Mapa completo do PINNeAPPle
Volumes
Visão geral
Como este documento foi feito

Volume 1 — Núcleo, porta de entrada e catálogo
Sumário
1. pinneapple
2. pinneapple_core
3. pinneapple_catalog
4. pinneapple_registry
5. pinneapple_orchestration
6. pinneapple_models
7. pinneapple_train
8. pinneapple_solvers

Volume 2 — Física: problemas, equações e ciência
Sumário
1. pinneapple_physics
2. pinneapple_presets
3. pinneapple_problemdesign
4. pinneapple_science
5. pinneapple_condense
6. pinneapple_query

Volume 3 — Simulação numérica
Sumário
1. pinneapple_simulation

Volume 4 — Aprendizado de máquina e dados
Sumário
1. pinneapple_neural
2. pinneapple_data
3. pinneapple_pdb
4. pinneapple_adaptation
5. pinneapple_arena
6. pinneapple_worldmodel
7. pinneapple_perception
8. pinneapple_quantum

Volume 5 — Confiança, validação e decisão
Sumário
1. pinneapple_analysis
2. pinneapple_check
3. pinneapple_veriphysics
4. pinneapple_security
5. pinneapple_decision
6. pinneapple_reason

Volume 6 — Projeto e sistemas
Sumário
1. pinneapple_design
2. pinneapple_systems
3. pinneapple_twin3d

Volume 7 — Ferramentas, orquestração e produtos
Sumário
1. pinneapple_tools
2. pinneapple_blender
3. pinneapple_flow
4. pinneapple_hub
5. pinneapple_lab
6. pinneapple_llm
7. pinneapple_mcp
8. pinneapple_autolab
9. pinneapple_app

Volume 8 — Apps, exemplos, reproduções, scripts e CI
1. Apps (apps/)

2. Exemplos (`examples/`)
3. Reproduções de papers
4. Scripts (`scripts/`)
5. Benchmarks (`benchmarks/`)
6. Integração contínua (`.github/workflows/`)
7. Documentação existente (`docs/`)

Volume 9 — Índice alfabético, glossário e apêndice numérico
1. Índice alfabético
2. Glossário
3. Apêndice: números do inventário

Mapa completo do PINNeAPPle

Documento de referência, em português, de todo o repositório: 41 pacotes, 2.142 módulos, 2.757 classes e 6.130 funções públicas. Gerado a partir do inventário automático do commit `e53cdcfb04`.

Como o texto passa de 150 páginas, ele está dividido em volumes por camada. Este arquivo é o volume 0: visão geral, arquitetura e sumário.

Volumes

volume	conteúdo	pacotes
Volume 1 — Núcleo, porta de entrada e catálogo	A porta de entrada `pp.*`, os primitivos do núcleo, o catálogo de métodos, o registro de artefatos e os shims de compatibilidade.	`pinneapple`, `pinneapple_core`, `pinneapple_catalog`, `pinneapple_registry`, `pinneapple_orchestration`, `pinneapple_models`, `pinneapple_train`, `pinneapple_solvers`
Volume 2 — Física: problemas, equações e ciência	Como um problema físico é descrito e resolvido (`PhysicalProblem`, `pp.solve`), as equações, os presets industriais, o desenho de problemas, os experimentos por área, a condensação e as consultas físicas.	`pinneapple_physics`, `pinneapple_presets`, `pinneapple_problemdesign`, `pinneapple_science`, `pinneapple_condense`, `pinneapple_query`
Volume 3 — Simulação numérica	Solvers clássicos (FDM, FEM, FVM, espectral, SPH, LBM), pontes para solvers externos, eletromagnetismo, voo, multicorpos, veículos e rovers.	`pinneapple_simulation`
Volume 4 — Aprendizado de máquina e dados	Arquiteturas e treino de redes (PINN, operadores neurais, grafos), dados e amostragem, adaptação, modelos de mundo, percepção, computação quântica e a arena de benchmarks.	`pinneapple_neural`, `pinneapple_data`, `pinneapple_pdb`, `pinneapple_adaptation`, `pinneapple_arena`, `pinneapple_worldmodel`, `pinneapple_perception`, `pinneapple_quantum`
Volume 5 — Confiança, validação e decisão	A camada que confere cada resultado contra a física: validação, incerteza, inversão, o linter `pinneapple check`, registros de evidência, segurança, decisão e a cadeia de porquês.	`pinneapple_analysis`, `pinneapple_check`, `pinneapple_veriphysics`, `pinneapple_security`, `pinneapple_decision`, `pinneapple_reason`
Volume 6 — Projeto e sistemas	Geometria, otimização de projeto, projeto a partir de intenção, sistemas dinâmicos, séries temporais, co-simulação e gêmeos digitais.	`pinneapple_design`, `pinneapple_systems`, `pinneapple_twin3d`
Volume 7 — Ferramentas, orquestração e produtos	Ferramentas (benchmarks, exportação, visualização, automação de GUIs de engenharia, NVIDIA), o Flow, o Hub, o Lab, os LLMs, o servidor MCP, o laboratório autônomo e o app web.	`pinneapple_tools`, `pinneapple_blender`, `pinneapple_flow`, `pinneapple_hub`, `pinneapple_lab`, `pinneapple_llm`, `pinneapple_mcp`, `pinneapple_autolab`, `pinneapple_app`
Volume 8 — Apps, exemplos, reproduções, scripts e CI	64 apps, 88 entradas de exemplos, reproduções de papers, scripts, benchmarks e 12 workflows de CI.	—
Volume 9 — Índice alfabético, glossário e apêndice numérico	Índice de todas as classes e funções públicas, glossário e números.	—

Visão geral

O que é o PINNeAPPle

O PINNeAPPle é uma biblioteca Python de IA para física. A ideia central tem duas partes:

1. **Uma descrição, muitos métodos.** O problema físico é descrito uma vez (`pp.PhysicalProblem`, ou o nome de um preset). Depois ele é resolvido por redes informadas pela física (PINN), operadores neurais, redes de grafos ou solvers clássicos (diferenças finitas, elementos finitos, pontes para OpenFOAM e CalculiX), sempre pela mesma chamada: `pp.solve(problema, método)`.
2. **Uma camada de confiança.** Antes de alguém usar um resultado, ele é conferido contra a física: resíduo da equação, conservação, condições de contorno, comparação com referências e domínio de validade. A saída é uma decisão explícita (`APPROVED` / `REVIEW` / `REJECT` no `pp.trust_card`; `pass` / `warn` / `fail` no `pp.validate`).

Em volta desse núcleo há camadas de projeto (geometria, otimização, projeto a partir de um texto), sistemas (séries temporais, co-simulação, gêmeos digitais), orquestração (Flow, Lab, laboratório autônomo, LLMs, servidor MCP) e produtos (Hub, apps web). O pacote de topo `pinneapple` (importado como `pp`) é uma camada fina: cada nome dele delega para um dos pacotes `pinneapple_*`.

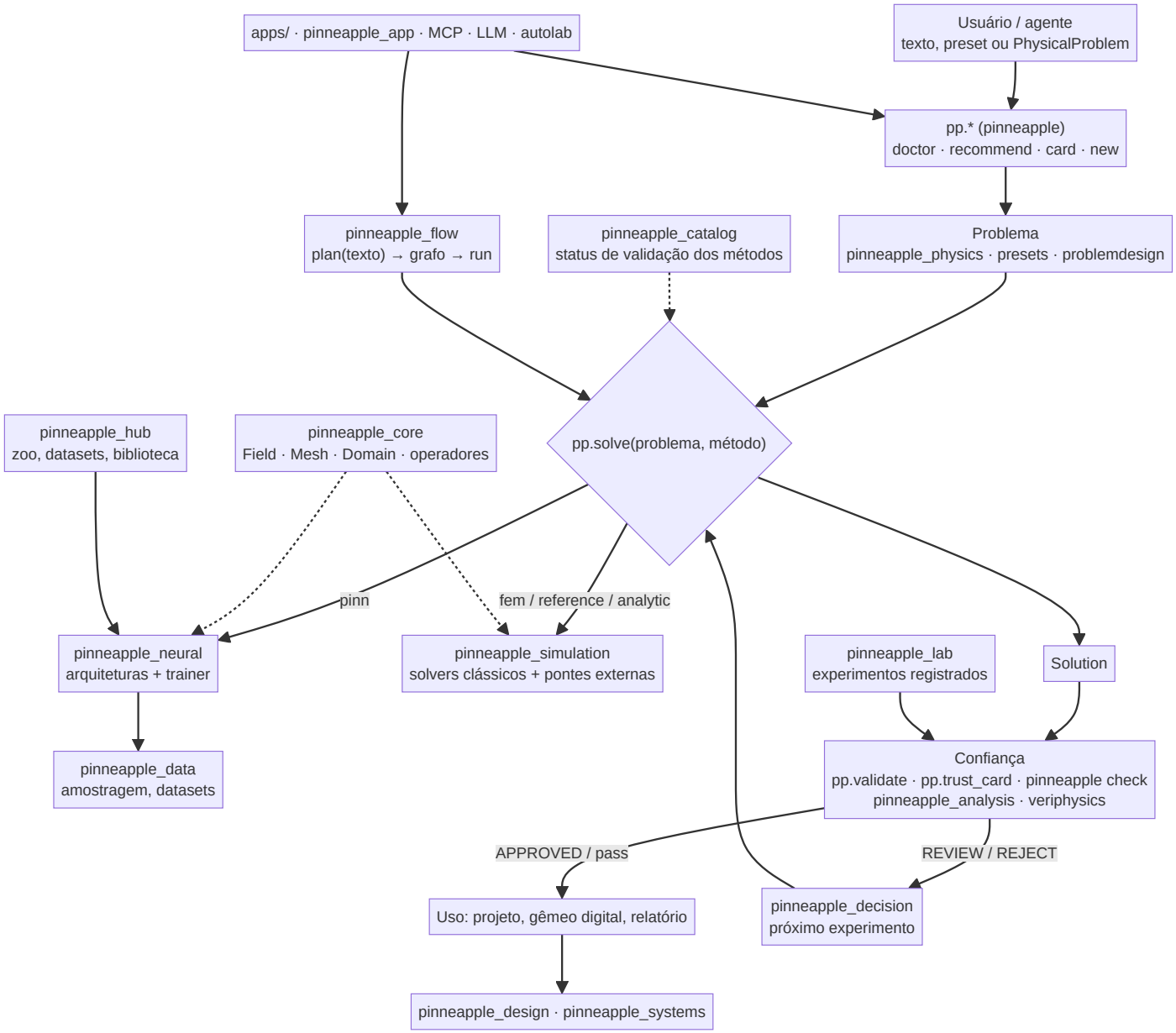
Arquitetura em camadas

A tabela resume as camadas na ordem em que um problema passa por elas. Cada camada depende só da interface da anterior (um `ProblemSpec`, coordenadas amostradas, a função de perda, um modelo treinado), e não de herança ou de um objeto compartilhado. Por isso dá para trocar uma peça sem mexer nas outras.

camada	pacotes principais	papel
Porta de entrada	`pinneapple` (`pp.*`), `pinneapple_core.onboarding`	`pp.doctor`, `pp.recommend`, `pp.card`, `pp.new`, `pp.solve`, `pp.compare`, `pp.validate`
Primitivos	`pinneapple_core`	`Field`, `Mesh`, `Domain`, `Geometry`, operadores diferenciais, motor de workflows
Problema	`pinneapple_physics` (`physical_problem`, `pde_environment`, `symbolic_pde`), `pinneapple_presets`, `pinneapple_problemdesign`	descrever o problema: equações, contornos, presets, desenho guiado
Solução clássica	`pinneapple_simulation`, `pinneapple_condense`	FDM, FEM, FVM, SPH, LBM, pontes externas, eletromagnetismo, voo, veículos, condensação
Aprendizado	`pinneapple_neural`, `pinneapple_data`, `pinneapple_adaptation`, `pinneapple_worldmodel`, `pinneapple_quantum`	arquiteturas, treino, dados, transferência, modelos de mundo
Confiança	`pinneapple_analysis`, `pinneapple_check`, `pinneapple_veriphysics`, `pinneapple_security`	validação, incerteza, inversão, linter de física em CI, registros de evidência, assinaturas
Decisão e raciocínio	`pinneapple_decision`, `pinneapple_reason`, `pinneapple_query`	qual experimento rodar a seguir, cadeia de porquês, consultas físicas com planejador
Projeto e sistemas	`pinneapple_design`, `pinneapple_systems`, `pinneapple_twin3d`	geometria, otimização, projeto por intenção, séries temporais, gêmeos digitais
Catálogo e registro	`pinneapple_catalog`, `pinneapple_registry`, `pinneapple_hub`	métodos com status de validação, recursos com licença, artefatos, zoo de modelos
Orquestração	`pinneapple_flow`, `pinneapple_lab`, `pinneapple_autolab`, `pinneapple_llm`, `pinneapple_mcp`, `pinneapple_orchestration`	grafos de nós, experimentos registrados, missões autônomas, agentes
Ferramentas e produtos	`pinneapple_tools`, `pinneapple_arena`, `pinneapple_blender`, `pinneapple_app`, `apps/`	benchmarks, exportação, visualização, automação de GUIs, apps web

Os pacotes `pinneapple_models`, `pinneapple_train` e `pinneapple_solvers` são shims de compatibilidade: só reexportam `pinneapple_neural.architectures`, `pinneapple_neural.trainer` e `pinneapple_simulation.numerical_solvers`, para que caminhos de import antigos continuem funcionando.

Como as peças se conectam



Em palavras:

- **`pp.*`** é a porta de entrada. `pp.doctor` diz o que falta instalar, `pp.recommend` sugere preset, método e modelo a partir de uma frase, `pp.card` mostra o que um preset recebe e devolve, e `pp.new` cria um projeto do usuário fora do repositório.
- **`pp.solve` / `pp.compare`** (em `pinneapple_physics.solving`) recebem um `PhysicalProblem`, um `ProblemSpec` ou o nome de um preset e despacham para o método pedido (`"pinn"`, `"fem"`, `"analytic"`, `"reference"`, `"exact"`, `"external"`; `pp.list_methods()` lista os registrados). O resultado é um `Solution`.
- **Confiança**: `pp.validate` (em `pinneapple_analysis.validation.validate`) e `pp.trust_card` (em `pinneapple_analysis.trust.field_card`) conferem resultados e modelos; `pinneapple check` roda as mesmas verificações em CI.
- **`pinneapple_flow`** transforma um texto num grafo de nós tipados (`plan`), e o executa sem interface ou no Flow Studio (`run`).
- **`pinneapple_hub`** guarda e entrega modelos treinados e verificados (`pp.zoo`), datasets (`pp.datasets`) e uma biblioteca pesquisável de ferramentas, geometrias, fluxos e exemplos.
- **`pinneapple_catalog`** lista cada solver, método de treino, equação e problema com o código, as referências e o status de validação calculado a partir dos testes (`validated` / `tested` / `untested`).
- **Apps** (`apps/` e `pinneapple_app`) são interfaces web e ferramentas de demonstração montadas sobre a biblioteca; o servidor MCP (`pinneapple_mcp`) expõe as mesmas funções para agentes de código.

A porta de entrada `pp.*`

`import pinneapple as pp` expõe 297 nomes (versão `0.7.0`). A tabela diz de onde cada um vem e o que faz.

nome	como é carregado	origem	o que faz
`pp.ActiveLearningConfig`	import direto	`pinneapple_data.ActiveLearningConfig`	Configuração do active learning: estratégia, número de candidatos, de pontos escolhidos por rodada, iniciais, iterações, peso de exploração e semente.
`pp.AdaptiveCollocationTrainer`	import direto	`pinneapple_data.AdaptiveCollocationTrainer`	Envolve um PINN e uma função física, mantém o próprio conjunto de colocação e acrescenta pontos escolhidos pela estratégia de active learning durante o treino.
`pp.AFNO`	import direto	`pinneapple_neural.architectures.AFNO`	Operador AFNO completo para campos 2D `(B, H, W, C)`: projeção de entrada, camadas AFNO e projeção de saída.
`pp.annulus`	import direto	`pinneapple_design.geometry.annulus`	Anel fechado (tubo com tampas planas) em torno do eixo z.
`pp.arena`	submódulo preguiçoso (_SUBMODULES)	`pinneapple_arena`	Pacote raiz: reexporta configuração, `Arena`, rankings, problemas, fábrica de modelos, treino, visualização, `define_problem`, `DatasetProblem`, presets de dataset, modo de decisão e NAS.
`pp.as_tensor`	preguiçoso (_LAZY_ATTRS)	`pinneapple_neural.utils.as_tensor`	Extrai o tensor de previsão de um Tensor, dicionário de campos, tupla ou objeto de saída (`PINNOutput`, `ModelOutput` etc.).
`pp.assess_drift`	preguiçoso (_LAZY_ATTRS)	`pinneapple_analysis.uncertainty.drift.assess_drift`	Checa um lote de predições quanto a perda de acurácia, má calibração, mudança de entrada e tendência do resíduo.
`pp.assimilation`	submódulo preguiçoso (_SUBMODULES)	`pinneapple_analysis.data_assimilation`	Assimilação de dados: 4D-Var por autograd sobre os métodos de ensemble (EKI) e filtros de Kalman, mais reconstrução GP e posicionamento de sensores.
`pp.astro`	submódulo preguiçoso (_SUBMODULES)	`pinneapple_physics.astro`	Pacote de astronomia posicional e astrofísica do observatório PINNeAPPle, em NumPy próprio e validado contra USNO NOVAS, JPL Horizons e skyfield; reexporta `Observer`, `Time`, `apparent` e afins.
`pp.auto_residual`	import direto	`pinneapple_physics.symbolic_pde.auto_residual`	Calcula valores do campo e derivadas nomeadas via autograd.
`pp.Backend`	import direto	`pinneapple_tools.compute_backends.Backend`	Namespace com as constantes de nome de backend (`TORCH`, `JAX`).
`pp.backend`	submódulo preguiçoso (_SUBMODULES)	`pinneapple_tools.compute_backends`	Suporte multi-backend: reexporta o seletor PyTorch/JAX e os utilitários JAX.
`pp.batched_inference`	import direto	`pinneapple_neural.trainer.batched_inference`	Inferência em lotes de um modelo sobre um tensor grande (`pinneapple_neural.predictor.infer`), com tamanho de lote, dispositivo e AMP opcionais.
`pp.benchmark`	submódulo preguiçoso (_SUBMODULES)	`pinneapple_analysis.benchmark`	Entrada do benchmark com orçamento igual (preparar -> validar -> rodar -> relatório); reexporta `run`, `prepare`, `load`, o registro e `ArrayProblem`.
`pp.BenchmarkConfig`	import direto	`pinneapple_tools.benchmark_suite.BenchmarkConfig`	Configuração do benchmark: número de pontos de colocação/contorno/inicial, épocas, taxa de aprendizado, pesos das perdas, dispositivo, semente e limiar de convergência.
`pp.BenchmarkResult`	import direto	`pinneapple_tools.benchmark_suite.BenchmarkResult`	Resultado de um par (problema, modelo): métricas, histórico, tempo, número de parâmetros e posição no ranking.

nome	como é carregado	origem	o que faz
pp.BenchmarkTaskBase	import direto	pinneapple_tools.benchmark_suite.BenchmarkTaskBase	Base abstrata de tarefa física: amostragem de colocação, contorno e condição inicial, resíduo da EDP, grade de avaliação e montagem de batch; os métodos abstratos levantam NotImplementedError.
pp.best_device	import direto	pinneapple_neural.trainer.best_device	Retorna o melhor dispositivo disponível: CUDA, depois MPS (se permitido), depois CPU.
pp.best_method	import direto	pinneapple_core.onboarding.catalog.best_method	O método mais barato que roda hoje para o preset: analítico, senão solver de referência, senão rede treinada.
pp.bodies	submódulo preguiçoso (_SUBMODULES)	pinneapple_design.geometry.bodies	Corpos fechados triangulados para escoamento externo: corpo de Ahmed, esfera, cilindro, caixa e anel.
pp.BoxConstraint	import direto	pinneapple_design.design_optimizer.BoxConstraint	Restrição de limites theta_min <= theta <= theta_max com barreira ReLU unilateral.
pp.build_digital_twin	import direto	pinneapple_systems.digital_twin.build_digital_twin	Fábrica de conveniência para DigitalTwin com as opções mais comuns.
pp.build_model	definido em pinneapple/__init__.py	pinneapple.build_model	Constrói um modelo pelo nome no ModelRegistry (ModelRegistry.build(name, **kwargs)).
pp.CADToCFDPipeline	import direto	pinneapple_simulation.numerical_solvers.CADToCFDPipeline	Pipeline completo: arquivo CAD → malha → solução NS → dados de treino PINN, com comparação contra um PINN.
pp.cae	submódulo preguiçoso (_SUBMODULES)	pinneapple_data.cae	Camada de dados CAE: lê malhas e resultados de OpenFOAM, CalculiX/Abaqus, Gmsh, VTK, STL e tabelas de pontos num único tipo Mesh, com qualidade de malha, Courant e distância à parede.
pp.card	import direto	pinneapple_core.onboarding.card	Monta o Card de um preset; nome desconhecido levanta KeyError com sugestões próximas.
pp.CauchyMisfit	import direto	pinneapple_analysis.inverse_problems.CauchyMisfit	Desajuste de Cauchy/Lorentz, de cauda pesada e muito robusto.
pp.CausalPINNTrainer	import direto	pinneapple_neural.trainer.CausalPINNTrainer	Treinador PINN que aplica os pesos causais ao resíduo da EDP junto com perdas de condição inicial/contorno; permite consultar e plotar os pesos.
pp.CausalWeightScheduler	import direto	pinneapple_neural.trainer.CausalWeightScheduler	Calcula pesos causais por fatia de tempo, w_k = exp(-epsilon * soma das perdas das fatias anteriores), e a perda ponderada e uma métrica de violação de causalidade.
pp.cfd	submódulo preguiçoso (_SUBMODULES)	pinneapple_simulation.numerical_solvers.external_flow	Escoamento externo em torno de qualquer corpo no OpenFOAM numa chamada: STL → snappyHexMesh → simpleFoam (k-omega SST) → forças, coeficientes de pele, linhas de corrente, cortes e cena 3D. Requer OpenFOAM instalado.
pp.CFDMesh	import direto	pinneapple_simulation.numerical_solvers.CFDMesh	Malha não estruturada para CFD, criada de STL, STEP ou CSG, com nós de contorno e plot 2D.
pp.channel_with_hole	import direto	pinneapple_design.geometry.channel_with_hole	Canal retangular com obstáculo circular.
pp.ChannelDomain2D	import direto	pinneapple_design.geometry.ChannelDomain2D	Canal retangular com entrada à esquerda e saída à direita.
pp.ChannelWithObstacleDomain2D	import direto	pinneapple_design.geometry.ChannelWithObstacleDomain2D	Canal retangular com obstáculo circular.
pp.circle	import direto	pinneapple_design.geometry.circle	Atalho: SDF de círculo.

nome	como é carregado	origem	o que faz
pp.closed_form	submódulo preguiçoso (_SUBMODULES)	pinneapple_physics.closed_form	Subpacote de modelos físicos em forma fechada (sem EDP), portados de ferramentas de produtos para serem compartilhados e testados; o __init__ importa só parte dos módulos (não importa eddy_coil, electromagnetics, heat_exchanger, pipe_insulation, pump_npsh, thin_airfoil).
pp.CollocationConfig	import direto	pinneapple_data.CollocationConfig	Configuração da geração de colocação: estratégia e número de pontos interiores, de contorno e de condição inicial, entre outros.
pp.CollocationSampler	import direto	pinneapple_data.CollocationSampler	Gerador único de pontos de colocação: construtores from_bounds, from_problem_spec, from_domain, from_mesh, from_stl, e sample/sample_adaptive que devolvem o dict de batch PINN.
pp.compare	import direto	pinneapple_physics.solving.compare	Resolve ou reaproveita cada método e pontua contra a referência nos mesmos pontos.
pp.Comparison	import direto	pinneapple_physics.solving.Comparison	Tabela de compare: métricas de cada método contra a referência nos mesmos pontos, com best.
pp.compile	preguiçoso (_LAZY_ATTRS)	pinneapple_physics.compile_api.compile	Prepara um PhysicalProblem, ProblemSpec ou nome de preset para resolver mais rápido.
pp.ComposedObsOperator	import direto	pinneapple_analysis.inverse_problems.ComposedObsOperator	Empilha vários operadores de observação num vetor único.
pp.CompositeObjective	import direto	pinneapple_design.design_optimizer.CompositeObjective	Soma ponderada de vários objetivos.
pp.CompositeRegularizer	import direto	pinneapple_analysis.inverse_problems.CompositeRegularizer	Soma ponderada de vários regularizadores.
pp.compute_isosurface	import direto	pinneapple_neural.predictor.compute_isosurface	Isosuperfície de um campo 3D por marching cubes (exige scikit-image).
pp.compute_pareto_front	import direto	pinneapple_design.design_optimizer.compute_pareto_front	Máscara booleana das soluções Pareto-ótimas.
pp.compute_streamlines	import direto	pinneapple_neural.predictor.compute_streamlines	Linhas de corrente 2D por integração RK4.
pp.ComputeBudget	preguiçoso (_LAZY_ATTRS)	pinneapple_analysis.cost.equal_budget.ComputeBudget	Orçamento de treino em segundos, FLOPs e/ou passos (None = sem limite).
pp.conformal_study	preguiçoso (_LAZY_ATTRS)	pinneapple_analysis.uncertainty.conformal_study.conformal_study	Calibra em (y_cal, pred_cal) e mede a cobertura em (y_test, pred_test).
pp.ConformalPredictor	import direto	pinneapple_analysis.uncertainty.ConformalPredictor	Preditor conformal split: calibra num conjunto reservado e dá intervalos com cobertura garantida.
pp.ConstraintSet	import direto	pinneapple_design.design_optimizer.ConstraintSet	Agrega várias restrições: soma de penalidades e checagem de todas.
pp.ContinuousAdjointSolver	import direto	pinneapple_design.design_optimizer.ContinuousAdjointSolver	Solver adjunto por diferenciação automática: resolve a equação adjunta, calcula a sensibilidade de forma e roda a otimização.
pp.core	submódulo preguiçoso (_SUBMODULES)	pinneapple_core	Ponto de entrada do pacote: exporta Domain, Geometry, Mesh, Field, FunctionField, Grid, PointCloud e os operadores, e carrega func, fem, loss, optim, data, transforms etc. preguiçosamente no primeiro acesso.
pp.count_gpus	import direto	pinneapple_neural.trainer.count_gpus	Retorna o número de GPUs CUDA disponíveis.
pp.csg_annulus	import direto	pinneapple_design.geometry.csg_annulus	Alias de annulus em pinneapple_design.geometry: domínio CSG em forma de coroa circular.

nome	como é carregado	origem	o que faz
pp.CSGCircle	import direto	pinneapple_design.geometry.CSGCircle	Círculo 2D.
pp.CSGDifference	import direto	pinneapple_design.geometry.CSGDifference	Diferença A menos B: max(sdf_a, -sdf_b).
pp.CSGEllipse	import direto	pinneapple_design.geometry.CSGEllipse	Elipse 2D alinhada aos eixos.
pp.CSGIntersection	import direto	pinneapple_design.geometry.CSGIntersection	Interseção booleana de duas formas: max das SDFs.
pp.CSGPolygon	import direto	pinneapple_design.geometry.CSGPolygon	Polígono qualquer dado por vértices ordenados.
pp.CSGRectangle	import direto	pinneapple_design.geometry.CSGRectangle	Retângulo alinhado aos eixos.
pp.CSGUnion	import direto	pinneapple_design.geometry.CSGUnion	União booleana de duas formas: min das SDFs.
pp.curl	preguiçoso (_LAZY_ATTRS)	pinneapple_core.operators.curl	Rotacional: vetor em 3D, escalar (componente z) em 2D.
pp.dashboard	submódulo preguiçoso (_SUBMODULES)	pinneapple_tools.visualization.dashboard	Painéis centrados numa animação para qualquer simulação: animação no meio, gráficos sincronizados ao cursor de tempo, KPIs, parâmetros e barra de cor; exporta HTML interativo, PNG e GIF/MP4.
pp.data	submódulo preguiçoso (_SUBMODULES)	pinneapple_data	Raiz do pacote: importa, cada um dentro de `try/except` (falha silenciosa), STL, `PhysicalSample`, armazenamento Zarr, collate, colocação, active learning, normalizadores, tipos UPD, splits e construtores de batch; os `datasets` são importados sem proteção.
pp.datasets	submódulo preguiçoso (_SUBMODULES)	pinneapple_hub.zoo.datasets	`pp.datasets`: datasets verificados em uma linha, guardados como npz sem pickle; os pequenos vêm no pacote, os outros são baixados do Hub, conferidos por SHA-256 e cacheados.
pp.DDPPINNTrainer	import direto	pinneapple_neural.trainer.DDPPINNTrainer	Treinador PINN multi-GPU com DDP: inicializa o grupo de processos, sincroniza gradientes e pode lançar workers via `mp.spawn`.
pp.DDPTrainerConfig	import direto	pinneapple_neural.trainer.DDPTrainerConfig	Configuração do treino DDP: backend, número de processos, endereço/porta do mestre, clip de gradiente, AMP e frequência de log.
pp.decide	preguiçoso (_LAZY_ATTRS)	pinneapple_decision.api.decide	Decide qual experimento rodar a seguir para o problema e objetivo dados.
pp.decision	submódulo preguiçoso (_SUBMODULES)	pinneapple_decision	Fachada do pacote: reexporta tipos, decisor, backends, laço de decisão, seletores, árvore, verificadores e a API `decide`/`execute`/`verify`/`run_tree`; importar o pacote não carrega torch.
pp.decision_engine	preguiçoso (_LAZY_ATTRS)	pinneapple_decision.api.decision_engine	Devolve o motor padrão; qualquer argumento (ou `reset=True`) cria um novo, sem carregar a evidência antiga.
pp.DEFAULT_MODELS	import direto	pinneapple_tools.benchmark_suite.DEFAULT_MODELS	Lista padrão de `ModelSpec` usada pela suíte de benchmarks (`pinneapple_tools.benchmark_suite.benchmark`).
pp.design	preguiçoso (_LAZY_ATTRS)	pinneapple_design.intent.design	Projeta a partir de uma meta em linguagem natural; devolve `DesignPackage`, `NeedsInput` ou `OutOfScope`.
pp.design_opt	submódulo preguiçoso (_SUBMODULES)	pinneapple_design.design_optimizer	Ponto de entrada do otimizador de projeto: reexporta o laço fechado (espaço de parâmetros, surrogate, otimizador, refinamento por PINN, Pareto) e a camada de estudos de projeto.

nome	como é carregado	origem	o que faz
pp.DesignOptConfig	import direto	pinneapple_design.design_optimizer.DesignOptConfig	Configuração do DesignOptLoop: iterações, configuração do otimizador interno, refinamento top-k e outros.
pp.DesignOptimizerConfig	import direto	pinneapple_design.design_optimizer.DesignOptimizerConfig	Configuração única dos back-ends (método gradient/bayesian/evolutionary, iterações, taxa de aprendizado, etc.).
pp.DesignOptLoop	import direto	pinneapple_design.design_optimizer.DesignOptLoop	Orquestra o laço fechado: parametriza, avalia com o surrogate, otimiza, refina por PINN e monta a frente de Pareto.
pp.DesignOptResult	import direto	pinneapple_design.design_optimizer.DesignOptResult	Resultado completo de um DesignOptLoop: melhor design, histórico, frente de Pareto, com gráfico de convergência e resumo.
pp.DigitalTwin	import direto	pinneapple_systems.digital_twin.DigitalTwin	Gêmeo digital que funde um modelo substituto treinado com dados de sensores vivos; gerencia fluxos, laço de atualização, callbacks, predição e histórico.
pp.DigitalTwinConfig	import direto	pinneapple_systems.digital_twin.DigitalTwinConfig	Configuração do DigitalTwin: dispositivo, lote, AMP, intervalo de atualização, fila, histórico, assimilação e parâmetros de anomalia.
pp.DirichletBC	import direto	pinneapple_physics.symbolic_pde.DirichletBC	Condição de Dirichlet suave por penalidade MSE.
pp.div	preguiçoso (_LAZY_ATTRS)	pinneapple_core.operators.div	Divergência de um campo vetorial.
pp.doctor	import direto	pinneapple_core.onboarding.doctor	pp.doctor: verifica a instalação contra um perfil (start, cfd, research, enterprise, all) e imprime um comando de correção por falha.
pp.dominant_frequency	preguiçoso (_LAZY_ATTRS)	pinneapple_physics.physics_audit.regime.dominant_frequency	Frequência do pico espectral mais forte de um sinal amostrado (por exemplo desprendimento de vórtices).
pp.DoMINO	import direto	pinneapple_physics.pinn_solver.DoMINO	Treinador de PINN por decomposição de domínio: particiona, combina as saídas e penaliza descontinuidade na interface.
pp.DragAdjointObjective	import direto	pinneapple_design.design_optimizer.DragAdjointObjective	Objetivo de força de arrasto (pressão + viscoso) aproximado como média ponderada sobre pontos de superfície.
pp.DragObjective	import direto	pinneapple_design.design_optimizer.DragObjective	Proxy de arrasto: média de |u|^2 + peso x p^2 sobre o campo (sem normais de superfície).
pp.dt	submódulo preguiçoso (_SUBMODULES)	pinneapple_systems.digital_twin	Runtime de gêmeo digital sobre modelos substitutos: reexporta DigitalTwin, estados, fluxos de sensores, filtros de Kalman, detectores de anomalia, pipeline de tempo real, reconstrução de sinais, condicionamento de telemetria e prognóstico (RUL).
pp.dynamics	submódulo preguiçoso (_SUBMODULES)	pinneapple_simulation.particle_dynamics	Simulação dinâmica diferenciável em PyTorch puro: corpos rígidos, MPM (MLS-MPM) e SPH; reexporta também os parâmetros de solo/roda de Bekker-Wong.
pp.EKIConfig	import direto	pinneapple_analysis.inverse_problems.EKIConfig	Configuração da EKI: tamanho do ensemble, iterações, ruído, espalhamento inicial, regularização, prior, tolerância de colapso e semente.
pp.ellipse	import direto	pinneapple_design.geometry.ellipse	Atalho: SDF de elipse.
pp.em	submódulo preguiçoso (_SUBMODULES)	pinneapple_simulation.electromagnetics	Pacote de eletromagnetismo em SI: referências em forma fechada, materiais, solvers FEM 2-D em A_z, FDTD, ondas e máquinas elétricas; reexporta as classes e funções principais dos submódulos.

nome	como é carregado	origem	o que faz
pp.ensemble	submódulo preguiçoso (_SUBMODULES)	pinneapple_physics.ensemble	Ensemble adaptativo de especialistas físicos: pesos atualizados online por erro contra referência, resíduo físico e custo, nos modos select ou combine, com intervalos conformais e detecção de suspeitos.
pp.EnsembleKalmanInversion	import direto	pinneapple_analysis.inverse_problems.EnsembleKalmanInversion	EKI padrão (Iglesias et al. 2013): inicializa o ensemble, aplica passos tipo Kalman e para por colapso do ensemble.
pp.EnsembleUQ	import direto	pinneapple_analysis.uncertainty.EnsembleUQ	Envelope de Deep Ensemble que dá média e incerteza a partir dos membros.
pp.env	submódulo preguiçoso (_SUBMODULES)	pinneapple_physics.pde_environment	Ponto de entrada do ambiente de problemas: reexporta ProblemSpec, PDETermSpec, as condições de contorno, ProblemBuilder, o registro de presets, presets acadêmicos/CFD/industriais, os resíduos de turbulência e a base de famílias de EDP. O bloco de presets estruturais é importado dentro de try/except ImportError. [sem teste direto]
pp.excitation	submódulo preguiçoso (_SUBMODULES)	pinneapple_systems.excitation	Sinais de excitação de atuador em NumPy (degrau, rampa, senoide, escada aleatória, PRBS, chirp, dente de serra, passeio aleatório suavizado, pulsos gaussianos, AM de dois tons) para identificação de sistemas e treino de surrogates; separa famílias de treino e famílias reservadas (TRAIN_FAMILIES, HELDOUT_FAMILIES).
pp.execute	preguiçoso (_LAZY_ATTRS)	pinneapple_decision.api.execute	Executa uma decisão emitida pelo mesmo motor, rechecando as restrições antes.
pp.Experiment	import direto	pinneapple_physics.experiment.Experiment	Método aplicado a um problema com semente, opções e referência opcional; run executa de forma reprodutível.
pp.ExperimentResult	import direto	pinneapple_physics.experiment.ExperimentResult	Registro de uma execução: config, fingerprint, solução, métricas, tempo, ambiente; salva e carrega em JSON.
pp.export	submódulo preguiçoso (_SUBMODULES)	pinneapple_tools.model_export	Exportação de modelos nn.Module para TorchScript e ONNX e de arrays de coordenadas/campos para CSV e NPZ. [sem teste]
pp.export_csv	import direto	pinneapple_tools.model_export.export_csv	Grava coordenadas e campos num arquivo CSV com cabeçalho.
pp.export_npz	import direto	pinneapple_tools.model_export.export_npz	Grava coordenadas e campos num .npz comprimido do NumPy.
pp.export_onnx	import direto	pinneapple_tools.model_export.export_onnx	Exporta qualquer nn.Module para ONNX com nomes de entrada/saída, opset e eixos dinâmicos.
pp.export_torchscript	import direto	pinneapple_tools.model_export.export_torchscript	Exporta qualquer nn.Module para TorchScript (trace com example_input, ou script se não houver exemplo ou o trace falhar).
pp.fea	submódulo preguiçoso (_SUBMODULES)	pinneapple_simulation.external_solvers.calculix.study	Estudos CalculiX em poucas linhas: malha (hexaedros, tetraedros gmsh ou deck existente), conjuntos por geometria, um passo de análise (estático, frequência, flambagem, calor), execução e resultados em arrays.
pp.FlowVisualizer	import direto	pinneapple_neural.predictor.FlowVisualizer	Classe de alto nível com linhas de corrente, contornos de pressão, módulo de velocidade e vorticidade a partir de um modelo.
pp.flux	preguiçoso (_LAZY_ATTRS)	pinneapple_core.operators.flux	Fluxo de um campo vetorial pela fronteira, integral f . n dS.
pp.FourierFeatureEmbedding	import direto	pinneapple_neural.architectures.FourierFeatureEmbedding	Embedding [cos, sin](2π B x) com matriz B aleatória ou treinável.
pp.func	submódulo preguiçoso (_SUBMODULES)	pinneapple_core.func	Transformações de função (grad, jacrev, jacfwd, jacobian, hessian, vmap) sobre torch.func com wrt por nome ou posição, e implicit_solve, que diferencia através de um solver por uma solução adjunta.

nome	como é carregado	origem	o que faz
pp.GaussianMisfit	import direto	pinneapple_analysis.inverse_problems.GaussianMisfit	Desajuste Gaussiano (L2/Mahalanobis).
pp.geom	submódulo preguiçoso (_SUBMODULES)	pinneapple_design.geometry	Subpacote de geometria (antigo pinneapple_geom): reexporta GeometrySpec, GeometryAsset, MeshData, SDFs, CSG, domínios físicos 2D/3D, malhadores, RBF e MeshCollocator; o construtor STL é importado de forma tolerante (vira None se faltar dependência).
pp.geometries	submódulo preguiçoso (_SUBMODULES)	pinneapple_design.geometry_library	Fachada da biblioteca de geometrias (exposta como pp.geometries): listar, construir, exportar, renderizar miniaturas, carregar arquivos pré-gerados e exportar a galeria web.
pp.geophysics	submódulo preguiçoso (_SUBMODULES)	pinneapple_simulation.geophysics	Blocos do sistema terrestre verificados contra soluções exatas ou testes padrão: águas rasas na esfera, equação de Richards e modelo de balanço de energia de duas camadas.
pp.get_backend	import direto	pinneapple_tools.compute_backends.get_backend	Devolve o nome do backend ativo.
pp.get_pde_family	import direto	pinneapple_physics.pde_environment.get_pde_family	Busca uma PDEFamily por tipo ou apelido (sem diferenciar caixa).
pp.get_physics_backend	preguiçoso (_LAZY_ATTRS)	pinneapple_core.backend.get_physics_backend	Devolve o objeto backend pelo nome ou o backend ativo.
pp.get_preset	import direto	pinneapple_physics.pde_environment.get_preset	Cria um preset pelo nome, repassando argumentos.
pp.get_rank	import direto	pinneapple_neural.trainer.get_rank	Retorna o rank do processo atual (0 fora de modo distribuído).
pp.get_rans_preset	import direto	pinneapple_physics.pde_environment.get_rans_preset	Cria um objeto de resíduo de turbulência pelo nome; ValueError para nome desconhecido.
pp.get_world_size	import direto	pinneapple_neural.trainer.get_world_size	Retorna o número total de processos (1 fora de modo distribuído).
pp.GlobalSensitivity	import direto	pinneapple_analysis.inverse_problems.GlobalSensitivity	Sensibilidade global por variância (índices de Sobol pelo método de Saltelli).
pp.gpu_info	import direto	pinneapple_neural.trainer.gpu_info	Retorna um dict de informações por GPU disponível.
pp.grad	preguiçoso (_LAZY_ATTRS)	pinneapple_core.operators.grad	Gradiente: (N, d) para escalar, (N, c, d) para c componentes.
pp.GradNormBalancer	import direto	pinneapple_neural.trainer.GradNormBalancer	GradNorm: ajusta pesos para manter as normas de gradiente de cada termo numa camada compartilhada proporcionais à média.
pp.HardBC	import direto	pinneapple_physics.symbolic_pde.HardBC	Condição de contorno rígida por função distância: u = g + phi * rede, satisfeita exatamente; embrulha o modelo.
pp.HashGridEncoding	import direto	pinneapple_neural.architectures.HashGridEncoding	Codificação hash multirresolução com interpolação entre os cantos do voxel em cada nível.
pp.HashGridMLP	import direto	pinneapple_neural.architectures.HashGridMLP	MLP pequeno sobre a codificação hash, para representar campos 3D.
pp.hessian	preguiçoso (_LAZY_ATTRS)	pinneapple_core.operators.hessian	Hessiana (N, d, d) de um campo escalar.

nome	como é carregado	origem	o que faz
pp.HeteroscedasticMisfit	import direto	pinneapple_analysis.inverse_problems.HeteroscedasticMisfit	Desajuste Gaussiano com variância de ruído por observação.
pp.HuberMisfit	import direto	pinneapple_analysis.inverse_problems.HuberMisfit	Desajuste de Huber (ou pseudo-Huber), robusto a outliers moderados.
pp.Hybrid	preguiçoso (_LAZY_ATTRS)	pinneapple_core.module.Hybrid	Solver mais correção neural (por exemplo modo `residual`), treinável através do solver.
pp.IdentifiabilityAnalyzer	import direto	pinneapple_analysis.inverse_problems.IdentifiabilityAnalyzer	Analisa a identificabilidade a partir dos autovalores da matriz de Fisher.
pp.identify_pde	import direto	pinneapple_physics.pde_environment.identify_pde	Pontua as famílias conhecidas contra uma descrição em texto livre por sobreposição de palavras-chave.
pp.infer_on_grid_1d	import direto	pinneapple_neural.predictor.infer_on_grid_1d	Avalia o modelo numa grade retangular x-t.
pp.infer_on_grid_2d	import direto	pinneapple_neural.predictor.infer_on_grid_2d	Avalia o modelo numa grade retangular x-y.
pp.inference	submódulo preguiçoso (_SUBMODULES)	pinneapple_neural.predictor	Inferência pós-treino (antigo `pinneapple_inference`): reexporta inferência em lote e em grade, gráficos de campos, erro, perda, comparação, fluxo 3D e otimização de projeto. [sem teste direto]
pp.info	definido em pinneapple/__init__.py	pinneapple.info	Imprime versão, dispositivos, número de presets e modelos, pacotes importáveis e dependências opcionais com o extra de pip.
pp.IntegralObsOperator	import direto	pinneapple_analysis.inverse_problems.IntegralObsOperator	Funcional integral aproximado por quadratura ponderada com núcleo opcional.
pp.integrate	preguiçoso (_LAZY_ATTRS)	pinneapple_core.operators.integrate	Integral sobre o suporte (trapézio na grade, P1 exato na malha, quadratura ou Monte Carlo na nuvem).
pp.inverse	submódulo preguiçoso (_SUBMODULES)	pinneapple_analysis.inverse_problems	Reexporta o kit de problemas inversos (modelos de ruído, regularizadores, operadores de observação, EKI, sensibilidade, termos faltantes, `InverseProblemSolver`, e o núcleo de identificabilidade/calibração/studio). O docstring ainda usa o nome antigo `pinneapple_inverse`.
pp.InverseProblemSolver	import direto	pinneapple_analysis.inverse_problems.InverseProblemSolver	Resolvedor unificado de problemas inversos com informação física.
pp.InverseSolverConfig	import direto	pinneapple_analysis.inverse_problems.InverseSolverConfig	Configuração do `InverseProblemSolver`: método, iterações, taxa de aprendizado e opções de EKI.
pp.is_distributed	import direto	pinneapple_neural.trainer.is_distributed	Diz se há um grupo de processos distribuído ativo.
pp.IteratedEKI	import direto	pinneapple_analysis.inverse_problems.IteratedEKI	EKI regularizada por Tikhonov (TEKI, Chada et al. 2020).
pp.jacobian	preguiçoso (_LAZY_ATTRS)	pinneapple_core.operators.jacobian	Jacobiana `(N, c, d)` de um campo.
pp.JAXBackend	import direto	pinneapple_tools.compute_backends.JAXBackend	Conjunto de utilitários JAX para PINNs sobre modelos Equinox ou Flax: `jit_pinn`, `vmap_residual`, `grad_fn` e conversões de tensores.

nome	como é carregado	origem	o que faz
`pp.jit_pinn`	import direto	`pinneapple_tools.compute_backends.jit_pinn`	Alias de módulo para `JAXBackend.jit_pinn` (compila com JIT o resíduo do PINN).
`pp.KOmegaSSTResiduals`	import direto	`pinneapple_physics.pde_environment.KOmegaSSTResiduals`	Resíduos do modelo k-omega SST (RANS) para PINN, em 2D ou 3D conforme a entrada, com viscosidade turbulenta e resíduos de canal.
`pp.laplacian`	preguiçoso (`_LAZY_ATTRS`)	`pinneapple_core.operators.laplacian`	Soma das segundas derivadas, por componente em campos vetoriais.
`pp.LCurveSelector`	import direto	`pinneapple_analysis.inverse_problems.LCurveSelector`	Escolhe lambda pela máxima curvatura da curva L, chamando uma função de treino para cada valor.
`pp.LidDrivenCavityDomain2D`	import direto	`pinneapple_design.geometry.LidDrivenCavityDomain2D`	Cavidade quadrada com tampa móvel.
`pp.LinearObsOperator`	import direto	`pinneapple_analysis.inverse_problems.LinearObsOperator`	Operador linear `H(u) = A u` (ex.: tomografia).
`pp.list_methods`	import direto	`pinneapple_physics.solving.list_methods`	Nomes dos métodos registrados, opcionalmente filtrados por tipo.
`pp.list_models`	definido em `pinneapple/__init__.py`	`pinneapple.list_models`	Lista em ordem os nomes de modelos registrados no `ModelRegistry`.
`pp.list_pde_families`	import direto	`pinneapple_physics.pde_environment.list_pde_families`	Lista ordenada dos tipos canônicos de EDP.
`pp.list_presets`	import direto	`pinneapple_physics.pde_environment.list_presets`	Lista ordenada de todos os presets registrados.
`pp.LocalSensitivity`	import direto	`pinneapple_analysis.inverse_problems.LocalSensitivity`	Sensibilidade linearizada em torno de um ponto nominal, com Jacobiano por diferenças finitas opcional.
`pp.loss`	submódulo preguiçoso (`_SUBMODULES`)	`pinneapple_core.loss`	Termos de perda como funções simples (`pde`, `boundary`, `supervised`, `conservation`, `energy`, `symmetry`, `inverse`), soma ponderada (`combine`) e o `Balancer` de pesos dinâmicos escolhido por nome.
`pp.LossRatioBalancer`	import direto	`pinneapple_neural.trainer.LossRatioBalancer`	Mantém as razões entre perdas (via EMA) próximas de valores-alvo em relação à perda da EDP.
`pp.lshape`	import direto	`pinneapple_design.geometry.lshape`	Domínio em L.
`pp.MassConservationConstraint`	import direto	`pinneapple_design.design_optimizer.MassConservationConstraint`	Penaliza o quadrado médio da divergência estimada do campo de velocidade.
`pp.maxwell`	submódulo preguiçoso (`_SUBMODULES`)	`pinneapple_physics.maxwell_checks`	Verificações das equações de Maxwell sobre campos numa grade (de PINN, FNO, solver ou medição), sem referência: resíduos relativos com teste em `2h`, balanço de Poynting, contorno PEC, reciprocidade e passividade; alimenta `pinneapple check` (`physics = "maxwell"`).
`pp.maybe_compile`	import direto	`pinneapple_neural.trainer.maybe_compile`	Aplica `torch.compile` se o PyTorch suportar; caso contrário devolve o modelo original.
`pp.MCDropoutWrapper`	import direto	`pinneapple_analysis.uncertainty.MCDropoutWrapper`	Envolve um `nn.Module` e liga dropout na inferência por hooks; avisa quando não há ativação oculta para descartar.
`pp.MeshGraphNet`	import direto	`pinneapple_neural.architectures.MeshGraphNet`	GNN encoder-processador-decoder para simulação em malha não estruturada com conectividade fixa.

nome	como é carregado	origem	o que faz
pp.meta	submódulo preguiçoso (_SUBMODULES)	pinneapple_adaptation.meta_learning	Reexporta configurações, amostrador e treinadores de meta-learning e oferece meta_train (constrói e treina) e meta_adapt (adapta a uma tarefa nova).
pp.MetaBenchmarkConfig	import direto	pinneapple_tools.benchmark_suite.MetaBenchmarkConfig	Configuração do benchmark de meta-aprendizado: épocas meta, passos internos, taxas de aprendizado, tarefas por lote, valores de K, algoritmos, pontos e pesos de perda.
pp.MetaBenchmarkFamily	import direto	pinneapple_tools.benchmark_suite.MetaBenchmarkFamily	Uma família paramétrica de EDPs: fábricas de tarefas de treino e avaliação, faixas de parâmetros e parâmetros de avaliação.
pp.MetaBenchmarkPipeline	import direto	pinneapple_tools.benchmark_suite.MetaBenchmarkPipeline	Pipeline que roda o meta-treino e a avaliação K-shot, monta leaderboard, curvas de erro por K e salva/carrega resultados.
pp.MetaBenchmarkResult	import direto	pinneapple_tools.benchmark_suite.MetaBenchmarkResult	Resultado de uma família, parâmetro de avaliação, algoritmo e valor de K: métricas e tempo.
pp.MethodNotAvailable	import direto	pinneapple_physics.solving.MethodNotAvailable	Exceção: o método pedido não pode resolver este problema.
pp.metrics	submódulo preguiçoso (_SUBMODULES)	pinneapple_physics.metrics	pp.metrics: L2, L2 relativo, RMSE, MAE, erro máximo, L∞ relativo e R^2 por campo, mais resumo, métrica agregada para rankings e erro de rollout temporal.
pp.ModelRegistry	import direto	pinneapple_neural.architectures.ModelRegistry	Registro central por nome: register (decorador), list, families, spec, build e build_with_report.
pp.models	submódulo preguiçoso (_SUBMODULES)	pinneapple_models	[shim] Reexporta registro, catálogo e arquiteturas de pinneapple_neural.architectures para código legado; marcado como obsoleto (use pinneapple_neural.architectures). [sem teste direto]
pp.ModelSpec	import direto	pinneapple_tools.benchmark_suite.ModelSpec	Descrição de um modelo candidato: nome, fábrica que o constrói, descrição, tipo de entrada (input_kind) e família.
pp.ModifiedMLP	import direto	pinneapple_neural.architectures.ModifiedMLP	MLP com Fourier features e portas U/V tipo highway em cada camada.
pp.molecular	submódulo preguiçoso (_SUBMODULES)	pinneapple_tools.visualization.molecular	Visualização molecular: proteínas e moléculas, sozinhas ou adsorvidas em superfícies metálicas, como imagens, trajetórias MP4, cenas glTF e painéis de dinâmica molecular.
pp.MPMSimulator	import direto	pinneapple_simulation.particle_dynamics.MPMSimulator	Simulador MPM diferenciável (nn.Module) com materiais elástico, fluido ou neve, em 2D/3D.
pp.MPMState	import direto	pinneapple_simulation.particle_dynamics.MPMState	Contêiner do estado das partículas MPM: posição, velocidade, gradiente de deformação F, matriz afim C e Jp.
pp.multibody	submódulo preguiçoso (_SUBMODULES)	pinneapple_simulation.multibody	Dinâmica de corpos rígidos em coordenadas reduzidas: uma base flutuante com árvores de juntas de revolução; reexporta a árvore e os utilitários de braço.
pp.naca_parametric	import direto	pinneapple_design.design_optimizer.naca_parametric	Gera a superfície de um aerofólio NACA de 4 dígitos como coordenadas de pontos de controle (pinneapple_design.geometry.gen.airfoil).
pp.NeumannBC	import direto	pinneapple_physics.symbolic_pde.NeumannBC	Condição de Neumann suave pela derivada normal via autograd.

nome	como é carregado	origem	o que faz
`pp.new`	definido em `pinneapple/__init__.py`	`pinneapple.new`	Cria um projeto do usuário fora do repositório (`pinneapple_core.onboarding.new_project`); recusa sobrescrever.
`pp.new_project`	import direto	`pinneapple_core.onboarding.new_project`	Escreve um projeto independente a partir de um template e preset e devolve a pasta.
`pp.non_hermitian`	submódulo preguiçoso (`_SUBMODULES`)	`pinneapple_physics.non_hermitian`	Física não hermitiana: dímeros e cadeias PT-simétricos, pontos excepcionais (EPs) e se eles podem ser sintonizados; numpy, sympy para a parte simbólica e matplotlib para as figuras.
`pp.NSFlowSolver`	import direto	`pinneapple_simulation.numerical_solvers.NSFlowSolver`	Solver de Navier-Stokes incompressível na malha, com condições de contorno e exportação de dados PINN.
`pp.NTKWeightBalancer`	import direto	`pinneapple_neural.trainer.NTKWeightBalancer`	Pesos inversamente proporcionais ao traço do NTK de cada termo, estimado por Hutchinson.
`pp.Parameter`	import direto	`pinneapple_physics.physical_problem.Parameter`	Parâmetro físico nomeado com valor, unidade, papel (fixo, projeto, incerto, desconhecido), limites e distribuição.
`pp.ParetoFront`	import direto	`pinneapple_design.design_optimizer.ParetoFront`	Contêiner de uma frente de Pareto: objetivos, parâmetros e máscara, com filtro, hipervolume e gráfico.
`pp.ParticleSystem`	import direto	`pinneapple_simulation.particle_dynamics.ParticleSystem`	Base de sistemas de partículas com busca de vizinhos por força bruta O(N²); `forward` levanta NotImplementedError.
`pp.pde_from_sympy`	import direto	`pinneapple_physics.symbolic_pde.pde_from_sympy`	Cria um `SymbolicPDE` a partir de uma expressão de resíduo SymPy.
`pp.PeriodicBC`	import direto	`pinneapple_physics.symbolic_pde.PeriodicBC`	Periodicidade exata numa coordenada por mudança de variável (seno/cosseno).
`pp.PhysicalProblem`	import direto	`pinneapple_physics.physical_problem.PhysicalProblem`	Problema físico completo (geometria, EDP, condições, parâmetros, solver, intenção) com validação, serialização, fingerprint e adaptadores.
`pp.physics_audit`	submódulo preguiçoso (`_SUBMODULES`)	`pinneapple_physics.physics_audit`	Auditoria física: qual modelo usar, quando confiar nele e quando sua formulação física deve ser revista; `diagnose` junta avaliação e diagnóstico num caso. Numpy puro, sem torch.
`pp.PhysicsDomain2D`	import direto	`pinneapple_design.geometry.PhysicsDomain2D`	Base dos domínios 2D: SDF, amostragem interna e por região, lote PINN e manifesto de condições.
`pp.PhysicsModule`	preguiçoso (`_LAZY_ATTRS`)	`pinneapple_core.module.PhysicsModule`	Classe base de modelos e solvers com `kind`, `describe()`, `num_parameters()` e um `fit` simples.
`pp.PhysicsOptimizer`	preguiçoso (`_LAZY_ATTRS`)	`pinneapple_core.optim.PhysicsOptimizer`	Parâmetros com limites, objetivo e restrições, minimizados pelo método escolhido em `minimize`.
`pp.PhysicsScenario`	import direto	`pinneapple_worldmodel.PhysicsScenario`	Especificação completa de um problema físico para o world model.
`pp.PhysicsSurrogate`	import direto	`pinneapple_design.design_optimizer.PhysicsSurrogate`	Envoltório fino de um `nn.Module` theta -> u com previsão, jacobiano, treino em dataset e checkpoint.
`pp.PhysicsValidator`	import direto	`pinneapple_analysis.validation.PhysicsValidator`	Orquestra uma suíte de checagens físicas sobre um modelo treinado.
`pp.PhysicsWorldModel`	import direto	`pinneapple_worldmodel.PhysicsWorldModel`	World model FNO: `(state_t, context) -> state_{t+1}` com formulação residual.

nome	como é carregado	origem	o que faz
pp.pinn	submódulo preguiçoso (_SUBMODULES)	pinneapple_physics.pinn_solver	Subpacote do compilador de PINN; reexporta compile_problem, pesos de perda, DoMINO e utilidades de PINN estocástico.
pp.PINNArenaBenchmark	import direto	pinneapple_tools.benchmark_suite.PINNArenaBenchmark	Suíte que roda todas as combinações tarefa × modelo, monta leaderboard, melhor modelo por problema, tabela-resumo, gráficos e salva/carrega resultados.
pp.PINNRefinement	import direto	pinneapple_design.design_optimizer.PINNRefinement	Refina a previsão do surrogate treinando uma PINN com perda de dados + resíduo de EDP, para um ou para os k melhores designs.
pp.plot_isosurface_3d	import direto	pinneapple_neural.predictor.plot_isosurface_3d	Desenha a isosuperfície 3D de um campo escalar previsto.
pp.plot_streamlines_2d_from_model	import direto	pinneapple_neural.predictor.plot_streamlines_2d_from_model	Desenha linhas de corrente do campo de velocidade previsto por um modelo.
pp.plot_volume_slice	import direto	pinneapple_neural.predictor.plot_volume_slice	Desenha uma fatia 2D de um campo 3D previsto.
pp.PointObsOperator	import direto	pinneapple_analysis.inverse_problems.PointObsOperator	Avalia o modelo num conjunto fixo de posições de sensores.
pp.presets	submódulo preguiçoso (_SUBMODULES)	pinneapple_presets	Presets industriais: problemas de engenharia já montados, resolvidos e validados com o PINNeAPPle, acessíveis por pp.presets.names()/load(); reexporta o registro e as classes base.
pp.profiles	import direto	pinneapple_core.onboarding.profiles	Perfis aceitos pelo doctor e a frase que descreve cada um.
pp.propagate_tolerances	preguiçoso (_LAZY_ATTRS)	pinneapple_analysis.uncertainty.tolerance.propagate	Propaga tolerâncias independentes por fn via Monte Carlo.
pp.propagate_uncertainty	preguiçoso (_LAZY_ATTRS)	pinneapple_analysis.uncertainty.tolerance.propagate_space	Propaga entradas com qualquer distribuição por fn.
pp.qualitative	submódulo preguiçoso (_SUBMODULES)	pinneapple_design.qualitative	Reexporta preview, part_sensitivity, Assembly e os modelos qualitativos.
pp.Quantity	import direto	pinneapple_physics.physical_problem.Quantity	Quantidade de interesse que o usuário quer da solução, com unidade e campo de origem.
pp.quantum	submódulo preguiçoso (_SUBMODULES)	pinneapple_quantum	Fachada do Módulo Quântico (PQM): reexporta configuração de circuito, backends, modelos, perdas de Schrödinger, datasets, treinador e presets, e oferece os atalhos build_circuit e backend_info. [experimental] [sem teste]
pp.quickstart	definido em pinneapple/__init__.py	pinneapple.quickstart	Imprime o resumo de um preset (EDP, campos, coordenadas, domínio, condições) e os próximos passos sugeridos; devolve o spec.
pp.recommend	import direto	pinneapple_core.onboarding.recommend	pp.recommend: transforma uma frase em preset, método e modelo, com o motivo e os termos que casaram; é pontuação por palavras-chave e pode errar. [sem teste direto]
pp.rectangle	import direto	pinneapple_design.geometry.rectangle	Atalho: SDF de retângulo.

nome	como é carregado	origem	o que faz
`pp.register_method`	import direto	`pinneapple_physics.solving.register_method`	Decorador que registra um método de solução sob um nome e um tipo.
`pp.register_preset`	import direto	`pinneapple_physics.pde_environment.register_preset`	Decorador que registra uma função fábrica de preset por nome.
`pp.reporting`	submódulo preguiçoso (`_SUBMODULES`)	`pinneapple_tools.reporting`	Relatórios: estilo de figuras e construtor de PDF (`paperkit`) e datasheet técnico (`datasheet`); não é importado automaticamente pelo pacote.
`pp.ResidualBasedAL`	import direto	`pinneapple_data.ResidualBasedAL`	Escolhe pontos de colocação onde o resíduo da EDP é maior, pegando os top-k (RAR) ou sorteando com peso proporcional ao resíduo (RAD).
`pp.RigidBody`	import direto	`pinneapple_simulation.particle_dynamics.RigidBody`	Corpo rígido único com massa, inércia e integrador simplético.
`pp.RigidBodyState`	import direto	`pinneapple_simulation.particle_dynamics.RigidBodyState`	Contêiner mutável do estado (posição, orientação, velocidades) de vários corpos rígidos.
`pp.RigidBodySystem`	import direto	`pinneapple_simulation.particle_dynamics.RigidBodySystem`	Sistema de vários corpos rígidos com gravidade opcional.
`pp.run_parallel_sweep`	import direto	`pinneapple_neural.trainer.run_parallel_sweep`	Executa um sweep em grade em paralelo (threads ou processos) e devolve a lista de resultados.
`pp.run_tree`	preguiçoso (`_LAZY_ATTRS`)	`pinneapple_decision.api.run_tree`	Percorre a árvore de decisão de Physics AI (por padrão `physics_ai_tree()`).
`pp.science`	submódulo preguiçoso (`_SUBMODULES`)	`pinneapple_science`	Ponto de entrada (`pp.science`): reexporta `atlas`, `experiments`, `run` e os tipos centrais.
`pp.score_against_reference`	preguiçoso (`_LAZY_ATTRS`)	`pinneapple_analysis.verification.physics_confidence_score.score_against_reference`	Pontua as predições de vários modelos contra um conjunto de referência separado.
`pp.security`	submódulo preguiçoso (`_SUBMODULES`)	`pinneapple_security`	Ponto de entrada do pacote; o docstring é o índice das capacidades e diz as dependências (biblioteca padrão e NumPy; PyTorch para DP-SGD e carga segura; `cryptography` para Ed25519 e AES-GCM).
`pp.SelfAdaptiveWeights`	import direto	`pinneapple_neural.trainer.SelfAdaptiveWeights`	Pesos de perda aprendíveis (SA-PINN) atualizados por subida de gradiente, crescendo nos termos mais difíceis.
`pp.sensitivity`	submódulo preguiçoso (`_SUBMODULES`)	`pinneapple_analysis.sensitivity`	Análise de sensibilidade global e local; reexporta Sobol, Morris, curvas de efeito, o estudo completo e as funções de teste.
`pp.Sequential`	preguiçoso (`_LAZY_ATTRS`)	`pinneapple_core.module.Sequential`	Encadeia componentes (módulos, solvers ou funções), a saída de um é a entrada do próximo.
`pp.set_backend`	import direto	`pinneapple_tools.compute_backends.set_backend`	Define o backend ativo, validando o nome contra o registro de `pinneapple_core.backend`; nome desconhecido levanta `ValueError`.
`pp.ShapeParametrization`	import direto	`pinneapple_design.design_optimizer.ShapeParametrization`	Parametrização de forma para otimização por gradiente (pontos de controle FFD, curvas de Bézier ou família NACA), com deformação de malha.
`pp.SineLayer`	import direto	`pinneapple_neural.architectures.SineLayer`	Camada senoidal com inicialização de pesos de Sitzmann et al.

nome	como é carregado	origem	o que faz
pp.SIREN	import direto	pinneapple_neural.architectures.SIREN	Rede SIREN com omega_0 configurável e última camada linear por padrão.
pp.sobol_indices	preguiçoso (_LAZY_ATTRS)	pinneapple_analysis.sensitivity.sobol.sobol_indices	Índices de Sobol de primeira ordem e totais (e de segunda ordem, opcional) de fn.
pp.Solution	import direto	pinneapple_physics.solving.Solution	Resultado de solve: função predict, tempo, histórico, modelo, grade e tipo do método, com metadata uniforme.
pp.solve	import direto	pinneapple_physics.solving.solve	Resolve o problema com o método pedido (nome registrado ou callable).
pp.SolverModule	preguiçoso (_LAZY_ATTRS)	pinneapple_core.module.SolverModule	Um solver como componente, com parâmetros treináveis opcionais; from_method usa um backend de pp.solve (não diferenciável).
pp.solvers	submódulo preguiçoso (_SUBMODULES)	pinneapple_solvers	[shim] Pacote de compatibilidade (obsoleto) que reexporta os solvers de pinneapple_simulation.numerical_solvers; os solvers opcionais viram None se o import falhar. [sem teste direto do __init__]
pp.SpalartAllmarasResiduals	import direto	pinneapple_physics.pde_environment.SpalartAllmarasResiduals	Resíduos do modelo de uma equação de Spalart-Allmaras.
pp.SparsityRegularizer	import direto	pinneapple_analysis.inverse_problems.SparsityRegularizer	Regularizador L1/LASSO com aproximação suave.
pp.SPHParticles	import direto	pinneapple_simulation.particle_dynamics.SPHParticles	SPH fracamente compressível: densidade por soma de kernel, pressão de Tait, viscosidade e gravidade.
pp.StructuralObjective	import direto	pinneapple_design.design_optimizer.StructuralObjective	Proxy de tensão de von Mises pela média dos quadrados do campo.
pp.StudentTMisfit	import direto	pinneapple_analysis.inverse_problems.StudentTMisfit	Log-verossimilhança negativa de Student-t; nu controla a robustez.
pp.StudyModel	preguiçoso (_LAZY_ATTRS)	pinneapple_analysis.study_models.StudyModel	Envelope de modelo para estudos: predição, nomes, unidades, limites, desvio opcional e construtores de função, zoo e ONNX.
pp.Subdomain	import direto	pinneapple_physics.pinn_solver.Subdomain	Subdomínio retangular com sobreposição e testes de pertinência e interface.
pp.SubdomainPINN	import direto	pinneapple_physics.pinn_solver.SubdomainPINN	PINN de um único subdomínio.
pp.suggest_problem_spec	import direto	pinneapple_physics.pde_environment.suggest_problem_spec	Sugere argumentos de construção de ProblemSpec a partir de uma descrição em texto livre.
pp.SurrogateConfig	import direto	pinneapple_design.design_optimizer.SurrogateConfig	Configuração do PhysicsSurrogate: modelo (nome registrado ou MLP), dimensões e camadas.
pp.SweepConfig	import direto	pinneapple_neural.trainer.SweepConfig	Configuração do sweep em grade: grade de parâmetros, número de jobs, backend (threads/processos) e timeout.
pp.symbolic	submódulo preguiçoso (_SUBMODULES)	pinneapple_physics.symbolic_pde	Compilador simbólico de EDP: transforma resíduos SymPy em funções PyTorch com autograd e oferece condições de contorno rígidas e suaves; reexporta SymbolicPDE, HardBC etc. [sem teste direto do **init**]
pp.SymbolicPDE	import direto	pinneapple_physics.symbolic_pde.SymbolicPDE	Resíduo de EDP simbólico compilado para função PyTorch (autograd ou grade).
pp.t_junction	import direto	pinneapple_design.geometry.t_junction	Junção em T: canal horizontal mais ramo vertical.

nome	como é carregado	origem	o que faz
pp.terramechanics	submódulo preguiçoso (_SUBMODULES)	pinneapple_simulation.terramechanics	Terramecânica de rodas de rovers planetários: solos com proveniência, modelo de roda de Wong-Reece e extensões, terreno deformável com sulcos, bancada de roda única com dados publicados e substitutos PINN do contato.
pp.ThermalEfficiencyObjective	import direto	pinneapple_design.design_optimizer.ThermalEfficiencyObjective	Eficiência térmica (fluxo de calor sobre queda de pressão proxy), devolvida negativa para minimizar.
pp.ThroughputMonitor	import direto	pinneapple_neural.trainer.ThroughputMonitor	Mede amostras/s, memória da GPU e utilização (com pynvml, se houver) por época.
pp.TikhonovRegularizer	import direto	pinneapple_analysis.inverse_problems.TikhonovRegularizer	Regularizador de Tikhonov/ridge, com prior e covariância opcionais.
pp.TimeMarchingTrainer	import direto	pinneapple_neural.trainer.TimeMarchingTrainer	Treina uma rede por janela de tempo, congelando a predição no fim de cada janela como condição inicial da seguinte; avalia e plota o resultado completo.
pp.to_jsonable	preguiçoso (_LAZY_ATTRS)	pinneapple_core.jsonable.to_jsonable	Devolve uma cópia de um objeto que json.dumps aceita, com opção de arredondar e tratar não-finitos.
pp.TotalVariationRegularizer	import direto	pinneapple_analysis.inverse_problems.TotalVariationRegularizer	Variação total anisotrópica para parâmetros de campo 1D/2D.
pp.train	submódulo preguiçoso (_SUBMODULES)	pinneapple_train	[shim] Pacote de compatibilidade (marcado como obsoleto) que reexporta o treinador, callbacks e métricas de pinneapple_neural.trainer; nomes opcionais viram None se o import falhar. [sem teste direto do __init__]
pp.TrainConfig	import direto	pinneapple_neural.trainer.TrainConfig	Configuração do Trainer: épocas, taxa de aprendizado, clip, AMP, dispositivo, semente, determinismo, log, DDP e validação com física.
pp.Trainer	import direto	pinneapple_neural.trainer.Trainer	Treinador genérico: fit(train_loader, val_loader, cfg) roda épocas com perdas tensor/dict, métricas, early stopping, checkpoint e log JSONL.
pp.transfer	submódulo preguiçoso (_SUBMODULES)	pinneapple_adaptation.transfer_learning	Reexporta TransferConfig, TransferTrainer, utilitários de congelamento, ParametricFamilyTransfer e PhysicsTransferAdapter. [sem teste direto do __init__]
pp.TransferBenchmarkConfig	import direto	pinneapple_tools.benchmark_suite.TransferBenchmarkConfig	Configuração: épocas e taxas da fonte e do ajuste fino, estratégias, aquecimento, pontos e pesos das perdas.
pp.TransferBenchmarkPipeline	import direto	pinneapple_tools.benchmark_suite.TransferBenchmarkPipeline	Pipeline que roda todas as estratégias de transferência e os baselines do zero, monta leaderboard, gráficos e salva/carrega resultados.
pp.TransferBenchmarkResult	import direto	pinneapple_tools.benchmark_suite.TransferBenchmarkResult	Resultado de (cenário, alvo, estratégia): métricas, histórico, tempo e parâmetros treináveis e totais.
pp.TransferScenario	import direto	pinneapple_tools.benchmark_suite.TransferScenario	Um cenário de transferência: uma tarefa fonte e N tarefas alvo, com rótulos.
pp.transforms	submódulo preguiçoso (_SUBMODULES)	pinneapple_core.transforms	Transformações invertíveis e componíveis (>>) sobre tabelas, PhysicsDataset e, para as de escala, PhysicalProblem, registrando proveniência a cada apply; as não-escalares levantam NotImplementedError em nível de problema.
pp.trust_card	preguiçoso (_LAZY_ATTRS)	pinneapple_analysis.trust.field_card.trust_card	Roda field_checks e produz a decisão de deploy e as seis perguntas do cartão de confiança.

nome	como é carregado	origem	o que faz
pp.TwoPhaseConfig	import direto	pinneapple_neural.trainer.TwoPhaseConfig	Configuração das duas fases: épocas e taxas de cada fase, modo combinado e rampa do peso da física.
pp.TwoPhaseHistory	import direto	pinneapple_neural.trainer.TwoPhaseHistory	Histórico de perdas das duas fases (incluindo componentes EDP e contorno na fase 2).
pp.TwoPhaseTrainer	import direto	pinneapple_neural.trainer.TwoPhaseTrainer	Executa a fase supervisionada e depois a fase de física, com otimizadores e históricos separados.
pp.uncertainty	submódulo preguiçoso (_SUBMODULES)	pinneapple_analysis.uncertainty	Quantificação de incerteza (antigo pinneapple_uq); reexporta aleatória, epistêmica, conformal, calibração, quantis, reamostragem, drift, tolerâncias e uq_predict.
pp.UnnormModel	import direto	pinneapple_neural.trainer.UnnormModel	Envolve um modelo treinado em alvos normalizados para devolver predições na escala física (y * std + mean).
pp.uq	submódulo preguiçoso (_SUBMODULES)	pinneapple_analysis.uncertainty	Quantificação de incerteza (antigo pinneapple_uq); reexporta aleatória, epistêmica, conformal, calibração, quantis, reamostragem, drift, tolerâncias e uq_predict.
pp.uq_predict	import direto	pinneapple_analysis.uncertainty.uq_predict	Interface unificada de predição com incerteza (método escolhido por method).
pp.use_backend	preguiçoso (_LAZY_ATTRS)	pinneapple_core.backend.use_backend	Gerenciador de contexto que roda um bloco num backend e restaura o anterior.
pp.validate	preguiçoso (_LAZY_ATTRS)	pinneapple_analysis.validation.validate.pp_validate	Uma chamada, um relatório de validação de um modelo (zoo, PyTorch, ONNX ou função) contra um problema, com credibilidade por fator e decisão pass / warn / fail (pp_validate = _PPValidate()).
pp.validate_model	import direto	pinneapple_analysis.validation.validate_model	Validação rápida de uma vez a partir de um ProblemSpec ou argumentos explícitos.
pp.vehicle	submódulo preguiçoso (_SUBMODULES)	pinneapple_simulation.vehicle	Veículos planetários com rodas: descrição (chassi + módulos de canto) com CAD, modelo multicorpo construído a partir dela e simulador em terreno deformável com rodas de Wong-Reece.
pp.verify	preguiçoso (_LAZY_ATTRS)	pinneapple_decision.api.verify	Verifica um resultado; só resultados verificados viram evidência.
pp.veriphysics	submódulo preguiçoso (_SUBMODULES)	pinneapple_veriphysics	Camada de verificação e evidência do Veriphysics dentro do PINNeAPPle; o __init__ só documenta os submódulos (decisão, relatório, aplicabilidade, escopo, robustez, recomendação, ledger, Validation Lab).
pp.viz	submódulo preguiçoso (_SUBMODULES)	pinneapple_tools.visualization.studio	Studio do PINNeAPPle: visualização 3D realista de qualquer geometria e sua física (cena, visualizador web, Blender, PyVista); exporta Scene e atalhos de render.
pp.vmap_residual	import direto	pinneapple_tools.compute_backends.vmap_residual	Alias de módulo para JAXBackend.vmap_residual (vetoriza um resíduo de um ponto sobre um lote).
pp.WeightMinimizationObjective	import direto	pinneapple_design.design_optimizer.WeightMinimizationObjective	Proxy de volume pela norma L2 do vetor de parâmetros.
pp.WeightScheduler	import direto	pinneapple_neural.trainer.WeightScheduler	Interface única: step(losses) aplica os pesos atuais, atualiza conforme o método e expõe parâmetros aprendíveis quando houver.
pp.WeightSchedulerConfig	import direto	pinneapple_neural.trainer.WeightSchedulerConfig	Configuração do agendamento de pesos: método (de self_adaptive até auto), pesos iniciais, frequência, taxas, limites e decaimento EMA.

nome	como é carregado	origem	o que faz
`pp.worldmodel`	submódulo preguiçoso (`_SUBMODULES`)	`pinneapple_worldmodel`	Ponto de entrada do pacote: reexporta o pipeline, o orquestrador, cenários, modelos e fábricas; o docstring descreve o pipeline de cinco estágios do modelo de fundação física.
`pp.WorldModelConfig`	import direto	`pinneapple_worldmodel.WorldModelConfig`	Hiperparâmetros do `PhysicsWorldModel`.
`pp.WorldModelDataset`	import direto	`pinneapple_worldmodel.WorldModelDataset`	Dataset de previsão de um ou vários passos construído de uma lista de trajetórias.
`pp.WorldModelTrainer`	import direto	`pinneapple_worldmodel.WorldModelTrainer`	Treina um `PhysicsWorldModel` num `WorldModelDataset`.
`pp.zoo`	submódulo preguiçoso (`_SUBMODULES`)	`pinneapple_hub.zoo`	`pp.zoo`: modelos pré-treinados e verificados em uma linha (listar, carregar, prever, avaliar, validar, consultar checagens, exportar, fazer benchmark e fine-tune com guarda).

Como este documento foi feito

1. `docs/pt/_mapa/inventario.py` percorre o repositório por AST e grava `inventario.json`.
2. O texto em português de cada pacote, módulo e símbolo foi escrito lendo o código e está em `docs/pt/_mapa/prosa/*.json`; `docs/pt/_mapa/checar_prosa.py` confere que nada ficou sem texto.
3. `docs/pt/_mapa/gerar.py` monta os volumes. Quando falta texto em português para um símbolo, a tabela mostra a primeira linha da docstring original (em inglês).
4. `docs/pt/_mapa/pdf.js` gera o PDF.

Volume 1 — Núcleo, porta de entrada e catálogo

> Parte do Mapa completo do PINNeAPPle. Gerado por `docs/pt/_mapa/gerar.py` a partir do inventário do commit `e53cdcfb04`.

A porta de entrada `pp.*`, os primitivos do núcleo, o catálogo de métodos, o registro de artefatos e os shims de compatibilidade.

Sumário

1. pinneapple
2. pinneapple_core
3. pinneapple_catalog
4. pinneapple_registry
5. pinneapple_orchestration
6. pinneapple_models
7. pinneapple_train
8. pinneapple_solvers

1. `pinneapple`

Objetivo

`pinneapple` é a fachada da biblioteca: o `import pinneapple as pp` que o usuário escreve primeiro. Ele não implementa física própria. Reexporta, em blocos `try/except`, funções e classes dos pacotes de verdade: presets e specs de problemas (`pinneapple_physics`), arquiteturas e treino (`pinneapple_neural`), amostragem (`pinneapple_data`), geometria e otimização de projeto (`pinneapple_design`), incerteza, validação e problemas inversos (`pinneapple_analysis`), gêmeo digital (`pinneapple_systems`), solvers e dinâmica (`pinneapple_simulation`), exportação e benchmarks (`pinneapple_tools`) e world models (`pinneapple_worldmodel`).

O problema que resolve é dar um ponto de entrada curto e estável para uma biblioteca grande. Além das reexportações, um `__getattr__` de módulo dá acesso preguiçoso a dezenas de subpacotes por apelido (`pp.solvers`, `pp.cfd`, `pp.zoo`, `pp.datasets`, `pp.presets`, `pp.vehicle`, `pp.decision` etc.) e a funções isoladas (`pp.decide`, `pp.trust_card`, `pp.grad`, `pp.sobol_indices`...): o pacote só é importado no primeiro acesso, e `import pinneapple` não importa a camada de decisão.

Duas funções próprias ajudam no começo: `quickstart` mostra o resumo de um preset (EDP, campos, domínio, condições) e os próximos passos sugeridos (`pp.solve`, `pp.card`, `pp.compare`, `pp.new`), e `info` mostra versão, dispositivos, pacotes importáveis e dependências opcionais com o extra de pip que as instala. O subpacote `pinneapple.check` é só um shim que expõe o linter de física de `pinneapple_check`, inclusive como `python -m pinneapple.check`.

Números do inventário

módulos	linhas	classes públicas	funções públicas	testes em arquivos que importam o pacote	módulos sem teste direto
3	866	0	2	431 (39 arquivos)	2

Submódulos

submó-dulo	o que faz	módu-los	classes + funções	tes-tes*
`check`	Shim que reexporta o linter de física `pinneapple_check` e permite rodá-lo com `python -m pinneapple.check [paths]`.	2	0	0

* maior número de testes que importam um módulo do submódulo.

Módulos e API pública

`pinneapple`

Fachada `pp`: reexporta símbolos dos pacotes da biblioteca, dá acesso preguiçoso a subpacotes e funções por apelido e define `quickstart` e `info`.

`pinneapple/__init__.py` · 854 linhas · testes: 431

nome	tipo	assinatura	o que faz
`quickstart`	função	`(problem_id='burgers_1d', **problem_kwargs)`	Imprime o resumo de um preset (EDP, campos, coordenadas, domínio, condições) e os próximos passos sugeridos; devolve o spec.
`info`	função	`()`	Imprime versão, dispositivos, número de presets e modelos, pacotes importáveis e dependências opcionais com o extra de pip.

`pinneapple.check`

Shim: reexporta tudo de `pinneapple_check` (o linter de física). [shim] [sem teste direto]

`pinneapple/check/__init__.py` · 4 linhas · **sem teste direto**

`pinneapple.check.__main__`

Permite `python -m pinneapple.check [paths]`, que chama `check_main` de `pinneapple_check.cli`. [shim] [sem teste direto]

`pinneapple/check/__main__.py` · 8 linhas · **sem teste direto**

Exemplos de uso reais do repositório

Primeiro resultado: Burgers com PINN contra a solução exata — `examples/first_example/first_example.py`

```python
import pinneapple as pp

prob = pp.PhysicalProblem.from_preset("burgers_1d", nu=0.01 / np.pi)   # u_t + u u_x = nu u_xx, x in [-1, 1]
pinn = pp.solve(prob, "pinn", epochs=1500, seed=0)                     # train the network (about 45 s)
exact = pp.solve(prob, "analytic")                                     # Cole-Hopf closed form

x = np.linspace(-1, 1, 201)
pts = np.stack([x, np.full_like(x, 0.75)], axis=1)                     # (x, t = 0.75): the steep front
u_pinn, u_exact = pinn.predict(pts)[:, 0], exact.predict(pts)[:, 0]
print(f"relative L2 error at t = 0.75: {pp.metrics.relative_l2(u_pinn, u_exact):.3f}")
```

Testes e status de validação

- Módulos com pelo menos um teste que os importa: 1 de 3.
- Funções de teste em arquivos que importam algum módulo do pacote: 431.

Limitações conhecidas

- As reexportações ficam em blocos `try/except` que ignoram falhas: se um pacote ou dependência faltar, o nome simplesmente não existe em `pp`, sem aviso.
- O acesso preguiçoso só falha no primeiro uso (`pp.<apelido>`), com o ImportError do pacote de destino.
- Alguns apelidos apontam para shims de compatibilidade (`pp.models` → `pinneapple_models`, `pp.train` → `pinneapple_train`, `pp.solvers` → `pinneapple_solvers`).
- `pinneapple.check` e `pinneapple.check.__main__` não têm teste direto e dependem de `pinneapple_check` instalado.

2. `pinneapple_core`

Objetivo

O `pinneapple_core` reúne as primitivas compartilhadas da biblioteca: `Domain` (a região do problema), `Geometry` (um domínio com fronteiras nomeadas e um gerador de malha), `Mesh` (malha simplicial 1D/2D/3D com cálculo P1 exato) e `Field`/`FunctionField` (valores de uma grandeza numa grade, malha, nuvem de pontos ou numa função torch, como uma PINN). Todo campo responde `gradient()`, `divergence()`, `interpolate(x)` e `integrate()`, seja qual for a discretização. O módulo `operators` oferece uma função por operador (`grad`, `div`, `curl`, `laplacian`, `jacobian`, `hessian`, `integrate`, `flux`) que funciona igual em todas essas representações.

Sobre essas primitivas, o pacote traz as peças de treino e otimização usadas pelo resto do projeto: `pp.func` (transformações de `torch.func` com `wrt` por nome e diferenciação implícita através de solvers), `pp.loss` (termos de perda funcionais e um `Balancer` com estratégias como GradNorm, NTK e ReLoBRaLo), `pp.transforms` (transformações invertíveis com proveniência, inclusive adimensionalização de um `PhysicalProblem`), `data` (`PhysicsDataset`, `DataLoader` e amostradores de colocação, fronteira, malha, trajetória, adaptativo e ativo), `optim` (`PhysicsOptimizer` com SLSQP, penalidade, Lagrangiano aumentado, evolução diferencial etc.), `module` (`PhysicsModule`, `Sequential`, `SolverModule` e `Hybrid` para compor redes e solvers) e `fem` (elementos finitos P1 diferenciáveis em torch, para gradientes de forma).

O pacote `pinneapple` (o `pp.*`) reexporta boa parte disso de forma preguiçosa: `pp.loss`, `pp.func`, `pp.transforms`, `pp.PhysicsOptimizer`, `pp.PhysicsModule`, `pp.grad`/`pp.div`/`pp.laplacian`, `pp.use_backend` e o `PhysicsBackend` (mesma API sobre torch e jax). O `__init__` carrega os submódulos que dependem de torch só no primeiro uso.

Três subpacotes atendem o resto do ecossistema. `onboarding` é a primeira hora do usuário: `pp.doctor` (checa a instalação por perfil), `pp.recommend` (frase em inglês ou português vira preset, método e snippet, por pontuação de palavras-chave), `card`/catálogo de presets de `pinneapple_physics` e `pp.new` (gera um projeto independente). `service` dá às apps em `apps/*` e ao servidor MCP fila de jobs em SQLite, chaves de API, medição de uso e uploads, só com a biblioteca padrão. `workflows` orquestra simulações como um DAG de passos (adaptadores `command`, `python`, `openfoam`, `calculix`; executores `local`, `docker`, `slurm`), com retomada, cache por hash de conteúdo e geração de datasets a partir das execuções.

Números do inventário

módulos	linhas	classes públicas	funções públicas	testes em arquivos que importam o pacote	módulos sem teste direto
34	8.747	71	92	233 (20 arquivos)	15

Pasta de testes dedicada: `tests/pinneapple_core` (132 funções de teste).

Submódulos

submódulo	o que faz	módulos	classes + funções	testes*
backend	`PhysicsBackend`: uma API de arrays, derivadas e solves sobre torch, jax ou outro motor registrado.	1	7	5

submódulo	o que faz	módulos	classes + funções	testes*
`data`	API de dados física: `PhysicsDataset`, `DataLoader` e amostradores (colocação, fronteira, malha, trajetória, adaptativo, ativo).	1	10	25
`domain`	`Domain`: caixa alinhada aos eixos, opcionalmente cortada por uma função distância com sinal.	1	1	0
`fem`	Elementos finitos P1 densos e diferenciáveis em torch (rigidez, massa, Poisson) para gradientes de forma.	1	5	25
`field`	`Field`, `Grid`, `PointCloud` e `FunctionField`: valores numa grade, malha, nuvem ou função, com gradiente, integral e interpolação.	1	4	0
`func`	`pp.func`: `grad`/`jacobian`/`hessian`/`vmap` com `wrt` por nome, e diferenciação implícita (`implicit_solve`).	1	8	10
`geometry`	`Geometry`: um `Domain` com fronteiras nomeadas e um gerador de malha.	1	1	0
`jsonable`	Conversão de resultados para JSON seguro (NumPy para Python, NaN/Inf para `None`, arredondamento).	1	2	8
`loss`	`pp.loss`: termos de perda funcionais e o `Balancer` de pesos escolhido por nome.	1	11	22
`mesh`	`Mesh`: malhas simpliciais (intervalos, triângulos, tetraedros) com cálculo P1.	1	1	0
`module`	`PhysicsModule` e composição de redes e solvers (`Sequential`, `SolverModule`, `Hybrid`).	1	5	7
`onboarding`	A primeira hora: `doctor`, `recommend`, catálogo de presets (`card`) e `new_project`, com CLI.	6	32	34
`operators`	Operadores físicos (`grad`, `div`, `curl`, `laplacian`, `jacobian`, `hessian`, `integrate`, `flux`) para todas as representações.	1	8	0
`optim`	`PhysicsOptimizer`: parâmetros, objetivo e restrições escritos uma vez e minimizados por várias famílias de otimizador.	1	6	8
`service`	Peças de runtime para servidores de apps (fila de jobs, chaves de API, medição de uso, uploads), só com a biblioteca padrão.	6	18	30
`transforms`	`pp.transforms`: transformações invertíveis e componíveis para tabelas, datasets e problemas, com proveniência.	1	10	13
`workflows`	Workflows de simulação como DAG de passos, com executores e adaptadores plugáveis, execuções rastreáveis e datasets a partir das execuções.	7	34	64

* maior número de testes que importam um módulo do submódulo.

Módulos e API pública

`pinneapple_core`

Ponto de entrada do pacote: exporta `Domain`, `Geometry`, `Mesh`, `Field`, `FunctionField`, `Grid`, `PointCloud` e os operadores, e carrega `func`, `fem`, `loss`, `optim`, `data`, `transforms` etc. preguiçosamente no primeiro acesso.

`pinneapple_core/__init__.py` · 49 linhas · testes: 79

`pinneapple_core.backend`

Define `PhysicsBackend`, interface única de arrays, derivadas e solves, com implementações para torch e jax (float64 por padrão) e um registro para novos motores; a classe base levanta `NotImplementedError` nos métodos abstratos.

`pinneapple_core/backend.py` · 233 linhas · testes: 5 · palavras-chave no código (heurística; o texto acima diz o status real): levanta `NotImplementedError` em algum caminho

nome	tipo	assinatura	o que faz
`PhysicsBackend`	classe	`(name, xp)`	Interface base que todo motor implementa (arrays, `solve`, `integrate`, `grad`, `jacobian`, `vmap`, `jit`); os métodos abstratos levantam `NotImplementedError`.
`TorchPhysicsBackend`	classe	`() -> None`	Implementação de `PhysicsBackend` sobre torch.
`JaxPhysicsBackend`	classe	`() -> None`	Implementação de `PhysicsBackend` sobre jax, com `jax_enable_x64` ligado para casar com os outros motores.
`register_backend`	função	`(name, cls) -> None`	Registra uma classe de backend sob um nome para `set_backend`/`use_backend`; instanciada sob demanda, uma vez.

nome	tipo	assinatura	o que faz
list_backends	função	() -> list	Lista os nomes de backends registrados.
get_physics_backend	função	(name=None) -> PhysicsBackend	Devolve o objeto backend pelo nome ou o backend ativo.
use_backend	função	(name)	Gerenciador de contexto que roda um bloco num backend e restaura o anterior.

pinneapple_core.data

API de dados para PINN, operador neural e problema inverso: um DataLoader recebe datasets e amostradores e gera um Batch por passo, marcando requires_grad nos pontos de colocação.

pinneapple_core/data.py · 432 linhas · testes: 25 · palavras-chave no código (heurística; o texto acima diz o status real): levanta NotImplementedError em algum caminho

nome	tipo	assinatura	o que faz
Batch	classe	(groups, step, meta)	Dados de um passo de treino, um grupo por fonte (batch["col"]["x"]), com atalhos x/y quando há uma só fonte.
PhysicsDataset	classe	(inputs, targets=None, *, coords=None, fields=None, domain=None, name='', provenance=None…)	Dataset rotulado inputs[i] -> targets[i] com nomes de coordenadas e campos, domínio e proveniência; suporta split.
Sampler	classe	(coords)	Base dos amostradores que geram pontos sob demanda; subclasses implementam sample (a base levanta NotImplementedError).
CollocationSampler	classe	(domain, strategy='uniform', seed=0) -> None	Pontos interiores para o resíduo da PDE, com estratégia uniform, lhs ou sobol.
BoundarySampler	classe	(domain, part=None, seed=0) -> None	Pontos de fronteira com normais unitárias externas, na fronteira toda ou numa parte nomeada de uma Geometry.
MeshSampler	classe	(mesh, mode='nodes', seed=0) -> None	Pontos ligados a uma malha: nós, centros de células ou pontos aleatórios dentro das células.
TrajectorySampler	classe	(states, n_in=1, n_out=1, stride=1, seed=0) -> None	Janelas de trajetórias (B, T, d): n_in estados como entrada e os n_out seguintes como alvo.
AdaptiveSampler	classe	(base, residual_fn, pool_size=20000, exponent=1.0, uniform_fraction=0.2, seed=0) -> None	Amostragem adaptativa pelo resíduo: sorteia de um conjunto de candidatos com probabilidade crescente no resíduo, mais uma fração uniforme.
ActiveSampler	classe	(base, score_fn, n_candidates=2000) -> None	Seleção por aquisição: sorteia candidatos e devolve os de maior score_fn.
DataLoader	classe	(sources, batch_size=256, steps=None, shuffle=True, drop_last=False, seed=0, physics_awar…)	Gera lotes de datasets e amostradores; com physics_aware liga requires_grad, ajusta dtype/dispositivo e anexa nomes de coordenadas.

pinneapple_core.domain

Domain como caixa em qualquer dimensão ou caixa cortada por SDF (formas CSG 2D de pinneapple_design): pertinência, volume e amostragem de interior e fronteira; fronteira de domínios não-caixa só em 2D (NotImplementedError). [sem teste direto]

pinneapple_core/domain.py · 190 linhas · **sem teste direto** · palavras-chave no código (heurística; o texto acima diz o status real): levanta NotImplementedError em algum caminho

nome	tipo	assinatura	o que faz
Domain	classe	(lo, hi, *, sdf=None, shape=None) -> None	Região de R^d: caixa ou caixa cortada por SDF, com pertinência, volume e amostragem de interior e fronteira.

pinneapple_core.fem

Elementos finitos P1 em torch com matrizes densas, de modo que a solução é diferenciável em relação às coordenadas dos vértices; pensado para malhas de até alguns milhares de nós.

pinneapple_core/fem.py · 110 linhas · testes: 25

nome	tipo	assinatura	o que faz
`cell_geometry`	função	`(points, cells) -> tuple`	Volumes e gradientes baricêntricos de cada célula, diferenciáveis nas coordenadas dos pontos.
`stiffness`	função	`(points, cells, kappa=1.0) -> Tensor`	Matriz de rigidez densa `(N, N)` com coeficiente `kappa`.
`mass`	função	`(points, cells) -> Tensor`	Matriz de massa consistente densa `(N, N)`.
`integrate_p1`	função	`(points, cells, values) -> Tensor`	Integral do interpolante P1 de valores nodais, diferenciável nos pontos e nos valores.
`solve_poisson`	função	`(points, cells, source, dirichlet_nodes, dirichlet_values=0.0, kappa=1.0) -> Tensor`	Resolve `-div(kappa grad u) = source` com Dirichlet em nós dados, diferenciável pela solução linear.

`pinneapple_core.field`

`Field` liga valores ao suporte (`Grid`, `Mesh` ou `PointCloud`) e oferece gradiente, divergência, interpolação e integral; `FunctionField` dá a mesma interface para uma função torch via autograd. [sem teste direto]

`pinneapple_core/field.py` · 407 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Grid`	classe	`(axes) -> None`	Grade produto tensorial definida por eixos 1D estritamente crescentes.
`PointCloud`	classe	`(points, *, domain=None, weights=None, k_neighbors=None) -> None`	Pontos dispersos com pesos de quadratura opcionais e vizinhos para derivadas.
`Field`	classe	`(values, support, name=None) -> None`	Valores de uma grandeza numa `Grid`, `Mesh` ou `PointCloud`, com gradiente, divergência, interpolação e integral.
`FunctionField`	classe	`(fn, domain=None, dim=None) -> None`	Campo definido por uma função torch (expressão analítica ou rede, como numa PINN), com derivadas por autograd.

`pinneapple_core.func`

Transformações de função (`grad`, `jacrev`, `jacfwd`, `jacobian`, `hessian`, `vmap`) sobre `torch.func` com `wrt` por nome ou posição, e `implicit_solve`, que diferencia através de um solver por uma solução adjunta.

`pinneapple_core/func.py` · 180 linhas · testes: 10

nome	tipo	assinatura	o que faz
`grad`	função	`(fn, wrt=0, *, has_aux=False) -> Callable`	Gradiente de uma função escalar em relação ao(s) argumento(s) `wrt`, por nome ou posição.
`jacrev`	função	`(fn, wrt=0, *, has_aux=False) -> Callable`	Jacobiana por modo reverso (barata com poucas saídas).
`jacfwd`	função	`(fn, wrt=0, *, has_aux=False) -> Callable`	Jacobiana por modo direto (barata com poucas entradas).
`jacobian`	função	`(fn, wrt=0, *, mode='auto') -> Callable`	Jacobiana com `mode` `rev`, `fwd` ou `auto`.
`hessian`	função	`(fn, wrt=0) -> Callable`	Hessiana de uma função escalar em relação a `wrt`.
`vmap`	função	`(fn, in_dims=0, out_dims=0, **kwargs) -> Callable`	Vetoriza uma função sobre uma dimensão de lote (`torch.func.vmap`).
`newton_solve`	função	`(residual, x0, *params, tol=1e-10, max_iter=50) -> Tensor`	Iteração de Newton amortecida para `residual(x, *params) = 0` com Jacobiana densa.
`implicit_solve`	função	`(residual, x0, *params, solver=None, tol=1e-10, max_iter=50) -> Tensor`	Resolve `residual(x, *params) = 0` e diferencia pela solução com o teorema da função implícita (uma solução adjunta no backward).

`pinneapple_core.geometry`

`Geometry` acrescenta fronteiras nomeadas (`x_min`, `x_max`, ...) e geração de malha a um `Domain`; malhar domínios não-caixa só funciona em 2D (`NotImplementedError`). [sem teste direto]

pinneapple_core/geometry.py · 114 linhas · **sem teste direto** · palavras-chave no código (heurística; o texto acima diz o status real): levanta NotImplementedError em algum caminho

nome	tipo	assinatura	o que faz
Geometry	classe	(domain, boundaries=None) -> None	Um Domain com partes de fronteira nomeadas e um método mesh.

pinneapple_core.jsonable

Implementação única da conversão de resultados para JSON usada por apps, servidor MCP, lab e ferramentas de dados: escalares e arrays NumPy viram Python, NaN/Inf viram None.

pinneapple_core/jsonable.py · 62 linhas · testes: 8

nome	tipo	assinatura	o que faz
round_sig	função	(x, digits=6, small=0.001) -> float	Arredonda por casas decimais para valores grandes e por dígitos significativos para valores pequenos.
to_jsonable	função	(obj, *, digits=None, nonfinite=None, drop_private=False, unknown='keep') -> Any	Devolve uma cópia de um objeto que json.dumps aceita, com opção de arredondar e tratar não-finitos.

pinneapple_core.loss

Termos de perda como funções simples (pde, boundary, supervised, conservation, energy, symmetry, inverse), soma ponderada (combine) e o Balancer de pesos dinâmicos escolhido por nome.

pinneapple_core/loss.py · 266 linhas · testes: 22

nome	tipo	assinatura	o que faz
Balancer	classe	(strategy='fixed', names=None, model=None, weights=None, update_every=100, *, schedule=No…)	Ponderação de perdas escolhida por nome de estratégia (fixa, aprendida, GradNorm, NTK, ReLoBRaLo, SoftAdapt, currículo etc.).
causal_weights	função	(residual, t, n_chunks=32, epsilon=1.0) -> Tensor	Pesos causais por ponto, que decaem com o resíduo acumulado de fatias de tempo anteriores.
pde	função	(residual, *, weights=None, t=None, causal_epsilon=None, n_chunks=32) -> Tensor	Resíduo quadrático médio, com pesos opcionais e versão causal no tempo.
boundary	função	(pred, target=0.0) -> Tensor	Erro quadrático médio em relação aos valores de contorno ou inicial.
supervised	função	(pred, target, kind='mse', *, delta=1.0) -> Tensor	Ajuste a dados com mse, mae, huber ou relative_l2.
conservation	função	(quantity, target=0.0, scale=None) -> Tensor	Desvio quadrático de uma grandeza conservada em relação ao alvo, normalizado por escala.
energy	função	(e, reference=None, mode='conserve') -> Tensor	Penalidade sobre a energia ao longo do tempo: conservada ou não crescente.
symmetry	função	(model, x, transform, output_transform=None) -> Tensor	Penalidade por quebrar uma simetria: mean((f(T x) - S f(x))^2).
inverse	função	(pred, observed, params=None, prior=None, reg=0.0) -> Tensor	Desajuste às observações mais regularização dos parâmetros desconhecidos em direção a um prior.
combine	função	(terms, weights=None) -> Tensor	Soma ponderada de termos nomeados; peso para termo ausente levanta KeyError.
list_strategies	função	() -> List[str]	Lista os nomes de estratégias aceitos pelo Balancer.

pinneapple_core.mesh

Mesh com pontos (N, d) e células (M, d+1); campos nodais P1, integral exata para dados P1 e gradiente exato para dados lineares. [sem teste direto]

pinneapple_core/mesh.py · 291 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Mesh`	classe	`(points, cells) -> None`	Malha simplicial não estruturada em R^d (d = 1, 2 ou 3) com construtores estruturados e cálculo P1.

pinneapple_core.module

`PhysicsModule` é o `nn.Module` da física: redes, solvers e híbridos se compõem com `Sequential`, `SolverModule` e `Hybrid`, e têm um `fit` simples.

`pinneapple_core/module.py` · 234 linhas · testes: 7

nome	tipo	assinatura	o que faz
`PhysicsModule`	classe	`(kind)`	Classe base de modelos e solvers com `kind`, `describe()`, `num_parameters()` e um `fit` simples.
`Lambda`	classe	`(fn) -> None`	Envolve uma função simples sem parâmetros como componente.
`Sequential`	classe	`(*parts) -> None`	Encadeia componentes (módulos, solvers ou funções), a saída de um é a entrada do próximo.
`SolverModule`	classe	`(fn, params=None, trainable=(), name=None) -> None`	Um solver como componente, com parâmetros treináveis opcionais; `from_method` usa um backend de `pp.solve` (não diferenciável).
`Hybrid`	classe	`(solver, corrector, mode='residual', corrector_input=None, detach_solver=False) -> None`	Solver mais correção neural (por exemplo modo `residual`), treinável através do solver.

pinneapple_core.onboarding

Subpacote da primeira hora com a biblioteca: reexporta `doctor`, `recommend`, `card` e `new_project`.

`pinneapple_core/onboarding/__init__.py` · 38 linhas · testes: 34

pinneapple_core.onboarding.catalog

Catálogo de presets: completa o registro de `pinneapple_physics.pde_environment.presets` com descrição, tags, método que roda hoje e um snippet executável; a busca é por palavras-chave (inglês ou português), não um modelo.

`pinneapple_core/onboarding/catalog.py` · 817 linhas · testes: 34

nome	tipo	assinatura	o que faz
`Card`	classe	`(name, domain, description, tags, coordinates, fields, spatial_dim, transient, domain_bou…)`	Descrição de um preset: domínio, coordenadas, campos, condições, solver, método, snippet, presets relacionados e disponibilidade.
`best_method`	função	`(name) -> str`	O método mais barato que roda hoje para o preset: analítico, senão solver de referência, senão rede treinada.
`needs_geometry_tags`	função	`(name) -> bool`	Diz se o preset liga condições de contorno a tags de geometria nomeadas.
`tag_names`	função	`(name) -> Tuple[str, ...]`	Tags de geometria citadas pelas condições do preset.
`default_geometry_note`	função	`(name) -> str`	Nota sobre qual geometria canônica o preset fornece para as fronteiras com tag, ou string vazia.
`trainable`	função	`(name) -> bool`	Diz se dá para treinar uma rede neste preset só pelo nome.
`probe`	função	`(name) -> Tuple[bool, str]`	Tenta uma vez o método mais barato do preset e informa se funcionou, com o motivo.
`runnable`	função	`(name) -> bool`	Diz se o preset produz um campo nesta máquina agora, sem geometria extra nem solver a instalar.
`working_method`	função	`(name) -> str`	O método que funciona aqui para o preset (o sondado após fallback, senão o preferido).

nome	tipo	assinatura	o que faz
`solver_available`	função	`(solver) -> bool`	Diz se esta máquina consegue rodar o solver de referência nomeado.
`solver_requirement`	função	`(solver) -> str`	Comando de instalação de um solver de referência ausente, ou string vazia.
`snippet`	função	`(name, method=None, *, train=False) -> str`	Menor programa correto para um preset, executável como impresso.
`standalone_snippet`	função	`(name) -> str`	Programa que resolve a mesma física do preset descrita do zero.
`card`	função	`(name, *, with_related=True, check=True) -> Card`	Monta o `Card` de um preset; nome desconhecido levanta `KeyError` com sugestões próximas.
`describe`	função	`(name) -> Dict[str, Any]`	O `card` de um preset como dicionário pronto para JSON.
`catalogue`	função	`() -> List[Card]`	Todos os presets registrados como cards, ordenados por domínio e nome.
`domains`	função	`() -> Dict[str, List[str]]`	Nomes de presets agrupados por domínio.
`format_card`	função	`(c, width=78) -> str`	Formata um card como texto simples, sem cor nem caixas.
`search`	função	`(text, top=5, *, min_score=0.0) -> List[Dict[str, Any]]`	Ordena presets contra um texto livre (inglês ou português) por pontuação de palavras-chave.

pinneapple_core.onboarding.cli

Comandos `pinneapple doctor | recommend | card | new`, todos offline e com saída de texto simples.

`pinneapple_core/onboarding/cli.py` · 153 linhas · testes: 34

nome	tipo	assinatura	o que faz
`doctor_main`	função	`(argv) -> int`	Implementa o subcomando `pinneapple doctor`.
`recommend_main`	função	`(argv) -> int`	Implementa o subcomando `pinneapple recommend`.
`card_main`	função	`(argv) -> int`	Implementa o subcomando `pinneapple card`.
`new_main`	função	`(argv) -> int`	Implementa o subcomando `pinneapple new`.
`main`	função	`(argv=None) -> int`	Despacha `pinneapple <doctor|recommend|card|new> ...`.

pinneapple_core.onboarding.doctor

`pp.doctor`: verifica a instalação contra um perfil (`start`, `cfd`, `research`, `enterprise`, `all`) e imprime um comando de correção por falha.

`pinneapple_core/onboarding/doctor.py` · 319 linhas · testes: 34

nome	tipo	assinatura	o que faz
`Check`	classe	`(name, ok, detail, fix, required)`	Um item verificado, com resultado (`None` se não pôde rodar), detalhe, correção e se é obrigatório.
`DoctorReport`	classe	`(profile, checks)`	Relatório do `doctor` para um perfil, com a lista de checagens e as falhas.
`profiles`	função	`() -> Dict[str, str]`	Perfis aceitos pelo `doctor` e a frase que descreve cada um.
`doctor`	função	`(profile='start', *, verbose=True) -> DoctorReport`	Verifica a instalação contra um perfil e, com `verbose`, imprime o relatório.

pinneapple_core.onboarding.newproject

`pp.new` / `pinneapple new`: escreve um projeto independente deste repositório, com modelos `plain` (só YAML), `research` e `cfd`.

`pinneapple_core/onboarding/newproject.py` · 842 linhas · testes: 34

nome	tipo	assinatura	o que faz
`plan`	função	`(name, template='plain', preset='burgers_1d') -> Dict[str, Any]`	Mostra o que `new_project` escreveria, sem escrever (ensaio).
`new_project`	função	`(name, *, dir=None, template='plain', preset='burgers_1d', verbose=True) -> str`	Escreve um projeto independente a partir de um template e preset e devolve a pasta.

pinneapple_core.onboarding.recommend

`pp.recommend`: transforma uma frase em preset, método e modelo, com o motivo e os termos que casaram; é pontuação por palavras-chave e pode errar. [sem teste direto]

`pinneapple_core/onboarding/recommend.py` · 299 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Recommendation`	classe	`(query, preset, method, model, confidence, reason, snippet, matched, alternatives, next_s…)`	Resultado do `recommend`: preset, método, modelo, confiança, motivo, snippet, termos casados, alternativas e próximos passos.
`recommend`	função	`(text, *, top=3) -> Recommendation`	Transforma a descrição de um problema em preset, método e modelo, com os motivos.

pinneapple_core.operators

Operadores físicos com a mesma chamada em campo contínuo (autograd), grade, malha simplicial e nuvem de pontos; em `Field` devolvem `Field`, em funções devolvem tensores com grafo. [sem teste direto]

`pinneapple_core/operators.py` · 275 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`grad`	função	`(f, x=None)`	Gradiente: `(N, d)` para escalar, `(N, c, d)` para `c` componentes.
`div`	função	`(f, x=None)`	Divergência de um campo vetorial.
`jacobian`	função	`(f, x=None)`	Jacobiana `(N, c, d)` de um campo.
`curl`	função	`(f, x=None)`	Rotacional: vetor em 3D, escalar (componente z) em 2D.
`hessian`	função	`(f, x=None)`	Hessiana `(N, d, d)` de um campo escalar.
`laplacian`	função	`(f, x=None)`	Soma das segundas derivadas, por componente em campos vetoriais.
`integrate`	função	`(f, **kwargs)`	Integral sobre o suporte (trapézio na grade, P1 exato na malha, quadratura ou Monte Carlo na nuvem).
`flux`	função	`(f, where=None, *, domain=None, n=20000, seed=0)`	Fluxo de um campo vetorial pela fronteira, `integral f . n dS`.

pinneapple_core.optim

`PhysicsOptimizer`: o problema (parâmetros com limites, objetivo, restrições de igualdade e desigualdade) é escrito uma vez e `minimize(method)` escolhe a família (SLSQP, adjunto, penalidade, Lagrangiano aumentado, evolução diferencial etc.).

`pinneapple_core/optim.py` · 395 linhas · testes: 8

nome	tipo	assinatura	o que faz
`Constraint`	classe	`(fn, kind, name)`	Restrição `fn(x) <= 0` (`ineq`) ou `fn(x) == 0` (`eq`).
`OptimizeResult`	classe	`(x, params, fun, violation, feasible, n_evals, method, family, success, message, wall_tim…)`	Resultado de `minimize`: ponto, objetivo, violação, viabilidade, avaliações, método, tempo e histórico.
`PhysicsOptimizer`	classe	`(parameters, objective, constraints=(), *, backend='torch', argument='vector', tol=0.001)…)`	Parâmetros com limites, objetivo e restrições, minimizados pelo método escolhido em `minimize`.

nome	tipo	assinatura	o que faz
`ineq`	função	`(fn, name='') -> Constraint`	Cria uma restrição de desigualdade `fn(x) <= 0`.
`eq`	função	`(fn, name='') -> Constraint`	Cria uma restrição de igualdade `fn(x) == 0`.
`list_methods`	função	`() -> List[str]`	Lista os métodos aceitos por `PhysicsOptimizer.minimize`.

pinneapple_core.service

Peças de runtime para servidores (apps, servidor MCP, serviços próprios) só com a biblioteca padrão; as rotas FastAPI ficam em `apps/_shared/appkit.py`.

`pinneapple_core/service/__init__.py` · 45 linhas · testes: 30

nome	tipo	assinatura	o que faz
`data_dir`	função	`(app, prefix=None, environ=None) -> Path`	Diretório de estado de uma app: `<PREFIX>_DATA_DIR`, senão `$PINNEAPPLE_APP_DATA/<app>`, senão `/data/<app>`.

pinneapple_core.service.apikeys

Chaves de API criadas uma vez (a chave em texto só aparece na criação), guardadas como SHA-256 e revogáveis; clientes enviam `X-API-Key`. [sem teste direto]

`pinneapple_core/service/apikeys.py` · 84 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`APIKeyStore`	classe	`(db_path, app='')`	Store SQLite de chaves de API: criar, listar, verificar e revogar.
`hash_key`	função	`(key) -> str`	SHA-256 de uma chave, usado para guardar e procurar.

pinneapple_core.service.db

Utilitários SQLite dos stores de serviço: uma conexão por thread, WAL e transações `BEGIN IMMEDIATE`. [sem teste direto]

`pinneapple_core/service/db.py` · 47 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`LocalDB`	classe	`(db_path, schema='')`	Uma conexão SQLite por thread para o mesmo arquivo, com esquema aplicado.
`connect`	função	`(db_path) -> sqlite3.Connection`	Abre conexão em autocommit com WAL e timeout de espera.
`transaction`	função	`(conn)`	Gerenciador de contexto de transação `BEGIN IMMEDIATE` com rollback em erro.

pinneapple_core.service.jobs

Fila de jobs persistente em SQLite com workers em thread ou processo, progresso, logs, cancelamento cooperativo, novas tentativas e retomada após reinício. [sem teste direto]

`pinneapple_core/service/jobs.py` · 510 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`JobCancelled`	classe	`()`	Levantada dentro de um job por `ctx.progress`/`ctx.check_cancel` após pedido de cancelamento.
`JobTimeout`	classe	`()`	Levantada dentro de um job em thread que passou do limite de tempo.
`JobContext`	classe	`(db_path, job_id, params, workdir, attempt, deadline=None)`	O que um job em execução vê: id, parâmetros, diretório de trabalho, progresso e log.

nome	tipo	assinatura	o que faz
`JobQueue`	classe	`(db_path, *, app='', max_concurrent=2, executor='thread', workdir=None, poll_interval=0.2…)`	Fila persistente em SQLite com registro de funções, `submit`, `wait`, cancelamento, novas tentativas e limite de concorrência entre workers.

pinneapple_core.service.metering

Medição de uso por app, principal e unidade, com limites gratuitos por mês (UTC) vindos do código e do ambiente; só mede, não cobra. [sem teste direto]

`pinneapple_core/service/metering.py` · 110 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`UsageLimitExceeded`	classe	`(unit, used, amount, limit, period)`	Erro de uso acima do limite gratuito do período para uma unidade.
`UsageMeter`	classe	`(db_path, app, limits=None)`	Registra e verifica uso por principal e unidade (`consume`, `usage`, `totals`).
`current_period`	função	`(ts=None) -> str`	Período corrente em UTC no formato ano-mês.
`limits_from_env`	função	`(prefix, defaults, environ=None) -> dict[str, float | None]`	Limites gratuitos por unidade, com padrões sobrescritos por `<PREFIX>_FREE_<UNIT>`.

pinneapple_core.service.uploads

Armazena uploads com limite de tamanho, sufixos permitidos, nomes seguros e SHA-256 em `<root>/<id>/`. [sem teste direto]

`pinneapple_core/service/uploads.py` · 106 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`UploadTooLarge`	classe	`()`	Erro de upload acima do tamanho máximo.
`UploadRejected`	classe	`()`	Erro de upload recusado (por exemplo sufixo não permitido).
`UploadStore`	classe	`(root, max_bytes=50 * 2 ** 20, allowed_suffixes=None)`	Guarda uploads em disco com metadados, e permite ler, listar, apagar e podar por idade.
`safe_name`	função	`(name) -> str`	Reduz um nome de arquivo ao nome base com caracteres seguros, nunca vazio nem oculto.

pinneapple_core.transforms

Transformações invertíveis e componíveis (`>>`) sobre tabelas, `PhysicsDataset` e, para as de escala, `PhysicalProblem`, registrando proveniência a cada `apply`; as não-escalares levantam `NotImplementedError` em nível de problema.

`pinneapple_core/transforms.py` · 636 linhas · testes: 13 · palavras-chave no código (heurística; o texto acima diz o status real): levanta `NotImplementedError` em algum caminho

nome	tipo	assinatura	o que faz
`NotInvertible`	classe	`()`	Erro de transformação (ou passo de composição) sem inversa.
`Transform`	classe	`()`	Base das transformações: `forward`/`inverse` em tabelas e `params()` para proveniência.
`Compose`	classe	`(steps) -> None`	Sequência de transformações; `inverse` roda na ordem reversa.
`Scale`	classe	`(scale, shift=None) -> None`	Escala afim de colunas nomeadas; num `PhysicalProblem` reescala coordenadas, domínio e parâmetros.
`Nondimensionalize`	classe	`(problem=None, *, length=None, time=None, field=None, scale=None, shift=None) -> None`	Leva um problema a variáveis adimensionais e permite trazer predições de volta às unidades originais.

nome	tipo	assinatura	o que faz
`Coordinate`	classe	`(kind, inputs, outputs, center=None) -> None`	Mudança de coordenadas em colunas: polar, cilíndrica, esférica ou log.
`Symmetry`	classe	`(kind, coords, vectors=(), *, axis=None, angle=0.0, center=None) -> None`	Operação de simetria (reflexão, rotação, com campos vetoriais) para aumento de dados ou teste de equivariância.
`Periodic`	classe	`(coord, period, harmonics=1, lo=0.0) -> None`	Troca uma coordenada periódica por `cos`/`sin` da fase, tornando a rede periódica por construção.
`FourierFeatures`	classe	`(coords, n_features=16, scale=1.0, seed=0, prefix='ff') -> None`	Features de Fourier aleatórias de algumas colunas.
`list_pde_rules`	função	`() -> List[str]`	Tipos de PDE cujos parâmetros `Scale`/`Nondimensionalize` sabem reescalar.

pinneapple_core.workflows

Orquestra ferramentas existentes (solvers, Docker, Slurm) como DAG de passos; reexporta `Workflow`, `Step`, `WorkflowRunner`, executores, adaptadores, templates e `dataset_from_run`.

`pinneapple_core/workflows/__init__.py` · 79 linhas · testes: 64

pinneapple_core.workflows.adapters

Adaptadores por `kind` de passo (`command`, `python`, `openfoam`, `calculix`): preparam o diretório, rodam comandos pelo executor e devolvem saídas em JSON.

`pinneapple_core/workflows/adapters.py` · 285 linhas · testes: 20

nome	tipo	assinatura	o que faz
`StepFailed`	classe	`()`	Erro de passo que rodou e falhou (saída não zero, erro do solver, saída ausente).
`StepContext`	classe	`(run_id, step, workdir, executor, upstream, log, should_stop, attempt)`	Contexto de um passo: id da execução, passo, diretório, executor, saídas a montante, log e tentativa.
`StepCancelled`	classe	`()`	Erro de passo cancelado.
`Adapter`	classe	`(kind)`	Base abstrata de um tipo de passo (`kind`) com verificação de disponibilidade e `run`.
`CommandAdapter`	classe	`()`	Passo `command`: roda `argv` com `{param}` preenchidos e lê `outputs.json` se existir.
`PythonAdapter`	classe	`()`	Passo `python`: chama `module:fn` no processo (só executor local) ou roda um script com o executor.
`OpenFOAMAdapter`	classe	`()`	Passo `openfoam`: copia um caso, aplica edições, roda blockMesh e o solver e lê a saúde do log.
`CalculiXAdapter`	classe	`()`	Passo `calculix`: copia um `.inp` e roda `ccx`.
`resolve_function`	função	`(ref) -> Callable`	Importa e devolve a função de uma referência `module:function`.
`register_adapter`	função	`(adapter) -> None`	Registra um adaptador para tornar executáveis passos do seu `kind`.
`get_adapter`	função	`(kind) -> Adapter`	Devolve o adaptador registrado para um `kind`.

pinneapple_core.workflows.dag

Definição do workflow como DAG (`Step`, `Workflow`): validação de dependências e ciclos, ordem, JSON e expansão por tabela de casos com preenchimento de `{param}`. [sem teste direto]

`pinneapple_core/workflows/dag.py` · 208 linhas · **sem teste direto** · palavras-chave no código (heurística; o texto acima diz o status real): placeholder/stub citado no docstring

nome	tipo	assinatura	o que faz
`WorkflowError`	classe	`()`	Erro de workflow inválido (passo desconhecido, id duplicado, ciclo).
`Step`	classe	`(id, kind, params, needs, executor, executor_options, retries, timeout, inputs, group)`	Um passo do DAG: id, `kind`, parâmetros, dependências, executor, tentativas, timeout e arquivos de entrada.
`Workflow`	classe	`(name, steps, fail_fast, max_parallel, metadata)`	Conjunto de passos com validação, ordem topológica, `fail_fast`, paralelismo máximo e salvar/carregar JSON.
`fill`	função	`(obj, values) -> Any`	Preenche `{name}` em strings, recursivamente; uma string que é só um placeholder recebe o valor tal qual.
`workflow_from_cases`	função	`(name, cases, template, *, id_column=None, final=(), **kw) -> Workflow`	Copia os passos de um template uma vez por caso de uma tabela, com ids `<caso>.<passo>`.

pinneapple_core.workflows.datasets

Leva uma tabela de casos a um workflow e as execuções terminadas a uma tabela de treino com `DatasetSpec` (marcado `partial` se faltam casos). [sem teste direto]

`pinneapple_core/workflows/datasets.py` · 115 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`read_cases`	função	`(path)`	Lê uma tabela de casos (CSV, Parquet, HDF5, JSON) e o contrato anexado, se houver.
`id_column_of`	função	`(df, contract=None) -> str | None`	Descobre a coluna de id dos casos pelo contrato ou por nomes comuns.
`dataset_from_run`	função	`(runner, run_id, cases, *, output_step, path, id_column=None, outputs=None, skeleton=None…)`	Grava a tabela de dataset de uma execução (parâmetros + saídas numéricas do passo) e devolve o `DatasetSpec`.

pinneapple_core.workflows.engine

`WorkflowRunner`: executa workflows com estado e log por passo em SQLite, novas tentativas, retomada e pulo de passos já feitos por hash de conteúdo. [sem teste direto]

`pinneapple_core/workflows/engine.py` · 367 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`WorkflowRunner`	classe	`(root, *, cache=True, input_root=None)`	Executa e retoma workflows, guarda estado, logs e cache por hash em SQLite sob uma raiz.
`sha256_file`	função	`(path) -> str`	SHA-256 do conteúdo de um arquivo.
`step_hash`	função	`(step, upstream_hashes, root=None) -> str`	Hash de conteúdo de um passo (tipo, parâmetros, opções, arquivos de entrada, hashes a montante).

pinneapple_core.workflows.executors

Executores plugáveis que rodam o comando de um passo: `local` (subprocesso), `docker` (sem rede por padrão) e `slurm` (script sbatch, só escrito por padrão com `dry_run=True`). [sem teste direto]

`pinneapple_core/workflows/executors.py` · 236 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ExecResult`	classe	`(returncode, seconds, command, dry_run, timed_out, cancelled, details)`	Resultado da execução de um comando: código de saída, tempo, comando, ensaio, timeout, cancelamento.
`Executor`	classe	`(**options)`	Base de executores: rodam um comando de um passo; subclasses definem `name` e `run`.
`LocalExecutor`	classe	`()`	Executor por subprocesso local, com timeout e parada sob pedido.
`DockerExecutor`	classe	`()`	Executor `docker run --rm` com o diretório do passo em /work, sem rede por padrão e com limites.

nome	tipo	assinatura	o que faz
`SlurmExecutor`	classe	`()`	Executor que escreve um script sbatch por passo (submete só fora do modo de ensaio).
`register_executor`	função	`(name, cls) -> None`	Registra uma classe de executor sob um nome usável em `executor=`.
`get_executor`	função	`(name, **options) -> Executor`	Instancia o executor registrado com as opções dadas.

`pinneapple_core.workflows.templates`

Templates de workflow seguros para servidor público (`slab_heat`, `beam_calculix`, `cavity_openfoam`), que validam cada caso contra o espaço de parâmetros.

`pinneapple_core/workflows/templates.py` · 213 linhas · testes: 20

nome	tipo	assinatura	o que faz
`Template`	classe	`(id, title, description, space, steps, output_step, final, outputs, solver)`	Template de workflow: espaço de parâmetros, passos por caso, passo de saída e solver necessário.
`slab_heat_solve`	função	`(kw, ctx) -> dict`	Perfil de temperatura da placa em `t_end` pela série exata; escreve `profile.csv`.
`collect_outputs`	função	`(kw, ctx) -> dict`	Resumo sobre todos os passos a montante (contagem, mínimo e máximo de cada saída numérica); escreve `summary.csv`.
`cantilever_deck`	função	`(kw, ctx) -> dict`	Gera o `job.inp` de uma viga em balanço (hexaedros C3D8I) e a deflexão de ponta de Euler-Bernoulli.
`cantilever_compare`	função	`(kw, ctx) -> dict`	Compara a deflexão máxima do CalculiX com a de Euler-Bernoulli e devolve a razão.

Exemplos de uso reais do repositório

PINN de Poisson sobre as primitivas — `examples/core_primitives/01_poisson_pinn.py`

```
from pinneapple_core import Field, FunctionField, Geometry

torch.manual_seed(0)
geom = Geometry.box([0, 0], [1, 1])
net = torch.nn.Sequential(
    torch.nn.Linear(2, 48), torch.nn.Tanh(),
    torch.nn.Linear(48, 48), torch.nn.Tanh(),
    torch.nn.Linear(48, 1),
)
u = FunctionField(net, geom.domain)

source = lambda x: 2 * np.pi**2 * torch.sin(np.pi * x[:, 0]) * torch.sin(np.pi * x[:, 1])
x_int = torch.tensor(geom.sample_interior(2000, seed=0), dtype=torch.float32)
x_bnd = torch.tensor(geom.sample_boundary(400, seed=1), dtype=torch.float32)
```

DataLoader com amostradores e pp.loss — `examples/data_api/01_pinn_operator_inverse.py`

```
geom = Geometry.box([0.0], [1.0])
net = mlp(1, 1)
loader = DataLoader({"col": CollocationSampler(geom, "lhs"), "bc": BoundarySampler(geom)}, batch_size={"col":
256, "bc": 2}, steps=1500)
opt = torch.optim.Adam(net.parameters(), lr=3e-3)
for batch in loader:
    x = batch["col"]["x"]                                          # already requires_grad
    r = -second(net(x), x) - PI**2 * torch.sin(PI * x)
    loss = L.combine({"pde": L.pde(r), "bc": L.boundary(net(batch["bc"]["x"]))}, {"bc": 10.0})
    opt.zero_grad(); loss.backward(); opt.step()
```

Workflow com template e dataset — `docs/guides/workflow_runner.md`

```python
from pinneapple_core.workflows import Step, Workflow, WorkflowRunner, TEMPLATES, dataset_from_run

runner = WorkflowRunner("work/runner")
t = TEMPLATES["slab_heat"]
cases = t.example_cases(20)                       # or read_cases("campaign.parquet")
run = runner.run(t.workflow(cases, max_parallel=4))
runner.run(run_id=run["id"])                      # resume (no-op when everything finished)
spec, path = dataset_from_run(runner, run["id"], cases, output_step="solve", path="slab.parquet")
```

Testes e status de validação

- Módulos com pelo menos um teste que os importa: 19 de 34.
- Funções de teste em arquivos que importam algum módulo do pacote: 233.

Limitações conhecidas

- `fem` usa matrizes densas: o docstring diz que serve para malhas de até alguns milhares de nós (casos de referência, protótipos de otimização de forma, testes).
- Domínios e geometrias não-caixa só são suportados em 2D (formas CSG de `pinneapple_design`); medir ou malhar a fronteira em outras dimensões levanta `NotImplementedError`.
- `PhysicsBackend` base e `Sampler` base levantam `NotImplementedError`; o backend jax exige jax instalado e liga float64 globalmente.
- Em `transforms`, só `Scale` e `Nondimensionalize` têm forma em nível de problema; as demais levantam `NotImplementedError` ao serem aplicadas a um `PhysicalProblem`.
- `recommend` e `search` do onboarding são pontuação por palavras-chave sobre um catálogo curado, não um modelo; podem errar e dizem quais termos casaram.
- Os adaptadores `openfoam` e `calculix` e os templates `beam_calculix`/`cavity_openfoam` precisam de OpenFOAM ou `ccx` instalados; o executor `slurm` só escreve o script por padrão (`dry_run=True`).
- `service` mede uso mas não cobra; o cancelamento de jobs em modo thread é cooperativo (só vale quando o job chama `ctx.progress`/`ctx.check_cancel`).
- Vários módulos não têm teste que os importe diretamente (`domain`, `field`, `mesh`, `geometry`, `operators`, `onboarding.recommend`, os stores de `service` e partes de `workflows`); são exercitados pelos testes que importam o pacote ou o subpacote.

3. `pinneapple_catalog`

Objetivo

`pinneapple_catalog` é o catálogo do que o PINNeAPPle conhece, sempre com a fonte. Ele tem duas partes. `resources` lista recursos públicos de Physics AI (datasets, conjuntos de CAD/geometria, modelos pré-treinados e benchmarks hospedados), cada um com licença conferida, classificação de uso e, quando houver, o repositório no Hugging Face. `methods` lista cada solver, método de treino, equação física e problema (preset) da biblioteca, com o caminho do código, as referências bibliográficas e o status de validação.

O problema que o pacote resolve é de rastreabilidade e de licença. Um componente que quer um recurso público pergunta primeiro ao catálogo: `require_allowed` aplica a mesma trava de licença de `pinneapple_neural._licencas` e recusa recursos sem uso comercial (ou com licença não verificada) quando `PINNEAPPLE_COMMERCIAL_MODE=1`. `fetch` baixa o recurso do Hugging Face Hub só depois dessa trava.

No lado dos métodos, o status de validação não é escrito à mão. O script `scripts/build_method_status.py` varre `tests/` e grava `pinneapple_catalog/method_status.json`. Um item é `validated` quando algum teste o compara com uma referência independente (forma fechada, valor publicado ou solução manufaturada), `tested` quando há testes sem essa comparação, e `untested` caso contrário. `validated_methods()` devolve só os validados; a página pública `docs/validation_status.md` é conferida contra essa varredura por teste. Os caminhos de código apontam para os outros pacotes (`pinneapple_simulation`, `pinneapple_neural`, `pinneapple_physics`, `pinneapple_systems`) e os problemas apontam para os presets de `pinneapple_physics/pde_environment/presets/`.

Números do inventário

módulos	linhas	classes públicas	funções públicas	testes em arquivos que importam o pacote	módulos sem teste direto
3	1.024	2	8	16 (3 arquivos)	1

Submódulos

submódulo	o que faz	módulos	classes + funções	testes*
`methods`	Inventário de solvers, métodos de treino, equações e problemas, com código, referências e status de validação lido de `method_status.json`.	1	5	8
`resources`	Catálogo de recursos públicos com licença conferida, filtros, trava de licença e download pelo Hugging Face Hub.	1	5	0

* maior número de testes que importam um módulo do submódulo.

Módulos e API pública

`pinneapple_catalog`

Raiz do pacote: reexporta a API de `resources` (`Resource`, `list_resources`, `get_resource`, `require_allowed`, `fetch`); `methods` é importado à parte.

`pinneapple_catalog/__init__.py` · 11 linhas · testes: 8

`pinneapple_catalog.methods`

Lista fixa de `Method` (IDs S = solver, T = treino, E = equação, P = problema) com caminho do código, referências citadas no código e referências acrescentadas, e funções para consultar o status de validação.

`pinneapple_catalog/methods.py` · 569 linhas · testes: 8

nome	tipo	assinatura	o que faz
`Method`	classe	`(id, name, code, probes, refs_in_code, refs_added, equations)`	Registro imutável de um item do catálogo: id, nome, caminhos de código, termos de busca nos testes (`probes`), referências e, para problemas, as equações usadas; expõe `category` e `references`.
`get_method`	função	`(method_id) -> Method`	Devolve o `Method` de um id ou levanta `KeyError` se o id não existe.
`list_methods`	função	`(category=None) -> List[Method]`	Lista todos os métodos, ou só os de uma categoria (`solver`, `training`, `equation`, `problem`).
`method_status`	função	`() -> Dict[str, Dict]`	Lê `method_status.json` e devolve, por id, o status e os testes de referência e outros testes; devolve dict vazio se o arquivo não existe.
`validated_methods`	função	`(category=None) -> List[Method]`	Lista oficial: só os métodos cujo status na última varredura é `validated`.

`pinneapple_catalog.resources`

Lista fixa de `Resource` com licença, uso comercial (`yes`/`no`/`unverified`), classificação (`use_now`, `product`, `reference`, `skip`), filtros, trava de licença e download. [sem teste direto pelo inventário; é exercitado via `tests/test_catalog_resources.py`, que importa pela raiz]

`pinneapple_catalog/resources.py` · 444 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Resource`	classe	`(id, name, kind, domain, summary, license, commercial, url, rating, paper, size, hf_repo,…)`	Registro imutável de um recurso público (tipo, domínio, licença, uso comercial, URL, classificação, artigo, tamanho, repositório HF, notas, tags); `research_only` é verdadeiro sempre que o uso comercial não é `yes`.

nome	tipo	assinatura	o que faz
`list_resources`	função	`(kind=None, domain=None, rating=None, commercial_only=False) -> List[Resource]`	Filtra os recursos por tipo, domínio e classificação; `commercial_only` mantém só os de uso comercial `yes`.
`get_resource`	função	`(resource_id) -> Resource`	Devolve o `Resource` de um id ou levanta `KeyError` listando os ids conhecidos.
`require_allowed`	função	`(resource_id) -> Resource`	Trava de licença: avisa para recursos só de pesquisa e os recusa em modo comercial, via `require_research_use` de `pinneapple_neural._licencas`.
`fetch`	função	`(resource_id, *, allow_patterns=None, local_dir=None, token=None, revision=None) -> str`	Passa pela trava de licença e baixa o recurso com `huggingface_hub.snapshot_download`, devolvendo o caminho local; aceita `allow_patterns` para baixar só parte.

Exemplos de uso reais do repositório

Recursos sem licença comercial contam como só pesquisa — `tests/test_catalog_resources.py`

```python
from pinneapple_catalog import fetch, get_resource, list_resources, require_allowed
from pinneapple_neural._licencas import ResearchOnlyError, ResearchOnlyWarning


def test_unknown_or_missing_license_counts_as_research_only():
    assert get_resource("pressnet").research_only  # no LICENSE file
    assert get_resource("drivaernet").research_only  # CC BY-NC
    assert not get_resource("airfrans").research_only  # ODbL allows commercial use


def test_commercial_only_filter_excludes_research_only():
    assert all(not r.research_only for r in list_resources(commercial_only=True))
```

Status de validação cobre todo o catálogo de métodos — `tests/test_catalog_methods.py`

```python
def test_status_covers_every_item_and_validated_is_a_subset():
    status = method_status()
    assert {m.id for m in list_methods()} <= set(status)
    for m in validated_methods():
        assert status[m.id]["status"] == "validated"
        assert status[m.id]["reference_tests"], m.id
```

Testes e status de validação

- Módulos com pelo menos um teste que os importa: 2 de 3.
- Funções de teste em arquivos que importam algum módulo do pacote: 16.

Limitações conhecidas

- As listas de recursos e de métodos são fixas no código; incluir um item novo exige editar `resources.py` ou `methods.py`.
- O status de validação depende de rodar `scripts/build_method_status.py`; se `method_status.json` estiver desatualizado, `validated_methods()` também estará. Sem o arquivo, `method_status()` devolve vazio.
- A detecção de testes é por presença de strings (`probes`) nos arquivos de teste, não por execução.
- `fetch` exige o extra `hub` (`huggingface_hub`) e só funciona para recursos com `hf_repo`; os demais levantam `ValueError` apontando a URL de origem.
- Licença não verificada (`unverified`) é tratada como não comercial; em `PINNEAPPLE_COMMERCIAL_MODE=1` esses recursos são recusados.
- Alguns caminhos de código e docstrings citam o repositório irmão `PINNeAPPle-CFD`, que pode não estar presente; o teste pula esses caminhos nesse caso.

- `pinneapple_catalog.methods` não é reexportado pela raiz do pacote; é preciso importar `pinneapple_catalog.methods` diretamente.

4. `pinneapple_registry`

Objetivo

`pinneapple_registry` é um registro local de artefatos, que roda sem rede: guarda modelos versionados (com estágios e relatório de confiança), datasets de simulação, execuções de experimentos num SQLite e o histórico de definições de problemas. Também gera repositórios de modelo para o Triton Inference Server a partir de modelos exportados em ONNX.

Ele complementa o `pinneapple_hub`. O hub publica um modelo pronto no Hugging Face Hub, o que exige rede e autenticação. O registro versiona cada checkpoint intermediário no disco local, junto com datasets e execuções, e um modelo feito aqui pode depois ser enviado ao hub com `pinneapple_hub.push_to_hub`. `ArtifactRegistry` junta os quatro stores (`ModelStore`, `DatasetStore`, `ExperimentStore`, `ProblemStore`) sob uma mesma raiz.

A outra metade do pacote é o gateway de modelos. `ModelGateway` publica modelos físicos como endpoints versionados: surrogates do Surrogate Studio/arena (`pinneapple_analysis.surrogate`), modelos do zoo (`pp.zoo`), grafos ONNX e modelos remotos. Todos rodam por um único `predict` com política para entradas fora da distribuição (avisar ou recusar), e cada chamada é registrada. `gateway_api` dá a forma REST (FastAPI, `/v1/models/...`) usada pelos apps que servem modelos, e `GatewayClient` chama essa API de Python só com a biblioteca padrão. A regra de confiança vem do `ModelStore`: um relatório de confiança com decisão REJECT bloqueia a publicação em `staging`/`production`, a não ser que haja uma justificativa registrada.

Números do inventário

módulos	linhas	classes públicas	funções públicas	testes em arquivos que importam o pacote	módulos sem teste direto
10	1.699	10	3	92 (10 arquivos)	3

Submódulos

submódulo	o que faz	módulos	classes + funções	testes*
`dataset_store`	Guarda arrays de uma simulação (coordenadas e campos) por problema e cenário, com estatísticas automáticas.	1	1	0
`experiment_memory`	Memória persistente de tentativas anteriores para um loop de pesquisa autônomo, com busca léxica de casos semelhantes.	1	2	19
`experiment_store`	Rastreador de execuções em SQLite: início, métricas por passo, fim e melhor execução.	1	1	10
`gateway`	`ModelGateway`: publica modelos como endpoints versionados, roda com política OOD e registra cada chamada.	1	2	10
`gateway_api`	Roteador FastAPI com a forma REST `/v1/models/...` do gateway.	1	2	10
`gateway_client`	Cliente Python (só `urllib`) para qualquer servidor com a API do gateway.	1	2	10
`model_store`	Store local e versionado de modelos treinados, com estágios, relatório de confiança e exportação ONNX.	1	1	13
`problem_store`	Definições de problemas versionadas, por preset (reconstruível) ou snapshot (só histórico).	1	1	0
`registry`	`ArtifactRegistry`: une os quatro stores sob uma raiz e gera repositório do Triton.	1	1	0

* maior número de testes que importam um módulo do submódulo.

Módulos e API pública

pinneapple_registry

Reexporta `ArtifactRegistry`, `ModelStore`, `ModelGateway`, `DatasetStore`, `ExperimentStore`, `ExperimentMemory`, `ExperimentRecord`, `ProblemStore` e `VALID_STAGES`.

`pinneapple_registry/__init__.py` · 45 linhas · testes: 50

pinneapple_registry.dataset_store

`DatasetStore` grava coordenadas e campos de uma execução por problema/cenário em disco com estatísticas resumo; é o complemento leve do pipeline Zarr de `pinneapple_data`. [sem teste direto pelo inventário]

`pinneapple_registry/dataset_store.py` · 99 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`DatasetStore`	classe	`(root)`	Store de datasets de simulação por problema e cenário, com `save`, `load`, `list_scenarios` e `stats`.

pinneapple_registry.experiment_memory

`ExperimentMemory` registra tentativas (`ExperimentRecord`) no `ExperimentStore` e busca tentativas parecidas por similaridade TF-IDF do texto (léxica, não semântica); serve a um loop de pesquisa autônomo que não está neste repositório.

`pinneapple_registry/experiment_memory.py` · 252 linhas · testes: 19

nome	tipo	assinatura	o que faz
`ExperimentRecord`	classe	`(problem_description, physics_domain, approach_summary, outcome, confidence_score, metric…)`	Uma tentativa registrada de resolver um problema: descrição, domínio, abordagem, resultado, confiança, métricas e lições.
`ExperimentMemory`	classe	`(store=None, *, db_path=None)`	Memória persistente de tentativas sobre o `ExperimentStore`, com busca de casos semelhantes e lições por domínio.

pinneapple_registry.experiment_store

`ExperimentStore`: rastreador de execuções em SQLite com experimentos, execuções, métricas por passo e métricas finais.

`pinneapple_registry/experiment_store.py` · 144 linhas · testes: 10

nome	tipo	assinatura	o que faz
`ExperimentStore`	classe	`(db_path)`	Rastreador de execuções em SQLite: cria experimentos e execuções, grava métricas e acha a melhor execução.

pinneapple_registry.gateway

`ModelGateway` serve surrogates, modelos do zoo, ONNX e modelos remotos por um `predict` com política OOD (`warn`/`refuse`), cache, log em `_runs.jsonl`, métricas de latência e exportação; tipos sem exportação levantam `NotImplementedError`.

`pinneapple_registry/gateway.py` · 457 linhas · testes: 10 · palavras-chave no código (heurística; o texto acima diz o status real): levanta `NotImplementedError` em algum caminho

nome	tipo	assinatura	o que faz
`ModelGateway`	classe	`(root, *, cache_size=8)`	Publica, resolve, roda, registra e exporta modelos versionados com política OOD.
`ood_verdict`	função	`(flag, policy='warn') -> str`	Decide o que `predict` faz com uma linha avaliada: "in range", "warn" ou "refuse".

pinneapple_registry.gateway_api

Monta um `APIRouter` FastAPI com listagem, descrição, predição, log de execuções, métricas e exportação, mapeando erros para 404/422/403/409.

`pinneapple_registry/gateway_api.py` · 144 linhas · testes: 10

nome	tipo	assinatura	o que faz
`count_rows`	função	`(inputs) -> int`	Conta as linhas que uma requisição pede, antes de carregar o modelo (para medição).
`gateway_router`	função	`(gateway, *, prefix='/v1', on_rows=None, caller=None, max_rows=10000, tags=('models',))`	Cria o roteador FastAPI do gateway com prefixo, hook de medição, identificação do chamador e limite de linhas.

pinneapple_registry.gateway_client

`GatewayClient` chama `/v1/models/...` com `urllib` e levanta `GatewayError` com status HTTP e mensagem.

`pinneapple_registry/gateway_client.py` · 83 linhas · testes: 10

nome	tipo	assinatura	o que faz
`GatewayError`	classe	`(status, message)`	Erro de uma chamada ao gateway, com status HTTP e mensagem do servidor.
`GatewayClient`	classe	`(base_url, *, api_key=None, timeout=30.0, headers=None)`	Cliente HTTP da API do gateway: `models`, `describe`, `predict`, `runs`, `metrics`.

pinneapple_registry.model_store

`ModelStore` grava modelos em `<root>/<problema>/v<data_hora>/` com `metadata.json`, ponteiro `latest`, estágios, relatório de confiança (REJECT bloqueia publicação) e exportação ONNX.

`pinneapple_registry/model_store.py` · 246 linhas · testes: 13

nome	tipo	assinatura	o que faz
`ModelStore`	classe	`(root)`	Store versionado de modelos com estágios, comparação por métrica, relatório de confiança e exportação ONNX.

pinneapple_registry.problem_store

`ProblemStore` versiona problemas: por preset (reconstruído com `get_preset`) ou snapshot em dict, que não é garantido voltar a ser `ProblemSpec`. [sem teste direto pelo inventário]

`pinneapple_registry/problem_store.py` · 98 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ProblemStore`	classe	`(root)`	Store versionado de definições de problemas, por preset ou snapshot, com histórico.

pinneapple_registry.registry

`ArtifactRegistry` cria os quatro stores sob uma raiz, resume o conteúdo e exporta um modelo como diretório do Triton (`config.pbtxt` + `1/model.onnx`). [sem teste direto pelo inventário; testado via o pacote]

`pinneapple_registry/registry.py` · 131 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ArtifactRegistry`	classe	`(root)`	Raiz única com os stores de modelos, datasets, experimentos e problemas, mais exportação para Triton e resumo.

Exemplos de uso reais do repositório

Publicar e servir um modelo no gateway — `docs/guides/surrogates_arena_gateway.md`

```python
from pinneapple_registry import ModelGateway

gw = ModelGateway("./models")
v = gw.publish("airfoil-cl-cd", "airfoil_model", ood_policy="refuse")      # or a SurrogateModel
gw.publish("airfoil-zoo", "airfoil_cst_rans_mlp", kind="pinneapple.zoo")    # zoo models too
r = gw.predict("airfoil-cl-cd", [{"T0": 0.15, "T1": 0.12, "T2": 0.13, "M0": 0.02, "M1": 0.05, "M2": 0.04,
                                  "alpha_deg": 4.0}])
print(r["outputs"], r["ood"], r["latency_ms"], gw.metrics()["by_version"])
gw.export("airfoil-cl-cd", None, "onnx", "airfoil.onnx")                    # PyTorch surrogates only
```

Versionar e promover um modelo — `tests/test_artifact_registry.py`

```python
reg = _registry()
model = ComponentRegistry.build("HeatExchangerVanillaPINN", in_dim=2, out_dim=1, hidden=(8, 8))
v1 = reg.models.save("heat_exchanger_demo", model, metadata={"metrics": {"rmse": 0.5}})
v2 = reg.models.save("heat_exchanger_demo", model, metadata={"metrics": {"rmse": 0.1}})
assert reg.models.latest("heat_exchanger_demo") == v2
reg.models.promote("heat_exchanger_demo", v2, "production")
assert reg.models.metadata("heat_exchanger_demo", v2)["stage"] == "production"
```

Testes e status de validação

- Módulos com pelo menos um teste que os importa: 7 de 10.
- Funções de teste em arquivos que importam algum módulo do pacote: 92.

Limitações conhecidas

- `ProblemStore.save_snapshot` não garante reconstruir um `ProblemSpec`: `load_snapshot` devolve só o dict.
- `ExperimentMemory.query_similar` é similaridade léxica (TF-IDF, via scikit-learn), não semântica; o loop de pesquisa autônomo que a usa não está no repositório.
- `ModelGateway.export` só exporta surrogates PyTorch e ONNX; outros tipos levantam `NotImplementedError`. ONNX exige `onnxruntime`; `gateway_api` exige FastAPI.
- Um surrogate é desserializado (pickle) depois de checar o sha256 do cartão: o docstring avisa para nunca publicar um diretório que você não produziu, e publicar um modelo remoto faz o servidor chamar a URL dada (só chamadores confiáveis devem publicar).
- A exportação para Triton só gera o diretório (`config.pbtxt` com tipo FP32 + ONNX); não roda nem testa um servidor Triton.
- Sem teste direto pelo inventário: `dataset_store`, `problem_store`, `registry` (cobertos indiretamente por `tests/test_artifact_registry.py` via o pacote).

5. `pinneapple_orchestration`

Objetivo

`pinneapple_orchestration` é uma camada opcional de pipelines no estilo Prefect. Ela não traz lógica nova: só encadeia o que já existe em outros pacotes. Os dados vão para o `DatasetStore` e os modelos para o registro de artefatos de `pinneapple_registry`. Os componentes treináveis vêm de `pinneapple_systems.component_library` e o gêmeo digital de `pinneapple_systems.digital_twin`.

O pacote é pequeno de propósito: são cinco flows. `flow_data_generation` roda N vezes uma função de amostragem dada pelo usuário e grava cada cenário. `flow_training` monta um componente registrado, treina e registra a execução num `ExperimentStore`. `flow_benchmark` ranqueia todas as arquiteturas de um

`component_type`. `flow_registry_publish` promove um modelo de estágio (`development` para `staging`/`production`), sempre com confirmação explícita. `flow_digital_twin_prep` monta um `DigitalTwin` a partir de um modelo do registro, só se ele estiver no estágio exigido.

Cada flow chama uma task fina em `tasks/`, e cada task delega ao pacote dono da lógica (por exemplo `ComponentModel.fit`, que usa o `Trainer` comum). Todo flow roda dentro de `flow_run_manifest`, que grava um JSON por execução com parâmetros, SHA do git, log das tasks e artefatos.

O Prefect não é obrigatório. Com o extra `pinneapple[orchestration]` instalado, os decoradores `flow`/`task` são os do Prefect (retries, agendamento, UI). Sem ele, `_prefect_compat` troca os decoradores por funções que não fazem nada, e cada flow vira uma função Python comum, chamável e testável.

Números do inventário

módulos	linhas	classes públicas	funções públicas	testes em arquivos que importam o pacote	módulos sem teste direto
16	591	1	11	6 (1 arquivos)	14

Submódulos

submódulo	o que faz	módulos	classes + funções	testes*
`_prefect_compat`	Shim que usa os decoradores `flow`/`task` do Prefect quando instalado e, senão, decoradores que devolvem a função sem mudança.	1	0	0
`common`	Utilitários comuns aos flows; hoje só o manifesto de proveniência de cada execução.	2	2	0
`flows`	Os cinco flows públicos (geração de dados, treino, benchmark, publicação no registro, preparo de gêmeo digital).	6	5	6
`tasks`	As tasks finas por trás de cada flow, que delegam a `pinneapple_registry` e `pinneapple_systems`.	6	5	0

* maior número de testes que importam um módulo do submódulo.

Módulos e API pública

`pinneapple_orchestration`

Ponto de entrada do pacote: reexporta `PREFECT_AVAILABLE`, os subpacotes `tasks` e `flows`, `flow_run_manifest` e `FlowRunManifest`.

`pinneapple_orchestration/__init__.py` · 44 linhas · testes: 6

`pinneapple_orchestration._prefect_compat`

[shim] Importa `flow` e `task` do Prefect se existir; senão define decoradores de passagem que aceitam `@flow` e `@flow(...)` e ignoram os kwargs. Define `PREFECT_AVAILABLE`. [sem teste direto]

`pinneapple_orchestration/_prefect_compat.py` · 36 linhas · **sem teste direto** · palavras-chave no código (heurística; o texto acima diz o status real): shim ou obsoleto

`pinneapple_orchestration.common`

Subpacote que reexporta `flow_run_manifest` e `FlowRunManifest`. [sem teste direto]

`pinneapple_orchestration/common/__init__.py` · 6 linhas · **sem teste direto**

`pinneapple_orchestration.common.provenance`

Gerenciador de contexto que grava `<manifest_dir>/<flow_name>/<run_id>.json` ao fim de cada flow (sucesso ou falha), com parâmetros, SHA do git, log de tasks, artefatos e duração. [sem teste direto]

pinneapple_orchestration/common/provenance.py · 95 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
FlowRunManifest	classe	(flow_name, run_id, parameters, git_sha, started_at, finished_at, status, task_log, artif…)	Registro de uma execução de flow: nome, id, parâmetros, SHA do git, horários, status, log de tasks, artefatos e erro.
flow_run_manifest	função	(flow_name, parameters=None, *, manifest_dir='data/manifests', repo_dir=None) -> Iterator…)	Gerenciador de contexto que cria um FlowRunManifest, marca sucesso ou falha e grava o JSON no fim.

pinneapple_orchestration.flows

Subpacote que reexporta os cinco flows.

pinneapple_orchestration/flows/__init__.py · 16 linhas · testes: 6

pinneapple_orchestration.flows.flow_01_data_generation

Flow que roda sample_fn N vezes e grava cada resultado como cenário num DatasetStore. [sem teste direto; coberto por tests/test_orchestration.py via flows]

pinneapple_orchestration/flows/flow_01_data_generation.py · 31 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
flow_data_generation	função	(dataset_store, problem_id, sample_fn, n_scenarios, *, manifest_dir='data/manifests') -> …)	Gera n_scenarios cenários com sample_fn, grava no DatasetStore e devolve os ids e o run_id.

pinneapple_orchestration.flows.flow_02_training

Flow que monta e treina um componente registrado e registra a execução num ExperimentStore opcional.

pinneapple_orchestration/flows/flow_02_training.py · 36 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
flow_training	função	(component_name, train_loader, val_loader=None, *, experiment_store=None, epochs=100, lr=…)	Treina o componente component_name com os loaders dados e devolve o modelo, o resultado do fit e o run_id.

pinneapple_orchestration.flows.flow_03_benchmark

Flow que ranqueia todas as arquiteturas registradas para um component_type.

pinneapple_orchestration/flows/flow_03_benchmark.py · 29 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
flow_benchmark	função	(component_type, train_loader, val_loader=None, *, epochs=50, lr=0.001, manifest_dir='dat…)	Ranqueia as arquiteturas de um component_type e guarda o ranking no manifesto.

pinneapple_orchestration.flows.flow_04_registry_publish

Flow terminal que salva um modelo no registro e o promove de estágio só com confirm=True.

pinneapple_orchestration/flows/flow_04_registry_publish.py · 32 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
flow_registry_publish	função	(registry, problem_id, model, *, metadata=None, stage='staging', confirm=False, manifest_…)	Publica um modelo no registro e devolve a versão; levanta PermissionError se o estágio não é development e confirm não é True.

pinneapple_orchestration.flows.flow_05_digital_twin_prep

Flow que monta um `DigitalTwin` a partir de um modelo do registro, exigindo por padrão o estágio `production`.

`pinneapple_orchestration/flows/flow_05_digital_twin_prep.py` · 36 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`flow_digital_twin_prep`	função	`(registry, problem_id, field_names, coord_names=None, *, version=None, require_stage='pro…)`	Devolve um `DigitalTwin` montado com o modelo do registro, checando o estágio exigido.

pinneapple_orchestration.tasks

Subpacote que reexporta as cinco tasks. [sem teste direto]

`pinneapple_orchestration/tasks/__init__.py` · 16 linhas · **sem teste direto**

pinneapple_orchestration.tasks.benchmark_tasks

Task que só chama `pinneapple_systems.component_library.benchmark_component_type`; não há lógica de benchmark aqui. [sem teste direto]

`pinneapple_orchestration/tasks/benchmark_tasks.py` · 25 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`benchmark_component_type_task`	função	`(component_type, train_loader, val_loader=None, *, epochs=50, lr=0.001) -> ComponentBench…)`	Repassa a chamada para `benchmark_component_type` e devolve um `ComponentBenchmarkResult`.

pinneapple_orchestration.tasks.data_generation_tasks

Task genérica de geração de dados: roda um thunk `sample_fn() -> (coords, fields)` dado pelo usuário e salva cada execução no `DatasetStore`. [sem teste direto]

`pinneapple_orchestration/tasks/data_generation_tasks.py` · 43 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`generate_scenarios_task`	função	`(dataset_store, problem_id, sample_fn, n_scenarios, *, scenario_prefix='scenario') -> Lis…)`	Roda `sample_fn()` `n_scenarios` vezes, salva cada resultado como cenário `<prefixo>_NNNN` e devolve os ids.

pinneapple_orchestration.tasks.digital_twin_tasks

Task que carrega um modelo do registro e monta um `DigitalTwin`, recusando com `PermissionError` modelos fora do estágio exigido. [sem teste direto]

`pinneapple_orchestration/tasks/digital_twin_tasks.py` · 48 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`prepare_digital_twin_task`	função	`(registry, problem_id, field_names, coord_names=None, *, version=None, require_stage='pro…)`	Escolhe a versão (a mais recente por padrão), confere o estágio, carrega o modelo e devolve um `DigitalTwin`.

pinneapple_orchestration.tasks.registry_tasks

Task que salva o modelo em `development` e só promove para outro estágio com `confirm=True`; um `trust_report` REJECT bloqueia a promoção salvo `override_reason`. [sem teste direto]

`pinneapple_orchestration/tasks/registry_tasks.py` · 47 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
publish_model_task	função	(registry, problem_id, model, *, metadata=None, stage='staging', confirm=False, trust_rep…)	Salva o modelo em development e promove para stage só com confirm=True; devolve a versão.

pinneapple_orchestration.tasks.training_tasks

Task fina que monta um componente via ComponentRegistry, chama ComponentModel.fit() e registra métricas num ExperimentStore. [sem teste direto]

pinneapple_orchestration/tasks/training_tasks.py · 51 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
train_component_task	função	(component_name, train_loader, val_loader=None, *, experiment_store=None, epochs=100, lr=…)	Monta o componente, treina com fit, registra best_val no ExperimentStore (se dado) e devolve modelo, resultado e run_id.

Exemplos de uso reais do repositório

Gerar cenários com uma função de amostragem — tests/test_orchestration.py

```
        def test_flow_data_generation(tmp_path):
            reg = _registry()

            def sample_fn():
                return np.random.rand(20, 2).astype(np.float32), {"u": np.random.rand(20, 1).astype(np.float32)}

            result = flow_data_generation(reg.datasets, "pipe_flow", sample_fn, 3, manifest_dir=str(tmp_path /
"manifests"))
            assert len(result["scenarios"]) == 3
            assert reg.datasets.list_scenarios("pipe_flow") == sorted(result["scenarios"])
```

Promoção de estágio exige confirm=True — tests/test_orchestration.py

```
            try:
                flow_registry_publish(reg, "valve_demo", model, stage="production", confirm=False,
                                      manifest_dir=str(tmp_path / "manifests"))
                assert False, "expected PermissionError"
            except PermissionError:
                pass

            result = flow_registry_publish(reg, "valve_demo", model, stage="production", confirm=True,
                                           manifest_dir=str(tmp_path / "manifests"))
            assert reg.models.metadata("valve_demo", result["version"])["stage"] == "production"
```

Testes e status de validação

- Módulos com pelo menos um teste que os importa: 2 de 16.
- Funções de teste em arquivos que importam algum módulo do pacote: 6.

Limitações conhecidas

- Prefect é opcional (extra pinneapple[orchestration]); sem ele os flows rodam como funções comuns, sem retries, agendamento nem UI.
- Sem Prefect, os kwargs dos decoradores (por exemplo retries) são ignorados em silêncio.
- flow_run_manifest grava por padrão em data/manifests relativo ao diretório atual e chama git rev-parse HEAD; fora de um repositório git o SHA fica None.
- A geração de dados não configura solvers sozinha: o usuário precisa passar um thunk sample_fn sem argumentos.
- flow_registry_publish não repassa trust_report nem override_reason para publish_model_task; para usar esse portão é preciso chamar a task direto.

- Os testes (`tests/test_orchestration.py`) importam só `flows`; tasks, `common` e `_prefect_compat` não têm teste direto.
- Depende de `pinneapple_registry` e `pinneapple_systems` (importados no topo dos módulos de flows e tasks).

6. `pinneapple_models`

Objetivo

`pinneapple_models` é um shim de compatibilidade. Ele não implementa nenhum modelo: só reexporta, de `pinneapple_neural.architectures`, o registro (`ModelRegistry`), o catálogo (`ModelCatalog`), a classe base `BaseModel` e algumas arquiteturas (`VanillaPINN`, `SIREN`, `ModifiedMLP`, `FourierNeuralOperator`, `FNO2d`, `DeepONet`, `MeshGraphNet`, `GraphBatch`, `GraphOutput`).

O problema que resolve é manter código antigo funcionando. Exemplos e scripts escritos com `from pinneapple_models import ...` continuam rodando depois que as arquiteturas foram para `pinneapple_neural`. O próprio docstring diz que código novo deve importar direto de `pinneapple_neural.architectures`.

Além dos nomes planos, o pacote cria pseudo-namespaces antigos (`pinns`, `neural_operators`, `graphnn`), um alias `catalog = ModelCatalog`, uma função `registry()` e tenta importar `register_all` do catálogo de `pinneapple_neural` (se falhar, vira uma função vazia). O submódulo `pinneapple_models.registry` existe só para que o caminho `pinneapple_models.registry.ModelRegistry` funcione, como em `pinneapple_train` e `pinneapple_solvers`.

Números do inventário

módulos	linhas	classes públicas	funções públicas	testes em arquivos que importam o pacote	módulos sem teste direto
2	74	0	1	1 (1 arquivos)	1

Pasta de testes dedicada: `tests/pinneapple_models` (1 funções de teste).

Submódulos

submódulo	o que faz	módulos	classes + funções	testes*
`registry`	Submódulo de compatibilidade que reexporta `ModelRegistry` de `pinneapple_neural.architectures.registry`.	1	0	1

* maior número de testes que importam um módulo do submódulo.

Módulos e API pública

`pinneapple_models`

[shim] Reexporta registro, catálogo e arquiteturas de `pinneapple_neural.architectures` para código legado; marcado como obsoleto (use `pinneapple_neural.architectures`). [sem teste direto]

`pinneapple_models/__init__.py` · 64 linhas · **sem teste direto** · palavras-chave no código (heurística; o texto acima diz o status real): shim ou obsoleto

nome	tipo	assinatura	o que faz
`registry`	função	`()`	Função legada que devolve a classe `ModelRegistry` de `pinneapple_neural` (não cria registro novo).

`pinneapple_models.registry`

[shim] Expõe `ModelRegistry` no caminho antigo `pinneapple_models.registry`, criado porque `tests/pinneapple_models/test_registry.py` não coletava sem ele.

pinneapple_models/registry.py · 10 linhas · testes: 1

Exemplos de uso reais do repositório

Teste de fumaça do registro pelo caminho antigo — tests/pinneapple_models/test_registry.py

```
def test_models_registry_smoke():
    from pinneapple_models.registry import ModelRegistry

    try:
        from pinneapple_models.register_all import register_all
        register_all()
    except Exception:
        pass

    names = ModelRegistry.list()
    assert isinstance(names, list)
```

Tour do registro via imports legados — examples/architectures/00_registry_tour.py

```
from pinneapple_models import ModelCatalog
from pinneapple_models import register_all
from pinneapple_models import ModelRegistry


def _short(s: str, n: int = 90) -> str:
    s = (s or "").strip().replace("\n", " ")
    return s if len(s) <= n else s[: n - 1] + "…"


def main() -> None:
    register_all()

    print("\n=== Families ===")
    print(ModelRegistry.families())
```

Testes e status de validação

- Módulos com pelo menos um teste que os importa: 1 de 2.
- Funções de teste em arquivos que importam algum módulo do pacote: 1.

Limitações conhecidas

- É só um shim: todo o comportamento vem de `pinneapple_neural.architectures`; qualquer mudança lá afeta este pacote.
- Marcado como obsoleto no docstring; código novo deve importar de `pinneapple_neural.architectures`.
- Se `pinneapple_neural.architectures._catalog.register_all` não puder ser importado, `register_all` vira uma função que não faz nada, em silêncio.
- `XTFC` no namespace `pinns` fica `None` se o import de `pinneapple_neural.architectures.pinns.xtfc` falhar.
- O teste existente tenta importar `pinneapple_models.register_all` como módulo, que não existe; o erro é engolido por `try/except`.
- O `__init__` não tem teste direto; só o submódulo `registry` é coberto, por um teste de fumaça.

7. `pinneapple_train`

Objetivo

`pinneapple_train` é um shim de compatibilidade. Ele não tem código próprio: só reexporta o treinador que hoje mora em `pinneapple_neural.trainer`. Existe para que código antigo que faz `from pinneapple_train import Trainer, TrainConfig` (ou importa `pinneapple_train.trainer`, `.losses`, `.metrics`) continue

funcionando.

O `__init__` reexporta `Trainer`, `TrainConfig`, `EarlyStopping`, `ModelCheckpoint`, `Metrics`, `Metric`, `MetricBundle` e `default_metrics`. Nomes opcionais (`TimeMarchingTrainer`, `RunLogger`, `Checkpoint`, `save_checkpoint`, `SweepConfig`, `run_parallel_sweep`, `AdaptiveSweepConfig`, `run_adaptive_sweep`) vêm dentro de `try/except` e viram `None` se o import falhar.

Os submódulos `trainer.py`, `losses.py` e `metrics.py` foram criados porque exemplos (`examples/time_series/*`, `examples/trainer/01_audited_training.py`), ferramentas de benchmark e a base de conhecimento de `pinneapple_problemdesign` importavam esses caminhos, que antes não existiam. Código novo deve importar direto de `pinneapple_neural.trainer`.

Números do inventário

módulos	linhas	classes públicas	funções públicas	testes em arquivos que importam o pacote	módulos sem teste direto
4	109	0	0	1 (1 arquivos)	1

Pasta de testes dedicada: `tests/pinneapple_train` (1 funções de teste).

Submódulos

submódulo	o que faz	módulos	classes + funções	testes*
`losses`	Reexporta `SupervisedLoss`, `PhysicsLossHook`, `CombinedLoss` e `build_loss` de `pinneapple_neural.trainer.losses`.	1	0	1
`metrics`	Reexporta as métricas (`MSE`, `MAE`, `RMSE`, `R2`, `RelL2`, `MaxError`, `default_metrics` etc.) de `pinneapple_neural.trainer.metrics`.	1	0	1
`trainer`	Reexporta `Trainer` e `TrainConfig` de `pinneapple_neural.trainer.trainer`.	1	0	1

* maior número de testes que importam um módulo do submódulo.

Módulos e API pública

`pinneapple_train`

[shim] Pacote de compatibilidade (marcado como obsoleto) que reexporta o treinador, callbacks e métricas de `pinneapple_neural.trainer`; nomes opcionais viram `None` se o import falhar. [sem teste direto do `__init__`]

`pinneapple_train/__init__.py` · 65 linhas · **sem teste direto** · palavras-chave no código (heurística; o texto acima diz o status real): shim ou obsoleto

`pinneapple_train.losses`

[shim] Reexporta as perdas (`SupervisedLoss`, `PhysicsLossHook`, `CombinedLoss`, `build_loss`) de `pinneapple_neural.trainer.losses`.

`pinneapple_train/losses.py` · 8 linhas · testes: 1

`pinneapple_train.metrics`

[shim] Reexporta as métricas de regressão e `default_metrics` de `pinneapple_neural.trainer.metrics`.

`pinneapple_train/metrics.py` · 14 linhas · testes: 1

`pinneapple_train.trainer`

[shim] Reexporta `Trainer` e `TrainConfig` de `pinneapple_neural.trainer.trainer`, para o caminho `pinneapple_train.trainer` que exemplos e ferramentas já usavam.

pinneapple_train/trainer.py · 22 linhas · testes: 1

Exemplos de uso reais do repositório

Teste mínimo do treinador pelo caminho antigo — `tests/pinneapple_train/test_trainer_minimal.py`

```
from pinneapple_train.trainer import Trainer, TrainConfig
from pinneapple_train.losses import CombinedLoss, SupervisedLoss
from pinneapple_train.metrics import default_metrics

...
    combined = CombinedLoss(supervised=SupervisedLoss("mse"), physics=None)

    def loss_fn(model_, y_hat, batch):
        return combined(model_, y_hat, batch)

    tr = Trainer(model=model, loss_fn=loss_fn, metrics=default_metrics())
```

Exemplo de séries temporais que usa o shim — `examples/time_series/01_FNO.py`

```
from pinneapple_train.trainer import Trainer, TrainConfig
from pinneapple_train.losses import CombinedLoss, SupervisedLoss
from pinneapple_train.metrics import default_metrics
```

Testes e status de validação

- Módulos com pelo menos um teste que os importa: 3 de 4.
- Funções de teste em arquivos que importam algum módulo do pacote: 1.

Limitações conhecidas

- É só um shim: todo o comportamento está em `pinneapple_neural.trainer`; o pacote é marcado como obsoleto.
- Vários nomes (`TimeMarchingTrainer`, `RunLogger`, `Checkpoint`, `SweepConfig`, `run_parallel_sweep`, `AdaptiveSweepConfig` etc.) são importados com `except Exception` e viram `None` em silêncio se o módulo de origem falhar.
- O `__init__` não tem teste direto; só os submódulos são exercitados por `tests/pinneapple_train/test_trainer_minimal.py`.
- Os submódulos reexportam só uma parte de `pinneapple_neural.trainer`; nomes fora das listas `__all__` não estão disponíveis por este caminho.

8. `pinneapple_solvers`

Objetivo

`pinneapple_solvers` é um shim de compatibilidade. Não tem código próprio: reexporta os solvers numéricos que hoje vivem em `pinneapple_simulation.numerical_solvers`, para que código antigo com `from pinneapple_solvers import ...` continue funcionando.

O `__init__` traz `SolverBase`, `SolverOutput`, `SolverRegistry` e `FDMSolver` diretamente. `FEMSolver`, `FVMSolver`, `LBMSolver`, `SpectralSolver`, `SPHSolver` e `FFTSolver` vêm dentro de `try/except` e viram `None` se o import falhar. `registry` é apenas um apelido para a classe `SolverRegistry`.

O submódulo `fft.py` foi criado porque `tests/pinneapple_solvers/test_fft.py` importava `pinneapple_solvers.fft`, que não existia. O comentário no `__init__` registra outro bug corrigido: antes ele importava um nome inexistente (`FFTProcessor`), e o erro era engolido pelo `except`. Código novo deve importar de `pinneapple_simulation.numerical_solvers`.

Números do inventário

módu-los	li-nhas	classes públicas	funções públicas	testes em arquivos que importam o pacote	módulos sem teste direto
2	71	0	0	1 (1 arquivos)	1

Pasta de testes dedicada: `tests/pinneapple_solvers` (4 funções de teste).

Submódulos

submódulo	o que faz	módulos	classes + funções	testes*
`fft`	Reexporta `FFTSolver` de `pinneapple_simulation.numerical_solvers.fft`.	1	0	1

* maior número de testes que importam um módulo do submódulo.

Módulos e API pública

`pinneapple_solvers`

[shim] Pacote de compatibilidade (obsoleto) que reexporta os solvers de `pinneapple_simulation.numerical_solvers`; os solvers opcionais viram `None` se o import falhar. [sem teste direto do `__init__`]

`pinneapple_solvers/__init__.py` · 61 linhas · **sem teste direto** · palavras-chave no código (heurística; o texto acima diz o status real): shim ou obsoleto

`pinneapple_solvers.fft`

[shim] Reexporta `FFTSolver` de `pinneapple_simulation.numerical_solvers.fft`, para o caminho usado em `tests/pinneapple_solvers/test_fft.py`.

`pinneapple_solvers/fft.py` · 10 linhas · testes: 1

Exemplos de uso reais do repositório

Teste de fumaça do FFTSolver pelo caminho antigo — `tests/pinneapple_solvers/test_fft.py`

```python
import torch
from pinneapple_solvers.fft import FFTSolver

def test_fft_solver_smoke():
    solver = FFTSolver()
    x = torch.randn(4, 128)
    out = solver(x)
    assert hasattr(out, "result")
    assert out.result.shape[0] == 4
```

Testes e status de validação

- Módulos com pelo menos um teste que os importa: 1 de 2.
- Funções de teste em arquivos que importam algum módulo do pacote: 1.

Limitações conhecidas

- É só um shim obsoleto: todo o comportamento está em `pinneapple_simulation.numerical_solvers`.
- `FEMSolver`, `FVMSolver`, `LBMSolver`, `SpectralSolver`, `SPHSolver` e `FFTSolver` são importados com `except Exception` e viram `None` em silêncio se o módulo de origem falhar.
- READMEs de exemplos (`examples/use_cases/TEMPLATE/README.md`, `examples/use_cases/drill_pipe_surrogate/README.md`) ainda citam `FEnicsBridge`, `OpenFOAMBridge` e

`pinneapple_solvers.openfoam_bridge`, que este shim não exporta.
- O `__init__` não tem teste direto; `tests/pinneapple_solvers/` testa `pinneapple_solvers.fft` e o `FDMSolver` pelo caminho novo.

Volume 2 — Física: problemas, equações e ciência

> Parte do Mapa completo do PINNeAPPle. Gerado por `docs/pt/_mapa/gerar.py` a partir do inventário do commit `e53cdcfb04`.

Como um problema físico é descrito e resolvido (`PhysicalProblem`, `pp.solve`), as equações, os presets industriais, o desenho de problemas, os experimentos por área, a condensação e as consultas físicas.

Sumário

1. pinneapple_physics
2. pinneapple_presets
3. pinneapple_problemdesign
4. pinneapple_science
5. pinneapple_condense
6. pinneapple_query

1. `pinneapple_physics`

Objetivo

`pinneapple_physics` é o pacote onde um problema físico é definido e resolvido. Ele junta a especificação de EDPs (`pde_environment`: `ProblemSpec`, condições de contorno e iniciais, presets como Navier-Stokes, calor, Burgers, elasticidade, modelos RANS e uma base de famílias de EDP), o compilador de PINN (`pinn_solver`), o compilador SymPy para autograd (`symbolic_pde`) e modelos de forma fechada (`closed_form`). O `__init__` reexporta esses nomes e mantém apelidos antigos (`environment`, `pinn`, `symbolic`) por compatibilidade.

O caminho principal é: montar um `ProblemSpec` (à mão, com `define_problem` ou com `get_preset`), compilar as perdas com `compile_physics`/`compile_problem` e treinar com `solve_pde`, um laço Adam próprio que amostra pontos de colocação a cada passo. `pipeline(task)` junta preset, modelo do registro de `pinneapple_neural` e `solve_pde` numa chamada e devolve um `PhysicsPipeline` com `.predict()`. Condições com `selector_type="tag"` exigem geometria real (por exemplo `STLDomainBatchBuilder` de `pinneapple_design`); sem ela, `solve_pde` levanta `TagConditionsUnresolved` antes de treinar. Os pontos de entrada `solve`/`compare` (de `solving.py`) e `Experiment` também são reexportados: é por aqui que `pp.solve(problem, "analytic")` usa as soluções de `closed_form`.

Além do núcleo PINN, o pacote traz camadas que julgam previsões pela física, sem solução de referência: `darcy_checks` e `maxwell_checks` (verificações usadas pelo comando `pinneapple check` e por relatórios de confiança APPROVED/REVIEW/REJECT), `physics_audit` (premissas de cada modelo, números adimensionais, monitor de validade, atribuição de falhas, recomendações e roteamento) e `multiscale` (filtros LES, fechamentos, resíduos multiescala e seleção de modelos sensível à escala). `non_hermitian` cobre dímeros PT-simétricos e pontos excepcionais; `weather` é um caso de uso de previsão global com ERA5 do WeatherBench2.

Esta parte do mapa cobre o `__init__` do pacote e os submódulos `closed_form`, `darcy_checks`, `maxwell_checks`, `multiscale`, `non_hermitian`, `physics_audit`, `pinn_solver` e `weather`. Os demais submódulos (`pde_environment`, `symbolic_pde`, `solving`, `ensemble`, etc.) são descritos nas outras partes.

Esta parte cobre quatro submódulos de `pinneapple_physics`. `astro` é um observatório em NumPy próprio (escalas de tempo, efeméride JPL, posições aparentes, fenômenos, satélites, pequenos corpos, estrelas, lentes gravitacionais, fichas bilíngues) com modelos de IA conferidos contra referências independentes. `blackhole` simula acreção em torno de um buraco negro e treina um previsor de 'tempo do buraco negro' com horizonte

de confiança. `pde_environment` define a linguagem de problemas do framework (`ProblemSpec`, condições, `ProblemBuilder`) e o catálogo de presets registrados usados por `get_preset` e pelos solvers. `symbolic_pde` compila EDPs escritas em SymPy ou em texto para resíduos PyTorch e condições de contorno, e liga sistemas de texto a um `PINNProblem`.

Esta parte cobre a camada de uso direto do `pinneapple_physics`: a descrição única de um problema (`PhysicalProblem`), a porta de entrada `pp.solve`/`pp.compare` com registro de métodos, `pp.compile` (reuso de derivadas no resíduo), `Experiment` reprodutível com métricas padronizadas e envio a MLflow/W&B, ensembles adaptativos de modelos físicos com aprendizado online, geração de scripts FDM e FEniCS independentes, ponte para o DeepXDE, problemas de artigos (PINNsFormer, X-TFC) e módulos de domínio: térmica de placa de circuito (PCB Hotspot), elasticidade axissimétrica, desgaste de Archard e projeção conservativa.

Números do inventário

módulos	linhas	classes públicas	funções públicas	testes em arquivos que importam o pacote	módulos sem teste direto
166	43.959	169	763	1.075 (105 arquivos)	52

Pasta de testes dedicada: `tests/pinneapple_physics` (48 funções de teste).

Submódulos

submódulo	o que faz	módulos	classes + funções	testes *
`advection_diffusion_1d`	Banco de teste 1D periódico de advecção-difusão com solução exata espectral, esquemas de diferenças finitas e resíduo da EDP.	1	1	13
`astro`	Astronomia posicional e astrofísica do observatório PINNeAPPle, validada contra NOVAS, Horizons e skyfield, com substitutos neurais e fichas para o público.	24	216	71
`blackhole`	Simulação hidrodinâmica de acreção em buraco negro, dataset, previsor U-Net com horizonte de confiança, avaliação de habilidade, traçado de raios e visualização 3D.	12	60	19
`closed_form`	Modelos de engenharia em forma fechada (aletas, vigas e fadiga, difusão em membrana, Helmholtz, dissipadores, PCB, escoamento em tubos, bocal de foguete, Burgers exato, etc.) para verdade de referência rápida sem resolver EDP.	19	91	103
`codegen`	Gera arquivos Python independentes (FDM em NumPy ou FEniCS/dolfinx) a partir de um `ProblemSpec`.	3	4	13
`compile_api`	`pp.compile`: prepara um problema para resolver mais rápido reaproveitando derivadas dentro de cada avaliação da perda.	1	3	5
`conservation`	Projeção exata que força a integral ponderada da saída de um modelo a um valor conservado.	1	3	6
`darcy_checks`	Verificações físicas sem referência de campos de Darcy 2-D (resíduo de volumes finitos, limites certificados de erro, balanço de fluxo, contorno, envelope de coeficientes) e decisão de implantação.	1	31	26
`deepxde_bridge`	Traduz um `ProblemSpec` em objetos reais do DeepXDE e treina com ele, como backend alternativo.	2	4	9
`ensemble`	Ensemble adaptativo de modelos físicos (PINN, operadores, GNN, solvers) escolhidos ou combinados online pelo erro observado.	1	9	68
`ensemble_viz`	Gera um GIF animado de uma execução do ensemble adaptativo.	1	1	3
`experiment`	`Experiment`: execução reprodutível de um método num problema, com métricas e proveniência.	1	3	0
`maxwell_checks`	Verificações das equações de Maxwell e das leis de balanço (Gauss, Faraday, Ampère-Maxwell, continuidade, Poynting, PEC, reciprocidade, passividade) em campos de qualquer origem.	1	19	51
`metrics`	Métricas de erro padrão da biblioteca (`pp.metrics`), por campo e agregadas.	1	11	36
`multiscale`	Física multiescala para ensembles adaptativos: filtros LES, solver de vorticidade 2-D, fechamentos de subfiltro, resíduos multiescala, consistência entre escalas e roteamento sensível à escala.	11	76	45
`non_hermitian`	Dímeros e cadeias PT-simétricos, pontos excepcionais e a verificação simbólica de leis de sintonia do EP contra um teorema de impossibilidade.	4	19	8
`online_learning`	Algoritmos de aprendizado online com especialistas (Fixed-Share, AdaHedge, conformal adaptativo) usados pelos métodos adaptativos.	1	4	0

submódulo	o que faz	módulos	classes + funções	testes *
`paper_problems`	Problemas de teste do PINNsFormer em grade (x, t) com solução analítica.	1	7	5
`paper_problems_xtfc`	Problemas de teste do artigo X-TFC com equação, domínio, solução exata e configurações relatadas.	1	4	10
`pcb`	Modelo térmico de placa de circuito montada (motor do app PCB Hotspot): solver de volumes finitos, what-if e calibração.	3	10	16
`pde_environment`	Especificação de problemas (`ProblemSpec`, condições, `ProblemBuilder`), catálogo de presets por domínio, resíduos de turbulência e base de famílias de EDP.	28	154	154
`physical_problem`	`PhysicalProblem`: descrição única de um problema físico com parâmetros, unidades, quantidades de interesse e fingerprint.	1	3	12
`physics_audit`	Auditoria física de modelos: premissas e condições de validade, números adimensionais, monitor de validade sem verdade de referência, atribuição de falhas, recomendações, grafo de conhecimento e roteamento.	8	65	37
`pinn_solver`	Compilador de PINN: traduz um `ProblemSpec` em funções de perda, mais DoMINO (decomposição de domínio), PINN estocástico, fábrica SymPy legada e leitura de dados UPD.	19	41	121
`solid`	Pós-processamento de mecânica dos sólidos por autograd (deformações, tensões, resíduo de equilíbrio, von Mises).	2	6	8
`solving`	`pp.solve` e `pp.compare`: ponto de entrada único para resolver um problema por qualquer método registrado e comparar métodos.	1	12	71
`symbolic_pde`	Compilador de EDPs simbólicas (SymPy e texto) para resíduos PyTorch, com condições de contorno rígidas e suaves e ponte para PINN.	6	24	17
`tracking`	Adaptador que envia um `ExperimentResult` ao MLflow ou ao Weights & Biases.	1	3	3
`tribology`	Desgaste por deslizamento de uma barra contra contraface pela lei de Archard.	2	3	20
`weather`	Caso de uso de previsão do tempo global: dados ERA5 do WeatherBench2, U-Net esférica, treino por rollout, métricas de habilidade, eventos extremos e animações.	7	38	8

* maior número de testes que importam um módulo do submódulo.

Módulos e API pública

`pinneapple_physics`

Ponto de entrada do pacote: reexporta `pde_environment`, `pinn_solver`, `symbolic_pde`, `closed_form`, `solve`/`compare` e `Experiment`, mantém apelidos antigos e define os auxiliares `define_problem`, `compile_physics`, `solve_pde` e `pipeline`.

`pinneapple_physics/__init__.py` · 604 linhas · testes: 206

nome	tipo	assinatura	o que faz
`TagConditionsUnresolved`	classe	`()`	Erro (`ValueError`) levantado por `solve_pde` quando há condições `selector_type="tag"` sem pontos e máscaras fornecidos pelo chamador, para não treinar sem essas condições.
`PhysicsPipeline`	classe	`(model, spec, history)`	Resultado de `pipeline`: guarda modelo treinado, `ProblemSpec` e histórico, e oferece `.predict(x)` que devolve um array numpy.
`compile_physics`	função	`(spec, **kwargs)`	Atalho para `compile_problem(spec, **kwargs)`: compila o `ProblemSpec` em perdas PINN ponderadas.
`identify`	função	`(description)`	Atalho para `identify_pde`: identifica a família de EDP a partir de uma descrição em linguagem natural.
`define_problem`	função	`(preset=None, **spec_kwargs) -> 'ProblemSpec'`	Monta um `ProblemSpec`: a partir de um preset nomeado (sobrescrevendo atributos) ou diretamente dos argumentos.
`solve_pde`	função	`(spec, model, *, epochs=5000, device='cpu', lr=0.001, n_collocation=2048, seed=0, weights…)`	Treina um modelo num `ProblemSpec` com um laço Adam próprio, amostrando colocação e pontos de condição (`all`/`callable`) a cada passo; aceita pontos fixos `x_col` e máscaras de tags.
`pipeline`	função	`(task, *, architecture='modified_mlp', hidden_dim=64, n_layers=4, epochs=2000, device='cp…)`	Uma chamada para tarefa nomeada: `get_preset` + `ModelRegistry.build` + `solve_pde`, devolvendo um `PhysicsPipeline`.

pinneapple_physics.advection_diffusion_1d

Advecção-difusão periódica 1D em [0, 2π) com solução exata por modos de Fourier, esquemas upwind e Lax-Wendroff e resíduo da EDP de qualquer campo previsto; serve de banco de teste para ensembles.

pinneapple_physics/advection_diffusion_1d.py · 80 linhas · testes: 13

nome	tipo	assinatura	o que faz
AdvectionDiffusion1D	classe	(nx, nt, t_end)	Banco de teste: sorteia casos, dá a solução exata, roda esquemas FD e calcula o resíduo da EDP de um campo previsto.

pinneapple_physics.astro

Pacote de astronomia posicional e astrofísica do observatório PINNeAPPle, em NumPy próprio e validado contra USNO NOVAS, JPL Horizons e skyfield; reexporta Observer, Time, apparent e afins.

pinneapple_physics/astro/__init__.py · 54 linhas · testes: 71

pinneapple_physics.astro.astrometry

Posições aparentes do Sol, Lua, planetas e estrelas de catálogo para um observador na Terra (tempo de luz, deflexão, aberração), mais refração, massa de ar e extinção. [sem teste direto]

pinneapple_physics/astro/astrometry.py · 194 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
Observer	classe	(lat, lon, elevation_m, pressure_mbar, temperature_c, name)	Um lugar na Terra: latitude/longitude geodésicas, altitude e o tempo (pressão, temperatura) usado pela refração; dá a posição em ITRS e GCRS.
Apparent	classe	(body, ra, dec, ra_astrometric, dec_astrometric, distance_km, light_time_s, range_rate_km…)	Resultado de apparent: RA/Dec aparentes e astrométricas, distância, tempo de luz, taxa de distância, altitude/azimute e ângulo horário.
refraction_deg	função	(alt_true_deg, pressure_mbar=1010.0, temperature_c=10.0)	Refração atmosférica (graus) para uma altitude verdadeira, fórmula de Saemundsson com correção de pressão e temperatura; zero abaixo de -1 grau.
airmass	função	(alt_apparent_deg)	Massa de ar relativa (Kasten & Young 1989), finita até o horizonte e infinita abaixo dele.
extinction_mag	função	(alt_apparent_deg, k_v=0.25)	Extinção atmosférica em magnitudes (banda V) para um coeficiente k_v por massa de ar.
apparent	função	(body, t, observer=None, eph=None, *, deflection=True, refraction=True) -> Apparent	Posição aparente de um corpo do Sistema Solar num instante (escalar ou vetor), geocêntrica se não houver observador.
star_apparent	função	(ra_j2000, dec_j2000, t, observer=None, *, pm_ra_mas=0.0, pm_dec_mas=0.0, parallax_mas=0.…)	Posições aparentes de estrelas de catálogo (ICRS J2000) num instante, com movimento próprio e paralaxe.

pinneapple_physics.astro.conditions

Condições de observação a partir de previsão numérica aberta (Open-Meteo, CC BY 4.0): ensemble de nebulosidade, probabilidade de céu limpo, calibração e modo de céu 'perfeito'/'real'. Depende de rede.

pinneapple_physics/astro/conditions.py · 126 linhas · testes: 25

nome	tipo	assinatura	o que faz
fetch_ensemble	função	(lat, lon, model='icon_seamless', hours=48, timeout=15.0)	Baixa da API de ensemble da Open-Meteo as variáveis horárias (nuvens, umidade, vento) de um modelo para um lugar.
members	função	(resp, var) -> np.ndarray	Matriz (membros, horas) de uma variável do ensemble, com a rodada de controle e cada membro.
summarize	função	(resp, clear_threshold=30.0) -> dict	Resume o ensemble: probabilidade de céu limpo por hora (nuvens abaixo de um limiar), umidade e vento em altitude.
calibrate	função	(p_forecast, observed_clear, bins=10) -> dict	Confiabilidade de probabilidades passadas de céu limpo contra o observado, com mapa de correção por faixa de probabilidade.

nome	tipo	assinatura	o que faz
fetch_cloud_forecast	função	(lat, lon, days=3, timeout=8.0) -> dict	Previsão horária de cobertura de nuvens para os próximos dias pela API da Open-Meteo; levanta erro sem rede.
cloud_at	função	(forecast, iso_utc) -> float | None	Cobertura de nuvens num instante, interpolada linearmente na previsão horária; None fora dela.
sky_view	função	(mode, forecast, iso_utc) -> dict	O que o céu deixa ver numa sessão: modo 'perfect' ignora o tempo, 'real' usa a previsão e esconde a vista quando nublado.

pinneapple_physics.astro.conjunction

Triagem de conjunções: aproximação mínima no movimento relativo de Clohessy-Wiltshire, probabilidade de colisão 2D e propagação de dois corpos com J2.

pinneapple_physics/astro/conjunction.py · 160 linhas · testes: 8

nome	tipo	assinatura	o que faz
ClosestApproach	classe	(t_tca_s, position_km, velocity_km_s, miss_distance_km, relative_speed_km_s, perpendicula…)	Resultado da aproximação mínima: instante, posição e velocidade relativas, distância de passagem e velocidade relativa.
cw_mean_motion	função	(a_km, mu_km3_s2=398600.4418) -> float	Movimento médio n = sqrt(mu/a^3) da órbita circular de referência.
cw_closest_approach	função	(n, state0, duration_s, n_samples=2000) -> ClosestApproach	Aproximação mínima num intervalo para um estado relativo de Clohessy-Wiltshire em torno de uma órbita de referência.
collision_probability_2d	função	(xm, zm, sigma_x, sigma_z, r_c) -> float	Probabilidade de uma gaussiana 2D (média e desvios dados) cair dentro de um disco de raio r_c.
collision_probability_centered	função	(r_c, sigma) -> float	Forma fechada (CDF de Rayleigh) da probabilidade de colisão para gaussiana isotrópica centrada.
screen_cw_conjunction	função	(n, state0, *, duration_s=None, n_samples=2000, hard_body_radius_km=0.01, sigma_km=1.0) -…)	Aproximação mínima mais probabilidade de colisão para um encontro CW (um período de referência por padrão).
two_body_j2_rhs	função	(mu, J2, Re)	Lado direito f(t, estado) do movimento de dois corpos com aceleração de J2.
propagate_two_body_j2	função	(state0, t_eval, *, mu=398600.4418, J2=0.00108262668, Re=6378.137, rtol=1e-11, atol=1e-08…)	Propaga estados de dois corpos com J2 a partir de um estado inicial, com integrador DOP853.

pinneapple_physics.astro.decay

Decaimento orbital de satélites em órbita baixa por arrasto atmosférico: integração de referência, envelope Monte Carlo, um PINN de decaimento com verificações e comparação com SGP4. Usa NRLMSIS (pymsis) quando disponível.

pinneapple_physics/astro/decay.py · 258 linhas · testes: 16

nome	tipo	assinatura	o que faz
DensityTable	classe	(h_km=None, **kw)	Log-densidade atmosférica numa grade de altitudes, interpolada em NumPy e PyTorch para avaliar modelos rápido.
exponential_density	função	(h_km)	Densidade atmosférica por um modelo exponencial em faixas de altitude (tabela interna), usada sem NRLMSIS.
msis_available	função	() -> bool	Diz se o pacote opcional pymsis (NRLMSIS) pode ser importado.
orbit_mean_density	função	(h_km, date='2026-10-13T00:00', f107=150.0, f107a=150.0, ap=15.0, inclination_deg=51.6, n…)	Densidade NRLMSIS 2.0 média ao longo da órbita (latitudes e horas locais) em cada altitude.
ballistic_from_bstar	função	(bstar_per_er) -> float	Coeficiente balístico B = C_D A / m implícito num B* do SGP4.
reference_decay	função	(h0_km, B, days, table, n=200, reentry_km=150.0)	Altitude versus tempo por integração Runge-Kutta adaptativa da EDO de decaimento.
envelope	função	(h0_km, B, days, *, f107=150.0, f107a=None, ap=15.0, date='2026-10-13T00:00', inclination…)	Faixa Monte Carlo do decaimento sobre entradas incertas (fluxo solar, atividade geomagnética, coeficiente balístico): mediana e percentis 5 e 95.

nome	tipo	assinatura	o que faz
`train_decay_pinn`	função	`(h0_km=415.0, B=0.005, days=60.0, *, f107=150.0, ap=15.0, date='2026-10-13T00:00', inclin…)`	Treina o PINN de decaimento e devolve relatório com previsão, referência, envelope, verificações e veredito.
`sgp4_decay`	função	`(line1, line2, days=30.0, step_min=10.0)`	Altitude média versus tempo prevista pelo SGP4 a partir de um TLE (média por órbita).
`compare_with_sgp4`	função	`(line1, line2, days=30.0, f107=70.0, ap=8.0, B=ISS_B) -> dict`	Coloca lado a lado o decaimento deste modelo (NRLMSIS e B físico) e o previsto pelo SGP4 do mesmo TLE.

pinneapple_physics.astro.ephemeris

Efeméride do Sistema Solar sobre um kernel JPL DE (domínio público) lido por `.spk`; localiza o kernel por variável de ambiente ou pelo pacote `skyfield_data`.

`pinneapple_physics/astro/ephemeris.py` · 77 linhas · testes: 71

nome	tipo	assinatura	o que faz
`Ephemeris`	classe	`(path=None)`	Efeméride sobre um kernel SPK: estados baricêntricos e relativos (km, km/s) dos corpos por data TDB.
`find_kernel`	função	`() -> str`	Acha o arquivo do kernel DE: variável `PINNEAPPLE_DE_KERNEL` ou o `de421.bsp` do pacote `skyfield_data`.
`default_ephemeris`	função	`(path=None) -> Ephemeris`	Cria uma `Ephemeris` com o kernel padrão (ou o caminho dado).

pinneapple_physics.astro.facts

Fichas de objetos do céu para o público geral, em inglês e português: números com fonte, curiosidades, história, missões, quiz, busca e 'como ver hoje'.

`pinneapple_physics/astro/facts.py` · 746 linhas · testes: 30

nome	tipo	assinatura	o que faz
`sources`	função	`() -> dict`	Todas as fontes que um número ou curiosidade pode citar (título, editor, URL, licença).
`ids`	função	`() -> dict`	Todos os objetos com ficha e seu tipo (planetas, estrelas, Messier, constelações, satélites).
`resolve`	função	`(ref) -> str`	Converte um id ou referência do observatório no id da ficha; levanta KeyError se não houver ficha.
`names`	função	`(oid) -> dict`	Nomes de exibição de uma ficha em inglês e português.
`numbers`	função	`(oid) -> dict`	Números-chave de uma ficha, cada um com valor, unidade e fonte.
`comparison`	função	`(key, number, lang='en') -> str | None`	Comparação cotidiana calculada a partir de um número (fonte: calculado).
`quiz`	função	`(oid, lang='en', seed=0, n=3) -> list[dict]`	Quiz curto de múltipla escolha gerado a partir dos números da ficha.
`card`	função	`(ref, lang='en') -> dict`	Ficha completa de um objeto no idioma pedido: nome, resumo, números, curiosidades com fonte, história, missões, quiz e fontes.
`list_cards`	função	`(kind=None, lang='en') -> list[dict]`	Lista as fichas (id, tipo, nome), opcionalmente de um tipo.
`search`	função	`(query, lang=None, limit=12) -> list[dict]`	Busca fichas pelo nome em inglês ou português, id ou designação, ignorando acentos e caixa.
`did_you_know`	função	`(lang='en', seed=None) -> dict`	Uma curiosidade aleatória das fichas escritas à mão, para dica de abertura.
`validate`	função	`() -> list[str]`	Lista problemas do conjunto de fichas (fonte por número, dois idiomas, número de curiosidades, história, fontes conhecidas).
`position`	função	`(ref)`	RA/Dec J2000 de um objeto fixo (estrela, Messier, rótulo de constelação); None para corpos do Sistema Solar e satélites.
`tonight`	função	`(ref, observer, t, hours=24.0, step_min=10.0, eph=None) -> dict | None`	Como ver um objeto nas próximas horas: altitude atual, nascer, culminação, ocaso e melhor altura com céu escuro.

pinneapple_physics.astro.frames

Rotações de referencial: viés, precessão (IAU 2006), nutação (IAU 1980, termos acima de 5 mas), posição geodésica WGS84, coordenadas horizontais e eclípticas.

pinneapple_physics/astro/frames.py · 202 linhas · testes: 16

nome	tipo	assinatura	o que faz
fundamental_args	função	(T)	Argumentos de Delaunay (D, M, M', F, Omega) para séculos julianos TT desde J2000.
nutation	função	(jd_tt)	Nutação em longitude e obliquidade (rad).
mean_obliquity	função	(jd_tt)	Obliquidade média da eclíptica (IAU 2006) numa data TT, em radianos.
rot_x	função	(a)	Matriz de rotação em torno do eixo x (vetorizada no ângulo).
rot_y	função	(a)	Matriz de rotação em torno do eixo y (vetorizada no ângulo).
rot_z	função	(a)	Matriz de rotação em torno do eixo z (vetorizada no ângulo).
precession_matrix	função	(jd_tt)	Matriz de precessão J2000 média para equador/equinócio médio da data (IAU 2006).
nutation_matrix	função	(jd_tt)	Matriz do equador médio da data para o verdadeiro.
npb_matrix	função	(jd_tt)	Matriz combinada GCRS para equador e equinócio verdadeiros da data (N P B).
ecliptic_matrix	função	(jd_tt=None)	Matriz GCRS para eclíptica (da data, ou a eclíptica média J2000 se a data for None).
geodetic_to_itrs	função	(lat_deg, lon_deg, height_m=0.0)	Posição do observador (km) no referencial terrestre a partir de latitude, longitude e altitude.
itrs_to_geodetic	função	(r)	Inversa de geodetic_to_itrs por iteração de Bowring.
radec	função	(v)	Vetores para RA, Dec (graus) e norma.
from_radec	função	(ra_deg, dec_deg)	Vetor unitário a partir de RA e Dec em graus.
equatorial_to_horizontal	função	(ha_rad, dec_rad, lat_rad)	Ângulo horário, declinação e latitude para altitude e azimute.
horizontal_to_equatorial	função	(alt_deg, az_deg, lat_deg)	Altitude e azimute para ângulo horário e declinação.
angular_separation	função	(ra1, dec1, ra2, dec2)	Separação angular de grande círculo (fórmula de Vincenty).
position_angle	função	(ra1, dec1, ra2, dec2)	Ângulo de posição (norte por leste) do ponto 2 visto do ponto 1.

pinneapple_physics.astro.journey

Geometria de uma 'viagem' do observador até um planeta, lua, o Sol ou uma estrela: posições heliocêntricas, distância, tempos de viagem, magnitude e cena 3D.

pinneapple_physics/astro/journey.py · 193 linhas · testes: 19

nome	tipo	assinatura	o que faz
galactic	função	(v)	Coordenadas galácticas (l, b) de um vetor ICRS.
body_position	função	(body, t, eph=None) -> np.ndarray	Posição heliocêntrica ICRS de um corpo do Sistema Solar (geométrica, sem tempo de luz).
observer_position	função	(observer, t, eph=None) -> np.ndarray	Posição heliocêntrica ICRS de um lugar na Terra (Terra pela efeméride mais o local em GCRS).
star_position	função	(ra_deg, dec_deg, parallax_mas) -> np.ndarray	Posição heliocêntrica de uma estrela a partir da direção J2000 e da paralaxe.

nome	tipo	assinatura	o que faz
travel_times	função	(distance_km) -> dict	Segundos para percorrer uma distância nas velocidades da tabela SPEEDS.
apparent_magnitude_star	função	(abs_mag, distance_km)	Magnitude aparente m = M + 5 log10(d / 10 pc).
angular_diameter_deg	função	(radius_km, distance_km)	Diâmetro angular (graus) de um corpo de raio dado a uma distância.
plan	função	(observer, target, t, standoff=None, n_profile=60, eph=None) -> dict	Monta a viagem do observador até um alvo numa data: distância, tempo de luz, perfil de magnitude e tamanho.
scene	função	(t, observer=None, max_parallax_err=0.2, eph=None) -> dict	Tudo o que a viagem 3D desenha: planetas, Sol e Lua com raios, e as estrelas brilhantes com paralaxe utilizável.

pinneapple_physics.astro.lensing

Demonstrações de lente gravitacional no céu real: um buraco negro de Schwarzschild na frente das estrelas, renderizado com o traçador de geodésicas de `blackhole.raytrace`, mais fórmulas fechadas (deflexão, raio de Einstein, sombra vs EHT).

pinneapple_physics/astro/lensing.py · 366 linhas · testes: 32

nome	tipo	assinatura	o que faz
deflection_angle_rad	função	(mass_kg, impact_parameter_m)	Deflexão da luz em campo fraco, 4GM/(c^2 b).
einstein_radius_rad	função	(mass_kg, d_lens_m, d_source_m)	Raio de Einstein (rad) para massa e distâncias da lente e da fonte.
shadow_diameter_rad	função	(mass_kg, distance_m)	Diâmetro angular da sombra de Schwarzschild, 2 sqrt(27) GM/(c^2 D).
eht_comparison	função	() -> list[dict]	Compara o diâmetro de sombra previsto com o anel observado pelo EHT para cada objeto da tabela interna, em sigmas.
sun_deflection_arcsec	função	(elongation_deg)	Deflexão da luz estelar pelo Sol a uma elongação dada (Eddington 1919).
star_splat	função	(v, mag_limit=6.5, sigma_deg=0.1, mag_white=1.5)	Largura gaussiana e brilho de pico para desenhar uma estrela de magnitude dada.
catalog_sky	função	(width=4096, height=2048, mag_limit=6.5, milky_way=0.32, star_sigma_deg=0.1) -> np.ndarray	Imagem RGB equiretangular do catálogo de estrelas brilhantes sobre o brilho da Via Láctea.
tone_map	função	(img, exposure=1.6, gamma=0.75) -> np.ndarray	Mapeia radiância linear do céu para valores de exibição em [0, 1] com ombro suave e gama.
bright_stars_in_field	função	(ra_deg, dec_deg, radius_deg, mag_limit=3.5) -> int	Conta estrelas do catálogo mais brilhantes que um limite dentro de um raio.
lensed_sky	função	(ra_deg, dec_deg, fov_deg=30.0, width=480, height=270, r_cam=30.0, sky=None, supersample=…)	Renderiza o céu real em torno de (ra, dec) visto através de um buraco negro de Schwarzschild.
trace_thin_disc	função	(width=640, height=360, inclination_deg=80.0, r_cam=60.0, fov_deg=32.0, max_crossings=3, …)	Traçado de raios exato para trás num disco equatorial fino em torno de um buraco negro de Schwarzschild.
disc_image	função	(tr, r_in=6.0, r_out=20.0, doppler=True, t_peak=3900.0, exposure=2.4, gamma=0.6, opacity=…)	Cor e intensidade do disco para os raios traçados, com Doppler e desvio gravitacional opcionais.
accretion_view	função	(ra_deg, dec_deg, width=640, height=360, inclination_deg=86.0, r_cam=60.0, fov_deg=42.0, …)	Buraco negro com disco de acreção fino na frente do céu real, com geodésicas exatas, Doppler e bloom.
lensing_view	função	(ra_deg, dec_deg, fov_deg=60.0, width=480, height=270, r_cam=120.0, sky=None, disc='auto'…)	Vista pronta para exibição de um buraco negro na frente do céu, com ou sem disco ('auto').

pinneapple_physics.astro.nbody

Integradores clássicos de N corpos para Sol-planetas(-Lua), com diagnósticos de conservação de energia e momento angular. [sem teste direto]

pinneapple_physics/astro/nbody.py · 116 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
initial_state	função	(jd_tdb, bodies=SOLAR_SYSTEM, eph=None)	Posições, velocidades baricêntricas e GM dos corpos numa data, lidos da efeméride.
accelerations	função	(r, mu)	Acelerações gravitacionais mútuas de N corpos.
energy	função	(r, v, mu)	Energia total (por unidade de G) do sistema de N corpos.
angular_momentum	função	(r, v, mu)	Momento angular total do sistema, ponderado por GM.
integrate	função	(r0, v0, mu, t_end_s, dt_s, method='yoshida4', n_out=200)	Integra de t=0 a t_end_s (Yoshida-4 por padrão) e devolve trajetórias e históricos de erro de energia e momento angular.

pinneapple_physics.astro.neural_propagator

Propagador aprendido para asteroides próximos da Terra: uma rede corrige o desvio da órbita kepleriana (no referencial R/T/N) para muitos corpos, com relatório de confiança.

pinneapple_physics/astro/neural_propagator.py · 240 linhas · testes: 16

nome	tipo	assinatura	o que faz
NeuralPropagator	classe	(net=None, xm=None, xs=None, ys=None, info=None)	Rede que prevê a correção à órbita kepleriana de um asteroide; treina, prevê, salva e carrega.
sample_population	função	(n, seed=0)	Sorteia elementos osculadores parecidos com os de NEOs.
kepler_positions	função	(body, jd)	Posições e velocidades keplerianas (ICRF) de um SmallBody nas datas pedidas.
features	função	(body, tau)	Entradas da rede: elementos, ângulos em seno/cosseno, tempo normalizado e anomalia média kepleriana.
rtn_basis	função	(rk, vk)	Vetores unitários radial, transversal e normal da órbita kepleriana.
make_dataset	função	(n_bodies=160, n_t=60, seed=0, cache=None)	Monta entradas e alvos: o desvio da órbita kepleriana no referencial R/T/N.
tisserand	função	(r_km, v_km_s, a_jup_au=5.2026)	Parâmetro de Tisserand em relação a Júpiter a partir de estados heliocêntricos.
evaluate	função	(model, bodies, n_t=60)	Erros do propagador aprendido e de Kepler sozinho contra o integrador completo, por corpo, com tempo de execução.
trust_report	função	(folder=MODEL_DIR, real=None) -> dict	Veredito da rede distribuída a partir da avaliação em corpos não vistos (precisa superar Kepler por margem fixa).

pinneapple_physics.astro.observing

Sessão de observação: visibilidade de um alvo na noite, nascer/trânsito/ocaso, brilho do céu (Krisciunas & Schaefer), óptica do telescópio, slews GoTo, rotação de campo e recortes de survey HiPS.

pinneapple_physics/astro/observing.py · 356 linhas · testes: 21

nome	tipo	assinatura	o que faz
Telescope	classe	(name, aperture_mm, focal_mm, kind, central_obstruction)	Telescópio: nome, abertura, focal, tipo e obstrução central.
Eyepiece	classe	(name, focal_mm, afov_deg)	Ocular: nome, focal e campo aparente.
Camera	classe	(name, width_mm, height_mm, pixel_um)	Câmera: nome, tamanho do sensor e tamanho do pixel.
target_altaz	função	(target, t, observer, refraction=True, eph=None)	Altitude, azimute e RA/Dec da data de um corpo ou de um par (ra, dec) J2000.
rise_transit_set	função	(target, observer, jd_start, days=1.0, eph=None) -> dict	Nascer, trânsito superior e ocaso de corpos, estrelas e objetos de céu profundo num intervalo.
extinction	função	(alt_apparent_deg, k=None) -> dict	Extinção por banda numa altitude aparente (k vezes massa de ar de Kasten-Young).
nl_to_mag	função	(b_nl)	Brilho superficial de nanolamberts para magnitude V por segundo de arco quadrado.

nome	tipo	assinatura	o que faz
mag_to_nl	função	(m)	Inversa de nl_to_mag: magnitude por arcsec^2 para nanolamberts.
moon_brightness_nl	função	(moon_phase_angle_deg, moon_sep_deg, moon_alt_deg, target_alt_deg, k=0.172)	Brilho do céu acrescentado pelo luar no alvo (Krisciunas & Schaefer 1991).
dark_sky_nl	função	(target_alt_deg, k=0.172, b_zenith_nl=79.0)	Brilho natural do céu noturno no alvo (KS91).
twilight_mag	função	(sun_alt_deg)	Brilho aproximado do zênite no crepúsculo a partir da altitude do Sol (ajuste linear).
sky_brightness	função	(sun_alt_deg, moon_phase_angle_deg, moon_sep_deg, moon_alt_deg, target_alt_deg, k=0.172, …)	Brilho do céu no alvo: céu escuro, luar e crepúsculo somados.
visibility	função	(target, observer, t, hours=24.0, step_min=5.0, min_alt=30.0, eph=None) -> dict	A noite de um alvo: amostras de altitude, massa de ar, Sol, Lua, brilho do céu, eventos e a melhor janela.
optics	função	(telescope, eyepiece=None, camera=None, naked_eye_limit=6.0, wavelength_nm=550.0) -> dict	O que um telescópio mostra: aumento, campo, pupila de saída, limites de resolução e magnitude limite.
axis_time	função	(delta_deg, rate_deg_s, accel_deg_s2) -> float	Tempo para mover um eixo com perfil de velocidade trapezoidal (ou triangular).
hour_angle_dec	função	(alt_deg, az_deg, lat_deg)	Ângulo horário e declinação a partir de altitude e azimute.
slew	função	(start_altaz, target_altaz, lat_deg, mount='altaz', rate_deg_s=4.0, accel_deg_s2=2.0, set…)	Simula um slew GoTo de montagem alt-az ou equatorial, com flip de meridiano e recusa abaixo da altitude mínima.
parallactic_angle	função	(lat_deg, ha_deg, dec_deg)	Ângulo paralático no alvo.
field_rotation_rate	função	(lat_deg, alt_deg, az_deg)	Taxa de rotação de campo de uma montagem alt-az.
survey_url	função	(ra_deg, dec_deg, fov_deg, width=512, height=512, rotation_deg=0.0, survey=SURVEY) -> str	URL de um recorte JPEG de um survey HiPS (CDS hips2fits) em torno de (ra, dec).
survey_cutout	função	(ra_deg, dec_deg, fov_deg, width=512, height=512, cache_dir=None, timeout=8.0, survey=SUR…)	Bytes JPEG do recorte, com cache em disco; levanta OSError sem rede e sem cache.

pinneapple_physics.astro.orbits

Órbitas de dois corpos: elementos osculadores a partir do vetor de estado, equação de Kepler, propagação e órbitas heliocêntricas dos planetas pela efeméride JPL.

pinneapple_physics/astro/orbits.py · 121 linhas · testes: 27

nome	tipo	assinatura	o que faz
solve_kepler	função	(M, e, tol=1e-14, maxiter=50)	Anomalia excêntrica a partir da anomalia média (Newton) para órbitas elípticas.
elements_from_state	função	(r, v, mu=GM_SUN) -> dict	Elementos keplerianos osculadores a partir de posição e velocidade.
state_from_elements	função	(el, mu=GM_SUN, dt_s=0.0)	Posição e velocidade após um intervalo de movimento keplerianо puro.
heliocentric_state	função	(body, jd_tdb, frame='ecliptic', eph=None)	Posição e velocidade heliocêntricas de um corpo, na eclíptica J2000 ou em eixos ICRF.
planet_elements	função	(body, jd_tdb, eph=None) -> dict	Elementos heliocêntricos osculadores de um planeta (geocêntricos para a Lua).
orbit_polyline	função	(body, jd_tdb, n=256, eph=None)	Uma volta da órbita osculadora em UA, para desenhar.
light_time_and_doppler	função	(range_km, range_rate_km_s, rest_wavelength_nm=656.281)	Tempo de luz de ida e desvio Doppler relativístico de uma linha espectral.

pinneapple_physics.astro.phenomena

Fenômenos de almanaque: nascer/trânsito/ocaso, crepúsculos, fase da Lua, magnitudes, elongações, oposições, equinócios, eclipses e trânsitos de Mercúrio e Vênus. [sem teste direto]

pinneapple_physics/astro/phenomena.py · 435 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
find_roots	função	(f, jd0, jd1, step, tol=0.5 * SECOND)	Todas as trocas de sinal de uma função de data num intervalo, refinadas até a tolerância.
find_minima	função	(f, jd0, jd1, step, tol=1.0 * SECOND)	Mínimos locais de uma função de data, refinados por seção áurea.
horizon_altitude	função	(body, distance_km=None) -> float	Altitude geométrica do centro do corpo no nascer/ocaso (refração padrão mais semidiâmetro).
rise_transit_set	função	(body, observer, jd_start, days=1.0, eph=None)	Nascer, trânsito e ocaso de um corpo do Sistema Solar num intervalo.
twilight_events	função	(observer, jd_start, days=1.0, eph=None)	Nascer/pôr do Sol e início/fim dos crepúsculos civil, náutico e astronômico.
sky_state	função	(sun_alt_deg) -> str	Classifica o céu pela altitude do Sol: dia, crepúsculo civil, náutico, astronômico ou noite.
phase_angle	função	(body, t, eph=None)	Ângulo Sol-corpo-Terra e distância Sol-corpo.
illuminated_fraction	função	(body, t, eph=None)	Fração iluminada do disco a partir do ângulo de fase.
moon_phase	função	(t, eph=None) -> dict	Fração iluminada, ângulo de fase, elongação, longitude de fase e ângulo do limbo brilhante da Lua.
magnitude	função	(body, t, eph=None)	Magnitude visual do Sol, Lua e planetas (Saturno sem o termo dos anéis).
planet_events	função	(body, jd0, jd1, eph=None)	Oposições, conjunções e, para Mercúrio e Vênus, maiores elongações num intervalo.
lunar_phase_events	função	(jd0, jd1, eph=None)	Datas de lua nova, quarto crescente, cheia e quarto minguante.
seasons	função	(jd0, jd1, eph=None)	Equinócios e solstícios num intervalo.
lunar_eclipses	função	(jd0, jd1, eph=None)	Eclipses lunares com tipo e magnitudes umbral e penumbral.
solar_eclipses	função	(jd0, jd1, observer=None, eph=None)	Eclipses solares (critério geocêntrico) com tipo aproximado e circunstâncias locais opcionais.
planetary_transits	função	(jd0, jd1, eph=None)	Trânsitos de Mercúrio e Vênus pelo disco do Sol (geocêntricos).

pinneapple_physics.astro.satellites

Satélites terrestres a partir de TLEs públicos (CelesTrak): propagação SGP4 (pacote sgp4), TEME para referencial terrestre, ângulos de visada, sombra da Terra, magnitude e passagens visíveis. [sem teste direto]

pinneapple_physics/astro/satellites.py · 246 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
Satellite	classe	(name, line1, line2)	Satélite definido por um TLE; dá magnitude padrão e estados TEME e ITRS via SGP4.
gmst82	função	(t)	GMST pela expressão IAU 1982 usada pelo referencial TEME do SGP4.
parse_tle	função	(text) -> list[Satellite]	Lê texto de TLEs (com ou sem linha de nome) e devolve uma lista de Satellite.
fetch_tles	função	(group='stations', cache_dir=None, max_age_hours=6.0, timeout=15.0) -> tuple[list[Satelli…)	Baixa os TLEs de um grupo do CelesTrak com cache em disco; informa erro quando não há rede nem cache.
sun_itrs	função	(t, eph=None)	Posição geocêntrica do Sol no referencial terrestre.
look	função	(sat, observer, t, eph=None) -> dict	Altitude, azimute, distância, ponto subsatélite, iluminação e magnitude, vetorizado no tempo.
magnitude	função	(std_mag, range_km, phase_rad, sunlit=1.0)	Brilho de esfera difusa em função da distância e do ângulo de fase.
find_passes	função	(sat, observer, t0, days=1.0, min_alt=10.0, step_s=20.0, eph=None) -> list[dict]	Passagens acima do horizonte num intervalo, com visibilidade e magnitude máxima.

pinneapple_physics.astro.smallbodies

Asteroides e cometas: elementos do JPL Small-Body Database, propagador de alta precisão no campo do Sol, planetas e Lua (DE421, com termo relativístico), aproximações à Terra e validação.

pinneapple_physics/astro/smallbodies.py · 222 linhas · testes: 16

nome	tipo	assinatura	o que faz
SmallBody	classe	(name, epoch_jd, a_au, e, i_deg, node_deg, peri_deg, M_deg, kind, neo, pha, H, diameter_k…)	Asteroide ou cometa com elementos orbitais, classificação (NEO, PHA), magnitude H e dados de aproximação.
Trajectory	classe	(body, jd0, jd1, eph=None, rtol=1e-11, gr=True)	Uma integração DOP853 adaptativa com saída densa; posições em qualquer instante custam só uma interpolação.
load_body	função	(key) -> SmallBody	Carrega um corpo empacotado a partir do JSON do JPL SBDB.
available_bodies	função	()	Chaves dos corpos empacotados, NEOs primeiro.
propagate	função	(body, jd_out, eph=None, rtol=1e-11, gr=True)	Posições e velocidades heliocêntricas ICRF nas datas TDB pedidas.
close_approaches	função	(body, jd0, jd1, target='earth', max_dist_au=0.2, step_days=0.25, eph=None, traj=None)	Mínimos de distância a um alvo (Terra por padrão), refinados por seção áurea.
horizons_vectors	função	(path=None)	Lê uma tabela VECTORS do JPL Horizons (CSV).
cad_reference	função	(body, year)	Registro de aproximação do JPL para um ano, com a faixa de 3 sigma.
validation_summary	função	(eph=None) -> dict	Propagador contra referências independentes: Ceres vs Horizons e a passagem de Apophis em 2029 vs a tabela do JPL.

pinneapple_physics.astro.spaceweather

Clima espacial para observadores: previsão de Kp com incerteza calibrada (rede de quantis conformalizada) e probabilidade de ver aurora.

pinneapple_physics/astro/spaceweather.py · 193 linhas · testes: 16

nome	tipo	assinatura	o que faz
load_history	função	()	Carrega o histórico empacotado de Kp e F10.7.
geomagnetic_latitude	função	(lat_deg, lon_deg)	Latitude geomagnética de dipolo centrado (IGRF-13 2020).
oval_boundary	função	(kp)	Latitude geomagnética do limite equatorial da aurora visível para um Kp (tabela NOAA SWPC).
kp_needed	função	(maglat_deg, horizon_margin_deg=0.0)	Kp necessário para ver aurora numa latitude geomagnética.
build_xy	função	(kp, f107, horizon)	Monta entradas e alvos de Kp para um horizonte de previsão.
train_forecaster	função	(horizon=1, seed=0, epochs=300)	Treina a rede de quantis de Kp, conformalizada e avaliada nos dois últimos meses.
forecast	função	(model, kp_hist, f107_now)	Previsão (baixo, mediana, alto) de Kp a partir do histórico recente.
aurora_probability	função	(lo, med, hi, maglat_deg)	Probabilidade de Kp atingir o necessário, a partir dos quantis da previsão.
fetch_live_kp	função	(timeout=10.0)	Baixa os últimos dias de Kp planetário da NOAA SWPC; levanta erro sem rede.

pinneapple_physics.astro.spk

Leitor pequeno de kernels JPL SPK (arquivos DAF, segmentos tipo 2 e 3 de Chebyshev), mapeado em memória.

pinneapple_physics/astro/spk.py · 124 linhas · testes: 16

nome	tipo	assinatura	o que faz
Segment	classe	(target, center, frame, kind, start_et, end_et, init, intlen, rsize, n, coeffs)	Segmento de um kernel SPK (coeficientes de Chebyshev) que avalia estado num tempo.
SPK	classe	(path)	Kernel SPK mapeado em memória; encadeia segmentos por centros comuns para dar o estado alvo-centro.

pinneapple_physics.astro.stars

Catálogo de estrelas brilhantes (Yale BSC5) e astrofísica estelar: distâncias, magnitude absoluta, temperatura pela cor, correção bolométrica, luminosidade, raio, espectros de Planck e cores de corpo negro.

pinneapple_physics/astro/stars.py · 198 linhas · testes: 23

nome	tipo	assinatura	o que faz
StarCatalog	classe	(hr, name, desig, con, ra, dec, pmra, pmdec, v, bv, sp, plx, plx_dyn, rv)	Catálogo de estrelas em arrays: HR, nome, designação, posição, movimento próprio, magnitude, cor, tipo espectral, paralaxe.
load_bright_stars	função	() -> StarCatalog	Carrega o catálogo BSC5 empacotado como StarCatalog.
load_constellations	função	() -> list	Figuras das constelações da IAU com chave, nome latino, nomes por idioma e ponto de rótulo.
constellation_names	função	(lang=None) -> dict	Nomes das constelações por chave em latim (en) e português do Brasil (pt).
planck_lambda	função	(wavelength_m, temperature_k)	Radiância espectral de Planck B_lambda.
wien_peak_m	função	(temperature_k)	Comprimento de onda de pico de Wien (m) para uma temperatura.
blackbody_rgb	função	(temperature_k)	Cor sRGB de um corpo negro (reusa o ajuste do renderizador de buraco negro).
blackbody_spectrum	função	(temperature_k, n=300, lam_min_nm=100.0, lam_max_nm=3000.0)	Espectro de corpo negro com pico, fluxo total e cor RGB.
teff_from_bv	função	(bv)	Temperatura efetiva a partir de B-V (Ballesteros 2012).
bolometric_correction	função	(teff)	Correção bolométrica BC_V de corpo negro, com zero no Sol.
star_properties	função	(cat, i, extinction_av=0.0) -> dict	Distância, magnitude absoluta, temperatura, luminosidade e raio de uma estrela do catálogo.
hr_diagram	função	(cat, max_rel_parallax_error=None, min_parallax_mas=5.0)	Pontos (B-V, M_V) do diagrama HR para estrelas com paralaxe utilizável.

pinneapple_physics.astro.stars_ml

Aprendizado de máquina sobre o catálogo de estrelas brilhantes (mistura gaussiana e k-NN no diagrama HR), conferido contra a classificação espectral do próprio catálogo.

pinneapple_physics/astro/stars_ml.py · 154 linhas · testes: 16

nome	tipo	assinatura	o que faz
luminosity_group	função	(sp)	Extrai a classe de luminosidade de um tipo espectral (None se não houver).
spectral_class	função	(sp)	Índice da classe espectral OBAFGKM de um tipo espectral.
dataset	função	(cat=None, min_parallax_mas=5.0)	Dados do catálogo com paralaxe boa: B-V, M_V, grupo de luminosidade e classe espectral.
gmm_fit	função	(X, k=3, iters=200, seed=0)	Mistura gaussiana por EM com covariâncias completas.
cross_validate	função	(X, y, k=15, folds=5, seed=0)	Validação cruzada de um classificador k-NN sobre os dados padronizados.

nome	tipo	assinatura	o que faz
`analyse`	função	`(seed=0) -> dict`	Roda a análise: agrupamentos GMM no diagrama HR nomeados pela classe de luminosidade majoritária e sua pureza.

`pinneapple_physics.astro.surrogate`

Substituto neural informado por física (PINN) para propagação de órbitas, conferido contra o integrador clássico Yoshida-4, com limiares de confiança.

`pinneapple_physics/astro/surrogate.py` · 268 linhas · testes: 32

nome	tipo	assinatura	o que faz
`reference_trajectory`	função	`(body, jd_tdb, days, n=400, eph=None)`	Posições e velocidades heliocêntricas de referência pela integração de N corpos Yoshida-4.
`train_orbit_pinn`	função	`(body='mars', jd_tdb=2461327.0, days=365.0, **kwargs) -> dict`	Treina o PINN substituto de órbita (com trava para uso em threads) e devolve o relatório.

`pinneapple_physics.astro.timescales`

Escalas de tempo para astronomia posicional: UTC, TAI, TT, TDB, UT1 e tempo sideral; a classe `Time` guarda as datas julianas derivadas.

`pinneapple_physics/astro/timescales.py` · 216 linhas · testes: 9

nome	tipo	assinatura	o que faz
`Time`	classe	`(utc, dut1)`	Instante (ou vetor de instantes) em UTC com as escalas derivadas como datas julianas.
`tai_minus_utc`	função	`(jd_utc)`	TAI-UTC em segundos (NaN antes de 1972).
`delta_t_model`	função	`(year)`	TT - UT1 pelos polinômios de Espenak & Meeus (2006); usado antes de 1972 e como reserva.
`dut1`	função	`(jd_utc)`	UT1-UTC numa data, de uma tabela IERS finals2000A se houver (zero sem tabela).
`tdb_minus_tt`	função	`(jd_tt)`	TDB - TT pela aproximação periódica de 2 termos.
`jd_from_datetime`	função	`(dt) -> float`	Data juliana a partir de um `datetime` (UTC se sem fuso).
`datetime_from_jd`	função	`(jd) -> datetime`	`datetime` UTC a partir de uma data juliana.
`julian_centuries`	função	`(jd)`	Séculos julianos desde J2000.
`era`	função	`(jd_ut1)`	Ângulo de rotação da Terra (IAU 2000).
`gmst_rad`	função	`(t)`	Tempo sideral médio de Greenwich (IAU 2006).
`gast_rad`	função	`(t)`	Tempo sideral aparente de Greenwich (GMST mais equação dos equinócios).
`gast_hours`	função	`(t)`	Tempo sideral aparente de Greenwich em horas.

`pinneapple_physics.astro.validation`

Verificações de precisão do observatório contra referências independentes (NOVAS, Horizons, almanaque USNO, eclipses NASA, passagens da ISS, skyfield), devolvidas como linhas de dados.

`pinneapple_physics/astro/validation.py` · 198 linhas · testes: 16

nome	tipo	assinatura	o que faz
`check_novas`	função	`()`	Compara posições aparentes com casos gravados do USNO NOVAS.
`check_horizons`	função	`()`	Compara posições com casos gravados do JPL Horizons.

nome	tipo	assinatura	o que faz
`check_usno_almanac`	função	`()`	Compara nascer/pôr do Sol, fases e estações com tabelas do USNO.
`check_eclipses`	função	`()`	Compara horários de eclipses e trânsitos com o catálogo da NASA.
`check_iss_passes`	função	`()`	Compara passagens previstas da ISS com uma referência gravada.
`check_skyfield_live`	função	`(n=24, seed=1)`	Compara alt/az de todos os corpos com o skyfield em épocas e observadores aleatórios (opcional).
`check_integrators`	função	`()`	Confere conservação de energia e momento angular do integrador de N corpos e a diferença para o DE421.
`run_all`	função	`(include_skyfield=True) -> list[dict]`	Roda todas as verificações e devolve as linhas de resultado.

`pinneapple_physics.blackhole`

Acreção em buraco negro: simulações hidrodinâmicas e previsão do 'tempo do buraco negro' com horizonte de confiança, reproduzindo e estendendo Duarte, Nemmen & Navarro (2022); reexporta `AccretionFlow`, `RIAFConfig`, `bondi_pw`, `torus_state`.

`pinneapple_physics/blackhole/__init__.py` · 20 linhas · testes: 19

`pinneapple_physics.blackhole.capture`

Grava uma cena Twin3D em quadros PNG/GIF com Chromium headless (Playwright, dependência opcional), para docs e posts. [sem teste direto]

`pinneapple_physics/blackhole/capture.py` · 79 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`capture_twin`	função	`(folder, steps, out_dir=None, *, field='', cmap='turbo', view='', zoom=1.0, size=(1280, 7…)`	Tira screenshots da cena exportada numa pasta em cada passo de tempo e devolve os caminhos PNG.

`pinneapple_physics.blackhole.convergence`

Estudo de resolução das simulações de acreção: remapeamento conservativo entre grades, estatísticas por janela de tempo e ordem observada, para saber se os dados de treino do previsor estão convergidos.

`pinneapple_physics/blackhole/convergence.py` · 262 linhas · testes: 19

nome	tipo	assinatura	o que faz
`window_mask`	função	`(grid) -> np.ndarray`	Células de uma grade cujo centro cai na janela física de previsão.
`cell_volumes`	função	`(grid) -> np.ndarray`	Volume 2 pi r^2 dr d(cos theta) de cada célula.
`remap`	função	`(a, faces_f, faces_c) -> np.ndarray`	Remapeamento conservativo (ponderado por volume) de médias de célula entre duas grades esféricas do mesmo domínio.
`remap_r`	função	`(a, rf_f, rf_c) -> np.ndarray`	Remapeamento de um perfil radial por sobreposição em log r.
`psd_slope`	função	`(x, dt, f_lo, f_hi) -> float`	Inclinação log-log do espectro de Welch entre duas frequências.
`run_statistics`	função	`(folder, t0, t1, log_rho_range=LOG_RHO_RANGE) -> dict`	Estatísticas de uma rodada numa janela de tempo: séries de escalares e médias temporais de perfis e campos.
`resolution_study`	função	`(runs, *, t0=2000.0, t1=10000.0, log=print) -> dict`	Estudo de resolução sobre rodadas em várias grades e sementes, com razão de refinamento constante.

`pinneapple_physics.blackhole.dataset`

Dataset do 'tempo do buraco negro': quadros de densidade de trajetórias simuladas inteiras, quantizados em uint16 e empacotados em npz determinístico para o model zoo.

pinneapple_physics/blackhole/dataset.py · 170 linhas · testes: 19

nome	tipo	assinatura	o que faz
encode	função	(log10_rho) -> np.ndarray	Log10 da densidade para uint16; levanta erro fora da faixa de quantização.
decode	função	(q) -> np.ndarray	Coluna uint16 de volta para log10 da densidade (float32).
deterministic_npz	função	(arrays) -> bytes	npz com datas e ordem fixas no zip, para o SHA-256 não mudar entre builds iguais.
build_arrays	função	(runs, conv=None, *, splits=SPLITS, t0=T0, t1=T1) -> dict[str, np.ndarray]	Monta os arrays do dataset do zoo a partir das pastas de simulação e do relatório de convergência.
write_npz	função	(arrays, path) -> Path	Grava todos os splits num único arquivo npz.
write_split_files	função	(arrays, folder) -> dict[str, Path]	Grava um npz por split, para baixar só o split usado e manter arquivos pequenos.
load_trajectories	função	(split, path=None) -> dict[int, dict]	Carrega as trajetórias inteiras de um split, por semente, com quadros em ordem de tempo.
window_volumes	função	(path=None) -> np.ndarray	Volumes das células da janela de previsão.

pinneapple_physics.blackhole.figures

Figuras do estudo: habilidade contra tempo de antecedência e estudo de resolução, com matplotlib. [sem teste direto]

pinneapple_physics/blackhole/figures.py · 121 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
plot_skill	função	(report, path, model=None, title='') -> str	Figura de erro (MAE) contra antecedência para modelo e baselines, e da habilidade contra persistência com intervalo e horizonte.
plot_convergence	função	(report, path) -> str	Figura de perfis médios de densidade e momento angular por grade, e da taxa média de acreção com intervalo.

pinneapple_physics.blackhole.forecast

Previsão do 'tempo do buraco negro': a U-Net de Duarte et al. (2022), treino com perda de rollout, ruído e penalidade de massa, rollouts, horizonte de confiança e envelope de massa.

pinneapple_physics/blackhole/forecast.py · 473 linhas · testes: 19

nome	tipo	assinatura	o que faz
DensityCodec	classe	(log_min, log_max, r_cells, theta_trim)	Normalização logarítmica e recorte compartilhados entre treino e inferência (salvos com o checkpoint).
DuarteUNet	classe	(filters=32, frames=5, residual=False, padding='zeros')	A U-Net de Duarte et al. (2022) que prevê o próximo bloco de quadros de densidade.
TrainConfig	classe	(filters, frames, residual, padding, epochs, batch_size, lr, alpha, clip, loss, rollout_b…)	Configuração de treino: arquitetura, épocas, lote, taxa, perda, blocos de rollout, ruído, pesos de massa, bf16, semente e tempo máximo.
MassEnvelope	classe	(rate_max, margin)	Limite aprendido de quão rápido a massa na janela de previsão pode mudar; aponta violações.
MassProjection	classe	(codec, cell_volume, envelope, frame_dt)	Projeta quadros previstos no orçamento de massa visto no treino, reescalando quadros que ganham ou perdem massa demais.
make_blocks	função	(x, k=5, stride=1, out_blocks=1) -> tuple[np.ndarray, np.ndarray]	Transforma quadros (T, H, W) em pares entrada/alvo de blocos de k quadros, com alvos para rollout de vários passos.
duarte_loss	função	(pred, target, alpha=8.0)	MAE na janela mais alfa vezes MAE nos pixels de alta densidade.
duarte_regional_loss	função	(pred, target, regions=None)	MSE na janela mais MAE ponderado nas quatro regiões do artigo.
train_forecaster	função	(X, Y, Xv, Yv, cfg, codec, out_dir, *, log=print, cell_volume=None, init=None) -> dict[st…)	Treina com Adam e a perda de Duarte, guardando os pesos com melhor perda de validação.

nome	tipo	assinatura	o que faz
`train_on_trajectories`	função	`(train, val, cfg, out_dir, *, cell_volume=None, val_blocks=1, val_stride=1, init=None, lo…)`	Treina a partir de trajetórias inteiras, sem blocos cruzando trajetórias e com validação em trajetórias separadas.
`evaluate_loss`	função	`(model, X, Y, cfg, batch=32) -> float`	Perda da previsão desenrolada sobre todos os blocos alvo.
`save_forecaster`	função	`(path, model, cfg, codec, meta) -> None`	Salva pesos, configuração, codec e metadados num checkpoint torch.
`load_forecaster`	função	`(path) -> tuple[DuarteUNet, DensityCodec, dict]`	Carrega um checkpoint e devolve modelo em modo eval, codec e conteúdo do checkpoint.
`rollout`	função	`(model, x0, n_blocks, *, clip=False, mass_projection=None) -> np.ndarray`	Previsão iterativa: cada bloco previsto volta como entrada, opcionalmente com projeção de massa.
`lead_time_scores`	função	`(pred, truth, last_input, mean_state, weights=None) -> dict[str, np.ndarray]`	Métricas por antecedência: MAE do modelo, da persistência e da média temporal, correlação de anomalia e de tendência.
`trust_horizon`	função	`(scores, frame_dt, acc_min=0.6, block=1) -> dict[str, float]`	Antecedência até a qual a previsão é útil (correlação de anomalia acima do limiar e erro abaixo das referências).
`window_mass`	função	`(rho, cell_volume) -> np.ndarray`	Massa na janela para quadros de densidade e volumes de célula.

`pinneapple_physics.blackhole.hydro`

Hidrodinâmica viscosa de acreção em torno de um buraco negro de Schwarzschild (potencial de Paczynski-Wiita), axissimétrica em (r, theta) com v_phi, em PyTorch; inclui toro de equilíbrio e solução de Bondi. [sem teste direto]

`pinneapple_physics/blackhole/hydro.py` · 506 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`RIAFConfig`	classe	`(nr, ntheta, r_in, r_out, gamma, alpha, viscosity, cfl, rho_floor, p_floor, v_max, t_cap,…)`	Configuração da simulação: grade, raios, gama, viscosidade alfa, CFL, pisos, contorno externo, backend, parâmetros do toro e perturbação.
`AccretionFlow`	classe	`(cfg)`	Fluxo de acreção viscoso 2.5D (axissimétrico com v_phi); conversões primitivas/conservadas, passo de tempo, balanços e `run` que salva snapshots.
`torus_state`	função	`(flow) -> np.ndarray`	Estado inicial de toro em equilíbrio com momento angular em lei de potência, imerso numa atmosfera.
`bondi_pw`	função	`(r, *, cs_inf, gamma=1.4, rho_inf=1.0)`	Solução exata de acreção transônica de Bondi no potencial de Paczynski-Wiita, por busca de raiz.

`pinneapple_physics.blackhole.hydro_numba`

Kernels Numba para `AccretionFlow` quando `RIAFConfig(backend="numba")`; requer o pacote `numba`. [sem teste direto]

`pinneapple_physics/blackhole/hydro_numba.py` · 321 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`NumbaStepper`	classe	`(flow)`	Guarda a geometria de um `AccretionFlow` e avança o estado com os kernels Numba.
`rhs`	função	`(W, r, th, rf, thf, area_r, area_t, vol, inv_r, geo_t, Rc, gamma, alpha, visc, dU, fluxes)`	Kernel Numba: variação dU nas células internas e fluxos de massa e momento angular nos contornos.
`prim_to_cons`	função	`(W, Rc, gamma, U)`	Kernel Numba: primitivas internas para variáveis conservadas.
`cons_to_prim`	função	`(U, W, Rc, r, gamma, rho_floor, p_floor, v_max, t_cap)`	Kernel Numba: conservadas para primitivas, com pisos e limites; devolve quantas células foram limitadas.
`boundaries`	função	`(W, outer, has_outer)`	Kernel Numba que preenche as células fantasmas: cópia nos contornos radiais (sem entrada de massa onde não há contorno externo dado) e reflexão nos polos com troca de sinal de v_theta.

nome	tipo	assinatura	o que faz
time_step	função	(W, r, dr, dth, gamma, alpha, visc, cfl)	Kernel Numba que calcula o passo de tempo pelos limites hidrodinâmico e viscoso (CFL).

pinneapple_physics.blackhole.raytrace

Traçado de raios relativístico de um fluxo de acreção em torno de buraco negro de Schwarzschild (vista estilo 'Interstellar'), com campo de estrelas, bloom e renderização com Doppler.

pinneapple_physics/blackhole/raytrace.py · 316 linhas · testes: 19

nome	tipo	assinatura	o que faz
RayPaths	classe	(ray, ir, ith, ds, kdotv, lapse, captured, escape_dir, shape)	Onde cada raio cruza a região emissora, em ordem câmera-longe, com dados para emissão e captura.
Camera	classe	(width, height, r_cam, inclination_deg, fov_deg, roll_deg, dphi, ds_in, ds_out, max_phi)	Câmera em torno do buraco negro: resolução, distância, inclinação, campo de visão; traça os raios.
blackbody_rgb	função	(T) -> np.ndarray	Cor sRGB aproximada de um corpo negro (ajuste de Tanner Helland).
star_field	função	(width=4096, height=2048, n=9000, seed=3) -> np.ndarray	Mapa de estrelas equiretangular determinístico com faixa tênue tipo Via Láctea.
bloom	função	(img, strength=0.35, sigma=6.0) -> np.ndarray	Brilho cinematográfico: soma uma cópia borrada das partes brilhantes.
render	função	(paths, rho, temperature, velocity=None, *, doppler=True, kappa=0.0, exposure=1.0, scale=…)	Renderiza um quadro a partir dos caminhos dos raios, densidade, temperatura e velocidade (Doppler opcional).

pinneapple_physics.blackhole.runs

Gera, retoma e lê as simulações de acreção usadas pelo previsor (issue #399).

pinneapple_physics/blackhole/runs.py · 89 linhas · testes: 19

nome	tipo	assinatura	o que faz
load_run	função	(folder) -> dict	Lê uma rodada: tempos, quadros, taxa de acreção, grade e configuração.
simulate_run	função	(cfg, folder, *, t_end=10000.0, every=10.0, chunk=50, name=None, log=print) -> str	Roda (ou retoma) uma simulação numa pasta, salvando em blocos com checkpoint.

pinneapple_physics.blackhole.skill

Habilidade do previsor contra persistência, climatologia e DMD, com intervalos bootstrap e verificações físicas de massa e fluxo.

pinneapple_physics/blackhole/skill.py · 178 linhas · testes: 19

nome	tipo	assinatura	o que faz
DMDBaseline	classe	(trajectories, rank)	Baseline DMD de posto r (via pinneapple_neural), iterada a partir de um quadro, mantendo congelada a parte fora do subespaço.
window_mass	função	(x, codec, vol) -> np.ndarray	Massa na janela para quadros normalizados.
implied_inflow	função	(x, codec, vol, dt, shells) -> np.ndarray	Fluxo líquido de massa para dentro pelas cascas, inferido da variação da massa contida entre quadros.
evaluate_skill	função	(models, train, val, test, *, frame_dt, cell_volume, blocks=20, start_every=5, block_M=10…)	Avalia modelos contra persistência, climatologia e DMD em vários postos, por antecedência, com verificações físicas.

pinneapple_physics.blackhole.twin

Vista 3D de gêmeo digital de um fluxo de acreção axissimétrico com o PINNeAPPle-Twin3D: painéis em corte e esferas.

pinneapple_physics/blackhole/twin.py · 120 linhas · testes: 19

nome	tipo	assinatura	o que faz
CutawayMesh	classe	(r_faces, theta_faces, *, r_view, phi_cut_deg=270.0, n_phi=72, offset=(0.0, 0.0, 0.0))	Geometria de um painel em corte e o mapa dos campos de célula (nr, ntheta) para seus vértices.
sphere	função	(radius, center=(0, 0, 0), n=24) -> tuple[np.ndarray, np.ndarray]	Malha de esfera (vértices e faces) de raio e centro dados.
accretion_scene	função	(grid, panels, times, *, field='log10_density', unit='', r_view=60.0, gap=0.35, title='Bl…)	Cena Twin3D com um painel em corte por campo, dispostos lado a lado, ao longo do tempo.

pinneapple_physics.closed_form

Subpacote de modelos físicos em forma fechada (sem EDP), portados de ferramentas de produtos para serem compartilhados e testados; o `__init__` importa só parte dos módulos (não importa `eddy_coil`, `electromagnetics`, `heat_exchanger`, `pipe_insulation`, `pump_npsh`, `thin_airfoil`).

pinneapple_physics/closed_form/__init__.py · 73 linhas · testes: 103

pinneapple_physics.closed_form.burgers

Solução exata da equação de Burgers viscosa (Cole-Hopf) e invíscida (características) com condição inicial `-sin(pi x)` em [-1, 1].

pinneapple_physics/closed_form/burgers.py · 73 linhas · testes: 80

nome	tipo	assinatura	o que faz
burgers_sine_exact	função	(x, t, nu, n_nodes=120) -> np.ndarray	Calcula `u(x, t)` da Burgers viscosa com condição inicial senoidal por quadratura (Cole-Hopf).
burgers_sine_inviscid_exact	função	(x, t) -> np.ndarray	Calcula a solução de entropia da Burgers invíscida com condição inicial senoidal.

pinneapple_physics.closed_form.cantilever_fatigue

Tensão de flexão de viga em balanço e vida em fadiga pela relação de Basquin (equações de livro-texto, Shigley e Dowling).

pinneapple_physics/closed_form/cantilever_fatigue.py · 73 linhas · testes: 32

nome	tipo	assinatura	o que faz
stress_from_moment_pa	função	(width_m, thickness_m, moment_n_m) -> float	Tensão `sigma = M c / I` numa seção retangular sólida; base comum das outras funções de tensão.
cantilever_root_bending_stress_pa	função	(width_m, thickness_m, load_n, length_m) -> float	Tensão de flexão na raiz engastada da viga em balanço, onde o momento é máximo (`M = F L`).
cantilever_bending_stress_at_position_pa	função	(width_m, thickness_m, load_n, length_m, x_from_root_m) -> float	Perfil de tensão de flexão ao longo da mesma viga, numa posição medida a partir da raiz.
basquin_fatigue_life_cycles	função	(stress_amplitude_pa, fatigue_strength_coeff_pa, fatigue_strength_exponent) -> float	Ciclos até a falha pela equação de Basquin, dada a amplitude de tensão e as constantes de fadiga do material.
basquin_allowable_stress_pa	função	(target_life_cycles, fatigue_strength_coeff_pa, fatigue_strength_exponent) -> float	Basquin invertida: maior amplitude de tensão que ainda atinge a vida alvo em ciclos.

pinneapple_physics.closed_form.compressible

Escoamento compressível 1-D de gás caloricamente perfeito: razões isentrópicas, choque normal, pressão de Pitot e `Cp_max` newtoniano modificado.

pinneapple_physics/closed_form/compressible.py · 78 linhas · testes: 15

nome	tipo	assinatura	o que faz
isentropic_ratios	função	(M, gamma=1.4) -> dict[str, np.ndarray]	Razões de estagnação/estática (T, p, rho) e razão de áreas A/A* num número de Mach (NACA 1135).
normal_shock	função	(M, gamma=1.4) -> dict[str, np.ndarray]	Saltos através de um choque normal para Mach a montante M >= 1 (NACA 1135).
pitot_pressure_ratio	função	(M, gamma=1.4)	Razão entre a pressão de Pitot e a pressão estática do escoamento livre.
cp_max_newtonian	função	(M, gamma=1.4)	Coeficiente de pressão no ponto de estagnação da teoria newtoniana modificada (Lees 1955).

pinneapple_physics.closed_form.eddy_coil

Impedância de uma bobina de correntes parasitas com núcleo de ar sobre uma placa condutora (Dodd & Deeds, 1968).

pinneapple_physics/closed_form/eddy_coil.py · 69 linhas · testes: 27

nome	tipo	assinatura	o que faz
skin_depth	função	(f, sigma, mu_r=1.0) -> float	Profundidade de penetração 1 / sqrt(pi f mu sigma).
coil_impedance	função	(f, *, r1, r2, l1, l2, turns, sigma, thickness=math.inf) -> dict	Impedância da bobina no ar, sua variação sobre a placa, impedância normalizada e profundidade de penetração.

pinneapple_physics.closed_form.electromagnetics

Soluções fechadas dos presets eletromagnéticos, registradas como solução "analytic" de pp.solve.

pinneapple_physics/closed_form/electromagnetics.py · 125 linhas · testes: 41

nome	tipo	assinatura	o que faz
two_dielectric_capacitor	função	(y, V, d, d1, eps1, eps2)	Potencial num capacitor de placas paralelas com dois dielétricos (Griffiths, ex. 4.7).
wire_az	função	(x, y, a, J, nu=1.0)	Potencial vetor A_z de um condutor redondo com densidade de corrente uniforme (lei de Ampère).
tm_cavity_mode	função	(x, y, t, m, n, Lx, Ly, eps=1.0, mu=1.0)	Onda estacionária TM_mn numa cavidade retangular PEC (campos Ez, Hx, Hy).
skin_effect_slab	função	(x, delta, A0=1.0)	Fasor do campo que se difunde num semiespaço condutor (efeito pelicular).
cylinder_tm_total	função	(x, y, k0, radius, eps_r)	Campo Ez total de uma onda plana espalhada por um cilindro dielétrico (série de funções cilíndricas, via pinneapple_simulation).

pinneapple_physics.closed_form.fin_array_conduction

Modelo fechado de condução e convecção em conjunto de aletas (Incropera), generalizado para qualquer arranjo de superfícies estendidas.

pinneapple_physics/closed_form/fin_array_conduction.py · 125 linhas · testes: 20

nome	tipo	assinatura	o que faz
fin_array_profile_params	função	(k_material_w_mk, h_conv_w_m2k, n_fins, fin_thickness_m, fin_length_m, transverse_extent_…)	Calcula uma vez as grandezas intermediárias do modelo de aletas, compartilhadas pelo valor escalar e pelo perfil espacial.
fin_array_base_temperature_c	função	(params, t_ambient_c, power_w) -> float	Temperatura na base do conjunto de aletas (verdade de referência escalar).
fin_array_temperature_at_position_c	função	(params, t_ambient_c, x_from_root_m, corrected_length_m=None) -> float	Perfil de temperatura ao longo da aleta pela mesma equação 1-D, com correção de ponta adiabática.

pinneapple_physics.closed_form.heat

Condução transiente 1-D numa placa (série de Fourier) e aleta de pino com ponta convectiva (Incropera).

pinneapple_physics/closed_form/heat.py · 79 linhas · testes: 17

nome	tipo	assinatura	o que faz
slab_transient	função	(x, t, L, alpha, T_left, T_right, T0, n_terms=None) -> np.ndarray	Temperatura T(x, t) numa placa com temperaturas impostas nas faces e temperatura inicial uniforme.
pin_fin_theta	função	(xi, h, k, D, L)	Excesso de temperatura adimensional ao longo de uma aleta de pino com ponta convectiva.
pin_fin_heat	função	(h, k, D, L, T_base, T_inf) -> float	Calor dissipado [W] por uma aleta de pino com ponta convectiva.

pinneapple_physics.closed_form.heat_exchanger

Dimensionamento de trocadores de calor: efetividade-NTU, fator de correção da LMTD e coeficientes do lado dos tubos e do casco.

pinneapple_physics/closed_form/heat_exchanger.py · 155 linhas · testes: 41

nome	tipo	assinatura	o que faz
effectiveness	função	(ntu, cr, arrangement='counterflow', n_shell=1) -> float	Efetividade epsilon(NTU, C_r) para contracorrente, paralelo, casco-tubo ou fluxo cruzado.
ntu_from_effectiveness	função	(eps, cr, arrangement='counterflow') -> float	Inverte effectiveness por bisseção para obter o NTU.
lmtd_correction_1_2	função	(t_hot_in, t_hot_out, t_cold_in, t_cold_out) -> float	Fator F de correção da LMTD de um trocador 1-2 (Bowman et al., 1940).
gnielinski	função	(re, pr) -> dict[str, float]	Número de Nusselt do lado dos tubos (Gnielinski; laminar desenvolvido abaixo de Re = 2300).
hausen_mean_nusselt	função	(re, pr, d_over_l) -> float	Nusselt médio laminar com entrada térmica num tubo de parede isotérmica (Hausen, 1943).
mean_nusselt_from_temperatures	função	(re, pr, d_over_l, t_wall, t_in, t_out) -> float	Nusselt médio de um tubo isotérmico a partir das temperaturas de entrada e saída (balanço de energia).
kern_shell	função	(m_dot, mu, k, cp, *, shell_id, baffle_spacing, tube_od, pitch, layout='triangular') -> d…)	Coeficiente do lado do casco pelo método de Kern (área de fluxo cruzado, diâmetro equivalente, Re, Nu, h).
one_two_exchanger_ode	função	(ntu, cr, n=4001, profiles=False)	Efetividade de um trocador 1-2 integrando as três equações de energia por diferenças finitas (modelo de Nagle), sem a fórmula fechada.

pinneapple_physics.closed_form.helmholtz_resonator

Equação de sintonia de ressonador de Helmholtz (porta de caixa acústica bass-reflex).

pinneapple_physics/closed_form/helmholtz_resonator.py · 50 linhas · testes: 4

nome	tipo	assinatura	o que faz
helmholtz_resonant_frequency_hz	função	(port_radius_m, port_length_m, cabinet_volume_l, end_correction_factor=1.7) -> float	Frequência de ressonância para uma porta de raio e comprimento dados num volume de caixa, com fator de correção de extremidade.
port_volume_m3	função	(port_radius_m, port_length_m) -> float	Volume de ar da porta, usado como restrição ou objetivo de projeto.

pinneapple_physics.closed_form.membrane_diffusion

Transporte difusivo controlado por membrana (primeira lei de Fick), como em adesivos transdérmicos de reservatório.

pinneapple_physics/closed_form/membrane_diffusion.py · 148 linhas · testes: 6

nome	tipo	assinatura	o que faz
`steady_state_release_rate_mg_day`	função	`(diffusion_coeff_m2_s, partition_coeff, reservoir_conc_kg_m3, area_m2, membrane_thickness…)`	Taxa de liberação em regime permanente (mg/dia) através da membrana limitante.
`depletion_time_days`	função	`(diffusion_coeff_m2_s, partition_coeff, membrane_thickness_m, reservoir_thickness_m) -> f…)`	Tempo de esgotamento do reservatório; depende só das espessuras e da permeabilidade, não da área nem da concentração.
`concentration_at_membrane_depth`	função	`(reservoir_conc_kg_m3, partition_coeff, membrane_thickness_m, z_from_reservoir_m) -> float`	Perfil linear de concentração dentro da membrana em regime permanente.
`post_depletion_decay_rate_constant_per_day`	função	`(diffusion_coeff_m2_s, partition_coeff, membrane_thickness_m, reservoir_thickness_m) -> f…)`	Constante de decaimento de primeira ordem depois que o excesso de fármaco do reservatório acaba.
`simulate_release_timeseries`	função	`(diffusion_coeff_m2_s, partition_coeff, reservoir_conc_kg_m3, area_m2, membrane_thickness…)`	Série temporal de liberação e esgotamento combinando as fórmulas do módulo.

`pinneapple_physics.closed_form.particle_drag`

Coeficiente de arrasto de uma esfera rígida em escoamento uniforme por correlações empíricas.

`pinneapple_physics/closed_form/particle_drag.py` · 32 linhas · testes: 14

nome	tipo	assinatura	o que faz
`clift_sphere_cd`	função	`(re) -> float`	Arrasto da esfera por Clift, Grace & Weber (1978), faixa 20 < Re <= 260 (avisa fora dela).
`schiller_naumann_cd`	função	`(re) -> float`	Arrasto da esfera por Schiller & Naumann (1933), Re < 800 (avisa acima).

`pinneapple_physics.closed_form.pcb_thermal`

Blocos térmicos de placas de circuito: condutividade efetiva do empilhamento, vias térmicas, convecção e radiação nas faces, e dados de encapsulamentos para o modelo de dois resistores JEDEC.

`pinneapple_physics/closed_form/pcb_thermal.py` · 179 linhas · testes: 16

nome	tipo	assinatura	o que faz
`Layer`	classe	`(name, kind, thickness_m, coverage)`	Camada do empilhamento (cobre ou dielétrico) com espessura e cobertura de cobre; dá `k_xy` e `k_z` pela regra da fração de área.
`Environment`	classe	`(t_ambient_c, air_velocity_m_s, orientation, emissivity, extra)`	Condições do ambiente da placa: temperatura, velocidade do ar (0 = convecção natural), orientação e emissividade.
`stackup`	função	`(n_copper=4, thickness_mm=1.6, outer_oz=1.0, inner_oz=1.0, outer_coverage=0.3, inner_cove…)`	Monta um empilhamento simétrico de camadas de sinal, planos e dielétrico.
`effective_conductivity`	função	`(layers) -> Dict[str, float]`	Condutividade efetiva da placa: regra paralela no plano e regra em série na espessura.
`via_copper_fraction`	função	`(n_vias, drill_mm, plating_um, area_m2) -> float`	Fração de área de cobre dos barris de vias metalizadas (não preenchidas) sob uma área.
`convection_coefficients`	função	`(width_m, depth_m, t_surface_c, env) -> Dict[str, float]`	Coeficientes h das faces superior e inferior (convecção natural ou forçada mais radiação linearizada).

`pinneapple_physics.closed_form.pipe_flow`

Escoamento desenvolvido em tubos e canais: fatores de atrito de Darcy, perda de carga e altura da primeira célula para um y+ alvo.

`pinneapple_physics/closed_form/pipe_flow.py` · 169 linhas · testes: 29

nome	tipo	assinatura	o que faz
`laminar`	função	`(Re) -> float`	Fator de atrito laminar `64 / Re` (avisa para Re >= 2300).
`blasius`	função	`(Re) -> float`	Correlação de Blasius para tubo liso (avisa fora de 4e3 a 1e5).
`colebrook`	função	`(Re, rel_rough=0.0, tol=1e-14, max_iter=100) -> float`	Fator de atrito de Darcy pela equação implícita de Colebrook, resolvida por iteração.
`haaland`	função	`(Re, rel_rough=0.0) -> float`	Aproximação explícita de Haaland para a equação de Colebrook.
`dean_number`	função	`(Re, D, Rc) -> float`	Número de Dean de um tubo curvo.
`curved_pipe_ratio`	função	`(De) -> float`	Razão entre o atrito laminar de tubo curvo e de tubo reto (White, 1929).
`friction_factor`	função	`(Re, rel_rough=0.0) -> float`	Escolhe `laminar` abaixo de Re = 2300 e `colebrook` acima (sem aviso na faixa de transição).
`darcy_pressure_drop`	função	`(f, L, D, rho, U) -> float`	Perda de pressão de Darcy-Weisbach num duto.
`parallel_plates_pressure_drop`	função	`(mu, L, h, U) -> float`	Perda de pressão laminar entre placas paralelas infinitas (Poiseuille plano).
`first_cell_height`	função	`(y_plus, Re, D, U, nu, f=None, rel_rough=0.0) -> float`	Altura da primeira célula junto à parede para atingir um y+ desejado.

pinneapple_physics.closed_form.pipe_insulation

Perda de calor em tubo isolado e espessura econômica de isolamento (método ISO 12241 / VDI 2055).

`pinneapple_physics/closed_form/pipe_insulation.py` · 151 linhas · testes: 27

nome	tipo	assinatura	o que faz
`air`	função	`(t_c) -> dict[str, float]`	Propriedades do ar (nu, k, Pr, beta) interpoladas de tabela numa temperatura.
`surface_coefficient`	função	`(ts, ta, d, eps, wind=0.0) -> dict[str, float]`	Coeficiente externo de um cilindro horizontal no ar: radiação mais convecção mista.
`heat_loss`	função	`(d_pipe, thickness, t_in, t_amb, *, k0, k1=0.0, eps=0.9, wind=0.0) -> dict[str, float]`	Perda de calor por metro, temperatura de superfície e coeficientes de um tubo isolado ou nu.
`radial_fd`	função	`(d_pipe, thickness, t_in, t_surface, *, k0, k1=0.0, n=400) -> dict[str, np.ndarray]`	Perfil de temperatura no isolamento por volumes finitos radiais com k(T) e iterações de Picard.
`crf`	função	`(rate, years) -> float`	Fator de recuperação de capital.
`economic_thickness`	função	`(d_pipe, t_in, t_amb, thicknesses, *, k0, k1, eps, wind, hours, energy_price, efficiency,…)`	Custo anual (energia + isolamento anualizado) por espessura e escolha da espessura econômica.

pinneapple_physics.closed_form.plate_fin_heatsink

Modelo térmico e hidráulico de dissipador de aletas planas com correlações publicadas (Teertstra, Bar-Cohen, Lee, Muzychka); os resultados trazem avisos quando uma correlação sai da faixa.

`pinneapple_physics/closed_form/plate_fin_heatsink.py` · 309 linhas · testes: 18

nome	tipo	assinatura	o que faz
`HeatSinkGeometry`	classe	`(base_width, base_depth, base_thickness, n_fins, fin_thickness, fin_height, material)`	Geometria do dissipador em metros (base, número, espessura e altura das aletas, material), com `fin_gap` e `mass_kg`.
`OperatingPoint`	classe	`(power_w, t_ambient_c, air_velocity_m_s, source_width, source_depth, tim_resistance_k_m2_…)`	Ponto de operação: potência, temperatura ambiente, velocidade do ar, tamanho da fonte e resistência da interface térmica.
`air_properties`	função	`(t_c, p_pa=101325.0) -> Dict[str, float]`	Propriedades do ar a 1 atm interpoladas da tabela de Incropera, com densidade de gás ideal.
`geometry_errors`	função	`(g, op) -> List[str]`	Lista os erros de geometria ou de ponto de operação (material desconhecido, dimensões inválidas).
`forced_convection`	função	`(g, v_approach, air) -> Dict[str, float]`	Convecção forçada em canais de placas paralelas (Teertstra) e perda de pressão.

nome	tipo	assinatura	o que faz
`natural_convection`	função	`(g, dt_k, air) -> Dict[str, float]`	Convecção natural entre placas verticais isotérmicas (Bar-Cohen & Rohsenow).
`spreading_resistance`	função	`(k, tb, source_area, plate_area, r_conv) -> Dict[str, float]`	Resistência de espalhamento da base (Lee et al., 1995), incluindo condução 1-D pela base.
`evaluate`	função	`(g, op, max_iter=60, h_scale=1.0, t_root_guess_c=None) -> Dict[str, object]`	Resolve a rede de resistências térmicas e devolve temperatura da fonte, resistências e avisos.

`pinneapple_physics.closed_form.pump_npsh`

Bomba centrífuga num sistema de sucção e recalque: ponto de operação, NPSH disponível contra requerido e margem de cavitação.

`pinneapple_physics/closed_form/pump_npsh.py` · 94 linhas · testes: 27

nome	tipo	assinatura	o que faz
`iapws97_psat`	função	`(t_c) -> float`	Pressão de saturação da água pela IAPWS-IF97.
`water_density`	função	`(t_c) -> dict[str, float]`	Densidade e viscosidade da água interpoladas de tabela numa temperatura.
`atmosphere`	função	`(z_m) -> float`	Pressão atmosférica ISO 2533 numa altitude (troposfera).
`pipe_loss`	função	`(q, d, length, k_minor, rough, rho, mu) -> dict[str, float]`	Perda de carga de um tubo com perdas localizadas numa vazão.
`fit_quadratic`	função	`(q, y) -> np.ndarray`	Ajuste por mínimos quadrados de uma curva quadrática na vazão a pontos de catálogo.
`duty_point`	função	`(head_coef, h_static, system_loss, q_max) -> float`	Vazão onde a altura da bomba iguala a do sistema (bisseção).
`npsh_available`	função	`(p_surface, t_c, z_suction, h_loss) -> dict[str, float]`	NPSH disponível no flange de sucção.

`pinneapple_physics.closed_form.rocket_nozzle`

Bocal de foguete ideal: escoamento isentrópico 1-D de gás caloricamente perfeito num bocal convergente-divergente (Sutton & Biblarz).

`pinneapple_physics/closed_form/rocket_nozzle.py` · 114 linhas · testes: 9

nome	tipo	assinatura	o que faz
`area_ratio`	função	`(M, gamma) -> float`	Razão de áreas `A/A*` num número de Mach.
`mach_from_area_ratio`	função	`(ratio, gamma, *, supersonic=True, tol=1e-12, max_iter=300) -> float`	Inverte `A/A*` por bisseção, no ramo supersônico (padrão) ou subsônico.
`pressure_ratio`	função	`(M, gamma) -> float`	Razão entre pressão estática e de estagnação num Mach.
`temperature_ratio`	função	`(M, gamma) -> float`	Razão entre temperatura estática e de estagnação num Mach.
`choked_mass_flow`	função	`(p0, throat_area, gamma, R_gas, T0) -> float`	Vazão mássica numa garganta bloqueada.
`characteristic_velocity`	função	`(gamma, R_gas, T0) -> float`	Velocidade característica c*, que só depende do gás e da temperatura da câmara.
`thrust_coefficient`	função	`(p_exit, p0, p_atm, ratio, gamma) -> float`	Coeficiente de empuxo (termo de quantidade de movimento mais termo de pressão).
`specific_impulse`	função	`(cf, c_star, g0=G0) -> float`	Impulso específico `Isp = C_F c* / g0`.
`nozzle_performance`	função	`(*, gamma, R_gas, T_chamber, p_chamber, area_ratio_exit, p_atm=101325.0) -> dict`	Estado de saída e desempenho de um bocal ideal expandido até uma razão de áreas.

pinneapple_physics.closed_form.thin_airfoil

Aerodinâmica 2-D fechada: sustentação de linha média NACA de 4 dígitos pela teoria de aerofólio fino e arrasto de perfil por correlação.

pinneapple_physics/closed_form/thin_airfoil.py · 61 linhas · testes: 4

nome	tipo	assinatura	o que faz
naca4_camber_slope	função	(x, m, p) -> np.ndarray	Inclinação dz/dx da linha média NACA de 4 dígitos.
thin_airfoil_cl	função	(m, p, alpha_deg, n_quad=4001) -> float	Coeficiente de sustentação pela teoria de aerofólio fino, por quadratura trapezoidal.
turbulent_flat_plate_cf	função	(reynolds) -> float	Coeficiente médio de atrito de placa plana totalmente turbulenta.
hoerner_profile_cd	função	(t_c, reynolds) -> float	Arrasto de perfil de uma seção aerodinâmica com fator de forma de Hoerner.

pinneapple_physics.codegen

Pacote que reexporta os geradores de scripts FDM e FEniCS; os scripts gerados expõem solve(params) -> dict e rodam sem importar a biblioteca. [sem teste direto do __init__]

pinneapple_physics/codegen/__init__.py · 51 linhas · **sem teste direto**

pinneapple_physics.codegen.fdm_script_generator

Gera um script NumPy de diferenças finitas explícitas para três formas curadas (calor 1D, Burgers 1D, Poisson/Laplace 2D); formas fora disso levantam NotImplementedError.

pinneapple_physics/codegen/fdm_script_generator.py · 548 linhas · testes: 13 · palavras-chave no código (heurística; o texto acima diz o status real): levanta NotImplementedError em algum caminho

nome	tipo	assinatura	o que faz
PDEParser	classe	()	Classifica um ProblemSpec (ou objeto equivalente) num dos três modelos FDM suportados, levantando NotImplementedError fora deles.
FDMScriptGenerator	classe	(parser=None) -> None	Renderiza o texto de um script FDM independente e executável (generate) ou grava em disco (write).

pinneapple_physics.codegen.fenics_script_generator

Gera scripts UFL/dolfinx para difusão linear, Stokes (Taylor-Hood) ou elasticidade linear com contato por penalidade, escolhidos por uma heurística de 3 vias.

pinneapple_physics/codegen/fenics_script_generator.py · 614 linhas · testes: 13

nome	tipo	assinatura	o que faz
FEniCSScriptGenerator	classe	()	Renderiza scripts dolfinx independentes para difusão, Stokes ou elasticidade conforme a forma física do problema.
classify_physics_shape	função	(spec) -> str	Heurística que devolve "diffusion", "stokes" ou "elasticity" a partir de pde.kind e dos campos; o padrão é difusão.

pinneapple_physics.compile_api

pp.compile: memoriza derivadas de primeira ordem dentro de uma avaliação da perda, para cada campo ser derivado uma vez por passo; as perdas são as mesmas. Fusão de operadores e planejamento de memória não são feitos.

pinneapple_physics/compile_api.py · 137 linhas · testes: 5

nome	tipo	assinatura	o que faz
`Benchmark`	classe	`(problem, epochs, repeats, baseline_s, compiled_s, speedup, max_param_diff, max_param_abs…)`	Resultado de `CompiledProblem.benchmark`: tempos base e compilado, speedup, diferença dos pesos e chamadas de derivada economizadas.
`CompiledProblem`	classe	`(problem, optimize='physics') -> None`	Problema com as otimizações decididas; seus `solve` e `loss_fn` usam o cache de derivadas.
`compile`	função	`(problem, optimize='physics', **preset_kwargs) -> CompiledProblem`	Prepara um `PhysicalProblem`, `ProblemSpec` ou nome de preset para resolver mais rápido.

pinneapple_physics.conservation

Conservação como restrição dura: projeta a saída de um modelo (aditiva ou multiplicativamente) para que a integral ponderada iguale um total prescrito.

`pinneapple_physics/conservation.py` · 83 linhas · testes: 6

nome	tipo	assinatura	o que faz
`ConservationProjection`	classe	`(model, weights, mode='additive', target_fn=None)`	Módulo torch que aplica um modelo e projeta a saída para conservar exatamente o total dado por `target_fn`.
`integral`	função	`(field, weights, dims=1) -> Tensor`	Soma ponderada nas últimas `dims` dimensões de um campo.
`project_integral`	função	`(field, weights, target, mode='additive', dims=None, eps=1e-30) -> Tensor`	Corrige um campo (modo aditivo ou multiplicativo) para que sua integral ponderada iguale o alvo.

pinneapple_physics.darcy_checks

Verificações físicas de campos de Darcy 2-D (`-div(a grad u) = f`, `u = 0` no contorno) sem solução de referência, na mesma discretização do solver de referência; alimenta `pinneapple check` (`physics = "darcy"`) e o relatório de confiança.

`pinneapple_physics/darcy_checks.py` · 857 linhas · testes: 26

nome	tipo	assinatura	o que faz
`CoefficientEnvelope`	classe	`(a_min, a_max, ranges, rel_tol)`	Envelope dos coeficientes de treino (faixa de `a` e de estatísticas do campo) para dizer se uma entrada está fora da distribuição.
`DarcyCheck`	classe	`(name, kind, values, value, limit, passed, critical, detail)`	Uma verificação sobre um lote: valores por amostra, agregado, limite, aprovado/reprovado e se é crítica.
`UseProfile`	classe	`(name, use, limits, reasons)`	Perfil de uso de uma previsão com os limites físicos que esse uso exige e o motivo de cada um.
`darcy_operator_apply`	função	`(u, a) -> np.ndarray`	Aplica o operador de volumes finitos de `-div(a grad u)` a `u`.
`darcy_residual`	função	`(u, a, f=1.0) -> np.ndarray`	Resíduo por célula `f - A_a u`; zero para a solução do próprio solver.
`residual_error_bound`	função	`(u, a, f=1.0) -> np.ndarray`	Limite certificado do erro relativo na norma de energia, só a partir do resíduo.
`residual_l2_bound`	função	`(u, a, f=1.0) -> np.ndarray`	Limite certificado do erro relativo em L2, só a partir do resíduo.
`boundary_flux_balance`	função	`(u, a, f=1.0) -> dict[str, np.ndarray]`	Fluxo líquido pelo contorno comparado com a integral da fonte (conservação discreta).
`boundary_values`	função	`(u) -> np.ndarray`	Valores de `u` nos quatro lados, extrapolados das duas células vizinhas.
`coefficient_stats`	função	`(a) -> dict[str, np.ndarray]`	Estatísticas por amostra de um campo de coeficientes, usadas na verificação de aplicabilidade.
`darcy_checks`	função	`(a, u, f=1.0, *, reference=None, envelope=None, tolerances=None, quantile=0.9, physics_ki…)`	Roda todas as verificações físicas (e, opcionalmente, envelope e comparação com referência) num lote de previsões.
`physics_metrics`	função	`(a, u, f=1.0) -> dict[str, np.ndarray]`	Valores por amostra das métricas físicas (`energy_bound`, `l2_bound`, `bc`, `flux`, `positivity`).
`calibrate_tolerances`	função	`(a, u, f=1.0, *, quantile=0.99, floor=None) -> dict[str, float]`	Calcula limites a partir das próprias previsões do modelo em entradas de validação dentro da distribuição.

nome	tipo	assinatura	o que faz
profile_verdicts	função	(a, u, f=1.0, *, profiles=None, quantile=0.9) -> dict[str, Any]	Para cada perfil de uso, compara o quantil de cada métrica com o limite e decide APPROVED ou REJECT.
sample_checks	função	(a, u, f=1.0, *, tolerances=None, envelope=None) -> list[dict[str, Any]]	Verificações de uma única previsão no formato que pinneapple check usa.
sample_verdict	função	(checks) -> str	PASS se todas as verificações passam, senão FAIL (status de saída de pinneapple check).
estimated_error	função	(u, a, f=1.0) -> np.ndarray	Mapa do erro de u estimado só pelo resíduo.
flux_magnitude	função	(u, a) -> np.ndarray	Módulo do fluxo |a grad u| por célula a partir dos fluxos de face do solver.
estimated_flux_error	função	(u, a, f=1.0) -> np.ndarray	Mapa do erro estimado do fluxo, relativo ao maior fluxo.
region_masks	função	(n, wall_cells=2, interior_from=8) -> dict[str, np.ndarray]	Máscaras booleanas de região de parede e de interior numa grade n x n.
regional_errors	função	(u, a, f=1.0, *, wall_cells=2, interior_from=8) -> dict[str, np.ndarray]	Erro relativo estimado sem referência nas regiões de parede e interior.
use_verdicts	função	(a, u, f=1.0, *, envelope=None, wall_cells=2, interior_from=8) -> dict[str, dict[str, Any…)	Para uma previsão, quais usos ela atende e por quê, só pelo resíduo e pelo envelope.
trust_report	função	(checks, *, model='', kinds=None)	Decisão de implantação (TrustReport: APPROVED / REVIEW / REJECT) de uma lista de DarcyCheck.
darcy_trust_report	função	(a, u, f=1.0, *, model='', kinds=…, **kwargs)	Roda darcy_checks e devolve (TrustReport, checks).
save_darcy_npz	função	(path, a, u, **extra)	Grava um campo de Darcy no formato de grade que pinneapple check lê.
check_config_entry	função	(name, path, *, f=1.0, tolerances=None, envelope=None, coefficient='a', solution='u') -> …)	Gera uma tabela [[result]] de pinneapple-check.toml para uma previsão de Darcy salva.
nodal_darcy_residual	função	(u, k, f=1.0, dx=1.0, *, cropped=1, cross_denominator=None) -> np.ndarray	Resíduo da forma nodal (diferenças finitas, layout Darcy2D do PhysicsNeMo) nos nós com vizinhos conhecidos.
nodal_boundary_values	função	(u, *, cropped=1) -> np.ndarray	Valores de contorno extrapolados até os nós fantasmas nulos, no layout nodal.
nodal_flux_balance	função	(u, k, f=1.0, dx=1.0, *, cropped=1) -> np.ndarray	Desequilíbrio relativo de um balanço de volume de controle em torno dos nós armazenados.
solve_nodal_darcy	função	(k, f=1.0, dx=1.0, *, cross_denominator=None) -> np.ndarray	Resolve diretamente a forma nodal; serve de referência para nodal_darcy_residual.
check_nodal_darcy_batch	função	(k, u, f=1.0, dx=1.0, *, cropped=1, alt_cross_denominators=None, tolerances=None, floor_f…)	Verifica um lote de pares de treino nodais (k, u) antes de alguém treinar com eles.

pinneapple_physics.deepxde_bridge

Pacote da ponte DeepXDE ↔ PINNeAPPle; reexporta solve_with_deepxde e utilitários do adapter.

pinneapple_physics/deepxde_bridge/__init__.py · 60 linhas · testes: 9

pinneapple_physics.deepxde_bridge.adapter

Traduz ProblemSpec em geometria, dados, rede FNN e dde.Model reais do DeepXDE e chama o treino do DeepXDE; só burgers e poisson são traduzidos, o resto levanta NotImplementedError. DeepXDE é dependência opcional. [sem teste direto]

pinneapple_physics/deepxde_bridge/adapter.py · 468 linhas · **sem teste direto** · palavras-chave no código (heurística; o texto acima diz o status real): levanta NotImplementedError em algum caminho

nome	tipo	assinatura	o que faz
DeepXDESolveResult	classe	(deepxde_model, spec, final_loss, train_time_s)	Resultado de solve_with_deepxde: o modelo DeepXDE, o spec, a perda final e o tempo de treino.
require_deepxde	função	() -> Any	Importa deepxde ou levanta erro claro se não estiver instalado.

nome	tipo	assinatura	o que faz
`is_deepxde_available`	função	`() -> bool`	Diz se o `deepxde` pode ser importado.
`solve_with_deepxde`	função	`(spec, *, n_domain=2000, n_boundary=200, n_initial=100, layers=(50, 50, 50, 50), epochs=1…)`	Traduz o `spec` num problema DeepXDE e o treina de verdade com os hiperparâmetros dados.

pinneapple_physics.ensemble

Ensemble adaptativo de especialistas físicos: pesos atualizados online por erro contra referência, resíduo físico e custo, nos modos `select` ou `combine`, com intervalos conformais e detecção de suspeitos.

`pinneapple_physics/ensemble.py` · 636 linhas · testes: 68

nome	tipo	assinatura	o que faz
`Expert`	classe	`(name, predict, cost, kind)`	Um especialista do ensemble: nome, função `predict(query)`, custo por chamada e tipo.
`EnsembleRun`	classe	`(names, errors, ensemble_errors, weights, active, coverage, costs, predictions, suspect)`	Histórico de uma execução do ensemble (erros, pesos, ativo, cobertura, custos, suspeitos) com `summary` e `switches`.
`PhysicsEnsemble`	classe	`(experts, mode='combine', etas=(0.5, 2.0, 8.0), alphas=(0.0, 0.01, 0.05), point_weights=N…)`	Seleção ou combinação online de modelos físicos com pesos Fixed-Share/AdaHedge, resíduo físico, custo e intervalos conformais.
`relative_l2`	função	`(pred, ref, weights=None) -> float`	Erro L2 relativo, com pesos por ponto opcionais.
`from_callable`	função	`(name, fn, cost=None, kind='numerical') -> Expert`	Cria um `Expert` a partir de uma função qualquer.
`from_solution`	função	`(name, solution, points, cost=None, shape=None) -> Expert`	Cria um `Expert` a partir de um resultado de `pp.solve`, avaliado nos pontos de cada consulta.
`from_torch`	função	`(name, model, to_input, from_output=None, cost=None, kind='neural') -> Expert`	Cria um `Expert` a partir de um `torch.nn.Module` (FNO, DeepONet, MeshGraphNet, PINN) com adaptadores de entrada e saída.
`residual_baseline`	função	`(experts, domain_queries, residual_fn) -> dict[str, float]`	Resíduo físico mediano de cada especialista em casos do seu próprio domínio, referência para `PhysicsEnsemble(residual_baseline=...)`.
`fit_static_weights`	função	`(predictions, references, n_folds=5, point_weights=None, seed=0) -> dict[str, Any]`	Ensemble offline: pesos convexos que minimizam o erro quadrático num conjunto de validação, com validação cruzada.

pinneapple_physics.ensemble_viz

Anima com matplotlib uma `EnsembleRun`: previsão vs. referência caso a caso, modelo escolhido e evolução dos pesos, gravando um GIF.

`pinneapple_physics/ensemble_viz.py` · 131 linhas · testes: 3

nome	tipo	assinatura	o que faz
`animate_ensemble`	função	`(run, references, path='ensemble.gif', *, x=None, times=None, frames_per_case=4, fps=10, …)`	Grava um GIF da execução do ensemble e devolve o caminho.

pinneapple_physics.experiment

`Experiment` fixa as sementes (Python, NumPy, PyTorch determinístico), resolve, pontua contra uma referência e grava registro JSON com fingerprint, versões e máquina. [sem teste direto pelo inventário; testado via `pp.Experiment`]

`pinneapple_physics/experiment.py` · 174 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ExperimentResult`	classe	`(config, problem_fingerprint, solution, metrics, wall_time_s, started_at, environment, n_…)`	Registro de uma execução: config, fingerprint, solução, métricas, tempo, ambiente; salva e carrega em JSON.
`Experiment`	classe	`(problem, method, options, seed, reference, reference_options, n_eval, metric_names, name…)`	Método aplicado a um problema com semente, opções e referência opcional; `run` executa de forma reprodutível.

nome	tipo	assinatura	o que faz
`run_all`	função	`(experiments) -> Dict[str, ExperimentResult]`	Roda vários experimentos e devolve os resultados por nome.

pinneapple_physics.maxwell_checks

Verificações das equações de Maxwell sobre campos numa grade (de PINN, FNO, solver ou medição), sem referência: resíduos relativos com teste em 2h, balanço de Poynting, contorno PEC, reciprocidade e passividade; alimenta `pinneapple check` (`physics = "maxwell"`).

`pinneapple_physics/maxwell_checks.py` · 549 linhas · testes: 51

nome	tipo	assinatura	o que faz
`MaxwellCheck`	classe	`(name, key, value, limit, passed, critical, detail, kind)`	Resultado de uma verificação: nome, chave, valor, limite, aprovado, criticidade e detalhe.
`FieldSet`	classe	`(grid, fields, *, eps=EPS0, mu=MU0, sigma=0.0, omega=None, interior=1)`	Campos numa grade tensorial com derivadas por diferenças centrais, máscara de interior e versão engrossada.
`relative_residual`	função	`(terms, mask=None, floor=0.0) -> float`	Resíduo relativo `||soma dos termos|| / soma das normas`; NaN quando a lei não restringe o campo.
`gauss_electric`	função	`(grid, fields, *, eps=EPS0, tol=None, **kw) -> MaxwellCheck`	Verifica a lei de Gauss elétrica `div D - rho`.
`gauss_magnetic`	função	`(grid, fields, *, mu=MU0, tol=None, **kw) -> MaxwellCheck`	Verifica a lei de Gauss magnética `div B`.
`faraday`	função	`(grid, fields, *, mu=MU0, omega=None, tol=None, **kw) -> MaxwellCheck`	Verifica a lei de Faraday `curl E + dB/dt`.
`ampere_maxwell`	função	`(grid, fields, *, eps=EPS0, mu=MU0, sigma=0.0, omega=None, tol=None, **kw) -> MaxwellCheck`	Verifica a lei de Ampère-Maxwell `curl H - J - dD/dt`.
`continuity`	função	`(grid, fields, *, omega=None, tol=None, **kw) -> MaxwellCheck`	Verifica a equação de continuidade da carga `d rho/dt + div J`.
`poynting_balance`	função	`(grid, fields, *, eps=EPS0, mu=MU0, sigma=0.0, omega=None, tol=None) -> MaxwellCheck`	Verifica o teorema de Poynting sobre a caixa da grade.
`tangential_continuity`	função	`(F1, F2, normal, *, surface_current=None, tol=None, name='E') -> MaxwellCheck`	Verifica `n x (F1 - F2) = K` em pontos de interface.
`pec_boundary`	função	`(grid, fields, faces, *, tol=None) -> MaxwellCheck`	Verifica `n x E = 0` nas faces declaradas PEC.
`reciprocity`	função	`(S, *, tol=None) -> MaxwellCheck`	Verifica `S = S^T` de uma rede recíproca.
`passivity`	função	`(S, *, lossless=False, tol=None) -> MaxwellCheck`	Verifica passividade (maior valor singular de S no máximo 1) ou unitariedade se sem perdas.
`maxwell_checks`	função	`(grid, fields, *, eps=EPS0, mu=MU0, sigma=0.0, omega=None, laws=None, pec_faces=(), refer…)`	Roda todas as verificações que os campos fornecidos permitem e, opcionalmente, compara com uma referência.
`maxwell_trust_report`	função	`(grid, fields, *, model='', **kwargs)`	Roda `maxwell_checks` e devolve `(TrustReport, checks)` com a decisão de implantação.
`sample_checks`	função	`(grid, fields, **kwargs) -> list[dict[str, Any]]`	Verificações no formato que `pinneapple check` usa.
`sample_verdict`	função	`(checks) -> str`	PASS ou FAIL a partir da lista de verificações.
`save_maxwell_npz`	função	`(path, grid, fields, **extra)`	Grava eixos e componentes de campo (fasores como pares `_re`/`_im`) no formato lido por `pinneapple check`.
`check_config_entry`	função	`(name, path, *, parameters=None, tolerances=None) -> str`	Gera um bloco `[[result]]` de `pinneapple-check.toml` para um campo salvo.

pinneapple_physics.metrics

`pp.metrics`: L2, L2 relativo, RMSE, MAE, erro máximo, L∞ relativo e R^2 por campo, mais resumo, métrica agregada para rankings e erro de rollout temporal.

`pinneapple_physics/metrics.py` · 182 linhas · testes: 36

nome	tipo	assinatura	o que faz
`l2`	função	`(pred, true)`	Norma euclidiana do erro por campo.
`relative_l2`	função	`(pred, true)`	Erro L2 relativo por campo (`nan` quando a referência é nula).
`rmse`	função	`(pred, true)`	Raiz do erro quadrático médio por campo.
`mae`	função	`(pred, true)`	Erro absoluto médio por campo.
`max_abs`	função	`(pred, true)`	Maior erro absoluto por campo (norma L∞ do erro).
`relative_linf`	função	`(pred, true)`	Erro máximo relativo ao máximo da referência, por campo.
`r2`	função	`(pred, true)`	Coeficiente de determinação por campo.
`per_field`	função	`(metric, pred, true, fields) -> dict[str, float]`	Aplica uma métrica e devolve `{campo: valor}`.
`summary`	função	`(pred, true, fields, names=…) -> dict[str, dict[str, float]]`	Várias métricas de uma vez no formato `{métrica: {campo: valor}}` das tabelas de benchmark.
`pooled`	função	`(pred, true) -> dict[str, float]`	Um número por métrica sobre todos os campos juntos, para rankings.
`rollout_relative_l2`	função	`(pred_seq, true_seq, *, aggregate='steps') -> float`	Erro L2 relativo de um rollout temporal (eixo 0 é o tempo), agregado por passos ou no total.

`pinneapple_physics.multiscale`

Subpacote de física multiescala para ensembles adaptativos (filtros, fechamentos, resíduos, consistência, seleção por escala); numpy puro, exceto `learned` (PyTorch).

`pinneapple_physics/multiscale/__init__.py` · 90 linhas · testes: 29

`pinneapple_physics.multiscale.closures`

Fechamentos de subfiltro para LES 2-D em forma de vorticidade (Smagorinsky, dinâmico, gradiente) e testes a priori, a posteriori e de saúde sem rótulo; a classe base interna levanta `NotImplementedError` em `nu_t`.

`pinneapple_physics/multiscale/closures.py` · 381 linhas · testes: 29 · palavras-chave no código (heurística; o texto acima diz o status real): levanta `NotImplementedError` em algum caminho

nome	tipo	assinatura	o que faz
`Smagorinsky`	classe	`(cs, delta, name)`	Fechamento de Smagorinsky `nu_t = (cs delta)^2 |S|`.
`DynamicSmagorinsky`	classe	`(delta, test_ratio, test_kind, name, last_c)`	Smagorinsky dinâmico com coeficiente calculado por filtro de teste (identidade de Germano, média no domínio).
`GradientModel`	classe	`(delta, coef, name)`	Modelo de gradiente (Clark) para a tensão de subfiltro.
`NoModel`	classe	`(name)`	Sem modelo de subfiltro (DNS sub-resolvida).
`AprioriReport`	classe	`(closure, filter, stress_correlation, tendency_correlation, transfer_true, transfer_model…)`	Resultado do teste a priori: correlações de tensão e tendência, transferência de energia e retroespalhamento.
`AposterioriReport`	classe	`(closure, times, spectrum_error, field_error, dissipation_les, dissipation_ref, stable, t…)`	Resultado do teste a posteriori: erros de espectro e campo, dissipação, estabilidade e crescimento de enstrofia.
`ClosureHealth`	classe	`(score, ok, reasons, enstrophy_ratio, tail_ratio, germano)`	Pontuação de saúde de um fechamento sem rótulos, com motivos e indicadores.
`strain`	função	`(w, solver) -> dict[str, np.ndarray]`	Gradientes de velocidade, componentes de deformação e `|S|` a partir da vorticidade.
`stress_tendency`	função	`(a, b, solver) -> np.ndarray`	Tendência de vorticidade `-curl(div tau)` de uma tensão desviadora.
`true_subfilter`	função	`(w_fine, filt, n_coarse=None, length=2 * np.pi) -> dict[str, np.ndarray]`	Termos exatos de subfiltro de um campo resolvido para um filtro dado.

nome	tipo	assinatura	o que faz
apriori_test	função	(w_fine, filt, closure, n_coarse, length=2 * np.pi) -> AprioriReport	Teste a priori: compara tensão e tendência do fechamento com os termos exatos de subfiltro.
spectrum_error	função	(w, w_ref, kmax, length=2 * np.pi, kmin=1) -> float	Diferença média de log10 E(k) entre dois espectros numa faixa de números de onda.
aposteriori_test	função	(les, w0_bar, ref_bar, times, dt=0.005, t_long=None) -> AposterioriReport	Teste a posteriori: roda a LES com o fechamento e compara com a referência filtrada ao longo do tempo.
closure_health	função	(w, w0, kc, length=2 * np.pi, *, forced=False, max_enstrophy_growth=1.05, tail_band=(0.05…)	Saúde de uma previsão LES sem rótulos (crescimento de enstrofia, acúmulo espectral, erro de Germano).
germano_error	função	(w, closure, solver, test_ratio=2.0, test_kind='sharp', delta=None) -> float	Erro do fechamento na identidade de Germano entre o filtro da grade e um filtro de teste.

pinneapple_physics.multiscale.comparison

Compara ensembles num fluxo de consultas multiescala: seleção cega à escala contra sensível à escala, com e sem verificações e modelos que ligam escalas; gera tabelas e figuras.

pinneapple_physics/multiscale/comparison.py · 439 linhas · testes: 29

nome	tipo	assinatura	o que faz
EnsembleConfig	classe	(aware, router, checks, experts, error_tolerance, cost_weight)	Configuração de um ensemble: sensível à escala ou não, roteador, verificações, especialistas, tolerância e peso de custo.
evaluate_experts	função	(path, queries, initial_states, experts, checks=None, n_out=None) -> dict[str, np.ndarray]	Roda cada especialista em cada consulta uma vez e guarda saídas e verificações (falhas viram NaN).
adapt_output	função	(raw, source, target, aware, length=2 * np.pi) -> np.ndarray	Adapta uma saída filtrada num filtro de origem ao filtro pedido, de forma cega ou sensível à escala.
raw_residual	função	(pred, nu, dt, length=2 * np.pi) -> float	Resíduo relativo da equação de vorticidade na escala da grade.
closed_multiscale_residual	função	(pred, nu, dt, filt, h, length=2 * np.pi) -> float	Resíduo da equação filtrada em duas larguras, com a tendência de subfiltro estimada pelo modelo de gradiente.
run_ensembles	função	(queries, references, table, profiles, configs, *, dt, h, length=2 * np.pi, cost_weight=0…)	Roda cada configuração no mesmo fluxo de consultas, reaproveitando as saídas já avaliadas.
summarise_runs	função	(runs, profiles, configs, widths, *, wrong_factor=1.5, wrong_margin=0.03, solver=None, or…)	Resume por configuração o erro médio, erro por largura, escolhas erradas e custo.
closure_flag_table	função	(tables, targets, names, kc, length=2 * np.pi) -> list[dict[str, Any]]	Tabela de como as verificações sem rótulo marcaram cada especialista e o erro quando marcado ou não.
markdown_tables	função	(rows, widths, flags=(), sc_rows=(), bench_rows=()) -> str	Gera as tabelas em Markdown dos resumos e estudos.
summary_figure	função	(rows, widths, path, sc_rows=(), colors=…, unit='h') -> Path	Figura de quatro painéis com erro por largura, erro contra custo, escolhas erradas e estudo de condicionamento.
frontier_figure	função	(points, path, baselines=(), colors=…, title=…) -> Path	Figura de fronteira custo-erro por configuração.

pinneapple_physics.multiscale.consistency

Consistência entre escalas: uma previsão fina e uma grossa do mesmo estado devem concordar depois de levadas à mesma escala. [sem teste direto]

pinneapple_physics/multiscale/consistency.py · 114 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
ConsistencyReport	classe	(passed, field_error, spectrum_error, integrals, failed, tolerance, reasons)	Resultado da verificação de consistência: aprovado, erros de campo e espectro, integrais e motivos.
cross_scale_consistency	função	(fine, coarse, coarse_filter, fine_filter=None, *, length=2 * np.pi, tol=0.05, integrals=…)	Filtra e restringe a previsão fina para a escala da grossa e compara campo, espectro e integrais.

pinneapple_physics.multiscale.filters

Filtros espaciais (caixa, gaussiano, espectral) em grades periódicas com funções de transferência exatas, restrição, prolongamento, refiltragem e erro de comutação.

`pinneapple_physics/multiscale/filters.py` · 271 linhas · testes: 29

nome	tipo	assinatura	o que faz
`Filter`	classe	`(kind, delta)`	Filtro de largura `delta` e tipo dado; `delta = 0` é a identidade.
`transfer`	função	`(kind, k, delta) -> np.ndarray`	Função de transferência 1-D de um filtro.
`wavenumbers`	função	`(n, length=2 * np.pi) -> np.ndarray`	Números de onda angulares de uma grade periódica, na ordem da FFT.
`apply_filter`	função	`(u, filt, length=2 * np.pi, axes=None) -> np.ndarray`	Filtra um campo periódico nos eixos dados.
`restrict`	função	`(u, n, axes=None) -> np.ndarray`	Restrição de Fourier para uma grade periódica mais grossa, preservando a média.
`prolong`	função	`(u, n, axes=None) -> np.ndarray`	Interpolação de Fourier (preenchimento com zeros) para uma grade mais fina.
`post_filter_transfer`	função	`(source, target, k, max_gain=2.0) -> np.ndarray`	Transferência do pós-filtro que leva de um filtro de origem a um alvo, com ganho limitado.
`realizability_mismatch`	função	`(source, target, kmax, *, slope=-2.0, n=4096, max_gain=2.0, kmin=1.0) -> float`	Fração do filtro alvo que não pode ser obtida por pós-filtragem de um campo já filtrado.
`refilter`	função	`(u, source, target, length=2 * np.pi, axes=None, max_gain=2.0) -> np.ndarray`	Converte um campo filtrado num filtro para outro filtro, até onde for possível.
`filter_variable_width`	função	`(u, kind, delta, length=2 * np.pi, axes=None, decimals=10) -> np.ndarray`	Filtro cuja largura varia no espaço.
`spectral_derivative`	função	`(u, axis, length=2 * np.pi, order=1) -> np.ndarray`	Derivada espectral exata do interpolante trigonométrico.
`commutation_error`	função	`(u, kind, delta, axis=-1, length=2 * np.pi, axes=None) -> np.ndarray`	Erro de comutação entre filtrar e derivar (não nulo para largura variável).
`low_modes`	função	`(kmax) -> list[tuple[int, int]]`	Lista dos modos 2-D de baixa frequência num corte quadrado, um de cada par `k, -k`.
`fourier_basis`	função	`(modes, n, length=2 * np.pi) -> np.ndarray`	Base real de cossenos e senos para os modos dados numa grade n x n.
`fourier_coefficients`	função	`(w, modes, length=2 * np.pi) -> np.ndarray`	Coeficientes de um campo periódico na base de Fourier.

pinneapple_physics.multiscale.learned

Modelos aprendidos que ligam escalas, em PyTorch: FNO condicionado pela largura do filtro, LES diferenciável, fechamento CNN treinado a posteriori e PINN de Fourier-Galerkin.

`pinneapple_physics/multiscale/learned.py` · 501 linhas · testes: 29

nome	tipo	assinatura	o que faz
`ScaleConditionedFNO2d`	classe	`(in_channels=1, out_channels=1, width=32, modes=12, layers=4, conditioned=True, delta_ref…)`	FNO periódico cujas camadas são moduladas pela largura do filtro; `conditioned=False` dá a versão cega à escala.
`TorchVorticityLES2D`	classe	`(n, length=2 * math.pi, dtype=torch.float64)`	Solver pseudo-espectral 2-D de vorticidade diferenciável (RK4 com fator integrante) com fechamento opcional.
`LearnedClosure`	classe	`(hidden=16, layers=3, kernel=5)`	Fechamento CNN periódico que prevê o fluxo de vorticidade de subfiltro.
`FourierGalerkinPINN`	classe	`(kmax=4, hidden=256, scale=5.0, t_ref=0.3, length=2 * math.pi)`	Rede informada pela física para as grandes escalas da vorticidade 2-D, treinada só no resíduo de Galerkin truncado, sem dados.
`torch_transfer`	função	`(kind, k, delta) -> torch.Tensor`	Função de transferência de filtro em torch.
`torch_filter`	função	`(x, kind, delta, length=2 * math.pi) -> torch.Tensor`	Filtra um tensor (B, C, n, n) com largura escalar ou por amostra.

nome	tipo	assinatura	o que faz
`torch_refilter`	função	`(x, kind, d_from, d_to, length=2 * math.pi, max_gain=2.0) -> torch.Tensor`	Leva um tensor filtrado numa largura para outra, por amostra.
`filtering_consistency_loss`	função	`(model, x, d_small, d_large, kind='gaussian', length=2 * math.pi) -> torch.Tensor`	Perda de consistência: filtrar a saída na largura pequena deve dar a saída na largura grande.
`fit_scale_conditioned`	função	`(model, inputs, targets, kind='gaussian', steps=2000, batch=16, lr=0.002, consistency_wei…)`	Treina um modelo condicionado por escala em várias larguras, com perda relativa mais consistência.
`train_closure_aposteriori`	função	`(closure, solver, w0, nu, reference, dt, save_steps, steps=300, batch=8, lr=0.001, seed=0…)`	Treina um fechamento a posteriori, rolando a LES e comparando com a referência filtrada e seus espectros.
`as_numpy_closure`	função	`(closure, n, length=2 * math.pi)`	Embrulha um fechamento aprendido para uso em `VorticityNS2D` (numpy, sem gradiente).
`periodic_cnn2d`	função	`(in_channels=2, out_channels=2, width=32, dilations=(1, 2, 4, 8, 4, 2, 1)) -> nn.Sequenti…)`	CNN 2-D dilatada com preenchimento circular e GELU.
`torus_graph`	função	`(n, offsets=(1, 2, 4)) -> tuple[torch.Tensor, torch.Tensor]`	Arestas de um grafo de grade periódica com vizinhos em várias distâncias.
`fit_supervised`	função	`(forward, params, X, Y, steps, seed=0, lr=0.002, batch=16, log='') -> list[float]`	Treino supervisionado com Adam e agenda cosseno no erro quadrático relativo.

`pinneapple_physics.multiscale.ns2d`

Navier-Stokes 2-D incompressível em forma de vorticidade no quadrado periódico, para DNS e LES, com espectro de energia e resíduo da equação de vorticidade.

`pinneapple_physics/multiscale/ns2d.py` · 218 linhas · testes: 45

nome	tipo	assinatura	o que faz
`VorticityNS2D`	classe	`(n, nu, length, forcing, drag, closure, dealias)`	Solver pseudo-espectral 2-D de vorticidade com forçamento, arrasto linear e fechamento opcional.
`velocity_from_vorticity`	função	`(w, length=2 * np.pi) -> tuple[np.ndarray, np.ndarray]`	Velocidade `(u, v)` de um campo de vorticidade periódico.
`energy_spectrum`	função	`(w, length=2 * np.pi, kmax=None) -> tuple[np.ndarray, np.ndarray]`	Espectro de energia cinética por cascas a partir da vorticidade.
`random_vorticity`	função	`(rng, n, k0=4.0, energy=0.5, length=2 * np.pi, batch=None) -> np.ndarray`	Vorticidade aleatória com espectro de pico em `k0` e energia total dada.
`exact_advection`	função	`(w, length=2 * np.pi) -> np.ndarray`	Termo `div(u w)` sem aliasing, calculado numa grade 3/2 mais fina.
`vorticity_residual`	função	`(w_prev, w_next, dt, nu, length=2 * np.pi, forcing=None, drag=0.0, closure_tendency=None)…)`	Resíduo da equação de vorticidade entre dois instantâneos (trapezoidal no tempo).

`pinneapple_physics.multiscale.residuals`

Resíduos físicos avaliados em várias larguras de filtro, separados em partes de subfiltro, comutação e modelo. [sem teste direto]

`pinneapple_physics/multiscale/residuals.py` · 120 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`MultiscaleResidual`	classe	`(filters, raw, subfilter, commutation, model)`	Curva de resíduo por largura de filtro, com exportação para dicionário e Markdown.
`multiscale_residuals`	função	`(state, residual_fn, filters, *, length=2 * np.pi, subfilter_fn=None, commutation_fn=None…)`	Calcula a curva de resíduo de qualquer equação a partir de funções de resíduo, subfiltro e comutação.
`ns2d_multiscale_residuals`	função	`(w_prev, w_next, dt, nu, filters, *, length=2 * np.pi, forcing=None, drag=0.0, subfilter=…)`	Resíduo multiescala da equação de vorticidade 2-D entre dois instantâneos.

pinneapple_physics.multiscale.scales

Seleção de modelos sensível à escala: cada modelo declara as escalas que resolve e cada consulta as que precisa. [sem teste direto]

pinneapple_physics/multiscale/scales.py · 245 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ScaleProfile`	classe	`(name, filter, grid_spacing, closure, re_range, time_step, fidelity, cost, notes)`	Escalas que um modelo representa: filtro da saída, espaçamento da grade, fechamento, faixa de Reynolds, passo de tempo, fidelidade e custo.
`ScaleQuery`	classe	`(filter, re, time_scale, kmax)`	Escalas que uma consulta pede: filtro da saída, Reynolds, escala de tempo e número de onda máximo.
`ScaleCheck`	classe	`(name, score, covered, mismatch, reasons)`	Resultado da compatibilidade de escala de um modelo com uma consulta.
`ScaleRoute`	classe	`(chosen, weights, checks, awake, rung, escalated, reasons)`	Decisão de roteamento: modelo escolhido, pesos, verificações, especialistas ativos e escalonamento.
`ScaleAwareRouter`	classe	`(profiles, query_fn=None, *, strength=1.0, floor=0.0001, abstain_below=0.05, ladder=True,…)`	Roteador que pondera especialistas pela compatibilidade de escala, com escada de fidelidade (surrogate, LES, solver resolvido).
`scale_compatibility`	função	`(profile, query, *, mismatch_tol=0.25, re_band=0.15, slope=-2.0) -> ScaleCheck`	Diz se um perfil pode responder a uma consulta e quão bem suas premissas de filtragem combinam.

pinneapple_physics.multiscale.turbulence

Turbulência 2-D em decaimento como benchmark multiescala: dados de referência, conjuntos filtrados, fluxos de consultas e benchmark de fechamentos.

pinneapple_physics/multiscale/turbulence.py · 295 linhas · testes: 29

nome	tipo	assinatura	o que faz
`DecayingTurbulence2D`	classe	`(length, n_out, t_out, spinup, k0, energy, re_bands, widths, width_probabilities, kinds, …)`	Configuração do benchmark (padrões do experimento em `docs/core_concepts/multiscale_ensembles.md`) com geração de dados e consultas.
`benchmark_closures`	função	`(bench, data, closures, cases_per_band=8, times=(0.1, 0.2, 0.3), t_long=2.0, dt=0.005) ->…)`	Avalia fechamentos em casos separados de cada faixa de Reynolds na grade LES.
`scale_conditioning_study`	função	`(bench, data, idx, conditioned, fixed, blind=None, widths=(1, 2, 4, 6, 8), kind='gaussian…)`	Erros em cada largura de filtro para modelos condicionados, fixos e cegos.

pinneapple_physics.multiscale.viz

Animação de uma rodada de ensemble sensível à escala (escala pedida, modelo escolhido, espectros, pesos). [sem teste direto]

pinneapple_physics/multiscale/viz.py · 111 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`animate_multiscale_ensemble`	função	`(references, predictions, active, names, weights, widths, labels, cutoffs, path='multisca…)`	Gera um GIF com um quadro por consulta da rodada do ensemble.

pinneapple_physics.non_hermitian

Física não hermitiana: dímeros e cadeias PT-simétricos, pontos excepcionais (EPs) e se eles podem ser sintonizados; numpy, sympy para a parte simbólica e matplotlib para as figuras.

pinneapple_physics/non_hermitian/__init__.py · 29 linhas · testes: 8

pinneapple_physics.non_hermitian.ep_tuning

Verifica simbolicamente se um parâmetro de controle move o EP de um dímero PT-simétrico segundo uma lei desejada, contra o teorema de impossibilidade (registro Zenodo citado no código), e mostra três rotas para contorná-lo.

`pinneapple_physics/non_hermitian/ep_tuning.py` · 226 linhas · testes: 8

nome	tipo	assinatura	o que faz
`EPTuningReport`	classe	`(ep_law, target, residual, exact, degree, perturbation_class, assumptions, theorem_applie…)`	Relatório da análise: lei do EP, alvo, resíduo, se é exata, classe da perturbação, premissas e rotas.
`classify_coupling`	função	`(b_ab, b_ba=None, lam=None) -> str`	Classifica o acoplamento como constante, só de fase, recíproco polinomial, recíproco não polinomial ou não recíproco.
`analyze_ep_tuning`	função	`(b_ab, b_ba=None, target=None, lam=None, dimension=2) -> EPTuningReport`	Checagem simbólica de uma lei de sintonia do EP contra o alvo e o teorema.
`coupling_classes`	função	`() -> list[tuple[str, str, Any, Any]]`	As quatro classes de acoplamento do artigo como leis sympy prontas para `analyze_ep_tuning`.
`route_a_sqrt_coupling`	função	`(J, G) -> Callable[[float], complex]`	Rota A: acoplamento `sqrt(J^2 + lam J G)`, que dá exatamente a lei linear em `gamma_EP^2`.
`route_c_asymmetric`	função	`(J, G)`	Rota C: acoplamento não recíproco que dá a lei linear, mas quebra `H12 = conj(H21)`.
`feedback_coupling`	função	`(J, G, lam, b0=None, iters=60, tol=1e-14) -> tuple[float, int]`	Rota B: acoplamento atualizado por realimentação cujo ponto fixo é o acoplamento da rota A.
`theorem_profile`	função	`()`	O teorema como `ModelProfile` de `physics_audit`, para auditar se ele restringe um projeto.
`design_features`	função	`(H, polynomial) -> dict[str, float]`	Características de um Hamiltoniano de dispositivo para a auditoria de `theorem_profile`.

pinneapple_physics.non_hermitian.pt_dimer

Dímeros e cadeias PT-simétricos: Hamiltoniano, espectro, EPs (fórmula fechada e numérico) e coalescência de autovetores. [sem teste direto]

`pinneapple_physics/non_hermitian/pt_dimer.py` · 185 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`PTDimer`	classe	`(coupling, coupling_ba, onsite)`	Sistema de dois sítios com ganho e perda, acoplamento constante ou função do parâmetro de controle, recíproco ou não.
`PTChain`	classe	`(n, b, b_back)`	Cadeia de N sítios com ganho no primeiro, perda no último e acoplamento entre vizinhos.
`pt_violation`	função	`(H) -> float`	Medida de violação da simetria PT de um Hamiltoniano (0 se simétrico).
`eigenvector_overlap`	função	`(H) -> float`	Maior sobreposição normalizada entre autovetores à direita (1 num EP).
`petermann_factor`	função	`(H) -> np.ndarray`	Fator de Petermann de cada modo (1 para matriz hermitiana, diverge num EP).

pinneapple_physics.non_hermitian.viz

Figuras matplotlib para dímeros PT e sintonia de EP.

`pinneapple_physics/non_hermitian/viz.py` · 121 linhas · testes: 8

nome	tipo	assinatura	o que faz
`plot_spectrum`	função	`(ax, dimer, lam=0.0, gamma_max=None, n=400)`	Partes real e imaginária dos autovalores contra gamma, com o EP e a sobreposição de autovetores.
`plot_ep_laws`	função	`(ax, J=1.0, G=0.5, lam=None)`	`gamma_EP^2(lam)` de cada classe de acoplamento contra o alvo linear.
`plot_coupling_plane`	função	`(ax, J=1.0, G=0.5, lam=None)`	Trajetória do acoplamento no plano complexo; `gamma_EP` é a distância à origem.

nome	tipo	assinatura	o que faz
`plot_chain_threshold`	função	`(ax, sizes=(2, 3, 4, 5), b_values=None)`	Limiar de quebra PT de cadeias de N sítios contra `|b|^2`.
`ep_tuning_figure`	função	`(path, J=1.0, G=0.5) -> str`	Monta e grava a figura 2x2 da sintonia de EP e devolve o caminho.

`pinneapple_physics.online_learning`

Fixed-Share, AdaHedge, inferência conformal adaptativa e grade de Fixed-Share ponderada por AdaHedge, compartilhados por previsão de séries e ensembles físicos; decisões só com perdas já observadas. [sem teste direto]

`pinneapple_physics/online_learning.py` · 126 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`FixedShare`	classe	`(n, eta, alpha)`	Média exponencialmente ponderada com fixed share, que acompanha a melhor sequência de especialistas.
`AdaHedge`	classe	`(k)`	Hedge sem parâmetros que ajusta a taxa de aprendizado sozinho.
`AdaptiveConformal`	classe	`(level=0.9, gamma=0.01, window=500)`	Inferência conformal adaptativa sobre resíduos absolutos, mantendo a cobertura sob mudança de distribuição.
`FixedShareGrid`	classe	`(n, etas=(0.5, 2.0, 8.0), alphas=(0.0, 0.01, 0.05))`	Grade de agregadores Fixed-Share (eta, alpha) ponderada por AdaHedge.

`pinneapple_physics.paper_problems`

Problemas do PINNsFormer (convecção, reação, onda) como o código oficial os define: domínio, grade de treino e teste e solução analítica.

`pinneapple_physics/paper_problems.py` · 114 linhas · testes: 5

nome	tipo	assinatura	o que faz
`GridCase`	classe	`(name, equation, x_range, t_range, exact, n_train, n_test, bc, lbfgs_steps, coef, source)`	Um problema de teste em grade tensorial (x, t) com referência analítica e grades de treino e teste.
`tensor_grid`	função	`(x_range, t_range, nx, nt) -> dict[str, np.ndarray]`	Grade `meshgrid(x, t)` achatada como no `get_data` oficial do PINNsFormer.
`time_sequence`	função	`(points, k=5, dt=0.0001) -> np.ndarray`	Pseudo-sequência do PINNsFormer: repete cada (x, t) k vezes com t + i·dt.
`convection_exact`	função	`(x, t, beta=50.0)`	Solução exata da convecção u = sin(x − βt).
`reaction_exact`	função	`(x, t, rho=5.0)`	Solução logística exata da equação de reação.
`wave_exact`	função	`(x, t, c2=4.0)`	Solução exata da equação da onda 1D do PINNsFormer.
`pinnsformer_case`	função	`(name) -> GridCase`	Devolve o `GridCase` do PINNsFormer pelo nome.

`pinneapple_physics.paper_problems_xtfc`

Problemas do artigo X-TFC (Schiassi et al.) com resíduo, solução exata e configurações relatadas; não contém resultados do artigo.

`pinneapple_physics/paper_problems_xtfc.py` · 214 linhas · testes: 10

nome	tipo	assinatura	o que faz
`XTFCCase`	classe	`(name, problem, exact, n_train, n_neurons, activation, weight_range, paper_iterations, pa…)`	Um problema do X-TFC: problema, solução exata, pontos de treino, neurônios, ativação e configurações do artigo.
`p6_rhs`	função	`(X)`	Termo fonte do problema 6 do X-TFC.
`p7_rhs`	função	`(X)`	Termo fonte do problema 7 do X-TFC.

nome	tipo	assinatura	o que faz
`xtfc_case`	função	`(name) -> XTFCCase`	Devolve o `XTFCCase` pelo nome.

pinneapple_physics.pcb

Reexporta `Board`, `Component`, `solve`, `evaluate`, `what_if`, `calibrate` do modelo térmico de PCB.

`pinneapple_physics/pcb/__init__.py` · 16 linhas · testes: 10

pinneapple_physics.pcb.engine

Motor do PCB Hotspot: avaliação com faixa de incerteza do coeficiente de convecção, estudos what-if, escopo do modelo e calibração por Ensemble Kalman Inversion; sem surrogate aprendido.

`pinneapple_physics/pcb/engine.py` · 392 linhas · testes: 16

nome	tipo	assinatura	o que faz
`model_scope`	função	`(comps, env) -> Dict[str, Any]`	Lista o que o modelo cobre, para que lado cada simplificação erra e o que fazer, para aparecer em todo relatório.
`evaluate`	função	`(board, comps, env, *, quick=False, n_cells=60) -> Dict[str, Any]`	Avalia a placa: solve rápido em grade grossa ou completo com faixa de incerteza e verificações.
`field_3d`	função	`(board, out) -> Dict[str, Any]`	Monta o campo de temperatura 3D da placa para visualização a partir da saída do solver.
`what_if`	função	`(board, comps, env, n_cells=48) -> Dict[str, Any]`	Resolve de novo com correções comuns e mostra o efeito no componente crítico e nas junções.
`calibrate`	função	`(board, comps, env, measurements, *, noise_std_c=1.0, n_ensemble=16, n_iterations=8, seed…)`	Ajusta fatores de resfriamento superficial e espalhamento no plano a temperaturas medidas por Ensemble Kalman Inversion.

pinneapple_physics.pcb.solver

Modelo térmico 3D de volumes finitos de uma PCB em camadas com componentes JEDEC de dois resistores, vias térmicas, convecção+radiação iterada e contato com chassi; resolve com CG precondicionado por AMG.

`pinneapple_physics/pcb/solver.py` · 341 linhas · testes: 16

nome	tipo	assinatura	o que faz
`Component`	classe	`(name, x_mm, y_mm, power_w, package, side, tj_max_c, theta_jb, theta_jc, w_mm, d_mm, h_mm…)`	Componente montado: posição, potência, encapsulamento, lado, resistências térmicas, dimensões e vias.
`Board`	classe	`(width_mm, depth_mm, n_copper, thickness_mm, outer_oz, inner_oz, outer_coverage, inner_co…)`	Placa: dimensões, camadas de cobre, espessura, cobertura de cobre e contato com chassi nas bordas.
`validate`	função	`(board, comps) -> List[str]`	Lista problemas de entrada da placa e dos componentes.
`overlaps`	função	`(comps) -> List[str]`	Pares de componentes do mesmo lado cujas pegadas se sobrepõem.
`solve`	função	`(board, comps, env, *, n_cells=60, h_override=None, k_xy_scale=1.0, h_scale=1.0, max_iter…)`	Temperaturas em regime permanente da placa, das junções e balanço de energia.

pinneapple_physics.pde_environment

Ponto de entrada do ambiente de problemas: reexporta `ProblemSpec`, `PDETermSpec`, as condições de contorno, `ProblemBuilder`, o registro de presets, presets acadêmicos/CFD/industriais, os resíduos de turbulência e a base de famílias de EDP. O bloco de presets estruturais é importado dentro de `try/except ImportError`. [sem teste direto]

`pinneapple_physics/pde_environment/__init__.py` · 105 linhas · **sem teste direto**

pinneapple_physics.pde_environment.builder

`ProblemBuilder`: API fluente para montar um `ProblemSpec` (domínio, campos, EDP, condições, amostragem, solver) sem mexer nos dataclasses, e registrá-lo como preset.

`pinneapple_physics/pde_environment/builder.py` · 638 linhas · testes: 26

nome	tipo	assinatura	o que faz
`ProblemBuilder`	classe	`(name) -> None`	Construtor fluente de `ProblemSpec`: encadeia domínio, coordenadas, campos, EDP, parâmetros, condições, dados, amostragem e solver, e termina em `build()`.

pinneapple_physics.pde_environment.capabilities

Base de conhecimento de famílias de EDP: mapeia descrições em texto livre para tipos (`kind`) de EDP e parâmetros sugeridos de `ProblemSpec`, por contagem simples de palavras-chave; usada pelo agente de design de problemas. [sem teste direto]

`pinneapple_physics/pde_environment/capabilities.py` · 314 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`PDEFamily`	classe	`(kind, aliases, description, default_params, coords_hint, fields_hint, tags)`	Descritor de uma família de EDP conhecida: tipo, apelidos, descrição, parâmetros padrão, dicas de coordenadas e campos, tags.
`list_pde_families`	função	`() -> List[str]`	Lista ordenada dos tipos canônicos de EDP.
`get_pde_family`	função	`(kind_or_alias) -> Optional[PDEFamily]`	Busca uma `PDEFamily` por tipo ou apelido (sem diferenciar caixa).
`identify_pde`	função	`(description) -> List[Tuple[str, float]]`	Pontua as famílias conhecidas contra uma descrição em texto livre por sobreposição de palavras-chave.
`suggest_problem_spec`	função	`(description) -> Dict[str, Any]`	Sugere argumentos de construção de `ProblemSpec` a partir de uma descrição em texto livre.

pinneapple_physics.pde_environment.condition_sampling

Amostragem de pontos para condições cuja região é dada por uma função seletora, e normais externas da caixa do domínio.

`pinneapple_physics/pde_environment/condition_sampling.py` · 68 linhas · testes: 15

nome	tipo	assinatura	o que faz
`sample_condition_points`	função	`(select, domain_bounds, coords, n, rng) -> np.ndarray`	Até n pontos do domínio aceitos por uma função seletora (máscara booleana).
`box_face_normals`	função	`(points, domain_bounds, coords, time_coord='t') -> Tuple[np.ndarray, np.ndarray]`	Normais unitárias externas da caixa do domínio nos pontos (zero na coordenada de tempo).

pinneapple_physics.pde_environment.conditions

`ConditionSpec`, a especificação genérica de restrição (Dirichlet, Neumann, Robin, interface, condição inicial, dados), e as fábricas que a constroem.

`pinneapple_physics/pde_environment/conditions.py` · 495 linhas · testes: 26

nome	tipo	assinatura	o que faz
`ConditionSpec`	classe	`(name, kind, fields, selector_type, selector, value_fn, weight, order, deriv_coord, inter…)`	Especificação genérica de restrição: tipo, campos, seletor, função de valor, peso e dados extras (ordem, coeficientes de interface, tração).
`DirichletBC`	função	`(name, fields=None, selector_type='all', selector=None, value_fn=None, weight=1.0) -> Con…)`	Fábrica de condição de Dirichlet (valor imposto).

nome	tipo	assinatura	o que faz
NeumannBC	função	(name, fields=None, selector_type='all', selector=None, value_fn=None, weight=1.0, order=…)	Fábrica de condição de Neumann (derivada normal, ou derivada repetida numa coordenada quando order > 1).
RobinBC	função	(name, fields=None, selector_type='all', selector=None, value_fn=None, weight=1.0) -> Con…)	Fábrica de condição de Robin (combinação de valor e derivada normal).
InterfaceBC	função	(name, fields, selector_type='all', selector=None, value_fn=None, weight=1.0, k_a=1.0, k_…)	Fábrica de condição de interface entre dois subdomínios (continuidade de valor e de fluxo com k_a, k_b).
InitialCondition	função	(name, fields=None, selector_type='all', selector=None, value_fn=None, weight=1.0) -> Con…)	Fábrica de condição inicial.
DataConstraint	função	(name, fields=None, selector_type='all', selector=None, value_fn=None, weight=1.0) -> Con…)	Fábrica de restrição de dados (perda supervisionada em pontos dados).

pinneapple_physics.pde_environment.environment_typing

Apelidos de tipo CoordNames e FieldNames (tuplas de strings) usados pelas specs.

pinneapple_physics/pde_environment/environment_typing.py · 6 linhas · testes: 26

pinneapple_physics.pde_environment.presets

Subpacote de presets de problemas; reexporta o registro (get_preset, list_presets, register_preset) e o preset de terramecânica.

pinneapple_physics/pde_environment/presets/__init__.py · 14 linhas · testes: 21

pinneapple_physics.pde_environment.presets._curve_geometry

Geometria paramétrica de curvas e polígonos para os presets com tags cujo domínio não é caixa nem cilindro: pá de cascata em arco circular, silhueta tipo Ahmed body e aerofólio NACA simétrico. Módulo privado. [sem teste direto]

pinneapple_physics/pde_environment/presets/_curve_geometry.py · 227 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
circular_arc_cascade_blade_points	função	(n, rng, *, flow_angle_in_deg, flow_angle_out_deg, chord, x_le, y_le) -> np.ndarray	Amostra n pontos no contorno de uma pá de cascata em arco circular definida pelos ângulos de entrada/saída e pela corda.
ahmed_body_polygon	função	(car_length, car_height, *, slant_angle_deg=_AHMED_DEFAULT_SLANT_ANGLE_DEG, n_arc=24, x0=…)	Polígono fechado da silhueta de carro inspirada no Ahmed body.
naca4_symmetric_polygon	função	(chord, thickness_ratio, *, n=80, x0=0.0, y0=0.0) -> np.ndarray	Polígono fechado de um aerofólio NACA de 4 dígitos simétrico.
polygon_perimeter_sample	função	(vertices, n, rng) -> np.ndarray	Amostra pontos uniformes por comprimento de arco ao longo de um polígono fechado.
polygon_contains	função	(vertices, X) -> np.ndarray	Teste vetorizado de ponto dentro de polígono por lançamento de raio.

pinneapple_physics.pde_environment.presets._utils

Auxiliar interno dos presets: constantes de Lamé a partir de E e nu. Fora da API pública. [sem teste direto]

pinneapple_physics/pde_environment/presets/_utils.py · 10 linhas · **sem teste direto**

pinneapple_physics.pde_environment.presets.academics

Presets acadêmicos registrados: Laplace 2D, Poisson 2D (fonte via ctx['source_fn']) e Burgers 1D.

pinneapple_physics/pde_environment/presets/academics.py · 113 linhas · testes: 37

nome	tipo	assinatura	o que faz
laplace_2d_default	função	() -> ProblemSpec	Preset de Laplace 2D com Dirichlet na fronteira marcada por tag.
poisson_2d_default	função	() -> ProblemSpec	Preset de Poisson 2D; a fonte f(x,y) vem de ctx['source_fn'].
burgers_1d_default	função	(nu=0.01) -> ProblemSpec	Preset de Burgers 1D viscoso com viscosidade nu.

pinneapple_physics.pde_environment.presets.astroml_bridge

Invólucro fino sobre o astroML (dependência opcional): periodogramas de Lomb-Scargle, deconvolução extrema, Bayesian Blocks, histograma de Knuth, ajustes de reta com erros, datasets e verificações sintéticas. Os periodogramas do astroML são marcados como deprecated (avisos silenciados).

pinneapple_physics/pde_environment/presets/astroml_bridge.py · 576 linhas · testes: 12 · palavras-chave no código (heurística; o texto acima diz o status real): shim ou obsoleto

nome	tipo	assinatura	o que faz
Periodogram	classe	(freq, power, method, n_terms, n_obs, false_alarm_levels, meta)	Periodograma numa grade de frequências, com pico, melhor período, nível de falso alarme e conversão para amostra.
XDResult	classe	(weights, means, covariances, log_likelihood, model)	Resultado da deconvolução extrema: mistura gaussiana da distribuição sem erro; permite amostrar.
LineFit	classe	(slope, intercept, covariance, chi2, dof, method)	Reta ajustada com inclinação, intercepto, covariância, chi2 e graus de liberdade.
astroml_available	função	() -> bool	Diz se o astroML pode ser importado.
frequency_grid	função	(t, period_range=None, f_min=None, f_max=None, oversample=10.0, n_max=200000) -> np.ndarr…)	Grade uniforme de frequências para o periodograma de amostras irregulares.
lomb_scargle_periodogram	função	(t, y, dy, freq=None, *, period_range=None, oversample=10.0, generalized=True, significan…)	Periodograma de Lomb-Scargle generalizado (média flutuante) do astroML.
multiterm_periodogram	função	(t, y, dy, freq=None, *, n_terms=3, period_range=None, oversample=10.0, time_unit='d') ->…)	Periodograma multi-harmônico do astroML.
bootstrap_peak_distribution	função	(t, y, dy, freq, n_bootstraps=100, random_state=0, generalized=True) -> np.ndarray	Pico mais alto do periodograma em reamostragens bootstrap dos dados.
best_period	função	(t, y, dy, *, period_range=None, n_terms=1, oversample=10.0, refine=True) -> float	Período do pico mais alto do periodograma.
light_curve_sample	função	(t, y, dy, *, band='', time_unit='d', value_unit='mag', **provenance)	Curva de luz irregular como PhysicalSample de domínio de pontos (não precisa de astroML).
xd_fit	função	(X, Xerr, n_components, *, max_iter=200, tol=1e-05, random_state=0) -> XDResult	Deconvolução extrema (XDGMM do astroML) de dados com covariância de erro por ponto.
bayesian_block_edges	função	(t, x=None, sigma=None, *, fitness='events', p0=0.05, gamma=None) -> np.ndarray	Bordas de bins por Bayesian Blocks (Scargle et al. 2013).
knuth_histogram	função	(data) -> tuple[np.ndarray, np.ndarray, float]	Histograma com largura de bin ótima bayesiana de Knuth.
fit_line_with_errors	função	(x, y, dy) -> LineFit	Mínimos quadrados ponderados com erro só em y.
fit_line_tls	função	(x, y, dx, dy, initial=None) -> LineFit	Ajuste de reta com erros nos dois eixos (TLS, Hogg, Bovy & Lang 2010).
load_dataset	função	(name, *, data_home=None, download_if_missing=False, **kwargs)	Chama um fetcher de astroML.datasets por nome curto; não baixa nada por padrão.
check_period_recovery	função	(seed=0, period=0.613, n=150, noise=0.15) -> dict	Verificação sintética: período recuperado de uma senoide irregular contra o verdadeiro.
check_xd_means	função	(seed=0, n=600) -> dict	Verificação sintética: erro das médias da deconvolução extrema em duas gaussianas 2D.
check_block_step	função	(seed=0, t_step=6.3, p0=0.001) -> dict	Verificação sintética: borda de Bayesian Blocks contra um degrau numa série ruidosa.
check_line_fits	função	(seed=0) -> dict	Verificação sintética: erro relativo da inclinação dos ajustes WLS e TLS.

nome	tipo	assinatura	o que faz
validate	função	(cases=None, log=print, **kw) -> list[dict]	Roda as verificações sintéticas e registra uma tabela; precisa de astroML, sem rede.

pinneapple_physics.pde_environment.presets.astrophysics

Presets de astrofísica e sistemas espaciais (Kepler, Clohessy-Wiltshire, J2, atitude, Lane-Emden, NFW, Sod, CR3BP, deflexão de luz, disco de Shakura-Sunyaev), cada um com solução de referência em `meta`, verificada com sympy segundo o docstring.

pinneapple_physics/pde_environment/presets/astrophysics.py · 1.486 linhas · testes: 72

nome	tipo	assinatura	o que faz
kepler_exact_state	função	(t, mu, a, e) -> Dict[str, np.ndarray]	Trajetória kepleriana exata de dois corpos nos tempos dados (perigeu em t=0).
kepler_two_body_orbit	função	(mu=398600.4418, a=8000.0, e=0.15) -> ProblemSpec	Preset de órbita kepleriana de dois corpos em formulação cartesiana plana.
cw_exact_velocity	função	(t, n, x0, y0, z0, vx0, vy0, vz0) -> Dict[str, np.ndarray]	Velocidade relativa de Clohessy-Wiltshire em forma fechada.
space_debris_cw_relative_motion	função	(n=0.0011, x0=1.0, y0=0.0, z0=0.2, vx0=0.0, vy0=-0.0015, vz0=0.0008) -> ProblemSpec	Preset das equações de Clohessy-Wiltshire para movimento relativo de detritos e operações de proximidade.
satellite_j2_perturbation	função	(mu=398600.4418, J2=0.00108262668, Re=6378.137, a=7000.0, e=0.001, inclination_deg=98.7) …)	Preset de órbita LEO com perturbação J2.
spacecraft_attitude_euler_rotation	função	(I1=100.0, I3=150.0, w1_0=0.05, w3_0=0.5) -> ProblemSpec	Preset de dinâmica de atitude sem torque de corpo rígido axissimétrico.
lane_emden_exact_theta	função	(xi, n) -> np.ndarray	Solução fechada de Lane-Emden para n em {0, 1, 5}; levanta erro para outros n.
lane_emden_polytrope	função	(n=1.0, xi_min=0.001, xi_max=3.0) -> ProblemSpec	Preset da equação de Lane-Emden para estrela politrópica.
nfw_potential_exact	função	(r, G, rho_s, rs) -> np.ndarray	Potencial gravitacional NFW em forma fechada.
nfw_source_fn	função	(G, rho_s, rs)	Função fonte 4 pi G rho_NFW para o tipo de EDP 'poisson'.
nfw_dark_matter_potential	função	(G=1.0, rho_s=1.0, rs=1.0, r_max=10.0) -> ProblemSpec	Preset de Poisson para o potencial de um halo de matéria escura NFW.
sod_exact_solution	função	(x, t, gamma=1.4, rho_l=1.0, u_l=0.0, p_l=1.0, rho_r=0.125, u_r=0.0, p_r=0.1, x0=0.5) -> …)	Solução de Riemann exata do tubo de choque de Sod (algoritmo de Toro).
sod_shock_tube_astro	função	(gamma=1.4, x0=0.5, t_end=0.2) -> ProblemSpec	Preset do tubo de choque de Sod 1D (Euler compressível).
cr3bp_omega_gradient	função	(x, y, mu)	Gradiente do potencial efetivo do problema restrito de três corpos no referencial sinódico.
cr3bp_collinear_lagrange_point	função	(mu, x0, tol=1e-14, max_iter=100) -> float	Ponto de Lagrange colinear por Newton-Raphson 1D a partir de um chute no ramo certo.
cr3bp_lagrange_points	função	(mu) -> Dict[str, tuple]	Os cinco pontos de Lagrange do CR3BP plano para uma razão de massa.
cr3bp_planar_synodic	função	(mu=0.012150585609624, dx0=0.0001, dy0=0.0, n_periods=4.0) -> ProblemSpec	Preset do CR3BP plano Terra-Lua no referencial sinódico, adimensional.
schwarzschild_light_bending_u_exact	função	(phi, m, b) -> np.ndarray	Solução perturbativa de primeira ordem da geodésica nula de Schwarzschild, u(phi).
schwarzschild_deflection_angle_weak_field	função	(GM, c, b) -> float	Deflexão da luz em campo fraco, 4GM/(c^2 b).
schwarzschild_light_bending_weak_field	função	(GM=1.32712440018e+20, c=299792458.0, b=695700000.0) -> ProblemSpec	Preset da equação da órbita do fóton u(phi) no limite de campo fraco.
shakura_sunyaev_flux_exact	função	(r, GM, Mdot, R_in) -> np.ndarray	Perfil radial de fluxo do disco alfa de Shakura-Sunyaev.
shakura_sunyaev_teff_exact	função	(r, GM, Mdot, R_in, sigma_SB=5.670374419e-08) -> np.ndarray	Perfil de temperatura efetiva do disco de Shakura-Sunyaev.

nome	tipo	assinatura	o que faz
shakura_sunyaev_accretion_disk	função	(M_bh_solar=10.0, Mdot_edd_frac=0.1, eta=0.1, r_out_factor=10000.0) -> ProblemSpec	Preset do disco de acreção fino e opticamente espesso de Shakura-Sunyaev.

pinneapple_physics.pde_environment.presets.astropy_bridge

Unidades e transformações de coordenadas com `astropy`: constantes astronômicas em SI com procedência, quantidades com unidade, conversão para SI e transformação GCRS/ITRS.

pinneapple_physics/pde_environment/presets/astropy_bridge.py · 296 linhas · testes: 22

nome	tipo	assinatura	o que faz
astro_constant_si	função	(name) -> float	Valor em SI de uma constante da tabela ASTRO_CONSTANTS.
constant_provenance	função	(name) -> str	Referência bibliográfica do astropy para uma constante.
with_units	função	(value, unit_str=None) -> 'u.Quantity'	Anexa unidades astropy a um número ou analisa uma string de unidade.
to_si	função	(quantity, target_unit=None) -> float	Converte uma Quantity (ou string como "5 Msun") em float SI ou na unidade pedida.
transform_coordinates	função	(x, y, z, obstime, frame_in='gcrs', frame_out='itrs', vx=None, vy=None, vz=None, unit='km…)	Transforma posição (e velocidade) cartesiana entre GCRS e ITRS com astropy.coordinates.

pinneapple_physics.pde_environment.presets.cfd

Presets de CFD: Navier-Stokes incompressível 2D e 3D, cavidade 3D com tampa móvel, canal 3D de Poiseuille e tubo de Hagen-Poiseuille. [sem teste direto]

pinneapple_physics/pde_environment/presets/cfd.py · 459 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
ns_incompressible_2d_default	função	(Re=100.0, Umax=1.0) -> ProblemSpec	Preset de Navier-Stokes incompressível 2D transiente (u, v, p) com número de Reynolds dado.
ns_incompressible_3d_default	função	(Re=100.0, Umax=1.0) -> ProblemSpec	Preset de Navier-Stokes incompressível 3D transiente (u, v, w, p).
lid_driven_cavity_3d	função	(Re=100.0, size=1.0, lid_velocity=1.0) -> ProblemSpec	Preset da cavidade 3D com tampa móvel.
channel_flow_3d	função	(Re=100.0, length=2.0, height=1.0, width=1.0, Umax=1.0) -> ProblemSpec	Preset de escoamento de Poiseuille num canal retangular 3D.
pipe_flow_3d	função	(Re=100.0, radius=0.5, length=2.0, Umax=1.0) -> ProblemSpec	Preset de escoamento de Hagen-Poiseuille num tubo cilíndrico.

pinneapple_physics.pde_environment.presets.electromagnetics

Presets eletromagnéticos com solução fechada registrada em `closed_form.electromagnetics` (via `pp.solve(problem, "analytic")`): capacitor de dois dielétricos, fio, cavidade TM, efeito pelicular e espalhamento por cilindro. [sem teste direto]

pinneapple_physics/pde_environment/presets/electromagnetics.py · 184 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
em_two_dielectric_capacitor	função	(V=1.0, eps1=1.0, eps2=4.0, d1=0.5, h=0.02) -> ProblemSpec	Preset de capacitor com dois dielétricos na forma integral (electrostatics_integral).
em_two_dielectric_capacitor_differential	função	(V=1.0, eps1=1.0, eps2=4.0, d1=0.5) -> ProblemSpec	Preset do mesmo capacitor na forma diferencial (electrostatics).
em_wire_az	função	(a=0.25, J=1.0, nu=1.0) -> ProblemSpec	Preset de magnetostática do potencial A_z de um fio com corrente uniforme.
em_tm_cavity	função	(m=1, n=1, periods=0.5) -> ProblemSpec	Preset de modo TM numa cavidade (maxwell_tm).

nome	tipo	assinatura	o que faz
em_tm_cavity_integral	função	(m=1, n=1, periods=0.5, h=0.02) -> ProblemSpec	Preset do modo TM na forma integral (maxwell_tm_integral).
em_skin_effect	função	(delta=0.2, omega=1.0, nu=1.0, length=1.0) -> ProblemSpec	Preset 1D de efeito pelicular numa placa condutora.
em_cylinder_scattering	função	(k0=2.0 * math.pi, radius=0.25, eps_cyl=4.0, eps_cyl_i=0.0, half_width=1.0) -> ProblemSpec	Preset de espalhamento TM por cilindro dielétrico.

pinneapple_physics.pde_environment.presets.engineering

Presets de engenharia: bocal e carcaça de foguete, asa, aerodinâmica de carro, freio, suspensão, dissipador de CPU, PCB, ventoinha, forno industrial, refratário, combustão e data center.

pinneapple_physics/pde_environment/presets/engineering.py · 1.169 linhas · testes: 7

nome	tipo	assinatura	o que faz
rocket_nozzle_cfd	função	(gamma=1.4, R_gas=287.0, T_inlet=3500.0, p_inlet=10000000.0, throat_radius=0.1, exit_radi…)	Preset de escoamento compressível axissimétrico num bocal convergente-divergente.
rocket_structural	função	(E=200000000000.0, nu=0.3, p_internal=10000000.0, T_inner=800.0, T_outer=293.0, alpha_T=1…)	Preset da carcaça do motor sob pressão interna e gradiente térmico.
aircraft_wing_aerodynamics	função	(Re=5000000.0, Ma=0.3, alpha_deg=5.0, chord=1.0, rho_inf=1.225, U_inf=102.0, nu_air=1.5e-…)	Preset 2D de aerodinâmica de aerofólio tipo NACA em baixo Mach.
aircraft_wing_structural	função	(E=70000000000.0, nu=0.33, lift_load=50000.0, span=5.0, thickness=0.1) -> ProblemSpec	Preset de tensão plana da longarina de asa sob carga de sustentação.
car_external_aero	função	(U_inf=33.3, rho=1.225, nu_air=1.5e-05, car_length=4.5, car_height=1.4) -> ProblemSpec	Preset 2D de aerodinâmica externa de um carro simplificado.
car_brake_thermal	função	(k_disc=55.0, rho_disc=7100.0, cp_disc=500.0, q_friction=2000000.0, T_ambient=293.0, h_co…)	Preset de condução transiente num disco de freio em frenagem de emergência.
car_suspension_fatigue	função	(E=210000000000.0, nu=0.3, F_vertical=8000.0, F_lateral=3000.0, arm_length=0.35, arm_widt…)	Preset de tensão plana para fadiga de braço de suspensão.
cpu_heatsink_thermal	função	(k_aluminium=205.0, k_copper=400.0, q_cpu=200.0, T_ambient=293.0, h_fin=50.0, die_area=0.…)	Preset 3D de condução/convecção permanente num dissipador de CPU.
pcb_thermal	função	(k_pcb=0.3, k_z_pcb=0.25, q_components=None, T_ambient=308.0, h_natural=10.0, board_lengt…)	Preset 2D de espalhamento de calor numa PCB com várias fontes.
fan_cooler_cfd	função	(rpm=2000.0, blade_radius=0.05, hub_radius=0.01, rho=1.225, nu_air=1.5e-05, n_blades=7) -…)	Preset 2D de escoamento radial numa ventoinha de PC.
industrial_furnace_thermal	função	(T_hot_gas=1600.0, T_ambient=300.0, k_refractory=1.5, k_insulation=0.1, h_hot_gas=50.0, h…)	Preset 3D de transferência de calor permanente na parede de um forno industrial.
refractory_lining	função	(layers=None, T_hot=1700.0, T_cold=350.0, total_thickness=0.5) -> ProblemSpec	Preset de parede refratária multicamada com condutividades diferentes.
furnace_combustion_zone	função	(T_flame=2200.0, T_wall=1600.0, rho_flue=0.24, nu_flue=0.0002, U_inlet=5.0, furnace_lengt…)	Preset simplificado de zona de combustão com liberação volumétrica de calor.
datacenter_airflow_2d	função	(U_cold_aisle=2.5, T_cold_air=291.0, T_hot_max=318.0, Q_rack=20000.0, rack_height=2.0, ra…)	Preset 2D de resfriamento de fileira de racks (corredor frio/quente).
datacenter_server_thermal	função	(Q_cpu=250.0, Q_gpu=400.0, Q_ram=20.0, T_inlet_air=295.0, U_fan=3.0, h_forced=120.0, k_bo…)	Preset 2D de placa de servidor com condução e convecção forçada.
datacenter_cfd_3d	função	(room_length=20.0, room_width=10.0, room_height=3.0, n_racks=20, Q_per_rack=15000.0, T_su…)	Preset 3D simplificado de sala de data center para prever pontos quentes.

pinneapple_physics.pde_environment.presets.industry

Presets industriais 3D/2D genéricos: condução de calor permanente e transiente, elasticidade linear, Darcy, Helmholtz acústico, onda ultrassônica e reação-difusão. [sem teste direto]

pinneapple_physics/pde_environment/presets/industry.py · 347 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
steady_heat_conduction_3d_default	função	() -> ProblemSpec	Preset de condução de calor 3D permanente (tipo 'poisson').
transient_heat_3d_default	função	() -> ProblemSpec	Preset de equação do calor 3D transiente.

nome	tipo	assinatura	o que faz
`linear_elasticity_3d_default`	função	`() -> ProblemSpec`	Preset de elasticidade linear 3D com constantes de Lamé unitárias.
`darcy_pressure_only_3d_default`	função	`() -> ProblemSpec`	Preset de Darcy 3D só com pressão.
`helmholtz_acoustics_3d_default`	função	`() -> ProblemSpec`	Preset de Helmholtz 3D para acústica.
`wave_ultrasound_3d_default`	função	`() -> ProblemSpec`	Preset de equação da onda 3D (ultrassom).
`reaction_diffusion_2d_default`	função	`() -> ProblemSpec`	Preset de reação-difusão 2D montado como equação do calor; o termo de reação deve vir de `ctx['source_fn']`.

`pinneapple_physics.pde_environment.presets.multidisciplinary`

Presets multidisciplinares: água rasa atmosférica, giro oceânico de Stommel, fônons, fratura por campo de fase, Black-Scholes, Heston, farmacocinética, difusão de fármaco, SIR e dinâmica de opinião. [sem teste direto]

`pinneapple_physics/pde_environment/presets/multidisciplinary.py` · 1.100 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`climate_atmosphere_2d`	função	`(g=9.81, f=0.0001, H0=1000.0, U_ref=10.0, lon_min=0.0, lon_max=360.0, lat_min=-90.0, lat_…)`	Preset de água rasa 2D para dinâmica atmosférica.
`climate_ocean_gyre`	função	`(beta=2e-11, tau0=0.1, r=1e-07, rho0=1025.0, H=500.0, L=1000000.0, W=500000.0) -> Problem…)`	Preset de giro oceânico pelo modelo de Stommel no plano beta.
`crystal_phonon`	função	`(k=150.0, tau=1e-12, vg=3000.0, T_hot=320.0, T_cold=280.0, L_domain=1e-06) -> ProblemSpec`	Preset 1D de transporte de fônons (Boltzmann simplificado).
`material_fracture_2d`	função	`(E=210000000000.0, nu=0.3, Gc=2700.0, l0=0.005, Lx=1.0, Ly=1.0) -> ProblemSpec`	Preset 2D de fratura por campo de fase (Bourdin-Francfort-Marigo).
`black_scholes_1d`	função	`(sigma=0.2, r=0.05, K=100.0, T=1.0) -> ProblemSpec`	Preset da EDP de Black-Scholes para opção de compra europeia.
`heston_pde_2d`	função	`(kappa=2.0, theta=0.04, sigma_v=0.3, rho=-0.7, r=0.05, K=100.0, T=1.0, S_max=300.0, v_max…)`	Preset da EDP de Heston (volatilidade estocástica) 2D.
`pk_two_compartment`	função	`(k12=0.5, k21=0.3, kel=0.2, V1=10.0, dose=500.0, t_end=24.0) -> ProblemSpec`	Preset de farmacocinética de dois compartimentos (sistema de EDOs).
`drug_diffusion_tissue`	função	`(D=1e-10, lam=5e-05, C0=1.0, Lx=0.01, Ly=0.005, t_end=3600.0) -> ProblemSpec`	Preset 2D de difusão de fármaco com metabolismo de primeira ordem.
`sir_epidemic`	função	`(beta=0.3, gamma=0.1, N=1000000.0, I0=100.0, t_end=200.0) -> ProblemSpec`	Preset do modelo epidêmico SIR (sistema de EDOs).
`opinion_dynamics_2d`	função	`(D=0.01, alpha=1.0, Lx=1.0, Ly=1.0, t_end=10.0) -> ProblemSpec`	Preset 2D de dinâmica de opinião (Hegselmann-Krause contínuo).

`pinneapple_physics.pde_environment.presets.registry`

Registro que mapeia nomes de presets para funções fábrica, com decorador de registro, criação por nome e listagem.

`pinneapple_physics/pde_environment/presets/registry.py` · 106 linhas · testes: 119

nome	tipo	assinatura	o que faz
`register_preset`	função	`(name)`	Decorador que registra uma função fábrica de preset por nome.
`get_preset`	função	`(name, **kwargs) -> ProblemSpec`	Cria um preset pelo nome, repassando argumentos.
`list_presets`	função	`() -> List[str]`	Lista ordenada de todos os presets registrados.

`pinneapple_physics.pde_environment.presets.solid_mechanics`

Presets de mecânica dos sólidos: elasticidade axissimétrica, cilindro de parede grossa de Lamé (com solução analítica) e acoplamento roscado TC50 (corpos BOX e PIN, carga combinada). [sem teste direto]

`pinneapple_physics/pde_environment/presets/solid_mechanics.py` · 669 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
axisymmetric_linear_elasticity_2d_default	função	(r_min=10.0, r_max=50.0, z_min=0.0, z_max=100.0, E=210000000000.0, nu=0.3, p_inner=0.0, p…)	Preset de elasticidade linear axissimétrica em (r, z).
thick_walled_cylinder_lame_default	função	(a=20.0, b=60.0, L=100.0, p_a=100000000.0, p_b=0.0, E=210000000000.0, nu=0.3) -> ProblemS…)	Preset do cilindro de parede grossa sob pressão (problema de Lamé).
lame_analytical	função	(r, a, b, p_a, p_b, E, nu) -> dict	Solução analítica de Lamé para o cilindro de parede grossa.
threaded_coupling_tc50_box_default	função	(clearance=0.1, thread_height=0.8, offset=1.0, E=210000000000.0, nu=0.3, traction_top=100…)	Preset 2D axissimétrico do corpo BOX de um acoplamento roscado TC50.
threaded_coupling_tc50_rotating_default	função	(body='BOX', E=210000000000.0, nu=0.3, p_inner=20000000.0, F_axial=500000.0, T_torque=400…)	Preset do acoplamento TC50 sob carga combinada (pressão, força axial, torque, rotação).
threaded_coupling_tc50_pin_default	função	(thread_height=0.8, E=210000000000.0, nu=0.3, traction_top=100000000.0) -> ProblemSpec	Preset do corpo PIN do acoplamento TC50 (mesma física do BOX, outra geometria).

pinneapple_physics.pde_environment.presets.structural

Presets estruturais: estado plano de tensão e deformação, von Mises, elasticidade 3D, acoplamento rotativo sob torção e termoelasticidade 2D. [sem teste direto]

pinneapple_physics/pde_environment/presets/structural.py · 429 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
plane_stress_2d_default	função	(E=210000000000.0, nu=0.3, load_x=0.0, load_y=-1000.0) -> ProblemSpec	Preset de elasticidade linear em estado plano de tensão.
plane_strain_2d_default	função	(E=210000000000.0, nu=0.3) -> ProblemSpec	Preset de elasticidade linear em estado plano de deformação.
von_mises_2d_default	função	(E=210000000000.0, nu=0.3) -> ProblemSpec	Preset de tensão plana com a tensão de von Mises como campo extra.
linear_elasticity_3d	função	(E=210000000000.0, nu=0.3) -> ProblemSpec	Preset de elasticidade linear 3D para suportes e estruturas gerais.
rotary_coupling_torsion_default	função	(E=210000000000.0, nu=0.3, r_inner=0.05, r_outer=0.08, torque=10000.0, axial_force=500000…)	Preset de acoplamento rotativo sob torção e tração axial combinadas.
thermoelasticity_2d_default	função	(E=70000000000.0, nu=0.33, alpha_T=2.3e-05, dT=100.0) -> ProblemSpec	Preset 2D de termoelasticidade (expansão térmica por variação de temperatura).

pinneapple_physics.pde_environment.presets.tag_geometry

Geometria real para presets com condições `selector_type="tag"`: só para presets cujo domínio é caixa ou cilindro descrito pelos próprios parâmetros; os demais continuam levantando `TagConditionsUnresolved` de propósito.

pinneapple_physics/pde_environment/presets/tag_geometry.py · 816 linhas · testes: 9

nome	tipo	assinatura	o que faz
build_tag_batch	função	(name, spec, *, n_col=20000, n_bc_per_face=3000, seed=7, user_ctx=None)	Monta um lote com geometria real para um preset com fixture; KeyError se não houver fixture.
tag_solve_kwargs	função	(name, spec, *, n_bc_per_face=1000, seed=7) -> Dict[str, Any]	Argumentos extras de `solve_pde` que um preset precisa: vazio sem tags, a geometria da fixture, ou KeyError.
solve_pde_kwargs_from_batch	função	(batch) -> Dict[str, Any]	Tira `x_col` do lote e converte arrays NumPy em tensores torch para passar a `solve_pde`.
per_tag_losses	função	(spec, model, batch, *, w_pde=1.0, w_bc=10.0) -> dict[str, float]	Cada termo de perda de `compile_problem(spec)` num lote (total, EDP e um por condição), como floats.
preset_name	função	(spec) -> str	Chave de registro de onde a spec veio (`problem_id` ou `meta['preset']`), ou vazio.
has_default_geometry	função	(spec) -> bool	Diz se as condições com tag da spec podem ser satisfeitas pela própria fixture.
default_geometry	função	(spec, *, n_bc_per_face=1000, seed=7) -> Dict[str, Any]	Argumentos de `solve_pde` que satisfazem as tags com a caixa ou cilindro do preset, ou vazio.

nome	tipo	assinatura	o que faz
`default_geometry_report`	função	`(spec) -> str`	Uma linha dizendo qual geometria um preset usa para suas fronteiras com tag.

`pinneapple_physics.pde_environment.presets.terramechanics`

Resíduos físicos e preset do substituto PINN de Bekker-Wong (interação roda-solo) em terramecânica. [sem teste direto]

`pinneapple_physics/pde_environment/presets/terramechanics.py` · 200 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`TerramechanicsResiduals`	classe	`(c_Pa=1400.0, phi_deg=30.0, R_m=0.125, b_m=0.06, n_phys=256, R_factor=0.06, slip_range=(0…)`	Resíduos físicos chamáveis do substituto PINN de Bekker-Wong (parâmetros do solo e da roda, faixas de escorregamento e afundamento).
`bekker_wong_surrogate_2d`	função	`(c_Pa=1400.0, phi_deg=30.0, R_m=0.125, b_m=0.06) -> ProblemSpec`	Devolve o `ProblemSpec` do substituto PINN 2D de Bekker-Wong.

`pinneapple_physics.pde_environment.presets.turbomachinery`

Presets de turbomáquinas: análise de linha média de compressor axial multiestágio, cascata 2D de Euler compressível e estágio 3D em coordenadas cilíndricas.

`pinneapple_physics/pde_environment/presets/turbomachinery.py` · 480 linhas · testes: 7

nome	tipo	assinatura	o que faz
`axial_compressor_meanline`	função	`(num_stages=5, pressure_ratio=3.0, mass_flow_rate=4.37, rpm=10000.0, inlet_total_pressure…)`	Preset 1D de linha média termodinâmica de compressor axial multiestágio.
`axial_compressor_cascade_2d`	função	`(inlet_mach=0.5, flow_angle_in_deg=50.0, flow_angle_out_deg=20.0, chord=0.1, pitch_to_cho…)`	Preset 2D de Euler compressível numa cascata de pás de compressor.
`axial_compressor_stage_3d`	função	`(rpm=10000.0, pressure_ratio_stage=1.4, mass_flow_rate=4.37, hub_radius=0.1, tip_radius=0…)`	Preset 3D de um estágio de compressor em coordenadas cilíndricas.

`pinneapple_physics.pde_environment.scales`

`ScaleSpec`: parâmetros mínimos e opcionais de adimensionalização (L, U, alpha).

`pinneapple_physics/pde_environment/scales.py` · 18 linhas · testes: 69

nome	tipo	assinatura	o que faz
`ScaleSpec`	classe	`(L, U, alpha)`	Parâmetros opcionais de escala: comprimento, velocidade e difusividade característicos.

`pinneapple_physics.pde_environment.spec`

Dataclasses centrais `PDETermSpec` (descritor da EDP para o compilador) e `ProblemSpec` (especificação completa do problema).

`pinneapple_physics/pde_environment/spec.py` · 51 linhas · testes: 154

nome	tipo	assinatura	o que faz
`PDETermSpec`	classe	`(kind, fields, coords, params, meta)`	Descritor da EDP usado pelo compilador: tipo, campos, coordenadas, parâmetros e metadados.
`ProblemSpec`	classe	`(name, dim, coords, fields, pde, conditions, sample_defaults, scales, field_ranges, refer…)`	Especificação completa do problema: dimensão, coordenadas, campos, EDP, condições, amostragem, escalas, limites do domínio, solver e metadados.

`pinneapple_physics.pde_environment.turbulence_presets`

Resíduos de fechamento de turbulência RANS/LES para treino de PINN: k-omega SST (2D/3D), Spalart-Allmaras e WALE (LES 3D), mais seleção por nome.

pinneapple_physics/pde_environment/turbulence_presets.py · 992 linhas · testes: 49

nome	tipo	assinatura	o que faz
KOmegaSSTResiduals	classe	(nu=1e-05, rho=1.0, consts=None) -> None	Resíduos do modelo k-omega SST (RANS) para PINN, em 2D ou 3D conforme a entrada, com viscosidade turbulenta e resíduos de canal.
SpalartAllmarasResiduals	classe	(nu=1e-05) -> None	Resíduos do modelo de uma equação de Spalart-Allmaras.
WALEResiduals	classe	(nu=1e-05, cw=0.325, delta=1.0, spatial_indices=(0, 1, 2), t_index=3) -> None	Fechamento de subgrade WALE (Nicoud & Ducros 1999) como resíduo LES algébrico 3D.
get_rans_preset	função	(model_name='k-omega-sst', **kwargs)	Cria um objeto de resíduo de turbulência pelo nome; ValueError para nome desconhecido.

pinneapple_physics.pde_environment.turbulence_selector

Seletor unificado de modelo de turbulência: só despacho e validação sobre os resíduos PINN de turbulence_presets e o parâmetro Smagorinsky Cs dos solvers LBM; não implementa física nova.

pinneapple_physics/pde_environment/turbulence_selector.py · 171 linhas · testes: 26

nome	tipo	assinatura	o que faz
TurbulenceModel	classe	()	Enum dos fechamentos disponíveis (laminar, k-omega SST, WALE, Smagorinsky).
get_turbulence_closure	função	(model, dim=3, solver_family='pinn', **kwargs) -> Optional[Any]	Devolve o objeto de fechamento certo (PINN) ou o Cs (LBM) para a família de solver; ValueError para combinação não suportada.

pinneapple_physics.physical_problem

PhysicalProblem reúne geometria, EDP, condições, parâmetros com unidade e papel, quantidades de interesse e fingerprint; adapta de/para ProblemSpec, PhysicsCase e o ProblemSpec do problem design.

pinneapple_physics/physical_problem.py · 534 linhas · testes: 12

nome	tipo	assinatura	o que faz
Parameter	classe	(name, value, unit, role, bounds, distribution, description)	Parâmetro físico nomeado com valor, unidade, papel (fixo, projeto, incerto, desconhecido), limites e distribuição.
Quantity	classe	(name, unit, field, description)	Quantidade de interesse que o usuário quer da solução, com unidade e campo de origem.
PhysicalProblem	classe	(name, coords, fields, pde, conditions, domain_bounds, geometry, geometry_kind, parameter…)	Problema físico completo (geometria, EDP, condições, parâmetros, solver, intenção) com validação, serialização, fingerprint e adaptadores.

pinneapple_physics.physics_audit

Auditoria física: qual modelo usar, quando confiar nele e quando sua formulação física deve ser revista; diagnose junta avaliação e diagnóstico num caso. Numpy puro, sem torch.

pinneapple_physics/physics_audit/__init__.py · 171 linhas · testes: 37

nome	tipo	assinatura	o que faz
Diagnosis	classe	(model, features, audit, validity, attribution, recommendations, unrepresented)	Resultado de diagnose: auditoria de premissas, validade, atribuição, recomendações e mecanismos não representados, com relatório legível.
diagnose	função	(profile, features, prediction=None, query=None, monitor=None, error=None, error_toleranc…)	Audita as premissas do perfil para o caso, roda o monitor na previsão, atribui a causa da falha e recomenda ações.

pinneapple_physics.physics_audit.assumptions

Premissas físicas de um modelo, as condições em que cada uma vale e um auditor que as checa caso a caso. [sem teste direto; usado via physics_audit]

pinneapple_physics/physics_audit/assumptions.py · 420 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
Condition	classe	(feature, lo, hi, band, log)	Condição lo <= feature <= hi com borda suave; classifica como válida, marginal ou violada e dá uma pontuação.
Assumption	classe	(name, description, conditions, breaks_into, missing_mechanism)	Simplificação em que um modelo se apoia, com o fenômeno que aparece quando ela falha e o mecanismo que falta.
AssumptionCheck	classe	(name, status, score, evidence, breaks_into, missing_mechanism, features)	Resultado da verificação de uma premissa num caso.
Envelope	classe	(ranges, samples, sample_features, band)	Onde um modelo foi treinado ou validado: faixas por característica e, opcionalmente, amostras para distância ao vizinho mais próximo.
ModelProfile	classe	(name, family, equations, assumptions, mechanisms, envelope, cost, notes)	O que a auditoria sabe de um modelo: família, equações, premissas, mecanismos, envelope e custo.
AssumptionAudit	classe	(model, status, compatibility, checks, envelope, triggered, missing_mechanisms)	Resultado da auditoria de um modelo num caso: status, compatibilidade, verificações e mecanismos ausentes.
laminar	função	(re_crit=2300.0, feature='Re', geometry='pipe') -> Assumption	Premissa de escoamento laminar abaixo do Reynolds de transição.
turbulent	função	(re_min=4000.0, re_max=math.inf, feature='Re') -> Assumption	Premissa de escoamento totalmente turbulento acima de um Reynolds mínimo.
hydraulically_smooth	função	(eps_plus_max=5.0) -> Assumption	Premissa de parede hidraulicamente lisa (Reynolds de rugosidade baixo).
incompressible	função	(ma_max=0.3) -> Assumption	Premissa de incompressibilidade por limite no número de Mach.
creeping_flow	função	(re_max=1.0) -> Assumption	Premissa de escoamento de Stokes (Reynolds muito baixo).
continuum	função	(kn_max=0.01) -> Assumption	Premissa de meio contínuo por limite no número de Knudsen.
boussinesq	função	(max_beta_dt=0.1) -> Assumption	Premissa de Boussinesq por limite em beta dT.
constant_properties	função	(max_dt_over_t=0.1) -> Assumption	Premissa de propriedades constantes por limite na variação relativa de temperatura.
small_strain	função	(max_strain=0.01) -> Assumption	Premissa de pequenas deformações.
linear_elastic	função	(max_stress_ratio=1.0) -> Assumption	Premissa de material linear elástico por limite na razão de tensões.
lumped_capacitance	função	(bi_max=0.1) -> Assumption	Premissa de capacitância concentrada por limite no número de Biot.
forced_convection	função	(ri_max=0.1) -> Assumption	Premissa de convecção forçada por limite no número de Richardson.
diffusion_dominated	função	(pe_max=1.0) -> Assumption	Premissa de transporte dominado por difusão (Péclet baixo).
advection_dominated	função	(pe_min=10.0) -> Assumption	Premissa de transporte dominado por advecção (Péclet alto).
attached_flow	função	(alpha_stall_deg=12.0) -> Assumption	Premissa de escoamento colado abaixo do ângulo de estol.
shallow_water	função	(max_aspect=0.05) -> Assumption	Premissa de águas rasas por limite na razão de aspecto.
get_assumption	função	(a) -> Assumption	Devolve uma Assumption a partir de um nome do catálogo ou do próprio objeto.
audit_assumptions	função	(profile, features) -> AssumptionAudit	Checa todas as premissas e o envelope de um perfil para as características de um caso.

pinneapple_physics.physics_audit.attribution

Atribuição de falhas físicas: a causa provável de uma previsão errada ou suspeita, com evidências e confiança. [sem teste direto; usado via `physics_audit`]

`pinneapple_physics/physics_audit/attribution.py` · 246 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Cause`	classe	`(code, title, probability, score, evidence)`	Uma causa candidata com código, título, probabilidade, pontuação e evidências.
`Attribution`	classe	`(failed, causes, confidence, evidence_available, error, notes)`	Resultado da atribuição: se falhou, causas ordenadas, confiança e evidências disponíveis.
`attribute_failure`	função	`(*, validity=None, audit=None, error=None, error_tolerance=0.05, residual_baseline=None, …)`	Pondera as evidências de um caso (validade, auditoria, erro, resolução, medição, dispersão) e estima a causa.

pinneapple_physics.physics_audit.improvement

Melhoria guiada pela física: quando nenhum modelo basta, qual experimento ou mudança vale mais. [sem teste direto; usado via `physics_audit`]

`pinneapple_physics/physics_audit/improvement.py` · 179 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Recommendation`	classe	`(action_id, action, cause, priority, gain, cost, rationale, details, cases)`	Ação recomendada com causa, prioridade, ganho, custo e justificativa.
`design_points`	função	`(features, spread=0.15, n=5, seed=0, keys=None) -> list[dict[str, float]]`	Pontos de projeto de alta fidelidade em torno de um caso, por hipercubo latino.
`recommend`	função	`(attribution, audit=None, graph=None, features=None, model=None, min_probability=0.1) -> …)`	Lista ações ordenadas para um caso, nomeando o mecanismo que falta quando há grafo de conhecimento.
`improvement_plan`	função	`(cases, top=None) -> list[Recommendation]`	Agrega recomendações de muitos casos, somando prioridades da mesma ação.

pinneapple_physics.physics_audit.knowledge

Grafo de conhecimento físico: fenômenos, equações, premissas, parâmetros, mecanismos e famílias de modelos e suas relações. [sem teste direto; usado via `physics_audit`]

`pinneapple_physics/physics_audit/knowledge.py` · 328 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Node`	classe	`(id, kind, label, data)`	Nó do grafo com id, tipo, rótulo e dados.
`Edge`	classe	`(src, rel, dst, data)`	Aresta do grafo com origem, relação, destino e dados.
`PhysicsKnowledgeGraph`	classe	`()`	Grafo de conhecimento com consultas sobre mecanismos, premissas, fenômenos disparados e mecanismos não representados.
`default_graph`	função	`(models=()) -> PhysicsKnowledgeGraph`	Catálogo padrão de premissas, fenômenos e equações, mais os modelos dados.

pinneapple_physics.physics_audit.monitor

Monitor de validade física: se uma previsão ainda é plausível sem verdade de referência (resíduo, conservação, limites, contorno, invariantes, extrapolação). [sem teste direto; usado via `physics_audit`]

`pinneapple_physics/physics_audit/monitor.py` · 260 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`CheckResult`	classe	`(name, kind, value, tolerance, passed, critical, detail)`	Resultado de uma verificação de validade: valor, tolerância, aprovado e criticidade.
`ValidityCheck`	classe	`(name, kind, fn, tolerance, critical, describe)`	Verificação `fn(prediction, query) -> valor` que passa quando o valor não excede a tolerância.
`ValidityReport`	classe	`(status, results, alarms, index)`	Relatório de uma previsão: status, resultados, alarmes e índice.
`PhysicsValidityMonitor`	classe	`(checks, persistence=3, drift_ratio=3.0, drift_alpha=0.3, baseline_cases=10, baselines=No…)`	Roda um conjunto de verificações em cada previsão e guarda o histórico, com persistência e detecção de deriva.
`residual_check`	função	`(residual_fn, tolerance, name='pde_residual', critical=False) -> ValidityCheck`	Cria uma verificação sobre a norma do resíduo da EDP.
`conservation_check`	função	`(total_fn, target_fn, rel_tol=0.01, name='conservation', critical=False, abs_tol=0.0) -> …)`	Cria uma verificação de desequilíbrio relativo de uma grandeza conservada.
`bounds_check`	função	`(lo=None, hi=None, name='bounds', select=None, max_fraction=0.0, critical=True) -> Validi…)`	Cria uma verificação da fração de valores fora de `[lo, hi]` (crítica por padrão).
`boundary_check`	função	`(predict_boundary, boundary_value, rel_tol=0.01, name='boundary_condition', critical=Fals…)`	Cria uma verificação do desvio RMS relativo entre a previsão no contorno e a condição de contorno.
`constitutive_check`	função	`(fn, tolerance, name='constitutive', critical=False) -> ValidityCheck`	Cria uma verificação de qualquer relação constitutiva que a previsão deve satisfazer.
`invariant_check`	função	`(invariant_fn, expected, rel_tol=0.01, name='invariant', critical=False, abs_tol=0.0) -> …)`	Cria uma verificação da variação relativa de um invariante.
`envelope_check`	função	`(envelope, features_fn, min_score=0.5, name='training_envelope') -> ValidityCheck`	Cria uma verificação de extrapolação a partir da pontuação do envelope de treino.

`pinneapple_physics.physics_audit.regime`

Números adimensionais que dizem em que física um caso está (Re, Ma, Pe, Kn, Gr, Pr, Ra, Ri, Bi, Fr, St, Da, rugosidade).

`pinneapple_physics/physics_audit/regime.py` · 178 linhas · testes: 31

nome	tipo	assinatura	o que faz
`reynolds`	função	`(velocity, length, nu=None, *, rho=None, mu=None) -> float`	Número de Reynolds.
`mach`	função	`(velocity, sound_speed=None, *, gamma=1.4, R=287.05, T=None) -> float`	Número de Mach, com velocidade do som dada ou de gás ideal.
`peclet`	função	`(velocity, length, diffusivity) -> float`	Número de Péclet.
`knudsen`	função	`(mean_free_path, length) -> float`	Número de Knudsen.
`grashof`	função	`(g, beta, dT, length, nu) -> float`	Número de Grashof.
`prandtl`	função	`(nu, alpha) -> float`	Número de Prandtl.
`rayleigh`	função	`(g, beta, dT, length, nu, alpha) -> float`	Número de Rayleigh.
`richardson`	função	`(g, beta, dT, length, velocity) -> float`	Número de Richardson.
`biot`	função	`(h, length, k) -> float`	Número de Biot.
`froude`	função	`(velocity, length, g=9.81) -> float`	Número de Froude.
`strouhal`	função	`(frequency, length, velocity) -> float`	Número de Strouhal.
`dominant_frequency`	função	`(signal, dt=1.0, discard=0.5, window=True, refine=True, pad=1) -> float`	Frequência do pico espectral mais forte de um sinal amostrado (por exemplo desprendimento de vórtices).
`damkohler`	função	`(reaction_rate, length, velocity) -> float`	Número de Damköhler.

nome	tipo	assinatura	o que faz
`roughness_reynolds`	função	`(re, eps_d, f=None) -> float`	Reynolds de rugosidade `eps+`, com atrito estimado por Haaland se não for dado.
`regime_features`	função	`(*, velocity=None, length=None, nu=None, sound_speed=None, diffusivity=None, alpha=None, …)`	Calcula todos os números adimensionais cujas entradas foram dadas, num dicionário de características.

`pinneapple_physics.physics_audit.routing`

Roteamento de modelos ciente da física: pondera cada modelo pelo quanto o caso atual cabe nas suas premissas, não só pelos erros passados. [sem teste direto; usado via `physics_audit`]

`pinneapple_physics/physics_audit/routing.py` · 142 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`RouteDecision`	classe	`(chosen, weights, compatibility, awake, escalate, audits, reasons)`	Decisão de roteamento: modelo escolhido, pesos, compatibilidades, modelos ativos, escalonamento e motivos.
`PhysicsAwareRouter`	classe	`(profiles, features_fn=None, strength=1.0, floor=0.0001, abstain_below=0.05, escalate_bel…)`	Roteador que usa a compatibilidade física como prior dos pesos do ensemble, com especialistas adormecidos e escalonamento ao solver.

`pinneapple_physics.pinn_solver`

Subpacote do compilador de PINN; reexporta `compile_problem`, pesos de perda, DoMINO e utilidades de PINN estocástico.

`pinneapple_physics/pinn_solver/__init__.py` · 28 linhas · testes: 1

`pinneapple_physics.pinn_solver.compiler`

Reexporta `compile_problem`, `LossWeights`, `AdaptiveWeights`, `SingleBatchDataset` e `dict_collate`.

`pinneapple_physics/pinn_solver/compiler/__init__.py` · 11 linhas · testes: 5

`pinneapple_physics.pinn_solver.compiler.autograd_ops`

Operadores diferenciais por autograd (gradiente, jacobiano, divergente, laplaciano, derivada temporal, derivada normal) com cache opcional de derivadas.

`pinneapple_physics/pinn_solver/compiler/autograd_ops.py` · 153 linhas · testes: 8

nome	tipo	assinatura	o que faz
`derivative_cache`	função	`() -> Iterator[Dict[str, int]]`	Gerenciador de contexto que memoiza primeiras derivadas dentro de uma avaliação de perda e conta acertos e falhas.
`ensure_tensor`	função	`(y)`	Extrai um tensor da saída do modelo (tensor puro ou objeto com atributo de saída).
`grad`	função	`(y, x) -> torch.Tensor`	Gradiente de um campo escalar em relação às entradas, com grafo para derivadas de ordem maior.
`jacobian`	função	`(Y, x) -> torch.Tensor`	Jacobiano de um campo vetorial em relação às entradas.
`divergence`	função	`(V, x, coord_indices=None) -> torch.Tensor`	Divergente de um campo vetorial.
`laplacian`	função	`(y, x, coord_indices=None) -> torch.Tensor`	Laplaciano (soma das segundas derivadas não mistas).
`time_derivative`	função	`(y, x, t_index) -> torch.Tensor`	Derivada em relação à coordenada de tempo.
`norm_dot_grad`	função	`(y, x, normals) -> torch.Tensor`	Derivada normal `n . grad y`.
`mse`	função	`(a, b) -> torch.Tensor`	Erro quadrático médio entre dois tensores.

pinneapple_physics.pinn_solver.compiler.compile

O compilador central: transforma um `ProblemSpec` numa função de perda por termo, com resíduos para dezenas de tipos de EDP (Laplace, Poisson, Navier-Stokes, elasticidade, Maxwell, Black-Scholes, órbitas, etc.) e perdas de condições Dirichlet, Neumann, Robin, interface, inicial e dados; tipo desconhecido levanta `ValueError`.

`pinneapple_physics/pinn_solver/compiler/compile.py` · 2.604 linhas · testes: 121

nome	tipo	assinatura	o que faz
`compile_problem`	função	`(spec, *, weights=None, cache_derivatives=False)`	Compila o `spec` em `loss_fn(model, y_hat, batch) -> {nome: perda}`; `cache_derivatives=True` compartilha derivadas entre termos.

pinneapple_physics.pinn_solver.compiler.dataset

Dataset de lote único e função de collate para lotes PINN em dicionário. [sem teste direto]

`pinneapple_physics/pinn_solver/compiler/dataset.py` · 89 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`SingleBatchDataset`	classe	`(batch)`	Embrulha um lote pronto (por exemplo de `STLDomainBatchBuilder`) num `Dataset`, ciclando pontos de contorno, inicial e dados.
`dict_collate`	função	`(batch) -> Dict[str, Any]`	Junta uma lista de lotes PINN em dicionário (usa `pinneapple_data.collate` se disponível).

pinneapple_physics.pinn_solver.compiler.em_kinds

Tipos eletromagnéticos do compilador: equações de Maxwell em forma diferencial (eletrostática, magnetostática, correntes parasitas, TM, 3-D, frequência) e integral (volumes de controle). [sem teste direto]

`pinneapple_physics/pinn_solver/compiler/em_kinds.py` · 261 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`em_residuals`	função	`(kind, *, fields, xcol, coords, params, ctx, model, field_names, split_fields) -> list[to…)`	Devolve a lista de resíduos nos pontos de colocação para um tipo eletromagnético.

pinneapple_physics.pinn_solver.compiler.loss

Pesos de perda: `LossWeights` fixos lidos por `compile_problem` e `AdaptiveWeights` autonormalizados para laços de treino próprios.

`pinneapple_physics/pinn_solver/compiler/loss.py` · 107 linhas · testes: 20

nome	tipo	assinatura	o que faz
`LossWeights`	classe	`(w_pde, w_bc, w_ic, w_data)`	Pesos fixos dos termos de EDP, contorno, inicial e dados.
`AdaptiveWeights`	classe	`(base_weights, momentum, eps, max_ratio, enabled, _ema)`	Pesos adaptativos por nome de termo, a partir de média móvel exponencial defasada da perda bruta, limitados por `max_ratio`.

pinneapple_physics.pinn_solver.domino

DoMINO: decomposição de domínio para PINN, com um PINN por subdomínio retangular sobreposto e perda de continuidade na interface. [sem teste direto]

`pinneapple_physics/pinn_solver/domino.py` · 385 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Subdomain`	classe	`(bounds, overlap=0.1)`	Subdomínio retangular com sobreposição e testes de pertinência e interface.
`SubdomainPINN`	classe	`(subdomain, model)`	PINN de um único subdomínio.

nome	tipo	assinatura	o que faz
`DoMINO`	classe	`(subdomains, model_factory, interface_weight=10.0)`	Treinador de PINN por decomposição de domínio: particiona, combina as saídas e penaliza descontinuidade na interface.

`pinneapple_physics.pinn_solver.factory`

Fábrica PINN legada baseada em SymPy; reexporta `NeuralNetwork`, `PINN`, `PINNFactory`, `PINNProblemSpec` e indica `VanillaPINN` de `pinneapple_neural` como modelo preferido.

`pinneapple_physics/pinn_solver/factory/__init__.py` · 23 linhas · testes: 7

`pinneapple_physics.pinn_solver.factory.autodiff`

Cálculo e cache de derivadas por autograd exigidas pelos átomos `Derivative` do SymPy. [sem teste direto]

`pinneapple_physics/pinn_solver/factory/autodiff.py` · 166 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`DerivativeKey`	classe	`(base, wrt)`	Chave estável para guardar derivadas em cache.
`DerivativeComputer`	classe	`(independent_vars, dependent_vars, device, coord_scales=None) -> None`	Calcula e guarda as derivadas por autograd pedidas pelas expressões SymPy.
`ensure_requires_grad`	função	`(inputs) -> None`	Liga `requires_grad=True` em todos os tensores de entrada.

`pinneapple_physics.pinn_solver.factory.pinn_factory`

Fábrica de PINN: compila equações simbólicas e monta uma perda unificada; `NeuralNetwork` e `PINN` são shims de compatibilidade sobre `VanillaPINN`. [shim parcial]

`pinneapple_physics/pinn_solver/factory/pinn_factory.py` · 412 linhas · testes: 1

nome	tipo	assinatura	o que faz
`NeuralNetwork`	classe	`(num_inputs, num_outputs, num_layers, num_neurons, activation=nn.Tanh())`	[shim] MLP compatível com a API antiga `(num_inputs, num_outputs, num_layers, num_neurons)`, sobre `VanillaPINN`.
`PINN`	classe	`(neural_network, inverse_params_names=None, initial_guesses=None, dtype=torch.float32)`	[shim] Rede mais parâmetros inversos treináveis, mantida só por compatibilidade de API.
`PINNProblemSpec`	classe	`(pde_residuals, independent_vars, dependent_vars, conditions, inverse_params, loss_weight…)`	Especificação PINN independente de modelo: resíduos em string, variáveis, condições, parâmetros inversos, pesos e sinais exógenos.
`PINNFactory`	classe	`(spec)`	Compila as equações SymPy em resíduos torch e gera uma função de perda unificada.

`pinneapple_physics.pinn_solver.factory.signals`

Provedores de sinais exógenos (dados de sensor, forçamento) para `PINNFactory`. [sem teste direto]

`pinneapple_physics/pinn_solver/factory/signals.py` · 62 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`TabulatedSignal`	classe	`(x, values, *, input_index=0, extrapolate='clamp')`	Interpolação linear por partes de um sinal amostrado, avaliada em torch, com extrapolação constante ou linear.

`pinneapple_physics.pinn_solver.factory.sympy_backend`

Compilador de equações SymPy para lambdas torch via `lambdify`; o gradiente não atravessa a avaliação simbólica (para isso o código indica `SymbolicPDE`). [sem teste direto]

`pinneapple_physics/pinn_solver/factory/sympy_backend.py` · 243 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`CompiledEquation`	classe	`(expr, derivatives, call, args_order, exogenous, runtime_constants)`	Equação compilada: expressão, derivadas necessárias, função chamável e ordem dos argumentos.
`SympyTorchCompiler`	classe	`(independent_vars, dependent_vars, inverse_params=None, extra_namespace=None)`	Compila strings de equações em lambdas torch usando SymPy.
`SympyBackend`	classe	`(*, extra_namespace=None) -> None`	[shim] Embrulho de compatibilidade que só devolve um `SympyTorchCompiler`.

`pinneapple_physics.pinn_solver.io`

Reexporta mapeamentos e o leitor de dados UPD; `UPDItem`, `ConditionSpec`, `SamplingSpec` e `Batch` vêm de `pinneapple_data.upd_types`. [sem teste direto]

`pinneapple_physics/pinn_solver/io/__init__.py` · 28 linhas · **sem teste direto**

`pinneapple_physics.pinn_solver.io.mappings`

Mapeamentos entre coordenadas e variáveis UPD e as variáveis independentes e dependentes do PINN, com transformações (segundos, graus para radianos, normalização). [sem teste direto]

`pinneapple_physics/pinn_solver/io/mappings.py` · 197 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`CoordMapping`	classe	`(ind_vars, coord_sources, coord_transform, normalize_to_unit, _deriv_scale)`	Mapeia coordenadas UPD para variáveis independentes do PINN, com transformação e escala de derivadas.
`VarMapping`	classe	`(dep_vars, upd_to_dep, enforce_present)`	Mapeia nomes de variáveis UPD para variáveis dependentes do PINN.
`PINNMapping`	classe	`(coord, var)`	Mapeamento completo de coordenadas e variáveis.
`build_default_mapping_atmosphere`	função	`(dep_vars, ind_vars=None, use_lev=False, time_transform='seconds_since_start', angle_tran…)`	Mapeamento padrão para fragmentos de reanálise atmosférica UPD (tempo, lat, lon e, opcionalmente, nível).

`pinneapple_physics.pinn_solver.io.upd_dataset`

Leitor de dataset UPD (Zarr + JSON, via xarray) que produz amostras prontas para PINN. [sem teste direto]

`pinneapple_physics/pinn_solver/io/upd_dataset.py` · 253 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`UPDDataset`	classe	`(item, mapping, device='cpu', dtype=torch.float32)`	Lê UPD e gera pontos de colocação, pontos de condição e pares supervisionados opcionais.

`pinneapple_physics.pinn_solver.schemas`

Pacote vazio que contém o adaptador de schema. [sem teste direto]

`pinneapple_physics/pinn_solver/schemas/__init__.py` · 1 linhas · **sem teste direto**

`pinneapple_physics.pinn_solver.schemas.schema_adapter`

Adaptador de Physical Schema (UPD) para `PINNProblemSpec` e especificações de amostragem; o próprio docstring descreve premissas de MVP. [sem teste direto]

`pinneapple_physics/pinn_solver/schemas/schema_adapter.py` · 325 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`SchemaAdapterOptions`	classe	`(default_loss_weights, default_condition_weight, build_sampling_from_conditions)`	Opções da conversão de schema: pesos padrão de perda, peso de condição e se cria amostragem a partir das condições.

nome	tipo	assinatura	o que faz
`SchemaAdapter`	classe	`(mapping, options=None) -> None`	Converte um schema físico em `PINNProblemSpec` (para `PINNFactory`) e, opcionalmente, em especificações de amostragem para `UPDDataset`.

pinneapple_physics.pinn_solver.stochastic

Treino de PINN estocástico condicionado por código latente: a rede aprende uma distribuição de saídas físicas e casa média e covariância do ensemble com estatísticas reais.

`pinneapple_physics/pinn_solver/stochastic.py` · 378 linhas · testes: 15

nome	tipo	assinatura	o que faz
`LatentConditionedModel`	classe	`(base, latent_dim) -> None`	Embrulha um modelo para aceitar também um código latente `xi`, concatenado à entrada.
`CorrelatedLatentField`	classe	`(bounds, coord_order, spatial_axes, periodic_axes, latent_dim, grid_res=(64, 24, 32), len…)`	Campo latente espacialmente correlacionado, alternativa ao sorteio independente por ponto.
`sample_latent`	função	`(n, latent_dim, device=None, dtype=torch.float32) -> torch.Tensor`	Sorteia códigos latentes `N(0, I)`.
`ensemble_forward`	função	`(model, x, latent_dim, n_samples, latent_sampler=None) -> torch.Tensor`	Avalia várias amostras latentes em cada ponto numa única passagem em lote.
`mean_covariance_loss`	função	`(model, x, latent_dim, n_samples, mean_target, cov_target=None, cov_index_pairs=None, fie…)`	Perda que casa a média e, opcionalmente, a covariância Monte Carlo do ensemble latente com estatísticas fornecidas.

pinneapple_physics.solid

Reexporta as funções de elasticidade axissimétrica. [sem teste direto do `__init__`]

`pinneapple_physics/solid/__init__.py` · 13 linhas · **sem teste direto**

pinneapple_physics.solid.axisymmetric

Elasticidade linear axissimétrica em (r, z) por autograd: deformações, tensões, resíduo de equilíbrio e von Mises, com torção opcional.

`pinneapple_physics/solid/axisymmetric.py` · 122 linhas · testes: 8

nome	tipo	assinatura	o que faz
`lame_parameters`	função	`(E, nu) -> tuple[float, float]`	Calcula (λ, μ) a partir de E e ν.
`strains_rz`	função	`(u, x, *, r_min=None, create_graph=True) -> dict[str, torch.Tensor]`	Deformações pequenas em (r, z) por autograd, com componentes de torção quando há `u_theta`.
`stresses_rz`	função	`(u, x, E, nu, *, r_min=None, create_graph=True) -> dict[str, torch.Tensor]`	Tensões pela lei de Hooke a partir das deformações.
`equilibrium_residual_rz`	função	`(u, x, E, nu, *, r_min=None) -> torch.Tensor`	Resíduo pontual das equações de equilíbrio axissimétricas.
`von_mises_from_components`	função	`(s_rr, s_zz, s_tt, s_rz=0.0, s_rt=0.0, s_tz=0.0) -> Any`	Tensão de von Mises a partir das componentes de tensão.
`von_mises_rz`	função	`(u, x, E, nu, *, r_min=None, create_graph=False) -> torch.Tensor`	Tensão de von Mises de um campo de deslocamento `u`.

pinneapple_physics.solving

`pp.solve`/`pp.compare` com registro de métodos (`pinn`, `fem`, `analytic`, `reference`, `exact`, `external`) e de soluções analíticas; toda `Solution` tem `predict` e `metadata` uniformes.

`pinneapple_physics/solving.py` · 594 linhas · testes: 71

nome	tipo	assinatura	o que faz
`MethodNotAvailable`	classe	`()`	Exceção: o método pedido não pode resolver este problema.
`Solution`	classe	`(problem, method, predict_fn, wall_time_s, history, info, model, grid, kind)`	Resultado de `solve`: função `predict`, tempo, histórico, modelo, grade e tipo do método, com `metadata` uniforme.
`Comparison`	classe	`(problem, reference, n_points, rows)`	Tabela de `compare`: métricas de cada método contra a referência nos mesmos pontos, com `best`.
`as_problem`	função	`(problem, **preset_kwargs) -> PhysicalProblem`	Converte preset, `ProblemSpec` ou `PhysicalProblem` em `PhysicalProblem`.
`register_method`	função	`(name, kind='custom')`	Decorador que registra um método de solução sob um nome e um tipo.
`list_methods`	função	`(kind=None) -> List[str]`	Nomes dos métodos registrados, opcionalmente filtrados por tipo.
`list_kinds`	função	`() -> Dict[str, List[str]]`	Métodos agrupados por tipo.
`register_analytic`	função	`(name, matcher)`	Decorador que registra uma solução fechada para os problemas em que `matcher` é verdadeiro.
`list_analytic`	função	`() -> List[str]`	Nomes das soluções analíticas registradas.
`analytic_for`	função	`(problem) -> Optional[str]`	Nome da solução fechada conhecida para o problema, ou `None`.
`solve`	função	`(problem, method='pinn', *, preset_kwargs=None, **options) -> Solution`	Resolve o problema com o método pedido (nome registrado ou callable).
`compare`	função	`(problem, methods, *, reference='reference', options=None, points=None, n_points=4096, se…)`	Resolve ou reaproveita cada método e pontua contra a referência nos mesmos pontos.

pinneapple_physics.symbolic_pde

Compilador simbólico de EDP: transforma resíduos SymPy em funções PyTorch com autograd e oferece condições de contorno rígidas e suaves; reexporta `SymbolicPDE`, `HardBC` etc. [sem teste direto do **init**]

`pinneapple_physics/symbolic_pde/__init__.py` · 52 linhas · **sem teste direto**

pinneapple_physics.symbolic_pde.bc

Imposição de condições de contorno: rígidas por construção (`HardBC` com função distância, `PeriodicBC`, `MultiPeriodicBC`) e suaves por penalidade (`DirichletBC`, `NeumannBC`). [sem teste direto]

`pinneapple_physics/symbolic_pde/bc.py` · 447 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`HardBC`	classe	`(distance_fn, bc_value_fn) -> None`	Condição de contorno rígida por função distância: u = g + phi * rede, satisfeita exatamente; embrulha o modelo.
`PeriodicBC`	classe	`(period, dim=0) -> None`	Periodicidade exata numa coordenada por mudança de variável (seno/cosseno).
`MultiPeriodicBC`	classe	`(periods) -> None`	Periodicidade exata em dois ou mais eixos de uma vez.
`DirichletBC`	classe	`(value_fn, boundary_fn=None, weight=1.0) -> None`	Condição de Dirichlet suave por penalidade MSE.
`NeumannBC`	classe	`(flux_fn, normal_fn, boundary_fn=None, weight=1.0) -> None`	Condição de Neumann suave pela derivada normal via autograd.

pinneapple_physics.symbolic_pde.compiler

`SymbolicPDE`: percorre a árvore SymPy e avalia o resíduo com `torch.autograd.grad` (gradientes fluem pela expressão), ou em grade com diferenças finitas/espectral. Levanta NotImplementedError para tipos SymPy não suportados e para coordenada solta no modo grade.

`pinneapple_physics/symbolic_pde/compiler.py` · 771 linhas · testes: 5 · palavras-chave no código (heurística; o texto acima diz o status real): levanta `NotImplementedError` em algum caminho

nome	tipo	assinatura	o que faz
SymbolicPDE	classe	(expr, coord_syms, field_syms, param_syms=None, grad_method='autograd', grid=None) -> None	Resíduo de EDP simbólico compilado para função PyTorch (autograd ou grade).
pde_from_sympy	função	(expr, coords, fields, params=None, grad_method='autograd', grid=None) -> SymbolicPDE	Cria um SymbolicPDE a partir de uma expressão de resíduo SymPy.
auto_residual	função	(model, coords_tensor, derivative_fns) -> Dict[str, torch.Tensor]	Calcula valores do campo e derivadas nomeadas via autograd.

pinneapple_physics.symbolic_pde.gradient_backends

Backends de derivada plugáveis para SymbolicPDE em dados de grade: mapeia coordenadas para eixos e compõe derivadas repetidas e mistas usando fd_derivative/spectral_derivative do PINO.

pinneapple_physics/symbolic_pde/gradient_backends.py · 130 linhas · testes: 5

nome	tipo	assinatura	o que faz
GridAxisSpec	classe	(coord_names, dims, L)	Liga nomes de coordenadas SymPy a eixos e comprimentos de um tensor de campo em grade.
grid_derivative	função	(field, wrt, *, grid, method) -> torch.Tensor	Derivada parcial de ordem qualquer de um campo em grade por diferenças finitas ou método espectral.

pinneapple_physics.symbolic_pde.pinn

Converte um sistema de equações analisado por .text num PINNProblem de pinneapple_analysis, com resíduo por autograd e coeficientes desconhecidos treináveis (PINN inverso).

pinneapple_physics/symbolic_pde/pinn.py · 194 linhas · testes: 10

nome	tipo	assinatura	o que faz
PINNWithParameters	classe	()	Placeholder substituído na primeira utilização pela classe torch que junta a rede e os coeficientes treináveis do problema inverso. [placeholder]
pinn_problem	função	(system, bounds, *, coefficients=None, unknowns=None, width=32, depth=4, exact=None, name…)	Monta (PINNProblem, modelo) para um sistema numa caixa, com coeficientes desconhecidos opcionais e solução exata opcional.
has_field	função	(expr) -> bool	Diz se a expressão contém algum campo (função não definida do SymPy).

pinneapple_physics.symbolic_pde.text

Equações como texto, analisadas com segurança para SymPy via ast (sem eval/sympify): atalhos u_t, u_xx, d(...), lap(...), sistemas com condições, classificação de família e compilação sem lambdify.

pinneapple_physics/symbolic_pde/text.py · 486 linhas · testes: 17

nome	tipo	assinatura	o que faz
EquationSyntaxError	classe	()	Erro de texto que não é uma equação aceita; a mensagem diz o quê e onde.
Context	classe	(coords, fields, time)	Contexto de análise: coordenadas, campos e coordenada de tempo.
Equation	classe	(text, lhs, rhs, coefficients, kind, on)	Uma equação analisada: texto, lados esquerdo e direito, coeficientes, tipo e onde vale; dá o resíduo e as derivadas.
EquationSystem	classe	(ctx, pdes, conditions, coefficients, unknowns)	Sistema de EDPs e condições, com coeficientes e incógnitas; lista problemas, substitui valores e classifica.
parse_expression	função	(text, ctx) -> tuple[sp.Expr, set[str]]	Analisa um texto sem '=' como expressão SymPy e devolve os coeficientes usados.
parse_equation	função	(text, ctx, *, kind='pde', on='') -> Equation	Analisa um texto 'lhs = rhs' como Equation; uma EDP precisa envolver um campo.
parse_system	função	(pdes, conditions=None, *, coords, fields, coefficients=None, unknowns=None, time='t') ->…)	Analisa textos de EDP e condições num EquationSystem; o erro nomeia o primeiro texto inválido.

nome	tipo	assinatura	o que faz
`classify`	função	`(system) -> dict[str, Any]`	Família de EDP de um sistema de um campo (calor, onda, Poisson, Burgers etc. ou 'custom'), ordens e linearidade.
`derivative_key`	função	`(dv) -> str`	Chave de ambiente de uma derivada de campo (ex.: `u__xx`).
`derivative_keys`	função	`(expr) -> dict[str, tuple[str, str]]`	Mapa chave -> (campo, letras) de todas as derivadas de campo na expressão.
`compile_expr`	função	`(expr, backend='numpy')`	Função env -> valor construída como árvore de closures (sem eval nem lambdify).

`pinneapple_physics.tracking`

Envia parâmetros, métricas, tempo e o registro de um `ExperimentResult` ao MLflow, W&B ou a um callable; os trackers são dependências opcionais.

`pinneapple_physics/tracking.py` · 82 linhas · testes: 3

nome	tipo	assinatura	o que faz
`flat_metrics`	função	`(result) -> Dict[str, float]`	Achata as métricas de um resultado em chaves `<métrica>/<campo>`.
`flat_params`	função	`(result) -> Dict[str, str]`	Achata a config de um resultado em parâmetros texto.
`log_result`	função	`(result, tracker, **kw) -> Any`	Envia o resultado ao tracker ("mlflow", "wandb" ou callable) e devolve o id da execução.

`pinneapple_physics.tribology`

Reexporta o modelo de desgaste de Archard.

`pinneapple_physics/tribology/__init__.py` · 5 linhas · testes: 20

`pinneapple_physics.tribology.archard`

Desgaste de uma barra abaulada contra contraface rígida: lei de Archard local com pressão de camada de Winkler redistribuída a cada passo, do running-in ao desgaste estacionário. [sem teste direto pelo inventário]

`pinneapple_physics/tribology/archard.py` · 136 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`WearMaterial`	classe	`(name, k, hardness_MPa, E_GPa)`	Material de desgaste com coeficiente k, dureza e módulo; `K` dá a taxa específica de desgaste.
`BarWear`	classe	`(material, load_N, length_mm, width_mm, crown_radius_mm, layer_mm, nx)`	Barra abaulada pressionada com carga F; `run` integra o desgaste ao longo da distância de deslizamento.
`winkler_parabolic_contact`	função	`(F_N, width_mm, R_mm, k_w) -> dict[str, float]`	Solução fechada do contato de folga parabólica numa camada de Winkler.

`pinneapple_physics.weather`

Subpacote de previsão global do tempo; o `__init__` está vazio (os módulos são importados diretamente). [sem teste direto]

`pinneapple_physics/weather/__init__.py` · 1 linhas · **sem teste direto**

`pinneapple_physics.weather.data`

Baixa ERA5 do WeatherBench2 (bucket público, grade 64 x 32) para um array local normalizado e calcula a climatologia para escores de anomalia. [sem teste direto]

`pinneapple_physics/weather/data.py` · 176 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Era5Store`	classe	`(path)`	Leitura preguiçosa (memory map) do ERA5 normalizado gravado por `download`.
`channel_names`	função	`(levels=LEVELS) -> list[str]`	Nomes dos canais: variáveis de altitude em cada nível mais as de superfície.
`download`	função	`(dest, years=(1990, 2022), levels=LEVELS, url=ERA5_64x32, log=print, normalization=None) …)`	Baixa e normaliza o ERA5 para `state.npy`, `constants.npy`, `times.npy` e `meta.json`.
`climatology`	função	`(store, url=CLIMATOLOGY_64x32) -> np.ndarray`	Climatologia normalizada do WeatherBench2, guardada em cache ao lado do store.
`climatology_at`	função	`(clim, time) -> np.ndarray`	Seleciona a climatologia do dia do ano e da hora de um instante.

`pinneapple_physics.weather.evaluate`

Habilidade de previsões globais por antecedência (RMSE e ACC ponderados por área) e horizonte útil de cada previsão.

`pinneapple_physics/weather/evaluate.py` · 154 linhas · testes: 8

nome	tipo	assinatura	o que faz
`PublishedForecasts`	classe	`(store, names=None)`	Previsões publicadas no WeatherBench2 na grade 64 x 32, no layout de canais do store (só inicializações 00 e 12 UTC).
`rmse`	função	`(f, o, w) -> np.ndarray`	RMSE ponderado por área em latitude e longitude.
`acc`	função	`(f, o, clim, w) -> np.ndarray`	Coeficiente de correlação de anomalias ponderado por área.
`first_crossing`	função	`(leads_h, values, threshold, below=True) -> float`	Antecedência interpolada em que uma curva cruza um limiar pela primeira vez.
`horizons`	função	`(leads_h, acc_curve, rmse_curve, clim_rmse) -> dict`	Horizonte útil (ACC acima de 0,6) e horizonte sem habilidade (RMSE acima do da climatologia).
`score_forecasts`	função	`(store, clim, forecasts, init_idx, steps, variables=SCORED) -> dict`	Pontua várias previsões (RMSE e ACC por variável e antecedência) a partir de inicializações dadas.
`persistence`	função	`(store)`	Previsão de referência por persistência do estado inicial.
`climatology_forecast`	função	`(store, clim)`	Previsão de referência pela climatologia.

`pinneapple_physics.weather.events`

Estudos de caso de tempo extremo e os estados ERA5 em torno deles, do store de treino ou do ERA5 ARCO em resolução completa médio na grade do modelo.

`pinneapple_physics/weather/events.py` · 156 linhas · testes: 8

nome	tipo	assinatura	o que faz
`Event`	classe	`(key, title, init, peak, var, region, center, what, score, vmin, vmax, extra_inits)`	Descrição de um evento extremo: inicialização, pico, variável, região, centro do globo e forma de resumir.
`by_key`	função	`(key) -> Event`	Busca um evento do catálogo pela chave.
`coarsen`	função	`(field, src_lat, src_lon, lat, lon) -> np.ndarray`	Média ponderada por área de um campo fino em cada célula do modelo.
`arco_states`	função	`(store, times, log=print) -> np.ndarray`	Estados normalizados na grade do store lidos do ERA5 ARCO, com precipitação de 6 h somada.
`event_states`	função	`(store, event, leads, cache_dir=None, log=print) -> tuple[np.ndarray, np.ndarray]`	Instantes e estados normalizados em torno de um evento, do store ou do ARCO.
`region_mask`	função	`(lat, lon, box) -> np.ndarray`	Máscara booleana de uma caixa lat/lon, tratando a passagem por 360 graus.

`pinneapple_physics.weather.model`

Rede de previsão global numa grade lat-lon: U-Net com convoluções esféricas que prevê a variação de 6 horas do estado.

pinneapple_physics/weather/model.py · 122 linhas · testes: 8

nome	tipo	assinatura	o que faz
SConv	classe	(cin, cout, k=3)	Convolução esférica: preenchimento periódico em longitude e através dos polos antes da convolução.
Block	classe	(cin, cout)	Bloco residual com duas convoluções esféricas 3x3, group norm e GELU.
SphereUNet	classe	(n_state, n_extra, widths=(128, 192, 256, 384), blocks=2, patch=1, refine=64)	U-Net esférica, com opção de dobrar células em canais (patch) e camada de suavização em resolução total.
sphere_pad	função	(x, p) -> torch.Tensor	Preenche um tensor (B, C, lat, lon): circular em longitude e através dos polos em latitude.
toa_insolation	função	(times, lat, lon) -> np.ndarray	Cosseno do ângulo zenital solar cortado em 0 (insolação normalizada no topo da atmosfera).
static_fields	função	(lat, lon, constants) -> np.ndarray	Campos fixos de entrada: constantes (máscara terra-mar, orografia) mais seno e cosseno de latitude e longitude.

pinneapple_physics.weather.train

Treino da rede de previsão: um passo de 6 horas primeiro, depois rollouts mais longos, com MSE ponderado por área e por canal. [sem teste direto]

pinneapple_physics/weather/train.py · 198 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
Forecaster	classe	(store, model)	Rede treinada com o necessário para fazer rollout a partir de um instante ERA5, salvar e carregar.
train	função	(store_path, out, *, train_years=(1990, 2017), val_years=(2018, 2019), widths=(192, 256, …)	Treina por um tempo de parede dividido por uma agenda de comprimentos de rollout e guarda o melhor checkpoint na validação.

pinneapple_physics.weather.viz

Visualizações de previsão contra verdade: globos lado a lado, antecedência correndo e faixa de habilidade (ACC), em GIF e figuras.

pinneapple_physics/weather/viz.py · 399 linhas · testes: 8

nome	tipo	assinatura	o que faz
display_units	função	(var, x) -> np.ndarray	Converte variáveis para unidades de exibição (°C, decâmetros, hPa, mm).
coastlines	função	() -> list[np.ndarray]	Carrega as linhas de costa do arquivo coastline_110m.npz do pacote.
orthographic	função	(field, lat, lon, center=(-15.0, -50.0), size=420)	Imagem do campo no hemisfério visível centrado num ponto (projeção ortográfica).
project_lines	função	(lines, center, size=420)	Projeta linhas (costas) na mesma projeção ortográfica.
regional	função	(field, lat, lon, box, size=(360, 260))	Imagem equirretangular interpolada de uma região.
regional_lines	função	(lines, box, size=(360, 260))	Projeta linhas na imagem regional.
forecast_gif	função	(path, var, truth, forecast, lat, lon, times, *, skill_leads_h, skill_acc, horizon, cente…)	GIF animado com ERA5 e previsão lado a lado por antecedência e a faixa de habilidade.
fill_gaps	função	(x, y) -> np.ndarray	Interpolação linear sobre NaNs (modelos publicados a cada 12 h num eixo de 6 h).
skill_figure	função	(path, leads_h, curves, horizons_by_model, var, metric='acc', dpi=110)	Figura de habilidade por antecedência de cada modelo com o horizonte útil marcado.
upsample_sphere	função	(field, lat, lon, factor=4)	Reamostragem cúbica de um campo lat/lon, periódica em longitude, só para exibição.
earth2_gif	função	(path, var, truth, forecast, lat, lon, times, *, lead_hours, horizon, title, center=(20.0…)	Animação em estilo de globos iluminados com rotação, quadros interpolados e barra de antecedência colorida pelo horizonte.

Exemplos de uso reais do repositório

Treinar um preset com `solve_pde` (pontos de tag vindos de geometria) — examples/pde_environment/07_tagged_presets_real_geometry.py

```
batch = build_tag_batch(name, spec, n_col=4000, n_bc_per_face=500, seed=1)
print("tag_masks sizes:", {k: int(v.sum()) for k, v in batch["ctx"]["tag_masks"].items()})

before = per_tag_losses(spec, model, batch)
print("per-tag losses BEFORE training:", before)

kwargs = solve_pde_kwargs_from_batch(batch)
result = pp.solve_pde(spec, model, epochs=epochs, n_collocation=4000, seed=1, lr=1e-3, **kwargs)
losses = result["history"]["loss"]
```

Sintonia de ponto excepcional e auditoria das premissas do teorema — examples/non_hermitian/ep_tuning.py

```
for _key, title, b, b_ba in coupling_classes():
    print(f"\n== {title}")
    print(analyze_ep_tuning(b, b_ba))

print("\n== numerical check (J = 1, G = 0.5)")
d_a = PTDimer(route_a_sqrt_coupling(1.0, 0.5))
ab, ba = route_c_asymmetric(1.0, 0.5)
d_c = PTDimer(ab, ba)
d_l = PTDimer(lambda x: 1.0 + 0.25 * x)
```

Verificações de Darcy sem referência sobre previsões de outro framework — examples/vs_physicsnemo/07_physicsnemo_trains_pinneapple_checks/run.py

```
ref = dc.darcy_checks(data["a"], u, DARCY["f"], reference=data["u"], envelope=envelope)
drift = dc.darcy_checks(data["a"], u, DARCY["f"], tolerances=calibrated[m], physics_kind="drift")
```

EDP simbólica com condição de contorno rígida — examples/pinn_solver/03_symbolic_pde_hard_bc.py

```
hard_bc = HardBC(distance_fn=phi, bc_value_fn=g_bc)

# ---- Base network (input: 2, output: 1) ----
base_net = MLP(in_dim=2, out_dim=1, width=width, depth=depth).to(device=device, dtype=dtype)

# ---- Wrapped model: satisfies u=0 on boundary by construction ----
model = hard_bc.wrap_model(base_net)

# ---- Residual function from SymPy expression ----
residual_fn = pde.to_residual_fn(model)
```

Simular uma rodada de acreção — examples/black_hole_weather/simulate.py

```
cfg = RIAFConfig(nr=a.nr, ntheta=a.ntheta, alpha=a.alpha, viscosity=a.viscosity, torus_a=a.torus_a,
seed=a.seed,
                 perturbation_kind=a.perturbation_kind, backend=a.backend, cfl=a.cfl)
simulate_run(cfg, os.path.join(a.out, a.name), t_end=a.t_end, every=a.every, chunk=a.chunk, name=a.name,
             log=lambda m: print(m, flush=True))
```

Equação em texto (docstring de symbolic_pde.text) — pinneapple_physics/symbolic_pde/text.py

```
sys_ = parse_system(["u_t = alpha*lap(u)"], [{"kind": "dirichlet", "on": "x_min", "equation": "u = 0"}],
                    coords=["x", "t"], fields=["u"], coefficients={"alpha": 0.1})
sys_.classify()["family"]       # "heat"
```

Testes e status de validação

- Módulos com pelo menos um teste que os importa: 114 de 166.
- Funções de teste em arquivos que importam algum módulo do pacote: 1.075.

Itens do catálogo de métodos (`pinneapple_catalog/method_status.json`) cujo código está neste pacote:

id	método	status	testes contra referência	outros testes
E55	S-N curve + Goodman + Miner	validated	1	14
E58	Fin array conduction	tested	0	4
E63	Helmholtz resonator	tested	0	4
E64	Membrane diffusion	tested	0	5
E65	Cantilever fatigue	validated	1	4
S38	Maxwell physics checks of predicted fields (Gauss, Faraday, Ampere-Maxwell, continuity, Poynting, interfaces, reciprocity, passivity)	validated	4	2
T10	Domain decomposition: XPINN, DoMINO/D3M	tested	0	2

Limitações conhecidas

- `solve_pde` não deriva regiões de condições `selector_type="tag"` a partir de `domain_bounds`: sem pontos e máscaras vindos de geometria real, levanta `TagConditionsUnresolved`.
- `compile_problem` só aceita os tipos de EDP implementados em `compile.py/em_kinds.py`; um tipo desconhecido levanta `ValueError("Unsupported PDE kind")`. Nos tipos eletromagnéticos diferenciais, saltos dos mapas de material não são derivados (interfaces só são impostas pelos tipos integrais).
- A fábrica SymPy (`pinn_solver.factory`) é caminho legado: `NeuralNetwork`, `PINN` e `SympyBackend` são shims de compatibilidade, e `SympyTorchCompiler` usa `lambdify`, opaco ao autograd.
- Vários módulos não têm teste direto: `pinn_solver.domino`, `pinn_solver.io`, `pinn_solver.schemas`, `factory.autodiff/signals/sympy_backend`, `compiler.dataset/em_kinds`, `multiscale.consistency/residuals/scales/viz`, `non_hermitian.pt_dimer`, `weather.data/train` e os submódulos de `physics_audit` (testados só via o `__init__`).
- Dependências opcionais: `multiscale.learned`, `pinn_solver` e `weather` precisam de PyTorch; `weather` e `pinn_solver.io` usam xarray/Zarr e acesso aos buckets públicos do Google Cloud; as figuras usam matplotlib (e scipy em `weather.viz`); `non_hermitian.ep_tuning` usa sympy.
- `weather` trabalha na grade grossa 64 x 32 do WeatherBench2; as previsões publicadas para comparação existem só nas inicializações 00 e 12 UTC.
- As correlações de `closed_form` valem nas faixas citadas nos docstrings e emitem avisos fora delas (por exemplo `particle_drag`, `pipe_flow`, `plate_fin_heatsink`); o `__init__` de `closed_form` não importa todos os módulos.
- As classes de fechamento de `multiscale.closures` herdam de uma base interna cujo `nu_t` levanta `NotImplementedError`; `schema_adapter` declara premissas de MVP no próprio docstring.
- Dependências opcionais: `sgp4` (satélites e comparação de decaimento), `pymsis` (NRLMSIS), `skyfield/skyfield_data` (kernel DE421, tabela IERS e validação), `numba` (backend rápido da hidrodinâmica), `playwright` (captura), `matplotlib` (figuras), `astropy` e `astroML` (bridges).
- Várias funções de `astro` usam a rede (Open-Meteo, CelesTrak, NOAA SWPC, CDS hips2fits) e levantam erro sem conexão quando não há cache.
- Módulos sem teste direto: `astro.astrometry`, `astro.nbody`, `astro.phenomena`, `astro.satellites`, `blackhole.hydro`, `blackhole.hydro_numba`, `blackhole.capture`, `blackhole.figures` e vários presets (`cfd`, `industry`, `multidisciplinary`, `solid_mechanics`, `structural`, `electromagnetics`, `terramechanics`).
- `pde_environment/__init__.py` importa `drill_pipe_torsion_default` de `presets.structural`, mas o módulo define `rotary_coupling_torsion_default`; o `try/except ImportError` engole a falha e nenhum preset estrutural é reexportado pelo pacote.

- `symbolic_pde.compiler` levanta NotImplementedError para tipos de expressão SymPy não suportados e para coordenada solta em expressões avaliadas no modo grade.
- `tag_geometry` só fornece geometria para presets com domínio de caixa ou cilindro; os outros presets com condições por tag continuam levantando `TagConditionsUnresolved` de propósito.
- `astroml_bridge` está marcado como deprecated no inventário: os periodogramas do astroML delegam ao `astropy` e emitem avisos de depreciação, que o módulo silencia.
- Aproximações declaradas nos docstrings: magnitude de Saturno sem o termo dos anéis, nutação truncada (IAU 1980, termos acima de 5 mas), brilho do crepúsculo por ajuste linear, correção bolométrica de corpo negro; `capabilities.identify_pde` usa só contagem de palavras-chave.
- `codegen.fdm_script_generator` cobre só calor 1D, Burgers 1D e Poisson/Laplace 2D; outras formas levantam `NotImplementedError`. O gerador FEniCS escolhe por heurística e cai em difusão por padrão; os scripts FEniCS precisam do dolfinx instalado para rodar.
- `deepxde_bridge` traduz apenas `burgers` e `poisson`; outros tipos de EDP e condições não suportadas levantam `NotImplementedError`. DeepXDE é dependência opcional e o `adapter` não tem teste direto.
- `pp.compile` só reaproveita derivadas de primeira ordem; fusão de operadores, batching de kernel e planejamento de memória não são feitos (dito no docstring).
- O método `reference` de `pp.solve` levanta `MethodNotAvailable` se o solver clássico não devolve campo em grade sobre todas as coordenadas.
- `tracking` depende de MLflow ou W&B instalados e configurados; são importados só quando usados.
- O modelo de PCB assume placa em ar livre, coeficientes de convecção com faixa de ±20 % assumida e valores típicos de encapsulamento quando faltam θJB/θJC; `model_scope` lista essas simplificações.
- Sem teste direto pelo inventário: `experiment`, `online_learning`, `deepxde_bridge.adapter`, `tribology.archard`, `codegen/__init__`, `solid/__init__` (vários são testados indiretamente via os pacotes que os reexportam).

2. `pinneapple_presets`

Objetivo

`pinneapple_presets` reúne presets industriais: problemas de engenharia já montados, resolvidos e validados com o PINNeAPPle. Cada preset é uma subclasse de `Preset` com uma ficha (entradas em SI com fonte do valor padrão e domínio de validade, saídas), um solver, checagens que alimentam um trust report, uma decisão (OK / NOT OK), referências citáveis para validação e, quando cabe, otimização, inversão, variantes de projeto e cena 3D. Os casos cobrem aerodinâmica (perfil a Re 4e6), eletromagnetismo (correntes parasitas, motores IPM e SPM), térmica (dissipador, PCB, isolamento, casco-tubo), estruturas (vaso de pressão ASME, passarela de aço) e fluidos (golpe de aríete, rede de água, bomba e NPSH).

O uso é `pp.presets.load(nome).run(**params)`, a CLI `python -m pinneapple_presets` ou o nó `preset.run` do Flow Studio. Uma corrida fora do domínio de validade é sinalizada. `surrogate` constrói um processo gaussiano ARD por saída a partir de um DOE com o solver real e mede o erro em corridas retidas; `run(fidelity="surrogate")` usa esse modelo.

`report` gera a pasta de cada preset em `examples/industrial_presets/` (resultados, figuras, README, manifesto do Lab) e `registry.write_index` grava o `pinneapple_presets/index.json` que o Hub lê. Os presets usam solvers e modelos de outros pacotes (por exemplo o substituto do zoo para o perfil, `pinneapple_core.jsonable` no índice, a cena de estúdio em `viz3d`).

Números do inventário

módulos	linhas	classes públicas	funções públicas	testes em arquivos que importam o pacote	módulos sem teste direto
21	5.247	22	31	50 (3 arquivos)	16

Submódulos

submódulo	o que faz	módulos	classes + funções	testes*
`__main__`	CLI do pacote.	1	1	27

submódulo	o que faz	módulos	classes + funções	testes*
`cases`	Os presets concretos (um módulo por problema de engenharia).	14	20	4
`core`	Classes base: `Param`, `Output`, `Reference`, `Result`, `Preset`.	1	10	0
`registry`	Registro de presets e índice do Hub.	1	6	0
`report`	Geração da documentação e dos resultados de cada preset.	1	4	27
`surrogate`	Substituto por processo gaussiano ARD.	1	6	46
`viz3d`	Primitivas e imagens 3D dos resultados.	1	6	0

* maior número de testes que importam um módulo do submódulo.

Módulos e API pública

`pinneapple_presets`

Presets industriais: problemas de engenharia já montados, resolvidos e validados com o PINNeAPPle, acessíveis por `pp.presets.names()`/`load()`; reexporta o registro e as classes base.

`pinneapple_presets/__init__.py` · 19 linhas · testes: 27

`pinneapple_presets.__main__`

CLI `python -m pinneapple_presets` com os comandos list, card, run (opcionalmente `--surrogate`), validate, surrogate, report e index.

`pinneapple_presets/__main__.py` · 83 linhas · testes: 27

nome	tipo	assinatura	o que faz
`main`	função	`(argv=None) -> int`	Interpreta os argumentos da CLI e executa o comando pedido, devolvendo o código de saída.

`pinneapple_presets.cases`

Pacote vazio que agrupa os presets concretos. [sem teste direto]

`pinneapple_presets/cases/__init__.py` · 1 linhas · **sem teste direto**

`pinneapple_presets.cases.airfoil`

Perfil de asa/pá a Re 4e6: sustentação, arrasto e momento a partir do substituto verificado do zoo treinado com OpenFOAM RANS, com teoria de perfil fino como checagem.

`pinneapple_presets/cases/airfoil.py` · 290 linhas · testes: 4

nome	tipo	assinatura	o que faz
`AirfoilSection`	classe	`()`	Preset do perfil de asa: modelo do zoo, forma NACA, solução, checagens, decisão, validação, otimização, inversão, variantes, figuras e cena.
`PresetCache`	classe	`()`	Cache em atributo de classe do modelo do zoo carregado, para não recarregá-lo a cada corrida.
`thin_airfoil_zero_lift`	função	`(code) -> float`	Ângulo de sustentação nula de uma linha de arqueamento NACA 4 dígitos pela teoria de perfil fino (quadratura numérica).

`pinneapple_presets.cases.eddy_current`

Inspeção por correntes parasitas de chapa aeronáutica: impedância da sonda, profundidade de penetração e sensibilidade ao afinamento, com checagem por diferenças finitas. [sem teste direto]

`pinneapple_presets/cases/eddy_current.py` · 308 linhas · **sem teste direto**

nome	tipo	assina-tura	o que faz
`EddyCurrentInspection`	classe	`()`	Preset de inspeção por correntes parasitas, com solução analítica e checagem por diferenças finitas.

`pinneapple_presets.cases.footbridge`

Passarela de aço: resistência à multidão, flecha, flambagem lateral-torcional e vibração de pedestres (EN 1990 A2). [sem teste direto]

`pinneapple_presets/cases/footbridge.py` · 375 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`SteelFootbridge`	classe	`()`	Preset da passarela de aço com checagens de resistência, flecha, flambagem e conforto.
`section_class_ok`	função	`(h, b, tf, tw, fy) -> tuple`	Verifica os limites de classe 3 da EN 1993-1-1 Tabela 5.2 para alma e mesa da seção.

`pinneapple_presets.cases.heatsink`

Dissipador de aletas planas em ar forçado canalizado para módulo de potência: rede de resistências mais base em volumes finitos 3D. [sem teste direto]

`pinneapple_presets/cases/heatsink.py` · 361 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`PlateFinHeatsink`	classe	`()`	Preset do dissipador de aletas com rede de resistências e base em volumes finitos 3D.

`pinneapple_presets.cases.insulation`

Isolamento de linha de vapor: perda de calor, temperatura de superfície e espessura econômica (ISO 12241, VDI 2055). [sem teste direto]

`pinneapple_presets/cases/insulation.py` · 279 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`SteamLineInsulation`	classe	`()`	Preset do isolamento de linha de vapor com espessura econômica.

`pinneapple_presets.cases.ipm_traction`

Motor de tração de ímãs internos (geometria do Toyota Prius 2004): torque, ondulação, MTPA e tensão em velocidade por elementos finitos. [sem teste direto]

`pinneapple_presets/cases/ipm_traction.py` · 252 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`IPMTraction`	classe	`()`	Preset do motor de tração IPM resolvido por FEA.
`machine`	função	`(n_gap) -> IPMMachine`	Monta a máquina `IPMMachine` da geometria de referência com `n_gap` elementos no entreferro.

`pinneapple_presets.cases.pcb`

Temperaturas de junção numa placa de potência (estágio DC-DC/acionamento) por volumes finitos 3D do empilhamento. [sem teste direto]

`pinneapple_presets/cases/pcb.py` · 266 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`PowerPCB`	classe	`()`	Preset da placa de potência com temperaturas de junção por volumes finitos 3D.

pinneapple_presets.cases.pipe_network

Rede de distribuição de água: pressões, vazões e dimensionamento de menor custo pelo algoritmo do gradiente global. [sem teste direto]

pinneapple_presets/cases/pipe_network.py · 289 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`WaterNetwork`	classe	`()`	Preset da rede de água com algoritmo do gradiente global e dimensionamento de menor custo.

pinneapple_presets.cases.pressure_vessel

Casco e tampos de vaso de pressão cilíndrico pela ASME VIII Div. 1 (UG-27, UG-32), conferidos por Lamé e FEM 3D. [sem teste direto]

pinneapple_presets/cases/pressure_vessel.py · 346 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`PressureVessel`	classe	`()`	Preset do vaso de pressão pela ASME VIII com checagem por Lamé e FEM de anel.
`ring_mesh`	função	`(ri, ro, h, nr, nt, nz) -> HexMesh`	Malha hexaédrica de 8 nós de um quarto de anel.
`lame`	função	`(p, ri, ro, r, E, nu) -> dict`	Tensões de Lamé num raio e deslocamento radial em deformação plana.

pinneapple_presets.cases.pump_npsh

Bomba centrífuga aspirando de um poço: ponto de operação, NPSH disponível contra requerido e margem contra cavitação. [sem teste direto]

pinneapple_presets/cases/pump_npsh.py · 276 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`PumpNPSH`	classe	`()`	Preset da bomba centrífuga com NPSH e margem de cavitação.

pinneapple_presets.cases.shell_tube

Resfriador casco-tubo de água em circuito fechado (TEMA E, 1 passe no casco, 2 nos tubos) avaliado por ε-NTU. [sem teste direto]

pinneapple_presets/cases/shell_tube.py · 304 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ShellTubeCooler`	classe	`()`	Preset do resfriador casco-tubo avaliado por ε-NTU.

pinneapple_presets.cases.spm_motor

Servo motor de ímãs superficiais: torque, ondulação, cogging e força contra-eletromotriz por FEM magnetostático 2D não linear. [sem teste direto]

pinneapple_presets/cases/spm_motor.py · 292 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`SPMMotor`	classe	`()`	Preset do servo motor de ímãs superficiais por FEM magnetostático 2D.

pinneapple_presets.cases.water_hammer

Golpe de aríete no fechamento de válvula no fim de uma adutora de aço, pelo método das características, com teoria de coluna rígida como checagem. [sem teste direto]

pinneapple_presets/cases/water_hammer.py · 369 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`WaterHammer`	classe	`()`	Preset do golpe de aríete pelo método das características, com estudo de convergência.
`rigid_column_max_head`	função	`(*, length, diameter, friction_factor, q0, h_reservoir, h_downstream, t_close, closure_ex…)`	Carga máxima na válvula para o mesmo fechamento pela teoria de coluna rígida.

pinneapple_presets.core

Base dos presets: parâmetros com domínio de validade e fonte, saídas, referências citáveis, especificação de substituto, `Result` com trust report e decisão, e a classe `Preset` (cujo `solve` levanta NotImplementedError até ser implementado). [sem teste direto]

`pinneapple_presets/core.py` · 342 linhas · **sem teste direto** · palavras-chave no código (heurística; o texto acima diz o status real): levanta `NotImplementedError` em algum caminho

nome	tipo	assinatura	o que faz
`Param`	classe	`(name, unit, default, source, lo, hi, help, choices)`	Uma entrada do preset: unidade, padrão com fonte, domínio de validade `lo..hi`, ajuda e escolhas.
`Output`	classe	`(name, unit, help)`	Uma saída do preset com unidade e ajuda.
`Reference`	classe	`(name, params, output, value, unit, source, kind, tol)`	Valor de referência citável de uma saída (analítico, norma, benchmark, experimento ou solver independente) com tolerância.
`SurrogateSpec`	classe	`(inputs, outputs, n_train, n_test, bounds, log, log_outputs)`	Quais entradas e saídas o substituto cobre e o tamanho do DOE.
`Result`	classe	`(preset, params, outputs, trust, decision, fidelity, warnings, meta, fields)`	Saídas de uma corrida com trust report, decisão, fidelidade, avisos e proveniência.
`Preset`	classe	`(name, title, sector, problem, question, params, outputs, validity, pipeline, library, su…)`	Classe base: atributos da ficha e os métodos `solve`, `checks`, `decide`, `references` (e opcionalmente `scene`, `convergence`, `figures`) a implementar.
`check`	função	`(name, ok, value=None, limit=None, source='', reason='', critical=False) -> Check`	Cria uma checagem do trust report a partir de um passa/falha (None = não rodou).
`environment`	função	`() -> dict`	Versões e hardware da corrida, gravados com cada resultado.
`lhs`	função	`(n, bounds, seed=0, log=()) -> np.ndarray`	Amostra por hipercubo latino de `n` pontos numa caixa, com eixos opcionais em escala log.
`call_quiet`	função	`(fn, *a, **kw)`	Chama uma função suprimindo avisos.

pinneapple_presets.registry

Registro dos presets embutidos (importados sob demanda) e de usuário, e o índice `pinneapple_presets/index.json` que o Hub lê. [sem teste direto]

`pinneapple_presets/registry.py` · 97 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`register`	função	`(cls) -> type`	Decorador que adiciona um preset de usuário com nome único.
`names`	função	`() -> list[str]`	Nomes de todos os presets (embutidos e registrados), em ordem.
`load`	função	`(name) -> Preset`	Instancia o preset pelo nome, sugerindo nomes próximos se não existir.
`list_presets`	função	`(sector=None) -> list[dict]`	Nome, título, setor e capacidades de cada preset, com filtro de setor.
`index`	função	`() -> dict`	Tudo que o Hub mostra: ficha, resultados medidos e caminhos de figuras de cada preset.
`write_index`	função	`(path=INDEX_PATH) -> str`	Grava o índice JSON do Hub.

pinneapple_presets.report

Resolve, valida e documenta um preset em `examples/industrial_presets/<name>/`: resultados, figuras, imagens 3D, substituto, inversão, otimização, variantes, README e manifesto do Lab.

`pinneapple_presets/report.py` · 322 linhas · testes: 27

nome	tipo	assinatura	o que faz
`write_report`	função	`(preset, out_dir=None, *, surrogate='load', three_d=True, lab=True) -> dict`	Gera a pasta de documentação de um preset, com substituto construído, carregado ou pulado.
`compare_variants`	função	`(p, variants, res, root, three_d=True) -> dict`	Roda todas as variantes de projeto e gera tabela, gráfico de barras e galeria 3D.
`readme`	função	`(p, s) -> str`	Texto do README de um preset (ficha + resultados medidos).
`write_catalogue`	função	`(root=None) -> str`	Gera `examples/industrial_presets/README.md` com uma linha por preset a partir dos resumos gravados.

pinneapple_presets.surrogate

Substituto de um preset: DOE por hipercubo latino rodado com o solver real, um processo gaussiano ARD por saída, erro medido em corridas retidas e a caixa de treino como domínio de validade.

`pinneapple_presets/surrogate.py` · 232 linhas · testes: 46

nome	tipo	assinatura	o que faz
`ARDGP`	classe	`()`	Processo gaussiano com kernel exponencial quadrático e um comprimento de escala por entrada, ajustado por máxima verossimilhança marginal.
`PresetSurrogate`	classe	`(meta, models, test=None)`	Substituto treinado de um preset: um GP ARD por saída, com predição e detecção de fora da caixa.
`surrogate_path`	função	`(name) -> str`	Caminho onde o substituto de um preset é gravado.
`load_surrogate`	função	`(name) -> PresetSurrogate | None`	Carrega o substituto gravado de um preset, ou None.
`build_surrogate`	função	`(preset, *, seed=0, save=True, tol=0.05, n_train=None, n_test=None) -> PresetSurrogate`	Roda o DOE com o solver real, ajusta, mede o erro retido e grava o modelo.
`serve_spec`	função	`(name) -> dict | None`	O que um servidor de modelos precisa para expor o substituto: entradas com limites e unidades, saídas, erro e chamada Python.

pinneapple_presets.viz3d

Imagens 3D de resultados de presets na cena de estúdio da biblioteca: primitivas de malha (caixa, tubo, extrusão) e gravação de imagens e visualizador HTML. [sem teste direto]

`pinneapple_presets/viz3d.py` · 144 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`box`	função	`(lo, hi, n=(1, 1, 1))`	Caixa fechada alinhada aos eixos como malha (V, F).
`tube`	função	`(path, radius, n_around=16)`	Tubo aberto ao longo de uma polilinha, com o índice do ponto de cada vértice.
`extrude`	função	`(section, x)`	Prisma de um polígono 2D extrudado ao longo de estações.
`merge`	função	`(parts)`	Concatena várias malhas numa só.
`stills`	função	`(scene, out_dir, name, modes=(), size=(1100, 700), views=None, frame=None) -> dict[str, s…)`	Grava o visualizador HTML autocontido e imagens estáticas de uma cena.
`field_on`	função	`(idx, values) -> np.ndarray`	Valores por vértice a partir de valores por estação.

Exemplos de uso reais do repositório

Uso de um preset (docstring do pacote) — `pinneapple_presets/__init__.py`

```
import pinneapple as pp
pp.presets.names()                                  # catalogue
p = pp.presets.load("water_hammer_valve_closure")
p.card()                                            # data sheet: inputs (SI, defaults with sources), outputs
r = p.run(t_close=4.0)                              # solver; fidelity="surrogate" for the trained surrogate
print(r)                                            # outputs, decision, trust report
p.validate()                                        # reference cases with the measured error
```

Testes e status de validação

- Módulos com pelo menos um teste que os importa: 5 de 21.
- Funções de teste em arquivos que importam algum módulo do pacote: 50.

Limitações conhecidas

- `Preset.solve` levanta NotImplementedError na classe base; todo preset novo precisa implementar `solve`, `checks`, `decide` e `references`.
- O domínio `lo..hi` de cada parâmetro é o domínio de validade do solver e das correlações; corridas fora dele são sinalizadas, não extrapoladas com garantia.
- O substituto só vale dentro da caixa de treino e tem erro retido medido por preset; no catálogo gravado, o golpe de aríete aparece com 18 % de erro hold-out máximo, e vários presets não têm substituto porque a forma fechada já é instantânea.
- O perfil de asa usa um substituto do zoo de OpenFOAM RANS como solver; o catálogo registra que o OpenFOAM não estava instalado para conferir os ótimos.
- Os módulos de casos, exceto `airfoil`, e `core`, `registry` e `viz3d` não têm teste que os importe diretamente; são exercitados pelos testes da CLI e de `report`/`surrogate`.

3. `pinneapple_problemdesign`

Objetivo

`pinneapple_problemdesign` ajuda a transformar uma descrição de problema em linguagem natural numa especificação estruturada e num plano de solução com o PINNeAPPle. O núcleo é o `DesignAgent`, que conversa com um LLM por estágios, preenche um `ProblemSpec` (dados, física, geometria, validação, implantação, restrições) e aplica uma política de não invenção: o que o usuário não informou vira `Gap` (pergunta) ou suposição marcada, nunca um valor inventado. O resultado é um `DesignReport` (Markdown ou JSON) com um plano FNO-first, PINN-first ou CFD-first.

A partir do spec, `codegen.build_pinneapple_spec` gera um `PinneappleSpec` com tipo de EDP, campos, modelo e código de pipeline; `method_selection` sugere método numérico e fechamento de turbulência com base no regime de escoamento (usando `pinneapple_analysis` e o registro de solvers de `pinneapple_simulation`); `knowledge.physics_knowledge` liga fenômenos a equações e presets de `pinneapple_physics`; e `preset_authoring` deixa um LLM propor uma EDP nova, aceitando-a só se passar numa checagem por solução manufaturada.

A parte agêntica compõe peças de outros pacotes sem alterá-las: `UnifiedPhysicsAgent` liga a elicitação ao `PhysicsOrchestrator` de `pinneapple_worldmodel`, e `AutonomousResearchAgent` roda iterações com memória de experimentos (`pinneapple_registry`) e validação por guardrail (`pinneapple_llm`). Tudo isso passa pela `ApprovalGate` de `autonomy`, que exige aprovação humana para ações consequentes (custo, ferramenta licenciada, efeitos fora do sandbox) em qualquer nível de autonomia. Os provedores de LLM incluídos são Gemini e um gateway HTTP (Cosmos); qualquer objeto com `generate` serve.

Números do inventário

módulos	linhas	classes públicas	funções públicas	testes em arquivos que importam o pacote	módulos sem teste direto
28	5.642	40	21	123 (9 arquivos)	19

Submódulos

submódulo	o que faz	módulos	classes + funções	testes*
`agent`	Agente de elicitação `DesignAgent`.	1	1	0
`autonomy`	Trava de aprovação humana para ações consequentes.	1	5	41
`cli`	CLI interativa do agente com Gemini.	1	1	0
`codegen`	Gera `PinneappleSpec` e código a partir do `ProblemSpec`.	1	1	0
`elicitation`	Estágios, perguntas de lacuna e validadores da elicitação.	4	3	0
`extractor`	Extração JSON de requisitos via LLM.	1	1	0
`knowledge`	Construtores de plano, base de fenômenos físicos e mapa de capacidades.	4	7	22
`merge`	Mescla de dados extraídos no spec.	1	1	0
`method_selection`	Recomendação de método numérico e turbulência pelo regime.	1	3	11
`policy`	Política de não invenção.	1	1	0
`preset_authoring`	Rascunho de preset por LLM verificado por solução manufaturada.	1	5	15
`protocol`	Protocolo de LLM e provedor Gemini.	1	4	20
`providers`	Provedores de LLM adicionais (Cosmos).	2	3	0
`renderers`	Relatórios em Markdown e JSON.	3	2	0
`research_loop`	Agente de pesquisa autônomo sob a trava de aprovação.	1	3	9
`schema`	Dataclasses do problema, plano e relatório.	1	16	70
`state`	Estado da conversa de design.	1	2	0
`unified_agent`	Elicitação + `PhysicsOrchestrator` num laço único.	1	2	0

* maior número de testes que importam um módulo do submódulo.

Módulos e API pública

`pinneapple_problemdesign`

Reexporta o agente de design, o esquema do problema, o estado, o protocolo de LLM, o gerador de código e a trava de autonomia; `UnifiedPhysicsAgent` e `AutonomousResearchAgent` só são exportados se as dependências opcionais (`pinneapple_worldmodel`, `pinneapple_registry`) importarem.

`pinneapple_problemdesign/__init__.py` · 62 linhas · testes: 58

`pinneapple_problemdesign.agent`

`DesignAgent`: elicitação interativa do problema via LLM, com extração estruturada em JSON, mescla segura no `ProblemSpec` e política de não invenção (o que é desconhecido vira `Gap`). [sem teste direto]

`pinneapple_problemdesign/agent.py` · 194 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`DesignAgent`	classe	`(llm, use_orchestrator_bridge=True)`	Agente que conduz a elicitação por estágios: ingere mensagens, extrai campos, gera perguntas de lacuna e, ao fim, o relatório.

pinneapple_problemdesign.autonomy

Trava de segurança do laço de pesquisa autônomo: classifica ações consequentes (custo, ferramenta licenciada, efeito fora do sandbox) e exige aprovação humana nelas em qualquer nível de autonomia, falhando fechado.

pinneapple_problemdesign/autonomy.py · 481 linhas · testes: 41

nome	tipo	assinatura	o que faz
AutonomyLevel	classe	()	Nível de autonomia do laço (SUPERVISED, SEMI_AUTONOMOUS, FULL_AUTONOMOUS).
ConsequentialActionClassifier	classe	(*, name_patterns=None, category_patterns=None, tag_patterns=None, exceptions=None) -> No…)	Classificador explícito (por padrões de nome, categoria e tag, sem ML) que decide se uma chamada de ferramenta é consequente.
ApprovalRequest	classe	(tool_name, tool_args, reason, autonomy_level, is_consequential)	Tudo que o aprovador humano precisa para decidir uma ação travada.
ApprovalDecision	classe	(approved, reason, requires_human)	Resultado de ApprovalGate.check: aprovado, motivo e se exige humano.
ApprovalGate	classe	(autonomy_level, approver_fn=None, iteration_report_every=5, confidence_pause_threshold=N…)	A trava: decide por chamada de ferramenta se pode prosseguir, pede aprovação quando necessário e falha fechado sem aprovador.

pinneapple_problemdesign.cli

CLI interativa mínima do agente de design usando GeminiProvider. [sem teste direto]

pinneapple_problemdesign/cli.py · 39 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
main	função	()	Laço interativo no terminal com o DesignAgent.

pinneapple_problemdesign.codegen

Traduz um ProblemSpec elicitado num PinneappleSpec concreto (tipo de EDP, coordenadas, campos, modelo, treino) com código de pipeline gerado; o código gerado contém TODOs para valores e DataLoader não resolvidos. [sem teste direto]

pinneapple_problemdesign/codegen.py · 374 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
build_pinneapple_spec	função	(design_spec) -> PinneappleSpec	Traduz um ProblemSpec num PinneappleSpec com objetos e código do PINNeAPPle.

pinneapple_problemdesign.elicitation

Subpacote de elicitação: reexporta STAGES_ORDER, Stage, build_stage_gaps e validate_and_suggest. [sem teste direto]

pinneapple_problemdesign/elicitation/__init__.py · 6 linhas · **sem teste direto**

pinneapple_problemdesign.elicitation.questions

Perguntas de lacuna (Gap) específicas de cada estágio da elicitação. [sem teste direto]

pinneapple_problemdesign/elicitation/questions.py · 101 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
build_stage_gaps	função	(stage, spec_dict) -> List[Gap]	Lista as lacunas abertas de um estágio dado o spec atual.

pinneapple_problemdesign.elicitation.stages

Definição dos estágios da elicitação (intake, entradas/saídas e tempo, dados, ...) e da ordem `STAGES_ORDER`. [sem teste direto]

`pinneapple_problemdesign/elicitation/stages.py` · 76 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Stage`	classe	`(name, description, required_fields)`	Um estágio de elicitação: nome, descrição e campos obrigatórios.

pinneapple_problemdesign.elicitation.validators

Validação de um `ProblemSpec`: avisos e suposições sugeridas. [sem teste direto]

`pinneapple_problemdesign/elicitation/validators.py` · 42 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`validate_and_suggest`	função	`(spec) -> Tuple[List[str], List[Assumption]]`	Devolve avisos e suposições sugeridas para um `ProblemSpec`.

pinneapple_problemdesign.extractor

Extração em JSON de requisitos a partir do texto do usuário via LLM. [sem teste direto]

`pinneapple_problemdesign/extractor.py` · 140 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`extract_from_text`	função	`(llm, user_text) -> Tuple[Dict[str, Any], Dict[str, Any]]`	Pede ao LLM um JSON estruturado a partir do texto do usuário e devolve o dicionário extraído e metadados.

pinneapple_problemdesign.knowledge

Subpacote de conhecimento: reexporta `PINNEAPPLE_CAPABILITIES` e os construtores de plano. [sem teste direto]

`pinneapple_problemdesign/knowledge/__init__.py` · 10 linhas · **sem teste direto**

pinneapple_problemdesign.knowledge.mapping

Construtor de plano: escolhe entre plano FNO-first, PINN-first ou CFD-first pelo `task_type`, e lista (só leitura) ferramentas registradas no `PhysicsOrchestrator` do `pinneapple_worldmodel` quando disponível.

`pinneapple_problemdesign/knowledge/mapping.py` · 483 linhas · testes: 22

nome	tipo	assinatura	o que faz
`available_orchestrator_tools`	função	`(spec) -> List[Dict[str, object]]`	Consulta só de leitura das ferramentas registradas no `PhysicsOrchestrator`.
`build_plan`	função	`(spec, gaps, use_orchestrator_bridge=True) -> Plan`	Escolhe o construtor de plano pelo `task_type` do spec.
`build_plan_pinn_first`	função	`(spec, gaps) -> Plan`	Plano para solução de EDP e problema inverso via `PINNFactory`.
`build_plan_fno_first`	função	`(spec, gaps) -> Plan`	Plano FNO-first para previsão e aprendizado de operadores.
`build_plan_cfd_first`	função	`(spec, gaps) -> Plan`	Plano para aerodinâmica externa/CFD com o solver Lattice-Boltzmann, presets RANS/LES e pós-processamento de vórtices.

pinneapple_problemdesign.knowledge.physics_knowledge

Base de conhecimento fenômeno -> equação governante -> parâmetros típicos -> suposições, cada entrada ligada a um preset real.

pinneapple_problemdesign/knowledge/physics_knowledge.py · 615 linhas · testes: 6

nome	tipo	assinatura	o que faz
PhenomenonEntry	classe	(phenomenon, governing_equation, typical_parameters, assumptions, preset_module, preset_f…)	Um registro de conhecimento físico rastreável até um preset real.
lookup_phenomenon	função	(name_or_keyword) -> List[PhenomenonEntry]	Busca entradas da base por palavra-chave, sem diferenciar maiúsculas.

pinneapple_problemdesign.knowledge.pinneapple_capabilities

Dicionário PINNEAPPLE_CAPABILITIES com os módulos do PINNeAPPle por área (treino, modelos, séries temporais, CFD...) usado no texto dos planos. [sem teste direto]

pinneapple_problemdesign/knowledge/pinneapple_capabilities.py · 41 linhas · **sem teste direto**

pinneapple_problemdesign.merge

Mescla profunda de um dicionário extraído dentro de um ProblemSpec existente. [sem teste direto]

pinneapple_problemdesign/merge.py · 33 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
merge_into_spec	função	(spec, partial) -> None	Mescla um dicionário parcial no ProblemSpec, no lugar.

pinneapple_problemdesign.method_selection

Recomenda método numérico e fechamento de turbulência a partir do regime de escoamento calculado, com uma tabela de decisão explícita e heurística (o docstring avisa que não é sistema especialista validado); recommend_method_from_spec hoje sempre devolve None porque o ProblemSpec não guarda os números necessários.

pinneapple_problemdesign/method_selection.py · 346 linhas · testes: 11 · palavras-chave no código (heurística; o texto acima diz o status real): placeholder/stub citado no docstring

nome	tipo	assinatura	o que faz
MethodRecommendation	classe	(numerical_method, turbulence_model, flow_regime, rationale, confidence)	Par recomendado (método numérico, modelo de turbulência) com regime, justificativa e confiança.
recommend_method	função	(velocity, length_scale, kinematic_viscosity, *, geometry='internal_pipe', compressible=F…)	Recomenda solver e fechamento de turbulência a partir de velocidade, escala e viscosidade.
recommend_method_from_spec	função	(spec) -> Optional[MethodRecommendation]	Extrai os parâmetros do ProblemSpec e chama recommend_method, ou devolve None se faltarem.

pinneapple_problemdesign.policy

Política de não invenção: aplica suposições e lacunas sugeridas ao estado do design. [sem teste direto]

pinneapple_problemdesign/policy.py · 67 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
apply_non_invention_policy	função	(state, *, unknown_fields, assumptions_suggested, gaps_suggested) -> None	Registra campos desconhecidos como lacunas e suposições no estado, sem inventar valores.

pinneapple_problemdesign.preset_authoring

Rascunho de preset assistido por LLM: o LLM propõe uma EDP, ela é convertida em sympy e só é marcada verificada se passar num teste de solução manufaturada automático; não produz presets confiáveis para produção.

pinneapple_problemdesign/preset_authoring.py · 707 linhas · testes: 15

nome	tipo	assinatura	o que faz
`PresetDraftError`	classe	`(stage, message, *, raw='') -> None`	Erro quando a proposta do LLM não vira uma EDP real e checável.
`ManufacturedSolutionCheck`	classe	`(exact_solution_used, residual_at_exact_solution, tolerance, passed, n_points, notes)`	Resultado de uma checagem por solução manufaturada: solução usada, resíduo, tolerância e se passou.
`DraftedPreset`	classe	`(phenomenon_description, proposed_equation_latex_or_sympy, sympy_pde, proposed_bcs, propo…)`	Preset proposto pelo LLM com a EDP em sympy, condições, resultado da verificação e citações.
`run_manufactured_solution_check`	função	`(sympy_pde, coord_syms, field_syms, *, param_syms=None, params=None, tolerance=0.0001, n_…)`	Roda uma checagem de autoconsistência por solução manufaturada sobre uma EDP sympy.
`draft_preset`	função	`(phenomenon_description, *, llm_provider='anthropic', model=None, use_knowledge_base=True…)`	Pede a um LLM (padrão `anthropic`) uma EDP para um fenômeno e a verifica por solução manufaturada.

pinneapple_problemdesign.protocol

Protocolo `LLMProvider`, mensagens/respostas e o `GeminiProvider` (SDK Google Generative AI).

`pinneapple_problemdesign/protocol.py` · 100 linhas · testes: 20

nome	tipo	assinatura	o que faz
`LLMMessage`	classe	`(role, content)`	Mensagem de LLM com papel e conteúdo.
`LLMResponse`	classe	`(text, raw)`	Resposta de LLM com texto e objeto bruto.
`LLMProvider`	classe	`()`	Interface única de provedor de LLM (`generate`).
`GeminiProvider`	classe	`(model='gemini-1.5-pro')`	Provedor Gemini via SDK Google Generative AI.

pinneapple_problemdesign.providers

Define outra interface `LLMProvider` (Protocol) para os provedores do pacote. [sem teste direto]

`pinneapple_problemdesign/providers/__init__.py` · 47 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`LLMProvider`	classe	`()`	Interface de provedor (Protocol) usada pelo pacote, duplicada em relação a `protocol.LLMProvider`.

pinneapple_problemdesign.providers.cosmos_provider

Provedor LLM que chama um gateway HTTP (Cosmos) via `requests`. [sem teste direto]

`pinneapple_problemdesign/providers/cosmos_provider.py` · 187 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`CosmosProviderConfig`	classe	`(base_url, api_key, timeout_s, route)`	Configuração do provedor Cosmos: URL, chave, timeout e rota.
`CosmosProvider`	classe	`(base_url=None, *, api_key=None, timeout_s=None, route=None, session=None)`	Provedor LLM que chama um gateway HTTP Cosmos.

pinneapple_problemdesign.renderers

Subpacote de renderizadores do `DesignReport` (Markdown e JSON). [sem teste direto]

`pinneapple_problemdesign/renderers/__init__.py` · 5 linhas · **sem teste direto**

pinneapple_problemdesign.renderers.report_json

Renderiza um `DesignReport` em JSON. [sem teste direto]

`pinneapple_problemdesign/renderers/report_json.py` · 35 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
render_json_report	função	(report, *, indent=2) -> str	Serializa o DesignReport em JSON.

pinneapple_problemdesign.renderers.report_md

Renderiza um DesignReport em Markdown com avisos. [sem teste direto]

pinneapple_problemdesign/renderers/report_md.py · 133 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
render_markdown_report	função	(report, warnings) -> str	Gera o relatório Markdown do design com os avisos.

pinneapple_problemdesign.research_loop

AutonomousResearchAgent: ciclo de pesquisa autônomo sobre o UnifiedPhysicsAgent, com ApprovalGate em cada iteração e ferramenta, memória de experimentos (pinneapple_registry) e validação por confiança física e guardrail.

pinneapple_problemdesign/research_loop.py · 423 linhas · testes: 9

nome	tipo	assinatura	o que faz
IterationRecord	classe	(iteration, problem_description, approval, agent_result, guardrail_report, confidence, no…)	O que aconteceu numa iteração do laço (aprovação, resultado, guardrail, confiança).
ResearchLoopResult	classe	(status, iterations, final_confidence, final_result, memory_record_ids, prior_lessons)	Resultado completo de AutonomousResearchAgent.run: status, iterações, confiança final e memória.
AutonomousResearchAgent	classe	(llm, *, autonomy_level=AutonomyLevel.SUPERVISED, approver_fn=None, memory=None, physics_…)	Agente de pesquisa autônomo que itera até a confiança alvo ou o limite de iterações, sob a trava de aprovação.

pinneapple_problemdesign.schema

Esquema do design de problema: ProblemSpec e suas partes (dados, física, geometria, validação, implantação, restrições), Gap, Plan, PinneappleSpec e DesignReport.

pinneapple_problemdesign/schema.py · 290 linhas · testes: 70

nome	tipo	assinatura	o que faz
Assumption	classe	(text, confidence, needs_confirmation)	Suposição com confiança e se precisa de confirmação.
Risk	classe	(text, level, mitigation)	Risco com nível e mitigação.
DataSpec	classe	(sources, format, sampling, variables_observed, target_variables, known_quality_issues, m…)	Especificação dos dados: fontes, formato, amostragem, variáveis, qualidade e divisão.
PhysicsSpec	classe	(governing_equations, boundary_conditions, initial_conditions, constraints, parameters_kn…)	Especificação física: equações, condições, restrições, parâmetros, unidades, turbulência e método numérico.
GeometrySpec	classe	(domain, representation, sensors, coordinate_system, cad_format, voxel_resolution, aoa_sw…)	Especificação da geometria: domínio, representação, sensores, CAD e varredura de ângulo de ataque.
ValidationSpec	classe	(primary_metrics, acceptance_criteria, robustness_tests, ood_scenarios)	Especificação de validação: métricas, critérios de aceitação, robustez e cenários fora da distribuição.
DeploymentSpec	classe	(environment, latency_budget_ms, update_policy, monitoring)	Especificação de implantação: ambiente, latência, atualização e monitoramento.
ConstraintsSpec	classe	(hardware, max_training_time, interpretability, compliance)	Restrições: hardware, tempo de treino, interpretabilidade e conformidade.
ProblemSpec	classe	(title, goal, task_type, inputs, outputs, horizon, input_window, frequency, domain_contex…)	Especificação completa do problema (objetivo, tipo de tarefa, entradas/saídas e todas as partes acima).
Gap	classe	(id, question, severity, rationale, how_to_obtain, resolved)	Lacuna de informação com pergunta, severidade, justificativa e como obter.
PlanStep	classe	(title, why, actions, pinneapple_modules, exit_criteria)	Passo de plano: título, motivo, ações, módulos PINNeAPPle e critério de saída.

nome	tipo	assinatura	o que faz
`Plan`	classe	`(recommended_approach, alternatives, steps, go_no_go)`	Plano: abordagem recomendada, alternativas, passos e critérios go/no-go.
`PinneappleSpec`	classe	`(pde_kind, pde_confidence, coords, fields, pde_params, environment_kwargs, model_name, mo…)`	Objetos concretos da API do PINNeAPPle derivados do spec (EDP, campos, modelo, treino, código).
`DesignReport`	classe	`(spec, gaps, plan, pinneapple_spec, created_at)`	Relatório final do design: spec, lacunas, plano e `PinneappleSpec`.
`uses_pinn_approach`	função	`(spec) -> bool`	Diz se o spec deve ser resolvido com um pipeline PINN de resíduo de EDP.
`uses_cfd_approach`	função	`(spec) -> bool`	Diz se o spec é um problema de aerodinâmica externa/CFD que recebe o plano CFD-first.

`pinneapple_problemdesign.state`

Estado da conversa de design (`DesignState`, `Turn`). [sem teste direto]

`pinneapple_problemdesign/state.py` · 33 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Turn`	classe	`(role, content)`	Um turno da conversa (papel, conteúdo).
`DesignState`	classe	`(spec, gaps, plan, history, stage, done)`	Estado do design: spec, lacunas, plano, histórico, estágio e se terminou.

`pinneapple_problemdesign.unified_agent`

Compõe `DesignAgent` (elicitação) e `PhysicsOrchestrator` do `pinneapple_worldmodel` (execução de ferramentas) num laço único, sem mexer nos internos de nenhum dos dois. [sem teste direto]

`pinneapple_problemdesign/unified_agent.py` · 572 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`UnifiedAgentResult`	classe	`(status, design_report, problem_statement, execution_result, design_state, gaps, question…)`	Resultado agrupado de `UnifiedPhysicsAgent.run`.
`UnifiedPhysicsAgent`	classe	`(llm, *, use_orchestrator_bridge=True, orchestrator=None, orchestrator_verbose=True) -> N…)`	Compõe um `DesignAgent` com um `PhysicsOrchestrator`: elicita o problema, monta o `ProblemStatement` e executa as ferramentas.

Exemplos de uso reais do repositório

Relatório de design sem LLM — `examples/problem_designer/03_offline_spec_to_report.py`

```
# 2) Run validators (warnings + safe assumptions)
warnings, assumptions = validate_and_suggest(spec)
spec.assumptions.extend(assumptions)

# 3) Build plan and report
gaps = []  # if you already know gaps, add them here; otherwise keep empty.
plan = build_plan_fno_first(spec, gaps)
report = DesignReport(spec=spec, gaps=gaps, plan=plan)

md = render_markdown_report(report, warnings)
js = render_json_report(report)
```

CLI interativa do agente — `pinneapple_problemdesign/cli.py`

```
def main():
    llm = GeminiProvider()
    agent = DesignAgent(llm=llm)
    state = agent.start()
```

```
print("Pinneapple Design CLI (type 'exit' to quit)\n")

while True:
    user = input("You: ").strip()
    if user.lower() in ("exit", "quit"):
        break

    agent.ingest_user_message(state, user)
    out = agent.step(state)
```

Testes e status de validação

- Módulos com pelo menos um teste que os importa: 9 de 28.
- Funções de teste em arquivos que importam algum módulo do pacote: 123.

Limitações conhecidas

- O agente depende de um LLM: `GeminiProvider` precisa do SDK Google Generative AI, `CosmosProvider` de um gateway HTTP com `requests`, `draft_preset` usa `anthropic` por padrão.
- `UnifiedPhysicsAgent` e `AutonomousResearchAgent` dependem de `pinneapple_worldmodel` (e `pinneapple_registry`); se não importarem, o `__init__` simplesmente não os exporta.
- `method_selection` é uma heurística explícita, não um sistema especialista validado (docstring); `recommend_method_from_spec` devolve None para todo spec atual porque o `ProblemSpec` não tem campos numéricos de velocidade, escala e viscosidade.
- `preset_authoring` só rascunha presets: o docstring diz que não produz nada confiável para produção, e descoberta automática de física está fora de escopo.
- O código gerado por `codegen` contém TODOs (valores físicos não resolvidos, DataLoader, tensor de teste) que o usuário precisa preencher.
- Há duas interfaces `LLMProvider` (em `protocol` e em `providers`).
- Muitos módulos não têm teste direto: `agent`, `codegen`, `unified_agent`, `elicitation.*`, `extractor`, `policy`, `merge`, `providers.*`, `renderers.*`, `state`, `cli`.

4. `pinneapple_science`

Objetivo

O `pinneapple_science` (exposto como `pp.science`) organiza experimentos por área da ciência, em nível acadêmico (didático, conferido contra valor de livro-texto) e de pesquisa (configurável, com estudo de convergência e incerteza). Cada experimento declara área, nível, tipo (`theory`, uma derivação numérica conferida contra forma fechada ou valor publicado, ou `practice`, uma bancada virtual com instrumentos ruidosos e ajuste com incerteza) e as dimensões em que roda: 0-D (concentrado), 1-D, 2-D, 3-D e 3-D + tempo.

Todo resultado (`core.Result`) carrega comparações com referências citadas, checagens físicas e um relatório de confiança (`pinneapple_analysis.trust.TrustReport`) com decisão final. O pacote também traz o atlas das áreas (`atlas`), um arquivo de dados versionado (`data/atlas.json`) que marca cada subárea como disponível, parcial ou planejada e aponta para o código que já a cobre em outros pacotes: um experimento de `pp.science`, um módulo da biblioteca, um preset de problema ou um experimento do Lab. O atlas aponta para o código existente e não o copia.

Hoje os experimentos implementados estão no subpacote `em` (eletromagnetismo), construído sobre `pp.em` (`pinneapple_simulation.electromagnetics`) e `pp.maxwell` (`pinneapple_physics.maxwell_checks`): eletrostática e magnetostática, indução e circuitos, ondas e óptica, bancadas virtuais e estudos de pesquisa (FDTD 3-D configurável, incerteza por Monte Carlo, problema inverso e surrogate). Blocos numéricos comuns ficam em `numerics`, instrumentos virtuais e ajustes em `instruments`, e convenções de figura em `plots`.

O módulo `report` liga o pacote ao `pinneapple_lab`: roda cada experimento em todas as dimensões, grava os resultados em `examples/science/`, escreve o manifesto `experiment.json`, registra o item no Lab e carimba as figuras com o selo da curadoria. A linha de comando é `python -m pinneapple_science atlas | list | run | report | table`.

Números do inventário

módulos	linhas	classes públicas	funções públicas	testes em arquivos que importam o pacote	módulos sem teste direto
16	4.253	33	49	19 (1 arquivos)	10

Submódulos

submódulo	o que faz	módulos	classes + funções	testes*
`__main__`	Linha de comando `python -m pinneapple_science`: atlas, lista, execução, relatório e tabela.	1	1	0
`atlas`	Atlas das áreas da ciência: status por subárea e ponteiros para o código que as cobre, com validação.	1	4	0
`core`	Forma comum de um experimento científico: `Experiment`, `Result`, `Reference` e `Comparison`.	1	7	19
`em`	Experimentos de eletromagnetismo: estática, indução, ondas, bancadas e estudos de pesquisa.	7	34	19
`instruments`	Instrumentos virtuais de laboratório com ruído, erro de calibração e resolução, e os ajustes com incerteza GUM.	1	10	19
`numerics`	Blocos numéricos: malhas tensoriais graduadas, solver de volumes finitos para potencial, energia de campo e autovalores de Helmholtz.	1	8	19
`plots`	Convenções de figura do pacote (paleta, mapas de cor, eixos discretos).	1	7	0
`registry`	Registro dos experimentos por nome, com filtros e execução.	1	5	0
`report`	Resultados versionados em `examples/science/`, manifestos e selos do Lab, e o catálogo com o atlas.	1	6	0

* maior número de testes que importam um módulo do submódulo.

Módulos e API pública

`pinneapple_science`

Ponto de entrada (`pp.science`): reexporta `atlas`, `experiments`, `run` e os tipos centrais.

`pinneapple_science/__init__.py` · 25 linhas · testes: 19

`pinneapple_science.__main__`

Linha de comando `python -m pinneapple_science` com os subcomandos `atlas`, `list`, `run`, `report` e `table`. [sem teste direto]

`pinneapple_science/__main__.py` · 73 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`main`	função	`(argv=None) -> int`	Monta o `argparse` dos subcomandos e despacha para atlas, registro e relatório.

`pinneapple_science.atlas`

Atlas das áreas: carrega `data/atlas.json` (esquema `pinneapple.science.atlas/1`) e valida que cada ponteiro aponta para código, preset ou experimento existente. [sem teste direto; coberto por `tests/test_science.py` via o pacote]

`pinneapple_science/atlas.py` · 273 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Area`	classe	`(id, title, branch, status, description, subareas)`	Uma área do atlas: id, título, ramo, status, descrição e subáreas com seus ponteiros.
`Atlas`	classe	`(schema, version, statuses, dims, areas)`	O atlas inteiro: busca de área, linhas de tabela, contagens por status e saída em Markdown ou HTML.
`load`	função	`(path=ATLAS_PATH) -> Atlas`	Carrega o atlas a partir do arquivo de dados versionado.
`validate`	função	`(atlas=None, *, check_presets=True, check_lab=True) -> list[str]`	Lista inconsistências do atlas: status desconhecido, módulos inexistentes, presets ou experimentos não registrados.

`pinneapple_science.core`

Forma comum de um experimento: o que ele declara (área, nível, tipo, dimensões) e o que toda execução devolve (comparações, checagens, figuras e decisão de confiança). `Experiment._run` da base levanta `NotImplementedError`.

`pinneapple_science/core.py` · 348 linhas · testes: 19 · palavras-chave no código (heurística; o texto acima diz o status real): levanta `NotImplementedError` em algum caminho

nome	tipo	assinatura	o que faz
`Reference`	classe	`(value, source, kind, unit)`	Valor de referência e sua origem (forma fechada, valor publicado, CODATA, limite assintótico, conservação...).
`Comparison`	classe	`(quantity, computed, reference, rel_error, tol, passed, kind, source, unit, how, abs_erro…)`	Comparação de uma grandeza calculada com a referência: erros, tolerância, se passou e fonte.
`Result`	classe	`(experiment, title, area, topic, level, kind, dim, params, values, comparisons, checks, f…)`	O que uma execução devolve: valores, comparações, checagens, figuras, filmes, visualizador, dados e relatório de confiança; salva em pasta.
`Experiment`	classe	`(name, title, area, topic, level, kind, dims, description, references, params, quick)`	Classe base: subclasses declaram os atributos e implementam `_run(res, dim, p)`; `run` e `run_all` executam e montam a confiança.
`jsonable`	função	`(v) -> Any`	Converte para tipos JSON simples (numpy para números e listas; NaN e inf para None).
`observed_order`	função	`(errors, ratio=2.0) -> list[float]`	Ordens de convergência observadas de uma escada de refinamento.
`central_derivative`	função	`(f, x, h) -> float`	Derivada por diferença central de uma função escalar.

`pinneapple_science.em`

Subpacote de experimentos de eletromagnetismo, construído sobre `pp.em` e `pp.maxwell`. [sem teste direto]

`pinneapple_science/em/__init__.py` · 3 linhas · **sem teste direto**

`pinneapple_science.em.bench`

Bancadas virtuais (prática): medir μ0 com um solenoide e uma sonda Hall, e uma capacitância pela descarga RC no osciloscópio, cada uma com orçamento de incerteza GUM. [sem teste direto]

`pinneapple_science/em/bench.py` · 220 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Mu0Solenoid`	classe	`()`	Experimento `em.bench_mu0_solenoid`: mede μ0 a partir de B(I) no centro de um solenoide com instrumentos virtuais.
`RCDischarge`	classe	`()`	Experimento `em.bench_rc_discharge`: mede uma capacitância pela descarga de um circuito RC num osciloscópio virtual.

pinneapple_science.em.fdtd_study

Estudo FDTD 3-D + tempo configurável por dicionário (fontes, materiais, escada de refinamento), com potência irradiada por transformada campo próximo-campo distante, snapshots e checagens de Maxwell.

pinneapple_science/em/fdtd_study.py · 199 linhas · testes: 19

nome	tipo	assinatura	o que faz
FDTDConfig	classe	(f0, size, cells_per_wavelength, npml, bandwidth, periods, sources, materials)	Configuração do estudo FDTD: frequência, tamanho em comprimentos de onda, resolução, CPML, fontes e materiais.
FDTDRun	classe	(config, dx, n, power, current_moment, ntff, frames, frame_times, volume, volume_grid, sl…)	Resultado do estudo: potência em f0, momento de corrente, quadros de E_z, campos de volume e diagrama de radiação (pattern).
run_fdtd	função	(cfg, *, snapshots=24, volume_half=3, volume_steps=24, maxwell_block=True, snap_window=(2…)	Roda o estudo configurado e devolve potência, snapshots dos planos centrais e campos em volume.
ladder_values	função	(values) -> list[float]	Converte uma sequência de valores da escada de refinamento em floats.

pinneapple_science.em.induction

Indução e circuitos (acadêmico): lei de Faraday com um ímã caindo por uma espira (3-D + tempo) e circuitos RC, RL e RLC (0-D). [sem teste direto]

pinneapple_science/em/induction.py · 271 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
FaradayInduction	classe	()	Experimento em.faraday_induction: calcula fluxo e FEM de um ímã cilíndrico caindo pela espira e compara com a referência.
Circuits	classe	()	Experimento em.circuits_rc_rl_rlc: integra as respostas ao degrau de RC, RL e RLC e confere constantes de tempo e frequências.
magnet_rings	função	(a, length, Br, n_rings, n_seg, z_center=0.0)	Discretiza um ímã cilíndrico em anéis poligonais da corrente superficial ligada K = Br/μ0.
loop_flux	função	(segs, I_ring, R, n_path=64) -> float	Fluxo através de uma espira circular pela circulação do potencial vetor (teorema de Stokes).

pinneapple_science.em.research

Eletromagnetismo em nível de pesquisa: FDTD 3-D com convergência e ordem observada, incerteza por Monte Carlo, permissividade por problema inverso e surrogate com erro de validação. [sem teste direto]

pinneapple_science/em/research.py · 548 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
FDTDStudy3D	classe	()	Experimento em.research_fdtd_3d: estudo FDTD configurável com convergência de malha (Richardson e GCI).
WaveguideUQ	classe	()	Experimento em.research_uq_waveguide: propaga por Monte Carlo as tolerâncias de fabricação até as frequências de corte de um guia.
InversePermittivity	classe	()	Experimento em.research_inverse_permittivity: recupera permissividade e perda de uma placa a partir do espectro de reflexão.
HallSurrogate	classe	()	Experimento em.research_surrogate_hall: processo gaussiano treinado em soluções de Hall, com erro em dados retidos.
slice_viewer	função	(run, path, title='FDTD: E_z in 3-D + time') -> str	Gera uma página three.js de arquivo único com os três planos centrais e E_z animado no tempo.

pinneapple_science.em.statics

Eletro e magnetostática (acadêmico): Coulomb e superposição, lei de Gauss, capacitor de placas com efeito de borda, Biot-Savart, lei de Ampère e efeito Hall, cada um com escada de refinamento contra a fórmula de livro.

pinneapple_science/em/statics.py · 901 linhas · testes: 19

nome	tipo	assinatura	o que faz
`CoulombSuperposition`	classe	`()`	Experimento `em.coulomb_superposition`: anel e linhas de carga por superposição de cargas pontuais contra a fórmula exata.
`GaussLaw`	classe	`()`	Experimento `em.gauss_law`: fluxo de E por quadratura numa superfície fechada contra a carga envolvida sobre ε0.
`ParallelPlateCapacitor`	classe	`()`	Experimento `em.parallel_plate_capacitor`: capacitância pela energia de uma solução de volumes finitos, contra Palmer e Kirchhoff.
`BiotSavart`	classe	`()`	Experimento `em.biot_savart`: integral de Biot-Savart numérica para fio, espira e solenoide.
`AmpereLaw`	classe	`()`	Experimento `em.ampere_law`: circulação de B em caminhos fechados contra μ0 vezes a corrente envolvida.
`HallEffect`	classe	`()`	Experimento `em.hall_effect`: potencial 2-D numa barra Hall com tensor de condutividade de Drude.
`ring_potential`	função	`(points, Q, R) -> np.ndarray`	Potencial exato de um anel uniformemente carregado (integral elíptica, Jackson).
`palmer_strip_capacitance`	função	`(w, d, eps_r=1.0) -> float`	Capacitância por unidade de comprimento de duas fitas finas com correção de borda de Palmer (1937), assintótica para w >> d.
`kirchhoff_disc_capacitance`	função	`(a, d, eps_r=1.0) -> float`	Capacitância de dois discos coaxiais finos pela fórmula de Kirchhoff (1877), assintótica para d << a.
`strip_capacitor_2d`	função	`(w, d, h, L=40.0, growth=1.08)`	Capacitância de duas fitas finas por volumes finitos num quarto de domínio graduado.
`disc_capacitor`	função	`(a, d, h, *, cartesian, L=12.0)`	Capacitância de dois discos coaxiais, em solução axissimétrica ou 3-D cartesiana num oitavo do domínio.
`hall_bar`	função	`(L, W, beta, n, V=1.0, nx=None)`	Potencial numa barra Hall com contatos nas pontas e bordas isolantes, para um parâmetro de Hall dado.

`pinneapple_science.em.waves`

Ondas e óptica (acadêmico): radiação de dipolo, onda plana em meio com perdas, Fresnel e Snell, modos de guia de onda, ressonância de cavidade e espalhamento de Mie, via FDTD ou autovalores contra formas fechadas. [sem teste direto]

`pinneapple_science/em/waves.py` · 599 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`DipoleRadiation`	classe	`()`	Experimento `em.dipole_radiation`: campos e fluxo de Poynting de um dipolo hertziano, em 3-D e 3-D + tempo.
`LossyPlaneWave`	classe	`()`	Experimento `em.lossy_plane_wave`: mede atenuação e fase de um pulso num dielétrico condutor por FDTD 1-D.
`FresnelSnell`	classe	`()`	Experimento `em.fresnel_snell`: coeficientes de reflexão s e p e lei de Snell por FDTD 2-D.
`WaveguideModes`	classe	`()`	Experimento `em.waveguide_modes`: frequências de corte TE e TM de um guia WR-90 por autovalores de Helmholtz.
`CavityResonance`	classe	`()`	Experimento `em.cavity_resonance`: ressonâncias de uma cavidade PEC pelo espectro de uma sonda em FDTD.
`MieScattering`	classe	`()`	Experimento `em.mie_scattering`: eficiências de espalhamento e extinção de uma esfera pela série de Mie contra valores publicados.
`dipole_cartesian`	função	`(X, Y, Z, Il, omega)`	Campos fasoriais E e H em componentes cartesianas de um dipolo hertziano na origem.

`pinneapple_science.instruments`

Instrumentos virtuais (erro de ganho e offset sorteados uma vez, ruído por leitura, resolução) e ajustes lineares e não lineares com incerteza GUM tipo A e tipo B.

`pinneapple_science/instruments.py` · 194 linhas · testes: 19

nome	tipo	assinatura	o que faz
`Instrument`	classe	`(name, unit, noise, gain_u, offset_u, resolution, seed, _rng, _gain, _offset)`	Instrumento de leitura com ruído, incerteza de ganho e de offset e resolução conhecidos só pela especificação.
`Oscilloscope`	classe	`(rate, duration, v_range, bits, noise, gain_u, timebase_u, seed)`	Osciloscópio digital virtual: amostra um sinal com ruído, erro de ganho, ADC de `bits` e erro de base de tempo.

nome	tipo	assinatura	o que faz
`LinearFit`	classe	`(slope, intercept, slope_u, intercept_u, cov, chi2, dof, residuals)`	Resultado de ajuste linear: inclinação, intercepto, incertezas, covariância, qui-quadrado e resíduos.
`CurveFit`	classe	`(params, errors, cov, chi2, dof, residuals)`	Resultado de ajuste não linear: parâmetros, erros, covariância, qui-quadrado e resíduos.
`FieldProbe`	função	`(**kw) -> Instrument`	Cria um `Instrument` que representa uma sonda Hall de campo magnético (tesla).
`Ammeter`	função	`(**kw) -> Instrument`	Cria um `Instrument` amperímetro (unidade A).
`Voltmeter`	função	`(**kw) -> Instrument`	Cria um `Instrument` voltímetro (unidade V).
`linear_fit`	função	`(x, y, sigma_y=None, *, through_origin=False, scale_by_chi2=True) -> LinearFit`	Mínimos quadrados ponderados y = a x + b, com incertezas da covariância escaladas pelo qui-quadrado reduzido.
`curve_fit`	função	`(model, x, y, p0, sigma_y=None) -> CurveFit`	Mínimos quadrados não lineares (scipy `curve_fit`) com a mesma regra de incerteza.
`combine`	função	`(*relative_uncertainties) -> float`	Soma quadrática de incertezas relativas independentes (GUM).

`pinneapple_science.numerics`

Blocos numéricos compartilhados: malhas tensoriais graduadas, volumes finitos para `div(eps grad phi) = -rho` em 1-D, 2-D e 3-D, energia de campo, autovalores de Helmholtz, picos espectrais e escala de GP.

`pinneapple_science/numerics.py` · 244 linhas · testes: 19

nome	tipo	assinatura	o que faz
`TensorGrid`	classe	`(axes)`	Coordenadas dos nós por eixo (1, 2 ou 3 eixos), com malha e larguras duais.
`graded_axis`	função	`(core, h, length, growth=1.12, start=0.0) -> np.ndarray`	Gera nós com passo fixo numa região central e crescimento geométrico até o fim do domínio.
`solve_potential`	função	`(grid, fixed, values, *, eps=None, rho=None, tol=1e-10, direct=None, mean='harmonic') -> …)`	Resolve `-div(eps grad phi) = rho` com nós de potencial fixo e faces de simetria.
`field_energy`	função	`(grid, phi, eps=None, mean='harmonic') -> float`	Energia discreta do campo 1/2 Σ ε |∇φ|² dV com os fluxos de face do esquema.
`helmholtz_eigs`	função	`(nx, ny, a, b, kind, n=6, inside=None, modes=False)`	Menores números de onda de corte de uma seção de guia (TM ou TE) por autovalores de Helmholtz.
`spectral_peaks`	função	`(signal, dt, f_max, *, n_peaks=8, min_rel=0.001, pad=8) -> np.ndarray`	Frequências dos picos mais fortes do espectro de um sinal, com janela de Hann, zero-padding e interpolação parabólica.
`refine_ladder`	função	`(base, levels, ratio=2) -> Sequence[int]`	Escada de refinamento `base * ratio**i`.
`gp_loo_scale`	função	`(gp) -> float`	Fator de escala do desvio preditivo de um processo gaussiano a partir dos resíduos leave-one-out.

`pinneapple_science.plots`

Convenções de figura: paleta fixa (calculado e referência), mapas sequencial e divergente, eixos discretos; nada é salvo aqui. [sem teste direto]

`pinneapple_science/plots.py` · 83 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`style`	função	`(ax) -> None`	Aplica o estilo do pacote a um eixo matplotlib.
`figure`	função	`(ncols=1, nrows=1, *, width=4.2, height=3.3, three_d=(), title='')`	Cria figura e lista plana de eixos, com eixos 3-D nos índices pedidos.
`badge`	função	`(ax, text, ok=True, loc='lower right') -> None`	Caixa de texto com um resultado medido, com moldura indicando passou ou falhou.
`close`	função	`(fig) -> None`	Fecha uma figura matplotlib.
`log_ticks_ok`	função	`(values) -> bool`	Diz se todos os valores são positivos (válidos para eixo log).

nome	tipo	assinatura	o que faz
`to_rgb`	função	`(fig) -> np.ndarray`	Converte a figura num array RGB, para quadros de animação.
`integer_ticks`	função	`(ax, values) -> None`	Põe rótulos inteiros simples num eixo x logarítmico.

`pinneapple_science.registry`

Registro dos experimentos por nome; os módulos são importados no primeiro uso. [sem teste direto]

`pinneapple_science/registry.py` · 68 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`register`	função	`(cls) -> type[Experiment]`	Decorador de classe que adiciona um experimento ao catálogo.
`get`	função	`(name) -> Experiment`	Instancia o experimento pelo nome, com erro listando os conhecidos.
`names`	função	`() -> list[str]`	Lista os nomes dos experimentos registrados.
`experiments`	função	`(area=None, *, level=None, kind=None, dim=None) -> list[dict[str, Any]]`	Cartões dos experimentos que casam com os filtros (área, dimensão etc.).
`run`	função	`(name, dim=None, *, quick=False, **params) -> Result`	Roda um experimento numa dimensão (a primeira por padrão) com parâmetros sobre os padrões.

`pinneapple_science.report`

Resultados versionados: uma pasta por experimento em `examples/science/` com todas as dimensões, manifesto do Lab, selo e README, mais o catálogo e o atlas. [sem teste direto]

`pinneapple_science/report.py` · 204 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`folder_of`	função	`(name, root=None) -> str`	Caminho da pasta de resultados de um experimento.
`write`	função	`(name, *, root=None, quick=False, register=True, seal=True) -> dict[str, Any]`	Roda um experimento em todas as dimensões, salva, escreve README e manifesto, registra no Lab e carimba as figuras.
`readme`	função	`(name, results, sealed, item) -> str`	Gera o README de um experimento a partir dos resultados.
`table`	função	`(root=None) -> list[dict[str, Any]]`	Tabela experimento x dimensão x grandeza x referência x erro medido a partir dos `result.json` versionados.
`write_catalogue`	função	`(root=None, register=True) -> str`	Escreve `examples/science/README.md`, `ATLAS.md`, `atlas.html` e o manifesto da pasta.
`write_all`	função	`(*, root=None, quick=False, register=True) -> list[dict[str, Any]]`	Roda `write` para todos os experimentos e depois escreve o catálogo.

Exemplos de uso reais do repositório

Atlas, filtros e execução de um experimento — `docs/guides/science.md`

```python
import pinneapple as pp

pp.science.atlas()                                  # every area and subarea, with its status and pointers
pp.science.experiments(area="electromagnetism", dim="3D+t")
r = pp.science.run("em.faraday_induction")          # a magnet falling through a loop, 3-D in time
print(r)                                            # computed vs reference, physics checks, trust decision
r.save("out/faraday")                               # result.json, figures, the animation (MP4 and GIF)
```

Estudo FDTD 3-D configurado por dicionário — `docs/guides/science.md`

```python
from pinneapple_science.em.fdtd_study import FDTDConfig, run_fdtd
```

```
cfg = FDTDConfig.from_dict({"f0": 1e9, "size": 2.4, "cells_per_wavelength": 14,
                            "sources": [{"kind": "dipole", "component": "Ez", "at": [0, 0, 0]}],
                            "materials": [{"shape": "box", "center": [0.5, 0, 0], "size": [0.2, 0.6, 0.6],
                                           "eps_r": 6.0, "sigma": 0.01}]})
run = run_fdtd(cfg, snapshots=24)            # run.power (W at f0), run.pattern(theta), run.slices, run.volume
```

Todo experimento rápido passa nas suas referências — `tests/test_science.py`

```
@pytest.mark.parametrize("name", FAST)
def test_experiment_passes_its_references(name):
    e = S.get(name)
    for d in e.dims:
        r = e.run(d, quick=True)
        assert r.comparisons, f"{name} [{d}] compares nothing"
        bad = [c.quantity for c in r.comparisons if not c.passed] + [k["name"] for k in r.checks if k["passed"]
is False]
        assert not bad, f"{name} [{d}]: {bad}"
        assert r.decision == "APPROVED"
```

Testes e status de validação

- Módulos com pelo menos um teste que os importa: 6 de 16.
- Funções de teste em arquivos que importam algum módulo do pacote: 19.

Limitações conhecidas

- Só a área de eletromagnetismo (`em`) tem experimentos implementados; as demais áreas aparecem no atlas como parciais ou planejadas, com ponteiros para código de outros pacotes.
- As fórmulas de Palmer e de Kirchhoff usadas como referência do capacitor são assintóticas; o guia `docs/guides/science.md` diz que a borda da placa fina limita a ordem dos volumes finitos a cerca de 1.
- O dipolo ao lado de uma esfera converge em primeira ordem (esfera em escada), segundo o guia; os estudos de pesquisa reportam GCI e incerteza em vez de esconder esse erro.
- `Experiment._run` da classe base levanta `NotImplementedError`; cada experimento precisa implementá-lo.
- A maioria dos módulos não tem teste direto; `tests/test_science.py` cobre o pacote pelo registro e roda os experimentos rápidos em modo `quick`.
- `report.write` depende do `pinneapple_lab` (manifesto, registro e selo) e escreve em `examples/science/`; ajustes não lineares usam scipy e as figuras usam matplotlib.

5. `pinneapple_condense`

Objetivo

`pinneapple_condense` implementa a "Adaptive Physics Condensation" (o `__init__` diz "work in progress"): resolver problemas físicos grandes feitos de blocos repetidos sem montar e resolver a malha fina inteira. O caso de referência é um encapsulamento de chip sintético com várias camadas (substrato, bumps C4, interposer com TSVs, microbumps, die, TIM, tampa de cobre), montado a partir de uma célula unitária repetida. Os materiais são valores representativos, não dados de um produto real.

A ideia central é a condensação estática (complemento de Schur): o interior de cada bloco é eliminado uma vez por tipo distinto de bloco, e o problema global fica só nas interfaces, que ainda podem ser reduzidas a poucos modos por face (POD ou modos de restrição). Sobre isso o pacote constrói uma hierarquia que reaproveita subárvores idênticas (`hierarchy`), um estimador de erro com refinamento por face, uma correção exata por PCG, uma versão transiente Craig-Bampton (`transient`), operadores de bloco aprendidos com verificação e fallback exato (`learned`), a homogeneização Keff como linha de base (`homogenize`), um método de "prior + patches de exceção" (`exceptions`) e uma versão algébrica que condensa qualquer matriz esparsa (`algebraic`).

O pacote também tem as soluções de referência (volumes finitos de condução em `fv`, escoamento 2D em grade MAC em `flow`), discretizações sem malha RBF-FD/GFDM (`meshless`, `meshless2d`) e os estudos medidos: `benchmark` gera os números de `docs/research/physics_condensation.md` e `falsify` faz os testes pré-registra-

dos de `docs/research/condensation_falsification.md`; `figures` desenha os gráficos.

Liga-se ao resto do repositório por `query`, que fornece os métodos (cache, limite rigoroso, superposição, condensado, referência) do motor de consultas físicas `pinneapple_query.Engine`; pelos nós `condense.*` e `query` de `pinneapple_flow`; e por `pinneapple_analysis.validation`. Os exemplos `examples/physics_condensation` e `examples/condensation_falsification` rodam os estudos.

Números do inventário

módulos	linhas	classes públicas	funções públicas	testes em arquivos que importam o pacote	módulos sem teste direto
18	6.006	29	115	27 (3 arquivos)	3

Submódulos

submódulo	o que faz	módulos	classes + funções	testes*
`algebraic`	Condensação algébrica exata de qualquer operador esparso montado, particionado em blocos por posição, com faces reduzidas por modos POD.	1	6	6
`benchmark`	Estudos medidos do benchmark de condensação (convergência, modos, refinamento, escala, aprendido, transiente), cada um devolvendo números em JSON.	1	14	0
`exceptions`	Método "prior + patches de exceção": uma solução barata (Keff ou coluna 1D) corrigida exatamente só nos blocos de maior resíduo.	1	6	14
`falsify`	Testes de falsificação pré-registrados do método (térmicos T1-T10, CFD F0-F3, sem malha M1-M4, patches E1).	1	21	6
`figures`	Figuras matplotlib do benchmark a partir do JSON de `benchmark`.	1	8	0
`flow`	Escoamento 2D incompressível e calor em grade escalonada (MAC) por volumes finitos: Stokes/Oseen, Picard para Navier-Stokes e equação escalar.	1	9	6
`fv`	Volumes finitos centrados na célula para condução estacionária e transiente numa caixa de voxels, e o solver de referência.	1	8	21
`hierarchy`	Condensação hierárquica aninhada com memoização de subárvores idênticas.	1	2	14
`homogenize`	Homogeneização por condutividade efetiva (Keff), a linha de base que a condensação precisa superar.	1	7	14
`learned`	Operador de bloco reduzido aprendido (POD + regressão GP) com indicadores de erro e fallback para condensação exata.	1	10	14
`meshless`	Solução sem malha (RBF-FD/GFDM) do encapsulamento a partir da geometria implícita, com pares de nós nas interfaces.	1	9	14
`meshless2d`	Discretizações 2D sem malha dos casos de CFD da falsificação (advecção-difusão, Stokes, condução híbrida FV/RBF-FD).	1	9	6
`package`	Encapsulamento de chip multicamada sintético feito de um bloco repetido: camadas, variantes, mapas de potência.	1	7	27
`query`	Métodos de consulta física sobre o encapsulamento para `pinneapple_query.Engine` (cache, limite, superposição, condensado, referência).	1	8	7
`query_benchmark`	Demonstração medida do motor de consultas sobre o encapsulamento contra a referência fina.	1	1	0
`substructure`	Condensação estática de blocos repetidos, interfaces reduzidas, recuperação sob demanda, estimativa de erro e refinamento.	1	15	21
`transient`	Condução transiente com redução Craig-Bampton por bloco e Euler implícito, mais a referência fina.	1	4	14

* maior número de testes que importam um módulo do submódulo.

Módulos e API pública

`pinneapple_condense`

Pacote da Adaptive Physics Condensation; o `__init__` só tem o docstring "work in progress" e não reexporta nada. [experimental]

`pinneapple_condense/__init__.py` · 2 linhas · testes: 27

pinneapple_condense.algebraic

Aplica o método de `substructure` a qualquer matriz esparsa (simétrica ou não, ponto de sela ou não): separadores pelo grafo de acoplamento, blocos comparados por conteúdo e faces reduzidas por modos.

`pinneapple_condense/algebraic.py` · 299 linhas · testes: 6

nome	tipo	assinatura	o que faz
`Partition`	classe	`(block, sep, interiors, adjacent, shape, size, origin, coords, fieldid)`	Partição das incógnitas: bloco de cada incógnita, máscara de separador e, por bloco, interiores e separadores adjacentes em ordem canônica.
`Condensation`	classe	`(A, part, timings)`	Condensação algébrica exata de `A x = b` sobre uma `Partition`, fatorando uma vez cada bloco de conteúdo distinto.
`Reduced`	classe	`(cond, fc, modes)`	Problema do separador reduzido por Galerkin sobre modos de face (exato quando cada face mantém todos os modos).
`partition`	função	`(A, coords, block_size, origin=(0.0, 0.0), fieldid=None, primal=None, h=None) -> Partition`	Divide as incógnitas em blocos de lado `block_size` e define os separadores pelo grafo de acoplamento, com `primal` opcional.
`faces`	função	`(part, A) -> dict`	Agrupa os separadores pelo conjunto de blocos que acoplam: faces entre dois blocos com posições canônicas; cantos e incógnitas primais ficam completos.
`face_modes`	função	`(snapshots, fc, m) -> dict`	Modos POD por tipo de face a partir de snapshots: a constante mais os `m - 1` principais vetores singulares.

pinneapple_condense.benchmark

Estudos medidos do benchmark de condensação; cada função devolve números JSON e pode rodar num processo novo (`isolated`) para medir memória de pico. [sem teste]

`pinneapple_condense/benchmark.py` · 396 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`peak_rss_gb`	função	`() -> float`	Memória residente de pico do processo atual, em GB.
`isolated`	função	`(func, timeout=3600, module=…, **kwargs) -> dict`	Roda `<module>.<func>(**kwargs)` num processo Python novo e devolve o resultado mais o pico de memória desse processo.
`mesh_convergence`	função	`(shape=(8, 8), levels=((4, 1), (8, 2), (16, 4)), seed=1) -> dict`	Soluções de referência em resoluções crescentes, ordem observada, extrapolação de Richardson e erro da resolução padrão.
`accuracy_vs_modes`	função	`(shape=(12, 12), ms=(1, 2, 4, 6, 8, 12, 16, 24), seed=1, modes='pod') -> dict`	Compara o condensado com a referência (Tmax, campo de junção, calores de interface, balanço de energia, estimativa de erro) para vários números de modos `m`.
`refinement_study`	função	`(shape=(12, 12), m0=2, steps=6, seed=1, hot=True) -> dict`	Refinamento de faces guiado pelo erro (Doerfler) contra aumentar `m` uniformemente, com hotspot menor que o passo.
`correction_study`	função	`(nb=32, ms=(6, 12), seed=0, power_scale=4.0, rtol=1e-09, every=10) -> dict`	Correção exata (`Condensed.correct`) num caso grande com hotspot: iterações até a solução fina e erro de Tmax ao longo delas.
`homogenization_study`	função	`(shape=(12, 12), seed=1) -> dict`	O que a homogeneização Keff apaga em três casos: potência suave, hotspot menor que o passo e bumps com vazios.
`scaling_point`	função	`(nb, m=12, reference=True, flat=True, hier=True, seed=2) -> dict`	Um tamanho de encapsulamento `nb x nb`: tempo e resultado de referência, condensação plana e hierarquia.
`configurations`	função	`(nb=64, m=12, n_configs=20, seed=0, check=2) -> dict`	Constrói uma vez e resolve vários mapas de potência novos, para separar custo de construção e custo por mapa.
`learned_study`	função	`(nb=32, m=12, n_train=48, seed=1, defects=4, exact_reference=True, tol_T=0.01) -> dict`	Operadores de bloco aprendidos com verificação contra a condensação exata por bloco, com defeitos.
`transient_study`	função	`(shape=(6, 6), dt=0.0002, steps=150, mk=((8, 8), (16, 16), (32, 32)), seed=1) -> dict`	Transiente Craig-Bampton contra a referência fina, com potência alternando entre dois mapas.
`root_compressibility`	função	`(nb=64, m=12, tol=1e-08) -> dict`	Posto numérico dos blocos fora da diagonal do complemento de Schur raiz: quanto uma representação H-matriz economizaria.

nome	tipo	assinatura	o que faz
`meshless_study`	função	`(nb=2, hfs=(6, 8, 10, 12), methods=('gfdm', 'rbf_fd'), interfaces=('pairs', 'naive'), ls_…)`	Trilha 1b: nuvens RBF-FD e GFDM contra a referência FV para vários espaçamentos, tratamentos de interface e colocação por mínimos quadrados.
`exceptions_study`	função	`(shape=(12, 12), seed=1, budgets=…) -> dict`	Trilha 1c: prior Keff com patches escolhidos por resíduo contra patches aleatórios no mesmo orçamento, e o prior de coluna 1D.

`pinneapple_condense.exceptions`

Trilha 1c: um campo prior barato (Keff ou coluna 1D) e solução exata só nos patches de blocos onde o resíduo do prior é grande, com correção final de energia.

`pinneapple_condense/exceptions.py` · 190 linhas · testes: 14

nome	tipo	assinatura	o que faz
`prior`	função	`(pkg, kind='keff') -> tuple[np.ndarray, float]`	Calcula o campo prior nas células finas e o seu custo em segundos.
`block_indicator`	função	`(pkg, sysm, T, f) -> np.ndarray`	Norma do resíduo `K T - f` sobre as células de cada bloco.
`correct`	função	`(pkg, T0, blocks, *, halo=1, blend='core', sysm=None, f=None) -> tuple[np.ndarray, dict]`	Resolve o problema fino nos patches em volta dos blocos escolhidos, com `T0` como dado de Dirichlet fora.
`energy_correct`	função	`(pkg, T, sysm) -> tuple[np.ndarray, float]`	Soma `alpha * phi` (resposta a um mapa uniforme) para que o fluxo de saída Robin iguale a potência.
`run`	função	`(pkg, *, budget=0.1, select='residual', halo=1, blend='core', prior_kind='keff', energy=T…)`	O método inteiro para um orçamento (fração de blocos escolhidos): campo e custos.
`pareto`	função	`(pkg, budgets=…, *, halos=(0, 1), selects=('residual', 'random'), prior_kind='keff', seed…)`	Erro contra fração de células resolvidas exatamente, por regra de seleção e halo, com a referência.

`pinneapple_condense.falsify`

Estudos de falsificação pré-registrados do método de condensação; cada função é um teste que devolve números JSON e deve rodar num processo novo.

`pinneapple_condense/falsify.py` · 989 linhas · testes: 6

nome	tipo	assinatura	o que faz
`t1_reference`	função	`(nb, seed=2, m=12, rtol=1e-10, fine=True, maxiter=2000) -> dict`	Temperatura de junção de pico de referência: solução fina por AMG e condensação exata por PCG com o histórico de convergência.
`t1_point`	função	`(nb, m, seed=2) -> dict`	Hierarquia memoizada com `m` modos: temperatura de pico e tempos de construção e solução.
`f0_validation`	função	`(poiseuille=(16, 32, 64), cavity=(32, 64, 128), nusselt=(16, 32, 64)) -> dict`	Valida o solver de escoamento contra Poiseuille plano, a cavidade com Re = 100 e o número de Nusselt de placas paralelas.
`f2_stokes`	função	`(n_pins=16, cells=16, pin=8, ms=(1, 2, 4, 8, 16), n_snap=24, seed=0) -> dict`	F2/F2r: Stokes num arranjo de pinos, condensação algébrica exata contra solução direta global e depois faces reduzidas por snapshots.
`f1_advection`	função	`(n_pins=8, cells=16, pin=8, pes=(0.1, 1, 10, 100, 1000), ms=(1, 2, 4, 8, 16), n_snap=24, …)`	F1/F1r: advecção-difusão estacionária no arranjo de pinos com varredura do número de Péclet, condensação exata do operador não simétrico e faces reduzidas.
`f3_cht`	função	`(n_pins=(8, 4), cells=16, pin=8, res=(1, 10, 50, 100), ms=(2, 4, 8, 16), n_snap=16, tol=1…)`	F3/F3r: placa fria com aletas, Navier-Stokes por Picard com cada iterado condensado, depois a energia conjugada e faces reduzidas.
`thermal_case`	função	`(pkg, m=12, rtol=1e-10) -> dict`	Um encapsulamento: referência FV, condensação com `m` modos (erro de Tmax, efetividade da estimativa) e condensação exata por PCG.
`t2_contrast`	função	`(nb=32, ratios=(1000.0, 10000.0, 100000.0), m=12, seed=2) -> dict`	T2: aumenta o contraste de condutividade cobre/underfill para várias razões.
`t3_anisotropy`	função	`(nb=32, factors=(100.0, 0.01), m=12, seed=2) -> dict`	T3: TIM e interposer com razão de condutividade no plano / através da espessura.
`t8_adversarial`	função	`(nb=16, m=12, seed=2) -> dict`	T8: mapas de potência diferentes do conjunto de snapshots, com a mesma potência total.
`t9_thin`	função	`(nb=16, m=12, seed=2) -> dict`	T9: camadas finas (TIM e die mais finos).

nome	tipo	assinatura	o que faz
`t4_no_repetition`	função	`(nb=32, m=12, seed=1, n_train=48) -> dict`	T4: todos os blocos diferentes; caminho exato (cada bloco condensado mais correção PCG) contra o caminho aprendido.
`t5_variants`	função	`(nb=64, Vs=(1, 4, 16, 64, 256), m=12, seed=0) -> dict`	T5: `V` variantes discretas em posições aleatórias; construção da hierarquia contra a condensação plana.
`t6_t7_nonlinear`	função	`(nb=16, case='k', seed=2, power_scale=2.0) -> dict`	T6 (condutividade k(T) de silício e cobre) ou T7 (resistência de contato R_c(T) entre die e TIM).
`t10_pitch`	função	`(nb=16, pitches=(4e-05, 2e-05), ms=(2, 4, 8, 12, 16, 24), n_snap=24, seed=2) -> dict`	T10: colunas de blocos alternando o passo, por condensação algébrica, contra o passo uniforme.
`m1_advection`	função	`(pins=4, densities=(8, 16, 32), truth_cells=64, pes=(1, 10, 100), methods=('rbf_fd', 'gfd…)`	M1: advecção-difusão sem malha em nuvens de pontos contra FV com o mesmo número de incógnitas.
`m2_stokes`	função	`(pins=4, densities=(8, 16, 32), truth_cells=64, methods=('rbf_fd', 'gfdm'), seed=0) -> di…)`	M2: Stokes colocado sem malha nos pontos de fluido; queda de pressão contra FV.
`m3_hybrid`	função	`(pins=2, densities=(8, 16, 32), truth_cells=64, k_pin=400.0, k_mat=1.0, R_c=0.001, stenci…)`	M3: híbrido FV nas interfaces + sem malha no resto contra FV em toda parte, com erro separado por região e sensibilidade ao estêncil.
`m4_meshless_condensed`	função	`(pins=4, cells=16, seed=0) -> dict`	M4: matrizes sem malha condensadas por `algebraic` (condução híbrida e advecção-difusão).
`exception_patches`	função	`(A, b, x0, x_true, block, nbshape, budgets, select='residual', halo=1, seed=0)`	Versão genérica de prior + patches de exceção para qualquer sistema `A x = b`, por resíduo ou aleatório.
`e1_cfd`	função	`(n_pins=8, fine=16, coarse=4, budgets=…, Pe_b=100.0, Re=50.0, seed=0) -> dict`	E1: patches por resíduo contra aleatórios na advecção-difusão e no campo de energia da placa fria, com prior numa grade mais grossa.

`pinneapple_condense.figures`

Figuras matplotlib do benchmark de condensação, lidas do JSON de `benchmark`. [sem teste]

`pinneapple_condense/figures.py` · 214 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`scaling`	função	`(points, path) -> str`	Tempo e memória de pico contra o número de blocos (dois painéis).
`accuracy`	função	`(res, path) -> str`	Gráfico dos erros do condensado contra o número de modos de interface.
`refinement`	função	`(res, path) -> str`	Gráfico do erro contra graus de liberdade de interface, refinamento guiado contra uniforme.
`keff_maps`	função	`(maps, path) -> str`	Mapas da temperatura de junção fina, das médias homogeneizadas por bloco e da diferença.
`transient`	função	`(res, path) -> str`	Gráfico do transiente Craig-Bampton contra a referência.
`meshless_convergence`	função	`(res, path, mesh_error=None) -> str`	Erro de Tmax e de balanço de energia contra número de nós, por método e tratamento de interface.
`meshless_fields`	função	`(res, method, path) -> str`	Nuvem de pontos numa fatia vertical colorida pela temperatura e o mapa de erro da junção.
`pareto`	função	`(res, path) -> str`	Erro contra fração de células resolvidas exatamente (patches de exceção), por regra e halo.

`pinneapple_condense.flow`

Escoamento 2D incompressível e calor em grade MAC por volumes finitos, montado como um sistema esparso: referência e operador dos testes de falsificação de CFD.

`pinneapple_condense/flow.py` · 330 linhas · testes: 6

nome	tipo	assinatura	o que faz
`Grid`	classe	`(nx, ny, h, solid, walls, closed, lid, index)`	Grade MAC: tamanho, espaçamento, células sólidas, tipo de paredes, cavidade fechada/tampa móvel e a numeração das incógnitas.
`ScalarIndex`	classe	`(nx, ny)`	Numeração das incógnitas escalares por célula da grade.
`assemble_oseen`	função	`(g, nu, inlet=None, adv=None, force=(0.0, 0.0)) -> tuple[sp.csr_matrix, np.ndarray]`	Monta o sistema de Oseen (Stokes quando `adv` é None) com perfil de entrada, velocidade advectiva e força de corpo.
`split`	função	`(g, x) -> tuple[np.ndarray, np.ndarray, np.ndarray]`	Separa o vetor solução em `u`, `v` e `p` (NaN nas células sólidas).
`solve`	função	`(A, b) -> np.ndarray`	Solução direta esparsa (`spsolve`) de `A x = b`.
`picard`	função	`(g, nu, inlet=None, tol=1e-08, maxiter=200, relax=1.0, linear_solver=None, force=(0.0, 0.…)`	Navier-Stokes por iteração de Picard (Oseen) a partir de Stokes, com histórico da variação relativa e tempos por iteração.
`assemble_scalar`	função	`(g, u, v, k, rhoc, q, T_in=0.0, scheme='hybrid') -> tuple[sp.csr_matrix, np.ndarray]`	Monta `rho c (u . grad T) - div(k grad T) = q` em todas as células, esquema híbrido ou upwind.
`scalar_coords`	função	`(g) -> np.ndarray`	Coordenadas dos centros das células da grade.
`pin_array`	função	`(n_pins, cells, pin, h, inlet_cells=0, outlet_cells=0) -> np.ndarray`	Máscara sólida de um arranjo de pinos quadrados, com colunas vazias opcionais antes e depois.

`pinneapple_condense.fv`

Volumes finitos centrados na célula para condução estacionária e transiente numa caixa de voxels (contato, Robin, portas laterais), e o solver de referência.

`pinneapple_condense/fv.py` · 174 linhas · testes: 21

nome	tipo	assinatura	o que faz
`System`	classe	`(K, shape, ports, g_top, g_bottom, Gz, C)`	Operador de condução montado: `K`, forma, blocos de portas, condutâncias Robin de topo e base, condutâncias verticais e capacidade `C`.
`SolveInfo`	classe	`(method, seconds, iterations, residual)`	Informação de uma solução: método, segundos, iterações e resíduo.
`assemble`	função	`(k, dz, R, dx, h_top, h_bottom, *, ports=False, rho_c=None, dy=None) -> System`	Monta o operador de condução de uma caixa de voxels (condutividade, espessuras, resistências de contato, Robin, portas e capacidade opcionais).
`assemble_package`	função	`(pkg, *, transient=False) -> System`	O encapsulamento inteiro como uma caixa adiabática nos lados: o modelo de referência.
`source_vector`	função	`(pkg, power=None) -> np.ndarray`	Vetor de fonte de calor (W) de cada célula: o mapa de potência na linha ativa.
`factor`	função	`(K)`	LU esparsa de uma matriz simétrica (grau mínimo em `K + K^T`, pivoteamento simétrico).
`solve`	função	`(K, f, *, method='auto', rtol=1e-11, maxiter=4000) -> tuple[np.ndarray, SolveInfo]`	Resolve `K T = f`: direto (SuperLU) até 6e4 incógnitas, senão gradientes conjugados com multigrid algébrico (`pyamg`) ou Jacobi.
`solve_package`	função	`(pkg, power=None, **kw) -> tuple[np.ndarray, SolveInfo]`	Elevação de temperatura de referência de cada célula do encapsulamento.

`pinneapple_condense.hierarchy`

Condensação hierárquica: divide o arranjo de blocos ao meio recursivamente, funde filhos eliminando faces compartilhadas e memoiza operadores de subárvores idênticas.

`pinneapple_condense/hierarchy.py` · 356 linhas · testes: 14

nome	tipo	assinatura	o que faz
`Hierarchy`	classe	`(pkg, m=6, *, modes='pod', keep_root=True)`	Condensação aninhada de um encapsulamento com `m` modos por face: preparação, passagem para cima das cargas, solução, observáveis e recuperação de um bloco.
`estimate`	função	`(h, sol, chunk=4096) -> np.ndarray`	Indicador de erro por face de uma solução hierárquica, como em `Condensed.estimate`.

pinneapple_condense.homogenize

Homogeneização Keff: cada camada com inclusões vira uma camada anisotrópica (regra paralela em z, limite de Hashin-Shtrikman no plano); é a linha de base que apaga o que ocorre dentro do bloco.

`pinneapple_condense/homogenize.py` · 121 linhas · testes: 14

nome	tipo	assinatura	o que faz
`effective_layer`	função	`(k_m, k_i, phi) -> tuple[float, float]`	Condutividades efetivas `(k_xy, k_z)` de uma camada com fração `phi` de prismas alinhados.
`effective_properties`	função	`(stack, variant=BASE) -> dict[str, np.ndarray]`	Propriedades efetivas por linha z de um bloco: `kxy`, `kz`, `rho_c`, `dz` e resistência de contato.
`homogenized_system`	função	`(pkg, cells=1, transient=False)`	Sistema do encapsulamento homogeneizado com `cells x cells` células laterais por bloco.
`coarse_power`	função	`(pkg, cells=1, power=None) -> np.ndarray`	Mapa de potência agregado na grade grossa homogeneizada.
`solve_homogenized`	função	`(pkg, cells=1, power=None)`	Campo de temperatura grosso e informação da solução.
`homogenized_observables`	função	`(pkg, T, sysm, cells=1) -> dict[str, np.ndarray]`	Por bloco: temperatura média de junção, calor die/TIM, calor na camada de microbumps e estimativa do calor por bump.
`compare`	função	`(pkg, ref, hom) -> dict[str, float]`	Mede o que a homogeneização apaga contra os observáveis de referência (pico, médias, espalhamento dentro do bloco, calores).

pinneapple_condense.learned

Operador de bloco reduzido aprendido em função de parâmetros (POD + `ParametricPOD` com GP), nunca usado às cegas: blocos incertos ou fora da caixa de treino são condensados exatamente e uma auditoria verifica o resultado.

`pinneapple_condense/learned.py` · 310 linhas · testes: 14

nome	tipo	assinatura	o que faz
`LearnedBlocks`	classe	`(stack, m, bounds, to_variant, n_train, regressor, seed, timings)`	Mapa aprendido `theta -> operador reduzido do bloco`; `fit()` condensa exatamente amostras Latin hypercube e treina a regressão.
`AdaptiveResult`	classe	`(obs, tb, exact_blocks, checked, failed, rounds, timings, flagged_std, flagged_ood, estim…)`	Resultado de `solve_adaptive`: observáveis, blocos exatos, verificados e reprovados, rodadas, tempos e estimativa.
`default_variant`	função	`(theta) -> Variant`	Converte três parâmetros (microbump, contato die/TIM e TIM/tampa, interposer) numa `Variant`.
`reduced_block`	função	`(stack, variant, Phi) -> dict[str, np.ndarray]`	Operador reduzido exato de um bloco sobre os modos de interface `Phi` (uma fatoração, sem cache).
`assemble_blocks`	função	`(Sr, dofs, n) -> sp.csc_matrix`	Monta a matriz global de interface a partir dos operadores reduzidos por bloco e seus graus de liberdade.
`solve_blocks`	função	`(ops, pb, dofs, n)`	Monta, fatora e resolve o problema de interface; devolve valores de porta por bloco e tempos.
`block_observables`	função	`(ops, tb, pb, n2) -> dict[str, np.ndarray]`	Observáveis por bloco (temperaturas de junção e fluxos) a partir dos valores de porta e das potências.
`solve_adaptive`	função	`(learned, theta, power, *, tol_std=0.05, tol_T=0.01, n_hot=8, n_audit=16, max_rounds=5, s…)`	Condensação aprendida com o laço de verificação: previsão, substituição por exato onde incerto, solução, auditoria e novas rodadas.
`solve_exact`	função	`(learned, theta, power) -> tuple[dict, float]`	Todos os blocos condensados exatamente: a referência do caminho aprendido.

nome	tipo	assinatura	o que faz
variant_field	função	(shape, seed=0, *, defects=0, defect_k=0.05, bounds=None) -> np.ndarray	Parâmetros de bloco aleatórios e suaves dentro dos limites, com defects microbumps quase abertos fora da caixa de treino.

pinneapple_condense.meshless

Trilha 1b: solução sem malha do encapsulamento direto da geometria implícita (nuvem de pontos, pares de nós com continuidade de fluxo e lei de contato nas interfaces, estênceis RBF-FD ou GFDM).

pinneapple_condense/meshless.py · 637 linhas · testes: 14

nome	tipo	assinatura	o que faz
Cloud	classe	(X, region, kind, normal, partner, area, k, R, seconds, info)	Nuvem de pontos: posições, região, tipo, normal, parceiro de interface, área, condutividade e resistência.
Geometry	classe	(pkg)	Descrição implícita do encapsulamento: material em cada ponto, camada, bloco, inclusões e fontes.
MeshlessSolution	classe	(cloud, T, A, b, timings, stencils)	Solução sem malha: nuvem, temperaturas, sistema montado, tempos e estênceis.
point_cloud	função	(pkg, *, h_xy=None, per_layer=3, h_max=2.5e-05, per_feature=4, band_h=2.0, gap=0.3, jitte…)	Gera os nós a partir da geometria implícita (retículo com jitter por região, pares em interfaces, densidade adaptativa opcional).
region_stencils	função	(X, region, Q, q_region, n, box, reach, normal=None)	Estênceis dos n vizinhos mais próximos da mesma região, com lados do encapsulamento como planos espelho.
assemble	função	(cloud, pkg, *, method='rbf_fd', degree=2, n=None, interface='pairs', oversample=False)	Monta o sistema de colocação (A, b, estênceis) para o método e tratamento de interface escolhidos.
solve	função	(pkg, cloud=None, *, method='rbf_fd', degree=2, n=None, interface='pairs', solver='auto',…)	Nuvem, montagem e solução (LU esparsa, ou GMRES com ILU acima de 2e5 nós), com estimativa de condicionamento opcional.
evaluate	função	(sol, pkg, P, *, degree=None) -> np.ndarray	Temperatura em pontos quaisquer, interpolada com estênceis da região de cada ponto.
observables	função	(sol, pkg) -> dict	Temperaturas de junção, calor die/TIM, fluxos de saída e balanço de energia da solução sem malha.

pinneapple_condense.meshless2d

Discretizações 2D sem malha (RBF-FD/GFDM) dos casos de CFD da falsificação, em nuvens de um arranjo de pinos quadrados construídas para dar matrizes idênticas em blocos idênticos.

pinneapple_condense/meshless2d.py · 339 linhas · testes: 6

nome	tipo	assinatura	o que faz
pin_sdf	função	(X, pins, pin_frac, offset=(1.0, 0.0))	Distância com sinal ao pino quadrado mais próximo (negativa dentro).
cloud	função	(pins, h, pin_frac=0.5, jitter=0.25, inlet_blocks=1, outlet_blocks=1, seed=0, fluid_only=…)	Pontos de retículo com jitter por célula unitária, iguais em todas as células, opcionalmente só fluido e com pontos de parede.
bilinear_mac	função	(g, u, v, X)	Velocidade de uma solução MAC de flow interpolada em pontos.
snap_to_box	função	(X, box, h)	Move para o contorno os pontos a menos de h/2 dele.
knn	função	(X, Q, n, tree=None)	Os n vizinhos mais próximos de cada ponto, com desempate determinístico pelo deslocamento.
advection_diffusion	função	(X, box, h, vel, alpha, q, *, method='rbf_fd', n=15, degree=2)	Monta vel . grad T - alpha lap T = q por colocação, com T = 0 na entrada e gradiente normal zero no resto.
stokes	função	(X, wall, box, h, nu=1.0, *, method='rbf_fd', n=20, degree=2)	Monta Stokes colocado (u, v, p) nos pontos de fluido, com não deslizamento nas paredes dos pinos.
hybrid_conduction	função	(pins, cells, pin_frac, k_pin, k_mat, R_c, *, band=2, point_h=None, jitter=0.25, n=12, de…)	Condução com resistência de contato: FV numa faixa em volta das interfaces e RBF-FD no resto, acoplados por interpolação.
fv_conduction	função	(pins, cells, pin_frac, k_pin, k_mat, R_c, q_pin=1.0, h_top=1.0)	O mesmo problema de condução só com volumes finitos (referência e linha de base).

pinneapple_condense.package

Encapsulamento de chip multicamada sintético feito de um bloco repetido (um C4, um TSV, um microbump), com variantes de bloco e mapas de potência; materiais representativos, não de um produto real.

pinneapple_condense/package.py · 253 linhas · testes: 27

nome	tipo	assinatura	o que faz
Layer	classe	(name, thickness, nz, matrix, inclusion, side)	Uma camada da pilha: nome, espessura, número de células em z, material da matriz, material e lado da inclusão.
Variant	classe	(inclusion_k, removed, contact)	O que muda num bloco sem mudar a geometria: fator de condutividade da inclusão, inclusões removidas, fator de contato.
Stack	classe	(layers, pitch, n, contact, h_top, h_bottom, active, materials, anisotropy)	A pilha de camadas com passo do bloco, células por bloco, resistências de contato, coeficientes Robin, camada ativa e materiais.
Package	classe	(stack, layout, variants, power)	Arranjo lateral de blocos sobre uma pilha, com layout de variantes e potência por célula ativa.
reference_stack	função	(n=8, vertical=1) -> Stack	A pilha do benchmark (passo de 40 um, cerca de 1 mm de altura, 8 x 8 células por lado de bloco por padrão).
hotspot_power	função	(stack, shape, total, *, hotspots=3, background=0.4, width=0.08, seed=0, centres=None) ->…)	Mapa de potência não uniforme: fundo uniforme mais hotspots gaussianos.
uniform_package	função	(shape, stack=None, *, total_power=None, power_density=1000000.0, seed=0, hotspots=3) -> …)	Encapsulamento de blocos idênticos com mapa de potência com hotspots.

pinneapple_condense.query

Trilha 2: métodos de consulta física sobre o encapsulamento para `pinneapple_query.Engine` (temperatura em ponto, pico em região, calor por feature, R_ja), do mais barato ao mais caro.

pinneapple_condense/query.py · 411 linhas · testes: 7

nome	tipo	assinatura	o que faz
State	classe	(power, obs, method, uncertainty, seconds, solver=None, sol=None, field=None, obs_unc=Non…)	Configuração resolvida: observáveis por bloco, método de origem, incerteza e o necessário para ler pontos internos.
PackageModel	classe	(pkg, m=8, reference_cells=2000000)	Maquinário comum dos métodos: o encapsulamento, os estados guardados, as hierarquias e a extração de quantidades.
Cached	classe	(model)	Método `cached`: lê um estado já resolvido com as mesmas condições, sem física nova.
Bound	classe	(model)	Método `bound`: intervalo rigoroso pela monotonicidade da M-matriz de condução.
Superposition	classe	(model, max_states=64)	Método `superposition`: combinação linear de estados resolvidos por mínimos quadrados nos mapas de potência, com resíduo limitado.
Condensed	classe	(model, expected_s=1.0)	Método `condensed`: nova solução com a hierarquia memoizada em `m`, incerteza |T_m - T_2m|.
Reference	classe	(model)	Método `reference`: solução fina por volumes finitos, o último recurso.
package_engine	função	(pkg, m=12, reference_cells=2000000, calibrate=True) -> tuple[Engine, PackageModel]	Monta um motor de consultas sobre um encapsulamento com esses métodos, calibrando opcionalmente o método condensado.

pinneapple_condense.query_benchmark

Demonstração medida do motor de consultas sobre o encapsulamento: milhares de consultas comparadas com a referência fina. [sem teste]

pinneapple_condense/query_benchmark.py · 125 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
query_demo	função	(nb=48, m=12, n_points=2000, n_interior=100, seed=0, reference=True) -> dict	Roda o conjunto de consultas e reporta latência por método, método escolhido, erro e se os intervalos rigorosos contêm a referência.

pinneapple_condense.substructure

Condensação estática dos blocos repetidos (complemento de Schur por bloco distinto), interfaces reduzidas a poucos modos, recuperação sob demanda, estimativa de erro e refinamento.

pinneapple_condense/substructure.py · 661 linhas · testes: 21

nome	tipo	assinatura	o que faz
BlockModel	classe	(stack, variant, S, W, Y, Z, ports, Om, obs_names, seconds, O_p, O_t)	Forma condensada de um bloco distinto: complemento de Schur, mapas de carga e de recuperação e observáveis.
CondensedState	classe	(t, pb, seconds)	Configuração condensada resolvida: graus de liberdade de interface e potências dos blocos.
Condensed	classe	(pkg, m=6, *, refined_x=None, refined_y=None, modes='pod', shift=0.0, face_m=None)	O encapsulamento condensado: operadores reaproveitados, m modos por face, sistema global fatorado uma vez, com estimativa, correção e refinamento.
block_functionals	função	(stack, variant, sysb) -> tuple[sp.csr_matrix, list[str]]	Linhas dos observáveis lineares de um bloco: temperaturas de junção e calores verticais em die/TIM, microbump, TSV, C4 e saída Robin.
block_model	função	(stack, variant=BASE, *, cache=True, shift=0.0) -> BlockModel	Condensa um bloco (com cache por pilha, variante e shift).
clear_cache	função	() -> None	Limpa o cache de modelos de bloco.
face_weights	função	(stack) -> np.ndarray	Área (m²) de cada nó de porta de uma face.
trace_snapshots	função	(stack, patch=4, shifts=(0.0,), *, global_size=16, global_maps=24) -> dict[str, np.ndarra…)	Traços de interface usados para construir os modos de face (com cache por pilha).
layered_modes	função	(stack, orient, m, shifts=(0.0,), *, groups='layer') -> np.ndarray	m modos de face que nunca ligam duas camadas: uma constante por camada e modos POD por camada.
interface_modes	função	(bm, orient, m, method='pod', shifts=(0.0,)) -> np.ndarray	m modos ortonormais (ponderados por área) de uma face compartilhada, o primeiro constante; POD por padrão.
reference_observables	função	(pkg, T) -> dict[str, np.ndarray]	Os mesmos observáveis de Condensed.observables calculados a partir de um campo fino completo.
energy_norm	função	(K, e) -> float	Norma de energia sqrt(e^T K e) de um vetor de erro.
errors	função	(obs, ref, total_power) -> dict[str, float]	Erros dos observáveis contra uma referência: Tmax, temperaturas de junção, calores relativos e balanço de energia.
tmax_cells	função	(T, active_k) -> tuple[float, float]	Máximo sobre todas as células e sobre a linha ativa (iguais pelo princípio do máximo discreto quando a potência é não negativa).
iterate_refinement	função	(pkg, m, tol, *, fraction=0.5, max_steps=6, power=None)	Resolve, estima e refina faces até a estimativa relativa de energia ficar abaixo de tol.

pinneapple_condense.transient

Condução transiente: redução Craig-Bampton de cada bloco (modos de restrição mais modos de interface fixa), Euler implícito com passo fixo, e a referência fina.

pinneapple_condense/transient.py · 199 linhas · testes: 14

nome	tipo	assinatura	o que faz
CBBlock	classe	(K, M, W, G, Ot, Op, lam)	Matrizes Craig-Bampton de um bloco (rigidez, massa, carga, acoplamentos, observáveis e autovalores).
CBTransient	classe	(pkg, m=8, k=8, shifts=TRANSIENT_SHIFTS)	Modelo Craig-Bampton do encapsulamento com m modos de interface e k modos internos por bloco.
cb_block	função	(pkg_stack, variant, Phi_f, k) -> CBBlock	Redução Craig-Bampton de um bloco sobre os modos de interface Phi_f e k modos de interface fixa.

nome	tipo	assinatura	o que faz
`reference_transient`	função	`(pkg, powers, dt, every=1) -> dict`	Euler implícito na grade fina com o mesmo passo (fatoração única ou AMG-CG para grades grandes).

Exemplos de uso reais do repositório

Condensação exata igual à referência fina — `tests/test_condense.py`

```
        def test_exact_condensation_equals_reference(case):
            pkg, T, ref = case
            c = Condensed(pkg, m=10 ** 6).prepare()
            st = c.solve()
            assert np.abs(c.field(st) - T).max() < 1e-7 * T.max()
            e = errors(c.observables(st), ref, pkg.power.sum())
            assert e["err_Tmax"] < 1e-7 and e["err_q_ubump"] < 1e-7 and abs(e["energy_balance"]) < 1e-8
            assert c.timings["distinct_blocks"] == 3                    # three variants, twelve blocks: three
factorisations
```

Estudos do benchmark rodados por nome — `examples/physics_condensation/run_benchmark.py`

```
        from pinneapple_condense import benchmark as B
        from pinneapple_condense import figures as F

        HERE = os.path.dirname(os.path.abspath(__file__))
        RES, FIG = os.path.join(HERE, "results"), os.path.join(HERE, "figures")

        FULL = {
            "mesh": ("mesh_convergence", {}),
            "accuracy": ("accuracy_vs_modes", {}),
```

Testes e status de validação

- Módulos com pelo menos um teste que os importa: 15 de 18.
- Funções de teste em arquivos que importam algum módulo do pacote: 27.

Limitações conhecidas

- O próprio pacote se declara "work in progress"; o `__init__` não reexporta nada e cada submódulo é importado direto.
- O caso de teste é um encapsulamento de chip sintético: geometria simplificada (inclusões quadradas, um C4 por bloco, tampa sem balanço) e materiais representativos, não de um produto real.
- A reutilização de blocos depende de blocos idênticos; quando todos diferem, é preciso o caminho aprendido (`learned`) ou condensar cada bloco (mais caro).
- Os estudos de `benchmark` e `falsify` são pesados (o exemplo cita cerca de uma hora em 4 núcleos e ~12 GB para o maior caso) e são pensados para rodar em processo isolado.
- `fv.solve` usa `pyamg` para problemas grandes se estiver instalado; sem ele cai para gradientes conjugados com Jacobi.
- `flow` e `meshless2d` são 2D e servem como referência e operador dos testes de falsificação, não como solver de CFD geral.
- `benchmark`, `figures` e `query_benchmark` não têm teste direto.

6. `pinneapple_query`

Objetivo

`pinneapple_query` é o motor genérico de consultas físicas (Physics Query Engine). A ideia: pedir uma grandeza física (por exemplo `T`) num lugar físico estável (por exemplo `die@0.3,0.7`), sob certas condições (por exemplo um mapa de potência), com uma precisão-alvo. A resposta traz o valor, a unidade, a incerteza, o domínio de

validade, a proveniência e o método que a produziu, e diz se o valor foi *lido* de um estado já resolvido (`read`), *previsto* para condições não resolvidas (`predicted`) ou obtido por uma nova solução (`solved`).

O problema que resolve é escolher, a cada pergunta, o caminho mais barato que ainda cumpre a precisão pedida. Um `Planner` pergunta a cada `Method` registrado por uma `Offer` (se pode responder, a que custo esperado, com que limite de erro, dentro de que validade) e escolhe a oferta válida mais barata cujo limite atende o alvo; se nenhuma atende, fica com a de menor limite de erro; um método de último recurso (`fallback`, tipicamente uma solução completa) só entra quando ninguém oferece. O `Engine` guarda um log de cada resposta (método, tipo, latência) para auditoria e aceita ganchos (`hooks`).

O pacote é pequeno e só tem a parte genérica: `core` (consulta, ofertas, planejador, respostas) e `refs` (referências físicas independentes de malha, como `ubump[12,40]`, `interface:die/tim` ou `lid:top`). Os métodos de uma física concreta ficam junto dela: para o encapsulamento de chips, `pinneapple_condense.query.package_engine` registra os métodos `cached`, `superposition`, `bound`, `condensed` e `reference`. No resto da plataforma, o nó `query.package` de `pinneapple_flow` expõe isso como um passo de fluxo, e `examples/physics_query/run_demo.py` é a demonstração medida (resultado gravado em `examples/physics_query/results/query_demo.json`). É uma linha de pesquisa ("track 2" no CHANGELOG), não uma API estável de `pp.*`.

Números do inventário

módulos	linhas	classes públicas	funções públicas	testes em arquivos que importam o pacote	módulos sem teste direto
3	252	7	2	7 (1 arquivos)	2

Submódulos

submódulo	o que faz	módulos	classes + funções	testes*
`core`	Consulta (`Query`), oferta (`Offer`), resposta (`Answer`), interface de método (`Method`), planejador (`Planner`) e motor (`Engine`).	1	7	0
`refs`	Referências físicas estáveis (`PhysicalRef`) e o parser das suas formas em texto (`parse_ref`).	1	2	0

* maior número de testes que importam um módulo do submódulo.

Módulos e API pública

`pinneapple_query`

Fachada do pacote: reexporta `Engine`, `Planner`, `Method`, `Offer`, `Answer`, `Query`, `condition_key`, `PhysicalRef` e `parse_ref`; o docstring mostra o uso com `pinneapple_condense.query.package_engine`.

`pinneapple_query/__init__.py` · 17 linhas · testes: 7

`pinneapple_query.core`

Motor de consultas: o `Planner` coleta ofertas dos métodos e escolhe a mais barata que cumpre a precisão; o `Engine` executa, mede o tempo, registra as ofertas e grava um log de auditoria. `Method` é só interface (levanta NotImplementedError); [sem teste direto] — coberto por `tests/test_query.py` via `pinneapple_condense.query`.

`pinneapple_query/core.py` · 155 linhas · **sem teste direto** · palavras-chave no código (heurística; o texto acima diz o status real): levanta `NotImplementedError` em algum caminho

nome	tipo	assinatura	o que faz
`Query`	classe	`(quantity, where, time, conditions, accuracy, rigorous)`	Uma consulta: grandeza, referência física (`where`), tempo, condições, precisão absoluta desejada e se só valem métodos com limite rigoroso; `key` é o hash das condições.

nome	tipo	assinatura	o que faz
`Offer`	classe	`(method, cost_s, bound, valid, rigorous, why)`	Oferta de um método para uma consulta: custo esperado em segundos, limite de erro esperado, se é válida, se o limite é rigoroso e o porquê.
`Answer`	classe	`(value, unit, uncertainty, kind, method, validity, provenance, seconds, offers, rigorous,…)`	Resposta: valor, unidade, incerteza, tipo (`read`, `predicted` ou `solved`), método, validade, proveniência, tempo gasto, ofertas consideradas e, se rigorosa, o intervalo.
`Method`	classe	`()`	Interface de um modo de responder: `offer` decide barato se pode responder e `answer` calcula; ambos levantam NotImplementedError na classe base.
`Planner`	classe	`(methods, fallback=None)`	Escolhe o método: a oferta válida mais barata que atende a precisão; senão a de menor limite de erro; o `fallback` só quando nenhum método oferece (sem fallback, levanta LookupError).
`Engine`	classe	`(planner, units=None, default_accuracy=None)`	Ponto de entrada `query(quantity, where, ...)`: aplica a precisão padrão por grandeza, consulta o `Planner`, mede a latência, anexa as ofertas, chama os ganchos e grava o log de auditoria.
`condition_key`	função	`(conditions) -> str`	Hash SHA-256 estável (16 caracteres) das condições de uma consulta; arrays entram pelos seus bytes.

`pinneapple_query.refs`

Referências físicas independentes de malha (componente + coordenada fracionária local, índice de bloco, faixa de blocos, interface ou face) e o parser das formas de texto. [sem teste direto] — `tests/test_query.py` testa `parse_ref` pela fachada do pacote.

`pinneapple_query/refs.py` · 80 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`PhysicalRef`	classe	`(component, local, index, span, kind)`	Referência física imutável: componente, coordenadas locais fracionárias, índice de feature repetida, faixa de blocos e tipo (`point`, `feature`, `region`, `interface` ou `face:...`); `str()` volta à forma de texto.
`parse_ref`	função	`(where) -> PhysicalRef`	Converte textos como `die@0.31,0.72`, `ubump[12,40]`, `interface:die/tim[3:9,0:4]`, `region:die[...]` ou `lid:top` em `PhysicalRef`; rejeita coordenadas locais fora de [0, 1].

Exemplos de uso reais do repositório

Referências físicas e seus tipos — `tests/test_query.py`

```python
def test_physical_references():
    r = parse_ref("die@0.25,0.5")
    assert (r.component, r.local, r.kind) == ("die", (0.25, 0.5), "point")
    assert parse_ref("ubump[3,4]").index == (3, 4) and parse_ref("ubump[3,4]").kind == "feature"
    i = parse_ref("interface:die/tim[0:2,1:3]")
    assert i.kind == "interface" and i.span == ((0, 2), (1, 3)) and i.component == "die/tim"
    assert parse_ref("lid:top").kind == "face:top"
    assert str(parse_ref("die@0.25,0.5,1")) == "die@0.25,0.5,1"
    with pytest.raises(ValueError):
        parse_ref("die@1.5,0.2")                                  # local coordinates are fractions
```

Resolver uma vez, depois ler do estado resolvido — `tests/test_query.py`

```python
def test_solved_then_read_with_physical_points(engine):
    pkg, eng, model = engine
    a = eng.query("Tj_max", "region:die", accuracy=1e-9)          # nothing solved: the reference solves
    assert a.kind == "solved" and a.method == "reference"
    b = eng.query("T", "die@0.4,0.6")
    c = eng.query("T", "die@0.4,0.6,0.5")
    assert b.kind == c.kind == "read" and b.method == "cached"
```

Testes e status de validação

- Módulos com pelo menos um teste que os importa: 1 de 3.

- Funções de teste em arquivos que importam algum módulo do pacote: 7.

Limitações conhecidas

- O pacote só traz a parte genérica: sem métodos concretos ele não responde nada. Hoje os únicos métodos estão em `pinneapple_condense.query` (encapsulamento de chips).
- `Method.offer` e `Method.answer` levantam NotImplementedError: cada física precisa implementar sua própria subclasse.
- Os módulos `core` e `refs` não têm teste que os importe diretamente; a cobertura vem de `tests/test_query.py`, que passa pelos métodos de `pinneapple_condense`.
- As formas de referência (`die@...`, `ubump[...]`, `tsv[...]`, `c4[...]`, `lid:top`) foram pensadas para encapsulamento de chips; outra geometria precisa definir seus próprios nomes de componente.
- A escolha do `Planner` confia nos custos e limites de erro que cada método declara na oferta; o motor não verifica se esses limites são verdadeiros.
- Sem precisão-alvo (nem na consulta nem em `default_accuracy`), o planejador escolhe a oferta de menor limite de erro, que pode ser a mais cara.
- É linha de pesquisa ("track 2" no CHANGELOG), sem entrada em `pp.*`.

Volume 3 — Simulação numérica

> Parte do Mapa completo do PINNeAPPle. Gerado por `docs/pt/_mapa/gerar.py` a partir do inventário do commit `e53cdcfb04`.

Solvers clássicos (FDM, FEM, FVM, espectral, SPH, LBM), pontes para solvers externos, eletromagnetismo, voo, multicorpos, veículos e rovers.

Sumário

1. pinneapple_simulation

1. `pinneapple_simulation`

Objetivo

`pinneapple_simulation` reúne os back-ends de simulação física da biblioteca e a geração de dados a partir deles. É onde ficam os solvers clássicos (diferenças finitas, elementos finitos, volumes finitos, lattice Boltzmann, métodos espectrais, partículas), as pontes para ferramentas externas (OpenFOAM, FEniCS, CalculiX, MATLAB, Modelica) e simuladores de domínio como dinâmica de partículas, multicorpos, terramecânica, veículos, eletromagnetismo e geofísica.

O problema que resolve é duplo. Primeiro, dar soluções de referência confiáveis para conferir modelos aprendidos (PINNs, operadores neurais, surrogates); muitos módulos trazem verificações contra formas fechadas ou estudos de convergência. Segundo, gerar dados de treino: `generate_pinn_dataset` monta pontos de colocação, contorno e dados a partir de um `ProblemSpec`, e módulos como `operator_datasets`, `cylinder_wake` e `thermal_channel_datasets` geram os datasets do zoo.

Esta parte cobre o `__init__` do pacote, `numerical_solvers` e `rover`. O `__init__` reexporta os solvers 3D, o registro, a dinâmica de partículas e as pontes externas, mantém os aliases antigos (`solvers`, `dynamics`, `integrations`; o antigo pacote `pinneapple_solvers` é um shim que aponta para `numerical_solvers`) e oferece `simulate`/`generate_data` por nome de cenário. `numerical_solvers` tem o `SolverRegistry`, que dá à Arena e aos pipelines de treino um catálogo único de solvers numéricos, de sinal e de partículas. `rover` é um rover lunar rocker-bogie com forças de Bekker-Wong e, em `rover.autolab`, a demo do Autonomous Physics Lab, que liga planejamento, simulação, estimação de solo e relatório de confiança.

Ligações com o resto: o pacote de topo `pinneapple` reexporta símbolos daqui e dá acesso preguiçoso a `pp.solvers`, `pp.dynamics`, `pp.cfd` (escoamento externo no OpenFOAM), `pp.terramechanics`, `pp.vehicle` e outros. Os solvers leem `ProblemSpec` de `pinneapple_physics`, e `symbolic_fd` resolve sistemas escritos em texto pelo `parse_system` de `pinneapple_physics.symbolic_pde`. Os resultados de escoamento podem virar cenas 3D do studio de visualização de `pinneapple_tools`.

Esta parte cobre três submódulos de `pinneapple_simulation`. `accelerated_foam` acelera as soluções de pressão do OpenFOAM trocando só o chute inicial do solver linear (plugin C++ `pinneWarmStart`, projeção/POD/rede aprendida em Python) e compara com o OpenFOAM puro na mesma tolerância. `electromagnetics` é uma biblioteca de eletromagnetismo em SI: referências em forma fechada, FEM 2-D em A_z, eletrostática, correntes parasitas, FDTD, ondas, Mie, máquinas elétricas (SPM e IPM, acionamento, térmica), antenas Vivaldi com solver externo, células periódicas e compact ranges por óptica física, com suítes de verificação. `geophysics` traz blocos do sistema terrestre verificados contra soluções conhecidas: águas rasas na esfera, equação de Richards e balanço de energia de duas camadas.

Esta parte cobre quatro submódulos de `pinneapple_simulation`. `external_solvers` liga o PINNeAPPle a programas de simulação de fora (OpenFOAM, CalculiX, OpenRadioss, ANSYS Fluent, FEniCS, MATLAB, Modelica/FMI, MuJoCo, Genesis, NVIDIA Newton, FluidX3D, GeoTaichi, TurboDesigner) e lê formatos abertos de CFD/FEM;

quase tudo termina num `PhysicalSample` (UPD) para treino e checagem. `flight` é um simulador de voo 6-DOF com aerodinâmica de vortex lattice, trim, piloto automático, decolagem e turbulência. `meshless` compara métodos sem malha (Kansa, RBF-FD, PINN) com FD e FEM no mesmo problema 2-D. `pipelines` encadeia FreeCAD, malha e OpenFOAM num fluxo reproduzível, com validação contra correlações publicadas, varredura e surrogate.

Esta parte cobre a física de corpos e partículas do `pinneapple_simulation`: dinâmica multicorpo própria (`multibody`), simuladores de partículas em PyTorch (`particle_dynamics`: corpos rígidos, MPM, GeoMPM, SPH, dinâmica molecular), robôs no NVIDIA Newton com checagens independentes (`robot_sim`), terramecânica de rodas planetárias (`terramechanics`) e o rover completo em terreno deformável (`vehicle`, com o VIPER). O fio comum é sempre ter uma referência independente para checar o resultado: modelo analítico, balanço de energia, dados publicados ou substituto PINN comparado com o modelo de origem.

Números do inventário

módulos	linhas	classes públicas	funções públicas	testes em arquivos que importam o pacote	módulos sem teste direto
298	66.085	364	931	1.119 (109 arquivos)	113

Pasta de testes dedicada: `tests/pinneapple_simulation` (16 funções de teste).

Submódulos

submódulo	o que faz	módulos	classes + funções	testes*
`accelerated_foam`	Acelera solves de pressão do OpenFOAM com chutes iniciais melhores (plugin C++ e ferramentas Python), mantendo o solver e o teste de parada do OpenFOAM.	9	46	20
`electromagnetics`	Eletromagnetismo em SI: formas fechadas, FEM 2-D, FDTD, ondas, máquinas elétricas, antenas, estruturas periódicas e refletores, com verificação.	56	345	88
`external_solvers`	Pontes para solvers e ferramentas externas (execução, montagem de casos, leitura de resultados) e leitores de formatos CFD/FEM, com saída em `PhysicalSample`.	66	192	151
`flight`	Dinâmica de voo 6-DOF: aeronave rígida com vortex lattice, motores, trem de pouso, trim, controle, decolagem, turbulência e cenas 3-D.	17	108	19
`geophysics`	Modelos do sistema terrestre: águas rasas na esfera, equação de Richards no solo e balanço de energia climático de duas camadas.	4	19	11
`meshless`	Métodos sem malha e com malha resolvendo os mesmos problemas 2-D, e o estudo de refinamento que os compara.	8	19	22
`multibody`	Árvore multicorpo de base flutuante com juntas de revolução e utilitários de braço de base fixa.	3	18	45
`numerical_solvers`	Solvers numéricos registráveis (FDM, FEM, FVM, LBM, espectral, SPH, X-TFC, decomposições de sinal), pontes OpenFOAM/FEniCS, casos prontos de escoamento e estruturas, e geradores de datasets.	73	295	206
`particle_dynamics`	Simuladores de partículas diferenciáveis em PyTorch: corpos rígidos, MPM, GeoMPM, SPH e dinâmica molecular.	8	21	151
`pipelines`	Pipelines de ponta a ponta que encadeiam CAD (FreeCAD), malhador e solver (OpenFOAM) num fluxo checado e reproduzível.	14	59	14
`robot_sim`	Robôs simulados no NVIDIA Newton (com backend Isaac Sim) e checados contra o modelo multicorpo próprio.	6	17	39
`rover`	Rover lunar rocker-bogie com terreno, contato Bekker-Wong, simulador quase-estático, verificações e visualização, mais a demo de laboratório autônomo em `rover.autolab`.	17	88	21
`terramechanics`	Contato roda-solo para rovers: solos, Wong-Reece, terreno deformável, bancada e substitutos PINN.	8	38	34
`vehicle`	Veículo planetário com rodas (VIPER) do spec ao CAD, simulação, dashboard e animação.	8	28	38

* maior número de testes que importam um módulo do submódulo.

Módulos e API pública

`pinneapple_simulation`

Ponto de entrada do pacote: reexporta solvers, dinâmica de partículas e pontes externas, mantém os aliases antigos (`solvers`, `dynamics`, `integrations`) e oferece `simulate`/`generate_data` por nome de cenário. [sem teste direto]

`pinneapple_simulation/__init__.py` · 163 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`simulate`	função	`(scenario, **kwargs) -> Any`	Roda um cenário pelo nome (`heat_3d`, `ns_3d`, `mpm`, `sph`, ...): instancia a classe com os kwargs e chama `run` se existir; senão devolve a instância.
`generate_data`	função	`(scenario, n_samples=10, **kwargs) -> list`	Chama `simulate` `n_samples` vezes e devolve a lista de saídas.

`pinneapple_simulation.accelerated_foam`

Pacote que acelera as soluções de pressão do OpenFOAM trocando só o chute inicial do solver linear (plugin C++ `pinneWarmStart` + ferramentas Python); reexporta `LduSystem`, `ProjectionWarmStart`, `PODWarmStart`, `galerkin` e os leitores de sistemas.

`pinneapple_simulation/accelerated_foam/__init__.py` · 41 linhas · testes: 15

`pinneapple_simulation.accelerated_foam.benchmark`

As comparações por trás de `docs/core_concepts/openfoam_acceleration.md`: OpenFOAM puro contra `pinneWarmStart` (mesmo solver interno e tolerâncias) e o benchmark offline dos sistemas extraídos. [sem teste]

`pinneapple_simulation/accelerated_foam/benchmark.py` · 387 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`openfoam_comparison`	função	`(work_dir, lib, n=128, end_time=0.5, inners=('PCG', 'GAMG'), tolerance=1e-06, rel_tol=0.0…)`	Roda a cavidade com tampa móvel (`icoFoam`) com solves de pressão puros e com chute aquecido, e devolve uma linha por execução (iterações, tempos, diferenças de campo).
`solver_entries`	função	`(fv_solution, field_p) -> dict`	Lê as entradas `chave valor;` do bloco `solvers { <field_p> { ... } }` de um texto `fvSolution`.
`set_solver_entries`	função	`(fv_solution, field_p, entries) -> str`	Substitui o corpo do bloco do solver de `field_p` em um texto `fvSolution`, sem mexer nas outras entradas.
`prepare_tutorial`	função	`(tutorial_dir, case_dir, p_solver, field_p, libs=(), setup=('blockMesh',), end_time=None,…)`	Copia um tutorial instalado do OpenFOAM, define o solver de pressão, força escrita ASCII com 17 dígitos e roda os comandos de preparo (`blockMesh` etc.).
`read_solver_dict`	função	`(tutorial_dir, field_p) -> dict`	Lê os controles do solver de `field_p` definidos pelo próprio tutorial (como strings).
`tutorial_comparison`	função	`(tutorial_dir, work_dir, lib, application, field_p, compare_fields=('U',), setup=('blockM…)`	Roda um tutorial instalado de três jeitos: solver original, o mesmo solver envolvido por `pinneWarmStart` e uma referência com tolerância muito apertada.
`offline_benchmark`	função	`(systems, guesses, solver_specs, tolerance=1e-06, rel_tol=0.0) -> list[dict]`	Resolve de novo, em ordem, os sistemas de uma execução com cada par (chute inicial, solver) no teste de parada do OpenFOAM.
`default_solver_specs`	função	`(include_amg=True, include_torch=True) -> dict`	Dicionário padrão de solvers do benchmark offline: PCG-DIC e PCG-Jacobi em NumPy, mais AMG (pyamg) e CG em PyTorch quando instalados.
`default_guesses`	função	`(n_snapshots=8) -> dict`	Dicionário padrão de chutes iniciais: o campo anterior (padrão do OpenFOAM) e a projeção nas últimas `n_snapshots` soluções.
`markdown_table`	função	`(rows, columns=None) -> str`	Formata uma lista de linhas (dicts) como tabela Markdown para a documentação.

pinneapple_simulation.accelerated_foam.build

Compila a biblioteca de solver linear `pinneWarmStart` (fonte C++ em `plugin/`) contra um OpenFOAM instalado e guarda o `.so` em cache em `~/.cache/pinneapple/accelerated_foam`.

`pinneapple_simulation/accelerated_foam/build.py` · 155 linhas · testes: 15

nome	tipo	assinatura	o que faz
`include_dirs`	função	`(openfoam_src=None, work=None) -> list[Path]`	Diretórios de cabeçalhos do OpenFOAM (de `$FOAM_SRC` ou de uma árvore de fontes); levanta `FileNotFoundError` se não houver nenhum.
`compile_flags`	função	`(label_size=None, precision=None, api=None) -> list[str]`	Flags de compilação C++ conforme tamanho de label, precisão e versão da API do OpenFOAM (lidas das variáveis `WM_*`/`FOAM_API` quando não informadas).
`build_plugin`	função	`(openfoam_src=None, out_dir=None, force=False, cxx=None) -> Path`	Compila `libpinneWarmStart.so` e devolve o caminho (usa o cache, a menos que `force`).
`find_plugin`	função	`(openfoam_src=None) -> Path | None`	Devolve `$PINNEAPPLE_WARMSTART_LIB` se definido, senão uma compilação em cache; `None` se não der para compilar.

pinneapple_simulation.accelerated_foam.case

Casos pequenos do OpenFOAM escritos do zero (cavidade com tampa, `icoFoam`), um executor de comandos e um parser de log para os benchmarks de solução de pressão.

`pinneapple_simulation/accelerated_foam/case.py` · 395 linhas · testes: 20

nome	tipo	assinatura	o que faz
`FieldSolves`	classe	`(n_solves, iterations, per_solve, initial_residuals, final_residuals)`	Resumo dos solves lineares de um campo lidos de um log (número de solves, iterações, resíduos inicial e final).
`RunResult`	classe	`(case_dir, wall_time, execution_time, solves, log)`	Resultado de uma execução: pasta do caso, tempo de parede, ExecutionTime, solves por campo e o texto do log.
`find_bashrc`	função	`(bashrc=None) -> str | None`	Localiza o script de ambiente do OpenFOAM (`bashrc` dado, `$FOAM_BASHRC` ou um local usual).
`openfoam_available`	função	`(solver='icoFoam') -> bool`	Diz se o solver indicado do OpenFOAM pode ser executado (no PATH ou via script de ambiente).
`plain_solver`	função	`(kind='PCG', tolerance=1e-06, rel_tol=0.0, **extra) -> dict`	Dicionário do solver de pressão do próprio OpenFOAM: `PCG` (pré-condicionador DIC) ou `GAMG`.
`warm_solver`	função	`(inner='PCG', tolerance=1e-06, rel_tol=0.0, guess='projection', n_snapshots=8, **extra) -…)`	Dicionário do wrapper `pinneWarmStart` em volta do mesmo solver interno e das mesmas tolerâncias de `plain_solver`.
`write_cavity`	função	`(case_dir, n=64, end_time=0.25, nu=0.01, lid=1.0, courant=0.5, p_solver=None, p_final_sol…)`	Escreve uma cavidade 2-D com tampa móvel (quadrado unitário, `n` x `n` células) para o `icoFoam`.
`run_command`	função	`(case_dir, command, bashrc=None, log_name=None, env=None) -> tuple[str, float]`	Executa um comando do OpenFOAM na pasta do caso e devolve (texto do log, segundos de parede); levanta erro em caso de falha.
`parse_log`	função	`(log) -> tuple[dict[str, FieldSolves], float]`	Extrai de um log de solver os solves lineares por campo e o ExecutionTime final.
`run_case`	função	`(case_dir, solver='icoFoam', bashrc=None, mesh=True, env=None) -> RunResult`	Roda `blockMesh` (opcional) e o solver, e devolve o log interpretado como `RunResult`.
`time_dirs`	função	`(case_dir) -> list[str]`	Lista as pastas de tempo (nomes numéricos) de um caso.
`read_field`	função	`(case_dir, name, time_name=None) -> np.ndarray`	Lê o campo interno `name` em um instante (padrão: o último) como array NumPy.
`field_difference`	função	`(case_a, case_b, name, time_name=None) -> float`	Diferença L2 relativa de um campo entre duas execuções no mesmo instante.

pinneapple_simulation.accelerated_foam.guess_server

Serve chutes iniciais calculados em Python a um solver OpenFOAM em execução (`pinneWarmStart` com `guess external`) por dois named pipes.

pinneapple_simulation/accelerated_foam/guess_server.py · 138 linhas · testes: 15

nome	tipo	assinatura	o que faz
GuessServer	classe	(pipe_dir, guessers)	Servidor em thread de fundo que responde pedidos de chute em pipe_dir/request usando um objeto chutador por campo; usado como context manager.

pinneapple_simulation.accelerated_foam.ldu

Lê os sistemas lduMatrix gravados por pinneWarmStart e os converte para matrizes esparsas SciPy/PyTorch, com o resíduo normalizado do OpenFOAM.

pinneapple_simulation/accelerated_foam/ldu.py · 197 linhas · testes: 15

nome	tipo	assinatura	o que faz
LduSystem	classe	(lower_addr, upper_addr, diag, upper, source, psi0, psi, lower, meta, _csr)	Um sistema linear A x = b no formato em que o OpenFOAM o entrega ao solver (diag, upper, lower, endereçamento), com produto, conversão para CSR e para PyTorch.
from_csr	função	(A, b) -> LduSystem	Cria um LduSystem a partir de uma matriz esparsa simétrica, para testes e casos sintéticos.
norm_factor	função	(system, x, Ax=None) -> float	Fator de normalização do resíduo usado pelo OpenFOAM.
normalised_residual	função	(system, x, nf=None) -> float	Resíduo normalizado sum|b - A x| / normFactor, o mesmo que o OpenFOAM imprime e compara com tolerance.
read_system	função	(directory) -> LduSystem	Lê um sistema gravado por pinneWarmStart (meta.txt e system.bin).
read_dump	função	(dump_dir, field_name='p') -> list[LduSystem]	Lê todos os sistemas de um campo em uma pasta de dump, na ordem dos solves.

pinneapple_simulation.accelerated_foam.learned

Correção aprendida do chute inicial: uma pequena rede em grafo (PyTorch) sobre o grafo da matriz, com salvaguarda que mantém a projeção se a rede piorar o resíduo. [experimental] O docstring avisa que o efeito esperado é modesto.

pinneapple_simulation/accelerated_foam/learned.py · 162 linhas · testes: 15

nome	tipo	assinatura	o que faz
GraphGuessNet	classe	(hidden=32, layers=4, seed=0)	Rede de passagem de mensagens sobre o grafo normalizado da matriz, que prevê a correção por célula.
LearnedWarmStart	classe	(n_snapshots=8, hidden=32, layers=4, seed=0)	Chute por projeção mais correção aprendida (com salvaguarda), com a mesma interface fit/guess/update dos outros warm starts.
projection_inputs	função	(systems, n_snapshots=8) -> list[np.ndarray]	Calcula o chute por projeção de cada sistema de uma execução a partir das próprias soluções convergidas (dados de treino).

pinneapple_simulation.accelerated_foam.solvers

Solvers de Krylov (PCG em NumPy e CG em PyTorch) para os sistemas de pressão extraídos, com o mesmo teste de parada do OpenFOAM, para comparar contagens de iterações com os logs.

pinneapple_simulation/accelerated_foam/solvers.py · 230 linhas · testes: 15

nome	tipo	assinatura	o que faz
SolveResult	classe	(x, iterations, initial_residual, final_residual, wall_time, converged, history, setup_ti…)	Resultado de um solve: solução, iterações, resíduos inicial e final, tempo, convergência, histórico e tempo de preparo.
jacobi	função	(system)	Pré-condicionador de Jacobi (divisão pela diagonal).
dic	função	(system)	Pré-condicionador DIC do OpenFOAM (Cholesky incompleto diagonal), aplicado por duas soluções triangulares esparsas.
amg	função	(system, **kwargs)	Pré-condicionador AMG por agregação suavizada do pyamg (dependência opcional), aplicado a -A.

nome	tipo	assinatura	o que faz
pcg	função	(system, x0=None, preconditioner='dic', tolerance=1e-06, rel_tol=0.0, max_iter=1000, min_…)	Gradiente conjugado pré-condicionado em NumPy com o algoritmo e o teste de parada do PCG do OpenFOAM.
torch_pcg	função	(system, x0=None, tolerance=1e-06, rel_tol=0.0, max_iter=1000, device=None, dtype='float6…)	CG com pré-condicionador de Jacobi em PyTorch (GPU quando houver), em float64 ou precisão mista.

pinneapple_simulation.accelerated_foam.warm_start

Chutes iniciais para uma sequência de sistemas de pressão: projeção de Galerkin nas últimas soluções e em uma base POD de execuções anteriores, nunca piores que o campo atual na norma de energia.

pinneapple_simulation/accelerated_foam/warm_start.py · 121 linhas · testes: 15

nome	tipo	assinatura	o que faz
ProjectionWarmStart	classe	(n_snapshots=8)	Projeção de Galerkin de cada novo sistema no espaço das últimas n_snapshots soluções e do campo atual (o mesmo algoritmo do plugin C++).
PODWarmStart	classe	(energy=0.99999, max_rank=32, n_recent=0)	Projeção de Galerkin em uma base POD de soluções guardadas (outras execuções ou passos anteriores) mais o campo atual.
galerkin	função	(system, basis, drop=1e-12) -> np.ndarray | None	Resolve (V^T A V) c = V^T b e devolve x = V c, descartando colunas dependentes; None se a base estiver vazia.

pinneapple_simulation.electromagnetics

Pacote de eletromagnetismo em SI: referências em forma fechada, materiais, solvers FEM 2-D em A_z, FDTD, ondas e máquinas elétricas; reexporta as classes e funções principais dos submódulos.

pinneapple_simulation/electromagnetics/__init__.py · 67 linhas · testes: 88

pinneapple_simulation.electromagnetics.analytic

Referências em forma fechada para verificar os solvers 2-D em A_z: fio, coaxial, cilindro e anel magnetizados, efeito pelicular e solução por subdomínios em camadas radiais.

pinneapple_simulation/electromagnetics/analytic.py · 226 linhas · testes: 22

nome	tipo	assinatura	o que faz
Layer	classe	(r_in, r_out, mu_r, Br_cos, Br_sin, J_cos, J_sin)	Uma coroa circular r_in < r < r_out com permeabilidade relativa, remanência radial e densidade de corrente de um harmônico.
RadialLayers	classe	(layers, p)	Solução exata de um harmônico p >= 1 em camadas concêntricas (potencial, campo polar e torque).
wire_field	função	(r, current, radius, R_outer)	A_z e |B| de um fio redondo com corrente uniforme dentro de um círculo com A = 0.
coax_field	função	(r, current, a, b, c)	|B| de um cabo coaxial (condutor interno +I, externo -I).
magnetized_cylinder	função	(x, y, Br, a, R, alpha=0.0)	A_z de um cilindro uniformemente magnetizado no ar com A = 0 em um raio externo; devolve (A, B interno).
magnetized_ring	função	(x, y, Br, a, b, r0, R, alpha=0.0)	A_z de um anel uniformemente magnetizado entre dois círculos com A = 0, por superposição de cilindros.
skin_effect_wire	função	(r, current, radius, sigma, omega, mu=MU0)	Densidade de corrente fasorial e impedância por metro de um fio redondo com efeito pelicular (funções de Bessel).
round_conductor_eddy_loss_low_freq	função	(B_peak, omega, sigma, a) -> float	Perda média por correntes parasitas por metro em um condutor redondo sob campo transversal uniforme, no limite de baixa frequência.

pinneapple_simulation.electromagnetics.antenna

Antenas impressas end-fire (Vivaldi antipodal) com solver FEM externo (EMerge): limites físicos, checagens, executor, surrogates, laço de projeto e visualização 3-D.

pinneapple_simulation/electromagnetics/antenna/__init__.py · 30 linhas · testes: 31

pinneapple_simulation.electromagnetics.antenna.ava

Modelo paramétrico aberto de antena Vivaldi antipodal (AVA) para o solver FEM EMerge, escrito do zero, com CLI; o EMerge é dependência opcional.

pinneapple_simulation/electromagnetics/antenna/ava.py · 386 linhas · testes: 31

nome	tipo	assinatura	o que faz
InvalidGeometry	classe	()	Exceção para parâmetros que não descrevem uma antena construível (metal sobreposto, fendas atravessando o braço etc.).
AVAParams	classe	(Lb, W, Wa, R, xs, Wg, Lo, nslot, Ls, ws, ps, dir_a, dir_b, dir_x, res, er, tand, h, wf, …)	Parâmetros de projeto da AVA em milímetros (placa, flare, fendas, diretor, substrato, malha).
Layout	classe	(params, top, bottom, director, wf, min_metal)	Polígonos condutores (topo, fundo, diretor) e dimensões derivadas de um projeto.
microstrip_width	função	(z0, er, h) -> float	Largura de uma microfita de impedância dada (Pozar eq. 3.197).
layout	função	(p) -> Layout	Gera os polígonos condutores de um projeto; levanta InvalidGeometry se não for construível.
use_superlu	função	() -> None	Força o solver direto SuperLU do EMerge.
nearfield_planes	função	(p) -> dict[str, dict[str, np.ndarray]]	Os dois planos de corte de campo próximo do projeto (plano da placa e plano do eixo).
plane_points	função	(pl) -> np.ndarray	Pontos (nv, nu, 3) de um plano de nearfield_planes.
simulate	função	(params, freqs_ghz, out_dir=None, *, superlu=True, log='WARNING') -> list[dict[str, Any]]	Monta o projeto no EMerge, resolve cada frequência e devolve uma linha por frequência.
main	função	(argv=None) -> int	Ponto de entrada da CLI que simula um projeto e grava result.json.

pinneapple_simulation.electromagnetics.antenna.bounds

Limites físicos de diretividade de antenas end-fire (Hansen-Woodyard e ordinária) e de abertura broadside, comprimento mínimo e frequência de viabilidade.

pinneapple_simulation/electromagnetics/antenna/bounds.py · 87 linhas · testes: 23

nome	tipo	assinatura	o que faz
to_dbi	função	(x)	Converte ganho linear para dBi.
from_dbi	função	(g)	Converte dBi para ganho linear.
wavelength	função	(f_hz)	Comprimento de onda no vácuo em metros.
endfire_directivity	função	(length_m, f_hz, kind='hw')	Diretividade máxima (linear) de uma antena end-fire de comprimento dado.
endfire_directivity_dbi	função	(length_m, f_hz, kind='hw')	O mesmo limite de diretividade end-fire em dBi.
aperture_gain	função	(area_m2, f_hz, efficiency=1.0)	Ganho de abertura broadside (linear).
aperture_gain_dbi	função	(area_m2, f_hz, efficiency=1.0)	Ganho de abertura broadside em dBi.
min_length	função	(gain_dbi, f_hz, kind='hw')	Menor comprimento end-fire cujo limite atinge um ganho alvo.
feasibility_frequency	função	(gain_dbi, length_m, kind='hw')	Menor frequência em que uma antena end-fire de comprimento dado pode atingir o ganho alvo.

pinneapple_simulation.electromagnetics.antenna.checks

Checagens de consistência de um resultado de antena (ganho realizado, eficiência, pico vs. boresight, limite end-fire, S11 interpolado, fechamento de potência) como Check/TrustReport de pinneapple_analysis.trust.

pinneapple_simulation/electromagnetics/antenna/checks.py · 172 linhas · testes: 23

nome	tipo	assinatura	o que faz
check_realized_gain	função	(g_dbi, g_real_dbi, s11_db, *, tol_db=0.1) -> Check	Confere o ganho realizado contra ganho e S11.
check_efficiency_ratio	função	(g_dbi, d_dbi, eff, *, tol_db=0.1) -> Check	Confere a eficiência de radiação como G/D.
check_efficiency_le_one	função	(eff, *, tol=0.01) -> Check	Checagem crítica: eficiência de radiação no máximo 1.
check_peak_ge_boresight	função	(peak_dbi, bore_dbi, *, tol_db=0.05) -> Check	Confere que o ganho de pico não é menor que o de boresight.
check_directivity_limit	função	(d_dbi, length_m, f_hz, *, margin_db=0.5, kind='hw') -> Check	Confere a diretividade abaixo do limite end-fire para o comprimento.
check_s11_resolved	função	(n_solved, *, interpolated, min_points=5) -> Check	Exige um número mínimo de frequências resolvidas quando o S11 vem de interpolação racional.
check_power_closure	função	(p_pattern, p_surface, *, rtol=0.1) -> Check	Compara a potência do padrão de campo distante com o fluxo de Poynting no campo próximo.
antenna_checks	função	(row, *, length_m=None, n_solved=None, interpolated=False, min_points=5, tol_db=0.1, marg…)	Aplica todas as checagens a uma linha da tabela e devolve um TrustReport.

pinneapple_simulation.electromagnetics.antenna.design

Laço de projeto da antena: DOE (LHS/Sobol), NSGA-II nos surrogates, aprendizado ativo verificado pelo solver, correção de viés multifidelidade e cartão de confiança por previsão.

pinneapple_simulation/electromagnetics/antenna/design.py · 326 linhas · testes: 23

nome	tipo	assinatura	o que faz
OptimizationOutcome	classe	(designs, Gmin, S11max, area_cm2, n_evals, wall_s, method, notes)	Resultado da otimização: projetos, ganho mínimo, S11 máximo, área, avaliações, tempo e método.
ALRound	classe	(round, n_train_designs, new_tags, rmse_prospective, mean_std, coverage95)	Registro de uma rodada de aprendizado ativo (tamanho do treino, novos projetos, erro prospectivo, cobertura).
ava_transform	função	(sample, *, fixed=None) -> dict[str, Any]	Converte uma amostra crua do espaço de projeto em parâmetros da AVA (liga/desliga diretor e fendas).
is_buildable	função	(params) -> bool	Diz se os parâmetros geram um layout de AVA válido.
training_box	função	(designs, keys=('Lb', 'W', 'Wa'), ranges=None) -> dict[str, tuple[float, float]]	Estreita as faixas de busca para o intervalo coberto pelos projetos de treino.
doe	função	(n, ranges=None, *, method='lhs', seed=0, transform=ava_transform, valid=is_buildable, ma…)	Gera n projetos válidos de uma amostra space-filling, contando os sorteados e inválidos.
band_metrics	função	(models, params, freqs, *, gain='Greal_bore_dBi', s11='S11_dB') -> dict[str, np.ndarray]	Ganho mínimo previsto, seu desvio e S11 máximo na banda para cada projeto.
optimize	função	(models, freqs, *, ranges=None, fixed=None, budget=600, seed=0, method='nsga2', s11_max=-…)	Busca multiobjetivo (NSGA-II) nos surrogates de ganho e S11.
active_learning	função	(simulate, table, freqs, *, target='Greal_bore_dBi', rounds=2, per_round=2, ranges=None, …)	Aprendizado ativo guiado pela variância, com verificação pelo simulador.
multi_fidelity	função	(coarse, fine, target) -> dict[str, Any]	Viés grosso-para-fino de uma grandeza por frequência, validado deixando um projeto fino de fora.
prediction_card	função	(model, params, f_ghz, train, *, target='Greal_bore_dBi', std_max_db=1.0, lodo_rmse=None,…)	Cartão de confiança (TrustReport) de uma previsão do surrogate.

pinneapple_simulation.electromagnetics.antenna.package

Lê um pacote de estudo externo (tabela de resultados + script opcional) de uma pasta local, sem copiar nada para o repositório, e reexecuta um de seus projetos.

pinneapple_simulation/electromagnetics/antenna/package.py · 83 linhas · testes: 23

nome	tipo	assinatura	o que faz
`Package`	classe	`(root, table_path, table, script, argv, flags, length_col)`	Pacote de estudo carregado: pasta, tabela, script, argumentos e flags; cria o simulador e descreve o conteúdo.
`load_package`	função	`(root) -> Package`	Carrega o pacote de uma pasta (usando `package.json` se houver).

`pinneapple_simulation.electromagnetics.antenna.plots`

Figuras matplotlib de um estudo de antena end-fire (diretividade vs. L/lambda com limites, RMSE LODO, paridade, layout). [sem teste]

`pinneapple_simulation/electromagnetics/antenna/plots.py` · 87 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`directivity_vs_length`	função	`(table, ax=None, *, column='D_peak_dBi', target_dbi=10.0, length_col='Lb', label=None)`	Plota uma coluna contra L/lambda com os limites Hansen-Woodyard e ordinário e o alvo.
`lodo_rmse_bars`	função	`(results, ax=None, *, title='leave-one-design-out RMSE')`	Barras de RMSE leave-one-design-out por modelo.
`parity_plot`	função	`(result, model, ax=None)`	Gráfico de paridade (previsto vs. simulado) de um modelo.
`layout_plot`	função	`(lay, ax=None, *, colors=…)`	Desenha os condutores de topo, fundo e diretor de um `Layout`.

`pinneapple_simulation.electromagnetics.antenna.runner`

Executa um script de simulador paramétrico externo, um processo por amostra (e por frequência), com timeout, memória de pico e fallback para SuperLU, e lê/grava as tabelas de resultados.

`pinneapple_simulation/electromagnetics/antenna/runner.py` · 309 linhas · testes: 34

nome	tipo	assinatura	o que faz
`RunRecord`	classe	`(tag, freqs, status, returncode, wall_s, max_rss_mb, rows, message, retried)`	Registro de um processo: projeto, frequências, status, código de retorno, tempo, memória e linhas.
`CLISimulator`	classe	`(argv, workdir, *, per_frequency=True, timeout=3600.0, params_flag='--params', freq_flag=…)`	Executor que chama um script de linha de comando por projeto (e por frequência) e coleta resultados.
`ResultTable`	classe	`(rows)`	Tabela de linhas (projeto, frequência) com acesso por coluna, tag, frequência e filtros.
`ava_simulator`	função	`(workdir, **kw) -> CLISimulator`	Um `CLISimulator` sobre o modelo AVA aberto (SuperLU, uma frequência por processo).
`read_table`	função	`(path) -> ResultTable`	Lê uma tabela de resultados (CSV, TSV ou separada por espaços).
`write_table`	função	`(table, path) -> str`	Grava uma `ResultTable` em arquivo.

`pinneapple_simulation.electromagnetics.antenna.scene3d`

Vistas 3-D de uma Vivaldi antipodal a partir dos campos resolvidos, sobre a cena studio de `pinneapple_tools.visualization.studio` (glTF/USD, visualizador three.js, Blender).

`pinneapple_simulation/electromagnetics/antenna/scene3d.py` · 333 linhas · testes: 8

nome	tipo	assinatura	o que faz
`FarField`	classe	`(f_ghz, theta, phi, U, P_rad, P_acc)`	Campo distante em grade (theta, phi) com potências irradiada e aceita; dá diretividade e ganho em dBi.
`antenna_surfaces`	função	`(params, *, copper_mm=COPPER_MM)`	Substrato e cobre do projeto como superfícies `Surface` do studio.
`read_farfield`	função	`(path) -> FarField`	Lê um arquivo de campo distante gravado por `ava`.
`farfield_metrics`	função	`(ff) -> dict[str, float]`	Diretividade e ganho de pico e de boresight em dBi.

nome	tipo	assinatura	o que faz
balloon	função	(ff, centre, radius, *, g_max=None, dyn_db=30.0, quantity='gain')	Malha de 'balão' de radiação colorida por ganho e diretividade.
read_nearfield	função	(path) -> dict[str, dict[str, np.ndarray]]	Lê um arquivo de campo próximo gravado por ava.
plane_surface	função	(points, name, group='cuts')	Superfície do studio a partir de uma grade estruturada de pontos.
solved_files	função	(run_dir) -> dict[float, dict[str, str]]	Encontra os arquivos de campo distante (e próximo) por frequência numa pasta de execução.
frequency_scene	função	(params, files, *, rows=(), dyn_db=30.0, title='') -> tuple[Any, dict]	Cena com antena, balão de ganho e cortes de |E| sobre as frequências resolvidas.
phase_scene	função	(params, nf_path, *, n_phase=24, component=0, title='')	Animação dos planos de corte ao longo de um período em uma frequência.
build_3d	função	(params, run_dir, out_dir, *, rows=(), name='design', render=True, size=(960, 540), sampl…)	Gera todas as saídas 3-D de um projeto a partir dos arquivos de suas execuções.

pinneapple_simulation.electromagnetics.antenna.study

Estudos completos de uma antena end-fire como chamadas de biblioteca (usadas pelo exemplo e pelo Flow Studio): limites, estudo do modelo aberto, reprodução de pacote externo e renderização 3-D.

pinneapple_simulation/electromagnetics/antenna/study.py · 228 linhas · testes: 8

nome	tipo	assinatura	o que faz
limits_report	função	(target_dbi=10.0, f_min_ghz=2.0, f_max_ghz=4.0, n=5, lengths_mm=(150.0, 206.0, 260.0)) ->…)	Comprimentos end-fire mínimos para um ganho alvo na banda e frequência de viabilidade de comprimentos dados (sem solver).
check_summary	função	(table, *, n_solved=None, interpolated=False, margin_db=0.5) -> dict[str, Any]	Contagens de resultados das checagens e de quantas vezes cada uma falhou.
validate_targets	função	(table, targets=TARGETS) -> dict[str, sg.ValidationResult]	Validação leave-one-design-out dos surrogates padrão para cada alvo presente na tabela.
study_figure	função	(table, vals, path, *, title='', target_dbi=10.0, column='D_peak_dBi') -> str	Figura de diretividade vs. L/lambda com limites e RMSE LODO por modelo.
open_study	função	(out, *, freqs=(2.0, 2.5, 3.0, 3.5, 4.0), n_designs=16, n_fine=4, coarse_res=0.25, fine_r…)	Estudo completo do modelo aberto (precisa do EMerge): DOE, execuções, checagens, surrogates, otimização, aprendizado ativo e relatórios.
package_study	função	(path, out, *, rerun=None, rerun_freqs=1, interpolated=False, timeout=3600.0, target_dbi=…)	Reproduz um pacote externo: checagens, limites, LODO, figuras e reexecução de um projeto.
render_3d	função	(designs, out, *, freqs=(2.0, 2.5, 3.0, 3.5, 4.0), res=0.25, nf_step=2.0, timeout=3600.0,…)	Resolve projetos nomeados com campo próximo e gera suas saídas 3-D.

pinneapple_simulation.electromagnetics.antenna.surrogates

Surrogates de uma grandeza de antena sobre (projeto, frequência) — média por frequência, Hansen-Woodyard + offset, regressão linear física e GP — e validação honesta (LODO, holdout); a classe base levanta `NotImplementedError` nos métodos abstratos.

pinneapple_simulation/electromagnetics/antenna/surrogates.py · 301 linhas · testes: 23 · palavras-chave no código (heurística; o texto acima diz o status real): levanta `NotImplementedError` em algum caminho

nome	tipo	assinatura	o que faz
FrequencyMean	classe	()	Linha de base: média dos alvos de treino em cada frequência.
HWOffset	classe	()	Limite de Hansen-Woodyard mais um offset ajustado em dB.
LinearPhysics	classe	(ridge=0.001)	Regressão ridge sobre as features físicas padronizadas.

nome	tipo	assinatura	o que faz
`GPModel`	classe	`(prior='hw', restarts=4, seed=0, features=None)`	Processo gaussiano sobre as features físicas, com o limite HW (ou constante) como média a priori.
`ValidationResult`	classe	`(target, scheme, rows, failed)`	Previsões fora da amostra de cada modelo, com erros, RMSE, cobertura de 95% e melhor modelo.
`physical_features`	função	`(params, f_ghz) -> np.ndarray`	Features físicas de cada (projeto, frequência): log(L/lambda), larguras relativas, fendas e diretor.
`columns`	função	`(*names)`	Cria uma função de features que usa os parâmetros de projeto nomeados diretamente.
`default_models`	função	`() -> dict[str, Callable[[], _Model]]`	Dicionário dos quatro modelos padrão (média, HW + offset, linear, GP).
`lodo`	função	`(table, target, models=None) -> ValidationResult`	Validação leave-one-design-out.
`holdout`	função	`(table, target, test_tags, models=None) -> ValidationResult`	Treina sem os projetos de teste e prevê esses projetos.

pinneapple_simulation.electromagnetics.constants

Constantes físicas do eletromagnetismo em SI, valores CODATA 2022 (NIST), com `mu0` medido após a redefinição do SI de 2019.

`pinneapple_simulation/electromagnetics/constants.py` · 29 linhas · testes: 36

pinneapple_simulation.electromagnetics.eddy

Correntes parasitas harmônicas no tempo em 2-D (A_z) com condutores sólidos e correntes líquidas impostas, meios lineares, montagem P1 e solver direto. [sem teste direto]

`pinneapple_simulation/electromagnetics/eddy.py` · 93 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`TimeHarmonicSolution`	classe	`(A, u, J_nodal, loss, geometry)`	Resultado do solve harmônico: A complexo, tensões por metro dos condutores, J nodal, perda e geometria.
`solve_time_harmonic`	função	`(mesh, nu, sigma, omega, conductors, currents, J_source=None, dirichlet=None, dirichlet_v…)`	Resolve A complexo com condutores sólidos (máscaras de elementos) e seus fasores de corrente líquida.

pinneapple_simulation.electromagnetics.electrostatics

Eletrostática 2-D em triângulos P1 (`-div(eps grad phi) = rho`) com eletrodos a potencial fixo, reaproveitando o operador do solver magnetostático; só geometria planar.

`pinneapple_simulation/electromagnetics/electrostatics.py` · 61 linhas · testes: 26

nome	tipo	assinatura	o que faz
`ElectrostaticSolution`	classe	`(phi, E, eps, energy, geometry)`	Resultado eletrostático: potencial, campo E, permissividade, energia e geometria; `D` dá o deslocamento elétrico.
`solve_electrostatic`	função	`(mesh, eps_r, electrodes=None, rho=None, dirichlet=None, dirichlet_values=None) -> Electr…)`	Calcula o potencial para permissividades por elemento, densidade de carga e eletrodos `{nome: (nós, potencial)}`.
`capacitance`	função	`(sol, V) -> float`	Capacitância por metro `C = 2 W / V^2` de um eletrodo a `V` com os outros aterrados.

pinneapple_simulation.electromagnetics.fdtd

Solver FDTD das equações de Maxwell na grade de Yee em 1-D, 2-D e 3-D (um só código), com contornos PEC/PMC/periódico/Bloch/CPML, fontes TF/SF, monitores DFT e transformada campo próximo-distante.

`pinneapple_simulation/electromagnetics/fdtd.py` · 912 linhas · testes: 10

nome	tipo	assinatura	o que faz
YeeGrid	classe	(shape, spacing, courant, dt, allow_unstable)	Grade de Yee: número de nós por eixo, espaçamentos e passo de tempo (fração do limite de Courant).
Probe	classe	(component, index, values, times)	Sonda que registra um componente de campo em um nó ao longo do tempo.
DFTMonitor	classe	(components, freqs, region, data)	Monitor que acumula a DFT de componentes de campo numa região durante a simulação.
PlaneWaveTFSF	classe	(axis, polarization, waveform, box)	Caixa campo total / campo espalhado que injeta uma onda plana com polarização e forma de onda dadas.
FDTD	classe	(grid, boundaries=None, *, npml=10, pml_order=3.0, pml_kappa_max=1.0, pml_alpha_max=None,…)	Solver leapfrog de Yee: materiais, fontes, correntes, sondas, monitores, ondas planas, avanço no tempo, energia e snapshots.
NearToFarField	classe	(sim, freqs, box)	Transformada campo próximo-distante em uma caixa fechada de planos de nós (grades 3-D).
fill_fraction	função	(grid, inside, oversample=8) -> np.ndarray	Fração de cada célula de nó que fica dentro de uma forma dada, por sobreamostragem.
courant_limit	função	(spacing, dims=None) -> float	Maior passo de tempo estável da grade de Yee nos eixos ativos.
numerical_dispersion	função	(omega, dt, spacing, direction, n=1.0) -> float	Número de onda numérico do esquema de Yee em uma direção e frequência (relação de dispersão resolvida por bisseção/Newton).
gaussian_pulse	função	(t0, tau) -> Callable[[float], float]	Forma de onda de pulso gaussiano.
modulated_gaussian	função	(f0, bandwidth, t0=None) -> Callable[[float], float]	Forma de onda de gaussiana modulada por seno (média zero, sem DC).
ricker	função	(fp, t0=None) -> Callable[[float], float]	Forma de onda de Ricker com frequência de pico dada.
sinusoid	função	(f0, ramp_periods=3.0) -> Callable[[float], float]	Seno ligado com rampa suave ao longo de alguns períodos.
poynting_flux_dft	função	(mon, axis, spacing) -> np.ndarray	Potência média através de um plano monitor calculada dos fasores DFT de E e H.

pinneapple_simulation.electromagnetics.fem

Magnetostática 2-D no potencial vetor A_z com triângulos P1: montagem, restrições de Dirichlet e (anti)periódicas e solver de Newton com busca linear no funcional de energia, para ferro não linear e ímãs.

pinneapple_simulation/electromagnetics/fem.py · 244 linhas · testes: 36

nome	tipo	assinatura	o que faz
Geometry	classe	(area, gx, gy, rows, cols, n)	Áreas e gradientes baricêntricos por elemento de uma malha P1, com índices de montagem esparsa e operadores de rigidez, carga e rotacional.
MagnetostaticSolution	classe	(A, B, nu, energy_functional, iterations, residual, geometry, tangent, P, info)	Resultado magnetostático: A, B, relutividade, funcional de energia, iterações, resíduo e dados da tangente; coenergy dá a coenergia.
flux_density	função	(mesh, A) -> np.ndarray	Densidade de fluxo B por elemento a partir do A_z nodal.
constraint_map	função	(n, dirichlet=None, dirichlet_values=None, periodic=None)	Mapa esparso A = P a + A_D que aplica nós de Dirichlet e nós escravos periódicos/antiperiódicos.
solve_magnetostatic	função	(mesh, nu, J=None, Br=None, nonlinear=(), dirichlet=None, dirichlet_values=None, periodic…)	Resolve o problema magnetostático em A_z com correntes, remanência, materiais não lineares e restrições.

pinneapple_simulation.electromagnetics.hysteresis

Magnetização não linear: curva de saturação de Fröhlich-Kennelly (compatível com o solver de Newton) e modelo de histerese de Jiles-Atherton.

pinneapple_simulation/electromagnetics/hysteresis.py · 198 linhas · testes: 26

nome	tipo	assinatura	o que faz
`FrohlichKennelly`	classe	`(alpha, beta)`	Curva B-H de Fröhlich-Kennelly em forma fechada, com a mesma interface de `BHCurve` (h, nu, dnu, energia).
`HysteresisLoop`	classe	`(H, M)`	Laço de histerese amostrado (H, M) com B, perda por ciclo, coercividade e remanência.
`JilesAtherton`	classe	`(Ms, a, k, c, alpha)`	Parâmetros do modelo de Jiles-Atherton, com curva anisterética, dM/dH, traçado de histórico de H e laço maior.
`langevin`	função	`(x)`	Função de Langevin `coth x - 1/x`, com série perto de zero.

pinneapple_simulation.electromagnetics.machine

Máquina de fluxo radial com ímãs de superfície (ou rotor de relutância) em FEM 2-D A_z sobre a malha polar com banda móvel: torque, cogging, perdas e sensibilidades para otimização topológica.

`pinneapple_simulation/electromagnetics/machine.py` · 473 linhas · testes: 47

nome	tipo	assinatura	o que faz
`ScaledCurve`	classe	`(curve, w)`	Relutividade interpolada por densidade entre ar e ferro com pesos por elemento (para otimização topológica do ferro).
`MachineSolution`	classe	`(theta, currents, mesh, sol, J)`	Solução de uma posição do rotor: ângulo, correntes, malha, solução magnetostática e J.
`SPMMachine`	classe	`(pole_pairs, slots, r_rotor, magnet_thickness, magnet_arc, magnetization, Br, mu_r_magnet…)`	Máquina com ímãs de superfície ou rotor de relutância dentro de um estator ranhurado: malha, enrolamento, solve, torque e gradientes de projeto.
`belt_winding`	função	`(slots, pole_pairs, phases=3) -> list[tuple[int, int]]`	Enrolamento de camada única em faixas de fase de 60 graus: (fase, sinal) por ranhura.

pinneapple_simulation.electromagnetics.machines

Máquinas elétricas: seções transversais paramétricas, malhas conformes, FEA 2-D não linear e pós-processamento; reexporta geometria, malhador, `IPMMachine` e dados de aço/ímã.

`pinneapple_simulation/electromagnetics/machines/__init__.py` · 27 linhas · testes: 48

pinneapple_simulation.electromagnetics.machines.design

Projeto do rotor da máquina IPM em V com o núcleo de `pinneapple_design.design_optimizer`: FEA como avaliador, GP com aprendizado ativo, NSGA-II, reverificação por FEA e comparação com um GA só com FEA (precisa de torch).

`pinneapple_simulation/electromagnetics/machines/design.py` · 175 linhas · testes: 2

nome	tipo	assinatura	o que faz
`rotor_evaluator`	função	`(**kwargs)`	Cria um `IPMRotorEvaluator` (classe construída no primeiro uso, pois o núcleo de projeto importa torch).
`rotor_design_problem`	função	`(evaluator=None, ripple_max=0.3)`	Problema de projeto: maximizar torque, minimizar massa de ímã, com ripple abaixo de um limite.
`hybrid_vs_fea_only`	função	`(problem, *, n_initial=24, rounds=3, n_verify=8, ga_budget=400, surrogate_budget=1500, se…)`	Roda o estudo híbrido com surrogate e o estudo só com FEA no mesmo problema e os compara com qualidade igual.

pinneapple_simulation.electromagnetics.machines.drive

Modelo de acionamento de máquina síncrona: regime dq, MTPA e enfraquecimento de campo com limites do inversor, mapa de eficiência e energia em ciclo de condução (WLTC).

`pinneapple_simulation/electromagnetics/machines/drive.py` · 484 linhas · testes: 11

nome	tipo	assinatura	o que faz
`LinearDQ`	classe	`(pole_pairs, psi_m, L_d, L_q, core_loss_coeffs)`	Máquina com indutâncias constantes, com MTPA e torque máximo em forma fechada para verificar o otimizador.
`FluxMap`	classe	`(pole_pairs, i_d, i_q, psi_d, psi_q, T, loss_i_d, loss_i_q, S_iron, magnet_per_f2, loss_c…)`	Máquina tabelada da FEA: fluxos dq, torque e coeficientes de perda numa grade (i_d, i_q), com leitura/gravação em npz.
`Drive`	classe	`(machine, R, I_max, V_max, n_d, L_end)`	Máquina alimentada por inversor com limites de corrente e tensão, resistência e indutância de cabeça de bobina; calcula pontos de operação e mapa de eficiência.
`Vehicle`	classe	`(mass, c_rr, cd_a, wheel_radius, gear_ratio, gear_efficiency, rotating_mass_factor, air_d…)`	Veículo longitudinal (massa, rolamento, arrasto, roda, relação de marcha) que converte velocidade em demanda do motor.
`v_max_svpwm`	função	`(v_dc) -> float`	Maior amplitude de tensão de fase do SVPWM na faixa linear.
`copper_resistance`	função	`(series_turns, conductor_area, stack_length, end_length, temperature=20.0, rho20=1.724e-0…)`	Resistência CC de fase a partir de espiras, área do condutor, comprimentos e temperatura.
`prius_phase_resistance`	função	`(temperature=100.0) -> tuple[float, dict]`	Resistência de fase do motor do Prius 2004 a partir dos dados de enrolamento do pyleecan (comprimento da cabeça de bobina estimado).
`wltc_class3b`	função	`() -> tuple[np.ndarray, np.ndarray]`	Ciclo WLTC classe 3b: tempo e velocidade do veículo.
`cycle_energy`	função	`(drive, vehicle, t, v, envelope_speeds=25) -> dict`	Energia do lado da bateria ao longo de um perfil de velocidade (motorização, regeneração e freio de atrito).

`pinneapple_simulation.electromagnetics.machines.endwinding`

Cabeças de bobina fora da seção 2-D: caminhos 3-D, resistência e indutância de dispersão (fórmula de Neumann), no ar e com espelho de ferro.

`pinneapple_simulation/electromagnetics/machines/endwinding.py` · 77 linhas · testes: 5

nome	tipo	assinatura	o que faz
`coil_end_path`	função	`(r_c, phi_a, phi_b, overhang, n_arc=48) -> np.ndarray`	Polilinha 3-D de uma cabeça de bobina entre dois ângulos.
`end_winding_paths`	função	`(layout, slots, coil_pitch, r_c, overhang) -> list[tuple[int, np.ndarray]]`	Caminhos (fase, polilinha) de todas as cabeças de bobina de um enrolamento de camada única.
`end_winding_inductance`	função	`(geometry, layout, turns_per_coil, coil_pitch, overhang, pieces_per_metre=300.0) -> dict`	Indutâncias própria e mútua de fase das cabeças de bobina e a dispersão dq, no ar e com espelho.

`pinneapple_simulation.electromagnetics.machines.export`

Exporta malhas e campos de máquinas para VTK (`.vtu`, ParaView).

`pinneapple_simulation/electromagnetics/machines/export.py` · 58 linhas · testes: 6

nome	tipo	assinatura	o que faz
`write_vtu`	função	`(path, points, cells, cell_data=None, point_data=None) -> Path`	Grava uma grade não estruturada VTK de triângulos ou prismas com dados de célula e de ponto.
`solution_to_vtu`	função	`(path, ms) -> Path`	Grava uma solução 2-D de máquina (regiões, B, |B|, A_z) em `.vtu`.

`pinneapple_simulation.electromagnetics.machines.geometry`

Seções transversais paramétricas de máquinas de fluxo radial: rotor com ímãs internos em V (parametrização `HoleM50` do pyleecan), ranhuras de fundo redondo e o motor do Prius 2004.

`pinneapple_simulation/electromagnetics/machines/geometry.py` · 320 linhas · testes: 18

nome	tipo	assinatura	o que faz
`Polygon`	classe	`(name, xy, magnetization)`	Polígono fechado nomeado (com magnetização opcional).

nome	tipo	assinatura	o que faz
SectorGeometry	classe	(r_in, r_out, span, fill, polygons, arc_nodes_in, arc_nodes_out)	Setor anular com polígonos sobre uma região de fundo, base do malhador.
VRotor	classe	(poles, r_out, r_in, H0, H1, H2, H3, W0, W1, W2, W3, W4, magnet_segments)	Rotor IPM com um V de dois ímãs por polo.
RoundBottomSlots	classe	(slots, r_in, r_out, W0, H0, H1, W1, W2, H2, R1)	Lâmina do estator com ranhuras abertas de fundo redondo.
MachineGeometry	classe	(rotor, stator, length, stacking_factor)	Rotor + estator + entreferro, com comprimento e fator de empilhamento.
polygon_area	função	(xy) -> float	Área com sinal de um polígono fechado.
arc	função	(center, radius, a0, a1, h, even=False) -> np.ndarray	Pontos de um arco circular com espaçamento máximo dado.
prius_2004	função	() -> MachineGeometry	Geometria do motor de tração do Toyota Prius 2004 com as dimensões do arquivo do pyleecan.
geometry_errors	função	(sector, tol=1e-09) -> list[str]	Motivos pelos quais um setor não é uma seção válida (lista vazia se válido).

pinneapple_simulation.electromagnetics.machines.getdp

Verificação cruzada independente da FEA da máquina com gmsh e GetDP, chamados como executáveis externos (2-D e meia pilha 3-D para dispersão de extremidade); só funciona com `gmsh` e `getdp` no PATH.

pinneapple_simulation/electromagnetics/machines/getdp.py · 469 linhas · testes: 3

nome	tipo	assinatura	o que faz
GmshGetDPError	classe	()	Erro levantado quando o gmsh ou o GetDP falham.
available	função	() -> bool	Diz se `gmsh` e `getdp` estão no PATH.
write_geo	função	(path, geometry, theta=0.0, n_gap=960, h_max=None, grading=0.2, h=0.001) -> dict	Escreve a geometria gmsh da máquina inteira num ângulo do rotor.
write_pro	função	(path, geometry, info, steel, Br, mu_r, layout, turns_per_slot, currents, parallel_paths=…)	Escreve o problema GetDP (A_z magnetostático) para as regiões da geometria.
solve	função	(geometry, theta, currents, steel, Br, mu_r, layout, turns_per_slot, n_gap=960, workdir=N…)	Malha com gmsh e resolve com GetDP; devolve torque, fluxos concatenados, elementos e tempos.
write_geo_3d	função	(path, geometry, n_gap, layers_stack, end_air, layers_air) -> dict	Extruda a seção em prismas sobre meia pilha, com extensão de ar além da face.
write_pro_3d	função	(path, info, steel, Br, mu_r, stacking_factor, pole_half_angle, r_gap, with_end_air) -> N…)	Escreve o problema GetDP 3-D em potencial escalar magnético com ferro não linear por Picard.
end_leakage_3d	função	(geometry, steel, Br, mu_r, n_gap=240, layers_stack=6, end_air=0.04, layers_air=6, workdi…)	Razão de fluxo no entreferro com e sem a região de ar além da face (efeito de extremidade 3-D).

pinneapple_simulation.electromagnetics.machines.interop

Interoperabilidade dos modelos de máquina: pyleecan, FEMM (script Lua), MATLAB, DXF, ElmerFEM e os contratos de dados do PINNeAPPle; FEMM e Elmer não são executados aqui.

pinneapple_simulation/electromagnetics/machines/interop.py · 357 linhas · testes: 8

nome	tipo	assinatura	o que faz
from_pyleecan	função	(machine) -> tuple[MachineGeometry, dict]	Lê um arquivo `MachineIPMSM` do pyleecan em `MachineGeometry` e dados de enrolamento.
to_pyleecan	função	(geometry) -> dict	Escreve os parâmetros nos nomes de campo do pyleecan.
cross_section_polygons	função	(geometry, h=0.001) -> list[tuple[str, np.ndarray, float | None]]	Todos os polígonos da seção completa com região e ângulo de magnetização.
femm_lua	função	(geometry, steel, magnet, layout, turns_per_slot, currents=(0.0, 0.0, 0.0), magnet_temper…)	Gera um script Lua do FEMM 4.2 para a máquina completa.
read_femm_lua_polygons	função	(script) -> list[np.ndarray]	Relê os polígonos traçados por um script de `femm_lua` (usado para checá-lo).

nome	tipo	assinatura	o que faz
to_mat	função	(path, **arrays) -> Path	Grava arrays nomeados num arquivo .mat do MATLAB.
from_mat	função	(path) -> dict	Lê um arquivo .mat do MATLAB como dict.
to_dxf	função	(path, geometry, h=0.001, unit_scale=1000.0) -> Path	Grava a seção como polilinhas fechadas em DXF R12, uma camada por tipo de região.
read_dxf_polylines	função	(path, unit_scale=1000.0) -> dict[str, list[np.ndarray]]	Lê as polilinhas fechadas de um DXF R12 por camada.
rotor_parameter_space	função	(name='v-ipm-rotor')	ParameterSpace das variáveis de projeto do rotor.
fluxmap_dataset	função	(fm, name='ipm-flux-map')	Converte um mapa de fluxo em tabela pandas com seu DatasetSpec.
elmer_case	função	(directory, geometry, steel, magnet, layout, turns_per_slot, currents=(0.0, 0.0, 0.0), th…)	Escreve um caso ElmerFEM (geometria gmsh e case.sif) da máquina; não é rodado aqui.

pinneapple_simulation.electromagnetics.machines.mesher

Malhas de Delaunay conformes de setores da máquina e sua montagem numa malha giratória com banda móvel no entreferro. [sem teste direto]

pinneapple_simulation/electromagnetics/machines/mesher.py · 573 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
SectorMesh	classe	(points, tris, region, polygon, arc_in, arc_out, side0, side1)	Malha de um setor: pontos, triângulos, regiões e nós dos arcos e lados; min_angle dá a qualidade.
MachineMesh	classe	(geometry, n_gap=960, h_max=None, grading=0.2, smooth=3, mirror=True)	Malha da máquina completa, copiando setores de polo e de ranhura e unindo-os pela banda móvel.
mesh_sector	função	(geom, size, n_in, n_out, smooth=3, mirror=False, max_rounds=60) -> SectorMesh	Malha um setor com tamanho de elemento função do raio, respeitando todas as arestas dos polígonos.
replicate_sector	função	(sm, copies, span) -> Mesh2D	Monta o círculo completo a partir de cópias giradas de uma malha de setor.

pinneapple_simulation.electromagnetics.machines.pmsm

Máquina síncrona de ímãs internos (IPM): FEA 2-D A_z não linear e pós-processamento (torque, cogging, fluxo, transformada de Park, L_d/L_q, MTPA, perdas). [sem teste direto]

pinneapple_simulation/electromagnetics/machines/pmsm.py · 361 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
Winding	classe	(turns_per_slot, parallel_paths, layout)	Enrolamento de camada única: espiras por ranhura, caminhos paralelos e layout (fase, sinal).
IPMMachine	classe	(geometry, steel, magnet, magnet_temperature, winding, n_gap, h_max, grading, tol)	Modelo FEA da máquina IPM (geometria, aço, ímã, temperatura, enrolamento, malha) com solve, torque, fluxos e grandezas dq.

pinneapple_simulation.electromagnetics.machines.robustness

Inclinação em degraus (multi-fatia 2-D), tolerâncias de fabricação e vista 3-D do campo de uma máquina IPM.

pinneapple_simulation/electromagnetics/machines/robustness.py · 132 linhas · testes: 6

nome	tipo	assinatura	o que faz
step_offsets	função	(steps, step_angle) -> np.ndarray	Ângulos dos segmentos do rotor centrados em zero.
step_skew_factor	função	(h, steps, step_angle, period)	Fator do skew em degraus sobre um harmônico.
skewed_no_load	função	(machine, thetas, steps, step_angle) -> dict	Torque de cogging e fluxos concatenados do rotor com skew.

nome	tipo	assinatura	o que faz
skewed_torque	função	(machine, i_d, i_q, thetas, steps, step_angle) -> np.ndarray	Torque do rotor com skew com correntes fixas no referencial dq.
tolerance_model	função	(base, I_peak=250.0, gamma_deg=50.0, n_gap=288, n_positions=6, variables=TOLERANCE_VARIAB…)	Função que leva amostras de tolerâncias a torque médio e ripple via FEA (NaN para geometrias inválidas).
skewed_field_3d	função	(machine, theta, currents, steps, step_angle, layers_per_step=1) -> dict	Malha de prismas da pilha com o campo 2-D de cada segmento.

pinneapple_simulation.electromagnetics.machines.steel

Dados de catálogo de aço elétrico (M400-50A) e ímã (NdFeB do pyleecan) com fontes, curva de pilha e ajuste de perdas no ferro. [sem teste direto]

pinneapple_simulation/electromagnetics/machines/steel.py · 191 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
SteelGrade	classe	(name, H, B, loss_f, loss_B, loss_W_per_kg, thickness, resistivity, density, source)	Grau de aço: curva B-H, tabela de perdas, espessura, resistividade, densidade e fonte.
MagnetGrade	classe	(name, Br20, mu_r, alpha_Br, resistivity, density, source)	Grau de ímã: Br a 20 °C, mu_r, coeficiente de temperatura, resistividade e densidade; dá Br(T) e Hc.
LossFit	classe	(model, rms_rel_error, max_rel_error, fitted)	Resultado do ajuste de perdas: modelo, erros relativos e parâmetros ajustados.
TableIronLoss	classe	(grade)	Perda específica interpolada da tabela de catálogo (mesma interface de IronLossModel).
stacked	função	(curve, kf, n=200) -> BHCurve	Curva B-H de uma pilha de lâminas com fator de empilhamento.
fit_iron_loss	função	(grade, classical_from_resistivity=True, B_max=1.8, f_max=2500.0) -> LossFit	Ajuste por mínimos quadrados do modelo de perdas à tabela do grau.

pinneapple_simulation.electromagnetics.machines.thermal

Rede térmica concentrada de uma máquina de fluxo radial e checagem de desmagnetização dos ímãs; os dados térmicos padrão são valores típicos declarados como hipóteses.

pinneapple_simulation/electromagnetics/machines/thermal.py · 283 linhas · testes: 8

nome	tipo	assinatura	o que faz
ThermalNetwork	classe	(names, C, links, boundaries)	Rede genérica de nós isotérmicos com capacidades, condutâncias e contornos; regime permanente e transiente.
MachineThermalData	classe	(k_lam, k_winding, k_cu, copper_fill, h_gap, h_end, jacket_GA, coolant_T, k_magnet_contac…)	Dados de material e contorno da rede da máquina (valores típicos de livro, declarados como hipóteses).
intrinsic_coercivity	função	(Hcj20, temperature, beta=-0.0055) -> float	Coercividade intrínseca do ímã em função da temperatura.
cylinder_conductance	função	(k, r1, r2, length, fraction=1.0) -> float	Condutância de condução radial através de uma casca cilíndrica.
machine_network	função	(geometry, data=None, end_length=0.16) -> ThermalNetwork	Rede de sete nós de uma máquina com camisa de água.
reverse_field	função	(B, angle, Br, mu_r) -> np.ndarray	Campo oposto à magnetização em elementos de ímã lineares.
demagnetisation_margin	função	(machine, ms, Hcj, margin_ratio=0.9) -> dict	Pior campo reverso nos ímãs de uma solução e margem até Hcj.
demagnetisation_refinement	função	(geometry, magnet_temperature, currents_d, Hcj, gradings=(0.2, 0.1, 0.05, 0.025), n_gap=9…)	Estudo de malha da margem de desmagnetização com refino crescente.
magnet_temperature_sweep	função	(geometry, temperatures, I_peak=250.0, n_gap=960, n_positions=12, n_flux=24, magnet=None)…)	Desempenho da máquina em função da temperatura do ímã.
iron_split	função	(m, I_peak=250.0, gamma_deg=50.3, rpm=1500.0, steps=24) -> dict	Frações da perda no ferro na coroa, nos dentes e no rotor num ponto de operação.

pinneapple_simulation.electromagnetics.machines.verification

Verificação da FEA de máquinas: formas fechadas, consistência interna de torque/fluxo/indutância e comparação com a curva torque-ângulo publicada do Prius 2004 (Yang et al. 2013).

pinneapple_simulation/electromagnetics/machines/verification.py · 365 linhas · testes: 30

nome	tipo	assinatura	o que faz
case	função	(name)	Decorador que registra um caso de verificação de máquinas.
run	função	(names=None, quick=False) -> list[CaseResult]	Executa os casos de verificação de máquinas e devolve CaseResult.
prius_torque_curve	função	(n_gap=960, I_peak=250.0, phi0_deg=YANG_2013_PHI0_DEG, n_positions=12) -> dict	Torque médio (tensor de Maxwell e dq) do modelo do Prius em função do ângulo de corrente.
yang_comparison	função	(curve) -> dict	Erros ponto a ponto contra Yang et al. 2013 e o deslocamento de ângulo que minimiza o erro RMS.

pinneapple_simulation.electromagnetics.materials

Materiais magnéticos para magnetostática 2-D: meios lineares, ímãs permanentes e curvas B-H não lineares (PCHIP e Brauer), sem histerese.

pinneapple_simulation/electromagnetics/materials.py · 147 linhas · testes: 26

nome	tipo	assinatura	o que faz
LinearMaterial	classe	(mu_r, sigma)	Meio linear isotrópico com permeabilidade relativa e condutividade.
Magnet	classe	(Br, mu_r, direction, sigma)	Ímã permanente linear B = mu0 mu_r H + Br.
BHCurve	classe	(B, H)	Curva B-H de valor único a partir de pontos, cúbica monótona em H(B), com inclinação do vácuo acima do último ponto.
BrauerCurve	classe	(k1, k2, k3)	Curva de Brauer nu(s) = k1 exp(k2 s) + k3 com energia e derivada em forma fechada.
illustrative_steel	função	(mu_r_init=4000.0, Js=2.0, n=60) -> BHCurve	Curva de aço saturante suave gerada por fórmula (ilustrativa, não é dado de catálogo).

pinneapple_simulation.electromagnetics.media

Meios elétricos e magnéticos para problemas de ondas: lineares, anisotrópicos e dispersivos (Drude, Lorentz, Debye), na convenção exp(+j omega t).

pinneapple_simulation/electromagnetics/media.py · 212 linhas · testes: 36

nome	tipo	assinatura	o que faz
Drude	classe	(omega_p, gamma)	Polo de Drude (elétrons livres: metais e plasmas).
Lorentz	classe	(delta_eps, omega0, delta)	Polo de Lorentz (ressonância ligada).
Debye	classe	(delta_eps, tau)	Polo de Debye (relaxação).
Medium	classe	(eps_inf, mu_r, sigma, sigma_m, poles)	Meio linear isotrópico com eps_inf, mu_r, condutividades elétrica e magnética e polos dispersivos; dá eps_r, índice, k e impedância.
Anisotropic	classe	(eps_r, mu_r, sigma)	Meio linear anisotrópico com tensores eps_r e mu_r 3x3, rotação, reciprocidade e índices.
drude_metal	função	(omega_p, gamma=0.0, eps_inf=1.0) -> Medium	Atalho que cria um Medium de metal de Drude.
is_passive	função	(medium, omegas, tol=1e-12) -> bool	Testa passividade (Im(eps_r) <= 0) nas frequências dadas.
rotation	função	(axis, angle) -> np.ndarray	Matriz de rotação em torno de um eixo (fórmula de Rodrigues).

pinneapple_simulation.electromagnetics.mesh

Malhas de triângulos para magnetostática 2-D: malhador polar por anéis com banda móvel para rotação, malha retangular estruturada e importador do gmsh (opcional).

pinneapple_simulation/electromagnetics/mesh.py · 301 linhas · testes: 26

nome	tipo	assinatura	o que faz
Mesh2D	classe	(points, tris, region, region_names, layer, boundary, rings)	Malha P1 de triângulos com um nome de região por elemento, máscaras, centroides e áreas.
Ring	classe	(r, n, offset, rotating)	Círculo de n nós igualmente espaçados num raio (com deslocamento angular e flag de rotação).
zipper	função	(ia, ang_a, ib, ang_b, closed=True) -> np.ndarray	Triangula a faixa entre dois anéis de nós quaisquer.
polar_mesh	função	(rings, classify, theta=0.0, span=TWO_PI, region_names=None) -> Mesh2D	Malha o disco (ou setor) entre anéis concêntricos, classificando regiões.
rect_mesh	função	(x, y, classify=None) -> Mesh2D	Malha triangular estruturada de um retângulo com diagonais alternadas.
mesh_from_gmsh	função	(dim=2, model=None) -> Mesh2D	Converte o modelo gmsh atual (já malhado) em Mesh2D.
boundary_nodes	função	(tris) -> np.ndarray	Nós que estão em arestas pertencentes a um só triângulo.

pinneapple_simulation.electromagnetics.mie

Espalhamento de Mie de uma onda plana por uma esfera homogênea (e por uma esfera condutora perfeita), série de Bohren & Huffman.

pinneapple_simulation/electromagnetics/mie.py · 127 linhas · testes: 26

nome	tipo	assinatura	o que faz
MieResult	classe	(x, m, an, bn, qext, qsca, qback, g)	Resultado de Mie: parâmetros, coeficientes a_n/b_n e eficiências de extinção, espalhamento, retroespalhamento e g.
mie_coefficients	função	(m_bh, x, n_max=None)	Coeficientes a_n, b_n na convenção de Bohren & Huffman.
mie	função	(m, x, n_max=None) -> MieResult	Eficiências de uma esfera homogênea com índice na convenção do pacote.
mie_pec	função	(x, n_max=None) -> MieResult	Eficiências de uma esfera condutora perfeita.
rayleigh_qsca	função	(m, x) -> float	Limite de esfera pequena (Rayleigh) da eficiência de espalhamento.

pinneapple_simulation.electromagnetics.periodic

Estruturas periódicas (arranjos infinitos, FSS) em execuções de célula unitária Floquet e as campanhas construídas sobre elas.

pinneapple_simulation/electromagnetics/periodic/__init__.py · 15 linhas · testes: 5

pinneapple_simulation.electromagnetics.periodic.benchmark

Campanhas de células unitárias e benchmark com orçamento igual de um surrogate de operador (FNO) contra otimização bayesiana no solver.

pinneapple_simulation/electromagnetics/periodic/benchmark.py · 212 linhas · testes: 5

nome	tipo	assinatura	o que faz
cell_simulator	função	(workdir, scans, *, phi=0.0, timeout=3600.0, log=None, resume=False) -> CLISimulator	CLISimulator sobre tcda com todas as frequências e ângulos de varredura num processo por célula.
campaign	função	(n, freqs, scans, workdir, *, ranges=None, fixed=None, method='lhs', seed=0, tag='c', log…)	Gera e resolve n células de uma amostragem do espaço de parâmetros.

nome	tipo	assinatura	o que faz
to_arrays	função	(table, names, freqs, scans, column='S11_dB') -> tuple[np.ndarray, np.ndarray, list[str]]	Converte a tabela em arrays X (parâmetros) e Y (frequência x varredura).
worst_case	função	(Y) -> np.ndarray	Pior reflexão ativa na banda e na varredura por célula.
fit_fno	função	(X, Y, *, wall_s=60.0, seed=0, val_frac=0.15, width=24, threads=1)	Treina um FNO (de pinneapple_analysis.surrogate) dos parâmetros ao mapa completo por tempo limitado.
heldout	função	(predict, X, Y) -> dict[str, float]	Erros do modelo em células nunca vistas.
equal_budget	função	(solve, ranges, budget, *, n_verify=4, seed=0, fno_wall_s=60.0, n_candidates=4000, log=pr…)	Compara as duas estratégias com o mesmo número de chamadas ao solver.
solver_from	função	(sim, names, freqs, scans, fixed=None)	Cria o callable solve de equal_budget a partir de um cell_simulator.

pinneapple_simulation.electromagnetics.periodic.tcda

Célula unitária genérica de arranjo de dipolos fortemente acoplados (TCDA) para execuções Floquet no EMerge, escrita do zero, com CLI.

pinneapple_simulation/electromagnetics/periodic/tcda.py · 165 linhas · testes: 5

nome	tipo	assinatura	o que faz
InvalidCell	classe	()	Exceção para parâmetros que não descrevem uma célula construível.
TCDACell	classe	(a, h, w, g, s, t_s, d_s, er_s, z_port, res, air)	Parâmetros da célula TCDA (lado, altura, dipolo, gap, superstrato, porta, malha) com checagem.
simulate	função	(cell, freqs_ghz, scans_deg=((0.0, 0.0),), *, log='WARNING') -> list[dict]	Reflexão ativa da célula em cada frequência e ângulo de varredura.
main	função	(argv=None) -> int	Ponto de entrada da CLI que simula uma célula e grava o resultado.

pinneapple_simulation.electromagnetics.postprocess

Pós-processamento de soluções A_z: torque por tensor de Maxwell (Arkkio) e por coenergia, fluxo concatenado, FEM induzida, perdas no ferro e perdas por correntes parasitas nos ímãs.

pinneapple_simulation/electromagnetics/postprocess.py · 285 linhas · testes: 22

nome	tipo	assinatura	o que faz
CoilSide	classe	(region, phase, turns, sign)	Lado de bobina: região, fase, espiras e orientação.
IronLossModel	classe	(k_h, alpha, beta, k_c, k_e, density)	Modelo de perda no ferro por kg (histerese, clássica e excesso) com densidade.
torque_maxwell_stress	função	(mesh, B, r1, r2, length=1.0, elements=None) -> float	Torque de Arkkio sobre tudo dentro de uma coroa de ar.
torque_maxwell_stress_gradient	função	(mesh, geometry, r1, r2, elements, length=1.0) -> np.ndarray	O torque de Arkkio como forma quadrática do potencial nodal, com valor e gradiente (para adjuntos).
torque_coenergy	função	(coenergy_at, theta, delta, length=1.0, stencil='central') -> float	Torque por trabalho virtual (derivada da coenergia em relação ao ângulo).
region_integral_A	função	(geometry, tris, A, mask) -> float	Integral de A sobre os elementos de uma máscara.
flux_linkage	função	(mesh, geometry, A, coil_sides, n_phases, length=1.0, parallel_paths=1) -> np.ndarray	Fluxo concatenado por fase a partir dos lados de bobina.
back_emf	função	(psi, theta, omega_mech) -> np.ndarray	FEM induzida por derivada espectral do fluxo concatenado em um período.
classical_eddy_coefficient	função	(sigma, thickness, density) -> float	Coeficiente clássico de perdas parasitas de lâminas finas.
element_flux_density_history	função	(B_steps, thetas, rotor_mask) -> np.ndarray	Empilha B por passo, levando os elementos do rotor para o referencial do rotor.

nome	tipo	assinatura	o que faz
`flux_density_ellipses`	função	`(B_hist, mask, n_harmonics=None)`	Semieixos da elipse descrita por cada harmônico de B por elemento.
`iron_loss`	função	`(B_hist, f_fundamental, model, area, mask, length=1.0, n_harmonics=None)`	Perda no ferro (total e densidade por elemento) a partir do histórico de B em um período.
`iron_loss_components`	função	`(B_hist, area, mask, alpha, beta, density, length=1.0, n_harmonics=None) -> np.ndarray`	Somas independentes da frequência que dão a perda no ferro em qualquer velocidade a partir de um período de solução.
`magnet_eddy_loss_resistance_limited`	função	`(mesh, geometry, A_hist, period, sigma, pieces, length=1.0)`	Perda média por correntes parasitas em peças de ímã, limitada por resistência.

pinneapple_simulation.electromagnetics.reflector

Antenas refletoras e compact ranges por óptica física: geometria, correntes PO, campo próximo e métricas de zona quieta.

`pinneapple_simulation/electromagnetics/reflector/__init__.py` · 18 linhas · testes: 6

pinneapple_simulation.electromagnetics.reflector.analytic

Formas fechadas que verificam o código de óptica física: campo no eixo de um disco PEC e taper de abertura por óptica geométrica.

`pinneapple_simulation/electromagnetics/reflector/analytic.py` · 55 linhas · testes: 6

nome	tipo	assinatura	o que faz
`disk_on_axis`	função	`(z, radius, k, E0=1.0) -> np.ndarray`	E_x exato no eixo de um disco PEC sob onda plana normal (corrente PO, integral exata).
`go_aperture_taper_db`	função	`(x, y, focal, q, feed_axis) -> np.ndarray`	Amplitude relativa por óptica geométrica na abertura de um paraboloide alimentado por um feed `cos^q`.

pinneapple_simulation.electromagnetics.reflector.compact_range

Compact range de refletor offset único: feed, refletor, zona quieta, avaliação por PO e varreduras de parâmetros; os valores padrão são um exemplo genérico.

`pinneapple_simulation/electromagnetics/reflector/compact_range.py` · 153 linhas · testes: 20

nome	tipo	assinatura	o que faz
`CompactRange`	classe	`(freq_hz, reflector, feed_gain_dbi, pol, qz_z, qz_diameter, qz_centre_y, feed_offset, mes…)`	Projeto de compact range: frequência, refletor, ganho e polarização do feed, plano e diâmetro da zona quieta, parâmetros de malha.
`scaled`	função	`(cr, side=None, roll=None, feed_gain_dbi=None) -> CompactRange`	Cópia do range com outro tamanho de seção, raio de borda enrolada ou ganho de feed, mantendo a folga abaixo da seção.
`qz_field`	função	`(cr, points_xy=None) -> dict[str, Any]`	Campo PO no plano da zona quieta.
`evaluate`	função	`(cr, *, amp_tol=0.5, phase_tol=5.0) -> dict[str, Any]`	Métricas da zona quieta do range com tamanho de malha e custo.
`sweep`	função	`(base, gains, sides, rolls=(None,), *, log=None)`	Executa todas as combinações de ganho, tamanho e raio de borda e devolve uma `ResultTable`.
`go_check`	função	`(cr) -> dict[str, float]`	Compara a amplitude PO na zona quieta com o taper de óptica geométrica.

pinneapple_simulation.electromagnetics.reflector.geometry

Refletores paraboloides offset com bordas enroladas opcionais, como malhas de triângulos para a óptica física.

`pinneapple_simulation/electromagnetics/reflector/geometry.py` · 172 linhas · testes: 6

nome	tipo	assinatura	o que faz
`OffsetReflector`	classe	`(focal, half_x, half_y, centre_y, n, roll, end_angle, blend, roll_aspect)`	Seção de paraboloide offset (contorno superelipse) com borda enrolada opcional; dá o foco e o centro.
`ReflectorMesh`	classe	`(vertices, faces, centres, normals, areas, roll)`	Malha do refletor: vértices, faces, centros, normais, áreas e marca de borda enrolada.
`superellipse_radius`	função	`(phi, a, b, n)`	Raio de uma superelipse em função do ângulo.
`mesh_reflector`	função	`(r, h) -> ReflectorMesh`	Malha triangular do refletor com arestas de tamanho aproximado `h`.
`flat_disk`	função	`(radius, h) -> ReflectorMesh`	Disco plano malhado, para validar a PO contra a forma fechada.

pinneapple_simulation.electromagnetics.reflector.po

Óptica física sobre superfície PEC triangulada: campos de feed `cos^q` e de onda plana, correntes PO e campo irradiado exato com a função de Green diádica completa.

`pinneapple_simulation/electromagnetics/reflector/po.py` · 118 linhas · testes: 6

nome	tipo	assinatura	o que faz
`wavenumber`	função	`(f_hz) -> float`	Número de onda no vácuo para uma frequência.
`q_from_gain`	função	`(gain_dbi) -> float`	Expoente `q` de um feed `cos^q` a partir da diretividade em dBi.
`gain_from_q`	função	`(q) -> float`	Diretividade em dBi de um feed `cos^q`.
`cosq_feed`	função	`(points, k, *, position, axis, q, pol=(0.0, 1.0, 0.0))`	Campos E e H de um feed `cos^q` em pontos dados.
`plane_wave`	função	`(points, k, direction, pol, amplitude=1.0)`	Campos E e H de uma onda plana.
`po_currents`	função	`(normals, centres, H_inc, source=None)`	Correntes PO `J = 2 n x H` nos triângulos iluminados.
`radiate`	função	`(centres, areas, J, obs, k, chunk=2000000)`	Campo E irradiado pelas correntes nos pontos de observação, perto ou longe.

pinneapple_simulation.electromagnetics.reflector.qz

Métricas de zona quieta de um campo colimado: variação pico a pico, taper, ripple, ripple de fase e diâmetro utilizável.

`pinneapple_simulation/electromagnetics/reflector/qz.py` · 69 linhas · testes: 6

nome	tipo	assinatura	o que faz
`disk_grid`	função	`(diameter, step, centre=(0.0, 0.0)) -> np.ndarray`	Pontos de uma grade quadrada dentro de um disco.
`qz_metrics`	função	`(xy, E_co, diameter, *, centre=(0.0, 0.0), E_x=None, amp_tol=0.5, phase_tol=5.0) -> dict[…)`	Métricas do campo copolar complexo sobre o disco da zona quieta.

pinneapple_simulation.electromagnetics.reflector.study

Estudos de compact range: validação do código PO, varredura (ganho do feed x tamanho da seção), surrogate GP com erro fora da amostra e mapa de trade-off da qualidade da zona quieta.

`pinneapple_simulation/electromagnetics/reflector/study.py` · 212 linhas · testes: 6

nome	tipo	assinatura	o que faz
`validate`	função	`(base, *, disk_radius_wl=3.0, mesh_densities=(8, 12, 16), rolls_wl=(3.0, 6.0), log=print)…)`	As três checagens de validação (disco PO, PO vs. GO, convergência de malha) na frequência do range.
`fit_surrogates`	função	`(table, metrics=METRICS, *, test_frac=0.25, seed=0)`	Ajusta um GP por métrica e raio de borda, com dois erros fora da amostra.
`tradeoff_map`	função	`(table, models, path, *, roll, qz_diameter, amp_tol=0.5, phase_tol=5.0, mark=None, metric…)`	Mapas previstos da métrica e do diâmetro utilizável sobre (ganho, tamanho), com a região viável.

nome	tipo	assinatura	o que faz
qz_figure	função	(cr, path, *, length_unit=1.0, unit_name='m', title='') -> dict[str, Any]	Figura da amplitude e fase copolar na zona quieta, com cortes; devolve as métricas.
qz_study	função	(base, gains, sides, rolls, out, *, mark=None, side_unit=1.0, side_label='section side [m…)	Varredura, surrogates e um mapa de trade-off por raio de borda numa pasta de saída.

pinneapple_simulation.electromagnetics.reflector.study_report

Estudo de compact range que se planeja sozinho: do problema às checagens e varreduras necessárias (por limiares de números adimensionais), à execução com cache em JSON e à folha de dados HTML/PDF.

pinneapple_simulation/electromagnetics/reflector/study_report.py · 1.018 linhas · testes: 14

nome	tipo	assinatura	o que faz
Problem	classe	(freq_hz, focal, section, offset, roll, feed_gain_dbi, qz_diameter, qz_z, n, pols, band_h…)	Projeto de compact range e o que ele deve entregar; aceita leitura de texto livre com unidades (from_text).
PlannedStep	classe	(key, title, reason, required)	Passo planejado do estudo: chave, título, motivo e se é obrigatório.
from_text	função	(text, **defaults) -> Problem	Atalho para Problem.from_text.
plan	função	(problem) -> list[PlannedStep]	Lista os passos que o problema exige e por quê.
field_on_points	função	(cr, pts, *, mesh_J=None) -> np.ndarray	Campo PO em pontos absolutos.
qz_metrics_at	função	(cr, z=None, *, amp_tol=None, phase_tol=None) -> dict[str, float]	Métricas de disco no plano da zona quieta numa posição z.
cut_metrics_at	função	(cr, z=None, *, half=None, points=None, cuts=('horizontal', 'vertical')) -> dict[str, dic…)	Métricas pico a pico e polarização cruzada em cortes pelo centro da zona quieta.
main_roll	função	(p) -> float	Raio de borda enrolada principal do problema.
run	função	(p, out='cr_study', *, steps=None, parallel=0, log=print, reuse=True) -> dict[str, Any]	Executa o plano (ou os passos nomeados) e guarda os números em study.json.
report	função	(p, study, out='report', *, pdf_name=None) -> dict[str, str | None]	Renderiza o estudo como folha de dados técnica (HTML, e PDF quando há navegador).

pinneapple_simulation.electromagnetics.reflector.study_report_figures

Figuras do relatório de compact range (geometria, iluminação, validação, zona quieta, varreduras), desenhadas a partir dos números guardados. [sem teste direto]

pinneapple_simulation/electromagnetics/reflector/study_report_figures.py · 431 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
build_figures	função	(p, study, out) -> dict[str, str]	Gera todas as figuras do relatório como {chave: caminho png}.

pinneapple_simulation.electromagnetics.reflector.study_report_text

Texto da folha de dados do estudo de compact range: cada frase é calculada a partir dos números, com regras fixas e sem modelo de linguagem. [sem teste direto]

pinneapple_simulation/electromagnetics/reflector/study_report_text.py · 578 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
build_datasheet	função	(p, study, figs) -> Datasheet	Monta o Datasheet (de pinneapple_tools.reporting.datasheet) a partir do problema, do estudo e das figuras.

pinneapple_simulation.electromagnetics.statics

Eletrostática e magnetostática em forma fechada (capacitores, fios, espiras, solenoides, indutâncias) e integração de Biot-Savart de correntes filamentares, vetorizadas.

pinneapple_simulation/electromagnetics/statics.py · 293 linhas · testes: 50

nome	tipo	assinatura	o que faz
parallel_plate_capacitance	função	(area, gap, eps_r=1.0) -> float	Capacitância de placas paralelas sem efeito de borda.
coax_capacitance	função	(a, b, eps_r=1.0) -> float	Capacitância por metro de uma linha coaxial.
coax_inductance	função	(a, b, mu_r=1.0) -> float	Indutância externa por metro de uma linha coaxial.
spherical_capacitance	função	(a, b=math.inf, eps_r=1.0) -> float	Capacitância de esferas concêntricas (ou esfera isolada).
point_charge_field	função	(points, q, at=(0.0, 0.0, 0.0), eps_r=1.0)	Potencial e campo de uma carga pontual.
line_charge_field	função	(points, lam, eps_r=1.0)	Campo E de uma linha de carga infinita no eixo z.
infinite_wire_B	função	(points, current)	Campo B de um fio reto infinito no eixo z.
polyline_segments	função	(vertices) -> tuple[np.ndarray, np.ndarray]	Segmentos (início, fim) de uma polilinha aberta.
circle_segments	função	(radius, n=360, center=(0.0, 0.0, 0.0), z=0.0)	Círculo aproximado por segmentos retos.
helix_segments	função	(radius, length, turns, n_per_turn=72, z0=None)	Bobina helicoidal ao longo de z em segmentos retos.
biot_savart	função	(points, starts, ends, current=1.0, chunk=4096)	Campo B exato de segmentos filamentares finitos (Biot-Savart).
vector_potential	função	(points, starts, ends, current=1.0, chunk=4096)	Potencial vetor A exato de segmentos filamentares (gauge de Coulomb).
flux_through_path	função	(path_starts, path_ends, starts, ends, current=1.0, gauss=4) -> float	Fluxo magnético através de um caminho poligonal fechado pela circulação de A (Stokes).
loop_on_axis_B	função	(z, radius, current)	B axial de uma espira circular.
loop_field	função	(points, radius, current)	B de uma espira circular em qualquer ponto fora do fio, com integrais elípticas completas.
finite_solenoid_on_axis_B	função	(z, radius, length, n_turns, current)	B axial de um solenoide finito bem enrolado.
helmholtz_coil_center_B	função	(radius, current, turns=1) -> float	Campo no centro de um par de Helmholtz.
mutual_inductance_coaxial_loops	função	(r1, r2, d) -> float	Indutância mútua de duas espiras coaxiais (fórmula de Maxwell).
loop_self_inductance	função	(R, a, internal=True) -> float	Autoindutância de uma espira de fio fino.
filament_inductance	função	(paths, wire_radius, pieces_per_metre=2000.0, gauss=6, mirror_z=None) -> np.ndarray	Matriz de indutâncias de condutores finos ao longo de polilinhas (fórmula de Neumann).

pinneapple_simulation.electromagnetics.units

Dimensões SI das grandezas eletromagnéticas e verificação dimensional das equações de Maxwell e de unidades declaradas pelo usuário.

pinneapple_simulation/electromagnetics/units.py · 145 linhas · testes: 26

nome	tipo	assinatura	o que faz
UnitProblem	classe	(where, message)	Um problema de unidade encontrado: onde e mensagem.
dimension	função	(unit) -> Dim	Vetor de dimensão de uma expressão de unidade.
term_dimension	função	(term, quantities=QUANTITIES) -> Dim	Dimensão de um termo (produto de grandezas e operadores).
check_equations	função	(equations=EQUATIONS, quantities=QUANTITIES) -> list[UnitProblem]	Verifica que todos os termos de cada lei têm a mesma dimensão e devolve as violações.
check_units	função	(declared) -> list[UnitProblem]	Compara unidades declaradas de grandezas nomeadas com a tabela de referência.

pinneapple_simulation.electromagnetics.verification

Suíte de verificação do eletromagnetismo: cada caso compara um solver com um resultado independente (forma fechada, valor publicado, outra biblioteca, limite) e mede erro e ordem de convergência; gera a tabela de `docs/guides/electromagnetics.md`.

`pinneapple_simulation/electromagnetics/verification.py` · 893 linhas · testes: 12

nome	tipo	assinatura	o que faz
`CaseResult`	classe	`(name, what, reference, error, tolerance, orders, expected_order, detail, seconds)`	Resultado de um caso de verificação: erro, tolerância, ordens observadas e esperada, detalhes e tempo; `passed` e `row`.
`case`	função	`(name)`	Decorador que registra uma função como caso de verificação.
`observed_orders`	função	`(errors, ratio=2.0) -> list[float]`	Ordens de convergência observadas entre erros sucessivos de um refinamento.
`run`	função	`(names=None, quick=False) -> list[CaseResult]`	Executa os casos registrados (todos ou os nomeados, opcionalmente em modo rápido).
`to_markdown`	função	`(results) -> str`	Renderiza os resultados como tabela Markdown.
`write_markdown_block`	função	`(path, results, marker='verification-table') -> None`	Substitui, num arquivo Markdown, o texto entre marcadores pela tabela dos resultados.

pinneapple_simulation.electromagnetics.waves

Eletromagnetismo de ondas em forma fechada (convenção `exp(+j omega t)`): Fresnel, multicamadas, efeito pelicular, guias e cavidades, dipolos, espalhamento por cilindro e plásmons de superfície.

`pinneapple_simulation/electromagnetics/waves.py` · 372 linhas · testes: 51

nome	tipo	assinatura	o que faz
`MultilayerResult`	classe	`(r, t, R, T)`	Coeficientes de reflexão e transmissão (amplitude e potência) de uma pilha de camadas.
`free_space_wavelength`	função	`(f) -> float`	Comprimento de onda no vácuo para uma frequência.
`fresnel`	função	`(n1, n2, theta_i)`	Coeficientes de Fresnel (rs, rp, ts, tp) em uma interface plana.
`brewster_angle`	função	`(n1, n2) -> float`	Ângulo de Brewster.
`critical_angle`	função	`(n1, n2) -> float`	Ângulo crítico da reflexão interna total.
`multilayer`	função	`(n_layers, d_layers, wavelength, theta_i=0.0, pol='s', n_in=1.0, n_out=1.0) -> Multilayer…)`	Reflexão e transmissão de uma pilha de camadas planas pelo método das matrizes características.
`skin_depth`	função	`(omega, sigma, mu_r=1.0, eps_r=1.0, exact=False)`	Profundidade pelicular (forma de bom condutor ou exata).
`surface_impedance`	função	`(omega, sigma, mu_r=1.0)`	Impedância de superfície de um bom condutor.
`rect_waveguide_cutoff`	função	`(m, n, a, b, eps_r=1.0, mu_r=1.0) -> float`	Frequência de corte de um modo TE/TM de guia retangular.
`rect_waveguide_modes`	função	`(a, b, n_modes=10, eps_r=1.0, max_index=8)`	Modos mais baixos de um guia retangular ordenados por corte.
`rect_te_mn_fields`	função	`(x, y, m, n, a, b)`	Perfil transversal (não normalizado) do modo TE_mn de um guia retangular.
`circular_waveguide_cutoff`	função	`(kind, m, n, radius, eps_r=1.0) -> float`	Frequência de corte de um modo TE/TM de guia circular.
`circular_waveguide_modes`	função	`(radius, n_modes=10, eps_r=1.0, max_m=6, max_n=4)`	Modos mais baixos de um guia circular ordenados por corte.
`propagation_constant`	função	`(f, f_cutoff, eps_r=1.0, mu_r=1.0)`	Constante de propagação acima do corte (ou evanescente abaixo).
`rect_cavity_resonance`	função	`(m, n, p, a, b, d, eps_r=1.0, mu_r=1.0) -> float`	Ressonância de um modo de cavidade retangular PEC.

nome	tipo	assinatura	o que faz
`rect_cavity_modes`	função	`(a, b, d, n_modes=10, eps_r=1.0, max_index=6)`	Ressonâncias mais baixas de uma cavidade retangular com multiplicidade.
`cylindrical_cavity_resonance`	função	`(kind, m, n, p, radius, length, eps_r=1.0) -> float`	Ressonância TE/TM de uma cavidade cilíndrica fechada.
`rect_cavity_te101_q`	função	`(a, b, d, sigma, eps_r=1.0) -> float`	Fator Q por perdas no condutor do modo TE101.
`hertzian_dipole_fields`	função	`(r, theta, current_moment, omega)`	Campos completos de um dipolo hertziano.
`hertzian_dipole_power`	função	`(current_moment, omega) -> float`	Potência irradiada média de um dipolo hertziano.
`hertzian_dipole_radiation_resistance`	função	`(length, wavelength) -> float`	Resistência de radiação de um dipolo curto.
`dipole_pattern`	função	`(theta, length_wavelengths=0.5)`	Padrão de campo distante de um dipolo fino alimentado no centro.
`cylinder_tm_coefficients`	função	`(ka, eps_r, n_max=None)`	Coeficientes do espalhamento TM_z por um cilindro dielétrico ou PEC.
`cylinder_scattering_tm`	função	`(x, y, k0, radius, eps_r, E0=1.0, total=True)`	Campo Ez fasorial do espalhamento de uma onda plana por um cilindro circular.
`half_wave_dipole_radiation_resistance`	função	`(eta=ETA0) -> float`	Resistência de radiação do dipolo de meia onda.
`half_wave_dipole_directivity`	função	`() -> float`	Diretividade do dipolo de meia onda.
`spp_wavenumber`	função	`(omega, eps_metal, eps_dielectric=1.0)`	Número de onda de um plásmon polariton de superfície numa interface plana.
`plasma_dispersion_omega`	função	`(k, omega_p)`	Relação de dispersão de ondas transversais num plasma sem colisões.

pinneapple_simulation.external_solvers

Ponto de entrada das integrações externas: reexporta OpenFOAM e, com imports opcionais (`try/except`), os leitores CFD, MATLAB, Modelica, FEniCS, MuJoCo, Genesis e Newton, para o pacote importar mesmo sem as ferramentas instaladas.

`pinneapple_simulation/external_solvers/__init__.py` · 226 linhas · testes: 16

pinneapple_simulation.external_solvers.ansys

Pacote do acoplamento com ANSYS Fluent em modo batch (journal `.jou`); reexporta `journal_builder` e `runner`. [sem teste]

`pinneapple_simulation/external_solvers/ansys/__init__.py` · 23 linhas · **sem teste direto**

pinneapple_simulation.external_solvers.ansys.journal_builder

Gera o texto de um journal TUI do Fluent (ler caso/malha, condições de contorno simples, inicializar, iterar, gravar) a partir de `FluentCaseConfig`. O docstring separa comandos de alta e média confiança e lista o que não foi implementado; nunca foi rodado contra um Fluent real.

`pinneapple_simulation/external_solvers/ansys/journal_builder.py` · 188 linhas · testes: 15 · palavras-chave no código (heurística; o texto acima diz o status real): placeholder/stub citado no docstring

nome	tipo	assinatura	o que faz
`FluentCaseConfig`	classe	`(output_case_file, case_file, mesh_file, steady, iterations, time_steps, max_iterations_p…)`	Configuração de uma rodada batch do Fluent: caso ou malha de entrada, regime permanente/transiente, iterações e condições de velocity-inlet/pressure-outlet.
`build_journal`	função	`(config) -> str`	Gera o texto do journal TUI do Fluent para uma `FluentCaseConfig`, na ordem ler, contorno, inicializar, resolver, gravar e sair.

pinneapple_simulation.external_solvers.ansys.runner

Lança o executável `fluent` real com o journal gerado, guarda logs e entrega o resultado ao leitor `fluent_mesh_to_upd`. Só os caminhos de geração de journal e de 'executável não encontrado' são testados (não há Fluent licenciado no ambiente).

`pinneapple_simulation/external_solvers/ansys/runner.py` · 182 linhas · testes: 15

nome	tipo	assinatura	o que faz
`FluentRunConfig`	classe	`(executable, precision, n_procs, work_dir, timeout_s, extra_args)`	Como invocar o executável do Fluent (nome, precisão, número de processos, pasta, timeout), sem dizer o que simular.
`FluentRunResult`	classe	`(success, upd_result, exit_code, wall_time_s, stdout_log_path, stderr_log_path, journal_p…)`	Resultado de uma rodada do Fluent: sucesso, `PhysicalSample` lido, código de saída, tempo, caminhos de log e journal, e erro de leitura.
`run_fluent_case`	função	`(case_config, run_config=None) -> FluentRunResult`	Monta o journal, lança o Fluent em batch e empacota o resultado com `fluent_mesh_to_upd`; falha explicitamente se o executável não existir.

pinneapple_simulation.external_solvers.calculix

Ponte para o CalculiX (FEM estrutural implícito): decks `.inp`, execução do `ccx`, leitura de `.frd` e um `ccx` virtual.

`pinneapple_simulation/external_solvers/calculix/__init__.py` · 23 linhas · testes: 14

pinneapple_simulation.external_solvers.calculix.frd

Lê e escreve arquivos de resultado `.frd` do CalculiX (ASCII) e lê deslocamentos nodais de `.dat`.

`pinneapple_simulation/external_solvers/calculix/frd.py` · 135 linhas · testes: 7

nome	tipo	assinatura	o que faz
`FrdResults`	classe	`(node_ids, coords, steps)`	Resultados lidos de um `.frd`: ids e coordenadas dos nós e os passos com seus campos (acesso por `field`).
`read_frd`	função	`(path) -> FrdResults`	Lê um `.frd` ASCII do CalculiX em `FrdResults`.
`write_frd`	função	`(path, node_ids, coords, elements=None, element_type='C3D8', fields=None, step_time=1.0, …)`	Escreve um `.frd` ASCII com nós, elementos e blocos `DISP`/`STRESS`.
`read_dat_displacements`	função	`(path) -> Dict[int, np.ndarray]`	Lê deslocamentos nodais de um arquivo `.dat` (saída `*NODE PRINT`).

pinneapple_simulation.external_solvers.calculix.inp

Contêiner simples de modelo CalculiX, escritor e leitor de decks `.inp`, e uma malha hexaédrica de viga em balanço. [sem teste direto]

`pinneapple_simulation/external_solvers/calculix/inp.py` · 205 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`CalculixModel`	classe	`(nodes, elements, element_type, nsets, material, E, nu, density, boundary, cloads, title)`	Modelo CalculiX mínimo: nós, elementos, conjuntos, material elástico, apoios e cargas pontuais.
`write_inp`	função	`(model, path, *, job_outputs=('U',), stress_outputs=('S',)) -> str`	Escreve um `CalculixModel` como deck `.inp` com pedidos de saída de deslocamento e tensão.
`read_inp`	função	`(path) -> CalculixModel`	Lê malha, material, apoios e cargas pontuais de um deck `.inp`, seguindo `*INCLUDE`.
`expand_target`	função	`(model, target) -> List[int]`	Devolve os ids de nós por trás de um alvo de `*BOUNDARY`/`*CLOAD` (id de nó ou nome de NSET).
`cantilever_hex_mesh`	função	`(L=1.0, b=0.1, h=0.1, nx=20, ny=2, nz=2, element_type='C3D20R') -> CalculixModel`	Cria uma viga em balanço de hexaedros ao longo de x, engastada em x = 0, com elementos de 8 ou 20 nós.

pinneapple_simulation.external_solvers.calculix.runner

Executa `ccx` com o binário nativo se estiver no PATH, senão com uma imagem Docker construída localmente a partir de um `Dockerfile` (Debian `calculix-ccx`).

`pinneapple_simulation/external_solvers/calculix/runner.py` · 100 linhas · testes: 14

nome	tipo	assinatura	o que faz
`CcxRun`	classe	`(returncode, stdout, frd, dat)`	Resultado de uma execução do `ccx`: código de retorno, saída e caminhos do `.frd` e `.dat`.
`docker_available`	função	`() -> bool`	Diz se o Docker responde (`docker info`).
`ensure_image`	função	`(image=IMAGE, attempts=3) -> str`	Constrói a imagem Docker do CalculiX se faltar, tentando registries alternativos a cada falha.
`ccx_backend`	função	`() -> str | None`	Diz qual backend está disponível: 'native', 'docker' ou None.
`run_ccx`	função	`(workdir, job, *, threads=1, timeout=3600.0, backend=None) -> CcxRun`	Roda `ccx -i <job>` numa pasta que contém `<job>.inp`, pelo backend escolhido.

pinneapple_simulation.external_solvers.calculix.study

Estudos CalculiX em poucas linhas: malha (hexaedros, tetraedros gmsh ou deck existente), conjuntos por geometria, um passo de análise (estático, frequência, flambagem, calor), execução e resultados em arrays.

`pinneapple_simulation/external_solvers/calculix/study.py` · 503 linhas · testes: 7

nome	tipo	assinatura	o que faz
`Material`	classe	`(name, E, nu, density, conductivity, expansion, plastic)`	Material do estudo (E, nu, densidade, condutividade, expansão, plasticidade) e seu trecho de deck.
`FEModel`	classe	`(nodes, elements, element, material, title)`	Modelo de elementos finitos do estudo: malha (caixa, hexmesh, `.inp` ou gmsh), seleção de nós e faces por geometria, e o deck.
`Static`	classe	`(fix, loads, pressure, gravity, nlgeom, increments, prescribed)`	Passo estático linear ou não linear: apoios, forças, pressões, gravidade e deslocamentos prescritos.
`Frequency`	classe	`(n_modes, fix)`	Passo de frequências naturais e modos de vibração (requer densidade).
`Buckle`	classe	`(n_modes, fix, loads, pressure)`	Passo de flambagem linear: fatores de carga das cargas de referência e formas de flambagem.
`Heat`	classe	`(temperature, film, flux, initial)`	Passo de condução de calor permanente com temperaturas, convecção (film) e fluxos.
`Result`	classe	`(model, u, stress, strain, reactions, peeq, temperature, frequencies, buckling_factors, m…)`	Resultados de um estudo: deslocamentos, tensões, deformações, reações, temperaturas, frequências e modos, com von Mises e deslocamento máximo.
`solve`	função	`(model, step, workdir, *, threads=2, keep=True) -> Result`	Escreve `workdir/job.inp`, roda o `ccx` e lê os resultados num `Result`.
`geo_plate_with_hole`	função	`(L, W, t, d) -> str`	Gera a geometria (texto para gmsh) de uma placa com furo central.
`geo_l_bracket`	função	`(a=0.1, b=0.08, w=0.04, t=0.01, hole=0.012, fillet=0.01) -> str`	Gera a geometria (texto para gmsh) de uma cantoneira em L com filete e furo de parafuso.

pinneapple_simulation.external_solvers.calculix.virtual_ccx

Um `ccx` substituto (`pinneapple-ccx`): mesma linha de comando, lê o `.inp` e escreve o `.frd` com campos previstos por um modelo (PINN/surrogate), recusando pontos fora do envelope de validade ou caindo no `ccx` real com `--fallback`.

`pinneapple_simulation/external_solvers/calculix/virtual_ccx.py` · 117 linhas · testes: 7

nome	tipo	assinatura	o que faz
`OutOfEnvelope`	classe	`()`	Erro levantado quando o caso pedido sai do envelope de validade do modelo preditor.

nome	tipo	assinatura	o que faz
`check_envelope`	função	`(model, coords, envelope, tol=1e-06) -> None`	Confere caixa da geometria, módulo E e cargas contra o envelope declarado pelo preditor e levanta `OutOfEnvelope` se sair.
`predict_to_frd`	função	`(inp_path, predictor, frd_path=None) -> str`	Lê um `.inp`, prevê os campos nodais com o preditor e escreve o `.frd` correspondente.
`load_predictor`	função	`(spec) -> Callable`	Carrega um preditor a partir de uma string `pacote.modulo:atributo`.
`main`	função	`(argv=None) -> int`	Linha de comando `pinneapple-ccx -i job [--predictor ...] [--fallback]`, espelhando `ccx -i job`.

`pinneapple_simulation.external_solvers.campaign_cases`

Escreve os arquivos de uma campanha de simulação planejada: tabelas de casos, um runner Python, casos OpenFOAM (aerofólio CST ou cópias de um caso base) e decks CalculiX, mais contratos e README; usado pelo app Campaign Planner.

`pinneapple_simulation/external_solvers/campaign_cases.py` · 349 linhas · testes: 7

nome	tipo	assinatura	o que faz
`write_case_tables`	função	`(cases, out_dir, space=None, stem='cases', formats=('csv', 'parquet', 'json')) -> list[Pa…)`	Grava a tabela de casos em CSV, Parquet e/ou JSON, com o espaço de parâmetros ao lado.
`write_python_runner`	função	`(out_dir, parameters, outputs=()) -> Path`	Gera `run_cases.py`, que percorre os casos e chama a função `simulate` do usuário (o modelo gerado levanta `NotImplementedError` até ser preenchido).
`airfoil_space`	função	`(budget=40) -> ParameterSpace`	Espaço de parâmetros da campanha de aerofólio: pesos CST de espessura e arqueamento e ângulo de ataque.
`airfoil_feasible`	função	`(cases) -> np.ndarray`	Marca as formas de aerofólio válidas (superfícies que não se cruzam e permanecem suaves).
`airfoil_existing`	função	`()`	Devolve como DataFrame as rodadas OpenFOAM já existentes no dataset do zoo `airfoil_rans_cst_re4e6`.
`write_openfoam_airfoil`	função	`(cases, out_dir, reynolds=AIRFOIL_REYNOLDS, iterations=1200, limit=None) -> list[Path]`	Escreve um caso simpleFoam por linha (aerofólio CST) com `params.json`, `coefficients.py` e `Allrun`.
`write_openfoam_from_base`	função	`(base, cases, out_dir, edits, derived=None, solver='simpleFoam', procs=1, pre=('checkMesh…)`	Copia um caso OpenFOAM base aplicando edições de dicionário por caso e grava o `plan.json` do runner de campanhas.
`cantilever_space`	função	`(budget=30) -> ParameterSpace`	Espaço de parâmetros de uma viga em balanço com carga na ponta (dimensões, material, carga), esbelta para valer a teoria de vigas.
`write_calculix_cantilever`	função	`(cases, out_dir, elements_per_height=2) -> list[Path]`	Escreve um `job.inp` CalculiX por caso da viga em balanço, com a deflexão de Euler-Bernoulli em `params.json`.
`write_calculix_from_template`	função	`(template, cases, out_dir, name='job.inp') -> list[Path]`	Preenche os marcadores `{parametro}` de um deck CalculiX para cada caso.
`write_campaign`	função	`(plan, out_dir, *, outputs=(), units=None, openfoam=None, calculix=None, openfoam_base=No…)`	Grava um `CampaignPlan` inteiro: tabelas, runner, casos OpenFOAM/CalculiX que se aplicam, contratos e README.

`pinneapple_simulation.external_solvers.cfd_formats`

Pacote de leitores de formatos abertos CFD/FEM além do OpenFOAM: CGNS, Exodus II, malha Fluent (só nós), ponte Abaqus `.inp`/`.odb` e exportações do STAR-CCM+. [sem teste direto do `__init__`]

`pinneapple_simulation/external_solvers/cfd_formats/__init__.py` · 48 linhas · **sem teste direto**

`pinneapple_simulation.external_solvers.cfd_formats._abaqus_odb_export_script`

Script que roda dentro do Python do próprio Abaqus (`abaqus python ...`) para exportar campos de um `.odb` para `.npz`; nunca é importado pelo código normal. [sem teste]

pinneapple_simulation/external_solvers/cfd_formats/_abaqus_odb_export_script.py · 87 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
main	função	()	Lê os argumentos JSON, abre o .odb com a API do Abaqus e grava os campos pedidos num .npz.

pinneapple_simulation.external_solvers.cfd_formats.abaqus_reader

Lê nós e elementos de um .inp do Abaqus e faz a ponte para resultados .odb chamando o Abaqus instalado como subprocesso; empacota tudo como PhysicalSample.

pinneapple_simulation/external_solvers/cfd_formats/abaqus_reader.py · 320 linhas · testes: 5

nome	tipo	assinatura	o que faz
read_abaqus_inp_mesh	função	(path)	Extrai os blocos *NODE e *ELEMENT de um .inp do Abaqus.
abaqus_inp_to_upd	função	(path)	Empacota a malha de um .inp (sem campos de resultado) como PhysicalSample.
export_odb_fields	função	(odb_path, out_npz_path, *, step_name=None, frame_index=-1, field_outputs=('U', 'S'), aba…)	Exporta campos de um .odb para .npz rodando o Python do Abaqus instalado.
load_exported_odb_npz	função	(npz_path)	Carrega o .npz exportado de um .odb como PhysicalSample.

pinneapple_simulation.external_solvers.cfd_formats.cgns_reader

Lê coordenadas e campos de solução de arquivos CGNS/HDF5 e os empacota como PhysicalSample com domínio de malha.

pinneapple_simulation/external_solvers/cfd_formats/cgns_reader.py · 186 linhas · testes: 14

nome	tipo	assinatura	o que faz
read_cgns_mesh_and_fields	função	(path, *, fields=None, zone_index=0)	Lê coordenadas de nós e campos de solução de um arquivo CGNS/HDF5.
cgns_to_upd	função	(path, *, fields=None, zone_index=0)	Lê um CGNS/HDF5 e empacota como PhysicalSample com geometry.nodes.

pinneapple_simulation.external_solvers.cfd_formats.convert

Despacho por formato de arquivos CFD/FEM/CAD: nuvem de pontos portátil de qualquer formato suportado, superfície triangulada dos formatos com conectividade, e reparo/simplificação na conversão.

pinneapple_simulation/external_solvers/cfd_formats/convert.py · 142 linhas · testes: 6

nome	tipo	assinatura	o que faz
detect_format	função	(filename) -> str	Devolve o nome do formato pelo sufixo do arquivo, ou ValueError listando os suportados.
read_point_bundle	função	(path, fmt=None) -> dict[str, Any]	Lê um formato volumétrico como dicionário de coordenadas, campos, proveniência e formato.
tet_boundary_faces	função	(tets) -> np.ndarray	Devolve os triângulos usados por um só tetraedro, ou seja, a fronteira de uma malha tetraédrica.
read_surface_mesh	função	(path, fmt=None)	Lê uma superfície triangulada (MeshData) de STL, OBJ, PLY, STEP ou .inp do Abaqus.
convert_surface	função	(path, out_path, *, fmt=None, repair=False, simplify_target_faces=None) -> dict[str, Any]	Lê uma superfície, opcionalmente repara e simplifica, e grava em STL, OBJ ou PLY.

pinneapple_simulation.external_solvers.cfd_formats.exodus_reader

Lê coordenadas e campos nodais de Exodus II clássico (NetCDF-3) via scipy.io.netcdf_file; arquivos NetCDF-4/HDF5 levantam NotImplementedError com a explicação, e a conectividade de elementos não é lida.

pinneapple_simulation/external_solvers/cfd_formats/exodus_reader.py · 177 linhas · testes: 5 · palavras-chave no código (heurística; o texto acima diz o status real): levanta NotImplementedError em algum caminho

nome	tipo	assinatura	o que faz
`read_exodus`	função	`(path, *, fields=None, time_index=-1)`	Lê coordenadas e campos nodais de um Exodus II em NetCDF clássico.
`exodus_to_upd`	função	`(path, *, fields=None, time_index=-1)`	Lê um Exodus II e empacota como `PhysicalSample` com domínio de malha.

`pinneapple_simulation.external_solvers.cfd_formats.fluent_mesh_reader`

Lê só as coordenadas de nós (seção 10, ASCII) de malhas `.msh` do Fluent/Gambit; células, faces, zonas e seções binárias levantam `NotImplementedError` de propósito.

`pinneapple_simulation/external_solvers/cfd_formats/fluent_mesh_reader.py` · 162 linhas · testes: 5 · palavras-chave no código (heurística; o texto acima diz o status real): levanta `NotImplementedError` em algum caminho

nome	tipo	assinatura	o que faz
`read_fluent_mesh_nodes`	função	`(path)`	Lê as coordenadas de nós das seções 10 de um `.msh` ASCII do Fluent/Gambit.
`fluent_mesh_to_upd`	função	`(path)`	Empacota os nós de um `.msh` do Fluent como `PhysicalSample` sem campos.

`pinneapple_simulation.external_solvers.cfd_formats.starccm_reader`

Lê exportações do STAR-CCM+: CGNS (reaproveita `cgns_reader`) e tabelas texto/CSV (parser próprio). O formato nativo `.sim`/`.ccm` não é suportado, por ser binário proprietário.

`pinneapple_simulation/external_solvers/cfd_formats/starccm_reader.py` · 243 linhas · testes: 9 · palavras-chave no código (heurística; o texto acima diz o status real): placeholder/stub citado no docstring

nome	tipo	assinatura	o que faz
`read_starccm_cgns`	função	`(path, *, fields=None, zone_index=0)`	Lê uma exportação CGNS do STAR-CCM+, delegando a `read_cgns_mesh_and_fields`.
`starccm_cgns_to_upd`	função	`(path, *, fields=None, zone_index=0)`	Empacota uma exportação CGNS do STAR-CCM+ como `PhysicalSample`, delegando a `cgns_to_upd`.
`read_starccm_table`	função	`(path, *, coord_names=('X', 'Y', 'Z'))`	Lê uma tabela texto/CSV exportada do STAR-CCM+, com cabeçalhos que podem ter unidades.
`starccm_table_to_upd`	função	`(path, *, coord_names=('X', 'Y', 'Z'))`	Empacota uma tabela exportada do STAR-CCM+ como `PhysicalSample` com as coordenadas como nós.

`pinneapple_simulation.external_solvers.fenics`

Pacote da integração FEniCS; reexporta a camada de workflow de `fenics.solver`.

`pinneapple_simulation/external_solvers/fenics/__init__.py` · 30 linhas · **sem teste direto**

`pinneapple_simulation.external_solvers.fenics.solver`

Camada de workflow sobre `FEniCSBridge` (em `pinneapple_simulation.numerical_solvers.fenics_bridge`): configuração, solve com empacotamento em `PhysicalSample` e varreduras de parâmetros. Requer FEniCS instalado.

`pinneapple_simulation/external_solvers/fenics/solver.py` · 180 linhas · testes: 151

nome	tipo	assinatura	o que faz
`FEniCSConfig`	classe	`(pde, domain, bcs, params, solver_opts)`	Configuração de um solve FEniCS: identificador da EDP, domínio, condições de contorno, parâmetros e opções do solver.
`FEniCSWorkflow`	classe	`(base_config) -> None`	Atalho para solves FEniCS repetidos sobre uma configuração base, com `solve` e `sweep` em grade de parâmetros.
`solve_and_package`	função	`(config)`	Roda um solve via `FEniCSBridge` e devolve um `PhysicalSample`.
`dof_to_upd`	função	`(dof_vector, mesh_coords, field_name='u')`	Empacota um vetor de DOFs do FEniCS e as coordenadas da malha como `PhysicalSample`.

pinneapple_simulation.external_solvers.fluidx3d

Ponte para o FluidX3D (LBM em GPU, OpenCL): escreve o `main_setup()` de um túnel de vento no checkout do usuário, compila e roda com `make.sh` e lê forças e campos `.vtk`. O PINNeAPPle não distribui código do FluidX3D, e o uso segue a licença dele (proíbe uso comercial e militar).

`pinneapple_simulation/external_solvers/fluidx3d.py` · 225 linhas · testes: 9

nome	tipo	assinatura	o que faz
`WindTunnelSetup`	classe	`(length_m, velocity_m_s, nu_m2_s, rho_kg_m3, stl, box, memory_mb, body_cells, lbm_velocit…)`	Descrição de um túnel de vento no FluidX3D: corpo STL (ou esfera), velocidade, viscosidade, caixa, memória, tempo e exportações.
`fluidx3d_home`	função	`() -> str | None`	Devolve a pasta do checkout do FluidX3D (`FLUIDX3D_HOME`) ou None.
`fluidx3d_available`	função	`() -> bool`	Diz se há um checkout configurado e um driver OpenCL instalado.
`write_setup`	função	`(setup, home=None) -> str`	Substitui o `main_setup()` de `src/setup.cpp` pelo gerado, guardando backup do original.
`enable_features`	função	`(features, home=None) -> str`	Ativa `#defines` em `src/defines.hpp` (com backup) e desliga `BENCHMARK`.
`run_fluidx3d`	função	`(setup=None, home=None, device=None, timeout=24 * 3600, log='log.pinneapple') -> subproce…)`	Escreve setup e defines, compila e roda com `make.sh`, gravando a saída num log.
`parse_forces`	função	`(text) -> np.ndarray`	Extrai as linhas (t, Fx, Fy, Fz) impressas pelo setup gerado.
`read_results`	função	`(home=None, log='log.pinneapple', export_dir=None) -> dict`	Lê o histórico de forças do log e o último campo exportado de cada grandeza.
`results_to_samples`	função	`(results, source='fluidx3d') -> list`	Converte os últimos campos exportados numa lista de `PhysicalSample` em grade regular.

pinneapple_simulation.external_solvers.genesis

Pacote da integração com Genesis AI (simulação multifísica para robótica); reexporta configuração, construtor de cena, runner e adaptadores. [sem teste]

`pinneapple_simulation/external_solvers/genesis/__init__.py` · 70 linhas · **sem teste direto**

pinneapple_simulation.external_solvers.genesis.adapter

Converte dicionários de trajetória do Genesis em `PhysicalSample` (UPD). [sem teste]

`pinneapple_simulation/external_solvers/genesis/adapter.py` · 96 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`genesis_traj_to_upd`	função	`(traj, time_array=None, field_map=None, meta_extra=None, dt=None)`	Converte um dicionário de trajetória do Genesis num `PhysicalSample`.
`genesis_trajs_to_upd`	função	`(trajs, time_array=None, field_map=None, meta_extra=None, dt=None) -> list`	Converte uma lista de trajetórias do Genesis em `PhysicalSamples`.

pinneapple_simulation.external_solvers.genesis.runner

Executa o laço `scene.step()` de uma cena Genesis e coleta estados (corpo rígido, partículas MPM/SPH ou função própria) por passo. [sem teste]

`pinneapple_simulation/external_solvers/genesis/runner.py` · 125 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`GenesisRunner`	classe	`(config, state_fn=None, controller_fn=None) -> None`	Executa uma cena Genesis e coleta observações de trajetória por passo de tempo.

pinneapple_simulation.external_solvers.genesis.scene_builder

Configuração de cena Genesis (backend, passo, ambientes paralelos, entidades, opções de corpo rígido/MPM/SPH) e a função que a materializa num `gs.Scene`. [sem teste]

`pinneapple_simulation/external_solvers/genesis/scene_builder.py` · 180 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`EntitySpec`	classe	`(morph, material, surface, vis_mode, name)`	Especificação de uma entidade da cena Genesis (morph, material, superfície, nome).
`GenesisConfig`	classe	`(backend, precision, dt, substeps, n_steps, n_envs, env_spacing, gravity, requires_grad, …)`	Configuração completa de uma cena Genesis: backend, precisão, passo, ambientes, gravidade, entidades e opções por física.
`build_scene`	função	`(config)`	Materializa uma `GenesisConfig` num `gs.Scene`.

pinneapple_simulation.external_solvers.geotaichi

Ponte para o GeoTaichi (MPM, DEM, FEM, IGA; licença GPL-3.0), rodado como programa separado: executa um script e lê os quadros de partículas `.npz`, que também servem para o solver `geo_mpm` do próprio PINNeAPPle.

`pinneapple_simulation/external_solvers/geotaichi.py` · 89 linhas · testes: 4

nome	tipo	assinatura	o que faz
`geotaichi_home`	função	`() -> str | None`	Devolve a pasta do checkout GeoTaichi (`GEOTAICHI_HOME`) ou None.
`geotaichi_available`	função	`() -> bool`	Diz se há um checkout do GeoTaichi configurado e se o Taichi é importável pelo interpretador que vai rodá-lo.
`run_geotaichi`	função	`(script, workdir, args=(), python=None, timeout=24 * 3600, log='log.geotaichi') -> subpro…)`	Roda um script do GeoTaichi num processo próprio dentro de `workdir`, com log.
`read_geotaichi`	função	`(path, kind='MPM', allow_pickle=False) -> list[dict]`	Lê os quadros de partículas de uma rodada; entradas pickled são puladas, salvo `allow_pickle=True`.
`frames_to_samples`	função	`(frames, source='geotaichi', fields=None, times=None) -> list`	Converte cada quadro em um `PhysicalSample`: posições como nuvem de pontos e os demais arrays como campos.

pinneapple_simulation.external_solvers.matlab

Pacote da integração MATLAB: Engine API, runner por subprocesso e E/S de `.mat`.

`pinneapple_simulation/external_solvers/matlab/__init__.py` · 27 linhas · **sem teste direto**

pinneapple_simulation.external_solvers.matlab.engine

Invólucro fino sobre `matlab.engine.MatlabEngine` (chamar funções, rodar scripts, avaliar, trocar variáveis). Requer a Engine API do MATLAB.

`pinneapple_simulation/external_solvers/matlab/engine.py` · 88 linhas · testes: 151

nome	tipo	assinatura	o que faz
`MATLABEngine`	classe	`(start_options=None) -> None`	Invólucro da Engine do MATLAB com `run_function`, `run_script`, `eval`, `get`, `put` e `quit`.

pinneapple_simulation.external_solvers.matlab.field_reader

Converte a saída `.mat` de um solver MATLAB em `PhysicalSample`. [sem teste]

`pinneapple_simulation/external_solvers/matlab/field_reader.py` · 61 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
mat_to_upd	função	(mat_path, *, field_keys=None, coord_keys=None, units=None)	Carrega um .mat e empacota seus arrays como PhysicalSample.

pinneapple_simulation.external_solvers.matlab.mat_io

Lê e grava arquivos .mat com scipy.io. [sem teste]

pinneapple_simulation/external_solvers/matlab/mat_io.py · 34 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
load_mat	função	(path) -> Dict[str, np.ndarray]	Carrega um .mat como dicionário de arrays numpy.
save_mat	função	(path, arrays) -> None	Grava um dicionário de arrays (ou escalares) num .mat.

pinneapple_simulation.external_solvers.matlab.runner

Roda scripts e funções MATLAB por subprocesso em modo -batch, trocando dados por .mat. [sem teste]

pinneapple_simulation/external_solvers/matlab/runner.py · 89 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
run_matlab_script	função	(script_path, *, matlab_executable='matlab', timeout=300, extra_flags=None) -> subprocess…)	Roda um script MATLAB por subprocesso em modo -batch.
run_matlab_function	função	(func_name, args, out_mat_path, *, matlab_executable='matlab', timeout=300) -> None	Chama uma função MATLAB por um script-invólucro gerado e grava a saída num .mat.

pinneapple_simulation.external_solvers.modelica

Pacote da integração Modelica: simulação de FMUs (fmpy), OpenModelica (OMPython ou omc direto), inspeção de FMUs, compatibilidade FMI e varreduras.

pinneapple_simulation/external_solvers/modelica/__init__.py · 35 linhas · testes: 5

pinneapple_simulation.external_solvers.modelica.fmi_compat

Roda FMUs por simulate_fmu em cada interface que oferecem (Co-Simulation, Model Exchange) e monta a tabela de compatibilidade pedida pela lista de ferramentas FMI.

pinneapple_simulation/external_solvers/modelica/fmi_compat.py · 122 linhas · testes: 2

nome	tipo	assinatura	o que faz
CompatRow	classe	(fmu, fmi_version, interface, exporter, platform, passed, detail)	Uma linha da tabela de compatibilidade FMI: FMU, versão, interface, exportador, plataforma, se passou e detalhe.
platform_tag	função	() -> str	Devolve a plataforma binária FMI desta máquina (por exemplo x86_64-linux).
check_fmu	função	(fmu, fmi_type, exporter='', stop_time=None, tol=None) -> CompatRow	Simula uma FMU numa interface e informa se funcionou.
check_fmus	função	(fmus, exporter='', interfaces=…) -> list[CompatRow]	Aplica check_fmu a cada FMU e a cada interface que ela implementa.
markdown_table	função	(rows) -> str	Monta tabelas Markdown por ferramenta exportadora e versão FMI, com células 'CS, ME'.

pinneapple_simulation.external_solvers.modelica.fmi_runner

Simula uma FMU com fmpy e devolve as saídas como arrays ou como PhysicalSample.

pinneapple_simulation/external_solvers/modelica/fmi_runner.py · 107 linhas · testes: 2

nome	tipo	assinatura	o que faz
FMUSimConfig	classe	(start_time, stop_time, step_size, output_interval, parameters, output_variables, fmi_typ…)	Configuração de uma simulação de FMU: tempos, passo, intervalo de saída, parâmetros, variáveis e tipo FMI.

nome	tipo	assinatura	o que faz
simulate_fmu	função	(fmu_path, config) -> Dict[str, np.ndarray]	Simula uma FMU com fmpy e devolve as variáveis de saída como arrays.
fmu_to_upd	função	(fmu_path, config)	Simula uma FMU e empacota o resultado como PhysicalSample.

pinneapple_simulation.external_solvers.modelica.fmu_inspect

Inspeção estática de um .fmu antes de usá-lo: versão FMI, interfaces, binários por plataforma, símbolos exportados de ELF64 contra os exigidos pelo padrão e consistência do modelDescription.

pinneapple_simulation/external_solvers/modelica/fmu_inspect.py · 233 linhas · testes: 6

nome	tipo	assinatura	o que faz
BinaryCheck	classe	(platform, path, format, exported, missing)	Resultado da checagem de um binário da FMU: plataforma, caminho, formato, funções exportadas e faltantes.
FMUReport	classe	(path, fmi_version, model_name, interfaces, binaries, problems, n_variables)	Relatório de inspeção de uma FMU: versão, modelo, interfaces, binários, problemas e número de variáveis.
elf_exported_functions	função	(data) -> List[str]	Lista as funções globais exportadas na tabela .dynsym de um objeto ELF64 little-endian.
inspect_fmu	função	(path) -> FMUReport	Inspeciona estaticamente um .fmu sem carregar nem executar código.

pinneapple_simulation.external_solvers.modelica.om_runner

Carrega e simula um modelo Modelica com OpenModelica via OMPython e empacota o resultado como PhysicalSample. [sem teste]

pinneapple_simulation/external_solvers/modelica/om_runner.py · 211 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
OMSimConfig	classe	(model_name, start_time, stop_time, number_of_intervals, tolerance, parameters, output_va…)	Configuração de uma simulação OpenModelica: modelo, tempos, intervalos, tolerância, parâmetros e saídas.
simulate_openmodelica	função	(model_file, config, *, working_dir=None) -> Dict[str, np.ndarray]	Carrega e simula um modelo Modelica com OpenModelica e devolve arrays por variável.
om_to_upd	função	(model_file, config, **sim_kwargs)	Simula com OpenModelica e devolve um PhysicalSample.

pinneapple_simulation.external_solvers.modelica.omc

Usa o compilador omc sem OMPython, local ou pela imagem Docker oficial, para gerar FMUs e simular arquivos .mo.

pinneapple_simulation/external_solvers/modelica/omc.py · 140 linhas · testes: 5

nome	tipo	assinatura	o que faz
omc_available	função	() -> str | None	Diz se o omc está disponível: 'local', 'docker' ou None.
run_omc_script	função	(script, workdir, timeout=1800.0) -> str	Roda um script .mos do omc numa pasta e devolve a saída, com RuntimeError em caso de erro.
build_fmu	função	(models, model_name, out_dir, fmi_version='2.0', fmu_type='cs', libraries=()) -> Path	Compila um modelo de arquivos .mo numa FMU.
simulate_mo	função	(models, model_name, stop_time=1.0, start_time=0.0, number_of_intervals=500, tolerance=1e…)	Simula um modelo com omc e devolve {variavel: array} incluindo time.

pinneapple_simulation.external_solvers.modelica.result_reader

Converte dicionários de resultado Modelica/FMI em PhysicalSample. [sem teste]

pinneapple_simulation/external_solvers/modelica/result_reader.py · 53 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
modelica_result_to_upd	função	(result, *, time_key='time', field_keys=None, units=None, source='modelica')	Empacota um dicionário de resultado Modelica como PhysicalSample.

pinneapple_simulation.external_solvers.modelica.sweep

Varreduras de parâmetros de uma FMU por hipercubo latino, gerando dados de treino e teste para surrogates de um modelo Modelica.

pinneapple_simulation/external_solvers/modelica/sweep.py · 86 linhas · testes: 5

nome	tipo	assinatura	o que faz
FMUDataset	classe	(names, params, time, outputs, bounds)	Dataset de uma varredura de FMU: nomes, parâmetros, tempo, saídas e limites, com split, save e load.
latin_hypercube	função	(n, d, seed=0) -> np.ndarray	Gera n pontos em [0, 1]^d por hipercubo latino.
fmu_sweep	função	(fmu, ranges, n, stop_time, output_interval, outputs, seed=0, fixed=None, start_time=0.0,…)	Simula n amostras de hipercubo latino dos intervalos de parâmetros numa grade de saída comum.

pinneapple_simulation.external_solvers.mujoco

Pacote da integração MuJoCo: carregamento de modelo, runner de trajetórias, rollouts paralelos MJX e adaptadores UPD. [sem teste]

pinneapple_simulation/external_solvers/mujoco/__init__.py · 53 linhas · **sem teste direto**

pinneapple_simulation.external_solvers.mujoco.adapter

Converte dicionários de trajetória MuJoCo em PhysicalSample. [sem teste]

pinneapple_simulation/external_solvers/mujoco/adapter.py · 78 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
trajectory_to_upd	função	(traj, field_map=None, meta_extra=None)	Converte um dicionário de trajetória MuJoCo num PhysicalSample.
trajectories_to_upd	função	(trajs, field_map=None, meta_extra=None) -> list	Converte uma lista de trajetórias MuJoCo em PhysicalSamples.

pinneapple_simulation.external_solvers.mujoco.loader

Configuração MuJoCo e funções para carregar o MjModel (arquivo ou XML) e alocar o MjData. [sem teste]

pinneapple_simulation/external_solvers/mujoco/loader.py · 92 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
MuJoCoConfig	classe	(model_path, model_xml, assets, dt, n_steps, n_substeps, gravity, extra_opts)	Configuração de uma simulação MuJoCo: modelo (arquivo ou XML), passo, número de passos, gravidade e opções.
load_model	função	(config)	Carrega o MjModel a partir da configuração.
make_data	função	(model)	Aloca um MjData novo para o modelo.

pinneapple_simulation.external_solvers.mujoco.mjx_backend

Backend MJX (JAX/GPU) que roda muitos ambientes MuJoCo em paralelo com jax.vmap para gerar datasets. Requer mujoco e jax. [sem teste]

pinneapple_simulation/external_solvers/mujoco/mjx_backend.py · 118 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`MJXParallelRunner`	classe	`(config, n_envs=256, device=None) -> None`	Roda N ambientes MuJoCo independentes em paralelo com MJX e `jax.vmap`, com saída em arrays ou UPD.

pinneapple_simulation.external_solvers.mujoco.runner

Roda um modelo MuJoCo com controlador opcional e coleta a trajetória (qpos, qvel etc.). [sem teste]

`pinneapple_simulation/external_solvers/mujoco/runner.py` · 137 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`MuJoCoRunner`	classe	`(config, controller=None) -> None`	Roda um modelo MuJoCo com controlador opcional e coleta trajetórias.

pinneapple_simulation.external_solvers.newton_physics

Pacote do backend de corpos rígidos NVIDIA Newton (XPBD, semi-implícito, Featherstone, MuJoCo-Warp) com checagens analíticas.

`pinneapple_simulation/external_solvers/newton_physics/__init__.py` · 28 linhas · testes: 8

pinneapple_simulation.external_solvers.newton_physics.runner

Integra uma `RigidScene` com um solver do Newton e grava a trajetória; velocidades iniciais com solvers de coordenadas de junta levantam `NotImplementedError`. [sem teste direto]

`pinneapple_simulation/external_solvers/newton_physics/runner.py` · 182 linhas · **sem teste direto** · palavras-chave no código (heurística; o texto acima diz o status real): levanta `NotImplementedError` em algum caminho

nome	tipo	assinatura	o que faz
`RigidTrajectory`	classe	`(names, t, pos, quat, vel, omega, mass, inertia, com, gravity, solver, dt, wall_time_s, m…)`	Estados gravados de uma simulação de corpos rígidos (posições, quatérnios, velocidades, massas, inércias) com busca de corpo por nome.
`simulate`	função	`(scene, duration, dt=0.001, solver='xpbd', iterations=8, record_every=10, device='cpu', l…)`	Integra a cena por um tempo com o solver escolhido e grava a trajetória a cada `record_every` passos.

pinneapple_simulation.external_solvers.newton_physics.scene

Descrição de cena de corpos rígidos (corpos, formas, juntas, chão) em Python puro, que só importa Newton/Warp ao chamar `build`.

`pinneapple_simulation/external_solvers/newton_physics/scene.py` · 280 linhas · testes: 8

nome	tipo	assinatura	o que faz
`Shape`	classe	`(kind, size, vertices, faces, boxes)`	Uma forma de colisão de um corpo no referencial do corpo (esfera, caixa, cápsula, cilindro, malha ou composta).
`Body`	classe	`(name, shape, pos, quat, vel, omega, density, mass, mu, restitution)`	Um corpo rígido da cena: forma, pose, velocidades iniciais, densidade ou massa, atrito e restituição.
`Joint`	classe	`(kind, parent, child, anchor, axis, damping)`	Uma junta entre dois corpos (revoluta ou fixa) com âncora, eixo e amortecimento.
`RigidScene`	classe	`(gravity, bodies, joints, ground_mu, ground_restitution)`	Cena de corpos, juntas e chão opcional, montada num modelo Newton só quando pedida.

pinneapple_simulation.external_solvers.newton_physics.validation

Checa um solver do Newton contra resultados fechados: queda livre, atrito, pêndulo, quique, giro livre e contato em repouso. [sem teste direto]

`pinneapple_simulation/external_solvers/newton_physics/validation.py` · 109 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
free_fall	função	(solver='xpbd', dt=0.001, **kw) -> dict	Checa queda livre de uma esfera contra z(t) = z0 - g t²/2.
sliding_friction	função	(solver='xpbd', dt=0.001, mu=0.4, v0=2.0, **kw) -> dict	Checa a distância de parada de uma caixa com atrito de Coulomb contra v0²/(2 mu g).
pendulum_period	função	(solver='xpbd', dt=0.001, length=1.0, amp_deg=5.0, **kw) -> dict	Checa o período de um pêndulo em pequena amplitude contra a fórmula fechada.
bounce	função	(solver='xpbd', dt=0.0001, e=0.5, **kw) -> dict	Checa a altura do primeiro quique contra e² vezes a altura de queda.
torque_free_spin	função	(solver='xpbd', dt=0.001, **kw) -> dict	Checa a conservação do momento angular de uma caixa girando sem gravidade.
resting_contact	função	(solver='xpbd', dt=0.001, **kw) -> dict	Checa que uma caixa em repouso no chão não afunda nem escorrega.
validate	função	(solver='xpbd', cases=None, log=print, **kw) -> list[dict]	Roda as checagens analíticas para um solver e devolve e registra uma tabela.

pinneapple_simulation.external_solvers.openfoam

Pacote OpenFOAM: montagem de casos, execução, amostragem, leitura binária, empacotamento UPD, arquivos `.splash` e campanhas. [sem teste direto do `__init__`]

pinneapple_simulation/external_solvers/openfoam/__init__.py · 8 linhas · **sem teste direto**

pinneapple_simulation.external_solvers.openfoam.binary_reader

Primitivas para ler arquivos FoamFile em formato binário (cabeçalho, listas, pontos, faces, internalField); formato ASCII ou tipos de lista não suportados levantam `NotImplementedError`. [sem teste direto]

pinneapple_simulation/external_solvers/openfoam/binary_reader.py · 254 linhas · **sem teste direto** · palavras-chave no código (heurística; o texto acima diz o status real): levanta `NotImplementedError` em algum caminho

nome	tipo	assinatura	o que faz
FoamFileHeader	classe	(klass, fmt, body_start)	Cabeçalho de um FoamFile: classe, formato e posição onde começa o corpo.
read_header	função	(data) -> FoamFileHeader	Localiza o fim do bloco `FoamFile {...}` e lê suas chaves.
read_binary_block	função	(data, pos, n_components, dtype=_SCALAR_DTYPE) -> Tuple[np.ndarray, int]	Lê um bloco binário `<count>(<bytes>)` a partir de uma posição.
read_label_list	função	(data, path_hint='') -> np.ndarray	Lê uma lista binária de rótulos (inteiros) de um FoamFile.
read_vector_field_points	função	(data, path_hint='') -> np.ndarray	Lê uma lista binária de pontos/vetores de um FoamFile.
read_face_compact_list	função	(data, path_hint='') -> Tuple[np.ndarray, np.ndarray]	Lê uma `faceCompactList` (offsets e índices de pontos).
read_internal_field	função	(data, path_hint='') -> Tuple[np.ndarray, bool, Optional[int]]	Lê o `internalField` de um campo volumétrico, uniforme ou não.

pinneapple_simulation.external_solvers.openfoam.campaign

Campanhas de parâmetros de rodadas OpenFOAM divididas entre várias máquinas: copia o caso base, aplica edições, roda com MPI, amostra linhas e grava um JSON por rodada, com trava por arquivo e publicação opcional por git.

pinneapple_simulation/external_solvers/openfoam/campaign.py · 686 linhas · testes: 22

nome	tipo	assinatura	o que faz
CampaignRun	classe	(id, params, mesh, worker, group)	Uma rodada da campanha: id, parâmetros, malha, worker e grupo.
Campaign	classe	(base, runs, name, solver, procs, end_time, pre, derived, edits, area_patch, setup, sampl…)	Plano de campanha (caso base, rodadas, solver, edições, amostragem, resultados) com leitura/escrita JSON, preparação e execução por worker.

nome	tipo	assinatura	o que faz
`GitPublisher`	classe	`(name=None, email=None, remote='origin', tries=6)`	Faz commit e push de um arquivo de resultado no branch atual, com rebase sobre o que outros workers enviaram.
`evaluate`	função	`(expr, names) -> float | list`	Avalia uma expressão aritmética restrita sobre nomes e funções de `math`.
`format_template`	função	`(template, values) -> str`	Formata um template restrito a `{nome}` e `{nome:spec}` numéricos, sem acesso a atributos.
`resolve`	função	`(params, derived=None) -> dict`	Devolve os parâmetros mais os valores derivados, na ordem dada.
`get_entry`	função	`(text, key) -> str`	Lê o valor de uma chave `a/b/c` num texto de dicionário OpenFOAM.
`set_entry`	função	`(text, key, value) -> str`	Define uma chave `a/b/c` num texto de dicionário OpenFOAM, criando níveis faltantes.
`edit_file`	função	`(path, key, value, root=None) -> None`	Define uma chave num arquivo de dicionário OpenFOAM, exigindo que o arquivo esteja dentro de `root` se dado.
`reset_nonuniform`	função	`(case, n_cells) -> list[str]`	Após troca de malha, reinicia campos de `0/` com número errado de valores para o valor mais comum.
`n_cells`	função	`(case) -> int`	Devolve o número de células da malha de um caso.
`area_patch`	função	`(case, patch, area, centre, normal, from_patches=None, rest='wall', plane_tol=0.03, bashr…)`	Reconstrói um patch de contorno com as faces de um plano mais próximas do centro até atingir a área pedida.
`write_decompose`	função	`(case, n, method='auto', bashrc=None) -> str`	Escreve o `decomposeParDict` para n processos (scotch se houver, senão `simple`).
`parse_solver_log`	função	`(text, every=200) -> dict`	Lê do log a saúde de uma rodada transiente: último tempo, Courant, limites de fração de fase, término e primeiro erro fatal.
`worker_commands`	função	`(plan, n, procs=4, publish_git=True) -> list[str]`	Gera uma linha de shell por máquina para rodar cada fatia da campanha.
`main`	função	`(argv=None) -> None`	Linha de comando do runner de campanhas (`--worker`, `--shard`, `--procs`, `--publish-git`, `--commands`).

`pinneapple_simulation.external_solvers.openfoam.case_builder`

Copia um caso-modelo OpenFOAM e escreve `0/T` e `transportProperties` de um cenário (problema térmico com obstáculo). [sem teste]

`pinneapple_simulation/external_solvers/openfoam/case_builder.py` · 105 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`OpenFOAMCaseTemplate`	classe	`(template_dir, obstacle_stl)`	Caso-modelo OpenFOAM (pasta e STL do obstáculo) com validação dos arquivos exigidos.
`stage_case_for_scenario`	função	`(*, tpl, out_case_dir, scenario) -> None`	Copia o caso-modelo para a pasta de saída e escreve `0/T` e `transportProperties` do cenário.

`pinneapple_simulation.external_solvers.openfoam.export_bundle`

Monta uma pasta-pacote copiando geometria, specs, pontos derivados e sensores amostrados; os nomes de arquivo estão fixos para um projeto de canal com cilindro. [sem teste]

`pinneapple_simulation/external_solvers/openfoam/export_bundle.py` · 38 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`export_bundle`	função	`(*, project_dir, bundle_dir) -> None`	Copia geometria, specs, pontos derivados e sensores de um projeto para uma pasta-pacote.

`pinneapple_simulation.external_solvers.openfoam.field_reader`

Lê o internalField de campos OpenFOAM (por padrão `p` e `U`) e empacota como `PhysicalSample`. [sem teste direto]

`pinneapple_simulation/external_solvers/openfoam/field_reader.py` · 173 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`openfoam_case_to_upd`	função	`(case_dir, *, time=None, fields=('p', 'U'))`	Lê o internalField dos campos pedidos num tempo e empacota como `PhysicalSample`.

`pinneapple_simulation.external_solvers.openfoam.mesh_reader`

Reconstrói centros e tamanhos de célula a partir de um polyMesh binário, no contrato de malha dos dataloaders de `pinneapple_data`. [sem teste direto]

`pinneapple_simulation/external_solvers/openfoam/mesh_reader.py` · 125 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`MeshGeometry`	classe	`(nodes, cell_size, cell_delta, bounds_min, bounds_max, n_points, n_faces, n_internal_face…)`	Geometria mínima de malha (centros, tamanhos de célula, limites, contagens) no contrato dos dataloaders.
`load_mesh`	função	`(source, prefix='constant/polyMesh/') -> MeshGeometry`	Constrói centros e tamanhos de célula a partir de um polyMesh binário (pasta ou arquivo).

`pinneapple_simulation.external_solvers.openfoam.runner`

Executa um caso OpenFOAM via bash (blockMesh, snappyHexMesh opcional, solver serial ou MPI, comandos extras). [sem teste]

`pinneapple_simulation/external_solvers/openfoam/runner.py` · 43 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`OpenFOAMRunConfig`	classe	`(foam_bashrc, solver, n_procs, use_snappy, extra_cmds)`	Como rodar um caso OpenFOAM: bashrc, solver, número de processos, snappyHexMesh e comandos extras.
`run_openfoam_case`	função	`(case_dir, cfg) -> None`	Roda malha, solver (serial ou MPI) e comandos extras num caso OpenFOAM.

`pinneapple_simulation.external_solvers.openfoam.sampling`

Escreve um `sampleDict` de nuvem de pontos, roda `postProcess -func sample` e lê os `.raw` resultantes em DataFrame. [sem teste]

`pinneapple_simulation/external_solvers/openfoam/sampling.py` · 99 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`write_sample_dict_cloud`	função	`(case_dir, *, set_name, points, fields) -> Path`	Escreve `system/sampleDict` para amostragem em nuvem nos pontos dados.
`run_sampling`	função	`(case_dir, foam_bashrc, *, time=None) -> None`	Roda `postProcess -func sample` (último tempo por padrão).
`read_sampled_scalar_field`	função	`(case_dir, *, set_name, field) -> pd.DataFrame`	Lê um campo escalar amostrado (`x y z valor`) em DataFrame.

`pinneapple_simulation.external_solvers.openfoam.splash_packer`

Empacota um caso OpenFOAM terminado num arquivo `.splash` (zip + `splash-manifest.json` com tamanho e sha256 por arquivo), lê o manifesto e verifica a integridade.

`pinneapple_simulation/external_solvers/openfoam/splash_packer.py` · 138 linhas · testes: 9

nome	tipo	assinatura	o que faz
`pack_splash_archive`	função	`(case_dir, out_path, *, case_name=None, summary=None, generated_by=…, splash_version='1.0…)`	Compacta um caso OpenFOAM terminado num `.splash` com manifesto de tamanhos e hashes.
`read_splash_manifest`	função	`(splash_path) -> Dict[str, Any]`	Lê só o `splash-manifest.json` de um `.splash`.
`verify_splash_archive`	função	`(splash_path) -> Iterable[str]`	Lista os arquivos cujo tamanho ou sha256 não bate com o manifesto (vazio se íntegro).

pinneapple_simulation.external_solvers.openradioss

Ponte para o OpenRadioss (FEA explícito): edição de deck para DoE, execução via Docker e leitores de resultados ANIM->VTK.

`pinneapple_simulation/external_solvers/openradioss/__init__.py` · 18 linhas · testes: 2

pinneapple_simulation.external_solvers.openradioss.deck

Edita decks OpenRadioss: lê e troca espessuras de `/PROP/SHELL`, escreve o engine com saída ANIM e prepara a pasta de um caso. [sem teste]

`pinneapple_simulation/external_solvers/openradioss/deck.py` · 124 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`read_shell_thickness`	função	`(deck_text, prop_id) -> float`	Lê a espessura `Thick` de um `/PROP/SHELL/<id>`.
`set_shell_thickness`	função	`(deck_text, thickness_by_prop) -> str`	Devolve uma cópia do deck com as espessuras trocadas por id de propriedade.
`write_engine_anim`	função	`(path, run_name, t_end, anim_dt, *, extra_anim=('/ANIM/VECT/DISP',), header='', tail='') …)`	Escreve um arquivo engine que pede saída ANIM em vez de H3D.
`stage_case`	função	`(template_starter, engine_file, out_dir, thickness_by_prop, *, extra_files=None) -> Path`	Cria a pasta de um caso com o starter de espessura alterada e o engine original.

pinneapple_simulation.external_solvers.openradioss.log_reader

Lê do log do engine os erros de balanço de energia (`ERR=...%`).

`pinneapple_simulation/external_solvers/openradioss/log_reader.py` · 38 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`read_energy_errors`	função	`(case_dir_or_log) -> np.ndarray`	Devolve cada `ERR=...%` impresso pelo engine, em ordem.
`read_energy_error`	função	`(case_dir_or_log) -> float`	Devolve o maior |ERR| de balanço de energia do log, ou `nan`.

pinneapple_simulation.external_solvers.openradioss.runner

Roda starter, engine e conversão ANIM->VTK do OpenRadioss via Docker, um caso ou vários em paralelo limitado. [sem teste]

`pinneapple_simulation/external_solvers/openradioss/runner.py` · 111 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`OpenRadiossDockerConfig`	classe	`(openradioss_dir, image, arch, threads, docker_bin, extra_path)`	Configuração da execução via Docker: pasta do OpenRadioss, imagem, arquitetura, threads e binário do docker.
`run_openradioss_case`	função	`(case_dir, run_name, cfg, *, starter=None, engine=None) -> Path`	Roda um caso preparado (starter, engine e conversão) e levanta erro se o solver falhar.
`run_openradioss_cases`	função	`(case_dirs, run_name, cfg, *, max_parallel=4, on_done=None) -> List[Path]`	Roda vários casos com paralelismo limitado, um solver de uma thread por caso.
`docker_available`	função	`(cfg) -> bool`	Diz se o Docker configurado responde.

pinneapple_simulation.external_solvers.openradioss.vtk_reader

Lê resultados OpenRadioss convertidos por `anim_to_vtk` (VTK legado ASCII) e séries de deslocamento. [sem teste]

`pinneapple_simulation/external_solvers/openradioss/vtk_reader.py` · 73 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`RadiossMesh`	classe	`(points, node_ids, cells, cell_types, part_ids, time)`	Malha lida de um VTK do OpenRadioss: pontos, ids de nós, células, tipos, partes e tempo.

nome	tipo	assinatura	o que faz
`read_vtk_mesh`	função	`(path) -> RadiossMesh`	Lê um arquivo VTK legado ASCII convertido por `anim_to_vtk`.
`read_case_displacements`	função	`(frames_dir) -> Tuple[np.ndarray, np.ndarray]`	Devolve tempos e deslocamentos (T, N, 3) a partir de arquivos `disp_XXX.txt`/`time_XXX.txt`.

pinneapple_simulation.external_solvers.turbodesigner

Pacote da ponte TurboDesigner (análise de linha média de compressor axial). [sem teste]

`pinneapple_simulation/external_solvers/turbodesigner/__init__.py` · 43 linhas · **sem teste direto**

pinneapple_simulation.external_solvers.turbodesigner.bridge

Envolve o pacote `turbodesigner` para obter campos por estação de um compressor axial como fonte de dados para PINN; sem o pacote, usa um cálculo analítico simplificado (`fallback_if_missing=True`). [sem teste]

`pinneapple_simulation/external_solvers/turbodesigner/bridge.py` · 400 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`TurboDesignerConfig`	classe	`(pressure_ratio, num_stages, mass_flow_rate, rpm, inlet_total_pressure, inlet_total_tempe…)`	Parâmetros de um compressor axial no TurboDesigner (razão de pressão, estágios, vazão, rotação, condições de entrada).
`TurboDesignerWorkflow`	classe	`(base_config) -> None`	Atalho para solves repetidos do TurboDesigner sobre uma configuração base, com `solve`, `solve_upd` e `sweep`.
`run_turbodesigner`	função	`(cfg, *, fallback_if_missing=True) -> Dict[str, np.ndarray]`	Roda a análise de linha média e devolve arrays por estação; sem o pacote, usa o cálculo analítico simplificado.
`turbodesigner_to_upd`	função	`(cfg, **run_kwargs)`	Roda o TurboDesigner e empacota o resultado como `PhysicalSample`.

pinneapple_simulation.flight

Dinâmica de voo: aeronave rígida 6-DOF com aerodinâmica de vortex lattice, empuxo, gravidade e trem de pouso, com trim, derivadas de estabilidade, piloto/piloto automático, decolagem, turbulência e cenas 3-D.

`pinneapple_simulation/flight/__init__.py` · 59 linhas · testes: 9

pinneapple_simulation.flight.aero

Aerodinâmica quase estacionária da aeronave inteira a partir do vortex lattice de `pinneapple_design.aero.vlm`, com superfícies de controle, deriva vertical e correções de fuselagem (Raymer). [sem teste direto]

`pinneapple_simulation/flight/aero.py` · 239 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ControlSurface`	classe	`(name, surface, eta_in, eta_out, cf_c, antisymmetric, tau)`	Painéis do lattice movidos por um comando (asa ou cauda, faixa de envergadura, razão de corda, simétrico ou antissimétrico).
`Fin`	classe	`(area, height, ac, rudder_cf_c, sweep_deg, stall_deg)`	Deriva vertical modelada como uma linha sustentadora: área, altura, centro aerodinâmico, leme e enflechamento.
`AeroModel`	classe	`(af, cg, cd0, fin, controls, cma_fus, cnb_fus, cd_gear, cd_flap_per_rad, nc, ns_wing, ns_…)`	Modelo aerodinâmico da aeronave, construído a partir do airframe, que dá forças e momentos no CG em eixos do corpo.
`flap_effectiveness`	função	`(cf_c, correction=0.85) -> float`	Eficácia de um flap simples pela teoria de perfil fino, vezes uma correção empírica.
`helmbold_lift_slope`	função	`(aspect_ratio, sweep_half_chord_deg=0.0, eta=0.95) -> float`	Inclinação da curva de sustentação de uma asa finita pela forma de Helmbold/DATCOM (Raymer).
`fuselage_munk_factor`	função	`(wing_qc_fraction) -> float`	Fator K_fus de Raymer em função da posição da asa ao longo da fuselagem.
`fuselage_moments`	função	`(length, max_width, volume, wing_area, mac, span, depth, wing_qc_fraction)`	Contribuições da fuselagem a Cm_alpha e Cn_beta pelas equações de Raymer.

pinneapple_simulation.flight.aircraft

A aeronave como corpo rígido 6-DOF: propriedades de massa, aerodinâmica, motores, trem de pouso e controles, com a avaliação de forças e a derivada do estado.

pinneapple_simulation/flight/aircraft.py · 155 linhas · testes: 9

nome	tipo	assinatura	o que faz
Controls	classe	(elevator, aileron, rudder, flap, throttle, steer, brake, gear_down)	Posições dos comandos (profundor, aileron, leme, flap, manete, direção, freio, trem) com convenção de sinais.
Aircraft	classe	(name, af, aero, mass, cg, inertia, engines, gear, cabin, tail_point, wingtip, clmax_take…)	A aeronave completa: airframe, aerodinâmica, massa, CG, inércia, motores, trem, pontos de cabine e limites de sustentação, com avaliação de forças.

pinneapple_simulation.flight.catalog

Quatro aeronaves prontas (light_ga, regional_turboprop, a320, motor_glider) montadas com a geometria paramétrica e os modelos de peso e arrasto da biblioteca; números publicados e premissas ficam em Aircraft.info, nada ajustado a ensaio em voo.

pinneapple_simulation/flight/catalog.py · 383 linhas · testes: 9

nome	tipo	assinatura	o que faz
ExplicitAirframe	classe	(wing_z, htail_spec, vtail_spec)	Um Airframe com altura da asa e caudas dadas explicitamente (por exemplo asa alta com cauda em T).
profile_cd0	função	(re, tc) -> float	Arrasto de perfil por atrito de placa plana turbulenta vezes fator de forma de espessura.
light_ga	função	() -> Aircraft	Constrói a aeronave leve monomotor a pistão (classe Cessna 172).
regional_turboprop	função	() -> Aircraft	Constrói o turboélice regional bimotor de asa alta (classe ATR 72-600).
a320	função	() -> Aircraft	Constrói o jato bimotor de corredor único (classe A320).
motor_glider	função	() -> Aircraft	Constrói o motoplanador de grande alongamento.
build	função	(name) -> Aircraft	Constrói uma aeronave do catálogo pelo nome.
names	função	() -> list[str]	Lista os nomes das aeronaves do catálogo.

pinneapple_simulation.flight.control

Modelos de piloto e piloto automático: atitude de arfagem, altitude, velocidade, asas niveladas/proa, amortecedor de guinada e o cronograma de decolagem.

pinneapple_simulation/flight/control.py · 221 linhas · testes: 9

nome	tipo	assinatura	o que faz
LoopDesign	classe	(omega_pitch, zeta_pitch, omega_roll, zeta_roll, omega_yaw, zeta_yaw, pitch_integral, q_f…)	Parâmetros de projeto das malhas de controle (frequências e amortecimentos de arfagem, rolamento e guinada).
InnerLoops	classe	(ac, derivs, design=None)	Malhas internas de atitude de uma aeronave, que dão incrementos de profundor, aileron e leme.
StickFixed	classe	(controls)	Controlador que mantém os comandos congelados nos valores de trim.
Autopilot	classe	(ac, trim, derivs, design=None, altitude=None, heading=None, speed=None, omega_h=0.25, ta…)	Piloto automático com manutenção de altitude, asas niveladas/proa, amortecedor de guinada e manete automática.
TakeoffSchedule	classe	(V_R, theta_rotation, rotation_rate, V_climb, theta_max, theta_min, runway_heading, flap,…)	Cronograma de decolagem: V_R, atitude e taxa de rotação, velocidade de subida, limites de arfagem, proa da pista, flap e manete.

nome	tipo	assinatura	o que faz
`TakeoffPilot`	classe	`(ac, sched, derivs, design=None)`	Piloto de decolagem: manete total e flap, mantém o eixo da pista no solo, roda em V_R e sobe na velocidade alvo.
`wrap`	função	`(a) -> float`	Normaliza um ângulo para o intervalo [-pi, pi).

pinneapple_simulation.flight.encounter

Encontros com turbulência e rajadas em voo e estatísticas de fator de carga, com comparação à fórmula de Pratt-Walker. [sem teste direto]

`pinneapple_simulation/flight/encounter.py` · 140 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`gust_sensitivity`	função	`(ac, V, rho, cl_alpha) -> float`	Sensibilidade dn/dw_rajada de uma asa fixa em escoamento quase estacionário.
`stall_angle`	função	`(ac, trim_alpha, trim_CL, cl_alpha) -> float`	Ângulo de ataque em que a sustentação linear atinge o CLmax limpo da aeronave.
`encounter_metrics`	função	`(ac, hist, t_skip=5.0, alpha_stall=None) -> dict[str, Any]`	Estatísticas de um encontro após um tempo inicial, opcionalmente checando o tempo acima do estol.
`turbulence_encounter`	função	`(ac, V, altitude, spec=None, wind=None, duration=60.0, dt=0.01, controller='autopilot', f…)`	Voa uma aeronave por turbulência durante um tempo, com piloto automático ou comandos fixos.
`discrete_gust`	função	`(ac, V, altitude, U, H=None, duration=None, dt=0.005)`	Resposta com comandos fixos a uma rajada vertical 1-cosseno, devolvendo histórico e pico de dn.
`pratt_check`	função	`(ac, V, altitude, U_de) -> dict[str, float]`	Compara o pico de dn simulado de uma rajada 1-cosseno com a fórmula de Pratt-Walker.

pinneapple_simulation.flight.figures

Figuras estáticas (PNG, matplotlib) dos resultados: espectros de turbulência, trajetórias de decolagem, vento cruzado, fatores de carga e checagem de sensibilidade a rajada.

`pinneapple_simulation/flight/figures.py` · 212 linhas · testes: 9

nome	tipo	assinatura	o que faz
`psd_figure`	função	`(curves, path) -> str`	Figura com um painel por espectro: medido contra o alvo.
`takeoff_figure`	função	`(runs, metrics, path) -> str`	Figura de decolagem: altura da roda e velocidade contra distância, arfagem contra tempo.
`lateral_figure`	função	`(runs, path, title='Crosswind takeoff') -> str`	Figura de desvio lateral, ângulo de rolamento e leme contra tempo (vento cruzado).
`encounter_traces_figure`	função	`(runs, path, window=20.0, t0=10.0, notes=None) -> str`	Pequenos múltiplos de incremento de fator de carga e rolamento, uma linha por aeronave.
`encounter_bars_figure`	função	`(table, path) -> str`	Barras de RMS do fator de carga no CG e na cabine por aeronave.
`gust_sensitivity_figure`	função	`(table, pratt, path) -> str`	Figura que compara RMS medido com a previsão quase estacionária e o pico de rajada com Pratt-Walker.
`table_markdown`	função	`(rows, cols, digits=None) -> str`	Monta uma tabela Markdown a partir de linhas em dicionário.

pinneapple_simulation.flight.gear

Contato do trem de pouso com pista plana: pernas mola-amortecedor com rolamento, frenagem e força lateral do pneu. [sem teste direto]

`pinneapple_simulation/flight/gear.py` · 96 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
GearLeg	classe	(name, contact, stiffness, damping, mu_roll, mu_brake, mu_side, slip_ref, steerable, brak…)	Uma perna do trem de pouso: ponto de contato, rigidez, amortecimento, coeficientes de atrito, direção e freio.
size_gear	função	(legs, mass, cg, static_compression=0.25, zeta=0.6, g=9.80665) -> list[GearLeg]	Dimensiona mola e amortecedor de cada perna para uma compressão estática dada sob sua parcela do peso.
gear_forces	função	(legs, x, C, cg, steer=0.0, brake=0.0, ground_z=0.0)	Forças, momentos e cargas normais de todas as pernas para um estado da aeronave.

pinneapple_simulation.flight.mass

Massa, centro de gravidade e tensor de inércia a partir de uma lista de componentes (ponto, barra, placa, cilindro). [sem teste direto]

pinneapple_simulation/flight/mass.py · 83 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
Component	classe	(name, mass, center, inertia)	Um componente de massa com centro e inércia própria.
point	função	(name, mass, center) -> Component	Cria um componente de massa pontual.
rod	função	(name, mass, center, length, axis=0) -> Component	Cria um componente barra fina ao longo de um eixo.
plate	função	(name, mass, center, span, chord, taper=1.0) -> Component	Cria uma placa fina no plano x-y (asa, estabilizador), opcionalmente afilada.
fin_plate	função	(name, mass, center, height, chord) -> Component	Cria uma placa fina no plano x-z (deriva vertical).
cylinder	função	(name, mass, center, length, radius, shell=False) -> Component	Cria um cilindro ao longo de x (fuselagem, nacele), maciço ou casca fina.
mass_properties	função	(components) -> tuple[float, np.ndarray, np.ndarray]	Calcula massa total, CG e inércia em torno do CG.
balance	função	(components, x_target, movable) -> list[Component]	Desloca um componente ao longo de x para pôr o CG numa posição alvo.

pinneapple_simulation.flight.propulsion

Motores: empuxo de turbofan e de hélice (pistão ou turboélice) contra velocidade e altitude, e a direção do empuxo. [sem teste direto]

pinneapple_simulation/flight/propulsion.py · 74 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
Turbofan	classe	(static_thrust, position, cant_deg, name)	Turbofan de alto bypass com queda de empuxo instalado com velocidade e altitude.
Propeller	classe	(power, diameter, efficiency, figure_of_merit, kind, position, cant_deg, name)	Hélice movida por pistão ou turboélice: potência com a altitude, empuxo estático e empuxo em voo.
thrust_direction_body	função	(cant_deg) -> np.ndarray	Vetor unitário do empuxo em eixos do corpo para um ângulo de inclinação do motor.

pinneapple_simulation.flight.rigid_body

Equações 6-DOF de corpo rígido em eixos do corpo com quatérnio de atitude, conversões de ângulos e passo RK4.

pinneapple_simulation/flight/rigid_body.py · 116 linhas · testes: 9

nome	tipo	assinatura	o que faz
quat_multiply	função	(a, b) -> np.ndarray	Produto de dois quatérnios.
quat_to_dcm	função	(q) -> np.ndarray	Matriz de rotação corpo -> NED a partir do quatérnio.

nome	tipo	assinatura	o que faz
`euler_to_quat`	função	`(phi, theta, psi) -> np.ndarray`	Converte rolamento, arfagem e guinada (3-2-1) em quatérnio.
`quat_to_euler`	função	`(q) -> tuple[float, float, float]`	Converte quatérnio em ângulos de Euler, tratando o gimbal lock.
`state_vector`	função	`(pos=(0, 0, 0), vel_body=(0, 0, 0), euler=(0, 0, 0), omega=(0, 0, 0)) -> np.ndarray`	Monta o vetor de estado de 13 componentes (posição, velocidade, quatérnio, rotação).
`rigid_body_derivative`	função	`(x, F_b, M_b, mass, inertia, inertia_inv=None, k_norm=1.0) -> np.ndarray`	Derivada temporal do estado de 13 componentes para força e momento totais no corpo.
`gravity_body`	função	`(q, mass, g=G0) -> np.ndarray`	Força peso em eixos do corpo.
`rk4_step`	função	`(f, t, x, dt) -> np.ndarray`	Um passo de Runge-Kutta clássico com renormalização do quatérnio.
`kinetic_energy`	função	`(x, mass, inertia) -> float`	Energia cinética de translação e rotação do estado.
`angular_momentum_ned`	função	`(x, inertia) -> np.ndarray`	Momento angular em torno do CG em eixos NED.

`pinneapple_simulation.flight.simulate`

Integração no tempo de uma `Aircraft` com controlador e campo de vento, gravando o histórico de voo, e checagem de energia do integrador.

`pinneapple_simulation/flight/simulate.py` · 165 linhas · testes: 9

nome	tipo	assinatura	o que faz
`FlightHistory`	classe	`(aircraft, t, x, euler, airspeed, alpha, beta, CL, nz, ny, f_spec, thrust, gear_loads, co…)`	Histórico gravado de um voo (estados, ângulos, velocidades, fatores de carga, empuxo, cargas do trem, comandos, vento, eventos) com resumo.
`simulate`	função	`(ac, x0, controller, t_end, dt=0.01, wind=None, record_every=1, record_gamma=False, stop=…)`	Integra a aeronave no tempo com um controlador e um vento opcional e devolve o `FlightHistory`.
`energy_check`	função	`(ac, x0, t_end=5.0, dt=0.01) -> dict[str, float]`	Checa conservação de energia e momento do integrador com aerodinâmica, motores e trem desligados.
`rms`	função	`(a) -> float`	Valor RMS de um array.

`pinneapple_simulation.flight.takeoff`

Decolagem: corrida, rotação em V_R, saída do chão e subida até a barreira de 35 ft, com a referência fechada de Raymer.

`pinneapple_simulation/flight/takeoff.py` · 184 linhas · testes: 9

nome	tipo	assinatura	o que faz
`default_schedule`	função	`(ac, kind='transport') -> TakeoffSchedule`	Cronograma de decolagem padrão a partir da velocidade de estol com flap de decolagem.
`takeoff_derivatives`	função	`(ac, V) -> dict[str, float]`	Derivadas de controle em torno da subida com flap de decolagem, para projetar os ganhos do piloto.
`simulate_takeoff`	função	`(ac, schedule=None, wind=None, t_max=90.0, dt=0.01, h_stop=400 * FT, design=None, record_…)`	Simula a decolagem completa da aeronave até uma altura de parada.
`takeoff_metrics`	função	`(ac, hist) -> dict[str, Any]`	Extrai do histórico as métricas de decolagem (distâncias, velocidades, tempos).
`raymer_takeoff`	função	`(ac, V_LOF=None, mu=0.02, t_rotation=3.0, h_obstacle=SCREEN, theta_ground=0.0) -> dict[st…)`	Distância de decolagem pela forma fechada de Raymer com o empuxo, sustentação e arrasto desta aeronave.
`raymer_variants`	função	`(ac, metrics, kind='light') -> dict[str, dict[str, float]]`	As duas referências fechadas de Raymer citadas nos resultados de decolagem.

pinneapple_simulation.flight.trim

Trim em voo reto, equilíbrio no solo, derivadas de estabilidade e modos dinâmicos (autovalores da linearização) de uma `Aircraft`.

`pinneapple_simulation/flight/trim.py` · 190 linhas · testes: 9

nome	tipo	assinatura	o que faz
`TrimResult`	classe	`(V, altitude, gamma, alpha, theta, elevator, throttle, x, controls, residual, CL, feasibl…)`	Resultado de trim: velocidade, altitude, ângulos, profundor, manete, estado, resíduo e viabilidade.
`trim`	função	`(ac, V, altitude, gamma=0.0, flap=0.0, gear_down=False, psi=0.0) -> TrimResult`	Acha o trim de voo reto e nivelado (ou em subida) com asas niveladas.
`ground_equilibrium`	função	`(ac, flap=0.0, throttle=0.0, psi=0.0, x_pos=0.0, y_pos=0.0) -> np.ndarray`	Estado de repouso sobre o trem: altura e arfagem com força vertical e momento nulos.
`stability_derivatives`	função	`(ac, tr, d_angle=math.radians(0.5), d_rate=0.01, d_ctl=math.radians(1.0)) -> dict[str, fl…)`	Derivadas aerodinâmicas em torno do trim por diferenças finitas.
`modes`	função	`(ac, tr, eps=1e-05) -> dict[str, object]`	Autovalores da linearização de 12 estados em torno do trim, com comandos fixos.
`classify_modes`	função	`(eigs, tol=1e-06) -> dict[str, dict[str, float]]`	Nomeia os pares complexos de autovalores por frequência, amortecimento e período.

pinneapple_simulation.flight.turbulence

Turbulência atmosférica e rajadas (MIL-F-8785C / MIL-HDBK-1797): especificações, espectros, filtros de Dryden, campos congelados 1-D e 3-D (von Karman), vento com cisalhamento e rajada 1-cosseno. `WindField.velocity` é interface abstrata (`NotImplementedError`).

`pinneapple_simulation/flight/turbulence.py` · 476 linhas · testes: 19 · palavras-chave no código (heurística; o texto acima diz o status real): levanta `NotImplementedError` em algum caminho

nome	tipo	assinatura	o que faz
`TurbulenceSpec`	classe	`(sigma_u, sigma_v, sigma_w, L_u, L_v, L_w, model, altitude, severity)`	Intensidades RMS e comprimentos de escala das três componentes da turbulência e a forma espectral.
`WindField`	classe	`()`	Interface de campo de vento: `velocity(t, pontos)` em NED; soma com `+`.
`SumWind`	classe	`(fields)`	Soma de vários campos de vento.
`ConstantWind`	classe	`(ned)`	Vento uniforme e constante.
`ShearWind`	classe	`(w20, from_deg, z0, ground_z)`	Vento médio com perfil logarítmico MIL-F-8785C e direção meteorológica.
`FrozenLine`	classe	`(spec, length, dx, heading_deg, origin, seed, series)`	Uma realização congelada das três componentes ao longo de uma linha reta.
`DrydenFilter`	classe	`(spec, V, dt, seed=0)`	Filtros de forma de Dryden (MIL-HDBK-1797) excitados por ruído branco, para um ponto voando a velocidade V.
`FrozenField3D`	classe	`(spec, n_shells, n_dirs, k_min, k_max, dx_min, heading_deg, seed)`	Turbulência 3-D isotrópica e sem divergência de von Karman por modos de Fourier aleatórios.
`DiscreteGust`	classe	`(amplitude, length, start, shape, direction, heading_deg, origin)`	Rajada discreta 1-cosseno (ou rampa) encontrada quando a aeronave passa um ponto do trajeto.
`turbulence_spec`	função	`(altitude, severity='moderate', model='von_karman', *, w20=None, probability=None) -> Tur…)`	Intensidades e escalas da turbulência numa altitude para uma severidade e modelo.
`psd`	função	`(spec, Omega, component='w') -> np.ndarray`	PSD espacial unilateral de uma componente numa frequência espacial.
`psd_temporal`	função	`(spec, omega, V, component='w') -> np.ndarray`	PSD temporal unilateral vista a uma velocidade V (campo congelado).
`welch_psd`	função	`(x, d, nseg=16)`	PSD unilateral de um sinal amostrado pelo método de Welch.
`path_axes`	função	`(heading_deg) -> np.ndarray`	Vetores unitários u, v, w do trajeto em NED para uma proa.

nome	tipo	assinatura	o que faz
`von_karman_energy`	função	`(k, L, sigma=1.0)`	Espectro de energia isotrópico de von Karman normalizado.
`isotropic_line_spectra`	função	`(k1, L, sigma=1.0)`	Espectros longitudinal e transversal 1-D do campo isotrópico, por quadratura.
`pratt_gust_load`	função	`(wing_loading, cl_alpha, mac, U_de, V_e, rho0=1.225, rho=1.225, g=9.80665)`	Incremento de fator de carga de rajada pela fórmula de Pratt-Walker.

pinneapple_simulation.flight.verification

Verificação dos blocos da dinâmica de voo como números citáveis: turbulência, trim, conservação e lattice. [sem teste direto]

`pinneapple_simulation/flight/verification.py` · 142 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`band_ratios`	função	`(W, P, target, lo, hi, nbins=8)`	Razões medido/alvo de PSD médias por banda de frequência.
`turbulence_checks`	função	`(seed=3) -> dict[str, Any]`	Checa variância e PSD de cada gerador de turbulência contra o alvo.
`trim_checks`	função	`(names=None, hold=10.0) -> list[dict[str, Any]]`	Checa resíduo de trim e deriva de velocidade, altitude e fator de carga com comandos fixos em cada aeronave.
`conservation_check`	função	`() -> dict[str, Any]`	Checa erro de energia e momento angular do integrador num corpo girando sob gravidade.
`lattice_check`	função	`() -> list[dict[str, Any]]`	Compara a sustentação do modelo com lattice no laço com o vortex lattice estacionário.
`run`	função	`() -> dict[str, Any]`	Roda todas as verificações e devolve um dicionário.

pinneapple_simulation.flight.visuals

Cenas 3-D de um voo (aeronave na trajetória, pista, rajadas como setas, carga do lattice e HUD) como Scene de `pinneapple_tools.visualization.studio`.

`pinneapple_simulation/flight/visuals.py` · 236 linhas · testes: 9

nome	tipo	assinatura	o que faz
`body_vertices`	função	`(ac, V_airframe) -> np.ndarray`	Converte pontos em eixos do airframe para o referencial do corpo em torno do CG.
`aircraft_surfaces`	função	`(ac, group='aircraft', parts=None)`	Geometria visual da aeronave como superfícies do studio no referencial do corpo.
`lattice_surface`	função	`(ac, group='aircraft', lift=0.6)`	Painéis do vortex lattice como uma folha acima da superfície, para colorir pela carga.
`runway_surfaces`	função	`(length=3000.0, width=45.0, x0=-300.0, grass=1500.0)`	Superfícies da pista com marcas e grama ao redor.
`frame_indices`	função	`(hist, n_frames) -> np.ndarray`	Índices do histórico usados como quadros da animação.
`flight_scene`	função	`(ac, hist, *, n_frames=240, wind=None, runway=True, runway_length=None, gust_grid=7, gust…)`	Cena 3-D animada de um voo, com pista, rajadas, carga e HUD opcionais.
`gust_wisps`	função	`(ac, hist, wind, n_lat=5, n_vert=3, spacing=None, exaggeration=0.6, ahead=0.15)`	Linhas estáticas que mostram o campo de rajada congelado ao longo do trajeto.
`scene_summary`	função	`(sc) -> dict[str, Any]`	Resume uma cena gerada: número de superfícies, de triângulos e de quadros da animação.

pinneapple_simulation.geophysics

Blocos do sistema terrestre verificados contra soluções exatas ou testes padrão: águas rasas na esfera, equação de Richards e modelo de balanço de energia de duas camadas.

`pinneapple_simulation/geophysics/__init__.py` · 40 linhas · testes: 11

pinneapple_simulation.geophysics.energy_balance

Modelo de balanço de energia de duas camadas do aquecimento médio global (Held et al. 2010; Geoffroy et al. 2013), com integração exata, resposta ao degrau, ECS/TCR e ajuste de parâmetros. [sem teste direto]

`pinneapple_simulation/geophysics/energy_balance.py` · 136 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`TwoLayerEBM`	classe	`(lam, gamma, C, C0, eps)`	Modelo de duas camadas (oceano superior e profundo) com simulação, modos, resposta ao degrau, ECS e TCR.
`fit_two_layer`	função	`(T_obs, forcing, dt=1.0, fit_eps=False, guess=None, sigma=None) -> dict[str, object]`	Ajuste por mínimos quadrados dos parâmetros a uma série de aquecimento forçada.

pinneapple_simulation.geophysics.richards

Fluxo não saturado no solo: equação de Richards 1-D (forma mista) com os modelos de retenção e condutividade clássicos. [sem teste direto]

`pinneapple_simulation/geophysics/richards.py` · 316 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`VanGenuchten`	classe	`(theta_r, theta_s, alpha, n, Ks, l)`	Solo de van Genuchten-Mualem: saturação efetiva, umidade, condutividade e capacidade.
`BrooksCorey`	classe	`(theta_r, theta_s, h_b, lam, Ks)`	Solo de Brooks-Corey.
`ClappHornberger`	classe	`(theta_s, psi_s, b, Ks, theta_r)`	Solo de Clapp-Hornberger (mesmas leis de potência de Campbell; `Campbell` é um alias).
`Gardner`	classe	`(theta_r, theta_s, alpha, Ks)`	Solo de Gardner (condutividade exponencial), que tem perfil estacionário exato.
`Boundary`	classe	`(kind, value)`	Condição de contorno: carga, fluxo ou drenagem livre, com valor constante ou função do tempo.
`RichardsResult`	classe	`(z, t, h, theta, flux_top, flux_bottom, mass_balance_ratio, n_steps, n_picard)`	Resultado: perfis de carga e umidade, fluxos no topo e no fundo, balanço de massa e contagens de passos.
`solve_richards`	função	`(soil, depth, n_cells, h0, t_end, top, bottom, dt0=1.0, dt_min=1e-06, dt_max=None, t_out=…)`	Resolve uma coluna de solo no tempo com passo adaptativo e iteração de Picard.
`gardner_steady_infiltration`	função	`(soil, z_above_table, q0) -> np.ndarray`	Perfil de carga estacionário exato acima do lençol para o solo de Gardner.

pinneapple_simulation.geophysics.sphere_shallow_water

Equações de águas rasas na esfera em rotação: solver espectral (vorticidade-divergência, grade gaussiana) e os casos de teste de Williamson et al. (1992).

`pinneapple_simulation/geophysics/sphere_shallow_water.py` · 298 linhas · testes: 11

nome	tipo	assinatura	o que faz
`SphericalHarmonics`	classe	`(trunc, nlat=None, nlon=None, radius=EARTH['a'])`	Transformadas espectrais escalares e vetoriais com truncamento triangular numa grade gaussiana.
`SphereShallowWater`	classe	`(sh, zeta, delta, phi, f, phi_s, omega, g, hyperdiffusion, t, history)`	Estado e integrador do modelo de águas rasas (vorticidade, divergência, geopotencial), com diagnósticos.
`legendre_tables`	função	`(mu, M, N) -> tuple[np.ndarray, np.ndarray]`	Tabelas das funções de Legendre associadas normalizadas e suas derivadas.
`stable_dt`	função	`(trunc, gh_max, u_max, radius=EARTH['a'], safety=0.7) -> float`	Passo de tempo RK4 explícito estável para a truncagem e velocidades dadas.
`williamson_case2`	função	`(sh, alpha=0.0, gh0=29400.0, **kw) -> SphereShallowWater`	Caso 2 de Williamson: escoamento zonal geostrófico estacionário.
`williamson_case5`	função	`(sh, **kw) -> SphereShallowWater`	Caso 5 de Williamson: escoamento zonal sobre montanha cônica.
`williamson_case6`	função	`(sh, R=4, omega_w=7.848e-06, K=7.848e-06, h0=8000.0, **kw) -> SphereShallowWater`	Caso 6 de Williamson: onda de Rossby-Haurwitz.

nome	tipo	assinatura	o que faz
rossby_haurwitz_speed	função	(R=4, omega_w=7.848e-06, omega=EARTH['omega']) -> float	Velocidade de fase angular da onda de Rossby-Haurwitz no modelo barotrópico não divergente.
height_errors	função	(sh, h, h_ref) -> dict[str, float]	Erros normalizados l1, l2 e l_inf de Williamson et al.

pinneapple_simulation.meshless

Métodos sem malha e com malha para os mesmos problemas 2-D e o estudo que os compara; reexporta domínios, problemas, métodos e meshless_study.

pinneapple_simulation/meshless/__init__.py · 23 linhas · testes: 14

pinneapple_simulation.meshless.contracts

Converte um estudo meshless no contrato StudyResult dos apps (pinneapple_data.contracts): tabela por método e espaçamento, convergência, vencedores e checagens. [sem teste direto]

pinneapple_simulation/meshless/contracts.py · 50 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
meshless_checks	função	(res) -> list	Checagens do estudo: ordem observada contra a esperada onde a solução é suave, e estabilidade.
study_result	função	(res, *, study_id, title, limits=(), app='meshless')	Monta o StudyResult de um estudo meshless com tabela, métricas, checagens e proveniência.

pinneapple_simulation.meshless.domains

Domínios 2-D (por SDF, polígonos com furos, presets ou nuvem de pontos enviada) e geração de conjuntos de nós para o estudo. [sem teste direto]

pinneapple_simulation/meshless/domains.py · 218 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
NodeSet	classe	(X, boundary, h)	Conjunto de nós com marca de contorno e espaçamento h.
Domain2D	classe	(name, sdf, pieces, lo, hi, description, cloud, eval_points)	Domínio 2-D com SDF, contorno, caixa, nuvem opcional e pontos de avaliação, que gera nós e amostras.
polygon_domain	função	(vertices, name='polygon', holes=()) -> Domain2D	Cria um domínio poligonal, opcionalmente com furos circulares.
domain	função	(name) -> Domain2D	Devolve um domínio preset: square, lshape, disk, annulus ou plate_hole.
point_cloud_domain	função	(X, boundary, *, eval_fraction=0.1, seed=0, name='point cloud') -> Domain2D	Cria um domínio a partir de uma nuvem de pontos enviada, separando uma fração para avaliação.

pinneapple_simulation.meshless.methods

Cinco formas de resolver o mesmo problema 2-D: Kansa (RBF global), RBF-FD, diferenças finitas com Shortley-Weller, FEM P1 e PINN, com custo de montagem, solve, número de condição e amplificação de ruído.

pinneapple_simulation/meshless/methods.py · 427 linhas · testes: 2

nome	tipo	assinatura	o que faz
Discretization	classe	(method, nodes, evaluate, setup_s, L, K, M, kansa, info, options)	Operador discretizado de um método nos seus nós, com interpolador, matrizes, custo de montagem e informações.
default_eps	função	(domain) -> float	Parâmetro de forma padrão da multiquádrica (3 / diâmetro do domínio).
rbf_fd_weights	função	(X, Q, idx, op='lap', degree=4) -> np.ndarray	Pesos RBF-FD 2-D dos estênceis para o Laplaciano ou interpolação.
fd_nodes	função	(domain, h)	Nós de grade dentro do domínio e cruzamentos das linhas de grade com o contorno.

nome	tipo	assinatura	o que faz
`discretize`	função	`(method, domain, nodes, **options) -> Discretization`	Monta o operador de um método sobre os nós (FD gera sua própria grade).
`solve`	função	`(problem, disc, *, seed=0) -> dict[str, Any]`	Resolve o problema na discretização (permanente ou calor por Crank-Nicolson) e devolve valores e custos.

pinneapple_simulation.meshless.pinn

PINN no conjunto de nós do estudo: nós internos como pontos de colocação, nós de contorno com Dirichlet, treino limitado por tempo de parede para comparar com os outros métodos no mesmo custo. [sem teste direto]

`pinneapple_simulation/meshless/pinn.py` · 89 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`train_pinn`	função	`(problem, disc, *, budget_s=5.0, width=32, depth=3, lr=0.002, bc_weight=10.0, n_times=8, …)`	Treina uma PINN nos nós do estudo dentro de um orçamento de tempo de parede.

pinneapple_simulation.meshless.problems

Problemas lineares 2-D com Dirichlet (Poisson, Helmholtz, calor) definidos por expressões em x, y e t; com solução manufaturada, a fonte e o contorno saem por sympy.

`pinneapple_simulation/meshless/problems.py` · 147 linhas · testes: 22

nome	tipo	assinatura	o que faz
`Problem`	classe	`(kind, exact, source, boundary, initial, k, alpha, t_end, n_steps)`	Problema linear 2-D (Poisson, Helmholtz ou calor) com solução exata opcional, fonte, contorno e condição inicial.

pinneapple_simulation.meshless.stencils

Pesos de estêncil em qualquer dimensão para discretizações sem malha: RBF-FD (spline poli-harmônica + polinômios) e GFDM (mínimos quadrados ponderados).

`pinneapple_simulation/meshless/stencils.py` · 105 linhas · testes: 2

nome	tipo	assinatura	o que faz
`monomials`	função	`(dim, degree) -> list[tuple[int, ...]]`	Lista os expoentes de todos os monômios de grau total até `degree` em `dim` variáveis.
`stencil_weights`	função	`(X, Q, idx, ops=('lap',), *, method='rbf_fd', degree=2, batch=2000) -> dict[str, np.ndarr…)`	Calcula pesos de estêncil por operador (eval, derivadas, Laplaciano) por RBF-FD ou GFDM.

pinneapple_simulation.meshless.study

Estudo de refinamento: para cada espaçamento e método mede erro, tempos, número de condição e amplificação de ruído, depois ordem de convergência, varredura do parâmetro de forma do Kansa e onde cada método vence. [sem teste direto, mas exercitado por `tests/test_meshless.py` via o pacote]

`pinneapple_simulation/meshless/study.py` · 258 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`MeshlessStudy`	classe	`(problem, domain, rows, convergence, eps_sweep, wins, notes, reference, fields, skipped, …)`	Resultado do estudo: linhas por método e espaçamento, convergência, varredura de forma, vencedores, notas, referência e campos.
`meshless_study`	função	`(problem, dom, h=None, *, methods=…, options=None, pinn_levels='finest', n_eval=2000, tol…)`	Roda todos os métodos em todos os espaçamentos e monta o `MeshlessStudy`.

pinneapple_simulation.multibody

Dinâmica de corpos rígidos em coordenadas reduzidas: uma base flutuante com árvores de juntas de revolução; reexporta a árvore e os utilitários de braço.

pinneapple_simulation/multibody/__init__.py · 26 linhas · testes: 45

pinneapple_simulation.multibody.arm

Braços seriais de base fixa sobre `MultibodyTree`: dinâmica inversa e direta, referência analítica do braço plano de dois elos, trajetórias quínticas, balanço de energia e checagem de tensão na raiz dos elos.

pinneapple_simulation/multibody/arm.py · 161 linhas · testes: 39

nome	tipo	assinatura	o que faz
`fixed_base_inverse_dynamics`	função	`(tree, q, qd, qdd) -> np.ndarray`	Calcula os torques de junta ao longo de uma trajetória (q, qd, qdd) com a base fixa na origem.
`fixed_base_forward`	função	`(tree, q0, qd0, tau_fn, t) -> tuple[np.ndarray, np.ndarray]`	Integra (RK4) os ângulos e velocidades de junta sob torques dados por `tau_fn(t, q, qd)`.
`planar_two_link_torques`	função	`(m1, m2, l1, lc1, lc2, I1, I2, g, q, qd, qdd) -> np.ndarray`	Torques em forma fechada de um braço plano de dois elos num plano vertical, usados como referência independente.
`quintic`	função	`(q0, q1, T, t) -> tuple[np.ndarray, np.ndarray, np.ndarray]`	Gera uma trajetória quíntica repouso-a-repouso de `q0` a `q1` em `T` segundos (posição, velocidade, aceleração).
`energy_balance`	função	`(tree, t, q, qd, tau) -> dict`	Compara o trabalho dos torques (trapézio) com a variação de energia cinética + potencial e devolve o desvio relativo.
`link_world_points`	função	`(tree, q, points) -> dict[str, np.ndarray]`	Posições no mundo de pontos fixos em cada corpo, para cada configuração.
`lumped_link`	função	`(name, length, mass_per_m, point_masses=(), width=0.05)`	Cria um `Body` de elo reto: viga uniforme mais massas pontuais (motor, carga), com massa, centro de massa e inércia (eixos paralelos).
`link_root_checks`	função	`(tau, section, *, yield_strength, fatigue_strength=None, safety=3.0, names=None) -> tuple…)`	Tensão de flexão na raiz de cada elo a partir dos torques de junta, checada contra escoamento/segurança e, opcionalmente, fadiga.

pinneapple_simulation.multibody.tree

Árvore multicorpo de base flutuante com juntas de revolução: cinemática, jacobianos, matriz de massa, forças de viés, limites de junta por mola de penalidade e passo de tempo linearmente implícito. [sem teste]

pinneapple_simulation/multibody/tree.py · 332 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Body`	classe	`(name, mass, com, inertia, parent_joint)`	Corpo rígido com massa, centro de massa e tensor de inércia no referencial do corpo.
`RevoluteJoint`	classe	`(name, parent, child, origin, axis, R0, limits, damping)`	Junta de revolução entre pai e filho, com origem, eixo, rotação inicial, limites (mola de penalidade) e amortecimento.
`Kinematics`	classe	`(R, p, c, axis, o, Jv, Jw, omega, vel)`	Poses e velocidades no mundo de todos os corpos para um estado, com jacobianos.
`MultibodyTree`	classe	`(gravity=(0.0, 0.0, -1.62))`	Árvore multicorpo (o primeiro corpo é a base flutuante) com cinemática, matriz de massa, forças de viés, gravidade, passo de tempo e energia.
`cross`	função	`(a, b)`	Produto vetorial de arrays (..., 3), mais barato que `np.cross` para arrays pequenos.
`skew`	função	`(a)`	Matriz antissimétrica de um vetor 3D.
`quat_to_matrix`	função	`(q)`	Converte quatérnio em matriz de rotação.
`axis_angle`	função	`(axis, angle)`	Matriz de rotação a partir de eixo e ângulo.

nome	tipo	assinatura	o que faz
`quat_mul`	função	`(a, b)`	Produto de dois quatérnios.
`quat_from_matrix`	função	`(R)`	Converte matriz de rotação em quatérnio.

`pinneapple_simulation.numerical_solvers`

Reúne os solvers numéricos (FDM, FEM, FVM, LBM, espectral, partículas, decomposições de sinal, X-TFC) e reexporta o registro, o gerador de datasets PINN e os solvers 3D; as pontes OpenFOAM e FEniCS são importadas só se disponíveis.

`pinneapple_simulation/numerical_solvers/__init__.py` · 73 linhas · testes: 111

nome	tipo	assinatura	o que faz
`openfoam_available`	função	`() -> bool`	Diz se a ponte OpenFOAM foi importada com sucesso.
`fenics_available`	função	`() -> bool`	Diz se a ponte FEniCS foi importada com sucesso.

`pinneapple_simulation.numerical_solvers._bc`

Lê as condições de Dirichlet de um `ProblemSpec` e devolve índices e valores nos nós de uma malha ou grade, para os solvers baseados em grade. [sem teste direto]

`pinneapple_simulation/numerical_solvers/_bc.py` · 82 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`dirichlet_conditions`	função	`(spec) -> List`	Filtra as condições de tipo `dirichlet` de um `ProblemSpec`.
`select`	função	`(cond, pts, edges=None, boundary=None) -> torch.Tensor`	Índices dos pontos aos quais uma condição se aplica.
`values`	função	`(cond, pts, field=0) -> torch.Tensor`	Valores de Dirichlet nos pontos para um campo (coluna de um `value_fn` multicampo).
`boundary_rect`	função	`(nodes, edges) -> torch.Tensor`	União dos índices de nós de todas as arestas de um retângulo.

`pinneapple_simulation.numerical_solvers.adapters`

Pacote dos adaptadores UPD (malha e sinal) usados para alimentar os solvers. [sem teste direto]

`pinneapple_simulation/numerical_solvers/adapters/__init__.py` · 20 linhas · **sem teste direto**

`pinneapple_simulation.numerical_solvers.adapters.upd_mesh`

Converte uma malha de superfície (vértices, faces) em topologia mínima de FVM ou em entradas de montagem FEM (nós e conectividade); versão MVP. [sem teste direto]

`pinneapple_simulation/numerical_solvers/adapters/upd_mesh.py` · 163 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`mesh_to_fvm_topology`	função	`(mesh, *, device=None, dtype=None) -> Dict[str, torch.Tensor]`	Monta uma topologia FVM mínima a partir de uma malha de triângulos ou quadriláteros.
`mesh_to_fem_assembly_inputs`	função	`(mesh, *, device=None, dtype=None) -> Dict[str, torch.Tensor]`	Devolve nós e conectividade para montagem FEM; a matriz K e o vetor f ficam por conta do usuário.

`pinneapple_simulation.numerical_solvers.adapters.upd_signal`

Extrai sinais 1D de campos físicos de objetos tipo `PhysicalSample` (duck typing) e empilha vários em lote com padding. [sem teste direto]

`pinneapple_simulation/numerical_solvers/adapters/upd_signal.py` · 184 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`list_field_vars`	função	`(sample) -> List[str]`	Lista as variáveis de campo de um objeto tipo `PhysicalSample`.
`list_signal_axes`	função	`(sample, var) -> List[str]`	Nomes dos eixos de um campo, com fallback `dim0`, `dim1`, ...
`extract_1d_signal`	função	`(sample, *, var, axis=-1, reduce='mean', index=None, device=None, dtype=None) -> Tuple[to…)`	Extrai um sinal 1D de um campo multidimensional por redução (média etc.) ou índice.
`to_signal_batch`	função	`(samples, *, var, axis=-1, reduce='mean', index=None, device=None, dtype=None) -> Tuple[t…)`	Converte uma lista de amostras num tensor de lote (B, Tmax) com padding.

pinneapple_simulation.numerical_solvers.base

Define `SolverOutput` (resultado, perdas, extras) e `SolverBase`, a classe base `nn.Module` de todos os solvers registrados.

`pinneapple_simulation/numerical_solvers/base.py` · 34 linhas · testes: 151

nome	tipo	assinatura	o que faz
`SolverOutput`	classe	`(result, losses, extras)`	Saída padrão dos solvers: tensor principal, perdas e metadados extras.
`SolverBase`	classe	`()`	Classe base (`nn.Module`) dos solvers numéricos, com um utilitário `mse`.

pinneapple_simulation.numerical_solvers.beam_bvp_fdm

Viga de Euler-Bernoulli por diferenças finitas de 4ª ordem: deflexão, momento, cortante e tensões (flexão, cisalhamento, von Mises) para três tipos de apoio.

`pinneapple_simulation/numerical_solvers/beam_bvp_fdm.py` · 296 linhas · testes: 5

nome	tipo	assinatura	o que faz
`SectionProps`	classe	`(I_m4, c_m, Q_m3, t_m, A_m2)`	Propriedades da seção transversal para cálculo de tensões, com construtores para retângulo, retângulo vazado e círculos.
`BeamBVPFDMSolver`	classe	`(nx=200, bc_type=BC_SIMPLY_SUPPORTED)`	Wrapper fino `SolverBase`/registro de `solve_euler_bernoulli_beam_1d`.
`compute_stresses`	função	`(moment, shear, section)`	Tensão de flexão, de cisalhamento e de von Mises a partir de momento, cortante e seção.
`second_derivative`	função	`(f, dz) -> np.ndarray`	Derivada segunda de 2ª ordem em todos os pontos, inclusive nas bordas.
`solve_euler_bernoulli_beam_1d`	função	`(L_m=10.0, E_Pa=200000000000.0, I_m4=8.33e-06, q_N_per_m=1000.0, nx=200, bc_type=BC_SIMPL…)`	Resolve EI w'''' = q com carga uniforme para um dos três tipos de apoio e devolve deflexão, esforços e tensões.

pinneapple_simulation.numerical_solvers.beam_fe

Viga de Euler-Bernoulli por elementos finitos de Hermite: deflexão estática sob carga uniforme e modos naturais (frequências e formas).

`pinneapple_simulation/numerical_solvers/beam_fe.py` · 74 linhas · testes: 27

nome	tipo	assinatura	o que faz
`beam_matrices`	função	`(L, EI, m, n_el)`	Matrizes globais de rigidez e massa consistente e posições dos nós de uma viga uniforme.
`beam_static`	função	`(L, EI, q, n_el=40, ends=('pinned', 'pinned')) -> dict`	Deflexão sob carga uniforme com cargas nodais consistentes.
`beam_modes`	função	`(L, EI, m, n_el=40, n_modes=4, ends=('pinned', 'pinned')) -> dict`	Frequências naturais (Hz) e formas modais normalizadas pela massa.

pinneapple_simulation.numerical_solvers.bekker_wong

Solver numérico de Bekker-Wong para interação roda rígida / solo deformável: tensões no contato, forças (F_x, F_z, M_y), afundamento a partir da carga e geração de datasets; inclui versão em lote em PyTorch.

pinneapple_simulation/numerical_solvers/bekker_wong.py · 347 linhas · testes: 50

nome	tipo	assinatura	o que faz
BekkerWongSolver	classe	(soil=None, wheel=None, rear_angle_ratio=1.0 / 3.0)	Solver de Bekker-Wong para uma roda e um solo: tensões, forças, afundamento a partir da carga e geração de dataset.
bekker_wong_forces_torch	função	(slip, z, *, R, b, c, tan_phi, K, k_c, k_phi, n, a0=0.4, a1=0.15, rear_angle_ratio=1.0 / …)	Forças (F_x, F_z, M_y) de Bekker-Wong em lote, por quadratura de Gauss-Legendre por partes em PyTorch.

pinneapple_simulation.numerical_solvers.ceemdan

Decomposição CEEMDAN (EMD em conjunto com ruído adaptativo) para sinais 1D; o docstring diz que é uma versão aproximada. [sem teste direto]

pinneapple_simulation/numerical_solvers/ceemdan.py · 103 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
CEEMDANSolver	classe	(n_ensembles=50, noise_std=0.2, max_imfs=10, seed=None)	Wrapper registrado de ceemdan_1d.
ceemdan_1d	função	(x, *, n_ensembles=50, noise_std=0.2, max_imfs=10, seed=None) -> np.ndarray	CEEMDAN aproximado; devolve as IMFs (K, T).

pinneapple_simulation.numerical_solvers.cfd_pipeline

Pipeline CAD→CFD: carrega STEP/STL ou CSG, gera malha (gmsh se instalado, senão malha estruturada simples), resolve Navier-Stokes incompressível na malha e exporta dados de treino PINN.

pinneapple_simulation/numerical_solvers/cfd_pipeline.py · 954 linhas · testes: 151

nome	tipo	assinatura	o que faz
CFDMesh	classe	(nodes, elements, boundary_tags) -> None	Malha não estruturada para CFD, criada de STL, STEP ou CSG, com nós de contorno e plot 2D.
NSFlowSolver	classe	(mesh, nu=0.001, rho=1.0, solver='direct') -> None	Solver de Navier-Stokes incompressível na malha, com condições de contorno e exportação de dados PINN.
CADToCFDPipeline	classe	(nu=0.001, rho=1.0) -> None	Pipeline completo: arquivo CAD → malha → solução NS → dados de treino PINN, com comparação contra um PINN.

pinneapple_simulation.numerical_solvers.compressible_fv

Volumes finitos para Euler / Navier-Stokes compressível em 2D e 3D cartesianos (PyTorch), com Riemann HLLC e limitadores; inclui a solução exata do tubo de choque de Sod.

pinneapple_simulation/numerical_solvers/compressible_fv.py · 299 linhas · testes: 15

nome	tipo	assinatura	o que faz
CompressibleConfig	classe	(shape, lengths, gamma, Pr, mu_ref, omega, cfl, limiter, bc, inflow, p_out, device, dtype…)	Configuração do solver compressível: grade, gás, viscosidade, CFL, limitador, contornos e pisos de densidade/pressão.
CompressibleFV	classe	(cfg)	Solver de volumes finitos compressível: conversões primitivas/conservativas, contornos, RHS, passo de tempo e número de Mach.
sod_exact	função	(x, t, left=(1.0, 0.0, 1.0), right=(0.125, 0.0, 0.1), gamma=1.4, x0=0.5)	Solução exata do problema de Riemann 1D (Toro, cap. 4): rho, u, p.

pinneapple_simulation.numerical_solvers.cylinder_wake

Esteira de cilindro confinado com o LBM D2Q9: uma simulação, a verificação (perfil de canal e estudo de resolução) e a geração do dataset do zoo cylinder_wake_lbm.

pinneapple_simulation/numerical_solvers/cylinder_wake.py · 178 linhas · testes: 3

nome	tipo	assinatura	o que faz
probe_strouhal	função	(signal, dt) -> float	Número de Strouhal de um registro de sonda pelo pico do espectro com janela de Hann e interpolação gaussiana.
simulate	função	(Re, D=SETUP['D'], height=SETUP['height'], length=SETUP['length'], u_in=SETUP['u_in'], co…)	Uma simulação da esteira: tempos, quadros de vorticidade, sonda e métricas da esteira.
verify	função	(Ds=(20, 30, 40), Re=100.0, channel_Re=2.0) -> dict	Verificação: perfil de Poiseuille desenvolvido em duas grades e estudo de resolução da esteira.
generate	função	(out, workers=1, name='cylinder_wake_lbm') -> dict	Roda o plano de simulações e grava os npz e o meta.json do dataset.

pinneapple_simulation.numerical_solvers.dfsph

SPH livre de divergência (DFSPH, Bender & Koschier) com resolução de divergência e de densidade por passo. [sem teste direto]

pinneapple_simulation/numerical_solvers/dfsph.py · 317 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
DFSPHSolver	classe	(h=0.04, mass=1.0, rho0=1000.0, mu=0.02, gravity=None, boundary_lo=None, boundary_hi=None…)	Solver DFSPH com laços de correção de divergência e de densidade por passo.

pinneapple_simulation.numerical_solvers.discrete_time_pinn

PINN de tempo discreto (Raissi et al., Parte II) com Runge-Kutta implícito de q estágios para Burgers 1D e Allen-Cahn 1D, treinado com L-BFGS.

pinneapple_simulation/numerical_solvers/discrete_time_pinn.py · 408 linhas · testes: 7

nome	tipo	assinatura	o que faz
DiscreteTimeRKNet	classe	(q, hidden_layers=(50, 50, 50, 50), activation='tanh')	Rede MLP tanh que prevê os q estágios RK e o estado final num ponto x.
DiscreteTimeResult	classe	(model, A, b, c, q, dt, x_span, loss_history)	Resultado do treino: modelo, tabela RK, passo, histórico de perda e preditores de estágio e final.
DiscreteTimePINNSolver	classe	(pde='burgers_1d', q=32, hidden_layers=(30, 30, 30), activation='tanh', n_boundary=20, bc…)	Wrapper fino SolverBase/registro de train_discrete_time.
stage_derivatives	função	(model, x)	U, U_x e U_xx da rede em todos os estágios via AD em modo direto.
discrete_time_rk_residual	função	(model, x, u_n_data, dt, A, b, nonlinear_op) -> 'tuple[torch.Tensor, torch.Tensor]'	Resíduos de consistência IRK dos estágios e do estado final.
train_discrete_time	função	(pde, x_n, u_n, dt, q=32, x_span=None, hidden_layers=(30, 30, 30), activation='tanh', par…)	Treina a rede para um passo IRK de Burgers ou Allen-Cahn 1D a partir de dados em t_n, com L-BFGS.

pinneapple_simulation.numerical_solvers.eddy_current_fdm

Diferenças finitas 2D axissimétricas para magnetostática e correntes parasitas: potencial vetor A_theta, densidade de corrente induzida e campo B_z.

pinneapple_simulation/numerical_solvers/eddy_current_fdm.py · 240 linhas · testes: 74

nome	tipo	assinatura	o que faz
EddyCurrentFDMSolver	classe	(nr=60, nz=120)	Wrapper fino SolverBase/registro do solver axissimétrico de correntes parasitas.
build_axisymmetric_helmholtz_system	função	(r, z, k2, rhs_2d)	Monta o sistema esparso complexo do operador axissimétrico.
annular_current_source	função	(r, z, r_center, z_center, width_z, width_r, n_turns, current_amp) -> np.ndarray	Densidade de corrente azimutal uniforme numa faixa anular (aproximação de bobina concentrada).
solve_axisymmetric_eddy_current	função	(r, z, omega, mu, sigma, source_current_density) -> np.ndarray	Resolve o potencial vetor A_theta(r, z) para uma frequência; omega = 0 dá o caso magnetostático.

nome	tipo	assinatura	o que faz
eddy_current_density	função	(A, omega, sigma) -> np.ndarray	Corrente parasita induzida J = -i omega sigma A.
axial_flux_density	função	(A, r, z) -> np.ndarray	Campo B_z a partir de A_theta por diferenças centrais.

pinneapple_simulation.numerical_solvers.eemd

EMD simples e EEMD (EMD em conjunto) para sinais 1D. [sem teste direto]

pinneapple_simulation/numerical_solvers/eemd.py · 167 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
EEMDSolver	classe	(n_ensembles=50, noise_std=0.2, max_imfs=10, seed=None)	Wrapper registrado de eemd_1d.
emd_1d	função	(x, *, max_imfs=10, max_sift=50, stop_thresh=0.05) -> List[np.ndarray]	EMD mínima para sinais 1D.
eemd_1d	função	(x, *, n_ensembles=50, noise_std=0.2, max_imfs=10, seed=None) -> np.ndarray	EEMD 1D; devolve as IMFs (K, T).

pinneapple_simulation.numerical_solvers.elasticity3d_fdm

Elasticidade linear 3D estática (Navier-Cauchy) por diferenças finitas e relaxação de Jacobi amortecida; só aceita deslocamento prescrito em todas as faces. [sem teste direto]

pinneapple_simulation/numerical_solvers/elasticity3d_fdm.py · 182 linhas · **sem teste direto** · palavras-chave no código (heurística; o texto acima diz o status real): placeholder/stub citado no docstring

nome	tipo	assinatura	o que faz
Elasticity3DFDMSolver	classe	(n_iter=20000, omega=1.0, tol=1e-10)	Wrapper fino SolverBase/registro de solve_static_elasticity_3d.
lame_parameters	função	(E, nu) -> Tuple[float, float]	Parâmetros de Lamé (lambda, mu) a partir de E e nu.
solve_static_elasticity_3d	função	(lam, mu, body_force, u_bc, dx, dy, dz, n_iter=20000, omega=1.0, tol=1e-10) -> dict	Resolve Navier-Cauchy estático numa grade por Jacobi amortecido.

pinneapple_simulation.numerical_solvers.etdrk4

Integrador ETDRK4 (diferenças exponenciais, Runge-Kutta de 4ª ordem) para problemas rígidos semilineares com parte linear diagonal, com detecção de explosão numérica.

pinneapple_simulation/numerical_solvers/etdrk4.py · 151 linhas · testes: 17

nome	tipo	assinatura	o que faz
SimulationBlowUp	classe	(message, *, step, t)	Exceção levantada quando o estado fica não finito ou grande demais.
ETDRK4Result	classe	(t, u, steps, blew_up, meta)	Resultado da integração: tempos, estados, passos, flag de explosão e metadados.
ETDRK4	classe	(linear, nonlinear, dt, *, contour_points=32, radius=1.0)	Integrador ETDRK4 de passo fixo com coeficientes por integral de contorno.
check_finite	função	(u, *, limit=1e+37, step=-1, t=float('nan')) -> None	Levanta SimulationBlowUp se houver NaN/inf ou magnitude acima do limite.
fourier_wavenumbers	função	(n, length=2 * np.pi, *, real=False) -> np.ndarray	Números de onda angulares de uma grade periódica na ordem da FFT.

pinneapple_simulation.numerical_solvers.external_flow

Escoamento externo em torno de qualquer corpo no OpenFOAM numa chamada: STL → snappyHexMesh → simpleFoam (k-omega SST) → forças, coeficientes de pele, linhas de corrente, cortes e cena 3D. Requer OpenFOAM instalado.

pinneapple_simulation/numerical_solvers/external_flow.py · 728 linhas · testes: 40

nome	tipo	assinatura	o que faz
CaseFields	classe	(case, time=None)	Malha reconstruída e últimos U, p de um caso, com forças de parede, interpolação de velocidade, linhas de corrente e cortes.
FlowResult	classe	(coefficients, surface, bodies, lines, line_speed, slices, info, geometry)	Resultado de um escoamento externo (coeficientes, superfície, linhas, cortes) com exportação para cena 3D, gravação e leitura.
ExternalFlow	classe	(bodies, speed, alpha, beta, nu, rho, ref_area, ref_length, half_model, ground, ground_mo…)	Escoamento incompressível permanente em torno de corpos: escreve, roda e lê o caso OpenFOAM.
flow_directions	função	(alpha_deg, beta_deg=0.0) -> Tuple[np.ndarray, np.ndarray, np.ndarray]	Vetores unitários de arrasto, sustentação e lateral para os ângulos dados.
write_flow_fields	função	(case, speed, alpha_deg, beta_deg, nu, iterations, procs, walls, symmetry, ground=None, g…)	Escreve campos iniciais, propriedades, controlDict, esquemas e solvers de um caso de escoamento externo.
snappy_dict	função	(geometry, features, surfaces, regions, location) -> str	Texto do snappyHexMeshDict (castellate + snap, sem camadas).
run_case	função	(case, procs=4, log=print, all_times=False) -> Dict[str, float]	Roda blockMesh, snappyHexMesh, checkMesh e simpleFoam em paralelo e reconstrói; devolve o tempo de cada etapa.
latest_time	função	(case) -> str	Último diretório de tempo com campo U de um caso; erro se não houver.
openfoam_available	função	() -> bool	Diz se as ferramentas OpenFOAM usadas pelo módulo estão no sistema.

pinneapple_simulation.numerical_solvers.fdm

Solver FDM agnóstico ao problema: escolhe o esquema pelo `pde.kind` do `ProblemSpec` (Poisson, Helmholtz, calor, onda, Burgers, advecção) e aplica condições Dirichlet, Neumann e Robin.

pinneapple_simulation/numerical_solvers/fdm.py · 919 linhas · testes: 3

nome	tipo	assinatura	o que faz
FDMSolver	classe	(nx=64, ny=64, nt=500, iters=8000, omega=1.5, theta=0.5, tol=1e-09)	Solver FDM que escolhe o esquema pelo tipo de EDP do `ProblemSpec`.

pinneapple_simulation.numerical_solvers.fdm3d

Solvers 3D por diferenças finitas: condução de calor, Navier-Stokes incompressível (projeção), onda elástica/acústica, cavidade com tampa móvel e canal com gradiente de pressão.

pinneapple_simulation/numerical_solvers/fdm3d.py · 911 linhas · testes: 151

nome	tipo	assinatura	o que faz
SolverOutput3D	classe	(u, coords, meta)	Saída dos solvers FDM 3D: campo, coordenadas e metadados.
HeatConfig3D	classe	(nx, ny, nz, nt, lx, ly, lz, dt, alpha, ic_fn, source_fn)	Configuração do solver de calor 3D.
HeatConduction3D	classe	(cfg=None)	Equação do calor 3D com Euler explícito e diferenças centrais.
NavierStokesConfig3D	classe	(nx, ny, nz, nt, lx, ly, lz, dt, nu, rho, pressure_iters, ic_fn, force_fn)	Configuração do solver Navier-Stokes 3D.
NavierStokes3D	classe	(cfg=None)	Navier-Stokes 3D incompressível pelo método de projeção.
ElasticWaveConfig3D	classe	(nx, ny, nz, nt, lx, ly, lz, dt, c, ic_u_fn, ic_du_fn, source_fn)	Configuração do solver de onda elástica 3D.
ElasticWave3D	classe	(cfg=None)	Equação de onda elástica (acústica) 3D com integração Störmer-Verlet.
LidDrivenCavityConfig3D	classe	(nx, ny, nz, nt, size, dt, nu, rho, lid_velocity, lid_axis, pressure_iters)	Configuração da cavidade 3D com tampa móvel.
LidDrivenCavitySolver3D	classe	(cfg=None)	Escoamento 3D em cavidade com tampa móvel.

nome	tipo	assinatura	o que faz
`ChannelFlowConfig3D`	classe	`(nx, ny, nz, nt, lx, ly, lz, dt, nu, rho, dp_dx, pressure_iters)`	Configuração do canal 3D com gradiente de pressão.
`ChannelFlowSolver3D`	classe	`(cfg=None)`	Escoamento 3D em canal movido a gradiente de pressão (tipo Poiseuille).

pinneapple_simulation.numerical_solvers.fem

Solver FEM agnóstico ao problema (elementos Q1 em malha estruturada): Poisson/Laplace, Helmholtz e elasticidade linear axissimétrica, com CG ou solução direta.

`pinneapple_simulation/numerical_solvers/fem.py` · 368 linhas · testes: 8

nome	tipo	assinatura	o que faz
`FEMSolver`	classe	`(nx=32, ny=32, solver='cg', tol=1e-08, max_iter=4000)`	Solver FEM Q1 que lê o tipo de EDP e as condições do `ProblemSpec`.

pinneapple_simulation.numerical_solvers.fenics_bridge

Ponte para FEniCS/dolfinx (com fallback para o FEniCS legado): monta malha, espaço de funções e forma variacional a partir do `ProblemSpec` e extrai os campos. Dependência opcional.

`pinneapple_simulation/numerical_solvers/fenics_bridge.py` · 647 linhas · testes: 151

nome	tipo	assinatura	o que faz
`FEnicsBridge`	classe	`(mesh_nx=32, mesh_ny=32, element_degree=1, solver_backend='dolfinx') -> None`	Formula e resolve problemas FEM com FEniCS/dolfinx a partir de um `ProblemSpec`.

pinneapple_simulation.numerical_solvers.fft

Solver FFT (PyTorch) que devolve espectro em magnitude/fase ou complexo, com rFFT para entrada real. [sem teste direto]

`pinneapple_simulation/numerical_solvers/fft.py` · 85 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`FFTSolver`	classe	`(*, dims=None, real_input=True, output='magnitude_phase', norm=None)`	Solver FFT em PyTorch que devolve magnitude/fase ou espectro complexo.

pinneapple_simulation.numerical_solvers.free_surface

Escoamento com superfície livre líquido-gás no OpenFOAM (interFoam, VOF): nível inicial, entradas, fronteiras abertas, k-omega SST e amostragem de linhas com média temporal. Requer OpenFOAM.

`pinneapple_simulation/numerical_solvers/free_surface.py` · 626 linhas · testes: 12

nome	tipo	assinatura	o que faz
`Phase`	classe	`(name, rho, nu)`	Uma fase (nome, densidade, viscosidade cinemática).
`SampleLine`	classe	`(name, start, end, n)`	Linha de amostragem (nome, início, fim, número de pontos).
`FreeSurfaceFlow`	classe	`(regions, roles, mesh, inlet_velocity, level, liquid, gas, sigma, gravity, p_ref, turbule…)`	Caso interFoam completo: regiões, papéis dos patches, nível, fases, turbulência; escreve, roda e lê perfis.
`LiveLineSampler`	classe	`(case, lines, fields, keep_every=None, end_time=float('inf'), out_name='lines', poll_s=3.…)`	Amostra linhas de um caso em execução a cada tempo gravado e pode apagar o tempo depois.
`split_surface`	função	`(V, F, boxes, rest='walls') -> dict[str, tuple[np.ndarray, np.ndarray]]`	Divide uma superfície fechada em regiões nomeadas por caixas.
`cell_centres`	função	`(poly_dir) -> np.ndarray`	Centros de células de uma polyMesh ASCII.

nome	tipo	assinatura	o que faz
write_profile	função	(case, tname, idx, res, fields, out_name='lines') -> None	Grava em CSV os valores amostrados de cada linha num tempo.
sample_lines	função	(case, lines, fields) -> int	Amostra todas as gravações de um caso terminado nas linhas dadas.
read_profiles	função	(case, names=None, t_start=0.0, t_end=float('inf'), out_name='lines') -> dict	Lê os perfis amostrados por linha, com tempos, campos e médias.
time_average	função	(profile, field, window=None, origin='min') -> tuple[np.ndarray, np.ndarray]	Média temporal de um campo ao longo de uma linha, opcionalmente cortada numa janela.

pinneapple_simulation.numerical_solvers.fvm

Solver FVM agnóstico ao problema em grade 2D estruturada de centro de célula: calor/difusão, convecção-difusão e Burgers.

pinneapple_simulation/numerical_solvers/fvm.py · 345 linhas · testes: 5

nome	tipo	assinatura	o que faz
FVMSolver	classe	(nx=64, ny=64, nt=200, dt=None, safety=0.4)	Solver FVM 2D de centro de célula guiado pelo ProblemSpec.

pinneapple_simulation.numerical_solvers.gated_xtfc

X-TFC com funções de base radial gaussianas cujos centros e larguras seguem uma divisão suave do domínio, para BVPs lineares 1D de 2ª ordem com camadas finas (problema direto e inverso).

pinneapple_simulation/numerical_solvers/gated_xtfc.py · 309 linhas · testes: 8

nome	tipo	assinatura	o que faz
LinearBVP1D	classe	(a, b, c, f, u_left, u_right, nu, exact)	BVP linear a u'' + b u' + c u = f em [0, 1] com Dirichlet nas pontas.
GatedLayout	classe	(centers, widths, collocation, splits, eps_b)	Centros, larguras, pontos de colocação e divisões da base RBF com portas.
GatedSolution	classe	(problem, layout, coef, solve_seconds)	Solução X-TFC com portas: coeficientes, predição e resíduo.
InverseFit	classe	(nu, splits, eps_scales, log_evidence, eta, solution, history, seconds)	Resultado do problema inverso: nu, portas, evidência, solução, histórico e desvio preditivo.
convection_diffusion	função	(nu) -> LinearBVP1D	Problema de convecção-difusão 1D com solução exata (eq. 1-2 do artigo).
twin_boundary_layer	função	(nu) -> LinearBVP1D	Problema com camada limite dupla e solução exata (Sec. 4.4 do artigo).
gated_layout	função	(splits, eps_scales, nu, n_centers, n_colloc=None, k=1.5) -> GatedLayout	Monta centros, larguras e pontos de colocação para uma ou várias divisões.
solve_forward	função	(problem, layout, lam=1e-12) -> GatedSolution	Resolve o problema direto por mínimos quadrados regularizados.
validation_energy	função	(sol, x_val) -> float	Resíduo médio quadrático na forma forte nos pontos de validação.
search_gates_nested	função	(problem, x_val, *, n_centers, n_colloc=None, k=1.5, lam=1e-12, xs_bounds=(0.8, 0.999), e…)	Busca de Brent aninhada sobre a posição da divisão e a escala de largura.
log_evidence	função	(Phi, y, *, eta0=1e-07, eta_bounds=(1e-12, 0.01), max_iter=50, damping=0.5) -> dict	Evidência bayesiana de um modelo linear com precisão a priori por ponto fixo.
fit_inverse	função	(make_problem, x_data, y_data, *, sigma, beta_pde, n_centers, n_colloc, k=1.5, log10_nu_b…)	Infere nu (e as portas) maximizando a evidência com o otimizador bayesiano da biblioteca.
search_gates_bo	função	(problem, x_val, *, split_bounds, eps_bounds=(10.0, 100.0), n_centers, n_colloc=None, k=1…)	Várias divisões: minimiza a energia do resíduo com otimização bayesiana.

pinneapple_simulation.numerical_solvers.heat_fv_reference

Soluções de referência independentes (volumes finitos, solução esparsa direta) para a placa 2D e o bloco 3D aquecidos do exemplo de convecção inversa; usadas só para conferir o PINN.

pinneapple_simulation/numerical_solvers/heat_fv_reference.py · 123 linhas · testes: 3

nome	tipo	assinatura	o que faz
`footprint`	função	`(x, y, xc, yc, half, width)`	Indicador suave de uma pegada quadrada de dispositivo (bordas tanh).
`footprint_area`	função	`(xc, yc, half, width, lx, ly, n=2000) -> float`	Área da pegada suave sobre a placa.
`plate_2d`	função	`(h, *, lx, ly, t, k, power, t_air, device, nx=200, ny=120)`	Temperatura da placa 2D nos centros das células.
`block_3d`	função	`(h, *, lx, ly, lz, k, power, t_air, device, nx=40, ny=40, nz=12)`	Temperatura do bloco 3D nos centros das células.
`interp_grid`	função	`(xs, ys, field, px, py)`	Interpolação bilinear de um campo 2D de centro de célula.
`convergence`	função	`(fn, coarse, fine, **kw) -> float`	Maior diferença de temperatura entre duas grades.

pinneapple_simulation.numerical_solvers.hilbert_huang

Transformada de Hilbert-Huang (EMD + Hilbert) para séries 1D; o docstring a chama de MVP para pesquisa. [sem teste direto]

`pinneapple_simulation/numerical_solvers/hilbert_huang.py` · 225 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`HilbertHuangSolver`	classe	`(*, max_imfs=8, max_sift=50, stop_mean_tol=0.001, dt=1.0)`	HHT para sinais 1D: EMD e depois amplitude e frequência instantâneas por Hilbert.

pinneapple_simulation.numerical_solvers.immersed_boundary_fdm

Navier-Stokes incompressível por fronteira imersa (IBM) e diferenças finitas em geometria 3D arbitrária, para escoamento interno (domínio fechado) ou externo (corpo imerso).

`pinneapple_simulation/numerical_solvers/immersed_boundary_fdm.py` · 964 linhas · testes: 2

nome	tipo	assinatura	o que faz
`ImmersedBoundaryFDMSolver`	classe	`(flow_type='internal', **solver_kwargs)`	Wrapper fino `SolverBase`/registro dos solvers IBM.
`field_divergence`	função	`(u, v, w, dx, dy, dz) -> np.ndarray`	Divergência por diferenças centrais de um campo de velocidade.
`interpolate_to_points`	função	`(grid_axes, field, query_points, fill_value=0.0) -> np.ndarray`	Interpolação trilinear de um campo de grade em pontos arbitrários.
`solve_ibm_internal_flow`	função	`(wall_points, bbox, *, Re=100.0, U_in=1.0, rho=1.0, nx=32, ny=32, nz=32, n_iter=500, n_fr…)`	IBM/FDM para domínio fechado (tampa móvel ou canal).
`solve_ibm_external_flow`	função	`(wall_points, bbox, *, Re=100.0, U_in=1.0, rho=1200.0, nx=32, ny=32, nz=32, n_iter=1000, …)`	IBM/FDM para escoamento livre em torno de um corpo imerso.
`solve_ibm_navier_stokes`	função	`(wall_points, bbox, flow_type='internal', **kwargs) -> Dict[str, Any]`	Despacha para o solver interno ou externo conforme `flow_type`.

pinneapple_simulation.numerical_solvers.internal_flow

Escoamento interno em tubos, curvas e misturadores estáticos Kenics no OpenFOAM numa chamada, com malha O-grid, perda de carga e fator de atrito comparados a correlações. Requer OpenFOAM.

`pinneapple_simulation/numerical_solvers/internal_flow.py` · 817 linhas · testes: 35

nome	tipo	assinatura	o que faz
`Route`	classe	`(D, segments)`	Linha central de um tubo: trechos retos e curvas, em diâmetros.
`InternalResult`	classe	`(route, info, sections, profiles, slices, lines, line_speed, wall, timings)`	Resultado do escoamento interno: seções, perfis, perda de carga, fator de atrito e cena 3D.
`InternalFlow`	classe	`(route, Re, nu, turbulent, mixer, mixer_start_D, scalar, inlet, n_core, n_radial, wall_gr…)`	Caso OpenFOAM de um tubo num dado Reynolds, com misturador Kenics e escalar passivo opcionais.
`kenics_elements`	função	`(route_D, n_elements, start_D, length_D=1.5, thickness_D=0.06, width_D=0.98, nu=10, ns=48…)`	Superfície fechada dos elementos de um misturador estático Kenics.

nome	tipo	assinatura	o que faz
oblock_mesh_dict	função	(route, n_core=8, n_radial=10, wall_grading=0.25, axial_cell_D=0.2, core=0.36, convert=1.…)	blockMeshDict da malha O-grid varrida ao longo da rota.
pipe_surface	função	(st, r, nth=64, sub=4) -> tuple[np.ndarray, np.ndarray]	Parede triangulada de uma seção circular varrida.
ito_bend_loss	função	(Re, radius_ratio, angle_deg=90.0) -> float	Coeficiente de perda de uma curva em regime turbulento (Ito 1960).
colebrook	função	(Re, rel_roughness=0.0) -> float	Fator de atrito de Darcy por Colebrook-White (delega a pinneapple_physics.closed_form).

pinneapple_simulation.numerical_solvers.irk_gauss_legendre

Tabelas de Butcher de Runge-Kutta implícito de Gauss-Legendre (ordem 2q), validação das condições de ordem, cache e um passo IRK com Newton.

pinneapple_simulation/numerical_solvers/irk_gauss_legendre.py · 236 linhas · testes: 7

nome	tipo	assinatura	o que faz
build_gauss_legendre_tableau	função	(q) -> Tuple[np.ndarray, np.ndarray, np.ndarray]	Constrói a tabela (A, b, c) de Gauss-Legendre de q estágios.
validate_tableau	função	(A, b, c, q, tol=1e-09) -> dict	Confere as condições de ordem e levanta AssertionError se falharem.
get_irk_tableau	função	(q, cache_dir=None) -> Tuple[np.ndarray, np.ndarray, np.ndarray]	Constrói ou carrega do cache e valida a tabela IRK.
step_irk	função	(rhs_fn, t, y, h, A, b, c, jac_fn=None, newton_tol=1e-10, newton_max_iter=25) -> np.ndarr…)	Um passo de Runge-Kutta totalmente implícito com Newton nos estágios.

pinneapple_simulation.numerical_solvers.isph

SPH incompressível (ISPH, Cummins & Rudman) por projeção de pressão predição-correção. [sem teste direto]

pinneapple_simulation/numerical_solvers/isph.py · 264 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
ISPHSolver	classe	(h=0.04, mass=1.0, rho0=1000.0, mu=0.02, gravity=None, boundary_lo=None, boundary_hi=None…)	Solver ISPH com projeção de pressão.

pinneapple_simulation.numerical_solvers.jax_cfd_solver

Adaptador para o jax-cfd do Google: Navier-Stokes 2D incompressível diferenciável, com condições iniciais Taylor-Green ou camada de cisalhamento. Dependência opcional (jax, jax_cfd).

pinneapple_simulation/numerical_solvers/jax_cfd_solver.py · 358 linhas · testes: 9

nome	tipo	assinatura	o que faz
JAXCFDSolver	classe	(nx=64, ny=64, viscosity=0.01, density=1.0, domain=None, max_velocity=1.0, cfl_safety=0.5…)	Navier-Stokes 2D incompressível sobre jax_cfd.base, também a partir de um ProblemSpec.
jax_cfd_available	função	() -> bool	Diz se jax e jax_cfd podem ser importados.
taylor_green_velocity_fns	função	() -> Tuple[Callable, Callable]	Funções de velocidade do vórtice de Taylor-Green 2D.
shear_layer_velocity_fns	função	(sigma=15.0, width=0.05) -> Tuple[Callable, Callable]	Funções de velocidade de uma camada de cisalhamento dupla 2D.

pinneapple_simulation.numerical_solvers.kuramoto_sivashinsky

Equação de Kuramoto-Sivashinsky 1D periódica integrada com ETDRK4 (configuração de Kassam & Trefethen, dealiasing 2/3).

pinneapple_simulation/numerical_solvers/kuramoto_sivashinsky.py · 60 linhas · testes: 7

nome	tipo	assinatura	o que faz
`KuramotoSivashinsky1D`	classe	`(n, length, dt)`	Sistema KS 1D: grade, estado inicial, passo e trajetória com ETDRK4.

pinneapple_simulation.numerical_solvers.lbm

Lattice Boltzmann D2Q9 e D3Q19 (BGK, Smagorinsky opcional) com máscaras de obstáculo (cilindro, retângulo, aerofólios NACA).

`pinneapple_simulation/numerical_solvers/lbm.py` · 739 linhas · testes: 14

nome	tipo	assinatura	o que faz
`LBMSolver`	classe	`(nx=128, ny=64, Re=100.0, u_in=0.05, rho_out=1.0, obstacle_mask=None, Cs=0.0)`	Solver LBM D2Q9 com obstáculo opcional, também a partir de um `ProblemSpec`.
`LBMSolver3D`	classe	`(nx=32, ny=32, nz=32, Re=100.0, u_in=0.05, obstacle_mask=None, Cs=0.0)`	Solver LBM D3Q19.
`cylinder_mask`	função	`(nx, ny, cx, cy, r) -> torch.Tensor`	Máscara de obstáculo circular.
`rectangle_mask`	função	`(nx, ny, x0, x1, y0, y1) -> torch.Tensor`	Máscara de obstáculo retangular.
`airfoil_naca_mask`	função	`(nx, ny, chord, aoa_deg=0.0, naca='0012') -> torch.Tensor`	Máscara aproximada de aerofólio NACA 4 dígitos centrada no domínio.
`naca4_mask`	função	`(nx, ny, naca='4412', chord=80.0, aoa_deg=0.0, x_le=None, y_le=None, n_pts=400) -> torch.…)`	Máscara de aerofólio NACA 4 dígitos com arqueamento e ângulo de ataque.
`airfoil_naca_mask_3d`	função	`(nx, ny, nz, chord, aoa_deg=0.0, naca='0012', z0=0, z1=None) -> torch.Tensor`	Máscara 3D: o perfil NACA extrudado ao longo da envergadura.

pinneapple_simulation.numerical_solvers.lbm_tunnel

Túnel de vento 3D por lattice Boltzmann em PyTorch com técnicas popularizadas pelo FluidX3D (D3Q19/D3Q27, SRT/TRT, DDF shifting, armazenamento em 16 bits), voxelização de STL e E/S VTK.

`pinneapple_simulation/numerical_solvers/lbm_tunnel.py` · 479 linhas · testes: 9

nome	tipo	assinatura	o que faz
`Units`	classe	`(m, s, kg)`	Conversão entre unidades SI e de rede.
`WindTunnelLBM`	classe	`(shape, nu, u_in=(0.0, 0.0, 0.0), velocity_set_name='D3Q19', collision='srt', smagorinsky…)`	Escoamento LBM 3D numa caixa: sólidos, passo, campos, força sobre o sólido e exportação VTK.
`velocity_set`	função	`(name='D3Q19')`	Vetores, pesos e opostos de D3Q19 ou D3Q27.
`sphere`	função	`(shape, center, radius) -> torch.Tensor`	Máscara sólida de uma esfera.
`box`	função	`(shape, lo, hi) -> torch.Tensor`	Máscara sólida de uma caixa.
`voxelize_stl`	função	`(mesh, shape, center, size, axis=0) -> torch.Tensor`	Máscara sólida de uma superfície fechada (via trimesh), escalada e centrada.
`write_vtk`	função	`(path, data, origin=(0.0, 0.0, 0.0), spacing=1.0) -> None`	Grava VTK legado binário de pontos estruturados.
`read_vtk`	função	`(path) -> dict`	Lê VTK legado de pontos estruturados (binário ou ASCII).
`drag_sphere_schiller_naumann`	função	`(re) -> float`	Coeficiente de arrasto de esfera por Schiller & Naumann.
`stokes_sphere_field`	função	`(shape, center, radius, u_inf)`	Campo de velocidade de Stokes em torno de uma esfera fixa.

pinneapple_simulation.numerical_solvers.meshfree

Métodos sem malha: colocação RBF (Kansa) para Poisson e aproximação por mínimos quadrados móveis (MLS). Levanta NotImplementedError para kernels RBF sem laplaciano analítico.

pinneapple_simulation/numerical_solvers/meshfree.py · 377 linhas · testes: 8 · palavras-chave no código (heurística; o texto acima diz o status real): levanta NotImplementedError em algum caminho

nome	tipo	assinatura	o que faz
RBFCollocationSolver	classe	(rbf='gaussian', eps=3.0, coeff=1.0, reg=1e-08)	Colocação RBF (Kansa) sem malha, também a partir de um ProblemSpec.
MLSApproximation	classe	(degree=1, support_radius=0.2, weight='quartic')	Aproximação por mínimos quadrados móveis, com avaliação e gradiente.

pinneapple_simulation.numerical_solvers.nbody_rebound

Wrapper do integrador gravitacional de N corpos REBOUND (rebound.Simulation). Dependência opcional.

pinneapple_simulation/numerical_solvers/nbody_rebound.py · 270 linhas · testes: 4

nome	tipo	assinatura	o que faz
NBodySolver	classe	(bodies, integrator='ias15', G=1.0, dt=None, move_to_com=True)	Wrapper de rebound.Simulation com integração, passo e saída padrão.

pinneapple_simulation.numerical_solvers.neighbors

Busca de vizinhos por hashing em grade uniforme para métodos de partículas, em formato comprimido; versão MVP. [sem teste direto]

pinneapple_simulation/numerical_solvers/neighbors.py · 113 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
NeighborList	classe	(j, indptr)	Lista de vizinhos comprimida (índices e ponteiros).
build_neighbor_list	função	(pos, *, h, max_neighbors=64) -> NeighborList	Constrói a lista de vizinhos 2D/3D por grade uniforme.

pinneapple_simulation.numerical_solvers.nonlinear_beam_fem

FEM de elementos espectrais para viga de Euler-Bernoulli geometricamente não linear (Von Karman) transiente, com Newmark-beta e Newton-Raphson a cada passo, e frequências naturais.

pinneapple_simulation/numerical_solvers/nonlinear_beam_fem.py · 843 linhas · testes: 2

nome	tipo	assinatura	o que faz
NonlinearBeamNewmarkFEM	classe	(num_elements, nodes_per_element, length, area, moment_of_inertia, E, density, damping_co…)	Motor de elementos espectrais + Newmark para a viga não linear: run no tempo e frequências naturais.
NonlinearBeamFEMSolver	classe	(num_elements=50, nodes_per_element=NODES_PER_ELEMENT)	Wrapper fino SolverBase/registro do solver de viga não linear.
gauss_lobatto_legendre_points	função	(n_points) -> np.ndarray	Pontos de Gauss-Lobatto-Legendre em [-1, 1].
generate_1d_mesh	função	(num_elements, nodes_per_element, dofpn, length)	Malha 1D uniforme com nós GLL em cada elemento.
gauss_legendre_quadrature	função	(n_points)	Pontos e pesos de Gauss-Legendre em [-1, 1].
lagrange_shape_functions_1d	função	(nodes_per_element, n_quad_points)	Funções de forma de Lagrange e derivadas nos pontos de quadratura.
hermite_shape_functions_1d	função	(n_quad_points, h)	Funções de forma cúbicas de Hermite e derivadas nos pontos de quadratura.
neumann_boundary_time_series	função	(num_elements, nodes_per_element, dofpn, conn, boundary_n, timesteps, delta_t, f_drive)	Pré-calcula as cargas de Neumann em cada passo de tempo.
dirichlet_boundary_values	função	(num_elements, nodes_per_element, dofpn, conn, boundary_d)	Pré-distribui os valores de Dirichlet no layout de elementos.
apply_neumann_bc	função	(i, t, load_vec_elem, boundary_n_series, ratio)	Soma a carga de Neumann ao vetor de um elemento.
apply_dirichlet_bc	função	(i, newton_iter, stiff_mat_elem, load_vec_elem, boundary_d_values)	Impõe Dirichlet na matriz tangente e no resíduo de um elemento por eliminação.

nome	tipo	assinatura	o que faz
`cantilever_frequency_roots`	função	`(n) -> np.ndarray`	Raízes de cos(x)cosh(x) = -1 para checar frequências de viga em balanço.
`solve_nonlinear_beam_transient`	função	`(boundary, L_m, A_m2, I_m4, E_Pa, rho_kg_m3, damping_coeff=0.0, axial_dist_load_N_per_m=0…)`	Resolve o problema transiente da viga não linear para um dicionário de contornos.
`acceleration_from_displacement`	função	`(w, dt) -> np.ndarray`	Derivada segunda no tempo de um histórico de deslocamento.

pinneapple_simulation.numerical_solvers.ns3d_spectral

Solver pseudoespectral de Navier-Stokes 3D incompressível na caixa periódica [0, 2π)^3 (forma rotacional, dealiasing 2/3, RK4, fator integrante para a viscosidade). [sem teste direto]

`pinneapple_simulation/numerical_solvers/ns3d_spectral.py` · 103 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`SpectralNS3D`	classe	`(N, nu, dt)`	Solver pseudoespectral NS 3D: transformadas, rotacional, projeção, RHS e laço RK4.
`taylor_green_ic`	função	`(N)`	Condição inicial de Taylor-Green 3D na grade N^3.

pinneapple_simulation.numerical_solvers.openfoam_bridge

Ponte OpenFOAM: gera casos a partir do `ProblemSpec`, roda o solver e extrai os campos como arrays para datasets. Requer OpenFOAM para rodar.

`pinneapple_simulation/numerical_solvers/openfoam_bridge.py` · 751 linhas · testes: 151

nome	tipo	assinatura	o que faz
`OpenFOAMBridge`	classe	`(solver='simpleFoam', n_cores=1, n_iterations=500, openfoam_bin=None) -> None`	Gera casos, roda o solver OpenFOAM e extrai campos.
`generate_case`	função	`(problem_spec, case_dir, mesh_cfg=None) -> Path`	Cria o diretório de caso OpenFOAM a partir de um `ProblemSpec`.
`run_openfoam`	função	`(case_dir, solver='simpleFoam', n_cores=1, n_iter=500) -> 'subprocess.CompletedProcess[st…)`	Roda um solver OpenFOAM no diretório do caso.
`extract_fields`	função	`(case_dir, time='latestTime', field_names=None) -> Dict[str, 'Any']`	Lê arrays de campos de uma rodada terminada.
`openfoam_to_dataset`	função	`(case_dir, field_names=None, time='latestTime') -> Dict[str, 'Any']`	Extrai os campos e devolve um dicionário plano de dataset.

pinneapple_simulation.numerical_solvers.operator_benchmarks

Solvers de referência e distribuições de entrada dos benchmarks clássicos de aprendizado de operadores: campos gaussianos periódicos, Burgers 1D, Navier-Stokes 2D em vorticidade, advecção-difusão e Darcy 2D.

`pinneapple_simulation/numerical_solvers/operator_benchmarks.py` · 277 linhas · testes: 132

nome	tipo	assinatura	o que faz
`FourierInterpolant`	classe	`(values, *, max_modes=None, dtype=None)`	Avalia um campo periódico de grade em pontos arbitrários com autograd.
`periodic_grf`	função	`(n_samples, shape, *, alpha, tau, sigma=None, seed=None) -> np.ndarray`	Campos gaussianos aleatórios periódicos com covariância tipo Matérn.
`threshold_coefficient`	função	`(field, high=12.0, low=3.0) -> np.ndarray`	Coeficiente constante por partes (alto/baixo) pelo sinal do campo (entradas Darcy do FNO).
`spectral_downsample`	função	`(u, factor, axes) -> np.ndarray`	Valores pontuais a cada `factor` nós de uma grade periódica.
`burgers_1d`	função	`(u0, nu, t_end, *, dt=0.0001, n_save=1, dealias=True) -> tuple[np.ndarray, np.ndarray]`	Burgers viscoso no intervalo periódico [0, 1).
`li_forcing`	função	`(n) -> np.ndarray`	Forçante de Li et al. 2021 na grade periódica n x n.
`navier_stokes_2d_vorticity`	função	`(w0, nu, t_end, *, forcing=None, dt=0.001, n_save=1, dealias=True) -> tuple[np.ndarray, n…)`	Navier-Stokes 2D em vorticidade no quadrado unitário periódico.

nome	tipo	assinatura	o que faz
advection_diffusion_2d	função	(u0, velocity, kappa, dt, n_steps) -> np.ndarray	Advecção-difusão 2D periódica, exata no tempo para os modos resolvidos.
darcy_2d	função	(a, f=1.0, *, axis_weights=(1.0, 1.0)) -> np.ndarray	Resolve -div(a grad u) = f no quadrado unitário com u = 0 na borda.
darcy_matrix	função	(a, axis_weights=(1.0, 1.0)) -> sp.csc_matrix	Matriz de volumes finitos de darcy_2d.

pinneapple_simulation.numerical_solvers.operator_datasets

Gera os datasets de aprendizado de operadores (Burgers 1D, Darcy 2D, Navier-Stokes 2D) com estudo de convergência e npz determinísticos.

pinneapple_simulation/numerical_solvers/operator_datasets.py · 309 linhas · testes: 17

nome	tipo	assinatura	o que faz
burgers_inputs	função	(n_samples, n, seed) -> np.ndarray	Condições iniciais aleatórias do benchmark de Burgers.
fourier_pad	função	(u, n_new, axes) -> np.ndarray	Interpolação trigonométrica de dados periódicos para mais pontos.
solve_burgers	função	(u0, dt=None) -> np.ndarray	Resolve Burgers com os parâmetros do dataset.
darcy_inputs	função	(n_samples, n, seed, *, tau=None, high=None, low=None) -> np.ndarray	Coeficientes de entrada do benchmark de Darcy, com opção de deslocar a distribuição.
block_average	função	(x, factor) -> np.ndarray	Médias por bloco numa grade factor vezes mais grossa.
solve_darcy	função	(a, refine=None) -> np.ndarray	Resolve Darcy numa grade refinada e devolve médias de célula.
ns_inputs	função	(n_samples, n, seed) -> np.ndarray	Vorticidades iniciais aleatórias do benchmark de Navier-Stokes.
solve_ns	função	(w0, n_solve=None, dt=None) -> np.ndarray	Resolve Navier-Stokes e devolve os valores na grade armazenada a cada tempo.
rel_l2	função	(a, b) -> np.ndarray	Erro L2 relativo por amostra.
write_npz	função	(path, arrays) -> None	Grava npz comprimido com timestamps fixos (hash depende só dos dados).
darcy_set	função	(n_samples, seed, *, n=None, tau=None, high=None, low=None, workers=1) -> tuple[np.ndarra…)	Pares (a, u) de Darcy como no dataset do zoo darcy_2d_li2021, opcionalmente com entradas deslocadas.
generate	função	(out, workers=1, only=None) -> None	Gera os três datasets (ou só um) num diretório.
convergence	função	(out, workers=1, n_samples=4, only=None) -> dict	Estudo de resolução e passo de tempo contra referências mais finas.

pinneapple_simulation.numerical_solvers.particles

Partículas lagrangianas em líquido (DEM de esferas macias, arrasto, empuxo, dispersão turbulenta) num escoamento prescrito de tanque agitado; o docstring diz que serve para tendências, não para números de projeto.

pinneapple_simulation/numerical_solvers/particles.py · 284 linhas · testes: 20

nome	tipo	assinatura	o que faz
Liquid	classe	(rho, mu)	Líquido (densidade, viscosidade).
StirredTank	classe	(R, H, D_imp, C, swirl, loop, turbulence, T_L, blades, pitch_deg, down_pumping)	Tanque cilíndrico agitado com escoamento analítico de rotação e circulação.
DEM	classe	(x, d, rho_p, liq=WATER, e=0.4, t_c=None, dt=0.0005, g=9.81, seed=0)	Esferas iguais com contato mola-amortecedor, arrasto, empuxo e paredes do tanque.
drag_factor	função	(Re) -> np.ndarray	Correção de Schiller-Naumann ao arrasto de Stokes.
terminal_velocity	função	(d, rho_p, liq=WATER, g=9.81) -> float	Velocidade de sedimentação de uma esfera isolada.

nome	tipo	assinatura	o que faz
packed_bed	função	(tank, n, d, seed=0) -> np.ndarray	Posições iniciais em leito no fundo do tanque.
suspend	função	(tank, n=20000, d=0.003, rho_p=1200.0, rpm=lambda t: min(150.0, 6.0 * t), t_end=30.0, dt=…)	Simula a suspensão de um leito enquanto o impelidor acelera.
zwietering_njs	função	(tank, d, rho_p, mass_ratio, liq=WATER, S=6.0, g=9.81) -> float	Rotação de suspensão mínima pela correlação de Zwietering (1958).

pinneapple_simulation.numerical_solvers.pipe_network

Escoamento permanente em rede de tubos com malhas (distribuição de água) pelo algoritmo do gradiente global, com a rede de dois anéis de Alperovits & Shamir e otimização de diâmetros por algoritmo genético.

pinneapple_simulation/numerical_solvers/pipe_network.py · 184 linhas · testes: 27

nome	tipo	assinatura	o que faz
Network	classe	(junctions, reservoirs, pipes, headloss, coords)	Rede de tubos: nós, reservatórios, tubos e lei de perda (Hazen-Williams ou Darcy-Weisbach).
solve_network	função	(net, tol=1e-10, max_iter=100) -> dict	Cargas, pressões, vazões, velocidades e perdas, com resíduos de continuidade e energia.
two_loop_network	função	(diameters_in=(18, 10, 16, 4, 16, 10, 10, 1), headloss='HW', roughness=None) -> Network	A rede de dois anéis de Alperovits & Shamir (1977).
optimize_diameters	função	(net, sizes_m, unit_cost, p_min, *, pop=100, generations=400, seed=0, penalty=1000000.0, …)	Diâmetros discretos de custo mínimo com pressão mínima, por algoritmo genético.

pinneapple_simulation.numerical_solvers.plane_stress_fem

Elementos finitos de estado plano de tensão (triângulos de deformação constante) sobre qualquer malha triangular, com patch test e fator de concentração de Heywood.

pinneapple_simulation/numerical_solvers/plane_stress_fem.py · 137 linhas · testes: 39

nome	tipo	assinatura	o que faz
PlaneStressSolution	classe	(V, F, u, stress, area, reaction)	Solução de estado plano: deslocamentos, tensões, reações, tensão nodal e von Mises.
heywood_kt_net	função	(d_over_w) -> float	Fator de concentração de tensão de furo central em tira tracionada (Heywood).
boundary_edges	função	(F) -> np.ndarray	Arestas que pertencem a um só triângulo.
solve_plane_stress	função	(V, F, *, E, nu, thickness, fixed, traction=(), body_force=(0.0, 0.0)) -> PlaneStressSolu…)	Resolve estado plano de tensão com nós fixos, trações e força de corpo.
patch_test	função	(V, F, *, E=1.0, nu=0.3) -> float	Patch test: desvio relativo máximo das tensões para um estado uniforme.

pinneapple_simulation.numerical_solvers.problem_runner

Gera datasets de treino PINN a partir de um `ProblemSpec`: pontos de colocação, contorno e condição inicial, e dados de referência opcionais via solvers FDM registrados.

pinneapple_simulation/numerical_solvers/problem_runner.py · 570 linhas · testes: 22

nome	tipo	assinatura	o que faz
generate_pinn_dataset	função	(problem_spec, *, n_col=None, n_bc=None, n_ic=None, n_data=0, seed=0, solver_cfg_override…)	Gera o dataset de treino PINN (colocação, contorno, inicial, dados) a partir de um `ProblemSpec`.

pinneapple_simulation.numerical_solvers.reaction_diffusion

Reação-difusão 1D periódica u_t = D u_xx + R(u) integrada com ETDRK4 (difusão exata, reação explícita).

pinneapple_simulation/numerical_solvers/reaction_diffusion.py · 65 linhas · testes: 4

nome	tipo	assinatura	o que faz
`ReactionDiffusion1D`	classe	`(reaction, diffusion, n, length, dt)`	Sistema de reação-difusão 1D: grade, estado inicial, passo e trajetória com ETDRK4.

`pinneapple_simulation.numerical_solvers.registry`

Registro de solvers por nome e família (`SolverRegistry`), `register_all` que importa cada módulo isoladamente e devolve as falhas, e o catálogo `SolverCatalog`.

`pinneapple_simulation/numerical_solvers/registry.py` · 137 linhas · testes: 206

nome	tipo	assinatura	o que faz
`SolverSpec`	classe	`(name, family, cls, description, tags)`	Registro de um solver: nome, família, classe, descrição e tags.
`SolverRegistry`	classe	`(_REG)`	Registro global de solvers: decorador `register`, `spec`, `list` e `build`.
`SolverCatalog`	classe	`(registry)`	Catálogo tipo dicionário sobre `SolverRegistry` que chama `register_all` ao ser criado.
`register_all`	função	`() -> Dict[str, str]`	Importa cada módulo de solver isoladamente e devolve as falhas de importação.

`pinneapple_simulation.numerical_solvers.shallow_water_fv`

Águas rasas 2D por volumes finitos com paredes, comportas, sensores virtuais, checagens de saúde, exportação CSV e comparação com experimentos; inclui as soluções de Ritter e Stoker.

`pinneapple_simulation/numerical_solvers/shallow_water_fv.py` · 352 linhas · testes: 4

nome	tipo	assinatura	o que faz
`Sensor`	classe	`(name, kind, x, y, z, faces, values)`	Sensor virtual (profundidade, pressão, velocidade ou força de parede) e seus valores.
`ShallowWater2D`	classe	`(nx, ny, dx, dy=None, *, g=G, rho=RHO, cfl=0.45, second_order=True)`	Águas rasas 2D com paredes, comportas, sensores, relatório de saúde, CSV e comparação.
`ritter`	função	`(x, t, h0, x0=0.0, g=G)`	Rompimento de barragem em leito seco (Ritter 1892).
`stoker`	função	`(x, t, h0, h1, x0=0.0, g=G)`	Rompimento de barragem em leito molhado (Stoker 1957).

`pinneapple_simulation.numerical_solvers.solid_fem`

Elasticidade linear 3D com hexaedros de 8 nós e modos incompatíveis (Wilson-Taylor), malhas de caixa, engastes, cargas, tensões nodais, figura de pós-processamento e exportação para CalculiX.

`pinneapple_simulation/numerical_solvers/solid_fem.py` · 415 linhas · testes: 47

nome	tipo	assinatura	o que faz
`HexMesh`	classe	`(nodes, elements, shape, size)`	Malha de hexaedros com seleção de nós e faces de contorno.
`FEMResult`	classe	`(mesh, u, stress, reactions, info)`	Resultado FEM 3D: deslocamentos, tensões, reações, von Mises e superfície.
`SolidFEM`	classe	`(mesh, E, nu, incompatible, _fixed, _f)`	Problema de elasticidade 3D: engastes, cargas, pressão, solução e exportação CalculiX.
`box_mesh`	função	`(L, W, H, nx, ny, nz, origin=(0.0, 0.0, 0.0)) -> HexMesh`	Malha estruturada de hexaedros de uma caixa.
`elasticity_matrix`	função	`(E, nu) -> np.ndarray`	Matriz constitutiva isotrópica 6x6 (Voigt) a partir de E e nu.
`von_mises`	função	`(s) -> np.ndarray`	Tensão de von Mises de tensões em notação de Voigt.
`fea_figure`	função	`(res, field='von_mises', scale=None, bands=12, title='', unit='MPa', factor=1e-06, view=(…)`	Figura da malha deformada colorida por faixas com legenda de pós-processador.

pinneapple_simulation.numerical_solvers.spectral

Solver espectral MVP: Poisson 2D periódico via FFT.

pinneapple_simulation/numerical_solvers/spectral.py · 49 linhas · testes: 5

nome	tipo	assinatura	o que faz
SpectralSolver	classe	()	Wrapper registrado de poisson_periodic_fft.
poisson_periodic_fft	função	(f, *, Lx=1.0, Ly=1.0, eps=1e-12) -> torch.Tensor	Resolve o Poisson 2D periódico via FFT.

pinneapple_simulation.numerical_solvers.sph

SPH fracamente compressível (WCSPH) MVP. [sem teste direto]

pinneapple_simulation/numerical_solvers/sph.py · 167 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
SPHState	classe	(pos, vel, rho, p)	Estado SPH: posição, velocidade, densidade e pressão.
SPHSolver	classe	(h=0.04, mass=1.0, rho0=1000.0, c0=20.0, mu=0.02, gravity=None, boundary_lo=None, boundar…)	Solver WCSPH MVP com passo e saída padrão.

pinneapple_simulation.numerical_solvers.sph_boundaries

Ajudantes de contorno para SPH (MVP): caixa alinhada aos eixos, força de repulsão de parede e reflexão rígida. [sem teste direto]

pinneapple_simulation/numerical_solvers/sph_boundaries.py · 88 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
BoxBoundary	classe	(lo, hi)	Caixa alinhada aos eixos usada como contorno.
wall_repulsion_force	função	(pos, vel, boundary, *, k=50.0, d0=0.02) -> torch.Tensor	Força de repulsão suave quando a partícula chega perto da parede.
reflect_box	função	(pos, vel, lo, hi, *, restitution=0.0) -> Tuple[torch.Tensor, torch.Tensor]	Reflexão rígida das partículas nas paredes da caixa.

pinneapple_simulation.numerical_solvers.ssa

Análise espectral singular (SSA) de séries 1D. [sem teste direto]

pinneapple_simulation/numerical_solvers/ssa.py · 73 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
SSASolver	classe	(L=64, r=10)	Wrapper registrado de ssa_decompose.
ssa_decompose	função	(x, L=64, r=10) -> Tuple[np.ndarray, np.ndarray, np.ndarray]	Componentes reconstruídas, valores singulares e U da SSA.

pinneapple_simulation.numerical_solvers.sst

Transformada sincronizada (synchrosqueezing) simplificada baseada em STFT; o docstring a chama de MVP. [sem teste direto]

pinneapple_simulation/numerical_solvers/sst.py · 89 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
SSTSolver	classe	(fs=1.0, nperseg=128, noverlap=None, nfft=None)	Wrapper registrado de sst_stft_1d.
sst_stft_1d	função	(x, *, fs=1.0, nperseg=128, noverlap=None, nfft=None, eps=1e-08) -> Tuple[np.ndarray, np.…]	Transformada tipo synchrosqueezing a partir da STFT; devolve espectro, frequências e tempos.

pinneapple_simulation.numerical_solvers.stable_fluids

Stable Fluids (Stam): escoamento incompressível 2D incondicionalmente estável em grade periódica, com advecção semi-lagrangiana, difusão implícita, projeção e confinamento de vorticidade.

pinneapple_simulation/numerical_solvers/stable_fluids.py · 198 linhas · testes: 17

nome	tipo	assinatura	o que faz
StableFluidsState	classe	(velocity, scalars, t)	Estado: velocidade, escalares transportados e tempo.
StableFluids2D	classe	(nx, ny, *, length=(2 * np.pi, 2 * np.pi), viscosity=0.0, confinement=0.0, scalar_dissipa…)	Caixa periódica 2D com as etapas do Stable Fluids e laço de simulação.
stable_time_step	função	(max_velocity, dx, *, courant=0.5, viscosity=0.0, ndim=2, implicit_diffusion=True) -> flo…)	Passo de tempo pelo CFL advectivo, limitado pela difusão explícita se for o caso.

pinneapple_simulation.numerical_solvers.stiff_kinetics_ode

Wrapper de EDOs rígidas para redes de reação (cinética química ou biológica) usando métodos implícitos do SciPy (Radau por padrão).

pinneapple_simulation/numerical_solvers/stiff_kinetics_ode.py · 100 linhas · testes: 7

nome	tipo	assinatura	o que faz
solve_stiff_reaction_network	função	(rate_fn, y0, t_span, t_eval=None, jac_fn=None, method='Radau', rtol=1e-09, atol=1e-12, c…)	Integra dy/dt = rate_fn(t, y) com método implícito rígido e corta valores negativos.

pinneapple_simulation.numerical_solvers.stl

Decomposição sazonal-tendência por Loess (STL) via statsmodels, que é importado no topo do módulo. [sem teste direto]

pinneapple_simulation/numerical_solvers/stl.py · 40 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
STLSolver	classe	(period=24, robust=True)	Wrapper registrado da STL do statsmodels.

pinneapple_simulation.numerical_solvers.symbolic_fd

Diferenças finitas para qualquer sistema de equações lido por `parse_system` numa caixa: estacionário (Newton) ou método das linhas (BDF), em 1D, 2D ou 3D.

pinneapple_simulation/numerical_solvers/symbolic_fd.py · 498 linhas · testes: 17

nome	tipo	assinatura	o que faz
FDError	classe	()	Erro: o sistema não cabe neste esquema de diferenças finitas (e por quê).
FDResult	classe	(axes, fields, t, info)	Resultado: eixos, campos, tempos e informações, com estado final e pontos.
FDOperator	classe	()	Sistema compilado numa grade: lados direitos, condições de face e estado inicial.
compile_system	função	(system, bounds, *, n=64, coefficients=None, keep=()) -> FDOperator	Compila um sistema lido para diferenças finitas.
initial_state	função	(op, t0) -> dict[tuple[str, bool], np.ndarray]	Valores iniciais de cada campo na grade com as faces impostas.
solve_fd	função	(system, bounds, *, n=64, n_save=41, coefficients=None, relax=20000.0, rtol=1e-06, atol=1…)	Resolve o sistema numa caixa (Newton estacionário ou BDF) e devolve `FDResult`.

pinneapple_simulation.numerical_solvers.thermal_channel_2d

CFD 2D transiente de canal aquecido (Navier-Stokes incompressível + energia) em grade MAC, com atuadores variáveis no tempo (velocidade de entrada e potência do aquecedor). [sem teste direto]

pinneapple_simulation/numerical_solvers/thermal_channel_2d.py · 231 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ChannelConfig`	classe	`(nx, ny, L, Re, Pr, heater, dt, gamma_safety, record_every)`	Configuração do canal aquecido: grade, Re, Pr, aquecedor, passo.
`ThermalChannel2D`	classe	`(cfg=None)`	Simulador do canal aquecido: passo, campos, grandezas de interesse e laço.

pinneapple_simulation.numerical_solvers.thermal_channel_datasets

Gera e verifica o dataset do zoo `thermal_channel_cfd`: trajetórias atuadas do canal aquecido e estudo de convergência.

pinneapple_simulation/numerical_solvers/thermal_channel_datasets.py · 181 linhas · testes: 2

nome	tipo	assinatura	o que faz
`verify`	função	`() -> dict`	Verificação: Poiseuille, Nusselt desenvolvido e estudo em três grades.
`make_actuation`	função	`(family, seed)`	Gera os atuadores de uma trajetória por família (treino ou não vistas).
`eval_act`	função	`(spec, t, which) -> float`	Avalia um atuador (linear por partes ou chirp) num tempo.
`simulate`	função	`(args) -> dict`	Uma trajetória: aquecimento e horizonte gravado.
`generate`	função	`(out, workers=1, name='thermal_channel_cfd') -> dict`	Roda o plano e grava os npz e o `meta.json` do dataset.

pinneapple_simulation.numerical_solvers.tvd_advection

Esquema upwind TVD de 2ª ordem com limitador de van Leer para advecção escalar 1D.

pinneapple_simulation/numerical_solvers/tvd_advection.py · 69 linhas · testes: 5

nome	tipo	assinatura	o que faz
`van_leer_limiter`	função	`(r) -> np.ndarray`	Limitador de fluxo TVD de van Leer.
`tvd_advection_rhs`	função	`(f, v, dz) -> np.ndarray`	Lado direito TVD upwind de 2ª ordem para df/dt = -v df/dz.

pinneapple_simulation.numerical_solvers.vmd

Decomposição variacional de modos (VMD) para sinais 1D. [sem teste direto]

pinneapple_simulation/numerical_solvers/vmd.py · 143 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`VMDSolver`	classe	`(K=4, alpha=2000.0, tau=0.0, DC=False, init='uniform', tol=1e-07, max_iter=500)`	Wrapper registrado de `vmd_1d`.
`vmd_1d`	função	`(x, *, K=4, alpha=2000.0, tau=0.0, DC=False, init='uniform', tol=1e-07, max_iter=500) -> …)`	VMD 1D; devolve os modos e as frequências centrais.

pinneapple_simulation.numerical_solvers.water_hammer_moc

Golpe de aríete num sistema reservatório-tubo-válvula pelo método das características, com as fórmulas de Korteweg, Joukowsky e Michaud.

pinneapple_simulation/numerical_solvers/water_hammer_moc.py · 130 linhas · testes: 17

nome	tipo	assinatura	o que faz
`wave_speed`	função	`(K, rho, D, e, E, c1=1.0) -> float`	Velocidade da onda de pressão num tubo elástico (Korteweg).
`joukowsky_head`	função	`(a, dv) -> float`	Sobrepressão de uma mudança instantânea de velocidade (Joukowsky).

nome	tipo	assinatura	o que faz
`michaud_head`	função	`(a, dv, length, t_close) -> float`	Estimativa de Michaud para fechamento lento.
`valve_closure_moc`	função	`(*, length, diameter, wave_speed, friction_factor, q0, h_reservoir, h_downstream=0.0, t_c…)`	Cargas e vazões transientes após o fechamento da válvula, por características.

pinneapple_simulation.numerical_solvers.wavelet

Atributos por decomposição wavelet (coeficientes de detalhe) via PyWavelets, que é importado no topo do módulo. [sem teste direto]

`pinneapple_simulation/numerical_solvers/wavelet.py` · 63 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`WaveletSolver`	classe	`(wavelet='db4', level=4, mode='periodization')`	Wrapper registrado de `wavedec_features`.
`wavedec_features`	função	`(x, wavelet='db4', level=4, mode='periodization') -> Tuple[np.ndarray, Dict[str, Any]]`	Concatena os coeficientes de detalhe wavelet como atributos.

pinneapple_simulation.numerical_solvers.xtfc_general

X-TFC geral numa caixa: qualquer número de dimensões e campos, restrições de valor e derivada nas faces, resíduos lineares ou não lineares.

`pinneapple_simulation/numerical_solvers/xtfc_general.py` · 335 linhas · testes: 10

nome	tipo	assinatura	o que faz
`Constraint`	classe	`(dim, at, order)`	Restrição de derivada de uma ordem numa face de uma dimensão.
`XTFCProblem`	classe	`(bounds, fields, constraint_fn, residual, derivatives, linear)`	Problema diferencial para X-TFC: limites, campos, restrições e resíduo.
`XTFCResult`	classe	`(beta, iterations, converged, step_norms, lstsq_seconds, total_seconds, n_neurons, _probl…)`	Resultado X-TFC: coeficientes, convergência, predição, resíduo e erro das restrições.
`activation_derivative`	função	`(name, z, n) -> np.ndarray`	Derivada de ordem n da ativação (tanh ou logística).
`switching_functions`	função	`(constraints) -> np.ndarray`	Coeficientes das funções de troca das restrições numa dimensão.
`solve_xtfc`	função	`(problem, points, *, n_neurons, activation='tanh', weight_range=(-1.0, 1.0), seed=0, max_…)`	Resolve o problema por X-TFC nos pontos de treino.
`grid`	função	`(bounds, n, kind='uniform') -> np.ndarray`	Grade tensorial uniforme ou de Chebyshev-Gauss-Lobatto.

pinneapple_simulation.numerical_solvers.xtfc_ivp

X-TFC (Theory of Functional Connections extrema) para sistemas de EDOs de valor inicial, com colocação de Chebyshev e mínimos quadrados iterados. [sem teste direto]

`pinneapple_simulation/numerical_solvers/xtfc_ivp.py` · 438 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`XTFCIVPSolver`	classe	`(n_basis=50, n_collocation=100, activation='tanh', max_iter=50, tol=1e-10, seed=0, colloc…)`	Wrapper fino `SolverBase`/registro de `solve_xtfc_ode`.
`chebyshev_gauss_lobatto`	função	`(n, lo=-1.0, hi=1.0) -> Array`	Pontos de Chebyshev-Gauss-Lobatto num intervalo.
`xtfc_predict`	função	`(t_query, w, b, beta, y0, activation, t0, tf) -> Array`	Avalia a expressão TFC treinada em tempos arbitrários a partir dos parâmetros salvos.
`solve_xtfc_ode`	função	`(rhs_fn, y0, t_span, n_basis=50, n_collocation=100, activation='tanh', max_iter=50, tol=1…)`	Resolve um PVI de EDOs por X-TFC.

pinneapple_simulation.numerical_solvers.xtfc_pde

X-TFC para EDPs 1D em espaço + tempo num retângulo (calor 1D e Burgers 1D) com condição inicial e duas Dirichlet.

pinneapple_simulation/numerical_solvers/xtfc_pde.py · 584 linhas · testes: 6

nome	tipo	assinatura	o que faz
XTFCPDESolver	classe	(pde='burgers_1d', n_basis=60, n_collocation_x=25, n_collocation_t=25, activation='tanh',…)	Wrapper fino SolverBase/registro de solve_xtfc_pde.
xtfc_pde_predict	função	(x_query, t_query, wx, wt, b, beta, pde, activation, x_span, t_span) -> Array	Avalia a expressão TFC 2D treinada em pontos (x, t).
solve_xtfc_pde	função	(pde, x_span=None, t_span=None, n_basis=60, n_collocation_x=25, n_collocation_t=25, activ…)	Resolve calor 1D ou Burgers 1D por X-TFC.

pinneapple_simulation.numerical_solvers.xtfc_subdomain

X-TFC encadeado em subdomínios de largura logarítmica para EDOs de valor inicial extremamente rígidas.

pinneapple_simulation/numerical_solvers/xtfc_subdomain.py · 395 linhas · testes: 7

nome	tipo	assinatura	o que faz
XTFCSubdomainSolver	classe	(n_subdomains=12, first_width=None, spacing='log', n_basis=30, n_collocation=40, activati…)	Wrapper fino SolverBase/registro de solve_xtfc_subdomains.
log_subdomain_boundaries	função	(t0, tf, n_subdomains, first_width=None) -> Array	Fronteiras de subdomínios com larguras crescentes em escala logarítmica.
solve_xtfc_subdomains	função	(rhs_fn, y0, t_span, n_subdomains=12, first_width=None, spacing='log', n_basis=30, n_coll…)	Resolve um PVI rígido por subdomínios encadeados com continuidade.

pinneapple_simulation.particle_dynamics

Simulação dinâmica diferenciável em PyTorch puro: corpos rígidos, MPM (MLS-MPM) e SPH; reexporta também os parâmetros de solo/roda de Bekker-Wong.

pinneapple_simulation/particle_dynamics/__init__.py · 61 linhas · **sem teste direto**

pinneapple_simulation.particle_dynamics._utils

Utilitário privado que monta o vetor de gravidade (padrão -9,81 no eixo y) usado pelos simuladores de partículas. [sem teste]

pinneapple_simulation/particle_dynamics/_utils.py · 25 linhas · **sem teste direto**

pinneapple_simulation.particle_dynamics.geo_mpm

MPM para geomecânica (colapso granular, deslizamentos, rompimento de barragem, sólidos moles): MLS-MPM com APIC em unidades físicas, materiais areia (Drucker-Prager), fluido e elástico, com quadros no formato do GeoTaichi.

pinneapple_simulation/particle_dynamics/geo_mpm.py · 355 linhas · testes: 4

nome	tipo	assinatura	o que faz
Sand	classe	(friction_deg, E, nu, rho)	Material areia: plasticidade de Drucker-Prager sem coesão, com ângulo de atrito, E, nu e densidade.
Fluid	classe	(K, rho, gamma)	Material fluido fracamente compressível, pressão de J (tipo Tait).
Elastic	classe	(E, nu, rho)	Material sólido hiperelástico corrotacional fixo.
GeoMPM	classe	(domain, dx, gravity=None, floor='friction', walls='slip', wall_friction=0.5, cfl=0.3, fl…)	Simulador MLS-MPM 2D/3D em unidades físicas com corpos, paredes (slip, sticky ou atrito), passo estável por CFL e gravação de quadros.

nome	tipo	assinatura	o que faz
box	função	(lo, hi)	Corpo retangular (ou caixa): devolve um teste de pertencimento para add_body.
write_particle_frame	função	(out, i, fr) -> str	Grava um quadro de partículas em .npz no layout do GeoTaichi.
read_particle_frames	função	(out, kind='MPM', allow_pickle=False) -> list[dict]	Lê os quadros de uma corrida deste módulo ou do GeoTaichi (MPM ou DEM); allow_pickle só para arquivos confiáveis.
runout	função	(position, axis=0, floor_band=None, quantile=0.995) -> float	Posição da frente de um depósito ao longo de um eixo, por um quantil alto robusto a partículas soltas.

pinneapple_simulation.particle_dynamics.molecular_dynamics

Dinâmica molecular NVE com potencial de Lennard-Jones par a par e integração velocity-Verlet, em PyTorch.

pinneapple_simulation/particle_dynamics/molecular_dynamics.py · 274 linhas · testes: 8

nome	tipo	assinatura	o que faz
MDState	classe	(pos, vel)	Contêiner de posições e velocidades de um sistema de N partículas.
MolecularDynamicsSimulator	classe	(epsilon=1.0, sigma=1.0, mass=1.0, cutoff=None, box_size=None, dim=3) -> None	Simulador mínimo NVE de Lennard-Jones com velocity-Verlet, raio de corte e caixa periódica opcionais.
lj_potential	função	(dist, epsilon, sigma) -> torch.Tensor	Potencial de par de Lennard-Jones U(r) = 4ε((σ/r)^12 - (σ/r)^6).
lj_force_magnitude	função	(dist, epsilon, sigma) -> torch.Tensor	Módulo analítico da força radial de Lennard-Jones, F(r) = -dU/dr.

pinneapple_simulation.particle_dynamics.mpm

Método do Ponto Material (MLS-MPM) diferenciável para sólidos elásticos, fluidos viscosos e neve com plasticidade de estiramento crítico, com etapas P2G, atualização da grade e G2P.

pinneapple_simulation/particle_dynamics/mpm.py · 481 linhas · testes: 151

nome	tipo	assinatura	o que faz
MPMState	classe	(pos, vel=None, F=None, C=None, Jp=None) -> None	Contêiner do estado das partículas MPM: posição, velocidade, gradiente de deformação F, matriz afim C e Jp.
MPMSimulator	classe	(grid_resolution=64, dt=0.0001, material='elastic', dim=2, E=10000.0, nu=0.2, rho=1.0, gr…)	Simulador MPM diferenciável (nn.Module) com materiais elástico, fluido ou neve, em 2D/3D.

pinneapple_simulation.particle_dynamics.particles

Simulação por partículas: SPH fracamente compressível (equação de Tait, viscosidade laminar) sobre uma classe base com busca de vizinhos O(N²); a base levanta NotImplementedError em forward, e o DEM citado no docstring não está implementado aqui.

pinneapple_simulation/particle_dynamics/particles.py · 334 linhas · testes: 151 · palavras-chave no código (heurística; o texto acima diz o status real): levanta NotImplementedError em algum caminho

nome	tipo	assinatura	o que faz
ParticleSystem	classe	(n_particles, dim=2, device='cpu') -> None	Base de sistemas de partículas com busca de vizinhos por força bruta O(N²); forward levanta NotImplementedError.
SPHParticles	classe	(n_particles, smoothing_length, rho0=1000.0, c0=10.0, nu=0.0001, dim=2, gamma=7.0, gravit…)	SPH fracamente compressível: densidade por soma de kernel, pressão de Tait, viscosidade e gravidade.

pinneapple_simulation.particle_dynamics.rigid_body

Dinâmica de corpos rígidos diferenciável em 2D/3D com integração Euler simplética, para aprendizado de robôs e acoplamento com PINNs.

pinneapple_simulation/particle_dynamics/rigid_body.py · 306 linhas · testes: 151

nome	tipo	assinatura	o que faz
RigidBodyState	classe	(n_bodies=1, dim=2, device='cpu', dtype=torch.float32) -> None	Contêiner mutável do estado (posição, orientação, velocidades) de vários corpos rígidos.
RigidBody	classe	(mass, inertia, dim=2) -> None	Corpo rígido único com massa, inércia e integrador simplético.
RigidBodySystem	classe	(bodies, gravity=(0.0, -9.81, 0.0)) -> None	Sistema de vários corpos rígidos com gravidade opcional.

pinneapple_simulation.particle_dynamics.terramechanics

Dataclasses de parâmetros de solo e roda para a interação roda-solo de Bekker-Wong.

pinneapple_simulation/particle_dynamics/terramechanics.py · 48 linhas · testes: 28

nome	tipo	assinatura	o que faz
SoilParams	classe	(c, phi_deg, K, k_c, k_phi, n, rho, g, a0, a1)	Parâmetros de solo de Bekker-Wong (coesão, atrito, módulos de afundamento, densidade).
WheelParams	classe	(R, b, n_wheels, mass_rover)	Geometria de roda rígida e massa do rover, com peso por roda.

pinneapple_simulation.pipelines

Pacote de pipelines de ponta a ponta que encadeiam ferramentas externas (CAD, malhador, solver). [sem teste direto do __init__]

pinneapple_simulation/pipelines/__init__.py · 5 linhas · **sem teste direto**

pinneapple_simulation.pipelines.freecad_openfoam

Pipelines headless e reproduzíveis de um modelo FreeCAD paramétrico a um resultado OpenFOAM checado; reexporta especificação, estudos, varredura e surrogate.

pinneapple_simulation/pipelines/freecad_openfoam/__init__.py · 29 linhas · testes: 14

pinneapple_simulation.pipelines.freecad_openfoam.__main__

Linha de comando `python -m pinneapple_simulation.pipelines.freecad_openfoam` com `run`, `sweep` e `build`. [sem teste]

pinneapple_simulation/pipelines/freecad_openfoam/__main__.py · 46 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
main	função	(argv=None) -> int	Interpreta os subcomandos run, sweep e build e devolve o código de saída.

pinneapple_simulation.pipelines.freecad_openfoam.cad

Passo de CAD: documento FreeCAD paramétrico de entrada, faces com patches de contorno nomeados por regras na saída, STL multirregião; backend real via `freecadcmd` ou um `FakeFreeCAD` em NumPy para testes.

pinneapple_simulation/pipelines/freecad_openfoam/cad.py · 365 linhas · testes: 14

nome	tipo	assinatura	o que faz
PatchSpec	classe	(name, where, kind)	Um patch de contorno: a regra que seleciona suas faces e o tipo (inlet, outlet, wall, symmetry).
Face	classe	(index, type, area, centre, points, triangles)	Uma face do sólido exportado: tipo, área, centro, pontos e triângulos, com normal.
CadGeometry	classe	(faces, info)	Faces de um sólido em mm com as informações do backend (volume, validade, arquivos).
FreeCADBackend	classe	(freecadcmd=None, timeout_s=600.0)	Backend que roda o FreeCAD headless via freecadcmd, com timeout por tarefa.

nome	tipo	assinatura	o que faz
`FakeFreeCAD`	classe	`()`	Backend falso que constrói os modelos em NumPy com as mesmas faces e parâmetros, sem FreeCAD.
`rule_eval`	função	`(expr, names, functions=None) -> Any`	Avalia uma regra de patch restrita (números, nomes, aritmética, comparações, lógica e algumas funções de `math`).
`assign_patches`	função	`(geom, patches, parameters=None) -> dict[str, list[int]]`	Atribui cada face ao primeiro patch cuja regra ela satisfaz; falha se sobrar face sem patch.
`write_stl`	função	`(path, geom, assignment, scale=0.001) -> Path`	Escreve um STL ASCII com um `solid` por patch, convertido de mm para m.
`find_freecadcmd`	função	`() -> str | None`	Procura o `freecadcmd` (variável de ambiente, AppImage em `/opt/freecad` ou PATH).
`get_backend`	função	`(name='auto')`	Escolhe o backend 'freecad', 'fake' ou 'auto'.

`pinneapple_simulation.pipelines.freecad_openfoam.case`

Modelos de caso OpenFOAM para a geometria malhada: `simpleFoam` ou `pimpleFoam` incompressível, laminar ou k-omega SST, com escalar passivo opcional.

`pinneapple_simulation/pipelines/freecad_openfoam/case.py` · 221 linhas · testes: 14

nome	tipo	assinatura	o que faz
`ScalarSpec`	classe	`(name, D, inlet, walls, initial)`	Escalar passivo: difusividade, valor na entrada, condições nas paredes e valor inicial.
`Physics`	classe	`(velocity, nu, rho, turbulence, solver, iterations, residual_tol, end_time, dt, max_coura…)`	Física do escoamento incompressível: velocidade, viscosidade, densidade, turbulência, solver, iterações, passo e escalar.
`write_internal_case`	função	`(case, kinds, inlet_normals, physics, procs=1, size=None) -> dict[str, Any]`	Escreve campos `0.orig`, dicionários de `constant/` e `system/` e o `Allrun` de um caso de escoamento interno.

`pinneapple_simulation.pipelines.freecad_openfoam.freecad_scripts.fc_job`

Script que roda dentro do Python do FreeCAD (`freecadcmd`), sem importar o PINNeAPPle: constrói os documentos paramétricos dos modelos e exporta STEP e faces trianguladas. [sem teste]

`pinneapple_simulation/pipelines/freecad_openfoam/freecad_scripts/fc_job.py` · 175 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`sheet`	função	`(doc, parameters)`	Cria a planilha `Params` com um alias por parâmetro.
`cylinder`	função	`(doc, name, radius, height, placement)`	Cria um `Part::Cylinder` com raio, altura e posição dados por expressões.
`build_pipe_bend`	função	`(doc, p)`	Constrói o modelo de curva de tubo: perna reta, curva de raio Rc e perna de saída.
`build_heated_pipe`	função	`(doc, p)`	Constrói o tubo reto com trecho de entrada não aquecido e trecho aquecido.
`build_spheroid`	função	`(doc, p)`	Constrói um esferoide centrado na origem (esfera quando L = D).
`set_parameters`	função	`(doc, parameters)`	Define os valores dos aliases das planilhas de um documento.
`faces_of`	função	`(shape, tol)`	Triangula as faces de um sólido e devolve tipo, área, centro, pontos e triângulos.
`main`	função	`()`	Lê a tarefa (build ou export) e grava o resultado em JSON.

`pinneapple_simulation.pipelines.freecad_openfoam.mesh`

Malha da geometria exportada: blockMesh + snappyHexMesh, ou `cartesianMesh` do cfMesh quando instalado, com decomposição e scripts `Allmesh`/`Allrun`.

`pinneapple_simulation/pipelines/freecad_openfoam/mesh.py` · 193 linhas · testes: 14

nome	tipo	assinatura	o que faz
`MeshSpec`	classe	`(cells_across, length, level, mesher, feature_angle, max_non_ortho)`	Especificação da malha: células através de um comprimento, nível de superfície, malhador, ângulo de feature e não ortogonalidade máxima.
`has_cfmesh`	função	`(bashrc=None) -> bool`	Diz se o cfMesh (`cartesianMesh`) está instalado.
`split_procs`	função	`(procs, size) -> tuple[int, int, int]`	Divide o número de processos em (nx, ny, nz) pelos lados mais longos da caixa.
`decompose_dict`	função	`(case, procs, size) -> None`	Escreve um `decomposeParDict` hierárquico.
`allmesh_script`	função	`(procs, mesher='snappy') -> str`	Gera o script `Allmesh` para snappyHexMesh ou cfMesh.
`allrun_script`	função	`(solver, procs, scalar=False) -> str`	Gera o script `Allrun` que copia `0.orig` e roda o solver, em paralelo se pedido.
`write_internal_mesh`	função	`(case, stl, patches, inside, cell_size, spec, procs=1, bashrc=None) -> dict[str, Any]`	Escreve os dicionários e o `Allmesh` de uma malha interna a partir do STL fechado.

`pinneapple_simulation.pipelines.freecad_openfoam.models`

Modelos FreeCAD paramétricos que o pipeline traz (pipe_bend, heated_pipe, spheroid), com regras de patch padrão e um gêmeo NumPy para testes. [sem teste direto]

`pinneapple_simulation/pipelines/freecad_openfoam/models.py` · 196 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Model`	classe	`(name, title, flow, solid, parameters, patches, fake)`	Modelo paramétrico: nome, tipo de escoamento, sólido, parâmetros com limites, regras de patch e gêmeo falso.
`build_document`	função	`(template, output, parameters=None, backend='auto') -> dict[str, Any]`	Escreve o documento paramétrico de um modelo com FreeCAD ou um documento falso.

`pinneapple_simulation.pipelines.freecad_openfoam.pipeline`

Descreve um estudo FreeCAD -> OpenFOAM (`PipelineSpec`) e o transforma num workflow de `pinneapple_tools.automation` com passos cad, case, mesh, solve, post e verify, retomável. [sem teste direto]

`pinneapple_simulation/pipelines/freecad_openfoam/pipeline.py` · 231 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`PipelineSpec`	classe	`(id, title, template, document, solid, flow, parameters, inputs, patches, physics, mesh, …)`	Um estudo FreeCAD -> OpenFOAM: modelo, parâmetros, patches, física, malha, seções, grandezas derivadas e referências.
`evaluate`	função	`(expr, names) -> Any`	Avalia um número ou uma expressão com as funções de referência.
`resolve_physics`	função	`(physics, inputs) -> dict[str, Any]`	Avalia em ordem as expressões dos valores físicos a partir das entradas.
`run_pipeline`	função	`(spec, folder, inputs=None, approve=True, approver='pipeline')`	Roda ou retoma o estudo numa pasta com as entradas dadas e devolve o registro da rodada.

`pinneapple_simulation.pipelines.freecad_openfoam.post`

Pós-processamento de um caso de escoamento interno terminado: pressões, fluxos, seções, forças, y+, Courant e figuras, lido com o leitor de volumes finitos de `pinneapple_check`. [sem teste direto]

`pinneapple_simulation/pipelines/freecad_openfoam/post.py` · 322 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Section`	classe	`(name, origin, normal, radius)`	Disco de amostragem (centro, normal, raio) para médias numa seção transversal.
`CaseResult`	classe	`(case, scalar=None)`	Campos do último tempo de um caso na malha (centros, volumes, U, p, escalar, patches) com amostragem pela célula mais próxima.

nome	tipo	assinatura	o que faz
`section_average`	função	`(res, sec) -> dict[str, float]`	Médias de pressão, fluxo e escalar na parte do disco dentro do fluido.
`wall_loads`	função	`(case, walls, nu, rho) -> dict[str, Any]`	Forças de pressão e viscosas nas paredes e y+ das primeiras células.
`read_case`	função	`(case, kinds, rho, sections=(), scalar=None, nu=1e-06) -> dict[str, Any]`	Todas as grandezas medidas de um caso terminado (pressões em Pa).
`plane_figure`	função	`(res, path, origin, normal, title, rho, scalar_label=None, n=360) -> Path`	Figura de velocidade, pressão e escalar num plano através do domínio.
`mesh_figure`	função	`(res, path, origin, normal, title) -> Path`	Figura das células cortadas por um plano, coloridas pelo tamanho.
`geometry_figure`	função	`(geom, assignment, path, title) -> Path`	Figura 3-D do sólido exportado com faces coloridas por patch.

pinneapple_simulation.pipelines.freecad_openfoam.report

Validação de um estudo: o mesmo caso em malhas cada vez mais finas, extrapolação de Richardson (GCI), erro contra a referência, tabela e figuras. [sem teste direto]

`pinneapple_simulation/pipelines/freecad_openfoam/report.py` · 154 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`mesh_study`	função	`(factory, resolutions, folder, reference, inputs=None, h_of=None, **kw) -> dict[str, Any]`	Roda o caso em várias resoluções, faz extrapolação de Richardson e compara com a referência.
`mesh_study_figure`	função	`(studies, path, labels=None) -> Path`	Figura por estudo de malha: valor contra tamanho de célula, referência, tolerância e valor extrapolado.
`sweep_figure`	função	`(rows, inputs, output, study, path, unit='') -> Path`	Figura da varredura e das previsões do surrogate contra o solver nos casos de treino e de teste.
`validation_markdown`	função	`(studies) -> str`	Tabela Markdown de validação (uma linha por malha e o valor extrapolado).

pinneapple_simulation.pipelines.freecad_openfoam.steps

As três ações do pipeline que não são comandos OpenFOAM puros (cad, prepare, post) como funções de parâmetros e pasta, embrulhadas pelos adaptadores de automação.

`pinneapple_simulation/pipelines/freecad_openfoam/steps.py` · 340 linhas · testes: 14

nome	tipo	assinatura	o que faz
`point_inside`	função	`(V, F, p) -> bool`	Diz se um ponto está dentro de uma malha triangular fechada (paridade de cruzamentos em três raios).
`auto_procs`	função	`(cells, cores=None, per_proc=20000.0) -> int`	Número de processos para uma malha: um por bloco de células, no máximo os núcleos disponíveis.
`cad_step`	função	`(params, step_dir) -> dict[str, Any]`	Define parâmetros, recalcula, exporta, atribui patches e grava `geometry.stl` e `geometry.png`.
`prepare_step`	função	`(params, step_dir) -> dict[str, Any]`	Gera dicionários de malha, o caso e os scripts `Allmesh`/`Allrun`.
`post_step`	função	`(params, step_dir) -> dict[str, Any]`	Grandezas medidas, figuras, grandezas derivadas e erros contra referências de um caso terminado.

pinneapple_simulation.pipelines.freecad_openfoam.studies

Estudos validados do pipeline como fábricas de `PipelineSpec` (curva de tubo, tubo aquecido, esfera), cada um comparado a forma fechada ou correlação publicada. [sem teste direto]

`pinneapple_simulation/pipelines/freecad_openfoam/studies.py` · 116 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
pipe_bend	função	(cells_across=16, backend='auto', procs='auto', bend_window=True) -> PipelineSpec	Fábrica do estudo de escoamento laminar numa curva de tubo.
heated_pipe	função	(cells_across=16, backend='auto', procs='auto') -> PipelineSpec	Fábrica do estudo de entrada térmica laminar com parede a temperatura constante.
sphere	função	(surface_level=4, backend='auto', procs='auto') -> PipelineSpec	Fábrica do estudo de esfera em escoamento uniforme a baixo Reynolds.

pinneapple_simulation.pipelines.freecad_openfoam.sweep

Varreduras/DoE sobre as entradas de uma PipelineSpec e um surrogate dos resultados avaliado em casos separados. [sem teste direto]

pinneapple_simulation/pipelines/freecad_openfoam/sweep.py · 152 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
Sweep	classe	(spec, design, folder, ids)	Uma PipelineSpec rodada em cada ponto de um desenho de experimentos, com salvar, carregar, estado, pendentes, execução e tabela.
doe	função	(bounds, n, method='lhs', seed=0, fixed=None) -> list[dict[str, float]]	Gera n pontos numa caixa de parâmetros por hipercubo latino, Sobol ou grade.
case_id	função	(i, inputs) -> str	Gera o id de um caso da varredura (c000, c001, ...).
fit_surrogate	função	(rows, inputs, outputs, *, test_frac=0.25, val_frac=0.15, seed=0, budget=5.0, methods='au…)	Ajusta um surrogate das saídas a partir das entradas nos casos terminados, com conjunto de teste separado (pinneapple_analysis.surrogate).

pinneapple_simulation.robot_sim

Robot Sim do PINNeAPPle: robôs no NVIDIA Newton com as checagens do PINNeAPPle (torques contra um modelo multicorpo independente, energia, convergência em dt, cargas estruturais) e um backend Isaac Sim com a mesma interface.

pinneapple_simulation/robot_sim/__init__.py · 18 linhas · testes: 39

pinneapple_simulation.robot_sim.arms

Braços prontos para o robot sim (o TwoLinkArm de tubos de alumínio), montados com os dados de material e modelos de elo do PINNeAPPle, para que as mesmas massas vão ao Newton (URDF), ao Isaac Sim (UsdPhysics) e à árvore multicorpo própria. [sem teste direto]

pinneapple_simulation/robot_sim/arms.py · 74 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
TwoLinkArm	classe	(l1, l2, tube, wall, material, motor, payload, shoulder_z, base_mass, effort_limits, grav…)	Braço plano de dois elos no plano x-z com tubos quadrados de alumínio, motor e carga; gera o spec para Newton/Isaac e a árvore multicorpo.

pinneapple_simulation.robot_sim.checks

Checagem em uma chamada de um movimento de pegar-e-colocar de um braço no robot sim, usada pelo nó lib.robot.arm_check do Flow Studio.

pinneapple_simulation/robot_sim/checks.py · 50 linhas · testes: 39

nome	tipo	assinatura	o que faz
arm_pick_and_place	função	(payload=5.0, duration=0.8, q_start=(-0.6, 1.2), q_end=(0.9, -0.4), dt=0.0005, kp=(350.0,…)	Simula e checa um movimento de pegar-e-colocar do braço (torques contra modelo independente, energia, cargas na raiz dos elos).

pinneapple_simulation.robot_sim.learn

Pequeno laço de aprendizado em CPU (método da entropia cruzada, CEM) contra rollouts do `RobotSim`.

`pinneapple_simulation/robot_sim/learn.py` · 48 linhas · testes: 39

nome	tipo	assinatura	o que faz
`CEMResult`	classe	`(best, best_cost, history, evaluations)`	Resultado do CEM: melhores parâmetros, custo, histórico e número de avaliações.
`cem`	função	`(objective, mean, std, *, iterations=6, population=8, elite=0.25, lower=None, upper=None,…)`	Minimiza `objective(params)` pelo método da entropia cruzada, com limites opcionais.

pinneapple_simulation.robot_sim.sim

Simulação de robôs com NVIDIA Newton (`SolverFeatherstone`, CPU ou GPU via Warp), gravando torques, energia, poses e IMU; o `IsaacSimBackend` só escreve um script e levanta NotImplementedError/RuntimeError fora do Isaac Sim. [sem teste direto]

`pinneapple_simulation/robot_sim/sim.py` · 275 linhas · **sem teste direto** · palavras-chave no código (heurística; o texto acima diz o status real): levanta `NotImplementedError` em algum caminho

nome	tipo	assinatura	o que faz
`RobotRun`	classe	`(t, q, qd, tau, body_pos, body_quat, body_vel, body_omega, body_names, joint_names, masse…)`	Registro de uma corrida (tempo, q, qd, torques, poses e velocidades dos corpos, saturação) com energia, trabalho do atuador e IMU.
`RobotSim`	classe	`(builder_fn, *, gravity=9.81, ground=True, device='cpu', name='robot')`	Modelo Newton de um robô (base fixa ou flutuante) construído de URDF, MJCF, USD ou spec, com `run` e controladores.
`IsaacSimBackend`	classe	`(usd_path, robot_prim)`	Mesma interface `run` para o Isaac Sim; aqui só escreve o script autônomo e levanta erro fora do Isaac Sim.
`newton_available`	função	`() -> bool`	Diz se o NVIDIA Newton (e Warp) pode ser importado.
`torque_trajectory`	função	`(fn)`	Controlador em malha aberta com torques `fn(t)`.
`pd_tracking`	função	`(reference, kp, kd, feedforward=None)`	Controlador PD de rastreamento com feedforward opcional.
`convergence_order`	função	`(errors, dts) -> float`	Ordem observada de precisão: inclinação de log(erro) contra log(dt) por mínimos quadrados.

pinneapple_simulation.robot_sim.spec

Robô como dados simples (elos, juntas, acionamentos), exportável como URDF e como UsdPhysics para Omniverse/Isaac Sim. [sem teste direto]

`pinneapple_simulation/robot_sim/spec.py` · 108 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Link`	classe	`(name, mass, size, position, com, inertia, box_offset, color, collide, shape)`	Elo do robô: massa, tamanho, posição, centro de massa, inércia, forma e colisão.
`Joint`	classe	`(name, parent, child, anchor, axis, limits_deg, drive, stiffness, damping, max_force, max…)`	Junta do robô: pai, filho, âncora, eixo, limites, acionamento e limites de força/velocidade.
`ArticulationSpec`	classe	`(name, links, joints, fixed_base, gravity, ground, info)`	Articulação completa (elos, juntas, base fixa, gravidade, chão) como dados simples.
`box_inertia`	função	`(mass, size) -> np.ndarray`	Inércia principal de uma caixa sólida em torno do centro.
`cylinder_inertia`	função	`(mass, radius, length) -> np.ndarray`	Inércia principal de um cilindro sólido com eixo em y.
`spec_to_urdf`	função	`(spec) -> str`	Escreve o URDF do spec: inerciais, colisores e visuais caixa/cilindro, juntas de revolução com limites.

pinneapple_simulation.rover

Rover lunar de seis rodas com suspensão rocker-bogie sobre terreno lunar, com forças roda-solo de Bekker-Wong; reexporta geometria, terreno, contato e simulador.

pinneapple_simulation/rover/__init__.py · 31 linhas · testes: 21

pinneapple_simulation.rover.autolab

Demo do Autonomous Physics Lab, "o rover que aprende o planeta": uma missão em linguagem natural roda o laço planejar → dirigir → aprender o solo → replanejar → verificar num sítio sintético de verdade conhecida.

pinneapple_simulation/rover/autolab/__init__.py · 38 linhas · testes: 8

pinneapple_simulation.rover.autolab.estimation

O que o rover mede (carga, deslizamento, afundamento, torque com ruído de sensor) e a crença gaussiana sobre os parâmetros de solo de cada unidade, atualizada por EKF iterado.

pinneapple_simulation/rover/autolab/estimation.py · 281 linhas · testes: 8

nome	tipo	assinatura	o que faz
Sensors	classe	(sinkage_sd, slip_sd, torque_rel, torque_abs, load_rel, seed)	Níveis de ruído (um desvio-padrão) das medidas do rover; hipóteses declaradas.
UnitBelief	classe	(base, mean, cov, n_obs, history, chi2_dof, scale)	Crença gaussiana de uma unidade de solo: média, covariância, observações e histórico.
SoilBelief	classe	(prior_soils, wheel, g=1.62, prior_sd=PRIOR_SD, obs_sd_sinkage=0.008, obs_sd_torque=(2.0,…)	Crenças sobre os parâmetros de solo de cada unidade, com predição, atualização, calibração e checagens.
soil_theta	função	(soil) -> np.ndarray	Vetor de parâmetros (log10 k_phi, n0, n1, phi) de um solo.
theta_soil	função	(base, theta) -> Soil	Solo a partir de um vetor de parâmetros, com limites físicos.
measure	função	(run, sensors=None, wheels=None, every=4, min_speed=0.02, max_slip=0.7, slip_window=0.3, …)	Medidas com ruído de uma rodada do veículo, por roda e amostra.

pinneapple_simulation.rover.autolab.mission

Converte uma missão em texto em MissionSpec (parser de regras ou LLM, só aceitando valores citados no texto) e roda o laço do laboratório com um conjunto fixo de ferramentas. [sem teste direto]

pinneapple_simulation/rover/autolab/mission.py · 393 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
MissionSpec	classe	(text, goal, start, slip_limit, risk_max, objective, speed, max_replans, parser, quotes, …)	Especificação da missão: meta, partida, limites de deslizamento e risco, objetivo, velocidade e valores padrão usados.
RuleParser	classe	()	Parser determinístico com a mesma interface extract de um adaptador LLM (palavras-chave em inglês e português).
StepLog	classe	(stage, tool, summary, seconds)	Registro de uma etapa do laço: estágio, ferramenta, resumo e tempo.
MissionResult	classe	(spec, routes, runs, belief, planners, measurements, comparison, report, log, retreat, ou…)	Resultado da missão: rotas, rodadas, crença, comparação, relatório e números.
RoverMission	classe	(site, text, *, llm=None, spec=None, dt=0.004, terrain_cell=0.03, planner_cell=0.5, n_sam…)	O laço do laboratório para um pedido: levantar o sítio, planejar, dirigir, estimar o solo, comparar modelos e rodar tudo.
parse_mission	função	(text, site, llm=None) -> MissionSpec	Converte o pedido em MissionSpec, completando com os padrões do sítio.
traction_monitor	função	(slip_limit, window_s=2.0, output_dt=0.05, front=None, t_min=2.0)	Regra de parada: deslizamento médio das rodas dianteiras acima do limite encerra a rodada como travada.
retreat_point	função	(run, site, back=2.0) -> tuple	Ponto para onde o rover recua após travar (o recuo em si não é simulado).

nome	tipo	assinatura	o que faz
`to_json`	função	`(obj)`	Cópia serializável em JSON das saídas da missão.

`pinneapple_simulation.rover.autolab.planner`

Planejamento de rota sensível a risco sobre o sítio a partir da crença de solo do rover (tabelas por declividade, grafo de custo, mapa de risco). [sem teste direto]

`pinneapple_simulation/rover/autolab/planner.py` · 227 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`UnitTables`	classe	`(grades, slip, energy, mean_slip, mean_energy)`	Tabelas por declividade de deslizamento e energia de uma unidade de solo.
`Route`	classe	`(waypoints, cells, length_m, energy_J, max_risk, mean_risk, max_slip, profile, belief_lab…)`	Rota planejada: pontos, células, comprimento, energia, risco e deslizamento.
`RiskPlanner`	classe	`(site, belief, wheel, mass, slip_limit=0.4, g=1.62, n_wheels=4, cell=0.5, n_samples=24, r…)`	Planejador sobre o sítio com a crença de solo: métricas de aresta, grafo de custo, plano, avaliação e mapa de risco.
`densify`	função	`(P, step)`	Adensa uma polilinha com passo máximo.
`simplify`	função	`(P, tol, accept=None)`	Simplificação de Douglas-Peucker com veto opcional de atalhos.

`pinneapple_simulation.rover.autolab.render`

Renderizações Blender Cycles da missão (exige `bpy`): modelo VIPER do repositório, terreno com sulcos das rodadas, cores por risco ou solo, e animação da travessia. [sem teste direto]

`pinneapple_simulation/rover/autolab/render.py` · 322 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`available`	função	`() -> bool`	Diz se o Blender Cycles está disponível.
`import_viper`	função	`(sc, path=VIPER_GLB)`	Importa o glb do VIPER sob um objeto vazio.
`pose_at`	função	`(run, k)`	Posição e rotação do rover numa amostra de saída.
`place`	função	`(root, run, k)`	Posiciona o objeto do rover na pose de uma amostra.
`terrain_grid`	função	`(ter, x_range, y_range, step)`	Grade de alturas do terreno numa faixa.
`terrain_colors`	função	`(site, X, Y, planner=None, mode='risk', shade_with=None)`	Cores do terreno por solo ou pelo risco aprendido.
`build_terrain`	função	`(site, runs, cell=0.03, upto=None)`	Terreno deformável novo com os sulcos das rodadas repetidos.
`route_lines`	função	`(runs, routes, learned_planner=None)`	Legendas de uma renderização: como cada rota foi planejada e o que foi medido.
`render_overview`	função	`(site, runs, routes, planner, out_png, *, size=(1600, 900), samples=48, step=0.06, lines=…)`	Renderização geral do sítio com rotas e o rover.
`render_hero`	função	`(site, runs, out_png, *, size=(1600, 900), samples=96, window=6.0, cell=0.02, grains=2000…)`	Close do travamento da primeira rodada.
`render_traverse`	função	`(site, runs, out_dir, *, size=(800, 450), samples=16, every_s=2.5, step=0.06, fps=12, gif…)`	Quadros das rodadas com câmera de perseguição, depois MP4 e GIF.

`pinneapple_simulation.rover.autolab.report`

Relatório da missão: `summary.json` com todos os números, `trust_report.md`, `report.md` e figuras. [sem teste direto]

`pinneapple_simulation/rover/autolab/report.py` · 206 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`run_info`	função	`() -> dict`	Versões e hardware da execução.
`mission_numbers`	função	`(mission, res) -> dict`	Números medidos e previstos da missão, rotulados pela origem.
`write_report`	função	`(mission, res, out_dir, figures=True, info=None) -> dict`	Grava números, figuras e relatórios da missão num diretório.
`report_markdown`	função	`(n) -> str`	Texto Markdown do relatório a partir dos números.

`pinneapple_simulation.rover.autolab.site`

Sítio planetário sintético com verdade conhecida: relevo, unidades de solo e o que um mapa orbital diria delas (inclusive rótulos errados). [sem teste direto]

`pinneapple_simulation/rover/autolab/site.py` · 190 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Mound`	classe	`(x, y, sx, sy, height)`	Colina gaussiana.
`Patch`	classe	`(unit, x, y, a, b, angle_deg)`	Elipse de uma unidade de solo.
`SoilUnit`	classe	`(name, truth, orbital, orbital_note, color)`	Unidade mapeada: nome, solo verdadeiro (oculto) e solo do mapa orbital.
`SyntheticSite`	classe	`(x_range, y_range, units, patches, mounds, tilt_deg, tilt_dir_deg, start, goal, descripti…)`	Sítio sintético: altura, gradiente, unidades de solo, verdade, prior orbital e terreno deformável.
`soft_sand`	função	`() -> Soil`	A areia fofa oculta do sítio de demonstração.
`demo_site`	função	`() -> SyntheticSite`	O sítio de demonstração com crista, passagem de areia fofa mal rotulada e terreno firme.

`pinneapple_simulation.rover.autolab.verify`

Relatório de confiança da missão: se funcionou, se a física se mantém (incluindo teste de passo de tempo) e se o solo aprendido é crível. [sem teste direto]

`pinneapple_simulation/rover/autolab/verify.py` · 182 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`monitor_slip`	função	`(run, front, window_s=2.0, t_min=2.0)`	Deslizamento médio das rodas dianteiras na janela, como o monitor de tração decide.
`timestep_check`	função	`(mission, route, seconds=6.0) -> dict`	Repete um trecho da rota com dt e dt/2 para checar o passo de tempo.
`verify_mission`	função	`(mission, routes, runs, planners, belief, meas_all, comparison, do_timestep=True)`	Monta as checagens do relatório de confiança da missão.

`pinneapple_simulation.rover.autolab.visuals`

Figuras matplotlib da missão: mapas de sítio e risco, convergência da crença, modelos de roda contra medidas e travessias. [sem teste direto]

`pinneapple_simulation/rover/autolab/visuals.py` · 194 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`plot_site_map`	função	`(site, planners, routes, runs, path, slip_limit=0.4)`	Mapa de solos verdadeiros, relevo, rotas e risco antes e depois do aprendizado.
`plot_learning`	função	`(belief, unit, truth, path, title='')`	Média ± 2 dp de cada parâmetro de solo após cada atualização do filtro.
`plot_model_comparison`	função	`(comparison, path)`	Afundamento e torque contra deslizamento: medido, analítico e surrogate.
`plot_traverse`	função	`(runs, site, path, slip_limit=0.4, front=(0, 1))`	Deslizamento, afundamento, energia e elevação por distância em cada rodada.

nome	tipo	assinatura	o que faz
`plot_route_risk`	função	`(planners, routes, path, slip_limit=0.4, risk_max=0.05)`	Deslizamento e risco previstos ao longo de cada rota.

`pinneapple_simulation.rover.autolab.wheel_models`

Leis roda-solo no ponto de operação que o rover precisa (tabela por declividade) e comparação entre lei analítica, surrogate e medidas. [sem teste direto]

`pinneapple_simulation/rover/autolab/wheel_models.py` · 128 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`GradeTable`	classe	`(grades, slip, sinkage, torque, energy_per_m, max_dp_over_w)`	Pontos de operação de um rover quase-estático num solo, por declividade.
`grade_table`	função	`(soil, wheel, mass, g=1.62, n_wheels=4, grades=GRADES, slips=SLIPS, n_gl=12) -> GradeTable`	Calcula deslizamento, afundamento, torque e energia por metro em cada declividade.
`surrogate_steady_state`	função	`(predict, load, slip, z_max=0.2, iters=50)`	Afundamento em que um modelo de contato equilibra a carga, por bisseção.
`compare_contact_models`	função	`(measured, soil_believed, wheel, surrogate=None, g=1.62, in_range=None) -> dict`	Pontua a lei analítica e um surrogate contra pontos medidos.
`max_climbable_grade`	função	`(table, slip_limit) -> float`	Maior declividade vencida com deslizamento até o limite.

`pinneapple_simulation.rover.checks`

Verificação do modelo do rover contra formas fechadas (afundamento de Bekker), o solver isolado de Bekker-Wong e o equilíbrio de forças e momentos.

`pinneapple_simulation/rover/checks.py` · 149 linhas · testes: 12

nome	tipo	assinatura	o que faz
`single_wheel_check`	função	`(state, rover, soils, rear_angle_ratio=1.0 / 3.0) -> dict`	Compara cada roda carregada com o solver isolado de Bekker-Wong.
`pressure_only`	função	`(soil) -> SoilParams`	Solo só com a lei pressão-afundamento (cisalhamento desligado).
`bekker_rigid_wheel_sinkage`	função	`(W, b, D, k, n) -> float`	Forma fechada de Bekker para o afundamento estático de roda rígida.
`static_sinkage_check`	função	`(rover=None, soil=None) -> dict`	Afundamento estático do rover contra a forma fechada e o solver isolado.
`balance_check`	função	`(state, rover) -> dict`	Força e momento líquidos recalculados das forças nas rodas, adimensionalizados.
`surrogate_vs_numerical`	função	`(states, surrogate, numerical, soils) -> dict`	Erros de um surrogate contra o solver numérico nos estados dados.
`dp_grade_estimate`	função	`(rover, soil, s_limit, g=1.62, rear_angle_ratio=1.0 / 3.0, tol=0.01) -> dict`	Declividade em que seis rodas no deslizamento limite equilibram o peso, por bisseção.

`pinneapple_simulation.rover.contact`

Leis de força roda-solo por roda: Bekker-Wong vetorizado (referência) ou um surrogate qualquer, que registra entradas fora da faixa de treino. [sem teste direto]

`pinneapple_simulation/rover/contact.py` · 167 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`BekkerWongContact`	classe	`(wheel_radius, wheel_width, rear_angle_ratio=1.0 / 3.0, method='gauss', n_gl=48)`	Forças de Bekker-Wong para N rodas de uma vez, com solo por roda.
`RangeLog`	classe	`(n_calls, n_out, slip_out, sinkage_out, soil_mismatch, worst)`	Contagem de entradas do surrogate dentro e fora da faixa de treino.
`SurrogateContact`	classe	`(predict, slip_range, sinkage_range, soil, wheel_radius, wheel_width)`	Envolve um preditor `predict(slip, z)` treinado num solo e roda, com checagem de faixa.

pinneapple_simulation.rover.dynamics

Simulador quase-estático do rover rocker-bogie: equilíbrio por passo com forças de Bekker-Wong, trajetórias, energia e declividade máxima. [sem teste direto]

pinneapple_simulation/rover/dynamics.py · 380 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
StepState	classe	(t, pose, v, omega, s_cmd, wheel_centre, contact_point, normal, tangent, sinkage, slip, F…)	Estado convergido de um passo, com arrays por roda.
RoverSimulator	classe	(rover, terrain, contact=None, g=LUNAR_G, s_limit=0.95, conv_tol=1e-06)	Rover rocker-bogie quase-estático com rodas de Bekker-Wong sobre LunarTerrain.
Trajectory	classe	(states, rover, mode, failed_at, terrain)	Sequência de estados com distância, energia, tabela por roda, resumo e CSV.

pinneapple_simulation.rover.geometry

Rover rocker-bogie paramétrico: parâmetros genéricos, cinemática dos elos e malhas exportáveis em glTF/STL. [sem teste direto]

pinneapple_simulation/rover/geometry.py · 237 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
RoverParams	classe	(wheel_radius, wheel_width, n_grousers, grouser_height, track, wheelbase, pivot_height, b…)	Parâmetros de um rover lunar pequeno genérico, com massa, inércia e checagem.
Pose	classe	(x, y, z, yaw, pitch, roll, rocker, bogie_l, bogie_r, extra)	Configuração do rover: origem, atitude e ângulos da suspensão.
rot_x	função	(a)	Matriz de rotação em torno de x.
rot_y	função	(a)	Matriz de rotação em torno de y.
rot_z	função	(a)	Matriz de rotação em torno de z.
body_rotation	função	(yaw, pitch, roll) -> np.ndarray	Rotação corpo-mundo a partir de guinada, arfagem e rolagem.
quat_from_matrix	função	(R) -> np.ndarray	Quatérnio (w, x, y, z) de uma matriz de rotação.
link_frames	função	(p, pose) -> dict	Posições no mundo dos pivôs e centros de roda, mais rotações dos elos.
rover_meshes	função	(p) -> dict	Malhas de cada peça no seu próprio referencial.
assembled_mesh	função	(p, pose, wheel_spin=None)	Todas as peças em coordenadas do mundo.
export_rover	função	(p, path, pose=None) -> str	Grava o rover como .glb ou .stl.

pinneapple_simulation.rover.terrain

Terreno lunar: função de altura com rampa, crateras e rochas, mais regiões de solo. [sem teste direto]

pinneapple_simulation/rover/terrain.py · 207 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
Crater	classe	(x, y, diameter, rim_ratio, depth_ratio, crest_width)	Cratera com borda e profundidade relativas.
Rock	classe	(x, y, radius, height)	Rocha com raio e altura.
SoilRegion	classe	(soil, kind, params, name)	Solo dentro de um círculo ou caixa.
LunarTerrain	classe	(slope_deg, slope_dir_deg, ramp, craters, rocks, soil, regions)	Função de altura e mapa de solo: normal, declividade, índice de solo, grade e malha.

nome	tipo	assinatura	o que faz
`random_rocks`	função	`(rng, n, x_range, y_range, radius_range, height_ratio_range, keep_out=None) -> list[Rock]`	Rochas com posições e raios uniformes (sem lei publicada de tamanho).
`random_craters`	função	`(rng, n, x_range, y_range, diameter_range, depth_ratio_range=(0.05, 0.15), rim_ratio=0.01…)`	Crateras degradadas com diâmetros por lei de potência truncada.

`pinneapple_simulation.rover.viz`

Gráficos, animações GIF/MP4 e cena 3D (gêmeo digital) de uma trajetória do rover.

`pinneapple_simulation/rover/viz.py` · 225 linhas · testes: 12

nome	tipo	assinatura	o que faz
`plot_wheel_series`	função	`(traj, path, title='', x='distance') -> str`	Seis painéis por roda (afundamento, deslizamento, tração na barra, carga normal, torque, potência) ao longo da trajetória.
`plot_rover_series`	função	`(trajs, path, title='') -> str`	Arfagem, rolagem e energia de várias trajetórias contra a distância.
`animate`	função	`(traj, path_gif, *, color_by='sinkage', path_mp4=None, frames=80, window=(1.7, 1.1), fps=…)`	Animação 3D seguindo o rover, com rodas coloridas por afundamento ou deslizamento.
`twin_scene`	função	`(traj, title='Lunar rover', margin=1.5, grid=(160, 100))`	Cena `pinneapple_twin3d.Scene` da trajetória para o visualizador 3D.

`pinneapple_simulation.terramechanics`

Terramecânica de rodas de rovers planetários: solos com proveniência, modelo de roda de Wong-Reece e extensões, terreno deformável com sulcos, bancada de roda única com dados publicados e substitutos PINN do contato.

`pinneapple_simulation/terramechanics/__init__.py` · 37 linhas · testes: 34

`pinneapple_simulation.terramechanics.mobility`

Triagem de mobilidade de roda com a PINN `bekker_wong_surrogate_2d`: treina, acha o afundamento que sustenta a carga, lê tração na barra contra escorregamento e rechecagem de cada restrição física.

`pinneapple_simulation/terramechanics/mobility.py` · 159 linhas · testes: 5

nome	tipo	assinatura	o que faz
`contact_entry_angle`	função	`(R_m, sinkage_m) -> float`	Ângulo de entrada θ1 = arccos(1 - z/R) de uma roda rígida afundada z.
`mohr_coulomb_traction_limit`	função	`(c_Pa, phi_deg, R_m, b_m, sinkage_m, Fz_N) -> float`	Maior tração na barra permitida pela resistência ao cisalhamento do solo, c A + Fz tan(φ).
`train_wheel_surrogate`	função	`(*, c_Pa, phi_deg, R_m, b_m, epochs=2000, n_collocation=1024, architecture='modified_mlp'…)`	Treina a PINN `bekker_wong_surrogate_2d` para um solo e uma roda.
`analyze_wheel_surrogate`	função	`(model, spec, *, c_Pa, phi_deg, R_m, b_m, Fz_N, slip_min=0.0, slip_max=0.7, n_slip=15, n_…)`	Tração contra escorregamento no afundamento que sustenta `Fz_N`, com checagem de cada restrição física.
`mobility_analysis`	função	`(*, c_Pa=1400.0, phi_deg=30.0, R_m=0.125, b_m=0.06, Fz_N=50.0, slip_min=0.0, slip_max=0.7…)`	Treina, checa as salvaguardas e analisa a mobilidade da roda em uma chamada.

`pinneapple_simulation.terramechanics.soils`

Banco de solos para modelos roda-solo (Bekker, Janosi-Hanamoto, Wong-Reece, escorregamento-afundamento, descarregamento) em que cada parâmetro registra a fonte ('secondary' ou 'estimate'). [sem teste direto]

`pinneapple_simulation/terramechanics/soils.py` · 222 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Soil`	classe	`(name, c, phi_deg, K, k_c, k_phi, n0, rho, n1, a0, a1, c3, Ky, k_u0, A_u, description, so…)`	Parâmetros de um solo com proveniência por parâmetro (`sources`).
`register_soil`	função	`(soil) -> Soil`	Registra um solo no banco.
`get_soil`	função	`(name) -> Soil`	Busca um solo pelo nome.
`list_soils`	função	`() -> list[str]`	Lista os nomes dos solos registrados.

`pinneapple_simulation.terramechanics.surrogate`

Substituto PINN de um modelo de contato roda-solo, (escorregamento, afundamento) -> (tração, carga normal, torque), treinado com termos físicos unilaterais e conjunto de teste separado, salvo como TorchScript + JSON.

`pinneapple_simulation/terramechanics/surrogate.py` · 292 linhas · testes: 20

nome	tipo	assinatura	o que faz
`ContactSurrogate`	classe	`(net, x_lo, x_hi, y_lo, y_hi, slip_range, sinkage_range, meta)`	Rede treinada com sua normalização e caixa de treino; prevê forças, diz se está no domínio e salva.
`PhysicsMismatchWarning`	classe	`()`	Aviso de que o substituto foi treinado com uma física diferente do modelo com que é usado.
`ZooContactSurrogate`	classe	`(zoo_model)`	Adapta um modelo de contato do model zoo (`pp.zoo.load`) à interface de `ContactSurrogate`.
`load_surrogate`	função	`(stem) -> ContactSurrogate`	Carrega um `ContactSurrogate` salvo (TorchScript + JSON).
`contact_net`	função	`(hidden=96, depth=4, n_fourier=16, scale=2.0, seed=0)`	Rede MLP residual com features de Fourier aleatórias e tanh, 2 entradas -> 3 saídas.
`train_contact_surrogate`	função	`(model, slip_range=(0.0, 0.8), sinkage_range=(0.002, 0.06), *, n_grid=40, n_lhs=2000, n_t…)`	Treina o substituto em grade + hipercubo latino com termos físicos e devolve o relatório de erro no teste separado.
`compare_on_states`	função	`(sur, model, slip, z) -> dict`	Compara substituto e referência em estados dados e conta os que estão fora da caixa de treino.

`pinneapple_simulation.terramechanics.terrain`

Terreno deformável para simulação roda-solo: memória plástica de afundamento (sulcos, multipassagem), marcas de garras, mapas de tensão de contato e rochas rígidas.

`pinneapple_simulation/terramechanics/terrain.py` · 341 linhas · testes: 20

nome	tipo	assinatura	o que faz
`RigidRock`	classe	`(x, y, a, b, c, burial, yaw, z0, seed)`	Rocha rígida elipsoidal parcialmente enterrada, com altura, penetração e malha.
`DeformableTerrain`	classe	`(base, x_range, y_range, cell=0.01, rebound=None, relief=0.0, relief_seed=0, berm=0.3, wa…)`	Grade de memória de afundamento sobre uma superfície base, com sulcos, bermas, marcas de garras e rochas.
`fractal_relief`	função	`(shape, cell, amplitude, seed=0, beta=2.2, min_wavelength=0.05) -> np.ndarray`	Superfície aleatória gaussiana com espectro em lei de potência e RMS dado.

`pinneapple_simulation.terramechanics.testbed`

Bancada de roda única: tração, torque e afundamento contra escorregamento a carga fixa, comparação com medições publicadas embarcadas e calibração por mínimos quadrados com pontos retidos.

`pinneapple_simulation/terramechanics/testbed.py` · 253 linhas · testes: 20

nome	tipo	assinatura	o que faz
`TestbedData`	classe	`(dataset, wheel_kind, wheel, soil, load, slip, drawbar_pull, sinkage, torque, source)`	Pontos medidos de uma roda num solo a uma carga (tração, afundamento, torque, fonte).

nome	tipo	assinatura	o que faz
`ValidationReport`	classe	`(data, model, curve, errors)`	Relatório de validação: dados, modelo, curva e erros.
`list_datasets`	função	`() -> list[tuple[str, str]]`	Lista os conjuntos embarcados (dataset, tipo de roda).
`load_dataset`	função	`(dataset, wheel, lambda_grouser=1.0) -> TestbedData`	Carrega os pontos embarcados ('ding2011', 'hurrell2025', 'wanghan2016') para roda lisa ou com garras.
`run_testbed`	função	`(model, load, slips) -> dict`	Tração, torque, afundamento e DP/W do modelo contra escorregamento a carga fixa.
`validate`	função	`(data, soil=None, g=9.81, **model_kw) -> ValidationReport`	Roda a bancada nos escorregamentos medidos e relata os erros contra a medição.
`holdout_split`	função	`(data, every=2, offset=0) -> np.ndarray`	Máscara de treino que separa pontos para ajuste e pontos retidos.
`calibrate`	função	`(data, params=('n0', 'n1', 'K'), bounds=None, g=9.81, train=None) -> dict`	Ajuste por mínimos quadrados de parâmetros do solo à tração e ao afundamento medidos.
`calibrate_holdout`	função	`(data, params=('n0', 'n1', 'K'), every=2, offset=0, g=9.81) -> dict`	Calibração de escorregamento-afundamento com pontos retidos para medir o erro fora do ajuste.
`plot_validation`	função	`(reports, path, title='', calibrated=None) -> str`	Gráfico de tração/carga e afundamento contra escorregamento: pontos medidos e curvas do modelo.

pinneapple_simulation.terramechanics.wheel

Geometria de roda rígida para terramecânica: raio, largura, garras (retas ou chevron), massa e inércia de giro. [sem teste direto]

`pinneapple_simulation/terramechanics/wheel.py` · 68 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Grousers`	classe	`(count, height, thickness, shape, chevron_angle_deg, shear_radius_factor)`	Garras (lugs) de uma roda rígida: número, altura, espessura, forma e ângulo chevron.
`Wheel`	classe	`(radius, width, grousers, mass, spin_inertia)`	Roda rígida: raio, largura, garras, massa e inércia de giro.

pinneapple_simulation.terramechanics.wong_reece

Modelo de Wong-Reece roda rígida / solo deformável com escorregamento-afundamento, garras, multipassagem, bulldozing e força lateral. [sem teste direto]

`pinneapple_simulation/terramechanics/wong_reece.py` · 335 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ContactResult`	classe	`(Fx, Fy, Fz, My, theta_f, theta_r, z_eff, bulldozing, j_exit, theta, sigma, tau_x, tau_y)`	Forças e momento por ponto de operação, com as tensões no arco quando pedidas.
`WongReeceModel`	classe	`(soil, wheel, g=1.62, bulldozing='lateral', n_gl=24, slip_sinkage='drive')`	Roda de Wong-Reece num solo: tensões normal e cisalhante, forças, regime permanente e configuração física registrável.
`bearing_capacity_factors`	função	`(phi_deg) -> tuple[np.ndarray, np.ndarray]`	Fatores de capacidade de carga de Terzaghi (N_c, N_γ) nas formas fechadas de Wong (2008).
`bulldozing_resistance`	função	`(soil, depth, width, g)`	Resistência de bulldozing de uma cunha de solo empurrada por uma face.
`slip_ratio`	função	`(v_long, omega, radius, v_eps=0.001)`	Escorregamento longitudinal (tração ou frenagem) a partir da velocidade do centro e da rotação da roda.
`slip_angle`	função	`(v_long, v_lat, v_eps=0.001)`	Ângulo de deriva de uma roda a partir das componentes da velocidade do centro.
`physics_mismatch`	função	`(recorded, current, rtol=1e-09) -> list[str]`	Lista as chaves em que uma configuração física gravada difere da atual.

pinneapple_simulation.vehicle

Veículos planetários com rodas: descrição (chassi + módulos de canto) com CAD, modelo multicorpo construído a partir dela e simulador em terreno deformável com rodas de Wong-Reece.

pinneapple_simulation/vehicle/__init__.py · 28 linhas · testes: 38

pinneapple_simulation.vehicle.build

Constrói o modelo multicorpo de um `VehicleSpec`: chassi como base flutuante e, por canto, braço de suspensão, módulo de direção e roda, com massas e inércias tiradas do CAD. [sem teste direto]

pinneapple_simulation/vehicle/build.py · 60 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`VehicleModel`	classe	`(spec, tree, susp, steer, wheel, wheel_body, origins)`	Modelo multicorpo de um veículo: spec, árvore, índices de juntas e corpos, com transformações para o CAD.
`build_vehicle`	função	`(spec, gravity=1.62) -> VehicleModel`	Constrói o `VehicleModel` com corpos nomeados como no CAD (`chassis`, `arm_<c>`, `steer_<c>`, `wheel_<c>`).

pinneapple_simulation.vehicle.dashboard

Dashboard centrado em animação de uma corrida do veículo (sobre `pinneapple_tools.visualization.dashboard`): curvas da bancada, escorregamento por roda, PINN contra Wong-Reece e cena 3D. [sem teste direto]

pinneapple_simulation/vehicle/dashboard.py · 240 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`render_frames`	função	`(run, terrain, n=150, *, field=None, clim=None, size=(1280, 720), camera=None, preset='lu…)`	Renderiza `n` quadros distribuídos pela corrida com o renderizador planetário.
`vehicle_dashboard`	função	`(run, frames, ks, *, model=None, surrogate=None, title=None, subtitle='', field_label=Non…)`	Monta o dashboard da corrida com gráficos, tiles, painel de parâmetros e animação.
`vehicle_scene3d`	função	`(run, ks, terrain=None, cell=0.03, preset='lunar_low_sun') -> dict`	Payload 3D ao vivo: corpos do CAD, suas poses e o terreno final com sulcos.

pinneapple_simulation.vehicle.scenarios

Sítios lunares e estudos prontos: terreno plano ou inclinado, cratera com rochas e varredura de rampas em processos paralelos.

pinneapple_simulation/vehicle/scenarios.py · 87 linhas · testes: 17

nome	tipo	assinatura	o que faz
`flat_site`	função	`(x_range=(-3.0, 12.0), y_range=(-3.0, 3.0), cell=0.01, relief=0.005, seed=2, slope_deg=0.…)`	Regolito plano (ou com inclinação uniforme) com relevo aleatório fino.
`crater_site`	função	`(crater=(5.0, 0.5, 4.0), depth_ratio=0.08, rim_ratio=0.015, rock=(2.0, 0.65, 0.16, 0.13, …)`	Cratera degradada com borda e uma rocha sobre regolito.
`grade_sweep`	função	`(factory, soil='GRC-1', grades=(0, 5, 10, 15, 20), duration=12.0, speed=0.1, workers=4, c…)`	Sobe rampas uniformes em linha reta em processos paralelos e resume cada caso.

pinneapple_simulation.vehicle.sim

Simulação no tempo de um veículo com rodas em terreno deformável: dinâmica multicorpo mais rodas de Wong-Reece, rochas por penalidade, atuadores e seguimento de rota. [sem teste direto]

pinneapple_simulation/vehicle/sim.py · 571 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`VehicleRun`	classe	`(model, t, q, v, wheel, joints, chassis, stresses, terrain_events, meta)`	Amostras gravadas de uma corrida (estado, rodas, juntas, chassi, tensões) com distância, energia, resumo, CSV e reprodução do terreno.
`VehicleSimulator`	classe	`(model, terrain, soil, g=1.62, soil_damping=0.7, rock_stiffness=300000.0, rock_damping=40…)`	Simulador de veículo em terreno deformável com rodas de Wong-Reece, rochas, atuadores e seguimento de rota.

pinneapple_simulation.vehicle.spec

Descrição de veículo compartilhada por CAD, modelo multicorpo e simulador: chassi e módulos de canto (suspensão -> direção -> roda motriz) com limites de junta e de atuador. [sem teste direto]

pinneapple_simulation/vehicle/spec.py · 81 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
Actuator	classe	(torque_limit, speed_limit, kp, ki, kd)	Limites de torque e velocidade e ganhos de um servo de junta.
CornerModule	classe	(name, susp_pivot, susp_axis, steer_origin, steer_axis, wheel_centre, wheel_axis, susp_li…)	Um canto do veículo: pivô e eixo da suspensão, direção, centro e eixo da roda, limites e atuadores.
VehicleSpec	classe	(name, wheel, corners, total_mass, max_speed, sources, cad)	Descrição completa do veículo: roda, cantos, massa total, velocidade máxima, fontes e CAD.

pinneapple_simulation.vehicle.viper

NASA VIPER em escala real a partir de números públicos (fontes em VIPER_SOURCES, estimativas marcadas), com CAD paramétrico próprio.

pinneapple_simulation/vehicle/viper.py · 323 linhas · testes: 32

nome	tipo	assinatura	o que faz
viper	função	(wheel=None, total_mass=450.0) -> VehicleSpec	VehicleSpec do VIPER na altura nominal em terreno plano.
viper_cad	função	(spec) -> VehicleCAD	CAD paramétrico do VIPER (chassi, braços, direção, rodas) com massas.

pinneapple_simulation.vehicle.viz

Imagens e animações de um VehicleRun: CAD posado pelo estado multicorpo, terreno com sulcos reproduzido a cada instante, tensões de contato em cores, GIF/MP4 e gráficos por roda.

pinneapple_simulation/vehicle/viz.py · 352 linhas · testes: 11

nome	tipo	assinatura	o que faz
posed_parts	função	(model, q, *, skip=())	Malhas no mundo de cada peça do CAD para uma posição generalizada q.
terrain_at	função	(run, terrain, t_end, cache=None) -> DeformableTerrain	Cópia do terreno com os sulcos e marcas da corrida até t_end.
frame_view	função	(run, terrain, k, *, field=None, clim=None, half_window=(3.2, 2.4), far_margin=14.0, far_…)	Monta a vista de superfície de um quadro: veículo posado, terreno próximo e campo distante.
far_surface	função	(ter, X, Y, relief=0.03, feather=1.0)	Altura do solo no campo distante, misturando o terreno deformável com a superfície base.
follow_camera	função	(run, k, offset=(-4.4, -5.0, 2.1), look=(0.5, 0.0, 0.75), smooth=15)	Câmera que segue o veículo com rumo suavizado.
render_frame	função	(run, terrain, k, path, *, field=None, clim=None, camera=None, size=(1600, 900), title=No…)	Renderiza um quadro da corrida em imagem.
animate	função	(run, terrain, gif, *, mp4=None, field=None, clim=None, frames=120, fps=15, size=(1280, 7…)	GIF (e MP4 com ffmpeg) da corrida.
write_gif	função	(path, imgs, fps, size=None) -> str	Grava um GIF animado de quadros RGB, com redução opcional.
beauty_view	função	(spec, terrain=None, preset='lunar_south_pole', earth=False, horizon=4000.0, sun_azimuth_…)	Vista de superfície usada por beauty.
beauty	função	(spec, path, *, terrain=None, camera=BEAUTY_CAMERA, size=(1600, 900), title='', preset='l…)	Imagem do veículo na pose de referência sobre um trecho de terreno.

nome	tipo	assinatura	o que faz
`plot_run`	função	`(run, path, title='')`	Gráficos por roda contra distância (afundamento, escorregamento, carga, tração, torque, força lateral) e arfagem/rolagem do chassi.
`plot_testbed`	função	`(curves, path, title='')`	Curvas de roda única contra escorregamento.
`chassis_rotation`	função	`(run, k)`	Matriz de rotação do chassi na amostra `k`.

Exemplos de uso reais do repositório

Warm start por projeção economiza iterações do PCG — `tests/test_accelerated_foam.py`

```
    systems = _sequence()
    ws = warm_start.ProjectionWarmStart(6)
    x_prev = np.zeros(systems[0].n_cells)
    it_plain = it_warm = 0
    for s in systems:
        A = s.to_csr()
        xs = spsolve(A.tocsc(), s.source)
        g = ws.guess(s, x_prev)

        def enorm(e, A=A):
            return float(-(e @ (A @ e)))  # A is negative definite

        assert enorm(g - xs) <= enorm(x_prev - xs) * (1 + 1e-9) + 1e-12
        it_plain += solvers.pcg(s, x_prev, "dic", 1e-8).iterations
        r = solvers.pcg(s, g, "dic", 1e-8)
```

Máquina SPM: cogging e torque por dois métodos — `examples/motor_magnetics/01_spm_cogging_and_torque.py`

```
m = SPMMachine(n_theta=360, slot_opening=2 / 15, slot_width=0.4)  # edges on the 1-degree grid
I_RATED = 30.0  # A peak, 20 turns per slot
summary = {"machine": {"pole_pairs": m.pole_pairs, "slots": m.slots, "length_m": m.length,
                       "airgap_m": m.airgap, "Br_T": m.Br, "n_theta": m.n_theta}}

# --- cogging torque over one period ------------------------------------------------------
period = m.cogging_period()
th_cog = np.linspace(0.0, period, 16, endpoint=False)
T_cog = m.cogging(th_cog)
check = [math.radians(d) for d in (1.3, 4.1, 9.7)]
cog_pairs = [(math.degrees(t), m.torque(m.solve(t)), m.torque_coenergy(t, (0.0, 0.0, 0.0))) for t in check]
summary["cogging"] = {"period_deg": math.degrees(period), "peak_to_peak_Nm": float(np.ptp(T_cog)),
                      "maxwell_vs_coenergy_Nm": cog_pairs}
```

Águas rasas na esfera (caso 6 de Williamson) — `examples/geophysics/01_sphere_and_assimilation.py`

```
def main(days: float = 4.0, out: str = "sphere_and_assimilation.png") -> dict:
    sh = SphericalHarmonics(42)
    sw = williamson_case6(sh)
    sw.run(days * 86400, 600.0, diagnostics_every=86400)
    d0, d1 = sw.history[0], sw.history[-1]
    drift = {k: (d1[k] - d0[k]) / d0[k] for k in ("mass", "energy", "potential_enstrophy")}
```

Corpos rígidos com NVIDIA Newton: validar o solver e rodar uma cena — `examples/rigid_body/newton_quickstart.py`

```
from pinneapple_simulation.external_solvers.newton_physics import RigidScene, simulate, validate

# 1. the solver setting against closed-form results
validate("xpbd")
```

```
# 2. a crate sliding to rest next to a pendulum
scene = RigidScene().ground(mu=0.4)
scene.box("crate", (0.1, 0.1, 0.1), pos=(0, 0, 0.1), vel=(2.0, 0, 0), mu=0.4)
a = np.radians(20)
bob = scene.sphere("bob", 0.05, pos=(1.0 + np.sin(a), 0, 2.0 - np.cos(a)))
scene.revolute(-1, bob, anchor=(1.0, 0, 2.0), axis=(0, 1, 0))
traj = simulate(scene, duration=2.0, dt=1e-3)
```

Viga em balanço CalculiX: malha, .frd e leitura — `examples/calculix_pyvista_cantilever.py`

```
from pinneapple_simulation.external_solvers.calculix import cantilever_hex_mesh, read_frd, write_frd

L, B, H, E, P = 1.0, 0.1, 0.1, 210e9, 1000.0

model = cantilever_hex_mesh(L, B, H, nx=20, ny=3, nz=3, element_type="C3D8")
ids, xyz = model.node_array()
x, z = xyz[:, 0], xyz[:, 2]
inertia = B * H ** 3 / 12
deflection = P * x ** 2 * (3 * L - x) / (6 * E * inertia)
stress = np.zeros((len(ids), 6))
stress[:, 0] = P * (L - x) * (z - H / 2) / inertia

out = tempfile.mkdtemp()
frd = read_frd(write_frd(os.path.join(out, "beam.frd"), ids, xyz, model.elements, "C3D8",
                         {"DISP": np.stack([0 * x, 0 * x, -deflection], 1), "STRESS": stress}))
```

Estudo meshless: ordens de convergência e vencedores — `tests/test_meshless.py`

```
        def test_study_reports_orders_wins_and_the_kansa_sweep():
            r = meshless_study(Problem("poisson", exact=U), domain("square"), h=[0.1, 0.05], methods=("kansa", "rbf_fd",
"fem"))
            d = r.to_dict()
            assert {w["criterion"].split(" ")[0] for w in d["wins"]} >= {"Accuracy", "Cheapest", "Fastest", "Best",
"Least"}
            assert d["convergence"]["fem"]["order"] == pytest.approx(2.0, abs=0.3)
```

Testes e status de validação

- Módulos com pelo menos um teste que os importa: 185 de 298.
- Funções de teste em arquivos que importam algum módulo do pacote: 1.119.

Itens do catálogo de métodos (`pinneapple_catalog/method_status.json`) cujo código está neste pacote:

id	método	status	testes contra referência	outros testes
S1	Finite differences 1D/2D/3D	validated	2	11
S2	Finite elements (linear) + nonlinear beam FEM	validated	6	15
S3	Finite volumes	validated	2	2
S4	Spectral (Fourier pseudo-spectral)	validated	3	2
S5	Lattice Boltzmann D2Q9 / D3Q19 (+ Smagorinsky LES)	validated	2	21
S6	SPH / ISPH / DFSPH	untested	0	0
S7	Meshfree RBF collocation (Kansa)	validated	1	1
S8	X-TFC (IVP, PDE, subdomains)	validated	4	8

id	método	status	testes contra referência	outros testes
S9	Discrete-time PINN / implicit Runge-Kutta Gauss-Legendre	tested	0	2
S10	Stiff kinetics ODE	validated	1	0
S11	TVD advection	validated	1	0
S12	Immersed boundary (FDM)	validated	1	1
S13	Beam BVP FDM / 3D elasticity FDM	validated	2	0
S14	Eddy current FDM	validated	3	3
S15	Bekker-Wong terramechanics	validated	1	19
S16	N-body (REBOUND)	validated	1	2
S17	JAX-CFD	tested	0	3
S18	Particle dynamics: rigid body, MPM, molecular dynamics	tested	0	13
S24	External solvers: OpenFOAM, FEniCS, ANSYS/CFD formats, MATLAB, Modelica/FMI, MuJoCo, Genesis, TurboDesigner	validated	1	80
S26	2D shallow-water finite volumes (MUSCL + HLL, wet/dry, walls, gates, virtual sensors, health checks)	validated	1	3
S27	ETDRK4 exponential integrator for stiff semilinear PDEs (contour phi-functions, blow-up guard)	validated	5	1
S28	Stable fluids 2D (semi-Lagrangian advection, spectral projection, vorticity confinement)	validated	2	4
S29	2-D A_z magnetostatics, P1 FEM with Newton (nonlinear B-H, magnets, (anti-)periodic constraints)	validated	1	6
S30	Time-harmonic 2-D eddy currents (A_z, solid conductors with imposed net currents)	validated	1	1
S31	Surface-PM machine model: Maxwell-stress and co-energy torque, cogging, back-EMF, losses	validated	1	8
S32	2-D electrostatics, P1 FEM (capacitance from field energy)	validated	2	0
S33	Biot-Savart integration of straight filament segments, loop and solenoid fields	validated	3	0
S34	Mie series for dielectric and perfectly conducting spheres	validated	2	0
S35	Fresnel coefficients and characteristic matrices of planar multilayers	validated	2	0
S36	Jiles-Atherton hysteresis model (RK4 along an applied field path)	validated	1	0
S37	Dispersive media (Drude, Lorentz, Debye) and their auxiliary-differential-equation coefficients	validated	1	0
S40	Yee FDTD in 1-D/2-D/3-D (CPML, Courant check, Drude/Lorentz/Debye ADE, PEC/PMC/periodic/Bloch, TF/SF plane waves, DFT monitors, sub-cell averaging)	validated	4	6
S41	Near-to-far-field transform on a closed box (FDTD radiation patterns)	validated	1	0

Limitações conhecidas

- `accelerated_foam` precisa de um OpenFOAM instalado e de um compilador C++ para o plugin `pinneWarmStart`; numa instalação só binária é preciso a árvore de fontes da mesma versão.
- `accelerated_foam.learned` é experimental: o próprio docstring diz que uma rede local só remove resíduo de alta frequência e que o efeito esperado é modesto; pyamg e PyTorch são opcionais.
- Antenas e células periódicas (`antenna.ava`, `periodic.tcda`, `antenna.study.open_study`) dependem do solver externo EMerge (`pip install emerge`); a classe base de `antenna.surrogates` levanta `NotImplementedError`

nos métodos abstratos.

- A verificação cruzada `machines.getdp` exige `gmsh` e `getdp` no PATH; FEMM e ElmerFEM só têm os arquivos de entrada gerados (`interop.femm_lua`, `interop.elmer_case`), não são executados.
- `machines.design` importa torch (via `pinneapple_design.design_optimizer`); `mesh.mesh_from_gmsh` precisa do gmsh; figuras precisam de matplotlib.
- Dados de aço e ímã (`machines.steel`) e as dimensões do Prius 2004 vêm do pyleecan; `materials.illustrative_steel` e os dados térmicos padrão de `machines.thermal` são ilustrativos/hipóteses, não dados de um fornecedor.
- `electrostatics` é só planar (sem peso axisimétrico); `eddy` assume meios lineares.
- Vários módulos não têm teste direto: `accelerated_foam.benchmark`, `antenna.plots`, `eddy`, `machines.mesher`, `machines.pmsm`, `machines.steel`, `reflector.study_report_figures`, `reflector.study_report_text`, `geophysics.energy_balance` e `geophysics.richards`.
- Quase todas as pontes de `external_solvers` dependem de programas externos ou pacotes opcionais (OpenFOAM, CalculiX/Docker, Fluent, Abaqus, MATLAB, fmpy/OMPython/omc, MuJoCo/JAX, Genesis, Newton/Warp, FluidX3D com OpenCL, GeoTaichi/Taichi, FEniCS, turbodesigner); o `__init__` importa com `try/except` e deixa os nomes como None quando falta a dependência.
- ANSYS Fluent nunca foi rodado de verdade: só a geração do journal e o erro de executável ausente são testados, e o journal cobre só velocity-inlet e pressure-outlet, sem seleção de modelos físicos.
- Leitores de formato com escopo estreito: Fluent `.msh` só lê nós ASCII (resto levanta `NotImplementedError`), Exodus só NetCDF clássico e sem conectividade, STAR-CCM+ nativo `.sim`/`.ccm` não é suportado, e o leitor binário OpenFOAM recusa formato ASCII.
- Vários módulos não têm teste direto: Genesis, MuJoCo, MATLAB (exceto via import amplo), TurboDesigner, OpenRadioss (deck, runner, vtk), boa parte de `openfoam` (runner, sampling, case_builder, export_bundle, binary_reader), `flight.aero/gear/mass/propulsion/encounter/verification` e grande parte de `pipelines.freecad_openfoam`.
- `openfoam.export_bundle` e `openfoam.case_builder` têm nomes de arquivo e campos fixos de um projeto específico (canal com cilindro, problema térmico), não são genéricos.
- `turbodesigner.run_turbodesigner` cai num cálculo analítico simplificado quando o pacote `turbodesigner` falta (`fallback_if_missing=True` por padrão), o que pode passar despercebido.
- FluidX3D tem licença que proíbe uso comercial e militar, e GeoTaichi é GPL-3.0; o PINNeAPPle só os chama como programas separados, sem distribuir o código.
- `newton_physics.simulate` levanta `NotImplementedError` para velocidades iniciais com solvers de coordenadas de junta (featherstone, mujoco); o catálogo de `flight` usa premissas declaradas em `Aircraft.info` e não é ajustado a ensaio em voo.
- `simulate` passa os kwargs direto ao construtor da classe e só chama `run` se ela existir; as classes de `fdm3d` recebem `cfg` e têm `solve`, então o uso do docstring (`simulate("heat_3d", nx=32, ...)`) não bate com essas assinaturas. `simulate/generate_data` não têm teste direto.
- Várias dependências são opcionais: OpenFOAM (`external_flow`, `internal_flow`, `free_surface`, `openfoam_bridge`), FEniCS/dolfinx, jax/jax_cfd, rebound, gmsh, trimesh, Blender `bpy` (`rover.autolab.render`). `stl` e `wavelet` importam statsmodels e pywt no topo; `register_all` captura e devolve essas falhas em vez de abortar.
- Vários módulos se declaram MVP ou aproximados: `spectral`, `sph`, `sph_boundaries`, `sst`, `hilbert_huang`, `neighbors`, `ceemdan`, `adapters.upd_mesh`.
- `meshfree.RBFCollocationSolver` levanta NotImplementedError para kernels RBF sem laplaciano analítico (aceita só gaussian, multiquadric, thin_plate, imq).
- `elasticity3d_fdm` aceita só deslocamento prescrito em todas as faces; o contorno livre (tração nula) não está implementado, segundo o próprio docstring.
- `SolverCatalog.get` chama `self.registry.get`, mas `SolverRegistry` não define `get` (tem `spec`).
- `particles` é feito para tendências de processo, não para números de projeto calibrados (docstring). O rover é quase-estático e usa parâmetros genéricos; em `autolab`, os ruídos de sensor são hipóteses declaradas e o recuo após travar não é simulado.
- Muitos módulos não têm teste que os importe diretamente (por exemplo SPH/ISPH/DFSPH, decomposições de sinal, `xtfc_ivp`, `thermal_channel_2d`, `ns3d_spectral` e os submódulos do rover e do `autolab` fora de `estimation`).
- `robot_sim` depende do NVIDIA Newton/Warp (`newton_available`); o `IsaacSimBackend` só escreve um script e levanta erro fora de uma máquina com Isaac Sim e GPU RTX.

- `ParticleSystem.forward` levanta NotImplementedError (classe base); o docstring de `particles` cita DEM, mas só o SPH está implementado.
- Vários módulos centrais não têm teste direto: `multibody.tree`, `terramechanics.wong_reece`, `terramechanics.soils`, `vehicle.sim`, `vehicle.build`, `vehicle.spec`, `vehicle.dashboard`, `robot_sim.sim`, `robot_sim.spec`.
- Os solos em `terramechanics.soils` trazem valores 'secondary' (não conferidos com a tabela original) ou 'estimate' (estimativa de engenharia), segundo o próprio docstring.
- O VIPER é um modelo paramétrico a partir de números públicos; o docstring avisa que os PDFs da NASA não puderam ser abertos e que alguns valores são estimativas.
- `read_particle_frames` com `allow_pickle=True` só deve ser usado com arquivos confiáveis (GeoTaichi grava objetos em pickle).
- Os substitutos PINN de contato só valem dentro da caixa de treino; fora dela `compare_on_states` e `in_range` sinalizam, e `PhysicsMismatchWarning` avisa física diferente.

Volume 4 — Aprendizado de máquina e dados

> Parte do Mapa completo do PINNeAPPle. Gerado por `docs/pt/_mapa/gerar.py` a partir do inventário do commit `e53cdcfb04`.

Arquiteturas e treino de redes (PINN, operadores neurais, grafos), dados e amostragem, adaptação, modelos de mundo, percepção, computação quântica e a arena de benchmarks.

Sumário

1. pinneapple_neural
2. pinneapple_data
3. pinneapple_pdb
4. pinneapple_adaptation
5. pinneapple_arena
6. pinneapple_worldmodel
7. pinneapple_perception
8. pinneapple_quantum

1. `pinneapple_neural`

Objetivo

`pinneapple_neural` é o pacote de redes neurais do PINNeAPPle. Ele junta três antigos pacotes: `architectures` (antes `pinneapple_models`) com o zoológico de modelos, `trainer` (antes `pinneapple_train`) com o laço de treino e utilitários de HPC, e `predictor` (antes `pinneapple_inference`) com inferência e gráficos. Os nomes antigos `models`, `train` e `inference` continuam como apelidos.

O problema que resolve é dar uma porta única para construir, treinar e usar modelos de física com IA. As arquiteturas cobrem PINNs (vanilla, inverso, bayesiano, X-TFC, XPINN, difusão informada por física), operadores neurais (FNO 1D/2D/3D, DeepONet, U-NO, Transolver, PINO), redes em grafo (MeshGraphNet e dinâmica em malha), modelos de tempo contínuo, séries temporais, computação de reservatório, autoencoders e modelos de ordem reduzida (POD, DMD, OpInf, SINDy e o estudo `rom_study`). Cada família tem um catálogo próprio e se registra no `ModelRegistry` global quando `architectures` é importado; `ModelRegistry.build(nome, **kwargs)` normaliza nomes de argumentos entre famílias.

Os atalhos da raiz (`build_model`, `train_model`, `predict`, `train_and_predict`) encadeiam registro, `Trainer` e inferência em lote. Em volta disso há peças mais novas: `callbacks` leva os checks de `pp.validate` para dentro do treino em Lightning, PhysicsNeMo e Hugging Face; `physicsnemo_bridge` liga modelos treinados no PhysicsNeMo às ferramentas de UQ, gêmeo digital e checagem do PINNeAPPle; `lpm` traz blocos de um modelo para um espaço de projeto inteiro; `workflows` oferece fluxos prontos (FNO3d em volumes densos, versão paramétrica e surrogate híbrido com pós-modelo físico).

O pacote é usado pelo resto da biblioteca: a fachada `pp.*` e os solvers montam modelos pelo registro, `pinneapple_systems.component_library` e `pinneapple_orchestration` treinam componentes com o mesmo `Trainer`, e os resultados de ROM viram contratos de `pinneapple_data.contracts` para os apps. Integrações com licença restrita (Noether/Emmi AI) são só para pesquisa e bloqueadas em modo comercial por `_licencas`.

O submódulo `pinneapple_neural.trainer` reúne a maquinaria de treino do PINNeAPPle. O centro é o `Trainer` genérico (perdas, métricas, callbacks, checkpoint, auditoria, AMP e DDP), cercado por estratégias específicas de PINN: treino em duas fases, time-marching, treino causal, PINN inverso e gray-box, colocação adaptativa (RAR), balanceamento automático de pesos de perda (`loss_balancer`, `weight_scheduler`), otimizadores especiais (MultiAdam, quase-Newton auto-escalado, L-BFGS multi-rodada), reinícios múltiplos, incerteza heteroscedástica, restrições de monotonicidade e utilitários de paralelismo/HPC (DDP, FSDP, SLURM).

Números do inventário

módulos	linhas	classes públicas	funções públicas	testes em arquivos que importam o pacote	módulos sem teste direto
198	36.217	334	150	438 (49 arquivos)	152

Pasta de testes dedicada: `tests/pinneapple_neural` (36 funções de teste).

Submódulos

submódulo	o que faz	módulos	classes + funções	testes*
`_licencas`	Guardas de licença: avisa quando um componente só para pesquisa é usado e o bloqueia com `PINNEAPPLE_COMMERCIAL_MODE=1`.	1	4	20
`architectures`	Zoológico de arquiteturas por família, com catálogos, adaptadores de batch e o `ModelRegistry` global.	147	270	64
`callbacks`	Checks físicos de `pp.validate` como métricas de treino em Lightning, PhysicsNeMo e Hugging Face.	1	3	4
`lpm`	Blocos de um 'Large Physics Model': codificação espacial, código de geometria, ensemble e sinal fora-da-distribuição.	1	6	5
`physicsnemo_bridge`	Cola entre PhysicsNeMo e PINNeAPPle: UQ, gêmeo digital, active learning com retreino e operadores em grade.	4	15	19
`predictor`	Inferência pós-treino em lote e em grade, e gráficos de campos, erros e escoamentos.	4	27	0
`trainer`	Treinadores, perdas, métricas, balanceadores de perda, otimizadores e utilitários de paralelismo/HPC para treinar PINNs e surrogates.	34	132	151
`utils`	Helpers `as_tensor` (tensor de qualquer saída de modelo) e `fit_supervised` (laço Adam mínimo).	1	2	5
`workflows`	Fluxos prontos de treino e previsão: FNO3d em volumes densos (simples e paramétrico) e surrogate híbrido com pós-modelo físico.	4	21	10

* maior número de testes que importam um módulo do submódulo.

Módulos e API pública

`pinneapple_neural`

Raiz do pacote: importa `architectures`, `trainer` e `predictor`, mantém os apelidos antigos `models`, `train` e `inference`, reexporta os símbolos principais e define os atalhos `build_model`, `train_model`, `predict` e `train_and_predict`.

`pinneapple_neural/__init__.py` · 256 linhas · testes: 20

nome	tipo	assinatura	o que faz
`build_model`	função	`(name, **kwargs)`	Atalho para `ModelRegistry.build(name, **kwargs)`.
`train_model`	função	`(model, loss_fn, train_loader, val_loader=None, *, epochs=5000, device='cpu', **cfg_kwarg…)`	Atalho que monta um `TrainConfig` e chama `Trainer.fit`; usa o loader de treino como validação se nenhum for dado.
`predict`	função	`(model, x, *, device='cpu', batch_size=4096)`	Move modelo e entrada para o dispositivo e roda `batched_inference`.
`train_and_predict`	função	`(model, loss_fn, train_loader, x_test, *, val_loader=None, epochs=5000, device='cpu')`	Treina com `train_model` e prevê em `x_test`; devolve um dicionário com os dois resultados.

`pinneapple_neural._licencas`

Guardas de licença para integrações de terceiros só para pesquisa (hoje, Noether/Emmi AI): avisa com `ResearchOnlyWarning` e bloqueia com `ResearchOnlyError` quando `PINNEAPPLE_COMMERCIAL_MODE=1`.

`pinneapple_neural/_licencas.py` · 44 linhas · testes: 20

nome	tipo	assinatura	o que faz
`ResearchOnlyWarning`	classe	`()`	Aviso emitido quando um componente só para pesquisa é instanciado.
`ResearchOnlyError`	classe	`()`	Erro levantado ao pedir um componente só para pesquisa em modo comercial.
`commercial_mode`	função	`() -> bool`	Diz se a variável `PINNEAPPLE_COMMERCIAL_MODE` vale 1.
`require_research_use`	função	`(component, notice=NOETHER_NOTICE) -> None`	Levanta `ResearchOnlyError` em modo comercial; fora dele, emite `ResearchOnlyWarning` com o aviso de licença.

`pinneapple_neural.architectures`

Subpacote de arquiteturas (antigo `pinneapple_models`): reexporta `BaseModel`, `ModelRegistry`, `ModelCatalog`, `select_adapter`, SIREN, ModifiedMLP, HashGridMLP, AFNO e MeshGraphNet, e chama `register_all()` ao ser importado para popular o registro global.

`pinneapple_neural/architectures/__init__.py` · 43 linhas · testes: 28

`pinneapple_neural.architectures._registry_bridge`

Função auxiliar que copia um dicionário `nome -> classe` de uma família para o `ModelRegistry` global, pulando apelidos da mesma classe. [sem teste direto]

`pinneapple_neural/architectures/_registry_bridge.py` · 39 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`register_family_registry`	função	`(family_registry, *, family, description_getter=None, tags_getter=None, capabilities_gett…)`	Registra cada classe de um dicionário de família no `ModelRegistry`, com descrição, tags e capacidades opcionais.

`pinneapple_neural.architectures.adapters`

Adaptadores que chamam qualquer modelo a partir de um batch em dicionário, sem refatorar cada implementação; usados pela Arena via `select_adapter`. [sem teste direto]

`pinneapple_neural/architectures/adapters/__init__.py` · 27 linhas · **sem teste direto**

`pinneapple_neural.architectures.adapters.ae`

Adaptador para autoencoders: lê `batch['x']` como entrada a reconstruir. [sem teste direto]

`pinneapple_neural/architectures/adapters/ae.py` · 28 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`AutoEncoderAdapter`	classe	`()`	Adaptador que chama autoencoders com `batch['x']`.

`pinneapple_neural.architectures.adapters.base`

Define o protocolo `ModelAdapter`, um adaptador padrão (usa `forward_batch` ou `model(x)`) e `select_adapter`, que testa os adaptadores em ordem. [sem teste direto]

`pinneapple_neural/architectures/adapters/base.py` · 63 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ModelAdapter`	classe	`()`	Protocolo de adaptador: `can_handle(spec)` e `forward_batch(model, batch)`.
`select_adapter`	função	`(spec) -> ModelAdapter`	Devolve o primeiro adaptador que aceita o `ModelSpec`, ou o adaptador padrão.

pinneapple_neural.architectures.adapters.conv

Adaptador para modelos de grade (Conv1D/2D/3D, AFNO, UNO em modo grade) que lê `batch['u_grid']`. [sem teste direto]

`pinneapple_neural/architectures/adapters/conv.py` · 31 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ConvAdapter`	classe	`()`	Adaptador para modelos de grade que lê `u_grid`.

pinneapple_neural.architectures.adapters.dynamics

Adaptador para modelos de evolução no tempo (Neural ODE, Latent ODE, ODE-RNN, CDE, SDE, Symplectic RNN) que lê `x0` e `t`. [sem teste direto]

`pinneapple_neural/architectures/adapters/dynamics.py` · 32 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`DynamicsAdapter`	classe	`()`	Adaptador para modelos de dinâmica que chama `forward(x0, t)`.

pinneapple_neural.architectures.adapters.gnn

Adaptador para modelos em grafo/malha: passa `batch['graph']` ou os tensores `x`/`edge_index`. [sem teste direto]

`pinneapple_neural/architectures/adapters/gnn.py` · 39 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`GNNAdapter`	classe	`()`	Adaptador para modelos em grafo ou malha.

pinneapple_neural.architectures.adapters.operators

Adaptador para operadores neurais (DeepONet, FNO-1D, GNO, UNO, PINO) que escolhe as chaves do batch conforme o tipo. [sem teste direto]

`pinneapple_neural/architectures/adapters/operators.py` · 49 linhas · **sem teste direto**

nome	tipo	assina-tura	o que faz
`OperatorAdapter`	classe	`()`	Adaptador para operadores neurais que monta a chamada certa para DeepONet, FNO, GNO, UNO ou PINO.

pinneapple_neural.architectures.adapters.pinn

Adaptador para PINNs pontuais: usa `batch['x']` ou `batch['x_col']`, ou delega a `forward_batch`. [sem teste direto]

`pinneapple_neural/architectures/adapters/pinn.py` · 37 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`PINNAdapter`	classe	`()`	Adaptador para PINNs pontuais (entrada `x` ou `x_col`).

pinneapple_neural.architectures.adapters.ts

Adaptador para modelos de série temporal: lê o histórico `x` (B, T, D) e chaves opcionais como `x_mark` e `y0`. [sem teste direto]

`pinneapple_neural/architectures/adapters/ts.py` · 44 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`TimeSeriesAdapter`	classe	`()`	Adaptador para modelos de série temporal (histórico `x` e chaves opcionais).

pinneapple_neural.architectures.afno

Adaptive Fourier Neural Operator (Guibas et al.) para campos 2D em grade `(B, H, W, C)`: mistura de tokens bloco-diagonal no espaço de Fourier mais MLP de canais. [sem teste direto]

pinneapple_neural/architectures/afno.py · 264 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
AFNOLayer	classe	(hidden_dim, n_modes_h=12, n_modes_w=12, mlp_ratio=4.0, dropout=0.0, num_blocks=8, hidden…)	Uma camada AFNO: mistura bloco-diagonal no espaço de Fourier com limiar de esparsidade mais MLP de canais.
AFNO	classe	(in_channels, out_channels, hidden_dim=64, n_layers=4, n_modes_h=12, n_modes_w=12, mlp_ra…)	Operador AFNO completo para campos 2D `(B, H, W, C)`: projeção de entrada, camadas AFNO e projeção de saída.

pinneapple_neural.architectures.autoencoders

Família de autoencoders; reexporta `AutoencoderCatalog`, `AEBase` e `AEOutput`. [sem teste direto]

pinneapple_neural/architectures/autoencoders/__init__.py · 5 linhas · **sem teste direto**

pinneapple_neural.architectures.autoencoders.ae_2d

Autoencoder convolucional 2D para imagens e campos 2D, com saída redimensionada para (H, W) por interpolação. [sem teste direto]

pinneapple_neural/architectures/autoencoders/ae_2d.py · 133 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
Autoencoder2D	classe	(in_channels, latent_dim, img_size, base_channels=32, norm='group', gn_groups=8, output_a…)	Autoencoder Conv2D para campos 2D, com latente vetorial.

pinneapple_neural.architectures.autoencoders.ae_rom_hybrid

Autoencoder denso com dinâmica linear contínua no latente (dz/dt = K z + c, passo de Euler), perdas de previsão, rollout e estabilidade. [sem teste direto]

pinneapple_neural/architectures/autoencoders/ae_rom_hybrid.py · 320 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
AEROMHybrid	classe	(input_dim, latent_dim, hidden=(512, 256), activation='gelu', rom_weight=1.0, pred_weight…)	Autoencoder com dinâmica latente linear (K, c), ajuste por mínimos quadrados e penalidade de estabilidade.

pinneapple_neural.architectures.autoencoders.base

Contrato comum dos autoencoders: `encode`, `decode`, `forward` e `loss_from_parts`; os dois primeiros levantam `NotImplementedError` na base (classe abstrata).

pinneapple_neural/architectures/autoencoders/base.py · 51 linhas · testes: 5 · palavras-chave no código (heurística; o texto acima diz o status real): levanta `NotImplementedError` em algum caminho

nome	tipo	assinatura	o que faz
AEOutput	classe	(x_hat, z, losses, extras)	Saída padronizada dos autoencoders: `x_hat`, `z`, perdas e extras.
AEBase	classe	()	Classe base abstrata dos autoencoders (encode/decode/forward/perdas).

pinneapple_neural.architectures.autoencoders.dense_ae

Autoencoder MLP simples para vetores, com decoder espelhado. [sem teste direto]

pinneapple_neural/architectures/autoencoders/dense_ae.py · 54 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`DenseAutoencoder`	classe	`(input_dim, latent_dim, hidden=(512, 256), activation='gelu')`	Autoencoder MLP para vetores.

pinneapple_neural.architectures.autoencoders.kae

Autoencoder estilo MMD-WAE: penalidade MMD (kernel RBF, sem viés) no latente para casar com um prior simples. [sem teste direto]

`pinneapple_neural/architectures/autoencoders/kae.py` · 177 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`KAEAutoencoder`	classe	`(input_dim, latent_dim, hidden=(512, 256), activation='gelu', mmd_weight=1.0, mmd_sigma=1…)`	Autoencoder com penalidade MMD no latente (um ou vários kernels RBF; banda pela mediana se `mmd_sigma <= 0`).

pinneapple_neural.architectures.autoencoders.koopman_pi_ae

Autoencoder de Koopman: operador linear treinável no latente, opção de gerador contínuo (expm(A*dt)), perdas de latente, previsão e rollout. [sem teste direto]

`pinneapple_neural/architectures/autoencoders/koopman_pi_ae.py` · 331 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`PhysicsInformedKoopmanAutoencoder`	classe	`(input_dim, latent_dim, hidden=(512, 256), activation='gelu', koopman_weight=1.0, *, pred…)`	Autoencoder de Koopman com operador linear no latente e perdas de consistência e previsão.

pinneapple_neural.architectures.autoencoders.registry

Registro da família de autoencoders (com apelidos) e `AutoencoderCatalog`; registra no `ModelRegistry` com `input_kind='autoencoder'`. [sem teste direto]

`pinneapple_neural/architectures/autoencoders/registry.py` · 70 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`AutoencoderCatalog`	classe	`(registry)`	Catálogo da família: `list`, `get` e `build` por nome.
`register_into_global`	função	`() -> None`	Registra os autoencoders no `ModelRegistry` global.

pinneapple_neural.architectures.autoencoders.vae

Autoencoder variacional MLP (beta-VAE) para vetores, com log-variância limitada. [sem teste direto]

`pinneapple_neural/architectures/autoencoders/vae.py` · 122 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`VariationalAutoencoder`	classe	`(input_dim, latent_dim, hidden=(512, 256), beta=1.0, activation='gelu', clamp_logvar=(-10…)`	VAE MLP com reparametrização e peso `beta` no termo KL.

pinneapple_neural.architectures.base

Contrato mínimo dos modelos: `ModelOutput` e `BaseModel` com `forward` (abstrato, levanta `NotImplementedError`), `forward_batch` padrão e checkpoints.

`pinneapple_neural/architectures/base.py` · 93 linhas · testes: 5 · palavras-chave no código (heurística; o texto acima diz o status real): levanta `NotImplementedError` em algum caminho

nome	tipo	assinatura	o que faz
`ModelOutput`	classe	`(y, losses, extras)`	Saída padrão de modelo: `y`, perdas e extras.
`BaseModel`	classe	`(family, name)`	Classe base de todos os modelos: `forward` abstrato, `forward_batch` padrão e salvar/carregar checkpoint.

pinneapple_neural.architectures.benchmarks

Família de modelos de referência para benchmarks (MLPs genéricos). [sem teste direto]

pinneapple_neural/architectures/benchmarks/__init__.py · 5 linhas · **sem teste direto**

pinneapple_neural.architectures.benchmarks.generic_pinn_models

Modelos-base genéricos para comparar arquiteturas: MLP, MLP com Fourier features, SIREN, MLP residual e linear. [sem teste direto]

pinneapple_neural/architectures/benchmarks/generic_pinn_models.py · 122 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
FourierFeatures	classe	(in_dim, num_f=64, scale=10.0)	Camada de features aleatórias de Fourier com escala fixa.
GenericMLP	classe	(in_dim=3, out_dim=1, width=128, depth=4, act='tanh')	MLP genérico de referência (tanh por padrão).
GenericFourierMLP	classe	(in_dim=3, out_dim=1, num_f=64, scale=10.0, width=128, depth=4)	MLP genérico com Fourier features na entrada.
Sine	classe	(w0=30.0)	Ativação seno com frequência w0.
GenericSIREN	classe	(in_dim=3, out_dim=1, width=128, depth=4, w0=30.0)	SIREN genérica de referência.
GenericResMLP	classe	(in_dim=3, out_dim=1, width=128, depth=6)	MLP residual genérico de referência.
GenericLinear	classe	(in_dim=3, out_dim=1)	Modelo linear de referência.

pinneapple_neural.architectures.benchmarks.registry

Registra os modelos de benchmark no ModelRegistry como bench_mlp, bench_fourier_mlp, bench_siren, bench_res_mlp e bench_linear. [sem teste direto]

pinneapple_neural/architectures/benchmarks/registry.py · 32 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
register_into_global	função	() -> None	Registra os cinco modelos de benchmark no ModelRegistry global.

pinneapple_neural.architectures.catalog

ModelCatalog: agrega os catálogos de todas as famílias num único objeto. [sem teste direto]

pinneapple_neural/architectures/catalog.py · 33 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
ModelCatalog	classe	(autoencoders, pinns, transformers, recurrent, convolutions, neural_operators, continuous…)	Dataclass com um catálogo por família (autoencoders, pinns, transformers, recurrent, convolutions, neural_operators, continuous, graphnn, reservoir_computing, classical_ts, rom, physics_aware).

pinneapple_neural.architectures.classical_ts

Família de modelos clássicos de série temporal e filtragem. [sem teste direto]

pinneapple_neural/architectures/classical_ts/__init__.py · 5 linhas · **sem teste direto**

pinneapple_neural.architectures.classical_ts.arima

ARIMA(p,d,q) implementado sem dependências externas, ajustado por regressão. [sem teste direto]

pinneapple_neural/architectures/classical_ts/arima.py · 250 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
ARIMA	classe	(dim=1, p=3, d=0, q=0, l2=1e-06, use_bias=True)	Modelo ARIMA(p,d,q) com fit, forecast e forward, sem bibliotecas externas.

pinneapple_neural.architectures.classical_ts.base

Saída ClassicalTSOutput e base ClassicalTSBase com mse para a família. [sem teste direto]

pinneapple_neural/architectures/classical_ts/base.py · 34 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
ClassicalTSOutput	classe	(y, losses, extras)	Saída padrão dos modelos clássicos de série temporal.
ClassicalTSBase	classe	()	Base da família com o helper mse.

pinneapple_neural.architectures.classical_ts.ekf

Filtro de Kalman estendido em lote, com funções de transição/observação e jacobianos dados pelo usuário. [sem teste direto]

pinneapple_neural/architectures/classical_ts/ekf.py · 125 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
ExtendedKalmanFilter	classe	(f, h, Q, R, F_jac, H_jac)	Filtro de Kalman estendido com jacobianos F_jac e H_jac.

pinneapple_neural.architectures.classical_ts.enkf

Filtro de Kalman por ensemble (EnKF). [sem teste direto]

pinneapple_neural/architectures/classical_ts/enkf.py · 122 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
EnsembleKalmanFilter	classe	(f, h, Q, R, *, ensemble_size=64)	Filtro de Kalman por ensemble com ensemble_size membros.

pinneapple_neural.architectures.classical_ts.kalman

Filtro de Kalman linear em lote, com controle B opcional. [sem teste direto]

pinneapple_neural/architectures/classical_ts/kalman.py · 130 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
KalmanFilter	classe	(A, H, Q, R, *, B=None)	Filtro de Kalman linear em lote.

pinneapple_neural.architectures.classical_ts.registry

Registro da família classical_ts e ClassicalTSCatalog. [sem teste direto]

pinneapple_neural/architectures/classical_ts/registry.py · 67 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
ClassicalTSCatalog	classe	(registry)	Catálogo da família classical_ts.
register_into_global	função	() -> None	Registra a família classical_ts no ModelRegistry global.

pinneapple_neural.architectures.classical_ts.tcn

Rede convolucional temporal (TCN) para regressão sequência a sequência. [sem teste direto]

pinneapple_neural/architectures/classical_ts/tcn.py · 89 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
TCN	classe	(in_dim, out_dim, *, channels=(64, 64, 64), kernel=3, dropout=0.1)	Rede convolucional temporal para regressão sequência a sequência.

pinneapple_neural.architectures.classical_ts.ukf

Filtro de Kalman unscented (pontos sigma) em lote, com jitter crescente na fatoração de Cholesky. [sem teste direto]

pinneapple_neural/architectures/classical_ts/ukf.py · 254 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
UnscentedKalmanFilter	classe	(f, h, Q, R, *, alpha=0.001, beta=2.0, kappa=0.0, jitter=1e-06, jitter_growth=10.0, max_c…)	Filtro de Kalman unscented em lote.

pinneapple_neural.architectures.classical_ts.var

Modelo VAR(p) ajustado por regressão ridge em forma fechada. [sem teste direto]

pinneapple_neural/architectures/classical_ts/var.py · 165 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
VAR	classe	(dim, p=1, l2=1e-06, use_bias=True)	Modelo VAR(p) ajustado por ridge em forma fechada.

pinneapple_neural.architectures.continuous

Família de modelos de tempo contínuo e probabilísticos (ODE/SDE/CDE neurais, hamiltonianos, GP neural). [sem teste direto]

pinneapple_neural/architectures/continuous/__init__.py · 5 linhas · **sem teste direto**

pinneapple_neural.architectures.continuous.base

Saída `ContOutput` e base `ContinuousModelBase` com `mse` e `gaussian_nll`. [sem teste direto]

pinneapple_neural/architectures/continuous/base.py · 38 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
ContOutput	classe	(y, losses, extras)	Saída padrão dos modelos de tempo contínuo.
ContinuousModelBase	classe	()	Base da família com mse e gaussian_nll.

pinneapple_neural.architectures.continuous.bayesian_rnn

RNN bayesiana aproximada por MC Dropout, com dropout travado ao longo do tempo. [sem teste direto]

pinneapple_neural/architectures/continuous/bayesian_rnn.py · 200 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
LockedDropout	classe	(p=0.0)	Dropout com a mesma máscara em todos os passos de tempo.
BayesianRNN	classe	(input_dim, out_dim, hidden_dim=128, num_layers=2, dropout=0.1, cell='gru', min_logvar=-1…)	RNN (GRU/LSTM) com média e log-variância; predict_mc faz amostragem por MC Dropout.

pinneapple_neural.architectures.continuous.deep_state_space

Modelo de espaço de estados profundo variacional (filtragem), com NLL gaussiana e KL entre gaussianas diagonais. [sem teste direto]

pinneapple_neural/architectures/continuous/deep_state_space.py · 214 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ContOutput`	classe	`(y, losses, extras)`	Cópia local da saída padrão (y, perdas, extras) usada pelo modelo de espaço de estados.
`DeepStateSpaceModel`	classe	`(input_dim, out_dim, latent_dim=32, hidden_dim=128, mlp_depth=2, min_logvar=-10.0, max_lo…)`	Modelo de espaço de estados profundo variacional treinado por ELBO (NLL + `beta_kl` * KL).
`gaussian_nll_diag`	função	`(mu, logvar, y) -> torch.Tensor`	NLL de uma gaussiana diagonal.
`kl_diag_gaussians`	função	`(mu_q, logvar_q, mu_p, logvar_p) -> torch.Tensor`	Divergência KL entre duas gaussianas diagonais.

`pinneapple_neural.architectures.continuous.hamiltonian`

Hamiltonian Neural Network: aprende H(q,p) e deriva a dinâmica por autograd. [sem teste direto]

`pinneapple_neural/architectures/continuous/hamiltonian.py` · 69 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`HamiltonianNeuralNetwork`	classe	`(dim_q, hidden=128, num_layers=3)`	Rede que aprende H(q,p) e devolve dq/dt = dH/dp, dp/dt = -dH/dq.

`pinneapple_neural.architectures.continuous.latent_ode`

Latent ODE: encoder GRU-ODE para tempos irregulares, dinâmica latente por ODE neural e decoder gaussiano. [sem teste direto]

`pinneapple_neural/architectures/continuous/latent_ode.py` · 249 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`LatentODE`	classe	`(obs_dim, latent_dim=32, hidden=128, *, ode_method='dopri5', enc_method='dopri5', min_log…)`	Latent ODE com `forward` (reconstrução e NLL) e `forecast`.

`pinneapple_neural.architectures.continuous.neural_cde`

Equação diferencial controlada neural (CDE) sem depender de `torchcde`. [sem teste direto]

`pinneapple_neural/architectures/continuous/neural_cde.py` · 156 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`NeuralCDE`	classe	`(input_dim, hidden_dim, out_dim, *, f_hidden=128, g_hidden=128, num_layers=2, activation=…)`	CDE neural com solver Euler por padrão.

`pinneapple_neural.architectures.continuous.neural_gp`

Processo gaussiano neural: features aleatórias de Fourier e cabeça linear bayesiana com posterior condicionável. [sem teste direto]

`pinneapple_neural/architectures/continuous/neural_gp.py` · 248 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`NeuralGaussianProcess`	classe	`(in_dim, out_dim, *, embed_dim=64, mlp_hidden=128, mlp_layers=2, rff_features=512, rff_le…)`	GP neural com features RFF e cabeça linear bayesiana; `condition` atualiza o posterior.

`pinneapple_neural.architectures.continuous.neural_ode`

Neural ODE simples integrada numa grade de tempos com Euler ou RK4. [sem teste direto]

`pinneapple_neural/architectures/continuous/neural_ode.py` · 91 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`NeuralODE`	classe	`(state_dim, hidden=128, num_layers=3, method='rk4')`	ODE neural dy/dt = f(t, y) integrada com Euler ou RK4.

pinneapple_neural.architectures.continuous.neural_sde

SDE neural integrada por Euler–Maruyama, com difusão diagonal ou completa e vários caminhos. [sem teste direto]

pinneapple_neural/architectures/continuous/neural_sde.py · 264 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
NeuralSDE	classe	(state_dim, *, hidden=128, num_layers=3, diffusion='diag', activation='tanh', sigma_eps=1…)	SDE neural com deriva e difusão aprendidas, simulada por Euler–Maruyama.

pinneapple_neural.architectures.continuous.ode_rnn

ODE-RNN: integra o estado oculto entre observações e atualiza com GRUCell. [sem teste direto]

pinneapple_neural/architectures/continuous/ode_rnn.py · 54 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
ODERNN	classe	(input_dim, hidden_dim=128)	ODE-RNN para séries com observações irregulares.

pinneapple_neural.architectures.continuous.registry

Registro da família continuous e ContinuousCatalog. [sem teste direto]

pinneapple_neural/architectures/continuous/registry.py · 80 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
ContinuousCatalog	classe	(registry)	Catálogo da família continuous.
register_into_global	função	() -> None	Registra a família continuous no ModelRegistry global.

pinneapple_neural.architectures.continuous.symplectic_ode

Symplectic ODE-Net: hamiltoniano aprendido com passo simplético e rollout. [sem teste direto]

pinneapple_neural/architectures/continuous/symplectic_ode.py · 147 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
SymplecticODENet	classe	(dim_q, hidden=128)	Rede com hamiltoniano aprendido, campo vetorial, passo simplético e rollout.

pinneapple_neural.architectures.continuous.symplectic_rnn

RNN simplética com atualização tipo leapfrog (energias cinética e potencial separadas). [sem teste direto]

pinneapple_neural/architectures/continuous/symplectic_rnn.py · 128 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
SymplecticRNN	classe	(dim_q, *, hidden=128, num_layers=2, activation='tanh')	RNN simplética com gradientes de T(p) e V(q) aprendidos.

pinneapple_neural.architectures.convolutions

Família de modelos convolucionais 1D/2D/3D. [sem teste direto]

pinneapple_neural/architectures/convolutions/__init__.py · 5 linhas · **sem teste direto**

pinneapple_neural.architectures.convolutions.base

Saída ConvOutput e base ConvModelBase com mse. [sem teste direto]

pinneapple_neural/architectures/convolutions/base.py · 36 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ConvOutput`	classe	`(y, losses, extras)`	Saída padrão dos modelos convolucionais.
`ConvModelBase`	classe	`()`	Base da família convolucional com `mse`.

pinneapple_neural.architectures.convolutions.conv1d

Modelo Conv1D flexível: previsão causal, remoção de ruído, aprendizado de operador e surrogate de EDP 1D. [sem teste direto]

`pinneapple_neural/architectures/convolutions/conv1d.py` · 265 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Conv1DModel`	classe	`(in_channels, out_channels, *, mode='denoise', cond_channels=0, hidden_channels=64, num_b…)`	Modelo Conv1D com modos `denoise`, causal, operador e surrogate de EDP 1D.

pinneapple_neural.architectures.convolutions.conv2d

Rede Conv2D residual (estilo ResNet) para regressão de campos. [sem teste direto]

`pinneapple_neural/architectures/convolutions/conv2d.py` · 148 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Conv2DModel`	classe	`(in_channels, out_channels, hidden_channels=32, num_blocks=4, kernel_size=3, dropout=0.0,…)`	CNN 2D residual para regressão.

pinneapple_neural.architectures.convolutions.conv3d

Rede Conv3D residual para regressão de volumes, com skip global opcional. [sem teste direto]

`pinneapple_neural/architectures/convolutions/conv3d.py` · 174 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ResidualBlock3D`	classe	`(channels, kernel_size=3, dropout=0.0, norm_groups=8) -> None`	Bloco residual 3D (Conv, GroupNorm, GELU, Dropout); a soma residual fica a cargo de quem chama.
`Conv3DModel`	classe	`(in_channels, out_channels, hidden_channels=32, num_blocks=4, kernel_size=3, dropout=0.0,…)`	CNN 3D residual para regressão de volumes.

pinneapple_neural.architectures.convolutions.registry

Registro da família `convolutions` e `ConvolutionCatalog`. [sem teste direto]

`pinneapple_neural/architectures/convolutions/registry.py` · 60 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ConvolutionCatalog`	classe	`(registry)`	Catálogo da família `convolutions`.
`register_into_global`	função	`() -> None`	Registra a família `convolutions` no `ModelRegistry` global.

pinneapple_neural.architectures.graphnn

Família de redes em grafo; reexporta `GraphModelBase`, `GraphBatch`, `GraphOutput`, `MeshGraphNet`, `MeshDynamicsMGN`, `MeshGraph`, `Normalizer` e `GraphCatalog`. [sem teste direto]

`pinneapple_neural/architectures/graphnn/__init__.py` · 8 linhas · **sem teste direto**

pinneapple_neural.architectures.graphnn.base

Formato mínimo de batch de grafo (`GraphBatch`, sem dependências externas), saída `GraphOutput` e base `GraphModelBase`.

`pinneapple_neural/architectures/graphnn/base.py` · 54 linhas · testes: 15

nome	tipo	assinatura	o que faz
`GraphBatch`	classe	`(x, edge_index, pos, edge_attr, mask)`	Batch de grafo com `x`, `edge_index` e, opcionalmente, `pos`, `edge_attr` e `mask`.
`GraphOutput`	classe	`(y, losses, extras)`	Saída padrão das GNNs: `y` por nó, perdas e extras.
`GraphModelBase`	classe	`()`	Base das redes em grafo com `mse`.

`pinneapple_neural.architectures.graphnn.equivariant_gnn`

GNN E(n)-equivariante no estilo EGNN, que pode atualizar as posições dos nós.

`pinneapple_neural/architectures/graphnn/equivariant_gnn.py` · 229 linhas · testes: 5

nome	tipo	assinatura	o que faz
`EquivariantGNN`	classe	`(node_dim, pos_dim, out_dim, *, hidden=128, layers=4, edge_dim=0, dropout=0.0, update_pos…)`	GNN E(n)-equivariante (EGNN) com atualização opcional das posições.

`pinneapple_neural.architectures.graphnn.gnn`

Rede de passagem de mensagens (MPNN) simples para previsão por nó.

`pinneapple_neural/architectures/graphnn/gnn.py` · 123 linhas · testes: 5

nome	tipo	assinatura	o que faz
`GraphNeuralNetwork`	classe	`(node_dim, out_dim, *, edge_dim=0, hidden=128, layers=4, dropout=0.0)`	MPNN simples com features de aresta opcionais.

`pinneapple_neural.architectures.graphnn.gnn_ode`

ODE neural em grafo: o campo vetorial é uma GNN, com features de Fourier do tempo opcionais; os métodos `dopri5`/`adams` exigem `torchdiffeq`. [sem teste direto]

`pinneapple_neural/architectures/graphnn/gnn_ode.py` · 212 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`FourierTimeFeatures`	classe	`(n_frequencies, *, max_frequency=10.0)`	Features [sin, cos](2π f t) do tempo escalar.
`GraphNeuralODE`	classe	`(node_dim, *, edge_dim=0, hidden=128, layers=3, method='rk4', time_features=0, time_max_f…)`	ODE neural em grafo integrada com RK4 por padrão ou com `torchdiffeq` (`dopri5`, `adams`, adjunto opcional).

`pinneapple_neural.architectures.graphnn.graphcast`

Rede inspirada no GraphCast (encoder, processador em malha, decoder) para malha fixa ou grafos variáveis. [sem teste direto]

`pinneapple_neural/architectures/graphnn/graphcast.py` · 289 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`GraphCast`	classe	`(node_in, out_dim, *, hidden=256, processor_blocks=8, pos_dim=0, edge_dim=0, dropout=0.0)`	Modelo inspirado no GraphCast com blocos de processamento em malha.

`pinneapple_neural.architectures.graphnn.mesh_graph_net`

MeshGraphNet (Pfaff et al., ICLR 2021) encoder-processador-decoder com um único tipo de aresta (malha fixa); o conjunto de arestas no espaço do mundo, usado para contato, não está implementado. O inventário marca 'placeholder', mas o código é uma implementação real.

`pinneapple_neural/architectures/graphnn/mesh_graph_net.py` · 375 linhas · testes: 15 · palavras-chave no código (heurística; o texto acima diz o status real): placeholder/stub citado no docstring

nome	tipo	assinatura	o que faz
MeshGraphNet	classe	(node_in_dim, out_dim, *, edge_in_dim=0, pos_dim=0, hidden_dim=128, n_layers=2, n_message…)	GNN encoder-processador-decoder para simulação em malha não estruturada com conectividade fixa.

pinneapple_neural.architectures.graphnn.mgn_dynamics

Dinâmica transiente sobre MeshGraphNet: malha estática, normalizadores, passo autorregressivo de (velocidade, pressão), treino de um passo e avaliação de rollout.

pinneapple_neural/architectures/graphnn/mgn_dynamics.py · 270 linhas · testes: 6

nome	tipo	assinatura	o que faz
MeshGraph	classe	(edge_index, edge_attr, node_type)	Uma malha estática: topologia, features brutas das arestas e tipo de cada nó; from_mesh monta a partir de triângulos.
Normalizer	classe	(dim, eps=1e-08) -> None	Padronizador por feature com estatísticas fixas guardadas como buffers.
MeshDynamicsMGN	classe	(vel_dim=2, num_node_types=4, edge_dim=3, out_p=True, **mgn_kwargs) -> None	MeshGraphNet autorregressivo para (velocidade, pressão) em malha transiente: step, loss e rollout.
triangles_to_edges	função	(cells) -> torch.Tensor	Gera edge_index bidirecional e sem duplicatas a partir de triângulos.
edge_features	função	(pos, edge_index) -> torch.Tensor	Deslocamento relativo entre nós da aresta e sua norma.
train_dynamics	função	(model, data, *, steps, lr=0.0001, lr_final=1e-06, noise_std=0.02, device='cpu', log_ever…)	Treino de um passo em pares (trajetória, tempo) aleatórios, com Adam, ruído de entrada e decaimento exponencial da taxa de aprendizado.
eval_rollout	função	(model, data, *, n_steps, device='cpu', normal_type=0) -> dict[str, float]	RMSE da velocidade nos nós NORMAL, em rollout livre e em um passo, média sobre trajetórias.
frozen_ic_rmse	função	(data, *, n_steps, normal_type=0) -> float	RMSE de referência quando a velocidade fica congelada no valor inicial.

pinneapple_neural.architectures.graphnn.registry

Registro da família graphnn e GraphCatalog. [sem teste direto]

pinneapple_neural/architectures/graphnn/registry.py · 64 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
GraphCatalog	classe	(registry)	Catálogo da família graphnn.
register_into_global	função	() -> None	Registra a família graphnn no ModelRegistry global.

pinneapple_neural.architectures.graphnn.spatiotemporal_gnn

GNN espaço-temporal: GNN em cada instante seguida de uma GRU por nó ao longo do tempo. [sem teste direto]

pinneapple_neural/architectures/graphnn/spatiotemporal_gnn.py · 117 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
SpatiotemporalGNN	classe	(node_in, out_dim, *, edge_dim=0, spatial_hidden=128, spatial_layers=3, temporal_hidden=1…)	GNN espacial em cada instante, GRU por nó ao longo do tempo e cabeça por nó.

pinneapple_neural.architectures.graphnn.utils

Utilitários de scatter-add e de achatamento de vários grafos num grafo único com deslocamento de índices. [sem teste direto]

pinneapple_neural/architectures/graphnn/utils.py · 84 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`scatter_add`	função	`(src, index, dim_size) -> torch.Tensor`	Scatter-add para grafos compartilhados no batch.
`scatter_add_flat`	função	`(src, index, dim_size) -> torch.Tensor`	Scatter-add para um grafo achatado (batch de grafos variáveis).
`flatten_graph_batch`	função	`(x_list, edge_index_list, pos_list=None, edge_attr_list=None)`	Junta grafos por amostra num único grafo, deslocando os índices dos nós.

pinneapple_neural.architectures.group_b_registry

Registro do 'grupo B' (SIREN, ModifiedMLP, HashGridMLP, AFNO) com apelidos e capacidades; MeshGraphNet fica na família `graphnn`. [sem teste direto]

`pinneapple_neural/architectures/group_b_registry.py` · 86 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`GroupBCatalog`	classe	`(registry)`	Catálogo do grupo B (SIREN, ModifiedMLP, HashGridMLP, AFNO).
`register_into_global`	função	`() -> None`	Registra o grupo B no `ModelRegistry` com a família `group_b`.

pinneapple_neural.architectures.hash_grid

Codificação multirresolução por tabela hash (estilo Instant-NGP) e MLP que a usa para campos 3D. [sem teste direto]

`pinneapple_neural/architectures/hash_grid.py` · 219 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`HashGridEncoding`	classe	`(in_dim=3, n_levels=16, n_features_per_level=2, log2_hashmap_size=19, base_resolution=16,…)`	Codificação hash multirresolução com interpolação entre os cantos do voxel em cada nível.
`HashGridMLP`	classe	`(in_dim=3, out_dim=1, hidden_dim=64, n_hidden=2, n_levels=8, n_features_per_level=2, log2…)`	MLP pequeno sobre a codificação hash, para representar campos 3D.

pinneapple_neural.architectures.instantiate

Normaliza nomes de kwargs entre famílias (`in_dim`, `input_dim`, `in_channels`…), traduz `hidden_dim`/`n_layers` para os nomes de cada classe e descarta argumentos não aceitos. [sem teste direto, mas usado por `ModelRegistry.build`]

`pinneapple_neural/architectures/instantiate.py` · 173 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`InstantiateReport`	classe	`(original, normalized, dropped, used_aliases)`	Diagnóstico da normalização: kwargs originais, normalizados, descartados e apelidos usados.
`normalize_kwargs`	função	`(kwargs) -> Tuple[Dict[str, Any], Dict[str, str]]`	Converte apelidos de kwargs para os nomes canônicos.
`filter_supported_kwargs`	função	`(model_cls, kwargs) -> Tuple[Dict[str, Any], Dict[str, Any]]`	Mantém só os kwargs aceitos por `model_cls.__init__` (nenhum é descartado se ele aceita `**kwargs`).
`instantiate`	função	`(model_cls, kwargs=None, *, strict=False) -> Tuple[BaseModel, InstantiateReport]`	Instancia a classe com kwargs normalizados e devolve o modelo e um `InstantiateReport`; com `strict=True` falha se algo for descartado.

pinneapple_neural.architectures.modified_mlp

MLP modificado (Wang et al.): embedding de Fourier aleatório e conexões tipo highway U/V. [sem teste direto]

`pinneapple_neural/architectures/modified_mlp.py` · 136 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`FourierFeatureEmbedding`	classe	`(in_dim, n_fourier=64, sigma=1.0, trainable=False)`	Embedding [cos, sin](2π B x) com matriz `B` aleatória ou treinável.

nome	tipo	assinatura	o que faz
ModifiedMLP	classe	(in_dim, out_dim, hidden_dim=128, n_layers=6, n_fourier=32, sigma=1.0, activation=nn.Tanh…)	MLP com Fourier features e portas U/V tipo highway em cada camada.

pinneapple_neural.architectures.neural_operators

Família de operadores neurais; reexporta FNO, FNO2d, MLPFNOSurrogate, Transolver e as classes Noether (que ficam None se emmiai-noether não estiver instalado).

pinneapple_neural/architectures/neural_operators/__init__.py · 51 linhas · testes: 2

pinneapple_neural.architectures.neural_operators.base

Saída OperatorOutput e base NeuralOperatorBase com mse.

pinneapple_neural/architectures/neural_operators/base.py · 40 linhas · testes: 5

nome	tipo	assinatura	o que faz
OperatorOutput	classe	(y, losses, extras)	Saída padrão dos operadores neurais.
NeuralOperatorBase	classe	()	Base dos operadores neurais com mse.

pinneapple_neural.architectures.neural_operators.deeponet

DeepONet não empilhado (Lu et al. 2021): rede branch e rede trunk, um conjunto de coeficientes por canal de saída.

pinneapple_neural/architectures/neural_operators/deeponet.py · 79 linhas · testes: 5

nome	tipo	assinatura	o que faz
DeepONet	classe	(branch_dim, trunk_dim, out_dim, hidden=128, modes=64, depth=1, trunk_activation=False)	DeepONet branch-trunk não empilhado, com um conjunto de coeficientes por canal de saída.
mlp	função	(in_dim, hidden, out_dim, depth=1, act=nn.GELU, last_act=False) -> nn.Sequential	Monta um MLP a partir de uma lista de larguras ou de depth camadas de largura fixa.

pinneapple_neural.architectures.neural_operators.fno

Fourier Neural Operators 1D, 2D e 3D e um híbrido FNO + decoder MLP para avaliar em pontos espalhados.

pinneapple_neural/architectures/neural_operators/fno.py · 530 linhas · testes: 39

nome	tipo	assinatura	o que faz
FourierNeuralOperator	classe	(in_channels, out_channels, width=64, modes=16, layers=4, *, use_grid=False)	FNO 1D.
FNO2d	classe	(in_channels, out_channels, width=32, modes1=12, modes2=12, layers=4, *, use_grid=True)	FNO 2D para campos em grade, com canais de coordenadas opcionais.
FNO3d	classe	(in_channels, out_channels, width=32, modes1=8, modes2=8, modes3=8, layers=4, *, use_grid…)	FNO 3D para campos 3D ou 2D + tempo.
MLPFNOSurrogate	classe	(in_channels, out_channels, fno_width=32, fno_modes=12, fno_layers=4, mlp_hidden=128, mlp…)	Backbone FNO com decoder MLP para avaliar em pontos espalhados.

pinneapple_neural.architectures.neural_operators.fno_export

Exportação ONNX de FNOs: troca as convoluções espectrais 2D por versões em aritmética real e confere a saída com onnxruntime.

pinneapple_neural/architectures/neural_operators/fno_export.py · 121 linhas · testes: 9

nome	tipo	assinatura	o que faz
RealSpectralConv2d	classe	(conv, H, W)	Convolução espectral 2D reescrita em aritmética real para uma grade H x W fixa (exportável para ONNX).

nome	tipo	assinatura	o que faz
`to_exportable`	função	`(model, grid) -> nn.Module`	Copia o modelo trocando cada convolução espectral 2D pela versão real.
`export_onnx`	função	`(model, example, path, *, grid=None, opset=17, atol=0.0001) -> float`	Exporta para ONNX com eixo de batch dinâmico, compara com `onnxruntime` e devolve a maior diferença absoluta (falha acima de `atol`).

pinneapple_neural.architectures.neural_operators.gno

Galerkin Neural Operator em malha: aprende uma base Φ(x), projeta com pesos de quadratura, aplica o operador nos coeficientes e reconstrói. [sem teste direto]

`pinneapple_neural/architectures/neural_operators/gno.py` · 146 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`GalerkinNeuralOperator`	classe	`(in_dim, out_dim, basis_dim=64, basis_hidden=128, op_hidden=128, op_depth=2, orthonormali…)`	Operador de Galerkin com base aprendida e projeção por quadratura.

pinneapple_neural.architectures.neural_operators.ms_deeponet

DeepONet multiescala: um trunk por escala com coordenadas esticadas (ideia do MscaleDNN).

`pinneapple_neural/architectures/neural_operators/ms_deeponet.py` · 69 linhas · testes: 5

nome	tipo	assinatura	o que faz
`MultiScaleDeepONet`	classe	`(branch_dim, trunk_dim, out_dim, *, hidden=128, scales=(32, 64, 128), scale_factors=None,…)`	DeepONet com um trunk por escala espacial e branch comum.

pinneapple_neural.architectures.neural_operators.noether_bridge

[experimental; só pesquisa] Envolve os modelos de `emmiai-noether` (UPT, AB-UPT, Transformer, Transolver e variantes aero) como `NeuralOperatorBase`; cada instância passa por `require_research_use` e é recusada em modo comercial.

`pinneapple_neural/architectures/neural_operators/noether_bridge.py` · 296 linhas · testes: 5

nome	tipo	assinatura	o que faz
`NoetherModelConfig`	classe	`(model_name, config_kwargs, device)`	Config mínima repassada sem mudanças às classes de config do Noether.
`NoetherUPT`	classe	`(**config_kwargs)`	Wrapper do Universal Physics Transformer do Noether (só pesquisa).
`NoetherABUPT`	classe	`(**config_kwargs)`	Wrapper do AB-UPT do Noether (só pesquisa; pesos CC BY-NC).
`NoetherTransformer`	classe	`(**config_kwargs)`	Wrapper do Transformer simples do Noether (só pesquisa).
`NoetherTransolver`	classe	`(**config_kwargs)`	Wrapper do Transolver do Noether (só pesquisa).
`NoetherAeroUPT`	classe	`(**config_kwargs)`	Wrapper do UPT com roteamento superfície/volume para aerodinâmica (só pesquisa).
`NoetherAeroABUPT`	classe	`(**config_kwargs)`	Wrapper do AB-UPT com roteamento superfície/volume (só pesquisa).
`NoetherAeroTransformer`	classe	`(**config_kwargs)`	Wrapper do Transformer para CFD aerodinâmico (só pesquisa).
`NoetherAeroTransolver`	classe	`(**config_kwargs)`	Wrapper do Transolver para CFD aerodinâmico (só pesquisa).

pinneapple_neural.architectures.neural_operators.pino

PINO: wrapper que soma perdas físicas a um operador, mais derivadas espectrais/diferenças finitas em grade uniforme e uma perda de resíduo genérica. [sem teste direto]

`pinneapple_neural/architectures/neural_operators/pino.py` · 232 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`PhysicsInformedNeuralOperator`	classe	`(operator, *, lambda_pde=1.0, lambda_bc=1.0, lambda_ic=1.0, lambda_physics=1.0, prefix='p…)`	Wrapper PINO: roda o operador e soma as perdas físicas (EDP, contorno, inicial) pesadas.
`GridSpec`	classe	`(L, dims)`	Descreve a grade uniforme: tamanho físico por eixo e quais dimensões de `u` são eixos.
`ResidualSpec`	classe	`(residual, orders, method)`	Especifica o resíduo da EDP, as ordens de derivada e o método (espectral ou diferenças finitas).
`pino_physics_fn`	função	`(u, *, grid, spec, params=None, bc_mask=None, bc_value=None, ic_mask=None, ic_value=None,…)`	Calcula derivadas na grade, avalia o resíduo e devolve as perdas `pde`, `bc`, `ic` e `physics_total`.
`spectral_derivative`	função	`(u, *, dim, L, order=1) -> torch.Tensor`	Derivada de ordem `order` ao longo de um eixo via FFT (domínio periódico).
`fd_derivative`	função	`(u, *, dim, dx, order=1) -> torch.Tensor`	Derivada por diferenças finitas ao longo de um eixo.
`compute_derivatives`	função	`(u, *, grid, orders=(1, 2), method='spectral') -> Dict[Tuple[int, int], torch.Tensor]`	Dicionário de derivadas por (eixo, ordem), pelo método espectral ou de diferenças finitas.

`pinneapple_neural.architectures.neural_operators.registry`

Registro da família `neural_operators` com capacidades por chave (tipo de entrada e chaves esperadas); `transolver` fica reservado para o bridge Noether.

`pinneapple_neural/architectures/neural_operators/registry.py` · 123 linhas · testes: 8

nome	tipo	assinatura	o que faz
`NeuralOperatorCatalog`	classe	`(registry)`	Catálogo da família `neural_operators`.
`register_into_global`	função	`() -> None`	Registra os operadores no `ModelRegistry` com tipo de entrada e chaves esperadas por modelo.

`pinneapple_neural.architectures.neural_operators.transolver`

Transolver nativo em PyTorch (Physics-Attention por fatias) para nuvens de pontos, e uma versão leve `TransolverLite`. [sem teste direto]

`pinneapple_neural/architectures/neural_operators/transolver.py` · 145 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`PhysicsAttention`	classe	`(dim, heads=8, slices=32, dropout=0.0)`	Atenção física multi-cabeça: agrupa pontos em fatias, atende entre fatias e devolve aos pontos.
`TransolverBlock`	classe	`(dim, heads, slices, mlp_ratio=2, dropout=0.0)`	Bloco Transolver: Physics-Attention mais MLP com normalização.
`Transolver`	classe	`(in_dim, out_dim, dim=128, depth=4, heads=8, slices=32, mlp_ratio=2, dropout=0.0)`	Operador em nuvem de pontos (B, N, in_dim) -> (B, N, out_dim).
`TransolverLite`	classe	`(in_dim=5, out_dim=101, d_model=64, n_heads=4, n_slices=8, n_blocks=3, dropout=0.1)`	Versão leve: projeção linear, alguns blocos de atenção por fatias e cabeça linear por nó.

`pinneapple_neural.architectures.neural_operators.uno`

U-NO (operador em forma de U) para grades 1D/2D/3D e para pontos irregulares via kNN, unificado em `UniversalUNO`. [sem teste direto]

`pinneapple_neural/architectures/neural_operators/uno.py` · 494 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`SpectralConvND`	classe	`(dim, in_channels, out_channels, modes)`	Convolução espectral 1D/2D/3D (rfftn) nos modos de baixa frequência.
`OperatorBlockND`	classe	`(dim, width, modes)`	Bloco de operador espectral com caminho local.

nome	tipo	assinatura	o que faz
`DownBlockND`	classe	`(dim, in_ch, out_ch, modes, depth)`	Bloco de descida (redução de resolução) do U-NO em grade.
`UpBlockND`	classe	`(dim, in_ch, skip_ch, out_ch, modes, depth)`	Bloco de subida com skip do U-NO em grade.
`UNONdGrid`	classe	`(dim, in_channels, out_channels, width=64, levels=3, depth_per_level=2, modes=None, add_c…)`	U-NO para grades regulares (B, Cin, *espaço) -> (B, Cout, *espaço).
`PointKernelOperator`	classe	`(dim, in_ch, out_ch, hidden=64, k=16)`	Operador contínuo em pontos irregulares por passagem de mensagens kNN com kernel aprendido.
`PointOperatorBlock`	classe	`(dim, width, k=16)`	Bloco residual de operador em pontos.
`UNONdPoints`	classe	`(dim, in_channels, out_channels, width=128, levels=3, depth_per_level=2, k=16, pool_ratio…)`	U-NO para pontos irregulares com pirâmide multiescala por subamostragem.
`UniversalUNO`	classe	`(dim, in_channels, out_channels, *, mesh_mode='auto', width_grid=64, levels_grid=3, depth…)`	U-NO unificado que escolhe o modo grade ou pontos (`mesh_mode='auto'`).

pinneapple_neural.architectures.physics_aware

Família de redes com estrutura física (PANN, SPN). [sem teste direto]

`pinneapple_neural/architectures/physics_aware/__init__.py` · 5 linhas · **sem teste direto**

pinneapple_neural.architectures.physics_aware.base

Saída `PhysicsAwareOutput` e base `PhysicsAwareBase` com `mse`. [sem teste direto]

`pinneapple_neural/architectures/physics_aware/base.py` · 25 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`PhysicsAwareOutput`	classe	`(y, losses, extras)`	Saída padrão da família `physics_aware`.
`PhysicsAwareBase`	classe	`()`	Base da família com `mse`.

pinneapple_neural.architectures.physics_aware.pann

Envolve um backbone e soma uma perda física opcional, com componentes prefixados por `physics/`. [sem teste direto]

`pinneapple_neural/architectures/physics_aware/pann.py` · 135 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`PhysicsAwareNeuralNetwork`	classe	`(backbone, *, physics_loss_fn=None, physics_weight=1.0, physics_components_prefix='physic…)`	Backbone com função de perda física opcional e peso `physics_weight`.

pinneapple_neural.architectures.physics_aware.registry

Registro da família `physics_aware` e `PhysicsAwareCatalog`. [sem teste direto]

`pinneapple_neural/architectures/physics_aware/registry.py` · 47 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`PhysicsAwareCatalog`	classe	`(registry)`	Catálogo da família `physics_aware`.
`register_into_global`	função	`() -> None`	Registra a família `physics_aware` no `ModelRegistry` global.

pinneapple_neural.architectures.physics_aware.spn

Rede que aplica um projetor à saída do backbone para preservar uma estrutura (por exemplo, uma restrição). [sem teste direto]

`pinneapple_neural/architectures/physics_aware/spn.py` · 53 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`StructurePreservingNetwork`	classe	`(backbone, *, projector)`	Backbone seguido de um `projector` que impõe uma estrutura à saída.

pinneapple_neural.architectures.pinns

Família de PINNs; reexporta Vanilla, Inverse, PIELM, PINN-LSTM, PINNsFormer, VPINN, XPINN, XTFC, PINN bayesiano e difusão informada por física. [sem teste direto do __init__]

`pinneapple_neural/architectures/pinns/__init__.py` · 57 linhas · **sem teste direto**

pinneapple_neural.architectures.pinns.base

Saída `PINNOutput` e base `PINNBase` com gradientes por autograd, derivada normal, `physics_loss` e exportação TorchScript/ONNX.

`pinneapple_neural/architectures/pinns/base.py` · 261 linhas · testes: 5

nome	tipo	assinatura	o que faz
`PINNOutput`	classe	`(y, losses, extras)`	Saída padrão dos PINNs: `y`, perdas e extras.
`PINNBase`	classe	`()`	Base dos PINNs com gradiente, derivada normal, `predict`, `physics_loss` e exportação TorchScript/ONNX.

pinneapple_neural.architectures.pinns.bpinn

PINN bayesiano por inferência variacional de campo médio (camadas lineares bayesianas, ELBO, previsão por ensemble). [sem teste direto]

`pinneapple_neural/architectures/pinns/bpinn.py` · 389 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`BayesianLinear`	classe	`(in_features, out_features, *, prior_std=1.0, init_rho=-3.0, bias=True)`	Camada linear com pesos gaussianos (média e rho), amostragem e termo KL.
`BPINNConfig`	classe	`(in_dim, out_dim, hidden, activation, prior_std, init_rho, noise_pde, noise_obs, kl_weigh…)`	Configuração do `BayesianPINN` (arquitetura, prior, ruídos, peso do KL, parâmetros inversos).
`BayesianPINN`	classe	`(in_dim=2, out_dim=1, hidden=(64, 64, 64), activation='tanh', *, config=None, prior_std=1…)`	PINN bayesiano por VI de campo médio, com `elbo_loss` e previsão por ensemble (média e desvio).

pinneapple_neural.architectures.pinns.inverse

PINN inverso do catálogo, com parâmetros físicos treináveis e gancho de compatibilidade com `PINNFactory`. [sem teste direto]

`pinneapple_neural/architectures/pinns/inverse.py` · 118 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`InversePINN`	classe	`(in_dim, out_dim, hidden=(128, 128, 128, 128), activation='tanh', inverse_params=None, in…)`	PINN com parâmetros físicos desconhecidos treináveis; `as_factory_model` o adapta ao `PINNFactory`.

pinneapple_neural.architectures.pinns.physics_diffusion

Modelo de difusão informado por física: escalonadores VP e EDM, denoiser MLP, perda de score matching e guias por resíduo de EDP ou dados. As bases `NoiseSchedulerBase` e `GuidanceBase` levantam `NotImplementedError` (são abstratas). [sem teste direto]

`pinneapple_neural/architectures/pinns/physics_diffusion.py` · 787 linhas · **sem teste direto** · palavras-chave no código (heurística; o texto acima diz o status real): levanta `NotImplementedError` em algum caminho

nome	tipo	assinatura	o que faz
NoiseSchedulerBase	classe	()	Protocolo de escalonador de ruído; os métodos levantam NotImplementedError.
VPNoiseScheduler	classe	(T=1000, beta_min=0.0001, beta_max=0.02)	Escalonador VP-DDPM com beta linear.
EDMNoiseScheduler	classe	(sigma_min=0.002, sigma_max=80.0, rho=7.0)	Escalonador EDM (Karras et al.) com sigmas entre sigma_min e sigma_max.
SinusoidalEmbedding	classe	(dim)	Embedding senoidal do nível de ruído ou passo de tempo.
FiLMLayer	classe	(feat_dim, cond_dim)	Modulação FiLM das ativações condicionada ao nível de ruído.
MLPDenoiser	classe	(x_dim, t_emb_dim=64, hidden=(256, 256, 256), activation='gelu', predict_x0=False)	Denoiser MLP condicionado no tempo para campos 1D.
DSMLoss	classe	(denoiser, scheduler, loss_weight_fn=None)	Perda de denoising score matching.
GuidanceBase	classe	()	Protocolo de guia tipo DPS; o gradiente levanta NotImplementedError na base.
PDEResidualGuidance	classe	(residual_fn, weight=0.1)	Guia que empurra a amostra para reduzir o resíduo de uma EDP.
DataConsistencyGuidance	classe	(mask, y_obs, weight=0.5)	Guia de consistência com observações mascaradas (assimilação de dados, problemas inversos).
ComposedGuidance	classe	(*guidances)	Soma de várias guias.
PIDiffConfig	classe	(x_dim, t_emb_dim, hidden, activation, scheduler_type, T, beta_min, beta_max, sigma_min, …)	Configuração do PhysicsInformedDiffusion.
PhysicsInformedDiffusion	classe	(x_dim=64, hidden=(256, 256, 256), activation='gelu', *, denoiser=None, scheduler=None, c…)	Modelo de difusão com amostragem, ensemble, perda de treino e previsão guiada por física.

pinneapple_neural.architectures.pinns.pielm

Extreme learning machine informada por física: camada oculta aleatória fixa e saída ajustada por ridge. [sem teste direto]

pinneapple_neural/architectures/pinns/pielm.py · 141 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
PIELM	classe	(in_dim, out_dim, hidden_dim=1024, activation='tanh', freeze_random=True)	ELM informada por física: camada oculta aleatória e fit_ridge para a saída.
PIELMFactoryAdapter	classe	(pielm, *, inverse_params_names=None, initial_guesses=None, dtype=torch.float32)	Adapta um PIELM à interface do PINNFactory, com ajuste ridge a partir das entradas.

pinneapple_neural.architectures.pinns.pinn_lstm

Híbrido PINN-LSTM que reorganiza os pontos em sequências e aplica um LSTM. [sem teste direto]

pinneapple_neural/architectures/pinns/pinn_lstm.py · 75 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
PINNLSTM	classe	(in_dim, out_dim, *, seq_len, hidden_dim=128, num_layers=2, dropout=0.0, inverse_params_n…)	PINN que passa pontos agrupados em sequências de seq_len por um LSTM.

pinneapple_neural.architectures.pinns.pinnsformer

PINNsFormer: PINN baseado em Transformer que trata pontos como sequência.

pinneapple_neural/architectures/pinns/pinnsformer.py · 212 linhas · testes: 1

nome	tipo	assinatura	o que faz
PINNsFormer	classe	(in_dim, out_dim, *, seq_len=32, d_model=128, nhead=4, num_layers=4, dim_feedforward=256,…)	PINN Transformer com embedding posicional e máscaras configuráveis.

pinneapple_neural.architectures.pinns.registry

Registro da família pinns e PINNCatalog. [sem teste direto]

pinneapple_neural/architectures/pinns/registry.py · 78 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
PINNCatalog	classe	(registry)	Catálogo da família pinns.
register_into_global	função	() -> None	Registra a família pinns no ModelRegistry global.

pinneapple_neural.architectures.pinns.vanilla

PINN MLP básico (forma forte), com parâmetros inversos opcionais exigidos pela perda do PINNFactory.

pinneapple_neural/architectures/pinns/vanilla.py · 94 linhas · testes: 13

nome	tipo	assinatura	o que faz
VanillaPINN	classe	(in_dim, out_dim, hidden=(128, 128, 128, 128), activation='tanh', *, inverse_params_names…)	PINN MLP com ativação configurável e parâmetros inversos opcionais.

pinneapple_neural.architectures.pinns.vpinn

PINN variacional (forma fraca) e um adaptador de perda para o PINNFactory. [sem teste direto]

pinneapple_neural/architectures/pinns/vpinn.py · 119 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
VPINN	classe	(in_dim, out_dim, hidden=(128, 128, 128), activation='tanh', *, inverse_params_names=None…)	PINN variacional (forma fraca).
pinn_factory_adapter	função	(loss_fn)	Converte uma função de perda em physics_fn no formato esperado pelo PINNFactory.

pinneapple_neural.architectures.pinns.xpinn

XPINN: uma sub-rede por subdomínio, com perdas de interface (valor e fluxo). [sem teste direto]

pinneapple_neural/architectures/pinns/xpinn.py · 126 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
SubnetWrapper	classe	(in_dim, out_dim, hidden=(128, 128, 128, 128), activation='tanh')	Sub-rede de um subdomínio (um VanillaPINN envolvido).
XPINN	classe	(n_subdomains, in_dim, out_dim, hidden=(128, 128, 128, 128), activation='tanh', interface…)	PINN com decomposição de domínio: uma sub-rede por subdomínio e perdas de interface.

pinneapple_neural.architectures.pinns.xtfc

X-TFC: features aleatórias com saída ajustada por ridge e expressões restritas (biblioteca tfc, opcional) que satisfazem contorno/condição inicial exatamente.

pinneapple_neural/architectures/pinns/xtfc.py · 681 linhas · testes: 4

nome	tipo	assinatura	o que faz
TFCConstrainedExpression	classe	(tfc_obj, dim=1)	Envolve a biblioteca tfc para montar expressões restritas 1D ou nD que cumprem exatamente as condições.

nome	tipo	assinatura	o que faz
XTFCConfig	classe	(in_dim, out_dim, rf_dim, activation, freeze_random, rf_kind, rff_sigma, use_bias, init_s…)	Configuração do XTFC (features aleatórias, regularização, uso de TFC).
XTFC	classe	(in_dim, out_dim, rf_dim=2048, activation=None, freeze_random=True, g_fn=None, B_fn=None,…)	PINN X-TFC: features aleatórias congeladas, saída por ridge e forma restrita g + B·N.
XTFCFactoryModel	classe	(xtfc, inverse_params_names=None, initial_guesses=None, dtype=torch.float32)	Wrapper com forward(x) -> Tensor compatível com o PINNFactory e o Trainer.
tfc_available	função	() -> bool	Diz se a biblioteca tfc está instalada.
build_xtfc	função	(in_dim, out_dim, *, rf_dim=2048, activation=None, freeze_random=True, g_fn=None, B_fn=No…)	Construtor de conveniência do XTFC, com TFC opcional.

pinneapple_neural.architectures.recurrent

Família de modelos recorrentes de previsão (GRU, LSTM, Seq2Seq). [sem teste direto]

pinneapple_neural/architectures/recurrent/__init__.py · 5 linhas · **sem teste direto**

pinneapple_neural.architectures.recurrent.base

Saída RNNOutput e base RecurrentModelBase com mse. [sem teste direto]

pinneapple_neural/architectures/recurrent/base.py · 36 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
RNNOutput	classe	(y, losses, extras)	Saída padrão dos modelos recorrentes.
RecurrentModelBase	classe	()	Base dos modelos recorrentes com mse.

pinneapple_neural.architectures.recurrent.gru

Previsores GRU e GRU bidirecional com horizonte fixo, pooling e embedding de tempo opcionais. [sem teste direto]

pinneapple_neural/architectures/recurrent/gru.py · 199 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
GRUModel	classe	(in_dim, out_dim, horizon, hidden_dim=128, num_layers=2, dropout=0.0, *, pool='last', tim…)	Previsor GRU para horizon passos.
BiGRUModel	classe	(in_dim, out_dim, horizon, hidden_dim=128, num_layers=2, dropout=0.0, *, pool='last', tim…)	Previsor GRU bidirecional.

pinneapple_neural.architectures.recurrent.lstm

Previsores LSTM e LSTM bidirecional com horizonte fixo. [sem teste direto]

pinneapple_neural/architectures/recurrent/lstm.py · 201 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
LSTMModel	classe	(in_dim, out_dim, horizon, hidden_dim=128, num_layers=2, dropout=0.0, *, pool='last', tim…)	Previsor LSTM para horizon passos.
BiLSTMModel	classe	(in_dim, out_dim, horizon, hidden_dim=128, num_layers=2, dropout=0.0, *, pool='last', tim…)	Previsor LSTM bidirecional.

pinneapple_neural.architectures.recurrent.registry

Registro da família recurrent e RecurrentCatalog. [sem teste direto]

pinneapple_neural/architectures/recurrent/registry.py · 57 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`RecurrentCatalog`	classe	`(registry)`	Catálogo da família `recurrent`.
`register_into_global`	função	`() -> None`	Registra a família `recurrent` no `ModelRegistry` global.

pinneapple_neural.architectures.recurrent.seq2seq

Previsor Seq2Seq (encoder-decoder GRU ou LSTM) com covariáveis futuras opcionais. [sem teste direto]

`pinneapple_neural/architectures/recurrent/seq2seq.py` · 215 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Seq2SeqRNN`	classe	`(in_dim, out_dim, horizon, hidden_dim=128, num_layers=2, dropout=0.0, cell='gru', future_…)`	Previsor encoder-decoder recorrente.

pinneapple_neural.architectures.register_all

Popula o `ModelRegistry` importando o registro de cada família; idempotente, ignora famílias com dependência faltando (log em debug) e só avisa em colisão de nomes. [sem teste direto]

`pinneapple_neural/architectures/register_all.py` · 44 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`register_all`	função	`() -> None`	Chama `register_into_global` de cada família uma única vez.

pinneapple_neural.architectures.registry

`ModelSpec` e `ModelRegistry`: registro central por nome, com família, tags, tipo de entrada e chaves esperadas; `build` passa pela normalização de kwargs de `instantiate`.

`pinneapple_neural/architectures/registry.py` · 101 linhas · testes: 64

nome	tipo	assinatura	o que faz
`ModelSpec`	classe	`(name, family, cls, description, tags, input_kind, supports_physics_loss, expects, predic…)`	Ficha de um modelo registrado: nome, família, classe, descrição, tags, tipo de entrada, suporte a perda física, chaves esperadas e campos previstos.
`ModelRegistry`	classe	`(_REGISTRY)`	Registro central por nome: `register` (decorador), `list`, `families`, `spec`, `build` e `build_with_report`.

pinneapple_neural.architectures.reservoir_computing

Família de computação de reservatório e modelos de features aleatórias com saída por ridge (ELM, ESN, RBF, Koopman/EDMD). [sem teste direto]

`pinneapple_neural/architectures/reservoir_computing/__init__.py` · 5 linhas · **sem teste direto**

pinneapple_neural.architectures.reservoir_computing.base

Saída `RCOutput` e base `RCBase` com `mse` e solução ridge. [sem teste direto]

`pinneapple_neural/architectures/reservoir_computing/base.py` · 48 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`RCOutput`	classe	`(y, losses, extras)`	Saída padrão dos modelos de reservatório.
`RCBase`	classe	`()`	Base com `mse` e `ridge_solve`.

pinneapple_neural.architectures.reservoir_computing.elm

Extreme Learning Machine: camada oculta aleatória e saída por ridge em forma fechada. [sem teste direto]

`pinneapple_neural/architectures/reservoir_computing/elm.py` · 88 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ExtremeLearningMachine`	classe	`(in_dim, out_dim, hidden_dim=1024, activation='tanh', l2=1e-06, freeze_random=True)`	ELM com `fit` por ridge e `forward`.

pinneapple_neural.architectures.reservoir_computing.esn

Echo State Network com reservatório vazado (leak), saída por ridge e previsão autorregressiva. [sem teste direto]

`pinneapple_neural/architectures/reservoir_computing/esn.py` · 340 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`EchoStateNetwork`	classe	`(in_dim, out_dim, reservoir_dim=1024, spectral_radius=0.9, leak=1.0, input_scale=1.0, l2=…)`	ESN com `fit` por ridge e `predict_autoregressive`.

pinneapple_neural.architectures.reservoir_computing.esn_rc

Variante de ESN com leitura aumentada por termos polinomiais do estado. [sem teste direto]

`pinneapple_neural/architectures/reservoir_computing/esn_rc.py` · 113 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ESNRC`	classe	`(in_dim, out_dim, reservoir_dim=1024, *, spectral_radius=0.9, leak=1.0, input_scale=1.0, …)`	ESN com leitura polinomial do estado.

pinneapple_neural.architectures.reservoir_computing.hybrid_rbf

Rede RBF híbrida: y = [Φ(x), x, 1] W, resolvida num único sistema ridge. [sem teste direto]

`pinneapple_neural/architectures/reservoir_computing/hybrid_rbf.py` · 158 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`HybridRBFNetwork`	classe	`(in_dim, out_dim, num_centers=512, sigma=None, l2=1e-06, learn_centers=False, use_linear=…)`	Rede RBF com termos linear e bias, resolvida por ridge.

pinneapple_neural.architectures.reservoir_computing.koopman

Operador de Koopman estilo EDMD num espaço elevado (RFF ou polinomial), ajustado por ridge. [sem teste direto]

`pinneapple_neural/architectures/reservoir_computing/koopman.py` · 155 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`KoopmanOperator`	classe	`(in_dim, *, lift_dim=1024, lift='rff', poly_degree=2, rff_lengthscale=1.0, l2=1e-06, use_…)`	Operador de Koopman EDMD: eleva x, ajusta K por ridge e faz rollout.

pinneapple_neural.architectures.reservoir_computing.rbf

Rede RBF gaussiana com leitura por ridge e opção de ajuste por gradiente. [sem teste direto]

`pinneapple_neural/architectures/reservoir_computing/rbf.py` · 263 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`RBFNetwork`	classe	`(in_dim, out_dim, num_centers=512, sigma=None, l2=1e-06, learn_centers=False, *, normaliz…)`	Rede RBF gaussiana com `fit` (ridge) e `fit_gd` (gradiente).
`sigma_is_default`	função	`(log_sigma, eps=1e-06) -> bool`	Diz se a largura `log_sigma` ainda está no valor padrão (sem ajuste).

pinneapple_neural.architectures.reservoir_computing.registry

Registro da família `reservoir_computing` e `ReservoirCatalog`. [sem teste direto]

pinneapple_neural/architectures/reservoir_computing/registry.py · 70 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
ReservoirCatalog	classe	(registry)	Catálogo da família reservoir_computing.
register_into_global	função	() -> None	Registra a família reservoir_computing no ModelRegistry global.

pinneapple_neural.architectures.rom

Família de modelos de ordem reduzida (POD, DMD, HAVOK, OpInf, SINDy, Koopman, ROM neural, POD paramétrico) e o estudo rom_study.

pinneapple_neural/architectures/rom/__init__.py · 39 linhas · testes: 25

pinneapple_neural.architectures.rom.base

Saída ROMOutput e base ROMBase com mse. [sem teste direto]

pinneapple_neural/architectures/rom/base.py · 31 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
ROMOutput	classe	(y, losses, extras)	Saída padrão dos ROMs.
ROMBase	classe	()	Base dos ROMs com mse.

pinneapple_neural.architectures.rom.contracts

Converte resultados do estudo de ROM nos contratos abertos de pinneapple_data.contracts (ModelRef, StudyResult), para outros apps usarem o ROM. [sem teste direto]

pinneapple_neural/architectures/rom/contracts.py · 120 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
model_ref	função	(rm, *, name, uri, payload=None, row=None, split='test', dataset_hash='', dataset_name=''…)	Monta o ModelRef de um ReducedModel: entradas com faixa de treino, saídas, métricas e SHA-256 do arquivo.
rom_checks	função	(res, tolerance) -> list	Lista os checks do estudo com valores: melhor ROM contra o baseline, contra a tolerância e divergência dos passos no tempo.
study_result	função	(res, *, study_id, title, tolerance, models=(), artifacts=(), limits=(), app='rom', datas…)	Monta o StudyResult do estudo: uma linha por (método, posto), o melhor modelo e os checks.

pinneapple_neural.architectures.rom.deep_uq_rom

[placeholder] Esqueleto ('MVP scaffold' no docstring) de preditor latente que devolve média e log-variância. [sem teste direto]

pinneapple_neural/architectures/rom/deep_uq_rom.py · 61 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
DeepUQROM	classe	(r, hidden=256, layers=2, dropout=0.0)	Preditor latente pequeno que devolve média e log-variância (esqueleto).

pinneapple_neural.architectures.rom.dmd

Decomposição em modos dinâmicos truncada (em lote, estabilizada, com análise espectral) e DMD com controle (DMDc, NumPy). [sem teste direto]

pinneapple_neural/architectures/rom/dmd.py · 346 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
DynamicModeDecomposition	classe	(r=64, center='global', l2=0.0)	DMD truncada de posto r com autovalores, frequências e rollout.

nome	tipo	assinatura	o que faz
`DMDc`	classe	`(affine=False)`	DMD com controle (Proctor, Brunton & Kutz 2016) em NumPy.

`pinneapple_neural.architectures.rom.havok`

HAVOK: embedding de Hankel, SVD e dinâmica linear por DMD no subespaço. [sem teste direto]

`pinneapple_neural/architectures/rom/havok.py` · 225 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`HAVOK`	classe	`(delays=50, r=64, center=True, svd_eps=1e-12, decode_mode='last')`	Modelo HAVOK com `fit` e `forward` (rollout no subespaço).

`pinneapple_neural.architectures.rom.koopman`

Operador de Koopman treinável (completo ou em blocos) e autoencoder de Koopman profundo. [sem teste direto]

`pinneapple_neural/architectures/rom/koopman.py` · 269 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`KoopmanOperator`	classe	`(k_dim, structure='full', num_blocks=4)`	Matriz de Koopman treinável, completa ou bloco-diagonal.
`KoopmanAutoencoder`	classe	`(in_dim, k_dim=128, hidden=128, n_layers=3, activation='tanh', k_structure='full', num_bl…)`	Autoencoder de Koopman profundo com perdas de reconstrução, previsão e linearidade.

`pinneapple_neural.architectures.rom.neural_rom`

ROM totalmente neural: encoder não linear, dinâmica latente (MLP) e decoder. [sem teste direto]

`pinneapple_neural/architectures/rom/neural_rom.py` · 237 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`NeuralROM`	classe	`(in_dim, r=32, hidden_enc=128, n_layers_enc=3, dynamics='mlp', hidden_dyn=128, n_layers_d…)`	ROM neural com encoder, dinâmica latente, decoder e rollout.

`pinneapple_neural.architectures.rom.opinf`

Operator Inference para dinâmica latente (termos linear, quadrático e bias, com regularização), em tempo discreto ou contínuo. [sem teste direto]

`pinneapple_neural/architectures/rom/opinf.py` · 387 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`OperatorInference`	classe	`(r, *, use_quadratic=True, use_bias=True, l2_linear=1e-06, l2_quad=1e-06, l2_bias=0.0, ce…)`	Operator Inference com operadores A, H e b, ajuste discreto ou contínuo e rollout.

`pinneapple_neural.architectures.rom.parametric`

ROM paramétrico: parâmetros de projeto -> campo completo sem rodar o solver (base POD + regressão dos coeficientes) e amostragem por hipercubo latino. [sem teste direto]

`pinneapple_neural/architectures/rom/parametric.py` · 198 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ParametricPOD`	classe	`(r, energy, regressor, kernel, smoothing, normalize, seed, mean_, basis_, sv_)`	Base POD com regressão dos coeficientes nos parâmetros (RBF ou GP), com predição, erro de projeção e serialização em arrays.

nome	tipo	assinatura	o que faz
latin_hypercube	função	(n, bounds, seed=0) -> tuple[np.ndarray, list[str]]	n pontos de hipercubo latino numa caixa de parâmetros; devolve pontos e nomes.

pinneapple_neural.architectures.rom.pod

POD por SVD completa ou randomizada, com ajuste incremental, para compressão. [sem teste direto]

pinneapple_neural/architectures/rom/pod.py · 252 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
POD	classe	(r=64, center=True, energy=None, randomised=False, random_seed=0)	POD por SVD completa ou randomizada, com encode, decode e erro de reconstrução.

pinneapple_neural.architectures.rom.registry

Registro da família rom e ROMCatalog. [sem teste direto]

pinneapple_neural/architectures/rom/registry.py · 82 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
ROMCatalog	classe	(registry)	Catálogo da família rom.
register_into_global	função	() -> None	Registra a família rom no ModelRegistry global.

pinneapple_neural.architectures.rom.rom_hybrid

ROM híbrido: POD, Operator Inference no latente e corretor neural do erro de truncamento. [sem teste direto]

pinneapple_neural/architectures/rom/rom_hybrid.py · 177 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
ROMHybrid	classe	(field_dim, r=64, l2=1e-06, use_corrector=True, corrector_hidden=64)	ROM híbrido POD + OpInf + corretor MLP.

pinneapple_neural.architectures.rom.sindy

SINDy para dinâmica latente: biblioteca de funções e regressão esparsa (STLSQ). [sem teste direto]

pinneapple_neural/architectures/rom/sindy.py · 323 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
SINDy	classe	(r, *, library='poly', poly_degree=2, threshold=0.05, n_iter=10, l2=0.0001, solver='stlsq…)	SINDy no latente com biblioteca polinomial, limiar de esparsidade e integrador RK4.

pinneapple_neural.architectures.rom.steppers

Funções de passo step(u) -> u_next de ROMs lineares (POD+DMD, POD+OpInf) no formato que pp.validate audita em rollout. [sem teste direto]

pinneapple_neural/architectures/rom/steppers.py · 48 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
pod_dmd_stepper	função	(snapshots, r=8)	Monta o passo POD + DMD exata e devolve também o maior módulo de autovalor (acima de 1, o ROM amplifica).
pod_opinf_stepper	função	(train, energy) -> tuple[Callable[[np.ndarray], np.ndarray], int]	Monta o passo POD + OpInf linear e devolve o posto escolhido pela energia.

pinneapple_neural.architectures.rom.study

Estudo de ROM: ajusta vários métodos e postos em snapshots de treino e mede erro, tempo e ganho de velocidade em snapshots separados. Exportação ONNX levanta `NotImplementedError` para métodos e opções sem suporte.

`pinneapple_neural/architectures/rom/study.py` · 787 linhas · testes: 11 · palavras-chave no código (heurística; o texto acima diz o status real): levanta `NotImplementedError` em algum caminho

nome	tipo	assinatura	o que faz
`Split`	classe	`(train, test, how)`	Índices de treino e teste e o modo de separação.
`ReducedModel`	classe	`(method, mode, rank, arrays, config, fit_s, _cache)`	ROM ajustado com `predict`, manifesto, forma portátil `.npz` e exportação ONNX quando suportada.
`ROMStudyResult`	classe	`(mode, split, singular_values, rows, projection, skipped, budget_s, hf_seconds, metric, m…)`	Resultado do estudo: linhas por método e posto, valores singulares, erro de projeção, melhor modelo e frente de Pareto.
`default_ranks`	função	`(n_train, n_values, sv=None, max_rank=64) -> list[int]`	Postos padrão: potências de 2 até o posto dos dados, mais os postos que guardam 99 %, 99,9 % e 99,99 % da energia.
`split_snapshots`	função	`(snaps, *, how='auto', test_fraction=0.2, seed=0, test_idx=None) -> Split`	Separa snapshots de teste (previsão, intercalado ou aleatório para dados paramétricos).
`field_scales`	função	`(snaps, train) -> np.ndarray`	Uma escala por valor: o RMS de cada campo nos snapshots de treino.
`fit_reduced_model`	função	`(snaps, train, method, rank=None, *, budget_s=5.0, seed=0, mode=None, scale='auto', log_p…)`	Ajusta um ROM de um método e posto nos snapshots de treino.
`rom_study`	função	`(snaps, *, methods=None, ranks=None, split='auto', test_fraction=0.2, test_idx=None, budg…)`	Ajusta todos os métodos em todos os postos e pontua cada um nos snapshots separados.

pinneapple_neural.architectures.rom.toy_systems

Sistemas lineares sintéticos para demos e testes de ROM. [sem teste direto]

`pinneapple_neural/architectures/rom/toy_systems.py` · 27 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`make_linear_system`	função	`(D, seed=0) -> torch.Tensor`	Operador linear aleatório contrativo (maior valor singular 0,95).
`generate_sequences`	função	`(B, T, D, A, noise=0.01, seed=123) -> torch.Tensor`	Gera trajetórias ruidosas de x_{k+1} = A x_k, determinísticas para uma `seed`.

pinneapple_neural.architectures.rom.trusted

Surrogate latente que diz quando não confiar: limite de erro conformal, teste de envelope e resíduo físico por consulta, convertidos num `TrustReport`.

`pinneapple_neural/architectures/rom/trusted.py` · 192 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`TrustedLatentSurrogate`	classe	`(rom, spread, members, subsample, neighbours, alpha, alpha_envelope, alpha_residual, resi…)`	POD latente com limite de erro, p-valor de envelope e resíduo físico por consulta; `trust_report` gera o relatório de confiança.

pinneapple_neural.architectures.rom.views

Visão de um snapshot separado na malha: campo de referência, previsão, diferença, regiões de contorno e carga, pontos de maior erro e um texto curto.

`pinneapple_neural/architectures/rom/views.py` · 122 linhas · testes: 11

nome	tipo	assinatura	o que faz
zones	função	(snaps, points, grow=0.1) -> dict[str, np.ndarray]	Regiões de comparação do erro: conjuntos nomeados de nós crescidos (zona de engaste, de carga) e o resto do corpo.
reconstruction_view	função	(snaps, model, index, *, reported=None, grow=0.1, max_faces=None, n_hotspots=3, full_labe…)	Visão de um snapshot contra a previsão de um ReducedModel.
prediction_view	função	(snaps, index, pred, *, label='Prediction', note='', reported=None, grow=0.1, max_faces=N…)	Visão de um snapshot contra uma previsão qualquer, com marcadores, regiões e pontos de maior erro.

pinneapple_neural.architectures.siren

SIREN (Sitzmann et al. 2020): MLP com ativação seno e inicialização própria, para campos de alta frequência. [sem teste direto]

pinneapple_neural/architectures/siren.py · 135 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
SineLayer	classe	(in_features, out_features, bias=True, is_first=False, omega_0=30.0)	Camada senoidal com inicialização de pesos de Sitzmann et al.
SIREN	classe	(in_dim, out_dim, hidden_dim=256, n_layers=5, omega_0=30.0, outermost_linear=True, final_…)	Rede SIREN com omega_0 configurável e última camada linear por padrão.

pinneapple_neural.architectures.transformers

Família de Transformers para séries temporais (Autoformer, FEDformer, Informer, TFT, TimesNet, Transformer). [sem teste direto]

pinneapple_neural/architectures/transformers/__init__.py · 5 linhas · **sem teste direto**

pinneapple_neural.architectures.transformers.autoformer

Autoformer mínimo: decomposição tendência/sazonal e auto-correlação via FFT. [sem teste direto]

pinneapple_neural/architectures/transformers/autoformer.py · 367 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
SeriesDecomp	classe	(kernel_size=25)	Decomposição por média móvel: sazonal + tendência.
AutoCorrelation	classe	(topk_factor=1.0, dropout=0.0)	Auto-correlação via FFT com agregação por atraso top-k.
AutoCorrelationLayer	classe	(d_model, nhead, topk_factor=1.0, dropout=0.0)	Camada multi-cabeça de auto-correlação com projeções.
EncoderLayer	classe	(d_model, nhead, d_ff, kernel_size, dropout, topk_factor)	Camada de encoder do Autoformer.
DecoderLayer	classe	(d_model, nhead, d_ff, kernel_size, dropout, topk_factor)	Camada de decoder do Autoformer que acumula a tendência.
Autoformer	classe	(in_dim, out_dim, horizon, d_model=128, nhead=4, num_layers=2, kernel_size=25, dropout=0.…)	Modelo Autoformer completo para previsão de horizon passos.

pinneapple_neural.architectures.transformers.base

Saída TSOutput e base TimeSeriesModelBase, cujo forward levanta NotImplementedError (abstrata). [sem teste direto]

pinneapple_neural/architectures/transformers/base.py · 48 linhas · **sem teste direto** · palavras-chave no código (heurística; o texto acima diz o status real): levanta NotImplementedError em algum caminho

nome	tipo	assinatura	o que faz
TSOutput	classe	(y, losses, extras)	Saída padrão dos Transformers de série temporal.
TimeSeriesModelBase	classe	()	Base abstrata de previsão de sequências com mse.

pinneapple_neural.architectures.transformers.fedformer

Modelo rápido inspirado no FEDformer: seleção de frequências top-k e MLP. Não é o FEDformer completo. [sem teste direto]

pinneapple_neural/architectures/transformers/fedformer.py · 89 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
FEDformer	classe	(in_dim, out_dim, horizon, d_model=128, topk_freq=32)	Modelo rápido inspirado no FEDformer (frequências top-k e MLP).

pinneapple_neural.architectures.transformers.informer

Encoder Transformer com destilação opcional, inspirado no Informer (usa atenção padrão, não ProbSparse). [sem teste direto]

pinneapple_neural/architectures/transformers/informer.py · 132 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
SinusoidalPositionalEncoding	classe	(d_model, max_len=10000)	Codificação posicional senoidal.
Informer	classe	(in_dim, out_dim, horizon, d_model=128, nhead=4, num_layers=4, dim_feedforward=256, dropo…)	Encoder Transformer com destilação opcional e cabeça de previsão.

pinneapple_neural.architectures.transformers.registry

Registro da família `transformers` e `TransformerCatalog`. [sem teste direto]

pinneapple_neural/architectures/transformers/registry.py · 61 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
TransformerCatalog	classe	(registry)	Catálogo da família `transformers`.
register_into_global	função	() -> None	Registra a família `transformers` no `ModelRegistry` global.

pinneapple_neural.architectures.transformers.tft

Versão leve do Temporal Fusion Transformer: seleção de variáveis por grupo, encoder causal, decoder com consultas futuras e portas GRN/GLU. [sem teste direto]

pinneapple_neural/architectures/transformers/tft.py · 254 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
GRN	classe	(d, hidden, dropout=0.0)	Gated Residual Network simplificada.
GLU	classe	(d)	Gated Linear Unit para portas de skip.
GroupVSN	classe	(d_model, num_groups, hidden, dropout=0.0)	Rede de seleção de variáveis por grupo.
TemporalFusionTransformer	classe	(in_dim, out_dim, horizon, d_model=128, nhead=4, num_enc_layers=2, num_dec_layers=2, drop…)	TFT simplificado para previsão multi-passo.

pinneapple_neural.architectures.transformers.timesnet

Previsor no estilo TimesNet: períodos por FFT, mapas 2D e blocos inception. [sem teste direto]

pinneapple_neural/architectures/transformers/timesnet.py · 199 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
TimesNet	classe	(in_dim, out_dim, horizon, d_model=128, num_blocks=2, top_k_periods=3, head_window=64, dr…)	Previsor no estilo TimesNet.

pinneapple_neural.architectures.transformers.transformer

Encoder Transformer causal para previsão autorregressiva. [sem teste direto]

pinneapple_neural/architectures/transformers/transformer.py · 113 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
VanillaTransformer	classe	(in_dim, out_dim, d_model=128, nhead=4, num_layers=4, dim_feedforward=256, dropout=0.1, p…)	Encoder Transformer causal com codificação posicional aprendida ou senoidal.

pinneapple_neural.callbacks

Roda os checks de pp.validate durante o treino em PyTorch Lightning, PhysicsNeMo e Hugging Face: registra métricas a cada N passos, avisa (ou para) quando uma métrica física piora enquanto a perda cai e grava relatório e trust card no fim.

pinneapple_neural/callbacks.py · 274 linhas · testes: 4

nome	tipo	assinatura	o que faz
PhysicsWarning	classe	()	Aviso: uma métrica física piorou em vários checks seguidos enquanto a perda de treino melhorava.
PhysicsMonitor	classe	(problem, data=None, *, reference=None, query=None, level='quick', every=100, patience=3,…)	Núcleo sem framework: roda os checks de pp.validate a cada every passos, acompanha as métricas e avisa ou para após patience pioras.
PhysicsNeMoHook	classe	(monitor)	Gancho para um laço de treino PhysicsNeMo, chamado após cada passo do otimizador.

pinneapple_neural.lpm

Blocos de um 'Large Physics Model': um modelo para um espaço de projeto inteiro, com codificação de Fourier, vizinhança multiescala, código de geometria (DeepSets), cabeças em ensemble para incerteza e sinal fora-da-distribuição.

pinneapple_neural/lpm.py · 212 linhas · testes: 5

nome	tipo	assinatura	o que faz
FourierEncoding	classe	(n_freq=6, include_input=True)	Codificação posicional [x, sin(2^k π x), cos(2^k π x)] por coordenada.
GeometryEncoder	classe	(d, width=64, code=32)	Encoder DeepSets: MLP por ponto em (posição, normal) e pooling média + máximo para um código de geometria.
Design	classe	(points, normals, query, params, target)	Uma simulação do espaço de projeto: geometria, parâmetros de operação, pontos de consulta e campo-alvo; ellipse e square criam exemplos.
LargePhysicsModel	classe	(dim, n_fields, n_params=0, radii=(0.1, 0.4, 1.2), n_freq=6, width=128, code=32, n_heads=…)	Modelo que combina codificação local, código de geometria e parâmetros num tronco com várias cabeças; predict dá média e dispersão e ood mede distância aos projetos de treino.
multiscale_neighbourhood	função	(query, points, normals, radii)	Para cada ponto de consulta e raio: deslocamento médio aos vizinhos, normal média e ocupação.
fit	função	(model, designs, *, epochs=2000, lr=0.002, freeze_geometry=False, seed=0) -> list[float]	Treina ou faz fine-tuning (com freeze_geometry=True) numa lista de projetos, cada cabeça num reamostra bootstrap.

pinneapple_neural.physicsnemo_bridge

Integração PhysicsNeMo <-> PINNeAPPle, promovida de examples/vs_physicsnemo/ e examples/pinneapple_and_physicsnemo/: adaptadores (UQ, gêmeo digital), laço de active learning + retreino e operadores de campo em grade.

pinneapple_neural/physicsnemo_bridge/__init__.py · 86 linhas · testes: 19

pinneapple_neural.physicsnemo_bridge.active_learning_bridge

Laço 'active learning seleciona -> retreina -> seleciona' (padrão 2 do README de exemplos), com a seleção delegada a `ResidualBasedAL` de `pinneapple_data`.

`pinneapple_neural/physicsnemo_bridge/active_learning_bridge.py` · 226 linhas · testes: 19

nome	tipo	assinatura	o que faz
`RetrainConfig`	classe	`(n_rounds, n_epochs_per_round, batch_size, lr, selection_mode, device, verbose)`	Configuração da parte de retreino do laço (rodadas, épocas por rodada, batch, taxa, modo de seleção).
`ALRoundResult`	classe	`(round_index, n_points_added, n_points_total, mean_train_loss, train_losses)`	Registro de uma rodada: pontos adicionados, total e perdas de treino.
`ALRetrainResult`	classe	`(rounds, x_collocation, final_loss)`	Resultado do laço completo: rodadas, pontos de colocação finais e perda final.
`active_learning_retrain_loop`	função	`(model, loss_fn, residual_fn, bounds, *, x_initial=None, al_config=None, retrain_config=N…)`	Alterna retreino do modelo nos pontos atuais e seleção de novos pontos por resíduo.

pinneapple_neural.physicsnemo_bridge.adapters

Adaptadores para usar um modelo treinado no PhysicsNeMo em PINNeAPPle: UQ por MC Dropout e `DigitalTwin`; o import de `physicsnemo` é opcional e feito só onde é preciso. [sem teste direto]

`pinneapple_neural/physicsnemo_bridge/adapters.py` · 254 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`PhysicsNemoModelAdapter`	classe	`(model, field_names) -> None`	Envolve um modelo para que `forward` devolva um único tensor (junta saídas em dicionário).
`is_physicsnemo_available`	função	`() -> bool`	Diz se `physicsnemo` (ou o nome antigo `modulus`) pode ser importado.
`require_physicsnemo_model`	função	`(model) -> Any`	Confere que o modelo é compatível com PhysicsNeMo; exige o pacote instalado.
`wrap_for_uq`	função	`(model, field_names=None, *, mc_dropout_config=None) -> MCDropoutWrapper`	Envolve um modelo treinado num `MCDropoutWrapper` para quantificar incerteza.
`build_digital_twin_for_model`	função	`(model, field_names, coord_names=None, *, config=None) -> DigitalTwin`	Envolve um modelo treinado num `DigitalTwin`.

pinneapple_neural.physicsnemo_bridge.field_operators

Constrói FNO e MeshGraphNet do próprio PhysicsNeMo numa grade n x n, treina sob um `ComputeBudget` e expõe como preditor NumPy `predict(a) -> u` para os checks de PINNeAPPle.

`pinneapple_neural/physicsnemo_bridge/field_operators.py` · 241 linhas · testes: 3

nome	tipo	assinatura	o que faz
`GridFieldPredictor`	classe	`(module, config, name)`	Modelo PhysicsNeMo treinado numa grade n x n exposto como preditor NumPy, com normalização, salvar e carregar.
`TrainingResult`	classe	`(run, epochs, final_loss, history)`	O que `train_grid_operator` gastou e alcançou (épocas, perda final, histórico).
`grid_graph`	função	`(n) -> tuple[np.ndarray, np.ndarray]`	Grafo de uma grade n x n centrada nas células: `edge_index` bidirecional e `edge_attr` (dx, dy, |d|).
`physicsnemo_fno`	função	`(n, *, latent_channels=32, num_fno_layers=4, num_fno_modes=12, decoder_layers=1, decoder_…)`	Cria um FNO 2D do PhysicsNeMo (coeficiente -> solução) como `GridFieldPredictor`.
`physicsnemo_meshgraphnet`	função	`(n, *, processor_size=15, hidden_dim=64, aggregation='sum', seed=0) -> GridFieldPredictor`	Cria um MeshGraphNet do PhysicsNeMo sobre o grafo da grade.
`train_grid_operator`	função	`(pred, a, u, budget, *, batch_size=8, lr=0.001, weight_decay=1e-05, seed=0, evaluate=None…)`	Treina o preditor em pares (a, u) até gastar o `ComputeBudget`, com perda L2 relativa por amostra.

pinneapple_neural.predictor

Inferência pós-treino (antigo `pinneapple_inference`): reexporta inferência em lote e em grade, gráficos de campos, erro, perda, comparação, fluxo 3D e otimização de projeto. [sem teste direto]

`pinneapple_neural/predictor/__init__.py` · 69 linhas · **sem teste direto**

pinneapple_neural.predictor.infer

Inferência em lote para evitar falta de memória, inferência com NumPy e avaliação em grades 1D+tempo e 2D, com o contêiner `InferenceResult`. [sem teste direto]

`pinneapple_neural/predictor/infer.py` · 190 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`InferenceResult`	classe	`(coords, fields, grid_shape, model_id, metadata)`	Contêiner da inferência: coordenadas, campos, forma da grade, id do modelo e metadados.
`batched_inference`	função	`(model, x, *, batch_size=4096, device=None, use_amp=False, amp_dtype=torch.float16) -> to…)`	Roda o modelo em lotes sobre um tensor grande, com AMP opcional.
`infer`	função	`(model, x, *, device='cpu', field_names=None) -> np.ndarray`	Passo direto com entrada NumPy e saída NumPy.
`infer_on_grid_1d`	função	`(model, x_range, t_range, *, nx=200, nt=200, device='cpu', field_names=None, coord_names=…)`	Avalia o modelo numa grade retangular x-t.
`infer_on_grid_2d`	função	`(model, x_range, y_range, *, nx=100, ny=100, device='cpu', field_names=None, coord_names=…)`	Avalia o modelo numa grade retangular x-y.

pinneapple_neural.predictor.postprocess

Visualização 3D de campos: linhas de corrente por RK4, isosuperfícies (marching cubes, exige `scikit-image`), fatias de volume e a classe `FlowVisualizer`. [sem teste direto]

`pinneapple_neural/predictor/postprocess.py` · 762 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`FlowVisualizer`	classe	`(model, coord_ranges, device='cpu') -> None`	Classe de alto nível com linhas de corrente, contornos de pressão, módulo de velocidade e vorticidade a partir de um modelo.
`compute_streamlines`	função	`(u_field, v_field, x_grid, y_grid, seed_points=None, n_seeds=20, max_length=1.0, dt=0.01)…)`	Linhas de corrente 2D por integração RK4.
`compute_isosurface`	função	`(field_3d, x, y, z, level=0.0) -> Tuple[np.ndarray, np.ndarray]`	Isosuperfície de um campo 3D por marching cubes (exige `scikit-image`).
`plot_streamlines_2d_from_model`	função	`(model, x_range=(0.0, 1.0), y_range=(0.0, 1.0), n_grid=50, n_seeds=30, figsize=(10, 6), t…)`	Desenha linhas de corrente do campo de velocidade previsto por um modelo.
`plot_isosurface_3d`	função	`(model, x_range=(0.0, 1.0), y_range=(0.0, 1.0), z_range=(0.0, 1.0), n_grid=30, level=0.0,…)`	Desenha a isosuperfície 3D de um campo escalar previsto.
`plot_volume_slice`	função	`(model, axis='z', slice_val=0.5, x_range=(0.0, 1.0), y_range=(0.0, 1.0), z_range=(0.0, 1.…)`	Desenha uma fatia 2D de um campo 3D previsto.

pinneapple_neural.predictor.visualize

Gráficos com matplotlib dos resultados de inferência: campos 1D/2D, mapas de erro, curvas de perda, comparação de modelos, fatias de velocidade e vorticidade, perfil de linha central, convergência e frente de Pareto. [sem teste direto]

`pinneapple_neural/predictor/visualize.py` · 1.103 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
plot_field_1d	função	(result, field_name, *, ax=None, title=None, cmap='RdBu_r', save_path=None, show=False) -…)	Mapa de cores x vs t de um campo 1D+tempo.
plot_field_2d	função	(result, field_name, *, ax=None, title=None, cmap='viridis', save_path=None, show=False) …)	Mapa de cores x vs y de um campo 2D.
plot_error_map_1d	função	(result, reference, field_name, *, ax=None, title=None, save_path=None, show=False) -> Any	Erro absoluto contra uma referência numa grade x-t.
plot_error_map_2d	função	(result, reference, field_name, *, ax=None, title=None, save_path=None, show=False) -> Any	Erro absoluto contra uma referência numa grade 2D.
plot_loss_curve	função	(history, *, keys=None, ax=None, title='Training loss', log_scale=True, save_path=None, s…)	Curvas de perda de treino e validação a partir do histórico.
plot_model_comparison_1d	função	(results, field_name, *, reference=None, title=None, cmap='RdBu_r', save_path=None, show=…)	Comparação lado a lado de vários modelos num campo 1D+tempo.
plot_model_comparison_2d	função	(results, field_name, *, reference=None, title=None, cmap='viridis', save_path=None, show…)	Comparação lado a lado de vários modelos num campo 2D.
render_visualizations	função	(viz_cfgs, *, model_results, problem_spec=None, reference_result=None, out_dir='.') -> Di…)	Gera os gráficos pedidos numa lista de configuração (YAML) e devolve os caminhos dos arquivos.
plot_velocity_slice	função	(u, x, y, z, *, slice_axis='z', slice_idx=None, component=0, title=None, cmap='RdBu_r', a…)	Fatia 2D de um componente da velocidade num campo 3D.
plot_velocity_magnitude_slice	função	(u, x, y, z, *, slice_axis='z', slice_idx=None, title=None, cmap='viridis', ax=None, save…)	Fatia 2D do módulo da velocidade.
plot_streamlines_2d	função	(u, v, x, y, *, density=1.5, title=None, background_field=None, cmap='viridis', ax=None, …)	Linhas de corrente de um campo de velocidade 2D com fundo opcional.
plot_centerline_velocity	função	(u_centerline, coord, *, reference_coord=None, reference_u=None, reference_label='Ghia et…)	Perfil de velocidade na linha central com referência da literatura opcional.
plot_vorticity_slice	função	(u, v, x, y, z, *, slice_axis='z', slice_idx=None, title=None, cmap='RdBu_r', ax=None, sa…)	Vorticidade ω_z = dv/dx - du/dy numa fatia 2D.
plot_internal_flow_summary	função	(solver_output, *, time_step=-1, slice_axis='z', title='Internal Flow Summary', save_path…)	Figura de vários painéis para escoamento interno 3D (cavidade, canal).
plot_design_opt_convergence	função	(history_objectives, *, title=…, xlabel='Iteration', ylabel='Objective', log_scale=False,…)	Curva de convergência do objetivo ao longo das iterações de otimização.
plot_pareto_front_2d	função	(objectives, pareto_mask, *, xlabel='Objective 1', ylabel='Objective 2', title='Pareto Fr…)	Dispersão com os pontos Pareto-ótimos destacados.

pinneapple_neural.trainer

Fachada do subpacote: reexporta treinadores (`Trainer`, `TwoPhaseTrainer`, `TimeMarchingTrainer`, `CausalPINNTrainer`, `DDPPINNTrainer`), balanceadores de perda, otimizadores e utilitários de HPC num só lugar.

`pinneapple_neural/trainer/__init__.py` · 191 linhas · testes: 1

pinneapple_neural.trainer.adaptive_sweep

Busca adaptativa de hiperparâmetros: usa TPE do Optuna (com poda opcional) quando o pacote está instalado e cai para busca aleatória sem ele. [sem teste]

`pinneapple_neural/trainer/adaptive_sweep.py` · 178 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
AdaptiveSweepConfig	classe	(param_space, n_trials, direction, seed, pruning)	Configuração da busca adaptativa: espaço de parâmetros, número de tentativas, direção (minimizar/maximizar), semente e poda.
run_adaptive_sweep	função	(trial_fn, cfg) -> List[Dict[str, Any]]	Executa a busca adaptativa (TPE do Optuna ou busca aleatória como fallback) e devolve a lista de resultados por tentativa.

pinneapple_neural.trainer.advisor

Diagnóstico de treino: lê históricos de perda e métricas finais, detecta sinais (platô, divergência, oscilação, desequilíbrio BC/física etc.) e devolve sugestões ordenadas com patches de configuração. [sem teste]

pinneapple_neural/trainer/advisor.py · 378 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
Suggestion	classe	(code, priority, description, config_patch, rationale)	Uma recomendação acionável: código, prioridade, descrição, patch de configuração e justificativa.
DiagnosticReport	classe	(model_name, signals, suggestions, metrics_summary, diagnosis_text)	Saída completa do diagnóstico de uma execução: sinais detectados, sugestões, resumo de métricas e texto de diagnóstico.
TrainingAdvisor	classe	(plateau_rel_std_thresh=0.005, divergence_rel_thresh=-0.05, slow_start_thresh=0.05, high_…)	Analisa histórico de perda e métricas finais com limiares configuráveis e produz sugestões ordenadas; apply_suggestions aplica os patches a uma configuração.

pinneapple_neural.trainer.audit

Auditabilidade: semente global, modo determinístico, impressão digital do ambiente e um logger JSONL por execução. [sem teste]

pinneapple_neural/trainer/audit.py · 76 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
RunLogger	classe	(out_dir, run_name, flush_every)	Logger JSONL por execução que também salva a configuração e artefatos no diretório da execução.
set_seed	função	(seed) -> None	Fixa a semente de random, NumPy e PyTorch (incluindo CUDA).
set_deterministic	função	(deterministic=True) -> None	Liga ou desliga o modo determinístico do PyTorch/cuDNN.
env_fingerprint	função	() -> Dict[str, Any]	Retorna um dict com versões de Python, plataforma, PyTorch e disponibilidade/versão de CUDA para registrar o ambiente.

pinneapple_neural.trainer.callbacks

Callbacks simples de treino: parada antecipada (EarlyStopping) e salvamento do melhor modelo (ModelCheckpoint). [sem teste]

pinneapple_neural/trainer/callbacks.py · 67 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
EarlyStopping	classe	(monitor, patience, min_delta, mode, best, bad_epochs, stop)	Para o treino quando a métrica monitorada não melhora por patience épocas, com min_delta e modo min/max.
ModelCheckpoint	classe	(path, monitor, mode, best)	Salva o state_dict do modelo (com extras e melhor valor) no caminho dado sempre que a métrica monitorada melhora.

pinneapple_neural.trainer.causal

Treino causal para EDPs temporais (Wang et al. 2022): pesos exponencialmente decrescentes por fatia de tempo, conforme o resíduo acumulado nas fatias anteriores. [sem teste]

pinneapple_neural/trainer/causal.py · 408 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
CausalWeightScheduler	classe	(n_time_chunks=100, epsilon=1.0, update_every=100)	Calcula pesos causais por fatia de tempo, w_k = exp(-epsilon * soma das perdas das fatias anteriores), e a perda ponderada e uma métrica de violação de causalidade.
CausalPINNTrainer	classe	(model, epsilon=1.0, n_time_chunks=50, update_every=100)	Treinador PINN que aplica os pesos causais ao resíduo da EDP junto com perdas de condição inicial/contorno; permite consultar e plotar os pesos.

pinneapple_neural.trainer.checkpoint

Salva e carrega um checkpoint com estado do modelo, do otimizador, configuração, metadados e normalizadores via `torch.save`.

`pinneapple_neural/trainer/checkpoint.py` · 40 linhas · testes: 3

nome	tipo	assinatura	o que faz
`Checkpoint`	classe	`(model_state, optim_state, cfg, meta, normalizers)`	Dataclass com estado do modelo, estado do otimizador, configuração, metadados e normalizadores.
`save_checkpoint`	função	`(path, ckpt) -> None`	Grava um `Checkpoint` em disco com `torch.save`, criando o diretório se preciso.
`load_checkpoint`	função	`(path) -> Checkpoint`	Lê um arquivo salvo e reconstrói o `Checkpoint`.

pinneapple_neural.trainer.collocation

Amostragem adaptativa de pontos de colocação (RAR): um conjunto ativo é reamostrado de um pool grande com densidade puxada para onde o resíduo da EDP é alto; inclui utilitários de escala por canal e filtragem de NaN. [sem teste]

`pinneapple_neural/trainer/collocation.py` · 413 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`AdaptiveCollocationConfig`	classe	`(n_active, n_pool, resample_every, residual_frac, residual_batch, replace)`	Configuração do RAR: pontos ativos, tamanho do pool, frequência de reamostragem, fração guiada pelo resíduo e tamanho do lote de avaliação.
`AdaptiveCollocationSampler`	classe	`(pool, config=None, residual_fn=None) -> None`	Mantém um pool de candidatos e um subconjunto ativo, reamostrado periodicamente com probabilidade proporcional ao resíduo ao quadrado misturada com uniforme.
`estimate_channel_scales`	função	`(Y, quantile=0.95, min_scale=1e-06, channel_names=None) -> List[float]`	Estima a escala de cada canal de saída como o quantil q de |Y|, para normalizar canais de magnitudes muito diferentes.
`filter_nan_rows`	função	`(*tensors, warn=True) -> Tuple[torch.Tensor, ...]`	Remove, de vários tensores ao mesmo tempo, as linhas onde algum tem NaN ou Inf.

pinneapple_neural.trainer.datamodule

Monta DataLoaders de treino/validação/teste com split consistente (`pinneapple_data.splits`), adaptadores de item e pipeline de pré-processamento. [sem teste]

`pinneapple_neural/trainer/datamodule.py` · 200 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ItemAdapter`	classe	`()`	Interface que converte itens do dataset em dicts de lote com `x` e, opcionalmente, `y`, `meta` e `physics`.
`FnAdapter`	classe	`(x_fn, y_fn, meta_fn=None)`	Adaptador por funções: `x_fn`, `y_fn` e `meta_fn` opcional aplicados a cada amostra (por exemplo `PhysicalSample`).
`AdaptedSequenceDataset`	classe	`(samples, adapter)`	Dataset que embrulha uma sequência Python e aplica o adaptador em `__getitem__`.
`DataModule`	classe	`(dataset, split, group_ids, batch_size, num_workers, pin_memory, persistent_workers, drop…)`	Constrói e guarda em cache os DataLoaders de treino/validação/teste com split e pré-processamento consistentes.

pinneapple_neural.trainer.distributed

Treino multi-GPU/multinó com DistributedDataParallel do PyTorch, lançado por `torchrun` ou por `mp.spawn`. [sem teste]

`pinneapple_neural/trainer/distributed.py` · 323 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`DDPTrainerConfig`	classe	`(backend, world_size, master_addr, master_port, gradient_sync_every, find_unused_paramete…)`	Configuração do treino DDP: backend, número de processos, endereço/porta do mestre, clip de gradiente, AMP e frequência de log.
`DDPPINNTrainer`	classe	`(model, config=None)`	Treinador PINN multi-GPU com DDP: inicializa o grupo de processos, sincroniza gradientes e pode lançar workers via `mp.spawn`.
`is_distributed`	função	`() -> bool`	Diz se há um grupo de processos distribuído ativo.
`get_rank`	função	`() -> int`	Retorna o rank do processo atual (0 fora de modo distribuído).
`get_world_size`	função	`() -> int`	Retorna o número total de processos (1 fora de modo distribuído).

pinneapple_neural.trainer.graybox

Treino PINN gray-box (estilo AI-Aristotle): rede principal aprende o estado e redes auxiliares pequenas aprendem um termo desconhecido da equação; opcionalmente destila esse termo em fórmula com PySR. [sem teste]

`pinneapple_neural/trainer/graybox.py` · 280 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`GrayBoxNet`	classe	`(n_in=1, n_out=1, width=32, depth=3)`	MLP pequena e rasa que aproxima um termo funcional desconhecido de uma EDP/EDO, mantida simples para facilitar a destilação simbólica.
`GrayBoxConfig`	classe	`(epochs_supervised, epochs_joint, lr_main, lr_graybox, lr_params, w_data, w_pde, pde_warm…)`	Configuração do treino gray-box: épocas das duas fases, taxas de aprendizado, pesos de dados e EDP, aquecimento da EDP, clip e EMA.
`train_graybox`	função	`(main_model, graybox_nets, known_residual_fn, obs_coords, obs_values, collocation_sampler…)`	Treino em duas fases: ajusta a rede principal só aos dados e depois treina rede principal, redes gray-box e parâmetros desconhecidos contra dados + resíduo da EDP.
`distill_to_symbolic`	função	`(graybox_net, coord_bounds, n_samples=2000, var_names=None, n_iterations=40) -> Dict[str,…)`	Amostra a rede gray-box no domínio e roda regressão simbólica PySR para obter uma expressão fechada; exige o pacote opcional `pysr`.

pinneapple_neural.trainer.heteroscedastic

Quantificação de incerteza heteroscedástica: a rede prevê média e log-variância por ponto (perda NLL gaussiana) e, com MC-dropout, separa incerteza aleatória e epistêmica. [sem teste]

`pinneapple_neural/trainer/heteroscedastic.py` · 136 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`HeteroscedasticWrapper`	classe	`(base_network, n_out_physical)`	Envolve uma rede com saída 2*n para expor `forward_uq` (média e log-variância); `forward` devolve só a média.
`init_logvar_bias`	função	`(output_layer, n_out_physical, bias_value=-4.6) -> None`	Inicializa o viés das colunas de log-variância da última camada para começar com incerteza pequena.
`heteroscedastic_nll_loss`	função	`(pred_mean, pred_logvar, target) -> torch.Tensor`	Perda NLL gaussiana com log-variância limitada a [-10, 10].
`compute_aleatoric_epistemic_uq`	função	`(model, coords, n_mc=50, forward_kwargs=None) -> Dict[str, torch.Tensor]`	Decompõe a incerteza preditiva em aleatória e epistêmica com amostras MC-dropout, pela lei da variância total.

pinneapple_neural.trainer.hpc

Utilitários de HPC: FSDP, ZeRO-1, CUDA graphs, ganchos de compressão de gradiente (PowerSGD, Top-K), geração de comandos `torchrun` e scripts SLURM, profiler e busca de batch máximo. [sem teste]

`pinneapple_neural/trainer/hpc.py` · 819 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`FSDPConfig`	classe	`(sharding_strategy, mixed_precision, min_num_params, activation_checkpointing, cpu_offloa…)`	Configuração do FSDP: estratégia de sharding, precisão mista, tamanho mínimo de módulo, checkpointing de ativações e offload para CPU.
`CUDAGraphModule`	classe	`(module, warmup_steps=3, device='cuda') -> None`	Envolve um módulo para capturar e reexecutar um CUDA graph na inferência com formato de entrada fixo.
`TorchRunConfig`	classe	`(n_nodes, n_procs_per_node, master_addr, master_port, rdzv_backend, max_restarts)`	Configuração de um lançamento `torchrun` (nós, processos por nó, endereço, porta, rendezvous, reinícios).
`SLURMConfig`	classe	`(job_name, n_nodes, n_gpus_per_node, n_cpus_per_task, time_limit, partition, account, mem…)`	Configuração de um script batch SLURM (nome, nós, GPUs, CPUs, tempo, partição, conta, memória, módulos, ambiente conda).
`ProfilerConfig`	classe	`(trace_dir, record_shapes, profile_memory, with_flops, with_stack, schedule_wait, schedul…)`	Configuração do `PINNeAPPleProfiler`: diretório de traces, o que registrar e agenda de profiling.
`PINNeAPPleProfiler`	classe	`(config=None) -> None`	Context manager sobre `torch.profiler.profile` que registra tempo, memória e FLOPs por operador e exporta trace Chrome.
`AutoBatchSizeFinder`	classe	`(trial_fn, lo=1, hi=8192, verbose=True) -> None`	Busca binária do maior tamanho de lote que cabe na memória da GPU, usando uma função de teste fornecida.
`wrap_fsdp`	função	`(model, cfg=None, *, process_group=None) -> nn.Module`	Envolve um modelo com Fully Sharded Data Parallel; devolve o modelo sem mudança se FSDP não estiver disponível.
`wrap_zero_optimizer`	função	`(optimizer_class, params, **optimizer_kwargs) -> torch.optim.Optimizer`	Cria um otimizador com `ZeroRedundancyOptimizer` (ZeRO-1), que divide o estado do otimizador entre ranks.
`register_powersgd_hook`	função	`(model, *, matrix_approximation_rank=1, start_powersgd_iter=10, use_error_feedback=True) …)`	Registra no modelo DDP o gancho de compressão de gradiente PowerSGD (baixo posto).
`register_topk_hook`	função	`(model, *, compression_ratio=0.1) -> None`	Registra no modelo DDP um gancho que envia só a fração Top-K de gradientes de maior módulo.
`build_torchrun_cmd`	função	`(script, cfg, *, script_args='') -> str`	Gera a string de comando `torchrun` para um script de treino.
`build_slurm_script`	função	`(torchrun_cmd, slurm, *, output_dir='slurm_logs') -> str`	Gera um script batch SLURM completo que executa um comando `torchrun`.

`pinneapple_neural.trainer.inference`

Inferência mínima: carrega pesos de um checkpoint num modelo e faz predição em modo `eval` sem gradiente. [sem teste]

`pinneapple_neural/trainer/inference.py` · 22 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`load_checkpoint`	função	`(model, path, map_location='cpu') -> Dict[str, Any]`	Carrega os pesos da chave `model` de um arquivo `torch.save` no modelo e devolve o dict lido.
`predict`	função	`(model, x, device=None) -> torch.Tensor`	Roda o modelo em modo `eval`, sem gradiente, no dispositivo indicado.

`pinneapple_neural.trainer.inverse_pinn`

PINN inverso: otimiza ao mesmo tempo os pesos da rede e parâmetros físicos escalares desconhecidos, a partir de sensores esparsos e de um resíduo de EDP/EDO fornecido pelo usuário.

`pinneapple_neural/trainer/inverse_pinn.py` · 218 linhas · testes: 6

nome	tipo	assinatura	o que faz
`InverseParam`	classe	`(initial_value, log_space=False, name='param')`	Escalar treinável, opcionalmente em espaço log para manter o parâmetro positivo.
`InversePINN`	classe	`(model, inverse_params, residual_fn, sensor_points, sensor_values, domain_bounds, n_col=2…)`	Treina pesos da rede e parâmetros físicos desconhecidos contra resíduo nos pontos de colocação mais erro nos sensores, com parada antecipada.

pinneapple_neural.trainer.lbfgs_finetune

Ajuste fino com L-BFGS em várias rodadas independentes, cada uma com um closure novo (novo mini-lote), em vez de uma fase longa única.

pinneapple_neural/trainer/lbfgs_finetune.py · 112 linhas · testes: 6

nome	tipo	assinatura	o que faz
LBFGSFinetuneResult	classe	(round_losses, round_iters)	Registro por rodada do L-BFGS multi-rodada (perdas e iterações), com a perda final.
multi_round_lbfgs	função	(params, closure_builder, n_rounds, n_steps_per_round, lr=1.0, history_size=50, line_sear…)	Executa N fases L-BFGS independentes, cada uma com um closure recém-construído, com callbacks opcionais ao fim de cada rodada.

pinneapple_neural.trainer.loss_balancer

Balanceamento automático de pesos de perda para PINNs: ReLoBRaLo, SoftAdapt, Lagrangiano aumentado, Inverse Dirichlet, LR annealing, PCGrad, pesos conjuntos via softplus e um meta-agendador `AutoBalancer` que troca de método quando o treino estagna.

pinneapple_neural/trainer/loss_balancer.py · 1.229 linhas · testes: 8

nome	tipo	assinatura	o que faz
ReLoBRaLo	classe	(loss_names, *, tau=0.1, alpha=0.999, update_every=1, clip_min=0.001, clip_max=1000.0, in…)	Balanceamento ReLoBRaLo: pesos por softmax das razões de perda relativas, com memória aleatória (lookback) e suavização.
SoftAdapt	classe	(loss_names, *, tau=0.1, beta=0.0, normalized=True, update_every=1, clip_min=0.001, clip_…)	SoftAdapt: pesos por softmax das taxas de variação de cada perda, opcionalmente normalizados.
AugmentedLagrangian	classe	(loss_names, *, pde_key='pde', rho=0.1, lambda_init=1.0, lambda_min=0.0, lambda_max=10000…)	Trata a perda da EDP como restrição e atualiza multiplicadores de Lagrange com penalidade `rho`, após um aquecimento.
InverseDirichlet	classe	(model, loss_names, *, shared_layer=None, update_every=50, alpha=0.9, clip_min=0.001, cli…)	Pesos a partir das estatísticas (variância) dos gradientes de cada termo numa camada compartilhada.
LRAnnealing	classe	(model, loss_names, *, reference='pde', update_every=100, alpha=0.1, clip_min=0.001, clip…)	Algoritmo 1 de Wang, Teng & Perdikaris (2021): ajusta pesos pela razão entre o gradiente máximo do termo de referência e o gradiente médio de cada termo.
PCGrad	classe	(model, loss_names, *, update_every=1, weights=None) -> None	PCGrad: projeta gradientes conflitantes entre termos de perda antes de combiná-los.
AutoBalancerConfig	classe	(method, check_every, health_window, plateau_rel_thresh, divergence_thresh, max_switches,…)	Configuração do meta-agendador: método inicial, janela de saúde, limiares de platô e divergência, máximo de trocas e método de fallback.
AutoBalancer	classe	(model, loss_names, *, config=None, pde_key='pde') -> None	Meta-agendador que monitora a saúde do treino e troca de método de balanceamento quando há platô ou divergência.
JointAdaptiveWeights	classe	(loss_names, *, initial_weights=None, sp_beta=1.0, max_raw_init=20.0, scale=None) -> None	Pesos de perda treináveis junto com o modelo, parametrizados por softplus.
AutoPDEWeightCalibrator	classe	(target_ratio=0.01, *, once=True, clip_min=1e-20, clip_max=1000000.0) -> None	Calibra o peso da perda da EDP para atingir uma razão-alvo em relação às perdas de contorno/dados.

pinneapple_neural.trainer.losses

Perdas supervisionadas e com física: perda supervisionada (MSE/MAE), gancho de perda física e combinação ponderada, com helper para a Arena. [sem teste]

pinneapple_neural/trainer/losses.py · 148 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
SupervisedLoss	classe	(kind)	Perda supervisionada entre predição e y: MAE quando kind é mae, MSE nos demais casos.
PhysicsLossHook	classe	(physics_loss_fn)	Gancho que chama uma função de perda física do usuário com modelo, predição e lote.
CombinedLoss	classe	(supervised, physics, w_supervised, w_physics)	Soma ponderada da perda supervisionada e da física, devolvendo dict com total e componentes.
build_loss	função	(*, problem_spec, model_capabilities=None, weights=None, supervised_kind='mse', physics_l…)	Monta um CombinedLoss para a Arena, ligando a física só quando o problema e o modelo a suportam.

pinneapple_neural.trainer.metrics

Métricas de avaliação (MSE, MAE, RMSE, R2, L2 relativo, erro máximo) e agregadores; as classes base Metric e Metrics levantam NotImplementedError por serem abstratas. [sem teste]

pinneapple_neural/trainer/metrics.py · 109 linhas · **sem teste direto** · palavras-chave no código (heurística; o texto acima diz o status real): levanta NotImplementedError em algum caminho

nome	tipo	assinatura	o que faz
Metric	classe	(name)	Classe base de métrica com name; __call__ levanta NotImplementedError.
MSE	classe	(name)	Erro quadrático médio.
MAE	classe	(name)	Erro absoluto médio.
RMSE	classe	(name)	Raiz do erro quadrático médio.
R2	classe	(name, eps)	Coeficiente de determinação R2, com eps no denominador.
RelL2	classe	(name, eps)	Erro L2 relativo, ||y_hat - y|| / ||y||, com eps.
MaxError	classe	(name)	Maior erro absoluto.
Metrics	classe	()	Interface base de conjunto de métricas; compute levanta NotImplementedError.
MetricBundle	classe	(metrics)	Aplica uma lista de Metric e devolve um dict nome -> valor.
RegressionMetrics	classe	(eps)	Conjunto fixo de métricas de regressão (MSE, MAE, RMSE, R2) num único compute.
default_metrics	função	() -> List[Metric]	Retorna a lista padrão de métricas.
regression_metrics_bundle	função	() -> Metrics	Retorna um objeto Metrics padrão para regressão.

pinneapple_neural.trainer.metrics_cfg

Constrói um MetricBundle a partir de uma lista de nomes ou dicts vinda de configuração YAML. [sem teste]

pinneapple_neural/trainer/metrics_cfg.py · 81 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
build_metrics_from_cfg	função	(cfg) -> Metrics	Converte nomes como mse, rmse, rel_l2, r2, mae, max_error (ou dicts com name) num MetricBundle.

pinneapple_neural.trainer.monotonic

Restrições de monotonicidade para surrogates de regressão: penalidade hinge sobre derivadas entrada-saída e medida da fração de violações.

pinneapple_neural/trainer/monotonic.py · 55 linhas · testes: 2

nome	tipo	assinatura	o que faz
monotonicity_penalty	função	(model, z, cols, signs, *, scale=None, margin=0.0, output=0, create_graph=True) -> torch.…)	Penalidade hinge quadrática média que é zero quando cada derivada amostrada tem o sinal exigido com folga margin.
monotone_violation_share	função	(model, z, cols, signs, output=0) -> list[float]	Fração de linhas onde a derivada da saída em cada coluna restrita tem o sinal errado.

pinneapple_neural.trainer.multiadam

Otimizador MultiAdam (Yao et al., ICML 2023): Adam com momentos separados por termo de perda, para que um termo muito grande não domine o passo.

pinneapple_neural/trainer/multiadam.py · 103 linhas · testes: 4

nome	tipo	assinatura	o que faz
MultiAdam	classe	(params, lr=0.001, betas=(0.99, 0.99), eps=1e-08)	Otimizador Adam com um par de momentos (m, v) por termo de perda; step recebe a lista de perdas e calcula os gradientes.

pinneapple_neural.trainer.multistart

Treino com múltiplos reinícios (sementes diferentes), vigia de NaN e escolha do melhor modelo por uma função de score. [sem teste]

pinneapple_neural/trainer/multistart.py · 363 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
MultiRestartConfig	classe	(n_restarts, seed_base, nan_patience, max_failed, verbose, keep_all)	Configuração dos reinícios: número, semente base, paciência de NaN, máximo de falhas, verbosidade e se guarda todos os resultados.
RestartResult	classe	(model, history, score, restart_idx, seed, nan_detected)	Resultado de um reinício: modelo, histórico, score, índice, semente e se houve NaN.
MultiRestartResult	classe	(best, all_results, n_valid, n_nan)	Resultado agregado: melhor reinício, todos os resultados (opcional) e contagens de válidos e com NaN.
MultiRestartTrainer	classe	(model_factory, train_fn, *, config=None, score_fn=None, on_restart_end=None) -> None	Roda uma função de treino várias vezes com sementes diferentes, descarta execuções com NaN e devolve a de menor score.

pinneapple_neural.trainer.noether_trainer

Treinador para modelos Noether (UPT, AB-UPT, Transolver) com dados do PINNeAPPle, num loop PyTorch próprio ou delegando ao HydraRunner do Noether (dependência opcional).

pinneapple_neural/trainer/noether_trainer.py · 344 linhas · testes: 5

nome	tipo	assinatura	o que faz
NoetherTrainConfig	classe	(epochs, batch_size, lr, weight_decay, field_weights, loss_fn, warmup_steps, grad_clip, d…)	Configuração do treino Noether: épocas, lote, taxa de aprendizado, decaimento, pesos por campo, aquecimento, clip, rastreador e checkpoints.
NoetherTrainResult	classe	(train_losses, val_losses, best_val_loss, best_epoch, checkpoint_path, extras)	Resumo do treino Noether: perdas de treino/validação, melhor perda e época, caminho do checkpoint e extras.
NoetherSurrogateTrainer	classe	(model, train_loader, val_loader=None, config=None) -> None	Treina um modelo Noether (UPT, AB-UPT, Transolver) com DataLoaders do PINNeAPPle; from_hydra_config delega tudo ao HydraRunner do Noether.

pinneapple_neural.trainer.normalizers

[shim] Reexporta Normalizer, StandardScaler e MinMaxScaler de pinneapple_data.transforms. [sem teste]

pinneapple_neural/trainer/normalizers.py · 5 linhas · **sem teste direto**

pinneapple_neural.trainer.parallel

Paralelismo e otimização de GPU: escolha de dispositivo, AMP, prefetch CUDA, acumulação de gradiente, torch.compile, DataParallel, sweep em grade paralelo, checkpointing de gradiente e monitor de throughput. [sem teste]

pinneapple_neural/trainer/parallel.py · 508 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`AMPContext`	classe	`(device='cuda', dtype=torch.bfloat16, enabled=True) -> None`	Context manager de precisão mista (`torch.amp`), que vira no-op fora de CUDA.
`CUDAPrefetcher`	classe	`(loader, device='cuda') -> None`	Pré-carrega o próximo lote para a GPU num stream CUDA separado, sobrepondo cópia e computação.
`GradAccumConfig`	classe	`(accumulate_steps, clip_grad_norm)`	Configuração da acumulação de gradiente: número de passos e clip de norma.
`GradAccumTrainer`	classe	`(model, optimizer, cfg=None, amp_ctx=None) -> None`	Acumula gradientes de micro-lotes e dá o passo do otimizador a cada N micro-lotes.
`SweepConfig`	classe	`(param_grid, n_jobs, backend, timeout_per_trial)`	Configuração do sweep em grade: grade de parâmetros, número de jobs, backend (threads/processos) e timeout.
`ThroughputMonitor`	classe	`(device_index=0) -> None`	Mede amostras/s, memória da GPU e utilização (com `pynvml`, se houver) por época.
`best_device`	função	`(prefer_cuda=True, allow_mps=True) -> torch.device`	Retorna o melhor dispositivo disponível: CUDA, depois MPS (se permitido), depois CPU.
`count_gpus`	função	`() -> int`	Retorna o número de GPUs CUDA disponíveis.
`gpu_info`	função	`() -> List[Dict[str, Any]]`	Retorna um dict de informações por GPU disponível.
`maybe_compile`	função	`(model, *, mode='default', fullgraph=False, dynamic=False) -> nn.Module`	Aplica `torch.compile` se o PyTorch suportar; caso contrário devolve o modelo original.
`wrap_data_parallel`	função	`(model, device_ids=None) -> nn.Module`	Envolve o modelo em `DataParallel` quando há mais de uma GPU.
`unwrap_model`	função	`(model) -> nn.Module`	Remove o invólucro DataParallel/DDP e devolve o modelo cru.
`run_parallel_sweep`	função	`(trial_fn, cfg) -> List[Dict[str, Any]]`	Executa um sweep em grade em paralelo (threads ou processos) e devolve a lista de resultados.
`enable_gradient_checkpointing`	função	`(model) -> nn.Module`	Liga checkpointing de gradiente em modelos que o suportam (estilo HuggingFace) ou envolvendo camadas.

`pinneapple_neural.trainer.preprocess`

Pipeline de pré-processamento de lotes com passos ajustados no fold de treino (normalização, preenchimento de NaN, winsorização, escala robusta, log, diferenciação, features de solver). [sem teste]

`pinneapple_neural/trainer/preprocess.py` · 445 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`PreprocessStep`	classe	`()`	Protocolo de passo de pré-processamento com `fit` e `apply`.
`PreprocessPipeline`	classe	`(steps)`	Sequência de passos ajustados nos lotes de treino e aplicados a cada lote, com `state_dict` para salvar.
`NormalizeStep`	classe	`(key, dim, store_key, enabled, scaler)`	Ajusta um `StandardScaler` numa chave do lote no treino e normaliza em `apply`.
`SolverFeatureStep`	classe	`(solver, mode, select_var_dim, reduce_fft_to, fit_noop)`	Calcula features com um solver (por exemplo FFT) a partir de `x` e as anexa ou substitui no lote.
`MissingValueStep`	classe	`(key, strategy, enabled)`	Preenche NaN em `x` por forward fill, backward fill ou zero.
`WinsorizeStep`	classe	`(key, q_low, q_high, enabled, low_, high_)`	Corta valores por quantis ajustados no fold de treino (winsorização).
`RobustScaleStep`	classe	`(key, eps, enabled, median_, iqr_)`	Escala robusta (x - mediana) / (IQR + eps), ajustada no fold de treino.
`LogStep`	classe	`(key, enabled, clamp_min)`	Transformação log opcional para sinais não negativos.
`DifferencingStep`	classe	`(key, time_dim, order, enabled, anchor_key)`	Diferenciação ao longo do tempo para estacionariedade; reduz o comprimento da sequência em `order`.

pinneapple_neural.trainer.self_scaled_qn

Otimizadores quase-Newton auto-escalados (BFGS, SSBFGS, SSBroyden) com Hessiana inversa densa e busca de linha strong-Wolfe, conforme Urban et al. (JCP 2025).

pinneapple_neural/trainer/self_scaled_qn.py · 186 linhas · testes: 4

nome	tipo	assinatura	o que faz
SelfScaledQuasiNewton	classe	(params, variant='ssbroyden', max_iter=20, tolerance_grad=1e-12, tolerance_change=1e-14, …)	Otimizador BFGS/SSBFGS/SSBroyden com Hessiana inversa densa e busca strong-Wolfe, usado como torch.optim.LBFGS com closure.
self_scaled_parameters	função	(s, y, g, alpha, Hy, n, variant)	Calcula (tau, phi) de uma atualização auto-escalada, conforme as equações 11-23 do artigo.

pinneapple_neural.trainer.splits

[shim] Reexporta SplitSpec e split_indices de pinneapple_data.splits. [sem teste]

pinneapple_neural/trainer/splits.py · 5 linhas · **sem teste direto**

pinneapple_neural.trainer.time_marching

Time-marching sequencial (Wight & Zhao 2020): divide o tempo em janelas e treina uma rede por janela, usando a solução final de uma como condição inicial da próxima. [sem teste]

pinneapple_neural/trainer/time_marching.py · 388 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
TimeMarchingTrainer	classe	(model_factory, t_start, t_end, n_windows=10, epochs_per_window=2000, n_col=1000, ic_weig…)	Treina uma rede por janela de tempo, congelando a predição no fim de cada janela como condição inicial da seguinte; avalia e plota o resultado completo.

pinneapple_neural.trainer.trainer

Trainer unificado para PINNs e modelos supervisionados: perdas que retornam tensor ou dict com total, AMP, DDP opcional, callbacks, checkpoint e log de auditoria.

pinneapple_neural/trainer/trainer.py · 397 linhas · testes: 151

nome	tipo	assinatura	o que faz
TrainConfig	classe	(epochs, lr, weight_decay, grad_clip, amp, device, seed, deterministic, log_dir, run_name…)	Configuração do Trainer: épocas, taxa de aprendizado, clip, AMP, dispositivo, semente, determinismo, log, DDP e validação com física.
Trainer	classe	(model, loss_fn, metrics=None, preprocess=None, early_stopping=None, checkpoint=None)	Treinador genérico: fit(train_loader, val_loader, cfg) roda épocas com perdas tensor/dict, métricas, early stopping, checkpoint e log JSONL.

pinneapple_neural.trainer.two_phase

Treinador em duas fases: primeiro ajuste supervisionado a dados de solver/medição, depois imposição de resíduos da EDP e condições de contorno. [sem teste]

pinneapple_neural/trainer/two_phase.py · 414 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
TwoPhaseConfig	classe	(phase1_epochs, phase1_lr, phase2_epochs, phase2_lr, weight_decay, grad_clip, device, amp…)	Configuração das duas fases: épocas e taxas de cada fase, modo combinado e rampa do peso da física.
TwoPhaseHistory	classe	(phase1_epochs, phase1_loss, phase2_epochs, phase2_loss, phase2_pde_loss, phase2_bc_loss)	Histórico de perdas das duas fases (incluindo componentes EDP e contorno na fase 2).

nome	tipo	assinatura	o que faz
`TwoPhaseTrainer`	classe	`(model, supervised_loss_fn, physics_loss_fn, config=None)`	Executa a fase supervisionada e depois a fase de física, com otimizadores e históricos separados.
`UnnormModel`	classe	`(model, Y_mean, Y_std)`	Envolve um modelo treinado em alvos normalizados para devolver predições na escala física (y * std + mean).

pinneapple_neural.trainer.viz

[shim] `plot_history` mantido por compatibilidade: delega a `pinneapple_viz.pinn` quando disponível e cai para um gráfico matplotlib mínimo. [sem teste]

`pinneapple_neural/trainer/viz.py` · 57 linhas · **sem teste direto** · palavras-chave no código (heurística; o texto acima diz o status real): shim ou obsoleto

nome	tipo	assinatura	o que faz
`plot_history`	função	`(history, keys=('train_total', 'val_total'), *, log_scale=True, show=True)`	Plota o histórico de perdas em escala log, via `pinneapple_viz.pinn` quando disponível.

pinneapple_neural.trainer.weight_scheduler

Agendamento automático de pesos de perda (SA-PINN, GradNorm, razão de perdas, NTK) com interface única `WeightScheduler`, que também despacha para os métodos de `loss_balancer`.

`pinneapple_neural/trainer/weight_scheduler.py` · 799 linhas · testes: 11

nome	tipo	assinatura	o que faz
`WeightSchedulerConfig`	classe	`(method, initial_weights, update_every, lr, alpha, clip_min, clip_max, ema_decay)`	Configuração do agendamento de pesos: método (de `self_adaptive` até `auto`), pesos iniciais, frequência, taxas, limites e decaimento EMA.
`SelfAdaptiveWeights`	classe	`(loss_names, config) -> None`	Pesos de perda aprendíveis (SA-PINN) atualizados por subida de gradiente, crescendo nos termos mais difíceis.
`GradNormBalancer`	classe	`(model, loss_names, shared_layer=None, config=None) -> None`	GradNorm: ajusta pesos para manter as normas de gradiente de cada termo numa camada compartilhada proporcionais à média.
`LossRatioBalancer`	classe	`(loss_names, target_ratios=None, config=None) -> None`	Mantém as razões entre perdas (via EMA) próximas de valores-alvo em relação à perda da EDP.
`NTKWeightBalancer`	classe	`(model, loss_names, n_proj=50, config=None) -> None`	Pesos inversamente proporcionais ao traço do NTK de cada termo, estimado por Hutchinson.
`WeightScheduler`	classe	`(model, loss_names, config=None) -> None`	Interface única: `step(losses)` aplica os pesos atuais, atualiza conforme o método e expõe parâmetros aprendíveis quando houver.

pinneapple_neural.utils

Helpers comuns: `as_tensor` extrai o tensor de previsão de qualquer saída de modelo e `fit_supervised` é um laço Adam mínimo em pares (x, y).

`pinneapple_neural/utils.py` · 92 linhas · testes: 5

nome	tipo	assinatura	o que faz
`as_tensor`	função	`(out, *, attrs=OUTPUT_TENSOR_ATTRS) -> torch.Tensor`	Extrai o tensor de previsão de um Tensor, dicionário de campos, tupla ou objeto de saída (`PINNOutput`, `ModelOutput` etc.).
`fit_supervised`	função	`(model, x, y, *, steps=1000, lr=0.001, batch=32, loss='mse', weight_decay=0.0, seed=0, fo…)`	Treina o modelo para mapear x em y com Adam em mini-batches aleatórios (perda MSE ou L2 relativa) e devolve o histórico.

pinneapple_neural.workflows

Workflows prontos de treino e previsão sobre as arquiteturas: operador de volume denso (simples e paramétrico) e surrogate híbrido com pós-modelo físico. [sem teste direto do `__init__`]

pinneapple_neural/workflows/__init__.py · 56 linhas · **sem teste direto**

pinneapple_neural.workflows.dense_volume_operator

Workflow de treino/previsão de um FNO3d em série temporal real de volumes (T, C, D, H, W), com o truque de pushforward para estabilizar o rollout autorregressivo.

pinneapple_neural/workflows/dense_volume_operator.py · 248 linhas · testes: 3

nome	tipo	assinatura	o que faz
DenseVolumeOperatorConfig	classe	(in_channels, out_channels, width, modes, layers, epochs, lr, val_frac, batch_size, rollo…)	Configuração do treino do FNO3d em volumes densos (largura, modos, camadas, épocas, validação, rollout, pushforward).
train_dense_volume_operator	função	(frames, cfg, *, seed=0) -> Tuple[Checkpoint, List[Dict[str, Any]]]	Treina um FNO3d numa série de volumes e devolve o checkpoint e o histórico.
load_dense_volume_operator	função	(checkpoint, *, device='cpu') -> nn.Module	Reconstrói o FNO3d a partir do checkpoint (também dos checkpoints antigos de mesmo formato).
predict_single_step	função	(checkpoint, input_frame, *, device='cpu') -> np.ndarray	Um passo de previsão em unidades reais, o regime em que a rede é mais precisa.
rollout_dense_volume_operator	função	(checkpoint, initial_frame, n_steps, *, device='cpu') -> np.ndarray	Rollout autorregressivo a partir de um quadro real; devolve n_steps + 1 quadros.

pinneapple_neural.workflows.hybrid_surrogate_physics

Modelo híbrido: um surrogate aprende só as grandezas intermediárias e um pós-modelo físico explícito (nunca treinado, trocável sem retreino) calcula a grandeza derivada; a incerteza passa por Monte Carlo. Só NumPy (scikit-learn opcional).

pinneapple_neural/workflows/hybrid_surrogate_physics.py · 517 linhas · testes: 10

nome	tipo	assinatura	o que faz
PhysicsPostModel	classe	(name, source)	Pós-modelo físico explícito, com nome e fonte científica, aplicado às grandezas intermediárias.
CallablePostModel	classe	(fn, name, source)	Envolve uma função simples como PhysicsPostModel.
SurrogatePrediction	classe	(mean, std, sampler)	Distribuição preditiva do surrogate: média, desvio e amostrador opcional.
Surrogate	classe	()	Interface mínima de surrogate: fit(X, Y) e predict(X).
GaussianProcessSurrogate	classe	(*, n_restarts=2, random_state=0, use_sklearn=None)	Um processo gaussiano por coluna de saída (scikit-learn se houver, senão um GP em NumPy).
PCASurrogate	classe	(base, *, n_components=None, variance=None)	Reduz as saídas por PCA, ajusta o surrogate base nos escores e volta ao espaço original.
TransformedSurrogate	classe	(base, forward, inverse, name='transform')	Ajusta o surrogate base em forward(Y) e desfaz com inverse na previsão.
PerQuantitySurrogate	classe	(blocks)	Um surrogate por bloco de colunas de saída.
HybridPrediction	classe	(intermediates, derived_at_mean, derived_mean, derived_std, derived_lower, derived_upper,…)	Saída do híbrido: intermediárias, grandeza derivada na média, média e desvio por Monte Carlo e intervalo.
HybridSurrogatePhysics	classe	(surrogate, post_model, *, param_names=None)	Compõe o surrogate com o pós-modelo; with_post_model troca a física sem retreinar.

pinneapple_neural.workflows.parametric_dense_volume_operator

Extensão multicaso de dense_volume_operator: um FNO3d treinado em vários casos que diferem num parâmetro físico theta, usado como canal constante de entrada, com normalização global.

pinneapple_neural/workflows/parametric_dense_volume_operator.py · 382 linhas · testes: 3

nome	tipo	assinatura	o que faz
`ParametricCase`	classe	`(name, frames, theta)`	Um caso de treino: sequência de volumes e o parâmetro físico `theta` que a gerou.
`ParametricDenseVolumeOperatorConfig`	classe	`(state_channels, theta_dim, width, modes, layers, epochs, lr, val_frac, batch_size, rollo…)`	Configuração do treino paramétrico (canais de estado, dimensão de theta, escala log de theta, pushforward).
`train_parametric_dense_volume_operator`	função	`(cases, cfg, *, seed=0) -> Tuple[Checkpoint, List[Dict[str, Any]]]`	Treina um `FNO3d` condicionado em theta em todos os casos juntos.
`load_parametric_dense_volume_operator`	função	`(checkpoint, *, device='cpu') -> nn.Module`	Reconstrói o `FNO3d` paramétrico a partir do checkpoint.
`predict_single_step_parametric`	função	`(checkpoint, input_frame, theta, *, device='cpu') -> np.ndarray`	Um passo de previsão num `theta` dado, inclusive não visto no treino.
`rollout_parametric_dense_volume_operator`	função	`(checkpoint, initial_frame, theta, n_steps, *, device='cpu') -> np.ndarray`	Rollout autorregressivo num `theta` dado, recolocando o canal de theta a cada passo.

Exemplos de uso reais do repositório

Montar um modelo pelo registro com tamanho genérico — `tests/pinneapple_neural/test_registry_size_kwargs.py`

```
        @pytest.mark.parametrize("name", ["vanilla_pinn", "bench_mlp"])
        def test_size_kwargs_are_honoured(name):
            model, report = ModelRegistry.build_with_report(name, in_dim=2, out_dim=1, hidden_dim=16, n_layers=2)
            assert not ({"hidden_dim", "n_layers"} & set(report.dropped)), report.dropped
            assert _linears(model) == [(2, 16), (16, 16), (16, 1)]            # two hidden layers of width 16
```

Estudo de ROM em snapshots separados — `tests/test_rom_study.py`

```
        def test_dmd_and_opinf_forecast_a_linear_wave_and_time_interpolation_cannot():
            r = rom_study(_wave(), ranks=[2, 4, 8], methods=["persistence", "pod_rbf", "dmd", "opinf"],
timing_repeats=2)
            err = {(x["method"], x["rank"]): x["error_mean"] for x in r.rows}
            assert err[("persistence", None)] > 0.5                           # the waves move: the last snapshot is
wrong
            assert err[("opinf", 4)] < 1e-3          # rank 4 holds the two waves (OpInf works on centred coefficients)
            assert err[("dmd", 8)] < 1e-3            # uncentred DMD: the mean is one more mode
            assert err[("dmd", 4)] > 0.1 and err[("opinf", 2)] > 0.1
```

Checks físicos num laço de treino PhysicsNeMo — `tests/test_training_callbacks.py`

```
        def physicsnemo_loop(darcy, w, mon, steps=STEPS):
            a, u, RL = darcy
            net = fno(u)
            opt = torch.optim.Adam(net.parameters(), 3e-3)
            hook, u2, step = PhysicsNeMoHook(mon), float((u[:64] ** 2).mean()), 0
            while step < steps:
                for batch in loader(a, u):
                    loss = loss_fn(net, batch, RL, w, u2)
                    opt.zero_grad()
                    loss.backward()
                    opt.step()
                    step += 1
                    if hook(net, step, loss) or step == steps:
```

Trainer com perda supervisionada + física — `examples/end_to_end/03_pinn_physics_loss_training.py`

```
combined = CombinedLoss(
    supervised=SupervisedLoss("mse"),
    physics=PhysicsLossHook(physics_loss_fn),
    w_supervised=1.0,
    w_physics=1.0,
```

```
)

def loss_fn(model, y_hat, batch):
    return combined(model, y_hat, batch)

trainer = Trainer(model=model, loss_fn=loss_fn, metrics=default_metrics())
```

MultiAdam recebe os termos de perda no step — `tests/test_multiadam.py`

```
opt = MultiAdam([p])
opt.step([p.sum() ** 2, (p - 1).sum() ** 2])
```

Testes e status de validação

- Módulos com pelo menos um teste que os importa: 46 de 198.
- Funções de teste em arquivos que importam algum módulo do pacote: 438.

Itens do catálogo de métodos (`pinneapple_catalog/method_status.json`) cujo código está neste pacote:

id	método	status	testes contra referência	outros testes
S21	Digital twin state estimation: KF/EKF/UKF/EnKF, prognostics, signal reconstruction	tested	2	39
T1	Base trainer: AMP, gradient accumulation, auto batch size, CUDA graphs, profiler	tested	0	9
T2	Distributed: DDP, FSDP, SLURM, torchrun, PowerSGD	untested	0	0
T3	Causal training	untested	0	0
T4	Time marching	untested	0	0
T5	Two phases Adam -> L-BFGS + L-BFGS finetune	tested	0	6
T6	Multi-restart	untested	0	0
T7	Adaptive residual-based collocation	untested	0	0
T8	Loss balancing: ReLoBRaLo, SoftAdapt, PCGrad, Augmented Lagrangian, Inverse Dirichlet, LR Annealing, AutoBalancer	validated	1	6
T9	Weight schedulers: SA-PINN, GradNorm, LossRatio, NTK	tested	0	2
T10	Domain decomposition: XPINN, DoMINO/D3M	tested	0	2
T11	Inverse PINN (learnable physical parameters)	tested	0	3
T12	Gray-box + symbolic regression (PySR)	untested	0	0
T13	Uncertainty: heteroscedastic, B-PINN, MC Dropout, deep ensembles	tested	0	20
T14	SINDy	tested	0	2
T15	Adaptive hyperparameter sweep / architecture search (arena NAS)	untested	0	0
T16	Noether trainer (research only)	tested	0	1
T17	Post-training advisor / audit	untested	0	0
T20	Self-scaled quasi-Newton: BFGS / SSBFGS / SSBroyden (new, 2026-09-24)	validated	1	3
T22	Large Physics Model blocks (Fourier encoding, multi-scale neighbourhoods, geometry code, ensemble UQ, OOD score, fine-tuning)	tested	0	1
T23	MultiAdam: per-loss-term Adam moments for multi-term PINN losses	validated	1	3

Limitações conhecidas

- Boa parte das arquiteturas não tem teste direto (testes=0 no inventário): famílias inteiras como `recurrent`, `transformers`, `reservoir_computing`, `classical_ts`, `continuous` e `convolutions`, além de `predictor`, `siren`, `hash_grid`, `afno` e `modified_mlp`.
- Muitos modelos se descrevem como 'MVP' no docstring (filtros de Kalman, ARIMA, Neural ODE/SDE/CDE, GNNs, Koopman); `rom.deep_uq_rom` é explicitamente um esqueleto ('scaffold').
- Dependências opcionais: `emmiai-noether` (bridge Noether), `physicsnemo` (field_operators e `require_physicsnemo_model`), `tfc` (X-TFC), `torchdiffeq` (métodos `dopri5`/`adams` de `GraphNeuralODE`), `onnxruntime` (`export_onnx`), `scikit-image` (isosuperfícies), `matplotlib` (gráficos), `lightning`/`transformers` (callbacks).
- O bridge Noether é só para pesquisa: emite `ResearchOnlyWarning` sempre e levanta `ResearchOnlyError` com `PINNEAPPLE_COMMERCIAL_MODE=1`; as classes viram `None` se o pacote não estiver instalado.
- `MeshGraphNet` implementa só o tipo de aresta da malha; o conjunto de arestas no espaço do mundo (contato) do artigo original não existe.
- `register_all` ignora em silêncio (log em debug) famílias cujo import falha, então um modelo pode simplesmente não aparecer no registro.
- A exportação ONNX de `ReducedModel` só cobre alguns métodos e opções e levanta `NotImplementedError` nos demais (por exemplo, RBF que não seja thin-plate spline).
- O adaptador padrão de `adapters.base` usa `batch.get('x') or batch.get('x_col')`, o que falha com tensores de mais de um elemento; na prática ele depende de o modelo ter `forward_batch`.
- Muitos módulos não têm teste direto (por exemplo `causal`, `time_marching`, `two_phase`, `hpc`, `distributed`, `parallel`, `collocation`, `multistart`, `graybox`, `heteroscedastic`).
- `Metric` e `Metrics` em `metrics.py` são bases abstratas e levantam NotImplementedError.
- Dependências opcionais: Optuna em `adaptive_sweep` (sem ele cai para busca aleatória), PySR em `distill_to_symbolic`, Noether em `noether_trainer`, matplotlib nos gráficos e `pynvml` no `ThroughputMonitor`.
- O docstring de `hpc.py` cita `CUDAGraphContext`, `PowerSGDCompressor` e `TopKGradCompressor`, mas o módulo expõe só `CUDAGraphModule` e as funções `register_powersgd_hook`/`register_topk_hook`; o docstring também fala em estender `pinneapple_train.parallel`.
- FSDP, ZeRO, CUDA graphs, DDP e ganchos de compressão exigem GPU e grupo de processos inicializado; fora disso há fallback ou nada acontece.
- `normalizers.py`, `splits.py` e `viz.py` são shims de compatibilidade que reexportam ou delegam para `pinneapple_data` e `pinneapple_viz`.

2. `pinneapple_data`

Objetivo

`pinneapple_data` é a camada de dados do PINNeAPPle. Ela define o formato comum de dados físicos, o UPD (Unified Physical Data), cujo contêiner é `PhysicalSample`: estado (de preferência um `xarray.Dataset`), geometria, esquema das equações, domínio e proveniência. Em volta desse formato há gravação e leitura (`serialization`, `zarr_store` e variantes com cache, shards e prefetch), `DataLoaders` do PyTorch, splits, normalizadores, validação e geração de pontos de colocação e batches para treinar PINNs (`collocation`, `active_learning`, `adapters.pinn_batch_builders`, `upd_types`).

O pacote também concentra os datasets. `datasets` tem um registro de datasets prontos (física gerada analiticamente ou por solver, geometria, séries temporais), carregadores de dados reais da web e clientes de fontes externas; vários destes últimos caem em dados sintéticos quando a fonte não responde. Há ainda leitores de datasets públicos (DeepMind `meshgraphnets`), um amostrador de campos aleatórios gaussianos para benchmarks de operadores, geradores sintéticos (`synth`) e a auditoria física de datasets (`audit`), que verifica se os dados de fato resolvem a EDP declarada antes do treino.

Uma terceira parte é a camada de dados de engenharia usada pelos apps (`apps/`): leitura de malhas e resultados CAE de OpenFOAM, CalculiX/Abaqus, Gmsh, VTK e STL (`cae`), qualidade de malha, comparação de resultados, pré-voo de simulações (`preflight`), extração de metadados e convergência (`simulation_metadata`), saúde de dados de sensores (`data_health`), padronização de unidades e fontes (`unified`, `physical_units`), snapshots

para modelos de ordem reduzida, linhagem de modelos (`lineage`), preenchimento de datasheets e formulários a partir de documentos (`formfill`, `udr1`) e otimização de processo só com dados. Apps como `simulation_preflight`, `mesh_quality`, `data_health` e `model_lineage` importam estes módulos diretamente.

Os contratos em `contracts` (`ParameterSpace`, `DatasetSpec`, `ModelRef`, `StudyResult`) e o design de experimentos em `doe` ligam os apps entre si: um desenho de experimentos vira campanha de simulação (`pinneapple_simulation.external_solvers.campaign_cases`), as execuções viram dataset e o dataset vira modelo substituto. O pacote se apoia em `pinneapple_design.geometry` para geometria, em `pinneapple_physics` para `ProblemSpec` e presets, e em `pinneapple_security.archives` para expandir uploads com segurança; `pinneapple_train` reexporta `splits` e `transforms`.

Números do inventário

módulos	linhas	classes públicas	funções públicas	testes em arquivos que importam o pacote	módulos sem teste direto
115	29.019	129	256	507 (54 arquivos)	70

Pasta de testes dedicada: `tests/pinneapple_data` (33 funções de teste).

Submódulos

submódulo	o que faz	módulos	classes + funções	testes*
`active_learning`	Escolha adaptativa de pontos de colocação de PINN por resíduo, variância ou combinação.	1	5	22
`adapters`	Adaptadores de geometria, shards UPD, batches PINN e arquivos `.splash` para o formato do pacote.	5	19	0
`audit`	Auditoria física de datasets de treino: EDP satisfeita, vazamento, duplicatas, diversidade e restrições.	1	10	49
`augmentation`	Aumento de nuvens de pontos com ruído, rotação/escala e interpolação.	1	3	0
`cache`	Cache LRU em memória limitado por número de itens.	1	2	0
`cache_bytes`	Cache LRU em memória limitado por bytes.	1	3	1
`cae`	Leitura de malhas e resultados CAE (OpenFOAM, CalculiX/Abaqus, Gmsh, VTK, STL), qualidade de malha, comparação de resultados e exportação neutra.	13	74	62
`chunking`	Fatiamento de tensores grandes em pedaços.	1	2	0
`collate`	Collate e transferência de dispositivo de batches PINN e supervisionados.	1	3	0
`collocation`	Geração de pontos de colocação a partir de limites, `ProblemSpec`, domínios, malhas ou STL.	1	2	7
`contracts`	Contratos de dados abertos entre apps (`ParameterSpace`, `DatasetSpec`, `ModelRef`, `StudyResult`) e tabelas com sidecar.	4	27	175
`data_health`	Relatório de qualidade física de dados de sensores, ensaios e planta.	1	6	37
`dataloaders`	Construção de `DataLoader` do PyTorch para shards UPD e `PhysicalSample`.	1	3	0
`datasets`	Registro de datasets prontos (física, geometria, séries temporais, reais e externos) e downloads.	9	28	10
`device`	Pin de memória e troca de dispositivo de amostras.	1	2	0
`doe`	Design de experimentos e planejamento de campanhas de simulação.	4	10	21
`expressions`	Avaliador seguro de expressões de restrição, sem `eval`.	1	5	22
`formfill`	Preenchimento de formulários e datasheets de engenharia a partir de documentos, com fonte de cada valor.	14	31	44
`geometry_features`	Nuvem de pontos e grade SDF a partir de malha triangular.	1	3	0
`grf_sampler`	Campos aleatórios gaussianos 1D/2D para funções de entrada de benchmarks de operador.	1	5	0
`lineage`	Linhagem de modelos de engenharia (digital thread) com checagens e exportação PROV.	3	4	13
`noether_dataset`	Ponte entre datasets do Noether e `PhysicalSample`.	1	3	5
`parameter_sampling`	Amostragem de espaços de parâmetros (LHS, Sobol, Halton, Saltelli).	1	2	5

submódulo	o que faz	mó-dulos	classes + funções	tes-tes*
physical_sample	Contêiner PhysicalSample alinhado ao UPD.	1	1	28
physical_units	Unidades, conversão para SI e inferência de grandeza física, só com a biblioteca padrão.	1	12	52
physics_case	PhysicsCase: geometria + spec + solver + resultados referenciados juntos.	1	2	47
prefetch_adaptive	Streaming de store Zarr UPD com prefetch adaptativo.	1	2	0
preflight	Checagem de casos OpenFOAM e decks CalculiX antes de rodar o solver.	4	11	13
process_optimizer	Otimização de processo industrial só com dados (gradient boosting).	1	7	16
serialization	Gravação e leitura de PhysicalSample em .pt, HDF5 e Zarr.	1	9	11
simulation_metadata	Extração de metadados e avaliação de convergência de simulações OpenFOAM e CalculiX.	5	25	25
snapshots	Matrizes de snapshots para modelos de ordem reduzida.	1	3	31
splits	Divisão treino/validação/teste aleatória, temporal ou por grupo.	1	2	0
stl_import	Leitor de STL sem dependências.	1	3	0
synth	Geradores de dados sintéticos (EDP, distribuições, curvas, imagens, geometria).	15	20	0
transforms	Normalizadores (StandardScaler, MinMaxScaler).	1	3	2
udr1	Atalhos do formulário ASME U-DR-1 sobre formfill.	1	3	36
unified	Padronização de várias fontes de dados de engenharia num esquema físico único.	1	9	24
upd_dataset	Dataset UPD com validação e persistência.	1	1	5
upd_types	Tipos puros do pipeline UPD (shard, amostragem, batch).	1	5	0
validators	Validação de PhysicalSample.	1	3	0
zarr_cached_store	Store Zarr UPD com cache LRU por contagem.	1	2	0
zarr_cached_store_bytes	Store Zarr UPD com cache LRU por bytes.	1	2	0
zarr_iterable	IterableDataset sobre store Zarr UPD com divisão por worker.	1	1	0
zarr_prefetch	IterableDataset Zarr com prefetch em thread e transferência para GPU.	1	2	0
zarr_shard_iterable	IterableDataset balanceado sobre stores Zarr fatiados em shards.	1	2	0
zarr_shards	Escritor de stores Zarr UPD fatiados por chave.	1	2	0
zarr_store	Store Zarr colunar do UPD.	1	1	1

* maior número de testes que importam um módulo do submódulo.

Módulos e API pública

pinneapple_data

Raiz do pacote: importa, cada um dentro de try/except (falha silenciosa), STL, PhysicalSample, armazenamento Zarr, collate, colocação, active learning, normalizadores, tipos UPD, splits e construtores de batch; os datasets são importados sem proteção.

pinneapple_data/__init__.py · 117 linhas · testes: 78

pinneapple_data.active_learning

Active learning para escolher pontos de colocação de PINN: seleção por resíduo (RAR/RAD), por variância de um modelo com incerteza, combinação ponderada com diversidade, e um treinador que refina a colocação a cada al_every épocas.

pinneapple_data/active_learning.py · 674 linhas · testes: 22

nome	tipo	assinatura	o que faz
`ActiveLearningConfig`	classe	`(strategy, n_candidates, n_select, n_initial, n_iterations, exploitation_weight, seed)`	Configuração do active learning: estratégia, número de candidatos, de pontos escolhidos por rodada, iniciais, iterações, peso de exploração e semente.
`ResidualBasedAL`	classe	`(config, bounds) -> None`	Escolhe pontos de colocação onde o resíduo da EDP é maior, pegando os top-k (RAR) ou sorteando com peso proporcional ao resíduo (RAD).
`VarianceBasedAL`	classe	`(config, bounds) -> None`	Escolhe pontos onde a variância da saída do modelo é maior; exige um modelo com incerteza que exponha `variance_fn` (ensemble, rede bayesiana).
`CombinedAL`	classe	`(config, bounds, residual_weight=0.6, variance_weight=0.3, diversity_weight=0.1) -> None`	Pontua candidatos por soma ponderada de resíduo, variância e diversidade (distância aos pontos já escolhidos), todos normalizados.
`AdaptiveCollocationTrainer`	classe	`(model, physics_fn, sampler, al_config=None, optimizer=None, device='cpu') -> None`	Envolve um PINN e uma função física, mantém o próprio conjunto de colocação e acrescenta pontos escolhidos pela estratégia de active learning durante o treino.

`pinneapple_data.adapters`

Reexporta os adaptadores de geometria, de shards UPD, de batch PINN e de arquivos `.splash`; os de geometria e UPD ficam em `try/except`. [sem teste]

`pinneapple_data/adapters/__init__.py` · 63 linhas · **sem teste direto**

`pinneapple_data.adapters.geom_adapter`

Carrega geometria (arquivo, STL, objeto) como `GeometryAsset` via `pinneapple_design.geometry` e a anexa a um `PhysicalSample`; o docstring ainda usa o nome antigo `pinneapple_geom`. [sem teste]

`pinneapple_data/adapters/geom_adapter.py` · 286 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`GeometryLoadOptions`	classe	`(scale, translate, rotate_euler_deg, repair, compute_normals, boundary_labels)`	Opções de carga de geometria: escala, translação, rotação em graus de Euler, reparo, cálculo de normais e rótulos de contorno.
`load_geometry_asset`	função	`(geom, *, options=None) -> Any`	Carrega ou monta um `GeometryAsset` a partir de caminho, STL ou especificação, delegando ao registro de geometria de `pinneapple_design`.
`stl_to_upd`	função	`(path, *, repair=True, compute_normals=True, units='m') -> Any`	Lê um STL com `trimesh` e devolve um `PhysicalSample` de domínio malha com `vertices` e `faces` como tensores e a proveniência do arquivo.
`attach_geometry`	função	`(sample, geom_asset) -> Any`	Anexa um `GeometryAsset` a uma amostra (dict ou objeto) e define o tipo de domínio como `mesh` se ainda não houver.

`pinneapple_data.adapters.pinn_batch_builders`

Converte fontes diferentes (bundle do Arena, solver, medições reais) para o batch em dict usado pelo compilador PINN (`x_col`, `x_bc`, `y_bc`, `x_ic`, `y_ic`, `x_data`, `y_data`, `ctx`). [sem teste]

`pinneapple_data/adapters/pinn_batch_builders.py` · 149 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`PINNBatch`	classe	`(batch)`	Invólucro fino de um batch PINN em memória (o dict de tensores).
`build_from_bundle`	função	`(bundle, *, n_collocation=4096, n_boundary=2048, n_data=2048, device=None, dtype=torch.fl…)`	Monta um batch PINN a partir de um `BundleData` de `pinneapple_arena`, sorteando pontos de colocação, contorno e dados.
`build_from_solver`	função	`(problem_spec, geometry, solver_cfg, *, device=None, dtype=torch.float32) -> PINNBatch`	Monta um batch PINN chamando um `solver_fn` da configuração ou usando tensores já prontos nela.
`build_from_real_data`	função	`(adapter_cfg, *, device=None, dtype=torch.float32) -> PINNBatch`	Monta um batch PINN a partir de medições reais, via `loader_fn` ou tensores `x_data`/`y_data` dados diretamente.

pinneapple_data.adapters.splash_archive_adapter

Lê um caso `.splash` (caso OpenFOAM zipado) para arrays NumPy: nuvem de pontos para PINN ou volumes densos em grade para operadores neurais como `FNO3d`; código promovido de um projeto derivado. [sem teste]

`pinneapple_data/adapters/splash_archive_adapter.py` · 315 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`SplashMesh`	classe	`(n_cells, n_points, n_faces, n_internal_faces, cell_centers, cell_size, cell_delta, bound…)`	Malha reconstruída de um `.splash`: contagens de células, pontos e faces, centros e tamanhos de célula e limites do domínio.
`FieldSnapshot`	classe	`(time, fields)`	Campos de um instante de tempo de um caso `.splash` (tempo + dict de arrays).
`SplashCase`	classe	`(path, mesh, time_dirs, transport, meta)`	Caso `.splash` aberto: malha, diretórios de tempo e propriedades; lê campos, instantâneos, nuvem de pontos e volume denso.
`load_mesh`	função	`(zf, prefix='constant/polyMesh/') -> SplashMesh`	Monta centros e tamanhos de célula a partir do polyMesh binário dentro do zip `.splash` aberto.
`save_mesh_cache`	função	`(mesh, path) -> None`	Grava um `SplashMesh` num `.npz` comprimido para reaproveitar depois.
`load_mesh_cache`	função	`(path) -> SplashMesh`	Lê de volta um `SplashMesh` gravado por `save_mesh_cache`.
`open_case`	função	`(path, mesh_cache_dir=None) -> SplashCase`	Abre um arquivo `.splash`, reconstruindo a malha ou lendo-a do cache.
`load_dense_volumes`	função	`(splash_path, field_components, nx, ny, nz, *, mesh_cache_dir=None) -> Tuple[np.ndarray, …)`	Lê todos os diretórios de tempo de um `.splash` como série temporal de volumes densos `nx×ny×nz`, para malha de um único bloco hexaédrico estruturado.

pinneapple_data.adapters.upd_adapter

Carrega shards UPD (Zarr + JSON de metadados) como `xarray.Dataset` e os converte em `PhysicalSample` ou anexa o estado a uma amostra existente. [sem teste]

`pinneapple_data/adapters/upd_adapter.py` · 288 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`load_upd_item`	função	`(upd) -> Tuple[xr.Dataset, Dict[str, Any]]`	Lê um shard UPD (par de caminhos, dict ou dataset pronto) e devolve o `xarray.Dataset` e os metadados.
`upd_to_physical_sample`	função	`(upd, *, schema_override=None, provenance_extra=None) -> Any`	Converte um shard UPD em `PhysicalSample`, com esquema opcional sobrescrito e proveniência extra.
`attach_upd_state`	função	`(sample, upd) -> Any`	Anexa estado, esquema, domínio e proveniência de um shard UPD a uma amostra existente (dict ou objeto).

pinneapple_data.audit

Auditoria física de um dataset de treino só com NumPy/SciPy: verifica se os dados resolvem a própria EDP, vazamento entre splits, duplicatas, diversidade, restrições e unidades, e gera um cartão de auditoria.

`pinneapple_data/audit.py` · 519 linhas · testes: 49

nome	tipo	assinatura	o que faz
`AuditCheck`	classe	`(id, status, summary, value, threshold, evidence)`	Resultado de uma verificação da auditoria: id, status (passa/falha/não se aplica), resumo, valor, limite e evidência.
`AuditCard`	classe	`(dataset, splits, physics, checks, tolerances)`	Cartão de auditoria de um dataset com todas as verificações; dá a decisão geral e exporta para dict, JSON e Markdown.
`DatasetPhysics`	classe	`(name, columns, check, tolerances, units)`	Física por amostra de um dataset: função `check(sample)` que devolve erros relativos por nome, e as tolerâncias de cada um.
`darcy_consistency`	função	`(a, u, f=1.0) -> dict[str, float]`	Verifica uma amostra de Darcy na discretização de `operator_benchmarks.darcy_2d`: distância à solução do próprio problema discreto e erro nas condições de contorno.

nome	tipo	assinatura	o que faz
`effective_sample_count`	função	`(x) -> float`	Razão de participação do espectro de componentes principais das amostras: aproxima quantas direções independentes os dados cobrem.
`near_duplicates`	função	`(x, near=0.001) -> dict[str, Any]`	Acha duplicatas exatas e quase duplicatas dentro de um conjunto, relativas à distância mediana ao vizinho mais próximo.
`leakage`	função	`(train, test, near=0.001) -> dict[str, Any]`	Para cada amostra de teste acha a de treino mais próxima e marca como cópia (vazamento) as que ficam perto demais.
`constraint_mask`	função	`(expr, cols) -> np.ndarray`	Avalia elemento a elemento onde uma relação declarada (como `"Cd > 0"` ou `"a == b * c"`) vale; serve também para checar predições.
`audit_dataset`	função	`(splits, *, name='dataset', physics=None, inputs=None, units=None, expected_units=None, c…)`	Audita splits `{split: {coluna: array}}` com física, restrições, diversidade, vazamento e unidades, e devolve um `AuditCard`.
`audit_zoo_dataset`	função	`(name, *, offline=None, **kw) -> AuditCard`	Audita uma entrada de `pp.datasets` usando a física e as restrições declaradas no seu catálogo.

`pinneapple_data.augmentation`

Aumento de dados para nuvens de pontos `(coords, fields)`: ruído gaussiano, rotação/escala de coordenadas e novos pontos por interpolação, sem acoplamento a framework. [sem teste]

`pinneapple_data/augmentation.py` · 139 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`augment_noise`	função	`(coords, fields, n_copies, std_fraction=0.05, seed=0) -> Dict[str, Any]`	Faz `n_copies` cópias do dataset somando ruído gaussiano proporcional ao desvio-padrão de cada campo; coordenadas ficam iguais.
`augment_transform`	função	`(coords, fields, n_copies, max_rotation_deg=15.0, scale_range=(0.9, 1.1), seed=0) -> Dict…)`	Faz `n_copies` cópias com rotação no plano dos dois primeiros eixos e escala isotrópica das coordenadas; os campos não são rotacionados.
`augment_interpolation`	função	`(coords, fields, n_new_points, method='linear', seed=0) -> Dict[str, Any]`	Gera pontos novos dentro do fecho convexo da nuvem e estima os campos por interpolação dispersa, com vizinho mais próximo onde a interpolação dá NaN.

`pinneapple_data.cache`

Cache LRU em memória, seguro para threads, limitado por número de itens. [sem teste]

`pinneapple_data/cache.py` · 120 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`CacheStats`	classe	`(hits, misses, evictions)`	Contadores do `LRUCache`: acertos, falhas e despejos.
`LRUCache`	classe	`(max_items=256)`	Cache LRU seguro para threads limitado por `max_items`, com `get`, `put` e `clear`.

`pinneapple_data.cache_bytes`

Cache LRU seguro para threads limitado por bytes, para tensores e amostras.

`pinneapple_data/cache_bytes.py` · 179 linhas · testes: 1

nome	tipo	assinatura	o que faz
`ByteCacheStats`	classe	`(hits, misses, evictions, bytes_in_use)`	Contadores do `ByteLRUCache`: acertos, falhas, despejos e bytes em uso.
`ByteLRUCache`	classe	`(max_bytes=512 * 1024 * 1024, weigher=default_weigher)`	Cache LRU que limita o total de bytes (`max_bytes`) em vez do número de itens, despejando os menos usados.
`default_weigher`	função	`(value) -> int`	Estima o tamanho em bytes de um valor: `nbytes` de tensor, soma para dicts e amostras, e um valor fixo de reserva para o resto.

pinneapple_data.cae

Camada de dados CAE: lê malhas e resultados de OpenFOAM, CalculiX/Abaqus, Gmsh, VTK, STL e tabelas de pontos num único tipo `Mesh`, com qualidade de malha, Courant e distância à parede.

`pinneapple_data/cae/__init__.py` · 14 linhas · testes: 44

pinneapple_data.cae.compare

Compara dois resultados campo a campo (simulação, experimento, predição de IA ou referência analítica), interpolando o candidato nas posições da referência e calculando MAE, RMSE, erro máximo, viés, L2 relativo, NRMSE e R^2.

`pinneapple_data/cae/compare.py` · 420 linhas · testes: 24

nome	tipo	assinatura	o que faz
`Interpolator`	classe	`(src_pts, dst_pts, method='auto')`	Interpola um campo candidato em pontos de consulta: cópia direta se as posições coincidem, linear (Delaunay) dentro do domínio e vizinho mais próximo fora.
`locations`	função	`(m, field) -> Tuple[np.ndarray, str, Optional[np.ndarray]]`	Devolve onde um campo vive (centros de célula ou nós), o tipo de posição e os pesos (volumes de célula, se houver).
`cell_centres_volumes`	função	`(m) -> Tuple[np.ndarray, Optional[np.ndarray]]`	Calcula (e guarda em cache na malha) centros e volumes de célula pela geometria de volumes finitos.
`field_value`	função	`(m, f) -> np.ndarray`	Devolve os valores de um campo da malha, ou de uma componente dele quando o nome a indica.
`fields_of`	função	`(m) -> Dict[str, Dict[str, Any]]`	Lista os campos de uma malha com posição (ponto/célula), número de componentes, unidade e tamanho.
`match_fields`	função	`(ref, cand, mapping=None) -> List[Tuple[str, str]]`	Pareia campos de referência e candidato: mapeamento explícito, depois mesmo nome sem caixa, depois apelidos (U~velocity, p~pressure, T~temperature, DISP~displacement).
`tolerance_verdicts`	função	`(result, tolerance) -> Optional[str]`	Marca cada campo de um resultado de `compare` como `pass`/`fail` pelo erro L2 relativo contra uma tolerância e devolve o veredito geral.
`compare`	função	`(ref, cand, mapping=None, method='auto', fields=None) -> Dict[str, Any]`	Compara referência e candidato em todos os campos pareados e devolve métricas por campo e componente; levanta `ValueError` se não houver campo em comum.
`surface`	função	`(m, max_tris=300000) -> Optional[Dict[str, Any]]`	Extrai a superfície de contorno em triângulos, com a célula de cada um, para exibição; em casos 2D do OpenFOAM mantém só um lado `empty`.
`views`	função	`(ref, res, max_points=60000) -> Dict[str, Any]`	Monta o payload de exibição por campo: superfície colorida, gráfico de linha ou mapa de pontos 2D/3D.

pinneapple_data.cae.dataset

Dataset físico neutro: normaliza qualquer resultado suportado em geometria, malha, coordenadas e campos com grandeza e unidade, e exporta para VTU, HDF5, Parquet, CSV, NPZ, JSON ou dataset PINNeAPPle (Zarr UPD).

`pinneapple_data/cae/dataset.py` · 507 linhas · testes: 14

nome	tipo	assinatura	o que faz
`quantity_of`	função	`(name) -> Tuple[Optional[str], Optional[str]]`	Infere grandeza física e unidade a partir do nome de um campo, por padrões de nome.
`poly_cells`	função	`(m)`	Converte células OpenFOAM em células VTK (hexaedro, prisma, tetraedro, pirâmide) e as demais em poliedros VTK.
`field_scalar`	função	`(values) -> Tuple[Optional[np.ndarray], str]`	Reduz um campo a um escalar por ponto/célula: o próprio valor, a magnitude de um vetor ou a tensão de von Mises de um tensor simétrico.
`build_dataset`	função	`(m, *, unit_system='auto', to_si=False, rho=None, location='auto', files=None) -> Dict[st…)`	Normaliza uma `Mesh` no dataset neutro (dict com arrays e metadados), com sistema de unidades, conversão opcional para SI e escolha de posição.

nome	tipo	assinatura	o que faz
`describe`	função	`(ds) -> Dict[str, Any]`	Descrição do dataset segura para JSON (o esquema mostrado ao usuário), sem arrays.
`export`	função	`(ds, fmt) -> Tuple[bytes, str, str]`	Serializa o dataset no formato pedido e devolve (conteúdo, tipo de mídia, extensão).
`write_vtu`	função	`(ds) -> bytes`	Escreve o dataset como VTK XML não estruturado (`.vtu`), incluindo poliedros quando a malha é OpenFOAM.
`write_hdf5`	função	`(ds) -> bytes`	Escreve o dataset em HDF5 com malha, campos e metadados (requer `h5py`).
`write_parquet`	função	`(ds) -> bytes`	Escreve o dataset como tabela Parquet com unidades nos metadados (requer `pandas` e `pyarrow`).
`write_csv`	função	`(ds) -> bytes`	Escreve o dataset como CSV com unidades no cabeçalho e uma linha de comentário com esquema e origem.
`write_npz`	função	`(ds) -> bytes`	Escreve pontos, centros de célula, campos e metadados num `.npz` comprimido.
`write_json`	função	`(ds, max_values=2000000) -> bytes`	Escreve a descrição em JSON e inclui os valores só se o total couber em `max_values`.
`write_pinneapple`	função	`(ds) -> bytes`	Escreve o dataset como um `PhysicalSample` num store Zarr UPD zipado, que se lê com `serialization.load_zarr`.
`preview`	função	`(ds, max_fields=6, max_tris=200000) -> Dict[str, Any]`	Payload de exibição: superfície de contorno colorida pelos primeiros campos reduzidos a escalar.

`pinneapple_data.cae.foam`

Leitores OpenFOAM: polyMesh (pontos, faces, owner, neighbour, boundary) e campos de volume de um diretório de tempo, em ASCII ou binário, com ou sem gzip; casos decompostos (`processor*`) não são remontados e nada é executado.

`pinneapple_data/cae/foam.py` · 372 linhas · testes: 6

nome	tipo	assinatura	o que faz
`header`	função	`(text) -> Dict[str, str]`	Lê o cabeçalho `FoamFile` de um arquivo OpenFOAM como dict de texto.
`read_points`	função	`(b) -> np.ndarray`	Lê o arquivo `points` do polyMesh (ASCII ou binário) como array (n, 3).
`read_labels`	função	`(b) -> np.ndarray`	Lê uma lista de rótulos inteiros (como `owner` ou `neighbour`) do polyMesh.
`read_faces`	função	`(b) -> Tuple[np.ndarray, np.ndarray]`	Lê o arquivo `faces` como lista plana de nós mais offsets (suporta `faceList` e `faceCompactList`).
`read_boundary`	função	`(b) -> List[Dict[str, Any]]`	Lê o arquivo `boundary` e devolve os patches com nome, tipo, número de faces e face inicial.
`read_polymesh`	função	`(fs, root='') -> Mesh`	Monta uma `Mesh` a partir de `constant/polyMesh` do upload; levanta `ValueError` se o polyMesh não estiver presente.
`dims_unit`	função	`(dims) -> Optional[str]`	Converte o vetor de dimensões OpenFOAM (`[0 2 -1 0 0 0 0]`) em texto de unidade.
`read_boundary_values`	função	`(b, mesh, width) -> Optional[np.ndarray]`	Valor em cada face de contorno lido do `boundaryField`: o `value` escrito, zero para `noSlip`, o valor da célula vizinha quando nada está escrito, NaN para patches `empty`.
`read_field`	função	`(b, n_cells) -> Tuple[np.ndarray, Optional[str], str]`	Lê o `internalField` de um campo de volume: valores por célula, unidade e classe.
`time_dirs`	função	`(fs, root='') -> List[str]`	Lista os diretórios de tempo numéricos do caso em ordem crescente.
`read_time`	função	`(fs, mesh, root='', time=None, only=None) -> Dict[str, Any]`	Carrega os campos de volume de um diretório de tempo (o último, por padrão) em `mesh.cell_data`.
`boundary_conditions`	função	`(fs, mesh, root='', time=None) -> Dict[str, Dict[str, str]]`	Devolve patch → campo → condição de contorno como escrita (por exemplo `fixedValue uniform (1 0 0)`, `noSlip`), por padrão do primeiro diretório de tempo.

pinneapple_data.cae.frd

Leitor do arquivo de resultados CalculiX `.frd` (ASCII): nós, elementos e blocos de resultado (DISP, STRESS, NDTEMP, FLUX...); guarda o último incremento de cada resultado.

`pinneapple_data/cae/frd.py` · 117 linhas · testes: 6

nome	tipo	assinatura	o que faz
`read_frd`	função	`(b) -> Mesh`	Lê um `.frd` ASCII e devolve uma `Mesh` com os resultados do último incremento em `point_data` e a lista de incrementos em `source`.

pinneapple_data.cae.geometry

Geometria de volumes finitos de qualquer malha, calculada como o `primitiveMesh` do OpenFOAM (centros e vetores de área de face, centros e volumes de célula); converte malhas FE em faces/owner/neighbour. [sem teste direto]

`pinneapple_data/cae/geometry.py` · 249 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`FaceMesh`	classe	`(points, face_nodes, face_offsets, owner, neighbour, n_cells, patch_of_face, patch_names,…)`	Malha na forma de faces: nós de face, owner, neighbour, patches e tipos, faces não-manifold e blocos invertidos.
`signed_volume_sign`	função	`(btype, pts, conn) -> np.ndarray`	Sinal de uma estimativa simples do volume de cada elemento, por decomposição em tetraedros a partir do primeiro canto.
`oriented_blocks`	função	`(mesh) -> List[str]`	Inverte a ordem de nós de blocos inteiros cuja maioria dos elementos sai com volume negativo (convenções diferentes entre VTK, Gmsh e Abaqus) e devolve os nomes dos blocos invertidos.
`face_mesh`	função	`(mesh) -> FaceMesh`	Devolve o polyMesh OpenFOAM como está, ou converte uma malha FE de volume em faces (internas primeiro, depois contorno).
`face_geometry`	função	`(fm) -> Dict[str, np.ndarray]`	Calcula centros de face, vetores de área e os grupos de cantos por decomposição em triângulos.
`cell_geometry`	função	`(fm, fg) -> Dict[str, np.ndarray]`	Calcula centros e volumes de célula por decomposição em pirâmides a partir das faces.
`cell_adjacency`	função	`(fm) -> sp.csr_matrix`	Matriz esparsa de adjacência entre células que compartilham uma face interna.
`regions`	função	`(adj) -> np.ndarray`	Rótulo de componente conexa de cada célula a partir da matriz de adjacência.
`element_adjacency_by_nodes`	função	`(mesh) -> sp.csr_matrix`	Adjacência entre células que compartilham pelo menos um nó (para malhas de superfície e para a checagem de conectividade).
`solution_dirs`	função	`(fm, fg) -> Optional[np.ndarray]`	Indica em quais direções não há patch `empty` (casos OpenFOAM 2D/1D).

pinneapple_data.cae.inp

Leitor de decks Abaqus/CalculiX `.inp`: blocos de palavra-chave com número de linha, nós, elementos em tipos VTK, conjuntos de nós e elementos e `*INCLUDE` resolvido a partir do upload.

`pinneapple_data/cae/inp.py` · 296 linhas · testes: 6

nome	tipo	assinatura	o que faz
`Block`	classe	`(keyword, params, data, line, file)`	Um bloco de palavra-chave do deck: palavra-chave, parâmetros, linhas de dados, linha e arquivo de origem.

nome	tipo	assinatura	o que faz
`Deck`	classe	`(blocks, includes, missing_includes)`	Deck lido: lista de blocos, includes resolvidos e includes faltantes; `find` busca blocos por palavra-chave.
`vtk_type`	função	`(abq) -> Optional[str]`	Traduz um tipo de elemento Abaqus (como `C3D8`) para o tipo VTK correspondente, ou `None`.
`read_deck`	função	`(text, name='model.inp', files=None, _depth=0) -> Deck`	Lê o texto de um deck em blocos, guardando o número de linha e resolvendo `*INCLUDE` a partir dos arquivos dados.
`resolve_sets`	função	`(deck, kind) -> Dict[str, List[str]]`	Resolve conjuntos NSET ou ELSET (que podem conter outros conjuntos) em listas de rótulos de membros.
`deck_mesh`	função	`(deck) -> Mesh`	Converte um `Deck` numa `Mesh` com nós, blocos de elementos e conjuntos.
`deck_conditions`	função	`(deck) -> List[Dict[str, object]]`	Lista apoios e cargas do deck por nó: `*BOUNDARY` (graus de liberdade restritos) e `*CLOAD`/`*CFLUX` (soma de forças ou fluxos nodais), entre outros.

pinneapple_data.cae.io

Lê qualquer upload (dict caminho → bytes) para uma `Mesh`: caso OpenFOAM, `.inp`, `.frd`, STL, Gmsh/VTK e outros formatos do `meshio`, e tabelas de pontos; `read_any` escolhe o leitor. [sem teste direto]

`pinneapple_data/cae/io.py` · 296 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`FormatError`	classe	`()`	Erro para formato não reconhecido ou dependência faltando (por exemplo `meshio`).
`kind_of`	função	`(fs) -> Tuple[str, Optional[str]]`	Detecta o tipo do upload (`openfoam`, `inp`, `frd`, `stl`, `meshio`, `points`) e o arquivo principal ou raiz do caso.
`read_any`	função	`(fs, fields=True, time=None) -> Mesh`	Lê o upload com o leitor certo, incluindo os campos do último (ou do pedido) diretório de tempo no caso OpenFOAM.
`read_stl`	função	`(b) -> Mesh`	Lê STL binário ou ASCII como `Mesh` de superfície de triângulos.
`read_meshio`	função	`(name, b) -> Mesh`	Lê um formato suportado pelo `meshio` (Gmsh, VTK/VTU e outros) via arquivo temporário; levanta `FormatError` sem `meshio`.
`read_points`	função	`(name, b) -> Mesh`	Lê uma tabela de pontos com colunas de campos (CSV/TXT, Parquet, NPZ, HDF5) como nuvem de pontos com `point_data`.

pinneapple_data.cae.metrics

Métricas de escoamento por célula numa malha de volumes finitos: número de Courant e distância à parede mais próxima. [sem teste direto]

`pinneapple_data/cae/metrics.py` · 109 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`courant_number`	função	`(face_area_vectors, face_owner, face_neighbour, U, cell_volumes, dt, where='auto') -> np.…)`	Número de Courant por célula na forma de volumes finitos, `0.5 dt Σ|U_f·S_f| / V_P`.
`wall_distance`	função	`(cell_centres, wall_face_centres, wall_face_normals=None) -> np.ndarray`	Distância de cada centro de célula ao centro de face de parede mais próximo (`cKDTree`), opcionalmente projetada na normal da face.
`mesh_courant_number`	função	`(mesh, U, dt, where='auto') -> np.ndarray`	Aplica `courant_number` a uma `Mesh`, calculando antes a geometria das faces e células.
`mesh_wall_distance`	função	`(mesh, patches=None, project=True) -> np.ndarray`	Aplica `wall_distance` a todas as células de uma `Mesh` em relação aos patches dados (padrão: todos os do tipo `wall`).

pinneapple_data.cae.model

Contêiner comum de malha e resultado (`Mesh`) usado pelos leitores CAE, pela qualidade de malha, pelo comparador e pela exportação; aceita células FE em blocos ou polyMesh OpenFOAM.

`pinneapple_data/cae/model.py` · 173 linhas · testes: 39

nome	tipo	assinatura	o que faz
`CellBlock`	classe	`(type, conn, ids)`	Bloco de células de um único tipo VTK com conectividade de nós; dá versão linear, cantos e dimensão.
`PolyMesh`	classe	`(face_nodes, face_offsets, owner, neighbour, n_cells, patches)`	polyMesh OpenFOAM: faces (lista plana + offsets), owner/neighbour e patches de contorno.
`Mesh`	classe	`(points, blocks, poly, boundary_blocks, cell_sets, node_sets, point_data, cell_data, unit…)`	Malha com resultados: pontos, blocos FE ou polyMesh, conjuntos, campos em pontos e células, unidades, valores de contorno e origem.

`pinneapple_data.cae.quality`

Qualidade de malha: métricas de volumes finitos como o `checkMesh` do OpenFOAM, métricas de forma para elementos FE, conectividade e checagens de superfície STL; cada métrica tem limite e fonte em `METRICS`.

`pinneapple_data/cae/quality.py` · 306 linhas · testes: 16

nome	tipo	assinatura	o que faz
`fv_metrics`	função	`(mesh) -> Dict[str, Any]`	Calcula as métricas de volumes finitos (não-ortogonalidade, skewness, razão de aspecto, volumes, entre outras) a partir da geometria de faces e células.
`element_metrics`	função	`(mesh) -> Dict[str, np.ndarray]`	Calcula Jacobiano escalado, skewness e razão de arestas de cada célula da dimensão principal da malha.
`surface_checks`	função	`(mesh) -> Dict[str, Any]`	Checa uma superfície de triângulos/quadriláteros: arestas abertas e não-manifold, orientação, faces degeneradas e cascas.

`pinneapple_data.cae.report`

Relatório de saúde da malha: checagens com veredito, estatísticas e histogramas, regiões com problema (aglomerados de células ruins) e o que fazer; também o payload da vista 3D.

`pinneapple_data/cae/report.py` · 371 linhas · testes: 8

nome	tipo	assinatura	o que faz
`build`	função	`(mesh, target='auto', max_regions=12) -> Dict[str, Any]`	Monta o relatório de saúde da malha com limites de CFD (`cfd`), de elementos finitos (`fea`) ou escolhidos automaticamente.
`view_payload`	função	`(mesh, rep, max_tris=250000) -> Dict[str, Any]`	Superfície de contorno com o valor da métrica da célula dona de cada face, mais as células das regiões com problema.
`worst_cells`	função	`(mesh, rep, n=40) -> List[Dict[str, Any]]`	As `n` piores células de cada métrica do relatório, respeitando o sentido (maior ou menor é pior).

`pinneapple_data.cae.upload`

Expande um upload (arquivos soltos e zips) em `{caminho relativo: bytes}` com limites de tamanho, mantendo malhas e resultados e descartando lixo e pastas `processor*`.

`pinneapple_data/cae/upload.py` · 45 linhas · testes: 62

nome	tipo	assinatura	o que faz
`expand`	função	`(files, max_unzipped=MAX_UNZIPPED) -> Dict[str, bytes]`	Expande arquivos e zips (via leitor seguro de `pinneapple_security.archives`) num dict caminho → bytes, respeitando `max_unzipped`.

`pinneapple_data.chunking`

Especificação de fatiamento (`ChunkSpec`) e iteração de tensores em pedaços ao longo de um ou mais eixos, para processar arrays grandes em streaming. [sem teste]

`pinneapple_data/chunking.py` · 112 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ChunkSpec`	class e	`(mode, chunk_size, dim, dims, chunk_sizes)`	Como fatiar um tensor: modo (`none`, `time`, `spatial`), tamanho do pedaço e eixo, ou vários eixos com tamanhos próprios.

nome	tipo	assinatura	o que faz
`iter_chunks`	função	`(x, spec) -> Iterator[torch.Tensor]`	Itera sobre os pedaços de um tensor conforme um `ChunkSpec`.

pinneapple_data.collate

Funções de collate: junta batches PINN (`collocation`, `conditions`, `data`, `meta`), move-os para um dispositivo e junta `PhysicalSample` para treino supervisionado. [sem teste]

`pinneapple_data/collate.py` · 132 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`collate_pinn_batches`	função	`(batches) -> Dict[str, Any]`	Concatena uma lista de dicts de batch PINN num único dict (chaves `collocation`, `conditions`, `data`, `meta`).
`move_batch_to_device`	função	`(batch, device) -> Dict[str, Any]`	Move todos os tensores de um batch PINN para o dispositivo dado, sem mexer em `meta`.
`collate_upd_supervised`	função	`(samples) -> Dict[str, Any]`	Empilha `state['x']` e `state['y']` de uma lista de `PhysicalSample` em `{x, y, meta}` para treino supervisionado.

pinneapple_data.collocation

Geração automática de pontos de colocação para PINN a partir de limites, `ProblemSpec`, domínios de `pinneapple_design.geometry`, malhas ou STL, com estratégias `uniform`, `lhs`, `sobol` e `adaptive` (por resíduo).

`pinneapple_data/collocation.py` · 558 linhas · testes: 7

nome	tipo	assinatura	o que faz
`CollocationConfig`	classe	`(n_col, n_bc, n_ic, n_data, strategy, seed, oversample_bc, include_time, t_range)`	Configuração da geração de colocação: estratégia e número de pontos interiores, de contorno e de condição inicial, entre outros.
`CollocationSampler`	classe	`(*, bounds, coord_names, sdf_fn=None, boundary_samplers=None, condition_value_fns=None, c…)`	Gerador único de pontos de colocação: construtores `from_bounds`, `from_problem_spec`, `from_domain`, `from_mesh`, `from_stl`, e `sample`/`sample_adaptive` que devolvem o dict de batch PINN.

pinneapple_data.contracts

Contratos de dados abertos compartilhados pelos apps (`ParameterSpace`, `DatasetSpec`, `ModelRef`, `StudyResult`) e leitura/escrita de tabelas com sidecar `.contract.json`; o que um app exporta outro importa.

`pinneapple_data/contracts/__init__.py` · 51 linhas · testes: 175

pinneapple_data.contracts.expressions

Reexporta a avaliação segura de expressões de `pinneapple_data.expressions` (que só depende de NumPy). [shim] [sem teste]

`pinneapple_data/contracts/expressions.py` · 12 linhas · **sem teste direto**

pinneapple_data.contracts.models

Os quatro contratos como modelos pydantic com `kind` e `schema_version`: ida e volta em JSON/YAML, arquivos, JSON Schema e hash de conteúdo.

`pinneapple_data/contracts/models.py` · 675 linhas · testes: 40

nome	tipo	assinatura	o que faz
`Contract`	classe	`(schema_version, metadata)`	Base dos contratos: conversão para/de dict e JSON, salvar/carregar arquivo, JSON Schema e hash do conteúdo.
`Parameter`	classe	`(name, kind, low, high, values, value, unit, log, group, default, description)`	Um parâmetro de estudo: tipo, limites ou valores, unidade, escala log, grupo e padrão; converte entre coordenada unitária e valor.

nome	tipo	assinatura	o que faz
ParameterSpace	classe	(kind, name, description, parameters, constraints, budget)	Parâmetros de um estudo e restrições entre eles; converte de/para o cubo unitário, testa viabilidade e lê/escreve tabelas.
Column	classe	(name, role, unit, dtype, description)	Coluna de um dataset: nome, papel (entrada, saída, id), unidade, tipo e descrição.
FileRef	classe	(path, format, sha256, size, rows, description)	Referência a um arquivo do dataset com formato, SHA-256, tamanho e número de linhas; of preenche a partir do disco.
Provenance	classe	(app, study_id, source, run_ids, library_version, created, notes)	De onde veio um contrato: app, id do estudo, fonte, ids de execução, versão da biblioteca, data e notas.
Splits	classe	(train, val, test, seed, id_column)	Divisão em splits: ids explícitos por split ou frações com semente.
DatasetSpec	classe	(kind, name, version, description, columns, files, n_rows, parameter_space, splits, prove…)	Tabela de casos ou amostras: colunas, arquivos, splits, espaço de parâmetros amostrado e proveniência; confere os hashes dos arquivos.
Variable	classe	(name, unit, low, high, description)	Variável de entrada ou saída de um modelo com unidade e faixa.
DatasetRef	classe	(name, version, content_hash, uri)	Referência ao dataset de treino de um modelo: nome, versão, hash e URI.
ModelRef	classe	(kind, name, version, model_type, uri, sha256, framework, inputs, outputs, dataset, metri…)	Onde está um modelo treinado e para que pode ser usado: entradas, saídas, dataset, métricas e faixa de validade (in_range).
CheckRecord	classe	(name, status, value, threshold, evidence, detail)	Registro de uma checagem; PASS/FAIL/WARN exigem valor ou evidência, para só contar como rodada se rodou.
Table	classe	(columns, file)	Tabela pequena embutida (colunas de mesmo tamanho) ou referência a arquivo; converte para/de DataFrame.
StudyResult	classe	(kind, id, study_type, app, title, status, parameter_space, dataset, models, table, metri…)	Resultado de um estudo (DOE, campanha, UQ, sensibilidade, otimização, calibração, validação): tabela, métricas, checagens e artefatos.
from_dict	função	(data) -> Contract	Constrói o contrato indicado por data['kind'].
load	função	(path) -> Contract	Lê um contrato de JSON ou YAML, qualquer que seja dos quatro.
validate	função	(data) -> list[str]	Lista os problemas de um contrato dado como dict, uma linha legível por problema (vazia quando válido).
json_schemas	função	() -> dict[str, dict[str, Any]]	Devolve o JSON Schema de cada contrato, por tipo.
export_schemas	função	(directory) -> list[Path]	Grava <Kind>.schema.json de cada contrato no diretório dado.
sha256_file	função	(path, chunk=1 << 20) -> str	Calcula o SHA-256 de um arquivo lendo em blocos.

pinneapple_data.contracts.tables

Tabelas em disco com o contrato: CSV, Parquet e HDF5 com sidecar JSON; Parquet e HDF5 também guardam o contrato nos metadados. [sem teste direto]

pinneapple_data/contracts/tables.py · 150 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
sidecar_path	função	(path) -> Path	Caminho do sidecar <arquivo>.contract.json de uma tabela.
table_format	função	(path) -> str	Deduz o formato da tabela (csv, parquet, hdf5, json) pela extensão.
write_sidecar	função	(path, contract) -> Path	Grava o contrato em <caminho>.contract.json.
read_sidecar	função	(path) -> Contract | None	Lê o contrato do sidecar, ou None se não existe.
write_table	função	(df, path, contract=None) -> Path	Grava um DataFrame (ou dict de colunas) como CSV, Parquet, HDF5 ou JSON, junto com o sidecar do contrato.
read_table	função	(path)	Lê uma tabela e seu contrato, preferindo o sidecar aos metadados embutidos.

nome	tipo	assinatura	o que faz
`convert`	função	`(src, dst) -> Path`	Copia uma tabela para outro formato (pela extensão) levando o contrato junto.

pinneapple_data.data_health

Relatório de saúde de dados de engenharia (sensores, ensaios, planta): lê tabelas em vários formatos e verifica unidades, lacunas, eixo de tempo, valores fisicamente impossíveis, códigos de erro, unidade incoerente e outros problemas, com sugestões.

`pinneapple_data/data_health.py` · 998 linhas · testes: 37

nome	tipo	assinatura	o que faz
`Issue`	classe	`(category, severity, title, detail, column, affected_fraction, count, spans, suggestion, …)`	Um problema encontrado: categoria, severidade, título, detalhe, coluna, fração afetada, trechos, sugestão e evidência.
`LoadInfo`	classe	`(format, rows, columns, delimiter, decimal, encoding, sheet, sheets, units_from_file, tru…)`	Como a tabela foi lida: formato, linhas, colunas, delimitador, separador decimal, codificação, planilha, unidades do arquivo e notas.
`load_table`	função	`(data, filename, sheet=None, max_rows=1000000) -> Tuple[pd.DataFrame, LoadInfo]`	Lê uma tabela enviada (CSV/TSV com detecção de delimitador e vírgula decimal, Excel, Parquet, JSON, HDF5) e devolve o DataFrame e um `LoadInfo`.
`detect_time_column`	função	`(df) -> Tuple[Optional[str], Optional[pd.Series], str]`	Procura a coluna de tempo pelo nome e pelo conteúdo e devolve o nome, a série convertida e uma explicação.
`analyze`	função	`(df, units=None, time_column=None, n_bins=160, series_points=600) -> Dict[str, Any]`	Gera o relatório de saúde de um DataFrame (pronto para JSON): unidades, completude, eixo de tempo, validade física, sinais travados e outros problemas; levanta `ValueError` para tabela vazia.
`example_chiller_plant`	função	`(seed=7, days=7, inject=True) -> Tuple[pd.DataFrame, Dict[str, Any]]`	Gera uma semana de dados de minuto em minuto de uma planta de água gelada com falhas injetadas e devolve a tabela e o gabarito das falhas.

pinneapple_data.dataloaders

Monta `DataLoader` do PyTorch a partir de um shard UPD (Zarr + JSON) ou de um `PhysicalSample` em grade; amostras em malha ainda não são suportadas. [sem teste]

`pinneapple_data/dataloaders.py` · 353 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`DataLoaderSpec`	classe	`(batch_size, num_workers, shuffle, pin_memory, drop_last)`	Parâmetros do `DataLoader`: tamanho de batch, workers, shuffle, pin_memory, entre outros.
`build_upd_dataloader`	função	`(*, zarr_path, meta_path, mapping, sampling, loader=None, device='cpu', dtype=torch.float…)`	Cria um `DataLoader` sobre um shard UPD, envolvendo-o num `Dataset` do Torch conforme o `DataLoaderSpec`.
`build_physical_sample_dataloader`	função	`(sample, *, mapping, sampling, loader=None, device='cpu', dtype=torch.float32, length=100…)`	Cria um `DataLoader` a partir de um `PhysicalSample` em grade (xarray); levanta `ValueError` para casos não suportados, como malha.

pinneapple_data.datasets

Datasets prontos (física, geometria, séries temporais) num registro único, com `load_dataset`, `list_datasets`, `dataset_info` e `dataset_ids`; os módulos `real_world` e `external_sources` são opcionais e carregados em `try/except`.

`pinneapple_data/datasets/__init__.py` · 152 linhas · testes: 10

nome	tipo	assinatura	o que faz
`load_dataset`	função	`(dataset_id, **kwargs) -> Dict[str, np.ndarray]`	Carrega um dataset registrado pelo id e devolve um dict de arrays NumPy; kwargs vão para o gerador.

nome	tipo	assinatura	o que faz
`list_datasets`	função	`(category=None, verbose=True) -> List[DatasetInfo]`	Lista (e imprime, se `verbose`) os datasets registrados, opcionalmente filtrando por categoria.
`dataset_info`	função	`(dataset_id) -> DatasetInfo`	Devolve os metadados (`DatasetInfo`) de um dataset.
`dataset_ids`	função	`(category=None) -> List[str]`	Devolve os ids registrados, opcionalmente por categoria.

`pinneapple_data.datasets.deepmind_mgn`

Datasets `meshgraphnets` da DeepMind (Pfaff et al., ICLR 2021): download parcial de `meta.json` + prefixo do TFRecord, leitura de trajetórias e conversão de `cylinder_flow` para tensores; exige `tfrecord`.

`pinneapple_data/datasets/deepmind_mgn.py` · 116 linhas · testes: 6

nome	tipo	assinatura	o que faz
`download_prefix`	função	`(dataset, split, dest, nbytes) -> Path`	Baixa `meta.json` e os primeiros `nbytes` do TFRecord de um split; como o formato é sequencial, o prefixo contém trajetórias inteiras.
`read_trajectories`	função	`(data_dir, split, max_traj) -> list[dict[str, np.ndarray]]`	Lê até `max_traj` trajetórias do TFRecord como dicts de arrays, com formas vindas de `meta.json`; descarta o último registro truncado.
`cylinder_flow_to_tensors`	função	`(traj, node_type_map=CYLINDER_TYPE_MAP) -> tuple[MeshGraph, torch.Tensor, torch.Tensor]`	Converte uma trajetória `cylinder_flow` em `(MeshGraph, velocidade (T,N,2), pressão (T,N,1))`, agrupando tipos de nó.

`pinneapple_data.datasets.download`

Baixa uma URL uma vez para um cache local, com novas tentativas, só com a biblioteca padrão; escreve em arquivo temporário e renomeia ao terminar.

`pinneapple_data/datasets/download.py` · 85 linhas · testes: 4

nome	tipo	assinatura	o que faz
`cache_path`	função	`(url, cache_dir) -> Path`	Caminho do arquivo de cache de uma URL: `<cache_dir>/<md5(url)><ext>`.
`fetch_cached`	função	`(url, cache_dir, *, tries=4, timeout=60, backoff=2.0) -> Path`	Devolve uma cópia local da URL, baixando só se ainda não estiver no cache.

`pinneapple_data.datasets.external_sources`

Clientes de fontes externas (Materials Project, NOMAD, NASA, NOAA, Copernicus, Brazil Data Cube, CPTEC/INPE, ONS, NREL, DesignSafe, OpenFOAM, SU2, NASA POWER, Open-Meteo) que registram datasets; vários caem em dados sintéticos quando a fonte não está disponível.

`pinneapple_data/datasets/external_sources.py` · 1.168 linhas · testes: 5

nome	tipo	assinatura	o que faz
`MaterialsProjectClient`	classe	`(api_key=None) -> None`	Cliente da API do Materials Project (requer `mp-api` e chave `MP_API_KEY`); sem acesso, devolve dados sintéticos marcados com `_synthetic`.
`NOMADClient`	classe	`()`	Cliente da API REST pública do NOMAD (cálculos ab initio/DFT).
`NASAClient`	classe	`()`	Cliente do portal de dados abertos da NASA: exoplanetas e temperatura GISS.
`NOAAClient`	classe	`()`	Cliente NOAA NCEI/ERDDAP: temperatura da superfície do mar e anomalia de temperatura, com reserva sintética.
`CopernicusClient`	classe	`()`	Cliente Copernicus (CDS): tenta `cdsapi` e, sem ele, gera dados sintéticos tipo ERA5 (`fetch_era5_synthetic`).
`BrazilDataCubeClient`	classe	`()`	Cliente da API STAC do Brazil Data Cube: lista coleções; a série de NDVI é sintética.

nome	tipo	assinatura	o que faz
`CPTECClient`	classe	`()`	Cliente CPTEC/INPE; a série de precipitação devolvida é sintética.
`ONSClient`	classe	`()`	Cliente de dados abertos do ONS; a carga elétrica devolvida é sintética, com sazonalidade diária e semanal.
`NRELClient`	classe	`(api_key=None) -> None`	Cliente NREL para recurso solar e eólico (dados reais exigem `NREL_API_KEY`; sem ela, sintéticos).
`DesignSafeClient`	classe	`()`	Cliente DesignSafe-CI para registros de terremoto/estrutural.
`OpenFOAMClient`	classe	`(openfoam_data_dir=None) -> None`	Devolve dados sintéticos com as curvas típicas de tutoriais OpenFOAM (arrasto de cilindro, perfil de canal, cavidade) ou lê casos reais se `openfoam_data_dir` for dado.
`SU2Client`	classe	`(su2_results_dir=None) -> None`	Devolve dados aerodinâmicos representativos de tutoriais SU2 (polar NACA 0012, Cp do RAE 2822) ou lê resultados reais de `su2_results_dir`.
`NASAPowerClient`	classe	`()`	Cliente da API diária NASA POWER (meteorologia e recurso solar, sem chave); erros de rede são levantados; converte para arrays e `PhysicalSample`.
`OpenMeteoClient`	classe	`()`	Cliente da API histórica Open-Meteo (reanálise ERA5 horária, sem chave); erros de rede são levantados.

`pinneapple_data.datasets.geometry`

Datasets de geometria gerados analiticamente (pontos de contorno e interior): `naca0012`, `cylinder_2d`, `lshape_2d`, `channel_with_obstacle`. [sem teste direto]

`pinneapple_data/datasets/geometry.py` · 269 linhas · **sem teste direto**

`pinneapple_data.datasets.physics`

Datasets de física gerados analiticamente ou por método das linhas com SciPy (Burgers, calor, Poisson, onda, Kovasznay, Allen-Cahn, Helmholtz) e um solver pseudo-espectral de Burgers periódico para pares de operador.

`pinneapple_data/datasets/physics.py` · 450 linhas · testes: 4

nome	tipo	assinatura	o que faz
`burgers_periodic_solve`	função	`(u0, T, nu, *, length=1.0, refine=None, cfl=0.5) -> np.ndarray`	Resolve Burgers viscoso periódico por pseudo-espectral de Fourier com regra 2/3 e Runge-Kutta de 4ª ordem com fator integrante.
`burgers_periodic_pairs`	função	`(n, nx=128, T=1.0, nu=0.01, seed=0, *, length=1.0, refine=None)`	Gera `n` pares aleatórios `(u(x,0), u(x,T))` de Burgers periódico para aprendizado de operadores, com condições iniciais somas de senos.

`pinneapple_data.datasets.real_world`

Datasets reais carregados da web ou de bibliotecas opcionais (sunspots, CO2, ETT, Jena, scikit-learn, seaborn, statsmodels...), com reserva sintética quando a fonte falha; alguns itens são sintéticos por natureza. [sem teste]

`pinneapple_data/datasets/real_world.py` · 1.665 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`load_real_dataset`	função	`(dataset_id) -> Dict[str, np.ndarray]`	Carrega um dataset real pelo id, baixando na primeira chamada e guardando em memória no processo.

`pinneapple_data.datasets.registry`

Registro central (`DatasetRegistry`) dos datasets prontos, com metadados `DatasetInfo`. [sem teste direto]

`pinneapple_data/datasets/registry.py` · 65 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`DatasetInfo`	classe	`(id, name, category, description, fields, tags, license, reference)`	Metadados de um dataset: id, nome, categoria, descrição, campos, tags, licença e referência.
`DatasetRegistry`	classe	`(_store)`	Registro em nível de classe: `register`, `load`, `get_info`, `list_all` e `ids`.

pinneapple_data.datasets.timeseries

Datasets de séries temporais gerados por EDOs ou funções sintéticas (Lorenz-63, massa-mola, Van der Pol, Rössler, pêndulo duplo, Lorenz-96, seno com ruído). [sem teste direto]

`pinneapple_data/datasets/timeseries.py` · 355 linhas · **sem teste direto**

pinneapple_data.device

Fixa em memória (pin) e move entre CPU e GPU todos os tensores de `fields` e `coords` de uma amostra tipo `PhysicalSample` (objeto ou dict). [sem teste]

`pinneapple_data/device.py` · 148 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`pin_sample`	função	`(sample) -> Any`	Fixa em memória paginável (pinned) todos os tensores de `fields` e `coords` de uma amostra.
`to_device_sample`	função	`(sample, device, *, dtype=None, non_blocking=True) -> Any`	Move todos os tensores de `fields` e `coords` de uma amostra para o dispositivo dado.

pinneapple_data.doe

Design de experimentos para campanhas de simulação: desenhos, cobertura, redundância, prioridade e plano de campanha, sobre os contratos `ParameterSpace`/`StudyResult`/`DatasetSpec` e `scipy.stats.qmc`.

`pinneapple_data/doe/__init__.py` · 20 linhas · testes: 21

pinneapple_data.doe.campaign

Planeja uma campanha de simulação: quais casos rodar, em que ordem, e o que acrescentam ao que já rodou; gera `StudyResult` e esqueleto de `DatasetSpec`. [sem teste direto]

`pinneapple_data/doe/campaign.py` · 174 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`CampaignPlan`	classe	`(space, design, cases, unit, coverage_planned, coverage_existing, coverage_combined, redu…)`	Plano de campanha: casos com prioridade, cobertura antes/depois, redundâncias removidas; exporta resumo, checagens, `StudyResult` e esqueleto de `DatasetSpec`.
`plan_campaign`	função	`(space, n=None, method='lhs', *, existing=None, existing_id_column=None, outputs=(), seed…)`	Desenha `n` casos, descarta os que repetem execuções existentes, ordena por prioridade e mede a cobertura do plano, do existente e de ambos.

pinneapple_data.doe.coverage

Mede quão bem um conjunto de casos cobre o espaço de parâmetros (no cubo unitário), quais casos planejados repetem outros e em que ordem rodá-los.

`pinneapple_data/doe/coverage.py` · 119 linhas · testes: 14

nome	tipo	assinatura	o que faz
`coverage`	função	`(unit, names=None, *, n_probe=2 ** 14, seed=0, bins_2d=None, levels=None) -> dict[str, An…)`	Métricas de cobertura de pontos em [0,1]^d: discrepância L2 centrada, distância mínima, distância média ao vizinho e outras.
`redundancy`	função	`(planned_unit, existing_unit=None, *, tol=None, existing_ids=None) -> dict[str, Any]`	Marca casos planejados que quase repetem um caso existente ou outro planejado, com o vizinho mais próximo e a distância.

nome	tipo	assinatura	o que faz
`prioritize`	função	`(planned_unit, existing_unit=None) -> dict[str, Any]`	Ordem de execução por maximin guloso a partir do que já existe, para que qualquer prefixo da lista cubra o espaço de forma espalhada.

pinneapple_data.doe.designs

Desenhos de experimentos sobre um `ParameterSpace`: `lhs`, `maximin`, `sobol`, `halton`, `random`, fatorial completo e fracionário, entre outros, filtrados por limites e restrições. [sem teste direto]

`pinneapple_data/doe/designs.py` · 283 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Design`	classe	`(cases, unit, method, info)`	Casos de um desenho: valores (DataFrame com todos os parâmetros), coordenadas unitárias e como foram gerados.
`design`	função	`(space, n, method='lhs', *, seed=0, existing=None, outputs=(), levels=3, n_restarts=20, f…)`	Gera `n` casos do espaço pelo método escolhido, considerando casos já existentes quando dados.
`fractional_factorial_unit`	função	`(k, n) -> tuple[np.ndarray, dict]`	Desenho fatorial fracionário de dois níveis 2^(k-p) com pelo menos `n` execuções, em coordenadas unitárias.
`greedy_maximin`	função	`(points, n, fixed=None, weights_fixed=None) -> np.ndarray`	Escolhe `n` pontos um a um, cada um o mais distante dos já escolhidos e dos fixos (com pesos opcionais).
`min_distance`	função	`(unit) -> float`	Menor distância entre dois pontos de um conjunto (NaN se houver menos de dois).

pinneapple_data.expressions

Avaliação segura de expressões de restrição (como `"chord - 2 * thickness >= 0.01"`): interpreta a árvore nó a nó, sem `eval`, só com aritmética, comparações, lógica, nomes de colunas e algumas funções matemáticas.

`pinneapple_data/expressions.py` · 131 linhas · testes: 22

nome	tipo	assinatura	o que faz
`ExpressionError`	classe	`()`	Erro para expressão com construção proibida ou nome que não é parâmetro.
`names_in`	função	`(expression) -> set[str]`	Nomes de parâmetros usados numa expressão (sem os nomes de função).
`evaluate`	função	`(expression, values, functions=None) -> np.ndarray`	Avalia a expressão com valores (escalares ou colunas) e devolve um array booleano ou numérico.
`check`	função	`(expression, names, functions=(), strings=False) -> None`	Levanta `ExpressionError` se a expressão sair da gramática permitida, dos nomes dados ou das funções permitidas.
`names_in_tree`	função	`(tree) -> set[str]`	Nomes de variáveis numa árvore AST, excluindo os nomes chamados como função.

pinneapple_data.formfill

Preenche formulários de engenharia a partir dos documentos que têm os dados: lê datasheets e especificações, acha cada item com a fonte, relata conflitos e lacunas e preenche o PDF; modelo embutido ASME U-DR-1 e outros datasheets.

`pinneapple_data/formfill/__init__.py` · 28 linhas · testes: 44

pinneapple_data.formfill.extract

Lê documentos (texto e tabelas via pdfplumber, OCR opcional), extrai candidatos pelos padrões do `FormSpec` e compila uma decisão por item, guardando documento, página e trecho de origem. [sem teste direto]

`pinneapple_data/formfill/extract.py` · 698 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Document`	classe	`(name, pages, tables, role, has_text, ocr_pages, ocr_conf, unread_pages)`	Documento lido: nome, texto por página, tabelas, papel, páginas por OCR e confiança, páginas não lidas.
`Candidate`	classe	`(key, value, raw, doc, page, snippet, method, confidence, unit, si, gauge, ocr)`	Valor candidato para um item: valor, texto bruto, documento, página, trecho, método, confiança, unidade e valor em SI.
`Decision`	classe	`(key, label, section, status, value, display, chosen, candidates, required, note)`	Decisão para um item: status, valor escolhido, candidatos, se é obrigatório e nota.
`Compilation`	classe	`(decisions, tables, documents, spec)`	Resultado da compilação: decisões, tabelas, documentos; dá lacunas, conflitos, respostas, resumo e o registro para outros formulários.
`read_document`	função	`(source, name='', role='', ocr='auto', max_ocr_pages=30) -> Document`	Lê um PDF (caminho ou bytes) ou texto puro; com `ocr='auto'` lê páginas escaneadas com Tesseract quando disponível.
`parse_value`	função	`(f, text) -> Optional[Dict[str, Any]]`	Interpreta um texto como resposta a um campo (quantidade, número, booleano, escolha...) ou devolve `None`.
`extract_rules`	função	`(doc, spec) -> List[Candidate]`	Gera candidatos para cada item do spec cujo rótulo aparece no documento (tabelas primeiro, depois linhas de texto), mais as linhas das tabelas do spec.
`check_answer`	função	`(f, value) -> Any`	Valida a resposta de um engenheiro para um campo; `ValueError` cita as opções aceitas.
`compile`	função	`(docs, candidates, spec, overrides=None) -> Compilation`	Uma decisão por item: documentos anteriores na lista vencem conflitos (que ainda são relatados) e respostas do engenheiro vencem tudo.

`pinneapple_data.formfill.fill`

Escreve uma compilação na cópia do usuário de um PDF preenchível (via `pypdf`); itens em `spec.left_blank` (data, assinatura) nunca são preenchidos. [sem teste direto]

`pinneapple_data/formfill/fill.py` · 177 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`pdf_widgets`	função	`(pdf) -> List[Dict[str, Any]]`	Lista os widgets de formulário de um PDF: nome qualificado, tipo, página, tooltip, estados de botão e grupo de rádio.
`form_values`	função	`(comp, spec=None) -> Tuple[Dict[str, str], Dict[str, str]]`	Calcula `(campos de texto, estados de botão)` para o PDF a partir das decisões e linhas de tabela.
`fill_pdf`	função	`(blank_pdf, comp, spec=None, extra_text=None) -> bytes`	Devolve o formulário preenchido como bytes de PDF.
`read_filled`	função	`(pdf) -> Dict[str, Any]`	Lê nome do campo → valor de um formulário preenchido, para checagem e ida e volta.

`pinneapple_data.formfill.llm`

Extração opcional com um LLM local servido pelo Ollama: só aceita respostas cuja citação existe no texto do documento e contém o valor; o resto é rejeitado e listado. [sem teste direto]

`pinneapple_data/formfill/llm.py` · 230 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`OllamaError`	classe	`()`	Erro de comunicação ou resposta inválida do servidor Ollama.
`OllamaClient`	classe	`(url=None, model=None, timeout=600.0, num_ctx=16384)`	Cliente mínimo da API HTTP do Ollama (`/api/chat`, `/api/tags`), com chat em saída JSON estruturada.
`document_text`	função	`(doc, pages=None) -> str`	Texto do documento com marcação de página e tabelas como linhas `célula | célula`.
`verify_quote`	função	`(doc, quote, page) -> Optional[int]`	Página onde a citação aparece no texto do documento (preferindo a página declarada), ou `None`.
`extract_with_llm`	função	`(doc, spec, client=None, max_chars=24000) -> Tuple[List[Candidate], List[Dict[str, Any]]]`	Extrai candidatos de um documento com o LLM e devolve os aceitos e as respostas rejeitadas com o motivo.

pinneapple_data.formfill.ocr

OCR de páginas escaneadas com Tesseract: renderiza, corrige inclinação, apaga linhas de grade de tabela antes da leitura e recoloca cada palavra na célula; exige o programa `tesseract`. [sem teste direto]

`pinneapple_data/formfill/ocr.py` · 241 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`OcrPage`	classe	`(lines, tables, confidence, word_conf, angle)`	Página lida por OCR: linhas de texto, tabelas, confiança média e por palavra e ângulo de correção.
`ocr_available`	função	`() -> bool`	Diz se o executável do Tesseract está disponível.
`ocr_image`	função	`(img) -> OcrPage`	Faz OCR de uma imagem de página (PIL).
`ocr_pdf_page`	função	`(pdf_bytes, index, dpi=DPI) -> OcrPage`	Faz OCR da página `index` (base 0) de um PDF.

pinneapple_data.formfill.pipeline

Uma chamada dos documentos ao formulário compilado (regras + LLM opcional) e utilitários de front-end: validar respostas do usuário e nome do arquivo de saída. [sem teste direto]

`pinneapple_data/formfill/pipeline.py` · 66 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`compile_documents`	função	`(docs, spec, client=None, overrides=None, max_ocr_pages=None) -> Tuple[Compilation, List[…]`	Lê cada documento `(nome, dados, papel)`, extrai candidatos por regras (e LLM, se houver) e compila; falha do LLM não derruba a chamada.
`check_overrides`	função	`(spec, raw) -> Dict[str, Any]`	Valida as respostas do usuário contra o spec; chave desconhecida ou valor inválido levanta `ValueError`.
`output_filename`	função	`(comp, prefix='', day=None) -> str`	Nome seguro do arquivo preenchido: `<spec id>_<valor da chave>_<AAAAMMDD>.pdf`.

pinneapple_data.formfill.report

Gera o datasheet compilado em PDF com `reportlab` (itens com valor e fonte, pendências primeiro), marcado como rascunho para revisão de engenharia. [sem teste direto]

`pinneapple_data/formfill/report.py` · 166 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`render_datasheet`	função	`(comp, spec=None, title=None) -> bytes`	Renderiza uma `Compilation` como PDF de datasheet; exige `reportlab`.

pinneapple_data.formfill.spec

Define o que um formulário pede (`FormSpec`, `Field`, `TableSpec`): rótulos, padrões de texto, tipo de campo, campo do PDF; specs vêm de modelos embutidos, JSON ou de qualquer PDF preenchível. [sem teste direto]

`pinneapple_data/formfill/spec.py` · 259 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Field`	classe	`(key, label, section, kind, pdf, states, patterns, quantity, required, source_hint, optio…)`	Um item do formulário: chave, rótulo, seção, tipo (`text`, `quantity`, `number`, `bool`, `choice`, `multi`), campo do PDF, padrões e opções.
`TableSpec`	classe	`(key, label, columns, header, combine, defaults, pdf_rows, description)`	Item de linhas repetidas (lista de bocais, lista de linhas) achado como tabela cujo cabeçalho casa com os padrões e preenchido linha a linha no PDF.
`FormSpec`	classe	`(id, title, fields, sections, tables, description, notes_field, left_blank, filename_key,…)`	Especificação completa de um formulário: campos, seções, tabelas e metadados; salva/carrega JSON e pode ser gerada de um PDF preenchível (`from_pdf`).

pinneapple_data.formfill.specs

Registro dos modelos de formulário embutidos (`asme_u-dr-1`, `psv`, `shell_tube`, `tank`, `pump`), carregados sob demanda. [sem teste direto]

`pinneapple_data/formfill/specs/__init__.py` · 41 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`get_spec`	função	`(spec_id) -> FormSpec`	Devolve o `FormSpec` de um modelo embutido pelo id; `KeyError` lista os ids conhecidos.
`list_specs`	função	`() -> List[Dict[str, object]]`	Lista os modelos embutidos com título, norma, resumo, entradas, saída, número de itens, obrigatórios e tabelas.

pinneapple_data.formfill.specs.asme_udr1

Modelo embutido do ASME BPVC Seção VIII Div. 1 Form U-DR-1: campos, padrões de texto e mapeamento para os campos do PDF preenchível (revisão 07/25). [sem teste direto]

`pinneapple_data/formfill/specs/asme_udr1.py` · 326 linhas · **sem teste direto**

pinneapple_data.formfill.specs.psv

Modelo embutido de datasheet de válvula de alívio de pressão, com os dados pedidos por API 520 Parte I e API 526; a saída é um datasheet gerado pela ferramenta. [sem teste direto]

`pinneapple_data/formfill/specs/psv.py` · 115 linhas · **sem teste direto**

pinneapple_data.formfill.specs.pump

Modelo embutido de datasheet de bomba centrífuga com os dados que a API 610 pede ao comprador. [sem teste direto]

`pinneapple_data/formfill/specs/pump.py` · 110 linhas · **sem teste direto**

pinneapple_data.formfill.specs.shell_tube

Modelo embutido de datasheet de trocador casco-tubo com os dados de uma folha TEMA e da API 660, lendo colunas de lado casco e lado tubo. [sem teste direto]

`pinneapple_data/formfill/specs/shell_tube.py` · 110 linhas · **sem teste direto**

pinneapple_data.formfill.specs.tank

Modelo embutido de datasheet de tanque de armazenamento atmosférico com os dados que a API 650 pede (Anexo L). [sem teste direto]

`pinneapple_data/formfill/specs/tank.py` · 117 linhas · **sem teste direto**

pinneapple_data.geometry_features

Features de geometria a partir de malha triangular: nuvem de pontos na superfície e grade grosseira de distância assinada (SDF), via `trimesh`. [sem teste]

`pinneapple_data/geometry_features.py` · 86 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`sample_point_cloud`	função	`(vertices, faces, n_samples=2048) -> np.ndarray`	Amostra pontos uniformes (ponderados por área) na superfície da malha, limitando o número em malhas muito grosseiras.
`compute_sdf_grid`	função	`(vertices, faces, grid_resolution=16, bounds=None) -> Dict[str, Any]`	Avalia a distância assinada numa grade regular sobre a caixa da malha (ou limites dados).

nome	tipo	assinatura	o que faz
`featurize_mesh`	função	`(vertices, faces, n_point_cloud_samples=2048, sdf_grid_resolution=16) -> Dict[str, Any]`	Nuvem de pontos + grade SDF numa chamada, relatando erros por etapa em vez de levantar.

pinneapple_data.grf_sampler

Amostrador de campos aleatórios gaussianos (GRF) para funções de entrada de benchmarks de operador (como no DeepONet): 1D com uma única fatoração de Cholesky por lote e spline cúbica, 2D por síntese espectral com FFT. [sem teste]

`pinneapple_data/grf_sampler.py` · 177 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`GRFDraw`	classe	`(x_grid, u_grid, spline)`	Uma função amostrada u(x) em [x_min, x_max], consultável em qualquer ponto; `sensors` avalia nos sensores.
`GRFDraw2D`	classe	`(x_grid, y_grid, u_grid, interp)`	Um campo 2D amostrado u(x,y) em [0,1]×[0,1]; `sensors` avalia nos pontos dados.
`sample_grf_batch`	função	`(n_draws, x_min=0.0, x_max=1.0, n_grid=1000, length_scale=0.2, jitter=1e-10, seed=None) -…)`	Sorteia vários caminhos GRF 1D de média zero com covariância RBF, compartilhando uma fatoração de Cholesky numa grade fina.
`sample_grf_paths`	função	`(n_draws, x_min=0.0, x_max=1.0, n_grid=1000, length_scale=0.2, jitter=1e-10, seed=None) -…)`	Apelido de `sample_grf_batch`.
`sample_grf_2d_batch`	função	`(n_draws, nx=64, ny=64, length_scale=0.2, seed=None) -> List[GRFDraw2D]`	Sorteia vários campos GRF 2D de média zero e variância unitária por síntese espectral com FFT.

pinneapple_data.lineage

Linhagem de modelos de engenharia (digital thread): artefatos e derivações, checagens, perguntas, exportação W3C PROV e detecção automática a partir dos arquivos de um projeto.

`pinneapple_data/lineage/__init__.py` · 7 linhas · testes: 13

pinneapple_data.lineage.detect

Detecta a linhagem de um projeto pelos arquivos: casos OpenFOAM, Gmsh `.geo→.msh`, decks e resultados CalculiX, datasets exportados (ligados por sha256), relatórios e model cards, mais um `lineage.json` declarado. [sem teste direto]

`pinneapple_data/lineage/detect.py` · 304 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`detect_project`	função	`(fs, project='') -> Lineage`	Monta um `Lineage` a partir de um dict caminho → bytes, ligando artefatos pelos arquivos e hashes.

pinneapple_data.lineage.graph

Grafo de linhagem: artefatos (CAD, malha, simulação, dataset, modelo, predição...) ligados por arestas 'derivado de', com checagens (referências quebradas, ciclos, arquivos alterados, derivações velhas) e exportação PROV-JSON e Markdown. [sem teste direto]

`pinneapple_data/lineage/graph.py` · 365 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Artifact`	classe	`(id, kind, name, file, version, software, parameters, timestamp, hash, owner, origin, inp…)`	Um artefato da linhagem: id, tipo, arquivo, versão, software, parâmetros, hash, dono, entradas; mede completude e verifica o hash.
`Lineage`	classe	`(artifacts=(), project='')`	Grafo de artefatos: adiciona, lê/escreve JSON, navega a montante e a jusante, roda checagens e exporta relatório, PROV e Markdown.

nome	tipo	assinatura	o que faz
sha256	função	(b) -> str	SHA-256 em hexadecimal de bytes.

pinneapple_data.noether_dataset

Ponte de dados entre o framework Noether e o PINNeAPPle: usa datasets do Noether (como DrivAerML) como Dataset do PyTorch e converte listas de PhysicalSample para o formato {campo: tensor} do Noether.

pinneapple_data/noether_dataset.py · 216 linhas · testes: 5

nome	tipo	assinatura	o que faz
NoetherDatasetBridge	classe	(noether_dataset, input_fields=None, target_fields=None, transform=None, device=None) -> …)	Envolve um dataset do Noether (ou qualquer dataset que devolva {campo: tensor}) como torch.utils.data.Dataset comum, sem Hydra.
PhysicalSampleDataset	classe	(samples, input_fields=None, output_fields=None, include_coords=False, transform=None) ->…)	Dataset PyTorch em que cada PhysicalSample vira um item {campo: tensor}, com campos e coordenadas expostos.
split_physical_samples	função	(samples, val_fraction=0.1, seed=42, **dataset_kwargs)	Divide uma lista de PhysicalSample em datasets de treino e validação com fração e semente.

pinneapple_data.parameter_sampling

Amostragem de espaços de parâmetros (constantes físicas, magnitudes de contorno, dimensões) por LHS, Sobol, Halton ou aleatória via scipy.stats.qmc, e conjunto tipo Saltelli para sensibilidade de Sobol sem SALib.

pinneapple_data/parameter_sampling.py · 118 linhas · testes: 5

nome	tipo	assinatura	o que faz
sample_parameters	função	(param_ranges, n_samples, method='lhs', seed=0) -> list[dict[str, float]]	Sorteia n_samples pontos de um espaço {nome: (min, max)} pelo método escolhido (lhs por padrão) e devolve uma lista de dicts.
saltelli_perturbation_sweep	função	(nominal, param_ranges, n_base=8, seed=0) -> list[dict[str, float]]	Monta o conjunto de amostras no estilo Saltelli (matrizes A, B e trocas de coluna) para índices de Sobol de primeira ordem e total.

pinneapple_data.physical_sample

Dataclass PhysicalSample, o contêiner padrão (alinhado ao UPD) de dados físicos: estado (xarray ou dict), geometria, esquema, domínio, proveniência e extras, com ponte para dict de treino.

pinneapple_data/physical_sample.py · 241 linhas · testes: 28

nome	tipo	assinatura	o que faz
PhysicalSample	classe	(state, geometry, schema, domain, provenance, extras)	Amostra física unificada: estado, geometria, esquema de equações/condições, domínio e proveniência; diz o tipo de domínio, lista variáveis, resume e converte para dict de treino (to_train_dict).

pinneapple_data.physical_units

Unidades e grandezas físicas só com a biblioteca padrão: interpreta unidades de cabeçalhos, converte para SI, infere a grandeza de uma coluna e guarda limites físicos rígidos e típicos por grandeza.

pinneapple_data/physical_units.py · 436 linhas · testes: 52

nome	tipo	assinatura	o que faz
Unit	classe	(symbol, factor, dim, offset, gauge)	Uma unidade com fator e deslocamento para SI, vetor de dimensão e indicação de pressão manométrica; converte para/de SI.
UnitError	classe	()	Erro de unidade desconhecida ou mal formada.

nome	tipo	assinatura	o que faz
`Quantity`	classe	`(name, si_unit, dim, keywords, hard, typical, display)`	Uma grandeza física: nome, unidade SI, dimensão, palavras-chave, limites rígidos, faixa típica e unidade de exibição.
`QuantityGuess`	classe	`(quantity, confidence, reason, alternatives, is_difference)`	Palpite de grandeza para uma coluna: grandeza, confiança, motivo, alternativas e se é diferença.
`parse_unit`	função	`(text) -> Unit`	Interpreta uma expressão de unidade (`°C`, `kg/m^3`, `W/(m·K)`, `psig`…) como `Unit`; levanta `UnitError` para token desconhecido.
`try_parse_unit`	função	`(text) -> Optional[Unit]`	Como `parse_unit`, mas devolve `None` em vez de levantar erro.
`convert`	função	`(values, src, dst, difference=False)`	Converte valores (escalar ou array) de uma unidade para outra, com opção para diferenças (sem deslocamento).
`dim_str`	função	`(dim) -> str`	Texto legível de um vetor de dimensão (por exemplo `kg·m^-3`, ou `dimensionless`).
`split_header`	função	`(header) -> Tuple[str, Optional[str]]`	Separa o cabeçalho de coluna em nome e texto de unidade (`[°C]`, `(bar)`, , `m3/h`, `in kW`, sufixos como `_degC`).
`infer_quantity`	função	`(name, unit=None) -> QuantityGuess`	Infere a grandeza física de uma coluna combinando a dimensão da unidade com palavras-chave do nome.
`describe_unit`	função	`(u) -> Dict[str, object]`	Descrição em dict de uma unidade: símbolo, fator e deslocamento SI, dimensão e se é manométrica.
`si_unit_for`	função	`(quantity) -> str`	Unidade SI de uma grandeza registrada em `QUANTITIES`.

pinneapple_data.physics_case

`PhysicsCase`: ponte mínima que referencia juntos geometria, `ProblemSpec`, escolha de solver e resultados de um caso físico, e compara resultados com um benchmark sem emitir veredito.

`pinneapple_data/physics_case.py` · 405 linhas · testes: 47

nome	tipo	assinatura	o que faz
`BenchmarkComparison`	classe	`(benchmark, x_vars, y_vars, n_reference_points, n_compared_points, rmse, relative_l2_erro…)`	Comparação numérica (RMSE, erro relativo etc.) produzida por `validate_against_benchmark`; o critério de aprovação fica em outro pacote.
`PhysicsCase`	classe	`(name, geometry, geometry_kind, physics, solver, solver_config, results, reference_benchm…)`	Um caso físico com campos opcionais (geometria, spec, solver, resultados, benchmark); valida contra benchmark e converte para/de dict.

pinneapple_data.prefetch_adaptive

`IterableDataset` que lê amostras de um store Zarr UPD com cache e prefetch adaptativo: uma thread produtora ajusta quantas amostras mantém na fila. [sem teste]

`pinneapple_data/prefetch_adaptive.py` · 281 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`AdaptivePrefetchConfig`	classe	`(queue_max, min_target_fill, max_target_fill, target_fill_init, control_period_s, increas…)`	Configuração do prefetch adaptativo: capacidade máxima da fila e faixa e passos do alvo de preenchimento.
`AdaptivePrefetchZarrUPDIterable`	classe	`(root, *, fields=None, coords=None, dtype=None, start=0, end=None, stride=1, sample_ctor=…)`	Itera amostras de um store Zarr UPD em cache com uma fila limitada abastecida por thread que ajusta o alvo conforme o consumo.

pinneapple_data.preflight

Pré-voo de simulação: checa um caso OpenFOAM ou um deck CalculiX/Abaqus antes de gastar horas de solver e devolve status PASS/WARNING/FAIL com achados que apontam arquivo e linha.

`pinneapple_data/preflight/__init__.py` · 74 linhas · testes: 13

nome	tipo	assinatura	o que faz
`preflight`	função	`(fs) -> Dict[str, Any]`	Detecta se o upload é caso OpenFOAM ou deck `.inp` e roda as regras correspondentes, devolvendo o relatório.
`is_result_file`	função	`(path) -> bool`	Diz se um arquivo é resultado de solver (diretório de tempo diferente de `0`, resultado CalculiX ou VTK) e não entrada.
`drop_results`	função	`(fs) -> Dict[str, bytes]`	Remove do upload os arquivos de resultado de solver.
`attach_snippets`	função	`(report, fs, context=4, width=160) -> None`	Acrescenta a cada achado com arquivo e linha as linhas de código ao redor (`snippet`).

`pinneapple_data.preflight.calculix`

Regras de pré-voo para decks CalculiX/Abaqus planos: malha, seções e materiais, propriedades por procedimento, consistência de unidades, apoios e cargas, condições iniciais e passos; regras FAIL confirmadas rodando CalculiX em decks quebrados. [sem teste direto]

`pinneapple_data/preflight/calculix.py` · 450 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`check`	função	`(fs, main) -> Report`	Lê o deck principal (com includes) e aplica todas as regras CalculiX, devolvendo um `Report`.

`pinneapple_data.preflight.common`

Tipos `Finding` e `Report` e utilitários de texto compartilhados pelas regras OpenFOAM e CalculiX. [sem teste direto]

`pinneapple_data/preflight/common.py` · 83 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Finding`	classe	`(section, status, title, detail, fix, file, line, entity, rule, evidence)`	Um achado do pré-voo: seção, status, título, detalhe, correção sugerida, arquivo, linha, entidade, regra e evidência.
`Report`	classe	`(solver, findings, info)`	Relatório de pré-voo: solver, lista de achados e informações; `add` acrescenta e `to_dict` calcula o status geral.
`line_of`	função	`(text, pattern, flags=re.M) -> Optional[int]`	Número da linha onde um padrão regex aparece no texto, ou `None`.
`num`	função	`(v) -> Optional[float]`	Último número de uma entrada OpenFOAM (`'nu [0 2 -1 0 0 0 0] 1e-05'` → `1e-05`).
`vec`	função	`(v) -> Optional[List[float]]`	Lê um vetor entre parênteses de uma entrada OpenFOAM como lista de floats, ou `None`.

`pinneapple_data.preflight.openfoam`

Regras de pré-voo para casos OpenFOAM: malha, propriedades de transporte/termo, condições de contorno e iniciais, esquemas, solvers lineares e passo de tempo; cada regra cita o que o OpenFOAM faz quando violada. [sem teste direto]

`pinneapple_data/preflight/openfoam.py` · 551 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`check`	função	`(fs, root='') -> Report`	Lê os dicionários do caso e aplica as regras do solver indicado no `controlDict` (ou só regras genéricas se o solver não está no conjunto).

`pinneapple_data.process_optimizer`

Otimização de processo industrial só com dados (sem modelo físico): limpa o histórico com o relatório de saúde, treina um `HistGradientBoostingRegressor` do scikit-learn que prevê o KPI a partir das alavancas e do contexto, valida em divisão temporal e sugere ajustes das alavancas.

`pinneapple_data/process_optimizer.py` · 436 linhas · testes: 16

nome	tipo	assinatura	o que faz
`suggest_roles`	função	`(columns) -> Dict[str, Any]`	Sugere pelos nomes das colunas qual é o alvo, quais são alavancas e quais são contexto (para revisão humana).
`clean_for_modeling`	função	`(df, report) -> Tuple[pd.DataFrame, List[str]]`	Tabela numérica ordenada no tempo com os problemas apontados pelo relatório de saúde marcados como ausentes.
`time_series_cv`	função	`(X, y, n_folds=4, l2_grid=L2_GRID, seed=0) -> Dict[str, Any]`	Validação cruzada encadeada no tempo (cada dobra treina no passado e testa no bloco seguinte) para escolher a regularização L2.
`fit_and_optimize`	função	`(data, target, levers, context, goal='minimize', constraints=(), bounds=None, max_move=0.…)`	Treina na parte antiga do histórico, avalia e otimiza as alavancas na parte mais recente e devolve modelo, métricas e recomendações.
`plant_physics`	função	`(oat, rh, load, chws_sp, cws_sp, pump_pct)`	Comportamento 'verdadeiro' da planta do exemplo, oculto do modelo de ML.
`example_plant_operations`	função	`(seed=11, days=60) -> Tuple[pd.DataFrame, Dict[str, Any]]`	Gera dados de exemplo de uma planta de água gelada operada com hábitos manuais de setpoint.
`true_saving`	função	`(result) -> Dict[str, Any]`	No exemplo, mede a economia real das recomendações usando a física verdadeira da planta.

`pinneapple_data.serialization`

Grava e lê listas de `PhysicalSample` em `.pt` (com gzip opcional), HDF5 e Zarr (via `UPDZarrStore`), compatível com o formato UPD novo e o legado (`fields`/`coords`/`meta`).

`pinneapple_data/serialization.py` · 338 linhas · testes: 11

nome	tipo	assinatura	o que faz
`save_manifest`	função	`(path, manifest) -> None`	Grava um manifesto JSON, criando o diretório se preciso.
`load_manifest`	função	`(path) -> Dict[str, Any]`	Lê um manifesto JSON.
`save_pt`	função	`(samples, path, *, compress=False) -> None`	Grava amostras num arquivo PyTorch `.pt`, opcionalmente comprimido com gzip.
`load_pt`	função	`(path) -> List[PhysicalSample]`	Lê amostras de um `.pt`, detectando sozinho se está comprimido.
`save_hdf5`	função	`(samples, path) -> None`	Grava amostras em HDF5 como blobs serializados pelo torch, com atributos de manifesto (requer `h5py`); o nome aparece definido duas vezes no módulo e vale a segunda definição.
`save_hdf5`	função	`(samples, path) -> None`	Grava amostras em HDF5 como blobs serializados pelo torch, com atributos de manifesto (requer `h5py`); o nome aparece definido duas vezes no módulo e vale a segunda definição.
`load_hdf5`	função	`(path) -> List[PhysicalSample]`	Lê amostras de um HDF5 gravado por `save_hdf5` (requer `h5py`).
`save_zarr`	função	`(samples, root, *, compressor='default') -> None`	Grava amostras num store Zarr de diretório via `UPDZarrStore` (versão corrigida; antes usava uma classe inexistente).
`load_zarr`	função	`(root)`	Lê amostras de um store Zarr via `UPDZarrStore`.

`pinneapple_data.simulation_metadata`

Extrai metadados de simulação (solver, malha, passo de tempo, condições de contorno, parâmetros e convergência) dos arquivos de um caso OpenFOAM, deck CalculiX/Abaqus ou tabela de resíduos.

`pinneapple_data/simulation_metadata/__init__.py` · 346 linhas · testes: 25

nome	tipo	assinatura	o que faz
`read_upload`	função	`(files) -> dict[str, bytes]`	Transforma arquivos enviados (zips expandidos com limites) em dict caminho relativo → bytes.
`parse_residual_table`	função	`(text) -> dict[str, Any] | None`	Lê uma tabela genérica de resíduos (exportada por outros solvers) achando o cabeçalho com a coluna de iteração; `None` se não reconhece.
`extract`	função	`(fs) -> dict[str, Any]`	Monta o registro de metadados a partir do upload, detectando OpenFOAM, `.inp` ou tabela de resíduos; levanta `ValueError` sem arquivos.

pinneapple_data.simulation_metadata.calculix

Lê decks CalculiX/Abaqus (`.inp`) e arquivos de execução CalculiX (`.sta`, `.cvg`, `.dat`, log) e avalia a convergência. [sem teste direto]

`pinneapple_data/simulation_metadata/calculix.py` · 323 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`parse_inp`	função	`(text, include=None) -> Dict[str, Any]`	Interpreta um deck em modelo, passos, materiais, condições de contorno e cargas, resolvendo `*INCLUDE` enviados.
`parse_sta`	função	`(text) -> List[Dict[str, Any]]`	Lê o arquivo `.sta` (incrementos por passo, tentativas, iterações, tempos).
`parse_cvg`	função	`(text) -> List[Dict[str, Any]]`	Lê o arquivo `.cvg` (iterações com elementos de contato e resíduos de força).
`parse_dat`	função	`(text) -> Dict[str, Any]`	Do `.dat`, maior deslocamento impresso (ou mín./máx. de outros resultados) no último tempo de cada conjunto.
`parse_ccx_log`	função	`(text) -> Dict[str, Any]`	Do log do CalculiX: versão, se terminou, erros, número de avisos, tempo total e threads.
`assess`	função	`(sta, cvg, deck, log) -> Dict[str, Any]`	Dá o status de uma execução CalculiX (`failed`, `completed`, `incomplete`...) com motivos e recomendações.

pinneapple_data.simulation_metadata.convergence

Avalia convergência a partir de históricos de resíduos e devolve status (`converged`, `still_falling`, `stalled`, `diverged`, `crashed`...), motivos e o que fazer. [sem teste direto]

`pinneapple_data/simulation_metadata/convergence.py` · 228 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`downsample`	função	`(x, series, k=1500) -> Dict[str, Any]`	Reduz séries de resíduos a no máximo `k` pontos (sempre incluindo o último) para exibição.
`assess_steady`	função	`(log, targets) -> Dict[str, Any]`	Avalia uma execução em regime permanente pelos resíduos contra as metas do `fvSolution`.
`assess_transient`	função	`(log, tolerances, max_co, algorithm, reached_end) -> Dict[str, Any]`	Avalia uma execução transiente: se chegou ao fim, falhas, Courant máximo e tolerâncias dos solvers lineares.
`assess_table`	função	`(iters, residuals, is_log10) -> Dict[str, Any]`	Avalia um histórico genérico de resíduos (exportações de Fluent, STAR-CCM+, SU2).

pinneapple_data.simulation_metadata.foam_dict

Parser tolerante de dicionários OpenFOAM (`controlDict`, `fvSchemes`, `fvSolution`, `0/U`...) em dicts aninhados; diretivas e macros são registradas, nunca executadas.

`pinneapple_data/simulation_metadata/foam_dict.py` · 176 linhas · testes: 16

nome	tipo	assinatura	o que faz
`parse`	função	`(text, max_list=2000) -> Dict[str, Any]`	Interpreta um arquivo FoamFile em dicts aninhados, resumindo listas longas como `<list of N>`.
`header`	função	`(text) -> Dict[str, str]`	Lê só o cabeçalho FoamFile (classe, objeto, formato, nota).
`scalar`	função	`(v) -> Optional[float]`	Último número de uma entrada (`'uniform 0'` → `0.0`).
`vector`	função	`(v) -> Optional[List[float]]`	Lê um vetor de uma entrada (`'uniform (10 0 0)'` → `[10, 0, 0]`).
`words`	função	`(v) -> List[str]`	Separa uma entrada em palavras (lista vazia para `None` ou dict).
`boundary`	função	`(text) -> List[Dict[str, Any]]`	Lê os patches de um arquivo `polyMesh/boundary`: nome, tipo, nFaces, startFace e grupos.

pinneapple_data.simulation_metadata.openfoam

Extração de caso e log OpenFOAM para o registro de metadados: log do solver, raiz do caso, tabelas de `postProcessing` e resumo por seções.

`pinneapple_data/simulation_metadata/openfoam.py` · 523 linhas · testes: 9

nome	tipo	assinatura	o que faz
`parse_log`	função	`(text) -> dict[str, Any]`	Interpreta um log de solver OpenFOAM (ou de blockMesh/checkMesh): passos, resíduos, Courant, erros.
`find_root`	função	`(fs) -> str | None`	Acha a raiz do caso: o diretório com `system/controlDict` de caminho mais curto, ou `None`.
`parse_dat`	função	`(text) -> tuple[list[str], np.ndarray]`	Interpreta uma tabela de function object (`forceCoeffs.dat`, `probes`…) em nomes de coluna e array, expandindo vetores entre parênteses.
`read_dat`	função	`(path) -> tuple[list[str], np.ndarray]`	Aplica `parse_dat` ao arquivo do caminho dado.
`extract_case`	função	`(fs, root) -> dict[str, Any]`	Lê os arquivos do caso (controlDict, esquemas, solução, propriedades, malha, campos iniciais, logs) num registro bruto.
`summarise`	função	`(case) -> dict[str, Any]`	Transforma a extração bruta do caso nas seções do registro de metadados.

pinneapple_data.snapshots

Matrizes de snapshots para modelos de ordem reduzida: vários resultados de um modelo (tempos ou pontos de parâmetro) na mesma malha empilhados em `X`, lidos de caso OpenFOAM, série de arquivos de malha ou arrays.

`pinneapple_data/snapshots.py` · 413 linhas · testes: 31

nome	tipo	assinatura	o que faz
`SnapshotSet`	classe	`(X, fields, times, params, param_names, locations, source, meta, cells, snapshot_points)`	`X` (n_snapshots, n_values) com o significado de linhas e colunas: tempos, parâmetros, campos, malha; seleciona, resume e exporta `.npz`.
`snapshots_from_arrays`	função	`(arrays, *, times=None, params=None, param_names=None, locations=None, where=None, source…)`	Monta um `SnapshotSet` a partir de arrays de campos com os snapshots no primeiro eixo, opcionalmente com malha.
`load_snapshots`	função	`(files, *, fields=None, param_columns=None) -> SnapshotSet`	Lê um upload (lista de `(nome, bytes)`) num `SnapshotSet`, conforme os formatos do docstring do módulo.

pinneapple_data.splits

Estratégias de divisão treino/validação/teste (aleatória, temporal, por grupo); é a fonte única, `pinneapple_train.splits` reexporta daqui. [sem teste direto]

`pinneapple_data/splits.py` · 104 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`SplitSpec`	classe	`(method, train, val, test, seed)`	Estratégia de split: `random` (embaralha), `time` (mantém a ordem, teste no fim) ou `group` (sem vazamento entre grupos), com razões e semente.
`split_indices`	função	`(n, spec, *, group_ids=None) -> Dict[str, np.ndarray]`	Devolve os índices de cada split para `n` amostras segundo o `SplitSpec` (exige `group_ids` na estratégia por grupo).

pinneapple_data.stl_import

Leitor de STL ASCII e binário sem dependências, com deduplicação de vértices, para ingestão em servidor. [sem teste direto]

`pinneapple_data/stl_import.py` · 93 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`STLMesh`	classe	`(verts, faces)`	Malha STL lida: vértices `(V,3)` float32 e faces `(F,3)` int64.
`load_stl_bytes`	função	`(data, *, dedup_eps=1e-06) -> STLMesh`	Lê STL (binário ou ASCII) de bytes, deduplicando vértices com tolerância `dedup_eps`; levanta `ValueError` para dados curtos demais.
`load_stl`	função	`(path, *, dedup_eps=1e-06) -> STLMesh`	Lê STL de um arquivo em disco via `load_stl_bytes`.

pinneapple_data.synth

Geradores de dados sintéticos (EDP/EDO, EDP simbólica com diferenças finitas, distribuições, ajuste de curvas, imagens, geometria com trimesh e CadQuery) com interface comum e catálogo por nome. [sem teste]

`pinneapple_data/synth/__init__.py` · 38 linhas · **sem teste direto**

pinneapple_data.synth.base

Tipos base dos geradores sintéticos: `SynthConfig`, `SynthOutput` e o protocolo `SynthGenerator`. [sem teste]

`pinneapple_data/synth/base.py` · 66 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`SynthConfig`	classe	`(seed, device, dtype)`	Configuração comum dos geradores: semente, dispositivo e tipo de dado.
`SynthOutput`	classe	`(samples, extras)`	Saída padrão de um gerador: lista de amostras e extras.
`SynthGenerator`	classe	`()`	Protocolo de gerador sintético: precisa implementar `generate` devolvendo `SynthOutput`.

pinneapple_data.synth.curvefit

Gerador que ajusta um polinômio com regularização ridge a dados 1D e sintetiza novas amostras com ruído. [sem teste]

`pinneapple_data/synth/curvefit.py` · 153 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`CurveFitSynthGenerator`	classe	`(cfg=None)`	Ajusta uma tendência polinomial (ridge) a pares (x, y) e gera novos y em novos x.

pinneapple_data.synth.distributions

Gerador que amostra entradas de distribuições simples: gaussiana, uniforme e mistura de gaussianas. [sem teste]

`pinneapple_data/synth/distributions.py` · 154 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`DistributionSynthGenerator`	classe	`(cfg=None)`	Amostra de distribuição gaussiana, uniforme ou mistura de gaussianas com parâmetros configuráveis.

pinneapple_data.synth.geometry

Gerador de variantes de uma malha triangular existente (trimesh) por transformações aleatórias controladas. [sem teste]

`pinneapple_data/synth/geometry.py` · 221 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`GeometrySynthGenerator`	classe	`(cfg=None)`	Gera variantes de uma malha (escala, rotação, ruído etc.) que mantêm a forma geral.

pinneapple_data.synth.geometry_cadquery

Síntese de geometria com CadQuery: variantes de um modelo paramétrico e variantes guiadas por um STL modelo. [sem teste]

pinneapple_data/synth/geometry_cadquery.py · 532 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
ParametricCadQuerySynthGenerator	classe	(cfg=None)	Sorteia parâmetros, constrói o sólido com template_fn do CadQuery, tessela em malha e devolve amostras tipo PhysicalSample.
STLTemplateSynthGenerator	classe	(cfg=None)	Lê um STL/malha modelo, usa a caixa envolvente e, se houver CadQuery, gera primitivas (caixa, cilindro...) parecidas; senão, deforma a malha.

pinneapple_data.synth.images

Gerador de tarefas de reconstrução de imagem e inpainting: entrada corrompida e reconstrução como amostras tipo PhysicalSample. [sem teste]

pinneapple_data/synth/images.py · 262 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
ImageReconstructionSynthGenerator	classe	(cfg=None)	Gera pares imagem corrompida / reconstruída em modos como inpainting suave dentro de uma máscara.

pinneapple_data.synth.operators

Operadores pequenos usados pelos geradores sintéticos: ruído gaussiano, máscara de bloco e normalização para [0, 1]. [sem teste]

pinneapple_data/synth/operators/__init__.py · 6 linhas · **sem teste direto**

pinneapple_data.synth.operators.masks

Gera máscara retangular aleatória numa grade 2D. [sem teste]

pinneapple_data/synth/operators/masks.py · 38 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
random_block_mask	função	(H, W, frac=0.2, seed=42) -> torch.Tensor	Gera uma máscara booleana com um único bloco retangular ocupando a fração frac da altura e largura, em posição sorteada com gerador determinístico.

pinneapple_data.synth.operators.noise

Soma ruído gaussiano a tensores com gerador determinístico. [sem teste]

pinneapple_data/synth/operators/noise.py · 28 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
add_gaussian_noise	função	(x, std=0.01, seed=42) -> torch.Tensor	Soma ruído normal de média zero e desvio std a um tensor.

pinneapple_data.synth.operators.transforms

Normalização de tensores para [0, 1]. [sem teste]

pinneapple_data/synth/operators/transforms.py · 28 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
normalize_01	função	(x, eps=1e-08) -> torch.Tensor	Reescala um tensor para [0, 1] pelo mínimo e máximo globais, com epsilon para tensor constante.

pinneapple_data.synth.pde

Gerador sintético de EDP/EDO simples: calor 1D, advecção 1D e logística, com diferenças finitas. [sem teste]

pinneapple_data/synth/pde.py · 254 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
SimplePhysicalSample	classe	(fields, coords, meta)	Contêiner mínimo com fields, coords e meta, para o módulo funcionar sem o PhysicalSample completo.
PDESynthGenerator	classe	(cfg=None)	Gera trajetórias de calor 1D, advecção 1D ou EDO logística a partir de domínio, grade, tempo e condição inicial.

pinneapple_data.synth.pde_symbolic

Interpreta uma EDP escrita como texto (forma de resíduo, como "u_t - alpha*u_xx") e gera trajetórias e resíduo com diferenças finitas. [sem teste]

pinneapple_data/synth/pde_symbolic.py · 498 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
SymbolicFDSynthGenerator	classe	(cfg=None)	Converte uma equação simbólica em operador de diferenças finitas e gera um dataset sintético de trajetórias.
make_fd_residual_fn	função	(equation, *, parameters=None, bc='periodic')	Cria uma função que avalia o resíduo simbólico trocando u_t, u_x, u_xx por diferenças finitas.

pinneapple_data.synth.registry

SynthCatalog: fábrica que constrói geradores sintéticos por nome a partir de um registro. [sem teste]

pinneapple_data/synth/registry.py · 73 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
SynthCatalog	classe	(registry)	Registro nome → classe de gerador; list mostra os nomes e build instancia (levanta KeyError para nome desconhecido).

pinneapple_data.synth.sample_adapter

Converte saídas dos geradores sintéticos em PhysicalSample quando possível, ou num substituto mínimo. [sem teste]

pinneapple_data/synth/sample_adapter.py · 192 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
PhysicalSampleLike	classe	(fields, coords, meta)	Protocolo mínimo de amostra: fields, coords e meta.
has_pinnego_physical_sample	função	() -> bool	Diz se pinneapple_data.physical_sample.PhysicalSample pode ser importado (o nome da função ainda usa o nome antigo do projeto).
to_physical_sample	função	(sample_like, *, device=None, dtype=None) -> Any	Conversão de melhor esforço para PhysicalSample (ou um dataclass substituto) a partir de objeto ou dict.

pinneapple_data.transforms

Normalizadores para pré-processamento (fonte única; pinneapple_train.normalizers e pinneapple_train.preprocess reexportam daqui): base abstrata, StandardScaler e MinMaxScaler.

pinneapple_data/transforms.py · 140 linhas · testes: 2 · palavras-chave no código (heurística; o texto acima diz o status real): levanta NotImplementedError em algum caminho

nome	tipo	assinatura	o que faz
`Normalizer`	classe	`()`	Base abstrata de normalizadores com `fit`/`transform`/`inverse` (e apelidos `encode`/`decode`) e estado serializável; os métodos base levantam `NotImplementedError`.
`StandardScaler`	classe	`(eps, mean_, std_)`	Normaliza para média zero e variância unitária; aceita estilo de método de classe (`StandardScaler.fit(x)`) e de instância.
`MinMaxScaler`	classe	`(eps, min_, max_)`	Reescala cada feature para [0, 1] e desfaz a escala com `inverse`.

pinneapple_data.udr1

Atalhos do ASME Form U-DR-1 sobre o motor genérico `formfill` (modelo `asme_u-dr-1`): extrair, compilar requisitos e preencher o formulário.

`pinneapple_data/udr1/__init__.py` · 30 linhas · testes: 36

nome	tipo	assinatura	o que faz
`extract_rules`	função	`(doc) -> List[Candidate]`	Aplica `formfill.extract_rules` ao documento com o spec do U-DR-1.
`compile_requirements`	função	`(docs, candidates, overrides=None) -> Compilation`	Compila os candidatos com o spec do U-DR-1, aceitando respostas do engenheiro como `overrides`.
`fill_form`	função	`(blank_pdf, comp) -> bytes`	Preenche o PDF em branco do U-DR-1 com a compilação.

pinneapple_data.unified

Padronizador de dados de engenharia: vários arquivos com convenções diferentes viram um esquema físico único (UPD), com mapeamento revisado por pessoa, conversão exata de unidades, fusos para UTC e alinhamento num eixo de tempo comum.

`pinneapple_data/unified.py` · 584 linhas · testes: 24

nome	tipo	assinatura	o que faz
`canonical_name`	função	`(base, quantity) -> str`	Gera o nome canônico de uma coluna a partir do cabeçalho e da grandeza (por exemplo `'CHW Supply Temp'` → `chw_supply_temperature`).
`conversion_text`	função	`(src, dst, difference=False) -> str`	Texto legível de uma conversão de unidade, por exemplo `°F → °C: (x − 32) × 0.5556`.
`propose_mapping`	função	`(df, source, system='metric', time_column=None, units=None, tz=None) -> Dict[str, Any]`	Rascunha o mapeamento de uma fonte a partir dos cabeçalhos (coluna de tempo, grandeza, unidade, nome e unidade alvo), para revisão humana.
`make_recipe`	função	`(mappings, system='metric', resample='auto', layout='wide', agg='mean') -> Dict[str, Any]`	Monta a receita (`pinneapple.recipe/1`) a partir dos mapeamentos, com sistema alvo, reamostragem, layout e agregação.
`standardize`	função	`(sources, recipe) -> Dict[str, Any]`	Aplica a receita às fontes e devolve tabelas larga e longa, manifesto, notas por fonte e concordância entre fontes.
`write`	função	`(result, fmt='csv', layout='wide') -> Tuple[bytes, str, str]`	Serializa o resultado padronizado e devolve (bytes, tipo de mídia, nome de arquivo).
`to_physical_sample`	função	`(result)`	Converte a tabela larga num `PhysicalSample` (xarray com coordenada `time` e unidades nos atributos).
`example_sources`	função	`(seed=3) -> List[Tuple[str, bytes]]`	Três fontes de exemplo da mesma planta de água gelada (CSV de BMS, JSON de SCADA, Excel de bancada) com convenções diferentes.
`before_after_series`	função	`(result, k=500) -> Dict[str, Any]`	Por variável, valores brutos de cada fonte e valores padronizados no eixo UTC comum, reduzidos a no máximo `k` pontos.

pinneapple_data.upd_dataset

`UPDDataset` endurecido: lista de amostras com versão de esquema no manifesto, validação e salvar/carregar em pt, zarr ou hdf5.

`pinneapple_data/upd_dataset.py` · 192 linhas · testes: 5

nome	tipo	assinatura	o que faz
`UPDDataset`	classe	`(samples, manifest)`	Dataset de `PhysicalSample` com validação e persistência em `.pt`, Zarr ou HDF5.

pinneapple_data.upd_types

Tipos de dados puros do pipeline UPD (shard, especificações de amostragem de condições, batch PINN) e conversão de `datetime64` para segundos; o docstring ainda cita o pacote antigo `pinneapple_pinn`. [sem teste]

`pinneapple_data/upd_types.py` · 103 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`UPDItem`	classe	`(zarr_path, meta_path)`	Um item de shard UPD: caminho do store Zarr e do JSON de metadados; lê os metadados e abre o dataset.
`ConditionSpec`	classe	`(name, type, equation, n, options)`	Como amostrar uma condição PINN: `initial`, `boundary`, `interior` ou `slice`, com número de pontos e variáveis.
`SamplingSpec`	classe	`(n_collocation, conditions, n_data, replace, seed)`	Plano de amostragem de um shard UPD: pontos de colocação e lista de `ConditionSpec`.
`Batch`	classe	`(collocation, conditions, data)`	Batch de treino PINN: tensores de colocação, de cada condição e pares de dados supervisionados.
`seconds_since`	função	`(t, t0=None) -> np.ndarray`	Converte um array `datetime64` em segundos (float64) desde `t0` (padrão: o menor tempo).

pinneapple_data.validators

Validação de `PhysicalSample`: unidades (política `strict`/`warn`/`off`), faixas, não negatividade e coordenadas monótonas. [sem teste]

`pinneapple_data/validators.py` · 161 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ValidationIssue`	classe	`(level, message, field)`	Um problema de validação: nível (`error` ou `warning`), mensagem e campo.
`validate_physical_sample`	função	`(sample, *, units_policy='warn', required_units=None, ranges=None, non_negative=None, mon…)`	Valida uma amostra e devolve a lista de problemas encontrados.
`assert_valid_physical_sample`	função	`(*args, **kwargs) -> None`	Valida uma amostra e levanta `ValueError` se houver algum erro.

pinneapple_data.zarr_cached_store

Envolve o `UPDZarrStore` com cache LRU por amostra e por campo, limitado por número de itens. [sem teste]

`pinneapple_data/zarr_cached_store.py` · 231 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ZarrCacheConfig`	classe	`(max_samples, max_fields, enable_field_cache)`	Limites do cache por contagem: máximo de amostras e de leituras de campo, e se o cache de campo está ligado.
`CachedUPDZarrStore`	classe	`(root, *, cache=None, mode='r')`	Store Zarr UPD com cache LRU: `count`, `manifest`, `read_sample` e estatísticas do cache.

pinneapple_data.zarr_cached_store_bytes

Envolve o `UPDZarrStore` com cache LRU limitado por bytes (`ByteLRUCache`) por amostra e por campo. [sem teste]

`pinneapple_data/zarr_cached_store_bytes.py` · 226 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ZarrByteCacheConfig`	classe	`(max_sample_bytes, max_field_bytes, enable_field_cache)`	Limites do cache em bytes para amostras inteiras e para campos.
`CachedUPDZarrStoreBytes`	classe	`(root, *, cache=None, mode='r')`	Store Zarr UPD com cache limitado por bytes: `count`, `manifest`, `read_sample` e estatísticas.

pinneapple_data.zarr_iterable

`IterableDataset` que lê `PhysicalSample` de um `UPDZarrStore`, abrindo o store em cada worker e dividindo o trabalho entre workers. [sem teste]

`pinneapple_data/zarr_iterable.py` · 69 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ZarrUPDIterable`	classe	`(root, fields, coords, start, stop, step, shuffle, seed)`	Itera as amostras de um store Zarr UPD com divisão por worker do `DataLoader`, sem exigir `__len__`.

pinneapple_data.zarr_prefetch

`IterableDataset` sobre store Zarr UPD com cache, thread produtora em segundo plano, memória fixada e transferência assíncrona para GPU. [sem teste]

`pinneapple_data/zarr_prefetch.py` · 205 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`PrefetchConfig`	classe	`(prefetch, queue_max, use_sample_cache, pin_memory, target_device, transfer_non_blocking)`	Controles do prefetch: tamanho da fila, pin_memory, dispositivo alvo e transferência não bloqueante.
`PrefetchZarrUPDIterable`	classe	`(root, *, fields=None, coords=None, dtype=None, start=0, end=None, stride=1, sample_ctor=…)`	Lê amostras do Zarr numa thread produtora e entrega em CPU ou já movidas para a GPU.

pinneapple_data.zarr_shard_iterable

`IterableDataset` que lê o `index.json` de um store Zarr UPD fatiado em shards e distribui as partes entre os workers. [sem teste]

`pinneapple_data/zarr_shard_iterable.py` · 162 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ShardAwareConfig`	classe	`(pin_memory, target_device, transfer_non_blocking, use_sample_cache)`	Configuração da iteração por shards: pin_memory, dispositivo, transferência não bloqueante e embaralhamento.
`ShardAwareZarrUPDIterable`	classe	`(root, *, fields=None, coords=None, dtype=None, sample_ctor=None, cache=None, cfg=None)`	Itera as partes de shard atribuídas a cada worker, uma de cada vez, a partir do `index.json`.

pinneapple_data.zarr_shards

Escritor de stores Zarr UPD fatiados por chave (tempo, região, regime) em `shards/<chave>/part=NNNN/data.zarr` com `index.json`. [sem teste]

`pinneapple_data/zarr_shards.py` · 154 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ShardSpec`	classe	`(key_fn, max_per_shard)`	Nomeação de shards: função `key_fn(sample)` que devolve a chave (por exemplo `time=2020-01`) e tamanho de parte.
`UPDZarrShardedWriter`	classe	`(root, shard_spec, *, shards_dirname='shards', index_filename='index.json')`	Escreve amostras em vários shards por chave e partes numeradas, e grava o índice.

pinneapple_data.zarr_store

Store Zarr do UPD em layout colunar (cada campo empilhado entre amostras, coordenadas e manifesto opcionais) para persistir `PhysicalSample`.

`pinneapple_data/zarr_store.py` · 396 linhas · testes: 1

nome	tipo	assinatura	o que faz
`UPDZarrStore`	class e	`(root, mode='r')`	Store Zarr UPD mínimo: escreve amostras, itera, conta, lista campos e coordenadas e lê o manifesto.

Exemplos de uso reais do repositório

Criar um PhysicalSample em grade e convertê-lo para dict de treino — `examples/data_pipeline/01_physical_sample_basics.py`

```
from pinneapple_data.physical_sample import PhysicalSample

ds = xr.Dataset(
    data_vars=dict(
        T2M=(("t","x"), np.random.randn(24, 16).astype("float32")),
        U10M=(("t","x"), np.random.randn(24, 16).astype("float32")),
    ),
    coords=dict(t=np.arange(24), x=np.arange(16)),
)

sample = PhysicalSample(
    state=ds,
    domain={"type": "grid"},
    schema={"governing": {"type": "PDE", "name": "toy"}},
    provenance={"source": "demo"},
)
```

Planejar uma campanha que não repete execuções existentes — `tests/pinneapple_data/test_doe.py`

```
    plan = plan_campaign(space, method="maximin", seed=1, existing=existing, existing_id_column="run_id")
    assert len(plan.cases) == 12 and list(plan.cases["priority"]) == list(range(1, 13))
    assert plan.cases["case_id"].iloc[0] == "case_001"
    s = plan.summary()
    assert s["coverage"]["combined"]["fill_distance"] < s["coverage"]["existing"]["fill_distance"]
    res = plan.study_result()
    assert isinstance(res, StudyResult) and res.passed and res.status == "planned"
    assert StudyResult.from_json(res.to_json()) == res
    skel = plan.dataset_skeleton(outputs=["y"], units={"y": "K"})
```

Pré-voo de um caso OpenFOAM ou deck CalculiX enviado — `tests/test_preflight.py`

```
from pinneapple_data.cae.upload import expand
from pinneapple_data.preflight import preflight


def _run(name):
    with open(os.path.join(EX, name), "rb") as f:
        return preflight(expand([(name, f.read())]))
```

Testes e status de validação

- Módulos com pelo menos um teste que os importa: 45 de 115.
- Funções de teste em arquivos que importam algum módulo do pacote: 507.

Limitações conhecidas

- A raiz do pacote importa quase tudo dentro de `try/except Exception: pass`: se uma dependência opcional faltar (xarray, zarr, torch), o nome simplesmente não existe em `pinneapple_data`, sem aviso.
- Muitas dependências são opcionais e só exigidas pelo módulo que as usa: `trimesh` (geometria, `geometry_features`), `meshio`, `h5py`, `pyarrow`, `pydantic` (contratos), `pypdf`/`pdfplumber`/`reportlab` (formfill), o programa `tesseract` (OCR), `tfrecord` (DeepMind), `cadquery`, `mp-api`, `cdsapi`, e um servidor Ollama local para `formfill.llm`.

- Vários clientes de `datasets.external_sources` e carregadores de `datasets.real_world` devolvem dados sintéticos quando a fonte não está disponível (marcados com `_synthetic` ou `source: synthetic`); alguns datasets de `real_world` são sintéticos por definição. Não use esses dados como medições reais sem conferir a marcação.
- `dataloaders.build_physical_sample_dataloader` só aceita amostras em grade; amostras em malha ainda não são suportadas.
- `transforms.Normalizer` é base abstrata: `fit`, `transform` e `inverse` levantam `NotImplementedError` e precisam de subclasse.
- Há nomes antigos nos docstrings: `pinneapple_geom` (hoje `pinneapple_design.geometry`), `pinneapple_pinn` (citado em `upd_types` e `adapters.pinn_batch_builders`) e `pinnego` em `has_pinnego_physical_sample`. Em `serialization`, `save_hdf5` aparece definido duas vezes e vale a segunda definição. O exemplo de `formfill.pipeline` cita o id `api610_pump`, mas o registro usa `pump`.
- Muitos módulos não têm teste direto: adaptadores, caches de Zarr, iteráveis e prefetch, `synth` inteiro, `augmentation`, `geometry_features`, `grf_sampler`, `validators`, `upd_types` e vários submódulos de `cae`, `formfill`, `preflight`, `lineage` e `simulation_metadata` (estes são exercitados pelos testes dos pacotes-pai e dos apps).
- `cae.foam` não remonta casos decompostos (`processor*`), e `cae.upload` descarta essas pastas; diretivas OpenFOAM (`#include`, `#codeStream`) são registradas, não executadas.

3. `pinneapple_pdb`

Objetivo

O `pinneapple_pdb` (physics database) tem duas partes independentes. A primeira é o `PhysicalDatasetBuilder`, que monta datasets físicos a partir de hubs de dados da Terra (NASA CMR via `earthaccess`): busca coleções e granules, escolhe variáveis e janela espaço-temporal, abre os dados com xarray, padroniza dimensões, valida unidades, faixas e monotonicidade, acrescenta variáveis derivadas (vorticidade e divergência), fatia em janelas de tempo e blocos espaciais com tags de regime e grava tudo em Zarr no formato UPD, com um catálogo e um esquema físico (equações, contornos, política de unidades) vindo de templates como `atmosphere_reanalysis_v1`.

A segunda parte é `benchmarks`, um catálogo pequeno e curado de datasets de referência resolvidos por nome (`get_benchmark("lane_emden_n1.5")`), no mesmo padrão `@register_x`/`get_x`/`list_x` do registro de presets de `pinneapple_physics`. Cada entrada é recalculada ao ser pedida (integração com SciPy ou solução fechada) e conferida contra um valor publicado ou uma solução exata; se a conferência falhar, a função levanta erro em vez de devolver um perfil não verificado. As entradas cobrem Lane-Emden (n=1.5 e n=3), sustentação de vórtice de Polhamus no ângulo de Concorde (teoria, não dado de voo), camada-limite de Blasius, condução transiente em parede plana com Bi=1 e três casos com as equações do PDEBench (advecção 1D, difusão-reação 1D, Darcy 2D) sem baixar as trajetórias do PDEBench.

O catálogo de benchmarks é usado pelo guardrail físico de `pinneapple_llm` e por exemplos como `examples/use_cases/concorde_high_aoa`; o builder é usado pelos exemplos de `examples/physics_db/`. O pacote não tem CLI: o `pyproject.toml` registra que o antigo script `pinneapple_pdb.cli:main` foi removido por não existir.

Números do inventário

módulos	linhas	classes públicas	funções públicas	testes em arquivos que importam o pacote	módulos sem teste direto
7	1.929	9	14	100 (5 arquivos)	5

Submódulos

submódulo	o que faz	módulos	classes + funções	testes*
`benchmarks`	Catálogo curado, por nome, de datasets de referência (x, y) recalculados e verificados a cada uso.	1	5	35

submódulo	o que faz	módulos	classes + funções	testes*
`builder`	`PhysicalDatasetBuilder`: busca dados em hubs da Terra (earthaccess/CMR), valida, deriva, fatia e grava Zarr UPD com catálogo.	1	5	0
`derived`	Variáveis derivadas (vorticidade e divergência) a partir de U/V em grade lat/lon.	1	3	0
`shard`	Fatiamento por janelas de tempo e blocos espaciais, com tags de regime (trópicos, polar).	1	6	0
`templates`	Templates de esquema físico (por exemplo reanálise atmosférica).	1	1	0
`validate`	Padronização de dimensões e validação de unidades, faixas e monotonicidade.	1	3	0

* maior número de testes que importam um módulo do submódulo.

Módulos e API pública

`pinneapple_pdb`

Ponto de entrada: reexporta o builder e suas specs, `schema_templates` e a API do catálogo de benchmarks; não tem docstring de módulo.

`pinneapple_pdb/__init__.py` · 36 linhas · testes: 65

`pinneapple_pdb.benchmarks`

Catálogo de referências por nome com oito entradas registradas, cada uma recalculada e conferida contra valor publicado ou solução exata (levanta erro se divergir); o docstring ainda fala em "FIVE entries", desatualizado.

`pinneapple_pdb/benchmarks.py` · 984 linhas · testes: 35

nome	tipo	assinatura	o que faz
`BenchmarkEntry`	classe	`(name, description, reference_source, x_vars, y_vars, reference_x, reference_y, verificat…)`	Um dataset de referência nomeado: descrição, fonte, variáveis x/y, arrays de referência e nota de verificação.
`register_benchmark`	função	`(name)`	Decorador que registra uma fábrica sem argumentos de `BenchmarkEntry` sob um nome.
`get_benchmark`	função	`(name) -> BenchmarkEntry`	Resolve um benchmark pelo nome, construindo e verificando a entrada.
`list_benchmarks`	função	`() -> List[str]`	Lista ordenada dos nomes de benchmarks registrados.
`benchmark_catalog`	função	`() -> Dict[str, BenchmarkEntry]`	Dicionário nome -> `BenchmarkEntry` de todos os benchmarks, construídos e verificados de novo.

`pinneapple_pdb.builder`

Construtor de datasets físicos a partir de NASA CMR/`earthaccess` (extra opcional `pinneapple[earthdata]`, importado sob demanda): busca, inspeção de granule, seleção, validação, derivadas, fatiamento e escrita Zarr UPD com catálogo; `build` exige login no Earthdata. [sem teste direto]

`pinneapple_pdb/builder.py` · 539 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`HubQuery`	classe	`(provider, short_name, keyword)`	Parâmetros de busca no hub: provedor, nome curto e palavra-chave.
`SpaceTime`	classe	`(time_start, time_end, bbox, stride, chunking)`	Extensão temporal e espacial (bbox) com passo e chunking.
`VariableSelection`	classe	`(include, exclude, packs)`	Seleção de variáveis por inclusão, exclusão e pacotes.
`PhysicalSchema`	classe	`(physical_system, governing_equations, ics, bcs, forcings, units_policy, regime_tags, val…)`	Metadados físicos: sistema, equações, CI, CC, forçantes, política de unidades, tags e validade.

nome	tipo	assinatura	o que faz
`PhysicalDatasetBuilder`	classe	`(packs=None)`	Builder MVP: busca e inspeção de granules, sharding por tempo e blocos, derivadas e validações, saída Zarr com catálogo.

pinneapple_pdb.derived

Calcula vorticidade e divergência de vento U/V numa grade lat/lon e aplica as derivadas pedidas numa spec. [sem teste direto]

`pinneapple_pdb/derived.py` · 116 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`DerivedSpec`	classe	`(derived, u_name, v_name)`	Quais variáveis derivadas acrescentar e os nomes de U e V.
`add_vorticity_divergence`	função	`(ds, u_name=None, v_name=None) -> xr.Dataset`	Acrescenta vorticidade e divergência calculadas de U/V em grade lat/lon.
`apply_derived`	função	`(ds, spec) -> xr.Dataset`	Aplica as derivadas de uma spec e devolve um novo dataset.

pinneapple_pdb.shard

Gera janelas de tempo não sobrepostas e blocos lat/lon alinhados, recorta o dataset e marca tags `tropics`/`polar` pela latitude. [sem teste direto]

`pinneapple_pdb/shard.py` · 72 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ShardSpec`	classe	`(time_window, tile_deg, add_regime_tags)`	Janela de tempo, tamanho de bloco em graus e se adiciona tags de regime.
`iter_time_windows`	função	`(time_values, window) -> Iterator[Tuple[pd.Timestamp, pd.Timestamp]]`	Gera pares (início, fim) de janelas de tempo não sobrepostas.
`subset_time`	função	`(ds, a, b) -> xr.Dataset`	Recorta o intervalo de tempo [a, b).
`iter_tiles`	função	`(lat_vals, lon_vals, dlat, dlon)`	Gera limites de blocos lat/lon alinhados à grade de passos dados.
`subset_tile`	função	`(ds, lat0, lat1, lon0, lon1) -> xr.Dataset`	Recorta um bloco espacial; supõe coordenadas ordenadas.
`regime_tags_for`	função	`(ds) -> list[str]`	Tags de regime (`tropics`, `polar`) pela extensão em latitude.

pinneapple_pdb.templates

Devolve o template `atmosphere_reanalysis_v1` (equações primitivas hidrostáticas); a política pede SI mas nenhuma conversão automática é feita. [sem teste direto]

`pinneapple_pdb/templates.py` · 28 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`schema_templates`	função	`() -> Dict[str, Dict[str, Any]]`	Mapa id de template -> dicionário compatível com `PhysicalSchema`.

pinneapple_pdb.validate

Padroniza nomes e ordem de dimensões (time, lev, lat, lon) e devolve a lista de problemas de unidades, faixas e monotonicidade. [sem teste direto]

`pinneapple_pdb/validate.py` · 154 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ValidationSpec`	classe	`(require_units, ranges, enforce_lev_monotonic, enforce_time_monotonic)`	Spec de validação: exigir unidades, faixas por variável e monotonicidade de nível e tempo.

nome	tipo	assinatura	o que faz
`standardize_dims`	função	`(ds) -> xr.Dataset`	Padroniza nomes de dimensões e transpõe variáveis para (time, lev, lat, lon).
`validate_dataset`	função	`(ds, spec) -> List[str]`	Valida o dataset contra a spec e devolve as mensagens de problema (vazia se válido).

Exemplos de uso reais do repositório

Inspecionar um granule de um hub da Terra — `examples/physics_db/00_quickstart_inspect.py`

```
b = PhysicalDatasetBuilder()

# 1) Search collections (keep keyword broad at first)
collections = b.list_collections(keyword="MERRA", limit=10)
print("\nTop matches:")
for c in collections[:10]:
    print(f"- {c.get('short_name')} | provider={c.get('provider')} | {c.get('title','')[:80]}")

# 2) Pick one (edit as needed)
# Many users end up using MERRA-2 products; pick a short_name shown above.
b.set_dataset(provider=collections[0].get("provider"), short_name=collections[0].get("short_name"))
```

Usar um benchmark do catálogo como referência teórica — `examples/use_cases/concorde_high_aoa/concorde_high_aoa_pipeline.py`

```
bench = get_benchmark("concorde_high_aoa")

fig, ax = plt.subplots(figsize=(9, 5.5), facecolor=DARK_BG)
ax.set_facecolor(DARK_BG)
for spine in ax.spines.values():
    spine.set_edgecolor("#444")
ax.tick_params(colors="white")

ax.plot(
    bench.reference_x[:, 0], bench.reference_y[:, 0],
    "-", color="white", lw=1.5,
```

Testes e status de validação

- Módulos com pelo menos um teste que os importa: 2 de 7.
- Funções de teste em arquivos que importam algum módulo do pacote: 100.

Limitações conhecidas

- O builder depende do extra opcional `earthaccess` (`pip install pinneapple[earthdata]`), de acesso à rede e de login no Earthdata (`build` chama `login`); também usa xarray, pandas e um engine como `pydap`.
- Os módulos do builder (`builder`, `derived`, `shard`, `templates`, `validate`) não têm teste que os importe diretamente; os testes do pacote cobrem sobretudo `benchmarks`.
- O catálogo de benchmarks é pequeno e curado; as entradas PDEBench reproduzem só as equações e parâmetros de exemplo, não as trajetórias reais do PDEBench.
- `concorde_high_aoa` é a teoria de Polhamus avaliada na razão de aspecto aproximada do Concorde, não dado de túnel de vento ou de voo.
- O docstring de `benchmarks` diz "FIVE entries", mas há oito registradas; um dataset LES de canal citado no docstring não foi incluído por viver fora do repositório.
- A política de unidades do template pede SI, mas nenhuma conversão é feita: as unidades dos granules são gravadas como vieram (`converted: False`).
- Não há CLI: o script `pinneapple_pdb.cli:main` foi removido do `pyproject.toml` porque o módulo não existe.

4. pinneapple_adaptation

Objetivo

pinneapple_adaptation reúne duas formas de reaproveitar um PINN já treinado em um problema físico parecido, sem treinar do zero. A primeira é transfer learning (transfer_learning, antigo pinneapple_transfer): pegar um modelo treinado, congelar parte das camadas e ajustar o resto no novo problema, por exemplo Burgers com outra viscosidade. A segunda é meta-learning (meta_learning, antigo pinneapple_meta): treinar uma inicialização compartilhada, com MAML ou Reptile, que se adapta a qualquer membro de uma família de EDPs em poucos passos de gradiente.

O pacote trabalha com qualquer torch.nn.Module e com funções de perda física no formato (model, batch) -> perda. Na prática, essas funções vêm do compilador de problemas de pinneapple_physics (compile_problem sobre um ProblemSpec de um preset, como laplace_2d_default ou burgers_1d_default), como mostra o teste tests/test_adaptation_and_active_learning.py. O catálogo de métodos (pinneapple_catalog.methods) lista o meta-learning como método de treino (item T18).

Na raiz há dois atalhos: fine_tune, que monta um TransferConfig e um TransferTrainer, chama prepare() e depois finetune(); e adapt_model, que escolhe entre meta-learning (quando o alvo é um dict com chave support) e transfer learning. Os nomes antigos transfer e meta continuam como apelidos dos submódulos, para compatibilidade.

Números do inventário

módulos	linhas	classes públicas	funções públicas	testes em arquivos que importam o pacote	módulos sem teste direto
13	2.946	10	9	154 (2 arquivos)	2

Submódulos

submódulo	o que faz	módulos	classes + funções	testes *
meta_learning	Meta-learning (MAML e Reptile) sobre famílias paramétricas de EDPs: amostrador de tarefas, treinadores, configurações e um invólucro MetaModel para adaptar e prever.	6	8	151
transfer_learning	Transfer learning e fine-tuning de PINNs: congelamento de camadas, taxas de aprendizado por camada, descongelamento progressivo, interpolação de pesos entre variantes e adaptação de domínio por MMD.	6	9	151

* maior número de testes que importam um módulo do submódulo.

Módulos e API pública

pinneapple_adaptation

Raiz do pacote: reexporta as classes de transfer_learning e meta_learning, mantém os apelidos antigos transfer/meta e define os atalhos fine_tune e adapt_model.

pinneapple_adaptation/__init__.py · 106 linhas · testes: 3

nome	tipo	assinatura	o que faz
fine_tune	função	(source_model, target_physics_fn, target_data=None, *, strategy='finetune', epochs=500, l…)	Atalho que cria TransferConfig + TransferTrainer, chama prepare() e finetune() e devolve o dict de resultado (modelo, histórico, métricas).
adapt_model	função	(model, target, *, mode='auto', **kwargs)	Adapta um modelo a uma tarefa nova: usa meta_adapt se o alvo for um dict com support (ou mode='meta'), senão fine_tune.

pinneapple_adaptation.meta_learning

Reexporta configurações, amostrador e treinadores de meta-learning e oferece `meta_train` (constrói e treina) e `meta_adapt` (adapta a uma tarefa nova).

`pinneapple_adaptation/meta_learning/__init__.py` · 113 linhas · testes: 3

nome	tipo	assinatura	o que faz
`meta_train`	função	`(model, sampler, *, algorithm='reptile', **cfg_kwargs)`	Constrói um `ReptileTrainer` (padrão) ou `MAMLTrainer`, executa `train()` e devolve o treinador já treinado.
`meta_adapt`	função	`(meta_trainer, task, n_steps=10)`	Chama `adapt` do treinador meta-treinado para uma tarefa nova e devolve uma cópia adaptada do modelo.

pinneapple_adaptation.meta_learning.config

Dataclasses `MAMLConfig` e `ReptileConfig` com os hiperparâmetros dos laços interno e externo, validados em `__post_init__`.

`pinneapple_adaptation/meta_learning/config.py` · 134 linhas · testes: 151

nome	tipo	assinatura	o que faz
`MAMLConfig`	classe	`(n_inner_steps, inner_lr, outer_lr, n_tasks_per_batch, n_meta_epochs, first_order, device…)`	Hiperparâmetros do MAML: passos e taxa do laço interno, taxa externa, tarefas por lote, épocas, flag `first_order`, dispositivo, semente e checkpoint.
`ReptileConfig`	classe	`(n_inner_steps, inner_lr, outer_lr, n_tasks_per_batch, n_meta_epochs, device, seed, epsil…)`	Hiperparâmetros do Reptile, incluindo o passo de interpolação `epsilon` (validado no intervalo (0, 1]).

pinneapple_adaptation.meta_learning.maml

Treinador MAML para famílias de EDPs, em versão de primeira ordem (padrão) ou de segunda ordem; usa `torch.func.functional_call` quando existe e cai para cópia profunda + SGD em PyTorch antigo.

`pinneapple_adaptation/meta_learning/maml.py` · 480 linhas · testes: 151

nome	tipo	assinatura	o que faz
`MAMLTrainer`	classe	`(model, config, task_sampler) -> None`	Treina um modelo com MAML sobre uma distribuição de tarefas de EDP, com `meta_update`, `train`, `adapt`, `save` e `load`.

pinneapple_adaptation.meta_learning.meta_model

Invólucro `MetaModel` de um modelo meta-treinado: adapta a uma função física nova, faz predição e salva/carrega o estado.

`pinneapple_adaptation/meta_learning/meta_model.py` · 191 linhas · testes: 151

nome	tipo	assinatura	o que faz
`MetaModel`	classe	`(model, meta_type='reptile', device='cpu') -> None`	Invólucro de um modelo meta-treinado (rótulo `reptile` ou `maml`) para adaptar em poucos passos, prever e serializar.

pinneapple_adaptation.meta_learning.reptile

Treinador Reptile: treina em cada tarefa separadamente e interpola os pesos compartilhados na direção dos pesos de cada tarefa, sem gradientes de segunda ordem.

`pinneapple_adaptation/meta_learning/reptile.py` · 294 linhas · testes: 151

nome	tipo	assinatura	o que faz
`ReptileTrainer`	classe	`(model, config, task_sampler) -> None`	Treinador Reptile com `meta_update`, `train`, `adapt` (devolve cópia adaptada), `save` e `load`.

pinneapple_adaptation.meta_learning.task_sampler

PDETaskSampler: sorteia parâmetros de EDP em intervalos dados e monta tarefas com conjuntos de suporte e de consulta para o meta-treino.

pinneapple_adaptation/meta_learning/task_sampler.py · 196 linhas · testes: 151

nome	tipo	assinatura	o que faz
PDETaskSampler	classe	(param_ranges, physics_fn_factory, data_factory=None, n_support=64, n_query=64, input_dim…)	Gera tarefas de meta-treino: parâmetros sorteados uniformemente, função física da fábrica e pontos de suporte/consulta (de data_factory ou uniformes em [0,1]^d).

pinneapple_adaptation.transfer_learning

Reexporta TransferConfig, TransferTrainer, utilitários de congelamento, ParametricFamilyTransfer e PhysicsTransferAdapter. [sem teste direto do __init__]

pinneapple_adaptation/transfer_learning/__init__.py · 49 linhas · **sem teste direto**

pinneapple_adaptation.transfer_learning.adapter

PhysicsTransferAdapter: combina a perda física do domínio alvo com uma regularização pela física de origem e uma perda MMD (kernel RBF) entre features dos dois domínios.

pinneapple_adaptation/transfer_learning/adapter.py · 346 linhas · testes: 151

nome	tipo	assinatura	o que faz
PhysicsTransferAdapter	classe	(model, source_spec, target_spec, source_physics_fn=None, source_weight=0.1, mmd_bandwidt…)	Adapta um PINN de um domínio físico de origem a um alvo relacionado, somando perda alvo, regularização da física de origem e perda MMD entre features.

pinneapple_adaptation.transfer_learning.config

Dataclass TransferConfig com estratégia de transferência (finetune, feature_extract, partial_freeze, progressive), prefixos de congelamento, taxas de aprendizado e pesos de perda.

pinneapple_adaptation/transfer_learning/config.py · 98 linhas · testes: 151

nome	tipo	assinatura	o que faz
TransferConfig	classe	(strategy, freeze_prefix, unfreeze_prefix, layer_lr_scale, base_lr, finetune_lr, epochs, …)	Configuração de um fine-tuning: estratégia, prefixos a congelar/descongelar, escala de taxa por camada, taxas base e de ajuste, épocas, pesos de perda física/dados e warmup.

pinneapple_adaptation.transfer_learning.freeze

Funções para congelar/descongelar parâmetros por prefixo de nome, montar grupos de otimizador com taxa de aprendizado por camada e contar parâmetros treináveis. [sem teste direto]

pinneapple_adaptation/transfer_learning/freeze.py · 202 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
freeze_layers	função	(model, prefixes) -> int	Congela (requires_grad=False) os parâmetros cujo nome começa por algum prefixo e devolve quantos foram congelados.
unfreeze_layers	função	(model, prefixes) -> int	Descongela os parâmetros cujo nome começa por algum prefixo e devolve quantos foram alterados.
freeze_all_except	função	(model, trainable_prefixes) -> None	Congela todos os parâmetros que não casam com os prefixos treináveis.
layer_lr_groups	função	(model, base_lr, scale_dict) -> List[dict]	Monta grupos de parâmetros para o otimizador com taxa de aprendizado por camada (taxa base vezes a escala do prefixo).

nome	tipo	assinatura	o que faz
`count_trainable`	função	`(model) -> Dict[str, int]`	Conta parâmetros treináveis, congelados e totais de um modelo.

`pinneapple_adaptation.transfer_learning.parametric`

`ParametricFamilyTransfer`: guarda variantes de um PINN ajustadas para valores diferentes de um parâmetro físico e interpola linearmente os pesos entre as duas mais próximas.

`pinneapple_adaptation/transfer_learning/parametric.py` · 272 linhas · testes: 151

nome	tipo	assinatura	o que faz
`ParametricFamilyTransfer`	classe	`(base_model, param_name) -> None`	Coleção de PINNs ajustados por valor de um parâmetro escalar: devolve a variante mais próxima, interpola pesos entre vizinhas e salva/carrega a família inteira.

`pinneapple_adaptation.transfer_learning.trainer`

`TransferTrainer`: copia o modelo de origem, aplica a estratégia de congelamento (`prepare`) e roda o laço de fine-tuning, inclusive com cronograma de descongelamento progressivo.

`pinneapple_adaptation/transfer_learning/trainer.py` · 465 linhas · testes: 151

nome	tipo	assinatura	o que faz
`TransferTrainer`	classe	`(source_model, config) -> None`	Treinador de transferência: copia o modelo de origem, aplica a estratégia (`prepare`), ajusta no problema alvo (`finetune`) ou com descongelamento por época (`progressive_unfreeze`).

Exemplos de uso reais do repositório

Fine-tuning de um PINN num preset de Laplace — `tests/test_adaptation_and_active_learning.py`

```
from pinneapple_adaptation import fine_tune

spec = laplace_2d_default()
loss_fn = compile_problem(spec)
bounds = spec.domain_bounds
coords = list(spec.coords)
torch.manual_seed(0)
model = torch.nn.Sequential(torch.nn.Linear(2, 32), torch.nn.Tanh(), torch.nn.Linear(32, 1))

def target_physics_fn(m, batch):
    x_col = torch.as_tensor(
        np.stack([np.random.uniform(*bounds[c], size=256) for c in coords], axis=1).astype(np.float32)
    )
    x_col.requires_grad_(True)
    y = m(x_col)
    return loss_fn(m, y, _empty_batch(x_col, len(coords), 1))
```

Meta-treino Reptile sobre Burgers com viscosidade variável — `tests/test_adaptation_and_active_learning.py`

```
sampler = PDETaskSampler(param_ranges={"nu": (0.001, 0.05)},
                         physics_fn_factory=physics_fn_factory, input_dim=2, seed=0)
torch.manual_seed(0)
model = torch.nn.Sequential(torch.nn.Linear(2, 32), torch.nn.Tanh(), torch.nn.Linear(32, 1))

trainer = meta_train(model, sampler, algorithm="reptile",
                     n_meta_epochs=5, n_tasks_per_batch=2, n_inner_steps=3, seed=0)
assert hasattr(trainer, "adapt"), "meta_train() should return a trainer with .adapt()"
assert len(trainer._history) == 5, "meta_train() must have actually run .train(), not just constructed the trainer"
```

```python
assert all(np.isfinite(h["meta_loss"]) for h in trainer._history)

task = sampler.sample_batch(1)[0]
adapted = trainer.adapt(task, n_steps=5)
```

Testes e status de validação

- Módulos com pelo menos um teste que os importa: 11 de 13.
- Funções de teste em arquivos que importam algum módulo do pacote: 154.

Itens do catálogo de métodos (`pinneapple_catalog/method_status.json`) cujo código está neste pacote:

id	método	status	testes contra referência	outros testes
T18	Meta-learning: MAML, Reptile	tested	0	6
T19	Transfer learning (freeze, adapters, parametric)	tested	0	3

Limitações conhecidas

- O docstring da raiz cita `fast_adapt` no exemplo de uso, mas essa função não existe; o nome real é `meta_adapt`.
- `MAMLTrainer` e `ReptileTrainer` não usam o mesmo contrato de função física: MAML chama `physics_fn(forward_ou_modelo, x)` e Reptile chama `physics_fn(model, batch) -> (perda, dict)`. Uma mesma fábrica só serve aos dois se tratar as duas formas.
- No PyTorch sem `torch.func`, o MAML usa um caminho alternativo (cópia profunda + SGD) que só faz a aproximação de primeira ordem.
- Em `PhysicsTransferAdapter`, `source_spec` e `target_spec` só ficam guardados como referência; não entram no cálculo, a menos que se faça subclasse.
- `ParametricFamilyTransfer.interpolate_weights` não extrapola: fora do intervalo registrado, ou com menos de duas variantes, devolve a variante mais próxima.
- `fine_tune` e `meta_train` tinham bugs (não chamavam `prepare()` / `train()`); foram corrigidos e agora têm teste de regressão, mas o `__init__` de `transfer_learning` e o módulo `freeze` não têm teste direto.

5. `pinneapple_arena`

Objetivo

`pinneapple_arena` é o laboratório de comparação de modelos do PINNeAPPle. A peça central é a classe `Arena`: ela lê uma configuração YAML/JSON (`ArenaConfig`) com um problema físico e uma lista de modelos, treina todos sob o mesmo problema, avalia contra a solução de referência e gera ranking, tabela de métricas e figuras. O problema que resolve é responder, com números do próprio código, qual arquitetura funciona melhor para uma EDP ou um conjunto de dados, sem que cada usuário escreva seu próprio laço de treino e avaliação.

O pacote se apoia no resto da biblioteca: os modelos vêm do `ModelRegistry` de `pinneapple_neural` (via `build_model`), as perdas físicas compiladas vêm de `pinneapple_physics` quando o problema tem `physics_preset`, os dados podem vir de `pinneapple_data` (`DatasetConfig`, `DatasetProblem`, `dataset_bench`), e os passos opcionais de problema inverso e de quantificação de incerteza chamam `pinneapple_analysis`. O pacote é acessível de forma preguiçosa como `pp.arena`, mas não é reexportado em `pinneapple/__init__`.

Além do fluxo padrão (treinar tudo e ordenar por erro relativo), há modos extras: `physics_aware_rank` marca modelos com resíduo de EDP fora do grupo sem reordenar o ranking; o modo de decisão (`decision_mode`) usa `pinneapple_decision` para escolher qual modelo treinar a seguir dentro de um orçamento; `nas` faz busca de arquitetura e hiperparâmetros com Optuna sobre o catálogo existente; `regimes` compara métodos de `pinneapple_analysis.surrogate` por regime de dados; `autoresearch` roda um laço automático de experimentos sobre um único `train.py`; e `custom_builder.define_problem` cria problemas novos a partir de uma EDP em texto.

Dois subpacotes servem aplicativos. `pointwise` contém o protocolo, o catálogo de problemas, os dados de referência, as perdas e os adaptadores do app `apps/pinn_arena` (regressão de campo ponto a ponto). `external` é um catálogo de bibliotecas de terceiros de ML para física (DeepXDE, PINA, NeuralOperator, MACE, etc.): cada entrada é só uma referência (repositório, licença, instalação), e os runners importam a biblioteca real na hora, rodam uma chamada mínima sobre dados sintéticos e informam com honestidade o status (`ok`, `not_installed`, `skipped_heavy`, `manual_install`, `failed`). Nada de código de terceiros é copiado.

Números do inventário

módulos	linhas	classes públicas	funções públicas	testes em arquivos que importam o pacote	módulos sem teste direto
54	21.853	69	243	99 (12 arquivos)	39

Submódulos

submódulo	o que faz	módulos	classes + funções	testes*
`arena`	Classe `Arena` (orquestrador do benchmark) e as funções de ranking `rank_by_accuracy` e `physics_aware_rank`.	1	3	8
`autoresearch`	Laço automático de experimentos que edita só o bloco TUNABLE de um `train.py` e mantém as mudanças que melhoram a métrica.	1	8	5
`config`	Dataclasses da configuração da Arena (`ArenaConfig` e seções: problema, modelos, treino, saída, inverso, UQ, dataset, decisão).	1	10	13
`custom_builder`	API declarativa `define_problem` para criar problemas a partir de uma EDP escrita como texto.	1	2	0
`dataset_bench`	Presets de benchmark por dataset de `pinneapple_data` e a função de uma linha `benchmark_dataset`.	1	3	0
`dataset_problems`	`DatasetProblem`: um `ArenaProblem` alimentado por um dataset real de `pinneapple_data`.	1	1	0
`decision_mode`	Modo de decisão: o motor de `pinneapple_decision` escolhe qual modelo configurado treinar a seguir.	1	8	16
`external`	Catálogo de bibliotecas externas de ML para física, com runners de inferência e de fine-tune de demonstração.	28	186	5
`model_factory`	Constrói qualquer modelo suportado a partir de um `ModelConfig`, via `ModelRegistry`.	1	5	10
`nas`	Busca de arquitetura e hiperparâmetros (Optuna TPE ou busca aleatória) sobre o catálogo existente.	1	4	5
`pointwise`	Protocolo, catálogos, dados de referência, perdas e adaptadores do app `apps/pinn_arena`.	11	35	16
`problems`	Registro dos problemas físicos embutidos da Arena, com solução analítica e resíduos PINN.	1	27	15
`regimes`	Arena de regimes: compara métodos por regime de dados sob um protocolo comum.	1	3	19
`showcase`	Demonstração: quatro arquiteturas PINN no mesmo problema de Burgers, ranqueadas por acurácia e pela visão física.	1	2	5
`trainer`	Motor de treino unificado (PINN, supervisionado, operador em grade, grafo), avaliação, UQ e inverso.	1	10	25
`visualizer`	Figuras em tema escuro dos resultados da Arena (campos, curvas de perda, tabela, linhas de corrente, UQ).	1	5	0

* maior número de testes que importam um módulo do submódulo.

Módulos e API pública

`pinneapple_arena`

Pacote raiz: reexporta configuração, `Arena`, rankings, problemas, fábrica de modelos, treino, visualização, `define_problem`, `DatasetProblem`, presets de dataset, modo de decisão e NAS.

`pinneapple_arena/__init__.py` · 157 linhas · testes: 52

pinneapple_arena.arena

Orquestrador principal: prepara dados do problema ou dataset, treina e avalia cada modelo configurado, roda UQ/inverso opcionais, gera figuras e imprime o resumo.

pinneapple_arena/arena.py · 628 linhas · testes: 8

nome	tipo	assinatura	o que faz
Arena	classe	(config, device=None)	Executor do benchmark multimodelo; criado de YAML, JSON ou ArenaConfig, com run, run_decision, physics_aware_summary, results e compare.
rank_by_accuracy	função	(train_results, eval_results, field_names) -> List[Tuple[str, float]]	Ranking padrão: lista (nome, erro relativo médio) em ordem crescente, do melhor para o pior.
physics_aware_rank	função	(train_results, eval_results, field_names, n_std=2.0, residual_threshold=None) -> Dict[st…)	Visão opcional que mantém o ranking por acurácia e marca modelos PINN cujo resíduo de EDP é atípico no grupo (mediana + n_std·desvio) ou passa de residual_threshold.

pinneapple_arena.autoresearch

Modo autoresearch (padrão do karpathy/autoresearch): um proponente edita o bloco TUNABLE de train.py, cada tentativa roda com orçamento de tempo, e só as mudanças que reduzem a métrica ficam; tudo vai para results.tsv.

pinneapple_arena/autoresearch.py · 284 linhas · testes: 5

nome	tipo	assinatura	o que faz
Trial	classe	(index, metric, status, kept, description, seconds, changes)	Registro de uma tentativa: índice, métrica, status (ok, timeout, crash, no_metric), se foi mantida, descrição, duração e mudanças.
RandomSearchProposer	classe	(space, seed=0, max_changes=2)	Proponente sem LLM e reprodutível: sorteia um ou dois parâmetros de um espaço discreto por tentativa.
LLMProposer	classe	(provider='anthropic', model=None, llm_call=None, history_len=15, **llm_kwargs)	Proponente que pede a um LLM (provedor de pinneapple_llm ou llm_call próprio) a próxima edição do bloco TUNABLE, dado program.md e o histórico.
AutoResearch	classe	(workdir, proposer, budget_s=300.0, python=sys.executable)	Laço de experimentos sobre um diretório de trabalho: propõe, aplica, roda, mantém ou reverte, e guarda o melhor script em best_train.py.
read_tunables	função	(code) -> Dict[str, Any]	Lê os valores NOME = valor do bloco # >>> TUNABLE ... # <<< TUNABLE; erro se o bloco não existe.
apply_tunables	função	(code, changes) -> str	Reescreve valores só dentro do bloco TUNABLE; nomes desconhecidos são rejeitados.
run_trial	função	(workdir, budget_s, *, python=sys.executable, grace_s=30.0)	Roda train.py uma vez num subprocesso com orçamento de tempo e devolve (métrica, status, fim do stdout, segundos).
init_workdir	função	(path, *, overwrite=False) -> str	Cria program.md e um train.py modelo (PINN de Poisson 1D com erro relativo contra sin(pi x)).

pinneapple_arena.config

Configuração estruturada da Arena em dataclasses com from_dict, carregável de YAML/JSON e exportável para YAML.

pinneapple_arena/config.py · 329 linhas · testes: 13

nome	tipo	assinatura	o que faz
NetworkConfig	classe	(hidden, activation, width, modes, modes2, layers, use_grid, hidden_dim, n_layers, n_mess…)	Hiperparâmetros de arquitetura (camadas ocultas, ativação, modos FNO, passos de mensagem, omega_0, transformer etc.).
TrainingConfig	classe	(epochs, lr, weight_decay, grad_clip, optimizer, scheduler, batch_size, seed)	Hiperparâmetros de treino de um modelo: épocas, taxa de aprendizado, decaimento, clipping, otimizador, scheduler, lote e semente.
ModelConfig	classe	(name, type, network, training)	Especificação completa de um modelo do benchmark: nome, tipo no registro, rede e treino.

nome	tipo	assinatura	o que faz
`ProblemConfig`	classe	`(name, params, physics_preset, grid_n, n_col, n_bc, n_train_supervised, n_mesh_nodes)`	Escolha do problema físico, parâmetros, preset físico opcional e tamanhos de amostragem (grade, colocação, contorno, nós de malha).
`OutputConfig`	classe	`(dir, prefix, save_figures, dpi, dark_theme, show)`	Configuração de saída: diretório, prefixo, salvar figuras, dpi, tema escuro e exibição.
`InverseConfig`	classe	`(enabled, params, n_obs, noise_std, method, n_iters, lambda_reg, sensor_locations)`	Configuração do problema inverso (parâmetros desconhecidos, observações, ruído, método, iterações, regularização, sensores).
`UQConfig`	classe	`(enabled, method, n_samples, dropout_rate, coverage)`	Configuração da quantificação de incerteza (método, amostras, dropout, cobertura).
`DatasetConfig`	classe	`(dataset_id, input_fields, output_fields, n_train, n_val, split_seed)`	Usa um dataset de `pinneapple_data` no lugar de um problema embutido (campos de entrada/saída, tamanhos de treino/validação, semente).
`DecisionConfig`	classe	`(enabled, budget, stop_when_passed, thresholds, evidence_path, problem)`	Ativa o modo de decisão: orçamento, parar ao passar, limiares, caminho de evidências e fatos do problema.
`ArenaConfig`	classe	`(problem, models, output, inverse, uq, dataset, decision)`	Configuração de topo que junta problema, modelos, saída, inverso, UQ, dataset e decisão; lê de dict, YAML ou JSON e grava YAML.

`pinneapple_arena.custom_builder`

API de alto nível para problemas próprios: a EDP vem como texto (`u_xx`, `u_t`, ...), as condições de contorno como dicts, e perdas, amostragem e pipeline são gerados automaticamente. [sem teste]

`pinneapple_arena/custom_builder.py` · 661 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`EasyArenaProblem`	classe	`(name, description, coords, fields, pde_evaluator, bcs, params, analytical_fn, bc_weight)`	`ArenaProblem` gerado por `define_problem`, com resíduo montado a partir da EDP em texto e método `solve` que roda a Arena.
`define_problem`	função	`(name, coords, fields, pde, bcs, params=None, description='', analytical=None, bc_weight=…)`	Define um problema físico de forma declarativa (coordenadas, campos, EDP em texto, contornos, parâmetros, solução analítica opcional) e o registra.

`pinneapple_arena.dataset_bench`

Tabela `DATASET_PRESETS` com configuração recomendada para cada dataset de `pinneapple_data` e a função `benchmark_dataset` que monta e roda a Arena a partir dela. [sem teste]

`pinneapple_arena/dataset_bench.py` · 723 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`list_benchmarks`	função	`(category=None, verbose=True) -> List[str]`	Imprime e devolve os ids de presets disponíveis, com filtro opcional por categoria.
`get_benchmark_preset`	função	`(dataset_id) -> Dict[str, Any]`	Devolve o dict de preset de um `dataset_id`.
`benchmark_dataset`	função	`(dataset_id, *, models=None, epochs=None, mode=None, n_train=None, n_bc=None, grid_n=None…)`	Roda um benchmark completo da Arena sobre um dataset de `pinneapple_data`, usando o preset e aceitando substituições (modelos, épocas, modo, tamanhos).

`pinneapple_arena.dataset_problems`

`DatasetProblem` converte datasets de `pinneapple_data` (EDP em grade, séries temporais, regressão, geometria) em problemas da Arena, inclusive o modo híbrido `pinn_data`. [sem teste]

`pinneapple_arena/dataset_problems.py` · 520 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`DatasetProblem`	classe	`(dataset_id, name, description, input_fields, output_fields, mode, dataset_params, pinn_r…)`	`ArenaProblem` alimentado por um dataset real; achata grades em `(N, in_dim)` e, no modo `pinn_data`, soma resíduo físico à perda de dados.

pinneapple_arena.decision_mode

A Arena como laboratório de decisão: em vez de treinar tudo, o laço DECIDE/EXECUTA/VALIDA/APRENDE de `pinneapple_decision` escolhe o próximo modelo, valida `rel_l2` e resíduo e para ao passar ou ao fim do orçamento.

`pinneapple_arena/decision_mode.py` · 320 linhas · testes: 16

nome	tipo	assinatura	o que faz
`ArenaExecutor`	classe	`(arena)`	Executor do laço de decisão: treina e avalia o modelo escolhido com o próprio código da Arena.
`ArenaDecisionReport`	classe	`(budget, stop_reason, ran, not_trained, accepted, ranking, evidence, facts)`	Relatório do modo de decisão: orçamento, motivo da parada, modelos rodados e não treinados, aceito, ranking, evidências e fatos.
`catalog_family`	função	`(model_type) -> Optional[str]`	Mapeia um tipo de modelo da Arena para uma chave de `ARCHITECTURE_CATALOG`, ou `None`.
`arena_choice`	função	`(arena) -> Tuple[PhysicsChoice, Dict[str, Optional[str]]]`	Monta a `PhysicsChoice` "arena_model" (opções = nomes dos modelos configurados) e a família de cada um; exige ao menos dois modelos.
`arena_model_rules`	função	`(families)`	Regras da escolha `arena_model`: pontuação do `ModelSelector` da família, menos uma penalidade para famílias que já falharam.
`default_decider`	função	`(families) -> PhysicsDecider`	`PhysicsDecider` padrão com backend de regras (seletores existentes mais as regras de `arena_model`).
`arena_problem_facts`	função	`(arena, overrides=None) -> Dict[str, Any]`	Fatos que a Arena realmente conhece sobre o problema; o resto fica como desconhecido, e `overrides` têm prioridade.
`run_decision_mode`	função	`(arena, *, budget=3, decider=None, verifier=None, store=None, problem=None, stop_when_pas…)`	Roda o laço de decisão sobre os modelos configurados da Arena e devolve um `ArenaDecisionReport`.

pinneapple_arena.external

Catálogo externo: referências a bibliotecas de terceiros de ML para física e a cola que as executa; cada runner importa a biblioteca real e informa um `ModelResult` com status honesto.

`pinneapple_arena/external/__init__.py` · 68 linhas · testes: 5

nome	tipo	assinatura	o que faz
`licence_counts`	função	`() -> dict[str, int]`	Número de entradas do catálogo por licença ("none found" quando o repositório não tem arquivo de licença).

pinneapple_arena.external.__main__

Linha de comando `python -m pinneapple_arena.external` para listar o catálogo e rodar inferência ou fine-tune de demonstração localmente, com saída JSON/CSV. [sem teste]

`pinneapple_arena/external/__main__.py` · 134 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`cmd_list`	função	`(args) -> None`	Subcomando que lista os modelos do catálogo, com filtros por categoria e domínio.
`print_result`	função	`(r) -> None`	Imprime um `ModelResult` de forma legível no terminal.
`cmd_run`	função	`(args) -> None`	Subcomando que roda inferência ou fine-tune nos modelos selecionados e grava relatório JSON/CSV opcional.
`main`	função	`() -> None`	Ponto de entrada da linha de comando do catálogo externo.

pinneapple_arena.external.arch_meta

Rótulos das famílias de arquitetura (campo `arch` de `ModelSpec`). [sem teste]

`pinneapple_arena/external/arch_meta.py` · 16 linhas · **sem teste direto**

pinneapple_arena.external.benchmark

Harness de benchmark/comparação do catálogo externo, chamável pela API para as telas Benchmark e Compare do frontend. [sem teste]

`pinneapple_arena/external/benchmark.py` · 53 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`select_slugs`	função	`(slugs=None, category=None, domain=None) -> list[str]`	Seleciona slugs do catálogo por lista explícita, categoria ou domínio.
`run_benchmark`	função	`(slugs, include_heavy=False) -> dict`	Roda a inferência de demonstração em cada slug e agrega os resultados.
`run_compare`	função	`(slugs, include_heavy=False) -> dict`	Roda o benchmark dos slugs escolhidos e devolve as especificações junto com resultados e resumo, para comparação lado a lado.

pinneapple_arena.external.dataset_probe

Ação "probe" do catálogo de datasets externos: carrega uma amostra real de um subconjunto e relata campos e formas, com o mesmo contrato de honestidade dos modelos. [sem teste]

`pinneapple_arena/external/dataset_probe.py` · 123 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`local_commands`	função	`(spec, subset, split) -> str`	Comandos exatos para baixar e carregar localmente um subconjunto/split de um dataset externo.
`probe_dataset`	função	`(spec, subset, split, stream=False, dataset_root=None) -> ModelResult`	Carrega a amostra 0 de um subconjunto/split, de cópia local (`EXTERNAL_CATALOG_DATASET_ROOT`) ou por streaming, e devolve um `ModelResult`.

pinneapple_arena.external.datasets_data

Fonte única do catálogo de datasets externos (contraparte de `models_data`). [sem teste]

`pinneapple_arena/external/datasets_data.py` · 87 linhas · **sem teste direto**

pinneapple_arena.external.demo_datasets

Geradores de dados sintéticos padronizados, um por forma de problema físico, reutilizados pelas demonstrações dos runners (cada `ModelSpec` aponta um via `dataset_fn`). [sem teste]

`pinneapple_arena/external/demo_datasets.py` · 230 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ds_pde_1d`	função	`(n=64) -> dict`	Pontos de colocação 1D sintéticos para PINNs gerais.
`ds_burgers`	função	`(nx=64, nt=32) -> dict`	Grade (x, t) e condição inicial sintéticas de Burgers 1D.
`ds_grid_2d`	função	`(nx=16, ny=16) -> dict`	Grade 2D genérica para simuladores de campo.
`ds_molecular`	função	`(n_atoms=6) -> dict`	Sistema molecular pequeno (posições e espécies) para potenciais de ML.
`ds_weather_grid`	função	`(nlat=16, nlon=32, nlevels=4) -> dict`	Grade lat/lon/níveis sintética para modelos de tempo e clima.
`ds_em_grid`	função	`(nx=20, ny=20) -> dict`	Grade de permissividade relativa para solvers eletromagnéticos.

nome	tipo	assinatura	o que faz
`ds_seismic`	função	`(nx=32, nz=32) -> dict`	Modelo de velocidade sísmica 2D para propagação de onda.
`ds_spin_lattice`	função	`(n_sites=6) -> dict`	Cadeia de spins pequena para estados quânticos neurais.
`ds_power_system`	função	`() -> dict`	Sistema de potência de 3 barras para modelos de rede elétrica.
`ds_traffic`	função	`(nx=50, nt=50) -> dict`	Grade de densidade de tráfego em (x, t).
`ds_epidemic`	função	`(n_days=60) -> dict`	Curva epidêmica SIR diária.
`ds_combustion`	função	`(n_species=5, n_t=50) -> dict`	Estado de combustão (espécies ao longo do tempo).
`ds_cosmology`	função	`(n_particles=200) -> dict`	Partículas N-corpos para modelos de cosmologia.
`ds_room`	função	`() -> dict`	Dimensões de uma sala retangular (ShoeBox) para acústica.
`ds_graph_mesh`	função	`(n_nodes=30) -> dict`	Grafo de malha em anel para redes de grafos.
`ds_rigidbody`	função	`(n=8) -> dict`	Estado de partículas livres para motores de corpo rígido.
`ds_image_field`	função	`(size=32) -> dict`	Campo escalar em imagem quadrada.
`ds_timeseries`	função	`(n=100) -> dict`	Série temporal genérica.
`ds_pedestrians`	função	`(n=20) -> dict`	Estado de uma multidão de pedestres (posições, velocidades, destinos).
`ds_airfoil`	função	`() -> dict`	Descrição de um aerofólio NACA para modelos aerodinâmicos.
`ds_option_pricing`	função	`(n=64) -> dict`	Grade de Black-Scholes para precificação de opções.
`ds_lorenz`	função	`(n_steps=2000, dt=0.01) -> dict`	Trajetória caótica do Lorenz-63, usada para descobrir (PySINDy) e integrar (Diffrax) a mesma EDO.
`ds_tle`	função	`() -> dict`	Conjunto TLE real e válido da ISS (época ilustrativa), copiado de um TLE público, para propagação orbital.
`ds_quantum_circuit`	função	`(n_qubits=2) -> dict`	Circuito quântico de par de Bell com poucos qubits.
`ds_crystal`	função	`() -> dict`	Célula unitária cúbica sintética em arrays simples (sem pymatgen/ASE); o runner monta a estrutura real se precisar.

`pinneapple_arena.external.domain_meta`

Metadados por domínio físico: rótulo, glifo e descrição. [sem teste]

`pinneapple_arena/external/domain_meta.py` · 56 linhas · **sem teste direto**

`pinneapple_arena.external.finetune`

Runners de fine-tune/retreino, um por modelo instalável via pip: modo "finetune" (laço curto e real de gradiente em CPU) ou "reconfigure" (re-roda o solver com parâmetros do usuário, ou só confirma o import em frameworks que exigem checkpoint). [sem teste]

`pinneapple_arena/external/finetune.py` · 1.013 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`finetune_deepxde`	função	`(params) -> ModelResult`	Retreina a FNN do DeepXDE do problema 1D de demonstração por algumas iterações Adam e registra a curva de perda.
`finetune_neuraloperator`	função	`(params) -> ModelResult`	Treina um FNO do NeuralOperator numa tarefa de brinquedo de aprender um operador filtro passa-baixa.
`finetune_pina`	função	`(params) -> ModelResult`	Treina a `FeedForward` do PINA numa regressão de brinquedo y=x^2.

nome	tipo	assinatura	o que faz
`finetune_sciann`	função	`(params) -> ModelResult`	Treina um `Functional` do SciANN numa regressão de brinquedo y=x^2.
`finetune_jax_md`	função	`(params) -> ModelResult`	Roda passos de minimização de energia FIRE do JAX-MD em esferas macias.
`finetune_ceviche`	função	`(params) -> ModelResult`	Otimiza a grade de permissividade do Ceviche por subida de gradiente com autograd.
`finetune_netket`	função	`(params) -> ModelResult`	Roda iterações VMC do NetKet otimizando um estado quântico neural RBM.
`finetune_pinnicle`	função	`(params) -> ModelResult`	Retreina a FNN do PINNICLE (backend DeepXDE) como substituto de um treino real de geleira.
`finetune_pybamm`	função	`(params) -> ModelResult`	Re-roda a simulação de descarga SPM do PyBaMM com corrente e duração escolhidas.
`finetune_socialforce`	função	`(params) -> ModelResult`	Re-roda o simulador Social Force com o número de pedestres escolhido.
`finetune_phiflow`	função	`(params) -> ModelResult`	Monta uma grade do PhiFlow e roda passos de relaxação de brinquedo.
`finetune_jax_cfd`	função	`(params) -> ModelResult`	Monta uma grade do JAX-CFD e roda passos de difusão explícita.
`finetune_taichi`	função	`(params) -> ModelResult`	Roda lançamentos de um kernel de difusão num campo Taichi.
`finetune_warp`	função	`(params) -> ModelResult`	Roda lançamentos de kernel de integração Euler explícita sobre partículas no NVIDIA Warp.
`finetune_devito`	função	`(params) -> ModelResult`	Monta uma grade Devito e roda passos explícitos de diferenças finitas.
`finetune_deepwave`	função	`(params) -> ModelResult`	Propaga uma onda escalar com Deepwave num modelo de velocidade constante.
`finetune_pyroomacoustics`	função	`(params) -> ModelResult`	Reconstrói uma sala ShoeBox com dimensões escolhidas e recalcula a resposta ao impulso.
`finetune_ml4gw`	função	`(params) -> ModelResult`	Regera formas de onda IMRPhenomD do ml4gw para as massas escolhidas.
`finetune_reservoirflow`	função	`(params) -> ModelResult`	Reconstrói uma grade de reservatório do ReservoirFlow com a permeabilidade escolhida.
`finetune_photontorch`	função	`(params) -> ModelResult`	Reconstrói um `Waveguide` do Photontorch com comprimento e índice efetivo escolhidos.
`finetune_neuralfoil`	função	`(params) -> ModelResult`	Roda o NeuralFoil num aerofólio NACA com ângulo e Reynolds escolhidos.
`finetune_mujoco_mjx`	função	`(params) -> ModelResult`	Roda passos de física MJX num sistema de um corpo com junta livre (pesado: exige `include_heavy`).
`finetune_brax`	função	`(params) -> ModelResult`	Só confirma o import do Brax; a nota explica que um fine-tune real seria treinar uma política de RL.
`finetune_torax`	função	`(params) -> ModelResult`	Só confirma o import do TORAX; um run real exige um CONFIG de transporte completo.
`finetune_physicsnemo`	função	`(params) -> ModelResult`	Treina um FNO do PhysicsNeMo numa tarefa de brinquedo de operador de deslocamento 1D (pesado).
`finetune_mace`	função	`(params) -> ModelResult`	Para MACE: só faz o import real e devolve uma nota; o fine-tune real precisa de dados rotulados e da CLI/checkpoint da própria biblioteca (pesado).
`finetune_nequip`	função	`(params) -> ModelResult`	Para NequIP: só faz o import real e devolve uma nota; o fine-tune real precisa de dados rotulados e da CLI/checkpoint da própria biblioteca (pesado).
`finetune_deepmd_kit`	função	`(params) -> ModelResult`	Para DeePMD-kit: só faz o import real e devolve uma nota; o fine-tune real precisa de dados rotulados e da CLI/checkpoint da própria biblioteca (pesado).
`finetune_schnetpack`	função	`(params) -> ModelResult`	Para SchNetPack: só faz o import real e devolve uma nota; o fine-tune real precisa de dados rotulados e da CLI/checkpoint da própria biblioteca (pesado).
`finetune_neuralgcm`	função	`(params) -> ModelResult`	Para NeuralGCM: só faz o import real e devolve uma nota; o fine-tune real precisa de dados rotulados e da CLI/checkpoint da própria biblioteca (pesado).
`finetune_pysindy`	função	`(params) -> ModelResult`	Reajusta o SINDy do PySINDy com limiar de esparsidade escolhido na trajetória Lorenz-63.
`finetune_diffrax`	função	`(params) -> ModelResult`	Reintegra o sistema de Lorenz com o Diffrax para `rho` e tempo final escolhidos.
`finetune_tensorcircuit_ng`	função	`(params) -> ModelResult`	Roda descida de gradiente por diferenças finitas num circuito do TensorCircuit-NG.

nome	tipo	assinatura	o que faz
`finetune_dsgp4`	função	`(params) -> ModelResult`	Repropaga o TLE da ISS com dSGP4 em instantes escolhidos.
`finetune_chgnet`	função	`(params) -> ModelResult`	Re-roda o CHGNet pré-treinado numa célula cúbica com elemento e parâmetro de rede escolhidos.
`run_finetune`	função	`(slug, params=None) -> ModelResult`	Despacha o fine-tune de um slug; sem rotina registrada, devolve `manual_install` (repositório sem pip) ou `finetune_not_supported` com o comando local.

`pinneapple_arena.external.installer`

Utilitários de estado de instalação usados pelos runners e pela API. [sem teste]

`pinneapple_arena/external/installer.py` · 31 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`is_installed`	função	`(module_name) -> bool`	Diz se um módulo existe sem importá-lo (import de bibliotecas pesadas tem efeitos colaterais).
`pip_install`	função	`(package, timeout=900) -> tuple[bool, str]`	`pip install` de melhor esforço, usado só quando o usuário pede explicitamente (`install=true`).

`pinneapple_arena.external.local_command`

Monta o comando exato que o usuário roda na própria máquina quando o app não consegue executar um modelo (GPU pesada, repositório só git, ou fine-tune não ligado). [sem teste]

`pinneapple_arena/external/local_command.py` · 37 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`local_command_for`	função	`(spec, action='infer', **params) -> str`	Linha de comando de `python -m pinneapple_arena.external` para inferência ou fine-tune, ou os passos de clone e build de um repositório de instalação manual.
`pip_install_command`	função	`(spec) -> str | None`	Comando `pip install` de um `ModelSpec`, ou `None` se não há pacote pip.

`pinneapple_arena.external.models_data`

Fonte única do catálogo de modelos externos (`MODELS`, `MODELS_BY_SLUG`), gerada uma vez a partir do catálogo HTML original e mantida à mão a partir daí. [sem teste]

`pinneapple_arena/external/models_data.py` · 1.302 linhas · **sem teste direto**

`pinneapple_arena.external.result`

Contrato de resultado de todo runner (inferência e fine-tune): `ModelResult`, construtores por status e utilitários. [sem teste]

`pinneapple_arena/external/result.py` · 141 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ModelResult`	classe	`(name, slug, category, domains, repo, status, dataset_name, dataset_description, input_sh…)`	Resultado de um runner: identidade do modelo, status, dataset, formas de entrada/saída, resumo, métricas (inclui `loss_curve` no fine-tune), erro, nota e comando local.
`Timer`	classe	`()`	Gerenciador de contexto que mede o tempo decorrido, legível a qualquer momento, inclusive dentro de um `except`.
`summarize_array`	função	`(arr) -> dict`	Resumo de um array (média, desvio, mínimo, máximo, tamanho) para o campo `output_summary`.
`not_installed`	função	`(name, slug, category, domains, repo, pip_hint, dataset=None, local_command=None) -> Mode…)`	Constrói um `ModelResult` com status `not_installed` e a dica de pip.

nome	tipo	assinatura	o que faz
manual_install	função	(name, slug, category, domains, repo, instructions, dataset=None, local_command=None) -> …)	Constrói um ModelResult com status manual_install e as instruções de clone/build.
skipped_heavy	função	(name, slug, category, domains, repo, pip_hint, dataset=None, local_command=None) -> Mode…)	Constrói um ModelResult com status skipped_heavy (só roda com include_heavy=True).
failed	função	(name, slug, category, domains, repo, exc, duration, dataset=None) -> ModelResult	Constrói um ModelResult com status failed e a exceção real.
ok	função	(name, slug, category, domains, repo, dataset, duration, output_shape=None, output_summar…)	Constrói um ModelResult com status ok, formas, resumo, nota e métricas da chamada real.

pinneapple_arena.external.runners

Subpacote dos runners de inferência por área (arquivo __init__ vazio). [sem teste]

pinneapple_arena/external/runners/__init__.py · 1 linhas · **sem teste direto**

pinneapple_arena.external.runners.applied_domains

Runners de domínios aplicados (acústica, epidemiologia, tráfego, energia elétrica, ondas gravitacionais, prognóstico, multidões, combustão, aerodinâmica, finanças); a maioria é código de artigo só com instalação manual. Cada função devolve um ModelResult. [sem teste]

pinneapple_arena/external/runners/applied_domains.py · 236 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
infer_simvascular	função	(ds) -> ModelResult	SimVascular / svFSI: sem pacote pip utilizável aqui; só devolve manual_install com as instruções de instalação, sem executar nada.
infer_pyroomacoustics	função	(ds) -> ModelResult	Pyroomacoustics: monta uma sala ShoeBox com fonte e microfone e calcula a resposta ao impulso.
infer_rirpinn	função	(ds) -> ModelResult	RIRPINN: sem pacote pip utilizável aqui; só devolve manual_install com as instruções de instalação, sem executar nada.
infer_epipinn	função	(ds) -> ModelResult	EpiPINN: sem pacote pip utilizável aqui; só devolve manual_install com as instruções de instalação, sem executar nada.
infer_pise	função	(ds) -> ModelResult	PISE (Physics-Informed Traffic State Estimation): sem pacote pip utilizável aqui; só devolve manual_install com as instruções de instalação, sem executar nada.
infer_pinn_ac_opf	função	(ds) -> ModelResult	PINN for AC Optimal Power Flow: sem pacote pip utilizável aqui; só devolve manual_install com as instruções de instalação, sem executar nada.
infer_piml_converter	função	(ds) -> ModelResult	PIML Converter: sem pacote pip utilizável aqui; só devolve manual_install com as instruções de instalação, sem executar nada.
infer_ml4gw	função	(ds) -> ModelResult	ml4gw: gera formas de onda IMRPhenomD de binárias de buracos negros numa grade de frequência.
infer_physics_informed_am	função	(ds) -> ModelResult	Physics-Informed AM (hybrid thermal model): sem pacote pip utilizável aqui; só devolve manual_install com as instruções de instalação, sem executar nada.
infer_pinn_swe_open	função	(ds) -> ModelResult	PINN_SWE_open: sem pacote pip utilizável aqui; só devolve manual_install com as instruções de instalação, sem executar nada.
infer_pinn_corrosion_fatigue	função	(ds) -> ModelResult	PINN for Corrosion-Fatigue Prognosis: sem pacote pip utilizável aqui; só devolve manual_install com as instruções de instalação, sem executar nada.
infer_pinn_wind_bearing	função	(ds) -> ModelResult	PINN for Wind Turbine Main Bearing Fatigue: sem pacote pip utilizável aqui; só devolve manual_install com as instruções de instalação, sem executar nada.
infer_pinnicle	função	(ds) -> ModelResult	PINNICLE: confirma o import com o backend DeepXDE e monta uma FNN pequena sobre dados sintéticos.
infer_physnet_thermal	função	(ds) -> ModelResult	PhysNet Thermal Models: sem pacote pip utilizável aqui; só devolve manual_install com as instruções de instalação, sem executar nada.
infer_neuralsolvers	função	(ds) -> ModelResult	NeuralSolvers: sem pacote pip utilizável aqui; só devolve manual_install com as instruções de instalação, sem executar nada.
infer_pinn_multivp	função	(ds) -> ModelResult	PINN-MULTIVP: sem pacote pip utilizável aqui; só devolve manual_install com as instruções de instalação, sem executar nada.

nome	tipo	assinatura	o que faz
`infer_socialforce`	função	`(ds) -> ModelResult`	Social Force: simula alguns passos de atualização de uma multidão de pedestres.
`infer_spdiff`	função	`(ds) -> ModelResult`	SPDiff: sem pacote pip utilizável aqui; só devolve `manual_install` com as instruções de instalação, sem executar nada.
`infer_stiff_pinn`	função	`(ds) -> ModelResult`	Stiff-PINN: sem pacote pip utilizável aqui; só devolve `manual_install` com as instruções de instalação, sem executar nada.
`infer_crnn`	função	`(ds) -> ModelResult`	CRNN (Chemical Reaction Neural Network): sem pacote pip utilizável aqui; só devolve `manual_install` com as instruções de instalação, sem executar nada.
`infer_neuralfoil`	função	`(ds) -> ModelResult`	NeuralFoil: roda a inferência aerodinâmica num aerofólio NACA 2412.
`infer_pinn_option_pricing`	função	`(ds) -> ModelResult`	PINN Option Pricing: sem pacote pip utilizável aqui; só devolve `manual_install` com as instruções de instalação, sem executar nada.

`pinneapple_arena.external.runners.dynamics_robotics_astro`

Runners de dinâmica não linear e identificação de sistemas, motores diferenciáveis de robótica e modelos astro (fluidos astrofísicos, órbitas, oceano). Cada função devolve um `ModelResult`. [sem teste]

`pinneapple_arena/external/runners/dynamics_robotics_astro.py` · 210 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`infer_pysindy`	função	`(ds) -> ModelResult`	PySINDy: ajusta um modelo SINDy à trajetória Lorenz-63 e relata as equações descobertas.
`infer_diffrax`	função	`(ds) -> ModelResult`	Diffrax: integra a EDO de Lorenz-63 com o solver Dopri5.
`infer_genesis`	função	`(ds, include_heavy=False) -> ModelResult`	Genesis: só confirma que a biblioteca importa e devolve a versão; a nota explica o que um run real exigiria. Pesado: pulado sem `include_heavy=True`.
`infer_nimble_physics`	função	`(ds) -> ModelResult`	Nimble Physics: monta um mundo vazio com gravidade e lê o vetor de estado inicial.
`infer_dojo_jl`	função	`(ds) -> ModelResult`	Dojo.jl: sem pacote pip utilizável aqui; só devolve `manual_install` com as instruções de instalação, sem executar nada.
`infer_astronomix`	função	`(ds) -> ModelResult`	astronomix (jf1uids): só confirma que a biblioteca importa e devolve a versão; a nota explica o que um run real exigiria.
`infer_dsgp4`	função	`(ds) -> ModelResult`	dSGP4: propaga o TLE da ISS em alguns instantes.
`infer_veros`	função	`(ds) -> ModelResult`	Veros: só confirma que a biblioteca importa e devolve a versão; a nota explica o que um run real exigiria.
`infer_diffpd`	função	`(ds) -> ModelResult`	DiffPD: sem pacote pip utilizável aqui; só devolve `manual_install` com as instruções de instalação, sem executar nada.
`infer_difftaichi`	função	`(ds) -> ModelResult`	DiffTaichi: sem pacote pip utilizável aqui; só devolve `manual_install` com as instruções de instalação, sem executar nada.
`infer_diffsph`	função	`(ds) -> ModelResult`	diffSPH: só confirma que a biblioteca importa e devolve a versão; a nota explica o que um run real exigiria.

`pinneapple_arena.external.runners.em_photonics`

Runners de eletromagnetismo e fotônica (Ceviche, Photontorch, hPINN). Cada função devolve um `ModelResult`. [sem teste]

`pinneapple_arena/external/runners/em_photonics.py` · 68 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`infer_ceviche`	função	`(ds) -> ModelResult`	Ceviche: resolve `fdfd_ez` com fonte pontual e devolve o campo Ez complexo.
`infer_photontorch`	função	`(ds) -> ModelResult`	Photontorch: monta um componente `Waveguide` simples.
`infer_hpinn`	função	`(ds) -> ModelResult`	hPINN: sem pacote pip utilizável aqui; só devolve `manual_install` com as instruções de instalação, sem executar nada.

pinneapple_arena.external.runners.fluids_structural

Runners de fluidos e estruturas (simuladores diferenciáveis e redes de grafos de malha); muitos são só instalação manual. Cada função devolve um `ModelResult`. [sem teste]

`pinneapple_arena/external/runners/fluids_structural.py` · 163 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`infer_phiflow`	função	`(ds) -> ModelResult`	PhiFlow: monta uma `CenteredGrid` 2D e lê seus valores.
`infer_jax_cfd`	função	`(ds) -> ModelResult`	JAX-CFD: monta uma `Grid`; a nota avisa que um `step()` completo exige campo de velocidade inicial.
`infer_jax_fluids`	função	`(ds) -> ModelResult`	JAX-Fluids: sem pacote pip utilizável aqui; só devolve `manual_install` com as instruções de instalação, sem executar nada.
`infer_deepflame`	função	`(ds) -> ModelResult`	DeepFlame: sem pacote pip utilizável aqui; só devolve `manual_install` com as instruções de instalação, sem executar nada.
`infer_fluidx3d`	função	`(ds) -> ModelResult`	FluidX3D: sem pacote pip utilizável aqui; só devolve `manual_install` com as instruções de instalação, sem executar nada.
`infer_lesnets`	função	`(ds) -> ModelResult`	LESnets: sem pacote pip utilizável aqui; só devolve `manual_install` com as instruções de instalação, sem executar nada.
`infer_twophasepinn`	função	`(ds) -> ModelResult`	twophasePINN: sem pacote pip utilizável aqui; só devolve `manual_install` com as instruções de instalação, sem executar nada.
`infer_meshgraphnets_pytorch`	função	`(ds) -> ModelResult`	MeshGraphNets (PyTorch port): sem pacote pip utilizável aqui; só devolve `manual_install` com as instruções de instalação, sem executar nada.
`infer_meshgraphnets_original`	função	`(ds) -> ModelResult`	MeshGraphNets (original, DeepMind): sem pacote pip utilizável aqui; só devolve `manual_install` com as instruções de instalação, sem executar nada.
`infer_gns`	função	`(ds) -> ModelResult`	GNS — Graph Network Simulator: sem pacote pip utilizável aqui; só devolve `manual_install` com as instruções de instalação, sem executar nada.
`infer_graingnn`	função	`(ds) -> ModelResult`	GrainGNN: sem pacote pip utilizável aqui; só devolve `manual_install` com as instruções de instalação, sem executar nada.
`infer_taichi`	função	`(ds) -> ModelResult`	Taichi: preenche um campo 2D com um kernel na CPU.
`infer_warp`	função	`(ds) -> ModelResult`	NVIDIA Warp: lança um kernel que soma 1 a um array na CPU.
`infer_dl4to`	função	`(ds) -> ModelResult`	DL4TO: sem pacote pip utilizável aqui; só devolve `manual_install` com as instruções de instalação, sem executar nada.

pinneapple_arena.external.runners.geophysics_energy

Runners de geofísica e energia (Devito, Deepwave, TORAX, PyBaMM, ReservoirFlow). Cada função devolve um `ModelResult`. [sem teste]

`pinneapple_arena/external/runners/geophysics_energy.py` · 111 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`infer_devito`	função	`(ds) -> ModelResult`	Devito: monta `Grid` e `Function` e carrega o modelo de velocidade.
`infer_deepwave`	função	`(ds) -> ModelResult`	Deepwave: monta o tensor do modelo de velocidade pronto para propagação.
`infer_torax`	função	`(ds, include_heavy=False) -> ModelResult`	TORAX: só confirma que a biblioteca importa e devolve a versão; a nota explica o que um run real exigiria. Pesado: pulado sem `include_heavy=True`.
`infer_pybamm`	função	`(ds) -> ModelResult`	PyBaMM: monta e resolve uma descarga de Single Particle Model.
`infer_reservoirflow`	função	`(ds) -> ModelResult`	ReservoirFlow: monta uma grade 2D pequena de reservatório.

pinneapple_arena.external.runners.materials2_and_misc

Segunda leva de potenciais de materiais e outros domínios (biomoléculas, circuitos quânticos, operadores neurais novos, esteira de turbinas, incêndios). Cada função devolve um `ModelResult`. [sem teste]

pinneapple_arena/external/runners/materials2_and_misc.py · 202 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
infer_openfold	função	(ds) -> ModelResult	OpenFold: sem pacote pip utilizável aqui; só devolve manual_install com as instruções de instalação, sem executar nada.
infer_tensorcircuit_ng	função	(ds) -> ModelResult	TensorCircuit-NG: monta um circuito de preparação GHZ e calcula e o estado.
infer_transolver	função	(ds) -> ModelResult	Transolver: sem pacote pip utilizável aqui; só devolve manual_install com as instruções de instalação, sem executar nada.
infer_u_no	função	(ds) -> ModelResult	U-NO: sem pacote pip utilizável aqui; só devolve manual_install com as instruções de instalação, sem executar nada.
infer_torchmd	função	(ds) -> ModelResult	TorchMD: só confirma que a biblioteca importa e devolve a versão; a nota explica o que um run real exigiria.
infer_fairchem	função	(ds, include_heavy=False) -> ModelResult	FAIRChem (Open Catalyst Project): só confirma que a biblioteca importa e devolve a versão; a nota explica o que um run real exigiria. Pesado: pulado sem include_heavy=True.
infer_mattersim	função	(ds, include_heavy=False) -> ModelResult	MatterSim: só confirma que a biblioteca importa e devolve a versão; a nota explica o que um run real exigiria. Pesado: pulado sem include_heavy=True.
infer_chgnet	função	(ds) -> ModelResult	CHGNet: carrega o checkpoint pré-treinado e prevê energia e forças de uma célula sintética.
infer_orb_models	função	(ds) -> ModelResult	Orb models: só confirma que a biblioteca importa e devolve a versão; a nota explica o que um run real exigiria.
infer_grainlearning	função	(ds) -> ModelResult	GrainLearning: só confirma que a biblioteca importa e devolve a versão; a nota explica o que um run real exigiria.
infer_wakenet	função	(ds) -> ModelResult	WakeNet: sem pacote pip utilizável aqui; só devolve manual_install com as instruções de instalação, sem executar nada.
infer_bpinn_wildfire	função	(ds) -> ModelResult	BPINN-Wildfire: sem pacote pip utilizável aqui; só devolve manual_install com as instruções de instalação, sem executar nada.

pinneapple_arena.external.runners.materials_md

Runners de dinâmica molecular e potenciais interatômicos (JAX-MD, MACE, NequIP, Allegro, DeePMD-kit, SchNetPack). Cada função devolve um ModelResult. [sem teste]

pinneapple_arena/external/runners/materials_md.py · 121 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
infer_jax_md	função	(ds) -> ModelResult	JAX-MD: monta o par de funções de deslocamento em espaço livre e aplica shift_fn.
infer_mace	função	(ds, include_heavy=False) -> ModelResult	MACE: só confirma que a biblioteca importa e devolve a versão; a nota explica o que um run real exigiria. Pesado: pulado sem include_heavy=True.
infer_nequip	função	(ds, include_heavy=False) -> ModelResult	NequIP: só confirma que a biblioteca importa e devolve a versão; a nota explica o que um run real exigiria. Pesado: pulado sem include_heavy=True.
infer_allegro	função	(ds) -> ModelResult	Allegro: sem pacote pip utilizável aqui; só devolve manual_install com as instruções de instalação, sem executar nada.
infer_deepmd_kit	função	(ds, include_heavy=False) -> ModelResult	DeePMD-kit: só confirma que a biblioteca importa e devolve a versão; a nota explica o que um run real exigiria. Pesado: pulado sem include_heavy=True.
infer_schnetpack	função	(ds, include_heavy=False) -> ModelResult	SchNetPack: monta um módulo de representação SchNet (pesado: exige include_heavy).

pinneapple_arena.external.runners.pinn_general

Runners de bibliotecas gerais de PINN e operadores neurais (DeepXDE, PINA, PhysicsNeMo, SciANN, NeuralOperator, DeepONet, PINO). Cada função devolve um ModelResult. [sem teste]

pinneapple_arena/external/runners/pinn_general.py · 167 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`infer_deepxde`	função	`(ds) -> ModelResult`	DeepXDE: monta um problema de Poisson 1D com FNN pequena, compila e faz `predict` (sem treino).
`infer_pina`	função	`(ds) -> ModelResult`	PINA: faz uma passada direta de uma `FeedForward` pequena.
`infer_physicsnemo`	função	`(ds, include_heavy=False) -> ModelResult`	NVIDIA PhysicsNeMo: só confirma que a biblioteca importa e devolve a versão; a nota explica o que um run real exigiria. Pesado: pulado sem `include_heavy=True`.
`infer_sciann`	função	`(ds) -> ModelResult`	SciANN: monta um `SciModel` pequeno e faz `predict`.
`infer_neuralpde_jl`	função	`(ds) -> ModelResult`	NeuralPDE.jl: sem pacote pip utilizável aqui; só devolve `manual_install` com as instruções de instalação, sem executar nada.
`infer_pinns_torch`	função	`(ds) -> ModelResult`	PINNs-Torch: sem pacote pip utilizável aqui; só devolve `manual_install` com as instruções de instalação, sem executar nada.
`infer_pinns_original`	função	`(ds) -> ModelResult`	PINNs (original reference code): sem pacote pip utilizável aqui; só devolve `manual_install` com as instruções de instalação, sem executar nada.
`infer_pinn_tfi_hss`	função	`(ds) -> ModelResult`	PINN_TFI-HSS: sem pacote pip utilizável aqui; só devolve `manual_install` com as instruções de instalação, sem executar nada.
`infer_neuraloperator`	função	`(ds) -> ModelResult`	NeuralOperator: faz uma passada direta de um FNO 1D pequeno.
`infer_deeponet_original`	função	`(ds) -> ModelResult`	DeepONet (original): sem pacote pip utilizável aqui; só devolve `manual_install` com as instruções de instalação, sem executar nada.
`infer_deeponet_fno_benchmark`	função	`(ds) -> ModelResult`	DeepONet-FNO benchmark: sem pacote pip utilizável aqui; só devolve `manual_install` com as instruções de instalação, sem executar nada.
`infer_pino`	função	`(ds) -> ModelResult`	PINO (Physics-Informed Neural Operator): sem pacote pip utilizável aqui; só devolve `manual_install` com as instruções de instalação, sem executar nada.

`pinneapple_arena.external.runners.quantum_cosmology`

Runners de física quântica e cosmologia (NetKet, FermiNet, FlowPM/JaxPM, CAMELS). Cada função devolve um `ModelResult`. [sem teste]

`pinneapple_arena/external/runners/quantum_cosmology.py` · 68 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`infer_netket`	função	`(ds) -> ModelResult`	NetKet: monta um estado quântico neural RBM com amostrador Metropolis e tira amostras MC.
`infer_fermine t`	função	`(ds) -> ModelResult`	FermiNet: sem pacote pip utilizável aqui; só devolve `manual_install` com as instruções de instalação, sem executar nada.
`infer_flowpm_jaxpm`	função	`(ds) -> ModelResult`	FlowPM / JaxPM: sem pacote pip utilizável aqui; só devolve `manual_install` com as instruções de instalação, sem executar nada.
`infer_camels`	função	`(ds) -> ModelResult`	CAMELS: sem pacote pip utilizável aqui; só devolve `manual_install` com as instruções de instalação, sem executar nada.

`pinneapple_arena.external.runners.registry`

Tabela de despacho que liga cada slug ao par (gerador de dados, função `infer_*`), usada pela API para inferência, benchmark e comparação. [sem teste]

`pinneapple_arena/external/runners/registry.py` · 268 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`call_infer`	função	`(slug, include_heavy=False) -> ModelResult`	Monta o dataset de demonstração e roda a função `infer_*` real do slug, passando `include_heavy` só a quem aceita; resultados não-ok recebem o comando local.
`get_dataset_preview`	função	`(slug) -> dict`	Metadados e prévia JSON dos arrays do dataset de demonstração de um slug, sem rodar inferência.

pinneapple_arena.external.runners.rigidbody_robotics

Runners de corpo rígido e robótica (Brax, MuJoCo MJX), ambos pesados. Cada função devolve um `ModelResult`. [sem teste]

`pinneapple_arena/external/runners/rigidbody_robotics.py` · 65 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`infer_brax`	função	`(ds, include_heavy=False) -> ModelResult`	Brax: só confirma que a biblioteca importa e devolve a versão; a nota explica o que um run real exigiria. Pesado: pulado sem `include_heavy=True`.
`infer_mujoco_mjx`	função	`(ds, include_heavy=False) -> ModelResult`	MuJoCo MJX: monta um modelo de um corpo e roda um `mjx.forward()` (pesado: exige `include_heavy`).

pinneapple_arena.external.runners.weather_climate

Runners de tempo e clima (GraphCast, Pangu-Weather, FourCastNet só manuais; NeuralGCM pesado). Cada função devolve um `ModelResult`. [sem teste]

`pinneapple_arena/external/runners/weather_climate.py` · 62 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`infer_graphcast`	função	`(ds) -> ModelResult`	GraphCast: sem pacote pip utilizável aqui; só devolve `manual_install` com as instruções de instalação, sem executar nada.
`infer_pangu_weather`	função	`(ds) -> ModelResult`	Pangu-Weather: sem pacote pip utilizável aqui; só devolve `manual_install` com as instruções de instalação, sem executar nada.
`infer_fourcastnet`	função	`(ds) -> ModelResult`	FourCastNet: sem pacote pip utilizável aqui; só devolve `manual_install` com as instruções de instalação, sem executar nada.
`infer_neuralgcm`	função	`(ds, include_heavy=False) -> ModelResult`	NeuralGCM: só confirma que a biblioteca importa e devolve a versão; a nota explica o que um run real exigiria. Pesado: pulado sem `include_heavy=True`.

pinneapple_arena.external.schema

Modelo de dados do catálogo externo: `ModelSpec`, `DatasetSpec` e metadados de domínio e família de arquitetura. [sem teste]

`pinneapple_arena/external/schema.py` · 111 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ModelSpec`	classe	`(slug, name, org, arch, domains, desc, tags, repo, ref, pip_package, import_name, heavy, …)`	Uma entrada do catálogo de modelos: slug, nome, organização, família, domínios, repositório, pacote pip, nome de import, se é pesado ou só manual, gerador de dados, licença e citação.
`DomainMeta`	classe	`(key, label, glyph, desc)`	Metadados de um domínio físico: chave, rótulo, glifo e descrição.
`ArchMeta`	classe	`(key, label)`	Metadados de uma família de arquitetura: chave e rótulo.
`SubDataset`	classe	`(name, label, domains, dims, desc)`	Um subconjunto de um dataset externo: nome, rótulo, domínios, dimensões e descrição.
`DatasetSpec`	classe	`(slug, name, org, desc, tags, repo, ref, size, subsets, splits, pip_package, import_name,…)`	Uma entrada do catálogo de datasets externos: repositório, tamanho, subconjuntos, splits, pacote, streaming, comando de download e licença.

pinneapple_arena.external.status

Checagem barata de estado de instalação, sem inferência, para o selo de cada cartão do catálogo. [sem teste]

`pinneapple_arena/external/status.py` · 29 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`install_state`	função	`(spec) -> str`	Estado de instalação de um `ModelSpec` (`installed`, `not_installed`, `heavy` ou `manual_only`), sem importar nem rodar o modelo.

pinneapple_arena.model_factory

Constrói qualquer modelo suportado pela Arena a partir de um `ModelConfig`, via `ModelRegistry.build()` de `pinneapple_neural`, com construtores fixos de reserva para os tipos mais comuns.

`pinneapple_arena/model_factory.py` · 324 linhas · testes: 10

nome	tipo	assinatura	o que faz
`is_graph_model`	função	`(cfg) -> bool`	Diz se o tipo do modelo é de grafo (MeshGraphNet, GNN etc.).
`is_pinn_model`	função	`(cfg) -> bool`	Diz se o tipo do modelo é da família PINN (treinado por resíduo físico).
`is_inverse_model`	função	`(cfg) -> bool`	Diz se o tipo do modelo é de problema inverso.
`is_operator_model`	função	`(cfg) -> bool`	Diz se o tipo do modelo é um operador neural (FNO, DeepONet etc.).
`build_model`	função	`(cfg, in_dim=2, out_dim=1, edge_in_dim=0) -> nn.Module`	Instancia o modelo de um `ModelConfig` com as dimensões de entrada/saída (e de aresta para grafos).

pinneapple_arena.nas

Busca conjunta de arquitetura, hiperparâmetros e taxa de aprendizado sobre o catálogo do `ModelRegistry`, com Optuna (TPE + poda pela mediana) ou busca aleatória; cada tentativa treina um modelo do zero, não é NAS diferenciável.

`pinneapple_arena/nas.py` · 447 linhas · testes: 5

nome	tipo	assinatura	o que faz
`ArchitectureCandidate`	classe	`(name, param_space, build_kwargs_fn)`	Uma arquitetura buscável: chave no `ModelRegistry`, seu espaço de hiperparâmetros e a função que monta os kwargs de construção.
`ArchitectureSearchSpace`	classe	`(candidates, lr_param_space)`	Arquiteturas candidatas, com o espaço de cada uma e uma faixa de taxa de aprendizado comum.
`NASResult`	classe	`(best_architecture, best_hyperparams, best_score, all_trials, study)`	Resultado da busca: melhor arquitetura, melhores hiperparâmetros, melhor pontuação, todas as tentativas e o `optuna.Study` bruto.
`search_architecture`	função	`(problem, search_space=None, n_trials=20, physics_aware=False, lam=1.0, epochs_per_trial=…)`	Roda a busca sobre um `ArenaProblem`, treinando cada tentativa com `train_pinn` e pontuando pelo erro (mais resíduo físico se `physics_aware`).

pinneapple_arena.pointwise

Benchmarks de campo ponto a ponto do app `apps/pinn_arena`: coordenadas `(N, in_dim)` entram, campos `(N, out_dim)` saem; reúne catálogo de problemas, dados de referência, perdas, adaptadores, treino e leaderboard.

`pinneapple_arena/pointwise/__init__.py` · 71 linhas · testes: 16

pinneapple_arena.pointwise.adapters

Adaptadores que fazem arquiteturas de grade ou sequência aceitarem dados ponto a ponto, tratando cada ponto como amostra própria (`PointwiseAdapter`, `ClosedFormAdapter`, lista `NOT_APPLICABLE`).

`pinneapple_arena/pointwise/adapters.py` · 181 linhas · testes: 16

nome	tipo	assinatura	o que faz
`ArchitectureNotApplicable`	classe	`()`	Exceção: a arquitetura não pode ser treinada em dados coordenada→campo (por exemplo modelos não supervisionados de trajetória).

nome	tipo	assinatura	o que faz
`adapt_model`	função	`(arch_id, model, in_dim, out_dim) -> Any`	Envolve o modelo para aceitar `(N, in_dim)` e devolver `(N, out_dim)`.
`check_applicable`	função	`(arch_id) -> None`	Levanta `ArchitectureNotApplicable` se a arquitetura está na lista de não aplicáveis.

`pinneapple_arena.pointwise.architectures`

Catálogo `ARCHITECTURES_CATALOG` das arquiteturas do registro que o app pode escolher, com descrições; algumas entradas têm chave renomeada no registro. [sem teste]

`pinneapple_arena/pointwise/architectures.py` · 1.722 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`registry_key`	função	`(arch_id) -> str`	Chave de `pp.build_model` de uma entrada do catálogo (trata os renomes `symm_*`→`noether_*` e `icosahedral_gnn`→`graphcast`).
`catalog_status`	função	`() -> dict[str, bool]`	`{id: registrado}`: se o registro do PINNeAPPle tem o modelo de cada entrada.

`pinneapple_arena.pointwise.datasets`

Dados de referência dos problemas do app: solução analítica quando existe, senão solução numérica própria; alguns campos (colisão, aerofólio 3D, esteira de cilindro) são ilustrativos ou de ordem reduzida, e o metadado diz isso. [sem teste]

`pinneapple_arena/pointwise/datasets.py` · 3.180 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`DatasetGenerator`	classe	`(problem_id, parameters)`	Gera a solução de referência de um problema: `generate(n)` devolve `coords`, `solution` e `metadata` com a origem da solução.

`pinneapple_arena.pointwise.geometries`

Catálogo `GEOMETRIES_CATALOG` de geometrias 3D prontas para experimentos 3D. [sem teste]

`pinneapple_arena/pointwise/geometries.py` · 503 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`get_geometry_mesh_config`	função	`(geometry_id, resolution) -> dict`	Dict de configuração de malha de um preset de geometria numa resolução, compatível com os construtores de geometria.

`pinneapple_arena.pointwise.leaderboard`

Ranking global de arquiteturas sobre muitos experimentos do app, com métricas médias e pontuações 0-100 por objetivo. [sem teste]

`pinneapple_arena/pointwise/leaderboard.py` · 212 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`aggregate`	função	`(experiments, families=None) -> dict`	Agrega experimentos concluídos numa entrada por arquitetura, ordenada por L2 relativo médio; custo, memória e velocidade são relativos ao conjunto da chamada.

`pinneapple_arena.pointwise.losses`

Perdas de resíduo de EDP dos problemas do app: primeiro resíduo nativo em autograd com os parâmetros do experimento, depois o preset compilado de `pinneapple_physics`, senão nenhuma (só dados). [sem teste]

`pinneapple_arena/pointwise/losses.py` · 732 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`PhysicsLossGenerator`	classe	`(problem_id, parameters)`	Compila a perda física (resíduo de EDP) de um problema e descreve qual fonte foi usada.

`pinneapple_arena.pointwise.problems`

Catálogo `PROBLEMS_CATALOG` de problemas físicos do app, com equações, parâmetros, condições de contorno/iniciais e a chave de preset quando existe. [sem teste]

`pinneapple_arena/pointwise/problems.py` · 1.875 linhas · **sem teste direto**

`pinneapple_arena.pointwise.schema`

Descritores pydantic v2 dos catálogos do app: problemas, presets de geometria e arquiteturas. [sem teste]

`pinneapple_arena/pointwise/schema.py` · 175 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`PhysicsArea`	classe	`()`	Enum das áreas físicas dos problemas.
`Dimension`	classe	`()`	Enum da dimensão do problema (1D, 2D, 3D, com ou sem tempo).
`DifficultyLevel`	classe	`()`	Enum do nível de dificuldade do problema.
`ArchitectureFamily`	classe	`()`	Enum das famílias de arquitetura do catálogo.
`GeometryType`	classe	`()`	Enum dos tipos de geometria 3D.
`ProblemParameter`	classe	`(name, default, min_val, max_val, description, unit)`	Parâmetro escalar configurável de um problema: nome, padrão, limites, descrição e unidade.
`ColumnHints`	classe	`(coord_cols, output_cols, time_col, csv_header, notes)`	Layout esperado de colunas de um dataset do problema, para guiar uploads CSV/NPZ/JSON.
`PhysicsProblem`	classe	`(id, name, description, domain, dimension, equations, applications, initial_conditions, b…)`	Descritor completo de um problema físico do catálogo (equações, aplicações, condições, parâmetros, dificuldade, preset).
`GeometryPreset`	classe	`(id, name, description, geometry_type, dimensions, supported_problems, mesh_resolution_de…)`	Geometria 3D pré-montada disponível para experimentos 3D.
`MeshInfo`	classe	`(n_points, n_elements, bounds, quality_score)`	Estatísticas resumidas de uma malha gerada (pontos, elementos, limites, qualidade).
`ArchitectureInfo`	classe	`(id, name, family, description, detailed_description, supports_physics_loss, supports_3d,…)`	Metadados de uma arquitetura do catálogo (família, descrição, suporte a perda física e 3D, uso, referência).

`pinneapple_arena.pointwise.tables`

Leitura de tabelas numéricas enviadas ao app (CSV/TSV/TXT com separador detectado, JSON de registros, NPZ).

`pinneapple_arena/pointwise/tables.py` · 111 linhas · testes: 16

nome	tipo	assinatura	o que faz
`parse_numeric_table`	função	`(filename, content) -> tuple[list[str], np.ndarray, int]`	Devolve `(colunas, array float32, linhas puladas)` de uma tabela enviada, com formato pelo sufixo do arquivo.

`pinneapple_arena.pointwise.training`

Protocolo de treino e pontuação de uma arquitetura no app: perda de dados mais resíduo de EDP balanceado em pontos de colocação, LR cosseno, parada antecipada na validação e nota no teste; nunca treina um substituto com o nome da arquitetura. [sem teste]

pinneapple_arena/pointwise/training.py · 706 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
ArchitectureUnavailable	classe	()	Exceção: o registro não conseguiu construir a arquitetura para os dados deste problema; ela é reportada como falha.
best_device	função	() -> str	Melhor dispositivo disponível (cuda > mps > cpu) como string.
device_info	função	() -> dict	Dispositivo de cálculo e, para CUDA, nome, memória e versão.
field_metrics	função	(y_pred, y_true) -> dict	Métricas de erro da previsão contra a referência, agrupadas sobre pontos e campos (l2_relative, l1, etc.).
prediction_quality	função	(y_pred, y_true, metrics) -> dict	Heurística que detecta saídas degeneradas (constante, zero, NaN) e dá uma nota de qualidade; não é checagem física.
problem_in_dim	função	(problem_id, parameters=None) -> int	Número de coordenadas de entrada de um problema.
problem_out_dim	função	(problem_id) -> int	Número de campos de saída de um problema.
instantiate_model	função	(arch_id, in_dim, out_dim) -> Any	Constrói a arquitetura pelo registro (pp.build_model) para (N, in_dim) -> (N, out_dim), com adaptadores quando preciso.
model_forward	função	(model, X_t) -> Any	Passada direta unificada que lida com os vários tipos de retorno dos modelos.
train_torch_model	função	(model, X_train, y_train, X_val, y_val, X_col, physics_loss_fn, n_epochs, lr, device, pro…)	Treina um modelo torch com perda de dados e resíduo físico opcional, parada antecipada e restauração dos melhores pesos.
train_closed_form_model	função	(model, X_train, y_train, X_val, y_val, progress_callback=None) -> dict[str, Any]	Treina por fit() um modelo de ClosedFormAdapter (ELM, RBF híbrida, leitura de ESN).
run_inverse_solver	função	(model, problem_id, inverse_cfg, coords, solution, device) -> dict	Envolve o modelo no InverseProblemSolver e chama invert() para recuperar parâmetros da EDP.
predict	função	(model, X, device='cpu') -> tuple[np.ndarray, float]	Previsão de um modelo treinado sem gradientes e o tempo em milissegundos.
rank_models	função	(results) -> list[dict]	Ordena resultados concluídos por L2 relativo (rank 1..n); os que falharam ficam depois, com rank None.

pinneapple_arena.problems

Registro dos problemas físicos embutidos da Arena, cada um com solução analítica, resíduos PINN e dados supervisionados; a classe base ArenaProblem levanta NotImplementedError em pinn_residuals e supervised_data.

pinneapple_arena/problems.py · 1.206 linhas · testes: 15 · palavras-chave no código (heurística; o texto acima diz o status real): levanta NotImplementedError em algum caminho

nome	tipo	assinatura	o que faz
ArenaProblem	classe	(name, description, domain, input_dim, output_dim, physics_preset)	Classe base de problema: analytical, pinn_residuals, supervised_data (abstratos, NotImplementedError), compiled_losses e to_problem_spec via preset de pinneapple_physics.
KovasznayNS	classe	()	Escoamento de Kovasznay (Navier-Stokes 2D estacionário) com solução analítica, campos u, v, p.
Stokes2D	classe	()	Escoamento de Stokes 2D (Poiseuille) com solução analítica u = 1-y², v = 0, p = -2x.
Heat1D	classe	()	Equação do calor 1D com solução analítica exponencial-senoidal.
Heat2D	classe	()	Equação do calor 2D com solução analítica.
ConvectionDiffusion1D	classe	()	Convecção-difusão 1D estacionária com solução analítica de camada-limite.
DarcyFlow2D	classe	()	Darcy 2D (Poisson para a pressão) com solução analítica sin(πx)sin(πy).
Laplace2D	classe	()	Laplace 2D com solução analítica sin(πx)sinh(πy)/sinh(π).

nome	tipo	assinatura	o que faz
`Poisson2D`	classe	`()`	Poisson 2D com solução analítica sin(πx)sin(πy).
`Helmholtz2D`	classe	`()`	Helmholtz 2D com solução fabricada sin(πx)sin(πy).
`Biharmonic2D`	classe	`()`	Equação biharmônica 2D (placa fina) com solução fabricada.
`AdvectionDiffusion1D`	classe	`()`	Advecção-difusão 1D transiente com solução analítica.
`Wave1D`	classe	`()`	Onda 1D com solução analítica sin(πx)cos(πct).
`KleinGordon1D`	classe	`()`	Klein-Gordon 1D linear com solução analítica cos(kx)cos(ωt).
`NLS1D`	classe	`()`	Schrödinger não linear 1D (sóliton brilhante), resolvido nas partes real e imaginária.
`Burgers1D`	classe	`()`	Burgers 1D viscoso com referência exata pela transformação de Cole-Hopf (quadratura de Gauss-Hermite).
`KdV1D`	classe	`()`	Korteweg-de Vries 1D com solução de sóliton.
`AllenCahn`	classe	`()`	Allen-Cahn 1D (campo de fase) com interface em tanh; a classe é definida duas vezes no arquivo e só a segunda definição é registrada.
`FisherKPP1D`	classe	`()`	Fisher-KPP 1D com onda viajante exata (o docstring a chama de Ablowitz-Zeidler).
`FitzHughNagumo1D`	classe	`()`	FitzHugh-Nagumo 1D (meio excitável, campos v e w).
`LinearElasticity2D`	classe	`()`	Elasticidade linear 2D em deformação plana com solução fabricada.
`BlackScholes1D`	classe	`()`	EDP de Black-Scholes 1D para opção de compra europeia, referência pela fórmula BS.
`AllenCahn`	classe	`()`	Allen-Cahn 1D (campo de fase) com interface em tanh; a classe é definida duas vezes no arquivo e só a segunda definição é registrada.
`get_problem`	função	`(name) -> ArenaProblem`	Devolve a instância registrada de um problema pelo nome; `KeyError` com a lista se não existe.
`register_problem`	função	`(problem) -> 'ArenaProblem'`	Registra uma instância própria de `ArenaProblem` para a Arena achá-la pelo nome.
`list_problems`	função	`() -> List[str]`	Nomes dos problemas registrados, em ordem alfabética.
`list_problems_by_domain`	função	`() -> Dict[str, List[str]]`	Problemas registrados agrupados pelo domínio físico.

`pinneapple_arena.regimes`

Arena de regimes: treina cada método de `pinneapple_analysis.surrogate.methods` com mesmos dados, mesmo orçamento, mesma regra de parada e várias sementes, e mede por regime (interpolação, poucos dados, extrapolação, mudança de geometria, frentes íngremes) mais custo e violações físicas; regime impossível é marcado indisponível.

`pinneapple_arena/regimes.py` · 387 linhas · testes: 19

nome	tipo	assinatura	o que faz
`ArenaResult`	classe	`(problem, protocol, regimes, methods, rankings, overall, baselines, audit, created, hardw…)`	Resultado da arena de regimes: problema, protocolo, regimes, métodos, rankings, visão geral, baselines, auditoria, hardware e tempo; exporta dict e Markdown.
`arena_markdown`	função	`(r, title='') -> str`	Relatório comparativo em Markdown a partir de `ArenaResult.to_dict()` ou de um arquivo de resultado gravado.
`run_regime_arena`	função	`(data, *, methods='auto', budget=10.0, seeds=3, regimes='auto', few_data_fraction=0.25, s…)`	Roda a arena de regimes sobre os dados e devolve um `ArenaResult`.

pinneapple_arena.showcase

Demonstração com quatro arquiteturas PINN contrastantes no mesmo problema `burgers_1d`, ranqueadas por acurácia e pela visão física da Arena; usa só peças da própria Arena.

`pinneapple_arena/showcase.py` · 158 linhas · testes: 5

nome	tipo	assinatura	o que faz
`field_fit_slice`	função	`(xy_eval, y_ref, y_pred, grid_n) -> dict[str, Any]`	Referência e previsão na última linha de tempo da grade de avaliação (t = 1 para `burgers_1d`).
`run_benchmark`	função	`(problem_name=PROBLEM_NAME, architectures=None, seed=None, progress_cb=None, n_col=N_COL,…)`	Treina cada arquitetura nos mesmos pontos de colocação, avalia, ranqueia por acurácia e por `physics_aware_rank`, reportando progresso.

pinneapple_arena.trainer

Motor de treino unificado: PINN (resíduo próprio ou perdas compiladas), supervisionado, operador em grade completa, grafo, mais avaliação, UQ e problema inverso via `pinneapple_analysis`.

`pinneapple_arena/trainer.py` · 616 linhas · testes: 25

nome	tipo	assinatura	o que faz
`TrainResult`	classe	`(name, model, train_losses, train_time, metrics, physics_residual, uq_result, inverse_res…)`	Resultado de treino de um modelo: modelo, perdas, tempo, métricas, resíduo físico, resultados de UQ e inverso, extras.
`train_pinn`	função	`(model, cfg, pinn_residuals_fn, xy_int, xy_bc, uv_bc, problem_params, device='cpu', log_i…)`	Treina um modelo PINN com resíduo de EDP em pontos internos e perda de contorno.
`train_supervised`	função	`(model, cfg, X_train, Y_train, device='cpu', log_interval=100) -> TrainResult`	Treina um modelo com MSE em pares (entrada, alvo).
`grid_layout`	função	`(x) -> Tuple[np.ndarray, int, int]`	Ordem das linhas de `x` que forma uma grade completa (y-major); `ValueError` se os pontos não formam grade.
`train_grid_operator`	função	`(model, cfg, X_grid, Y_grid, device='cpu', log_interval=100) -> TrainResult`	Treina um modelo tipo FNO sobre campos inteiros: cada passo vê a grade de treino completa e ordenada.
`train_graph`	função	`(model, cfg, node_feats, edge_index, edge_attr, node_targets, device='cpu', log_interval=…)`	Treina um modelo de grafo de forma supervisionada nos nós.
`evaluate_model`	função	`(result, cfg, xy_eval, Y_ref, field_names, device='cpu', node_positions=None, edge_index=…)`	Avalia um modelo treinado nos pontos de avaliação contra a referência e devolve métricas por campo.
`run_uq`	função	`(result, xy_eval, uq_cfg, device='cpu') -> Optional[Any]`	Roda quantificação de incerteza com `pinneapple_analysis.uq_predict`.
`run_inverse`	função	`(result, xy_eval, Y_ref, inv_cfg, device='cpu') -> Optional[Any]`	Roda o problema inverso com `pinneapple_analysis.invert`.
`load_pinneapple_dataset`	função	`(dataset_id, input_fields, output_fields, n_train=1000, n_val=200, split_seed=42)`	Carrega um dataset de `pinneapple_data` e divide em arrays de treino e validação.

pinneapple_arena.visualizer

Figuras em tema escuro dos resultados da Arena, salvas como PNG no diretório de saída. [sem teste]

`pinneapple_arena/visualizer.py` · 452 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`plot_field_comparison`	função	`(eval_results, field_names, xy_eval, grid_n, problem_name, dark_theme=True, dpi=150, save…)`	Painel de campos: referência, cada modelo e o erro.
`plot_loss_curves`	função	`(train_results, dark_theme=True, dpi=150, save_path=None, show=False)`	Curvas de perda de treino de cada modelo.
`plot_metrics_table`	função	`(eval_results, field_names, train_results, dark_theme=True, dpi=150, save_path=None, show…)`	Tabela de métricas por modelo e campo, como figura.
`plot_streamlines`	função	`(eval_results, xy_eval, grid_n, u_field='u', v_field='v', field_names=None, problem_name=…)`	Linhas de corrente do campo de velocidade previsto por cada modelo.

nome	tipo	assinatura	o que faz
plot_uq	função	(uq_result, xy_eval, field_names, grid_n=None, title='UQ', dark_theme=True, dpi=150, save…)	Média ± desvio de um UQResult de pinneapple_analysis.

Exemplos de uso reais do repositório

Arena a partir de configuração programática — examples/arena_pipelines/04_arena_from_yaml.py

```
from pinneapple_arena import Arena, ArenaConfig

BUILTIN_CONFIGS = {
    "kovasznay_ns": {
        "problem": {
            "name": "kovasznay_ns",
            "params": {"re": 40.0},
            "grid_n": 40, "n_col": 2000, "n_bc": 500,
            "n_train_supervised": 200, "n_mesh_nodes": 600,
        },
        "models": [
            {"name": "VanillaPINN", "type": "vanilla_pinn",
             "network": {"hidden": [128, 128, 128, 128], "activation": "tanh"},
             "training": {"epochs": 2000, "lr": 5e-4, "grad_clip": 1.0}},
```

Problema próprio com define_problem — pinneapple_arena/custom_builder.py

```
from pinneapple_arena import define_problem
import numpy as np

prob = define_problem(
    name="reaction_diffusion",
    description="eps·u'' − k·u = f  on [0,1],  u(0)=u(1)=0",
    coords={"x": (0.0, 1.0)},
    fields=["u"],
    pde="eps * u_xx - k * u + (eps * pi**2 + k) * sin(pi * x)",
    bcs=[
        {"type": "dirichlet", "at": "x_min", "field": "u", "value": 0.0},
        {"type": "dirichlet", "at": "x_max", "field": "u", "value": 0.0},
    ],
    params={"eps": 0.1, "k": 1.0},
    analytical=lambda x, eps=0.1, k=1.0: {"u": np.sin(np.pi * x)},
)
prob.solve(models=["VanillaPINN", "SIREN"], epochs=3000)
```

Autoresearch com proponente aleatório — pinneapple_arena/autoresearch.py

```
from pinneapple_arena.autoresearch import AutoResearch, RandomSearchProposer, init_workdir
init_workdir("runs/poisson")  # program.md + train.py (1D Poisson PINN, rel-L2 vs sin(pi x))
AutoResearch("runs/poisson", RandomSearchProposer({"LR": [1e-3, 3e-3, 1e-2], "WIDTH": [16, 32, 64]}),
             budget_s=60).run(max_trials=20)
```

Testes e status de validação

- Módulos com pelo menos um teste que os importa: 15 de 54.
- Funções de teste em arquivos que importam algum módulo do pacote: 99.

Itens do catálogo de métodos (pinneapple_catalog/method_status.json) cujo código está neste pacote:

id	método	status	testes contra referência	outros testes
T15	Adaptive hyperparameter sweep / architecture search (arena NAS)	untested	0	0
T21	Autoresearch loop (fixed-budget trials, keep/revert, random or LLM proposer)	tested	0	2

Limitações conhecidas

- Muitos módulos não têm teste direto: `custom_builder`, `dataset_bench`, `dataset_problems`, `visualizer`, quase todo o subpacote `external` e a maior parte de `pointwise` (só `adapters` e `tables` têm).
- O catálogo `external` depende de bibliotecas opcionais de terceiros: sem elas o resultado é `not_installed`; entradas pesadas só rodam com `include_heavy=True`; muitos runners apenas devolvem `manual_install` (código de artigo só via git, Julia, C++) ou só confirmam o import, sem inferência real.
- Os runners de fine-tune do catálogo externo são demonstrações curtas em CPU sobre dados de brinquedo; no modo "reconfigure" não há gradiente, e para MACE, NequIP, DeePMD-kit, SchNetPack e NeuralGCM só há import real.
- Em `problems.py`, o docstring cita `lid_driven_cavity_ns`, mas esse problema não é registrado; a classe `AllenCahn` aparece duas vezes (a primeira com um alias `AreraProblem`) e só a segunda vale.
- A classe base `ArenaProblem` levanta `NotImplementedError` em `pinn_residuals` e `supervised_data`: problemas próprios precisam implementá-los (ou usar `define_problem`).
- `nas` não é NAS diferenciável: cada tentativa treina um modelo do zero, e sem Optuna cai para busca aleatória.
- Em `pointwise.datasets`, alguns campos de referência (colisão, aerofólio 3D, esteira de cilindro) são ilustrativos ou de ordem reduzida, não soluções das equações completas; `prediction_quality` é heurística, não checagem física.
- No `leaderboard`, as notas de custo, memória e velocidade são relativas às entradas da mesma chamada e dependem do hardware; o L2 médio mistura problemas diferentes se não for filtrado.

6. `pinneapple_worldmodel`

Objetivo

O `pinneapple_worldmodel` tenta montar um "modelo de mundo" físico: uma rede que aprende `f(state_t, descritor) -> state_{t+1}`, onde o estado é um campo espacial (temperatura, velocidade, pressão) e o descritor resume a PDE, os parâmetros e o domínio. O fluxo principal é o `PhysicsAIPipeline`: gera dados de várias fontes (`PhysicsDatasetFactory`, com solvers clássicos, dados de PINN, PDE simbólica e colocação), treina um especialista por cenário (`SpecialistTrainer` -> `ModelZoo`), faz meta-aprendizado (`MetaLearner`, MAML/Reptile), monta um modelo de fundação (`WaMaModel` por média de pesos e `PhysicsFoundationModel`, um FNO com codificador de descritor e adaptadores LoRA) e avalia num `PhysicsBenchmark` com tarefas embutidas.

A base é simples e roda em CPU: cenários embutidos (`BUILTIN_SCENARIOS`: `burgers_1d`, `wave_1d`, `heat_2d`, `advection_2d`, `ns2d_cavity`, `heat_multiscale`), o `PhysicsSimulator` (tenta solvers de `pinneapple_simulation` e cai para diferenças finitas em PyTorch), o `WorldModelDataset` de pares (estado, próximo estado) e o `PhysicsWorldModel` (FNO residual condicionado a contexto). Há ainda um orquestrador por ferramentas (`PhysicsOrchestrator` + `PhysicsToolRegistry`) que escolhe, por regras fixas para cada tipo de problema (forward, inverse, design, forecast, uncertainty, discovery, digital_twin, world_model), uma cadeia de chamadas a outros pacotes.

Um segundo eixo é a fábrica de dados sintéticos para IA física (`SyntheticDataFactory`, "9 estágios"): cenário a partir de JSON ou de uma frase (`ScenarioGenerator`, extração por palavras-chave), solver (`PhysicsSolverBridge`), renderização em RGB/térmico/profundidade/IR (`PhysicsRenderer`), câmeras virtuais (`CameraSystem`), aleatorização para reduzir a diferença sim-real (`RealityRandomizer`), realce fotorrealista que nunca toca os campos físicos (`PhotorealisticEnhancer`), empacotamento em Zarr/vídeo/JSON (`DatasetPackager`) e datasets prontos para treino (`TrainingHooks`).

Os módulos mais novos e mais testados são `physical_world` (vídeo sintético de uma bola quicando com rótulos exatos, um world model pequeno treinável em CPU e checagens físicas que julgam qualquer vídeo, inclusive de um modelo de fundação como NVIDIA Cosmos via `CosmosAdapter`), `inspection_data` (inspeção térmica sintética com rótulo físico e cenas OpenUSD) e `robot_perception` (movimentos de braço simulados no Robot Sim/Newton, vistos por câmera aleatorizada). Eles são usados pelos exemplos em `examples/nvidia/`. O pacote se apoia em `pinneapple_simulation`, `pinneapple_physics`, `pinneapple_analysis`, `pinneapple_neural`, `pinneapple_adaptation`, `pinneapple_data` e `pinneapple_tools`, quase sempre com import opcional e fallback.

Números do inventário

módulos	linhas	classes públicas	funções públicas	testes em arquivos que importam o pacote	módulos sem teste direto
31	15.016	90	49	94 (6 arquivos)	23

Pasta de testes dedicada: `tests/pinneapple_worldmodel` (8 funções de teste).

Submódulos

submódulo	o que faz	módulos	classes + funções	testes*
`benchmark`	`PhysicsBenchmark`: avalia um modelo em tarefas embutidas com trajetórias de referência, RMSE, L2 relativo e checagens de conservação.	1	3	0
`camera_system`	Câmeras virtuais (RGB, térmica, profundidade, IR) que projetam campos 2D em observações de sensor.	1	3	0
`curriculum`	Treino em estágios de complexidade crescente, avançando quando a perda de validação atinge a meta.	1	3	0
`dataset`	`WorldModelDataset` de pares (estado, contexto) -> próximo estado e o `DatasetBuilder` simulador -> dataset.	1	3	20
`dataset_factory`	`PhysicsDatasetFactory`: gera dados de várias fontes (solver, PINN, simbólico, colocação) num `DatasetCatalog`.	1	4	0
`dataset_packager`	Grava cada amostra na estrutura canônica de dataset de IA física (Zarr, vídeos, VTK, JSONs) com manifesto.	1	4	0
`generate_datasets`	Funções e CLI para gerar e carregar catálogos de datasets do world model.	1	4	0
`geometry`	Geometrias 2D/3D com CSG, regiões de contorno nomeadas e domínios prontos (cavidade, canal, cilindro, L, anel).	1	17	0
`inspection_data`	Dados sintéticos de inspeção térmica com rótulo físico, variações de cena em OpenUSD e checagem do rótulo.	1	11	39
`mega_model`	`PhysicsFoundationModel`: FNO com codificador de descritor por transformer e adaptadores LoRA.	1	5	0
`meta_learning`	Meta-aprendizado MAML/Reptile sobre tarefas amostradas de um catálogo.	1	3	0
`model`	`PhysicsWorldModel`: FNO residual condicionado a contexto, `(state_t, context) -> state_{t+1}`.	1	2	20
`model_zoo`	Registro de especialistas treinados com consulta, checkpoints e ensembles (média de saídas ou de pesos).	1	4	0
`orchestrator`	`PhysicsOrchestrator`: resolve um `ProblemStatement` encadeando ferramentas do registro por planos fixos por tipo de problema.	1	3	20
`photorealistic_enhancer`	Realce visual de quadros renderizados sem tocar dados físicos; backend padrão é identidade.	1	3	0
`physical_world`	Vídeo sintético com rótulos exatos, world model pequeno em CPU e checagens físicas de vídeo a partir dos pixels.	1	12	39
`physics_renderer`	Renderiza trajetórias de campos em quadros RGB, térmicos, de profundidade e IR, e em vídeo ou PNG.	1	3	0
`physics_solver_bridge`	Despacha um cenário para um backend de solver e empacota a saída como verdade de referência.	1	4	0
`physics_tools`	Inventário das capacidades dos pacotes como ferramentas chamáveis (`PhysicsToolRegistry`).	1	2	35
`pipeline`	`PhysicsAIPipeline`: ponto de entrada do fluxo dados -> especialistas -> meta -> fundação -> benchmark.	1	3	0
`reality_randomizer`	Aumentos de realismo (ruído, JPEG, iluminação, sombras, oclusões, blur, distorção, vinheta) só nas imagens.	1	2	0
`robot_perception`	Dados de percepção de robô: movimentos de braço simulados (Newton), linhas do tempo OpenUSD, câmera aleatorizada e checagens.	1	13	39
`scenario`	`PhysicsScenario` e o registro `BUILTIN_SCENARIOS` de problemas para treinar o world model.	1	1	0
`scenario_generator`	Converte JSON ou uma frase (por palavras-chave) num `ScenarioSpec` com geometria, contorno e sensores.	1	2	0

submódulo	o que faz	mó-dulos	classes + funções	tes-tes*
`simulator`	`PhysicsSimulator`: gera trajetórias de um cenário com solvers do projeto ou diferenças finitas embutidas.	1	2	0
`specialist_trainer`	Treina um `PhysicsWorldModel` por cenário e preenche o `ModelZoo`.	1	2	0
`synthetic_factory`	`SyntheticDataFactory`: encadeia os estágios de cenário a dataset multimodal, com atalhos por caso.	1	7	0
`train`	Entrada de treino com modos specialist, meta, foundation e pipeline (funções e CLI).	1	5	0
`trainer`	Laço de treino do world model com perda de rollout multi-passo e perda física opcional.	1	3	0
`training_hooks`	Datasets PyTorch sobre amostras empacotadas para codificador de vídeo, decodificador físico, PINN inversa e operador neural.	1	6	0

* maior número de testes que importam um módulo do submódulo.

Módulos e API pública

`pinneapple_worldmodel`

Ponto de entrada do pacote: reexporta o pipeline, o orquestrador, cenários, modelos e fábricas; o docstring descreve o pipeline de cinco estágios do modelo de fundação física.

`pinneapple_worldmodel/__init__.py` · 409 linhas · testes: 11

`pinneapple_worldmodel.benchmark`

Suíte de benchmark com tarefas embutidas (`BUILTIN_TASKS`, como `heat_2d_smooth`, `burgers_1d_shock`, `ns2d_lid_driven`) avaliadas por RMSE, L2 relativo, passo de divergência no rollout e checagens físicas. [sem teste direto]

`pinneapple_worldmodel/benchmark.py` · 564 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`BenchmarkTask`	classe	`(name, scenario_name, n_trajectories, rollout_steps, seed, physics_checks, metrics, descr…)`	Uma tarefa de benchmark: cenário, número de trajetórias, passos de rollout, semente, checagens físicas e métricas.
`BenchmarkResult`	classe	`(task_name, next_step_rmse, rollout_rmse, rel_l2, div_step, physics_scores, n_trajectorie…)`	Resultado de uma tarefa: RMSE de um passo e de rollout, L2 relativo, passo de divergência e escores físicos.
`PhysicsBenchmark`	classe	`(device='cpu', verbose=True) -> None`	Avalia modelos com `forward(state_t, descriptor)` nas tarefas do benchmark.

`pinneapple_worldmodel.camera_system`

Estágio 5 da fábrica sintética: projeta campos 2D em observações de câmeras virtuais com pose, campo de visão, ruído e profundidade de bits. [sem teste direto]

`pinneapple_worldmodel/camera_system.py` · 392 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`CameraConfig`	classe	`(name, sensor_type, position, look_at, fov_deg, resolution, fps, near_clip, far_clip, noi…)`	Configuração de uma câmera virtual: tipo de sensor, pose, campo de visão, resolução, fps, ruído e bits.
`MultiCameraArray`	classe	`(cameras, sync_fps)`	Conjunto de câmeras que observam o mesmo campo simulado, sincronizadas.
`CameraSystem`	classe	`(array=None, domain_bounds=None) -> None`	Projeta campos físicos em observações no plano de cada câmera.

pinneapple_worldmodel.curriculum

Currículo de treino por estágios de cenários (de calor 2D a Navier-Stokes) que avança ao atingir a perda-alvo de validação. [sem teste direto]

`pinneapple_worldmodel/curriculum.py` · 283 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`CurriculumStage`	classe	`(scenarios, n_samples, epochs, target_val_loss, lr, rollout_steps, description)`	Um estágio do currículo: cenários, amostras, épocas, perda-alvo, taxa de aprendizado e passos de rollout.
`CurriculumConfig`	classe	`(stages, model_config, device, batch_size, patience, save_dir, verbose)`	Configuração completa do currículo: estágios, modelo, dispositivo, lote, paciência e pasta de saída.
`PhysicsCurriculum`	classe	`(config) -> None`	Treina um world model passando pelos estágios do currículo.

pinneapple_worldmodel.dataset

Transforma trajetórias simuladas em amostras `state_t`, `state_tp1`, `params`, `context` normalizadas; `DatasetBuilder` orquestra simulação e montagem.

`pinneapple_worldmodel/dataset.py` · 382 linhas · testes: 20

nome	tipo	assinatura	o que faz
`WorldModelDataset`	classe	`(trajectories, *, horizon=1, normalize=True, transform=None) -> None`	Dataset de previsão de um ou vários passos construído de uma lista de trajetórias.
`DatasetConfig`	classe	`(scenarios, n_samples_per_scenario, horizon, normalize, validate_physics, save_dir, devic…)`	Configuração do `DatasetBuilder`: cenários, amostras por cenário, horizonte, normalização e validação física.
`DatasetBuilder`	classe	`(config) -> None`	Orquestra simulação e montagem de um `WorldModelDataset`.

pinneapple_worldmodel.dataset_factory

Fábrica multi-fonte de dados (solver, PINN via `pinneapple_physics`, PDE simbólica, colocação de `pinneapple_data`) com catálogo por cenário e fonte; fontes indisponíveis são puladas. [sem teste direto]

`pinneapple_worldmodel/dataset_factory.py` · 725 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`FactoryConfig`	classe	`(sources, scenarios, n_samples_per_scenario, horizon, validate_physics, device, save_dir,…)`	Configuração da `PhysicsDatasetFactory`: fontes, cenários, amostras, horizonte, validação e pasta.
`DatasetEntry`	classe	`(name, dataset, source, scenario, physics_tags, n_samples, generation_time_s, metadata)`	Uma fatia do catálogo: dataset, fonte, cenário, tags físicas, número de amostras e tempo de geração.
`DatasetCatalog`	classe	`() -> None`	Registro de todos os datasets gerados, consultável por cenário e fonte.
`PhysicsDatasetFactory`	classe	`(config) -> None`	Gera datasets de treino combinando as fontes de dados disponíveis.

pinneapple_worldmodel.dataset_packager

Estágio 8: grava cada amostra em `sample_XXXXXX/` com campos em Zarr, vídeos, `mesh.vtk` e JSONs de metadados, PDE, contorno e câmera, mais um manifesto. [sem teste direto]

`pinneapple_worldmodel/dataset_packager.py` · 439 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`PackagerConfig`	classe	`(output_dir, format, zarr_chunks, zarr_compressor, save_vtk, save_videos, sample_id_digit…)`	Como o empacotador escreve o dataset: formato, chunks e compressor Zarr, VTK, vídeos e sobrescrita.

nome	tipo	assinatura	o que faz
PackagedSample	classe	(sample_id, sample_dir, field_paths, video_paths, metadata_path)	Metadados de uma amostra empacotada: id, pasta e caminhos de campos, vídeos e metadados.
DatasetManifest	classe	(output_dir) -> None	Índice de todas as amostras de um dataset de IA física.
DatasetPackager	classe	(config=None) -> None	Escreve uma amostra física na estrutura canônica do dataset.

pinneapple_worldmodel.generate_datasets

Gera catálogos de trajetórias multi-fonte para treinar o world model, via funções ou `python -m pinneapple_worldmodel.generate_datasets`. [sem teste direto]

`pinneapple_worldmodel/generate_datasets.py` · 441 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
generate	função	(scenarios=None, sources=None, n_samples=100, output_dir='./data/worldmodel', device='cpu…)	Gera datasets multi-fonte e devolve um `DatasetCatalog`.
generate_via_factory	função	(scenarios=None, sources=None, n_samples=100, output_dir='./data/worldmodel', device='cpu…)	Gera datasets pela `PhysicsDatasetFactory` de alto nível.
load_catalog	função	(path) -> DatasetCatalog	Carrega do disco um `DatasetCatalog` salvo.
main	função	() -> None	Ponto de entrada da linha de comando de geração de datasets.

pinneapple_worldmodel.geometry

Sistema de geometria componível (primitivas 2D/3D, CSG, regiões de contorno, domínios prontos) usado pela fábrica de dados e pelo treinador; o docstring cita `make_pipe_bend`, que não existe no módulo. [sem teste direto]

`pinneapple_worldmodel/geometry.py` · 569 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
GeometryBase	classe	()	Base abstrata das geometrias (pertinência e amostragem de interior e fronteira).
Rectangle	classe	(lo, hi) -> None	Retângulo alinhado aos eixos.
Circle	classe	(center, radius) -> None	Círculo 2D preenchido.
Polygon	classe	(vertices) -> None	Polígono 2D preenchido definido por vértices ordenados.
Box3D	classe	(lo, hi) -> None	Caixa 3D alinhada aos eixos.
Sphere	classe	(center, radius) -> None	Esfera 3D preenchida.
Union	classe	(a, b) -> None	União CSG: pontos dentro de *a* ou de *b*.
Intersection	classe	(a, b) -> None	Interseção CSG: pontos dentro de *a* e de *b*.
Difference	classe	(a, b) -> None	Diferença CSG: pontos dentro de *a* e fora de *b*.
BoundaryRegion	classe	(name, bc_type, value, sampler)	Parte nomeada da fronteira com tipo e valor de condição de contorno e um amostrador.
PhysicsDomain	classe	(geometry, boundaries, name, tags)	Domínio completo: geometria, regiões de contorno, nome e tags.
make_unit_square	função	(*, with_boundary_labels=True) -> PhysicsDomain	Quadrado unitário com rótulos de contorno, domínio padrão de benchmark.
make_cavity	função	(L=1.0) -> PhysicsDomain	Cavidade com tampa móvel (u=1 em y=L).

nome	tipo	assinatura	o que faz
`make_channel`	função	`(length=4.0, height=1.0, *, inlet_velocity=1.0) -> PhysicsDomain`	Domínio de canal de Poiseuille com velocidade de entrada.
`make_channel_with_cylinder`	função	`(length=4.0, height=1.0, cyl_center=(1.0, 0.5), cyl_radius=0.1) -> PhysicsDomain`	Canal 2D com obstáculo circular.
`make_l_shaped`	função	`() -> PhysicsDomain`	Domínio em L (benchmark de concentração de tensão).
`make_annulus`	função	`(inner_r=0.3, outer_r=1.0, center=(0.0, 0.0)) -> PhysicsDomain`	Domínio anular para problemas de calor ou eletromagnetismo em geometria cilíndrica.

`pinneapple_worldmodel.inspection_data`

Inspeção térmica sintética: placa aquecida com vazio subsuperficial resolvida por volumes finitos, variações de câmera gravadas como estágios OpenUSD e verificação de que o aumento manteve o rótulo físico; precisa de `pxr`.

`pinneapple_worldmodel/inspection_data.py` · 224 linhas · testes: 39

nome	tipo	assinatura	o que faz
`Variation`	classe	`(gain, offset, noise, blur, zoom, angle_deg, shift, glint)`	Parâmetros de uma variação da câmera sintética: ganho, offset, ruído, blur, zoom, ângulo, deslocamento e brilho especular.
`thermal_plate`	função	`(defect, n=48, *, k=1.0, k_void=0.15, q=1.0) -> dict`	Condução estacionária no quadrado unitário com disco de baixa condutividade; devolve campo, campo sem defeito e o rótulo.
`sample_variation`	função	`(rng, *, glint_p=0.2) -> Variation`	Sorteia uma `Variation`, com brilho especular com probabilidade `glint_p`.
`camera_transform`	função	`(var, H, W)`	Transformação placa -> pixel e sua inversa para a câmera da variação.
`write_variation_stage`	função	`(path, plate, var, *, cmap='gray') -> str`	Grava a placa e a variação como um estágio OpenUSD e devolve o caminho.
`thermal_image`	função	`(stage_path, var, H=64, W=64, seed=0) -> np.ndarray`	Imagem `(H, W)` da câmera de um estágio de variação.
`nominal_image`	função	`(stage_path, plate, var, H=64, W=64) -> np.ndarray`	Imagem que a mesma câmera veria na placa sem defeito, a referência do inspetor.
`label_check`	função	`(image, nominal, var, label_xy, *, tol=0.08) -> dict`	Verifica se a imagem ainda mostra a anomalia física onde o rótulo indica, pelo resíduo contra a imagem nominal.
`inspection_dataset`	função	`(n, seed, workdir, *, glint_p=0.25, H=64, W=64, keep_stages=0) -> dict`	Gera `n` amostras com vazio aleatório, campo térmico, variação em USD, rótulo em pixels e checagem do rótulo.
`inspection_sample`	função	`(x=0.4, y=0.6, r=0.08, seed=0, glint=False, usd_path='inspection_sample.usda') -> dict`	Uma amostra de inspeção térmica com vazio em posição dada, variação de câmera e checagem do rótulo.
`cleanup`	função	`(paths) -> None`	Remove os arquivos indicados que existirem.

`pinneapple_worldmodel.mega_model`

Modelo de fundação generalista: FNO com condicionamento FiLM por um codificador transformer de descritor físico e adaptadores LoRA; suporta grades 1D e 2D, 3D ainda não. [sem teste direto]

`pinneapple_worldmodel/mega_model.py` · 668 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`FoundationConfig`	classe	`(n_modes, width, depth, context_dim, descriptor_vocab, n_heads, n_context_layers, lora_ra…)`	Hiperparâmetros do `PhysicsFoundationModel` (modos, largura, profundidade, contexto, cabeças, posto LoRA, incerteza aleatória).
`LoRALinear`	classe	`(in_features, out_features, rank=8) -> None`	Camada linear com adaptação de baixo posto para adaptação rápida de domínio.
`PhysicsDescriptorEncoder`	classe	`(context_dim=64, max_scalar_features=32, n_heads=4, n_layers=2, dropout=0.1) -> None`	Codifica um dicionário descritor da física num vetor de contexto de tamanho fixo (transformer).
`FoundationFNOBlock`	classe	`(width, n_modes, context_dim, lora_rank=8, dropout=0.1, spatial_dim=2) -> None`	Bloco FNO com injeção de contexto e LoRA opcional.

nome	tipo	assinatura	o que faz
`PhysicsFoundationModel`	classe	`(config, *, n_fields=1, grid_shape=(64, 64)) -> None`	Modelo de fundação generalista para previsão de campos condicionada a descritor.

`pinneapple_worldmodel.meta_learning`

Meta-aprendizado MAML/Reptile para adaptação rápida a novo domínio, com caminho opcional por `pinneapple_adaptation.meta_learning`; o docstring cita classes `MAMLWorldModel`/`ReptileWorldModel` que não existem no arquivo. [sem teste direto]

`pinneapple_worldmodel/meta_learning.py` · 512 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`MetaConfig`	classe	`(algorithm, n_meta_epochs, n_inner_steps, inner_lr, outer_lr, n_tasks_per_batch, n_suppor…)`	Configuração do `MetaLearner`: algoritmo, épocas, passos internos, taxas e tamanhos de suporte e consulta.
`TaskDistribution`	classe	`(catalog, n_support=32, n_query=32, device='cpu') -> None`	Sorteia tarefas (suporte, consulta) de um `DatasetCatalog`.
`MetaLearner`	classe	`(config, model_config=None) -> None`	Meta-aprendizado MAML/Reptile de um world model.

`pinneapple_worldmodel.model`

Arquitetura `PhysicsWorldModel`: codifica o contexto, concatena ao estado, aplica camadas de Fourier e prevê o incremento sobre o estado atual (1D com `rfft`, 2D com `rfft2`).

`pinneapple_worldmodel/model.py` · 320 linhas · testes: 20

nome	tipo	assinatura	o que faz
`WorldModelConfig`	classe	`(n_modes, width, depth, context_dim, embed_dim, activation, dropout, rollout_steps)`	Hiperparâmetros do `PhysicsWorldModel`.
`PhysicsWorldModel`	classe	`(config, *, n_fields, grid_shape) -> None`	World model FNO: `(state_t, context) -> state_{t+1}` com formulação residual.

`pinneapple_worldmodel.model_zoo`

Registro de especialistas por cenário com consulta por nome, tag ou métrica, salvar/carregar e ensembles por média de saídas ou de pesos. [sem teste direto]

`pinneapple_worldmodel/model_zoo.py` · 494 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ZooEntry`	classe	`(name, model, scenario, physics_tags, metrics, n_params, trained_at, checkpoint_path, met…)`	Entrada do zoo: modelo, cenário, tags, métricas, parâmetros, data de treino e checkpoint.
`EnsembleModel`	classe	`(models, weights=None, aggregation='mean') -> None`	Ensemble que combina as saídas de vários especialistas `PhysicsWorldModel`.
`WaMaModel`	classe	`()`	Modelo obtido pela média dos pesos de vários especialistas.
`ModelZoo`	classe	`(root_dir=None) -> None`	Registro de especialistas treinados com consulta, persistência e ensembles.

`pinneapple_worldmodel.orchestrator`

Orquestrador orientado a objetivo: escolhe um plano fixo por `kind` do `ProblemStatement` e executa as ferramentas do `PhysicsToolRegistry`, com um `tool_gate` opcional que só permite ou bloqueia chamadas.

`pinneapple_worldmodel/orchestrator.py` · 766 linhas · testes: 20

nome	tipo	assinatura	o que faz
`ProblemStatement`	classe	`(kind, description, pde_hint, domain_hint, scenarios, params, observations, model, n_samp…)`	Descrição de alto nível de um problema físico: tipo, descrição, dicas de PDE e domínio, cenários, parâmetros e saídas.
`OrchestratorResult`	classe	`(kind, artifacts, plan, elapsed_s, logs)`	Saída de `PhysicsOrchestrator.solve`: artefatos, plano, tempo e logs.
`PhysicsOrchestrator`	classe	`(registry=None, verbose=True, tool_gate=None) -> None`	Resolve problemas físicos encadeando ferramentas do registro segundo o tipo do problema.

`pinneapple_worldmodel.photorealistic_enhancer`

Estágio 7: realce de imagens que nunca lê campos físicos; o backend padrão `stub` devolve a entrada, `local_diffusion` precisa de `diffusers` e `cosmos` é [placeholder] ainda indisponível (cai para `stub`). [sem teste direto]

`pinneapple_worldmodel/photorealistic_enhancer.py` · 283 linhas · **sem teste direto** · palavras-chave no código (heurística; o texto acima diz o status real): placeholder/stub citado no docstring

nome	tipo	assinatura	o que faz
`EnhancerConfig`	classe	`(backend, strength, style_prompt, negative_prompt, model_id, device, batch_size, apply_to…)`	Configuração do realce: backend, intensidade, prompts, modelo, dispositivo e sensores.
`PhotorealisticEnhancer`	classe	`(config=None) -> None`	Realça quadros renderizados sem tocar dados físicos, com fallback para o backend identidade.
`register_enhancer`	função	`(name, fn) -> None`	Registra uma função de realce personalizada sob um nome.

`pinneapple_worldmodel.physical_world`

Vídeo sintético de bola quicando com física exata e aleatorização de domínio, um world model conv+GRU treinável em CPU, edições não-físicas e checagens físicas a partir dos pixels, além de um adaptador HTTP para NVIDIA Cosmos.

`pinneapple_worldmodel/physical_world.py` · 489 linhas · testes: 39

nome	tipo	assinatura	o que faz
`Video`	classe	`(frames, dt, px_per_m, labels, params)`	Uma cena: quadros `(T, H, W)`, passo de tempo, pixels por metro, rótulos e parâmetros.
`WorldModel`	classe	`(H=64, W=96, latent=32, seed=0)`	Codificador conv, dinâmica latente GRU e decodificador, treinado para prever o próximo quadro; pequeno para CPU.
`StateEncoder`	classe	`(H=64, W=96, seed=0)`	CNN pequena que lê o centro do objeto num quadro, treinada nos rótulos exatos.
`CosmosAdapter`	classe	`(base_url, api_key=None, timeout=600.0)`	Cliente HTTP de um NIM NVIDIA Cosmos: envia prompt (e vídeo opcional) e decodifica o vídeo devolvido em quadros.
`bouncing_ball_scene`	função	`(*, seed=0, T=48, fps=24.0, H=64, W=96, randomize=True, defect=False, e=None) -> Video`	Bola lançada quicando no chão, renderizada com aleatorização de domínio e rótulos exatos.
`dataset`	função	`(n, *, seed=0, **kw) -> list[Video]`	Gera `n` vídeos de `bouncing_ball_scene` com sementes sucessivas.
`track_object`	função	`(frames) -> dict[str, np.ndarray]`	Centro e área do objeto de maior contraste em cada quadro (NaN quando não há).
`physics_checks`	função	`(video, *, dt=None, px_per_m=None, g=G, g_rtol=0.1, area_rtol=0.25, max_speed=10.0) -> di…)`	Checagens físicas de um vídeo de bola a partir dos pixels (gravidade ajustada, área, velocidade), com veredito.
`video_check`	função	`(seed=0, edit='none') -> dict[str, Any]`	Gera um vídeo sintético, opcionalmente editado de forma não física, e o julga com `physics_checks`.
`corrupt`	função	`(video, kind, seed=0) -> Video`	Edições não físicas: `time_warp`, `teleport`, `fade`, `gravity_flip`.
`state_pairs`	função	`(videos) -> tuple[np.ndarray, np.ndarray]`	Pares `(s_t, s_t+1)` com posição e velocidade em metros tirados dos rótulos exatos.
`render_like`	função	`(video, x_m, y_m) -> np.ndarray`	Renderiza uma trajetória com a câmera e a aparência de um vídeo dado.

pinneapple_worldmodel.physics_renderer

Estágio 4: converte campos em quadros RGB, térmicos, de profundidade e IR por colormaps e monta vídeos `.mp4` (PNG se não houver ffmpeg). [sem teste direto]

`pinneapple_worldmodel/physics_renderer.py` · 444 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`RendererConfig`	classe	`(resolution, fps, sensors, colormap_rgb, colormap_thermal, colormap_depth, colormap_ir, d…)`	Configuração de renderização: resolução, fps, sensores, colormaps, dpi e qualidade de vídeo.
`RenderResult`	classe	`(frames, video_paths, png_dirs, field_stats)`	Saída de `PhysicsRenderer.render()`: quadros, caminhos de vídeo e PNG e estatísticas de campo.
`PhysicsRenderer`	classe	`(config=None) -> None`	Renderiza trajetórias de campos em quadros e vídeos por sensor.

pinneapple_worldmodel.physics_solver_bridge

Estágios 2 e 3: despacha o cenário para `builtin` (PyTorch), OpenFOAM, FEniCS, PINN ou FNO, com aviso e retorno ao `builtin` quando o backend falta, e empacota a saída em Zarr (ou NumPy). [sem teste direto]

`pinneapple_worldmodel/physics_solver_bridge.py` · 837 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`SolverOutput`	classe	`(fields, t_coords, x_coords, y_coords, z_coords, params, solver_name, metadata)`	Histórico multivariado de campos (u, v, w, p, T, C) com coordenadas, parâmetros e nome do solver.
`SolverBridgeConfig`	classe	`(solver, generate_temperature, generate_concentration, openfoam_case_dir, fenics_problem,…)`	Configuração do bridge: solver, campos extras, caso OpenFOAM, problema FEniCS, checkpoints PINN/FNO.
`PhysicsSolverBridge`	classe	`(config=None) -> None`	Despacha um cenário para o backend de solver adequado.
`GroundTruthPackager`	classe	`(output_dir, format='zarr', zarr_chunks=None, overwrite=True) -> None`	Organiza `SolverOutput` na estrutura canônica de dataset (Zarr ou NumPy).

pinneapple_worldmodel.physics_tools

Cataloga capacidades dos pacotes como `PhysicsTool` com categoria, esquemas de entrada/saída e função, usadas pelo orquestrador; ferramentas cujo pacote falta ficam indisponíveis.

`pinneapple_worldmodel/physics_tools.py` · 983 linhas · testes: 35

nome	tipo	assinatura	o que faz
`PhysicsTool`	classe	`(name, category, description, input_schema, output_schema, module_path, fn, tags)`	Uma capacidade registrada: nome, categoria, descrição, esquemas, módulo, função e tags.
`PhysicsToolRegistry`	classe	`() -> None`	Catálogo das capacidades como ferramentas chamáveis, com busca por categoria, tag ou texto.

pinneapple_worldmodel.pipeline

`PhysicsAIPipeline`: roda os cinco estágios (dados, especialistas, meta-aprendizado, fundação, benchmark) a partir de um `PhysicsAIConfig`. [sem teste direto]

`pinneapple_worldmodel/pipeline.py` · 498 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`PhysicsAIConfig`	classe	`(scenarios, sources, n_samples, device, save_dir, specialist_epochs, specialist_lr, speci…)`	Configuração completa do `PhysicsAIPipeline`.
`PhysicsAIPipelineResult`	classe	`(catalog, zoo, meta_model, mega_model, benchmark, elapsed_s)`	Saídas do pipeline: catálogo, zoo, meta-modelo, mega-modelo, benchmark e tempo.

nome	tipo	assinatura	o que faz
`PhysicsAIPipeline`	classe	`(config) -> None`	Constrói o world model generalista rodando todos os estágios.

`pinneapple_worldmodel.reality_randomizer`

Estágio 6: aplica aumentos de imagem com probabilidade e intensidade configuráveis, sem tocar os arrays físicos. [sem teste direto]

`pinneapple_worldmodel/reality_randomizer.py` · 328 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`RandomizerConfig`	classe	`(seed, p_sensor_noise, gaussian_std, poisson_scale, p_jpeg, jpeg_quality_lo, jpeg_quality…)`	Quais aumentos aplicar, com que probabilidade e intensidade.
`RealityRandomizer`	classe	`(config=None) -> None`	Aplica aumentos de diferença sim-real a sequências de imagens.

`pinneapple_worldmodel.robot_perception`

Movimentos de braço de dois elos simulados no Robot Sim (NVIDIA Newton), gravados como linha do tempo OpenUSD, renderizados por câmera aleatorizada e rotulados com os ângulos exatos, com checagens de percepção.

`pinneapple_worldmodel/robot_perception.py` · 283 linhas · testes: 39

nome	tipo	assinatura	o que faz
`CameraVariation`	classe	`(zoom, offset, gain, noise, blur, albedo, background, occluder)`	Variação da câmera sintética: zoom, offset, ganho, ruído, blur, albedo, fundo e oclusor.
`ImageRegressor`	classe	`(H, W, n_out, seed=0)`	CNN pequena de uma imagem cinza para `n_out` números.
`forward_points`	função	`(arm, q) -> dict[str, np.ndarray]`	Cotovelo e ponto da ferramenta no plano x-z para ângulos de junta dados.
`arm_motions`	função	`(arm, n, seed, *, duration=0.8, dt=0.001, fps=30.0, kp=(350.0, 110.0), kd=(5.0, 2.5), q_b…)`	Gera `n` movimentos ponto a ponto aleatórios simulados no Newton, amostrados em `fps`.
`sample_camera`	função	`(rng, *, occluder_p=0.3) -> CameraVariation`	Sorteia uma `CameraVariation`, com oclusor com probabilidade `occluder_p`.
`write_motion_stage`	função	`(path, arm, motion, var, *, payload=None) -> str`	Grava o movimento como linha do tempo OpenUSD, com carga opcional presa à ferramenta.
`camera_frames`	função	`(path, arm, motion, var, H=72, W=72, seed=0) -> np.ndarray`	Quadros cinza `(T, H, W)` do estágio em cada instante registrado.
`visibility`	função	`(arm, motion, var, *, margin=0.03) -> np.ndarray`	Por quadro, se cotovelo e ferramenta estão na imagem e não atrás do oclusor.
`payload_track_check`	função	`(frames, arm, motion, var, *, tol_px=2.0, offset=(0.0, 0.0)) -> dict`	Verifica se cada quadro mostra a peça carregada onde a cinemática a coloca.
`arm_camera_sample`	função	`(payload=5.0, seed=0, occluder=False, usd_path='arm_camera_sample.usda') -> dict`	Uma amostra rotulada de percepção: movimento, estágio USD com câmera, quadros e checagens.
`angle_features`	função	`(q) -> np.ndarray`	Converte ângulos de junta em features `sin`/`cos`.
`feature_angles`	função	`(f) -> np.ndarray`	Inverso de `angle_features`: recupera os ângulos por `arctan2`.
`perception_checks`	função	`(q_pred, motion, arm, *, qd_max=12.0) -> dict`	Erros de uma sequência de ângulos prevista para um movimento, com limite de velocidade de junta.

`pinneapple_worldmodel.scenario`

Define `PhysicsScenario` (PDE, grade, tempo, faixas de parâmetros, CI/CC, solver) e o registro de cenários embutidos. [sem teste direto]

`pinneapple_worldmodel/scenario.py` · 253 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`PhysicsScenario`	classe	`(name, pde_kind, grid_shape, t_span, n_steps, domain_bounds, param_ranges, ic_type, bc_ty…)`	Especificação completa de um problema físico para o world model.

`pinneapple_worldmodel.scenario_generator`

Estágio 1: transforma JSON, nome ou frase (extração heurística por palavras-chave, por exemplo número de Reynolds) num `ScenarioSpec` completo. [sem teste direto]

`pinneapple_worldmodel/scenario_generator.py` · 360 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ScenarioSpec`	classe	`(scenario, domain, geometry, fluid, boundary_conditions, initial_conditions, sensor_confi…)`	Especificação estendida: cenário, domínio, geometria, fluido, contorno, condições iniciais e sensores.
`ScenarioGenerator`	classe	`(default_grid_2d=(64, 64), default_n_steps=32, device='cpu') -> None`	Converte prompt, JSON, dicionário ou nome num `ScenarioSpec`.

`pinneapple_worldmodel.simulator`

Gera trajetórias (`T+1, C, *grid`) tentando solvers de `pinneapple_simulation` (FDM, LBM, FEM) e caindo para solvers PyTorch embutidos de calor, Burgers, onda, advecção e NS2D. [sem teste direto]

`pinneapple_worldmodel/simulator.py` · 630 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`TrajectoryData`	classe	`(states, params, scenario_name, metadata)`	Uma trajetória: sequência de campos, parâmetros, cenário e metadados.
`PhysicsSimulator`	classe	`(scenario, *, device='cpu', verbose=False) -> None`	Gera trajetórias de um `PhysicsScenario` com o backend disponível.

`pinneapple_worldmodel.specialist_trainer`

Treina um especialista por cenário com validação física opcional (`pinneapple_analysis.validation`) e transferência (`pinneapple_adaptation`); o docstring ainda cita pacotes com nomes antigos. [sem teste direto]

`pinneapple_worldmodel/specialist_trainer.py` · 423 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`SpecialistConfig`	classe	`(model, epochs, lr, batch_size, device, val_fraction, patience, grad_clip, rollout_steps,…)`	Configuração do treino de especialistas (épocas, taxa, lote, paciência, rollout, transferência, UQ).
`SpecialistTrainer`	classe	`(config, *, physics_loss_fn=None, pretrained_zoo=None) -> None`	Treina especialistas e preenche um `ModelZoo`.

`pinneapple_worldmodel.synthetic_factory`

Fábrica de dados sintéticos de ponta a ponta (cenário, simulação, renderização, câmeras, aleatorização, realce, empacotamento, hooks de treino) com atalhos para tubo, cilindro, calor e mistura. [sem teste direto]

`pinneapple_worldmodel/synthetic_factory.py` · 525 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`SyntheticFactoryConfig`	classe	`(scenario_input, n_samples, output_dir, device, render_fps, sensors, render_resolution, a…)`	Configuração única de todos os estágios da fábrica sintética.
`FactoryResult`	classe	`(config, spec, samples, output_dir, elapsed_s)`	Saída de `SyntheticDataFactory.generate_dataset()`: configuração, spec, amostras, pasta e tempo.
`SyntheticDataFactory`	classe	`(config=None) -> None`	Fábrica de dados sintéticos de ponta a ponta para IA física.

nome	tipo	assinatura	o que faz
generate_pipe_flow	função	(n_samples=20, output_dir='./pipe_dataset', **kw) -> FactoryResult	Gera um dataset de escoamento em tubo (NS2D, Re de 100 a 5000).
generate_cylinder_flow	função	(n_samples=20, output_dir='./cylinder_dataset', **kw) -> FactoryResult	Gera um dataset de escoamento ao redor de cilindro (esteira de von Kármán).
generate_heat_transfer	função	(n_samples=20, output_dir='./heat_dataset', **kw) -> FactoryResult	Gera um dataset de condução de calor 2D.
generate_chemical_mixing	função	(n_samples=20, output_dir='./mixing_dataset', **kw) -> FactoryResult	Gera um dataset de mistura química (advecção-difusão).

pinneapple_worldmodel.train

Entrada de treino com quatro modos (specialist, meta, foundation, pipeline), como funções e CLI. [sem teste direto]

pinneapple_worldmodel/train.py · 593 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
train_specialist	função	(catalog, output_dir, *, epochs=50, batch_size=32, lr=0.001, device='cpu', patience=10, n…)	Treina um especialista por cenário do catálogo.
train_meta	função	(catalog, output_dir, *, algorithm='reptile', n_meta_epochs=100, n_inner_steps=5, inner_l…)	Meta-treina uma inicialização compartilhada sobre o catálogo.
train_foundation	função	(catalog, output_dir, *, base_ckpt=None, epochs=30, batch_size=16, lr=0.0005, device='cpu…)	Monta e ajusta o PhysicsFoundationModel com LoRA.
run_pipeline	função	(scenarios=None, sources=None, n_samples=100, output_dir='./checkpoints/worldmodel', devi…)	Roda o pipeline completo de cinco estágios.
main	função	() -> None	Ponto de entrada da linha de comando de treino.

pinneapple_worldmodel.trainer

Laço de treino do PhysicsWorldModel com perda de rollout autoregressivo, perda física opcional e validação por RMSE e L2 relativo. [sem teste direto]

pinneapple_worldmodel/trainer.py · 421 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
WorldModelLoss	classe	(rollout_steps=1, rollout_weight=0.5, physics_loss_fn=None, physics_weight=0.01) -> None	Perda de rollout multi-passo com termo físico opcional.
WorldModelTrainConfig	classe	(epochs, lr, weight_decay, batch_size, val_fraction, device, amp, grad_clip, rollout_step…)	Hiperparâmetros do WorldModelTrainer.
WorldModelTrainer	classe	(model, config, *, physics_loss_fn=None) -> None	Treina um PhysicsWorldModel num WorldModelDataset.

pinneapple_worldmodel.training_hooks

Estágio 9: datasets PyTorch sobre o dataset empacotado para quatro objetivos (codificador de vídeo, decodificador de campo, PINN inversa, operador neural); sem arquivo de embedding, o decodificador recebe embedding zero. [sem teste direto]

pinneapple_worldmodel/training_hooks.py · 478 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
PhysicsAIDataset	classe	(dataset_dir, fields=('u', 'v', 'p', 'T'), sensors=(), max_samples=None, device='cpu') ->…)	Dataset base sobre as amostras empacotadas (campos e sensores escolhidos).
CosmosEncoderDataset	classe	(dataset_dir, video_sensors=('rgb', 'thermal'), field_names=('u', 'v', 'p', 'T'), **kwarg…)	Dataset para treinar um codificador vídeo -> embedding latente.
PhysicsDecoderDataset	classe	(dataset_dir, embedding_dim=512, field_names=('u', 'v', 'p', 'T'), device='cpu', max_samp…)	Dataset para treinar um decodificador embedding -> campos físicos (embedding zero se não houver arquivo).

nome	tipo	assinatura	o que faz
`InversePINNDataset`	classe	`(dataset_dir, param_keys=None, video_sensor='rgb', **kwargs) -> None`	Dataset para PINN inversa: quadros de vídeo -> parâmetros físicos.
`NeuralOperatorDataset`	classe	`(dataset_dir, field_names=('u', 'v', 'p'), horizon=1, subsample_t=1, device='cpu', max_sa…)`	Dataset de pares (estado, próximo estado) para operadores neurais.
`TrainingHooks`	classe	`(dataset_dir, device='cpu') -> None`	Fábrica de conveniência para todos os tipos de dataset de treino.

Exemplos de uso reais do repositório

Gerar dados e treinar especialistas — `examples/worldmodel/01_generate_train_evaluate.py`

```
from pinneapple_worldmodel import train_specialist

t0 = time.perf_counter()
zoo = train_specialist(
    catalog      = catalog,
    output_dir   = Path(cfg["output_dir"]) / "checkpoints",
    epochs       = cfg["epochs"],
    batch_size   = cfg["batch_size"],
    lr           = cfg["lr"],
    device       = cfg["device"],
    patience     = max(cfg["epochs"] // 3, 5),
    n_modes      = cfg["n_modes"],
    width        = cfg["width"],
```

Vídeo sintético julgado por checagens físicas — `examples/nvidia/04_worldmodel_physics_video/run.py`

```
    with info.step("synthetic videos (exact physics + randomization)"):
        train = dataset(n_train, seed=100, defect=False)
        test = [v for v in dataset(n_test, seed=5000)]
        test_def = dataset(max(2, n_test // 5), seed=7000, defect=True)
    with info.step("checks on ground truth and edits"):
        gt = [physics_checks(v) for v in test + test_def]
        bad = {k: [physics_checks(corrupt(v, k)) for v in test] for k in KINDS}
```

Testes e status de validação

- Módulos com pelo menos um teste que os importa: 8 de 31.
- Funções de teste em arquivos que importam algum módulo do pacote: 94.

Limitações conhecidas

- A maior parte dos módulos não tem teste que os importe diretamente; os testados são `physical_world`, `inspection_data`, `robot_perception`, `physics_tools`, `orchestrator`, `dataset` e `model`.
- Os docstrings citam pacotes com nomes antigos que não existem no repositório (`pinneapple_pinn`, `pinneapple_symbolic`, `pinneapple_validate`, `pinneapple_uq`, `pinneapple_transfer`, `pinneapple_meta`); o código usa `pinneapple_physics`, `pinneapple_analysis`, `pinneapple_adaptation` e `pinneapple_simulation`. Também citam `MAMLWorldModel`, `ReptileWorldModel` e `make_pipe_bend`, que não existem.
- `PhysicsFoundationModel` só suporta grades 1D e 2D (3D ainda não, segundo o docstring).
- `PhotorealisticEnhancer`: o backend padrão é identidade, `local_diffusion` exige `diffusers` e `cosmos` é placeholder ainda indisponível; backends ausentes caem para identidade com aviso.
- `PhysicsSolverBridge` e `PhysicsSimulator` caem para solvers PyTorch embutidos (com aviso) quando OpenFOAM, FEniCS ou checkpoints PINN/FNO não estão disponíveis.
- `ScenarioGenerator.from_prompt` e o `PhysicsOrchestrator` usam heurísticas: extração por palavras-chave e um plano fixo por tipo de problema, sem modelo de linguagem.
- `inspection_data` e `robot_perception` precisam de OpenUSD (`pxr`); `robot_perception` precisa do NVIDIA Newton; `CosmosAdapter` precisa de um endpoint NIM Cosmos e de `imageio` com ffmpeg.

- Em `training_hooks`, `PhysicsDecoderDataset` devolve um embedding zero como placeholder quando os embeddings não foram pré-calculados.

7. `pinneapple_perception`

Objetivo

`pinneapple_perception` extrai observações físicas de imagens, vídeo e áudio para usar como alvos de `DataConstraint` ou como geometria de contorno no resto do PINNeAPPle. É a direção inversa de `pinneapple_worldmodel`, que renderiza simulações em imagens: aqui dados reais entram e saem campos de velocidade, contornos e frequências modais.

As três técnicas centrais dependem só de numpy/scipy. `video_piv` faz PIV por correlação cruzada (a técnica padrão da mecânica dos fluidos experimental), e o resultado (x, y, u, v) entra direto em `solve_pde/compile_problem` como restrição de dados, por exemplo num preset de Navier-Stokes incompressível. `image_geometry` extrai contornos de fotos ou scans de seções de peças para `pinneapple_design.geometry`. `audio_modal` encontra frequências de ressonância de um teste de impacto para calibrar presets de dinâmica estrutural.

Há também ferramentas de apoio e um fluxo de sensoriamento remoto: `piv_tools` lê vídeos com ffmpeg e orienta a escolha de parâmetros do PIV; `sentinel2` busca imagens Sentinel-2 no Google Earth Engine e `change_detection` compara datas por banda e por NDVI e gera relatório HTML. Esse fluxo veio de um produto de monitoramento por satélite dos PINNeAPPle-apps, com correções documentadas no docstring.

Números do inventário

módulos	linhas	classes públicas	funções públicas	testes em arquivos que importam o pacote	módulos sem teste direto
7	1.037	6	19	29 (3 arquivos)	2

Submódulos

submódulo	o que faz	módulos	classes + funções	testes*
`audio_modal`	Frequências dominantes de vibração a partir de áudio.	1	1	0
`change_detection`	Detecção de mudança multiespectral (bandas e NDVI) com figura e relatório.	1	10	13
`image_geometry`	Contorno e ajuste de círculo a partir de imagem.	1	2	0
`piv_tools`	Leitura de vídeo, grade, resumo e guia de parâmetros para PIV.	1	6	6
`sentinel2`	Busca de imagens Sentinel-2 no Google Earth Engine.	1	4	13
`video_piv`	PIV por correlação cruzada para campos de velocidade.	1	2	6

* maior número de testes que importam um módulo do submódulo.

Módulos e API pública

`pinneapple_perception`

Fachada do pacote: reexporta PIV (`piv_velocity_field`, `piv_velocity_sequence`), extração de contorno de imagem e frequências dominantes de áudio. O sinalizador 'experimental' do inventário vem da expressão 'experimental fluid dynamics' no docstring, não de um aviso.

`pinneapple_perception/__init__.py` · 55 linhas · testes: 23 · palavras-chave no código (heurística; o texto acima diz o status real): experimental

pinneapple_perception.audio_modal

Extrai as frequências dominantes de vibração/acústica de uma gravação por picos da FFT com refinamento parabólico sub-bin, para calibrar presets de dinâmica estrutural. [sem teste direto; exercitado via `tests/test_perception.py`]

`pinneapple_perception/audio_modal.py` · 95 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`extract_dominant_frequencies`	função	`(waveform, sample_rate, n_peaks=5, min_freq=0.0, max_freq=None, window='hann') -> List[Tu…)`	Devolve os N picos de frequência mais proeminentes como (frequência em Hz, amplitude), ordenados por amplitude.

pinneapple_perception.change_detection

Detecção de mudança entre duas imagens multiespectrais do mesmo local: diferença média por banda em refletância, mudança de NDVI com banda NIR real, figura de comparação e relatório HTML. O módulo remove o NIR inventado (placeholder) do notebook original; não é ele próprio um placeholder.

`pinneapple_perception/change_detection.py` · 288 linhas · testes: 13 · palavras-chave no código (heurística; o texto acima diz o status real): placeholder/stub citado no docstring

nome	tipo	assinatura	o que faz
`BandChangeResult`	classe	`(band, mean_abs_diff_raw, mean_abs_diff_reflectance, threshold, exceeds_threshold)`	Resultado da checagem de uma banda: diferença média bruta e em refletância, limiar e se foi excedido.
`NdviChangeResult`	classe	`(mean_abs_diff, threshold, exceeds_threshold)`	Resultado da mudança de NDVI: diferença média absoluta, limiar e se foi excedido.
`PairAnalysis`	classe	`(date_a, date_b, band_results, ndvi_result, any_change, narrative_html)`	Análise de um par de datas: resultados por banda, resultado de NDVI (se houver), se houve mudança e narrativa HTML.
`band_difference`	função	`(img_a, img_b, band_index, band_name, threshold=DEFAULT_BAND_DIFF_THRESHOLD, reflectance_…)`	Diferença média absoluta de uma banda entre duas imagens, convertida em refletância e comparada ao limiar.
`compute_ndvi`	função	`(red, nir, eps=1e-06) -> np.ndarray`	NDVI = (NIR − RED) / (NIR + RED + eps) a partir de bandas reais; não fabrica nenhuma banda.
`ndvi_change`	função	`(ndvi_a, ndvi_b, threshold=DEFAULT_NDVI_THRESHOLD) -> NdviChangeResult`	Compara dois mapas de NDVI pela diferença média absoluta contra um limiar.
`analyze_pair`	função	`(rgb_a, rgb_b, date_a, date_b, nir_a=None, nir_b=None, band_diff_threshold=DEFAULT_BAND_D…)`	Analisa um par de imagens: sempre as três bandas RGB e o NDVI só quando há NIR real das duas datas.
`rgb_preview_stretch`	função	`(img, normalizer=…) -> np.ndarray`	Contraste de exibição para prévias RGB (divide por constante e corta em [0, 1]); sem significado físico.
`render_comparison_figure`	função	`(rgb_a, rgb_b, date_a, date_b, ndvi_a=None, ndvi_b=None) -> bytes`	Gera em PNG a grade de comparação (par RGB e diferença, mapas por banda e, se houver, NDVI antes/depois/delta).
`render_html_report`	função	`(pairs, images_png, lon, lat, buffer_m, has_ndvi) -> str`	Monta um relatório HTML autocontido com as análises dos pares e as figuras.

pinneapple_perception.image_geometry

Extrai uma nuvem de pontos de contorno de uma imagem (foto ou scan de seção de peça) e ajusta um círculo por mínimos quadrados, para usar como geometria de domínio. [sem teste direto; exercitado via `tests/test_perception.py`]

`pinneapple_perception/image_geometry.py` · 114 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`extract_boundary_points`	função	`(image, threshold=None, units_per_pixel=1.0, ordered=True) -> np.ndarray`	Extrai o contorno da região de primeiro plano da imagem como nuvem de pontos, opcionalmente ordenada e em unidades físicas.

nome	tipo	assinatura	o que faz
estimate_bounding_circle	função	(points) -> Tuple[float, float, float]	Ajuste algébrico de círculo (método de Kasa) a uma nuvem de pontos de contorno; devolve centro e raio.

pinneapple_perception.piv_tools

Utilitários em volta do PIV: decodifica vídeo (ffmpeg/ffprobe) ou arquivos de imagem em quadros, calcula a grade de vetores, resume um resultado e sugere janela/margem a partir do deslocamento esperado.

pinneapple_perception/piv_tools.py · 128 linhas · testes: 6

nome	tipo	assinatura	o que faz
FFmpegMissing	classe	()	Erro quando ffmpeg ou ffprobe não está no PATH.
decode_video	função	(path) -> np.ndarray	Lê um vídeo como quadros em tons de cinza (T, H, W) usando ffprobe e ffmpeg.
decode_image_frames	função	(files) -> np.ndarray	Converte pares (nome, bytes) de imagens em quadros em tons de cinza, ordenados pelo nome.
piv_grid_count	função	(height, width, window_size, search_margin, step=None, *, margin=None) -> tuple[int, int]	Número de linhas e colunas de janelas de interrogação que o PIV usa num quadro H x W.
summarize_piv	função	(seq, frame_shape, window_size, search_margin, step=None) -> dict	Resume contagens e velocidades de um resultado de piv_velocity_sequence.
piv_parameter_guide	função	(displacement_px, width_px, height_px) -> dict	Sugere janela, margem de busca e passo para um deslocamento esperado em pixels por quadro, com avisos.

pinneapple_perception.sentinel2

Busca imagens Sentinel-2 de refletância de superfície de um ponto no Google Earth Engine (bandas B4, B3, B2 e o NIR real B8) para change_detection; exige earthengine-api e credenciais.

pinneapple_perception/sentinel2.py · 136 linhas · testes: 13

nome	tipo	assinatura	o que faz
EarthEngineUnavailable	classe	()	Erro quando ee.Initialize() falha (sem credencial, sem projeto ou API desativada), com a mensagem real e os passos de correção.
Sentinel2Frame	classe	(date, rgb, nir)	Uma imagem Sentinel-2: data, array RGB e banda NIR.
ee_initialize	função	(project=None) -> None	Inicializa o Earth Engine com conta de serviço (variáveis de ambiente) ou com a credencial persistente padrão.
fetch_sentinel2_sequence	função	(lon, lat, buffer_m=5000.0, start_date='2024-01-01', end_date='2025-07-27', max_cloud_pct…)	Busca as N imagens mais recentes e com menos nuvens em torno de um ponto e as devolve como Sentinel2Frame.

pinneapple_perception.video_piv

Velocimetria por imagem de partículas (PIV) por correlação cruzada via FFT com refinamento sub-pixel: transforma quadros de vídeo de um escoamento em campo de velocidade pronto para DataConstraint.

pinneapple_perception/video_piv.py · 221 linhas · testes: 6 · palavras-chave no código (heurística; o texto acima diz o status real): experimental

nome	tipo	assinatura	o que faz
piv_velocity_field	função	(frame_a, frame_b, window_size=32, search_margin=16, step=None, dt=1.0, units_per_pixel=1…)	Campo de velocidade PIV entre dois quadros: janelas, correlação cruzada normalizada, pico sub-pixel e conversão por dt e units_per_pixel.
piv_velocity_sequence	função	(frames, dt=1.0, **piv_kwargs) -> Dict[str, np.ndarray]	Roda o PIV em cada par consecutivo de quadros e empilha os resultados com uma coluna de tempo t.

Exemplos de uso reais do repositório

PIV entre dois quadros — `pinneapple_perception/__init__.py`

```
>>> from pinneapple_perception import piv_velocity_field
>>> field = piv_velocity_field(frame_a, frame_b, dt=1/30, units_per_pixel=1e-3)
>>> # field["x"], field["y"], field["u"], field["v"] -- feed directly into
>>> # a DataConstraint on a navier_stokes_incompressible-family preset.
```

Frequências modais conhecidas recuperadas — `tests/test_perception.py`

```
peaks = extract_dominant_frequencies(waveform, sr, n_peaks=2)
assert len(peaks) == 2
freqs_found = sorted(p[0] for p in peaks)
assert abs(freqs_found[0] - f1) < 0.1, f"expected ~{f1} Hz, got {freqs_found[0]}"
assert abs(freqs_found[1] - f2) < 0.1, f"expected ~{f2} Hz, got {freqs_found[1]}"
```

Testes e status de validação

- Módulos com pelo menos um teste que os importa: 5 de 7.
- Funções de teste em arquivos que importam algum módulo do pacote: 29.

Limitações conhecidas

- `sentinel2` exige o pacote `earthengine-api` (ee), credenciais e um projeto com a API do Earth Engine ativada; sem isso levanta `EarthEngineUnavailable`.
- `piv_tools.decode_video` exige `ffmpeg` e `ffprobe` no PATH (senão levanta `FFmpegMissing`).
- `compute_ndvi` exige a banda NIR real (B8): não há fallback que invente NIR; sem NIR o NDVI simplesmente não é calculado.
- O limiar quantitativo de mudança usa escala de refletância 10000 por padrão; `reflectance_scale=3000` reproduz o comportamento antigo do notebook, que o docstring chama de fisicamente inconsistente.
- `extract_boundary_points` usa por padrão a média da imagem como limiar, adequado só para imagens aproximadamente bimodais.
- `audio_modal` e `image_geometry` não têm teste que os importe pelo caminho do módulo; são testados pela fachada em `tests/test_perception.py`.

8. `pinneapple_quantum`

Objetivo

`pinneapple_quantum` (PINNeAPPle Quantum Module, versão 0.1.0) é uma extensão experimental de aprendizado de máquina informado por física híbrido clássico-quântico. Ele usa circuitos quânticos variacionais (VQC) como ansatz de função de onda, ou como camada dentro de uma rede clássica, e os treina com perdas da equação de Schrödinger.

Os fluxos previstos são: VQ-PINN (o circuito é ψ(x) e a perda é o quociente de Rayleigh, como no VQE), PINN híbrida (MLP clássica + circuito), aprendizado de Hamiltoniana a partir de energias medidas e kernel quântico para métodos de kernel clássicos. `pipeline.presets.solve()` imita a API `solve` do PINNeAPPle para esses casos.

A execução pode usar PennyLane, Qiskit/Aer ou um simulador clássico de vetor de estado em PyTorch. Se as bibliotecas quânticas não estiverem instaladas, o pacote cai automaticamente para o simulador clássico, então a API toda continua disponível. O pacote é isolado do resto do repositório: não importa outros pacotes `pinneapple_*` e nenhum teste o importa.

Números do inventário

módulos	linhas	classes públicas	funções públicas	testes em arquivos que importam o pacote	módulos sem teste direto
16	2.636	17	24	0 (0 arquivos)	16

Submódulos

submódulo	o que faz	módulos	classes + funções	testes*
`backends`	Backends PennyLane, Qiskit e simulador clássico.	2	7	0
`circuits`	Configuração e construção de circuitos variacionais.	2	4	0
`data`	Estados, observáveis, datasets e amostradores de domínio.	2	7	0
`loss`	Perdas da equação de Schrödinger e potenciais.	2	10	0
`models`	`QuantumModel` e `HybridModel` como módulos PyTorch.	3	2	0
`pipeline`	Presets de alto nível e `solve()`.	2	5	0
`training`	Treinador híbrido e gradiente parameter-shift.	2	4	0

* maior número de testes que importam um módulo do submódulo.

Módulos e API pública

`pinneapple_quantum`

Fachada do Módulo Quântico (PQM): reexporta configuração de circuito, backends, modelos, perdas de Schrödinger, datasets, treinador e presets, e oferece os atalhos `build_circuit` e `backend_info`. [experimental] [sem teste]

`pinneapple_quantum/__init__.py` · 210 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`build_circuit`	função	`(n_qubits=4, depth=2, ansatz='hardware_efficient', encoding='angle', n_observables=1, bac…)`	Atalho que monta um `QuantumModel` a partir de argumentos simples (qubits, profundidade, ansatz, codificação, backend).
`backend_info`	função	`(provider='pennylane') -> dict`	Devolve o dict de informações do backend pedido (ou do fallback resolvido).

`pinneapple_quantum.backends`

Subpacote dos backends de execução quântica (reexporta `backend`). [sem teste]

`pinneapple_quantum/backends/__init__.py` · 4 linhas · **sem teste direto**

`pinneapple_quantum.backends.backend`

Adaptador de backend quântico: PennyLane, Qiskit/Aer ou simulador clássico de vetor de estado em PyTorch; `get_backend` cai para o clássico se a biblioteca não estiver instalada. A base `QuantumBackend` levanta NotImplementedError. [sem teste]

`pinneapple_quantum/backends/backend.py` · 324 linhas · **sem teste direto** · palavras-chave no código (heurística; o texto acima diz o status real): levanta `NotImplementedError` em algum caminho

nome	tipo	assinatura	o que faz
`BackendConfig`	classe	`(provider, device_name, diff_method, shots)`	Configuração do backend: provedor, nome do dispositivo, método de diferenciação e número de shots.
`QuantumBackend`	classe	`(config)`	Base abstrata de backend; `name` e `build_qnode` levantam NotImplementedError.
`PennyLaneBackend`	classe	`()`	Backend PennyLane (simuladores CPU/GPU e QPUs reais).

nome	tipo	assinatura	o que faz
`QiskitBackend`	classe	`()`	Backend Qiskit/Aer via plugin PennyLane-Qiskit; sem ele, usa o backend clássico.
`ClassicalBackend`	classe	`()`	Simulador clássico de vetor de estado em PyTorch, sem dependências quânticas.
`ClassicalQNode`	classe	`(circuit_fn, n_qubits)`	Executa uma descrição de circuito gravada aplicando as portas como unitárias complexas a um vetor de estado.
`get_backend`	função	`(config=None) -> 'QuantumBackend'`	Cria o backend a partir da configuração, caindo para o clássico se PennyLane ou Qiskit não estiverem instalados.

`pinneapple_quantum.circuits`

Subpacote de construção de circuitos (reexporta `base`). [sem teste]

`pinneapple_quantum/circuits/__init__.py` · 8 linhas · **sem teste direto**

`pinneapple_quantum.circuits.base`

Define o circuito variacional (ansatz e codificação) e o constrói como QNode do PennyLane ou como gravação de portas para o simulador clássico; inclui `ClassicalVQC`, uma aproximação por MLP. [sem teste]

`pinneapple_quantum/circuits/base.py` · 300 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`QuantumCircuitConfig`	classe	`(n_qubits, depth, ansatz, encoding, n_observables)`	Especificação do circuito variacional: qubits, profundidade, ansatz (`hardware_efficient`, `strongly_entangling`, `efficient_su2`), codificação (`angle`, `amplitude`, `iqp`) e número de observáveis.
`ClassicalVQC`	classe	`(config)`	Aproximação por MLP de um circuito variacional, com saída em [-1, 1]; o próprio docstring diz que não reproduz a dinâmica das portas.
`build_pennylane_circuit`	função	`(config, backend)`	Constrói um QNode do PennyLane para a configuração dada.
`build_classical_circuit`	função	`(config, backend)`	Constrói o mesmo circuito gravando as portas num protocolo que o `ClassicalQNode` executa.

`pinneapple_quantum.data`

Subpacote de dados quânticos (reexporta `quantum_dataset`). [sem teste]

`pinneapple_quantum/data/__init__.py` · 20 linhas · **sem teste direto**

`pinneapple_quantum.data.quantum_dataset`

Camada de dados quânticos: estados, observáveis, datasets de colocação e de cadeia de spins (Ising), amostradores de domínio 1D/2D e autoestados analíticos do oscilador harmônico. [sem teste]

`pinneapple_quantum/data/quantum_dataset.py` · 290 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`QuantumState`	classe	`(coords, psi_real, psi_imag, meta)`	Estado quântico puro psi(x) sobre um domínio espacial, com partes real e imaginária e norma.
`QuantumObservable`	classe	`(expectation, variance, operator_names)`	Observável medido: valor esperado, variância e nomes dos operadores.
`QuantumCollocationDataset`	classe	`(x_col, x_bc, psi_exact=None, batch_size=256)`	Dataset de pontos de colocação e de contorno para treinar PINNs quânticas, com solução exata opcional.
`SpinChainDataset`	classe	`(n_spins=4, n_samples=1024, J=1.0, h=0.5, seed=42)`	Configurações aleatórias de spins ±1 com energia de Ising (acoplamento J e campo h).
`sample_domain_1d`	função	`(x_min=0.0, x_max=1.0, n_col=1000, n_bc=50, device='cpu', dtype=torch.float32) -> Tuple[t…)`	Amostra pontos de colocação e de contorno num intervalo 1D.

nome	tipo	assinatura	o que faz
sample_domain_2d	função	(x_min=-2.0, x_max=2.0, y_min=-2.0, y_max=2.0, n_col=4096, n_bc=200, device='cpu', dtype=…)	Amostra pontos de colocação e de contorno num retângulo 2D.
exact_eigenstates_1d_harmonic	função	(x, n_state=0, omega=1.0, hbar=1.0, mass=1.0) -> Tuple[torch.Tensor, float]	Calcula o n-ésimo autoestado analítico do oscilador harmônico 1D e sua energia.

pinneapple_quantum.loss

Subpacote de perdas físicas (reexporta `schrodinger`). [sem teste]

pinneapple_quantum/loss/__init__.py · 26 linhas · **sem teste direto**

pinneapple_quantum.loss.schrodinger

Perdas físicas para a equação de Schrödinger: potenciais prontos, energia variacional (quociente de Rayleigh), resíduo forte independente e dependente do tempo, normalização e valor esperado de Hamiltoniana matricial. [sem teste]

pinneapple_quantum/loss/schrodinger.py · 344 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
harmonic_potential	função	(x, omega=1.0, mass=1.0) -> torch.Tensor	Potencial do oscilador harmônico, V = ½ m ω² |x|².
infinite_well_potential	função	(x, width=1.0) -> torch.Tensor	Poço infinito: V = 0 no domínio, com as paredes impostas por condições de Dirichlet.
coulomb_potential	função	(x, charge=1.0, eps=0.001) -> torch.Tensor	Potencial coulombiano regularizado na origem (átomo hidrogenoide).
double_well_potential	função	(x, a=1.0, b=0.0) -> torch.Tensor	Poço duplo (x² − a)² + b x, simétrico ou assimétrico.
get_potential	função	(name_or_fn) -> Callable	Resolve um potencial pelo nome ou devolve a função se já for chamável.
energy_loss	função	(model, x, potential='harmonic_oscillator', hbar=1.0, mass=1.0, **potential_kwargs) -> Tu…)	Energia variacional (quociente de Rayleigh) da equação de Schrödinger independente do tempo; devolve energia e norma.
pde_residual_loss	função	(model, x, eigenvalue, potential='harmonic_oscillator', hbar=1.0, mass=1.0, **potential_k…)	Resíduo forte −(ħ²/2m)∇²ψ + Vψ − Eψ para um autovalor dado.
time_dependent_loss	função	(model, x, t, potential='harmonic_oscillator', hbar=1.0, mass=1.0, **potential_kwargs) ->…)	Resíduo da equação de Schrödinger dependente do tempo, com ψ complexo como duas saídas.
normalization_loss	função	(model, x, domain_volume=1.0) -> torch.Tensor	Restrição suave para ∫|ψ|² dx ≈ 1.
hamiltonian_expectation	função	(model, x, H_matrix) -> torch.Tensor	Valor esperado discreto ⟨ψ|H|ψ⟩ para Hamiltonianas em forma de matriz.

pinneapple_quantum.models

Subpacote de modelos (reexporta `QuantumModel` e `HybridModel`). [sem teste]

pinneapple_quantum/models/__init__.py · 5 linhas · **sem teste direto**

pinneapple_quantum.models.hybrid_model

`HybridModel`: combina uma rede clássica e um `QuantumModel` num grafo diferenciável, em série (clássico primeiro ou quântico primeiro) ou em paralelo com fusão linear. [sem teste]

pinneapple_quantum/models/hybrid_model.py · 112 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`HybridModel`	classe	`(classical_net, quantum_model, mode='classical_first', out_dim=None)`	Modelo híbrido que compõe rede clássica e circuito quântico em modo `classical_first`, `quantum_first` ou `parallel`.

pinneapple_quantum.models.quantum_model

`QuantumModel`: circuito variacional exposto como `nn.Module` do PyTorch, devolvendo valores esperados de Pauli-Z por amostra. [sem teste]

`pinneapple_quantum/models/quantum_model.py` · 133 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`QuantumModel`	classe	`(circuit_config, backend_config=None, output_scale=1.0)`	Circuito variacional como `nn.Module`, com pesos treináveis e escala de saída.

pinneapple_quantum.pipeline

Subpacote de presets de alto nível (reexporta `presets`). [sem teste]

`pinneapple_quantum/pipeline/__init__.py` · 16 linhas · **sem teste direto**

pinneapple_quantum.pipeline.presets

Presets prontos: VQ-PINN, PINN híbrido, aprendizado de Hamiltoniana, kernel quântico e um `solve()` que imita a API do PINNeAPPle. [sem teste]

`pinneapple_quantum/pipeline/presets.py` · 490 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`vq_pinn`	função	`(equation='schrodinger_1d', potential='harmonic_oscillator', n_qubits=4, depth=3, ansatz=…)`	Monta e treina uma VQ-PINN para Schrödinger com o potencial escolhido e devolve resultados (energia estimada e exata, histórico, modelo).
`hybrid_pinn`	função	`(classical_in=1, classical_hidden=32, n_qubits=4, depth=2, potential='harmonic_oscillator…)`	Treina uma PINN híbrida: MLP clássica codifica a entrada para o circuito quântico.
`learn_hamiltonian`	função	`(data, labels, n_spins=4, epochs=2000, lr=0.001, device='cpu', seed=42) -> Dict[str, Any]`	Ajusta uma matriz Hamiltoniana simétrica tal que ⟨ψ|H|ψ⟩ aproxime as energias medidas.
`quantum_kernel`	função	`(x_train, x_test=None, n_qubits=4, encoding='angle', backend='pennylane') -> torch.Tensor`	Matriz de kernel a partir das expectativas de Pauli-Z do circuito (cosseno ao quadrado); no backend clássico usa um kernel RBF.
`solve`	função	`(problem='quantum', equation='schrodinger', potential='harmonic', method='vq_pinn', backe…)`	Ponto de entrada no estilo `solve()` do PINNeAPPle que despacha para `vq_pinn` ou `hybrid_pinn`.

pinneapple_quantum.training

Subpacote de treino (reexporta `trainer`). [sem teste]

`pinneapple_quantum/training/__init__.py` · 9 linhas · **sem teste direto**

pinneapple_quantum.training.trainer

Treinador híbrido clássico-quântico com perda física, de contorno e de normalização; a opção de gradiente por parameter-shift não está implementada e cai para backprop com aviso. [sem teste]

`pinneapple_quantum/training/trainer.py` · 345 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`QTrainerConfig`	classe	`(epochs, lr, optimizer, quantum_gradient, lambda_norm, lambda_bc, print_every, device)`	Configuração do treinador quântico: épocas, taxa, otimizador, modo de gradiente quântico e pesos de normalização e contorno.
`QTrainHistory`	classe	`(epochs, total_loss, pde_loss, energy, norm_sq)`	Histórico do treino: perda total, perda da EDP, energia e norma por época.
`QTrainer`	classe	`(model, physics_loss_fn, bc_loss_fn=None, config=None)`	Treinador híbrido que minimiza perda física + contorno + normalização sobre pontos de colocação.
`parameter_shift_gradient`	função	`(loss_fn, weights, shift=math.pi / 2) -> torch.Tensor`	Gradiente exato de `loss_fn` em relação a `weights` pela regra parameter-shift.

Exemplos de uso reais do repositório

Quick start do docstring do pacote — `pinneapple_quantum/__init__.py`

```
>>> cfg = pqm.QuantumCircuitConfig(n_qubits=4, depth=3)
>>> model = pqm.QuantumModel(cfg)

>>> # Physics loss
>>> x = torch.linspace(-3, 3, 500).unsqueeze(-1)
>>> energy, norm = pqm.loss.energy_loss(model, x, "harmonic_oscillator")

>>> # One-line solve
>>> result = pqm.solve(
...     problem="quantum",
...     equation="schrodinger",
...     potential="harmonic",
...     method="vq_pinn",
...     n_qubits=4, depth=3, epochs=2000,
... )
```

Testes e status de validação

- Módulos com pelo menos um teste que os importa: 0 de 16.
- Funções de teste em arquivos que importam algum módulo do pacote: 0.

Limitações conhecidas

- Nenhum módulo do pacote tem teste.
- `QTrainerConfig.quantum_gradient='parameter_shift'` não está implementado no treino: o `QTrainer` emite aviso e usa backprop comum, porque a reconstrução anterior dava gradientes errados com o termo laplaciano de `energy_loss`.
- `quantum_kernel` não calcula a fidelidade exata $|\langle\varphi(x_i)|\varphi(x_j)\rangle|^2$, e sim um proxy pelas expectativas de Pauli-Z; no backend clássico troca por um kernel RBF.
- `ClassicalVQC` é uma aproximação por MLP, não uma simulação fiel; a simulação fiel é o `ClassicalQNode` (vetor de estado).
- PennyLane e Qiskit/Aer são opcionais; sem eles tudo roda no simulador clássico, que avalia amostra por amostra.
- `QuantumBackend` é base abstrata (`name` e `build_qnode` levantam NotImplementedError).

Volume 5 — Confiança, validação e decisão

> Parte do Mapa completo do PINNeAPPle. Gerado por `docs/pt/_mapa/gerar.py` a partir do inventário do commit `e53cdcfb04`.

A camada que confere cada resultado contra a física: validação, incerteza, inversão, o linter `pinneapple check`, registros de evidência, segurança, decisão e a cadeia de porquês.

Sumário

1. pinneapple_analysis
2. pinneapple_check
3. pinneapple_veriphysics
4. pinneapple_security
5. pinneapple_decision
6. pinneapple_reason

1. `pinneapple_analysis`

Objetivo

`pinneapple_analysis` reúne as ferramentas que olham para um modelo já treinado (ou para um conjunto de resultados de simulação) e perguntam: ele é confiável, quanto custa, quais parâmetros importam e onde ele pode ser usado. Esta parte do mapa cobre a raiz do pacote e os submódulos de incerteza (`uncertainty`), confiança (`trust`), sensibilidade (`sensitivity`), custo (`cost`), benchmark com orçamento igual (`benchmark`), surrogates (`surrogate`), assimilação de dados (`data_assimilation`, `state_estimation`), grafos de prova (`proof`), reprodução de artigos (`reproduction`), busca semântica (`retrieval`) e os contratos de estudo (`study_models`, `study_contracts`). Os submódulos `validation` e `inverse_problems` ficam em outra parte.

A raiz oferece dois atalhos: `analyze_model` (validação física + quantificação de incerteza numa chamada) e `invert` (problema inverso com erro gaussiano e regularização de Tikhonov via `InverseProblemSolver`). O submódulo `uncertainty` era o antigo `pinneapple_uq` e junta MC Dropout, ensembles, cabeças aleatóricas, predição conformal, calibração, regressão quantílica, propagação de tolerâncias, caos polinomial, drift e o limite de erro a partir do resíduo da EDP.

Vários submódulos seguem a mesma disciplina: todo número vem de uma execução medida, com dados de teste que nunca escolhem o modelo, e cada resultado vira checagens registradas (`Check`, `TrustReport`, cartão de confiança de seis perguntas do `pinneapple_lab`). O `benchmark` compara métodos (PINN, FNO, DeepONet, MeshGraphNet, MLP) sob o mesmo `ComputeBudget`, com portões de correção e um conselheiro que sugere ajustes; o `surrogate` treina todos os candidatos aplicáveis num conjunto de dados e empacota o melhor com um portão OOD.

A ligação com o resto vem pelos atalhos de `pinneapple` (`pp.benchmark`, `pp.uq`/`pp.uncertainty`, `pp.sensitivity`, `pp.assimilation`, `pp.ComputeBudget`, `pp.trust_card`), pelos contratos de `pinneapple_data.contracts` (`ParameterSpace`, `ModelRef`, `StudyResult`) usados pelos apps de incerteza, sensibilidade e surrogate, pelo zoo de `pinneapple_hub` (modelos `zoo:<nome>`) e pelo `pinneapple_reason`, que monta seu relatório de confiança com `pinneapple_analysis.trust.TrustReport`.

Esta parte cobre quatro submódulos de `pinneapple_analysis`. `inspection` gera sinais sintéticos de ensaio não destrutivo e treina PINNs com a mesma EDP. `inverse_problems` estima parâmetros a partir de medições, diz antes se eles são identificáveis e compara métodos (mínimos quadrados, adjunto, autograd, bayesiano, PINN, EKI). `validation` produz o relatório de `pp.validate` e as checagens que o alimentam, além da injeção de fa-

lhas e da arena red team / blue team. `verification` reúne os blocos do motor de verificação física (análise dimensional, convergência de malha, qualidade de malha, escore de confiança, recomendação de arquitetura, solver e ferramentas), servidos por `apps/verified_physics` e usados por outros apps.

Números do inventário

módulos	linhas	classes públicas	funções públicas	testes em arquivos que importam o pacote	módulos sem teste direto
159	42.004	249	450	939 (83 arquivos)	56

Pasta de testes dedicada: `tests/pinneapple_analysis` (169 funções de teste).

Submódulos

submódulo	o que faz	módulos	classes + funções	testes *
`benchmark`	Benchmarks de métodos de IA física com orçamento igual: preparar artefatos, validar, rodar com cache e gerar relatório com checagens e diagnósticos.	18	113	44
`cost`	Mede custo computacional (tempo, memória, FLOPs, escala, orçamentos, regressões) e oferece o laço de treino que para num orçamento fixo.	4	25	55
`data_assimilation`	Assimilação de dados: 4D-Var forte por autograd, modelos caóticos de teste e reconstrução de campo por processo gaussiano com posicionamento guloso de sensores.	4	13	7
`inspection`	Suíte de ensaio não destrutivo (NDE): gera sinais de inspeção sintéticos para seis modalidades e treina uma PINN com a mesma EDP que gerou os dados.	8	21	13
`inverse_problems`	Problemas inversos: modelos diretos, identificabilidade, calibração, estimadores (mínimos quadrados, adjunto, autograd, MCMC, EKI, PINN), regularização, ruído, sensibilidade e descoberta de termos faltantes.	19	95	151
`proof`	Grafos de prova: argumentos matemáticos como grafos de hipóteses, lemas e teoremas com checagens automáticas e status que se propaga.	3	14	11
`reproduction`	Ferramentas neutras para reproduzir números de artigos: métricas de erro, estatística por semente e regra de status fixada antes da execução.	1	10	5
`retrieval`	Busca semântica local sobre um corpus real (docstrings, presets, código e, opcionalmente, artigos do arXiv) com embeddings de sentence-transformers.	3	11	32
`sensitivity`	Análise de sensibilidade global e local: Sobol com intervalos, Morris, curvas de efeito, derivadas e a regra de quais parâmetros podem ser congelados.	6	21	28
`state_estimation`	Filtros de Kalman estendido (EKF) e de ensemble (EnKF) para estimar estado sequencialmente a partir de observações.	2	4	22
`study_contracts`	Converte espaços de parâmetros, modelos e resultados de estudos de incerteza e sensibilidade para os contratos de `pinneapple_data.contracts`.	1	7	4
`study_models`	Interface única `StudyModel` (`predict(X) -> (n, m)`) com nomes, unidades e faixa de validade) para estudos de incerteza e sensibilidade.	1	3	25
`surrogate`	Estudo de surrogates a partir de resultados de simulação: escolhe candidatos, compara com orçamento igual, mede em dados não vistos e empacota o melhor.	11	41	32
`trust`	Pontuação de confiança antes de consumir a predição de um modelo: `TrustGate`, cartão de confiança de campos e `TrustReport` com decisão de deploy.	4	11	13
`uncertainty`	Quantificação de incerteza (antigo `pinneapple_uq`): aleatória, epistêmica, conformal, calibração, quantis, tolerâncias, drift, caos polinomial e limites a partir do resíduo.	21	79	185
`validation`	Validação de modelos de IA física: o relatório único `pp.validate`, checagens clássicas de PINN (conservação, contorno, simetria), métricas de campo e espectrais, testes metamórficos, rollout, injeção de falhas e a arena red team / blue team.	33	151	151
`verification`	Blocos do motor de verificação física: números adimensionais, convergência de malha (GCI), qualidade de malha, rotulagem de geometria, guarda de fora-de-distribuição, grafo de conhecimento, escore de confiança, recomendações e pipeline com LLM.	19	78	43

* maior número de testes que importam um módulo do submódulo.

Módulos e API pública

`pinneapple_analysis`

Raiz do pacote de análise de modelos: reexporta validação, UQ e problemas inversos e oferece os atalhos `analyze_model` e `invert`.

`pinneapple_analysis/__init__.py` · 173 linhas · testes: 56

nome	tipo	assinatura	o que faz
`analyze_model`	função	`(model, spec, x_test, *, uq_method='mc_dropout', run_validate=True, run_uq=True) -> dict`	Roda validação física e quantificação de incerteza numa chamada e devolve um dicionário combinado.
`invert`	função	`(model, y_obs, sensor_locs, *, noise_std=0.01, lambda_reg=0.001, method='adam', n_iters=2…)`	Atalho de problema inverso com desajuste gaussiano e regularização de Tikhonov, via `InverseProblemSolver`.

`pinneapple_analysis.benchmark`

Entrada do benchmark com orçamento igual (preparar -> validar -> rodar -> relatório); reexporta `run`, `prepare`, `load`, o registro e `ArrayProblem`.

`pinneapple_analysis/benchmark/__init__.py` · 34 linhas · testes: 44

`pinneapple_analysis.benchmark.__main__`

Permite rodar `python -m pinneapple_analysis.benchmark`, repassando os argumentos à CLI. [sem teste]

`pinneapple_analysis/benchmark/__main__.py` · 7 linhas · **sem teste direto**

`pinneapple_analysis.benchmark.advisor`

Diagnósticos medidos no próprio run (históricos, níveis de orçamento, checagens, dados) com severidade, evidência e edição de configuração sugerida. [sem teste]

`pinneapple_analysis/benchmark/advisor.py` · 222 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Diagnosis`	classe	`(severity, topic, subject, finding, evidence, suggestion, edit)`	Um diagnóstico: severidade, tópico, evidência medida, sugestão e, quando possível, a edição exata da configuração.
`diagnose`	função	`(cfg, plan, extracted, checks, arch_cfgs=None, loss_terms=None) -> list[Diagnosis]`	Gera a lista de diagnósticos a partir da configuração, do plano, das métricas extraídas e das checagens.
`apply_suggestions`	função	`(cfg, diagnoses, path) -> BenchmarkConfig`	Grava uma nova configuração com todas as edições sugeridas, para o usuário revisar e rodar.

`pinneapple_analysis.benchmark.api`

API `pp.benchmark`: `prepare`, `run`, `load`, `report` e `BenchmarkResult`, com portões de correção e o conselheiro. [sem teste direto]

`pinneapple_analysis/benchmark/api.py` · 284 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`BenchmarkResult`	classe	`(ws, cfg, plan, arch, banners, blocked)`	Resultado de um benchmark aberto: tabela, relatório, checagens, diagnósticos, exportações e checagem pontual de determinismo.
`prepare`	função	`(config=None, out_dir='benchmark_out', *, force=False, adopt=False, log=print, **kwargs) …)`	Cria e valida os artefatos (config do solver, geometria, dados, custo, hashes) em `out_dir/prepared/` antes de qualquer treino.

nome	tipo	assinatura	o que faz
`run`	função	`(config=None, out_dir='benchmark_out', *, force=False, prepare_missing=True, stages=…, lo…)`	Roda o benchmark; retoma do cache no mesmo `out_dir` e recusa uma configuração diferente sem `force`.
`load`	função	`(out_dir) -> BenchmarkResult`	Abre uma pasta de benchmark terminada ou parcial.
`report`	função	`(out_dir) -> BenchmarkResult`	Abre a pasta com `load` e regenera o relatório.

pinneapple_analysis.benchmark.builtin

Importa e registra os problemas e métodos embutidos; `airfoil` e `inverse_heat` só entram se suas dependências opcionais importarem. [sem teste]

`pinneapple_analysis/benchmark/builtin/__init__.py` · 9 linhas · **sem teste direto**

pinneapple_analysis.benchmark.builtin.airfoil

Problema `airfoil` (aerofólio 2-D, OpenFOAM simpleFoam k-omega SST, Re 4e6) com métodos MLP, POD + MLP, MeshGraphNet e PINN por geometria; `prepare` exige OpenFOAM. [sem teste]

`pinneapple_analysis/benchmark/builtin/airfoil.py` · 654 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`AirfoilProblem`	classe	`()`	Problema registrado `airfoil`: prepara dados OpenFOAM, verifica o solver, define colunas e métricas do aerofólio 2-D.
`MGNMethod`	classe	`()`	Método `mgn` (MeshGraphNet) cujo treino é delegado aos ganchos do problema.
`MLPMethod`	classe	`()`	Método `mlp` que prevê os coeficientes integrais via ganchos do problema.
`MLPFieldsMethod`	classe	`()`	Método `mlp_fields` (POD + MLP) que prevê campos via ganchos do problema.
`PINNRansMethod`	classe	`()`	Método `pinn_rans`: PINN RANS por instância (uma geometria e um ângulo).
`openfoam_available`	função	`() -> bool`	Diz se o bashrc do OpenFOAM existe (`FOAM_BASHRC` ou o caminho padrão).

pinneapple_analysis.benchmark.builtin.airfoil_impl

Implementação do benchmark do aerofólio vinda do experimento 09 do PINNeAPPle-Benchmark: treino de MLP em ensemble, MeshGraphNet, POD + MLP e PINNs de escoamento potencial e RANS. [sem teste]

`pinneapple_analysis/benchmark/builtin/airfoil_impl.py` · 599 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`PotentialNet`	classe	`(cfg, alpha_deg)`	Rede de escoamento potencial com circulação treinável e condição de parede imposta na forma da função de corrente.
`RansNet`	classe	`(cfg, alpha_deg, wall)`	Rede (u, v, p) com não-deslizamento exato na parede, para a PINN RANS.
`EddyViscosity`	classe	`(model, p0, e)`	Viscosidade turbulenta por comprimento de mistura de Prandtl com amortecimento de van Driest, congelada a cada passo (linearização de Picard).
`set_data_dir`	função	`(prepared) -> None`	Define a pasta de dados preparados usada pelas funções de treino.
`load_runs`	função	`(dirs)`	Carrega os runs aplicando o filtro do app aero (status utilizável, sem estol profundo, oscilantes promediados).
`split`	função	`(recs, frac=0.15, seed=3)`	Divisão do app aero: uma fração das formas LHS reservada com todos os seus ângulos; os polares NACA à parte.
`splits`	função	`()`	Monta os conjuntos treino, validação (formas sorteadas), teste, NACA e OOD a partir dos runs.

nome	tipo	assinatura	o que faz
`eval_splits`	função	`(final, S)`	Lista os conjuntos a avaliar: teste, NACA e OOD no run final, só validação na busca.
`coefs`	função	`(recs)`	Extrai a matriz (Cl, Cd, Cm) dos registros.
`coef_scores`	função	`(true, pred) -> dict`	Erros absolutos de Cl e Cm e erro relativo de Cd, por run e em média.
`field_scores`	função	`(true, pred) -> dict`	L2 relativo por run da velocidade e da pressão nos nós do grafo.
`train_mlp`	função	`(cfg, seed, budget, *, final) -> dict`	Treina o ensemble de MLPs dos coeficientes dentro do orçamento e devolve as métricas.
`graph_data`	função	`(lay, recs)`	Monta features de nós e arestas e alvos de campo e coeficientes para o MeshGraphNet.
`train_mgn`	função	`(cfg, seed, budget, *, final, batch=4) -> dict`	Treina o MeshGraphNet (campos e coeficientes) dentro do orçamento.
`train_mlp_fields`	função	`(cfg, seed, budget, *, final) -> dict`	Baseline de campo: POD dos campos de treino e MLP dos parâmetros de forma e ângulo para os coeficientes POD.
`find_case`	função	`(case) -> dict`	Acha um registro de caso pelo id nos conjuntos de avaliação.
`collocation_pool`	função	`(shape, d_min) -> torch.Tensor`	Centros de célula da O-grid do caso dentro do raio do domínio, usados como pontos de colocação.
`wall_distance`	função	`(x, y, p0, e)`	Distância ao polígono da parede, diferenciável quase em toda parte.
`rans_residuals`	função	`() -> list[str]`	Resíduos das equações RANS estacionárias incompressíveis com viscosidade turbulenta (continuidade e momento x e y).
`train_pinn`	função	`(cfg, seed, budget, *, case, kind='potential', ckpt=None) -> dict`	Treina uma PINN (potencial ou RANS) para um aerofólio num ângulo, sem dados de simulação.

`pinneapple_analysis.benchmark.builtin.fields`

Problemas de operador campo -> campo em grade, o `ArrayProblem` para arrays do usuário e os métodos genéricos FNO, DeepONet e PINN por instância; ganchos base levantam NotImplementedError.

`pinneapple_analysis/benchmark/builtin/fields.py` · 614 linhas · testes: 29 · palavras-chave no código (heurística; o texto acima diz o status real): levanta `NotImplementedError` em algum caminho

nome	tipo	assinatura	o que faz
`FieldData`	classe	`(prepared)`	Arrays preparados com normalizadores gaussianos pontuais calculados só no treino.
`FieldOperatorProblem`	classe	`()`	Base dos problemas grade -> grade; reporta L2 relativo do campo e erro relativo de uma integral nos conjuntos PINN, teste e OOD.
`ArrayProblem`	classe	`(id, *, train, val, test, ood=None, data_cost_s=None, qoi=None, pinn_setup=None, title=No…)`	Envolve arrays do próprio usuário como problema de benchmark; sem `pinn_setup` o PINN levanta NotImplementedError.
`FNOMethod`	classe	`()`	Método `fno` (Fourier Neural Operator) para problemas de campo, com checagem de modos de Fourier.
`DeepONetMethod`	classe	`()`	Método `deeponet` para problemas de campo, com checagem das dimensões de branch e trunk.
`PINNMethod`	classe	`()`	Método `pinn`: uma PINN por instância via `pinn_setup` do problema ou seu próprio `train_pinn`.
`train_operator`	função	`(problem, prepared, method, cfg, seed, budget, *, final=True, **_) -> dict`	Treina um operador (FNO ou DeepONet) com Adam dentro do orçamento e devolve as métricas.
`train_field_pinn`	função	`(problem, prepared, cfg, seed, budget, *, split, instance, **_)`	Treina uma PINN para uma instância usando `PINNFactory` e avalia contra o campo verdadeiro.
`pinn_checks`	função	`(problem, prepared, cfg) -> list[Check]`	Portões da PINN: derivadas autograd contra diferenças finitas, resíduo de uma solução exata e ajuste de um conjunto mínimo.
`finite`	função	`(x) -> bool`	Diz se um valor converte para float finito.

pinneapple_analysis.benchmark.builtin.inverse_heat

Problema `inverse_heat`: estimar o coeficiente de convecção h de uma placa a partir de 8 termopares. [sem teste]

`pinneapple_analysis/benchmark/builtin/inverse_heat.py` · 201 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`InverseHeatProblem`	classe	`()`	Problema registrado `inverse_heat`: prepara dados de volume finito, verifica o solver e define as métricas do coeficiente h.

pinneapple_analysis.benchmark.builtin.inverse_heat_impl

Implementação do benchmark de calor inverso: PINN da placa contra surrogate + calibração (MLP, DeepONet) e contra o volume finito com busca de seção áurea. [sem teste]

`pinneapple_analysis/benchmark/builtin/inverse_heat_impl.py` · 228 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Surrogate`	classe	`(method, cfg, xy)`	Rede que mapeia (log h, potência) normalizados para o campo de temperatura 60 x 100 normalizado.
`Data`	classe	`()`	Carrega os dados preparados e normaliza as entradas do surrogate.
`set_data_dir`	função	`(prepared, smoke=False) -> None`	Define a pasta de dados preparados e o modo smoke.
`single_thread`	função	`()`	Força uma thread (OMP e torch) no processo atual.
`fv_field`	função	`(h, power)`	Campo de temperatura da placa pelo modelo de volume finito para um h e uma potência.
`readings`	função	`(power, h_true, seed)`	Leituras ruidosas dos termopares interpoladas do campo de referência.
`generate`	função	`(out, n_train, n_val, workers=4) -> dict`	Gera em paralelo os campos de treino e validação por hipercubo latino em (h, potência) e mede o custo.
`train_surrogate`	função	`(method, cfg, seed, budget)`	Treina o surrogate MLP ou DeepONet dentro do orçamento.
`calibrate`	função	`(field_fn, power, t_meas, x, y)`	Busca de seção áurea de h que minimiza o desajuste nos sensores de um campo dado.
`surrogate_job`	função	`(method, cfg, seed, budget, *, final) -> dict`	Job de surrogate + calibração: treina e, no run final, estima h nos casos.
`pinn_job`	função	`(cfg, seed, budget, *, kind, case) -> dict`	Job da PINN da placa num caso: estima h e mede erros de campo e de ponto quente.
`classical_job`	função	`(seed) -> dict`	Busca de seção áurea usando o próprio modelo de volume finito como 'surrogate'.

pinneapple_analysis.benchmark.builtin.standard

Problemas padrão de aprendizado de operador (Burgers, Darcy, Navier-Stokes de Li et al. 2021) com formulação PINN de condições rígidas e solução exata para o portão de resíduo.

`pinneapple_analysis/benchmark/builtin/standard.py` · 478 linhas · testes: 16 · palavras-chave no código (heurística; o texto acima diz o status real): levanta `NotImplementedError` em algum caminho

nome	tipo	assinatura	o que faz
`Burgers`	classe	`()`	Problema `burgers` (1-D periódico, u(.,0) -> u(.,T)) com formulação PINN e solução exata.
`Darcy`	classe	`()`	Problema `darcy` (-div(a grad u) = 1, u = 0 na borda, a -> u) com formulação PINN.
`NavierStokes`	classe	`()`	Problema `navier_stokes` (vorticidade 2-D periódica, w(.,0) -> w(.,T)) com formulação PINN.
`Bilinear`	classe	`(a)`	Interpolante bilinear de um campo centrado em células e seu gradiente, sem autograd.

nome	tipo	assinatura	o que faz
`HardNet`	função	`(problem, cfg, init=None, t_end=10.0)`	Cria uma `NeuralNetwork` da biblioteca com features de entrada e condições iniciais/de contorno rígidas.

`pinneapple_analysis.benchmark.checks`

Portões de correção de um run: checagens de arquitetura, de cada job, de determinismo, de dispersão entre sementes e o status de cada linha. [sem teste direto]

`pinneapple_analysis/benchmark/checks.py` · 175 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`architecture_gates`	função	`(plan) -> list[Check]`	Roda as checagens de arquitetura de cada par problema/método; um portão que quebra conta como falho.
`job_checks`	função	`(rec) -> list[Check]`	Checagens de um job terminado, a partir do seu arquivo de run.
`training_gates`	função	`(ws, plan) -> tuple[list[Check], dict[str, list[Check]]]`	Todas as checagens de jobs agregadas por (problema, método) mais as checagens entre runs.
`determinism_check`	função	`(plan, pid, mid, steps=20) -> Check`	Roda um job duas vezes com a mesma semente e orçamento de passos; os resultados devem bater.
`seed_spread`	função	`(values, subj, what) -> Check`	Checa se a dispersão relativa entre sementes está dentro da tolerância do `pinneapple_veriphysics`.
`row_status`	função	`(arch, train, skipped, did_not_converge) -> str`	Status de uma linha da tabela: checks skipped, INVALID, did not converge ou com avisos.
`summarise`	função	`(checks) -> dict[str, Any]`	Conta checagens por status.

`pinneapple_analysis.benchmark.cli`

Comando `pinneapple benchmark prepare|run|report` (também `python -m pinneapple_analysis.benchmark`).

`pinneapple_analysis/benchmark/cli.py` · 64 linhas · testes: 16

nome	tipo	assinatura	o que faz
`main`	função	`(argv=None) -> int`	Ponto de entrada da CLI `pinneapple benchmark prepare|run|report`.

`pinneapple_analysis.benchmark.config`

`BenchmarkConfig`: os mesmos campos vindos de kwargs Python ou de YAML, resolvidos e com hash. [sem teste direto]

`pinneapple_analysis/benchmark/config.py` · 148 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`BenchmarkConfig`	classe	`(problems, methods, budget, problem_budgets, budget_levels, seeds, n_trials, search_fract…)`	Tudo de que um run depende (problemas, métodos, orçamentos, sementes, busca, FLOPs iguais, workers); lê e grava YAML e calcula hash.

`pinneapple_analysis.benchmark.registry`

Classes base `Problem` e `Method`, `Check`, `Column` e o registro com decoradores `register_problem`/`register_method`; os ganchos não implementados levantam NotImplementedError. [sem teste direto]

`pinneapple_analysis/benchmark/registry.py` · 251 linhas · **sem teste direto** · palavras-chave no código (heurística; o texto acima diz o status real): levanta `NotImplementedError` em algum caminho

nome	tipo	assinatura	o que faz
`Check`	classe	`(name, stage, subject, passed, value, threshold, detail, severity)`	Um portão ou item de validação; `passed` None significa não aplicável, sempre com motivo.

nome	tipo	assinatura	o que faz
`Column`	classe	`(key, label, set, metric, kind, pct)`	Uma coluna da tabela: uma métrica das instâncias de um conjunto, de campo ou integral.
`Problem`	classe	`(**options)`	Classe base de problema de benchmark; subclasses definem `id`, `title` e os ganchos de que precisam.
`Method`	classe	`(id, title, per_instance, outputs)`	Classe base de método de benchmark (espaço de hiperparâmetros, treino, checagens de arquitetura).
`register_problem`	função	`(problem_id)`	Decorador que registra uma classe de problema sob um id.
`register_method`	função	`(method_id)`	Decorador que registra uma classe de método sob um id.
`get_problem`	função	`(p) -> Problem`	Devolve a instância de problema registrada para um id (ou a própria instância).
`get_method`	função	`(m) -> Method`	Devolve a instância de método registrada para um id (ou a própria instância).
`problems`	função	`() -> list[str]`	Lista os ids de problemas registrados.
`methods`	função	`() -> list[str]`	Lista os ids de métodos registrados.

`pinneapple_analysis.benchmark.report`

Relatório do benchmark: tabelas com n em cada cabeçalho, custo de dados, break-even, seções de checagens, validação, cartão de confiança e diagnósticos, figuras e summary.json.

`pinneapple_analysis/benchmark/report.py` · 648 linhas · testes: 16

nome	tipo	assinatura	o que faz
`fmt_s`	função	`(x) -> str`	Formata segundos em µs, ms, s ou h.
`collect`	função	`(plan) -> dict`	Lê os arquivos de run e monta {(problema, método): {nível: {semente: métricas}}} mais custos.
`equal_flops_table`	função	`(plan, col, pid) -> dict | None`	Linhas do braço de FLOPs iguais: métrica principal nas instâncias comuns a todos e no conjunto completo.
`cell`	função	`(seeds, set_, metric) -> dict | None`	Célula da tabela: média e desvio entre sementes de uma métrica num conjunto.
`build_tables`	função	`(plan, col, row_status) -> dict`	Monta as linhas por problema com células, custos e status.
`markdown`	função	`(plan, tables, checks, validation, trust, diagnoses, banners, studies=None) -> str`	Escreve o relatório Markdown completo (hardware, tabelas, checagens, validação, confiança, diagnósticos, estudos).
`write_csv`	função	`(path, plan, tables) -> None`	Grava as tabelas em CSV, uma linha por célula.
`figures`	função	`(plan, col, tables, fig_dir, banners) -> list[str]`	Desenha as figuras do benchmark (erro contra orçamento, resumo) com matplotlib.
`summary_metrics`	função	`(prob)`	Linhas da figura de resumo de um problema: rótulo, tipo, métricas de teste e OOD.
`to_dataframe`	função	`(plan, tables)`	Converte as tabelas num `pandas.DataFrame`.
`collect_studies`	função	`(plan) -> dict`	Lê dos runs o estudo de orçamento e as ablações (auditoria do setup).
`studies_markdown`	função	`(plan, studies) -> list[str]`	Seções Markdown do estudo de orçamento e das ablações.
`study_metric_key`	função	`(prob, method_id, pts) -> str | None`	Escolhe a métrica da curva do estudo de orçamento: a métrica de convergência do próprio método, ou a primeira coluna.
`studies_figure`	função	`(plan, studies, fig_dir) -> list[str]`	Figura das curvas do estudo de orçamento, um painel por par problema/método.

`pinneapple_analysis.benchmark.runner`

Maquinário do benchmark: workspace em out_dir, estágio de preparo, cache de jobs e estágios de busca, final e FLOPs iguais, um processo por job. [sem teste direto]

pinneapple_analysis/benchmark/runner.py · 397 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
ConfigMismatch	classe	()	Erro quando o out_dir já tem uma configuração diferente e force não foi dado.
PreparationError	classe	()	Erro quando os artefatos preparados faltam, estão velhos ou falharam na validação.
Workspace	classe	(out_dir, config)	A pasta out_dir: caminhos de runs e preparados, guarda de configuração e leitura de resultados.
Plan	classe	(ws, cfg, problems, methods)	Especificações dos jobs de um benchmark (busca, final, estudo, ablação, FLOPs iguais), com ids compatíveis com o experimento 09.
prepare_problem	função	(ws, problem, *, n_workers=1, force=False, adopt=False, log=print) -> dict[str, Any]	Cria ou reutiliza os artefatos de um problema, verifica o solver, checa os dados e roda a validação; devolve o manifesto.
require_prepared	função	(ws, problem, allow_failed) -> dict	Exige artefatos preparados, atuais e com hashes corretos; senão levanta PreparationError.
run_job	função	(spec, out_dir, objects=None) -> str	Roda um job neste processo e grava runs/<id>.json com o resultado ou o traceback.
execute_jobs	função	(ws, jobs, *, n_workers, objects=None, log=print) -> None	Executa os jobs pendentes, os mais longos primeiro, em processos paralelos ou em série.
lvl	função	(x) -> str	Rótulo de nível de orçamento no formato L0.333.

pinneapple_analysis.benchmark.validation

Aplica a pilha de validação do PINNeAPPle (auditoria física, envelope, GCI, malha, y+, preflight) aos artefatos preparados e monta o cartão de confiança.

pinneapple_analysis/benchmark/validation.py · 159 linhas · testes: 16

nome	tipo	assinatura	o que faz
physics_audit_check	função	(subject, name, family, equations, assumptions, features) -> Check	Checagem de auditoria física de uma formulação (família, equações, hipóteses).
envelope_checks	função	(subject, train, test, ood, features=None) -> list[Check]	Checagens de envelope de aplicabilidade: teste dentro e OOD fora do envelope do treino.
gci_check	função	(subject, quantity, values_fine_to_coarse, sizes_fine_to_coarse, max_gci=0.05) -> Check	Checagem do índice de convergência de malha (GCI) de uma grandeza contra um limite.
mesh_quality_check	função	(subject, mesh, target='cfd') -> Check	Checagem de qualidade de malha para um alvo (por padrão CFD).
preflight_check	função	(subject, case_dir) -> Check	Checagem preflight de um caso de solver.
yplus_check	função	(subject, first_cell, length, velocity, nu, target=1.0) -> Check	y+ da primeira célula pela estimativa de placa plana contra um alvo.
field_features	função	(x) -> dict[str, float]	Descritores escalares de um campo de entrada (amplitude e centróide espectral) para o envelope.
field_validation	função	(problem, out, grid_study=None) -> list[Check]	Checagens de validação de um problema de campo, incluindo estudo de grade opcional.
trust_card	função	(out_dir, checks, tier, limitations) -> dict[str, Any]	Responde as seis perguntas do cartão de confiança do pinneapple_lab a partir das checagens do benchmark.

pinneapple_analysis.cost

Validação de custo computacional por operação; reexporta profiler, equal_budget e torch_ops.

pinneapple_analysis/cost/__init__.py · 50 linhas · testes: 42

pinneapple_analysis.cost.equal_budget

Comparação com orçamento igual: ComputeBudget, laço train_within_budget, contagem de FLOPs (inclusive autograd.grad), busca aleatória e custo amortizado.

pinneapple_analysis/cost/equal_budget.py · 446 linhas · testes: 55

nome	tipo	assinatura	o que faz
ComputeBudget	classe	(wall_s, flops, steps)	Orçamento de treino em segundos, FLOPs e/ou passos (None = sem limite).
BudgetRun	classe	(steps, wall_s, flops, flops_per_step, stopped_by, budget, history, eval_wall_s)	O que um treino com orçamento gastou e como foi (passos, tempo, FLOPs, motivo da parada, histórico).
SearchTrial	classe	(config, score, info)	Uma tentativa da busca: configuração, score e informações.
SearchResult	classe	(best_config, best_score, trials, n_trials, seed)	Resultado da busca: melhor configuração, melhor score e todas as tentativas.
AmortizedCost	classe	(data_s, train_s, infer_s, per_query_train_s)	Custo de responder novas consultas (dados, treino, inferência) com total, por consulta e ponto de equilíbrio.
count_flops	função	(fn) -> tuple[int, dict[str, int], Any]	Executa fn() uma vez e conta os FLOPs de forward e backward, inclusive torch.autograd.grad.
cosine_lr	função	(progress, lr0, final_fraction=0.03) -> float	Taxa de aprendizado com decaimento cosseno ao longo do orçamento.
set_lr	função	(optimizer, lr) -> None	Define a taxa de aprendizado em todos os grupos do otimizador.
train_within_budget	função	(step, budget, *, flops_per_step=None, count_first_step=True, evaluate=None, eval_at=(), …)	Chama step(progress) até gastar o orçamento, com avaliações, log e checkpoints opcionais.
equal_budget_search	função	(space, objective, *, n_trials, seed=0, minimize=True) -> SearchResult	Busca aleatória sobre uma grade com exatamente n_trials configurações distintas.
relative_l2	função	(pred, true, *, sample_axis=0, eps=1e-12) -> np.ndarray	Erro L2 relativo por amostra.
time_per_query	função	(fn, *, repeats=20, warmup=3, n_per_call=1, no_grad=True) -> dict[str, float]	Tempo mediano de fn() por instância respondida, por padrão sob no_grad.
hardware_record	função	() -> dict[str, Any]	Registro de CPU, núcleos, memória, GPU, versões e threads para acompanhar uma tabela de orçamento.

pinneapple_analysis.cost.profiler

Mede custo de uma operação, ajusta lei de escala cost ~ n^k, checa orçamentos e detecta regressões contra uma linha de base salva.

pinneapple_analysis/cost/profiler.py · 437 linhas · testes: 11

nome	tipo	assinatura	o que faz
CostRecord	classe	(name, n_runs, time_s, time_min_s, time_iqr_s, times_s, peak_mem_bytes, mem_source, flops…)	Custo medido de uma operação: tempos, dispersão, memória de pico, FLOPs e tamanho.
ScalingFit	classe	(quantity, exponent, ci, r2, coefficient, declared, expected_exponent, verdict)	Ajuste log-log de uma grandeza contra o tamanho, com expoente, intervalo, R^2 e veredito contra a complexidade declarada.
ScalingStudy	classe	(name, sizes, records, fits)	Medidas de uma operação em vários tamanhos e seus ajustes de escala.
Budget	classe	(max_time_s, max_mem_bytes, max_flops, max_time_exponent)	Limites de tempo, memória, FLOPs e expoente de uma operação; limites None não são checados.
BudgetExceeded	classe	()	Exceção levantada quando uma operação passa do orçamento.
Regression	classe	(name, quantity, baseline, current, ratio)	Uma regressão de custo detectada contra a linha de base (grandeza, valores e razão).
CostLedger	classe	()	Registro de operações nomeadas com seus custos; salva uma linha de base e compara runs posteriores.
measure	função	(fn, *args, repeats=5, warmup=1, name=None, flops=True, memory=True, size=None, **kwargs)…)	Mede o custo de fn(*args, **kwargs) com repetições e aquecimento.
expected_exponent	função	(declared, sizes) -> float	Inclinação log-log da complexidade declarada nos tamanhos usados.
scaling_study	função	(make_call, sizes, *, name='operation', repeats=5, warmup=1, declared=None, declared_flop…)	Mede uma operação em vários tamanhos e ajusta cost ~ n^k para tempo e FLOPs.

pinneapple_analysis.cost.torch_ops

Custo por etapa de um passo de treino PINN e tabela de custo por operador PyTorch via profiler. [sem teste]

pinneapple_analysis/cost/torch_ops.py · 84 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
pinn_operation_costs	função	(model, x, *, repeats=5, warmup=1, optimizer=None) -> Dict[str, CostRecord]	Custo de cada etapa de um passo PINN em pontos de colocação.
profile_ops	função	(fn, *, top=15, sort_by='self_cpu_time_total') -> List[Dict[str, Any]]	Tabela por operador PyTorch de uma chamada: chamadas, tempo próprio de CPU e FLOPs.

pinneapple_analysis.data_assimilation

Assimilação de dados: 4D-Var por autograd sobre os métodos de ensemble (EKI) e filtros de Kalman, mais reconstrução GP e posicionamento de sensores.

pinneapple_analysis/data_assimilation/__init__.py · 16 linhas · testes: 6

pinneapple_analysis.data_assimilation.models

Modelos caóticos pequenos e diferenciáveis em PyTorch (Lorenz-96, Lorenz-63, passo RK4) para testar assimilação. [sem teste direto]

pinneapple_analysis/data_assimilation/models.py · 39 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
rk4	função	(f, x, dt) -> torch.Tensor	Um passo de Runge-Kutta de 4ª ordem de f.
lorenz96_tendency	função	(x, F=8.0) -> torch.Tensor	Tendência dx/dt do modelo Lorenz-96 com forçante F.
lorenz96_step	função	(x, dt=0.05, F=8.0) -> torch.Tensor	Um passo RK4 do Lorenz-96.
lorenz63_step	função	(x, dt=0.01, sigma=10.0, rho=28.0, beta=8.0 / 3.0) -> torch.Tensor	Um passo RK4 do Lorenz-63.

pinneapple_analysis.data_assimilation.sensor_placement

Reconstrução de campo escalar por processo gaussiano (kernel Matérn) a partir de sensores pontuais e escolha gulosa dos próximos sensores.

pinneapple_analysis/data_assimilation/sensor_placement.py · 236 linhas · testes: 7

nome	tipo	assinatura	o que faz
GaussianProcessField	classe	(lengthscales=1.0, variance=1.0, nu=2.5, noise=1e-10, mean=0.0) -> None	Regressão por processo gaussiano de um campo escalar a partir de sensores pontuais, com LOO e ajuste de hiperparâmetros.
SensorPlacement	classe	(indices, points, integrated_variance)	Resultado do posicionamento guloso: índices, pontos e variância integrada.
matern_kernel	função	(A, B, lengthscales=1.0, variance=1.0, nu=2.5) -> np.ndarray	Covariância de Matérn anisotrópica entre dois conjuntos de pontos.
greedy_sensor_placement	função	(gp, candidates, n_sensors, domain=None, weights=None, existing=None) -> SensorPlacement	Escolhe sensores entre candidatos minimizando gulosamente a variância posterior integrada.

pinneapple_analysis.data_assimilation.var4d

4D-Var de restrição forte sobre qualquer modelo PyTorch diferenciável, com o adjunto vindo do autograd, checagem de gradiente e experimento gêmeo em Lorenz-96. [sem teste direto]

pinneapple_analysis/data_assimilation/var4d.py · 174 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Observation`	classe	`(step, y, H, R)`	Observações num passo do modelo, com operador H e covariância de erro R.
`Var4DResult`	classe	`(x0, trajectory, cost_initial, cost_final, cost_history, n_iterations, background_term, o…)`	Resultado do 4D-Var: estado inicial analisado, trajetória, histórico do custo e termos de fundo e observação.
`Var4D`	classe	`(model, B_sqrt, dtype=torch.float64)`	4D-Var de restrição forte sobre um modelo `model(x) -> x_next` com covariância de fundo dada pela raiz.
`gradient_check`	função	`(var, x0, xb, obs, n_steps=None, eps=1e-06, seed=0) -> dict[str, float]`	Compara a derivada direcional do custo pelo autograd com uma diferença finita central.
`twin_experiment`	função	`(n=40, window_steps=20, obs_every=2, obs_fraction=0.5, obs_std=1.0, background_std=1.0, d…)`	Experimento gêmeo de 4D-Var no Lorenz-96 e RMSE de fundo, análise e previsão.

pinneapple_analysis.inspection

Pacote de inspeção/NDE: reexporta `InspectionPINN`, `train_generic_inspection_pinn` e as funções `generate_*`, `make_*_physics_loss` e `train_*` de seis modalidades (correntes parasitas, MFL, acústica, EMAT, ondas guiadas, phased array). Todos os sinais são sintéticos e ilustrativos, nunca dados medidos.

`pinneapple_analysis/inspection/__init__.py` · 151 linhas · testes: 13

pinneapple_analysis.inspection.acoustic

Gera, por diferenças finitas no tempo (FDTD 1D), o espectro de condição de uma barra golpeada (impact-echo/ressonância), em que um defeito reduz a velocidade de onda local; inclui perda física da equação da onda e treino da PINN. [sem teste]

`pinneapple_analysis/inspection/acoustic.py` · 227 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`generate_acoustic_synthetic`	função	`(*, L=1.0, c0=5000.0, n_points=60, duration=0.0015, defect_position=0.6, defect_width=0.0…)`	Simula por FDTD 1D a barra sem e com defeito e devolve séries temporais, espectros e os dados de treino da PINN para o deslocamento `u(x, t)`.
`make_acoustic_physics_loss`	função	`(x_grid, c_field) -> PhysicsLossFn`	Cria a perda física `(model, batch) -> (loss, comps)` da equação da onda `u_tt - c(x)^2 u_xx = 0` via autograd, com `c(x)` tirado do mapa FDTD fixo.
`train_acoustic`	função	`(synthetic_data=None, *, hidden=(64, 64, 64), epochs=200, lr=0.001, batch_size=32, device…)`	Gera os dados (se não vierem), monta uma `InspectionPINN` de entrada (x, t) e treina com a perda acústica; devolve modelo, saída do treino e dados.

pinneapple_analysis.inspection.base

Base comum da inspeção: a MLP `InspectionPINN` (subclasse de `PINNBase`) e o wrapper de treino que usa o `Trainer` de `pinneapple_neural` com `CombinedLoss` (supervisionada + hook de física). [sem teste]

`pinneapple_analysis/inspection/base.py` · 251 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`InspectionPINN`	classe	`(in_dim, out_dim, hidden=(64, 64, 64), activation='tanh', *, modality='generic', dtype=to…)`	MLP coordenadas -> sinal de inspeção, no padrão de `VanillaPINN`; a física entra por um resíduo externo (`make_*_physics_loss`).
`train_generic_inspection_pinn`	função	`(model, synthetic_data, physics_loss_fn=None, *, epochs=200, lr=0.001, batch_size=32, val…)`	Converte um dict de arrays em lotes `{x, y, x_col}`, liga a perda física opcional por `PhysicsLossHook` e chama `Trainer.fit`, devolvendo o dict do treino.

pinneapple_analysis.inspection.eddy_current

Sinal sintético de varredura de sonda de correntes parasitas, construído sobre o solver axissimétrico de Helmholtz complexo de `pinneapple_simulation.numerical_solvers.eddy_current_fdm`, mais o treino da PINN com esse mesmo resíduo. A varredura lateral é uma aproximação axissimétrica de um problema 3D. [sem

teste]

`pinneapple_analysis/inspection/eddy_current.py` · 281 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`generate_eddy_current_synthetic`	função	`(scan_positions=None, *, nr=24, nz=40, r_bounds=(0.001, 0.05), z_bounds=(-0.03, 0.03), om…)`	Gera um conjunto sintético de varredura de sonda de correntes parasitas resolvendo o Helmholtz axissimétrico para cada posição de defeito.
`make_eddy_current_physics_loss`	função	`(omega, mu, sigma) -> PhysicsLossFn`	Cria a perda física com as partes real e imaginária do operador de Helmholtz complexo axissimétrico, com J=0 (válido no volume do condutor).
`train_eddy_current`	função	`(synthetic_data=None, *, hidden=(64, 64, 64), epochs=200, lr=0.001, batch_size=32, device…)`	Gera os dados se preciso, monta uma `InspectionPINN` e treina com a perda do resíduo de Helmholtz.

`pinneapple_analysis.inspection.emat`

Gerador simplificado de sinal EMAT: a força de Lorentz vira um termo de força escalar (Gaussiana no espaço, tone-burst no tempo) numa equação da onda elástica 1D; não resolve o acoplamento eletromagnético-elástico completo. [sem teste]

`pinneapple_analysis/inspection/emat.py` · 267 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`generate_emat_synthetic`	função	`(*, L=0.3, rho=7800.0, G0=80000000000.0, n_points=60, duration=6e-05, coil_position=0.05,…)`	Gera um sinal EMAT pitch-catch sintético: um tone-burst tipo Lorentz lança uma onda elástica que um defeito (queda local de rigidez) reflete/atenua.
`make_emat_physics_loss`	função	`(x_grid, c_field, coil_x0, footprint_sigma, burst_freq, n_cycles, t0_burst, J0_over_rho) …)`	Cria a perda física da equação da onda forçada `u_tt - c(x)^2 u_xx - F(x,t) = 0`, com `F` em forma fechada.
`train_emat`	função	`(synthetic_data=None, *, hidden=(64, 64, 64), epochs=200, lr=0.001, batch_size=32, device…)`	Gera os dados EMAT se preciso e treina uma `InspectionPINN` com a perda da onda forçada.

`pinneapple_analysis.inspection.guided_wave_ultrasonic`

Ondas guiadas em 1D com a equação dispersiva tipo Klein-Gordon `u_tt = c0^2 u_xx - beta(x) u`, como substituto qualitativo das curvas de Rayleigh-Lamb; o defeito é um aumento local de `beta`. [sem teste]

`pinneapple_analysis/inspection/guided_wave_ultrasonic.py` · 210 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`generate_guided_wave_synthetic`	função	`(*, L=1.0, c0=3000.0, beta0=0.0, n_points=80, duration=0.0006, burst_freq=50000.0, n_cycl…)`	Gera um conjunto pulso-eco de onda guiada: um tone-burst injetado em x=0 é parcialmente refletido por um aumento local de `beta(x)`.
`make_guided_wave_physics_loss`	função	`(x_grid, beta_field, c0) -> PhysicsLossFn`	Cria a perda física da equação dispersiva `u_tt - c0^2 u_xx + beta(x) u = 0`.
`train_guided_wave_ultrasonic`	função	`(synthetic_data=None, *, hidden=(64, 64, 64), epochs=200, lr=0.001, batch_size=32, device…)`	Gera os dados de onda guiada se preciso e treina uma `InspectionPINN` com a perda dispersiva.

`pinneapple_analysis.inspection.magnetic_flux_leakage`

Verdade de referência por diferenças finitas 2D para MFL (vazamento de fluxo magnético) com permeabilidade não linear (saturação tipo Frohlich-Kennelly, iteração de Picard) e treino PIML com um resíduo simplificado. [sem teste]

`pinneapple_analysis/inspection/magnetic_flux_leakage.py` · 308 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`solve_mfl_nonlinear`	função	`(nx, ny, Lx, Ly, phi0, mu_r_max, H_sat, defect_mask, n_picard=6) -> Tuple[np.ndarray, np.…)`	Resolve o potencial escalar magnetostático não linear do MFL por iteração de Picard, com a região de defeito fixa em `mu0`.
`generate_magnetic_flux_leakage_synthetic`	função	`(*, nx=36, ny=14, Lx=0.2, Ly=0.02, phi0=500.0, mu_r_max=500.0, H_sat=2000.0, defect_cente…)`	Resolve a verdade de referência MFL e lê o campo de vazamento `Bz` na superfície como sinal de varredura ao longo de x.
`make_mfl_physics_loss`	função	`(x_grid, y_grid, mu_field) -> PhysicsLossFn`	Cria uma perda física simplificada `mu * laplaciano(phi)`, que descarta o termo cruzado `grad(mu)·grad(phi)` (exata só onde `mu` varia pouco).
`train_magnetic_flux_leakage`	função	`(synthetic_data=None, *, hidden=(64, 64, 64), epochs=200, lr=0.001, batch_size=32, device…)`	Gera os dados MFL se preciso e treina uma `InspectionPINN` com a perda magnetostática simplificada.

pinneapple_analysis.inspection.phased_array_ultrasonic

B-scan sintético de um array linear em pulso-eco (wavelets de Ricker) reconstruído por delay-and-sum / Total Focusing Method; a perda física de Helmholtz é declarada como regularizador, não lei exata. [sem teste]

`pinneapple_analysis/inspection/phased_array_ultrasonic.py` · 205 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`generate_phased_array_synthetic`	função	`(*, n_elements=24, array_length=0.02, c=5900.0, burst_freq=5000000.0, defect_x=0.0, defec…)`	Gera um B-scan pulso-eco sintético de um array de elementos e sua reconstrução por delay-and-sum (TFM).
`make_phased_array_physics_loss`	função	`(k0) -> PhysicsLossFn`	Cria a perda de consistência de Helmholtz 2D `laplaciano(p) + k0^2 p = 0` aplicada à intensidade da imagem (um regularizador, não lei física exata).
`train_phased_array_ultrasonic`	função	`(synthetic_data=None, *, hidden=(64, 64, 64), epochs=200, lr=0.001, batch_size=32, device…)`	Gera o B-scan se preciso e treina uma `InspectionPINN` com o regularizador de Helmholtz.

pinneapple_analysis.inverse_problems

Reexporta o kit de problemas inversos (modelos de ruído, regularizadores, operadores de observação, EKI, sensibilidade, termos faltantes, `InverseProblemSolver`, e o núcleo de identificabilidade/calibração/studio). O docstring ainda usa o nome antigo `pinneapple_inverse`.

`pinneapple_analysis/inverse_problems/__init__.py` · 261 linhas · testes: 17

pinneapple_analysis.inverse_problems.calibration

Calibração de modelo: separa medições de ajuste e de validação, faz mínimos quadrados ponderados, roda a identificabilidade no ponto calibrado (parâmetro não estimável fica sem intervalo) e dá intervalos por Jacobiano e por bootstrap. [sem teste direto]

`pinneapple_analysis/inverse_problems/calibration.py` · 241 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`CalibrationResult`	classe	`(parameters, fit, metrics, rows, identifiability, bootstrap, warnings, seconds)`	Resultado da calibração: parâmetros, ajuste, métricas, linhas, identificabilidade, bootstrap, avisos e tempo; `theta` dá os valores e `to_dict` serializa.
`residual_metrics`	função	`(pred, obs) -> dict[str, Any]`	Calcula RMSE, MAE, erro máximo, viés, chi2 e chi2 por ponto da predição contra as medições, no total e por observável.
`calibrate`	função	`(model, obs, parameters=None, *, holdout=None, holdout_fraction=0.0, holdout_by=None, los…)`	Calibra os parâmetros contra as observações com validação separada, perda robusta opcional, bootstrap e identificabilidade.

pinneapple_analysis.inverse_problems.contracts

Converte relatórios de identificabilidade, calibrações e comparações de métodos em `StudyResult` de `pinneapple_data.contracts`, e parâmetros em/de `ParameterSpace`, para encadear os apps.

`pinneapple_analysis/inverse_problems/contracts.py` · 246 linhas · testes: 13

nome	tipo	assinatura	o que faz
`parameters_from_space`	função	`(space, nominal=None) -> list[Parameter]`	Cria a lista de `Parameter` a partir de um `ParameterSpace`; parâmetros categóricos não são estimáveis e levantam erro.
`to_parameter_space`	função	`(params, name, description='', groups=None)`	Cria um `ParameterSpace`: parâmetros livres viram contínuos nos seus limites e os fixos viram constantes.
`identifiability_study`	função	`(report, params, *, app='identifiability', title='', model='', study_id=None)`	Converte um relatório de identificabilidade em `StudyResult` (uma linha por parâmetro, posto e condicionamento como métricas).
`calibration_study`	função	`(result, params, *, app='calibration', title='', model='', model_spec=None, study_id=None)`	Converte um resultado de calibração em `StudyResult`, com um `ModelRef` do modelo calibrado quando há métricas de validação.
`inverse_study`	função	`(comparison, params, *, app='inverse', title='', study_id=None)`	Converte a saída de `compare_methods` em `StudyResult` com uma linha por método e parâmetro.

pinneapple_analysis.inverse_problems.counterfactual_attribution

Atribuição exata por valores de Shapley de uma variação medida de um escalar entre poucas causas candidatas, via rodadas contrafactuais de qualquer modelo; inclui um classificador de ambiguidade quando duas causas não se distinguem.

`pinneapple_analysis/inverse_problems/counterfactual_attribution.py` · 152 linhas · testes: 7

nome	tipo	assinatura	o que faz
`CandidateCause`	classe	`(name, apply_baseline, apply_current, signature_measurements)`	Causa candidata: nome, funções que a colocam no estado de base ou atual, e medições que a assinam.
`ShapleyAttributionResult`	classe	`(measured_delta, contributions, residual, baseline_value, current_value)`	Resultado da atribuição: variação medida, contribuição por causa, resíduo de fechamento (deve ser ~0) e valores de base e atual.
`ExactShapleyAttributor`	classe	`(value_fn, baseline_state)`	Avalia uma função de valor em todas as combinações de causas e devolve a decomposição exata de Shapley da variação.
`AmbiguityResult`	classe	`(indeterminate, reason, recommended_measurement)`	Diz se a atribuição é indeterminada, por quê, e qual medição é recomendada para desempatar.
`classify_ambiguity`	função	`(contributions, contribution_uncertainty, signature_measurements, available_measurements)…)`	Decide se as contribuições são indistinguíveis dadas as incertezas e as medições disponíveis.

pinneapple_analysis.inverse_problems.ensemble_kalman

Ensemble Kalman Inversion (EKI) sem derivadas: atualiza um ensemble de parâmetros só com avaliações do modelo direto; inclui a versão regularizada por Tikhonov (TEKI).

`pinneapple_analysis/inverse_problems/ensemble_kalman.py` · 454 linhas · testes: 151

nome	tipo	assinatura	o que faz
`EKIConfig`	classe	`(n_ensemble, n_iterations, noise_cov, noise_std, init_spread, lambda_reg, prior_mean, pri…)`	Configuração da EKI: tamanho do ensemble, iterações, ruído, espalhamento inicial, regularização, prior, tolerância de colapso e semente.
`EKIHistory`	classe	`(iterations, data_misfit, ensemble_spread, theta_mean_history)`	Histórico de convergência da EKI (desajuste, espalhamento, média de theta por iteração).
`EnsembleKalmanInversion`	classe	`(forward_fn, config=None) -> None`	EKI padrão (Iglesias et al. 2013): inicializa o ensemble, aplica passos tipo Kalman e para por colapso do ensemble.

nome	tipo	assinatura	o que faz
`IteratedEKI`	classe	`()`	EKI regularizada por Tikhonov (TEKI, Chada et al. 2020).

`pinneapple_analysis.inverse_problems.estimation`

Estimadores comuns a identificabilidade, calibração e studio: mínimos quadrados não lineares em parâmetros escalados, L-BFGS-B com gradiente exato (adjunto ou autograd) e Metropolis adaptativo. [sem teste direto]

`pinneapple_analysis/inverse_problems/estimation.py` · 284 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Prior`	classe	`(mean, std, L, weight)`	Prior Gaussiano nos parâmetros escalados mais penalidade opcional de suavidade `weight * |L z|^2`.
`FitResult`	classe	`(z, theta, pred, chi2, prior_penalty, jac, nfev, success, message, seconds, derivatives, …)`	Resultado de um ajuste: z, theta, predições, chi2, penalidade do prior, Jacobiano, avaliações e diagnóstico; `dof` dá os graus de liberdade.
`PosteriorSamples`	classe	`(names, samples_z, samples, acceptance, n_evaluations, seconds)`	Amostras da posterior (em z e unidades físicas) com taxa de aceitação; `summary` dá média, mediana, desvio, intervalos e correlações.
`least_squares_fit`	função	`(model, params, obs, *, z0=None, free=None, prior=None, loss='linear', max_nfev=200, deri…)`	Minimiza chi2 (mais prior) nos parâmetros escalados por região de confiança, com subconjunto livre e perdas robustas opcionais.
`gradient_fit`	função	`(model, params, obs, *, z0=None, prior=None, gradient='adjoint', max_iter=300) -> FitResu…)`	L-BFGS-B no desajuste com gradiente exato pelo adjunto do modelo ou por autograd.
`metropolis`	função	`(model, params, obs, *, z0, cov0=None, n_steps=4000, burn=None, prior=None, seed=0, thin=…)`	Metropolis de passeio aleatório adaptativo (Haario et al. 2001) em z, com verossimilhança Gaussiana e limite de tempo.

`pinneapple_analysis.inverse_problems.forward`

Linguagem comum dos problemas inversos: `Parameter`, `ParameterSet`, `Observations`, a base `ForwardModel` (cujo `predict` levanta NotImplementedError até ser sobrescrito) e modelos por função, expressão de texto ou tabela; mais o Jacobiano em coordenadas escaladas. [sem teste direto]

`pinneapple_analysis/inverse_problems/forward.py` · 484 linhas · **sem teste direto** · palavras-chave no código (heurística; o texto acima diz o status real): levanta `NotImplementedError` em algum caminho

nome	tipo	assinatura	o que faz
`Parameter`	classe	`(name, value, lower, upper, unit, log, fixed, description, truth)`	Um parâmetro ajustável com valor nominal, limites, unidade, escala log opcional, flag de fixo e valor verdadeiro opcional.
`ParameterSet`	classe	`(parameters)`	Conjunto de parâmetros livres e o mapa entre theta (dict em unidades físicas) e z (vetor escalado).
`Observations`	classe	`(inputs, sigma, values, observable, meta)`	Medições: entradas por coluna, observável de cada linha, valores (opcionais antes do experimento) e sigma do ruído; permite subconjunto, divisão e concatenação.
`ForwardModel`	classe	`(name, inputs, outputs, cost, description)`	Classe base de modelo direto: define entradas, saídas e parâmetros padrão; `predict` levanta NotImplementedError se não for sobrescrito.
`CallableModel`	classe	`(fn, *, inputs, outputs=('y',), parameters=(), name='callable', cost='cheap', description…)`	Embrulha qualquer função Python ou solver externo `fn(theta, inputs)` como modelo direto.
`ExpressionModel`	classe	`(expressions, *, inputs, parameters, name='expression', description='')`	Modelo escrito como texto, uma expressão por observável.
`TabulatedModel`	classe	`(table, *, parameters, inputs, output, name='simulated results', smoothing=0.0, observabl…)`	Usa uma tabela de resultados simulados como modelo, interpolando sobre parâmetros e entradas.
`count_evaluations`	função	`(model) -> int`	Devolve quantas vezes o modelo foi avaliado (contador interno `_n_eval`).

nome	tipo	assinatura	o que faz
jacobian	função	(model, params, z, obs, *, method='auto', rel_step=0.0001) -> tuple[np.ndarray, np.ndarra…)	Calcula predições e Jacobiano em relação a z, pelo método do modelo, autograd ou diferenças finitas.

pinneapple_analysis.inverse_problems.heat_diffusivity

Recupera a difusividade alpha da equação do calor 1D a partir de leituras ruidosas num instante, com uma PINN que aprende campo e alpha juntos, e relata a informação de Fisher/limite de Cramér-Rao.

pinneapple_analysis/inverse_problems/heat_diffusivity.py · 162 linhas · testes: 4

nome	tipo	assinatura	o que faz
u_exact	função	(t, x, alpha) -> np.ndarray	Solução exata exp(-pi^2 alpha t) sin(pi x) da equação do calor do caso de teste.
identifiability_report	função	(t_obs, x_obs, alpha, noise_std) -> dict[str, Any]	Informação de Fisher e limite de Cramér-Rao para alpha a partir das leituras com ruído Gaussiano.
run_inverse_discovery	função	(true_alpha=TRUE_ALPHA, initial_guess=INITIAL_GUESS, noise_std=NOISE_STD, n_observations=…)	Gera as leituras, treina rede e alpha juntos e devolve um resultado serializável com histórico, alpha recuperado e identificabilidade.

pinneapple_analysis.inverse_problems.heat_transfer

Transferência de calor inversa: estima o coeficiente de convecção h de uma aleta (1D, também com leituras do usuário) e de uma placa 2D por PINN; o caso 3D não é treinado aqui, é servido a partir de resultados gravados em examples/use_cases/fin_convection_inverse/results/3d.

pinneapple_analysis/inverse_problems/heat_transfer.py · 325 linhas · testes: 15

nome	tipo	assinatura	o que faz
InputError	classe	()	Erro de entrada inválida (subclasse de ValueError) dos casos de transferência de calor.
fin_exact	função	(xi, h, k, d, length) -> np.ndarray	Temperatura adimensional de uma aleta pino com ponta convectiva (Incropera, Tabela 3.4, caso A).
fin_q_exact	função	(h, k, d, length, dt) -> float	Calor que sai da base da aleta pela solução analítica.
run_fin	função	(*, k, d_mm, length_mm, t_base, t_air, sensors_mm, readings, h_guess=100.0, steps=2000, s…)	Aprende o perfil de temperatura e h de uma aleta a partir de leituras de termopares.
fin_synthetic_readings	função	(*, k, d_mm, length_mm, t_base, t_air, sensors_mm, h_true, noise, seed=0) -> list[float]	Gera leituras sintéticas de termopares da aleta analítica mais ruído Gaussiano.
run_plate	função	(*, power, h_true, noise, h_guess=60.0, steps=1500, seed=0, progress=None, budget=None, w…)	Gera leituras sintéticas da placa pelo modelo de volumes finitos e treina a PINN para T(x, y) e h, comparando com o modelo.
plate_precomputed	função	() -> dict	Carrega o resultado gravado da placa 2D (campo da PINN e de referência) da pasta de resultados do exemplo.
block_precomputed	função	() -> dict	Carrega o resultado gravado do bloco 3D (campo da PINN, referência e sorteios de h) da pasta de resultados do exemplo.

pinneapple_analysis.inverse_problems.identifiability

Responde se os parâmetros podem ser recuperados das medições antes de estimar: informação de Fisher, sensibilidade local, verossimilhança de perfil e posterior, mais projeto D-ótimo de novas medições. (A flag 'experimental' vem de 'experimental design', não de status.) [sem teste direto]

pinneapple_analysis/inverse_problems/identifiability.py · 457 linhas · **sem teste direto** · palavras-chave no código (heurística; o texto acima diz o status real): experimental

nome	tipo	assinatura	o que faz
`IdentifiabilityReport`	classe	`(parameters, analysis_point, data, n_observations, rank, n_parameters, condition_number, …)`	Relatório de identificabilidade: posto, condicionamento, autovalores, correlações, ambiguidades e status por parâmetro.
`d_optimal_design`	função	`(F, Jc_w, n_pick=3, distinct=True) -> list[dict[str, Any]]`	Seleção gulosa D-ótima de medições candidatas a partir da informação de Fisher, com o ganho de log det de cada escolha.
`profile_likelihood`	função	`(model, params, obs, z_best, chi2_min, std_z, *, n_points=15, max_nfev=60, time_budget=60…)`	Perfil de verossimilhança de cada parâmetro livre, com intervalo de 95% e veredito.
`analyze_identifiability`	função	`(model, obs, parameters=None, *, candidates=None, n_suggest=3, profile='auto', n_profile=…)`	Roda os métodos de identificabilidade sobre modelo e medições e sugere novas medições.

`pinneapple_analysis.inverse_problems.missing_term`

Identificação de termos faltantes numa EDP parcialmente conhecida: biblioteca de candidatos, regressão esparsa SINDy (STRidge/LASSO), análise do resíduo de uma PINN treinada e um termo neural treinado em conjunto.

`pinneapple_analysis/inverse_problems/missing_term.py` · 500 linhas · testes: 151

nome	tipo	assinatura	o que faz
`CandidateLibrary`	classe	`(poly_order=2, include_trig=False, custom_terms=None) -> None`	Monta a matriz Θ de termos candidatos (monômios, trigonométricos, personalizados) a partir de dados de colocação.
`SINDyResult`	classe	`(coefficients, active_terms, active_coefficients, residual_norm, n_terms_selected)`	Resultado da regressão SINDy: coeficientes, termos ativos e norma do resíduo; `equation` escreve a forma simbólica.
`SINDyIdentifier`	classe	`(threshold=0.001, max_iter=20, alpha=1e-05, method='stridge') -> None`	Identifica termos faltantes por regressão esparsa (Brunton et al. 2016).
`ResidualAnalysisResult`	classe	`(residual_field, sindy_result, coords, mean_abs_residual, max_abs_residual)`	Saída de `ResidualAnalyzer.analyze`: campo de resíduo, resultado SINDy opcional e estatísticas.
`ResidualAnalyzer`	classe	`(pde_residual_fn, library=None, identifier=None) -> None`	Avalia o resíduo da EDP conhecida numa PINN treinada e opcionalmente roda SINDy sobre ele.
`NeuralTermConfig`	classe	`(hidden_dims, activation, term_weight, data_weight, n_iters, lr, device)`	Configuração do treino conjunto PINN + termo neural.
`NeuralTermDiscovery`	classe	`(known_residual_fn, config=None) -> None`	Descobre um termo faltante parametrizado por uma rede neural e permite destilá-lo.

`pinneapple_analysis.inverse_problems.models`

Modelos diretos físicos para identificabilidade, calibração e inversão: aleta analítica, aleta em volumes finitos com zonas de `h` (com adjunto), placa dissipadora, resfriamento concentrado e cinética de Arrhenius. [sem teste direto]

`pinneapple_analysis/inverse_problems/models.py` · 476 linhas · **sem teste direto** · palavras-chave no código (heurística; o texto acima diz o status real): experimental

nome	tipo	assinatura	o que faz
`PinFinModel`	classe	`(d_mm=5.0, length_mm=50.0)`	Aleta pino analítica com ponta convectiva (caso do Inverse Heat Lab), com autograd e caminho PINN para `h`.
`RodModel`	classe	`(d_mm=5.0, length_mm=100.0, n_zones=1, n_cells=200)`	Aleta em volumes finitos com `h` por zonas (um campo), com sensibilidades exatas, gradiente adjunto e gêmeo em torch.
`PlateModel`	classe	`(nx=60, ny=36)`	Placa dissipadora do Inverse Heat Lab resolvida por volumes finitos, com cache por `h`.
`LumpedCoolingModel`	classe	`(area_m2=0.01)`	Curva de resfriamento concentrado; `h` e C só aparecem como `h A / C`, logo os dois livres são estruturalmente ambíguos.
`ArrheniusModel`	classe	`()`	Conversão de primeira ordem de Arrhenius; A e Ea são ambíguos com uma só temperatura.

pinneapple_analysis.inverse_problems.noise_models

Modelos de ruído / termos de desajuste de dados em PyTorch: Gaussiano, Huber, Cauchy, Student-t e heteroscedástico.

`pinneapple_analysis/inverse_problems/noise_models.py` · 260 linhas · testes: 151

nome	tipo	assinatura	o que faz
`DataMisfitBase`	classe	`()`	Base abstrata de termo de desajuste / log-verossimilhança negativa.
`GaussianMisfit`	classe	`(noise_cov=None, noise_std=1.0, reduction='mean') -> None`	Desajuste Gaussiano (L2/Mahalanobis).
`HuberMisfit`	classe	`(delta=1.0, noise_std=1.0, smooth=True) -> None`	Desajuste de Huber (ou pseudo-Huber), robusto a outliers moderados.
`CauchyMisfit`	classe	`(gamma=1.0, noise_std=1.0) -> None`	Desajuste de Cauchy/Lorentz, de cauda pesada e muito robusto.
`StudentTMisfit`	classe	`(nu=5.0, noise_std=1.0) -> None`	Log-verossimilhança negativa de Student-t; `nu` controla a robustez.
`HeteroscedasticMisfit`	classe	`(log_noise_var) -> None`	Desajuste Gaussiano com variância de ruído por observação.

pinneapple_analysis.inverse_problems.obs_operator

Operadores de observação H (estado -> observáveis): pontual, linear, integral e composto. O docstring cita um `BoundaryObsOperator` que não existe no arquivo.

`pinneapple_analysis/inverse_problems/obs_operator.py` · 284 linhas · testes: 151

nome	tipo	assinatura	o que faz
`ObsOperatorBase`	classe	`()`	Base abstrata de operador de observação; oferece `jacobian_theta`.
`PointObsOperator`	classe	`(sensor_locations, output_indices=None) -> None`	Avalia o modelo num conjunto fixo de posições de sensores.
`LinearObsOperator`	classe	`(A, x_eval) -> None`	Operador linear `H(u) = A u` (ex.: tomografia).
`IntegralObsOperator`	classe	`(x_quad, weights, kernel_fn=None) -> None`	Funcional integral aproximado por quadratura ponderada com núcleo opcional.
`ComposedObsOperator`	classe	`(operators, x_obs_list=None) -> None`	Empilha vários operadores de observação num vetor único.

pinneapple_analysis.inverse_problems.presets

Problemas inversos prontos com resposta conhecida (medições sintéticas com semente fixa) e construção de modelos a partir de JSON, para apps e agentes. [sem teste direto]

`pinneapple_analysis/inverse_problems/presets.py` · 326 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Preset`	classe	`(id, title, summary, model_spec, parameters, observations, validation, candidates, method…)`	Problema pronto: modelo em JSON, parâmetros, observações, validação, métodos e lição; `build` monta os objetos.
`model_from_spec`	função	`(spec) -> ForwardModel`	Constrói um `ForwardModel` a partir de um dict JSON (aleta, expressão, tabela etc.).
`synthetic`	função	`(model, params, obs, seed, rel_sigma=False) -> Observations`	Gera observações sintéticas: modelo na verdade mais ruído N(0, sigma) com semente, arredondado à resolução do instrumento.
`get_preset`	função	`(name) -> Preset`	Devolve um preset pelo nome.
`list_presets`	função	`(tag=None) -> list[dict[str, Any]]`	Lista os presets, opcionalmente filtrados por tag.

pinneapple_analysis.inverse_problems.regularization

Regularizadores diferenciáveis em PyTorch (Tikhonov, L1, variação total, composto) e escolha automática de lambda pela curva L.

pinneapple_analysis/inverse_problems/regularization.py · 329 linhas · testes: 151

nome	tipo	assinatura	o que faz
RegularizerBase	classe	()	Base abstrata de regularizador R(θ) com peso.
TikhonovRegularizer	classe	(lambda_reg, prior=None, covariance=None) -> None	Regularizador de Tikhonov/ridge, com prior e covariância opcionais.
SparsityRegularizer	classe	(lambda_reg, eps=1e-06) -> None	Regularizador L1/LASSO com aproximação suave.
TotalVariationRegularizer	classe	(lambda_reg, eps=1e-06) -> None	Variação total anisotrópica para parâmetros de campo 1D/2D.
CompositeRegularizer	classe	(terms=None) -> None	Soma ponderada de vários regularizadores.
LCurveResult	classe	(lambdas, data_misfits, regularization_norms, optimal_lambda, curvatures)	Resultado da curva L: lambdas, desajustes, normas, curvaturas e lambda ótimo.
LCurveSelector	classe	(train_fn, lambdas=None) -> None	Escolhe lambda pela máxima curvatura da curva L, chamando uma função de treino para cada valor.

pinneapple_analysis.inverse_problems.sensitivity

Análise de sensibilidade: Jacobiano local e matriz de Fisher, análise de identificabilidade por autovalores com critérios D/A/E e índices de Sobol globais pelo método de Saltelli.

pinneapple_analysis/inverse_problems/sensitivity.py · 439 linhas · testes: 151 · palavras-chave no código (heurística; o texto acima diz o status real): experimental

nome	tipo	assinatura	o que faz
LocalSensitivityResult	classe	(jacobian, fisher_information, singular_values, sensitivity_indices, param_names)	Resultado da sensibilidade local: Jacobiano, Fisher, valores singulares e índices.
LocalSensitivity	classe	(forward_fn, noise_std=1.0, param_names=None) -> None	Sensibilidade linearizada em torno de um ponto nominal, com Jacobiano por diferenças finitas opcional.
IdentifiabilityResult	classe	(eigenvalues, eigenvectors, condition_number, d_optimality, pseudo_d_optimality, a_optima…)	Resultado da análise de identificabilidade por Fisher: autovalores, condicionamento, critérios D/A/E e índices identificáveis.
IdentifiabilityAnalyzer	classe	(tol=1e-06) -> None	Analisa a identificabilidade a partir dos autovalores da matriz de Fisher.
SobolResult	classe	(S1, ST, param_names, n_samples)	Índices de Sobol de primeira ordem e totais.
GlobalSensitivity	classe	(forward_fn, param_bounds, n_samples=1024, seed=None) -> None	Sensibilidade global por variância (índices de Sobol pelo método de Saltelli).

pinneapple_analysis.inverse_problems.solver

InverseProblemSolver: liga modelo (PINN ou nn.Module), operador de observação, desajuste e regularização, otimizando por Adam, L-BFGS ou EKI.

pinneapple_analysis/inverse_problems/solver.py · 426 linhas · testes: 151

nome	tipo	assinatura	o que faz
InverseSolverConfig	classe	(method, n_iters, lr, lbfgs_max_iter, device, grad_clip, print_every, eki_n_ensemble, eki…)	Configuração do InverseProblemSolver: método, iterações, taxa de aprendizado e opções de EKI.
InverseSolverResult	classe	(theta_opt, final_misfit, final_regularization, final_total_loss, loss_history, n_iters, …)	Resultado: theta ótimo, desajuste, regularização, perda total, histórico, método e convergência.

nome	tipo	assinatura	o que faz
`InverseProblemSolver`	classe	`(model, obs_operator, data_misfit=None, regularizer=None, config=None, extra_loss_fn=None…)`	Resolvedor unificado de problemas inversos com informação física.

`pinneapple_analysis.inverse_problems.studio`

Interface única para formular um problema inverso, resolvê-lo com vários métodos (`least_squares`, `adjoint`, `autodiff`, `bayesian`, `pinn`) e ranqueá-los pelo erro nas medições de validação. [sem teste direto]

`pinneapple_analysis/inverse_problems/studio.py` · 317 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`InverseProblem`	classe	`(model, observations, parameters, validation, name, description, prior_std, smoothness, f…)`	Problema inverso: modelo, observações, parâmetros, validação, prior e zonas de campo.
`available_methods`	função	`(problem) -> dict[str, dict[str, Any]]`	Diz quais métodos se aplicam ao problema e por que não, quando não se aplicam.
`solve_inverse`	função	`(problem, method, *, identifiability=None, pinn_steps=1500, mcmc_steps=3000, seed=0, prog…)`	Resolve o problema com um método e devolve um dict serializável com estimativas, incerteza, resíduos e métricas.
`compare_methods`	função	`(problem, methods=…, *, identifiability=True, pinn_steps=1500, seed=0, progress=None) -> …)`	Roda vários métodos sobre os mesmos dados e os ranqueia pelo RMSE de validação.

`pinneapple_analysis.proof`

Entrada dos grafos de prova; reexporta `ProofGraph` e as fábricas de checagem.

`pinneapple_analysis/proof/__init__.py` · 29 linhas · testes: 11

`pinneapple_analysis.proof.graph`

Modelo de grafo de prova com nós (hipótese, definição, lema, teorema, conjectura), checagens exatas ou numéricas e status ASSUMED/PROVED/ARGUED/SUPPORTED/REFUTED/OPEN. [sem teste direto]

`pinneapple_analysis/proof/graph.py` · 334 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`CheckOutcome`	classe	`(name, passed, exact, detail, counterexample, seconds)`	Resultado de uma checagem: passou, exata ou não, detalhe, contraexemplo e tempo.
`Check`	classe	`(name, fn)`	Uma checagem nomeada com a função que a executa.
`Node`	classe	`(id, kind, statement, depends_on, checks, argument, covered, supported_by, status, outcom…)`	Um nó do grafo (hipótese, definição, lema, teorema, conjectura) com dependências, checagens e status.
`ProofGraph`	classe	`(title='')`	O grafo de prova: adiciona nós, executa em ordem de dependência, escreve relatório e desenha em Mermaid ou matplotlib.
`symbolic_identity`	função	`(name, lhs, rhs=0, simplify=None) -> Check`	Checagem exata: `simplify(lhs - rhs) == 0` no sympy.
`symbolic_true`	função	`(name, statement) -> Check`	Checagem exata: uma relação sympy que simplifica para True.
`exhaustive`	função	`(name, predicate, domain, describe='') -> Check`	Checagem exata: um predicado sobre todos os elementos de um conjunto finito.
`numeric`	função	`(name, predicate, sampler, n=2000, seed=0) -> Check`	Checagem numérica: um predicado em amostras aleatórias (evidência; falha é contraexemplo).
`counterexample_search`	função	`(name, violation, bounds, tol=1e-08, seed=0, maxiter=300, restarts=3) -> Check`	Busca de contraexemplo por evolução diferencial; valor abaixo de `-tol` refuta.
`custom`	função	`(name, fn) -> Check`	Checagem livre que devolve (passou, exata, detalhe).

pinneapple_analysis.proof.pt_chains

Grafo de prova que testa se o teorema de impossibilidade de sintonia linear de ponto excepcional vale para tetrâmeros PT; o teorema fica ARGUED e cadeias N >= 5 seguem como conjectura. [experimental]

`pinneapple_analysis/proof/pt_chains.py` · 262 linhas · testes: 4

nome	tipo	assinatura	o que faz
`chain_hamiltonian`	função	`(gamma, bonds) -> np.ndarray`	Hamiltoniano de uma cadeia PT de N sítios com ganho e perda nas pontas.
`threshold_numeric`	função	`(bonds, gmax=None, tol=1e-11) -> float`	Menor gamma em que o espectro deixa de ser real, por varredura e bissecção na matriz.
`tetramer_threshold_branches`	função	`(A, B) -> list[float]`	Valores candidatos de gamma² dos ramos I e II do limiar do tetrâmero.
`build_pt_chain_graph`	função	`(max_deg=4, n_numeric=300, search_iters=120, searches=True) -> ProofGraph`	Monta o grafo de prova do tetrâmero PT; `searches=False` omite as buscas de contraexemplo lentas.

pinneapple_analysis.reproduction

Avaliador único de métricas de erro de artigos, estatística por semente sem escolha da melhor e regra de status (reproduzido, parcial, não reproduzido, escala reduzida).

`pinneapple_analysis/reproduction.py` · 139 linhas · testes: 5

nome	tipo	assinatura	o que faz
`SeedStats`	classe	`(values, median, min, max, n_failed)`	Valor de cada semente com mediana e mín-máx; sem escolha da melhor semente.
`rel_l2`	função	`(pred, ref) -> float`	Erro L2 relativo; `inf` quando a predição não é finita.
`rmae`	função	`(pred, ref) -> float`	Erro absoluto relativo `sum |pred - ref| / sum |ref|`.
`max_abs`	função	`(pred, ref) -> float`	Maior erro absoluto.
`mean_abs`	função	`(pred, ref) -> float`	Erro absoluto médio.
`mse`	função	`(pred, ref) -> float`	Erro quadrático médio.
`rmse`	função	`(pred, ref) -> float`	Raiz do erro quadrático médio.
`evaluate`	função	`(pred, ref, metrics=('rel_l2', 'rmae', 'max_abs')) -> dict[str, float]`	Calcula as métricas nomeadas de `pred` contra `ref`.
`seed_stats`	função	`(values) -> SeedStats`	Mediana e mín-máx entre sementes; semente divergente conta como `inf`.
`status`	função	`(ours_median, paper, *, spread=None, ordering_holds=None, reduced_scale=False, factor=2.0…)`	Status de um alvo pela regra do protocolo: reproduced, partially reproduced, not reproduced ou reduced-scale.

pinneapple_analysis.retrieval

Busca semântica local; oferece `build_default_index` e exige o extra `retrieval` (sentence-transformers).

`pinneapple_analysis/retrieval/__init__.py` · 58 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`build_default_index`	função	`(*, model_name=DEFAULT_MODEL_NAME, include_literature=False, literature_topics=None, lite…)`	Monta numa chamada um índice sobre o corpus padrão, opcionalmente com artigos do arXiv.

pinneapple_analysis.retrieval.corpus

Fontes reais do corpus: docstrings de módulos, resumos de presets, arquivos de código fatiados e artigos do arXiv buscados ao vivo (opcional).

pinneapple_analysis/retrieval/corpus.py · 360 linhas · testes: 32

nome	tipo	assinatura	o que faz
Document	classe	(doc_id, source, title, text)	Um documento do corpus: id, fonte, título e texto.
fetch_literature_documents	função	(query, *, k=5, timeout_s=30) -> List[Document]	Busca artigos reais no arXiv para uma consulta; falha de rede levanta erro.
build_literature_corpus	função	(topics=None, *, k_per_topic=5, timeout_s=30) -> List[Document]	Busca artigos para vários tópicos, sem duplicatas; pula com aviso um tópico que falhar.
build_corpus	função	(*, include_literature=False, literature_topics=None, literature_k_per_topic=5) -> List[D…)	Monta o corpus atual (docstrings, presets e, se pedido, literatura), recalculado a cada chamada.
iter_repo_files	função	(repo_root, *, extensions=DEFAULT_CODE_EXTENSIONS, exclude_dirs=DEFAULT_EXCLUDE_DIRS, max…)	Percorre um repositório e lista arquivos pelas extensões, pulando pastas excluídas e arquivos grandes.
chunk_text	função	(text, *, max_chars=4000, overlap=200) -> List[str]	Fatia texto em janelas de caracteres com sobreposição, preferindo cortar em linha em branco.
file_fingerprint	função	(path) -> str	Hash sha256 do conteúdo atual de um arquivo, para detectar mudanças.
code_repo_documents	função	(repo_root, *, repo_name=None, extensions=DEFAULT_CODE_EXTENSIONS, exclude_dirs=DEFAULT_E…)	Gera documentos de código-fonte fatiados de um clone local de repositório.

pinneapple_analysis.retrieval.index

Índice semântico local com embeddings de sentence-transformers e similaridade de cosseno; salva e carrega o índice.

pinneapple_analysis/retrieval/index.py · 133 linhas · testes: 17

nome	tipo	assinatura	o que faz
SearchResult	classe	(document, score)	Um resultado de busca: documento e score de similaridade.
RetrievalIndex	classe	(documents, embeddings, model_name) -> None	Índice consultável com o embedding de todos os documentos; build, search, save e load.

pinneapple_analysis.sensitivity

Análise de sensibilidade global e local; reexporta Sobol, Morris, curvas de efeito, o estudo completo e as funções de teste.

pinneapple_analysis/sensitivity/__init__.py · 42 linhas · testes: 24

pinneapple_analysis.sensitivity.effects

Curvas de dependência parcial e ICE, jacobiano local, recomendação de congelar parâmetros e verificação direta do congelamento. [sem teste direto]

pinneapple_analysis/sensitivity/effects.py · 157 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
effect_curves	função	(fn, space, n_grid=21, n_background=128, n_ice=20, seed=0, outputs=None) -> dict	Curvas de dependência parcial e ICE de cada entrada.
local_derivatives	função	(fn, x0, space=None, torch_fn=None, rel_step=0.0001, outputs=None) -> dict	Jacobiano dy/dx num ponto, por autograd quando há função torch, senão por diferenças finitas.

nome	tipo	assinatura	o que faz
`freeze_recommendation`	função	`(indices, threshold=0.01, morris_result=None) -> list[dict]`	Por entrada: freeze, inconclusive ou keep, pelo intervalo do índice total contra um limiar.
`verify_freeze`	função	`(fn, space, frozen, n=1024, seed=0, at=None) -> dict`	Compara o modelo com todas as entradas variando e com as congeladas fixas; devolve razão de variância e mudanças pontuais.

pinneapple_analysis.sensitivity.morris

Triagem de Morris por efeitos elementares, com custo `r (d + 1)` avaliações e intervalos bootstrap de mu*.

`pinneapple_analysis/sensitivity/morris.py` · 111 linhas · testes: 13

nome	tipo	assinatura	o que faz
`MorrisResult`	classe	`(names, outputs, r, levels, delta, n_evals, mu, mu_star, sigma, mu_star_ci, level, effect…)`	Resultado de Morris: mu, mu*, sigma, intervalos de mu* e ranking por saída.
`morris_design`	função	`(d, r=20, levels=4, seed=0) -> tuple[np.ndarray, np.ndarray, np.ndarray]`	Trajetórias de Morris no hipercubo unitário, a entrada movida em cada passo e o sinal.
`morris`	função	`(fn, space, r=20, levels=4, seed=0, n_boot=500, level=0.95, outputs=None) -> MorrisResult`	Triagem de Morris de `fn` sobre o espaço com `r` trajetórias.

pinneapple_analysis.sensitivity.sobol

Índices de Sobol de primeira ordem, totais e de segunda ordem com intervalos bootstrap, a partir de um modelo, de avaliações externas ou de uma amostra existente. [sem teste direto]

`pinneapple_analysis/sensitivity/sobol.py` · 255 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`SobolIndices`	classe	`(names, outputs, n_base, n_evals, variance, S1, ST, S1_ci, ST_ci, S2, S2_ci, level, n_boo…)`	Índices de Sobol S1, ST (e S2) por saída e entrada, com intervalos bootstrap.
`saltelli_design`	função	`(space, n, second_order=False, seed=0) -> tuple[np.ndarray, dict]`	Entradas do desenho de Saltelli empilhadas e o layout para lê-las de volta.
`sobol_from_evaluations`	função	`(Y, layout, outputs=None, n_boot=200, level=0.95, seed=0, X=None) -> SobolIndices`	Índices a partir de saídas avaliadas em outro lugar sobre o desenho de Saltelli.
`sobol_indices`	função	`(fn, space, n=1024, second_order=False, n_boot=200, level=0.95, seed=0, outputs=None) -> …)`	Índices de Sobol de primeira ordem e totais (e de segunda ordem, opcional) de `fn`.
`first_order_given_data`	função	`(X, y, names=None, n_bins=None, n_boot=200, level=0.95, seed=0) -> dict`	Índices de primeira ordem a partir de uma amostra existente, por classes de mesma contagem com correção de viés.
`sobol_from_runs`	função	`(space, n, run_ids, Y, second_order=False, seed=0, n_boot=200, level=0.95, outputs=None) …)`	Índices a partir de runs do desenho feitos fora (OpenFOAM, CalculiX); run faltante levanta ValueError.

pinneapple_analysis.sensitivity.study

Estudo de sensibilidade completo (Morris, Sobol, curvas, derivadas, regra de congelamento e sua checagem) e a previsão do número de avaliações. [sem teste direto]

`pinneapple_analysis/sensitivity/study.py` · 136 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`study_cost`	função	`(d, n_sobol=1024, second_order=False, r_morris=20, n_grid=15, n_background=64, n_verify=5…)`	Número de avaliações do modelo em cada parte do estudo de sensibilidade.
`sensitivity_study`	função	`(model, space, outputs=None, *, n_sobol=1024, second_order=False, r_morris=20, n_grid=15,…)`	Roda o estudo de sensibilidade completo sobre um modelo e um espaço de parâmetros.

pinneapple_analysis.sensitivity.testfunctions

Funções de teste com índices de Sobol em forma fechada (Ishigami, função G de Sobol) para conferir estimadores.

pinneapple_analysis/sensitivity/testfunctions.py · 53 linhas · testes: 28

nome	tipo	assinatura	o que faz
ishigami	função	(X, a=7.0, b=0.1) -> np.ndarray	Função de Ishigami.
ishigami_space	função	() -> list[dict]	Espaço de parâmetros da função de Ishigami.
ishigami_indices	função	(a=7.0, b=0.1) -> dict	Variância e índices de Sobol exatos da função de Ishigami.
sobol_g	função	(X, a) -> np.ndarray	Função G de Sobol com coeficientes a.
sobol_g_space	função	(d) -> list[dict]	Espaço de parâmetros da função G com d entradas.
sobol_g_indices	função	(a) -> dict	Variância e índices exatos da função G.

pinneapple_analysis.state_estimation

Blocos de estimação sequencial de estado; código movido de pinneapple_systems.digital_twin, que agora mantém um shim de reexportação.

pinneapple_analysis/state_estimation/__init__.py · 49 linhas · testes: 22 · palavras-chave no código (heurística; o texto acima diz o status real): shim ou obsoleto

pinneapple_analysis.state_estimation.kalman

Filtro de Kalman estendido e filtro de Kalman de ensemble estocástico, mais a análise EnKF com localização de Gaspari-Cohn.

pinneapple_analysis/state_estimation/kalman.py · 346 linhas · testes: 6 · palavras-chave no código (heurística; o texto acima diz o status real): shim ou obsoleto

nome	tipo	assinatura	o que faz
ExtendedKalmanFilter	classe	(n_state, n_obs, f, h, Q=None, R=None, F_jac=None, H_jac=None, *, eps_fd=1e-05) -> None	Filtro de Kalman estendido com jacobianos dados ou por diferenças finitas.
EnsembleKalmanFilter	classe	(n_state, n_obs, f, h, Q=None, R=None, n_ens=100, inflation=1.02, seed=0) -> None	Filtro de Kalman de ensemble estocástico com inflação.
gaspari_cohn	função	(d) -> np.ndarray	Correlação de suporte compacto de Gaspari-Cohn para localização.
enkf_analysis	função	(ens, y, hx, r_var, *, loc_xy=None, loc_yy=None, inflation=1.0, rng=None) -> np.ndarray	Passo de análise EnKF com observações perturbadas, localização opcional e inflação.

pinneapple_analysis.study_contracts

Ponte entre estudos de incerteza/sensibilidade e os contratos compartilhados (ParameterSpace, ModelRef, StudyResult).

pinneapple_analysis/study_contracts.py · 237 linhas · testes: 4

nome	tipo	assinatura	o que faz
is_parameter_space	função	(obj) -> bool	Diz se um objeto é um contrato ParameterSpace.
space_from_contract	função	(space) -> tuple[list[dict], dict[str, Any]]	Lê (parâmetros, fixos) de um ParameterSpace.
space_to_contract	função	(params, name='study', fixed=None, units=None)	Grava os parâmetros de um estudo como ParameterSpace, com as distribuições nos metadados.
model_from_ref	função	(ref, onnx=None, outputs=None) -> StudyModel	Cria um StudyModel de uma referência zoo:<nome> ou ONNX.
model_to_ref	função	(model)	Cria um ModelRef de um modelo de estudo, com faixa de validade e métricas do cartão.
uncertainty_result	função	(out, model, study_id, app='uncertainty', title='')	Converte o resultado de uncertainty_study num StudyResult de incerteza.

nome	tipo	assinatura	o que faz
sensitivity_result	função	(out, model, study_id, app='sensitivity', title='')	Converte o resultado de sensitivity_study num StudyResult de sensibilidade.

pinneapple_analysis.study_models

StudyModel: um só envelope de modelo para estudos (função, modelo do zoo, ONNX ou funções de teste embutidas).

pinneapple_analysis/study_models.py · 211 linhas · testes: 25

nome	tipo	assinatura	o que faz
StudyModel	classe	(name, kind, input_names, output_names, predict_fn, units, bounds, description, torch_fn,…)	Envelope de modelo para estudos: predição, nomes, unidades, limites, desvio opcional e construtores de função, zoo e ONNX.
builtin	função	(name) -> StudyModel	Modelo de teste embutido (ishigami ou sobol_g) com índices de Sobol exatos nos metadados.
columns	função	(table, names) -> np.ndarray	Empilha colunas nomeadas de uma tabela num array (n, k).

pinneapple_analysis.surrogate

Surrogates a partir de resultados de simulação; reexporta dados, divisões, candidatos, estudo, limites e o modelo empacotado.

pinneapple_analysis/surrogate/__init__.py · 40 linhas · testes: 32

pinneapple_analysis.surrogate.contracts

Leitura de tabelas com papéis de coluna (DatasetSpec) e conversão do estudo de surrogate e da arena de regimes para os contratos compartilhados.

pinneapple_analysis/surrogate/contracts.py · 217 linhas · testes: 4

nome	tipo	assinatura	o que faz
read_table_with_roles	função	(path, *, max_rows=None)	Lê uma tabela e os papéis de coluna do DatasetSpec embutido ou do arquivo .contract.json.
data_from_spec	função	(df, spec, *, group=None, grid_shape=None, meta=None) -> SurrogateData	Monta SurrogateData a partir dos papéis de coluna do spec.
dataset_spec	função	(data, name, *, app='', description='')	Cria um DatasetSpec para os dados de surrogate.
model_ref	função	(model, *, name, version, uri, app='', dataset_name='')	Cria um ModelRef para um modelo de surrogate publicado.
study_result	função	(study, *, id, app='surrogate', ref=None, title='')	Converte o resultado do estudo de surrogate num StudyResult.
arena_study_result	função	(result, *, id, app='arena', title='')	Converte o resultado da arena de regimes num StudyResult, uma linha por método e regime.

pinneapple_analysis.surrogate.data

Dados de treino de surrogate (SurrogateData): parâmetros de entrada para escalares ou campos em grade, lidos de tabelas, arrays ou snapshots.

pinneapple_analysis/surrogate/data.py · 365 linhas · testes: 13

nome	tipo	assinatura	o que faz
SurrogateData	classe	(X, Y, input_names, output_names, coords, coord_names, groups, meta)	Dados de surrogate: entradas X e saídas Y escalares ou campos, com nomes, coordenadas, grupos e hash.
grid_coords	função	(shape, ranges=None) -> np.ndarray	Coordenadas de uma grade produto tensorial na ordem ij.

nome	tipo	assinatura	o que faz
`inspect_table`	função	`(df, roles=None) -> dict[str, Any]`	Diz o que uma tabela oferece: colunas numéricas, campos, colunas de texto e entradas/saídas sugeridas.
`read_table`	função	`(source, name=None, *, fmt=None)`	Lê uma tabela de caminho ou bytes (CSV, Parquet, JSON, HDF5, NPZ) num DataFrame.
`field_columns`	função	`(columns, name) -> list[str]`	Colunas de um campo indexado, em ordem.
`from_table`	função	`(df, inputs, outputs, *, group=None, grid_shape=None, coords=None, coord_names=(), meta=N…)`	Escolhe entradas e saídas de um DataFrame; descarta linhas com valores faltantes.
`from_arrays`	função	`(arrays, *, input_names=None, output_names=None, meta=None) -> SurrogateData`	Cria `SurrogateData` de um dicionário de arrays (layout de export NPZ ou HDF5).
`lattice_order`	função	`(points) -> tuple[tuple[int, ...], np.ndarray] | None`	Detecta se os pontos formam uma grade produto tensorial e devolve forma e ordem.
`from_snapshots`	função	`(snaps, *, meta=None) -> SurrogateData`	Converte um `SnapshotSet` paramétrico em reticulado num conjunto de campos.
`snapshot_values`	função	`(snaps, data, Y) -> np.ndarray`	Leva campos previstos no reticulado de volta às linhas no layout dos snapshots.

pinneapple_analysis.surrogate.examples

Conjuntos de exemplo sem download: aerofólio RANS do zoo, campos de Burgers exatos, viga cantilever 3-D do ROM Lab e um baseline de diferenças finitas. [sem teste direto]

`pinneapple_analysis/surrogate/examples.py` · 126 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`airfoil_rans`	função	`() -> SurrogateData`	Conjunto do zoo de aerofólios CST com RANS (entradas CST e ângulo, saídas Cl e Cd).
`burgers_fields`	função	`(n=96, nu_range=(0.004, 0.1), nx=48, nt=12, t_max=0.9, seed=0) -> SurrogateData`	Campos exatos de Burgers viscoso para várias viscosidades.
`cantilever_3d`	função	`(path, n_timed=2) -> SurrogateData`	Viga cantilever 3-D FEM do ROM Lab como conjunto de campos (deslocamento e von Mises).
`burgers_fdm_baseline`	função	`(nx=201, cfl=0.4)`	Baseline numérico de diferenças finitas para `burgers_fields`, interpolado na grade do conjunto.

pinneapple_analysis.surrogate.heldout

Erros de um surrogate de campo nos dados reservados, por campo, por região do corpo e nas grandezas escalares que o engenheiro lê do campo.

`pinneapple_analysis/surrogate/heldout.py` · 100 linhas · testes: 13

nome	tipo	assinatura	o que faz
`region_rel_l2`	função	`(pred, true, regions) -> dict[str, float]`	L2 relativo restrito aos pontos de cada região.
`heldout_report`	função	`(snaps, data, model, idx, *, grow=0.1) -> dict[str, Any]`	Relatório de erros do modelo nas amostras reservadas: por campo, região, amostra e grandezas escalares.

pinneapple_analysis.surrogate.limits

Onde o surrogate pode ser usado: limites de validade, portão OOD por k-vizinhos e onde os erros se concentram. [sem teste direto]

`pinneapple_analysis/surrogate/limits.py` · 147 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`OODGate`	classe	`(X_train, names, knn_limit=3.0)`	Portão OOD: limites de validade das entradas e distância de k-vizinhos normalizada.
`per_sample_error`	função	`(pred, true, scale) -> np.ndarray`	Erro por amostra: RMS das saídas normalizadas pelo desvio do treino.

nome	tipo	assinatura	o que faz
limitations	função	(gate, X_eval, err, sets, X_train, *, field_err=None, coords=None, coord_names=(), metric…)	Resume onde o surrogate erra: erros por conjunto, por região de entrada e mapa de erro de campo.

pinneapple_analysis.surrogate.methods

Candidatos de surrogate (ridge, GP, GBT, MLP, FNO, DeepONet, PINN), regras de aplicabilidade e treino sob ComputeBudget.

pinneapple_analysis/surrogate/methods.py · 502 linhas · testes: 18 · palavras-chave no código (heurística; o texto acima diz o status real): levanta NotImplementedError em algum caminho

nome	tipo	assinatura	o que faz
TrainContext	classe	(Xtr, Ytr, Xva, Yva, budget, seed, grid_shape, coords, pde, options)	O que um método recebe: arrays, escaladores do treino, orçamento, semente, layout do campo e EDP.
Fitted	classe	(method, n_params, info)	Um candidato treinado com predict(X_raw) -> Y_raw e tamanho em bytes.
NonNegative	classe	(inner, mask)	Envelope que treina na raiz quadrada de saídas marcadas e prevê seu quadrado, evitando valores negativos.
Candidate	classe	(id, title, family, iterative, fit, check)	Registro de um candidato: id, título, família, se é iterativo, função de treino e checagem de aplicabilidade.
candidate_ids	função	() -> list[str]	Lista os ids dos candidatos registrados.
applicable	função	(kind, n_train, grid_shape=(), has_pde=False) -> dict[str, str | None]	Para cada candidato, None se aplicável ou o motivo de não ser.
fit_candidate	função	(method, ctx) -> tuple[Fitted, float]	Treina um candidato e devolve o modelo e o tempo de treino.

pinneapple_analysis.surrogate.model

SurrogateModel: predição com portão OOD, salvar/carregar, exportar (só candidatos PyTorch; outros levantam NotImplementedError) e cartão do modelo. [sem teste direto]

pinneapple_analysis/surrogate/model.py · 179 linhas · **sem teste direto** · palavras-chave no código (heurística; o texto acima diz o status real): levanta NotImplementedError em algum caminho

nome	tipo	assinatura	o que faz
SurrogateModel	classe	(fitted, gate, *, input_names, output_names, grid_shape=(), coords=None, coord_names=(), …)	Surrogate treinado e empacotado: predição com portão OOD, salvar/carregar, ModelRef e export TorchScript/ONNX.

pinneapple_analysis.surrogate.pde

Declarações de EDP (heat, burgers, advection) que dizem qual resíduo o candidato PINN soma à perda de dados. [sem teste direto]

pinneapple_analysis/surrogate/pde.py · 142 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
PDEDeclaration	classe	(type, coord_index, coefficients)	Declaração de EDP (tipo, coordenadas, coeficientes) com resíduo em torch e por diferenças finitas.
parse_pde	função	(decl, coord_names, input_names) -> PDEDeclaration | None	Valida uma declaração de EDP contra os nomes de coordenadas e entradas; None passa adiante.

pinneapple_analysis.surrogate.splits

Divisões treino/validação/teste e um conjunto de mudança de distribuição retirado antes da divisão aleatória. [sem teste direto]

pinneapple_analysis/surrogate/splits.py · 105 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Splits`	classe	`(train, val, test, shift, shift_kind, shift_description, seed, info)`	Índices de treino, validação, teste e mudança de distribuição, com descrição do tipo de mudança.
`make_splits`	função	`(data, *, test_frac=0.2, val_frac=0.15, shift='auto', shift_frac=0.15, seed=0) -> Splits`	Cria índices disjuntos; o conjunto de mudança sai antes da divisão aleatória.

pinneapple_analysis.surrogate.study

`run_study`: treina cada candidato aplicável nos mesmos dados e orçamento, mede em dados não vistos, escolhe pela validação e empacota com limitações e `TrustReport`.

`pinneapple_analysis/surrogate/study.py` · 380 linhas · testes: 13

nome	tipo	assinatura	o que faz
`StudyResult`	classe	`(data, splits, budget, seeds, methods, best, model, limitations, trust, metric, created, …)`	Resultado do estudo de surrogate: métodos, melhor, modelo, limitações, confiança, ranking e Markdown.
`evaluate`	função	`(kind, pred, true, scale, output_names) -> dict[str, Any]`	Métricas das predições achatadas contra a verdade (nRMSE para escalares, L2 relativo para campos).
`run_study`	função	`(data, *, methods='auto', budget=10.0, seeds=1, splits=None, split_options=None, pde=None…)`	Roda o estudo de surrogate completo sob um orçamento por run de treino.
`nonnegative_mask`	função	`(data) -> np.ndarray | None`	Máscara das saídas declaradas não negativas em `meta['nonnegative']`.

pinneapple_analysis.trust

Pontuação de confiança antes de consumir predições; reexporta `TrustGate`, `TrustReport` e o cartão de campo. [sem teste direto do **init**]

`pinneapple_analysis/trust/__init__.py` · 20 linhas · **sem teste direto**

pinneapple_analysis.trust.field_card

Cartão de confiança de um campo em grade: checagens de sanidade, física, referência e generalização que viram decisão de deploy.

`pinneapple_analysis/trust/field_card.py` · 358 linhas · testes: 12

nome	tipo	assinatura	o que faz
`FieldCheck`	classe	`(name, kind, passed, value, threshold, critical, detail)`	Uma checagem de campo (sanity, physics, reference, generalization); `passed` None quando não pôde rodar.
`FieldTrustCard`	classe	`(checks, stages, decision, reasons, name)`	Cartão de confiança de um campo: checagens, estágios, decisão e motivos, com saída em Markdown.
`field_checks`	função	`(grid, values, *, pde=None, source=None, alpha=1.0, time_coord='t', dirichlet=None, refer…)`	Roda as checagens de um campo escalar em grade; checagem sem entrada aparece como não executada, com motivo.
`trust_card`	função	`(grid, values, *, name='field', is_ml=True, limitations=(), min_coverage=0.5, **kwargs) -…)`	Roda `field_checks` e produz a decisão de deploy e as seis perguntas do cartão de confiança.

pinneapple_analysis.trust.trust_gate

`TrustGate`: combina OOD, resíduo físico e variância de ensemble num escore antes de a predição alimentar um gêmeo digital.

`pinneapple_analysis/trust/trust_gate.py` · 199 linhas · testes: 10

nome	tipo	assinatura	o que faz
`TrustScore`	classe	`(combined, ood_score, residual_score, ensemble_score, details)`	Escore combinado e as partes OOD, resíduo e ensemble de uma predição.

nome	tipo	assinatura	o que faz
`TrustGate`	classe	`(*, ood_weight=0.4, residual_weight=0.4, ensemble_weight=0.2, ood_level=0.01)`	Ajusta o domínio de treino e pontua uma predição combinando OOD, resíduo e variância de ensemble.

pinneapple_analysis.trust.trust_report

`TrustReport`: junta checagens independentes numa decisão APPROVED, REVIEW ou REJECT com cobertura e score; API com nomes em português. [experimental]

`pinneapple_analysis/trust/trust_report.py` · 179 linhas · testes: 13 · palavras-chave no código (heurística; o texto acima diz o status real): experimental

nome	tipo	assinatura	o que faz
`Check`	classe	`(name, status, critical, weight, value, limit, source, reason)`	Uma checagem do relatório (status supports, contradicts ou not_run, crítica, peso, valor, limite, fonte).
`TrustReport`	classe	`(checks, min_coverage, min_score, model, details)`	Junta checagens e calcula cobertura, score e decisão APPROVED/REVIEW/REJECT com motivos.
`checagem_erro_relativo`	função	`(nome, erro, limite, *, fonte='', critica=False, peso=1.0, motivo_nao_executada='') -> Ch…)`	Cria uma checagem de acurácia: erro relativo menor ou igual ao limite.
`checagem_booleana`	função	`(nome, passou, *, fonte='', critica=False, peso=1.0, motivo_nao_executada='') -> Check`	Cria uma checagem booleana de consistência física ou comparação qualitativa.
`from_trust_score`	função	`(ts, *, limite_sub_score=0.7, fonte='TrustGate') -> List[Check]`	Converte o `TrustScore` do `TrustGate` em checagens de OOD, resíduo e ensemble.

pinneapple_analysis.uncertainty

Quantificação de incerteza (antigo `pinneapple_uq`); reexporta aleatória, epistêmica, conformal, calibração, quantis, reamostragem, drift, tolerâncias e `uq_predict`.

`pinneapple_analysis/uncertainty/__init__.py` · 186 linhas · testes: 39

pinneapple_analysis.uncertainty.aleatoric

Incerteza aleatória por regressão heteroscedástica: cabeça de log-variância e perda NLL gaussiana.

`pinneapple_analysis/uncertainty/aleatoric.py` · 164 linhas · testes: 157

nome	tipo	assinatura	o que faz
`AleatoricHead`	classe	`(base, out_dim, in_dim, *, hidden=64) -> None`	Envolve um modelo determinístico e acrescenta uma cabeça que aprende a log-variância.
`aleatoric_nll_loss`	função	`(mean, log_var, target, *, min_log_var=-10.0, max_log_var=10.0) -> Tensor`	Perda NLL gaussiana heteroscedástica com log-variância limitada.

pinneapple_analysis.uncertainty.applicability

Domínio de aplicabilidade (portão de densidade k-NN + faixa) e regra que manda para o solver de alta fidelidade os casos que o modelo não pode responder.

`pinneapple_analysis/uncertainty/applicability.py` · 163 linhas · testes: 8

nome	tipo	assinatura	o que faz
`ApplicabilityDomain`	classe	`(names, lo, hi, center, scale, k, level, threshold, n_ref, bounds_source, _tree, _gate)`	Portão de densidade k-NN e de faixa sobre entradas padronizadas.
`route_cases`	função	`(status, pred, lower, upper, tolerance=None, limits=None) -> list[dict]`	Regra de roteamento por caso: surrogate ou high_fidelity, com os motivos.
`routing_summary`	função	`(routes) -> dict`	Contagens de cada rota e de cada código de motivo.

pinneapple_analysis.uncertainty.calibration

Métricas de calibração e nitidez para regressão probabilística (ECE, cobertura, curva de calibração, NLL gaussiana).

pinneapple_analysis/uncertainty/calibration.py · 302 linhas · testes: 185

nome	tipo	assinatura	o que faz
`CalibrationMetrics`	classe	`()`	Métodos estáticos de calibração e nitidez (ECE, cobertura, dados de curva, nitidez, NLL).

pinneapple_analysis.uncertainty.conformal

Predição conformal split para modelos PyTorch, com quantil de amostra finita e checagem de tamanho mínimo de calibração.

pinneapple_analysis/uncertainty/conformal.py · 281 linhas · testes: 151

nome	tipo	assinatura	o que faz
`ConformalPredictor`	classe	`(model, alpha=0.1) -> None`	Preditor conformal split: calibra num conjunto reservado e dá intervalos com cobertura garantida.
`conformal_quantile`	função	`(scores, alpha) -> float`	Quantil conformal split de qualquer array de escores de não conformidade.
`conformal_quantile_valid`	função	`(n, alpha) -> bool`	Diz se `n` escores de calibração bastam para a garantia de amostra finita no nível `alpha`.

pinneapple_analysis.uncertainty.conformal_study

Intervalos conformais para predições NumPy de qualquer modelo, com cobertura medida num teste separado e cobertura por grupo.

pinneapple_analysis/uncertainty/conformal_study.py · 195 linhas · testes: 8

nome	tipo	assinatura	o que faz
`ConformalStudy`	classe	`(alpha, q, normalized, n_cal, n_test, coverage, coverage_ci, mean_width, median_width, le…)`	Calibração conformal e sua avaliação reservada (cobertura, intervalo, larguras, cobertura por grupo).
`conformal_scores`	função	`(y, pred, scale=None) -> np.ndarray`	Escores de não conformidade `|y - pred|`, divididos pela escala quando dada.
`conformal_quantile`	função	`(scores, alpha=0.1) -> float`	Quantil conformal de amostra finita dos escores.
`conformal_interval`	função	`(pred, q, scale=None) -> tuple[np.ndarray, np.ndarray]`	Intervalo `pred -+ q` (vezes a escala quando normalizado).
`min_calibration_size`	função	`(alpha) -> int`	Menor conjunto de calibração com quantil finito no nível `1 - alpha`.
`coverage_by_group`	função	`(covered, groups, labels=None, level=0.95, width=None) -> list[dict]`	Cobertura medida por grupo, mostrando subcobertura escondida pela garantia marginal.
`quantile_bins`	função	`(x, n_bins=5) -> tuple[np.ndarray, list[str]]`	Classes de mesma contagem de `x` com rótulos.
`conformal_study`	função	`(y_cal, pred_cal, y_test, pred_test, alpha=0.1, scale_cal=None, scale_test=None, level=0.…)`	Calibra em `(y_cal, pred_cal)` e mede a cobertura em `(y_test, pred_test)`.
`split_indices`	função	`(n, fractions=(0.5, 0.5), seed=0) -> list[np.ndarray]`	Conjuntos disjuntos aleatórios de índices com as frações dadas.

pinneapple_analysis.uncertainty.core

Tipo `UQResult` e a interface unificada `uq_predict` que despacha para MC Dropout, ensemble e outros métodos. [sem teste direto]

pinneapple_analysis/uncertainty/core.py · 265 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`UQResult`	classe	`(mean, std, aleatoric_std, epistemic_std, samples, metadata)`	Contêiner de predição com média, desvio, partes aleatória e epistêmica, amostras e intervalos.
`uq_predict`	função	`(model, x, method='mc_dropout', device=None, **kwargs) -> UQResult`	Interface unificada de predição com incerteza (método escolhido por `method`).

pinneapple_analysis.uncertainty.decomposition

Separa a variância preditiva em parte aleatória e epistêmica a partir de passes estocásticos de um modelo com cabeça aleatória.

`pinneapple_analysis/uncertainty/decomposition.py` · 143 linhas · testes: 6

nome	tipo	assinatura	o que faz
`decompose_uncertainty`	função	`(model, x, *, n_samples=100, has_aleatoric=True, device=None) -> UQResult`	Separa a incerteza preditiva em aleatória e epistêmica.

pinneapple_analysis.uncertainty.distributions

Entradas incertas de um estudo: uma distribuição por parâmetro, amostrada por Monte Carlo, hipercubo latino ou Sobol.

`pinneapple_analysis/uncertainty/distributions.py` · 187 linhas · testes: 8

nome	tipo	assinatura	o que faz
`UncertainParameter`	classe	`(name, dist, params, unit, nominal)`	Uma entrada incerta com distribuição, parâmetros, unidade, valor nominal, ppf, cdf e suporte.
`as_parameter`	função	`(spec, name=None) -> UncertainParameter`	Cria um parâmetro a partir de dict, par (low, high) uniforme ou `UncertainParameter`.
`as_space`	função	`(space) -> list[UncertainParameter]`	Cria a lista de parâmetros de lista, dicionário ou objeto com `parameters`.
`to_values`	função	`(space, U) -> np.ndarray`	Leva pontos do hipercubo unitário a valores pelos inversos das CDFs.
`unit_sample`	função	`(d, n, method='lhs', seed=0) -> np.ndarray`	Amostra o hipercubo unitário por MC, hipercubo latino ou Sobol embaralhado.
`sample_space`	função	`(space, n, method='lhs', seed=0) -> np.ndarray`	Amostra `n` pontos dos parâmetros do espaço.

pinneapple_analysis.uncertainty.drift

Monitoramento de drift de um surrogate em uso: acurácia, calibração, mudança de entrada e tendência do resíduo, com decisão ok/review/retrain.

`pinneapple_analysis/uncertainty/drift.py` · 208 linhas · testes: 6

nome	tipo	assinatura	o que faz
`DriftThresholds`	classe	`(nrmse_review, nrmse_retrain, bias_z, coverage_z, ks_alpha, oob_review, oob_retrain, tren…)`	Limiares da decisão ok/review/retrain de `assess_drift`.
`DriftReport`	classe	`(n, nrmse, bias, bias_z, level, coverage, ks_stat, ks_pvalue, out_of_bounds, trend_tau, t…)`	Resultado de `assess_drift` com nRMSE, viés, cobertura, KS, fora de faixa, tendência e decisão.
`assess_drift`	função	`(y_true, y_pred, sigma=None, X_ref=None, X_new=None, bounds=None, level=0.95, thresholds=…)`	Checa um lote de predições quanto a perda de acurácia, má calibração, mudança de entrada e tendência do resíduo.

pinneapple_analysis.uncertainty.ensemble

Incerteza por Deep Ensemble: média e dispersão de vários modelos treinados de forma independente.

`pinneapple_analysis/uncertainty/ensemble.py` · 251 linhas · testes: 151

nome	tipo	assinatura	o que faz
`EnsembleConfig`	classe	`(n_members, diversity_reg, seed)`	Configuração do Deep Ensemble (número de membros, regularização de diversidade, semente).
`EnsembleUQ`	classe	`(models, config=None) -> None`	Envelope de Deep Ensemble que dá média e incerteza a partir dos membros.

`pinneapple_analysis.uncertainty.margin`

Estudo de margem por Monte Carlo: um projeto que passa nas dimensões nominais ainda passa com as tolerâncias de fabricação?

`pinneapple_analysis/uncertainty/margin.py` · 130 linhas · testes: 6

nome	tipo	assinatura	o que faz
`corner_stackup`	função	`(fn, nominal, tolerances) -> dict[str, Any]`	Avalia `fn` nos 2^d cantos da caixa de tolerâncias.
`summarize_samples`	função	`(values, limit, marginal_fraction=0.05) -> dict[str, Any]`	Estatísticas da distribuição e veredito pelo p95 contra um limite superior.
`margin_study`	função	`(fn, nominal, tolerances, limit, *, n=200, n_small=50, dist='uniform', seed=None, check_s…)`	Estudo de margem Monte Carlo contra um limite superior, com checagens do próprio estudo.

`pinneapple_analysis.uncertainty.mc_dropout`

MC Dropout para incerteza epistêmica: envolve qualquer `nn.Module` e mantém o dropout ativo na inferência.

`pinneapple_analysis/uncertainty/mc_dropout.py` · 414 linhas · testes: 163

nome	tipo	assinatura	o que faz
`MCDropoutConfig`	classe	`(n_samples, dropout_p, seed)`	Configuração do MC Dropout (amostras, probabilidade, semente).
`MCDropoutWrapper`	classe	`(model, config) -> None`	Envolve um `nn.Module` e liga dropout na inferência por hooks; avisa quando não há ativação oculta para descartar.
`MCDropout`	classe	`(model, config=None) -> None`	Classe de conveniência que aplica MC Dropout a qualquer modelo chamável.

`pinneapple_analysis.uncertainty.pce`

Caos polinomial (Legendre) por mínimos quadrados: média, variância e índices de Sobol a partir de uma regressão.

`pinneapple_analysis/uncertainty/pce.py` · 123 linhas · testes: 8

nome	tipo	assinatura	o que faz
`PCEResult`	classe	`(names, degree, n_terms, n_evals, mean, std, S1, ST, loo_q2, alphas, coef)`	Expansão ajustada: média, desvio, índices S1/ST, Q^2 leave-one-out e coeficientes.
`multi_indices`	função	`(d, degree) -> np.ndarray`	Todos os multi-índices de grau total até `degree`.
`legendre_design`	função	`(xi, alphas) -> np.ndarray`	Matriz de desenho de produtos de Legendre ortonormais.
`n_terms`	função	`(d, degree) -> int`	Número de termos de uma expansão de grau total em `d` variáveis.
`polynomial_chaos`	função	`(fn, space, degree=3, n=None, seed=0, max_terms=500) -> PCEResult`	Ajusta um caos de Legendre a `fn` em amostras de hipercubo latino do espaço.

`pinneapple_analysis.uncertainty.posterior_metrics`

Métricas de comparação de posteriores para solvers inversos generativos (MMD, Wasserstein fatiado, espectro radial, CRPS).

`pinneapple_analysis/uncertainty/posterior_metrics.py` · 172 linhas · testes: 7

nome	tipo	assinatura	o que faz
`normalize_weights`	função	`(weights, n) -> np.ndarray`	Valida e normaliza pesos de amostras (uniformes quando None).
`weighted_mmd`	função	`(pred, ref, pred_weights=None, ref_weights=None) -> float`	MMD ponderado com kernel gaussiano e banda pela mediana das distâncias.
`weighted_wasserstein_1d`	função	`(x, y, wx=None, wy=None) -> float`	Distância de Wasserstein-1 entre duas amostras 1-D ponderadas.
`grf_direction`	função	`(h, w, rng, alpha=2.0, tau=3.0) -> np.ndarray`	Direção aleatória suave de norma unitária vinda de um campo gaussiano aleatório.
`sliced_wasserstein`	função	`(pred, ref, pred_weights=None, ref_weights=None, n_projections=128, seed=0, alpha=2.0, ta…)`	Wasserstein fatiado entre ensembles de campos por projeções GRF aleatórias.
`radial_power_spectrum`	função	`(samples, weights=None) -> np.ndarray`	Espectro de potência radial médio ponderado de amostras 2-D.
`spectral_relative_geomean`	função	`(pred_spectrum, ref_spectrum, eps=1e-10) -> float`	Média geométrica do erro relativo entre dois espectros.
`evaluate_posterior`	função	`(pred, ref, pred_weights=None, ref_weights=None, *, n_projections=128, seed=0, swd_mean=N…)`	Compara um ensemble posterior previsto com um de referência (menor é melhor).
`crps_gaussian`	função	`(mu, sigma, y) -> np.ndarray`	CRPS em forma fechada para uma preditiva gaussiana.
`crps_ensemble`	função	`(samples, y, *, fair=False) -> np.ndarray`	CRPS de ensemble, com opção de normalização justa.

`pinneapple_analysis.uncertainty.quantile`

Regressão quantílica: cabeça de quantis, perda pinball e taxa de cruzamento de quantis.

`pinneapple_analysis/uncertainty/quantile.py` · 212 linhas · testes: 161

nome	tipo	assinatura	o que faz
`QuantileConfig`	classe	`(quantiles)`	Configuração dos níveis de quantil.
`QuantileHead`	classe	`(base, cfg, hidden_dim=64, rearrange=True) -> None`	Envolve um modelo pontual e projeta a saída em vários quantis, com reordenação opcional.
`QuantileLoss`	classe	`(quantiles) -> None`	Perda pinball de múltiplos quantis compatível com o Trainer.
`pinball_loss_torch`	função	`(y_pred, y_true, q) -> torch.Tensor`	Perda pinball de um nível de quantil.
`crossing_rate`	função	`(y_pred, quantiles) -> float`	Fração de predições com quantis fora de ordem.

`pinneapple_analysis.uncertainty.reachability`

Viabilidade de encontro por conjuntos alcançáveis (risco de colisão, rendezvous); emite ResearchOnlyWarning e é recusado em modo comercial. [experimental]

`pinneapple_analysis/uncertainty/reachability.py` · 115 linhas · testes: 7

nome	tipo	assinatura	o que faz
`BodyState`	classe	`(position, velocity, pos_uncertainty, vel_uncertainty, max_accel, latency)`	Estado de um corpo com incertezas de posição e velocidade, aceleração máxima e latência; dá centro e raio da bola alcançável.
`encounter_feasibility`	função	`(a, b, horizon, *, contact_distance=0.0, n=2001, refine=True) -> Dict[str, object]`	Margem entre as bolas alcançáveis de dois corpos no horizonte; diz se o contato é possível.
`time_to_reach_region`	função	`(chaser, target_center, target_radius=0.0, horizon=1000000.0) -> Optional[float]`	Primeiro instante em que uma região alvo estática fica dentro da bola alcançável.

`pinneapple_analysis.uncertainty.resampling`

Intervalos de confiança por reamostragem para dados dependentes: bootstrap em blocos e por grupo, skill score e jackknife.

`pinneapple_analysis/uncertainty/resampling.py` · 206 linhas · testes: 9

nome	tipo	assinatura	o que faz
block_bootstrap	função	(x, stat=np.mean, block=20, n=2000, seed=0, level=0.95) -> tuple[float, float, float]	Bootstrap de blocos móveis de uma estatística para amostra com dependência serial.
group_bootstrap	função	(x, groups, stat=np.mean, n=2000, seed=0, level=0.95) -> tuple[float, float, float]	Bootstrap por grupos inteiros com reposição.
skill_score	função	(err_model, err_ref, *, block=None, groups=None, n=2000, seed=0, level=0.95) -> tuple[flo…)	Skill score 1 - E|e_model| / E|e_ref| com intervalo bootstrap pareado.
jackknife	função	(samples, fn, n_blocks=20)	Estimativa jackknife de um bloco removido e erro padrão.

pinneapple_analysis.uncertainty.residual_bound

Transforma o resíduo da EDP de uma predição (sem precisar da resposta verdadeira) num limite de erro, garantido em norma de energia ou calibrado por conformal.

pinneapple_analysis/uncertainty/residual_bound.py · 242 linhas · testes: 19

nome	tipo	assinatura	o que faz
LinearSPDProblem	classe	(A, f, L, c, weight, shape, _L_solve)	Problema linear A u = f com A SPD, com resíduo, norma de energia, norma dual e solução.
CoverageReport	classe	(label, coverage, target, n, holds, tolerance, median_bound, note)	Relatório da cobertura medida de um limite contra o alvo.
ResidualLoss	classe	(problems)	Perda física diferenciável (resíduo relativo) sobre um conjunto fixo de problemas, em PyTorch esparso.
ResidualErrorCalibrator	classe	(alpha=0.1, mode='ratio')	Mapa conformal split de uma medida de resíduo para um limite de erro.
darcy_problem	função	(a, f=1.0) -> LinearSPDProblem	Problema de Darcy na discretização de operator_benchmarks.darcy_2d.
heat_conduction_problem	função	(k, q) -> LinearSPDProblem	Condução estacionária com T = 0 na borda, mesmo operador de Darcy.
energy_error_bound	função	(problem, u) -> dict[str, float]	Limite garantido do erro em norma de energia a partir só do resíduo.
coverage_study	função	(res, err, *, alpha=0.1, n_cal=None, n_splits=200, seed=0) -> dict[str, float]	Cobertura do limite em modo ratio sobre várias divisões aleatórias de calibração e teste.

pinneapple_analysis.uncertainty.study

Estudo de incerteza de uma saída: intervalos conformais medidos, portão de aplicabilidade, triagem de casos e propagação de entradas incertas.

pinneapple_analysis/uncertainty/study.py · 220 linhas · testes: 12

nome	tipo	assinatura	o que faz
uncertainty_study	função	(model, output, X_cal, y_cal, X_test, y_test, *, alpha=0.1, scale='absolute', X_ref=None,…)	Roda o estudo de incerteza de uma saída de um StudyModel.

pinneapple_analysis.uncertainty.tolerance

Propagação Monte Carlo de tolerâncias de entrada por qualquer modelo (JCGM 101), com probabilidade de ficar dentro dos limites e intervalo de Wilson.

pinneapple_analysis/uncertainty/tolerance.py · 166 linhas · testes: 20

nome	tipo	assinatura	o que faz
ToleranceResult	classe	(mean, std, percentiles, n, inputs, outputs, p_within, p_within_ci, p_within_each)	Resultado de propagate: média, desvio, percentis e probabilidade de ficar dentro dos limites.
wilson_interval	função	(k, n, level=0.95) -> tuple[float, float]	Intervalo de score de Wilson para uma proporção binomial.
propagate	função	(fn, nominal, tolerances, n=10000, dist='normal', limits=None, seed=0, level=0.95, percen…)	Propaga tolerâncias independentes por fn via Monte Carlo.

nome	tipo	assinatura	o que faz
`propagate_space`	função	`(fn, space, n=4096, method='lhs', limits=None, seed=0, level=0.95, percentiles=(1, 5, 50,…)`	Propaga entradas com qualquer distribuição por `fn`.
`src`	função	`(X, y) -> tuple[np.ndarray, float]`	Coeficientes de regressão padronizados e R^2 do ajuste linear: quanto da dispersão cada tolerância explica.

pinneapple_analysis.validation

Reexporta as checagens clássicas (`PhysicsValidator`, `ConservationCheck`, `BoundaryCheck`, `SymmetryCheck`, `validate_model`), comparações com solução analítica/solver, funções espectrais, metamórficas e do relatório-artefato. O docstring vem depois de `from __future__` e por isso não vira docstring do módulo.

`pinneapple_analysis/validation/__init__.py` · 82 linhas · testes: 21

pinneapple_analysis.validation.analytical

Compara as predições de um modelo com uma solução analítica em pontos aleatórios do domínio, ou com dados de um solver de referência (RMSE, L2 relativo, erro máximo). [sem teste direto]

`pinneapple_analysis/validation/analytical.py` · 187 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`compare_to_analytical`	função	`(model, analytical_fn, coord_names, domain_bounds, n_points=5000, field_names=None, devic…)`	Compara as predições do modelo com uma função analítica em pontos amostrados no domínio.
`validate_against_solver`	função	`(model, solver_data, coord_names, field_names, device='cpu') -> dict`	Compara as predições do modelo com dados de referência de um solver.

pinneapple_analysis.validation.arena

Red team / blue team para validação: geradores adversariais de casos errados porém plausíveis contra as checagens, e um laço que as endurece; o módulo é chamável como `run_arena`.

`pinneapple_analysis/validation/arena/__init__.py` · 43 linhas · testes: 12

pinneapple_analysis.validation.arena.__main__

Ponto de entrada `python -m pinneapple_analysis.validation.arena`: roda a arena em Darcy 2D e Burgers 1D. [sem teste direto]

`pinneapple_analysis/validation/arena/__main__.py` · 49 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`main`	função	`(argv=None) -> int`	Executa a arena pela linha de comando e devolve o código de saída.

pinneapple_analysis.validation.arena.blue

Blue team: limiares calibrados com taxa de falso alarme declarada (teste conformal dividido combinado) e endurecimento com checagens extras até rejeitar as fugas guardadas. [sem teste direto]

`pinneapple_analysis/validation/arena/blue.py` · 201 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Defence`	classe	`(checks, thresholds, kind, alpha, alpha_cal, scales, statistic_threshold, n_calibration, …)`	Conjunto de checagens com um limiar cada (rejeita quando algum valor passa do limiar), com estatística combinada e descrição.
`score_matrix`	função	`(checks, cases) -> dict[str, np.ndarray]`	Calcula o valor de cada checagem em cada caso.

nome	tipo	assinatura	o que faz
`calibrate_scores`	função	`(checks, values, alpha=0.01, *, safety=0.5, seed=0, false_alarms=None, kind=…, floor=0.01…)`	Calibra os limiares a partir de valores já calculados nos casos limpos, mantendo os falsos alarmes guardados abaixo do limiar.
`calibrate`	função	`(checks, clean_cases, alpha=0.01, *, safety=0.5, seed=0, false_alarms=()) -> Defence`	Teste conformal dividido combinado das checagens sobre casos limpos.
`harden`	função	`(current, library, clean_cases, *, escapes=(), false_alarms=(), alpha=0.01, safety=0.5, s…)`	Calibra a defesa e adiciona checagens da biblioteca até rejeitar todas as fugas guardadas (ou nenhuma ajudar).

`pinneapple_analysis.validation.arena.cases`

Guarda casos de regressão da arena e os reexecuta (usado por `pinneapple check --arena-cases`). [sem teste direto]

`pinneapple_analysis/validation/arena/cases.py` · 108 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ReplayResult`	classe	`(path, problem, rows)`	Resultado da reexecução de um arquivo de casos; aponta regressões.
`save_cases`	função	`(path, problem, defence, findings, meta=None) -> Path`	Grava problema, defesa e achados num arquivo de casos.
`load_cases`	função	`(path) -> dict[str, Any]`	Lê um arquivo de casos.
`builtin_cases`	função	`() -> list[Path]`	Lista os arquivos de casos que vêm com a biblioteca (um por problema).
`load_defence`	função	`(path) -> tuple[Any, Defence]`	Devolve o problema e a defesa endurecida guardados num arquivo de casos.
`replay`	função	`(path) -> ReplayResult`	Regenera cada caso guardado e o confere contra a defesa guardada.

`pinneapple_analysis.validation.arena.generators`

Geradores do red team: famílias parametrizadas de saídas erradas porém plausíveis que a busca de evasão explora. [sem teste direto]

`pinneapple_analysis/validation/arena/generators.py` · 164 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Generator`	classe	`(name, bounds, fn, problems, physics_consistent, description)`	Família parametrizada de saídas erradas porém plausíveis, com limites e flag de consistência física.
`get_generator`	função	`(name) -> Generator`	Devolve um gerador pelo nome.
`generators_for`	função	`(problem) -> list[Generator]`	Lista os geradores aplicáveis a um problema.

`pinneapple_analysis.validation.arena.loop`

Laço da arena: rodadas de red team e depois blue team, o relatório e a auditoria de um modelo externo contra uma defesa. [sem teste direto]

`pinneapple_analysis/validation/arena/loop.py` · 396 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ArenaBudget`	classe	`(n_bases, evals, fa_seeds, fa_evals, n_calibration, n_holdout, n_mutation_cases, n_datase…)`	Quanto o red team busca e quantos casos limpos o blue team usa para calibrar.
`RoundReport`	classe	`(round, defence_before, defence_after, mutation_before, mutation_after, escapes, attempts…)`	Relatório de uma rodada: defesa antes/depois, fugas, falsos alarmes e escore de mutação.
`ArenaReport`	classe	`(problem, alpha, min_error, rounds, defence, stored, settings, _problem)`	Relatório completo da arena, com rodadas, defesa final, casos guardados e saída em Markdown/JSON.

nome	tipo	assinatura	o que faz
`ModelAudit`	classe	`(problem, defence, true_error, rejected, seeds, min_error)`	Saídas de um modelo julgadas por uma defesa contra o solver de referência (fugas e falsos alarmes).
`run_arena`	função	`(problem='darcy_2d', checks=None, *, budget=None, rounds=1, alpha=0.01, safety=0.5, min_e…)`	Roda rodadas de red team e blue team num problema e devolve o `ArenaReport`.
`audit_model`	função	`(problem, predict, defence=None, *, n_cases=50, seed=0, min_error=0.1, varied=True, **pro…)`	Passa um modelo externo (FNO, MeshGraphNet, PINN…) por uma defesa.

`pinneapple_analysis.validation.arena.red`

Red team: busca fugas (erradas mas aceitas) e falsos alarmes (corretas mas rejeitadas) contra uma defesa, mais o escore de mutação. [sem teste direto]

`pinneapple_analysis/validation/arena/red.py` · 178 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Finding`	classe	`(kind, problem, seed, variation, generator, params, true_error, values, margin, found, me…)`	Uma fuga, um falso alarme ou a melhor tentativa de um deles, com parâmetros e erro verdadeiro.
`evasion_search`	função	`(problem, defence, generator, base, *, budget=200, method='de', seed=0, min_error=0.1) ->…)`	Busca a pior saída de um gerador que a defesa ainda aceita.
`false_alarm_search`	função	`(problem, defence, *, seeds, budget=100, method='de', search_seed=0) -> list[Finding]`	Para cada semente, busca o caso correto com maior margem de rejeição da defesa.
`mutation_score`	função	`(matrix, thresholds, min_error=0.1) -> dict[str, Any]`	Fração dos mutantes com erro relevante que os limiares rejeitam, no total e por falha.
`mutation_escapes`	função	`(matrix, thresholds, *, seed, min_error=0.1, per_fault=1) -> list[Finding]`	Lista os mutantes relevantes que passam, como achados reproduzíveis.

`pinneapple_analysis.validation.arena.search`

Minimizadores caixa-preta no cubo unitário para o red team: evolução diferencial (SciPy), CMA-ES e L-BFGS-B por diferenças finitas. [sem teste direto]

`pinneapple_analysis/validation/arena/search.py` · 130 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`SearchResult`	classe	`(x, fun, evaluations, method)`	Resultado de uma minimização: ponto, valor, avaliações e método.
`cmaes`	função	`(f, dim, budget, seed=0, sigma0=0.3, x0=None) -> SearchResult`	CMA-ES (mu/mu_w, lambda) no cubo [0, 1]^dim.
`differential_evolution`	função	`(f, dim, budget, seed=0) -> SearchResult`	Evolução diferencial do SciPy dentro de um orçamento de avaliações.
`lbfgs`	função	`(f, dim, budget, seed=0) -> SearchResult`	L-BFGS-B com gradiente por diferenças finitas a partir de alguns pontos iniciais.
`minimize`	função	`(f, dim, budget, method='de', seed=0) -> SearchResult`	Minimiza uma função no cubo unitário com o método escolhido (`de`, `cmaes` ou `lbfgs`).

`pinneapple_analysis.validation.boundary`

Checagens de condições de contorno de uma PINN treinada (Dirichlet, Neumann, periodicidade) consultando o modelo em pontos de borda.

`pinneapple_analysis/validation/boundary.py` · 255 linhas · testes: 151

nome	tipo	assinatura	o que faz
`BoundaryCheck`	classe	`(device='cpu') -> None`	Checa condições de contorno consultando o modelo em pontos de borda.

`pinneapple_analysis.validation.conservation`

Checagens de leis de conservação (massa, energia, integral genérica) por integração numérica sobre o domínio.

`pinneapple_analysis/validation/conservation.py` · 375 linhas · testes: 151

nome	tipo	assinatura	o que faz
`ConservationCheck`	classe	`(device='cpu') -> None`	Checa leis de conservação por integração numérica no domínio.

pinneapple_analysis.validation.core

Estruturas de dados do validador clássico: `CheckResult` e o `ValidationReport` agregado (não confundir com o de `validate.py`). [sem teste direto]

`pinneapple_analysis/validation/core.py` · 175 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`CheckResult`	classe	`(name, passed, value, threshold, description, unit)`	Resultado de uma checagem física: nome, aprovado, valor, limiar e descrição.
`ValidationReport`	classe	`(model_name, timestamp, checks)`	Relatório agregado do validador clássico, com resumo em tabela, `to_dict` e `save`.

pinneapple_analysis.validation.error_map

Mostra onde um campo está errado: erro ponto a ponto contra a referência, aglomerados de erro alto (hotspots), participação por região e um texto que compara com o número global.

`pinneapple_analysis/validation/error_map.py` · 182 linhas · testes: 9

nome	tipo	assinatura	o que faz
`ErrorMap`	classe	`(error, rel_l2, max_error, rms_error, hotspots, regions, unit, label)`	Resultado de `error_map`: erro por ponto, L2 relativo, erro máximo e RMS, hotspots e regiões.
`pointwise_error`	função	`(pred, ref) -> np.ndarray`	Calcula `|pred - ref|` por ponto (norma Euclidiana para vetores).
`error_hotspots`	função	`(points, err, *, n=5, quantile=0.9, link=None, weights=None) -> list[dict[str, Any]]`	Agrupa os pontos de erro acima de um quantil em aglomerados e devolve os de maior erro.
`grow_region`	função	`(points, idx, radius) -> np.ndarray`	Devolve os pontos a menos de um raio de um conjunto de índices.
`error_by_region`	função	`(err, regions, *, weights=None) -> list[dict[str, Any]]`	Por região nomeada: participação nos pontos e no erro quadrático e razão do RMS local para o global.
`error_map`	função	`(points, pred, ref, *, regions=None, weights=None, n_hotspots=5, quantile=0.9, unit='', l…)`	Calcula erro ponto a ponto, hotspots e participação por região de uma predição contra a referência.

pinneapple_analysis.validation.fault_injection

Injeção de falhas: mede com que frequência cada checagem de validação pega defeitos conhecidos de modelo e de dados.

`pinneapple_analysis/validation/fault_injection/__init__.py` · 36 linhas · testes: 20

pinneapple_analysis.validation.fault_injection.__main__

Ponto de entrada `python -m pinneapple_analysis.validation.fault_injection`: regenera a matriz de detecção (poder das checagens). [sem teste direto]

`pinneapple_analysis/validation/fault_injection/__main__.py` · 59 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`main`	função	`(argv=None) -> int`	Regenera a matriz de detecção pela linha de comando e devolve o código de saída.

pinneapple_analysis.validation.fault_injection.checks

As checagens em teste, como objetos `ValidityCheck` de `physics_audit.monitor` que pontuam um `Case`, incluindo checagens de conjunto de dados. [sem teste direto]

`pinneapple_analysis/validation/fault_injection/checks.py` · 289 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`score`	função	`(check, case) -> float`	Valor de uma checagem num caso (`inf` quando não dá para calcular).
`flags`	função	`(check, case, tolerance=None) -> bool`	Diz se a checagem rejeita o caso, com tolerância opcional.
`check_library`	função	`(problem) -> dict[str, Any]`	Todas as checagens disponíveis para um problema, com tolerância ajustada à mão.
`default_checks`	função	`(problem) -> list`	As checagens que um usuário roda hoje (rodada 0 da arena).
`dataset_checks`	função	`(problem, case_checks=None) -> list`	Checagens de um conjunto de dados inteiro: quase-duplicatas, vazamento treino/teste e pior valor por amostra.

`pinneapple_analysis.validation.fault_injection.faults`

Biblioteca de falhas: defeitos controlados da saída de um modelo ou de um conjunto de dados, cada um com uma magnitude. [sem teste direto]

`pinneapple_analysis/validation/fault_injection/faults.py` · 248 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Fault`	classe	`(name, target, apply, magnitudes, documented, unit, description)`	Uma falha controlada de modelo ou de dados, com magnitudes e função `apply`.
`model_faults`	função	`(problem=None) -> list[Fault]`	Lista as falhas de saída de modelo, opcionalmente filtradas por problema.
`data_faults`	função	`(problem=None) -> list[Fault]`	Lista as falhas de conjunto de dados, opcionalmente filtradas por problema.
`get_fault`	função	`(name) -> Fault`	Devolve uma falha pelo nome.

`pinneapple_analysis.validation.fault_injection.matrix`

Matriz de detecção: cada checagem contra cada falha e magnitude e contra casos limpos, com taxas de detecção, falso alarme, AUC e magnitude mínima detectável. [sem teste direto]

`pinneapple_analysis/validation/fault_injection/matrix.py` · 282 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`DetectionMatrix`	classe	`(problem, level, tolerances, clean, scores, errors, faults, min_rate, trials)`	Valores das checagens em casos limpos e em cada falha e magnitude, com taxas de detecção, falso alarme e AUC.
`FaultInjectionReport`	classe	`(problem, model, data, settings)`	Relatório da injeção de falhas (nível modelo e nível dados), com poder das checagens e saída em Markdown.
`auc`	função	`(clean, faulty) -> float`	AUC ROC de um escore separando casos com falha de casos limpos (Mann-Whitney).
`fault_rng`	função	`(seed, fault, magnitude, case_seed) -> np.random.Generator`	Fluxo aleatório de uma injeção de falha, para reproduzir um teste isolado.
`run_fault_injection`	função	`(problem='darcy_2d', *, checks=None, data_checks=None, model_faults=None, data_faults=Non…)`	Monta a matriz de detecção das checagens contra a biblioteca de falhas num problema.

`pinneapple_analysis.validation.fault_injection.problems`

Problemas de benchmark com solver de referência verificado (Darcy 2D, Burgers 1D) como fonte de casos corretos; a base `BenchmarkProblem` tem ganchos que levantam NotImplementedError até a subclasse implementar. [sem teste direto]

`pinneapple_analysis/validation/fault_injection/problems.py` · 274 linhas · **sem teste direto** · palavras-chave no código (heurística; o texto acima diz o status real): levanta `NotImplementedError` em algum caminho

nome	tipo	assinatura	o que faz
`Case`	classe	`(problem, inputs, field, params, seed, variation, meta)`	Um par entrada/saída de um problema, reproduzível por semente e variação.

nome	tipo	assinatura	o que faz
`Dataset`	classe	`(train, test)`	Divisão de treino e teste de casos.
`BenchmarkProblem`	classe	`(**options)`	Base de problema com solver de referência verificado; subclasses implementam a solução e o espaço de variação.
`DarcyProblem`	classe	`()`	Problema de Darcy `-div(a grad u) = f` no quadrado unitário com `u = 0` na borda.
`BurgersProblem`	classe	`()`	Problema de Burgers viscoso 1D periódico.
`get_problem`	função	`(problem, **options) -> BenchmarkProblem`	Devolve um problema pelo nome (`darcy_2d`, `burgers_1d`) ou o próprio objeto.

pinneapple_analysis.validation.field_metrics

Métricas de erro no estilo PDEBench para campos em grade, só NumPy (RMSE, nRMSE, erro de contorno, conservado, por faixa de frequência, Pearson).

`pinneapple_analysis/validation/field_metrics.py` · 154 linhas · testes: 17

nome	tipo	assinatura	o que faz
`FieldErrorReport`	classe	`(rmse, nrmse, max_error, crmse, brmse, frmse_low, frmse_mid, frmse_high, bands, vrmse, pe…)`	Conjunto de métricas de erro de um campo em grade, serializável com `to_dict`.
`spectral_error_shells`	função	`(pred, target, ndim) -> np.ndarray`	Erro quadrático médio por casca de número de onda radial inteiro.
`field_error_metrics`	função	`(pred, target, *, ndim, bands=(4, 12), eps=1e-12) -> FieldErrorReport`	Calcula o `FieldErrorReport` de uma predição contra o alvo.

pinneapple_analysis.validation.gallery

Final comum dos exemplos de `examples/validation/`: relatório do modelo bom, trust card, saídas lidas por `pinneapple check` e uma falha real pega numa variante quebrada.

`pinneapple_analysis/validation/gallery.py` · 67 linhas · testes: 4

nome	tipo	assinatura	o que faz
`GalleryResult`	classe	`(name, family, physics, good, bad, caught, expected, files)`	Resultado de um exemplo da galeria: relatórios bom e ruim, se a falha foi pega e arquivos gravados.
`conclude`	função	`(name, good, bad, *, expect, family, physics, out_dir=None) -> GalleryResult`	Fecha um exemplo da galeria e levanta `AssertionError` se a checagem esperada não pegar a falha.

pinneapple_analysis.validation.metamorphic

Testes metamórficos para substitutos: relações que a física garante (simetria, invariância, monotonicidade, superposição, consistência de resolução), testáveis sem solução de referência; a base `Relation.violations` levanta NotImplementedError.

`pinneapple_analysis/validation/metamorphic.py` · 497 linhas · testes: 9 · palavras-chave no código (heurística; o texto acima diz o status real): levanta `NotImplementedError` em algum caminho

nome	tipo	assinatura	o que faz
`Relation`	classe	`(name, tol, output, atol, scale, per_sample, kind)`	Base de relação metamórfica; `violations` devolve a violação relativa por amostra e a evidência.
`Symmetry`	classe	`(transform_in, transform_out)`	Relação de simetria `f(T x) = S(x, f(x))`.
`Similarity`	classe	`(transform_in, dimensionless)`	Relação de semelhança: uma grandeza adimensional não muda sob a transformação da entrada.
`Invariance`	classe	`(transform_in)`	Relação de invariância `f(T x) = f(x)`.
`Monotonicity`	classe	`(input, direction, step, increase, bounds)`	A saída não diminui (ou não aumenta) quando uma entrada cresce.

nome	tipo	assinatura	o que faz
`Superposition`	classe	`(affine, homogeneity, linear_inputs, combine)`	Relação de superposição (aditividade e, opcionalmente, homogeneidade) de modelos lineares ou afins.
`ResolutionConsistency`	classe	`(restrict, quantity)`	Grandezas integrais devem ser iguais quando a entrada muda de resolução.
`RelationResult`	classe	`(name, kind, passed, worst, tol, n_tested, counterexample, search, note)`	Resultado de uma relação: aprovação, pior violação, contraexemplo e nota.
`MetamorphicReport`	classe	`(results)`	Relatório de todas as relações metamórficas testadas.
`box_sampler`	função	`(bounds, fixed=None, log=()) -> Sampler`	Amostrador uniforme (ou log-uniforme) numa caixa de entradas nomeadas.
`field_sampler`	função	`(make) -> Sampler`	Amostrador de lotes de campos a partir de um gerador de uma amostra.
`reflect_field`	função	`(axis) -> Callable[[np.ndarray], np.ndarray]`	Cria a reflexão de um lote de campos ao longo de um eixo espacial.
`rotate_field`	função	`(k=1, axes=(0, 1)) -> Callable[[np.ndarray], np.ndarray]`	Cria a rotação de um lote de campos por quartos de volta.
`check_relations`	função	`(model, relations, sampler, *, n=64, seed=0, search='random', max_examples=50) -> Metamor…)`	Roda cada relação em amostras aleatórias e, com `search='hypothesis'`, faz busca por propriedades (cai para busca aleatória se `hypothesis` faltar).
`relations_from_spec`	função	`(spec) -> list[Relation]`	Cria relações de um modelo tabular a partir de uma lista declarativa (TOML/JSON).

`pinneapple_analysis.validation.physics_learning`

Verifica se um modelo aprendeu a física ou só uma perda baixa: escolhe pontos informativos, checa resíduo discreto, respostas a intervenções e suficiência das entradas, e compara com as métricas usuais.

`pinneapple_analysis/validation/physics_learning.py` · 499 linhas · testes: 7

nome	tipo	assinatura	o que faz
`Finding`	classe	`(check, passed, value, limit, detail, provenance, points)`	Um achado da verificação: checagem, aprovação, valor, limite e detalhe.
`VerificationReport`	classe	`(model, findings, points, reasons, insufficient, trust)`	Relatório de verificação de aprendizado da física, com achados, pontos e confiança.
`select_points`	função	`(case, theta, k=48, seed=0, interventions=INTERVENTIONS, pool=3000) -> tuple[np.ndarray, …)`	Escolhe pontos informativos pela resposta da referência a cada intervenção e por cotas perto de interfaces, fonte e bordas.
`random_points`	função	`(case, k, seed=0) -> np.ndarray`	Sorteia pontos aleatórios do caso (base de comparação).
`fv_residual`	função	`(model, case, th, rows) -> np.ndarray`	Resíduo de volumes finitos dos valores do próprio modelo nas células dadas, relativo à escala de calor.
`structural_checks`	função	`(model, case, theta, X, probes=True) -> list[Finding]`	Checagens no campo do modelo que não precisam de referência: resíduo, interfaces, bordas e balanço global de calor.
`intervention_checks`	função	`(model, case, theta, X, interventions=INTERVENTIONS) -> list[Finding]`	Compara a resposta do modelo a cada intervenção com a resposta da referência.
`sufficiency`	função	`(model, case, theta, X) -> tuple[Finding, list[dict]]`	Aponta parâmetros físicos fora das entradas do modelo que mudam a temperatura de referência acima da tolerância.
`verify`	função	`(model, case=None, theta=None, *, k=48, seed=0, points=None, probes=True) -> Verification…)`	Roda todas as checagens do módulo num modelo e devolve o `VerificationReport`.
`baselines`	função	`(model, case=None, *, n_theta=12, n_points=400, seed=1) -> dict`	Calcula o que as métricas usuais dizem (erro L2 em pontos aleatórios e perda física média).
`detection_study`	função	`(severities=(0.25, 0.5, 1.0, 2.0), budgets=(4, 8, 16, 48), *, seed=0, extra_models=None) …)`	Estudo de detecção do verificador contra as métricas usuais, com modelos defeituosos e de controle.

pinneapple_analysis.validation.physics_learning_cases

Caso de verificação para `physics_learning`: seção transversal die/TIM/tampa aquecida na junção, com solver de referência, modelos de controle, modelos defeituosos, substituto POD+GP e PINN.

`pinneapple_analysis/validation/physics_learning_cases.py` · 359 linhas · testes: 7

nome	tipo	assinatura	o que faz
`Section`	classe	`(nx, nz, _cache)`	Grade e solver de referência da seção transversal die/TIM/tampa.
`Model`	classe	`(name, fn, inputs, defect, reinit)`	Modelo sob teste `fn(X, theta) -> T`, com entradas, defeito e cópia não treinada opcional.
`good_models`	função	`(sec, amplitudes=(0.003,), modes=(1,)) -> list[Model]`	Modelos de controle: a referência e a referência com erros suaves que respeitam contorno e simetria.
`defective_models`	função	`(sec, severity=1.0) -> list[Model]`	Modelos precisos pelas métricas usuais mas errados num aspecto físico, com severidade ajustável.
`training_distribution`	função	`(n, rng, *, correlated=True) -> list[dict]`	Amostras de parâmetros como as de treino, com resfriamento correlacionado à potência.
`surrogate_model`	função	`(sec, n_train=48, *, correlated=True, seed=0) -> Model`	Substituto POD + processo Gaussiano do campo, treinado em soluções de referência.
`pinn_model`	função	`(sec, *, hard_sides=False, adam_steps=3000, lbfgs_steps=500, n_collocation=3000, seed=0, …)`	PINN do caso nominal treinada com `train_pinn`, com uma saída por camada e condições de interface e contorno.

pinneapple_analysis.validation.pinn

Validação de PINN: resíduo em pontos nunca usados no treino, detectores dos modos de falha conhecidos, comparação com solução exata e um monitor durante o treino; traz problemas prontos (Burgers, Poisson, Kovasznay).

`pinneapple_analysis/validation/pinn.py` · 576 linhas · testes: 8

nome	tipo	assinatura	o que faz
`PINNProblem`	classe	`(name, bounds, residual, boundary, initial, exact, lhs, forcing, time_axis, scale, residu…)`	Descrição de um problema para `validate_pinn`: domínio, resíduo, contorno, condição inicial, solução exata e escalas.
`PINNReport`	classe	`(problem, checks, evidence)`	Relatório de validação de PINN com checagens, evidência e modos de falha detectados.
`PINNMonitor`	classe	`(problem, *, every=100, warmup=0, patience=2, stop_on=…, n_points=1024, grid=(32, 32), **…)`	Roda as checagens de `validate_pinn` a cada tantas épocas durante o treino, com parada opcional.
`d`	função	`(u, X, i, order=1)`	Derivada de ordem dada de `u` em relação a uma coordenada, por autograd.
`validate_pinn`	função	`(model, problem, *, n_points=4096, grid=None, train_points=None, tolerances=None, seed=0)…)`	Valida uma PINN treinada num problema e devolve o `PINNReport`.
`mlp`	função	`(n_in, n_out, width=32, depth=4)`	Cria uma MLP tanh, a rede usual de PINN.
`train_pinn`	função	`(problem, model=None, *, n_collocation=2000, n_condition=400, adam_steps=1500, lbfgs_step…)`	Treina uma PINN com restrições suaves (Adam e depois L-BFGS) e monitor opcional.
`cole_hopf`	função	`(x, t, nu, n=2001) -> np.ndarray`	Solução exata de Burgers viscoso com `u(x, 0) = -sin(pi x)` pela transformada de Cole-Hopf.
`burgers_1d`	função	`(nu=0.05 / math.pi) -> PINNProblem`	Problema de Burgers 1D (Raissi et al. 2019, viscosidade maior por padrão) com solução exata.
`poisson_2d`	função	`(kx=1, ky=2, exact=True) -> PINNProblem`	Problema de Poisson 2D manufaturado no quadrado unitário.
`kovasznay`	função	`(re=40.0) -> PINNProblem`	Escoamento de Kovasznay (Navier-Stokes estacionário) com saídas (u, v, p).

pinneapple_analysis.validation.report_artifact

O relatório de validação como artefato: esquema JSON versionado, hashes de proveniência, diferença entre dois relatórios, seção para model card do Hugging Face, badge e assinaturas.

pinneapple_analysis/validation/report_artifact.py · 341 linhas · testes: 26

nome	tipo	assinatura	o que faz
ReportDiff	classe	(old, new, lost, gained, changed, added, removed, moved, envelope, decision)	Diferença entre dois relatórios: checagens perdidas, ganhas, alteradas e decisão.
as_dict	função	(report) -> dict[str, Any]	Converte relatório (objeto, dict ou caminho JSON) em dict.
check_report	função	(report) -> list[str]	Lista os problemas de um relatório contra o esquema e o vocabulário de IDs; vazio se válido.
hash_model	função	(model) -> str | None	SHA-256 dos parâmetros de um modelo ou do arquivo ONNX; None para um callable simples.
hash_data	função	(*objs) -> str | None	SHA-256 dos dados usados num relatório.
diff	função	(old, new, *, rtol=0.1) -> ReportDiff	Calcula o que mudou entre dois relatórios de validação.
model_card_section	função	(report, *, title='Physics validation') -> str	Gera um bloco Markdown para model card do Hugging Face.
shields_endpoint	função	(report, label='physics-validated') -> dict[str, Any]	Gera o JSON de um badge de endpoint do shields.io.
badge	função	(report, label='physics-validated') -> str	Gera um badge SVG autocontido com o resultado das checagens.
sign_report	função	(report, key) -> dict[str, Any]	Assina o relatório com pinneapple_security.signing.sign_json.
verify_report	função	(envelope, key) -> dict[str, Any]	Verifica a assinatura de um envelope e devolve o relatório, levantando erro se não confere.

pinneapple_analysis.validation.rollout

Validação de rollout de substitutos autorregressivos: deriva de invariantes, espectro de energia ao longo do horizonte, tempo de divergência ('trust horizon') e estatísticas de longo prazo.

pinneapple_analysis/validation/rollout.py · 326 linhas · testes: 7

nome	tipo	assinatura	o que faz
RolloutReport	classe	(checks, evidence)	Relatório das checagens de rollout com evidência.
rollout	função	(step, u0, n_steps) -> np.ndarray	Gera a trajetória de um modelo autorregressivo, parando cedo se o estado deixar de ser finito.
invariant_series	função	(traj, ndim, invariants=('mass', 'energy')) -> dict[str, np.ndarray]	Séries temporais dos invariantes (massa, energia) de uma trajetória.
invariant_drift	função	(pred, ndim, ref=None, *, dt=1.0, invariants=('mass', 'energy'), window=1) -> dict[str, d…)	Taxa e deriva máxima de cada invariante, e desvio contra a referência se houver.
spectral_drift	função	(pred, ndim, ref=None, *, ref_spectrum=None, periodic=True, window=1, floor=1e-06) -> dic…)	Viés de energia em alto número de onda a cada horizonte, contra a referência.
divergence_time	função	(pred, ref, ndim, *, dt=1.0, err_tol=0.2, spec_tol=0.25, inv_tol=0.05, periodic=True, inv…)	Primeiro horizonte em que cada critério cruza a tolerância, e o 'trust horizon' (o mais cedo).
long_run_statistics	função	(pred, ref, ndim, *, dt=1.0, burn_in=0, periodic=True) -> dict[str, Any]	Compara medidas invariantes de uma corrida longa do modelo com uma corrida longa de referência.
validate_rollout	função	(pred, ndim, ref=None, *, dt=1.0, conserved=(), invariants=('mass', 'energy'), long_ref=N…)	Roda as checagens rollout.* de uma trajetória (aprova / falha / não aplicável).

pinneapple_analysis.validation.schema

Vocabulário do relatório de validação num só lugar: status, fatores e níveis de credibilidade e os IDs estáveis de checagens usados por pp.validate, trust card e pinneapple check.

pinneapple_analysis/validation/schema.py · 120 linhas · testes: 9

nome	tipo	assinatura	o que faz
CheckSpec	classe	(id, factor, title, rule, strength, critical, better)	Especificação de uma checagem: ID, fator de credibilidade, título, regra, força e criticidade.

nome	tipo	assinatura	o que faz
spec_of	função	(check_id) -> CheckSpec	Devolve a especificação de um ID de checagem; sub-checagens herdam da família.

pinneapple_analysis.validation.spectral

Fidelidade espectral de um campo previsto (só NumPy): espectro de energia, viés em alto número de onda, erro de fase, coerência, funções de estrutura e erros de gradiente.

pinneapple_analysis/validation/spectral.py · 369 linhas · testes: 9

nome	tipo	assinatura	o que faz
SpectralComparison	classe	(k, E_pred, E_ref, ratio, phase_error, coherence, bands, band_ratio, band_phase_error, ba…)	Comparação espectral: espectros, razão, erro de fase, coerência e viés por faixa.
StructureFunctions	classe	(r, S, S3_signed, flatness)	Funções de estrutura por defasagem, com flatness.
SpectralFidelityReport	classe	(rel_l2, spectrum, structure_pred, structure_ref, gradients, tol, checks)	Relatório de fidelidade espectral com L2 relativo, espectro, funções de estrutura e gradientes.
energy_spectrum	função	(u, ndim=None, *, periodic=True, window='hann', kmax=None) -> tuple[np.ndarray, np.ndarra…)	Espectro de energia por cascas de um campo.
fit_power_law	função	(k, E, kmin, kmax) -> tuple[float, float]	Ajusta uma lei de potência E = C k^slope num intervalo de k.
spectral_comparison	função	(pred, ref, ndim, *, periodic=True, window='hann', bands=(1 / 3, 2 / 3), floor=1e-10) -> …)	Compara os espectros de predição e referência.
high_wavenumber_bias	função	(pred, ref, ndim, **kw) -> float	Viés de energia na faixa alta de número de onda.
structure_functions	função	(u, ndim, *, orders=(2, 3), lags=None, periodic=True) -> StructureFunctions	Calcula funções de estrutura ao longo de cada eixo espacial.
gradient_metrics	função	(pred, ref, ndim, spacing=1.0, *, periodic=False, vector_axis=None, walls=(), conductivit…)	Erros RMS relativos de grandezas baseadas em gradiente.
spectral_fidelity	função	(pred, ref, ndim, *, periodic=True, window='hann', bands=(1 / 3, 2 / 3), tol=0.25, spacin…)	Fidelidade espectral: aprova quando o viés de alta frequência fica dentro da tolerância.

pinneapple_analysis.validation.symmetry

Checagens de simetria (reflexão e rotação 2D) de uma PINN treinada.

pinneapple_analysis/validation/symmetry.py · 280 linhas · testes: 151

nome	tipo	assinatura	o que faz
SymmetryCheck	classe	(device='cpu') -> None	Checa propriedades de simetria espacial de um modelo treinado.

pinneapple_analysis.validation.validate

pp.validate: uma chamada escolhe as checagens que se aplicam ao modelo e ao problema, roda-as e devolve um ValidationReport com evidência, credibilidade por fator e decisão de implantação, em níveis quick, standard e thorough.

pinneapple_analysis/validation/validate.py · 759 linhas · testes: 23

nome	tipo	assinatura	o que faz
ValidationCheck	classe	(id, status, summary, value, threshold, evidence)	Uma checagem do relatório: ID, status, resumo, valor, limiar e evidência.
ValidationReport	classe	(model, problem, level, checks, provenance)	Relatório de pp.validate com decisão, veredito, credibilidade e saídas JSON, HTML e trust card.
as_field_model	função	(model) -> Callable[[np.ndarray], np.ndarray]	Converte modelo torch, ONNX ou callable num operador de campo em NumPy.
validate	função	(model, problem=None, data=None, reference=None, level='standard', *, query=None, method=…)	Valida um modelo num problema no nível pedido e devolve o ValidationReport.

pinneapple_analysis.validation.validator

PhysicsValidator: orquestra uma suíte de checagens (conservação, contorno, simetria, personalizada) sobre um modelo treinado, inclusive a partir de um ProblemSpec.

pinneapple_analysis/validation/validator.py · 357 linhas · testes: 151

nome	tipo	assinatura	o que faz
PhysicsValidator	classe	(model, coord_names, domain_bounds, device='cpu') -> None	Orquestra uma suíte de checagens físicas sobre um modelo treinado.
validate_model	função	(model, spec, coord_names=None, domain_bounds=None, device='cpu') -> ValidationReport	Validação rápida de uma vez a partir de um ProblemSpec ou argumentos explícitos.

pinneapple_analysis.verification

Reexporta os blocos de verificação física trazidos do produto physics_verification_engine; o serviço HTTP sobre eles é apps/verified_physics. (A flag 'experimental' vem de uma citação ao guardrail experimental do PhysicsNeMo, não do status.) [sem teste direto do pacote; os submódulos têm testes]

pinneapple_analysis/verification/__init__.py · 366 linhas · **sem teste direto** · palavras-chave no código (heurística; o texto acima diz o status real): experimental

pinneapple_analysis.verification.architecture_critique

Revisão adversarial de um desenho de arquitetura de IA física por um LLM, contra uma lista fixa de modos de falha, com veredito e severidade por categoria.

pinneapple_analysis/verification/architecture_critique.py · 267 linhas · testes: 12

nome	tipo	assinatura	o que faz
CritiqueFinding	classe	(category, verdict, severity, reasoning)	Um achado da revisão: categoria, veredito, severidade e justificativa.
AdversarialReviewReport	classe	(architecture_description, findings, overall_reasoning, provider, raw_llm_response)	Relatório da revisão adversarial, com achados, justificativa geral e resposta bruta do LLM.
run_adversarial_review	função	(architecture_description, spec=None, *, provider='ollama', model=None, api_key=None, con…)	Roda a revisão adversarial de uma descrição de arquitetura contra a lista fixa de modos de falha, usando um provedor de LLM.

pinneapple_analysis.verification.architecture_recommendation

Recomenda qual família de arquitetura neural do PINNeAPPle usar primeiro, a partir de disponibilidade de dados, solver, generalização por parâmetro/geometria e problema direto ou inverso; nunca afirma acurácia, só adequação ao escopo documentado.

pinneapple_analysis/verification/architecture_recommendation.py · 453 linhas · testes: 19

nome	tipo	assinatura	o que faz
ArchitectureCandidate	classe	(name, registry_key, category, when_to_use, strengths, weaknesses, source, implementation…)	Uma família de arquitetura e quando seu propósito documentado se aplica, com forças, fraquezas e fonte.
ArchitectureRecommendation	classe	(recommended, reasoning, decision_path, alternatives, caveats)	Resultado da recomendação, com caminho de decisão auditável, alternativas e ressalvas.
recommend_architecture	função	(*, n_high_fidelity_simulations=0, has_lots_of_data=None, has_analytical_solver=False, ne…)	Recomenda a família de arquitetura a usar primeiro a partir das características reais do problema.

pinneapple_analysis.verification.causal_discrepancy

Dois itens do roadmap: `run_intervention` reexecuta o pipeline com um parâmetro alterado e relata variações percentuais reais; `fit_discrepancy_model` ajusta uma correção de discrepância de forma (Kennedy & O'Hagan 2001) por uma MLP. Cada intervenção retreina uma PINN do zero.

`pinneapple_analysis/verification/causal_discrepancy.py` · 483 linhas · testes: 5

nome	tipo	assinatura	o que faz
`InterventionRunResult`	classe	`(param_value, final_loss, loss_pct_change, residual_value, residual_pct_change, guardrail…)`	Resultado de uma rodada de intervenção: valor do parâmetro, perda, resíduo, confiabilidade do guardrail e variações percentuais.
`InterventionReport`	classe	`(base_description, param_name, baseline_rejected, rejection_reason, query_point, baseline…)`	Relatório da intervenção: rodada de base, rodadas perturbadas e motivo de rejeição se a base não for confiável.
`DiscrepancyModel`	classe	`(physics_model, correction_model, in_dim, out_dim, n_train, n_test, variance_explained_te…)`	Modelo `y(x) ~= physics_model(x) + delta(x)` com a correção `delta` dada por uma MLP.
`run_intervention`	função	`(base_description, param_name, param_values, *, physical_parameters=None, provider='ollam…)`	Roda o pipeline como base e de novo para cada valor do parâmetro, relatando variações percentuais reais.
`fit_discrepancy_model`	função	`(spec, physics_model, reference_x, reference_y, *, hidden_dim=32, n_layers=2, epochs=400,…)`	Ajusta uma MLP de discrepância sobre o resíduo real entre referência e modelo físico.

pinneapple_analysis.verification.convergence

Automação de estudo de convergência de malha: extrapolação de Richardson e Grid Convergence Index de Roache com três malhas, e um driver genérico que chama `solve_fn(resolução)`.

`pinneapple_analysis/verification/convergence.py` · 477 linhas · testes: 43

nome	tipo	assinatura	o que faz
`ConvergenceResult`	classe	`(observed_order, extrapolated_value, gci_fine, gci_coarse, asymptotic_ratio, is_asymptoti…)`	Resultado de extrapolação de Richardson com três malhas: ordem observada, valor extrapolado, GCI e checagem de regime assintótico.
`MeshIndependenceStudyResult`	classe	`(resolutions, values, refinement_ratios, refinement_ratio_used, convergence)`	Estudo completo: valores por resolução e o `ConvergenceResult` das três malhas mais finas.
`richardson_extrapolate`	função	`(f_coarse, f_medium, f_fine, r, *, p_assumed=None, safety_factor=_DEFAULT_SAFETY_FACTOR, …)`	Extrapola por Richardson uma grandeza em três resoluções e calcula o GCI de Roache (1994).
`mesh_independence_study`	função	`(solve_fn, resolutions, quantity_extractor, *, refinement_ratio=None, p_assumed=None, saf…)`	Chama um solver qualquer em resoluções crescentes e extrapola as três mais finas.

pinneapple_analysis.verification.dimensional_analysis

Calcula números adimensionais por definições de livro (Reynolds, Prandtl, Péclet, Mach, Froude, Weber, Damköhler, correlação de Nusselt) e classifica o regime de escoamento conforme a geometria indicada.

`pinneapple_analysis/verification/dimensional_analysis.py` · 175 linhas · testes: 27

nome	tipo	assinatura	o que faz
`DimensionlessNumbers`	classe	`(reynolds, prandtl, peclet_heat, peclet_mass, mach, froude, weber, damkohler, nusselt_cor…)`	Os números adimensionais calculáveis com os dados fornecidos; campos ficam `None` quando faltam entradas.
`compute_dimensionless_numbers`	função	`(*, velocity=None, length=None, density=None, dynamic_viscosity=None, kinematic_viscosity…)`	Calcula todos os números adimensionais que o subconjunto de parâmetros permite.
`classify_flow_regime`	função	`(re, *, geometry='internal_pipe') -> str`	Classifica o regime de escoamento pelo Reynolds com limiares próprios da geometria informada.

pinneapple_analysis.verification.evidence_graph

Grafo de evidências de uma execução (networkx): liga o registro de proveniência ao escore de confiança e às comparações entre solvers, e explica em texto por que confiar.

pinneapple_analysis/verification/evidence_graph.py · 235 linhas · testes: 8

nome	tipo	assinatura	o que faz
build_evidence_graph	função	(provenance, confidence=None, comparisons=None) -> 'nx.DiGraph'	Monta o grafo de evidências de uma execução.
evidence_summary	função	(g) -> Dict[str, Any]	Resume o grafo: contagem de evidências por tipo e por apoio/contradição, sem recalcular a confiabilidade.
explain_trust	função	(g) -> str	Gera um texto 'por que confiar' a partir das evidências reais do grafo.

pinneapple_analysis.verification.forecast_skill

Skill de previsão contra uma previsão de referência, com intervalos por bootstrap em blocos móveis e médias em lotes que respeitam a correlação serial.

pinneapple_analysis/verification/forecast_skill.py · 96 linhas · testes: 19

nome	tipo	assinatura	o que faz
SkillResult	classe	(leads, diff, skill_score, horizon, n_starts, block)	Resultado de skill por antecedência: diferenças, escore de skill, horizonte e tamanho de bloco.
block_bootstrap_indices	função	(n, block, n_boot, rng) -> np.ndarray	Índices reamostrados por blocos móveis de amostras consecutivas.
block_bootstrap_mean	função	(d, block, n_boot=2000, seed=0, level=0.95) -> tuple[float, float, float]	Média e intervalo por bootstrap em blocos móveis.
block_bootstrap_ratio	função	(a, b, block, n_boot=2000, seed=0, level=0.95) -> tuple[float, float, float]	Escore de skill 1 - mean(a)/mean(b) e seu intervalo por bootstrap pareado em blocos.
batch_means_ci	função	(x, n_batches=8, level=0.95) -> tuple[float, float]	Média de uma série correlacionada e meia-largura do intervalo por médias em lotes (t de Student).
skill_vs_lead	função	(err_model, err_ref, leads, block, *, n_boot=2000, level=0.95, seed=0) -> SkillResult	Compara duas previsões pareadas por antecedência e calcula o horizonte em que o modelo é melhor com a confiança pedida.

pinneapple_analysis.verification.geometry_intelligence

Rotulagem semântica de regiões de uma malha de superfície (entrada, saída, parede) por segmentação geométrica por normais, com desambiguação opcional por LLM, e sugestão de tipos de condição de contorno.

pinneapple_analysis/verification/geometry_intelligence.py · 756 linhas · testes: 9

nome	tipo	assinatura	o que faz
BoundaryLoop	classe	(n_vertices, closed, perimeter, is_planar, planarity_rms_deviation, enclosed_area, circul…)	Um laço de borda fechado de um patch de superfície, com perímetro, planaridade, área e circularidade.
SurfaceRegion	classe	(region_id, face_ids, n_faces, area, area_fraction_of_total_surface, area_fraction_of_lar…)	Um patch conexo da segmentação por normais, com as propriedades geométricas usadas pelo classificador.
GeometryClassification	classe	(regions, semantic_labels, boundary_condition_types, ambiguous_region_ids, llm_used, llm_…)	Resultado da classificação: regiões, rótulos semânticos, tipos de condição de contorno e regiões ambíguas.
classify_geometry	função	(mesh, problem_description='', *, provider=None, model=None, api_key=None, angle_toleranc…)	Classifica cada superfície de borda da malha numa região com sentido físico, com etapa opcional de LLM.

pinneapple_analysis.verification.geometry_ood_guardrail

Sinaliza quando uma geometria de entrada está fora da distribuição das geometrias de referência, por distância de Mahalanobis sobre descritores de forma. (A flag 'experimental' vem da citação ao PhysicsNeMo.)

pinneapple_analysis/verification/geometry_ood_guardrail.py · 456 linhas · testes: 39 · palavras-chave no código (heurística; o texto acima diz o status real): experimental

nome	tipo	assinatura	o que faz
GeometryOODResult	classe	(score, mahalanobis_sq, dof, status, n_reference, feature_values, feature_z_scores, top_d…)	Saída de uma consulta: escore, Mahalanobis ao quadrado, status e descritores que mais desviam.
GeometryOODGuardrail	classe	(*, warn_p=0.05, reject_p=0.01)	Ajusta uma distribuição Gaussiana por descritor sobre geometrias de referência e pontua geometrias novas.
extract_geometry_features	função	(mesh) -> Dict[str, float]	Extrai de uma malha os descritores de forma listados em FEATURE_NAMES.
default_reference_meshes	função	() -> List[MeshData]	Catálogo genérico de malhas de referência (caixa, cilindro, canal) gerado por pinneapple_design.

pinneapple_analysis.verification.knowledge_graph

Grafo de conhecimento físico (networkx) construído na hora a partir do registro de presets e dos arquivos de teste de validação (lidos por AST): fenômeno, equação, parâmetros, preset, referência, método de verificação. (As flags vêm de palavras no texto, não do status.)

pinneapple_analysis/verification/knowledge_graph.py · 644 linhas · testes: 14 · palavras-chave no código (heurística; o texto acima diz o status real): experimental; placeholder/stub citado no docstring

nome	tipo	assinatura	o que faz
build_knowledge_graph	função	() -> 'nx.MultiDiGraph'	Constrói o grafo de conhecimento a partir do registro de presets e dos testes de validação atuais.
find_presets_for_phenomenon	função	(g, phenomenon) -> List[str]	Lista os presets cuja equação pertence a um fenômeno.
find_verified_equations	função	(g) -> List[str]	Lista os tipos de EDP com ao menos uma aresta de verificação confirmada por AST.
explain_preset	função	(g, preset_name) -> Dict[str, Any]	Resumo estruturado de um preset (equação, fenômeno, parâmetros, citações, verificação) com texto pronto para contexto de LLM.
graph_stats	função	(g) -> Dict[str, Any]	Contagens de auditoria do grafo, incluindo frações de presets com citação e com verificação.

pinneapple_analysis.verification.mesh_intelligence

Avalia se uma malha triangular de superfície serve para a física pedida: razão de aspecto e assimetria por triângulo, estimativa da altura da primeira célula por y+ e relatório do que falhou; ortogonalidade é declarada não calculável em malha de superfície.

pinneapple_analysis/verification/mesh_intelligence.py · 475 linhas · testes: 14

nome	tipo	assinatura	o que faz
TriangleQualityStats	classe	(aspect_ratio_convention, aspect_ratio, aspect_ratio_mean, aspect_ratio_max, worst_aspect…)	Métricas de qualidade por triângulo e suas estatísticas resumo.
OrthogonalityResult	classe	(computable, reason)	Indica que a ortogonalidade não é calculável numa malha de superfície e por quê.
YPlusEstimate	classe	(computable, reason, target_yplus, reynolds_length, skin_friction_coefficient, wall_shear…)	Estimativa da altura da primeira célula para um y+ alvo, com campos None quando faltam entradas.
MeshQualityReport	classe	(n_faces, n_vertices, triangle_quality, orthogonality, yplus, is_good_enough, failures, t…)	Relatório de qualidade da malha: métricas, estimativa de y+, se é boa o bastante e o que falhou.
estimate_required_first_cell_height	função	(*, reynolds_length, velocity, kinematic_viscosity, target_yplus=1.0) -> YPlusEstimate	Estima a altura da primeira célula para atingir o y+ alvo numa camada limite turbulenta de placa plana.
assess_mesh_quality	função	(mesh, *, aspect_ratio_threshold=100.0, skewness_threshold=0.95, max_bad_element_fraction…)	Avalia se a malha é boa o bastante e devolve um relatório estruturado.

pinneapple_analysis.verification.nozzle_crosscheck

Triagem de bocal de foguete por dois métodos sem código comum: uma PINN do preset `rocket_nozzle_cfd` e o bocal isentrópico ideal 1D, com a discordância entre eles.

`pinneapple_analysis/verification/nozzle_crosscheck.py` · 146 linhas · testes: 3

nome	tipo	assinatura	o que faz
`scaled_field_head`	função	`(net, scale, offset)`	Embrulha uma rede para que `y = net(x) * scale + offset` por campo de saída.
`train_nozzle_pinn`	função	`(*, throat_radius, exit_radius, nozzle_length, p_chamber, T_chamber, gamma, R_gas, epochs…)`	Treina a PINN do preset `rocket_nozzle_cfd`, roda o guardrail e devolve as médias no plano de saída.
`screen_nozzle`	função	`(*, throat_radius, exit_radius, p_chamber, T_chamber, gamma=1.2, R_gas=287.0, nozzle_leng…)`	Estimativa do bocal ideal e, com `run_pinn`, a estimativa da PINN e a discordância entre as duas.

pinneapple_analysis.verification.physics_confidence_score

`PhysicsConfidenceScore`: escore agregado e transparente (média simples) só das checagens já executadas que o chamador passa, sempre com a fração de cobertura.

`pinneapple_analysis/verification/physics_confidence_score.py` · 777 linhas · testes: 36

nome	tipo	assinatura	o que faz
`CalibrationSummary`	classe	`(ece, coverage, target_coverage, sharpness)`	Números de calibração de UQ já calculados (ECE, cobertura, nitidez).
`ConfidenceComponent`	classe	`(name, score, source_summary)`	Uma checagem já executada convertida em escore de 0 a 1 por fórmula documentada.
`PhysicsConfidenceScore`	classe	`(overall_score, components, coverage)`	Escore agregado (média simples dos componentes) com a cobertura das checagens executadas.
`compute_physics_confidence`	função	`(*, guardrail_report=None, convergence_result=None, calibration_metrics=None, benchmark_c…)`	Agrega as checagens fornecidas num `PhysicsConfidenceScore`.
`score_against_reference`	função	`(name, x, y_ref, preds, *, x_vars, y_vars, sigma=None, convergence=None, guardrail=None, …)`	Pontua as predições de vários modelos contra um conjunto de referência separado.

pinneapple_analysis.verification.pipeline

Pipeline descrição do problema -> análise dimensional -> preset rascunhado por LLM -> treino da PINN -> guardrail -> seleção do melhor candidato; bloqueante, o treino acontece dentro da chamada.

`pinneapple_analysis/verification/pipeline.py` · 166 linhas · testes: 6

nome	tipo	assinatura	o que faz
`CandidateResult`	classe	`(architecture, final_loss, guardrail_trustworthy, guardrail_checks, residual_value, model)`	Um candidato treinado: arquitetura, perda final, resultado do guardrail, resíduo e modelo.
`AnalysisResult`	classe	`(provenance, best_candidate, all_candidates, rejected, rejection_reason)`	Resultado do pipeline: proveniência, melhor candidato, todos os candidatos e rejeição.
`rank_key`	função	`(candidate) -> tuple`	Chave de ordenação: não confiável primeiro descartado, depois resíduo recalculado e perda de treino.
`guardrail_checks_as_dicts`	função	`(report) -> list[dict[str, Any]]`	Converte as checagens de um `GuardrailReport` em dicts simples.
`analyze`	função	`(description, *, physical_parameters=None, flow_geometry='internal_pipe', provider='ollam…)`	Roda o pipeline completo para uma descrição de problema.

pinneapple_analysis.verification.provenance

Registro estruturado de proveniência de uma execução (descrição, adimensionais, preset rascunhado, arquitetura etc.), com gravar e carregar em JSON. (A flag 'placeholder' vem do texto 'not a placeholder'.)

pinneapple_analysis/verification/provenance.py · 67 linhas · testes: 8 · palavras-chave no código (heurística; o texto acima diz o status real): placeholder/stub citado no docstring

nome	tipo	assinatura	o que faz
ProvenanceRecord	classe	(run_id, created_unix, problem_description, dimensionless_numbers, flow_regime, drafted_p…)	Registro de proveniência de uma execução, com to_dict, save e load.

pinneapple_analysis.verification.solver_orchestration

Orquestração entre famílias de solvers do PINNeAPPle: lista o que está disponível na máquina, recomenda uma família para um ProblemSpec e compara dois resultados nos mesmos pontos.

pinneapple_analysis/verification/solver_orchestration.py · 729 linhas · testes: 19

nome	tipo	assinatura	o que faz
SolverRecommendation	classe	(recommended_family, reasoning, matched_rule, fallback_families, caveats)	Resultado de select_solver_family: família recomendada, regra usada, alternativas e ressalvas.
ComparisonReport	classe	(label_a, label_b, n_points, max_abs_diff, mean_abs_diff, rmse, relative_rmse, worst_inde…)	Comparação numérica entre dois resultados nos mesmos pontos (diferença máxima, média, RMSE).
list_available_solver_families	função	() -> Dict[str, dict]	Verifica na hora quais famílias de solver este processo consegue usar na máquina.
select_solver_family	função	(spec, dimensionless_numbers=None, *, catalog=None) -> SolverRecommendation	Recomenda uma família de solver disponível para o ProblemSpec e explica por quê.
compare_solvers	função	(spec, model_or_result_a, model_or_result_b, *, query_points=None, labels=('solver_a', 's…)	Compara dois resultados do mesmo ProblemSpec nos mesmos pontos.

pinneapple_analysis.verification.tool_recommendation

Recomenda ferramentas externas (CFD, FEM, multifísica, malha, visualização, UQ) cujo escopo documentado cobre o problema, com catálogo inicial de ferramentas, empresas e orientação comprar-ou-construir.

pinneapple_analysis/verification/tool_recommendation.py · 601 linhas · testes: 18

nome	tipo	assinatura	o que faz
ExternalTool	classe	(name, category, license, typical_cost_tier, pde_kinds, dims, strengths, caveats, source)	Uma ferramenta externa nomeada e seu escopo documentado.
ToolRecommendation	classe	(recommended, reasoning, alternatives, caveats)	Resultado de recommend_tools: recomendadas, justificativa, alternativas e ressalvas.
Player	classe	(name, focus_areas, tool_keys, notes)	Uma empresa ou ecossistema e suas áreas de foco, ligado às ferramentas do catálogo.
recommend_tools	função	(spec, dimensionless_numbers=None, *, catalog=None, category=None) -> ToolRecommendation	Recomenda ferramentas externas cujo escopo documentado cobre o ProblemSpec.
get_tools_by_category	função	(category, *, catalog=None) -> List[str]	Lista as ferramentas do catálogo numa categoria (ex.: malha, visualização, UQ).
get_player_for_tool	função	(tool_key, *, player_catalog=None) -> Optional[str]	Diz qual empresa faz a ferramenta, ou None para projetos comunitários.
buy_vs_build_recommendation	função	(need, *, guidance=None) -> Optional[str]	Busca a orientação qualitativa comprar-ou-construir para uma necessidade; None se não coberta.
list_buy_vs_build_needs	função	(*, guidance=None) -> List[str]	Lista as necessidades cobertas pela tabela comprar-ou-construir.

pinneapple_analysis.verification.uncertainty_inverse

Liga as classes reais de UQ e de problemas inversos ao pipeline de verificação: relatório de incerteza por tipo e estimativa de parâmetros; o inverso só suporta heat_equation_steady com o parâmetro k, e outros casos levantam NotImplementedError.

pinneapple_analysis/verification/uncertainty_inverse.py · 752 linhas · testes: 13 · palavras-chave no código (heurística; o texto acima diz o status real): levanta NotImplementedError em algum caminho

nome	tipo	assinatura	o que faz
`InverseParamAdapter`	classe	`(base, unknown_param_names, initial_guess=None) -> None`	Dá a qualquer modelo um `nn.ParameterDict` `inverse_params`, como `InverseProblemSolver` exige.
`UncertaintyReport`	classe	`(epistemic_std, aleatoric_std, parameter_std, total_std, method, n_samples, metadata)`	Relatório de incerteza separado por tipo (epistêmica, aleatória, de parâmetro, total).
`InverseProblemResult`	classe	`(estimated_params, parameter_uncertainty, converged, final_misfit, method, n_iters, loss_…)`	Resultado da estimativa de parâmetros: valores, incerteza, convergência, desajuste e histórico.
`quantify_uncertainty`	função	`(spec, model, *, method='ensemble', n_samples=20, models=None, x_eval=None, n_eval=200, s…)`	Quantifica a incerteza preditiva de um modelo treinado com as classes reais de `pinneapple_analysis.uncertainty`.
`solve_inverse_problem`	função	`(spec, observed_data, unknown_param_names, *, model=None, initial_guess=None, n_outer=5, …)`	Estima parâmetros a partir de dados esparsos com `InverseProblemSolver` num esquema alternado campo/parâmetro.

Exemplos de uso reais do repositório

Correntes parasitas: varredura sintética e treino da PINN — `examples/inspection/01_eddy_current_pinn.py`

```
data = generate_eddy_current_synthetic(
    nr=24, nz=40, scan_positions=np.linspace(-0.01, 0.01, 9),
)
...
out = train_eddy_current(
    synthetic_data=data,
    hidden=[64, 64, 64],
    epochs=60,
    lr=1e-3,
    batch_size=32,
    log_dir="examples/_out/inspection_eddy_current",
    run_name="eddy_current_demo",
)
```

Problema inverso a partir de um preset — `examples/quickstart/05_inverse_problem.py`

```
import pinneapple_analysis.inverse_problems as pinv

preset = pinv.get_preset("fin_h")
built = preset.build()
problem = pinv.InverseProblem(
    model=built["model"], observations=built["observations"],
    parameters=built["parameters"], validation=built["validation"], name="fin_h",
)
result = pinv.solve_inverse(problem, "least_squares")
comparison = pinv.compare_methods(problem, methods=["least_squares", "bayesian"])
print("best:", comparison["best"])
```

pp.validate num FNO exportado em ONNX (Darcy 2D) — `examples/validation/02_fno_darcy_onnx.py`

```
data = {"train": {"a": a[:n_train], "u": u[:n_train]}, "test": {"a": a[n_train:], "u": u[n_train:]}}
good = pp.validate(str(path), "darcy_2d", data=data, level="standard")
shifted, _ = darcy_set(8, n_grid, seed=7, high=100.0, low=1.0)
bad = pp.validate(str(path), "darcy_2d", data={"train": data["train"]}, query=shifted, level="standard")
return conclude("02_fno_darcy_onnx", good, bad, expect=CATCHES, family=FAMILY, physics=PHYSICS, out_dir=out_dir)
```

Números adimensionais conferidos com cálculo à mão — `tests/pinneapple_analysis/test_verification_dimensional_analysis.py`

```
def test_reynolds_number_water_in_pipe_matches_hand_calculation():
    # rho=998 kg/m3, U=2 m/s, L=0.05 m (pipe diameter), mu=1.002e-3 Pa.s
    # Re = rho*U*L/mu = 998*2*0.05/1.002e-3 = 99600.8 (hand-computed)
```

```
    dn = compute_dimensionless_numbers(
        velocity=2.0, length=0.05, density=998.0, dynamic_viscosity=1.002e-3,
    )
    assert dn.reynolds == pytest.approx(99600.8, rel=1e-3)
```

Benchmark com orçamento igual: PINN, FNO e DeepONet em Burgers — `examples/benchmark/01_equal_budget.py`

```
import pinneapple as pp

full = "--full" in sys.argv
res = pp.benchmark.run(
    problems=["burgers"],
    methods=["pinn", "fno", "deeponet"],
    budget=pp.ComputeBudget(wall_s=300),          # per training run, one core; or flops=..., steps=...
    budget_levels=[1 / 9, 1 / 3, 1.0],            # error-versus-budget curve
    seeds=3, n_trials=4, search_fraction=1 / 3,   # the same search effort for every method
    equal_flops=True,                             # the extra equal-FLOPs arm
    n_workers=4, threads_per_run=1,
    smoke=not full,
    out_dir="results/benchmark_burgers" + ("" if full else "_smoke"),
)
print(res.table().to_string())
```

Grafo de prova do tetrâmero PT — `examples/proof_graph/pt_chain_no_go.py`

```
from pinneapple_analysis.proof.pt_chains import build_pt_chain_graph

g = build_pt_chain_graph(max_deg=4, n_numeric=300, search_iters=40)
status = g.run()
print(g.report())
print("\nMermaid:\n" + g.to_mermaid())
```

Testes e status de validação

- Módulos com pelo menos um teste que os importa: 103 de 159.
- Funções de teste em arquivos que importam algum módulo do pacote: 939.

Limitações conhecidas

- `inspection`: todos os sinais são sintéticos e ilustrativos; as simplificações (EMAT escalar, Klein-Gordon no lugar de Rayleigh-Lamb, varredura axissimétrica em correntes parasitas, MFL sem termo cruzado, Helmholtz como regularizador no phased array) estão declaradas nos docstrings. Os submódulos não têm teste direto; só o pacote é importado por `tests/test_inspection.py`.
- `inverse_problems`: convivem duas camadas, o kit PyTorch antigo (`InverseProblemSolver`, misfits, regularizadores, EKI) e o núcleo novo em NumPy/SciPy (`forward`, `estimation`, `identifiability`, `calibration`, `studio`, `presets`); vários módulos do núcleo novo não têm teste direto.
- `inverse_problems.heat_transfer`: o caso 3D não é treinado sob demanda, é lido de resultados gravados; o docstring de `obs_operator` cita um `BoundaryObsOperator` inexistente.
- `validation`: há duas classes `ValidationReport` diferentes (`core.ValidationReport` do validador clássico e `validate.ValidationReport` de `pp.validate`).
- `validation.arena` e `validation.fault_injection`: só os pacotes são importados por testes; os submódulos (blue, red, loop, matrix, faults etc.) não têm teste direto, e só Darcy 2D e Burgers 1D vêm prontos.
- `validation.metamorphic`: a busca por propriedades usa `hypothesis` se instalado; senão cai para busca aleatória.
- `verification`: vários módulos (`pipeline`, `causal_discrepancy`, `architecture_critique`, `geometry_intelligence`) dependem de um provedor de LLM (`ollama` por padrão, servidor local); `evidence_graph` e `knowledge_graph` exigem `networkx`.
- `verification.uncertainty_inverse`: `solve_inverse_problem` só suporta `heat_equation_steady` com o parâmetro `k`; o resto levanta NotImplementedError.

- `verification.nozzle_crosscheck`: o README de `apps/nozzle_screen` avisa que a PINN e o bocal ideal discordam hoje, porque o resíduo do preset está em unidades SI brutas; a metade PINN é uma demonstração.
- `benchmark.builtin.airfoil` precisa do OpenFOAM para `prepare` (`/usr/share/openfoam/etc/bashrc` ou `FOAM_BASHRC`); `airfoil` e `inverse_heat` só são registrados se suas dependências opcionais importarem.
- Vários módulos do benchmark (`advisor`, `api`, `runner`, `checks`, `config`, `registry`, `airfoil*`, `inverse_heat*`) não têm teste que os importe diretamente; a cobertura vem pelo `__init__` e pela CLI.
- Os ganchos das classes base `Problem`, `Method`, `FieldOperatorProblem` e dos candidatos de surrogate levantam NotImplementedError até serem implementados; `SurrogateModel.export` só funciona para os candidatos PyTorch (mlp, fno, deeponet, pinn).
- `retrieval` exige o extra `retrieval` (sentence-transformers, modelo local padrão `all-MiniLM-L6-v2`) e, com `include_literature=True`, rede para o arXiv; o `__init__` de `retrieval` não tem teste direto.
- `uncertainty.reachability` é marcado RESEARCH ONLY: emite ResearchOnlyWarning e é recusado com `PINNEAPPLE_COMMERCIAL_MODE=1`; modela espaço livre, sem gravidade.
- `trust.trust_report` é marcado experimental e tem API em português (`decisao`, `motivos`, `checagem_*`); as regras de decisão são escolhas de projeto, não normas.
- `proof.pt_chains` não prova o teorema por completo: o status fica ARGUED, e cadeias com N >= 5 seguem como conjectura apoiada só por busca de contraexemplos.
- A garantia conformal só vale com calibração suficiente (`n >= ceil(1/alpha) - 1`); com menos pontos o quantil é cortado no maior escore e a garantia não se aplica.

2. `pinneapple_check`

Objetivo

`pinneapple_check` é um linter de física para CI. Ele falha um pull request quando um resultado de simulação ou um modelo de Physics-AI viola a física: conservação, unidades, condições de contorno e iniciais, resíduo da EDP, limites, uso fora da distribuição de treino, regressão contra uma referência gravada e integridade dos artefatos. Cada violação tem um id de regra estável (`PC101`…), severidade, explicação e dica de correção.

O uso é pela linha de comando `pinneapple check [paths]` (ou `python -m pinneapple_check`), pela GitHub Action (`action.yml` na raiz do repositório) ou por Python com `pinneapple_check.run`. Funciona sem configuração (descobre casos OpenFOAM e arquivos de campo e infere a física pelos nomes) ou com `pinneapple-check.toml`. A saída sai em texto, JSON, SARIF, JUnit, anotações do GitHub ou resumo Markdown, e uma baseline permite adotar o checador num repositório que já tem violações.

O pacote importa só NumPy na partida e reaproveita módulos de outros pacotes sem executar seus `__init__` (`_reuse`): leitores OpenFOAM e de unidades de `pinneapple_data`, o envelope de treino de `pinneapple_physics.physics_audit`, métricas de `pinneapple_analysis`, o catálogo de métodos (`pinneapple_catalog`, regra PC901), `pinneapple_security` para hashes, assinaturas e varredura de checkpoints, e os estágios do trust card de `pinneapple_lab`. O relatório vira evidência para o trust card e para o mapa de aplicabilidade de `pinneapple_veriphysics`. O mesmo `cli.main` serve de despachante do comando `pinneapple` (doctor, recommend, card, new, design, benchmark).

Números do inventário

módulos	linhas	classes públicas	funções públicas	testes em arquivos que importam o pacote	módulos sem teste direto
23	4.547	22	71	144 (12 arquivos)	11

Submódulos

submódulo	o que faz	módulos	classes + funções	testes*
`__main__`	Entrada `python -m pinneapple_check`.	1	0	0
`_reuse`	Importa módulos isolados de outros pacotes PINNeAPPle sem seus `__init__`.	1	14	0
`balances`	Balanços globais de massa, calor e força em grades tensoriais.	1	3	0
`baseline`	Baseline para falhar só em violações novas.	1	3	0
`checks`	As checagens de resultados, modelos, datasets e integridade.	6	15	22

submódulo	o que faz	módulos	classes + funções	testes*
`cli`	Linha de comando `pinneapple check` e despachante `pinneapple`.	1	2	54
`config`	Leitura de `pinneapple-check.toml` e descoberta de alvos.	1	10	5
`engine`	Orquestra configuração, alvos e resultados num `Report`.	1	1	5
`evidence`	Trust card e checagens de salvaguarda para o veriphysics.	1	2	22
`fields`	Leitura de resultados em `FieldData`.	1	12	16
`fv`	Operadores e resíduos de volumes finitos em malhas OpenFOAM.	1	4	16
`models`	Carga segura de modelos como funções.	1	5	22
`numerics`	Derivadas em grade, nuvem de pontos e modelo.	1	5	5
`physics`	Equações governantes e tabela de parâmetros.	1	4	5
`report`	Formatos de saída (texto, JSON, SARIF, JUnit, GitHub, resumo).	1	7	22
`result`	`Outcome` e `Report`.	1	3	0
`rules`	Tabela de regras PC com seleção por prefixo.	1	3	0

* maior número de testes que importam um módulo do submódulo.

Módulos e API pública

`pinneapple_check`

Ponto de entrada do `pinneapple check`, um linter de física para CI: reexporta `run` e as classes de relatório; importa só NumPy na partida.

`pinneapple_check/__init__.py` · 39 linhas · testes: 74

`pinneapple_check.__main__`

Permite `python -m pinneapple_check [paths]`, equivalente a `pinneapple check [paths]`. [sem teste direto]

`pinneapple_check/__main__.py` · 8 linhas · **sem teste direto**

`pinneapple_check._reuse`

Carrega módulos isolados de outros pacotes (leitores OpenFOAM e de unidades do `pinneapple_data`, envelope do `physics_audit`, métricas do `pinneapple_analysis`, catálogo, estágios do `pinneapple_lab`, checagens Darcy/Maxwell) sem executar o `__init__` desses pacotes. [sem teste direto]

`pinneapple_check/_reuse.py` · 98 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`leaf`	função	`(dotted) -> types.ModuleType`	Importa um módulo pelo nome pontilhado sem executar o `__init__` dos pacotes pais.
`foam`	função	`()`	Leitor OpenFOAM de `pinneapple_data.cae.foam`.
`cae_io`	função	`()`	Módulo de E/S CAE de `pinneapple_data.cae.io`.
`cae_geometry`	função	`()`	Geometria de malha de `pinneapple_data.cae.geometry`.
`units`	função	`()`	Unidades físicas de `pinneapple_data.physical_units`.
`envelope_cls`	função	`()`	Classe `Envelope` de `pinneapple_physics.physics_audit.assumptions`.
`field_metrics`	função	`()`	Métricas de campo de `pinneapple_analysis.validation.field_metrics`.
`spectral`	função	`()`	Métricas espectrais de `pinneapple_analysis.validation.spectral`.
`metamorphic`	função	`()`	Testes metamórficos de `pinneapple_analysis.validation.metamorphic`.

nome	tipo	assinatura	o que faz
`dataset_audit`	função	`()`	Auditoria de dataset de `pinneapple_data.audit`.
`method_catalog`	função	`()`	Catálogo de métodos de `pinneapple_catalog.methods`.
`trust_stages`	função	`()`	Estágios do trust card (`STAGES`) de `pinneapple_lab.curation`.
`darcy_checks`	função	`()`	Checagens de Darcy de `pinneapple_physics.darcy_checks`.
`maxwell_checks`	função	`()`	Checagens de Maxwell de `pinneapple_physics.maxwell_checks`.

`pinneapple_check.balances`

Balanços globais na fronteira de uma caixa em grade tensorial: fluxo líquido de volume, fluxo líquido de calor e força resultante, cada um com o fluxo absoluto total para normalizar. [sem teste direto]

`pinneapple_check/balances.py` · 85 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`grid_mass`	função	`(coords, names, vel) -> dict`	Fluxo líquido de volume pela fronteira da grade e o fluxo absoluto total.
`grid_heat`	função	`(coords, names, T, k, q) -> dict`	Fluxo de calor que entra no domínio (k dT/dn) mais a fonte.
`grid_force`	função	`(coords, names, disp, E, nu, f) -> dict`	Resultante das trações de fronteira σ·n mais a força de corpo (deformação plana em 2D).

`pinneapple_check.baseline`

Linha de base: adota o checador num repositório que já tem violações e falha só nas novas. [sem teste direto]

`pinneapple_check/baseline.py` · 34 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`load_baseline`	função	`(path) -> set[str]`	Lê o arquivo de baseline como conjunto de impressões de violações.
`write_baseline`	função	`(report, path) -> int`	Grava as violações atuais do relatório como baseline e devolve quantas.
`apply_baseline`	função	`(report, path) -> None`	Marca no relatório as violações que já estão na baseline.

`pinneapple_check.checks`

Pacote das checagens executadas por `run`: resultados, modelos, conjuntos de dados e integridade de artefatos. [sem teste direto]

`pinneapple_check/checks/__init__.py` · 9 linhas · **sem teste direto**

`pinneapple_check.checks.common`

Checagens comuns a resultados e modelos: valores finitos (PC403), limites declarados (PC401), admissibilidade física (PC402), unidades e referências. [sem teste direto]

`pinneapple_check/checks/common.py` · 227 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Ctx`	classe	`(cfg, target, enabled, outcomes)`	Contexto de um alvo onde os resultados são coletados, com regras ativas, tolerâncias e localização.
`fmt`	função	`(x) -> str`	Formata um número para mensagens.
`admissible`	função	`(name, unit) -> tuple[float | None, float | None, str] | None`	Limites físicos rígidos (baixo, alto, descrição) inferidos do nome e da unidade de um campo.
`check_values`	função	`(ctx, name, v, unit, loc, where=None) -> None`	Aplica PC403 (não finito), PC401 (limites declarados) e PC402 (admissibilidade) a um campo.

nome	tipo	assinatura	o que faz
check_units	função	(ctx, name, file_unit, declared, values, loc, used) -> None	Confere a unidade do arquivo contra a declarada e a plausibilidade das magnitudes (regras PC5xx).
rel_l2	função	(a, b) -> float	Erro L2 relativo entre dois arrays.

pinneapple_check.checks.datasets

Checagens de um conjunto de treino ([[dataset]]) usando a auditoria de pinneapple_data.audit: regras PC451 (física/restrições), PC452 (diversidade/duplicatas) e PC453 (vazamento treino/teste). [sem teste direto]

pinneapple_check/checks/datasets.py · 53 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
check_dataset	função	(ctx, t) -> None	Roda a auditoria física de um conjunto de treino e emite PC451-PC453.

pinneapple_check.checks.integrity

Integridade de artefatos com pinneapple_security: manifestos SHA-256 (PC801, PC804), assinaturas Ed25519 destacadas (PC802) e pins de hash por alvo. [sem teste direto]

pinneapple_check/checks/integrity.py · 112 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
check_pin	função	(ctx, path, expected) -> None	Confere o hash SHA-256 de um arquivo contra o pin declarado.
check_integrity	função	(ctx, cfg) -> None	Confere manifestos e assinaturas declarados na configuração.

pinneapple_check.checks.models

Checagens de um modelo: carga segura, status do método no catálogo, entradas dentro da faixa de treino, física das predições em pontos amostrados (resíduo, divergência, balanços), condições de contorno/iniciais e regressão. [sem teste direto]

pinneapple_check/checks/models.py · 397 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
read_table	função	(path) -> tuple[dict[str, np.ndarray], dict[str, int]]	Lê colunas de um CSV ou arrays de um NPZ, com o número de linha de cada registro do CSV.
check_model	função	(ctx, t) -> LoadedModel | None	Carrega e checa um alvo de modelo, devolvendo o LoadedModel ou None.

pinneapple_check.checks.results

Checagens de um resultado gravado (caso OpenFOAM, grade, nuvem de pontos, tabela de QoI): valores, unidades, contorno, resíduos da EDP, balanços e regressão contra referência.

pinneapple_check/checks/results.py · 707 linhas · testes: 22

nome	tipo	assinatura	o que faz
evaluate_expr	função	(value, coords, n) -> np.ndarray	Avalia um valor declarado: número, vetor ou expressão das coordenadas (ex.: 'sin(pi*x)').
check_result	função	(ctx, t) -> FieldData | None	Carrega e checa um alvo de resultado, devolvendo o FieldData ou None.
report_equations	função	(ctx, eqs, why, where, rule_div, rule_res, tol_div, tol_res, loc, subject_prefix='') -> N…)	Emite os resultados de divergência e resíduo de um conjunto de equações.
check_balance	função	(ctx, rule, what, b, tol, loc, unit='', closed_scale=None, subject=None) -> None	Emite o resultado de um balanço global contra a tolerância.

pinneapple_check.cli

Linha de comando `pinneapple check [paths]` (formatos, seleção de regras, baseline, `--explain`) e o despachante `pinneapple <comando>` (doctor/recommend/card/new, check, design, benchmark).

`pinneapple_check/cli.py` · 207 linhas · testes: 54

nome	tipo	assinatura	o que faz
`check_main`	função	`(argv=None) -> int`	Executa `pinneapple check` com os argumentos e devolve o código de saída.
`main`	função	`(argv=None) -> int`	Despacha `pinneapple <comando>`: doctor/recommend/card/new, check, e design/benchmark quando instalados.

pinneapple_check.config

Configuração por `pinneapple-check.toml` (ou `[tool.pinneapple.check]` no `pyproject.toml`): alvos de resultado, modelo, dataset e integridade, seleção de regras, tolerâncias e descoberta automática sem configuração.

`pinneapple_check/config.py` · 534 linhas · testes: 5

nome	tipo	assinatura	o que faz
`ConfigError`	classe	`()`	Erro de configuração inutilizável (código de saída 2).
`Reference`	classe	`(path, fields, rel_l2, qoi)`	Referência gravada para regressão: caminho, campos, tolerância L2 relativa e QoIs.
`ResultTarget`	classe	`(kind, time, fields, flux, conserved, envelope)`	Alvo de resultado: tipo, tempo, campos, fluxos, quantidades conservadas e envelope.
`ModelTarget`	classe	`(architecture, model_config, hub, zoo, inputs, outputs, train_range, train_samples, queri…)`	Alvo de modelo: arquitetura, hub/zoo, entradas/saídas, faixa de treino, consultas, domínio, método e relações.
`DatasetTarget`	classe	`(zoo, inputs, constraints, train, test, audit)`	Alvo de dataset: zoo, entradas, restrições, partições treino/teste e opções de auditoria.
`Integrity`	classe	`(manifests, public_key, signed, line)`	Configuração de integridade: manifestos, chave pública e arquivos assinados.
`Config`	classe	`(root, file, select, ignore, extend_select, fail_on, baseline, severity, exclude, results…)`	Configuração completa: raiz, seleção/ignorar de regras, fail_on, baseline, severidades, exclusões e alvos.
`find_config`	função	`(start) -> tuple[Path | None, dict[str, Any]]`	Sobe a partir de `start` até o primeiro `pinneapple-check.toml` ou `pyproject.toml` com a seção do checador.
`load_config`	função	`(file=None, start=None, root=None) -> Config`	Interpreta um arquivo de configuração (encontrado a partir de `start` se não for dado).
`discover`	função	`(paths, exclude, root) -> list[ResultTarget]`	Descobre resultados sob os caminhos quando a configuração não declara nenhum: casos OpenFOAM e arquivos de campo.

pinneapple_check.engine

Executa as checagens: configuração -> alvos -> resultados (`Outcome`) -> `Report`, aplicando seleção de regras e baseline.

`pinneapple_check/engine.py` · 93 linhas · testes: 5

nome	tipo	assinatura	o que faz
`run`	função	`(paths=None, *, config=None, select=None, ignore=None, extend_select=None, baseline=None,…)`	Checa os alvos da configuração e devolve o `Report`.

pinneapple_check.evidence

Traduz o relatório em evidência: trust card por alvo (os seis estágios de `pinneapple_lab.curation`) e as checagens de salvaguarda lidas por `pinneapple_veriphysics.applicability.build_applicability_map`.

`pinneapple_check/evidence.py` · 66 linhas · testes: 22

nome	tipo	assinatura	o que faz
`trust_card`	função	`(report) -> dict[str, dict[str, Any]]`	Trust card por alvo: para cada estágio, título, pergunta, status e evidências.
`guardrail_checks`	função	`(report, target) -> list[dict[str, Any]]`	Lista `{name, passed, detail}` para o veriphysics, uma entrada por nome de evidência.

pinneapple_check.fields

Lê resultados de qualquer formato (OpenFOAM, NPZ/HDF5, meshio, tabela de QoI) para uma estrutura única, `FieldData`.

`pinneapple_check/fields.py` · 434 linhas · testes: 16

nome	tipo	assinatura	o que faz
`LoadError`	classe	`()`	Erro ao ler um resultado.
`FieldData`	classe	`(kind, fields, units, files, source, axes, points, mesh, geom, boundary, patches, phi, ti…)`	Estrutura única de um resultado: campos, unidades, eixos ou pontos, malha, contorno, fluxo `phi`, tempo, snapshots e QoIs.
`scalar`	função	`(v) -> np.ndarray`	Converte um valor em array escalar.
`velocity_components`	função	`(fd) -> list[np.ndarray] | None`	Velocidade como lista de componentes (de `U`/`u` ou de escalares `u, v, w`).
`velocity_name`	função	`(fd) -> str | None`	Nome do campo de velocidade encontrado, ou None.
`load_result`	função	`(path, kind=None, time=None, fields=None) -> FieldData`	Lê um resultado escolhendo o leitor pelo tipo ou pela extensão.
`load_openfoam`	função	`(case, time=None, only=None) -> FieldData`	Lê um caso OpenFOAM (malha, campos, patches, `phi`) num instante.
`fv_geometry`	função	`(mesh) -> dict[str, np.ndarray]`	Centros e vetores de área das faces, centros e volumes das células, e direções resolvidas.
`read_surface_scalar`	função	`(b, poly) -> tuple[np.ndarray, np.ndarray]`	Lê um `surfaceScalarField` (`phi`) nas faces internas e de fronteira.
`load_arrays`	função	`(path, kind=None) -> FieldData`	Lê NPZ/HDF5 como grade (se houver coordenadas 1D) ou nuvem de pontos, com unidades opcionais.
`load_meshio`	função	`(path) -> FieldData`	Lê uma malha via meshio para `FieldData`.
`load_qoi`	função	`(path) -> FieldData`	Lê uma tabela de quantidades de interesse.

pinneapple_check.fv

Operadores de volumes finitos sobre uma malha OpenFOAM (teorema de Gauss nas faces de cada célula) e os resíduos conservativos e balanços de fronteira construídos sobre eles.

`pinneapple_check/fv.py` · 189 linhas · testes: 16

nome	tipo	assinatura	o que faz
`FV`	classe	`(fd)`	Operadores de volumes finitos (fluxo, gradiente, laplaciano, somas por patch) sobre uma malha OpenFOAM.
`fv_residuals`	função	`(fv, physics, p)`	Resíduos por célula (por unidade de volume) das equações da física, com os motivos das que não rodaram.
`fv_mass_balance`	função	`(fv) -> dict[str, Any] | None`	Balanço de massa na fronteira usando `phi` ou a velocidade interpolada.
`fv_heat_balance`	função	`(fv, p) -> dict[str, Any] | None`	Balanço de calor na fronteira da malha.

pinneapple_check.models

Carrega um modelo como função simples `(N, n_inputs) -> {saída: (N,)}`, com varredura de segurança do checkpoint antes de carregar.

pinneapple_check/models.py · 192 linhas · testes: 22

nome	tipo	assinatura	o que faz
ModelLoadError	classe	()	Erro ao carregar um modelo.
UnsafeCheckpoint	classe	(report)	Checkpoint recusado pela varredura de segurança, com o relatório.
LoadedModel	classe	(fn, inputs, outputs, train_range=None, file=None)	Modelo carregado como função, com entradas, saídas, faixa de treino e arquivo.
load_model	função	(target) -> LoadedModel	Carrega o modelo de um alvo (checkpoint, hub ou zoo).
scan	função	(p)	Varre um checkpoint com pinneapple_security antes de carregar.

pinneapple_check.numerics

Derivadas de campos escalares nomeados em três discretizações (grade, nuvem de pontos por mínimos quadrados, modelo por diferenças centrais) atrás de uma interface única.

pinneapple_check/numerics.py · 243 linhas · testes: 5

nome	tipo	assinatura	o que faz
GridD	classe	(axes, comps)	Derivadas por diferenças finitas numa grade tensorial.
PointD	classe	(points, comps, axes, k=None)	Derivadas por ajuste quadrático de mínimos quadrados ponderados em nuvem de pontos; pontos na casca ficam de fora.
ModelD	classe	(f, X, inputs, h, spans=None)	Derivadas por diferenças centrais de um modelo nos pontos X.
components	função	(fields, vector_ndim, ncomp=3) -> dict[str, np.ndarray]	Separa campos vetoriais em escalares nomeados (U -> u, v, w).
rel_rms	função	(residual, terms, mask=None, floor=0.0) -> float	RMS do resíduo sobre RMS da soma dos termos absolutos (0 = exato, 1 = termos não se equilibram).

pinneapple_check.physics

Equações governantes (escoamento incompressível, calor, elasticidade, conservação, Darcy, Maxwell) escritas sobre os provedores de derivadas, mais a tabela de parâmetros com dimensões e padrões; o docstring ainda diz 'quatro físicas'.

pinneapple_check/physics.py · 195 linhas · testes: 5

nome	tipo	assinatura	o que faz
Equation	classe	(name, residual, terms, mask, description, scale)	Uma equação avaliada: nome, resíduo, termos, máscara, descrição e escala.
parse_parameters	função	(physics, given) -> tuple[dict[str, Any], list[tuple[str, str]]]	Converte parâmetros para SI e lista os de dimensão errada (regra PC504).
infer_physics	função	(names) -> str | None	Infere a física só pelos nomes dos campos (ex.: U e p -> escoamento incompressível).
equations	função	(physics, D, p, conserved=None) -> tuple[list[Equation], list[str]]	Resíduos da física num provedor de derivadas, com os motivos das equações que não puderam rodar.

pinneapple_check.report

Formatos de saída: text, json, sarif (GitHub code scanning), junit, github (anotações) e summary em Markdown para o job summary.

pinneapple_check/report/__init__.py · 216 linhas · testes: 22

nome	tipo	assinatura	o que faz
render	função	(report, fmt, **kw) -> str	Renderiza o relatório no formato pedido.

nome	tipo	assinatura	o que faz
`text`	função	`(report, color=None, show_passed=False, show_fixes=True, **_) -> str`	Saída de texto legível, com cores e dicas de correção.
`to_json`	função	`(report, **_) -> str`	Saída JSON do relatório.
`sarif`	função	`(report, **_) -> str`	Saída SARIF para o GitHub code scanning.
`junit`	função	`(report, **_) -> str`	Saída JUnit XML.
`github`	função	`(report, **_) -> str`	Anotações por workflow commands do GitHub.
`summary`	função	`(report, **_) -> str`	Resumo em Markdown para o job summary do GitHub Action.

`pinneapple_check.result`

O que uma checagem produz: um `Outcome` por (regra, alvo, sujeito), aprovado ou não, e o `Report` com código de saída. [sem teste direto]

`pinneapple_check/result.py` · 119 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Location`	classe	`(file, line, column)`	Arquivo, linha e coluna de uma violação.
`Outcome`	classe	`(rule, target, subject, passed, message, location, value, threshold, severity, baselined,…)`	Uma avaliação de regra: aprovada ou violação, mensagem, valor, limite, severidade e dica.
`Report`	classe	`(outcomes, targets, config_file, root, notes)`	Relatório com todos os resultados, violações, erros, avisos, código de saída e trust card.

`pinneapple_check.rules`

Tabela de regras do `pinneapple check`: cada regra tem id (`PC101`), nome, severidade padrão, estágio do trust card, explicação e dica de correção; seleção por prefixos no estilo ruff. [sem teste direto]

`pinneapple_check/rules.py` · 290 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Rule`	classe	`(id, name, title, severity, stage, evidence, explanation, fix)`	Regra do checador: id, nome, severidade, estágio do trust card, explicação e correção.
`get_rule`	função	`(rule_id) -> Rule`	Busca uma regra pelo id.
`select_rules`	função	`(select=None, ignore=None, extend_select=None) -> set[str]`	Ids de regras ativas por prefixos no estilo ruff (`select`, `ignore`, `extend_select`).

Exemplos de uso reais do repositório

Uso por Python (docstring do pacote) — `pinneapple_check/__init__.py`

```python
from pinneapple_check import run
report = run(["cases/"])
assert not report.errors, [o.message for o in report.errors]
```

GitHub Action e linha de comando — `README.md`

```yaml
# .github/workflows/physics-check.yml, in a job with `permissions: security-events: write`
- uses: actions/checkout@v4
- uses: PINNeAPPle-Labs/PINNeAPPle@main    # pin a tag or a commit SHA

pinneapple check cases/ models/            # locally: same rules, readable output, exit code 1 on a violation
```

Testes e status de validação

- Módulos com pelo menos um teste que os importa: 12 de 23.
- Funções de teste em arquivos que importam algum módulo do pacote: 144.

Limitações conhecidas

- Os resíduos e balanços têm erro de discretização próprio; em malhas grossas o valor é dominado por ele, por isso as tolerâncias padrão são frouxas (docs/guides/validation_methods_reference.md).
- Condições de contorno checadas são só Dirichlet; Neumann só aparece pelos balanços. Um resíduo pequeno não detecta condição de contorno ou parâmetro errado que a EDP ainda satisfaz.
- A detecção de fora-da-distribuição (PC601/PC602) usa a caixa de treino e distâncias padronizadas; não cobre bem entradas correlacionadas.
- Unidades precisam estar no arquivo ou declaradas; a admissibilidade (PC402) infere a grandeza pelo nome e unidade do campo.
- torch só é necessário para carregar checkpoints `.pt`; `design` e `benchmark` no despachante só funcionam se esses pacotes estiverem instalados.
- Muitos submódulos (`checks.*`, `rules`, `result`, `baseline`, `balances`, `_reuse`) não têm teste que os importe diretamente; são exercitados pelos testes de `run`/CLI.
- O docstring de `physics` fala em 'quatro físicas', mas o código trata seis (escoamento incompressível, calor, elasticidade, conservação, Darcy, Maxwell).

3. `pinneapple_veriphysics`

Objetivo

`pinneapple_veriphysics` é a camada de verificação e evidência do Veriphysics, extraída para dentro do PINNeAPPle (Apache-2.0); a fila de execução, a API e a cobrança ficam no Veriphysics. O pacote responde a pergunta "posso usar este resultado, para quê e com que evidência?" em vez de dar só aprovado/reprovado.

A regra central é não inventar números nem vereditos: `decision`, `applicability` e `evidence_report` só classificam e renderizam fatos que o pipeline já calculou, e uma checagem que não rodou fica `NOT_RUN`. O `ledger` reforça isso: uma checagem só conta como aprovada se há um registro íntegro, com hash, de que ela rodou. `scope` traduz vereditos em usos atendidos e não atendidos, e `robustness` mede extrapolação, ensemble, sensibilidade a pesos e deslocamento de parâmetros com treino real.

O pacote se liga ao resto assim: o `validation_lab` roda as regras do `pinneapple check` (`pinneapple_check`) e as agrupa em verificação numérica, validação contra dados e consistência física; `build_applicability_map` lê as checagens de salvaguarda de `pinneapple_check.evidence`; `recommend` usa `pinneapple_analysis.verification` e um rascunho de LLM (`pinneapple_llm.draft_problem`) para sugerir solver e arquitetura; `evidence_package` gera relatórios e `StudyResult` (`pinneapple_data.contracts`). Os apps `apps/validation_lab` e `apps/evidence_report` são as interfaces web sobre esses módulos.

Números do inventário

módulos	linhas	classes públicas	funções públicas	testes em arquivos que importam o pacote	módulos sem teste direto
13	3.411	17	38	98 (11 arquivos)	1

Pasta de testes dedicada: `tests/pinneapple_veriphysics` (49 funções de teste).

Submódulos

submódulo	o que faz	módulos	classes + funções	testes*
`applicability`	Mapa de aplicabilidade com checklist de oito itens e envelope de variáveis.	1	5	37
`decision`	`DecisionRecord` com recomendação, trust score e alternativas.	1	1	18

submódulo	o que faz	módu-los	classes + funções	tes-tes*
`evidence_package`	Relatório e pacote zip de evidência reprodutível.	1	9	22
`evidence_report`	PDF do Evidence Report.	1	1	18
`example_cases`	Casos de exemplo fixos para o recomendador.	1	2	0
`execution_log`	Coletor de log de passos do pipeline.	1	3	5
`ledger`	Ledger de evidência com hashes.	1	8	22
`mesh_views`	Vistas 3D de erro contra referência em malha.	1	1	3
`recommend`	Formulação do problema e recomendação de método.	1	3	1
`robustness`	Estudos de robustez medidos.	1	6	5
`scope`	Escopo de uso de um resultado.	1	4	5
`validation_lab`	`pinneapple check` agrupado em três categorias com ledger.	1	12	42

* maior número de testes que importam um módulo do submódulo.

Módulos e API pública

`pinneapple_veriphysics`

Camada de verificação e evidência do Veriphysics dentro do PINNeAPPle; o `__init__` só documenta os submódulos (decisão, relatório, aplicabilidade, escopo, robustez, recomendação, ledger, Validation Lab).

`pinneapple_veriphysics/__init__.py` · 35 linhas · testes: 8

`pinneapple_veriphysics.applicability`

Mapa de aplicabilidade de uma corrida: classifica fatos já produzidos em VERIFIED, INFERRED ou UNSUPPORTED, responde um checklist fixo de oito itens e registra o envelope de variáveis testado; não roda checagem nem inventa número.

`pinneapple_veriphysics/applicability.py` · 488 linhas · testes: 37

nome	tipo	assinatura	o que faz
`Tolerances`	classe	`(plateau_rel, spread_rel, ece, coverage)`	Tolerâncias de aceitação do mapa (platô, dispersão, ECE, cobertura), substituíveis por critérios acordados.
`ChecklistItem`	classe	`(key, question, status, tier, summary, evidence)`	Um item do checklist de revisão: pergunta, status, nível (tier), resumo e evidência.
`VariableEnvelope`	classe	`(name, kind, verified_range, verified_point, extrapolation_supported, note)`	Faixa ou ponto verificado de uma variável e se a extrapolação é suportada.
`ApplicabilityMap`	classe	`(checklist, envelope, verified, inferred, unsupported, tolerances)`	O mapa completo: checklist, envelope e listas verificado/inferido/sem suporte, com `to_dict` e `render`.
`build_applicability_map`	função	`(*, architecture, coords, fields, domain_bounds, physical_parameters, guardrail_checks, c…)`	Classifica os fatos de uma corrida (checagens de salvaguarda, confiança, perda, domínio) num `ApplicabilityMap`.

`pinneapple_veriphysics.decision`

`DecisionRecord`, a saída para humanos de uma corrida: recomendação, trust score e cobertura, evidência por checagem e alternativas, montado só de objetos que o orquestrador já produziu.

`pinneapple_veriphysics/decision.py` · 157 linhas · testes: 18

nome	tipo	assinatura	o que faz
`DecisionRecord`	classe	`(run_id, problem_description, recommended_strategy, reasoning, trust_score, trust_coverag…)`	Registro de decisão de uma corrida: estratégia recomendada, raciocínio, trust score, alternativas e ferramentas externas, com `render`.

pinneapple_veriphysics.evidence_package

Evidência de engenharia: transforma arquivos de experimentos e simulações num relatório reprodutível (JSON, HTML, PDF, figuras) e num pacote zip com manifesto SHA-256 verificável.

pinneapple_veriphysics/evidence_package.py · 638 linhas · testes: 22

nome	tipo	assinatura	o que faz
Source	classe	(name, kind, sha256, bytes, raw, url, data, contract, problems)	Um arquivo de entrada com tipo detectado, SHA-256, tamanho, dados lidos e problemas.
load_source	função	(name, data, url='') -> Source	Lê um arquivo e decide o que ele é; conteúdo desconhecido é mantido como arquivo referenciado.
build_report	função	(meta, sources) -> dict[str, Any]	Monta o relatório como dados a partir dos metadados e das fontes (baseline contra candidato, testes).
make_figures	função	(report, outdir) -> dict[str, str]	Figuras PNG com os números do próprio relatório.
render_html	função	(report, figures=None) -> str	HTML autocontido do relatório com figuras embutidas.
render_pdf	função	(report, figures=None, path=None) -> bytes	PDF do relatório via pinneapple_tools.reporting.paperkit.Paper (precisa de reportlab).
to_study_result	função	(report, app='evidence') -> dict[str, Any]	Converte o relatório num StudyResult (pinneapple_data.contracts) para o próximo app.
build_package	função	(report, sources, *, html=None, pdf=None, figures=None) -> bytes	Zip com relatório, ledger, figuras, fontes originais e MANIFEST.json com SHA-256 de cada arquivo.
verify_package	função	(data) -> list[str]	Lista os problemas de um pacote (vazio quando todos os arquivos batem com o manifesto).

pinneapple_veriphysics.evidence_report

Gera o Evidence Report em PDF a partir de um DecisionRecord já calculado (precisa do opcional reportlab).

pinneapple_veriphysics/evidence_report.py · 206 linhas · testes: 18

nome	tipo	assinatura	o que faz
render_evidence_report_pdf	função	(decision) -> bytes	Renderiza um DecisionRecord como PDF e devolve os bytes.

pinneapple_veriphysics.example_cases

Catálogo fixo de casos de exemplo para o recomendador (Burgers 1D e dois casos de turbomáquina), que contorna o LLM para esses casos conhecidos porque o preenchimento de kwargs pelo LLM se mostrou instável. [sem teste direto]

pinneapple_veriphysics/example_cases.py · 228 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
ExampleCase	classe	(key, label, preset, preset_kwargs, description, physical_parameters, flow_geometry, epoc…)	Um caso de exemplo: preset, kwargs, parâmetros físicos, geometria, épocas, colocação e limiar de resíduo.
get_example_case	função	(key) -> ExampleCase	Busca um caso pela chave ou levanta ValueError listando as chaves registradas.

pinneapple_veriphysics.execution_log

Coletor de log de execução seguro para threads, para a API/UI verem os passos reais do pipeline enquanto ele roda.

pinneapple_veriphysics/execution_log.py · 51 linhas · testes: 5

nome	tipo	assinatura	o que faz
LogEntry	classe	(timestamp_unix, message)	Uma entrada do log: instante e mensagem.

nome	tipo	assinatura	o que faz
`LogCollector`	classe	`() -> None`	Coletor passado a `analyze()`/`formulate_and_recommend()` para observar os passos; pode ser lido de outra thread.
`emit`	função	`(log, message) -> None`	Registra no log se houver um, senão não faz nada.

pinneapple_veriphysics.ledger

Ledger de evidência só de acréscimo: uma checagem só aparece como aprovada se há um registro íntegro (com hash) de que ela rodou.

`pinneapple_veriphysics/ledger.py` · 242 linhas · testes: 22

nome	tipo	assinatura	o que faz
`EvidenceEntry`	classe	`(check_id, name, category, tool, tool_version, inputs_sha256, outputs, started_at, finish…)`	Uma execução registrada de checagem: ferramenta, hash das entradas, saídas, tempos, resultado e hash da entrada.
`EvidenceLedger`	classe	`(entries)`	Lista só de acréscimo de `EvidenceEntry`; o status de uma checagem é o da última entrada válida.
`canonical_json`	função	`(obj) -> str`	JSON canônico (chaves ordenadas, sem espaços) para hashes estáveis.
`sha256_bytes`	função	`(data) -> str`	SHA-256 de bytes.
`sha256_json`	função	`(obj) -> str`	SHA-256 do JSON canônico de um objeto.
`sha256_file`	função	`(path, chunk=1 << 20) -> str`	SHA-256 de um arquivo lido em blocos.
`utc_now`	função	`(t=None) -> str`	Instante UTC em texto ISO.
`record_checks`	função	`(ledger, checks, *, prefix, inputs_sha256, started_at, finished_at, tool='', tool_version…)`	Registra no ledger as checagens de um trust card.

pinneapple_veriphysics.mesh_views

Vistas 3D das comparações com referência de uma corrida do Validation Lab: predição, referência e erro pontual na malha (OpenFOAM, VTU/VTK) com as regiões de maior erro.

`pinneapple_veriphysics/mesh_views.py` · 164 linhas · testes: 3

nome	tipo	assinatura	o que faz
`mesh_views`	função	`(workdir, config=None, *, report=None, max_faces=None, n_regions=8, mesh_unit='m') -> lis…)`	Uma vista 3D por alvo de resultado com referência: campo, erros e regiões.

pinneapple_veriphysics.recommend

Metade de formulação e escolha de método do `pipeline.analyze()`: números adimensionais, regime, família de solver, arquitetura e ferramentas externas, com uma chamada `draft_problem` de LLM (`pinneapple_llm`) e sem treino.

`pinneapple_veriphysics/recommend.py` · 295 linhas · testes: 1

nome	tipo	assinatura	o que faz
`CandidateDetail`	classe	`(key, name, category, is_recommended, when_to_use, license, typical_cost_tier, player, st…)`	Uma opção documentada (arquitetura ou ferramenta externa) com prós, contras, licença e custo típico.
`RecommendationResult`	classe	`(rejected, rejection_reason, dimensionless_numbers, flow_regime, drafted_preset, drafted_…)`	Resultado da formulação e recomendação: rejeição, números adimensionais, regime, preset rascunhado, família de solver e candidatos.
`formulate_and_recommend`	função	`(description, *, physical_parameters=None, flow_geometry='internal_pipe', provider='ollam…)`	Formula o problema a partir de uma descrição e recomenda família de solver, arquitetura e ferramentas externas.

pinneapple_veriphysics.robustness

Estudos de robustez medidos que alimentam o mapa de aplicabilidade: extrapolação, ensemble (UQ não calibrada sem referência), sensibilidade aos pesos da perda, cobertura das condições e deslocamento de parâmetros.

`pinneapple_veriphysics/robustness.py` · 300 linhas · testes: 5

nome	tipo	assinatura	o que faz
`ensemble_study`	função	`(spec, build_model, best_model, *, n_members, epochs, n_collocation, n_eval=2048, margins…)`	Treina sementes extras e mede a discordância entre membros no interior e fora do domínio.
`extrapolation_study`	função	`(spec, model, *, residual_threshold, ensemble=None, margins=DEFAULT_MARGINS, n_eval=2048,…)`	Resíduo (e dispersão do ensemble) em cascas fora do domínio de treino, dando o limite de extrapolação medido.
`weight_sensitivity_study`	função	`(spec, build_model, best_model, *, epochs, n_collocation, factors=DEFAULT_WEIGHT_FACTORS,…)`	Retreina com o peso da EDP escalado e mede quanto a predição muda.
`calibration_against_reference`	função	`(members, reference_x, reference_y) -> dict[str, Any]`	Calibração do ensemble contra dados de referência via `CalibrationMetrics`.
`condition_coverage`	função	`(spec, n=50000) -> dict[str, Any]`	Conta quantos pontos cada condição amostrada automaticamente seleciona com o mesmo amostrador do solver.
`parameter_shift_study`	função	`(make_spec, base_kwargs, build_model, base_model, *, residual_threshold, epochs, n_colloc…)`	Generalização em parâmetros: quanto o modelo se afasta quando parâmetros físicos são deslocados.

pinneapple_veriphysics.scope

Escopo de uso: quais usos um resultado atende, onde falha e por quê, além de um veredito único aprova/reprova.

`pinneapple_veriphysics/scope.py` · 148 linhas · testes: 5

nome	tipo	assinatura	o que faz
`UseScope`	classe	`(use, ok, reason, why, evidence, values, limits, fraction_ok, precondition)`	Um uso de um resultado: utilizável, não utilizável ou não avaliado, com motivo e evidência.
`ScopeOfUse`	classe	`(subject, uses, figure, notes)`	Os usos que um resultado atende ou não, com figura e notas opcionais.
`scope_from_verdicts`	função	`(subject, verdicts, *, evidence='estimated', figure=None, notes=()) -> ScopeOfUse`	Monta um `ScopeOfUse` a partir de vereditos por uso.
`scope_from_batch`	função	`(subject, verdicts, *, min_fraction=0.9, evidence='estimated', figure=None, notes=()) -> …)`	Monta um `ScopeOfUse` sobre um lote de vereditos por amostra (uso OK se a fração mínima passar).

pinneapple_veriphysics.validation_lab

Validation Lab: roda as checagens do `pinneapple check` sobre predições, resultados ou um modelo do zoo e relata em três grupos (verificação numérica, validação contra dados, consistência física), com registro no ledger para cada checagem.

`pinneapple_veriphysics/validation_lab.py` · 459 linhas · testes: 42

nome	tipo	assinatura	o que faz
`CodeNotAllowed`	classe	`()`	Erro quando a configuração importaria ou desserializaria código enviado pelo usuário.
`LabReport`	classe	`(check_report, ledger, config_text, inputs, started_at, finished_at, versions, title)`	Relatório do lab: relatório do check, ledger, configuração, entradas, versões, categorias e decisão.
`category_of`	função	`(rule_id) -> str | None`	Grupo de uma regra do `pinneapple check` (None para PC002, que é cobertura).
`to_toml`	função	`(doc) -> str`	Escreve um `pinneapple-check.toml` a partir de um dicionário.
`model_ref_target`	função	`(ref) -> dict[str, Any]`	Tabela `[[model]]` a partir de uma referência a modelo do zoo.

nome	tipo	assinatura	o que faz
`config_from_spec`	função	`(spec) -> str`	Gera `pinneapple-check.toml` a partir de uma descrição simples do que checar.
`assert_no_code`	função	`(cfg) -> None`	Recusa alvos que executariam código das entradas (função Python, checkpoint, download do Hub).
`write_inputs`	função	`(root, files) -> list[str]`	Grava arquivos enviados sob `root`, recusando nomes que saiam dele.
`tables_to_csv`	função	`(root, config_text=None) -> tuple[str | None, dict[str, str]]`	Converte Parquet e tabelas JSON em CSV ao lado dos originais.
`run_validation`	função	`(workdir, config=None, *, allow_code=False, title='Validation') -> LabReport`	Roda o `pinneapple check` nas entradas de `workdir` e devolve o `LabReport`.
`summarize`	função	`(lab) -> dict[str, Any]`	Visão compacta de um relatório do lab: decisão e contagens por grupo.
`render_html`	função	`(lab) -> str`	HTML autocontido de um relatório do lab, sem scripts nem arquivos externos.

Exemplos de uso reais do repositório

Escopo de uso e Evidence Report para modelos PhysicsNeMo — `examples/vs_physicsnemo/07_physicsnemo_trains_pinneapple_checks/run.py`

```
    from pinneapple_veriphysics.decision import DecisionRecord
    from pinneapple_veriphysics.evidence_report import render_evidence_report_pdf
    from pinneapple_veriphysics.scope import scope_from_batch
...
        verdicts = [dc.use_verdicts(data["a"][i], preds[(r, m)][i], DARCY["f"], envelope=envelope)
                    for i in range(len(data["a"]))]
        label = {"in_distribution": "test set"}.get(r, SHIFTS.get(r, {}).get("label", r))
        scopes[r] = scope_from_batch(f"PhysicsNeMo {LABEL[m]}, {label}", verdicts, evidence=evidence)
```

Validation Lab na API do app — `apps/validation_lab/validation_lab/api.py`

```
            (Path(d) / CONFIG_NAME).write_text(config, encoding="utf-8")
            with _HEAVY:
                lab = run_validation(d, allow_code=allow_code).to_dict()
                lab["views3d"] = _views(d, lab)
        except HTTPException:
            raise
        except CodeNotAllowed as e:
            raise HTTPException(422, str(e)) from e
```

Testes e status de validação

- Módulos com pelo menos um teste que os importa: 12 de 13.
- Funções de teste em arquivos que importam algum módulo do pacote: 98.

Limitações conhecidas

- `evidence_report` e `evidence_package.render_pdf` precisam do opcional `reportlab`.
- `recommend.formulate_and_recommend` depende de um LLM via `pinneapple_llm` (provedor padrão `ollama`); o docstring de `example_cases` registra que o LLM preenche kwargs de preset de forma instável, por isso há o catálogo fixo.
- `example_cases` cobre só Burgers 1D e dois casos de turbomáquina; presets com seletores por tag (`industrial_furnace_thermal`, `pipe_flow_3d`) ficam fora de escopo, segundo o docstring.
- A dispersão do ensemble em `robustness` é UQ não calibrada a menos que haja dados de referência; um resíduo pequeno fora do domínio não prova que o modelo acerta a solução verdadeira.
- `validation_lab` recusa por padrão alvos que executariam código do usuário (função Python, checkpoint, download do Hub); só modelos do zoo e arquivos de dados são aceitos sem `allow_code`.
- `example_cases` não tem teste direto e `recommend` tem pouca cobertura de teste.

4. pinneapple_security

Objetivo

O pinneapple_security cuida da segurança de dados e de processo em IA física científica e industrial. Ele junta, num só pacote e quase só com a biblioteca padrão e NumPy, as peças que um projeto precisa para provar de onde vieram dados e modelos e quem mexeu neles: digests SHA-256 e manifestos Merkle (integrity), assinaturas Ed25519 ou HMAC em envelopes DSSE (signing), atestações in-toto/SLSA e SBOM CycloneDX (attestation), trilha de auditoria encadeada por hash com atributos ALCOA+ (audit) e criptografia AES-256-GCM em repouso (encryption).

Para dados sensíveis há detecção e redação de dados pessoais (inclusive CPF e CNPJ com dígitos verificadores), pseudonimização com chave, k-anonimato e limpeza de logs (privacy), privacidade diferencial com contador RDP e DP-SGD para PyTorch (dp), rótulos de classificação e política de manuseio (classification) e um mapa de capacidades para controles de NIST CSF 2.0, IEC 62443, ISO 27001, 21 CFR Part 11, LGPD/GDPR, SSDF e SLSA (compliance), que mapeia evidências mas não certifica.

Do lado de modelos e operação, model_security varre checkpoints pickle sem executá-los e carrega com weights_only=True, além de medir sensibilidade adversarial de surrogates; anomaly e cps detectam injeção de dados falsos em sensores industriais por resíduos físicos (balanços de massa, leis de vazão) com CUSUM, e cps_bench é um benchmark reprodutível de uma rede de água de três tanques comparando o detector físico com detectores só de dados.

Ele se liga ao resto da biblioteca por integrate (versões assinadas no ModelStore de pinneapple_registry, model cards e TrustReports assinados, experimentos pp.Experiment atestados) e às apps web por web (cabeçalhos de segurança e CSP, limite de corpo, rate limit, bloqueio de login, IP por proxy confiável, CSRF, CORS, URLs de saída seguras), passwords (hash de senha scrypt/argon2id/bcrypt) e archives (extração de zip sem zip-slip nem zip bomb). Há também uma CLI: python -m pinneapple_security.

Números do inventário

módulos	linhas	classes públicas	funções públicas	testes em arquivos que importam o pacote	módulos sem teste direto
20	3.398	33	93	105 (10 arquivos)	0

Pasta de testes dedicada: tests/pinneapple_security (60 funções de teste).

Submódulos

submódulo	o que faz	módulos	classes + funções	testes*
__main__	CLI python -m pinneapple_security: manifestos, assinaturas, varreduras, limpeza de log, SBOM e hash de senha.	1	1	10
anomaly	Detecção de dados de sensor manipulados por resíduo físico com CUSUM e detecção de replay.	1	3	6
archives	Extração segura de zip: sem zip-slip, sem links e com limites contra zip bomb; gunzip limitado.	1	7	22
attestation	Atestações in-toto/SLSA de execuções e experimentos e SBOM CycloneDX do ambiente.	1	7	10
audit	Trilha de auditoria append-only em JSON lines encadeada por hash (ou HMAC).	1	1	54
classification	Rótulos de dados (nível, dados pessoais, controle de exportação) e política de manuseio.	1	4	10
compliance	Mapa capacidade -> controle de frameworks de auditoria e relatório de autoavaliação.	1	3	10
cps	Resíduos físicos caixa-cinza (balanço de massa, leis de vazão) com CUSUM contra injeção de dados falsos.	1	9	5
cps_bench	Benchmark reprodutível de rede de água de três tanques com ataques e detectores de base só de dados.	1	7	5
dp	Privacidade diferencial: mecanismos de Laplace e Gauss, estatísticas privadas, contador RDP e DP-SGD.	1	7	9

submódulo	o que faz	módulos	classes + funções	testes*
`encryption`	Criptografia AES-256-GCM em repouso para bytes, arquivos e arrays, com chave por scrypt ou aleatória.	1	9	6
`integrate`	Integra a segurança ao `ModelStore`, aos model cards e aos experimentos (assinar, verificar, atestar).	1	8	10
`integrity`	Digests SHA-256 de arquivos, arrays e registros e manifestos Merkle de pastas.	1	8	14
`model_security`	Varredura de checkpoints por execução de código, carga segura e sensibilidade adversarial.	1	6	6
`passwords`	Hash e verificação de senhas de login (scrypt, argon2id, bcrypt).	1	4	56
`privacy`	Detecção e redação de PII, pseudonimização, k-anonimato e limpeza de logs.	1	11	9
`secrets_scan`	Busca de credenciais em código, notebooks, configs e casos de solver.	1	4	10
`signing`	Assinaturas Ed25519 ou HMAC-SHA256, envelopes DSSE e assinaturas destacadas de arquivo.	1	12	25
`web`	Endurecimento HTTP das apps ASGI: cabeçalhos, CSP, limites, rate limit, login, CSRF, CORS e URLs de saída.	1	15	40

* maior número de testes que importam um módulo do submódulo.

Módulos e API pública

`pinneapple_security`

Ponto de entrada do pacote; o docstring é o índice das capacidades e diz as dependências (biblioteca padrão e NumPy; PyTorch para DP-SGD e carga segura; `cryptography` para Ed25519 e AES-GCM).

`pinneapple_security/__init__.py` · 67 linhas · testes: 64

`pinneapple_security.__main__`

CLI com comandos `manifest`, `verify-manifest`, `keygen`, `sign`, `verify`, `scan-model`, `scan-secrets`, `sanitize-log`, `sbom` e `hash-password`, com código de saída 1 quando algo falha.

`pinneapple_security/__main__.py` · 143 linhas · testes: 10

nome	tipo	assinatura	o que faz
`main`	função	`(argv=None) -> int`	Analisa os argumentos e executa o subcomando da CLI de segurança.

`pinneapple_security.anomaly`

Detecta injeção de dados falsos, spoofing e replay em fluxos industriais: CUSUM bilateral sobre o resíduo físico padronizado e um escore de replay por janelas repetidas.

`pinneapple_security/anomaly.py` · 102 linhas · testes: 6

nome	tipo	assinatura	o que faz
`ResidualCUSUM`	classe	`(k, h, mu, sigma, fitted, s_hi, s_lo, t, alarms)`	CUSUM bilateral sobre um resíduo físico: aprende média e desvio de dados confiáveis e acusa alarmes.
`balance_residual`	função	`(inflows, outflows, accumulation=None) -> np.ndarray`	Resíduo de conservação por amostra: soma das entradas menos saídas menos variação do armazenamento.
`replay_score`	função	`(x, window=50) -> dict[str, Any]`	Menor distância normalizada entre a última janela e janelas anteriores; perto de 0 indica replay.

`pinneapple_security.archives`

Abre zips enviados com segurança: nomes relativos seguros, sem links simbólicos e com limites de membros, tamanho total, tamanho por membro e razão de compressão.

`pinneapple_security/archives.py` · 153 linhas · testes: 22

nome	tipo	assinatura	o que faz
`UnsafeArchive`	classe	`()`	Erro de arquivo compactado que quebra uma regra de segurança ou um limite.
`ArchiveLimits`	classe	`(max_members, max_total_bytes, max_member_bytes, max_ratio)`	Limites de número de membros, bytes totais, bytes por membro e razão de compressão.
`check_member_name`	função	`(name) -> str`	Devolve o nome do membro como caminho POSIX relativo seguro, ou levanta `UnsafeArchive`.
`check_zip`	função	`(z, limits=DEFAULT_LIMITS) -> list[zipfile.ZipInfo]`	Checa nomes, links e tamanhos declarados e devolve os membros de arquivo.
`safe_extract_zip`	função	`(source, dest, limits=DEFAULT_LIMITS) -> list[Path]`	Extrai um zip sob o destino com todas as checagens e devolve os arquivos escritos.
`read_members`	função	`(source, limits=DEFAULT_LIMITS, skip=None) -> dict[str, bytes]`	Lê todos os membros em memória com as mesmas checagens e limites contados.
`gunzip`	função	`(data, max_bytes=1 << 30) -> bytes`	Descompacta gzip recusando saída acima de `max_bytes` sem produzi-la antes.

pinneapple_security.attestation

Gera declarações in-toto v1 com predicado SLSA Provenance v1 para arquivos e experimentos, com commit git, e um SBOM CycloneDX 1.5 das distribuições instaladas.

`pinneapple_security/attestation.py` · 153 linhas · testes: 10

nome	tipo	assinatura	o que faz
`subject`	função	`(name, path=None, sha256=None) -> dict[str, Any]`	Monta um sujeito in-toto (nome e digest SHA-256).
`git_commit`	função	`(path='.') -> dict[str, Any] | None`	Commit do repositório que contém o caminho e se a árvore está suja (`None` fora do git).
`provenance`	função	`(external_parameters, dependencies=(), started_on=None, finished_on=None, builder_id=…, i…)`	Monta o predicado SLSA Provenance v1 (parâmetros, dependências, construtor, tempos).
`statement`	função	`(subjects, predicate, predicate_type=SLSA_PROVENANCE) -> dict[str, Any]`	Monta uma declaração in-toto v1 com sujeitos e predicado.
`attest_files`	função	`(artifacts, parameters, inputs=(), repo='.') -> dict[str, Any]`	Declaração para arquivos de saída construídos de arquivos de entrada com parâmetros.
`attest_experiment`	função	`(result, artifacts=(), inputs=(), repo='.') -> dict[str, Any]`	Declaração para o resultado de um `pp.Experiment` e seus arquivos salvos.
`sbom`	função	`(name='pinneapple-environment', version='', packages=None) -> dict[str, Any]`	SBOM CycloneDX 1.5 das distribuições Python instaladas (todas ou as listadas).

pinneapple_security.audit

Trilha de auditoria append-only em que cada registro carrega o hash do anterior; com chave vira cadeia HMAC; é base para trilhas no estilo 21 CFR Part 11, não um sistema certificado.

`pinneapple_security/audit.py` · 133 linhas · testes: 54

nome	tipo	assinatura	o que faz
`AuditLog`	classe	`(path, key=None)`	Log de auditoria em JSON lines encadeado por hash ou HMAC, com `append` e `verify`.

pinneapple_security.classification

Rótulos de dados (`DataLabel`) e uma política padrão (`HandlingPolicy`) que diz se pode publicar, compartilhar, treinar, enviar à nuvem ou exportar, e por quê; o rótulo registra o que o dono declarou.

`pinneapple_security/classification.py` · 106 linhas · testes: 10

nome	tipo	assinatura	o que faz
`DataLabel`	classe	`(level, owner, personal_data, export_control, licence, retention_days, allowed_countries,…)`	Rótulo de um artefato: nível, dono, dados pessoais, controle de exportação, licença, retenção e países.
`Decision`	classe	`(allowed, reasons, requires)`	Resposta da política: permitido ou não, motivos e aprovações exigidas.

nome	tipo	assinatura	o que faz
`HandlingPolicy`	classe	`(publish_levels, share_levels, cloud_levels)`	Regras padrão (editáveis) que decidem ações sobre um artefato rotulado.
`combine`	função	`(labels) -> DataLabel`	Rótulo de algo derivado de vários: nível mais restrito e união de dados pessoais e controles.

pinneapple_security.compliance

Mapeia cada capacidade do pacote para controles de NIST CSF 2.0, IEC 62443-3-3, ISO/IEC 27001, 21 CFR Part 11, LGPD/GDPR, SSDF e SLSA e gera autoavaliação; mapeia, não certifica.

`pinneapple_security/compliance.py` · 106 linhas · testes: 10

nome	tipo	assinatura	o que faz
`controls_for`	função	`(capability) -> list[dict[str, Any]]`	Lista os controles de framework para os quais uma capacidade gera evidência.
`assess`	função	`(evidence, frameworks=()) -> dict[str, Any]`	Autoavaliação: marca como coberto o controle com ao menos uma evidência válida.
`report_markdown`	função	`(result, title=…) -> str`	Formata o resultado de `assess` como relatório Markdown.

pinneapple_security.cps

Detector caixa-cinza de injeção de dados falsos em sistemas ciberfísicos: componentes de resíduo físico com constantes identificadas de dados confiáveis, CUSUM por canal e limiar calibrado a um orçamento de falsos alarmes; a base `CusumAlarm` levanta `NotImplementedError`.

`pinneapple_security/cps.py` · 305 linhas · testes: 5 · palavras-chave no código (heurística; o texto acima diz o status real): levanta `NotImplementedError` em algum caminho

nome	tipo	assinatura	o que faz
`ResidualComponent`	classe	`(name)`	Base de um componente que mapeia colunas de sensores a uma série de resíduo (`fit`, `residual`).
`MassBalance`	classe	`(level, inflows, outflows, dt, window, area, name)`	Balanço de armazenamento de um volume numa janela: nível versus vazões integradas.
`FlowLaw`	classe	`(flow, up, down, exponent, midpoint, coef, name)`	Lei de vazão por diferença de carga (válvula, orifício, tubo ou vertedouro) com constantes ajustadas.
`CusumAlarm`	classe	`(k, h, upper_only)`	Lógica de alarme comum: escores padronizados, CUSUM e limiar calibrado; subclasses definem `z` e `channels`.
`PhysicsResidualDetector`	classe	`(components, k, h, mu_, sigma_)`	Detector com resíduos físicos caixa-cinza e CUSUM bilateral por canal.
`cusum`	função	`(z, k=0.5, h=None, upper_only=False) -> tuple[np.ndarray, list[int]]`	CUSUM bilateral de escores padronizados, com estatística, máximo entre canais e alarmes.
`roc_auc`	função	`(scores, labels) -> float`	Área sob a curva ROC pelo U de Mann-Whitney, descartando NaN.
`window_auc`	função	`(stat_attacked, episodes, stat_clean) -> float`	AUC por episódio: máximo da estatística em cada ataque contra o máximo em janelas limpas.
`episode_metrics`	função	`(alarms, episodes, n, samples_per_day=1440, grace=60) -> dict[str, Any]`	Taxa e atraso de detecção por episódio de ataque e falsos alarmes por dia fora deles.

pinneapple_security.cps_bench

Benchmark aberto de uma rede de água por gravidade de três tanques com seis ataques e três detectores de base sem física (PCA, AR, isolation forest), todos com o mesmo alarme CUSUM.

`pinneapple_security/cps_bench.py` · 331 linhas · testes: 5

nome	tipo	assinatura	o que faz
`PCAResidualDetector`	classe	`(variance)`	Detector de base pela estatística Q de um modelo PCA dos sensores e incrementos.

nome	tipo	assinatura	o que faz
ARResidualDetector	classe	(lags, ridge, upper_only)	Gêmeo linear só de dados: prevê cada sensor de defasagens de todos (ridge) e aplica CUSUM.
IsolationForestDetector	classe	(n_estimators, seed, max_fit)	Isolation forest sobre sensores e incrementos; precisa de scikit-learn.
simulate_three_tank	função	(days=7.0, seed=0, dt=60.0, substeps=12, demand_scale=1.0, pump_band=(1.0, 2.5)) -> dict[…]	Simula a planta de três tanques e devolve as colunas de sensores e a verdade.
inject	função	(D, kind, episodes) -> dict[str, np.ndarray]	Cópia dos dados com as leituras reescritas em cada episódio de ataque.
physics_detector	função	(dt=60.0, window=1, k=0.5) -> PhysicsResidualDetector	Gêmeo caixa-cinza da planta: três balanços de armazenamento e duas leis de válvula.
run_benchmark	função	(seed=0, train_days=7.0, calib_days=7.0, test_days=10.0, episodes=20, length=240, false_a…)	Ajusta, calibra e testa todos os detectores contra todos os ataques nos regimes nominal e deslocado.

pinneapple_security.dp

Privacidade diferencial: mecanismos de Laplace e Gauss, média e histograma privados, contador RDP do mecanismo gaussiano amostrado e DP-SGD com `torch.func`.

pinneapple_security/dp.py · 155 linhas · testes: 9

nome	tipo	assinatura	o que faz
RDPAccountant	classe	()	Contador de privacidade Rényi do mecanismo gaussiano amostrado, convertido para (ε, δ).
DPSGD	classe	(model, optimizer, loss_fn, noise_multiplier, max_grad_norm, sample_rate, seed=None)	Envoltório DP-SGD: gradientes por amostra recortados mais ruído gaussiano em cada `step`.
laplace_mechanism	função	(value, sensitivity, epsilon, rng=None) -> Any	Mecanismo de Laplace (ε-DP puro).
gaussian_sigma	função	(sensitivity, epsilon, delta) -> float	Desvio do ruído gaussiano para (ε, δ)-DP pela calibração clássica (válida para ε ≤ 1).
gaussian_mechanism	função	(value, sensitivity, epsilon, delta, rng=None) -> Any	Mecanismo gaussiano ((ε, δ)-DP).
dp_mean	função	(x, lower, upper, epsilon, rng=None) -> float	Média ε-DP com valores recortados a limites conhecidos.
dp_histogram	função	(x, bins, epsilon, rng=None) -> np.ndarray	Contagens ε-DP por faixa, com contagens negativas zeradas.

pinneapple_security.encryption

AES-256-GCM para bytes, arquivos e arrays, com chave derivada de senha por scrypt ou aleatória; precisa de `cryptography`.

pinneapple_security/encryption.py · 106 linhas · testes: 6

nome	tipo	assinatura	o que faz
DecryptionError	classe	()	Erro de decriptação (chave errada ou dado adulterado).
derive_key	função	(passphrase, salt) -> bytes	Deriva uma chave de 32 bytes de uma senha e um sal por scrypt.
new_key	função	() -> bytes	Gera uma chave aleatória de 32 bytes.
encrypt_bytes	função	(data, secret) -> bytes	Cifra bytes com AES-256-GCM.
decrypt_bytes	função	(blob, secret) -> bytes	Decifra bytes cifrados por `encrypt_bytes`.
encrypt_file	função	(path, secret, out=None, remove_plain=False) -> str	Cifra um arquivo, opcionalmente apagando o original.
decrypt_file	função	(path, secret, out=None) -> str	Decifra um arquivo cifrado.
encrypt_array	função	(a, secret) -> bytes	Cifra um array NumPy.

nome	tipo	assinatura	o que faz
`decrypt_array`	função	`(blob, secret)`	Decifra um array cifrado por `encrypt_array`.

`pinneapple_security.integrate`

Liga a segurança ao resto da biblioteca: salvar versões assinadas e varridas no `ModelStore`, reassinar e verificar versões, assinar model cards e atestar experimentos.

`pinneapple_security/integrate.py` · 134 linhas · testes: 10

nome	tipo	assinatura	o que faz
`secure_save`	função	`(store, problem_id, model, signing_key, *, metadata=None, stage='development', trust_repo…)`	`ModelStore.save` mais manifesto assinado, varredura do checkpoint e registro de auditoria.
`resign_version`	função	`(store, problem_id, version, signing_key, *, actor='', audit=None, reason='') -> str`	Reassina uma versão após mudança legítima (por exemplo `promote`).
`verify_version`	função	`(store, problem_id, version, key) -> dict[str, Any]`	Verifica assinatura, manifesto, arquivos e varredura do checkpoint de uma versão.
`sign_record`	função	`(obj, key) -> dict[str, Any]`	Assina um registro JSON num envelope.
`verify_record`	função	`(envelope, key, expected=None) -> Any`	Verifica um envelope de registro e devolve o conteúdo.
`sign_model_card`	função	`(card, key, path=None) -> dict[str, Any]`	Assina um model card (com TrustReport) e grava ao lado do card se pedido.
`verify_model_card`	função	`(card, envelope, key) -> dict[str, Any]`	Verifica a assinatura de um model card contra o card atual.
`attest_and_sign_experiment`	função	`(result, key, directory=None, inputs=(), repo='.') -> dict[str, Any]`	Declaração in-toto/SLSA assinada para o resultado de um `pp.Experiment`.

`pinneapple_security.integrity`

Digests SHA-256 de bytes, arquivos, registros (JSON canônico) e arrays, raiz Merkle e manifesto de pastas.

`pinneapple_security/integrity.py` · 139 linhas · testes: 14

nome	tipo	assinatura	o que faz
`Manifest`	classe	`(files, algorithm, root)`	Arquivos de uma pasta com digests e tamanhos e a raiz Merkle.
`sha256_bytes`	função	`(data) -> str`	SHA-256 de bytes.
`sha256_file`	função	`(path) -> str`	SHA-256 de um arquivo.
`canonical_json`	função	`(obj) -> bytes`	JSON canônico em bytes (chaves ordenadas, separadores compactos, UTF-8).
`digest_record`	função	`(obj) -> str`	SHA-256 do JSON canônico de um registro.
`digest_array`	função	`(a) -> str`	SHA-256 de dtype, forma e bytes de um array NumPy ou tensor.
`merkle_root`	função	`(leaves) -> str`	Raiz Merkle SHA-256 de digests folha.
`build_manifest`	função	`(folder, include=(), exclude=…) -> Manifest`	Manifesto de todos os arquivos sob uma pasta, com filtros de inclusão e exclusão.

`pinneapple_security.model_security`

Lê os opcodes pickle sem executar e lista os globais importados, marca perigosos (`os`, `subprocess`, `eval`…), carrega com `torch.load(weights_only=True)` e mede sensibilidade adversarial por PGD.

`pinneapple_security/model_security.py` · 165 linhas · testes: 6

nome	tipo	assinatura	o que faz
`UnsafeModelError`	classe	`()`	Erro de checkpoint inseguro ou com digest divergente.
`ScanReport`	classe	`(path, globals, dangerous, unknown, errors)`	Relatório da varredura: globais encontrados, perigosos, desconhecidos e erros.
`scan_pickle_bytes`	função	`(data, name='<bytes>') -> ScanReport`	Varre um fluxo pickle em bytes sem executá-lo.
`scan_checkpoint`	função	`(path) -> ScanReport`	Varre um arquivo `torch.save` (cada membro `.pkl`) ou um pickle simples.
`safe_load`	função	`(path, map_location='cpu', expected_sha256=None)`	Verifica digest, varre e carrega com `torch.load(weights_only=True)`.
`adversarial_sensitivity`	função	`(model, X, eps=0.01, steps=20, step_frac=0.25, input_range=None) -> dict[str, Any]`	Pior mudança relativa da saída sob perturbação L∞ de tamanho `eps` (busca PGD).

`pinneapple_security.passwords`

Hash de senha com scrypt da biblioteca padrão por padrão, argon2id se `argon2-cffi` estiver instalado e verificação de hashes bcrypt.

`pinneapple_security/passwords.py` · 119 linhas · testes: 56

nome	tipo	assinatura	o que faz
`is_hash`	função	`(value) -> bool`	Diz se o valor parece um hash verificável, e não uma senha em texto.
`hash_password`	função	`(password, scheme='auto') -> str`	Hash salgado de uma senha por scrypt, argon2 ou o melhor disponível.
`verify_password`	função	`(password, stored) -> bool`	Verifica uma senha contra o hash guardado; formatos desconhecidos dão `False`.
`needs_rehash`	função	`(stored) -> bool`	Diz se o hash está mais fraco que os parâmetros atuais.

`pinneapple_security.privacy`

Encontra e-mails, telefones, IPs, cartões (Luhn), IBANs e CPF/CNPJ; redige, pseudonimiza por HMAC, mede k-anonimato e limpa logs; a detecção por regex tem falsos negativos.

`pinneapple_security/privacy.py` · 180 linhas · testes: 9

nome	tipo	assinatura	o que faz
`Finding`	classe	`(kind, start, end, text)`	Um achado de PII: tipo, posição e texto.
`valid_cpf`	função	`(s) -> bool`	Valida os dígitos verificadores de um CPF.
`valid_cnpj`	função	`(s) -> bool`	Valida os dígitos verificadores de um CNPJ.
`luhn_ok`	função	`(s) -> bool`	Valida um número pelo algoritmo de Luhn.
`iban_ok`	função	`(s) -> bool`	Valida um IBAN pelo módulo 97.
`find_pii`	função	`(text, kinds=PII_KINDS) -> list[Finding]`	Achados de PII sem sobreposição num texto.
`redact`	função	`(text, kinds=PII_KINDS, label=…) -> str`	Substitui cada achado de PII por um rótulo.
`pseudonymize`	função	`(value, key, prefix='id_', length=16) -> str`	Token determinístico com chave (HMAC-SHA256 truncado) para um valor.
`pseudonymize_column`	função	`(values, key, prefix='id_') -> list[str]`	Pseudonimiza uma coluna inteira.
`k_anonymity`	função	`(rows, quasi_identifiers, k=5) -> dict[str, Any]`	Mede o k da tabela pelos quase-identificadores e lista os grupos menores que `k`.
`sanitize_log`	função	`(text, extra=(), keep_paths=True) -> tuple[str, dict[str, int]]`	Remove de um log identificadores pessoais e de infraestrutura e conta o que trocou.

pinneapple_security.secrets_scan

Procura credenciais em textos e árvores de arquivos por regras e por entropia de Shannon.

pinneapple_security/secrets_scan.py · 116 linhas · testes: 10

nome	tipo	assinatura	o que faz
SecretFinding	classe	(rule, path, line, excerpt)	Um achado de credencial: regra, caminho, linha e trecho.
shannon_entropy	função	(s) -> float	Entropia de Shannon de uma string.
scan_text	função	(text, path='<text>', entropy_threshold=4.5) -> list[SecretFinding]	Procura credenciais num texto por regras e entropia.
scan_path	função	(root, exts=None, max_bytes=5000000, entropy_threshold=4.5) -> list[SecretFinding]	Procura credenciais em todos os arquivos sob um caminho.

pinneapple_security.signing

Assinaturas Ed25519 (precisa de `cryptography`) ou HMAC-SHA256 (biblioteca padrão) sobre bytes, JSON canônico em envelope DSSE e arquivos com assinatura `.sig` destacada.

pinneapple_security/signing.py · 189 linhas · testes: 25

nome	tipo	assinatura	o que faz
VerificationError	classe	()	Erro de assinatura inválida.
VerifyKey	classe	(algorithm, raw)	Metade pública (Ed25519) ou segredo compartilhado (HMAC) para verificar.
SigningKey	classe	(algorithm, raw)	Chave de assinatura Ed25519 ou HMAC, com `public()` e salvar/carregar.
pae	função	(payload_type, payload) -> bytes	Codificação de pré-autenticação DSSE.
generate_key	função	() -> SigningKey	Gera uma chave Ed25519 nova (precisa de `cryptography`).
hmac_key	função	(secret=None) -> SigningKey	Chave de segredo compartilhado (biblioteca padrão); quem verifica também pode assinar.
sign_bytes	função	(data, key) -> dict[str, str]	Assina bytes e devolve a assinatura em dicionário.
verify_bytes	função	(data, signature, key) -> None	Verifica a assinatura de bytes ou levanta `VerificationError`.
sign_json	função	(payload, key, payload_type=PAYLOAD_TYPE) -> dict[str, Any]	Envelope DSSE sobre o JSON canônico de um payload.
verify_json	função	(envelope, key) -> Any	Verifica um envelope DSSE e devolve o payload.
sign_file	função	(path, key, sig_path=None) -> str	Assinatura destacada `<path>.sig` sobre o SHA-256 e o nome do arquivo.
verify_file	função	(path, key, sig_path=None) -> dict[str, Any]	Verifica a assinatura destacada de um arquivo.

pinneapple_security.web

Endurecimento HTTP das apps web ASGI (FastAPI/Starlette) sem dependências além da biblioteca padrão na importação: middlewares, rate limit, bloqueio de login, CSRF, CORS, CSP e checagem de URLs de saída contra SSRF.

pinneapple_security/web.py · 496 linhas · testes: 40

nome	tipo	assinatura	o que faz
SecurityHeaders	classe	(app, csp=None, hsts=…, frame_options='DENY', extra=None)	Middleware ASGI que adiciona cabeçalhos de segurança (CSP, HSTS, frame, etc.) a toda resposta.
BodyLimit	classe	(app, max_bytes, limits=None)	Middleware ASGI que responde 413 para corpo acima do limite, contando durante o streaming.

nome	tipo	assinatura	o que faz
`RateRule`	classe	`(name, count, seconds, methods, prefixes)`	Regra de taxa: `count` requisições por `seconds`, filtrada por método e prefixo de caminho.
`RateLimiter`	classe	`(rules=(), max_keys=50000, clock=time.monotonic)`	Token buckets por regra e chave, com memória limitada.
`LoginThrottle`	classe	`(max_failures, window, lock_seconds, clock, max_keys, _fails, _locked, _lock)`	Bloqueia IP ou usuário por um tempo após falhas de login repetidas numa janela.
`UnsafeURL`	classe	`()`	Erro de URL que o servidor não deve buscar.
`AuthAudit`	classe	`(app_name, path=None, key=None)`	Eventos de autenticação e administração como uma linha JSON de log cada.
`client_ip`	função	`(peer, forwarded_for, trusted=DEFAULT_TRUSTED_PROXIES) -> str`	Endereço do cliente; `X-Forwarded-For` só vale vindo de proxy confiável.
`inline_hashes`	função	`(page) -> tuple[set[str], set[str]]`	Hashes CSP dos scripts inline e dos handlers inline de uma página HTML.
`build_csp`	função	`(script_hashes=(), handler_hashes=(), *, connect_src=(), img_src=(), frame_ancestors="'no…)`	Monta a Content-Security-Policy de uma página de app.
`parse_rate`	função	`(spec, default=None) -> tuple[float, float] | None`	Interpreta especificações de taxa como `30/60`, `30/min`, `5/s`; `off` dá `None`.
`same_origin_ok`	função	`(method, headers, allowed_origins=()) -> bool`	Checagem CSRF: falso para requisição de mudança de estado vinda de outro site.
`check_cors_origins`	função	`(origins, allow_credentials) -> list[str]`	Valida uma lista de origens CORS e recusa `*` com credenciais.
`check_frame_origins`	função	`(origins) -> list[str]`	Valida origens permitidas para embutir ou ser embutido.
`check_outbound_url`	função	`(url, allow_hosts=(), resolve=None) -> str`	Recusa URL que não seja http(s) para endereço público, salvo hosts permitidos (contra SSRF).

Exemplos de uso reais do repositório

Pipeline seguro: rotular, redigir, manifestar e cifrar — `examples/security/01_secure_physics_ai_pipeline.py`

```
            label = classification.DataLabel(level="confidential", owner="Plant A", personal_data=True)
            print("publish raw data?", classification.HandlingPolicy().check("publish", label).reasons)
            clean_notes = [privacy.redact(t) for t in notes]
            data_dir = os.path.join(out_dir, "dataset")
            os.makedirs(data_dir)
            np.savez(os.path.join(data_dir, "ops.npz"), q_in=q_in, speed=speed, q_out=q_out)
            with open(os.path.join(data_dir, "notes.txt"), "w") as f:
                f.write("\n".join(clean_notes))
            man = integrity.build_manifest(data_dir)
            man.save(os.path.join(out_dir, "dataset.manifest.json"))
            signing.sign_file(os.path.join(out_dir, "dataset.manifest.json"), key)
            data_key = encryption.new_key()                      # held by the plant's key manager, not stored with the
data
            encryption.encrypt_file(os.path.join(data_dir, "ops.npz"), data_key, out=os.path.join(out_dir,
"ops.npz.enc"))
```

Benchmark de injeção de dados falsos (três tanques) — `examples/security/02_physics_residual_fdi.py`

```
        from pinneapple_security.cps_bench import ATTACKS, run_benchmark


        def _fmt(v, nd=2):
            return "  -  " if v is None else f"{v:.{nd}f}"


        def main():
            ap = argparse.ArgumentParser(description=__doc__.splitlines()[0])
            ap.add_argument("--seeds", type=int, nargs="+", default=[0])
            ap.add_argument("--out", default=None)
```

```
a = ap.parse_args()
runs = [run_benchmark(seed=s) for s in a.seeds]
```

Testes e status de validação

- Módulos com pelo menos um teste que os importa: 20 de 20.
- Funções de teste em arquivos que importam algum módulo do pacote: 105.

Limitações conhecidas

- Ed25519 e AES-GCM precisam de `cryptography` (`pip install pinneapple[security]`); `DPSGD`, `safe_load` e `adversarial_sensitivity` precisam de PyTorch; argon2id precisa de `argon2-cffi`; `IsolationForestDetector` precisa de scikit-learn.
- `compliance` mapeia capacidades para controles e gera autoavaliação, mas não certifica: os controles são cumpridos pelos processos da organização.
- `audit` sem chave detecta acidentes e edições ingênuas, não um falsificador que recalcula todos os hashes; também não é um sistema certificado de 21 CFR Part 11.
- A detecção de PII por regex tem falsos negativos (nomes e endereços em texto livre): é uma rede de segurança, não garantia de anonimato.
- Com chave HMAC (`hmac_key`), quem pode verificar também pode assinar; o docstring recomenda usar só dentro de uma organização.
- `gaussian_sigma`/`gaussian_mechanism` usam a calibração clássica, válida para $\varepsilon \leq 1$; os limites de `dp_mean`/`dp_histogram` não podem vir dos próprios dados.
- O ataque `coordinated_replay` do `cps_bench` é fisicamente consistente e invisível ao detector de resíduo físico (o docstring diz que é relatado para mostrar o limite e é pego por `anomaly.replay_score`).
- `HandlingPolicy` traz regras padrão a ajustar por organização, e o `DataLabel` registra o que o dono declarou, sem decidir a classificação.

5. `pinneapple_decision`

Objetivo

`pinneapple_decision` é uma camada de decisão probabilística sobre o pipeline de Physics AI do PINNeAPPle. Ela não prevê resultados físicos: decide qual experimento rodar a seguir (qual modelo, qual estratégia de treino, como validar). Um LLM ou um backend de regras offline dá escores a um conjunto fechado de opções; não há geração nem interpretação de texto livre.

O fluxo é OBSERVAR -> ENTENDER -> DECIDIR -> EXECUTAR -> VALIDAR -> APRENDER. O `adapter` converte o problema (um `ProblemSpec` de `pinneapple_problemdesign` ou um dict livre) em fatos canônicos com proveniência. As restrições excluem opções inviáveis com motivo (geometria não suportada, sem implementação no PINNeAPPle, fora do orçamento, licença só para pesquisa em modo comercial, já falhou). O `PhysicsDecider` pontua as opções restantes e remove viés de posição e de rótulo sem rótulos (nível L0, inspirado no AnyJev). Cada resultado executado passa por um verificador (por limiares ou VeriPhysics) e só resultados verificados viram evidência.

O pacote liga-se ao resto pelo `pp.decide` (API em `api.py`), pela recomendação de arquitetura de `pinneapple_analysis` (carregada de forma preguiçosa pelo `ModelSelector`) e pelo VeriPhysics como verificador. A árvore `physics_ai_tree` encadeia perguntas de fato (há solução analítica? há dados de referência?) com escolhas e validações. Importar o pacote não carrega torch.

Números do inventário

módulos	linhas	classes públicas	funções públicas	testes em arquivos que importam o pacote	módulos sem teste direto
12	2.499	37	20	44 (3 arquivos)	9

Submódulos

submódulo	o que faz	módulos	classes + funções	testes*
`adapter`	Converte `ProblemSpec` e dicts livres em fatos canônicos com proveniência.	1	2	0
`api`	API `pp.decide/execute/verify/run_tree` sobre um motor padrão.	1	5	16
`backends`	Backends de pontuação por regras ou por log-probabilidade de LLM.	1	3	0
`constraints`	Restrições duras que excluem opções com motivo.	1	5	0
`debias`	Remoção de viés L0 por rotações cíclicas e prior.	1	8	21
`decider`	Decisor que combina restrições, pontuação e remoção de viés.	1	1	0
`loop`	Laço decidir-executar-verificar e fachada `PhysicsDecisionEngine`.	1	7	0
`schema`	Tipos de escolha, decisão, resultado, verificação e evidência.	1	10	0
`selectors`	Seletores de modelo, treino e validação, e perguntas de fato.	1	4	0
`tree`	Árvore de decisão declarativa do fluxo de Physics AI.	1	9	0
`verifiers`	Verificadores por limiar e via VeriPhysics.	1	3	0

* maior número de testes que importam um módulo do submódulo.

Módulos e API pública

`pinneapple_decision`

Fachada do pacote: reexporta tipos, decisor, backends, laço de decisão, seletores, árvore, verificadores e a API `decide/execute/verify/run_tree`; importar o pacote não carrega torch.

`pinneapple_decision/__init__.py` · 116 linhas · testes: 44

`pinneapple_decision.adapter`

Adaptador de problema: converte `ProblemSpec` reais e dicts livres (por exemplo `reynolds`, `geometry`, `target`) nas chaves canônicas que regras e restrições leem, registrando o que foi dado, inferido (com a regra) ou desconhecido. [sem teste direto]

`pinneapple_decision/adapter.py` · 508 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`AdaptedProblem`	classe	`(problem, given, inferred, unknown, unrecognized, source)`	Resultado da adaptação: problema canônico e o que foi dado, inferido, desconhecido ou não reconhecido, com relatório de proveniência.
`adapt_problem`	função	`(obj) -> AdaptedProblem`	Adapta um `ProblemSpec`, um dict livre ou None para a camada de decisão; é idempotente em dicts canônicos.

`pinneapple_decision.api`

API de nível de módulo usada por `pp.decide(...)`: mantém um motor padrão e expõe `decide`, `execute`, `verify` e `run_tree`.

`pinneapple_decision/api.py` · 59 linhas · testes: 16

nome	tipo	assinatura	o que faz
`decision_engine`	função	`(executor=None, verifier=None, decider=None, store=None, *, reset=False) -> PhysicsDecisi…)`	Devolve o motor padrão; qualquer argumento (ou `reset=True`) cria um novo, sem carregar a evidência antiga.
`decide`	função	`(problem=None, objective=None, *, state=None, choice=None, engine=None) -> Decision`	Decide qual experimento rodar a seguir para o problema e objetivo dados.
`execute`	função	`(decision, problem=None, *, engine=None) -> ExecutionResult`	Executa uma decisão emitida pelo mesmo motor, rechecando as restrições antes.
`verify`	função	`(result, *, engine=None) -> Verification`	Verifica um resultado; só resultados verificados viram evidência.
`run_tree`	função	`(problem, tree=None, *, max_experiments=3, engine=None)`	Percorre a árvore de decisão de Physics AI (por padrão `physics_ai_tree()`).

pinneapple_decision.backends

Backends de pontuação: transformam (estado, escolha, opções em ordem) em um escore por opção, por regras determinísticas ou por log-probabilidade de um LLM; nunca geram texto. [sem teste direto]

`pinneapple_decision/backends.py` · 153 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ScoringBackend`	classe	`(name, has_label_prior)`	Protocolo de backend: `score` devolve um escore por opção na ordem mostrada e `rationale` explica.
`RuleBasedBackend`	classe	`(rules=None)`	Backend determinístico e offline que usa as regras dos seletores; invariante à ordem e sem prior de rótulo.
`LogProbLLMBackend`	classe	`(logprob_fn, *, template=DEFAULT_TEMPLATE, name='llm-logprob')`	Pontua cada opção pela log-probabilidade do próximo token da sua letra num prompt; `from_transformers` cria um a partir de um LM local do Hugging Face (opcional).

pinneapple_decision.constraints

Restrições duras sobre opções (geometria suportada, implementação existente, orçamento, licença comercial, falha anterior): excluem com motivo legível, nunca reordenam; rodam antes da pontuação e antes da execução. [sem teste direto]

`pinneapple_decision/constraints.py` · 137 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Constraint`	classe	`(name, check)`	Restrição nomeada com uma função `check` que devolve None (mantém) ou um motivo (exclui).
`NoFeasibleOptionError`	classe	`(choice, excluded)`	Erro quando todas as opções de uma escolha foram excluídas pelas restrições.
`ConstraintViolationError`	classe	`()`	Erro quando a opção de uma decisão deixa de passar nas restrições no momento da execução.
`resolve`	função	`(constraint) -> Constraint`	Converte um nome de restrição registrado (ou um objeto) em `Constraint`.
`apply_constraints`	função	`(choice, state) -> Tuple[List[str], Dict[str, str]]`	Devolve as opções viáveis na ordem da escolha e um dict de opções excluídas com o motivo.

pinneapple_decision.debias

Remoção de viés nível L0 sem rótulos (inspirada no AnyJev, implementação própria): média sobre rotações cíclicas da ordem das opções e correção por prior.

`pinneapple_decision/debias.py` · 161 linhas · testes: 21

nome	tipo	assinatura	o que faz
`BatchPrior`	classe	`(min_prior_n=8)`	Média corrente das distribuições L0 por (escolha, conjunto de opções viáveis), usada como prior a partir de um mínimo de amostras.
`log_softmax`	função	`(xs) -> List[float]`	Log-softmax numericamente estável de uma lista de escores.
`softmax_dict`	função	`(logits) -> Dict[str, float]`	Softmax de um dict opção -> logit.
`normalize`	função	`(p) -> Dict[str, float]`	Normaliza um dict de pesos para somar 1.
`rotations`	função	`(options, max_permutations=None) -> List[List[str]]`	Lista as rotações cíclicas da ordem das opções, opcionalmente limitadas e igualmente espaçadas.
`raw_distribution`	função	`(score_fn, options) -> Dict[str, float]`	Distribuição de uma única ordem das opções (softmax dos escores), sem remoção de viés.
`permutation_marginalized`	função	`(score_fn, options, *, combine='geometric', max_permutations=None) -> Tuple[Dict[str, flo…)`	Distribuição média sobre rotações cíclicas, combinando por média geométrica (padrão) ou aritmética, com diagnósticos.

nome	tipo	assinatura	o que faz
apply_prior	função	(dist, prior, strength=1.0) -> Dict[str, float]	Corrige uma distribuição dividindo por um prior elevado a strength e renormalizando.

pinneapple_decision.decider

PhysicsDecider: aplica restrições, pontua, remove viés (L0) e produz uma Decision com distribuição sobre as opções viáveis. [sem teste direto]

pinneapple_decision/decider.py · 119 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
PhysicsDecider	classe	(backend=None, *, debias=True, prior='content_free', prior_strength=1.0, combine='geometr…)	Decide a opção de uma PhysicsChoice: restrições, pontuação pelo backend, L0 (ou raw) e montagem da Decision.

pinneapple_decision.loop

Laço OBSERVAR -> ENTENDER -> DECIDIR -> EXECUTAR -> VALIDAR -> APRENDER, com garantias: só decisões emitidas pelo laço executam, restrições são rechecadas e só resultados verificados viram evidência. [sem teste direto]

pinneapple_decision/loop.py · 215 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
ValidationRequiredError	classe	()	Erro quando um resultado é oferecido como evidência antes de ser verificado.
Executor	classe	()	Protocolo de executor: execute(decision, problem) devolve um ExecutionResult.
CallableExecutor	classe	(fn)	Executor que embrulha uma função fn(decision, problem) que devolve ExecutionResult ou dict.
EvidenceStore	classe	(path=None)	Registro de evidências em memória, opcionalmente salvo em JSON.
DecisionLoop	classe	(executor=None, verifier=None, decider=None, store=None)	Laço completo decidir/executar/verificar com histórico de evidências, execução em passos e percurso de árvore.
PhysicsDecisionEngine	classe	()	Fachada de alto nível: decide(problem, objective=...) ou decide(state=...), mais execute e verify.
choice_for_objective	função	(objective, has_previous_result=False) -> PhysicsChoice	Escolhe a PhysicsChoice adequada a partir do texto do objetivo (padrão: modelo, ou estratégia de treino se já há resultado).

pinneapple_decision.schema

Tipos centrais da camada de decisão: níveis de decisão, metadados de opção, escolhas, resultados de execução, verificações, decisões, evidências e estado. [sem teste direto]

pinneapple_decision/schema.py · 284 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
DecisionLevel	classe	()	Nível de leitura dos escores: RAW (ranking), L0 (sem viés, sem rótulos), L1 (calibrado, ainda não implementado) e FACT (resposta lida do problema).
OptionInfo	classe	(name, description, implementation, representations, supports_varying_geometry, relative_…)	Metadados de uma opção (implementação real, representações de geometria, custo relativo, fonte, licença), usados por regras e restrições.
PhysicsChoice	classe	(name, question, options, constraints, option_info)	Pergunta tipada com conjunto fechado de opções e restrições.
ExecutionResult	classe	(status, metrics, diagnostics, artifacts, decision_id)	O que um executor devolve: status, métricas, tags de diagnóstico, artefatos e id da decisão.

nome	tipo	assinatura	o que faz
`Verification`	classe	`(passed, checks, tags, source, notes)`	Resultado de verificar uma execução: passou ou não, checagens que de fato rodaram, tags, fonte e notas.
`Decision`	classe	`(choice, question, selected, probabilities, level, requires_validation, calibrated_confid…)`	Decisão: opção escolhida, probabilidades, nível, exclusões, justificativa e diagnósticos; `calibrated_confidence` só é aceito no nível L1.
`Evidence`	classe	`(decision, result, verification)`	Um registro decisão + resultado + verificação.
`DecisionState`	classe	`(problem, previous_result, verification, history, facts)`	Tudo que um backend pode ver: o problema adaptado, o resultado anterior, sua verificação e o histórico.
`problem_from_spec`	função	`(spec) -> Dict[str, Any]`	Mapeia um `ProblemSpec` de `pinneapple_problemdesign` para o dict de problema, sem inventar valores numéricos.
`dumps`	função	`(obj) -> str`	Serializa objetos da camada de decisão em JSON.

`pinneapple_decision.selectors`

As três decisões do MVP (modelo, estratégia de treino, estratégia de validação) e perguntas de fato da árvore, cada uma com opções, metadados e regras de pontuação. [sem teste direto]

`pinneapple_decision/selectors.py` · 358 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ModelSelector`	classe	`()`	Escolha do modelo físico (PINN, XPINN, DeepONet, FNO, PINO, MeshGraphNet etc.) com metadados por opção e regras.
`TrainingStrategySelector`	classe	`()`	Escolha da próxima estratégia de treino a partir dos diagnósticos do resultado anterior.
`ValidationStrategySelector`	classe	`()`	Escolha da estratégia de validação do resultado.
`FactQuestion`	classe	`(name, question, fact, hints=None)`	Pergunta sim/não sobre um fato do problema; as regras só rodam quando o fato não é conhecido.

`pinneapple_decision.tree`

Árvore de decisão declarativa do fluxo de Physics AI (solução analítica? dados de referência? ...), percorrida pelo `DecisionLoop`; nós de fato são respondidos pelo problema quando conhecido. [sem teste direto]

`pinneapple_decision/tree.py` · 297 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ChoiceNode`	classe	`(id, choice, edges, fact, fact_options, plan_key, plan_updates)`	Nó da árvore que faz uma escolha (ou lê um fato) e segue a aresta da opção selecionada.
`ExecuteNode`	classe	`(id, next, plan_updates)`	Nó da árvore que executa o plano atual.
`ValidateNode`	classe	`(id, on_pass, on_fail, plan_updates)`	Nó da árvore que verifica o último resultado e segue para `on_pass` ou `on_fail`.
`TerminalNode`	classe	`(id, outcome, plan_updates)`	Nó final da árvore com o desfecho.
`DecisionTree`	classe	`(name, root, nodes)`	Árvore de decisão: nome, nó raiz e nós por id.
`TreeStep`	classe	`(node, kind, selected, resolved_by, decision_id, passed, note)`	Um passo do percurso: nó, tipo, opção escolhida, quem resolveu (fato ou decisor) e resultado da validação.
`TreeRun`	classe	`(tree, outcome, path, plan, experiments, evidence, facts)`	Resultado do percurso: desfecho, caminho, plano, experimentos, evidências e fatos.
`physics_ai_tree`	função	`() -> DecisionTree`	Constrói a árvore padrão do fluxo de Physics AI descrita no docstring do módulo.
`run_tree`	função	`(loop, problem, tree=None, *, max_experiments=3, max_steps=200) -> TreeRun`	Percorre uma árvore com um `DecisionLoop`, até um nó terminal ou os limites de experimentos e passos.

`pinneapple_decision.verifiers`

Verificadores que transformam `ExecutionResult` em `Verification`: por limiares de métricas ou a partir de um registro do VeriPhysics/`PhysicsConfidenceScore`. [sem teste direto]

`pinneapple_decision/verifiers.py` · 92 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Verifier`	classe	`()`	Protocolo de verificador: `verify(result)` devolve uma `Verification`.
`ThresholdVerifier`	classe	`(thresholds=None)`	Exige cada métrica configurada <= seu limiar; métrica ausente é checagem que não rodou, e nenhuma checagem significa não aprovado.
`VeriPhysicsVerifier`	classe	`(min_trust=70.0, min_coverage=0.5, fallback=None)`	Lê um registro do VeriPhysics ou um `PhysicsConfidenceScore` nos artefatos e aprova por confiança e cobertura mínimas; sem esse artefato usa o verificador de fallback.

Exemplos de uso reais do repositório

Restrição de geometria exclui FNO/PINO — `tests/test_decision_adapter_tree_arena.py`

```python
def test_user_airfoil_example_excludes_fno_pino_with_reason():
    d = PhysicsDecider().decide(USER_AIRFOIL, ModelSelector.choice())
    for opt in ("fno", "pino"):
        assert opt in d.excluded and opt not in d.probabilities
        assert "must_support_geometry" in d.excluded[opt]
```

Motor de decisão com executor próprio — `pinneapple_decision/README.md`

```
engine = PhysicsDecisionEngine(executor=CallableExecutor(run_experiment))  # your function
d = engine.decide(airfoil, objective="choose physics model")
result = engine.execute(d)             # constraints are re-checked here
verification = engine.verify(result)   # ThresholdVerifier or VeriPhysicsVerifier
```

Testes e status de validação

- Módulos com pelo menos um teste que os importa: 3 de 12.
- Funções de teste em arquivos que importam algum módulo do pacote: 44.

Limitações conhecidas

- O nível L1 (confiança calibrada em resultados rotulados) não está implementado; as probabilidades L0 não são confiança calibrada.
- Pacote descrito como MVP no README: só três decisões (modelo, estratégia de treino, estratégia de validação) além das perguntas de fato da árvore.
- `LogProbLLMBackend.from_transformers` depende de um LM local do Hugging Face (opcional); o backend padrão é o de regras.
- `VeriPhysicsVerifier` só lê um registro já presente nos artefatos do resultado; sem ele, recorre ao verificador de fallback (`ThresholdVerifier` por padrão) e diz isso em `source`.
- Opções sem implementação no PINNeAPPle são excluídas pela restrição `requires_implementation`.
- A maioria dos módulos não tem teste direto; os testes importam o pacote pela fachada e cobrem sobretudo `api` e `debias`.

6. pinneapple_reason

Objetivo

pinneapple_reason calcula a cadeia de porquês de um projeto de engenharia. Parte de requisitos e de um conceito ingênuo (por exemplo telhas planas, ou uma bomba acima do poço) e repete o laço: checagem física com números -> primeiro modo de falha encontrado (mecanismo + o número que o prova) -> contramedida (novo elemento, com custo e efeito esperado) -> checar tudo de novo, até que todos os requisitos passem. O resultado é uma Chain com cada versão do projeto e o motivo de cada mudança.

O pacote tem um modelo de dados (chain), uma biblioteca de modos de falha e contramedidas por domínio físico (library e modes: estruturas, calor, água em telhados, fluidos em tubos, corrosão), o motor (engine), relatórios em Markdown/HTML com figuras (report, narrative), o relatório de confiança (trust) e casos prontos (cases: telhado, passarela, placa de potência, adutora bombeada). Os textos existem em português e inglês.

Um LLM pode atuar só como conselheiro (llm.LLMAdvisor): propõe nomes de contramedidas da biblioteca ou um conceito inicial, mas nunca fornece um número que decida algo; toda margem, custo e mecanismo vem das checagens.

Liga-se ao resto do repositório pelo TrustReport de pinneapple_analysis.trust e pelo cartão de seis perguntas de pinneapple_lab.curation, por solvers e fórmulas de pinneapple_simulation (golpe de aríete por MOC) e pinneapple_physics (placa de circuito, dissipador), e é usado pelo nó reason do pinneapple_flow, pelas ferramentas de pinneapple_mcp, pelo workbench de pinneapple_llm e pelos presets industriais de pinneapple_presets.

Números do inventário

módulos	linhas	classes públicas	funções públicas	testes em arquivos que importam o pacote	módulos sem teste direto
18	3.757	17	60	17 (1 arquivos)	8

Submódulos

submódulo	o que faz	módulos	classes + funções	testes*
__main__	CLI python -m pinneapple_reason: roda um caso, lista a biblioteca ou explica uma cadeia salva.	1	1	0
cases	Casos prontos em escalas e áreas diferentes, cada um a partir do conceito ingênuo.	1	5	17
chain	Modelo de dados da cadeia de porquês (requisito, conceito, verificação, achado, opção, passo, cadeia) e textos bilíngues.	1	13	17
engine	Motor do raciocínio: checa tudo, acha a primeira falha, testa contramedidas, escolhe a melhor e repete.	1	3	0
library	Biblioteca de modos de falha e contramedidas, com registro, validade e casos de resposta conhecida.	1	11	17
llm	Conselheiro LLM que só propõe contramedidas da biblioteca e um conceito inicial, filtrados.	1	2	17
modes	Modos de falha e contramedidas embutidos, um módulo por domínio físico.	7	20	17
narrative	Conta a cadeia em palavras, um parágrafo por versão com o número que prova cada afirmação.	1	2	0
props	Constantes, propriedades e relações de forma fechada compartilhadas pelos modos de falha, com fonte.	1	6	17
report	Relatórios da cadeia (chain.json, report.md, report.html, trust.json) e figuras.	1	4	0
trust	Confiança da cadeia como TrustReport e como cartão de seis estágios.	1	5	0

* maior número de testes que importam um módulo do submódulo.

Módulos e API pública

`pinneapple_reason`

Raiz do pacote: reexporta o modelo de dados, o motor e a biblioteca e oferece `write_report`, `trust_report`, `run_case`, `verify` e `describe_library`.

`pinneapple_reason/__init__.py` · 77 linhas · testes: 17

nome	tipo	assinatura	o que faz
`write_report`	função	`(chain, out_dir, lang='pt', **kw)`	Grava chain.json, report.md, report.html, figuras e cartão de confiança em `out_dir`.
`trust_report`	função	`(chain, **kw)`	Atalho para o `TrustReport` da cadeia (ver `pinneapple_reason.trust`).
`run_case`	função	`(name, **kw) -> Chain`	Roda o raciocínio de um dos casos de `cases.CASES`.
`verify`	função	`(concept, requirements) -> list[Verification]`	Roda todos os modos de falha dos requisitos sobre um conceito, sem contramedidas.
`describe_library`	função	`(lang='pt') -> dict`	Descreve a biblioteca: modos de falha por domínio e suas contramedidas.

`pinneapple_reason.__main__`

CLI `python -m pinneapple_reason run|modes|explain`. [sem teste]

`pinneapple_reason/__main__.py` · 45 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`main`	função	`(argv=None) -> int`	Ponto de entrada da CLI com os subcomandos run, modes e explain.

`pinneapple_reason.cases`

Quatro casos prontos (telhado, passarela de aço, placa com MOSFET, adutora bombeada) que devolvem requisitos, conceito ingênuo e título.

`pinneapple_reason/cases.py` · 150 linhas · testes: 17

nome	tipo	assinatura	o que faz
`roof`	função	`()`	Caso telhado de casa em clima quente e úmido.
`footbridge`	função	`()`	Caso passarela de aço de 30 m com duas vigas soldadas.
`power_board`	função	`()`	Caso chave MOSFET de 20 A numa placa de 4 camadas dentro de um gabinete a 40 °C.
`pumped_main`	função	`()`	Caso estação de bombeamento do poço ao reservatório e adutora por gravidade até uma válvula.
`get`	função	`(name)`	Devolve o caso pelo nome; nome desconhecido levanta KeyError.

`pinneapple_reason.chain`

Modelo de dados serializável da cadeia de porquês e auxiliares de texto em português e inglês.

`pinneapple_reason/chain.py` · 409 linhas · testes: 17

nome	tipo	assinatura	o que faz
`Requirement`	classe	`(id, text, modes, limits, source)`	O que o projeto deve cumprir e os modos de falha que o impediriam.
`Concept`	classe	`(name, params, elements, description, sources, version)`	Uma versão do projeto: parâmetros em SI, elementos presentes e origem de cada valor.
`Validity`	classe	`(name, value, lo, hi, note)`	Uma condição de aplicabilidade do modelo usado numa checagem (`lo <= value <= hi`).
`Verification`	classe	`(mode, requirement, passed, demand, capacity, unit, margin, demand_label, capacity_label,…)`	Resultado de um modo de falha checado num conceito: demanda, capacidade, margem, mecanismo, método e fontes.

nome	tipo	assinatura	o que faz
`Finding`	classe	`(requirement, mode, title, verification)`	O primeiro modo de falha que ocorre, o requisito que quebra e o número que o prova.
`Option`	classe	`(countermeasure, element, title, feasible, reason, changes, cost, effect, target_margin_b…)`	Uma contramedida testada: o que mudou, custo, efeito nas margens de todos os requisitos ou por que não se aplica.
`Step`	classe	`(version, concept, verifications, finding, options, chosen, why)`	Uma versão do projeto com todas as checagens, a primeira falha, as opções testadas e a escolhida.
`Chain`	classe	`(title, requirements, steps, status, notes, meta)`	O raciocínio inteiro: requisitos, versões e porquês; explica, salva e carrega.
`tr`	função	`(pt, en) -> dict[str, str]`	Cria um texto em português e inglês.
`text_of`	função	`(t, lang='pt') -> str`	Devolve o texto no idioma pedido, com recuo para o outro idioma.
`param_text`	função	`(name, value, lang='pt') -> str`	Formata `nome = valor unidade` com o nome do parâmetro no idioma.
`element_label`	função	`(name, lang='pt') -> str`	Nome de um elemento no idioma.
`cost_text`	função	`(cost, lang='pt') -> str`	Descreve em palavras o custo de uma contramedida.

pinneapple_reason.engine

Reasoner e reason: o laço checar -> primeira falha -> contramedidas -> escolher a mais barata que corrige sem quebrar nada -> checar. [sem teste direto]

pinneapple_reason/engine.py · 211 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Advisor`	classe	`()`	Protocolo de conselheiro que propõe nomes de contramedidas para um achado.
`Reasoner`	classe	`(library=LIBRARY, advisor=None, max_steps=12, progress=None)`	Executa o laço de raciocínio sobre a biblioteca, com cache de verificações e limite de passos.
`reason`	função	`(requirements, concept, title='', *, advisor=None, max_steps=12, library=LIBRARY, progres…)`	Constrói a cadeia de porquês para os requisitos a partir do conceito ingênuo.

pinneapple_reason.library

Classes FailureMode, Countermeasure, Library e o registro global LIBRARY, com auxiliares de validade e checagens internas.

pinneapple_reason/library.py · 246 linhas · testes: 17

nome	tipo	assinatura	o que faz
`Infeasible`	classe	`(pt, en)`	Exceção: a contramedida não pode ser aplicada a este conceito (mensagem bilíngue).
`Need`	classe	`(name, unit, label)`	Um parâmetro que um modo de falha lê, com unidade SI e significado.
`FailureMode`	classe	`(name, domain, title, summary, evaluate_fn, needs, sources, countermeasures, uncertain, k…)`	Um modo de falha e sua checagem quantitativa, com fontes e casos de resposta conhecida.
`Countermeasure`	classe	`(name, element, title, effect, addresses, apply_fn, cost_fn, unit_cost, unit_cost_note, r…)`	Uma mudança que atua no mecanismo de um modo de falha, com aplicação, custo, requisitos e exclusões.
`Library`	classe	`()`	Coleção de modos e contramedidas, consultável por modo e domínio.
`param`	função	`(p, name, default=None) -> Any`	Lê um parâmetro do dicionário; sem valor padrão, levanta KeyError se faltar.
`self_check`	função	`(name, passed, value=None, limit=None, source='', reason='') -> Check`	Checagem interna de uma conta (resíduo, fechamento, concordância com solução independente).
`register_mode`	função	`(m, replace=False) -> FailureMode`	Registra um modo de falha na biblioteca global.

nome	tipo	assinatura	o que faz
`register_countermeasure`	função	`(c, replace=False) -> Countermeasure`	Registra uma contramedida na biblioteca global.
`validity`	função	`(name, value, lo=None, hi=None, note='') -> Validity`	Cria uma condição de validade `Validity`.
`iter_known_answers`	função	`(names=None)`	Itera pares (modo, caso) de resposta conhecida da biblioteca.

pinneapple_reason.llm

`LLMAdvisor`: envolve um objeto com `extract(text, schema)` e filtra o que ele devolve para nomes da biblioteca e parâmetros declarados.

`pinneapple_reason/llm.py` · 63 linhas · testes: 17

nome	tipo	assinatura	o que faz
`ExtractLLM`	classe	`()`	Protocolo de LLM com `extract(text, schema) -> dict`.
`LLMAdvisor`	classe	`(llm, max_proposals=3)`	Conselheiro LLM: propõe contramedidas existentes e um conceito inicial marcado como premissa.

pinneapple_reason.modes

Pacote dos modos embutidos, importados no primeiro uso da biblioteca. [sem teste direto]

`pinneapple_reason/modes/__init__.py` · 6 linhas · **sem teste direto**

pinneapple_reason.modes._util

Auxiliares dos modos: veredito demanda contra capacidade com margem, dimensionamento até passar e formatação.

`pinneapple_reason/modes/_util.py` · 69 linhas · testes: 17

nome	tipo	assinatura	o que faz
`verdict`	função	`(*, demand, capacity, unit, scale=None, demand_label, capacity_label, mechanism, method, …)`	Cria uma verificação que passa quando a demanda é menor ou igual à capacidade, com margem relativa.
`q`	função	`(value, unit, label) -> list`	Empacota uma grandeza como [valor, unidade, rótulo].
`target_margin`	função	`(p) -> float`	Margem de projeto alvo do caso (`design_margin`).
`size_to_pass`	função	`(mode, p, values, make, limits=None, add=()) -> tuple[dict[str, Any], Any]`	Primeiro candidato da lista cujas mudanças fazem o modo passar com a margem de projeto.
`fmt`	função	`(x, nd=3) -> str`	Formata um número com algarismos significativos (infinito como ∞).

pinneapple_reason.modes.corrosion

Corrosão galvânica entre dois metais num eletrólito aerado, com contramedidas de kit isolante e revestimento do cátodo. [sem teste direto]

`pinneapple_reason/modes/corrosion.py` · 91 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`faraday_rate`	função	`(i, metal) -> float`	Taxa de perda de parede de um metal pela lei de Faraday a partir da densidade de corrente.

pinneapple_reason.modes.fluids

Líquidos em tubos: cavitação de bomba (NPSH), golpe de aríete no fechamento de válvula e separação de coluna, via MOC. [sem teste direto]

pinneapple_reason/modes/fluids.py · 328 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
npsh	função	(p) -> dict	Calcula NPSH disponível e requerido da sucção da bomba (pressão atmosférica, vapor, perdas).
main_wave_speed	função	(p) -> float	Velocidade da onda de pressão na adutora pela elasticidade do tubo e da água.
transient	função	(p, instantaneous_frictionless=False) -> dict	Simula o transiente de fechamento de válvula pelo método das características.

pinneapple_reason.modes.structural

Estruturas: flexão, flecha, flambagem lateral-torcional e ressonância por pedestres de vigas de aço, flambagem de coluna, fadiga e sucção de vento em telhado.

pinneapple_reason/modes/structural.py · 571 linhas · testes: 17

nome	tipo	assinatura	o que faz
i_section	função	(h, b, tf, tw) -> dict	Propriedades de uma seção I (área, inércias, torção, empenamento, módulo).
crowd_load	função	(span) -> float	Carga distribuída de multidão de passarela pela EN 1991-2.
girder_loads	função	(p) -> dict	Cargas permanentes e variáveis por viga a partir dos parâmetros do caso.
chi	função	(lam, alpha) -> float	Fator de redução por esbeltez da EN 1993-1-1.
m_cr	função	(sec, L_cr, C1=1.0) -> float	Momento crítico elástico de viga I duplamente simétrica.
frequency	função	(p) -> dict	Primeira frequência de flexão da passarela e massa modal.
pedestrian_response	função	(p) -> dict	Resposta da passarela à excitação de pedestres, com amortecedor de massa sintonizado opcional.

pinneapple_reason.modes.thermal

Calor: condensação sob o telhado em noite clara, ganho radiante de telhado ao sol e aquecimento Joule de um dispositivo de potência.

pinneapple_reason/modes/thermal.py · 547 linhas · testes: 17

nome	tipo	assinatura	o que faz
roof_network	função	(p, case) -> dict	Temperaturas e fluxos estacionários do telhado no caso noite ou dia.
electrothermal	função	(p) -> dict	Temperatura de junção com P = I² R_on(T_j) no caminho térmico do conceito, por ponto fixo.

pinneapple_reason.modes.water

Água: chuva atravessando as sobreposições das telhas (capilaridade, pressão do vento, lâmina d'água).

pinneapple_reason/modes/water.py · 213 linhas · testes: 17

nome	tipo	assinatura	o que faz
lap_heads	função	(p) -> dict	Cargas de água atuando na sobreposição das telhas e a subida da sobreposição.
minimum_slope	função	(p, margin=0.0) -> float	Menor inclinação em que a sobreposição segura a carga com margem, por bissecção.

pinneapple_reason.narrative

Texto explicativo da cadeia, versão por versão, com as opções testadas. [sem teste direto]

pinneapple_reason/narrative.py · 60 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
option_line	função	(o, lang) -> str	Linha de texto de uma opção testada: se corrige, margens antes e depois, custo e o que quebra.
explain	função	(chain, lang='pt') -> str	Texto completo da cadeia: requisitos e cada versão com sua falha e escolha.

pinneapple_reason.props

Propriedades do ar, pressão de saturação, ponto de orvalho, temperatura do céu, convecção por vento e atmosfera padrão, cada uma com fonte.

pinneapple_reason/props.py · 76 linhas · testes: 17

nome	tipo	assinatura	o que faz
air	função	(t_c, p_pa=101325.0) -> dict[str, float]	Propriedades do ar numa temperatura (k, nu, Pr, rho, cp, alpha, beta).
saturation_pressure	função	(t_c) -> float	Pressão de vapor de saturação da água (forma de Magnus).
dew_point	função	(t_c, rh) -> float	Temperatura de ponto de orvalho do ar com umidade relativa dada.
sky_temperature	função	(t_air_k) -> float	Temperatura efetiva de céu claro (Swinbank 1963).
wind_convection	função	(v) -> float	Coeficiente de convecção externa com vento (Watmuff et al. 1977).
atmospheric_pressure	função	(z_m) -> float	Pressão da atmosfera padrão na altitude (ISO 2533).

pinneapple_reason.report

Gera chain.json, report.md, report.html autocontido, trust.json e figuras (linha do tempo, margens, demanda x capacidade). [sem teste direto]

pinneapple_reason/report.py · 487 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
timeline_figure	função	(chain, path, lang='pt') -> Path	Figura da linha do tempo: cartões das versões e setas com o que falhou e o que foi acrescentado.
margins_figure	função	(chain, path, lang='pt') -> Path	Figura das margens de cada checagem em cada versão.
demand_capacity_figure	função	(chain, path, lang='pt') -> Path	Figura de demanda contra capacidade de cada modo ao longo das versões.
write_report	função	(chain, out_dir, lang='pt', card=True) -> dict[str, Path]	Grava a cadeia, as figuras, o relatório de confiança e os relatórios; devolve os caminhos.

pinneapple_reason.trust

Checagens de resposta conhecida, de incerteza das premissas e da versão final, viradas em `TrustReport` e cartão de seis estágios. [sem teste direto]

pinneapple_reason/trust.py · 108 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
known_answer_checks	função	(modes) -> list[Check]	Checa cada modo contra seus casos de resposta conhecida.
uncertainty_checks	função	(chain) -> list[Check]	Pior margem de cada checagem final quando uma premissa incerta varia pela sua dispersão relativa.
chain_checks	função	(chain) -> list[tuple[Check, str]]	Pares (checagem, tipo) da versão final da cadeia.
trust_report	função	(chain, **kw) -> TrustReport	`TrustReport` com as checagens da cadeia.

nome	tipo	assinatura	o que faz
`trust_card`	função	`(chain, out_dir='.', report=None) -> dict[str, dict[str, Any]]`	Responde as seis perguntas do cartão de confiança a partir das checagens da cadeia.

Exemplos de uso reais do repositório

Cadeia de porquês dos quatro casos com relatório e confiança — `examples/chain_of_whys/chain_of_whys.py`

```
for name in a.case or list(cases.CASES):
    print(f"\n=== {name}")
    chain = run_case(name, progress=lambda m: print("  ", m))
    paths = write_report(chain, os.path.join(a.out, name), lang=a.lang)
    rep = trust_report(chain)
    print(chain.explain(a.lang))
```

Testes e status de validação

- Módulos com pelo menos um teste que os importa: 10 de 18.
- Funções de teste em arquivos que importam algum módulo do pacote: 17.

Limitações conhecidas

- A biblioteca cobre poucos domínios (estruturas, calor, água em telhados, fluidos em tubos, corrosão galvânica); um projeto fora deles precisa de novos modos registrados com `register_mode`/`register_countermeasure`.
- O motor escolhe a primeira falha na ordem dada e a contramedida mais barata que corrige sem quebrar o resto; o resultado depende dessa ordem e dos custos unitários da biblioteca, e para em `max_steps` (padrão 12).
- Os modos usam modelos simplificados (forma fechada, MOC, volume finito pequeno) com condições de validade registradas; fora delas a checagem não é confiável.
- O conselheiro LLM é opcional e só propõe nomes; sem LLM, só as contramedidas da biblioteca são testadas.
- `engine`, `report`, `trust`, `narrative`, `__main__`, `modes.fluids` e `modes.corrosion` não têm teste que os importe diretamente; nenhum caso pronto usa o modo de corrosão.
- Relatórios e confiança dependem de `matplotlib`, `pinneapple_analysis.trust` e `pinneapple_lab.curation`; alguns modos chamam `pinneapple_simulation` e `pinneapple_physics`.

Volume 6 — Projeto e sistemas

> Parte do Mapa completo do PINNeAPPle. Gerado por `docs/pt/_mapa/gerar.py` a partir do inventário do commit `e53cdcfb04`.

Geometria, otimização de projeto, projeto a partir de intenção, sistemas dinâmicos, séries temporais, co-simulação e gêmeos digitais.

Sumário

1. pinneapple_design
2. pinneapple_systems
3. pinneapple_twin3d

1. `pinneapple_design`

Objetivo

`pinneapple_design` reúne construção de geometria e otimização de projeto guiada por física. Esta parte cobre o `__init__` do pacote e os submódulos `geometry`, `bricks` e `thermal`.

`geometry` (antigo `pinneapple_geom`) dá a base geométrica de todo o PINNeAPPle: SDFs 2D/3D, CSG, domínios físicos prontos (canal, cavidade, tubo, L, anel, junção T), malhadores 2D/3D com gmsh, limpeza e reparo de malhas, curvatura discreta, voxelização, importação STL/STEP/IGES/meshio/CadQuery, amostradores de pontos de colocação (`MeshCollocator`, construtores de lote por tags usados pelas presets de `pinneapple_physics.pde_environment`) e CAD paramétrico de veículos e peças para CFD e render.

`bricks` é um estudo completo com blocos de montar: catálogo e massas, estática por programação linear, FEA em voxels, replay de corpo rígido com o NVIDIA Newton, metas (balanço, ponte, torre), busca evolutiva NSGA-II ou guiada por surrogate GP, validação contra fórmulas fechadas e saídas visuais (LDraw, glTF, Blender, GIF/MP4).

`thermal` traz o motor do app HeatSink Sizer (`apps/heatsink_sizer` reexporta estes módulos), com rede de fórmulas fechadas, base em volumes finitos e surrogates MLP só para triagem, e o estudo `board_cooling`, um agente (via `pinneapple_autolab`) que lê um pedido, amostra, aprende um surrogate POD+GP, otimiza, verifica numa malha fina e entrega relatório de confiança.

O `__init__` reexporta `geometry` e `design_optimizer` e oferece `build_domain`, `sample_domain` e `optimize`. O pacote se apoia em `pinneapple_physics` (fórmulas fechadas, PCB), `pinneapple_simulation` (Newton, `solid_fem`, LBM), `pinneapple_neural` (treinador, ROM, GP) e `pinneapple_tools` (visualização).

Esta parte cobre cinco submódulos de `pinneapple_design`. `aero` faz o projeto aerodinâmico completo (perfis CST, casos OpenFOAM, surrogates MLP e MeshGraphNet, aeronave leve e avião comercial, busca NSGA-II e verificação por CFD). `design_optimizer` é o núcleo genérico de otimização de projeto: problemas com variáveis, objetivos, restrições e avaliadores, métodos comparáveis sob o mesmo orçamento, verificação em alta fidelidade e presets. `geometry_library` é a biblioteca de geometrias 3D paramétricas exposta como `pp.geometries`. `intent` transforma uma meta em linguagem natural (`pp.design`) num pacote de projeto verificado, usando `design_optimizer` e a biblioteca de geometrias. `surrogate_inverse` traz cinco casos didáticos de projeto inverso por surrogate sobre fórmulas de `pinneapple_physics.closed_form`, servidos por apps.

Esta parte cobre três submódulos do `pinneapple_design`. `fitmesh` leva de uma descrição em texto a CAD verificado, malhas planejadas para o propósito e um relatório único, com ajuda opcional de modelo de linguagem. `qualitative` faz uma prévia física qualitativa de geometrias e mudanças de peças antes da simulação, e de-

pois a confere quantitativamente. `topology` faz otimização topológica por densidade (SIMP, filtro, projeção de Heaviside, OC e MMA) em problemas estruturais, térmicos, de Stokes e de rotor de máquina elétrica.

Números do inventário

módulos	linhas	classes públicas	funções públicas	testes em arquivos que importam o pacote	módulos sem teste direto
196	48.153	269	740	744 (54 arquivos)	86

Pasta de testes dedicada: `tests/pinneapple_design` (88 funções de teste).

Submódulos

submódulo	o que faz	módulos	classes + funções	testes *
`aero`	Projeto aerodinâmico de ponta a ponta: perfis CST, malha O-grid e casos OpenFOAM, surrogates (MLP e MeshGraphNet), desempenho da aeronave leve e do avião comercial, busca NSGA-II e verificação por CFD.	23	111	72
`bricks`	Blocos de montar: geometria, estática, dinâmica, FEA, metas, busca automática de projetos e visualizações.	14	108	24
`design_optimizer`	Otimização de projeto guiada por física: espaço de parâmetros, objetivos e restrições, otimizadores (gradiente, bayesiano, evolutivo), estudos de projeto com orçamento de avaliações, verificação em alta fidelidade e presets de dissipador e perfil.	18	98	179
`fitmesh`	Geometria e malha adequadas ao propósito: lê o problema, sugere geometrias de referência, gera CAD checado, planeja e gera malhas e escreve um relatório.	12	38	21
`geometry`	Base geométrica: SDFs, CSG, domínios físicos, malhas, E/S de CAD, operações de malha, amostradores e CAD paramétrico de veículos.	56	255	195
`geometry_library`	Biblioteca de geometrias 3D paramétricas e realistas (aeronaves, veículos, navios, turbinas, prédios, anatomia) com registro, construtores, checagens de malha, exportadores (STL, OBJ, glTF, USD, STEP), renderizador por software e arquivos pré-gerados no Hugging Face Hub.	15	91	42
`intent`	Projeto a partir da intenção: uma meta em linguagem natural vira especificação com unidades, plano de física, otimização multifidelidade, verificação com solver completo, robustez e um pacote de projeto com relatório, geometria e arquivos de checagem.	19	84	24
`qualitative`	Prévia qualitativa de como cada geometria ou mudança de peça afeta o resultado, por modelos físicos baratos, com checagem quantitativa.	5	25	13
`surrogate_inverse`	Projeto inverso guiado por surrogate em física de forma fechada: amostra, treina uma MLP pequena, desce o gradiente no design pela rede congelada e reconfere o resultado contra a física; cinco casos resolvidos.	8	104	94
`thermal`	Projeto térmico: motor e dimensionador de dissipadores, surrogates e o estudo de resfriamento de placa.	19	75	46
`topology`	Otimização topológica baseada em densidade com filtros, projeção, OC e MMA.	6	17	9

* maior número de testes que importam um módulo do submódulo.

Módulos e API pública

`pinneapple_design`

Ponto de entrada do pacote de projeto: reexporta `geometry` e `design_optimizer` (com os apelidos antigos `geom` e `design_opt`) e oferece três atalhos de integração.

`pinneapple_design/__init__.py` · 179 linhas · testes: 34

nome	tipo	assinatura	o que faz
`build_domain`	função	`(name, dim=2, **kwargs)`	Devolve um domínio físico pelo nome: chama `get_domain_3d` se `dim == 3` e `get_domain` caso contrário.

nome	tipo	assinatura	o que faz
sample_domain	função	(domain, n_interior=4096, n_boundary=512)	Chama domain.sample_interior e domain.sample_boundary e devolve os pontos de colocação internos e de contorno; os domínios de gen.domains/domains3d não têm sample_boundary (só sample_boundary_region), então só funciona com objetos que tenham esse método, como as formas de csg.
optimize	função	(param_space, surrogate, objective, *, constraints=None, method='bayesian', n_trials=100,…)	Atalho de otimização de projeto: embrulha o modelo em PhysicsSurrogate se preciso, monta DesignOptConfig e roda DesignOptLoop.

pinneapple_design.aero

Pacote de projeto aerodinâmico: formas de perfil, malhas O-grid, casos OpenFOAM, desempenho de aeronave e busca guiada por surrogate.

pinneapple_design/aero/__init__.py · 2 linhas · testes: 17

pinneapple_design.aero.aircraft

Converte a polar de uma seção em números de aeronave leve monomotor a hélice (classe Cessna 172 por padrão): velocidade máxima, CO2 por 100 km, velocidade de estol e checagem de requisitos.

pinneapple_design/aero/aircraft.py · 154 linhas · testes: 14

nome	tipo	assinatura	o que faz
Aircraft	classe	(mass, wing_area, aspect_ratio, oswald, cd0_rest, power_kw, prop_efficiency, bsfc, cruise…)	Dataclass com os dados da aeronave leve (massa, área e alongamento da asa, Oswald, CD0 do resto, potência, hélice, consumo, cruzeiro), com peso e corda derivados.
Requirements	classe	(max_stall_speed, min_thickness, max_abs_cm, max_cruise_cl_ratio)	Requisitos do design: velocidade de estol máxima, espessura mínima, |Cm| máximo e razão máxima entre CL de cruzeiro e CLmax.
Polar	classe	(alpha, cl, cd, cm)	Polar de seção a partir de coeficientes em poucos ângulos: cl(alpha) e cd, cm como funções de cl na parte colada.
isa_density	função	(h) -> float	Densidade do ar na atmosfera padrão (ISA, troposfera) na altitude h.
performance	função	(polar, ac=Aircraft(), req=Requirements()) -> Dict[str, Any]	Calcula o desempenho da aeronave a partir da polar: velocidade máxima, consumo e CO2 por 100 km, velocidade de estol e grandezas de cruzeiro.
evaluate	função	(shape_props, polar, ac=Aircraft(), req=Requirements()) -> Dict[str, Any]	Desempenho mais o requisito geométrico; o design é viável quando todas as checagens passam.

pinneapple_design.aero.aircraft3d

Avalia a aeronave inteira em 3D: vortex lattice (asa + cauda, trimada), arrasto de seção faixa a faixa pelo surrogate de CFD, arrasto e pesos por build-up de Raymer, CG, ponto neutro e estol pela seção crítica.

pinneapple_design/aero/aircraft3d.py · 295 linhas · testes: 6

nome	tipo	assinatura	o que faz
Requirements3D	classe	(max_stall_speed, min_thickness, min_static_margin, max_static_margin, max_tail_incidence…)	Requisitos do design 3D: estol, espessura, margem estática mínima e máxima, incidência da cauda, estação de início do estol, envergadura e razão de CL de cruzeiro.
Aircraft3D	classe	(x, wing_polar, tail_polar, ac=Aircraft(), req=Requirements3D(), base_weights=None, nc=4,…)	Avalia um design 3D a partir das polares de seção (do surrogate de CFD): condição de voo, arrasto, estol e avaliação completa.
airframe_from	função	(x, base=None) -> Airframe	Monta um Airframe a partir do vetor de design (6 pesos CST + parâmetros de planta).
baseline_x	função	() -> np.ndarray	Vetor de design de referência: perfil NACA 2412 mais a planta de base.
wing_weight	função	(af, Wdg, q, tc, nz=5.7, Wfw=70 * 3.785 * 0.72) -> float	Massa da asa (kg) pela fórmula de Raymer para aviação geral, a partir da massa bruta de projeto, pressão dinâmica e combustível na asa.

nome	tipo	assinatura	o que faz
`htail_weight`	função	`(af, Wdg, q, nz=5.7) -> float`	Massa da cauda horizontal pela fórmula de Raymer (aviação geral).
`vtail_weight`	função	`(af, Wdg, q, nz=5.7) -> float`	Massa da cauda vertical pela fórmula de Raymer (aviação geral).
`fuselage_wetted`	função	`(af)`	Área molhada da fuselagem e seu fator de forma, integrando o perímetro das seções.
`fuselage_cd0`	função	`(af, V, rho, mu=1.75e-05, wet=None) -> float`	CD0 da fuselagem (atrito turbulento x fator de forma x área molhada sobre a área da asa).
`vtail_cd0`	função	`(af, V, rho, mu=1.75e-05, vt=None) -> float`	CD0 da cauda vertical por build-up de componente.

pinneapple_design.aero.airframe

Geometria paramétrica 3D de aeronave leve (fuselagem, asa com o perfil otimizado, caudas por coeficiente de volume, hélice, trem de pouso) com exportação para glTF binário, OpenUSD e STL.

`pinneapple_design/aero/airframe.py` · 511 linhas · testes: 20

nome	tipo	assinatura	o que faz
`Part`	classe	`(name, vertices, faces, material, group, aero)`	Uma peça da malha da aeronave: nome, vértices, faces, material, grupo (a hélice gira) e se entra na forma limpa de CFD.
`Airframe`	classe	`(length, width, height, wing_area, aspect_ratio, taper, sweep_le, dihedral, twist, incide…)`	Aeronave leve paramétrica (fuselagem, asa, caudas por coeficiente de volume) que gera suas peças com `build` e expõe envergadura, corda média e dados das caudas.
`vertex_normals`	função	`(V, F) -> np.ndarray`	Normais por vértice somando as normais das faces vizinhas.
`to_glb`	função	`(parts, scalars=None, fields=None) -> bytes`	Escreve glTF 2.0 binário com materiais PBR e, opcionalmente, campos por vértice (ex.: pressão de superfície).
`to_usda`	função	`(parts, title='aircraft') -> str`	Escreve OpenUSD em texto, uma Mesh por peça com materiais UsdPreviewSurface (Omniverse, usdview).
`to_stl`	função	`(parts, solid='aircraft') -> bytes`	STL binário das peças dadas, uma superfície fechada por peça.

pinneapple_design.aero.airliner

Geometria paramétrica de avião comercial de corredor único (classe A320/737) com a mesma interface de `Airframe`, para usar o mesmo pipeline de vortex lattice e exportação. [sem teste]

`pinneapple_design/aero/airliner.py` · 310 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Airliner`	classe	`(length, width, height, wing_area, aspect_ratio, taper, sweep_le, dihedral, twist, incide…)`	Avião comercial paramétrico (fuselagem circular com janelas e pintura, asa enflechada, turbofans, caudas) com a mesma interface de `Airframe`.
`sweep_le_from_qc`	função	`(sweep_qc, ar, taper) -> float`	Converte o enflechamento a 1/4 da corda em enflechamento do bordo de ataque, dados alongamento e afilamento.

pinneapple_design.aero.airliner3d

Física do avião comercial (classe A320): cruzeiro transônico com Prandtl-Glauert, arrasto de onda (Korn + regra de Lock), alcance de Breguet com MTOW iterado, e lado de baixa velocidade. [sem teste]

`pinneapple_design/aero/airliner3d.py` · 340 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Mission`	classe	`(passengers, mass_per_pax, range_km, cruise_altitude, tsfc, reserve, oew_rest, high_lift_…)`	Missão de projeto do avião comercial: passageiros, massa por passageiro, alcance, altitude de cruzeiro, consumo específico do turbofan e reserva.
`RequirementsAL`	classe	`(max_span, max_vref_kt, buffet_margin, min_static_margin, max_static_margin, max_stall_st…)`	Requisitos do avião comercial: envergadura máxima (código de aeródromo), velocidade de aproximação, margem de buffet, margem estática e estação de estol.
`AirlinerModel`	classe	`(x, wing_polar, tail_polar, mission=Mission(), req=RequirementsAL(), nc=4, ns_wing=12, ns…)`	Avalia um design de avião comercial: faixas da asa, arrasto de onda, cruzeiro, arrasto parasita, estol, volume de combustível e avaliação completa.
`isa`	função	`(h)`	Atmosfera padrão até a estratosfera baixa: densidade, velocidade do som e viscosidade (Sutherland) na altitude `h`.
`airliner_from`	função	`(x) -> Airliner`	Monta um `Airliner` a partir do vetor de design (seção CST + planta + Mach + posição da asa).
`baseline_x_al`	função	`() -> np.ndarray`	Vetor de design de referência do avião comercial (NACA 2412 + planta de base).
`wing_weight_transport`	função	`(af, mtow, tc, nz=3.75) -> float`	Massa da asa pela fórmula de Raymer para aviões de transporte, com um fator de correção do módulo.

pinneapple_design.aero.buildup

Arrasto de sustentação nula (CD0) de uma aeronave da biblioteca de geometrias a partir da malha: atrito turbulento de placa plana por componente, fatores de forma de Raymer, interferência e excrescências. [sem teste]

`pinneapple_design/aero/buildup.py` · 57 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`sutherland_mu`	função	`(T) -> float`	Viscosidade dinâmica do ar pela lei de Sutherland, em Pa.s.
`friction_drag`	função	`(geometry, ac, *, rho, V, T, mach, components=None, interference=1.08) -> dict[str, Any]`	CD0 (sobre a área da asa) dos corpos de uma `Geometry` da biblioteca cujos nomes casam com os componentes dados.

pinneapple_design.aero.case

Monta, roda e lê um caso OpenFOAM 2D (simpleFoam, k-omega SST resolvido até a parede) de um perfil em um ângulo de ataque; também empacota o caso em zip pronto para rodar.

`pinneapple_design/aero/case.py` · 302 linhas · testes: 22

nome	tipo	assinatura	o que faz
`write_case`	função	`(case, shape, alpha_deg, reynolds=4000000.0, iterations=1200, tu=0.001, grid=GridSpec(), …)`	Escreve o diretório do caso OpenFOAM (polyMesh, condições de contorno, controle) para uma forma, ângulo e Reynolds.
`read_internal`	função	`(path) -> np.ndarray`	Lê o internalField de um arquivo de campo ASCII do OpenFOAM como array (n,) ou (n, 3).
`wall_geometry`	função	`(P2) -> Dict[str, np.ndarray]`	Por face de parede: centro, comprimento, normal para dentro do corpo e distância do primeiro centro de célula.
`forces`	função	`(P2, p_wall, U_first, nu, alpha_deg) -> Dict[str, Any]`	Cl, Cd, Cm (a 1/4 da corda) e as distribuições de Cp e Cf a partir das células junto à parede.
`time_dirs`	função	`(case)`	Lista os diretórios de tempo numéricos de um caso OpenFOAM, em ordem.
`read_coefficients`	função	`(case, P2, nu, alpha_deg, ni) -> Dict[str, Any]`	Coeficientes em cada escrita guardada (checagem de convergência) e o resultado completo na última.
`run_case`	função	`(case, shape, alpha_deg, reynolds=4000000.0, iterations=1200, grid=GridSpec(), timeout=18…)`	Escreve e roda o simpleFoam e devolve coeficientes, veredito de convergência e tempo.
`case_zip`	função	`(shape, alpha_deg, reynolds=4000000.0, iterations=1200, grid=None) -> bytes`	Caso OpenFOAM pronto para rodar (com script de coeficientes e `Allrun`) como bytes de zip.

pinneapple_design.aero.case3d

Caso OpenFOAM 3D da aeronave inteira (meio modelo com plano de simetria): STL do `Airframe`, snappyHexMesh, simpleFoam com funções de parede e leitura de forças, Cp, Cf e linhas de corrente. [sem teste]

`pinneapple_design/aero/case3d.py` · 197 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`write_case3d`	função	`(case, af, alpha_deg=2.0, speed=55.0, nu=1.5e-05, iterations=800, level_wing=(5, 6), proc…)`	Escreve o caso 3D: STLs dos corpos do `Airframe`, snappyHexMesh e configuração do simpleFoam.
`run_case3d`	função	`(case, procs=4, log=print) -> Dict[str, float]`	Roda o caso 3D em paralelo (delegando a `run_case`).
`read_result3d`	função	`(case, info, af, n_lines=34) -> Dict[str, Any]`	Lê forças (pressão + atrito), Cp e Cf por face, linhas de corrente, esteira e fatias no plano de simetria.
`surface_scalars`	função	`(af, parts, surface, key='cp') -> Dict[str, np.ndarray]`	Passa um campo de superfície do CFD (cp ou cf) para os vértices da malha visual, por interpolação de faces vizinhas e espelhamento.

pinneapple_design.aero.dataset

Conjunto de treino OpenFOAM dos surrogates de perfil: plano de rodadas (hipercubo latino de formas x ângulos + polares NACA), execução de uma rodada e regra do valor estacionário.

`pinneapple_design/aero/dataset.py` · 92 linhas · testes: 13

nome	tipo	assinatura	o que faz
`plan`	função	`(n_shapes, n_alpha, seed=7) -> list[dict[str, Any]]`	Lista de rodadas: formas válidas por hipercubo latino x ângulos em faixas, mais as polares NACA de referência; determinística pela semente.
`run_job`	função	`(job, out) -> tuple[str, Any, Any]`	Roda um job no OpenFOAM e grava `out/runs/<id>.json` (e `.npz` com campos de célula quando rodou).
`steady_values`	função	`(runs, max_osc=MAX_CL_OSC) -> list[dict[str, Any]]`	Cl, Cd, Cm estacionários de cada rodada; rodadas instáveis são médias das últimas escritas ou descartadas se oscilam demais.

pinneapple_design.aero.field_dataset

Grava o dataset de campos do zoo (`airfoil_rans_cst_re4e6_fields`): campos de célula p, Ux, Uy, nut de cada rodada da campanha OpenFOAM, em arquivos `.npz` por split.

`pinneapple_design/aero/field_dataset.py` · 98 linhas · testes: 4

nome	tipo	assinatura	o que faz
`write_field_dataset`	função	`(run_dir, out, name=…, shard_rows=SHARD_ROWS) -> dict`	Escreve o dataset de campos do zoo (train, test, naca_reference em `.npz` e `meta.json`) a partir dos arquivos de rodada da campanha.

pinneapple_design.aero.geometry

Formas de perfil CST (Kulfan) com 6 pesos de espessura e arqueamento, ajuste de NACA 4 dígitos, propriedades derivadas, validade, hipercubo latino e alças de edição.

`pinneapple_design/aero/geometry.py` · 153 linhas · testes: 72

nome	tipo	assinatura	o que faz
`Loop`	classe	`(xy, n_te)`	Laço fechado de parede para a O-grid: base do bordo de fuga, superfície inferior e superior.
`cst_surface`	função	`(w, x, te) -> np.ndarray`	Avalia uma superfície CST (função de classe x Bernstein + espessura do bordo de fuga) nas estações x.
`surfaces`	função	`(shape, x) -> tuple[np.ndarray, np.ndarray]`	y das superfícies superior e inferior nas estações x (corda 1).

nome	tipo	assinatura	o que faz
`fit_cst`	função	`(xu, yu, xl, yl) -> np.ndarray`	Pesos CST por mínimos quadrados para coordenadas dadas de extradorso e intradorso.
`naca4`	função	`(code, n=200) -> np.ndarray`	Ajuste CST de um perfil NACA 4 dígitos, ex.: `naca4("2412")`.
`properties`	função	`(shape) -> dict[str, float]`	Espessura e arqueamento máximos e suas posições, raio do bordo de ataque e área da seção.
`valid`	função	`(shape) -> tuple[bool, str]`	Testa se a forma é válida (superfícies não se cruzam, curvatura limitada) e devolve o motivo quando não é.
`outline`	função	`(shape, n=80) -> dict[str, list]`	Contorno do perfil em espaçamento cosseno, como dicionário de listas (para o navegador).
`latin_hypercube`	função	`(n, bounds, rng) -> np.ndarray`	Amostra `n` pontos por hipercubo latino dentro dos limites dados.
`wall_loop`	função	`(shape, n_surf=80, n_te=4) -> Loop`	Constrói o `Loop` de parede da forma, com pontos agrupados no bordo de ataque e de fuga.
`handles`	função	`(shape, xh=HANDLE_X) -> tuple[np.ndarray, np.ndarray]`	y das superfícies superior e inferior nas estações de alça (três por superfície).
`shape_from_handles`	função	`(y_upper, y_lower, xh=HANDLE_X) -> np.ndarray`	Forma CST cujas superfícies passam pelas alturas de alça dadas.
`out_of_bounds`	função	`(shape) -> list`	Nomes dos pesos CST fora da caixa de treino dos surrogates (`BOUNDS`).

pinneapple_design.aero.geometry_onnx

Exporta a geometria CST como grafos ONNX float64 (geometria e conversão de alças para forma) que reproduzem as funções NumPy, para uso no navegador.

`pinneapple_design/aero/geometry_onnx.py` · 188 linhas · testes: 13

nome	tipo	assinatura	o que faz
`torch_geometry`	função	`(n_outline=N_OUTLINE)`	Módulo torch que leva a forma (n, 6) ao dicionário de saídas de geometria em float64.
`torch_from_handles`	função	`()`	Módulo torch que leva alturas de alça (n, 6) à forma CST (n, 6), como `shape_from_handles`.
`export_onnx`	função	`(directory, *, presets=None) -> dict[str, Any]`	Grava `cst_geometry.onnx`, `cst_from_handles.onnx` e `cst_geometry.json` no diretório e devolve o JSON.
`reference`	função	`(shapes) -> dict[str, np.ndarray]`	Valores NumPy que os grafos ONNX devem reproduzir (usado nos testes).

pinneapple_design.aero.mesh

Gera a O-grid estruturada em volta do perfil, com a mesma topologia para todo design, e escreve direto o polyMesh do OpenFOAM.

`pinneapple_design/aero/mesh.py` · 182 linhas · testes: 21

nome	tipo	assinatura	o que faz
`GridSpec`	classe	`(n_surf, n_te, nj, first, radius, span)`	Parâmetros da O-grid: células ao longo da superfície e do bordo de fuga, camadas até o campo distante, altura da primeira célula e raio externo.
`grid_points`	função	`(shape, g=GridSpec()) -> np.ndarray`	Pontos da grade (nj+1, ni, 2), com j = 0 na parede.
`topology`	função	`(g=GridSpec()) -> Dict[str, np.ndarray]`	Faces da grade 2D na ordem do OpenFOAM, como índices (i, j) dos pontos.
`write_polymesh`	função	`(shape, case, g=GridSpec()) -> np.ndarray`	Escreve `constant/polyMesh` para a forma e devolve os pontos.
`cell_centres`	função	`(P2) -> np.ndarray`	Centros das células (nj, ni, 2) a partir dos pontos da grade.

pinneapple_design.aero.optimize

Busca de projeto NSGA-II sobre forma do perfil + área de asa no surrogate, com três objetivos (velocidade máxima, CO2, velocidade de estol), viabilidade e nível de confiança por design.

pinneapple_design/aero/optimize.py · 183 linhas · testes: 23

nome	tipo	assinatura	o que faz
Engine	classe	(bundle)	Junta os surrogates treinados e a avaliação da aeronave: carrega o bundle, prevê coeficientes (MLP e GNN), avalia e faz a busca NSGA-II.
baseline	função	(engine, aircraft, req, name='NACA 2412') -> Dict[str, Any]	Avalia o design de referência (um perfil NACA com a área de asa da aeronave).
summarize	função	(rec) -> Dict[str, Any]	Forma compacta de um design enviada ao navegador para a nuvem de designs.
infeasibility_reasons	função	(designs) -> dict[str, int]	Conta, por requisito violado, por que os designs de uma busca ficaram fora do conjunto de Pareto.

pinneapple_design.aero.optimize3d

Busca NSGA-II da aeronave inteira (perfil + planta + posição da asa), avaliada com vortex lattice, surrogate de seção e build-up de componentes.

pinneapple_design/aero/optimize3d.py · 107 linhas · testes: 6

nome	tipo	assinatura	o que faz
Engine3D	classe	(engine)	Motor da busca da aeronave inteira: pesos de base, avaliação 3D de um lote e busca NSGA-II.
summarize3d	função	(r) -> Dict[str, Any]	Forma compacta de um registro da busca 3D (velocidades, CO2, viabilidade, margem estática, etc.).

pinneapple_design.aero.optimize_airliner

Busca NSGA-II do avião comercial: seção da asa, planta, Mach de cruzeiro e posição da asa; objetivos CO2 por passageiro-km, Mach e velocidade de aproximação. [sem teste]

pinneapple_design/aero/optimize_airliner.py · 102 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
EngineAL	classe	(engine)	Motor da busca do avião comercial: avaliação de um lote de designs com a missão e busca NSGA-II.
summarize_al	função	(r) -> Dict[str, Any]	Forma compacta de um registro da busca do avião comercial (CO2 por passageiro-km, Mach, Vref, MTOW, etc.).

pinneapple_design.aero.panel

Método de painéis de Hess-Smith (escoamento potencial incompressível) em volta de um perfil CST: a referência invíscida do benchmark de perfil.

pinneapple_design/aero/panel.py · 128 linhas · testes: 31

nome	tipo	assinatura	o que faz
PanelResult	classe	(nodes, sigma, gamma, alpha_deg, x_mid, y_mid, cp, cl_kutta_joukowski, cl_pressure, cd_pr…)	Resultado do método de painéis: nós, intensidades de fonte, vórtice, pontos médios, com cálculo de velocidade em pontos.
airfoil_nodes	função	(shape, n_per_side=100, close_te=False) -> np.ndarray	Nós dos painéis em sentido horário, agrupados nas pontas; opcionalmente fecha o bordo de fuga rombudo.
panel_solve	função	(shape=None, alpha_deg=0.0, *, nodes=None, n_per_side=100, close_te=False) -> PanelResult	Resolve o escoamento potencial em volta de um perfil CST (ou de nós dados) num ângulo de ataque.

pinneapple_design.aero.render_blender

Renderização fotorrealista de um `.glb` de aeronave com Blender Cycles, com opção de colorir por Cp/Cf e desenhar linhas de corrente; exige `bpy`. [sem teste]

`pinneapple_design/aero/render_blender.py` · 250 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`render`	função	`(glb, out, view='hero', samples=96, size=(1600, 900), cp=False, lines_npz=None, sun_eleva…)`	Renderiza um `.glb` com Blender Cycles numa vista, com opção de campo Cp/Cf, modo estúdio e linhas de corrente.
`main`	função	`(argv=None)`	Ponto de entrada de linha de comando do render (argparse).

pinneapple_design.aero.surrogate

Surrogates do escoamento no perfil: uma MeshGraphNet nas células da O-grid (campos + Cl, Cd, Cm) e um ensemble de MLPs cuja dispersão é a incerteza.

`pinneapple_design/aero/surrogate.py` · 134 linhas · testes: 9

nome	tipo	assinatura	o que faz
`Layout`	classe	`(grid, i_step, max_dist, j_sel, i_sel, edge_index)`	Subconjunto fixo de células da O-grid perto do perfil usado como grafo, com as arestas multiescala.
`AeroGNN`	classe	`(node_in, edge_in=3, hidden=48, mp=8, n_fields=4)`	MeshGraphNet que prevê os campos p, Ux, Uy, log(1+nut/nu) por nó e, por uma leitura agregada na parede, Cl, Cd, Cm.
`MLPEnsemble`	classe	`(k=5, width=96)`	K MLPs pequenas em (forma, alpha) -> (Cl, log Cd, Cm); a dispersão entre elas é a incerteza.
`node_features`	função	`(lay, shape, alpha, pos=None) -> np.ndarray`	Atributos de nó do grafo: posição, log da distância à parede, nível, flag de parede, direção do escoamento e pesos da forma.
`edge_features`	função	`(lay, pos) -> np.ndarray`	Atributos de aresta: direção unitária e log do comprimento.
`coef_targets`	função	`(cl, cd, cm)`	Empilha (Cl, log Cd, Cm) como alvos de treino.
`coef_from_targets`	função	`(t)`	Inverte `coef_targets`: devolve Cl, Cd, Cm.

pinneapple_design.aero.train

Treina os dois surrogates de perfil com as rodadas OpenFOAM e mede o erro em formas nunca vistas (split por forma) e nas polares NACA.

`pinneapple_design/aero/train.py` · 334 linhas · testes: 13

nome	tipo	assinatura	o que faz
`load_runs`	função	`(dirs, max_osc=MAX_CL_OSC) -> list[dict[str, Any]]`	Carrega os registros de rodada utilizáveis de cada diretório, com coeficientes estacionários e o caminho dos campos.
`split`	função	`(recs, frac=0.15, seed=3)`	Divide em (treino, teste, referência) separando formas inteiras para teste.
`coefs`	função	`(recs) -> np.ndarray`	Array (n, 3) de (Cl, Cd, Cm) dos registros.
`metrics`	função	`(true, pred) -> dict[str, dict[str, float]]`	Erros médio e máximo e R^2 por coeficiente; para Cd também erro relativo médio e percentil 90.
`train_mlp`	função	`(tr, k=5, width=96, epochs=6000, seed=0)`	Treina k MLPs em reamostragens bootstrap (Adam + agenda cosseno) e devolve modelo e normalização.
`predict_mlp`	função	`(model, norm, recs)`	Média e dispersão do ensemble para (Cl, Cd, Cm).
`graph_data`	função	`(lay, recs)`	Monta atributos de nó e aresta e alvos de campo e coeficiente de cada rodada a partir dos `.npz`.

nome	tipo	assinatura	o que faz
`train_gnn`	função	`(tr, te, minutes=60.0, seed=0, batch=4, log=print, *, epochs=None, hidden=48, mp=8, layou…)`	Treina a rede de grafos por um orçamento de tempo e/ou épocas, com taxa de aprendizado em cosseno.
`predict_gnn`	função	`(model, norm, lay, recs)`	Prevê (Cl, Cd, Cm) com a GNN e calcula o RMSE de cada campo contra as rodadas.
`train_surrogates`	função	`(recs, gnn_minutes=60.0, gnn_from=None, log=print, *, mlp_k=5, mlp_width=96, mlp_epochs=6…)`	Divide os dados, treina os dois surrogates e mede; devolve `(bundle, metrics)`.

pinneapple_design.aero.verify

Confere a busca com OpenFOAM: escolhe designs ao longo da frente de Pareto, roda suas polares e compara surrogate x CFD; as rodadas servem para aprendizado ativo. [sem teste]

`pinneapple_design/aero/verify.py` · 85 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`pick`	função	`(front, n) -> list[int]`	Escolhe `n` designs espaçados ao longo da frente, sempre com as duas pontas.
`cfd_eval`	função	`(runs, shape, area, ac, req) -> dict[str, Any]`	Desempenho e checagens da aeronave a partir da polar OpenFOAM de uma forma com dada área de asa.
`verification_jobs`	função	`(designs, chosen, round_)`	Gera os jobs OpenFOAM (um por design e ângulo) no formato do dataset.
`compare_design`	função	`(engine, did, d, runs, ac, req) -> dict[str, Any]`	Compara surrogate e OpenFOAM para um design da busca.

pinneapple_design.aero.vlm

Vortex lattice (ferraduras) para asa + cauda horizontal de um `Airframe`; a solução linear dá sustentação, momento, carregamento, arrasto induzido, trimagem e ponto neutro.

`pinneapple_design/aero/vlm.py` · 257 linhas · testes: 6

nome	tipo	assinatura	o que faz
`Lattice`	classe	`(P1, P2, C, nrm0, surf, strip, chord, yc, dy, strip_surf, tail_rot)`	Malha de ferraduras: pontos dos vórtices ligados, pontos de controle, normais, superfície (asa ou cauda) e faixa.
`VLMSolution`	classe	`(L, G, s_ref, c_ref, b_ref, x_ref, CL, Cm, CLw, cl_strip, Q)`	Solução do VLM linear em alpha e incidência da cauda: coeficientes, trimagem, cl por faixa e ponto neutro.
`build_lattice`	função	`(af, nc=5, ns_wing=14, ns_tail=6, tail=True) -> Lattice`	Monta o `Lattice` da asa e da cauda de um `Airframe` (enflechamento, diedro, torção, arqueamento).
`horseshoe_induced`	função	`(P, L, legs=True)`	Velocidade induzida pelas ferraduras do lattice em pontos dados (perna ligada e pernas livres).
`solve`	função	`(af, x_ref, nc=5, ns_wing=14, ns_tail=6, tail=True) -> VLMSolution`	Resolve o VLM para o `Airframe` e devolve um `VLMSolution`.
`streamlines`	função	`(sol, alpha, it, seeds, steps=220, h=0.08, core=0.08) -> np.ndarray`	Linhas de corrente do escoamento potencial (corrente livre + ferraduras com núcleo dessingularizado).

pinneapple_design.bricks

Subpacote de blocos de montar (tipo LEGO): geometria, estática, dinâmica, FEA, metas de projeto e um laço de projeto que se modifica sozinho; reexporta as classes principais.

`pinneapple_design/bricks/__init__.py` · 67 linhas · testes: 24

pinneapple_design.bricks.assembly

Montagens de blocos numa grade de pinos: posicionamento, conectividade pino-tubo e regras de validade (sem sobreposição, apoio).

`pinneapple_design/bricks/assembly.py` · 315 linhas · testes: 24

nome	tipo	assinatura	o que faz
`Brick`	classe	`(part, x, y, z, rot, colour)`	Um bloco colocado na grade (peça, posição x/y/z, rotação, cor), com pegada, células ocupadas, caixa e centro de massa em mm.
`Ground`	classe	`(kind, regions)`	O chão sob a montagem: mesa lisa (com atrito) ou placa-base com pinos, opcionalmente só em regiões retangulares.
`Connection`	classe	`(lower, upper, z, rect, studs)`	Contato entre dois blocos (ou bloco e chão) numa altura, com o retângulo e os pinos que se encaixam; `clutch` dá a capacidade de tração.
`Assembly`	classe	`(bricks=None, ground=None, name='')`	Lista de blocos sobre o chão, com adição/remoção, mapa de ocupação, conexões, grafo, componentes, validação, massa e centro de massa.
`colour_rgb`	função	`(name) -> tuple[float, float, float]`	Converte o nome de uma cor da paleta em RGB.
`stack`	função	`(part_name, n, *, offsets=None, colours=('red',), ground=None) -> Assembly`	Monta `n` cópias de uma peça empilhadas, cada uma deslocada em x por `offsets[k]` pinos.

pinneapple_design.bricks.closed_form

Fórmulas fechadas de referência (tombamento, empilhamento harmônico, balanço preso por pinos, viga biapoiada) usadas para validar `statics`, `dynamics` e `fea`.

`pinneapple_design/bricks/closed_form.py` · 77 linhas · testes: 24

nome	tipo	assinatura	o que faz
`harmonic`	função	`(n) -> float`	Número harmônico H_n.
`harmonic_overhang`	função	`(n, length) -> float`	Balanço máximo de `n` blocos soltos iguais, um por nível: (L/2)·H_n.
`harmonic_stack`	função	`(n, length, edge=0.0) -> np.ndarray`	Posições x da borda esquerda de cada bloco da pilha harmônica sobre uma mesa que termina em `edge`.
`tip_over_offset`	função	`(length) -> float`	Maior balanço de um bloco uniforme sozinho na borda da mesa: metade do comprimento.
`stud_cantilever_tension`	função	`(weight, d, studs_x, x1) -> float`	Tração igual por pino que segura uma peça em balanço, girando em torno da borda do retalho de contato.
`beam_reactions`	função	`(span, w, P) -> float`	Reação em cada apoio de uma viga biapoiada simétrica.
`beam_moment`	função	`(x, span, w, P) -> float`	Momento fletor numa posição da viga biapoiada com carga central.
`beam_midspan_deflection`	função	`(span, E, inertia, P=0.0, w=0.0) -> float`	Flecha no meio do vão de uma viga biapoiada (Euler-Bernoulli).
`tower_tip_force`	função	`(weight, half_base, height) -> float`	Força horizontal na altura `height` que tomba um bloco rígido: F = W·b/h.

pinneapple_design.bricks.dynamics

Reprodução de corpo rígido de uma montagem com o NVIDIA Newton (`pinneapple_simulation.external_solvers.newton_physics`): blocos presos viram um corpo, juntas quebradas são soltas. Depende do Newton estar instalado.

`pinneapple_design/bricks/dynamics.py` · 239 linhas · testes: 24

nome	tipo	assinatura	o que faz
`Replay`	classe	`(t, pos, quat, broken, max_displacement_mm, moved, solver, wall_time_s, max_tilt_deg)`	Resultado de uma simulação Newton: trajetórias, juntas quebradas, deslocamento e inclinação máximos; `collapsed` diz se a montagem caiu.
`table_boxes`	função	`(asm, margin_mm=80.0) -> list[tuple[float, float, float, float]]`	Regiões do chão recortadas em volta da montagem, em mm.
`clusters`	função	`(asm, broken=()) -> tuple[list[list[int]], set[int]]`	Grupos de blocos unidos por juntas intactas e quais deles estão soldados a uma placa-base.
`rigid_scene`	função	`(asm, broken=(), joint=None, dead_loads=())`	Monta a `RigidScene` do Newton: um corpo rígido por grupo de blocos, cargas descendentes como pesos mortos.

nome	tipo	assinatura	o que faz
`replay`	função	`(asm, broken=(), *, duration=0.8, dt=0.001, solver='xpbd', iterations=16, frames=40, load…)`	Simula a montagem solta em repouso, com as juntas `broken` liberadas e as cargas aplicadas.
`harmonic_threshold`	função	`(n=4, length_mm=63.8, *, factors=…, duration=0.8, mu=0.4, tol_mm=1.0, solver='xpbd') -> l…)`	Testa no Newton a pilha harmônica com os deslocamentos escalados por fatores e compara com a fórmula fechada (fica de pé para f <= 1).
`cross_check`	função	`(goal, candidates, *, max_n=30, duration=0.4, tol_mm=2.0, dt=0.00025, iterations=32, seed…)`	Confere se o Newton concorda com o veredito da estática para uma amostra de candidatos.
`friction_creep`	função	`(asm, loads, *, dts=(0.001, 0.00025), duration=0.6, iterations=32) -> list[dict]`	Mede o deslocamento de uma montagem que deveria ficar parada sob carga lateral, para vários passos de tempo (artefato do atrito XPBD).

pinneapple_design.bricks.fea

Elementos finitos lineares elásticos (Hex8 com modos incompatíveis, `solid_fem` de `pinneapple_simulation`) sobre a montagem voxelizada.

`pinneapple_design/bricks/fea.py` · 127 linhas · testes: 24

nome	tipo	assinatura	o que faz
`VoxelModel`	classe	`(mesh, element_brick, element_volume)`	Malha hexaédrica da montagem com o bloco e o volume de cada elemento.
`FEAResult`	classe	`(fem, model, brick_displacement, brick_von_mises, max_displacement)`	Resultado da FEA: solução, modelo, deslocamento e tensão de von Mises por bloco, deslocamento máximo.
`voxel_mesh`	função	`(asm, per_stud=2) -> VoxelModel`	Gera a malha hexaédrica das células ocupadas, `per_stud` elementos por passo de pino.
`solve`	função	`(asm, loads=(), *, E=E_ABS_MPA, nu=NU_ABS, per_stud=2, gravity=True, supports='ground', f…)`	Resolve elasticidade estática linear da montagem sob gravidade e cargas pontuais (ABS por padrão).

pinneapple_design.bricks.goals

Metas de projeto para o laço automático (balanço, ponte, torre): cada uma avalia uma montagem com a estática e explica por que falha. A classe base `Goal` levanta NotImplementedError nos métodos a sobrescrever.

`pinneapple_design/bricks/goals.py` · 285 linhas · testes: 24 · palavras-chave no código (heurística; o texto acima diz o status real): levanta `NotImplementedError` em algum caminho

nome	tipo	assinatura	o que faz
`Evaluation`	classe	`(score, feasible, violation, n_bricks, status, explanation, metrics, critical)`	Avaliação de uma montagem: pontuação, viabilidade, violação numa escala comum, status, explicação e métricas.
`Goal`	classe	`(joint=None)`	Classe base das metas: define chão, semente, cargas, avaliação e objetivos para o NSGA-II; `ground`, `seed` e `score_and_check` levantam NotImplementedError.
`CantileverGoal`	classe	`(max_bricks=10, joint=None)`	Meta de alcançar o mais longe possível além da borda de uma mesa com no máximo `max_bricks` peças, sob gravidade.
`BridgeGoal`	classe	`(gap=12, load_n=1.0, max_bricks=24, joint=None)`	Meta de vencer um vão entre duas mesas carregando `load_n` newtons no meio do tabuleiro, com o menor número de peças.
`TowerGoal`	classe	`(max_bricks=16, push_n=0.01, base=6, joint=None)`	Meta da torre mais alta numa base dada que resiste a um empurrão no topo, sobre mesa lisa (pode tombar ou quebrar).
`reference_cantilever`	função	`(n, length_mm=63.8) -> dict[str, float]`	Alcances de referência em fórmula fechada para `n` peças: bloco único, empilhamento harmônico e o limite do centro de massa.

pinneapple_design.bricks.ldraw

Intercâmbio LDraw: escreve e lê arquivos .ldr/.mpd e carrega formas reais de peças da biblioteca LDraw (baixada sob demanda).

`pinneapple_design/bricks/ldraw.py` · 281 linhas · testes: 24

nome	tipo	assinatura	o que faz
`ReadReport`	classe	`(assembly, skipped)`	Resultado da leitura de um arquivo LDraw: a montagem e a lista de linhas ignoradas.
`PartsLibrary`	classe	`(root)`	Resolve e triangula arquivos .dat da biblioteca LDraw (sem impor o enrolamento BFC).
`to_ldr`	função	`(asm, name='model.ldr', *, steps=True, author='PINNeAPPle') -> str`	Texto LDraw da montagem, com um `0 STEP` por camada se `steps`.
`write_ldr`	função	`(asm, path, **kw) -> str`	Grava o texto de `to_ldr` num arquivo.
`write_mpd`	função	`(models, path) -> str`	Grava várias montagens num documento multi-peça (.mpd).
`from_ldr`	função	`(text, *, ground=None, tol=0.5) -> ReadReport`	Lê texto .ldr/.mpd: peças do catálogo alinhadas à grade viram blocos; o resto vai para `skipped`.
`read_ldr`	função	`(path, **kw) -> ReadReport`	Lê um arquivo .ldr/.mpd do disco com `from_ldr`.
`default_cache`	função	`() -> str`	Pasta padrão de cache da biblioteca de peças LDraw.
`fetch_parts_library`	função	`(cache_dir=None, urls=LIBRARY_URLS, timeout=120.0) -> str`	Baixa e descompacta a biblioteca LDraw uma vez; levanta RuntimeError se nenhuma URL responder.

`pinneapple_design.bricks.mesh`

Malhas de triângulos de blocos e montagens e exportação para glTF (cor por bloco), STL e USD.

`pinneapple_design/bricks/mesh.py` · 119 linhas · testes: 24

nome	tipo	assinatura	o que faz
`box_mesh`	função	`(lo, hi) -> tuple[np.ndarray, np.ndarray]`	Caixa fechada com normais para fora e quatro vértices por face.
`cylinder_mesh`	função	`(cx, cy, z0, r, h, n=16) -> tuple[np.ndarray, np.ndarray]`	Cilindro fechado com lateral suave e tampas planas.
`merge`	função	`(meshes) -> tuple[np.ndarray, np.ndarray]`	Junta várias malhas (vértices, faces) numa só.
`brick_mesh`	função	`(b, *, studs=True, segments=16, local=False) -> tuple[np.ndarray, np.ndarray]`	Malha de um bloco colocado, em mm, com ou sem pinos.
`assembly_meshes`	função	`(asm, **kw)`	Lista `(vértices_mm, faces, srgb)` de cada bloco da montagem.
`studio_scene`	função	`(asm, title='brick build', fields=None)`	Monta a montagem como uma `Scene` do estúdio de visualização (metros, z para cima), com um material por cor.
`write_gltf`	função	`(asm, path, title='brick build') -> str`	Grava um glTF binário (.glb) com um nó por bloco e sua cor.
`write_usd`	função	`(asm, path, title='brick build') -> str`	Grava a montagem em USD.
`write_stl`	função	`(asm, path, title='brick build') -> str`	Grava um STL binário da montagem inteira em mm.

`pinneapple_design.bricks.parts`

Dimensões nominais dos blocos, catálogo de peças, modelo de massa, cores e o modelo de junta por pino (`JointModel`).

`pinneapple_design/bricks/parts.py` · 215 linhas · testes: 24

nome	tipo	assinatura	o que faz
`PartType`	classe	`(name, nx, ny, h, ldraw, mass, mass_source, smooth)`	Tipo de peça retangular: `nx` x `ny` pinos, `h` placas de altura (3 = bloco, 1 = placa), com massa e código LDraw.
`Colour`	classe	`(name, ldraw, rgb)`	Uma cor da paleta, com código LDraw e RGB.
`JointModel`	classe	`(clutch_n, clutch_band, mu, shear_n, edge_inset_mm, source, notes)`	Resistência de uma junta pino-tubo e atrito de contato usados pela estática; o padrão de `clutch_n` é 0,98 N com faixa assumida de 0,5 a 1,5 vez.

nome	tipo	assinatura	o que faz
`fit_mass_model`	função	`() -> np.ndarray`	Ajusta por mínimos quadrados um modelo de casca `m = a·pinos + b·perímetro·altura` às massas publicadas.
`estimate_mass`	função	`(nx, ny, h) -> float`	Estima a massa de uma peça com o modelo de casca.
`mass_model_residuals`	função	`() -> dict[str, float]`	Erro relativo do modelo de casca em cada massa publicada.
`part`	função	`(name) -> PartType`	Busca uma peça do catálogo pelo nome ou pelo arquivo LDraw.
`smooth_block`	função	`(nx, ny, h, mass=None) -> PartType`	Cria um bloco liso, sem pinos (sem encaixe), para testes de empilhamento solto.
`colour`	função	`(name_or_code) -> Colour`	Busca uma cor pelo nome ou código.

`pinneapple_design.bricks.report`

Resumos de um experimento com blocos: sensibilidade ao modelo de junta, gráfico de comparação de estratégias e relatório em Markdown.

`pinneapple_design/bricks/report.py` · 191 linhas · testes: 24

nome	tipo	assinatura	o que faz
`sensitivity`	função	`(goal, asm, factors=None, shear_caps=(None, 1.0, 0.3)) -> list[dict]`	Reavalia uma montagem com o encaixe escalado na faixa assumida e com cisalhamento por pino limitado.
`plot_comparison`	função	`(summaries, path, *, ylabels=None, display=None) -> str`	Gráfico da melhor pontuação viável contra o número de avaliações, por meta e por estratégia (mediana entre sementes).
`write_report`	função	`(path, *, validation, comparisons, best, sensitivities, references=None, files=None, join…)`	Escreve o relatório Markdown: checagens fechadas, comparação de estratégias, melhores montagens, sensibilidade e limites.

`pinneapple_design.bricks.search`

O laço que se modifica: muta uma montagem, simula cada candidato e guarda o que a física aceita (evolução NSGA-II ou busca guiada por GP).

`pinneapple_design/bricks/search.py` · 371 linhas · testes: 24

nome	tipo	assinatura	o que faz
`Candidate`	classe	`(asm, ev, edit, generation, parent, index)`	Um candidato avaliado: montagem, avaliação, edição aplicada, geração e pai.
`SearchResult`	classe	`(strategy, goal, seed, budget, history, generations, best_curve, wall_time_s, meta)`	Resultado de uma busca: histórico, gerações, curva do melhor, tempo e utilitários por geração.
`mutate`	função	`(goal, asm, rng, last=None, edit=None) -> tuple[Assembly, str]`	Aplica uma edição aleatória (ou a pedida) e devolve a nova montagem e o nome da edição.
`repair`	função	`(goal, asm) -> Assembly`	Remove peças sobrepostas ou flutuando para que a maioria das edições dê um candidato montável.
`evolve`	função	`(goal, budget=200, *, seed=0, mu=10, lam=10, progress=None) -> SearchResult`	Evolução (mu + lambda) com seleção NSGA-II e orçamento de avaliações físicas.
`surrogate_search`	função	`(goal, budget=200, *, seed=0, mu=10, lam=10, pool=8, kappa=1.0, warmup=20, progress=None)…)`	Mesmo laço, mas um surrogate GP ranqueia `pool * lam` filhos por média + kappa·desvio e só os melhores são simulados.
`compare`	função	`(goal_factory, *, budget=200, seeds=(0, 1, 2, 3, 4), strategies=('evolution', 'surrogate'…)`	Roda cada estratégia com cada semente no mesmo orçamento e resume mediana e faixa.

`pinneapple_design.bricks.statics`

Equilíbrio estático de uma montagem por programação linear: forças de contato, utilização das juntas, falhas progressivas, tombamento e capacidade de carga.

`pinneapple_design/bricks/statics.py` · 589 linhas · testes: 24

nome	tipo	assinatura	o que faz
`Body`	classe	`(lo, hi, mass, com, label)`	Corpo rígido em caixa (limites, massa, centro de massa).
`Interface`	classe	`(lower, upper, z, rect, studs, label)`	Retalho de contato entre dois corpos (ou corpo e chão) numa altura, com pinos.
`Load`	classe	`(body, point, force, label)`	Carga pontual aplicada num corpo.
`Structure`	classe	`(bodies, interfaces, loads, gravity)`	Estrutura para a análise: corpos, interfaces, cargas e gravidade.
`StaticsResult`	classe	`(status, utilisation, interface_utilisation, interface_compression, interface_tension, bo…)`	Resultado da estática: status, utilização das juntas, forças, ponto crítico, explicação e tombamento.
`FailureRound`	classe	`(result, broken, new)`	Uma rodada da falha progressiva: resultado e juntas quebradas.
`from_assembly`	função	`(asm, loads=(), broken=()) -> Structure`	Converte uma `Assembly` em `Structure` (caixas pelas células).
`analyse`	função	`(structure, joint=None) -> StaticsResult`	Acha o equilíbrio sob gravidade e cargas com a menor utilização de pino máxima.
`capacity`	função	`(structure, joint=None, lam_max=1000000.0) -> float`	Maior fator sobre as cargas que a estrutura ainda carrega com todos os pinos dentro do limite.
`convex_hull`	função	`(points) -> np.ndarray`	Fecho convexo anti-horário de pontos 2D (cadeia monótona de Andrew).
`signed_distance_inside`	função	`(poly, p) -> float`	Distância de um ponto à borda de um polígono convexo (positiva dentro).
`centre_of_pressure`	função	`(s, members, z0) -> np.ndarray | None`	Onde a resultante dos pesos e cargas de um grupo cruza um plano horizontal.
`tipping_report`	função	`(s, joint=None) -> dict | None`	Pior polígono de apoio deixado pelo centro de pressão, tratando as juntas como inquebráveis.
`analyse_assembly`	função	`(asm, loads=(), joint=None, broken=()) -> StaticsResult`	Roda `analyse` numa montagem com gravidade, cargas e juntas liberadas.
`progressive_failure`	função	`(asm, loads=(), joint=None, max_rounds=50) -> list[FailureRound]`	Solta as juntas mais usadas, rodada a rodada, até a montagem ficar de pé ou virar mecanismo.
`top_load`	função	`(asm, force, *, at=None, label='load') -> Load`	Cria uma carga pontual na face de cima da peça mais alta em um ponto.
`single_stud_torsion`	função	`(asm, loads=(), joint=None, spread_mm=1.5) -> dict`	Verifica se um veredito depende de juntas de pino único sem rigidez à torção, resolvendo de novo com três pontos de contato.

`pinneapple_design.bricks.validation`

Compara a física dos blocos com as fórmulas fechadas e junta os resultados num relatório de confiança.

`pinneapple_design/bricks/validation.py` · 146 linhas · testes: 24

nome	tipo	assinatura	o que faz
`harmonic_limit_lp`	função	`(n=6, L=63.8, tol=0.0001, joint=None) -> float`	Maior fator dos deslocamentos harmônicos que a LP ainda acha estável (por bissecção).
`validate`	função	`(*, newton=None, fea=True, joint=None) -> dict`	Roda todas as checagens fechadas (estática, FEA e, se instalado, Newton).

`pinneapple_design.bricks.visuals`

Imagens do laço: linha do tempo da evolução, visualizador web twin3d, GIF/MP4, renders no Blender e instruções de montagem.

`pinneapple_design/bricks/visuals.py` · 508 linhas · testes: 24

nome	tipo	assinatura	o que faz
`Item`	classe	`(brick, pos, quat, value)`	Um bloco num quadro: posição, orientação e valor de cor.

nome	tipo	assinatura	o que faz
`Marker`	classe	`(size, pos, quat)`	Marcador (por exemplo, junta quebrada) num quadro.
`Frame`	classe	`(items, caption, short, kind, generation, markers, scenery)`	Um quadro da animação: itens, legenda, tipo, geração, marcadores e cenário.
`build_frame`	função	`(asm, values=None, **kw) -> Frame`	Cria um quadro da montagem em repouso, opcionalmente colorido pela utilização.
`scenery`	função	`(asms, thickness=80.0, margin=48.0) -> list[tuple[np.ndarray, np.ndarray]]`	Blocos de mesa sob as montagens para o cenário.
`utilisation`	função	`(asm, loads=(), joint=None) -> np.ndarray`	Pior utilização de pino de cada bloco (0 = sem tração, 1 = no limite).
`replay_frames`	função	`(asm, *, loads=(), joint=None, frames=14, duration=0.6, caption='', short='', generation=…)`	Falha progressiva seguida de replay Newton, um quadro por amostra, juntas quebradas em vermelho.
`evolution_timeline`	função	`(result, goal, *, generations=None, hold=3, replay=True, frames_per_replay=14, duration=0…)`	Quadros de uma `SearchResult`: o melhor de cada geração e, opcionalmente, um rejeitado desabando.
`frame_bounds`	função	`(frames, margin=6.0) -> tuple[np.ndarray, np.ndarray]`	Caixa que envolve todos os quadros.
`timeline_scene`	função	`(frames, title='self-modifying brick build', source='')`	Cena twin3d com um passo de tempo por quadro, para exportar como visualizador web.
`frame_geometry`	função	`(frame, *, colour_by='auto', ghost=None)`	Quadro como geometria para o renderizador de `geometry_library.render`.
`render_frames`	função	`(frames, *, size=(960, 540), azimuth=250.0, elevation=20.0, colour_by='auto', ssaa=1, lab…)`	Renderiza por software os quadros em RGB com uma câmera fixa.
`write_gif`	função	`(path, images, fps=8.0) -> str`	Grava as imagens como GIF.
`write_mp4`	função	`(path, images, fps=8.0) -> str`	Grava as imagens como MP4 H.264 com imageio-ffmpeg ou o programa `ffmpeg`.
`blender_available`	função	`() -> bool`	Diz se o módulo `bpy` do Blender está disponível.
`render_blender`	função	`(asm, out, *, values=None, view='iso', samples=64, size=(1280, 720), title=None, tables=N…)`	Render Cycles da montagem nas cores dos blocos ou colorida por utilização.
`turntable_blender`	função	`(asm, folder, *, frames=24, elevation=0.45, samples=32, size=(960, 540), gif=None, mp4=No…)`	Renders Cycles em volta da montagem, com GIF/MP4 opcionais.
`building_steps`	função	`(asm) -> list[list[int]]`	Índices dos blocos por passo de montagem, uma camada por passo.
`write_instructions`	função	`(asm, folder, *, title='Building instructions', size=(800, 520)) -> str`	Gera imagens por passo, lista de peças, arquivo LDraw e `instructions.md`.

pinneapple_design.design_optimizer

Ponto de entrada do otimizador de projeto: reexporta o laço fechado (espaço de parâmetros, surrogate, otimizador, refinamento por PINN, Pareto) e a camada de estudos de projeto.

`pinneapple_design/design_optimizer/__init__.py` · 116 linhas · testes: 36

pinneapple_design.design_optimizer.adjoint

Otimização de forma por adjunto contínuo com autograd do PyTorch: parametrização de forma (FFD, Bézier, NACA), solver adjunto que calcula dJ/d(forma) e um objetivo de arrasto.

`pinneapple_design/design_optimizer/adjoint.py` · 549 linhas · testes: 151

nome	tipo	assinatura	o que faz
`ShapeParametrization`	classe	`(control_points, device='cpu') -> None`	Parametrização de forma para otimização por gradiente (pontos de controle FFD, curvas de Bézier ou família NACA), com deformação de malha.
`ContinuousAdjointSolver`	classe	`(primal_model, pde_residual_fn, objective_fn) -> None`	Solver adjunto por diferenciação automática: resolve a equação adjunta, calcula a sensibilidade de forma e roda a otimização.
`DragAdjointObjective`	classe	`(surface_pts, nu, alpha=0.0) -> None`	Objetivo de força de arrasto (pressão + viscoso) aproximado como média ponderada sobre pontos de superfície.

pinneapple_design.design_optimizer.airfoil_drag

Estudo resolvido: reduz o arrasto de perfil de uma seção NACA 4 dígitos mantendo a sustentação, com MLP surrogate treinada na verdade de forma fechada (perfil fino + Hoerner) e o `DesignOptLoop`.

`pinneapple_design/design_optimizer/airfoil_drag.py` · 175 linhas · testes: 5

nome	tipo	assinatura	o que faz
`section_aero`	função	`(m, t_c, p=CAMBER_POSITION, alpha_deg=ANGLE_OF_ATTACK_DEG, reynolds=REYNOLDS) -> dict[str…)`	(Cl, Cd) de um design pela verdade de forma fechada.
`airfoil_outline`	função	`(m, t_c, p=CAMBER_POSITION, n_pts=160) -> dict[str, list[float]]`	Pontos de superfície da seção em escala real.
`train_surrogate`	função	`(seed, n_train=N_TRAIN_SAMPLES, n_val=N_VAL_SAMPLES, epochs=SURROGATE_EPOCHS) -> dict[str…)`	Treina a MLP (m, t/c) -> (Cl, Cd) e valida num lote com outra semente.
`run_design_optimization`	função	`(n_iterations=150, seed=None, surrogate_epochs=SURROGATE_EPOCHS) -> dict[str, Any]`	Treina o surrogate, roda o laço fechado e devolve um resultado JSON com base, final, trajetória e erros.

pinneapple_design.design_optimizer.compare

Compara métodos de otimização no mesmo problema, mesmo orçamento e mesmas sementes, usando hipervolume normalizado como métrica de convergência. [sem teste]

`pinneapple_design/design_optimizer/compare.py` · 158 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`MethodComparison`	classe	`(problem, results, ideal, nadir, budget, rows, curves)`	Resultado de uma comparação de métodos: tabela, união das avaliações, frentes, vencedor e exportação para dicionário.
`compare_methods`	função	`(problem, methods, *, budget=300, seeds=(0,), tol=0.01, options=None, progress=None) -> M…)`	Roda cada método uma vez por semente com o mesmo orçamento e tabula o resultado.

pinneapple_design.design_optimizer.constraints

Restrições de projeto como penalidades diferenciáveis: caixa, conservação de massa (divergência), razão entre parâmetros, suavidade (variação total) e um conjunto que as agrega.

`pinneapple_design/design_optimizer/constraints.py` · 254 linhas · testes: 151

nome	tipo	assinatura	o que faz
`ConstraintBase`	classe	`()`	Base abstrata de restrição, com `penalty` e `satisfied`.
`BoxConstraint`	classe	`(theta_min, theta_max, weight=100.0) -> None`	Restrição de limites `theta_min <= theta <= theta_max` com barreira ReLU unilateral.
`MassConservationConstraint`	classe	`(weight=10.0, coords=None, n_neighbors=None) -> None`	Penaliza o quadrado médio da divergência estimada do campo de velocidade.
`GeometricConstraint`	classe	`(min_val=0.1, max_val=10.0, param_pairs=None, weight=50.0) -> None`	Restringe razões entre pares de parâmetros a um intervalo.
`ManufacturabilityConstraint`	classe	`(smoothness_weight=1.0) -> None`	Penaliza variação de alta frequência do vetor de projeto pela variação total.
`ConstraintSet`	classe	`(constraints)`	Agrega várias restrições: soma de penalidades e checagem de todas.

pinneapple_design.design_optimizer.evaluators

Avaliadores que ligam modelos e solvers a um `DesignProblem`: modelo do `pp.zoo`, módulo torch diferenciável e comando externo via CSV. [sem teste]

`pinneapple_design/design_optimizer/evaluators.py` · 168 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ZooEvaluator`	classe	`(name, space, fixed=None, derived=None)`	Usa um modelo do `pp.zoo` como avaliador (fidelidade surrogate, não diferenciável por este caminho); designs fora da faixa de validade ficam inválidos.
`TorchModuleEvaluator`	classe	`(module, outputs, *, input_transform=None, output_transform=None, name='torch-module', fi…)`	Usa um módulo torch como avaliador diferenciável, com transformações de entrada e saída opcionais.
`CommandEvaluator`	classe	`(command, space, outputs, *, timeout=600.0, name='external solver', fidelity='reference',…)`	Roda um solver externo por lote: o comando lê um CSV de designs e escreve um CSV de saídas, sem shell, em diretório temporário.
`from_model_ref`	função	`(ref, space, *, fixed=None, outputs=None) -> Evaluator`	Cria o avaliador certo a partir de um `ModelRef` ou dicionário com `uri` (zoo, TorchScript, etc.).

pinneapple_design.design_optimizer.methods

Métodos de otimização sobre um `DesignProblem` sob o mesmo orçamento: random (hipercubo latino), NSGA-II próprio e do pymoo, SciPy DE e SLSQP, bayesiano (ParEGO) e gradiente por autograd. [sem teste]

`pinneapple_design/design_optimizer/methods.py` · 501 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`OptimizationResult`	classe	`(problem, method, seed, budget, evaluations, wall_s, status, message, notes)`	Todas as avaliações de uma rodada de um método, com custo, frente de Pareto, melhor design e curva de hipervolume.
`normalise_front`	função	`(F, ideal, nadir) -> np.ndarray`	Normaliza objetivos entre os pontos ideal e nadir.
`weight_vectors`	função	`(n_obj, n, seed=0) -> np.ndarray`	Gera `n` vetores de peso no simplex (espaçados para dois objetivos, Dirichlet com semente acima disso).
`available_methods`	função	`(problem=None) -> dict[str, dict[str, Any]]`	Lista cada método com sua família e se pode rodar aqui (pymoo instalado, avaliador diferenciável).
`run`	função	`(problem, method='nsga2', *, budget=400, seed=0, initial=None, progress=None, **options) …)`	Roda um método no problema com no máximo `budget` avaliações; métodos impossíveis voltam com status skipped.

pinneapple_design.design_optimizer.objective

Objetivos de projeto sobre um campo previsto: proxies de arrasto, eficiência térmica, tensão e peso, e soma ponderada.

`pinneapple_design/design_optimizer/objective.py` · 138 linhas · testes: 151

nome	tipo	assinatura	o que faz
`ObjectiveBase`	classe	`()`	Classe base abstrata para objetivos de projeto.
`DragObjective`	classe	`(pressure_weight=1.0, viscous_weight=0.1) -> None`	Proxy de arrasto: média de |u|^2 + peso x p^2 sobre o campo (sem normais de superfície).
`ThermalEfficiencyObjective`	classe	`(heat_field_idx=0, pressure_drop_weight=0.5) -> None`	Eficiência térmica (fluxo de calor sobre queda de pressão proxy), devolvida negativa para minimizar.
`StructuralObjective`	classe	`()`	Proxy de tensão de von Mises pela média dos quadrados do campo.
`WeightMinimizationObjective`	classe	`()`	Proxy de volume pela norma L2 do vetor de parâmetros.
`CompositeObjective`	classe	`(terms=None) -> None`	Soma ponderada de vários objetivos.

pinneapple_design.design_optimizer.optimizer

Espaço de parâmetros em caixa e os três otimizadores do laço: gradiente (Adam/L-BFGS), bayesiano com processo gaussiano e evolutivo (CMA-ES ou AG de reserva).

`pinneapple_design/design_optimizer/optimizer.py` · 666 linhas · testes: 179

nome	tipo	assinatura	o que faz
`ParamSpace`	classe	`(bounds, x0)`	Espaço de parâmetros limitado por caixa, com recorte aos limites.
`OptState`	classe	`(step, best_x, best_y, last_x, last_y)`	Retrato do estado do otimizador em uma iteração.
`DesignOptimizerConfig`	classe	`(method, n_iters, lr, grad_optimizer, grad_clip, n_initial_random, acquisition, xi, kappa…)`	Configuração única dos back-ends (método gradient/bayesian/evolutionary, iterações, taxa de aprendizado, etc.).
`GradientDesignOptimizer`	classe	`(theta_init, cfg) -> None`	Otimizador por gradiente que aplica um passo Adam ou L-BFGS por chamada de `step`.
`BayesianDesignOptimizer`	classe	`(cfg, seed=0) -> None`	Otimizador bayesiano sequencial com processo gaussiano (scikit-learn ou GP NumPy de reserva).
`EvolutionaryDesignOptimizer`	classe	`(cfg, seed=0, multi_objective=False) -> None`	Otimizador populacional: CMA-ES quando `cma` está instalado, senão um AG (mu+lambda) com torneio, cruzamento e mutação.

pinneapple_design.design_optimizer.pareto

Utilidades multiobjetivo: dominância, máscara de Pareto, ordenação não dominada e distância de aglomeração do NSGA-II, representantes da frente e contêiner `ParetoFront`.

`pinneapple_design/design_optimizer/pareto.py` · 341 linhas · testes: 9

nome	tipo	assinatura	o que faz
`ParetoFront`	classe	`(objectives, params, mask)`	Contêiner de uma frente de Pareto: objetivos, parâmetros e máscara, com filtro, hipervolume e gráfico.
`pareto_dominates`	função	`(a, b) -> bool`	True se a solução `a` domina `b` (minimização).
`compute_pareto_front`	função	`(objectives) -> np.ndarray`	Máscara booleana das soluções Pareto-ótimas.
`fast_non_dominated_sort`	função	`(objectives) -> list[np.ndarray]`	Divide as soluções em frentes sucessivas (NSGA-II).
`crowding_distance`	função	`(objectives) -> np.ndarray`	Distância de aglomeração do NSGA-II dentro de uma frente.
`representatives`	função	`(objectives, senses, names, ids=None, balanced='balanced', weights=None) -> dict[str, Any]`	Designs representativos de uma frente: o melhor em cada objetivo e um compromisso equilibrado.

pinneapple_design.design_optimizer.pipeline

Laço fechado de otimização guiada por física (`DesignOptLoop`): avalia o surrogate, otimiza com o back-end escolhido, refina por PINN e monta a frente de Pareto.

`pinneapple_design/design_optimizer/pipeline.py` · 502 linhas · testes: 151

nome	tipo	assinatura	o que faz
`DesignOptConfig`	classe	`(n_iterations, optimizer_cfg, refine_top_k, save_history, verbose, convergence_tol, seed,…)`	Configuração do `DesignOptLoop`: iterações, configuração do otimizador interno, refinamento top-k e outros.
`DesignOptResult`	classe	`(best_theta, best_objective, history_theta, history_objectives, history_all_objectives, p…)`	Resultado completo de um `DesignOptLoop`: melhor design, histórico, frente de Pareto, com gráfico de convergência e resumo.
`DesignOptLoop`	classe	`(param_space, surrogate, objective, *, constraints=None, refinement=None, cfg=None) -> No…)`	Orquestra o laço fechado: parametriza, avalia com o surrogate, otimiza, refina por PINN e monta a frente de Pareto.

pinneapple_design.design_optimizer.presets

Problemas prontos: dissipador de aletas planas (fidelidades surrogate, forma fechada e FVM) e perfil CST (ensemble do zoo), com bases de comparação e geração de corpos/arquivos STL e STEP.

pinneapple_design/design_optimizer/presets.py · 466 linhas · testes: 31 · palavras-chave no código (heurística; o texto acima diz o status real): levanta NotImplementedError em algum caminho

nome	tipo	assinatura	o que faz
HeatsinkOperating	classe	(power_w, t_ambient_c, air_velocity_m_s, source_width_mm, source_depth_mm, tim_k_mm2_w, m…)	Ponto de operação do dissipador: potência, temperatura ambiente, velocidade do ar, tamanho da fonte, resistência de interface.
HeatsinkLimits	classe	(t_source_max_c, pressure_drop_max_pa, fin_stress_max_mpa, fin_gap_min_mm, fin_aspect_max)	Limites do dissipador: temperatura da fonte, queda de pressão, tensão na aleta e limites de extrusão.
CostModel	classe	(material_per_kg, material_yield, per_fin, fixed, currency)	Modelo de custo unitário com valores assumidos (preço do material x massa / rendimento + custo por aleta + fixo).
HeatsinkEvaluator	classe	(fidelity='surrogate', operating=None, cost=None)	Avaliador do dissipador: geometria em forma fechada mais o modelo térmico/hidráulico da fidelidade escolhida.
AirfoilEvaluator	classe	()	Avaliador do perfil: ensemble airfoil_cst_rans_mlp (média e dispersão de Cl, Cd, Cm) mais espessuras em forma fechada.
heatsink_problem	função	(fidelity='surrogate', operating=None, limits=None, cost=None, variables=None, objectives…)	Monta o problema de dimensionamento do dissipador de aletas planas na fidelidade pedida.
heatsink_baseline	função	() -> dict[str, float]	Design convencional de partida do dissipador para comparação.
airfoil_problem	função	(cl_min=0.6, cm_min=-0.12, aft_min=0.012, alpha_range=(0.0, 8.0)) -> DesignProblem	Problema do perfil CST: minimizar Cd e maximizar altura da longarina com restrições de Cl, Cm e espessura traseira.
airfoil_baseline	função	() -> dict[str, float]	Design de referência do perfil: NACA 2412 (ajuste CST) a 4 graus.
preset_info	função	(name) -> dict[str, Any]	O que um formulário precisa para montar um preset: variáveis, padrões, objetivos possíveis, base e referência.
plate_fin_bodies	função	(design) -> list[tuple[str, np.ndarray, np.ndarray]]	Malhas fechadas (metros) da placa base e de cada aleta do dissipador.
airfoil_bodies	função	(design, span=0.2, chord=1.0, n=81) -> list[tuple[str, np.ndarray, np.ndarray]]	Segmento de asa extrudado e fechado da seção CST.
preset_bodies	função	(preset, design) -> list[tuple[str, np.ndarray, np.ndarray]]	Corpos sólidos de um design de preset (heatsink ou airfoil).
design_file	função	(bodies, fmt, name='design') -> bytes	Bytes de STL ou STEP do design, escritos pelos exportadores da biblioteca de geometrias.

pinneapple_design.design_optimizer.refinement

Refina previsões do surrogate continuando o treino de uma PINN com perda de dados + resíduo de EDP.

pinneapple_design/design_optimizer/refinement.py · 225 linhas · testes: 151

nome	tipo	assinatura	o que faz
RefinementConfig	classe	(n_epochs, lr, pde_weight, bc_weight, data_weight, n_collocation, verbose)	Configuração do refinamento por PINN: épocas, taxa de aprendizado e pesos das perdas.
RefinementResult	classe	(theta, u_refined, loss_history, improvement_ratio)	Resultado de um refinamento: parâmetros, saída refinada, histórico de perda e razão de melhora.
PINNRefinement	classe	(pinn_model, pde_residual_fn, domain=None, cfg=None) -> None	Refina a previsão do surrogate treinando uma PINN com perda de dados + resíduo de EDP, para um ou para os k melhores designs.

pinneapple_design.design_optimizer.report

Leva estudos de projeto para dentro e fora dos contratos de dados compartilhados (ParameterSpace, StudyResult, ModelRef) e treina surrogates GP a partir de tabelas do usuário. [sem teste]

pinneapple_design/design_optimizer/report.py · 344 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
SurrogateFit	classe	(space, gp, outputs, metrics, n_train, n_test)	Surrogate GP treinado numa tabela, com seu erro em dados separados.

nome	tipo	assinatura	o que faz
to_parameter_space	função	(space, name, description='', budget=None)	Converte um espaço de projeto em pinneapple_data.contracts.ParameterSpace.
designs_frame	função	(problem, ev, idx=None, status=None)	Tabela pandas com um design por linha: variáveis, saídas, viabilidade, fidelidade e status de verificação.
study_result	função	(problem, result, *, study_id, app, title='', candidates=(), verification=None, compariso…)	Empacota uma otimização ou comparação de métodos como StudyResult, registrando só as checagens que rodaram.
surrogate_from_table	função	(df, inputs, outputs, *, test_fraction=0.2, seed=0, integer=(), bounds=None) -> Surrogate…)	Ajusta um GPSurrogate por saída a partir das linhas de uma tabela e mede o erro em linhas separadas.
model_ref_for_surrogate	função	(fit, name, uri, dataset_name='')	ModelRef de um surrogate treinado em tabela, com faixa de validade e métricas.
table_evaluator	função	(space, df, outputs, *, tol=1e-06, name='your solver results') -> FunctionEvaluator	Avaliador de referência que responde a partir de uma tabela de resultados de solver, casando linhas pelos valores das variáveis.
verify_front	função	(problem, ev, front, n, reference=None)	Escolhe candidatos ao longo da frente e os reavalia com a referência, quando existe.
write_design_files	função	(folder, problem, ev, candidates, status=None, *, labels=None) -> list[str]	Grava os arquivos do estudo numa pasta: avaliações, candidatos com status, variáveis para rodar no solver e o ParameterSpace.
chart_points	função	(problem, ev, front=(), *, limit=2000, labels=None) -> list[dict[str, Any]]	Seleciona e afina pontos para um gráfico de trade-off, mantendo sempre a frente.
verify_uploaded_results	função	(space, candidates, results, objectives=(), constraints=(), *, tol=1e-06)	Verifica candidatos exportados com resultados de solver rodados fora, casando linhas por valores das variáveis.

pinneapple_design.design_optimizer.spec

Monta problemas de projeto a partir de specs JSON simples (fonte preset, zoo ou tabela), como os enviados por um app ou API. [sem teste]

pinneapple_design/design_optimizer/spec.py · 205 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
SpecError	classe	()	Erro de spec que não vira problema, com mensagem legível.
objectives_from	função	(items) -> list[Objective]	Converte a lista de objetivos da spec em Objective; exige ao menos um.
constraints_from	função	(items) -> list[Constraint]	Converte a lista de restrições da spec em Constraint com limites inferior/superior.
zoo_problem	função	(model, objectives, constraints=(), *, variables=None, fixed=None) -> DesignProblem	Problema sobre um modelo do pp.zoo: cada entrada não fixa vira variável limitada pela faixa de validade.
problem_from_spec	função	(spec, table=None) -> tuple[DesignProblem, Any]	Monta (problem, fit) a partir de uma spec; fit só existe para fonte tabela.
reference_for_spec	função	(spec) -> Evaluator | None	Referência de alta fidelidade da spec: o motor FVM para o dissipador, None nos outros casos.

pinneapple_design.design_optimizer.study

Descrição do problema de projeto usada por todos os métodos: variáveis, espaço, objetivos, restrições, avaliador, registro das avaliações, relatório de viabilidade e hipervolume.

pinneapple_design/design_optimizer/study.py · 494 linhas · testes: 49 · palavras-chave no código (heurística; o texto acima diz o status real): levanta NotImplementedError em algum caminho

nome	tipo	assinatura	o que faz
Variable	classe	(name, lower, upper, kind, unit, description)	Uma variável de projeto em [lower, upper]; kind="integer" arredonda.
DesignSpace	classe	(variables)	Lista ordenada de variáveis: limites, reparo, mapeamento ao cubo unitário, amostragem e conversão de/para ParameterSpace.

nome	tipo	assinatura	o que faz
`Objective`	classe	`(name, sense, unit, label)`	Minimizar ou maximizar uma saída nomeada do avaliador.
`Constraint`	classe	`(name, upper, lower, unit, kind, label)`	Restrição `lower <= saída <= upper` com violação normalizada, margem e tipo (física, fabricação, validade).
`Evaluator`	classe	`(name, fidelity, outputs, differentiable, relaxable, description)`	Base de avaliador: mapeia designs X (n, d) em saídas nomeadas, com fidelidade e flag de diferenciável; `evaluate` é abstrato.
`FunctionEvaluator`	classe	`(fn, outputs, *, name='function', fidelity='model', torch_fn=None, description='', relaxa…)`	Avaliador a partir de uma função vetorizada (e, opcionalmente, de sua versão torch).
`Evaluations`	classe	`(X, Y, F, G, feasible, relaxed, fidelity, wall_s, tag)`	Todos os designs avaliados num estudo, em ordem: matriz de objetivos, violações, viabilidade e frente de Pareto.
`DesignProblem`	classe	`(name, space, objectives, constraints, evaluator, description, notes)`	Problema de otimização: variáveis, objetivos, restrições e avaliador, com registro das avaliações.
`constants_of`	função	`(ps) -> dict[str, Any]`	Parâmetros constantes de um `ParameterSpace` ou dicionário.
`feasibility_report`	função	`(problem, ev, idx, active_tol=0.05) -> list[dict[str, Any]]`	Por design: valor, margem e status (violada, ativa, inativa) de cada restrição.
`hypervolume`	função	`(F, ref, n_mc=20000, seed=0) -> float`	Hipervolume dominado pelas linhas de F até `ref`: exato para 1 e 2 objetivos, Monte Carlo acima.

`pinneapple_design.design_optimizer.surrogate`

Envoltório `PhysicsSurrogate` que dá uma interface uniforme (previsão, jacobiano, treino, salvar/carregar) a qualquer `nn.Module` que mapeia parâmetros em campo.

`pinneapple_design/design_optimizer/surrogate.py` · 268 linhas · testes: 151

nome	tipo	assinatura	o que faz
`SurrogateConfig`	classe	`(model_name, in_dim, out_channels, hidden_dim, n_layers, device, checkpoint_path)`	Configuração do `PhysicsSurrogate`: modelo (nome registrado ou MLP), dimensões e camadas.
`PhysicsSurrogate`	classe	`(model, cfg=None) -> None`	Envoltório fino de um `nn.Module` theta -> u com previsão, jacobiano, treino em dataset e checkpoint.

`pinneapple_design.design_optimizer.verify`

Confirmação em alta fidelidade dos designs otimizados e otimização assistida por surrogate GP com aprendizado ativo. [sem teste]

`pinneapple_design/design_optimizer/verify.py` · 254 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`VerificationReport`	classe	`(rows, errors, reference, wall_s, evaluations)`	Relatório de verificação: linhas por design, erros por saída, dados da referência e tempo gasto.
`GPSurrogate`	classe	`(problem, outputs, validity=None, name='gp-surrogate', safe=None, kappa=2.0)`	Processo gaussiano por saída (Matern 5/2, ARD, scikit-learn) em entradas no cubo unitário; saídas positivas em escala log.
`ActiveLearningResult`	classe	`(problem, reference_evals, rounds, final, verification, wall_s, notes)`	Resultado da otimização assistida: avaliações de referência, rodadas, resultado final e verificação.
`select_candidates`	função	`(ev, front, n) -> list[int]`	Escolhe `n` designs espaçados ao longo de uma frente, com as duas pontas.
`verify_candidates`	função	`(problem, ev, idx, reference, reference_problem=None) -> VerificationReport`	Reavalia designs escolhidos com a referência e compara cada saída (erro relativo, viabilidade, status).
`surrogate_assisted`	função	`(problem, reference, *, outputs=None, method='nsga2', n_initial=24, rounds=3, n_verify=6,…)`	Otimiza num GP da referência e o melhora onde o otimizador procura (aprendizado ativo).

pinneapple_design.fitmesh

Reexporta a sessão e as etapas do fluxo fitmesh: leitura do problema, catálogo, sugestões, CAD, plano e geração de malha, relatório.

pinneapple_design/fitmesh/__init__.py · 26 linhas · testes: 21

pinneapple_design.fitmesh.api

API HTTP (FastAPI) sobre `FitMeshSession`: uma sessão por estudo, uma chamada por etapa, mais arquivos e relatório.

pinneapple_design/fitmesh/api.py · 146 linhas · testes: 12

nome	tipo	assinatura	o que faz
`create_app`	função	`(root='fitmesh_runs')`	Cria o app FastAPI das sessões fitmesh numa pasta raiz.

pinneapple_design.fitmesh.cad

Constrói as variantes em CadQuery, exporta STEP e STL e verifica cada uma (limites, BRep válido, volume exato, STL fechado, caixa, comparação com referências publicadas). [sem teste direto pelo inventário]

pinneapple_design/fitmesh/cad.py · 115 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`CadVariant`	classe	`(index, params, step, stl, checks, comparison, volume, area, bbox)`	Uma variante CAD construída: parâmetros, arquivos STEP/STL, checagens, comparação, volume, área e caixa, com `verdict`.
`build_variant`	função	`(tpl, params, out_dir, index, accuracy='design') -> CadVariant`	Constrói uma variante a partir do modelo e dos parâmetros, exporta e roda as checagens.
`compare_with_references`	função	`(tpl, params) -> list[dict[str, Any]]`	Diferença relativa de cada dimensão compartilhada com cada objeto de referência publicado.

pinneapple_design.fitmesh.catalog

Catálogo de modelos de geometria paramétricos (asa NACA, corpo de Ahmed, cilindro, esfera, curva de tubo, degrau, placa com furo, viga, dissipador) com parâmetros, limites e referências publicadas. [sem teste direto pelo inventário]

pinneapple_design/fitmesh/catalog.py · 460 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Param`	classe	`(name, value, unit, typical, limits, role, sensitivity)`	Parâmetro de modelo com valor de referência, unidade, faixa típica, limites, papel e sensibilidade.
`Reference`	classe	`(citation, what, params)`	Objeto de referência publicado: citação, o que foi medido e suas dimensões.
`Template`	classe	`(key, title, physics, keywords, description, params, references, quantities, build, volum…)`	Modelo de geometria paramétrico com física, palavras-chave, parâmetros, referências, construtor CadQuery, volume exato e comprimentos característicos.
`naca4_points`	função	`(m, p, t, chord, n=80) -> np.ndarray`	Pontos de uma seção NACA 4 dígitos fechada com espaçamento cosseno.
`polygon_area`	função	`(pts) -> float`	Área de um polígono 2D.

pinneapple_design.fitmesh.context

Lê uma descrição livre (inglês ou português) em fatos com fonte (física, fluido, velocidade, tamanho, propósito) e gera as perguntas do que falta; calcula a altura da primeira célula para um y+.

pinneapple_design/fitmesh/context.py · 208 linhas · testes: 12

nome	tipo	assinatura	o que faz
`Fact`	classe	`(value, source)`	Um fato do problema com valor e fonte (texto, resposta, padrão).
`ProblemContext`	classe	`(text, facts)`	Fatos lidos do problema, com física, precisão, propriedades do fluido, Reynolds e perguntas pendentes.
`parse`	função	`(text, llm=None) -> ProblemContext`	Extrai fatos de uma descrição livre em inglês ou português, opcionalmente com um LLM.
`wall_unit_height`	função	`(y_plus, speed, length, nu, re, internal=False) -> dict[str, float]`	Altura da primeira célula para um y+ alvo pelo atrito de placa plana.

`pinneapple_design.fitmesh.layers`

Dimensiona camadas prismáticas como pilha geométrica, resolvendo a quantidade que falta. [sem teste direto pelo inventário]

`pinneapple_design/fitmesh/layers.py` · 81 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`layer_total`	função	`(first_height, expansion, n_layers) -> float`	Espessura total de n camadas em progressão geométrica.
`size_layers`	função	`(first_height, expansion=None, n_layers=None, total_thickness=None) -> dict[str, float]`	Resolve a pilha de camadas para a quantidade que falta (razão, número ou espessura total).

`pinneapple_design.fitmesh.llm`

Ajuda opcional de modelo de linguagem (Ollama local, Anthropic ou OpenAI) em cada etapa; tudo que volta é validado, e código CadQuery livre roda em processo isolado com lista de imports e timeout.

`pinneapple_design/fitmesh/llm.py` · 251 linhas · testes: 12

nome	tipo	assinatura	o que faz
`LLMAssist`	classe	`(provider, model, api_key, conversation_store, call, provider_kwargs)`	Escolha do modelo de linguagem (provedor, modelo, chave ou função `call` própria).
`read_problem`	função	`(llm, text) -> dict[str, Any]`	Pede ao LLM os fatos de um texto livre.
`rank_geometries`	função	`(llm, ctx) -> dict[str, Any]`	Pede ao LLM para escolher modelos do catálogo real (chaves desconhecidas são descartadas).
`propose_parameters`	função	`(llm, tpl, ctx, base) -> list[dict[str, Any]]`	Pede ao LLM parâmetros e faixas, cortados nos limites do modelo.
`free_geometry`	função	`(llm, description) -> dict[str, Any]`	Pede ao LLM um script CadQuery quando nenhum modelo serve, checado estaticamente.
`check_code`	função	`(code) -> list[str]`	Checagens estáticas do código: sintaxe, imports permitidos, sem dunder ou E/S, presença de `build()`.
`run_free_geometry`	função	`(code, params, out_dir, tol, timeout=120) -> dict[str, Any]`	Roda o CadQuery escrito pelo modelo num interpretador isolado (`python -I`) com timeout.
`review_mesh_plan`	função	`(llm, plan, ctx) -> MeshPlan`	Pede ao LLM sugestões sobre o plano de malha; só alguns campos podem mudar.

`pinneapple_design.fitmesh.mesh_gen`

Gera as malhas planejadas com Gmsh (sólidos e seções 2D) ou OpenFOAM blockMesh+snappyHexMesh (escoamento 3D) e mede a qualidade via `pinneapple_data.cae`. [sem teste direto pelo inventário]

`pinneapple_design/fitmesh/mesh_gen.py` · 407 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`MeshResult`	classe	`(variant, level, tool, path, files, cells, seconds, quality, layers, error)`	Resultado de uma malha: ferramenta, arquivos, células, tempo, qualidade, camadas e erro, com `verdict`.
`generate`	função	`(tpl, var, plan, out_dir, level=1.0, procs=1, max_cells=3000000.0, log=print) -> MeshResu…)`	Gera a malha de uma variante num nível de refinamento do plano.
`quality`	função	`(path, target) -> dict[str, Any]`	Mede a qualidade de uma malha contra o alvo.

pinneapple_design.fitmesh.mesh_plan

Recomenda uma malha adequada ao propósito (triagem, projeto, verificação): método, tamanhos, camadas por y+, domínio, estimativa de células e custo e família para GCI. [sem teste direto pelo inventário]

pinneapple_design/fitmesh/mesh_plan.py · 264 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
MeshPlan	classe	(kind, tool, elements, purpose, sizes, layers, domain, estimated_cells, estimated_cost, f…)	Plano de malha: tipo, ferramenta, elementos, tamanhos, camadas, domínio, estimativas, família e justificativa.
mesh_kind	função	(tpl, ctx) -> str	Escolhe o tipo de malha (sólido, escoamento 3D, seção 2D) para o modelo e o contexto.
plan_mesh	função	(tpl, params, ctx, budget_cells=5000000.0) -> MeshPlan	Monta o plano de malha adequado ao propósito dentro do orçamento de células.

pinneapple_design.fitmesh.report

Calcula o índice de convergência de malha (GCI) e monta o relatório final JSON+HTML com veredito por etapa. [sem teste direto pelo inventário]

pinneapple_design/fitmesh/report.py · 226 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
gci	função	(values, sizes, safety=None) -> dict[str, Any]	Índice de convergência de malha a partir de três malhas (Roache; Celik et al. 2008).
build_report	função	(session) -> dict[str, Any]	Reúne todas as etapas de uma sessão num relatório com veredito por etapa.
render_html	função	(rep) -> str	Renderiza o relatório em HTML.
write_report	função	(rep, stem) -> dict[str, str]	Grava o relatório em JSON e HTML.

pinneapple_design.fitmesh.session

`FitMeshSession`: o estudo interativo passo a passo, da descrição ao relatório. [sem teste direto pelo inventário]

pinneapple_design/fitmesh/session.py · 295 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
FitMeshSession	classe	(text, out_dir, llm=None, log=print)	Estudo interativo: perguntas, escolha de geometria e parâmetros, CAD, plano, malhas, resultados e relatório.

pinneapple_design.fitmesh.suggest

Ranqueia modelos de geometria para o problema e propõe parâmetros a variar, faixas e amostras por hipercubo latino.

pinneapple_design/fitmesh/suggest.py · 94 linhas · testes: 12

nome	tipo	assinatura	o que faz
suggest_geometries	função	(ctx, top=3) -> list[dict[str, Any]]	Modelos ranqueados por física e palavras-chave, com o motivo e as referências.
scale_to_context	função	(tpl, ctx) -> dict[str, float]	Valores de referência escalados para o tamanho dado no problema.
suggest_parameters	função	(tpl, ctx, base=None) -> list[dict[str, Any]]	Cada parâmetro com recomendação de variar ou manter, faixa proposta e motivo.
sample	função	(tpl, base, ranges, n, seed=0, include_base=True) -> list[dict[str, float]]	Amostras por hipercubo latino nas faixas escolhidas, com a base primeiro.

pinneapple_design.geometry

Subpacote de geometria (antigo `pinneapple_geom`): reexporta `GeometrySpec`, `GeometryAsset`, `MeshData`, SDFs, CSG, domínios físicos 2D/3D, malhadores, RBF e `MeshCollocator`; o construtor STL é importado de forma tolerante (vira `None` se faltar dependência).

`pinneapple_design/geometry/__init__.py` · 228 linhas · testes: 1

pinneapple_design.geometry.bodies

Corpos fechados triangulados para escoamento externo: corpo de Ahmed, esfera, cilindro, caixa e anel.

`pinneapple_design/geometry/bodies.py` · 141 linhas · testes: 43

nome	tipo	assinatura	o que faz
`ahmed_body`	função	`(slant_deg=25.0, scale=1.0, clearance=0.05, n_nose=10) -> Mesh`	Gera o corpo de Ahmed com a traseira inclinada em `slant_deg`, a uma folga acima do chão.
`sphere`	função	`(radius=0.5, centre=(0.0, 0.0, 0.0), n=48) -> Mesh`	Esfera UV.
`cylinder`	função	`(radius=0.5, length=2.0, axis='y', n=48) -> Mesh`	Cilindro fechado centrado na origem, no eixo pedido.
`box`	função	`(size=(1.0, 1.0, 1.0), centre=(0.0, 0.0, 0.0)) -> Mesh`	Caixa alinhada aos eixos.
`annulus`	função	`(r_inner, r_outer, height, n=72, z0=0.0) -> Mesh`	Anel fechado (tubo com tampas planas) em torno do eixo z.

pinneapple_design.geometry.builders

Reexporta o construtor de lotes a partir de STL e os construtores analíticos. [sem teste]

`pinneapple_design/geometry/builders/__init__.py` · 15 linhas · **sem teste direto**

pinneapple_design.geometry.builders.analytic_domain_batch_builder

Gera lotes de treino (colocação e contorno com tags) sem malha, para domínios que são caixas, cilindros, anéis ou caixas com curvas embutidas, sem recentralizar as coordenadas. [sem teste]

`pinneapple_design/geometry/builders/analytic_domain_batch_builder.py` · 709 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`sample_box_tag_batch`	função	`(spec, tag_faces, *, n_col=20000, n_bc_per_face=3000, seed=7, user_ctx=None) -> Dict[str,…)`	Amostra um lote numa caixa (de qualquer dimensão) descrita por `spec.domain_bounds`, com pontos de contorno por face com tag.
`sample_cylinder_tag_batch`	função	`(spec, *, axis_coord, cross_coords, radius, tag_faces, n_col=20000, n_bc_per_face=3000, s…)`	Amostra um lote num cilindro de seção circular extrudado ao longo de um eixo (ex.: `pipe_flow_3d`).
`sample_axisymmetric_wall_tag_batch`	função	`(spec, *, axis_coord, radial_coord, r_wall_fn, dr_wall_fn, tag_faces, n_col=20000, n_bc_p…)`	Amostra um domínio axissimétrico (r, z) cujo raio de parede varia com z (ex.: bocal cônico).
`sample_annulus_tag_batch`	função	`(spec, tag_faces, *, cross_coords, inner_radius, outer_radius, n_col=20000, n_bc_per_face…)`	Amostra uma seção anular, com tags para o círculo interno e o externo.
`sample_box_with_curve_tag_batch`	função	`(spec, *, shape_coords, box_tag_faces=None, curve_tags=None, inside_body_fn=None, n_col=2…)`	Amostra uma caixa 2D com corpos ou curvas embutidos (ex.: perfil de aerofólio) que não são faces da caixa.

pinneapple_design.geometry.builders.stl_domain_batch_builder

Constrói lotes de treino PINN (colocação, contorno, inicial, dados) a partir de um STL via trimesh, com teste de dentro/fora configurável e tags heurísticas de entrada/saída/parede; recentraliza a malha. [sem teste]

`pinneapple_design/geometry/builders/stl_domain_batch_builder.py` · 463 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`VoxelInsideConfig`	classe	`(pitch, target_resolution, fill)`	Configuração do teste de dentro/fora por voxelização.
`SDFConfig`	classe	`(mode, sdf_fn, inside_if_leq)`	Configuração do teste de dentro/fora por SDF (proximidade da malha ou função dada).
`TagHeuristics`	classe	`(enabled, inlet_outlet_axis, inlet_is_min, outlet_is_min, plane_eps_frac, require_normal_…)`	Regras heurísticas para marcar pontos de contorno como entrada, saída, parede ou fixo.
`STLDomainBatchConfig`	classe	`(n_col, n_bc, n_ic, n_data, normalize_unit_box, inside_mode, max_rejection_rounds, oversa…)`	Configuração completa do lote STL: quantidades de pontos, modo de dentro/fora, dispositivo e tags.
`STLDomainBatchBuilder`	classe	`(cfg)`	Carrega o STL, normaliza e amostra o lote de treino com `build`.

`pinneapple_design.geometry.core`

Reexporta `GeometrySpec`, `GeometryAsset`, `MeshData` e as fábricas de ativos.

`pinneapple_design/geometry/core/__init__.py` · 12 linhas · **sem teste direto**

`pinneapple_design.geometry.core.geometry`

Define a especificação declarativa de geometria e o contêiner unificado de ativo geométrico com transformações e grupos de contorno. [sem teste direto]

`pinneapple_design/geometry/core/geometry.py` · 422 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`GeometrySpec`	classe	`(kind, name, path, params)`	Especificação declarativa (tipo, nome, caminho, parâmetros) com validação e impressão digital.
`GeometryAsset`	classe	`(mesh, bounds, units, boundary_groups, meta)`	Contêiner de geometria com malha, limites, unidades, grupos de contorno e transformações.

`pinneapple_design.geometry.core.mesh`

Contêiner leve de malha de triângulos (só numpy) com transformações, normais, áreas e apoio a amostragem para PINN.

`pinneapple_design/geometry/core/mesh.py` · 566 linhas · testes: 174

nome	tipo	assinatura	o que faz
`MeshData`	classe	`(vertices, faces, normals)`	Malha de triângulos (vértices, faces, normais) com limites, transformações, normais e áreas.

`pinneapple_design.geometry.core.registry`

Registro de construtores de ativos e a fábrica `build_geometry_asset` (arquivo ou primitiva). [sem teste direto]

`pinneapple_design/geometry/core/registry.py` · 288 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`build_geometry_asset`	função	`(spec, *, options=None) -> GeometryAsset`	Constrói um `GeometryAsset` a partir de um `GeometrySpec` ou dicionário.
`load_geometry_asset`	função	`(geom, *, options=None) -> GeometryAsset`	Atalho: aceita caminho, `GeometryAsset` ou dicionário e devolve um `GeometryAsset`.

`pinneapple_design.geometry.csg`

Geometria sólida construtiva 2D por SDF para domínios de colocação: formas básicas, união, interseção e diferença com amostragem por rejeição. A base `SDFShape` levanta NotImplementedError nos métodos abstratos.

`pinneapple_design/geometry/csg.py` · 570 linhas · testes: 195 · palavras-chave no código (heurística; o texto acima diz o status real): levanta `NotImplementedError` em algum caminho

nome	tipo	assinatura	o que faz
SDFShape	classe	(bounds_min, bounds_max)	Classe base de formas 2D por SDF, com contains, on_boundary, amostragem e operadores +, *, -.
CSGRectangle	classe	(x_min, y_min, x_max, y_max) -> None	Retângulo alinhado aos eixos.
CSGCircle	classe	(center_x, center_y, radius) -> None	Círculo 2D.
CSGEllipse	classe	(center_x, center_y, a, b) -> None	Elipse 2D alinhada aos eixos.
CSGPolygon	classe	(vertices) -> None	Polígono qualquer dado por vértices ordenados.
CSGUnion	classe	()	União booleana de duas formas: min das SDFs.
CSGIntersection	classe	()	Interseção booleana de duas formas: max das SDFs.
CSGDifference	classe	()	Diferença A menos B: max(sdf_a, -sdf_b).
lshape	função	(width1, height1, width2, height2) -> CSGDifference	Domínio em L.
annulus	função	(cx, cy, r_inner, r_outer) -> CSGDifference	Domínio anular (círculo externo menos interno).
channel_with_hole	função	(length, height, hole_cx, hole_cy, hole_r) -> CSGDifference	Canal retangular com obstáculo circular.
t_junction	função	(h_length, h_height, v_x_start, v_width, v_height) -> CSGUnion	Junção em T: canal horizontal mais ramo vertical.

pinneapple_design.geometry.gen

Reexporta build_primitive e as funções de CadQuery.

pinneapple_design/geometry/gen/__init__.py · 17 linhas · testes: 9

pinneapple_design.geometry.gen.airfoil

Gera a superfície de um aerofólio NACA de 4 dígitos como tensor de pontos. [sem teste direto]

pinneapple_design/geometry/gen/airfoil.py · 81 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
naca_parametric	função	(m=0.0, p=0.0, t_c=0.12, n_pts=100) -> torch.Tensor	Gera pontos da superfície de um NACA 4 dígitos a partir de curvatura, posição e espessura.

pinneapple_design.geometry.gen.cadquery_advanced

Modelos CadQuery paramétricos avançados (curva de tubo com flange, tê, redutor, serpentina, ventilador, engrenagem, dissipador de pinos) e reproduções dos exemplos clássicos da documentação CadQuery. Exige CadQuery.

pinneapple_design/geometry/gen/cadquery_advanced.py · 292 linhas · testes: 9

nome	tipo	assinatura	o que faz
pipe_bend_flanged	função	(p)	Curva de tubo com raio e ângulo dados e flange parafusada em cada ponta.
pipe_tee	função	(p)	Tê igual: tubo principal em X e ramo em +Z.
concentric_reducer	função	(p)	Redutor cônico oco de D1 para D2.
helical_coil	função	(p)	Tubo sólido varrido ao longo de uma hélice.
axial_fan	função	(p)	Cubo com pás de perfil NACA com torção, em padrão polar.

nome	tipo	assinatura	o que faz
involute_tooth_profile	função	(module, teeth, pressure_deg=20.0, n=12) -> List[Tuple[float, float]]	Contorno fechado de uma engrenagem de dentes retos com flancos evolventes.
spur_gear	função	(p)	Engrenagem de dentes retos extrudada a partir do perfil evolvente.
pin_fin_heatsink	função	(p)	Dissipador com base e matriz de pinos.
occ_bottle	função	(p)	Garrafa clássica do OCC (exemplo da documentação CadQuery).
pillow_block	função	(p)	Mancal paramétrico (exemplo da documentação CadQuery).
lego_brick	função	(p)	Bloco de montar com pinos em cima e tubos embaixo (exemplo da documentação CadQuery).
register_advanced_templates	função	(reg)	Registra todos esses construtores num CadQueryRegistry com seus esquemas de parâmetros.

pinneapple_design.geometry.gen.cadquery_gen

Importação de malhas via CadQuery, especificação paramétrica de CAD, registro de construtores e exportação STEP/STL. CadQuery é dependência opcional.

pinneapple_design/geometry/gen/cadquery_gen.py · 404 linhas · testes: 18

nome	tipo	assinatura	o que faz
CadQueryTessellationOptions	classe	(linear_deflection, angular_deflection)	Controles de tesselação (desvios linear e angular).
ParametricCadSpec	classe	(builder_id, params, schema, meta)	Especificação declarativa de CAD: id do construtor, parâmetros e esquema.
CadQueryBuildOptions	classe	(tessellation, heal, center_to_origin, normalize_to_unit_box)	Opções de construção: tesselação, reparo, centralização e normalização.
CadQueryRegistry	classe	(builders, schemas)	Registro de construtores CAD paramétricos e seus esquemas.
cadquery_available	função	() -> bool	Diz se o CadQuery está instalado.
cadquery_to_trimesh	função	(cq_obj, *, options=None)	Converte um objeto CadQuery em memória num trimesh.Trimesh.
build_mesh_from_step	função	(step_path, *, options=None) -> MeshData	Lê um STEP com CadQuery e devolve MeshData.
build_mesh_from_cadquery_object	função	(cq_obj, *, options=None) -> MeshData	Converte um objeto CadQuery em MeshData.
build_mesh_from_parametric_spec	função	(spec, registry, *, options=None) -> MeshData	Constrói MeshData a partir de um ParametricCadSpec usando o registro.
register_default_templates	função	(reg) -> CadQueryRegistry	Registra alguns modelos de exemplo, como prova de conceito.
export_cadquery	função	(cq_obj, out_path, *, fmt=None) -> Path	Exporta um objeto CadQuery para STEP/STL ou outro formato suportado.

pinneapple_design.geometry.gen.car2d

Perfil lateral 2D paramétrico de carro (tipo corpo de Ahmed, sem rodas) e sua rasterização em máscara/SDF.

pinneapple_design/geometry/gen/car2d.py · 92 linhas · testes: 20

nome	tipo	assinatura	o que faz
CarProfile	classe	(slant_deg, windshield_deg, hood, clearance, diffuser_deg, nose, height, slant_length)	Perfil de carro com ângulos de traseira, para-brisa e difusor; dá contorno, altura frontal, máscara e SDF.
sample_designs	função	(n, seed=0) -> list[dict[str, float]]	Amostra projetos por hipercubo latino em DESIGN_SPACE.

pinneapple_design.geometry.gen.domains

Domínios físicos 2D prontos para treinar PINNs (canal, canal com obstáculo, cavidade, L, anel, vários obstáculos, junção T, SDF livre), com regiões de contorno nomeadas e lote PINN. A base levanta NotImplementedError nos métodos abstratos.

pinneapple_design/geometry/gen/domains.py · 754 linhas · testes: 151 · palavras-chave no código (heurística; o texto acima diz o status real): levanta NotImplementedError em algum caminho

nome	tipo	assinatura	o que faz
BoundaryRegion	classe	(name, kind, conditions)	Região de contorno nomeada com tipo e condições.
PhysicsDomain2D	classe	(bounds_min, bounds_max, boundary_regions)	Base dos domínios 2D: SDF, amostragem interna e por região, lote PINN e manifesto de condições.
ChannelDomain2D	classe	(length=2.0, height=1.0, origin=(0.0, 0.0), inlet_velocity=1.0)	Canal retangular com entrada à esquerda e saída à direita.
ChannelWithObstacleDomain2D	classe	(length=2.0, height=1.0, origin=(0.0, 0.0), obstacle_center=(0.5, 0.5), obstacle_radius=0…)	Canal retangular com obstáculo circular.
LidDrivenCavityDomain2D	classe	(size=1.0, origin=(0.0, 0.0), lid_velocity=1.0)	Cavidade quadrada com tampa móvel.
LShapeDomain2D	classe	(size=2.0, origin=(-1.0, -1.0))	Domínio em L para problemas de calor/Laplace.
AnnularDomain2D	classe	(center=(0.0, 0.0), r_inner=0.2, r_outer=1.0, inner_condition=None, outer_condition=None)	Domínio anular entre dois círculos.
MultiObstacleDomain2D	classe	(length=3.0, height=1.0, origin=(0.0, 0.0), obstacles=None, inlet_velocity=1.0)	Canal com vários obstáculos circulares.
TJunctionDomain2D	classe	(main_length=3.0, main_height=0.5, branch_width=0.5, branch_length=1.0, origin=(0.0, 0.0)…)	Junção em T: canal principal horizontal e ramo vertical.
SDFDomain2D	classe	(sdf_fn, *, bounds_min, bounds_max, boundary_samplers=None, boundary_conditions=None)	Domínio flexível a partir de qualquer função SDF e amostradores por região.
get_domain	função	(name, **kwargs) -> PhysicsDomain2D	Instancia um domínio 2D pelo nome do registro (ex.: channel_2d).
list_domains	função	() -> List[str]	Lista os nomes dos domínios 2D registrados.

pinneapple_design.geometry.gen.domains3d

Domínios físicos 3D prontos para PINNs (cavidade, duto, tubo), com regiões de contorno e grade estruturada. Os alvos de contorno de get_pinn_batch são zeros de marcação; a base levanta NotImplementedError.

pinneapple_design/geometry/gen/domains3d.py · 597 linhas · testes: 151 · palavras-chave no código (heurística; o texto acima diz o status real): levanta NotImplementedError em algum caminho

nome	tipo	assinatura	o que faz
PhysicsDomain3D	classe	(bounds_min, bounds_max, boundary_regions)	Base dos domínios 3D: SDF, amostragem, lote PINN e grade estruturada.
LidDrivenCavityDomain3D	classe	(size=1.0, origin=(0.0, 0.0, 0.0), lid_velocity=1.0, lid_axis='x')	Cubo com tampa móvel na face de cima.
ChannelDomain3D	classe	(length=2.0, height=1.0, width=1.0, origin=(0.0, 0.0, 0.0), inlet_velocity=1.0, flow_prof…)	Duto retangular com perfil de entrada uniforme ou outro.
PipeFlowDomain3D	classe	(radius=0.5, length=2.0, origin=(0.0, 0.0, 0.0), inlet_velocity=1.0)	Tubo de seção circular com escoamento em x.
get_domain_3d	função	(name, **kwargs) -> PhysicsDomain3D	Instancia um domínio 3D pelo nome do registro.
list_domains_3d	função	() -> List[str]	Lista os nomes dos domínios 3D registrados.

pinneapple_design.geometry.gen.genetic_sdf

Geometria implícita 'genética' por SDF em torch: um genoma de parâmetros define a forma (hoje só união de círculos). [sem teste]

pinneapple_design/geometry/gen/genetic_sdf.py · 110 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
GeneticSDF	classe	(sdf, genome, bounds_min, bounds_max, dim)	Domínio SDF chamável com genoma explícito e limites.
sdf_union	função	(a, b) -> torch.Tensor	União de SDFs (mínimo) em torch.
sdf_intersection	função	(a, b) -> torch.Tensor	Interseção de SDFs (máximo) em torch.
make_union_circles_sdf	função	(*, n_circles=3, seed=0, bounds_min=(0.0, 0.0), bounds_max=(1.0, 1.0), r_min=0.12, r_max=…)	Cria um domínio aleatório como união de círculos, opcionalmente cortado pela caixa.

pinneapple_design.geometry.gen.primitives

Construtores de malhas primitivas via trimesh e geometrias arquitetadas (tecido, trança), com fábrica genérica e booleanas. project_points_to_surface é marcado como placeholder.

pinneapple_design/geometry/gen/primitives.py · 753 linhas · testes: 66 · palavras-chave no código (heurística; o texto acima diz o status real): placeholder/stub citado no docstring

nome	tipo	assinatura	o que faz
MeshBuilder	classe	()	Tipo (protocolo) de um construtor de malha registrado.
register_builder	função	(*names)	Decorador que registra um construtor sob um ou mais nomes.
list_builders	função	() -> Tuple[str, ...]	Lista todos os nomes de construtores registrados, com apelidos.
build_mesh	função	(name, **params) -> MeshData	Fábrica geral: chama o construtor, aplica transformações e booleana opcional (cai para concatenação se não houver motor booleano).
build_primitive	função	(name, **params) -> MeshData	Constrói uma primitiva pelo nome e devolve MeshData.
project_points_to_surface	função	(points, surface, *, method='nearest') -> np.ndarray	[placeholder] Projeta pontos na superfície por ponto mais próximo (trimesh ou KD-tree); o docstring diz que deveria levantar erro, mas devolve (pontos, distâncias).

pinneapple_design.geometry.gen.sdf_shapes

Biblioteca de SDFs 2D e 3D em numpy: formas básicas, booleanas (também suaves), transformações e a classe SDF orientada a objeto.

pinneapple_design/geometry/gen/sdf_shapes.py · 629 linhas · testes: 151

nome	tipo	assinatura	o que faz
SDF	classe	(fn, *args, **kwargs)	Envoltório orientado a objeto de uma função SDF, com transformações e amostragem.
sdf2d_circle	função	(p, center, radius) -> np.ndarray	SDF de círculo 2D.
sdf2d_rectangle	função	(p, center, half_extents) -> np.ndarray	SDF de retângulo alinhado aos eixos.
sdf2d_rounded_rectangle	função	(p, center, half_extents, radius) -> np.ndarray	SDF de retângulo com cantos arredondados.
sdf2d_ellipse	função	(p, center, semi_axes, iterations=4) -> np.ndarray	SDF de elipse por refinamento de Newton.
sdf2d_capsule	função	(p, a, b, radius) -> np.ndarray	SDF de cápsula 2D entre dois pontos.

nome	tipo	assinatura	o que faz
sdf2d_line_segment	função	(p, a, b, thickness=0.0) -> np.ndarray	SDF de um segmento, com espessura opcional.
sdf2d_triangle	função	(p, v0, v1, v2) -> np.ndarray	SDF de triângulo 2D.
sdf2d_annulus	função	(p, center, r_inner, r_outer) -> np.ndarray	SDF de anel 2D.
sdf2d_sector	função	(p, center, radius, angle_start, angle_end) -> np.ndarray	SDF de setor circular.
sdf2d_convex_polygon	função	(p, vertices) -> np.ndarray	SDF de polígono convexo.
sdf3d_sphere	função	(p, center, radius) -> np.ndarray	SDF de esfera.
sdf3d_box	função	(p, center, half_extents) -> np.ndarray	SDF de caixa alinhada aos eixos.
sdf3d_cylinder	função	(p, center, radius, height, axis=2) -> np.ndarray	SDF de cilindro finito num eixo.
sdf3d_torus	função	(p, center, R, r, axis=2) -> np.ndarray	SDF de toro.
sdf3d_capsule	função	(p, a, b, radius) -> np.ndarray	SDF de cápsula 3D.
sdf3d_rounded_box	função	(p, center, half_extents, radius) -> np.ndarray	SDF de caixa com arestas arredondadas.
sdf3d_cone	função	(p, apex, height, radius) -> np.ndarray	SDF de cone com base fechada.
sdf_union	função	(a, b) -> np.ndarray	União: min(a, b).
sdf_intersection	função	(a, b) -> np.ndarray	Interseção: max(a, b).
sdf_difference	função	(a, b) -> np.ndarray	Diferença: max(a, -b).
sdf_smooth_union	função	(a, b, k=0.1) -> np.ndarray	União suave com raio de mistura k.
sdf_smooth_intersection	função	(a, b, k=0.1) -> np.ndarray	Interseção suave.
sdf_smooth_difference	função	(a, b, k=0.1) -> np.ndarray	Diferença suave.
sdf_translate	função	(sdf_fn, offset) -> Callable[[np.ndarray], np.ndarray]	Translada uma SDF.
sdf_scale	função	(sdf_fn, factor) -> Callable[[np.ndarray], np.ndarray]	Escala uniformemente uma SDF.
sdf_rotate_2d	função	(sdf_fn, angle_rad) -> Callable[[np.ndarray], np.ndarray]	Gira uma SDF 2D.
sdf_mirror_x	função	(sdf_fn) -> Callable[[np.ndarray], np.ndarray]	Espelha uma SDF trocando o sinal de x.
sdf_mirror_y	função	(sdf_fn) -> Callable[[np.ndarray], np.ndarray]	Espelha uma SDF trocando o sinal de y.
sdf_onion	função	(sdf_fn, thickness) -> Callable[[np.ndarray], np.ndarray]	Cria uma casca de espessura dada.
sdf_elongate_2d	função	(sdf_fn, dx=0.0, dy=0.0) -> Callable[[np.ndarray], np.ndarray]	Alonga uma forma 2D em x e/ou y.
sdf_repeat_2d	função	(sdf_fn, period, repetitions=None) -> Callable[[np.ndarray], np.ndarray]	Repete uma SDF 2D com período dado.
circle	função	(center=(0.0, 0.0), radius=0.5) -> SDF	Atalho: SDF de círculo.
rectangle	função	(center=(0.0, 0.0), half_extents=(0.5, 0.5)) -> SDF	Atalho: SDF de retângulo.
ellipse	função	(center=(0.0, 0.0), semi_axes=(0.5, 0.3)) -> SDF	Atalho: SDF de elipse.
annulus	função	(center=(0.0, 0.0), r_inner=0.2, r_outer=0.5) -> SDF	Atalho: SDF de anel.

nome	tipo	assinatura	o que faz
capsule2d	função	(a=(0.0, -0.5), b=(0.0, 0.5), radius=0.2) -> SDF	Atalho: SDF de cápsula 2D.
sphere3d	função	(center=(0.0, 0.0, 0.0), radius=0.5) -> SDF	Atalho: SDF de esfera.
box3d	função	(center=(0.0, 0.0, 0.0), half_extents=(0.5, 0.5, 0.5)) -> SDF	Atalho: SDF de caixa 3D.
cylinder3d	função	(center=(0.0, 0.0, 0.0), radius=0.5, height=1.0, axis=2) -> SDF	Atalho: SDF de cilindro.
torus3d	função	(center=(0.0, 0.0, 0.0), R=0.5, r=0.15, axis=2) -> SDF	Atalho: SDF de toro.

pinneapple_design.geometry.gen.threaded_connection

Malhas 2D axissimétricas (r, z) de conexões rosqueadas rotativas (perfil API, TC50/TC46/TC38) geradas com gmsh. [sem teste]

pinneapple_design/geometry/gen/threaded_connection.py · 442 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
ThreadProfile	classe	(name, tpi, taper, h_factor, clearance_factor)	Parâmetros de uma forma de rosca (fios por polegada, conicidade, altura, folga).
ConnectionSpec	classe	(name, r_bore, r_outer, z_thread_start, z_thread_end, z_shoulder_top, thread, body, n_thr…)	Especificação de um corpo (BOX ou PIN) da conexão: raios, posições e rosca.
ThreadedConnectionMesher	classe	(spec, lc=1.0, lc_thread=None)	Gera a malha 2D axissimétrica com gmsh e amostra pontos.
TC50Geometry	classe	(body='BOX', lc=0.8, lc_thread=None)	Conexão TC50 (diâmetro grande).
TC46Geometry	classe	(body='BOX', lc=0.8, lc_thread=None)	Conexão TC46 (diâmetro médio).
TC38Geometry	classe	(body='BOX', lc=0.8, lc_thread=None)	Conexão TC38 (padrão).
build_thread_polygon	função	(spec, n_threads=None) -> List[Tuple[float, float]]	Monta o polígono (r, z) da superfície rosqueada.
get_connection_geometry	função	(connection, body='BOX', lc=0.8) -> ThreadedConnectionMesher	Fábrica: devolve o malhador de uma conexão pelo nome.

pinneapple_design.geometry.io

Ajudantes de E/S com imports tolerantes (STEP, IGES, CadQuery); build_parametric_part não existe em cadquery_bridge, então sempre vira None. [sem teste]

pinneapple_design/geometry/io/__init__.py · 34 linhas · **sem teste direto**

pinneapple_design.geometry.io.cadquery_bridge

Converte um sólido CadQuery em malha por tesselação e embrulha como PhysicalSample. [sem teste]

pinneapple_design/geometry/io/cadquery_bridge.py · 39 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
cadquery_solid_to_upd	função	(solid, *, tess_linear_deflection=0.2, tess_angular_deflection=0.3, units='mm', meta=None…)	Tessela um sólido CadQuery e devolve um PhysicalSample com vértices e faces.

pinneapple_design.geometry.io.iges

Importa arquivos IGES em MeshData via gmsh, com cache.

pinneapple_design/geometry/io/iges.py · 157 linhas · testes: 6

nome	tipo	assinatura	o que faz
`IgesImportConfig`	classe	`(kind, mesh_size, algorithm_2d, algorithm_3d, curvature_refine, optimize, heal_shapes)`	Configuração do import IGES (tipo, tamanho de malha, algoritmos, reparo).
`iges_to_mesh`	função	`(iges_path, *, cfg=None, cache_dir=None) -> MeshData`	Converte um IGES em `MeshData` via gmsh.

`pinneapple_design.geometry.io.meshio_bridge`

Carrega e salva malhas com meshio (VTK/VTU/MSH/XDMF) e empacota como `PhysicalSample`. [sem teste]

`pinneapple_design/geometry/io/meshio_bridge.py` · 149 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`load_meshio`	função	`(path, *, require_triangles=True) -> MeshData`	Lê uma malha com meshio e devolve `MeshData` de triângulos.
`save_meshio`	função	`(mesh, path, *, file_format=None, point_data=None, cell_data=None) -> None`	Salva `MeshData` com meshio.
`meshio_to_upd`	função	`(path)`	Lê malha e dados de ponto e empacota como amostra UPD.

`pinneapple_design.geometry.io.sdf_meshing`

Utilitários para malhar geometrias implícitas 2D: pontos de contorno, contornos por marching squares e malha triangular via gmsh. [sem teste]

`pinneapple_design/geometry/io/sdf_meshing.py` · 172 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`SDFGrid2D`	classe	`(bounds_min, bounds_max, resolution)`	Grade 2D (limites e resolução) para avaliar uma SDF.
`sample_boundary_points_sdf2d`	função	`(sdf, *, grid=None, n=2048, rng=None, band=0.01) -> np.ndarray`	Amostra pontos de contorno por rejeição numa faixa fina em torno do zero.
`marching_squares_contours`	função	`(sdf, *, grid=None, level=0.0) -> list[np.ndarray]`	Extrai polilinhas do nível `sdf = level`.
`sdf2d_to_tri_mesh`	função	`(sdf, *, grid=None, mesh_size=0.03, cache_path=None) -> MeshData`	Gera malha triangular 2D do conjunto de nível zero com gmsh.

`pinneapple_design.geometry.io.step`

Importa arquivos STEP em `MeshData` via gmsh, com cache. [sem teste]

`pinneapple_design/geometry/io/step.py` · 132 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`StepImportConfig`	classe	`(kind, mesh_size, algorithm_2d, algorithm_3d, curvature_refine, optimize)`	Configuração do import STEP.
`step_to_mesh`	função	`(step_path, *, cfg=None, cache_dir=None) -> MeshData`	Converte um STEP em `MeshData` via gmsh.

`pinneapple_design.geometry.io.stl`

Carregador de STL via trimesh que devolve `MeshData`. [sem teste]

`pinneapple_design/geometry/io/stl.py` · 27 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`load_stl`	função	`(path, *, repair=True, compute_normals=True) -> MeshData`	Lê um STL, opcionalmente repara e calcula normais.

`pinneapple_design.geometry.io.trimesh_bridge`

Adaptador entre `trimesh.Trimesh` e `MeshData` (carga, reparo e conversão).

pinneapple_design/geometry/io/trimesh_bridge.py · 111 linhas · testes: 8

nome	tipo	assinatura	o que faz
TrimeshBridge	classe	()	Adaptador que carrega arquivos via trimesh e converte para MeshData.

pinneapple_design.geometry.mesh

Reexporta os malhadores 2D e 3D. [sem teste]

pinneapple_design/geometry/mesh/__init__.py · 26 linhas · **sem teste direto**

pinneapple_design.geometry.mesh.mesher_2d

Geração de malhas 2D: retângulo estruturado, malha de SDF via gmsh, triangulação de polígono e relatório de qualidade. [sem teste direto]

pinneapple_design/geometry/mesh/mesher_2d.py · 436 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
Mesh2D	classe	(vertices, triangles, boundary_edges, boundary_points, meta)	Malha 2D de triângulos com estatísticas de qualidade e amostragem interna.
mesh_rectangle_structured	função	(x_range=(0.0, 1.0), y_range=(0.0, 1.0), nx=20, ny=20) -> Mesh2D	Malha triangular estruturada de um retângulo.
mesh_sdf_2d	função	(sdf_fn, bounds_min, bounds_max, *, mesh_size=0.05, mesh_size_boundary=None, boundary_n_p…)	Malha 2D de qualidade dentro de um domínio SDF usando gmsh.
mesh_polygon_2d	função	(vertices, *, mesh_size=None, holes=None) -> Optional['Mesh2D']	Triangula um polígono com Delaunay do scipy ou com a biblioteca triangle se houver.
mesh_quality_report	função	(mesh) -> Dict[str, Any]	Relatório de qualidade de uma malha 2D.

pinneapple_design.geometry.mesh.mesher_3d

Malhas tetraédricas a partir de superfícies ou SDFs, nuvens de pontos para física e reconstrução de superfície. [sem teste direto]

pinneapple_design/geometry/mesh/mesher_3d.py · 839 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
Mesh3D	classe	(vertices, tets, surface_faces, boundary_groups, meta)	Malha tetraédrica com faces de superfície e grupos de contorno.
PhysicsPointCloud3D	classe	(interior, boundary, boundary_normals, near_wall, near_wall_dist, boundary_groups, meta)	Pontos internos, de contorno e perto da parede para treino físico.
mesh_from_surface	função	(surface, *, mesh_size=0.0, mesh_size_boundary=0.0, boundary_layer=False, bl_thickness=0.…)	Gera malha de volume a partir de uma superfície fechada (gmsh), com camada-limite opcional.
mesh_from_sdf_3d	função	(sdf_fn, bounds_min, bounds_max, *, resolution=64, mesh_size=0.0, clean=True, volume=True…)	Extrai a superfície de uma SDF 3D por marching cubes e malha o volume.
sample_physics_points	função	(mesh, *, n_interior=10000, n_boundary=2000, n_near_wall=0, near_wall_thickness=0.0, curv…)	Gera pontos de colocação prontos para física a partir de uma malha 3D.
surface_from_pointcloud	função	(points, *, normals=None, method='poisson', depth=8, alpha=0.0, density_quantile=0.05) ->…)	Reconstrói uma superfície de uma nuvem de pontos (Poisson via open3d ou alpha shape via Delaunay).

pinneapple_design.geometry.mesh_collocator

Amostrador de pontos de colocação 3D para PINN a partir de uma malha (interior, contorno com normais, suporte a corpo oco).

pinneapple_design/geometry/mesh_collocator.py · 388 linhas · testes: 151

nome	tipo	assinatura	o que faz
`MeshCollocatorConfig`	classe	`(n_interior, n_boundary, domain, boundary_mode, hollow, seed, max_rejection_tries)`	Configuração do `MeshCollocator` (quantidades, domínio, modo de contorno, oco, semente).
`CollocationBatch3D`	classe	`(X_interior, X_boundary, normals_boundary, boundary_groups)`	Lote de colocação 3D: pontos internos, de contorno, normais e grupos.
`MeshCollocator`	classe	`(mesh, config=None, *, inner_mesh=None)`	Amostra pontos internos e de contorno de uma malha 3D, como arrays ou tensores.

pinneapple_design.geometry.ops

Reexporta as operações de malha: reparo, simplificação, remalha, limpeza, atributos e voxelização. [sem teste]

`pinneapple_design/geometry/ops/__init__.py` · 53 linhas · **sem teste direto**

pinneapple_design.geometry.ops.clean

Pipeline de limpeza de malhas 3D (fusão de vértices, degenerados, buracos, auto-interseção, suavização, decimação), com pymeshlab/open3d opcionais. [sem teste]

`pinneapple_design/geometry/ops/clean.py` · 602 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`CleanMeshConfig`	classe	`(merge_vertices, merge_tol, remove_degenerate, remove_duplicates, remove_unreferenced, fi…)`	Configuração completa do pipeline `clean_mesh`.
`mesh_quality_report_3d`	função	`(mesh) -> Dict[str, Any]`	Relatório de qualidade de uma malha de superfície 3D.
`smooth_mesh`	função	`(mesh, *, method='taubin', iterations=5, lam=0.5, mu=-0.53) -> MeshData`	Suavização Laplaciana ou de Taubin.
`remove_outlier_vertices`	função	`(mesh, *, nb_neighbors=20, std_ratio=2.0) -> MeshData`	Remove vértices atípicos por estatística de vizinhos.
`keep_largest_component`	função	`(mesh) -> MeshData`	Mantém só o maior componente conexo.
`decimate_mesh`	função	`(mesh, *, target_faces=None, ratio=0.5) -> MeshData`	Reduz o número de faces da malha.
`remove_self_intersections`	função	`(mesh) -> MeshData`	Remove faces auto-intersectantes com pymeshlab.
`clean_mesh`	função	`(mesh, *, config=None, verbose=False) -> MeshData`	Roda o pipeline completo de limpeza.

pinneapple_design.geometry.ops.curvature

Geometria diferencial discreta em malhas de triângulos: áreas mistas, Laplaciano cotangente, Laplace-Beltrami e curvaturas.

`pinneapple_design/geometry/ops/curvature.py` · 188 linhas · testes: 5

nome	tipo	assinatura	o que faz
`boundary_vertices`	função	`(mesh_or_vertices, faces=None) -> np.ndarray`	Máscara dos vértices em arestas de borda.
`euler_characteristic`	função	`(mesh_or_vertices, faces=None) -> int`	Característica de Euler V - E + F.
`mixed_areas`	função	`(mesh_or_vertices, faces=None) -> np.ndarray`	Área mista de Voronoi por vértice.
`cotan_laplacian`	função	`(mesh_or_vertices, faces=None) -> sparse.csr_matrix`	Matriz Laplaciana cotangente simétrica.
`laplace_beltrami`	função	`(f, mesh_or_vertices, faces=None) -> np.ndarray`	Laplace-Beltrami pontual de um campo nos vértices.
`gaussian_curvature`	função	`(mesh_or_vertices, faces=None, *, boundary='geodesic') -> np.ndarray`	Curvatura gaussiana por defeito angular.

nome	tipo	assinatura	o que faz
mean_curvature	função	(mesh_or_vertices, faces=None, *, signed=True, boundary='nan') -> np.ndarray	Curvatura média por vértice a partir de Δx = -2Hn.
principal_curvatures	função	(mesh_or_vertices, faces=None) -> Tuple[np.ndarray, np.ndarray]	Curvaturas principais k1 e k2 a partir de H e K.

pinneapple_design.geometry.ops.features

Normais de face e vértice, áreas e um indicador simples de curvatura. [sem teste]

pinneapple_design/geometry/ops/features.py · 88 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
compute_face_normals	função	(mesh) -> np.ndarray	Normais das faces.
compute_vertex_normals	função	(mesh) -> np.ndarray	Normais dos vértices ponderadas pela área.
compute_face_areas	função	(mesh) -> np.ndarray	Áreas das faces.
compute_curvature_proxy	função	(mesh) -> np.ndarray	Indicador leve de curvatura por face.

pinneapple_design.geometry.ops.lbm_bridge

Voxeliza uma malha de superfície (STL/trimesh) numa máscara booleana 3D de obstáculo para o solver LBM, com ângulo de ataque.

pinneapple_design/geometry/ops/lbm_bridge.py · 249 linhas · testes: 7

nome	tipo	assinatura	o que faz
mesh_to_obstacle_mask	função	(mesh, bounds, resolution, *, aoa_deg=0.0, rotation_axis='y', oracle_supersample=2, devic…)	Gera a máscara sólida 3D do LBM a partir de uma malha, com rotação por ângulo de ataque.

pinneapple_design.geometry.ops.meshfree

Utilitários sem malha: interpolação por funções de base radial e SDF implícita reconstruída de nuvem de pontos.

pinneapple_design/geometry/ops/meshfree.py · 397 linhas · testes: 151

nome	tipo	assinatura	o que faz
RBFInterpolator	classe	(kernel='gaussian', eps=1.0, reg=1e-10) -> None	Interpolação de dados espalhados por RBF, com gradiente.
ImplicitSurfaceRBF	classe	(kernel='gaussian', eps=2.0, offset=0.01, reg=1e-08) -> None	Reconstrói uma SDF aproximada de pontos com normais e amostra dentro/contorno.

pinneapple_design.geometry.ops.occ_features

Operações paramétricas diretas do OpenCASCADE (filete, chanfro, casca) sobre sólidos CadQuery/OCC.

pinneapple_design/geometry/ops/occ_features.py · 206 linhas · testes: 9

nome	tipo	assinatura	o que faz
fillet_edges	função	(solid, radius, edge_selector=None)	Aplica filete de raio constante nas arestas de um sólido.
chamfer_edges	função	(solid, distance, edge_selector=None)	Aplica chanfro nas arestas de um sólido.
shell_solid	função	(solid, thickness, faces_to_remove=None)	Esvazia um sólido numa casca de espessura dada.

pinneapple_design.geometry.ops.pointcloud

Utilitários de nuvem de pontos: amostrar de malha e de SDF 2D. [sem teste]

pinneapple_design/geometry/ops/pointcloud.py · 80 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`PointCloud`	classe	`(points, normals, features)`	Nuvem de pontos com normais e atributos.
`mesh_to_pointcloud`	função	`(mesh, *, n_surface=4096, seed=0, device=None, dtype=torch.float32, include_normals=True)…)`	Amostra uma nuvem de pontos da superfície de uma malha.
`sample_pointcloud_from_mesh`	função	`(points, cells, *, n=8192, seed=0) -> np.ndarray`	Atalho usado pelo webapp para amostrar pontos de uma malha.
`sdf2d_to_pointcloud`	função	`(sdf, *, grid=None, n_boundary=4096, band=0.01, seed=0, device=None, dtype=torch.float32)…)`	Amostra pontos de contorno de uma SDF 2D.

`pinneapple_design.geometry.ops.remesh`

Remalha de superfície isotrópica via pymeshlab (melhor esforço). [sem teste]

`pinneapple_design/geometry/ops/remesh.py` · 58 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`remesh_surface`	função	`(mesh, *, max_edge_length=None, target_edge_length=None) -> MeshData`	Remalha a superfície com comprimento de aresta alvo.

`pinneapple_design.geometry.ops.repair`

Reparo de malha por melhor esforço com trimesh (duplicados, degenerados, normais, buracos).

`pinneapple_design/geometry/ops/repair.py` · 61 linhas · testes: 1

nome	tipo	assinatura	o que faz
`repair_mesh`	função	`(mesh, *, remove_duplicates=True, remove_degenerate=True, fix_normals=True, fill_holes=Fa…)`	Repara a malha com trimesh.

`pinneapple_design.geometry.ops.simplify`

Simplificação de malha por trimesh ou decimação quádrica do Open3D.

`pinneapple_design/geometry/ops/simplify.py` · 187 linhas · testes: 1

nome	tipo	assinatura	o que faz
`simplify_mesh`	função	`(mesh, *, target_faces=10000, backend='auto', on_missing='error') -> MeshData`	Simplifica a malha até `target_faces`, com escolha de backend.

`pinneapple_design.geometry.ops.voxelize`

Grades de voxels e amostragem sem malha (domínio, SDF, nuvem de pontos) para gerar pontos de colocação. [sem teste]

`pinneapple_design/geometry/ops/voxelize.py` · 284 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`VoxelGrid`	classe	`(data, origin, voxel_size, dim, meta)`	Grade regular de voxels 2D/3D com conversão mundo-voxel e centros ocupados.
`voxelize_domain`	função	`(bounds, resolution, device=torch.device('cpu')) -> VoxelGrid`	Cria uma grade totalmente ocupada para um domínio retangular.
`voxelize_sdf`	função	`(sdf_fn, bounds, resolution, *, threshold=0.0, device=torch.device('cpu')) -> VoxelGrid`	Voxeliza uma SDF: voxel ocupado onde sdf <= limiar.
`voxelize_pointcloud`	função	`(points, bounds, resolution, *, values=None, aggregate='mean', device=torch.device('cpu')…)`	Rasteriza uma nuvem de pontos numa grade de voxels.
`voxelgrid_to_collocation`	função	`(grid, threshold=0.5, include_values=False)`	Extrai os centros dos voxels ocupados como pontos de colocação.
`sample_voxelgrid`	função	`(grid, n_samples, strategy='random', threshold=0.5) -> torch.Tensor`	Amostra pontos de colocação de uma grade de voxels.

pinneapple_design.geometry.optimize.loop

Envoltório amigável para UI que usa `EvolutionaryDesignOptimizer` (CMA-ES/GA) com parâmetros em dicionário. [sem teste]

`pinneapple_design/geometry/optimize/loop.py` · 124 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`GeometryOptimizer`	classe	`(space, *, seed=0, sigma0=0.2)`	Otimizador de geometria com interface ask/tell/run sobre `EvolutionaryDesignOptimizer`.

pinneapple_design.geometry.retrieval

Busca de peças parecidas: descritor de forma D2 sem ML e índice por similaridade de cosseno, também sobre embeddings pré-calculados.

`pinneapple_design/geometry/retrieval.py` · 122 linhas · testes: 3

nome	tipo	assinatura	o que faz
`GeometryIndex`	classe	`(ids, vectors, captions)`	Índice de similaridade de cosseno (força bruta) sobre vetores de peças, com busca por legenda.
`shape_descriptor`	função	`(mesh, n_pairs=20000, bins=48, seed=0) -> np.ndarray`	Histograma D2 mais extensões principais, normalizado; invariante a rotação, translação e escala.
`load_caption_index`	função	`(parquet_paths, id_column='abc_id', limit=None) -> GeometryIndex`	Monta um índice a partir de shards parquet com `abc_id`, `caption` e `embedding`.

pinneapple_design.geometry.sample

Reexporta os amostradores baricêntricos, de caixa, de superfície e de partículas.

`pinneapple_design/geometry/sample/__init__.py` · 31 linhas · testes: 3

pinneapple_design.geometry.sample.barycentric

Amostragem e interpolação baricêntrica em triângulos. [sem teste]

`pinneapple_design/geometry/sample/barycentric.py` · 66 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`sample_barycentric_uv`	função	`(n, rng=None) -> Tuple[np.ndarray, np.ndarray, np.ndarray]`	Amostra pesos baricêntricos uniformes num triângulo.
`sample_points_on_triangles`	função	`(tri_v0, tri_v1, tri_v2, n, rng=None) -> np.ndarray`	Amostra pontos uniformes num triângulo.
`interpolate_on_triangles`	função	`(values_v0, values_v1, values_v2, w0, w1, w2) -> np.ndarray`	Interpola valores por vértice com pesos baricêntricos.

pinneapple_design.geometry.sample.grids

Amostragem uniforme e por hipercubo latino em caixas. [sem teste]

`pinneapple_design/geometry/sample/grids.py` · 63 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`sample_uniform_box`	função	`(bounds_min, bounds_max, n, rng=None) -> np.ndarray`	Amostras uniformes numa caixa.
`sample_latin_hypercube_box`	função	`(bounds_min, bounds_max, n, rng=None) -> np.ndarray`	Amostras por hipercubo latino numa caixa.

pinneapple_design.geometry.sample.marching_squares

Extrai o contorno de uma SDF 2D em torch por marching squares e amostra pontos dele. [sem teste]

pinneapple_design/geometry/sample/marching_squares.py · 86 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
marching_squares_boundary	função	(sdf, *, bounds_min=(0.0, 0.0), bounds_max=(1.0, 1.0), resolution=256, device=None, dtype…)	Devolve os vértices da polilinha de contorno.
sample_boundary_points	função	(sdf, n, *, bounds_min=(0.0, 0.0), bounds_max=(1.0, 1.0), resolution=256, device=None, dt…)	Amostra n pontos de contorno a partir do contorno extraído.

pinneapple_design.geometry.sample.particles

Amostradores de partículas para SPH e conjuntos baseados em partículas. [sem teste]

pinneapple_design/geometry/sample/particles.py · 51 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
sample_box_particles	função	(*, lo, hi, spacing, jitter=0.0, device=None, dtype=torch.float32) -> torch.Tensor	Grade regular de partículas numa caixa, com tremor opcional.
sample_circle_particles	função	(*, center, radius, spacing, jitter=0.0, device=None, dtype=torch.float32) -> torch.Tensor	Partículas 2D dentro de um círculo.

pinneapple_design.geometry.sample.points

Amostragem de pontos na superfície de malhas de triângulos (por área ou por pesos). [sem teste]

pinneapple_design/geometry/sample/points.py · 108 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
sample_surface_points	função	(mesh, n, *, rng=None, return_normals=True, return_face_id=True)	Amostra pontos uniformes na superfície, com normais e índice da face.
sample_surface_points_weighted	função	(mesh, n, *, face_weights, rng=None, return_normals=True, return_face_id=True)	Amostra pontos com pesos por face dados pelo usuário.

pinneapple_design.geometry.trough

Seção longitudinal de uma calha de metal fundido como malha 2D do OpenFOAM (blockMesh): fundo inclinado, vertedouro, jato entrando por cima e ar acima.

pinneapple_design/geometry/trough.py · 115 linhas · testes: 12

nome	tipo	assinatura	o que faz
TroughSection	classe	(length, slope_deg, weir_height, top, inlet_x, inlet_width, cell, thickness)	Geometria da calha; gera o blockMeshDict, a gravidade inclinada, o nível do banho e grava a malha.

pinneapple_design.geometry.vehicle_cad

CAD paramétrico de veículos tipo rover: primitivas de malha fechada, peças com material e massa, propriedades de massa e montagem em corpos para simulação multicorpo.

pinneapple_design/geometry/vehicle_cad.py · 715 linhas · testes: 19

nome	tipo	assinatura	o que faz
CADPart	classe	(name, body, vertices, faces, material, mass, group)	Peça com malha, corpo, material, massa e grupo; calcula propriedades de massa.
VehicleCAD	classe	(title, parts, frames, notes)	Conjunto de peças agrupadas em corpos, com propriedades de massa, poses, cena e exportação.
orient_outward	função	(mesh) -> Mesh	Inverte o enrolamento de uma malha fechada com volume negativo.
signed_volume	função	(V, F) -> float	Volume com sinal de uma malha fechada.
box	função	(size, center=(0, 0, 0), R=None) -> Mesh	Caixa como malha, com rotação opcional.

nome	tipo	assinatura	o que faz
beveled_box	função	(size, center=(0, 0, 0), bevel=0.02, n=3, R=None) -> Mesh	Caixa com arestas arredondadas.
cylinder	função	(p0, p1, radius, n=32) -> Mesh	Cilindro (ou tronco de cone) fechado entre dois pontos.
tube_shell	função	(axis_p0, axis_p1, r_in, r_out, n=48) -> Mesh	Tubo anular fechado (aro de roda).
revolve_closed	função	(profile, axis_origin=(0, 0, 0), axis=(0, 0, 1), n=48) -> Mesh	Sólido de revolução de um perfil fechado.
paraboloid_dish	função	(diameter, depth, thickness, center, axis, n=40, m=8) -> Mesh	Refletor de antena parabólica como casca.
loft	função	(rings, cap='fan') -> Mesh	Sólido fechado passando por anéis de pontos.
offset_polygon	função	(poly, d) -> np.ndarray	Desloca um polígono convexo para dentro ou para fora.
chamfered_rect	função	(sx, sy, c=0.0) -> np.ndarray	Retângulo com cantos chanfrados a 45 graus.
extrude_polygon	função	(poly, z0, z1, chamfer_top=0.0, chamfer_bottom=0.0, R=None, t=(0, 0, 0)) -> Mesh	Prisma de um polígono convexo com chanfros em cima e embaixo.
chamfered_box	função	(size, center=(0, 0, 0), edge=0.0, top=0.0, bottom=0.0, R=None) -> Mesh	Caixa com chanfros planos a 45 graus.
box_beam	função	(p0, p1, section0, section1=None, up=(0, 0, 1), chamfer=0.0) -> Mesh	Viga de seção caixão reta, com afunilamento e chanfro opcionais.
ladder_rail	função	(p0, p1, width, rail=(0.03, 0.04), rungs=4, side=(0, 1, 0), rung=(0.02, 0.02)) -> Mesh	Trilho em escada: dois trilhos paralelos ligados por travessas.
helix_sweep	função	(origin, axis, r_in, r_out, thickness, pitch, length, n_per_turn=24, phase=0.0) -> Mesh	Hélice de seção retangular (rosca, broca).
mli_blanket	função	(width, height, thickness=0.012, wrinkle=0.004, n=(28, 14), seed=0) -> Mesh	Manta de isolamento multicamada com face externa enrugada.
rotate	função	(mesh, R, about=(0, 0, 0)) -> Mesh	Gira uma malha.
translate	função	(mesh, t) -> Mesh	Translada uma malha.
rot	função	(axis, angle_rad)	Matriz de rotação em torno de um eixo.
merge	função	(meshes) -> Mesh	Junta várias malhas.
mesh_mass_properties	função	(V, F, mass) -> tuple[np.ndarray, np.ndarray, float]	Centro de massa, tensor de inércia e volume de uma malha fechada de densidade uniforme.
combine_mass	função	(items) -> tuple[float, np.ndarray, np.ndarray]	Combina massa, centro de massa e inércia de vários itens.
rover_wheel	função	(radius, width, *, rim_thickness=0.004, grousers=0, grouser_height=0.0, grouser_thickness…)	Roda rígida de rover (aro, garras, raios, cubo) como lista de peças.
solar_panel	função	(width, height, thickness=0.025, cells=(6, 4), gap=0.012, frame=0.03)	Painel solar com moldura, substrato e células.
radiator	função	(width, depth, thickness=0.02, fins=12, fin_height=0.03)	Placa radiadora com aletas.
camera_head	função	(width, depth, height, baseline, lens_radius=0.022, lights=2)	Cabeça de câmera com par estéreo e barras de LED.
actuator	função	(p0, p1, radius, cap_radius=None, cap_length=None, flange=0.012, housing='white_paint', t…)	Atuador rotativo com carcaça, flanges e tampa do motor.
pan_tilt_head	função	(width=0.36, depth=0.15, height=0.12, baseline=0.24, lens_radius=0.02, lights=6, aft_came…)	Cabeça de mastro com pan-tilt, câmeras e LEDs.
drill_assembly	função	(rail_height=1.3, rail_width=0.16, rail_offset=0.075, auger_radius=0.0125, flight_radius=…)	Broca planetária com trilho de implantação, cabeça e trado.

pinneapple_design.geometry.vehicles3d

Veículos 3D paramétricos para CFD e render: carro de passeio, carro de fórmula e lançador de dois estágios.

pinneapple_design/geometry/vehicles3d.py · 687 linhas · testes: 4

nome	tipo	assinatura	o que faz
RoadCar	classe	(length, width, roof_height, clearance, hood_height, deck_height, windshield_deg, slant_d…)	Sedã/cupê paramétrico (fastback ou notchback), com rodas, corpos para CFD e área frontal.
LaunchVehicle	classe	(length, diameter, fairing_length, fairing_diameter, interstage_x, fin_root, fin_tip, fin…)	Lançador de dois estágios com coifa ogiva e quatro aletas; inclui estimativa de Barrowman.
FormulaCar	classe	(wheelbase, front_axle_x, ride_height, front_track, rear_track, wheel_radius, front_tyre_…)	Carro de roda aberta paramétrico (não de nenhuma equipe), com asas, assoalho, difusor e rodas.
road	função	(length=9.0, width=6.5, x0=-2.2) -> Part	Plano de asfalto em z = 0 (só para desenho).
naca4	função	(m=0.06, p=0.4, t=0.1, n=40) -> np.ndarray	Perfil NACA 4 dígitos fechado com bordo de fuga finito.

pinneapple_design.geometry_library

Fachada da biblioteca de geometrias (exposta como `pp.geometries`): listar, construir, exportar, renderizar miniaturas, carregar arquivos pré-gerados e exportar a galeria web.

pinneapple_design/geometry_library/__init__.py · 96 linhas · testes: 34

nome	tipo	assinatura	o que faz
list	função	(category=None) -> builtins.list[str]	Nomes das geometrias da biblioteca, opcionalmente filtrados por categoria.
categories	função	() -> dict[str, builtins.list[str]]	Dicionário categoria -> nomes das geometrias.
export	função	(name, path, fmt=None, lod='light', **params) -> str	Constrói a geometria com os parâmetros dados e grava no formato pedido (ou pela extensão).
render	função	(name, path=None, lod='light', size=(960, 640), **kw)	Renderiza a miniatura de uma geometria com parâmetros padrão (câmera e parâmetros físicos opcionais).
thumbnail	função	(name)	Caminho da miniatura PNG versionada, renderizando-a se não existir.
export_web	função	(directory, **kw) -> dict[str, Any]	Gera o índice da galeria e as miniaturas para o navegador.

pinneapple_design.geometry_library.core

Registro das entradas, especificação de parâmetros com faixas e fontes, o corpo fechado `Body` e o objeto `Geometry` devolvido por `build`.

pinneapple_design/geometry_library/core.py · 321 linhas · testes: 21

nome	tipo	assinatura	o que faz
Param	classe	(name, default, unit, range, doc, choices)	Um parâmetro físico: padrão, unidade, faixa admissível e o que controla.
Source	classe	(what, ref, estimate)	De onde vem um número do mundo real; `estimate=True` marca valor escolhido por julgamento, sem fonte publicada.
Body	classe	(name, vertices, faces, materials, group, aero, closed)	Uma superfície fechada (estanque, normais para fora) ou uma folha aberta só para desenho, com material por corpo ou por face.
GeometrySpec	classe	(name, title, category, description, builder, params, dimensions, frame, sources, physics…)	Especificação de uma entrada: nome, título, categoria, construtor, parâmetros, dimensões, fontes e usos físicos.
Geometry	classe	(name, bodies, params, lod, spec)	Geometria construída: corpos fechados com materiais, em metros, com malha, dimensões, volume, checagens, exportação e renderização.
register	função	(spec) -> GeometrySpec	Adiciona uma `GeometrySpec` ao registro, recusando nomes repetidos e chaves de dimensão inválidas.
get_spec	função	(name) -> GeometrySpec	Devolve a especificação registrada de um nome (carrega as entradas se preciso).
params_hash	função	(name, params, lod) -> str	Hash curto (SHA-256) de nome, nível de detalhe e parâmetros.

nome	tipo	assinatura	o que faz
`build`	função	`(name, lod='light', **params) -> Geometry`	Constrói uma geometria da biblioteca no nível de detalhe `light` ou `fine` com os parâmetros físicos dados.
`names`	função	`() -> list[str]`	Lista ordenada dos nomes registrados.
`info`	função	`(name) -> dict[str, Any]`	Informações de uma entrada: dimensões, parâmetros com faixas, fontes e usos.
`summary_table`	função	`() -> str`	Tabela Markdown da biblioteca (nome, título, tamanho, usos).

`pinneapple_design.geometry_library.entries`

Pacote de entradas: cada módulo dentro dele registra suas geometrias ao ser importado (descoberta automática). [sem teste]

`pinneapple_design/geometry_library/entries/__init__.py` · 7 linhas · **sem teste direto**

`pinneapple_design.geometry_library.entries._adapt`

Adaptador que transforma as listas de peças dos modelos anteriores (`Part` de aero, `CADPart` de veículo) em corpos fechados da biblioteca, soldando peças por material. [sem teste]

`pinneapple_design/geometry_library/entries/_adapt.py` · 103 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`bodies_from_items`	função	`(items, prefix='') -> list[Body]`	Agrupa peças por grupo, solda vértices e separa em cascas conexas, cada uma virando um `Body` com material por face.
`base_name`	função	`(name, material) -> str`	Nome da pele de onde veio uma peça dividida por material (ex.: `fuselage_window` -> `fuselage`).
`drop_degenerate`	função	`(bodies) -> list[Body]`	Remove faces degeneradas (vértices repetidos) dos corpos.
`close_sheets`	função	`(bodies, thickness=0.004) -> list[Body]`	Dá espessura às folhas abertas dos modelos antigos para que todo corpo seja uma casca fechada.

`pinneapple_design.geometry_library.entries.aircraft_new`

Registra três aeronaves movidas a rotor a partir de dados publicados: turboélice regional classe ATR 72-600, helicóptero leve classe H145 e drone quadricóptero classe F450. [sem teste]

`pinneapple_design/geometry_library/entries/aircraft_new.py` · 880 linhas · **sem teste direto**

`pinneapple_design.geometry_library.entries.industrial_bio`

Registra geometrias industriais e biomédicas de referência: dissipador de pinos, rotor centrífugo fechado, prédio alto padrão CAARC e arco da aorta torácica. [sem teste]

`pinneapple_design/geometry_library/entries/industrial_bio.py` · 656 linhas · **sem teste direto**

`pinneapple_design.geometry_library.entries.legacy`

Registra as primeiras entradas, embrulhando modelos já existentes no repositório: avião comercial A320, carro de passeio, carro de Fórmula, lançador de dois estágios e o rover NASA VIPER. [sem teste]

`pinneapple_design/geometry_library/entries/legacy.py` · 288 linhas · **sem teste direto**

`pinneapple_design.geometry_library.entries.marine_rail_space`

Registra entradas marinhas, ferroviárias e espaciais: navio KCS em escala real, casco Wigley analítico, trem de alta velocidade classe ICE 3 e lançador classe Falcon 9. [sem teste]

`pinneapple_design/geometry_library/entries/marine_rail_space.py` · 890 linhas · **sem teste direto**

pinneapple_design.geometry_library.entries.wind_turbine

Registra a turbina eólica de referência NREL 5 MW (Jonkman et al. 2009): pás pela tabela publicada de corda/torção/perfil, cubo, nacele e torre. [sem teste]

pinneapple_design/geometry_library/entries/wind_turbine.py · 160 linhas · **sem teste direto**

pinneapple_design.geometry_library.exporters

Escritores de arquivo determinísticos: STL binário, OBJ + MTL, glTF binário, OpenUSD ASCII e STEP AP214 (B-rep facetado), mais leitura de STL e checagem do STEP via OpenCASCADE.

pinneapple_design/geometry_library/exporters.py · 180 linhas · testes: 21

nome	tipo	assinatura	o que faz
`normalise_format`	função	`(fmt) -> str`	Normaliza o nome do formato (aceita aliases como usd e stp) e valida.
`stl_bytes`	função	`(V, F, header) -> bytes`	STL binário de uma malha (V, F) com cabeçalho de 80 bytes.
`to_stl`	função	`(geom, aero_only=False) -> bytes`	Bytes STL de uma `Geometry` (opcionalmente só os corpos aerodinâmicos).
`to_obj`	função	`(geom) -> bytes`	Texto OBJ de uma `Geometry`, um grupo por superfície com referência ao MTL.
`to_mtl`	função	`(geom) -> bytes`	Arquivo MTL com as cores dos materiais usados.
`to_glb`	função	`(geom) -> bytes`	Bytes glTF binário da cena da geometria com a tabela de materiais.
`to_usda`	função	`(geom) -> bytes`	Texto OpenUSD da cena da geometria.
`to_step`	função	`(geom, aero_only=False) -> bytes`	Arquivo STEP AP214 com um `FACETED_BREP` por corpo fechado (folhas abertas são ignoradas).
`export`	função	`(geom, path, fmt=None) -> str`	Grava a geometria num caminho no formato dado ou deduzido da extensão.
`read_stl`	função	`(path)`	Lê STL binário como triângulos (m, 3, 3).
`step_check`	função	`(path) -> dict`	Relê um STEP com OpenCASCADE (`cadquery`): número de sólidos e volume total.

pinneapple_design.geometry_library.kit

Blocos de construção de superfícies fechadas para as entradas: lofts, sólidos de revolução, seções NACA e asas/pás, caixas arredondadas, cilindros, tubos, extrusões e rotações.

pinneapple_design/geometry_library/kit.py · 481 linhas · testes: 21

nome	tipo	assinatura	o que faz
`rot`	função	`(axis, angle_deg) -> np.ndarray`	Matriz de rotação em torno de um eixo por um ângulo em graus (Rodrigues).
`tf`	função	`(mesh, R=None, t=(0, 0, 0), about=(0, 0, 0), scale=1.0) -> Mesh`	Aplica escala, rotação e translação a uma malha.
`mirror`	função	`(mesh, axis=1) -> Mesh`	Espelha uma malha num eixo, invertendo a ordem das faces.
`with_mirror`	função	`(mesh, axis=1) -> list[Mesh]`	Lista com a malha e sua cópia espelhada.
`merge`	função	`(meshes) -> Mesh`	Junta várias malhas numa só.
`ear_clip_fast`	função	`(P2) -> np.ndarray`	Triangulação por ear clipping de polígono simples anti-horário, com testes vetorizados.
`loft`	função	`(rings, start='cap', end='cap') -> Mesh`	Superfície fechada por anéis, com pontas planas (cap) ou em polo.
`revolve`	função	`(x, r, n=48, axis='x', center=(0.0, 0.0, 0.0), phase=0.0) -> Mesh`	Sólido de revolução de um meridiano r(x) em torno de um eixo.

nome	tipo	assinatura	o que faz
superellipse	função	`(hw, hh, n, e=2.0, zc=0.0, yc=0.0, top_e=None) -> np.ndarray`	Anel de pontos de uma superelipse (com expoente separado para a metade de cima, opcional).
body_of_sections	função	`(xs, hw, zb, zt, n=48, e=2.4, e_top=None) -> Mesh`	Corpo tipo fuselagem com seções superelípticas ao longo de x.
naca4	função	`(m=0.0, p=0.4, t=0.12, n=40, closed_te=True) -> np.ndarray`	Contorno NACA 4 dígitos em sentido anti-horário, com opção de bordo de fuga fechado.
thick_te	função	`(sec, te) -> np.ndarray`	Abre o bordo de fuga até uma espessura finita.
section_ring	função	`(sec2d, chord, twist_deg, le, frame=None, pivot=0.25) -> np.ndarray`	Posiciona uma seção de corda unitária em 3D com torção e bordo de ataque dados.
wing	função	`(stations, n=36, frame=None, tip='cap') -> Mesh`	Loft de asa ou pá por estações (bordo de ataque, corda, torção, espessura, arqueamento), fechado nas pontas.
blend_sections	função	`(a, b, w) -> np.ndarray`	Interpola linearmente duas seções com peso `w`.
round_section	função	`(n) -> np.ndarray`	Círculo unitário no formato de seção, com o mesmo número de pontos de `naca4`.
box	função	`(size, center=(0, 0, 0), R=None) -> Mesh`	Caixa fechada de tamanho dado, com rotação e centro opcionais.
rounded_box	função	`(size, center=(0, 0, 0), radius=0.05, n_side=4, R=None) -> Mesh`	Caixa com arestas e cantos arredondados (soma de Minkowski de caixa e esfera).
cylinder	função	`(p0, p1, radius, n=32, r1=None) -> Mesh`	Cilindro (ou tronco de cone) fechado entre dois pontos.
ellipsoid	função	`(center, radii, nu=24, nv=14, R=None) -> Mesh`	Elipsoide fechado com semieixos dados.
tube	função	`(path, radius, n=16, radii=None) -> Mesh`	Tubo fechado varrido ao longo de uma polilinha com referenciais de transporte paralelo.
tube_loop	função	`(path, radius, n=16) -> Mesh`	Tubo fechado ao longo de uma polilinha fechada (anéis, aros).
torus	função	`(center, R_major, r_minor, nu=48, nv=16, axis=(0, 0, 1)) -> Mesh`	Toro fechado com raios maior e menor.
ear_clip	função	`(poly) -> np.ndarray`	Triangula um polígono simples por ear clipping, em qualquer sentido.
extrude	função	`(poly2d, z0, z1, plane='xy') -> Mesh`	Prisma: polígono extrudado num eixo com tampas trianguladas.
sweep_profile	função	`(profile2d, path, up=(0, 0, 1)) -> Mesh`	Perfil 2D fechado varrido ao longo de um caminho, com tampas (trilhos, alças, nervuras).
is_closed_volume	função	`(mesh) -> bool`	True se a malha encerra volume positivo.

pinneapple_design.geometry_library.meshops

Operações de malha em NumPy/SciPy: soldar vértices, orientar normais, estanqueidade, volume, autointerseção, solidificar folhas e união booleana (via `manifold3d`).

`pinneapple_design/geometry_library/meshops.py` · 413 linhas · testes: 42

nome	tipo	assinatura	o que faz
concat	função	`(meshes) -> Mesh`	Concatena malhas reindexando as faces.
compact	função	`(V, F) -> Mesh`	Remove vértices não referenciados.
weld	função	`(V, F, tol=1e-09) -> Mesh`	Funde vértices coincidentes por tolerância relativa e remove faces colapsadas.
area	função	`(V, F) -> float`	Área total da malha.
volume	função	`(V, F) -> float`	Volume encerrado com sinal (positivo para normais para fora).
face_normals	função	`(V, F, unit=True) -> np.ndarray`	Normais das faces, unitárias ou não.
edge_stats	função	`(V, F) -> dict[str, int]`	Conta arestas de borda, não manifold e com orientação inconsistente.

nome	tipo	assinatura	o que faz
components	função	(F, n_vertices) -> np.ndarray	Rótulo de componente conexo de cada face.
orient_outward	função	(V, F) -> Mesh	Inverte cada componente fechado com volume negativo.
quality	função	(V, F, closed=True) -> dict[str, float]	Resumo de qualidade da malha: contagens de arestas, estanqueidade, volume e área.
self_intersections	função	(V, F, max_pairs=20000000, return_pairs=False)	Conta pares de triângulos não adjacentes que se cruzam, com busca por k-d trees.
union	função	(meshes) -> Mesh	União booleana de malhas fechadas com manifold3d.
crease_normals	função	(V, F, angle_deg=38.0) -> np.ndarray	Normais por canto que mantêm arestas vivas acima de um ângulo.
solidify	função	(V, F, thickness)	Transforma uma folha aberta numa casca fina fechada.
fix_winding	função	(V, F) -> Mesh	Torna a orientação das faces consistente em cada componente e orienta para fora.
refan_concave_caps	função	(V, F, materials=None, min_valence=5)	Retriangula tampas planas em leque que se dobram sobre si (ex.: tampa de perfil arqueado).

pinneapple_design.geometry_library.render

Renderizador por software (NumPy + Pillow, sem GPU) para miniaturas, folha de contatos e GIF giratório, com sombras, oclusão ambiente e supersampling. [sem teste]

pinneapple_design/geometry_library/render.py · 362 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
rasterize	função	(sx, sy, depth, F, W, H, persp=True, max_frag=3000000)	Rasteriza triângulos num z-buffer com id de triângulo e coordenadas baricêntricas por pixel.
render_geometry	função	(geom, path=None, size=(960, 640), azimuth=None, elevation=None, ssaa=2, background='stud…)	Renderiza uma Geometry e devolve uma imagem PIL (e grava se path for dado).
contact_sheet	função	(geoms, path, cols=4, tile=(480, 330), title='') -> str	Grade de miniaturas com nomes, em PNG.
turntable	função	(geoms, path, frames=24, cols=4, tile=(240, 165), duration_ms=90) -> str	GIF animado com todas as geometrias girando numa grade.

pinneapple_design.geometry_library.store

Arquivos pré-gerados: manifesto com SHA-256, cópias locais, cache e Hugging Face Hub; load busca nessa ordem e reconstrói pelo código se o arquivo faltar.

pinneapple_design/geometry_library/store.py · 138 linhas · testes: 21

nome	tipo	assinatura	o que faz
file_name	função	(name, lod, fmt) -> str	Nome do arquivo pré-gerado: <nome>.<lod>.<formato>.
thumbnail_path	função	(name) -> Path	Caminho da miniatura PNG de uma entrada.
sha256	função	(path) -> str	SHA-256 de um arquivo, lido em blocos.
read_manifest	função	() -> dict[str, Any]	Lê o manifesto de arquivos (ou devolve um vazio).
write_manifest	função	(m) -> None	Grava o manifesto de arquivos em JSON.
cache_dir	função	() -> Path	Diretório de cache das geometrias (PINNEAPPLE_GEOMETRY_CACHE ou ~/.cache/pinneapple/geometries).
offline	função	() -> bool	True se o modo offline estiver ligado por variável de ambiente.
hub_url	função	(name, fname) -> str	URL do arquivo no dataset do Hugging Face Hub.

nome	tipo	assinatura	o que faz
load	função	(name, format='stl', lod='light', *, allow_build=True, allow_download=True) -> Path	Caminho do arquivo pré-gerado de uma entrada, buscando repositório, cache e Hub, com conferência de hash.

pinneapple_design.geometry_library.web

Gera o índice JSON da galeria e miniaturas JPEG de cada entrada para o navegador.

pinneapple_design/geometry_library/web.py · 75 linhas · testes: 13

nome	tipo	assinatura	o que faz
index	função	() -> dict[str, Any]	Índice da galeria com o que cada cartão precisa (título, dimensões, arquivo, fontes, parâmetros, código).
export_web	função	(directory, *, thumb_size=(360, 240)) -> dict[str, Any]	Grava index.json e thumbs/<nome>.jpg no diretório e devolve o índice.

pinneapple_design.intent

Fachada do projeto a partir da intenção (pp.design): reexporta os estágios formular, planejar, otimizar, verificar e avaliar, e os objetos tipados de cada um.

pinneapple_design/intent/__init__.py · 73 linhas · testes: 24

pinneapple_design.intent.api

Função design(text): de uma frase a um pacote de projeto verificado, a uma lista de perguntas (NeedsInput) ou a uma recusa (OutOfScope). [sem teste]

pinneapple_design/intent/api.py · 193 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
NeedsInput	classe	(spec)	Pedido dentro do escopo mas com valor crítico faltando: traz as perguntas a responder.
design	função	(text, *, answers=None, llm=None, template=None, seed=0, out_dir=None, verify=True, n_scr…)	Projeta a partir de uma meta em linguagem natural; devolve DesignPackage, NeedsInput ou OutOfScope.

pinneapple_design.intent.cli

Linha de comando pinneapple design "<meta>" para rodar o pipeline a partir do shell, com respostas a perguntas e opção de pular a verificação. [sem teste]

pinneapple_design/intent/cli.py · 64 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
main	função	(argv=None) -> int	Ponto de entrada da CLI pinneapple design (argparse).

pinneapple_design.intent.figures

Figuras do pacote de projeto em matplotlib (Agg): frente da otimização e figuras por template (loop, suporte, dissipador, asa), cada uma dizendo de qual modelo vêm os números. [sem teste]

pinneapple_design/intent/figures.py · 300 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
pareto_figure	função	(ctx, x, y, xlabel, ylabel, out_dir) -> str | None	Gráfico de todas as avaliações do modelo reduzido (viáveis e inviáveis) com o design escolhido.
loop_figures	função	(t, spec, d, ctx, out_dir) -> dict[str, str]	Figuras do template do loop (pista, perfis de velocidade e força normal).
loop_gif	função	(t, d, traj, out_dir, *, n_frames=48, size=(720, 480)) -> str	GIF do carro dando a volta, a partir da trajetória do Newton.
bracket_figures	função	(t, spec, d, ctx, out_dir) -> dict[str, str]	Figuras do template do suporte (otimização e resultado do CalculiX quando rodou).

nome	tipo	assinatura	o que faz
`heatsink_figures`	função	`(t, spec, d, ctx, out_dir) -> dict[str, str]`	Figuras do template do dissipador (otimização e verificação).
`wing_figures`	função	`(t, spec, d, ctx, out_dir) -> dict[str, str]`	Figuras do template da asa (comparação linha sustentadora x vortex lattice).

pinneapple_design.intent.materials

Materiais com propriedades citadas (cada número com o documento de origem) para os templates; valores típicos de datasheet, não admissíveis de norma.

`pinneapple_design/intent/materials.py` · 128 linhas · testes: 21

nome	tipo	assinatura	o que faz
`Prop`	classe	`(value, unit, source, note)`	Uma propriedade de material: valor, unidade, fonte e nota.
`Material`	classe	`(key, label, category, props, processes, notes)`	Um material com categoria, propriedades citadas e processos, com `has` e `cite`.
`heat_sink_material`	função	`(key) -> Material`	Material do próprio modelo de dissipador (condutividade e densidade como a física usa).
`get`	função	`(key) -> Material`	Devolve um material pela chave ou levanta `KeyError`.
`candidates`	função	`(*needs, category=None) -> list[Material]`	Materiais que têm valor citado para todas as propriedades pedidas.
`find`	função	`(text) -> list[str]`	Chaves de materiais citados num texto livre.

pinneapple_design.intent.package

`DesignPackage`: tudo para construir e checar o design e a gravação em pasta (design.json, relatório HTML e PDF, geometria STL/STEP/glTF e pasta `check/`). [sem teste]

`pinneapple_design/intent/package.py` · 400 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`DesignPackage`	classe	`(spec, plan, optimisation, candidate, design, robustness, alternatives, verification, ass…)`	Pacote de projeto completo (spec, plano, otimização, candidato, robustez, verificação, avaliação) com veredito, resumo e gravação em pasta.
`make_geometry`	função	`(name, title, bodies)`	Cria uma `Geometry` da biblioteca a partir de corpos, para exportar STL/STEP/glTF.
`write_html`	função	`(pkg, path) -> str`	Escreve o relatório HTML autocontido do pacote.
`write_pdf`	função	`(pkg, path) -> str`	Escreve um PDF curto com veredito, design, verificação e figuras.

pinneapple_design.intent.parse

Converte intenção em `RequirementSpec`: escolhe o template por palavras-chave, lê números com unidades e preenche os slots; um LLM pode ser plugado via `IntentLLM`, mas o parser determinístico não precisa de rede.

`pinneapple_design/intent/parse.py` · 416 linhas · testes: 21

nome	tipo	assinatura	o que faz
`OutOfScope`	classe	`(text, kind, reason, needs, closest)`	O pedido não pode ser projetado aqui: diz o motivo e o que seria preciso.
`Mention`	classe	`(value, unit, start, end, quote)`	Um número achado no texto com a unidade, posição e trecho citado.
`IntentLLM`	classe	`()`	Protocolo de LLM plugável: qualquer objeto com `extract(text, schema) -> dict`.
`CallLLM`	classe	`(provider='anthropic', model=None, **kw)`	Adaptador de `pinneapple_llm.call_llm` (Anthropic, OpenAI, Ollama) para `IntentLLM`.

nome	tipo	assinatura	o que faz
`register`	função	`(template) -> ProblemTemplate`	Adiciona uma instância de template ao registro usado por `formulate`.
`templates`	função	`() -> dict[str, ProblemTemplate]`	Dicionário dos templates registrados (carrega os embutidos).
`get_template`	função	`(name) -> ProblemTemplate`	Devolve um template pelo nome ou levanta `KeyError`.
`find_quantities`	função	`(text) -> list[Mention]`	Acha cada número do texto com a unidade que o segue; ignora números colados em letras.
`llm_schema`	função	`(tpls) -> dict[str, Any]`	Esquema JSON enviado ao LLM: templates possíveis e seus slots.
`select_template`	função	`(text, tpls=None) -> list[tuple[str, float]]`	Ordena os templates por pontuação de palavras-chave no texto.
`out_of_scope`	função	`(text) -> tuple[str, str] | None`	Detecta pedidos fora de escopo ou recusados por palavras da lista `OUT_OF_SCOPE`.
`formulate`	função	`(text, *, llm=None, answers=None, template=None) -> RequirementSpec | OutOfScope`	Transforma um pedido em `RequirementSpec` (ou `OutOfScope`), aceitando respostas a perguntas e template forçado.

`pinneapple_design.intent.physics`

Subpacote com a física própria dos templates (suporte em L e pista de loop).

`pinneapple_design/intent/physics/__init__.py` · 1 linhas · testes: 21

`pinneapple_design.intent.physics.bracket`

Suporte em L sob carga na ponta: teoria de viga, modelo de elementos finitos hexaédricos próprio e modelo CalculiX (gmsh) com o filete e os furos de parafuso.

`pinneapple_design/intent/physics/bracket.py` · 181 linhas · testes: 21

nome	tipo	assinatura	o que faz
`beam`	função	`(P, b, w, t, E)`	Viga em balanço sob carga na ponta: tensão de flexão na raiz e deflexão na ponta (vetorizado).
`l_mesh`	função	`(a, b, w, t, n_t=2, h=None, n_w=4, fillet=0.0)`	Malha estruturada de hexaedros do L (filete em escada), com nós engastados e carregados.
`hex_model`	função	`(P, a, b, w, t, E, nu, n_t=2, fillet=0.0)`	Resolve o L com `SolidFEM` (C3D8I): deflexão na ponta e tensão nominal na raiz.
`geo_shelf_bracket`	função	`(a, b, w, t, fillet, hole) -> str`	Script gmsh (OpenCASCADE) do L com canto interno filetado e dois furos de parafuso.
`gmsh_exe`	função	`() -> str | None`	Caminho do executável `gmsh`, ou None se não achado.
`export_cad`	função	`(geo, workdir) -> dict[str, str]`	Gera STEP (B-rep exato) e STL a partir do script gmsh.
`calculix_model`	função	`(P, a, b, w, t, fillet, hole, E, nu, rho, size, workdir)`	Malha com gmsh (C3D10), aplica engaste e carga e resolve com CalculiX; devolve pico de von Mises, deflexão e reação.
`section_stress`	função	`(m, r, b, w, t, z_frac=0.45)`	Tensão média na face externa do braço numa seção, e o valor de viga M c / I no mesmo ponto.
`mesh_sizes`	função	`(t) -> tuple[float, float]`	Dois tamanhos de tetraedro quadrático (cerca de dois e três elementos na espessura).

`pinneapple_design.intent.physics.loop_track`

Pista de loop para um carrinho: curva da pista (com clotoides), modelo 1-D do carro ao longo dela, forma fechada e a cena de corpo rígido no NVIDIA Newton.

`pinneapple_design/intent/physics/loop_track.py` · 390 linhas · testes: 21

nome	tipo	assinatura	o que faz
`LoopTrack`	classe	`(radius, clothoid, run_in, run_out, width, pitch)`	Pista em cinco trechos (lançamento, clotoide, arco, clotoide, saída) com curvatura, altura, amostragem e pegada.

nome	tipo	assinatura	o que faz
`Run`	classe	`(completes, v_top, v_exit, n_min, n_max, top_margin, energy_lost, s_fail, profile)`	Resultado do modelo 1-D para um lote de designs, com os perfis do primeiro.
`NewtonLoopResult`	classe	`(completes, v_top, max_gap, s_reached, t, pos, vel, s, wall_s, solver, dt, meta)`	Resultado da simulação Newton: se completou, velocidade no topo, maior folga e trajetória.
`along_track`	função	`(radius, clothoid, v0, mass, *, crr=0.02, cd=0.6, area=0.0006, com_height=0.008, wheel_in…)`	Integra d(v^2)/ds ao longo da pista para um lote de designs; falha se o carro para ou perde contato.
`closed_form`	função	`(radius, clothoid, v0, mass, *, crr=0.02, cd=0.6, area=0.0006, com_height=0.008, wheel_in…)`	Estimativa de triagem: energia até o topo com perdas, condição no topo e carga de pico.
`track_meshes`	função	`(track, thickness=0.003, rail=0.008, n=600)`	Malhas fechadas da tira e dos dois trilhos laterais da pista.
`car_mesh`	função	`(length, width, height, wheel_r)`	Carrinho simples (corpo arredondado + quatro rodas) para desenhos e arquivos.
`newton_scene`	função	`(track, mass, v0, *, radius=0.006, mu=0.0, thickness=0.003, rail=0.008, n=900)`	Monta a cena de corpo rígido: pista como malha estática e carro como esfera lançada em +x.
`newton_check`	função	`(track, mass, v0, *, dt=2.5e-05, duration=None, iterations=12, record_every=None, radius=…)`	Roda o carro no loop no NVIDIA Newton e mede se fica na pista, velocidade no topo e folga máxima.

`pinneapple_design.intent.pipeline`

Estágios RequirementSpec -> PhysicsPlan -> otimização multifidelidade (triagem em forma fechada + NSGA-II ou GP assistido) -> verificação com solver completo, robustez por Monte Carlo e veredito. [sem teste]

`pinneapple_design/intent/pipeline.py` · 450 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`PhysicsPlan`	classe	`(template, title, physics, assumptions, variables, objectives, failure_modes, fidelities,…)`	Plano de física de um template: física, hipóteses, variáveis, objetivos, modelos por fidelidade e modos de falha.
`CallLog`	classe	`(calls, wall)`	Registro de chamadas de modelo por fidelidade: designs avaliados e tempo.
`Candidate`	classe	`(x, design, outputs, closed_form, rank)`	Um design candidato com saídas do modelo reduzido e da forma fechada.
`OptimisationResult`	classe	`(candidates, method, n_screen, n_feasible_screen, n_reduced, log, study, notes)`	Resultado da otimização: candidatos, método, contagens de triagem e registro de chamadas.
`Robustness`	classe	`(p_success, ci, n, per_mode, margin_p05, inputs, sensitivity, level, calibration)`	Probabilidade de sucesso com intervalo de confiança, por modo de falha, e margens no percentil 5.
`Verification`	classe	`(ran, label, reason, result, comparison)`	Resultado da verificação com o solver completo e comparação com modelos reduzido e fechado.
`Assessment`	classe	`(trust, verdict, probability, ci, assumptions, reasons)`	Avaliação final: relatório de confiança, veredito, probabilidade, hipóteses e motivos.
`plan`	função	`(spec, template) -> PhysicsPlan`	Monta o `PhysicsPlan` do template para uma spec sem perguntas abertas.
`decode`	função	`(vs, x) -> dict[str, Any]`	Converte um vetor de design em dicionário de variáveis decodificadas.
`optimise`	função	`(spec, template, *, seed=0, n_screen=2000, budget=600, n_candidates=3, slow_budget=48, pr…)`	Faz triagem, otimiza e devolve até `n_candidates` designs viáveis.
`robustness`	função	`(spec, template, cand, *, n=2000, seed=0, level=REDUCED, log=None, calibration=None, opti…)`	Probabilidade de todas as margens >= 0 sob as entradas incertas, por Monte Carlo.
`verify`	função	`(spec, template, cand, *, log=None, workdir=None, options=None) -> Verification`	Roda o solver completo no candidato (se instalado) e compara com os modelos reduzido e fechado.
`assess`	função	`(spec, template, cand, rob, ver, *, agree_tol=0.1) -> Assessment`	Combina verificação e robustez num `TrustReport` e num veredito.

`pinneapple_design.intent.spec`

Objetos de requisito tipados: `Quantity` em SI com proveniência (declarado, respondido, padrão, faltando), `Slot`, `Question` e `RequirementSpec`. [sem teste]

`pinneapple_design/intent/spec.py` · 241 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`SpecError`	classe	`()`	Valor inaceitável: unidade desconhecida, dimensão errada ou fora da faixa do template.
`Quantity`	classe	`(name, value, unit, provenance, source, quote)`	Um número em SI (ou uma escolha) com proveniência, fonte e trecho citado.
`Slot`	classe	`(name, label, unit, display_unit, keywords, cues, critical, default, default_source, boun…)`	Um valor que um template precisa ou pode usar: unidade SI, faixa, palavras-chave, padrão e criticidade.
`Question`	classe	`(slot, text, unit, choices, why)`	Uma pergunta de acompanhamento para um slot, com unidade, escolhas e motivo.
`ObjectiveChoice`	classe	`(name, sense, label, keywords)`	Um objetivo que o template pode otimizar (saída e sentido min/max).
`RequirementSpec`	classe	`(text, template, values, objectives, questions, issues, unsupported, parser, match_score)`	Especificação de requisitos: texto, template, valores, objetivos, perguntas e problemas encontrados.
`to_si`	função	`(value, unit) -> float`	Converte um valor da unidade dada para SI.
`from_si`	função	`(value, unit) -> float`	Converte um valor em SI para a unidade dada.
`same_dimension`	função	`(a, b) -> bool`	True se duas unidades têm a mesma dimensão física.

pinneapple_design.intent.template

Classe base `ProblemTemplate` e seus tipos (fidelidade, modo de falha, variável, incerteza, resultado completo): como um tipo de pedido se liga a física, variáveis, modelos e falhas. [sem teste]

`pinneapple_design/intent/template.py` · 218 linhas · **sem teste direto** · palavras-chave no código (heurística; o texto acima diz o status real): levanta `NotImplementedError` em algum caminho

nome	tipo	assinatura	o que faz
`Fidelity`	classe	`(level, label, method, modules, executables, check)`	Um nível de modelo (closed_form, reduced, full) com rótulo, método e dependências; sabe se está disponível.
`FailureMode`	classe	`(name, label, description, critical)`	Um modo de falha: saída que guarda a margem (>= 0 não ocorre), rótulo e se é crítico.
`DesignVariable`	classe	`(name, lower, upper, unit, label, integer, choices)`	Variável de projeto com limites, unidade e opção inteira (escolhas discretas como índice).
`Uncertain`	classe	`(name, nominal, spread, dist, unit, source)`	Entrada incerta do estudo de robustez: nominal e meia-largura uniforme ou desvio-padrão normal.
`FullResult`	classe	`(outputs, checks, level, label, wall_s, artifacts, notes, compare, compare_options, compa…)`	O que o solver de alta fidelidade mediu para um design.
`ProblemTemplate`	classe	`(name, title, domain, keywords, physics, assumptions, slots, objectives, failure_modes, f…)`	Classe base dos templates; subclasses definem atributos e implementam `evaluate` e os ganchos do pacote (alguns levantam `NotImplementedError` na base).

pinneapple_design.intent.templates

Templates embutidos; `builtin()` devolve loop do carrinho, suporte, dissipador e asa de planador. [sem teste]

`pinneapple_design/intent/templates/__init__.py` · 12 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`builtin`	função	`() -> list`	Lista das instâncias dos templates embutidos.

pinneapple_design.intent.templates.bracket

Template de suporte em L (cantoneira) que segura uma carga: espessura, largura, perna de fixação e material; modelos viga, FE hexaédrico e CalculiX. [sem teste]

`pinneapple_design/intent/templates/bracket.py` · 346 linhas · **sem teste direto**

nome	tipo	assina-tura	o que faz
`BracketTemplate`	classe	`()`	Template de suporte em L: variáveis, avaliação nas três fidelidades, verificação CalculiX, incertezas e itens do pacote.

`pinneapple_design.intent.templates.example_spring`

Exemplo didático de template: mola helicoidal de compressão (Shigley), usado no guia `docs/guides/design_from_intent.md`; não é registrado por padrão.

`pinneapple_design/intent/templates/example_spring.py` · 91 linhas · testes: 21

nome	tipo	assinatura	o que faz
`CoilSpringTemplate`	classe	`()`	Template de exemplo de mola helicoidal (tensão com fator de Wahl, rigidez, fio de piano).

`pinneapple_design.intent.templates.glider_wing`

Template de asa de planador modelo: envergadura, alongamento, afilamento e seção para menor taxa de afundamento, com linha sustentadora elíptica e vortex lattice. [sem teste]

`pinneapple_design/intent/templates/glider_wing.py` · 297 linhas · **sem teste direto**

nome	tipo	assina-tura	o que faz
`GliderWingTemplate`	classe	`()`	Template de asa de planador modelo, avaliado por linha sustentadora elíptica e vortex lattice grosso e fino.
`profile_cd0`	função	`(re, tc)`	Coeficiente de arrasto de perfil da asa por atrito de placa plana vezes fator de forma.

`pinneapple_design.intent.templates.heatsink`

Template de dissipador de aletas planas (passivo ou com ventilador), reaproveitando a física de forma fechada e o FVM 3-D do dimensionador de dissipadores. [sem teste]

`pinneapple_design/intent/templates/heatsink.py` · 327 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`HeatSinkTemplate`	classe	`()`	Template de dissipador de aletas planas, com forma fechada e FVM 3-D como verificação.

`pinneapple_design.intent.templates.loop_car`

Template do carrinho no loop: raio, entrada em clotoide, velocidade de lançamento, massa e tira da pista; modelos balanço de energia, EDO 1-D e NVIDIA Newton. [sem teste]

`pinneapple_design/intent/templates/loop_car.py` · 469 linhas · **sem teste direto**

nome	tipo	assina-tura	o que faz
`LoopCarTemplate`	classe	`()`	Template do carrinho no loop, com envelope de atrito e arrasto, EDO 1-D e verificação no Newton.

`pinneapple_design.qualitative`

Reexporta `preview`, `part_sensitivity`, `Assembly` e os modelos qualitativos.

`pinneapple_design/qualitative/__init__.py` · 36 linhas · testes: 13

`pinneapple_design.qualitative.descriptors`

Descritores geométricos de corpos triangulados fechados (volume, área molhada, área frontal, esbeltez, perfil de seções) e `Assembly` de peças nomeadas. [sem teste direto pelo inventário]

pinneapple_design/qualitative/descriptors.py · 259 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
Descriptors	classe	(volume, wetted_area, centroid, extents, length, frontal_area, equivalent_diameter, slend…)	Descritores geométricos de um corpo (volume, áreas, centroide, extensões, esbeltez, esfericidade e outros).
Assembly	classe	(parts)	Peças nomeadas de uma geometria que podem ser escaladas, movidas, removidas ou trocadas.
as_mesh	função	(geom) -> tuple[np.ndarray, np.ndarray]	Obtém (vértices, faces) de tupla, objeto tipo trimesh ou Assembly.
extrude	função	(polygon, depth=1.0, axis='y') -> tuple[np.ndarray, np.ndarray]	Prisma fechado a partir de um polígono 2D, para tratar seções 2D como corpos.
face_geometry	função	(V, F)	Área, normal unitária e centroide de cada face.
describe	função	(geom, direction=(1.0, 0.0, 0.0)) -> Descriptors	Calcula os Descriptors de um corpo fechado para uma direção de escoamento.
section_profile	função	(geom, axis=0, n_slices=40, n_grid=48)	Propriedades de seção ao longo de um eixo por voxelização.

pinneapple_design.qualitative.models

Modelos físicos baratos avaliados face a face: escoamento externo (Newtoniano modificado e atrito), resfriamento convectivo, viga engastada e fórmulas do usuário; a classe base levanta NotImplementedError em evaluate (interface). [sem teste direto pelo inventário]

pinneapple_design/qualitative/models.py · 501 linhas · **sem teste direto** · palavras-chave no código (heurística; o texto acima diz o status real): levanta NotImplementedError em algum caminho

nome	tipo	assinatura	o que faz
Quantity	classe	(name, unit, better, pt, en)	Quantidade de um modelo qualitativo com unidade, sentido de melhora e nomes pt/en.
Evaluation	classe	(quantities, terms, by_part, face_field, field_label, notes, caveats, mesh, extra)	Resultado de um modelo numa geometria: quantidades, termos, contribuição por peça, campo na superfície e ressalvas.
QualitativeModel	classe	(**conditions)	Classe base dos modelos qualitativos com condições padrão; evaluate é abstrato.
ExternalFlow	classe	()	Arrasto e sustentação estimados por pressão Newtoniana modificada, separação e atrito de placa plana.
ConvectiveCooling	classe	()	Estimativa do calor dissipado por convecção de um corpo.
Cantilever	classe	()	Estimativa da deflexão de viga engastada com carga na ponta.
ScalingModel	classe	(formulas, better=None, direction=(1.0, 0.0, 0.0), **conditions)	Modelo com fórmulas do usuário em função dos descritores geométricos.
sheltered_faces	função	(V, F, normals, centroids, direction, grid=96, tol=0.05) -> np.ndarray	Marca faces de barlavento escondidas atrás de outras superfícies vistas de montante.

pinneapple_design.qualitative.preview

Compara geometrias ou mudanças de peça para um objetivo, com direção, mecanismo, confiança e mapa na superfície; quantify roda a conta exata só onde vale e confere a leitura qualitativa.

pinneapple_design/qualitative/preview.py · 523 linhas · testes: 13

nome	tipo	assinatura	o que faz
Objective	classe	(quantity, sense, model)	Objetivo: quantidade, sentido (min/max) e modelo.
VariantPreview	classe	(name, evaluation, change, mechanisms, parts, confidence, reasons, good)	Prévia de uma variante: avaliação, mudança, mecanismos, peças, confiança e se é boa.
QualitativePreview	classe	(model, objective, baseline, variants, geometries, lang)	Comparação qualitativa de geometrias com ranking, texto, figura e quantify.

nome	tipo	assinatura	o que faz
`QuantitativeCheck`	classe	`(preview, values, seconds)`	Confere a prévia contra valores quantitativos: concordância de direção e correlação de ranking.
`PartSensitivity`	classe	`(objective, model, base_value, rows, lang)`	Sensibilidade do objetivo a crescer cada peça em cada eixo.
`parse_objective`	função	`(objective, model=None) -> Objective`	Interpreta objetivos como "minimizar arrasto" ou `("tip_deflection", "min")`.
`preview`	função	`(geometries, objective, conditions=None, baseline=None, model=None, lang='pt') -> Qualita…)`	Compara geometrias para um objetivo e devolve a `QualitativePreview`.
`part_sensitivity`	função	`(assembly, objective, conditions=None, step=0.2, model=None, lang='pt', axes='xyz', about…)`	Mede quanto o objetivo muda ao crescer cada peça de um `Assembly`.

`pinneapple_design.qualitative.quantitative`

Contrapartes quantitativas: elasticidade 3D por FEM em voxels com hexaedros de modos incompatíveis de Wilson, para checar prévias de viga engastada.

`pinneapple_design/qualitative/quantitative.py` · 132 linhas · testes: 13

nome	tipo	assinatura	o que faz
`hex8_incompatible_stiffness`	função	`(size, E, nu) -> np.ndarray`	Rigidez 24×24 de um tijolo com modos incompatíveis de Wilson condensados.
`voxel_fem_cantilever`	função	`(geom, E=70000000000.0, nu=0.3, load=1000.0, axis=None, load_axis=2, clamp='min', n_slice…)`	Deflexão na ponta, rigidez e tensão de von Mises de uma viga engastada voxelizada por FEM 3D.

`pinneapple_design.surrogate_inverse`

Projeto inverso por surrogate com cinco casos (dissipador, disco de freio, adesivo transdérmico, braço de suspensão, duto bass-reflex) que compartilham o mesmo método e são servidos por apps em `apps/`.

`pinneapple_design/surrogate_inverse/__init__.py` · 38 linhas · testes: 91

`pinneapple_design.surrogate_inverse.brake_rotor`

Caso do disco de freio ventilado: as aletas radiais entre os anéis tratadas como arranjo de aletas (ferro fundido) sob carga contínua, com Monte Carlo das tolerâncias de usinagem.

`pinneapple_design/surrogate_inverse/brake_rotor.py` · 269 linhas · testes: 94

nome	tipo	assinatura	o que faz
`RotorDesign`	classe	`(vane_count, vane_gap_m, vane_thickness_m)`	Design do disco: número, vão e espessura das aletas radiais, com espaçamento derivado.
`OptimizationResult`	classe	`(design, surrogate_predicted_temp_c)`	Resultado da otimização do disco: design e temperatura prevista.
`profile_params`	função	`(design, power_w, h_conv) -> dict[str, float]`	Grandezas da fórmula com elevação na raiz da aleta e comprimento radial corrigido.
`rotor_temperature_c`	função	`(design, power_w, h_conv) -> float`	Verdade: temperatura do anel de atrito sob calor contínuo.
`thermal_resistance_k_w`	função	`(design, h_conv) -> float`	Resistência térmica do disco (elevação por watt).
`vane_temperature_at_radial_position_c`	função	`(design, power_w, h_conv, r_fraction) -> float`	Perfil de temperatura ao longo da aleta, da borda interna (mais quente) à externa.
`thermal_field`	função	`(design, power_w, h_conv) -> dict[str, float]`	Parâmetros para colorir as aletas pelo perfil radial.
`constraint_violations`	função	`(design) -> dict[str, float]`	Violações com sinal (espaçamento mínimo, limites de vão e espessura).
`design_to_dict`	função	`(design) -> dict`	Dicionário do design com contagem arredondada e espaçamento.
`generate_dataset`	função	`(n_samples, seed=0) -> tuple[torch.Tensor, torch.Tensor]`	Designs aleatórios com perfis de resfriamento, rotulados pela fórmula.

nome	tipo	assinatura	o que faz
`train_surrogate`	função	`(n_samples=6000, epochs=1500, seed=0) -> SurrogateFit`	Treina a MLP (saída em log) no dataset do disco.
`predict_temperature_c`	função	`(surrogate, design, power_w, h_conv) -> float`	Temperatura do disco prevista pelo surrogate.
`optimize_design`	função	`(surrogate, power_w, h_conv, steps=400, lr=0.05, penalty_weight=100.0) -> OptimizationRes…)`	Minimiza a temperatura prevista sobre (N, vão, t), como no caso do dissipador.
`verify_design`	função	`(design, power_w, h_conv, surrogate_predicted_temp_c) -> VerificationCard`	Reconfere o design com a fórmula e as regras de projeto.
`monte_carlo_uncertainty`	função	`(design, power_w, h_conv, safety_threshold_c, n_samples=200, seed=None, thickness_toleran…)`	Monte Carlo das tolerâncias de espessura e vão pela fórmula, com veredito no percentil 95.
`mesh`	função	`(design) -> meshes.Mesh`	Malha dos dois anéis e das aletas, em milímetros.
`export_stl_bytes`	função	`(design) -> bytes`	Bytes STL do disco ventilado.

pinneapple_design.surrogate_inverse.core

Maquinário comum: MLP surrogate com entradas normalizadas, treino, congelamento, descida Adam nas variáveis de projeto e cartão de verificação que reconfere contra a verdade de forma fechada.

`pinneapple_design/surrogate_inverse/core.py` · 267 linhas · testes: 16

nome	tipo	assinatura	o que faz
`MLPSurrogate`	classe	`(bounds, hidden=48, n_layers=3, log_output=False)`	MLP `tanh` com entradas mapeadas dos limites para [-1, 1] e saída padronizada (opcionalmente em log).
`SurrogateFit`	classe	`(model, val_mean_rel_err, val_p95_rel_err, val_max_rel_err, n_train, n_val, epochs, secon…)`	Surrogate treinado com seus erros relativos em linhas separadas.
`CheckResult`	classe	`(name, verdict, detail, value, limit)`	Resultado de uma checagem: nome, veredito (PASS, WARNING, FAIL), detalhe, valor e limite.
`VerificationCard`	classe	`(checks, values, covered, not_covered)`	Design final reconferido contra a verdade de forma fechada, com veredito geral, cobertura explícita e `TrustReport`.
`fit_surrogate`	função	`(X, y, bounds, *, hidden=48, n_layers=3, epochs=1500, lr=0.003, seed=0, log_output=False,…)`	Treina a MLP com Adam em lote cheio e decaimento cosseno, separando as últimas linhas para validação.
`freeze`	função	`(model) -> nn.Module`	Congela os pesos do surrogate para que só o design se mova.
`descend`	função	`(loss_fn, x0, *, bounds, steps=400, lr=0.01, reparam='clamp', project=None) -> dict[str, …)`	Adam nas variáveis de projeto para minimizar uma perda, com reparametrização por clamp ou sigmoide.
`banded`	função	`(name, error, pass_limit, warn_limit, *, pass_text, warn_text, fail_text) -> CheckResult`	Veredito por faixas: PASS até `pass_limit`, WARNING até `warn_limit`, FAIL acima.
`relative_error`	função	`(predicted, reference, floor) -> float`	Erro relativo |previsto - referência| / max(|referência|, piso).
`constraint_checks`	função	`(violations, ok_text=…, skip=()) -> list[CheckResult]`	Uma checagem por violação com sinal (<= 0 satisfeita).

pinneapple_design.surrogate_inverse.heatsink_fins

Caso do dissipador de CPU com aletas retas em base fixa de alumínio: fórmula 1-D de arranjo de aletas como verdade, surrogate na resistência térmica e checagem por volumes finitos 2-D numa célula de aleta.

`pinneapple_design/surrogate_inverse/heatsink_fins.py` · 264 linhas · testes: 91

nome	tipo	assinatura	o que faz
`HeatsinkDesign`	classe	`(fin_count, fin_height_m, fin_thickness_m)`	Design do dissipador: número, altura e espessura das aletas, com espaçamento derivado.
`OptimizationResult`	classe	`(design, surrogate_predicted_temp_c, steps_run)`	Resultado da otimização do dissipador: design, temperatura prevista e passos.

nome	tipo	assinatura	o que faz
`profile_params`	função	`(design, power_w, h_conv) -> dict[str, float]`	Grandezas intermediárias da fórmula de aletas (m, comprimento corrigido, resistências, elevação na raiz).
`junction_temperature_c`	função	`(design, power_w, h_conv) -> float`	Verdade: temperatura da base (junção), usada nos rótulos e na verificação.
`thermal_resistance_k_w`	função	`(design, h_conv) -> float`	Resistência térmica (T_j - T_amb) / P, independente da potência neste modelo.
`fin_temperature_at_height_c`	função	`(design, power_w, h_conv, z_above_root_m) -> float`	Perfil 1-D de temperatura ao longo da altura da aleta.
`thermal_field`	função	`(design, power_w, h_conv) -> dict[str, float]`	O que um visualizador precisa para colorir as aletas pelo perfil.
`constraint_violations`	função	`(design) -> dict[str, float]`	Violações com sinal das regras de projeto (<= 0 satisfeita).
`design_to_dict`	função	`(design) -> dict`	Dicionário do design com contagem arredondada e espaçamento.
`generate_dataset`	função	`(n_samples, seed=0) -> tuple[torch.Tensor, torch.Tensor]`	Designs uniformes rotulados com a resistência térmica da fórmula.
`train_surrogate`	função	`(n_samples=6000, epochs=1500, seed=0) -> SurrogateFit`	Treina a MLP em log R.
`predict_temperature_c`	função	`(surrogate, design, power_w, h_conv) -> float`	Temperatura de junção prevista pelo surrogate.
`optimize_design`	função	`(surrogate, power_w, h_conv, steps=400, lr=0.05, penalty_weight=100.0) -> OptimizationRes…)`	Minimiza a temperatura prevista sobre (N, H, t) a partir da base.
`verify_design`	função	`(design, power_w, h_conv, surrogate_predicted_temp_c) -> VerificationCard`	Reroda a fórmula no design final: erro do surrogate, regras de projeto e limite de plausibilidade.
`fin_cell_check`	função	`(design, power_w, h_conv) -> dict`	Compara a fórmula com condução por volumes finitos 2-D num passo de aleta.
`mesh`	função	`(design) -> meshes.Mesh`	Malha da placa base com as aletas, em milímetros.
`export_stl_bytes`	função	`(design) -> bytes`	Bytes STL do dissipador.

`pinneapple_design.surrogate_inverse.meshes`

Malhas triangulares das cinco peças em milímetros (para visualizador ou STL), como corpos fechados concatenados, sem união booleana.

`pinneapple_design/surrogate_inverse/meshes.py` · 93 linhas · testes: 88

nome	tipo	assinatura	o que faz
`merge`	função	`(parts) -> Mesh`	Concatena malhas reindexando as faces.
`bbox_size`	função	`(mesh) -> tuple[float, float, float]`	Tamanho da caixa envolvente (x, y, z) da malha.
`to_stl`	função	`(mesh, name) -> bytes`	STL binário com cabeçalho iniciado pelo nome da peça.
`fin_array`	função	`(base_w_mm, base_d_mm, base_t_mm, n_fins, fin_h_mm, fin_t_mm) -> Mesh`	Placa base com aletas retas distribuídas na largura.
`vented_rotor`	função	`(r_mean_mm, radial_len_mm, ring_t_mm, gap_mm, n_vanes, vane_t_mm, n=96) -> Mesh`	Dois anéis de atrito unidos por aletas radiais no vão de ventilação.
`layered_disc`	função	`(radius_mm, layer_thicknesses_mm, n=96) -> Mesh`	Discos coaxiais empilhados (ex.: reservatório sob membrana).
`rectangular_beam`	função	`(width_mm, thickness_mm, length_mm) -> Mesh`	Caixa simples representando a viga.
`flanged_tube`	função	`(radius_mm, length_mm, flange_radius_mm, flange_t_mm, n=64) -> Mesh`	Duto cilíndrico com disco de flange na base.

`pinneapple_design.surrogate_inverse.reflex_port`

Caso do duto bass-reflex: raio e comprimento que sintonizam a caixa numa frequência (ressonador de Helmholtz), com varredura do trade-off volume x precisão.

pinneapple_design/surrogate_inverse/reflex_port.py · 232 linhas · testes: 88

nome	tipo	assinatura	o que faz
PortDesign	classe	(radius_m, length_m)	Design do duto: raio e comprimento.
OptimizationResult	classe	(design, surrogate_predicted_freq_hz)	Resultado da otimização do duto.
FrontierPoint	classe	(volume_weight, design, surrogate_predicted_freq_hz, real_achieved_freq_hz, real_freq_err…)	Um ponto da varredura volume x precisão.
resonant_frequency_hz	função	(design, cabinet_volume_l) -> float	Verdade: frequência de ressonância de Helmholtz.
acoustic_field	função	(design, cabinet_volume_l) -> dict[str, float]	Um valor por design (o modelo concentrado não tem campo): a sintonia dentro da faixa alvo.
constraint_violations	função	(design, cabinet_volume_l) -> dict[str, float]	Violações com sinal das regras do duto.
design_to_dict	função	(design) -> dict	Dicionário do design do duto.
generate_dataset	função	(n_samples, seed=0) -> tuple[torch.Tensor, torch.Tensor]	Designs aleatórios rotulados com a frequência de Helmholtz.
train_surrogate	função	(n_samples=6000, epochs=1500, seed=0) -> SurrogateFit	Treina a MLP que prevê a frequência.
predict_frequency_hz	função	(surrogate, design, cabinet_volume_l) -> float	Frequência prevista pelo surrogate.
optimize_design	função	(surrogate, cabinet_volume_l, target_freq_hz, steps=400, lr=0.05, penalty_weight=100.0) -…)	Minimiza o erro relativo de frequência ao quadrado mais penalidade de volume do duto.
sweep_pareto_frontier	função	(surrogate, cabinet_volume_l, target_freq_hz, n_points=8, steps=400, lr=0.05, penalty_wei…)	Varre o peso lambda e monta o trade-off volume x precisão.
verify_design	função	(design, cabinet_volume_l, target_freq_hz, surrogate_predicted_freq_hz) -> VerificationCa…)	Reconfere o design com a fórmula de Helmholtz e as regras.
mesh	função	(design) -> meshes.Mesh	Malha do duto com flange, em milímetros.
export_stl_bytes	função	(design) -> bytes	Bytes STL do duto.

pinneapple_design.surrogate_inverse.suspension_arm

Caso do braço de suspensão como viga em balanço retangular: menor largura x espessura que atinge uma vida à fadiga (lei de Basquin), com sensibilidades locais.

pinneapple_design/surrogate_inverse/suspension_arm.py · 255 linhas · testes: 88

nome	tipo	assinatura	o que faz
ArmDesign	classe	(width_m, thickness_m)	Design do braço: largura e espessura da seção retangular.
OptimizationResult	classe	(design, surrogate_predicted_stress_pa)	Resultado da otimização do braço.
SensitivityResult	classe	(width_m, thickness_m, load_n, life_cycles, mass_kg, d_life_d_width, d_life_d_thickness, …)	Sensibilidades locais (elasticidades) de vida e massa em relação a largura e espessura.
bending_stress_pa	função	(design, load_n) -> float	Verdade: tensão de flexão na raiz engastada.
bending_stress_at_position_pa	função	(design, load_n, x_from_root_m) -> float	Tensão de flexão ao longo do braço, máxima na raiz e nula na ponta.
fatigue_life_cycles	função	(design, load_n) -> float	Vida à fadiga em ciclos pela lei de Basquin.
allowable_stress_for_life_pa	função	(target_life_cycles) -> float	Tensão admissível para atingir a vida alvo.
stress_field	função	(design, load_n) -> dict[str, float]	Dados para colorir o braço pela tensão ao longo do comprimento.
constraint_violations	função	(design, load_n, target_life_cycles) -> dict[str, float]	Violações com sinal das regras do braço.

nome	tipo	assinatura	o que faz
design_to_dict	função	(design) -> dict	Dicionário do design do braço.
generate_dataset	função	(n_samples, seed=0) -> tuple[torch.Tensor, torch.Tensor]	Designs aleatórios rotulados com a tensão de flexão.
train_surrogate	função	(n_samples=6000, epochs=1500, seed=0) -> SurrogateFit	Treina a MLP em log da tensão.
predict_stress_pa	função	(surrogate, design, load_n) -> float	Tensão prevista pelo surrogate.
optimize_design	função	(surrogate, load_n, target_life_cycles, steps=500, lr=0.0006, penalty_weight=5e-11) -> Op…)	Minimiza w t mais penalidade quando a tensão prevista passa da admissível.
verify_design	função	(design, load_n, target_life_cycles, surrogate_predicted_stress_pa) -> VerificationCard	Reconfere o design com a forma fechada e as regras.
compute_sensitivity	função	(width_m, thickness_m, load_n, step_m=H_M) -> SensitivityResult	Derivadas de vida e massa por diferenças centrais na forma fechada (nunca no surrogate).
sensitivity_to_dict	função	(r) -> dict	Dicionário das sensibilidades com um resumo em texto.
mesh	função	(design) -> meshes.Mesh	Malha do braço como caixa simples em milímetros (sem filetes nem furos).
export_stl_bytes	função	(design) -> bytes	Bytes STL do braço.

pinneapple_design.surrogate_inverse.transdermal_patch

Caso do adesivo de liberação controlada por membrana (lei de Fick): área, espessura da membrana e concentração para uma taxa alvo e tempo de uso, com a menor área.

pinneapple_design/surrogate_inverse/transdermal_patch.py · 249 linhas · testes: 88

nome	tipo	assinatura	o que faz
PatchDesign	classe	(area_m2, membrane_thickness_m, reservoir_conc_kg_m3)	Design do adesivo: área, espessura da membrana e concentração do reservatório.
OptimizationResult	classe	(design, surrogate_predicted_rate_mg_day)	Resultado da otimização do adesivo.
release_rate_mg_day	função	(design) -> float	Verdade: taxa de liberação estacionária.
depletion_time_days	função	(design) -> float	Tempo até esgotar o reservatório.
reservoir_mass_mg	função	(design) -> float	Massa de fármaco no reservatório.
concentration_at_membrane_depth	função	(design, z_from_reservoir_m) -> float	Perfil linear estacionário de concentração na membrana.
decay_rate_constant_per_day	função	(design) -> float	Constante de primeira ordem da cauda após o esgotamento.
simulate_release_timeseries	função	(design, duration_days=None, n_points=100) -> list[dict[str, float]]	Série temporal de taxa, liberação acumulada e reservatório restante.
concentration_field	função	(design) -> dict[str, float]	Dados para colorir a membrana pelo perfil de concentração.
constraint_violations	função	(design, target_wear_days) -> dict[str, float]	Violações com sinal (taxa, tempo de uso, limites).
design_to_dict	função	(design) -> dict	Dicionário do design do adesivo.
generate_dataset	função	(n_samples, seed=0) -> tuple[torch.Tensor, torch.Tensor]	Designs aleatórios rotulados com a taxa de liberação.
train_surrogate	função	(n_samples=6000, epochs=1500, seed=0) -> SurrogateFit	Treina a MLP em log da taxa.
predict_rate_mg_day	função	(surrogate, design) -> float	Taxa prevista pelo surrogate.
optimize_design	função	(surrogate, target_rate_mg_day, target_wear_days, steps=1200, lr=0.15, weight_rate=80.0, …)	Minimiza área relativa mais erros de taxa e de tempo de uso num espaço limitado por sigmoide.

nome	tipo	assinatura	o que faz
`verify_design`	função	`(design, target_rate_mg_day, target_wear_days, surrogate_predicted_rate_mg_day) -> Verifi…)`	Reconfere o design com a lei de Fick e as regras.
`mesh`	função	`(design) -> meshes.Mesh`	Malha do disco do reservatório com a membrana, em milímetros.
`export_stl_bytes`	função	`(design) -> bytes`	Bytes STL do adesivo.

`pinneapple_design.thermal`

Subpacote de projeto térmico: dimensionamento de dissipadores (física + triagem por surrogate) e o estudo de resfriamento de placa.

`pinneapple_design/thermal/__init__.py` · 2 linhas · testes: 3

`pinneapple_design.thermal.board_cooling`

'Engenheiro que se auto-otimiza': resfria uma peça superaquecida numa placa com dissipador e ventiladores; reexporta `run_mission` e as etapas do estudo.

`pinneapple_design/thermal/board_cooling/__init__.py` · 29 linhas · testes: 10

`pinneapple_design.thermal.board_cooling.agent`

Agente do estudo de resfriamento: pedido em linguagem natural, plano (modelo fixo ou LLM via `pinneapple_autolab`), ferramentas controladas e evidências. [sem teste direto]

`pinneapple_design/thermal/board_cooling/agent.py` · 197 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`t_read_mission`	função	`(state, text) -> dict[str, Any]`	Ferramenta que lê o pedido de resfriamento e guarda a missão no estado.
`t_baseline`	função	`(state) -> dict[str, Any]`	Ferramenta que resolve a placa nua em ar parado como ponto de partida.
`t_screen_natural`	função	`(state, gaps_mm=(4.0, 6.0, 8.0, 11.0)) -> dict[str, Any]`	Testa em ar parado o maior dissipador que cabe, para alguns espaçamentos de aleta; se nem o melhor atende, conclui que precisa de ar forçado.
`t_sample`	função	`(state, n=64, seed=0, workers=4) -> dict[str, Any]`	Ferramenta que roda soluções de referência em projetos amostrados.
`t_learn`	função	`(state, test_fraction=0.2, seed=0, max_error_k=3.0, min_coverage=0.9, add=32, max_n=320, …)`	Ajusta o surrogate e mede numa amostra separada; enquanto erro ou cobertura não atendem, adiciona soluções de referência até um limite.
`t_optimize`	função	`(state, rounds=5, k_sigma=2.0, seed=0) -> dict[str, Any]`	Ferramenta que roda o laço aprender-otimizar-verificar.
`t_verify`	função	`(state) -> dict[str, Any]`	Ferramenta que verifica o projeto escolhido numa malha fina.
`t_deliver`	função	`(state, out_dir='out', render=True, samples=24) -> dict[str, Any]`	Ferramenta que gera figuras, cena 3D, renders e relatório.
`toolbox`	função	`() -> Toolbox`	Monta o `Toolbox` com as ferramentas acima.
`template_plan`	função	`(text, *, n=64, out_dir='out', render=True, workers=4) -> TemplatePlanner`	Plano determinístico (sem LLM) com a sequência padrão de etapas.
`run_mission`	função	`(text, *, out_dir='out', planner='template', n=64, render=True, workers=4, llm=None, prov…)`	Planeja e roda o estudo; com `planner='llm'` cai para o plano fixo se o modelo não responder ou o plano for inválido.

`pinneapple_design.thermal.board_cooling.learn`

Aproximação rápida da referência acoplada: temperaturas de junção e mapa de temperatura da placa (POD + processo gaussiano), com avaliação em amostra separada. [sem teste direto]

pinneapple_design/thermal/board_cooling/learn.py · 107 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
CoolingSurrogate	classe	(t_air_c, modes, tj_, map_, fit_seconds, n_train)	Surrogate das temperaturas de junção e do mapa da placa, com ajuste, previsão e posto do mapa.
errors	função	(pred_tj, pred_map, Y, maps) -> dict[str, Any]	Calcula os erros de previsão de temperaturas e mapas contra a referência.
holdout	função	(data, t_air_c, *, test_fraction=0.2, seed=0, modes=16) -> dict[str, Any]	Treina numa parte das soluções e mede no resto: erros, cobertura da faixa de ±2 desvios e tempo de previsão.

pinneapple_design.thermal.board_cooling.mission

A missão de resfriamento: placa, peça quente, limites, espaço disponível, orçamento de ventilação e espaço de projeto; inclui leitura determinística do pedido. [sem teste direto]

pinneapple_design/thermal/board_cooling/mission.py · 212 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
CoolingDesign	classe	(base_width_mm, base_depth_mm, base_thickness_mm, n_fins, fin_thickness_mm, height_mm, ai…)	Um projeto: dissipador sobre a peça quente mais a velocidade do ar dos ventiladores (0 = convecção natural).
DesignSpace	classe	(lo, hi, min_gap_mm, min_fin_height_mm)	Limites das variáveis de projeto e regras geométricas (espaçamento e altura de aleta).
CoolingMission	classe	(board, components, hot, t_ambient_c, tj_limit_c, max_width_mm, max_depth_mm, max_height_…)	Placa, componentes, peça quente, temperaturas, limites de espaço e de potência de ventilação.
default_mission	função	(power_w=40.0) -> CoolingMission	Missão padrão: placa controladora com um SoC de potência power_w, reguladores e memória.
parse_mission	função	(text) -> CoolingMission	Lê um pedido em português ou inglês; os números citados substituem os padrões e provenance diz de onde veio cada campo.

pinneapple_design.thermal.board_cooling.optimize

Busca com restrições no surrogate, verificação na referência e o laço entre eles. [sem teste direto]

pinneapple_design/thermal/board_cooling/optimize.py · 141 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
constraint_violation	função	(mission, X, tj_upper, fan_w) -> np.ndarray	Soma das violações de restrição (temperatura, ventilação, espaçamento), cada uma escalada.
optimize	função	(mission, sur, *, k_sigma=2.0, seed=0, maxiter=120, popsize=24) -> dict[str, Any]	Acha o projeto de menor massa no surrogate por evolução diferencial, com margem pessimista k_sigma.
design_loop	função	(mission, data, *, rounds=5, k_sigma=2.0, seed=0, log=None) -> dict[str, Any]	Laço aprender → otimizar → verificar na referência → adicionar o ponto, até confirmar.

pinneapple_design.thermal.board_cooling.report

Entregáveis do estudo de resfriamento: figuras, cena e visualizador 3D, renders no Blender (opcional), relatório e relatório de confiança. [sem teste direto]

pinneapple_design/thermal/board_cooling/report.py · 377 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
figure_maps	função	(state, path) -> str	Figura com os mapas de temperatura da placa.
figure_evolution	função	(state, path) -> str	Figura da temperatura máxima ao longo do estudo: base, ar parado, otimizador e ótimos verificados.
figure_holdout	função	(state, path) -> str	Figura do surrogate contra a referência na amostra separada e do erro do mapa no projeto verificado.
impact_gif	função	(state, path, hold=6) -> str	Animação 2D do mapa da placa da placa nua ao projeto verificado.

nome	tipo	assinatura	o que faz
render_3d	função	(state, out_dir, *, render=True, samples=24, size=(1280, 720)) -> dict[str, Any]	glTF/USD do projeto, visualizador da linha do tempo e, com Blender, imagens e animação.
write_report	função	(state, out_dir, files, extra) -> str	Escreve o relatório do estudo.
deliver	função	(state, out_dir, *, render=True, samples=24) -> dict[str, Any]	Gera todos os entregáveis numa pasta.

pinneapple_design.thermal.board_cooling.scene

Cena 3D da placa resfriada: gabinete, ventiladores, mapa de temperatura, peças, dissipador e ar aquecido. [sem teste direto]

pinneapple_design/thermal/board_cooling/scene.py · 323 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
fan_positions	função	(mission) -> tuple[list[tuple[float, float, float]], float]	Centros e raio dos três ventiladores na parede de entrada.
heat_sink_mesh	função	(mission, design, n_max, field3d=None)	Vértices, faces e temperatura por vértice do dissipador.
air_paths	função	(mission, result, n_x=18, heights=(4.0, 12.0, 22.0))	Caminhos do ar e temperatura média de cada faixa por balanço de energia.
board_scene	função	(mission, result, *, title='', paths=True)	Um estado como cena estática com o campo de temperatura.
timeline_scene	função	(mission, states, *, title='', t_range=T_RANGE)	Os estados como quadros de uma animação, com painel de temperatura, massa e potência de ventilação.

pinneapple_design.thermal.board_cooling.simulate

Simulação de referência da placa resfriada: modelo 3D da placa acoplado ao modelo do dissipador.

pinneapple_design/thermal/board_cooling/simulate.py · 183 linhas · testes: 10

nome	tipo	assinatura	o que faz
map_axes	função	(board) -> tuple[np.ndarray, np.ndarray]	Centros (y, x) das células do mapa em mm.
sink_mass_and_air	função	(mission, d) -> dict[str, float]	Massa, perda de carga e potência do ar do dissipador por fórmula fechada.
simulate	função	(mission, design, *, fidelity='train', pessimistic=True, tol=0.0001, max_iter=12) -> dict…)	Temperaturas de junção, mapa da placa e checagens de um projeto (None = placa nua em ar parado).
response	função	(mission, result) -> np.ndarray	Vetor de respostas que o surrogate aprende: temperaturas de junção nominais e pessimistas.
simulate_many	função	(mission, X, *, fidelity='train', workers=4) -> dict[str, np.ndarray]	Soluções de referência de vários projetos em paralelo.

pinneapple_design.thermal.board_cooling.verify

Verificação independente do projeto escolhido numa malha fina e o relatório de confiança.

pinneapple_design/thermal/board_cooling/verify.py · 109 linhas · testes: 10

nome	tipo	assinatura	o que faz
verify	função	(mission, design, *, coarse=None, prediction=None, holdout=None, train_X=None, test_X=Non…)	Resolve o projeto na malha fina e monta o relatório de confiança com previsão, amostra separada e cobertura.

pinneapple_design.thermal.fin_array_speed

Mede quanto um surrogate treinado é mais rápido que uma solução numérica e quão longe estão, numa varredura de dissipadores de aletas curados.

pinneapple_design/thermal/fin_array_speed.py · 160 linhas · testes: 3

nome	tipo	assinatura	o que faz
design_by_name	função	(name) -> dict[str, Any]	Busca um dos projetos de CURATED_DESIGNS (KeyError se não existir).
design_of	função	(spec) -> hf.HeatsinkDesign	Converte uma especificação em HeatsinkDesign.
train_surrogate	função	(n_samples=6000, epochs=1500, seed=0)	Treina o surrogate de heatsink_fins e devolve o modelo e o erro relativo de validação.
time_surrogate	função	(model, d, power_w, h_conv, reps=200) -> dict[str, float]	Tempo médio de uma passada do surrogate e a temperatura prevista.
reference_solve	função	(d, power_w, h_conv, refine=True) -> dict[str, Any]	Solução de volumes finitos de um passo de aleta (fin_cell), cronometrada.
measure_design	função	(model, spec, timing_reps=200, refine=True) -> dict[str, Any]	Fórmula fechada, surrogate e referência cronometrados para um projeto.
summarize	função	(designs) -> dict[str, Any]	Resume as medições de todos os projetos.
run_sweep	função	(model, progress_cb=None, timing_reps=200, refine=True) -> dict[str, Any]	Mede todos os projetos de CURATED_DESIGNS.
thermal_field	função	(name='baseline') -> dict[str, Any]	Campo térmico de um projeto mais a temperatura da base.

pinneapple_design.thermal.fin_cell

Condução em volumes finitos num passo de aleta de um dissipador de aletas retas: checagem independente da fórmula 1D de pinneapple_physics.closed_form.fin_array_conduction.

pinneapple_design/thermal/fin_cell.py · 147 linhas · testes: 16

nome	tipo	assinatura	o que faz
FinCellResult	classe	(t_junction_c, t_fin_root_c, t_fin_tip_c, energy_balance_rel_error, n_cells, grid_change_…)	Resultado: temperaturas de junção, raiz e ponta da aleta, erro de balanço de energia e mudança com a malha.
solve_fin_cell	função	(*, n_fins, fin_thickness_m, fin_height_m, fin_spacing_m, depth_m, base_thickness_m, k_w_…)	Resolve a condução permanente num passo de aleta (power_w é a potência do conjunto todo).

pinneapple_design.thermal.heatsink

Dimensionamento de dissipadores de aletas planas (motor do app HeatSink Sizer); reexporta o motor, o dimensionador e os surrogates.

pinneapple_design/thermal/heatsink/__init__.py · 18 linhas · **sem teste direto**

pinneapple_design.thermal.heatsink.engine

Motor físico do HeatSink Sizer: avalia um projeto com a rede de fórmulas fechadas e a base resolvida em volumes finitos 3D, com faixa pessimista/otimista de convecção.

pinneapple_design/thermal/heatsink/engine.py · 318 linhas · testes: 46

nome	tipo	assinatura	o que faz
DesignInput	classe	(base_width_mm, base_depth_mm, base_thickness_mm, n_fins, fin_thickness_mm, fin_height_mm…)	Entrada de geometria do dissipador (base, aletas, material).
OperatingInput	classe	(power_w, t_ambient_c, air_velocity_m_s, source_width_mm, source_depth_mm, source_x_mm, s…)	Condições de operação (potência, ambiente, velocidade do ar, fonte, TIM, limite).
base_theta	função	(g, op, r_conv, n=40) -> Dict[str, Any]	Campo de temperatura da base para uma resistência de convecção.
physics_resistance	função	(design, op, n=36) -> Dict[str, float]	Resistência base-ambiente (sem TIM) pela física resolvida, mais perda de carga: o alvo do surrogate.
field_3d	função	(g, op, fvm, nominal, t_amb, max_cells=64, n_z=14) -> Dict[str, Any]	Temperaturas na superfície do dissipador inteiro para o visualizador 3D.

nome	tipo	assinatura	o que faz
model_scope	função	(mode) -> Dict[str, Any]	Descreve o que o modelo cobre e para que lado erra cada simplificação; vai em todo relatório.
evaluate_design	função	(design, op, *, grid=48, map_size=40) -> Dict[str, Any]	Avalia um projeto completo (nominal, pessimista e otimista) e devolve resultado e mapas.

pinneapple_design.thermal.heatsink.field

Solução 3D em volumes finitos da condução na placa-base do dissipador.

pinneapple_design/thermal/heatsink/field.py · 111 linhas · testes: 18

nome	tipo	assinatura	o que faz
solve_base_field	função	(*, base_width, base_depth, base_thickness, k, power_w, r_fins_k_w, source_width, source_…)	Resolve o campo de temperatura da base com a fonte de calor e a resistência das aletas.

pinneapple_design.thermal.heatsink.monotone

Treina o surrogate de dissipador com restrições de monotonicidade (modelo do zoo `heatsink_natural_mlp`). [sem teste direto]

pinneapple_design/thermal/heatsink/monotone.py · 163 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
random_designs	função	(mode, n, rng) -> np.ndarray	Linhas de atributos de projetos aleatórios no intervalo, amostradas log-uniformemente.
split_indices	função	(n) -> dict[str, np.ndarray]	Divisão do app em teste, calibração, validação e treino.
train_monotone	função	(mode='natural', *, seed=0, epochs=600, lbfgs_rounds=4, lbfgs_steps=200, n_random=20000, …)	Treina o surrogate com penalidade de monotonicidade e ajuste L-BFGS e devolve o pacote a salvar.

pinneapple_design.thermal.heatsink.sizer

Dimensionamento: requisitos entram, projetos verificados saem; triagem por surrogate (limite superior conformal) ou por fórmula fechada, depois verificação física.

pinneapple_design/thermal/heatsink/sizer.py · 251 linhas · testes: 18

nome	tipo	assinatura	o que faz
SizingRequest	classe	(power_w, t_limit_c, t_ambient_c, air_velocity_m_s, source_width_mm, source_depth_mm, tim…)	Pedido de dimensionamento: potência, limites, ar, fonte, envelope, materiais e processo.
size	função	(req, surrogates) -> Dict[str, Any]	Amostra candidatos, faz a triagem, arredonda e verifica os melhores com o motor físico.

pinneapple_design.thermal.heatsink.surrogate

Surrogate neural (MLP) do motor físico, usado só para triagem; um modelo por modo (forçado/natural), treinado e salvo em `ARTIFACTS`. [sem teste direto]

pinneapple_design/thermal/heatsink/surrogate.py · 334 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
Surrogate	classe	(mode, model, x_mean, x_std, y_mean, y_std, conformal_log_q, report)	Modelo treinado com normalização e quantis conformais; prevê, salva e carrega.
n_fins_for	função	(width_mm, gap_mm, t_mm)	Número de aletas que cabem numa largura com espaçamento e espessura dados.
features	função	(mode, W, D, tb, N, t, H, k, sw, sd, Ta, Q, V=None) -> np.ndarray	Mapa vetorizado de atributos do surrogate.
reference_solve	função	(params, mode='forced') -> dict | None	Roda o motor físico num projeto dado nas colunas do conjunto de dados.
reference_solve_batch	função	(X, names, output='r_base_k_w', mode='forced') -> np.ndarray	`reference_solve` em cada linha; NaN onde o motor rejeita o projeto.

nome	tipo	assinatura	o que faz
`generate_dataset`	função	`(mode, n, seed=0, workers=4) -> List[Dict[str, float]]`	Gera o conjunto de dados de treino com o motor físico em paralelo.
`rows_to_xy`	função	`(mode, rows) -> Tuple[np.ndarray, np.ndarray]`	Converte linhas do conjunto em matrizes X, Y.
`train`	função	`(mode, rows, seed=0, epochs=400) -> Surrogate`	Treina o surrogate de um modo.
`load_surrogates`	função	`() -> Dict[str, Surrogate]`	Carrega os surrogates salvos que existirem em `ARTIFACTS`.
`main`	função	`(n_forced=6000, n_natural=5000, workers=4, reuse=False) -> None`	Gera os dados, treina e grava os surrogates e o relatório de confiança.

`pinneapple_design.thermal.heatsink.trust`

Relatório de confiança dos surrogates de dissipador (erros, convergência, cobertura). [sem teste direto]

`pinneapple_design/thermal/heatsink/trust.py` · 202 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`surrogate_report`	função	`(sur, *, X, Y, split, history, train_seconds, n_rows) -> Dict`	Monta o relatório de confiança de um surrogate a partir dos dados e do histórico.

`pinneapple_design.topology`

Reexporta problemas, filtros, otimizadores e o driver de otimização topológica.

`pinneapple_design/topology/__init__.py` · 19 linhas · testes: 9

`pinneapple_design.topology.driver`

Laço de otimização: variáveis → simetria → filtro de densidade → projeção de Heaviside → problema, com adjunto, continuação de beta e checagem por diferenças finitas. [sem teste direto pelo inventário]

`pinneapple_design/topology/driver.py` · 127 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`DesignChain`	classe	`(n_el, filt=None, symmetry=None, beta=0.0, eta=0.5)`	Cadeia `x_phys = H_beta(W (S x))` e seu adjunto.
`TopOptResult`	classe	`(x, x_phys, objective, history)`	Resultado: densidades de projeto e físicas, objetivo e histórico.
`optimize`	função	`(problem, volfrac, filt=None, optimizer='oc', max_iter=200, tol=0.01, x0=None, symmetry=N…)`	Roda a otimização topológica com fração de volume, filtro, OC ou MMA, simetria e continuação de beta.
`finite_difference_check`	função	`(fun, x, idx, h=1e-06) -> np.ndarray`	Erro relativo do gradiente adjunto contra diferenças centrais em entradas escolhidas.

`pinneapple_design.topology.filters`

Filtro linear de densidade, mapa de simetria e projeção de Heaviside com derivadas. [sem teste direto pelo inventário]

`pinneapple_design/topology/filters.py` · 115 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`DensityFilter`	classe	`(centroids, r_min, volumes=None)`	Filtro linear de densidade com raio `r_min`, com construtor para grades e adjunto.
`SymmetryMap`	classe	`(S)`	Mapa 0/1 de poucas variáveis de projeto para densidades de elemento (identidade, rotacional).
`heaviside`	função	`(x, beta, eta=0.5) -> np.ndarray`	Projeção de Heaviside suave com nitidez beta e limiar eta.
`heaviside_derivative`	função	`(x, beta, eta=0.5) -> np.ndarray`	Derivada da projeção de Heaviside.

`pinneapple_design.topology.magnetic`

Otimiza o ferro do rotor para torque no modelo 2D A_z de `pinneapple_simulation.electromagnetics`, com sensibilidade por adjunto em cada ângulo.

`pinneapple_design/topology/magnetic.py` · 79 linhas · testes: 9

nome	tipo	assinatura	o que faz
`RotorTorqueProblem`	classe	`(machine, thetas, I_peak, gamma, objective='mean', T_ref=1.0)`	Problema de ferro do rotor: torque médio ou ripple sobre ângulos, com restrição de torque médio.

`pinneapple_design.topology.optimizers`

Regras de atualização: critério de otimalidade (OC) e método das assíntotas móveis (MMA). [sem teste direto pelo inventário]

`pinneapple_design/topology/optimizers.py` · 117 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`MMA`	classe	`(n, m, xmin=0.0, xmax=1.0, move=0.5, asyinit=0.5, asyincr=1.2, asydecr=0.7, c=1000.0, d=1…)`	Método das assíntotas móveis para problemas com restrições de desigualdade e limites.
`oc_update`	função	`(x, dfdx, dvdx, volume_target, volume_of, move=0.2, eta=0.5, xmin=0.0, xmax=1.0)`	Um passo do critério de otimalidade com bisseção no multiplicador de volume.

`pinneapple_design.topology.problems`

Problemas em grades de elementos Q4: compliance estrutural (MBB, viga engastada), compliance térmica e dissipação de Stokes com penalização de Brinkman. [sem teste direto pelo inventário]

`pinneapple_design/topology/problems.py` · 264 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Q4Grid`	classe	`(nelx, nely, h=1.0)`	Grade estruturada de elementos Q4 quadrados com numeração e montagem.
`Compliance2D`	classe	`(nelx, nely, fixed_dofs, loads, penal=3.0, E0=1.0, Emin=1e-09, nu=0.3)`	Compliance estrutural em estado plano de tensão com SIMP; construtores `mbb` e `cantilever`.
`ThermalCompliance2D`	classe	`(nelx=60, nely=60, sink_fraction=0.1, q=1.0, penal=3.0, kmin=0.001)`	Placa com fonte de calor uniforme e sumidouro numa parte da borda esquerda; minimiza a compliance térmica.
`StokesDissipation2D`	classe	`(nelx, nely, inlets, outlets, mu=1.0, alpha_max=None, alpha_min=None, q=0.1, penalty=1000…)`	Escoamento de Stokes com penalização de Brinkman minimizando a dissipação.
`elasticity_ke`	função	`(nu=0.3, E=1.0, h=1.0, thickness=1.0) -> np.ndarray`	Rigidez 8×8 de um elemento Q4 em estado plano de tensão.
`conduction_ke`	função	`(h=1.0) -> np.ndarray`	Matriz de condução 4×4 de um elemento Q4.

Exemplos de uso reais do repositório

Vortex lattice do avião comercial da biblioteca de geometrias — `examples/geometry_library/03_airliner_vlm.py`

```
g = pp.geometries.build("airliner_a320")
p = g.params
ac = Airliner(length=p["length"], wing_area=p["wing_area"], aspect_ratio=p["aspect_ratio"], taper=p["taper"],
              sweep_le=p["sweep_le"], dihedral=p["dihedral"], twist=p["twist"], fan_diameter=p["fan_diameter"],
              engine_station=p["engine_station"], sharklets=p["sharklets"])
x_ref = ac.mac_le_x + 0.25 * ac.mac
sol = solve(ac, x_ref)
print(g)
```

Preset do dissipador no estudo de projeto (forma fechada) — `tests/test_design_studies.py`

```
        def test_heatsink_geometry_matches_library_closed_forms():
            from pinneapple_design.thermal.heatsink.engine import DesignInput
            from pinneapple_physics.closed_form.cantilever_fatigue import stress_from_moment_pa

            p = presets.heatsink_problem("closed_form")
            b = presets.heatsink_baseline()
            ev = p.evaluate(p.space.from_dicts([b]))
            d = DesignInput(**b)
            assert ev.Y["mass_g"][0] == pytest.approx(d.geometry().mass_kg() * 1e3, rel=1e-12)
```

Surrogate do surrogate_inverse com erro em dados separados — `tests/pinneapple_design/test_surrogate_inverse.py`

```
        def test_fit_surrogate_learns_a_power_law_and_reports_held_out_error():
            rng = np.random.default_rng(0)
            X = torch.tensor(rng.uniform([1.0, 1.0], [10.0, 5.0], size=(2000, 2)), dtype=torch.float32)
            y = X[:, 0] ** 2 / X[:, 1]
            fit = core.fit_surrogate(X, y, [(1.0, 10.0), (1.0, 5.0)], epochs=1000, log_output=True, hidden=32,
 n_layers=2)
            assert fit.n_val == 200 and fit.n_train == 1800
            assert fit.val_mean_rel_err < 0.03 and fit.val_p95_rel_err >= fit.val_mean_rel_err
            with torch.no_grad():
                assert float(fit.model(torch.tensor([[4.0, 2.0]]))[0]) == pytest.approx(8.0, rel=0.05)
```

Montar e amostrar um domínio — `docs/core_concepts/geometry_domain.md`

```
        from pinneapple_design import build_domain, sample_domain

        domain = build_domain("lid_driven_cavity_2d")   # dim=2 by default
        x_int, x_bnd = sample_domain(domain, n_interior=4096, n_boundary=512)
```

Laço de projeto com blocos de montar — `examples/bricks/self_modifying_bricks.py`

```
        from pinneapple_design.bricks import (
            BridgeGoal,
            CantileverGoal,
            TowerGoal,
            compare,
            dynamics,
            ldraw,
            mesh,
            reference_cantilever,
            report,
            statics,
            validation,
            visuals,
        )
```

Estudo de resfriamento de placa — `examples/autolab/self_optimizing_engineer.py`

```
        from pinneapple_design.thermal.board_cooling import DEFAULT_TEXT, run_mission
        ...
                run = run_mission(a.text, out_dir=a.out, planner=a.planner, n=n, render=render, workers=a.workers)
                st = run.state
                m = st["read_mission"]["mission"]
                v, h = st["verify"], st["learn"]["holdout"]
```

Testes e status de validação

- Módulos com pelo menos um teste que os importa: 110 de 196.
- Funções de teste em arquivos que importam algum módulo do pacote: 744.

Itens do catálogo de métodos (`pinneapple_catalog/method_status.json`) cujo código está neste pacote:

id	método	status	testes contra referência	outros testes
S39	Density-based topology optimisation (SIMP, filter, projection, OC, MMA; compliance, thermal, Stokes-Brinkman, machine rotor)	validated	1	2

Limitações conhecidas

- `aero.case`, `aero.case3d`, `aero.dataset` e `aero.verify` precisam do OpenFOAM instalado para rodar; `render_blender` precisa do `bpy` (Blender).
- Os modelos de aeronave são de ordem reduzida (polar elíptica, build-up de Raymer, vortex lattice com esteira plana sem roll-up); o docstring de `vlm.streamlines` avisa que a esteira não mostra o enrolamento.
- Dependências opcionais mudam o que roda: `pymoo-nsga2` exige pymoo, o otimizador evolutivo usa `cma` se houver, o bayesiano e o `GPSurrogate` usam scikit-learn; métodos impossíveis voltam com status `skipped`.
- `Evaluator.torch` e os avaliadores não diferenciáveis levantam `NotImplementedError` (métodos por gradiente só funcionam com avaliadores diferenciáveis, como a fidelidade surrogate do dissipador).
- `meshops.union` exige `manifold3d`; `exporters.step_check` exige `cadquery` (OpenCASCADE); o carregamento do Hub depende de rede (há modo offline e reconstrução pelo código).
- A verificação completa do `intent` depende de programas externos: CalculiX e gmsh para o suporte, NVIDIA Newton para o loop; sem eles o relatório de confiança diz que a checagem completa não rodou.
- No template da asa de planador, a fidelidade 'full' é um vortex lattice fino; o docstring diz que um modelo RANS seria o próximo passo e não roda aqui.
- Os cinco casos usam fórmulas de livro texto com hipóteses declaradas; o docstring do pacote diz que nenhum foi validado contra medições.
- `sample_domain` chama `domain.sample_boundary`, mas `PhysicsDomain2D`/`PhysicsDomain3D` só têm `sample_boundary_region`; com os domínios de `build_domain` a chamada deve falhar (o exemplo da documentação usa exatamente esse caminho).
- `geometry.io.__init__` tenta importar `build_parametric_part` de `cadquery_bridge`, que não define esse nome; o valor fica sempre `None`.
- `gen.primitives.project_points_to_surface` é marcado como placeholder; o docstring diz que deveria levantar erro, mas devolve uma tupla `(pontos, distâncias)` em vez de um array.
- Muitas partes dependem de pacotes opcionais: trimesh, meshio, gmsh, CadQuery/OCC, pymeshlab, open3d, NVIDIA Newton (`bricks.dynamics`), Blender/`bpy` e ffmpeg (renders), `pinneapple_autolab` (agente). Os `__init__` usam imports tolerantes que viram `None` em silêncio.
- Vários módulos não têm teste direto: os construtores de lote (`builders`), `core.geometry`, `core.registry`, `io.*` (exceto IGES e trimesh), `mesh.mesher_2d`/`mesher_3d`, `ops.clean`, `ops.voxelize`, `sample.*`, `threaded_connection`, `genetic_sdf` e quase todo `thermal.board_cooling` e `thermal.heatsink.surrogate`/`monotone`/`trust`.
- `bricks.parts.JointModel` usa encaixe de 0,98 N por pino com faixa assumida (0,5 a 1,5 vez); os resultados da estática dependem dessa hipótese, e `bricks.report.sensitivity` existe para medi-la. `dynamics.friction_creep` documenta um artefato do atrito XPBD.
- `domains3d.get_pinn_batch` devolve alvos de contorno zerados (marcação), não valores físicos; o `STLDomainBatchBuilder` recentraliza a malha, o que quebra funções de valor que assumem coordenadas literais (motivo de existir `analytic_domain_batch_builder`).
- Os surrogates de `thermal.heatsink` só servem para triagem: `load_surrogates` só carrega o que existir em `ARTIFACTS` (`apps/heatsink_sizer/artifacts` ou `PINNEAPPLE_HEATSINK_ARTIFACTS`), e todo projeto recomendado é reverificado pelo motor físico.
- `fitmesh` depende de ferramentas externas opcionais: CadQuery e trimesh para CAD, Gmsh e OpenFOAM (blockMesh, snappyHexMesh, checkMesh) para malhas, FastAPI para a API.
- O docstring de `mesh_gen` avisa que camadas do snappyHexMesh podem colapsar perto de arestas vivas ou folgas finas.
- Geometria livre escrita por LLM roda em processo isolado com lista de imports e timeout, e fica marcada como sem referência publicada.
- Os modelos qualitativos são estimativas baratas (Newtoniano modificado, correlações), feitas para direção e ordem de grandeza; a área projetada é exata só para corpos convexos (limite superior nos demais).
- A otimização topológica usa grades estruturadas de Q4 em 2D; `oc` só trata a restrição de volume, outras restrições exigem `mma`.

- Muitos módulos não têm teste direto pelo inventário: em `fitmesh` (`cad`, `catalog`, `layers`, `mesh_gen`, `mesh_plan`, `report`, `session`), em `qualitative` (`descriptors`, `models`) e em `topology` (`driver`, `filters`, `optimizers`, `problems`); são exercitados via os `__init__` em `tests/test_fitmesh.py`, `tests/test_qualitative_preview.py` e nos testes de topologia.

2. `pinneapple_systems`

Objetivo

`pinneapple_systems` reúne as ferramentas para sistemas físicos com vários componentes que evoluem no tempo. O docstring do pacote lista cinco submódulos: `time_series` (previsão de sinais físicos, antigo `pinneapple_timeseries`), `cosimulation` (co-simulação diferenciável em grafo, antigo `pinneapple_cosim`), `digital_twin` (gêmeo digital com streams de sensores, filtros de Kalman e detecção de anomalias), `component_modeling` (controle, MPC, incerteza, resíduos de PDE e empacotamento ONNX) e `process_components` (modelos de operações unitárias de planta). Há também o módulo `excitation`, com sinais de excitação de atuador.

O problema que o pacote resolve é juntar, num só lugar, modelos aprendidos e modelos físicos de partes diferentes de um sistema. A co-simulação liga nós (redes PyTorch, PINNs com resíduo físico, funções analíticas, caixas-pretas, modelos do `ModelRegistry`, PDEs em SymPy, previsores de séries) por portas nomeadas, executa o grafo no tempo resolvendo laços algébricos e treina tudo junto por retropropagação no tempo. A parte de séries temporais vai do preparo dos dados (imputação, outliers, reamostragem, EDA, auditoria estatística) a modelos (baselines, ML tabular, LSTM/GRU, N-BEATS, TCN, TFT, FNO, decomposição FFT/HHT, ARIMA, modelos fundacionais Chronos e TimesFM), validação temporal sem vazamento, métricas e visualização.

Dentro de `time_series` há dois laboratórios de estudo maiores. O `forecast_lab` define uma interface comum e um protocolo rígido (teste fixado antes, tuning em walk-forward com purga, testes de Diebold-Mariano com correção de comparações múltiplas) para comparar baselines, modelos físicos, modelos de dados e híbridos em séries quase aleatórias; ele usa `synthetic` e `predictability`. O subpacote `enso` aplica esse protocolo à previsão do El Niño, com dados abertos baixados e verificados por hash, osciladores conceituais, um modelo acoplado intermediário e PINNs.

O pacote se apoia no resto do repositório: usa `pinneapple_neural` (arquiteturas, `ModelRegistry`, `Trainer`, `TrainConfig`, `WeightScheduler`, `InversePINN`), `pinneapple_physics` (compilação de `ProblemSpec` em perda física), `pinneapple_simulation.numerical_solvers` (decomposições clássicas), `pinneapple_analysis` (regressão por quantis) e os renderizadores Blender (`pinneapple_blender`, `pinneapple_tools`). A raiz oferece os atalhos `forecast` e `cosim_from_models`, e mantém os aliases antigos `timeseries` e `cosim`.

Esta parte de `pinneapple_systems` cobre quatro submódulos voltados a sistemas em operação. `digital_twin` é o runtime genérico de gêmeo digital (fluxos de sensores, filtros de Kalman, anomalias, prognóstico, padrões industriais) mais a camada de comissionamento `product` (contrato, preflight, backtest, evidência, política de deriva) e duas demos completas (`pipe_loop`, o gêmeo que sabe quando errou, e `recession`, o gêmeo de rio com dados do USGS). `operations` reúne uma planta de referência (tanque aquecido) e três ferramentas sobre ela (runtime de gêmeo com EnKF, detecção física de anomalias, MPC contra PID), usadas pelos apps `twin_runtime`, `anomaly_detector`, `control_lab` e `predictive_twin`. `process_components` traz modelos físicos de operações unitárias independentes (gás real via CoolProp, turbomáquinas, válvulas, trocadores, tubulações, vigas, cinética, golpe de aríete). `sports` aplica física a dados públicos de futebol e compara com baselines só de dados.

Esta parte cobre dois submódulos do `pinneapple_systems`. `component_library` é um registro de componentes de planta nomeados (tubo, trocador de calor, tanque, célula de bateria, bomba, válvula). Cada componente junta uma arquitetura já existente do `ModelRegistry` com uma restrição física opcional (`Physics`), e os componentes são agrupados em toolboxes versionadas e instaláveis. `component_modeling` traz ferramentas que funcionam com qualquer modelo diferenciável, seja qual for a física que ele representa: controle PID e MPC, incerteza (SWAG, ensemble profundo, MC-Dropout), resíduos físicos genéricos por autograd e empacotamento ONNX para borda com runtime sem torch.

Números do inventário

módulos	linhas	classes públicas	funções públicas	testes em arquivos que importam o pacote	módulos sem teste direto
182	37.262	301	515	579 (40 arquivos)	78

Submódulos

submódulo	o que faz	módulos	classes + funções	testes*
`component_library`	Registro de componentes de planta nomeados, com restrição física, benchmark por tipo e toolboxes instaláveis.	7	15	36
`component_modeling`	Controle, incerteza, resíduos físicos e exportação para borda para qualquer modelo diferenciável.	8	12	165
`cosimulation`	Motor de co-simulação diferenciável em grafo: nós com portas, conexões, motor por passos de tempo com laços algébricos, gravador, perdas e treinador por BPTT.	10	23	151
`digital_twin`	Runtime de gêmeo digital (fluxos, assimilação, anomalias, RUL, reconstrução, padrões AAS/DTDL/NGSI-LD/Ditto/OPC UA), camada de comissionamento `product` e as demos `pipe_loop` e `recession`.	33	129	172
`excitation`	Sinais de excitação de atuador em NumPy para identificação de sistemas e treino de surrogates.	1	12	6
`operations`	Planta de tanque aquecido e ferramentas de operação sobre ela: runtime de gêmeo EnKF, detecção física de anomalias, controle preditivo, ROM com entradas, serviço ao vivo e contratos de dados.	10	73	25
`process_components`	Modelos físicos independentes de operações unitárias para gêmeos de processo (gás real, compressores, válvulas, trocadores, tubos, vigas, reações, fadiga, golpe de aríete).	17	112	171
`sports`	Física aplicada a dados públicos de futebol (voo da bola, controle de campo, cinemática, xG), com estudos contra modelos só de dados.	12	87	16
`time_series`	Previsão de séries temporais de ponta a ponta: preparo, EDA, modelos, decomposição, validação, métricas, previsibilidade, previsão adaptativa e os estudos `forecast_lab` e `enso`.	83	351	151

* maior número de testes que importam um módulo do submódulo.

Módulos e API pública

`pinneapple_systems`

Raiz do pacote de sistemas multi-componente: importa os submódulos (`time_series`, `cosimulation`, `digital_twin`, `component_modeling`, `process_components`), reexporta seus nomes públicos, cria aliases antigos (`timeseries`, `cosim`) e define os atalhos `forecast` e `cosim_from_models`. [sem teste direto]

`pinneapple_systems/__init__.py` · 280 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`cosim_from_models`	função	`(models_dict, connections) -> 'CoSimGraph'`	Atalho que embrulha cada `nn.Module` de um dicionário num `PINNeAPPleModelNode` com portas fixas `input`/`output` e liga as conexões dadas como pares de strings.
`forecast`	função	`(data, horizon, *, model='auto', **kwargs)`	Atalho de previsão: escolhe uma classe por nome (`lstm`, `gru`, `nbeats`, `tft`, `tcn`, `xgboost`; `auto` vira `lstm`), chama `fit(data)` e `predict(horizon)`.

`pinneapple_systems.component_library`

Reexporta `ComponentRegistry`, `ComponentModel`, `Physics`, `Toolbox`, `ToolboxRegistry` e o benchmark, e registra os seis componentes de referência ao ser importado.

`pinneapple_systems/component_library/__init__.py` · 44 linhas · testes: 36

pinneapple_systems.component_library.base

`ComponentModel`: um `BaseModel` que envolve uma arquitetura registrada, conhece sua `Physics` e oferece `fit` e `evaluate` (RMSE, MAPE, R^2) em uma chamada. [sem teste direto pelo inventário]

`pinneapple_systems/component_library/base.py` · 173 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ComponentModel`	classe	`(*, architecture, architecture_kwargs=None, physics=None)`	Componente de planta nomeado: arquitetura existente + `Physics` opcional, com `fit` e `evaluate`.

pinneapple_systems.component_library.benchmark

Treina todas as arquiteturas registradas sob um `component_type` e as ranqueia pela perda final de validação. A marca 'placeholder' do inventário é falso positivo: o docstring diz 'not a display placeholder'. [sem teste direto pelo inventário]

`pinneapple_systems/component_library/benchmark.py` · 77 linhas · **sem teste direto** · palavras-chave no código (heurística; o texto acima diz o status real): placeholder/stub citado no docstring

nome	tipo	assinatura	o que faz
`ComponentBenchmarkResult`	classe	`(component_type, ranked)`	Ranking das arquiteturas de um tipo de componente, com `best`.
`benchmark_component_type`	função	`(component_type, train_loader, val_loader=None, *, epochs=50, lr=0.001, eval_coords=None,…)`	Treina cada arquitetura de um `component_type` e ranqueia pela perda de validação.

pinneapple_systems.component_library.components

Seis componentes de referência (tubo, trocador de calor, casco de tanque, célula de bateria, bomba, válvula), cada um com uma combinação diferente de física e arquitetura (`vanilla_pinn` ou `modified_mlp`); a válvula é só por dados. [sem teste direto pelo inventário]

`pinneapple_systems/component_library/components.py` · 126 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`PipeVanillaPINN`	classe	`(in_dim=3, out_dim=3, hidden=(128, 128, 128, 128), **kwargs)`	Componente tubo: VanillaPINN com restrição de escoamento incompressível.
`HeatExchangerVanillaPINN`	classe	`(in_dim=2, out_dim=1, hidden=(64, 64, 64), **kwargs)`	Componente trocador de calor: VanillaPINN com resíduo de condução de calor.
`TankShellVanillaPINN`	classe	`(in_dim=2, out_dim=2, hidden=(64, 64, 64), **kwargs)`	Componente casco de tanque: VanillaPINN com resíduo de elasticidade linear.
`BatteryCellVanillaPINN`	classe	`(in_dim=2, out_dim=1, hidden=(64, 64, 64), **kwargs)`	Componente célula de bateria: VanillaPINN com resíduo de difusão de espécies.
`PumpModifiedMLP`	classe	`(in_dim=3, out_dim=3, hidden_dim=128, n_layers=6, **kwargs)`	Componente bomba: ModifiedMLP com restrição de escoamento incompressível.
`ValveModifiedMLP`	classe	`(in_dim=2, out_dim=1, hidden_dim=64, n_layers=3, **kwargs)`	Componente válvula: ModifiedMLP só com dados, sem restrição física.

pinneapple_systems.component_library.physics

`Physics`: liga um componente à sua restrição, reduzida a uma `loss_fn` no formato do `Trainer`, com parâmetros guardados para inspeção. [sem teste direto pelo inventário]

`pinneapple_systems/component_library/physics.py` · 88 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Physics`	classe	`(name, loss_fn, params)`	Restrição física nomeada reduzida a uma `loss_fn`, com parâmetros para inspeção.

pinneapple_systems.component_library.registry

ComponentRegistry guarda componentes por component_type, permitindo várias arquiteturas concorrentes no mesmo tipo, e os constrói por nome. [sem teste direto pelo inventário]

pinneapple_systems/component_library/registry.py · 122 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
ComponentSpec	classe	(component_type, name, cls, physics, description, tags)	Registro de um componente: tipo, nome, classe, física, descrição e tags.
ComponentRegistry	classe	(_REGISTRY)	Registro central de componentes: registra, lista por tipo, dá a spec e constrói.
register_component	função	(*, component_type, name, physics=None, description='', tags=None)	Decorador de módulo equivalente a ComponentRegistry.register.

pinneapple_systems.component_library.toolbox

Toolbox: pacote nomeado e versionado de tipos de componentes com dependências; ToolboxRegistry registra e mantém o conjunto instalado. Registra a 'Fluid Mechanics Toolbox' e a 'Process Equipment Toolbox'. [sem teste direto pelo inventário]

pinneapple_systems/component_library/toolbox.py · 110 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
Toolbox	classe	(name, version, component_types, description, depends_on)	Pacote nomeado e versionado de tipos de componentes, com dependências e lista de componentes.
ToolboxRegistry	classe	(_REGISTRY, _INSTALLED)	Registro de toolboxes com instalar, desinstalar e consultar as instaladas.

pinneapple_systems.component_modeling

Reexporta controle, MPC, SWAG, ensemble, MC-Dropout, resíduos físicos e exportação para borda; o docstring explica que registro, classe base e geração de malha ficam em outros pacotes. [sem teste direto do __init__]

pinneapple_systems/component_modeling/__init__.py · 60 linhas · **sem teste direto**

pinneapple_systems.component_modeling.bayesian

SWAGApproximation: posterior Gaussiana diagonal sobre os pesos via SWAG, a partir de snapshots ao longo do treino, com previsão por amostragem de pesos.

pinneapple_systems/component_modeling/bayesian.py · 127 linhas · testes: 151

nome	tipo	assinatura	o que faz
SWAGApproximation	classe	(model)	Posterior aproximada SWAG (diagonal) sobre os pesos de um modelo, com fit e predict_with_uncertainty.

pinneapple_systems.component_modeling.control

Controlador PID discreto com limites de saída e um executor de malha fechada para planta diferenciável ou caixa-preta, com ruído de processo e medição e observador opcional.

pinneapple_systems/component_modeling/control.py · 113 linhas · testes: 165

nome	tipo	assinatura	o que faz
PIDController	classe	(kp, ki, kd, setpoint, output_limits=None, bias=0.0)	PID discreto padrão com bias e limites de saída.
run_closed_loop	função	(plant, *, setpoint, kp=1.0, ki=0.1, kd=0.0, steps=50, dt=0.1, process_noise=0.0, measure…)	Fecha uma malha de controle PID em torno de uma planta e devolve os históricos.

pinneapple_systems.component_modeling.edge

Exporta um modelo para ONNX e compacta com `manifest.json` num zip; `EdgeRuntime` roda esse pacote só com onnxruntime e NumPy.

`pinneapple_systems/component_modeling/edge.py` · 96 linhas · testes: 151

nome	tipo	assinatura	o que faz
`EdgeRuntime`	classe	`(zip_path)`	Runtime mínimo de borda que carrega o zip exportado e roda inferência ONNX sem torch.
`export_edge_package`	função	`(model, output_dir, *, name, sample_input, opset_version=17, extra_manifest=None) -> Dict…)`	Exporta o modelo para ONNX e compacta com um manifesto num pacote de borda.

pinneapple_systems.component_modeling.ensemble

`DeepEnsemble`: N cópias da mesma arquitetura com inicializações diferentes, treinadas separadamente; a média e o desvio entre membros dão a incerteza epistêmica.

`pinneapple_systems/component_modeling/ensemble.py` · 84 linhas · testes: 151

nome	tipo	assinatura	o que faz
`DeepEnsemble`	classe	`(model_factory, n_members=5, base_seed=0)`	Ensemble profundo com `fit` e `predict` devolvendo média e desvio entre membros.

pinneapple_systems.component_modeling.mc_dropout

Incerteza por MC-Dropout: várias passagens com dropout ativo, reduzidas a média e desvio; erro se o modelo não tem dropout. [sem teste direto]

`pinneapple_systems/component_modeling/mc_dropout.py` · 41 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`mc_dropout_uncertainty`	função	`(model, x, n_samples=50) -> Tuple[torch.Tensor, torch.Tensor]`	Média e desvio de várias passagens com dropout ativo.

pinneapple_systems.component_modeling.mpc

MPC de horizonte deslizante baseado em gradiente em torno de um componente diferenciável. [sem teste direto pelo inventário]

`pinneapple_systems/component_modeling/mpc.py` · 119 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`run_mpc`	função	`(plant, *, in_dim, setpoint, control_input_index=0, observed_output_index=0, horizon=10, …)`	MPC de horizonte deslizante que otimiza a entrada de controle por gradiente a cada passo.

pinneapple_systems.component_modeling.physics_residuals

Resíduos de EDP fechados por autograd para qualquer modelo coordenadas→campos: continuidade incompressível, condução de calor, elasticidade linear e difusão de espécies; saídas insuficientes levantam `NotImplementedError`.

`pinneapple_systems/component_modeling/physics_residuals.py` · 126 linhas · testes: 8 · palavras-chave no código (heurística; o texto acima diz o status real): levanta `NotImplementedError` em algum caminho

nome	tipo	assinatura	o que faz
`incompressible_continuity_residual`	função	`(model, coords) -> torch.Tensor`	Resíduo de div(u) = 0 para as primeiras saídas de velocidade.
`heat_conduction_residual`	função	`(model, coords, *, k=50.0, Q=0.0) -> torch.Tensor`	Resíduo estacionário k·ΔT + Q.

nome	tipo	assinatura	o que faz
`linear_elasticity_residual`	função	`(model, coords, *, E=210000000000.0, nu=0.3) -> torch.Tensor`	Resíduo de equilíbrio de Navier-Lamé sem força de corpo.
`species_diffusion_residual`	função	`(model, coords, *, D=1e-05, k=0.0) -> torch.Tensor`	Resíduo estacionário de difusão-reação D·ΔC − k·C.

pinneapple_systems.cosimulation

Motor nativo de co-simulação diferenciável: nós com portas nomeadas, conexões, grafo com análise de topologia, motor por passos de tempo, gravador de trajetórias, perdas e treinador; o `__init__` só reexporta. [sem teste direto do `__init__`]

`pinneapple_systems/cosimulation/__init__.py` · 86 linhas · **sem teste direto**

pinneapple_systems.cosimulation.adapters

Nós de co-simulação que ligam o motor ao resto do PINNeAPPle: problema físico compilado (`ProblemSpec`), modelos do `ModelRegistry`, PDE simbólica em SymPy e modelos de séries temporais.

`pinneapple_systems/cosimulation/adapters.py` · 432 linhas · testes: 151

nome	tipo	assinatura	o que faz
`PINNeProblemNode`	classe	`(name, model, spec, compiled_fn, coord_ports, field_ports, extra_input_ports=None, physic…)`	Nó apoiado num `ProblemSpec` do PINNeAPPle e na perda física compilada por `compile_problem`; separa portas de coordenadas e de campos e expõe `physics_loss`.
`PINNeAPPleModelNode`	classe	`(name, model, input_ports, output_ports, batch_port_map=None) -> None`	Embrulha qualquer `BaseModel` (ou `nn.Module`) do `ModelRegistry` como nó, mapeando portas de entrada e saída para o batch do modelo; `from_registry` constrói pelo nome.
`SymbolicPDENode`	classe	`(name, model, symbolic_pde, coord_ports, field_ports, physics_weight=1.0) -> None`	Nó cujo resíduo físico é uma expressão SymPy convertida em resíduo com autograd sobre as portas de coordenadas e campos.
`TimeSeriesCoSimNode`	classe	`(name, model, input_len, horizon, extra_input_ports=None, extra_output_ports=None) -> None`	Embrulha um `TSModelBase` (LSTM, TFT, FFT+LSTM…) como nó: recebe uma janela de `input_len` e devolve a previsão de `horizon` passos.

pinneapple_systems.cosimulation.connection

Define `Connection`, a aresta imutável `no.porta -> no.porta` do grafo de co-simulação, com parser de strings. [sem teste direto]

`pinneapple_systems/cosimulation/connection.py` · 81 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Connection`	classe	`(src_node, src_port, dst_node, dst_port)`	Aresta dirigida e imutável entre a porta de saída de um nó e a porta de entrada de outro; `parse` lê strings `no.porta`.

pinneapple_systems.cosimulation.engine

`CoSimEngine`: executa o grafo passo a passo no tempo, na ordem das componentes fortemente conexas, iterando laços algébricos com Gauss-Seidel ou Jacobi e preservando o grafo do autograd.

`pinneapple_systems/cosimulation/engine.py` · 257 linhas · testes: 151

nome	tipo	assinatura	o que faz
`CoSimEngine`	classe	`(graph, recorder=None, loop_solver='gauss_seidel', max_iter=50, tol=1e-06, retain_graph=F…)`	Motor de co-simulação por passos de tempo sobre um `CoSimGraph`: inicializa portas, executa cada passo (com iteração de laço até `tol` ou `max_iter`) e grava no `TrajectoryRecorder`.

pinneapple_systems.cosimulation.graph

CoSimGraph: registro de nós e conexões com validação de portas, ordem de execução em cache e detecção de ciclos e laços algébricos.

pinneapple_systems/cosimulation/graph.py · 187 linhas · testes: 151

nome	tipo	assinatura	o que faz
CoSimGraph	classe	() -> None	Registro de nós e conexões com cache de topologia: ordem de execução, ciclos, laços algébricos, conexões de entrada/saída e parâmetros treináveis.

pinneapple_systems.cosimulation.losses

Perdas compostas para treinar grafos de co-simulação: ajuste a dados, resíduos físicos dos nós PINN e consistência entre portas conectadas.

pinneapple_systems/cosimulation/losses.py · 225 linhas · testes: 151

nome	tipo	assinatura	o que faz
DataLoss	classe	(weight=1.0) -> None	Erro quadrático médio entre valores previstos nas portas e observações.
PhysicsLoss	classe	(weight=1.0) -> None	Soma dos resíduos físicos (physics_loss) de todos os nós PINN do grafo.
CouplingLoss	classe	(weight=0.1) -> None	Penaliza inconsistências entre os valores das portas ligadas por uma conexão.
CoSimLoss	classe	(data_weight=1.0, physics_weight=1.0, coupling_weight=0.1) -> None	Perda composta e ponderada: dados + física + acoplamento.

pinneapple_systems.cosimulation.node

Abstrações de nó: base abstrata CoSimNode e nós concretos para nn.Module, função Python/NumPy, PINN com resíduo físico e caixa-preta sem gradiente.

pinneapple_systems/cosimulation/node.py · 291 linhas · testes: 151

nome	tipo	assinatura	o que faz
CoSimNode	classe	(name, input_ports, output_ports) -> None	Classe base abstrata de todo nó: nome, portas de entrada e saída, step, reset, physics_loss e parameters.
TorchNode	classe	(name, model, input_ports, output_ports) -> None	Nó que embrulha um nn.Module; os gradientes passam por step().
AnalyticalNode	classe	(name, fn, input_ports, output_ports) -> None	Nó que embrulha uma função Python/NumPy pura; as saídas saem desligadas do autograd.
PINNNode	classe	(name, model, input_ports, output_ports, physics_fn, physics_weight=1.0) -> None	TorchNode com uma função de resíduo físico e peso próprio, usada pela PhysicsLoss.
BlackBoxNode	classe	(name, fn, input_ports, output_ports) -> None	Nó opaco (por exemplo um simulador externo) totalmente desligado do autograd.
split_output_ports	função	(out, ports) -> Dict[str, torch.Tensor]	Divide a última dimensão de um tensor de saída igualmente entre as portas dadas.

pinneapple_systems.cosimulation.recorder

Gravação de trajetórias no tempo por (nó, porta) durante uma execução de co-simulação, com acesso como listas, arrays ou tensores.

pinneapple_systems/cosimulation/recorder.py · 177 linhas · testes: 151

nome	tipo	assinatura	o que faz
Trajectory	classe	(node, port) -> None	Série temporal de um único sinal (nó, porta), com tempos e valores como arrays ou tensor.
TrajectoryRecorder	classe	() -> None	Grava saídas de portas com carimbo de tempo para vários nós; permite escolher o que observar e resumir as trajetórias.

pinneapple_systems.cosimulation.trainer

CoSimTrainer: treina por BPTT (desenrolando n_unroll passos) todos os parâmetros do grafo com TrainConfig, WeightScheduler, parada antecipada e checkpoint de pinneapple_neural.

pinneapple_systems/cosimulation/trainer.py · 331 linhas · testes: 151

nome	tipo	assinatura	o que faz
CoSimTrainer	classe	(graph, engine, criterion=None, weight_scheduler_cfg=None, early_stopping=None, checkpoin…)	Treina um grafo de co-simulação diferenciável com a pilha de treino do PINNeAPPle (BPTT, balanceamento de pesos, parada antecipada, checkpoint) e guarda o histórico.

pinneapple_systems.cosimulation.utils

Algoritmos de grafo para a topologia da co-simulação: Tarjan (componentes fortemente conexas), ordem por condensação e ordenação topológica. [sem teste direto]

pinneapple_systems/cosimulation/utils.py · 180 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
tarjan_scc	função	(adj) -> List[List[str]]	Encontra as componentes fortemente conexas de um grafo de adjacência com o algoritmo de Tarjan.
condensation_order	função	(adj) -> Tuple[List[List[str]], List[List[str]]]	Calcula a ordem de execução pela condensação das componentes fortemente conexas e devolve também os laços.
topological_sort	função	(adj) -> List[str]	Ordenação topológica para grafos acíclicos (DAG).

pinneapple_systems.digital_twin

Runtime de gêmeo digital sobre modelos substitutos: reexporta DigitalTwin, estados, fluxos de sensores, filtros de Kalman, detectores de anomalia, pipeline de tempo real, reconstrução de sinais, condicionamento de telemetria e prognóstico (RUL).

pinneapple_systems/digital_twin/__init__.py · 121 linhas · testes: 15

pinneapple_systems.digital_twin.assimilation

Subpacote de assimilação de dados: só reexporta ExtendedKalmanFilter e EnsembleKalmanFilter. [sem teste direto]

pinneapple_systems/digital_twin/assimilation/__init__.py · 5 linhas · **sem teste direto**

pinneapple_systems.digital_twin.assimilation.kalman

[shim] Shim de compatibilidade: a implementação real de ExtendedKalmanFilter/EnsembleKalmanFilter mudou para pinneapple_analysis.state_estimation.kalman; aqui só se reexportam os nomes para os chamadores antigos.

pinneapple_systems/digital_twin/assimilation/kalman.py · 20 linhas · testes: 157 · palavras-chave no código (heurística; o texto acima diz o status real): shim ou obsoleto

pinneapple_systems.digital_twin.conditioning

Condicionamento causal de telemetria, leitura a leitura: checagem de faixa, rejeição de outliers (Hampel causal com predição de um passo), limite de taxa, filtro passa-baixa e preenchimento de lacunas, com flags do que foi feito. [sem teste direto] — tests/test_telemetry_conditioning.py usa via pinneapple_systems.digital_twin.

pinneapple_systems/digital_twin/conditioning.py · 195 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`SignalProfile`	classe	`(kind, tau_s, hampel_window, hampel_k, max_consecutive_outliers, max_rate, valid_range, m…)`	Configuração de como condicionar um tipo de sinal (constante de tempo, janela e limiar de Hampel, taxa máxima, faixa válida, lacuna máxima); há presets em `PROFILES`.
`ConditionedSample`	classe	`(t, raw, value, flags)`	Uma leitura condicionada: tempo, valor bruto, valor final e flags; `ok` diz se o valor é finito e não velho.
`StreamingConditioner`	classe	`(profile='generic', **overrides)`	Condiciona um sinal, uma leitura por vez, de forma causal (custo O(janela) por leitura); `update` processa, `fill` preenche lacunas.
`TelemetryConditioner`	classe	`(profiles=None, default='generic')`	Conjunto de condicionadores por sensor e campo para um gêmeo inteiro; `condition_observation` limpa uma `Observation`.

pinneapple_systems.digital_twin.io

Subpacote de entrada/saída: reexporta `Sensor`, `SensorRegistry` e os adaptadores de fluxo de `stream`.

`pinneapple_systems/digital_twin/io/__init__.py` · 26 linhas · testes: 8

pinneapple_systems.digital_twin.io.mqtt_broker

Broker MQTT 3.1.1 embutido e pequeno (QoS 0 e 1, mensagens retidas, curingas +/#, usuário/senha opcional) para demos, testes e borda de um ativo; sem persistência, sem QoS 2, sem will e sem TLS.

`pinneapple_systems/digital_twin/io/mqtt_broker.py` · 263 linhas · testes: 14

nome	tipo	assinatura	o que faz
`MQTTBroker`	classe	`(host='127.0.0.1', port=1883, username=None, password=None, max_packet=256 * 1024)`	Broker MQTT embutido que roda numa thread com laço de eventos próprio; `start`/`stop`, publica e entrega a assinantes.
`topic_matches`	função	`(filt, topic) -> bool`	Testa se um tópico casa com um filtro MQTT (`+` um nível, `#` o resto; tópicos `$` exigem primeiro nível explícito).

pinneapple_systems.digital_twin.io.sensors

Cadastro de sensores do gêmeo: posição, calibração (offset e escala), ruído e campos; cada `Sensor` converte uma leitura bruta em `Observation` calibrada.

`pinneapple_systems/digital_twin/io/sensors.py` · 151 linhas · testes: 151

nome	tipo	assinatura	o que faz
`Sensor`	classe	`(sensor_id, coords, field_names, calibration_offset, calibration_scale, noise_std, transf…)`	Um sensor físico: id, coordenadas, campos, calibração, ruído e transformação; `read` gera uma `Observation` calibrada.
`SensorRegistry`	classe	`() -> None`	Cadastro de todos os sensores do gêmeo: adiciona, remove, lista ativos, lê em lote e monta o vetor de observações.

pinneapple_systems.digital_twin.io.stream

Adaptadores de fluxo de dados em tempo real (arquivo JSONL/CSV/Parquet, MQTT, HTTP, Kafka, OPC UA, Modbus TCP e um fluxo sintético) que emitem `Observation` numa fila consumida pelo `DigitalTwin`; MQTT, Kafka, OPC UA e Modbus pedem bibliotecas opcionais.

`pinneapple_systems/digital_twin/io/stream.py` · 576 linhas · testes: 172

nome	tipo	assinatura	o que faz
`BaseStream`	classe	`(sensor_id, field_names) -> None`	Classe base abstrata dos adaptadores de fluxo: liga a fila de saída e define `start`/`stop`.
`FileWatchStream`	classe	`(path, sensor_id, field_names, *, poll_interval=1.0, format='jsonl') -> None`	Consulta periodicamente um arquivo JSON-lines (ou CSV/Parquet, com pandas) e emite as linhas novas.

nome	tipo	assinatura	o que faz
`MQTTStream`	classe	`(broker, topic, sensor_id, field_names, *, port=1883, keepalive=60, username=None, passwo…)`	Assina um tópico MQTT e emite observações (requer `paho-mqtt`).
`HTTPPollStream`	classe	`(url, sensor_id, field_names, *, poll_interval=5.0, headers=None, transform=None) -> None`	Consulta um endpoint REST por GET em intervalos e emite observações.
`KafkaStream`	classe	`(bootstrap_servers, topic, sensor_id, field_names, *, group_id='pinneapple_dt', auto_offs…)`	Lê mensagens de um tópico Apache Kafka (requer `kafka-python`).
`OPCUAStream`	classe	`(server_url, node_ids, sensor_id, field_names, *, poll_interval=1.0) -> None`	Consulta tags de um servidor OPC UA em intervalos (requer `asyncua`).
`ModbusStream`	classe	`(host, port, register_map, sensor_id, field_names, *, poll_interval=1.0, device_id=1) -> …)`	Consulta registradores holding de um servidor Modbus TCP (requer `pymodbus`).
`MockStream`	classe	`(sensor_id, field_names, generator_fn, *, tick_interval=0.1, coords=None) -> None`	Fluxo sintético a partir de uma função geradora, para testes e simulação.

pinneapple_systems.digital_twin.io.usgs

Cliente da API pública "Instantaneous Values" do USGS (vazão, nível de rios dos EUA), com cache curto em memória; falhas levantam `USGSDataError`, nada é preenchido. Os dados são baixados em tempo de execução.

`pinneapple_systems/digital_twin/io/usgs.py` · 100 linhas · testes: 8

nome	tipo	assinatura	o que faz
`USGSDataError`	classe	`()`	Erro quando o serviço do USGS não responde ou não devolve dados utilizáveis.
`Series`	classe	`(site_id, site_name, parameter_cd, unit, timestamps, values, fetched_at, from_cache)`	Série temporal baixada de um posto: id e nome do posto, parâmetro, unidade, tempos, valores e se veio do cache.
`monitoring_url`	função	`(site_id) -> str`	URL da página pública de um posto do USGS, para o leitor conferir os dados.
`parse_iv`	função	`(payload, site_id, parameter_cd) -> Series`	Extrai a primeira série de um JSON da API IV, pulando leituras malformadas e ordenando por tempo.
`fetch_iv`	função	`(site_id, parameter_cd=DISCHARGE, period='P5D', timeout_s=10.0, cache_ttl_s=90.0) -> Seri…)`	Baixa um parâmetro de um posto num período ISO 8601 recente, com timeout e cache em memória.

pinneapple_systems.digital_twin.monitoring

Subpacote de monitoramento: reexporta os detectores de anomalia e o `AnomalyMonitor`. [sem teste direto]

`pinneapple_systems/digital_twin/monitoring/__init__.py` · 17 linhas · **sem teste direto**

pinneapple_systems.digital_twin.monitoring.anomaly

Detecção de anomalias que compara previsões com observações: limiar fixo, z-score móvel, distância de Mahalanobis e checagem dos resíduos de inovação do filtro, combinados num monitor composto.

`pinneapple_systems/digital_twin/monitoring/anomaly.py` · 415 linhas · testes: 166

nome	tipo	assinatura	o que faz
`AnomalyEvent`	classe	`(timestamp, sensor_id, field_name, observed, predicted, score, detector, metadata)`	Registro de uma anomalia: tempo, sensor, campo, valor observado e previsto, escore e detector.
`ThresholdDetector`	classe	`(thresholds, rel_thresholds=None, eps=1e-08) -> None`	Detector simples por limiar absoluto (e opcionalmente relativo) por campo.
`ZScoreDetector`	classe	`(z_threshold=3.0, window_size=100, min_samples=10) -> None`	Detector por z-score numa janela móvel de referência.
`MahalanobisDetector`	classe	`(threshold=3.0, field_order=None) -> None`	Detector multivariado por distância de Mahalanobis, com covariância atualizável.
`ResidualDetector`	classe	`(z_threshold=3.0, window_size=100, min_samples=10) -> None`	Checa os resíduos de inovação do filtro de assimilação com z-score móvel.
`AnomalyMonitor`	classe	`() -> None`	Monitor composto que roda vários detectores em sequência e guarda os eventos recentes.

pinneapple_systems.digital_twin.pipe_loop

Demo 3 do Autonomous Physics Lab: gêmeo de um circuito de água bombeada e aquecida (cotovelo e junção em T) que distingue processo mudado, sensor errado e modelo fora do domínio; reexporta `run_mission`, `run_benchmark`, a planta e o diagnóstico.

`pinneapple_systems/digital_twin/pipe_loop/__init__.py` · 31 linhas · testes: 7

pinneapple_systems.digital_twin.pipe_loop.agent

Missão do gêmeo como ferramentas controladas de `pinneapple_autolab` (montar o modelo rápido, calibrar alarmes, monitorar, pedir nova simulação, benchmark, relatório de confiança, entrega), com planejador determinístico ou LLM. [sem teste direto] — `tests/test_autolab_twin.py` usa via pacote.

`pinneapple_systems/digital_twin/pipe_loop/agent.py` · 217 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`t_read_mission`	função	`(state, text) -> dict[str, Any]`	Ferramenta que lê o pedido em texto, monta o `Loop` e marca quais hipóteses (processo, sensor, domínio, benchmark) foram pedidas.
`t_build_twin`	função	`(state, n_train=120, n_test=40, seed=0) -> dict[str, Any]`	Ferramenta que ajusta o modelo rápido (`Surrogate`) e mede o erro em pontos de operação separados (hold-out) e a cobertura de 2 desvios.
`t_calibrate`	função	`(state, n_runs=12) -> dict[str, Any]`	Ferramenta que calibra os limiares do `Diagnoser` em corridas saudáveis.
`t_monitor`	função	`(state) -> dict[str, Any]`	Ferramenta que roda e diagnostica os casos de vitrine (`SHOWCASE`) e conta os acertos.
`t_request_simulation`	função	`(state, n_new=40, widen=0.25) -> dict[str, Any]`	Para cada caso diagnosticado fora do domínio, roda a referência, compara com as leituras, amplia o envelope, adiciona soluções novas, reajusta o modelo e diagnostica de novo.
`t_benchmark`	função	`(state, n_per_kind=20, n_calibration=12, workers=4) -> dict[str, Any]`	Ferramenta que roda o benchmark controlado e devolve acurácia, número de casos e casos com alarme falso.
`t_trust`	função	`(state) -> dict[str, Any]`	Ferramenta que monta o relatório de confiança e sua decisão.
`t_deliver`	função	`(state, out_dir='out', render=True, samples=16) -> dict[str, Any]`	Ferramenta que gera figuras, cena 3-D, visualizador, renders e o relatório.
`toolbox`	função	`() -> Toolbox`	Monta o `Toolbox` `pipe_twin` com as oito ferramentas e seus resumos.
`template_plan`	função	`(text, *, out_dir='out', render=True, n_per_kind=20, workers=4) -> TemplatePlanner`	Plano determinístico (`TemplatePlanner`) com a ordem padrão das ferramentas.
`run_mission`	função	`(text=DEFAULT_TEXT, *, out_dir='out', planner='template', render=True, n_per_kind=20, wor…)`	Executa a missão inteira com planejador `template` ou `llm` e devolve o `Run` com o estado de cada passo.

pinneapple_systems.digital_twin.pipe_loop.benchmark

Casos controlados com resposta conhecida (normal, processo, sensor, domínio) e a matriz de confusão do diagnóstico, com escores de localização, alarmes falsos e atrasos. [sem teste direto]

`pinneapple_systems/digital_twin/pipe_loop/benchmark.py` · 123 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`make_cases`	função	`(n_per_kind=20, seed=0) -> list[Scenario]`	Sorteia cenários controlados das quatro classes (normal, processo, sensor, domínio) com semente.
`run_benchmark`	função	`(loop=None, *, n_per_kind=20, n_calibration=12, seed=0, n_train=120, workers=4, log=None)…)`	Ajusta o modelo rápido, calibra em corridas saudáveis, diagnostica cada caso e devolve matriz de confusão, escores de isolamento, alarmes falsos, atrasos e tempos.

pinneapple_systems.digital_twin.pipe_loop.diagnose

Como o gêmeo sabe que errou: resíduos físicos estruturados, modelo rápido POD + processo gaussiano, EKF de perda extra por tubo e domínio de validade, combinados num diagnóstico por janela. [sem teste direto]

pinneapple_systems/digital_twin/pipe_loop/diagnose.py · 313 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
Residuals	classe	(loop)	Resíduos físicos de um vetor de leituras (massa no T, Darcy por tubo, bomba, válvulas, energia), em unidades de sensor, com seu desvio de ruído e jacobiano.
Surrogate	classe	(n_train, seed, model, lo, hi)	Modelo rápido POD + processo gaussiano das dez leituras sobre NORMAL_BOX, com desvio preditivo e excesso fora do envelope.
LossTracker	classe	(loop, q_walk=0.02, k0_sigma=0.5)	EKF do coeficiente de perda extra de cada tubo (passeio aleatório), observado pela queda de pressão medida dada a vazão medida.
Diagnoser	classe	(loop, surrogate, window, thresholds, ood_threshold)	Transforma evidência em janela numa hipótese (normal, processo, sensor, domínio) com alvo; limiares calibrados em corridas saudáveis.

pinneapple_systems.digital_twin.pipe_loop.plant

A planta e seu simulador de referência quase estacionário: bomba (leis de afinidade), Darcy-Weisbach/Colebrook, perdas localizadas, balanço de energia, dez sensores com ruído e falhas injetadas (obstrução, deriva, ruído).

pinneapple_systems/digital_twin/pipe_loop/plant.py · 258 linhas · testes: 7

nome	tipo	assinatura	o que faz
Pipe	classe	(length, diameter, roughness, k_minor, ua_w_k)	Um tubo: comprimento, diâmetro, rugosidade, perda localizada e UA; calcula área, perda de carga e vazão a partir da perda.
Loop	classe	(pipe_A, pipe_B, pipe_C, h0_m, k_pump, kv0_B, kv0_C, z_B, z_C, t_room_c, p_atm_head_m, wa…)	Geometria e máquinas do circuito (três tubos, bomba, válvulas, cotas, ambiente) em SI.
Faults	classe	(k_extra, bias, noise_gain)	O que está errado num instante: perda extra por tubo, viés e ganho de ruído por sensor (tudo zero = saudável).
Scenario	classe	(name, kind, seed, duration_s, dt_s, onset_s, ramp_s, target, magnitude, mode)	Caso de teste controlado: histórico de operação e, opcionalmente, uma falha com início e rampa.
water	função	(t_c) -> tuple[float, float]	Densidade e viscosidade dinâmica da água numa temperatura em °C (tabela de White).
steady_state	função	(loop, u, k_extra=None) -> dict[str, float]	Estado verdadeiro da planta para entradas dadas: vazões, cargas, pressões e temperaturas.
readings	função	(state) -> np.ndarray	Valores verdadeiros dos sensores em unidades de sensor (bar, m3/h, °C) na ordem SENSORS.
quasi_steady_valid	função	(loop, stroke_s, factor=10.0) -> bool	Diz se o modelo quase estacionário vale para um curso de válvula (curso ≥ fator × 2L/a).
operating_history	função	(sc) -> tuple[np.ndarray, dict[str, np.ndarray]]	Movimentos lentos do operador dentro de NORMAL_BOX (soma de senos); num cenário de domínio leva uma entrada para fora da caixa.
run_plant	função	(loop, sc) -> dict[str, Any]	Simula um cenário: entradas, valores verdadeiros, leituras com ruído e a falha aplicada a cada passo.
with_loop	função	(loop, **kw) -> Loop	Cópia de um Loop com campos trocados.

pinneapple_systems.digital_twin.pipe_loop.report

Entregáveis da demo: figuras de sala de controle, matriz de confusão, relatório de confiança, GIF, cena 3-D (visualizador web, glTF, USD, renders Blender opcionais) e relatório em texto. [sem teste direto]

pinneapple_systems/digital_twin/pipe_loop/report.py · 341 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
trust_report	função	(state) -> TrustReport	Monta o relatório de confiança (TrustReport) com checagens e decisão a partir do estado das ferramentas.

nome	tipo	assinatura	o que faz
`control_room`	função	`(case, path, *, title='', cursor=None) -> str`	Figura de sala de controle de um caso: leituras, previsões e evidência ao longo do tempo (PNG).
`figure_confusion`	função	`(b, path) -> str`	Figura da matriz de confusão do benchmark.
`figure_request`	função	`(rs, path) -> str | None`	Figura do pedido de nova simulação (antes e depois de ampliar o modelo), ou `None` se não houve.
`impact_gif`	função	`(case, path, n=14) -> str`	GIF 2-D do impacto de um caso ao longo do tempo (usa imageio).
`render_3d`	função	`(case, out_dir, *, render, samples, size=(1280, 720)) -> dict[str, Any]`	Gera visualizador web, glTF e USD da cena; renders Blender só se `render` e se `bpy` estiver instalado.
`deliver`	função	`(state, out_dir, *, render=True, samples=16) -> dict[str, Any]`	Gera todos os entregáveis (figuras, GIF, 3-D, `trust_report.json`, relatório) numa pasta.
`write_report`	função	`(state, out_dir, files, env) -> str`	Escreve o relatório em texto com os números das ferramentas e o ambiente da execução.

`pinneapple_systems.digital_twin.pipe_loop.scene`

Cena 3-D esquemática do circuito, quadro a quadro: tubos coloridos pela pressão de referência, linhas de corrente do cotovelo por lei de potência 1/7 e sensores coloridos pelo veredito. [sem teste direto]

`pinneapple_systems/digital_twin/pipe_loop/scene.py` · 181 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`tube`	função	`(path, r=R_PIPE, n_theta=20, n_seg=40)`	Malha triangulada de um tubo ao longo de uma polilinha, com a fração de comprimento de cada vértice.
`elbow_streamlines`	função	`(v_bulk, n_r=3, n_t=8)`	Linhas pelo cotovelo e trechos vizinhos em alguns raios, com a velocidade da lei de potência 1/7.
`loop_scene`	função	`(run, diagnosis, *, n_frames=24, title='')`	Cena animada de uma corrida monitorada: pressões nas paredes, veredito nos sensores e HUD com a evidência.

`pinneapple_systems.digital_twin.product`

Camada de comissionamento do gêmeo como produto: contrato (`TwinSpec`), contrato do modelo, estimador EnKF de estado latente, preflight + backtest, evidência em arquivo e política de deriva; reexporta tudo.

`pinneapple_systems/digital_twin/product/__init__.py` · 83 linhas · testes: 24

`pinneapple_systems.digital_twin.product.assimilation`

Estimador do gêmeo: EnKF da plataforma sobre o estado latente que o modelo não conhece (incrustação, espessura, viés), com o próprio modelo como operador de observação e banda de 95 % pela dispersão do ensemble. [sem teste direto] — coberto via `pinneapple_systems.digital_twin.product`.

`pinneapple_systems/digital_twin/product/assimilation.py` · 418 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`TwinEstimate`	classe	`(theta, theta_std, innovation, n_obs)`	Um instante de análise: estado latente, sua dispersão, inovação e número de observações.
`LatentStateTwin`	classe	`(model, spec, *, n_ens=64, inflation=1.02, seed=0) -> None`	Estima o estado latente de um ativo a partir da spec e do modelo; dá banda e média nos sensores de hold-out, tendência e projeção de cruzamento de limite.

`pinneapple_systems.digital_twin.product.commission`

Os dois portões antes de o gêmeo rodar: `preflight` (checagens baratas do contrato e do histórico) e `backtest` (replay do histórico medido contra sensores nunca assimilados).

`pinneapple_systems/digital_twin/product/commission.py` · 588 linhas · testes: 24

nome	tipo	assinatura	o que faz
`read_timestamp`	função	`(frame) -> float`	Lê o carimbo de tempo de um frame, sob qualquer das chaves convencionais.
`read_readings`	função	`(frame, spec) -> dict[str, float]`	Valores de sensores presentes num frame, ignorando chaves que não são sensores.
`preflight`	função	`(spec, history) -> CheckReport`	Checa se o gêmeo pode ser comissionado (unidades, observabilidade, canal travado, cobertura da faixa) antes de gastar um backtest.
`backtest`	função	`(spec, model, history, *, n_ens=64, inflation=1.02, seed=0, warmup_steps=0, calibrate_fra…)`	Repassa o histórico pelo gêmeo e mede contra os sensores de hold-out (erro, viés, cobertura da banda, antecedência de alarme).

`pinneapple_systems.digital_twin.product.evidence`

Contêineres da evidência: itens e relatório de checklist, métricas por campo com sua fonte, relatório de rastreamento e o arquivo `TwinEvidence` com a lista do que não foi verificado; não calcula física. [sem teste direto]

`pinneapple_systems/digital_twin/product/evidence.py` · 284 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`CheckItem`	classe	`(check_id, status, title, detail, why, fix, source)`	Uma linha do checklist: id, status, título, detalhe, porquê, correção e fonte.
`CheckReport`	classe	`(title, items)`	Checklist ordenado com veredito: qualquer `FAIL` bloqueia o comissionamento.
`FieldMetrics`	classe	`(field_name, unit, n, rmse, mae, bias, p95_abs, coverage, span, window_s, coverage_raw, b…)`	Concordância entre gêmeo e realidade num campo, em observações nunca assimiladas, com o número de comparações e a janela.
`TrackingReport`	classe	`(metrics, gates, breaches, alarm_lead_s, forecast_error_s, final_latent, n_steps)`	Métricas por campo, veredito dos portões, violações e antecedência dos alarmes.
`TwinEvidence`	classe	`(asset_id, spec_digest, model_id, created_unix, preflight, tracking, decisions, unverifie…)`	Tudo o que um gêmeo comissionado produz num arquivo, ligado ao digest da spec, incluindo o que não foi verificado.

`pinneapple_systems.digital_twin.product.model`

Contrato do modelo `model(X, theta) -> {campo: array}` e o adaptador `BiasModel` para encaixar substitutos que ignoram `theta`. [sem teste direto]

`pinneapple_systems/digital_twin/product/model.py` · 155 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`TwinModelError`	classe	`()`	Erro quando o modelo não cumpre o contrato do gêmeo.
`BiasModel`	classe	`(model, field_names) -> None`	Envolve um `model(X)` simples para cumprir o contrato, usando `theta` como viés de saída.
`as_array_dict`	função	`(out, field_names) -> dict[str, np.ndarray]`	Normaliza a saída de um modelo para `{campo: array (n,) float32}`.
`ensure_twin_model`	função	`(model, field_names, n_latent)`	Devolve um chamável que cumpre o contrato, envolvendo o modelo quando preciso.

`pinneapple_systems.digital_twin.product.policy`

Política de deriva: consistência da banda de 95 % do gêmeo numa janela móvel convertida em modo NORMAL → WATCH → ALARM → RECALIBRATE com histerese; nunca re-treina sozinho. [sem teste direto]

`pinneapple_systems/digital_twin/product/policy.py` · 166 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`TwinMode`	classe	`()`	Enum dos modos do gêmeo: NORMAL, WATCH, ALARM, RECALIBRATE.

nome	tipo	assinatura	o que faz
DriftPolicy	classe	(window, target_coverage, hard_break, max_sustained, recalibrate_after, hysteresis)	Limiares da escada de consistência (janela, cobertura alvo, quebra forte, sustentação, recalibração, histerese).
DriftMonitor	classe	(policy, _window, _mode, _off_run, _alarm_run, _ok_run, decisions)	Consistência móvel da banda do gêmeo convertida em modo com histerese; registra as decisões.

pinneapple_systems.digital_twin.product.runner

Superfície do produto: commission (preflight + backtest + evidência), run_live, describe (padrões industriais) e export_scene (cena pinneapple_twin3d). [sem teste direto] — coberto via pacote product.

pinneapple_systems/digital_twin/product/runner.py · 346 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
RunResult	classe	(n_steps, n_observed, n_holdout, final_mode, coverage, latent, modes_seen, decisions)	O que aconteceu na execução: passos, observados, hold-out, modo final, cobertura, estado latente e decisões.
commission	função	(spec, model, history, *, n_ens=64, inflation=1.02, seed=0, warmup_steps=0, calibrate_fra…)	Roda preflight e backtest e devolve o arquivo de evidência; determinístico para uma semente.
run_live	função	(spec, model, frames, *, n_ens=64, inflation=1.02, seed=0, policy=None, evidence=None, th…)	Assimila uma sequência de frames, monitora a consistência e registra o que o gêmeo decidiu.
describe	função	(spec, latent, fields=None)	Descreve o gêmeo para os padrões industriais de standards (AAS, DTDL, NGSI-LD, Ditto, OPC UA).
export_scene	função	(spec, frames, folder, *, mesh=None, field_on_mesh=None, times=None) -> str	Escreve uma cena pinneapple_twin3d do gêmeo: sensores com séries medidas e envelopes (malha opcional via trimesh).

pinneapple_systems.digital_twin.product.spec

O contrato do gêmeo (TwinSpec): campos, sensores, modelo, portões de aceitação e um digest; carrega e grava YAML (PyYAML opcional). [sem teste direto]

pinneapple_systems/digital_twin/product/spec.py · 438 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
SpecError	classe	()	Erro quando a spec não é utilizável (afirmação faltando, contradição, campo inobservável).
FieldSpec	classe	(name, unit, kind, lo, hi, alarm_above, alarm_below)	Uma grandeza que o gêmeo estima: nome, unidade, tipo, faixa e limites de alarme.
SensorSpec	classe	(sensor_id, unit, field, lo, hi, at, max_gap_s, noise_sigma, holdout, driver)	Uma medição disponível ao gêmeo, com envelope, ruído, se é hold-out e se é driver.
ModelSpec	classe	(model_id, version, accuracy_floor, validity)	O modelo por trás do gêmeo, sua versão, a precisão que promete e a validade.
AcceptanceGates	classe	(max_rmse_frac, max_bias_frac, min_coverage, min_alarm_lead_s)	Limiares que decidem se o gêmeo entra em operação (erro e viés relativos, cobertura mínima, antecedência mínima de alarme).
TwinSpec	classe	(asset_id, asset_name, asset_type, fields, sensors, model, asset_unit, coord_names, sampl…)	Definição completa e checável de um gêmeo; tem digest, leitura/escrita em dict e YAML e consultas sobre sensores e campos.

pinneapple_systems.digital_twin.prognostics

Manutenção preditiva: estimativa de vida útil remanescente (RUL) por tendência linear ou exponencial de um indicador de saúde, com intervalo de confiança e aviso de extrapolação pouco confiável.

pinneapple_systems/digital_twin/prognostics.py · 628 linhas · testes: 15

nome	tipo	assinatura	o que faz
`HealthIndicatorHistory`	classe	`(timestamps, values, source)`	Série temporal escalar de um indicador de saúde/degradação, montada de arrays ou de um `DigitalTwin`.
`RULEstimate`	classe	`(estimated_rul, rul_confidence_interval, failure_time_estimate, degradation_model_used, f…)`	Resultado da estimativa de RUL: vida remanescente, intervalo de confiança, tempo de falha, modelo usado, qualidade do ajuste e se a extrapolação é confiável.
`estimate_rul`	função	`(health_indicator_series, timestamps=None, failure_threshold=None, model='linear', confid…)`	Ajusta uma tendência linear ou exponencial ao indicador de saúde e extrapola até o limiar de falha.
`estimate_rul_from_twin`	função	`(twin, failure_threshold, *, detector=None, sensor_id=None, field_name=None, n=None, mode…)`	Atalho que estima a RUL direto de um `DigitalTwin` vivo (escores de anomalia ou um campo observado).

pinneapple_systems.digital_twin.realtime_pipeline

Pipeline de inferência em tempo real que impõe tempo de inferência ≤ período dos dados, com políticas `warn`, `skip`, `throttle` ou `raise` e estatísticas de latência. [sem teste]

`pinneapple_systems/digital_twin/realtime_pipeline.py` · 470 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`LatencyBudgetExceeded`	classe	`()`	Erro levantado quando a inferência passa do período dos dados e a política é `raise`.
`LatencyStats`	classe	`(n_frames, n_skipped, n_budget_violations, mean_inference_ms, max_inference_ms, budget_ms…)`	Estatísticas de latência: frames, descartes, violações, média e máximo de inferência, orçamento e utilização.
`RealtimePipelineConfig`	classe	`(data_period, latency_policy, buffer_maxlen, latency_alpha, throttle_min_points, throttle…)`	Configuração do pipeline: período, política de latência, buffer, suavização, throttling, dispositivo e AMP.
`RealtimePipeline`	classe	`(model, data_period, field_names, coord_names=None, *, config=None, on_state=None, on_lat…)`	Pipeline que envolve um modelo substituto, recebe leituras por `push` e mantém o último estado dentro do orçamento de latência.

pinneapple_systems.digital_twin.recession

Gêmeo de rio por recessão de vazão de base: calibra Q0 e k no passado recente de um posto, prevê o presente e sinaliza um desvio sustentado (veredito `DIVERGENCE_DETECTED`, `MATCHES_FORECAST` ou `NO_CLEAN_RECESSION`).

`pinneapple_systems/digital_twin/recession.py` · 195 linhas · testes: 8

nome	tipo	assinatura	o que faz
`recession`	função	`(q0, k_per_hour, hours) -> float | np.ndarray`	Curva de recessão Q0·exp(-k·t), com t em horas.
`fit_recession`	função	`(hours, values) -> dict[str, float]`	Ajusta Q0 e k por mínimos quadrados em ln Q e devolve R² e média/desvio dos resíduos.
`forecast`	função	`(calibration, hours) -> np.ndarray`	Avalia a curva calibrada em novas horas.
`detect_divergence`	função	`(hours, actual, predicted, residual_mean, residual_std, n_sigma=N_SIGMA, sustained_minute…)`	Acha a primeira excursão sustentada do resíduo além de n desvios da calibração.
`run_twin`	função	`(timestamps, values, *, stage_timestamps=None, stage_values=None, calibration_hours=CALIB…)`	Pipeline completo numa série: separa calibração e comparação, calibra, prevê, compara e dá o veredito; levanta ValueError se a série for curta.
`synthetic_recession`	função	`(hours=96.0, step_minutes=15.0, q0=2000.0, k_per_hour=0.004, noise=0.005, bump_at_hour=No…)`	Registro sintético de posto (recessão exponencial com ruído e degrau opcional) para testes e demos offline.

pinneapple_systems.digital_twin.signal_reconstruction

Reconstrução de sinais limpos a partir das observações do gêmeo: espectral (FFT), SSA, wavelets, interpolação e via modelo físico. [sem teste]

`pinneapple_systems/digital_twin/signal_reconstruction.py` · 587 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ReconstructionResult`	classe	`(sensor_id, field_name, method, timestamps, signal, raw_timestamps, raw_values, quality, …)`	Saída de uma reconstrução: sensor, campo, método, tempos e sinal reconstruídos, dados brutos e métricas de qualidade.
`SignalReconstructor`	classe	`(twin_or_observations, *, default_method='ssa', default_n_out=512) -> None`	Atalho que reconstrói sinais a partir do histórico de observações de um `DigitalTwin` (ou de uma lista de `Observation`).
`reconstruct_spectral`	função	`(timestamps, values, *, n_harmonics=10, out_times=None, min_points=8) -> Tuple[np.ndarray…)`	Reconstrução por FFT: ajusta os harmônicos dominantes e avalia numa grade densa.
`reconstruct_ssa`	função	`(timestamps, values, *, window=0, n_components=5, out_times=None) -> Tuple[np.ndarray, np…)`	Reconstrução por SSA: decompõe e mantém os componentes dominantes (supressão de ruído).
`reconstruct_wavelet`	função	`(timestamps, values, *, wavelet='db4', level=4, threshold_mode='soft', threshold_sigma_mu…)`	Remoção de ruído por limiar suave de wavelets (requer PyWavelets).
`reconstruct_interpolate`	função	`(timestamps, values, *, kind='cubic', out_times=None, n_out=512) -> Tuple[np.ndarray, np.…)`	Preenchimento de lacunas e reamostragem por interpolação SciPy (linear, cúbica, PCHIP etc.).
`reconstruct_model_based`	função	`(timestamps, values, *, model, sensor_coord, coord_names, field_name, field_index=0, out_…)`	Usa um modelo físico treinado (PyTorch) para reconstruir a série num ponto do espaço consultando `model(x, y, t)`.

pinneapple_systems.digital_twin.standards

Interoperabilidade: descreve o gêmeo uma vez e publica em AAS (BaSyx), DTDL v3, NGSI-LD, Eclipse Ditto e um servidor OPC UA (asyncua).

`pinneapple_systems/digital_twin/standards.py` · 322 linhas · testes: 31

nome	tipo	assinatura	o que faz
`TwinProperty`	classe	`(name, value, unit, kind, description, writable)`	Uma propriedade do gêmeo: nome, valor, unidade, tipo (parâmetro, telemetria, estado…), descrição e se é gravável.
`TwinModel`	classe	`(name, family, status, compatibility, violated, probable_cause)`	Um modelo por trás do gêmeo e sua validade física no ponto de operação atual (pode vir de uma auditoria).
`TwinDescription`	classe	`(id, name, asset_type, description, manufacturer, properties, models, relationships)`	Descrição neutra de um gêmeo (id, nome, tipo, propriedades, modelos, relações) usada por todos os exportadores.
`OpcUaTwinServer`	classe	`(desc, endpoint=…, namespace='urn:pinneapple:twins')`	Serve o gêmeo por OPC UA (asyncua) numa thread: um objeto por gêmeo, uma variável por propriedade; `update` grava valores.
`to_aas`	função	`(desc, as_json=True)`	Monta um Asset Administration Shell (metamodelo v3) com o SDK Eclipse BaSyx e devolve o JSON ou o objeto.
`to_dtdl`	função	`(desc, version=1) -> dict[str, Any]`	Gera uma Interface DTDL v3 com Properties, Telemetry e Relationships.
`to_ngsi_ld`	função	`(desc, context=…, observed_at=None) -> dict[str, Any]`	Gera uma entidade NGSI-LD com Property e Relationship.
`to_ditto_thing`	função	`(desc, namespace='org.pinneapple') -> dict[str, Any]`	Gera um Thing do Eclipse Ditto com atributos e features.

pinneapple_systems.digital_twin.state

Estruturas de estado: `Observation` (leitura bruta de um sensor) e `SystemState` (estimativa fundida com covariância, resíduos e histórico). [sem teste direto]

`pinneapple_systems/digital_twin/state.py` · 137 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Observation`	classe	`(timestamp, sensor_id, values, coords, metadata)`	Uma observação bruta de um sensor: tempo, id, valores, coordenadas e metadados.
`SystemState`	classe	`(timestamp, fields, coords, covariance, residuals, metadata, history)`	Estimativa corrente do sistema: campos, coordenadas, covariância, resíduos e histórico; converte de/para vetor plano.

pinneapple_systems.digital_twin.twin

Classe central `DigitalTwin`: envolve um modelo treinado, consome fluxos de sensores, faz assimilação (nenhuma, EKF ou EnKF), detecta anomalias e mantém o estado vivo; PyTorch é opcional.

`pinneapple_systems/digital_twin/twin.py` · 650 linhas · testes: 151

nome	tipo	assinatura	o que faz
`DigitalTwinConfig`	classe	`(device, batch_size, use_amp, update_interval, obs_queue_maxsize, max_history, assimilati…)`	Configuração do `DigitalTwin`: dispositivo, lote, AMP, intervalo de atualização, fila, histórico, assimilação e parâmetros de anomalia.
`DigitalTwin`	classe	`(model, field_names, coord_names=None, *, config=None, assimilation_filter=None) -> None`	Gêmeo digital que funde um modelo substituto treinado com dados de sensores vivos; gerencia fluxos, laço de atualização, callbacks, predição e histórico.
`build_digital_twin`	função	`(model, field_names, *, coord_names=None, device='cpu', update_interval=0.5, anomaly_z_th…)`	Fábrica de conveniência para `DigitalTwin` com as opções mais comuns.

pinneapple_systems.excitation

Sinais de excitação de atuador em NumPy (degrau, rampa, senoide, escada aleatória, PRBS, chirp, dente de serra, passeio aleatório suavizado, pulsos gaussianos, AM de dois tons) para identificação de sistemas e treino de surrogates; separa famílias de treino e famílias reservadas (`TRAIN_FAMILIES`, `HELDOUT_FAMILIES`).

`pinneapple_systems/excitation.py` · 152 linhas · testes: 6

nome	tipo	assinatura	o que faz
`shape_step`	função	`(tau, t0=0.3, **_)`	Degrau unitário em `tau = t0`.
`shape_ramp`	função	`(tau, t1=0.8, **_)`	Rampa linear de 0 em `tau = 0` até 1 em `tau = t1`, depois constante.
`shape_sinusoid`	função	`(tau, freq=2.0, phase=0.0, **_)`	Senoide `0.5 + 0.5 sin(2 pi freq tau + phase)`.
`shape_piecewise_constant`	função	`(tau, rng=None, n_seg=5, **_)`	Escada aleatória com `n_seg` níveis uniformes entre bordas aleatórias ordenadas.
`shape_prbs`	função	`(tau, rng=None, dwell=0.08, **_)`	Sequência binária pseudoaleatória (PRBS) em {0, 1}, trocando de nível só nos instantes do relógio `dwell`.
`shape_chirp`	função	`(tau, f0=0.5, f1=6.0, **_)`	Chirp linear com frequência instantânea de `f0` a `f1` ao longo de `tau`.
`shape_sawtooth`	função	`(tau, freq=3.0, **_)`	Dente de serra crescente `(freq * tau) mod 1`.
`shape_smoothed_random_walk`	função	`(tau, rng=None, **_)`	Passeio aleatório gaussiano suavizado por média móvel e reescalado para [0, 1].
`shape_gaussian_bumps`	função	`(tau, rng=None, n_bumps=4, width=0.05, **_)`	Soma de `n_bumps` gaussianas com centros e amplitudes aleatórios, normalizada pelo pico.
`shape_am_two_tone`	função	`(tau, f_carrier=5.0, f_mod=0.7, **_)`	Modulação em amplitude: produto de uma portadora e de uma moduladora senoidais.
`rate_limit`	função	`(series, tau_steps) -> np.ndarray`	Filtro passa-baixa de primeira ordem que limita a taxa de variação do sinal (constante de tempo em amostras).
`actuator_series`	função	`(shape, n_steps, *, lo=0.0, hi=1.0, seed=0, rate_limit_steps=0, **kwargs) -> np.ndarray`	Amostra uma forma (pelo nome em `SHAPES` ou callable) em `n_steps` pontos, escala para `[lo, hi]` e opcionalmente aplica `rate_limit`.

pinneapple_systems.operations

Operações: rodar modelos físicos validados e substitutos ao lado de dados de planta. Uma planta comum (tanque aquecido) e três ferramentas sobre ela: runtime de gêmeo, detecção física de anomalias e controle preditivo, todas falando `pinneapple_data.contracts`.

`pinneapple_systems/operations/__init__.py` · 94 linhas · testes: 24

pinneapple_systems.operations.anomaly

Detecção física de anomalias em séries de sensores: quatro famílias (estatística robusta, AR + CUSUM, balanços de conservação, resíduos de modelo físico) calibradas numa janela declarada normal, fundidas em eventos com severidade e explicações, mais um benchmark rotulado de falhas.

`pinneapple_systems/operations/anomaly.py` · 573 linhas · testes: 14

nome	tipo	assinatura	o que faz
`BalanceRule`	classe	`(name, accumulation, inflow, outflow, units, description)`	Lei de conservação sobre colunas: d(acumulação)/dt = entrada - saída, com unidades e descrição.
`DetectorConfig`	classe	`(normal_until, sensors, methods, balances, constants, window, threshold, ar_order, cusum_…)`	O que rodar e quão estrito ser: fim da janela normal, sensores, métodos, balanços, constantes, janela, limiar, ordem AR, parâmetros CUSUM.
`Channel`	classe	`(method, name, variables, residual, centre, spread, units, description, score, threshold)`	Uma saída de detector: série de resíduos, centro e dispersão da janela normal e escore normalizado.
`Event`	classe	`(start, end, i0, i1, severity, peak_ratio, methods, channels, variables, explanations, ev…)`	Evento de anomalia fundido: intervalo, severidade, razão de pico, métodos, canais, variáveis, explicações e evidência.
`DetectionResult`	classe	`(t, channels, flags, events, config, normal_samples)`	Resultado de `detect`: tempos, canais, flags, eventos, configuração e amostras normais.
`LabelledInterval`	classe	`(kind, start, end, target)`	Intervalo anormal rotulado `[start, end)` lido dos dados, com a mesma interface de um `Fault` da planta.
`tank_balance_rules`	função	`() -> list[BalanceRule]`	Balanços de massa e energia do tanque aquecido como `BalanceRule`.
`tank_constants`	função	`(p=None) -> dict[str, float]`	Constantes do tanque (área, densidade, cp, UA) usadas nas fórmulas dos balanços.
`windowed_balance`	função	`(t, acc, net_in, w) -> np.ndarray`	Resíduo de balanço numa janela: variação da acumulação dividida pelo tempo menos a média trapezoidal do líquido de entrada.
`severity_of`	função	`(ratio) -> str`	Severidade pela razão pico/limiar: baixa, média, alta ou crítica.
`detect`	função	`(data, config, model=None, model_states=(), model_inputs=()) -> DetectionResult`	Roda os detectores configurados sobre uma tabela (DataFrame ou dict com `t`) e devolve canais, flags e eventos.
`benchmark_faults`	função	`(t0=7200.0) -> list[Fault]`	As falhas do benchmark depois de um período inicial sem falha, com magnitudes pequenas de propósito.
`fault_benchmark`	função	`(seed=0, t0=7200.0, dt=10.0) -> tuple[dict[str, np.ndarray], list[Fault]]`	Uma corrida rotulada: operação normal inicial e depois as falhas do benchmark separadas por períodos normais.
`intervals_from_labels`	função	`(t, labels) -> list[LabelledInterval]`	Converte uma coluna de rótulos em intervalos contíguos de anomalia.
`score_events`	função	`(result, faults, grace=600.0, evaluate_from=None) -> dict`	Escores ponto a ponto e por evento contra os intervalos verdadeiros de falha.
`run_benchmark`	função	`(seeds=(101, 102, 103), methods_sets=None, window=12, threshold=5.0) -> dict`	Pontua cada família de detectores sozinha e todas juntas em corridas rotuladas separadas, sem ajustar limiares na parte com falha.

pinneapple_systems.operations.contracts

Converte modelos e estudos de operações para `pinneapple_data.contracts`: modelos viram `ModelRef` (com métricas de hold-out e SHA-256) e estudos de gêmeo, anomalia e controle viram `StudyResult` com checagens do que rodou.

`pinneapple_systems/operations/contracts.py` · 192 linhas · testes: 14

nome	tipo	assinatura	o que faz
save_model	função	(model, path, name, *, app='', dataset=None, trust_decision='UNCHECKED') -> ModelRef	Grava um modelo de gêmeo ou ControlledOpInf em JSON e devolve seu ModelRef (também gravado ao lado).
load_model	função	(ref, root='.')	Abre o modelo apontado por um ModelRef, conferindo o SHA-256.
check	função	(name, passed, value=None, threshold=None, evidence='', detail='', warn=False) -> CheckRe…)	Cria um CheckRecord; passed=None significa que a checagem não rodou (NOT_RUN).
new_id	função	(prefix) -> str	Gera um id curto com prefixo e parte aleatória (uuid).
twin_study	função	(runtime, *, app='twin', model_ref=None, title='') -> StudyResult	Converte a execução de um gêmeo num StudyResult (estimativas, alertas, checagens de validade, revalidações).
anomaly_study	função	(result, *, app='anomaly', benchmark=None, title='') -> StudyResult	Converte uma detecção (e opcionalmente um benchmark) num StudyResult.
control_study	função	(comparison, *, app='control', title='') -> StudyResult	Converte uma comparação de controladores num StudyResult, uma linha por controlador.

pinneapple_systems.operations.control

Controle do tanque aquecido: MPC linear e não linear contra PID e regras com histerese, em malha fechada na mesma planta simulada, só em modo simulação/recomendação (nada é enviado à planta); CasADi é opcional.

pinneapple_systems/operations/control.py · 658 linhas · testes: 14

nome	tipo	assinatura	o que faz
ControlSpec	classe	(dt, horizon, setpoints, h_bounds, T_max, q_in_bounds, Q_bounds, dq_max, dQ_max, h_scale,…)	Objetivos, restrições e limites dos atuadores do controle (passo, horizonte, setpoints, limites, pesos).
ControlScenario	classe	(duration, x0, T_in_steps, T_amb, faults, noise, seed)	Lado da planta num estudo em malha fechada: duração, estado inicial, perturbações, falhas e ruído, comum a todos os controladores.
PIDBaseline	classe	(spec, params=None, kp_h=0.02, ki_h=0.0001, kp_T=60000.0, ki_T=400.0)	Baseline PID: malha de nível na bomba e de temperatura no aquecedor, com feed-forward e anti-windup.
RuleBasedBaseline	classe	(spec, band_h=0.05, band_T=0.5, q_low=0.004, q_high=0.01, Q_on=600000.0)	Baseline por regras com histerese: bomba em duas vazões e aquecedor liga/desliga.
LinearMPC	classe	()	MPC com linearização sucessiva: um QP no horizonte resolvido com SLSQP.
NonlinearMPC	classe	(spec, params=None, model=None, backend='auto')	MPC não linear nos balanços do tanque (CasADi/IPOPT se disponível, senão SciPy) ou sobre qualquer modelo com step.
make_estimator	função	(spec, params=None, seed=0) -> TwinRuntime	Estimador de estado lido pelos MPCs: gêmeo EnKF do tanque com vazamento não medido e eficiência do aquecedor.
run_closed_loop	função	(controller, spec, scenario, params=None) -> dict	Simula um controlador num cenário e devolve trajetórias, planos (MPC) e métricas de desempenho.
performance	função	(tr, spec) -> dict	Métricas de uma trajetória: erros integrados, violações de restrição, energia, curso dos atuadores e tempo de solução.
compare_controllers	função	(spec=None, scenario=None, params=None, controllers=…, surrogate=None) -> dict	Roda os controladores nomeados (pid, rules, lmpc, nmpc, smpc) no mesmo cenário.
recommend	função	(estimate, reading, spec=None, params=None, t=0.0, method='nmpc') -> dict	Propõe as próximas ações para uma pessoa revisar, a partir da estimativa e da leitura atual; nada é enviado.

pinneapple_systems.operations.exchanger

Segunda planta de referência: trocador água/água em contracorrente com incrustação, usado pela camada de comissionamento de digital_twin.product; gera um histórico claramente sintético. [sem teste direto] — usado em tests/test_digital_twin_product.py via pacote.

pinneapple_systems/operations/exchanger.py · 158 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`OperatingPoint`	classe	`(t_hi, t_ci, m_h, m_c)`	Ponto de operação do trocador: temperaturas de entrada e vazões quente e fria.
`PlantConfig`	classe	`(ua_clean, flow_sensitivity, tau_fouling_s, rf_final)`	O trocador como construído, incluindo o erro de modelo com que o gêmeo tem de conviver.
`exchanger_model`	função	`(X, theta, *, ua_clean) -> dict[str, np.ndarray]`	`model(X, theta)` dado ao gêmeo: temperaturas de saída a partir do ponto de operação, com a resistência de incrustação em `theta`.
`plant_outlets`	função	`(op, rf, cfg) -> tuple[float, float]`	A planta real: o modelo mais um UA dependente da vazão que o modelo não conhece.
`fouling_at`	função	`(t_s, cfg) -> float`	Incrustação assintótica no tempo: rápida no início, depois patamar.
`generate_history`	função	`(hours=72.0, sampling_s=300.0, *, seed=7, cfg=None, op=None, t_start=1767225600.0) -> lis…)`	Histórico sintético e determinístico da planta com incrustação, oscilação de vazão e ruído de instrumento.

pinneapple_systems.operations.live

[experimental no OPC UA] Roda um `TwinRuntime` como serviço: fonte simulada via MQTT (broker embutido ou externo), MQTT externo, pushes REST ou um poller OPC UA experimental; também faz replay sobre uma tabela gravada.

`pinneapple_systems/operations/live.py` · 251 linhas · testes: 11 · palavras-chave no código (heurística; o texto acima diz o status real): experimental

nome	tipo	assinatura	o que faz
`LiveTwin`	classe	`(twin, *, source='simulated', topic=None, broker_host='127.0.0.1', broker_port=0, embedde…)`	Gêmeo vivo de um ativo: liga fonte (simulada, MQTT, REST ou OPC UA experimental), consumidor e armazenamento; permite injetar falhas e tirar snapshot.
`make_twin`	função	`(model, *, asset='tank-1', store=None, rules=None, sigma_obs=None, sigma_process=None, se…)`	Cria um `TwinRuntime` do tanque aquecido com os níveis de ruído dos seus sensores.
`replay`	função	`(table, model, *, asset='tank-1', store=None, rules=None, forecast_steps=60, revalidate=T…)`	Roda um gêmeo sobre uma tabela gravada e devolve histórico, alertas, previsão final e, se pedido, re-pontuação do modelo.
`demo_record`	função	`(duration=7200.0, dt=10.0, seed=4, faults=None) -> dict`	Corrida gravada do tanque simulado com um vazamento na segunda metade (exemplo offline do app de gêmeo).
`default_model`	função	`(kind='lumped', dt=10.0)`	Modelo padrão do gêmeo: só o `LumpedTankModel`; outro tipo levanta ValueError.

pinneapple_systems.operations.plant

Tanque aquecido de buffer: planta concentrada (0-D) com balanços de massa e energia, sensores com ruído, falhas e resíduos de balanço calculados de sinais medidos.

`pinneapple_systems/operations/plant.py` · 290 linhas · testes: 25

nome	tipo	assinatura	o que faz
`TankParams`	classe	`(area, k_out, ua, rho, cp, h_min, h_max, q_in_max, q_max_heater)`	Parâmetros físicos do tanque aquecido (SI, temperaturas em °C).
`Fault`	classe	`(kind, start, end, magnitude, target)`	Falha ativa em `[start, end)` segundos; `target` indica o sensor para falhas de sensor.
`Scenario`	classe	`(duration, dt, x0, T_in, T_amb, q_in, Q)`	Cenário de operação: como entradas e perturbações evoluem no tempo.
`SensorNoise`	classe	`(sigma)`	Desvio-padrão do ruído branco de cada sensor.
`TankSimulator`	classe	`(params=None, x0=(1.2, 25.0), faults=(), noise=None, seed=0)`	Simulador passo a passo do tanque com sensores e falhas, para malha aberta, malha fechada ou fonte de demo.
`tank_rhs`	função	`(x, u, d, p, eta=1.0, valve=1.0, q_leak=0.0) -> np.ndarray`	Derivadas [dh/dt, dT/dt] do tanque para estado, entradas e perturbações (vetorizado).

nome	tipo	assinatura	o que faz
`tank_step`	função	`(x, u, d, p, dt, substeps=4, **fault_kw) -> np.ndarray`	Avança o tanque `dt` segundos com RK4 clássico.
`steady_state`	função	`(p, q_in, Q, T_in, T_amb) -> np.ndarray`	Equilíbrio [h, T] para entradas constantes (forma fechada).
`inputs_for_steady_state`	função	`(p, h, T, T_in, T_amb) -> np.ndarray`	Entradas [q_in, Q] que mantêm o tanque num [h, T] dado (forma fechada).
`measure`	função	`(truth, t, faults, noise, rng, frozen) -> dict[str, float]`	Leituras de um passo: valor verdadeiro + ruído, depois falhas de sensor.
`simulate`	função	`(scenario, params=None, faults=(), noise=None, seed=0) -> dict[str, np.ndarray]`	Corrida em malha aberta de um cenário; devolve sensores medidos, estados verdadeiros e a falha ativa.
`demo_scenario`	função	`(duration=3600.0, dt=10.0, seed=0) -> Scenario`	Perfil realista de turno: degraus de vazão, aquecedor por agenda e temperatura de entrada variando devagar; determinístico por semente.
`mass_balance_residual`	função	`(t, h, q_in, q_out, area) -> np.ndarray`	Resíduo do balanço de massa a partir de sinais medidos; um vazamento o torna igual a menos a vazão vazada.
`energy_balance_residual`	função	`(t, h, T, q_in, q_out, Q, T_in, T_amb, p) -> np.ndarray`	Resíduo do balanço de energia em forma conservativa, em W.
`with_params`	função	`(p, **changes) -> TankParams`	Cópia de `TankParams` com campos trocados.

pinneapple_systems.operations.rom

Operator inference com entradas: modelo reduzido polinomial em tempo discreto ajustado por regressão ridge, validado em trajetórias inteiras separadas; serializa em JSON e tem passo CasADi opcional.

`pinneapple_systems/operations/rom.py` · 178 linhas · testes: 11

nome	tipo	assinatura	o que faz
`ControlledOpInf`	classe	`(state_names, input_names, dt, W, x_mean, x_scale, v_mean, v_scale, quadratic, bilinear, …)`	Modelo reduzido ajustado (matrizes, escalas, caixa de treino, validação); `step`, `rollout`, `casadi_step` e JSON.
`fit_controlled_opinf`	função	`(trajectories, state_names, input_names, dt, *, holdout=1, ridge=1e-06, quadratic=True, b…)`	Ajusta o modelo em todas menos as últimas trajetórias e valida nelas (erros de um passo e de rollout).
`residual_sigma`	função	`(model, trajectories) -> dict[str, float]`	Desvio dos resíduos de um passo por estado: o ruído de processo que um filtro com este modelo deve assumir.

pinneapple_systems.operations.series

Cadeia ponta a ponta num ativo (Campaign Planner → corridas → Surrogate Studio → incerteza → otimizador → gêmeo), cada passagem um arquivo de contrato; diz para cada passo se o app existe em `apps/` ou rodou como substituto de biblioteca.

`pinneapple_systems/operations/series.py` · 339 linhas · testes: 14

nome	tipo	assinatura	o que faz
`operating_space`	função	`() -> ParameterSpace`	`ParameterSpace` das entradas de operação do tanque excitadas pela campanha.
`app_available`	função	`(pattern, apps_dir) -> str | None`	Nome do diretório `apps/` de um app da série, se existir neste checkout.
`run_series_flow`	função	`(workdir, *, seed=0, n_runs=12, steps_per_run=12, hold=300.0, dt=20.0, T_min=28.0, q_min=…)`	Roda a cadeia inteira e grava cada contrato em `workdir`; devolve um resumo em JSON.

pinneapple_systems.operations.twin

Runtime de gêmeo digital: EnKF com parâmetros no estado, previsões com bandas 5-95 %, alertas por regras e teste de consistência (NIS), monitoramento de validade e revalidação, histórico em SQLite ou Postgres (psycopg opcional).

`pinneapple_systems/operations/twin.py` · 602 linhas · testes: 25

nome	tipo	assinatura	o que faz
`LumpedTankModel`	classe	`(params=None, dt=10.0, estimate=('q_leak', 'eta'))`	Balanços de massa e energia do tanque como modelo de gêmeo, opcionalmente com parâmetros (vazamento, eficiência) no estado.
`SurrogateModel`	classe	`(rom)`	Um `ControlledOpInf` usado como modelo de gêmeo (sem parâmetros no estado).
`AlertRules`	classe	`(limits, param_limits, param_sigmas, nis_probability, nis_consecutive, forecast_steps, dr…)`	Limiares de alerta: limites de estado e de parâmetro, teste NIS, horizonte de previsão, deriva e revisão.
`Alert`	classe	`(t, kind, severity, message, data, cleared)`	Um alerta: tempo, tipo, severidade, mensagem, dados e se foi limpo.
`TwinStore`	classe	`(url=':memory:')`	Histórico de leituras, estimativas, alertas e revalidações (SQLite por padrão, Postgres opcional).
`TwinRuntime`	classe	`(model, obs_names, sigma_obs, sigma_process, *, x0=None, sigma0=None, n_ens=48, inflation…)`	Assimila leituras uma a uma no modelo e mantém estado, previsões, alertas e validade do gêmeo.
`model_from_dict`	função	`(d)`	Reconstrói um modelo de gêmeo do JSON (`lumped_tank` ou `controlled_opinf`).

pinneapple_systems.process_components

Modelos físicos de operações unitárias para gêmeos de processo: gás real, turbomáquinas, válvulas, trocadores, tubulações, vigas, cinética de reação, fluidos não newtonianos, tensões, stick-slip, fadiga e um motor genérico de equações; módulos independentes entre si.

`pinneapple_systems/process_components/__init__.py` · 272 linhas · testes: 58

pinneapple_systems.process_components.beam_nonlinear_fem

Viga Euler-Bernoulli geometricamente não linear (Von Karman), estática, por Newton-Raphson em passos de carga, reutilizando o motor transiente de `pinneapple_simulation` no limite estático (massa e amortecimento zero).

`pinneapple_systems/process_components/beam_nonlinear_fem.py` · 172 linhas · testes: 6

nome	tipo	assinatura	o que faz
`StaticNonlinearBeamResult`	classe	`(z, u_m, w_m, theta_rad, bending_moment, bending_stress, load_step_deflections, iteration…)`	Resultado da viga não linear: deslocamentos, rotação, momento, tensão, deflexão por passo de carga, iterações e passos não convergidos.
`solve_nonlinear_beam_static`	função	`(boundary, L_m, A_m2, I_m4, E_Pa, fiber_distance_m, axial_dist_load_N_per_m=0.0, transver…)`	Resolve a viga Von Karman estática por Newton-Raphson em passos de carga e recupera a tensão de flexão na fibra externa.

pinneapple_systems.process_components.beam_statics

Soluções fechadas de viga Euler-Bernoulli linear estática para casos padrão de balanço e biapoiada (cargas uniforme, parcial e triangular), mais propriedades de seção retangular e Von Mises.

`pinneapple_systems/process_components/beam_statics.py` · 196 linhas · testes: 19

nome	tipo	assinatura	o que faz
`BeamResult`	classe	`(x, deflection, slope, bending_moment, shear_force)`	Resultado da viga linear: posições, deflexão, inclinação, momento fletor e cortante.
`rectangular_section_properties`	função	`(width_m, height_m) -> tuple[float, float]`	Momento de inércia I e primeiro momento Q de uma seção retangular.
`von_mises_stress_rectangular_section`	função	`(bending_moment, shear_force, I, Q, width_m, height_m) -> np.ndarray`	Tensão de Von Mises numa seção retangular a partir de momento e cortante.
`solve_beam`	função	`(case, x, L, E, I, q0, load_extent=None) -> BeamResult`	Despacha para a solução fechada do caso de viga pedido (com `load_extent` nos casos de carga parcial).

pinneapple_systems.process_components.control_valve

Dimensionamento de válvula de controle pela IEC 60534-2-1 (compressível com fator de expansão e bloqueio, e incompressível simples), característica inerente, vazamento de sede e atraso do atuador. [sem teste direto] — testado via `tests/test_process_components.py` pelo pacote.

`pinneapple_systems/process_components/control_valve.py` · 115 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ValveSpec`	classe	`(name, Cv_max_us, xT, characteristic, equal_percentage_R, actuator_time_constant_s, leaka…)`	Especificação da válvula: Cv máximo, xT, característica, atuador e vazamento de sede.
`ValveFlowResult`	classe	`(mass_flow_kg_s, choked, x, Y, Cv_effective_us)`	Resultado de vazão compressível: vazão mássica, se bloqueou, x, Y e Cv efetivo.
`installed_cv`	função	`(travel_fraction, spec) -> float`	Cv numa fração de curso pela característica inerente (sem vazamento).
`effective_cv`	função	`(travel_fraction, spec, leakage_cv_us_override=None) -> float`	Cv que de fato passa vazão, incluindo o piso de vazamento de sede (pode sobrescrever o vazamento).
`compressible_mass_flow`	função	`(Cv_us, P1_Pa, P2_Pa, rho1_kg_m3, k_isentropic, xT) -> ValveFlowResult`	Vazão mássica compressível pela IEC 60534-2-1 com fator de expansão e bloqueio, em SI.
`incompressible_mass_flow`	função	`(Cv_us, P1_Pa, P2_Pa, rho_kg_m3) -> float`	Vazão de líquido simples w = Cv_SI·√(ΔP·ρ), sem modelo de flashing/cavitação.
`actuator_response_rhs`	função	`(travel_actual, travel_command, spec) -> float`	Derivada do curso real para o atraso de primeira ordem do atuador.

pinneapple_systems.process_components.curved_path_geometry

Gerador de trajetória curva "build-and-hold" (vertical, ganho de inclinação com curvatura constante, trecho mantido) com profundidade vertical e afastamento horizontal; só em um plano (2D). A marca de placeholder do inventário vem da frase "não implementado aqui" sobre o caso 3D.

`pinneapple_systems/process_components/curved_path_geometry.py` · 113 linhas · testes: 7 · palavras-chave no código (heurística; o texto acima diz o status real): placeholder/stub citado no docstring

nome	tipo	assinatura	o que faz
`CurvedPathProfile`	classe	`(s_m, inclination_deg, tvd_m, hd_m, n_bins, vertical_length_m, total_length_m, bin_width_…)`	Perfil discretizado da trajetória: comprimento medido, inclinação, profundidade vertical, afastamento e larguras.
`inclination_at_depth`	função	`(s_m, vertical_length_m, build_length_m, final_inclination_deg) -> np.ndarray`	Inclinação em cada profundidade medida: vertical, linear no ganho e mantida depois.
`build_and_hold_profile`	função	`(total_length_m, build_length_m, final_inclination_deg, hold_length_m, bin_width_m) -> Cu…)`	Discretiza a trajetória build-and-hold e integra profundidade vertical e afastamento horizontal.
`circular_arc_tvd_hd`	função	`(s_within_build_m, build_length_m, final_inclination_deg) -> tuple`	Profundidade e afastamento em forma fechada no trecho de ganho (arco circular), para conferir a integração.

pinneapple_systems.process_components.explicit_equation_system

Motor genérico e isolado de "modelo como equações nomeadas": avalia expressões com AST seguro, ordena por dependência (detecta ciclos) e calibra parâmetros limitados por mínimos quadrados.

`pinneapple_systems/process_components/explicit_equation_system.py` · 393 linhas · testes: 171

nome	tipo	assinatura	o que faz
`ExplicitEquationError`	classe	`()`	Erro do motor de equações explícitas.
`Definition`	classe	`(name, lhs, rhs, rhs_symbols)`	Uma equação já analisada: nome, símbolo do lado esquerdo, expressão e símbolos usados.
`AnalysisResult`	classe	`(valid, errors, evaluation_order)`	Resultado da validação: se é válido, erros e ordem de avaliação.

nome	tipo	assinatura	o que faz
ParameterSpec	classe	(symbol, value, fit, min, max)	Um parâmetro: símbolo, valor, se deve ser ajustado e limites.
CalibrationResult	classe	(fitted_parameters, start, bounds, n_evaluations, final_scaled_residual_ssr, termination_…)	Resultado da calibração: parâmetros ajustados, ponto inicial, limites, avaliações, resíduo final e mensagem.
safe_eval	função	(expr, env) -> Any	Avalia uma expressão por AST permitindo só operações e funções de uma lista branca.
build_definitions	função	(inputs, parameters, intermediates, outputs, equations) -> tuple[list[Definition], list[s…)	Valida e monta as definições a partir de dicts {name, equation}, acumulando todos os erros.
evaluation_order	função	(definitions) -> list[Definition]	Ordenação topológica das definições com detecção de ciclos.
evaluate	função	(input_values, parameter_values, order) -> dict[str, Any]	Avalia todas as definições na ordem dada e devolve o ambiente completo de símbolos.
analyze	função	(inputs, parameters, intermediates, outputs, equations) -> AnalysisResult	Validação a seco: monta e ordena o sistema e reporta erros, sem calcular números.
calibrate	função	(input_columns, parameters, order, measured_rows, target_symbols) -> CalibrationResult	Calibração não linear limitada (Trust Region Reflective) dos parâmetros com fit=True contra dados medidos, reavaliando o sistema completo.

pinneapple_systems.process_components.fatigue_analysis

Vida em fadiga por curva S-N (Wöhler), correção de tensão média de Goodman e dano acumulado pela regra de Miner.

pinneapple_systems/process_components/fatigue_analysis.py · 108 linhas · testes: 8

nome	tipo	assinatura	o que faz
MinerDamageResult	classe	(N_f, damage_this_period, cumulative_damage, remaining_life_fraction)	Resultado de Miner: vida N_f, dano do período, dano acumulado e fração de vida restante.
sn_curve_cycles_to_failure	função	(sigma_a, A, m) -> ArrayLike	Ciclos até a falha pela curva S-N para uma amplitude totalmente reversa.
goodman_equivalent_amplitude	função	(sigma_alt, sigma_mean, S_ut) -> ArrayLike	Amplitude equivalente totalmente reversa pela linha de Goodman modificada.
goodman_safety_ratio	função	(sigma_alt, sigma_mean, S_e, S_ut) -> ArrayLike	Razão de segurança de Goodman (< 1 dentro da região segura).
miners_rule_damage	função	(sigma_alt, sigma_mean, n_cycles, A_sn, m_sn, S_ut, prior_damage_fraction=0.0) -> MinerDa…)	Vida S-N corrigida por Goodman, dano do período e dano acumulado de Miner (limitado a [0, 1]).

pinneapple_systems.process_components.heat_exchanger

Trocador de calor por efetividade-NTU (contracorrente e paralelo, formas fechadas) e resposta transiente de capacitância concentrada. [sem teste direto] — testado via pacote em tests/test_process_components.py.

pinneapple_systems/process_components/heat_exchanger.py · 113 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
HeatExchangerSpec	classe	(name, UA_design_W_K, design_dP_Pa, design_m_dot_hot_kg_s, design_rho_hot_kg_m3, thermal_…)	Especificação do trocador: UA de projeto, queda de pressão e vazão de projeto, massa térmica, capacidade do lado frio e arranjo.
HeatExchangerResult	classe	(Q_W, T_hot_out_K, T_cold_out_K, effectiveness, NTU, dP_Pa, UA_effective_W_K)	Resultado: calor trocado, temperaturas de saída, efetividade, NTU, queda de pressão e UA efetivo.
steady_state	função	(spec, m_dot_hot_kg_s, T_hot_in_K, cp_hot_J_kgK, T_cold_in_K, rho_hot_kg_m3, UA_multiplie…)	Solução estacionária efetividade-NTU, com multiplicador de UA (incrustação).
transient_rhs	função	(T_hot_out_current_K, spec, m_dot_hot_kg_s, T_hot_in_K, cp_hot_J_kgK, T_cold_in_K, rho_ho…)	Derivada da temperatura de saída quente, relaxando para o valor estacionário com a constante de tempo da massa térmica.

pinneapple_systems.process_components.non_newtonian_pipe_flow

Reologia Herschel-Bulkley, Reynolds generalizado e fator de atrito de Metzner-Reed, e integração 1D da queda de pressão num conduto inclinado. A marca de placeholder do inventário vem da frase "não implementado aqui" sobre a correlação de Dodge-Metzner.

pinneapple_systems/process_components/non_newtonian_pipe_flow.py · 155 linhas · testes: 10 · palavras-chave no código (heurística; o texto acima diz o status real): placeholder/stub citado no docstring

nome	tipo	assinatura	o que faz
PressureProfile	classe	(s_m, tvd_m, P_Pa, rho_eq_kg_m3, rho_kg_m3, friction_factor, reynolds_number)	Perfil de pressão ao longo do conduto: posição, profundidade, pressão, densidade equivalente, atrito e Reynolds.
herschel_bulkley_stress	função	(tau_y, K, n, gamma_dot) -> ArrayLike	Tensão de Herschel-Bulkley τ = τ_y + K·γ̇^n.
effective_viscosity	função	(tau_y, K, n, gamma_dot) -> ArrayLike	Viscosidade efetiva τ/γ̇.
generalized_reynolds_number	função	(rho, v, D, K, n) -> ArrayLike	Reynolds generalizado de Metzner-Reed.
metzner_reed_friction_factor	função	(Re_gen) -> ArrayLike	Fator de atrito laminar (16/Re) ou turbulento (Blasius/Metzner-Reed).
pressure_gradient	função	(rho, v, D_h, f, inclination_rad=np.pi / 2, g=9.81) -> ArrayLike	Gradiente de pressão: termo hidrostático mais atrito.
integrate_pressure_profile	função	(length_m, v_m_s, D_h_m, K_Pa_sn, n_flow_index, P_inlet_Pa, rho_profile=1000.0, inclinati…)	Integra a pressão ao longo do conduto por Euler explícito, com densidade e inclinação constantes ou funções da posição.

pinneapple_systems.process_components.pipe_network_1d

Escoamento 1D de gás real num único segmento de tubo: modo estacionário rápido (marcha célula a célula) e modo transiente de volumes finitos (momento quase estacionário, continuidade transiente), atrito de Colebrook-White.

pinneapple_systems/process_components/pipe_network_1d.py · 290 linhas · testes: 151

nome	tipo	assinatura	o que faz
PipeSpec	classe	(name, length_m, diameter_m, roughness_m, elevation_change_m, ambient_temperature_K, wall…)	Especificação do tubo: comprimento, diâmetro, rugosidade, desnível, troca de calor com o ambiente, células e multiplicador de atrito.
SteadyProfilePoint	classe	(x_m, P_Pa, T_K, rho_kg_m3, G_kg_m2s, mach)	Um ponto do perfil estacionário: posição, pressão, temperatura, densidade, fluxo mássico e Mach.
PipeState	classe	(P_Pa, T_K)	Estado transiente do tubo: pressões e temperaturas por célula.
TransientPipe	classe	(spec, gas)	Solver transiente de volumes finitos de um tubo de gás real, inicializável pelo perfil estacionário.
colebrook_white_f	função	(Re, relative_roughness) -> float	Fator de atrito de Darcy: laminar abaixo de Re 2300, senão Colebrook-White por Newton.
face_reynolds_number	função	(G_kg_m2s, D_m, viscosity_Pa_s) -> float	Número de Reynolds numa face a partir do fluxo mássico.
rapid_steady_state_profile	função	(spec, gas, m_dot_kg_s, P_in_Pa, T_in_K) -> list[SteadyProfilePoint]	Perfil estacionário marchando célula a célula a partir da entrada, sem integração no tempo.

pinneapple_systems.process_components.pipe_stress_mechanics

Tensões combinadas num tubo esbelto (Lamé, torção, flexão por curvatura, Von Mises triaxial), flambagem de Euler, flambagem de haste confinada (Paslay-Dawson), amplificação P-Delta e ciclo de flexão rotativa.

pinneapple_systems/process_components/pipe_stress_mechanics.py · 209 linhas · testes: 16

nome	tipo	assinatura	o que faz
`ConstrainedRodBucklingResult`	classe	`(F_critical_sinusoidal, F_critical_helical, mode)`	Cargas críticas senoidal e helicoidal de Paslay-Dawson e o modo.
`RotatingBendingCycleResult`	classe	`(theta_rad, sigma_von_mises, sigma_alternating, sigma_mean)`	Envelope de Von Mises numa rotação e as tensões alternada e média para fadiga.
`lame_hoop_stress_outer`	função	`(p_internal, r_outer, r_inner) -> ArrayLike`	Tensão circunferencial de Lamé na parede externa.
`lame_hoop_stress_inner`	função	`(p_internal, r_outer, r_inner) -> ArrayLike`	Tensão circunferencial de Lamé na parede interna (máxima).
`torsional_shear_stress`	função	`(torque, r_outer, J) -> ArrayLike`	Cisalhamento de torção máximo T·c/J na fibra externa.
`bending_stress_from_curvature`	função	`(curvature, E, r_outer) -> ArrayLike`	Tensão de flexão na fibra externa a partir da curvatura.
`von_mises_triaxial`	função	`(sigma_axial, sigma_hoop, sigma_radial=0.0, tau=0.0) -> ArrayLike`	Von Mises triaxial com cisalhamento no plano axial-circunferencial.
`euler_critical_buckling_load`	função	`(E, I, length, end_condition_factor=1.0) -> float`	Carga crítica de Euler com fator de condição de extremidade.
`constrained_rod_buckling_load`	função	`(E, I, lateral_load_per_length, radial_clearance) -> ConstrainedRodBucklingResult`	Cargas críticas de Paslay-Dawson para haste confinada numa folga cilíndrica.
`classify_buckling_mode`	função	`(compressive_force, result) -> ArrayLike`	Classifica o modo: 0 reto, 1 senoidal, 2 helicoidal.
`beam_column_moment_amplification_factor`	função	`(compressive_force, E, I, length, end_condition_factor=1.0, max_ratio=0.98) -> ArrayLike`	Fator de amplificação P-Delta 1/(1 - P/P_cr), com P/P_cr limitado.
`rotating_bending_stress_cycle`	função	`(sigma_axial, sigma_bending_amplitude, sigma_hoop=0.0, tau=0.0, n_angles=36) -> RotatingB…)`	Envelope de Von Mises numa rotação completa sob flexão rotativa e tensões alternada e média.

`pinneapple_systems.process_components.polytropic_path`

Integração do caminho politrópico de gás real (compressão ou expansão) consistente com ASME PTC 10, por integração direta passo a passo. [sem teste direto] — testado via pacote em `tests/test_process_components.py`.

`pinneapple_systems/process_components/polytropic_path.py` · 196 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`PolytropicPathResult`	classe	`(inlet, outlet, polytropic_work_J_kg, polytropic_efficiency, isentropic_work_J_kg, isentr…)`	Resultado do caminho politrópico: estados de entrada e saída, trabalhos, eficiências, razão de pressão e expoente.
`solve_path_from_pressure_ratio`	função	`(inlet, gas, P2_Pa, polytropic_efficiency) -> PolytropicPathResult`	Integra o caminho politrópico até uma pressão de saída conhecida (compressão ou expansão).
`solve_path_from_work`	função	`(inlet, gas, polytropic_work_J_kg, polytropic_efficiency, *, max_pressure_ratio=6.0) -> P…)`	Integra até atingir um trabalho politrópico alvo e resolve a pressão de saída.

`pinneapple_systems.process_components.reaction_kinetics`

Motor genérico de redes de reação por ação de massas (constantes Arrhenius, polinomiais ou catalíticas, especiação por pH), integração rígida e transporte 1D advecção-dispersão-reação em grade periódica.

`pinneapple_systems/process_components/reaction_kinetics.py` · 257 linhas · testes: 164

nome	tipo	assinatura	o que faz
`Reaction`	classe	`(name, rate_fn, stoichiometry)`	Uma reação: nome, função de taxa e estequiometria.
`ReactionNetwork`	classe	`(species, reactions)`	Rede de espécies e reações; `rhs` dá dC/dt.

nome	tipo	assinatura	o que faz
`IntegrationResult`	classe	`(t, C, success)`	Resultado da integração: tempos, concentrações e sucesso.
`AdvectionDispersionReactionSolver`	classe	`(network, n_grid, length_m, velocity_m_s, dispersion_m2_s)`	Transporte 1D advecção-dispersão-reação numa grade periódica, com reações de uma `ReactionNetwork`.
`mass_action_rate`	função	`(k, orders) -> RateFn`	Constrói a taxa de ação de massas k·∏C^ordem, com k constante ou função de T, pH ou concentrações.
`arrhenius_rate_constant`	função	`(A, Ea_over_R) -> Callable[..., float]`	Constante de Arrhenius k(T); levanta erro se chamada sem temperatura.
`quadratic_in_T`	função	`(a, b, c) -> Callable[[float], float]`	Função quadrática em T (por exemplo pKa(T) ajustado).
`linear_combination_rate_constant`	função	`(terms) -> Callable[..., float]`	Constante efetiva como combinação linear de concentrações de catalisadores.
`acid_fraction`	função	`(pH, pKa) -> float`	Fração protonada de um par ácido-base monoprótico (Henderson-Hasselbalch).
`base_fraction`	função	`(pH, pKa) -> float`	Fração desprotonada, igual a 1 - `acid_fraction`.
`diprotic_fractions`	função	`(pH, pKa1, pKa2) -> tuple[float, float, float]`	Frações de Bjerrum de um sistema diprótico.
`integrate_network`	função	`(network, C0, t_eval, T_K=None, pH=None, *, method='Radau', rtol=1e-09, atol=1e-14) -> In…)`	Integra a rede (Radau por padrão), mantendo concentrações não negativas e levantando RuntimeError se o integrador falhar.

pinneapple_systems.process_components.real_gas_eos

Propriedades de misturas de gás real por equação de estado de Helmholtz (GERG-2008 via backend HEOS do CoolProp; REFPROP opcional), envelope de validade, condições padrão e derivadas por diferença central fora de grafos de autograd.

`pinneapple_systems/process_components/real_gas_eos.py` · 259 linhas · testes: 151

nome	tipo	assinatura	o que faz
`OutOfEnvelopeError`	classe	`()`	Erro quando um estado cai fora do envelope de validade declarado.
`GasComposition`	classe	`(components, mole_fractions)`	Mistura de composição fixa (componentes e frações molares que somam 1).
`ValidityEnvelope`	classe	`(P_min_Pa, P_max_Pa, T_min_K, T_max_K)`	Limites (P, T) opcionais; fora deles o estado levanta `OutOfEnvelopeError` em vez de extrapolar.
`GasState`	classe	`(P_Pa, T_K, rho_kg_m3, Z, h_J_kg, s_J_kgK, cp_J_kgK, cv_J_kgK, k_isentropic, speed_of_sou…)`	Estado termodinâmico completo em SI (P, T, ρ, Z, h, s, cp, cv, k, som, viscosidade, massa molar).
`StandardConditions`	classe	`(P_Pa, T_K)`	Condições padrão de referência (presets ISO 13443 e 60 °F americano).
`state_from_PT`	função	`(gas, P_Pa, T_K, *, backend='HEOS', specify_phase='gas', envelope=None) -> GasState`	Estado completo a partir de pressão e temperatura (CoolProp).
`state_from_Ph`	função	`(gas, P_Pa, h_J_kg, *, backend='HEOS', specify_phase='gas', envelope=None) -> GasState`	Estado completo a partir de pressão e entalpia (flash na EOS).
`state_from_Ps`	função	`(gas, P_Pa, s_J_kgK, *, backend='HEOS', specify_phase='gas', envelope=None) -> GasState`	Estado completo a partir de pressão e entropia (ponto isentrópico).
`standard_volumetric_flow_to_mass_flow`	função	`(q_std_m3_s, gas, std) -> float`	Converte vazão volumétrica em condições padrão para vazão mássica.
`mass_flow_to_standard_volumetric_flow`	função	`(m_dot_kg_s, gas, std) -> float`	Converte vazão mássica para vazão volumétrica em condições padrão.
`central_difference`	função	`(gas, P_Pa, T_K, attr, wrt, *, rel_step=0.0001) -> float`	Derivada de uma propriedade por diferença central, para usar fora de grafos de autograd.

pinneapple_systems.process_components.similarity_map

Utilitários de mapas adimensionais de turbomáquinas: coeficientes de vazão e altura, Mach de ponta, Reynolds da máquina, margens de surge e choke e velocidade necessária para uma altura.

pinneapple_systems/process_components/similarity_map.py · 194 linhas · testes: 8

nome	tipo	assinatura	o que faz
MapCoefficients	classe	(name, D2_m, n_stages, phi_ref_bep, psi_ref_bep, psi_shape_a, psi0, eta_p_max, eta_p_shap…)	Coeficientes de um mapa adimensional de turbomáquina (diâmetro, estágios, curvas de ψ e η, surge, choke, correções).
MapEvaluation	classe	(phi, psi, eta_p, Mu2, Re, surge_margin_frac, choke_margin_frac)	Avaliação do mapa num ponto: φ, ψ, η, Mach de ponta, Reynolds e margens de surge e choke.
derive_psi_shape_a	função	(psi0, phi_ref_bep, psi_ref_bep) -> float	Curvatura a de ψ(φ) = ψ0 - a·φ² que passa pelo ponto de melhor eficiência.
make_map	função	(name, D2_m, phi_ref_bep, psi_ref_bep, psi0, eta_p_max, eta_p_shape_b, phi_surge, phi_cho…)	Monta MapCoefficients derivando a curvatura da curva de altura.
flow_coefficient	função	(Q1_m3_s, N_rpm, coeffs) -> float	Coeficiente de vazão φ = Q1/(N·D2³).
tip_speed_m_s	função	(N_rpm, coeffs) -> float	Velocidade de ponta do impelidor.
tip_mach_number	função	(N_rpm, coeffs, speed_of_sound_m_s) -> float	Mach de ponta U2/a1.
machine_reynolds_number	função	(N_rpm, coeffs, kinematic_viscosity_m2_s) -> float	Reynolds da máquina U2·D2/ν1.
evaluate_map	função	(coeffs, Q1_m3_s, N_rpm, speed_of_sound_m_s, kinematic_viscosity_m2_s, Re_ref=None) -> Ma…)	Avalia o mapa numa vazão e rotação, com correções de Mach e Reynolds e margens.
polytropic_head_from_psi	função	(psi, N_rpm, coeffs) -> float	Altura politrópica total = estágios·ψ·U2².
required_speed_for_head	função	(target_head_J_kg, Q1_m3_s, coeffs, speed_of_sound_m_s, kinematic_viscosity_m2_s, *, N_mi…)	Rotação que produz uma altura alvo a vazão fixa, por bisseção.

pinneapple_systems.process_components.torsional_stickslip

Simulação por diferenças finitas da onda torcional 1D num eixo flexível acionado numa ponta e com atrito de Stribeck na outra (mecanismo de stick-slip), com índice de stick-slip.

pinneapple_systems/process_components/torsional_stickslip.py · 179 linhas · testes: 6

nome	tipo	assinatura	o que faz
StickSlipResult	classe	(t_s, z_m, omega_field_rad_s, end_omega_ts_rad_s, surface_torque_ts_N_m, stick_slip_index)	Resultado: tempos, posições, campo de velocidade angular, velocidade da ponta, torque na superfície e índice de stick-slip.
stribeck_friction_torque	função	(omega, T_static, T_kinetic, omega_breakout) -> float	Torque de atrito de Stribeck (enfraquece com a velocidade).
simulate_torsional_stickslip	função	(length_m, G_Pa, J_m4, rho_kg_m3, omega_set_rad_s, T_static_N_m, T_kinetic_N_m, omega_bre…)	Resolve a onda torcional 1D por diferenças finitas explícitas, da ponta acionada à ponta com atrito e inércia concentrada.

pinneapple_systems.process_components.water_hammer

Golpe de aríete numa adutora bombeada: ponto de operação estacionário e transiente de fechamento de válvula pelo método das características, mais fórmulas de Joukowsky e período de onda.

pinneapple_systems/process_components/water_hammer.py · 191 linhas · testes: 39

nome	tipo	assinatura	o que faz
Pipeline	classe	(length, diameter, wave_speed, roughness, z_in, z_out, nu, rho)	Adutora: comprimento, diâmetro, celeridade, rugosidade, cotas, viscosidade e densidade.
PumpCurve	classe	(shutoff_head, k, sump_head)	Curva da bomba de rotação fixa H = H0 - k·Q|Q| com carga de sucção.
OperatingPoint	classe	(flow, velocity, reynolds, friction_factor, pump_head, valve_head, friction_loss, valve_c…)	Ponto de operação estacionário: vazão, velocidade, Reynolds, atrito, cargas, Cv da válvula e tanque.

nome	tipo	assinatura	o que faz
`TransientResult`	classe	`(t, x, head, flow, tau, dt, courant, op, info)`	Resultado do transiente MOC: tempos, posições, carga, vazão, abertura, passo, Courant e informações.
`operating_point`	função	`(line, pump, *, valve_k, tank_head=0.0) -> OperatingPoint`	Vazão onde a curva da bomba encontra atrito (Colebrook) mais perda da válvula aberta.
`linear_closure`	função	`(closure_time, start=0.0)`	Lei de fechamento τ(t): 1 até o início, depois linear até 0.
`moc_valve_closure`	função	`(line, pump, op, *, closure_time, t_end, n_reaches=40, friction=True, closure_start=0.0, …)`	Transiente de fechamento de válvula pelo método das características a partir do ponto de operação.
`joukowsky_head_rise`	função	`(wave_speed, dv) -> float`	Sobrepressão de Joukowsky a·ΔV/g.
`wave_period`	função	`(length, wave_speed) -> float`	Período da oscilação 4L/a.

pinneapple_systems.sports

Física aplicada a esportes sobre dados públicos de rastreamento e eventos; por enquanto só futebol. [sem teste direto]

`pinneapple_systems/sports/__init__.py` · 2 linhas · **sem teste direto**

pinneapple_systems.sports.football

Futebol: análises com física sobre dados públicos (voo da bola, controle de campo, cinemática de jogadores, gols esperados); ver `docs/guides/football_physics.md`.

`pinneapple_systems/sports/football/__init__.py` · 3 linhas · testes: 16

pinneapple_systems.sports.football.ball

Física do voo da bola (gravidade, arrasto quadrático com crise de arrasto, sustentação de Magnus), problema inverso a partir de posições rastreadas, identificabilidade por Cramér-Rao e modelo de bola rolando para passes rasteiros.

`pinneapple_systems/sports/football/ball.py` · 464 linhas · testes: 16

nome	tipo	assinatura	o que faz
`Ball`	classe	`(mass, diameter)`	Bola: massa e diâmetro; área, constante de arrasto e Reynolds.
`FlightFit`	classe	`(model, x0, v0, cd, magnus, g, t0, params, names, cov, rmse, n, success, extra)`	Resultado de `fit_flight`: parâmetros, erros-padrão de Laplace, RMSE, BIC e previsão.
`Flight`	classe	`(match_id, t, xyz, detected)`	Um voo aéreo recortado de um rastreamento: partida, tempos, posições e detecção.
`cd_crisis`	função	`(re, cd_sub=0.45, cd_super=0.18, re_crit=260000.0, width=25000.0)`	Curva suave de crise de arrasto entre Cd subcrítico e supercrítico.
`simulate`	função	`(x0, v0, t, *, cd=0.0, magnus=(0.0, 0.0, 0.0), g=G, ball=BALL, cd_fn=None, dt=0.005) -> n…)`	Posições de uma bola lançada com arrasto, Magnus e gravidade (RK4).
`fit_flight`	função	`(t, xyz, model='drag', *, sigma=None, ball=BALL, g=G, observe=(0, 1, 2), cd_prior=None, l…)`	Ajusta um modelo de voo a posições rastreadas (pula linhas NaN), com perda robusta.
`predict`	função	`(fit, t, ball=BALL) -> np.ndarray`	Posições de um voo ajustado em novos tempos.
`kinematic_extrapolation`	função	`(t, xyz, t_new, degree=2) -> np.ndarray`	Baseline só de dados: polinômio por eixo extrapolado.
`extract_flights`	função	`(track, *, match_id=None, z_on=0.6, min_apex=2.5, min_duration=0.8, max_duration=4.5, max…)`	Recorta voos aéreos de um rastreamento de bola por critérios de altura, duração e velocidade.
`fisher_identifiability`	função	`(x0, v0, t, *, cd=0.25, magnus=(0.0, 0.0, 0.0), sigma=0.3, observe=(0, 1, 2), ball=BALL, …)`	Desvios de Cramér-Rao dos parâmetros do voo para um ruído de observação.
`rolling_fit`	função	`(t, xy, *, ball=BALL) -> dict[str, float]`	Ajusta resistência ao rolamento e arrasto a um passe rasteiro em 2-D.
`rolling_travel_time`	função	`(distance, v0, decel, cd=0.0, ball=BALL)`	Tempo para uma bola rolando cobrir uma distância (infinito se parar antes).

nome	tipo	assinatura	o que faz
travel_time	função	(distance, *, speed=None, v0=None, decel=None)	Tempo de viagem da bola: velocidade constante ou bola rolando.

pinneapple_systems.sports.football.ball_study

Estudos do voo da bola: recuperação em voos simulados, identificabilidade 3-D vs 2-D, auditoria de gravidade em voos reais e previsão hold-out contra baselines de dados. [sem teste direto]

pinneapple_systems/sports/football/ball_study.py · 235 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
synthetic_flights	função	(n=40, *, noise=0.2, fps=10.0, seed=0, g=B.G) -> list[tuple[B.Flight, dict]]	Chutes simulados com arrasto, rotação e ruído, com a verdade de cada um.
synthetic_recovery	função	(n=40, *, noise=0.2, seed=0) -> dict[str, Any]	Ajusta modelos aos voos sintéticos e mede erro e cobertura dos intervalos de 95 %.
identifiability	função	(speed=22.0, elevation_deg=25.0, *, fps=25.0, sigma=0.2, duration=1.6) -> dict[str, Any]	Compara os desvios de Cramér-Rao de um passe visto em 3-D e em 2-D.
flights_from_skillcorner	função	(match_ids, root=None, **kw) -> list[B.Flight]	Voos aéreos de partidas SkillCorner.
gravity_audit	função	(flights, *, max_rmse=0.6, workers=1) -> dict[str, Any]	Ajusta a gravidade livre em cada voo para checar a consistência física dos dados.
holdout_prediction	função	(train, test, *, frac=0.5, train_fracs=(0.3, 0.5, 0.7), seed=0, workers=1) -> dict[str, A…)	Observa o início de cada voo de teste e prevê o resto; erros por modelo.
summarise	função	(res) -> list[dict[str, Any]]	Tabela por modelo de holdout_prediction, melhor aterrissagem primeiro.

pinneapple_systems.sports.football.data

Dados públicos de futebol (Metrica, SkillCorner, StatsBomb) baixados em tempo de execução, em cache fora do git, com SHA-256 de cada arquivo num manifesto.

pinneapple_systems/sports/football/data.py · 282 linhas · testes: 16

nome	tipo	assinatura	o que faz
MetricaMatch	classe	(game, t, period, players, names, ball, events, fps, gk)	Uma partida de amostra Metrica em metros (jogadores, bola, eventos, fps, goleiros).
cache_dir	função	(root=None) -> Path	Pasta de cache do futebol (criada se faltar).
manifest	função	(root=None) -> dict[str, dict[str, Any]]	Manifesto URL → caminho, SHA-256 e bytes de cada arquivo baixado.
fetch	função	(url, rel, root=None, *, timeout=120.0) -> Path	Baixa uma URL uma vez para o cache e registra o SHA-256.
metrica_match	função	(game, root=None) -> MetricaMatch	Carrega a partida 1 ou 2 da amostra Metrica (baixa uma vez).
skillcorner_matches	função	(root=None) -> list[dict]	Lista das partidas abertas do SkillCorner (id, times, data).
skillcorner_ball	função	(match_id, root=None) -> dict[str, np.ndarray]	Rastreamento da bola de uma partida SkillCorner (NaN quando falta).
statsbomb_shots	função	(seasons=None, root=None, *, max_matches=None) -> list[dict]	Todos os chutes com freeze frame 360 nas temporadas StatsBomb pedidas.

pinneapple_systems.sports.football.kin_study

Estudos de cinemática: suavização com verdade conhecida, incerteza de potência metabólica em dados reais e previsão de trajetória com e sem limites físicos. [sem teste direto]

pinneapple_systems/sports/football/kin_study.py · 175 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
synthetic_runs	função	(n=20, T=60.0, fps=25.0, *, seed=0, v_cap=9.0, a_cap=6.0, tau=1.0, sigma_a=3.0)	Trajetórias verdadeiras de jogadores por aceleração Ornstein-Uhlenbeck limitada.
synthetic_smoothing	função	(noise=(0.05, 0.2), fps=(25.0, 10.0), *, n=20, seed=0) -> list[dict[str, Any]]	RMSE de velocidade, aceleração e potência metabólica por método de suavização contra a verdade.
match_load	função	(match, *, team='Home', max_frames=None) -> dict[str, Any]	Potência metabólica de um time numa partida Metrica por jogador e método, e plausibilidade.
prediction_samples	função	(match, horizon, *, every=1.0, history=1.0) -> dict[str, np.ndarray]	Amostras para previsão de posição num horizonte, com estados de Kalman causal.
prediction_study	função	(train, test, horizon, *, sizes=(100, 300, 1000, 3000, None), seeds=(0, 1, 2)) -> dict[st…)	Erro final de deslocamento de cada preditor treinado em subconjuntos de tamanhos diferentes.

pinneapple_systems.sports.football.kinematics

Cinemática de jogadores: suavização Kalman-RTS com ruído ajustado por verossimilhança, plausibilidade física, potência metabólica (modelo de inclinação equivalente) e previsão de curto prazo com e sem limites físicos.

pinneapple_systems/sports/football/kinematics.py · 300 linhas · testes: 16

nome	tipo	assinatura	o que faz
MLPredictor	classe	(residual_of, project, max_iter, seed, _models)	Preditor de deslocamento só de dados: árvores com gradient boosting (scikit-learn) sobre history_features.
kalman_smooth	função	(xy, dt, *, sigma_pos=0.1, jerk=30.0, causal=False, max_gap=1.0) -> dict[str, np.ndarray]	Posição, velocidade e aceleração suavizadas por Kalman-RTS (ou filtro causal).
fit_noise	função	(xy, dt, sigmas=(0.01, 0.02, 0.05, 0.1, 0.2), jerks=(5.0, 10.0, 20.0, 40.0, 80.0))	Ruído de medição e densidade de jerk por máxima verossimilhança marginal (busca em grade).
finite_differences	função	(xy, dt) -> dict[str, np.ndarray]	Diferenças centrais das posições brutas (baseline sem suavização).
savgol_derivatives	função	(xy, dt, window=0.5, order=2) -> dict[str, np.ndarray]	Posição, velocidade e aceleração por Savitzky-Golay.
plausibility	função	(vel, acc, *, v_max=V_MAX, a_max=A_MAX) -> dict[str, float]	Fração de amostras acima dos limites humanos de velocidade ou aceleração.
minetti_cost	função	(i)	Custo energético de correr numa inclinação (Minetti et al. 2002).
metabolic_power	função	(speed, accel_fwd, *, terrain=1.0) -> np.ndarray	Potência metabólica em W/kg a partir de velocidade e aceleração (inclinação equivalente).
forward_acceleration	função	(vel, acc) -> np.ndarray	Componente da aceleração ao longo da velocidade.
reachable_radius	função	(speed0, horizon, *, v_max=V_MAX, a_max=A_MAX)	Maior distância alcançável num horizonte (limite físico de qualquer previsão).
predict_constant_velocity	função	(pos, vel, horizon)	Posição após o horizonte a velocidade constante.
predict_limited	função	(pos, vel, acc, horizon, *, v_max=V_MAX, a_max=A_MAX, tau=0.5, dt=0.04)	Previsão física: aceleração limitada que decai e velocidade limitada.
project_reachable	função	(pos, vel, pred, horizon, *, v_max=V_MAX, a_max=A_MAX)	Projeta previsões no conjunto alcançável.
history_features	função	(pos_hist, ball_rel=None) -> np.ndarray	Features para o preditor de ML: posições passadas relativas e, opcionalmente, a bola.
displacement_errors	função	(pred, true, pos=None, vel=None, horizon=None) -> dict[str, float]	Erro final médio e percentil 90, e fração de previsões fora do conjunto alcançável.

pinneapple_systems.sports.football.metrics

Escores de previsões binárias probabilísticas: log-loss, Brier, AUC, curvas de confiabilidade, ECE e bootstrap pareado de diferenças.

pinneapple_systems/sports/football/metrics.py · 78 linhas · testes: 16

nome	tipo	assinatura	o que faz
`log_loss`	função	`(p, y) -> float`	Log-loss de probabilidades binárias.
`brier`	função	`(p, y) -> float`	Escore de Brier.
`auc`	função	`(p, y) -> float`	AUC ROC pela fórmula de soma de postos (Mann-Whitney).
`reliability`	função	`(p, y, bins=10, strategy='quantile') -> dict[str, np.ndarray]`	Probabilidade média prevista, frequência observada e contagem por faixa.
`ece`	função	`(p, y, bins=10) -> float`	Erro de calibração esperado com faixas por quantil.
`score`	função	`(p, y) -> dict[str, float]`	Todos os escores de um modelo.
`paired_bootstrap`	função	`(p_a, p_b, y, metric='log_loss', n=2000, seed=0) -> dict[str, Any]`	Diferença de uma métrica entre dois modelos nos mesmos eventos com intervalo bootstrap de 95 %.

`pinneapple_systems.sports.football.pc_study`

Estudo de resultado de passes: controle de campo físico contra classificadores só de dados, com o mesmo conjunto de treino, e eficiência de dados. [sem teste direto]

`pinneapple_systems/sports/football/pc_study.py` · 131 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`physics_features`	função	`(ps, p) -> np.ndarray`	Features físicas de um passe: probabilidade de controle e tempos de interceptação menos tempo da bola.
`fit_models`	função	`(train, *, seed=0, which=…) -> dict[str, Any]`	Ajusta os modelos (parâmetros físicos ou estimadores) no conjunto de treino.
`predict_models`	função	`(models, ps) -> dict[str, np.ndarray]`	Probabilidades previstas por cada modelo ajustado num conjunto de passes.
`compare`	função	`(train, test, *, seed=0) -> dict[str, Any]`	Escores no teste, bootstrap pareado de log-loss contra o melhor modelo só de dados e parâmetros físicos ajustados.
`data_efficiency`	função	`(train, test, sizes=(25, 50, 100, 200, 400, None), seeds=(0, 1, 2), which=…) -> dict[str,…)`	Log-loss de teste de cada modelo treinado em subconjuntos aleatórios.
`with_params`	função	`(p, **kw) -> PC.PCParams`	Cópia de `PCParams` com campos trocados.

`pinneapple_systems.sports.football.pitch_control`

Controle de campo: probabilidade de o time atacante ganhar a bola num ponto, a partir da cinemática dos jogadores e do tempo de viagem da bola; extrai passes do Metrica e ajusta parâmetros por máxima verossimilhança.

`pinneapple_systems/sports/football/pitch_control.py` · 275 linhas · testes: 16

nome	tipo	assinatura	o que faz
`PCParams`	classe	`(tti, v_max, a_max, reaction, sigma, lam, kappa_def, gk_factor, ball, ball_speed, roll, d…)`	Parâmetros do controle de campo (velocidade e aceleração máximas, reação, incerteza, taxa de controle, goleiro, bola, horizonte).
`PassSet`	classe	`(att_pos, att_vel, def_pos, def_vel, att_gk, def_gk, ball, target, y, duration, meta)`	Conjunto de passes com o estado no chute (posições e velocidades dos 22 jogadores, bola, alvo, rótulo).
`time_to_intercept`	função	`(pos, vel, target, p) -> np.ndarray`	Tempo para jogadores chegarem a um alvo dada posição e velocidade.
`control_probability`	função	`(att_pos, att_vel, def_pos, def_vel, ball_pos, target, p=DEFAULT, att_gk=None, def_gk=Non…)`	Probabilidade de o time atacante controlar a bola no alvo (vetorizada).
`passes_from_metrica`	função	`(match, *, sigma_pos=0.1, jerk=30.0) -> PassSet`	Passes completados e interceptados de uma partida Metrica com estados de Kalman causal.
`fit_ball_travel`	função	`(ps, *, min_len=5.0) -> dict[str, Any]`	Ajusta o tempo de viagem da bola (rolando e velocidade constante) às durações observadas.

nome	tipo	assinatura	o que faz
fit_params	função	(ps, base, names=…, *, max_iter=200) -> tuple[PCParams, float]	Ajuste por máxima verossimilhança de parâmetros nomeados (Nelder-Mead em log).
geometric_features	função	(ps) -> tuple[np.ndarray, list[str]]	Features só de dados de um passe (distâncias, posições, contagens de defensores).
pitch_surface	função	(att_pos, att_vel, def_pos, def_vel, ball_pos, p=DEFAULT, *, nx=53, ny=34, pitch=(105.0, …)	Probabilidade de controle numa grade do campo para um quadro (para figuras).

pinneapple_systems.sports.football.plots

Figuras estáticas (matplotlib, PNG) dos estudos de futebol: campo, superfície de controle, voos 3-D, auditoria de gravidade, confiabilidade, eficiência de dados e erros de suavização. [sem teste]

pinneapple_systems/sports/football/plots.py · 224 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
color_of	função	(name) -> str	Cor fixa de um nome de modelo (cinza se desconhecido).
draw_pitch	função	(ax, length=105.0, width=68.0, color='#b9b7b0')	Desenha as linhas do campo em metros.
pitch_control_figure	função	(xs, ys, q, att_pos, att_vel, def_pos, def_vel, ball, target, path, title='')	Figura da superfície de controle com jogadores, bola e alvo.
flights_3d	função	(flights, path, title='')	Vista 3-D de voos observados e ajustados sobre o campo.
gravity_audit_figure	função	(audit, synthetic_g, path)	Figura da auditoria de gravidade.
reliability_figure	função	(rel, path, title='', models=None, lim=1.0)	Curvas de confiabilidade por modelo.
efficiency_figure	função	(rows, metric, path, title='', ylabel=None, models=None, flat=None)	Métrica de teste contra tamanho do treino por modelo, com faixa min-max.
smoothing_figure	função	(rows, path, metric='mp_rmse', ylabel='metabolic power RMSE [W/kg]')	Erros de suavização por método.
holdout_figure	função	(res, path, models=None)	Box plot dos erros de aterrissagem por modelo.

pinneapple_systems.sports.football.xg

Gols esperados com e sem física: baseline de geometria do chute contra features de tempo de interceptação calculadas do freeze frame 360 do StatsBomb.

pinneapple_systems/sports/football/xg.py · 239 linhas · testes: 16

nome	tipo	assinatura	o que faz
XGParams	classe	(v_shot, v_gk, reach_gk, reaction_gk, v_def, reach_def, reaction_def, n_mouth, n_path)	Parâmetros físicos do xG (velocidade do chute, do goleiro e dos defensores, alcance, reação, discretização).
shots_arrays	função	(shots) -> dict[str, Any]	Converte a tabela de chutes em arrays (posições em metros, rótulo, freeze frame).
standard_features	função	(A) -> tuple[np.ndarray, list[str]]	Features padrão de xG: distância, ângulo e contexto do chute.
physics_features	função	(A, p=None, *, chunk=200) -> tuple[np.ndarray, list[str]]	Features físicas: fração do gol aberta, margem do goleiro e bloqueadores.
freeze_geometry_features	função	(A) -> tuple[np.ndarray, list[str]]	Features geométricas do freeze frame sem física.
select_params	função	(A_train, grid_v_shot=(12.0, 16.0, 20.0, 25.0, 30.0), grid_v_gk=(3.0, 5.0, 7.0), folds=5)…)	Escolhe velocidades do chute e do goleiro por validação cruzada agrupada no treino.
compare	função	(A_train, A_test, *, params=None, seed=0) -> dict[str, Any]	Escores de teste dos modelos padrão, físicos e combinados, com o xG do StatsBomb como referência externa.
data_efficiency	função	(A_train, A_test, params, sizes=(100, 200, 400, 800, None), seeds=(0, 1, 2, 3, 4))	Log-loss de teste dos modelos padrão e padrão+física em subconjuntos do treino.

pinneapple_systems.time_series

Previsão de séries temporais de sinais físicos (antigo `pinneapple_timeseries`): o `__init__` reexporta specs, datamodule, catálogo, validação, auditoria, features, baselines, previsão adaptativa, wrappers clássicos, preparação, EDA, modelos, decomposição, tuning e visualização.

`pinneapple_systems/time_series/__init__.py` · 108 linhas · testes: 90

pinneapple_systems.time_series.adapters

Pacote vazio que agrupa adaptadores de forma. [sem teste]

`pinneapple_systems/time_series/adapters/__init__.py` · 1 linhas · **sem teste direto**

pinneapple_systems.time_series.adapters.shape

Adaptador `LastStepToHorizon`, que repete uma saída de um passo ao longo do horizonte. [sem teste]

`pinneapple_systems/time_series/adapters/shape.py` · 14 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`LastStepToHorizon`	classe	`(model, horizon)`	Módulo que repete a saída de um passo do modelo interno `horizon` vezes, para obter a forma (B, H, C).

pinneapple_systems.time_series.adaptive

Previsão adaptativa online: especialistas baratos (suavização exponencial, Holt, Holt-Winters, Theta, média móvel, AR ridge, regressor sobre defasagens) combinados ou alternados por pesos Fixed-Share calculados só com erros fora da amostra (prequencial), com intervalos conformais.

`pinneapple_systems/time_series/adaptive.py` · 447 linhas · testes: 9

nome	tipo	assinatura	o que faz
`SimpleExpSmoothing`	classe	`(alpha, level_)`	Especialista de suavização exponencial simples (previsão plana no nível).
`HoltDamped`	classe	`(alpha, beta, phi, level_, trend_)`	Especialista de Holt com tendência amortecida por `phi`.
`HoltWintersAdditive`	classe	`(season_length, alpha, beta, gamma, level_, trend_, seasonal_, n_)`	Especialista Holt-Winters aditivo com sazonalidade de comprimento `season_length`.
`Theta`	classe	`(alpha, level_, slope_, n_)`	Especialista Theta: suavização exponencial com deriva igual à metade da inclinação da regressão.
`MovingAverage`	classe	`(window, value_)`	Especialista que prevê a média das últimas `window` observações.
`RidgeAR`	classe	`(p, window, ridge, coef_, tail_)`	Especialista autorregressivo de ordem p com intercepto e regularização ridge, previsão recursiva.
`LagRegressorExpert`	classe	`(estimator, n_lags=12, refit_every=24, window=500)`	Transforma qualquer regressor tabular estilo scikit-learn em especialista sobre defasagens, com reajuste periódico.
`AdaptiveRun`	classe	`(origins, forecast, expert_forecasts, lower, upper, weights, active, expert_names, y)`	Resultado de `AdaptiveForecaster.run`: previsões, previsões por especialista, intervalos, pesos e especialista ativo por origem; calcula erros, cobertura e trocas.
`AdaptiveForecaster`	classe	`(experts, horizon=1, mode='combine', etas=(0.5, 2.0, 8.0), alphas=(0.0, 0.01, 0.05), loss…)`	Seleção e combinação online de previsores por pesos exponenciais Fixed-Share por horizonte, com intervalos conformais adaptativos.
`default_experts`	função	`(season_length=1) -> dict[str, Any]`	Conjunto diverso e barato de especialistas, com variantes de hiperparâmetros como especialistas separados.

pinneapple_systems.time_series.audit

Pacote de auditoria estatística de séries; reexporta `AuditReport`, `AuditSection` e `TSAuditor`. [sem teste direto]

pinneapple_systems/time_series/audit/__init__.py · 4 linhas · **sem teste direto**

pinneapple_systems.time_series.audit.report

Contêineres serializáveis (`AuditReport`, `AuditSection`) para os resultados dos diagnósticos. [sem teste]

pinneapple_systems/time_series/audit/report.py · 28 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`AuditSection`	classe	`(name, results)`	Seção nomeada de um relatório de auditoria, com dicionário de resultados.
`AuditReport`	classe	`(meta, sections)`	Contêiner serializável dos diagnósticos de uma série, organizado por seções.

pinneapple_systems.time_series.audit.tests

Testes diagnósticos de séries (ADF, KPSS, Phillips-Perron, ARCH, ACF/PACF, Chow, pontos de mudança via `ruptures`) que devolvem dicionários com `ok`/erro em vez de levantar exceção; depende de `statsmodels`. [sem teste]

pinneapple_systems/time_series/audit/tests.py · 315 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`TSAuditor`	classe	`(nlags, arch_lags)`	Roda a bateria de diagnósticos (estacionariedade, heterocedasticidade, autocorrelação, quebras de regime) e devolve um `AuditReport`.
`adf_test`	função	`(y, *, autolag='AIC', min_n=20) -> Dict[str, Any]`	Teste de raiz unitária Augmented Dickey-Fuller via `statsmodels`.
`kpss_test`	função	`(y, *, regression='c', nlags='auto', min_n=20) -> Dict[str, Any]`	Teste de estacionariedade KPSS via `statsmodels`.
`pp_test`	função	`(y, min_n=20) -> Dict[str, Any]`	Teste de Phillips-Perron; devolve erro explicativo se a versão do `statsmodels` não tiver a função.
`arch_test`	função	`(y, *, lags=12, min_n=30) -> Dict[str, Any]`	Teste ARCH de heterocedasticidade condicional.
`acf_pacf`	função	`(y, *, nlags=40, min_n=50) -> Dict[str, Any]`	Calcula autocorrelação e autocorrelação parcial até `nlags`.
`chow_test_mean_shift`	função	`(y, break_idx, min_n=20) -> Dict[str, Any]`	Teste de Chow simples para mudança de média num índice de quebra.
`changepoints_ruptures`	função	`(y, *, model='l2', max_breaks=5, min_n=50) -> Dict[str, Any]`	Detecção aproximada de múltiplos pontos de mudança com `ruptures`, se instalado.

pinneapple_systems.time_series.base

Tipos base: `TSOutput` (saída padrão com `y_hat`) e `TSModelBase` (`nn.Module` cujo `forward` levanta `NotImplementedError`). [sem teste direto]

pinneapple_systems/time_series/base.py · 26 linhas · **sem teste direto** · palavras-chave no código (heurística; o texto acima diz o status real): levanta `NotImplementedError` em algum caminho

nome	tipo	assinatura	o que faz
`TSOutput`	classe	`(y_hat, extras)`	Saída padrão dos modelos de séries: `y_hat` e um dicionário `extras`.
`TSModelBase`	classe	`()`	Base opcional (`nn.Module`) para modelos de séries; seu `forward` levanta `NotImplementedError`.

pinneapple_systems.time_series.baselines

Reexporta as baselines ingênuas (`NaiveForecaster`, `SeasonalNaiveForecaster`, `DriftForecaster`). [sem teste direto]

pinneapple_systems/time_series/baselines/__init__.py · 2 linhas · **sem teste direto**

pinneapple_systems.time_series.baselines.naive

Baselines sem aprendizado: último valor, sazonal ingênuo e deriva linear, com interface `fit`/`predict`. [sem teste direto]

`pinneapple_systems/time_series/baselines/naive.py` · 71 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`NaiveForecaster`	classe	`(last_value_)`	Prevê o último valor observado para todos os passos futuros.
`SeasonalNaiveForecaster`	classe	`(season_length, history_)`	Repete o valor da mesma estação no último ciclo.
`DriftForecaster`	classe	`(first_, last_, n_)`	Extrapola linearmente do primeiro ao último ponto de treino.

pinneapple_systems.time_series.classical_wrappers

Wrappers reais dos pacotes estatísticos clássicos: ARIMA/SARIMA de `statsmodels` e `pmdarima.auto_arima`, com intervalos de previsão.

`pinneapple_systems/time_series/classical_wrappers.py` · 244 linhas · testes: 10

nome	tipo	assinatura	o que faz
`StatsmodelsARIMAForecaster`	classe	`(order, seasonal_order, trend, enforce_stationarity, enforce_invertibility, fit_kwargs)`	Previsor ARIMA/SARIMA real sobre `statsmodels` SARIMAX, com intervalos e resumo.
`AutoARIMAForecaster`	classe	`(seasonal, m, max_p, max_q, max_d, max_P, max_Q, max_D, stepwise, suppress_warnings, auto…)`	Previsor sobre `pmdarima.auto_arima`, que escolhe a ordem automaticamente.

pinneapple_systems.time_series.csv_loader

Carrega um CSV de série temporal em tensores de treino/validação/teste (com preenchimento, normalização por canal e divisão cronológica) e gera um CSV sintético. [sem teste]

`pinneapple_systems/time_series/csv_loader.py` · 322 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`TimeSeriesScaler`	classe	`() -> None`	Normalizador z-score por canal: ajusta no treino e aplica a todas as partições.
`load_timeseries_csv`	função	`(path, *, target_cols=None, feature_cols=None, time_col=None, fill_method='ffill', normal…)`	Lê um CSV e devolve tensores de treino, validação e teste prontos para o PINNeAPPle.
`generate_synthetic_csv`	função	`(path, T=1000, seed=42, freq='D') -> Path`	Gera e salva um CSV sintético multivariado de série temporal.

pinneapple_systems.time_series.datamodule

`TSDataModule`: monta o dataset em janelas e loaders com divisões determinísticas e ordenadas no tempo (por índices ou holdout sequencial).

`pinneapple_systems/time_series/datamodule.py` · 80 linhas · testes: 151

nome	tipo	assinatura	o que faz
`TSDataModule`	classe	`(series, spec, batch_size, val_ratio, num_workers, pin_memory)`	Monta o dataset em janelas e os DataLoaders de treino/validação em ordem temporal.

pinneapple_systems.time_series.datasets

Pacote vazio que agrupa os datasets. [sem teste]

`pinneapple_systems/time_series/datasets/__init__.py` · 1 linhas · **sem teste direto**

pinneapple_systems.time_series.datasets.public

Séries reais de licença aberta baixadas de um commit fixo e verificadas por SHA-256 (câmbio USD/EUR diário, manchas solares anuais via `statsmodels`, agregados diários de vídeos em alta do YouTube EUA).

`pinneapple_systems/time_series/datasets/public.py` · 183 linhas · testes: 8

nome	tipo	assinatura	o que faz
`PublicSeries`	classe	`(name, time, values, source)`	Série pública carregada: nome, tempos, valores e fonte.
`YouTubeDaily`	classe	`(dates, n_entries, total_views, total_likes, category_counts, source)`	Agregados diários dos vídeos em alta: data, número de entradas, visualizações, curtidas e contagem por categoria.
`fetch_pinned`	função	`(key, cache_dir=None, *, local=None) -> bytes`	Bytes de um arquivo fixado (de arquivo local, cache ou rede), com SHA-256 verificado.
`load_usd_eur_daily`	função	`(cache_dir=None) -> PublicSeries`	Carrega a taxa USD por EUR em dias úteis (dias sem cotação são descartados).
`load_sunspots_yearly`	função	`(cache_dir=None) -> PublicSeries`	Carrega a série anual de manchas solares distribuída com `statsmodels`.
`log_returns`	função	`(values) -> np.ndarray`	Calcula os retornos logarítmicos de uma série de valores.
`youtube_daily_aggregates`	função	`(csv_text) -> YouTubeDaily`	Agrega as linhas de um CSV de vídeos em alta para uma linha por dia.
`load_youtube_trending_us`	função	`(cache_dir=None) -> YouTubeDaily`	Baixa (fixado e verificado) e agrega o CSV de vídeos em alta dos EUA.

pinneapple_systems.time_series.datasets.windowed

`WindowedTimeSeriesDataset`: corta uma série (T, F) em pares (janela de entrada, horizonte) com passo e deslocamento. [sem teste direto]

`pinneapple_systems/time_series/datasets/windowed.py` · 41 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`WindowedTimeSeriesDataset`	classe	`(series, spec)`	Dataset PyTorch de pares (janela de entrada, horizonte) cortados de uma série (T, F).

pinneapple_systems.time_series.decomposition

Reexporta os previsores por decomposição (FFT, FFT+NN, HHT+NN, FFT+LSTM, HHT+LSTM) e tenta reexportar solvers clássicos de decomposição de `pinneapple_simulation.numerical_solvers` (FFT, wavelet, HHT, EEMD, CEEMDAN, VMD, SSA, SST, STL). [sem teste direto]

`pinneapple_systems/time_series/decomposition/__init__.py` · 53 linhas · **sem teste direto**

pinneapple_systems.time_series.decomposition.fft_forecaster

Previsor só com FFT: ajusta as K frequências dominantes (com remoção de tendência opcional) e extrapola as senoides.

`pinneapple_systems/time_series/decomposition/fft_forecaster.py` · 86 linhas · testes: 151

nome	tipo	assinatura	o que faz
`FFTForecaster`	classe	`(n_harmonics=10, detrend=True)`	Ajusta as K componentes de frequência dominantes via FFT e as extrapola.

pinneapple_systems.time_series.decomposition.fft_lstm

Híbrido FFT + LSTM multivariado: um `FFTForecaster` por canal captura a parte periódica e uma LSTM prevê o resíduo; `FFTLSTMPipeline` orquestra os dois. [sem teste]

`pinneapple_systems/time_series/decomposition/fft_lstm.py` · 335 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`FFTDecomposer`	classe	`(n_harmonics=10, detrend=True, auto_harmonics=True, auto_threshold=0.05) -> None`	Ajusta um `FFTForecaster` por canal e calcula resíduos e extrapolações.
`ResidualLSTMConfig`	classe	`(input_len, horizon, n_features, n_targets, hidden_size, num_layers, dropout, layer_norm)`	Hiperparâmetros do `ResidualLSTMForecaster`.
`ResidualLSTMForecaster`	classe	`(cfg=None, **kw)`	LSTM treinada sobre os resíduos da FFT.
`FFTLSTMPipeline`	classe	`(*, n_harmonics=10, detrend=True, auto_harmonics=True, auto_threshold=0.05, input_len=64,…)`	Orquestra decomposição FFT, treino da LSTM nos resíduos e recomposição da previsão.

pinneapple_systems.time_series.decomposition.fft_nn

Previsor em dois estágios: FFT para a parte periódica global e uma rede neural treinada no resíduo.

`pinneapple_systems/time_series/decomposition/fft_nn.py` · 100 linhas · testes: 151

nome	tipo	assinatura	o que faz
`FFTNNForecaster`	classe	`(n_harmonics=10, detrend=True, input_len=32, horizon=16, nn_model=None, nn_epochs=50, nn_…)`	Previsor em dois estágios: FFT para a parte periódica e rede neural no resíduo.

pinneapple_systems.time_series.decomposition.hht_lstm

Híbrido Hilbert-Huang (EMD) + LSTM multivariado: IMFs dominantes por canal são extrapoladas e a LSTM prevê o resíduo; precisa de `PyEMD`. [sem teste]

`pinneapple_systems/time_series/decomposition/hht_lstm.py` · 341 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`HHTDecomposer`	classe	`(n_imfs=6, dominant_energy_ratio=0.8, detrend=True) -> None`	Ajusta um decompositor EMD por canal, calcula resíduos e extrapola as IMFs dominantes.
`HHTLSTMPipeline`	classe	`(n_imfs=6, dominant_energy_ratio=0.8, detrend=True, input_len=64, horizon=16, n_features=…)`	Orquestra decomposição HHT, LSTM nos resíduos e recomposição.

pinneapple_systems.time_series.decomposition.hht_nn

Previsor em dois estágios: EMD separa IMFs, as dominantes são extrapoladas e uma rede neural prevê o resto; precisa de `PyEMD`.

`pinneapple_systems/time_series/decomposition/hht_nn.py` · 172 linhas · testes: 151

nome	tipo	assinatura	o que faz
`HHTNNForecaster`	classe	`(n_imfs=5, input_len=32, horizon=16, nn_model=None, nn_epochs=50, nn_lr=0.001)`	Previsor em dois estágios: IMFs dominantes da EMD extrapoladas e rede neural no restante.

pinneapple_systems.time_series.eda

Reexporta as dez ferramentas gráficas de análise exploratória de `eda.plots`. [sem teste]

`pinneapple_systems/time_series/eda/__init__.py` · 26 linhas · **sem teste direto**

pinneapple_systems.time_series.eda.plots

Dez gráficos de EDA em matplotlib: tendência, rastreio de setpoint, estatísticas móveis, resposta ao degrau, espectro, ACF/PACF, testes de estacionariedade, pontos de mudança, correlação cruzada e matriz RGA. [sem teste]

`pinneapple_systems/time_series/eda/plots.py` · 463 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`plot_trend`	função	`(y, *, window=24, title='Trend Analysis', figsize=(12, 6), show=False) -> Figure`	Série bruta com média móvel e faixa de desvio-padrão móvel.
`track_setpoint`	função	`(actual, setpoint, *, tolerance=0.05, title='Setpoint Tracking', figsize=(12, 5), show=Fa…)`	Valor real contra setpoint, com sombreamento do desvio e faixa de tolerância.
`rolling_statistics`	função	`(y, windows=(8, 24, 96), *, stats=('mean', 'std'), title='Rolling Statistics', figsize=(1…)`	Sobreposição de estatísticas móveis em várias janelas.
`step_response`	função	`(y, step_idx=None, *, title='Step Response', figsize=(12, 5), show=False) -> Figure`	Mostra a resposta a um degrau (detectado ou dado) e estima ganho, tempo de subida e outras métricas.
`power_spectrum`	função	`(y, *, fs=1.0, n_top=5, title='FFT Power Spectrum', figsize=(12, 5), show=False) -> Figure`	Espectro de potência da FFT em escala log, anotando as frequências dominantes.
`plot_acf_pacf`	função	`(y, nlags=40, *, alpha=0.05, title='ACF / PACF', figsize=(12, 5), show=False) -> Figure`	ACF e PACF lado a lado com bandas de confiança.
`stationarity_report`	função	`(y, *, title='Stationarity Tests', figsize=(10, 4), show=False) -> Figure`	Roda ADF, KPSS e PP e mostra uma tabela-resumo.
`changepoint_plot`	função	`(y, n_bkps=3, *, model='rbf', title='Change Point Detection', figsize=(12, 5), show=False…)`	Mostra pontos de mudança detectados com `ruptures`.
`cross_correlation`	função	`(df, cols=None, max_lag=20, *, title='Cross-Correlation', figsize=(12, 6), show=False) ->…)`	Mapa de calor da correlação cruzada máxima e da defasagem do pico entre pares de colunas.
`rga_matrix`	função	`(K, mv_labels=None, cv_labels=None, *, title='Relative Gain Array (RGA)', figsize=(8, 6),…)`	Calcula e mostra a Relative Gain Array (Bristol) de uma matriz de ganhos.

pinneapple_systems.time_series.enso

Pacote do estudo de previsão do ENSO (El Niño): dados, modelos físicos, previsores informados por física e por dados, e avaliação; o `__init__` só tem o docstring.

`pinneapple_systems/time_series/enso/__init__.py` · 2 linhas · testes: 22

pinneapple_systems.time_series.enso.data

Baixa observações abertas do ENSO (OISST, ERA5 do WeatherBench 2, perfis WOD, conteúdo de calor NCEI, Niño 1+2 do `statsmodels`), reduz para produtos mensais do Pacífico tropical e grava em cache com o SHA-256 de cada arquivo-fonte num manifesto; precisa de rede na primeira execução.

`pinneapple_systems/time_series/enso/data.py` · 674 linhas · testes: 22

nome	tipo	assinatura	o que faz
`Manifest`	classe	`(path, expected=None)`	Manifesto URL -> {sha256, bytes} de cada arquivo-fonte de um produto; um manifesto anterior fixa os hashes esperados.
`Gridded`	classe	`(time, lat, lon, values, source)`	Campos mensais (nt, ny, nx) numa grade regular lat/lon, com seleção de região.
`manifests`	função	`(cache_dir=None) -> dict[str, Path]`	Lista os manifestos já gravados no cache, por produto.
`fetch`	função	`(url, *, timeout=300, retries=4) -> bytes`	Baixa os bytes de uma URL (respeitando o proxy), com novas tentativas e espera exponencial.
`month_range`	função	`(start, end) -> list[str]`	Lista de strings 'YYYY-MM' entre duas datas, inclusive.
`oisst_monthly`	função	`(start='1981-09', end=None, *, lat=(-30.0, 30.0), lon=(100.0, 300.0), res=1.0, cache_dir=…)`	Médias mensais de TSM do OISST v2.1 numa grade e região dadas.
`box_mean`	função	`(g, var, box) -> np.ndarray`	Média ponderada por cos(latitude) de uma variável dentro de uma caixa.
`nino_indices`	função	`(g, var='sst') -> dict[str, np.ndarray]`	Médias das caixas Niño 3.4, 3, 4 e 1+2 (não anomalias).
`coarsen`	função	`(g, ky, kx) -> Gridded`	Médias em blocos de ky x kx células, tolerante a NaN.

nome	tipo	assinatura	o que faz
anomalies	função	(x, time, base) -> tuple[np.ndarray, np.ndarray]	Anomalias em relação à climatologia mensal de um período base.
d20_from_profile	função	(z, T, *, iso=20.0, max_gap=60.0) -> float	Profundidade em que um perfil de temperatura cruza a isoterma de 20 °C (D20), por interpolação linear.
fetch_to_file	função	(url, path, *, timeout=1800, retries=4, chunk=1 << 22) -> str	Grava uma URL em arquivo por streaming e devolve o SHA-256.
wod_d20_casts	função	(years, kinds=('mrb',), *, lat=(-8.0, 8.0), lon=(120.0, 280.0), cache_dir=None, manifest=…)	D20 de cada perfil com controle de qualidade do World Ocean Database dentro da caixa.
d20_monthly	função	(casts, months, *, lat=(-5.0, 5.0), lon_edges=tuple(range(130, 281, 10)), min_casts=1, da…)	D20 média mensal por faixa de longitude a partir dos perfis.
d20_climatology	função	(casts, lon_edges, lat_edges, base) -> np.ndarray	Climatologia da D20 por célula (lat, lon) no período base.
era5_monthly	função	(variables=…, *, cache_dir=None, manifest=None, workers=8, derived=True, log=print) -> Gr…)	Médias mensais de campos ERA5 na grade do WeatherBench 2, com variáveis derivadas.
wind_stress	função	(u, v, rho_air=1.2, cd=0.0013)	Tensão do vento pela fórmula bulk com coeficiente de arrasto constante.
soi_from_mslp	função	(g, var='mean_sea_level_pressure', base=None) -> np.ndarray	Índice de Oscilação Sul (Taiti menos Darwin) a partir da pressão ao nível do mar.
ncei_heat_content	função	(box=(-5.0, 5.0, 120.0, 280.0), *, cache_dir=None, manifest=None) -> dict[str, Any]	Soma da anomalia de conteúdo de calor 0-700 m do NCEI na caixa, em médias trimestrais.
statsmodels_nino12	função	() -> dict[str, Any]	Série mensal Niño 1+2 distribuída com `statsmodels`.

pinneapple_systems.time_series.enso.dataset

Monta o conjunto de dados do estudo a partir das fontes em cache (índices mensais, mapas de anomalia de TSM, profundidade D20 preenchida por EOF, tensão do vento) e o converte em `SeriesData` do `forecast_lab`.

pinneapple_systems/time_series/enso/dataset.py · 226 linhas · testes: 22

nome	tipo	assinatura	o que faz
EnsoDataset	classe	(months, indices, sst_maps, grid, d20_band, d20_eq, d20_lon, d20_observed, taux, base, cl…)	Conjunto de dados do estudo: meses, índices, mapas de anomalia de TSM, D20, tensão do vento, período base e checagens.
eof_fill	função	(X, base_rows, n_eofs=4, ridge=0.1) -> np.ndarray	Preenche NaNs de cada linha com as EOFs dominantes por mínimos quadrados regularizados.
build_dataset	função	(start='1982-01', end='2024-12', base=('1982-01', '2011-12'), grid=None, *, cache_dir=Non…)	Constrói o `EnsoDataset` a partir das fontes em cache para um intervalo de meses.
to_series	função	(ds, columns=COLUMNS, name='enso')	Converte o `EnsoDataset` em `SeriesData` com as colunas pedidas e os mapas em `meta`.

pinneapple_systems.time_series.enso.evaluation

Escores específicos do ENSO sobre os do `forecast_lab`: correlação de anomalia por antecedência, skill por mês alvo (barreira de primavera), verificação categórica El Niño/neutro/La Niña e hindcast com validação cruzada em blocos.

pinneapple_systems/time_series/enso/evaluation.py · 96 linhas · testes: 22

nome	tipo	assinatura	o que faz
acc	função	(mean, y) -> list[float]	Correlação de anomalia (Pearson sobre as origens) por coluna de horizonte.
by_target_month	função	(mean, y, origins, month_of_year, horizons, min_n=6) -> dict[str, Any]	ACC e RMSE por (mês alvo, horizonte).
category	função	(x, threshold=0.5) -> np.ndarray	Classifica em La Niña (-1), neutro (0) ou El Niño (+1) por um limiar.
categorical	função	(mean, y, threshold=0.5) -> dict[str, Any]	Tabela de contingência 3 x 3 e taxas de acerto por categoria e horizonte.

nome	tipo	assinatura	o que faz
blocked_hindcast	função	(make, data, horizons, *, block=48, first=36, last=None, val_frac=0.15) -> dict[str, Any]	Previsões fora da amostra para cada origem com validação cruzada em blocos.

pinneapple_systems.time_series.enso.forecasters

Previsores do ENSO com a interface Forecaster do forecast_lab: oscilador de recarga, modelo linear inverso (LIM), CNN sobre mapas, modelo intermediário acoplado com EnKF e ensemble; ColumnSubset._fit levanta NotImplementedError porque fit é sobrescrito.

pinneapple_systems/time_series/enso/forecasters.py · 683 linhas · testes: 22 · palavras-chave no código (heurística; o texto acima diz o status real): levanta NotImplementedError em algum caminho

nome	tipo	assinatura	o que faz
ColumnSubset	classe	(inner, cols, name=None, family=None)	Embrulha um previsor para que ele veja só algumas colunas.
RechargeOscillatorForecaster	classe	(h_col='wwv', seasonal=(), multiplicative=False, stochastic=True, n_members=200, sub=4, s…)	Previsor pelo oscilador de recarga de Jin sobre (T, h), com ensemble estocástico opcional.
LIMForecaster	classe	(n_eofs=10, extra_cols=('wwv',), tau0=1, name=None)	Modelo linear inverso sobre as PCs dos mapas de TSM e índices extras.
MapCNNForecaster	classe	(n_lags=3, pool=2, scalar_cols=('wwv', 'h_west', 'h_east'), channels=(16, 32), hidden=64,…)	CNN sobre os últimos mapas mensais de anomalia de TSM mais escalares.
IntermediateModelForecaster	classe	(init='nudged', n_members=1, enkf_members=24, coupling=1.0, eps_a=0.1, c=2.7, r_days=900.…)	Previsor pelo modelo intermediário acoplado ajustado no treino, iniciado por nudging ou EnKF.
EnsembleForecaster	classe	(members, n_draw=100, seed=0, name=None, family=None)	Mistura de pesos iguais de membros ajustados juntos.
box_index	função	(maps, grid, box=(-5.0, 5.0, 190.0, 240.0)) -> np.ndarray	Média ponderada por cos(lat) de mapas numa caixa (padrão Niño 3.4).
ro_rhs	função	(x, R=-0.05, F1=0.02, F2=1.5, eps=0.03)	Campo vetorial do oscilador de recarga com coeficientes constantes, para os previsores de EDO.
fit_intermediate	função	(meta, end, *, eps_a=0.1, c=2.7, r_days=900.0, coupling=1.0, window=4, ridge=0.001, L_T=6…)	Ajusta o modelo acoplado intermediário às linhas de treino do estudo.

pinneapple_systems.time_series.enso.intermediate

Modelo acoplado intermediário simplificado do Pacífico tropical, no espírito de Zebiak & Cane (1987): oceano de águas rasas em gravidade reduzida, atmosfera de Gill e equação de TSM, com testes de verificação (onda de Kelvin, Rossby, Gill analítico, convergência).

pinneapple_systems/time_series/enso/intermediate.py · 607 linhas · testes: 22

nome	tipo	assinatura	o que faz
OceanGrid	classe	(lon0, lon1, dlon, lat_max, dlat)	Grade C do oceano: h e T nos centros, u e v nas faces.
ShallowWaterOcean	classe	(grid=None, c=2.7, H=150.0, r_days=900.0, sponge_deg=5.0, sponge_days=5.0, dtype='float64…)	Oceano linear de águas rasas em gravidade reduzida no plano beta equatorial.
GillAtmosphere	classe	(eps=0.1, c_a=60.0, n_modes=24, dlon=2.5, lon_offset=0.0)	Resposta de onda longa de Gill (1980) a um aquecimento.
CoupledParams	classe	(eps_T, gamma, kappa, mu, mu_y, L_T, noise_tau, noise_months, noise_lon, extra)	Parâmetros do modelo acoplado (coeficientes de TSM por longitude, ruído).
CoupledModel	classe	(params, ocean=None, atmosphere=None, dt_hours=8.0, atm_every=3, closure=None, atm_lat=40…)	Modelo acoplado linear oceano-atmosfera de anomalias sobre a OceanGrid.
gill_analytic	função	(x, y, L, eps, period) -> dict[str, np.ndarray]	Resposta de Gill em forma fechada, usada como referência.
sst_tendency	função	(T, h, taux_c, eps_T, gamma, kappa, S)	Tendência de TSM a partir de T, h e tensão do vento.
kelvin_wave_test	função	(grid=None, c=2.7, dt_hours=6.0, days=40.0, sigma_km=1500.0) -> dict	Teste: solta uma onda de Kelvin equatorial e mede sua velocidade.
rossby_wave_test	função	(grid=None, c=2.7, dt_hours=6.0, days=150.0, sigma_km=2500.0) -> dict	Teste: solta uma onda de Rossby longa n = 1 e mede sua propagação.

nome	tipo	assinatura	o que faz
gill_test	função	(nlons=(72, 144, 288, 576), eps=0.1, L=2.0, n_modes=12) -> dict	Teste: compara a resposta numérica de Gill com a analítica em várias resoluções.
convergence_study	função	(factors=(1, 2, 4), c=2.7) -> dict	Erro da velocidade da onda de Kelvin sob refinamento conjunto de dx, dy e dt.
forced_ocean_run	função	(ocean, taux, *, steps_per_month=91, tauy=None, state=None) -> tuple[np.ndarray, dict]	Integra só o oceano sob tensão do vento mensal prescrita.

pinneapple_systems.time_series.enso.oscillators

Modelos conceituais do ENSO: oscilador de recarga (com coeficientes sazonais, ajuste e simulação estocástica) e oscilador com atraso (raízes por Lambert-W, método dos passos).

pinneapple_systems/time_series/enso/oscillators.py · 264 linhas · testes: 22

nome	tipo	assinatura	o que faz
ROParams	classe	(coef, sigma_T, sigma_h, B, info)	Coeficientes do oscilador de recarga por mês (harmônicos anuais) e amplitudes de ruído.
ro_eigen	função	(R, F1, F2, eps) -> dict[str, float]	Autovalores, taxa de crescimento e período do oscilador de recarga com coeficientes constantes.
ro_coefficients	função	(p, month) -> dict[str, np.ndarray]	Coeficientes do oscilador em meses do calendário (fracionários).
ro_tendency	função	(p, X, month) -> np.ndarray	Tendência determinística (dT/dt, dh/dt) de estados num mês dado.
simulate_recharge	função	(p, x0, n_months, *, month0=0.0, sub=10, n_members=1, stochastic=True, seed=0, method='rk…)	Integra o oscilador de recarga, determinístico ou estocástico, com vários membros.
fit_recharge	função	(T, h, month=None, *, seasonal=(), multiplicative=False, lead=1, sub=4) -> ROParams	Ajusta o oscilador de recarga a séries mensais T e h.
do_rhs	função	(x, xt, a=1.0, b=0.75, c=1.0)	Lado direito do oscilador com atraso de Battisti-Hirst / Suarez-Schopf.
do_roots	função	(a, b, tau, branches=(0, -1, 1)) -> list[complex]	Raízes da equação característica linear do oscilador com atraso via Lambert-W.
simulate_delayed	função	(a, b, c, tau, history, n_steps, dt, method='rk4') -> np.ndarray	Integra o oscilador com atraso pelo método dos passos.
oscillation_stats	função	(x, dt, *, skip=0) -> dict[str, float]	Período e taxa de crescimento de uma série oscilante.

pinneapple_systems.time_series.enso.pinn

Redes informadas por física para o ENSO, treinadas com `InversePINN` de `pinneapple_neural` (pesos e parâmetros físicos desconhecidos otimizados juntos): identificação do oscilador de recarga, previsão por extrapolação de PINN e PINN de ondas equatoriais.

pinneapple_systems/time_series/enso/pinn.py · 626 linhas · testes: 22

nome	tipo	assinatura	o que faz
FourierMLP	classe	(in_dim, out_dim, hidden=64, layers=3, n_freq=8, out_scale=None, freq_max=None)	MLP sobre features de Fourier das entradas.
GroupedMLP	classe	(groups, in_dim, out_dim, hidden=32, layers=2, n_freq=4, freq_max=2.0, out_scale=None, se…)	G MLPs de Fourier independentes avaliadas em lote.
PINNRechargeForecaster	classe	(h_col='wwv', seasonal=(), closure=False, stochastic=False, n_members=200, epochs=4000, s…)	Previsor que integra o oscilador de recarga identificado por PINN no treino.
PINNExtrapolationForecaster	classe	(h_col='wwv', window=24, epochs=3000, n_col=512, hidden=32, chunk=256, seed=0, name=None)	Para cada origem, ajusta uma PINN aos últimos meses e às equações do oscilador e extrapola.
EquatorialWavePINN	classe	(lon_obs, h_obs, T_obs, lon_f, X0, X2, tau_eq, *, lon_w=120.0, lon_e=280.0, H=150.0, sst=…)	PINN para o oceano equatorial de ondas longas em dois modos mais uma equação de TSM.
identify_recharge	função	(T, h, month=None, *, seasonal=(), closure=False, epochs=4000, n_col=2048, hidden=64, lay…)	Identificação por PINN dos parâmetros do oscilador de recarga em séries mensais (T, h).
project_forcing	função	(taux, lat, c=2.7, H=150.0, rho=1025.0) -> tuple[np.ndarray, np.ndarray]	Projeções da forçante do vento nas funções de Hermite de ordem 0 e 2.

pinneapple_systems.time_series.enso.pipeline

Etapas do estudo ENSO que devolvem resultados serializáveis em JSON (checagens dos solvers, escores, painéis de casos, skill contra referência, barreira de primavera, retrato de fase, estudo PINN de águas rasas, quadros para render, tabelas Markdown).

pinneapple_systems/time_series/enso/pipeline.py · 419 linhas · testes: 22

nome	tipo	assinatura	o que faz
solver_checks	função	() -> dict[str, Any]	Roda os testes de Kelvin, Rossby, convergência e Gill e compara o oscilador de recarga simulado com a teoria.
enso_scores	função	(merged, month_of_year, threshold=0.5) -> dict[str, Any]	ACC por antecedência, skill por mês alvo e escores categóricos de cada modelo.
case_panels	função	(parts, merged, data, *, families=…, events=None, months_before=18, leads=(1, 3, 6, 9, 12…)	Painéis de plumas para as origens dos estudos de caso, por família de modelo.
skill_against	função	(merged, reference='climatology', alpha=0.05) -> dict[str, Any]	Skill de RMSE contra uma referência e testes de Diebold-Mariano com ajuste de Holm.
spring_barrier_hindcasts	função	(data, prot, makers, *, block=48) -> dict[str, Any]	Hindcasts em blocos com purga no período de desenvolvimento, estratificados por mês alvo.
phase_portrait	função	(data, prot, *, seasonal=('R', 'F1', 'F2', 'eps'), seed=0, onsets=…, n_path=48) -> dict[s…)	Retrato de fase (h, T) observado e trajetórias dos osciladores ajustados.
shallow_water_pinn_study	função	(ds, window=('1996-01', '1999-12'), *, epochs=6000, n_col=4096, pde_weight=100.0, spin_mo…)	Estudo da PINN de ondas equatoriais em dados sintéticos e observados.
slab_frames	função	(ds, oisst, casts, start, end, *, spin_months=36, lon=(121.0, 279.0), lat=(-19.5, 19.5)) …)	Quadros mensais (TSM e termoclina) para o render 3-D.
markdown_tables	função	(res, leads=(1, 3, 6, 9, 12, 18, 24)) -> dict[str, str]	Tabelas Markdown do estudo a partir dos resultados gravados.

pinneapple_systems.time_series.enso.plots

Figuras do estudo ENSO em matplotlib no estilo do `forecast_lab`: plumas, skill por antecedência, barreira de primavera, retrato de fase, Hovmöller e validação dos solvers. [sem teste]

pinneapple_systems/time_series/enso/plots.py · 217 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
plot_plumes	função	(panels, path, *, ncols=3) -> str	Plumas de distribuição de previsão a partir de uma origem por painel.
plot_skill_vs_lead	função	(curves, path, *, metrics=…, labels=None) -> str	Um painel por métrica com as curvas de skill contra antecedência.
plot_spring_barrier	função	(maps, path, *, ncols=3, key='acc') -> str	Mapas de calor do ACC por mês alvo e antecedência, um por modelo.
plot_phase_portrait	função	(traj, path) -> str	Retratos de fase (h, T) com setas de direção.
plot_hovmoller	função	(panels, path, *, ncols=3) -> str	Diagramas de Hovmöller (longitude x tempo).
plot_validation	função	(kelvin, gill, conv, ro, path) -> str	Figura das checagens dos solvers (Kelvin, Gill, convergência, oscilador).

pinneapple_systems.time_series.enso.render

Renders 3-D do Pacífico tropical durante um evento ENSO com o renderizador Blender Cycles da biblioteca (`pinneapple_tools.visualization.studio.render`) e codificação de vídeo com ffmpeg.

pinneapple_systems/time_series/enso/render.py · 131 linhas · testes: 22

nome	tipo	assinatura	o que faz
slab_scene	função	(lon, lat, sst, d20, *, d20_anom=None, band_deg=2.0, z_per_100m=1.0, x_per_deg=0.06, y_pe…)	Cena 3-D de uma fatia do oceano com TSM na superfície e termoclina em profundidade.
render_frames	função	(frames, out_dir, *, sst_range=(22.0, 31.0), size=(960, 540), samples=24, view=(-0.4, -1.…)	Renderiza uma sequência de quadros mensais em PNG.
encode_video	função	(paths, out, fps=8, crf=30) -> str	Codifica quadros PNG em vídeo MP4 H.264 com ffmpeg.

pinneapple_systems.time_series.enso.study

O estudo de previsão do ENSO: protocolo, zoo de modelos e uma chamada por modelo (para rodar em processos paralelos e juntar com merge_zoo), além de hindcasts de eventos. [sem teste direto]

pinneapple_systems/time_series/enso/study.py · 173 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
protocol	função	(months, test_start='2012-01', horizons=HORIZONS, min_history=36) -> Protocol	Cria o Protocol do estudo ENSO (início do teste, horizontes, histórico mínimo).
zoo	função	(quick=False) -> list[ModelSpec]	Lista de ModelSpec do zoo; quick reduz épocas e membros para testes rápidos.
spec	função	(name, quick=False) -> ModelSpec	Devolve o ModelSpec de um modelo do zoo pelo nome.
run_model	função	(name, data, prot, *, budget=FP.BUDGET, quick=False, case_origins=None, log=print, seed=0…)	Faz tuning, ajuste e avaliação de um modelo do zoo.
event_hindcast	função	(name, data, event_start, origins, *, quick=False, horizons=HORIZONS, min_history=36, val…)	Reajusta um modelo e prevê a partir de origens próximas a um evento.

pinneapple_systems.time_series.features

Reexporta o gerador de features (TSFeatureEngineer, defasagens, estatísticas móveis, Fourier). [sem teste direto]

pinneapple_systems/time_series/features/__init__.py · 2 linhas · **sem teste direto**

pinneapple_systems.time_series.features.engineering

Engenharia de features para ML tabular: defasagens, estatísticas móveis, harmônicos de Fourier, diferenças e janelas deslizantes (X, y). [sem teste direto]

pinneapple_systems/time_series/features/engineering.py · 110 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
TSFeatureEngineer	classe	(lags, rolling_windows, rolling_stats_set, fourier_periods, fourier_K)	Wrapper que gera um dicionário de features (defasagens, estatísticas móveis, Fourier).
make_lags	função	(y, lags) -> Dict[str, np.ndarray]	Cria features de defasagem de uma série 1D.
rolling_stats	função	(y, windows, stats=('mean', 'std', 'min', 'max')) -> Dict[str, np.ndarray]	Calcula estatísticas móveis da série em várias janelas.
fourier_features	função	(t, periods, K=1) -> Dict[str, np.ndarray]	Gera features de sazonalidade seno/cosseno para cada período.
rate_of_change	função	(y, orders=(1, 2)) -> Dict[str, np.ndarray]	Diferenças de primeira e maior ordem (velocidade, aceleração).
window_features	função	(y, input_len, horizon) -> tuple	Monta arrays de janelas deslizantes (X, y_alvo) para ML supervisionado.

pinneapple_systems.time_series.forecast_lab

Laboratório para prever séries quase aleatórias sem se enganar: uma interface comum para baselines, previsores por dados, dados + física, físicos e híbridos, e o protocolo que os compara (tuning em walk-forward com purga, período de teste intocado, escores pontuais e probabilísticos, testes de Diebold-Mariano com controle de comparações múltiplas).

pinneapple_systems/time_series/forecast_lab/__init__.py · 44 linhas · testes: 50

pinneapple_systems.time_series.forecast_lab.base

Interface comum dos previsores do forecast_lab: SeriesData, Forecast (média com desvio gaussiano ou amostras) e a base Forecaster, cujo _predict_mean levanta NotImplementedError se não for implementado. [sem teste direto]

pinneapple_systems/time_series/forecast_lab/base.py · 168 linhas · **sem teste direto** · palavras-chave no código (heurística; o texto acima diz o status real): levanta NotImplementedError em algum caminho

nome	tipo	assinatura	o que faz
SeriesData	classe	(obs, target, dt, name, meta)	Série observada (T, d), coluna alvo, passo de amostragem e metadados livres.
Forecast	classe	(mean, std, samples)	Distribuição preditiva em (origem, horizonte): média e desvio gaussiano ou amostras.
Forecaster	classe	()	Classe base dos previsores: subclasses implementam _fit e _predict_mean (ou sobrescrevem predict).
train_end	função	(train_origins, horizons) -> int	Índice final exclusivo dos dados que um ajuste nessas origens pode usar.
forecast_origins	função	(lo, hi, horizons, *, stride=1, min_history=0) -> np.ndarray	Origens válidas de previsão dentro de um intervalo, com passo e histórico mínimo.
targets_at	função	(data, origins, horizons) -> np.ndarray	Valores alvo observados em cada origem e horizonte.

pinneapple_systems.time_series.forecast_lab.baselines

Baselines que todo previsor precisa vencer: persistência, climatologia, AR(p), ARIMA, suavização exponencial, ingênuo sazonal e climatologia alinhada pela fase. [sem teste direto]

pinneapple_systems/time_series/forecast_lab/baselines.py · 278 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
Persistence	classe	()	Previsão de passeio aleatório: o último valor.
Climatology	classe	()	Média do treino, com dispersão igual ao seu desvio-padrão.
ARForecaster	classe	(max_order=20, criterion='aic', difference=False)	AR(p) por mínimos quadrados com ordem por AIC.
ARIMAForecaster	classe	(p=(0, 1, 2), d=(0, 1), q=(0, 1), max_fit_length=2000)	ARIMA via SARIMAX do statsmodels, ordem escolhida por AIC numa grade pequena.
ExponentialSmoothing	classe	()	Suavização exponencial simples com alpha que minimiza o erro de um passo.
SeasonalNaive	classe	(period=None)	Valor de um período (ou do menor número de períodos) antes do alvo.
PhaseClimatology	classe	(period=None, fit_cycles=1.0)	Ciclo médio alinhado à fase de cada origem.
dominant_period	função	(y, min_period=2.0, max_frac=0.25) -> float	Período do maior pico do periodograma.

pinneapple_systems.time_series.forecast_lab.hybrid

Previsores híbridos: modelo físico mais algo aprendido (correção do resíduo por GBM ou rede, e equação diferencial universal com termo neural). [sem teste direto]

pinneapple_systems/time_series/forecast_lab/hybrid.py · 181 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
ResidualCorrectionForecaster	classe	(physical, corrector='gbm', n_lags=8, lag_step=1, max_iter=200, learning_rate=0.05, max_l…)	Previsão física mais um corretor (GBM ou rede) treinado no resíduo.

nome	tipo	assinatura	o que faz
UDEForecaster	classe	(physical, hidden=64, layers=2, window=4, epochs=200, patience=20, lr=0.001, weight_decay…)	Equação diferencial universal: física conhecida mais um termo neural, treinada através de RK4.

pinneapple_systems.time_series.forecast_lab.learned

Previsores diretos por dados sobre janelas defasadas: gradient boosting e redes neurais (MLP, LSTM, TCN, transformer), com features físicas opcionais e, para o MLP, perda física. [sem teste direto]

pinneapple_systems/time_series/forecast_lab/learned.py · 355 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
LagFeatures	classe	(n_lags=8, lag_step=1, cols=None, extra=None)	Janelas defasadas das colunas observadas, com features extras opcionais.
GBMForecaster	classe	(n_lags=8, lag_step=1, max_iter=200, learning_rate=0.05, max_leaf_nodes=15, min_samples_l…)	Previsor direto com gradient boosting por histogramas do scikit-learn.
NeuralForecaster	classe	(arch='mlp', n_lags=16, lag_step=1, hidden=64, layers=2, dropout=0.0, weight_decay=0.0, l…)	Previsor neural direto multi-horizonte (MLP, LSTM, TCN ou transformer).
PhysicsLossMLP	classe	(physical, lam=0.1, **kw)	MLP que prevê todo o estado em cada passo e é penalizado por violar o modelo físico.
train_torch_regressor	função	(build, X_tr, Y_tr, X_va=None, Y_va=None, *, epochs=100, lr=0.001, weight_decay=0.0, batc…)	Treina um regressor torch com o Trainer de pinneapple_neural.

pinneapple_systems.time_series.forecast_lab.physical

Previsores físicos: equações governantes com parâmetros estimados só no treino (EDO, EDO com atraso, espaço de estados linear, deriva de EDE, EnKF, SINDy) e features físicas para modelos de dados.

pinneapple_systems/time_series/forecast_lab/physical.py · 564 linhas · testes: 22

nome	tipo	assinatura	o que faz
ODEForecaster	classe	(rhs, params, fit_params=None, sub=5, fit_steps=2, max_fit_origins=1500, divergence_facto…)	Previsor por EDO dx/dt = rhs(x) com parâmetros ajustados no treino.
DelayODEForecaster	classe	(rhs, params, tau, fit_params=None, dt_int=0.1, fit_steps=2, max_fit_origins=1500, diverg…)	Previsor por EDO com atraso, integrada por RK4.
LinearSSMForecaster	classe	(kind='ou', max_fit_length=4000, name=None, family=None)	Modelo linear gaussiano de espaço de estados (por exemplo Ornstein-Uhlenbeck) com filtro de Kalman.
SDEDriftForecaster	classe	(degree=3, n_samples=200, sub=10, closure=None, seed=0, name=None, family=None)	EDE com deriva polinomial ajustada aos incrementos, com fechamento neural opcional.
EnKFForecaster	classe	(physical, n_ens=50, inflation=1.02, q_frac=0.1, spinup=200, seed=0, name=None, family=No…)	Filtro de Kalman por ensemble estocástico sobre um modelo físico.
SINDyForecaster	classe	(threshold=0.05, l2=0.0001, library='poly', sub=5, divergence_factor=5.0, name=None, fami…)	Equações descobertas por SINDy na trajetória de treino, integradas para prever.
PhysicsFeatures	classe	(physical)	Features de entrada para modelos de dados calculadas por um previsor físico.

pinneapple_systems.time_series.forecast_lab.pipeline

Uma chamada por caso do estudo de séries quase aleatórias e as tabelas do guia, montadas a partir dos resultados gravados.

pinneapple_systems/time_series/forecast_lab/pipeline.py · 270 linhas · testes: 28

nome	tipo	assinatura	o que faz
fan_panels	função	(zoo, data, title, n_origins=2, families=('physical', 'hybrid', 'data')) -> list[dict]	Dados de gráfico em leque para os representantes de cada família.
surprise_record	função	(zoo, data, horizon_index=0) -> dict[str, Any] | None	Erro absoluto por origem de teste do melhor modelo e da melhor baseline.
run_case	função	(name, *, n=6000, budget=BUDGET, seed=0, cache_dir=None, n_surrogates=99, log=print) -> d…)	Roda um caso completo: previsibilidade, zoo de modelos, gêmeos treinados em surrogates e leques.

nome	tipo	assinatura	o que faz
run_overfitting	função	(name, *, n=6000, seed=0, cache_dir=None, schemes=…, log=print) -> dict[str, Any]	Roda o estudo de sobreajuste de um caso com vários esquemas de validação.
lyapunov_study	função	(dt_ints=(0.1, 0.05), total_times=(10000.0, 50000.0, 200000.0), seeds=(0, 1, 2), log=prin…)	Convergência da estimativa de Benettin do expoente de Lyapunov do Mackey-Glass.
memorise_vs_learn	função	(name, variants=…, *, horizon_index=0, window=200, n=6000, seed=0, cache_dir=None) -> dic…)	Compara duas variantes de treino nas últimas origens de treino e nas primeiras de teste.
markdown_tables	função	(results, overfit=()) -> dict[str, str]	Tabelas Markdown do guia a partir dos resultados gravados.

pinneapple_systems.time_series.forecast_lab.plots

Figuras estáticas (matplotlib, PNG) de um estudo de previsão: skill por horizonte, sobreajuste, testes com surrogates, PIT, leques, memorizar contra aprender e calendário de surpresas.

pinneapple_systems/time_series/forecast_lab/plots.py · 268 linhas · testes: 8

nome	tipo	assinatura	o que faz
plot_skill_vs_horizon	função	(results, path, *, lyapunov=None, ncols=4, floor=-0.5, titles=None, reference='best_basel…)	Um painel por série com o skill contra o horizonte de cada família.
plot_overfitting	função	(studies, path) -> str	Erros de treino, validação e teste de cada variante de treino.
plot_surrogate_tests	função	(reports, path, *, ncols=4) -> str	Erro de previsão não linear da série contra seus surrogates.
plot_pit	função	(entries, path, *, ncols=4) -> str	Histogramas de PIT.
plot_fan_charts	função	(panels, path, *, ncols=2) -> str	Distribuições de previsão no período de teste nos horizontes avaliados.
plot_memorise_vs_learn	função	(records, path) -> str	Verdade e previsões das duas variantes de memorise_vs_learn.
calendar_grid	função	(dates, values) -> dict	Organiza valores diários numa grade semana x dia da semana.
plot_surprise_calendar	função	(records, path, *, vmax=1.0) -> str	Mapas de calor em calendário dos erros do modelo e da baseline.

pinneapple_systems.time_series.forecast_lab.presets

Os zoos de séries e de modelos do estudo de previsão quase aleatória (examples/near_random_forecasting).

pinneapple_systems/time_series/forecast_lab/presets.py · 345 linhas · testes: 20

nome	tipo	assinatura	o que faz
Case	classe	(name, data, protocol, specs, truth, title)	Caso do estudo: dados, protocolo, specs dos modelos, verdade e título.
lorenz63_missing_xz_rhs	função	(x, sigma=10.0, rho=28.0, beta=8.0 / 3.0)	Lorenz-63 sem o termo -x z: modelo físico errado de propósito.
baseline_specs	função	(arima=True) -> list[ModelSpec]	Lista de ModelSpec das baselines (ARIMA opcional).
data_specs	função	(lag_steps=(1,), lags=(8, 16), seq_lags=32, epochs=60, patience=8, max_train=3000, cols=N…)	Specs de GBM, MLP, LSTM, TCN e transformer sobre janelas defasadas.
case	função	(name, *, n=6000, seed=0, cache_dir=None) -> Case	Constrói um caso do estudo pelo nome.
overfitting_variants	função	(c, physics=False) -> dict[str, Any]	Variantes de treino de um MLP (e de GBM), de muito sobreajustada a regularizada.

pinneapple_systems.time_series.forecast_lab.renders

Renders 3-D de um estudo de previsão via Blender Cycles (pinneapple_blender.cycles): paisagem de skill, cones de previsão, atratores e fibras de ensemble.

pinneapple_systems/time_series/forecast_lab/renders.py · 314 linhas · testes: 8

nome	tipo	assinatura	o que faz
landscape_tiles	função	(results, *, floor=-0.5, titles=None) -> list[dict[str, Any]]	Um ladrilho por caso com o skill de cada família por horizonte.
render_landscape	função	(tiles, path, *, ncols=4, resolution=(1600, 1000), samples=64, height_scale=1.2) -> str	Desenha os ladrilhos como superfícies 3-D de skill.
dense_forecast	função	(data, protocol, make, max_horizon, *, quantiles=(0.05, 0.25, 0.5, 0.75, 0.95)) -> dict[s…)	Ajusta e prevê todas as origens de teste em todos os horizontes, com quantis e cobertura.
render_cones	função	(lanes, path, *, resolution=(1280, 720), samples=64, frames_dir=None, n_frames=0, history…)	Desenha cones de previsão (faixas 50 % e 90 %) por raia.
embedding_views	função	(x, *, delay=None, n_points=3000, seed=0) -> dict[str, Any]	Imersões por atraso da série e de seus surrogates.
render_attractors	função	(views, path, *, resolution=(1280, 480), samples=64, titles=…) -> str	Renderiza as imersões por atraso como atratores 3-D.
ensemble_fibres	função	(step, x0, *, n_members=50, rel_perturbation=0.01, n_steps=300, scale=None, seed=0, compo…)	Integra membros perturbados de um estado e devolve suas trajetórias e o espalhamento.
render_ensembles	função	(panels, path, *, resolution=(1280, 900), samples=48) -> str	Desenha as fibras de ensemble com uma parede em um tempo de Lyapunov.

pinneapple_systems.time_series.forecast_lab.study

Protocolo de avaliação: fixa primeiro o período de teste, faz tuning em dobras walk-forward com purga no resto, ajusta uma vez, avalia uma vez no teste intocado e testa as diferenças.

pinneapple_systems/time_series/forecast_lab/study.py · 450 linhas · testes: 50

nome	tipo	assinatura	o que faz
Protocol	classe	(horizons, test_frac, val_frac, n_folds, fold_frac, inner_val_frac, min_history, test_str…)	Divisão de uma série em teste, validação e dobras de tuning, com horizontes e histórico mínimo.
ModelSpec	classe	(name, family, factory, grid)	Um previsor do zoo: fábrica, família e grade de configurações.
tune	função	(spec, data, protocol, *, budget=None, seed=0) -> dict[str, Any]	Avalia configurações nas dobras walk-forward com purga e escolhe a melhor.
evaluate	função	(model, data, origins, *, levels=LEVELS, keep=False) -> dict[str, Any]	Escores pontuais e probabilísticos por horizonte nas origens dadas.
skill_horizon	função	(horizons, skill, p_values=None, alpha=0.05) -> int | None	Maior horizonte em que o skill é positivo (e significativo, se houver p-valores).
run_zoo	função	(data, specs, protocol, *, budget=None, keep_forecasts=(), keep_models=False, log=print, …)	Faz tuning, ajuste e avaliação de todo o zoo e testa as diferenças.
merge_zoo	função	(parts) -> dict[str, Any]	Junta resultados de run_zoo rodados em partes.
overfitting_study	função	(data, protocol, variants, *, schemes=('purged_walk_forward',), schemes_for=None, log=pri…)	Compara erros de variantes de treino sob vários esquemas de validação.
surrogate_trained	função	(make, data, protocol, *, kind='iaaft', seed=0) -> dict[str, Any]	Skill de um modelo treinado em surrogates da série (gêmeo sem estrutura).

pinneapple_systems.time_series.foundation_models

Wrappers de modelos fundacionais pré-treinados reais (Amazon Chronos e Google TimesFM) baixados do HuggingFace Hub, para previsão zero-shot; dependem de chronos-forecasting e timesfm.

pinneapple_systems/time_series/foundation_models.py · 379 linhas · testes: 14 · palavras-chave no código (heurística; o texto acima diz o status real): levanta NotImplementedError em algum caminho

nome	tipo	assinatura	o que faz
ChronosForecaster	classe	(model_id='amazon/chronos-t5-tiny', device_map='cpu', dtype=None, **pipeline_kwargs)	Wrapper do Chronos pré-treinado da Amazon (pacote chronos-forecasting), com previsão por quantis e amostras.
TimesFMForecaster	classe	(model_id=…, max_context=512, max_horizon=128, torch_compile=False, **forecast_config_kwa…)	Wrapper do TimesFM pré-treinado do Google (pacote timesfm), com contexto e horizonte máximos.

pinneapple_systems.time_series.losses

Perda `mse_forecast` no formato de dicionário (`supervised`, `total`) usado pelos trainers. [sem teste]

`pinneapple_systems/time_series/losses.py` · 9 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`mse_forecast`	função	`(model, y_hat, batch)`	Perda MSE entre `y_hat` e o alvo do batch, no formato de dicionário dos trainers.

pinneapple_systems.time_series.metrics

Métricas de previsão em torch (MAE, MSE, RMSE, MAPE, sMAPE, R²) em duas assinaturas: `(y_hat, batch)` e `(y_hat, y)`. [sem teste direto]

`pinneapple_systems/time_series/metrics.py` · 104 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`mae`	função	`(y_hat, batch)`	Erro absoluto médio a partir de `(y_hat, batch)`.
`mse`	função	`(y_hat, batch)`	Erro quadrático médio a partir de `(y_hat, batch)`.
`rmse`	função	`(y_hat, batch)`	Raiz do erro quadrático médio a partir de `(y_hat, batch)`.
`mape`	função	`(y_hat, batch)`	Erro percentual absoluto médio (%) a partir de `(y_hat, batch)`.
`smape`	função	`(y_hat, batch)`	Erro percentual absoluto simétrico (%) a partir de `(y_hat, batch)`.
`r2`	função	`(y_hat, batch)`	Coeficiente de determinação R² a partir de `(y_hat, batch)`.
`mae_fn`	função	`(y_hat, y) -> torch.Tensor`	MAE entre dois tensores.
`mse_fn`	função	`(y_hat, y) -> torch.Tensor`	MSE entre dois tensores.
`rmse_fn`	função	`(y_hat, y) -> torch.Tensor`	RMSE entre dois tensores.
`mape_fn`	função	`(y_hat, y) -> torch.Tensor`	MAPE (%) entre dois tensores.
`smape_fn`	função	`(y_hat, y) -> torch.Tensor`	sMAPE (%) entre dois tensores.
`r2_fn`	função	`(y_hat, y) -> torch.Tensor`	R² entre dois tensores.
`default_ts_metrics`	função	`()`	Dicionário padrão das métricas `mae`, `mse`, `rmse`, `mape`, `smape` e `r2`.

pinneapple_systems.time_series.metrics_ext

Reexporta métricas pontuais, probabilísticas e de comparação de previsões.

`pinneapple_systems/time_series/metrics_ext/__init__.py` · 28 linhas · testes: 20

pinneapple_systems.time_series.metrics_ext.comparison

Comparação de previsões: skill score, teste de Diebold-Mariano (com correção HLN) e correções de múltiplas comparações (Holm, Benjamini-Hochberg).

`pinneapple_systems/time_series/metrics_ext/comparison.py` · 127 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`DMResult`	classe	`(statistic, p_value, p_value_better, mean_loss_difference, n, max_lag)`	Resultado do teste de Diebold-Mariano: estatística, p-valores, diferença média de perda, n e defasagem.
`skill_score`	função	`(score, reference) -> np.ndarray`	Skill `1 - score / referência`, elemento a elemento.
`diebold_mariano`	função	`(e1, e2, *, h=1, loss='se', max_lag=None, hln=True) -> DMResult`	Teste de Diebold-Mariano de igualdade de perda esperada entre duas séries de erros.
`holm`	função	`(p_values)`	P-valores ajustados pelo método step-down de Holm.
`benjamini_hochberg`	função	`(p_values)`	P-valores ajustados por Benjamini-Hochberg (q-valores).

pinneapple_systems.time_series.metrics_ext.point

Métricas pontuais em NumPy (MAE, RMSE, MAPE, sMAPE, MASE, MAE por horizonte) e o agregador `PointMetrics`.

`pinneapple_systems/time_series/metrics_ext/point.py` · 81 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`PointMetrics`	classe	`(season_length)`	Calcula de uma vez as métricas pontuais, incluindo MASE com `season_length`.
`mae`	função	`(y_pred, y_true) -> float`	Erro absoluto médio em NumPy.
`rmse`	função	`(y_pred, y_true) -> float`	Raiz do erro quadrático médio em NumPy.
`mape`	função	`(y_pred, y_true, eps=1e-08) -> float`	Erro percentual absoluto médio em NumPy.
`smape`	função	`(y_pred, y_true, eps=1e-08) -> float`	Erro percentual absoluto simétrico em NumPy.
`mase`	função	`(y_pred, y_true, y_train, season_length=1, eps=1e-08) -> float`	Erro absoluto escalado (MASE) pelo erro do ingênuo sazonal no treino.
`multi_horizon_mae`	função	`(y_pred, y_true) -> Dict[str, float]`	MAE global e por passo do horizonte.

pinneapple_systems.time_series.metrics_ext.probabilistic

Métricas probabilísticas: pinball, cobertura, largura de intervalo, PIT gaussiano e de ensemble, histograma de PIT e `ProbabilisticMetrics`.

`pinneapple_systems/time_series/metrics_ext/probabilistic.py` · 109 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ProbabilisticMetrics`	classe	`(quantiles)`	Calcula de uma vez as métricas probabilísticas para uma lista de quantis.
`pinball_loss`	função	`(y_pred_q, y_true, q) -> float`	Perda pinball de um quantil.
`coverage`	função	`(y_lo, y_hi, y_true) -> float`	Fração das observações dentro do intervalo [lo, hi].
`mean_interval_width`	função	`(y_lo, y_hi) -> float`	Largura média dos intervalos.
`pit_gaussian`	função	`(mu, sigma, y) -> np.ndarray`	Valores PIT para previsões gaussianas (média e desvio).
`pit_ensemble`	função	`(samples, y, rng=None) -> np.ndarray`	PIT aleatorizado de um ensemble (membros no eixo 0).
`pit_histogram`	função	`(pit, n_bins=10) -> Dict[str, object]`	Frequências relativas do PIT em classes iguais e seu desvio absoluto médio da uniforme.
`gaussian_interval_coverage`	função	`(mu, sigma, y, level=0.9) -> float`	Cobertura empírica do intervalo gaussiano de nível dado.
`ensemble_interval_coverage`	função	`(samples, y, level=0.9) -> float`	Cobertura empírica do intervalo de quantis de um ensemble.

pinneapple_systems.time_series.models

Reexporta os modelos de previsão (clássicos de ML, LSTM/GRU, N-BEATS, TCN, TFT); o `FNOForecaster` é importado só se disponível. [sem teste direto]

`pinneapple_systems/time_series/models/__init__.py` · 29 linhas · **sem teste direto**

pinneapple_systems.time_series.models.classical

Previsores de ML tabular com interface `fit(X, y)`/`predict(X)`: XGBoost, LightGBM, RandomForest, CatBoost, GPR e MLP do scikit-learn; as bibliotecas são importadas sob demanda.

pinneapple_systems/time_series/models/classical.py · 315 linhas · testes: 151 · palavras-chave no código (heurística; o texto acima diz o status real): levanta NotImplementedError em algum caminho

nome	tipo	assinatura	o que faz
ForecastModel	classe	()	Base abstrata dos previsores tabulares (fit, predict, predict_with_uncertainty, score); o fit base levanta NotImplementedError.
XGBoostForecaster	classe	(n_estimators=300, max_depth=6, learning_rate=0.05, subsample=0.8, colsample_bytree=0.8, …)	Regressor multi-saída XGBoost como previsor.
LightGBMForecaster	classe	(n_estimators=300, num_leaves=31, learning_rate=0.05, subsample=0.8, **kwargs)	Previsor sobre LightGBM.
RandomForestForecaster	classe	(n_estimators=200, max_depth=None, min_samples_leaf=1, **kwargs)	Previsor RandomForest do scikit-learn, com incerteza pela dispersão das árvores.
CatBoostForecaster	classe	(iterations=300, depth=6, learning_rate=0.05, **kwargs)	Previsor sobre CatBoost.
GPRForecaster	classe	(kernel=None, alpha=0.0001, normalize_y=True, **kwargs)	Regressão por processo gaussiano, naturalmente probabilística, indicada para poucos dados.
MLPForecaster	classe	(hidden_layer_sizes=(128, 64), activation='relu', max_iter=1000, learning_rate_init=0.001…)	Previsor MLP do scikit-learn (MLPRegressor).

pinneapple_systems.time_series.models.fno_forecaster

Previsor FNO-1D multi-horizonte sobre FourierNeuralOperator de pinneapple_neural, com histórico e preenchimento com zeros. [sem teste direto]

pinneapple_systems/time_series/models/fno_forecaster.py · 73 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
FNOForecastConfig	classe	(input_len, horizon, n_features, n_targets, width, modes, layers)	Configuração do FNOForecaster (comprimentos, features, largura, modos, camadas).
FNOForecaster	classe	(cfg)	Previsor de séries com FNO-1D: entrada (B, L_in, F), saída (B, H, C).

pinneapple_systems.time_series.models.nbeats

N-BEATS (Oreshkin et al., 2019) com pilhas de tendência, sazonalidade e genérica.

pinneapple_systems/time_series/models/nbeats.py · 127 linhas · testes: 151

nome	tipo	assinatura	o que faz
NBeatsConfig	classe	(input_len, horizon, n_features, stack_types, n_blocks, n_layers, layer_width, degree_of_…)	Configuração do N-BEATS (pilhas, blocos, camadas, grau do polinômio, harmônicos).
NBeats	classe	(cfg=None, **kw)	Rede N-BEATS com pilhas de tendência, sazonalidade e genérica que devolve TSOutput.

pinneapple_systems.time_series.models.recurrent

Previsores LSTM e GRU em PyTorch que saem do último estado oculto para todo o horizonte.

pinneapple_systems/time_series/models/recurrent.py · 64 linhas · testes: 151

nome	tipo	assinatura	o que faz
RecurrentConfig	classe	(input_len, horizon, n_features, n_targets, hidden_size, num_layers, dropout, bidirection…)	Configuração dos previsores LSTM/GRU.
LSTMForecaster	classe	(cfg=None, **kw)	Previsor multi-passo com LSTM (é um nn.Module, sem fit/predict).
GRUForecaster	classe	(cfg=None, **kw)	Previsor multi-passo com GRU (é um nn.Module, sem fit/predict).

pinneapple_systems.time_series.models.tcn

Rede convolucional temporal (TCN) com dilatações exponenciais.

pinneapple_systems/time_series/models/tcn.py · 74 linhas · testes: 151

nome	tipo	assinatura	o que faz
TCNConfig	classe	(input_len, horizon, n_features, n_targets, n_channels, n_layers, kernel_size, dropout)	Configuração da TCN.
TCNForecaster	classe	(cfg=None, **kw)	TCN com dilatações exponenciais que devolve TSOutput.

pinneapple_systems.time_series.models.tft

Temporal Fusion Transformer (Lim et al., 2021) em versão simplificada.

pinneapple_systems/time_series/models/tft.py · 197 linhas · testes: 151

nome	tipo	assinatura	o que faz
TFTConfig	classe	(input_len, horizon, n_features, n_targets, hidden_size, num_heads, num_lstm_layers, drop…)	Configuração do TFT.
TFTForecaster	classe	(cfg=None, **kw)	Temporal Fusion Transformer simplificado que devolve TSOutput.

pinneapple_systems.time_series.predictability

Medidas de previsibilidade para calcular antes de ajustar qualquer modelo: AR, entropias, expoente de Lyapunov (Rosenstein), surrogates (embaralhado, fase aleatória, IAAFT), erro de previsão não linear e relatório consolidado.

pinneapple_systems/time_series/predictability.py · 350 linhas · testes: 20

nome	tipo	assinatura	o que faz
ARFit	classe	(order, intercept, coef, sigma, r2_one_step, criterion)	Resultado do ajuste AR: ordem, coeficientes, sigma, R² de um passo e critério.
SurrogateTest	classe	(statistic, kind, observed, surrogates, p_value, alternative, reject_at_0_05)	Resultado de um teste com surrogates: estatística observada, valores nos surrogates e p-valor.
fit_ar	função	(x, max_order=20, criterion='aic') -> ARFit	AR(p) por mínimos quadrados com ordem escolhida por AIC/BIC na mesma amostra.
permutation_entropy	função	(x, order=5, delay=1, normalize=True) -> float	Entropia de Shannon dos padrões ordinais.
sample_entropy	função	(x, m=2, r=None) -> float	Entropia amostral -log(A/B).
delay_embedding	função	(x, dim, delay) -> np.ndarray	Imersão por atraso de uma série em dimensão e atraso dados.
first_acf_zero	função	(x, max_lag=None) -> int	Primeira defasagem em que a autocorrelação cai abaixo de 1/e (regra para o atraso).
rosenstein_lyapunov	função	(x, *, dim=5, delay=None, dt=1.0, min_tsep=None, n_steps=40, fit=(5, 30), max_points=3000…)	Maior expoente de Lyapunov estimado pelo método de Rosenstein.
shuffle_surrogate	função	(x, rng) -> np.ndarray	Surrogate por permutação: mantém a distribuição e destrói a estrutura temporal.
phase_randomized_surrogate	função	(x, rng) -> np.ndarray	Surrogate de Fourier: mesmo espectro, fases aleatórias.
iaaft_surrogate	função	(x, rng, n_iter=200, tol=1e-08) -> np.ndarray	Surrogate IAAFT (amplitude e espectro preservados iterativamente).
nonlinear_prediction_error	função	(x, *, dim=4, delay=1, horizon=1, k=10, train_frac=0.5) -> float	Erro fora da amostra de um previsor local por vizinhos mais próximos.
time_reversal_asymmetry	função	(x, lag=1) -> float	Terceiro momento normalizado dos incrementos; zero em média para processos reversíveis.
surrogate_test	função	(x, statistic, *, kind='iaaft', n_surrogates=99, alternative='less', seed=0, name=None) -…)	Teste de posto de uma estatística contra seus valores em surrogates.

nome	tipo	assinatura	o que faz
`baseline_errors`	função	`(x, horizons, train_frac=0.8) -> dict[str, dict[int, float]]`	RMSE da persistência e da climatologia por horizonte.
`predictability_report`	função	`(x, *, horizons=(1, 2, 4, 8, 16, 32), dt=1.0, max_ar_order=20, pe_order=5, embed_dim=4, n…)`	Relatório consolidado de previsibilidade (AR, entropias, Lyapunov, surrogates, baselines).

pinneapple_systems.time_series.preparation

Reexporta o pipeline de preparação (imputação, outliers, reamostragem). [sem teste direto]

`pinneapple_systems/time_series/preparation/__init__.py` · 6 linhas · **sem teste direto**

pinneapple_systems.time_series.preparation.imputer

`TimeSeriesImputer`: trata valores ausentes com estratégias de interpolação em camadas, respeitando um tamanho máximo de lacuna.

`pinneapple_systems/time_series/preparation/imputer.py` · 126 linhas · testes: 151

nome	tipo	assinatura	o que faz
`TimeSeriesImputer`	classe	`(strategy='layered', max_gap=None, verbose=False)`	Imputador em camadas de valores ausentes de séries temporais.

pinneapple_systems.time_series.preparation.outliers

`OutlierDetector`: detecta outliers (IQR móvel, z-score etc.) e os trata por interpolação, winsorização ou outra opção.

`pinneapple_systems/time_series/preparation/outliers.py` · 155 linhas · testes: 151

nome	tipo	assinatura	o que faz
`OutlierDetector`	classe	`(method='rolling_iqr', window=20, iqr_factor=1.5, z_threshold=3.0, treatment='interpolate…)`	Detector e tratador de outliers em séries temporais.

pinneapple_systems.time_series.preparation.resampler

`TimeSeriesResampler`: sobe ou desce a frequência de DataFrames numéricos indexados por tempo.

`pinneapple_systems/time_series/preparation/resampler.py` · 122 linhas · testes: 151

nome	tipo	assinatura	o que faz
`TimeSeriesResampler`	classe	`(target_freq, time_col=None, agg_method='mean', upsample_method='linear', numeric_only=Tr…)`	Reamostra um DataFrame indexado por tempo para a frequência alvo.

pinneapple_systems.time_series.problem

`ForecastProblemSpec`: definição formal e imutável do problema de previsão (colunas, horizonte, objetivo, métricas, quantis) a ser feita antes da modelagem. [sem teste direto]

`pinneapple_systems/time_series/problem.py` · 54 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ForecastProblemSpec`	classe	`(freq, time_col, target_cols, feature_cols, exog_past_cols, exog_future_cols, input_len, …)`	Definição formal do problema de previsão; `to_timeseries_spec` gera o `TimeSeriesSpec` de janelamento.

pinneapple_systems.time_series.registry

Catálogo de modelos de séries (`TSModelCatalog`) sobre o `ModelRegistry` e registro idempotente do previsor padrão `ts_fno`.

pinneapple_systems/time_series/registry.py · 82 linhas · testes: 151

nome	tipo	assinatura	o que faz
TSModelCatalog	classe	(families, default_name)	Catálogo que lista e constrói modelos de séries no ModelRegistry e monta o FNO padrão.

pinneapple_systems.time_series.solar_cycle

Envelope do ciclo solar a partir do número diário de manchas (SILSO): EMD/EEMD, a IMF mais próxima de 11 anos, sua amplitude de Hilbert e ajuste por ELM com restrição de periodicidade (TFC), estendido além dos dados.

pinneapple_systems/time_series/solar_cycle.py · 274 linhas · testes: 16

nome	tipo	assinatura	o que faz
Decomposition	classe	(imfs, method, backend, trials, noise_width)	Resultado da decomposição: IMFs, método, backend, tentativas e ruído.
HilbertResult	classe	(amplitude, phase, instantaneous_frequency)	Amplitude, fase e frequência instantâneas de uma IMF.
PeriodicFit	classe	(rmse, fitted, forecast_t, forecast, hidden_units, backend, endpoint_error)	Resultado do ajuste ELM periódico: RMSE, curva ajustada, previsão e erro no ponto final.
parse_silso_csv	função	(source)	Converte um CSV diário do SILSO em DataFrame com índice de datas.
fetch_silso_daily	função	(url=SILSO_DAILY_URL, timeout=30.0)	Baixa e lê a série diária atual do SILSO (rede; falha se não conseguir).
synthetic_daily	função	(start='2005-01-01', end='2025-06-30', *, period_years=11.0, peak=180.0, seed=0)	Série diária sintética com a forma da do SILSO, para testes e demos offline.
preprocess	função	(df)	Normaliza tempo para [0, 1] e valores por min-max.
decompose	função	(y, method='emd', *, max_imf=6, trials=16, noise_width=0.05, seed=0, backend='auto') -> D…)	Decomposição EMD (determinística) ou EEMD (com ruído).
imf_statistics	função	(imfs, dt_days=1.0) -> list[dict[str, Any]]	Estatísticas por IMF, incluindo período dominante em dias e anos.
closest_to_schwabe_cycle	função	(stats) -> tuple[dict | None, str | None]	Escolhe a IMF de período mais próximo de 11 anos e avisa se estiver longe.
hilbert_analysis	função	(imf, t) -> HilbertResult	Amplitude, fase e frequência instantâneas de uma IMF pela transformada de Hilbert.
periodic_elm_fit	função	(t, y, *, hidden_units=50, forecast_horizon_frac=1.0, n_forecast_points=None, seed=0, bac…)	Ajuste por mínimos quadrados de uma ELM com base que impõe f(0) = f(1), e extrapolação.
run_pipeline	função	(df, *, decomposition_method='emd', max_imf=6, eemd_trials=16, eemd_noise_width=0.05, imf…)	Executa o pipeline completo: preparo, decomposição, escolha da IMF, Hilbert e ajuste ELM.

pinneapple_systems.time_series.spec

TimeSeriesSpec: comprimento de entrada, horizonte, passo e deslocamento do alvo.

pinneapple_systems/time_series/spec.py · 13 linhas · testes: 151

nome	tipo	assinatura	o que faz
TimeSeriesSpec	classe	(input_len, horizon, stride, target_offset)	Especificação de janela: input_len, horizon, stride e target_offset.

pinneapple_systems.time_series.synthetic

Séries sintéticas com verdade conhecida para estudos de previsão: Lorenz-63, Lorenz-96, Mackey-Glass, Ornstein-Uhlenbeck, oscilador estocástico, poço duplo, ruído branco e passeio aleatório, com expoentes de Lyapunov e steppers.

pinneapple_systems/time_series/synthetic.py · 364 linhas · testes: 50

nome	tipo	assinatura	o que faz
SyntheticSeries	classe	(name, t, state, obs, dt, target, observed, params, noise_std, noise_level, lyapunov, kin…)	Série sintética com sua verdade: estado, observação, parâmetros, ruído e expoente de Lyapunov.
rk4_step	função	(f, x, dt)	Um passo de Runge-Kutta clássico (NumPy ou torch, em lote).
lorenz63_rhs	função	(x, sigma=10.0, rho=28.0, beta=8.0 / 3.0)	Lado direito das equações de Lorenz-63.
lorenz96_rhs	função	(x, F=8.0)	Lado direito das equações de Lorenz-96.
mackey_glass_rhs	função	(x, x_tau, beta=0.2, gamma=0.1, n=10.0)	Lado direito da equação com atraso de Mackey-Glass.
double_well_drift	função	(x, a=1.0, b=1.0)	Deriva de uma partícula no potencial quártico de poço duplo.
mackey_glass_step	função	(hist, dt, beta=0.2, gamma=0.1, n=10.0) -> np.ndarray	Avança históricos de Mackey-Glass por um passo RK4 (método dos passos).
largest_lyapunov_benettin	função	(step, x0, *, step_time, n_steps=20000, transient=2000, d0=1e-08, seed=0) -> float	Maior expoente de Lyapunov pelo método de Benettin.
mackey_glass_lyapunov	função	(*, dt_int=0.1, total_time=50000.0, transient_time=2000.0, tau=17.0, beta=0.2, gamma=0.1,…)	Maior expoente de Lyapunov do Mackey-Glass.
lorenz63	função	(n=6000, *, dt=0.05, sub=5, noise=0.1, seed=0, sigma=10.0, rho=28.0, beta=8.0 / 3.0, obse…)	Gera uma série de Lorenz-63 com ruído de observação.
lorenz96	função	(n=6000, *, K=40, F=8.0, dt=0.05, sub=5, noise=0.1, seed=0, lyapunov_steps=20000) -> Synt…)	Gera uma série de Lorenz-96 com K variáveis.
mackey_glass	função	(n=6000, *, dt=3.0, dt_int=0.1, tau=17.0, beta=0.2, gamma=0.1, n_exp=10.0, noise=0.1, see…)	Gera uma série de Mackey-Glass.
ornstein_uhlenbeck	função	(n=6000, *, dt=0.1, theta=1.0, sigma=1.0, noise=0.1, seed=0) -> SyntheticSeries	Gera um processo de Ornstein-Uhlenbeck por amostragem exata.
oscillator_transition	função	(omega, zeta, sigma, dt)	Discretização exata de um oscilador amortecido com ruído branco.
stochastic_oscillator	função	(n=6000, *, dt=0.25, omega=1.0, zeta=0.05, sigma=0.3, noise=0.1, seed=0) -> SyntheticSeri…)	Gera um oscilador amortecido forçado por ruído branco.
double_well	função	(n=6000, *, dt=0.1, sigma=0.7, sub=100, a=1.0, b=1.0, noise=0.1, seed=0) -> SyntheticSeri…)	Gera dinâmica de Langevin num poço duplo (Euler-Maruyama).
white_noise	função	(n=6000, *, seed=0) -> SyntheticSeries	Gera ruído branco (sem nada a prever).
random_walk	função	(n=6000, *, seed=0) -> SyntheticSeries	Gera um passeio aleatório.
ode_stepper	função	(rhs, dt, sub=1) -> Callable[[np.ndarray], np.ndarray]	Mapa que avança um lote de estados de uma EDO por dt com RK4.
mackey_glass_stepper	função	(dt_int=0.1, sub=10, **params) -> Callable[[np.ndarray], np.ndarray]	Mapa que avança históricos de Mackey-Glass.
linear_sde_stepper	função	(Phi, Q, seed=0) -> Callable[[np.ndarray], np.ndarray]	Mapa x -> Phi x + w para uma EDE linear discretizada.

pinneapple_systems.time_series.tuning

Reexporta os afinadores Optuna e temporal_split. [sem teste direto]

pinneapple_systems/time_series/tuning/__init__.py · 4 linhas · **sem teste direto**

pinneapple_systems.time_series.tuning.optuna_tuner

Busca de hiperparâmetros com Optuna para os previsores clássicos e neurais, com divisão temporal em três partes.

pinneapple_systems/time_series/tuning/optuna_tuner.py · 265 linhas · testes: 151

nome	tipo	assinatura	o que faz
ClassicalTuner	classe	(model_type='xgboost', n_trials=30, direction='minimize', metric='mae', timeout=None)	Busca de hiperparâmetros com Optuna para previsores de ML clássicos.

nome	tipo	assinatura	o que faz
`NeuralTuner`	classe	`(model_type='lstm', n_trials=20, epochs_per_trial=20, lr=0.001, batch_size=64, input_len=…)`	Busca de hiperparâmetros com Optuna para previsores neurais (LSTM etc.).
`temporal_split`	função	`(X, y, val_frac=0.15, test_frac=0.15)`	Divisão cronológica em treino, validação e teste.

`pinneapple_systems.time_series.uncertainty`

Reexporta `QuantileConfig`, `QuantileHead` e `QuantileLoss`. [shim] [sem teste]

`pinneapple_systems/time_series/uncertainty/__init__.py` · 2 linhas · **sem teste direto**

`pinneapple_systems.time_series.uncertainty.quantile`

[shim] Reexporta a regressão por quantis de `pinneapple_analysis.uncertainty.quantile` para usuários antigos de `pinneapple_timeseries`. [sem teste]

`pinneapple_systems/time_series/uncertainty/quantile.py` · 8 linhas · **sem teste direto**

`pinneapple_systems.time_series.validation`

Reexporta os divisores de validação e o executor de backtest.

`pinneapple_systems/time_series/validation/__init__.py` · 16 linhas · testes: 20

`pinneapple_systems.time_series.validation.backtest`

`BacktestRunner`: treina e avalia um modelo em cada dobra de validação temporal e agrega as métricas.

`pinneapple_systems/time_series/validation/backtest.py` · 111 linhas · testes: 151

nome	tipo	assinatura	o que faz
`BacktestConfig`	classe	`(reset_model_each_fold, shuffle_train, val_batch_size)`	Configuração do backtest (reiniciar modelo por dobra, embaralhar treino, batch de validação).
`BacktestResult`	classe	`(folds, agg)`	Resultado do backtest: métricas por dobra e agregadas.
`BacktestRunner`	classe	`(*, trainer_factory, model_factory, datamodule, splits, train_cfg, preprocess_factory=Non…)`	Treina e avalia um modelo em cada dobra temporal.

`pinneapple_systems.time_series.validation.splitters`

Divisores de validação temporal: janela expansiva, janela móvel, walk-forward e k-fold com purga/embargo, e k-fold embaralhado (mantido só para medir o otimismo).

`pinneapple_systems/time_series/validation/splitters.py` · 163 linhas · testes: 151

nome	tipo	assinatura	o que faz
`Split`	classe	`(fold, train_idx, val_idx, test_idx)`	Uma dobra de backtest em índices do dataset em janelas.
`ExpandingWindowSplitter`	classe	`(initial_train_size, val_size, step_size, gap, max_folds)`	Walk-forward com janela de treino crescente.
`RollingWindowSplitter`	classe	`(train_size, val_size, step_size, gap, max_folds)`	Walk-forward com janela de treino de tamanho fixo.
`PurgedWalkForwardSplitter`	classe	`(n_folds, val_size, purge, embargo, window, min_train)`	Walk-forward sobre origens de previsão com purga e embargo contra vazamento.
`PurgedKFoldSplitter`	classe	`(n_folds, purge, embargo)`	K dobras contíguas com purga antes e embargo depois da validação.
`ShuffledKFoldSplitter`	classe	`(n_folds, seed)`	K-fold embaralhado comum, que ignora o tempo; mantido só para medir o otimismo.

`pinneapple_systems.time_series.viz`

Reexporta os gráficos de previsão e a animação. [sem teste]

`pinneapple_systems/time_series/viz/__init__.py` · 18 linhas · **sem teste direto**

`pinneapple_systems.time_series.viz.animation`

Animação de previsão móvel com bandas de incerteza e exportação opcional para MP4. [sem teste]

`pinneapple_systems/time_series/viz/animation.py` · 114 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`animate_rolling_forecast`	função	`(y_true, y_pred_seq, y_std_seq=None, horizon=16, context_len=64, interval=80, sigmas=(1.0…)`	Anima a previsão móvel com bandas de incerteza; exporta MP4 opcionalmente.

`pinneapple_systems.time_series.viz.forecast`

Gráficos de previsão: previsão móvel com bandas, horizonte com contexto, paridade, resíduos e comparação de backtests. [sem teste]

`pinneapple_systems/time_series/viz/forecast.py` · 209 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`plot_rolling_forecast`	função	`(y_true, y_pred, y_std=None, sigmas=(1.0, 2.0, 3.0), title='Rolling Forecast', xlabel='Ti…)`	Previsão móvel com bandas de 1, 2 e 3 sigmas.
`plot_forecast_horizon`	função	`(y_context, y_true, y_pred, y_std=None, sigmas=(1.0, 2.0), title='Forecast', figsize=(12,…)`	Contexto, verdade futura e previsão com bandas.
`plot_parity`	função	`(y_true, y_pred, title='Parity Plot', figsize=(5, 5), ax=None)`	Dispersão real contra previsto com diagonal de referência.
`plot_residuals`	função	`(y_true, y_pred, title='Residual Analysis', figsize=(14, 4))`	Três painéis de resíduos: no tempo, histograma e Q-Q.
`plot_backtest`	função	`(y_true, predictions, title='Backtest Comparison', figsize=(14, 5))`	Sobrepõe previsões de vários modelos à verdade.

Exemplos de uso reais do repositório

Previsão adaptativa com especialistas padrão — `examples/time_series/adaptive_forecasting.py`

```
def main(out="adaptive_forecasting.png"):
    y = make_series()
    run = AdaptiveForecaster(default_experts(season_length=12), horizon=3).run(y, start=60)
    err = run.errors(h=1)
    for name in sorted(err, key=lambda k: err[k]["mae"]):
        print(f"{name:26s} MAE {err[name]['mae']:.3f}")
    print("interval coverage (90 % target):", round(run.coverage(1), 3))
    print("leading model changes (first 10):", run.switches()[:10])
```

Execução de um grafo de co-simulação com gravador — `examples/pinneapple_cosim/01_spring_mass_pinn.py`

```
engine_eval = CoSimEngine(
    graph_eval, recorder=recorder, loop_solver="gauss_seidel", max_iter=1
)
engine_eval.reset()
engine_eval.initialize_ports(_INIT_PORTS)

with torch.no_grad():
    engine_eval.run(T=T_END, dt=DT)

traj_x1 = recorder.get("mass1", "x1")
traj_v1 = recorder.get("mass1", "v1")
```

Sinal PRBS de dois níveis — `tests/test_excitation.py`

```
        def test_prbs_two_levels_and_switches_only_on_clock():
            n, dwell, lo, hi = 1000, 0.05, -2.0, 3.0
            u = actuator_series("prbs", n, lo=lo, hi=hi, seed=7, dwell=dwell)
            assert set(np.unique(u)) == {lo, hi}
            hold = int(round(dwell * n))
            switches = np.flatnonzero(np.diff(u)) + 1           # first sample of each new level
            assert switches.size > 0 and np.all(switches % hold == 0)
```

Gêmeo que sabe quando errou (missão completa) — `examples/autolab_digital_twin/twin_knows_when_wrong.py`

```
                run = run_mission(a.text, out_dir=a.out, planner=a.planner, render=render, n_per_kind=n,
 workers=a.workers)
                st = run.state
                b = st["benchmark"]["benchmark"]
                ex.log_metric("cause_accuracy", b["accuracy"], "fraction", f"{b['n_cases']} controlled cases")
                ex.log_metric("cases_with_false_alarm", b["cases_with_false_alarm"], "cases",
                              "alarm before the fault, or any alarm on a healthy case")
```

Comissionar o gêmeo do trocador de calor — `tests/test_digital_twin_product.py`

```
            spec = TS.from_yaml(APP / "spec_hx201.yaml")
            hist = generate_history(hours=36.0, sampling_s=600.0)

            def model(X, theta):
                return exchanger_model(X, theta, ua_clean=10_200.0)

            ev = commission(spec, model, hist, n_ens=48, seed=0, warmup_steps=10)
            assert ev.commissioned, ev.render()
            assert ev.tracking.alarm_lead_s.get("T_ho", 0) > 0
```

Gêmeo ao vivo do tanque via MQTT embutido — `tests/test_operations_plant_twin.py`

```
            lt = LiveTwin(make_twin(LumpedTankModel()), speed=1000.0).start()
            try:
                deadline = time.time() + 10
                while lt.twin.n_updates < 30 and time.time() < deadline:
                    time.sleep(0.1)
                assert lt.twin.n_updates >= 30 and lt.broker is not None and lt.broker.n_published >= 30
                f = lt.inject_fault("leak", 5e-4)
                assert f["kind"] == "leak" and lt.sim.faults
```

Testes e status de validação

- Módulos com pelo menos um teste que os importa: 104 de 182.
- Funções de teste em arquivos que importam algum módulo do pacote: 579.

Itens do catálogo de métodos (`pinneapple_catalog/method_status.json`) cujo código está neste pacote:

id	método	status	testes contra referência	outros testes
E22	Euler-Bernoulli beam	validated	1	11
E23	von Karman beam / plate	validated	2	6
E44	Reaction network / Arrhenius / mass action	tested	0	10
E47	Colebrook-White	tested	0	8

id	método	status	testes contra referência	outros testes
E49	Herschel-Bulkley + Metzner-Reed (non-Newtonian pipe flow)	tested	0	4
E51	Lame thick-walled cylinder	tested	0	3
E52	von Mises equivalent stress	tested	0	11
E53	Euler buckling + constrained rod buckling	validated	1	6
E54	Beam-column moment amplification factor	tested	0	3
E55	S-N curve + Goodman + Miner	validated	1	14
E56	Torsional stick-slip with Stribeck friction	tested	0	6
E57	Heat exchanger (effectiveness-NTU)	tested	0	8
E59	Control valve (Cv, compressible / incompressible)	tested	0	1
E60	Polytropic compression path	tested	0	1
E61	Real gas equation of state (CoolProp)	tested	0	2
E62	Compressor similarity map	validated	2	0
S20	Co-simulation (graph + engine, algebraic loops by Tarjan SCC)	tested	0	11
S21	Digital twin state estimation: KF/EKF/UKF/EnKF, prognostics, signal reconstruction	tested	2	39
S23	Explicit equation-system solver (evaluation order)	tested	0	1
S25	Causal telemetry conditioning (Hampel one-step, rate limit, first-order filter, gaps)	validated	1	6
T13	Uncertainty: heteroscedastic, B-PINN, MC Dropout, deep ensembles	tested	0	20

Limitações conhecidas

- O atalho `forecast` da raiz chama `cls(horizon=...)`, `fit(data)` e `predict(horizon)`, mas `LSTMForecaster`, `GRUForecaster`, `NBeats`, `TCNForecaster` e `TFTForecaster` são `nn.Module` sem `fit`/`predict`, e o padrão `auto` vira `lstm`; o atalho não tem teste e não parece funcionar como descrito.
- `cosim_from_models` cria todos os nós com portas fixas `input` e `output`; conexões como `fluid.T` do exemplo no docstring falham na validação de portas.
- Muitas dependências são opcionais e importadas sob demanda: `statsmodels`, `pmdarima`, `ruptures`, `PyEMD`, `xgboost`, `lightgbm`, `catboost`, `optuna`, `sympy`, `chronos-forecasting`, `timesfm`, Blender (`pinneapple_blender`) e ffmpeg; sem elas, as funções correspondentes falham ou devolvem erro.
- Os módulos `enso.data`, `datasets.public`, `solar_cycle.fetch_silso_daily` e `foundation_models` precisam de rede para baixar dados ou pesos; os dados do SILSO têm licença não comercial (CC BY-NC 4.0).
- `NotImplementedError`: `TSModelBase.forward`, `ForecastModel.fit`, `Forecaster._predict_mean` (base), `ColumnSubset._fit` (não usado) e `ChronosForecaster.predict_samples` para pipelines Chronos sem amostragem.
- `uncertainty` e `uncertainty.quantile` são shims que reexportam de `pinneapple_analysis`; o docstring de `foundation_models` cita um `time_series.classical_ts` que não existe no pacote.
- Vários módulos não têm teste direto: `cosimulation.connection`, `cosimulation.utils`, `audit`, `csv_loader`, `eda`, `viz`, `features`, `baselines.naive`, `fft_lstm`, `hht_lstm`, `metrics`, `enso.plots`, `enso.study` e partes do `forecast_lab` (`base`, `baselines`, `hybrid`, `learned`).
- `registry` importa `fno_forecaster` sem `try/except`, então importar `time_series` exige `pinneapple_neural` com o FNO, embora `models/__init__` trate esse import como opcional.

- `digital_twin.assimilation.kalman` é só um shim: os filtros EKF/EnKF vivem em `pinneapple_analysis.state_estimation.kalman`.
- Muitas partes dependem de bibliotecas opcionais: PyTorch (`DigitalTwin`, reconstrução por modelo), paho-mqtt, kafka-python, asyncua, pymodbus (fluxos), PyWavelets, PyYAML (spec), BaSyx (AAS), trimesh, CasADi (MPC/ROM), psycopg (Postgres), Blender `bpy` (renders) e CoolProp (todo `real_gas_eos` e o que depende dele).
- O broker MQTT embutido não tem persistência, QoS 2, will nem TLS; o próprio docstring manda usar um broker de produção diante de equipamento real. A fonte OPC UA de `operations.live` é marcada como experimental.
- `operations.control` funciona só em simulação/recomendação: `recommend` não envia nada à planta. As plantas de `operations` (tanque, trocador) e de `pipe_loop` são simuladas; o histórico do trocador é declaradamente sintético.
- Vários módulos não têm teste que os importe diretamente: `realtime_pipeline`, `signal_reconstruction`, `state`, os módulos internos de `product` e de `pipe_loop`, `ball_study`, `kin_study`, `pc_study` e `plots` (alguns são cobertos pelos testes do pacote pai).
- Escopos declarados nos docstrings: `pipe_network_1d` cobre um único segmento de tubo; o transporte de `reaction_kinetics` só aceita grade periódica; `curved_path_geometry` é só 2D (o caso 3D não está implementado); `recession` é um só reservatório exponencial, sem chuva nem roteamento.
- A política de deriva de `product` detecta que o gêmeo errou, não o porquê, e nunca re-treina sozinha; a spec não valida a física, só que existe uma afirmação falsificável.
- Futebol: os dados (Metrica, SkillCorner, StatsBomb) são baixados em tempo de execução e têm termos de uso próprios (StatsBomb não comercial). O veredito gravado em `docs/guides/football_physics.md` registra que a física não ajudou no xG nem no ponto de aterrissagem contra árvores treinadas.
- `physics_residuals` levanta `NotImplementedError` quando o modelo não tem saídas suficientes para o resíduo pedido; os resíduos são estacionários e simplificados (sem força de corpo, decaimento de primeira ordem).
- `SWAGApproximation` usa só a variância diagonal, não a parte de baixo posto do SWAG completo.
- `edge` exige onnx/onnxruntime; `EdgeRuntime` não importa torch nem pinneapple.
- Os seis componentes de referência são exemplos de arquitetura e física, sem dados reais; o exemplo usa alvos sintéticos aleatórios.
- O conjunto de toolboxes instaladas vive só na memória do processo (`ToolboxRegistry`).
- Sem teste direto pelo inventário: todos os módulos de `component_library` exceto o `__init__`, e `component_modeling` `mc_dropout`, `mpc` e o `__init__`.

3. `pinneapple_twin3d`

Objetivo

`pinneapple_twin3d` é a camada de visualização 3D de gêmeos digitais do PINNeAPPle. Você monta uma `Scene` em Python (partes de malha, campos por vértice ao longo do tempo, movimentos, visibilidade, propriedades físicas e sensores com séries e envelopes de alarme) e exporta para uma pasta estática: `scene.json`, `geometry.glb`, `fields.bin` e o visualizador web embutido (pasta `viewer/`). `serve` abre essa pasta por HTTP.

O pacote resolve o problema de mostrar resultados de simulação e de monitoramento sobre a geometria real, com o tempo como eixo. Há várias entradas de geometria: primitivas (`primitives`), a biblioteca de geometrias de `pinneapple_design` (`library`), casos OpenFOAM com valores por face (`openfoam`), scans limpos e cenas MuJoCo (`scan`). Além do glTF, `usd` exporta OpenUSD com campos como time samples e física `UsdPhysics`, para Omniverse e Isaac Sim; a cena também vira cena de `pp.viz` (`to_studio`) e pode ser renderizada no Blender.

O subpacote `wear` é um gerador completo de gêmeos de desgaste de revestimento (refratário, liner, tubo): especificação do vaso por zonas de revolução, dataset de espessura, previsão linear de vida restante com intervalo e confiança, backtest, otimização de paradas de manutenção (apoio à decisão), calibração a partir de registros e campanhas sintéticas claramente rotuladas. Segundo o docstring do pacote, o visualizador fica na biblioteca para ser reaproveitado pelo PINNeAPPle-CFD, pelos apps e por `pinneapple_systems.digital_twin`.

Números do inventário

módulos	linhas	classes públicas	funções públicas	testes em arquivos que importam o pacote	módulos sem teste direto
19	2.820	19	67	74 (7 arquivos)	4

Submódulos

submódulo	o que faz	módulos	classes + funções	testes*
`demo`	Cena de demonstração de curva de tubo com desgaste ilustrativo.	1	1	6
`gallery`	Galeria de peças variadas construídas com primitivas, com campos sintéticos.	1	11	16
`library`	Alias da biblioteca de geometrias de `pinneapple_design`.	1	0	0
`openfoam`	Leitura de casos OpenFOAM para cenas com valores por face.	1	6	6
`primitives`	Primitivas de malha (revolução, varredura, loft, asa, caixa, cilindro).	1	12	16
`scan`	Limpeza de malhas de scan e importação de cenas MuJoCo.	1	4	4
`scene`	Modelo da cena e exportação para glTF + campos + visualizador.	1	5	9
`serve`	Servidor HTTP da pasta exportada.	1	1	6
`usd`	Exportação OpenUSD com campos no tempo e física.	1	1	5
`wear`	Gerador de gêmeos de desgaste: modelo, previsão, otimização, calibração e presets.	9	45	30

* maior número de testes que importam um módulo do submódulo.

Módulos e API pública

`pinneapple_twin3d`

Fachada do pacote: reexporta `Scene` e `serve` para montar uma cena de gêmeo digital 3D em Python e abri-la no visualizador web embutido.

`pinneapple_twin3d/__init__.py` · 26 linhas · testes: 34

`pinneapple_twin3d.demo`

Cena de demonstração: curva de tubo de 90 graus com desgaste por erosão crescendo no tempo e dois sensores; o campo é ilustrativo, não resultado de CFD.

`pinneapple_twin3d/demo.py` · 68 linhas · testes: 6

nome	tipo	assinatura	o que faz
`pipe_bend_scene`	função	`(years=(0, 1, 2, 3, 4, 5)) -> Scene`	Monta a cena de demonstração da curva de tubo com campo de desgaste ilustrativo por ano e dois sensores.

`pinneapple_twin3d.gallery`

Galeria de gêmeos 3D de peças muito diferentes (canal de corrida, tanque, oleoduto, avião, foguete, drone, rover lunar, turbina eólica, satélite) montados com primitivas e campos sintéticos.

`pinneapple_twin3d/gallery.py` · 438 linhas · testes: 16

nome	tipo	assinatura	o que faz
`get`	função	`(key) -> Scene`	Retorna a cena da galeria pelo nome da função registrada; levanta KeyError para nome desconhecido.
`runner_channel`	função	`() -> Scene`	Canal de corrida de alto-forno: calha em U revestida de refratário do furo de gusa ao skimmer.
`mixing_tank`	função	`() -> Scene`	Tanque agitado com fundo abaulado, chicanas e impulsor de dois estágios; mistura de traçador no tempo.
`oil_pipeline`	função	`() -> Scene`	Oleoduto com duas curvas e uma depressão; perda de parede por corrosão/erosão ao longo dos anos.

nome	tipo	assinatura	o que faz
`airliner`	função	`() -> Scene`	Avião de corredor único (fuselagem, asa enflechada, cauda, dois motores); temperatura de pele e carga na raiz da asa num voo.
`rocket`	função	`() -> Scene`	Lançador com nariz ogival, tanques, bocal e aletas; aquecimento aerodinâmico e ablação do bocal na subida.
`drone`	função	`() -> Scene`	Quadricóptero; aquecimento de motores e bateria durante um voo.
`lunar_rover`	função	`() -> Scene`	Rover lunar de seis rodas num ciclo dia-noite lunar: temperatura do chassi e desgaste das rodas.
`wind_turbine`	função	`() -> Scene`	Turbina de três pás; erosão de bordo de ataque e fadiga ao longo dos anos.
`satellite`	função	`() -> Scene`	Satélite pequeno em órbita baixa; ciclos térmicos entre sol e eclipse.
`export_all`	função	`(folder) -> str`	Exporta todas as peças da galeria em subpastas e grava um `index.html` com links para elas.

pinneapple_twin3d.library

[shim] Alias de `pinneapple_design.geometry_library`: expõe a biblioteca de geometrias do lado do gêmeo. [sem teste]

`pinneapple_twin3d/library.py` · 8 linhas · **sem teste direto**

pinneapple_twin3d.openfoam

Lê um caso OpenFOAM (pontos, faces, boundary, diretórios de tempo) e monta uma cena com os patches de parede como superfícies e valores por face ao longo do tempo, como mapas de erosão.

`pinneapple_twin3d/openfoam.py` · 215 linhas · testes: 6

nome	tipo	assinatura	o que faz
`read_points`	função	`(case) -> np.ndarray`	Lê o arquivo `constant/polyMesh/points` do caso como array (N, 3).
`read_faces`	função	`(case) -> List[np.ndarray]`	Lê `polyMesh/faces` como lista de arrays de índices de vértices.
`read_boundary`	função	`(case) -> Dict[str, Dict[str, object]]`	Lê `polyMesh/boundary` e devolve tipo, face inicial e número de faces de cada patch.
`time_directories`	função	`(case) -> List[Tuple[float, str]]`	Lista os diretórios de tempo numéricos do caso, ordenados, como (tempo, nome).
`read_boundary_field`	função	`(path, patch, n_faces) -> Optional[np.ndarray]`	Lê os valores por face de um patch num arquivo volScalarField/volVectorField (vetores viram módulo).
`scene_from_case`	função	`(case, *, title=None, patches=None, face_values=None, units=None, boundary_fields=(), tim…)`	Monta uma cena a partir de um caso OpenFOAM, com patches de parede como partes e campos vindos de valores por face ou de campos de contorno.

pinneapple_twin3d.primitives

Primitivas de malha para montar peças sem CAD: revolução, varredura, loft, perfil de aerofólio, asa, caixa, cilindro, transformações, junção e checagem de qualidade.

`pinneapple_twin3d/primitives.py` · 183 linhas · testes: 16

nome	tipo	assinatura	o que faz
`revolve`	função	`(profile, n_theta=48, *, cap=False) -> Mesh`	Superfície de revolução de um perfil (r, y) em torno do eixo y, com tampas opcionais.
`sweep`	função	`(section, path, *, closed=True, up=(0, 1, 0)) -> Mesh`	Extruda uma seção 2D ao longo de um caminho 3D, aberta ou fechada.
`airfoil`	função	`(chord=1.0, thickness=0.12, n=24) -> np.ndarray`	Contorno fechado de aerofólio simétrico NACA 00xx.

nome	tipo	assinatura	o que faz
loft	função	(rings) -> Mesh	Malha de pele através de anéis com o mesmo número de pontos.
wing	função	(span, root_chord, tip_chord, *, sweep_deg=0.0, dihedral_deg=0.0, twist_deg=0.0, thicknes…)	Meia asa ou pá com enflechamento, diedro, torção e afilamento.
box	função	(size, center=(0, 0, 0)) -> Mesh	Caixa com tamanho e centro dados.
cylinder	função	(p0, p1, radius, n_theta=32, *, cap=True) -> Mesh	Cilindro (ou tronco de cone) entre dois pontos.
rotation	função	(axis, angle_deg) -> np.ndarray	Matriz de rotação 3x3 em torno de um eixo, ângulo em graus.
transform	função	(mesh, *, R=None, t=(0, 0, 0), scale=1.0) -> Mesh	Aplica rotação, translação e escala a uma malha.
mirror_z	função	(mesh) -> Mesh	Espelha a malha no plano z=0, corrigindo a orientação das faces.
merge	função	(meshes) -> Mesh	Junta várias malhas numa só, reindexando as faces.
mesh_quality	função	(mesh) -> dict	Números de sanidade da malha (coordenadas finitas, índices válidos, fração de triângulos degenerados, limites) usados nos testes.

pinneapple_twin3d.scan

Geometria de gêmeo a partir de escaneamentos: limpa malhas de scan (vértices duplicados, fragmentos, buracos, alinhamento do piso, metros) e converte cenas MuJoCo MJCF em partes; exige `trimesh`.

pinneapple_twin3d/scan.py · 199 linhas · testes: 4

nome	tipo	assinatura	o que faz
ScanReport	classe	(vertices_in, faces_in, vertices_out, faces_out, components_dropped, holes_filled, watert…)	Relatório da limpeza do scan: contagens antes/depois, fragmentos removidos, buracos preenchidos, estanqueidade e eixo vertical.
clean_scan	função	(path_or_mesh, *, unit='m', min_component_fraction=0.01, fill_holes=True, target_faces=No…)	Limpa uma malha de scan e devolve (malha trimesh em metros, ScanReport).
add_scan	função	(scene, name, path_or_mesh, *, group='scan', **kw) -> ScanReport	Limpa um scan e o adiciona como parte de uma cena, devolvendo o relatório.
scene_from_mjcf	função	(path, *, title=None, include_primitives=True) -> Scene	Cria uma parte por geom de uma cena MuJoCo MJCF, em coordenadas do mundo.

pinneapple_twin3d.scene

Núcleo do pacote: modelo da cena (partes, campos no tempo, movimentos, visibilidade, física, sensores) e exportação para `scene.json` + `geometry.glb` + `fields.bin` + visualizador, com USD opcional.

pinneapple_twin3d/scene.py · 360 linhas · testes: 9

nome	tipo	assinatura	o que faz
Part	classe	(name, vertices, faces, group, color, fields, motion, visibility, physics)	Uma parte da cena: vértices, faces, grupo, cor, campos, movimento, visibilidade e propriedades físicas.
PhysicsMaterial	classe	(name, static_friction, dynamic_friction, restitution, density)	Material de contato (atrito estático e dinâmico, restituição, densidade) no estilo UsdPhysics.
PhysicsProps	classe	(rigid_body, collider, kinematic, mass, density, center_of_mass, approximation, material,…)	Como a parte entra numa simulação física: corpo rígido, colisor, cinemático, massa, material e velocidades.
Sensor	classe	(id, position, label, unit, quantity, series, envelope)	Sensor com id, posição, unidade, grandeza, série temporal e envelope de alarme.

nome	tipo	assinatura	o que faz
`Scene`	classe	`(title, *, length_unit='m', times=None, time_unit='s', source='')`	Cena de gêmeo digital: adiciona partes, campos, movimentos, física e sensores e exporta para pasta (com visualizador, USD opcional, render Blender).

`pinneapple_twin3d.serve`

Serve por HTTP a pasta exportada (o visualizador precisa de HTTP, não `file://`).

`pinneapple_twin3d/serve.py` · 26 linhas · testes: 6

nome	tipo	assinatura	o que faz
`serve`	função	`(folder, port=8765, *, host='127.0.0.1', block=True)`	Serve a pasta exportada em `http://host:port/`; com `block=False` devolve o servidor rodando.

`pinneapple_twin3d.usd`

Exporta a cena como OpenUSD texto (`.usda`) com campos por vértice como time samples, sensores como esferas e propriedades `UsdPhysics`, para Omniverse/Isaac Sim/usdview.

`pinneapple_twin3d/usd.py` · 209 linhas · testes: 5

nome	tipo	assinatura	o que faz
`export_usd`	função	`(scene, path) -> str`	Grava a cena em `.usda` e devolve o caminho.

`pinneapple_twin3d.wear`

Gerador de gêmeos de desgaste (refratário, revestimento, tubo) a partir de uma especificação: spec -> dataset -> previsão -> otimização -> cena + relatório.

`pinneapple_twin3d/wear/__init__.py` · 21 linhas · testes: 30

`pinneapple_twin3d.wear.build`

Converte um `WearDataset` em cena 3D, contrato do hub (`wear_dataset.json`) e relatório de previsão/otimização (`report.json`). [sem teste direto]

`pinneapple_twin3d/wear/build.py` · 95 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`remaining_life_fields`	função	`(ds, cap, **kw) -> dict`	Vida restante por célula em cada leitura, limitada a `cap`.
`scene_from_wear`	função	`(ds, *, life_cap=None, **fit_kw) -> Scene`	Monta a cena 3D do dataset de desgaste, com campos de desgaste, fração e vida restante por célula.
`report`	função	`(ds, cfg=None, **fit_kw) -> dict`	Relatório que mantém separados o observado, o previsto e o recomendado.
`export_wear_twin`	função	`(ds, folder, *, cfg=None, usd=False, **fit_kw) -> str`	Grava cena 3D, `wear_dataset.json` e `report.json` numa pasta.

`pinneapple_twin3d.wear.calibrate`

Calibra as hipóteses do otimizador (horas de parada, custos, fração de recuperação) a partir de registros de paradas e do próprio dataset, informando a fonte e o erro padrão de cada parâmetro.

`pinneapple_twin3d/wear/calibrate.py` · 138 linhas · testes: 30

nome	tipo	assinatura	o que faz
`StopRecord`	classe	`(zones, duration_h, cost, x)`	Registro de uma parada de manutenção: zonas atendidas, duração, custo e posição x.

nome	tipo	assinatura	o que faz
`ParamReport`	classe	`(name, value, source, n, se, note)`	Relatório de um parâmetro calibrado: valor, fonte, tamanho de amostra, erro padrão e nota.
`Calibration`	classe	`(cfg, report)`	Resultado da calibração: `OptimizerConfig` ajustado e relatório por parâmetro.
`estimate_recovery`	função	`(ds, repair_drop=0.08, min_cells=20)`	Mediana da fração da espessura útil recuperada por célula reparada, e quantas células foram usadas.
`calibrate`	função	`(stops=(), ds=None, base=None) -> Calibration`	Ajusta tempos e custos de parada por mínimos quadrados e a recuperação a partir do dataset, mantendo o padrão quando o ajuste não é confiável.

pinneapple_twin3d.wear.forecast

Previsão de vida restante por célula por regressão linear da espessura residual numa janela recente, com intervalo t de Student de 90 % e índice de confiança; inclui histórico e backtest.

`pinneapple_twin3d/wear/forecast.py` · 185 linhas · testes: 30

nome	tipo	assinatura	o que faz
`CellFit`	classe	`(remaining, remaining_lo, remaining_hi, slope, r2, n, confidence)`	Ajuste de uma célula: vida restante com limites, inclinação, R2, número de pontos e confiança.
`ZoneForecast`	classe	`(zone, worst_fraction, n_over, min_remaining, rate, usable, critical_cell, confidence, r2…)`	Previsão de uma zona: pior fração de desgaste, células acima do limite, menor vida restante, taxa, célula crítica e confiança.
`t90`	função	`(df)`	Quantil t de Student para intervalo bilateral de 90 % por graus de liberdade, interpolado de uma tabela.
`confidence_index`	função	`(n, r2)`	Índice de confiança 0-3 (n/a, baixa, média, alta) a partir do número de pontos e do R2.
`fit_zone`	função	`(ds, zone, s, *, win_x=DEFAULT_WIN_X, min_pts=8, repair_drop=0.08) -> CellFit`	Ajuste linear por célula de uma zona na leitura `s`, parando no último reparo detectado.
`forecast`	função	`(ds, s=None, **kw) -> Dict[str, ZoneForecast]`	Previsão por zona na leitura indicada (por padrão a última).
`forecast_history`	função	`(ds, **kw) -> List[dict]`	Menor vida restante prevista recalculada em cada leitura, mostrando como a previsão evoluiu.
`backtest`	função	`(ds, **kw) -> dict`	Compara previsões passadas de quando a primeira célula atinge o limite com o que de fato aconteceu.
`status`	função	`(fraction) -> str`	Classifica a fração de desgaste em `normal`, `attention`, `alert` ou `critical`.

pinneapple_twin3d.wear.geometry

Malhas de células para superfícies de revolução: amostragem do perfil, uma quad por célula, mapeamento célula->vértice e centros das células.

`pinneapple_twin3d/wear/geometry.py` · 62 linhas · testes: 30

nome	tipo	assinatura	o que faz
`sample_profile`	função	`(profile, n_rows) -> np.ndarray`	Pontos igualmente espaçados em comprimento de arco ao longo do perfil (r, y).
`cell_mesh`	função	`(zone) -> Tuple[np.ndarray, np.ndarray]`	Vértices e faces de uma zona, uma quad (dois triângulos) por célula.
`cell_to_vertex`	função	`(values) -> np.ndarray`	Converte valores por célula em valores por vértice na ordem de `cell_mesh`.
`cell_centers`	função	`(zone) -> np.ndarray`	Posição no mundo do centro de cada célula, usada para posicionar sensores e pontos quentes.

pinneapple_twin3d.wear.model

Modelo de dados do gêmeo de desgaste: `ZoneSpec` (perfil de revolução cortado em células), `VesselSpec` e `WearDataset` (desgaste por zona no tempo, com contrato JSON e leitura de CSV). [sem teste direto]

`pinneapple_twin3d/wear/model.py` · 165 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ZoneSpec`	classe	`(name, label, profile, n_rows, n_sectors, e0, emin, center, theta0, group)`	Zona de revolução: perfil, linhas e setores, espessura nova e mínima, centro e grupo; `usable` dá a espessura útil.
`VesselSpec`	classe	`(key, title, zones, x_name, wear_unit, notes)`	Especificação do vaso: chave, título, zonas, nome da variável x e unidade de desgaste.
`WearDataset`	classe	`(spec, x, wear, origin, labels, note)`	Desgaste em mm por zona com forma (T, linhas, setores), NaN sem leitura; converte de/para o contrato JSON do hub e lê CSV.

pinneapple_twin3d.wear.optimize

Agendamento de manutenção (gunning/reparo) sobre a previsão: simula políticas reativa, calendário fixo e preditiva agrupada e recomenda a mais barata sem violação; apoio à decisão apenas.

`pinneapple_twin3d/wear/optimize.py` · 166 linhas · testes: 30

nome	tipo	assinatura	o que faz
`OptimizerConfig`	classe	`(horizon, heats_per_day, recovery_frac, setup_hours, hours_per_zone, cost_per_hour, cost_…)`	Parâmetros do otimizador: horizonte, recuperação, horas e custos de parada, custo de violação, margem de segurança e janelas.
`ZoneState`	classe	`(zone, deadline, life_full, ext)`	Estado de uma zona para o otimizador: prazo até o limite, vida completa e extensão por intervenção.
`Policy`	classe	`(id, name, schedule, stops, zones_served, downtime_hours, breach, cost, window)`	Uma política simulada: agenda de paradas, zonas atendidas, horas paradas, violação e custo.
`OptimizationResult`	classe	`(cfg, zones, policies, recommended, baseline, notes)`	Resultado: zonas, políticas simuladas, política recomendada, linha de base e notas.
`prepare`	função	`(zones, cfg) -> List[ZoneState]`	Converte previsões de zona em `ZoneState` usando a configuração.
`simulate`	função	`(zs, stops, cfg) -> dict`	Simula uma agenda de paradas e devolve custo, horas paradas e violação.
`fixed_policy`	função	`(zs, cfg) -> List[dict]`	Gera a agenda de paradas em calendário fixo.
`predictive_policy`	função	`(zs, cfg, window) -> List[dict]`	Gera paradas preditivas agrupando zonas cujos prazos caem dentro de uma janela.
`optimize`	função	`(forecasts, cfg=None) -> OptimizationResult`	Simula as políticas reativa, fixa e preditiva e recomenda a mais barata sem violação (ou com menor violação).
`explain`	função	`(res) -> List[dict]`	Explica em texto cada parada da política recomendada (zonas, prazo e margem).

pinneapple_twin3d.wear.presets

Especificações indicativas dos casos de referência (panela de aço, panela de gusa, conversor BOF, desgaseificador RH) e de um reator genérico de redução-fusão.

`pinneapple_twin3d/wear/presets.py` · 176 linhas · testes: 30

nome	tipo	assinatura	o que faz
`get`	função	`(key)`	Retorna (spec, pontos quentes) do preset pelo nome da função; levanta KeyError se não existir.
`fill_level`	função	`(rin, y0, y1, rho, mass, n=300) -> float`	Nível do banho para uma massa dada num vaso de raio interno variável, por integração em troncos.
`steel_ladle`	função	`()`	Preset indicativo de panela de aço.
`pig_iron_ladle`	função	`()`	Preset indicativo de panela de gusa.
`bof_converter`	função	`()`	Preset indicativo de conversor BOF.
`rh_degasser`	função	`()`	Preset indicativo de desgaseificador RH.
`smelting_reduction_reactor`	função	`()`	Preset genérico de reator de redução-fusão: cuba, zona de ventaneiras, cadinho e soleira.

`pinneapple_twin3d.wear.synthetic`

Campanhas de desgaste sintéticas (pontos quentes + lei de crescimento + reparos + ruído), sempre rotuladas SYNTHETIC; não são modelo físico. [sem teste direto]

`pinneapple_twin3d/wear/synthetic.py` · 80 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`HotSpot`	classe	`(zone, s, theta_deg, ds, dtheta_deg, amp)`	Ponto quente de desgaste numa zona: posição relativa, ângulo, larguras e amplitude.
`base_pattern`	função	`(spec, hotspots, base) -> Dict[str, np.ndarray]`	Padrão espacial de desgaste por zona a partir dos pontos quentes e de um nível base.
`synthetic_campaign`	função	`(spec, hotspots, *, n_readings=41, x_max=120.0, growth_power=1.0, base=0.2, noise=0.01, s…)`	Gera um `WearDataset` sintético com crescimento, reparos e ruído de medição.

Exemplos de uso reais do repositório

Gêmeo de desgaste para uma peça nova — `examples/use_cases/refractory_wear_twin/custom_part.py`

```
        def build(out: str) -> str:
            spec = tank_spec()
            hot = (HotSpot("wall", 0.7, None, 0.1, amp=0.6),          # a band all around at 70 % of the height
                   HotSpot("floor", 0.4, 90, 0.3, 35, amp=0.7))      # an impact pad at 90 deg, 40 % of the radius
            ds = synthetic_campaign(spec, hot, n_readings=25, x_max=80)
            print({k: round(v.min_remaining, 1) for k, v in forecast(ds).items()}, "->",
optimize(forecast(ds)).recommended.name)
            return export_wear_twin(ds, out)
```

Exportar e servir uma cena — `tests/test_twin3d.py`

```
        sc.export(str(tmp_path))
        httpd = serve(str(tmp_path), port=0, block=False)
```

Testes e status de validação

- Módulos com pelo menos um teste que os importa: 15 de 19.
- Funções de teste em arquivos que importam algum módulo do pacote: 74.

Limitações conhecidas

- A previsão de desgaste é extrapolação linear: não antecipa mudança de regime; a confiança só avalia a qualidade do ajuste e o intervalo de 90 % é de ajuste, não de predição.
- O otimizador de manutenção é apoio à decisão apenas; custos de violação e margem de segurança são entradas de negócio, nunca inferidas.
- Os presets usam geometria indicativa, não desenhos de uma planta específica; as campanhas sintéticas e o campo da demo são ilustrativos, não modelos físicos.
- `scan` exige `trimesh`; `render_blender` exige `bpy`.
- O exemplo no docstring de `pinneapple_twin3d.wear` usa `presets.get("bof")`, mas a chave registrada é `bof_converter` (o nome da função), então aquele trecho levantaria KeyError.
- Sem teste direto: `library`, `wear.build`, `wear.model` e `wear.synthetic` (são exercitados indiretamente pelos testes de `wear`).
- O visualizador precisa ser servido por HTTP; abrir o `index.html` por `file://` não carrega os dados.

Volume 7 — Ferramentas, orquestração e produtos

> Parte do Mapa completo do PINNeAPPle. Gerado por `docs/pt/_mapa/gerar.py` a partir do inventário do commit `e53cdcfb04`.

Ferramentas (benchmarks, exportação, visualização, automação de GUIs de engenharia, NVIDIA), o Flow, o Hub, o Lab, os LLMs, o servidor MCP, o laboratório autônomo e o app web.

Sumário

1. pinneapple_tools
2. pinneapple_blender
3. pinneapple_flow
4. pinneapple_hub
5. pinneapple_lab
6. pinneapple_llm
7. pinneapple_mcp
8. pinneapple_autolab
9. pinneapple_app

1. `pinneapple_tools`

Objetivo

`pinneapple_tools` é a caixa de ferramentas de apoio do PINNeAPPle: tudo o que fica em volta do treino e da solução de um problema físico, mas não é o modelo nem o solver em si. Reúne visualização no estilo CFD (`visualization`), exportação de modelos (`model_export`), o pipeline de literatura (`hpo_experiments`), a suíte de benchmarks (`benchmark_suite`) e a camada de backend PyTorch/JAX (`compute_backends`). Vários desses submódulos eram pacotes separados; o `__init__` ainda expõe os nomes antigos (`viz`, `export`, `researcher`, `arena`, `backend`) como aliases de compatibilidade.

Além disso, o pacote tem ferramentas de engenharia mais recentes: coeficientes aerodinâmicos a partir de pressão na superfície (`aero_coefficients`), qualidade de datasets e curadoria por modelo de visão-linguagem (`dataset_quality`), métricas e gráficos genéricos de avaliação (`model_evaluation`), relatórios técnicos em PDF/HTML (`reporting`), a política que decide se um problema precisa de surrogate (`surrogate_policy`) e a execução de scripts de solver de usuários em outro processo com limites de recurso (`sandbox`).

Três submódulos são sistemas inteiros. `automation` é o "agente de GUI de engenharia": executa um workflow em YAML (FreeCAD, CfdOF, OpenFOAM, ParaView, web) passo a passo, para quando um passo pede confirmação humana, e grava evidências (comandos, logs, SHA-256 dos arquivos, screenshots, checagens) num relatório HTML e num `StudyResult`. `model_services` publica modelos do zoo (`pp.zoo`), o trust card e o extrator de intenção como microserviços HTTP com as rotas do NVIDIA NIM, cada resposta com o seu relatório de confiança. `rea_bridge` usa a ferramenta externa REA para estudar implementações de referência, com uma trava de licença que decide o que pode ser reaproveitado.

O pacote se liga ao resto do repositório em vários pontos: `automation` usa `pinneapple_check` nas checagens, `pinneapple_simulation.pipelines.freecad_openfoam` nos adaptadores e `pinneapple_data.contracts.StudyResult` no relatório, e se registra como passo `guiagent` do runner de `pinneapple_core.workflows` (usado pelos apps `guiagent` e `simulation_runner`). `model_services` serve modelos

de pp.zoo e checagens de pinneapple_physics. rea_bridge.tools registra ferramentas no PhysicsToolRegistry de pinneapple_worldmodel para o loop de agente de pinneapple_llm. reporting é usado por estudos de pinneapple_simulation e por pinneapple_veriphysics.

Esta parte cobre quatro submódulos de pinneapple_tools. benchmark_suite (o "Arena") roda e compara modelos em problemas físicos: tarefas clássicas (Burgers, calor, onda, Poisson, Allen–Cahn, Taylor–Green, escoamentos), backends de treino (PyTorch nativo, registro de modelos, DeepXDE, JAX, PhysicsNeMo Sym), benchmarks de transferência e meta-aprendizado, pipelines declarativos e o protocolo do benchmark contra outros frameworks. hpo_experiments é, apesar do nome, o antigo pesquisador de literatura (arXiv, GitHub e um LLM Gemini). nvidia é a cola com PhysicsNeMo, OpenUSD/Omniverse, Isaac Sim e USD BRep. visualization reúne gráficos no estilo CFD, painéis animados, pontes para PyVista/Plotly/Rerun/Trame e o studio de cenas 3D.

Números do inventário

módulos	linhas	classes públicas	funções públicas	testes em arquivos que importam o pacote	módulos sem teste direto
177	34.350	155	384	465 (35 arquivos)	107

Pasta de testes dedicada: tests/pinneapple_tools (30 funções de teste).

Submódulos

submódulo	o que faz	módulos	classes + funções	testes*
aero_coefficients	Utilitários em NumPy para Cp, integração de forças de pressão/cisalhamento numa superfície triangulada e conversão para CL/CD/CM nos eixos do vento.	1	4	7
automation	Agente de GUI de engenharia: executa workflows estruturados em FreeCAD, CfdOF, OpenFOAM, ParaView e interfaces web e entrega arquivos mais a evidência do que aconteceu.	22	62	32
benchmark_suite	Arena de benchmarks: tarefas físicas, backends de treino, comparação multi-modelo, transferência, meta-aprendizado, pipelines declarativos e o benchmark entre frameworks.	52	118	155
compute_backends	Seletor global de backend (PyTorch por padrão, JAX opcional) e utilitários JAX de jit/vmap para resíduos de PINN.	3	8	156
dataset_quality	Checagens estatísticas de qualidade de datasets em nuvem de pontos e curadoria de sequências com um modelo de visão-linguagem (VLM).	3	16	16
hpo_experiments	Pesquisador automático de literatura (antigo pinneapple_researcher): descoberta no arXiv/GitHub, base de conhecimento, extração e reprodução com LLM Gemini.	27	44	0
model_evaluation	Métricas genéricas de avaliação de modelos (regressão, bootstrap, sensibilidade, calibração) e os gráficos correspondentes em matplotlib.	3	12	5
model_export	Exportação de modelos para TorchScript e ONNX e de campos (coordenadas + valores) para CSV e NPZ.	1	4	0
model_services	Modelos do PINNeAPPle servidos como microserviços HTTP com as rotas do NVIDIA NIM, cada resposta com relatório de confiança; inclui cliente e servidor.	6	17	39
nvidia	Integração com a pilha NVIDIA: stages OpenUSD, robôs para Isaac Sim, surrogates PhysicsNeMo, geometria exata USD BRep, figuras e registro de execução.	11	49	39
rea_bridge	Ponte para a ferramenta REA (engenharia reversa) com trava de licença, mapa de funcionalidades e livro de registro do que foi estudado.	6	20	22
reporting	Estilo de figuras e construtores de relatório técnico: PDF com reportlab (paperkit) e datasheet HTML/PDF/Markdown (datasheet).	3	14	14
sandbox	Executa um script de solver fornecido pelo usuário em outro processo com limites de recurso POSIX; não é uma fronteira de segurança.	2	1	13
surrogate_policy	Decide, pelo tempo medido de uma solução real, se um problema precisa de surrogate treinado ou se o solver pode ser chamado direto.	1	2	5
visualization	Visualização no estilo CFD, painéis animados, pontes para visualizadores 3D externos e o studio de cenas 3D realistas (web, Blender, PyVista).	35	165	46

* maior número de testes que importam um módulo do submódulo.

Módulos e API pública

`pinneapple_tools`

Ponto de entrada do pacote: reexporta visualização, exportação, benchmarks, backends e qualidade de dados, mantém aliases antigos (`viz`, `export`, `researcher`, `arena`, `backend`) e define três atalhos. [sem teste]

`pinneapple_tools/__init__.py` · 237 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`plot`	função	`(model, x, *, field_name='u', dim=None, **kwargs)`	Atalho que avalia o modelo em `x` sem gradiente e desenha a previsão com `plot_scalar` (1D ou 2D conforme a dimensão).
`export_model`	função	`(model, path, fmt='torchscript', **kwargs) -> str`	Atalho que despacha para `export_torchscript`, `export_onnx`, `export_csv` ou `export_npz` conforme `fmt`; formato desconhecido levanta `ValueError`.
`run_benchmark`	função	`(models, tasks=None, **bench_kwargs) -> 'BenchmarkResult'`	Atalho que monta um `BenchmarkConfig` e roda `PINNArenaBenchmark` sobre os modelos dados.

`pinneapple_tools.aero_coefficients`

Funções compartilhadas de aerodinâmica em NumPy puro: Cp, integração de força e momento sobre malha triangulada e rotação para os eixos do vento (CL, CD, CM).

`pinneapple_tools/aero_coefficients.py` · 293 linhas · testes: 7

nome	tipo	assinatura	o que faz
`AeroCoefficients`	classe	`(CL, CD, CM, CY)`	Dataclass com os coeficientes adimensionais CL, CD, CM e a força lateral CY, nos eixos do vento.
`compute_cp`	função	`(p, p_inf, q_inf) -> np.ndarray`	Calcula o coeficiente de pressão `Cp = (p - p_inf) / q_inf`; levanta `ValueError` se `q_inf` for zero.
`integrate_surface_forces`	função	`(vertices, faces, face_normals, face_pressure, face_shear=None, moment_center=None) -> Tu…)`	Soma pressão (e cisalhamento opcional) por face de uma superfície triangulada e devolve força e momento líquidos em torno de um centro.
`force_to_coefficients`	função	`(force, moment, q_inf, ref_area, ref_length, alpha_deg, *, drag_axis='x', lift_axis='z', …)`	Converte força/momento nos eixos do corpo em CL/CD/CM nos eixos do vento, girando pelo ângulo de ataque no plano (`drag_axis`, `lift_axis`).

`pinneapple_tools.automation`

Agente de GUI de engenharia ("Selenium para engenharia computacional"): expõe `Runner`, `Workflow`, `interpret`, `catalog` e afins para executar tarefas em softwares de engenharia com evidência.

`pinneapple_tools/automation/__init__.py` · 84 linhas · testes: 32

nome	tipo	assinatura	o que faz
`example_workflow`	função	`(name) -> Path`	Devolve o caminho do `workflow.yaml` de um exemplo versionado em `automation/examples/<name>/`.
`catalog`	função	`(folder=None) -> dict[str, Workflow]`	Carrega todos os workflows de uma pasta (por padrão os exemplos versionados), indexados pelo id.

`pinneapple_tools.automation.adapters`

Pacote de adaptadores, um por aplicação; importá-lo registra todos.

`pinneapple_tools/automation/adapters/__init__.py` · 16 linhas · testes: 16

`pinneapple_tools.automation.adapters.base`

Contrato comum dos adaptadores (`Adapter`, `ActionResult`, `StepContext`) e o registro global de adaptadores. [sem teste]

pinneapple_tools/automation/adapters/base.py · 135 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
ActionFailed	classe	()	Exceção para uma ação que rodou e falhou; a mensagem diz o motivo e o log diz onde.
ActionResult	classe	(outputs, artifacts, evidence, checks, actions)	Resultado de uma ação: saídas para os passos seguintes, arquivos produzidos, imagens de evidência, checagens e ações registradas.
StepContext	classe	(run_dir, step_dir, toolchain, display, workflow_dir, criteria, previous, timeout_s, comm…)	Contexto em que um passo roda: pastas, toolchain, display, critérios, registros anteriores, timeout, e helpers para rodar comandos com log.
Adapter	classe	(requires, actions, needs_display)	Classe base de um adaptador: declara o que requer, as ações que oferece e se precisa de display, e despacha run(action, params, ctx) para do_<ação>.
register	função	(adapter) -> Adapter	Registra uma instância de adaptador no registro global pelo nome.
get_adapter	função	(name) -> Adapter	Devolve o adaptador registrado com o nome dado.
list_adapters	função	() -> dict[str, Adapter]	Devolve o dicionário de todos os adaptadores registrados.
known_actions	função	() -> set[str]	Conjunto de todas as ações adaptador.ação conhecidas.

pinneapple_tools.automation.adapters.checks

Adaptador verify (avalia os critérios de aceitação do workflow sobre os arquivos do run) e adaptador web (abre uma UI web com Playwright e captura). [sem teste]

pinneapple_tools/automation/adapters/checks.py · 71 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
VerifyAdapter	classe	()	Adaptador verify: a ação criteria avalia cada critério do workflow e devolve as checagens.
WebAdapter	classe	()	Adaptador web: a ação capture abre uma página com Playwright e grava screenshot; só disponível com Playwright instalado.

pinneapple_tools.automation.adapters.freecad

Adaptadores FreeCAD e CfdOF: rodam um script dentro da GUI do FreeCAD no display virtual, capturam a tela e a vista 3D, e montam a análise CfdOF. [sem teste]

pinneapple_tools/automation/adapters/freecad.py · 195 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
FreeCADAdapter	classe	()	Adaptador freecad: ações open (descreve documento, parâmetros e sólido) e macro (roda uma macro com ARGS/RESULT).
CfdOFAdapter	classe	()	Adaptador cfdof: ação setup monta a análise CfdOF (laminar, estacionária, incompressível) e grava o caso de malha e o caso OpenFOAM.
run_in_freecad_gui	função	(script, args, ctx, name, view=None, hold_s=60.0) -> tuple[dict[str, Any], ActionResult]	Roda um script na GUI do FreeCAD com ARGS, espera o sinal de pronto, tira screenshots e devolve o RESULT e a evidência.

pinneapple_tools.automation.adapters.openfoam

Adaptador OpenFOAM pela linha de comando: roda Allmesh/Allrun, lê os logs com parse_log e aplica correções de compatibilidade do CfdOF com OpenFOAM v1912.

pinneapple_tools/automation/adapters/openfoam.py · 223 linhas · testes: 16

nome	tipo	assinatura	o que faz
`OpenFOAMAdapter`	classe	`()`	Adaptador `openfoam`: ações `mesh`, `solve`, `prepare` e `post`, com checagens de malha, fim do solver e convergência dos resíduos.
`patch_cfdof_allmesh`	função	`(mesh_case) -> str | None`	Edita o `Allmesh` do CfdOF para usar a sintaxe `-scale` em versões ESI sem `FOAM_API`; devolve a descrição da edição ou None.
`patch_cfdof_inlet`	função	`(case) -> list[str]`	Troca entradas `fixedNormalInletOutletVelocity` do CfdOF por `fixedValue` com a mesma velocidade normal (versões ESI antes da v2006).
`solver_log`	função	`(case) -> tuple[Path | None, dict[str, Any]]`	Encontra o log do solver de um caso (`log.simpleFoam`, `log.foamRun`, ...) e devolve o caminho e o conteúdo interpretado.
`residual_plot`	função	`(parsed, path, title) -> Path | None`	Desenha o histórico de resíduos de um log interpretado num PNG (evidência do tipo `plot`).

`pinneapple_tools.automation.adapters.paraview`

Adaptador ParaView via `pvpython`: abre um caso OpenFOAM no último tempo, exporta um campo de célula para CSV e renderiza uma fatia em PNG. [sem teste]

`pinneapple_tools/automation/adapters/paraview.py` · 70 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ParaViewAdapter`	classe	`()`	Adaptador `paraview`: ação `export` (CSV com uma linha por célula e imagem do tipo `render`).

`pinneapple_tools.automation.adapters.pipeline`

Adaptadores do pipeline FreeCAD -> OpenFOAM de `pinneapple_simulation`: só convertem os resultados desse pipeline em `ActionResult`. [sem teste]

`pinneapple_tools/automation/adapters/pipeline.py` · 57 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`FreeCADHeadlessAdapter`	classe	`()`	Adaptador `freecadcmd`: ação `export` roda o FreeCAD sem tela (ou um substituto NumPy com `backend: fake`) e exporta STEP e STL por patch.
`pipeline_call`	função	`(fn, params, ctx) -> ActionResult`	Chama uma função do pipeline com os parâmetros e a pasta do passo e converte erros comuns em `ActionFailed`.

`pinneapple_tools.automation.display`

Display virtual Xvfb, screenshots e os três drivers de GUI (xdotool, Playwright, pywinauto) usados quando a aplicação não tem API utilizável. [sem teste]

`pinneapple_tools/automation/display.py` · 278 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`GuiUnavailable`	classe	`()`	Exceção: o driver de GUI não pode rodar aqui (ferramenta ausente, plataforma errada, sem display).
`VirtualDisplay`	classe	`(width=1600, height=1000, depth=24, reuse=False)`	Display Xvfb que vive durante um bloco `with`; fornece o ambiente `DISPLAY` e screenshots via Pillow.
`XdotoolDriver`	classe	`(display)`	Encontra e age sobre janelas X11 com `xdotool` (esperar, levantar, teclar, clicar), registrando cada comando.
`PlaywrightDriver`	classe	`(headless=True, executable_path=None)`	Dirige uma UI web com Playwright (abrir, clicar, preencher, esperar texto, screenshot), registrando cada ação.
`PywinautoDriver`	classe	`(backend='uia')`	Interface para GUIs de desktop Windows via `pywinauto`; no Linux se declara indisponível.

`pinneapple_tools.automation.examples.channel_obstacle.build_project`

Script para `freecadcmd` que constrói o projeto FreeCAD paramétrico do exemplo `channel_obstacle` (canal com obstáculo, dimensões numa planilha). [sem teste]

`pinneapple_tools/automation/examples/channel_obstacle/build_project.py` · 33 linhas · **sem teste direto**

pinneapple_tools.automation.interpret

Transforma uma tarefa (frase, dict ou JSON/YAML) num workflow do catálogo e seus inputs, por regras e com LLM opcional (Ollama); uma frase só escolhe workflows revisados e define inputs.

pinneapple_tools/automation/interpret.py · 155 linhas · testes: 16

nome	tipo	assinatura	o que faz
Interpretation	classe	(workflow, inputs, method, notes, score)	Resultado da interpretação: workflow escolhido, inputs, método (regras ou LLM), notas e pontuação.
parse_quantity	função	(text, unit) -> float | None	Extrai o número de um texto e converte para a unidade pedida ("4 cm" -> 40 em mm); None se não houver número.
interpret	função	(task, catalog, llm=None, min_score=0.15) -> Interpretation	Converte a tarefa num workflow do catálogo e seus inputs, validando limites; recusa tarefas sem workflow correspondente.

pinneapple_tools.automation.record

Registros de um run: StepRecord por passo e RunRecord em volta, gravados como run.json; um passo só conta como aprovado se a ação rodou e todas as checagens e artefatos estão ok.

pinneapple_tools/automation/record.py · 171 linhas · testes: 14

nome	tipo	assinatura	o que faz
Check	classe	(name, passed, detail, value, threshold, source)	Uma checagem: nome, se passou, detalhe, valor medido, limite e fonte.
Evidence	classe	(path, kind, caption, sha256)	Uma imagem do que aconteceu, com o tipo explícito (screen, window, render, plot), legenda e SHA-256.
FileRecord	classe	(path, bytes, sha256, kind)	Registro de um arquivo produzido: caminho, tamanho, SHA-256 e tipo.
StepRecord	classe	(id, title, action, mode, state, started, ended, commands, logs, artifacts, evidence, che…)	Registro de um passo: estado, horários, comandos, logs, artefatos, evidências, checagens, saídas, confirmação e erro.
RunRecord	classe	(run_id, workflow, inputs, folder, state, created, finished, steps, environment, criteria…)	Registro do run inteiro (workflow, inputs, passos, ambiente, critérios, mensagem), com save/load em run.json.
file_record	função	(path, root) -> FileRecord	Calcula tamanho e SHA-256 de um arquivo; para pasta, tamanho total e hash sobre os hashes ordenados dos arquivos.

pinneapple_tools.automation.report

Relatório de evidências de um run: HTML autocontido com imagens embutidas e o mesmo conteúdo como StudyResult de pinneapple_data.contracts. [sem teste]

pinneapple_tools/automation/report.py · 234 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
write_report	função	(run, path) -> Path	Grava o relatório HTML autocontido do run (inputs, ferramentas, linha do tempo dos passos, critérios, erros).
to_study_result	função	(run)	Converte o run num StudyResult (study_type="runs", app="guiagent").
write_study_result	função	(run, path) -> Path	Grava o StudyResult do run em JSON.

pinneapple_tools.automation.runner

Executa um workflow passo a passo: pula passos já aprovados, bloqueia sem pré-requisitos, para em confirmações (expensive, overwrite, ...) e na primeira falha, e reescreve run.json a cada mudança. [sem teste direto]

pinneapple_tools/automation/runner.py · 290 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Runner`	classe	`(workflow, run_dir, inputs=None, toolchain=None, on_event=None, report=True)`	Executor de um workflow numa pasta de run, com inputs, aprovações e callback de eventos; `run()` executa e `decline()` encerra um run que pediu confirmação.
`plan`	função	`(workflow, toolchain=None) -> list[dict[str, Any]]`	Mostra o que o workflow precisaria sem rodar nada: por passo, adaptador, modo, se a aplicação está instalada e as confirmações pedidas.

pinneapple_tools.automation.runner_step

Registra o tipo de passo `guiagent` no runner de `pinneapple_core.workflows`, para que um passo de DAG rode um workflow de automação inteiro e devolva o veredito.

`pinneapple_tools/automation/runner_step.py` · 95 linhas · testes: 16

nome	tipo	assinatura	o que faz
`GuiAgentStep`	classe	`(toolchain=None)`	Implementação do passo `guiagent`: roda o workflow pedido na pasta do passo e devolve estado, critérios, métricas e caminhos dos relatórios.
`register`	função	`(toolchain=None) -> GuiAgentStep`	Registra o tipo de passo `guiagent` no runner de workflows e devolve o adaptador.

pinneapple_tools.automation.scripts.cfdof_setup

Script executado dentro da GUI do FreeCAD que monta uma análise CfdOF pela API Python (grupos de faces, material, malha gmsh) e grava os casos. [sem teste]

`pinneapple_tools/automation/scripts/cfdof_setup.py` · 112 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`on_plane`	função	`(face, plane)`	Diz se uma face do sólido está sobre um dos planos da caixa envolvente (`x_min`, `x_max`, ...), para classificar entrada, saída a paredes.

pinneapple_tools.automation.scripts.freecad_gui_job

Script lançado na GUI do FreeCAD que executa o job (script com `ARGS`/`RESULT`), ajusta a vista, sinaliza ao adaptador para tirar screenshots e sai. [sem teste]

`pinneapple_tools/automation/scripts/freecad_gui_job.py` · 68 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`pump`	função	`(seconds)`	Mantém o laço de eventos da GUI do FreeCAD rodando por alguns segundos.
`run_job`	função	`()`	Executa o script do job dentro do FreeCAD, grava o resultado e coordena os arquivos `ready`/`release` com o adaptador.

pinneapple_tools.automation.scripts.freecad_inspect

Script rodado dentro do FreeCAD que abre um documento e descreve objetos, parâmetros da planilha e o sólido pedido em `RESULT`. [sem teste]

`pinneapple_tools/automation/scripts/freecad_inspect.py` · 27 linhas · **sem teste direto**

pinneapple_tools.automation.scripts.paraview_export

Script para `pvpython` que abre um caso OpenFOAM, exporta um campo de célula para CSV e salva uma imagem da fatia. [sem teste]

`pinneapple_tools/automation/scripts/paraview_export.py` · 92 linhas · **sem teste direto**

pinneapple_tools.automation.toolchain

Descobre onde estão instalados FreeCAD, freecadcmd, CfdOF, OpenFOAM, gmsh e pvpython (variáveis de ambiente, locais usuais, PATH) e suas versões. [sem teste]

pinneapple_tools/automation/toolchain.py · 109 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
Toolchain	classe	(freecad, freecadcmd, cfdof, foam_bashrc, gmsh, pvpython)	Caminhos das aplicações de engenharia, com discover(), a lista do que falta e as versões encontradas.

pinneapple_tools.automation.verify

Critérios de aceitação de um run calculados dos arquivos produzidos (balanço de massa, vazão de entrada, queda de pressão, convergência, artefato, métrica, physics_lint) com as checagens de pinneapple_check.

pinneapple_tools/automation/verify.py · 176 linhas · testes: 30

nome	tipo	assinatura	o que faz
patch_fluxes	função	(case) -> dict[str, Any]	Fluxo volumétrico [m³/s] por patch de contorno no último tempo, mais o líquido e o total.
patch_mean	função	(case, field, patches) -> dict[str, float]	Média ponderada por área de um campo escalar em cada patch nomeado, no último tempo.
patch_area	função	(case, patch) -> float	Área [m²] de um patch de contorno da malha.
inflow	função	(case, inlet, velocity, tol=0.01) -> Check	Checa se a vazão na entrada bate com velocity vezes a área do patch, dentro de tol.
mass_balance	função	(case, inlet, outlet, tol=0.01) -> Check	Checa |entrada + saída| / |entrada| <= tol com os fluxos dos patches.
pressure_drop	função	(case, inlet, outlet, density=None, min_pa=None, max_pa=None) -> tuple[Check, float]	Calcula a queda de pressão média entre entrada e saída (cinemática ou vezes density) e checa contra min_pa/max_pa.
evaluate_criterion	função	(kind, params, run_dir, previous) -> Check	Avalia um critério do workflow pelo tipo e devolve um Check.

pinneapple_tools.automation.workflow

Workflow estruturado em YAML/JSON: inputs com limites, passos com pré-condições, artefatos esperados, confirmações e critérios, com templates {inputs.x}, {steps.id.out}. [sem teste direto]

pinneapple_tools/automation/workflow.py · 274 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
WorkflowError	classe	(errors)	Exceção para workflow ou inputs inválidos; errors lista todos os problemas encontrados.
ArtifactSpec	classe	(path, kind, min_bytes, description)	Arquivo ou pasta que um passo deve deixar, relativo à pasta do run, com tamanho mínimo.
InputSpec	classe	(default, min, max, unit, description, choices)	Especificação de um input: padrão, mínimo, máximo, unidade, descrição e escolhas; check valida um valor.
Step	classe	(id, action, title, description, params, after, requires, expects, confirm, timeout_s)	Um passo do workflow: ação adaptador.ação, parâmetros, dependências, requisitos, artefatos esperados, confirmações e timeout.
Criterion	classe	(id, kind, title, params)	Critério de aceitação do resultado, avaliado pela ação verify.criteria.
Workflow	classe	(id, title, steps, description, application, inputs, criteria, limits, source)	O workflow completo: carrega de arquivo, valida, resolve inputs e expõe passos e pasta de origem.
render	função	(value, ctx) -> Any	Preenche recursivamente os templates de um valor a partir do contexto; um template inteiro preserva o tipo do valor referido.

pinneapple_tools.benchmark_suite

Ponto de entrada da suíte de benchmarks: reexporta `Arena`, `PINNArenaBenchmark`, registros de tarefas e backends, relatórios e, dentro de `try/except`, os pipelines de física, séries temporais, transferência e meta-aprendizado.

`pinneapple_tools/benchmark_suite/__init__.py` · 108 linhas · testes: 2

pinneapple_tools.benchmark_suite.api

API de alto nível `Arena`: liga uma `ProblemSpec` (ou um preset, ou um YAML) ao pipeline de geração de dados, treino e métricas.

`pinneapple_tools/benchmark_suite/api.py` · 591 linhas · testes: 153

nome	tipo	assinatura	o que faz
`ArenaResult`	classe	`(spec, model, history, metrics, artifacts, model_id, elapsed_s)`	Resultado de `Arena.run()`: especificação, modelo treinado, histórico de perdas, métricas finais, artefatos e tempo gasto.
`ArenaCompareResult`	classe	`(spec, results)`	Resultado de `Arena.compare()` com vários modelos, com `leaderboard()` para ordenar por métrica.
`Arena`	classe	`(spec) -> None`	Executor de alto nível criado por `from_spec`, `from_preset` ou `from_yaml`, que treina um modelo (`run`) ou compara vários (`compare`).

pinneapple_tools.benchmark_suite.backends

Pacote dos backends de treino do Arena; importa e reexporta os cinco backends e o protocolo `Backend`. [sem teste]

`pinneapple_tools/benchmark_suite/backends/__init__.py` · 17 linhas · **sem teste direto**

pinneapple_tools.benchmark_suite.backends.base

Define o protocolo `Backend`: um objeto com `name` e um método `train(bundle, run_cfg)`.

`pinneapple_tools/benchmark_suite/backends/base.py` · 20 linhas · testes: 151

nome	tipo	assinatura	o que faz
`Backend`	classe	`(name)`	Protocolo de backend de treino: recebe um `BundleData` e uma configuração e devolve saídas de treino.

pinneapple_tools.benchmark_suite.backends.deepxde_backend

Backend DeepXDE registrado como `deepxde`: baseline supervisionado nos sensores do bundle (x, y, u, v, p opcional). [sem teste]

`pinneapple_tools/benchmark_suite/backends/deepxde_backend.py` · 113 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`DeepXDEBackend`	classe	`(name)`	Treina uma rede DeepXDE só com os dados de sensores e devolve `predict_fn` (N,2)->(N,3); falha com `RuntimeError` se `deepxde` não estiver instalado.

pinneapple_tools.benchmark_suite.backends.jax_pinn

Backend PINN completo em JAX para o caso `FlowObstacle2D` (Navier–Stokes 2D estacionário), registrado como `jax_pinn`. [sem teste]

`pinneapple_tools/benchmark_suite/backends/jax_pinn.py` · 335 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`JAXPINNBackend`	classe	`(name)`	Treina em JAX com resíduo de momento e continuidade, perda de contorno e perda opcional em sensores; devolve `predict_fn` e métricas de teste.

`pinneapple_tools.benchmark_suite.backends.native_pinn`

Backend PINN nativo em PyTorch para Navier–Stokes 2D estacionário, sem dependências externas além do torch. [sem teste]

`pinneapple_tools/benchmark_suite/backends/native_pinn.py` · 287 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`MLP`	classe	`(in_dim, out_dim, hidden, activation='tanh')`	Perceptron multicamada totalmente conectado com ativação escolhida por nome (tanh, relu, gelu, silu).
`NativePINNBackend`	classe	`()`	Treina um `MLP` em PyTorch com resíduo de Navier–Stokes nos pontos de colocação e perda de contorno do bundle.

`pinneapple_tools.benchmark_suite.backends.physicsnemo_sym`

Backend que monta um PINN de Navier–Stokes com as abstrações do PhysicsNeMo Sym (domínio, restrições, solver) e usa o laço de treino dele.

`pinneapple_tools/benchmark_suite/backends/physicsnemo_sym.py` · 240 linhas · testes: 2

nome	tipo	assinatura	o que faz
`PhysicsNeMoSymBackend`	classe	`()`	Treina um PINN 2D estacionário com PhysicsNeMo Sym; exige `nvidia-physicsnemo.sym` e não garante devolver um modelo PyTorch.

`pinneapple_tools.benchmark_suite.backends.pinneapple_models_backend`

Backend "multi-modelo" do Arena: treina qualquer modelo do `ModelRegistry` com batch unificado e perda física compilada quando suportada. [sem teste]

`pinneapple_tools/benchmark_suite/backends/pinneapple_models_backend.py` · 200 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`PinneappleModelsBackend`	classe	`()`	Carrega o bundle, monta o batch, compila a perda física se houver especificação, treina com o `Trainer` (opcionalmente DDP) e exporta previsões.

`pinneapple_tools.benchmark_suite.benchmark`

PINN Arena: benchmark multi-problema e multi-arquitetura que treina vários modelos em várias tarefas físicas e gera ranking, tabelas e gráficos. A classe base de tarefa levanta `NotImplementedError` nos métodos abstratos.

`pinneapple_tools/benchmark_suite/benchmark.py` · 1.309 linhas · testes: 155 · palavras-chave no código (heurística; o texto acima diz o status real): levanta `NotImplementedError` em algum caminho

nome	tipo	assinatura	o que faz
`BenchmarkConfig`	classe	`(n_col, n_bc, n_ic, epochs, lr, weight_pde, weight_bc, weight_ic, device, seed, n_eval, c…)`	Configuração do benchmark: número de pontos de colocação/contorno/inicial, épocas, taxa de aprendizado, pesos das perdas, dispositivo, semente e limiar de convergência.
`ModelSpec`	classe	`(name, factory, description, input_kind, family)`	Descrição de um modelo candidato: nome, fábrica que o constrói, descrição, tipo de entrada (`input_kind`) e família.
`BenchmarkResult`	classe	`(problem_id, model_id, metrics, history, elapsed_s, n_params, rank)`	Resultado de um par (problema, modelo): métricas, histórico, tempo, número de parâmetros e posição no ranking.
`BenchmarkTaskBase`	classe	`(task_id, in_dim, out_dim)`	Base abstrata de tarefa física: amostragem de colocação, contorno e condição inicial, resíduo da EDP, grade de avaliação e montagem de batch; os métodos abstratos levantam `NotImplementedError`.

nome	tipo	assinatura	o que faz
`NotApplicableError`	classe	`()`	Exceção para sinalizar que uma tarefa não monta batch para um dado `input_kind`; o próprio módulo não a levanta, só os chamadores.
`PINNArenaBenchmark`	classe	`(tasks, model_specs=None, config=None) -> None`	Suíte que roda todas as combinações tarefa × modelo, monta leaderboard, melhor modelo por problema, tabela-resumo, gráficos e salva/carrega resultados.
`all_model_specs`	função	`() -> List[ModelSpec]`	Gera um `ModelSpec` para cada modelo do registro de modelos, marcado com o `input_kind` real da família.

pinneapple_tools.benchmark_suite.bundle

Pacote de leitura e validação de bundles de dados do Arena (reexporta `loader` e `schema`). [sem teste]

`pinneapple_tools/benchmark_suite/bundle/__init__.py` · 7 linhas · **sem teste direto**

pinneapple_tools.benchmark_suite.bundle.loader

Carrega um bundle do disco (manifesto, condições, pontos de colocação e contorno, sensores) e o valida contra um esquema.

`pinneapple_tools/benchmark_suite/bundle/loader.py` · 149 linhas · testes: 2

nome	tipo	assinatura	o que faz
`BundleData`	classe	`(root, manifest, conditions, points_collocation, points_boundary, sensors)`	Contêiner imutável de um bundle validado: raiz, manifesto, condições, pontos de colocação, pontos de contorno e sensores.
`load_bundle`	função	`(bundle_root, *, schema, require_sensors=False) -> BundleData`	Lê e valida um bundle no disco segundo um `BundleSchema`, exigindo sensores quando pedido.

pinneapple_tools.benchmark_suite.bundle.schema

Esquema de validação de bundles lido de YAML: versão, arquivos, regiões, chaves do manifesto e colunas de sensores obrigatórias. [sem teste]

`pinneapple_tools/benchmark_suite/bundle/schema.py` · 119 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`BundleSchema`	classe	`(schema_version, required_files, required_regions, manifest_required_keys, sensors_requir…)`	Definição do esquema de um bundle, com `validate_bundle_root` para verificar uma pasta.
`load_bundle_schema`	função	`(path) -> BundleSchema`	Carrega um `BundleSchema` a partir de um arquivo YAML.

pinneapple_tools.benchmark_suite.crossframework

Partes neutras do benchmark entre frameworks (PINNeAPPle contra PhysicsNeMo/PhysicsNeMo-Sym, DeepXDE, neuraloperator): problemas, pontos e pesos compartilhados, avaliador único, estatísticas e registros de execução.

`pinneapple_tools/benchmark_suite/crossframework/__init__.py` · 17 linhas · testes: 20

pinneapple_tools.benchmark_suite.crossframework.darcy

Trilha B do benchmark entre frameworks: dados de Darcy (`darcy_2d_li2021`), normalização e o laço único de treino de operadores. [sem teste]

`pinneapple_tools/benchmark_suite/crossframework/darcy.py` · 182 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
DarcyData	classe	(a, u, a_mean, a_std, u_mean, u_std)	As três partições (treino, validação, teste) de Darcy com a normalização calculada no treino; x normaliza entradas e decode desfaz a normalização.
load_split	função	(split, resolution=64) -> tuple[np.ndarray, np.ndarray]	Devolve (a, u) de uma partição na resolução pedida, com média em blocos quando a resolução é menor.
load_darcy	função	(resolution=64, n_train=N_TRAIN) -> DarcyData	Carrega as três partições de Darcy e monta um DarcyData normalizado.
train_operator	função	(model, data, *, epochs, seed, batch_size=20, lr=0.001, weight_decay=0.0001, log=None) ->…)	Treina um operador (B,1,r,r)->(B,1,r,r) com o laço do protocolo e devolve histórico, tempos por época e previsões decodificadas.
run_track_b	função	(argv, *, framework, source, build, model_config) -> None	Corpo comum dos três executores da trilha B: cada executor só informa como sua biblioteca constrói o FNO.
inference_latency	função	(model, data, batch, repeats=20, warmup=3) -> float	Mediana do tempo de parede de uma passada direta sem gradientes sobre um lote de teste.

pinneapple_tools.benchmark_suite.crossframework.evaluate

Avaliador único do benchmark entre frameworks (problemas diretos, inverso e operador) e estatísticas sobre sementes.

pinneapple_tools/benchmark_suite/crossframework/evaluate.py · 141 linhas · testes: 20

nome	tipo	assinatura	o que faz
evaluate_pinn	função	(problem, u_test, u_stencil=None, *, reference=None) -> dict[str, float]	Métricas de uma execução direta a partir das previsões exportadas (erro na grade de teste e resíduo pelo estêncil).
evaluate_inverse	função	(nu_hat, nu_true, u_pred, u_ref) -> dict[str, float]	Execução inversa: erro relativo do coeficiente identificado e erro do campo nos pontos de teste.
per_sample_rel_l2	função	(pred, ref) -> np.ndarray	Erro L2 relativo por amostra, sobre todos os eixos menos o primeiro (aprendizado de operadores).
evaluate_operator	função	(pred, ref) -> dict[str, float]	Execução de operador: média, mediana e máximo do L2 relativo por amostra no teste.
seed_stats	função	(values) -> dict[str, object]	Mediana, mínimo, máximo e n dos valores finitos, contando à parte as falhas (NaN/inf) em n_failed.
paired_ratio	função	(a, b) -> dict[str, object]	Razão por semente a / b nas sementes em comum: mediana, extremos e quantas vezes a < b.
sign_test_p	função	(k, n) -> float	Valor-p exato bilateral do teste do sinal; o docstring avisa que com 5 sementes o menor p possível é 0.0625.
summarize	função	(rows, keys, by) -> list[dict[str, object]]	Agrupa linhas de resultado por campos e aplica seed_stats a cada métrica.

pinneapple_tools.benchmark_suite.crossframework.inverse

Trilha C do benchmark entre frameworks: identificar a viscosidade de Burgers a partir de amostras ruidosas.

pinneapple_tools/benchmark_suite/crossframework/inverse.py · 28 linhas · testes: 20

nome	tipo	assinatura	o que faz
observations	função	(n=C['n_obs'], noise_rel=C['noise_rel'], seed=C['obs_seed']) -> dict[str, np.ndarray]	Gera as observações do problema inverso: pontos (x, t), valores ruidosos e limpos e o desvio do ruído, com semente fixa do protocolo.

pinneapple_tools.benchmark_suite.crossframework.problems

Problemas PINN diretos e inverso do benchmark entre frameworks (Burgers, Poisson, Helmholtz, Allen–Cahn), com referências, conjuntos de pontos, grade de teste e resíduo por diferenças finitas.

pinneapple_tools/benchmark_suite/crossframework/problems.py · 314 linhas · testes: 20

nome	tipo	assinatura	o que faz
`PINNProblem`	classe	`(name, coords, bounds, params, time_dependent, fd_step, citation, test_grid, extra)`	Um problema do benchmark: coordenadas, limites, parâmetros, se depende do tempo, passo do estêncil, citação e grade de teste; expõe condição inicial, contorno, fonte, solução exata e resíduo.
`get_problem`	função	`(name) -> PINNProblem`	Busca um problema por nome no dicionário `PROBLEMS`; levanta `KeyError` com a lista disponível se não existir.
`make_point_set`	função	`(problem, *, n_interior=10000, n_boundary=400, seed=20260) -> dict[str, np.ndarray]`	Gera os pontos de treino que todo framework carrega (protocolo A1): colocação interna uniforme, pontos e valores de contorno e, se houver tempo, condição inicial.
`point_set_digest`	função	`(points) -> str`	SHA-256 dos arrays do conjunto de pontos, para cada execução registrar em quais pontos treinou.
`save_point_set`	função	`(points, path) -> str`	Grava o conjunto de pontos em `.npz` e devolve o seu digest.
`load_point_set`	função	`(path) -> dict[str, np.ndarray]`	Lê um conjunto de pontos de um arquivo `.npz`.
`test_points`	função	`(problem) -> np.ndarray`	Grade de teste estruturada (float64, N x 2), a mesma para todos os frameworks.
`residual_points`	função	`(problem, n=2000, seed=777) -> np.ndarray`	Pontos internos novos, nunca usados no treino, para medir o resíduo, afastados do contorno por um passo de estêncil.
`stencil`	função	`(problem, X) -> np.ndarray`	Estêncil de 5 pontos em torno de cada ponto de resíduo, no formato (N, 5, 2).
`residual_from_stencil`	função	`(problem, X, values) -> np.ndarray`	Calcula o resíduo da EDP a partir dos valores no estêncil por diferenças centrais de segunda ordem.
`allen_cahn_reference`	função	`(d=0.0001, n=2048, dt=0.0001, t_end=1.0, n_save=1000) -> tuple[np.ndarray, np.ndarray, np…)`	Solução de referência numérica de Allen–Cahn periódica em [-1, 1) com u(x,0) = x² cos(πx).
`callable_exact`	função	`(problem) -> Callable[[np.ndarray], np.ndarray]`	Devolve a solução de referência como função `X -> u` (vetor coluna), para APIs que pedem um callable de referência.
`describe`	função	`(problem) -> dict[str, object]`	Descrição do problema pronta para JSON, guardada em todo arquivo de resultado.

`pinneapple_tools.benchmark_suite.crossframework.protocol`

Configurações fixas do protocolo (trilhas A1, A2, B, C, lidas de `benchmarks/vs_frameworks/PROTOCOL.md`) e a exportação que todo executor produz.

`pinneapple_tools/benchmark_suite/crossframework/protocol.py` · 97 linhas · testes: 20

nome	tipo	assinatura	o que faz
`export_predictions`	função	`(problem, predict, batch=20000) -> dict[str, np.ndarray]`	Avalia a função de previsão em lotes na grade de teste e no estêncil de resíduo e devolve `u_test` e `u_stencil`.
`ensure_point_set`	função	`(problem, root) -> Path`	Cria uma única vez o conjunto de pontos A1 do problema em `root/points/` e reutiliza o mesmo arquivo depois.
`runner_parser`	função	`(description)`	Parser de linha de comando comum a todos os executores (o driver só os chama por ele).
`base_meta`	função	`(args, *, track, framework, source, config) -> dict`	Metadados iniciais de cada resultado (trilha, framework, fonte, configuração); o executor completa status, iterações e tempos.

`pinneapple_tools.benchmark_suite.crossframework.records`

Registros de execução do benchmark entre frameworks: ambiente, threads, carga da máquina, limites de tempo e esquema do arquivo de resultado. [sem teste]

pinneapple_tools/benchmark_suite/crossframework/records.py · 166 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Clock`	classe	`(wall_s, cpu_s, loadavg_before, loadavg_after)`	Tempo de parede e de CPU do processo de um bloco, com a carga média antes e depois.
`TimeCap`	classe	`(seconds, kind='cpu')`	Orçamento de tempo por execução, medido em CPU do processo ou em tempo de parede, com `restart`, `used` e `expired`.
`setup_threads`	função	`(n) -> None`	Fixa o número de threads do processo (variáveis OMP/MKL/OpenBLAS e `torch.set_num_threads`); deve ser chamado antes de importar o framework.
`loadavg`	função	`() -> list`	Devolve a carga média do sistema operacional.
`cpu_model`	função	`() -> str`	Devolve o nome do modelo de CPU da máquina.
`package_versions`	função	`(names) -> dict[str, str | None]`	Devolve a versão instalada de cada pacote pedido (None se ausente).
`environment`	função	`(threads) -> dict[str, Any]`	Registro da máquina e das bibliotecas, guardado junto aos resultados.
`write_run`	função	`(out_dir, run_id, meta, arrays) -> Path`	Grava `<run_id>.json` (metadados) e `<run_id>.npz` (previsões exportadas) na pasta de saída.
`read_run`	função	`(out_dir, run_id)`	Lê de volta os metadados e as previsões de uma execução gravada.
`validate_result`	função	`(meta) -> list`	Lista os campos obrigatórios ausentes nos metadados de um resultado (vazia quando válido).

pinneapple_tools.benchmark_suite.crossframework.weights

Pesos iniciais compartilhados da trilha de configuração idêntica (A1), gerados em numpy e copiados para camadas PyTorch.

pinneapple_tools/benchmark_suite/crossframework/weights.py · 48 linhas · testes: 20

nome	tipo	assinatura	o que faz
`layer_sizes`	função	`(in_dim=2, out_dim=1, width=50, depth=4) -> list[int]`	Lista de tamanhos de camada de um MLP a partir da dimensão de entrada, saída, largura e profundidade.
`xavier_uniform_init`	função	`(sizes, seed) -> list[tuple[np.ndarray, np.ndarray]]`	Gera pesos Xavier uniformes (Glorot & Bengio 2010) no layout de `torch.nn.Linear`, com bias zero, a partir de uma semente.
`load_into_linears`	função	`(linears, params) -> None`	Copia os pesos gerados para uma sequência de `torch.nn.Linear` (as formas precisam bater exatamente).
`count_parameters`	função	`(module) -> int`	Conta os parâmetros de um módulo PyTorch.

pinneapple_tools.benchmark_suite.deeponet_ground_truth

Os quatro solvers de referência do artigo DeepONet (Lu, Jin & Karniadakis 2019): dada uma função de entrada amostrada de um GRF, calculam a solução verdadeira de cada sistema dinâmico. [sem teste]

pinneapple_tools/benchmark_suite/deeponet_ground_truth.py · 155 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`solve_ode_linear`	função	`(draw, x_query) -> np.ndarray`	Solução exata da EDO linear pela antiderivada do próprio spline do GRF.
`solve_ode_nonlinear`	função	`(draw, x_query) -> np.ndarray`	Solução de referência da EDO não linear dirigida pela função de entrada, nos pontos pedidos.
`solve_pendulum`	função	`(draw, x_query, k=1.0) -> np.ndarray`	Solução do pêndulo forçado pela função de entrada; devolve o ângulo em cada ponto de consulta.
`pendulum_rk4_reference`	função	`(draw, x_max, k, n_steps=20000) -> float`	Integrador RK4 de passo fixo escrito à mão, só para conferir a integração adaptativa em scripts de validação (não para produção).

nome	tipo	assinatura	o que faz
`solve_diffusion_reaction`	função	`(draw, nx=100, nt=100, D=0.01, k=0.01, x_max=1.0, t_max=1.0) -> 'tuple[np.ndarray, np.nda…)`	Difusão-reação com Crank–Nicolson para a difusão, uma correção de Picard por passo para a reação e algoritmo de Thomas.

`pinneapple_tools.benchmark_suite.io`

Pacote de entrada/saída da suíte; reexporta os utilitários YAML. [sem teste]

`pinneapple_tools/benchmark_suite/io/__init__.py` · 6 linhas · **sem teste direto**

`pinneapple_tools.benchmark_suite.io.yamlx`

Funções finas para ler e escrever YAML como dicionário. [sem teste]

`pinneapple_tools/benchmark_suite/io/yamlx.py` · 56 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`load_yaml`	função	`(path) -> Dict[str, Any]`	Lê um arquivo YAML e devolve o conteúdo como dicionário.
`dump_yaml`	função	`(obj) -> str`	Serializa um objeto Python para texto YAML.

`pinneapple_tools.benchmark_suite.meta_benchmark`

Benchmark de meta-aprendizado para PINNs: treina MAML (primeira ordem) e Reptile em famílias paramétricas de EDPs e compara a adaptação em K passos com treino do zero.

`pinneapple_tools/benchmark_suite/meta_benchmark.py` · 937 linhas · testes: 151

nome	tipo	assinatura	o que faz
`MetaBenchmarkConfig`	classe	`(n_meta_epochs, n_inner_steps, inner_lr, outer_lr, n_tasks_per_batch, k_shots, algorithms…)`	Configuração do benchmark de meta-aprendizado: épocas meta, passos internos, taxas de aprendizado, tarefas por lote, valores de K, algoritmos, pontos e pesos de perda.
`MetaBenchmarkFamily`	classe	`(name, train_task_factory, eval_task_factory, param_ranges, eval_params)`	Uma família paramétrica de EDPs: fábricas de tarefas de treino e avaliação, faixas de parâmetros e parâmetros de avaliação.
`MetaBenchmarkResult`	classe	`(family_name, eval_params, algorithm, k_shot, metrics, elapsed_s)`	Resultado de uma família, parâmetro de avaliação, algoritmo e valor de K: métricas e tempo.
`MetaBenchmarkPipeline`	classe	`(families, config=None) -> None`	Pipeline que roda o meta-treino e a avaliação K-shot, monta leaderboard, curvas de erro por K e salva/carrega resultados.
`cfg_str`	função	`(results) -> str`	Texto curto com o número de passos do baseline `scratch_full`, tirado dos resultados.

`pinneapple_tools.benchmark_suite.physics_pipeline`

`PhysicsPipeline`: pipeline físico padrão em 6 passos (problema, geometria, geração de pontos e perdas, solver de referência, modelos, treino e exportação com ranking). [sem teste]

`pinneapple_tools/benchmark_suite/physics_pipeline.py` · 1.332 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`PhysicsPipeline`	classe	`(problem, geometry=None, load_generate_data='generate', source=None, metrics='auto', coll…)`	Pipeline declarativo que recebe um problema e uma lista de modelos, gera dados de referência com um solver numérico, treina cada modelo com a perda física e produz um `BenchmarkReport`.

`pinneapple_tools.benchmark_suite.pipeline.dataset_builder`

Reexportação fina de `pinneapple_data.adapters.pinn_batch_builders`, com o alias legado `BundleDataLike = PINNBatch`. [shim] [sem teste]

pinneapple_tools/benchmark_suite/pipeline/dataset_builder.py · 22 linhas · **sem teste direto**

pinneapple_tools.benchmark_suite.pipeline.model_selector

Ajudante para escolher modelos do `ModelRegistry` por tipo de entrada, chaves esperadas, suporte a perda física e tags. [sem teste]

pinneapple_tools/benchmark_suite/pipeline/model_selector.py · 39 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`select_models`	função	`(*, input_kind, expects=None, supports_physics_loss=None, tags_any=None) -> List[str]`	Lista os nomes de modelos registrados que batem com o `input_kind` e os filtros opcionais.

pinneapple_tools.benchmark_suite.registry

Registros globais de tarefas e backends do Arena, com decoradores de registro e funções de busca. [sem teste direto]

pinneapple_tools/benchmark_suite/registry.py · 77 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`register_task`	função	`(task_cls) -> Type['ArenaTask']`	Decorador que registra uma classe de tarefa pelo seu `task_id`.
`get_task`	função	`(task_id) -> Type['ArenaTask']`	Devolve a classe de tarefa registrada com o id dado.
`list_tasks`	função	`() -> List[str]`	Lista os ids de tarefas registradas.
`register_backend`	função	`(backend_cls) -> Type['Backend']`	Decorador que registra uma classe de backend pelo seu `name`.
`get_backend`	função	`(name) -> Type['Backend']`	Devolve a classe de backend registrada com o nome dado.
`list_backends`	função	`() -> List[str]`	Lista os nomes de backends registrados.

pinneapple_tools.benchmark_suite.report

Relatório estruturado dos benchmarks de física e de séries temporais: resultados por modelo, leaderboard, melhor modelo e exportação JSON. [sem teste direto]

pinneapple_tools/benchmark_suite/report.py · 148 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ModelRunResult`	classe	`(model_id, n_params, training_time_s, metrics, history, param_estimates, rank, error_mess…)`	Resultado de um modelo: id, número de parâmetros, tempo de treino, métricas, histórico, parâmetros estimados, posição e mensagem de erro.
`BenchmarkReport`	classe	`(benchmark_type, created_at, version, problem_info, config, model_results, leaderboard, b…)`	Relatório completo de um benchmark, com `to_dict`, `to_json`, `save` e `print_summary`.

pinneapple_tools.benchmark_suite.runner

Pacote dos executores de linha de comando do Arena (comparação, leaderboard, sweep, YAML, pipeline completo). [sem teste]

pinneapple_tools/benchmark_suite/runner/__init__.py · 25 linhas · **sem teste direto**

pinneapple_tools.benchmark_suite.runner.compare

Compara várias pastas de execução já concluídas, gerando tabela e gráficos. [sem teste]

pinneapple_tools/benchmark_suite/runner/compare.py · 151 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`compare_runs`	função	`(run_dirs, *, out_dir='runs/arena_compare', title='arena_compare', make_plots=True) -> Di…)`	Lê as pastas de execução, junta métricas, escreve um resumo e, opcionalmente, gráficos de comparação.

pinneapple_tools.benchmark_suite.runner.leaderboard

Leitura e atualização de um leaderboard em arquivo JSON. [sem teste]

`pinneapple_tools/benchmark_suite/runner/leaderboard.py` · 43 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`load_leaderboard`	função	`(path) -> List[Dict[str, Any]]`	Carrega o leaderboard de um arquivo JSON.
`update_leaderboard`	função	`(path, row) -> None`	Acrescenta uma linha ao leaderboard em JSON.

pinneapple_tools.benchmark_suite.runner.metrics

Utilitário pequeno para converter valores de métricas em float. [sem teste]

`pinneapple_tools/benchmark_suite/runner/metrics.py` · 30 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ensure_float_dict`	função	`(d) -> Dict[str, float]`	Converte para float os valores de um dicionário quando possível.

pinneapple_tools.benchmark_suite.runner.report

Grava os artefatos de uma execução (relatório, métricas, resumo) em disco. [sem teste]

`pinneapple_tools/benchmark_suite/runner/report.py` · 67 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`write_run_artifacts`	função	`(*, artifacts_dir, run_id, report, metrics, summary) -> Path`	Persiste relatório, métricas e resumo de uma execução na pasta de artefatos e devolve o caminho.

pinneapple_tools.benchmark_suite.runner.run_arena_yaml

Executa um experimento Arena completo a partir de um YAML unificado (preset do problema, vários modelos, métricas, inferência e visualização). [sem teste]

`pinneapple_tools/benchmark_suite/runner/run_arena_yaml.py` · 563 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`run_arena_experiment`	função	`(yaml_path, *, out_dir=None) -> Dict[str, Any]`	Lê o YAML, treina cada modelo do experimento, avalia e devolve um resumo por modelo.
`main`	função	`() -> None`	Ponto de entrada de linha de comando para `run_arena_experiment`.

pinneapple_tools.benchmark_suite.runner.run_benchmark

Executa um benchmark do Arena a partir de configurações de tarefa, execução e esquema de bundle. [sem teste]

`pinneapple_tools/benchmark_suite/runner/run_benchmark.py` · 132 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`run_benchmark`	função	`(*, artifacts_dir, task_cfg_path, run_cfg_path, bundle_schema_path) -> Dict[str, Any]`	Carrega o bundle, escolhe o backend, treina, calcula as métricas da tarefa e grava os artefatos.

pinneapple_tools.benchmark_suite.runner.run_models_from_cfg

Treina um modelo do registro a partir de uma configuração de execução (usado pelo sweep). [sem teste]

`pinneapple_tools/benchmark_suite/runner/run_models_from_cfg.py` · 265 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`run_one`	função	`(*, run_cfg, run_dir, bundle_root=None) -> Dict[str, Any]`	Roda uma configuração de modelo numa pasta de execução e devolve as métricas.

nome	tipo	assinatura	o que faz
`main`	função	`()`	Ponto de entrada de linha de comando para `run_one`.

pinneapple_tools.benchmark_suite.runner.run_pipeline

Pipeline de ponta a ponta configurado por YAML: geometria, malha, solver (OpenFOAM, FEniCS ou embutido), dataset, treino, inferência e relatório. [sem teste]

`pinneapple_tools/benchmark_suite/runner/run_pipeline.py` · 1.444 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`run_full_pipeline`	função	`(config_path_or_dict, *, out_dir=None) -> Dict[str, Any]`	Executa todas as etapas do pipeline a partir de um caminho de config ou dicionário e devolve o resumo.
`main`	função	`() -> None`	Ponto de entrada de linha de comando para `run_full_pipeline`.

pinneapple_tools.benchmark_suite.runner.run_sweep

Sweep de benchmark sobre vários modelos, com paralelismo entre modelos (um processo por GPU, fila, ou DDP por modelo via torchrun). [sem teste]

`pinneapple_tools/benchmark_suite/runner/run_sweep.py` · 208 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`SweepJob`	classe	`(model_name, run_cfg)`	Um trabalho do sweep: nome do modelo e sua configuração de execução.
`run_sweep`	função	`(*, models, base_run_cfg, bundle_root, out_dir='runs/arena_sweep', parallelism='process',…)`	Distribui os modelos entre GPUs ou processos e produz uma pasta de execução por modelo.

pinneapple_tools.benchmark_suite.tasks

Pacote das tarefas do Arena: reexporta `ArenaTask`, os casos de escoamento (obstáculo 2D, cavidade 3D) e as tarefas clássicas de benchmark.

`pinneapple_tools/benchmark_suite/tasks/__init__.py` · 29 linhas · testes: 2

pinneapple_tools.benchmark_suite.tasks.allen_cahn_1d

Tarefa de benchmark Allen–Cahn 1D com referência numérica opcional.

`pinneapple_tools/benchmark_suite/tasks/allen_cahn_1d.py` · 135 linhas · testes: 7

nome	tipo	assinatura	o que faz
`AllenCahn1DTask`	classe	`(eps=0.01, build_reference=True) -> None`	Tarefa Allen–Cahn 1D: amostragem de pontos, condições de contorno e inicial, resíduo da EDP e grade de avaliação.

pinneapple_tools.benchmark_suite.tasks.base

Base das tarefas do Arena e o resultado padrão de uma avaliação.

`pinneapple_tools/benchmark_suite/tasks/base.py` · 53 linhas · testes: 2

nome	tipo	assinatura	o que faz
`TaskResult`	classe	`(metrics, artifacts)`	Saída padrão de uma avaliação de tarefa: métricas e artefatos.
`ArenaTask`	classe	`(task_id)`	Classe base de tarefa do Arena; `compute_metrics` padrão calcula L2 relativo e erro máximo das saídas do backend.

pinneapple_tools.benchmark_suite.tasks.burgers_1d

Tarefa de benchmark Burgers 1D (viscosidade padrão 0.01/π) com referência numérica opcional.

pinneapple_tools/benchmark_suite/tasks/burgers_1d.py · 152 linhas · testes: 7

nome	tipo	assinatura	o que faz
Burgers1DTask	classe	(nu=0.01 / np.pi, build_reference=True) -> None	Tarefa Burgers 1D: amostragem, condições de contorno e inicial, resíduo, grade de avaliação e avaliação.

pinneapple_tools.benchmark_suite.tasks.flow_obstacle_2d

Tarefa de escoamento 2D estacionário em torno de obstáculo, avaliada por modelo torch, função de previsão ou arrays. [sem teste]

pinneapple_tools/benchmark_suite/tasks/flow_obstacle_2d.py · 263 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
FlowObstacle2DTask	classe	(task_id, n_collocation_eval, n_boundary_eval, n_sensors_eval)	Calcula resíduo de Navier–Stokes, erro de contorno e L2 em sensores conforme o que o backend devolveu.

pinneapple_tools.benchmark_suite.tasks.heat_1d

Tarefa de benchmark equação do calor 1D com solução exata.

pinneapple_tools/benchmark_suite/tasks/heat_1d.py · 85 linhas · testes: 153

nome	tipo	assinatura	o que faz
Heat1DTask	classe	(alpha=0.01) -> None	Tarefa do calor 1D: solução exata, amostragem, resíduo e grade de avaliação.

pinneapple_tools.benchmark_suite.tasks.lid_driven_cavity_3d

Benchmark de cavidade com tampa móvel 3D, com validação contra dados de linha central de Ghia et al. (1982). [sem teste]

pinneapple_tools/benchmark_suite/tasks/lid_driven_cavity_3d.py · 394 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
LidDrivenCavity3DTask	classe	(Re=100.0, size=1.0, lid_velocity=1.0) -> None	Tarefa de cavidade 3D: métricas, perfil de referência da literatura (Re mais próximo entre os tabelados) e comparação com a previsão.

pinneapple_tools.benchmark_suite.tasks.navier_stokes_tgv_2d

Tarefa de benchmark Navier–Stokes 2D com o vórtice de Taylor–Green (solução exata). [sem teste]

pinneapple_tools/benchmark_suite/tasks/navier_stokes_tgv_2d.py · 183 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
NavierStokesTGV2DTask	classe	(nu=0.01) -> None	Tarefa Taylor–Green 2D: campos exatos u, v, p, amostragem, resíduo, grade e avaliação.

pinneapple_tools.benchmark_suite.tasks.poisson_2d

Tarefa de benchmark Poisson 2D com solução exata e termo fonte. [sem teste]

pinneapple_tools/benchmark_suite/tasks/poisson_2d.py · 98 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
Poisson2DTask	classe	()	Tarefa Poisson 2D: solução exata, forçamento, amostragem, resíduo e grade de avaliação.

pinneapple_tools.benchmark_suite.tasks.wave_1d

Tarefa de benchmark equação da onda 1D com solução exata. [sem teste]

pinneapple_tools/benchmark_suite/tasks/wave_1d.py · 103 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
Wave1DTask	classe	(c=1.0) -> None	Tarefa da onda 1D: solução exata, amostragem, condições, resíduo e grade de avaliação.

pinneapple_tools.benchmark_suite.timeseries_pipeline

TimeSeriesBenchmarkSpec: benchmark declarativo de séries temporais que reaproveita datasets, modelos, treino e métricas existentes. [sem teste]

pinneapple_tools/benchmark_suite/timeseries_pipeline.py · 579 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
TimeSeriesBenchmarkSpec	classe	(source, models=('lstm',), metrics=('mse', 'mae', 'rmse'), target_cols=None, horizon=50, …)	Especificação declarativa (fonte, modelos, métricas, horizonte, janela) cujo run() treina cada modelo e devolve um BenchmarkReport.

pinneapple_tools.benchmark_suite.transfer_benchmark

Benchmark de transferência para PINNs: treina um modelo fonte e o ajusta a problemas alvo com várias estratégias, comparando com treino do zero.

pinneapple_tools/benchmark_suite/transfer_benchmark.py · 798 linhas · testes: 151

nome	tipo	assinatura	o que faz
TransferBenchmarkConfig	classe	(n_source_epochs, lr_source, n_finetune_epochs, lr_finetune, lr_base, lr_scratch, strateg…)	Configuração: épocas e taxas da fonte e do ajuste fino, estratégias, aquecimento, pontos e pesos das perdas.
TransferScenario	classe	(name, source_task, target_tasks, source_label, target_labels)	Um cenário de transferência: uma tarefa fonte e N tarefas alvo, com rótulos.
TransferBenchmarkResult	classe	(scenario_name, target_label, strategy, metrics, history, elapsed_s, n_params_trainable, …)	Resultado de (cenário, alvo, estratégia): métricas, histórico, tempo e parâmetros treináveis e totais.
TransferBenchmarkPipeline	classe	(scenarios, config=None) -> None	Pipeline que roda todas as estratégias de transferência e os baselines do zero, monta leaderboard, gráficos e salva/carrega resultados.

pinneapple_tools.compute_backends

Suporte multi-backend: reexporta o seletor PyTorch/JAX e os utilitários JAX.

pinneapple_tools/compute_backends/__init__.py · 44 linhas · **sem teste direto**

pinneapple_tools.compute_backends.backend

Seletor global do backend ativo: "torch" (padrão), "jax" ou outro nome registrado em pinneapple_core.backend.

pinneapple_tools/compute_backends/backend.py · 53 linhas · testes: 156

nome	tipo	assinatura	o que faz
Backend	class e	(TORCH, JAX)	Namespace com as constantes de nome de backend (TORCH, JAX).
set_backend	função	(name) -> None	Define o backend ativo, validando o nome contra o registro de pinneapple_core.backend; nome desconhecido levanta ValueError.
get_backend	função	() -> str	Devolve o nome do backend ativo.

pinneapple_tools.compute_backends.jax_backend

Utilitários JAX para PINNs (jit, vmap, gradiente, conversão torch<->jax); degrada sem erro de import quando JAX não está instalado.

pinneapple_tools/compute_backends/jax_backend.py · 268 linhas · testes: 151

nome	tipo	assinatura	o que faz
JAXBackend	classe	()	Conjunto de utilitários JAX para PINNs sobre modelos Equinox ou Flax: jit_pinn, vmap_residual, grad_fn e conversões de tensores.
jax_available	função	() -> bool	Diz se o JAX pode ser importado.
jit_pinn	função	(model_fn, residual_fn) -> Callable	Alias de módulo para JAXBackend.jit_pinn (compila com JIT o resíduo do PINN).
vmap_residual	função	(residual_fn, batch_size=64) -> Callable	Alias de módulo para JAXBackend.vmap_residual (vetoriza um resíduo de um ponto sobre um lote).
jax_pinn	função	(model_fn, residual_fn, vectorise=True) -> Callable	Atalho que compila com JIT e, opcionalmente, vetoriza com vmap o resíduo de um PINN.

pinneapple_tools.dataset_quality

Qualidade de datasets: reexporta as checagens estatísticas de checks e a curadoria por VLM de vlm_curation.

pinneapple_tools/dataset_quality/__init__.py · 64 linhas · **sem teste direto**

pinneapple_tools.dataset_quality.checks

Checagens genéricas sobre um dataset (coords, fields): completude, consistência, distribuição, outliers e faixas físicas, sem dependência de framework. [sem teste direto]

pinneapple_tools/dataset_quality/checks.py · 214 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
analyze_completeness	função	(fields) -> Dict[str, Any]	Fração de NaN por campo e uma nota geral de completude.
validate_consistency	função	(coords, fields, monotonic_name_hints=_MONOTONIC_NAME_HINTS) -> Dict[str, Any]	Checa se os campos têm o mesmo número de pontos que coords e, por heurística de nome, se campos cumulativos/temporais são não decrescentes.
analyze_distribution	função	(fields) -> Dict[str, Any]	Resumo estatístico por campo com scipy.stats.describe (média, variância, assimetria, curtose).
detect_outliers	função	(fields, random_state=42) -> Dict[str, Any]	Ajusta um IsolationForest em todos os campos numéricos juntos e reporta a fração de pontos marcados como outliers.
field_range_issues	função	(fields, ranges) -> list	Checa cada campo contra uma faixa física {nome: (lo, hi)} e devolve frases com os problemas.
build_1d_linear_interpolant	função	(z, y) -> 'Any'	Constrói um interpolante linear por partes diferenciável (torch) a partir de amostras 1D, para avaliar resíduos de EDP sobre dados estáticos.

pinneapple_tools.dataset_quality.vlm_curation

Curadoria de datasets sintéticos com um modelo de visão-linguagem: julga a plausibilidade física de um clipe por sequência e filtra pelo rótulo/confiança, sem inventar julgamentos (resposta inválida vira rótulo neutro com confiança 0.5).

pinneapple_tools/dataset_quality/vlm_curation.py · 504 linhas · testes: 16 · palavras-chave no código (heurística; o texto acima diz o status real): placeholder/stub citado no docstring

nome	tipo	assinatura	o que faz
CurationRecord	classe	(id, label, confidence, reason, raw_text)	Julgamento de uma sequência: id, rótulo, confiança, motivo e texto bruto do modelo.

nome	tipo	assinatura	o que faz
build_consistency_prompt	função	(pde_description, *, labels=DEFAULT_LABELS, extra_instructions=None) -> str	Monta o prompt que pede ao VLM para julgar a consistência física de um clipe contra uma descrição da física dada.
extract_json	função	(text) -> Optional[dict]	Extrai o primeiro objeto JSON {...} de um texto livre; devolve None em vez de levantar erro.
normalize_label	função	(label, *, labels=DEFAULT_LABELS, fallback=None) -> str	Normaliza um rótulo possivelmente malformado para o conjunto fechado de rótulos, com fallback no rótulo do meio.
clamp_confidence	função	(x, *, lo=0.0, hi=1.0) -> float	Converte para float e limita a [lo, hi]; valor ilegível vira o ponto médio.
parse_curation_response	função	(seq_id, text, *, labels=DEFAULT_LABELS, reason_max_chars=300, raw_text_max_chars=1200) -…)	Converte uma geração do VLM num CurationRecord sem nunca levantar erro.
load_vlm	função	(model_name, *, device=None, dtype=None, model_cls=None, attn_implementation='sdpa')	Carrega um modelo de chat de visão-linguagem do Hugging Face e seu processador (precisa do extra vlm_curation).
curate_sequence	função	(model, processor, clip_path, prompt, *, seq_id='', fps=4, max_new_tokens=512, labels=DEF…)	Roda uma geração do VLM sobre um clipe e interpreta o resultado.
curate_dataset	função	(index, *, out_dir, model_name='nvidia/Cosmos-Reason2-2B', model=None, processor=None, mo…)	Faz a curadoria de todas as sequências de um índice com um VLM e grava metadados por sequência e agregados.
filter_curated	função	(index, curation_records, *, keep_labels=('consistent',), min_confidence=0.55, id_field='…)	Filtra o índice para as entradas com rótulo aceito e confiança mínima, copiando os dados opcionalmente.

pinneapple_tools.hpo_experiments

Pesquisador automático de literatura (antigo pinneapple_researcher, apesar do nome "hpo"): descobre artigos e repositórios sobre um tema, monta uma base de conhecimento, extrai problema/solução com LLM e tenta reproduzir. [experimental] [sem teste]

pinneapple_tools/hpo_experiments/__init__.py · 14 linhas · **sem teste direto**

pinneapple_tools.hpo_experiments.agents

Pacote dos agentes LLM do pesquisador (extrator, reprodutor, verificador). [sem teste]

pinneapple_tools/hpo_experiments/agents/__init__.py · 6 linhas · **sem teste direto**

pinneapple_tools.hpo_experiments.agents.extractor

Agente que usa um LLM para extrair problema e solução estruturados dos artefatos da base de conhecimento. [sem teste]

pinneapple_tools/hpo_experiments/agents/extractor.py · 101 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
ExtractorAgent	classe	(provider)	Envia trechos de um artigo ou repositório ao provedor LLM e devolve um ExtractedProblemSolution.

pinneapple_tools.hpo_experiments.agents.prompts

Textos de sistema (prompts) dos agentes de extração, reprodução e verificação. [sem teste]

pinneapple_tools/hpo_experiments/agents/prompts.py · 125 linhas · **sem teste direto**

pinneapple_tools.hpo_experiments.agents.reproducer

Agente que pede ao LLM os arquivos de um projeto de reprodução a partir de um item e de trechos da base. [sem teste]

pinneapple_tools/hpo_experiments/agents/reproducer.py · 98 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ReproduceResult`	classe	`(project_dir, run_instructions)`	Resultado da reprodução: pasta do projeto gerado e instruções de execução.
`ReproducerAgent`	classe	`(provider)`	Gera, via LLM, um projeto de reprodução numa pasta de saída.

`pinneapple_tools.hpo_experiments.agents.verifier`

Agente que lê a árvore do projeto e os logs de execução e propõe diagnóstico e patches. [sem teste]

`pinneapple_tools/hpo_experiments/agents/verifier.py` · 75 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`VerifyResult`	classe	`(status, diagnosis, patches, notes)`	Resposta do verificador: status, diagnóstico, lista de patches e notas.
`VerifierAgent`	classe	`(provider)`	Pede ao LLM um diagnóstico de por que o projeto não roda e patches para corrigi-lo.

`pinneapple_tools.hpo_experiments.config`

Configuração do pesquisador: tema, número de artigos e repositórios, pesos do ranking, filtros do GitHub e arXiv, pastas e chunking. [sem teste]

`pinneapple_tools/hpo_experiments/config.py` · 42 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ResearcherConfig`	classe	`(topic, k_papers, k_repos, w_lexical, w_signals, w_freshness, w_quality, github_min_stars…)`	Dataclass com o tema da busca, quantidades, pesos heurísticos do ranking, filtros e diretórios de saída e cache.

`pinneapple_tools.hpo_experiments.kb`

Pacote da base de conhecimento (chunking e armazenamento). [sem teste]

`pinneapple_tools/hpo_experiments/kb/__init__.py` · 5 linhas · **sem teste direto**

`pinneapple_tools.hpo_experiments.kb.chunking`

Divide texto em pedaços com sobreposição para indexação. [sem teste]

`pinneapple_tools/hpo_experiments/kb/chunking.py` · 29 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`TextChunk`	classe	`(chunk_id, text)`	Um pedaço de texto com seu identificador.
`chunk_text`	função	`(text, *, chunk_chars=1800, overlap=200) -> List[TextChunk]`	Corta um texto em pedaços de tamanho fixo em caracteres, com sobreposição entre eles.

`pinneapple_tools.hpo_experiments.kb.store`

Armazenamento de uma execução da base: manifesto, artefatos (artigos e repositórios) e `chunks.jsonl` para RAG. [sem teste]

`pinneapple_tools/hpo_experiments/kb/store.py` · 142 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`KBStore`	classe	`(*, out_dir)`	Persiste uma execução da base: cria a pasta, grava manifesto, textos dos artefatos, pedaços em JSONL e o índice.
`slugify`	função	`(s) -> str`	Converte um texto num nome seguro para arquivo ou pasta.
`ensure_dir`	função	`(p) -> str`	Cria a pasta se não existir e devolve o caminho.

pinneapple_tools.hpo_experiments.models

Modelos de dados do pesquisador: itens ranqueados, resultado da descoberta, artefatos e índice da base, e problema/solução extraído. [sem teste]

pinneapple_tools/hpo_experiments/models.py · 75 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
RankedItem	classe	(type, id, title, url, score, why, meta)	Um artigo ou repositório ranqueado: tipo, id, título, URL, pontuação, justificativa e metadados.
DiscoveryResult	classe	(topic, papers, repos, meta)	Resultado da descoberta: tema, listas de artigos e de repositórios ranqueados e metadados.
KBArtifact	classe	(type, id, title, url, path, meta)	Um artefato (artigo ou repositório) persistido dentro de uma execução da base.
KBIndex	classe	(topic, run_dir, chunks_path, manifest_path)	Índice leve que aponta para a pasta da execução, o JSONL de pedaços e o manifesto.
ExtractedProblemSolution	classe	(source_type, source_id, title, problem, solution, equations, data_requirements, training…)	Saída do ExtractorAgent: problema, solução, equações, requisitos de dados, receita de treino, métricas e limitações.

pinneapple_tools.hpo_experiments.pipelines

Pacote dos pipelines do pesquisador (descoberta, base, extração, reprodução, exportação). [sem teste]

pinneapple_tools/hpo_experiments/pipelines/__init__.py · 8 linhas · **sem teste direto**

pinneapple_tools.hpo_experiments.pipelines.build_kb

Pipeline que baixa artigos (PDF do arXiv) e arquivos-chave de repositórios e grava tudo no KBStore. [sem teste]

pinneapple_tools/hpo_experiments/pipelines/build_kb.py · 141 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
build_kb	função	(cfg, discovery) -> KBIndex	Monta a base de conhecimento a partir do resultado da descoberta e devolve o KBIndex.

pinneapple_tools.hpo_experiments.pipelines.discover

Pipeline de descoberta: busca artigos no arXiv e repositórios no GitHub sobre o tema e os ranqueia por heurísticas. [sem teste]

pinneapple_tools/hpo_experiments/pipelines/discover.py · 201 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
discover	função	(cfg) -> DiscoveryResult	Busca e ranqueia artigos e repositórios para a configuração dada e devolve um DiscoveryResult.

pinneapple_tools.hpo_experiments.pipelines.export_problemdesign

Exporta soluções extraídas num esquema estilo ProblemDesign em JSON; o arquivo complementar só é escrito quando pinneapple_problemdesign pode ser importado. [sem teste]

pinneapple_tools/hpo_experiments/pipelines/export_problemdesign.py · 169 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
to_problemdesign_dict	função	(x) -> Dict[str, Any]	Converte um problema/solução extraído num dicionário estável (fonte, problema, solução, benchmark, pontos fortes e fracos, alternativas, melhorias).
export_problemdesign	função	(items, *, out_path) -> str	Grava os achados em JSON e, se pinneapple_problemdesign estiver disponível, um *_pinneapple_spec.json complementar.

pinneapple_tools.hpo_experiments.pipelines.extract_problem_solutions

Pipeline que roda o `ExtractorAgent` sobre os itens da base. [sem teste]

`pinneapple_tools/hpo_experiments/pipelines/extract_problem_solutions.py` · 39 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`extract_problem_solutions`	função	`(*, kb_index, out_dir=None, max_items=20) -> List[ExtractedProblemSolution]`	Extrai até `max_items` pares problema/solução da base e os grava opcionalmente numa pasta.

pinneapple_tools.hpo_experiments.pipelines.reproduce

Pipeline de reprodução: gera um projeto via LLM, compila, roda `run.py`/`main.py` e aplica patches do verificador em laço. [sem teste]

`pinneapple_tools/hpo_experiments/pipelines/reproduce.py` · 79 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`reproduce`	função	`(*, item, kb_index_dir, out_dir=None, max_fix_iters=2) -> str`	Gera e tenta fazer rodar um projeto de reprodução de um item, com até `max_fix_iters` rodadas de correção, e devolve a pasta.

pinneapple_tools.hpo_experiments.providers

Pacote dos provedores de LLM (só Gemini). [sem teste]

`pinneapple_tools/hpo_experiments/providers/__init__.py` · 4 linhas · **sem teste direto**

pinneapple_tools.hpo_experiments.providers.gemini_provider

Provedor LLM que chama a API Gemini (`generateContent`) diretamente, com modo JSON, ou via um gateway legado. [sem teste]

`pinneapple_tools/hpo_experiments/providers/gemini_provider.py` · 240 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`GeminiProviderConfig`	classe	`(api_base, api_version, model, api_key, timeout_s, gateway_base_url, gateway_route)`	Configuração imutável do provedor: base e versão da API, modelo, chave, timeout e dados do gateway opcional.
`GeminiProvider`	classe	`(*, api_key=None, model=None, api_base=None, api_version=None, timeout_s=None, gateway_ba…)`	Cliente Gemini com `generate`, que aceita um esquema JSON para respostas estruturadas; precisa de chave de API e de `requests`.

pinneapple_tools.hpo_experiments.ranking

Heurísticas de ranking: sobreposição léxica, frescor por decaimento exponencial e normalização. [sem teste]

`pinneapple_tools/hpo_experiments/ranking.py` · 71 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`tokenize`	função	`(text) -> List[str]`	Quebra um texto em tokens minúsculos.
`lexical_overlap_score`	função	`(query, doc) -> float`	Fração dos tokens únicos da consulta presentes no documento (0 a 1).
`safe_parse_iso8601`	função	`(dt) -> Optional[datetime]`	Converte uma data ISO 8601 em `datetime`, devolvendo None se falhar.
`freshness_score`	função	`(dt, *, half_life_days=365.0) -> float`	Pontuação de frescor por decaimento exponencial: 1 hoje, cerca de 0.5 na meia-vida.
`normalize`	função	`(xs) -> List[float]`	Normaliza uma lista de números para o intervalo [0, 1].
`sort_ranked`	função	`(items) -> List[RankedItem]`	Ordena itens ranqueados pela pontuação, do maior para o menor.

pinneapple_tools.hpo_experiments.sources

Pacote das fontes externas (arXiv e GitHub). [sem teste]

`pinneapple_tools/hpo_experiments/sources/__init__.py` · 12 linhas · **sem teste direto**

pinneapple_tools.hpo_experiments.sources.arxiv_pdf

Baixa o PDF de um artigo do arXiv, extrai o texto e o divide em seções por heurística. [sem teste]

`pinneapple_tools/hpo_experiments/sources/arxiv_pdf.py` · 79 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`arxiv_pdf_url`	função	`(arxiv_id, use_export_subdomain=True) -> str`	Monta a URL do PDF de um id do arXiv.
`download_pdf`	função	`(url, out_path, timeout_s=60) -> str`	Baixa um PDF para um caminho local.
`extract_pdf_text`	função	`(pdf_path) -> str`	Extrai o texto de um PDF.
`split_sections`	função	`(text) -> Dict[str, str]`	Divide o texto em seções procurando títulos por palavras-chave; o docstring avisa que não é perfeito.

pinneapple_tools.hpo_experiments.sources.arxiv_source

Busca mínima de artigos na API Atom do arXiv. [sem teste]

`pinneapple_tools/hpo_experiments/sources/arxiv_source.py` · 90 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ArxivPaper`	classe	`(arxiv_id, title, summary, url, published, authors, categories)`	Metadados de um artigo do arXiv: id, título, resumo, URL, data, autores e categorias.
`ArxivSource`	classe	`()`	Faz buscas na API Atom do arXiv e devolve `ArxivPaper`.

pinneapple_tools.hpo_experiments.sources.github_fetcher

Lê README, árvore de arquivos e arquivos-chave de um repositório pela API REST do GitHub. [sem teste]

`pinneapple_tools/hpo_experiments/sources/github_fetcher.py` · 179 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`RepoFile`	classe	`(path, sha, size, url)`	Um arquivo do repositório: caminho, sha, tamanho e URL.
`GithubFetcher`	classe	`(token=None, api_base='https://api.github.com')`	Cliente da API REST do GitHub para branch padrão, README, árvore recursiva, texto de arquivos e escolha de arquivos-chave.

pinneapple_tools.hpo_experiments.sources.github_source

Busca de repositórios pela API de busca do GitHub. [sem teste]

`pinneapple_tools/hpo_experiments/sources/github_source.py` · 90 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`GithubRepo`	classe	`(full_name, title, url, description, stars, forks, language, updated_at, license)`	Metadados de um repositório: nome, URL, descrição, estrelas, forks, linguagem, data de atualização e licença.
`GithubSource`	classe	`(token=None)`	Busca repositórios no GitHub; o token é opcional, mas recomendado (`GITHUB_TOKEN`).

pinneapple_tools.hpo_experiments.utils

Pacote de utilitários do pesquisador. [sem teste]

`pinneapple_tools/hpo_experiments/utils/__init__.py` · 10 linhas · **sem teste direto**

pinneapple_tools.hpo_experiments.utils.verify_runtime

Verificação de execução de um projeto: árvore de arquivos, `py_compile` e execução rápida de `run.py` ou `main.py` em subprocesso. [sem teste]

`pinneapple_tools/hpo_experiments/utils/verify_runtime.py` · 74 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`RunResult`	classe	`(ok, step, returncode, stdout, stderr)`	Resultado de uma etapa: ok, nome da etapa, código de retorno, stdout e stderr.
`project_tree`	função	`(cwd, max_files=400) -> str`	Lista em texto os arquivos do projeto, até um limite.
`py_compile_all`	função	`(cwd) -> RunResult`	Compila todos os `.py` do projeto com `python -m py_compile`.
`smoke_run`	função	`(cwd, timeout_s=180) -> RunResult`	Executa `run.py` ou `main.py` com timeout; se não houver ponto de entrada, devolve ok sem rodar nada.

pinneapple_tools.model_evaluation

Pacote de avaliação de modelos; o `__init__` está vazio e o conteúdo fica em `metrics` e `visualization`. [sem teste direto]

`pinneapple_tools/model_evaluation/__init__.py` · 1 linhas · **sem teste direto**

pinneapple_tools.model_evaluation.metrics

Métricas genéricas sobre arrays simples: regressão, perfis por eixo, MAPE, piores pontos, intervalo bootstrap, sensibilidades e calibração.

`pinneapple_tools/model_evaluation/metrics.py` · 262 linhas · testes: 5

nome	tipo	assinatura	o que faz
`regression_metrics`	função	`(y_true, y_pred, field_names=None) -> Dict[str, Any]`	RMSE, MAE, erro relativo e R^2, no total e por coluna de saída.
`axis_binned_profile`	função	`(coord, field, n_bins=40) -> Dict[str, np.ndarray]`	Agrupa um campo em faixas de uma coordenada e tira a média em cada faixa (por exemplo, perfil de velocidade na direção normal à parede).
`mape`	função	`(y_true, y_pred, eps=1e-08) -> float`	Erro percentual absoluto médio, como fração, com `eps` contra alvos perto de zero.
`worst_n_points`	função	`(coords, abs_residuals, field_names, n=20) -> List[Dict[str, Any]]`	Índices e coordenadas dos `n` pontos com maior erro absoluto somado entre os campos.
`bootstrap_confidence_interval`	função	`(residuals, n_boot=1000, ci=90.0, seed=42) -> Dict[str, Any]`	Intervalo de confiança bootstrap da média dos resíduos.
`one_at_a_time_sensitivity`	função	`(model_fn, base_inputs, param_columns, perturb_frac=0.1, default_nominal=1.0) -> Dict[str…)`	Sensibilidade de primeira ordem, uma coluna por vez, por diferença finita em torno da média.
`gradient_sensitivity`	função	`(model, x, n_out=None) -> Dict[str, Any]`	Sensibilidade exata por autograd: média de |d saída / d entrada| sobre as linhas de `x` (precisa de um `torch.nn.Module` diferenciável).
`calibration_curve`	função	`(y_true, y_pred, residual_std, levels=(0.5, 0.7, 0.8, 0.9, 0.95)) -> Dict[str, Any]`	Cobertura empírica de um intervalo gaussiano com `residual_std` em cada nível de confiança nominal.

pinneapple_tools.model_evaluation.visualization

Gráficos genéricos de avaliação em matplotlib: histogramas, real vs previsto e curva de calibração. [sem teste]

`pinneapple_tools/model_evaluation/visualization.py` · 95 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`plot_value_histograms`	função	`(fields, out_dir, bins=40) -> List[str]`	Um histograma por campo dos valores brutos (perfil de dataset sem verdade de referência).
`plot_error_histograms`	função	`(abs_residuals, field_names, out_dir, bins=40) -> List[str]`	Um histograma por campo de saída de |previsão - verdade|.
`plot_real_vs_predicted`	função	`(y_true, y_pred, out_path) -> str`	Dispersão previsto vs real com a reta y=x.
`plot_calibration_curve`	função	`(levels, empirical_coverage, out_path) -> str`	Cobertura empírica vs nominal contra a diagonal ideal.

pinneapple_tools.model_export

Exportação de modelos `nn.Module` para TorchScript e ONNX e de arrays de coordenadas/campos para CSV e NPZ. [sem teste]

`pinneapple_tools/model_export/__init__.py` · 197 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`export_torchscript`	função	`(model, path, example_input=None) -> str`	Exporta qualquer `nn.Module` para TorchScript (trace com `example_input`, ou script se não houver exemplo ou o trace falhar).
`export_onnx`	função	`(model, path, example_input, input_names=None, output_names=None, opset_version=17, dynam…)`	Exporta qualquer `nn.Module` para ONNX com nomes de entrada/saída, opset e eixos dinâmicos.
`export_csv`	função	`(coords, fields, path, *, coord_names=None, field_names=None) -> str`	Grava coordenadas e campos num arquivo CSV com cabeçalho.
`export_npz`	função	`(coords, fields, path, *, coord_names=None, field_names=None) -> str`	Grava coordenadas e campos num `.npz` comprimido do NumPy.

pinneapple_tools.model_services

Modelos prontos do PINNeAPPle como microserviços locais no formato NVIDIA NIM (`/v1/health`, `/v1/models`, `/v1/infer`, `/v1/chat/completions`), cada resposta com o relatório de confiança.

`pinneapple_tools/model_services/__init__.py` · 27 linhas · testes: 39

pinneapple_tools.model_services.__main__

Linha de comando `python -m pinneapple_tools.model_services serve` (host, porta, `--no-zoo`, `--api-key`). [sem teste]

`pinneapple_tools/model_services/__main__.py` · 27 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`main`	função	`(argv=None) -> int`	Lê os argumentos, monta o registro padrão e roda o servidor HTTP.

pinneapple_tools.model_services.client

Cliente único para Model Services e NVIDIA NIM (mesmas rotas): saúde, modelos, inferência e chat. [sem teste direto]

`pinneapple_tools/model_services/client.py` · 77 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ServiceError`	classe	`(status, message)`	Erro HTTP de um serviço, com status e mensagem.
`ModelServiceClient`	classe	`(base_url, api_key=None, timeout=60.0)`	Cliente HTTP de um endpoint `/v1` (local ou NIM com `api_key`): `ready`, `models`, `infer`, `chat`.

pinneapple_tools.model_services.grid

Serviços de operador em grade: um modelo de campo treinado (por exemplo um FNO do PhysicsNeMo) e o solver de referência na mesma rota `/v1/infer`, com as checagens de Darcy baseadas em resíduo.

`pinneapple_tools/model_services/grid.py` · 50 linhas · testes: 39

nome	tipo	assinatura	o que faz
`darcy_trust`	função	`(a, u, f=1.0, envelope=None) -> dict`	Roda as checagens de Darcy (`pinneapple_physics.darcy_checks`) sobre coeficiente e solução e devolve decisão e checagens.
`grid_operator_service`	função	`(predictor, name, *, envelope=None, f=1.0, card=None, task='Darcy-type field operator') -…)`	Embala um preditor de campo `a -> u` como `ModelService` com a confiança de Darcy e o envelope de treino.
`darcy_solver_service`	função	`(name='pinneapple-darcy-fv', f=1.0) -> ModelService`	O solver de volumes finitos de Darcy como serviço, com as mesmas checagens: o fallback quando o surrogate não é aprovado.

pinneapple_tools.model_services.registry

`ModelService` (função com card e função de confiança) e o registro dos serviços embutidos: zoo, trust card e `pinneapple-intent`. [sem teste direto]

`pinneapple_tools/model_services/registry.py` · 200 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ModelService`	classe	`(id, kind, task, predict, trust, card, version, calls)`	Um serviço: id, tipo (`infer` ou `chat`), tarefa, função de previsão, função de confiança e card; `run` devolve saídas, confiança, avisos e tempos.
`Registry`	classe	`(services=())`	Registro de serviços por id (`add`, `get`, `list`).
`jsonable`	função	`(o)`	Converte recursivamente arrays e escalares NumPy em tipos JSON, com não finitos virando None.
`zoo_service`	função	`(name) -> ModelService`	Serviço de um modelo do zoo: previsão do modelo e confiança vinda de `pp.zoo.validate` nos mesmos inputs.
`trust_card_service`	função	`() -> ModelService`	Serviço de checagens físicas: recebe eixos, valores e EDP e devolve o trust card do campo.
`intent_service`	função	`() -> ModelService`	Serviço `pinneapple-intent`: o extrator determinístico de `pinneapple_design.intent` atrás de uma rota de chat no estilo OpenAI.
`default_registry`	função	`(zoo=True) -> Registry`	Monta o registro com os serviços embutidos (o zoo é opcional).
`dumps`	função	`(o) -> bytes`	Serializa um objeto para bytes JSON usando `jsonable`.
`infer`	função	`(model, inputs_json) -> dict`	Faz uma requisição a um serviço embutido no próprio processo, sem servidor, e devolve a mesma resposta da rota HTTP.

pinneapple_tools.model_services.server

Servidor HTTP dos serviços só com a biblioteca padrão (`ThreadingHTTPServer`), com as rotas do NIM e chave de API opcional. [sem teste direto]

`pinneapple_tools/model_services/server.py` · 116 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`make_server`	função	`(registry, host='127.0.0.1', port=8000, api_key=None)`	Cria o `ThreadingHTTPServer` que serve o registro; com `api_key` exige `Authorization: Bearer`.

nome	tipo	assinatura	o que faz
`serve_in_thread`	função	`(registry, host='127.0.0.1', port=0, api_key=None)`	Inicia o servidor numa thread e devolve `(server, base_url)`.

pinneapple_tools.nvidia

Cola entre o PINNeAPPle e a pilha NVIDIA (PhysicsNeMo, Omniverse/OpenUSD, Isaac Sim, USD BRep): o PINNeAPPle fornece física, solvers de referência e checagens; o NVIDIA treina, mostra ou simula. Todas as dependências NVIDIA são opcionais.

`pinneapple_tools/nvidia/__init__.py` · 24 linhas · **sem teste direto**

pinneapple_tools.nvidia._brep_worker

Processo auxiliar isolado que constrói uma peça a partir de uma receita com `usd_brep` e a mede; roda no interpretador que tiver `usd-brep` (wheel CPython 3.12). [sem teste]

`pinneapple_tools/nvidia/_brep_worker.py` · 157 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`main`	função	`(req_path, out_path)`	Lê a requisição JSON, constrói e mede a peça e grava o resultado no arquivo de saída.

pinneapple_tools.nvidia.brep

Geometria exata com NVIDIA USD BRep: propriedades de massa do sólido exato, curvatura das faces, tesselações em várias tolerâncias com desvio medido e checagens de confiança da geometria.

`pinneapple_tools/nvidia/brep.py` · 247 linhas · testes: 39

nome	tipo	assinatura	o que faz
`PartRecipe`	classe	`(ops, result)`	Receita de peça como lista de operações (caixa, cilindro, esfera, união, diferença, transformações, filetes, extrusão), serializável em dicionário.
`BRepReport`	classe	`(raw, meshes, seconds)`	Relatório da avaliação de uma peça: dados brutos, malhas por tolerância e tempo; dá raio mínimo de curvatura e acesso às malhas.
`brep_python`	função	`() -> str | None`	Caminho do interpretador que tem `usd_brep`: o atual ou o de `$PINNEAPPLE_BREP_PYTHON`, se existir.
`brep_available`	função	`() -> bool`	Diz se há um interpretador com `usd_brep` disponível.
`evaluate_part`	função	`(recipe, *, chord_tolerances=(0.001, 0.0001), density=1.0, angle_deg=25.0, max_edge_lengt…)`	Constrói a receita com USD BRep (via processo auxiliar) e mede massa, curvatura e tesselações nas tolerâncias pedidas.
`geometry_checks`	função	`(rep, *, element_size=None, points_per_radius=4.0, volume_rtol=0.001) -> list[dict[str, A…)`	Checagens da geometria entregue a um solver ou surrogate: sólido manifold, erro de corda abaixo da tolerância, tamanho de malha e volume.
`planar_face_mesh`	função	`(V, F, *, axis=2, value, atol=1e-09) -> tuple[np.ndarray, np.ndarray]`	Extrai os triângulos de uma tesselação que estão num plano `x[axis] = value`, como malha 2D.
`vertex_normals`	função	`(V, F) -> np.ndarray`	Normais unitárias por vértice ponderadas pela área dos triângulos.
`mesh_thickness`	função	`(V, F, *, normals=None, eps=1e-07, chunk=256) -> np.ndarray`	Espessura de parede em cada vértice de uma malha fechada por lançamento de raio (Möller–Trumbore) ao longo da normal interna.
`point_mesh_distance`	função	`(P, V, F, chunk=128) -> np.ndarray`	Distância de cada ponto ao vértice ou centróide de triângulo mais próximo, um limite superior barato da distância à malha.

pinneapple_tools.nvidia.diagram

Desenha diagramas de arquitetura de um pipeline PINNeAPPle + NVIDIA só com matplotlib (sem graphviz).

`pinneapple_tools/nvidia/diagram.py` · 118 linhas · testes: 39

nome	tipo	assinatura	o que faz
`Block`	classe	`(key, label, owner, col, row, ran)`	Um bloco do diagrama: chave, rótulo, dono (PINNeAPPle ou NVIDIA), posição na grade e se a etapa rodou.
`architecture_diagram`	função	`(title, blocks, edges, path, *, subtitle='', width=2.6, height=1.0, dpi=150) -> str`	Desenha blocos e arestas (com rótulo opcional) e salva um PNG.

`pinneapple_tools.nvidia.figures`

Figuras de resultado comuns aos exemplos NVIDIA: painéis de campos, gráfico de checagens e curvas de treino. [sem teste]

`pinneapple_tools/nvidia/figures.py` · 140 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`field_panels`	função	`(fields, titles, path, *, cmaps=None, extent=(0, 1, 0, 1), suptitle='', units=None, share…)`	Desenha vários campos 2D lado a lado, com faixas de cor compartilhadas opcionais, e salva a figura.
`check_rows`	função	`(checks) -> list[dict[str, Any]]`	Normaliza checagens de vários formatos em linhas `{name, value, limit, passed, detail}`.
`checks_figure`	função	`(checks, path, *, title='Physics checks', decision='', dpi=130) -> str`	Gráfico de barras valor/limite por checagem em escala log, com linha tracejada no limite e decisão no título.
`curves_figure`	função	`(panels, path, *, suptitle='', dpi=130) -> str`	Figura com vários painéis de curvas (séries, escala log opcional, linhas horizontais).

`pinneapple_tools.nvidia.heatsink`

Projetos de dissipador de aletas planas avaliados de duas formas: sólido exato do USD BRep (volume, área, massa) e modelo térmico e hidráulico de forma fechada do PINNeAPPle.

`pinneapple_tools/nvidia/heatsink.py` · 113 linhas · testes: 39

nome	tipo	assinatura	o que faz
`Duty`	classe	`(power_w, t_ambient_c, air_velocity_m_s, source, base_t, length, width, material, root_fi…)`	Condição de operação do dissipador: potência, temperatura ambiente, velocidade do ar, fonte, base, dimensões, material e filete de raiz.
`evaluate_design`	função	`(n_fins, fin_height, fin_t, duty=None, *, brep=True, chord_tolerances=()) -> dict`	Avalia um projeto: resultado térmico/hidráulico de forma fechada e, com `brep`, volume, área e massa do sólido exato.
`sample_designs`	função	`(n, *, n_range=(6, 20), h_range=(0.015, 0.045), t_range=(0.001, 0.003), min_gap=0.002, wi…)`	Amostra projetos por hipercubo latino dentro das faixas, com folga mínima entre aletas.
`design_table`	função	`(designs, duty=None, *, brep=True) -> list[dict]`	Avalia uma lista de projetos e devolve uma linha por projeto.
`verify`	função	`(n_fins, fin_height, fin_t, duty=None, **kw) -> dict`	Avaliação decisiva (forma fechada + sólido exato) de um projeto proposto por um surrogate.
`surrogate_service`	função	`(net, X_train, *, name=…, t_ambient=25.0)`	Embala um surrogate de dissipador como Model Service do PINNeAPPle, com relatório de confiança que sinaliza projetos fora da região de treino.

`pinneapple_tools.nvidia.isaac`

Robôs prontos para Isaac Sim em OpenUSD (`UsdPhysics`), suas checagens e o script do Isaac Sim que reproduz um movimento; escrito para a API 5.x, que o 6.x ainda mantém como depreciada.

`pinneapple_tools/nvidia/isaac.py` · 239 linhas · testes: 39 · palavras-chave no código (heurística; o texto acima diz o status real): shim ou obsoleto

nome	tipo	assinatura	o que faz
`author_articulation`	função	`(stage, spec) -> str`	Escreve uma articulação (corpos, juntas, massas) em `/World/<nome>` de um stage USD e devolve o caminho do prim.
`validate_physics`	função	`(path, *, expect_bodies=None, expect_joints=None) -> dict`	`validate_stage` mais a leitura `UsdPhysics` e as checagens de massa e juntas de que o simulador depende.

nome	tipo	assinatura	o que faz
isaac_replay_script	função	(usd_path, robot_prim, traj_npz, out_npz, *, mode='effort', physics_dt=1.0 / 600.0, headl…)	Gera o script autônomo do Isaac Sim que reproduz uma trajetória (ângulos e torques) e grava o resultado; exige Isaac Sim com GPU RTX.
compare_replay	função	(npz, t, q, tau) -> dict	Compara a reprodução do Isaac Sim com o PINNeAPPle: erro máximo de rastreamento e erro RMS de esforço relativo ao torque de pico.

pinneapple_tools.nvidia.omniverse

Resultados do PINNeAPPle como stages OpenUSD para o Omniverse (escritos e checados com `usd-core`): campos como primvars, validador de stage e pré-visualização lida de volta do disco.

`pinneapple_tools/nvidia/omniverse.py` · 646 linhas · testes: 39

nome	tipo	assinatura	o que faz
FieldStage	classe	(path, title='PINNeAPPle result', *, meters_per_unit=1.0, fps=24.0)	Stage USD que guarda campos do PINNeAPPle (malhas, curvas, pontos, amostras no tempo, animação rígida e veredito de confiança) e é gravado com `save()`.
require_pxr	função	()	Importa `pxr` (OpenUSD) ou falha com dica de instalação.
field_colors	função	(values, cmap='inferno', vrange=None) -> np.ndarray	Cores RGB lineares de valores através de um mapa de cores do matplotlib (sRGB convertido para linear, como o USD espera).
grid_surface	função	(X, Y, Z=None) -> tuple[np.ndarray, np.ndarray]	Triângulos de uma grade estruturada de pontos (vértices e faces).
extrude_profile	função	(xy, length, n=2, axis=1) -> tuple[np.ndarray, np.ndarray]	Extruda um perfil 2D fechado ao longo de um eixo e devolve a parede lateral como triângulos.
tube	função	(path, radius, n_around=12) -> tuple[np.ndarray, np.ndarray]	Tubo de raio dado em torno de uma polilinha 3D, como malha de triângulos.
validate_stage	função	(path, *, required_primvars=None, compliance=True) -> dict[str, Any]	Reabre um arquivo USD e o checa (prims, malhas, curvas, amostras no tempo, eixo, unidade, primvars obrigatórios, conformidade).
usd_validators	função	(stage) -> dict[str, Any]	Roda todos os validadores registrados no OpenUSD (`pxr.UsdValidation`), inclusive os de `UsdPhysics`, sobre um stage.
read_stage_geometry	função	(path, time=None, exclude=()) -> list[dict[str, Any]]	Lê as malhas e curvas de um arquivo USD como arrays, com as cores `displayColor` num tempo dado.
render_stage_png	função	(path, png, *, time=None, elev=28, azim=-60, title='', colorbar=None, size=(7.5, 5.5), dp…)	Desenha em PNG (matplotlib) as malhas e curvas de um USD lido do disco, com suas cores.
kit_render_script	função	(usd_path, png_path, *, frames=60) -> str	Gera um script Python para rodar no Omniverse Kit/USD Composer que abre o stage e grava uma captura RTX; exige Omniverse com GPU RTX.
render_stage_gif	função	(path, gif, times, *, fps=6.0, label=None, **kw) -> str	GIF animado de um stage com amostras no tempo, um `render_stage_png` por tempo.
rasterize_topdown	função	(path, H, W, *, bounds=None, time=None, exclude=(), background=(0.0, 0.0, 0.0), view='top…)	Vista ortográfica (por padrão de cima) das malhas de um USD como imagem RGB (H, W, 3).

pinneapple_tools.nvidia.parts

Peças paramétricas (suporte em L, dissipador de aletas) como `PartRecipe`, com volumes de forma fechada para checar o sólido exato.

`pinneapple_tools/nvidia/parts.py` · 68 linhas · testes: 39

nome	tipo	assinatura	o que faz
l_bracket	função	(*, base=0.08, height=0.06, width=0.05, t=0.006, fillet=0.005, bolt_d=0.009, web_hole_d=0…)	Receita de um suporte em L com filete interno, furos de parafuso na base e furo na alma.
l_bracket_volume	função	(*, base=0.08, height=0.06, width=0.05, t=0.006, fillet=0.005, bolt_d=0.009, web_hole_d=0…)	Volume de forma fechada do suporte em L.
heat_sink	função	(*, n_fins, fin_height, fin_t, base_t=0.005, length=0.1, width=0.1, root_fillet=0.0) -> P…)	Receita de um dissipador de aletas planas igualmente espaçadas sobre uma base.

nome	tipo	assinatura	o que faz
heat_sink_geometry	função	(*, n_fins, fin_height, fin_t, base_t=0.005, length=0.1, width=0.1, root_fillet=0.0) -> d…)	Volume e áreas de forma fechada do dissipador (área molhada e área total).

pinneapple_tools.nvidia.physicsnemo

Surrogates pequenos do PhysicsNeMo (FullyConnected) para física tabular (poucas entradas, poucas saídas), treinados e checados à maneira do PINNeAPPle.

pinneapple_tools/nvidia/physicsnemo.py · 146 linhas · testes: 39

nome	tipo	assinatura	o que faz
MLPSurrogate	classe	(in_features, out_features, layer_size, num_layers, activation_fn, seed, x_lo, x_hi, y_lo…)	Surrogate MLP do PhysicsNeMo com normalização pela caixa de treino: fit, predict, in_box (domínio de validade), save e load.
sign_violations	função	(f, axes, axis, output, sign=1, *, tol=0.0) -> float	Fração das células de uma grade onde a diferença finita de uma saída ao longo de uma entrada tem o sinal oposto ao esperado (checagem de monotonia).
relative_errors	função	(pred, ref) -> np.ndarray	Erro RMS por saída relativo ao RMS da saída de referência.

pinneapple_tools.nvidia.runinfo

Registra o que uma execução usou: versões dos pacotes NVIDIA e PINNeAPPle, hardware e tempos por etapa.

pinneapple_tools/nvidia/runinfo.py · 89 linhas · testes: 39

nome	tipo	assinatura	o que faz
RunInfo	classe	(name='')	Registro de execução com tempos por etapa (step), marcação de componentes que rodaram ou não (mark), e exportação para dicionário ou arquivo.
environment	função	() -> dict[str, Any]	Python, sistema, CPU, GPU (CUDA via torch, se houver) e versões dos pacotes relevantes.

pinneapple_tools.rea_bridge

Estudo de implementações de referência com a ferramenta REA (Reverse Engineer Anything), sob uma trava de licença; é uma terceira fonte para o pipeline de pesquisa.

pinneapple_tools/rea_bridge/__init__.py · 33 linhas · testes: 22

pinneapple_tools.rea_bridge.client

Wrapper fino da CLI do REA: resolve o executável (PINNEAPPLE_REA_BIN, rea no PATH ou npx), roda comandos e interpreta o JSON, sem completar resultados. [sem teste direto]

pinneapple_tools/rea_bridge/client.py · 216 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
ReaUnavailable	classe	()	Exceção: nenhum executável do REA foi encontrado.
ReaError	classe	(message, *, returncode, stderr_tail)	Exceção: o REA rodou mas saiu com erro ou não imprimiu JSON.
ReaResult	classe	(command, payload, progress, stderr_tail)	Uma invocação do REA: comando, payload JSON, eventos de progresso e final do stderr, com acesso ao id de evidência, SHA-256 e limitações.
ReaClient	classe	(command=None, *, timeout_s=900.0, cwd=None)	Roda comandos do REA (versão, doctor, análise de app JavaScript, layout de binário, inspeção de artefato) e interpreta a saída.
resolve_rea_command	função	(env=None) -> List[str]	Devolve o prefixo de comando que roda o REA ou levanta ReaUnavailable.

pinneapple_tools.rea_bridge.evidence

Converte a evidência de análise JavaScript do REA num mapa compacto de funções, chamadas, timers e armazenamento, e localiza funcionalidades por palavra-chave. [sem teste direto]

pinneapple_tools/rea_bridge/evidence.py · 222 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
FunctionRef	classe	(node_id, name, module, start_line, end_line)	Referência a uma função: nó, nome, módulo e intervalo de linhas.
FeatureMap	classe	(evidence_id, subject_sha256, root, functions, calls, callers, timers, storage, limitatio…)	Mapa de funcionalidades de uma aplicação: funções, grafo de chamadas, timers, armazenamento, limitações do REA e módulos de terceiros.
FeatureHit	classe	(function, matched, snippet, callees, callers)	Uma função encontrada por locate: termos casados, trecho do código, chamadas e chamadores.
summarize_js_evidence	função	(evidence, *, vendor_globs=DEFAULT_VENDOR_GLOBS, include_vendor=False) -> FeatureMap	Constrói um FeatureMap a partir de um payload de evidência analyze_javascript_application.
locate	função	(fmap, keywords, *, root=None, max_hits=20, snippet_lines=40) -> List[FeatureHit]	Funções cujo nome ou corpo menciona alguma palavra-chave, ordenadas pelo número de ocorrências.
exported_functions_from_layout	função	(evidence) -> List[str]	Lista as funções exportadas de um binário ELF a partir da evidência de inspect-binary-layout.

pinneapple_tools.rea_bridge.harvest

Livro de registro JSON do que foi estudado, sob qual licença, e o que resultou; recusa achados adapted sob política clean_room ou reference_only. [sem teste direto]

pinneapple_tools/rea_bridge/harvest.py · 126 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
LicenseGateError	classe	()	Exceção: um achado pede mais reuso do que a licença do projeto permite.
Finding	classe	(title, summary, source_refs, status, method, pinneapple_refs, note)	Um achado (ideia, técnica ou funcionalidade): onde está no original, status (idea, adapted, reimplemented, exists, rejected) e o que o PINNeAPPle fez.
HarvestEntry	classe	(project, url, commit, licenses, policy, findings, rea_version, evidence_id, subject_sha2…)	Um projeto estudado: URL, commit, licenças, política, achados e dados da evidência do REA.
HarvestLedger	classe	(path)	Arquivo JSON de entradas por nome de projeto, com put, save e summary.

pinneapple_tools.rea_bridge.licenses

Trava de licença: classifica um projeto estudado em adapt, clean_room ou reference_only pela licença mais restritiva detectada. [sem teste direto]

pinneapple_tools/rea_bridge/licenses.py · 179 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
LicensePolicy	classe	(policy, licenses, attribution_required, reason, notes)	Política de reuso de um projeto (licenças, atribuição exigida, motivo); may_copy_code diz se código pode ser adaptado.
policy_for	função	(spdx_ids) -> LicensePolicy	Política mais restritiva sobre uma lista de identificadores SPDX.
detect_licenses	função	(root, *, max_bytes=200000) -> List[str]	Identificadores SPDX reconhecidos nos arquivos de licença e nos manifestos (package.json, pyproject.toml, setup.py) de uma pasta.
policy_for_path	função	(root) -> LicensePolicy	Detecta as licenças de uma pasta e devolve a política correspondente.

pinneapple_tools.rea_bridge.tools

Registra a ponte REA como ferramentas do `PhysicsToolRegistry` para o loop de agente de `pinneapple_llm`; as ferramentas REA só entram quando o executável existe. [sem teste direto]

`pinneapple_tools/rea_bridge/tools.py` · 94 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`register_rea_tools`	função	`(registry) -> List[str]`	Registra `reference_license_policy` e, se o REA estiver disponível, `rea_analyze_javascript` e `rea_locate_feature`; devolve os nomes.

pinneapple_tools.reporting

Relatórios: estilo de figuras e construtor de PDF (`paperkit`) e datasheet técnico (`datasheet`); não é importado automaticamente pelo pacote.

`pinneapple_tools/reporting/__init__.py` · 27 linhas · testes: 5

pinneapple_tools.reporting.datasheet

Datasheet técnico de um estudo, montado só a partir de dados (sem LLM): faixa de KPIs, seções numeradas, plano de execução, itens a confirmar; HTML, PDF via Chromium e Markdown.

`pinneapple_tools/reporting/datasheet.py` · 570 linhas · testes: 14

nome	tipo	assinatura	o que faz
`KPI`	classe	`(label, value, unit, note, tone)`	Uma célula da faixa de números principais: rótulo, valor, unidade, nota e tom.
`Table`	classe	`(rows, caption, header, widths, align, emphasis, note)`	Tabela de duas ou mais colunas com cabeçalho, legenda, larguras, alinhamento e nota.
`Figure`	classe	`(path, caption, wide, max_width)`	Imagem com legenda, em largura total ou dividindo a linha.
`Note`	classe	`(text, kind, title)`	Caixa de destaque do tipo `info`, `ok`, `warn` ou `fail`.
`Step`	classe	`(title, detail, status, cost)`	Um passo do plano de execução: o que faz, por que existe, custo e situação.
`Section`	classe	`(title, number, lede, blocks)`	Seção numerada com abertura e blocos (parágrafos, notas, tabelas, figuras, linhas, equações, listas).
`Datasheet`	classe	`(title, subtitle, eyebrow, generated, verified_by, kpis_, sections, plan_, confirm_, limi…)`	O documento inteiro; `build` grava HTML e, se houver Chromium ou Playwright, PDF; também gera Markdown.
`find_chrome`	função	`() -> str | None`	Procura um executável Chromium/Chrome capaz de imprimir HTML em PDF; None se não houver.

pinneapple_tools.reporting.paperkit

Estilo único de figuras matplotlib (ordem de cores fixa, mapas `seq_blue` e `div_blue_red`) e `Paper`, um construtor de relatório A4 em PDF com reportlab (opcional).

`pinneapple_tools/reporting/paperkit.py` · 429 linhas · testes: 5

nome	tipo	assinatura	o que faz
`Paper`	classe	`(title, subtitle='', authors='PINNeAPPle Labs', affiliation=…, date=None, report_id='', f…)`	Relatório técnico A4 com reportlab/platypus: título, resumo, seções, equações, figuras e tabelas numeradas, referências.
`setup_mpl`	função	`() -> None`	Aplica o estilo de relatório ao `matplotlib.rcParams` e registra os mapas `SEQ` e `DIV`.
`savefig`	função	`(fig, path, **kwargs) -> str`	Salva a figura criando as pastas, fecha e devolve o caminho.
`fmt`	função	`(x, nd=3) -> str`	Formata um número de forma compacta para uma célula de tabela.

nome	tipo	assinatura	o que faz
sanitize	função	(text) -> str	Troca sequências de sobrescritos/subscritos Unicode pela marcação <super>/<sub> do reportlab.
para	função	(text, style, **kw)	Cria um Paragraph do reportlab a partir do texto sanitizado.

pinneapple_tools.sandbox

Pacote para executar scripts de solver não confiáveis em outro processo com limites de recurso; o próprio docstring avisa que não é um sandbox real.

pinneapple_tools/sandbox/__init__.py · 11 linhas · **sem teste direto**

pinneapple_tools.sandbox.custom_solver_runner

Roda solve(params) de um script do usuário num subprocesso com RLIMIT_AS, RLIMIT_CPU e timeout de relógio; não restringe arquivos, rede nem syscalls.

pinneapple_tools/sandbox/custom_solver_runner.py · 318 linhas · testes: 13

nome	tipo	assinatura	o que faz
run_custom_solver	função	(script_path, params=None, *, timeout=60.0, memory_limit_mb=512) -> Dict[str, Any]	Executa a função solve(params) do script num subprocesso limitado e devolve o resultado ou o erro.

pinneapple_tools.surrogate_policy

Política baseada em tempo medido: decide se um problema precisa de surrogate cronometrando uma solução real, com cache JSON e limiar.

pinneapple_tools/surrogate_policy.py · 92 linhas · testes: 5

nome	tipo	assinatura	o que faz
SurrogatePolicyResult	classe	(problem_id, needs_model, reason, measured_s)	Resultado da política: problema, se precisa de modelo, motivo e tempo medido.
SurrogatePolicy	classe	(cache_path, *, slow_threshold_s=1.0)	Classifica problemas (sem solver direto, geometria cara, ou tempo medido acima de slow_threshold_s) e sugere uma arquitetura por heurística simples.

pinneapple_tools.visualization

Visualização no estilo CFD (antigo pinneapple_viz): reexporta estilo, campos 2D/3D, gráficos de PINN, saídas de solvers, animações, malhas, voxels e vórtices; os subpacotes dashboard e studio cobrem painéis animados e cenas 3D realistas.

pinneapple_tools/visualization/__init__.py · 116 linhas · **sem teste direto**

pinneapple_tools.visualization.animation

Animações da evolução no tempo de soluções de EDP (campo escalar e linhas de corrente), salvas em GIF/MP4 com matplotlib. [sem teste]

pinneapple_tools/visualization/animation.py · 205 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
animate_scalar_field	função	(trajectory, x=None, y=None, *, t_vals=None, title='Field evolution', field_label='u', cm…)	Anima uma série temporal de campos escalares 2D com escala de cor fixa e salva opcionalmente.
animate_streamlines	função	(u_traj, v_traj, x=None, y=None, *, speed_traj=None, t_vals=None, title='Velocity evoluti…)	Anima linhas de corrente de uma série temporal de campos de velocidade, coloridas pela velocidade.

nome	tipo	assinatura	o que faz
`make_gif`	função	`(frames, x, y, save_path, *, t_vals=None, cmap=DEFAULT_CMAP, dpi=80, fps=10) -> None`	Atalho que salva uma lista de quadros de campo 2D como GIF.

`pinneapple_tools.visualization.dashboard`

Painéis centrados numa animação para qualquer simulação: animação no meio, gráficos sincronizados ao cursor de tempo, KPIs, parâmetros e barra de cor; exporta HTML interativo, PNG e GIF/MP4.

`pinneapple_tools/visualization/dashboard/__init__.py` · 18 linhas · testes: 16

`pinneapple_tools.visualization.dashboard.html`

Exportação HTML interativa de um `Dashboard` num único arquivo autônomo (sem servidor, sem internet).

`pinneapple_tools/visualization/dashboard/html.py` · 429 linhas · testes: 5

nome	tipo	assinatura	o que faz
`payload`	função	`(db) -> dict`	Converte tudo que a página desenha em dados serializáveis em JSON.
`scene3d_payload`	função	`(bodies, poses, terrain=None, lighting=None, axes='z_up') -> dict`	Dados da vista 3D ao vivo (corpos, poses, terreno, iluminação) para a página.
`write_html`	função	`(db, path, *, fps=12, max_width=1280, quality=26) -> str`	Grava a página HTML autônoma do painel e devolve o caminho.

`pinneapple_tools.visualization.dashboard.spec`

Modelo de dados puro (arrays numpy e textos) do painel: animação, gráficos, KPIs, parâmetros e barra de cor; os exportadores ficam em `html` e `static`. [sem teste direto]

`pinneapple_tools/visualization/dashboard/spec.py` · 190 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Series`	classe	`(label, y, color, dash, x, width)`	Uma linha de gráfico, com um valor por quadro ou por ponto `x`.
`Marker`	classe	`(label, x, y, color)`	Um ponto que se move com os quadros num gráfico de fase.
`Chart`	classe	`(title, ylabel, xlabel, series, markers, x, kind, slot, ylim, note)`	Um gráfico do painel: título, eixos, séries, marcadores, tipo, posição (slot), limites e nota.
`Tile`	classe	`(label, value, unit, fmt, note)`	Um KPI: escalar fixo ou um valor por quadro que acompanha o cursor.
`Param`	classe	`(label, value, unit)`	Um parâmetro mostrado no painel lateral (rótulo, valor, unidade).
`ColorBar`	classe	`(title, cmap, lo, hi, saturation)`	Barra de cor ao lado da animação, com saturação ajustável para imitar uma sobreposição semitransparente.
`Dashboard`	classe	`(title, t, frames, charts, tiles, params, subtitle, colorbar, footer, time_label, time_un…)`	O painel completo (título, tempos, quadros, gráficos, KPIs, parâmetros, barra de cor, cena 3D), com `to_html`, `to_png` e `to_video`.

`pinneapple_tools.visualization.dashboard.static`

Exportações PNG e GIF/MP4 de um `Dashboard`, desenhadas com matplotlib no estilo escuro do studio.

`pinneapple_tools/visualization/dashboard/static.py` · 352 linhas · testes: 14

nome	tipo	assinatura	o que faz
`write_png`	função	`(db, path, k=None, size=(1920, 1080), dpi=100, font_scale=1.0) -> str`	Grava o painel num quadro dado (padrão: o último) como PNG.

nome	tipo	assinatura	o que faz
write_gif	função	(path, frames, fps, colors=255, loop=0) -> str	GIF animado de quadros RGB com uma paleta adaptativa única para todo o clipe.
hstack_gifs	função	(paths_or_frames, out, fps=10, colors=128, background='white', mismatch='error', loop=0)	Junta várias animações lado a lado num só GIF, quadro a quadro (por exemplo, duas execuções comparadas).
write_video	função	(db, gif=None, mp4=None, *, fps=12, size=(1920, 1080), gif_size=(1280, 720), gif_every=1,…)	Grava o painel animado como GIF e/ou MP4 (H.264).

pinneapple_tools.visualization.electromagnetics

Figuras e animações de campos eletromagnéticos: mapas com setas de Poynting, filmes de propagação (GIF com Pillow, MP4 com ffmpeg) e diagramas de radiação. [sem teste]

pinneapple_tools/visualization/electromagnetics.py · 134 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
field_map	função	(field, extent, path=None, *, outline=None, vectors=None, title='', label='', cmap='RdBu_…)	Desenha um campo 2D com contorno de material e setas opcionais, escala simétrica por padrão.
field_movie	função	(frames, extent, path, *, times=None, outline=None, title='', label='', cmap='RdBu_r', fp…)	Anima uma lista de campos 2D com escala de cor simétrica fixa, calculada de todos os quadros.
radiation_pattern	função	(theta, values, path=None, *, reference=None, labels=('computed', 'analytic'), db=True, f…)	Gráfico polar de um diagrama de campo distante normalizado, em dB opcional, contra uma referência opcional.

pinneapple_tools.visualization.fields

Gráficos de campos escalares e vetoriais 2D/3D no estilo CFD (contornos, fatias, setas, linhas de corrente, comparação e erro). [sem teste]

pinneapple_tools/visualization/fields.py · 361 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
plot_scalar	função	(x, y, field, *, title='Scalar field', label='', cmap=DEFAULT_CMAP, n_levels=64, show_con…)	Contorno preenchido de um campo escalar 2D, com linhas de contorno opcionais.
plot_scalar_3d	função	(x, y, z, field, *, slice_axis='z', slice_val=None, title='Scalar field (3-D slice)', cma…)	Plota uma fatia 2D de um campo escalar 3D ao longo de um eixo.
plot_vectors	função	(x, y, u, v, *, title='Vector field', scale=None, density=20, color_by_mag=True, cmap='ra…)	Gráfico de setas (quiver), coloridas pela magnitude, com campo escalar de fundo opcional.
plot_streamlines	função	(x, y, u, v, *, title='Streamlines', density=1.5, color_by_mag=True, cmap='plasma', linew…)	Linhas de corrente de um campo de velocidade em grade estruturada, com fundo opcional.
compare_fields	função	(x, y, fields, *, cmap=DEFAULT_CMAP, n_cols=3, figsize_per_panel=(5, 4), suptitle='', sho…)	Contornos de vários campos lado a lado para comparação.
plot_error	função	(x, y, pred, ref, *, relative=True, title='Prediction error', cmap='hot', show=False) -> …)	Mostra o erro ponto a ponto |pred − ref| (ou relativo) no domínio.

pinneapple_tools.visualization.integrations

Pontes dos resultados do PINNeAPPle para outros visualizadores 3D (Plotly, Rerun, Trame). [sem teste direto]

pinneapple_tools/visualization/integrations/__init__.py · 11 linhas · **sem teste direto**

pinneapple_tools.visualization.integrations.plotly_export

HTML 3D interativo e autônomo com Plotly (girar, zoom, valores ao passar o mouse), para compartilhar um resultado sem servidor.

pinneapple_tools/visualization/integrations/plotly_export.py · 94 linhas · testes: 7

nome	tipo	assinatura	o que faz
`scene_html`	função	`(scene, path, field=None, cmap='Turbo', title='', include_plotlyjs=True) -> str`	Exporta uma cena 3D (superfícies, campo opcional) para HTML Plotly.
`molecule_html`	função	`(mol, path, xyz=None, slab=None, metal='Au', title='', include_plotlyjs=True) -> str`	Exporta uma molécula, opcionalmente sobre uma placa metálica, para HTML Plotly.
`field_slices_html`	função	`(x, y, z, values, path, iso=None, cmap='Viridis', title='', include_plotlyjs=True) -> str`	Exporta um campo 3D em grade como três fatias medianas e, com `iso`, uma isosuperfície, em HTML Plotly.

pinneapple_tools.visualization.integrations.rerun_bridge

Ponte para o visualizador Rerun (rerun.io): envia resultados ao vivo ou grava num arquivo `.rrd`; exige `rerun-sdk`.

`pinneapple_tools/visualization/integrations/rerun_bridge.py` · 197 linhas · testes: 7

nome	tipo	assinatura	o que faz
`RerunLogger`	classe	`(app_id='pinneapple', path=None, spawn=False, connect=None)`	Registrador Rerun que envia pontos, malhas, cenas, moléculas, escalares, conjuntos de execuções, grafos e texto, para arquivo, visualizador novo ou já aberto.

pinneapple_tools.visualization.integrations.trame_app

App web Trame (Kitware) em volta de um plotter PyVista, servido a partir do Python, para malhas e campos grandes demais para um HTML autônomo.

`pinneapple_tools/visualization/integrations/trame_app.py` · 37 linhas · testes: 7

nome	tipo	assinatura	o que faz
`build_app`	função	`(plotter, title='PINNeAPPle', local_rendering=False, server_name=None)`	Cria um servidor trame que mostra a janela de render do plotter, com renderização no servidor ou no navegador (vtk.js).

pinneapple_tools.visualization.mesh

Utilitários de visualização de malhas e geometria (malha 2D, grupos de contorno, nuvem de pontos). [sem teste]

`pinneapple_tools/visualization/mesh.py` · 137 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`plot_mesh`	função	`(nodes, elems, *, node_values=None, cmap=DEFAULT_CMAP, edge_color='#4c9be8', edge_lw=0.5,…)`	Plota uma malha 2D de triângulos ou quadriláteros, com valores nodais opcionais.
`plot_boundary`	função	`(nodes, boundary_groups, *, title='Boundary groups', show=False) -> Figure`	Mostra os grupos de contorno rotulados de uma malha.
`plot_point_cloud`	função	`(points, values=None, *, cmap='plasma', s=3.0, title='Point cloud', show=False, dim3=Fals…)`	Dispersão 2D ou 3D de uma nuvem de pontos, colorida por valores opcionais.

pinneapple_tools.visualization.molecular

Visualização molecular: proteínas e moléculas, sozinhas ou adsorvidas em superfícies metálicas, como imagens, trajetórias MP4, cenas glTF e painéis de dinâmica molecular.

`pinneapple_tools/visualization/molecular.py` · 503 linhas · testes: 7

nome	tipo	assinatura	o que faz
`Molecule`	classe	`(xyz, elements, names, resnames, resids, chains, frames)`	Molécula com posições, elementos, nomes, resíduos, cadeias e quadros; lê PDB ou mdtraj e dá massas, raios, classes e traço C-alfa.
`StepLog`	classe	`(title='', color=None, stream=None)`	Log de console passo a passo de um pipeline: passos numerados, detalhes indentados, tempo decorrido e cor em terminal.

nome	tipo	assinatura	o que faz
residue_class	função	(resname) -> str	Classifica um resíduo pelo nome em hidrofóbico, polar, positivo, negativo, especial ou outro, para colorir.
fcc_slab	função	(metal='Au', facet='111', nx=10, ny=10, layers=4, a=None) -> tuple[np.ndarray, np.ndarray]	Posições atômicas de uma placa cúbica de face centrada com a faceta dada no topo, e os vetores da célula no plano.
slab_for_footprint	função	(xyz, metal='Au', facet='111', padding=8.0, layers=4) -> tuple[np.ndarray, np.ndarray, tu…)	Placa metálica que cobre a projeção x-y de uma molécula mais uma margem.
orient_on_surface	função	(xyz, masses=None, active_site=None, gap=3.0, surface_z=0.0) -> np.ndarray	Gira a molécula para deitar sua face mais larga sobre a superfície e a posiciona a uma distância dada.
rmsd	função	(frames, ref, align=True) -> np.ndarray	RMSD de cada quadro em relação a uma referência, com alinhamento de Kabsch opcional.
height_above	função	(frames, surface_z, masses=None) -> np.ndarray	Altura do centro de massa acima do plano da superfície, por quadro.
contacts	função	(frames, surface_z, cutoff=3.5) -> np.ndarray	Número de átomos a menos de um corte da superfície, por quadro.
render	função	(mol, path, *, xyz=None, slab=None, metal='Au', color_by='residue', style='spheres', size…)	Gera uma imagem estática da molécula, opcionalmente sobre uma placa, com cores e estilo escolhidos.
render_frames	função	(mol, frames, out_dir, *, slab=None, metal='Au', color_by='residue', style='spheres', siz…)	Gera um PNG por quadro de trajetória com câmera fixa ou em órbita lenta.
encode_mp4	função	(pngs, out, fps=12, crf=22) -> str	Codifica PNGs em MP4 H.264 com o ffmpeg empacotado pelo `imageio-ffmpeg`.
md_dashboard	função	(path, t_ps, *, potential=None, temperature=None, rmsd_A=None, height_A=None, contacts_n=…)	Figura com um painel por observável de dinâmica molecular ao longo do tempo (energia, temperatura, RMSD, altura, contatos).
to_glb	função	(mol, path, *, xyz=None, slab=None, metal='Au', max_atoms=20000) -> str	Exporta a cena como glTF binário (esferas por átomo agrupadas por classe e placa metálica) para o visualizador web.

pinneapple_tools.visualization.pinn

Visualizações específicas de PINN: curvas de treino, campos de resíduo, pontos de colocação e comparação previsão × referência. [sem teste]

`pinneapple_tools/visualization/pinn.py` · 341 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
plot_loss_history	função	(history, *, keys=None, log_scale=True, title='Training loss history', show=False) -> Fig…)	Plota curvas de perda de um histórico de treino, em escala log por padrão.
plot_multi_loss	função	(history, *, groups=None, log_scale=True, title='Loss components', show=False) -> Figure	Painéis separados por grupos de componentes de perda.
plot_collocation	função	(interior_pts, boundary_pts=None, initial_pts=None, data_pts=None, *, title='Collocation …)	Dispersão 2D dos conjuntos de pontos internos, de contorno, iniciais e de dados.
plot_pinn_prediction	função	(x, y, pred, ref=None, *, field_name='u', cmap=DEFAULT_CMAP, show=False) -> Figure	Painéis referência, previsão e erro (ou só a previsão, sem referência).
plot_pde_residual	função	(x, y, residual, *, title='PDE residual', cmap='hot', log_scale=True, show=False) -> Figu…)	Plota |resíduo da EDP| nos pontos de colocação, em escala log opcional.
plot_gradient_magnitude	função	(x, y, grad_x, grad_y, *, title='Gradient magnitude | ∇u|', cmap='plasma', show=False) -> …)	Plota |∇u| a partir das derivadas parciais.
predict_and_plot	função	(model, x_coords, y_coords, *, ref=None, field_name='u', device=None, cmap=DEFAULT_CMAP, …)	Roda a inferência do modelo numa grade e plota o painel de previsão.

pinneapple_tools.visualization.pyvista_bridge

Ponte PyVista: transforma malhas e campos do PINNeAPPle (inclusive resultados CalculiX e cenas `twin3d`) em imagens 3D de qualidade e arquivos VTK.

`pinneapple_tools/visualization/pyvista_bridge.py` · 195 linhas · testes: 7

nome	tipo	assinatura	o que faz
von_mises	função	(stress6) -> np.ndarray	Tensão equivalente de von Mises a partir das seis componentes do CalculiX.
grid_from_arrays	função	(points, cells, vtk_cell_type, point_data=None)	Monta uma grade não estruturada a partir de pontos, células e tipo de célula VTK.
calculix_grid	função	(model, frd=None, step=-1, warp=None)	Grade PyVista de um modelo CalculiX com deslocamento, |U|, tensões e von Mises, se presentes.
scene_to_multiblock	função	(scene, time_index=0, field_name=None)	Converte uma Scene do pinneapple_twin3d num pyvista.MultiBlock (uma superfície por peça).
export_vtk	função	(mesh, path) -> str	Grava .vtu, .vtp, .vtk ou .vtm (pronto para ParaView), com o formato escolhido pela extensão.
can_render	função	() -> bool	Testa num subprocesso se dá para criar um contexto OpenGL fora da tela (o VTK derruba o interpretador em vez de levantar erro).
render_png	função	(mesh, path, *, scalars=None, cmap='turbo', title='', show_edges=False, window_size=(1400…)	Renderiza uma malha em PNG fora da tela, com escalares, mapa de cor e câmera escolhidos.
render_gif	função	(mesh, path, *, scalars=None, cmap='turbo', title='', n_frames=60, elevation=25.0, window…)	Gira a câmera em volta da malha e grava um GIF animado.

pinneapple_tools.visualization.solver

Visualiza objetos SolverOutput de solvers FDM, FEM, FVM e sem malha. [sem teste]

pinneapple_tools/visualization/solver.py · 198 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
plot_solver_output	função	(output, *, x=None, y=None, title='Solver result', cmap=DEFAULT_CMAP, show=False) -> Figu…)	Escolhe automaticamente o gráfico adequado para um SolverOutput.
plot_fem_result	função	(output, *, component=0, title='FEM solution', cmap=DEFAULT_CMAP, show_mesh=True, show=Fa…)	Plota uma solução FEM sobre sua malha Q1.
plot_fvm_result	função	(output, *, frame=-1, title='FVM solution', cmap=DEFAULT_CMAP, show=False) -> Figure	Plota um passo de tempo de uma trajetória FVM.
plot_residuals	função	(output, *, title='Solver residuals', show=False) -> Figure	Gráfico de barras dos resíduos/perdas guardados em output.losses.

pinneapple_tools.visualization.studio

Studio do PINNeAPPle: visualização 3D realista de qualquer geometria e sua física (cena, visualizador web, Blender, PyVista); exporta Scene e atalhos de render.

pinneapple_tools/visualization/studio/__init__.py · 47 linhas · testes: 41

nome	tipo	assinatura	o que faz
render	função	(*a, **kw)	Atalho para o render Blender Cycles de studio.blender.render.
web_viewer	função	(*a, **kw)	Atalho para a página interativa de studio.web.web_viewer.
serve	função	(*a, **kw)	Atalho que grava o visualizador numa pasta temporária e o serve (studio.web.serve).

pinneapple_tools.visualization.studio.animation

Animação no tempo de uma Scene: corpos rígidos em trajetória, setas por quadro, rastros, campos por vértice e números na tela, para o visualizador web e o Blender.

pinneapple_tools/visualization/studio/animation.py · 434 linhas · testes: 27

nome	tipo	assinatura	o que faz
Animation	classe	(times, tracks, vectors, trails, surface_fields, hud, camera, follow_group, follow_offset…)	Dados por quadro sobre uma cena estática (trilhas, vetores, rastros, campos de superfície, HUD, câmera), com métodos para adicionar e consultar poses.

nome	tipo	assinatura	o que faz
`matrix_from_quat`	função	`(q) -> np.ndarray`	Matriz de rotação de um ou vários quaternions unitários [w, x, y, z].
`quat_from_matrix`	função	`(R) -> np.ndarray`	Quaternion unitário de uma matriz de rotação pelo método de Shepperd (estável numericamente).
`slerp`	função	`(q0, q1, f) -> np.ndarray`	Interpolação esférica entre quaternions unitários pelo arco mais curto.

pinneapple_tools.visualization.studio.blender

Renders Blender Cycles de qualquer `Scene` (exige `pip install bpy` ou rodar dentro do Blender): imagens estáticas, animações e quadros de alta qualidade.

`pinneapple_tools/visualization/studio/blender.py` · 1.296 linhas · testes: 27

nome	tipo	assinatura	o que faz
`render`	função	`(scene, out, *, field=None, field_range=None, lines=False, line_range=None, slice=None, s…)`	Renderiza uma cena (ou um `.glb`) em imagem com campos, linhas, fatias, vista e fundo escolhidos.
`blender_axes`	função	`(axes) -> np.ndarray`	Matriz de rotação dos eixos da cena para as coordenadas do Blender.
`render_animation`	função	`(scene, out_dir, *, frames=None, size=(960, 540), samples=16, camera='chase', camera_offs…)`	Renderiza a animação da cena quadro a quadro, com câmera de perseguição, HUD e sparklines opcionais.
`render_still`	função	`(scene, out_png, frame, *, size=(1920, 1080), samples=64, **kw) -> str`	Renderiza um único quadro da animação em alta qualidade como PNG.

pinneapple_tools.visualization.studio.colormap

Escala de cor CFD (jet) e barra de cor gravada numa imagem, com ticks idênticos aos do visualizador web.

`pinneapple_tools/visualization/studio/colormap.py` · 93 linhas · testes: 14

nome	tipo	assinatura	o que faz
`jet`	função	`(t) -> Tuple[float, float, float]`	Cor sRGB jet para t em [0, 1].
`ramp`	função	`(t, stops=None) -> tuple[float, float, float]`	Cor sRGB de uma escala linear por partes (padrão jet) em t.
`decimals`	função	`(lo, hi) -> int`	Número de casas decimais que distingue cinco ticks em [lo, hi] (mesma regra do JavaScript do studio).
`fmt`	função	`(v, nd) -> str`	Formata um rótulo de tick como o JavaScript imprimiria (decimal fixo ou notação exponencial).
`ticks`	função	`(lo, hi, n=5, nd=None) -> List[str]`	Rótulos de tick igualmente espaçados, idênticos aos do visualizador web.
`colorbar`	função	`(path, title, lo, hi, nd=None, lo_txt='', hi_txt='', corner='bottom-right', stops=None) -…)`	Desenha uma barra de cor com 5 ticks e título num canto de uma imagem existente.

pinneapple_tools.visualization.studio.compose

Mapas de cor para animações (jet e divergente), barras de cor gravadas em imagens e montagem de várias imagens ou vídeos em grade.

`pinneapple_tools/visualization/studio/compose.py` · 238 linhas · testes: 19

nome	tipo	assinatura	o que faz
`stops`	função	`(cmap)`	Pontos de parada da escala de cor pedida.
`cmap_color`	função	`(cmap, t) -> tuple[float, float, float]`	Cor sRGB de um mapa em t (NaN vira 0).
`symmetric_range`	função	`(values) -> tuple[float, float]`	Faixa simétrica (−m, m) com m o maior |valor| finito, para mapas divergentes.

nome	tipo	assinatura	o que faz
default_range	função	(values, cmap='jet') -> tuple[float, float]	Faixa padrão: mín..máx para jet, simétrica em torno de zero para mapa divergente.
colorbar_image	função	(width, title, lo, hi, cmap='jet', scale=1.0, background=(255, 255, 255, 225))	Barra de cor (título, gradiente, cinco ticks) como imagem RGBA do PIL.
draw_colorbars	função	(path, bars, corner='bottom-right', width_frac=0.3) -> str	Grava várias barras de cor empilhadas num canto de uma imagem (sobrescreve o arquivo).
compose_grid	função	(images, out_png, labels=None, ncols=2, colorbar=None, background=(233, 237, 242)) -> str	Monta várias imagens numa grade com rótulos e uma barra de cor comum.
compose_grid_video	função	(mp4_paths, out_mp4, labels=None, ncols=2, colorbar=None, crf=28, webm=False, fps=None, b…)	Monta vários vídeos numa grade com ffmpeg xstack.

pinneapple_tools.visualization.studio.fieldview

Campos sobre uma malha para as vistas 3D dos apps: superfície de contorno de uma malha pinneapple_data.cae com vários campos, deformação e regiões marcadas, como glTF e JSON.

pinneapple_tools/visualization/studio/fieldview.py · 179 linhas · testes: 9

nome	tipo	assinatura	o que faz
FieldView	classe	(scene, meta)	A vista pronta: cena e metadados, com glb() e to_dict().
colour_range	função	(values, kind='value') -> dict[str, Any]	Faixa de cor de um campo: valores de mín a máx; erros de 0 ao percentil 99, marcando quando houve corte.
length_factor	função	(unit, to='m') -> float | None	Fator de conversão de uma unidade de comprimento para outra (None se não for comprimento).
displacement_factor	função	(name, unit, components, mesh_unit='m') -> float | None	Se um campo vetorial nodal é um deslocamento, o fator que o converte para a unidade de comprimento da malha.
subsample	função	(points, n=MAX_MARKER_POINTS) -> np.ndarray	No máximo n pontos espalhados por igual no array (para marcadores).
field_view	função	(mesh, fields, *, deform=None, markers=(), max_faces=VIEW_MAX_FACES, title='', axes='z_up…)	Monta o FieldView de uma malha com campos, deformação e marcadores.

pinneapple_tools.visualization.studio.lighting

Presets de iluminação e céu compartilhados pelos renderizadores do studio (PyVista, Blender Cycles, three.js).

pinneapple_tools/visualization/studio/lighting.py · 171 linhas · testes: 6

nome	tipo	assinatura	o que faz
LightingPreset	classe	(name, sun_elevation_deg, sun_azimuth_deg, sun_intensity, sun_color, ambient, background_…)	Um preset de luz: elevação, azimute, intensidade e cor do Sol, ambiente, fundo, sombras e Terra no céu.
get_preset	função	(preset) -> LightingPreset	Devolve um LightingPreset pelo nome.
apply_pyvista	função	(plotter, preset='lunar_south_pole', target=(0.0, 0.0, 0.0), extent=10.0, earth=None, sha…)	Aplica o preset a um plotter PyVista: fundo, Sol direcional com sombras e Terra opcional.
add_earth_pyvista	função	(plotter, preset, target, extent)	Adiciona a Terra como esfera distante com o tamanho angular correto, iluminada pela direção do Sol.
threejs_config	função	(preset='lunar_south_pole') -> dict	O preset como dados simples para os visualizadores no navegador.
blender_world	função	(preset='lunar_south_pole', bpy=None)	Configura a cena do Blender: cor do mundo, lâmpada Sol com tamanho angular e intensidade do preset.

pinneapple_tools.visualization.studio.particles

Animações de partículas (milhares de esferas coloridas por um valor) dentro de equipamentos que podem se mover, renderizadas com Blender Cycles e compostas com gráficos ao vivo. [sem teste]

pinneapple_tools/visualization/studio/particles.py · 323 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
render_particle_frames	função	(frames, d, out_dir, *, equipment=None, field='speed', field_range=None, view=(-1.0, -1.2…)	Renderiza quadros de partículas com Blender Cycles, com equipamento, campo de cor e câmera.
compose_video	função	(images, times, out, *, charts=(), colorbar=None, cmap='viridis', clock='Time: {t:.1f}', …)	Compõe cada imagem com gráficos que se desenham ao longo do tempo, barra de cor e relógio, em GIF ou MP4.
encode_mp4	função	(pngs, out, fps=10) -> str	MP4 H.264 de uma sequência PNG usando o sequenciador do Blender (sem binário ffmpeg).

pinneapple_tools.visualization.studio.planetary

Renders PyVista de um veículo numa superfície planetária: terreno por mapa de alturas com fotometria e sombras do Sol, peças com materiais, rochas, mapas de campo e horizonte.

pinneapple_tools/visualization/studio/planetary.py · 348 linhas · testes: 17

nome	tipo	assinatura	o que faz
SurfaceView	classe	(X, Y, Z, parts, rocks, preset, terrain_field, field_alpha, field_glow, field_saturation,…)	Descrição completa da vista de superfície: terreno, peças, rochas, preset de luz, campos, mapa de cor, exposição, Terra e terreno distante.
horizon_distance	função	(height, planet_radius=MOON_RADIUS) -> float	Distância até o horizonte de uma esfera lisa vista de uma altura dada (padrão: raio da Lua).
ring_height	função	(view, x, y)	Altura do anel de horizonte em (x, y), misturando o terreno distante com o nível da borda.
terrain_height	função	(view, x, y)	Altura do solo renderizado em (x, y): trecho detalhado, grade distante e anel de horizonte.
border_points	função	(view, n=720) -> np.ndarray	Pontos na borda externa do solo renderizado.
border_visibility	função	(view, camera, size=(1600, 900), view_angle=35.0, samples=400, margin=0.0) -> dict	Verifica se a borda externa do solo aparece vista da câmera, projetando os pontos de borda.
render_surface	função	(view, path, camera, *, size=(1600, 900), view_angle=35.0, title='', subtitle='', plotter…)	Renderiza a vista de superfície com uma câmera dada, em arquivo ou como imagem.

pinneapple_tools.visualization.studio.pyvista_render

Renders fora da tela de uma Scene do studio com PyVista (VTK), a alternativa sem Blender para máquinas sem bpy.

pinneapple_tools/visualization/studio/pyvista_render.py · 157 linhas · testes: 4

nome	tipo	assinatura	o que faz
render_scene	função	(scene, path, *, field=None, cmap='jet', clim=None, lines=False, slice_name=None, view='i…)	Renderiza a cena com campo, linhas, fatia, vista e chão opcionais num PNG.

pinneapple_tools.visualization.studio.scene

Cena 3D para qualquer geometria: superfícies com materiais e campos por vértice, linhas de corrente e planos de fatia, com leitura (STL, OBJ, glTF, VTP) e escrita (glTF, USDA, STL).

pinneapple_tools/visualization/studio/scene.py · 682 linhas · testes: 46

nome	tipo	assinatura	o que faz
Surface	classe	(name, vertices, faces, material, fields, group)	Uma superfície triangulada com material, campos por vértice e grupo.
Lines	classe	(points, values, name, label, cmap, alpha, radius, color_range)	Polilinhas (por exemplo, linhas de corrente) com valores opcionais por ponto e mapa de cor.

nome	tipo	assinatura	o que faz
`Slice`	classe	`(name, origin, u, v, grid, label, group)`	Plano retangular com uma grade de valores (NaN onde não há valor).
`Scene`	classe	`(surfaces, lines, slices, axes, title, labels, lighting, materials, animation)`	A cena completa (superfícies, linhas, fatias, eixos, iluminação, materiais, animação), criada de arrays, arquivo, malha ou peças, com dizimação e save.
`decimate_surface`	função	`(srf, target_faces) -> 'Surface'`	Simplifica uma superfície por agrupamento de vértices numa grade, ajustada por bisseção para chegar perto do número de faces pedido.
`scalar_of`	função	`(a) -> np.ndarray`	Um valor por entrada: o próprio, a magnitude de um vetor ou a tensão de von Mises de um tensor de 6 componentes.
`vertex_normals`	função	`(V, F) -> np.ndarray`	Normais por vértice de uma malha de triângulos.
`read_stl`	função	`(path) -> List[Tuple[str, np.ndarray, np.ndarray]]`	Lê um STL e devolve (nome, vértices, faces) por sólido, com vértices duplicados fundidos.
`read_obj`	função	`(path) -> Tuple[np.ndarray, np.ndarray]`	Lê vértices e faces de um arquivo OBJ.
`read_gltf`	função	`(path, axes='z_up') -> List[Surface]`	Lê as malhas de um `.glb`/`.gltf` com transformações aplicadas, convertidas para os eixos da cena.
`read_vtp`	função	`(path, material='grey') -> Surface`	Lê um VTK PolyData (`.vtp`) com seus dados de ponto e célula; exige `vtk`.
`write_glb`	função	`(surfaces, T, materials, generator='PINNeAPPle', groups=None) -> bytes`	Escreve glTF 2.0 binário com materiais PBR, e cada campo por vértice como atributo.
`write_usda`	função	`(surfaces, materials, title='scene', root='Scene') -> str`	Escreve OpenUSD em texto: uma Mesh por superfície com materiais `UsdPreviewSurface` e campos como primvars.
`write_stl`	função	`(surfaces, solid='scene') -> bytes`	Escreve um STL binário de todas as superfícies.

pinneapple_tools.visualization.studio.shadows

Sombras do Sol e sombreamento de superfície planetária para terrenos por mapa de alturas, independentes do renderizador.

`pinneapple_tools/visualization/studio/shadows.py` · 171 linhas · testes: 17

nome	tipo	assinatura	o que faz
`terrain_self_shadow`	função	`(Z, cell, sun_dir, max_distance=None) -> np.ndarray`	Máscara booleana das células onde o próprio terreno esconde o Sol.
`points_in_terrain_shadow`	função	`(P, X, Y, Z, sun_dir, step=None) -> np.ndarray`	Para cada ponto 3D, diz se o terreno bloqueia o Sol.
`cast_shadow`	função	`(meshes, X, Y, Z, sun_dir, iterations=3, supersample=1) -> np.ndarray`	Máscara das células da grade que ficam na sombra de malhas de triângulos (por exemplo, um veículo).
`lunar_lambert`	função	`(normals, sun_dir, view_dir, L=1.0) -> np.ndarray`	Refletância Lunar-Lambert de McEwen (1991), sem o albedo, para normais, Sol e direção de vista.
`relief_noise`	função	`(X, Y, rms, wavelengths=(0.6, 8.0), n=40, seed=7, slope=1.2)`	Relevo aleatório suave ancorado no mundo, como soma de ondas planas com direções e fases aleatórias.
`grid_normals`	função	`(Z, cell) -> np.ndarray`	Normais de um mapa de alturas em grade regular.
`shade_terrain`	função	`(X, Y, Z, sun_dir, camera_position, albedo=(0.45, 0.44, 0.42), shadow=None, exposure=1.0,…)`	Cores do terreno com fotometria Lunar-Lambert, sombra, exposição e luz de preenchimento.

pinneapple_tools.visualization.studio.solver_view

Resultado de uma execução de solver (CalculiX ou OpenFOAM) como vista 3D para conferir antes de usar os números: malha, campos, forma deformada e regiões de contorno e carga.

`pinneapple_tools/visualization/studio/solver_view.py` · 153 linhas · testes: 6

nome	tipo	assinatura	o que faz
`find_result`	função	`(folder) -> tuple[str, Path] | None`	Identifica numa pasta um resultado CalculiX (`.frd`) ou um caso OpenFOAM; devolve None se não achar.
`solver_result_view`	função	`(folder, *, max_faces=VIEW_MAX_FACES) -> tuple[FieldView, dict[str, Any]]`	Monta o `FieldView` e um resumo do resultado do solver na pasta; levanta `ValueError` quando não há resultado.

pinneapple_tools.visualization.studio.web

Visualizador interativo de uma Scene no navegador (three.js embutido na biblioteca, sem internet).

`pinneapple_tools/visualization/studio/web.py` · 197 linhas · testes: 27

nome	tipo	assinatura	o que faz
`web_viewer`	função	`(scene, out_dir, title=None, max_faces=MAX_FACES, single_file=False, filename='index.html…)`	Grava o visualizador da cena numa pasta (ou num arquivo único) e devolve o caminho do `index.html`.
`serve`	função	`(scene, port=8000, open_browser=True, block=True, out_dir=None, max_faces=MAX_FACES)`	Grava o visualizador e o serve numa porta local, abrindo o navegador opcionalmente.
`screenshot`	função	`(viewer_dir, out_png, frame=None, size=(1280, 720), setup_js=None, timeout_ms=60000) -> s…)`	Captura um PNG do visualizador gravado com navegador headless (Playwright); devolve None se não for possível.

pinneapple_tools.visualization.style

Mapas de cor, temas escuro/claro e padrões de figura no estilo CFD. [sem teste]

`pinneapple_tools/visualization/style.py` · 118 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`use_cfd_style`	função	`(dark=True) -> None`	Aplica globalmente o tema CFD escuro ou claro no matplotlib.
`get_cmap`	função	`(name=DEFAULT_CMAP)`	Devolve um mapa de cor do matplotlib por apelido do PINNeAPPle ou nome padrão.
`make_figure`	função	`(nrows=1, ncols=1, **kwargs)`	Cria uma figura matplotlib já estilizada.

pinneapple_tools.visualization.trust_panels

Painéis de campo lado a lado com um veredito de confiança sob cada modelo: qual previsão dá para usar e onde ela quebra a física.

`pinneapple_tools/visualization/trust_panels.py` · 167 linhas · testes: 2

nome	tipo	assinatura	o que faz
`Panel`	classe	`(title, values, kind, verdict, verdicts, note, inset)`	Um painel: campo, resíduo (log) ou erro (linear a partir de 0), com veredito e nota.
`Row`	classe	`(label, panels)`	Uma linha da figura: rótulo e lista de painéis.
`trust_panels`	função	`(rows, *, title='', footer='', cell=2.4, residual_range=None, error_max=None, error_label…)`	Figura com uma linha por `Row`: painéis de campo, barra de cor, resíduos e erros, com vereditos.
`trust_panels_gif`	função	`(frames, path, *, duration_ms=1600, **kwargs) -> Path`	Uma figura `trust_panels` por quadro, gravada como GIF em laço.

pinneapple_tools.visualization.vortex

Identificação de vórtices e visualização de escoamento: vorticidade, critério Q, λ_2, enstrofia, dissipação e painéis prontos (inclusive para LBM).

`pinneapple_tools/visualization/vortex.py` · 675 linhas · testes: 10

nome	tipo	assinatura	o que faz
compute_vorticity_2d	função	(u, v, dx=1.0, dy=1.0) -> np.ndarray	Componente z da vorticidade ω_z = ∂v/∂x − ∂u/∂y por diferenças finitas.
compute_q_criterion_2d	função	(u, v, dx=1.0, dy=1.0) -> np.ndarray	Critério Q para escoamento 2D incompressível.
compute_vorticity_3d	função	(u, v, w, dx=1.0, dy=1.0, dz=1.0) -> np.ndarray	Vetor vorticidade ω = ∇×u completo para escoamento 3D.
compute_strain_rate_tensor_3d	função	(u, v, w, dx=1.0, dy=1.0, dz=1.0)	Tensor taxa de deformação simétrico S_ij para escoamento 3D.
compute_q_criterion_3d	função	(u, v, w, dx=1.0, dy=1.0, dz=1.0) -> np.ndarray	Critério Q 3D: Q = 0.5(‖Ω‖² − ‖S‖²).
compute_enstrophy	função	(vorticity) -> float	Enstrofia: 0.5 vezes a soma de |ω|².
compute_dissipation_rate	função	(strain_rate_tensor_components, nu) -> np.ndarray	Taxa de dissipação viscosa ε = 2ν S_ij S_ij.
compute_lambda2_3d	função	(u, v, w, dx=1.0, dy=1.0, dz=1.0) -> np.ndarray	Critério λ_2 de Jeong & Hussain (1995); núcleo de vórtice onde $\lambda_2 < 0$.
plot_vorticity	função	(x, y, omega, *, title='Vorticity ω_z', cmap='RdBu_r', symmetric=True, obstacle_mask=None…)	Mapa de vorticidade com mapa de cor divergente centrado em zero e máscara de obstáculo opcional.
plot_q_criterion_2d	função	(x, y, Q, vel_mag=None, *, title='Q-criterion', q_levels=30, highlight_positive=True, cma…)	Gráfico do campo Q 2D, destacando regiões Q > 0, com velocidade de fundo opcional.
plot_vortex_identification	função	(x, y, u, v, *, dx=1.0, dy=1.0, title=…, obstacle_mask=None, figsize=(11, 5), show=False)…)	Calcula Q a partir da velocidade e plota com linhas de corrente sobrepostas.
plot_q_criterion_3d	função	(x, y, z, Q, *, level=0.01, vel_mag=None, title='Q-criterion iso-surface', cmap='rainbow'…)	Isosuperfície 3D do critério Q, colorida opcionalmente pela velocidade.
plot_lbm_flow	função	(ux, uy, rho=None, obstacle_mask=None, *, dx=1.0, dy=1.0, title='LBM Flow Field', figsize…)	Painel de 4 gráficos para escoamento LBM: magnitude da velocidade, vorticidade, critério Q e pressão (via densidade), com obstáculos em cinza.
plot_flow_panel	função	(x, y, ux, uy, pressure=None, obstacle_mask=None, *, title='Flow Field Analysis', figsize…)	Painel de 4 gráficos: pressão, |v| com linhas de corrente, critério Q e vorticidade.

`pinneapple_tools.visualization.voxel`

Visualização de objetos `VoxelGrid` (fatias, ocupação 3D e histograma). [sem teste]

`pinneapple_tools/visualization/voxel.py` · 127 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
plot_voxel_slice	função	(grid, *, axis='z', index=None, cmap=DEFAULT_CMAP, title='', show=False) -> Figure	Plota uma fatia 2D de uma `VoxelGrid` 3D ao longo de um eixo.
plot_voxel_3d	função	(grid, *, threshold=0.5, cmap=DEFAULT_CMAP, alpha=0.6, title='Voxel occupancy', show=Fals…)	Dispersão 3D dos voxels ocupados acima de um limiar, limitada a um número máximo.
plot_voxel_histogram	função	(grid, *, bins=50, title='Voxel value distribution', show=False) -> Figure	Histograma dos valores não nulos dos voxels.

Exemplos de uso reais do repositório

Experimento Arena a partir de YAML — `examples/benchmark_suite/02_arena_from_yaml.py`

```python
from pinneapple_tools.benchmark_suite.runner.run_arena_yaml import run_arena_experiment


def main() -> None:
    repo_root = Path(__file__).resolve().parents[2]
    config_path = repo_root / "examples" / "benchmark_suite" / "configs" / "experiment_burgers_1d.yaml"
    out_dir = repo_root / "data" / "artifacts" / "experiments" / "burgers_1d"
```

```
    ...
    result = run_arena_experiment(config_path, out_dir=out_dir)
```

Campo de temperatura num stage OpenUSD, validado e pré-visualizado — examples/nvidia/07_omniverse_physicsnemo_coldplate/run.py

```
st = FieldStage(usd, "Cold plate, PhysicsNeMo FNO temperature")
path = st.add_mesh("ColdPlate", V, F, fields={"temperature": T.ravel(), "conductivity": (a_te[i] * K0).ravel(),
                                              "temperature_reference": (T_COOL + DT * u_te[i]).ravel(),
                                              "estimated_error": (DT * np.abs(est)).ravel()},
                   units={"temperature": "K", "conductivity": "W/m/K", "temperature_reference": "K",
                          "estimated_error": "K"}, color_by="temperature")
st.set_trust(path, decision, [c.to_dict() for c in checks], model="physicsnemo.models.fno.FNO",
             solver="pinneapple finite volumes")
st.save()
v = validate_stage(usd, required_primvars={path: ["temperature", "conductivity"]})
```

Painel animado de um campo no tempo — examples/quickstart/11_visualisation.py

```
from pinneapple_tools.visualization.dashboard import Chart, Dashboard, Series, Tile
...
dashboard = Dashboard(
    "Diffusing blobs",
    t,
    frames,
    subtitle="T(x, y, t): two Gaussian blobs diffusing and decaying",
    charts=[Chart("peak amplitude", "amplitude [a.u.]", "time [s]",
                  [Series("peak", np.asarray(peaks))])],
    tiles=[Tile("final peak", peaks[-1], "a.u."), Tile("frames", len(frames), "")],
)
written = dashboard.to_video(gif=f"{OUT}/04_field_4d.gif")
```

Coeficiente de pressão com `compute_cp` — tests/pinneapple_tools/test_aero_coefficients.py

```
def test_compute_cp_basic():
    p = np.array([100.0, 150.0, 50.0])
    p_inf = 100.0
    q_inf = 50.0
    cp = compute_cp(p, p_inf, q_inf)
    np.testing.assert_allclose(cp, [0.0, 1.0, -1.0])
```

Frase -> workflow do catálogo de automação — tests/test_automation.py

```
def test_sentence_picks_the_workflow_and_converts_units():
    cat = catalog()
    it = interpret("Run the channel with the obstacle height set to 5.5 cm at an inlet velocity of 3 cm/s", cat)
    assert it.workflow.id == "channel_obstacle" and it.method == "rules"
    assert it.inputs == {"obstacle_height_mm": pytest.approx(55.0), "inlet_velocity_mm_s": pytest.approx(30.0)}
    with pytest.raises(WorkflowError, match="outside"):
        interpret("channel obstacle height 95 mm", cat)
    with pytest.raises(WorkflowError, match="no committed workflow"):
        interpret("bake a lemon cake", cat)
```

Servir o zoo como microserviço NIM e consultar — examples/nvidia/06_model_services_zoo/run.py

```
    srv, url = serve_in_thread(default_registry())
    c = ModelServiceClient(url)
    try:
        with info.step("service discovery"):
            ready = c.ready()
```

Testes e status de validação

- Módulos com pelo menos um teste que os importa: 70 de 177.
- Funções de teste em arquivos que importam algum módulo do pacote: 465.

Limitações conhecidas

- `benchmark_suite/__init__.py` importa `PhysicsPipeline`, `TimeSeriesBenchmarkSpec`, `run_full_pipeline` e os benchmarks de transferência e meta-aprendizado dentro de `try/except Exception: pass`: se uma dependência falhar, o nome some sem aviso.
- Vários docstrings da `benchmark_suite` ainda citam nomes antigos de pacotes (`pinneapple_arena`, `pinneapple_environment`, `pinneapple_pinn`, `examples/pinneapple_arena/`), embora o código já importe os pacotes atuais.
- Os backends DeepXDE, JAX e PhysicsNeMo Sym dependem de pacotes opcionais e falham com `RuntimeError`/`ImportError` se eles não estiverem instalados; DeepXDE, JAX, nativo e multi-modelo não têm teste direto.
- `BenchmarkTaskBase` levanta `NotImplementedError` nos métodos abstratos; os runners de linha de comando (`runner/*`), `physics_pipeline` e `timeseries_pipeline` não têm teste direto.
- `hpo_experiments` não tem nenhum teste, depende de rede (arXiv, GitHub) e de uma chave da API Gemini, e o pipeline `reproduce` executa com `python run.py`/`main.py` código gerado por LLM, sem sandbox.
- `export_problemdesign` só escreve o arquivo complementar se `pinneapple_problemdesign` puder ser importado (o pacote existe no repositório; o import fica dentro de um try).
- O docstring de `pinneapple_tools.nvidia` lista os módulos `nim`, `cosmos` e `testing`, que não existem na pasta; Isaac Sim, Omniverse Kit RTX e USD BRep (wheel CPython 3.12) são externos e opcionais.
- Partes do `visualization` exigem pacotes opcionais (`bpy` para Blender, `pyvista`/`vtk`, `rerun-sdk`, `plotly`, `trame`, `playwright` para capturas, ffmpeg para MP4); muitos módulos de gráficos clássicos (`fields`, `pinn`, `solver`, `mesh`, `voxel`, `animation`, `style`) não têm teste direto.
- `sandbox.custom_solver_runner` não é uma fronteira de segurança: só limita memória virtual, CPU e tempo de relógio; o script ainda acessa arquivos, rede e pode criar processos (aviso do próprio docstring).
- `automation` depende de aplicações externas (FreeCAD, CfdOF, OpenFOAM, gmsh, ParaView, Xvfb, xdotool, Playwright); sem elas os passos ficam bloqueados. `PywinautoDriver` só funciona no Windows. Os adaptadores, o runner, o display e o relatório não têm teste direto (os testes cobrem workflow/interpretação/registro/verificação por outros módulos).
- As correções de compatibilidade em `adapters.openfoam` (`patch_cfdof_allmesh`, `patch_cfdof_inlet`) são específicas do OpenFOAM v1912 / versões ESI antigas com CfdOF.
- `dataset_quality.vlm_curation` precisa do extra `vlm_curation` (`transformers`, `accelerate`) e, para checkpoints hospedados, de GPU e download do Hugging Face; não há substituto offline para o julgamento do VLM.
- `compute_backends.jax_backend` exige JAX (e Equinox ou Flax para os modelos); sem JAX o módulo importa, mas as funções não rodam.
- `reporting.paperkit.Paper` precisa de `reportlab`; `reporting.datasheet` só gera PDF com Chromium/Chrome ou Playwright (sem eles, só HTML).
- O atalho `export_model` passa o modelo como primeiro argumento também para `csv`/`npz`, cujas funções esperam `coords` e `fields`; para esses formatos é melhor chamar `export_csv`/`export_npz` direto. `model_export` e o `__init__` do pacote não têm teste direto.
- `surrogate_policy.recommend_architecture` é uma heurística fixa (`vanilla_pinn` ou `modified_mlp` pelo número de observações), não um recomendador aprendido; `rea_bridge` precisa do REA (Node.js) instalado ou de `npx`.

2. `pinneapple_blender`

Objetivo

`pinneapple_blender` leva resultados do PINNeAPPle para o Blender. O lado Python puro (`export`) grava uma nuvem de pontos com um ou mais campos, opcionalmente em vários passos de tempo, como uma sequência de arquivos `.ply` com cores por mapa de cores. O importador nativo do Blender abre esses arquivos sem add-on.

Há duas formas de usar o Blender de verdade. `render.build_scene` chama uma instalação local do Blender em modo headless (`blender --background --python ...`), que roda o script `blender_addon/import_pinneapple_sequence.py` dentro do `bpy` do próprio Blender. O script importa cada quadro como um objeto e anima a visibilidade, e o resultado pode ser salvo em `.blend`. É o mesmo padrão de subprocesso usado pelo leitor de `.odb` do Abaqus em `cfd_formats`. Já `cycles` usa o módulo `bpy` instalado por pip, no mesmo processo, para montar cenas pequenas a partir de arrays NumPy (superfícies, tubos, pontos, planos, faixas, texto, câmera, luzes) e renderizar com Cycles na CPU. `frames_to_video` junta os quadros num MP4 com ffmpeg. Ele é usado pelos visuais de `examples/near_random_forecasting`.

Números do inventário

módulos	linhas	classes públicas	funções públicas	testes em arquivos que importam o pacote	módulos sem teste direto
5	616	1	7	15 (2 arquivos)	2

Submódulos

submódulo	o que faz	módulos	classes + funções	testes*
`blender_addon`	Script que roda dentro do Python do Blender para importar a sequência `.ply` e animar a visibilidade dos quadros.	1	1	0
`cycles`	Renderizador Cycles no mesmo processo via `bpy`: monta cenas a partir de arrays e gera imagens e vídeo.	1	3	8
`export`	Grava pontos e campos como sequência de `.ply` coloridos, sem precisar do Blender.	1	3	7
`render`	Ponte por subprocesso para uma instalação local do Blender, que monta e salva a cena.	1	1	7

* maior número de testes que importam um módulo do submódulo.

Módulos e API pública

`pinneapple_blender`

Pacote da ponte com o Blender; só o docstring descrevendo `export` e `render`. [sem teste direto]

`pinneapple_blender/__init__.py` · 15 linhas · **sem teste direto**

`pinneapple_blender.blender_addon.import_pinneapple_sequence`

Roda dentro do `bpy` do Blender (nunca importado pelo código normal): importa cada `frame_XXXX.ply` como objeto e põe keyframes de visibilidade; pode salvar `.blend`. [sem teste direto; exercitado pelos testes que exigem Blender]

`pinneapple_blender/blender_addon/import_pinneapple_sequence.py` · 95 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`main`	função	`()`	Lê os argumentos depois de `--`, importa os quadros `.ply`, anima a visibilidade e salva o `.blend` se pedido.

`pinneapple_blender.cycles`

`CyclesScene` monta uma cena Blender nova a partir de NumPy (malhas, superfícies, planos, tubos, pontos, faixas, texto, câmera, luzes) e renderiza com Cycles na CPU; exige `pip install bpy`.

`pinneapple_blender/cycles.py` · 292 linhas · testes: 8

nome	tipo	assinatura	o que faz
`CyclesScene`	classe	`(resolution=(960, 540), samples=64, background=(0.97, 0.97, 0.96), ambient=0.6, transpare…)`	Cena Blender renderizada com Cycles na CPU, com métodos para adicionar geometria, materiais, câmera, luzes e renderizar quadros.

nome	tipo	assinatura	o que faz
`available`	função	`() -> bool`	Diz se o módulo `bpy` pode ser importado.
`frames_to_video`	função	`(frames, path, *, fps=24, crf=28) -> str`	Junta quadros PNG `frame_%04d.png` num MP4 H.264 com ffmpeg.

`pinneapple_blender.export`

Grava nuvens de pontos com campos escalares/vetoriais como `.ply` só de vértices, com cores por colormap e campos extras como propriedades, um arquivo por quadro.

`pinneapple_blender/export.py` · 153 linhas · testes: 7

nome	tipo	assinatura	o que faz
`write_ply`	função	`(path, points, colors=None, scalar_fields=None) -> None`	Grava uma nuvem de pontos PLY só de vértices, com cores e campos escalares opcionais.
`export_scene`	função	`(points, field, out_dir, *, field_name='field', cmap='viridis', frame_index=0, extra_fiel…)`	Exporta um quadro como `<out_dir>/frame_<NNNN>.ply` colorido pelo campo.
`export_trajectory`	função	`(points, fields_over_time, out_dir, *, field_name='field', cmap='viridis') -> list`	Exporta uma sequência inteira: pontos fixos e um campo por quadro.

`pinneapple_blender.render`

Executa o Blender headless com o script do add-on para importar uma sequência `.ply` e opcionalmente salvar `.blend`; levanta erro claro se o executável não existe ou o Blender falha.

`pinneapple_blender/render.py` · 61 linhas · testes: 7

nome	tipo	assinatura	o que faz
`build_scene`	função	`(ply_dir, *, output_blend=None, blender_executable='blender', timeout=600) -> str`	Roda o Blender headless para importar a sequência `.ply` numa cena e opcionalmente salvar `.blend`.

Exemplos de uso reais do repositório

Exportar uma sequência .ply — `tests/test_blender_bridge.py`

```
def test_export_trajectory_writes_valid_ply_sequence_with_real_colors(tmp_path):
    rng = np.random.default_rng(0)
    points = rng.uniform(-1, 1, size=(20, 3)).astype(np.float32)
    fields = [np.sin(points[:, 0] + t) for t in np.linspace(0, np.pi, 3)]
    paths = export_trajectory(points, fields, str(tmp_path), field_name="u")

    assert len(paths) == 3
    for p in paths:
        assert Path(p).exists()
```

Cena Cycles a partir de arrays — `pinneapple_blender/cycles.py`

```
from pinneapple_blender.cycles import CyclesScene, available
if available():
    sc = CyclesScene(resolution=(960, 540), samples=64)
    sc.add_surface(X, Y, Z, colors)               # (ny, nx) grids, colors (ny, nx, 3) in 0..1
    sc.add_plane(z=0.0, size=(10, 6), color=(0.1, 0.3, 0.6), alpha=0.35)
    sc.camera(location=(8, -9, 6), look_at=(0, 0, 0))
    sc.render("landscape.png")
```

Testes e status de validação

- Módulos com pelo menos um teste que os importa: 3 de 5.
- Funções de teste em arquivos que importam algum módulo do pacote: 15.

Limitações conhecidas

- `render` precisa de um executável `blender` instalado localmente; os testes que o usam são pulados quando ele não existe.
- `cycles` precisa do módulo `bpy` (pip) e renderiza só na CPU; `frames_to_video` precisa do ffmpeg.
- Os `.ply` são só de vértices (sem faces); para ver como superfície é preciso montar geometry nodes no Blender.
- O docstring do add-on cita `render.render_sequence`, mas a função existente em `render.py` é `build_scene`.
- O `__init__` e o script do add-on não têm teste direto pelo inventário.

3. `pinneapple_flow`

Objetivo

`pinneapple_flow` representa fluxos de engenharia como grafos de nós tipados. Cada nó chama a biblioteca PINNeAPPle: geometria, equações, solvers clássicos, PINNs, operadores neurais, validação, gêmeos digitais, design. Um fluxo é um documento JSON (`pinneapple-flow/1`) com nós, arestas e parâmetros. Ele pode ser planejado a partir de uma descrição em texto (`pf.plan`), validado, executado sem interface (`pf.run` ou `python -m pinneapple_flow run`) ou editado e executado no Flow Studio (`apps/flow_studio`, servido por `python -m pinneapple_flow studio`).

O problema que ele resolve é ligar as muitas capacidades da biblioteca num caminho reprodutível e explicável, de 'o que eu quero' até métricas e figuras. A execução usa o motor de `pinneapple_core.workflows` (cache, logs, retentativas). Cada nó grava artefatos na pasta da execução, e o `FlowStore` guarda fluxos, versões, execuções e saídas por nó em SQLite ou em qualquer banco SQLAlchemy. Cada decisão automática do planejador ou dos nós segue uma regra heurística registrada (`rules`), que cita referências do registro curado (`references`) ou diz por que não tem citação. Essas regras e evidências são checadas por teste.

O pacote se liga ao resto pelos nós: chamadas curadas da biblioteca (`libcalls_catalog`, por exemplo `pp.solve` e `pp.compare`), nós gerados da API pública (`autocalls`) e nós que reproduzem estudos de `examples/*`. O módulo `coverage` mostra, para cada pacote e módulo `pinneapple*`, que nós o alcançam ou por que não há nó (shims, infraestrutura como `pinneapple_registry` e `pinneapple_mcp`). Outros pacotes e apps podem estender o fluxo com tipos de porta (`register_port_type`), referências e regras próprias. `FlowService` reúne as operações que o Flow Studio e outros front ends (como o app PINNeAPPle-CFD) expõem por HTTP.

Esta parte cobre a superfície do pacote (`__init__`), a CLI, a configuração por ambiente, a cobertura, o catálogo de chamadas, as regras, o serviço, o armazenamento, os modelos de equações e os tipos de porta. O motor, os nós e o planejador estão descritos em outra parte.

Esta parte cobre o núcleo do motor de fluxos do `pinneapple_flow` e os seus nós: o documento de fluxo (`schema`), o registro de tipos de nó com portas tipadas e formulários de parâmetros (`registry`), a execução sobre o motor de workflows de `pinneapple_core.workflows` (`runtime`), os nós embutidos (`nodes`) e os nós que reproduzem os estudos de `examples/*` (`nodes_ext`), os nós gerados a partir da API pública (`libcalls`, `autocalls`, `wrappers`), o planejador que transforma uma descrição em texto em propostas de fluxo (`planner`), a paleta do Flow Studio (`palette`, `studio`), os registros de decisão com referências checadas (`decisions`, `references`), a identificação de equações e o gêmeo digital automático (`identify`, `twin`), os surrogates treináveis (`learn`) e a ligação com o selo do PINNeAPPle Lab (`lab_status`, `examples`).

Números do inventário

módulos	linhas	classes públicas	funções públicas	testes em arquivos que importam o pacote	módulos sem teste direto
89	18.086	23	381	263 (26 arquivos)	58

Submódulos

submó-dulo	o que faz	mó-dulos	classes + funções	tes-tes*
`__main__`	CLI `python -m pinneapple_flow`: run, plan, validate, catalog, schema, examples, coverage, references, autocalls e studio.	1	10	10
`autocalls`	Gera nós `auto.<módulo>.<função>` a partir das funções públicas da biblioteca que podem ser chamadas com segurança por um formulário, gravados em `autocalls.json`.	1	6	10
`config`	Configurações por ambiente e `.env`: provedor de LLM, URL do banco, pastas e endereço do servidor.	1	5	0
`coverage`	Tabela de quais módulos da biblioteca são alcançados por nós de fluxo e o motivo dos que não são.	1	6	10
`decisions`	Registros de decisão de um fluxo (quem decidiu, por quê, com quais referências e evidências, com que confiança) e a seção de relatório em Markdown/HTML.	1	11	22
`examples`	Inventário de `examples/*` como fluxos: configurações rápidas, execução, comparações medidas e o documento `EXAMPLES_AS_FLOWS.md`.	1	15	8
`identify`	Identifica as equações que governam um problema a partir do texto (heurística offline ou LLM) e as descobre a partir de dados por regressão esparsa.	1	4	7
`lab_status`	Lê o selo do PINNeAPPle Lab de um item validado e diz que nós inserir no fluxo para subir de letra.	1	5	9
`learn`	Surrogates para nós de fluxo (blocos MLP, DeepONet, FNO, POD + MLP): construção, prévia como grafo de camadas, treino e busca de hiperparâmetros.	1	14	0
`libcalls`	Lista curada de chamadas públicas da biblioteca expostas como nós, com formulários gerados por introspecção e tipos de nó definidos pelo usuário.	1	10	10
`libcalls_catalog`	Lista curada de chamadas da biblioteca expostas como nós `lib.<chave>`.	1	0	0
`nodes`	Tipos de nó embutidos: problema, dados, solvers, PINN, surrogates, métodos, validação, sistemas, raciocínio, veículo, visualização.	20	91	27
`nodes_ext`	Tipos de nó que reproduzem os estudos de `examples/*` como fluxos, um módulo por domínio, e as capacidades do planejador para eles.	38	112	8
`palette`	A paleta de nós do Flow Studio: seções em ordem de fluxo de engenharia, títulos legíveis, resumos e ranking de busca.	1	12	29
`planner`	Planejador: de uma descrição do problema a propostas de fluxo, por heurística offline, por LLM validado ou pelo assistente guiado.	6	23	16
`references`	Registro curado de referências (`references.yaml`) e a checagem (registro, DOI/URL) por que passa toda referência citada.	1	10	13
`registry`	Tipos de nó: portas tipadas, esquemas de parâmetros, a chamada de biblioteca de cada um e o decorador `node` que os registra.	1	6	46
`rules`	Regras heurísticas por trás de cada decisão automática, com as referências e evidências que as sustentam.	1	3	13
`runtime`	Executa um fluxo sobre o motor de workflows de `pinneapple_core.workflows`, com cache, reexecução parcial, artefatos e ganchos de fim de execução.	1	15	9
`schema`	O documento de fluxo `pinneapple-flow/1` (nós, arestas, metadados): carregar, salvar, validar e JSON Schema.	1	9	27
`service`	`FlowService`: as operações do Flow Studio como métodos simples (planejar, validar, salvar, executar, resultados).	1	1	29
`store`	`FlowStore`: fluxos, versões, execuções, decisões e saídas por nó em SQLite ou banco SQLAlchemy.	1	3	23
`studio`	Sobe o Flow Studio (`apps/flow_studio`, FastAPI + frontend React) em localhost, exigindo login para outros endereços.	1	3	38
`templates`	Modelos de equações por família de EDP e dimensão, com condições, coeficientes e solução exata.	1	2	0
`twin`	Gêmeo digital montado automaticamente a partir de um sistema de equações identificado, sobre o `TwinRuntime` (filtro de Kalman por ensemble).	1	3	7
`types`	Tipos de porta do grafo e a regra de quais saídas podem alimentar quais entradas.	1	2	7
`wrappers`	Pontos de entrada com argumentos JSON para capacidades da biblioteca cuja API recebe objetos, para virarem nós de chamada de biblioteca.	1	18	0

* maior número de testes que importam um módulo do submódulo.

Módulos e API pública

`pinneapple_flow`

Superfície pública: `Flow`, `Node`, `Edge`, validação e schema, `REGISTRY` e `@node`, tipos de porta, e atalhos `run`, `plan`, `catalog`, `register_references`, `register_rule`; `FlowStore`, `Rule` e `NotRun` são carregados sob demanda.

`pinneapple_flow/__init__.py` · 91 linhas · testes: 231

nome	tipo	assinatura	o que faz
`register_references`	função	`(entries)`	Adiciona referências curadas para regras de outro pacote.
`register_rule`	função	`(r)`	Adiciona uma regra heurística que cita referências registradas.
`run`	função	`(flow, **kw)`	Executa um fluxo sem interface (via `runtime.run`).
`plan`	função	`(text, **kw)`	Gera propostas de fluxo para uma descrição (via `planner.plan`).
`catalog`	função	`() -> list[dict]`	Devolve todos os tipos de nó em JSON (portas, schema de parâmetros, disponibilidade).

`pinneapple_flow.__main__`

CLI com subcomandos `run`, `plan`, `validate`, `catalog`, `schema`, `examples`, `example-flows`, `coverage`, `references`, `autocalls` e `studio`, imprimindo o estado por nó.

`pinneapple_flow/__main__.py` · 212 linhas · testes: 10

nome	tipo	assinatura	o que faz
`cmd_run`	função	`(a) -> int`	Subcomando `run`: executa um fluxo de arquivo ou embutido e imprime estado, tempo e métricas por nó.
`cmd_plan`	função	`(a) -> int`	Subcomando `plan`: propostas para uma descrição, opcionalmente gravando o fluxo.
`cmd_validate`	função	`(a) -> int`	Subcomando `validate`: valida um arquivo de fluxo e lista os problemas.
`cmd_catalog`	função	`(a) -> int`	Subcomando `catalog`: lista os tipos de nó, em texto ou JSON.
`cmd_examples`	função	`(a) -> int`	Subcomando `examples`: lista os fluxos de exemplo embutidos.
`cmd_coverage`	função	`(a) -> int`	Subcomando `coverage`: escreve a tabela de cobertura em markdown.
`cmd_autocalls`	função	`(a) -> int`	Subcomando `autocalls`: regenera os nós gerados da API pública (`autocalls.json`).
`cmd_references`	função	`(a) -> int`	Subcomando `references`: lista o registro de referências; `--verify` resolve os DOIs online.
`cmd_studio`	função	`(a) -> int`	Subcomando `studio`: serve o backend e o frontend do Flow Studio.
`main`	função	`(argv=None) -> int`	Ponto de entrada da CLI: monta os subcomandos e despacha.

`pinneapple_flow.autocalls`

Varre os módulos públicos que nenhum nó curado cobre e gera nós `auto.*` só para funções com argumentos de tipo simples e sem caminhos, URLs, comandos, `eval` ou escrita de arquivo; o resultado fica em `autocalls.json` e é regenerado por `python -m pinneapple_flow autocalls`.

`pinneapple_flow/autocalls.py` · 236 linhas · testes: 10

nome	tipo	assinatura	o que faz
scan	função	(module) -> tuple[list[dict[str, Any]], str]	Gera as entradas de nó de um módulo, ou a razão de ele não ter nenhuma.
generate	função	(modules=None) -> dict[str, Any]	Varre os módulos (por padrão todos os que os nós curados não cobrem) e devolve os dados de autocalls.json.
smoke	função	(calls, *, timeout=30.0, workers=6) -> None	Chama cada função gerada com os padrões do formulário num interpretador novo em pasta vazia e grava o resultado em entry["check"].
prune	função	(data) -> None	Descarta entradas cuja checagem mostra que devolvem uma função ou classe, e dá essa razão ao módulo que ficar vazio.
load	função	() -> dict[str, Any]	Lê autocalls.json (ou devolve listas vazias se o arquivo não existe).
write	função	(data) -> None	Grava os dados em autocalls.json.

pinneapple_flow.config

Settings lido de variáveis PINNEAPPLE_FLOW_* (home, banco, pasta de dados, host/porta, LLM), leitor de .env e safe_data_path, que impede caminhos fora da pasta de dados. [sem teste direto pelo inventário]

pinneapple_flow/config.py · 99 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
Settings	classe	(host, port, db_url, llm_provider, llm_model, llm_base_url, llm_api_key)	Configurações do servidor e do LLM lidas do ambiente; public mascara a chave e a senha do banco.
load_dotenv	função	(path='.env', *, override=False) -> dict[str, str]	Lê linhas KEY=value de um .env para os.environ.
home	função	() -> Path	Pasta base do Flow (PINNEAPPLE_FLOW_HOME, padrão ~/.pinneapple_flow).
data_root	função	() -> Path	Única pasta de onde nós de ingestão podem ler arquivos.
safe_data_path	função	(name) -> Path	Caminho dentro da pasta de dados, recusando caminhos absolutos fora dela e ...

pinneapple_flow.coverage

Percorre todos os pacotes pinneapple* e seus módulos de primeiro nível, mapeia os nós que os chamam e anexa um motivo curado (REASONS) aos sem nó; gera tabela markdown e falha se faltar nó e motivo.

pinneapple_flow/coverage.py · 183 linhas · testes: 10

nome	tipo	assinatura	o que faz
packages	função	() -> list[str]	Lista os pacotes pinneapple* do repositório.
modules	função	(pkg) -> list[str]	Lista os módulos de primeiro nível de um pacote.
node_targets	função	(auto=True) -> dict[str, set[str]]	Mapa módulo → tipos de nó que o chamam.
coverage	função	(auto=True) -> list[dict[str, str]]	Uma linha por pacote e módulo com os nós que o alcançam ou o motivo de não haver.
counts	função	(rows=None) -> dict[str, int]	Contagens resumidas das linhas de cobertura.
markdown	função	() -> str	Tabela de cobertura em markdown.

pinneapple_flow.decisions

Define Decision e as fábricas de decisões de regra, de usuário e de LLM; a confiança vem de quem decidiu e da qualidade das referências (registro > verificada > não verificada), limitada a 0.35 sem referência nem evidência.

pinneapple_flow/decisions.py · 236 linhas · testes: 22

nome	tipo	assinatura	o que faz
`Decision`	classe	`(topic, what, decided_by, rationale, value, references, evidence, node, notes, uncited_re…)`	Registro de uma decisão do fluxo: tema, o que foi decidido, por quem, justificativa, valor, referências, evidências, confiança e rótulo.
`score`	função	`(d) -> Decision`	Calcula a confiança de uma decisão a partir de quem decidiu, das referências e das evidências; sem nenhuma, fica no máximo 0.35.
`rule_decision`	função	`(rule_id, *, what, node='', value=None, rationale='', topic=None, extra=()) -> Decision`	Cria a decisão tomada por uma regra heurística, com as referências e evidências da regra.
`user_decision`	função	`(node, topic, what, value=None, *, notes='', references=(), user='user') -> Decision`	Cria a decisão de uma edição do usuário, com notas e referências checadas no registro ou por DOI/URL.
`llm_decision`	função	`(provider_model, *, topic, what, node='', value=None, rationale='', references=()) -> Dec…)`	Cria a decisão proposta por um LLM; cada referência é checada e nada é aceito só pela palavra do modelo.
`edits_to_decisions`	função	`(edits) -> list[Decision]`	Converte as edições em `flow.meta["user_edits"]` em decisões de usuário.
`collect`	função	`(flow, reports=None) -> list[dict[str, Any]]`	Junta todas as decisões de um fluxo: do planejador/LLM, das edições do usuário e as tomadas pelos nós durante a execução.
`by_node`	função	`(decisions) -> dict[str, list[dict[str, Any]]]`	Agrupa decisões pelo nó a que se referem.
`summary`	função	`(decisions) -> dict[str, Any]`	Resumo das decisões: total, quantas sem referência, contagem por rótulo e referências citadas.
`report_markdown`	função	`(decisions, title='Decisions and references') -> str`	Gera a seção "Decisions and references" do relatório em Markdown.
`report_html`	função	`(decisions, title='Decisions and references') -> str`	Gera a mesma seção em HTML com referências clicáveis.

`pinneapple_flow.examples`

Liga cada pasta de `examples/*` aos seus fluxos em `flows/examples/`, roda os fluxos (completo ou rápido), grava os resultados em `measured.json` e gera `EXAMPLES_AS_FLOWS.md`; `problems` é o portão de CI.

`pinneapple_flow/examples.py` · 410 linhas · testes: 8

nome	tipo	assinatura	o que faz
`inventory`	função	`() -> dict[str, Any]`	Lê `flows/examples/inventory.json` (estado e razão de cada pasta de exemplo).
`measured`	função	`() -> dict[str, Any]`	Lê `flows/examples/measured.json` (resultados medidos das execuções dos fluxos de exemplo).
`example_folders`	função	`(root=None) -> list[str]`	Lista as pastas de `examples/*` que são exemplos (sem rascunhos, `_out`, `_runs` e pastas de resultados).
`flows`	função	`() -> list[dict[str, Any]]`	Lista todos os fluxos de exemplo com nome, caminho, título, domínio, nós e se têm configuração rápida.
`flow_path`	função	`(name) -> Path`	Caminho do JSON de um fluxo de exemplo pelo nome; `KeyError` se não existir.
`with_quick`	função	`(doc) -> dict[str, Any]`	Devolve o fluxo com a configuração rápida (`meta.quick`) aplicada.
`run_example`	função	`(name, *, quick=False, workdir=None, record=True, on_update=None) -> dict[str, Any]`	Roda um fluxo de exemplo (completo ou rápido) e, com `record`, grava resultado e comparações em `measured.json`.
`register_example_run`	função	`(result, flow) -> str`	Gancho de fim de execução que grava a execução de um fluxo de exemplo como manifesto de experimento e o importa no Lab do Flow Studio.
`register_hook`	função	`() -> None`	Adiciona `register_example_run` a `RUN_FINISHED_HOOKS` (uma vez só).
`scan`	função	`(example) -> dict[str, Any]`	Por script de um exemplo, lista as chamadas de biblioteca que ele importa e quais um nó declara.
`status`	função	`(example, inv=None, meas=None) -> dict[str, Any]`	Linha do inventário de um exemplo: fluxos, estado, razão e comparação medida.
`problems`	função	`() -> list[str]`	Portão de CI: pastas de exemplo sem fluxo nem razão, estados inválidos e fluxos que citam exemplo inexistente.

nome	tipo	assinatura	o que faz
`markdown`	função	`() -> str`	Gera o texto de `docs/flow_studio/EXAMPLES_AS_FLOWS.md`.
`planner_rows`	função	`(rows) -> list[str]`	Tabela do planejador: por exemplo com fluxo, se o planejador heurístico propõe a capacidade em primeiro lugar a partir do título.
`main`	função	`(argv) -> int`	CLI `python -m pinneapple_flow example-flows` (list, run, doc, check).

pinneapple_flow.identify

Classifica sistemas de equações candidatos para uma descrição (palavras-chave, unidades, valores explícitos e, opcionalmente, um LLM validado) e ajusta uma EDP a dados em grade por regressão esparsa (STRidge/SINDy).

`pinneapple_flow/identify.py` · 256 linhas · testes: 7

nome	tipo	assinatura	o que faz
`identify`	função	`(text, *, dimension=None, units=None, n=3, llm=None, call=None) -> dict[str, Any]`	Devolve sistemas de equações candidatos ordenados para uma descrição, com confiança, raciocínio e decisões; um LLM opcional acrescenta um candidato validado.
`discover_pde`	função	`(axes, values, coords, *, field='u', threshold=0.05, trim=3) -> dict[str, Any]`	Ajusta `u_t` (e `u_tt`) a termos candidatos por regressão esparsa sobre dados em grade e devolve a equação, coeficientes, R^2 e família.
`confirm`	função	`(candidate, found) -> dict[str, Any]`	Diz se a equação descoberta concorda com um candidato (mesma família e, quando possível, coeficientes).
`to_json`	função	`(res) -> str`	Serializa o resultado em JSON convertendo arrays numpy em listas.

pinneapple_flow.lab_status

Consulta `pinneapple_lab.curation` e `pinneapple_lab.seal` (detectados em tempo de execução) para mostrar a letra do selo e os próximos passos; o Flow Studio nunca dá nota sozinho.

`pinneapple_flow/lab_status.py` · 211 linhas · testes: 9

nome	tipo	assinatura	o que faz
`seal_status`	função	`(item, lab=None) -> dict[str, Any]`	Letra do selo de um item do Lab com sua razão; `None` quando o item nunca foi curado.
`nodes_for`	função	`(action='', command='', params=None, target='') -> list[dict[str, Any]]`	Tipos de nó registrados que executam uma ação do Lab (pelo tipo ou pela chamada de biblioteca que declaram), curados primeiro.
`next_steps`	função	`(item, lab=None) -> dict[str, Any]`	O que falta ao item para a próxima letra, com a ação do Lab e os nós que a executam.
`lab_items`	função	`(reports) -> list[str]`	Itens do Lab gravados pelos nós `validate.lab` de uma execução.
`insert_nodes`	função	`(flow, anchor, items) -> tuple[dict[str, Any], list[str]]`	Insere nós num fluxo e liga cada entrada obrigatória a partir de um nó âncora ou de nós compatíveis; devolve o fluxo e as entradas não ligadas.

pinneapple_flow.learn

Monta surrogates torch a partir das arquiteturas da biblioteca num formato único de dataset (`A`, `Y`, `grid`), normaliza, treina com Adam e mede erros em partições separadas. [sem teste]

`pinneapple_flow/learn.py` · 366 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`default_config`	função	`(kind, in_dim, out_dim) -> dict[str, Any]`	Arquitetura sugerida para um tipo de surrogate e os tamanhos dos dados.
`blocks_net`	função	`(in_dim, out_dim, blocks)`	Monta um `torch.nn.Sequential` a partir de blocos editáveis (`dense`, `residual`, `dropout`, `layernorm`) com camada linear final.

nome	tipo	assinatura	o que faz
build	função	(cfg, in_dim, out_dim, *, n_fields=1, grid=None)	Constrói o surrogate torch de uma configuração, mapeando (B, in_dim) para (B, out_dim) em unidades normalizadas.
count_parameters	função	(model) -> int	Conta os parâmetros treináveis de um modelo.
layer_graph	função	(model) -> dict[str, Any]	Grafo das camadas folha com contagem de parâmetros, agrupadas por bloco (ramo e tronco da DeepONet em grupos separados).
architecture	função	(cfg, in_dim, out_dim, *, n_fields=1, grid=None) -> dict[str, Any]	Prévia de uma arquitetura sem treinar: grafo de camadas, número de parâmetros e a configuração usada.
split	função	(n, val=0.15, test=0.15, seed=0)	Divide índices em treino, validação e teste de forma aleatória com semente.
rel_l2_rows	função	(pred, true) -> float	Média por amostra de ||pred - true|| / ||true||.
fit	função	(cfg, data, *, epochs=500, lr=0.002, batch=32, seed=0, n_fields=1, splits=None, callback=…)	Treina o surrogate com Adam e agenda cosseno, guardando os pesos de menor erro de validação; devolve modelo e métricas por partição.
save	função	(model, cfg, path, *, n_fields=1, meta=None) -> None	Salva configuração, tamanhos e pesos do surrogate com torch.save.
load	função	(obj)	Reconstrói um surrogate a partir do que save gravou.
search_space	função	(spec) -> dict[str, tuple]	Converte uma especificação simples de espaço de busca em parâmetros de AdaptiveSweepConfig.
apply_trial	função	(cfg, trial) -> tuple[dict[str, Any], dict[str, Any]]	Separa os valores de uma tentativa em mudanças de arquitetura e opções de treino.
tune	função	(cfg, data, space, *, n_trials=12, epochs=200, seed=0, n_fields=1, log=print)	Busca de hiperparâmetros (Optuna TPE se instalado, senão busca aleatória) minimizando o erro de validação em partições fixas.

pinneapple_flow.libcalls

Mantém CALLS, a lista de chamadas permitidas; gera o esquema de parâmetros a partir da assinatura e só executa entradas da lista, inclusive para nós custom.* criados pelo usuário.

pinneapple_flow/libcalls.py · 409 linhas · testes: 10

nome	tipo	assinatura	o que faz
LibCall	classe	(key, target, title, category, description, refs, inputs, exclude, overrides, init, requi…)	Entrada da lista permitida: alvo módulo:atributo, título, categoria, referências, entradas vindas de portas, parâmetros excluídos ou sobrescritos e tipo de saída.
resolve	função	(target)	Devolve o objeto chamável de um alvo da lista permitida.
introspect	função	(call) -> tuple[list[Param], list[str]]	Lê os parâmetros de tipo simples de uma chamada e os nomes que não puderam ser expostos.
summarize	função	(ctx, result, depth=0) -> dict[str, Any]	Transforma o que uma chamada devolveu em entradas de relatório e um resultado JSON.
spec_for	função	(call, *, type_=None, title=None, fixed=None, defaults=None, hidden=()) -> NodeSpec	Monta o NodeSpec de uma LibCall, com valores fixos, padrões e parâmetros ocultos opcionais.
wrapped	função	(call) -> list[str]	Para um wrapper de pinneapple_flow.wrappers, lê no código-fonte os módulos da biblioteca que ele importa.
custom_spec	função	(doc) -> NodeSpec	Cria um tipo de nó custom.* do usuário sobre uma entrada de CALLS; ValueError se citar outra coisa ou parâmetro inexistente.
library_specs	função	() -> list[NodeSpec]	Os NodeSpec de todas as entradas de CALLS.
auto_specs	função	() -> list[NodeSpec]	Os NodeSpec dos nós gerados em autocalls.json; os módulos só são importados quando o nó roda.
add	função	(*calls) -> None	Acrescenta LibCalls a CALLS, recusando chaves duplicadas.

pinneapple_flow.libcalls_catalog

Registra com `add(LibCall(...))` as chamadas curadas da biblioteca expostas como nós (solvers, comparação, validação etc.), com categoria na paleta, entradas por porta e referências; os formulários de parâmetros vêm das assinaturas. [sem teste direto pelo inventário]

`pinneapple_flow/libcalls_catalog.py` · 119 linhas · **sem teste direto**

pinneapple_flow.nodes

Importa os módulos de nós embutidos, registra os nós de chamada de biblioteca (`library_specs` e `auto_specs`) e depois os nós de exemplos de `nodes_ext`.

`pinneapple_flow/nodes/__init__.py` · 29 linhas · testes: 17

pinneapple_flow.nodes._common

Peças comuns dos nós embutidos: documentos de problema, artefatos de solução em grade, soluções exatas, interpolação e gráficos. [sem teste direto]

`pinneapple_flow/nodes/_common.py` · 186 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`coords_of`	função	`(geometry) -> list[str]`	Lista de coordenadas de uma geometria.
`bounds_of`	função	`(problem) -> dict[str, tuple[float, float]]`	Limites por eixo de um problema, incluindo `t` quando há tempo.
`system_of`	função	`(problem, *, coefficients=None)`	O `EquationSystem` analisado de um documento de problema.
`need_problem`	função	`(v) -> dict[str, Any]`	Garante que o valor é um problema com geometria e equações, senão falha o passo.
`grid_points`	função	`(axes, coords) -> np.ndarray`	Pontos de uma grade produto dos eixos, como matriz `(N, d)`.
`save_solution`	função	`(ctx, name, axes, values, coords, fields, method, label='', extra=None) -> dict[str, Any]`	Grava uma solução em grade como artefato npz com eixos, campos e método.
`load_solution`	função	`(v) -> dict[str, Any]`	Lê um artefato de solução em grade (eixos, valores, coordenadas, campos).
`exact_function`	função	`(problem)`	Função da solução exata do problema, ou `None`.
`interpolate`	função	`(sol, X) -> np.ndarray`	Interpola linearmente uma solução em grade em pontos dados.
`eval_axes`	função	`(problem, n=None, n_t=None) -> dict[str, np.ndarray]`	Grade de avaliação de um problema: pontos por eixo dados, a malha anexada ou um padrão por dimensão.
`plot_solution`	função	`(sol, title='', field=0)`	Figura de uma solução: curvas no tempo (1-D), mapa (2-D) ou cortes no plano médio (3-D).

pinneapple_flow.nodes.analysis

Nós de validação `validate.compare` (erros contra referência e checagens de física do trust card) e `validate.lab` (curadoria do PINNeAPPle Lab com letra A-F e selo na figura). [sem teste direto]

`pinneapple_flow/nodes/analysis.py` · 306 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`compare`	função	`(ctx, inputs, params)`	Nó `validate.compare`: mede cada candidato contra a referência nos mesmos pontos (L2 relativo, erro máximo e RMS) e as checagens de física.
`lab`	função	`(ctx, inputs, params)`	Nó `validate.lab`: grava a validação como execução do Lab, dá a letra A-F e carimba o selo na figura só nos níveis A e B.

pinneapple_flow.nodes.autolab

Nós do Autonomous Physics Lab (pinneapple_autolab): resfriamento de placa, gêmeo de tubulação que sabe quando erra e o aerodinamicista com OpenFOAM como referência. [sem teste direto]

pinneapple_flow/nodes/autolab.py · 143 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
board_cooling	função	(ctx, inputs, params)	Nó autolab.board_cooling: acha o dissipador mais leve e a velocidade do ventilador de uma placa dentro dos limites, com verificação em malhas finas e relatório de confiança.
pipe_twin	função	(ctx, inputs, params)	Nó autolab.pipe_twin: gêmeo de tubulação que distingue mudança de processo, sensor errado e modelo fora do domínio, medido em casos controlados.
aerodynamicist	função	(ctx, inputs, params)	Nó autolab.aerodynamicist: reduz o arrasto de um carro de corrida paramétrico mantendo downforce e balanço, com OpenFOAM como referência e surrogates comparados.

pinneapple_flow.nodes.cfd_pipeline

Nós do pipeline FreeCAD -> OpenFOAM (pinneapple_simulation.pipelines.freecad_openfoam): estudo, geometria, execução, varredura e surrogate da varredura; precisam do OpenFOAM instalado. [sem teste direto]

pinneapple_flow/nodes/cfd_pipeline.py · 172 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
study	função	(ctx, inputs, params)	Nó cfd.study: modelo FreeCAD paramétrico com regras de patches, malha, física, seções e referências.
geometry	função	(ctx, inputs, params)	Nó cfd.geometry: ajusta a planilha, recalcula, exporta e nomeia os patches, sem solver.
run	função	(ctx, inputs, params)	Nó cfd.run: CAD, malha, solução, pós-processamento e checagens de um caso.
sweep	função	(ctx, inputs, params)	Nó cfd.sweep: varredura de parâmetros retomável, um processo por caso.
surrogate	função	(ctx, inputs, params)	Nó cfd.surrogate: todos os surrogates aplicáveis com o mesmo orçamento, erro medido em casos separados.

pinneapple_flow.nodes.condense

Nó de condensação estática hierárquica de um encapsulamento de chip (pinneapple_condense). [sem teste direto]

pinneapple_flow/nodes/condense.py · 64 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
package_thermal	função	(ctx, inputs, params)	Nó condense.package_thermal: resolve um encapsulamento multicamada sintético condensando blocos repetidos e relata temperatura de junção e balanço de energia.

pinneapple_flow.nodes.data

Nós de dados: observações sintéticas de uma solução, leitura de CSV, conjuntos paramétricos e o benchmark Burgers com condições iniciais de campo aleatório gaussiano. [sem teste direto]

pinneapple_flow/nodes/data.py · 194 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
observe	função	(ctx, inputs, params)	Nó data.observe: observações pontuais de uma solução (sensores sintéticos) com ruído, guardando os coeficientes verdadeiros.
ingest	função	(ctx, inputs, params)	Nó data.ingest: lê medições de CSV colado ou de arquivo da pasta de dados.
parametric	função	(ctx, inputs, params)	Nó data.parametric: conjunto de treino para surrogates com um solve por diferenças finitas por amostra.

nome	tipo	assinatura	o que faz
grf_burgers	função	(ctx, inputs, params)	Nó data.grf_burgers: condições iniciais de campo aleatório gaussiano mapeadas para a solução de Burgers viscosa (solver pseudo-espectral).
dataset_of	função	(v)	Lê um dataset de operador (A, Y, grid) de um artefato, senão falha o passo.

pinneapple_flow.nodes.methods

Nós de métodos que envolvem implementações da biblioteca: convergência de malha, filtro de Kalman, propagação de incerteza, ROMs, MeshGraphNet, transferência, meta-aprendizado, arena de PINNs, NSGA-II e controle. [sem teste direto]

pinneapple_flow/nodes/methods.py · 657 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
convergence	função	(ctx, inputs, params)	Nó verify.convergence: três malhas aninhadas, ordem observada, extrapolação de Richardson e GCI.
kalman	função	(ctx, inputs, params)	Nó assimilate.filter: estima estados de canais ruidosos com os filtros de Kalman da biblioteca.
propagate	função	(ctx, inputs, params)	Nó uq.propagate: propaga incerteza uniforme nas entradas do surrogate por amostragem ou caos polinomial.
rom_fit	função	(ctx, inputs, params)	Nó rom.fit: ajusta ROMs aos snapshots e compara os erros de previsão com o mesmo orçamento.
meshgraphnet	função	(ctx, inputs, params)	Nó gnn.meshgraphnet: treina uma MeshGraphNet num campo de difusão em malhas aleatórias e compara com o estado inicial congelado.
transfer	função	(ctx, inputs, params)	Nó transfer.pinn: fine-tuning de uma PINN treinada no problema alvo contra treino do zero com as mesmas épocas.
meta_learn	função	(ctx, inputs, params)	Nó meta.learn: meta-aprende uma inicialização de PINN numa família de problemas e compara a adaptação com inicialização aleatória.
arena	função	(ctx, inputs, params)	Nó benchmark.arena: mesmos problemas, orçamento e épocas para todos os modelos, com L2 relativo e posição por problema.
nsga2	função	(ctx, inputs, params)	Nó optimize.nsga2: busca um problema de projeto com NSGA-II (ou um baseline) e relata a frente de Pareto.
control	função	(ctx, inputs, params)	Nó control.compare: compara controladores em malha fechada no tanque aquecido da biblioteca (só simulação, sem comando a equipamento).

pinneapple_flow.nodes.physics_learning

Nó que verifica se um modelo térmico aprendido conhece a física com um verificador independente da perda de treino (pinneapple_analysis.validation.physics_learning). [sem teste direto]

pinneapple_flow/nodes/physics_learning.py · 62 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
physics_learning	função	(ctx, inputs, params)	Nó analysis.physics_learning: checa resíduo, fluxo, contorno, balanço de energia, simetria, escala e intervenções causais contra referência de volumes finitos.

pinneapple_flow.nodes.presets

Nós de presets industriais (pinneapple_presets): rodar um caso pronto com parâmetros próprios e validá-lo contra os casos de referência.

pinneapple_flow/nodes/presets.py · 60 linhas · testes: 27

nome	tipo	assinatura	o que faz
run_preset	função	(ctx, inputs, params)	Nó preset.run: resolve um caso industrial pronto com as entradas do usuário.
validate_preset	função	(ctx, inputs, params)	Nó preset.validate: roda os casos de referência do preset e mede o erro.

pinneapple_flow.nodes.problem

Nós de problema: domínio (1-D, 2-D, 3-D), malha, sistema de equações em texto validado por parser seguro e o problema genérico que identifica as equações. [sem teste direto]

pinneapple_flow/nodes/problem.py · 227 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
geometry_doc	função	(params) -> dict[str, Any]	Monta o documento de geometria (dimensão, limites, tempo) a partir dos parâmetros, checando os intervalos.
domain	função	(ctx, inputs, params)	Nó geometry.domain: dimensão, limites por eixo e intervalo de tempo.
mesh	função	(ctx, inputs, params)	Nó geometry.mesh: grade estruturada com nós por eixo e níveis de tempo salvos.
equations	função	(ctx, inputs, params)	Nó equations.system: EDPs, condições e coeficientes em texto, analisados por parser seguro e validados.
require_problem	função	(v)	Atalho para need_problem.
generic	função	(ctx, inputs, params)	Nó problem.generic: identifica as equações de qualquer problema descrito, com confiança e referências, e as descobre a partir de dados quando houver.

pinneapple_flow.nodes.query

Nó de consulta física de um encapsulamento de chip, respondida pelo método válido mais barato com incerteza e procedência (pinneapple_query). [sem teste direto]

pinneapple_flow/nodes/query.py · 46 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
package_queries	função	(ctx, inputs, params)	Nó query.package: responde cada consulta pelo método mais barato que atinge a meta e diz qual, com incerteza.

pinneapple_flow.nodes.reason

Nós de raciocínio de engenharia (pinneapple_reason): requisito, conceito, modo de falha, verificação, contramedida, cadeia de porquês e relatório.

pinneapple_flow/nodes/reason.py · 183 linhas · testes: 17

nome	tipo	assinatura	o que faz
requirement	função	(ctx, inputs, params)	Nó reason.requirement: o que o projeto deve fazer e quais modos de falha o impediriam.
concept	função	(ctx, inputs, params)	Nó reason.concept: uma versão do projeto com parâmetros, elementos e origem de cada valor.
failure_mode	função	(ctx, inputs, params)	Nó reason.failure_mode: um modo de falha no conceito com demanda, capacidade e mecanismo.
verify	função	(ctx, inputs, params)	Nó reason.verify: todos os modos de falha de todos os requisitos e o primeiro que falha.
countermeasure	função	(ctx, inputs, params)	Nó reason.countermeasure: tenta as contramedidas da primeira falha e aplica a escolhida.
chain	função	(ctx, inputs, params)	Nó reason.chain: ciclo checar-corrigir a partir do conceito ingênuo até todos os requisitos passarem.
report	função	(ctx, inputs, params)	Nó reason.report: a cadeia como relatório com linha do tempo, margens, figuras e relatório de confiança.

pinneapple_flow.nodes.science

Nós que rodam um experimento de pinneapple_science contra a referência citada e desenham o atlas das áreas cobertas.

pinneapple_flow/nodes/science.py · 65 linhas · testes: 19

nome	tipo	assinatura	o que faz
`run_experiment`	função	`(ctx, inputs, params)`	Nó `science.run`: roda um experimento científico, cada grandeza contra sua referência citada.
`atlas`	função	`(ctx, inputs, params)`	Nó `science.atlas`: mapa das áreas científicas disponíveis, parciais e planejadas.

`pinneapple_flow.nodes.solvers`

Nós de solver: clássico (diferenças finitas, elementos finitos), OpenFOAM, PINN, PINN inverso e calibração por mínimos quadrados. [sem teste direto]

`pinneapple_flow/nodes/solvers.py` · 362 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`classical`	função	`(ctx, inputs, params)`	Nó `solver.classical`: resolve o sistema editado com método clássico e relata custo e resíduo.
`openfoam`	função	`(ctx, inputs, params)`	Nó `solver.openfoam`: roda um caso OpenFOAM (blockMesh + solver) e lê a saúde do log; precisa do OpenFOAM no PATH.
`pinn`	função	`(ctx, inputs, params)`	Nó `train.pinn`: treina uma PINN nas equações editadas (Adam e L-BFGS) e a valida em pontos novos.
`discover`	função	`(ctx, inputs, params)`	Nó `inverse.discover`: identifica coeficientes desconhecidos junto com o campo a partir de observações (PINN inversa).
`calibrate`	função	`(ctx, inputs, params)`	Nó `inverse.calibrate`: calibração por mínimos quadrados com o solver de diferenças finitas como modelo direto.

`pinneapple_flow.nodes.surrogates`

Nós de surrogate sobre `pinneapple_flow.learn`: treino com arquitetura editável, HPO, fine-tuning e índices de Sobol. [sem teste direto]

`pinneapple_flow/nodes/surrogates.py` · 274 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`surrogate`	função	`(ctx, inputs, params)`	Nó `surrogate.train`: treina um surrogate num dataset e relata erros de validação e teste.
`hpo`	função	`(ctx, inputs, params)`	Nó `surrogate.hpo`: busca arquitetura e treino na validação e retreina o melhor, sem usar o teste na busca.
`finetune`	função	`(ctx, inputs, params)`	Nó `train.finetune`: continua o treino de um surrogate com dados novos e compara o erro antes e depois.
`sensitivity`	função	`(ctx, inputs, params)`	Nó `uq.sensitivity`: índices de Sobol de uma grandeza do campo em relação às entradas do surrogate.

`pinneapple_flow.nodes.systems`

Nós de sistemas: gêmeo digital do tanque aquecido, previsão do ENSO (baixa dados e precisa de statsmodels), laboratório de previsão de séries e decolagem com a biblioteca de voo. [sem teste direto]

`pinneapple_flow/nodes/systems.py` · 424 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`twin`	função	`(ctx, inputs, params)`	Nó `twin.connect`: alimenta o runtime do gêmeo (Kalman por ensemble no tanque aquecido) com leituras e devolve estimativas, alertas e previsão.
`enso`	função	`(ctx, inputs, params)`	Nó `forecast.enso`: previsão do índice Niño 3.4 com o protocolo da biblioteca de ENSO; baixa as observações uma vez.
`forecast`	função	`(ctx, inputs, params)`	Nó `forecast.lab`: testes de previsibilidade e depois um conjunto de previsores em validação walk-forward com período de teste intocado.
`takeoff`	função	`(ctx, inputs, params)`	Nó `flight.takeoff`: simula uma decolagem com a biblioteca de voo 6-DOF e compara a distância com a forma fechada de Raymer.

pinneapple_flow.nodes.twin_nodes

Gêmeo digital automático: nó de dados ao vivo (sensores simulados, arquivo, MQTT, OPC UA) e nó que monta o gêmeo a partir das equações identificadas. [sem teste direto]

pinneapple_flow/nodes/twin_nodes.py · 279 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
stream	função	(ctx, inputs, params)	Nó data.stream: leituras de sensores simuladas, de arquivo ou ao vivo via MQTT / OPC UA.
twin_auto	função	(ctx, inputs, params)	Nó twin.auto: monta o gêmeo a partir das equações identificadas, com alertas e painel.

pinneapple_flow.nodes.vehicle

Nós do rover planetário de pinneapple_simulation.vehicle: veículo, solo, terreno, travessia multicorpo, modelo de roda Wong-Reece, PINN do model zoo, painel animado e a missão que aprende o solo.

pinneapple_flow/nodes/vehicle.py · 479 linhas · testes: 6

nome	tipo	assinatura	o que faz
vehicle_presets	função	() -> dict[str, Any]	Funções de pinneapple_simulation.vehicle que constroem um VehicleSpec.
build_spec	função	(v)	Constrói o veículo a partir do preset e da massa, com geometria de roda editável.
build_soil	função	(v)	Constrói o solo do banco de terramecânica com parâmetros sobrescritos.
build_site	função	(v)	Constrói o terreno pelo cenário escolhido.
load_run	função	(v) -> dict[str, Any]	Lê a travessia gravada por um nó vehicle.simulate, conferindo o checksum.
virtual_display	função	(size=(1920, 1080))	Display para pyvista: o atual ou um Xvfb iniciado durante o bloco.
vehicle_spec	função	(ctx, inputs, params)	Nó vehicle.spec: rover da biblioteca (NASA VIPER) com massa e roda editáveis.
soil	função	(ctx, inputs, params)	Nó terramechanics.soil: solo do banco de terramecânica (parâmetros de Bekker / Wong-Reece) com sobrescrita.
site	função	(ctx, inputs, params)	Nó scenario.site: terreno deformável com cratera e rocha, ou rampa plana.
simulate	função	(ctx, inputs, params)	Nó vehicle.simulate: dinâmica multicorpo do rover sobre o terreno com rodas Wong-Reece.
wheel_model	função	(ctx, inputs, params)	Nó terramechanics.wheel_model: modelo de roda Wong-Reece e suas curvas de bancada na carga estática.
build_wheel_model	função	(v)	Constrói o WongReeceModel a partir do solo, do veículo e das opções.
zoo_surrogate	função	(ctx, inputs, params)	Nó terramechanics.zoo_surrogate: PINN de contato da roda do model zoo (baixada uma vez, checksum verificado) com seus erros do model card.
dashboard	função	(ctx, inputs, params)	Nó vehicle.dashboard: painel animado da travessia com gráficos sincronizados, em HTML, PNG, MP4/GIF.
learn_planet	função	(ctx, inputs, params)	Nó rover.learn_planet: missão em que o rover atola, aprende o solo pelas rodas (EKF iterado) e replaneja, num terreno sintético de solos conhecidos.
ctx_metrics	função	(nums) -> dict[str, float]	Métricas numéricas planas do resumo de uma missão do rover.

pinneapple_flow.nodes.viz

Nós de visualização e exportação: gráficos, visualizador three.js, render (Blender Cycles se bpy existe, senão matplotlib) e relatório autocontido. [sem teste direto]

pinneapple_flow/nodes/viz.py · 237 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`plot`	função	`(ctx, inputs, params)`	Nó `viz.plot`: uma figura por solução (curvas, mapa de cores ou cortes).
`field3d`	função	`(ctx, inputs, params)`	Nó `viz.field3d`: visualização interativa three.js do campo.
`render`	função	`(ctx, inputs, params)`	Nó `viz.render`: imagem e animação de um campo 2-D com Blender Cycles ou, sem `bpy`, matplotlib.
`report`	função	`(ctx, inputs, params)`	Nó `export.report`: relatório autocontido com métricas, parâmetros e figuras embutidas.

pinneapple_flow.nodes_ext

Importa os módulos de nós por domínio que reproduzem `examples/*`, liga o gancho que grava execuções de exemplo como experimentos do Lab e coloca cada nó na seção da paleta da sua categoria. [sem teste direto]

`pinneapple_flow/nodes_ext/__init__.py` · 58 linhas · **sem teste direto**

pinneapple_flow.nodes_ext.aero

Nó de aerodinâmica de avião do exemplo `geometry_library/03_airliner_vlm`: malha de vórtices do `airliner_a320` e arrasto de atrito. [sem teste direto]

`pinneapple_flow/nodes_ext/aero.py` · 71 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`airliner_vlm`	função	`(ctx, inputs, params)`	Nó `aero.airliner_vlm`: inclinação da sustentação, ponto neutro, trimagem, arrasto induzido, CD0 da malha e L/D.

pinneapple_flow.nodes_ext.architectures

Nós dos exemplos `examples/architectures`: listagem do registro de modelos, construção por nome com passagem direta em tensores falsos e compressão POD com rollout DMD. [sem teste direto]

`pinneapple_flow/nodes_ext/architectures.py` · 172 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`registry_listing`	função	`(ctx, inputs, params)`	Nó `architectures.registry`: famílias e modelos do registro e o catálogo por família.
`forward`	função	`(ctx, inputs, params)`	Nó `architectures.forward`: um modelo do registro construído por nome e rodado em tensores falsos (forma de saída, parâmetros, perdas, gradiente).
`pod_dmd`	função	`(ctx, inputs, params)`	Nó `architectures.pod_dmd`: erros de reconstrução POD e de rollout POD + DMD num sistema linear sintético.

pinneapple_flow.nodes_ext.arena_pipelines

Nós de `examples/arena_pipelines`: registro de datasets, especificações de benchmark físico e de séries temporais e a execução multi-modelo da `Arena`. [sem teste direto]

`pinneapple_flow/nodes_ext/arena_pipelines.py` · 205 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`datasets`	função	`(ctx, inputs, params)`	Nó `arena.datasets`: ids de datasets por categoria e os arrays que cada id carrega.
`physics_spec`	função	`(ctx, inputs, params)`	Nó `arena.physics_spec`: todos os modelos da especificação num problema físico, métricas e leaderboard.
`timeseries_spec`	função	`(ctx, inputs, params)`	Nó `arena.timeseries_spec`: todos os modelos de previsão da especificação no mesmo corte lookback/horizonte.
`arena_run`	função	`(ctx, inputs, params)`	Nó `arena.run`: todos os modelos treinados e avaliados nos mesmos pontos, com erros por campo e tempo de treino.

pinneapple_flow.nodes_ext.astro

Nós de acreção em buraco negro (examples/black_hole_weather): simulação de toro RIAF e estatísticas de janela do estudo de resolução. [sem teste direto]

pinneapple_flow/nodes_ext/astro.py · 117 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
simulate	função	(ctx, inputs, params)	Nó blackhole.simulate: evolui o fluxo de acreção a partir de um toro de equilíbrio (potencial de Paczynski-Wiita).
statistics	função	(ctx, inputs, params)	Nó blackhole.statistics: taxa média de acreção, perfis e erro de persistência na janela.

pinneapple_flow.nodes_ext.autolab_caps

Registra capacidades do planejador para fluxos de exemplo cujos nós vivem em outros módulos (presets industriais, experimentos de ciência, missões do laboratório autônomo). [sem teste direto]

pinneapple_flow/nodes_ext/autolab_caps.py · 42 linhas · **sem teste direto**

pinneapple_flow.nodes_ext.benchmarks

Nós de benchmark: o benchmark de mesmo orçamento (pinneapple_analysis.benchmark.run) e a suíte de examples/benchmark_suite (Arena por YAML, transferência, meta-aprendizado). [sem teste direto]

pinneapple_flow/nodes_ext/benchmarks.py · 239 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
equal_budget	função	(ctx, inputs, params)	Nó benchmark.equal_budget: todos os métodos em todos os problemas com o mesmo ComputeBudget, busca e sementes.
arena_yaml	função	(ctx, inputs, params)	Nó benchmark.arena_yaml: preset, dados de referência e todos os modelos do YAML treinados nos mesmos pontos.
transfer	função	(ctx, inputs, params)	Nó benchmark.transfer: pré-treino numa fonte e fine-tuning com cada estratégia de congelamento contra treino do zero.
meta	função	(ctx, inputs, params)	Nó benchmark.meta: meta-treino por família de EDP e adaptação em K passos contra treino do zero.

pinneapple_flow.nodes_ext.bricks

Nós dos tijolos encaixáveis auto-modificáveis (examples/bricks): checagens que controlam o laço e as duas estratégias de busca com o mesmo orçamento. [sem teste direto]

pinneapple_flow/nodes_ext/bricks.py · 83 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
validate	função	(ctx, inputs, params)	Nó bricks.validate: valor simulado de cada checagem, se passou, e a decisão de confiança.
search	função	(ctx, inputs, params)	Nó bricks.search: as duas estratégias em cada semente com o mesmo orçamento e a melhor construção.

pinneapple_flow.nodes_ext.cfd

Nós de CFD com OpenFOAM: a calha de superfície livre com jato (interFoam) de examples/free_surface e a campanha de examples/openfoam_campaign. [sem teste direto]

pinneapple_flow/nodes_ext/cfd.py · 210 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
trough_jet	função	(ctx, inputs, params)	Nó cfd.trough_jet: um caso de inclinação da calha com jato (k médio perto do fundo, pico, profundidade, volume).
ratio	função	(ctx, inputs, params)	Nó cfd.ratio: razão de uma métrica de cada caso pela do caso base.

nome	tipo	assinatura	o que faz
campaign	função	(ctx, inputs, params)	Nó cfd.campaign: caso base e cada execução do plano com sua saúde e perfis amostrados.

pinneapple_flow.nodes_ext.core_ts

Nós de geofísica (água rasa na esfera, 4D-Var em Lorenz-96, Richards), do PhysicsOptimizer, do híbrido FEM + correção neural e de séries temporais. [sem teste direto]

pinneapple_flow/nodes_ext/core_ts.py · 270 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
earth_system	função	(ctx, inputs, params)	Nó geo.earth_system: onda de Rossby-Haurwitz, 4D-Var em Lorenz-96 e infiltração de Richards com balanço de massa.
torsion_optimizer	função	(ctx, inputs, params)	Nó core.torsion_optimizer: maximiza a rigidez torcional de um retângulo com vários otimizadores.
hybrid_fem	função	(ctx, inputs, params)	Nó core.hybrid_fem: FEM grosseira com corretor Conv1d treinado contra FEM fina.
regime_series	função	(n=900, seed=0) -> np.ndarray	Série do exemplo de previsão adaptativa: sazonal, depois tendência, depois passeio aleatório.
adaptive	função	(ctx, inputs, params)	Nó ts.adaptive: especialistas ponderados pelos erros recentes, MAE do conjunto, cobertura do intervalo e trocas de líder.
naive_baselines	função	(ctx, inputs, params)	Nó ts.naive_baselines: baselines ingênuo, sazonal ingênuo e drift numa série sintética.

pinneapple_flow.nodes_ext.design

Nós de projeto: otimização topológica (pinneapple_design.topology), checagem de gradiente, rotor de relutância, projeto a partir de intenção e prévia qualitativa do Ahmed body. [sem teste direto]

pinneapple_flow/nodes_ext/design.py · 321 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
topology2d	função	(ctx, inputs, params)	Nó design.topology2d: otimização topológica 2-D (viga MBB, balanço, térmica, Stokes).
topology_gradcheck	função	(ctx, inputs, params)	Nó design.topology_gradcheck: sensibilidades adjuntas contra diferenças centrais.
reluctance_rotor	função	(ctx, inputs, params)	Nó design.reluctance_rotor: núcleo de rotor de 4 polos por otimização topológica, reavaliado com ferro não linear.
from_intent	função	(ctx, inputs, params)	Nó design.from_intent: de um objetivo em linguagem natural a um projeto verificado, perguntas ou recusa.
ahmed_slant	função	(ctx, inputs, params)	Nó qualitative.ahmed_slant: variação de arrasto prevista por inclinação contra a medida, com baixa confiança perto da separação.

pinneapple_flow.nodes_ext.electromagnetics

Nós dos estudos de examples/motor_magnetics e examples/electrodynamics: tabela de verificação, máquina de ímãs na superfície e o motor IPM do Prius 2004 (torque, mapa de fluxo, acionamento, skew, tolerâncias). [sem teste direto]

pinneapple_flow/nodes_ext/electromagnetics.py · 512 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
em_verification	função	(ctx, inputs, params)	Nó em.verification: cada solver eletromagnético contra resultado independente, com erros e ordens observadas.
spm_machine	função	(ctx, inputs, params)	Nó em.spm_machine: máquina de ímãs na superfície com torque por tensor de Maxwell e por trabalho virtual.
prius_machine	função	(ctx, inputs, params)	Nó em.prius_machine: motor do Prius 2004, torque contra ângulo de corrente comparado com Yang et al. (IEMDC 2013).
prius_flux_map	função	(ctx, inputs, params)	Nó em.prius_flux_map: psi_d, psi_q e torque numa grade i_d x i_q, com perdas.

nome	tipo	assinatura	o que faz
prius_drive	função	(ctx, inputs, params)	Nó em.prius_drive: envelope MTPA + enfraquecimento de campo, mapa de eficiência e energia no ciclo WLTC.
prius_skew	função	(ctx, inputs, params)	Nó em.prius_skew: cogging, torque e ripple contra o skew em degraus e harmônicos da FEM contra o fator de skew.
prius_tolerances	função	(ctx, inputs, params)	Nó em.prius_tolerances: dispersão de torque e ripple pelas tolerâncias dos ímãs e do entreferro.

pinneapple_flow.nodes_ext.em_antenna

Nó da zona quieta de um compact range de antenas por óptica física, com validação e varredura de projeto. [sem teste direto]

pinneapple_flow/nodes_ext/em_antenna.py · 67 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
compact_range	função	(ctx, inputs, params)	Nó antenna.compact_range: zona quieta do projeto de referência, validação e varredura de projeto.

pinneapple_flow.nodes_ext.em_prius

Nós dos estudos longos do motor do Prius: projeto de rotor (NSGA-II com surrogate GP), rede térmica em ciclos WLTC, verificação cruzada com gmsh + GetDP e efeitos de extremidade; os dois últimos precisam de gmsh e GetDP. [sem teste direto]

pinneapple_flow/nodes_ext/em_prius.py · 315 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
rotor_design	função	(ctx, inputs, params)	Nó em.prius_rotor_design: busca híbrida com surrogate contra só FEA (hipervolume e chamadas de FEA).
prius_thermal	função	(ctx, inputs, params)	Nó em.prius_thermal: rede térmica concentrada com perdas WLTC e margens de desmagnetização por grau de NdFeB.
getdp_crosscheck	função	(ctx, inputs, params)	Nó em.prius_getdp_crosscheck: FEA do PINNeAPPle contra um modelo independente gmsh + GetDP da mesma máquina.
end_effects	função	(ctx, inputs, params)	Nó em.prius_end_effects: o estudo de efeitos de extremidade do exemplo 16 (precisa de gmsh e GetDP).

pinneapple_flow.nodes_ext.examples

O nó examples.compare, que confere números do fluxo contra os resultados versionados do exemplo com tolerância atol + rtol \|ref\|, o nó examples.flatten e ajudantes comuns dos nós de exemplo.

pinneapple_flow/nodes_ext/examples.py · 309 linhas · testes: 8

nome	tipo	assinatura	o que faz
Written	classe	(text)	Número de referência que guarda o texto como foi escrito (preserva as casas decimais para a tolerância de arredondamento).
save_object	função	(ctx, name, obj) -> dict[str, Any]	Grava um objeto Python como artefato pickle do nó.
load_object	função	(v) -> Any	Lê o objeto de um artefato de save_object (só artefatos escritos por nós desta execução).
examples_root	função	() -> Path | None	Pasta examples do checkout ($PINNEAPPLE_REPO/examples ou ao lado dos pacotes); None numa wheel instalada.
example_dir	função	(name) -> Path	Pasta de um exemplo pelo nome, falhando se não existir ou se a pasta examples não estiver presente.
doc_table_value	função	(ref) -> float	Primeiro número de uma célula de tabela Markdown em docs/, indicada por doc:<caminho>|<linha>|<coluna>.
reference_value	função	(folder, ref) -> float	O número de referência que uma expressão ref aponta numa pasta de exemplo (arquivo de resultados, manifesto, nó ou tabela de doc).
rounding_half_unit	função	(x) -> float	Meia unidade do último dígito de um número como o Python o escreve; zero com 12 ou mais dígitos significativos.

nome	tipo	assinatura	o que faz
`metric_value`	função	`(reports, name) -> float`	Valor de `node.metric` nos relatórios dos nós anteriores.
`compare`	função	`(ctx, inputs, params)`	Nó `examples.compare`: compara cada número medido pelo fluxo com o versionado no exemplo, dentro da tolerância.
`flatten`	função	`(ctx, inputs, params)`	Nó `examples.flatten`: transforma cada número de um resultado aninhado numa métrica para `examples.compare` checar.

`pinneapple_flow.nodes_ext.flight`

Nós de dinâmica de voo (`examples/flight_dynamics`) sobre `pinneapple_simulation.flight`: aeronaves, verificação, trimagem, decolagem, turbulência, rajada e cenas 3-D. [sem teste direto]

`pinneapple_flow/nodes_ext/flight.py` · 588 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`aircraft`	função	`(ctx, inputs, params)`	Nó `flight.aircraft`: modelos de aeronave (asa e cauda em malha de vórtices, fuselagem, trem, motores, massas com fontes).
`verification`	função	`(ctx, inputs, params)`	Nó `flight.verification`: as checagens do script de verificação do exemplo.
`trim_stability`	função	`(ctx, inputs, params)`	Nó `flight.trim_stability`: trimagem em cruzeiro, derivadas adimensionais e autovalores da linearização de 12 estados.
`takeoff_cases`	função	`(ctx, inputs, params)`	Nó `flight.takeoff_cases`: corrida, rotação, decolagem e subida, com forma fechada de Raymer, vento cruzado e rajada.
`turbulence_encounter`	função	`(ctx, inputs, params)`	Nó `flight.turbulence_encounter`: cada aeronave na mesma turbulência, com fatores de carga e acelerações na cabine.
`pratt`	função	`(ctx, inputs, params)`	Nó `flight.pratt`: pico de fator de carga numa rajada 1-cosseno contra a fórmula de Pratt-Walker.
`same_gust`	função	`(ctx, inputs, params)`	Nó `flight.same_gust`: mesma rajada para todas as aeronaves, com picos no CG e na cabine e as estimativas quase-estática e de Pratt.
`scene`	função	`(ctx, inputs, params)`	Nó `flight.scene`: cenas 3-D animadas dos voos como visualizadores three.js e entrada dos nós de render Blender.

`pinneapple_flow.nodes_ext.football`

Nós de futebol (`examples/sports`): problema inverso da trajetória da bola, suavização de rastreamento (dados simulados) e pitch control numa partida de amostra da Metrica (baixada uma vez). [sem teste direto]

`pinneapple_flow/nodes_ext/football.py` · 76 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`football_ball_flight_inverse`	função	`(ctx, inputs, params)`	Nó `football.ball_flight_inverse`: chutes simulados com ruído ajustados pela EDO de arrasto-Magnus, com cobertura dos intervalos e comparação de Cramér-Rao 3-D contra 2-D.
`football_tracking_smoothing`	função	`(ctx, inputs, params)`	Nó `football.tracking_smoothing`: erro de velocidade e potência metabólica por diferenças, Savitzky-Golay e suavizador Kalman-RTS.
`football_pass_pitch_control`	função	`(ctx, inputs, params)`	Nó `football.pass_pitch_control`: pitch control contra regressão logística e árvores em passes de partidas de amostra da Metrica.

`pinneapple_flow.nodes_ext.forecasting`

Nós de previsão: checagens do solver de ENSO (sem dados) e estudos de `forecast_lab` (caso, sobreajuste, Lyapunov de Mackey-Glass); alguns casos precisam de download e terminam `not_run` sem ele. [sem teste direto]

`pinneapple_flow/nodes_ext/forecasting.py` · 201 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`enso_solver_checks`	função	`(ctx, inputs, params)`	Nó `enso.solver_checks`: velocidades de ondas, ordem de convergência e resposta de Gill do modelo acoplado, e osciladores contra seus autovalores.
`forecast_case`	função	`(ctx, inputs, params)`	Nó `forecast_lab.case`: previsibilidade e todos os modelos num caso, com walk-forward purgado e teste intocado.
`lyapunov`	função	`(ctx, inputs, params)`	Nó `forecast_lab.lyapunov`: estimativa de Benettin do maior expoente de Lyapunov de Mackey-Glass.
`overfitting`	função	`(ctx, inputs, params)`	Nó `forecast_lab.overfitting`: pontuações de vários esquemas de validação em variantes sobreajustadas e regularizadas contra o teste intocado.

`pinneapple_flow.nodes_ext.geomechanics`

Nós de colapso de coluna granular (MPM contra a lei de Lube et al. 2005), PINN de inspeção por correntes parasitas e biblioteca de componentes. [sem teste direto]

`pinneapple_flow/nodes_ext/geomechanics.py` · 169 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`lube_2005`	função	`(a) -> float`	Lei de espalhamento do colapso 2-D de Lube et al. (2005) usada como referência.
`column_collapse`	função	`(ctx, inputs, params)`	Nó `geo.column_collapse`: colapso de coluna de areia seca por MPM, espalhamento final contra a lei de Lube.
`eddy_current`	função	`(ctx, inputs, params)`	Nó `inspection.eddy_current`: varredura sintética de sonda por FDM e uma `InspectionPINN` treinada nela.
`component_fit`	função	`(ctx, inputs, params)`	Nó `systems.component_fit`: componentes e toolboxes registrados e um componente ajustado a alvos aleatórios com semente (demonstração da API, não da física).

`pinneapple_flow.nodes_ext.lbm`

Nós de Lattice Boltzmann (`examples/lbm`): arrasto de esfera no túnel 3-D contra a lei de Stokes e contra Schiller & Naumann. [sem teste direto]

`pinneapple_flow/nodes_ext/lbm.py` · 153 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`sphere_stokes`	função	`(ctx, inputs, params)`	Nó `lbm.sphere_stokes`: esfera num cubo com campo de Stokes nas faces; arrasto por troca de momento contra 6 pi rho nu R U.
`sphere_drag`	função	`(ctx, inputs, params)`	Nó `lbm.sphere_drag`: Cd da esfera no túnel contra Schiller & Naumann (1933).

`pinneapple_flow.nodes_ext.learned_sim`

Nó de MeshGraphNet treinada num prefixo dos dados `cylinder_flow` da DeepMind, com rollout na validação. [sem teste direto]

`pinneapple_flow/nodes_ext/learned_sim.py` · 112 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`mgn_cylinder_flow`	função	`(ctx, inputs, params)`	Nó `gnn.mgn_cylinder_flow`: velocidade no passo seguinte do desprendimento de vórtices, em rollout, contra a velocidade congelada.

`pinneapple_flow.nodes_ext.non_hermitian`

Nós de física não-hermitiana (`examples/non_hermitian`): teorema no-go de sintonia de pontos excepcionais em dímeros PT-simétricos, checado simbólica e numericamente. [sem teste direto]

`pinneapple_flow/nodes_ext/non_hermitian.py` · 177 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ep_laws`	função	`(ctx, inputs, params)`	Nó `nonhermitian.ep_laws`: lei do ponto excepcional por classe de acoplamento e se o teorema se aplica.

nome	tipo	assinatura	o que faz
ep_numeric	função	(ctx, inputs, params)	Nó nonhermitian.ep_numeric: gamma_EP² obtido do hamiltoniano contra o alvo linear e ponto fixo com realimentação.
hypothesis_map	função	(ctx, inputs, params)	Nó nonhermitian.hypothesis_map: três projetos auditados contra as hipóteses do teorema.
ep_figure	função	(ctx, inputs, params)	Nó nonhermitian.ep_figure: figura de quatro painéis e limiar de quebra PT em cadeias uniformes.

pinneapple_flow.nodes_ext.pde_env

Nó do exemplo pde_environment: preset com seletor por etiqueta treinado via solve_pde, termos de perda por etiqueta antes e depois. [sem teste direto]

pinneapple_flow/nodes_ext/pde_env.py · 76 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
tagged_preset	função	(ctx, inputs, params)	Nó pde_env.tagged_preset: termos de perda por etiqueta de um preset antes e depois do treino na geometria real.

pinneapple_flow.nodes_ext.pinn_examples

Nós dos exemplos de chamada única: PINN de Burgers contra Cole-Hopf com pp.solve e FEM contra PINN em Poisson com pp.compare. [sem teste direto]

pinneapple_flow/nodes_ext/pinn_examples.py · 99 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
burgers_first	função	(ctx, inputs, params)	Nó pinn.burgers_first: perfil da PINN de Burgers na frente íngreme contra a solução de Cole-Hopf.
poisson_fem_vs_pinn	função	(ctx, inputs, params)	Nó pinn.poisson_fem_vs_pinn: FEM e PINN numa chamada contra a solução exata de Poisson.

pinneapple_flow.nodes_ext.pinn_solver

Nó que avalia a perda simbólica de PINNFactory num campo nulo (examples/pinn_solver). [sem teste direto]

pinneapple_flow/nodes_ext/pinn_solver.py · 57 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
factory_loss	função	(ctx, inputs, params)	Nó pinn_solver.factory_loss: perda de PINNFactory compilada pelo SymPy e avaliada no campo nulo.

pinneapple_flow.nodes_ext.problemdesign

Nó de projeto de problema sem LLM (examples/problem_designer): validadores, plano FNO-first e relatório de um ProblemSpec parcial. [sem teste direto]

pinneapple_flow/nodes_ext/problemdesign.py · 84 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
offline_report	função	(ctx, inputs, params)	Nó problemdesign.offline_report: validadores, plano e relatório de uma especificação parcial.

pinneapple_flow.nodes_ext.render

Nós de render de cenas animadas com Blender Cycles e grade de clipes com barra de cor; o bpy roda em processo separado e o vídeo precisa de ffmpeg. [sem teste direto]

pinneapple_flow/nodes_ext/render.py · 161 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`blender_clip`	função	`(ctx, inputs, params)`	Nó `render.blender_clip`: clipe Cycles de cada cena com HUD, em MP4, mais uma imagem.
`clip_grid`	função	`(ctx, inputs, params)`	Nó `render.clip_grid`: clipes lado a lado com rótulos e uma barra de cor.

`pinneapple_flow.nodes_ext.render_worker`

Script que roda o Blender em processo próprio para `nodes_ext.render`, porque importar `bpy` pode derrubar o processo e o Cycles prende o interpretador por minutos. [sem teste direto]

`pinneapple_flow/nodes_ext/render_worker.py` · 74 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`check`	função	`() -> int`	Confere se o `bpy` importa e funciona.
`render`	função	`(scenes_pkl, options_json, out_dir) -> int`	Renderiza as cenas de um pickle com as opções JSON na pasta de saída.
`main`	função	`(argv) -> int`	Entrada de linha de comando do worker (`--check` ou render).

`pinneapple_flow.nodes_ext.security`

Nó de segurança ciberfísica: resíduos físicos contra injeção de dados falsos na rede de três tanques, comparados com detectores baseados em dados. [sem teste direto]

`pinneapple_flow/nodes_ext/security.py` · 70 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`fdi_benchmark`	função	`(ctx, inputs, params)`	Nó `security.fdi_benchmark`: AUC, taxa de detecção, atraso e localização por detector, regime e ataque; a isolation forest precisa de scikit-learn.

`pinneapple_flow.nodes_ext.solver_registry`

Nós sobre o `SolverRegistry` (`examples/solvers`): decomposições de séries temporais, caixa SPH e Poisson por FDM e espectral. [sem teste direto]

`pinneapple_flow/nodes_ext/solver_registry.py` · 211 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`decompose`	função	`(ctx, inputs, params)`	Nó `solvers.decompose`: soma de senos com ruído decomposta por cada solver registrado.
`sph_box`	função	`(ctx, inputs, params)`	Nó `solvers.sph_box`: trajetória SPH, caixa final e fração de partículas dentro, com aviso se saírem.
`poisson_fdm_spectral`	função	`(ctx, inputs, params)`	Nó `solvers.poisson_fdm_spectral`: Poisson por SOR e por FFT, a diferença entre eles e cada um contra a solução exata.

`pinneapple_flow.nodes_ext.solvers`

Nós de `examples/numerical_solvers` e `examples/rigid_body`: gás de Lennard-Jones, ruptura de barragem em água rasa e caixa com atrito (precisa do NVIDIA Newton). [sem teste direto]

`pinneapple_flow/nodes_ext/solvers.py` · 172 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`lj_gas`	função	`(ctx, inputs, params)`	Nó `md.lj_gas`: gás de Lennard-Jones com Verlet e energia total contra a inicial.
`dam_break`	função	`(ctx, inputs, params)`	Nó `swe.dam_break`: ruptura de barragem em água rasa com relatório de saúde (volume, positividade, finitude).
`crate_and_pendulum`	função	`(ctx, inputs, params)`	Nó `newton.crate_and_pendulum`: distância de parada da caixa com atrito de Coulomb contra $v0^2/(2 mu g)$; precisa do NVIDIA Newton.

pinneapple_flow.nodes_ext.structures

Nós do CalculiX (examples/calculix_cantilever): deck, execução do ccx (nativo ou Docker) e leitura do .frd com comparação com Euler-Bernoulli. [sem teste direto]

pinneapple_flow/nodes_ext/structures.py · 146 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
deck	função	(ctx, inputs, params)	Nó calculix.deck: deck de entrada da viga em balanço.
solve	função	(ctx, inputs, params)	Nó calculix.solve: roda o ccx no deck e falha o passo se o retorno não for zero.
results	função	(ctx, inputs, params)	Nó calculix.results: deflexão na ponta, maior |sigma_xx| e deflexão de Euler-Bernoulli P L³/(3 E I).

pinneapple_flow.nodes_ext.surrogate_inverse

Nó de projeto inverso guiado por surrogate (pinneapple_design.surrogate_inverse, as ferramentas de apps/*_ai) com verificação na forma fechada. [sem teste direto]

pinneapple_flow/nodes_ext/surrogate_inverse.py · 59 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
surrogate_inverse	função	(ctx, inputs, params)	Nó design.surrogate_inverse: treina o surrogate do caso, otimiza o projeto por ele e verifica na forma fechada.

pinneapple_flow.nodes_ext.use_cases

Nó do problema inverso da aleta (examples/use_cases/fin_convection_inverse): coeficiente h por PINN e por mínimos quadrados. [sem teste direto]

pinneapple_flow/nodes_ext/use_cases.py · 137 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
pin_fin	função	(ctx, inputs, params)	Nó inverse.pin_fin: h e o campo da aleta aprendidos juntos por PINN contra ajuste de mínimos quadrados da forma fechada.

pinneapple_flow.nodes_ext.validation

Nós de validação: linter físico pinneapple_check, avaliação do model zoo, grafo de prova PT, arena de validação red/blue team e auditoria do solver de referência. [sem teste direto]

pinneapple_flow/nodes_ext/validation.py · 213 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
physics_check	função	(ctx, inputs, params)	Nó check.physics: roda todas as regras do linter físico nos alvos e compara as regras quebradas com a lista do exemplo.
zoo_evaluate	função	(ctx, inputs, params)	Nó zoo.evaluate: reproduz as métricas do model card, checagens físicas, comparação com XFOIL e fine-tune protegido.
pt_chain	função	(ctx, inputs, params)	Nó proof.pt_chain: grafo de hipóteses, lemas e teorema com checagens exatas e numéricas para tetrâmeros PT.
arena	função	(ctx, inputs, params)	Nó validate.arena: red team procura saídas erradas que passam e blue team ajusta as checagens até pegar todas.
audit	função	(ctx, inputs, params)	Nó validate.audit_model: o solver de referência julgado pela defesa entregue (sem escapes nem falsos alarmes).

pinneapple_flow.nodes_ext.validation_gallery

Nós da galeria de validação (examples/validation): um modelo bom e uma variante quebrada que uma checagem nomeada de pp.validate deve pegar. [sem teste direto]

pinneapple_flow/nodes_ext/validation_gallery.py · 197 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
pinn_heat	função	(ctx, inputs, params)	Nó gallery.pinn_heat: PINN de calor e variante com peso de contorno zero, que pinn.boundary deve pegar.
pod_dmd_wake	função	(ctx, inputs, params)	Nó gallery.pod_dmd_wake: POD + DMD bom e quebrado na esteira, que rollout.invariants deve pegar.
opinf_advection	função	(ctx, inputs, params)	Nó gallery.opinf_advection: ROMs de advecção-difusão com base completa e cortada, que rollout.spectrum deve pegar.
zoo_range	função	(ctx, inputs, params)	Nó gallery.zoo_range: modelo do zoo dentro e fora da faixa de validade, que applicability.range deve pegar.

pinneapple_flow.nodes_ext.viz

Nó "qualquer resultado em 3-D": lê arquivos CAE ou geometria, exporta glTF/USD, gera visualizador three.js e, com bpy, renderiza. [sem teste direto]

pinneapple_flow/nodes_ext/viz.py · 92 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
any_result_3d	função	(ctx, inputs, params)	Nó viz.any_result_3d: carrega, exporta, visualiza e renderiza qualquer resultado em 3-D.

pinneapple_flow.nodes_ext.worldmodel

Nó do modelo de mundo físico (pinneapple_worldmodel): datasets dos cenários, um especialista FNO por cenário e as tarefas de benchmark. [sem teste direto]

pinneapple_flow/nodes_ext/worldmodel.py · 69 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
specialists	função	(ctx, inputs, params)	Nó worldmodel.specialists: por cenário, parâmetros do especialista e erros de passo e rollout em cada tarefa.

pinneapple_flow.palette

Descreve a paleta do Flow Studio: seções, título e resumo de cada nó, dica de instalação e busca que coloca nós curados acima dos auto.*.

pinneapple_flow/palette.py · 299 linhas · testes: 29

nome	tipo	assinatura	o que faz
place	função	(type_, section) -> None	Coloca um tipo de nó registrado por outro módulo numa seção da paleta.
sections	função	() -> list[dict[str, Any]]	As seções da paleta em ordem, com título, ícone e descrição.
is_auto	função	(spec) -> bool	Diz se o tipo de nó é gerado (auto.*).
section_of	função	(spec) -> str	Seção de um tipo de nó: pela etiqueta section:, pelo mapa curado ou pela categoria.
package_of	função	(spec) -> str	Pacote da biblioteca que um nó envolve, pela primeira chamada declarada.
humanize	função	(name) -> str	Converte um nome em snake_case num título legível, mantendo siglas em maiúsculas.
summary	função	(text, limit=110) -> str	Primeira frase de uma docstring, sem marcação reST, cortada num limite de caracteres.
install_hint	função	(reason) -> str	O que fazer com um nó indisponível, a partir da razão.
entry	função	(spec, avail=None) -> dict[str, Any]	Campos de paleta de um tipo de nó (seção, título, resumo, etiquetas, pacote, curado, dica de instalação).
score	função	(e, query) -> float	Relevância de uma entrada do catálogo para uma busca; título pesa mais e nós curados ficam acima dos gerados.
search	função	(query, catalog=None, limit=50) -> list[dict[str, Any]]	Entradas do catálogo que casam com a busca, melhores primeiro.

nome	tipo	assinatura	o que faz
`layout`	função	`(catalog=None) -> list[dict[str, Any]]`	A paleta inteira como seções em ordem com seus tipos de nó.

pinneapple_flow.planner

Ponto de entrada `plan`: escolhe o planejador heurístico, o LLM ou `auto` (LLM se houver provedor configurado) e volta para a heurística quando o LLM falha, registrando o motivo.

`pinneapple_flow/planner/__init__.py` · 57 linhas · testes: 16

nome	tipo	assinatura	o que faz
`plan`	função	`(text, *, backend='auto', provider='', model='', base_url='', api_key='', n=3, call=None)…)`	Gera propostas de fluxo para uma descrição com o planejador escolhido, caindo para a heurística quando o LLM falha.

pinneapple_flow.planner.build

`FlowBuilder` para montar fluxos em código (nós, arestas, decisões) e `layout`, que posiciona os nós da esquerda para a direita.

`pinneapple_flow/planner/build.py` · 66 linhas · testes: 5

nome	tipo	assinatura	o que faz
`FlowBuilder`	classe	`(name, description='')`	Construtor de fluxos em código: adiciona nós, liga portas, registra decisões e devolve o fluxo ou uma proposta.
`layout`	função	`(nodes, edges, dx=270.0, dy=150.0)`	Posiciona os nós em colunas pelo caminho mais longo desde uma fonte e em linhas pela ordem de aparição.

pinneapple_flow.planner.extensions

Gancho público de extensão do planejador: outros pacotes declaram `Capability` (palavras-chave, nós, modelo de fluxo) e o planejador heurístico as propõe para descrições que casam, com uma decisão por escolha.

`pinneapple_flow/planner/extensions.py` · 168 linhas · testes: 5

nome	tipo	assinatura	o que faz
`Capability`	classe	`(id, title, domain, keywords, nodes, description, inputs, downstream, references, uncited…)`	Capacidade declarada para o planejador: id, título, domínio, palavras-chave com pesos, nós, referências, modelo de fluxo e pontuação mínima.
`register_capability`	função	`(cap) -> Capability`	Adiciona ou substitui (mesmo id) uma capacidade e sua regra de decisão.
`capabilities`	função	`() -> list[Capability]`	Todas as capacidades registradas, em ordem de id.
`score`	função	`(cap, text) -> tuple[float, list[str]]`	Pontuação de uma capacidade para um texto (soma dos pesos das palavras-chave encontradas), zero se faltar um grupo obrigatório.
`match`	função	`(text) -> list[tuple[float, Capability, list[str]]]`	Capacidades cuja pontuação atinge `min_score`, melhores primeiro.
`template_doc`	função	`(name, quick=False) -> dict[str, Any]`	Lê o fluxo de exemplo usado como modelo, opcionalmente na configuração rápida.
`proposal`	função	`(cap, text, det, hits, s) -> dict[str, Any]`	Monta a proposta de fluxo de uma capacidade com as decisões que explicam a escolha.
`proposals`	função	`(text, det=None, *, limit=2) -> list[dict[str, Any]]`	Fluxos das capacidades que melhor casam com o texto (no máximo `limit`).
`describe`	função	`() -> list[dict[str, Any]]`	Todas as capacidades em JSON, para o assistente da interface e o prompt do planejador LLM.

pinneapple_flow.planner.heuristic

Planejador offline e determinístico: palavras-chave em inglês e português, grandezas com unidades e valores de coeficientes detectam a física e o pedido, e cada proposta é um fluxo do catálogo.

pinneapple_flow/planner/heuristic.py · 573 linhas · testes: 11

nome	tipo	assinatura	o que faz
detect	função	(text) -> dict[str, Any]	Tudo o que as heurísticas leem no texto (família de EDP, dimensão, regime, condições, pedidos), mostrado ao usuário.
family_length_ok	função	(family) -> bool	Diz se a família aceita o comprimento do domínio dado (falso para burgers).
plan_heuristic	função	(text, *, max_proposals=3) -> dict[str, Any]	Plano heurístico completo: capacidades registradas primeiro, depois fluxos montados conforme a física e os pedidos detectados.
sensor_count	função	(text) -> int | None	Lê no texto o número de sensores (1 a 64), em inglês ou português, ou None.
sensor_layout	função	(n, dim) -> list	Posições de n sensores espalhadas no intervalo ou quadrado unitário.

pinneapple_flow.planner.llm

Planejador por LLM via pinneapple_llm.call_llm (Ollama, endpoint compatível com OpenAI, OpenAI ou Anthropic): catálogo e esquema vão no prompt, a resposta JSON é validada e reparada. [sem teste direto; os testes usam plan com call falso]

pinneapple_flow/planner/llm.py · 228 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
catalog_prompt	função	() -> str	Texto do catálogo de nós (portas, parâmetros e descrição) para o prompt.
build_prompt	função	(text, n=2) -> str	Prompt completo com o catálogo, o esquema e um exemplo de resposta JSON.
repair	função	(flow_doc) -> tuple[dict[str, Any], list[str]]	Reconstrói um fluxo válido a partir da saída do LLM e lista os reparos feitos.
plan_llm	função	(text, *, provider, model='', base_url='', api_key='', n=2, call=None) -> dict[str, Any]	Propostas de um LLM, cada uma validada e reparada; levanta erro em falha do provedor ou saída inútil.

pinneapple_flow.planner.wizard

Assistente guiado de "New flow" (problema, física, geometria/dados, métodos, validação, saídas) que transforma respostas numa descrição para o planejador heurístico.

pinneapple_flow/planner/wizard.py · 89 linhas · testes: 5

nome	tipo	assinatura	o que faz
describe	função	(answers) -> str	A descrição em texto que o planejador lê para as respostas do assistente.
plan_from_answers	função	(answers, *, max_proposals=3) -> dict[str, Any]	Resultado do planejador para as respostas do assistente, com a capacidade escolhida em primeiro lugar.

pinneapple_flow.references

Carrega o registro curado references.yaml, aceita referências de outros pacotes, verifica DOIs online (doi.org e Crossref) e formata citações.

pinneapple_flow/references.py · 216 linhas · testes: 13

nome	tipo	assinatura	o que faz
registry	função	() -> dict[str, dict[str, Any]]	Lê references.yaml mais as referências registradas por outros pacotes.
register_references	função	(entries) -> list[str]	Acrescenta referências curadas (dict ou YAML) checando o formato; recusa chave existente com conteúdo diferente.

nome	tipo	assinatura	o que faz
`ref`	função	`(key) -> dict[str, Any]`	Uma referência do registro pela chave; `KeyError` se não existir.
`link`	função	`(r) -> str`	Link clicável de uma referência: DOI, senão URL, senão vazio.
`problems`	função	`(entry) -> list[str]`	Lista o que está errado numa entrada do registro (vazia se bem formada).
`verify_doi`	função	`(doi, title='') -> dict[str, Any]`	Resolve um DOI online (doi.org e Crossref) e devolve `verified`, `not found` ou `unverified` (sem rede).
`verify_registry`	função	`() -> dict[str, dict[str, Any]]`	Verifica online todas as entradas do registro que têm DOI.
`check`	função	`(reference, *, online=None) -> dict[str, Any]`	Classifica uma referência citada como `registry`, `verified`, `not found` ou `unverified`.
`cite`	função	`(*keys, supports='') -> list[dict[str, Any]]`	Referências do registro para as chaves dadas.
`format_reference`	função	`(r) -> str`	Formata uma referência como texto (autores, ano, título, veículo, link, estado).

pinneapple_flow.registry

Define `Port`, `Param`, `NodeSpec` e `Registry` e o decorador `node`, que registra `fn(ctx, inputs, params)` como tipo de nó.

`pinneapple_flow/registry.py` · 256 linhas · testes: 46

nome	tipo	assinatura	o que faz
`Port`	classe	`(name, type, required, multiple, help)`	Porta de entrada ou saída de um nó: nome, tipo, se é obrigatória ou múltipla e ajuda.
`Param`	classe	`(name, kind, default, help, choices, minimum, maximum, max_length, advanced, label)`	Parâmetro de um nó com tipo, padrão, escolhas e limites; gera seu JSON Schema e checa valores.
`NodeSpec`	classe	`(type, title, category, fn, inputs, outputs, params, description, calls, requires, availa…)`	Especificação de um tipo de nó: função, portas, parâmetros, descrição, chamadas de biblioteca, requisitos e disponibilidade.
`Registry`	classe	`()`	Catálogo de tipos de nó: registrar, registrar nós do usuário, buscar e listar o catálogo.
`palette_entry`	função	`(spec, avail) -> dict[str, Any]`	Atalho para `palette.entry` de um `NodeSpec`.
`node`	função	`(type_, title, *, category, inputs=(), outputs=(), params=(), description='', calls='', r…)`	Decorador que registra `fn(ctx, inputs, params) -> {porta: valor}` como tipo de nó.

pinneapple_flow.rules

`RULES`: regras heurísticas (equações, condições de contorno, dimensão, método, arquitetura, hiperparâmetros, validação, gêmeo digital) que citam chaves do registro de referências ou dizem por que não citam, com evidências no repositório; `tests/test_flow_decisions.py` confere citações e caminhos.

`pinneapple_flow/rules.py` · 208 linhas · testes: 13

nome	tipo	assinatura	o que faz
`Rule`	classe	`(id, description, cites, uncited, evidence, topic)`	Regra heurística: id, descrição, citações ou motivo sem citação, evidências e tópico.
`register_rule`	função	`(r) -> Rule`	Registra uma regra de outro pacote, exigindo referências registradas ou um motivo e recusando id repetido diferente.
`rule`	função	`(rule_id) -> Rule`	Busca uma regra pelo id.

pinneapple_flow.runtime

Converte cada nó num `Step` do motor de workflows (ordem, logs, retentativas, cancelamento e reaproveitamento de saídas inalteradas); valores grandes viram artefatos em arquivo e o fim de cada execução passa por `on_run_finished`.

pinneapple_flow/runtime.py · 523 linhas · testes: 9

nome	tipo	assinatura	o que faz
NotRun	classe	(reason)	Exceção levantada por um nó que não pode rodar aqui: ele termina not_run e os nós que dependem dele também.
NodeContext	classe	(node_id, node_type, workdir, log, should_stop=lambda: False, meta=None)	O que a função de um nó recebe além de entradas e parâmetros: pasta, log e ajudantes para métricas, tabelas, figuras, decisões e artefatos.
FlowNodeAdapter	classe	()	Adaptador do motor que roda um nó no processo: resolve entradas a partir das saídas anteriores e chama o tipo registrado.
FlowRunResult	classe	(run_id, status, nodes, workdir, seconds, hooks)	Resultado de uma execução: id, estado, nós, pasta, segundos e o que cada gancho fez; dá acesso a saídas e relatórios.
default_workdir	função	() -> Path	Pasta de trabalho padrão ($PINNEAPPLE_FLOW_HOME/work ou ~/.pinneapple_flow/work).
artifact	função	(path, kind) -> dict[str, Any]	Cria o valor {"$artifact": caminho, "kind": tipo} que referencia um arquivo.
is_artifact	função	(v) -> bool	Diz se um valor é uma referência de artefato.
load_artifact	função	(v)	Lê o conteúdo de um artefato: npz como dict de arrays, json como objeto, pt como objeto torch, png como caminho.
to_workflow	função	(flow, *, nodes=None, force=frozenset(), token='', max_parallel=1) -> Workflow	Converte o fluxo no Workflow do motor; nós em force recebem um token novo para rodar de novo.
node_states	função	(runner, run_id, *, with_logs=False) -> dict[str, dict[str, Any]]	Estado, erro e saídas de cada nó de uma execução, marcando not_run e degraded.
register_custom_nodes	função	(docs) -> list[str]	Registra tipos de nó do usuário validados por custom_spec.
run	função	(flow, *, workdir=None, from_node=None, only=None, run_id=None, cache=True, max_parallel=…)	Roda um fluxo sem interface e devolve um FlowRunResult, com cache, re-execução a partir de um nó e paralelismo opcional.
on_run_finished	função	(result, flow) -> None	Ponto único por onde passa toda execução terminada; chama a exportação para o Lab e os ganchos registrados sem nunca falhar a execução.
lab_export_status	função	() -> str	Diz se pinneapple_lab.manifest.from_flow_run está disponível.
export_to_lab	função	(result, flow) -> str	Grava o manifesto de experimento da execução (experiment.json) com o from_flow_run do Lab.

pinneapple_flow.schema

O documento de fluxo pinneapple-flow/1 com Flow, Node, Edge e Issue; validate_flow acusa tipos e parâmetros desconhecidos, portas incompatíveis, entradas faltando e ciclos.

pinneapple_flow/schema.py · 307 linhas · testes: 27

nome	tipo	assinatura	o que faz
FlowError	classe	(message, issues=None)	Exceção para documento de fluxo malformado ou inválido, com a lista de problemas.
Node	classe	(id, type, params, position, label)	Um nó do fluxo: id, tipo, parâmetros, posição e rótulo.
Edge	classe	(source, source_port, target, target_port, id)	Uma aresta do fluxo: nó e porta de origem, nó e porta de destino.
Flow	classe	(name, nodes, edges, id, description, meta, schema)	O documento de fluxo com nós, arestas e metadados, com ordem topológica, ancestrais e leitura/gravação em JSON.
Issue	classe	(level, message, node, edge, param)	Um problema de validação com nível, mensagem e o nó, aresta ou parâmetro envolvido.
as_flow	função	(flow) -> Flow	Obtém um Flow a partir de um Flow, dict, string JSON ou caminho de arquivo.
validate_flow	função	(flow, registry=None) -> list[Issue]	Lista todos os problemas de um fluxo; lista vazia significa válido.
check_flow	função	(flow, registry=None) -> Flow	Devolve o fluxo ou levanta FlowError com os erros (avisos passam).

nome	tipo	assinatura	o que faz
`flow_json_schema`	função	`() -> dict[str, Any]`	JSON Schema (draft 2020-12) do documento de fluxo.

`pinneapple_flow.service`

`FlowService` reúne catálogo, paleta e preferências, plano, validação, prévia de equações e arquitetura, decisões, referências, selo do Lab, nós customizados, fluxos versionados, execuções em segundo plano com estado ao vivo e acesso seguro a artefatos.

`pinneapple_flow/service.py` · 352 linhas · testes: 29

nome	tipo	assinatura	o que faz
`FlowService`	classe	`(*, db_url=None, workdir=None, settings=None, max_runs=1)`	Operações do Flow Studio como métodos: catálogo, plano, validação, armazenamento, execuções e artefatos.

`pinneapple_flow.store`

`FlowStore` em SQLAlchemy (SQLite com WAL por padrão, Postgres opcional) com fluxos versionados, preferências, nós customizados, decisões e execuções; artefatos ficam em disco e o store guarda os caminhos.

`pinneapple_flow/store.py` · 261 linhas · testes: 23

nome	tipo	assinatura	o que faz
`FlowStore`	classe	`(url=None, *, table_prefix=None)`	Armazenamento de fluxos, versões, execuções, decisões e saídas por nó num banco SQLAlchemy, com prefixo de tabela opcional.
`default_url`	função	`() -> str`	URL padrão do banco: `PINNEAPPLE_FLOW_DB_URL` ou o SQLite na pasta home.
`redact`	função	`(url) -> str`	A URL com a senha substituída, para logs e a interface.

`pinneapple_flow.studio`

Serve o app `apps/flow_studio` com uvicorn; recusa ouvir fora de localhost sem `FLOW_USER` e hash de senha, salvo `allow_insecure`.

`pinneapple_flow/studio.py` · 64 linhas · testes: 38

nome	tipo	assinatura	o que faz
`app_dir`	função	`() -> Path`	Pasta do app Flow Studio (`$PINNEAPPLE_FLOW_APP` ou `apps/flow_studio` do checkout).
`load_app`	função	`()`	Carrega o app FastAPI de `apps/flow_studio/backend/app.py` (precisa do checkout do repositório).
`serve`	função	`(host=None, port=None, *, open_browser=True, env_file='.env', allow_insecure=False) -> int`	Sobe o Flow Studio com uvicorn e abre o navegador; recusa endereço não local sem login.

`pinneapple_flow.templates`

Modelos de equações em texto (gramática de `symbolic_pde.text`) para calor, onda, Poisson, Laplace, Helmholtz, Burgers, advecção-difusão e reação-difusão em 1 a 3 dimensões, e troca de dimensão. [sem teste direto pelo inventário]

`pinneapple_flow/templates.py` · 114 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`template`	função	`(family, dimension=1) -> dict[str, Any]`	PDEs, condições, coeficientes, domínio e solução exata de uma família numa dimensão.
`retarget`	função	`(equations, dimension) -> dict[str, Any]`	Leva as equações de um modelo para outra dimensão (troca 2D ↔ 3D).

pinneapple_flow.twin

PDETwinModel transforma o sistema identificado num modelo de gêmeo (campo em grade grossa, coeficientes como estados de passeio aleatório, um estado por sensor); simulate_sensors e run_twin simulam e assimilam leituras. Só primeira ordem no tempo.

pinneapple_flow/twin.py · 281 linhas · testes: 7

nome	tipo	assinatura	o que faz
PDETwinModel	classe	(system, bounds, *, n=21, dt=0.01, estimate=None, coefficients=None)	O sistema identificado como modelo de gêmeo: estados do campo em grade grossa, coeficientes estimados e sensores interpolados.
simulate_sensors	função	(plant, *, sensors, t_end, every=1, noise=0.0, coefficients=None, fault=None, seed=0) -> …)	Leituras simuladas de um ativo (em geral uma grade mais fina), com ruído gaussiano e falha opcional num instante.
run_twin	função	(model, readings, *, sensors, sigma_obs, sigma_process=0.0001, sigma_param=None, limits=N…)	Assimila leituras no modelo com o TwinRuntime e devolve o runtime e um resumo (estimativas, alertas, coeficientes calibrados).

pinneapple_flow.types

PORT_TYPES (geometry, mesh, equations, problem, solution, model, dataset, twin, vehicle, chain e outros), CONVERSIONS entre tipos diferentes e registro de novos tipos por outros pacotes.

pinneapple_flow/types.py · 71 linhas · testes: 7

nome	tipo	assinatura	o que faz
compatible	função	(source, target) -> bool	Diz se uma saída de um tipo pode alimentar uma entrada de outro.
register_port_type	função	(name, description, *, converts_to=(), converts_from=()) -> None	Adiciona um tipo de porta com as conversões permitidas, para nós de outros pacotes.

pinneapple_flow.wrappers

Funções com argumentos JSON que montam os objetos com os construtores da própria biblioteca e chamam a biblioteca; nada aqui calcula física. [sem teste direto]

pinneapple_flow/wrappers.py · 292 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
solve_preset	função	(preset='burgers_1d', method='analytic', architecture='modified_mlp', epochs=50, hidden_d…)	pp.solve num preset da biblioteca; para pinn, a arquitetura é qualquer variante PINN do registro de modelos.
compare_preset	função	(preset='burgers_1d', methods=('analytic', 'pinn'), reference='analytic', architecture='m…)	pp.compare de métodos num preset contra uma referência nos mesmos pontos.
lbm_channel	função	(nx=64, ny=32, Re=100.0, u_in=0.05, steps=200, save_every=50) -> dict	Escoamento em canal por Lattice Boltzmann via SolverRegistry.build('lbm').
beam_deflection	função	(L_m=1.0, E_Pa=200000000000.0, I_m4=8.33e-06, q_N_per_m=1000.0, nx=200, bc_type=0) -> dict	Deflexão de viga de Euler-Bernoulli por diferenças finitas (beam_bvp_fdm).
xtfc_pde	função	(pde='burgers_1d', n_basis=60, n_query=40) -> dict	Resolve uma EDP com o solver X-TFC (xtfc_pde).
wheel_on_soil	função	(soil='GRC-1', radius_m=0.25, width_m=0.2, load_N=100.0, slip=0.2, gravity=1.62) -> dict	Modelo roda-solo de Wong-Reece em regime permanente (pinneapple_simulation.terramechanics).
meshless_compare	função	(kind='poisson', exact='sin(pi*x)*sin(pi*y)', domain='square', methods=…, k=4.0, alpha=1.…)	Compara métodos sem malha e com malha num problema (meshless_study), com checagem das expressões pela biblioteca.
rom_compare	função	(Y=None, A=None, methods=(), ranks=(), budget_s=5.0, seed=0) -> dict	Compara modelos de ordem reduzida em snapshots paramétricos (rom_study).

nome	tipo	assinatura	o que faz
`parametric_pod`	função	`(A=None, Y=None, r=16, regressor='rbf', test_fraction=0.2, seed=0) -> dict`	Ajusta `ParametricPOD` a um dataset e mede o L2 relativo em amostras separadas.
`surrogate_study`	função	`(X=None, Y=None, example='none', methods=('ridge', 'gp', 'gbt'), budget=10.0, seeds=1) ->…)`	Compara surrogates clássicos (ridge, GP, árvores, rede) com `pinneapple_analysis.surrogate.run_study`.
`calculix_cantilever`	função	`(length_m=1.0, b_m=0.1, h_m=0.1, nx=20, element_type='C3D20R', run=True) -> dict`	Escreve o deck de uma viga em balanço com a biblioteca e roda o CalculiX se `ccx` (ou sua imagem Docker) existir; senão só escreve o deck.
`fmu_simulate`	função	`(file='', stop_time=1.0, step_size=0.01) -> dict`	Simula um FMU da pasta de dados (precisa de fmpy).
`newton_validate`	função	`(solver='xpbd', cases=…) -> dict`	Valida solvers de corpo rígido contra formas fechadas (precisa do NVIDIA Newton).
`openfoam_available`	função	`() -> dict`	Diz se o OpenFOAM está instalado (`blockMesh` no PATH).
`model_architecture`	função	`(name='fno', in_dim=1, out_dim=1, hidden=64) -> dict`	Constrói qualquer modelo do registro com argumentos padrão e tamanhos dados e conta seus parâmetros.
`doe_design`	função	`(parameters=None, n=16, method='lhs', seed=0, out_dir='') -> dict`	Plano de experimentos com `pinneapple_data.doe.design` sobre um espaço de parâmetros, gravado como CSV.
`geometry_export`	função	`(name='caarc_building', fmt='stl', lod='light', out_dir='') -> dict`	Constrói uma geometria paramétrica de `pinneapple_design.geometry_library` e a exporta (STL, OBJ, glTF, USD ou STEP).
`design_intent`	função	`(text=…, seed=0, budget=200, n_screen=500, out_dir='') -> dict`	`pinneapple_design.intent.design`: de um objetivo em linguagem natural a um projeto verificado, às perguntas que faltam ou à recusa.

Exemplos de uso reais do repositório

Rodar um fluxo, reaproveitar o cache e re-executar a partir de um nó — `tests/test_flow_runtime.py`

```python
def test_run_measures_against_exact_then_caches(home):
    f = heat_flow()
    r = pf.run(f, workdir=home / "w")
    assert r.status == "done", {k: v["error"] for k, v in r.nodes.items()}
    rel = r.report("val")["metrics"]["FD_rel_l2"]["value"]
    assert rel < 1e-3
    sol = r.load("fd", "solution")
    assert sol["values"].shape == (41, 11, 1)
    r2 = pf.run(f, workdir=home / "w")
    assert {v["status"] for v in r2.nodes.values()} == {"cached"}
    r3 = pf.run(f, workdir=home / "w", from_node="fd")
    st = {k: v["status"] for k, v in r3.nodes.items()}
    assert st["fd"] == "done" and st["val"] == "done" and st["geo"] == "cached"
```

Toda proposta do planejador heurístico valida contra o catálogo — `tests/test_flow_planner.py`

```python
def test_every_proposal_validates(text):
    res = plan(text, backend="heuristic")
    assert res["planner"] == "heuristic" and res["proposals"]
    for p in res["proposals"]:
        assert [i for i in pf.validate_flow(p["flow"]) if i.level == "error"] == []
        assert p["flow"]["meta"]["planner"] == "heuristic"
```

Nó indisponível termina not_run e propaga para baixo — `tests/test_flow_extensions.py`

```python
def test_unavailable_node_finishes_not_run_and_downstream_too(tmp_path):
    f = pf.Flow("x", nodes=[{"id": "a", "type": "test.missing_tool"}, {"id": "b", "type": "test.consumer"}],
                edges=[E("a", "b")])
    r = pf.run(f, workdir=tmp_path)
    assert r.nodes["a"]["status"] == "not_run" and "footool" in r.report("a")["not_run_reason"]
    assert r.nodes["b"]["status"] == "not_run" and "did not run" in r.report("b")["not_run_reason"]
```

Planejar e executar um gêmeo digital a partir de texto — `tests/test_flow_generic.py`

```
text = ("digital twin of a heated rod (1D) from 4 temperature sensors; detect if its thermal diffusivity "
        "changes")
res = pf.plan(text, backend="heuristic")
p = res["proposals"][0]
types = [n["type"] for n in p["flow"]["nodes"]]
assert types[:3] == ["problem.generic", "data.stream", "twin.auto"]
flow = pf.Flow.from_dict(p["flow"])
...
r = pf.run(flow, workdir=tmp_path)
assert r.status == "done", {k: v["error"] for k, v in r.nodes.items()}
m = r.report("twin")["metrics"]
```

Executar um fluxo de exemplo — `examples/flow_studio/run_examples.py`

```
def run_one(name: str, cache: bool = False) -> dict:
    flow = pf.Flow.load(FLOWS_DIR / f"{name}.json")
    t0 = time.time()
    res = pf.run(flow, workdir=WORK, cache=cache)
```

Uso pela linha de comando — `pinneapple_flow/__init__.py`

```
python -m pinneapple_flow run flow.json        # the same, from the shell
python -m pinneapple_flow studio               # the GUI on http://127.0.0.1:8765
```

Testes e status de validação

- Módulos com pelo menos um teste que os importa: 31 de 89.
- Funções de teste em arquivos que importam algum módulo do pacote: 263.

Limitações conhecidas

- Quase todos os módulos de `nodes` e `nodes_ext` não têm teste que os importe diretamente; são exercitados pelos testes de fluxos de exemplo e de execução, não por testes unitários próprios.
- Vários nós dependem de ferramentas externas e ficam indisponíveis sem elas: OpenFOAM (`solver.openfoam`, `cfd.*`), gmsh + GetDP (`em.prius_getdp_crosscheck`, `em.prius_end_effects`), NVIDIA Newton, Blender `bpy` e ffmpeg, CalculiX `ccx`, fmpy, scikit-learn, statsmodels.
- Alguns nós baixam dados na primeira execução (ENSO, partidas de amostra da Metrica, casos de `forecast_lab`, PINN do model zoo) e terminam `not_run` quando o download não é possível.
- `runtime.on_run_finished` diz que a exportação para o PINNeAPPle Lab só funciona quando `pinneapple_lab.manifest.from_flow_run` existe; senão `lab_export_status` informa que não está disponível e nada é gravado. `lab_status` também detecta a API do Lab em tempo de execução.
- `twin.PDETwinModel` só aceita sistemas de primeira ordem no tempo.
- Os nós de `nodes_ext` e `examples.example_dir` precisam da pasta `examples` de um checkout (ou `PINNEAPPLE_REPO`); numa wheel instalada eles falham. `studio.load_app` também depende do checkout com `apps/flow_studio`.
- `studio.serve` recusa ouvir fora de localhost sem `FLOW_USER` e hash de senha, salvo `allow_insecure` explícito.
- `learn` e `planner.llm` não têm teste direto; o planejador LLM depende de um provedor configurado e volta para a heurística quando falha.
- O Flow Studio e o servidor exigem o extra `pinneapple[flow]`; Postgres exige `psycopg`. Quando o servidor escuta fora de localhost, valem as proteções de `appkit` (usuário/senha, limites, CSRF).
- O planejador por LLM depende de um provedor configurado (`ollama`, `openai`, `openai_compatible`, `anthropic`); sem ele resta o planejador heurístico offline.
- Nós de ingestão só leem arquivos dentro de `PINNEAPPLE_FLOW_DATA` (`safe_data_path`); o acesso a artefatos fica restrito à pasta da execução.
- `templates` cobre só oito famílias de EDP em 1 a 3 dimensões; outras famílias ou dimensões levantam `ValueError`.

- Uma chamada da biblioteca só vira nó se estiver em `libcalls_catalog` ou nos `autocalls` gerados; `coverage` lista o que fica de fora e o motivo.
- `register_rule` recusa regras que citam referências não registradas ou que não têm citação nem motivo.
- Sem teste direto pelo inventário: `config`, `libcalls_catalog` e `templates` (exercitados indiretamente pelos testes de fluxo).

4. `pinneapple_hub`

Objetivo

`pinneapple_hub` reúne tudo o que o PINNeAPPle publica e distribui pronto: modelos treinados e verificados, datasets verificados, cartões de modelo e o índice do Hub de ferramentas. O pacote raiz expõe só o cliente genérico de hub: `push_to_hub` e `from_pretrained` sobre a infraestrutura do Hugging Face Hub (sem hospedagem própria), mais o `ModelCard`, contrato de reprodutibilidade em que `validation_metrics` calculadas são obrigatórias. `verified_card` cria esses cartões só a partir de uma rodada do `PhysicsGuardrail` de `pinneapple_llm`.

O subpacote `zoo` é o que aparece como `pp.zoo` e `pp.datasets`: modelos pré-treinados (aerofólio, dissipadores de calor, contato de roda de rover) com faixa de validade, métricas contra referência guardada, checagens físicas que alimentam um `TrustReport` de `pinneapple_analysis`, fine-tune com guarda no conjunto de teste original e benchmark via `pp.benchmark`. Os arquivos são verificados por SHA-256 contra o `manifest.json`, cacheados e baixados sob demanda do Hugging Face Hub. O zoo também exporta ONNX (com as checagens por consulta dentro do grafo), safetensors, Parquet para o visualizador de datasets do Hub, cartões README, o Space estático de demonstração e o pacote web do Playground.

O subpacote `library` gera o índice do Hub (app `apps/hub`): coleta ferramentas (funções públicas, nós do Flow Studio, receitas, ferramentas MCP), modelos, geometrias, datasets, fluxos e exemplos do repositório, cada item com um snippet executável cuja saída gravada é conferida por teste. Ele se liga a `pinneapple_flow` (nós, fluxos embutidos, links para o Flow Studio, composição de ferramentas próprias) e a `pinneapple_design` (biblioteca de geometrias).

Números do inventário

módulos	linhas	classes públicas	funções públicas	testes em arquivos que importam o pacote	módulos sem teste direto
31	7.260	22	170	183 (18 arquivos)	8

Submódulos

submódulo	o que faz	módulos	classes + funções	testes*
`hub`	`push_to_hub` e `from_pretrained`: envio e reconstrução de modelos no Hugging Face Hub com config e cartão.	1	2	0
`library`	Índice do Hub de ferramentas: coleta de itens, snippets executáveis, execuções gravadas, busca, composição e links com o Flow Studio.	10	64	22
`model_card`	`ModelCard`: contrato de reprodutibilidade com métricas de validação obrigatórias.	1	1	21
`verified_card`	Cartões de modelo cujas métricas vêm só de uma rodada do guardrail físico; recusa os demais.	1	4	6
`zoo`	`pp.zoo` e `pp.datasets`: modelos e datasets verificados, com checagens, exportação e publicação no Hugging Face Hub.	17	121	125

* maior número de testes que importam um módulo do submódulo.

Módulos e API pública

`pinneapple_hub`

Pacote raiz: reexporta `ModelCard`, `push_to_hub` e `from_pretrained`.

pinneapple_hub/__init__.py · 12 linhas · testes: 122

pinneapple_hub.hub

Envio e download de modelos PINNeAPPle no Hugging Face Hub, com o arquivo lateral pinneapple_config.json que permite reconstruir a arquitetura; exige o extra hub (huggingface_hub). [sem teste]

pinneapple_hub/hub.py · 151 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
push_to_hub	função	(model, repo_id, model_card, *, architecture=None, architecture_config=None, private=Fals…)	Envia pesos, config de arquitetura e cartão de modelo de um modelo treinado para um repositório do Hugging Face Hub.
from_pretrained	função	(repo_id, *, revision='main', device='cpu', token=None)	Baixa e reconstrói, via ModelRegistry, um modelo enviado com push_to_hub.

pinneapple_hub.library

Índice do Hub: todos os itens prontos do repositório numa lista gerada (data/web/hub/index.json), com busca, facetas, destaques, execução e verificação dos snippets.

pinneapple_hub/library/__init__.py · 303 linhas · testes: 22

nome	tipo	assinatura	o que faz
collect	função	(only=None) -> list[dict[str, Any]]	Itens de todas as fontes (sources.COLLECTORS), ids únicos, ordenados por tipo e título.
run_file	função	(item_id, runs=RUNS) -> Path	Arquivo que guarda a execução gravada de um item (snippet_runs/<id>.json).
read_runs	função	(path=RUNS) -> dict[str, Any]	Todas as execuções gravadas, por id de item.
write_run	função	(item_id, rec, runs=RUNS) -> Path	Grava a execução de um item no seu arquivo em snippet_runs/.
attach_runs	função	(items, runs) -> None	Liga a cada item a execução gravada do snippet atual (mesmo hash), senão a do exemplo da referência, senão not run.
facets	função	(items) -> dict[str, Any]	Contagens por tipo, subtipo, domínio, categoria e qualidade, para os filtros.
quality_key	função	(it) -> str	Valor de filtro da qualidade de um item: nota do Lab, decisão do zoo ou "not graded".
featured	função	(items, n=8) -> list[str]	Destaques por regra fixa: modelos aprovados do zoo, exemplos com selo A e figuras, geometrias com miniatura, depois fluxos.
build	função	(only=None, runs=None) -> dict[str, Any]	Monta o índice inteiro em memória (sem gravar).
write_index	função	(index, path=INDEX) -> Path	Grava o índice em JSON.
load_index	função	(path=INDEX, *, rebuild=False) -> dict[str, Any]	Lê o índice; o padrão é um cache reconstruído das fontes quando falta ou com rebuild=True.
get	função	(index, item_id) -> dict[str, Any]	Devolve um item do índice pelo id.
score	função	(it, query) -> float	Relevância de um item para uma consulta (0 = sem match); todas as palavras devem casar, com peso maior no título.
search	função	(index, query='', *, kind=None, domain=None, quality=None, subkind=None, category=None, l…)	Itens que casam com a consulta e os filtros (tipo, domínio, qualidade, subtipo, categoria), ordenados por relevância.
record_runs	função	(items, *, stale_only=False, timeout=300.0, ids=None, workers=4, log=print) -> dict[str, …)	Roda os snippets (todos, só os desatualizados ou ids escolhidos) em paralelo no sandbox e grava as saídas.
check	função	(index) -> list[str]	Problemas de um índice: snippet sem execução aprovada do código atual, arquivo ou link inexistente, entre outros.

pinneapple_hub.library.__main__

Linha de comando `python -m pinneapple_hub.library build | run | check` (e `flowruns`). [sem teste]

`pinneapple_hub/library/__main__.py` · 47 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`main`	função	`(argv=None) -> int`	Ponto de entrada da linha de comando do índice do Hub.

pinneapple_hub.library.categories

As categorias de tarefa das ferramentas do Hub: cada ferramenta recebe exatamente uma, pela regra de prefixo mais longa sobre a chave de origem; receitas são listadas uma a uma.

`pinneapple_hub/library/categories.py` · 174 linhas · testes: 14

nome	tipo	assinatura	o que faz
`key`	função	`(it) -> str`	Chave de origem de uma ferramenta, no formato que as regras de prefixo leem (`node:`, `lib:`, `fn:`, `mcp:`).
`category_of`	função	`(it) -> str | None`	Slug da categoria de uma ferramenta, ou `None` se nenhuma regra casa (um teste proíbe isso).
`table`	função	`() -> list[dict[str, str]]`	Tabela das categorias (slug, título e descrição).

pinneapple_hub.library.compose

"Crie a sua": compõe uma ferramenta a partir de blocos existentes (tipos de nó do Flow Studio, fluxo inicial opcional, geometria) e devolve o documento de fluxo e o Python que o roda.

`pinneapple_hub/library/compose.py` · 191 linhas · testes: 22

nome	tipo	assinatura	o que faz
`ComposeError`	classe	`()`	Erro de composição (`ValueError`): especificação inválida ou blocos demais.
`blocks`	função	`() -> list[dict[str, Any]]`	Blocos que o compositor oferece: todos os tipos de nó da paleta do Flow Studio, curados primeiro.
`compose`	função	`(spec) -> dict[str, Any]`	Monta o fluxo a partir da especificação (nome, blocos, início, parâmetros expostos), liga entradas e valida.
`code_for`	função	`(doc, *, geometry='', workdir='hub_tool', expose=None) -> str`	Python que reconstrói o fluxo, valida, roda sem interface e imprime as métricas de cada nó.

pinneapple_hub.library.flowlink

Liga o Hub ao Flow Studio sobre o mesmo registro: monta um fluxo pronto em volta de qualquer tipo de nó e gera links entre os dois apps.

`pinneapple_hub/library/flowlink.py` · 173 linhas · testes: 8

nome	tipo	assinatura	o que faz
`FlowLinkError`	classe	`()`	Erro ao montar um fluxo para um tipo de nó (por exemplo tipo desconhecido).
`bundled_flows_using`	função	`(type_) -> list[str]`	Nomes dos fluxos embutidos que contêm um nó do tipo dado.
`flow_for_node`	função	`(type_, params=None, *, report=True) -> dict[str, Any]`	Monta entradas → ferramenta → relatório para um tipo de nó, reaproveitando um fluxo embutido ou escolhendo produtores para as entradas.
`b64url`	função	`(obj) -> str`	Codifica um objeto JSON em base64 para URL.
`from_b64url`	função	`(text) -> Any`	Decodifica o base64 de URL de volta para o objeto JSON.
`flow_link`	função	`(flow, *, base) -> str`	URL do Flow Studio que abre um fluxo (`?flow=`).

nome	tipo	assinatura	o que faz
`add_link`	função	`(type_, params=None, *, base) -> str`	URL do Flow Studio que adiciona um nó a um fluxo escolhido pelo usuário (`?add=`).

`pinneapple_hub.library.flowruns`

Execuções gravadas dos fluxos embutidos (`pinneapple_flow/flows`), mostradas na página de cada fluxo e de cada tipo de nó usado; válidas enquanto o SHA-256 do arquivo do fluxo não muda.

`pinneapple_hub/library/flowruns.py` · 109 linhas · testes: 8

nome	tipo	assinatura	o que faz
`flow_hash`	função	`(name) -> str`	Hash SHA-256 curto do arquivo JSON de um fluxo embutido.
`read_flow_runs`	função	`(path=FLOW_RUNS) -> dict[str, Any]`	Lê `flow_runs.json` com as execuções gravadas.
`current`	função	`(runs, name) -> dict[str, Any] | None`	A execução gravada de um fluxo, só se foi gravada para o arquivo como está agora.
`record`	função	`(names=None, *, timeout=900.0, workers=2, log=print) -> dict[str, Any]`	Roda os fluxos embutidos no sandbox e grava `flow_runs.json`.
`runs_for_node`	função	`(runs, type_) -> list[dict[str, Any]]`	Cada nó de um tipo nas execuções atuais: fluxo, id do nó, status, métricas e resumo.

`pinneapple_hub.library.myhub`

"Meu Hub": ferramentas compostas e salvas por uma pessoa, como itens do Hub numa pasta local (um JSON cada).

`pinneapple_hub/library/myhub.py` · 87 linhas · testes: 14

nome	tipo	assinatura	o que faz
`MyHubError`	classe	`()`	Erro do Meu Hub (`ValueError`), por exemplo limite de ferramentas atingido ou ferramenta grande demais para salvar.
`MyHub`	classe	`(root)`	Pasta local de ferramentas próprias com `save`, `list`, `get` e `delete`.

`pinneapple_hub.library.sandbox`

Roda um snippet num interpretador separado e de vida curta, guardando o que imprimiu e as figuras; tem limites de tempo e, em POSIX, de memória, CPU e arquivo, mas não bloqueia rede.

`pinneapple_hub/library/sandbox.py` · 104 linhas · testes: 22

nome	tipo	assinatura	o que faz
`code_hash`	função	`(code) -> str`	SHA-256 curto de um snippet; uma execução gravada só é atual enquanto o snippet tem esse hash.
`run_snippet`	função	`(code, *, timeout=60.0, memory_mb=None, cpu_s=None, max_figures=4, env=None) -> dict[str,…)`	Executa o código em diretório temporário com matplotlib Agg, sem GPU e com limites, e devolve `ok`, stdout, segundos e figuras PNG.

`pinneapple_hub.library.snippets`

Código mínimo e executável de cada item do Hub, rodado por `python -m pinneapple_hub.library run` e pelos testes; curto, offline e rápido. [sem teste]

`pinneapple_hub/library/snippets.py` · 121 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`model`	função	`(name, example) -> str`	Snippet de um modelo do zoo: carrega, prevê num exemplo e valida.
`dataset`	função	`(name, split) -> str`	Snippet de um dataset: carrega um split ou mostra informações e licença.
`geometry`	função	`(name) -> str`	Snippet de uma geometria da biblioteca de geometrias.
`flow`	função	`(name) -> str`	Snippet que carrega e roda um fluxo embutido.
`node`	função	`(type_) -> str`	Snippet de um tipo de nó do Flow Studio.
`auto_call`	função	`(target, node_type, defaults=None) -> str`	Chamada gerada de uma função da biblioteca com os padrões do formulário do nó.
`api`	função	`(dotted) -> str`	Snippet de uma função pública pelo nome pontilhado.
`mcp`	função	`(name, *, call) -> str`	Snippet de uma ferramenta do servidor MCP.
`example`	função	`(folder) -> str`	Snippet de uma pasta de exemplo.
`lab_item`	função	`(key) -> str`	Snippet de um item do Lab.
`symbols`	função	`(code) -> list[str]`	Nomes `pp.xxx` chamados por um snippet (para tags e links).

`pinneapple_hub.library.sources`

Coletores dos itens do Hub: cada um lê uma fonte existente do repositório (manifesto do zoo, biblioteca de geometrias, fluxos, registro de nós, AGENTS.md, servidor MCP, exemplos); nada é escrito à mão. [sem teste]

`pinneapple_hub/library/sources.py` · 579 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`rel`	função	`(p) -> str`	Caminho relativo à raiz do repositório, em formato POSIX.
`domain`	função	`(name, tags=()) -> str`	Domínio físico de um item pela regra do Lab (`pinneapple_lab.manifest.domain_of`).
`used_by_examples`	função	`(*needles) -> list[str]`	Pastas de exemplo cujos scripts contêm um dos termos dados como string ou chamada.
`models`	função	`() -> list[dict[str, Any]]`	Itens de modelos do zoo (manifesto, cartões, exemplo do teste, grafos ONNX do navegador).
`datasets`	função	`() -> list[dict[str, Any]]`	Itens dos datasets do manifesto do zoo.
`geometries`	função	`() -> list[dict[str, Any]]`	Itens da biblioteca de geometrias de `pinneapple_design`.
`flows`	função	`() -> list[dict[str, Any]]`	Itens dos fluxos embutidos do Flow Studio.
`nodes`	função	`() -> list[dict[str, Any]]`	Itens de cada tipo de nó da paleta do Flow Studio (curados e chamadas de biblioteca geradas).
`node_for_callable`	função	`() -> dict[str, str]`	Mapa `módulo:função` → tipo de nó que a envolve.
`recipes`	função	`() -> list[dict[str, Any]]`	Itens de receitas: blocos Python do AGENTS.md e exemplos da referência de módulos.
`apis`	função	`() -> list[dict[str, Any]]`	Um item por linha da tabela "Use these APIs first" do AGENTS.md.
`agent_tools`	função	`() -> list[dict[str, Any]]`	Ferramentas do servidor MCP, lidas do código-fonte (funções `@server.tool()`).
`examples`	função	`() -> list[dict[str, Any]]`	Itens das pastas de exemplos.

pinneapple_hub.model_card

`ModelCard`: contrato de reprodutibilidade de todo modelo enviado ao hub (arquitetura, config de treino, linhagem dos dados, métricas de validação calculadas e decisão de confiança).

`pinneapple_hub/model_card.py` · 180 linhas · testes: 21

nome	tipo	assinatura	o que faz
`ModelCard`	classe	`(name, description, task, version, inputs, outputs, validity_range, known_limitations, ar…)`	Cartão de modelo com tarefa, entradas/saídas, faixa de validade, limitações, arquitetura, equações e métricas; `validate()` lista problemas e há exportação para dict, JSON e Markdown.

pinneapple_hub.verified_card

Cartões de modelo cujas `validation_metrics` vêm só de uma rodada do `PhysicsGuardrail` contra o modelo; cartão que não passa em `ModelCard.validate()` é recusado.

`pinneapple_hub/verified_card.py` · 122 linhas · testes: 6

nome	tipo	assinatura	o que faz
`CardRefused`	classe	`(problems)`	Exceção: o cartão gerado do guardrail não passou em `ModelCard.validate()`; `.problems` diz por quê.
`validation_metrics_from_report`	função	`(report) -> dict[str, float]`	Converte um relatório do guardrail em métricas: valor de cada checagem numérica e `<nome>_passed` (1.0/0.0) para as de passa/falha.
`card_from_guardrail`	função	`(spec, report, *, name, preset, preset_kwargs=None, architecture, architecture_config, tr…)`	Monta um `ModelCard` a partir de um relatório do guardrail e levanta `CardRefused` se ele não valida.
`certify_model`	função	`(preset, name, *, preset_kwargs=None, architecture='modified_mlp', architecture_config=No…)`	Constrói ou carrega um modelo para um preset, treina se não há pesos, roda o guardrail e devolve o cartão verificado.

pinneapple_hub.zoo

`pp.zoo`: modelos pré-treinados e verificados em uma linha (listar, carregar, prever, avaliar, validar, consultar checagens, exportar, fazer benchmark e fine-tune com guarda).

`pinneapple_hub/zoo/__init__.py` · 341 linhas · testes: 125

nome	tipo	assinatura	o que faz
`FinetuneRejectedError`	classe	`()`	Exceção: o modelo ajustado ficou pior que o original no teste original (use `force=True` para salvar mesmo assim).
`FinetuneResult`	classe	`(model, original, before, after, accepted, tolerance, history, new_data_metrics, saved_to)`	Resultado de `finetune`: modelo novo, original, métricas antes/depois, se foi aceito, tolerância, histórico e onde foi salvo.
`export_onnx`	função	`(name, path, *, checks=False, check_parity=True, offline=None) -> dict[str, Any]`	Exporta um modelo para ONNX com pré e pós-processamento dentro do grafo e confere paridade com o PyTorch.
`onnx_parity`	função	`(name, onnx_model, X=None, *, offline=None) -> float`	Maior diferença escalada entre um grafo ONNX e o modelo PyTorch.
`query_checks`	função	`(name, inputs=None, *, offline=None, **named) -> builtins.list`	Checagens de confiança e veredito para cada linha de entrada.
`check_spec`	função	`(name, *, offline=None) -> dict[str, Any]`	Especificação declarativa das checagens por consulta de um modelo.
`export_web`	função	`(name, directory, *, offline=None) -> dict[str, Any]`	Grafo ONNX com checagens mais o JSON de cartão e spec, para o navegador.
`list`	função	`(task=None) -> builtins.list`	Nomes dos modelos publicados, com filtro opcional por tarefa ou tag.

nome	tipo	assinatura	o que faz
`info`	função	`(name, *, offline=None, version=None) -> ModelCard`	Cartão do modelo: tarefa, entradas/saídas com unidades, faixa de validade, métricas, dados de treino, licença e citação.
`load`	função	`(name, *, offline=None, version=None) -> ZooModel`	Preditor pronto, com pesos conferidos pelo SHA-256 do manifesto; `version` carrega uma versão substituída.
`versions`	função	`(name) -> builtins.list[dict[str, str]]`	Todas as versões de um modelo, da mais nova para a mais antiga, com o motivo de cada substituição.
`predict`	função	`(name, inputs=None, **named) -> dict[str, np.ndarray]`	Atalho para `load(name).predict(...)`.
`catalogue`	função	`() -> str`	Página Markdown com todos os modelos e datasets: métricas, decisão de confiança, tamanho e link do Hub.
`status`	função	`(name) -> dict[str, str]`	De onde cada arquivo do modelo seria lido agora (bundled, cached ou remote), sem baixar.
`evaluate`	função	`(name, dataset=None, *, split=None, offline=None, rtol=None) -> dict[str, Any]`	Métricas do modelo contra os casos de referência guardados.
`validate`	função	`(name, inputs=None, *, offline=None)`	`TrustReport` do modelo: métricas do cartão reproduzidas, checagens físicas e faixa de validade.
`finetune`	função	`(name, new_data, *, epochs=200, lr=0.0001, batch=256, weight_decay=0.0, tolerance=0.02, s…)`	Ajusta o modelo com dados novos e só aceita se não piorar no teste original além da tolerância (com replay opcional).
`benchmark`	função	`(name, *, budget_s=60.0, seeds=1, baselines=('retrain',), out_dir='zoo_benchmark', force=…)`	Coloca o modelo do zoo em `pp.benchmark` contra a mesma arquitetura treinada do zero no orçamento dado.

`pinneapple_hub.zoo._artifacts`

Armazém de artefatos do zoo: manifesto, checagem SHA-256, cache local e download preguiçoso do Hugging Face Hub.

`pinneapple_hub/zoo/_artifacts.py` · 237 linhas · testes: 81

nome	tipo	assinatura	o que faz
`ZooError`	classe	`()`	Classe base dos erros do zoo.
`ArtifactNotPublishedError`	classe	`()`	O artefato está no manifesto mas ainda não está no Hugging Face Hub.
`ArtifactUnavailableError`	classe	`()`	O artefato não está empacotado nem em cache e não pode ser baixado (offline ou erro de rede).
`ChecksumMismatchError`	classe	`()`	O SHA-256 de um arquivo difere do manifesto: corrompido, truncado ou adulterado.
`sha256_file`	função	`(path, chunk=1 << 20) -> str`	SHA-256 de um arquivo, lido em blocos.
`manifest`	função	`() -> dict[str, Any]`	O `manifest.json` lido (relido quando o arquivo muda).
`cache_dir`	função	`() -> Path`	Pasta de cache local dos artefatos.
`is_offline`	função	`(offline=None) -> bool`	Diz se o zoo opera offline: pelo argumento, senão pelas variáveis `PINNEAPPLE_OFFLINE` ou `HF_HUB_OFFLINE`.
`entry`	função	`(kind, name, version=None) -> dict[str, Any]`	Entrada do manifesto de um modelo ou dataset, opcionalmente de uma versão específica.
`versions`	função	`(kind, name) -> list[dict[str, str]]`	Todas as versões de um item, da mais nova à mais antiga, com revisão e motivo de substituição.
`revisions`	função	`(ent) -> list[str]`	Revisões do Hub que guardam os arquivos de uma entrada, na ordem em que são tentadas.
`fetch`	função	`(kind, name, filename, *, offline=None, version=None) -> Path`	Caminho local de um arquivo de modelo ou dataset: empacotado, em cache ou baixado e conferido.
`status`	função	`(kind, name) -> dict[str, str]`	De onde cada arquivo de um item seria lido agora (bundled, cached ou remote), sem baixar.

pinneapple_hub.zoo._bench

pp.zoo.benchmark: compara o modelo publicado (sem treino) com a mesma arquitetura treinada do zero sob o ComputeBudget de pinneapple_analysis.benchmark. [sem teste]

pinneapple_hub/zoo/_bench.py · 224 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
run_benchmark	função	(model, *, budget_s, seeds, baselines, out_dir, force, log, **kwargs)	Monta o problema a partir do dataset do modelo e roda pp.benchmark com os métodos zoo_pretrained e zoo_retrain.

pinneapple_hub.zoo._catalog

Descrição estática de todo modelo e dataset publicado (fonte de manifest.json e dos model_card.json); só entra o que passou no portão do zoo. [sem teste]

pinneapple_hub/zoo/_catalog.py · 715 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
hf_repo	função	(kind, name) -> str	Repositório do Hub de um modelo/dataset: PINNeAPPle-Labs/<nome com hífens>.

pinneapple_hub.zoo._models

Adaptadores dos modelos do zoo: preditor com faixa de validade, métricas, checagens físicas e ganchos de fine-tune; a base ZooModel levanta NotImplementedError nos ganchos abstratos.

pinneapple_hub/zoo/_models.py · 657 linhas · testes: 7 · palavras-chave no código (heurística; o texto acima diz o status real): levanta NotImplementedError em algum caminho

nome	tipo	assinatura	o que faz
OutOfRangeWarning	classe	()	Aviso: entradas fora da faixa de validade (a previsão é extrapolação).
ZooModel	classe	(name, card, config, state, spec)	Classe base de um modelo carregado: predict, in_range, reference, avaliação e save; subclasses implementam construção e métricas.
AirfoilMLP	classe	()	Adaptador do ensemble MLP de aerofólio CST: prevê Cl, Cd, Cm e L/D com o desvio do ensemble.
AirfoilGNN	classe	()	Adaptador MeshGraphNet do campo próximo do aerofólio (p, Ux, Uy, viscosidade turbulenta) e coeficientes; não há modelo com esse adaptador no manifesto atual.
HeatsinkForced	classe	()	Adaptador do dissipador em convecção forçada: prevê resistência térmica e queda de pressão.
HeatsinkNatural	classe	()	Adaptador do dissipador em convecção natural, sem velocidade do ar nem queda de pressão, treinado com restrições de monotonicidade.
RoverContact	classe	()	Adaptador do contato roda-solo de rover: prevê Fx, Fz e My.
SupersededModelWarning	classe	()	Aviso: foi carregada uma versão substituída de um modelo do zoo.
load_dir	função	(directory) -> ZooModel	Carrega um modelo salvo com ZooModel.save (por exemplo uma cópia ajustada).
load_model	função	(name, *, offline=None, version=None) -> ZooModel	Carrega um modelo do zoo pelo nome, verificando os arquivos.
copy_model	função	(m) -> ZooModel	Cópia independente de um modelo carregado.
fit	função	(m, X, Y, *, epochs=300, lr=0.0001, batch=256, weight_decay=0.0, seed=0, progress=None) -…)	Adam sobre os alvos normalizados do próprio modelo (alvos NaN ignorados); devolve o histórico de perda.

pinneapple_hub.zoo.catalogue

Catálogo do zoo como uma página Markdown com métricas, tamanho e link do Hub de cada item. [sem teste]

pinneapple_hub/zoo/catalogue.py · 73 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
markdown	função	() -> str	Gera a página do catálogo em Markdown.

pinneapple_hub.zoo.checks

Checagens de confiança por consulta (faixa de validade, região esparsa, saída não física ou não finita, resíduo) com IDs de regra de pinneapple check, e o veredito pass/warn/fail que elas formam.

pinneapple_hub/zoo/checks.py · 358 linhas · testes: 34

nome	tipo	assinatura	o que faz
has_query_checks	função	(zm) -> bool	Diz se o modelo tem checagens por consulta; senão só in_range vale.
check_spec	função	(zm) -> dict[str, Any]	Descrição declarativa das checagens de um modelo (uma entrada por coluna de saída).
checks_module	função	(zm, predictor=None)	Módulo torch que mapeia linhas de consulta para valores das checagens; é o mesmo que vai para o grafo ONNX.
check_values	função	(zm, X) -> tuple[np.ndarray, np.ndarray]	Valores e auxiliares das checagens para linhas X em unidades físicas.
compare	função	(value, op, limit) -> bool	Compara valor op limite; NaN nunca passa.
evaluate	função	(spec, values, aux) -> dict[str, Any]	Status, detalhes e veredito para uma linha de valores (o navegador faz o mesmo com as saídas do grafo).
verdict	função	(checks) -> dict[str, str]	Pior status das checagens de confiança (restrições de projeto excluídas), com título e explicação.
query_checks	função	(model, inputs, *, offline=None) -> list[dict[str, Any]]	Checagens e veredito para cada linha de entrada de um modelo.

pinneapple_hub.zoo.datasets

pp.datasets: datasets verificados em uma linha, guardados como npz sem pickle; os pequenos vêm no pacote, os outros são baixados do Hub, conferidos por SHA-256 e cacheados.

pinneapple_hub/zoo/datasets.py · 199 linhas · testes: 8

nome	tipo	assinatura	o que faz
DatasetInfo	classe	(name, version, title, description, license, source, verification, splits, columns, citat…)	Metadados de um dataset: versão, título, descrição, licença, fonte, verificação, splits, colunas, citação e repositório.
DatasetSplit	classe	(data, name, split, ds_info)	Um split: {coluna: array} com nome, split e informações; tem ajudantes como xy.
audit_card	função	(name) -> dict[str, Any] | None	Cartão de auditoria física guardado de um dataset, ou None se não foi auditado.
audit	função	(name, *, offline=None, save=False, **kw)	Auditoria física de um dataset via pinneapple_data.audit.audit_zoo_dataset.
list	função	() -> builtins.list[str]	Nomes dos datasets publicados.
info	função	(name) -> DatasetInfo	DatasetInfo de um dataset.
split_files	função	(name, split=None) -> builtins.list[str]	Arquivos que guardam um split (data.npz, <split>.npz ou fragmentos <split>-NN.npz).
load	função	(name, split=None, *, offline=None)	Carrega um split (DatasetSplit) ou todos como dict.
status	função	(name) -> dict[str, str]	De onde os arquivos seriam lidos agora (bundled, cached ou remote), sem baixar.

pinneapple_hub.zoo.demo

Dados do Space estático de demonstração do zoo: varreduras de cada entrada com banda de incerteza e veredito das checagens, e a escrita dos arquivos do Space.

pinneapple_hub/zoo/demo.py · 189 linhas · testes: 21

nome	tipo	assinatura	o que faz
models	função	() -> list[str]	Nomes dos modelos que a demonstração oferece.
trust	função	(name) -> dict[str, Any]	Relatório de confiança do modelo como um todo: decisão e motivos.
controls	função	(name) -> list[dict[str, Any]]	Uma entrada por variável de entrada: nome, unidade, descrição e faixa.
sweep	função	(name, values, vary, *, n=61, margin=0.15) -> dict[str, Any]	Previsões e checagens ao varrer uma entrada um pouco além da faixa, com as outras fixas.
bands	função	(outputs) -> dict[str, dict[str, Any]]	Saídas principais e como desenhar a incerteza de cada uma (desvio do ensemble ou limite conformal).
precompute	função	(name, *, n=SWEEP_POINTS) -> dict[str, Any]	Tudo o que o Space mostra de um modelo sem rodá-lo: controles, saídas, varreduras e checagens.
export_space	função	(directory, models_=None) -> dict[str, Any]	Grava dados e runtime do Space estático num diretório.

pinneapple_hub.zoo.export

Exportação ONNX dos modelos do zoo com pré e pós-processamento no grafo e, opcionalmente, as checagens por consulta; também safetensors. Adaptador sem exportação levanta NotImplementedError.

pinneapple_hub/zoo/export.py · 317 linhas · testes: 55 · palavras-chave no código (heurística; o texto acima diz o status real): levanta NotImplementedError em algum caminho

nome	tipo	assinatura	o que faz
predictor_module	função	(zm)	Módulo torch que leva as entradas do cartão às saídas, base do grafo ONNX.
output_names	função	(zm) -> dict[str, list[str]]	Mapa saída ONNX → chaves de predict das suas colunas.
as_graph_outputs	função	(zm, pred) -> dict[str, np.ndarray]	Saída de predict reorganizada como as saídas do grafo.
output_scale	função	(zm) -> dict[str, np.ndarray]	Escala de tolerância de paridade por coluna (relativa para saídas positivas modeladas em log).
parity_error	função	(zm, ref, got) -> float	Maior |obtido - referência| / escala sobre todas as colunas de saída.
export_bytes	função	(zm, *, checks=False) -> bytes	Bytes do grafo ONNX de um modelo carregado.
export_onnx	função	(name, path, *, checks=False, check_parity=True, offline=None) -> dict[str, Any]	Exporta um modelo do zoo para ONNX num caminho, com checagem de paridade opcional.
parity_inputs	função	(zm, *, n_random=500, seed=7) -> np.ndarray	Entradas de teste e de referência independente do modelo mais pontos aleatórios dentro da faixa.
run_onnx	função	(data, X, query=None) -> dict[str, np.ndarray]	Roda um grafo no onnxruntime (CPU, uma thread) e devolve as saídas em float64.
onnx_parity	função	(name, onnx_model, X=None, *, offline=None) -> float	Maior diferença escalada entre as previsões do grafo e as do modelo PyTorch.
checks_parity	função	(name, onnx_model, X=None, *, offline=None) -> dict[str, float]	Compara as checagens de um grafo exportado com checks=True com as do Python.
export_safetensors	função	(name, path, *, offline=None) -> dict[str, Any]	Grava os pesos do modelo como safetensors.
load_safetensors	função	(name, path, *, offline=None) -> ZooModel	Carrega o modelo do zoo com pesos lidos de um arquivo safetensors.

pinneapple_hub.zoo.hub_cards

Cartões README dos repositórios do zoo no Hugging Face: front matter YAML lido pelo Hub e o texto para pessoas.

pinneapple_hub/zoo/hub_cards.py · 368 linhas · testes: 21

nome	tipo	assinatura	o que faz
space_url	função	() -> str	URL do Space de demonstração do zoo.
front_matter	função	(meta) -> str	Bloco YAML de front matter a partir de um dict.
parse_front_matter	função	(readme) -> dict[str, Any]	Lê o bloco YAML do topo de um README ({} se não há).
code_blocks	função	(readme, lang='python') -> list[str]	Blocos de código cercados de um README numa linguagem.
split_order	função	(name) -> list[str]	Splits de um dataset com train/val/cal/test primeiro e os outros na ordem do manifesto.
dataset_card	função	(name, layout) -> str	README de um repositório de dataset: front matter (configs, features, licença, tags) e o cartão.
model_example	função	(zm) -> dict[str, Any]	Um caso de referência do teste do modelo: entradas, saídas do modelo e referência.
model_card	função	(name, *, example, onnx_info=None, files=()) -> str	README de um repositório de modelo: metadados, uso, faixa de validade, cartão de confiança, relatório e arquivos.
space_readme	função	() -> str	README do Space estático de demonstração.
example_json	função	(example) -> str	Serializa o exemplo do modelo em JSON.

pinneapple_hub.zoo.parquet

Cópias Parquet determinísticas dos datasets do zoo para o visualizador do Hugging Face e `datasets.load_dataset`; os npz continuam a fonte de verdade.

`pinneapple_hub/zoo/parquet.py` · 281 linhas · testes: 21

nome	tipo	assinatura	o que faz
read_npz	função	(paths) -> dict[str, dict[str, np.ndarray]]	Lê npz com arrays `<split>__<coluna>` em {split: {coluna: array}}, juntando fragmentos.
plan_split	função	(arrays, *, explode_row_bytes=EXPLODE_ROW_BYTES, max_shard_bytes=MAX_SHARD_BYTES, row_gro…)	Decide como um split vira Parquet (explodir amostras grandes, fragmentos, grupos de linhas).
write_dataset	função	(name, data, out_dir, *, split_order=(), column_order=(), explode_row_bytes=EXPLODE_ROW_B…)	Escreve os arquivos Parquet de um dataset e devolve o layout.
read_parquet	função	(paths) -> dict[str, np.ndarray]	Lê de volta os arrays npz de um split a partir dos fragmentos Parquet (inverso de `write_dataset`).
parquet_files	função	(layout) -> list[str]	Todos os arquivos Parquet de um layout.
features_yaml	função	(schema) -> list[dict[str, Any]]	Lista `dataset_info.features` de um schema Parquet, no formato que `datasets` escreve.

pinneapple_hub.zoo.publish

Publica o zoo no Hugging Face Hub (um repositório por item em `PINNeAPPle-Labs` mais o Space) e verifica o que o Hub serve; usado por `scripts/zoo/publish_to_hub.py` e pelo workflow de publicação.

`pinneapple_hub/zoo/publish.py` · 611 linhas · testes: 49

nome	tipo	assinatura	o que faz
plan	função	(staging=None, *, need_files=True, hub_fallback=False, work=None, only=()) -> list[dict]	Lista de itens a publicar com tipo, nome, repositório, revisão e arquivos.
prepare	função	(it, work) -> dict	Grava os arquivos derivados de um item (Parquet e cartão, ou safetensors, ONNX e exemplo) e o README.
publish	função	(items, token, *, work=None, tags=True) -> list[str]	Envia cada item num commit, marca com a revisão e devolve os que falharam.
build_space	função	(out, app_dir, playground_dir, models=None) -> list[str]	Monta o Space estático de demonstração num diretório.
publish_space	função	(token, app_dir, playground_dir, *, work=None, from_hub=True) -> list[str]	Monta e envia o Space num commit.

nome	tipo	assinatura	o que faz
`verify_files`	função	`(items, allow_unpublished=False) -> int`	Baixa anonimamente cada arquivo do manifesto e confere o SHA-256.
`viewer_status`	função	`(repo, *, timeout=2400, interval=30, get=_get_json, sleep=time.sleep) -> dict[str, Any]`	Estado do visualizador de datasets de um repositório, esperando até o tempo limite.
`verify_viewer`	função	`(names=None, *, timeout=2400, interval=30, get=_get_json, sleep=time.sleep) -> int`	Confere o visualizador de todos os datasets.
`verify_model`	função	`(name, work) -> list[str]`	Problemas encontrados ao carregar do Hub e checar um modelo publicado.
`verify_models`	função	`(names=None) -> int`	`verify_model` para todos os modelos, anônimo, sem cópias empacotadas e com cache vazio.
`verify_space`	função	`(*, timeout=2400, interval=30, runtime=None, fetch=_fetch, sleep=time.sleep) -> int`	Espera o Space rodar e servir a página; 0 quando está OK.

`pinneapple_hub.zoo.relations`

Relações metamórficas que a física de cada modelo do zoo implica (simetria, monotonicidade), checadas sem solução de referência via `pinneapple_analysis.validation.metamorphic`.

`pinneapple_hub/zoo/relations.py` · 95 linhas · testes: 9

nome	tipo	assinatura	o que faz
`check_zoo_relations`	função	`(model, name=None, *, n=64, seed=0, search='random', relations=None) -> MetamorphicReport`	Roda as relações de um modelo do zoo e devolve um `MetamorphicReport`.

`pinneapple_hub.zoo.validation_reports`

Relatórios `pp.validate` guardados de cada modelo do zoo, mostrados no cartão e comparados com um novo antes de uma versão; uma checagem que deixa de passar bloqueia a publicação.

`pinneapple_hub/zoo/validation_reports.py` · 59 linhas · testes: 7

nome	tipo	assinatura	o que faz
`build`	função	`(name)`	Valida o modelo e grava o relatório.
`stored`	função	`(name) -> dict[str, Any] | None`	Lê o relatório guardado de um modelo, ou `None`.
`check_release`	função	`(name, model=None)`	Diferença entre o relatório atual (ou de um modelo novo) e o guardado, mais o relatório novo.
`card_section`	função	`(name) -> str`	Bloco Markdown do relatório guardado para o cartão do Hub (vazio se não há).

`pinneapple_hub.zoo.web`

Modelos do zoo para o navegador: grafos ONNX com checagens, cartões em JSON e o runtime JS (`onnx.js`, `zoo.js`) usado pelo Playground.

`pinneapple_hub/zoo/web/__init__.py` · 160 linhas · testes: 13

nome	tipo	assinatura	o que faz
`runtime_dir`	função	`() -> Path`	Pasta do runtime JS (`onnx.js`, `zoo.js`).
`reference_data`	função	`(zm) -> dict[str, Any]`	Pontos de referência para plotar ao lado das previsões (por exemplo rodadas OpenFOAM e polar XFOIL no aerofólio).
`export_web`	função	`(name, directory, *, offline=None) -> dict[str, Any]`	Grava `<nome>.onnx` e `<nome>.json` para o navegador e devolve o JSON.
`fixtures`	função	`(*, offline=None) -> dict[str, Any]`	Respostas do Python para entradas fixas (dentro e fora da faixa), usadas nos testes JS.
`export_bundle`	função	`(directory, models=MODELS, *, offline=None) -> dict[str, Any]`	Tudo o que o Playground carrega: cada modelo, grafos de geometria CST e `fixtures.json`.

`pinneapple_hub.zoo.web.__main__`

`python -m pinneapple_hub.zoo.web [DIR]`: grava o pacote web do zoo (padrão `data/web/zoo`). [sem teste]

`pinneapple_hub/zoo/web/__main__.py` · 23 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`main`	função	`(argv=None) -> int`	Ponto de entrada que chama `export_bundle` no diretório dado.

Exemplos de uso reais do repositório

Quickstart do zoo: listar, carregar e prever — `examples/model_zoo/quickstart.py`

```
import pinneapple as pp

print(pp.zoo.list())
print(pp.datasets.list())
card = pp.zoo.info("airfoil_cst_rans_mlp")
print(card.task)

model = pp.zoo.load("airfoil_cst_rans_mlp")
naca2412 = dict(T0=0.169591, T1=0.144959, T2=0.131433, M0=0.024922, M1=0.071088, M2=0.058960)  # NACA 2412 (CST)
alphas = np.array([0.0, 2.0, 4.0, 6.0])
pred = model.predict(**naca2412, alpha_deg=alphas)
```

Fine-tune com guarda no teste original — `examples/model_zoo/quickstart.py`

```
new = pp.datasets.load("airfoil_rans_cst_re4e6", split="naca_reference")    # stand-in for your own new runs
ft = pp.zoo.finetune("airfoil_cst_rans_mlp", new, epochs=50, lr=1e-5)
print(ft.summary())
if ft.accepted:
    ft.save(tempfile.mkdtemp())
else:
    print("not saved: the fine-tuned model is worse on the original test set (force=True overrides)")
```

Busca no índice do Hub — `pinneapple_hub/library/__init__.py`

```
from pinneapple_hub import library as hub
index = hub.load_index()                              # data/web/hub/index.json (generated, not committed)
hits = hub.search(index, "airfoil", kind="model")     # ranked items
item = hub.get(index, "geometry:f1_car")
```

Testes e status de validação

- Módulos com pelo menos um teste que os importa: 23 de 31.
- Funções de teste em arquivos que importam algum módulo do pacote: 183.

Limitações conhecidas

- `push_to_hub`/`from_pretrained` exigem o extra `hub` (`huggingface_hub`) e, para enviar, uma conta e token do Hugging Face; o módulo `hub.py` não tem teste direto.
- Datasets e modelos que não vêm empacotados precisam de rede no primeiro uso; offline, levantam `ArtifactUnavailableError`, e um arquivo com SHA-256 diferente do manifesto levanta `ChecksumMismatchError`.
- Re-auditar um cartão existente só refaz a checagem de esquema: os pesos e os kwargs exatos do preset não viajam no JSON do cartão, então as alegações físicas não podem ser re-executadas a partir dele (docs-tring de `verified_card`).
- O sandbox de snippets limita tempo e, em POSIX, memória, CPU e tamanho de arquivo, mas não bloqueia rede; o docstring pede rodar o app sem saída de rede quando os snippets vêm de visitantes.

- `ZooModel` levanta `NotImplementedError` nos ganchos abstratos e `export.predictor_module` levanta `NotImplementedError` para adaptador sem exportação ONNX; o adaptador `AirfoilGNN` existe mas nenhum modelo do manifesto atual o usa.
- Sem teste direto: `hub`, `library.snippets`, `library.sources`, `zoo._bench`, `zoo._catalog`, `zoo.catalogue` e as linhas de comando `__main__`.
- Os cartões do Hub declaram `inference: false`: nenhum provedor de inferência do Hub roda esses modelos; o Space de demonstração é estático e avalia ONNX no navegador.

5. `pinneapple_lab`

Objetivo

O `pinneapple_lab` é o laboratório de experimentos do PINNeAPPle. Ele roda muitos experimentos de forma reprodutível e acumula um banco de resultados e de datasets. Um experimento é uma classe com parâmetros padrão e um método `run(ctx)`; o runner dá a cada execução um id derivado do nome, da versão e dos parâmetros, grava entradas, saídas, métricas, checagens de validação, figuras e datasets numa pasta própria e indexa tudo em SQLite (`LabStore`). Rodar de novo com os mesmos parâmetros devolve o resultado em cache.

Por cima desse banco existe uma camada de curadoria. `curation.curate` avalia cada execução em dimensões calculadas só a partir do que foi gravado (checagens, métricas, arquivos, código, datasets, documentação e uma revisão humana opcional) e dá uma nota de A (flagship) a F (não utilizável). As notas são portões, não média ponderada: uma checagem que falhou nunca é escondida por uma figura bonita. `next_steps` diz o que falta para a próxima letra e qual ação do PINNeAPPle fecharia cada lacuna; `seal` carimba a letra em figuras, renders e vídeos; `explorer`, `html`, `server`, `reports` e `showcase` publicam tudo como página, API, relatório ou painel animado.

O pacote se liga ao resto da biblioteca de várias formas. Os experimentos embutidos em `pinneapple_lab.experiments` chamam solvers e modelos de outros pacotes (`pinneapple_simulation`, `pinneapple_physics`, `pinneapple_neural`, `pinneapple_design`) e conferem os resultados contra soluções exatas ou referências publicadas. O módulo `manifest` define o formato `experiment.json`, lido pelo site, pela curadoria e pelo Flow Studio, e converte para manifesto resultados de `pp.solve` (`from_solution`), tabelas de `pp.compare` (`from_comparison`), workflows de `pinneapple_core` e execuções do Flow Studio. Qualquer script de `examples/` vira item do Lab com `pinneapple_lab.experiment(...)` (`capture`).

A linha de comando `python -m pinneapple_lab` reúne tudo: `list`, `run`, `sweep`, `status`, `report`, `serve`, `curate`, `review`, `brief`, `seal`, `stamp`, `improve`, `register`, `scaffold`, `manifests --check` (o portão de CI dos manifestos), `showcase` e `export`.

Números do inventário

módulos	linhas	classes públicas	funções públicas	testes em arquivos que importam o pacote	módulos sem teste direto
42	11.775	45	119	96 (9 arquivos)	27

Submódulos

submódulo	o que faz	módulos	classes + funções	testes*
`__main__`	Linha de comando `python -m pinneapple_lab`: rodar, varrer, curar, carimbar, servir e exportar.	1	1	59
`capture`	`experiment(...)`: embrulha um script para que ele deixe um manifesto, um item do Lab e uma nota.	1	3	0
`context`	`RunContext`, o objeto que o experimento recebe para gravar tudo o que produz.	1	1	0
`curation`	Curadoria: avalia execuções em dimensões de evidência e dá notas A-F com prontidão por uso.	1	17	65
`dataset`	Datasets gerados pelas execuções, gravados em shards `.npz` com um cartão descritivo.	1	2	0

submódulo	o que faz	módulos	classes + funções	testes*
`experiments`	Experimentos embutidos (ODE, PDE, LBM, FEM, CFD, descoberta de leis, ROM, importação de resultados do repositório).	22	39	20
`explorer`	Página de experimentos com selo A-F, cartão de confiança, métricas, mídia e filtros.	1	11	6
`html`	Catálogo navegável do Lab em uma única página HTML autocontida.	1	6	14
`improve`	Comando `improve`: mostra as ações para a próxima letra e, se pedido, roda as executáveis e recura.	1	2	0
`manifest`	Formato `experiment.json` (manifesto de experimento), conversões, validação e portão de CI.	1	19	29
`next_steps`	Lacunas entre a letra atual e a próxima, cada uma com as ações do PINNeAPPle que a fecham.	1	9	31
`reports`	Relatórios em HTML, Markdown ou PDF de um item ou de um conjunto filtrado.	1	6	20
`results_rules`	Regras que extraem métricas e checagens dos arquivos de resultados que um exemplo versiona.	1	6	14
`runner`	Execução de um experimento ou de varreduras (grade ou amostras), com cache e paralelismo.	1	3	0
`seal`	Selo de qualidade PINNeAPPle: badge com a letra da curadoria, carimbado em figuras e vídeos.	1	21	39
`server`	Servidor HTTP (só biblioteca padrão) do catálogo, arquivos das execuções, API JSON e downloads.	1	2	26
`showcase`	Painel animado (MP4, GIF, PNG) com os melhores resultados, montado a partir da curadoria.	1	5	3
`spec`	Classe base `Experiment` e registro de experimentos por nome.	1	4	0
`store`	`LabStore`: pastas de execução mais índice SQLite, consultas, exportação de datasets e catálogo.	1	2	0
`uq`	Checagens de incerteza para o cartão de confiança: cobertura de intervalos, GCI e espalhamento de ensemble.	1	5	20

* maior número de testes que importam um módulo do submódulo.

Módulos e API pública

`pinneapple_lab`

Ponto de entrada do pacote: reexporta `Experiment`, `register`, `run`, `sweep`, `LabStore`, `experiment` e afins.

`pinneapple_lab/__init__.py` · 43 linhas · testes: 71

`pinneapple_lab.__main__`

Linha de comando `python -m pinneapple_lab` com subcomandos para rodar, varrer, curar, revisar, carimbar, servir, gerar relatórios e exportar datasets.

`pinneapple_lab/__main__.py` · 366 linhas · testes: 59

nome	tipo	assinatura	o que faz
`main`	função	`(argv=None) -> int`	Monta o `argparse` com todos os subcomandos e despacha para as funções do pacote; devolve o código de saída.

`pinneapple_lab.capture`

Embrulha um script ou exemplo (`with experiment(...) as ex:` ou decorador) para gravar parâmetros, métricas, checagens e arquivos num manifesto e registrá-lo no Lab. [sem teste direto]

`pinneapple_lab/capture.py` · 168 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Capture`	classe	`(name, *, title='', description='', domain='', methods=(), modules=(), params=None, out_d…)`	Contexto/decorador que acumula parâmetros, métricas (`log_metric`), checagens (`check`) e arquivos (`add_file`) e, ao sair, escreve o manifesto e registra o item no Lab.
`experiment`	função	`(name, **kw) -> Capture`	Cria um `Capture`, usável como `with experiment(...) as ex:` ou como `@experiment(...)`.
`register_manifest`	função	`(path, lab=None, *, curate=True) -> str`	Importa um manifesto válido no Lab como item `manifest/<id>`, recura e devolve a chave do item; recusa manifesto inválido.

pinneapple_lab.context

Define o `RunContext`, que o experimento recebe em `run(ctx)` e que grava tudo na pasta da execução e no índice. [sem teste direto]

`pinneapple_lab/context.py` · 217 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`RunContext`	classe	`(run_dir, params, experiment, run_id, seed=None)`	Objeto entregue a `Experiment.run`: dá os parâmetros e grava log, estágios, entradas, saídas, figuras, imagens, GIFs, métricas, checagens e datasets na pasta da execução.

pinneapple_lab.curation

Curadoria das execuções: avalia dimensões de evidência (validação, referência, baseline, física, incerteza, reprodutibilidade etc.) e atribui notas A-F com prontidão para produto, artigo, marketing ou dados de treino. Usa o vocabulário de `pinneapple_veriphysics.applicability`.

`pinneapple_lab/curation.py` · 970 linhas · testes: 65

nome	tipo	assinatura	o que faz
`Dimension`	classe	`(name, score, status, tier, evidence)`	Uma dimensão avaliada: nota 0-1 (ou None = não rodou), status, nível de evidência e lista de evidências.
`Assessment`	classe	`(run_id, experiment, status, tier, score, coverage, dimensions, readiness, gaps, reviewed…)`	Avaliação completa de uma execução: letra, nota, cobertura, dimensões, prontidão por uso e lacunas.
`infer_check_kind`	função	`(check) -> str`	Deduz de que uma checagem é evidência (referência, benchmark, física etc.) pelo nome, pelo valor de referência ou pelo texto de detalhe.
`load_reviews`	função	`(root) -> dict[str, Any]`	Lê as revisões humanas gravadas em `<lab>/reviews.json`.
`save_review`	função	`(root, experiment, **fields) -> dict[str, Any]`	Grava (mesclando) a revisão humana de um experimento em `<lab>/reviews.json`.
`assess_run`	função	`(store, row, *, meta=None, reviews=None, siblings=None) -> Assessment`	Avalia uma execução (uma linha de `LabStore.runs()`) e devolve seu `Assessment`.
`path_to_next`	função	`(tier, issue, checks, dims, evidence, ok, is_ml, ctx, *, reviewed=False) -> list[Any]`	Lista as lacunas entre a letra atual e a próxima, cada uma com ações do PINNeAPPle, a partir das mesmas evidências da nota.
`next_steps`	função	`(item, lab=None) -> list[Any]`	Lê do registro de curadoria o caminho para a próxima letra de um item, como lista de `Gap`.
`problem`	função	`(status, checks, error=None) -> dict[str, Any] | None`	Diz qual regra E ou F a execução cai (travou, divergiu, física falhou, contradiz a referência...) ou None se tudo passou.
`problem_headline`	função	`(issue) -> dict[str, Any]`	Resumo de um item E ou F: a checagem principal que falhou, a regra e a dica do próximo passo.
`check_line`	função	`(check, max_chars=64) -> str`	Uma linha que diz contra o que a checagem comparou (rótulo dado pelo experimento ou nome da checagem mais detalhe).
`headline`	função	`(checks, tier, labels=None) -> dict[str, Any]`	Evidência que um selo ou bloco do showcase pode imprimir, tirada só das checagens que passaram.
`trust_stages`	função	`(d, checks, metrics, dims, tier, readiness, meta, is_ml, limitations) -> dict[str, dict[s…)`	Responde às seis perguntas do cartão de confiança (dados, modelo, física, benchmark, incerteza...) com base nas evidências da execução.

nome	tipo	assinatura	o que faz
`evidence_stages`	função	`(d, checks, metrics, dims, is_ml, limitations) -> dict[str, dict[str, Any]]`	As cinco primeiras perguntas do cartão de confiança, que não dependem da letra e alimentam a regra C/D.
`plan_experiment`	função	`(runs, meta, review) -> dict[str, Any]`	Plano de um experimento: limitações, ações que fecham lacunas (com esforço) e a execução mais próxima de estar pronta para cada uso.
`assess_experiment`	função	`(runs) -> dict[str, Any]`	Avalia um experimento inteiro: melhor execução, contagem por letra e consistência entre execuções.
`curate`	função	`(store, path_json=None, path_md=None) -> dict[str, Any]`	Avalia todas as execuções e experimentos e escreve `curation.json` e `CURATION.md` na pasta do Lab.

pinneapple_lab.dataset

Datasets das execuções: amostras gravadas em shards `.npz` com índice `samples.jsonl` e `card.json` (esquema, unidades, proveniência, checksum). [sem teste direto]

`pinneapple_lab/dataset.py` · 145 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`DatasetWriter`	classe	`(folder, *, name, description='', units=None, shard_size=256, license='', provenance=None)`	Escreve amostras de um dataset em shards, com bufferização, e gera o cartão descritivo ao fechar.
`read_dataset`	função	`(folder) -> Iterator[dict[str, Any]]`	Itera sobre as amostras de uma pasta de dataset escrita por `DatasetWriter`.

pinneapple_lab.experiments

Pacote dos experimentos embutidos; cada módulo registra seus experimentos ao ser importado. [sem teste direto]

`pinneapple_lab/experiments/__init__.py` · 42 linhas · **sem teste direto**

pinneapple_lab.experiments.accretion

Experimentos de hidrodinâmica de buraco negro sobre `pinneapple_physics.blackhole`: acreção de Bondi e um toro acretando. [sem teste direto]

`pinneapple_lab/experiments/accretion.py` · 126 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`BondiAccretion`	classe	`()`	Experimento `bondi_accretion`: o solver hidro, iniciado na solução exata de Bondi (potencial de Paczynski-Wiita), deve mantê-la estacionária.
`AccretionFlowRun`	classe	`()`	Experimento `accretion_flow`: toro quente acretando num buraco negro de Schwarzschild, com filme de densidade, taxa de acreção e balanços; os quadros viram dataset.

pinneapple_lab.experiments.blackhole_forecast

Experimento de previsão do "tempo" de buraco negro (Duarte, Nemmen & Navarro 2022) que embrulha `examples/black_hole_weather`; precisa do checkout do repositório. [sem teste direto]

`pinneapple_lab/experiments/blackhole_forecast.py` · 86 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`BlackHoleForecast`	classe	`()`	Experimento `bh_forecast`: U-Net que prevê a densidade do fluxo de acreção, avaliada por horizonte contra persistência.

pinneapple_lab.experiments.bricks

Construções de blocos encaixáveis (`pinneapple_design.bricks`) que se modificam até atingir uma meta (balanço, ponte, torre), verificadas por equilíbrio estático. [sem teste direto]

pinneapple_lab/experiments/bricks.py · 97 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`BricksExperiment`	classe	`()`	Experimento `bricks`: verifica a física contra formas fechadas e depois muta a construção até cumprir a meta.
`make_goal`	função	`(case)`	Devolve a meta (`CantileverGoal`, `BridgeGoal` ou `TowerGoal`) de um caso pelo nome.

pinneapple_lab.experiments.calculix_cases

Casos de CalculiX via `pinneapple_simulation.external_solvers.calculix.study` (placa com furo, suporte em L, modal de viga...) contra referências em forma fechada. Depende de CalculiX/gmsh. [sem teste direto]

pinneapple_lab/experiments/calculix_cases.py · 283 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`CalculixCase`	classe	`()`	Experimento `calculix_case`: roda um caso de CalculiX e compara tensão, convergência de malha e equilíbrio com fórmulas fechadas.

pinneapple_lab.experiments.car

Aerodinâmica de um perfil 2-D de carro paramétrico em túnel de vento virtual (LBM D2Q9 com LES) e um surrogate de IA física treinado nessas execuções.

pinneapple_lab/experiments/car.py · 438 linhas · testes: 20

nome	tipo	assinatura	o que faz
`CarLBM`	classe	`()`	Experimento `car_lbm`: um desenho de carro no túnel virtual, com arrasto e sustentação, campos médios e filme de vorticidade; gera dataset.
`CarSurrogate`	classe	`()`	Experimento `car_surrogate`: treina um FNO (forma -> escoamento médio) e um ensemble de MLPs (parâmetros -> Cd, Cl) sobre as execuções `car_lbm`.
`simulate_car`	função	`(shape, *, length_cells=64, Re=500.0, u_in=0.08, Cs=0.1, steps=8000, save_every=50, keep_…)`	Roda o túnel de vento virtual para um perfil (ou máscara sólida) e devolve campos, históricos e coeficientes.

pinneapple_lab.experiments.cylinder

Escoamento em torno de cilindro por lattice-Boltzmann: regime, número de Strouhal e imagens de vorticidade rotuladas. [sem teste direto]

pinneapple_lab/experiments/cylinder.py · 61 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`CylinderLBM`	classe	`()`	Experimento `cylinder_lbm`: canal 2-D com cilindro no solver D2Q9, medindo regime e Strouhal pelo espectro refinado.

pinneapple_lab.experiments.discovery

Descoberta de leis físicas a partir de dados: Kepler com dados orbitais reais, pêndulo filmado, Lorenz no estilo AI-Lorenz e oscilador, com SINDy e ELM. [sem teste direto]

pinneapple_lab/experiments/discovery.py · 548 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`KeplerLaw`	classe	`()`	Experimento `kepler_law`: regressão esparsa sobre leis de potência redescobre a terceira lei de Kepler a partir de dados orbitais.

nome	tipo	assinatura	o que faz
PendulumVideo	classe	()	Experimento pendulum_video: lei -> vídeo -> lei -> vídeo; mede o ângulo nos quadros, descobre a equação por SINDy fraco e re-simula.
LorenzDiscovery	classe	()	Experimento lorenz_discovery: ajusta observações ruidosas de Lorenz por janelas com ELM e descobre as equações.
OscillatorDiscovery	classe	()	Experimento oscillator_discovery: usa trajetórias do oscilador já no banco e recupera ζ e ω por SINDy.
elm_fit	função	(t, y, n_features=300, ridge=1e-08, seed=0, scale=None)	Ajuste por mínimos quadrados com features tanh aleatórias fixas (extreme learning machine), com derivadas primeira e segunda.
windowed_elm	função	(t, y, window=200, overlap=40, **kw)	Ajustes ELM em janelas sobrepostas para registros longos e caóticos; devolve y e y' costurados.
weak_form_2nd_order	função	(t, u, du_hat, library, width, n_test, p=4)	Monta a forma fraca de u'' = f(u, u') com funções-teste, evitando derivar os dados (SINDy fraco).
sindy	função	(Theta, target, names, threshold=None, max_terms=3, min_contribution=0.03)	Seleção esparsa de lei: testa todos os subconjuntos de até max_terms termos e fica com o de menor BIC.

pinneapple_lab.experiments.em_pinn

PINNs nos presets de eletromagnetismo com solução fechada (em_*), resolvidos com pp.solve(problem, "pinn") e medidos com pp.compare. [sem teste direto]

pinneapple_lab/experiments/em_pinn.py · 109 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
EMPinn	classe	()	Experimento em_pinn: treina a PINN de um caso em_* e mede o erro relativo contra o campo exato.

pinneapple_lab.experiments.examples

Transforma cada script de examples/ em experimento do Lab: roda como um usuário e coleta figuras, JSON, arrays, saída de console e código.

pinneapple_lab/experiments/examples.py · 290 linhas · testes: 20

nome	tipo	assinatura	o que faz
ExampleScript	classe	()	Experimento example_script: roda um script de exemplo e guarda o que ele produziu como figuras, métricas, dataset artifacts e código.
discover_examples	função	(repo=REPO) -> list[dict[str, str]]	Lista os scripts executáveis de examples/**/*.py, sem helpers e pastas de rascunho, agrupados por pasta.

pinneapple_lab.experiments.landing

Importa como execuções do Lab os casos da landing page que vivem em repositórios companheiros públicos (PINNeAPPle-Benchmark, PINNeAPPle-Climate). [sem teste direto]

pinneapple_lab/experiments/landing.py · 292 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
BenchmarkCase	classe	()	Experimento benchmark_case: importa um experimento finalizado de repositório companheiro e reconfere a afirmação principal com o results.json dele.
source_root	função	(key, lab_root=None) -> str | None	Acha o checkout local de um repositório companheiro (variável de ambiente, pasta irmã ou clone raso).

pinneapple_lab.experiments.manifest_import

Experimento manifest: traz um experiment.json para o Lab como execução, para ser curado A-F como qualquer item. [sem teste direto]

pinneapple_lab/experiments/manifest_import.py · 45 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ManifestImport`	classe	`()`	Experimento `manifest`: grava parâmetros, métricas, checagens e cópias de figuras de um manifesto; falha se o manifesto registra erro.

`pinneapple_lab.experiments.ode`

Oscilador harmônico amortecido: integradores (Euler, Euler simplético, RK4) contra a solução exata. [sem teste direto]

`pinneapple_lab/experiments/ode.py` · 95 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Oscillator`	classe	`()`	Experimento `oscillator`: integra x'' + 2ζωx' + ω²x = 0 e compara com a solução exata; gera dataset de trajetórias.
`exact`	função	`(t, zeta, omega, x0=1.0, v0=0.0)`	Solução exata do oscilador amortecido nos casos sub, crítico e superamortecido.
`integrate`	função	`(method, zeta, omega, dt, t_end, x0=1.0, v0=0.0)`	Integra o oscilador com o método escolhido (`euler`, `symplectic`, `rk4`) e passo fixo.

`pinneapple_lab.experiments.particles`

Suspensão de sólidos em tanque agitado: partículas lagrangianas (DEM, arrasto, empuxo, dispersão) num escoamento analítico, renderizadas como vídeo de processo. [sem teste direto]

`pinneapple_lab/experiments/particles.py` · 156 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ParticleSuspension`	classe	`()`	Experimento `particle_suspension`: simula a suspensão e confere velocidade de sedimentação e rotação de suspensão contra Schiller-Naumann e Zwietering.

`pinneapple_lab.experiments.pde`

Experimentos de EDP resolvidos com os solvers da biblioteca e conferidos contra soluções exatas. [sem teste direto]

`pinneapple_lab/experiments/pde.py` · 49 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`HeatXTFC`	classe	`()`	Experimento `heat_xtfc`: equação do calor 1-D resolvida por X-TFC contra exp(-α π² t) sin(π x).

`pinneapple_lab.experiments.pipes`

Escoamento interno em tubos e misturadores estáticos com OpenFOAM (`internal_flow`): Hagen-Poiseuille, curva de 90 graus e misturador Kenics. Depende de OpenFOAM. [sem teste direto]

`pinneapple_lab/experiments/pipes.py` · 307 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`PipeFlow`	classe	`()`	Experimento `pipe_flow`: roda um caso do solver de tubo e compara fator de atrito e perda com 64/Re, Colebrook e Ito.

`pinneapple_lab.experiments.repo_results`

Importa como execuções validadas os resultados já produzidos por scripts do repositório, reconferindo-os contra sua referência. [sem teste direto]

`pinneapple_lab/experiments/repo_results.py` · 317 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`RepoResults`	classe	`()`	Experimento `repo_results`: lê os artefatos de uma fonte (`source`), reconfere e grava métricas, figuras e datasets.

`pinneapple_lab.experiments.repo_sources`

Mais fontes para `repo_results` (máquinas elétricas, aerofólio, CFD de avião, rover lunar, otimização topológica, modelo de clima etc.), cada uma transformando números versionados em checagens. [sem teste direto]

`pinneapple_lab/experiments/repo_sources.py` · 636 linhas · **sem teste direto**

`pinneapple_lab.experiments.rom`

Modelos de ordem reduzida (`pinneapple_neural.architectures.rom`) sobre os solvers do Lab: POD, DMD e Operator Inference na esteira de cilindro e ROM paramétrico de viga FEM. [sem teste direto]

`pinneapple_lab/experiments/rom.py` · 399 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ROMStudy`	classe	`()`	Experimento `rom_study`: roda um caso de ROM e confere energia dos modos POD e previsão contra persistência.
`cylinder_wake`	função	`(Re, steps, save_every, D=20)`	Gera snapshots de velocidade do escoamento em torno de cilindro (LBM D2Q9) em unidades de rede.
`cantilever`	função	`(H, W, angle_deg, *, L=2.0, mesh=(30, 3, 6), F=10000.0)`	Resolve por FEM sólido 3-D uma viga engastada de aço com carga na face livre e devolve o `FEMResult`.

`pinneapple_lab.experiments.structures`

Mecânica estrutural 3-D com o solver `solid_fem` (hexaedros de 8 nós com modos incompatíveis), conferida contra teoria de vigas e torção e contra CalculiX. [sem teste direto]

`pinneapple_lab/experiments/structures.py` · 225 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`SolidFEMExperiment`	classe	`()`	Experimento `solid_fem`: roda um caso (balanço, biapoiada, torção...) e compara flecha, tensão e taxa de torção com fórmulas fechadas e CalculiX.
`build`	função	`(case, refine=1)`	Monta malha, modelo e referências em forma fechada de um caso.

`pinneapple_lab.experiments.vehicles3d`

Veículos 3-D (carro e foguete) em OpenFOAM com RANS, renders no Blender e visualizador three.js. Depende de OpenFOAM e Blender. [sem teste direto]

`pinneapple_lab/experiments/vehicles3d.py` · 162 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`VehicleCFD`	classe	`()`	Experimento `vehicle_cfd`: gera o veículo, roda RANS em meio modelo e confere coeficientes contra faixas de referência ou estabilidade de Barrowman.

`pinneapple_lab.experiments.wear`

Desgaste por deslizamento de barras de vários materiais (`pinneapple_physics.tribology`), com Archard e pressão redistribuída. [sem teste direto]

`pinneapple_lab/experiments/wear.py` · 238 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`BarWear`	classe	`()`	Experimento `bar_wear`: desliza uma barra coroada até assentar e confere volume desgastado, contato inicial e taxa estacionária.
`BarWearRanking`	classe	`()`	Experimento `bar_wear_ranking`: compara todas as execuções `bar_wear` e confere o ranking dos materiais pelo coeficiente de Archard.

pinneapple_lab.explorer

Página de experimentos: cada item com selo A-F, cartão de confiança, métricas, próximos passos, figuras, vídeos e links, servida em `/experiments` ou exportada como HTML estático.

`pinneapple_lab/explorer.py` · 387 linhas · testes: 6

nome	tipo	assinatura	o que faz
`proper`	função	`(title) -> str`	Gera um título a partir de uma chave, com siglas em maiúsculas.
`collect`	função	`(store, examples=None) -> dict[str, Any]`	Reúne todos os experimentos da página (curadoria e manifestos), contagens por letra, facetas de filtro e lista de cobertura.
`coverage`	função	`(data) -> list[dict[str, Any]]`	Tabela experimento -> curado sim/não (e letra), uma linha por entrada.
`coverage_md`	função	`(data) -> str`	Gera em Markdown o relatório de cobertura da curadoria.
`filter_entries`	função	`(entries, *, letter=None, domain=None, method=None, ready=False, q=None, sort='letter') -…)`	Aplica os filtros da página (letra, domínio, método, pronto, busca, ordenação), também usados por `/api/experiments`.
`media_urls`	função	`(data, mode='server') -> dict[str, Any]`	Preenche as URLs de mídia, miniatura e selo, como rotas do servidor ou caminhos relativos do export estático.
`render`	função	`(data) -> str`	Renderiza a página com os dados embutidos no template.
`media_path`	função	`(lab_root, entry, item) -> str | None`	Resolve o arquivo por trás de um item de mídia (arquivo de execução ou do repositório).
`stamped_jpeg`	função	`(path, key, cur, width=1280) -> bytes | None`	Devolve a figura carimbada com o selo do item como JPEG de largura limitada.
`seal_png`	função	`(key, cur, lang='en') -> bytes`	Renderiza o selo de um item como bytes PNG.
`write_static`	função	`(store, out_dir, *, examples=None, media=True) -> str`	Escreve a mesma página como arquivos estáticos (`index.html` e `media/`).

pinneapple_lab.html

Catálogo navegável do Lab como uma página HTML autocontida, com dados e miniaturas embutidos.

`pinneapple_lab/html.py` · 197 linhas · testes: 14

nome	tipo	assinatura	o que faz
`thumbnail_bytes`	função	`(path, width=360) -> bytes | None`	Miniatura JPEG de uma imagem (primeiro quadro de GIF), ou None se não der para ler.
`showcase_score`	função	`(path) -> int`	Pontua o nome de uma figura por palavras-chave para escolher a mais vistosa.
`collect`	função	`(store, *, thumbs_per_run=1, max_runs_per_experiment=400, thumbs='embed') -> dict[str, An…)`	Reúne tudo o que o catálogo mostra, com miniaturas embutidas ou apontando para a rota `thumb/` do servidor.
`write_html`	função	`(store, path=None, *, standalone=True, **kw) -> str`	Escreve a página do catálogo, como documento completo ou só o corpo.
`render_page`	função	`(data, *, standalone=True) -> str`	Renderiza a página do catálogo com os dados de `collect` embutidos.
`write_site`	função	`(store, out_dir) -> str`	Escreve um catálogo público autocontido para hospedagem estática (GitHub Pages), sem caminhos locais.

pinneapple_lab.improve

Comando `improve`: imprime lacunas e ações para a próxima letra e, com `--run`, executa só as ações registradas como executáveis e recura. [sem teste direto]

`pinneapple_lab/improve.py` · 70 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`runnable_actions`	função	`(gaps) -> list[Any]`	Filtra as ações registradas e executáveis das lacunas, sem repetição.
`improve`	função	`(item, store, *, run=False, lang='en', log=print) -> int`	Imprime lacunas e ações de um item; com `run=True` roda os comandos do Lab e recura; devolve 0 ou 1.

pinneapple_lab.manifest

Manifestos de experimento (`experiment.json`, formato `pinneapple.experiment/1`): conversões de execuções, soluções, comparações e fluxos, validação, varredura do repositório e portão de CI. (A marca de placeholder é falso positivo: o docstring diz que métrica nunca é placeholder.)

`pinneapple_lab/manifest.py` · 650 linhas · testes: 29 · palavras-chave no código (heurística; o texto acima diz o status real): placeholder/stub citado no docstring

nome	tipo	assinatura	o que faz
`Manifest`	classe	`(id, title, description, domain, methods, modules, params, scripts, metrics, checks, file…)`	Dataclass do manifesto: id, título, domínio, métodos, módulos, parâmetros, scripts, métricas, checagens, arquivos e proveniência.
`save`	função	`(m, path) -> str`	Grava o manifesto em um caminho (uma pasta recebe `experiment.json`).
`load`	função	`(path) -> Manifest`	Lê o manifesto de um `experiment.json`, de uma pasta que o contém ou de uma pasta de execução do Lab.
`validate`	função	`(m, base=None) -> list[str]`	Lista os problemas de um manifesto (esquema, id, título, métricas finitas com `how`, arquivos existentes).
`from_run_dir`	função	`(d) -> Manifest`	Converte uma pasta de execução do Lab em manifesto.
`write_for_run`	função	`(d) -> str | None`	Escreve `experiment.json` numa pasta de execução terminada; nunca levanta exceção.
`from_workflow_run`	função	`(runner, run_id) -> Manifest`	Converte uma execução de `pinneapple_core.workflows` em manifesto.
`from_solution`	função	`(sol, title=None) -> Manifest`	Converte um resultado de `pp.solve` em manifesto (usado por `Solution.to_experiment`).
`from_comparison`	função	`(cmp, title=None) -> Manifest`	Converte uma tabela de `pp.compare` em manifesto, com todas as métricas de cada método.
`from_flow_run`	função	`(run, out_dir=None) -> Manifest`	Gancho do Flow Studio: converte uma execução de fluxo terminada em manifesto e, opcionalmente, grava.
`domain_of`	função	`(name, tags=()) -> str`	Deduz o domínio (por padrões de texto) a partir do nome e das tags; padrão `general`.
`methods_of`	função	`(text) -> list[str]`	Lista rótulos de métodos cujos nomes aparecem no texto, como faceta de filtro.
`manifest_from_dir`	função	`(folder, lab_curation=None) -> Manifest`	Monta um manifesto com o que uma pasta de exemplo já contém (README, docstring, imports, figuras, resultados).
`lab_items_for`	função	`(rel, cur) -> list[str]`	Lista os itens do Lab que avaliam uma pasta de exemplo, melhor letra primeiro.
`from_results`	função	`(folder, rules=None) -> Manifest`	Manifesto da pasta acrescido das métricas e checagens extraídas dos resultados versionados por `results_rules`.
`example_dirs`	função	`(root=None) -> list[str]`	Lista as pastas de `examples/*` que precisam ter manifesto.
`scan`	função	`(roots=None, *, lab=None) -> list[tuple[str, Manifest]]`	Encontra todos os manifestos sob as raízes dadas e todas as execuções do Lab.
`check_repo`	função	`(examples=None, lab=None) -> list[str]`	Portão de CI: uma linha por problema nos manifestos dos exemplos e nas execuções do Lab; vazio se tudo ok.

nome	tipo	assinatura	o que faz
`scaffold`	função	`(name, root=None, *, title='', domain='') -> str`	Cria `examples/<name>/` com `run.py` ligado a `pinneapple_lab.experiment`, README e manifesto inicial; recusa pasta existente.

pinneapple_lab.next_steps

Registro de tipos de lacuna e de ações: diz o que falta para a próxima letra e qual ação do PINNeAPPle fecha cada lacuna.

`pinneapple_lab/next_steps.py` · 399 linhas · testes: 31

nome	tipo	assinatura	o que faz
`Action`	classe	`(id, kind, target, command, text, text_pt, params, runnable, argv)`	Ação sugerida: chamada Python ou comando CLI, com experimento e parâmetros preenchidos e textos em inglês e português.
`Gap`	classe	`(type, to, stage, why, why_pt, short, short_pt, check, measured, actions)`	Lacuna entre o item e a próxima letra: tipo, estágio do cartão de confiança, porquê, checagem relacionada e ações.
`measured`	função	`(check) -> dict[str, Any] | None`	Extrai o valor medido de uma checagem e seu limite, ou None se não houver número.
`measured_text`	função	`(m, lang='en') -> str`	Formata o valor medido contra o limite como texto em inglês ou português.
`context_for`	função	`(experiment, params, run_dir, tags, is_ml, cls_params) -> dict[str, Any]`	Monta o contexto (resolução, semente, se é ML etc.) usado para preencher as ações de um item.
`make_action`	função	`(aid, ctx) -> Action | None`	Preenche uma ação registrada para o item, ou None se a condição dela não vale.
`make_gap`	função	`(gtype, to, ctx, *, check=None, why=None, meas=None) -> Gap`	Cria uma lacuna de tipo registrado com as ações preenchidas; `KeyError` para tipo desconhecido.
`group`	função	`(gaps) -> list[dict[str, Any]]`	Agrupa lacunas que compartilham as mesmas ações, na ordem em que aparecem.
`explain`	função	`(gaps, lang='en', title='') -> str`	Escreve as lacunas como texto, com porquê, valor medido e ações agrupadas.

pinneapple_lab.reports

Relatórios do Lab (HTML, Markdown ou PDF) de um item ou de vários filtrados, tudo tirado do registro e da curadoria. PDF precisa de `reportlab`.

`pinneapple_lab/reports.py` · 428 linhas · testes: 20

nome	tipo	assinatura	o que faz
`select`	função	`(cur, *, items=None, experiment=None, tier=None, use=None, ready=None, tag=None, tags_of=…)`	Seleciona chaves de itens da curadoria pelos filtros; `use` ordena por distância a esse uso.
`gather`	função	`(store, cur, key, n_figures=3) -> dict[str, Any]`	Junta tudo o que o relatório diz de um item.
`render_html`	função	`(store, groups, title, filters='') -> str`	Renderiza o relatório como HTML.
`render_md`	função	`(groups, title, filters='') -> str`	Renderiza o relatório como Markdown com tabela de itens.
`render_pdf`	função	`(groups, title, filters='') -> bytes`	Renderiza o relatório como PDF com `reportlab`.
`write_report`	função	`(store, out, *, keys=None, title=None, fmt=None, filters='', cur=None, **select_kw) -> str`	Seleciona os itens e escreve o relatório no formato indicado pela extensão ou por `fmt`.

pinneapple_lab.results_rules

Regras (`Metric`, `Flag`, `Bound`, `CheckList`) que transformam resultados versionados de um exemplo em métricas e checagens de manifesto, com limites citados como `arquivo:linha`. (A marca de placeholder é falso positivo.)

`pinneapple_lab/results_rules.py` · 219 linhas · testes: 14 · palavras-chave no código (heurística; o texto acima diz o status real): placeholder/stub citado no docstring

nome	tipo	assinatura	o que faz
`Metric`	classe	`(file, path, name, unit, how)`	Regra que lê uma métrica de um arquivo de resultados por caminho, com nome, unidade e `how`.
`Flag`	classe	`(file, path, name, kind, detail, invert)`	Regra de checagem a partir de um veredito booleano que o próprio estudo gravou.
`Bound`	classe	`(file, path, name, kind, literal, source, op, absolute, unit)`	Regra que compara um valor medido com o limite de aceitação citado no teste do estudo (`arquivo:linha`).
`CheckList`	classe	`(file, path, name, kind, value_key, reference_key, tolerance_key, limit_key, passed_key)`	Regra que lê uma lista de registros de checagem gravados pelo estudo (valor, referência, tolerância ou limite).
`extract`	função	`(folder, rules) -> tuple[list[dict], list[dict]]`	Aplica as regras e devolve (métricas, checagens); regra sem arquivo ou caminho não gera nada.
`check_citations`	função	`(repo) -> list[str]`	Confere que cada literal de `Bound` aparece na linha citada do arquivo citado.

pinneapple_lab.runner

Executa um experimento ou varreduras por grade ou amostragem (hipercubo latino), com cache por id e paralelismo por processos. [sem teste direto; coberto pelos testes do pacote]

`pinneapple_lab/runner.py` · 243 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`RunResult`	classe	`(run_id, experiment, status, dir, metrics, seconds, error)`	Resultado de uma execução: id, experimento, status (inclusive `cached:<status>`), pasta, métricas, tempo e erro.
`run`	função	`(experiment, params=None, *, root=None, force=False, tags=None, raise_errors=False) -> Ru…)`	Roda um experimento com parâmetros sobre os padrões; não repete execução já concluída, salvo `force`.
`sweep`	função	`(experiment, *, grid=None, samples=None, space=None, fixed=None, method='lhs', seed=0, n_…)`	Roda um experimento sobre uma grade e/ou amostras aleatórias do espaço, opcionalmente em paralelo (`n_jobs`).

pinneapple_lab.seal

Selo de qualidade PINNeAPPle: badge com a letra dada pela curadoria (nunca escolhida à mão), carimbado em figuras, renders, GIFs e vídeos. Vídeos usam ffmpeg.

`pinneapple_lab/seal.py` · 829 linhas · testes: 39

nome	tipo	assinatura	o que faz
`SealError`	classe	`()`	Erro quando o item não pode receber selo porque sua letra está fora das letras pedidas.
`Seal`	classe	`(item, tier, title, reason, reason_kind, curated, experiment, run_id, hint, reason_pt, hi…)`	Dados de um selo: item, letra, título, motivo (manchete da curadoria), tipo do motivo e execução.
`load_curation`	função	`(lab=None) -> dict[str, Any]`	Carrega o registro de curadoria de uma pasta, caminho, `LabStore`, dict ou do Lab padrão.
`parse_tiers`	função	`(tiers) -> tuple[str, ...]`	Converte `"A,B"`, lista ou None numa tupla de letras válidas.
`items`	função	`(lab=None, tiers=SEALABLE) -> list[str]`	Lista as chaves de itens curados com uma das letras pedidas.
`seal_for`	função	`(item, lab=None, tiers=None) -> Seal`	Monta o selo de um item curado; `KeyError` se desconhecido e `SealError` se a letra estiver fora do filtro.
`run_dir`	função	`(seal, lab=None) -> str`	Devolve a pasta da execução a que o selo se refere.
`draw_text`	função	`(draw, xy, text, font, fill, anchor='ls') -> None`	Desenha texto com uma marca inicial (✓, ◐, ✗ ou ↑) na fonte DejaVu Sans.
`render_seal`	função	`(seal, *, height=96, fmt='png', reason=True, compact=None, opacity=1.0, lang='en', next_s…)`	Renderiza o badge como imagem PIL RGBA ou SVG, com forma compacta em tamanhos pequenos e idioma en/pt.
`contrast_ratio`	função	`(fg, bg=BADGE_BG) -> float`	Razão de contraste WCAG 2 entre duas cores `#rrggbb`.

nome	tipo	assinatura	o que faz
save_seal	função	(seal, path, **kw) -> str	Grava o badge como SVG ou imagem raster, conforme a extensão.
pick_corner	função	(frames, w, h, margin, avoid=()) -> str	Escolhe o canto onde o badge cobre menos conteúdo, evitando caixas proibidas.
corner_scores	função	(frames, w, h, margin, avoid=()) -> dict[str, float]	Mede o quanto cada canto utilizável está ocupado.
stamp_image	função	(image, seal, *, corner='auto', size=0.24, opacity=0.95, margin=0.02, reason=True, avoid=…)	Carimba uma imagem PIL, array ou arquivo, no canto mais vazio ou numa faixa extra se todos estiverem ocupados.
stamp_figure	função	(fig, seal, *, corner='auto', size=0.24, opacity=0.95, margin=0.015, reason=True, extend=…)	Carimba uma figura matplotlib no lugar, evitando barras de cor.
sealed_savefig	função	(seal, **stamp_kw)	Context manager em que todo `Figure.savefig` carimba a figura antes de salvar.
stamp_video	função	(src, seal, out, *, corner='auto', size=0.24, opacity=0.95, margin=0.02, reason=True, ext…)	Carimba um GIF quadro a quadro ou um MP4/WebM/MOV via overlay do ffmpeg.
stamp_file	função	(path, seal, out=None, **kw) -> str	Carimba um arquivo de imagem, GIF ou vídeo e devolve o caminho de saída; recusa SVG e PDF.
stamp_all	função	(lab=None, out_dir='export/sealed', *, tiers=SEALABLE, only=None, lang='en', **kw) -> dic…)	Modo lote: carimba todas as figuras e vídeos dos itens nas letras pedidas, com selos SVG/PNG e manifesto.
legend_seals	função	(lang='en') -> list[Seal]	Um selo de exemplo por letra, com a linha que explica o significado da letra.
render_legend	função	(lang='en', height=72)	Renderiza a legenda dos seis selos, A a F, em fundo claro e escuro.

pinneapple_lab.server

Servidor HTTP só com biblioteca padrão: página do catálogo, arquivos das execuções, API JSON, miniaturas sob demanda e downloads de datasets.

`pinneapple_lab/server.py` · 382 linhas · testes: 26

nome	tipo	assinatura	o que faz
LabServer	classe	(root=None, *, refresh=30.0, user=None, password=None)	Catálogo e arquivos de uma pasta do Lab; `handler()` dá a classe que atende as requisições.
serve	função	(root=None, *, host='127.0.0.1', port=8093, refresh=30.0) -> None	Serve o catálogo até ser interrompido.

pinneapple_lab.showcase

Painel animado 1080 x 1350 com os melhores resultados do Lab (destaque, grade de itens A e fileira de B), gerado da curadoria como MP4, GIF e PNG.

`pinneapple_lab/showcase.py` · 589 linhas · testes: 3

nome	tipo	assinatura	o que faz
Tile	classe	(item, tier, figure, title, evidence, evidence_kind, seal, frame, lang)	Um bloco do painel: item, letra, figura, título, linha de evidência e selo.
evidence_line	função	(rec, lang='en', short=False) -> str	Escolhe a linha de evidência do bloco a partir da manchete da curadoria, no idioma pedido.
choose_tiles	função	(lab=None, picks=None, lang='en', n_grid=9, n_b=2) -> dict[str, Any]	Escolhe destaque, grade (A) e fileira B a partir da curadoria e das escolhas manuais `picks`.
fit_crop	função	(img, aspect, *, photo=None) -> tuple[tuple[int, int, int, int], bool]	Calcula o recorte de uma imagem para caber no bloco e se precisa de tarjas.
panel	função	(lab=None, *, lang='en', out_dir='docs/assets/showcase', picks=None, seconds=8.0, fps=15,…)	Renderiza o painel e devolve os arquivos escritos e os blocos usados.

pinneapple_lab.spec

Classe base `Experiment` e registro por nome; `Experiment.run` da base levanta `NotImplementedError` (deve ser sobrescrito). [sem teste direto]

`pinneapple_lab/spec.py` · 92 linhas · **sem teste direto** · palavras-chave no código (heurística; o texto acima diz o status real): levanta `NotImplementedError` em algum caminho

nome	tipo	assinatura	o que faz
`Experiment`	classe	`(name, version, description, tags, params, space, code_files, references, limitations, ca…)`	Classe base de experimento: defina `name`, `params` e `run(ctx)`; a versão entra no id da execução.
`register`	função	`(cls) -> type[Experiment]`	Decorador de classe que registra o experimento por nome para o runner e a CLI.
`get`	função	`(name) -> type[Experiment]`	Busca um experimento pelo nome, carregando os embutidos ou importando `modulo:Classe`.
`available`	função	`() -> list[str]`	Lista os nomes de todos os experimentos registrados.

pinneapple_lab.store

Banco do Lab: pastas `runs/<experimento>/<run_id>` mais índice `index.sqlite`, consultas, tabelas, exportação de datasets e catálogos. [sem teste direto; coberto pelos testes do pacote]

`pinneapple_lab/store.py` · 285 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`LabStore`	classe	`(root=None)`	Banco de experimentos numa raiz: indexa execuções, consulta, monta tabelas, lista e exporta datasets e escreve catálogos.
`default_root`	função	`() -> str`	Raiz padrão do Lab: `$PINNEAPPLE_LAB` ou `./lab`.

pinneapple_lab.uq

Checagens de incerteza para o cartão de confiança: cobertura de intervalos preditivos, índice de convergência de malha (GCI) e espalhamento de ensemble.

`pinneapple_lab/uq.py` · 95 linhas · testes: 20

nome	tipo	assinatura	o que faz
`coverage`	função	`(y_true, mean, std, level=0.9) -> float`	Fração dos valores verdadeiros dentro do intervalo gaussiano central de nível dado.
`coverage_check`	função	`(ctx, name, y_true, mean, std, level=0.9, tol=0.07) -> float`	Grava a cobertura empírica em dados não vistos e checa se fica perto do nível nominal.
`gci`	função	`(values, h, order=None, safety=None) -> dict`	Índice de convergência de malha do valor mais fino, com ordem observada para três malhas (Celik et al. 2008).
`gci_check`	função	`(ctx, name, values, h, order=None, max_band=0.05) -> dict`	Grava a faixa GCI e o valor extrapolado de Richardson e checa se a faixa fica abaixo de `max_band`.
`ensemble_spread`	função	`(values) -> float`	Desvio padrão relativo de resultados repetidos.

Exemplos de uso reais do repositório

Executar, reaproveitar o cache e varrer parâmetros — `tests/test_lab.py`

```python
def test_run_records_everything_and_is_cached(tmp_path):
    root = str(tmp_path)
    r = run("_test_square", {"a": 2.0}, root=root)
    assert r.status == "completed" and r.metrics["y_max"] == 2.0
    ...
    again = run("_test_square", {"a": 2.0}, root=root)
    assert again.status == "cached:completed" and again.run_id == r.run_id
```

Embrulhar um script de exemplo com pinneapple_lab.experiment — examples/chain_of_whys/chain_of_whys.py

```
            with pinneapple_lab.experiment(
                    "examples/chain_of_whys", title="Chain of whys",
                    description="Engineering reasoning from physics: naive concept -> computed failure mode ->
 countermeasure "
                                "-> check again, for a roof, a footbridge, a power board and a pumped main.",
                    domain="design and geometry", methods=["closed-form checks", "FD beam", "3-D board FV", "MOC"],
                    modules=["pinneapple_reason"], out_dir=HERE, register=False,
                    links={"guide": "docs/guides/chain_of_whys.md", "script":
 "examples/chain_of_whys/chain_of_whys.py"}) as ex:
                for name in a.case or list(cases.CASES):
                    ...
                    ex.check(f"{name}: every requirement passes on the final version", chain.resolved, kind="physics",
                             detail=" -> ".join(f"{m}:{c}" for _, m, c in chain.path()))
```

Varredura paralela e linha de comando — tests/test_lab.py

```
            res = sweep("oscillator", grid={"zeta": [0.1, 0.4], "method": ["rk4", "symplectic"]}, n_jobs=2, root=root)
            assert all(r.ok for r in res)
            assert cli(["--root", root, "status"]) == 0
            assert cli(["--root", root, "run", "oscillator", "-p", "zeta=0.3"]) == 0
            assert cli(["--root", root, "report"]) == 0
```

Testes e status de validação

- Módulos com pelo menos um teste que os importa: 15 de 42.
- Funções de teste em arquivos que importam algum módulo do pacote: 96.

Limitações conhecidas

- Vários experimentos embutidos dependem de ferramentas externas: CalculiX e gmsh (`calculix_case`), OpenFOAM (`pipe_flow`, `vehicle_cfd`), Blender (renders de `vehicle_cfd`) e ffmpeg (carimbo de vídeos e `showcase`).
- `bh_forecast`, `example_script` e `repo_results` leem arquivos do repositório e só funcionam num checkout do código-fonte; `benchmark_case` precisa de repositórios companheiros (variável de ambiente, pasta irmã ou clone raso).
- A maioria dos módulos de `pinneapple_lab.experiments` não tem teste direto; `runner`, `store`, `context` e `dataset` são exercitados pelos testes que importam o pacote (`tests/test_lab.py`), não por testes próprios.
- A curadoria só classifica a evidência que a execução gravou: um experimento sem checagens fica no máximo em D, e as letras A exigem revisão humana para virar produto, artigo ou marketing.
- `Experiment.run` da classe base levanta `NotImplementedError`; cada experimento precisa sobrescrevê-lo.
- Relatórios em PDF precisam de `reportlab`; selos e miniaturas usam Pillow e matplotlib.
- A curadoria reutiliza o vocabulário de `pinneapple_veriphysics.applicability`, importado de forma opcional.

6. `pinneapple_llm`

Objetivo

O `pinneapple_llm` usa modelos de linguagem (LLMs) para rascunhar partes de um pipeline de IA física, sempre atrás de uma camada de verificação com base física. A regra de projeto, repetida nos docstrings, é que o LLM só escolhe e parametriza algo que o PINNeAPPle já implementa e testa: um preset de `ProblemSpec` de `pinneapple_physics` (`draft_problem`), um gerador de geometria de `pinneapple_design.geometry` (`draft_geometry`, `draft_mesh_recipe`, `draft_cadquery_template`) ou os campos de `DigitalTwinConfig` de `pinneapple_systems` (`draft_digital_twin`). Ele nunca escreve física, CAD ou gêmeo digital do zero, e cada nome e argumento devolvido é conferido contra o registro real.

O `PhysicsGuardrail` (`guardrail`) é o mecanismo anti-alucinação: recalcula sinais numéricos independentes (resíduo da EDP em pontos novos, sanidade dimensional dos parâmetros, balanço de conservação quando há forma fechada e erro contra dados de referência) e se recusa a chamar de confiável um resultado que não passa. O próprio docstring avisa que isso não garante que o resultado esteja certo. `research` busca artigos e repositórios reais no arXiv e GitHub (o LLM só resume), `org_knowledge` responde perguntas a partir de um índice de busca local de `pinneapple_analysis.retrieval`, e `agent_loop` roda um laço de agente que chama ferramentas reais de `pinneapple_worldmodel`.

Todas as chamadas passam por `_dispatch.call_llm`, que fala com Anthropic, OpenAI (ou servidor compatível) e Ollama local. `local_llm` e `conversation_store` dão um caminho totalmente local (Ollama mais um log SQLite das conversas), e `finetune` faz fine-tuning LoRA de um modelo local com esse log.

Dois subpacotes maiores completam o pacote. `qa` é um pipeline de garantia de qualidade em estágios A-G (especificação física tipada, geração, validação estática, execução em sandbox, validadores científicos, comparação com referência independente e reparo limitado), com relatório reprodutível e níveis Q0-Q4 dados pelo que de fato rodou. `workbench` é o Agent Workbench: uma equipe de agentes de código e engenharia rodando num modelo local, com app web (FastAPI), ferramentas confinadas a uma pasta, aprovações, banco SQLite/PostgreSQL e fine-tuning nas próprias sessões.

Números do inventário

módulos	linhas	classes públicas	funções públicas	testes em arquivos que importam o pacote	módulos sem teste direto
49	10.750	66	99	242 (20 arquivos)	28

Pasta de testes dedicada: `tests/pinneapple_llm` (4 funções de teste).

Submódulos

submódulo	o que faz	módulos	classes + funções	testes*
`_dispatch`	Despacho de uma chamada de LLM para Anthropic, OpenAI/compatível ou Ollama, com log opcional.	1	1	0
`agent_loop`	Laço de agente em vários passos em que o LLM escolhe, a cada turno, qual ferramenta real chamar.	1	3	15
`cad_draft`	Rascunho de geometria nova pelo LLM: receitas CSG de primitivas ou templates paramétricos CadQuery.	1	7	22
`catalog_match`	`resolve_problem`: pré-filtro por palavras-chave mais LLM para achar um preset de EDP, sem nunca levantar exceção.	1	2	4
`conversation_store`	Log SQLite de todas as chamadas de LLM, para auditoria e como dataset de fine-tuning.	1	2	0
`draft`	Rascunho de `ProblemSpec`: o LLM escolhe e parametriza um preset existente de `pinneapple_physics`.	1	5	23
`finetune`	Fine-tuning LoRA de um modelo causal local (transformers + peft) com os dados do log de conversas.	1	3	0
`geometry_draft`	Rascunho de geometria: o LLM escolhe um gerador existente de `pinneapple_design.geometry`.	1	2	0
`guardrail`	`PhysicsGuardrail`: checagens numéricas recalculadas que decidem se um resultado pode ser chamado de confiável.	1	3	60
`local_llm`	Provedor local Ollama: iniciar servidor, baixar modelo e fazer chamadas de chat.	1	3	0
`org_knowledge`	Respostas do LLM fundamentadas nos resultados reais de um índice de busca local.	1	2	4
`qa`	Pipeline de garantia de qualidade (estágios A-G e níveis Q0-Q4) para artefatos gerados por LLM.	23	80	48
`research`	Busca real de literatura no arXiv e GitHub, com resumo opcional pelo LLM.	1	2	0
`twin_draft`	Rascunho de configuração de gêmeo digital e montagem de um gêmeo 3-D ao vivo.	1	3	0
`workbench`	Agent Workbench: equipe de agentes de código e engenharia em modelo local, com app web e fine-tuning.	12	47	37

* maior número de testes que importam um módulo do submódulo.

Módulos e API pública

`pinneapple_llm`

Ponto de entrada do pacote: reexporta as funções de rascunho, o `PhysicsGuardrail`, a busca de literatura e o caminho local.

`pinneapple_llm/__init__.py` · 61 linhas · testes: 128

`pinneapple_llm._dispatch`

Despacho de chamadas de LLM independente de provedor (`anthropic`, `openai`, `openai_compatible`, `ollama`), com log opcional por chamada num `ConversationStore`. [sem teste direto]

`pinneapple_llm/_dispatch.py` · 85 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`call_llm`	função	`(prompt, *, provider='anthropic', model=None, api_key=None, system='', json_mode=False, m…)`	Envia um prompt ao provedor pedido e devolve o texto bruto da resposta.

`pinneapple_llm.agent_loop`

Laço de agente em vários passos: o LLM decide a cada turno qual ferramenta real chamar (por padrão `PhysicsToolRegistry` de `pinneapple_worldmodel`), observa o resultado e segue.

`pinneapple_llm/agent_loop.py` · 331 linhas · testes: 15

nome	tipo	assinatura	o que faz
`AgentStep`	classe	`(tool_name, tool_args, tool_result, error, llm_reasoning)`	Um turno do laço: ferramenta escolhida, argumentos, resultado real (ou erro) e o raciocínio declarado pelo LLM.
`AgentLoopResult`	classe	`(steps, final_answer, stopped_reason)`	Rastro completo de uma execução de `run_agent_loop`.
`run_agent_loop`	função	`(goal, tool_registry=None, *, llm_provider='anthropic', model=None, max_steps=8, llm_call…)`	Roda o laço rumo a um objetivo: monta o prompt com histórico e ferramentas disponíveis, chama a ferramenta escolhida ou encerra com resposta final.

`pinneapple_llm.cad_draft`

Geração de geometria nova pelo LLM a partir de blocos já existentes: receitas CSG de primitivas de malha ou templates paramétricos CadQuery (sólido B-rep exportável em STEP).

`pinneapple_llm/cad_draft.py` · 633 linhas · testes: 22

nome	tipo	assinatura	o que faz
`CadRecipeResult`	classe	`(kind, recipe, name, params, reasoning, raw_response)`	Resultado de um rascunho de CAD: tipo (`mesh_recipe` ou `cadquery_template`), receita validada ou nome do template e parâmetros.
`draft_mesh_recipe`	função	`(description, *, provider='anthropic', model=None, api_key=None, conversation_store=None)…)`	Pede ao LLM uma receita de geometria (possivelmente combinada por CSG) para uma descrição em linguagem natural; não constrói a malha.
`draft_cadquery_template`	função	`(description, *, provider='anthropic', model=None, api_key=None, conversation_store=None)…)`	Pede ao LLM para escolher e parametrizar um template CadQuery paramétrico; requer `cadquery` opcional.
`build_recipe`	função	`(result)`	Constrói a geometria descrita: `MeshData` para receita de malha ou objeto CadQuery para template.
`export_recipe`	função	`(result, out_path, *, fmt=None) -> str`	Exporta a geometria construída para arquivo (STL via `trimesh`; STEP se `cadquery` estiver instalado).
`measure_part`	função	`(result) -> Dict[str, Any]`	Constrói uma receita de malha e mede volume, extensão, estanqueidade e se havia motor booleano disponível.

nome	tipo	assinatura	o que faz
`draft_part`	função	`(description, *, tier='quick', provider='ollama', model=None, api_key=None, max_attempts=…)`	Rascunha uma peça construível, tentando de novo quando o rascunho é recusado; nível `quick` (CSG) ou `engineering` (CadQuery).

pinneapple_llm.catalog_match

Frente em dois estágios para `draft_problem`: pré-filtro por sobreposição de palavras-chave e depois o LLM; se o LLM falhar, devolve o melhor casamento por palavras. Nunca levanta exceção.

`pinneapple_llm/catalog_match.py` · 128 linhas · testes: 4

nome	tipo	assinatura	o que faz
`ResolveResult`	classe	`(preset, kwargs, reasoning, used_llm, shortlist)`	Resultado da resolução: preset, kwargs, raciocínio, se o LLM foi usado e a lista curta.
`resolve_problem`	função	`(description, *, provider='anthropic', model=None, api_key=None, conversation_store=None,…)`	Resolve uma descrição para um preset de EDP registrado, com fallback determinístico por palavras-chave.

pinneapple_llm.conversation_store

Log SQLite (só biblioteca padrão) de cada chamada de LLM do pacote, para auditoria e como dataset de `finetune`. [sem teste direto]

`pinneapple_llm/conversation_store.py` · 120 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ConversationRecord`	classe	`(id, timestamp, module, provider, model, system_prompt, user_prompt, response, metadata)`	Um registro de conversa: horário, módulo, provedor, modelo, prompts, resposta e metadados.
`ConversationStore`	classe	`(db_path=_DEFAULT_DB_PATH)`	Invólucro fino de SQLite que grava, consulta e exporta conversas como JSONL.

pinneapple_llm.draft

Rascunho de `ProblemSpec` restrito a escolher e parametrizar um preset existente de `pinneapple_physics.list_presets()`; nomes e argumentos inventados são rejeitados.

`pinneapple_llm/draft.py` · 231 linhas · testes: 23

nome	tipo	assinatura	o que faz
`DraftResult`	classe	`(preset, kwargs, reasoning, raw_response)`	Resultado do rascunho: preset escolhido, kwargs, raciocínio e resposta bruta.
`DraftRejected`	classe	`(message, attempts)`	Exceção quando nenhuma tentativa produziu rascunho aceitável; `.attempts` diz por que cada uma foi recusada.
`draft_problem`	função	`(description, *, provider='anthropic', model=None, api_key=None, conversation_store=None)…)`	Pede ao LLM para escolher e parametrizar um preset para uma descrição em linguagem natural; não constrói o `ProblemSpec`.
`check_kwargs_in_bounds`	função	`(kwargs, bounds) -> Dict[str, float]`	Converte kwargs rascunhados para floats finitos e confere cada um contra limites (lo, hi).
`draft_preset_kwargs`	função	`(description, expected_preset, *, bounds=None, provider='ollama', model=None, api_key=Non…)`	Repete `draft_problem` até obter o preset esperado com kwargs dentro dos limites; levanta `DraftRejected` ao esgotar as tentativas.

pinneapple_llm.finetune

Fine-tuning LoRA de um modelo causal local no formato Hugging Face (transformers + peft, extra `finetune`) com os pares registrados no log; não faz fine-tuning direto de modelo Ollama. [sem teste direto]

`pinneapple_llm/finetune.py` · 116 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`FinetuneConfig`	classe	`(base_model, output_dir, lora_r, lora_alpha, lora_dropout, learning_rate, num_train_epoch…)`	Configuração do fine-tuning: modelo base, pasta de saída e hiperparâmetros de LoRA e treino.
`prepare_dataset`	função	`(store, out_jsonl, *, module=None) -> str`	Exporta os pares (prompt, resposta) do log para JSONL, para inspeção antes de treinar.
`finetune_lora`	função	`(dataset_jsonl, cfg)`	Roda um fine-tuning LoRA real com `transformers.Trainer` e `peft` sobre o JSONL preparado.

pinneapple_llm.geometry_draft

Rascunho de geometria: o LLM escolhe e parametriza um gerador existente (domínios 2-D/3-D nomeados ou `naca_parametric`), conferido contra o registro. [sem teste direto]

`pinneapple_llm/geometry_draft.py` · 121 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`GeometryDraftResult`	classe	`(generator, kind, kwargs, reasoning, raw_response)`	Resultado: gerador escolhido, tipo (`domain2d`, `domain3d`, `airfoil`), kwargs, raciocínio e resposta bruta.
`draft_geometry`	função	`(description, *, provider='anthropic', model=None, api_key=None, conversation_store=None)…)`	Pede ao LLM para escolher e parametrizar um gerador de geometria; não constrói a geometria.

pinneapple_llm.guardrail

`PhysicsGuardrail`: roda checagens independentes e recalculadas (sanidade de parâmetros, análise dimensional, resíduo da EDP, conservação de massa ou calor, erro contra referência) e só marca o resultado como confiável se passar. Não garante correção.

`pinneapple_llm/guardrail.py` · 1.069 linhas · testes: 60

nome	tipo	assinatura	o que faz
`CheckResult`	classe	`(name, passed, detail, value, threshold)`	Resultado de uma checagem: nome, se passou, detalhe, valor e limite.
`GuardrailReport`	classe	`(checks)`	Relatório do guardrail: checagens que rodaram, as puladas, veredito de confiança e resumo.
`PhysicsGuardrail`	classe	`(spec, *, residual_threshold=0.01, n_check_points=4096, conservation_n_points_per_face=20…)`	Executa a sequência fixa de checagens contra um modelo treinado e o `ProblemSpec` em que ele foi treinado.

pinneapple_llm.local_llm

Provedor local Ollama via HTTP: iniciar `ollama serve`, baixar o modelo e fazer chamadas de chat sem dados saindo da máquina. [sem teste direto]

`pinneapple_llm/local_llm.py` · 119 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`start_server`	função	`(*, host='127.0.0.1', port=11434, wait_s=10.0) -> subprocess.Popen`	Inicia `ollama serve` como subprocesso em segundo plano; falha se o `ollama` não estiver instalado.
`ensure_model_pulled`	função	`(model, *, host='127.0.0.1', port=11434, timeout=3600) -> None`	Roda `ollama pull` do modelo se ele ainda não estiver presente.
`call_ollama`	função	`(prompt, model, api_key=None, system='', *, host='127.0.0.1', port=11434, timeout=300, js…)`	Chamada de chat a um servidor Ollama local, com erro claro se ele não estiver acessível.

pinneapple_llm.org_knowledge

Respostas do LLM fundamentadas nos resultados reais de um `RetrievalIndex` local de `pinneapple_analysis.retrieval` ("LLM local que conhece o código da organização").

`pinneapple_llm/org_knowledge.py` · 133 linhas · testes: 4

nome	tipo	assinatura	o que faz
`OrgAnswer`	classe	`(question, context_used, answer)`	Resposta: pergunta, trechos de contexto realmente usados (fonte, título, score) e o texto da resposta.
`answer_with_context`	função	`(question, retrieval_index, *, top_k=5, provider='ollama', model='llama3.2:3b', api_key=N…)`	Busca no índice e pede ao LLM uma resposta baseada só nesses resultados.

pinneapple_llm.qa

Garantia de qualidade científica e industrial para artefatos do `pinneapple_llm`: estágios A-G e níveis Q0-Q4.

`pinneapple_llm/qa/__init__.py` · 51 linhas · testes: 48

pinneapple_llm.qa.examples

Exemplos de referência executáveis do pipeline de qualidade (importados sob demanda).

`pinneapple_llm/qa/examples/__init__.py` · 10 linhas · testes: 39

pinneapple_llm.qa.examples.pipe_wear_twin

Exemplo ponta a ponta: gêmeo digital 3-D de um tubo com desgaste de parede, construído com `pinneapple_twin3d` e passado pelos estágios A-G.

`pinneapple_llm/qa/examples/pipe_wear_twin.py` · 342 linhas · testes: 39

nome	tipo	assinatura	o que faz
`hollow_pipe_mesh`	função	`(*, radius=OUTER_DIAMETER_M / 2, thickness=WALL_THICKNESS_M, length=LENGTH_M, n_theta=64)…)`	Malha de tubo realmente oco (paredes externa e interna e duas coroas nas pontas) por revolução.
`solid_pipe_mesh`	função	`(*, radius=OUTER_DIAMETER_M / 2, length=LENGTH_M, n_theta=64) -> Tuple[np.ndarray, np.nda…)`	Cilindro maciço com as mesmas medidas externas, usado nos testes como falha que o validador de geometria precisa pegar.
`build_spec`	função	`(*, anomaly=False) -> PhysicalSpecification`	Monta a `PhysicalSpecification` do exemplo (com ou sem condição de anomalia).
`build_twin`	função	`(*, anomaly=False) -> Dict[str, Any]`	Monta a cena do gêmeo de desgaste com malhas, campos e sensores.
`export_twin`	função	`(bundle, output_dir=None) -> str`	Escreve os arquivos reais do visualizador e o contrato do hub; devolve o `scene.json`.
`analytical_reference`	função	`(bundle) -> ReferenceData`	Espessura residual em forma fechada para desgaste uniforme; é verificação de fórmulas, não de física.
`run_example`	função	`(*, output_dir=None, anomaly=False, write_report=True) -> Tuple[QualityReport, Dict[str, …)`	Roda os estágios A-G sobre o gêmeo sintético e devolve o relatório.

pinneapple_llm.qa.examples.pipe_wear_twin_main

Script que roda o exemplo de referência e imprime o relatório de qualidade; `--anomaly` demonstra a falha das invariantes. [sem teste direto]

`pinneapple_llm/qa/examples/pipe_wear_twin_main.py` · 37 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`main`	função	`(argv=None) -> int`	Lê argumentos, roda `run_example` e imprime o relatório; devolve 0 ou 1.

pinneapple_llm.qa.generation

Estágio B: captura as alegações estruturadas do LLM (premissas, dependências, unidades, equações, limitações) e as confronta com a especificação. [sem teste direto]

`pinneapple_llm/qa/generation.py` · 240 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`GeneratedArtifact`	classe	`(code, assumptions, dependencies, units, equations, limitations, provider, model, elapsed…)`	Proposta do LLM mais suas alegações estruturadas; não confiável por construção.
`build_prompt`	função	`(spec) -> str`	Monta um prompt que carrega a especificação inteira, não uma paráfrase.
`parse_generation_response`	função	`(raw, *, provider='', model='', elapsed_s=0.0) -> Tuple[GeneratedArtifact, StaticResult]`	Faz parse e checa o esquema da resposta do LLM; não levanta exceção com JSON malformado.
`check_claims`	função	`(artifact, spec) -> List[Finding]`	Confronta as declarações do modelo com a especificação e a tabela de unidades; devolve achados.
`validate_generated`	função	`(artifact, spec, *, filename='artifact.py') -> Tuple[List[Finding], Any]`	Checa as alegações e roda a validação estática completa do código devolvido.
`generate_artifact`	função	`(spec, *, provider='anthropic', model=None, api_key=None, conversation_store=None) -> Gen…)`	Pede a um provedor um artefato mais envelope de alegações, via `_dispatch.call_llm`.

`pinneapple_llm.qa.levels`

Níveis de qualidade Q0-Q4, cumulativos e dados só pelo que rodou; Q4 exige evidência operacional fornecida por humano. [sem teste direto]

`pinneapple_llm/qa/levels.py` · 182 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`evaluate_level`	função	`(report, spec=None) -> Tuple[str, List[str]]`	Devolve (nível, motivos), com os motivos de ter parado ali.
`blocking_reasons`	função	`(report) -> List[str]`	Motivos legíveis pelos quais o relatório não pode ser tratado como aprovado.
`level_description`	função	`(level) -> str`	Descrição textual de um nível.

`pinneapple_llm.qa.pipeline`

O pipeline de qualidade: liga os estágios A-G em uma execução reprodutível. [sem teste direto]

`pinneapple_llm/qa/pipeline.py` · 455 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`PipelineConfig`	classe	`(validators, run_artifacts, source_name, static_policy, repair_attempts)`	Configuração do pipeline: validadores, se roda artefatos, política estática e tentativas de reparo.
`QualityPipeline`	classe	`(spec, *, validators=None, sandbox=None, run_artifacts=False, static_policy=None, referen…)`	Executa os estágios sobre uma especificação e um artefato e devolve o `QualityReport`.

`pinneapple_llm.qa.references`

Estágio F: comparação com dados de referência independentes; recusa inventar referência ou limite de aceitação. [sem teste direto]

`pinneapple_llm/qa/references.py` · 202 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ReferenceData`	classe	`(name, origin, version, values, unit, limitations, used_for_calibration)`	Referência de comparação, com origem, versão e limitações obrigatórias.
`compare`	função	`(predictions, reference, criteria, *, subset='validation', groups=None, baseline=None) ->…)`	Compara previsões com a referência sob critérios dados pelo chamador; só as métricas pedidas bloqueiam.
`split_mask`	função	`(n, holdout_fraction=0.3, seed=0) -> np.ndarray`	Máscara booleana determinística que marca o subconjunto de validação (holdout).

pinneapple_llm.qa.repair

Estágio G: diagnóstico estruturado e reparo automático limitado, que rejeita reparos que mudem os critérios da especificação. [sem teste direto]

pinneapple_llm/qa/repair.py · 296 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
Diagnosis	classe	(failed_steps, findings, confirmed_cause, probable_cause, constraints_to_preserve, tests_…)	Por que uma tentativa foi rejeitada, num formato que o passo de reparo consome.
RepairAttempt	classe	(attempt, accepted, status, diagnosis, diff, notes, duration_s)	Registro de uma tentativa de reparo: aceita ou não, status, diagnóstico e diff.
RepairOutcome	classe	(accepted, artifact, attempts, results, stopped_because)	Resultado do laço de reparo: aceito, artefato final, tentativas e motivo da parada.
RepairLoop	classe	(*, evaluate, repair, spec=None, required=(), max_attempts=DEFAULT_MAX_ATTEMPTS, on_attem…)	Laço limitado gerar -> validar -> diagnosticar -> reparar.
diagnose	função	(results, *, required=()) -> Diagnosis	Junta os resultados de validadores que falharam num diagnóstico estruturado.

pinneapple_llm.qa.report

Relatório de qualidade reprodutível: status por estágio (sem booleano "ok"), hashes dos artefatos, versões e comando de reprodução. [sem teste direto]

pinneapple_llm/qa/report.py · 327 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
Status	classe	()	Enum de status: PASS, FAIL, BLOCKED, NOT_RUN, NOT_APPLICABLE.
Finding	classe	(id, status, message, severity, evidence, justification, limitations)	Um fato observado, com a evidência que o produziu.
StepOutcome	classe	(step, status, findings, duration_s, metadata)	Resultado de um estágio do pipeline, com seus achados.
ArtifactRecord	classe	(name, kind, path, sha256, size_bytes, metadata)	Registro de um artefato: nome, tipo, caminho, sha256, tamanho e metadados.
QualityReport	classe	(run_id, created_at, spec_id, spec_hash, criteria_hash, provider, model, steps, artifacts…)	Tudo o que uma execução do pipeline observou e o que ela pode afirmar; serializa para JSON.
environment_snapshot	função	() -> Dict[str, Any]	Versões e plataforma, para rastrear o relatório até um ambiente.

pinneapple_llm.qa.sandbox

Estágio D: execução controlada de artefatos em subprocesso com ambiente limpo e limites de recursos POSIX; não é sandbox de sistema operacional. [sem teste direto]

pinneapple_llm/qa/sandbox.py · 389 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
IsolationLevel	classe	()	Enum do nível de isolamento: bloqueado ou subprocesso.
SandboxLimits	classe	(timeout_s, cpu_s, memory_mb, file_size_mb, open_files, max_processes, write_bytes)	Tetos de recursos (CPU, memória, tamanho de arquivo etc.); None desliga o limite.
SandboxResult	classe	(status, isolation, workdir, exit_code, stdout, stderr, duration_s, timed_out, violations…)	Resultado da execução: status, código de saída, saídas, tempo, timeout, violações e arquivos produzidos.
SandboxExecutor	classe	(*, limits=None, allow_execution=False, allow_network=False, base_dir=None, keep_workdir=…)	Roda um artefato Python gerado num processo separado e limitado.
hash_bytes	função	(blob) -> str	SHA-256 em hexadecimal de um bloco de bytes.

pinneapple_llm.qa.spec

Estágio A: especificação física tipada e serializável, separando o que veio do usuário, de fonte citada, de premissa ou de estimativa. [sem teste direto]

`pinneapple_llm/qa/spec.py` · 496 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Provenance`	classe	`()`	Enum de proveniência: USER, SOURCE, ASSUMPTION, ESTIMATED.
`SpecError`	classe	`()`	Erro para especificação estruturalmente inválida.
`Quantity`	classe	`(value, unit, provenance, source, uncertainty)`	Número com unidade, proveniência e incerteza opcional.
`Invariant`	classe	`(name, kind, target, params, severity, description)`	Restrição física declarativa (tipo mais parâmetros) que o artefato deve cumprir.
`Observation`	classe	`(sensor_id, quantity, unit, position, timestamps, time_unit, values, provenance, holdout)`	Leitura de sensor: onde, o quê, quando e se pode ser usada para validação.
`GeometrySpec`	classe	`(kind, coordinate_system, dimensions, hollow, wall_thickness, notes)`	Geometria, domínio e sistema de coordenadas com unidades explícitas.
`PhysicalSpecification`	classe	`(objective, phenomenon, geometry, independent_variables, dependent_variables, parameters,…)`	Saída do estágio A: o pedido do usuário expresso de forma checável, com hash dos critérios.
`quantity_map`	função	`(**kwargs) -> Dict[str, Quantity]`	Atalho para montar um dicionário de `Quantity` a partir de tuplas (valor, unidade).

pinneapple_llm.qa.static

Estágio C: validação estática por AST e introspecção real dos módulos PINNeAPPle (sintaxe, imports, contratos de API, assinaturas, segurança, esquema JSON). Não é sandbox nem prova de correção. [sem teste direto]

`pinneapple_llm/qa/static.py` · 596 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`StaticPolicy`	classe	`(allowed_roots, forbidden_roots, forbidden_calls, forbidden_attr_calls, allowed_write_roo…)`	Política estática: raízes de import permitidas e proibidas, chamadas proibidas e pastas onde se pode escrever.
`StaticResult`	classe	`(findings, resolved_imports)`	Resultado da validação estática: achados e imports resolvidos.
`check_syntax`	função	`(source, filename='<artifact>') -> List[Finding]`	Checa a compilação; um `SyntaxError` vira achado.
`check_imports`	função	`(source, policy) -> Tuple[List[Finding], Dict[str, Any]]`	Recusa imports proibidos e relata os desconhecidos.
`check_api_contracts`	função	`(source, resolved) -> List[Finding]`	Resolve de verdade cada atributo `pinneapple_*` usado pelo artefato, pegando APIs inventadas.
`check_signatures`	função	`(source, resolved) -> List[Finding]`	Compara chamadas literais com a `inspect.signature` real.
`check_security`	função	`(source, policy, write_roots=()) -> List[Finding]`	Recusa criação de processos, execução de código, rede e escrita fora do escopo.
`check_json_artifact`	função	`(data, schema=None, name='artifact') -> List[Finding]`	Valida um artefato JSON contra um subconjunto pequeno de JSON Schema.
`validate_python_artifact`	função	`(source, *, policy=None, filename='<artifact>', write_roots=(), json_schema=None) -> Stat…)`	Roda todas as checagens estáticas e devolve um `StaticResult`.
`validate_json_artifact`	função	`(payload, schema=None, name='artifact') -> StaticResult`	Valida um payload JSON e devolve um `StaticResult`.
`load_json_file`	função	`(path) -> Any`	Lê um arquivo JSON.

pinneapple_llm.qa.units

Análise dimensional sem dependências para o pipeline, com gramática e tabela de unidades fixas; unidade desconhecida levanta `UnitError`. [sem teste direto]

`pinneapple_llm/qa/units.py` · 327 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`UnitError`	classe	`()`	Erro de unidade não interpretável ou conversão incompatível.
`Unit`	classe	`(name, dim, scale, offset)`	Unidade em escala SI (`si = valor * escala + offset`) com dimensão.
`parse_unit`	função	`(text) -> Unit`	Converte texto de unidade num `Unit`.
`are_compatible`	função	`(a, b) -> bool`	Diz se duas unidades têm a mesma dimensão física.
`convert`	função	`(value, from_unit, to_unit) -> float`	Converte um valor entre unidades compatíveis.
`dimension_of`	função	`(unit) -> Dim`	Devolve a dimensão de uma unidade.
`dimension_str`	função	`(unit) -> str`	Devolve a dimensão de uma unidade como texto.

pinneapple_llm.qa.validators

Validadores científicos do estágio E e seu registro; os embutidos se registram na importação. [sem teste direto]

`pinneapple_llm/qa/validators/__init__.py` · 19 linhas · **sem teste direto**

pinneapple_llm.qa.validators.base

Protocolo de validador, contexto compartilhado e registro; inaplicável vira NOT_APPLICABLE e falta de dado vira BLOCKED, nunca PASS. [sem teste direto]

`pinneapple_llm/qa/validators/base.py` · 164 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`SensorReading`	classe	`(id, position, unit, quantity, series, holdout, position_unit)`	Um sensor: posição, unidade e série no eixo de tempo do artefato.
`ValidationContext`	classe	`(spec, meshes, fields, field_units, times, time_unit, sensors, payload, model, extra)`	Tudo o que um validador pode ver (especificação, malhas, campos, tempos, sensores, modelo).
`ValidatorResult`	classe	`(name, status, findings, evidence, limitations, duration_s)`	Resultado de um validador: status, achados, evidência, limitações e tempo.
`register`	função	`(name) -> Callable[[Validator], Validator]`	Decorador que registra um validador por nome; nome repetido dá erro.
`available`	função	`() -> List[str]`	Lista os nomes dos validadores registrados.
`get`	função	`(name) -> Validator`	Devolve o validador pelo nome, com erro claro se desconhecido.
`run`	função	`(name, ctx) -> ValidatorResult`	Executa um validador pelo nome e mede o tempo.
`not_applicable`	função	`(name, reason, **evidence) -> ValidatorResult`	Monta um resultado NOT_APPLICABLE com o motivo.
`blocked`	função	`(name, reason, **evidence) -> ValidatorResult`	Monta um resultado BLOCKED com o motivo.

pinneapple_llm.qa.validators.geometry

Validação de geometria, incluindo a checagem de corpo oco: compara o volume da malha com o implicado pelas dimensões da especificação. [sem teste direto]

`pinneapple_llm/qa/validators/geometry.py` · 206 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`check_geometry`	função	`(ctx) -> ValidatorResult`	Valida as malhas do artefato (volume, oco vs maciço); NOT_APPLICABLE sem malhas.

`pinneapple_llm.qa.validators.guardrail`

Ponte para o `PhysicsGuardrail`: aplica-se só quando há modelo treinado e `ProblemSpec`. [sem teste direto]

`pinneapple_llm/qa/validators/guardrail.py` · 70 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`check_guardrail`	função	`(ctx) -> ValidatorResult`	Roda o `PhysicsGuardrail` e transforma cada checagem num achado.

`pinneapple_llm.qa.validators.numeric`

Integridade numérica: valores finitos, formas coerentes com tempos e malhas, campos degenerados. [sem teste direto]

`pinneapple_llm/qa/validators/numeric.py` · 108 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`check_numeric`	função	`(ctx) -> ValidatorResult`	Checa finitude, forma e degenerescência dos campos numéricos do artefato.

`pinneapple_llm.qa.validators.physics`

Avalia as invariantes físicas declaradas na especificação sobre os arrays do artefato; alvo ausente vira BLOCKED. [sem teste direto]

`pinneapple_llm/qa/validators/physics.py` · 203 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`check_physics`	função	`(ctx) -> ValidatorResult`	Avalia cada invariante declarada contra os dados do artefato.

`pinneapple_llm.qa.validators.sensors`

Consistência de sensores: posição, tempo e grandeza existentes, e separação entre leituras de calibração e de validação. [sem teste direto]

`pinneapple_llm/qa/validators/sensors.py` · 123 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`check_sensors`	função	`(ctx) -> ValidatorResult`	Confere as leituras de sensores contra o domínio, o eixo de tempo e as grandezas modeladas.

`pinneapple_llm.qa.validators.temporal`

Consistência temporal: ordenação, número de amostras e janela de validade. [sem teste direto]

`pinneapple_llm/qa/validators/temporal.py` · 99 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`check_temporal`	função	`(ctx) -> ValidatorResult`	Confere o eixo de tempo e as contagens de amostras; NOT_APPLICABLE em artefato estacionário.

`pinneapple_llm.qa.validators.units`

Consistência dimensional entre geometria, parâmetros, campos e sensores; não prova que as equações estão certas. [sem teste direto]

`pinneapple_llm/qa/validators/units.py` · 142 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`check_units`	função	`(ctx) -> ValidatorResult`	Confere que cada grandeza declarada tem unidade válida e compatível com a especificação.

`pinneapple_llm.research`

Busca de literatura e repositórios no arXiv e GitHub com as fontes reais de `pinneapple_tools.hpo_experiments.sources`; o LLM só resume ou ordena os resultados reais. [sem teste direto]

`pinneapple_llm/research.py` · 85 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ResearchReport`	classe	`(query, papers, repos, llm_summary)`	Relatório de busca: consulta, artigos, repositórios e resumo opcional do LLM.
`search_literature`	função	`(query, *, k_papers=5, k_repos=5, summarize=False, provider='anthropic', model=None, api_…)`	Busca artigos e repositórios reais e, opcionalmente, pede ao LLM um resumo deles.

`pinneapple_llm.twin_draft`

Rascunho de gêmeo digital: o LLM propõe valores para os campos de `DigitalTwinConfig`; o gêmeo é sempre construído por `build_digital_twin`. [sem teste direto]

`pinneapple_llm/twin_draft.py` · 156 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`TwinDraftResult`	classe	`(config_kwargs, reasoning, raw_response)`	Resultado: kwargs de configuração propostos, raciocínio e resposta bruta.
`draft_digital_twin`	função	`(description, *, provider='anthropic', model=None, api_key=None, conversation_store=None)…)`	Pede ao LLM kwargs de `DigitalTwinConfig` para uma descrição; não constrói o gêmeo.
`build_3d_live_twin`	função	`(model, field_names, *, export_dir, points, config_kwargs=None, **build_kwargs)`	Constrói um `DigitalTwin` real com um gancho que grava cada atualização como quadro `.ply` para `pinneapple_blender`.

`pinneapple_llm.workbench`

Agent Workbench: um agente líder e especialistas (explorador, pesquisador, programador, testador, revisor, engenheiro de física, relator) num modelo local.

`pinneapple_llm/workbench/__init__.py` · 22 linhas · testes: 20

`pinneapple_llm.workbench.__main__`

Linha de comando `python -m pinneapple_llm.workbench` (ou `pinneapple-workbench`): `serve`, `doctor`, `pull`, `run`, `eval`, `finetune`, `vscode`.

`pinneapple_llm/workbench/__main__.py` · 274 linhas · testes: 20

nome	tipo	assinatura	o que faz
`cmd_serve`	função	`(args)`	Sobe o app web com uvicorn.
`cmd_doctor`	função	`(args)`	Verifica máquina, servidor de modelo e modelo e diz o que corrigir.
`cmd_pull`	função	`(args)`	Baixa com Ollama o modelo recomendado (ou o indicado).
`cmd_run`	função	`(args)`	Roda uma tarefa no terminal, sem navegador.
`cmd_eval`	função	`(args)`	Mede um modelo nas tarefas embutidas de avaliação.
`cmd_finetune`	função	`(args)`	Exporta o dataset do banco, treina um adaptador LoRA ou o serve.
`cmd_vscode`	função	`(args)`	Instala a extensão do VS Code que abre o workbench ao lado do editor.
`main`	função	`(argv=None)`	Monta o `argparse` dos subcomandos e despacha.

pinneapple_llm.workbench.agents

Definições dos agentes: um Markdown por agente com front matter e prompt; versões do usuário ou do projeto substituem as embutidas pelo nome.

pinneapple_llm/workbench/agents.py · 65 linhas · testes: 20

nome	tipo	assinatura	o que faz
Agent	classe	(name, title, description, tools, prompt, source, extra)	Um agente: nome, título, descrição, ferramentas e prompt de sistema.
parse_agent	função	(text, source='') -> Agent	Lê um arquivo Markdown de agente (front matter mais corpo) num Agent.
load_agents	função	(workspace=None, user_dir=USER_DIR) -> tuple[dict[str, Agent], str]	Carrega todos os agentes (embutidos, do usuário, do workspace) e as regras comuns.

pinneapple_llm.workbench.backend

Backends de chat com ferramentas: Ollama, qualquer servidor compatível com OpenAI e um backend roteirizado para testes; Backend.chat da base levanta NotImplementedError.

pinneapple_llm/workbench/backend.py · 262 linhas · testes: 20 · palavras-chave no código (heurística; o texto acima diz o status real): levanta NotImplementedError em algum caminho

nome	tipo	assinatura	o que faz
ToolCall	classe	(name, arguments, id)	Uma chamada de ferramenta pedida pelo modelo: nome, argumentos e id.
Reply	classe	(content, tool_calls, usage)	Resposta do modelo: texto, chamadas de ferramenta e uso de tokens.
Backend	classe	(model, tool_mode)	Interface base de backend (chat, models), com modo de ferramenta nativo ou JSON.
OllamaBackend	classe	(model, host='http://127.0.0.1:11434', *, num_ctx=32768, temperature=0.2, tool_mode='nati…)	Backend sobre /api/chat de um servidor Ollama local.
OpenAIBackend	classe	(model, base_url='http://127.0.0.1:1234/v1', api_key='local', *, temperature=0.2, tool_mo…)	Backend sobre /v1/chat/completions de servidor compatível com OpenAI (LM Studio, llama.cpp, vLLM, MLX ou hospedado).
ScriptedBackend	classe	(script, model='scripted')	Backend que repete respostas roteirizadas ou chama uma função, para testes e demos sem modelo.
parse_json_tool_call	função	(text, tool_names) -> ToolCall | None	Lê uma chamada de ferramenta em JSON de uma resposta; None se não houver.
make_backend	função	(kind='ollama', model='', **kw) -> Backend	Cria o backend pelo tipo (ollama, openai, lmstudio, llamacpp, vllm, mlx, …).

pinneapple_llm.workbench.evaluate

Benchmark pequeno e checável do workbench na sua máquina e modelo: cada tarefa monta um projeto mínimo e confere o resultado mecanicamente.

pinneapple_llm/workbench/evaluate.py · 128 linhas · testes: 20

nome	tipo	assinatura	o que faz
TaskResult	classe	(name, passed, seconds, model_calls, prompt_tokens, completion_tokens, detail)	Resultado de uma tarefa: passou, tempo, chamadas ao modelo e tokens.
run_eval	função	(backend, names=None, on_event=None) -> list[TaskResult]	Roda as tarefas de avaliação com um backend e devolve os resultados.
format_results	função	(results, model) -> str	Formata os resultados como tabela de texto.
as_dicts	função	(results) -> list[dict]	Converte os resultados em lista de dicts.

pinneapple_llm.workbench.finetune

Fine-tuning do modelo local nas próprias sessões: exporta trajetórias do banco (com segredos mascarados) e treina LoRA com `mlx_lm` (Apple Silicon) ou TRL + PEFT (CUDA).

`pinneapple_llm/workbench/finetune.py` · 178 linhas · testes: 20

nome	tipo	assinatura	o que faz
`TrainConfig`	classe	`(model, adapter_path, iters, batch_size, num_layers, learning_rate, max_seq_length, lora_…)`	Configuração do treino: modelo, pasta do adaptador e hiperparâmetros de LoRA.
`redact`	função	`(text) -> str`	Mascara tokens e chaves que parecem segredos num texto.
`export_dataset`	função	`(store, out_dir, *, min_rating=0, mode='trajectories', agents=None, val_fraction=0.1, see…)`	Escreve `train.jsonl` e `valid.jsonl` a partir do banco, filtrando por avaliação; devolve as contagens.
`mlx_train_command`	função	`(data_dir, cfg) -> list[str]`	Monta a linha de comando de `mlx_lm.lora`.
`train_mlx`	função	`(data_dir, cfg=None, *, dry_run=False) -> str`	Faz fine-tuning LoRA com `mlx_lm` e devolve a pasta do adaptador.
`serve_mlx_command`	função	`(cfg=None, port=8080) -> list[str]`	Monta a linha de comando de `mlx_lm.server` com o adaptador.
`train_peft`	função	`(data_dir, cfg) -> str`	Faz fine-tuning LoRA com TRL + PEFT e devolve a pasta do adaptador.

pinneapple_llm.workbench.models

Escolhe o modelo local a partir da memória da máquina.

`pinneapple_llm/workbench/models.py` · 75 linhas · testes: 20

nome	tipo	assinatura	o que faz
`Machine`	classe	`(system, arch, memory_gb, apple_silicon)`	Dados da máquina: sistema, arquitetura, memória e se é Apple Silicon.
`Recommendation`	classe	`(min_memory_gb, model, approx_size_gb, context, note)`	Recomendação de modelo: memória mínima, tag Ollama, tamanho, contexto e nota.
`detect_machine`	função	`() -> Machine`	Detecta sistema, arquitetura e memória da máquina.
`recommend`	função	`(memory_gb=None) -> Recommendation`	Modelo padrão para a memória dada (ou detectada).

pinneapple_llm.workbench.runtime

O laço de agentes: o líder conversa, chama ferramentas, mantém um plano e delega a especialistas; cada passo vira evento armazenado. (A marca de placeholder vem da ferramenta `todo`, não de código incompleto.)

`pinneapple_llm/workbench/runtime.py` · 301 linhas · testes: 20 · palavras-chave no código (heurística; o texto acima diz o status real): placeholder/stub citado no docstring

nome	tipo	assinatura	o que faz
`Cancelled`	classe	`()`	Exceção usada para cancelar um turno em andamento.
`SessionSettings`	classe	`(max_steps, max_depth, context_chars, keep_recent, language)`	Limites da sessão: passos por agente, profundidade de delegação, tamanho de contexto e compactação.
`Session`	classe	`(backend, workspace, *, policy=None, settings=None, on_event=None, agents=None, common=No…)`	Uma sessão de trabalho: envia mensagens, roda o laço do líder, atende aprovações e perguntas e pode ser cancelada.

pinneapple_llm.workbench.server

App web FastAPI sobre `Session` e `Store`, com frontend estático; escuta em 127.0.0.1 por padrão e aceita token de acesso.

pinneapple_llm/workbench/server.py · 318 linhas · testes: 20

nome	tipo	assinatura	o que faz
Manager	classe	(store, backend_factory, default_workspace, language='')	Gerencia sessões vivas (uma thread por turno) sobre o banco.
create_app	função	(*, backend='ollama', model='', host='', base_url='', api_key='', workspace='.', db=DEFAU…)	Cria o app FastAPI com as rotas da API e o frontend.

pinneapple_llm.workbench.store

Banco do workbench (SQLite local ou PostgreSQL): sessões, eventos, chamadas ao modelo e avaliações para fine-tuning. [sem teste direto]

pinneapple_llm/workbench/store.py · 158 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
Store	classe	(url=DEFAULT_DB)	Acesso ao banco: cria e lista sessões, grava eventos, chamadas ao modelo e feedback, e dá estatísticas.

pinneapple_llm.workbench.tools

Ferramentas dos agentes confinadas a uma pasta: arquivos, busca, comandos, jobs em segundo plano, internet, GitHub e exibição de arquivos, com política de aprovação.

pinneapple_llm/workbench/tools.py · 355 linhas · testes: 37

nome	tipo	assinatura	o que faz
ToolError	classe	()	Erro de ferramenta devolvido ao agente.
ApprovalPolicy	classe	(mode)	Política de aprovação (ask, auto, yolo) que decide quais ações pedem confirmação.
Workspace	classe	(root, policy=None, approve=None, run_timeout=600, web=True)	Pasta que os agentes podem ler e alterar; implementa as ferramentas e pede aprovação nas ações protegidas.

pinneapple_llm.workbench.web

Busca na internet (SearXNG, Brave, Tavily ou DuckDuckGo) e leitura de páginas para os agentes.

pinneapple_llm/workbench/web.py · 142 linhas · testes: 20

nome	tipo	assinatura	o que faz
parse_duckduckgo	função	(page) -> list[dict]	Extrai título, URL e trecho dos resultados da página HTML do DuckDuckGo.
web_search	função	(query, max_results=8) -> list[dict]	Busca com o primeiro provedor configurado e devolve [{title, url, snippet}].
html_to_text	função	(page) -> str	Converte HTML em texto legível.
fetch_text	função	(url, max_chars=12000) -> str	Lê o texto de uma página ou arquivo; links blob do GitHub são lidos crus.

Exemplos de uso reais do repositório

Especificação física do pipeline de qualidade — docs/guides/pinneapple_llm_quality_pipeline.md

```
import pinneapple_llm.qa as qa

spec = qa.PhysicalSpecification(
    objective="Track pipe wall wear over five years",
    phenomenon="erosive wall thinning",
    geometry=qa.GeometrySpec(
        kind="hollow_pipe",
```

```
            dimensions={"outer_radius": qa.Quantity(0.10, "m"),
                        "length": qa.Quantity(2.0, "m")},
            hollow=True,
            wall_thickness=qa.Quantity(0.010, "m"),
        ),
```

resolve_problem sem credenciais cai no casamento por palavras — `tests/test_catalog_match.py`

```
def test_resolve_problem_falls_back_without_llm_credentials():
    # No API key is configured in this environment -- resolve_problem must
    # never raise, and must fall back to the deterministic keyword match.
    for var in ("ANTHROPIC_API_KEY", "OPENAI_API_KEY"):
        os.environ.pop(var, None)

    result = resolve_problem("burgers equation turbulence one dimensional")
    assert result.used_llm is False
    assert result.preset is not None
```

Subir o Agent Workbench com modelo local — `docs/guides/agent_workbench.md`

```
brew install ollama            # or the app from https://ollama.com; it uses the Mac's GPU (Metal)
ollama serve &                 # the Ollama app does this for you
pip install "pinneapple[workbench]"

python -m pinneapple_llm.workbench doctor       # memory, Ollama, model, Python packages: what to fix
python -m pinneapple_llm.workbench pull         # downloads the model recommended for your memory
python -m pinneapple_llm.workbench serve --workspace ~/code/my-project --language Português
```

Testes e status de validação

- Módulos com pelo menos um teste que os importa: 21 de 49.
- Funções de teste em arquivos que importam algum módulo do pacote: 242.

Limitações conhecidas

- O `PhysicsGuardrail` não garante que um resultado esteja correto (o docstring diz isso explicitamente); a checagem de conservação só existe para as famílias de EDP com balanço conhecido (massa e condução de calor).
- Os provedores hospedados (Anthropic, OpenAI) exigem chave de API e o extra `llm`; o caminho local exige Ollama instalado, sem fallback embutido.
- `finetune` e `workbench.finetune` dependem de extras pesados (`transformers`, `peft`, `trl`, `mlx_lm`); `finetune` não treina modelos Ollama diretamente.
- `cad_draft`: templates paramétricos e exportação STEP precisam de `cadquery`; sem motor booleano (`manifold3d`) as operações CSG viram concatenação das partes.
- `qa.sandbox` é isolamento por subprocesso com limites POSIX, não um sandbox de sistema operacional; `qa.static` não prova que o código faz o que diz.
- `qa.units` entende só uma gramática e uma tabela de unidades fixas; unidade desconhecida levanta `UnitError`.
- O nível Q4 do pipeline de qualidade não pode ser alcançado dentro do repositório: exige evidência operacional fornecida por humano.
- O workbench executa comandos na máquina do usuário; o servidor escuta em 127.0.0.1 e não deve ser exposto à rede. Modelos locais não igualam os maiores modelos hospedados em tarefas longas, segundo o próprio guia.

7. pinneapple_mcp

Objetivo

pinneapple_mcp expõe o PINNeAPPle como servidor MCP (Model Context Protocol) via stdio. Qualquer agente de código que fale MCP pode resolver, comparar e validar problemas físicos com a biblioteca, e cada resultado volta com suas verificações físicas e um cartão de confiança (APPROVED / REVIEW / REJECT).

As ferramentas ficam em pinneapple_mcp.tools como funções Python comuns, com argumentos e retornos em JSON, utilizáveis sem MCP. Elas chamam o resto da biblioteca: pp.solve e pp.list_methods (em pinneapple_physics.solving), o catálogo de métodos, os presets de problemas, pp.zoo e pp.datasets, o design por intenção (pinneapple_design.intent) e a cadeia de porquês (pinneapple_reason). Um problema pode ser um preset ou um 'problema em caixa' descrito campo a campo em JSON, com expressões das coordenadas analisadas por ast (nunca eval).

O servidor tem padrões seguros (config): trabalho longo (solve, benchmark, design, reason) roda como job num processo separado com limite de tempo, limite de memória e rede desligada; resultados vão para um diretório de trabalho e só se leem arquivos de raízes permitidas. import pinneapple_mcp não importa o SDK MCP; ele só é carregado por server (SDK 1.x FastMCP ou 2.x MCPServer).

Números do inventário

módulos	linhas	classes públicas	funções públicas	testes em arquivos que importam o pacote	módulos sem teste direto
8	1.541	2	31	30 (3 arquivos)	3

Submódulos

submódulo	o que faz	módulos	classes + funções	testes*
__main__	Permite rodar o servidor com python -m pinneapple_mcp.	1	0	0
config	Limites e locais do servidor (diretório de trabalho, tempo, memória, jobs, épocas, pontos, rede, raízes) com variáveis de ambiente.	1	3	10
expressions	Compila expressões aritméticas seguras das coordenadas em funções NumPy, sem eval.	1	2	10
jobs	Executa trabalho longo como jobs em processos separados, com estado gravado em disco.	1	5	0
problems	Monta um problema físico a partir de JSON: preset ou problema em caixa com condições de contorno.	1	2	0
server	Servidor MCP stdio que registra todas as ferramentas com assinaturas tipadas.	1	2	10
tools	As ferramentas MCP como funções puras com entrada e saída JSON.	1	17	30

* maior número de testes que importam um módulo do submódulo.

Módulos e API pública

pinneapple_mcp

Ponto de entrada do pacote; build_server e main importam server só quando chamados, para não exigir o SDK MCP no import.

pinneapple_mcp/__init__.py · 34 linhas · testes: 30

nome	tipo	assinatura	o que faz
build_server	função	(*args, **kwargs)	Importa server sob demanda e devolve o objeto servidor MCP.
main	função	(argv=None)	Importa server sob demanda e roda o servidor stdio.

pinneapple_mcp.__main__

Chama `server.main()` para `python -m pinneapple_mcp`. [sem teste direto]

`pinneapple_mcp/__main__.py` · 5 linhas · **sem teste direto**

pinneapple_mcp.config

`Config` com padrões seguros, cada um sobrescrevível por variável de ambiente `PINNEAPPLE_MCP_*`; `get_config`/`set_config` mantêm a configuração global.

`pinneapple_mcp/config.py` · 91 linhas · testes: 10

nome	tipo	assinatura	o que faz
`Config`	classe	`(workdir, max_seconds, max_memory_mb, max_jobs, max_epochs, max_points, max_file_mb, allo…)`	Configuração do servidor: diretório, tempo e memória por job, jobs simultâneos, épocas, pontos, tamanho de arquivo, rede e raízes de leitura.
`get_config`	função	`() -> Config`	Devolve a configuração ativa.
`set_config`	função	`(cfg) -> Config`	Define a configuração ativa e a devolve.

pinneapple_mcp.expressions

Analisa texto com `ast` aceitando só números, nomes de coordenadas, `pi`/`e`, operadores aritméticos e uma lista fixa de funções NumPy; qualquer outra coisa levanta `ValueError`.

`pinneapple_mcp/expressions.py` · 105 linhas · testes: 10

nome	tipo	assinatura	o que faz
`compile_expression`	função	`(text, coords) -> Callable[[np.ndarray], np.ndarray]`	Compila o texto numa função `f(X) -> (N,)` das coordenadas, ou levanta `ValueError`.
`value`	função	`(v, coords)`	Mantém números e compila strings como expressão.

pinneapple_mcp.jobs

Cada job roda num processo filho com `RLIMIT_AS`, conexões de rede não locais bloqueadas e diretório próprio com `request.json`, `status.json` e `log.txt`; o estado sobrevive a reinícios (job sem processo vira `lost`). [sem teste direto pelo inventário; exercitado por `tests/test_mcp_server.py` via `tools`]

`pinneapple_mcp/jobs.py` · 199 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`running`	função	`() -> list[str]`	Ids dos jobs com processo ainda vivo.
`submit`	função	`(kind, args, cfg, max_seconds=None) -> str`	Inicia um job de um tipo de `tools.JOB_KINDS` num novo processo e devolve o id.
`status`	função	`(job_id, cfg, wait_seconds=0.0) -> dict[str, Any]`	Estado de um job, seu resultado quando pronto e os arquivos do diretório, podendo esperar até `wait_seconds`.
`cancel`	função	`(job_id, cfg) -> dict[str, Any]`	Mata o processo de um job em execução.
`list_jobs`	função	`(cfg, limit=20) -> list[dict[str, Any]]`	Lista os jobs mais recentes com estado.

pinneapple_mcp.problems

Constrói `BuiltProblem` a partir de JSON: preset com `preset_kwargs` ou problema em caixa (domínio, campos, EDP, fonte, condições de Dirichlet, solução exata). [sem teste direto pelo inventário]

`pinneapple_mcp/problems.py` · 156 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`BuiltProblem`	classe	`(problem, ctx, exact, dirichlet, source, spec)`	Problema montado: `PhysicalProblem`, contexto, solução exata, condições de Dirichlet e fonte; `check_pde` diz qual EDP estacionária pode ser verificada.
`build_problem`	função	`(spec) -> BuiltProblem`	Monta um `BuiltProblem` a partir do JSON, ou levanta `ValueError` com o motivo.

pinneapple_mcp.server

Monta o servidor MCP (SDK 2.x `MCPServer` ou 1.x `FastMCP`) com todas as ferramentas e CLI `pinneapple-mcp` com opções de limites.

`pinneapple_mcp/server.py` · 284 linhas · testes: 10

nome	tipo	assinatura	o que faz
`build_server`	função	`(config=None)`	Cria o objeto servidor MCP com todas as ferramentas do PINNeAPPle.
`main`	função	`(argv=None) -> None`	CLI do servidor: lê opções de limites, ajusta a configuração e roda em stdio.

pinneapple_mcp.tools

Implementa as ferramentas: capacidades, catálogo de métodos, presets, `trust_card`, `solve` e `benchmark` como jobs, zoo, jobs, `design` e as ferramentas de `pinneapple_reason`.

`pinneapple_mcp/tools.py` · 667 linhas · testes: 30

nome	tipo	assinatura	o que faz
`capabilities`	função	`() -> dict[str, Any]`	Informa versão, APIs opcionais presentes, dispositivo, limites e diretório de trabalho.
`list_methods`	função	`(category='all', validated_only=False) -> dict[str, Any]`	Métodos de `solve` agrupados por tipo e o catálogo de métodos com status de validação.
`describe_method`	função	`(method) -> dict[str, Any]`	Descreve um método de `solve` com suas opções, ou um item do catálogo com código, referências e testes.
`list_presets`	função	`(contains=None) -> dict[str, Any]`	Lista os presets de problema utilizáveis em `solve` e `benchmark`.
`trust_card`	função	`(file=None, field=None, grid=None, values=None, pde=None, source=None, alpha=1.0, dirichl…)`	Verificações físicas de um campo em grade (inline ou arquivo) e o cartão de confiança com decisão.
`solve`	função	`(problem, method='pinn', options=None, reference=None, reference_options=None, grid_point…)`	Resolve um problema com um método num job; o resultado traz métricas, verificações, cartão de confiança e arquivos.
`benchmark`	função	`(problem, methods=('pinn', 'fem'), budget=None, seeds=1, reference=None, options=None, gr…)`	Roda vários métodos num problema com o mesmo orçamento de treino e pontua na mesma grade, num job.
`zoo_list`	função	`(task=None) -> dict[str, Any]`	Lista modelos pré-treinados verificados do zoo e datasets do hub.
`zoo_predict`	função	`(name, inputs) -> dict[str, Any]`	Previsões de um modelo do zoo, marcando as linhas fora da faixa de validade.
`zoo_validate`	função	`(name, inputs=None) -> dict[str, Any]`	Relatório de confiança de um modelo do zoo, opcionalmente checando se entradas estão na faixa válida.
`job_status`	função	`(job_id, wait_seconds=0.0) -> dict[str, Any]`	Estado de um job e, quando pronto, o resultado com os caminhos dos arquivos.
`list_jobs`	função	`(limit=20) -> dict[str, Any]`	Jobs mais recentes com estado.
`cancel_job`	função	`(job_id) -> dict[str, Any]`	Para um job em execução matando o processo.
`design`	função	`(text, answers=None, verify=True, seed=0, max_seconds=None, wait_seconds=0.0) -> dict[str…)`	Design a partir de um objetivo em linguagem natural: responde na hora `out_of_scope` ou `needs_input`, senão inicia um job que gera o pacote de design.
`reason`	função	`(case=None, requirements=None, concept=None, lang='pt', max_steps=12, max_seconds=None, w…)`	Roda a cadeia de porquês de `pinneapple_reason` num job, aplicando contramedidas até cada requisito passar.

nome	tipo	assinatura	o que faz
`reason_library`	função	`(domain=None, lang='pt') -> dict[str, Any]`	Lista os modos de falha por domínio físico, as contramedidas e os casos resolvidos de `pinneapple_reason`.
`reason_check`	função	`(mode, params, elements=(), limits=None, lang='pt') -> dict[str, Any]`	Avalia um modo de falha nos parâmetros dados (SI): demanda, capacidade, margem, mecanismo e fontes.

Exemplos de uso reais do repositório

Problema em caixa resolvido por FEM via ferramenta MCP — `tests/test_mcp_server.py`

```
PLATE = {"name": "plate", "domain": {"x": [0, 1], "y": [0, 1]}, "fields": ["T"], "pde": "laplace",
         "bcs": [{"kind": "dirichlet", "field": "T", "on": "y_max", "value": "sin(pi*x)"},
                 {"kind": "dirichlet", "field": "T", "on": "y_min", "value": 0},
                 {"kind": "dirichlet", "field": "T", "on": "x_boundary", "value": 0}],
         "exact": "sin(pi*x)*sinh(pi*y)/sinh(pi)"}
...
def test_solve_job_fem_with_checks(cfg):
    st = tools.solve(PLATE, "fem", {"n": 24}, grid_points_per_axis=25, wait_seconds=180)
    assert st["status"] == "done", st
    res = st["result"]
    assert res["reference"] == "exact" and res["metrics"]["relative_l2"]["T"] < 0.02
    assert res["decision"] == "APPROVED", res["trust_cards"]
```

Instalação e execução do servidor — `docs/guides/mcp_server.md`

```
pip install "pinneapple[mcp]"
pinneapple-mcp --help            # or: python -m pinneapple_mcp
```

Testes e status de validação

- Módulos com pelo menos um teste que os importa: 5 de 8.
- Funções de teste em arquivos que importam algum módulo do pacote: 30.

Limitações conhecidas

- O servidor precisa do extra `pinneapple[mcp]` (SDK MCP 1.x ou 2.x e `pydantic`); as ferramentas em `tools` funcionam sem ele.
- Limites padrão por job: 600 s, 8192 MB de espaço de endereçamento, 2 jobs simultâneos, 20000 épocas, 250000 pontos e arquivos de até 200 MB; o limite de memória usa `resource` e é ignorado em plataformas sem ele.
- A rede dentro dos jobs fica desligada substituindo `socket.connect` (só conexões locais e Unix passam), a menos que `PINNEAPPLE_MCP_ALLOW_NETWORK=1`.
- Expressões aceitam só aritmética das coordenadas e uma lista fixa de funções NumPy; qualquer outra construção é recusada.
- `problems.BuiltProblem.check_pde` só verifica resíduo para EDPs estacionárias `laplace`, `poisson` e `heat_equation`.
- `jobs`, `problems` e `__main__` não têm teste direto pelo inventário (são exercitados indiretamente por `tests/test_mcp_server.py`).

8. `pinneapple_autolab`

Objetivo

`pinneapple_autolab` é o "Autonomous Physics Lab": uma IA que administra evidências para tomar decisões de engenharia com um simulador de referência caro. Em vez de confiar num surrogate, o laboratório separa o que foi simulado (evidência) do que foi só previsto (rotulado como previsão até uma simulação confirmar), e entrega como recomendação sempre um projeto simulado, com o seu relatório de confiança e o livro de registro (ledger).

O pacote tem duas camadas. A primeira é o laço de agente (`core`): uma missão em linguagem natural vira um plano de chamadas controladas de biblioteca (`Tool`, `Toolbox`, `Step`), escolhidas por um planejador determinístico (`TemplatePlanner`) ou por um modelo de linguagem (`LLMPlanner`, `LLMStepPlanner`), e cada chamada fica registrada como `Evidence`. A segunda é o ciclo científico (`lab.AutonomousLab`): planejar, checar geometria e condições de contorno, simular, aprender (famílias de surrogates comparadas por validação cruzada, com incerteza), otimizar (NSGA-II sobre o surrogate), avaliar cada candidato (domínio validado, probabilidade de cumprir os limites, erro de campo esperado na região crítica), escolher a próxima simulação pelo que ainda é desconhecido, verificar e entregar.

As regras são impostas em código: orçamento de simulações com reserva para verificação (`Budget`, `BudgetExhaustedError`), limites relativos ao baseline que só valem depois de o baseline ser simulado (`UnresolvedLimitError`), condições de operação que o laboratório não pode mudar, e uma auditoria (`agent.audit_claims`) que exige que todo número da resposta final venha das ferramentas. O domínio de exemplo é `domains.racecar`: um carro de fórmula paramétrico (não o carro de nenhuma equipe) simulado em OpenFOAM, com a missão "menos arrasto mantendo a downforce".

Liga-se ao resto do repositório pelas variáveis e objetivos de `pinneapple_design.design_optimizer.study`, pela geometria `pinneapple_design.geometry.vehicles3d.FormulaCar`, pelo `TrustReport` de `pinneapple_analysis.trust`, por `pinneapple_llm` nos planejadores com LLM e pelo extrator `pinneapple_design.intent` na leitura de missões. É usado pelo nó `autolab` de `pinneapple_flow`, por agentes em `pinneapple_design.thermal.board_cooling` e `pinneapple_systems.digital_twin.pipe_loop`, e pelos exemplos `examples/autolab*`.

Números do inventário

módulos	linhas	classes públicas	funções públicas	testes em arquivos que importam o pacote	módulos sem teste direto
16	3.596	38	39	37 (5 arquivos)	10

Submódulos

submódulo	o que faz	módulos	classes + funções	testes*
`agent`	O laboratório dirigido por um agente: texto da missão -> `Mission` -> chamadas de ferramentas sobre o laboratório -> resposta auditada.	1	9	6
`core`	Laço de agente genérico: ferramentas, caixa de ferramentas, planejadores (modelo fixo ou LLM), execução e registro de evidências.	1	13	0
`dashboard`	Painel HTML com quatro áreas sincronizadas (geometria, física, modelos, decisões) sobre o estado real de uma sessão, estático, ao vivo ou em replay.	1	5	5
`domain`	Domínio validado: diz se um projeto está perto o bastante dos projetos simulados para a previsão valer.	1	2	0
`domains`	Simuladores e missões do laboratório, um módulo por domínio físico (hoje `racecar`).	2	11	11
`lab`	O laboratório autônomo: o ciclo planejar -> simular -> aprender -> otimizar -> avaliar -> verificar -> entregar sobre qualquer simulador.	1	2	10
`ledger`	Livro de registro de evidências: cada simulação e cada evento da sessão, em ordem, com orçamento.	1	4	0
`mission`	A missão em números: variáveis, objetivos, limites, baseline, orçamento e tolerâncias de aceitação.	1	4	0
`qualification`	Qualificação do solver antes da campanha: quanto da resposta depende da malha e do número de iterações.	1	1	0
`replay`	Simulador que responde a partir de um ledger gravado quando o projeto é exatamente o mesmo, e roda o real caso contrário.	1	1	6
`report`	Figuras e relatório de engenharia em Markdown de uma sessão, feitos só a partir do snapshot.	1	8	0
`simulator`	Interface do simulador de referência (`check`, `run`) e um simulador a partir de função Python.	1	3	0
`surrogates`	Surrogates do solver (GP, ensemble de MLP, superfície de resposta quadrática, POD de campo) comparados por validação cruzada.	1	9	0
`trust`	Triagem dos candidatos do otimizador (favorável, incerto, fora do domínio, improvável) e a regra de escolha da próxima simulação.	1	5	0

* maior número de testes que importam um módulo do submódulo.

Módulos e API pública

`pinneapple_autolab`

Ponto de entrada do Autonomous Physics Lab: reexporta missão, ledger, laboratório, simuladores, surrogates, domínio validado e triagem.

`pinneapple_autolab/__init__.py` · 70 linhas · testes: 31

`pinneapple_autolab.agent`

Laboratório dirigido por agente: lê a missão em texto, expõe uma caixa fechada de ferramentas (sem ferramenta para mudar condições, limites ou solver) e audita a resposta final.

`pinneapple_autolab/agent.py` · 488 linhas · testes: 6

nome	tipo	assinatura	o que faz
`MissionRefused`	classe	`()`	Exceção: o pedido não pode ser executado como escrito; traz o motivo e o que falta.
`MissionDomain`	classe	`(title, variables, baseline, output_words, units, operating_point, acceptance, conditions…)`	O que um simulador pode receber como pedido: variáveis, baseline, palavras que nomeiam saídas, ponto de operação fixo e tolerâncias padrão.
`MissionReading`	classe	`(mission, readings, assumptions)`	Resultado da leitura de uma missão: a `Mission`, os trechos lidos e as suposições feitas.
`LabPlanner`	classe	`(lab, n_doe=None)`	Planejador determinístico de passos sobre `lab_toolbox` (sem rede): baseline, projetos que preenchem o espaço, ciclos aprender/avaliar/simular, verificação e resposta.
`AgentResult`	classe	`(reading, refused, answer, audit, run, summary, wall_s, planner)`	Resultado de `run_agent`: leitura da missão, recusa, resposta, auditoria, execução, resumo, tempo e planejador usado.
`interpret_mission`	função	`(text, dom, llm=None) -> MissionReading`	Lê o texto numa `Mission` sobre o domínio dado; um LLM opcional só é aceito onde escolhe opções fechadas e cita palavras do texto.
`lab_toolbox`	função	`(lab, reading=None) -> Toolbox`	Caixa fechada de ferramentas com que o agente dirige o laboratório; cada ferramenta devolve o medido ou o previsto (rotulado).
`audit_claims`	função	`(answer, tool_log, lab, recommended_run=None, rtol=0.005) -> dict[str, Any]`	Confere se cada número da resposta aparece no que as ferramentas devolveram e se o run recomendado existe, teve sucesso e cumpre os limites.
`run_agent`	função	`(text, simulator, folder, *, domain, planner=None, provider=None, model=None, llm=None, m…)`	Interpreta o texto, monta o laboratório e deixa o agente dirigi-lo (planejador dado, LLM se houver provedor, senão `LabPlanner`).

`pinneapple_autolab.core`

Laço de agente missão -> plano -> chamadas controladas -> evidência: o agente nunca roda código próprio, só ferramentas registradas com argumentos checados. [sem teste direto]

`pinneapple_autolab/core.py` · 297 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Tool`	classe	`(name, fn, description, inputs, summary)`	Uma chamada de biblioteca permitida: `fn(state, **args) -> dict`, descrição, entradas aceitas e resumo para o log.
`Toolbox`	classe	`(kind, tools=None)`	Conjunto controlado de ferramentas de um tipo de missão (`add`, `names`, `check`, `describe`).
`Step`	classe	`(tool, args, why)`	Um passo do plano: ferramenta, argumentos e justificativa.
`Plan`	classe	`(mission, steps, planner, notes)`	Plano de uma missão: passos, planejador usado e notas.
`Planner`	classe	`()`	Interface base de planejador que produz um `Plan`.

nome	tipo	assinatura	o que faz
`TemplatePlanner`	classe	`(steps)`	Planejador determinístico com a sequência fixa de passos de um tipo de missão.
`LLMPlanner`	classe	`(fallback, *, provider='anthropic', model=None, llm=None, required=())`	Um modelo de linguagem ordena as ferramentas num plano JSON; ferramenta ou argumento desconhecido faz cair no plano modelo.
`Evidence`	classe	`(step, tool, args, why, summary, seconds, ok, error)`	Registro de uma chamada: passo, ferramenta, argumentos, justificativa, resumo do resultado, tempo, sucesso e erro.
`Run`	classe	`(plan, state, evidence)`	Resultado de uma execução: plano, estado com as saídas por ferramenta e a lista de evidências.
`StepPlanner`	classe	`(name)`	Interface de planejador que escolhe o próximo passo depois de ver os resultados (None quando termina).
`LLMStepPlanner`	classe	`(*, provider='anthropic', model=None, llm=None, rules='')`	Um modelo de linguagem escolhe cada próximo passo vendo missão, ferramentas e evidências; passo inválido é recusado e registrado.
`execute`	função	`(plan, toolbox, state=None, *, stop_on_error=True, log=None) -> Run`	Executa os passos do plano em ordem, só pela caixa de ferramentas, com estado compartilhado.
`act`	função	`(mission, toolbox, planner, state=None, *, max_steps=200, max_refusals=5, log=None) -> Run`	Laço decidir-e-agir: pede um passo, checa, executa, registra, até o planejador terminar ou atingir os limites de passos ou recusas.

`pinneapple_autolab.dashboard`

Painel da sessão a partir dos arquivos (`ledger.json`, `events.jsonl`, snapshot, resumo): página estática, servidor ao vivo, screenshot e vídeo do replay.

`pinneapple_autolab/dashboard.py` · 233 linhas · testes: 5

nome	tipo	assinatura	o que faz
`live_state`	função	`(folder, out_dir=None, tail=120) -> dict[str, Any]`	Estado da sessão lido dos arquivos, com um `status` do que está acontecendo agora.
`build_dashboard`	função	`(folder, out_dir, *, scenes=None, title='', state=None, live=False, replay=False, figures…)`	Grava `index.html` do painel com o estado embutido, opcionalmente ao vivo (polling de `state.json`) ou com replay.
`serve`	função	`(folder, out_dir, port=8120, host='127.0.0.1', block=True, **kw)`	Serve o painel ao vivo: a página e `/state.json` relido a cada requisição.
`capture`	função	`(html, png, *, width=1600, height=1100, full_page=True, replay_to=None, wait_ms=5000) -> …)`	Screenshot de uma página do painel (precisa de `playwright`).
`record_replay`	função	`(html, mp4, *, frames=60, fps=4.0, hold_s=3.0, width=1600, height=2040, out_width=1280, c…)`	Vídeo MP4 do replay do painel via ffmpeg (precisa de `playwright`).

`pinneapple_autolab.domain`

Domínio validado: distância de um projeto ao projeto simulado mais próximo, em unidades do espaçamento típico dos simulados, mais a faixa coberta por variável. [sem teste direto]

`pinneapple_autolab/domain.py` · 72 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`DomainVerdict`	classe	`(inside, ratio, nearest, reasons)`	Veredito de domínio: dentro ou fora, razão de distância, vizinho mais próximo e motivos em palavras.
`ValidatedDomain`	classe	`(U, names=None, bounds=None, max_ratio=1.5, margin=0.05)`	Domínio dos projetos simulados; `assess` diz se um projeto está dentro (razão até `max_ratio` e variáveis na faixa com margem).

`pinneapple_autolab.domains`

Pacote dos simuladores e missões por domínio físico. [sem teste direto]

`pinneapple_autolab/domains/__init__.py` · 2 linhas · **sem teste direto**

pinneapple_autolab.domains.racecar

Simulador do "AI Aerodynamicist": carro de fórmula paramétrico (`FormulaCar`, não de nenhuma equipe) em OpenFOAM (snappyHexMesh + simpleFoam k-omega SST), com a missão principal e o painel da sessão.

`pinneapple_autolab/domains/racecar.py` · 464 linhas · testes: 11

nome	tipo	assinatura	o que faz
`RaceCarAero`	classe	`(resolution='campaign', iterations=800, procs=4, n_lines=40, keep_cases=False, conditions…)`	Simulador de aerodinâmica externa do carro em OpenFOAM: checa o projeto, roda o caso e devolve forças, balanço e Cp de superfície por região.
`make_car`	função	`(params)`	Constrói um `FormulaCar` a partir dos parâmetros.
`force_convergence`	função	`(history, window=2) -> dict[str, float]`	Variação relativa de arrasto e downforce nos últimos intervalos gravados.
`solver_log_summary`	função	`(case) -> dict[str, Any]`	Resumo do log: última iteração, resíduos finais, se o checkMesh passou, células e tempos.
`render_run`	função	`(params, flow_npz, out_prefix, title='', size=(1200, 675)) -> dict[str, str]`	Renders PyVista de um projeto simulado: Cp de superfície e linhas de corrente.
`cycles_renders`	função	`(params, flow_npz, out_prefix, title='', samples=64, size=(1600, 900)) -> dict[str, str]`	Renders Blender Cycles de um projeto simulado (precisa de `bpy`; sem ele devolve {}).
`flow_scene`	função	`(params, flow_npz, title='')`	Cena do estúdio 3D de um projeto simulado (partes do carro, Cp/Cf, linhas de corrente, fatias).
`build_dashboard`	função	`(work, out_dir, summary, *, figures=None, figure_runs=None, viewers=True, replay=True, re…)`	Painel de uma sessão do carro com renders do projeto original e do recomendado, visualizadores 3D e a sessão gravada.
`recorded_dashboard`	função	`(results_dir, out_dir) -> str`	Painel de uma sessão gravada só a partir dos arquivos versionados, sem OpenFOAM e sem visualizador 3D.
`racecar_domain`	função	`()`	Descrição `MissionDomain` do que o simulador do carro pode receber, para `interpret_mission`.
`flagship_mission`	função	`(budget=30, reserve_verify=3) -> Mission`	A missão principal: menor arrasto a 70 m/s mantendo a downforce, dentro das restrições geométricas, validado por CFD.

pinneapple_autolab.lab

O laboratório autônomo sobre qualquer simulador, com cada etapa disponível como método e registrada no ledger; impõe orçamento, checagens antes de simular e recomendação só de projeto simulado.

`pinneapple_autolab/lab.py` · 546 linhas · testes: 10

nome	tipo	assinatura	o que faz
`AutonomousLab`	classe	`(mission, simulator, folder, *, families=…, field_family='gp', seed=0, nsga_budget=3000, …)`	O laboratório: missão, simulador, pasta, famílias de surrogates, região crítica e tolerância de campo; métodos para cada etapa do ciclo e `run()` para o ciclo inteiro.
`showcase`	função	`(decisions, baseline) -> dict[str, dict[str, Any]]`	Escolhe por regra fixa três projetos simulados que o laboratório julgou de forma diferente antes de simular (incerto, favorável, fora do domínio).

pinneapple_autolab.ledger

Livro de evidências: cada simulação (por quê, com o quê, custo, resultado) e cada evento, em ordem; é a única fonte de números para relatório, painel e agente. [sem teste direto]

`pinneapple_autolab/ledger.py` · 185 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`BudgetExhaustedError`	classe	`()`	Exceção: o orçamento de simulações da missão acabou.

nome	tipo	assinatura	o que faz
`RunRecord`	classe	`(id, purpose, params, ok, outputs, message, cost_s, started, reason, predicted, meta, fie…)`	Uma simulação de referência: id, propósito, parâmetros, sucesso, saídas, mensagem, custo, motivo e o que o surrogate previa antes.
`Ledger`	classe	`(folder, max_runs=None, max_wall_s=None, listeners=None)`	Runs e eventos de uma sessão numa pasta, com controle de orçamento, `events.jsonl` incremental e `ledger.json`.
`environment`	função	`(extra=None) -> dict[str, Any]`	Hardware e software da máquina: CPU, núcleos, memória, sistema, versões de Python e pacotes, commit do git.

`pinneapple_autolab.mission`

Missão em números: variáveis, objetivos, limites (absolutos ou relativos ao baseline), baseline, orçamento, aceitação e condições fixas. [sem teste direto]

`pinneapple_autolab/mission.py` · 161 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`UnresolvedLimitError`	classe	`()`	Exceção: um limite relativo ao baseline foi usado antes de o baseline ser simulado.
`Budget`	classe	`(max_runs, reserve_verify, max_wall_s)`	Orçamento: máximo de simulações, reserva para a verificação final e tempo total opcional.
`Limit`	classe	`(name, lower, upper, lower_frac_of_baseline, upper_frac_of_baseline, unit, kind, label)`	Limite `lower <= saída <= upper`, absoluto ou como fração do baseline, com unidade e tipo (físico, geometria, validade).
`Mission`	classe	`(title, statement, variables, objectives, constraints, baseline, budget, acceptance, cond…)`	O que o laboratório deve fazer; resolve limites relativos com as saídas do baseline e serializa de/para dict.

`pinneapple_autolab.qualification`

Qualificação do solver: roda o mesmo projeto em variantes de malha/iterações (fora do orçamento) e mede a diferença relativa de cada saída contra a referência. [sem teste direto]

`pinneapple_autolab/qualification.py` · 49 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`solver_qualification`	função	`(sim, params, variants, folder, reference, campaign, outputs=None) -> dict[str, Any]`	Roda as variantes num ledger próprio e devolve, por saída, a diferença relativa de cada variante em relação à `reference`.

`pinneapple_autolab.replay`

Simulador de replay: devolve a medição gravada quando o projeto está no ledger (sem interpolar) e roda o simulador real caso contrário.

`pinneapple_autolab/replay.py` · 68 linhas · testes: 6

nome	tipo	assinatura	o que faz
`ReplaySimulator`	classe	`(simulator, ledger_path, fields_dir=None, atol=1e-09)`	Envolve um simulador real e um ledger gravado; marca a origem do replay em `meta`.

`pinneapple_autolab.report`

Figuras e relatório de engenharia de uma sessão, só a partir do snapshot; nada aqui calcula um número físico. [sem teste direto]

`pinneapple_autolab/report.py` · 348 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`shrink_png`	função	`(path, colors=256) -> str`	Regrava um PNG com paleta adaptativa para reduzir o tamanho.

nome	tipo	assinatura	o que faz
pareto_front	função	(F) -> np.ndarray	Índices das linhas não dominadas (todas as colunas minimizadas), ordenados pela primeira coluna.
fig_pareto	função	(snap, path, x='drag_N', y='downforce_N', scale=0.001, units='kN', annotate=None) -> str	Figura arrasto x downforce com candidatos, frente do surrogate, projetos simulados, baseline e limite.
fig_models	função	(snap, path, outputs, labels=None, scale=None) -> str	Figura das previsões por validação cruzada contra a referência, com intervalos de 95 %, e notas das famílias.
fig_learning	função	(snap, path, outputs, objective, acceptance) -> str	Figura do erro de cada previsão feita antes da simulação e do melhor objetivo viável contra simulações gastas.
fig_field_triptych	função	(points, ref, pred, path, title='', clim=(-1.5, 1.0), err_max=0.3, label='Cp', mirror=Tru…)	Referência | previsão | |erro| de um campo de superfície, vistas de cima e de baixo.
fig_trust_cards	função	(cards, path, outputs, units=None) -> str	Um painel por caso de confiança: previsão ± 1,96 std, referência quando simulada e decisão.
write_report	função	(summary, path, figures=None, qualification=None, extra='') -> str	Relatório de engenharia em Markdown: missão, plano, qualificação, orçamento, modelos, decisões, recomendação e ambiente.

pinneapple_autolab.simulator

Contrato do simulador de referência (`check` antes, `run` com custo depois; falha nunca devolve saídas inventadas) e um simulador de função para testes. Os métodos da base levantam `NotImplementedError` (abstratos). [sem teste direto]

`pinneapple_autolab/simulator.py` · 91 linhas · **sem teste direto** · palavras-chave no código (heurística; o texto acima diz o status real): levanta `NotImplementedError` em algum caminho

nome	tipo	assinatura	o que faz
SimOutcome	classe	(outputs, ok, message, cost_s, fields, meta)	Resultado de uma simulação: saídas, sucesso, mensagem, custo, campos opcionais e metadados.
Simulator	classe	(name, fidelity, outputs, field_names, regions)	Classe base: define nome, fidelidade, saídas, campos e regiões; subclasses implementam `check` e `run`.
FunctionSimulator	classe	(fn, outputs, *, name='function', checker=None, fail=None, field_names=(), regions=None, …)	Simulador a partir de `fn(params) -> {saída: valor}`, com checador e falha simulada opcionais (testes, problemas analíticos).

pinneapple_autolab.surrogates

Surrogates com incerteza (GP Matern 5/2, ensemble de MLP, superfície quadrática, POD de campo) e a comparação por validação cruzada k-fold; a base `_Model` é abstrata (`NotImplementedError`). [sem teste direto]

`pinneapple_autolab/surrogates.py` · 260 linhas · **sem teste direto** · palavras-chave no código (heurística; o texto acima diz o status real): levanta `NotImplementedError` em algum caminho

nome	tipo	assinatura	o que faz
GPModel	classe	(restarts=1, seed=0)	Processo gaussiano Matern 5/2 com escalas ARD (scikit-learn), com desvio padrão preditivo.
MLPEnsemble	classe	(n=5, hidden=(32, 32), seed=0, max_iter=600)	n MLPs pequenos em reamostragens bootstrap (scikit-learn); desvio = espalhamento dos membros.
QuadraticRSM	classe	(alpha=1e-06)	Regressão ridge numa base quadrática completa; desvio = desvio residual do próprio ajuste.
ModelBoard	classe	(table, chosen, models, cv_pred, n_train, cv_y)	Tabela de notas por validação cruzada, família escolhida por saída e modelos ajustados.
FieldSurrogate	classe	(family='gp', energy=0.99, max_modes=12)	POD de campos em pontos fixos mais uma regressão por modo; desvio do campo vem dos modos.
make_model	função	(family, **kw) -> _Model	Cria um modelo da família pedida; família desconhecida levanta `ValueError`.
cross_validate	função	(family, U, y, k=5, seed=0) -> dict[str, Any]	Validação cruzada k-fold de uma família numa saída: métricas e previsões fora da amostra.
compare_models	função	(U, Y, families=…, k=5, seed=0, select='rmse') -> ModelBoard	Valida todas as famílias em todas as saídas, escolhe a de menor métrica por saída (penalizando as mal calibradas) e reajusta.

nome	tipo	assinatura	o que faz
`field_errors`	função	`(pred, ref, regions=None, std=None) -> dict[str, dict[str, float]]`	RMSE, erro máximo e cobertura de 95 % de um campo previsto, no total e por região.

`pinneapple_autolab.trust`

Triagem de candidatos: probabilidade de viabilidade, melhoria esperada restrita, domínio validado e erro de campo na região crítica, com quatro rótulos e a regra da próxima simulação. [sem teste direto]

`pinneapple_autolab/trust.py` · 262 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`CandidateAssessment`	classe	`(index, params, predicted, std, p_feasible, cei, domain, label, reasons, report, critical…)`	Avaliação de um candidato: previsão e desvio, P(viável), EI restrita, domínio, rótulo, motivos, relatório e, depois, a referência.
`NextExperiment`	classe	`(candidate, rule, reason, alternatives)`	A próxima simulação escolhida: candidato, regra, motivo e alternativas.
`assess_candidates`	função	`(X, names, pred, objectives, constraints, domain_verdicts, best_value, acceptance, model_…)`	Avalia os candidatos do otimizador e atribui `favourable`, `uncertain`, `out_of_domain` ou `unlikely`.
`choose_next`	função	`(cands, objective, exclude=()) -> NextExperiment`	Escolhe a próxima simulação pela regra escrita no docstring do módulo.
`verification_checks`	função	`(c, reference, ok, acceptance, constraints, message='') -> CandidateAssessment`	Atualiza uma avaliação com o resultado de referência: erros relativos medidos, confirmação e viabilidade real.

Exemplos de uso reais do repositório

Missão de brinquedo com simulador analítico — `tests/test_autolab.py`

```
def toy_sim(**kw):
    return FunctionSimulator(toy, ["drag", "down"], field_names=["cp"], regions={"tail": PTS > 0.7}, **kw)


def toy_mission(runs=12, reserve=1):
    return Mission("toy", "lower drag, keep downforce", [Variable("a", 0, 1), Variable("b", 0, 1)],
                   [Objective("drag", "min")], [Limit("down", lower_frac_of_baseline=1.0)],
                   baseline={"a": 0.5, "b": 0.5}, budget=Budget(runs, reserve),
                   acceptance={"drag": 0.03, "down": 0.03})
```

O laboratório na missão do carro (docstring do pacote) — `pinneapple_autolab/__init__.py`

```
from pinneapple_autolab import AutonomousLab
from pinneapple_autolab.domains.racecar import RaceCarAero, flagship_mission

lab = AutonomousLab(flagship_mission(), RaceCarAero(), "runs/aero", critical_region="diffuser")
summary = lab.run()
```

Testes e status de validação

- Módulos com pelo menos um teste que os importa: 6 de 16.
- Funções de teste em arquivos que importam algum módulo do pacote: 37.

Limitações conhecidas

- `domains.racecar` precisa de OpenFOAM (snappyHexMesh, simpleFoam); a campanha completa do exemplo leva horas em 4 núcleos (docstring de `examples/autolab_aero/autolab_aero.py`). Renders pedem PyVista e, para Cycles, `bpy`.
- `dashboard.capture` e `dashboard.record_replay` precisam de `playwright` (e ffmpeg para o vídeo).
- Os planejadores com LLM (`LLMPlanner`, `LLMStepPlanner`, `run_agent` com `provider`) precisam de chave/serviço de um provedor; sem isso usa-se o planejador determinístico.

- `LabPlanner` é declarado como substituto de um modelo de linguagem, não um agente inteligente: segue uma sequência fixa de etapas.
- `Simulator` e a base dos surrogates são abstratos (`NotImplementedError`); cada domínio novo precisa implementar `check` e `run`.
- Os surrogates dependem de scikit-learn; a recomendação final é sempre um projeto simulado, então o orçamento de simulações limita o que pode ser recomendado.
- `core`, `domain`, `ledger`, `mission`, `qualification`, `report`, `simulator`, `surrogates` e `trust` não têm teste direto (são exercitados pelos testes que importam o pacote raiz).

9. `pinneapple_app`

Objetivo

`pinneapple_app` é o aplicativo web de referência do PINNeAPPle: um backend FastAPI (`pinneapple_app/backend`) e um frontend React/Vite (`pinneapple_app/frontend`), com Dockerfiles, docker-compose, nginx e manifestos de deploy (Helm, k8s, monitoramento Prometheus). Ele deixa um usuário escolher um problema (preset do ambiente de EDPs ou equações próprias), gerar pontos de colocação e dados de referência, treinar vários modelos lado a lado e comparar os resultados.

O fluxo do backend é: `problem` define o problema; `collocation` e `data_pipeline` geram pontos e, opcionalmente, rodam solvers de `pinneapple_simulation`; `physics_adapter` transforma o problema em perda física (presets via `compile_problem`, equações customizadas via `SymbolicPDE` de `pinneapple_physics`); `experiment` treina cada modelo do `ModelRegistry` de `pinneapple_neural`, podendo re-treinar com o `TrainingAdvisor`; `surrogate_report` e `benchmark` produzem leaderboard, gráficos e um veredito de confiança medido em dados que o modelo não viu.

Os routers expõem isso como API REST e WebSocket (`/api/problems`, `/api/models`, `/api/experiments`), e um router admin protegido por token liga o app ao registro de artefatos (`pinneapple_registry`) para promover versões de modelo. É uma aplicação de demonstração: o armazenamento de experimentos é em memória e a autenticação admin é deliberadamente mínima.

Números do inventário

módulos	linhas	classes públicas	funções públicas	testes em arquivos que importam o pacote	módulos sem teste direto
18	3.672	29	53	48 (3 arquivos)	11

Submódulos

submódulo	o que faz	módulos	classes + funções	testes*
`backend`	Backend FastAPI: lógica de problemas, colocação, dados, física, experimentos e relatórios (`core`) e os routers da API (`routers`).	18	82	41

* maior número de testes que importam um módulo do submódulo.

Módulos e API pública

`pinneapple_app.backend`

Pacote do backend FastAPI do app (arquivo `__init__` vazio). [sem teste direto]

`pinneapple_app/backend/__init__.py` · 1 linhas · **sem teste direto**

`pinneapple_app.backend.core`

Subpacote com a lógica do backend: problemas, colocação, dados, física, experimentos, catálogo de modelos e relatórios (arquivo `__init__` vazio). [sem teste direto]

pinneapple_app/backend/core/__init__.py · 1 linhas · **sem teste direto**

pinneapple_app.backend.core.admin_auth

Portão de administração mínimo: um token compartilhado lido de PINNEAPPLE_ADMIN_TOKEN; sem a variável, as rotas admin respondem 503. [sem teste direto; exercitado via tests/test_admin_router.py]

pinneapple_app/backend/core/admin_auth.py · 31 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
require_admin	função	(x_admin_token=Header(default='')) -> None	Dependência FastAPI que exige o cabeçalho X-Admin-Token igual a PINNEAPPLE_ADMIN_TOKEN (401 se diferente, 503 se não configurado).

pinneapple_app.backend.core.benchmark

Gera o payload do benchmark de um ExperimentResult: leaderboard, gráficos PNG em base64 (matplotlib) e resumo em texto, trocando NaN/Inf por null. [sem teste]

pinneapple_app/backend/core/benchmark.py · 208 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
build_benchmark_payload	função	(result) -> Dict[str, Any]	Converte um ExperimentResult em dict JSON com leaderboard, gráficos em base64 e resumo.

pinneapple_app.backend.core.collocation

Geração de pontos de colocação (interior, contorno, inicial) num domínio caixa por uniforme, LHS, Sobol, Halton ou grade, ou por rejeição dentro de uma SDF.

pinneapple_app/backend/core/collocation.py · 208 linhas · testes: 7

nome	tipo	assinatura	o que faz
CollocationConfig	classe	(strategy, n_interior, n_boundary, n_initial, seed, use_geometry, geometry_sdf_fn, domain…)	Configuração da colocação: estratégia, quantidades de pontos, semente, uso de geometria/SDF, resolução de grade e parâmetros adaptativos.
generate_points	função	(config, bounds, *, include_time=False, t_range=(0.0, 1.0)) -> Dict[str, np.ndarray]	Gera pontos interiores, de contorno (com rótulo da face) e iniciais a partir da configuração e dos limites do domínio.

pinneapple_app.backend.core.data_pipeline

Pipeline de dados: opcionalmente roda um solver de pinneapple_simulation para gerar referência e amostra os pontos de colocação, devolvendo um DataBundle.

pinneapple_app/backend/core/data_pipeline.py · 163 linhas · testes: 7

nome	tipo	assinatura	o que faz
DataConfig	classe	(solver_key, n_snapshots, grid_resolution, t_end, use_solver)	Configuração da geração de dados: chave do solver, número de snapshots, resolução, tempo final e se usa o solver.
DataBundle	classe	(x_col, x_bnd, x_ic, u_ref, solver_outputs, meta)	Saída do pipeline: pontos de colocação, contorno e iniciais, referência interpolada, saídas do solver e metadados.
run_data_pipeline	função	(problem, collocation_cfg, data_cfg, *, device='cpu', verbose=True) -> DataBundle	Roda o solver (se pedido) e amostra os pontos, devolvendo um DataBundle.

pinneapple_app.backend.core.experiment

Executor de experimentos: treina vários modelos no mesmo problema, transmite progresso por gerador assíncrono, pode re-treinar com sugestões do TrainingAdvisor e mede tudo em dados separados.

pinneapple_app/backend/core/experiment.py · 475 linhas · testes: 7 · palavras-chave no código (heurística; o texto acima diz o status real): experimental

nome	tipo	assinatura	o que faz
ModelRunConfig	classe	(name, extra_kwargs, weight_override)	Configuração de um modelo dentro do experimento: nome, kwargs extras e sobrescrita de pesos.
ExperimentConfig	classe	(experiment_id, problem_name, models, metrics, epochs, lr, device, batch_size, seed, grad…)	Configuração completa de um experimento: problema, modelos, métricas, épocas, taxa, dispositivo, semente e opções de re-treino automático.
ModelResult	classe	(name, metrics, loss_history, physics_loss_history, bc_loss_history, train_time_s, n_para…)	Resultado de um modelo: métricas, históricos de perda, tempo, número de parâmetros, estado, erro, rodadas de re-treino, diagnóstico e relatório.
ExperimentResult	classe	(experiment_id, config, model_results, completed_at)	Resultado do experimento com os resultados por modelo; leaderboard ordena por L2 relativo em dados separados e depois por resíduo em pontos não vistos.
ExperimentRunner	classe	(config, data_bundle, problem)	Treina todos os modelos configurados e emite eventos de progresso por um gerador assíncrono.

pinneapple_app.backend.core.model_catalog

Catálogo de modelos: lista arquiteturas do ModelRegistry enriquecidas com metadados estáticos (família, descrição, suporte a perda física) e recomenda por família de problema. [sem teste]

pinneapple_app/backend/core/model_catalog.py · 195 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
ModelEntry	classe	(name, family, description, supports_physics_loss, tags, recommended_for)	Entrada do catálogo: nome, família, descrição, suporte a perda física, tags e problemas recomendados.
list_models	função	(family=None) -> List[ModelEntry]	Lista os modelos disponíveis, opcionalmente filtrando por família.
recommend_for_problem	função	(problem_family, n=5) -> List[str]	Devolve os N modelos recomendados para uma família de problema.
get_model_info	função	(name) -> Optional[ModelEntry]	Devolve a entrada de um modelo pelo nome, ou None.

pinneapple_app.backend.core.physics_adapter

Transforma um ProblemDefinition numa perda física treinável: PresetPhysics monta o lote que compile_problem espera e CustomPhysics compila equações do usuário com SymbolicPDE e impõe condições Dirichlet/Neumann/iniciais; substitui um antigo placeholder que zerava a física.

pinneapple_app/backend/core/physics_adapter.py · 552 linhas · testes: 7 · palavras-chave no código (heurística; o texto acima diz o status real): placeholder/stub citado no docstring; levanta NotImplementedError em algum caminho

nome	tipo	assinatura	o que faz
ConditionInfo	classe	(key, name, kind, fields, weight, n_points, how, target_rms)	Descrição de uma condição de contorno/inicial aplicada: chave, tipo, campos, peso, número de pontos, como foi imposta e RMS-alvo.
PhysicsAdapter	classe	()	Interface comum de física usada pelo executor e pelo relatório; build_batch e loss levantam NotImplementedError na base.
PresetPhysics	classe	(spec, *, w_physics=None, w_bc=None)	Adaptador para presets: monta o lote esperado por compile_problem com pontos exatamente nas faces da caixa e máscaras por condição.
CustomPhysics	classe	(problem, *, w_physics=None, w_bc=None)	Adaptador para problemas customizados: compila as equações com SymbolicPDE e impõe BCs Dirichlet/Neumann e ICs.
unwrap	função	(out) -> torch.Tensor	Converte a saída do modelo (tensor ou objeto com .y) num tensor (N, F).
spatial_coords	função	(coords) -> List[str]	Lista as coordenadas espaciais (exclui o tempo).
all_faces	função	(coords) -> List[Face]	Lista todas as faces min/max da caixa espacial.

nome	tipo	assinatura	o que faz
`faces_for_tag`	função	`(tag, coords) -> Optional[List[Face]]`	Traduz tags de contorno conhecidas em faces da caixa; None para tags que não seguem essa convenção.
`sample_faces`	função	`(bounds, coords, faces, n, rng) -> Tuple[np.ndarray, np.ndarray]`	Amostra pontos uniformes nas faces dadas, proporcionais à área, com normais unitárias externas.
`sample_initial`	função	`(bounds, coords, n, rng) -> np.ndarray`	Amostra pontos no instante inicial do domínio.
`sample_interior`	função	`(bounds, coords, n, rng) -> np.ndarray`	Amostra pontos uniformes no interior do domínio.
`characteristic_length`	função	`(bounds, coords) -> float`	Extensão espacial média do domínio (extensão do tempo para EDOs).
`parse_equation`	função	`(expr, coords, fields)`	Converte uma string como `u_t + u*u_x - 0.01*u_xx` numa expressão SymPy.
`parse_location`	função	`(location, bounds, coords) -> List[Face]`	Converte descrições de local (`all`, `x=0`, `x_min`, `left`…) em faces da caixa.
`build_physics`	função	`(problem, *, weight_override=None) -> Optional[PhysicsAdapter]`	Escolhe e cria o adaptador de física do problema, ou None quando ele não define física.

`pinneapple_app.backend.core.problem`

Camada de definição de problema: carrega presets do ambiente de EDPs de `pinneapple_physics` ou monta problemas customizados a partir de equações em texto, BCs e ICs.

`pinneapple_app/backend/core/problem.py` · 184 linhas · testes: 7

nome	tipo	assinatura	o que faz
`BoundaryConditionSpec`	classe	`(kind, location, value, field)`	Condição de contorno legível para problemas customizados: tipo, local, valor e campo.
`InitialConditionSpec`	classe	`(expression, field)`	Condição inicial legível: expressão e campo.
`EquationSpec`	classe	`(expression, label, field)`	Uma equação de resíduo da EDP (string ou SymPy), com rótulo e campo.
`ProblemDefinition`	classe	`(kind, name, spec, dim, domain_bounds, pde_family, is_time_dependent, preset_name, equati…)`	Contêiner único de problema do app (preset ou customizado): domínio, família, dependência do tempo, equações, BCs e ICs.
`load_preset`	função	`(preset_name) -> ProblemDefinition`	Carrega um `ProblemDefinition` a partir do registro de presets de `pinneapple_physics.pde_environment`.
`define_custom`	função	`(name, equations, bcs, ics=None, domain_bounds=None, dim=2, pde_family=FAMILY_GENERIC, is…)`	Monta um `ProblemDefinition` a partir de equações, BCs e ICs do usuário, sem exigir `ProblemSpec`.

`pinneapple_app.backend.core.problem_registry`

Registro estático de metadados por preset (família, descrição, tags, solver e modelos recomendados), com valores padrão para presets desconhecidos. [sem teste]

`pinneapple_app/backend/core/problem_registry.py` · 469 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`get_problem_meta`	função	`(preset_name) -> Dict[str, Any]`	Devolve os metadados de um preset, com padrões para presets desconhecidos.
`recommend_models`	função	`(preset_name) -> list`	Lista de modelos recomendados para o preset.
`recommended_solver`	função	`(preset_name) -> str`	Chave do solver recomendado para o preset.
`all_problems`	função	`() -> list`	Lista todos os presets com metadados.

pinneapple_app.backend.core.surrogate_report

Relatório "posso confiar neste modelo?": erro em teste separado, convergência, generalização, variáveis, pesos, incerteza (ensemble de snapshots e conformal) e checagens físicas, com veredito final.

pinneapple_app/backend/core/surrogate_report.py · 552 linhas · testes: 7

nome	tipo	assinatura	o que faz
error_section	função	(model, fields, X_test, Y_test, reference_source) -> Dict[str, Any]	Seção de erro: L2 relativo e erro absoluto por campo contra a referência no split de teste.
convergence_section	função	(loss_history) -> Dict[str, Any]	Seção de convergência: classifica o histórico de perda (convergiu, estagnou, divergiu, ainda melhorando, oscilando).
generalization_section	função	(model, X_train, Y_train, X_test, Y_test, res_train, res_unseen) -> Dict[str, Any]	Seção de generalização: erro treino vs teste e resíduo da EDP em treino vs pontos não vistos.
variables_section	função	(model, adapter, X_eval, supervised) -> Dict[str, Any]	Seção de variáveis: entradas, saídas, faixas previstas e sensibilidade às entradas.
weights_section	função	(model, adapter, final_raw, w_data, data_loss) -> Dict[str, Any]	Seção de pesos: pesos dos termos de perda, participação de cada termo e saúde dos parâmetros da rede.
uncertainty_section	função	(model, snapshots, fields, X_eval, X_cal, Y_cal, X_test, Y_test, coverage=0.9) -> Dict[st…)	Seção de incerteza: dispersão de snapshots de pesos do fim do treino e intervalos conformal com cobertura medida.
field_scales	função	(adapter, pred) -> np.ndarray	Escala de cada campo: máximo entre a dispersão prevista, a faixa declarada e o RMS dos valores impostos.
residual_scale	função	(model, adapter, X, pred) -> float	Tamanho típico dos termos da EDP, para normalizar o resíduo.
physics_checks_section	função	(model, adapter, X_eval, raw_unseen, raw_initial, pred_eval) -> List[Dict[str, Any]]	Checagens físicas (resíduo, condições de contorno/iniciais, conservação de massa, balanço de energia, limites físicos) com pass/warn/fail.
verdict	função	(report) -> Dict[str, Any]	Veredito final (trustworthy, use_with_caution ou not_trustworthy) com as razões.
build_report	função	(*, model, adapter, loss_history, final_raw_train, raw_initial, raw_unseen, res_train, X_…)	Monta todas as seções do relatório e o veredito.

pinneapple_app.backend.main

Ponto de entrada FastAPI: registra os routers, expõe /api/health, /api/metrics (Prometheus feito à mão) e /api/info, e serve o frontend React compilado quando existe.

pinneapple_app/backend/main.py · 112 linhas · testes: 41

nome	tipo	assinatura	o que faz
health	função	()	Endpoint de saúde do servidor.
metrics	função	() -> str	Endpoint Prometheus com tempo de atividade e total de requisições.
info	função	()	Resumo das capacidades: número de modelos e problemas, famílias e lista de recursos.

pinneapple_app.backend.routers

Subpacote dos routers da API (arquivo __init__ vazio). [sem teste direto]

pinneapple_app/backend/routers/__init__.py · 1 linhas · **sem teste direto**

pinneapple_app.backend.routers.admin

Router admin protegido por require_admin: resumo cruzando catálogo de problemas, registro de modelos (pinneapple_registry) e cenários de dados, e promoção de versão de modelo. [sem teste direto; exercitado via tests/test_admin_router.py]

pinneapple_app/backend/routers/admin.py · 84 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ModelSummaryItem`	classe	`(problem_id, latest_model_version, stage, dataset_scenarios)`	Item do resumo admin: problema, última versão de modelo, estágio e cenários de dados.
`AdminSummaryResponse`	classe	`(registry_root, n_presets, n_architectures, models)`	Resposta do resumo admin: raiz do registro, número de presets e arquiteturas, e modelos.
`PromoteRequest`	classe	`(problem_id, version, stage)`	Pedido de promoção: problema, versão e novo estágio.
`admin_summary`	função	`() -> AdminSummaryResponse`	Monta o resumo admin cruzando presets, registro de modelos e cenários.
`promote_model`	função	`(req) -> Dict[str, Any]`	Promove uma versão de modelo armazenada para um novo estágio no `ArtifactRegistry`.

`pinneapple_app.backend.routers.experiments`

Router de experimentos: lança em background, consulta status e resultados, lista e transmite progresso por WebSocket; guarda tudo num dict em memória. [sem teste direto]

`pinneapple_app/backend/routers/experiments.py` · 266 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`CollocationRequest`	classe	`(strategy, n_interior, n_boundary, n_initial, seed, use_geometry, domain_name)`	Corpo da requisição para configurar a colocação.
`DataRequest`	classe	`(solver_key, n_snapshots, grid_resolution, t_end, use_solver)`	Corpo da requisição para configurar a geração de dados.
`ModelRequest`	classe	`(name, extra_kwargs)`	Corpo da requisição para um modelo do experimento.
`ExperimentRequest`	classe	`(problem_name, custom_problem, collocation, data, models, metrics, epochs, lr, device, se…)`	Corpo completo para lançar um experimento (problema preset ou customizado, colocação, dados, modelos, métricas, treino).
`ExperimentStatusResponse`	classe	`(experiment_id, status, progress, message)`	Resposta de status: id, estado, progresso e mensagem.
`launch_experiment`	função	`(req, background_tasks)`	Lança um experimento em background e devolve o id na hora.
`get_status`	função	`(experiment_id)`	Devolve o status de um experimento.
`get_results`	função	`(experiment_id)`	Devolve os resultados de um experimento concluído.
`list_experiments`	função	`()`	Lista os experimentos guardados em memória.
`experiment_ws`	função	`(websocket, experiment_id)`	WebSocket que transmite eventos de progresso do treino em JSON.

`pinneapple_app.backend.routers.models`

Router de modelos: lista modelos e famílias, recomenda por família, lista métricas e devolve metadados de um modelo. [sem teste direto]

`pinneapple_app/backend/routers/models.py` · 84 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ModelListItem`	classe	`(name, family, description, supports_physics_loss, tags, recommended_for)`	Item da lista de modelos na resposta da API.
`MetricItem`	classe	`(key, label)`	Métrica disponível (chave e rótulo).
`list_models`	função	`(family=None)`	Lista modelos, opcionalmente por família.
`list_families`	função	`()`	Lista as famílias de modelos.
`recommend_models`	função	`(problem_family, n=5)`	Devolve os N modelos recomendados para uma família de problema.

nome	tipo	assinatura	o que faz
`list_metrics`	função	`()`	Lista as métricas de avaliação disponíveis.
`get_model`	função	`(name)`	Devolve metadados de um modelo (404 se não existir).

`pinneapple_app.backend.routers.problems`

Router de problemas: lista presets com metadados, detalha um preset e valida um problema customizado sem rodá-lo. [sem teste direto]

`pinneapple_app/backend/routers/problems.py` · 86 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`PresetListItem`	classe	`(name, family, description, tags, time_dependent, recommended_models, recommended_solver)`	Item da lista de presets: nome, família, descrição, tags, dependência do tempo e recomendações.
`CustomProblemRequest`	classe	`(name, equations, boundary_conditions, initial_conditions, domain_bounds, dim, pde_family…)`	Corpo para definir um problema customizado (equações, BCs, ICs, domínio, dimensão, família).
`list_problems`	função	`()`	Lista todos os presets com metadados.
`get_problem`	função	`(name)`	Devolve os metadados de um preset.
`validate_custom_problem`	função	`(req)`	Valida a definição de um problema customizado sem rodá-lo.

Exemplos de uso reais do repositório

Métricas e portão admin pela API — `tests/test_admin_router.py`

```
        resp = client.get("/api/admin/summary", headers={"X-Admin-Token": "test-token"})
```

Testes e status de validação

- Módulos com pelo menos um teste que os importa: 7 de 18.
- Funções de teste em arquivos que importam algum módulo do pacote: 48.

Limitações conhecidas

- Os experimentos ficam num dict em memória (`_EXPERIMENTS`); o próprio código diz para trocar por banco de dados em produção.
- A autenticação admin é um único token compartilhado em variável de ambiente; o docstring recomenda um provedor de autenticação real para uso multiusuário.
- `collocation.STRATEGIES` lista `adaptive` e `meshfree`, mas a amostragem interior só implementa uniforme, LHS, Sobol, Halton e grade; outras estratégias caem em LHS.
- Condições de contorno que nomeiam geometria real (tags fora da convenção de faces da caixa) não são aplicadas; ficam listadas como `unresolved` para o relatório.
- `PhysicsAdapter` é base abstrata (`build_batch` e `loss` levantam NotImplementedError).
- Vários módulos não têm teste direto (`benchmark`, `model_catalog`, `problem_registry` e os routers de experimentos, modelos e problemas).
- Depende de FastAPI; gráficos do benchmark usam matplotlib; o frontend só é servido se `frontend/dist` tiver sido compilado.

Volume 8 — Apps, exemplos, reproduções, scripts e CI

> Parte do Mapa completo do PINNeAPPle. Gerado por `docs/pt/_mapa/gerar.py` a partir do inventário do commit `e53cdcfb04`.

O que cada app, exemplo, reprodução de paper, script, benchmark e workflow de CI faz. A coluna *testes* conta funções de teste em arquivos que citam o caminho.

1. Apps (`apps/`)

64 itens.

item	o que faz	arquivos .py	testes
`apps/_shared`	Não é um app: é a cola (`appkit.py`) que todo app em `apps/` importa, com rotas HTTP comuns, login, fila de jobs, chaves de API, medição de uso e a vista 3-D de campos (`/shared/fieldview3d.js`). A lógica fica na biblioteca (`pinneapple_core.service`, `pinneapple_data.contracts`, `pinneapple_security`) e tem testes (por exemplo `tests/test_appkit_runtime.py`). Entra no `PYTHONPATH` de cada Dockerfile.	1	23
`apps/aero_optimizer`	Otimizador de projeto de avião de corredor único (classe A320/737): o NSGA-II busca 13 números (seção da asa em CST, área, alongamento, afilamento, enflechamento, torção, posição e Mach de cruzeiro) para menos CO_2 por passageiro-km, com checagem em OpenFOAM 3D e campos de escoamento. Usa sobretudo `pinneapple_design` (e `pinneapple_tools`). Roda com `cd apps/aero_optimizer && uvicorn aero_optimizer.api:app --port 8092`; nenhum teste cita a pasta.	9	0
`apps/anomaly_detector`	Detector de anomalias para engenheiros de planta: combina estatística robusta, modelo de séries temporais e balanços de massa e energia para dizer qual balanço quebrou, quanto e em quais sensores. A lógica é `pinneapple_systems.operations.anomaly` (com `pinneapple_data` e `pinneapple_core`). Roda com `uvicorn anomaly_detector.api:app --port 8111` (comando do Dockerfile) ou `docker run -p 8111:8111 anomaly-detector`.	2	4
`apps/benchmark_arena`	Arena de benchmarks de IA física: compara, sob um só protocolo, ML clássico, MLP, FNO, DeepONet, PINN e solvers numéricos em problemas públicos (campanha OpenFOAM de aerofólios do zoo, Burgers viscoso) e na tabela de runs do próprio usuário. Usa `pinneapple_arena.regimes.run_regime_arena`, `pinneapple_analysis` e `pinneapple_core`. Roda com `uvicorn benchmark_arena.api:app --port 8105` (Dockerfile); os resultados públicos se refazem com `python run_public_benchmarks.py --budget 10 --seeds 3`.	3	5
`apps/brakecool_ai`	Projeto inverso das aletas radiais de um disco de freio ventilado (número de aletas, folga e espessura) por um surrogate neural congelado, sempre reconferido contra a física, mais um Monte Carlo das tolerâncias de usinagem. A lógica está em `pinneapple_design.surrogate_inverse.brake_rotor`; a pasta só tem rotas e a página. Roda com `cd apps/brakecool_ai && PYTHONPATH=../..:../_shared uvicorn brakecool_ai.api:app --port 8161`.	2	5
`apps/calibration_lab`	Laboratório de calibração: ajusta parâmetros de um modelo a medições reais, dá intervalos só onde os dados os determinam e confere o ajuste em medições que ele não viu. Usa `pinneapple_analysis`. Roda com `cd apps/calibration_lab && uvicorn calibration_lab.api:app --port 8101` ou pela imagem Docker.	2	4
`apps/control_lab`	Laboratório de controle baseado em física: compara MPC linear (sobre os balanços de massa e energia) e outras estratégias com o que roda hoje (PID, liga/desliga). Funciona só em modo de simulação ou recomendação; não escreve em nenhum controlador real. A lógica é `pinneapple_systems.operations.control`; roda com `uvicorn control_lab.api:app --port 8112` (Dockerfile).	2	4
`apps/data_health`	Recebe um conjunto de dados (CSV, Excel, Parquet, JSON, HDF5), reconhece a grandeza física e a unidade de cada coluna e devolve um relatório de qualidade com nota, evidências e correções. Usa `pinneapple_data` e `pinneapple_core`. Roda com `cd apps/data_health && uvicorn data_health.api:app --port 8082`; nenhum teste cita a pasta.	2	0
`apps/data_standardizer`	Junta até 8 arquivos de fontes diferentes (unidades, fusos e convenções distintas) num só esquema físico em UTC e unidades coerentes, com uma receita JSON reaproveitável e checagem de concordância entre fontes. Usa `pinneapple_data`. Roda com `cd apps/data_standardizer && uvicorn data_standardizer.api:app --port 8083`; nenhum teste cita a pasta.	2	0
`apps/deploy`	Não é um app: são os arquivos para publicar vários apps numa só VM Linux, cada um no seu subdomínio, atrás de um Caddy com certificados Let's Encrypt (`docker-compose.yml`, variantes de pouca memória e de proxy existente, `Caddyfile`, `fail2ban`, `check_server.sh`). Sobe com `docker compose up -d`; a senha de login vem de `python -m pinneapple_security hash-password`.	0	9

item	o que faz	arquivos .py	testes
apps/design_optimization	Caso fixo de demonstração: uma seção de asa da família NACA 2418 redesenhada ao vivo para menos arrasto com a mesma sustentação, com passos de Adam através de um surrogate MLP e cada iterado reavaliado pela fórmula fechada. Usa `pinneapple_design.design_optimizer`. Roda com `PYTHONPATH=../_shared uvicorn design_optimization.api:app --port 8181`.	2	2
apps/design_requirements	Form Compiler: lê folhas de dados e especificações (PDF, texto, páginas escaneadas via OCR) e compila um formulário ou folha de dados, com a origem de cada valor e a lista do que falta ou se contradiz. Usa `pinneapple_data`. Roda com `cd apps/design_requirements && UDR_FORM_PDF=/caminho/u-dr-1.pdf uvicorn design_requirements.api:app --port 8085`; nenhum teste cita a pasta.	3	0
apps/engineering_optimizer	Otimização caixa-preta de projetos dentro de limites físicos e de fabricação, sobre um modelo rápido, com o ótimo confirmado por um modelo de alta fidelidade. Usa `pinneapple_design` e `pinneapple_data`. Roda com `cd apps/engineering_optimizer && PYTHONPATH=../_shared uvicorn engineering_optimizer.api:app --port 8100`.	2	3
apps/evidence_report	Transforma saídas de experimentos e simulações (relatórios do Validation Lab, ledgers de evidência, `pinneapple check --json`, tabelas de métricas) num relatório técnico reprodutível; um teste só conta como aprovado se de fato rodou. Usa `pinneapple_veriphysics`. Roda com `cd apps/evidence_report && uvicorn evidence_report.api:app --port 8113`.	2	5
apps/flow_studio	Canvas de nós no estilo n8n para fluxos de engenharia: descreve-se o problema, recebe-se um ou mais fluxos, editam-se equações e métodos, roda-se e inspeciona-se cada nó (métricas, figuras, vista 3-D). Toda a lógica é `pinneapple_flow`; a pasta tem só o shell HTTP e o frontend React. Roda com `python -m pinneapple_flow studio` (porta 8765); os mesmos fluxos rodam sem GUI com `python -m pinneapple_flow run flow.json`.	1	19
apps/flowtrace	Recebe um vídeo ou uma sequência de imagens de um escoamento com partículas e devolve o campo de velocidade por PIV de correlação cruzada. Usa `pinneapple_perception.video_piv` e `pinneapple_perception.piv_tools`. Roda com `cd apps/flowtrace && uvicorn flowtrace.api:app --port 8145` (vídeos pedem `ffmpeg`).	2	3
apps/governed_card	Emite um model card só depois de uma checagem física do modelo: treina (ou carrega) o modelo de um preset, roda o `PhysicsGuardrail` e monta um `ModelCard` cujas métricas são as daquele run; um card inválido é recusado. Usa `pinneapple_hub.verified_card`, `pinneapple_hub.model_card` e `pinneapple_physics`. Roda com `cd apps/governed_card && uvicorn governed_card.api:app --port 8148`.	2	3
apps/guiagent	Agente que executa estudos de engenharia em software com interface gráfica (CAD, pré-processador, solver, pós) dentro do seu contêiner, a partir de uma frase ou de um workflow, e devolve os arquivos com a evidência de cada passo; há uma aprovação antes do solver. Usa `pinneapple_tools.automation` e `pinneapple_security`. Roda com `docker build -f apps/guiagent/Dockerfile -t guiagent .` e `docker run -p 8115:8115 guiagent`.	2	2
apps/heatsink_inverse_ai	Demo de projeto inverso por gradiente de um dissipador de aletas retas para CPU (número, altura e espessura das aletas numa base fixa de 80 x 80 mm) através de um surrogate neural congelado, reconferido contra a física. O próprio README diz que não dimensiona um dissipador real (para isso existe `heatsink_sizer`). Lógica em `pinneapple_design.surrogate_inverse.heatsink_fins`; roda com `PYTHONPATH=../..:../_shared uvicorn heatsink_inverse_ai.api:app --port 8160`.	2	5
apps/heatsink_sizer	Protótipo SaaS (dia 1 da série) que dimensiona dissipadores de placas e aletas a partir de correlações publicadas de convecção forçada e natural, perda de carga e uma solução 3-D da base, e devolve projetos fabricáveis verificados pela física. Usa `pinneapple_design` e `pinneapple_physics`. Roda com `uvicorn heatsink_sizer.api:app --port 8080` ou `docker run -p 8080:8080 heatsink-sizer`; nenhum teste cita a pasta.	7	0
apps/helio_tfc	Estima para onde vai o envelope do ciclo solar: EMD do número diário de manchas solares, amplitude de Hilbert do componente de cerca de 11 anos e um ajuste por máquina de aprendizado extremo com valores de ponta amarrados (Theory of Functional Connections). O README avisa que é um método, não uma previsão validada. Usa `pinneapple_systems.time_series.solar_cycle`; roda com `cd apps/helio_tfc && uvicorn helio_tfc.api:app --port 8149`.	2	4
apps/hub	Catálogo de ferramentas prontas, modelos treinados, geometrias, datasets, fluxos e exemplos: buscar, ver o run gravado, copiar código, baixar arquivos, abrir no Flow Studio e rodar um modelo do zoo. Tudo vem do índice gerado por `pinneapple_hub.library`; a pasta só tem rotas e página. Roda com `PYTHONPATH=../..:../_shared:. uvicorn hub.api:app --port 8116` dentro de `apps/hub`.	2	14
apps/identifiability_lab	Antes de estimar parâmetros, verifica se as medições conseguem determiná-los (por exemplo, numa aleta só h/k é identificável) e diz o que medir em vez disso. Usa `pinneapple_analysis`. Roda com `cd apps/identifiability_lab && uvicorn identifiability_lab.api:app --port 8108`.	2	4
apps/interop_hub	Converte resultados de simulação (OpenFOAM, CalculiX, VTK/VTU, Gmsh, STL, CSV/HDF5/NPZ) num dataset físico neutro (`pinneapple.physical_dataset/1`, com unidades e metadados) e dali para VTU, HDF5, Parquet, CSV, NPZ, JSON ou Zarr. Usa `pinneapple_data`. Roda com `cd apps/interop_hub && uvicorn interop_hub.api:app --port 8091`; nenhum teste cita a pasta.	2	0
apps/inverse_heat	Estima o coeficiente de convecção h a partir de poucos termopares com uma PINN montada pelo `PINNFactory` (h é um parâmetro treinável), em 1D, 2D e 3D, e confere cada resposta contra uma solução independente. Usa `pinneapple_physics` e `pinneapple_analysis`. Roda com `cd apps/inverse_heat && uvicorn inverse_heat.api:app --port 8086 --workers 1`; nenhum teste cita a pasta.	7	0

item	o que faz	arquivos .py	testes
apps/inverse_studio	Um mesmo problema inverso (por exemplo o campo h(x) de uma aleta) resolvido por cinco métodos com os mesmos dados, cada um conferido. Reusa a física do Inverse Heat Lab via pinneapple_analysis.inverse_problems. Roda com cd apps/inverse_studio && uvicorn inverse_studio.api:app --port 8104.	2	3
apps/mesh_quality	Lê uma malha (OpenFOAM, Gmsh, VTK, STL, Abaqus/CalculiX e outros formatos meshio) e diz em segundos onde ela é ruim, quanto, e como corrigir, com veredito PASS/WARNING/FAIL e regiões problemáticas. Usa pinneapple_data. Roda com cd apps/mesh_quality && uvicorn mesh_quality.api:app --port 8088; nenhum teste cita a pasta.	2	0
apps/meshbridge	Recebe uma malha de CFD, FEM ou CAD (CGNS, Exodus II, Fluent ASCII, Abaqus .inp e outras) e devolve pontos e campos em NPZ ou uma superfície triangular reparada (STL/OBJ/PLY). Usa pinneapple_simulation.external_solvers.cfd_formats.convert. Roda com cd apps/meshbridge && uvicorn meshbridge.api:app --port 8141.	2	4
apps/meshless_lab	Mede, no problema do usuário (Poisson, Helmholtz ou calor em vários domínios ou numa nuvem de pontos), onde métodos sem malha, com malha e neurais ganham uns dos outros. Usa pinneapple_simulation e pinneapple_core. Roda com cd apps/meshless_lab && uvicorn meshless_lab.api:app --port 8109.	2	5
apps/model_benchmarking	Comparação fixa: quatro arquiteturas PINN (Vanilla, SIREN, PIELM, XTFC) treinadas ao vivo no mesmo Burgers viscoso burgers_1d e ranqueadas por precisão e por uma checagem do resíduo da EDP. Usa pinneapple_arena. Roda com PYTHONPATH=../_shared uvicorn model_benchmarking.api:app --port 8182; o estado fica em memória e se perde ao reiniciar.	2	2
apps/model_gateway	Publica modelos físicos (do zoo, ONNX, model.zip do Surrogate Studio ou remotos) atrás de endpoints versionados que checam cada consulta contra a faixa de validade, registram e medem o uso. Usa pinneapple_registry.ModelGateway e pinneapple_security. Roda com uvicorn model_gateway.api:app --port 8114 (Dockerfile).	2	6
apps/model_lineage	Mostra como um resultado foi produzido: o fio digital do CAD à previsão, com arquivo, versão, software, parâmetros, data, sha256 e origem de cada artefato, como um grafo consultável. Usa pinneapple_data. Roda com cd apps/model_lineage && uvicorn model_lineage.api:app --port 8090; nenhum teste cita a pasta.	2	0
apps/nozzle_screen	Triagem rápida de bocais de foguete: estimativa ideal isentrópica 1-D (pinneapple_physics.closed_form.rocket_nozzle) e uma checagem cruzada por PINN que avisa quando as duas discordam (pinneapple_analysis.verification.nozzle_crosscheck). Roda com cd apps/nozzle_screen && uvicorn nozzle_screen.api:app --port 8146.	2	2
apps/observatory	O céu e o sistema solar agora, vistos de onde o usuário está, em 3-D: estrelas do Yale Bright Star Catalogue, constelações, Sol, Lua, planetas e satélites, com modo de apontar o celular. Os números vêm de pinneapple_physics.astro. Roda com cd apps/observatory && uvicorn observatory.api:app --port 8094; os testes de navegador ficam na própria pasta (npx playwright test, workflow observatory.yml), e nenhum teste Python cita a pasta.	3	0
apps/orbitwatch	Compara duas imagens de satélite do mesmo lugar e gera um relatório HTML com a mudança de refletância por banda e de NDVI. Usa pinneapple_perception.change_detection e, para buscar Sentinel-2 no Google Earth Engine, pinneapple_perception.sentinel2. Roda com cd apps/orbitwatch && uvicorn orbitwatch.api:app --port 8150.	2	5
apps/parameter_discovery	Problema-teste fixo: uma PINN recupera a difusividade escondida de uma barra (equação do calor) a partir de leituras ruidosas, comparando a estimativa com o limite de Cramér-Rao. Usa pinneapple_analysis. Roda com PYTHONPATH=../_shared uvicorn parameter_discovery.api:app --port 8183.	2	2
apps/patchdose_ai	Exercício de transferência de massa (o README diz que não é ferramenta médica nem de dosagem): acha o menor adesivo de liberação por membrana para uma taxa e um tempo de uso, por um surrogate neural reconferido contra a lei de Fick. Lógica em pinneapple_design.surrogate_inverse.transdermal_patch; roda com PYTHONPATH=../..:../_shared uvicorn patchdose_ai.api:app --port 8162.	2	4
apps/pcb_hotspot	Protótipo SaaS (dia 2 da série) de análise térmica de placas de circuito: temperaturas de junção, pontos quentes e o que os corrige, em cerca de um segundo. Usa pinneapple_physics. Roda com uvicorn pcb_hotspot.api:app --port 8081 ou docker run -p 8081:8081 pcb-hotspot; nenhum teste cita a pasta.	4	0
apps/physics_copilot	Um problema de física em linguagem natural vira uma PINN treinada e o relatório recalculado do guardrail: análise dimensional, escolha de um preset registrado pelo LLM, treino e checagem do resíduo em pontos novos. Usa pinneapple_analysis.verification.pipeline (com pinneapple_llm). Roda com cd apps/physics_copilot && uvicorn physics_copilot.api:app --port 8142.	2	3
apps/physics_design	Otimiza projetos físicos paramétricos com objetivos e restrições explícitos e compara métodos por gradiente, evolutivos e bayesianos com o mesmo orçamento, mostrando quais candidatos o modelo de alta fidelidade confirma. Usa pinneapple_design. Roda com cd apps/physics_design && PYTHONPATH=../_shared uvicorn physics_design.api:app --port 8106.	2	2
apps/pinn_arena	Escolhe-se um problema físico e algumas arquiteturas PINNeAPPle, treina-se sob um protocolo, compara-se num conjunto separado e monta-se um leaderboard; uma segunda seção roda bibliotecas de terceiros pelo próprio pacote delas. Lógica em pinneapple_arena (backend FastAPI e frontend React). Roda com cd apps/pinn_arena && PYTHONPATH=../_shared uvicorn pinn_arena.api:app --port 8130 (o frontend se compila antes com npm run build).	21	25

item	o que faz	arquivos .py	testes
apps/playground	Site estático: desenhar um aerofólio, dimensionar um dissipador ou dirigir uma roda de rover lunar e ver um modelo do zoo responder no navegador (ONNX), com checagens físicas e veredito de confiança. Os modelos e checagens vêm de pp.zoo.export_web (pinneapple_hub.zoo.web) e pinneapple_design.geometry_library.web. Roda com python apps/playground/build.py --serve; testes JS com npm test e workflow playground.yml.	1	2
apps/predictive_twin	Gêmeo digital de um medidor real de rio (USGS 01646500, Potomac): baixa os últimos dias de vazão da API pública do USGS, calibra uma recessão exponencial, prevê as últimas 24 h e sinaliza quando o rio se afasta da previsão. Usa pinneapple_systems. Roda com PYTHONPATH=../_shared uvicorn predictive_twin.api:app --port 8184 (precisa de internet).	2	3
apps/ridetune_ai	Acha a seção mais leve de um braço de suspensão (modelado como viga em balanço) para uma meta de vida em fadiga, por um surrogate neural da tensão reconferido contra a fórmula fechada, com sensibilidades locais. Lógica em pinneapple_design.surrogate_inverse.suspension_arm; roda com PYTHONPATH=../..:../_shared uvicorn ridetune_ai.api:app --port 8163.	2	4
apps/rom_lab	Constrói modelos de ordem reduzida (POD + RBF, POD + GPR e outros) a partir de snapshots de simulação e mostra o erro em dados separados, por posto. Usa pinneapple_neural.architectures.rom, pinneapple_data, pinneapple_lab e pinneapple_core. Roda com cd apps/rom_lab && uvicorn rom_lab.api:app --port 8102.	3	4
apps/rover_mobility	Triagem de mobilidade de roda rígida: força de tração contra escorregamento por uma PINN de Bekker-Wong, com cada restrição física reconferida isoladamente. Usa pinneapple_simulation.terramechanics.mobility. Roda com cd apps/rover_mobility && uvicorn rover_mobility.api:app --port 8147.	2	2
apps/satscreen	Triagem rápida de conjunção de satélites: aproximação mínima e estimativa de probabilidade de colisão no movimento relativo de Clohessy-Wiltshire, mais propagação de dois corpos com J2; não treina PINN por requisição. Usa pinneapple_physics.astro.conjunction. Roda com cd apps/satscreen && uvicorn satscreen.api:app --port 8144.	2	3
apps/sensitivity_lab	Mostra quais parâmetros controlam de fato o resultado e quais podem ser congelados, com análise global e incerteza da resposta. Usa pinneapple_analysis, pinneapple_data e pinneapple_core. Roda com cd apps/sensitivity_lab && uvicorn sensitivity_lab.api:app --port 8099.	2	7
apps/simulation_campaign	Planeja campanhas de simulação: a partir de um espaço de parâmetros, gera um projeto de experimentos (hipercubo latino e outros), a ordem dos runs e os arquivos de caso. Usa pinneapple_data, pinneapple_simulation e pinneapple_security. Roda com cd apps/simulation_campaign && uvicorn simulation_campaign.api:app --port 8096.	2	8
apps/simulation_comparator	Compara um resultado com uma referência campo a campo (MAE, RMSE, máximo, normalizados) e mostra onde diferem; aceita OpenFOAM, VTK, CalculiX, Gmsh e tabelas de pontos, inclusive previsões de IA. Usa pinneapple_data. Roda com cd apps/simulation_comparator && uvicorn simulation_comparator.api:app --port 8089; nenhum teste cita a pasta.	2	0
apps/simulation_metadata	Lê (sem executar) um caso OpenFOAM, CalculiX/Abaqus ou um histórico de resíduos e devolve um registro estruturado (pinneapple.simmeta/1) com solver, malha, condições de contorno, parâmetros e veredito de convergência. Usa pinneapple_data. Roda com cd apps/simulation_metadata && uvicorn simulation_metadata.api:app --port 8084; nenhum teste cita a pasta.	2	0
apps/simulation_preflight	Antes de gastar horas de solver, confere se um caso OpenFOAM ou um deck CalculiX/Abaqus está montado certo (malha, materiais, contornos, condições iniciais, solver) e dá PASS/WARNING/FAIL. Usa pinneapple_data. Roda com cd apps/simulation_preflight && uvicorn simulation_preflight.api:app --port 8087.	2	5
apps/simulation_runner	Executa campanhas de simulação como cadeias de passos (gerar entrada, malhar, resolver, pós-processar, checar), registra o que terminou e retoma de onde parou, com executores locais ou Docker isolado. Usa pinneapple_core e pinneapple_tools. Roda com cd apps/simulation_runner && uvicorn simulation_runner.api:app --port 8110.	2	4
apps/soundshape_ai	Acha raio e comprimento do duto de uma caixa bass-reflex para uma frequência de sintonia, por um surrogate neural reconferido contra a fórmula de Helmholtz, com a curva de compromisso entre duto pequeno e sintonia exata. Lógica em pinneapple_design.surrogate_inverse.reflex_port; roda com PYTHONPATH=../..:../_shared uvicorn soundshape_ai.api:app --port 8164.	2	4
apps/surrogate_speed	Demo fixa com sete dissipadores: compara fórmula fechada, previsão de um surrogate (cronometrada) e uma solução de volumes finitos com índice de convergência de malha. O README avisa que as razões de velocidade são contra outro solver de referência que o do projeto original. Usa pinneapple_design; roda com PYTHONPATH=../_shared uvicorn surrogate_speed.api:app --port 8185.	2	2
apps/surrogate_studio	Transforma resultados de simulação existentes num modelo preditivo rápido, medido em runs que não viu e servido num endpoint versionado com a faixa de validade imposta. Usa pinneapple_analysis, pinneapple_registry e pinneapple_core. Roda com cd apps/surrogate_studio && uvicorn surrogate_studio.api:app --port 8097.	2	4
apps/text2part	Descreve-se uma peça em linguagem natural e recebe-se uma malha STL ou um CAD paramétrico STEP com suas medidas; o LLM só escolhe uma receita de um menu fixo, nunca escreve código CAD. Usa pinneapple_llm (draft_part, measure_part). Roda com cd apps/text2part && uvicorn text2part.api:app --port 8140 e precisa de um servidor Ollama.	2	4
apps/twin_ops	Exemplo de ponta a ponta (não é serviço web) de um gêmeo digital comissionado para um trocador casco-e-tubo com incrustação: contrato assinado, preflight que pode recusar, backtest separado, arquivo de evidência e escada de deriva. Usa pinneapple_systems. Roda com python apps/twin_ops/demo.py.	1	24

item	o que faz	arquivos .py	testes
apps/twin_runtime	Põe um modelo físico validado (tanque aquecido concentrado) ou um surrogate ao lado de dados operacionais, com assimilação por EnKF, e diz quando parar de confiar nele. Usa pinneapple_systems.operations, pinneapple_data e pinneapple_core. Roda com uvicorn twin_runtime.api:app --port 8103 (Dockerfile).	2	6
apps/uncertainty_lab	Mede quando uma previsão é confiável e quando o caso deve ir ao solver de alta fidelidade (fora dos dados, dados esparsos, erro maior que a margem ao limite). Usa pinneapple_analysis, pinneapple_data e pinneapple_core. Roda com cd apps/uncertainty_lab && uvicorn uncertainty_lab.api:app --port 8098.	2	8
apps/validation_lab	Verifica se um modelo ou resultado é consistente física e numericamente (equações, conservação, contornos, unidades, referência, faixa de treino) e grava o registro de cada checagem. Usa pinneapple_veriphysics, pinneapple_check, pinneapple_data, pinneapple_neural e pinneapple_lab. Roda com cd apps/validation_lab && uvicorn validation_lab.api:app --port 8107.	3	5
apps/verification_gap	Caso fixo de demonstração: um disco de freio que passa na checagem nominal e o que as tolerâncias de usinagem fazem com essa aprovação, por Monte Carlo sobre um modelo fechado. Usa pinneapple_design e pinneapple_analysis. Roda com PYTHONPATH=../_shared uvicorn verification_gap.api:app --port 8180.	2	2
apps/verified_physics	O LLM propõe a física e o PINNeAPPle executa e reconfere mecanicamente, com a evidência anexada; inclui arquiteturas candidatas, incerteza, intervenções causais e grafo de conhecimento. Usa pinneapple_analysis.verification. Roda com cd apps/verified_physics && uvicorn verified_physics.api:app --port 8143.	2	5
apps/zoo_space	Página estática (index.html, app.js, space.css) do demo do zoo publicada como Space no Hugging Face por scripts/zoo/publish_to_hub.py e pelo workflow publish-zoo.yml: mostra cada modelo do zoo numa varredura das entradas, com banda de incerteza e onde as checagens físicas avisam. Não tem README nem teste que cite a pasta.	0	0

2. Exemplos (examples/)

88 itens.

item	o que faz	arquivos .py	testes
examples/README.md	Índice curto da pasta examples/: diz para rodar os scripts a partir da raiz (por exemplo python examples/data_pipeline/01_physical_sample_basics.py) e aponta os tópicos que têm guia próprio, como meshgraphnet/ e studio/.	0	0
examples/architectures	Demos curadas do catálogo de modelos: tour do registro, autoencoders, séries temporais clássicas, PINNs (inclusive parâmetro inverso em EDO), FNO de brinquedo, ROM por POD/DMD, GNN por troca de mensagens e transformer de séries temporais. Usa pinneapple_neural e pinneapple_models; nenhum teste cita a pasta.	12	0
examples/arena_pipelines	Scripts numerados que listam os datasets e rodam benchmarks de física, de séries temporais e de Navier-Stokes (Kovasznay), arena a partir de YAML, galeria de problemas, problemas próprios e um pipeline de geometria 3D. Usa pinneapple_arena e pinneapple_neural; nenhum teste cita a pasta.	10	0
examples/autolab	Demo 2 do Autonomous Physics Lab: o 'engenheiro que se auto-otimiza' num problema térmico (self_optimizing_engineer.py). Usa pinneapple_lab e pinneapple_design; nenhum teste cita a pasta.	1	0
examples/autolab_aero	Estágio 1 do 'AI Aerodynamicist': o ciclo científico de aerodinâmica (hipótese, experimento, conclusão) sem agente. Usa pinneapple_autolab e pinneapple_lab; nenhum teste cita a pasta.	1	0
examples/autolab_aero_agent	Estágio 2 do 'AI Aerodynamicist': um agente lê a missão e opera o laboratório só por ferramentas controladas (agent_mission.py). Usa pinneapple_autolab e pinneapple_lab; nenhum teste cita a pasta.	1	0
examples/autolab_digital_twin	Demo 3 do Autonomous Physics Lab: um gêmeo digital que percebe quando está errado (twin_knows_when_wrong.py). Usa pinneapple_lab e pinneapple_systems; nenhum teste cita a pasta.	1	0
examples/autolab_rover	Demo 4 do Autonomous Physics Lab: o rover que aprende o solo do planeta enquanto anda (rover_learns_planet.py). Usa pinneapple_simulation e pinneapple_lab; nenhum teste cita a pasta.	1	0
examples/benchmark	Benchmark de mesmo orçamento numa só chamada: PINN contra FNO contra DeepONet no Burgers 1-D, configurado em bench.yaml. Usa a API de alto nível pinneapple; nenhum teste cita a pasta.	1	0
examples/benchmark_suite	Mostra o alcance de pinneapple_arena: benchmark de ponta a ponta (bundle, backend, métricas, leaderboard), troca de backend (nativo, DeepXDE, JAX, PhysicsNeMo), backend e tarefa próprios, XTFC, varredura de hiperparâmetros e gêmeos digitais. Usa também pinneapple_physics, pinneapple_neural e pinneapple_train; nenhum teste cita a pasta.	20	0

item	o que faz	arquivos .py	testes
examples/black_hole_weather	Reprodução e extensão de Duarte, Nemmen & Navarro (2022): simula o gás em volta de um buraco negro, treina a U-Net do paper e uma variante residual para prever o escoamento e mede por quanto tempo a previsão é confiável (com checagem de massa). Usa pinneapple_physics.blackhole; detalhes na seção de reproduções.	5	20
examples/bricks	Construções de blocos encaixáveis que se auto-modificam: simular, julgar pela física, redesenhar e repetir (self_modifying_bricks.py). Usa pinneapple e pinneapple_design.	1	20
examples/calculix_cantilever	Não é script: são os arquivos de uma execução de referência do CalculiX (ccx) de uma viga em balanço de 1,0 × 0,1 × 0,1 m (entrada beam.inp, resultados beam.frd, convergência e resumo). Serve de dado de entrada a outros exemplos; nenhum teste cita a pasta.	0	0
examples/calculix_pyvista_cantilever.py	Script único que lê o resultado CalculiX da viga em balanço e gera uma imagem PyVista da tensão de von Mises na viga deformada. Nenhum teste cita o arquivo.	1	0
examples/chain_of_whys	'Cadeia de porquês': quatro projetos em quatro escalas com o mesmo raciocínio causal. Usa pinneapple_reason e pinneapple_lab; nenhum teste cita a pasta.	1	0
examples/condensation_falsification	Campanha de falsificação da condensação física: os runs medidos de docs/research/condensation_falsification.md, com figuras e resultados gravados. Usa pinneapple_condense e pinneapple_lab; nenhum teste cita a pasta.	1	0
examples/core_primitives	PINN de Poisson e um operador de Poisson 1-D montados sobre as primitivas do núcleo (Domain, Geometry, FunctionField, Mesh, Field). Usa pinneapple_core e pinneapple_neural; nenhum teste cita a pasta.	2	0
examples/data_api	Uma PINN, um operador neural e um problema inverso treinados a partir da mesma API de loader. Usa pinneapple_core; nenhum teste cita a pasta.	1	0
examples/data_pipeline	Oito scripts da camada de dados: PhysicalSample, dataloaders UPD, prefetch e streaming para GPU, EDP sintética de ponta a ponta, variantes paramétricas em CadQuery, Zarr fragmentado com workers balanceados, portões de qualidade e lotes no formato do PINNFactory. Usa pinneapple_data e pinneapple_design; nenhum teste cita a pasta.	8	0
examples/design_from_intent	Projeto a partir da intenção: cinco pedidos passam pelo mesmo pipeline até um projeto verificado. Usa pinneapple e pinneapple_design; nenhum teste cita a pasta.	1	0
examples/electrodynamics	Soluções PINN de problemas clássicos de eletrostática, magnetostática e eletromagnetismo (capacitor de Laplace, dipolo, fio, onda 1D, guia de onda TM) com solução exata, mais galeria FDTD e uma série sobre um motor síncrono de ímãs permanentes (campo, acionamento, projeto do rotor, térmica, checagem cruzada com GetDP). Usa sobretudo pinneapple_simulation.	17	20
examples/end_to_end	Seis pipelines completos sem README: dataset UPD/Zarr até treino com solvers, malha até pontos de treino, treino com perda física via PINNFactory, CadQuery até Zarr e treino, treino e inferência pelo registro e pipeline STL de calor. Usa pinneapple_neural, pinneapple_data, pinneapple_design e pinneapple_physics; nenhum teste cita a pasta.	6	0
examples/enso_forecasting	Previsão do El Niño (ENSO) com modelos baseados em dados, físicos, informados pela física e híbridos sob um mesmo protocolo honesto (run_study.py, figuras e resultados gravados). Usa pinneapple_systems e pinneapple_core; guia em docs/guides/enso_forecasting.md.	1	20
examples/first_example	Primeiro resultado em cerca de um minuto numa CPU de laptop: resolve a equação de Burgers com uma PINN, compara com a solução exata e plota as duas (first_example.py). Usa pinneapple; não há teste que cite a pasta, mas o workflow tests.yml tem um job first-example.	1	0
examples/flight_dynamics	Verificação da biblioteca de dinâmica de voo: espectros de turbulência contra MIL-F-8785C, trimagem e estabilidade de quatro aeronaves, decolagem, encontros com turbulência, clipes renderizados, tabelas e visualizadores. Usa pinneapple_simulation, pinneapple e pinneapple_tools; guia em docs/guides/flight_dynamics.md.	8	20
examples/flow_studio	Gera os fluxos de exemplo do Flow Studio em pinneapple_flow/flows/ (build_examples.py), roda-os (run_examples.py) e tira capturas de tela, com resultados gravados em results/. Usa pinneapple_flow; nenhum teste cita a pasta.	3	0
examples/free_surface	Escoamento de superfície livre com interFoam (VOF): um jato caindo num canal de metal fundido com vertedouro, variando a inclinação do piso. Usa pp.cfd.FreeSurfaceFlow (pinneapple_simulation) e precisa de OpenFOAM; nenhum teste cita a pasta.	1	0
examples/freecad_openfoam	Pipelines FreeCAD para OpenFOAM: três estudos validados, uma varredura de parâmetros (sweep.csv) e um surrogate, com validation.json/validation.md gravados. Usa pinneapple_simulation e pinneapple_lab; nenhum teste cita a pasta.	1	0
examples/geomechanics	Método do ponto material: colapso de colunas de areia com várias razões de aspecto, alcance comparado com Lube et al. (2005) e quadros lidos pela ponte GeoTaichi. Usa pinneapple_simulation; resultados e limites em docs/core_concepts/geomechanics_mpm.md; nenhum teste cita a pasta.	1	0

item	o que faz	arquivos .py	testes
examples/geometry	Fluxos de geometria e malha para pipelines de PINN: carregar e reparar malhas, amostrar pontos de contorno e de colocação, amostragem por curvatura, CSG, ida e volta meshio e datasets CAD abertos. Usa pinneapple_design; nenhum teste cita a pasta.	8	0
examples/geometry_library	Três usos da biblioteca de geometrias: vento no edifício-padrão CAARC com OpenFOAM (via pp.cfd e STL), turbina eólica por BEM e avião por VLM. Usa pinneapple_design e pinneapple; nenhum teste cita a pasta.	3	0
examples/geophysics	Blocos de sistema terrestre num só script de cerca de um minuto em CPU: uma esfera e assimilação de dados. Usa pinneapple_simulation e pinneapple_analysis; nenhum teste cita a pasta.	1	0
examples/getting_started	Dez primeiros passos em IA física: cada script resolve um problema clássico (osciladores, calor, onda, logística, Lotka-Volterra, pêndulo, Van der Pol, Lorenz) com uma PINN usando só PyTorch, NumPy e Matplotlib, sem importar módulos PINNeAPPle. Nenhum teste cita a pasta.	10	0
examples/handoff_cases	Dois pacotes de caso autocontidos para testar um agente de simulação (ou um engenheiro) partindo do zero: aerofólio 2-D com RANS para maximizar L/D e jato de água num canal com VOF. Cada pasta tem tarefa, geometria, configuração, resultados calculados e referências; usa pinneapple_design e pinneapple_simulation; nenhum teste cita a pasta.	13	0
examples/hpo_experiments	Fluxo do pinneapple_researcher: descobrir papers (arXiv) e repositórios (GitHub), montar uma base de conhecimento, buscar localmente, extrair pares problema/solução com LLM e reproduzir um item num projeto local (04_reproduce.py pede GEMINI_API_KEY). Nenhum teste cita a pasta.	8	0
examples/industrial_presets	Não tem scripts: são os relatórios de presets industriais já montados, resolvidos e validados (perfil de asa, inspeção por correntes parasitas, motores, dissipador, PCB, vaso de pressão, bomba, trocador, tubulação de vapor, passarela, rede de água), com ficha, validação, surrogate, inversão e figuras. Regenera-se com python -m pinneapple_presets report all; nenhum teste cita a pasta.	0	0
examples/inspection	Ensaio não destrutivo por correntes parasitas de ponta a ponta com uma PINN. Usa pinneapple_analysis.inspection; nenhum teste cita a pasta.	1	0
examples/latent	Surrogate latente com cartão de confiança: erro, velocidade e calibração contra o solver completo. Usa pinneapple_neural e pinneapple_security; nenhum teste cita a pasta.	1	0
examples/lbm	Túnel de vento 3-D por lattice Boltzmann: arrasto numa esfera contra a lei de Stokes e a correlação de Schiller-Naumann, com distribuições guardadas em 32 e 16 bits. Usa pinneapple_simulation; nenhum teste cita a pasta.	1	0
examples/meshgraphnet	MeshGraphNet: difusão sintética em malhas de Delaunay e o escoamento em volta de cilindro do DeepMind (Pfaff et al. 2021) num orçamento reduzido, mais scripts de paridade com o PhysicsNeMo. O README diz que não reproduz o paper; só mostra que o pipeline aprende de dados reais. Usa pinneapple_neural e pinneapple_data.	6	6
examples/model_zoo	Início rápido do zoo de modelos: carregar um modelo pré-treinado, prever, comparar com a referência, validar, medir desempenho e fazer fine-tune, sobre o surrogate de aerofólio verificado que vem no pacote. Usa pinneapple; nenhum teste cita a pasta.	1	0
examples/modelica	Modelos Modelica como dados para IA física: .mo para FMU com OpenModelica, um surrogate MLP de resfriamento de panela de metal a partir de uma varredura em hipercubo latino e o coeficiente de descarga de um tanque identificado por PINN. Usa pinneapple_simulation.	4	6
examples/molecular	Uma proteína sobre uma superfície de ouro: orientar, montar a placa Au(111), rodar uma dinâmica molecular curta com OpenMM e visualizar. Usa pinneapple_tools; nenhum teste cita a pasta.	1	0
examples/motor_magnetics	Máquina de ímãs na superfície (4 polos, 24 ranhuras, ferro não linear): torque de cogging, torque contra ângulo de corrente, força contraeletromotriz e perdas com o FEM 2D em A_z. Usa pinneapple_simulation.electromag; nenhum teste cita a pasta.	1	0
examples/near_random_forecasting	Estudo de previsão de sistemas quase aleatórios (run_study.py) e o pacote visual dele: quatro renders Blender Cycles e duas figuras 2-D (render_visuals.py). Usa pinneapple_systems, pinneapple_core e pinneapple_blender.	2	20
examples/non_hermitian	Sintonia de pontos excepcionais em dímeros PT-simétricos: o teorema de impossibilidade (Zenodo 23287622) conferido simbólica e numericamente, as três rotas que o contornam e o mapa de hipóteses. Usa pinneapple_physics; nenhum teste cita a pasta.	1	0
examples/numerical_solvers	Treze scripts da camada de solvers numéricos: FFT, Hilbert-Huang, FEM e FVM (descritos como esqueletos no README), SPH, FDM, LBM, espectral, pipeline CAD-CFD, dinâmica molecular de gás Lennard-Jones e um gêmeo 3D de águas rasas no navegador. Usa pinneapple_solvers, pinneapple_train e pinneapple_twin; nenhum teste cita a pasta.	13	0
examples/nvidia	Exemplos que juntam o PINNeAPPle à pilha NVIDIA (Omniverse, PhysicsNeMo, simulação robótica, modelos de mundo, B-rep, serviços de modelos). O PINNeAPPle fica no centro (problema, solvers de referência, checagens físicas e decisão de confiança); cada run.py grava results/architecture.png, figuras, summary.json e run_info.json. A cola é pinneapple_tools.nvidia.	25	39
examples/openfoam_campaign	Campanha de parâmetros no OpenFOAM: um caso base e seis runs que mudam velocidade e elevação do jato, divididos entre duas máquinas, com status por python -m pinneapple_simulation.external_solvers.openfoam.campaign. Nenhum teste cita a pasta.	1	0

item	o que faz	arquivos .py	testes
examples/pde_environment	Ambientes de EDP montados à mão ou a partir de presets: Laplace 2D, Burgers 1D, elasticidade e calor 3D em STL, Navier-Stokes 2D, e validações de órbita de Kepler e de detritos espaciais (Clohessy-Wiltshire) contra a solução analítica. Usa pinneapple_physics, pinneapple_models e pinneapple_neural.	9	10
examples/physics_audit	Auditoria física em atrito de tubulação: modelos julgados pela física, não só pelos erros passados (pipe_regimes.py). Usa pinneapple_physics; nenhum teste cita a pasta.	1	0
examples/physics_check	Fixtures do comando pinneapple check: um caso que passa (pass/) e um que falha (fail/) para cada regra, com runs reais de icoFoam e campos analíticos com defeitos de propósito, regenerados por generate.py. Usa pinneapple_check; é o que o workflow physics-check.yml roda.	7	22
examples/physics_condensation	Trilha 1 da Adaptive Physics Condensation: o benchmark medido de docs/research/physics_condensation.md, com figuras e resultados gravados. Usa pinneapple_condense e pinneapple_lab; nenhum teste cita a pasta.	1	0
examples/physics_db	Descobrir datasets via CMR/earthaccess e NASA POWER, convertê-los em UPD fragmentado, carregar catálogos como PhysicalSample, amostrar pontos para PINN e treinar algo pequeno. Usa pinneapple_pdb, pinneapple_physics e pinneapple_data; nenhum teste cita a pasta.	7	0
examples/physics_ensemble	Ensembles de cinco famílias de modelos (FNO, PINN, GNN e solvers): reagir só a erros passados contra usar o resíduo físico do caso atual, turbulência multiescala e ruído de sensor, gerando GIFs comparativos. Usa pinneapple_physics e pinneapple_neural.	5	3
examples/physics_learning_verification	Trilha 3: o estudo medido de verificação de aprendizado físico de docs/research/physics_condensation.md (seção 6). Usa pinneapple_analysis e pinneapple_lab; nenhum teste cita a pasta.	1	0
examples/physics_module	Modelo híbrido: uma solução de elementos finitos grosseira mais uma correção neural, treinadas de ponta a ponta. Usa pinneapple_core; nenhum teste cita a pasta.	1	0
examples/physics_optimizer	Um problema de projeto com restrições, uma interface, seis otimizadores (01_constrained_shape_six_methods.py). Usa pinneapple_core; nenhum teste cita a pasta.	1	0
examples/physics_query	Trilha 2: a demonstração medida do motor de consultas físicas de docs/research/physics_condensation.md (seção 7). Usa pinneapple_lab; nenhum teste cita a pasta.	1	0
examples/pinn_solver	Camada específica de PINN: perda simbólica, compilador de EDP (Poisson 2D, Burgers 1D, onda 1D), contorno rígido, parâmetro inverso no calor 1D com PINNFactory, regressão só com dados, marcha no tempo, gêmeo climático com Open-Meteo e transferência de calor conjugada. Usa pinneapple_physics.pinn_solver; nenhum teste cita a pasta.	10	0
examples/pinneapple_and_physicsnemo	A equação da onda 1D resolvida lado a lado no PINNeAPPle e no PhysicsNeMo (pastas pinneapple/, physicsnemo/ e both/, esta com um benchmark de amostragem de geometria em EDO). Usa pinneapple_neural e pinneapple_physics; nenhum teste cita a pasta.	4	0
examples/pinneapple_benchmark	Benchmark de arquiteturas para EDPs (01_pde_comparison.py) e para problemas inversos (02_inverse_benchmark.py). Usa pinneapple_neural; nenhum teste cita a pasta.	2	0
examples/pinneapple_cosim	Cossimulação: sistema massa-mola de 2 graus de liberdade acoplado com dois modelos PINN. Usa pinneapple_cosim e pinneapple_models; nenhum teste cita a pasta.	1	0
examples/problem_designer	Elicitação, especificação e planejamento de problemas com pinneapple_problemdesign, de forma conversacional com LLM (Gemini ou provedor próprio) ou offline a partir de uma especificação pronta, até gerar relatórios em lote.	5	22
examples/proof_graph	Experimento de grafo de prova: o teorema de impossibilidade para sintonia linear de pontos excepcionais (dímeros PT, Zenodo 23287622) se estende a tetrâmeros PT com ligações moduladas de forma independente? Usa pinneapple_analysis; nenhum teste cita a pasta.	1	0
examples/qualitative	Prévia qualitativa de variantes seguida da checagem quantitativa: aletas de dissipador, rigidez por massa de viga e o ângulo traseiro do corpo de Ahmed. Usa pinneapple_design; nenhum teste cita a pasta.	3	0
examples/quickstart	Treze passos em sequência: o que está instalado, resolver e comparar, cartão de confiança, problema e dados próprios, problema inverso, usar o modelo, benchmark, incerteza e validação, preset industrial, gêmeo digital, visualização, texto para gêmeo e variantes. Usa sobretudo a API pinneapple; nenhum teste cita a pasta.	13	0
examples/reproductions	Reproduções de papers com protocolo congelado antes de rodar: X-TFC (Schiassi et al.) e Gated X-TFC (Dwivedi et al.), cada uma com TARGETS.md, RESULTS.md e dados brutos. Usa pinneapple_analysis.reproduction, pinneapple_simulation e pinneapple_physics; nenhum teste cita a pasta; detalhes na seção de reproduções.	3	0
examples/results_chassis	Não tem código: só figuras e JSON (pontos de colocação, campos de deslocamento e von Mises, histórico de perda) de um chassi, gerados por examples/scratch/generate_chassis_results.py. Nenhum teste cita a pasta.	0	0

item	o que faz	arquivos .py	testes
examples/rigid_body	Corpos rígidos com NVIDIA Newton: checar o solver e rodar uma cena pequena (newton_quickstart.py). Usa pinneapple_simulation; nenhum teste cita a pasta.	1	0
examples/science	Não tem scripts: é a tabela gerada dos experimentos de pinneapple_science (pp.science), cada grandeza calculada comparada com a referência e um veredito de confiança, a partir dos result.json gravados; hoje cobre eletromagnetismo (lei de Ampère, Gauss, Faraday, guias de onda e outros). Regenera-se com python -m pinneapple_science report all; nenhum teste cita a pasta.	0	0
examples/scratch	Scripts de trabalho e experimentais do desenvolvimento (trechos de benchmark, geração pontual de resultados, embutir resultados em relatórios, exportar PDF). O README avisa que são menos polidos que o resto; nenhum teste cita a pasta.	6	0
examples/security	Pipeline seguro de IA física de ponta a ponta e detecção de injeção de dados falsos pelo resíduo físico. Usa pinneapple_security e pinneapple_registry; nenhum teste cita a pasta.	2	0
examples/solver_api	O mesmo problema numa só chamada, por FEM clássico e por PINN, comparados com a solução exata. Usa pinneapple e pinneapple_physics; nenhum teste cita a pasta.	1	0
examples/solvers	Decomposições de séries temporais, uma caixa SPH e solvers de Poisson. Usa pinneapple_simulation e pinneapple_design; nenhum teste cita a pasta.	3	0
examples/sports	Física no esporte: voo da bola (arrasto, Magnus e gravidade a partir de posições rastreadas), controle de campo, cinemática de jogadores, gols esperados e um veredito sobre se a física prevê melhor que modelos só de dados. Usa pinneapple_systems e pinneapple_core; nenhum teste cita a pasta.	5	0
examples/studio	Qualquer resultado de simulação em 3D com três chamadas (carregar, exportar, renderizar), e CFD externo em corpo de Ahmed, avião, carro de rua, veículo lançador, carro de fórmula e um painel de rover lunar. Usa pinneapple, pinneapple_tools, pinneapple_simulation e pinneapple_design; guia em docs/core_concepts/studio.md.	7	6
examples/systems	Registro de componentes, caixa de ferramentas e ajuste informado pela física de ponta a ponta. Usa pinneapple_systems; nenhum teste cita a pasta.	1	0
examples/time_series	Fluxos de previsão de séries temporais: janelas e loaders determinísticos, FNO, pipeline completo, auditoria e features, backtest de baselines, incerteza por quantis, registro de modelos próprios e híbridos FFT/HHT com LSTM. Usa pinneapple_timeseries e pinneapple_train; nenhum teste cita a pasta.	10	0
examples/topology_optimization	Otimização topológica por SIMP: meia viga MBB e balanço, problemas de calor e de Stokes, e um rotor de relutância. Usa pinneapple_design.topology e pinneapple_simulation; nenhum teste cita a pasta.	3	0
examples/trainer	Treino com pinneapple_train: treino auditado, DataModule com divisões e pré-processamento, PINN de EDO consciente da física, retomar, inferir e plotar, e treino distribuído com torchrun. Nenhum teste cita a pasta.	5	0
examples/use_cases	Casos de uso maiores, cada um na sua pasta e com um TEMPLATE: antenas (zona quieta por óptica física), asa delta em alto ângulo de ataque por LBM, surrogates de colisão veicular e de desgaste, convecção inversa, gêmeo de canal aquecido, aerodinâmica de míssil, fábrica de dados sintéticos, gêmeos de desgaste de revestimento refratário, mecânica dos sólidos e terramecânica. Usa pinneapple_physics, pinneapple_simulation, pinneapple_neural e pinneapple_worldmodel.	34	17
examples/validation	Doze validações, uma por família de modelo: PINN de condução, FNO de Darcy exportado em ONNX, DeepONet/FNO de reação-difusão, MeshGraphNet de elasticidade, POD/DMD de esteira, OpInf, MLP do zoo, processo gaussiano, ODE neural, modelo generativo de Darcy, FNO do PhysicsNeMo e regressão modal. Usa pinneapple_analysis, pinneapple_neural e pinneapple_simulation.	12	4
examples/validation_arena	Roda um modelo externo (por exemplo o FNO ou o MeshGraphNet de um experimento PhysicsNeMo) pela defesa endurecida da arena de validação e compara cada saída. Usa pinneapple_analysis; nenhum teste cita a pasta.	2	0
examples/visualizations	Seis visualizações (escoamento em cilindro, calor 2D, vórtices, campo de fase, onda 2D, estrutural) e um run_all.py. Usa pinneapple_analysis e pinneapple_design; nenhum teste cita a pasta.	8	0
examples/vs_physicsnemo	Guia de comparação e integração PINNeAPPle contra PhysicsNeMo, com sete exemplos (incerteza em gêmeo digital, meta-transferência, pesos ativos, FNO do PhysicsNeMo, FNO combinado com gêmeo, MeshGraphNet validado, PhysicsNeMo treina e PINNeAPPle confere). As linhas medidas vêm de benchmarks/vs_frameworks/REPORT.md, só em CPU.	7	22
examples/weather_forecasting	Modelo global de previsão do tempo treinado com pinneapple_physics.weather em ERA5 (WeatherBench2), avaliado contra persistência, climatologia e previsões publicadas, com horizonte útil e eventos extremos. O README diz que o experimento está pela metade (só a fase 1 do treino foi feita).	2	6
examples/worldmodel	Modelo de mundo físico de ponta a ponta: gerar dados, treinar e avaliar. Usa pinneapple_worldmodel; nenhum teste cita a pasta.	1	0

3. Reproduções de papers

`examples/reproductions/xtfc_schiassi2021`

- **Paper:** E. Schiassi, C. Leake, M. De Florio, H. Johnston, R. Furfaro, D. Mortari, "Extreme Theory of Functional Connections: A Physics-Informed Neural Network Method for Solving Parametric Differential Equations", arXiv:2005.10632v1, 2020 (versão de periódico: Neurocomputing 457, 2021, que não foi lida)
- **Resultado gravado:** Os alvos foram congelados em `TARGETS.md` antes de qualquer run, e a regra de status é `pinneapple_analysis.reproduction.status`. Segundo `examples/reproductions/xtfc_schiassi2021/README.md` e `RESULTS.md`, 7 de 10 alvos foram reproduzidos, em escala completa e em CPU. Exemplos: ODE 1 com erro máximo mediano 4.4e-16 contra 6.66e-16 do paper; PDE 1 com 1.4e-13 contra 5.05e-13; PDE 7 (não linear 3D + t) com 2.4e-5 contra 2.98e-5. Não reproduzidos: ODE 2 (y1 mediano 0.91 contra 5.54e-10; a iteração de Gauss-Newton diverge na maioria das seeds) e PDE 6 (5.5e-6 contra 1.66e-8). Dados brutos em `results.json` e `baselines.json`.

`examples/reproductions/gated_xtfc_dwivedi2025`

- **Paper:** V. Dwivedi, E. Schiassi, M. Sigovan, B. Sixou, "Gated X-TFC: Soft Domain Decomposition for Forward and Inverse Problems in Sharp-Gradient PDEs", arXiv:2510.01039v1, 2025
- **Resultado gravado:** Alvos congelados em `TARGETS.md` antes de rodar; tudo medido por `run_gated.py`. Segundo `examples/reproductions/gated_xtfc_dwivedi2025/README.md`, só 2 de 9 alvos foram reproduzidos (faixa de fator 2): o problema direto com $\nu = 1e\text{-}2$ (6.7e-4 contra 4.95e-4 do paper) e a camada-limite dupla com $\nu = 1e\text{-}4$ (5.0e-3 contra 2.97e-3). Falharam os casos diretos com $\nu \leq 1e\text{-}3$ (por exemplo 3.9e-2 contra 5.86e-4 em $\nu = 1e\text{-}4$) e os quatro casos inversos (por exemplo 41 % de erro em $\hat{\nu}$ contra 10.3 % em $\nu = 0.1$). O README atribui a diferença a detalhes que o paper deixa em aberto (λ de Tikhonov, tipo de solução, apêndice ausente). Dados brutos em `part_*.json`.

`examples/black_hole_weather`

- **Paper:** Duarte, Nemmen & Navarro, "Black hole weather forecasting with deep learning: a pilot study", MNRAS 512, 5848, 2022 (arXiv:2102.06242)
- **Resultado gravado:** Reprodução e extensão (o README diz as duas coisas). Os dados PLUTO do paper não estavam acessíveis, então as simulações 2.5-D foram refeitas com `pinneapple_physics.blackhole`, e a U-Net do paper foi portada camada por camada (com 16 filtros em vez de 32). Segundo `examples/black_hole_weather/README.md`, a porta fiel não bate a persistência no espaçamento de quadros usado aqui (erro 0.038 contra 0.0137 da persistência em +1200 M), enquanto a variante residual, que é adição do PINNeAPPle, bate a persistência em todo o rollout (0.0076 em +1200 M, com correlação de tendência 0.61). Resumos por modelo ficam em `examples/black_hole_weather/results/`.

4. Scripts (`scripts/`)

24 itens.

item	o que faz	arquivos .py	testes
`scripts/README.md`	Runbook em português do MVP 'Flow Obstacle 2D': o passo a passo opcional Cosmos (spec e variações), Omniverse (USD e bundle), depois validar o bundle, rodar a PINNArena (nativa ou PhysicsNeMo), gerar o leaderboard e resumir os runs.	0	0
`scripts/app_helpers_baseline.txt`	Lista de duplicatas já existentes de helpers da biblioteca em `apps/` e `examples/`, usada como linha de base por `scripts/check_app_helpers.py`; os testes (por exemplo `tests/test_app_helpers.py`) falham só com duplicatas novas.	0	4
`scripts/arena`	Scripts de linha de comando do MVP para a arena: rodar um benchmark (`run_benchmark.py`, via `pinneapple_tools.benchmark_suite`), varreduras, comparar arenas, montar o leaderboard e resumir um run. Nenhum teste cita a pasta.	5	0
`scripts/astro`	Constroem os dados que vêm com `pinneapple_physics/astro/data`: catálogos compactos de estrelas e constelações, o dataset de fatos (a partir do d3-celestial) e o propagador neural de NEOs treinado.	3	0

item	o que faz	arquivos .py	testes
scripts/build_method_status.py	Calcula o status de validação de cada item de pinneapple_catalog.methods a partir das suítes de teste; o workflow release.yml confere que a página de status bate com essa varredura.	1	2
scripts/check_app_helpers.py	Aponta lógica reutilizável escrita em apps/ ou examples/ em vez de importada da biblioteca, comparando com app_helpers_baseline.txt.	1	0
scripts/check_dist.py	Confere as distribuições construídas: cada pacote da wheel está no sdist e na wheel, e importa quando o sdist é instalado num ambiente virtual limpo. Roda em tests.yml e release.yml.	1	0
scripts/check_example_imports.py	Confere que todo import pinneapple* em examples/ e templates/ resolve, com as exceções conhecidas em example_imports_baseline.txt.	1	0
scripts/check_experiment_manifests.py	Portão de CI (em tests.yml): toda pasta de examples/* tem um experiment.json válido e todo run do Lab é lido como manifesto válido.	1	0
scripts/check_release_version.py	Falha se a tag git, pyproject.toml, CITATION.cff, pinneapple.__version__ e CHANGELOG.md não concordam na versão; roda em release.yml.	1	0
scripts/cosmos	Integração com o Cosmos (LLM / World Foundation Model) por um endpoint HTTP JSON: cena para especificação, geração de variações e avaliação com refinamento. Parte opcional do MVP; nenhum teste cita a pasta.	3	0
scripts/dev	Atalhos de desenvolvimento do MVP: quickstart_mvp.sh (roda o fluxo Flow Obstacle 2D) e sanity_check_install.py (confere quais módulos importam).	1	0
scripts/example_imports_baseline.txt	Lista dos imports de exemplos que hoje não resolvem (por exemplo examples/architectures/50_timeseries_transformer_forecast_toy.py), usada como linha de base por check_example_imports.py e pelos testes.	0	2
scripts/fault_injection_matrix.py	Regenera a matriz de detecção das checagens de validação (#697): cada checagem contra a biblioteca de falhas em Darcy 2-D e Burgers 1-D.	1	0
scripts/geometry_library.py	Gera, verifica e publica os arquivos da biblioteca de geometrias (STL, glTF, OBJ, USD, STEP) com SHA-256 no manifesto; é chamado pelo job geometries de publish-zoo.yml.	1	21
scripts/issues	Tira um retrato das issues abertas do repositório para a página de contribuição (docs/org/contribute/index.html); roda no workflow docs.yml.	1	0
scripts/mvp	Scripts locais 'cola-tudo' do MVP: gerar os JSONs do bundle (conditions/manifest), validar o bundle contra o esquema e rodar a matriz da arena. Nenhum teste cita a pasta.	3	0
scripts/omniverse	Scripts que rodam dentro do Omniverse Kit (Omniverse Code / Isaac Sim): gerar cenários e exportar um bundle a partir do stage USD. Não rodam fora do Omniverse.	2	0
scripts/publish_issues.sh	Publica docs/dev/issues/*.md como issues no GitHub com gh, pulando as que já existem com o mesmo título.	0	0
scripts/reference	Monta a referência de módulos em docs/org/reference/ (pacotes, submódulos, docstrings e API pública lidos com ast) e roda os exemplos curados dessa referência, cada um num interpretador novo.	3	0
scripts/run_colab_notebooks.py	Executa os notebooks do Colab sem interface, do início ao fim, e confere que as cópias gravadas não têm saídas; é o job colab-notebooks de tests.yml.	1	0
scripts/update_imports.py	Script de migração de uma só vez: reescreve os imports Python para os caminhos novos dos 'mega-módulos'. Nenhum teste cita o arquivo.	1	0
scripts/validate_model_card.py	Portão de governança do hub de modelos: recusa o push de um model card sem validation_metrics calculadas ou sem fonte de referência; roda em tests.yml.	1	8
scripts/zoo	Constroem o zoo de modelos a partir das fontes gravadas: assets, model cards e manifest.json, datasets (esteira de cilindro por LBM, operadores Burgers/Darcy/Navier-Stokes, canal térmico, acreção em buraco negro), treinos (aerofólio por GNN, dissipador, contato de rover), relatórios de validação e a publicação no Hugging Face (publish_to_hub.py, usado por publish-zoo.yml).	10	28

5. Benchmarks (benchmarks/)

7 itens.

item	o que faz	arquivos .py	testes
`benchmarks/data_io_bench.py`	Mede a vazão de leitura de dados da camada `pinneapple_data` (store Zarr UPD, prefetch e cache de bytes). Não tem docstring nem teste que o cite.	1	0
`benchmarks/lbm_vortex_shedding.py`	Desprendimento de vórtices com o solver LBM D2Q9: número de Strouhal de um cilindro a Re = 100, comparado com o ajuste de Williamson (1996), e de um NACA 4412, em mais de uma altura de canal. Nenhum teste o cita.	1	0
`benchmarks/lpm_ellipse_design_space.py`	'Large Physics Model' num espaço de projeto exato: Cp de escoamento potencial em elipses, treinado em 30 projetos, avaliado em 10 inéditos, e com sinais de fora-da-distribuição conferidos num quadrado e num círculo maior. Roda com `python -m benchmarks.lpm_ellipse_design_space`.	1	0
`benchmarks/shard_balance_bench.py`	Mede o balanceamento de shards Zarr entre workers (`UPDZarrShardedWriter`, `ShardAwareZarrUPDIterable`) de `pinneapple_data`. Não tem docstring nem teste que o cite.	1	0
`benchmarks/ssqn_helmholtz.py`	Benchmark de paper: Helmholtz 2D (setup de Wang, Teng & Perdikaris 2021, usado por Urban et al. 2025) resolvido por PINN com os otimizadores quase-Newton BFGS, SSBFGS e SSBroyden contra Adam.	1	0
`benchmarks/ssqn_paper_benchmarks.py`	Vários problemas dos papers sobre otimizadores quase-Newton auto-escalados (Helmholtz, PK-PD rígido e outros, do repositório CrunchOptimizer/PINNs), cada um com solução de referência calculada aqui, comparando BFGS, SSBFGS e SSBroyden com Adam.	1	0
`benchmarks/vs_frameworks`	Benchmark entre frameworks (PINNeAPPle contra PhysicsNeMo / PhysicsNeMo-Sym, DeepXDE e neuraloperator) com protocolo gravado antes dos resultados (`PROTOCOL.md`): PINN com configuração idêntica e com receita própria, FNO em Darcy, Burgers inverso e ergonomia. O `REPORT.md` avisa que tudo rodou só em CPU, e que pelo teste do sinal nenhuma diferença é estatisticamente significativa.	15	0

6. Integração contínua (`.github/workflows/`)

workflow	nome	gatilhos	jobs	o que faz
`docs.yml`	Deploy Docs to GitHub Pages	push, workflow_dispatch, contents	deploy	Roda a cada push em `main` e à mão. Atualiza o retrato das issues abertas (`scripts/issues/build.py`), opcionalmente inclui o Playground, constrói a documentação e publica no GitHub Pages.
`draft-pdf.yml`	Draft PDF	push, pull_request, workflow_dispatch	paper	Roda em push para `main` e em pull requests que mexem em `paper/**`, e à mão. Gera o PDF de rascunho do artigo e o anexa como artefato.
`extras.yml`	Install extras	push, pull_request, workflow_dispatch	extras	Roda quando `pyproject.toml` ou `pinneapple/__init__.py` mudam (push em `main` ou PR) e à mão. Instala cada extra num ambiente limpo e confere que `pp.info()` roda sem pacote quebrado.
`observatory.yml`	Observatory (browser tests)	push, pull_request, workflow_dispatch	browser	Roda quando mudam `apps/observatory`, `apps/_shared`, `pinneapple_physics/astro` ou os testes do Observatory. Instala o app, passa um lint de texto fixo na interface e roda os testes de navegador com Playwright.
`physics-check.yml`	Physics check (pinneapple check)	push, pull_request, workflow_dispatch, contents	passing-fixtures, failing-fixtures, darcy-trust, unit-tests	Roda a cada push em `main`, em todo PR e à mão. Roda `pinneapple check` nas fixtures `examples/physics_check/pass` (deve passar) e `fail` (deve acusar exatamente as violações esperadas), checa um modelo de Darcy dentro e fora da faixa validada e roda os testes unitários.
`playground.yml`	Zoo web export and Playground (ONNX parity, JS runtime, browser smoke test)	push, pull_request, workflow_dispatch	onnx-parity, browser, docker	Roda quando mudam o Playground, os dados web, o zoo ou a biblioteca de geometrias. Confere a paridade ONNX contra PyTorch, o runtime JS do zoo contra o Python, um smoke test no navegador, o tamanho do site estático e a imagem Docker servindo a página.
`publish-zoo.yml`	Publish zoo to the Hugging Face Hub	workflow_dispatch, push, contents, group	publish, geometries	Roda à mão ou em push em `main` que muda o manifesto, os assets do zoo, `apps/zoo_space` ou a biblioteca de geometrias. Exige que a validação não regrida, confere SHA-256, envia modelos, datasets, o Space de demo e as geometrias ao Hugging Face (pula com aviso sem `HF_TOKEN`) e verifica os arquivos publicados baixando-os anonimamente.

workflow	nome	gatilhos	jobs	o que faz
release.yml	Release to PyPI	push	build, publish	Roda ao empurrar uma tag vX.Y.Z. Confere que tag, pyproject, CITATION e CHANGELOG concordam, que a página de status bate com o catálogo, constrói sdist e wheel, testa os imports num venv limpo e publica no PyPI.
security.yml	Security	pull_request, push, schedule, workflow_dispatch, contents (cron 17 5 * * 1)	scan, tests, image	Roda em PRs, em push em main, toda segunda-feira às 05:17 UTC (cron 17 5 * * 1) e à mão. Passa bandit, pip-audit, gitleaks, hadolint e Trivy, gera SBOMs, roda os testes de segurança por camada e confere que a imagem do HeatSink Sizer roda sem root, com sistema de arquivos só de leitura.
tests.yml	Tests (audit + core suite)	push, pull_request, workflow_dispatch	test, generated, first-example, colab-notebooks	Roda a cada push em main, em todo PR e à mão. O job test roda a suíte central em blocos que devem passar (contratos dos apps, Flow, apps de projeto inverso, arena de validação, entre outros); generated confere o índice do Hub e a cobertura do Flow Studio; first-example instala a wheel num venv limpo e roda o primeiro exemplo fora do repo; colab-notebooks roda os notebooks em modo rápido.
training-callbacks.yml	Training callbacks (Lightning, PhysicsNeMo, Hugging Face)	push, pull_request, workflow_dispatch, contents	callbacks	Roda quando mudam os callbacks de treino ou a validação de pinneapple_analysis. Instala Lightning, PhysicsNeMo e Hugging Face e confere que os callbacks dão as mesmas métricas em todos e avisam quando o peso do resíduo está errado.
validation-gallery.yml	Validation gallery	push, pull_request, workflow_dispatch, contents	gallery	Não estava na lista de entrada, mas existe. Roda quando mudam examples/validation, a validação de pinneapple_analysis, pinneapple_neural ou os solvers numéricos. Roda cada exemplo da galeria de validação em tamanho reduzido, confere que ele pega a própria falha e passa pinneapple check na configuração da galeria.

7. Documentação existente (docs/)

190 arquivos Markdown. Lista com o título de cada um:

arquivo	título
docs/SUMMARY.md	SUMMARY
docs/apps/anomaly_detector.md	Physics Anomaly Detector (day 28)
docs/apps/benchmark_arena.md	Physics AI Benchmark Arena
docs/apps/calibration_lab.md	Model Calibration Lab (day 18)
docs/apps/control_lab.md	Physics-Based Control Lab (day 29)
docs/apps/evidence_report.md	Engineering Evidence Report (day 30)
docs/apps/hub.md	PINNeAPPle Hub
docs/apps/identifiability_lab.md	Parameter Identifiability Lab (day 25)
docs/apps/inverse_studio.md	Inverse Model Studio (day 21)
docs/apps/meshless_lab.md	Meshless Method Lab (day 26)
docs/apps/model_gateway.md	Physics AI Model Gateway
docs/apps/rom_lab.md	Reduced Order Model Lab (day 19)
docs/apps/surrogate_studio.md	Surrogate Studio
docs/apps/twin_runtime.md	Digital Twin Runtime (day 20)
docs/apps/validation_lab.md	Physics Validation Lab (day 24)
docs/architecture/execution_model.md	Execution Model
docs/architecture/package_layers.md	Package Layers
docs/architecture/system_overview.md	System Overview
docs/archive/overview_en.md	PINNeAPPle — Library Overview
docs/archive/overview_pt.md	PINNeAPPle — Visão Geral da Biblioteca
docs/assets/apps/README.md	App images (30 apps with PINNeAPPle, days 1–14)
docs/assets/proof_graph/pt_chain_no_go_report.md	No linear exceptional-point tuning in PT-symmetric tetramers
docs/core_concepts/adaptive_forecasting.md	Adaptive forecasting: switching and combining models online
docs/core_concepts/astro.md	Positional astronomy and astrophysics (pinneapple_physics.astro)
docs/core_concepts/astroml.md	astroML: periodograms, deconvolved densities and fits with errors

arquivo	título
docs/core_concepts/backend.md	Backend
docs/core_concepts/black_hole_weather.md	Black-hole weather: forecasting accretion flows with a trust horizon
docs/core_concepts/core_primitives.md	Core primitives: Domain, Geometry, Mesh, Field
docs/core_concepts/dashboard.md	Animation-centred dashboards
docs/core_concepts/earth_system.md	Earth-system building blocks and data assimilation
docs/core_concepts/fitmesh.md	From a problem description to checked CAD, fit-for-purpose meshes and one report
docs/core_concepts/free_surface.md	Free-surface flow: runners, channels, launders
docs/core_concepts/geomechanics_mpm.md	Geomechanics with the material point method
docs/core_concepts/geometry_domain.md	Geometry & Domain
docs/core_concepts/lab.md	PINNeAPPle Lab: experiments at scale, and a database of results and datasets
docs/core_concepts/lbm_wind_tunnel.md	Lattice Boltzmann wind tunnel and the FluidX3D bridge
docs/core_concepts/lunar_rover_terramechanics.md	Lunar rover terramechanics in 3D
docs/core_concepts/model.md	Model
docs/core_concepts/modelica.md	Modelica models as Physics AI data
docs/core_concepts/molecular_visualization.md	Molecular visualisation: proteins on metal surfaces
docs/core_concepts/motor_and_topopt_audit.md	Audit: electric-motor magnetics and topology optimization
docs/core_concepts/multiscale_ensembles.md	Multiscale ensembles: scales, filters and closures
docs/core_concepts/newton_rigid_body.md	Rigid bodies with NVIDIA Newton
docs/core_concepts/non_hermitian.md	Non-Hermitian physics: PT-symmetric dimers and exceptional-point tuning
docs/core_concepts/openfoam_acceleration.md	Faster implicit pressure solves in OpenFOAM
docs/core_concepts/openfoam_campaigns.md	OpenFOAM parameter campaigns on several machines
docs/core_concepts/overview.md	Core Concepts
docs/core_concepts/physical_sample.md	PhysicalSample
docs/core_concepts/physics_audit.md	Physics audit: which model to use, when to trust it, when to revisit the physics
docs/core_concepts/physics_ensembles.md	Adaptive ensembles of physics models
docs/core_concepts/physics_optimizer.md	PhysicsOptimizer
docs/core_concepts/pinn.md	PINN / Physics
docs/core_concepts/problem_definition.md	ProblemDefinition
docs/core_concepts/proof_graphs.md	Proof graphs: mathematics with the same evidence discipline
docs/core_concepts/qualitative_preview.md	Qualitative preview: physical impact of geometry changes before the numbers
docs/core_concepts/reduced_order_models.md	Reduced-order models
docs/core_concepts/researcher_benchmarking.md	Researcher & Benchmarking
docs/core_concepts/security.md	Security for scientific and industrial Physics AI (`pinneapple_security`)
docs/core_concepts/solver.md	Solver
docs/core_concepts/studio.md	3D studio and external flow
docs/core_concepts/training_pipeline.md	Training Pipeline
docs/core_concepts/veriphysics.md	Verification and evidence (`pinneapple_veriphysics`)
docs/core_concepts/viewers_and_twin_standards.md	Other 3-D viewers and digital-twin standards
docs/dev/ANALISE_FEATURES_COMERCIAIS.md	O que as ferramentas comerciais e os outros CFDs têm que o PINNeAPPle ainda não tem
docs/dev/AUDIT_REPORT.md	Evidenced audit report
docs/dev/CATALOGO_METODOS.md	Catálogo de métodos do PINNeAPPle
docs/dev/LATENT_AND_CYBER_EVALUATION.md	Latent-space methods and cybersecurity applications: evaluation and roadmap
docs/dev/LIBRARY_CONSOLIDATION.md	Library consolidation: reusable logic outside the library
docs/dev/MARKET_SCAN_LOVABLE_PHYSICS_AI_2026-09-13.md	Pesquisa: Physics AI / Simulação / CAE / Digital Twin em lovable.app
docs/dev/PEDIDOS_2026-09-24.md	Pedidos e insights de 2026-09-24 — registro e decisões
docs/dev/REA_INTEGRATION.md	REA no PINNeAPPle — integração, fluxo de harvest e mapa do ecossistema
docs/dev/ROADMAP_PHYSICS_AI_HUB.md	PINNeAPPle → "the Hugging Face of Physics AI": full roadmap

arquivo	título
docs/dev/issues/001-geometric-similarity-score.md	
docs/dev/issues/002-active-simulation-loop.md	
docs/dev/issues/003-raw-geometry-field-surrogate.md	
docs/dev/issues/004-rom-fmu-export.md	
docs/dev/issues/005-pinneapple-mcp-server.md	
docs/dev/issues/006-topology-optimization-simp.md	
docs/dev/issues/007-solution-adaptive-amr.md	
docs/dev/issues/008-multi-fidelity-cokriging.md	
docs/dev/issues/009-hybrid-rom-online-calibration.md	
docs/dev/issues/010-pce-sobol.md	
docs/dev/issues/011-geometry-encoders.md	
docs/dev/issues/012-lbm-d3q27-cumulant-voxel.md	
docs/dev/issues/013-ifno-incremental-modes.md	
docs/dev/issues/014-multiadam.md	
docs/dev/issues/015-mapdl-rst-reader-pyfluent.md	
docs/dev/issues/016-browser-pde-preview.md	
docs/dev/issues/017-physicsnemo-missing-models.md	
docs/dev/issues/018-rea-native-ghidra.md	
docs/dev/issues/019-adjoint-mma-bspline.md	
docs/dev/issues/020-design-study-object.md	
docs/dev/issues/021-rbf-mesh-morphing.md	
docs/dev/issues/022-inference-cache-viewer.md	
docs/dev/issues/023-the-well-adapter.md	
docs/dev/issues/024-procedural-pde-datasets.md	
docs/dev/issues/025-neural-corrected-stepper.md	
docs/dev/issues/026-differentiable-linear-solves.md	
docs/dev/issues/027-nektar-bridge.md	
docs/dev/issues/028-jaxfluids-bridge.md	
docs/dev/issues/029-paraview-catalyst-insitu.md	
docs/dev/issues/030-nn-in-weak-forms.md	
docs/dev/issues/031-warp-backend.md	
docs/dev/issues/032-rbf-kriging-surrogates-outliers.md	
docs/dev/issues/033-onshape-configuration-sampler.md	
docs/dev/issues/034-rescale-job-connector.md	
docs/dev/issues/035-comsol-mph-bridge.md	
docs/dev/model_dataset_inventory.md	Model and dataset inventory for the zoo (2026-10-13)
docs/dev/pr/feat-rea-bridge.md	Summary
docs/getting_started/first_pinn.md	First PINN

arquivo	título
docs/getting_started/installation.md	Installation
docs/getting_started/project_layout.md	Project Layout
docs/getting_started/quickstart.md	Quickstart
docs/getting_started/start_here.md	Start here (docs)
docs/guides/adding_an_experiment.md	Adding an experiment / Como adicionar um experimento
docs/guides/agent_workbench.md	Agent Workbench: coding and engineering agents on a local model
docs/guides/antenna_design.md	Antenna design: an end-fire Vivaldi with an external FEM solver
docs/guides/autonomous_lab.md	Autonomous Physics Lab: an AI that manages evidence for engineering decisions
docs/guides/autonomous_physics_lab.md	Autonomous Physics Lab
docs/guides/campaign_planner.md	Simulation Campaign Planner: which simulations need to run
docs/guides/chain_of_whys.md	Chain of whys: why each element of a design exists, computed
docs/guides/check_power.md	Check power: how often each check catches a known defect
docs/guides/check_power_matrix.md	Check power: the detection matrix
docs/guides/choosing_a_method.md	Choosing a method: PINN, neural operator, graph network or the solver
docs/guides/colab_notebooks.md	Colab notebooks
docs/guides/compact_range.md	Compact range quiet zone: feed gain × reflector size
docs/guides/dataset_audit.md	Dataset physics audit: does the data solve its PDE?
docs/guides/design_from_intent.md	Design from intent
docs/guides/design_report.md	Design reports without an LLM: `Datasheet` and the self-planning study
docs/guides/design_studies.md	Design studies: one problem, many optimisers, verified candidates
docs/guides/electromagnetics.md	Electromagnetics and electrodynamics
docs/guides/engineering_design_optimizer.md	Engineering Design Optimizer (app, day 17)
docs/guides/engineering_gui_agent.md	Engineering GUI agent: run engineering software and keep the evidence
docs/guides/enso_forecasting.md	Predicting El Niño: physical, physics-informed, data-driven and hybrid models under one protocol
docs/guides/evidence_ledger.md	Evidence ledger: a test passed only if it ran
docs/guides/flight_dynamics.md	Flight dynamics, turbulence and take-off
docs/guides/flow_schema.md	The flow schema (`pinneapple-flow/1`) and how another app reuses it
docs/guides/fmi_compatibility.md	FMI import compatibility
docs/guides/football_physics.md	Football: where physics helps on public tracking data
docs/guides/freecad_openfoam_pipelines.md	FreeCAD -> OpenFOAM pipelines
docs/guides/geometry_library.md	Geometry library
docs/guides/identifiability_calibration_inversion.md	Identifiability, calibration and inverse problems
docs/guides/industrial_presets.md	Industrial presets
docs/guides/interlocking_bricks.md	Self-modifying brick builds
docs/guides/mcp_server.md	PINNeAPPle from any coding agent: the MCP server
docs/guides/metamorphic_tests.md	Metamorphic tests: checking a surrogate without ground truth
docs/guides/model_zoo.md	Pre-trained models and datasets (`pp.zoo`, `pp.datasets`)
docs/guides/near_random_forecasting.md	Forecasting near-random series without overfitting
docs/guides/observatory.md	The PINNeAPPle Observatory
docs/guides/physics_checks_in_ci.md	Physics checks in CI
docs/guides/physics_design_optimizer.md	Physics Design Optimizer (app, day 23)
docs/guides/physics_residual_attack_detection.md	Detect false data injection with physics residuals
docs/guides/physicsnemo_trust.md	PhysicsNeMo trains, PINNeAPPle says when to trust
docs/guides/pinn_validation.md	PINN validation: what the training loss does not say
docs/guides/pinneapple_llm_quality_pipeline.md	Quality pipeline for LLM-generated physics and twin artifacts
docs/guides/playground.md	Playground: physics surrogates in the browser
docs/guides/pp_validate.md	pp.validate: one call, one validation report
docs/guides/recreate_a_wear_twin.md	Recreate a 3D wear twin (refractory, liner, pipe wall)
docs/guides/residual_error_bound.md	From residual to error bound: how wrong is this prediction, without the true answer?

arquivo	título
docs/guides/rollout_validation.md	Rollout validation: when an autoregressive surrogate stops being trustworthy
docs/guides/rom_and_meshless_studies.md	Reduced-order models and meshless methods: measured, not assumed
docs/guides/rover_learns_the_planet.md	The rover that learns the planet
docs/guides/running_a_benchmark.md	Running a benchmark
docs/guides/scale_aware_ensembles.md	Choose a model by the scale of the question
docs/guides/science.md	Science by area (pp.science)
docs/guides/sensitivity_lab.md	Engineering Sensitivity Lab
docs/guides/spectral_fidelity.md	Spectral fidelity: what relative L2 does not see
docs/guides/surrogates_arena_gateway.md	Surrogates from simulation results: study, regime arena, gateway
docs/guides/topology_optimization.md	Topology optimisation
docs/guides/training_callbacks.md	Physics checks during training (Lightning, PhysicsNeMo, Hugging Face)
docs/guides/trusted_latent_surrogate.md	A latent surrogate that says when not to trust it
docs/guides/uncertainty_and_sensitivity.md	Uncertainty and sensitivity studies on any model
docs/guides/uncertainty_lab.md	Engineering Uncertainty Lab
docs/guides/unit_cell_benchmark.md	Periodic unit cell: an FNO over frequency × scan, and an equal-budget benchmark against Bayesian optimisation
docs/guides/validation_arena.md	Red team / blue team for the validation checks
docs/guides/validation_gallery.md	Validation gallery: one worked example per model family and per physics
docs/guides/validation_methods_reference.md	Validation methods reference
docs/guides/workflow_runner.md	Simulation Workflow Runner: campaigns as traceable, resumable workflows
docs/guides/zoo_catalogue.md	Zoo catalogue
docs/index.md	PINNeAPPle
docs/lab/coverage.md	Experiments coverage
docs/philosophy.md	Philosophy
docs/research/condensation_falsification.md	Falsification campaign for physics condensation — pre-registration
docs/research/physics_condensation.md	Adaptive Physics Condensation: geometry-independent thermal models of chip packages
docs/security/audit.md	Security audit: the PINNeAPPle web apps and their deployment
docs/security/runbook.md	Operator runbook: PINNeAPPle apps
docs/security/server.md	Server hardening: the host that runs the PINNeAPPle apps
docs/strategy/STRATEGY_PHYSICS_AI.md	Physics AI Strategy — structure and first implementation
docs/validation_status.md	Validation status
docs/why.md	Why PINNeAPPle?

Volume 9 — Índice alfabético, glossário e apêndice numérico

> Parte do Mapa completo do PINNeAPPle. Gerado por `docs/pt/_mapa/gerar.py` a partir do inventário do commit `e53cdcfb04`.

Índice de todas as classes e funções públicas de nível de módulo, o glossário e os números do inventário.

1. Índice alfabético

8.887 entradas. **C** = classe, **f** = função. O caminho é o módulo onde o nome é definido (ver o capítulo do pacote no volume indicado).

A

- `a320` f — `pinneapple_simulation.flight.catalog` (v03)
- `abaqus_inp_to_upd` f — `pinneapple_simulation.external_solvers.cfd_formats.abaqus_reader` (v03)
- `acc` f — `pinneapple_physics.weather.evaluate` (v02)
- `acc` f — `pinneapple_systems.time_series.enso.evaluation` (v06)
- `acceleration_from_displacement` f — `pinneapple_simulation.numerical_solvers.nonlinear_beam_fem` (v03)
- `accelerations` f — `pinneapple_physics.astro.nbody` (v02)
- `AcceptanceGates` C — `pinneapple_systems.digital_twin.product.spec` (v06)
- `accretion_scene` f — `pinneapple_physics.blackhole.twin` (v02)
- `accretion_view` f — `pinneapple_physics.astro.lensing` (v02)
- `AccretionFlow` C — `pinneapple_physics.blackhole.hydro` (v02)
- `AccretionFlowRun` C — `pinneapple_lab.experiments.accretion` (v07)
- `accuracy` f — `pinneapple_condense.figures` (v02)
- `accuracy_vs_modes` f — `pinneapple_condense.benchmark` (v02)
- `acf_pacf` f — `pinneapple_systems.time_series.audit.tests` (v06)
- `acid_fraction` f — `pinneapple_systems.process_components.reaction_kinetics` (v06)
- `acoustic_field` f — `pinneapple_design.surrogate_inverse.reflex_port` (v06)
- `act` f — `pinneapple_autolab.core` (v07)
- `Action` C — `pinneapple_lab.next_steps` (v07)
- `ActionFailed` C — `pinneapple_tools.automation.adapters.base` (v07)
- `ActionResult` C — `pinneapple_tools.automation.adapters.base` (v07)
- `activation_derivative` f — `pinneapple_simulation.numerical_solvers.xtfc_general` (v03)
- `active_learning` f — `pinneapple_simulation.electromagnetics.antenna.design` (v03)
- `active_learning_retrain_loop` f — `pinneapple_neural.physicsnemo_bridge.active_learning_bridge` (v04)
- `ActiveLearningConfig` C — `pinneapple_data.active_learning` (v04)
- `ActiveLearningResult` C — `pinneapple_design.design_optimizer.verify` (v06)
- `ActiveSampler` C — `pinneapple_core.data` (v01)
- `Actuator` C — `pinneapple_simulation.vehicle.spec` (v03)
- `actuator` f — `pinneapple_design.geometry.vehicle_cad` (v06)
- `actuator_response_rhs` f — `pinneapple_systems.process_components.control_valve` (v06)
- `actuator_series` f — `pinneapple_systems.excitation` (v06)
- `AdaHedge` C — `pinneapple_physics.online_learning` (v02)
- `adapt_model` f — `pinneapple_adaptation` (v04)
- `adapt_model` f — `pinneapple_arena.pointwise.adapters` (v04)
- `adapt_output` f — `pinneapple_physics.multiscale.comparison` (v02)
- `adapt_problem` f — `pinneapple_decision.adapter` (v05)
- `AdaptedProblem` C — `pinneapple_decision.adapter` (v05)

- AdaptedSequenceDataset C — pinneapple_neural.trainer.datamodule (v04)
- Adapter C — pinneapple_core.workflows.adapters (v01)
- Adapter C — pinneapple_tools.automation.adapters.base (v07)
- adaptive f — pinneapple_flow.nodes_ext.core_ts (v07)
- AdaptiveCollocationConfig C — pinneapple_neural.trainer.collocation (v04)
- AdaptiveCollocationSampler C — pinneapple_neural.trainer.collocation (v04)
- AdaptiveCollocationTrainer C — pinneapple_data.active_learning (v04)
- AdaptiveConformal C — pinneapple_physics.online_learning (v02)
- AdaptiveForecaster C — pinneapple_systems.time_series.adaptive (v06)
- AdaptivePrefetchConfig C — pinneapple_data.prefetch_adaptive (v04)
- AdaptivePrefetchZarrUPDIterable C — pinneapple_data.prefetch_adaptive (v04)
- AdaptiveResult C — pinneapple_condense.learned (v02)
- AdaptiveRun C — pinneapple_systems.time_series.adaptive (v06)
- AdaptiveSampler C — pinneapple_core.data (v01)
- AdaptiveSweepConfig C — pinneapple_neural.trainer.adaptive_sweep (v04)
- AdaptiveWeights C — pinneapple_physics.pinn_solver.compiler.loss (v02)
- add f — pinneapple_flow.libcalls (v07)
- add_earth_pyvista f — pinneapple_tools.visualization.studio.lighting (v07)
- add_gaussian_noise f — pinneapple_data.synth.operators.noise (v04)
- add_link f — pinneapple_hub.library.flowlink (v07)
- add_scan f — pinneapple_twin3d.scan (v06)
- add_vorticity_divergence f — pinneapple_pdb.derived (v04)
- adf_test f — pinneapple_systems.time_series.audit.tests (v06)
- admin_summary f — pinneapple_app.backend.routers.admin (v07)
- AdminSummaryResponse C — pinneapple_app.backend.routers.admin (v07)
- admissible f — pinneapple_check.checks.common (v05)
- advection_diffusion f — pinneapple_condense.meshless2d (v02)
- advection_diffusion_2d f — pinneapple_simulation.numerical_solvers.operator_benchmarks (v03)
- advection_dominated f — pinneapple_physics.physics_audit.assumptions (v02)
- AdvectionDiffusion1D C — pinneapple_arena.problems (v04)
- AdvectionDiffusion1D C — pinneapple_physics.advection_diffusion_1d (v02)
- AdvectionDispersionReactionSolver C — pinneapple_systems.process_components.reaction_kinetics (v06)
- adversarial_sensitivity f — pinneapple_security.model_security (v05)
- AdversarialReviewReport C — pinneapple_analysis.verification.architecture_critique (v05)
- Advisor C — pinneapple_reason.engine (v05)
- AEBase C — pinneapple_neural.architectures.autoencoders.base (v04)
- AEOutput C — pinneapple_neural.architectures.autoencoders.base (v04)
- AeroCoefficients C — pinneapple_tools.aero_coefficients (v07)
- aerodynamicist f — pinneapple_flow.nodes.autolab (v07)
- AeroGNN C — pinneapple_design.aero.surrogate (v06)
- AEROMHybrid C — pinneapple_neural.architectures.autoencoders.ae_rom_hybrid (v04)
- AeroModel C — pinneapple_simulation.flight.aero (v03)
- AFNO C — pinneapple_neural.architectures.afno (v04)
- AFNOLayer C — pinneapple_neural.architectures.afno (v04)
- Agent C — pinneapple_llm.workbench.agents (v07)
- agent_tools f — pinneapple_hub.library.sources (v07)
- AgentLoopResult C — pinneapple_llm.agent_loop (v07)
- AgentResult C — pinneapple_autolab.agent (v07)
- AgentStep C — pinneapple_llm.agent_loop (v07)
- aggregate f — pinneapple_arena.pointwise.leaderboard (v04)
- ahmed_body f — pinneapple_design.geometry.bodies (v06)
- ahmed_body_polygon f — pinneapple_physics.pde_environment.presets._curve_geometry (v02)
- ahmed_slant f — pinneapple_flow.nodes_ext.design (v07)
- air f — pinneapple_physics.closed_form.pipe_insulation (v02)
- air f — pinneapple_reason.props (v05)

- air_paths f — pinneapple_design.thermal.board_cooling.scene (v06)
- air_properties f — pinneapple_physics.closed_form.plate_fin_heatsink (v02)
- Aircraft C — pinneapple_design.aero.aircraft (v06)
- Aircraft C — pinneapple_simulation.flight.aircraft (v03)
- aircraft f — pinneapple_flow.nodes_ext.flight (v07)
- Aircraft3D C — pinneapple_design.aero.aircraft3d (v06)
- aircraft_surfaces f — pinneapple_simulation.flight.visuals (v03)
- aircraft_wing_aerodynamics f — pinneapple_physics.pde_environment.presets.engineering (v02)
- aircraft_wing_structural f — pinneapple_physics.pde_environment.presets.engineering (v02)
- airfoil f — pinneapple_twin3d.primitives (v06)
- airfoil_baseline f — pinneapple_design.design_optimizer.presets (v06)
- airfoil_bodies f — pinneapple_design.design_optimizer.presets (v06)
- airfoil_existing f — pinneapple_simulation.external_solvers.campaign_cases (v03)
- airfoil_feasible f — pinneapple_simulation.external_solvers.campaign_cases (v03)
- airfoil_naca_mask f — pinneapple_simulation.numerical_solvers.lbm (v03)
- airfoil_naca_mask_3d f — pinneapple_simulation.numerical_solvers.lbm (v03)
- airfoil_nodes f — pinneapple_design.aero.panel (v06)
- airfoil_outline f — pinneapple_design.design_optimizer.airfoil_drag (v06)
- airfoil_problem f — pinneapple_design.design_optimizer.presets (v06)
- airfoil_rans f — pinneapple_analysis.surrogate.examples (v05)
- airfoil_space f — pinneapple_simulation.external_solvers.campaign_cases (v03)
- AirfoilEvaluator C — pinneapple_design.design_optimizer.presets (v06)
- AirfoilGNN C — pinneapple_hub.zoo._models (v07)
- AirfoilMLP C — pinneapple_hub.zoo._models (v07)
- AirfoilProblem C — pinneapple_analysis.benchmark.builtin.airfoil (v05)
- AirfoilSection C — pinneapple_presets.cases.airfoil (v02)
- Airframe C — pinneapple_design.aero.airframe (v06)
- airframe_from f — pinneapple_design.aero.aircraft3d (v06)
- Airliner C — pinneapple_design.aero.airliner (v06)
- airliner f — pinneapple_twin3d.gallery (v06)
- airliner_from f — pinneapple_design.aero.airliner3d (v06)
- airliner_vlm f — pinneapple_flow.nodes_ext.aero (v07)
- AirlinerModel C — pinneapple_design.aero.airliner3d (v06)
- airmass f — pinneapple_physics.astro.astrometry (v02)
- aleatoric_nll_loss f — pinneapple_analysis.uncertainty.aleatoric (v05)
- AleatoricHead C — pinneapple_analysis.uncertainty.aleatoric (v05)
- Alert C — pinneapple_systems.operations.twin (v06)
- AlertRules C — pinneapple_systems.operations.twin (v06)
- all_faces f — pinneapple_app.backend.core.physics_adapter (v07)
- all_model_specs f — pinneapple_tools.benchmark_suite.benchmark (v07)
- all_problems f — pinneapple_app.backend.core.problem_registry (v07)
- allen_cahn_reference f — pinneapple_tools.benchmark_suite.crossframework.problems (v07)
- AllenCahn C — pinneapple_arena.problems (v04)
- AllenCahn C — pinneapple_arena.problems (v04)
- AllenCahn1DTask C — pinneapple_tools.benchmark_suite.tasks.allen_cahn_1d (v07)
- allmesh_script f — pinneapple_simulation.pipelines.freecad_openfoam.mesh (v03)
- allowable_stress_for_life_pa f — pinneapple_design.surrogate_inverse.suspension_arm (v06)
- allrun_script f — pinneapple_simulation.pipelines.freecad_openfoam.mesh (v03)
- along_track f — pinneapple_design.intent.physics.loop_track (v06)
- ALRetrainResult C — pinneapple_neural.physicsnemo_bridge.active_learning_bridge (v04)
- ALRound C — pinneapple_simulation.electromagnetics.antenna.design (v03)
- ALRoundResult C — pinneapple_neural.physicsnemo_bridge.active_learning_bridge (v04)
- AmbiguityResult C — pinneapple_analysis.inverse_problems.counterfactual_attribution (v05)
- amg f — pinneapple_simulation.accelerated_foam.solvers (v03)
- Ammeter f — pinneapple_science.instruments (v02)
- AmortizedCost C — pinneapple_analysis.cost.equal_budget (v05)

- AMPContext C — pinneapple_neural.trainer.parallel (v04)
- ampere_maxwell f — pinneapple_physics.maxwell_checks (v02)
- AmpereLaw C — pinneapple_science.em.statics (v02)
- analyse f — pinneapple_design.bricks.statics (v06)
- analyse f — pinneapple_physics.astro.stars_ml (v02)
- analyse_assembly f — pinneapple_design.bricks.statics (v06)
- AnalysisResult C — pinneapple_analysis.verification.pipeline (v05)
- AnalysisResult C — pinneapple_systems.process_components.explicit_equation_system (v06)
- analytic_for f — pinneapple_physics.solving (v02)
- analytical_reference f — pinneapple_llm.qa.examples.pipe_wear_twin (v07)
- AnalyticalNode C — pinneapple_systems.cosimulation.node (v06)
- analyze f — pinneapple_analysis.verification.pipeline (v05)
- analyze f — pinneapple_data.data_health (v04)
- analyze f — pinneapple_systems.process_components.explicit_equation_system (v06)
- analyze_completeness f — pinneapple_tools.dataset_quality.checks (v07)
- analyze_distribution f — pinneapple_tools.dataset_quality.checks (v07)
- analyze_ep_tuning f — pinneapple_physics.non_hermitian.ep_tuning (v02)
- analyze_identifiability f — pinneapple_analysis.inverse_problems.identifiability (v05)
- analyze_model f — pinneapple_analysis (v05)
- analyze_pair f — pinneapple_perception.change_detection (v04)
- analyze_wheel_surrogate f — pinneapple_simulation.terramechanics.mobility (v03)
- angle_features f — pinneapple_worldmodel.robot_perception (v04)
- angular_diameter_deg f — pinneapple_physics.astro.journey (v02)
- angular_momentum f — pinneapple_physics.astro.nbody (v02)
- angular_momentum_ned f — pinneapple_simulation.flight.rigid_body (v03)
- angular_separation f — pinneapple_physics.astro.frames (v02)
- animate f — pinneapple_simulation.rover.viz (v03)
- animate f — pinneapple_simulation.vehicle.viz (v03)
- animate_ensemble f — pinneapple_physics.ensemble_viz (v02)
- animate_multiscale_ensemble f — pinneapple_physics.multiscale.viz (v02)
- animate_rolling_forecast f — pinneapple_systems.time_series.viz.animation (v06)
- animate_scalar_field f — pinneapple_tools.visualization.animation (v07)
- animate_streamlines f — pinneapple_tools.visualization.animation (v07)
- Animation C — pinneapple_tools.visualization.studio.animation (v07)
- Anisotropic C — pinneapple_simulation.electromagnetics.media (v03)
- annular_current_source f — pinneapple_simulation.numerical_solvers.eddy_current_fdm (v03)
- AnnularDomain2D C — pinneapple_design.geometry.gen.domains (v06)
- annulus f — pinneapple_design.geometry.bodies (v06)
- annulus f — pinneapple_design.geometry.csg (v06)
- annulus f — pinneapple_design.geometry.gen.sdf_shapes (v06)
- anomalies f — pinneapple_systems.time_series.enso.data (v06)
- anomaly_study f — pinneapple_systems.operations.contracts (v06)
- AnomalyEvent C — pinneapple_systems.digital_twin.monitoring.anomaly (v06)
- AnomalyMonitor C — pinneapple_systems.digital_twin.monitoring.anomaly (v06)
- Answer C — pinneapple_query.core (v02)
- answer_with_context f — pinneapple_llm.org_knowledge (v07)
- antenna_checks f — pinneapple_simulation.electromagnetics.antenna.checks (v03)
- antenna_surfaces f — pinneapple_simulation.electromagnetics.antenna.scene3d (v03)
- any_result_3d f — pinneapple_flow.nodes_ext.viz (v07)
- aperture_gain f — pinneapple_simulation.electromagnetics.antenna.bounds (v03)
- aperture_gain_dbi f — pinneapple_simulation.electromagnetics.antenna.bounds (v03)
- api f — pinneapple_hub.library.snippets (v07)
- APIKeyStore C — pinneapple_core.service.apikeys (v01)
- apis f — pinneapple_hub.library.sources (v07)
- aposteriori_test f — pinneapple_physics.multiscale.closures (v02)
- AposterioriReport C — pinneapple_physics.multiscale.closures (v02)

- app_available f — pinneapple_systems.operations.series (v06)
- app_dir f — pinneapple_flow.studio (v07)
- Apparent C — pinneapple_physics.astro.astrometry (v02)
- apparent f — pinneapple_physics.astro.astrometry (v02)
- apparent_magnitude_star f — pinneapple_physics.astro.journey (v02)
- ApplicabilityDomain C — pinneapple_analysis.uncertainty.applicability (v05)
- ApplicabilityMap C — pinneapple_veriphysics.applicability (v05)
- applicable f — pinneapple_analysis.surrogate.methods (v05)
- apply_baseline f — pinneapple_check.baseline (v05)
- apply_constraints f — pinneapple_decision.constraints (v05)
- apply_derived f — pinneapple_pdb.derived (v04)
- apply_dirichlet_bc f — pinneapple_simulation.numerical_solvers.nonlinear_beam_fem (v03)
- apply_filter f — pinneapple_physics.multiscale.filters (v02)
- apply_neumann_bc f — pinneapple_simulation.numerical_solvers.nonlinear_beam_fem (v03)
- apply_non_invention_policy f — pinneapple_problemdesign.policy (v02)
- apply_prior f — pinneapple_decision.debias (v05)
- apply_pyvista f — pinneapple_tools.visualization.studio.lighting (v07)
- apply_suggestions f — pinneapple_analysis.benchmark.advisor (v05)
- apply_trial f — pinneapple_flow.learn (v07)
- apply_tunables f — pinneapple_arena.autoresearch (v04)
- ApprovalDecision C — pinneapple_problemdesign.autonomy (v02)
- ApprovalGate C — pinneapple_problemdesign.autonomy (v02)
- ApprovalPolicy C — pinneapple_llm.workbench.tools (v07)
- ApprovalRequest C — pinneapple_problemdesign.autonomy (v02)
- apriori_test f — pinneapple_physics.multiscale.closures (v02)
- AprioriReport C — pinneapple_physics.multiscale.closures (v02)
- arc f — pinneapple_simulation.electromagnetics.machines.geometry (v03)
- arch_test f — pinneapple_systems.time_series.audit.tests (v06)
- architecture f — pinneapple_flow.learn (v07)
- architecture_diagram f — pinneapple_tools.nvidia.diagram (v07)
- architecture_gates f — pinneapple_analysis.benchmark.checks (v05)
- ArchitectureCandidate C — pinneapple_analysis.verification.architecture_recommendation (v05)
- ArchitectureCandidate C — pinneapple_arena.nas (v04)
- ArchitectureFamily C — pinneapple_arena.pointwise.schema (v04)
- ArchitectureInfo C — pinneapple_arena.pointwise.schema (v04)
- ArchitectureNotApplicable C — pinneapple_arena.pointwise.adapters (v04)
- ArchitectureRecommendation C — pinneapple_analysis.verification.architecture_recommendation (v05)
- ArchitectureSearchSpace C — pinneapple_arena.nas (v04)
- ArchitectureUnavailable C — pinneapple_arena.pointwise.training (v04)
- ArchiveLimits C — pinneapple_security.archives (v05)
- ArchMeta C — pinneapple_arena.external.schema (v04)
- arco_states f — pinneapple_physics.weather.events (v02)
- ARDGP C — pinneapple_presets.surrogate (v02)
- are_compatible f — pinneapple_llm.qa.units (v07)
- Area C — pinneapple_science.atlas (v02)
- area f — pinneapple_design.geometry_library.meshops (v06)
- area_patch f — pinneapple_simulation.external_solvers.openfoam.campaign (v03)
- area_ratio f — pinneapple_physics.closed_form.rocket_nozzle (v02)
- Arena C — pinneapple_arena.arena (v04)
- Arena C — pinneapple_tools.benchmark_suite.api (v07)
- arena f — pinneapple_flow.nodes.methods (v07)
- arena f — pinneapple_flow.nodes_ext.validation (v07)
- arena_choice f — pinneapple_arena.decision_mode (v04)
- arena_markdown f — pinneapple_arena.regimes (v04)
- arena_model_rules f — pinneapple_arena.decision_mode (v04)
- arena_problem_facts f — pinneapple_arena.decision_mode (v04)

- arena_run f — pinneapple_flow.nodes_ext.arena_pipelines (v07)
- arena_study_result f — pinneapple_analysis.surrogate.contracts (v05)
- arena_yaml f — pinneapple_flow.nodes_ext.benchmarks (v07)
- ArenaBudget C — pinneapple_analysis.validation.arena.loop (v05)
- ArenaCompareResult C — pinneapple_tools.benchmark_suite.api (v07)
- ArenaConfig C — pinneapple_arena.config (v04)
- ArenaDecisionReport C — pinneapple_arena.decision_mode (v04)
- ArenaExecutor C — pinneapple_arena.decision_mode (v04)
- ArenaProblem C — pinneapple_arena.problems (v04)
- ArenaReport C — pinneapple_analysis.validation.arena.loop (v05)
- ArenaResult C — pinneapple_arena.regimes (v04)
- ArenaResult C — pinneapple_tools.benchmark_suite.api (v07)
- ArenaTask C — pinneapple_tools.benchmark_suite.tasks.base (v07)
- ARFit C — pinneapple_systems.time_series.predictability (v06)
- ARForecaster C — pinneapple_systems.time_series.forecast_lab.baselines (v06)
- ARIMA C — pinneapple_neural.architectures.classical_ts.arima (v04)
- ARIMAForecaster C — pinneapple_systems.time_series.forecast_lab.baselines (v06)
- arm_camera_sample f — pinneapple_worldmodel.robot_perception (v04)
- arm_motions f — pinneapple_worldmodel.robot_perception (v04)
- arm_pick_and_place f — pinneapple_simulation.robot_sim.checks (v03)
- ArmDesign C — pinneapple_design.surrogate_inverse.suspension_arm (v06)
- ArrayProblem C — pinneapple_analysis.benchmark.builtin.fields (v05)
- ARResidualDetector C — pinneapple_security.cps_bench (v05)
- arrhenius_rate_constant f — pinneapple_systems.process_components.reaction_kinetics (v06)
- ArrheniusModel C — pinneapple_analysis.inverse_problems.models (v05)
- ArticulationSpec C — pinneapple_simulation.robot_sim.spec (v03)
- Artifact C — pinneapple_data.lineage.graph (v04)
- artifact f — pinneapple_flow.runtime (v07)
- ArtifactNotPublishedError C — pinneapple_hub.zoo._artifacts (v07)
- ArtifactRecord C — pinneapple_llm.qa.report (v07)
- ArtifactRegistry C — pinneapple_registry.registry (v01)
- ArtifactSpec C — pinneapple_tools.automation.workflow (v07)
- ArtifactUnavailableError C — pinneapple_hub.zoo._artifacts (v07)
- arxiv_pdf_url f — pinneapple_tools.hpo_experiments.sources.arxiv_pdf (v07)
- ArxivPaper C — pinneapple_tools.hpo_experiments.sources.arxiv_source (v07)
- ArxivSource C — pinneapple_tools.hpo_experiments.sources.arxiv_source (v07)
- as_array_dict f — pinneapple_systems.digital_twin.product.model (v06)
- as_dict f — pinneapple_analysis.validation.report_artifact (v05)
- as_dicts f — pinneapple_llm.workbench.evaluate (v07)
- as_field_model f — pinneapple_analysis.validation.validate (v05)
- as_flow f — pinneapple_flow.schema (v07)
- as_graph_outputs f — pinneapple_hub.zoo.export (v07)
- as_mesh f — pinneapple_design.qualitative.descriptors (v06)
- as_numpy_closure f — pinneapple_physics.multiscale.learned (v02)
- as_parameter f — pinneapple_analysis.uncertainty.distributions (v05)
- as_problem f — pinneapple_physics.solving (v02)
- as_space f — pinneapple_analysis.uncertainty.distributions (v05)
- as_tensor f — pinneapple_neural.utils (v04)
- assemble f — pinneapple_condense.fv (v02)
- assemble f — pinneapple_condense.meshless (v02)
- assemble_blocks f — pinneapple_condense.learned (v02)
- assemble_oseen f — pinneapple_condense.flow (v02)
- assemble_package f — pinneapple_condense.fv (v02)
- assemble_scalar f — pinneapple_condense.flow (v02)
- assembled_mesh f — pinneapple_simulation.rover.geometry (v03)
- Assembly C — pinneapple_design.bricks.assembly (v06)

- Assembly C — pinneapple_design.qualitative.descriptors (v06)
- assembly_meshes f — pinneapple_design.bricks.mesh (v06)
- assert_no_code f — pinneapple_veriphysics.validation_lab (v05)
- assert_valid_physical_sample f — pinneapple_data.validators (v04)
- assess f — pinneapple_data.simulation_metadata.calculix (v04)
- assess f — pinneapple_design.intent.pipeline (v06)
- assess f — pinneapple_security.compliance (v05)
- assess_candidates f — pinneapple_autolab.trust (v07)
- assess_drift f — pinneapple_analysis.uncertainty.drift (v05)
- assess_experiment f — pinneapple_lab.curation (v07)
- assess_mesh_quality f — pinneapple_analysis.verification.mesh_intelligence (v05)
- assess_run f — pinneapple_lab.curation (v07)
- assess_steady f — pinneapple_data.simulation_metadata.convergence (v04)
- assess_table f — pinneapple_data.simulation_metadata.convergence (v04)
- assess_transient f — pinneapple_data.simulation_metadata.convergence (v04)
- Assessment C — pinneapple_design.intent.pipeline (v06)
- Assessment C — pinneapple_lab.curation (v07)
- assign_patches f — pinneapple_simulation.pipelines.freecad_openfoam.cad (v03)
- Assumption C — pinneapple_physics.physics_audit.assumptions (v02)
- Assumption C — pinneapple_problemdesign.schema (v02)
- AssumptionAudit C — pinneapple_physics.physics_audit.assumptions (v02)
- AssumptionCheck C — pinneapple_physics.physics_audit.assumptions (v02)
- astro_constant_si f — pinneapple_physics.pde_environment.presets.astropy_bridge (v02)
- astroml_available f — pinneapple_physics.pde_environment.presets.astroml_bridge (v02)
- Atlas C — pinneapple_science.atlas (v02)
- atlas f — pinneapple_flow.nodes.science (v07)
- atmosphere f — pinneapple_physics.closed_form.pump_npsh (v02)
- atmospheric_pressure f — pinneapple_reason.props (v05)
- attach_geometry f — pinneapple_data.adapters.geom_adapter (v04)
- attach_runs f — pinneapple_hub.library (v07)
- attach_snippets f — pinneapple_data.preflight (v04)
- attach_upd_state f — pinneapple_data.adapters.upd_adapter (v04)
- attached_flow f — pinneapple_physics.physics_audit.assumptions (v02)
- attest_and_sign_experiment f — pinneapple_security.integrate (v05)
- attest_experiment f — pinneapple_security.attestation (v05)
- attest_files f — pinneapple_security.attestation (v05)
- attribute_failure f — pinneapple_physics.physics_audit.attribution (v02)
- Attribution C — pinneapple_physics.physics_audit.attribution (v02)
- auc f — pinneapple_analysis.validation.fault_injection.matrix (v05)
- auc f — pinneapple_systems.sports.football.metrics (v06)
- audit f — pinneapple_flow.nodes_ext.validation (v07)
- audit f — pinneapple_hub.zoo.datasets (v07)
- audit_assumptions f — pinneapple_physics.physics_audit.assumptions (v02)
- audit_card f — pinneapple_hub.zoo.datasets (v07)
- audit_claims f — pinneapple_autolab.agent (v07)
- audit_dataset f — pinneapple_data.audit (v04)
- audit_model f — pinneapple_analysis.validation.arena.loop (v05)
- audit_zoo_dataset f — pinneapple_data.audit (v04)
- AuditCard C — pinneapple_data.audit (v04)
- AuditCheck C — pinneapple_data.audit (v04)
- AuditLog C — pinneapple_security.audit (v05)
- AuditReport C — pinneapple_systems.time_series.audit.report (v06)
- AuditSection C — pinneapple_systems.time_series.audit.report (v06)
- augment_interpolation f — pinneapple_data.augmentation (v04)
- augment_noise f — pinneapple_data.augmentation (v04)
- augment_transform f — pinneapple_data.augmentation (v04)

- `AugmentedLagrangian` C — `pinneapple_neural.trainer.loss_balancer` (v04)
- `aurora_probability` f — `pinneapple_physics.astro.spaceweather` (v02)
- `AuthAudit` C — `pinneapple_security.web` (v05)
- `author_articulation` f — `pinneapple_tools.nvidia.isaac` (v07)
- `auto_call` f — `pinneapple_hub.library.snippets` (v07)
- `auto_procs` f — `pinneapple_simulation.pipelines.freecad_openfoam.steps` (v03)
- `auto_residual` f — `pinneapple_physics.symbolic_pde.compiler` (v02)
- `auto_specs` f — `pinneapple_flow.libcalls` (v07)
- `AutoARIMAForecaster` C — `pinneapple_systems.time_series.classical_wrappers` (v06)
- `AutoBalancer` C — `pinneapple_neural.trainer.loss_balancer` (v04)
- `AutoBalancerConfig` C — `pinneapple_neural.trainer.loss_balancer` (v04)
- `AutoBatchSizeFinder` C — `pinneapple_neural.trainer.hpc` (v04)
- `AutoCorrelation` C — `pinneapple_neural.architectures.transformers.autoformer` (v04)
- `AutoCorrelationLayer` C — `pinneapple_neural.architectures.transformers.autoformer` (v04)
- `Autoencoder2D` C — `pinneapple_neural.architectures.autoencoders.ae_2d` (v04)
- `AutoEncoderAdapter` C — `pinneapple_neural.architectures.adapters.ae` (v04)
- `AutoencoderCatalog` C — `pinneapple_neural.architectures.autoencoders.registry` (v04)
- `Autoformer` C — `pinneapple_neural.architectures.transformers.autoformer` (v04)
- `AutonomousLab` C — `pinneapple_autolab.lab` (v07)
- `AutonomousResearchAgent` C — `pinneapple_problemdesign.research_loop` (v02)
- `AutonomyLevel` C — `pinneapple_problemdesign.autonomy` (v02)
- `AutoPDEWeightCalibrator` C — `pinneapple_neural.trainer.loss_balancer` (v04)
- `Autopilot` C — `pinneapple_simulation.flight.control` (v03)
- `AutoResearch` C — `pinneapple_arena.autoresearch` (v04)
- `ava_simulator` f — `pinneapple_simulation.electromagnetics.antenna.runner` (v03)
- `ava_transform` f — `pinneapple_simulation.electromagnetics.antenna.design` (v03)
- `available` f — `pinneapple_blender.cycles` (v07)
- `available` f — `pinneapple_lab.spec` (v07)
- `available` f — `pinneapple_llm.qa.validators.base` (v07)
- `available` f — `pinneapple_simulation.electromagnetics.machines.getdp` (v03)
- `available` f — `pinneapple_simulation.rover.autolab.render` (v03)
- `available_bodies` f — `pinneapple_physics.astro.smallbodies` (v02)
- `available_methods` f — `pinneapple_analysis.inverse_problems.studio` (v05)
- `available_methods` f — `pinneapple_design.design_optimizer.methods` (v06)
- `available_orchestrator_tools` f — `pinneapple_problemdesign.knowledge.mapping` (v02)
- `AVAParams` C — `pinneapple_simulation.electromagnetics.antenna.ava` (v03)
- `axial_compressor_cascade_2d` f — `pinneapple_physics.pde_environment.presets.turbomachinery` (v02)
- `axial_compressor_meanline` f — `pinneapple_physics.pde_environment.presets.turbomachinery` (v02)
- `axial_compressor_stage_3d` f — `pinneapple_physics.pde_environment.presets.turbomachinery` (v02)
- `axial_fan` f — `pinneapple_design.geometry.gen.cadquery_advanced` (v06)
- `axial_flux_density` f — `pinneapple_simulation.numerical_solvers.eddy_current_fdm` (v03)
- `axis_angle` f — `pinneapple_simulation.multibody.tree` (v03)
- `axis_binned_profile` f — `pinneapple_tools.model_evaluation.metrics` (v07)
- `axis_time` f — `pinneapple_physics.astro.observing` (v02)
- `axisymmetric_linear_elasticity_2d_default` f — `pinneapple_physics.pde_environment.presets.solid_mechanics` (v02)

B

- `b64url` f — `pinneapple_hub.library.flowlink` (v07)
- `back_emf` f — `pinneapple_simulation.electromagnetics.postprocess` (v03)
- `Backend` C — `pinneapple_llm.workbench.backend` (v07)
- `Backend` C — `pinneapple_tools.benchmark_suite.backends.base` (v07)
- `Backend` C — `pinneapple_tools.compute_backends.backend` (v07)
- `backend_info` f — `pinneapple_quantum` (v04)
- `BackendConfig` C — `pinneapple_quantum.backends.backend` (v04)
- `backtest` f — `pinneapple_systems.digital_twin.product.commission` (v06)

- backtest f — pinneapple_twin3d.wear.forecast (v06)
- BacktestConfig C — pinneapple_systems.time_series.validation.backtest (v06)
- BacktestResult C — pinneapple_systems.time_series.validation.backtest (v06)
- BacktestRunner C — pinneapple_systems.time_series.validation.backtest (v06)
- badge f — pinneapple_analysis.validation.report_artifact (v05)
- badge f — pinneapple_science.plots (v02)
- balance f — pinneapple_simulation.flight.mass (v03)
- balance_check f — pinneapple_simulation.rover.checks (v03)
- balance_residual f — pinneapple_security.anomaly (v05)
- Balancer C — pinneapple_core.loss (v01)
- BalanceRule C — pinneapple_systems.operations.anomaly (v06)
- Ball C — pinneapple_systems.sports.football.ball (v06)
- ballistic_from_bstar f — pinneapple_physics.astro.decay (v02)
- balloon f — pinneapple_simulation.electromagnetics.antenna.scene3d (v03)
- band_difference f — pinneapple_perception.change_detection (v04)
- band_metrics f — pinneapple_simulation.electromagnetics.antenna.design (v03)
- band_ratios f — pinneapple_simulation.flight.verification (v03)
- BandChangeResult C — pinneapple_perception.change_detection (v04)
- banded f — pinneapple_design.surrogate_inverse.core (v06)
- bands f — pinneapple_hub.zoo.demo (v07)
- BarWear C — pinneapple_lab.experiments.wear (v07)
- BarWear C — pinneapple_physics.tribology.archard (v02)
- BarWearRanking C — pinneapple_lab.experiments.wear (v07)
- base_fraction f — pinneapple_systems.process_components.reaction_kinetics (v06)
- base_meta f — pinneapple_tools.benchmark_suite.crossframework.protocol (v07)
- base_name f — pinneapple_design.geometry_library.entries._adapt (v06)
- base_pattern f — pinneapple_twin3d.wear.synthetic (v06)
- base_theta f — pinneapple_design.thermal.heatsink.engine (v06)
- baseline f — pinneapple_design.aero.optimize (v06)
- baseline_errors f — pinneapple_systems.time_series.predictability (v06)
- baseline_specs f — pinneapple_systems.time_series.forecast_lab.presets (v06)
- baseline_x f — pinneapple_design.aero.aircraft3d (v06)
- baseline_x_al f — pinneapple_design.aero.airliner3d (v06)
- baselines f — pinneapple_analysis.validation.physics_learning (v05)
- BaseModel C — pinneapple_neural.architectures.base (v04)
- BaseStream C — pinneapple_systems.digital_twin.io.stream (v06)
- basquin_allowable_stress_pa f — pinneapple_physics.closed_form.cantilever_fatigue (v02)
- basquin_fatigue_life_cycles f — pinneapple_physics.closed_form.cantilever_fatigue (v02)
- Batch C — pinneapple_core.data (v01)
- Batch C — pinneapple_data.upd_types (v04)
- batch_means_ci f — pinneapple_analysis.verification.forecast_skill (v05)
- batched_inference f — pinneapple_neural.predictor.infer (v04)
- BatchPrior C — pinneapple_decision.debias (v05)
- BatteryCellVanillaPINN C — pinneapple_systems.component_library.components (v06)
- bayesian_block_edges f — pinneapple_physics.pde_environment.presets.astroml_bridge (v02)
- BayesianDesignOptimizer C — pinneapple_design.design_optimizer.optimizer (v06)
- BayesianLinear C — pinneapple_neural.architectures.pinns.bpinn (v04)
- BayesianPINN C — pinneapple_neural.architectures.pinns.bpinn (v04)
- BayesianRNN C — pinneapple_neural.architectures.continuous.bayesian_rnn (v04)
- bbox_size f — pinneapple_design.surrogate_inverse.meshes (v06)
- beam f — pinneapple_design.intent.physics.bracket (v06)
- beam_column_moment_amplification_factor f — pinneapple_systems.process_components.pipe_stress_mechanics (v06)
- beam_deflection f — pinneapple_flow.wrappers (v07)
- beam_matrices f — pinneapple_simulation.numerical_solvers.beam_fe (v03)
- beam_midspan_deflection f — pinneapple_design.bricks.closed_form (v06)

- beam_modes f — pinneapple_simulation.numerical_solvers.beam_fe (v03)
- beam_moment f — pinneapple_design.bricks.closed_form (v06)
- beam_reactions f — pinneapple_design.bricks.closed_form (v06)
- beam_static f — pinneapple_simulation.numerical_solvers.beam_fe (v03)
- BeamBVPFDMSolver C — pinneapple_simulation.numerical_solvers.beam_bvp_fdm (v03)
- BeamResult C — pinneapple_systems.process_components.beam_statics (v06)
- bearing_capacity_factors f — pinneapple_simulation.terramechanics.wong_reece (v03)
- beauty f — pinneapple_simulation.vehicle.viz (v03)
- beauty_view f — pinneapple_simulation.vehicle.viz (v03)
- before_after_series f — pinneapple_data.unified (v04)
- bekker_rigid_wheel_sinkage f — pinneapple_simulation.rover.checks (v03)
- bekker_wong_forces_torch f — pinneapple_simulation.numerical_solvers.bekker_wong (v03)
- bekker_wong_surrogate_2d f — pinneapple_physics.pde_environment.presets.terramechanics (v02)
- BekkerWongContact C — pinneapple_simulation.rover.contact (v03)
- BekkerWongSolver C — pinneapple_simulation.numerical_solvers.bekker_wong (v03)
- belt_winding f — pinneapple_simulation.electromagnetics.machine (v03)
- Benchmark C — pinneapple_physics.compile_api (v02)
- benchmark f — pinneapple_hub.zoo (v07)
- benchmark f — pinneapple_mcp.tools (v07)
- benchmark_catalog f — pinneapple_pdb.benchmarks (v04)
- benchmark_closures f — pinneapple_physics.multiscale.turbulence (v02)
- benchmark_component_type f — pinneapple_systems.component_library.benchmark (v06)
- benchmark_component_type_task f — pinneapple_orchestration.tasks.benchmark_tasks (v01)
- benchmark_dataset f — pinneapple_arena.dataset_bench (v04)
- benchmark_faults f — pinneapple_systems.operations.anomaly (v06)
- BenchmarkCase C — pinneapple_lab.experiments.landing (v07)
- BenchmarkComparison C — pinneapple_data.physics_case (v04)
- BenchmarkConfig C — pinneapple_analysis.benchmark.config (v05)
- BenchmarkConfig C — pinneapple_tools.benchmark_suite.benchmark (v07)
- BenchmarkEntry C — pinneapple_pdb.benchmarks (v04)
- BenchmarkProblem C — pinneapple_analysis.validation.fault_injection.problems (v05)
- BenchmarkReport C — pinneapple_tools.benchmark_suite.report (v07)
- BenchmarkResult C — pinneapple_analysis.benchmark.api (v05)
- BenchmarkResult C — pinneapple_tools.benchmark_suite.benchmark (v07)
- BenchmarkResult C — pinneapple_worldmodel.benchmark (v04)
- BenchmarkTask C — pinneapple_worldmodel.benchmark (v04)
- BenchmarkTaskBase C — pinneapple_tools.benchmark_suite.benchmark (v07)
- bending_stress_at_position_pa f — pinneapple_design.surrogate_inverse.suspension_arm (v06)
- bending_stress_from_curvature f — pinneapple_systems.process_components.pipe_stress_mechanics (v06)
- bending_stress_pa f — pinneapple_design.surrogate_inverse.suspension_arm (v06)
- benjamini_hochberg f — pinneapple_systems.time_series.metrics_ext.comparison (v06)
- best_device f — pinneapple_arena.pointwise.training (v04)
- best_device f — pinneapple_neural.trainer.parallel (v04)
- best_method f — pinneapple_core.onboarding.catalog (v01)
- best_period f — pinneapple_physics.pde_environment.presets.astroml_bridge (v02)
- beveled_box f — pinneapple_design.geometry.vehicle_cad (v06)
- BHCurve C — pinneapple_simulation.electromagnetics.materials (v03)
- BiasModel C — pinneapple_systems.digital_twin.product.model (v06)
- BiGRUModel C — pinneapple_neural.architectures.recurrent.gru (v04)
- Biharmonic2D C — pinneapple_arena.problems (v04)
- Bilinear C — pinneapple_analysis.benchmark.builtin.standard (v05)
- bilinear_mac f — pinneapple_condense.meshless2d (v02)
- BiLSTMModel C — pinneapple_neural.architectures.recurrent.lstm (v04)
- BinaryCheck C — pinneapple_simulation.external_solvers.modelica.fmu_inspect (v03)
- biot f — pinneapple_physics.physics_audit.regime (v02)

- biot_savart f — pinneapple_simulation.electromagnetics.statics (v03)
- BiotSavart C — pinneapple_science.em.statics (v02)
- black_scholes_1d f — pinneapple_physics.pde_environment.presets.multidisciplinary (v02)
- blackbody_rgb f — pinneapple_physics.astro.stars (v02)
- blackbody_rgb f — pinneapple_physics.blackhole.raytrace (v02)
- blackbody_spectrum f — pinneapple_physics.astro.stars (v02)
- BlackBoxNode C — pinneapple_systems.cosimulation.node (v06)
- BlackHoleForecast C — pinneapple_lab.experiments.blackhole_forecast (v07)
- BlackScholes1D C — pinneapple_arena.problems (v04)
- blasius f — pinneapple_physics.closed_form.pipe_flow (v02)
- blend_sections f — pinneapple_design.geometry_library.kit (v06)
- blender_available f — pinneapple_design.bricks.visuals (v06)
- blender_axes f — pinneapple_tools.visualization.studio.blender (v07)
- blender_clip f — pinneapple_flow.nodes_ext.render (v07)
- blender_world f — pinneapple_tools.visualization.studio.lighting (v07)
- Block C — pinneapple_data.cae.inp (v04)
- Block C — pinneapple_physics.weather.model (v02)
- Block C — pinneapple_tools.nvidia.diagram (v07)
- block_3d f — pinneapple_simulation.numerical_solvers.heat_fv_reference (v03)
- block_average f — pinneapple_simulation.numerical_solvers.operator_datasets (v03)
- block_bootstrap f — pinneapple_analysis.uncertainty.resampling (v05)
- block_bootstrap_indices f — pinneapple_analysis.verification.forecast_skill (v05)
- block_bootstrap_mean f — pinneapple_analysis.verification.forecast_skill (v05)
- block_bootstrap_ratio f — pinneapple_analysis.verification.forecast_skill (v05)
- block_functionals f — pinneapple_condense.substructure (v02)
- block_indicator f — pinneapple_condense.exceptions (v02)
- block_model f — pinneapple_condense.substructure (v02)
- block_observables f — pinneapple_condense.learned (v02)
- block_precomputed f — pinneapple_analysis.inverse_problems.heat_transfer (v05)
- blocked f — pinneapple_llm.qa.validators.base (v07)
- blocked_hindcast f — pinneapple_systems.time_series.enso.evaluation (v06)
- blocking_reasons f — pinneapple_llm.qa.levels (v07)
- BlockModel C — pinneapple_condense.substructure (v02)
- blocks f — pinneapple_hub.library.compose (v07)
- blocks_net f — pinneapple_flow.learn (v07)
- bloom f — pinneapple_physics.blackhole.raytrace (v02)
- Board C — pinneapple_physics.pcb.solver (v02)
- board_cooling f — pinneapple_flow.nodes.autolab (v07)
- board_scene f — pinneapple_design.thermal.board_cooling.scene (v06)
- bodies_from_items f — pinneapple_design.geometry_library.entries._adapt (v06)
- Body C — pinneapple_design.bricks.statics (v06)
- Body C — pinneapple_design.geometry_library.core (v06)
- Body C — pinneapple_simulation.external_solvers.newton_physics.scene (v03)
- Body C — pinneapple_simulation.multibody.tree (v03)
- body_of_sections f — pinneapple_design.geometry_library.kit (v06)
- body_position f — pinneapple_physics.astro.journey (v02)
- body_rotation f — pinneapple_simulation.rover.geometry (v03)
- body_vertices f — pinneapple_simulation.flight.visuals (v03)
- BodyLimit C — pinneapple_security.web (v05)
- BodyState C — pinneapple_analysis.uncertainty.reachability (v05)
- bof_converter f — pinneapple_twin3d.wear.presets (v06)
- bolometric_correction f — pinneapple_physics.astro.stars (v02)
- bondi_pw f — pinneapple_physics.blackhole.hydro (v02)
- BondiAccretion C — pinneapple_lab.experiments.accretion (v07)
- bootstrap_confidence_interval f — pinneapple_tools.model_evaluation.metrics (v07)
- bootstrap_peak_distribution f — pinneapple_physics.pde_environment.presets.astroml_bridge (v02)

- `border_points` f — `pinneapple_tools.visualization.studio.planetary` (v07)
- `border_visibility` f — `pinneapple_tools.visualization.studio.planetary` (v07)
- `bounce` f — `pinneapple_simulation.external_solvers.newton_physics.validation` (v03)
- `bouncing_ball_scene` f — `pinneapple_worldmodel.physical_world` (v04)
- `Bound` C — `pinneapple_condense.query` (v02)
- `Bound` C — `pinneapple_lab.results_rules` (v07)
- `boundaries` f — `pinneapple_physics.blackhole.hydro_numba` (v02)
- `Boundary` C — `pinneapple_simulation.geophysics.richards` (v03)
- `boundary` f — `pinneapple_core.loss` (v01)
- `boundary` f — `pinneapple_data.simulation_metadata.foam_dict` (v04)
- `boundary_check` f — `pinneapple_physics.physics_audit.monitor` (v02)
- `boundary_conditions` f — `pinneapple_data.cae.foam` (v04)
- `boundary_edges` f — `pinneapple_simulation.numerical_solvers.plane_stress_fem` (v03)
- `boundary_flux_balance` f — `pinneapple_physics.darcy_checks` (v02)
- `boundary_nodes` f — `pinneapple_simulation.electromagnetics.mesh` (v03)
- `boundary_rect` f — `pinneapple_simulation.numerical_solvers._bc` (v03)
- `boundary_values` f — `pinneapple_physics.darcy_checks` (v02)
- `boundary_vertices` f — `pinneapple_design.geometry.ops.curvature` (v06)
- `BoundaryCheck` C — `pinneapple_analysis.validation.boundary` (v05)
- `BoundaryConditionSpec` C — `pinneapple_app.backend.core.problem` (v07)
- `BoundaryLoop` C — `pinneapple_analysis.verification.geometry_intelligence` (v05)
- `BoundaryRegion` C — `pinneapple_design.geometry.gen.domains` (v06)
- `BoundaryRegion` C — `pinneapple_worldmodel.geometry` (v04)
- `BoundarySampler` C — `pinneapple_core.data` (v01)
- `bounds_check` f — `pinneapple_physics.physics_audit.monitor` (v02)
- `bounds_of` f — `pinneapple_flow.nodes._common` (v07)
- `boussinesq` f — `pinneapple_physics.physics_audit.assumptions` (v02)
- `box` f — `pinneapple_design.geometry.bodies` (v06)
- `box` f — `pinneapple_design.geometry.vehicle_cad` (v06)
- `box` f — `pinneapple_design.geometry_library.kit` (v06)
- `box` f — `pinneapple_presets.viz3d` (v02)
- `box` f — `pinneapple_simulation.numerical_solvers.lbm_tunnel` (v03)
- `box` f — `pinneapple_simulation.particle_dynamics.geo_mpm` (v03)
- `box` f — `pinneapple_twin3d.primitives` (v06)
- `Box3D` C — `pinneapple_worldmodel.geometry` (v04)
- `box3d` f — `pinneapple_design.geometry.gen.sdf_shapes` (v06)
- `box_beam` f — `pinneapple_design.geometry.vehicle_cad` (v06)
- `box_face_normals` f — `pinneapple_physics.pde_environment.condition_sampling` (v02)
- `box_index` f — `pinneapple_systems.time_series.enso.forecasters` (v06)
- `box_inertia` f — `pinneapple_simulation.robot_sim.spec` (v03)
- `box_mean` f — `pinneapple_systems.time_series.enso.data` (v06)
- `box_mesh` f — `pinneapple_design.bricks.mesh` (v06)
- `box_mesh` f — `pinneapple_simulation.numerical_solvers.solid_fem` (v03)
- `box_sampler` f — `pinneapple_analysis.validation.metamorphic` (v05)
- `BoxBoundary` C — `pinneapple_simulation.numerical_solvers.sph_boundaries` (v03)
- `BoxConstraint` C — `pinneapple_design.design_optimizer.constraints` (v06)
- `BPINNConfig` C — `pinneapple_neural.architectures.pinns.bpinn` (v04)
- `bracket_figures` f — `pinneapple_design.intent.figures` (v06)
- `BracketTemplate` C — `pinneapple_design.intent.templates.bracket` (v06)
- `BrauerCurve` C — `pinneapple_simulation.electromagnetics.materials` (v03)
- `BrazilDataCubeClient` C — `pinneapple_data.datasets.external_sources` (v04)
- `brep_available` f — `pinneapple_tools.nvidia.brep` (v07)
- `brep_python` f — `pinneapple_tools.nvidia.brep` (v07)
- `BRepReport` C — `pinneapple_tools.nvidia.brep` (v07)
- `brewster_angle` f — `pinneapple_simulation.electromagnetics.waves` (v03)
- `Brick` C — `pinneapple_design.bricks.assembly` (v06)

- brick_mesh f — pinneapple_design.bricks.mesh (v06)
- BricksExperiment C — pinneapple_lab.experiments.bricks (v07)
- BridgeGoal C — pinneapple_design.bricks.goals (v06)
- brier f — pinneapple_systems.sports.football.metrics (v06)
- bright_stars_in_field f — pinneapple_physics.astro.lensing (v02)
- BrooksCorey C — pinneapple_simulation.geophysics.richards (v03)
- Buckle C — pinneapple_simulation.external_solvers.calculix.study (v03)
- Budget C — pinneapple_analysis.cost.profiler (v05)
- Budget C — pinneapple_autolab.mission (v07)
- BudgetExceeded C — pinneapple_analysis.cost.profiler (v05)
- BudgetExhaustedError C — pinneapple_autolab.ledger (v07)
- BudgetRun C — pinneapple_analysis.cost.equal_budget (v05)
- build f — pinneapple_data.cae.report (v04)
- build f — pinneapple_design.geometry_library.core (v06)
- build f — pinneapple_flow.learn (v07)
- build f — pinneapple_hub.library (v07)
- build f — pinneapple_hub.zoo.validation_reports (v07)
- build f — pinneapple_lab.experiments.structures (v07)
- build f — pinneapple_simulation.flight.catalog (v03)
- build_1d_linear_interpolant f — pinneapple_tools.dataset_quality.checks (v07)
- build_3d f — pinneapple_simulation.electromagnetics.antenna.scene3d (v03)
- build_3d_live_twin f — pinneapple_llm.twin_draft (v07)
- build_and_hold_profile f — pinneapple_systems.process_components.curved_path_geometry (v06)
- build_app f — pinneapple_tools.visualization.integrations.trame_app (v07)
- build_applicability_map f — pinneapple_veriphysics.applicability (v05)
- build_arrays f — pinneapple_physics.blackhole.dataset (v02)
- build_axisymmetric_helmholtz_system f — pinneapple_simulation.numerical_solvers.eddy_current_fdm (v03)
- build_benchmark_payload f — pinneapple_app.backend.core.benchmark (v07)
- build_circuit f — pinneapple_quantum (v04)
- build_classical_circuit f — pinneapple_quantum.circuits.base (v04)
- build_consistency_prompt f — pinneapple_tools.dataset_quality.vlm_curation (v07)
- build_corpus f — pinneapple_analysis.retrieval.corpus (v05)
- build_csp f — pinneapple_security.web (v05)
- build_dashboard f — pinneapple_autolab.dashboard (v07)
- build_dashboard f — pinneapple_autolab.domains.racecar (v07)
- build_dataset f — pinneapple_data.cae.dataset (v04)
- build_dataset f — pinneapple_systems.time_series.enso.dataset (v06)
- build_datasheet f — pinneapple_simulation.electromagnetics.reflector.study_report_text (v03)
- build_default_index f — pinneapple_analysis.retrieval (v05)
- build_default_mapping_atmosphere f — pinneapple_physics.pinn_solver.io.mappings (v02)
- build_definitions f — pinneapple_systems.process_components.explicit_equation_system (v06)
- build_digital_twin f — pinneapple_systems.digital_twin.twin (v06)
- build_digital_twin_for_model f — pinneapple_neural.physicsnemo_bridge.adapters (v04)
- build_document f — pinneapple_simulation.pipelines.freecad_openfoam.models (v03)
- build_domain f — pinneapple_design (v06)
- build_evidence_graph f — pinneapple_analysis.verification.evidence_graph (v05)
- build_figures f — pinneapple_simulation.electromagnetics.reflector.study_report_figures (v03)
- build_fmu f — pinneapple_simulation.external_solvers.modelica.omc (v03)
- build_frame f — pinneapple_design.bricks.visuals (v06)
- build_from_bundle f — pinneapple_data.adapters.pinn_batch_builders (v04)
- build_from_real_data f — pinneapple_data.adapters.pinn_batch_builders (v04)
- build_from_solver f — pinneapple_data.adapters.pinn_batch_builders (v04)
- build_gauss_legendre_tableau f — pinneapple_simulation.numerical_solvers.irk_gauss_legendre (v03)
- build_geometry_asset f — pinneapple_design.geometry.core.registry (v06)
- build_heated_pipe f — pinneapple_simulation.pipelines.freecad_openfoam.freecad_scripts.fc_job (v03)

- build_journal f — pinneapple_simulation.external_solvers.ansys.journal_builder (v03)
- build_kb f — pinneapple_tools.hpo_experiments.pipelines.build_kb (v07)
- build_knowledge_graph f — pinneapple_analysis.verification.knowledge_graph (v05)
- build_lattice f — pinneapple_design.aero.vlm (v06)
- build_literature_corpus f — pinneapple_analysis.retrieval.corpus (v05)
- build_loss f — pinneapple_neural.trainer.losses (v04)
- build_manifest f — pinneapple_security.integrity (v05)
- build_mesh f — pinneapple_design.geometry.gen.primitives (v06)
- build_mesh_from_cadquery_object f — pinneapple_design.geometry.gen.cadquery_gen (v06)
- build_mesh_from_parametric_spec f — pinneapple_design.geometry.gen.cadquery_gen (v06)
- build_mesh_from_step f — pinneapple_design.geometry.gen.cadquery_gen (v06)
- build_metrics_from_cfg f — pinneapple_neural.trainer.metrics_cfg (v04)
- build_model f — pinneapple_arena.model_factory (v04)
- build_model f — pinneapple_neural (v04)
- build_neighbor_list f — pinneapple_simulation.numerical_solvers.neighbors (v03)
- build_package f — pinneapple_veriphysics.evidence_package (v05)
- build_pennylane_circuit f — pinneapple_quantum.circuits.base (v04)
- build_physical_sample_dataloader f — pinneapple_data.dataloaders (v04)
- build_physics f — pinneapple_app.backend.core.physics_adapter (v07)
- build_pinneapple_spec f — pinneapple_problemdesign.codegen (v02)
- build_pipe_bend f — pinneapple_simulation.pipelines.freecad_openfoam.freecad_scripts.fc_job (v03)
- build_plan f — pinneapple_problemdesign.knowledge.mapping (v02)
- build_plan_cfd_first f — pinneapple_problemdesign.knowledge.mapping (v02)
- build_plan_fno_first f — pinneapple_problemdesign.knowledge.mapping (v02)
- build_plan_pinn_first f — pinneapple_problemdesign.knowledge.mapping (v02)
- build_plugin f — pinneapple_simulation.accelerated_foam.build (v03)
- build_primitive f — pinneapple_design.geometry.gen.primitives (v06)
- build_problem f — pinneapple_mcp.problems (v07)
- build_prompt f — pinneapple_flow.planner.llm (v07)
- build_prompt f — pinneapple_llm.qa.generation (v07)
- build_pt_chain_graph f — pinneapple_analysis.proof.pt_chains (v05)
- build_recipe f — pinneapple_llm.cad_draft (v07)
- build_report f — pinneapple_app.backend.core.surrogate_report (v07)
- build_report f — pinneapple_design.fitmesh.report (v06)
- build_report f — pinneapple_veriphysics.evidence_package (v05)
- build_scene f — pinneapple_blender.render (v07)
- build_scene f — pinneapple_simulation.external_solvers.genesis.scene_builder (v03)
- build_server f — pinneapple_mcp (v07)
- build_server f — pinneapple_mcp.server (v07)
- build_site f — pinneapple_flow.nodes.vehicle (v07)
- build_slurm_script f — pinneapple_neural.trainer.hpc (v04)
- build_soil f — pinneapple_flow.nodes.vehicle (v07)
- build_space f — pinneapple_hub.zoo.publish (v07)
- build_spec f — pinneapple_flow.nodes.vehicle (v07)
- build_spec f — pinneapple_llm.qa.examples.pipe_wear_twin (v07)
- build_spheroid f — pinneapple_simulation.pipelines.freecad_openfoam.freecad_scripts.fc_job (v03)
- build_stage_gaps f — pinneapple_problemdesign.elicitation.questions (v02)
- build_surrogate f — pinneapple_presets.surrogate (v02)
- build_tables f — pinneapple_analysis.benchmark.report (v05)
- build_tag_batch f — pinneapple_physics.pde_environment.presets.tag_geometry (v02)
- build_terrain f — pinneapple_simulation.rover.autolab.render (v03)
- build_thread_polygon f — pinneapple_design.geometry.gen.threaded_connection (v06)
- build_torchrun_cmd f — pinneapple_neural.trainer.hpc (v04)
- build_twin f — pinneapple_llm.qa.examples.pipe_wear_twin (v07)
- build_upd_dataloader f — pinneapple_data.dataloaders (v04)
- build_variant f — pinneapple_design.fitmesh.cad (v06)

- build_vehicle f — pinneapple_simulation.vehicle.build (v03)
- build_wheel_model f — pinneapple_flow.nodes.vehicle (v07)
- build_xtfc f — pinneapple_neural.architectures.pinns.xtfc (v04)
- build_xy f — pinneapple_physics.astro.spaceweather (v02)
- building_steps f — pinneapple_design.bricks.visuals (v06)
- builtin f — pinneapple_analysis.study_models (v05)
- builtin f — pinneapple_design.intent.templates (v06)
- builtin_cases f — pinneapple_analysis.validation.arena.cases (v05)
- BuiltProblem C — pinneapple_mcp.problems (v07)
- bulldozing_resistance f — pinneapple_simulation.terramechanics.wong_reece (v03)
- bundled_flows_using f — pinneapple_hub.library.flowlink (v07)
- BundleData C — pinneapple_tools.benchmark_suite.bundle.loader (v07)
- BundleSchema C — pinneapple_tools.benchmark_suite.bundle.schema (v07)
- Burgers C — pinneapple_analysis.benchmark.builtin.standard (v05)
- Burgers1D C — pinneapple_arena.problems (v04)
- Burgers1DTask C — pinneapple_tools.benchmark_suite.tasks.burgers_1d (v07)
- burgers_1d f — pinneapple_analysis.validation.pinn (v05)
- burgers_1d f — pinneapple_simulation.numerical_solvers.operator_benchmarks (v03)
- burgers_1d_default f — pinneapple_physics.pde_environment.presets.academics (v02)
- burgers_fdm_baseline f — pinneapple_analysis.surrogate.examples (v05)
- burgers_fields f — pinneapple_analysis.surrogate.examples (v05)
- burgers_first f — pinneapple_flow.nodes_ext.pinn_examples (v07)
- burgers_inputs f — pinneapple_simulation.numerical_solvers.operator_datasets (v03)
- burgers_periodic_pairs f — pinneapple_data.datasets.physics (v04)
- burgers_periodic_solve f — pinneapple_data.datasets.physics (v04)
- burgers_sine_exact f — pinneapple_physics.closed_form.burgers (v02)
- burgers_sine_inviscid_exact f — pinneapple_physics.closed_form.burgers (v02)
- BurgersProblem C — pinneapple_analysis.validation.fault_injection.problems (v05)
- buy_vs_build_recommendation f — pinneapple_analysis.verification.tool_recommendation (v05)
- by_key f — pinneapple_physics.weather.events (v02)
- by_node f — pinneapple_flow.decisions (v07)
- by_target_month f — pinneapple_systems.time_series.enso.evaluation (v06)
- ByteCacheStats C — pinneapple_data.cache_bytes (v04)
- ByteLRUCache C — pinneapple_data.cache_bytes (v04)

C

- cache_dir f — pinneapple_design.geometry_library.store (v06)
- cache_dir f — pinneapple_hub.zoo._artifacts (v07)
- cache_dir f — pinneapple_systems.sports.football.data (v06)
- cache_path f — pinneapple_data.datasets.download (v04)
- Cached C — pinneapple_condense.query (v02)
- CachedUPDZarrStore C — pinneapple_data.zarr_cached_store (v04)
- CachedUPDZarrStoreBytes C — pinneapple_data.zarr_cached_store_bytes (v04)
- CacheStats C — pinneapple_data.cache (v04)
- cad_reference f — pinneapple_physics.astro.smallbodies (v02)
- cad_step f — pinneapple_simulation.pipelines.freecad_openfoam.steps (v03)
- CadGeometry C — pinneapple_simulation.pipelines.freecad_openfoam.cad (v03)
- CADPart C — pinneapple_design.geometry.vehicle_cad (v06)
- cadquery_available f — pinneapple_design.geometry.gen.cadquery_gen (v06)
- cadquery_solid_to_upd f — pinneapple_design.geometry.io.cadquery_bridge (v06)
- cadquery_to_trimesh f — pinneapple_design.geometry.gen.cadquery_gen (v06)
- CadQueryBuildOptions C — pinneapple_design.geometry.gen.cadquery_gen (v06)
- CadQueryRegistry C — pinneapple_design.geometry.gen.cadquery_gen (v06)
- CadQueryTessellationOptions C — pinneapple_design.geometry.gen.cadquery_gen (v06)
- CadRecipeResult C — pinneapple_llm.cad_draft (v07)
- CADToCFDPipeline C — pinneapple_simulation.numerical_solvers.cfd_pipeline (v03)

- CadVariant C — pinneapple_design.fitmesh.cad (v06)
- cae_geometry f — pinneapple_check._reuse (v05)
- cae_io f — pinneapple_check._reuse (v05)
- calculix_cantilever f — pinneapple_flow.wrappers (v07)
- calculix_grid f — pinneapple_tools.visualization.pyvista_bridge (v07)
- calculix_model f — pinneapple_design.intent.physics.bracket (v06)
- CalculiXAdapter C — pinneapple_core.workflows.adapters (v01)
- CalculixCase C — pinneapple_lab.experiments.calculix_cases (v07)
- CalculixModel C — pinneapple_simulation.external_solvers.calculix.inp (v03)
- calendar_grid f — pinneapple_systems.time_series.forecast_lab.plots (v06)
- calibrate f — pinneapple_analysis.benchmark.builtin.inverse_heat_impl (v05)
- calibrate f — pinneapple_analysis.inverse_problems.calibration (v05)
- calibrate f — pinneapple_analysis.validation.arena.blue (v05)
- calibrate f — pinneapple_flow.nodes.solvers (v07)
- calibrate f — pinneapple_physics.astro.conditions (v02)
- calibrate f — pinneapple_physics.pcb.engine (v02)
- calibrate f — pinneapple_simulation.terramechanics.testbed (v03)
- calibrate f — pinneapple_systems.process_components.explicit_equation_system (v06)
- calibrate f — pinneapple_twin3d.wear.calibrate (v06)
- calibrate_holdout f — pinneapple_simulation.terramechanics.testbed (v03)
- calibrate_scores f — pinneapple_analysis.validation.arena.blue (v05)
- calibrate_tolerances f — pinneapple_physics.darcy_checks (v02)
- Calibration C — pinneapple_twin3d.wear.calibrate (v06)
- calibration_against_reference f — pinneapple_veriphysics.robustness (v05)
- calibration_curve f — pinneapple_tools.model_evaluation.metrics (v07)
- calibration_study f — pinneapple_analysis.inverse_problems.contracts (v05)
- CalibrationMetrics C — pinneapple_analysis.uncertainty.calibration (v05)
- CalibrationResult C — pinneapple_analysis.inverse_problems.calibration (v05)
- CalibrationResult C — pinneapple_systems.process_components.explicit_equation_system (v06)
- CalibrationSummary C — pinneapple_analysis.verification.physics_confidence_score (v05)
- call_infer f — pinneapple_arena.external.runners.registry (v04)
- call_llm f — pinneapple_llm._dispatch (v07)
- call_ollama f — pinneapple_llm.local_llm (v07)
- call_quiet f — pinneapple_presets.core (v02)
- callable_exact f — pinneapple_tools.benchmark_suite.crossframework.problems (v07)
- CallableExecutor C — pinneapple_decision.loop (v05)
- CallableModel C — pinneapple_analysis.inverse_problems.forward (v05)
- CallablePostModel C — pinneapple_neural.workflows.hybrid_surrogate_physics (v04)
- CallLLM C — pinneapple_design.intent.parse (v06)
- CallLog C — pinneapple_design.intent.pipeline (v06)
- Camera C — pinneapple_physics.astro.observing (v02)
- Camera C — pinneapple_physics.blackhole.raytrace (v02)
- camera_frames f — pinneapple_worldmodel.robot_perception (v04)
- camera_head f — pinneapple_design.geometry.vehicle_cad (v06)
- camera_transform f — pinneapple_worldmodel.inspection_data (v04)
- CameraConfig C — pinneapple_worldmodel.camera_system (v04)
- CameraSystem C — pinneapple_worldmodel.camera_system (v04)
- CameraVariation C — pinneapple_worldmodel.robot_perception (v04)
- Campaign C — pinneapple_simulation.external_solvers.openfoam.campaign (v03)
- campaign f — pinneapple_flow.nodes_ext.cfd (v07)
- campaign f — pinneapple_simulation.electromagnetics.periodic.benchmark (v03)
- CampaignPlan C — pinneapple_data.doe.campaign (v04)
- CampaignRun C — pinneapple_simulation.external_solvers.openfoam.campaign (v03)
- can_render f — pinneapple_tools.visualization.pyvista_bridge (v07)
- cancel f — pinneapple_mcp.jobs (v07)
- cancel_job f — pinneapple_mcp.tools (v07)

- `Cancelled` C — `pinneapple_llm.workbench.runtime` (v07)
- `Candidate` C — `pinneapple_analysis.surrogate.methods` (v05)
- `Candidate` C — `pinneapple_data.formfill.extract` (v04)
- `Candidate` C — `pinneapple_design.bricks.search` (v06)
- `Candidate` C — `pinneapple_design.intent.pipeline` (v06)
- `candidate_ids` f — `pinneapple_analysis.surrogate.methods` (v05)
- `CandidateAssessment` C — `pinneapple_autolab.trust` (v07)
- `CandidateCause` C — `pinneapple_analysis.inverse_problems.counterfactual_attribution` (v05)
- `CandidateDetail` C — `pinneapple_veriphysics.recommend` (v05)
- `CandidateLibrary` C — `pinneapple_analysis.inverse_problems.missing_term` (v05)
- `CandidateResult` C — `pinneapple_analysis.verification.pipeline` (v05)
- `candidates` f — `pinneapple_design.intent.materials` (v06)
- `canonical_json` f — `pinneapple_security.integrity` (v05)
- `canonical_json` f — `pinneapple_veriphysics.ledger` (v05)
- `canonical_name` f — `pinneapple_data.unified` (v04)
- `Cantilever` C — `pinneapple_design.qualitative.models` (v06)
- `cantilever` f — `pinneapple_lab.experiments.rom` (v07)
- `cantilever_3d` f — `pinneapple_analysis.surrogate.examples` (v05)
- `cantilever_bending_stress_at_position_pa` f — `pinneapple_physics.closed_form.cantilever_fatigue` (v02)
- `cantilever_compare` f — `pinneapple_core.workflows.templates` (v01)
- `cantilever_deck` f — `pinneapple_core.workflows.templates` (v01)
- `cantilever_frequency_roots` f — `pinneapple_simulation.numerical_solvers.nonlinear_beam_fem` (v03)
- `cantilever_hex_mesh` f — `pinneapple_simulation.external_solvers.calculix.inp` (v03)
- `cantilever_root_bending_stress_pa` f — `pinneapple_physics.closed_form.cantilever_fatigue` (v02)
- `cantilever_space` f — `pinneapple_simulation.external_solvers.campaign_cases` (v03)
- `CantileverGoal` C — `pinneapple_design.bricks.goals` (v06)
- `capabilities` f — `pinneapple_flow.planner.extensions` (v07)
- `capabilities` f — `pinneapple_mcp.tools` (v07)
- `Capability` C — `pinneapple_flow.planner.extensions` (v07)
- `capacitance` f — `pinneapple_simulation.electromagnetics.electrostatics` (v03)
- `capacity` f — `pinneapple_design.bricks.statics` (v06)
- `capsule2d` f — `pinneapple_design.geometry.gen.sdf_shapes` (v06)
- `Capture` C — `pinneapple_lab.capture` (v07)
- `capture` f — `pinneapple_autolab.dashboard` (v07)
- `capture_twin` f — `pinneapple_physics.blackhole.capture` (v02)
- `car_brake_thermal` f — `pinneapple_physics.pde_environment.presets.engineering` (v02)
- `car_external_aero` f — `pinneapple_physics.pde_environment.presets.engineering` (v02)
- `car_mesh` f — `pinneapple_design.intent.physics.loop_track` (v06)
- `car_suspension_fatigue` f — `pinneapple_physics.pde_environment.presets.engineering` (v02)
- `Card` C — `pinneapple_core.onboarding.catalog` (v01)
- `card` f — `pinneapple_core.onboarding.catalog` (v01)
- `card` f — `pinneapple_physics.astro.facts` (v02)
- `card_from_guardrail` f — `pinneapple_hub.verified_card` (v07)
- `card_main` f — `pinneapple_core.onboarding.cli` (v01)
- `card_section` f — `pinneapple_hub.zoo.validation_reports` (v07)
- `CardRefused` C — `pinneapple_hub.verified_card` (v07)
- `CarLBM` C — `pinneapple_lab.experiments.car` (v07)
- `CarProfile` C — `pinneapple_design.geometry.gen.car2d` (v06)
- `CarSurrogate` C — `pinneapple_lab.experiments.car` (v07)
- `Case` C — `pinneapple_analysis.validation.fault_injection.problems` (v05)
- `Case` C — `pinneapple_systems.time_series.forecast_lab.presets` (v06)
- `case` f — `pinneapple_simulation.electromagnetics.machines.verification` (v03)
- `case` f — `pinneapple_simulation.electromagnetics.verification` (v03)
- `case` f — `pinneapple_systems.time_series.forecast_lab.presets` (v06)
- `case_id` f — `pinneapple_simulation.pipelines.freecad_openfoam.sweep` (v03)

- `case_panels` f — `pinneapple_systems.time_series.enso.pipeline` (v06)
- `case_zip` f — `pinneapple_design.aero.case` (v06)
- `CaseFields` C — `pinneapple_simulation.numerical_solvers.external_flow` (v03)
- `CaseResult` C — `pinneapple_simulation.electromagnetics.verification` (v03)
- `CaseResult` C — `pinneapple_simulation.pipelines.freecad_openfoam.post` (v03)
- `cast_shadow` f — `pinneapple_tools.visualization.studio.shadows` (v07)
- `catalog` f — `pinneapple_flow` (v07)
- `catalog` f — `pinneapple_tools.automation` (v07)
- `catalog_family` f — `pinneapple_arena.decision_mode` (v04)
- `catalog_prompt` f — `pinneapple_flow.planner.llm` (v07)
- `catalog_sky` f — `pinneapple_physics.astro.lensing` (v02)
- `catalog_status` f — `pinneapple_arena.pointwise.architectures` (v04)
- `catalogue` f — `pinneapple_core.onboarding.catalog` (v01)
- `catalogue` f — `pinneapple_hub.zoo` (v07)
- `CatBoostForecaster` C — `pinneapple_systems.time_series.models.classical` (v06)
- `categorical` f — `pinneapple_systems.time_series.enso.evaluation` (v06)
- `categories` f — `pinneapple_design.geometry_library` (v06)
- `category` f — `pinneapple_systems.time_series.enso.evaluation` (v06)
- `category_of` f — `pinneapple_hub.library.categories` (v07)
- `category_of` f — `pinneapple_veriphysics.validation_lab` (v05)
- `CauchyMisfit` C — `pinneapple_analysis.inverse_problems.noise_models` (v05)
- `causal_weights` f — `pinneapple_core.loss` (v01)
- `CausalPINNTrainer` C — `pinneapple_neural.trainer.causal` (v04)
- `CausalWeightScheduler` C — `pinneapple_neural.trainer.causal` (v04)
- `Cause` C — `pinneapple_physics.physics_audit.attribution` (v02)
- `CavityResonance` C — `pinneapple_science.em.waves` (v02)
- `cb_block` f — `pinneapple_condense.transient` (v02)
- `CBBlock` C — `pinneapple_condense.transient` (v02)
- `CBTransient` C — `pinneapple_condense.transient` (v02)
- `ccx_backend` f — `pinneapple_simulation.external_solvers.calculix.runner` (v03)
- `CcxRun` C — `pinneapple_simulation.external_solvers.calculix.runner` (v03)
- `cd_crisis` f — `pinneapple_systems.sports.football.ball` (v06)
- `ceemdan_1d` f — `pinneapple_simulation.numerical_solvers.ceemdan` (v03)
- `CEEMDANSolver` C — `pinneapple_simulation.numerical_solvers.ceemdan` (v03)
- `cell` f — `pinneapple_analysis.benchmark.report` (v05)
- `cell_adjacency` f — `pinneapple_data.cae.geometry` (v04)
- `cell_centers` f — `pinneapple_twin3d.wear.geometry` (v06)
- `cell_centres` f — `pinneapple_design.aero.mesh` (v06)
- `cell_centres` f — `pinneapple_simulation.numerical_solvers.free_surface` (v03)
- `cell_centres_volumes` f — `pinneapple_data.cae.compare` (v04)
- `cell_geometry` f — `pinneapple_core.fem` (v01)
- `cell_geometry` f — `pinneapple_data.cae.geometry` (v04)
- `cell_mesh` f — `pinneapple_twin3d.wear.geometry` (v06)
- `cell_simulator` f — `pinneapple_simulation.electromagnetics.periodic.benchmark` (v03)
- `cell_to_vertex` f — `pinneapple_twin3d.wear.geometry` (v06)
- `cell_volumes` f — `pinneapple_physics.blackhole.convergence` (v02)
- `CellBlock` C — `pinneapple_data.cae.model` (v04)
- `CellFit` C — `pinneapple_twin3d.wear.forecast` (v06)
- `cem` f — `pinneapple_simulation.robot_sim.learn` (v03)
- `CEMResult` C — `pinneapple_simulation.robot_sim.learn` (v03)
- `central_derivative` f — `pinneapple_science.core` (v02)
- `central_difference` f — `pinneapple_systems.process_components.real_gas_eos` (v06)
- `centre_of_pressure` f — `pinneapple_design.bricks.statics` (v06)
- `certify_model` f — `pinneapple_hub.verified_card` (v07)
- `cfd_eval` f — `pinneapple_design.aero.verify` (v06)
- `CFDMesh` C — `pinneapple_simulation.numerical_solvers.cfd_pipeline` (v03)

- `CfdOFAdapter` C — `pinneapple_tools.automation.adapters.freecad` (v07)
- `cfg_str` f — `pinneapple_tools.benchmark_suite.meta_benchmark` (v07)
- `cgns_to_upd` f — `pinneapple_simulation.external_solvers.cfd_formats.cgns_reader` (v03)
- `Chain` C — `pinneapple_reason.chain` (v05)
- `chain` f — `pinneapple_flow.nodes.reason` (v07)
- `chain_checks` f — `pinneapple_reason.trust` (v05)
- `chain_hamiltonian` f — `pinneapple_analysis.proof.pt_chains` (v05)
- `chamfer_edges` f — `pinneapple_design.geometry.ops.occ_features` (v06)
- `chamfered_box` f — `pinneapple_design.geometry.vehicle_cad` (v06)
- `chamfered_rect` f — `pinneapple_design.geometry.vehicle_cad` (v06)
- `changepoint_plot` f — `pinneapple_systems.time_series.eda.plots` (v06)
- `changepoints_ruptures` f — `pinneapple_systems.time_series.audit.tests` (v06)
- `Channel` C — `pinneapple_systems.operations.anomaly` (v06)
- `channel_flow_3d` f — `pinneapple_physics.pde_environment.presets.cfd` (v02)
- `channel_names` f — `pinneapple_physics.weather.data` (v02)
- `channel_with_hole` f — `pinneapple_design.geometry.csg` (v06)
- `ChannelConfig` C — `pinneapple_simulation.numerical_solvers.thermal_channel_2d` (v03)
- `ChannelDomain2D` C — `pinneapple_design.geometry.gen.domains` (v06)
- `ChannelDomain3D` C — `pinneapple_design.geometry.gen.domains3d` (v06)
- `ChannelFlowConfig3D` C — `pinneapple_simulation.numerical_solvers.fdm3d` (v03)
- `ChannelFlowSolver3D` C — `pinneapple_simulation.numerical_solvers.fdm3d` (v03)
- `ChannelWithObstacleDomain2D` C — `pinneapple_design.geometry.gen.domains` (v06)
- `characteristic_length` f — `pinneapple_app.backend.core.physics_adapter` (v07)
- `characteristic_velocity` f — `pinneapple_physics.closed_form.rocket_nozzle` (v02)
- `Chart` C — `pinneapple_tools.visualization.dashboard.spec` (v07)
- `chart_points` f — `pinneapple_design.design_optimizer.report` (v06)
- `chassis_rotation` f — `pinneapple_simulation.vehicle.viz` (v03)
- `chebyshev_gauss_lobatto` f — `pinneapple_simulation.numerical_solvers.xtfc_ivp` (v03)
- `checagem_booleana` f — `pinneapple_analysis.trust.trust_report` (v05)
- `checagem_erro_relativo` f — `pinneapple_analysis.trust.trust_report` (v05)
- `Check` C — `pinneapple_analysis.benchmark.registry` (v05)
- `Check` C — `pinneapple_analysis.proof.graph` (v05)
- `Check` C — `pinneapple_analysis.trust.trust_report` (v05)
- `Check` C — `pinneapple_core.onboarding.doctor` (v01)
- `Check` C — `pinneapple_tools.automation.record` (v07)
- `check` f — `pinneapple_data.expressions` (v04)
- `check` f — `pinneapple_data.preflight.calculix` (v04)
- `check` f — `pinneapple_data.preflight.openfoam` (v04)
- `check` f — `pinneapple_flow.nodes_ext.render_worker` (v07)
- `check` f — `pinneapple_flow.references` (v07)
- `check` f — `pinneapple_hub.library` (v07)
- `check` f — `pinneapple_presets.core` (v02)
- `check` f — `pinneapple_systems.operations.contracts` (v06)
- `check_answer` f — `pinneapple_data.formfill.extract` (v04)
- `check_api_contracts` f — `pinneapple_llm.qa.static` (v07)
- `check_applicable` f — `pinneapple_arena.pointwise.adapters` (v04)
- `check_balance` f — `pinneapple_check.checks.results` (v05)
- `check_block_step` f — `pinneapple_physics.pde_environment.presets.astroml_bridge` (v02)
- `check_citations` f — `pinneapple_lab.results_rules` (v07)
- `check_claims` f — `pinneapple_llm.qa.generation` (v07)
- `check_code` f — `pinneapple_design.fitmesh.llm` (v06)
- `check_config_entry` f — `pinneapple_physics.darcy_checks` (v02)
- `check_config_entry` f — `pinneapple_physics.maxwell_checks` (v02)
- `check_cors_origins` f — `pinneapple_security.web` (v05)
- `check_dataset` f — `pinneapple_check.checks.datasets` (v05)
- `check_directivity_limit` f — `pinneapple_simulation.electromagnetics.antenna.checks` (v03)

- check_eclipses f — pinneapple_physics.astro.validation (v02)
- check_efficiency_le_one f — pinneapple_simulation.electromagnetics.antenna.checks (v03)
- check_efficiency_ratio f — pinneapple_simulation.electromagnetics.antenna.checks (v03)
- check_envelope f — pinneapple_simulation.external_solvers.calculix.virtual_ccx (v03)
- check_equations f — pinneapple_simulation.electromagnetics.units (v03)
- check_finite f — pinneapple_simulation.numerical_solvers.etdrk4 (v03)
- check_flow f — pinneapple_flow.schema (v07)
- check_fmu f — pinneapple_simulation.external_solvers.modelica.fmi_compat (v03)
- check_fmus f — pinneapple_simulation.external_solvers.modelica.fmi_compat (v03)
- check_frame_origins f — pinneapple_security.web (v05)
- check_geometry f — pinneapple_llm.qa.validators.geometry (v07)
- check_guardrail f — pinneapple_llm.qa.validators.guardrail (v07)
- check_horizons f — pinneapple_physics.astro.validation (v02)
- check_imports f — pinneapple_llm.qa.static (v07)
- check_integrators f — pinneapple_physics.astro.validation (v02)
- check_integrity f — pinneapple_check.checks.integrity (v05)
- check_iss_passes f — pinneapple_physics.astro.validation (v02)
- check_json_artifact f — pinneapple_llm.qa.static (v07)
- check_kwargs_in_bounds f — pinneapple_llm.draft (v07)
- check_library f — pinneapple_analysis.validation.fault_injection.checks (v05)
- check_line f — pinneapple_lab.curation (v07)
- check_line_fits f — pinneapple_physics.pde_environment.presets.astroml_bridge (v02)
- check_main f — pinneapple_check.cli (v05)
- check_member_name f — pinneapple_security.archives (v05)
- check_model f — pinneapple_check.checks.models (v05)
- check_nodal_darcy_batch f — pinneapple_physics.darcy_checks (v02)
- check_novas f — pinneapple_physics.astro.validation (v02)
- check_numeric f — pinneapple_llm.qa.validators.numeric (v07)
- check_outbound_url f — pinneapple_security.web (v05)
- check_overrides f — pinneapple_data.formfill.pipeline (v04)
- check_peak_ge_boresight f — pinneapple_simulation.electromagnetics.antenna.checks (v03)
- check_period_recovery f — pinneapple_physics.pde_environment.presets.astroml_bridge (v02)
- check_physics f — pinneapple_llm.qa.validators.physics (v07)
- check_pin f — pinneapple_check.checks.integrity (v05)
- check_power_closure f — pinneapple_simulation.electromagnetics.antenna.checks (v03)
- check_realized_gain f — pinneapple_simulation.electromagnetics.antenna.checks (v03)
- check_relations f — pinneapple_analysis.validation.metamorphic (v05)
- check_release f — pinneapple_hub.zoo.validation_reports (v07)
- check_repo f — pinneapple_lab.manifest (v07)
- check_report f — pinneapple_analysis.validation.report_artifact (v05)
- check_result f — pinneapple_check.checks.results (v05)
- check_rows f — pinneapple_tools.nvidia.figures (v07)
- check_s11_resolved f — pinneapple_simulation.electromagnetics.antenna.checks (v03)
- check_security f — pinneapple_llm.qa.static (v07)
- check_sensors f — pinneapple_llm.qa.validators.sensors (v07)
- check_signatures f — pinneapple_llm.qa.static (v07)
- check_skyfield_live f — pinneapple_physics.astro.validation (v02)
- check_spec f — pinneapple_hub.zoo (v07)
- check_spec f — pinneapple_hub.zoo.checks (v07)
- check_summary f — pinneapple_simulation.electromagnetics.antenna.study (v03)
- check_syntax f — pinneapple_llm.qa.static (v07)
- check_temporal f — pinneapple_llm.qa.validators.temporal (v07)
- check_units f — pinneapple_check.checks.common (v05)
- check_units f — pinneapple_llm.qa.validators.units (v07)
- check_units f — pinneapple_simulation.electromagnetics.units (v03)
- check_usno_almanac f — pinneapple_physics.astro.validation (v02)

- check_values f — pinneapple_check.checks.common (v05)
- check_values f — pinneapple_hub.zoo.checks (v07)
- check_xd_means f — pinneapple_physics.pde_environment.presets.astroml_bridge (v02)
- check_zip f — pinneapple_security.archives (v05)
- check_zoo_relations f — pinneapple_hub.zoo.relations (v07)
- CheckItem C — pinneapple_systems.digital_twin.product.evidence (v06)
- CheckList C — pinneapple_lab.results_rules (v07)
- ChecklistItem C — pinneapple_veriphysics.applicability (v05)
- CheckOutcome C — pinneapple_analysis.proof.graph (v05)
- Checkpoint C — pinneapple_neural.trainer.checkpoint (v04)
- CheckRecord C — pinneapple_data.contracts.models (v04)
- CheckReport C — pinneapple_systems.digital_twin.product.evidence (v06)
- CheckResult C — pinneapple_analysis.validation.core (v05)
- CheckResult C — pinneapple_design.surrogate_inverse.core (v06)
- CheckResult C — pinneapple_llm.guardrail (v07)
- CheckResult C — pinneapple_physics.physics_audit.monitor (v02)
- checks_figure f — pinneapple_tools.nvidia.figures (v07)
- checks_module f — pinneapple_hub.zoo.checks (v07)
- checks_parity f — pinneapple_hub.zoo.export (v07)
- CheckSpec C — pinneapple_analysis.validation.schema (v05)
- ChecksumMismatchError C — pinneapple_hub.zoo._artifacts (v07)
- chi f — pinneapple_reason.modes.structural (v05)
- choice_for_objective f — pinneapple_decision.loop (v05)
- ChoiceNode C — pinneapple_decision.tree (v05)
- choked_mass_flow f — pinneapple_physics.closed_form.rocket_nozzle (v02)
- choose_next f — pinneapple_autolab.trust (v07)
- choose_tiles f — pinneapple_lab.showcase (v07)
- chow_test_mean_shift f — pinneapple_systems.time_series.audit.tests (v06)
- ChronosForecaster C — pinneapple_systems.time_series.foundation_models (v06)
- chunk_text f — pinneapple_analysis.retrieval.corpus (v05)
- chunk_text f — pinneapple_tools.hpo_experiments.kb.chunking (v07)
- ChunkSpec C — pinneapple_data.chunking (v04)
- Circle C — pinneapple_worldmodel.geometry (v04)
- circle f — pinneapple_design.geometry.gen.sdf_shapes (v06)
- circle_segments f — pinneapple_simulation.electromagnetics.statics (v03)
- Circuits C — pinneapple_science.em.induction (v02)
- circular_arc_cascade_blade_points f — pinneapple_physics.pde_environment.presets._curve_geometry (v02)
- circular_arc_tvd_hd f — pinneapple_systems.process_components.curved_path_geometry (v06)
- circular_waveguide_cutoff f — pinneapple_simulation.electromagnetics.waves (v03)
- circular_waveguide_modes f — pinneapple_simulation.electromagnetics.waves (v03)
- cite f — pinneapple_flow.references (v07)
- clamp_confidence f — pinneapple_tools.dataset_quality.vlm_curation (v07)
- ClappHornberger C — pinneapple_simulation.geophysics.richards (v03)
- classical f — pinneapple_flow.nodes.solvers (v07)
- classical_eddy_coefficient f — pinneapple_simulation.electromagnetics.postprocess (v03)
- classical_job f — pinneapple_analysis.benchmark.builtin.inverse_heat_impl (v05)
- ClassicalBackend C — pinneapple_quantum.backends.backend (v04)
- ClassicalQNode C — pinneapple_quantum.backends.backend (v04)
- ClassicalTSBase C — pinneapple_neural.architectures.classical_ts.base (v04)
- ClassicalTSCatalog C — pinneapple_neural.architectures.classical_ts.registry (v04)
- ClassicalTSOutput C — pinneapple_neural.architectures.classical_ts.base (v04)
- ClassicalTuner C — pinneapple_systems.time_series.tuning.optuna_tuner (v06)
- ClassicalVQC C — pinneapple_quantum.circuits.base (v04)
- classify f — pinneapple_physics.symbolic_pde.text (v02)
- classify_ambiguity f — pinneapple_analysis.inverse_problems.counterfactual_attribution (v05)

- `classify_buckling_mode` f — `pinneapple_systems.process_components.pipe_stress_mechanics` (v06)
- `classify_coupling` f — `pinneapple_physics.non_hermitian.ep_tuning` (v02)
- `classify_flow_regime` f — `pinneapple_analysis.verification.dimensional_analysis` (v05)
- `classify_geometry` f — `pinneapple_analysis.verification.geometry_intelligence` (v05)
- `classify_modes` f — `pinneapple_simulation.flight.trim` (v03)
- `classify_physics_shape` f — `pinneapple_physics.codegen.fenics_script_generator` (v02)
- `clean_for_modeling` f — `pinneapple_data.process_optimizer` (v04)
- `clean_mesh` f — `pinneapple_design.geometry.ops.clean` (v06)
- `clean_scan` f — `pinneapple_twin3d.scan` (v06)
- `CleanMeshConfig` C — `pinneapple_design.geometry.ops.clean` (v06)
- `cleanup` f — `pinneapple_worldmodel.inspection_data` (v04)
- `clear_cache` f — `pinneapple_condense.substructure` (v02)
- `client_ip` f — `pinneapple_security.web` (v05)
- `clift_sphere_cd` f — `pinneapple_physics.closed_form.particle_drag` (v02)
- `climate_atmosphere_2d` f — `pinneapple_physics.pde_environment.presets.multidisciplinary` (v02)
- `climate_ocean_gyre` f — `pinneapple_physics.pde_environment.presets.multidisciplinary` (v02)
- `Climatology` C — `pinneapple_systems.time_series.forecast_lab.baselines` (v06)
- `climatology` f — `pinneapple_physics.weather.data` (v02)
- `climatology_at` f — `pinneapple_physics.weather.data` (v02)
- `climatology_forecast` f — `pinneapple_physics.weather.evaluate` (v02)
- `clip_grid` f — `pinneapple_flow.nodes_ext.render` (v07)
- `CLISimulator` C — `pinneapple_simulation.electromagnetics.antenna.runner` (v03)
- `Clock` C — `pinneapple_tools.benchmark_suite.crossframework.records` (v07)
- `close` f — `pinneapple_science.plots` (v02)
- `close_approaches` f — `pinneapple_physics.astro.smallbodies` (v02)
- `close_sheets` f — `pinneapple_design.geometry_library.entries._adapt` (v06)
- `closed_form` f — `pinneapple_design.intent.physics.loop_track` (v06)
- `closed_multiscale_residual` f — `pinneapple_physics.multiscale.comparison` (v02)
- `closest_to_schwabe_cycle` f — `pinneapple_systems.time_series.solar_cycle` (v06)
- `ClosestApproach` C — `pinneapple_physics.astro.conjunction` (v02)
- `closure_flag_table` f — `pinneapple_physics.multiscale.comparison` (v02)
- `closure_health` f — `pinneapple_physics.multiscale.closures` (v02)
- `ClosureHealth` C — `pinneapple_physics.multiscale.closures` (v02)
- `Cloud` C — `pinneapple_condense.meshless` (v02)
- `cloud` f — `pinneapple_condense.meshless2d` (v02)
- `cloud_at` f — `pinneapple_physics.astro.conditions` (v02)
- `clusters` f — `pinneapple_design.bricks.dynamics` (v06)
- `cmaes` f — `pinneapple_analysis.validation.arena.search` (v05)
- `cmap_color` f — `pinneapple_tools.visualization.studio.compose` (v07)
- `cmd_autocalls` f — `pinneapple_flow.__main__` (v07)
- `cmd_catalog` f — `pinneapple_flow.__main__` (v07)
- `cmd_coverage` f — `pinneapple_flow.__main__` (v07)
- `cmd_doctor` f — `pinneapple_llm.workbench.__main__` (v07)
- `cmd_eval` f — `pinneapple_llm.workbench.__main__` (v07)
- `cmd_examples` f — `pinneapple_flow.__main__` (v07)
- `cmd_finetune` f — `pinneapple_llm.workbench.__main__` (v07)
- `cmd_list` f — `pinneapple_arena.external.__main__` (v04)
- `cmd_plan` f — `pinneapple_flow.__main__` (v07)
- `cmd_pull` f — `pinneapple_llm.workbench.__main__` (v07)
- `cmd_references` f — `pinneapple_flow.__main__` (v07)
- `cmd_run` f — `pinneapple_arena.external.__main__` (v04)
- `cmd_run` f — `pinneapple_flow.__main__` (v07)
- `cmd_run` f — `pinneapple_llm.workbench.__main__` (v07)
- `cmd_serve` f — `pinneapple_llm.workbench.__main__` (v07)
- `cmd_studio` f — `pinneapple_flow.__main__` (v07)
- `cmd_validate` f — `pinneapple_flow.__main__` (v07)

- cmd_vscode f — pinneapple_llm.workbench.__main__ (v07)
- coarse_power f — pinneapple_condense.homogenize (v02)
- coarsen f — pinneapple_physics.weather.events (v02)
- coarsen f — pinneapple_systems.time_series.enso.data (v06)
- coastlines f — pinneapple_physics.weather.viz (v02)
- coax_capacitance f — pinneapple_simulation.electromagnetics.statics (v03)
- coax_field f — pinneapple_simulation.electromagnetics.analytic (v03)
- coax_inductance f — pinneapple_simulation.electromagnetics.statics (v03)
- code_blocks f — pinneapple_hub.zoo.hub_cards (v07)
- code_for f — pinneapple_hub.library.compose (v07)
- code_hash f — pinneapple_hub.library.sandbox (v07)
- code_repo_documents f — pinneapple_analysis.retrieval.corpus (v05)
- CodeNotAllowed C — pinneapple_veriphysics.validation_lab (v05)
- coef_from_targets f — pinneapple_design.aero.surrogate (v06)
- coef_scores f — pinneapple_analysis.benchmark.builtin.airfoil_impl (v05)
- coef_targets f — pinneapple_design.aero.surrogate (v06)
- coefficient_stats f — pinneapple_physics.darcy_checks (v02)
- CoefficientEnvelope C — pinneapple_physics.darcy_checks (v02)
- coefs f — pinneapple_analysis.benchmark.builtin.airfoil_impl (v05)
- coefs f — pinneapple_design.aero.train (v06)
- coil_end_path f — pinneapple_simulation.electromagnetics.machines.endwinding (v03)
- coil_impedance f — pinneapple_physics.closed_form.eddy_coil (v02)
- CoilSide C — pinneapple_simulation.electromagnetics.postprocess (v03)
- CoilSpringTemplate C — pinneapple_design.intent.templates.example_spring (v06)
- cole_hopf f — pinneapple_analysis.validation.pinn (v05)
- colebrook f — pinneapple_physics.closed_form.pipe_flow (v02)
- colebrook f — pinneapple_simulation.numerical_solvers.internal_flow (v03)
- colebrook_white_f f — pinneapple_systems.process_components.pipe_network_1d (v06)
- collate_pinn_batches f — pinneapple_data.collate (v04)
- collate_upd_supervised f — pinneapple_data.collate (v04)
- collect f — pinneapple_analysis.benchmark.report (v05)
- collect f — pinneapple_flow.decisions (v07)
- collect f — pinneapple_hub.library (v07)
- collect f — pinneapple_lab.explorer (v07)
- collect f — pinneapple_lab.html (v07)
- collect_outputs f — pinneapple_core.workflows.templates (v01)
- collect_studies f — pinneapple_analysis.benchmark.report (v05)
- collision_probability_2d f — pinneapple_physics.astro.conjunction (v02)
- collision_probability_centered f — pinneapple_physics.astro.conjunction (v02)
- collocation_pool f — pinneapple_analysis.benchmark.builtin.airfoil_impl (v05)
- CollocationBatch3D C — pinneapple_design.geometry.mesh_collocator (v06)
- CollocationConfig C — pinneapple_app.backend.core.collocation (v07)
- CollocationConfig C — pinneapple_data.collocation (v04)
- CollocationRequest C — pinneapple_app.backend.routers.experiments (v07)
- CollocationSampler C — pinneapple_core.data (v01)
- CollocationSampler C — pinneapple_data.collocation (v04)
- color_of f — pinneapple_systems.sports.football.plots (v06)
- ColorBar C — pinneapple_tools.visualization.dashboard.spec (v07)
- colorbar f — pinneapple_tools.visualization.studio.colormap (v07)
- colorbar_image f — pinneapple_tools.visualization.studio.compose (v07)
- Colour C — pinneapple_design.bricks.parts (v06)
- colour f — pinneapple_design.bricks.parts (v06)
- colour_range f — pinneapple_tools.visualization.studio.fieldview (v07)
- colour_rgb f — pinneapple_design.bricks.assembly (v06)
- Column C — pinneapple_analysis.benchmark.registry (v05)
- Column C — pinneapple_data.contracts.models (v04)

- `column_collapse` f — `pinneapple_flow.nodes_ext.geomechanics` (v07)
- `ColumnHints` C — `pinneapple_arena.pointwise.schema` (v04)
- `columns` f — `pinneapple_analysis.study_models` (v05)
- `columns` f — `pinneapple_simulation.electromagnetics.antenna.surrogates` (v03)
- `ColumnSubset` C — `pinneapple_systems.time_series.enso.forecasters` (v06)
- `combine` f — `pinneapple_core.loss` (v01)
- `combine` f — `pinneapple_science.instruments` (v02)
- `combine` f — `pinneapple_security.classification` (v05)
- `combine_mass` f — `pinneapple_design.geometry.vehicle_cad` (v06)
- `CombinedAL` C — `pinneapple_data.active_learning` (v04)
- `CombinedLoss` C — `pinneapple_neural.trainer.losses` (v04)
- `CommandAdapter` C — `pinneapple_core.workflows.adapters` (v01)
- `CommandEvaluator` C — `pinneapple_design.design_optimizer.evaluators` (v06)
- `commercial_mode` f — `pinneapple_neural._licencas` (v04)
- `commission` f — `pinneapple_systems.digital_twin.product.runner` (v06)
- `commutation_error` f — `pinneapple_physics.multiscale.filters` (v02)
- `compact` f — `pinneapple_design.geometry_library.meshops` (v06)
- `compact_range` f — `pinneapple_flow.nodes_ext.em_antenna` (v07)
- `CompactRange` C — `pinneapple_simulation.electromagnetics.reflector.compact_range` (v03)
- `compare` f — `pinneapple_condense.homogenize` (v02)
- `compare` f — `pinneapple_data.cae.compare` (v04)
- `compare` f — `pinneapple_design.bricks.search` (v06)
- `compare` f — `pinneapple_flow.nodes.analysis` (v07)
- `compare` f — `pinneapple_flow.nodes_ext.examples` (v07)
- `compare` f — `pinneapple_hub.zoo.checks` (v07)
- `compare` f — `pinneapple_llm.qa.references` (v07)
- `compare` f — `pinneapple_physics.solving` (v02)
- `compare` f — `pinneapple_systems.sports.football.pc_study` (v06)
- `compare` f — `pinneapple_systems.sports.football.xg` (v06)
- `compare_contact_models` f — `pinneapple_simulation.rover.autolab.wheel_models` (v03)
- `compare_controllers` f — `pinneapple_systems.operations.control` (v06)
- `compare_design` f — `pinneapple_design.aero.verify` (v06)
- `compare_fields` f — `pinneapple_tools.visualization.fields` (v07)
- `compare_methods` f — `pinneapple_analysis.inverse_problems.studio` (v05)
- `compare_methods` f — `pinneapple_design.design_optimizer.compare` (v06)
- `compare_models` f — `pinneapple_autolab.surrogates` (v07)
- `compare_on_states` f — `pinneapple_simulation.terramechanics.surrogate` (v03)
- `compare_preset` f — `pinneapple_flow.wrappers` (v07)
- `compare_replay` f — `pinneapple_tools.nvidia.isaac` (v07)
- `compare_runs` f — `pinneapple_tools.benchmark_suite.runner.compare` (v07)
- `compare_solvers` f — `pinneapple_analysis.verification.solver_orchestration` (v05)
- `compare_to_analytical` f — `pinneapple_analysis.validation.analytical` (v05)
- `compare_variants` f — `pinneapple_presets.report` (v02)
- `compare_with_references` f — `pinneapple_design.fitmesh.cad` (v06)
- `compare_with_sgp4` f — `pinneapple_physics.astro.decay` (v02)
- `Comparison` C — `pinneapple_physics.solving` (v02)
- `Comparison` C — `pinneapple_science.core` (v02)
- `comparison` f — `pinneapple_physics.astro.facts` (v02)
- `ComparisonReport` C — `pinneapple_analysis.verification.solver_orchestration` (v05)
- `compatible` f — `pinneapple_flow.types` (v07)
- `CompatRow` C — `pinneapple_simulation.external_solvers.modelica.fmi_compat` (v03)
- `Compilation` C — `pinneapple_data.formfill.extract` (v04)
- `compile` f — `pinneapple_data.formfill.extract` (v04)
- `compile` f — `pinneapple_physics.compile_api` (v02)
- `compile_documents` f — `pinneapple_data.formfill.pipeline` (v04)
- `compile_expr` f — `pinneapple_physics.symbolic_pde.text` (v02)

- `compile_expression` f — `pinneapple_mcp.expressions` (v07)
- `compile_flags` f — `pinneapple_simulation.accelerated_foam.build` (v03)
- `compile_physics` f — `pinneapple_physics` (v02)
- `compile_problem` f — `pinneapple_physics.pinn_solver.compiler.compile` (v02)
- `compile_requirements` f — `pinneapple_data.udr1` (v04)
- `compile_system` f — `pinneapple_simulation.numerical_solvers.symbolic_fd` (v03)
- `CompiledEquation` C — `pinneapple_physics.pinn_solver.factory.sympy_backend` (v02)
- `CompiledProblem` C — `pinneapple_physics.compile_api` (v02)
- `Compliance2D` C — `pinneapple_design.topology.problems` (v06)
- `Component` C — `pinneapple_physics.pcb.solver` (v02)
- `Component` C — `pinneapple_simulation.flight.mass` (v03)
- `component_fit` f — `pinneapple_flow.nodes_ext.geomechanics` (v07)
- `ComponentBenchmarkResult` C — `pinneapple_systems.component_library.benchmark` (v06)
- `ComponentModel` C — `pinneapple_systems.component_library.base` (v06)
- `ComponentRegistry` C — `pinneapple_systems.component_library.registry` (v06)
- `components` f — `pinneapple_check.numerics` (v05)
- `components` f — `pinneapple_design.geometry_library.meshops` (v06)
- `ComponentSpec` C — `pinneapple_systems.component_library.registry` (v06)
- `Compose` C — `pinneapple_core.transforms` (v01)
- `compose` f — `pinneapple_hub.library.compose` (v07)
- `compose_grid` f — `pinneapple_tools.visualization.studio.compose` (v07)
- `compose_grid_video` f — `pinneapple_tools.visualization.studio.compose` (v07)
- `compose_video` f — `pinneapple_tools.visualization.studio.particles` (v07)
- `ComposedGuidance` C — `pinneapple_neural.architectures.pinns.physics_diffusion` (v04)
- `ComposedObsOperator` C — `pinneapple_analysis.inverse_problems.obs_operator` (v05)
- `ComposeError` C — `pinneapple_hub.library.compose` (v07)
- `CompositeObjective` C — `pinneapple_design.design_optimizer.objective` (v06)
- `CompositeRegularizer` C — `pinneapple_analysis.inverse_problems.regularization` (v05)
- `compressible_mass_flow` f — `pinneapple_systems.process_components.control_valve` (v06)
- `CompressibleConfig` C — `pinneapple_simulation.numerical_solvers.compressible_fv` (v03)
- `CompressibleFV` C — `pinneapple_simulation.numerical_solvers.compressible_fv` (v03)
- `compute_aleatoric_epistemic_uq` f — `pinneapple_neural.trainer.heteroscedastic` (v04)
- `compute_cp` f — `pinneapple_tools.aero_coefficients` (v07)
- `compute_curvature_proxy` f — `pinneapple_design.geometry.ops.features` (v06)
- `compute_derivatives` f — `pinneapple_neural.architectures.neural_operators.pino` (v04)
- `compute_dimensionless_numbers` f — `pinneapple_analysis.verification.dimensional_analysis` (v05)
- `compute_dissipation_rate` f — `pinneapple_tools.visualization.vortex` (v07)
- `compute_enstrophy` f — `pinneapple_tools.visualization.vortex` (v07)
- `compute_face_areas` f — `pinneapple_design.geometry.ops.features` (v06)
- `compute_face_normals` f — `pinneapple_design.geometry.ops.features` (v06)
- `compute_isosurface` f — `pinneapple_neural.predictor.postprocess` (v04)
- `compute_lambda2_3d` f — `pinneapple_tools.visualization.vortex` (v07)
- `compute_ndvi` f — `pinneapple_perception.change_detection` (v04)
- `compute_pareto_front` f — `pinneapple_design.design_optimizer.pareto` (v06)
- `compute_physics_confidence` f — `pinneapple_analysis.verification.physics_confidence_score` (v05)
- `compute_q_criterion_2d` f — `pinneapple_tools.visualization.vortex` (v07)
- `compute_q_criterion_3d` f — `pinneapple_tools.visualization.vortex` (v07)
- `compute_sdf_grid` f — `pinneapple_data.geometry_features` (v04)
- `compute_sensitivity` f — `pinneapple_design.surrogate_inverse.suspension_arm` (v06)
- `compute_strain_rate_tensor_3d` f — `pinneapple_tools.visualization.vortex` (v07)
- `compute_streamlines` f — `pinneapple_neural.predictor.postprocess` (v04)
- `compute_stresses` f — `pinneapple_simulation.numerical_solvers.beam_bvp_fdm` (v03)
- `compute_vertex_normals` f — `pinneapple_design.geometry.ops.features` (v06)
- `compute_vorticity_2d` f — `pinneapple_tools.visualization.vortex` (v07)
- `compute_vorticity_3d` f — `pinneapple_tools.visualization.vortex` (v07)
- `ComputeBudget` C — `pinneapple_analysis.cost.equal_budget` (v05)

- concat f — pinneapple_design.geometry_library.meshops (v06)
- concentration_at_membrane_depth f — pinneapple_design.surrogate_inverse.transdermal_patch (v06)
- concentration_at_membrane_depth f — pinneapple_physics.closed_form.membrane_diffusion (v02)
- concentration_field f — pinneapple_design.surrogate_inverse.transdermal_patch (v06)
- concentric_reducer f — pinneapple_design.geometry.gen.cadquery_advanced (v06)
- Concept C — pinneapple_reason.chain (v05)
- concept f — pinneapple_flow.nodes.reason (v07)
- conclude f — pinneapple_analysis.validation.gallery (v05)
- Condensation C — pinneapple_condense.algebraic (v02)
- condensation_order f — pinneapple_systems.cosimulation.utils (v06)
- Condensed C — pinneapple_condense.query (v02)
- Condensed C — pinneapple_condense.substructure (v02)
- CondensedState C — pinneapple_condense.substructure (v02)
- Condition C — pinneapple_physics.physics_audit.assumptions (v02)
- condition_coverage f — pinneapple_veriphysics.robustness (v05)
- condition_key f — pinneapple_query.core (v02)
- ConditionedSample C — pinneapple_systems.digital_twin.conditioning (v06)
- ConditionInfo C — pinneapple_app.backend.core.physics_adapter (v07)
- ConditionSpec C — pinneapple_data.upd_types (v04)
- ConditionSpec C — pinneapple_physics.pde_environment.conditions (v02)
- conduction_ke f — pinneapple_design.topology.problems (v06)
- confidence_index f — pinneapple_twin3d.wear.forecast (v06)
- ConfidenceComponent C — pinneapple_analysis.verification.physics_confidence_score (v05)
- Config C — pinneapple_check.config (v05)
- Config C — pinneapple_mcp.config (v07)
- config_from_spec f — pinneapple_veriphysics.validation_lab (v05)
- ConfigError C — pinneapple_check.config (v05)
- ConfigMismatch C — pinneapple_analysis.benchmark.runner (v05)
- configurations f — pinneapple_condense.benchmark (v02)
- confirm f — pinneapple_flow.identify (v07)
- conformal_interval f — pinneapple_analysis.uncertainty.conformal_study (v05)
- conformal_quantile f — pinneapple_analysis.uncertainty.conformal (v05)
- conformal_quantile f — pinneapple_analysis.uncertainty.conformal_study (v05)
- conformal_quantile_valid f — pinneapple_analysis.uncertainty.conformal (v05)
- conformal_scores f — pinneapple_analysis.uncertainty.conformal_study (v05)
- conformal_study f — pinneapple_analysis.uncertainty.conformal_study (v05)
- ConformalPredictor C — pinneapple_analysis.uncertainty.conformal (v05)
- ConformalStudy C — pinneapple_analysis.uncertainty.conformal_study (v05)
- connect f — pinneapple_core.service.db (v01)
- Connection C — pinneapple_design.bricks.assembly (v06)
- Connection C — pinneapple_systems.cosimulation.connection (v06)
- ConnectionSpec C — pinneapple_design.geometry.gen.threaded_connection (v06)
- cons_to_prim f — pinneapple_physics.blackhole.hydro_numba (v02)
- ConsequentialActionClassifier C — pinneapple_problemdesign.autonomy (v02)
- conservation f — pinneapple_core.loss (v01)
- conservation_check f — pinneapple_physics.physics_audit.monitor (v02)
- conservation_check f — pinneapple_simulation.flight.verification (v03)
- ConservationCheck C — pinneapple_analysis.validation.conservation (v05)
- ConservationProjection C — pinneapple_physics.conservation (v02)
- ConsistencyReport C — pinneapple_physics.multiscale.consistency (v02)
- constant_properties f — pinneapple_physics.physics_audit.assumptions (v02)
- constant_provenance f — pinneapple_physics.pde_environment.presets.astropy_bridge (v02)
- constants_of f — pinneapple_design.design_optimizer.study (v06)
- ConstantWind C — pinneapple_simulation.flight.turbulence (v03)
- constellation_names f — pinneapple_physics.astro.stars (v02)
- constitutive_check f — pinneapple_physics.physics_audit.monitor (v02)

- constrained_rod_buckling_load f — pinneapple_systems.process_components.pipe_stress_mechanics (v06)
- ConstrainedRodBucklingResult C — pinneapple_systems.process_components.pipe_stress_mechanics (v06)
- Constraint C — pinneapple_core.optim (v01)
- Constraint C — pinneapple_decision.constraints (v05)
- Constraint C — pinneapple_design.design_optimizer.study (v06)
- Constraint C — pinneapple_simulation.numerical_solvers.xtfc_general (v03)
- constraint_checks f — pinneapple_design.surrogate_inverse.core (v06)
- constraint_map f — pinneapple_simulation.electromagnetics.fem (v03)
- constraint_mask f — pinneapple_data.audit (v04)
- constraint_violation f — pinneapple_design.thermal.board_cooling.optimize (v06)
- constraint_violations f — pinneapple_design.surrogate_inverse.brake_rotor (v06)
- constraint_violations f — pinneapple_design.surrogate_inverse.heatsink_fins (v06)
- constraint_violations f — pinneapple_design.surrogate_inverse.reflex_port (v06)
- constraint_violations f — pinneapple_design.surrogate_inverse.suspension_arm (v06)
- constraint_violations f — pinneapple_design.surrogate_inverse.transdermal_patch (v06)
- ConstraintBase C — pinneapple_design.design_optimizer.constraints (v06)
- constraints_from f — pinneapple_design.design_optimizer.spec (v06)
- ConstraintSet C — pinneapple_design.design_optimizer.constraints (v06)
- ConstraintsSpec C — pinneapple_problemdesign.schema (v02)
- ConstraintViolationError C — pinneapple_decision.constraints (v05)
- contact_entry_angle f — pinneapple_simulation.terramechanics.mobility (v03)
- contact_net f — pinneapple_simulation.terramechanics.surrogate (v03)
- contact_sheet f — pinneapple_design.geometry_library.render (v06)
- ContactResult C — pinneapple_simulation.terramechanics.wong_reece (v03)
- contacts f — pinneapple_tools.visualization.molecular (v07)
- ContactSurrogate C — pinneapple_simulation.terramechanics.surrogate (v03)
- Context C — pinneapple_physics.symbolic_pde.text (v02)
- context_for f — pinneapple_lab.next_steps (v07)
- continuity f — pinneapple_physics.maxwell_checks (v02)
- ContinuousAdjointSolver C — pinneapple_design.design_optimizer.adjoint (v06)
- ContinuousCatalog C — pinneapple_neural.architectures.continuous.registry (v04)
- ContinuousModelBase C — pinneapple_neural.architectures.continuous.base (v04)
- continuum f — pinneapple_physics.physics_audit.assumptions (v02)
- ContOutput C — pinneapple_neural.architectures.continuous.base (v04)
- ContOutput C — pinneapple_neural.architectures.continuous.deep_state_space (v04)
- Contract C — pinneapple_data.contracts.models (v04)
- contrast_ratio f — pinneapple_lab.seal (v07)
- control f — pinneapple_flow.nodes.methods (v07)
- control_probability f — pinneapple_systems.sports.football.pitch_control (v06)
- control_room f — pinneapple_systems.digital_twin.pipe_loop.report (v06)
- control_study f — pinneapple_systems.operations.contracts (v06)
- ControlledOpInf C — pinneapple_systems.operations.rom (v06)
- Controls C — pinneapple_simulation.flight.aircraft (v03)
- controls f — pinneapple_hub.zoo.demo (v07)
- controls_for f — pinneapple_security.compliance (v05)
- ControlScenario C — pinneapple_systems.operations.control (v06)
- ControlSpec C — pinneapple_systems.operations.control (v06)
- ControlSurface C — pinneapple_simulation.flight.aero (v03)
- Conv1DModel C — pinneapple_neural.architectures.convolutions.conv1d (v04)
- Conv2DModel C — pinneapple_neural.architectures.convolutions.conv2d (v04)
- Conv3DModel C — pinneapple_neural.architectures.convolutions.conv3d (v04)
- ConvAdapter C — pinneapple_neural.architectures.adapters.conv (v04)
- convection_coefficients f — pinneapple_physics.closed_form.pcb_thermal (v02)
- convection_diffusion f — pinneapple_simulation.numerical_solvers.gated_xtfc (v03)
- convection_exact f — pinneapple_physics.paper_problems (v02)

- ConvectionDiffusion1D C — pinneapple_arena.problems (v04)
- ConvectiveCooling C — pinneapple_design.qualitative.models (v06)
- convergence f — pinneapple_flow.nodes.methods (v07)
- convergence f — pinneapple_simulation.numerical_solvers.heat_fv_reference (v03)
- convergence f — pinneapple_simulation.numerical_solvers.operator_datasets (v03)
- convergence_order f — pinneapple_simulation.robot_sim.sim (v03)
- convergence_section f — pinneapple_app.backend.core.surrogate_report (v07)
- convergence_study f — pinneapple_systems.time_series.enso.intermediate (v06)
- ConvergenceResult C — pinneapple_analysis.verification.convergence (v05)
- ConversationRecord C — pinneapple_llm.conversation_store (v07)
- ConversationStore C — pinneapple_llm.conversation_store (v07)
- conversion_text f — pinneapple_data.unified (v04)
- convert f — pinneapple_data.contracts.tables (v04)
- convert f — pinneapple_data.physical_units (v04)
- convert f — pinneapple_llm.qa.units (v07)
- convert_surface f — pinneapple_simulation.external_solvers.cfd_formats.convert (v03)
- convex_hull f — pinneapple_design.bricks.statics (v06)
- ConvModelBase C — pinneapple_neural.architectures.convolutions.base (v04)
- ConvolutionCatalog C — pinneapple_neural.architectures.convolutions.registry (v04)
- ConvOutput C — pinneapple_neural.architectures.convolutions.base (v04)
- CoolingDesign C — pinneapple_design.thermal.board_cooling.mission (v06)
- CoolingMission C — pinneapple_design.thermal.board_cooling.mission (v06)
- CoolingSurrogate C — pinneapple_design.thermal.board_cooling.learn (v06)
- Coordinate C — pinneapple_core.transforms (v01)
- CoordMapping C — pinneapple_physics.pinn_solver.io.mappings (v02)
- coords_of f — pinneapple_flow.nodes._common (v07)
- CopernicusClient C — pinneapple_data.datasets.external_sources (v04)
- copper_resistance f — pinneapple_simulation.electromagnetics.machines.drive (v03)
- copy_model f — pinneapple_hub.zoo._models (v07)
- corner_scores f — pinneapple_lab.seal (v07)
- corner_stackup f — pinneapple_analysis.uncertainty.margin (v05)
- CornerModule C — pinneapple_simulation.vehicle.spec (v03)
- correct f — pinneapple_condense.exceptions (v02)
- correction_study f — pinneapple_condense.benchmark (v02)
- CorrelatedLatentField C — pinneapple_physics.pinn_solver.stochastic (v02)
- corrupt f — pinneapple_worldmodel.physical_world (v04)
- cosim_from_models f — pinneapple_systems (v06)
- CoSimEngine C — pinneapple_systems.cosimulation.engine (v06)
- CoSimGraph C — pinneapple_systems.cosimulation.graph (v06)
- CoSimLoss C — pinneapple_systems.cosimulation.losses (v06)
- CoSimNode C — pinneapple_systems.cosimulation.node (v06)
- CoSimTrainer C — pinneapple_systems.cosimulation.trainer (v06)
- cosine_lr f — pinneapple_analysis.cost.equal_budget (v05)
- CosmosAdapter C — pinneapple_worldmodel.physical_world (v04)
- CosmosEncoderDataset C — pinneapple_worldmodel.training_hooks (v04)
- CosmosProvider C — pinneapple_problemdesign.providers.cosmos_provider (v02)
- CosmosProviderConfig C — pinneapple_problemdesign.providers.cosmos_provider (v02)
- cosq_feed f — pinneapple_simulation.electromagnetics.reflector.po (v03)
- cost_text f — pinneapple_reason.chain (v05)
- CostLedger C — pinneapple_analysis.cost.profiler (v05)
- CostModel C — pinneapple_design.design_optimizer.presets (v06)
- CostRecord C — pinneapple_analysis.cost.profiler (v05)
- cotan_laplacian f — pinneapple_design.geometry.ops.curvature (v06)
- coulomb_potential f — pinneapple_quantum.loss.schrodinger (v04)
- CoulombSuperposition C — pinneapple_science.em.statics (v02)
- count_evaluations f — pinneapple_analysis.inverse_problems.forward (v05)

- count_flops f — pinneapple_analysis.cost.equal_budget (v05)
- count_gpus f — pinneapple_neural.trainer.parallel (v04)
- count_parameters f — pinneapple_flow.learn (v07)
- count_parameters f — pinneapple_tools.benchmark_suite.crossframework.weights (v07)
- count_rows f — pinneapple_registry.gateway_api (v01)
- count_trainable f — pinneapple_adaptation.transfer_learning.freeze (v04)
- counterexample_search f — pinneapple_analysis.proof.graph (v05)
- Countermeasure C — pinneapple_reason.library (v05)
- countermeasure f — pinneapple_flow.nodes.reason (v07)
- counts f — pinneapple_flow.coverage (v07)
- CoupledModel C — pinneapple_systems.time_series.enso.intermediate (v06)
- CoupledParams C — pinneapple_systems.time_series.enso.intermediate (v06)
- coupling_classes f — pinneapple_physics.non_hermitian.ep_tuning (v02)
- CouplingLoss C — pinneapple_systems.cosimulation.losses (v06)
- courant_limit f — pinneapple_simulation.electromagnetics.fdtd (v03)
- courant_number f — pinneapple_data.cae.metrics (v04)
- coverage f — pinneapple_data.doe.coverage (v04)
- coverage f — pinneapple_flow.coverage (v07)
- coverage f — pinneapple_lab.explorer (v07)
- coverage f — pinneapple_lab.uq (v07)
- coverage f — pinneapple_systems.time_series.metrics_ext.probabilistic (v06)
- coverage_by_group f — pinneapple_analysis.uncertainty.conformal_study (v05)
- coverage_check f — pinneapple_lab.uq (v07)
- coverage_md f — pinneapple_lab.explorer (v07)
- coverage_study f — pinneapple_analysis.uncertainty.residual_bound (v05)
- CoverageReport C — pinneapple_analysis.uncertainty.residual_bound (v05)
- cp_max_newtonian f — pinneapple_physics.closed_form.compressible (v02)
- CPTECClient C — pinneapple_data.datasets.external_sources (v04)
- cpu_heatsink_thermal f — pinneapple_physics.pde_environment.presets.engineering (v02)
- cpu_model f — pinneapple_tools.benchmark_suite.crossframework.records (v07)
- cr3bp_collinear_lagrange_point f — pinneapple_physics.pde_environment.presets.astrophysics (v02)
- cr3bp_lagrange_points f — pinneapple_physics.pde_environment.presets.astrophysics (v02)
- cr3bp_omega_gradient f — pinneapple_physics.pde_environment.presets.astrophysics (v02)
- cr3bp_planar_synodic f — pinneapple_physics.pde_environment.presets.astrophysics (v02)
- crate_and_pendulum f — pinneapple_flow.nodes_ext.solvers (v07)
- Crater C — pinneapple_simulation.rover.terrain (v03)
- crater_site f — pinneapple_simulation.vehicle.scenarios (v03)
- crease_normals f — pinneapple_design.geometry_library.meshops (v06)
- create_app f — pinneapple_design.fitmesh.api (v06)
- create_app f — pinneapple_llm.workbench.server (v07)
- creeping_flow f — pinneapple_physics.physics_audit.assumptions (v02)
- crf f — pinneapple_physics.closed_form.pipe_insulation (v02)
- Criterion C — pinneapple_tools.automation.workflow (v07)
- critical_angle f — pinneapple_simulation.electromagnetics.waves (v03)
- CritiqueFinding C — pinneapple_analysis.verification.architecture_critique (v05)
- cross f — pinneapple_simulation.multibody.tree (v03)
- cross_check f — pinneapple_design.bricks.dynamics (v06)
- cross_correlation f — pinneapple_systems.time_series.eda.plots (v06)
- cross_scale_consistency f — pinneapple_physics.multiscale.consistency (v02)
- cross_section_polygons f — pinneapple_simulation.electromagnetics.machines.interop (v03)
- cross_validate f — pinneapple_autolab.surrogates (v07)
- cross_validate f — pinneapple_physics.astro.stars_ml (v02)
- crossing_rate f — pinneapple_analysis.uncertainty.quantile (v05)
- crowd_load f — pinneapple_reason.modes.structural (v05)
- crowding_distance f — pinneapple_design.design_optimizer.pareto (v06)
- crps_ensemble f — pinneapple_analysis.uncertainty.posterior_metrics (v05)

- crps_gaussian f — pinneapple_analysis.uncertainty.posterior_metrics (v05)
- crystal_phonon f — pinneapple_physics.pde_environment.presets.multidisciplinary (v02)
- CSGCircle C — pinneapple_design.geometry.csg (v06)
- CSGDifference C — pinneapple_design.geometry.csg (v06)
- CSGEllipse C — pinneapple_design.geometry.csg (v06)
- CSGIntersection C — pinneapple_design.geometry.csg (v06)
- CSGPolygon C — pinneapple_design.geometry.csg (v06)
- CSGRectangle C — pinneapple_design.geometry.csg (v06)
- CSGUnion C — pinneapple_design.geometry.csg (v06)
- cst_surface f — pinneapple_design.aero.geometry (v06)
- Ctx C — pinneapple_check.checks.common (v05)
- ctx_metrics f — pinneapple_flow.nodes.vehicle (v07)
- CUDAGraphModule C — pinneapple_neural.trainer.hpc (v04)
- CUDAPrefetcher C — pinneapple_neural.trainer.parallel (v04)
- curate f — pinneapple_lab.curation (v07)
- curate_dataset f — pinneapple_tools.dataset_quality.vlm_curation (v07)
- curate_sequence f — pinneapple_tools.dataset_quality.vlm_curation (v07)
- CurationRecord C — pinneapple_tools.dataset_quality.vlm_curation (v07)
- curl f — pinneapple_core.operators (v01)
- current f — pinneapple_hub.library.flowruns (v07)
- current_period f — pinneapple_core.service.metering (v01)
- CurriculumConfig C — pinneapple_worldmodel.curriculum (v04)
- CurriculumStage C — pinneapple_worldmodel.curriculum (v04)
- curve_fit f — pinneapple_science.instruments (v02)
- curved_pipe_ratio f — pinneapple_physics.closed_form.pipe_flow (v02)
- CurvedPathProfile C — pinneapple_systems.process_components.curved_path_geometry (v06)
- CurveFit C — pinneapple_science.instruments (v02)
- CurveFitSynthGenerator C — pinneapple_data.synth.curvefit (v04)
- curves_figure f — pinneapple_tools.nvidia.figures (v07)
- custom f — pinneapple_analysis.proof.graph (v05)
- custom_spec f — pinneapple_flow.libcalls (v07)
- CustomPhysics C — pinneapple_app.backend.core.physics_adapter (v07)
- CustomProblemRequest C — pinneapple_app.backend.routers.problems (v07)
- cusum f — pinneapple_security.cps (v05)
- CusumAlarm C — pinneapple_security.cps (v05)
- cut_metrics_at f — pinneapple_simulation.electromagnetics.reflector.study_report (v03)
- CutawayMesh C — pinneapple_physics.blackhole.twin (v02)
- cw_closest_approach f — pinneapple_physics.astro.conjunction (v02)
- cw_exact_velocity f — pinneapple_physics.pde_environment.presets.astrophysics (v02)
- cw_mean_motion f — pinneapple_physics.astro.conjunction (v02)
- cycle_energy f — pinneapple_simulation.electromagnetics.machines.drive (v03)
- cycles_renders f — pinneapple_autolab.domains.racecar (v07)
- CyclesScene C — pinneapple_blender.cycles (v07)
- cylinder f — pinneapple_design.geometry.bodies (v06)
- cylinder f — pinneapple_design.geometry.vehicle_cad (v06)
- cylinder f — pinneapple_design.geometry_library.kit (v06)
- cylinder f — pinneapple_simulation.flight.mass (v03)
- cylinder f — pinneapple_simulation.pipelines.freecad_openfoam.freecad_scripts.fc_job (v03)
- cylinder f — pinneapple_twin3d.primitives (v06)
- cylinder3d f — pinneapple_design.geometry.gen.sdf_shapes (v06)
- cylinder_conductance f — pinneapple_simulation.electromagnetics.machines.thermal (v03)
- cylinder_flow_to_tensors f — pinneapple_data.datasets.deepmind_mgn (v04)
- cylinder_inertia f — pinneapple_simulation.robot_sim.spec (v03)
- cylinder_mask f — pinneapple_simulation.numerical_solvers.lbm (v03)
- cylinder_mesh f — pinneapple_design.bricks.mesh (v06)
- cylinder_scattering_tm f — pinneapple_simulation.electromagnetics.waves (v03)

- cylinder_tm_coefficients f — pinneapple_simulation.electromagnetics.waves (v03)
- cylinder_tm_total f — pinneapple_physics.closed_form.electromagnetics (v02)
- cylinder_wake f — pinneapple_lab.experiments.rom (v07)
- CylinderLBM C — pinneapple_lab.experiments.cylinder (v07)
- cylindrical_cavity_resonance f — pinneapple_simulation.electromagnetics.waves (v03)

D

- d f — pinneapple_analysis.validation.pinn (v05)
- d20_climatology f — pinneapple_systems.time_series.enso.data (v06)
- d20_from_profile f — pinneapple_systems.time_series.enso.data (v06)
- d20_monthly f — pinneapple_systems.time_series.enso.data (v06)
- d_optimal_design f — pinneapple_analysis.inverse_problems.identifiability (v05)
- dam_break f — pinneapple_flow.nodes_ext.solvers (v07)
- damkohler f — pinneapple_physics.physics_audit.regime (v02)
- Darcy C — pinneapple_analysis.benchmark.builtin.standard (v05)
- darcy_2d f — pinneapple_simulation.numerical_solvers.operator_benchmarks (v03)
- darcy_checks f — pinneapple_check._reuse (v05)
- darcy_checks f — pinneapple_physics.darcy_checks (v02)
- darcy_consistency f — pinneapple_data.audit (v04)
- darcy_inputs f — pinneapple_simulation.numerical_solvers.operator_datasets (v03)
- darcy_matrix f — pinneapple_simulation.numerical_solvers.operator_benchmarks (v03)
- darcy_operator_apply f — pinneapple_physics.darcy_checks (v02)
- darcy_pressure_drop f — pinneapple_physics.closed_form.pipe_flow (v02)
- darcy_pressure_only_3d_default f — pinneapple_physics.pde_environment.presets.industry (v02)
- darcy_problem f — pinneapple_analysis.uncertainty.residual_bound (v05)
- darcy_residual f — pinneapple_physics.darcy_checks (v02)
- darcy_set f — pinneapple_simulation.numerical_solvers.operator_datasets (v03)
- darcy_solver_service f — pinneapple_tools.model_services.grid (v07)
- darcy_trust f — pinneapple_tools.model_services.grid (v07)
- darcy_trust_report f — pinneapple_physics.darcy_checks (v02)
- DarcyCheck C — pinneapple_physics.darcy_checks (v02)
- DarcyData C — pinneapple_tools.benchmark_suite.crossframework.darcy (v07)
- DarcyFlow2D C — pinneapple_arena.problems (v04)
- DarcyProblem C — pinneapple_analysis.validation.fault_injection.problems (v05)
- dark_sky_nl f — pinneapple_physics.astro.observing (v02)
- Dashboard C — pinneapple_tools.visualization.dashboard.spec (v07)
- dashboard f — pinneapple_flow.nodes.vehicle (v07)
- Data C — pinneapple_analysis.benchmark.builtin.inverse_heat_impl (v05)
- data_dir f — pinneapple_core.service (v01)
- data_efficiency f — pinneapple_systems.sports.football.pc_study (v06)
- data_efficiency f — pinneapple_systems.sports.football.xg (v06)
- data_faults f — pinneapple_analysis.validation.fault_injection.faults (v05)
- data_from_spec f — pinneapple_analysis.surrogate.contracts (v05)
- data_root f — pinneapple_flow.config (v07)
- data_specs f — pinneapple_systems.time_series.forecast_lab.presets (v06)
- DataBundle C — pinneapple_app.backend.core.data_pipeline (v07)
- datacenter_airflow_2d f — pinneapple_physics.pde_environment.presets.engineering (v02)
- datacenter_cfd_3d f — pinneapple_physics.pde_environment.presets.engineering (v02)
- datacenter_server_thermal f — pinneapple_physics.pde_environment.presets.engineering (v02)
- DataConfig C — pinneapple_app.backend.core.data_pipeline (v07)
- DataConsistencyGuidance C — pinneapple_neural.architectures.pinns.physics_diffusion (v04)
- DataConstraint f — pinneapple_physics.pde_environment.conditions (v02)
- DataLabel C — pinneapple_security.classification (v05)
- DataLoader C — pinneapple_core.data (v01)
- DataLoaderSpec C — pinneapple_data.dataloaders (v04)
- DataLoss C — pinneapple_systems.cosimulation.losses (v06)

- DataMisfitBase C — pinneapple_analysis.inverse_problems.noise_models (v05)
- DataModule C — pinneapple_neural.trainer.datamodule (v04)
- DataRequest C — pinneapple_app.backend.routers.experiments (v07)
- Dataset C — pinneapple_analysis.validation.fault_injection.problems (v05)
- dataset f — pinneapple_hub.library.snippets (v07)
- dataset f — pinneapple_physics.astro.stars_ml (v02)
- dataset f — pinneapple_worldmodel.physical_world (v04)
- dataset_audit f — pinneapple_check._reuse (v05)
- dataset_card f — pinneapple_hub.zoo.hub_cards (v07)
- dataset_checks f — pinneapple_analysis.validation.fault_injection.checks (v05)
- dataset_from_run f — pinneapple_core.workflows.datasets (v01)
- dataset_ids f — pinneapple_data.datasets (v04)
- dataset_info f — pinneapple_data.datasets (v04)
- dataset_of f — pinneapple_flow.nodes.data (v07)
- dataset_spec f — pinneapple_analysis.surrogate.contracts (v05)
- DatasetBuilder C — pinneapple_worldmodel.dataset (v04)
- DatasetCatalog C — pinneapple_worldmodel.dataset_factory (v04)
- DatasetConfig C — pinneapple_arena.config (v04)
- DatasetConfig C — pinneapple_worldmodel.dataset (v04)
- DatasetEntry C — pinneapple_worldmodel.dataset_factory (v04)
- DatasetGenerator C — pinneapple_arena.pointwise.datasets (v04)
- DatasetInfo C — pinneapple_data.datasets.registry (v04)
- DatasetInfo C — pinneapple_hub.zoo.datasets (v07)
- DatasetManifest C — pinneapple_worldmodel.dataset_packager (v04)
- DatasetPackager C — pinneapple_worldmodel.dataset_packager (v04)
- DatasetPhysics C — pinneapple_data.audit (v04)
- DatasetProblem C — pinneapple_arena.dataset_problems (v04)
- DatasetRef C — pinneapple_data.contracts.models (v04)
- DatasetRegistry C — pinneapple_data.datasets.registry (v04)
- datasets f — pinneapple_flow.nodes_ext.arena_pipelines (v07)
- datasets f — pinneapple_hub.library.sources (v07)
- DatasetSpec C — pinneapple_arena.external.schema (v04)
- DatasetSpec C — pinneapple_data.contracts.models (v04)
- DatasetSplit C — pinneapple_hub.zoo.datasets (v07)
- DatasetStore C — pinneapple_registry.dataset_store (v01)
- DatasetTarget C — pinneapple_check.config (v05)
- DatasetWriter C — pinneapple_lab.dataset (v07)
- Datasheet C — pinneapple_tools.reporting.datasheet (v07)
- DataSpec C — pinneapple_problemdesign.schema (v02)
- datetime_from_jd f — pinneapple_physics.astro.timescales (v02)
- DDPPINNTrainer C — pinneapple_neural.trainer.distributed (v04)
- DDPTrainerConfig C — pinneapple_neural.trainer.distributed (v04)
- dean_number f — pinneapple_physics.closed_form.pipe_flow (v02)
- Debye C — pinneapple_simulation.electromagnetics.media (v03)
- decay_rate_constant_per_day f — pinneapple_design.surrogate_inverse.transdermal_patch (v06)
- DecayingTurbulence2D C — pinneapple_physics.multiscale.turbulence (v02)
- decide f — pinneapple_decision.api (v05)
- decimals f — pinneapple_tools.visualization.studio.colormap (v07)
- decimate_mesh f — pinneapple_design.geometry.ops.clean (v06)
- decimate_surface f — pinneapple_tools.visualization.studio.scene (v07)
- Decision C — pinneapple_data.formfill.extract (v04)
- Decision C — pinneapple_decision.schema (v05)
- Decision C — pinneapple_flow.decisions (v07)
- Decision C — pinneapple_security.classification (v05)
- decision_engine f — pinneapple_decision.api (v05)
- DecisionConfig C — pinneapple_arena.config (v04)

- DecisionLevel C — pinneapple_decision.schema (v05)
- DecisionLoop C — pinneapple_decision.loop (v05)
- DecisionRecord C — pinneapple_veriphysics.decision (v05)
- DecisionState C — pinneapple_decision.schema (v05)
- DecisionTree C — pinneapple_decision.tree (v05)
- Deck C — pinneapple_data.cae.inp (v04)
- deck f — pinneapple_flow.nodes_ext.structures (v07)
- deck_conditions f — pinneapple_data.cae.inp (v04)
- deck_mesh f — pinneapple_data.cae.inp (v04)
- decode f — pinneapple_design.intent.pipeline (v06)
- decode f — pinneapple_physics.blackhole.dataset (v02)
- decode_image_frames f — pinneapple_perception.piv_tools (v04)
- decode_video f — pinneapple_perception.piv_tools (v04)
- DecoderLayer C — pinneapple_neural.architectures.transformers.autoformer (v04)
- decompose f — pinneapple_flow.nodes_ext.solver_registry (v07)
- decompose f — pinneapple_systems.time_series.solar_cycle (v06)
- decompose_dict f — pinneapple_simulation.pipelines.freecad_openfoam.mesh (v03)
- decompose_uncertainty f — pinneapple_analysis.uncertainty.decomposition (v05)
- Decomposition C — pinneapple_systems.time_series.solar_cycle (v06)
- decrypt_array f — pinneapple_security.encryption (v05)
- decrypt_bytes f — pinneapple_security.encryption (v05)
- decrypt_file f — pinneapple_security.encryption (v05)
- DecryptionError C — pinneapple_security.encryption (v05)
- DeepEnsemble C — pinneapple_systems.component_modeling.ensemble (v06)
- DeepONet C — pinneapple_neural.architectures.neural_operators.deeponet (v04)
- DeepONetMethod C — pinneapple_analysis.benchmark.builtin.fields (v05)
- DeepStateSpaceModel C — pinneapple_neural.architectures.continuous.deep_state_space (v04)
- DeepUQROM C — pinneapple_neural.architectures.rom.deep_uq_rom (v04)
- DeepXDEBackend C — pinneapple_tools.benchmark_suite.backends.deepxde_backend (v07)
- DeepXDESolveResult C — pinneapple_physics.deepxde_bridge.adapter (v02)
- default_cache f — pinneapple_design.bricks.ldraw (v06)
- default_checks f — pinneapple_analysis.validation.fault_injection.checks (v05)
- default_config f — pinneapple_flow.learn (v07)
- default_decider f — pinneapple_arena.decision_mode (v04)
- default_ephemeris f — pinneapple_physics.astro.ephemeris (v02)
- default_eps f — pinneapple_simulation.meshless.methods (v03)
- default_experts f — pinneapple_systems.time_series.adaptive (v06)
- default_geometry f — pinneapple_physics.pde_environment.presets.tag_geometry (v02)
- default_geometry_note f — pinneapple_core.onboarding.catalog (v01)
- default_geometry_report f — pinneapple_physics.pde_environment.presets.tag_geometry (v02)
- default_graph f — pinneapple_physics.physics_audit.knowledge (v02)
- default_guesses f — pinneapple_simulation.accelerated_foam.benchmark (v03)
- default_metrics f — pinneapple_neural.trainer.metrics (v04)
- default_mission f — pinneapple_design.thermal.board_cooling.mission (v06)
- default_model f — pinneapple_systems.operations.live (v06)
- default_models f — pinneapple_simulation.electromagnetics.antenna.surrogates (v03)
- default_range f — pinneapple_tools.visualization.studio.compose (v07)
- default_ranks f — pinneapple_neural.architectures.rom.study (v04)
- default_reference_meshes f — pinneapple_analysis.verification.geometry_ood_guardrail (v05)
- default_registry f — pinneapple_tools.model_services.registry (v07)
- default_root f — pinneapple_lab.store (v07)
- default_schedule f — pinneapple_simulation.flight.takeoff (v03)
- default_solver_specs f — pinneapple_simulation.accelerated_foam.benchmark (v03)
- default_ts_metrics f — pinneapple_systems.time_series.metrics (v06)
- default_url f — pinneapple_flow.store (v07)
- default_variant f — pinneapple_condense.learned (v02)

- default_weigher f — pinneapple_data.cache_bytes (v04)
- default_workdir f — pinneapple_flow.runtime (v07)
- defective_models f — pinneapple_analysis.validation.physics_learning_cases (v05)
- Defence C — pinneapple_analysis.validation.arena.blue (v05)
- define_custom f — pinneapple_app.backend.core.problem (v07)
- define_problem f — pinneapple_arena.custom_builder (v04)
- define_problem f — pinneapple_physics (v02)
- Definition C — pinneapple_systems.process_components.explicit_equation_system (v06)
- deflection_angle_rad f — pinneapple_physics.astro.lensing (v02)
- DeformableTerrain C — pinneapple_simulation.terramechanics.terrain (v03)
- delay_embedding f — pinneapple_systems.time_series.predictability (v06)
- DelayODEForecaster C — pinneapple_systems.time_series.forecast_lab.physical (v06)
- deliver f — pinneapple_design.thermal.board_cooling.report (v06)
- deliver f — pinneapple_systems.digital_twin.pipe_loop.report (v06)
- delta_t_model f — pinneapple_physics.astro.timescales (v02)
- DEM C — pinneapple_simulation.numerical_solvers.particles (v03)
- demagnetisation_margin f — pinneapple_simulation.electromagnetics.machines.thermal (v03)
- demagnetisation_refinement f — pinneapple_simulation.electromagnetics.machines.thermal (v03)
- demand_capacity_figure f — pinneapple_reason.report (v05)
- demo_record f — pinneapple_systems.operations.live (v06)
- demo_scenario f — pinneapple_systems.operations.plant (v06)
- demo_site f — pinneapple_simulation.rover.autolab.site (v03)
- dense_forecast f — pinneapple_systems.time_series.forecast_lab.renders (v06)
- DenseAutoencoder C — pinneapple_neural.architectures.autoencoders.dense_ae (v04)
- DenseVolumeOperatorConfig C — pinneapple_neural.workflows.dense_volume_operator (v04)
- densify f — pinneapple_simulation.rover.autolab.planner (v03)
- DensityCodec C — pinneapple_physics.blackhole.forecast (v02)
- DensityFilter C — pinneapple_design.topology.filters (v06)
- DensityTable C — pinneapple_physics.astro.decay (v02)
- depletion_time_days f — pinneapple_design.surrogate_inverse.transdermal_patch (v06)
- depletion_time_days f — pinneapple_physics.closed_form.membrane_diffusion (v02)
- DeploymentSpec C — pinneapple_problemdesign.schema (v02)
- derivative_cache f — pinneapple_physics.pinn_solver.compiler.autograd_ops (v02)
- derivative_key f — pinneapple_physics.symbolic_pde.text (v02)
- derivative_keys f — pinneapple_physics.symbolic_pde.text (v02)
- DerivativeComputer C — pinneapple_physics.pinn_solver.factory.autodiff (v02)
- DerivativeKey C — pinneapple_physics.pinn_solver.factory.autodiff (v02)
- derive_key f — pinneapple_security.encryption (v05)
- derive_psi_shape_a f — pinneapple_systems.process_components.similarity_map (v06)
- DerivedSpec C — pinneapple_pdb.derived (v04)
- descend f — pinneapple_design.surrogate_inverse.core (v06)
- describe f — pinneapple_core.onboarding.catalog (v01)
- describe f — pinneapple_data.cae.dataset (v04)
- describe f — pinneapple_design.qualitative.descriptors (v06)
- describe f — pinneapple_flow.planner.extensions (v07)
- describe f — pinneapple_flow.planner.wizard (v07)
- describe f — pinneapple_systems.digital_twin.product.runner (v06)
- describe f — pinneapple_tools.benchmark_suite.crossframework.problems (v07)
- describe_library f — pinneapple_reason (v05)
- describe_method f — pinneapple_mcp.tools (v07)
- describe_unit f — pinneapple_data.physical_units (v04)
- Descriptors C — pinneapple_design.qualitative.descriptors (v06)
- Design C — pinneapple_data.doe.designs (v04)
- Design C — pinneapple_neural.lpm (v04)
- design f — pinneapple_data.doe.designs (v04)
- design f — pinneapple_design.intent.api (v06)

- design f — pinneapple_mcp.tools (v07)
- design_by_name f — pinneapple_design.thermal.fin_array_speed (v06)
- design_features f — pinneapple_physics.non_hermitian.ep_tuning (v02)
- design_file f — pinneapple_design.design_optimizer.presets (v06)
- design_intent f — pinneapple_flow.wrappers (v07)
- design_loop f — pinneapple_design.thermal.board_cooling.optimize (v06)
- design_of f — pinneapple_design.thermal.fin_array_speed (v06)
- design_points f — pinneapple_physics.physics_audit.improvement (v02)
- design_table f — pinneapple_tools.nvidia.heatsink (v07)
- design_to_dict f — pinneapple_design.surrogate_inverse.brake_rotor (v06)
- design_to_dict f — pinneapple_design.surrogate_inverse.heatsink_fins (v06)
- design_to_dict f — pinneapple_design.surrogate_inverse.reflex_port (v06)
- design_to_dict f — pinneapple_design.surrogate_inverse.suspension_arm (v06)
- design_to_dict f — pinneapple_design.surrogate_inverse.transdermal_patch (v06)
- DesignAgent C — pinneapple_problemdesign.agent (v02)
- DesignChain C — pinneapple_design.topology.driver (v06)
- DesignInput C — pinneapple_design.thermal.heatsink.engine (v06)
- DesignOptConfig C — pinneapple_design.design_optimizer.pipeline (v06)
- DesignOptimizerConfig C — pinneapple_design.design_optimizer.optimizer (v06)
- DesignOptLoop C — pinneapple_design.design_optimizer.pipeline (v06)
- DesignOptResult C — pinneapple_design.design_optimizer.pipeline (v06)
- DesignPackage C — pinneapple_design.intent.package (v06)
- DesignProblem C — pinneapple_design.design_optimizer.study (v06)
- DesignReport C — pinneapple_problemdesign.schema (v02)
- designs_frame f — pinneapple_design.design_optimizer.report (v06)
- DesignSafeClient C — pinneapple_data.datasets.external_sources (v04)
- DesignSpace C — pinneapple_design.design_optimizer.study (v06)
- DesignSpace C — pinneapple_design.thermal.board_cooling.mission (v06)
- DesignState C — pinneapple_problemdesign.state (v02)
- DesignVariable C — pinneapple_design.intent.template (v06)
- detect f — pinneapple_flow.planner.heuristic (v07)
- detect f — pinneapple_systems.operations.anomaly (v06)
- detect_divergence f — pinneapple_systems.digital_twin.recession (v06)
- detect_format f — pinneapple_simulation.external_solvers.cfd_formats.convert (v03)
- detect_licenses f — pinneapple_tools.rea_bridge.licenses (v07)
- detect_machine f — pinneapple_llm.workbench.models (v07)
- detect_outliers f — pinneapple_tools.dataset_quality.checks (v07)
- detect_project f — pinneapple_data.lineage.detect (v04)
- detect_time_column f — pinneapple_data.data_health (v04)
- detection_study f — pinneapple_analysis.validation.physics_learning (v05)
- DetectionMatrix C — pinneapple_analysis.validation.fault_injection.matrix (v05)
- DetectionResult C — pinneapple_systems.operations.anomaly (v06)
- DetectorConfig C — pinneapple_systems.operations.anomaly (v06)
- determinism_check f — pinneapple_analysis.benchmark.checks (v05)
- deterministic_npz f — pinneapple_physics.blackhole.dataset (v02)
- device_info f — pinneapple_arena.pointwise.training (v04)
- dew_point f — pinneapple_reason.props (v05)
- DFSPHSolver C — pinneapple_simulation.numerical_solvers.dfsph (v03)
- DFTMonitor C — pinneapple_simulation.electromagnetics.fdtd (v03)
- diagnose f — pinneapple_analysis.benchmark.advisor (v05)
- diagnose f — pinneapple_llm.qa.repair (v07)
- diagnose f — pinneapple_physics.physics_audit (v02)
- Diagnoser C — pinneapple_systems.digital_twin.pipe_loop.diagnose (v06)
- Diagnosis C — pinneapple_analysis.benchmark.advisor (v05)
- Diagnosis C — pinneapple_llm.qa.repair (v07)
- Diagnosis C — pinneapple_physics.physics_audit (v02)

- DiagnosticReport C — pinneapple_neural.trainer.advisor (v04)
- dic f — pinneapple_simulation.accelerated_foam.solvers (v03)
- dict_collate f — pinneapple_physics.pinn_solver.compiler.dataset (v02)
- did_you_know f — pinneapple_physics.astro.facts (v02)
- diebold_mariano f — pinneapple_systems.time_series.metrics_ext.comparison (v06)
- diff f — pinneapple_analysis.validation.report_artifact (v05)
- Difference C — pinneapple_worldmodel.geometry (v04)
- DifferencingStep C — pinneapple_neural.trainer.preprocess (v04)
- differential_evolution f — pinneapple_analysis.validation.arena.search (v05)
- DifficultyLevel C — pinneapple_arena.pointwise.schema (v04)
- diffusion_dominated f — pinneapple_physics.physics_audit.assumptions (v02)
- digest_array f — pinneapple_security.integrity (v05)
- digest_record f — pinneapple_security.integrity (v05)
- DigitalTwin C — pinneapple_systems.digital_twin.twin (v06)
- DigitalTwinConfig C — pinneapple_systems.digital_twin.twin (v06)
- dim_str f — pinneapple_data.physical_units (v04)
- Dimension C — pinneapple_arena.pointwise.schema (v04)
- Dimension C — pinneapple_lab.curation (v07)
- dimension f — pinneapple_simulation.electromagnetics.units (v03)
- dimension_of f — pinneapple_llm.qa.units (v07)
- dimension_str f — pinneapple_llm.qa.units (v07)
- DimensionlessNumbers C — pinneapple_analysis.verification.dimensional_analysis (v05)
- dims_unit f — pinneapple_data.cae.foam (v04)
- dipole_cartesian f — pinneapple_science.em.waves (v02)
- dipole_pattern f — pinneapple_simulation.electromagnetics.waves (v03)
- DipoleRadiation C — pinneapple_science.em.waves (v02)
- diprotic_fractions f — pinneapple_systems.process_components.reaction_kinetics (v06)
- directivity_vs_length f — pinneapple_simulation.electromagnetics.antenna.plots (v03)
- dirichlet_boundary_values f — pinneapple_simulation.numerical_solvers.nonlinear_beam_fem (v03)
- dirichlet_conditions f — pinneapple_simulation.numerical_solvers._bc (v03)
- DirichletBC f — pinneapple_physics.pde_environment.conditions (v02)
- DirichletBC C — pinneapple_physics.symbolic_pde.bc (v02)
- disc_capacitor f — pinneapple_science.em.statics (v02)
- disc_image f — pinneapple_physics.astro.lensing (v02)
- discover f — pinneapple_check.config (v05)
- discover f — pinneapple_flow.nodes.solvers (v07)
- discover f — pinneapple_tools.hpo_experiments.pipelines.discover (v07)
- discover_examples f — pinneapple_lab.experiments.examples (v07)
- discover_pde f — pinneapple_flow.identify (v07)
- DiscoveryResult C — pinneapple_tools.hpo_experiments.models (v07)
- DiscrepancyModel C — pinneapple_analysis.verification.causal_discrepancy (v05)
- discrete_gust f — pinneapple_simulation.flight.encounter (v03)
- discrete_time_rk_residual f — pinneapple_simulation.numerical_solvers.discrete_time_pinn (v03)
- DiscreteGust C — pinneapple_simulation.flight.turbulence (v03)
- DiscreteTimePINNSolver C — pinneapple_simulation.numerical_solvers.discrete_time_pinn (v03)
- DiscreteTimeResult C — pinneapple_simulation.numerical_solvers.discrete_time_pinn (v03)
- DiscreteTimeRKNet C — pinneapple_simulation.numerical_solvers.discrete_time_pinn (v03)
- Discretization C — pinneapple_simulation.meshless.methods (v03)
- discretize f — pinneapple_simulation.meshless.methods (v03)
- disk_grid f — pinneapple_simulation.electromagnetics.reflector.qz (v03)
- disk_on_axis f — pinneapple_simulation.electromagnetics.reflector.analytic (v03)
- displacement_errors f — pinneapple_systems.sports.football.kinematics (v06)
- displacement_factor f — pinneapple_tools.visualization.studio.fieldview (v07)
- display_units f — pinneapple_physics.weather.viz (v02)
- distill_to_symbolic f — pinneapple_neural.trainer.graybox (v04)
- DistributionSynthGenerator C — pinneapple_data.synth.distributions (v04)

- div f — pinneapple_core.operators (v01)
- divergence f — pinneapple_physics.pinn_solver.compiler.autograd_ops (v02)
- divergence_time f — pinneapple_analysis.validation.rollout (v05)
- DMDBaseline C — pinneapple_physics.blackhole.skill (v02)
- DMDc C — pinneapple_neural.architectures.rom.dmd (v04)
- DMResult C — pinneapple_systems.time_series.metrics_ext.comparison (v06)
- do_rhs f — pinneapple_systems.time_series.enso.oscillators (v06)
- do_roots f — pinneapple_systems.time_series.enso.oscillators (v06)
- doc_table_value f — pinneapple_flow.nodes_ext.examples (v07)
- docker_available f — pinneapple_simulation.external_solvers.calculix.runner (v03)
- docker_available f — pinneapple_simulation.external_solvers.openradioss.runner (v03)
- DockerExecutor C — pinneapple_core.workflows.executors (v01)
- doctor f — pinneapple_core.onboarding.doctor (v01)
- doctor_main f — pinneapple_core.onboarding.cli (v01)
- DoctorReport C — pinneapple_core.onboarding.doctor (v01)
- Document C — pinneapple_analysis.retrieval.corpus (v05)
- Document C — pinneapple_data.formfill.extract (v04)
- document_text f — pinneapple_data.formfill.llm (v04)
- doe f — pinneapple_simulation.electromagnetics.antenna.design (v03)
- doe f — pinneapple_simulation.pipelines.freecad_openfoam.sweep (v03)
- doe_design f — pinneapple_flow.wrappers (v07)
- dof_to_upd f — pinneapple_simulation.external_solvers.fenics.solver (v03)
- Domain C — pinneapple_core.domain (v01)
- domain f — pinneapple_flow.nodes.problem (v07)
- domain f — pinneapple_hub.library.sources (v07)
- domain f — pinneapple_simulation.meshless.domains (v03)
- Domain2D C — pinneapple_simulation.meshless.domains (v03)
- domain_of f — pinneapple_lab.manifest (v07)
- DomainMeta C — pinneapple_arena.external.schema (v04)
- domains f — pinneapple_core.onboarding.catalog (v01)
- DomainVerdict C — pinneapple_autolab.domain (v07)
- dominant_frequency f — pinneapple_physics.physics_audit.regime (v02)
- dominant_period f — pinneapple_systems.time_series.forecast_lab.baselines (v06)
- DoMINO C — pinneapple_physics.pinn_solver.domino (v02)
- double_well f — pinneapple_systems.time_series.synthetic (v06)
- double_well_drift f — pinneapple_systems.time_series.synthetic (v06)
- double_well_potential f — pinneapple_quantum.loss.schrodinger (v04)
- DownBlockND C — pinneapple_neural.architectures.neural_operators.uno (v04)
- download f — pinneapple_physics.weather.data (v02)
- download_pdf f — pinneapple_tools.hpo_experiments.sources.arxiv_pdf (v07)
- download_prefix f — pinneapple_data.datasets.deepmind_mgn (v04)
- downsample f — pinneapple_data.simulation_metadata.convergence (v04)
- dp_grade_estimate f — pinneapple_simulation.rover.checks (v03)
- dp_histogram f — pinneapple_security.dp (v05)
- dp_mean f — pinneapple_security.dp (v05)
- DPSGD C — pinneapple_security.dp (v05)
- draft_cadquery_template f — pinneapple_llm.cad_draft (v07)
- draft_digital_twin f — pinneapple_llm.twin_draft (v07)
- draft_geometry f — pinneapple_llm.geometry_draft (v07)
- draft_mesh_recipe f — pinneapple_llm.cad_draft (v07)
- draft_part f — pinneapple_llm.cad_draft (v07)
- draft_preset f — pinneapple_problemdesign.preset_authoring (v02)
- draft_preset_kwargs f — pinneapple_llm.draft (v07)
- draft_problem f — pinneapple_llm.draft (v07)
- DraftedPreset C — pinneapple_problemdesign.preset_authoring (v02)
- DraftRejected C — pinneapple_llm.draft (v07)

- `DraftResult` C — `pinneapple_llm.draft` (v07)
- `drag_factor` f — `pinneapple_simulation.numerical_solvers.particles` (v03)
- `drag_sphere_schiller_naumann` f — `pinneapple_simulation.numerical_solvers.lbm_tunnel` (v03)
- `DragAdjointObjective` C — `pinneapple_design.design_optimizer.adjoint` (v06)
- `DragObjective` C — `pinneapple_design.design_optimizer.objective` (v06)
- `draw_colorbars` f — `pinneapple_tools.visualization.studio.compose` (v07)
- `draw_pitch` f — `pinneapple_systems.sports.football.plots` (v06)
- `draw_text` f — `pinneapple_lab.seal` (v07)
- `DriftForecaster` C — `pinneapple_systems.time_series.baselines.naive` (v06)
- `DriftMonitor` C — `pinneapple_systems.digital_twin.product.policy` (v06)
- `DriftPolicy` C — `pinneapple_systems.digital_twin.product.policy` (v06)
- `DriftReport` C — `pinneapple_analysis.uncertainty.drift` (v05)
- `DriftThresholds` C — `pinneapple_analysis.uncertainty.drift` (v05)
- `drill_assembly` f — `pinneapple_design.geometry.vehicle_cad` (v06)
- `Drive` C — `pinneapple_simulation.electromagnetics.machines.drive` (v03)
- `drone` f — `pinneapple_twin3d.gallery` (v06)
- `drop_degenerate` f — `pinneapple_design.geometry_library.entries._adapt` (v06)
- `drop_results` f — `pinneapple_data.preflight` (v04)
- `Drude` C — `pinneapple_simulation.electromagnetics.media` (v03)
- `drude_metal` f — `pinneapple_simulation.electromagnetics.media` (v03)
- `drug_diffusion_tissue` f — `pinneapple_physics.pde_environment.presets.multidisciplinary` (v02)
- `DrydenFilter` C — `pinneapple_simulation.flight.turbulence` (v03)
- `ds_airfoil` f — `pinneapple_arena.external.demo_datasets` (v04)
- `ds_burgers` f — `pinneapple_arena.external.demo_datasets` (v04)
- `ds_combustion` f — `pinneapple_arena.external.demo_datasets` (v04)
- `ds_cosmology` f — `pinneapple_arena.external.demo_datasets` (v04)
- `ds_crystal` f — `pinneapple_arena.external.demo_datasets` (v04)
- `ds_em_grid` f — `pinneapple_arena.external.demo_datasets` (v04)
- `ds_epidemic` f — `pinneapple_arena.external.demo_datasets` (v04)
- `ds_graph_mesh` f — `pinneapple_arena.external.demo_datasets` (v04)
- `ds_grid_2d` f — `pinneapple_arena.external.demo_datasets` (v04)
- `ds_image_field` f — `pinneapple_arena.external.demo_datasets` (v04)
- `ds_lorenz` f — `pinneapple_arena.external.demo_datasets` (v04)
- `ds_molecular` f — `pinneapple_arena.external.demo_datasets` (v04)
- `ds_option_pricing` f — `pinneapple_arena.external.demo_datasets` (v04)
- `ds_pde_1d` f — `pinneapple_arena.external.demo_datasets` (v04)
- `ds_pedestrians` f — `pinneapple_arena.external.demo_datasets` (v04)
- `ds_power_system` f — `pinneapple_arena.external.demo_datasets` (v04)
- `ds_quantum_circuit` f — `pinneapple_arena.external.demo_datasets` (v04)
- `ds_rigidbody` f — `pinneapple_arena.external.demo_datasets` (v04)
- `ds_room` f — `pinneapple_arena.external.demo_datasets` (v04)
- `ds_seismic` f — `pinneapple_arena.external.demo_datasets` (v04)
- `ds_spin_lattice` f — `pinneapple_arena.external.demo_datasets` (v04)
- `ds_timeseries` f — `pinneapple_arena.external.demo_datasets` (v04)
- `ds_tle` f — `pinneapple_arena.external.demo_datasets` (v04)
- `ds_traffic` f — `pinneapple_arena.external.demo_datasets` (v04)
- `ds_weather_grid` f — `pinneapple_arena.external.demo_datasets` (v04)
- `DSMLoss` C — `pinneapple_neural.architectures.pinns.physics_diffusion` (v04)
- `duarte_loss` f — `pinneapple_physics.blackhole.forecast` (v02)
- `duarte_regional_loss` f — `pinneapple_physics.blackhole.forecast` (v02)
- `DuarteUNet` C — `pinneapple_physics.blackhole.forecast` (v02)
- `dump_yaml` f — `pinneapple_tools.benchmark_suite.io.yamlx` (v07)
- `dumps` f — `pinneapple_decision.schema` (v05)
- `dumps` f — `pinneapple_tools.model_services.registry` (v07)
- `dut1` f — `pinneapple_physics.astro.timescales` (v02)
- `Duty` C — `pinneapple_tools.nvidia.heatsink` (v07)

- duty_point f — pinneapple_physics.closed_form.pump_npsh (v02)
- DynamicModeDecomposition C — pinneapple_neural.architectures.rom.dmd (v04)
- DynamicsAdapter C — pinneapple_neural.architectures.adapters.dynamics (v04)
- DynamicSmagorinsky C — pinneapple_physics.multiscale.closures (v02)

E

- e1_cfd f — pinneapple_condense.falsify (v02)
- ear_clip f — pinneapple_design.geometry_library.kit (v06)
- ear_clip_fast f — pinneapple_design.geometry_library.kit (v06)
- EarlyStopping C — pinneapple_neural.trainer.callbacks (v04)
- earth2_gif f — pinneapple_physics.weather.viz (v02)
- earth_system f — pinneapple_flow.nodes_ext.core_ts (v07)
- EarthEngineUnavailable C — pinneapple_perception.sentinel2 (v04)
- EasyArenaProblem C — pinneapple_arena.custom_builder (v04)
- ece f — pinneapple_systems.sports.football.metrics (v06)
- EchoStateNetwork C — pinneapple_neural.architectures.reservoir_computing.esn (v04)
- ecliptic_matrix f — pinneapple_physics.astro.frames (v02)
- economic_thickness f — pinneapple_physics.closed_form.pipe_insulation (v02)
- eddy_current f — pinneapple_flow.nodes_ext.geomechanics (v07)
- eddy_current_density f — pinneapple_simulation.numerical_solvers.eddy_current_fdm (v03)
- EddyCurrentFDMSolver C — pinneapple_simulation.numerical_solvers.eddy_current_fdm (v03)
- EddyCurrentInspection C — pinneapple_presets.cases.eddy_current (v02)
- EddyViscosity C — pinneapple_analysis.benchmark.builtin.airfoil_impl (v05)
- Edge C — pinneapple_flow.schema (v07)
- Edge C — pinneapple_physics.physics_audit.knowledge (v02)
- edge_features f — pinneapple_design.aero.surrogate (v06)
- edge_features f — pinneapple_neural.architectures.graphnn.mgn_dynamics (v04)
- edge_stats f — pinneapple_design.geometry_library.meshops (v06)
- EdgeRuntime C — pinneapple_systems.component_modeling.edge (v06)
- edit_file f — pinneapple_simulation.external_solvers.openfoam.campaign (v03)
- edits_to_decisions f — pinneapple_flow.decisions (v07)
- EDMNoiseScheduler C — pinneapple_neural.architectures.pinns.physics_diffusion (v04)
- ee_initialize f — pinneapple_perception.sentinel2 (v04)
- eemd_1d f — pinneapple_simulation.numerical_solvers.eemd (v03)
- EEMDSolver C — pinneapple_simulation.numerical_solvers.eemd (v03)
- effect_curves f — pinneapple_analysis.sensitivity.effects (v05)
- effective_conductivity f — pinneapple_physics.closed_form.pcb_thermal (v02)
- effective_cv f — pinneapple_systems.process_components.control_valve (v06)
- effective_layer f — pinneapple_condense.homogenize (v02)
- effective_properties f — pinneapple_condense.homogenize (v02)
- effective_sample_count f — pinneapple_data.audit (v04)
- effective_viscosity f — pinneapple_systems.process_components.non_newtonian_pipe_flow (v06)
- effectiveness f — pinneapple_physics.closed_form.heat_exchanger (v02)
- efficiency_figure f — pinneapple_systems.sports.football.plots (v06)
- eht_comparison f — pinneapple_physics.astro.lensing (v02)
- eigenvector_overlap f — pinneapple_physics.non_hermitian.pt_dimer (v02)
- einstein_radius_rad f — pinneapple_physics.astro.lensing (v02)
- EKIConfig C — pinneapple_analysis.inverse_problems.ensemble_kalman (v05)
- EKIHistory C — pinneapple_analysis.inverse_problems.ensemble_kalman (v05)
- Elastic C — pinneapple_simulation.particle_dynamics.geo_mpm (v03)
- Elasticity3DFDMSolver C — pinneapple_simulation.numerical_solvers.elasticity3d_fdm (v03)
- elasticity_ke f — pinneapple_design.topology.problems (v06)
- elasticity_matrix f — pinneapple_simulation.numerical_solvers.solid_fem (v03)
- ElasticWave3D C — pinneapple_simulation.numerical_solvers.fdm3d (v03)
- ElasticWaveConfig3D C — pinneapple_simulation.numerical_solvers.fdm3d (v03)
- elbow_streamlines f — pinneapple_systems.digital_twin.pipe_loop.scene (v06)

- ElectrostaticSolution C — pinneapple_simulation.electromagnetics.electrostatics (v03)
- electrothermal f — pinneapple_reason.modes.thermal (v05)
- element_adjacency_by_nodes f — pinneapple_data.cae.geometry (v04)
- element_flux_density_history f — pinneapple_simulation.electromagnetics.postprocess (v03)
- element_label f — pinneapple_reason.chain (v05)
- element_metrics f — pinneapple_data.cae.quality (v04)
- elements_from_state f — pinneapple_physics.astro.orbits (v02)
- elf_exported_functions f — pinneapple_simulation.external_solvers.modelica.fmu_inspect (v03)
- ellipse f — pinneapple_design.geometry.gen.sdf_shapes (v06)
- ellipsoid f — pinneapple_design.geometry_library.kit (v06)
- elm_fit f — pinneapple_lab.experiments.discovery (v07)
- elmer_case f — pinneapple_simulation.electromagnetics.machines.interop (v03)
- em_cylinder_scattering f — pinneapple_physics.pde_environment.presets.electromagnetics (v02)
- em_residuals f — pinneapple_physics.pinn_solver.compiler.em_kinds (v02)
- em_skin_effect f — pinneapple_physics.pde_environment.presets.electromagnetics (v02)
- em_tm_cavity f — pinneapple_physics.pde_environment.presets.electromagnetics (v02)
- em_tm_cavity_integral f — pinneapple_physics.pde_environment.presets.electromagnetics (v02)
- em_two_dielectric_capacitor f — pinneapple_physics.pde_environment.presets.electromagnetics (v02)
- em_two_dielectric_capacitor_differential f — pinneapple_physics.pde_environment.presets.electromagnetics (v02)
- em_verification f — pinneapple_flow.nodes_ext.electromagnetics (v07)
- em_wire_az f — pinneapple_physics.pde_environment.presets.electromagnetics (v02)
- embedding_views f — pinneapple_systems.time_series.forecast_lab.renders (v06)
- emd_1d f — pinneapple_simulation.numerical_solvers.eemd (v03)
- emit f — pinneapple_veriphysics.execution_log (v05)
- EMPinn C — pinneapple_lab.experiments.em_pinn (v07)
- enable_features f — pinneapple_simulation.external_solvers.fluidx3d (v03)
- enable_gradient_checkpointing f — pinneapple_neural.trainer.parallel (v04)
- encode f — pinneapple_physics.blackhole.dataset (v02)
- encode_mp4 f — pinneapple_tools.visualization.molecular (v07)
- encode_mp4 f — pinneapple_tools.visualization.studio.particles (v07)
- encode_video f — pinneapple_systems.time_series.enso.render (v06)
- EncoderLayer C — pinneapple_neural.architectures.transformers.autoformer (v04)
- encounter_bars_figure f — pinneapple_simulation.flight.figures (v03)
- encounter_feasibility f — pinneapple_analysis.uncertainty.reachability (v05)
- encounter_metrics f — pinneapple_simulation.flight.encounter (v03)
- encounter_traces_figure f — pinneapple_simulation.flight.figures (v03)
- encrypt_array f — pinneapple_security.encryption (v05)
- encrypt_bytes f — pinneapple_security.encryption (v05)
- encrypt_file f — pinneapple_security.encryption (v05)
- end_effects f — pinneapple_flow.nodes_ext.em_prius (v07)
- end_leakage_3d f — pinneapple_simulation.electromagnetics.machines.getdp (v03)
- end_winding_inductance f — pinneapple_simulation.electromagnetics.machines.endwinding (v03)
- end_winding_paths f — pinneapple_simulation.electromagnetics.machines.endwinding (v03)
- endfire_directivity f — pinneapple_simulation.electromagnetics.antenna.bounds (v03)
- endfire_directivity_dbi f — pinneapple_simulation.electromagnetics.antenna.bounds (v03)
- energy f — pinneapple_core.loss (v01)
- energy f — pinneapple_physics.astro.nbody (v02)
- energy_balance f — pinneapple_simulation.multibody.arm (v03)
- energy_balance_residual f — pinneapple_systems.operations.plant (v06)
- energy_check f — pinneapple_simulation.flight.simulate (v03)
- energy_correct f — pinneapple_condense.exceptions (v02)
- energy_error_bound f — pinneapple_analysis.uncertainty.residual_bound (v05)
- energy_loss f — pinneapple_quantum.loss.schrodinger (v04)
- energy_norm f — pinneapple_condense.substructure (v02)
- energy_spectrum f — pinneapple_analysis.validation.spectral (v05)

- energy_spectrum f — pinneapple_physics.multiscale.ns2d (v02)
- Engine C — pinneapple_design.aero.optimize (v06)
- Engine C — pinneapple_query.core (v02)
- Engine3D C — pinneapple_design.aero.optimize3d (v06)
- EngineAL C — pinneapple_design.aero.optimize_airliner (v06)
- EnhancerConfig C — pinneapple_worldmodel.photorealistic_enhancer (v04)
- enkf_analysis f — pinneapple_analysis.state_estimation.kalman (v05)
- EnKFForecaster C — pinneapple_systems.time_series.forecast_lab.physical (v06)
- ensemble_fibres f — pinneapple_systems.time_series.forecast_lab.renders (v06)
- ensemble_forward f — pinneapple_physics.pinn_solver.stochastic (v02)
- ensemble_interval_coverage f — pinneapple_systems.time_series.metrics_ext.probabilistic (v06)
- ensemble_spread f — pinneapple_lab.uq (v07)
- ensemble_study f — pinneapple_veriphysics.robustness (v05)
- EnsembleConfig C — pinneapple_analysis.uncertainty.ensemble (v05)
- EnsembleConfig C — pinneapple_physics.multiscale.comparison (v02)
- EnsembleForecaster C — pinneapple_systems.time_series.enso.forecasters (v06)
- EnsembleKalmanFilter C — pinneapple_analysis.state_estimation.kalman (v05)
- EnsembleKalmanFilter C — pinneapple_neural.architectures.classical_ts.enkf (v04)
- EnsembleKalmanInversion C — pinneapple_analysis.inverse_problems.ensemble_kalman (v05)
- EnsembleModel C — pinneapple_worldmodel.model_zoo (v04)
- EnsembleRun C — pinneapple_physics.ensemble (v02)
- EnsembleUQ C — pinneapple_analysis.uncertainty.ensemble (v05)
- enso f — pinneapple_flow.nodes.systems (v07)
- enso_scores f — pinneapple_systems.time_series.enso.pipeline (v06)
- enso_solver_checks f — pinneapple_flow.nodes_ext.forecasting (v07)
- EnsoDataset C — pinneapple_systems.time_series.enso.dataset (v06)
- ensure_dir f — pinneapple_tools.hpo_experiments.kb.store (v07)
- ensure_float_dict f — pinneapple_tools.benchmark_suite.runner.metrics (v07)
- ensure_image f — pinneapple_simulation.external_solvers.calculix.runner (v03)
- ensure_model_pulled f — pinneapple_llm.local_llm (v07)
- ensure_point_set f — pinneapple_tools.benchmark_suite.crossframework.protocol (v07)
- ensure_requires_grad f — pinneapple_physics.pinn_solver.factory.autodiff (v02)
- ensure_tensor f — pinneapple_physics.pinn_solver.compiler.autograd_ops (v02)
- ensure_twin_model f — pinneapple_systems.digital_twin.product.model (v06)
- EntitySpec C — pinneapple_simulation.external_solvers.genesis.scene_builder (v03)
- entry f — pinneapple_flow.palette (v07)
- entry f — pinneapple_hub.zoo._artifacts (v07)
- env_fingerprint f — pinneapple_neural.trainer.audit (v04)
- Envelope C — pinneapple_physics.physics_audit.assumptions (v02)
- envelope f — pinneapple_physics.astro.decay (v02)
- envelope_check f — pinneapple_physics.physics_audit.monitor (v02)
- envelope_checks f — pinneapple_analysis.benchmark.validation (v05)
- envelope_cls f — pinneapple_check._reuse (v05)
- Environment C — pinneapple_physics.closed_form.pcb_thermal (v02)
- environment f — pinneapple_autolab.ledger (v07)
- environment f — pinneapple_presets.core (v02)
- environment f — pinneapple_tools.benchmark_suite.crossframework.records (v07)
- environment f — pinneapple_tools.nvidia.runinfo (v07)
- environment_snapshot f — pinneapple_llm.qa.report (v07)
- eof_fill f — pinneapple_systems.time_series.enso.dataset (v06)
- ep_figure f — pinneapple_flow.nodes_ext.non_hermitian (v07)
- ep_laws f — pinneapple_flow.nodes_ext.non_hermitian (v07)
- ep_numeric f — pinneapple_flow.nodes_ext.non_hermitian (v07)
- ep_tuning_figure f — pinneapple_physics.non_hermitian.viz (v02)
- Ephemeris C — pinneapple_physics.astro.ephemeris (v02)
- episode_metrics f — pinneapple_security.cps (v05)

- EPTuningReport C — pinneapple_physics.non_hermitian.ep_tuning (v02)
- eq f — pinneapple_core.optim (v01)
- equal_budget f — pinneapple_flow.nodes_ext.benchmarks (v07)
- equal_budget f — pinneapple_simulation.electromagnetics.periodic.benchmark (v03)
- equal_budget_search f — pinneapple_analysis.cost.equal_budget (v05)
- equal_flops_table f — pinneapple_analysis.benchmark.report (v05)
- Equation C — pinneapple_check.physics (v05)
- Equation C — pinneapple_physics.symbolic_pde.text (v02)
- equations f — pinneapple_check.physics (v05)
- equations f — pinneapple_flow.nodes.problem (v07)
- EquationSpec C — pinneapple_app.backend.core.problem (v07)
- EquationSyntaxError C — pinneapple_physics.symbolic_pde.text (v02)
- EquationSystem C — pinneapple_physics.symbolic_pde.text (v02)
- equatorial_to_horizontal f — pinneapple_physics.astro.frames (v02)
- EquatorialWavePINN C — pinneapple_systems.time_series.enso.pinn (v06)
- equilibrium_residual_rz f — pinneapple_physics.solid.axisymmetric (v02)
- EquivariantGNN C — pinneapple_neural.architectures.graphnn.equivariant_gnn (v04)
- era f — pinneapple_physics.astro.timescales (v02)
- era5_monthly f — pinneapple_systems.time_series.enso.data (v06)
- Era5Store C — pinneapple_physics.weather.data (v02)
- error_by_region f — pinneapple_analysis.validation.error_map (v05)
- error_hotspots f — pinneapple_analysis.validation.error_map (v05)
- error_map f — pinneapple_analysis.validation.error_map (v05)
- error_section f — pinneapple_app.backend.core.surrogate_report (v07)
- ErrorMap C — pinneapple_analysis.validation.error_map (v05)
- errors f — pinneapple_condense.substructure (v02)
- errors f — pinneapple_design.thermal.board_cooling.learn (v06)
- ESNRC C — pinneapple_neural.architectures.reservoir_computing.esn_rc (v04)
- estimate f — pinneapple_condense.hierarchy (v02)
- estimate_bounding_circle f — pinneapple_perception.image_geometry (v04)
- estimate_channel_scales f — pinneapple_neural.trainer.collocation (v04)
- estimate_mass f — pinneapple_design.bricks.parts (v06)
- estimate_recovery f — pinneapple_twin3d.wear.calibrate (v06)
- estimate_required_first_cell_height f — pinneapple_analysis.verification.mesh_intelligence (v05)
- estimate_rul f — pinneapple_systems.digital_twin.prognostics (v06)
- estimate_rul_from_twin f — pinneapple_systems.digital_twin.prognostics (v06)
- estimated_error f — pinneapple_physics.darcy_checks (v02)
- estimated_flux_error f — pinneapple_physics.darcy_checks (v02)
- ETDRK4 C — pinneapple_simulation.numerical_solvers.etdrk4 (v03)
- ETDRK4Result C — pinneapple_simulation.numerical_solvers.etdrk4 (v03)
- euler_characteristic f — pinneapple_design.geometry.ops.curvature (v06)
- euler_critical_buckling_load f — pinneapple_systems.process_components.pipe_stress_mechanics (v06)
- euler_to_quat f — pinneapple_simulation.flight.rigid_body (v03)
- eval_act f — pinneapple_simulation.numerical_solvers.thermal_channel_datasets (v03)
- eval_axes f — pinneapple_flow.nodes._common (v07)
- eval_rollout f — pinneapple_neural.architectures.graphnn.mgn_dynamics (v04)
- eval_splits f — pinneapple_analysis.benchmark.builtin.airfoil_impl (v05)
- evaluate f — pinneapple_analysis.reproduction (v05)
- evaluate f — pinneapple_analysis.surrogate.study (v05)
- evaluate f — pinneapple_condense.meshless (v02)
- evaluate f — pinneapple_data.expressions (v04)
- evaluate f — pinneapple_design.aero.aircraft (v06)
- evaluate f — pinneapple_hub.zoo (v07)
- evaluate f — pinneapple_hub.zoo.checks (v07)
- evaluate f — pinneapple_physics.astro.neural_propagator (v02)
- evaluate f — pinneapple_physics.closed_form.plate_fin_heatsink (v02)

- evaluate f — pinneapple_physics.pcb.engine (v02)
- evaluate f — pinneapple_simulation.electromagnetics.reflector.compact_range (v03)
- evaluate f — pinneapple_simulation.external_solvers.openfoam.campaign (v03)
- evaluate f — pinneapple_simulation.pipelines.freecad_openfoam.pipeline (v03)
- evaluate f — pinneapple_systems.process_components.explicit_equation_system (v06)
- evaluate f — pinneapple_systems.time_series.forecast_lab.study (v06)
- evaluate_criterion f — pinneapple_tools.automation.verify (v07)
- evaluate_design f — pinneapple_design.thermal.heatsink.engine (v06)
- evaluate_design f — pinneapple_tools.nvidia.heatsink (v07)
- evaluate_experts f — pinneapple_physics.multiscale.comparison (v02)
- evaluate_expr f — pinneapple_check.checks.results (v05)
- evaluate_inverse f — pinneapple_tools.benchmark_suite.crossframework.evaluate (v07)
- evaluate_level f — pinneapple_llm.qa.levels (v07)
- evaluate_loss f — pinneapple_physics.blackhole.forecast (v02)
- evaluate_map f — pinneapple_systems.process_components.similarity_map (v06)
- evaluate_model f — pinneapple_arena.trainer (v04)
- evaluate_operator f — pinneapple_tools.benchmark_suite.crossframework.evaluate (v07)
- evaluate_part f — pinneapple_tools.nvidia.brep (v07)
- evaluate_pinn f — pinneapple_tools.benchmark_suite.crossframework.evaluate (v07)
- evaluate_posterior f — pinneapple_analysis.uncertainty.posterior_metrics (v05)
- evaluate_skill f — pinneapple_physics.blackhole.skill (v02)
- Evaluation C — pinneapple_design.bricks.goals (v06)
- Evaluation C — pinneapple_design.qualitative.models (v06)
- evaluation_order f — pinneapple_systems.process_components.explicit_equation_system (v06)
- Evaluations C — pinneapple_design.design_optimizer.study (v06)
- Evaluator C — pinneapple_design.design_optimizer.study (v06)
- evasion_search f — pinneapple_analysis.validation.arena.red (v05)
- Event C — pinneapple_physics.weather.events (v02)
- Event C — pinneapple_systems.operations.anomaly (v06)
- event_hindcast f — pinneapple_systems.time_series.enso.study (v06)
- event_states f — pinneapple_physics.weather.events (v02)
- Evidence C — pinneapple_autolab.core (v07)
- Evidence C — pinneapple_decision.schema (v05)
- Evidence C — pinneapple_tools.automation.record (v07)
- evidence_line f — pinneapple_lab.showcase (v07)
- evidence_stages f — pinneapple_lab.curation (v07)
- evidence_summary f — pinneapple_analysis.verification.evidence_graph (v05)
- EvidenceEntry C — pinneapple_veriphysics.ledger (v05)
- EvidenceLedger C — pinneapple_veriphysics.ledger (v05)
- EvidenceStore C — pinneapple_decision.loop (v05)
- evolution_timeline f — pinneapple_design.bricks.visuals (v06)
- EvolutionaryDesignOptimizer C — pinneapple_design.design_optimizer.optimizer (v06)
- evolve f — pinneapple_design.bricks.search (v06)
- exact f — pinneapple_lab.experiments.ode (v07)
- exact_advection f — pinneapple_physics.multiscale.ns2d (v02)
- exact_eigenstates_1d_harmonic f — pinneapple_quantum.data.quantum_dataset (v04)
- exact_function f — pinneapple_flow.nodes._common (v07)
- ExactShapleyAttributor C — pinneapple_analysis.inverse_problems.counterfactual_attribution (v05)
- example f — pinneapple_hub.library.snippets (v07)
- example_chiller_plant f — pinneapple_data.data_health (v04)
- example_dir f — pinneapple_flow.nodes_ext.examples (v07)
- example_dirs f — pinneapple_lab.manifest (v07)
- example_folders f — pinneapple_flow.examples (v07)
- example_json f — pinneapple_hub.zoo.hub_cards (v07)
- example_plant_operations f — pinneapple_data.process_optimizer (v04)
- example_sources f — pinneapple_data.unified (v04)

- example_workflow f — pinneapple_tools.automation (v07)
- ExampleCase C — pinneapple_veriphysics.example_cases (v05)
- examples f — pinneapple_hub.library.sources (v07)
- examples_root f — pinneapple_flow.nodes_ext.examples (v07)
- ExampleScript C — pinneapple_lab.experiments.examples (v07)
- exception_patches f — pinneapple_condense.falsify (v02)
- exceptions_study f — pinneapple_condense.benchmark (v02)
- exchanger_model f — pinneapple_systems.operations.exchanger (v06)
- ExecResult C — pinneapple_core.workflows.executors (v01)
- execute f — pinneapple_autolab.core (v07)
- execute f — pinneapple_decision.api (v05)
- execute_jobs f — pinneapple_analysis.benchmark.runner (v05)
- ExecuteNode C — pinneapple_decision.tree (v05)
- ExecutionResult C — pinneapple_decision.schema (v05)
- Executor C — pinneapple_core.workflows.executors (v01)
- Executor C — pinneapple_decision.loop (v05)
- exhaustive f — pinneapple_analysis.proof.graph (v05)
- exodus_to_upd f — pinneapple_simulation.external_solvers.cfd_formats.exodus_reader (v03)
- expand f — pinneapple_data.cae.upload (v04)
- expand_target f — pinneapple_simulation.external_solvers.calculix.inp (v03)
- ExpandingWindowSplitter C — pinneapple_systems.time_series.validation.splitters (v06)
- expected_exponent f — pinneapple_analysis.cost.profiler (v05)
- Experiment C — pinneapple_lab.spec (v07)
- Experiment C — pinneapple_physics.experiment (v02)
- Experiment C — pinneapple_science.core (v02)
- experiment f — pinneapple_lab.capture (v07)
- experiment_ws f — pinneapple_app.backend.routers.experiments (v07)
- ExperimentConfig C — pinneapple_app.backend.core.experiment (v07)
- ExperimentMemory C — pinneapple_registry.experiment_memory (v01)
- ExperimentRecord C — pinneapple_registry.experiment_memory (v01)
- ExperimentRequest C — pinneapple_app.backend.routers.experiments (v07)
- ExperimentResult C — pinneapple_app.backend.core.experiment (v07)
- ExperimentResult C — pinneapple_physics.experiment (v02)
- ExperimentRunner C — pinneapple_app.backend.core.experiment (v07)
- experiments f — pinneapple_science.registry (v02)
- ExperimentStatusResponse C — pinneapple_app.backend.routers.experiments (v07)
- ExperimentStore C — pinneapple_registry.experiment_store (v01)
- Expert C — pinneapple_physics.ensemble (v02)
- explain f — pinneapple_lab.next_steps (v07)
- explain f — pinneapple_reason.narrative (v05)
- explain f — pinneapple_twin3d.wear.optimize (v06)
- explain_preset f — pinneapple_analysis.verification.knowledge_graph (v05)
- explain_trust f — pinneapple_analysis.verification.evidence_graph (v05)
- ExplicitAirframe C — pinneapple_simulation.flight.catalog (v03)
- ExplicitEquationError C — pinneapple_systems.process_components.explicit_equation_system (v06)
- exponential_density f — pinneapple_physics.astro.decay (v02)
- ExponentialSmoothing C — pinneapple_systems.time_series.forecast_lab.baselines (v06)
- export f — pinneapple_data.cae.dataset (v04)
- export f — pinneapple_design.geometry_library (v06)
- export f — pinneapple_design.geometry_library.exporters (v06)
- export_all f — pinneapple_twin3d.gallery (v06)
- export_bundle f — pinneapple_hub.zoo.web (v07)
- export_bundle f — pinneapple_simulation.external_solvers.openfoam.export_bundle (v03)
- export_bytes f — pinneapple_hub.zoo.export (v07)
- export_cad f — pinneapple_design.intent.physics.bracket (v06)
- export_cadquery f — pinneapple_design.geometry.gen.cadquery_gen (v06)

- export_csv f — pinneapple_tools.model_export (v07)
- export_dataset f — pinneapple_llm.workbench.finetune (v07)
- export_edge_package f — pinneapple_systems.component_modeling.edge (v06)
- export_model f — pinneapple_tools (v07)
- export_npz f — pinneapple_tools.model_export (v07)
- export_odb_fields f — pinneapple_simulation.external_solvers.cfd_formats.abaqus_reader (v03)
- export_onnx f — pinneapple_design.aero.geometry_onnx (v06)
- export_onnx f — pinneapple_hub.zoo (v07)
- export_onnx f — pinneapple_hub.zoo.export (v07)
- export_onnx f — pinneapple_neural.architectures.neural_operators.fno_export (v04)
- export_onnx f — pinneapple_tools.model_export (v07)
- export_predictions f — pinneapple_tools.benchmark_suite.crossframework.protocol (v07)
- export_problemdesign f — pinneapple_tools.hpo_experiments.pipelines.export_problemdesign (v07)
- export_recipe f — pinneapple_llm.cad_draft (v07)
- export_rover f — pinneapple_simulation.rover.geometry (v03)
- export_safetensors f — pinneapple_hub.zoo.export (v07)
- export_scene f — pinneapple_blender.export (v07)
- export_scene f — pinneapple_systems.digital_twin.product.runner (v06)
- export_schemas f — pinneapple_data.contracts.models (v04)
- export_space f — pinneapple_hub.zoo.demo (v07)
- export_stl_bytes f — pinneapple_design.surrogate_inverse.brake_rotor (v06)
- export_stl_bytes f — pinneapple_design.surrogate_inverse.heatsink_fins (v06)
- export_stl_bytes f — pinneapple_design.surrogate_inverse.reflex_port (v06)
- export_stl_bytes f — pinneapple_design.surrogate_inverse.suspension_arm (v06)
- export_stl_bytes f — pinneapple_design.surrogate_inverse.transdermal_patch (v06)
- export_to_lab f — pinneapple_flow.runtime (v07)
- export_torchscript f — pinneapple_tools.model_export (v07)
- export_trajectory f — pinneapple_blender.export (v07)
- export_twin f — pinneapple_llm.qa.examples.pipe_wear_twin (v07)
- export_usd f — pinneapple_twin3d.usd (v06)
- export_vtk f — pinneapple_tools.visualization.pyvista_bridge (v07)
- export_wear_twin f — pinneapple_twin3d.wear.build (v06)
- export_web f — pinneapple_design.geometry_library (v06)
- export_web f — pinneapple_design.geometry_library.web (v06)
- export_web f — pinneapple_hub.zoo (v07)
- export_web f — pinneapple_hub.zoo.web (v07)
- exported_functions_from_layout f — pinneapple_tools.rea_bridge.evidence (v07)
- ExpressionError C — pinneapple_data.expressions (v04)
- ExpressionModel C — pinneapple_analysis.inverse_problems.forward (v05)
- ExtendedKalmanFilter C — pinneapple_analysis.state_estimation.kalman (v05)
- ExtendedKalmanFilter C — pinneapple_neural.architectures.classical_ts.ekf (v04)
- ExternalFlow C — pinneapple_design.qualitative.models (v06)
- ExternalFlow C — pinneapple_simulation.numerical_solvers.external_flow (v03)
- ExternalTool C — pinneapple_analysis.verification.tool_recommendation (v05)
- extinction f — pinneapple_physics.astro.observing (v02)
- extinction_mag f — pinneapple_physics.astro.astrometry (v02)
- extract f — pinneapple_data.simulation_metadata (v04)
- extract f — pinneapple_lab.results_rules (v07)
- extract_1d_signal f — pinneapple_simulation.numerical_solvers.adapters.upd_signal (v03)
- extract_boundary_points f — pinneapple_perception.image_geometry (v04)
- extract_case f — pinneapple_data.simulation_metadata.openfoam (v04)
- extract_dominant_frequencies f — pinneapple_perception.audio_modal (v04)
- extract_fields f — pinneapple_simulation.numerical_solvers.openfoam_bridge (v03)
- extract_flights f — pinneapple_systems.sports.football.ball (v06)
- extract_from_text f — pinneapple_problemdesign.extractor (v02)
- extract_geometry_features f — pinneapple_analysis.verification.geometry_ood_guardrail (v05)

- extract_json f — pinneapple_tools.dataset_quality.vlm_curation (v07)
- extract_pdf_text f — pinneapple_tools.hpo_experiments.sources.arxiv_pdf (v07)
- extract_problem_solutions f — pinneapple_tools.hpo_experiments.pipelines.extract_problem_solutions (v07)
- extract_rules f — pinneapple_data.formfill.extract (v04)
- extract_rules f — pinneapple_data.udr1 (v04)
- extract_with_llm f — pinneapple_data.formfill.llm (v04)
- ExtractedProblemSolution C — pinneapple_tools.hpo_experiments.models (v07)
- ExtractLLM C — pinneapple_reason.llm (v05)
- ExtractorAgent C — pinneapple_tools.hpo_experiments.agents.extractor (v07)
- extrapolation_study f — pinneapple_veriphysics.robustness (v05)
- ExtremeLearningMachine C — pinneapple_neural.architectures.reservoir_computing.elm (v04)
- extrude f — pinneapple_design.geometry_library.kit (v06)
- extrude f — pinneapple_design.qualitative.descriptors (v06)
- extrude f — pinneapple_presets.viz3d (v02)
- extrude_polygon f — pinneapple_design.geometry.vehicle_cad (v06)
- extrude_profile f — pinneapple_tools.nvidia.omniverse (v07)
- Eyepiece C — pinneapple_physics.astro.observing (v02)

F

- f0_validation f — pinneapple_condense.falsify (v02)
- f1_advection f — pinneapple_condense.falsify (v02)
- f2_stokes f — pinneapple_condense.falsify (v02)
- f3_cht f — pinneapple_condense.falsify (v02)
- Face C — pinneapple_simulation.pipelines.freecad_openfoam.cad (v03)
- face_geometry f — pinneapple_data.cae.geometry (v04)
- face_geometry f — pinneapple_design.qualitative.descriptors (v06)
- face_mesh f — pinneapple_data.cae.geometry (v04)
- face_modes f — pinneapple_condense.algebraic (v02)
- face_normals f — pinneapple_design.geometry_library.meshops (v06)
- face_reynolds_number f — pinneapple_systems.process_components.pipe_network_1d (v06)
- face_weights f — pinneapple_condense.substructure (v02)
- FaceMesh C — pinneapple_data.cae.geometry (v04)
- faces f — pinneapple_condense.algebraic (v02)
- faces_for_tag f — pinneapple_app.backend.core.physics_adapter (v07)
- faces_of f — pinneapple_simulation.pipelines.freecad_openfoam.freecad_scripts.fc_job (v03)
- facets f — pinneapple_hub.library (v07)
- Fact C — pinneapple_design.fitmesh.context (v06)
- factor f — pinneapple_condense.fv (v02)
- factory_loss f — pinneapple_flow.nodes_ext.pinn_solver (v07)
- FactoryConfig C — pinneapple_worldmodel.dataset_factory (v04)
- FactoryResult C — pinneapple_worldmodel.synthetic_factory (v04)
- FactQuestion C — pinneapple_decision.selectors (v05)
- failed f — pinneapple_arena.external.result (v04)
- failure_mode f — pinneapple_flow.nodes.reason (v07)
- FailureMode C — pinneapple_design.intent.template (v06)
- FailureMode C — pinneapple_reason.library (v05)
- FailureRound C — pinneapple_design.bricks.statics (v06)
- FakeFreeCAD C — pinneapple_simulation.pipelines.freecad_openfoam.cad (v03)
- false_alarm_search f — pinneapple_analysis.validation.arena.red (v05)
- family_length_ok f — pinneapple_flow.planner.heuristic (v07)
- fan_cooler_cfd f — pinneapple_physics.pde_environment.presets.engineering (v02)
- fan_panels f — pinneapple_systems.time_series.forecast_lab.pipeline (v06)
- fan_positions f — pinneapple_design.thermal.board_cooling.scene (v06)
- far_surface f — pinneapple_simulation.vehicle.viz (v03)
- faraday f — pinneapple_physics.maxwell_checks (v02)

- faraday_rate f — pinneapple_reason.modes.corrosion (v05)
- FaradayInduction C — pinneapple_science.em.induction (v02)
- FarField C — pinneapple_simulation.electromagnetics.antenna.scene3d (v03)
- farfield_metrics f — pinneapple_simulation.electromagnetics.antenna.scene3d (v03)
- fast_non_dominated_sort f — pinneapple_design.design_optimizer.pareto (v06)
- fatigue_life_cycles f — pinneapple_design.surrogate_inverse.suspension_arm (v06)
- Fault C — pinneapple_analysis.validation.fault_injection.faults (v05)
- Fault C — pinneapple_systems.operations.plant (v06)
- fault_benchmark f — pinneapple_systems.operations.anomaly (v06)
- fault_rng f — pinneapple_analysis.validation.fault_injection.matrix (v05)
- FaultInjectionReport C — pinneapple_analysis.validation.fault_injection.matrix (v05)
- Faults C — pinneapple_systems.digital_twin.pipe_loop.plant (v06)
- fcc_slab f — pinneapple_tools.visualization.molecular (v07)
- fd_derivative f — pinneapple_neural.architectures.neural_operators.pino (v04)
- fd_nodes f — pinneapple_simulation.meshless.methods (v03)
- FDError C — pinneapple_simulation.numerical_solvers.symbolic_fd (v03)
- fdi_benchmark f — pinneapple_flow.nodes_ext.security (v07)
- FDMScriptGenerator C — pinneapple_physics.codegen.fdm_script_generator (v02)
- FDMSolver C — pinneapple_simulation.numerical_solvers.fdm (v03)
- FDOperator C — pinneapple_simulation.numerical_solvers.symbolic_fd (v03)
- FDResult C — pinneapple_simulation.numerical_solvers.symbolic_fd (v03)
- FDTD C — pinneapple_simulation.electromagnetics.fdtd (v03)
- FDTDConfig C — pinneapple_science.em.fdtd_study (v02)
- FDTDRun C — pinneapple_science.em.fdtd_study (v02)
- FDTDStudy3D C — pinneapple_science.em.research (v02)
- fea_figure f — pinneapple_simulation.numerical_solvers.solid_fem (v03)
- FEAResult C — pinneapple_design.bricks.fea (v06)
- feasibility_frequency f — pinneapple_simulation.electromagnetics.antenna.bounds (v03)
- feasibility_report f — pinneapple_design.design_optimizer.study (v06)
- feature_angles f — pinneapple_worldmodel.robot_perception (v04)
- featured f — pinneapple_hub.library (v07)
- FeatureHit C — pinneapple_tools.rea_bridge.evidence (v07)
- FeatureMap C — pinneapple_tools.rea_bridge.evidence (v07)
- features f — pinneapple_design.thermal.heatsink.surrogate (v06)
- features f — pinneapple_physics.astro.neural_propagator (v02)
- features_yaml f — pinneapple_hub.zoo.parquet (v07)
- featurize_mesh f — pinneapple_data.geometry_features (v04)
- FEDformer C — pinneapple_neural.architectures.transformers.fedformer (v04)
- feedback_coupling f — pinneapple_physics.non_hermitian.ep_tuning (v02)
- femm_lua f — pinneapple_simulation.electromagnetics.machines.interop (v03)
- FEModel C — pinneapple_simulation.external_solvers.calculix.study (v03)
- FEMResult C — pinneapple_simulation.numerical_solvers.solid_fem (v03)
- FEMSolver C — pinneapple_simulation.numerical_solvers.fem (v03)
- fenics_available f — pinneapple_simulation.numerical_solvers (v03)
- FEnicsBridge C — pinneapple_simulation.numerical_solvers.fenics_bridge (v03)
- FEniCSConfig C — pinneapple_simulation.external_solvers.fenics.solver (v03)
- FEniCSScriptGenerator C — pinneapple_physics.codegen.fenics_script_generator (v02)
- FEniCSWorkflow C — pinneapple_simulation.external_solvers.fenics.solver (v03)
- fetch f — pinneapple_catalog.resources (v01)
- fetch f — pinneapple_hub.zoo._artifacts (v07)
- fetch f — pinneapple_systems.sports.football.data (v06)
- fetch f — pinneapple_systems.time_series.enso.data (v06)
- fetch_cached f — pinneapple_data.datasets.download (v04)
- fetch_cloud_forecast f — pinneapple_physics.astro.conditions (v02)
- fetch_ensemble f — pinneapple_physics.astro.conditions (v02)
- fetch_iv f — pinneapple_systems.digital_twin.io.usgs (v06)

- fetch_literature_documents f — pinneapple_analysis.retrieval.corpus (v05)
- fetch_live_kp f — pinneapple_physics.astro.spaceweather (v02)
- fetch_parts_library f — pinneapple_design.bricks.ldraw (v06)
- fetch_pinned f — pinneapple_systems.time_series.datasets.public (v06)
- fetch_sentinel2_sequence f — pinneapple_perception.sentinel2 (v04)
- fetch_silso_daily f — pinneapple_systems.time_series.solar_cycle (v06)
- fetch_text f — pinneapple_llm.workbench.web (v07)
- fetch_tles f — pinneapple_physics.astro.satellites (v02)
- fetch_to_file f — pinneapple_systems.time_series.enso.data (v06)
- FFmpegMissing C — pinneapple_perception.piv_tools (v04)
- FFTDecomposer C — pinneapple_systems.time_series.decomposition.fft_lstm (v06)
- FFTForecaster C — pinneapple_systems.time_series.decomposition.fft_forecaster (v06)
- FFTLSTMPipeline C — pinneapple_systems.time_series.decomposition.fft_lstm (v06)
- FFTNNForecaster C — pinneapple_systems.time_series.decomposition.fft_nn (v06)
- FFTSolver C — pinneapple_simulation.numerical_solvers.fft (v03)
- Fidelity C — pinneapple_design.intent.template (v06)
- Field C — pinneapple_core.field (v01)
- Field C — pinneapple_data.formfill.spec (v04)
- field3d f — pinneapple_flow.nodes.viz (v07)
- field_3d f — pinneapple_design.thermal.heatsink.engine (v06)
- field_3d f — pinneapple_physics.pcb.engine (v02)
- field_checks f — pinneapple_analysis.trust.field_card (v05)
- field_colors f — pinneapple_tools.nvidia.omniverse (v07)
- field_columns f — pinneapple_analysis.surrogate.data (v05)
- field_difference f — pinneapple_simulation.accelerated_foam.case (v03)
- field_divergence f — pinneapple_simulation.numerical_solvers.immersed_boundary_fdm (v03)
- field_energy f — pinneapple_science.numerics (v02)
- field_error_metrics f — pinneapple_analysis.validation.field_metrics (v05)
- field_errors f — pinneapple_autolab.surrogates (v07)
- field_features f — pinneapple_analysis.benchmark.validation (v05)
- field_fit_slice f — pinneapple_arena.showcase (v04)
- field_map f — pinneapple_tools.visualization.electromagnetics (v07)
- field_metrics f — pinneapple_arena.pointwise.training (v04)
- field_metrics f — pinneapple_check._reuse (v05)
- field_movie f — pinneapple_tools.visualization.electromagnetics (v07)
- field_on f — pinneapple_presets.viz3d (v02)
- field_on_points f — pinneapple_simulation.electromagnetics.reflector.study_report (v03)
- field_panels f — pinneapple_tools.nvidia.figures (v07)
- field_range_issues f — pinneapple_tools.dataset_quality.checks (v07)
- field_rotation_rate f — pinneapple_physics.astro.observing (v02)
- field_sampler f — pinneapple_analysis.validation.metamorphic (v05)
- field_scalar f — pinneapple_data.cae.dataset (v04)
- field_scales f — pinneapple_app.backend.core.surrogate_report (v07)
- field_scales f — pinneapple_neural.architectures.rom.study (v04)
- field_scores f — pinneapple_analysis.benchmark.builtin.airfoil_impl (v05)
- field_slices_html f — pinneapple_tools.visualization.integrations.plotly_export (v07)
- field_validation f — pinneapple_analysis.benchmark.validation (v05)
- field_value f — pinneapple_data.cae.compare (v04)
- field_view f — pinneapple_tools.visualization.studio.fieldview (v07)
- FieldCheck C — pinneapple_analysis.trust.field_card (v05)
- FieldData C — pinneapple_analysis.benchmark.builtin.fields (v05)
- FieldData C — pinneapple_check.fields (v05)
- FieldErrorReport C — pinneapple_analysis.validation.field_metrics (v05)
- FieldMetrics C — pinneapple_systems.digital_twin.product.evidence (v06)
- FieldOperatorProblem C — pinneapple_analysis.benchmark.builtin.fields (v05)
- FieldProbe f — pinneapple_science.instruments (v02)

- fields_of f — pinneapple_data.cae.compare (v04)
- FieldSet C — pinneapple_physics.maxwell_checks (v02)
- FieldSnapshot C — pinneapple_data.adapters.splash_archive_adapter (v04)
- FieldSolves C — pinneapple_simulation.accelerated_foam.case (v03)
- FieldSpec C — pinneapple_systems.digital_twin.product.spec (v06)
- FieldStage C — pinneapple_tools.nvidia.omniverse (v07)
- FieldSurrogate C — pinneapple_autolab.surrogates (v07)
- FieldTrustCard C — pinneapple_analysis.trust.field_card (v05)
- FieldView C — pinneapple_tools.visualization.studio.fieldview (v07)
- fig_field_triptych f — pinneapple_autolab.report (v07)
- fig_learning f — pinneapple_autolab.report (v07)
- fig_models f — pinneapple_autolab.report (v07)
- fig_pareto f — pinneapple_autolab.report (v07)
- fig_trust_cards f — pinneapple_autolab.report (v07)
- Figure C — pinneapple_tools.reporting.datasheet (v07)
- figure f — pinneapple_science.plots (v02)
- figure_confusion f — pinneapple_systems.digital_twin.pipe_loop.report (v06)
- figure_evolution f — pinneapple_design.thermal.board_cooling.report (v06)
- figure_holdout f — pinneapple_design.thermal.board_cooling.report (v06)
- figure_maps f — pinneapple_design.thermal.board_cooling.report (v06)
- figure_request f — pinneapple_systems.digital_twin.pipe_loop.report (v06)
- figures f — pinneapple_analysis.benchmark.report (v05)
- filament_inductance f — pinneapple_simulation.electromagnetics.statics (v03)
- file_fingerprint f — pinneapple_analysis.retrieval.corpus (v05)
- file_name f — pinneapple_design.geometry_library.store (v06)
- file_record f — pinneapple_tools.automation.record (v07)
- FileRecord C — pinneapple_tools.automation.record (v07)
- FileRef C — pinneapple_data.contracts.models (v04)
- FileWatchStream C — pinneapple_systems.digital_twin.io.stream (v06)
- fill f — pinneapple_core.workflows.dag (v01)
- fill_form f — pinneapple_data.udr1 (v04)
- fill_fraction f — pinneapple_simulation.electromagnetics.fdtd (v03)
- fill_gaps f — pinneapple_physics.weather.viz (v02)
- fill_level f — pinneapple_twin3d.wear.presets (v06)
- fill_pdf f — pinneapple_data.formfill.fill (v04)
- fillet_edges f — pinneapple_design.geometry.ops.occ_features (v06)
- FiLMLayer C — pinneapple_neural.architectures.pinns.physics_diffusion (v04)
- Filter C — pinneapple_physics.multiscale.filters (v02)
- filter_curated f — pinneapple_tools.dataset_quality.vlm_curation (v07)
- filter_entries f — pinneapple_lab.explorer (v07)
- filter_nan_rows f — pinneapple_neural.trainer.collocation (v04)
- filter_supported_kwargs f — pinneapple_neural.architectures.instantiate (v04)
- filter_variable_width f — pinneapple_physics.multiscale.filters (v02)
- filtering_consistency_loss f — pinneapple_physics.multiscale.learned (v02)
- Fin C — pinneapple_simulation.flight.aero (v03)
- fin_array f — pinneapple_design.surrogate_inverse.meshes (v06)
- fin_array_base_temperature_c f — pinneapple_physics.closed_form.fin_array_conduction (v02)
- fin_array_profile_params f — pinneapple_physics.closed_form.fin_array_conduction (v02)
- fin_array_temperature_at_position_c f — pinneapple_physics.closed_form.fin_array_conduction (v02)
- fin_cell_check f — pinneapple_design.surrogate_inverse.heatsink_fins (v06)
- fin_exact f — pinneapple_analysis.inverse_problems.heat_transfer (v05)
- fin_plate f — pinneapple_simulation.flight.mass (v03)
- fin_q_exact f — pinneapple_analysis.inverse_problems.heat_transfer (v05)
- fin_synthetic_readings f — pinneapple_analysis.inverse_problems.heat_transfer (v05)
- fin_temperature_at_height_c f — pinneapple_design.surrogate_inverse.heatsink_fins (v06)
- FinCellResult C — pinneapple_design.thermal.fin_cell (v06)

- find f — pinneapple_design.intent.materials (v06)
- find_bashrc f — pinneapple_simulation.accelerated_foam.case (v03)
- find_case f — pinneapple_analysis.benchmark.builtin.airfoil_impl (v05)
- find_chrome f — pinneapple_tools.reporting.datasheet (v07)
- find_config f — pinneapple_check.config (v05)
- find_freecadcmd f — pinneapple_simulation.pipelines.freecad_openfoam.cad (v03)
- find_kernel f — pinneapple_physics.astro.ephemeris (v02)
- find_minima f — pinneapple_physics.astro.phenomena (v02)
- find_passes f — pinneapple_physics.astro.satellites (v02)
- find_pii f — pinneapple_security.privacy (v05)
- find_plugin f — pinneapple_simulation.accelerated_foam.build (v03)
- find_presets_for_phenomenon f — pinneapple_analysis.verification.knowledge_graph (v05)
- find_quantities f — pinneapple_design.intent.parse (v06)
- find_result f — pinneapple_tools.visualization.studio.solver_view (v07)
- find_root f — pinneapple_data.simulation_metadata.openfoam (v04)
- find_roots f — pinneapple_physics.astro.phenomena (v02)
- find_verified_equations f — pinneapple_analysis.verification.knowledge_graph (v05)
- Finding C — pinneapple_analysis.validation.arena.red (v05)
- Finding C — pinneapple_analysis.validation.physics_learning (v05)
- Finding C — pinneapple_data.preflight.common (v04)
- Finding C — pinneapple_llm.qa.report (v07)
- Finding C — pinneapple_reason.chain (v05)
- Finding C — pinneapple_security.privacy (v05)
- Finding C — pinneapple_tools.rea_bridge.harvest (v07)
- fine_tune f — pinneapple_adaptation (v04)
- finetune f — pinneapple_flow.nodes.surrogates (v07)
- finetune f — pinneapple_hub.zoo (v07)
- finetune_brax f — pinneapple_arena.external.finetune (v04)
- finetune_ceviche f — pinneapple_arena.external.finetune (v04)
- finetune_chgnet f — pinneapple_arena.external.finetune (v04)
- finetune_deepmd_kit f — pinneapple_arena.external.finetune (v04)
- finetune_deepwave f — pinneapple_arena.external.finetune (v04)
- finetune_deepxde f — pinneapple_arena.external.finetune (v04)
- finetune_devito f — pinneapple_arena.external.finetune (v04)
- finetune_diffrax f — pinneapple_arena.external.finetune (v04)
- finetune_dsgp4 f — pinneapple_arena.external.finetune (v04)
- finetune_jax_cfd f — pinneapple_arena.external.finetune (v04)
- finetune_jax_md f — pinneapple_arena.external.finetune (v04)
- finetune_lora f — pinneapple_llm.finetune (v07)
- finetune_mace f — pinneapple_arena.external.finetune (v04)
- finetune_ml4gw f — pinneapple_arena.external.finetune (v04)
- finetune_mujoco_mjx f — pinneapple_arena.external.finetune (v04)
- finetune_nequip f — pinneapple_arena.external.finetune (v04)
- finetune_netket f — pinneapple_arena.external.finetune (v04)
- finetune_neuralfoil f — pinneapple_arena.external.finetune (v04)
- finetune_neuralgcm f — pinneapple_arena.external.finetune (v04)
- finetune_neuraloperator f — pinneapple_arena.external.finetune (v04)
- finetune_phiflow f — pinneapple_arena.external.finetune (v04)
- finetune_photontorch f — pinneapple_arena.external.finetune (v04)
- finetune_physicsnemo f — pinneapple_arena.external.finetune (v04)
- finetune_pina f — pinneapple_arena.external.finetune (v04)
- finetune_pinnicle f — pinneapple_arena.external.finetune (v04)
- finetune_pybamm f — pinneapple_arena.external.finetune (v04)
- finetune_pyroomacoustics f — pinneapple_arena.external.finetune (v04)
- finetune_pysindy f — pinneapple_arena.external.finetune (v04)
- finetune_reservoirflow f — pinneapple_arena.external.finetune (v04)

- finetune_schnetpack f — pinneapple_arena.external.finetune (v04)
- finetune_sciann f — pinneapple_arena.external.finetune (v04)
- finetune_socialforce f — pinneapple_arena.external.finetune (v04)
- finetune_taichi f — pinneapple_arena.external.finetune (v04)
- finetune_tensorcircuit_ng f — pinneapple_arena.external.finetune (v04)
- finetune_torax f — pinneapple_arena.external.finetune (v04)
- finetune_warp f — pinneapple_arena.external.finetune (v04)
- FinetuneConfig C — pinneapple_llm.finetune (v07)
- FinetuneRejectedError C — pinneapple_hub.zoo (v07)
- FinetuneResult C — pinneapple_hub.zoo (v07)
- finite f — pinneapple_analysis.benchmark.builtin.fields (v05)
- finite_difference_check f — pinneapple_design.topology.driver (v06)
- finite_differences f — pinneapple_systems.sports.football.kinematics (v06)
- finite_solenoid_on_axis_B f — pinneapple_simulation.electromagnetics.statics (v03)
- first_acf_zero f — pinneapple_systems.time_series.predictability (v06)
- first_cell_height f — pinneapple_physics.closed_form.pipe_flow (v02)
- first_crossing f — pinneapple_physics.weather.evaluate (v02)
- first_order_given_data f — pinneapple_analysis.sensitivity.sobol (v05)
- fisher_identifiability f — pinneapple_systems.sports.football.ball (v06)
- FisherKPP1D C — pinneapple_arena.problems (v04)
- fit f — pinneapple_flow.learn (v07)
- fit f — pinneapple_hub.zoo._models (v07)
- fit f — pinneapple_neural.lpm (v04)
- fit_and_optimize f — pinneapple_data.process_optimizer (v04)
- fit_ar f — pinneapple_systems.time_series.predictability (v06)
- fit_ball_travel f — pinneapple_systems.sports.football.pitch_control (v06)
- fit_candidate f — pinneapple_analysis.surrogate.methods (v05)
- fit_controlled_opinf f — pinneapple_systems.operations.rom (v06)
- fit_crop f — pinneapple_lab.showcase (v07)
- fit_cst f — pinneapple_design.aero.geometry (v06)
- fit_discrepancy_model f — pinneapple_analysis.verification.causal_discrepancy (v05)
- fit_flight f — pinneapple_systems.sports.football.ball (v06)
- fit_fno f — pinneapple_simulation.electromagnetics.periodic.benchmark (v03)
- fit_intermediate f — pinneapple_systems.time_series.enso.forecasters (v06)
- fit_inverse f — pinneapple_simulation.numerical_solvers.gated_xtfc (v03)
- fit_iron_loss f — pinneapple_simulation.electromagnetics.machines.steel (v03)
- fit_line_tls f — pinneapple_physics.pde_environment.presets.astroml_bridge (v02)
- fit_line_with_errors f — pinneapple_physics.pde_environment.presets.astroml_bridge (v02)
- fit_mass_model f — pinneapple_design.bricks.parts (v06)
- fit_models f — pinneapple_systems.sports.football.pc_study (v06)
- fit_noise f — pinneapple_systems.sports.football.kinematics (v06)
- fit_params f — pinneapple_systems.sports.football.pitch_control (v06)
- fit_power_law f — pinneapple_analysis.validation.spectral (v05)
- fit_quadratic f — pinneapple_physics.closed_form.pump_npsh (v02)
- fit_recession f — pinneapple_systems.digital_twin.recession (v06)
- fit_recharge f — pinneapple_systems.time_series.enso.oscillators (v06)
- fit_reduced_model f — pinneapple_neural.architectures.rom.study (v04)
- fit_scale_conditioned f — pinneapple_physics.multiscale.learned (v02)
- fit_static_weights f — pinneapple_physics.ensemble (v02)
- fit_supervised f — pinneapple_neural.utils (v04)
- fit_supervised f — pinneapple_physics.multiscale.learned (v02)
- fit_surrogate f — pinneapple_design.surrogate_inverse.core (v06)
- fit_surrogate f — pinneapple_simulation.pipelines.freecad_openfoam.sweep (v03)
- fit_surrogates f — pinneapple_simulation.electromagnetics.reflector.study (v03)
- fit_two_layer f — pinneapple_simulation.geophysics.energy_balance (v03)
- fit_zone f — pinneapple_twin3d.wear.forecast (v06)

- FitMeshSession C — pinneapple_design.fitmesh.session (v06)
- FitResult C — pinneapple_analysis.inverse_problems.estimation (v05)
- Fitted C — pinneapple_analysis.surrogate.methods (v05)
- FitzHughNagumo1D C — pinneapple_arena.problems (v04)
- fix_winding f — pinneapple_design.geometry_library.meshops (v06)
- fixed_base_forward f — pinneapple_simulation.multibody.arm (v03)
- fixed_base_inverse_dynamics f — pinneapple_simulation.multibody.arm (v03)
- fixed_policy f — pinneapple_twin3d.wear.optimize (v06)
- FixedShare C — pinneapple_physics.online_learning (v02)
- FixedShareGrid C — pinneapple_physics.online_learning (v02)
- fixtures f — pinneapple_hub.zoo.web (v07)
- Flag C — pinneapple_lab.results_rules (v07)
- flags f — pinneapple_analysis.validation.fault_injection.checks (v05)
- flagship_mission f — pinneapple_autolab.domains.racecar (v07)
- flanged_tube f — pinneapple_design.surrogate_inverse.meshes (v06)
- flap_effectiveness f — pinneapple_simulation.flight.aero (v03)
- flat_disk f — pinneapple_simulation.electromagnetics.reflector.geometry (v03)
- flat_metrics f — pinneapple_physics.tracking (v02)
- flat_params f — pinneapple_physics.tracking (v02)
- flat_site f — pinneapple_simulation.vehicle.scenarios (v03)
- flatten f — pinneapple_flow.nodes_ext.examples (v07)
- flatten_graph_batch f — pinneapple_neural.architectures.graphnn.utils (v04)
- Flight C — pinneapple_systems.sports.football.ball (v06)
- flight_scene f — pinneapple_simulation.flight.visuals (v03)
- FlightFit C — pinneapple_systems.sports.football.ball (v06)
- FlightHistory C — pinneapple_simulation.flight.simulate (v03)
- flights_3d f — pinneapple_systems.sports.football.plots (v06)
- flights_from_skillcorner f — pinneapple_systems.sports.football.ball_study (v06)
- Flow C — pinneapple_flow.schema (v07)
- flow f — pinneapple_hub.library.snippets (v07)
- flow_benchmark f — pinneapple_orchestration.flows.flow_03_benchmark (v01)
- flow_coefficient f — pinneapple_systems.process_components.similarity_map (v06)
- flow_data_generation f — pinneapple_orchestration.flows.flow_01_data_generation (v01)
- flow_digital_twin_prep f — pinneapple_orchestration.flows.flow_05_digital_twin_prep (v01)
- flow_directions f — pinneapple_simulation.numerical_solvers.external_flow (v03)
- flow_for_node f — pinneapple_hub.library.flowlink (v07)
- flow_hash f — pinneapple_hub.library.flowruns (v07)
- flow_json_schema f — pinneapple_flow.schema (v07)
- flow_link f — pinneapple_hub.library.flowlink (v07)
- flow_path f — pinneapple_flow.examples (v07)
- flow_registry_publish f — pinneapple_orchestration.flows.flow_04_registry_publish (v01)
- flow_run_manifest f — pinneapple_orchestration.common.provenance (v01)
- flow_scene f — pinneapple_autolab.domains.racecar (v07)
- flow_training f — pinneapple_orchestration.flows.flow_02_training (v01)
- FlowBuilder C — pinneapple_flow.planner.build (v07)
- FlowError C — pinneapple_flow.schema (v07)
- FlowLaw C — pinneapple_security.cps (v05)
- FlowLinkError C — pinneapple_hub.library.flowlink (v07)
- FlowNodeAdapter C — pinneapple_flow.runtime (v07)
- FlowObstacle2DTask C — pinneapple_tools.benchmark_suite.tasks.flow_obstacle_2d (v07)
- FlowResult C — pinneapple_simulation.numerical_solvers.external_flow (v03)
- FlowRunManifest C — pinneapple_orchestration.common.provenance (v01)
- FlowRunResult C — pinneapple_flow.runtime (v07)
- flows f — pinneapple_flow.examples (v07)
- flows f — pinneapple_hub.library.sources (v07)
- FlowService C — pinneapple_flow.service (v07)

- FlowStore C — pinneapple_flow.store (v07)
- FlowVisualizer C — pinneapple_neural.predictor.postprocess (v04)
- fluent_mesh_to_upd f — pinneapple_simulation.external_solvers.cfd_formats.fluent_mesh_reader (v03)
- FluentCaseConfig C — pinneapple_simulation.external_solvers.ansys.journal_builder (v03)
- FluentRunConfig C — pinneapple_simulation.external_solvers.ansys.runner (v03)
- FluentRunResult C — pinneapple_simulation.external_solvers.ansys.runner (v03)
- Fluid C — pinneapple_simulation.particle_dynamics.geo_mpm (v03)
- fluidx3d_available f — pinneapple_simulation.external_solvers.fluidx3d (v03)
- fluidx3d_home f — pinneapple_simulation.external_solvers.fluidx3d (v03)
- flux f — pinneapple_core.operators (v01)
- flux_density f — pinneapple_simulation.electromagnetics.fem (v03)
- flux_density_ellipses f — pinneapple_simulation.electromagnetics.postprocess (v03)
- flux_linkage f — pinneapple_simulation.electromagnetics.postprocess (v03)
- flux_magnitude f — pinneapple_physics.darcy_checks (v02)
- flux_through_path f — pinneapple_simulation.electromagnetics.statics (v03)
- FluxMap C — pinneapple_simulation.electromagnetics.machines.drive (v03)
- fluxmap_dataset f — pinneapple_simulation.electromagnetics.machines.interop (v03)
- fmt f — pinneapple_check.checks.common (v05)
- fmt f — pinneapple_reason.modes._util (v05)
- fmt f — pinneapple_tools.reporting.paperkit (v07)
- fmt f — pinneapple_tools.visualization.studio.colormap (v07)
- fmt_s f — pinneapple_analysis.benchmark.report (v05)
- fmu_simulate f — pinneapple_flow.wrappers (v07)
- fmu_sweep f — pinneapple_simulation.external_solvers.modelica.sweep (v03)
- fmu_to_upd f — pinneapple_simulation.external_solvers.modelica.fmi_runner (v03)
- FMUDataset C — pinneapple_simulation.external_solvers.modelica.sweep (v03)
- FMUReport C — pinneapple_simulation.external_solvers.modelica.fmu_inspect (v03)
- FMUSimConfig C — pinneapple_simulation.external_solvers.modelica.fmi_runner (v03)
- FnAdapter C — pinneapple_neural.trainer.datamodule (v04)
- FNO2d C — pinneapple_neural.architectures.neural_operators.fno (v04)
- FNO3d C — pinneapple_neural.architectures.neural_operators.fno (v04)
- FNOForecastConfig C — pinneapple_systems.time_series.models.fno_forecaster (v06)
- FNOForecaster C — pinneapple_systems.time_series.models.fno_forecaster (v06)
- FNOMethod C — pinneapple_analysis.benchmark.builtin.fields (v05)
- foam f — pinneapple_check._reuse (v05)
- FoamFileHeader C — pinneapple_simulation.external_solvers.openfoam.binary_reader (v03)
- folder_of f — pinneapple_science.report (v02)
- follow_camera f — pinneapple_simulation.vehicle.viz (v03)
- football_ball_flight_inverse f — pinneapple_flow.nodes_ext.football (v07)
- football_pass_pitch_control f — pinneapple_flow.nodes_ext.football (v07)
- football_tracking_smoothing f — pinneapple_flow.nodes_ext.football (v07)
- footbridge f — pinneapple_reason.cases (v05)
- footprint f — pinneapple_simulation.numerical_solvers.heat_fv_reference (v03)
- footprint_area f — pinneapple_simulation.numerical_solvers.heat_fv_reference (v03)
- force_convergence f — pinneapple_autolab.domains.racecar (v07)
- force_to_coefficients f — pinneapple_tools.aero_coefficients (v07)
- forced_convection f — pinneapple_physics.closed_form.plate_fin_heatsink (v02)
- forced_convection f — pinneapple_physics.physics_audit.assumptions (v02)
- forced_ocean_run f — pinneapple_systems.time_series.enso.intermediate (v06)
- forces f — pinneapple_design.aero.case (v06)
- Forecast C — pinneapple_systems.time_series.forecast_lab.base (v06)
- forecast f — pinneapple_flow.nodes.systems (v07)
- forecast f — pinneapple_physics.astro.spaceweather (v02)
- forecast f — pinneapple_systems (v06)
- forecast f — pinneapple_systems.digital_twin.recession (v06)
- forecast f — pinneapple_twin3d.wear.forecast (v06)

- `forecast_case` f — `pinneapple_flow.nodes_ext.forecasting` (v07)
- `forecast_gif` f — `pinneapple_physics.weather.viz` (v02)
- `forecast_history` f — `pinneapple_twin3d.wear.forecast` (v06)
- `forecast_origins` f — `pinneapple_systems.time_series.forecast_lab.base` (v06)
- `Forecaster` C — `pinneapple_physics.weather.train` (v02)
- `Forecaster` C — `pinneapple_systems.time_series.forecast_lab.base` (v06)
- `ForecastModel` C — `pinneapple_systems.time_series.models.classical` (v06)
- `ForecastProblemSpec` C — `pinneapple_systems.time_series.problem` (v06)
- `form_values` f — `pinneapple_data.formfill.fill` (v04)
- `format_card` f — `pinneapple_core.onboarding.catalog` (v01)
- `format_reference` f — `pinneapple_flow.references` (v07)
- `format_results` f — `pinneapple_llm.workbench.evaluate` (v07)
- `format_template` f — `pinneapple_simulation.external_solvers.openfoam.campaign` (v03)
- `FormatError` C — `pinneapple_data.cae.io` (v04)
- `FormSpec` C — `pinneapple_data.formfill.spec` (v04)
- `FormulaCar` C — `pinneapple_design.geometry.vehicles3d` (v06)
- `formulate` f — `pinneapple_design.intent.parse` (v06)
- `formulate_and_recommend` f — `pinneapple_veriphysics.recommend` (v05)
- `forward` f — `pinneapple_flow.nodes_ext.architectures` (v07)
- `forward_acceleration` f — `pinneapple_systems.sports.football.kinematics` (v06)
- `forward_points` f — `pinneapple_worldmodel.robot_perception` (v04)
- `ForwardModel` C — `pinneapple_analysis.inverse_problems.forward` (v05)
- `fouling_at` f — `pinneapple_systems.operations.exchanger` (v06)
- `FoundationConfig` C — `pinneapple_worldmodel.mega_model` (v04)
- `FoundationFNOBlock` C — `pinneapple_worldmodel.mega_model` (v04)
- `fourier_basis` f — `pinneapple_physics.multiscale.filters` (v02)
- `fourier_coefficients` f — `pinneapple_physics.multiscale.filters` (v02)
- `fourier_features` f — `pinneapple_systems.time_series.features.engineering` (v06)
- `fourier_pad` f — `pinneapple_simulation.numerical_solvers.operator_datasets` (v03)
- `fourier_wavenumbers` f — `pinneapple_simulation.numerical_solvers.etdrk4` (v03)
- `FourierEncoding` C — `pinneapple_neural.lpm` (v04)
- `FourierFeatureEmbedding` C — `pinneapple_neural.architectures.modified_mlp` (v04)
- `FourierFeatures` C — `pinneapple_core.transforms` (v01)
- `FourierFeatures` C — `pinneapple_neural.architectures.benchmarks.generic_pinn_models` (v04)
- `FourierGalerkinPINN` C — `pinneapple_physics.multiscale.learned` (v02)
- `FourierInterpolant` C — `pinneapple_simulation.numerical_solvers.operator_benchmarks` (v03)
- `FourierMLP` C — `pinneapple_systems.time_series.enso.pinn` (v06)
- `FourierNeuralOperator` C — `pinneapple_neural.architectures.neural_operators.fno` (v04)
- `FourierTimeFeatures` C — `pinneapple_neural.architectures.graphnn.gnn_ode` (v04)
- `fractal_relief` f — `pinneapple_simulation.terramechanics.terrain` (v03)
- `fractional_factorial_unit` f — `pinneapple_data.doe.designs` (v04)
- `Frame` C — `pinneapple_design.bricks.visuals` (v06)
- `frame_bounds` f — `pinneapple_design.bricks.visuals` (v06)
- `frame_geometry` f — `pinneapple_design.bricks.visuals` (v06)
- `frame_indices` f — `pinneapple_simulation.flight.visuals` (v03)
- `frame_view` f — `pinneapple_simulation.vehicle.viz` (v03)
- `frames_to_samples` f — `pinneapple_simulation.external_solvers.geotaichi` (v03)
- `frames_to_video` f — `pinneapple_blender.cycles` (v07)
- `FrdResults` C — `pinneapple_simulation.external_solvers.calculix.frd` (v03)
- `free_fall` f — `pinneapple_simulation.external_solvers.newton_physics.validation` (v03)
- `free_geometry` f — `pinneapple_design.fitmesh.llm` (v06)
- `free_space_wavelength` f — `pinneapple_simulation.electromagnetics.waves` (v03)
- `FreeCADAdapter` C — `pinneapple_tools.automation.adapters.freecad` (v07)
- `FreeCADBackend` C — `pinneapple_simulation.pipelines.freecad_openfoam.cad` (v03)
- `FreeCADHeadlessAdapter` C — `pinneapple_tools.automation.adapters.pipeline` (v07)
- `FreeSurfaceFlow` C — `pinneapple_simulation.numerical_solvers.free_surface` (v03)

- `freeze` f — `pinneapple_design.surrogate_inverse.core` (v06)
- `freeze_all_except` f — `pinneapple_adaptation.transfer_learning.freeze` (v04)
- `freeze_geometry_features` f — `pinneapple_systems.sports.football.xg` (v06)
- `freeze_layers` f — `pinneapple_adaptation.transfer_learning.freeze` (v04)
- `freeze_recommendation` f — `pinneapple_analysis.sensitivity.effects` (v05)
- `Frequency` C — `pinneapple_simulation.external_solvers.calculix.study` (v03)
- `frequency` f — `pinneapple_reason.modes.structural` (v05)
- `frequency_grid` f — `pinneapple_physics.pde_environment.presets.astroml_bridge` (v02)
- `frequency_scene` f — `pinneapple_simulation.electromagnetics.antenna.scene3d` (v03)
- `FrequencyMean` C — `pinneapple_simulation.electromagnetics.antenna.surrogates` (v03)
- `freshness_score` f — `pinneapple_tools.hpo_experiments.ranking` (v07)
- `fresnel` f — `pinneapple_simulation.electromagnetics.waves` (v03)
- `FresnelSnell` C — `pinneapple_science.em.waves` (v02)
- `friction_creep` f — `pinneapple_design.bricks.dynamics` (v06)
- `friction_drag` f — `pinneapple_design.aero.buildup` (v06)
- `friction_factor` f — `pinneapple_physics.closed_form.pipe_flow` (v02)
- `FrohlichKennelly` C — `pinneapple_simulation.electromagnetics.hysteresis` (v03)
- `from_arrays` f — `pinneapple_analysis.surrogate.data` (v05)
- `from_assembly` f — `pinneapple_design.bricks.statics` (v06)
- `from_b64url` f — `pinneapple_hub.library.flowlink` (v07)
- `from_callable` f — `pinneapple_physics.ensemble` (v02)
- `from_comparison` f — `pinneapple_lab.manifest` (v07)
- `from_csr` f — `pinneapple_simulation.accelerated_foam.ldu` (v03)
- `from_dbi` f — `pinneapple_simulation.electromagnetics.antenna.bounds` (v03)
- `from_dict` f — `pinneapple_data.contracts.models` (v04)
- `from_flow_run` f — `pinneapple_lab.manifest` (v07)
- `from_intent` f — `pinneapple_flow.nodes_ext.design` (v07)
- `from_ldr` f — `pinneapple_design.bricks.ldraw` (v06)
- `from_mat` f — `pinneapple_simulation.electromagnetics.machines.interop` (v03)
- `from_model_ref` f — `pinneapple_design.design_optimizer.evaluators` (v06)
- `from_pretrained` f — `pinneapple_hub.hub` (v07)
- `from_pyleecan` f — `pinneapple_simulation.electromagnetics.machines.interop` (v03)
- `from_radec` f — `pinneapple_physics.astro.frames` (v02)
- `from_results` f — `pinneapple_lab.manifest` (v07)
- `from_run_dir` f — `pinneapple_lab.manifest` (v07)
- `from_si` f — `pinneapple_design.intent.spec` (v06)
- `from_snapshots` f — `pinneapple_analysis.surrogate.data` (v05)
- `from_solution` f — `pinneapple_lab.manifest` (v07)
- `from_solution` f — `pinneapple_physics.ensemble` (v02)
- `from_table` f — `pinneapple_analysis.surrogate.data` (v05)
- `from_text` f — `pinneapple_simulation.electromagnetics.reflector.study_report` (v03)
- `from_torch` f — `pinneapple_physics.ensemble` (v02)
- `from_trust_score` f — `pinneapple_analysis.trust.trust_report` (v05)
- `from_workflow_run` f — `pinneapple_lab.manifest` (v07)
- `front_matter` f — `pinneapple_hub.zoo.hub_cards` (v07)
- `frontier_figure` f — `pinneapple_physics.multiscale.comparison` (v02)
- `FrontierPoint` C — `pinneapple_design.surrogate_inverse.reflex_port` (v06)
- `froude` f — `pinneapple_physics.physics_audit.regime` (v02)
- `frozen_ic_rmse` f — `pinneapple_neural.architectures.graphnn.mgn_dynamics` (v04)
- `FrozenField3D` C — `pinneapple_simulation.flight.turbulence` (v03)
- `FrozenLine` C — `pinneapple_simulation.flight.turbulence` (v03)
- `FSDPConfig` C — `pinneapple_neural.trainer.hpc` (v04)
- `FullResult` C — `pinneapple_design.intent.template` (v06)
- `FunctionEvaluator` C — `pinneapple_design.design_optimizer.study` (v06)
- `FunctionField` C — `pinneapple_core.field` (v01)
- `FunctionRef` C — `pinneapple_tools.rea_bridge.evidence` (v07)

- FunctionSimulator C — pinneapple_autolab.simulator (v07)
- fundamental_args f — pinneapple_physics.astro.frames (v02)
- furnace_combustion_zone f — pinneapple_physics.pde_environment.presets.engineering (v02)
- fuselage_cd0 f — pinneapple_design.aero.aircraft3d (v06)
- fuselage_moments f — pinneapple_simulation.flight.aero (v03)
- fuselage_munk_factor f — pinneapple_simulation.flight.aero (v03)
- fuselage_wetted f — pinneapple_design.aero.aircraft3d (v06)
- FV C — pinneapple_check.fv (v05)
- fv_conduction f — pinneapple_condense.meshless2d (v02)
- fv_field f — pinneapple_analysis.benchmark.builtin.inverse_heat_impl (v05)
- fv_geometry f — pinneapple_check.fields (v05)
- fv_heat_balance f — pinneapple_check.fv (v05)
- fv_mass_balance f — pinneapple_check.fv (v05)
- fv_metrics f — pinneapple_data.cae.quality (v04)
- fv_residual f — pinneapple_analysis.validation.physics_learning (v05)
- fv_residuals f — pinneapple_check.fv (v05)
- FVMSolver C — pinneapple_simulation.numerical_solvers.fvm (v03)

G

- gain_from_q f — pinneapple_simulation.electromagnetics.reflector.po (v03)
- galactic f — pinneapple_physics.astro.journey (v02)
- galerkin f — pinneapple_simulation.accelerated_foam.warm_start (v03)
- GalerkinNeuralOperator C — pinneapple_neural.architectures.neural_operators.gno (v04)
- GalleryResult C — pinneapple_analysis.validation.gallery (v05)
- Gap C — pinneapple_lab.next_steps (v07)
- Gap C — pinneapple_problemdesign.schema (v02)
- Gardner C — pinneapple_simulation.geophysics.richards (v03)
- gardner_steady_infiltration f — pinneapple_simulation.geophysics.richards (v03)
- GasComposition C — pinneapple_systems.process_components.real_gas_eos (v06)
- gaspari_cohn f — pinneapple_analysis.state_estimation.kalman (v05)
- GasState C — pinneapple_systems.process_components.real_gas_eos (v06)
- gast_hours f — pinneapple_physics.astro.timescales (v02)
- gast_rad f — pinneapple_physics.astro.timescales (v02)
- gated_layout f — pinneapple_simulation.numerical_solvers.gated_xtfc (v03)
- GatedLayout C — pinneapple_simulation.numerical_solvers.gated_xtfc (v03)
- GatedSolution C — pinneapple_simulation.numerical_solvers.gated_xtfc (v03)
- gateway_router f — pinneapple_registry.gateway_api (v01)
- GatewayClient C — pinneapple_registry.gateway_client (v01)
- GatewayError C — pinneapple_registry.gateway_client (v01)
- gather f — pinneapple_lab.reports (v07)
- gauss_electric f — pinneapple_physics.maxwell_checks (v02)
- gauss_legendre_quadrature f — pinneapple_simulation.numerical_solvers.nonlinear_beam_fem (v03)
- gauss_lobatto_legendre_points f — pinneapple_simulation.numerical_solvers.nonlinear_beam_fem (v03)
- gauss_magnetic f — pinneapple_physics.maxwell_checks (v02)
- gaussian_curvature f — pinneapple_design.geometry.ops.curvature (v06)
- gaussian_interval_coverage f — pinneapple_systems.time_series.metrics_ext.probabilistic (v06)
- gaussian_mechanism f — pinneapple_security.dp (v05)
- gaussian_nll_diag f — pinneapple_neural.architectures.continuous.deep_state_space (v04)
- gaussian_pulse f — pinneapple_simulation.electromagnetics.fdtd (v03)
- gaussian_sigma f — pinneapple_security.dp (v05)
- GaussianMisfit C — pinneapple_analysis.inverse_problems.noise_models (v05)
- GaussianProcessField C — pinneapple_analysis.data_assimilation.sensor_placement (v05)
- GaussianProcessSurrogate C — pinneapple_neural.workflows.hybrid_surrogate_physics (v04)
- GaussLaw C — pinneapple_science.em.statics (v02)
- GBMForecaster C — pinneapple_systems.time_series.forecast_lab.learned (v06)
- gci f — pinneapple_design.fitmesh.report (v06)

- gci f — pinneapple_lab.uq (v07)
- gci_check f — pinneapple_analysis.benchmark.validation (v05)
- gci_check f — pinneapple_lab.uq (v07)
- gear_forces f — pinneapple_simulation.flight.gear (v03)
- GearLeg C — pinneapple_simulation.flight.gear (v03)
- GeminiProvider C — pinneapple_problemdesign.protocol (v02)
- GeminiProvider C — pinneapple_tools.hpo_experiments.providers.gemini_provider (v07)
- GeminiProviderConfig C — pinneapple_tools.hpo_experiments.providers.gemini_provider (v07)
- generalization_section f — pinneapple_app.backend.core.surrogate_report (v07)
- generalized_reynolds_number f — pinneapple_systems.process_components.non_newtonian_pipe_flow (v06)
- generate f — pinneapple_analysis.benchmark.builtin.inverse_heat_impl (v05)
- generate f — pinneapple_design.fitmesh.mesh_gen (v06)
- generate f — pinneapple_flow.autocalls (v07)
- generate f — pinneapple_simulation.numerical_solvers.cylinder_wake (v03)
- generate f — pinneapple_simulation.numerical_solvers.operator_datasets (v03)
- generate f — pinneapple_simulation.numerical_solvers.thermal_channel_datasets (v03)
- generate f — pinneapple_worldmodel.generate_datasets (v04)
- generate_1d_mesh f — pinneapple_simulation.numerical_solvers.nonlinear_beam_fem (v03)
- generate_acoustic_synthetic f — pinneapple_analysis.inspection.acoustic (v05)
- generate_artifact f — pinneapple_llm.qa.generation (v07)
- generate_case f — pinneapple_simulation.numerical_solvers.openfoam_bridge (v03)
- generate_chemical_mixing f — pinneapple_worldmodel.synthetic_factory (v04)
- generate_cylinder_flow f — pinneapple_worldmodel.synthetic_factory (v04)
- generate_data f — pinneapple_simulation (v03)
- generate_dataset f — pinneapple_design.surrogate_inverse.brake_rotor (v06)
- generate_dataset f — pinneapple_design.surrogate_inverse.heatsink_fins (v06)
- generate_dataset f — pinneapple_design.surrogate_inverse.reflex_port (v06)
- generate_dataset f — pinneapple_design.surrogate_inverse.suspension_arm (v06)
- generate_dataset f — pinneapple_design.surrogate_inverse.transdermal_patch (v06)
- generate_dataset f — pinneapple_design.thermal.heatsink.surrogate (v06)
- generate_eddy_current_synthetic f — pinneapple_analysis.inspection.eddy_current (v05)
- generate_emat_synthetic f — pinneapple_analysis.inspection.emat (v05)
- generate_guided_wave_synthetic f — pinneapple_analysis.inspection.guided_wave_ultrasonic (v05)
- generate_heat_transfer f — pinneapple_worldmodel.synthetic_factory (v04)
- generate_history f — pinneapple_systems.operations.exchanger (v06)
- generate_key f — pinneapple_security.signing (v05)
- generate_magnetic_flux_leakage_synthetic f — pinneapple_analysis.inspection.magnetic_flux_leakage (v05)
- generate_phased_array_synthetic f — pinneapple_analysis.inspection.phased_array_ultrasonic (v05)
- generate_pinn_dataset f — pinneapple_simulation.numerical_solvers.problem_runner (v03)
- generate_pipe_flow f — pinneapple_worldmodel.synthetic_factory (v04)
- generate_points f — pinneapple_app.backend.core.collocation (v07)
- generate_scenarios_task f — pinneapple_orchestration.tasks.data_generation_tasks (v01)
- generate_sequences f — pinneapple_neural.architectures.rom.toy_systems (v04)
- generate_synthetic_csv f — pinneapple_systems.time_series.csv_loader (v06)
- generate_via_factory f — pinneapple_worldmodel.generate_datasets (v04)
- GeneratedArtifact C — pinneapple_llm.qa.generation (v07)
- Generator C — pinneapple_analysis.validation.arena.generators (v05)
- generators_for f — pinneapple_analysis.validation.arena.generators (v05)
- generic f — pinneapple_flow.nodes.problem (v07)
- GenericFourierMLP C — pinneapple_neural.architectures.benchmarks.generic_pinn_models (v04)
- GenericLinear C — pinneapple_neural.architectures.benchmarks.generic_pinn_models (v04)
- GenericMLP C — pinneapple_neural.architectures.benchmarks.generic_pinn_models (v04)
- GenericResMLP C — pinneapple_neural.architectures.benchmarks.generic_pinn_models (v04)
- GenericSIREN C — pinneapple_neural.architectures.benchmarks.generic_pinn_models (v04)
- genesis_traj_to_upd f — pinneapple_simulation.external_solvers.genesis.adapter (v03)

- genesis_trajs_to_upd f — pinneapple_simulation.external_solvers.genesis.adapter (v03)
- GenesisConfig C — pinneapple_simulation.external_solvers.genesis.scene_builder (v03)
- GenesisRunner C — pinneapple_simulation.external_solvers.genesis.runner (v03)
- GeneticSDF C — pinneapple_design.geometry.gen.genetic_sdf (v06)
- geo_l_bracket f — pinneapple_simulation.external_solvers.calculix.study (v03)
- geo_plate_with_hole f — pinneapple_simulation.external_solvers.calculix.study (v03)
- geo_shelf_bracket f — pinneapple_design.intent.physics.bracket (v06)
- geodetic_to_itrs f — pinneapple_physics.astro.frames (v02)
- geomagnetic_latitude f — pinneapple_physics.astro.spaceweather (v02)
- geometric_features f — pinneapple_systems.sports.football.pitch_control (v06)
- GeometricConstraint C — pinneapple_design.design_optimizer.constraints (v06)
- geometries f — pinneapple_hub.library.sources (v07)
- Geometry C — pinneapple_condense.meshless (v02)
- Geometry C — pinneapple_core.geometry (v01)
- Geometry C — pinneapple_design.geometry_library.core (v06)
- Geometry C — pinneapple_simulation.electromagnetics.fem (v03)
- geometry f — pinneapple_flow.nodes.cfd_pipeline (v07)
- geometry f — pinneapple_hub.library.snippets (v07)
- geometry_checks f — pinneapple_tools.nvidia.brep (v07)
- geometry_doc f — pinneapple_flow.nodes.problem (v07)
- geometry_errors f — pinneapple_physics.closed_form.plate_fin_heatsink (v02)
- geometry_errors f — pinneapple_simulation.electromagnetics.machines.geometry (v03)
- geometry_export f — pinneapple_flow.wrappers (v07)
- geometry_figure f — pinneapple_simulation.pipelines.freecad_openfoam.post (v03)
- GeometryAsset C — pinneapple_design.geometry.core.geometry (v06)
- GeometryBase C — pinneapple_worldmodel.geometry (v04)
- GeometryClassification C — pinneapple_analysis.verification.geometry_intelligence (v05)
- GeometryDraftResult C — pinneapple_llm.geometry_draft (v07)
- GeometryEncoder C — pinneapple_neural.lpm (v04)
- GeometryIndex C — pinneapple_design.geometry.retrieval (v06)
- GeometryLoadOptions C — pinneapple_data.adapters.geom_adapter (v04)
- GeometryOODGuardrail C — pinneapple_analysis.verification.geometry_ood_guardrail (v05)
- GeometryOODResult C — pinneapple_analysis.verification.geometry_ood_guardrail (v05)
- GeometryOptimizer C — pinneapple_design.geometry.optimize.loop (v06)
- GeometryPreset C — pinneapple_arena.pointwise.schema (v04)
- GeometrySpec C — pinneapple_design.geometry.core.geometry (v06)
- GeometrySpec C — pinneapple_design.geometry_library.core (v06)
- GeometrySpec C — pinneapple_llm.qa.spec (v07)
- GeometrySpec C — pinneapple_problemdesign.schema (v02)
- GeometrySynthGenerator C — pinneapple_data.synth.geometry (v04)
- GeometryType C — pinneapple_arena.pointwise.schema (v04)
- GeoMPM C — pinneapple_simulation.particle_dynamics.geo_mpm (v03)
- geotaichi_available f — pinneapple_simulation.external_solvers.geotaichi (v03)
- geotaichi_home f — pinneapple_simulation.external_solvers.geotaichi (v03)
- germano_error f — pinneapple_physics.multiscale.closures (v02)
- get f — pinneapple_design.intent.materials (v06)
- get f — pinneapple_hub.library (v07)
- get f — pinneapple_lab.spec (v07)
- get f — pinneapple_llm.qa.validators.base (v07)
- get f — pinneapple_reason.cases (v05)
- get f — pinneapple_science.registry (v02)
- get f — pinneapple_twin3d.gallery (v06)
- get f — pinneapple_twin3d.wear.presets (v06)
- get_adapter f — pinneapple_core.workflows.adapters (v01)
- get_adapter f — pinneapple_tools.automation.adapters.base (v07)
- get_assumption f — pinneapple_physics.physics_audit.assumptions (v02)

- get_backend f — pinneapple_quantum.backends.backend (v04)
- get_backend f — pinneapple_simulation.pipelines.freecad_openfoam.cad (v03)
- get_backend f — pinneapple_tools.benchmark_suite.registry (v07)
- get_backend f — pinneapple_tools.compute_backends.backend (v07)
- get_benchmark f — pinneapple_pdb.benchmarks (v04)
- get_benchmark_preset f — pinneapple_arena.dataset_bench (v04)
- get_cmap f — pinneapple_tools.visualization.style (v07)
- get_config f — pinneapple_mcp.config (v07)
- get_connection_geometry f — pinneapple_design.geometry.gen.threaded_connection (v06)
- get_dataset_preview f — pinneapple_arena.external.runners.registry (v04)
- get_domain f — pinneapple_design.geometry.gen.domains (v06)
- get_domain_3d f — pinneapple_design.geometry.gen.domains3d (v06)
- get_entry f — pinneapple_simulation.external_solvers.openfoam.campaign (v03)
- get_example_case f — pinneapple_veriphysics.example_cases (v05)
- get_executor f — pinneapple_core.workflows.executors (v01)
- get_fault f — pinneapple_analysis.validation.fault_injection.faults (v05)
- get_generator f — pinneapple_analysis.validation.arena.generators (v05)
- get_geometry_mesh_config f — pinneapple_arena.pointwise.geometries (v04)
- get_irk_tableau f — pinneapple_simulation.numerical_solvers.irk_gauss_legendre (v03)
- get_method f — pinneapple_analysis.benchmark.registry (v05)
- get_method f — pinneapple_catalog.methods (v01)
- get_model f — pinneapple_app.backend.routers.models (v07)
- get_model_info f — pinneapple_app.backend.core.model_catalog (v07)
- get_pde_family f — pinneapple_physics.pde_environment.capabilities (v02)
- get_physics_backend f — pinneapple_core.backend (v01)
- get_player_for_tool f — pinneapple_analysis.verification.tool_recommendation (v05)
- get_potential f — pinneapple_quantum.loss.schrodinger (v04)
- get_preset f — pinneapple_analysis.inverse_problems.presets (v05)
- get_preset f — pinneapple_physics.pde_environment.presets.registry (v02)
- get_preset f — pinneapple_tools.visualization.studio.lighting (v07)
- get_problem f — pinneapple_analysis.benchmark.registry (v05)
- get_problem f — pinneapple_analysis.validation.fault_injection.problems (v05)
- get_problem f — pinneapple_app.backend.routers.problems (v07)
- get_problem f — pinneapple_arena.problems (v04)
- get_problem f — pinneapple_tools.benchmark_suite.crossframework.problems (v07)
- get_problem_meta f — pinneapple_app.backend.core.problem_registry (v07)
- get_rank f — pinneapple_neural.trainer.distributed (v04)
- get_rans_preset f — pinneapple_physics.pde_environment.turbulence_presets (v02)
- get_resource f — pinneapple_catalog.resources (v01)
- get_results f — pinneapple_app.backend.routers.experiments (v07)
- get_rule f — pinneapple_check.rules (v05)
- get_soil f — pinneapple_simulation.terramechanics.soils (v03)
- get_spec f — pinneapple_data.formfill.specs (v04)
- get_spec f — pinneapple_design.geometry_library.core (v06)
- get_status f — pinneapple_app.backend.routers.experiments (v07)
- get_task f — pinneapple_tools.benchmark_suite.registry (v07)
- get_template f — pinneapple_design.intent.parse (v06)
- get_tools_by_category f — pinneapple_analysis.verification.tool_recommendation (v05)
- get_turbulence_closure f — pinneapple_physics.pde_environment.turbulence_selector (v02)
- get_world_size f — pinneapple_neural.trainer.distributed (v04)
- getdp_crosscheck f — pinneapple_flow.nodes_ext.em_prius (v07)
- gill_analytic f — pinneapple_systems.time_series.enso.intermediate (v06)
- gill_test f — pinneapple_systems.time_series.enso.intermediate (v06)
- GillAtmosphere C — pinneapple_systems.time_series.enso.intermediate (v06)
- girder_loads f — pinneapple_reason.modes.structural (v05)
- git_commit f — pinneapple_security.attestation (v05)

- github f — pinneapple_check.report (v05)
- GithubFetcher C — pinneapple_tools.hpo_experiments.sources.github_fetcher (v07)
- GithubRepo C — pinneapple_tools.hpo_experiments.sources.github_source (v07)
- GithubSource C — pinneapple_tools.hpo_experiments.sources.github_source (v07)
- GitPublisher C — pinneapple_simulation.external_solvers.openfoam.campaign (v03)
- GliderWingTemplate C — pinneapple_design.intent.templates.glider_wing (v06)
- GlobalSensitivity C — pinneapple_analysis.inverse_problems.sensitivity (v05)
- GLU C — pinneapple_neural.architectures.transformers.tft (v04)
- gmm_fit f — pinneapple_physics.astro.stars_ml (v02)
- gmsh_exe f — pinneapple_design.intent.physics.bracket (v06)
- GmshGetDPError C — pinneapple_simulation.electromagnetics.machines.getdp (v03)
- gmst82 f — pinneapple_physics.astro.satellites (v02)
- gmst_rad f — pinneapple_physics.astro.timescales (v02)
- gnielinski f — pinneapple_physics.closed_form.heat_exchanger (v02)
- GNNAdapter C — pinneapple_neural.architectures.adapters.gnn (v04)
- go_aperture_taper_db f — pinneapple_simulation.electromagnetics.reflector.analytic (v03)
- go_check f — pinneapple_simulation.electromagnetics.reflector.compact_range (v03)
- Goal C — pinneapple_design.bricks.goals (v06)
- good_models f — pinneapple_analysis.validation.physics_learning_cases (v05)
- goodman_equivalent_amplitude f — pinneapple_systems.process_components.fatigue_analysis (v06)
- goodman_safety_ratio f — pinneapple_systems.process_components.fatigue_analysis (v06)
- gp_loo_scale f — pinneapple_science.numerics (v02)
- GPModel C — pinneapple_autolab.surrogates (v07)
- GPModel C — pinneapple_simulation.electromagnetics.antenna.surrogates (v03)
- GPRForecaster C — pinneapple_systems.time_series.models.classical (v06)
- GPSurrogate C — pinneapple_design.design_optimizer.verify (v06)
- gpu_info f — pinneapple_neural.trainer.parallel (v04)
- grad f — pinneapple_core.func (v01)
- grad f — pinneapple_core.operators (v01)
- grad f — pinneapple_physics.pinn_solver.compiler.autograd_ops (v02)
- GradAccumConfig C — pinneapple_neural.trainer.parallel (v04)
- GradAccumTrainer C — pinneapple_neural.trainer.parallel (v04)
- grade_sweep f — pinneapple_simulation.vehicle.scenarios (v03)
- grade_table f — pinneapple_simulation.rover.autolab.wheel_models (v03)
- graded_axis f — pinneapple_science.numerics (v02)
- GradeTable C — pinneapple_simulation.rover.autolab.wheel_models (v03)
- gradient_check f — pinneapple_analysis.data_assimilation.var4d (v05)
- gradient_fit f — pinneapple_analysis.inverse_problems.estimation (v05)
- gradient_metrics f — pinneapple_analysis.validation.spectral (v05)
- gradient_sensitivity f — pinneapple_tools.model_evaluation.metrics (v07)
- GradientDesignOptimizer C — pinneapple_design.design_optimizer.optimizer (v06)
- GradientModel C — pinneapple_physics.multiscale.closures (v02)
- GradNormBalancer C — pinneapple_neural.trainer.weight_scheduler (v04)
- graph_data f — pinneapple_analysis.benchmark.builtin.airfoil_impl (v05)
- graph_data f — pinneapple_design.aero.train (v06)
- graph_stats f — pinneapple_analysis.verification.knowledge_graph (v05)
- GraphBatch C — pinneapple_neural.architectures.graphnn.base (v04)
- GraphCast C — pinneapple_neural.architectures.graphnn.graphcast (v04)
- GraphCatalog C — pinneapple_neural.architectures.graphnn.registry (v04)
- GraphGuessNet C — pinneapple_simulation.accelerated_foam.learned (v03)
- GraphModelBase C — pinneapple_neural.architectures.graphnn.base (v04)
- GraphNeuralNetwork C — pinneapple_neural.architectures.graphnn.gnn (v04)
- GraphNeuralODE C — pinneapple_neural.architectures.graphnn.gnn_ode (v04)
- GraphOutput C — pinneapple_neural.architectures.graphnn.base (v04)
- grashof f — pinneapple_physics.physics_audit.regime (v02)
- gravity_audit f — pinneapple_systems.sports.football.ball_study (v06)

- gravity_audit_figure f — pinneapple_systems.sports.football.plots (v06)
- gravity_body f — pinneapple_simulation.flight.rigid_body (v03)
- GrayBoxConfig C — pinneapple_neural.trainer.graybox (v04)
- GrayBoxNet C — pinneapple_neural.trainer.graybox (v04)
- greedy_maximin f — pinneapple_data.doe.designs (v04)
- greedy_sensor_placement f — pinneapple_analysis.data_assimilation.sensor_placement (v05)
- grf_burgers f — pinneapple_flow.nodes.data (v07)
- grf_direction f — pinneapple_analysis.uncertainty.posterior_metrics (v05)
- GRFDraw C — pinneapple_data.grf_sampler (v04)
- GRFDraw2D C — pinneapple_data.grf_sampler (v04)
- Grid C — pinneapple_condense.flow (v02)
- Grid C — pinneapple_core.field (v01)
- grid f — pinneapple_simulation.numerical_solvers.xtfc_general (v03)
- grid_coords f — pinneapple_analysis.surrogate.data (v05)
- grid_derivative f — pinneapple_physics.symbolic_pde.gradient_backends (v02)
- grid_force f — pinneapple_check.balances (v05)
- grid_from_arrays f — pinneapple_tools.visualization.pyvista_bridge (v07)
- grid_graph f — pinneapple_neural.physicsnemo_bridge.field_operators (v04)
- grid_heat f — pinneapple_check.balances (v05)
- grid_layout f — pinneapple_arena.trainer (v04)
- grid_mass f — pinneapple_check.balances (v05)
- grid_normals f — pinneapple_tools.visualization.studio.shadows (v07)
- grid_operator_service f — pinneapple_tools.model_services.grid (v07)
- grid_points f — pinneapple_design.aero.mesh (v06)
- grid_points f — pinneapple_flow.nodes._common (v07)
- grid_surface f — pinneapple_tools.nvidia.omniverse (v07)
- GridAxisSpec C — pinneapple_physics.symbolic_pde.gradient_backends (v02)
- GridCase C — pinneapple_physics.paper_problems (v02)
- GridD C — pinneapple_check.numerics (v05)
- Gridded C — pinneapple_systems.time_series.enso.data (v06)
- GridFieldPredictor C — pinneapple_neural.physicsnemo_bridge.field_operators (v04)
- GridSpec C — pinneapple_design.aero.mesh (v06)
- GridSpec C — pinneapple_neural.architectures.neural_operators.pino (v04)
- GRN C — pinneapple_neural.architectures.transformers.tft (v04)
- Ground C — pinneapple_design.bricks.assembly (v06)
- ground_equilibrium f — pinneapple_simulation.flight.trim (v03)
- GroundTruthPackager C — pinneapple_worldmodel.physics_solver_bridge (v04)
- group f — pinneapple_lab.next_steps (v07)
- group_bootstrap f — pinneapple_analysis.uncertainty.resampling (v05)
- GroupBCatalog C — pinneapple_neural.architectures.group_b_registry (v04)
- GroupedMLP C — pinneapple_systems.time_series.enso.pinn (v06)
- GroupVSN C — pinneapple_neural.architectures.transformers.tft (v04)
- Grousers C — pinneapple_simulation.terramechanics.wheel (v03)
- grow_region f — pinneapple_analysis.validation.error_map (v05)
- GRUForecaster C — pinneapple_systems.time_series.models.recurrent (v06)
- GRUModel C — pinneapple_neural.architectures.recurrent.gru (v04)
- guardrail_checks f — pinneapple_check.evidence (v05)
- guardrail_checks_as_dicts f — pinneapple_analysis.verification.pipeline (v05)
- GuardrailReport C — pinneapple_llm.guardrail (v07)
- GuessServer C — pinneapple_simulation.accelerated_foam.guess_server (v03)
- GuiAgentStep C — pinneapple_tools.automation.runner_step (v07)
- GuidanceBase C — pinneapple_neural.architectures.pinns.physics_diffusion (v04)
- GuiUnavailable C — pinneapple_tools.automation.display (v07)
- gunzip f — pinneapple_security.archives (v05)
- gust_sensitivity f — pinneapple_simulation.flight.encounter (v03)
- gust_sensitivity_figure f — pinneapple_simulation.flight.figures (v03)

- gust_wisps f — pinneapple_simulation.flight.visuals (v03)

H

- haaland f — pinneapple_physics.closed_form.pipe_flow (v02)
- half_wave_dipole_directivity f — pinneapple_simulation.electromagnetics.waves (v03)
- half_wave_dipole_radiation_resistance f — pinneapple_simulation.electromagnetics.waves (v03)
- hall_bar f — pinneapple_science.em.statics (v02)
- HallEffect C — pinneapple_science.em.statics (v02)
- HallSurrogate C — pinneapple_science.em.research (v02)
- hamiltonian_expectation f — pinneapple_quantum.loss.schrodinger (v04)
- HamiltonianNeuralNetwork C — pinneapple_neural.architectures.continuous.hamiltonian (v04)
- handles f — pinneapple_design.aero.geometry (v06)
- HandlingPolicy C — pinneapple_security.classification (v05)
- HardBC C — pinneapple_physics.symbolic_pde.bc (v02)
- harden f — pinneapple_analysis.validation.arena.blue (v05)
- HardNet f — pinneapple_analysis.benchmark.builtin.standard (v05)
- hardware_record f — pinneapple_analysis.cost.equal_budget (v05)
- harmonic f — pinneapple_design.bricks.closed_form (v06)
- harmonic_limit_lp f — pinneapple_design.bricks.validation (v06)
- harmonic_overhang f — pinneapple_design.bricks.closed_form (v06)
- harmonic_potential f — pinneapple_quantum.loss.schrodinger (v04)
- harmonic_stack f — pinneapple_design.bricks.closed_form (v06)
- harmonic_threshold f — pinneapple_design.bricks.dynamics (v06)
- HarvestEntry C — pinneapple_tools.rea_bridge.harvest (v07)
- HarvestLedger C — pinneapple_tools.rea_bridge.harvest (v07)
- has_cfmesh f — pinneapple_simulation.pipelines.freecad_openfoam.mesh (v03)
- has_default_geometry f — pinneapple_physics.pde_environment.presets.tag_geometry (v02)
- has_field f — pinneapple_physics.symbolic_pde.pinn (v02)
- has_pinnego_physical_sample f — pinneapple_data.synth.sample_adapter (v04)
- has_query_checks f — pinneapple_hub.zoo.checks (v07)
- hash_bytes f — pinneapple_llm.qa.sandbox (v07)
- hash_data f — pinneapple_analysis.validation.report_artifact (v05)
- hash_key f — pinneapple_core.service.apikeys (v01)
- hash_model f — pinneapple_analysis.validation.report_artifact (v05)
- hash_password f — pinneapple_security.passwords (v05)
- HashGridEncoding C — pinneapple_neural.architectures.hash_grid (v04)
- HashGridMLP C — pinneapple_neural.architectures.hash_grid (v04)
- hausen_mean_nusselt f — pinneapple_physics.closed_form.heat_exchanger (v02)
- HAVOK C — pinneapple_neural.architectures.rom.havok (v04)
- header f — pinneapple_data.cae.foam (v04)
- header f — pinneapple_data.simulation_metadata.foam_dict (v04)
- headline f — pinneapple_lab.curation (v07)
- health f — pinneapple_app.backend.main (v07)
- HealthIndicatorHistory C — pinneapple_systems.digital_twin.prognostics (v06)
- Heat C — pinneapple_simulation.external_solvers.calculix.study (v03)
- Heat1D C — pinneapple_arena.problems (v04)
- Heat1DTask C — pinneapple_tools.benchmark_suite.tasks.heat_1d (v07)
- Heat2D C — pinneapple_arena.problems (v04)
- heat_conduction_problem f — pinneapple_analysis.uncertainty.residual_bound (v05)
- heat_conduction_residual f — pinneapple_systems.component_modeling.physics_residuals (v06)
- heat_loss f — pinneapple_physics.closed_form.pipe_insulation (v02)
- heat_sink f — pinneapple_tools.nvidia.parts (v07)
- heat_sink_geometry f — pinneapple_tools.nvidia.parts (v07)
- heat_sink_material f — pinneapple_design.intent.materials (v06)
- heat_sink_mesh f — pinneapple_design.thermal.board_cooling.scene (v06)
- HeatConduction3D C — pinneapple_simulation.numerical_solvers.fdm3d (v03)

- HeatConfig3D C — pinneapple_simulation.numerical_solvers.fdm3d (v03)
- heated_pipe f — pinneapple_simulation.pipelines.freecad_openfoam.studies (v03)
- HeatExchangerResult C — pinneapple_systems.process_components.heat_exchanger (v06)
- HeatExchangerSpec C — pinneapple_systems.process_components.heat_exchanger (v06)
- HeatExchangerVanillaPINN C — pinneapple_systems.component_library.components (v06)
- heatsink_baseline f — pinneapple_design.design_optimizer.presets (v06)
- heatsink_figures f — pinneapple_design.intent.figures (v06)
- heatsink_problem f — pinneapple_design.design_optimizer.presets (v06)
- HeatsinkDesign C — pinneapple_design.surrogate_inverse.heatsink_fins (v06)
- HeatsinkEvaluator C — pinneapple_design.design_optimizer.presets (v06)
- HeatsinkForced C — pinneapple_hub.zoo._models (v07)
- HeatSinkGeometry C — pinneapple_physics.closed_form.plate_fin_heatsink (v02)
- HeatsinkLimits C — pinneapple_design.design_optimizer.presets (v06)
- HeatsinkNatural C — pinneapple_hub.zoo._models (v07)
- HeatsinkOperating C — pinneapple_design.design_optimizer.presets (v06)
- HeatSinkTemplate C — pinneapple_design.intent.templates.heatsink (v06)
- HeatXTFC C — pinneapple_lab.experiments.pde (v07)
- heaviside f — pinneapple_design.topology.filters (v06)
- heaviside_derivative f — pinneapple_design.topology.filters (v06)
- height_above f — pinneapple_tools.visualization.molecular (v07)
- height_errors f — pinneapple_simulation.geophysics.sphere_shallow_water (v03)
- heldout f — pinneapple_simulation.electromagnetics.periodic.benchmark (v03)
- heldout_report f — pinneapple_analysis.surrogate.heldout (v05)
- helical_coil f — pinneapple_design.geometry.gen.cadquery_advanced (v06)
- heliocentric_state f — pinneapple_physics.astro.orbits (v02)
- helix_segments f — pinneapple_simulation.electromagnetics.statics (v03)
- helix_sweep f — pinneapple_design.geometry.vehicle_cad (v06)
- helmbold_lift_slope f — pinneapple_simulation.flight.aero (v03)
- Helmholtz2D C — pinneapple_arena.problems (v04)
- helmholtz_acoustics_3d_default f — pinneapple_physics.pde_environment.presets.industry (v02)
- helmholtz_coil_center_B f — pinneapple_simulation.electromagnetics.statics (v03)
- helmholtz_eigs f — pinneapple_science.numerics (v02)
- helmholtz_resonant_frequency_hz f — pinneapple_physics.closed_form.helmholtz_resonator (v02)
- hermite_shape_functions_1d f — pinneapple_simulation.numerical_solvers.nonlinear_beam_fem (v03)
- herschel_bulkley_stress f — pinneapple_systems.process_components.non_newtonian_pipe_flow (v06)
- hertzian_dipole_fields f — pinneapple_simulation.electromagnetics.waves (v03)
- hertzian_dipole_power f — pinneapple_simulation.electromagnetics.waves (v03)
- hertzian_dipole_radiation_resistance f — pinneapple_simulation.electromagnetics.waves (v03)
- hessian f — pinneapple_core.func (v01)
- hessian f — pinneapple_core.operators (v01)
- heston_pde_2d f — pinneapple_physics.pde_environment.presets.multidisciplinary (v02)
- heteroscedastic_nll_loss f — pinneapple_neural.trainer.heteroscedastic (v04)
- HeteroscedasticMisfit C — pinneapple_analysis.inverse_problems.noise_models (v05)
- HeteroscedasticWrapper C — pinneapple_neural.trainer.heteroscedastic (v04)
- hex8_incompatible_stiffness f — pinneapple_design.qualitative.quantitative (v06)
- hex_model f — pinneapple_design.intent.physics.bracket (v06)
- HexMesh C — pinneapple_simulation.numerical_solvers.solid_fem (v03)
- heywood_kt_net f — pinneapple_simulation.numerical_solvers.plane_stress_fem (v03)
- hf_repo f — pinneapple_hub.zoo._catalog (v07)
- HHTDecomposer C — pinneapple_systems.time_series.decomposition.hht_lstm (v06)
- HHTLSTMPipeline C — pinneapple_systems.time_series.decomposition.hht_lstm (v06)
- HHTNNForecaster C — pinneapple_systems.time_series.decomposition.hht_nn (v06)
- Hierarchy C — pinneapple_condense.hierarchy (v02)
- high_wavenumber_bias f — pinneapple_analysis.validation.spectral (v05)
- hilbert_analysis f — pinneapple_systems.time_series.solar_cycle (v06)
- HilbertHuangSolver C — pinneapple_simulation.numerical_solvers.hilbert_huang (v03)

- HilbertResult C — pinneapple_systems.time_series.solar_cycle (v06)
- history_features f — pinneapple_systems.sports.football.kinematics (v06)
- hmac_key f — pinneapple_security.signing (v05)
- hoerner_profile_cd f — pinneapple_physics.closed_form.thin_airfoil (v02)
- holdout f — pinneapple_design.thermal.board_cooling.learn (v06)
- holdout f — pinneapple_simulation.electromagnetics.antenna.surrogates (v03)
- holdout_figure f — pinneapple_systems.sports.football.plots (v06)
- holdout_prediction f — pinneapple_systems.sports.football.ball_study (v06)
- holdout_split f — pinneapple_simulation.terramechanics.testbed (v03)
- hollow_pipe_mesh f — pinneapple_llm.qa.examples.pipe_wear_twin (v07)
- holm f — pinneapple_systems.time_series.metrics_ext.comparison (v06)
- HoltDamped C — pinneapple_systems.time_series.adaptive (v06)
- HoltWintersAdditive C — pinneapple_systems.time_series.adaptive (v06)
- home f — pinneapple_flow.config (v07)
- homogenization_study f — pinneapple_condense.benchmark (v02)
- homogenized_observables f — pinneapple_condense.homogenize (v02)
- homogenized_system f — pinneapple_condense.homogenize (v02)
- horizon_altitude f — pinneapple_physics.astro.phenomena (v02)
- horizon_distance f — pinneapple_tools.visualization.studio.planetary (v07)
- horizons f — pinneapple_physics.weather.evaluate (v02)
- horizons_vectors f — pinneapple_physics.astro.smallbodies (v02)
- horizontal_to_equatorial f — pinneapple_physics.astro.frames (v02)
- horseshoe_induced f — pinneapple_design.aero.vlm (v06)
- HotSpot C — pinneapple_twin3d.wear.synthetic (v06)
- hotspot_power f — pinneapple_condense.package (v02)
- hour_angle_dec f — pinneapple_physics.astro.observing (v02)
- hpo f — pinneapple_flow.nodes.surrogates (v07)
- hr_diagram f — pinneapple_physics.astro.stars (v02)
- hstack_gifs f — pinneapple_tools.visualization.dashboard.static (v07)
- htail_weight f — pinneapple_design.aero.aircraft3d (v06)
- html_to_text f — pinneapple_llm.workbench.web (v07)
- HTTPPollStream C — pinneapple_systems.digital_twin.io.stream (v06)
- hub_url f — pinneapple_design.geometry_library.store (v06)
- HuberMisfit C — pinneapple_analysis.inverse_problems.noise_models (v05)
- HubQuery C — pinneapple_pdb.builder (v04)
- humanize f — pinneapple_flow.palette (v07)
- HWOffset C — pinneapple_simulation.electromagnetics.antenna.surrogates (v03)
- Hybrid C — pinneapple_core.module (v01)
- hybrid_conduction f — pinneapple_condense.meshless2d (v02)
- hybrid_fem f — pinneapple_flow.nodes_ext.core_ts (v07)
- hybrid_pinn f — pinneapple_quantum.pipeline.presets (v04)
- hybrid_vs_fea_only f — pinneapple_simulation.electromagnetics.machines.design (v03)
- HybridModel C — pinneapple_quantum.models.hybrid_model (v04)
- HybridPrediction C — pinneapple_neural.workflows.hybrid_surrogate_physics (v04)
- HybridRBFNetwork C — pinneapple_neural.architectures.reservoir_computing.hybrid_rbf (v04)
- HybridSurrogatePhysics C — pinneapple_neural.workflows.hybrid_surrogate_physics (v04)
- hydraulically_smooth f — pinneapple_physics.physics_audit.assumptions (v02)
- hypervolume f — pinneapple_design.design_optimizer.study (v06)
- hypothesis_map f — pinneapple_flow.nodes_ext.non_hermitian (v07)
- HysteresisLoop C — pinneapple_simulation.electromagnetics.hysteresis (v03)

I

- i_section f — pinneapple_reason.modes.structural (v05)
- iaaft_surrogate f — pinneapple_systems.time_series.predictability (v06)
- iapws97_psat f — pinneapple_physics.closed_form.pump_npsh (v02)
- iban_ok f — pinneapple_security.privacy (v05)

- id_column_of f — pinneapple_core.workflows.datasets (v01)
- identifiability f — pinneapple_systems.sports.football.ball_study (v06)
- identifiability_report f — pinneapple_analysis.inverse_problems.heat_diffusivity (v05)
- identifiability_study f — pinneapple_analysis.inverse_problems.contracts (v05)
- IdentifiabilityAnalyzer C — pinneapple_analysis.inverse_problems.sensitivity (v05)
- IdentifiabilityReport C — pinneapple_analysis.inverse_problems.identifiability (v05)
- IdentifiabilityResult C — pinneapple_analysis.inverse_problems.sensitivity (v05)
- identify f — pinneapple_flow.identify (v07)
- identify f — pinneapple_physics (v02)
- identify_pde f — pinneapple_physics.pde_environment.capabilities (v02)
- identify_recharge f — pinneapple_systems.time_series.enso.pinn (v06)
- ids f — pinneapple_physics.astro.facts (v02)
- iges_to_mesh f — pinneapple_design.geometry.io.iges (v06)
- IgesImportConfig C — pinneapple_design.geometry.io.iges (v06)
- illuminated_fraction f — pinneapple_physics.astro.phenomena (v02)
- illustrative_steel f — pinneapple_simulation.electromagnetics.materials (v03)
- ImageReconstructionSynthGenerator C — pinneapple_data.synth.images (v04)
- ImageRegressor C — pinneapple_worldmodel.robot_perception (v04)
- imf_statistics f — pinneapple_systems.time_series.solar_cycle (v06)
- ImmersedBoundaryFDMSolver C — pinneapple_simulation.numerical_solvers.immersed_boundary_fdm (v03)
- impact_gif f — pinneapple_design.thermal.board_cooling.report (v06)
- impact_gif f — pinneapple_systems.digital_twin.pipe_loop.report (v06)
- implicit_solve f — pinneapple_core.func (v01)
- ImplicitSurfaceRBF C — pinneapple_design.geometry.ops.meshfree (v06)
- implied_inflow f — pinneapple_physics.blackhole.skill (v02)
- import_viper f — pinneapple_simulation.rover.autolab.render (v03)
- improve f — pinneapple_lab.improve (v07)
- improvement_plan f — pinneapple_physics.physics_audit.improvement (v02)
- inclination_at_depth f — pinneapple_systems.process_components.curved_path_geometry (v06)
- include_dirs f — pinneapple_simulation.accelerated_foam.build (v03)
- incompressible f — pinneapple_physics.physics_audit.assumptions (v02)
- incompressible_continuity_residual f — pinneapple_systems.component_modeling.physics_residuals (v06)
- incompressible_mass_flow f — pinneapple_systems.process_components.control_valve (v06)
- index f — pinneapple_design.geometry_library.web (v06)
- index f — pinneapple_presets.registry (v02)
- industrial_furnace_thermal f — pinneapple_physics.pde_environment.presets.engineering (v02)
- ineq f — pinneapple_core.optim (v01)
- infeasibility_reasons f — pinneapple_design.aero.optimize (v06)
- Infeasible C — pinneapple_reason.library (v05)
- infer f — pinneapple_neural.predictor.infer (v04)
- infer f — pinneapple_tools.model_services.registry (v07)
- infer_allegro f — pinneapple_arena.external.runners.materials_md (v04)
- infer_astronomix f — pinneapple_arena.external.runners.dynamics_robotics_astro (v04)
- infer_bpinn_wildfire f — pinneapple_arena.external.runners.materials2_and_misc (v04)
- infer_brax f — pinneapple_arena.external.runners.rigidbody_robotics (v04)
- infer_camels f — pinneapple_arena.external.runners.quantum_cosmology (v04)
- infer_ceviche f — pinneapple_arena.external.runners.em_photonics (v04)
- infer_check_kind f — pinneapple_lab.curation (v07)
- infer_chgnet f — pinneapple_arena.external.runners.materials2_and_misc (v04)
- infer_crnn f — pinneapple_arena.external.runners.applied_domains (v04)
- infer_deepflame f — pinneapple_arena.external.runners.fluids_structural (v04)
- infer_deepmd_kit f — pinneapple_arena.external.runners.materials_md (v04)
- infer_deeponet_fno_benchmark f — pinneapple_arena.external.runners.pinn_general (v04)
- infer_deeponet_original f — pinneapple_arena.external.runners.pinn_general (v04)
- infer_deepwave f — pinneapple_arena.external.runners.geophysics_energy (v04)

- infer_deepxde f — pinneapple_arena.external.runners.pinn_general (v04)
- infer_devito f — pinneapple_arena.external.runners.geophysics_energy (v04)
- infer_diffpd f — pinneapple_arena.external.runners.dynamics_robotics_astro (v04)
- infer_diffrax f — pinneapple_arena.external.runners.dynamics_robotics_astro (v04)
- infer_diffsph f — pinneapple_arena.external.runners.dynamics_robotics_astro (v04)
- infer_difftaichi f — pinneapple_arena.external.runners.dynamics_robotics_astro (v04)
- infer_dl4to f — pinneapple_arena.external.runners.fluids_structural (v04)
- infer_dojo_jl f — pinneapple_arena.external.runners.dynamics_robotics_astro (v04)
- infer_dsgp4 f — pinneapple_arena.external.runners.dynamics_robotics_astro (v04)
- infer_epipinn f — pinneapple_arena.external.runners.applied_domains (v04)
- infer_fairchem f — pinneapple_arena.external.runners.materials2_and_misc (v04)
- infer_ferminet f — pinneapple_arena.external.runners.quantum_cosmology (v04)
- infer_flowpm_jaxpm f — pinneapple_arena.external.runners.quantum_cosmology (v04)
- infer_fluidx3d f — pinneapple_arena.external.runners.fluids_structural (v04)
- infer_fourcastnet f — pinneapple_arena.external.runners.weather_climate (v04)
- infer_genesis f — pinneapple_arena.external.runners.dynamics_robotics_astro (v04)
- infer_gns f — pinneapple_arena.external.runners.fluids_structural (v04)
- infer_graingnn f — pinneapple_arena.external.runners.fluids_structural (v04)
- infer_grainlearning f — pinneapple_arena.external.runners.materials2_and_misc (v04)
- infer_graphcast f — pinneapple_arena.external.runners.weather_climate (v04)
- infer_hpinn f — pinneapple_arena.external.runners.em_photonics (v04)
- infer_jax_cfd f — pinneapple_arena.external.runners.fluids_structural (v04)
- infer_jax_fluids f — pinneapple_arena.external.runners.fluids_structural (v04)
- infer_jax_md f — pinneapple_arena.external.runners.materials_md (v04)
- infer_lesnets f — pinneapple_arena.external.runners.fluids_structural (v04)
- infer_mace f — pinneapple_arena.external.runners.materials_md (v04)
- infer_mattersim f — pinneapple_arena.external.runners.materials2_and_misc (v04)
- infer_meshgraphnets_original f — pinneapple_arena.external.runners.fluids_structural (v04)
- infer_meshgraphnets_pytorch f — pinneapple_arena.external.runners.fluids_structural (v04)
- infer_ml4gw f — pinneapple_arena.external.runners.applied_domains (v04)
- infer_mujoco_mjx f — pinneapple_arena.external.runners.rigidbody_robotics (v04)
- infer_nequip f — pinneapple_arena.external.runners.materials_md (v04)
- infer_netket f — pinneapple_arena.external.runners.quantum_cosmology (v04)
- infer_neuralfoil f — pinneapple_arena.external.runners.applied_domains (v04)
- infer_neuralgcm f — pinneapple_arena.external.runners.weather_climate (v04)
- infer_neuraloperator f — pinneapple_arena.external.runners.pinn_general (v04)
- infer_neuralpde_jl f — pinneapple_arena.external.runners.pinn_general (v04)
- infer_neuralsolvers f — pinneapple_arena.external.runners.applied_domains (v04)
- infer_nimble_physics f — pinneapple_arena.external.runners.dynamics_robotics_astro (v04)
- infer_on_grid_1d f — pinneapple_neural.predictor.infer (v04)
- infer_on_grid_2d f — pinneapple_neural.predictor.infer (v04)
- infer_openfold f — pinneapple_arena.external.runners.materials2_and_misc (v04)
- infer_orb_models f — pinneapple_arena.external.runners.materials2_and_misc (v04)
- infer_pangu_weather f — pinneapple_arena.external.runners.weather_climate (v04)
- infer_phiflow f — pinneapple_arena.external.runners.fluids_structural (v04)
- infer_photontorch f — pinneapple_arena.external.runners.em_photonics (v04)
- infer_physics f — pinneapple_check.physics (v05)
- infer_physics_informed_am f — pinneapple_arena.external.runners.applied_domains (v04)
- infer_physicsnemo f — pinneapple_arena.external.runners.pinn_general (v04)
- infer_physnet_thermal f — pinneapple_arena.external.runners.applied_domains (v04)
- infer_piml_converter f — pinneapple_arena.external.runners.applied_domains (v04)
- infer_pina f — pinneapple_arena.external.runners.pinn_general (v04)
- infer_pinn_ac_opf f — pinneapple_arena.external.runners.applied_domains (v04)
- infer_pinn_corrosion_fatigue f — pinneapple_arena.external.runners.applied_domains (v04)
- infer_pinn_multivp f — pinneapple_arena.external.runners.applied_domains (v04)
- infer_pinn_option_pricing f — pinneapple_arena.external.runners.applied_domains (v04)

- `infer_pinn_swe_open` f — `pinneapple_arena.external.runners.applied_domains` (v04)
- `infer_pinn_tfi_hss` f — `pinneapple_arena.external.runners.pinn_general` (v04)
- `infer_pinn_wind_bearing` f — `pinneapple_arena.external.runners.applied_domains` (v04)
- `infer_pinnicle` f — `pinneapple_arena.external.runners.applied_domains` (v04)
- `infer_pinns_original` f — `pinneapple_arena.external.runners.pinn_general` (v04)
- `infer_pinns_torch` f — `pinneapple_arena.external.runners.pinn_general` (v04)
- `infer_pino` f — `pinneapple_arena.external.runners.pinn_general` (v04)
- `infer_pise` f — `pinneapple_arena.external.runners.applied_domains` (v04)
- `infer_pybamm` f — `pinneapple_arena.external.runners.geophysics_energy` (v04)
- `infer_pyroomacoustics` f — `pinneapple_arena.external.runners.applied_domains` (v04)
- `infer_pysindy` f — `pinneapple_arena.external.runners.dynamics_robotics_astro` (v04)
- `infer_quantity` f — `pinneapple_data.physical_units` (v04)
- `infer_reservoirflow` f — `pinneapple_arena.external.runners.geophysics_energy` (v04)
- `infer_rirpinn` f — `pinneapple_arena.external.runners.applied_domains` (v04)
- `infer_schnetpack` f — `pinneapple_arena.external.runners.materials_md` (v04)
- `infer_sciann` f — `pinneapple_arena.external.runners.pinn_general` (v04)
- `infer_simvascular` f — `pinneapple_arena.external.runners.applied_domains` (v04)
- `infer_socialforce` f — `pinneapple_arena.external.runners.applied_domains` (v04)
- `infer_spdiff` f — `pinneapple_arena.external.runners.applied_domains` (v04)
- `infer_stiff_pinn` f — `pinneapple_arena.external.runners.applied_domains` (v04)
- `infer_taichi` f — `pinneapple_arena.external.runners.fluids_structural` (v04)
- `infer_tensorcircuit_ng` f — `pinneapple_arena.external.runners.materials2_and_misc` (v04)
- `infer_torax` f — `pinneapple_arena.external.runners.geophysics_energy` (v04)
- `infer_torchmd` f — `pinneapple_arena.external.runners.materials2_and_misc` (v04)
- `infer_transolver` f — `pinneapple_arena.external.runners.materials2_and_misc` (v04)
- `infer_twophasepinn` f — `pinneapple_arena.external.runners.fluids_structural` (v04)
- `infer_u_no` f — `pinneapple_arena.external.runners.materials2_and_misc` (v04)
- `infer_veros` f — `pinneapple_arena.external.runners.dynamics_robotics_astro` (v04)
- `infer_wakenet` f — `pinneapple_arena.external.runners.materials2_and_misc` (v04)
- `infer_warp` f — `pinneapple_arena.external.runners.fluids_structural` (v04)
- `inference_latency` f — `pinneapple_tools.benchmark_suite.crossframework.darcy` (v07)
- `InferenceResult` C — `pinneapple_neural.predictor.infer` (v04)
- `infinite_well_potential` f — `pinneapple_quantum.loss.schrodinger` (v04)
- `infinite_wire_B` f — `pinneapple_simulation.electromagnetics.statics` (v03)
- `inflow` f — `pinneapple_tools.automation.verify` (v07)
- `info` f — `pinneapple` (v01)
- `info` f — `pinneapple_app.backend.main` (v07)
- `info` f — `pinneapple_design.geometry_library.core` (v06)
- `info` f — `pinneapple_hub.zoo` (v07)
- `info` f — `pinneapple_hub.zoo.datasets` (v07)
- `Informer` C — `pinneapple_neural.architectures.transformers.informer` (v04)
- `ingest` f — `pinneapple_flow.nodes.data` (v07)
- `init_logvar_bias` f — `pinneapple_neural.trainer.heteroscedastic` (v04)
- `init_workdir` f — `pinneapple_arena.autoresearch` (v04)
- `initial_state` f — `pinneapple_physics.astro.nbody` (v02)
- `initial_state` f — `pinneapple_simulation.numerical_solvers.symbolic_fd` (v03)
- `InitialCondition` f — `pinneapple_physics.pde_environment.conditions` (v02)
- `InitialConditionSpec` C — `pinneapple_app.backend.core.problem` (v07)
- `inject` f — `pinneapple_security.cps_bench` (v05)
- `inline_hashes` f — `pinneapple_security.web` (v05)
- `InnerLoops` C — `pinneapple_simulation.flight.control` (v03)
- `InputError` C — `pinneapple_analysis.inverse_problems.heat_transfer` (v05)
- `inputs_for_steady_state` f — `pinneapple_systems.operations.plant` (v06)
- `InputSpec` C — `pinneapple_tools.automation.workflow` (v07)
- `insert_nodes` f — `pinneapple_flow.lab_status` (v07)
- `inspect_fmu` f — `pinneapple_simulation.external_solvers.modelica.fmu_inspect` (v03)

- inspect_table f — pinneapple_analysis.surrogate.data (v05)
- inspection_dataset f — pinneapple_worldmodel.inspection_data (v04)
- inspection_sample f — pinneapple_worldmodel.inspection_data (v04)
- InspectionPINN C — pinneapple_analysis.inspection.base (v05)
- install_hint f — pinneapple_flow.palette (v07)
- install_state f — pinneapple_arena.external.status (v04)
- installed_cv f — pinneapple_systems.process_components.control_valve (v06)
- instantiate f — pinneapple_neural.architectures.instantiate (v04)
- instantiate_model f — pinneapple_arena.pointwise.training (v04)
- InstantiateReport C — pinneapple_neural.architectures.instantiate (v04)
- Instrument C — pinneapple_science.instruments (v02)
- integer_ticks f — pinneapple_science.plots (v02)
- integral f — pinneapple_physics.conservation (v02)
- IntegralObsOperator C — pinneapple_analysis.inverse_problems.obs_operator (v05)
- integrate f — pinneapple_core.operators (v01)
- integrate f — pinneapple_lab.experiments.ode (v07)
- integrate f — pinneapple_physics.astro.nbody (v02)
- integrate_network f — pinneapple_systems.process_components.reaction_kinetics (v06)
- integrate_p1 f — pinneapple_core.fem (v01)
- integrate_pressure_profile f — pinneapple_systems.process_components.non_newtonian_pipe_flow (v06)
- integrate_surface_forces f — pinneapple_tools.aero_coefficients (v07)
- IntegrationResult C — pinneapple_systems.process_components.reaction_kinetics (v06)
- Integrity C — pinneapple_check.config (v05)
- intent_service f — pinneapple_tools.model_services.registry (v07)
- IntentLLM C — pinneapple_design.intent.parse (v06)
- Interface C — pinneapple_design.bricks.statics (v06)
- interface_modes f — pinneapple_condense.substructure (v02)
- InterfaceBC f — pinneapple_physics.pde_environment.conditions (v02)
- IntermediateModelForecaster C — pinneapple_systems.time_series.enso.forecasters (v06)
- InternalFlow C — pinneapple_simulation.numerical_solvers.internal_flow (v03)
- InternalResult C — pinneapple_simulation.numerical_solvers.internal_flow (v03)
- interp_grid f — pinneapple_simulation.numerical_solvers.heat_fv_reference (v03)
- interpolate f — pinneapple_flow.nodes._common (v07)
- interpolate_on_triangles f — pinneapple_design.geometry.sample.barycentric (v06)
- interpolate_to_points f — pinneapple_simulation.numerical_solvers.immersed_boundary_fdm (v03)
- Interpolator C — pinneapple_data.cae.compare (v04)
- interpret f — pinneapple_tools.automation.interpret (v07)
- interpret_mission f — pinneapple_autolab.agent (v07)
- Interpretation C — pinneapple_tools.automation.interpret (v07)
- Intersection C — pinneapple_worldmodel.geometry (v04)
- intervals_from_labels f — pinneapple_systems.operations.anomaly (v06)
- intervention_checks f — pinneapple_analysis.validation.physics_learning (v05)
- InterventionReport C — pinneapple_analysis.verification.causal_discrepancy (v05)
- InterventionRunResult C — pinneapple_analysis.verification.causal_discrepancy (v05)
- intrinsic_coercivity f — pinneapple_simulation.electromagnetics.machines.thermal (v03)
- introspect f — pinneapple_flow.libcalls (v07)
- InvalidCell C — pinneapple_simulation.electromagnetics.periodic.tcda (v03)
- InvalidGeometry C — pinneapple_simulation.electromagnetics.antenna.ava (v03)
- Invariance C — pinneapple_analysis.validation.metamorphic (v05)
- Invariant C — pinneapple_llm.qa.spec (v07)
- invariant_check f — pinneapple_physics.physics_audit.monitor (v02)
- invariant_drift f — pinneapple_analysis.validation.rollout (v05)
- invariant_series f — pinneapple_analysis.validation.rollout (v05)
- inventory f — pinneapple_flow.examples (v07)
- inverse f — pinneapple_core.loss (v01)
- inverse_study f — pinneapple_analysis.inverse_problems.contracts (v05)

- `InverseConfig` C — `pinneapple_arena.config` (v04)
- `InverseDirichlet` C — `pinneapple_neural.trainer.loss_balancer` (v04)
- `InverseFit` C — `pinneapple_simulation.numerical_solvers.gated_xtfc` (v03)
- `InverseHeatProblem` C — `pinneapple_analysis.benchmark.builtin.inverse_heat` (v05)
- `InverseParam` C — `pinneapple_neural.trainer.inverse_pinn` (v04)
- `InverseParamAdapter` C — `pinneapple_analysis.verification.uncertainty_inverse` (v05)
- `InversePermittivity` C — `pinneapple_science.em.research` (v02)
- `InversePINN` C — `pinneapple_neural.architectures.pinns.inverse` (v04)
- `InversePINN` C — `pinneapple_neural.trainer.inverse_pinn` (v04)
- `InversePINNDataset` C — `pinneapple_worldmodel.training_hooks` (v04)
- `InverseProblem` C — `pinneapple_analysis.inverse_problems.studio` (v05)
- `InverseProblemResult` C — `pinneapple_analysis.verification.uncertainty_inverse` (v05)
- `InverseProblemSolver` C — `pinneapple_analysis.inverse_problems.solver` (v05)
- `InverseSolverConfig` C — `pinneapple_analysis.inverse_problems.solver` (v05)
- `InverseSolverResult` C — `pinneapple_analysis.inverse_problems.solver` (v05)
- `invert` f — `pinneapple_analysis` (v05)
- `involute_tooth_profile` f — `pinneapple_design.geometry.gen.cadquery_advanced` (v06)
- `IPMMachine` C — `pinneapple_simulation.electromagnetics.machines.pmsm` (v03)
- `IPMTraction` C — `pinneapple_presets.cases.ipm_traction` (v02)
- `iron_loss` f — `pinneapple_simulation.electromagnetics.postprocess` (v03)
- `iron_loss_components` f — `pinneapple_simulation.electromagnetics.postprocess` (v03)
- `iron_split` f — `pinneapple_simulation.electromagnetics.machines.thermal` (v03)
- `IronLossModel` C — `pinneapple_simulation.electromagnetics.postprocess` (v03)
- `is_artifact` f — `pinneapple_flow.runtime` (v07)
- `is_auto` f — `pinneapple_flow.palette` (v07)
- `is_buildable` f — `pinneapple_simulation.electromagnetics.antenna.design` (v03)
- `is_closed_volume` f — `pinneapple_design.geometry_library.kit` (v06)
- `is_deepxde_available` f — `pinneapple_physics.deepxde_bridge.adapter` (v02)
- `is_distributed` f — `pinneapple_neural.trainer.distributed` (v04)
- `is_graph_model` f — `pinneapple_arena.model_factory` (v04)
- `is_hash` f — `pinneapple_security.passwords` (v05)
- `is_installed` f — `pinneapple_arena.external.installer` (v04)
- `is_inverse_model` f — `pinneapple_arena.model_factory` (v04)
- `is_offline` f — `pinneapple_hub.zoo._artifacts` (v07)
- `is_operator_model` f — `pinneapple_arena.model_factory` (v04)
- `is_parameter_space` f — `pinneapple_analysis.study_contracts` (v05)
- `is_passive` f — `pinneapple_simulation.electromagnetics.media` (v03)
- `is_physicsnemo_available` f — `pinneapple_neural.physicsnemo_bridge.adapters` (v04)
- `is_pinn_model` f — `pinneapple_arena.model_factory` (v04)
- `is_result_file` f — `pinneapple_data.preflight` (v04)
- `isa` f — `pinneapple_design.aero.airliner3d` (v06)
- `isa_density` f — `pinneapple_design.aero.aircraft` (v06)
- `isaac_replay_script` f — `pinneapple_tools.nvidia.isaac` (v07)
- `IsaacSimBackend` C — `pinneapple_simulation.robot_sim.sim` (v03)
- `isentropic_ratios` f — `pinneapple_physics.closed_form.compressible` (v02)
- `ishigami` f — `pinneapple_analysis.sensitivity.testfunctions` (v05)
- `ishigami_indices` f — `pinneapple_analysis.sensitivity.testfunctions` (v05)
- `ishigami_space` f — `pinneapple_analysis.sensitivity.testfunctions` (v05)
- `isolated` f — `pinneapple_condense.benchmark` (v02)
- `IsolationForestDetector` C — `pinneapple_security.cps_bench` (v05)
- `IsolationLevel` C — `pinneapple_llm.qa.sandbox` (v07)
- `isotropic_line_spectra` f — `pinneapple_simulation.flight.turbulence` (v03)
- `ISPHSolver` C — `pinneapple_simulation.numerical_solvers.isph` (v03)
- `Issue` C — `pinneapple_data.data_health` (v04)
- `Issue` C — `pinneapple_flow.schema` (v07)
- `Item` C — `pinneapple_design.bricks.visuals` (v06)

- ItemAdapter C — pinneapple_neural.trainer.datamodule (v04)
- items f — pinneapple_lab.seal (v07)
- iter_chunks f — pinneapple_data.chunking (v04)
- iter_known_answers f — pinneapple_reason.library (v05)
- iter_repo_files f — pinneapple_analysis.retrieval.corpus (v05)
- iter_tiles f — pinneapple_pdb.shard (v04)
- iter_time_windows f — pinneapple_pdb.shard (v04)
- iterate_refinement f — pinneapple_condense.substructure (v02)
- IteratedEKI C — pinneapple_analysis.inverse_problems.ensemble_kalman (v05)
- IterationRecord C — pinneapple_problemdesign.research_loop (v02)
- ito_bend_loss f — pinneapple_simulation.numerical_solvers.internal_flow (v03)
- itrs_to_geodetic f — pinneapple_physics.astro.frames (v02)

J

- jacfwd f — pinneapple_core.func (v01)
- jackknife f — pinneapple_analysis.uncertainty.resampling (v05)
- jacobi f — pinneapple_simulation.accelerated_foam.solvers (v03)
- jacobian f — pinneapple_analysis.inverse_problems.forward (v05)
- jacobian f — pinneapple_core.func (v01)
- jacobian f — pinneapple_core.operators (v01)
- jacobian f — pinneapple_physics.pinn_solver.compiler.autograd_ops (v02)
- jacrev f — pinneapple_core.func (v01)
- jax_available f — pinneapple_tools.compute_backends.jax_backend (v07)
- jax_cfd_available f — pinneapple_simulation.numerical_solvers.jax_cfd_solver (v03)
- jax_pinn f — pinneapple_tools.compute_backends.jax_backend (v07)
- JAXBackend C — pinneapple_tools.compute_backends.jax_backend (v07)
- JAXCFDSolver C — pinneapple_simulation.numerical_solvers.jax_cfd_solver (v03)
- JaxPhysicsBackend C — pinneapple_core.backend (v01)
- JAXPINNBackend C — pinneapple_tools.benchmark_suite.backends.jax_pinn (v07)
- jd_from_datetime f — pinneapple_physics.astro.timescales (v02)
- jet f — pinneapple_tools.visualization.studio.colormap (v07)
- JilesAtherton C — pinneapple_simulation.electromagnetics.hysteresis (v03)
- jit_pinn f — pinneapple_tools.compute_backends.jax_backend (v07)
- job_checks f — pinneapple_analysis.benchmark.checks (v05)
- job_status f — pinneapple_mcp.tools (v07)
- JobCancelled C — pinneapple_core.service.jobs (v01)
- JobContext C — pinneapple_core.service.jobs (v01)
- JobQueue C — pinneapple_core.service.jobs (v01)
- JobTimeout C — pinneapple_core.service.jobs (v01)
- Joint C — pinneapple_simulation.external_solvers.newton_physics.scene (v03)
- Joint C — pinneapple_simulation.robot_sim.spec (v03)
- JointAdaptiveWeights C — pinneapple_neural.trainer.loss_balancer (v04)
- JointModel C — pinneapple_design.bricks.parts (v06)
- joukowsky_head f — pinneapple_simulation.numerical_solvers.water_hammer_moc (v03)
- joukowsky_head_rise f — pinneapple_systems.process_components.water_hammer (v06)
- json_schemas f — pinneapple_data.contracts.models (v04)
- jsonable f — pinneapple_science.core (v02)
- jsonable f — pinneapple_tools.model_services.registry (v07)
- julian_centuries f — pinneapple_physics.astro.timescales (v02)
- junction_temperature_c f — pinneapple_design.surrogate_inverse.heatsink_fins (v06)
- junit f — pinneapple_check.report (v05)

K

- k_anonymity f — pinneapple_security.privacy (v05)
- KAEAutoencoder C — pinneapple_neural.architectures.autoencoders.kae (v04)

- KafkaStream C — pinneapple_systems.digital_twin.io.stream (v06)
- kalman f — pinneapple_flow.nodes.methods (v07)
- kalman_smooth f — pinneapple_systems.sports.football.kinematics (v06)
- KalmanFilter C — pinneapple_neural.architectures.classical_ts.kalman (v04)
- KBArtifact C — pinneapple_tools.hpo_experiments.models (v07)
- KBIndex C — pinneapple_tools.hpo_experiments.models (v07)
- KBStore C — pinneapple_tools.hpo_experiments.kb.store (v07)
- KdV1D C — pinneapple_arena.problems (v04)
- keep_largest_component f — pinneapple_design.geometry.ops.clean (v06)
- keff_maps f — pinneapple_condense.figures (v02)
- kelvin_wave_test f — pinneapple_systems.time_series.enso.intermediate (v06)
- kenics_elements f — pinneapple_simulation.numerical_solvers.internal_flow (v03)
- kepler_exact_state f — pinneapple_physics.pde_environment.presets.astrophysics (v02)
- kepler_positions f — pinneapple_physics.astro.neural_propagator (v02)
- kepler_two_body_orbit f — pinneapple_physics.pde_environment.presets.astrophysics (v02)
- KeplerLaw C — pinneapple_lab.experiments.discovery (v07)
- kern_shell f — pinneapple_physics.closed_form.heat_exchanger (v02)
- key f — pinneapple_hub.library.categories (v07)
- kind_of f — pinneapple_data.cae.io (v04)
- kinematic_extrapolation f — pinneapple_systems.sports.football.ball (v06)
- Kinematics C — pinneapple_simulation.multibody.tree (v03)
- kinetic_energy f — pinneapple_simulation.flight.rigid_body (v03)
- kirchhoff_disc_capacitance f — pinneapple_science.em.statics (v02)
- kit_render_script f — pinneapple_tools.nvidia.omniverse (v07)
- kl_diag_gaussians f — pinneapple_neural.architectures.continuous.deep_state_space (v04)
- KleinGordon1D C — pinneapple_arena.problems (v04)
- knn f — pinneapple_condense.meshless2d (v02)
- known_actions f — pinneapple_tools.automation.adapters.base (v07)
- known_answer_checks f — pinneapple_reason.trust (v05)
- knudsen f — pinneapple_physics.physics_audit.regime (v02)
- knuth_histogram f — pinneapple_physics.pde_environment.presets.astroml_bridge (v02)
- KOmegaSSTResiduals C — pinneapple_physics.pde_environment.turbulence_presets (v02)
- KoopmanAutoencoder C — pinneapple_neural.architectures.rom.koopman (v04)
- KoopmanOperator C — pinneapple_neural.architectures.reservoir_computing.koopman (v04)
- KoopmanOperator C — pinneapple_neural.architectures.rom.koopman (v04)
- kovasznay f — pinneapple_analysis.validation.pinn (v05)
- KovasznayNS C — pinneapple_arena.problems (v04)
- kp_needed f — pinneapple_physics.astro.spaceweather (v02)
- KPI C — pinneapple_tools.reporting.datasheet (v07)
- kpss_test f — pinneapple_systems.time_series.audit.tests (v06)
- KuramotoSivashinsky1D C — pinneapple_simulation.numerical_solvers.kuramoto_sivashinsky (v03)

L

- l2 f — pinneapple_physics.metrics (v02)
- l_bracket f — pinneapple_tools.nvidia.parts (v07)
- l_bracket_volume f — pinneapple_tools.nvidia.parts (v07)
- l_mesh f — pinneapple_design.intent.physics.bracket (v06)
- lab f — pinneapple_flow.nodes.analysis (v07)
- lab_export_status f — pinneapple_flow.runtime (v07)
- lab_item f — pinneapple_hub.library.snippets (v07)
- lab_items f — pinneapple_flow.lab_status (v07)
- lab_items_for f — pinneapple_lab.manifest (v07)
- lab_toolbox f — pinneapple_autolab.agent (v07)
- label_check f — pinneapple_worldmodel.inspection_data (v04)
- LabelledInterval C — pinneapple_systems.operations.anomaly (v06)
- LabPlanner C — pinneapple_autolab.agent (v07)

- LabReport C — pinneapple_veriphysics.validation_lab (v05)
- LabServer C — pinneapple_lab.server (v07)
- LabStore C — pinneapple_lab.store (v07)
- ladder_rail f — pinneapple_design.geometry.vehicle_cad (v06)
- ladder_values f — pinneapple_science.em.fdtd_study (v02)
- LagFeatures C — pinneapple_systems.time_series.forecast_lab.learned (v06)
- lagrange_shape_functions_1d f — pinneapple_simulation.numerical_solvers.nonlinear_beam_fem (v03)
- LagRegressorExpert C — pinneapple_systems.time_series.adaptive (v06)
- Lambda C — pinneapple_core.module (v01)
- lame f — pinneapple_presets.cases.pressure_vessel (v02)
- lame_analytical f — pinneapple_physics.pde_environment.presets.solid_mechanics (v02)
- lame_hoop_stress_inner f — pinneapple_systems.process_components.pipe_stress_mechanics (v06)
- lame_hoop_stress_outer f — pinneapple_systems.process_components.pipe_stress_mechanics (v06)
- lame_parameters f — pinneapple_physics.solid.axisymmetric (v02)
- lame_parameters f — pinneapple_simulation.numerical_solvers.elasticity3d_fdm (v03)
- laminar f — pinneapple_physics.closed_form.pipe_flow (v02)
- laminar f — pinneapple_physics.physics_audit.assumptions (v02)
- landscape_tiles f — pinneapple_systems.time_series.forecast_lab.renders (v06)
- lane_emden_exact_theta f — pinneapple_physics.pde_environment.presets.astrophysics (v02)
- lane_emden_polytrope f — pinneapple_physics.pde_environment.presets.astrophysics (v02)
- langevin f — pinneapple_simulation.electromagnetics.hysteresis (v03)
- lap_heads f — pinneapple_reason.modes.water (v05)
- Laplace2D C — pinneapple_arena.problems (v04)
- laplace_2d_default f — pinneapple_physics.pde_environment.presets.academics (v02)
- laplace_beltrami f — pinneapple_design.geometry.ops.curvature (v06)
- laplace_mechanism f — pinneapple_security.dp (v05)
- laplacian f — pinneapple_core.operators (v01)
- laplacian f — pinneapple_physics.pinn_solver.compiler.autograd_ops (v02)
- LargePhysicsModel C — pinneapple_neural.lpm (v04)
- largest_lyapunov_benettin f — pinneapple_systems.time_series.synthetic (v06)
- LastStepToHorizon C — pinneapple_systems.time_series.adapters.shape (v06)
- LatencyBudgetExceeded C — pinneapple_systems.digital_twin.realtime_pipeline (v06)
- LatencyStats C — pinneapple_systems.digital_twin.realtime_pipeline (v06)
- LatentConditionedModel C — pinneapple_physics.pinn_solver.stochastic (v02)
- LatentODE C — pinneapple_neural.architectures.continuous.latent_ode (v04)
- LatentStateTwin C — pinneapple_systems.digital_twin.product.assimilation (v06)
- lateral_figure f — pinneapple_simulation.flight.figures (v03)
- latest_time f — pinneapple_simulation.numerical_solvers.external_flow (v03)
- latin_hypercube f — pinneapple_design.aero.geometry (v06)
- latin_hypercube f — pinneapple_neural.architectures.rom.parametric (v04)
- latin_hypercube f — pinneapple_simulation.external_solvers.modelica.sweep (v03)
- Lattice C — pinneapple_design.aero.vlm (v06)
- lattice_check f — pinneapple_simulation.flight.verification (v03)
- lattice_order f — pinneapple_analysis.surrogate.data (v05)
- lattice_surface f — pinneapple_simulation.flight.visuals (v03)
- launch_experiment f — pinneapple_app.backend.routers.experiments (v07)
- LaunchVehicle C — pinneapple_design.geometry.vehicles3d (v06)
- Layer C — pinneapple_condense.package (v02)
- Layer C — pinneapple_physics.closed_form.pcb_thermal (v02)
- Layer C — pinneapple_simulation.electromagnetics.analytic (v03)
- layer_graph f — pinneapple_flow.learn (v07)
- layer_lr_groups f — pinneapple_adaptation.transfer_learning.freeze (v04)
- layer_sizes f — pinneapple_tools.benchmark_suite.crossframework.weights (v07)
- layer_total f — pinneapple_design.fitmesh.layers (v06)
- layered_disc f — pinneapple_design.surrogate_inverse.meshes (v06)
- layered_modes f — pinneapple_condense.substructure (v02)

- Layout C — pinneapple_design.aero.surrogate (v06)
- Layout C — pinneapple_simulation.electromagnetics.antenna.ava (v03)
- layout f — pinneapple_flow.palette (v07)
- layout f — pinneapple_flow.planner.build (v07)
- layout f — pinneapple_simulation.electromagnetics.antenna.ava (v03)
- layout_plot f — pinneapple_simulation.electromagnetics.antenna.plots (v03)
- lbfgs f — pinneapple_analysis.validation.arena.search (v05)
- LBFGSFinetuneResult C — pinneapple_neural.trainer.lbfgs_finetune (v04)
- lbm_channel f — pinneapple_flow.wrappers (v07)
- LBMSolver C — pinneapple_simulation.numerical_solvers.lbm (v03)
- LBMSolver3D C — pinneapple_simulation.numerical_solvers.lbm (v03)
- LCurveResult C — pinneapple_analysis.inverse_problems.regularization (v05)
- LCurveSelector C — pinneapple_analysis.inverse_problems.regularization (v05)
- LduSystem C — pinneapple_simulation.accelerated_foam.ldu (v03)
- lead_time_scores f — pinneapple_physics.blackhole.forecast (v02)
- leaf f — pinneapple_check._reuse (v05)
- leakage f — pinneapple_data.audit (v04)
- learn_hamiltonian f — pinneapple_quantum.pipeline.presets (v04)
- learn_planet f — pinneapple_flow.nodes.vehicle (v07)
- learned_study f — pinneapple_condense.benchmark (v02)
- LearnedBlocks C — pinneapple_condense.learned (v02)
- LearnedClosure C — pinneapple_physics.multiscale.learned (v02)
- LearnedWarmStart C — pinneapple_simulation.accelerated_foam.learned (v03)
- least_squares_fit f — pinneapple_analysis.inverse_problems.estimation (v05)
- Ledger C — pinneapple_autolab.ledger (v07)
- legend_seals f — pinneapple_lab.seal (v07)
- legendre_design f — pinneapple_analysis.uncertainty.pce (v05)
- legendre_tables f — pinneapple_simulation.geophysics.sphere_shallow_water (v03)
- lego_brick f — pinneapple_design.geometry.gen.cadquery_advanced (v06)
- length_factor f — pinneapple_tools.visualization.studio.fieldview (v07)
- lensed_sky f — pinneapple_physics.astro.lensing (v02)
- lensing_view f — pinneapple_physics.astro.lensing (v02)
- level_description f — pinneapple_llm.qa.levels (v07)
- lexical_overlap_score f — pinneapple_tools.hpo_experiments.ranking (v07)
- lhs f — pinneapple_presets.core (v02)
- li_forcing f — pinneapple_simulation.numerical_solvers.operator_benchmarks (v03)
- LibCall C — pinneapple_flow.libcalls (v07)
- Library C — pinneapple_reason.library (v05)
- library_specs f — pinneapple_flow.libcalls (v07)
- licence_counts f — pinneapple_arena.external (v04)
- LicenseGateError C — pinneapple_tools.rea_bridge.harvest (v07)
- LicensePolicy C — pinneapple_tools.rea_bridge.licenses (v07)
- lid_driven_cavity_3d f — pinneapple_physics.pde_environment.presets.cfd (v02)
- LidDrivenCavity3DTask C — pinneapple_tools.benchmark_suite.tasks.lid_driven_cavity_3d (v07)
- LidDrivenCavityConfig3D C — pinneapple_simulation.numerical_solvers.fdm3d (v03)
- LidDrivenCavityDomain2D C — pinneapple_design.geometry.gen.domains (v06)
- LidDrivenCavityDomain3D C — pinneapple_design.geometry.gen.domains3d (v06)
- LidDrivenCavitySolver3D C — pinneapple_simulation.numerical_solvers.fdm3d (v03)
- light_curve_sample f — pinneapple_physics.pde_environment.presets.astroml_bridge (v02)
- light_ga f — pinneapple_simulation.flight.catalog (v03)
- light_time_and_doppler f — pinneapple_physics.astro.orbits (v02)
- LightGBMForecaster C — pinneapple_systems.time_series.models.classical (v06)
- LightingPreset C — pinneapple_tools.visualization.studio.lighting (v07)
- LIMForecaster C — pinneapple_systems.time_series.enso.forecasters (v06)
- Limit C — pinneapple_autolab.mission (v07)
- limitations f — pinneapple_analysis.surrogate.limits (v05)

- `limits_from_env` f — `pinneapple_core.service.metering` (v01)
- `limits_report` f — `pinneapple_simulation.electromagnetics.antenna.study` (v03)
- `line_charge_field` f — `pinneapple_simulation.electromagnetics.statics` (v03)
- `line_of` f — `pinneapple_data.preflight.common` (v04)
- `Lineage` C — `pinneapple_data.lineage.graph` (v04)
- `linear_closure` f — `pinneapple_systems.process_components.water_hammer` (v06)
- `linear_combination_rate_constant` f — `pinneapple_systems.process_components.reaction_kinetics` (v06)
- `linear_elastic` f — `pinneapple_physics.physics_audit.assumptions` (v02)
- `linear_elasticity_3d` f — `pinneapple_physics.pde_environment.presets.structural` (v02)
- `linear_elasticity_3d_default` f — `pinneapple_physics.pde_environment.presets.industry` (v02)
- `linear_elasticity_residual` f — `pinneapple_systems.component_modeling.physics_residuals` (v06)
- `linear_fit` f — `pinneapple_science.instruments` (v02)
- `linear_sde_stepper` f — `pinneapple_systems.time_series.synthetic` (v06)
- `LinearBVP1D` C — `pinneapple_simulation.numerical_solvers.gated_xtfc` (v03)
- `LinearDQ` C — `pinneapple_simulation.electromagnetics.machines.drive` (v03)
- `LinearElasticity2D` C — `pinneapple_arena.problems` (v04)
- `LinearFit` C — `pinneapple_science.instruments` (v02)
- `LinearMaterial` C — `pinneapple_simulation.electromagnetics.materials` (v03)
- `LinearMPC` C — `pinneapple_systems.operations.control` (v06)
- `LinearObsOperator` C — `pinneapple_analysis.inverse_problems.obs_operator` (v05)
- `LinearPhysics` C — `pinneapple_simulation.electromagnetics.antenna.surrogates` (v03)
- `LinearSPDProblem` C — `pinneapple_analysis.uncertainty.residual_bound` (v05)
- `LinearSSMForecaster` C — `pinneapple_systems.time_series.forecast_lab.physical` (v06)
- `LineFit` C — `pinneapple_physics.pde_environment.presets.astroml_bridge` (v02)
- `Lines` C — `pinneapple_tools.visualization.studio.scene` (v07)
- `Link` C — `pinneapple_simulation.robot_sim.spec` (v03)
- `link` f — `pinneapple_flow.references` (v07)
- `link_frames` f — `pinneapple_simulation.rover.geometry` (v03)
- `link_root_checks` f — `pinneapple_simulation.multibody.arm` (v03)
- `link_world_points` f — `pinneapple_simulation.multibody.arm` (v03)
- `Liquid` C — `pinneapple_simulation.numerical_solvers.particles` (v03)
- `list` f — `pinneapple_design.geometry_library` (v06)
- `list` f — `pinneapple_hub.zoo` (v07)
- `list` f — `pinneapple_hub.zoo.datasets` (v07)
- `list_adapters` f — `pinneapple_tools.automation.adapters.base` (v07)
- `list_analytic` f — `pinneapple_physics.solving` (v02)
- `list_available_solver_families` f — `pinneapple_analysis.verification.solver_orchestration` (v05)
- `list_backends` f — `pinneapple_core.backend` (v01)
- `list_backends` f — `pinneapple_tools.benchmark_suite.registry` (v07)
- `list_benchmarks` f — `pinneapple_arena.dataset_bench` (v04)
- `list_benchmarks` f — `pinneapple_pdb.benchmarks` (v04)
- `list_builders` f — `pinneapple_design.geometry.gen.primitives` (v06)
- `list_buy_vs_build_needs` f — `pinneapple_analysis.verification.tool_recommendation` (v05)
- `list_cards` f — `pinneapple_physics.astro.facts` (v02)
- `list_datasets` f — `pinneapple_data.datasets` (v04)
- `list_datasets` f — `pinneapple_simulation.terramechanics.testbed` (v03)
- `list_domains` f — `pinneapple_design.geometry.gen.domains` (v06)
- `list_domains_3d` f — `pinneapple_design.geometry.gen.domains3d` (v06)
- `list_experiments` f — `pinneapple_app.backend.routers.experiments` (v07)
- `list_families` f — `pinneapple_app.backend.routers.models` (v07)
- `list_field_vars` f — `pinneapple_simulation.numerical_solvers.adapters.upd_signal` (v03)
- `list_jobs` f — `pinneapple_mcp.jobs` (v07)
- `list_jobs` f — `pinneapple_mcp.tools` (v07)
- `list_kinds` f — `pinneapple_physics.solving` (v02)
- `list_methods` f — `pinneapple_catalog.methods` (v01)
- `list_methods` f — `pinneapple_core.optim` (v01)

- list_methods f — pinneapple_mcp.tools (v07)
- list_methods f — pinneapple_physics.solving (v02)
- list_metrics f — pinneapple_app.backend.routers.models (v07)
- list_models f — pinneapple_app.backend.core.model_catalog (v07)
- list_models f — pinneapple_app.backend.routers.models (v07)
- list_pde_families f — pinneapple_physics.pde_environment.capabilities (v02)
- list_pde_rules f — pinneapple_core.transforms (v01)
- list_presets f — pinneapple_analysis.inverse_problems.presets (v05)
- list_presets f — pinneapple_mcp.tools (v07)
- list_presets f — pinneapple_physics.pde_environment.presets.registry (v02)
- list_presets f — pinneapple_presets.registry (v02)
- list_problems f — pinneapple_app.backend.routers.problems (v07)
- list_problems f — pinneapple_arena.problems (v04)
- list_problems_by_domain f — pinneapple_arena.problems (v04)
- list_resources f — pinneapple_catalog.resources (v01)
- list_signal_axes f — pinneapple_simulation.numerical_solvers.adapters.upd_signal (v03)
- list_soils f — pinneapple_simulation.terramechanics.soils (v03)
- list_specs f — pinneapple_data.formfill.specs (v04)
- list_strategies f — pinneapple_core.loss (v01)
- list_tasks f — pinneapple_tools.benchmark_suite.registry (v07)
- live_state f — pinneapple_autolab.dashboard (v07)
- LiveLineSampler C — pinneapple_simulation.numerical_solvers.free_surface (v03)
- LiveTwin C — pinneapple_systems.operations.live (v06)
- lj_force_magnitude f — pinneapple_simulation.particle_dynamics.molecular_dynamics (v03)
- lj_gas f — pinneapple_flow.nodes_ext.solvers (v07)
- lj_potential f — pinneapple_simulation.particle_dynamics.molecular_dynamics (v03)
- llm_decision f — pinneapple_flow.decisions (v07)
- llm_schema f — pinneapple_design.intent.parse (v06)
- LLMAdvisor C — pinneapple_reason.llm (v05)
- LLMAssist C — pinneapple_design.fitmesh.llm (v06)
- LLMMessage C — pinneapple_problemdesign.protocol (v02)
- LLMPlanner C — pinneapple_autolab.core (v07)
- LLMProposer C — pinneapple_arena.autoresearch (v04)
- LLMProvider C — pinneapple_problemdesign.protocol (v02)
- LLMProvider C — pinneapple_problemdesign.providers (v02)
- LLMResponse C — pinneapple_problemdesign.protocol (v02)
- LLMStepPlanner C — pinneapple_autolab.core (v07)
- lmtd_correction_1_2 f — pinneapple_physics.closed_form.heat_exchanger (v02)
- Load C — pinneapple_design.bricks.statics (v06)
- load f — pinneapple_analysis.benchmark.api (v05)
- load f — pinneapple_data.contracts.models (v04)
- load f — pinneapple_design.geometry_library.store (v06)
- load f — pinneapple_flow.autocalls (v07)
- load f — pinneapple_flow.learn (v07)
- load f — pinneapple_hub.zoo (v07)
- load f — pinneapple_hub.zoo.datasets (v07)
- load f — pinneapple_lab.manifest (v07)
- load f — pinneapple_presets.registry (v02)
- load f — pinneapple_science.atlas (v02)
- load_agents f — pinneapple_llm.workbench.agents (v07)
- load_app f — pinneapple_flow.studio (v07)
- load_arrays f — pinneapple_check.fields (v05)
- load_artifact f — pinneapple_flow.runtime (v07)
- load_baseline f — pinneapple_check.baseline (v05)
- load_body f — pinneapple_physics.astro.smallbodies (v02)
- load_bright_stars f — pinneapple_physics.astro.stars (v02)

- `load_bundle` f — `pinneapple_tools.benchmark_suite.bundle.loader` (v07)
- `load_bundle_schema` f — `pinneapple_tools.benchmark_suite.bundle.schema` (v07)
- `load_caption_index` f — `pinneapple_design.geometry.retrieval` (v06)
- `load_cases` f — `pinneapple_analysis.validation.arena.cases` (v05)
- `load_catalog` f — `pinneapple_worldmodel.generate_datasets` (v04)
- `load_checkpoint` f — `pinneapple_neural.trainer.checkpoint` (v04)
- `load_checkpoint` f — `pinneapple_neural.trainer.inference` (v04)
- `load_config` f — `pinneapple_check.config` (v05)
- `load_constellations` f — `pinneapple_physics.astro.stars` (v02)
- `load_curation` f — `pinneapple_lab.seal` (v07)
- `load_darcy` f — `pinneapple_tools.benchmark_suite.crossframework.darcy` (v07)
- `load_dataset` f — `pinneapple_data.datasets` (v04)
- `load_dataset` f — `pinneapple_physics.pde_environment.presets.astroml_bridge` (v02)
- `load_dataset` f — `pinneapple_simulation.terramechanics.testbed` (v03)
- `load_defence` f — `pinneapple_analysis.validation.arena.cases` (v05)
- `load_dense_volume_operator` f — `pinneapple_neural.workflows.dense_volume_operator` (v04)
- `load_dense_volumes` f — `pinneapple_data.adapters.splash_archive_adapter` (v04)
- `load_dir` f — `pinneapple_hub.zoo._models` (v07)
- `load_dotenv` f — `pinneapple_flow.config` (v07)
- `load_exported_odb_npz` f — `pinneapple_simulation.external_solvers.cfd_formats.abaqus_reader` (v03)
- `load_forecaster` f — `pinneapple_physics.blackhole.forecast` (v02)
- `load_geometry_asset` f — `pinneapple_data.adapters.geom_adapter` (v04)
- `load_geometry_asset` f — `pinneapple_design.geometry.core.registry` (v06)
- `load_hdf5` f — `pinneapple_data.serialization` (v04)
- `load_history` f — `pinneapple_physics.astro.spaceweather` (v02)
- `load_index` f — `pinneapple_hub.library` (v07)
- `load_into_linears` f — `pinneapple_tools.benchmark_suite.crossframework.weights` (v07)
- `load_json_file` f — `pinneapple_llm.qa.static` (v07)
- `load_leaderboard` f — `pinneapple_tools.benchmark_suite.runner.leaderboard` (v07)
- `load_manifest` f — `pinneapple_data.serialization` (v04)
- `load_mat` f — `pinneapple_simulation.external_solvers.matlab.mat_io` (v03)
- `load_mesh` f — `pinneapple_data.adapters.splash_archive_adapter` (v04)
- `load_mesh` f — `pinneapple_simulation.external_solvers.openfoam.mesh_reader` (v03)
- `load_mesh_cache` f — `pinneapple_data.adapters.splash_archive_adapter` (v04)
- `load_meshio` f — `pinneapple_check.fields` (v05)
- `load_meshio` f — `pinneapple_design.geometry.io.meshio_bridge` (v06)
- `load_model` f — `pinneapple_check.models` (v05)
- `load_model` f — `pinneapple_hub.zoo._models` (v07)
- `load_model` f — `pinneapple_simulation.external_solvers.mujoco.loader` (v03)
- `load_model` f — `pinneapple_systems.operations.contracts` (v06)
- `load_object` f — `pinneapple_flow.nodes_ext.examples` (v07)
- `load_openfoam` f — `pinneapple_check.fields` (v05)
- `load_package` f — `pinneapple_simulation.electromagnetics.antenna.package` (v03)
- `load_parametric_dense_volume_operator` f — `pinneapple_neural.workflows.parametric_dense_volume_operator` (v04)
- `load_pinneapple_dataset` f — `pinneapple_arena.trainer` (v04)
- `load_point_set` f — `pinneapple_tools.benchmark_suite.crossframework.problems` (v07)
- `load_predictor` f — `pinneapple_simulation.external_solvers.calculix.virtual_ccx` (v03)
- `load_preset` f — `pinneapple_app.backend.core.problem` (v07)
- `load_pt` f — `pinneapple_data.serialization` (v04)
- `load_qoi` f — `pinneapple_check.fields` (v05)
- `load_real_dataset` f — `pinneapple_data.datasets.real_world` (v04)
- `load_result` f — `pinneapple_check.fields` (v05)
- `load_reviews` f — `pinneapple_lab.curation` (v07)
- `load_run` f — `pinneapple_flow.nodes.vehicle` (v07)
- `load_run` f — `pinneapple_physics.blackhole.runs` (v02)

- `load_runs` f — `pinneapple_analysis.benchmark.builtin.airfoil_impl` (v05)
- `load_runs` f — `pinneapple_design.aero.train` (v06)
- `load_safetensors` f — `pinneapple_hub.zoo.export` (v07)
- `load_snapshots` f — `pinneapple_data.snapshots` (v04)
- `load_solution` f — `pinneapple_flow.nodes._common` (v07)
- `load_source` f — `pinneapple_veriphysics.evidence_package` (v05)
- `load_split` f — `pinneapple_tools.benchmark_suite.crossframework.darcy` (v07)
- `load_stl` f — `pinneapple_data.stl_import` (v04)
- `load_stl` f — `pinneapple_design.geometry.io.stl` (v06)
- `load_stl_bytes` f — `pinneapple_data.stl_import` (v04)
- `load_sunspots_yearly` f — `pinneapple_systems.time_series.datasets.public` (v06)
- `load_surrogate` f — `pinneapple_presets.surrogate` (v02)
- `load_surrogate` f — `pinneapple_simulation.terramechanics.surrogate` (v03)
- `load_surrogates` f — `pinneapple_design.thermal.heatsink.surrogate` (v06)
- `load_table` f — `pinneapple_data.data_health` (v04)
- `load_timeseries_csv` f — `pinneapple_systems.time_series.csv_loader` (v06)
- `load_trajectories` f — `pinneapple_physics.blackhole.dataset` (v02)
- `load_upd_item` f — `pinneapple_data.adapters.upd_adapter` (v04)
- `load_usd_eur_daily` f — `pinneapple_systems.time_series.datasets.public` (v06)
- `load_vlm` f — `pinneapple_tools.dataset_quality.vlm_curation` (v07)
- `load_yaml` f — `pinneapple_tools.benchmark_suite.io.yamlx` (v07)
- `load_youtube_trending_us` f — `pinneapple_systems.time_series.datasets.public` (v06)
- `load_zarr` f — `pinneapple_data.serialization` (v04)
- `loadavg` f — `pinneapple_tools.benchmark_suite.crossframework.records` (v07)
- `LoadedModel` C — `pinneapple_check.models` (v05)
- `LoadError` C — `pinneapple_check.fields` (v05)
- `LoadInfo` C — `pinneapple_data.data_health` (v04)
- `local_command_for` f — `pinneapple_arena.external.local_command` (v04)
- `local_commands` f — `pinneapple_arena.external.dataset_probe` (v04)
- `local_derivatives` f — `pinneapple_analysis.sensitivity.effects` (v05)
- `LocalDB` C — `pinneapple_core.service.db` (v01)
- `LocalExecutor` C — `pinneapple_core.workflows.executors` (v01)
- `LocalSensitivity` C — `pinneapple_analysis.inverse_problems.sensitivity` (v05)
- `LocalSensitivityResult` C — `pinneapple_analysis.inverse_problems.sensitivity` (v05)
- `locate` f — `pinneapple_tools.rea_bridge.evidence` (v07)
- `Location` C — `pinneapple_check.result` (v05)
- `locations` f — `pinneapple_data.cae.compare` (v04)
- `LockedDropout` C — `pinneapple_neural.architectures.continuous.bayesian_rnn` (v04)
- `lodo` f — `pinneapple_simulation.electromagnetics.antenna.surrogates` (v03)
- `lodo_rmse_bars` f — `pinneapple_simulation.electromagnetics.antenna.plots` (v03)
- `loft` f — `pinneapple_design.geometry.vehicle_cad` (v06)
- `loft` f — `pinneapple_design.geometry_library.kit` (v06)
- `loft` f — `pinneapple_twin3d.primitives` (v06)
- `log_evidence` f — `pinneapple_simulation.numerical_solvers.gated_xtfc` (v03)
- `log_loss` f — `pinneapple_systems.sports.football.metrics` (v06)
- `log_result` f — `pinneapple_physics.tracking` (v02)
- `log_returns` f — `pinneapple_systems.time_series.datasets.public` (v06)
- `log_softmax` f — `pinneapple_decision.debias` (v05)
- `log_subdomain_boundaries` f — `pinneapple_simulation.numerical_solvers.xtfc_subdomain` (v03)
- `log_ticks_ok` f — `pinneapple_science.plots` (v02)
- `LogCollector` C — `pinneapple_veriphysics.execution_log` (v05)
- `LogEntry` C — `pinneapple_veriphysics.execution_log` (v05)
- `LoginThrottle` C — `pinneapple_security.web` (v05)
- `LogProbLLMBackend` C — `pinneapple_decision.backends` (v05)
- `LogStep` C — `pinneapple_neural.trainer.preprocess` (v04)
- `lomb_scargle_periodogram` f — `pinneapple_physics.pde_environment.presets.astroml_bridge` (v02)

- `long_run_statistics` f — `pinneapple_analysis.validation.rollout` (v05)
- `look` f — `pinneapple_physics.astro.satellites` (v02)
- `lookup_phenomenon` f — `pinneapple_problemdesign.knowledge.physics_knowledge` (v02)
- `Loop` C — `pinneapple_design.aero.geometry` (v06)
- `Loop` C — `pinneapple_systems.digital_twin.pipe_loop.plant` (v06)
- `loop_field` f — `pinneapple_simulation.electromagnetics.statics` (v03)
- `loop_figures` f — `pinneapple_design.intent.figures` (v06)
- `loop_flux` f — `pinneapple_science.em.induction` (v02)
- `loop_gif` f — `pinneapple_design.intent.figures` (v06)
- `loop_on_axis_B` f — `pinneapple_simulation.electromagnetics.statics` (v03)
- `loop_scene` f — `pinneapple_systems.digital_twin.pipe_loop.scene` (v06)
- `loop_self_inductance` f — `pinneapple_simulation.electromagnetics.statics` (v03)
- `LoopCarTemplate` C — `pinneapple_design.intent.templates.loop_car` (v06)
- `LoopDesign` C — `pinneapple_simulation.flight.control` (v03)
- `LoopTrack` C — `pinneapple_design.intent.physics.loop_track` (v06)
- `LoRALinear` C — `pinneapple_worldmodel.mega_model` (v04)
- `Lorentz` C — `pinneapple_simulation.electromagnetics.media` (v03)
- `lorenz63` f — `pinneapple_systems.time_series.synthetic` (v06)
- `lorenz63_missing_xz_rhs` f — `pinneapple_systems.time_series.forecast_lab.presets` (v06)
- `lorenz63_rhs` f — `pinneapple_systems.time_series.synthetic` (v06)
- `lorenz63_step` f — `pinneapple_analysis.data_assimilation.models` (v05)
- `lorenz96` f — `pinneapple_systems.time_series.synthetic` (v06)
- `lorenz96_rhs` f — `pinneapple_systems.time_series.synthetic` (v06)
- `lorenz96_step` f — `pinneapple_analysis.data_assimilation.models` (v05)
- `lorenz96_tendency` f — `pinneapple_analysis.data_assimilation.models` (v05)
- `LorenzDiscovery` C — `pinneapple_lab.experiments.discovery` (v07)
- `LossFit` C — `pinneapple_simulation.electromagnetics.machines.steel` (v03)
- `LossRatioBalancer` C — `pinneapple_neural.trainer.weight_scheduler` (v04)
- `LossTracker` C — `pinneapple_systems.digital_twin.pipe_loop.diagnose` (v06)
- `LossWeights` C — `pinneapple_physics.pinn_solver.compiler.loss` (v02)
- `LossyPlaneWave` C — `pinneapple_science.em.waves` (v02)
- `low_modes` f — `pinneapple_physics.multiscale.filters` (v02)
- `LRAnnealing` C — `pinneapple_neural.trainer.loss_balancer` (v04)
- `LRUCache` C — `pinneapple_data.cache` (v04)
- `lshape` f — `pinneapple_design.geometry.csg` (v06)
- `LShapeDomain2D` C — `pinneapple_design.geometry.gen.domains` (v06)
- `LSTMForecaster` C — `pinneapple_systems.time_series.models.recurrent` (v06)
- `LSTMModel` C — `pinneapple_neural.architectures.recurrent.lstm` (v04)
- `lube_2005` f — `pinneapple_flow.nodes_ext.geomechanics` (v07)
- `luhn_ok` f — `pinneapple_security.privacy` (v05)
- `luminosity_group` f — `pinneapple_physics.astro.stars_ml` (v02)
- `lumped_capacitance` f — `pinneapple_physics.physics_audit.assumptions` (v02)
- `lumped_link` f — `pinneapple_simulation.multibody.arm` (v03)
- `LumpedCoolingModel` C — `pinneapple_analysis.inverse_problems.models` (v05)
- `LumpedTankModel` C — `pinneapple_systems.operations.twin` (v06)
- `lunar_eclipses` f — `pinneapple_physics.astro.phenomena` (v02)
- `lunar_lambert` f — `pinneapple_tools.visualization.studio.shadows` (v07)
- `lunar_phase_events` f — `pinneapple_physics.astro.phenomena` (v02)
- `lunar_rover` f — `pinneapple_twin3d.gallery` (v06)
- `LunarTerrain` C — `pinneapple_simulation.rover.terrain` (v03)
- `lvl` f — `pinneapple_analysis.benchmark.runner` (v05)
- `lyapunov` f — `pinneapple_flow.nodes_ext.forecasting` (v07)
- `lyapunov_study` f — `pinneapple_systems.time_series.forecast_lab.pipeline` (v06)

M

- `m1_advection` f — `pinneapple_condense.falsify` (v02)

- m2_stokes f — pinneapple_condense.falsify (v02)
- m3_hybrid f — pinneapple_condense.falsify (v02)
- m4_meshless_condensed f — pinneapple_condense.falsify (v02)
- m_cr f — pinneapple_reason.modes.structural (v05)
- mach f — pinneapple_physics.physics_audit.regime (v02)
- mach_from_area_ratio f — pinneapple_physics.closed_form.rocket_nozzle (v02)
- Machine C — pinneapple_llm.workbench.models (v07)
- machine f — pinneapple_presets.cases.ipm_traction (v02)
- machine_network f — pinneapple_simulation.electromagnetics.machines.thermal (v03)
- machine_reynolds_number f — pinneapple_systems.process_components.similarity_map (v06)
- MachineGeometry C — pinneapple_simulation.electromagnetics.machines.geometry (v03)
- MachineMesh C — pinneapple_simulation.electromagnetics.machines.mesher (v03)
- MachineSolution C — pinneapple_simulation.electromagnetics.machine (v03)
- MachineThermalData C — pinneapple_simulation.electromagnetics.machines.thermal (v03)
- mackey_glass f — pinneapple_systems.time_series.synthetic (v06)
- mackey_glass_lyapunov f — pinneapple_systems.time_series.synthetic (v06)
- mackey_glass_rhs f — pinneapple_systems.time_series.synthetic (v06)
- mackey_glass_step f — pinneapple_systems.time_series.synthetic (v06)
- mackey_glass_stepper f — pinneapple_systems.time_series.synthetic (v06)
- MAE C — pinneapple_neural.trainer.metrics (v04)
- mae f — pinneapple_physics.metrics (v02)
- mae f — pinneapple_systems.time_series.metrics (v06)
- mae f — pinneapple_systems.time_series.metrics_ext.point (v06)
- mae_fn f — pinneapple_systems.time_series.metrics (v06)
- mag_to_nl f — pinneapple_physics.astro.observing (v02)
- Magnet C — pinneapple_simulation.electromagnetics.materials (v03)
- magnet_eddy_loss_resistance_limited f — pinneapple_simulation.electromagnetics.postprocess (v03)
- magnet_rings f — pinneapple_science.em.induction (v02)
- magnet_temperature_sweep f — pinneapple_simulation.electromagnetics.machines.thermal (v03)
- MagnetGrade C — pinneapple_simulation.electromagnetics.machines.steel (v03)
- magnetized_cylinder f — pinneapple_simulation.electromagnetics.analytic (v03)
- magnetized_ring f — pinneapple_simulation.electromagnetics.analytic (v03)
- MagnetostaticSolution C — pinneapple_simulation.electromagnetics.fem (v03)
- magnitude f — pinneapple_physics.astro.phenomena (v02)
- magnitude f — pinneapple_physics.astro.satellites (v02)
- MahalanobisDetector C — pinneapple_systems.digital_twin.monitoring.anomaly (v06)
- main f — pinneapple_analysis.benchmark.cli (v05)
- main f — pinneapple_analysis.validation.arena.__main__ (v05)
- main f — pinneapple_analysis.validation.fault_injection.__main__ (v05)
- main f — pinneapple_arena.external.__main__ (v04)
- main f — pinneapple_blender.blender_addon.import_pinneapple_sequence (v07)
- main f — pinneapple_check.cli (v05)
- main f — pinneapple_core.onboarding.cli (v01)
- main f — pinneapple_design.aero.render_blender (v06)
- main f — pinneapple_design.intent.cli (v06)
- main f — pinneapple_design.thermal.heatsink.surrogate (v06)
- main f — pinneapple_flow.__main__ (v07)
- main f — pinneapple_flow.examples (v07)
- main f — pinneapple_flow.nodes_ext.render_worker (v07)
- main f — pinneapple_hub.library.__main__ (v07)
- main f — pinneapple_hub.zoo.web.__main__ (v07)
- main f — pinneapple_lab.__main__ (v07)
- main f — pinneapple_llm.qa.examples.pipe_wear_twin_main (v07)
- main f — pinneapple_llm.workbench.__main__ (v07)
- main f — pinneapple_mcp (v07)
- main f — pinneapple_mcp.server (v07)

- main f — pinneapple_presets.__main__ (v02)
- main f — pinneapple_problemdesign.cli (v02)
- main f — pinneapple_reason.__main__ (v05)
- main f — pinneapple_science.__main__ (v02)
- main f — pinneapple_security.__main__ (v05)
- main f — pinneapple_simulation.electromagnetics.antenna.ava (v03)
- main f — pinneapple_simulation.electromagnetics.periodic.tcda (v03)
- main f — pinneapple_simulation.external_solvers.calculix.virtual_ccx (v03)
- main f — pinneapple_simulation.external_solvers.cfd_formats._abaqus_odb_export_script (v03)
- main f — pinneapple_simulation.external_solvers.openfoam.campaign (v03)
- main f — pinneapple_simulation.pipelines.freecad_openfoam.__main__ (v03)
- main f — pinneapple_simulation.pipelines.freecad_openfoam.freecad_scripts.fc_job (v03)
- main f — pinneapple_tools.benchmark_suite.runner.run_arena_yaml (v07)
- main f — pinneapple_tools.benchmark_suite.runner.run_models_from_cfg (v07)
- main f — pinneapple_tools.benchmark_suite.runner.run_pipeline (v07)
- main f — pinneapple_tools.model_services.__main__ (v07)
- main f — pinneapple_tools.nvidia._brep_worker (v07)
- main f — pinneapple_worldmodel.generate_datasets (v04)
- main f — pinneapple_worldmodel.train (v04)
- main_roll f — pinneapple_simulation.electromagnetics.reflector.study_report (v03)
- main_wave_speed f — pinneapple_reason.modes.fluids (v05)
- make_acoustic_physics_loss f — pinneapple_analysis.inspection.acoustic (v05)
- make_action f — pinneapple_lab.next_steps (v07)
- make_actuation f — pinneapple_simulation.numerical_solvers.thermal_channel_datasets (v03)
- make_annulus f — pinneapple_worldmodel.geometry (v04)
- make_backend f — pinneapple_llm.workbench.backend (v07)
- make_blocks f — pinneapple_physics.blackhole.forecast (v02)
- make_car f — pinneapple_autolab.domains.racecar (v07)
- make_cases f — pinneapple_systems.digital_twin.pipe_loop.benchmark (v06)
- make_cavity f — pinneapple_worldmodel.geometry (v04)
- make_channel f — pinneapple_worldmodel.geometry (v04)
- make_channel_with_cylinder f — pinneapple_worldmodel.geometry (v04)
- make_data f — pinneapple_simulation.external_solvers.mujoco.loader (v03)
- make_dataset f — pinneapple_physics.astro.neural_propagator (v02)
- make_eddy_current_physics_loss f — pinneapple_analysis.inspection.eddy_current (v05)
- make_emat_physics_loss f — pinneapple_analysis.inspection.emat (v05)
- make_estimator f — pinneapple_systems.operations.control (v06)
- make_fd_residual_fn f — pinneapple_data.synth.pde_symbolic (v04)
- make_figure f — pinneapple_tools.visualization.style (v07)
- make_figures f — pinneapple_veriphysics.evidence_package (v05)
- make_gap f — pinneapple_lab.next_steps (v07)
- make_geometry f — pinneapple_design.intent.package (v06)
- make_gif f — pinneapple_tools.visualization.animation (v07)
- make_goal f — pinneapple_lab.experiments.bricks (v07)
- make_guided_wave_physics_loss f — pinneapple_analysis.inspection.guided_wave_ultrasonic (v05)
- make_l_shaped f — pinneapple_worldmodel.geometry (v04)
- make_lags f — pinneapple_systems.time_series.features.engineering (v06)
- make_linear_system f — pinneapple_neural.architectures.rom.toy_systems (v04)
- make_map f — pinneapple_systems.process_components.similarity_map (v06)
- make_mfl_physics_loss f — pinneapple_analysis.inspection.magnetic_flux_leakage (v05)
- make_model f — pinneapple_autolab.surrogates (v07)
- make_phased_array_physics_loss f — pinneapple_analysis.inspection.phased_array_ultrasonic (v05)
- make_point_set f — pinneapple_tools.benchmark_suite.crossframework.problems (v07)
- make_recipe f — pinneapple_data.unified (v04)
- make_server f — pinneapple_tools.model_services.server (v07)
- make_splits f — pinneapple_analysis.surrogate.splits (v05)

- make_twin f — pinneapple_systems.operations.live (v06)
- make_union_circles_sdf f — pinneapple_design.geometry.gen.genetic_sdf (v06)
- make_unit_square f — pinneapple_worldmodel.geometry (v04)
- MAMLConfig C — pinneapple_adaptation.meta_learning.config (v04)
- MAMLTrainer C — pinneapple_adaptation.meta_learning.maml (v04)
- Manager C — pinneapple_llm.workbench.server (v07)
- Manifest C — pinneapple_lab.manifest (v07)
- Manifest C — pinneapple_security.integrity (v05)
- Manifest C — pinneapple_systems.time_series.enso.data (v06)
- manifest f — pinneapple_hub.zoo._artifacts (v07)
- manifest f — pinneapple_systems.sports.football.data (v06)
- manifest_from_dir f — pinneapple_lab.manifest (v07)
- ManifestImport C — pinneapple_lab.experiments.manifest_import (v07)
- manifests f — pinneapple_systems.time_series.enso.data (v06)
- manual_install f — pinneapple_arena.external.result (v04)
- ManufacturabilityConstraint C — pinneapple_design.design_optimizer.constraints (v06)
- ManufacturedSolutionCheck C — pinneapple_problemdesign.preset_authoring (v02)
- map_axes f — pinneapple_design.thermal.board_cooling.simulate (v06)
- MapCNNForecaster C — pinneapple_systems.time_series.enso.forecasters (v06)
- MapCoefficients C — pinneapple_systems.process_components.similarity_map (v06)
- mape f — pinneapple_systems.time_series.metrics (v06)
- mape f — pinneapple_systems.time_series.metrics_ext.point (v06)
- mape f — pinneapple_tools.model_evaluation.metrics (v07)
- mape_fn f — pinneapple_systems.time_series.metrics (v06)
- MapEvaluation C — pinneapple_systems.process_components.similarity_map (v06)
- marching_squares_boundary f — pinneapple_design.geometry.sample.marching_squares (v06)
- marching_squares_contours f — pinneapple_design.geometry.io.sdf_meshing (v06)
- margin_study f — pinneapple_analysis.uncertainty.margin (v05)
- margins_figure f — pinneapple_reason.report (v05)
- markdown f — pinneapple_analysis.benchmark.report (v05)
- markdown f — pinneapple_flow.coverage (v07)
- markdown f — pinneapple_flow.examples (v07)
- markdown f — pinneapple_hub.zoo.catalogue (v07)
- markdown_table f — pinneapple_simulation.accelerated_foam.benchmark (v03)
- markdown_table f — pinneapple_simulation.external_solvers.modelica.fmi_compat (v03)
- markdown_tables f — pinneapple_physics.multiscale.comparison (v02)
- markdown_tables f — pinneapple_systems.time_series.enso.pipeline (v06)
- markdown_tables f — pinneapple_systems.time_series.forecast_lab.pipeline (v06)
- Marker C — pinneapple_design.bricks.visuals (v06)
- Marker C — pinneapple_tools.visualization.dashboard.spec (v07)
- mase f — pinneapple_systems.time_series.metrics_ext.point (v06)
- mass f — pinneapple_core.fem (v01)
- mass_action_rate f — pinneapple_systems.process_components.reaction_kinetics (v06)
- mass_balance f — pinneapple_tools.automation.verify (v07)
- mass_balance_residual f — pinneapple_systems.operations.plant (v06)
- mass_flow_to_standard_volumetric_flow f — pinneapple_systems.process_components.real_gas_eos (v06)
- mass_model_residuals f — pinneapple_design.bricks.parts (v06)
- mass_properties f — pinneapple_simulation.flight.mass (v03)
- MassBalance C — pinneapple_security.cps (v05)
- MassConservationConstraint C — pinneapple_design.design_optimizer.constraints (v06)
- MassEnvelope C — pinneapple_physics.blackhole.forecast (v02)
- MassProjection C — pinneapple_physics.blackhole.forecast (v02)
- mat_to_upd f — pinneapple_simulation.external_solvers.matlab.field_reader (v03)
- match f — pinneapple_flow.planner.extensions (v07)
- match_fields f — pinneapple_data.cae.compare (v04)
- match_load f — pinneapple_systems.sports.football.kin_study (v06)

- Material C — pinneapple_design.intent.materials (v06)
- Material C — pinneapple_simulation.external_solvers.calculix.study (v03)
- material_fracture_2d f — pinneapple_physics.pde_environment.presets.multidisciplinary (v02)
- MaterialsProjectClient C — pinneapple_data.datasets.external_sources (v04)
- matern_kernel f — pinneapple_analysis.data_assimilation.sensor_placement (v05)
- MATLABEngine C — pinneapple_simulation.external_solvers.matlab.engine (v03)
- matrix_from_quat f — pinneapple_tools.visualization.studio.animation (v07)
- max_abs f — pinneapple_analysis.reproduction (v05)
- max_abs f — pinneapple_physics.metrics (v02)
- max_climbable_grade f — pinneapple_simulation.rover.autolab.wheel_models (v03)
- MaxError C — pinneapple_neural.trainer.metrics (v04)
- maxwell_checks f — pinneapple_check._reuse (v05)
- maxwell_checks f — pinneapple_physics.maxwell_checks (v02)
- maxwell_trust_report f — pinneapple_physics.maxwell_checks (v02)
- MaxwellCheck C — pinneapple_physics.maxwell_checks (v02)
- maybe_compile f — pinneapple_neural.trainer.parallel (v04)
- mc_dropout_uncertainty f — pinneapple_systems.component_modeling.mc_dropout (v06)
- MCDropout C — pinneapple_analysis.uncertainty.mc_dropout (v05)
- MCDropoutConfig C — pinneapple_analysis.uncertainty.mc_dropout (v05)
- MCDropoutWrapper C — pinneapple_analysis.uncertainty.mc_dropout (v05)
- mcp f — pinneapple_hub.library.snippets (v07)
- md_dashboard f — pinneapple_tools.visualization.molecular (v07)
- MDState C — pinneapple_simulation.particle_dynamics.molecular_dynamics (v03)
- mean_abs f — pinneapple_analysis.reproduction (v05)
- mean_covariance_loss f — pinneapple_physics.pinn_solver.stochastic (v02)
- mean_curvature f — pinneapple_design.geometry.ops.curvature (v06)
- mean_interval_width f — pinneapple_systems.time_series.metrics_ext.probabilistic (v06)
- mean_nusselt_from_temperatures f — pinneapple_physics.closed_form.heat_exchanger (v02)
- mean_obliquity f — pinneapple_physics.astro.frames (v02)
- measure f — pinneapple_analysis.cost.profiler (v05)
- measure f — pinneapple_simulation.rover.autolab.estimation (v03)
- measure f — pinneapple_systems.operations.plant (v06)
- measure_design f — pinneapple_design.thermal.fin_array_speed (v06)
- measure_part f — pinneapple_llm.cad_draft (v07)
- measured f — pinneapple_flow.examples (v07)
- measured f — pinneapple_lab.next_steps (v07)
- measured_text f — pinneapple_lab.next_steps (v07)
- media_path f — pinneapple_lab.explorer (v07)
- media_urls f — pinneapple_lab.explorer (v07)
- Medium C — pinneapple_simulation.electromagnetics.media (v03)
- members f — pinneapple_physics.astro.conditions (v02)
- memorise_vs_learn f — pinneapple_systems.time_series.forecast_lab.pipeline (v06)
- Mention C — pinneapple_design.intent.parse (v06)
- merge f — pinneapple_design.bricks.mesh (v06)
- merge f — pinneapple_design.geometry.vehicle_cad (v06)
- merge f — pinneapple_design.geometry_library.kit (v06)
- merge f — pinneapple_design.surrogate_inverse.meshes (v06)
- merge f — pinneapple_presets.viz3d (v02)
- merge f — pinneapple_twin3d.primitives (v06)
- merge_into_spec f — pinneapple_problemdesign.merge (v02)
- merge_zoo f — pinneapple_systems.time_series.forecast_lab.study (v06)
- merkle_root f — pinneapple_security.integrity (v05)
- Mesh C — pinneapple_core.mesh (v01)
- Mesh C — pinneapple_data.cae.model (v04)
- mesh f — pinneapple_design.surrogate_inverse.brake_rotor (v06)
- mesh f — pinneapple_design.surrogate_inverse.heatsink_fins (v06)

- mesh f — pinneapple_design.surrogate_inverse.reflex_port (v06)
- mesh f — pinneapple_design.surrogate_inverse.suspension_arm (v06)
- mesh f — pinneapple_design.surrogate_inverse.transdermal_patch (v06)
- mesh f — pinneapple_flow.nodes.problem (v07)
- Mesh2D C — pinneapple_design.geometry.mesh.mesher_2d (v06)
- Mesh2D C — pinneapple_simulation.electromagnetics.mesh (v03)
- Mesh3D C — pinneapple_design.geometry.mesh.mesher_3d (v06)
- mesh_convergence f — pinneapple_condense.benchmark (v02)
- mesh_courant_number f — pinneapple_data.cae.metrics (v04)
- mesh_figure f — pinneapple_simulation.pipelines.freecad_openfoam.post (v03)
- mesh_from_gmsh f — pinneapple_simulation.electromagnetics.mesh (v03)
- mesh_from_sdf_3d f — pinneapple_design.geometry.mesh.mesher_3d (v06)
- mesh_from_surface f — pinneapple_design.geometry.mesh.mesher_3d (v06)
- mesh_independence_study f — pinneapple_analysis.verification.convergence (v05)
- mesh_kind f — pinneapple_design.fitmesh.mesh_plan (v06)
- mesh_mass_properties f — pinneapple_design.geometry.vehicle_cad (v06)
- mesh_polygon_2d f — pinneapple_design.geometry.mesh.mesher_2d (v06)
- mesh_quality f — pinneapple_twin3d.primitives (v06)
- mesh_quality_check f — pinneapple_analysis.benchmark.validation (v05)
- mesh_quality_report f — pinneapple_design.geometry.mesh.mesher_2d (v06)
- mesh_quality_report_3d f — pinneapple_design.geometry.ops.clean (v06)
- mesh_rectangle_structured f — pinneapple_design.geometry.mesh.mesher_2d (v06)
- mesh_reflector f — pinneapple_simulation.electromagnetics.reflector.geometry (v03)
- mesh_sdf_2d f — pinneapple_design.geometry.mesh.mesher_2d (v06)
- mesh_sector f — pinneapple_simulation.electromagnetics.machines.mesher (v03)
- mesh_sizes f — pinneapple_design.intent.physics.bracket (v06)
- mesh_study f — pinneapple_simulation.pipelines.freecad_openfoam.report (v03)
- mesh_study_figure f — pinneapple_simulation.pipelines.freecad_openfoam.report (v03)
- mesh_thickness f — pinneapple_tools.nvidia.brep (v07)
- mesh_to_fem_assembly_inputs f — pinneapple_simulation.numerical_solvers.adapters.upd_mesh (v03)
- mesh_to_fvm_topology f — pinneapple_simulation.numerical_solvers.adapters.upd_mesh (v03)
- mesh_to_obstacle_mask f — pinneapple_design.geometry.ops.lbm_bridge (v06)
- mesh_to_pointcloud f — pinneapple_design.geometry.ops.pointcloud (v06)
- mesh_views f — pinneapple_veriphysics.mesh_views (v05)
- mesh_wall_distance f — pinneapple_data.cae.metrics (v04)
- MeshBuilder C — pinneapple_design.geometry.gen.primitives (v06)
- MeshCollocator C — pinneapple_design.geometry.mesh_collocator (v06)
- MeshCollocatorConfig C — pinneapple_design.geometry.mesh_collocator (v06)
- MeshData C — pinneapple_design.geometry.core.mesh (v06)
- MeshDynamicsMGN C — pinneapple_neural.architectures.graphnn.mgn_dynamics (v04)
- MeshGeometry C — pinneapple_simulation.external_solvers.openfoam.mesh_reader (v03)
- MeshGraph C — pinneapple_neural.architectures.graphnn.mgn_dynamics (v04)
- MeshGraphNet C — pinneapple_neural.architectures.graphnn.mesh_graph_net (v04)
- meshgraphnet f — pinneapple_flow.nodes.methods (v07)
- MeshIndependenceStudyResult C — pinneapple_analysis.verification.convergence (v05)
- MeshInfo C — pinneapple_arena.pointwise.schema (v04)
- meshio_to_upd f — pinneapple_design.geometry.io.meshio_bridge (v06)
- meshless_checks f — pinneapple_simulation.meshless.contracts (v03)
- meshless_compare f — pinneapple_flow.wrappers (v07)
- meshless_convergence f — pinneapple_condense.figures (v02)
- meshless_fields f — pinneapple_condense.figures (v02)
- meshless_study f — pinneapple_condense.benchmark (v02)
- meshless_study f — pinneapple_simulation.meshless.study (v03)
- MeshlessSolution C — pinneapple_condense.meshless (v02)
- MeshlessStudy C — pinneapple_simulation.meshless.study (v03)
- MeshPlan C — pinneapple_design.fitmesh.mesh_plan (v06)

- MeshQualityReport C — pinneapple_analysis.verification.mesh_intelligence (v05)
- MeshResult C — pinneapple_design.fitmesh.mesh_gen (v06)
- MeshSampler C — pinneapple_core.data (v01)
- MeshSpec C — pinneapple_simulation.pipelines.freecad_openfoam.mesh (v03)
- meta f — pinneapple_flow.nodes_ext.benchmarks (v07)
- meta_adapt f — pinneapple_adaptation.meta_learning (v04)
- meta_learn f — pinneapple_flow.nodes.methods (v07)
- meta_train f — pinneapple_adaptation.meta_learning (v04)
- MetaBenchmarkConfig C — pinneapple_tools.benchmark_suite.meta_benchmark (v07)
- MetaBenchmarkFamily C — pinneapple_tools.benchmark_suite.meta_benchmark (v07)
- MetaBenchmarkPipeline C — pinneapple_tools.benchmark_suite.meta_benchmark (v07)
- MetaBenchmarkResult C — pinneapple_tools.benchmark_suite.meta_benchmark (v07)
- metabolic_power f — pinneapple_systems.sports.football.kinematics (v06)
- MetaConfig C — pinneapple_worldmodel.meta_learning (v04)
- MetaLearner C — pinneapple_worldmodel.meta_learning (v04)
- MetaModel C — pinneapple_adaptation.meta_learning.meta_model (v04)
- metamorphic f — pinneapple_check._reuse (v05)
- MetamorphicReport C — pinneapple_analysis.validation.metamorphic (v05)
- Method C — pinneapple_analysis.benchmark.registry (v05)
- Method C — pinneapple_catalog.methods (v01)
- Method C — pinneapple_query.core (v02)
- method_catalog f — pinneapple_check._reuse (v05)
- method_status f — pinneapple_catalog.methods (v01)
- MethodComparison C — pinneapple_design.design_optimizer.compare (v06)
- MethodNotAvailable C — pinneapple_physics.solving (v02)
- MethodRecommendation C — pinneapple_problemdesign.method_selection (v02)
- methods f — pinneapple_analysis.benchmark.registry (v05)
- methods_of f — pinneapple_lab.manifest (v07)
- Metric C — pinneapple_lab.results_rules (v07)
- Metric C — pinneapple_neural.trainer.metrics (v04)
- metric_value f — pinneapple_flow.nodes_ext.examples (v07)
- metrica_match f — pinneapple_systems.sports.football.data (v06)
- MetricaMatch C — pinneapple_systems.sports.football.data (v06)
- MetricBundle C — pinneapple_neural.trainer.metrics (v04)
- MetricItem C — pinneapple_app.backend.routers.models (v07)
- Metrics C — pinneapple_neural.trainer.metrics (v04)
- metrics f — pinneapple_app.backend.main (v07)
- metrics f — pinneapple_design.aero.train (v06)
- metropolis f — pinneapple_analysis.inverse_problems.estimation (v05)
- metzner_reed_friction_factor f — pinneapple_systems.process_components.non_newtonian_pipe_flow (v06)
- mgn_cylinder_flow f — pinneapple_flow.nodes_ext.learned_sim (v07)
- MGNMethod C — pinneapple_analysis.benchmark.builtin.airfoil (v05)
- michaud_head f — pinneapple_simulation.numerical_solvers.water_hammer_moc (v03)
- microstrip_width f — pinneapple_simulation.electromagnetics.antenna.ava (v03)
- mie f — pinneapple_simulation.electromagnetics.mie (v03)
- mie_coefficients f — pinneapple_simulation.electromagnetics.mie (v03)
- mie_pec f — pinneapple_simulation.electromagnetics.mie (v03)
- MieResult C — pinneapple_simulation.electromagnetics.mie (v03)
- MieScattering C — pinneapple_science.em.waves (v02)
- min_calibration_size f — pinneapple_analysis.uncertainty.conformal_study (v05)
- min_distance f — pinneapple_data.doe.designs (v04)
- min_length f — pinneapple_simulation.electromagnetics.antenna.bounds (v03)
- MinerDamageResult C — pinneapple_systems.process_components.fatigue_analysis (v06)
- miners_rule_damage f — pinneapple_systems.process_components.fatigue_analysis (v06)
- minetti_cost f — pinneapple_systems.sports.football.kinematics (v06)

- `minimize` f — `pinneapple_analysis.validation.arena.search` (v05)
- `minimum_slope` f — `pinneapple_reason.modes.water` (v05)
- `MinMaxScaler` C — `pinneapple_data.transforms` (v04)
- `mirror` f — `pinneapple_design.geometry_library.kit` (v06)
- `mirror_z` f — `pinneapple_twin3d.primitives` (v06)
- `MissingValueStep` C — `pinneapple_neural.trainer.preprocess` (v04)
- `Mission` C — `pinneapple_autolab.mission` (v07)
- `Mission` C — `pinneapple_design.aero.airliner3d` (v06)
- `mission_numbers` f — `pinneapple_simulation.rover.autolab.report` (v03)
- `MissionDomain` C — `pinneapple_autolab.agent` (v07)
- `MissionReading` C — `pinneapple_autolab.agent` (v07)
- `MissionRefused` C — `pinneapple_autolab.agent` (v07)
- `MissionResult` C — `pinneapple_simulation.rover.autolab.mission` (v03)
- `MissionSpec` C — `pinneapple_simulation.rover.autolab.mission` (v03)
- `mixed_areas` f — `pinneapple_design.geometry.ops.curvature` (v06)
- `mixing_tank` f — `pinneapple_twin3d.gallery` (v06)
- `MJXParallelRunner` C — `pinneapple_simulation.external_solvers.mujoco.mjx_backend` (v03)
- `mli_blanket` f — `pinneapple_design.geometry.vehicle_cad` (v06)
- `MLP` C — `pinneapple_tools.benchmark_suite.backends.native_pinn` (v07)
- `mlp` f — `pinneapple_analysis.validation.pinn` (v05)
- `mlp` f — `pinneapple_neural.architectures.neural_operators.deeponet` (v04)
- `MLPDenoiser` C — `pinneapple_neural.architectures.pinns.physics_diffusion` (v04)
- `MLPEnsemble` C — `pinneapple_autolab.surrogates` (v07)
- `MLPEnsemble` C — `pinneapple_design.aero.surrogate` (v06)
- `MLPFieldsMethod` C — `pinneapple_analysis.benchmark.builtin.airfoil` (v05)
- `MLPFNOSurrogate` C — `pinneapple_neural.architectures.neural_operators.fno` (v04)
- `MLPForecaster` C — `pinneapple_systems.time_series.models.classical` (v06)
- `MLPMethod` C — `pinneapple_analysis.benchmark.builtin.airfoil` (v05)
- `MLPredictor` C — `pinneapple_systems.sports.football.kinematics` (v06)
- `MLPSurrogate` C — `pinneapple_design.surrogate_inverse.core` (v06)
- `MLPSurrogate` C — `pinneapple_tools.nvidia.physicsnemo` (v07)
- `MLSApproximation` C — `pinneapple_simulation.numerical_solvers.meshfree` (v03)
- `mlx_train_command` f — `pinneapple_llm.workbench.finetune` (v07)
- `MMA` C — `pinneapple_design.topology.optimizers` (v06)
- `mobility_analysis` f — `pinneapple_simulation.terramechanics.mobility` (v03)
- `moc_valve_closure` f — `pinneapple_systems.process_components.water_hammer` (v06)
- `MockStream` C — `pinneapple_systems.digital_twin.io.stream` (v06)
- `ModbusStream` C — `pinneapple_systems.digital_twin.io.stream` (v06)
- `Model` C — `pinneapple_analysis.validation.physics_learning_cases` (v05)
- `Model` C — `pinneapple_simulation.pipelines.freecad_openfoam.models` (v03)
- `model` f — `pinneapple_hub.library.snippets` (v07)
- `model_architecture` f — `pinneapple_flow.wrappers` (v07)
- `model_card` f — `pinneapple_hub.zoo.hub_cards` (v07)
- `model_card_section` f — `pinneapple_analysis.validation.report_artifact` (v05)
- `model_example` f — `pinneapple_hub.zoo.hub_cards` (v07)
- `model_faults` f — `pinneapple_analysis.validation.fault_injection.faults` (v05)
- `model_forward` f — `pinneapple_arena.pointwise.training` (v04)
- `model_from_dict` f — `pinneapple_systems.operations.twin` (v06)
- `model_from_ref` f — `pinneapple_analysis.study_contracts` (v05)
- `model_from_spec` f — `pinneapple_analysis.inverse_problems.presets` (v05)
- `model_ref` f — `pinneapple_analysis.surrogate.contracts` (v05)
- `model_ref` f — `pinneapple_neural.architectures.rom.contracts` (v04)
- `model_ref_for_surrogate` f — `pinneapple_design.design_optimizer.report` (v06)
- `model_ref_target` f — `pinneapple_veriphysics.validation_lab` (v05)
- `model_scope` f — `pinneapple_design.thermal.heatsink.engine` (v06)
- `model_scope` f — `pinneapple_physics.pcb.engine` (v02)

- model_to_ref f — pinneapple_analysis.study_contracts (v05)
- ModelAdapter C — pinneapple_neural.architectures.adapters.base (v04)
- ModelAudit C — pinneapple_analysis.validation.arena.loop (v05)
- ModelBoard C — pinneapple_autolab.surrogates (v07)
- ModelCard C — pinneapple_hub.model_card (v07)
- ModelCatalog C — pinneapple_neural.architectures.catalog (v04)
- ModelCheckpoint C — pinneapple_neural.trainer.callbacks (v04)
- ModelConfig C — pinneapple_arena.config (v04)
- ModelD C — pinneapple_check.numerics (v05)
- ModelEntry C — pinneapple_app.backend.core.model_catalog (v07)
- ModelGateway C — pinneapple_registry.gateway (v01)
- modelica_result_to_upd f — pinneapple_simulation.external_solvers.modelica.result_reader (v03)
- ModelListItem C — pinneapple_app.backend.routers.models (v07)
- ModelLoadError C — pinneapple_check.models (v05)
- ModelOutput C — pinneapple_neural.architectures.base (v04)
- ModelProfile C — pinneapple_physics.physics_audit.assumptions (v02)
- ModelRef C — pinneapple_data.contracts.models (v04)
- ModelRegistry C — pinneapple_neural.architectures.registry (v04)
- ModelRequest C — pinneapple_app.backend.routers.experiments (v07)
- ModelResult C — pinneapple_app.backend.core.experiment (v07)
- ModelResult C — pinneapple_arena.external.result (v04)
- ModelRunConfig C — pinneapple_app.backend.core.experiment (v07)
- ModelRunResult C — pinneapple_tools.benchmark_suite.report (v07)
- models f — pinneapple_hub.library.sources (v07)
- models f — pinneapple_hub.zoo.demo (v07)
- ModelSelector C — pinneapple_decision.selectors (v05)
- ModelService C — pinneapple_tools.model_services.registry (v07)
- ModelServiceClient C — pinneapple_tools.model_services.client (v07)
- ModelSpec C — pinneapple_arena.external.schema (v04)
- ModelSpec C — pinneapple_neural.architectures.registry (v04)
- ModelSpec C — pinneapple_systems.digital_twin.product.spec (v06)
- ModelSpec C — pinneapple_systems.time_series.forecast_lab.study (v06)
- ModelSpec C — pinneapple_tools.benchmark_suite.benchmark (v07)
- ModelStore C — pinneapple_registry.model_store (v01)
- ModelSummaryItem C — pinneapple_app.backend.routers.admin (v07)
- ModelTarget C — pinneapple_check.config (v05)
- ModelZoo C — pinneapple_worldmodel.model_zoo (v04)
- modes f — pinneapple_simulation.flight.trim (v03)
- ModifiedMLP C — pinneapple_neural.architectures.modified_mlp (v04)
- modulated_gaussian f — pinneapple_simulation.electromagnetics.fdtd (v03)
- modules f — pinneapple_flow.coverage (v07)
- mohr_coulomb_traction_limit f — pinneapple_simulation.terramechanics.mobility (v03)
- MolecularDynamicsSimulator C — pinneapple_simulation.particle_dynamics.molecular_dynamics (v03)
- Molecule C — pinneapple_tools.visualization.molecular (v07)
- molecule_html f — pinneapple_tools.visualization.integrations.plotly_export (v07)
- monitor_slip f — pinneapple_simulation.rover.autolab.verify (v03)
- monitoring_url f — pinneapple_systems.digital_twin.io.usgs (v06)
- monomials f — pinneapple_simulation.meshless.stencils (v03)
- monotone_violation_share f — pinneapple_neural.trainer.monotonic (v04)
- Monotonicity C — pinneapple_analysis.validation.metamorphic (v05)
- monotonicity_penalty f — pinneapple_neural.trainer.monotonic (v04)
- monte_carlo_uncertainty f — pinneapple_design.surrogate_inverse.brake_rotor (v06)
- month_range f — pinneapple_systems.time_series.enso.data (v06)
- moon_brightness_nl f — pinneapple_physics.astro.observing (v02)
- moon_phase f — pinneapple_physics.astro.phenomena (v02)
- morris f — pinneapple_analysis.sensitivity.morris (v05)

- morris_design f — pinneapple_analysis.sensitivity.morris (v05)
- MorrisResult C — pinneapple_analysis.sensitivity.morris (v05)
- motor_glider f — pinneapple_simulation.flight.catalog (v03)
- Mound C — pinneapple_simulation.rover.autolab.site (v03)
- move_batch_to_device f — pinneapple_data.collate (v04)
- MovingAverage C — pinneapple_systems.time_series.adaptive (v06)
- MPMSimulator C — pinneapple_simulation.particle_dynamics.mpm (v03)
- MPMState C — pinneapple_simulation.particle_dynamics.mpm (v03)
- MQTTBroker C — pinneapple_systems.digital_twin.io.mqtt_broker (v06)
- MQTTStream C — pinneapple_systems.digital_twin.io.stream (v06)
- MSE C — pinneapple_neural.trainer.metrics (v04)
- mse f — pinneapple_analysis.reproduction (v05)
- mse f — pinneapple_physics.pinn_solver.compiler.autograd_ops (v02)
- mse f — pinneapple_systems.time_series.metrics (v06)
- mse_fn f — pinneapple_systems.time_series.metrics (v06)
- mse_forecast f — pinneapple_systems.time_series.losses (v06)
- msis_available f — pinneapple_physics.astro.decay (v02)
- Mu0Solenoid C — pinneapple_science.em.bench (v02)
- MuJoCoConfig C — pinneapple_simulation.external_solvers.mujoco.loader (v03)
- MuJoCoRunner C — pinneapple_simulation.external_solvers.mujoco.runner (v03)
- multi_fidelity f — pinneapple_simulation.electromagnetics.antenna.design (v03)
- multi_horizon_mae f — pinneapple_systems.time_series.metrics_ext.point (v06)
- multi_indices f — pinneapple_analysis.uncertainty.pce (v05)
- multi_round_lbfgs f — pinneapple_neural.trainer.lbfgs_finetune (v04)
- MultiAdam C — pinneapple_neural.trainer.multiadam (v04)
- MultibodyTree C — pinneapple_simulation.multibody.tree (v03)
- MultiCameraArray C — pinneapple_worldmodel.camera_system (v04)
- multilayer f — pinneapple_simulation.electromagnetics.waves (v03)
- MultilayerResult C — pinneapple_simulation.electromagnetics.waves (v03)
- MultiObstacleDomain2D C — pinneapple_design.geometry.gen.domains (v06)
- MultiPeriodicBC C — pinneapple_physics.symbolic_pde.bc (v02)
- MultiRestartConfig C — pinneapple_neural.trainer.multistart (v04)
- MultiRestartResult C — pinneapple_neural.trainer.multistart (v04)
- MultiRestartTrainer C — pinneapple_neural.trainer.multistart (v04)
- multiscale_neighbourhood f — pinneapple_neural.lpm (v04)
- multiscale_residuals f — pinneapple_physics.multiscale.residuals (v02)
- MultiScaleDeepONet C — pinneapple_neural.architectures.neural_operators.ms_deeponet (v04)
- MultiscaleResidual C — pinneapple_physics.multiscale.residuals (v02)
- multiterm_periodogram f — pinneapple_physics.pde_environment.presets.astroml_bridge (v02)
- mutate f — pinneapple_design.bricks.search (v06)
- mutation_escapes f — pinneapple_analysis.validation.arena.red (v05)
- mutation_score f — pinneapple_analysis.validation.arena.red (v05)
- mutual_inductance_coaxial_loops f — pinneapple_simulation.electromagnetics.statics (v03)
- MyHub C — pinneapple_hub.library.myhub (v07)
- MyHubError C — pinneapple_hub.library.myhub (v07)

N

- n_cells f — pinneapple_simulation.external_solvers.openfoam.campaign (v03)
- n_fins_for f — pinneapple_design.thermal.heatsink.surrogate (v06)
- n_terms f — pinneapple_analysis.uncertainty.pce (v05)
- naca4 f — pinneapple_design.aero.geometry (v06)
- naca4 f — pinneapple_design.geometry.vehicles3d (v06)
- naca4 f — pinneapple_design.geometry_library.kit (v06)
- naca4_camber_slope f — pinneapple_physics.closed_form.thin_airfoil (v02)
- naca4_mask f — pinneapple_simulation.numerical_solvers.lbm (v03)
- naca4_points f — pinneapple_design.fitmesh.catalog (v06)

- naca4_symmetric_polygon f — pinneapple_physics.pde_environment.presets._curve_geometry (v02)
- naca_parametric f — pinneapple_design.geometry.gen.airfoil (v06)
- naive_baselines f — pinneapple_flow.nodes_ext.core_ts (v07)
- NaiveForecaster C — pinneapple_systems.time_series.baselines.naive (v06)
- names f — pinneapple_design.geometry_library.core (v06)
- names f — pinneapple_physics.astro.facts (v02)
- names f — pinneapple_presets.registry (v02)
- names f — pinneapple_science.registry (v02)
- names f — pinneapple_simulation.flight.catalog (v03)
- names_in f — pinneapple_data.expressions (v04)
- names_in_tree f — pinneapple_data.expressions (v04)
- NASAClient C — pinneapple_data.datasets.external_sources (v04)
- NASAPowerClient C — pinneapple_data.datasets.external_sources (v04)
- NASResult C — pinneapple_arena.nas (v04)
- NativePINNBackend C — pinneapple_tools.benchmark_suite.backends.native_pinn (v07)
- natural_convection f — pinneapple_physics.closed_form.plate_fin_heatsink (v02)
- navier_stokes_2d_vorticity f — pinneapple_simulation.numerical_solvers.operator_benchmarks (v03)
- NavierStokes C — pinneapple_analysis.benchmark.builtin.standard (v05)
- NavierStokes3D C — pinneapple_simulation.numerical_solvers.fdm3d (v03)
- NavierStokesConfig3D C — pinneapple_simulation.numerical_solvers.fdm3d (v03)
- NavierStokesTGV2DTask C — pinneapple_tools.benchmark_suite.tasks.navier_stokes_tgv_2d (v07)
- NBeats C — pinneapple_systems.time_series.models.nbeats (v06)
- NBeatsConfig C — pinneapple_systems.time_series.models.nbeats (v06)
- NBodySolver C — pinneapple_simulation.numerical_solvers.nbody_rebound (v03)
- ncei_heat_content f — pinneapple_systems.time_series.enso.data (v06)
- ndvi_change f — pinneapple_perception.change_detection (v04)
- NdviChangeResult C — pinneapple_perception.change_detection (v04)
- near_duplicates f — pinneapple_data.audit (v04)
- nearfield_planes f — pinneapple_simulation.electromagnetics.antenna.ava (v03)
- NearToFarField C — pinneapple_simulation.electromagnetics.fdtd (v03)
- Need C — pinneapple_reason.library (v05)
- need_problem f — pinneapple_flow.nodes._common (v07)
- needs_geometry_tags f — pinneapple_core.onboarding.catalog (v01)
- needs_rehash f — pinneapple_security.passwords (v05)
- NeedsInput C — pinneapple_design.intent.api (v06)
- NeighborList C — pinneapple_simulation.numerical_solvers.neighbors (v03)
- Network C — pinneapple_simulation.numerical_solvers.pipe_network (v03)
- NetworkConfig C — pinneapple_arena.config (v04)
- neumann_boundary_time_series f — pinneapple_simulation.numerical_solvers.nonlinear_beam_fem (v03)
- NeumannBC f — pinneapple_physics.pde_environment.conditions (v02)
- NeumannBC C — pinneapple_physics.symbolic_pde.bc (v02)
- NeuralCDE C — pinneapple_neural.architectures.continuous.neural_cde (v04)
- NeuralForecaster C — pinneapple_systems.time_series.forecast_lab.learned (v06)
- NeuralGaussianProcess C — pinneapple_neural.architectures.continuous.neural_gp (v04)
- NeuralNetwork C — pinneapple_physics.pinn_solver.factory.pinn_factory (v02)
- NeuralODE C — pinneapple_neural.architectures.continuous.neural_ode (v04)
- NeuralOperatorBase C — pinneapple_neural.architectures.neural_operators.base (v04)
- NeuralOperatorCatalog C — pinneapple_neural.architectures.neural_operators.registry (v04)
- NeuralOperatorDataset C — pinneapple_worldmodel.training_hooks (v04)
- NeuralPropagator C — pinneapple_physics.astro.neural_propagator (v02)
- NeuralROM C — pinneapple_neural.architectures.rom.neural_rom (v04)
- NeuralSDE C — pinneapple_neural.architectures.continuous.neural_sde (v04)
- NeuralTermConfig C — pinneapple_analysis.inverse_problems.missing_term (v05)
- NeuralTermDiscovery C — pinneapple_analysis.inverse_problems.missing_term (v05)
- NeuralTuner C — pinneapple_systems.time_series.tuning.optuna_tuner (v06)
- new_id f — pinneapple_systems.operations.contracts (v06)

- new_key f — pinneapple_security.encryption (v05)
- new_main f — pinneapple_core.onboarding.cli (v01)
- new_project f — pinneapple_core.onboarding.newproject (v01)
- newton_available f — pinneapple_simulation.robot_sim.sim (v03)
- newton_check f — pinneapple_design.intent.physics.loop_track (v06)
- newton_scene f — pinneapple_design.intent.physics.loop_track (v06)
- newton_solve f — pinneapple_core.func (v01)
- newton_validate f — pinneapple_flow.wrappers (v07)
- NewtonLoopResult C — pinneapple_design.intent.physics.loop_track (v06)
- next_steps f — pinneapple_flow.lab_status (v07)
- next_steps f — pinneapple_lab.curation (v07)
- NextExperiment C — pinneapple_autolab.trust (v07)
- nfw_dark_matter_potential f — pinneapple_physics.pde_environment.presets.astrophysics (v02)
- nfw_potential_exact f — pinneapple_physics.pde_environment.presets.astrophysics (v02)
- nfw_source_fn f — pinneapple_physics.pde_environment.presets.astrophysics (v02)
- nino_indices f — pinneapple_systems.time_series.enso.data (v06)
- nl_to_mag f — pinneapple_physics.astro.observing (v02)
- NLS1D C — pinneapple_arena.problems (v04)
- NOAAClient C — pinneapple_data.datasets.external_sources (v04)
- nodal_boundary_values f — pinneapple_physics.darcy_checks (v02)
- nodal_darcy_residual f — pinneapple_physics.darcy_checks (v02)
- nodal_flux_balance f — pinneapple_physics.darcy_checks (v02)
- Node C — pinneapple_analysis.proof.graph (v05)
- Node C — pinneapple_flow.schema (v07)
- Node C — pinneapple_physics.physics_audit.knowledge (v02)
- node f — pinneapple_flow.registry (v07)
- node f — pinneapple_hub.library.snippets (v07)
- node_features f — pinneapple_design.aero.surrogate (v06)
- node_for_callable f — pinneapple_hub.library.sources (v07)
- node_states f — pinneapple_flow.runtime (v07)
- node_targets f — pinneapple_flow.coverage (v07)
- NodeContext C — pinneapple_flow.runtime (v07)
- nodes f — pinneapple_hub.library.sources (v07)
- nodes_for f — pinneapple_flow.lab_status (v07)
- NodeSet C — pinneapple_simulation.meshless.domains (v03)
- NodeSpec C — pinneapple_flow.registry (v07)
- NoetherABUPT C — pinneapple_neural.architectures.neural_operators.noether_bridge (v04)
- NoetherAeroABUPT C — pinneapple_neural.architectures.neural_operators.noether_bridge (v04)
- NoetherAeroTransformer C — pinneapple_neural.architectures.neural_operators.noether_bridge (v04)
- NoetherAeroTransolver C — pinneapple_neural.architectures.neural_operators.noether_bridge (v04)
- NoetherAeroUPT C — pinneapple_neural.architectures.neural_operators.noether_bridge (v04)
- NoetherDatasetBridge C — pinneapple_data.noether_dataset (v04)
- NoetherModelConfig C — pinneapple_neural.architectures.neural_operators.noether_bridge (v04)
- NoetherSurrogateTrainer C — pinneapple_neural.trainer.noether_trainer (v04)
- NoetherTrainConfig C — pinneapple_neural.trainer.noether_trainer (v04)
- NoetherTrainResult C — pinneapple_neural.trainer.noether_trainer (v04)
- NoetherTransformer C — pinneapple_neural.architectures.neural_operators.noether_bridge (v04)
- NoetherTransolver C — pinneapple_neural.architectures.neural_operators.noether_bridge (v04)
- NoetherUPT C — pinneapple_neural.architectures.neural_operators.noether_bridge (v04)
- NoFeasibleOptionError C — pinneapple_decision.constraints (v05)
- NoiseSchedulerBase C — pinneapple_neural.architectures.pinns.physics_diffusion (v04)
- NOMADClient C — pinneapple_data.datasets.external_sources (v04)
- nominal_image f — pinneapple_worldmodel.inspection_data (v04)
- NoModel C — pinneapple_physics.multiscale.closures (v02)
- Nondimensionalize C — pinneapple_core.transforms (v01)
- nonlinear_prediction_error f — pinneapple_systems.time_series.predictability (v06)

- NonlinearBeamFEMSolver C — pinneapple_simulation.numerical_solvers.nonlinear_beam_fem (v03)
- NonlinearBeamNewmarkFEM C — pinneapple_simulation.numerical_solvers.nonlinear_beam_fem (v03)
- NonlinearMPC C — pinneapple_systems.operations.control (v06)
- NonNegative C — pinneapple_analysis.surrogate.methods (v05)
- nonnegative_mask f — pinneapple_analysis.surrogate.study (v05)
- norm_dot_grad f — pinneapple_physics.pinn_solver.compiler.autograd_ops (v02)
- norm_factor f — pinneapple_simulation.accelerated_foam.ldu (v03)
- normal_shock f — pinneapple_physics.closed_form.compressible (v02)
- normalise_format f — pinneapple_design.geometry_library.exporters (v06)
- normalise_front f — pinneapple_design.design_optimizer.methods (v06)
- normalised_residual f — pinneapple_simulation.accelerated_foam.ldu (v03)
- normalization_loss f — pinneapple_quantum.loss.schrodinger (v04)
- normalize f — pinneapple_decision.debias (v05)
- normalize f — pinneapple_tools.hpo_experiments.ranking (v07)
- normalize_01 f — pinneapple_data.synth.operators.transforms (v04)
- normalize_kwargs f — pinneapple_neural.architectures.instantiate (v04)
- normalize_label f — pinneapple_tools.dataset_quality.vlm_curation (v07)
- normalize_weights f — pinneapple_analysis.uncertainty.posterior_metrics (v05)
- Normalizer C — pinneapple_data.transforms (v04)
- Normalizer C — pinneapple_neural.architectures.graphnn.mgn_dynamics (v04)
- NormalizeStep C — pinneapple_neural.trainer.preprocess (v04)
- not_applicable f — pinneapple_llm.qa.validators.base (v07)
- not_installed f — pinneapple_arena.external.result (v04)
- NotApplicableError C — pinneapple_tools.benchmark_suite.benchmark (v07)
- Note C — pinneapple_tools.reporting.datasheet (v07)
- NotInvertible C — pinneapple_core.transforms (v01)
- NotRun C — pinneapple_flow.runtime (v07)
- nozzle_performance f — pinneapple_physics.closed_form.rocket_nozzle (v02)
- npb_matrix f — pinneapple_physics.astro.frames (v02)
- npsh f — pinneapple_reason.modes.fluids (v05)
- npsh_available f — pinneapple_physics.closed_form.pump_npsh (v02)
- NRELClient C — pinneapple_data.datasets.external_sources (v04)
- ns2d_multiscale_residuals f — pinneapple_physics.multiscale.residuals (v02)
- ns_incompressible_2d_default f — pinneapple_physics.pde_environment.presets.cfd (v02)
- ns_incompressible_3d_default f — pinneapple_physics.pde_environment.presets.cfd (v02)
- ns_inputs f — pinneapple_simulation.numerical_solvers.operator_datasets (v03)
- NSFlowSolver C — pinneapple_simulation.numerical_solvers.cfd_pipeline (v03)
- nsga2 f — pinneapple_flow.nodes.methods (v07)
- NTKWeightBalancer C — pinneapple_neural.trainer.weight_scheduler (v04)
- ntu_from_effectiveness f — pinneapple_physics.closed_form.heat_exchanger (v02)
- num f — pinneapple_data.preflight.common (v04)
- NumbaStepper C — pinneapple_physics.blackhole.hydro_numba (v02)
- numbers f — pinneapple_physics.astro.facts (v02)
- numeric f — pinneapple_analysis.proof.graph (v05)
- numerical_dispersion f — pinneapple_simulation.electromagnetics.fdtd (v03)
- nutation f — pinneapple_physics.astro.frames (v02)
- nutation_matrix f — pinneapple_physics.astro.frames (v02)

O

- Objective C — pinneapple_design.design_optimizer.study (v06)
- Objective C — pinneapple_design.qualitative.preview (v06)
- ObjectiveBase C — pinneapple_design.design_optimizer.objective (v06)
- ObjectiveChoice C — pinneapple_design.intent.spec (v06)
- objectives_from f — pinneapple_design.design_optimizer.spec (v06)
- oblock_mesh_dict f — pinneapple_simulation.numerical_solvers.internal_flow (v03)
- observables f — pinneapple_condense.meshless (v02)

- Observation C — pinneapple_analysis.data_assimilation.var4d (v05)
- Observation C — pinneapple_llm.qa.spec (v07)
- Observation C — pinneapple_systems.digital_twin.state (v06)
- Observations C — pinneapple_analysis.inverse_problems.forward (v05)
- observations f — pinneapple_tools.benchmark_suite.crossframework.inverse (v07)
- observe f — pinneapple_flow.nodes.data (v07)
- observed_order f — pinneapple_science.core (v02)
- observed_orders f — pinneapple_simulation.electromagnetics.verification (v03)
- Observer C — pinneapple_physics.astro.astrometry (v02)
- observer_position f — pinneapple_physics.astro.journey (v02)
- ObsOperatorBase C — pinneapple_analysis.inverse_problems.obs_operator (v05)
- oc_update f — pinneapple_design.topology.optimizers (v06)
- occ_bottle f — pinneapple_design.geometry.gen.cadquery_advanced (v06)
- OceanGrid C — pinneapple_systems.time_series.enso.intermediate (v06)
- ocr_available f — pinneapple_data.formfill.ocr (v04)
- ocr_image f — pinneapple_data.formfill.ocr (v04)
- ocr_pdf_page f — pinneapple_data.formfill.ocr (v04)
- OcrPage C — pinneapple_data.formfill.ocr (v04)
- ode_stepper f — pinneapple_systems.time_series.synthetic (v06)
- ODEForecaster C — pinneapple_systems.time_series.forecast_lab.physical (v06)
- ODERNN C — pinneapple_neural.architectures.continuous.ode_rnn (v04)
- Offer C — pinneapple_query.core (v02)
- offline f — pinneapple_design.geometry_library.store (v06)
- offline_benchmark f — pinneapple_simulation.accelerated_foam.benchmark (v03)
- offline_report f — pinneapple_flow.nodes_ext.problemdesign (v07)
- offset_polygon f — pinneapple_design.geometry.vehicle_cad (v06)
- OffsetReflector C — pinneapple_simulation.electromagnetics.reflector.geometry (v03)
- oil_pipeline f — pinneapple_twin3d.gallery (v06)
- oisst_monthly f — pinneapple_systems.time_series.enso.data (v06)
- ok f — pinneapple_arena.external.result (v04)
- OllamaBackend C — pinneapple_llm.workbench.backend (v07)
- OllamaClient C — pinneapple_data.formfill.llm (v04)
- OllamaError C — pinneapple_data.formfill.llm (v04)
- om_to_upd f — pinneapple_simulation.external_solvers.modelica.om_runner (v03)
- omc_available f — pinneapple_simulation.external_solvers.modelica.omc (v03)
- OMSimConfig C — pinneapple_simulation.external_solvers.modelica.om_runner (v03)
- on_plane f — pinneapple_tools.automation.scripts.cfdof_setup (v07)
- on_run_finished f — pinneapple_flow.runtime (v07)
- one_at_a_time_sensitivity f — pinneapple_tools.model_evaluation.metrics (v07)
- one_two_exchanger_ode f — pinneapple_physics.closed_form.heat_exchanger (v02)
- onnx_parity f — pinneapple_hub.zoo (v07)
- onnx_parity f — pinneapple_hub.zoo.export (v07)
- ONSClient C — pinneapple_data.datasets.external_sources (v04)
- ood_verdict f — pinneapple_registry.gateway (v01)
- OODGate C — pinneapple_analysis.surrogate.limits (v05)
- OPCUAStream C — pinneapple_systems.digital_twin.io.stream (v06)
- OpcUaTwinServer C — pinneapple_systems.digital_twin.standards (v06)
- open_case f — pinneapple_data.adapters.splash_archive_adapter (v04)
- open_study f — pinneapple_simulation.electromagnetics.antenna.study (v03)
- OpenAIBackend C — pinneapple_llm.workbench.backend (v07)
- openfoam f — pinneapple_flow.nodes.solvers (v07)
- openfoam_available f — pinneapple_analysis.benchmark.builtin.airfoil (v05)
- openfoam_available f — pinneapple_flow.wrappers (v07)
- openfoam_available f — pinneapple_simulation.accelerated_foam.case (v03)
- openfoam_available f — pinneapple_simulation.numerical_solvers (v03)
- openfoam_available f — pinneapple_simulation.numerical_solvers.external_flow (v03)

- openfoam_case_to_upd f — pinneapple_simulation.external_solvers.openfoam.field_reader (v03)
- openfoam_comparison f — pinneapple_simulation.accelerated_foam.benchmark (v03)
- openfoam_to_dataset f — pinneapple_simulation.numerical_solvers.openfoam_bridge (v03)
- OpenFOAMAdapter C — pinneapple_core.workflows.adapters (v01)
- OpenFOAMAdapter C — pinneapple_tools.automation.adapters.openfoam (v07)
- OpenFOAMBridge C — pinneapple_simulation.numerical_solvers.openfoam_bridge (v03)
- OpenFOAMCaseTemplate C — pinneapple_simulation.external_solvers.openfoam.case_builder (v03)
- OpenFOAMClient C — pinneapple_data.datasets.external_sources (v04)
- OpenFOAMRunConfig C — pinneapple_simulation.external_solvers.openfoam.runner (v03)
- OpenMeteoClient C — pinneapple_data.datasets.external_sources (v04)
- OpenRadiossDockerConfig C — pinneapple_simulation.external_solvers.openradioss.runner (v03)
- operating_history f — pinneapple_systems.digital_twin.pipe_loop.plant (v06)
- operating_point f — pinneapple_systems.process_components.water_hammer (v06)
- operating_space f — pinneapple_systems.operations.series (v06)
- OperatingInput C — pinneapple_design.thermal.heatsink.engine (v06)
- OperatingPoint C — pinneapple_physics.closed_form.plate_fin_heatsink (v02)
- OperatingPoint C — pinneapple_systems.operations.exchanger (v06)
- OperatingPoint C — pinneapple_systems.process_components.water_hammer (v06)
- OperatorAdapter C — pinneapple_neural.architectures.adapters.operators (v04)
- OperatorBlockND C — pinneapple_neural.architectures.neural_operators.uno (v04)
- OperatorInference C — pinneapple_neural.architectures.rom.opinf (v04)
- OperatorOutput C — pinneapple_neural.architectures.neural_operators.base (v04)
- opinf_advection f — pinneapple_flow.nodes_ext.validation_gallery (v07)
- opinion_dynamics_2d f — pinneapple_physics.pde_environment.presets.multidisciplinary (v02)
- optics f — pinneapple_physics.astro.observing (v02)
- OptimisationResult C — pinneapple_design.intent.pipeline (v06)
- optimise f — pinneapple_design.intent.pipeline (v06)
- OptimizationOutcome C — pinneapple_simulation.electromagnetics.antenna.design (v03)
- OptimizationResult C — pinneapple_design.design_optimizer.methods (v06)
- OptimizationResult C — pinneapple_design.surrogate_inverse.brake_rotor (v06)
- OptimizationResult C — pinneapple_design.surrogate_inverse.heatsink_fins (v06)
- OptimizationResult C — pinneapple_design.surrogate_inverse.reflex_port (v06)
- OptimizationResult C — pinneapple_design.surrogate_inverse.suspension_arm (v06)
- OptimizationResult C — pinneapple_design.surrogate_inverse.transdermal_patch (v06)
- OptimizationResult C — pinneapple_twin3d.wear.optimize (v06)
- optimize f — pinneapple_design (v06)
- optimize f — pinneapple_design.thermal.board_cooling.optimize (v06)
- optimize f — pinneapple_design.topology.driver (v06)
- optimize f — pinneapple_simulation.electromagnetics.antenna.design (v03)
- optimize f — pinneapple_twin3d.wear.optimize (v06)
- optimize_design f — pinneapple_design.surrogate_inverse.brake_rotor (v06)
- optimize_design f — pinneapple_design.surrogate_inverse.heatsink_fins (v06)
- optimize_design f — pinneapple_design.surrogate_inverse.reflex_port (v06)
- optimize_design f — pinneapple_design.surrogate_inverse.suspension_arm (v06)
- optimize_design f — pinneapple_design.surrogate_inverse.transdermal_patch (v06)
- optimize_diameters f — pinneapple_simulation.numerical_solvers.pipe_network (v03)
- OptimizerConfig C — pinneapple_twin3d.wear.optimize (v06)
- OptimizeResult C — pinneapple_core.optim (v01)
- Option C — pinneapple_reason.chain (v05)
- option_line f — pinneapple_reason.narrative (v05)
- OptionInfo C — pinneapple_decision.schema (v05)
- OptState C — pinneapple_design.design_optimizer.optimizer (v06)
- orbit_mean_density f — pinneapple_physics.astro.decay (v02)
- orbit_polyline f — pinneapple_physics.astro.orbits (v02)
- OrchestratorResult C — pinneapple_worldmodel.orchestrator (v04)
- OrgAnswer C — pinneapple_llm.org_knowledge (v07)

- `orient_on_surface` f — `pinneapple_tools.visualization.molecular` (v07)
- `orient_outward` f — `pinneapple_design.geometry.vehicle_cad` (v06)
- `orient_outward` f — `pinneapple_design.geometry_library.meshops` (v06)
- `oriented_blocks` f — `pinneapple_data.cae.geometry` (v04)
- `ornstein_uhlenbeck` f — `pinneapple_systems.time_series.synthetic` (v06)
- `OrthogonalityResult` C — `pinneapple_analysis.verification.mesh_intelligence` (v05)
- `orthographic` f — `pinneapple_physics.weather.viz` (v02)
- `oscillation_stats` f — `pinneapple_systems.time_series.enso.oscillators` (v06)
- `Oscillator` C — `pinneapple_lab.experiments.ode` (v07)
- `oscillator_transition` f — `pinneapple_systems.time_series.synthetic` (v06)
- `OscillatorDiscovery` C — `pinneapple_lab.experiments.discovery` (v07)
- `Oscilloscope` C — `pinneapple_science.instruments` (v02)
- `out_of_bounds` f — `pinneapple_design.aero.geometry` (v06)
- `out_of_scope` f — `pinneapple_design.intent.parse` (v06)
- `Outcome` C — `pinneapple_check.result` (v05)
- `OutlierDetector` C — `pinneapple_systems.time_series.preparation.outliers` (v06)
- `outline` f — `pinneapple_design.aero.geometry` (v06)
- `OutOfEnvelope` C — `pinneapple_simulation.external_solvers.calculix.virtual_ccx` (v03)
- `OutOfEnvelopeError` C — `pinneapple_systems.process_components.real_gas_eos` (v06)
- `OutOfRangeWarning` C — `pinneapple_hub.zoo._models` (v07)
- `OutOfScope` C — `pinneapple_design.intent.parse` (v06)
- `Output` C — `pinneapple_presets.core` (v02)
- `output_filename` f — `pinneapple_data.formfill.pipeline` (v04)
- `output_names` f — `pinneapple_hub.zoo.export` (v07)
- `output_scale` f — `pinneapple_hub.zoo.export` (v07)
- `OutputConfig` C — `pinneapple_arena.config` (v04)
- `oval_boundary` f — `pinneapple_physics.astro.spaceweather` (v02)
- `overfitting` f — `pinneapple_flow.nodes_ext.forecasting` (v07)
- `overfitting_study` f — `pinneapple_systems.time_series.forecast_lab.study` (v06)
- `overfitting_variants` f — `pinneapple_systems.time_series.forecast_lab.presets` (v06)
- `overlaps` f — `pinneapple_physics.pcb.solver` (v02)

P

- `p6_rhs` f — `pinneapple_physics.paper_problems_xtfc` (v02)
- `p7_rhs` f — `pinneapple_physics.paper_problems_xtfc` (v02)
- `pack_splash_archive` f — `pinneapple_simulation.external_solvers.openfoam.splash_packer` (v03)
- `Package` C — `pinneapple_condense.package` (v02)
- `Package` C — `pinneapple_simulation.electromagnetics.antenna.package` (v03)
- `package_engine` f — `pinneapple_condense.query` (v02)
- `package_of` f — `pinneapple_flow.palette` (v07)
- `package_queries` f — `pinneapple_flow.nodes.query` (v07)
- `package_study` f — `pinneapple_simulation.electromagnetics.antenna.study` (v03)
- `package_thermal` f — `pinneapple_flow.nodes.condense` (v07)
- `package_versions` f — `pinneapple_tools.benchmark_suite.crossframework.records` (v07)
- `PackagedSample` C — `pinneapple_worldmodel.dataset_packager` (v04)
- `PackageModel` C — `pinneapple_condense.query` (v02)
- `PackagerConfig` C — `pinneapple_worldmodel.dataset_packager` (v04)
- `packages` f — `pinneapple_flow.coverage` (v07)
- `packed_bed` f — `pinneapple_simulation.numerical_solvers.particles` (v03)
- `pae` f — `pinneapple_security.signing` (v05)
- `PairAnalysis` C — `pinneapple_perception.change_detection` (v04)
- `paired_bootstrap` f — `pinneapple_systems.sports.football.metrics` (v06)
- `paired_ratio` f — `pinneapple_tools.benchmark_suite.crossframework.evaluate` (v07)
- `palette_entry` f — `pinneapple_flow.registry` (v07)
- `palmer_strip_capacitance` f — `pinneapple_science.em.statics` (v02)
- `pan_tilt_head` f — `pinneapple_design.geometry.vehicle_cad` (v06)

- `Panel` C — `pinneapple_tools.visualization.trust_panels` (v07)
- `panel` f — `pinneapple_lab.showcase` (v07)
- `panel_solve` f — `pinneapple_design.aero.panel` (v06)
- `PanelResult` C — `pinneapple_design.aero.panel` (v06)
- `Paper` C — `pinneapple_tools.reporting.paperkit` (v07)
- `para` f — `pinneapple_tools.reporting.paperkit` (v07)
- `paraboloid_dish` f — `pinneapple_design.geometry.vehicle_cad` (v06)
- `parallactic_angle` f — `pinneapple_physics.astro.observing` (v02)
- `parallel_plate_capacitance` f — `pinneapple_simulation.electromagnetics.statics` (v03)
- `parallel_plates_pressure_drop` f — `pinneapple_physics.closed_form.pipe_flow` (v02)
- `ParallelPlateCapacitor` C — `pinneapple_science.em.statics` (v02)
- `Param` C — `pinneapple_design.fitmesh.catalog` (v06)
- `Param` C — `pinneapple_design.geometry_library.core` (v06)
- `Param` C — `pinneapple_flow.registry` (v07)
- `Param` C — `pinneapple_presets.core` (v02)
- `Param` C — `pinneapple_tools.visualization.dashboard.spec` (v07)
- `param` f — `pinneapple_reason.library` (v05)
- `param_text` f — `pinneapple_reason.chain` (v05)
- `Parameter` C — `pinneapple_analysis.inverse_problems.forward` (v05)
- `Parameter` C — `pinneapple_data.contracts.models` (v04)
- `Parameter` C — `pinneapple_physics.physical_problem` (v02)
- `parameter_shift_gradient` f — `pinneapple_quantum.training.trainer` (v04)
- `parameter_shift_study` f — `pinneapple_veriphysics.robustness` (v05)
- `parameters_from_space` f — `pinneapple_analysis.inverse_problems.contracts` (v05)
- `ParameterSet` C — `pinneapple_analysis.inverse_problems.forward` (v05)
- `ParameterSpace` C — `pinneapple_data.contracts.models` (v04)
- `ParameterSpec` C — `pinneapple_systems.process_components.explicit_equation_system` (v06)
- `parametric` f — `pinneapple_flow.nodes.data` (v07)
- `parametric_pod` f — `pinneapple_flow.wrappers` (v07)
- `ParametricCadQuerySynthGenerator` C — `pinneapple_data.synth.geometry_cadquery` (v04)
- `ParametricCadSpec` C — `pinneapple_design.geometry.gen.cadquery_gen` (v06)
- `ParametricCase` C — `pinneapple_neural.workflows.parametric_dense_volume_operator` (v04)
- `ParametricDenseVolumeOperatorConfig` C — `pinneapple_neural.workflows.parametric_dense_volume_operator` (v04)
- `ParametricFamilyTransfer` C — `pinneapple_adaptation.transfer_learning.parametric` (v04)
- `ParametricPOD` C — `pinneapple_neural.architectures.rom.parametric` (v04)
- `ParamReport` C — `pinneapple_twin3d.wear.calibrate` (v06)
- `params_hash` f — `pinneapple_design.geometry_library.core` (v06)
- `ParamSpace` C — `pinneapple_design.design_optimizer.optimizer` (v06)
- `ParaViewAdapter` C — `pinneapple_tools.automation.adapters.paraview` (v07)
- `pareto` f — `pinneapple_condense.exceptions` (v02)
- `pareto` f — `pinneapple_condense.figures` (v02)
- `pareto_dominates` f — `pinneapple_design.design_optimizer.pareto` (v06)
- `pareto_figure` f — `pinneapple_design.intent.figures` (v06)
- `pareto_front` f — `pinneapple_autolab.report` (v07)
- `ParetoFront` C — `pinneapple_design.design_optimizer.pareto` (v06)
- `parity_error` f — `pinneapple_hub.zoo.export` (v07)
- `parity_inputs` f — `pinneapple_hub.zoo.export` (v07)
- `parity_plot` f — `pinneapple_simulation.electromagnetics.antenna.plots` (v03)
- `parquet_files` f — `pinneapple_hub.zoo.parquet` (v07)
- `parse` f — `pinneapple_data.simulation_metadata.foam_dict` (v04)
- `parse` f — `pinneapple_design.fitmesh.context` (v06)
- `parse_agent` f — `pinneapple_llm.workbench.agents` (v07)
- `parse_ccx_log` f — `pinneapple_data.simulation_metadata.calculix` (v04)
- `parse_curation_response` f — `pinneapple_tools.dataset_quality.vlm_curation` (v07)
- `parse_cvg` f — `pinneapple_data.simulation_metadata.calculix` (v04)

- parse_dat f — pinneapple_data.simulation_metadata.calculix (v04)
- parse_dat f — pinneapple_data.simulation_metadata.openfoam (v04)
- parse_duckduckgo f — pinneapple_llm.workbench.web (v07)
- parse_equation f — pinneapple_app.backend.core.physics_adapter (v07)
- parse_equation f — pinneapple_physics.symbolic_pde.text (v02)
- parse_expression f — pinneapple_physics.symbolic_pde.text (v02)
- parse_forces f — pinneapple_simulation.external_solvers.fluidx3d (v03)
- parse_front_matter f — pinneapple_hub.zoo.hub_cards (v07)
- parse_generation_response f — pinneapple_llm.qa.generation (v07)
- parse_inp f — pinneapple_data.simulation_metadata.calculix (v04)
- parse_iv f — pinneapple_systems.digital_twin.io.usgs (v06)
- parse_json_tool_call f — pinneapple_llm.workbench.backend (v07)
- parse_location f — pinneapple_app.backend.core.physics_adapter (v07)
- parse_log f — pinneapple_data.simulation_metadata.openfoam (v04)
- parse_log f — pinneapple_simulation.accelerated_foam.case (v03)
- parse_mission f — pinneapple_design.thermal.board_cooling.mission (v06)
- parse_mission f — pinneapple_simulation.rover.autolab.mission (v03)
- parse_numeric_table f — pinneapple_arena.pointwise.tables (v04)
- parse_objective f — pinneapple_design.qualitative.preview (v06)
- parse_parameters f — pinneapple_check.physics (v05)
- parse_pde f — pinneapple_analysis.surrogate.pde (v05)
- parse_quantity f — pinneapple_tools.automation.interpret (v07)
- parse_rate f — pinneapple_security.web (v05)
- parse_ref f — pinneapple_query.refs (v02)
- parse_residual_table f — pinneapple_data.simulation_metadata (v04)
- parse_silso_csv f — pinneapple_systems.time_series.solar_cycle (v06)
- parse_solver_log f — pinneapple_simulation.external_solvers.openfoam.campaign (v03)
- parse_sta f — pinneapple_data.simulation_metadata.calculix (v04)
- parse_system f — pinneapple_physics.symbolic_pde.text (v02)
- parse_tiers f — pinneapple_lab.seal (v07)
- parse_tle f — pinneapple_physics.astro.satellites (v02)
- parse_unit f — pinneapple_data.physical_units (v04)
- parse_unit f — pinneapple_llm.qa.units (v07)
- parse_value f — pinneapple_data.formfill.extract (v04)
- Part C — pinneapple_design.aero.airframe (v06)
- Part C — pinneapple_twin3d.scene (v06)
- part f — pinneapple_design.bricks.parts (v06)
- part_sensitivity f — pinneapple_design.qualitative.preview (v06)
- ParticleSuspension C — pinneapple_lab.experiments.particles (v07)
- ParticleSystem C — pinneapple_simulation.particle_dynamics.particles (v03)
- Partition C — pinneapple_condense.algebraic (v02)
- partition f — pinneapple_condense.algebraic (v02)
- PartRecipe C — pinneapple_tools.nvidia.brep (v07)
- PartSensitivity C — pinneapple_design.qualitative.preview (v06)
- PartsLibrary C — pinneapple_design.bricks.ldraw (v06)
- PartType C — pinneapple_design.bricks.parts (v06)
- passes_from_metrica f — pinneapple_systems.sports.football.pitch_control (v06)
- passivity f — pinneapple_physics.maxwell_checks (v02)
- PassSet C — pinneapple_systems.sports.football.pitch_control (v06)
- Patch C — pinneapple_simulation.rover.autolab.site (v03)
- patch_area f — pinneapple_tools.automation.verify (v07)
- patch_cfdof_allmesh f — pinneapple_tools.automation.adapters.openfoam (v07)
- patch_cfdof_inlet f — pinneapple_tools.automation.adapters.openfoam (v07)
- patch_fluxes f — pinneapple_tools.automation.verify (v07)
- patch_mean f — pinneapple_tools.automation.verify (v07)
- patch_test f — pinneapple_simulation.numerical_solvers.plane_stress_fem (v03)

- PatchDesign C — pinneapple_design.surrogate_inverse.transdermal_patch (v06)
- PatchSpec C — pinneapple_simulation.pipelines.freecad_openfoam.cad (v03)
- path_axes f — pinneapple_simulation.flight.turbulence (v03)
- path_to_next f — pinneapple_lab.curation (v07)
- payload f — pinneapple_tools.visualization.dashboard.html (v07)
- payload_track_check f — pinneapple_worldmodel.robot_perception (v04)
- PCAResidualDetector C — pinneapple_security.cps_bench (v05)
- PCASurrogate C — pinneapple_neural.workflows.hybrid_surrogate_physics (v04)
- pcb_thermal f — pinneapple_physics.pde_environment.presets.engineering (v02)
- PCEResult C — pinneapple_analysis.uncertainty.pce (v05)
- pcg f — pinneapple_simulation.accelerated_foam.solvers (v03)
- PCGrad C — pinneapple_neural.trainer.loss_balancer (v04)
- PCParams C — pinneapple_systems.sports.football.pitch_control (v06)
- pd_tracking f — pinneapple_simulation.robot_sim.sim (v03)
- pde f — pinneapple_core.loss (v01)
- pde_from_sympy f — pinneapple_physics.symbolic_pde.compiler (v02)
- pde_residual_loss f — pinneapple_quantum.loss.schrodinger (v04)
- PDEDeclaration C — pinneapple_analysis.surrogate.pde (v05)
- PDEFamily C — pinneapple_physics.pde_environment.capabilities (v02)
- PDEParser C — pinneapple_physics.codegen.fdm_script_generator (v02)
- PDEResidualGuidance C — pinneapple_neural.architectures.pinns.physics_diffusion (v04)
- PDESynthGenerator C — pinneapple_data.synth.pde (v04)
- PDETaskSampler C — pinneapple_adaptation.meta_learning.task_sampler (v04)
- PDETermSpec C — pinneapple_physics.pde_environment.spec (v02)
- PDETwinModel C — pinneapple_flow.twin (v07)
- pdf_widgets f — pinneapple_data.formfill.fill (v04)
- peak_rss_gb f — pinneapple_condense.benchmark (v02)
- pec_boundary f — pinneapple_physics.maxwell_checks (v02)
- peclet f — pinneapple_physics.physics_audit.regime (v02)
- pedestrian_response f — pinneapple_reason.modes.structural (v05)
- pendulum_period f — pinneapple_simulation.external_solvers.newton_physics.validation (v03)
- pendulum_rk4_reference f — pinneapple_tools.benchmark_suite.deeponet_ground_truth (v07)
- PendulumVideo C — pinneapple_lab.experiments.discovery (v07)
- PennyLaneBackend C — pinneapple_quantum.backends.backend (v04)
- per_field f — pinneapple_physics.metrics (v02)
- per_sample_error f — pinneapple_analysis.surrogate.limits (v05)
- per_sample_rel_l2 f — pinneapple_tools.benchmark_suite.crossframework.evaluate (v07)
- per_tag_losses f — pinneapple_physics.pde_environment.presets.tag_geometry (v02)
- perception_checks f — pinneapple_worldmodel.robot_perception (v04)
- performance f — pinneapple_design.aero.aircraft (v06)
- performance f — pinneapple_systems.operations.control (v06)
- Periodic C — pinneapple_core.transforms (v01)
- periodic_cnn2d f — pinneapple_physics.multiscale.learned (v02)
- periodic_elm_fit f — pinneapple_systems.time_series.solar_cycle (v06)
- periodic_grf f — pinneapple_simulation.numerical_solvers.operator_benchmarks (v03)
- PeriodicBC C — pinneapple_physics.symbolic_pde.bc (v02)
- PeriodicFit C — pinneapple_systems.time_series.solar_cycle (v06)
- Periodogram C — pinneapple_physics.pde_environment.presets.astroml_bridge (v02)
- permutation_entropy f — pinneapple_systems.time_series.predictability (v06)
- permutation_marginalized f — pinneapple_decision.debias (v05)
- PerQuantitySurrogate C — pinneapple_neural.workflows.hybrid_surrogate_physics (v04)
- Persistence C — pinneapple_systems.time_series.forecast_lab.baselines (v06)
- persistence f — pinneapple_physics.weather.evaluate (v02)
- petermann_factor f — pinneapple_physics.non_hermitian.pt_dimer (v02)
- Phase C — pinneapple_simulation.numerical_solvers.free_surface (v03)
- phase_angle f — pinneapple_physics.astro.phenomena (v02)

- phase_portrait f — pinneapple_systems.time_series.enso.pipeline (v06)
- phase_randomized_surrogate f — pinneapple_systems.time_series.predictability (v06)
- phase_scene f — pinneapple_simulation.electromagnetics.antenna.scene3d (v03)
- PhaseClimatology C — pinneapple_systems.time_series.forecast_lab.baselines (v06)
- PhenomenonEntry C — pinneapple_problemdesign.knowledge.physics_knowledge (v02)
- PhotorealisticEnhancer C — pinneapple_worldmodel.photorealistic_enhancer (v04)
- physical_features f — pinneapple_simulation.electromagnetics.antenna.surrogates (v03)
- PhysicalDatasetBuilder C — pinneapple_pdb.builder (v04)
- PhysicalProblem C — pinneapple_physics.physical_problem (v02)
- PhysicalRef C — pinneapple_query.refs (v02)
- PhysicalSample C — pinneapple_data.physical_sample (v04)
- PhysicalSampleDataset C — pinneapple_data.noether_dataset (v04)
- PhysicalSampleLike C — pinneapple_data.synth.sample_adapter (v04)
- PhysicalSchema C — pinneapple_pdb.builder (v04)
- PhysicalSpecification C — pinneapple_llm.qa.spec (v07)
- Physics C — pinneapple_simulation.pipelines.freecad_openfoam.case (v03)
- Physics C — pinneapple_systems.component_library.physics (v06)
- physics_ai_tree f — pinneapple_decision.tree (v05)
- physics_audit_check f — pinneapple_analysis.benchmark.validation (v05)
- physics_aware_rank f — pinneapple_arena.arena (v04)
- physics_check f — pinneapple_flow.nodes_ext.validation (v07)
- physics_checks f — pinneapple_worldmodel.physical_world (v04)
- physics_checks_section f — pinneapple_app.backend.core.surrogate_report (v07)
- physics_detector f — pinneapple_security.cps_bench (v05)
- physics_features f — pinneapple_systems.sports.football.pc_study (v06)
- physics_features f — pinneapple_systems.sports.football.xg (v06)
- physics_learning f — pinneapple_flow.nodes.physics_learning (v07)
- physics_metrics f — pinneapple_physics.darcy_checks (v02)
- physics_mismatch f — pinneapple_simulation.terramechanics.wong_reece (v03)
- physics_resistance f — pinneapple_design.thermal.heatsink.engine (v06)
- physics_spec f — pinneapple_flow.nodes_ext.arena_pipelines (v07)
- PhysicsAdapter C — pinneapple_app.backend.core.physics_adapter (v07)
- PhysicsAIConfig C — pinneapple_worldmodel.pipeline (v04)
- PhysicsAIDataset C — pinneapple_worldmodel.training_hooks (v04)
- PhysicsAIPipeline C — pinneapple_worldmodel.pipeline (v04)
- PhysicsAIPipelineResult C — pinneapple_worldmodel.pipeline (v04)
- PhysicsArea C — pinneapple_arena.pointwise.schema (v04)
- PhysicsAttention C — pinneapple_neural.architectures.neural_operators.transolver (v04)
- PhysicsAwareBase C — pinneapple_neural.architectures.physics_aware.base (v04)
- PhysicsAwareCatalog C — pinneapple_neural.architectures.physics_aware.registry (v04)
- PhysicsAwareNeuralNetwork C — pinneapple_neural.architectures.physics_aware.pann (v04)
- PhysicsAwareOutput C — pinneapple_neural.architectures.physics_aware.base (v04)
- PhysicsAwareRouter C — pinneapple_physics.physics_audit.routing (v02)
- PhysicsBackend C — pinneapple_core.backend (v01)
- PhysicsBenchmark C — pinneapple_worldmodel.benchmark (v04)
- PhysicsCase C — pinneapple_data.physics_case (v04)
- PhysicsChoice C — pinneapple_decision.schema (v05)
- PhysicsConfidenceScore C — pinneapple_analysis.verification.physics_confidence_score (v05)
- PhysicsCurriculum C — pinneapple_worldmodel.curriculum (v04)
- PhysicsDataset C — pinneapple_core.data (v01)
- PhysicsDatasetFactory C — pinneapple_worldmodel.dataset_factory (v04)
- PhysicsDecider C — pinneapple_decision.decider (v05)
- PhysicsDecisionEngine C — pinneapple_decision.loop (v05)
- PhysicsDecoderDataset C — pinneapple_worldmodel.training_hooks (v04)
- PhysicsDescriptorEncoder C — pinneapple_worldmodel.mega_model (v04)
- PhysicsDomain C — pinneapple_worldmodel.geometry (v04)

- PhysicsDomain2D C — pinneapple_design.geometry.gen.domains (v06)
- PhysicsDomain3D C — pinneapple_design.geometry.gen.domains3d (v06)
- PhysicsEnsemble C — pinneapple_physics.ensemble (v02)
- PhysicsFeatures C — pinneapple_systems.time_series.forecast_lab.physical (v06)
- PhysicsFoundationModel C — pinneapple_worldmodel.mega_model (v04)
- PhysicsGuardrail C — pinneapple_llm.guardrail (v07)
- PhysicsInformedDiffusion C — pinneapple_neural.architectures.pinns.physics_diffusion (v04)
- PhysicsInformedKoopmanAutoencoder C — pinneapple_neural.architectures.autoencoders.koopman_pi_ae (v04)
- PhysicsInformedNeuralOperator C — pinneapple_neural.architectures.neural_operators.pino (v04)
- PhysicsKnowledgeGraph C — pinneapple_physics.physics_audit.knowledge (v02)
- PhysicsLoss C — pinneapple_systems.cosimulation.losses (v06)
- PhysicsLossGenerator C — pinneapple_arena.pointwise.losses (v04)
- PhysicsLossHook C — pinneapple_neural.trainer.losses (v04)
- PhysicsLossMLP C — pinneapple_systems.time_series.forecast_lab.learned (v06)
- PhysicsMaterial C — pinneapple_twin3d.scene (v06)
- PhysicsMismatchWarning C — pinneapple_simulation.terramechanics.surrogate (v03)
- PhysicsModule C — pinneapple_core.module (v01)
- PhysicsMonitor C — pinneapple_neural.callbacks (v04)
- physicsnemo_fno f — pinneapple_neural.physicsnemo_bridge.field_operators (v04)
- physicsnemo_meshgraphnet f — pinneapple_neural.physicsnemo_bridge.field_operators (v04)
- PhysicsNeMoHook C — pinneapple_neural.callbacks (v04)
- PhysicsNemoModelAdapter C — pinneapple_neural.physicsnemo_bridge.adapters (v04)
- PhysicsNeMoSymBackend C — pinneapple_tools.benchmark_suite.backends.physicsnemo_sym (v07)
- PhysicsOptimizer C — pinneapple_core.optim (v01)
- PhysicsOrchestrator C — pinneapple_worldmodel.orchestrator (v04)
- PhysicsPipeline C — pinneapple_physics (v02)
- PhysicsPipeline C — pinneapple_tools.benchmark_suite.physics_pipeline (v07)
- PhysicsPlan C — pinneapple_design.intent.pipeline (v06)
- PhysicsPointCloud3D C — pinneapple_design.geometry.mesh.mesher_3d (v06)
- PhysicsPostModel C — pinneapple_neural.workflows.hybrid_surrogate_physics (v04)
- PhysicsProblem C — pinneapple_arena.pointwise.schema (v04)
- PhysicsProps C — pinneapple_twin3d.scene (v06)
- PhysicsRenderer C — pinneapple_worldmodel.physics_renderer (v04)
- PhysicsResidualDetector C — pinneapple_security.cps (v05)
- PhysicsScenario C — pinneapple_worldmodel.scenario (v04)
- PhysicsSimulator C — pinneapple_worldmodel.simulator (v04)
- PhysicsSolverBridge C — pinneapple_worldmodel.physics_solver_bridge (v04)
- PhysicsSpec C — pinneapple_problemdesign.schema (v02)
- PhysicsSurrogate C — pinneapple_design.design_optimizer.surrogate (v06)
- PhysicsTool C — pinneapple_worldmodel.physics_tools (v04)
- PhysicsToolRegistry C — pinneapple_worldmodel.physics_tools (v04)
- PhysicsTransferAdapter C — pinneapple_adaptation.transfer_learning.adapter (v04)
- PhysicsValidator C — pinneapple_analysis.validation.validator (v05)
- PhysicsValidityMonitor C — pinneapple_physics.physics_audit.monitor (v02)
- PhysicsWarning C — pinneapple_neural.callbacks (v04)
- PhysicsWorldModel C — pinneapple_worldmodel.model (v04)
- picard f — pinneapple_condense.flow (v02)
- pick f — pinneapple_design.aero.verify (v06)
- pick_corner f — pinneapple_lab.seal (v07)
- PIDBaseline C — pinneapple_systems.operations.control (v06)
- PIDController C — pinneapple_systems.component_modeling.control (v06)
- PIDiffConfig C — pinneapple_neural.architectures.pinns.physics_diffusion (v04)
- PIELM C — pinneapple_neural.architectures.pinns.pielm (v04)
- PIELMFactoryAdapter C — pinneapple_neural.architectures.pinns.pielm (v04)
- pig_iron_ladle f — pinneapple_twin3d.wear.presets (v06)

- pillow_block f — pinneapple_design.geometry.gen.cadquery_advanced (v06)
- pin_array f — pinneapple_condense.flow (v02)
- pin_fin f — pinneapple_flow.nodes_ext.use_cases (v07)
- pin_fin_heat f — pinneapple_physics.closed_form.heat (v02)
- pin_fin_heatsink f — pinneapple_design.geometry.gen.cadquery_advanced (v06)
- pin_fin_theta f — pinneapple_physics.closed_form.heat (v02)
- pin_sample f — pinneapple_data.device (v04)
- pin_sdf f — pinneapple_condense.meshless2d (v02)
- pinball_loss f — pinneapple_systems.time_series.metrics_ext.probabilistic (v06)
- pinball_loss_torch f — pinneapple_analysis.uncertainty.quantile (v05)
- PinFinModel C — pinneapple_analysis.inverse_problems.models (v05)
- PINN C — pinneapple_physics.pinn_solver.factory.pinn_factory (v02)
- pinn f — pinneapple_flow.nodes.solvers (v07)
- pinn_checks f — pinneapple_analysis.benchmark.builtin.fields (v05)
- pinn_factory_adapter f — pinneapple_neural.architectures.pinns.vpinn (v04)
- pinn_heat f — pinneapple_flow.nodes_ext.validation_gallery (v07)
- pinn_job f — pinneapple_analysis.benchmark.builtin.inverse_heat_impl (v05)
- pinn_model f — pinneapple_analysis.validation.physics_learning_cases (v05)
- pinn_operation_costs f — pinneapple_analysis.cost.torch_ops (v05)
- pinn_problem f — pinneapple_physics.symbolic_pde.pinn (v02)
- PINNAdapter C — pinneapple_neural.architectures.adapters.pinn (v04)
- PINNArenaBenchmark C — pinneapple_tools.benchmark_suite.benchmark (v07)
- PINNBase C — pinneapple_neural.architectures.pinns.base (v04)
- PINNBatch C — pinneapple_data.adapters.pinn_batch_builders (v04)
- PINNCatalog C — pinneapple_neural.architectures.pinns.registry (v04)
- PINNeAPPleModelNode C — pinneapple_systems.cosimulation.adapters (v06)
- PinneappleModelsBackend C — pinneapple_tools.benchmark_suite.backends.pinneapple_models_backend (v07)
- PINNeAPPleProfiler C — pinneapple_neural.trainer.hpc (v04)
- PinneappleSpec C — pinneapple_problemdesign.schema (v02)
- PINNeProblemNode C — pinneapple_systems.cosimulation.adapters (v06)
- PINNExtrapolationForecaster C — pinneapple_systems.time_series.enso.pinn (v06)
- PINNFactory C — pinneapple_physics.pinn_solver.factory.pinn_factory (v02)
- PINNLSTM C — pinneapple_neural.architectures.pinns.pinn_lstm (v04)
- PINNMapping C — pinneapple_physics.pinn_solver.io.mappings (v02)
- PINNMethod C — pinneapple_analysis.benchmark.builtin.fields (v05)
- PINNMonitor C — pinneapple_analysis.validation.pinn (v05)
- PINNNode C — pinneapple_systems.cosimulation.node (v06)
- PINNOutput C — pinneapple_neural.architectures.pinns.base (v04)
- PINNProblem C — pinneapple_analysis.validation.pinn (v05)
- PINNProblem C — pinneapple_tools.benchmark_suite.crossframework.problems (v07)
- PINNProblemSpec C — pinneapple_physics.pinn_solver.factory.pinn_factory (v02)
- PINNRansMethod C — pinneapple_analysis.benchmark.builtin.airfoil (v05)
- PINNRechargeForecaster C — pinneapple_systems.time_series.enso.pinn (v06)
- PINNRefinement C — pinneapple_design.design_optimizer.refinement (v06)
- PINNReport C — pinneapple_analysis.validation.pinn (v05)
- PINNsFormer C — pinneapple_neural.architectures.pinns.pinnsformer (v04)
- pinnsformer_case f — pinneapple_physics.paper_problems (v02)
- PINNWithParameters C — pinneapple_physics.symbolic_pde.pinn (v02)
- pino_physics_fn f — pinneapple_neural.architectures.neural_operators.pino (v04)
- pip_install f — pinneapple_arena.external.installer (v04)
- pip_install_command f — pinneapple_arena.external.local_command (v04)
- Pipe C — pinneapple_systems.digital_twin.pipe_loop.plant (v06)
- pipe_bend f — pinneapple_simulation.pipelines.freecad_openfoam.studies (v03)
- pipe_bend_flanged f — pinneapple_design.geometry.gen.cadquery_advanced (v06)
- pipe_bend_scene f — pinneapple_twin3d.demo (v06)

- pipe_flow_3d f — pinneapple_physics.pde_environment.presets.cfd (v02)
- pipe_loss f — pinneapple_physics.closed_form.pump_npsh (v02)
- pipe_surface f — pinneapple_simulation.numerical_solvers.internal_flow (v03)
- pipe_tee f — pinneapple_design.geometry.gen.cadquery_advanced (v06)
- pipe_twin f — pinneapple_flow.nodes.autolab (v07)
- PipeFlow C — pinneapple_lab.experiments.pipes (v07)
- PipeFlowDomain3D C — pinneapple_design.geometry.gen.domains3d (v06)
- Pipeline C — pinneapple_systems.process_components.water_hammer (v06)
- pipeline f — pinneapple_physics (v02)
- pipeline_call f — pinneapple_tools.automation.adapters.pipeline (v07)
- PipelineConfig C — pinneapple_llm.qa.pipeline (v07)
- PipelineSpec C — pinneapple_simulation.pipelines.freecad_openfoam.pipeline (v03)
- PipeSpec C — pinneapple_systems.process_components.pipe_network_1d (v06)
- PipeState C — pinneapple_systems.process_components.pipe_network_1d (v06)
- PipeVanillaPINN C — pinneapple_systems.component_library.components (v06)
- pit_ensemble f — pinneapple_systems.time_series.metrics_ext.probabilistic (v06)
- pit_gaussian f — pinneapple_systems.time_series.metrics_ext.probabilistic (v06)
- pit_histogram f — pinneapple_systems.time_series.metrics_ext.probabilistic (v06)
- pitch_control_figure f — pinneapple_systems.sports.football.plots (v06)
- pitch_surface f — pinneapple_systems.sports.football.pitch_control (v06)
- pitot_pressure_ratio f — pinneapple_physics.closed_form.compressible (v02)
- piv_grid_count f — pinneapple_perception.piv_tools (v04)
- piv_parameter_guide f — pinneapple_perception.piv_tools (v04)
- piv_velocity_field f — pinneapple_perception.video_piv (v04)
- piv_velocity_sequence f — pinneapple_perception.video_piv (v04)
- pk_two_compartment f — pinneapple_physics.pde_environment.presets.multidisciplinary (v02)
- place f — pinneapple_flow.palette (v07)
- place f — pinneapple_simulation.rover.autolab.render (v03)
- plain_solver f — pinneapple_simulation.accelerated_foam.case (v03)
- Plan C — pinneapple_analysis.benchmark.runner (v05)
- Plan C — pinneapple_autolab.core (v07)
- Plan C — pinneapple_problemdesign.schema (v02)
- plan f — pinneapple_core.onboarding.newproject (v01)
- plan f — pinneapple_design.aero.dataset (v06)
- plan f — pinneapple_design.intent.pipeline (v06)
- plan f — pinneapple_flow (v07)
- plan f — pinneapple_flow.planner (v07)
- plan f — pinneapple_hub.zoo.publish (v07)
- plan f — pinneapple_physics.astro.journey (v02)
- plan f — pinneapple_simulation.electromagnetics.reflector.study_report (v03)
- plan f — pinneapple_tools.automation.runner (v07)
- plan_campaign f — pinneapple_data.doe.campaign (v04)
- plan_experiment f — pinneapple_lab.curation (v07)
- plan_from_answers f — pinneapple_flow.planner.wizard (v07)
- plan_heuristic f — pinneapple_flow.planner.heuristic (v07)
- plan_llm f — pinneapple_flow.planner.llm (v07)
- plan_mesh f — pinneapple_design.fitmesh.mesh_plan (v06)
- plan_split f — pinneapple_hub.zoo.parquet (v07)
- planar_face_mesh f — pinneapple_tools.nvidia.brep (v07)
- planar_two_link_torques f — pinneapple_simulation.multibody.arm (v03)
- planck_lambda f — pinneapple_physics.astro.stars (v02)
- plane_figure f — pinneapple_simulation.pipelines.freecad_openfoam.post (v03)
- plane_points f — pinneapple_simulation.electromagnetics.antenna.ava (v03)
- plane_strain_2d_default f — pinneapple_physics.pde_environment.presets.structural (v02)
- plane_stress_2d_default f — pinneapple_physics.pde_environment.presets.structural (v02)
- plane_surface f — pinneapple_simulation.electromagnetics.antenna.scene3d (v03)

- plane_wave f — pinneapple_simulation.electromagnetics.reflector.po (v03)
- PlaneStressSolution C — pinneapple_simulation.numerical_solvers.plane_stress_fem (v03)
- planet_elements f — pinneapple_physics.astro.orbits (v02)
- planet_events f — pinneapple_physics.astro.phenomena (v02)
- planetary_transits f — pinneapple_physics.astro.phenomena (v02)
- PlaneWaveTFSF C — pinneapple_simulation.electromagnetics.fdtd (v03)
- PlannedStep C — pinneapple_simulation.electromagnetics.reflector.study_report (v03)
- Planner C — pinneapple_autolab.core (v07)
- Planner C — pinneapple_query.core (v02)
- planner_rows f — pinneapple_flow.examples (v07)
- PlanStep C — pinneapple_problemdesign.schema (v02)
- plant_outlets f — pinneapple_systems.operations.exchanger (v06)
- plant_physics f — pinneapple_data.process_optimizer (v04)
- PlantConfig C — pinneapple_systems.operations.exchanger (v06)
- plasma_dispersion_omega f — pinneapple_simulation.electromagnetics.waves (v03)
- plate f — pinneapple_simulation.flight.mass (v03)
- plate_2d f — pinneapple_simulation.numerical_solvers.heat_fv_reference (v03)
- plate_fin_bodies f — pinneapple_design.design_optimizer.presets (v06)
- plate_precomputed f — pinneapple_analysis.inverse_problems.heat_transfer (v05)
- PlateFinHeatsink C — pinneapple_presets.cases.heatsink (v02)
- PlateModel C — pinneapple_analysis.inverse_problems.models (v05)
- platform_tag f — pinneapple_simulation.external_solvers.modelica.fmi_compat (v03)
- plausibility f — pinneapple_systems.sports.football.kinematics (v06)
- Player C — pinneapple_analysis.verification.tool_recommendation (v05)
- PlaywrightDriver C — pinneapple_tools.automation.display (v07)
- plot f — pinneapple_flow.nodes.viz (v07)
- plot f — pinneapple_tools (v07)
- plot_acf_pacf f — pinneapple_systems.time_series.eda.plots (v06)
- plot_backtest f — pinneapple_systems.time_series.viz.forecast (v06)
- plot_boundary f — pinneapple_tools.visualization.mesh (v07)
- plot_calibration_curve f — pinneapple_tools.model_evaluation.visualization (v07)
- plot_centerline_velocity f — pinneapple_neural.predictor.visualize (v04)
- plot_chain_threshold f — pinneapple_physics.non_hermitian.viz (v02)
- plot_collocation f — pinneapple_tools.visualization.pinn (v07)
- plot_comparison f — pinneapple_design.bricks.report (v06)
- plot_convergence f — pinneapple_physics.blackhole.figures (v02)
- plot_coupling_plane f — pinneapple_physics.non_hermitian.viz (v02)
- plot_design_opt_convergence f — pinneapple_neural.predictor.visualize (v04)
- plot_ep_laws f — pinneapple_physics.non_hermitian.viz (v02)
- plot_error f — pinneapple_tools.visualization.fields (v07)
- plot_error_histograms f — pinneapple_tools.model_evaluation.visualization (v07)
- plot_error_map_1d f — pinneapple_neural.predictor.visualize (v04)
- plot_error_map_2d f — pinneapple_neural.predictor.visualize (v04)
- plot_fan_charts f — pinneapple_systems.time_series.forecast_lab.plots (v06)
- plot_fem_result f — pinneapple_tools.visualization.solver (v07)
- plot_field_1d f — pinneapple_neural.predictor.visualize (v04)
- plot_field_2d f — pinneapple_neural.predictor.visualize (v04)
- plot_field_comparison f — pinneapple_arena.visualizer (v04)
- plot_flow_panel f — pinneapple_tools.visualization.vortex (v07)
- plot_forecast_horizon f — pinneapple_systems.time_series.viz.forecast (v06)
- plot_fvm_result f — pinneapple_tools.visualization.solver (v07)
- plot_gradient_magnitude f — pinneapple_tools.visualization.pinn (v07)
- plot_history f — pinneapple_neural.trainer.viz (v04)
- plot_hovmoller f — pinneapple_systems.time_series.enso.plots (v06)
- plot_internal_flow_summary f — pinneapple_neural.predictor.visualize (v04)
- plot_isosurface_3d f — pinneapple_neural.predictor.postprocess (v04)

- plot_lbm_flow f — pinneapple_tools.visualization.vortex (v07)
- plot_learning f — pinneapple_simulation.rover.autolab.visuals (v03)
- plot_loss_curve f — pinneapple_neural.predictor.visualize (v04)
- plot_loss_curves f — pinneapple_arena.visualizer (v04)
- plot_loss_history f — pinneapple_tools.visualization.pinn (v07)
- plot_memorise_vs_learn f — pinneapple_systems.time_series.forecast_lab.plots (v06)
- plot_mesh f — pinneapple_tools.visualization.mesh (v07)
- plot_metrics_table f — pinneapple_arena.visualizer (v04)
- plot_model_comparison f — pinneapple_simulation.rover.autolab.visuals (v03)
- plot_model_comparison_1d f — pinneapple_neural.predictor.visualize (v04)
- plot_model_comparison_2d f — pinneapple_neural.predictor.visualize (v04)
- plot_multi_loss f — pinneapple_tools.visualization.pinn (v07)
- plot_overfitting f — pinneapple_systems.time_series.forecast_lab.plots (v06)
- plot_pareto_front_2d f — pinneapple_neural.predictor.visualize (v04)
- plot_parity f — pinneapple_systems.time_series.viz.forecast (v06)
- plot_pde_residual f — pinneapple_tools.visualization.pinn (v07)
- plot_phase_portrait f — pinneapple_systems.time_series.enso.plots (v06)
- plot_pinn_prediction f — pinneapple_tools.visualization.pinn (v07)
- plot_pit f — pinneapple_systems.time_series.forecast_lab.plots (v06)
- plot_plumes f — pinneapple_systems.time_series.enso.plots (v06)
- plot_point_cloud f — pinneapple_tools.visualization.mesh (v07)
- plot_q_criterion_2d f — pinneapple_tools.visualization.vortex (v07)
- plot_q_criterion_3d f — pinneapple_tools.visualization.vortex (v07)
- plot_real_vs_predicted f — pinneapple_tools.model_evaluation.visualization (v07)
- plot_residuals f — pinneapple_systems.time_series.viz.forecast (v06)
- plot_residuals f — pinneapple_tools.visualization.solver (v07)
- plot_rolling_forecast f — pinneapple_systems.time_series.viz.forecast (v06)
- plot_route_risk f — pinneapple_simulation.rover.autolab.visuals (v03)
- plot_rover_series f — pinneapple_simulation.rover.viz (v03)
- plot_run f — pinneapple_simulation.vehicle.viz (v03)
- plot_scalar f — pinneapple_tools.visualization.fields (v07)
- plot_scalar_3d f — pinneapple_tools.visualization.fields (v07)
- plot_site_map f — pinneapple_simulation.rover.autolab.visuals (v03)
- plot_skill f — pinneapple_physics.blackhole.figures (v02)
- plot_skill_vs_horizon f — pinneapple_systems.time_series.forecast_lab.plots (v06)
- plot_skill_vs_lead f — pinneapple_systems.time_series.enso.plots (v06)
- plot_solution f — pinneapple_flow.nodes._common (v07)
- plot_solver_output f — pinneapple_tools.visualization.solver (v07)
- plot_spectrum f — pinneapple_physics.non_hermitian.viz (v02)
- plot_spring_barrier f — pinneapple_systems.time_series.enso.plots (v06)
- plot_streamlines f — pinneapple_arena.visualizer (v04)
- plot_streamlines f — pinneapple_tools.visualization.fields (v07)
- plot_streamlines_2d f — pinneapple_neural.predictor.visualize (v04)
- plot_streamlines_2d_from_model f — pinneapple_neural.predictor.postprocess (v04)
- plot_surprise_calendar f — pinneapple_systems.time_series.forecast_lab.plots (v06)
- plot_surrogate_tests f — pinneapple_systems.time_series.forecast_lab.plots (v06)
- plot_testbed f — pinneapple_simulation.vehicle.viz (v03)
- plot_traverse f — pinneapple_simulation.rover.autolab.visuals (v03)
- plot_trend f — pinneapple_systems.time_series.eda.plots (v06)
- plot_uq f — pinneapple_arena.visualizer (v04)
- plot_validation f — pinneapple_simulation.terramechanics.testbed (v03)
- plot_validation f — pinneapple_systems.time_series.enso.plots (v06)
- plot_value_histograms f — pinneapple_tools.model_evaluation.visualization (v07)
- plot_vectors f — pinneapple_tools.visualization.fields (v07)
- plot_velocity_magnitude_slice f — pinneapple_neural.predictor.visualize (v04)
- plot_velocity_slice f — pinneapple_neural.predictor.visualize (v04)

- plot_volume_slice f — pinneapple_neural.predictor.postprocess (v04)
- plot_vortex_identification f — pinneapple_tools.visualization.vortex (v07)
- plot_vorticity f — pinneapple_tools.visualization.vortex (v07)
- plot_vorticity_slice f — pinneapple_neural.predictor.visualize (v04)
- plot_voxel_3d f — pinneapple_tools.visualization.voxel (v07)
- plot_voxel_histogram f — pinneapple_tools.visualization.voxel (v07)
- plot_voxel_slice f — pinneapple_tools.visualization.voxel (v07)
- plot_wheel_series f — pinneapple_simulation.rover.viz (v03)
- po_currents f — pinneapple_simulation.electromagnetics.reflector.po (v03)
- POD C — pinneapple_neural.architectures.rom.pod (v04)
- pod_dmd f — pinneapple_flow.nodes_ext.architectures (v07)
- pod_dmd_stepper f — pinneapple_neural.architectures.rom.steppers (v04)
- pod_dmd_wake f — pinneapple_flow.nodes_ext.validation_gallery (v07)
- pod_opinf_stepper f — pinneapple_neural.architectures.rom.steppers (v04)
- PODWarmStart C — pinneapple_simulation.accelerated_foam.warm_start (v03)
- point f — pinneapple_simulation.flight.mass (v03)
- point_charge_field f — pinneapple_simulation.electromagnetics.statics (v03)
- point_cloud f — pinneapple_condense.meshless (v02)
- point_cloud_domain f — pinneapple_simulation.meshless.domains (v03)
- point_inside f — pinneapple_simulation.pipelines.freecad_openfoam.steps (v03)
- point_mesh_distance f — pinneapple_tools.nvidia.brep (v07)
- point_set_digest f — pinneapple_tools.benchmark_suite.crossframework.problems (v07)
- PointCloud C — pinneapple_core.field (v01)
- PointCloud C — pinneapple_design.geometry.ops.pointcloud (v06)
- PointD C — pinneapple_check.numerics (v05)
- PointKernelOperator C — pinneapple_neural.architectures.neural_operators.uno (v04)
- PointMetrics C — pinneapple_systems.time_series.metrics_ext.point (v06)
- PointObsOperator C — pinneapple_analysis.inverse_problems.obs_operator (v05)
- PointOperatorBlock C — pinneapple_neural.architectures.neural_operators.uno (v04)
- points_in_terrain_shadow f — pinneapple_tools.visualization.studio.shadows (v07)
- pointwise_error f — pinneapple_analysis.validation.error_map (v05)
- Poisson2D C — pinneapple_arena.problems (v04)
- Poisson2DTask C — pinneapple_tools.benchmark_suite.tasks.poisson_2d (v07)
- poisson_2d f — pinneapple_analysis.validation.pinn (v05)
- poisson_2d_default f — pinneapple_physics.pde_environment.presets.academics (v02)
- poisson_fdm_spectral f — pinneapple_flow.nodes_ext.solver_registry (v07)
- poisson_fem_vs_pinn f — pinneapple_flow.nodes_ext.pinn_examples (v07)
- poisson_periodic_fft f — pinneapple_simulation.numerical_solvers.spectral (v03)
- Polar C — pinneapple_design.aero.aircraft (v06)
- polar_mesh f — pinneapple_simulation.electromagnetics.mesh (v03)
- Policy C — pinneapple_twin3d.wear.optimize (v06)
- policy_for f — pinneapple_tools.rea_bridge.licenses (v07)
- policy_for_path f — pinneapple_tools.rea_bridge.licenses (v07)
- poly_cells f — pinneapple_data.cae.dataset (v04)
- Polygon C — pinneapple_simulation.electromagnetics.machines.geometry (v03)
- Polygon C — pinneapple_worldmodel.geometry (v04)
- polygon_area f — pinneapple_design.fitmesh.catalog (v06)
- polygon_area f — pinneapple_simulation.electromagnetics.machines.geometry (v03)
- polygon_contains f — pinneapple_physics.pde_environment.presets._curve_geometry (v02)
- polygon_domain f — pinneapple_simulation.meshless.domains (v03)
- polygon_perimeter_sample f — pinneapple_physics.pde_environment.presets._curve_geometry (v02)
- polyline_segments f — pinneapple_simulation.electromagnetics.statics (v03)
- PolyMesh C — pinneapple_data.cae.model (v04)
- polynomial_chaos f — pinneapple_analysis.uncertainty.pce (v05)
- polytropic_head_from_psi f — pinneapple_systems.process_components.similarity_map (v06)
- PolytropicPathResult C — pinneapple_systems.process_components.polytropic_path (v06)

- pooled f — pinneapple_physics.metrics (v02)
- Port C — pinneapple_flow.registry (v07)
- port_volume_m3 f — pinneapple_physics.closed_form.helmholtz_resonator (v02)
- PortDesign C — pinneapple_design.surrogate_inverse.reflex_port (v06)
- Pose C — pinneapple_simulation.rover.geometry (v03)
- pose_at f — pinneapple_simulation.rover.autolab.render (v03)
- posed_parts f — pinneapple_simulation.vehicle.viz (v03)
- position f — pinneapple_physics.astro.facts (v02)
- position_angle f — pinneapple_physics.astro.frames (v02)
- post_depletion_decay_rate_constant_per_day f — pinneapple_physics.closed_form.membrane_diffusion (v02)
- post_filter_transfer f — pinneapple_physics.multiscale.filters (v02)
- post_step f — pinneapple_simulation.pipelines.freecad_openfoam.steps (v03)
- PosteriorSamples C — pinneapple_analysis.inverse_problems.estimation (v05)
- PotentialNet C — pinneapple_analysis.benchmark.builtin.airfoil_impl (v05)
- power_board f — pinneapple_reason.cases (v05)
- power_spectrum f — pinneapple_systems.time_series.eda.plots (v06)
- PowerPCB C — pinneapple_presets.cases.pcb (v02)
- poynting_balance f — pinneapple_physics.maxwell_checks (v02)
- poynting_flux_dft f — pinneapple_simulation.electromagnetics.fdtd (v03)
- pp_test f — pinneapple_systems.time_series.audit.tests (v06)
- prandtl f — pinneapple_physics.physics_audit.regime (v02)
- pratt f — pinneapple_flow.nodes_ext.flight (v07)
- pratt_check f — pinneapple_simulation.flight.encounter (v03)
- pratt_gust_load f — pinneapple_simulation.flight.turbulence (v03)
- precession_matrix f — pinneapple_physics.astro.frames (v02)
- precompute f — pinneapple_hub.zoo.demo (v07)
- predict f — pinneapple_arena.pointwise.training (v04)
- predict f — pinneapple_hub.zoo (v07)
- predict f — pinneapple_neural (v04)
- predict f — pinneapple_neural.trainer.inference (v04)
- predict f — pinneapple_systems.sports.football.ball (v06)
- predict_and_plot f — pinneapple_tools.visualization.pinn (v07)
- predict_constant_velocity f — pinneapple_systems.sports.football.kinematics (v06)
- predict_frequency_hz f — pinneapple_design.surrogate_inverse.reflex_port (v06)
- predict_gnn f — pinneapple_design.aero.train (v06)
- predict_limited f — pinneapple_systems.sports.football.kinematics (v06)
- predict_mlp f — pinneapple_design.aero.train (v06)
- predict_models f — pinneapple_systems.sports.football.pc_study (v06)
- predict_rate_mg_day f — pinneapple_design.surrogate_inverse.transdermal_patch (v06)
- predict_single_step f — pinneapple_neural.workflows.dense_volume_operator (v04)
- predict_single_step_parametric f — pinneapple_neural.workflows.parametric_dense_volume_operator (v04)
- predict_stress_pa f — pinneapple_design.surrogate_inverse.suspension_arm (v06)
- predict_temperature_c f — pinneapple_design.surrogate_inverse.brake_rotor (v06)
- predict_temperature_c f — pinneapple_design.surrogate_inverse.heatsink_fins (v06)
- predict_to_frd f — pinneapple_simulation.external_solvers.calculix.virtual_ccx (v03)
- predictability_report f — pinneapple_systems.time_series.predictability (v06)
- prediction_card f — pinneapple_simulation.electromagnetics.antenna.design (v03)
- prediction_quality f — pinneapple_arena.pointwise.training (v04)
- prediction_samples f — pinneapple_systems.sports.football.kin_study (v06)
- prediction_study f — pinneapple_systems.sports.football.kin_study (v06)
- prediction_view f — pinneapple_neural.architectures.rom.views (v04)
- predictive_policy f — pinneapple_twin3d.wear.optimize (v06)
- predictor_module f — pinneapple_hub.zoo.export (v07)
- PrefetchConfig C — pinneapple_data.zarr_prefetch (v04)
- PrefetchZarrUPDIterable C — pinneapple_data.zarr_prefetch (v04)

- `preflight` f — `pinneapple_data.preflight` (v04)
- `preflight` f — `pinneapple_systems.digital_twin.product.commission` (v06)
- `preflight_check` f — `pinneapple_analysis.benchmark.validation` (v05)
- `PreparationError` C — `pinneapple_analysis.benchmark.runner` (v05)
- `prepare` f — `pinneapple_analysis.benchmark.api` (v05)
- `prepare` f — `pinneapple_hub.zoo.publish` (v07)
- `prepare` f — `pinneapple_twin3d.wear.optimize` (v06)
- `prepare_dataset` f — `pinneapple_llm.finetune` (v07)
- `prepare_digital_twin_task` f — `pinneapple_orchestration.tasks.digital_twin_tasks` (v01)
- `prepare_problem` f — `pinneapple_analysis.benchmark.runner` (v05)
- `prepare_step` f — `pinneapple_simulation.pipelines.freecad_openfoam.steps` (v03)
- `prepare_tutorial` f — `pinneapple_simulation.accelerated_foam.benchmark` (v03)
- `preprocess` f — `pinneapple_systems.time_series.solar_cycle` (v06)
- `PreprocessPipeline` C — `pinneapple_neural.trainer.preprocess` (v04)
- `PreprocessStep` C — `pinneapple_neural.trainer.preprocess` (v04)
- `Preset` C — `pinneapple_analysis.inverse_problems.presets` (v05)
- `Preset` C — `pinneapple_presets.core` (v02)
- `preset_bodies` f — `pinneapple_design.design_optimizer.presets` (v06)
- `preset_info` f — `pinneapple_design.design_optimizer.presets` (v06)
- `preset_name` f — `pinneapple_physics.pde_environment.presets.tag_geometry` (v02)
- `PresetCache` C — `pinneapple_presets.cases.airfoil` (v02)
- `PresetDraftError` C — `pinneapple_problemdesign.preset_authoring` (v02)
- `PresetListItem` C — `pinneapple_app.backend.routers.problems` (v07)
- `PresetPhysics` C — `pinneapple_app.backend.core.physics_adapter` (v07)
- `PresetSurrogate` C — `pinneapple_presets.surrogate` (v02)
- `pressure_drop` f — `pinneapple_tools.automation.verify` (v07)
- `pressure_gradient` f — `pinneapple_systems.process_components.non_newtonian_pipe_flow` (v06)
- `pressure_only` f — `pinneapple_simulation.rover.checks` (v03)
- `pressure_ratio` f — `pinneapple_physics.closed_form.rocket_nozzle` (v02)
- `PressureProfile` C — `pinneapple_systems.process_components.non_newtonian_pipe_flow` (v06)
- `PressureVessel` C — `pinneapple_presets.cases.pressure_vessel` (v02)
- `preview` f — `pinneapple_data.cae.dataset` (v04)
- `preview` f — `pinneapple_design.qualitative.preview` (v06)
- `prim_to_cons` f — `pinneapple_physics.blackhole.hydro_numba` (v02)
- `principal_curvatures` f — `pinneapple_design.geometry.ops.curvature` (v06)
- `print_result` f — `pinneapple_arena.external.__main__` (v04)
- `Prior` C — `pinneapple_analysis.inverse_problems.estimation` (v05)
- `prior` f — `pinneapple_condense.exceptions` (v02)
- `prioritize` f — `pinneapple_data.doe.coverage` (v04)
- `prius_2004` f — `pinneapple_simulation.electromagnetics.machines.geometry` (v03)
- `prius_drive` f — `pinneapple_flow.nodes_ext.electromagnetics` (v07)
- `prius_flux_map` f — `pinneapple_flow.nodes_ext.electromagnetics` (v07)
- `prius_machine` f — `pinneapple_flow.nodes_ext.electromagnetics` (v07)
- `prius_phase_resistance` f — `pinneapple_simulation.electromagnetics.machines.drive` (v03)
- `prius_skew` f — `pinneapple_flow.nodes_ext.electromagnetics` (v07)
- `prius_thermal` f — `pinneapple_flow.nodes_ext.em_prius` (v07)
- `prius_tolerances` f — `pinneapple_flow.nodes_ext.electromagnetics` (v07)
- `prius_torque_curve` f — `pinneapple_simulation.electromagnetics.machines.verification` (v03)
- `ProbabilisticMetrics` C — `pinneapple_systems.time_series.metrics_ext.probabilistic` (v06)
- `Probe` C — `pinneapple_simulation.electromagnetics.fdtd` (v03)
- `probe` f — `pinneapple_core.onboarding.catalog` (v01)
- `probe_dataset` f — `pinneapple_arena.external.dataset_probe` (v04)
- `probe_strouhal` f — `pinneapple_simulation.numerical_solvers.cylinder_wake` (v03)
- `Problem` C — `pinneapple_analysis.benchmark.registry` (v05)
- `Problem` C — `pinneapple_simulation.electromagnetics.reflector.study_report` (v03)
- `Problem` C — `pinneapple_simulation.meshless.problems` (v03)

- problem f — pinneapple_lab.curation (v07)
- problem_from_spec f — pinneapple_decision.schema (v05)
- problem_from_spec f — pinneapple_design.design_optimizer.spec (v06)
- problem_headline f — pinneapple_lab.curation (v07)
- problem_in_dim f — pinneapple_arena.pointwise.training (v04)
- problem_out_dim f — pinneapple_arena.pointwise.training (v04)
- ProblemBuilder C — pinneapple_physics.pde_environment.builder (v02)
- ProblemConfig C — pinneapple_arena.config (v04)
- ProblemContext C — pinneapple_design.fitmesh.context (v06)
- ProblemDefinition C — pinneapple_app.backend.core.problem (v07)
- ProblemParameter C — pinneapple_arena.pointwise.schema (v04)
- problems f — pinneapple_analysis.benchmark.registry (v05)
- problems f — pinneapple_flow.examples (v07)
- problems f — pinneapple_flow.references (v07)
- ProblemSpec C — pinneapple_physics.pde_environment.spec (v02)
- ProblemSpec C — pinneapple_problemdesign.schema (v02)
- ProblemStatement C — pinneapple_worldmodel.orchestrator (v04)
- ProblemStore C — pinneapple_registry.problem_store (v01)
- ProblemTemplate C — pinneapple_design.intent.template (v06)
- profile_cd0 f — pinneapple_design.intent.templates.glider_wing (v06)
- profile_cd0 f — pinneapple_simulation.flight.catalog (v03)
- profile_likelihood f — pinneapple_analysis.inverse_problems.identifiability (v05)
- profile_ops f — pinneapple_analysis.cost.torch_ops (v05)
- profile_params f — pinneapple_design.surrogate_inverse.brake_rotor (v06)
- profile_params f — pinneapple_design.surrogate_inverse.heatsink_fins (v06)
- profile_verdicts f — pinneapple_physics.darcy_checks (v02)
- ProfilerConfig C — pinneapple_neural.trainer.hpc (v04)
- profiles f — pinneapple_core.onboarding.doctor (v01)
- progressive_failure f — pinneapple_design.bricks.statics (v06)
- project_forcing f — pinneapple_systems.time_series.enso.pinn (v06)
- project_integral f — pinneapple_physics.conservation (v02)
- project_lines f — pinneapple_physics.weather.viz (v02)
- project_points_to_surface f — pinneapple_design.geometry.gen.primitives (v06)
- project_reachable f — pinneapple_systems.sports.football.kinematics (v06)
- project_tree f — pinneapple_tools.hpo_experiments.utils.verify_runtime (v07)
- projection_inputs f — pinneapple_simulation.accelerated_foam.learned (v03)
- ProjectionWarmStart C — pinneapple_simulation.accelerated_foam.warm_start (v03)
- prolong f — pinneapple_physics.multiscale.filters (v02)
- promote_model f — pinneapple_app.backend.routers.admin (v07)
- PromoteRequest C — pinneapple_app.backend.routers.admin (v07)
- ProofGraph C — pinneapple_analysis.proof.graph (v05)
- Prop C — pinneapple_design.intent.materials (v06)
- propagate f — pinneapple_analysis.uncertainty.tolerance (v05)
- propagate f — pinneapple_flow.nodes.methods (v07)
- propagate f — pinneapple_physics.astro.smallbodies (v02)
- propagate_space f — pinneapple_analysis.uncertainty.tolerance (v05)
- propagate_two_body_j2 f — pinneapple_physics.astro.conjunction (v02)
- propagation_constant f — pinneapple_simulation.electromagnetics.waves (v03)
- Propeller C — pinneapple_simulation.flight.propulsion (v03)
- proper f — pinneapple_lab.explorer (v07)
- properties f — pinneapple_design.aero.geometry (v06)
- proposal f — pinneapple_flow.planner.extensions (v07)
- proposals f — pinneapple_flow.planner.extensions (v07)
- propose_mapping f — pinneapple_data.unified (v04)
- propose_parameters f — pinneapple_design.fitmesh.llm (v06)
- Protocol C — pinneapple_systems.time_series.forecast_lab.study (v06)

- protocol f — pinneapple_systems.time_series.enso.study (v06)
- Provenance C — pinneapple_data.contracts.models (v04)
- Provenance C — pinneapple_llm.qa.spec (v07)
- provenance f — pinneapple_security.attestation (v05)
- ProvenanceRecord C — pinneapple_analysis.verification.provenance (v05)
- prune f — pinneapple_flow.autocalls (v07)
- psd f — pinneapple_simulation.flight.turbulence (v03)
- psd_figure f — pinneapple_simulation.flight.figures (v03)
- psd_slope f — pinneapple_physics.blackhole.convergence (v02)
- psd_temporal f — pinneapple_simulation.flight.turbulence (v03)
- pseudonymize f — pinneapple_security.privacy (v05)
- pseudonymize_column f — pinneapple_security.privacy (v05)
- pt_chain f — pinneapple_flow.nodes_ext.validation (v07)
- pt_violation f — pinneapple_physics.non_hermitian.pt_dimer (v02)
- PTChain C — pinneapple_physics.non_hermitian.pt_dimer (v02)
- PTDimer C — pinneapple_physics.non_hermitian.pt_dimer (v02)
- PublicSeries C — pinneapple_systems.time_series.datasets.public (v06)
- publish f — pinneapple_hub.zoo.publish (v07)
- publish_model_task f — pinneapple_orchestration.tasks.registry_tasks (v01)
- publish_space f — pinneapple_hub.zoo.publish (v07)
- PublishedForecasts C — pinneapple_physics.weather.evaluate (v02)
- pump f — pinneapple_tools.automation.scripts.freecad_gui_job (v07)
- PumpCurve C — pinneapple_systems.process_components.water_hammer (v06)
- pumped_main f — pinneapple_reason.cases (v05)
- PumpModifiedMLP C — pinneapple_systems.component_library.components (v06)
- PumpNPSH C — pinneapple_presets.cases.pump_npsh (v02)
- PurgedKFoldSplitter C — pinneapple_systems.time_series.validation.splitters (v06)
- PurgedWalkForwardSplitter C — pinneapple_systems.time_series.validation.splitters (v06)
- push_to_hub f — pinneapple_hub.hub (v07)
- py_compile_all f — pinneapple_tools.hpo_experiments.utils.verify_runtime (v07)
- PythonAdapter C — pinneapple_core.workflows.adapters (v01)
- PywinautoDriver C — pinneapple_tools.automation.display (v07)

Q

- q f — pinneapple_reason.modes._util (v05)
- Q4Grid C — pinneapple_design.topology.problems (v06)
- q_from_gain f — pinneapple_simulation.electromagnetics.reflector.po (v03)
- QiskitBackend C — pinneapple_quantum.backends.backend (v04)
- QTrainer C — pinneapple_quantum.training.trainer (v04)
- QTrainerConfig C — pinneapple_quantum.training.trainer (v04)
- QTrainHistory C — pinneapple_quantum.training.trainer (v04)
- quadratic_in_T f — pinneapple_systems.process_components.reaction_kinetics (v06)
- QuadraticRSM C — pinneapple_autolab.surrogates (v07)
- QualitativeModel C — pinneapple_design.qualitative.models (v06)
- QualitativePreview C — pinneapple_design.qualitative.preview (v06)
- quality f — pinneapple_design.fitmesh.mesh_gen (v06)
- quality f — pinneapple_design.geometry_library.meshops (v06)
- quality_key f — pinneapple_hub.library (v07)
- QualityPipeline C — pinneapple_llm.qa.pipeline (v07)
- QualityReport C — pinneapple_llm.qa.report (v07)
- quantify_uncertainty f — pinneapple_analysis.verification.uncertainty_inverse (v05)
- quantile_bins f — pinneapple_analysis.uncertainty.conformal_study (v05)
- QuantileConfig C — pinneapple_analysis.uncertainty.quantile (v05)
- QuantileHead C — pinneapple_analysis.uncertainty.quantile (v05)
- QuantileLoss C — pinneapple_analysis.uncertainty.quantile (v05)
- QuantitativeCheck C — pinneapple_design.qualitative.preview (v06)

- Quantity C — pinneapple_data.physical_units (v04)
- Quantity C — pinneapple_design.intent.spec (v06)
- Quantity C — pinneapple_design.qualitative.models (v06)
- Quantity C — pinneapple_llm.qa.spec (v07)
- Quantity C — pinneapple_physics.physical_problem (v02)
- quantity_map f — pinneapple_llm.qa.spec (v07)
- quantity_of f — pinneapple_data.cae.dataset (v04)
- QuantityGuess C — pinneapple_data.physical_units (v04)
- quantum_kernel f — pinneapple_quantum.pipeline.presets (v04)
- QuantumBackend C — pinneapple_quantum.backends.backend (v04)
- QuantumCircuitConfig C — pinneapple_quantum.circuits.base (v04)
- QuantumCollocationDataset C — pinneapple_quantum.data.quantum_dataset (v04)
- QuantumModel C — pinneapple_quantum.models.quantum_model (v04)
- QuantumObservable C — pinneapple_quantum.data.quantum_dataset (v04)
- QuantumState C — pinneapple_quantum.data.quantum_dataset (v04)
- quasi_steady_valid f — pinneapple_systems.digital_twin.pipe_loop.plant (v06)
- quat_from_matrix f — pinneapple_simulation.multibody.tree (v03)
- quat_from_matrix f — pinneapple_simulation.rover.geometry (v03)
- quat_from_matrix f — pinneapple_tools.visualization.studio.animation (v07)
- quat_mul f — pinneapple_simulation.multibody.tree (v03)
- quat_multiply f — pinneapple_simulation.flight.rigid_body (v03)
- quat_to_dcm f — pinneapple_simulation.flight.rigid_body (v03)
- quat_to_euler f — pinneapple_simulation.flight.rigid_body (v03)
- quat_to_matrix f — pinneapple_simulation.multibody.tree (v03)
- Query C — pinneapple_query.core (v02)
- query_checks f — pinneapple_hub.zoo (v07)
- query_checks f — pinneapple_hub.zoo.checks (v07)
- query_demo f — pinneapple_condense.query_benchmark (v02)
- Question C — pinneapple_design.intent.spec (v06)
- quickstart f — pinneapple (v01)
- quintic f — pinneapple_simulation.multibody.arm (v03)
- quiz f — pinneapple_physics.astro.facts (v02)
- qz_field f — pinneapple_simulation.electromagnetics.reflector.compact_range (v03)
- qz_figure f — pinneapple_simulation.electromagnetics.reflector.study (v03)
- qz_metrics f — pinneapple_simulation.electromagnetics.reflector.qz (v03)
- qz_metrics_at f — pinneapple_simulation.electromagnetics.reflector.study_report (v03)
- qz_study f — pinneapple_simulation.electromagnetics.reflector.study (v03)

R

- R2 C — pinneapple_neural.trainer.metrics (v04)
- r2 f — pinneapple_physics.metrics (v02)
- r2 f — pinneapple_systems.time_series.metrics (v06)
- r2_fn f — pinneapple_systems.time_series.metrics (v06)
- racecar_domain f — pinneapple_autolab.domains.racecar (v07)
- RaceCarAero C — pinneapple_autolab.domains.racecar (v07)
- radec f — pinneapple_physics.astro.frames (v02)
- radial_fd f — pinneapple_physics.closed_form.pipe_insulation (v02)
- radial_power_spectrum f — pinneapple_analysis.uncertainty.posterior_metrics (v05)
- RadialLayers C — pinneapple_simulation.electromagnetics.analytic (v03)
- radiate f — pinneapple_simulation.electromagnetics.reflector.po (v03)
- radiation_pattern f — pinneapple_tools.visualization.electromagnetics (v07)
- radiator f — pinneapple_design.geometry.vehicle_cad (v06)
- RadiossMesh C — pinneapple_simulation.external_solvers.openradioss.vtk_reader (v03)
- ramp f — pinneapple_tools.visualization.studio.colormap (v07)
- random_block_mask f — pinneapple_data.synth.operators.masks (v04)
- random_craters f — pinneapple_simulation.rover.terrain (v03)

- random_designs f — pinneapple_design.thermal.heatsink.monotone (v06)
- random_points f — pinneapple_analysis.validation.physics_learning (v05)
- random_rocks f — pinneapple_simulation.rover.terrain (v03)
- random_vorticity f — pinneapple_physics.multiscale.ns2d (v02)
- random_walk f — pinneapple_systems.time_series.synthetic (v06)
- RandomForestForecaster C — pinneapple_systems.time_series.models.classical (v06)
- RandomizerConfig C — pinneapple_worldmodel.reality_randomizer (v04)
- RandomSearchProposer C — pinneapple_arena.autoresearch (v04)
- RangeLog C — pinneapple_simulation.rover.contact (v03)
- rank_by_accuracy f — pinneapple_arena.arena (v04)
- rank_geometries f — pinneapple_design.fitmesh.llm (v06)
- rank_key f — pinneapple_analysis.verification.pipeline (v05)
- rank_models f — pinneapple_arena.pointwise.training (v04)
- RankedItem C — pinneapple_tools.hpo_experiments.models (v07)
- rans_residuals f — pinneapple_analysis.benchmark.builtin.airfoil_impl (v05)
- RansNet C — pinneapple_analysis.benchmark.builtin.airfoil_impl (v05)
- rapid_steady_state_profile f — pinneapple_systems.process_components.pipe_network_1d (v06)
- rasterize f — pinneapple_design.geometry_library.render (v06)
- rasterize_topdown f — pinneapple_tools.nvidia.omniverse (v07)
- rate_limit f — pinneapple_systems.excitation (v06)
- rate_of_change f — pinneapple_systems.time_series.features.engineering (v06)
- RateLimiter C — pinneapple_security.web (v05)
- RateRule C — pinneapple_security.web (v05)
- ratio f — pinneapple_flow.nodes_ext.cfd (v07)
- raw_distribution f — pinneapple_decision.debias (v05)
- raw_residual f — pinneapple_physics.multiscale.comparison (v02)
- rayleigh f — pinneapple_physics.physics_audit.regime (v02)
- rayleigh_qsca f — pinneapple_simulation.electromagnetics.mie (v03)
- raymer_takeoff f — pinneapple_simulation.flight.takeoff (v03)
- raymer_variants f — pinneapple_simulation.flight.takeoff (v03)
- RayPaths C — pinneapple_physics.blackhole.raytrace (v02)
- rbf_fd_weights f — pinneapple_simulation.meshless.methods (v03)
- RBFCollocationSolver C — pinneapple_simulation.numerical_solvers.meshfree (v03)
- RBFInterpolator C — pinneapple_design.geometry.ops.meshfree (v06)
- RBFNetwork C — pinneapple_neural.architectures.reservoir_computing.rbf (v04)
- RCBase C — pinneapple_neural.architectures.reservoir_computing.base (v04)
- RCDischarge C — pinneapple_science.em.bench (v02)
- RCOutput C — pinneapple_neural.architectures.reservoir_computing.base (v04)
- RDPAccountant C — pinneapple_security.dp (v05)
- reachable_radius f — pinneapple_systems.sports.football.kinematics (v06)
- ReaClient C — pinneapple_tools.rea_bridge.client (v07)
- Reaction C — pinneapple_systems.process_components.reaction_kinetics (v06)
- reaction_diffusion_2d_default f — pinneapple_physics.pde_environment.presets.industry (v02)
- reaction_exact f — pinneapple_physics.paper_problems (v02)
- ReactionDiffusion1D C — pinneapple_simulation.numerical_solvers.reaction_diffusion (v03)
- ReactionNetwork C — pinneapple_systems.process_components.reaction_kinetics (v06)
- read_abaqus_inp_mesh f — pinneapple_simulation.external_solvers.cfd_formats.abaqus_reader (v03)
- read_any f — pinneapple_data.cae.io (v04)
- read_binary_block f — pinneapple_simulation.external_solvers.openfoam.binary_reader (v03)
- read_boundary f — pinneapple_data.cae.foam (v04)
- read_boundary f — pinneapple_twin3d.openfoam (v06)
- read_boundary_field f — pinneapple_twin3d.openfoam (v06)
- read_boundary_values f — pinneapple_data.cae.foam (v04)
- read_case f — pinneapple_simulation.pipelines.freecad_openfoam.post (v03)
- read_case_displacements f — pinneapple_simulation.external_solvers.openradioss.vtk_reader (v03)
- read_cases f — pinneapple_core.workflows.datasets (v01)

- read_cgns_mesh_and_fields f — pinneapple_simulation.external_solvers.cfd_formats.cgns_reader (v03)
- read_coefficients f — pinneapple_design.aero.case (v06)
- read_dat f — pinneapple_data.simulation_metadata.openfoam (v04)
- read_dat_displacements f — pinneapple_simulation.external_solvers.calculix.frd (v03)
- read_dataset f — pinneapple_lab.dataset (v07)
- read_deck f — pinneapple_data.cae.inp (v04)
- read_document f — pinneapple_data.formfill.extract (v04)
- read_dump f — pinneapple_simulation.accelerated_foam.ldu (v03)
- read_dxf_polylines f — pinneapple_simulation.electromagnetics.machines.interop (v03)
- read_energy_error f — pinneapple_simulation.external_solvers.openradioss.log_reader (v03)
- read_energy_errors f — pinneapple_simulation.external_solvers.openradioss.log_reader (v03)
- read_exodus f — pinneapple_simulation.external_solvers.cfd_formats.exodus_reader (v03)
- read_face_compact_list f — pinneapple_simulation.external_solvers.openfoam.binary_reader (v03)
- read_faces f — pinneapple_data.cae.foam (v04)
- read_faces f — pinneapple_twin3d.openfoam (v06)
- read_farfield f — pinneapple_simulation.electromagnetics.antenna.scene3d (v03)
- read_femm_lua_polygons f — pinneapple_simulation.electromagnetics.machines.interop (v03)
- read_field f — pinneapple_data.cae.foam (v04)
- read_field f — pinneapple_simulation.accelerated_foam.case (v03)
- read_filled f — pinneapple_data.formfill.fill (v04)
- read_flow_runs f — pinneapple_hub.library.flowruns (v07)
- read_fluent_mesh_nodes f — pinneapple_simulation.external_solvers.cfd_formats.fluent_mesh_reader (v03)
- read_frd f — pinneapple_data.cae.frd (v04)
- read_frd f — pinneapple_simulation.external_solvers.calculix.frd (v03)
- read_geotaichi f — pinneapple_simulation.external_solvers.geotaichi (v03)
- read_gltf f — pinneapple_tools.visualization.studio.scene (v07)
- read_header f — pinneapple_simulation.external_solvers.openfoam.binary_reader (v03)
- read_inp f — pinneapple_simulation.external_solvers.calculix.inp (v03)
- read_internal f — pinneapple_design.aero.case (v06)
- read_internal_field f — pinneapple_simulation.external_solvers.openfoam.binary_reader (v03)
- read_label_list f — pinneapple_simulation.external_solvers.openfoam.binary_reader (v03)
- read_labels f — pinneapple_data.cae.foam (v04)
- read_ldr f — pinneapple_design.bricks.ldraw (v06)
- read_manifest f — pinneapple_design.geometry_library.store (v06)
- read_members f — pinneapple_security.archives (v05)
- read_meshio f — pinneapple_data.cae.io (v04)
- read_nearfield f — pinneapple_simulation.electromagnetics.antenna.scene3d (v03)
- read_npz f — pinneapple_hub.zoo.parquet (v07)
- read_obj f — pinneapple_tools.visualization.studio.scene (v07)
- read_parquet f — pinneapple_hub.zoo.parquet (v07)
- read_particle_frames f — pinneapple_simulation.particle_dynamics.geo_mpm (v03)
- read_point_bundle f — pinneapple_simulation.external_solvers.cfd_formats.convert (v03)
- read_points f — pinneapple_data.cae.foam (v04)
- read_points f — pinneapple_data.cae.io (v04)
- read_points f — pinneapple_twin3d.openfoam (v06)
- read_polymesh f — pinneapple_data.cae.foam (v04)
- read_problem f — pinneapple_design.fitmesh.llm (v06)
- read_profiles f — pinneapple_simulation.numerical_solvers.free_surface (v03)
- read_readings f — pinneapple_systems.digital_twin.product.commission (v06)
- read_result3d f — pinneapple_design.aero.case3d (v06)
- read_results f — pinneapple_simulation.external_solvers.fluidx3d (v03)
- read_run f — pinneapple_tools.benchmark_suite.crossframework.records (v07)
- read_runs f — pinneapple_hub.library (v07)
- read_sampled_scalar_field f — pinneapple_simulation.external_solvers.openfoam.sampling (v03)
- read_shell_thickness f — pinneapple_simulation.external_solvers.openradioss.deck (v03)

- read_sidecar f — pinneapple_data.contracts.tables (v04)
- read_solver_dict f — pinneapple_simulation.accelerated_foam.benchmark (v03)
- read_splash_manifest f — pinneapple_simulation.external_solvers.openfoam.splash_packer (v03)
- read_stage_geometry f — pinneapple_tools.nvidia.omniverse (v07)
- read_starccm_cgns f — pinneapple_simulation.external_solvers.cfd_formats.starccm_reader (v03)
- read_starccm_table f — pinneapple_simulation.external_solvers.cfd_formats.starccm_reader (v03)
- read_stl f — pinneapple_data.cae.io (v04)
- read_stl f — pinneapple_design.geometry_library.exporters (v06)
- read_stl f — pinneapple_tools.visualization.studio.scene (v07)
- read_surface_mesh f — pinneapple_simulation.external_solvers.cfd_formats.convert (v03)
- read_surface_scalar f — pinneapple_check.fields (v05)
- read_system f — pinneapple_simulation.accelerated_foam.ldu (v03)
- read_table f — pinneapple_analysis.surrogate.data (v05)
- read_table f — pinneapple_check.checks.models (v05)
- read_table f — pinneapple_data.contracts.tables (v04)
- read_table f — pinneapple_simulation.electromagnetics.antenna.runner (v03)
- read_table_with_roles f — pinneapple_analysis.surrogate.contracts (v05)
- read_time f — pinneapple_data.cae.foam (v04)
- read_timestamp f — pinneapple_systems.digital_twin.product.commission (v06)
- read_trajectories f — pinneapple_data.datasets.deepmind_mgn (v04)
- read_tunables f — pinneapple_arena.autoresearch (v04)
- read_upload f — pinneapple_data.simulation_metadata (v04)
- read_vector_field_points f — pinneapple_simulation.external_solvers.openfoam.binary_reader (v03)
- read_vtk f — pinneapple_simulation.numerical_solvers.lbm_tunnel (v03)
- read_vtk_mesh f — pinneapple_simulation.external_solvers.openradioss.vtk_reader (v03)
- read_vtp f — pinneapple_tools.visualization.studio.scene (v07)
- readings f — pinneapple_analysis.benchmark.builtin.inverse_heat_impl (v05)
- readings f — pinneapple_systems.digital_twin.pipe_loop.plant (v06)
- readme f — pinneapple_presets.report (v02)
- readme f — pinneapple_science.report (v02)
- ReadReport C — pinneapple_design.bricks.ldraw (v06)
- ReaError C — pinneapple_tools.rea_bridge.client (v07)
- RealityRandomizer C — pinneapple_worldmodel.reality_randomizer (v04)
- realizability_mismatch f — pinneapple_physics.multiscale.filters (v02)
- RealSpectralConv2d C — pinneapple_neural.architectures.neural_operators.fno_export (v04)
- RealtimePipeline C — pinneapple_systems.digital_twin.realtime_pipeline (v06)
- RealtimePipelineConfig C — pinneapple_systems.digital_twin.realtime_pipeline (v06)
- ReaResult C — pinneapple_tools.rea_bridge.client (v07)
- reason f — pinneapple_mcp.tools (v07)
- reason f — pinneapple_reason.engine (v05)
- reason_check f — pinneapple_mcp.tools (v07)
- reason_library f — pinneapple_mcp.tools (v07)
- Reasoner C — pinneapple_reason.engine (v05)
- ReaUnavailable C — pinneapple_tools.rea_bridge.client (v07)
- recession f — pinneapple_systems.digital_twin.recession (v06)
- RechargeOscillatorForecaster C — pinneapple_systems.time_series.enso.forecasters (v06)
- recipes f — pinneapple_hub.library.sources (v07)
- reciprocity f — pinneapple_physics.maxwell_checks (v02)
- recommend f — pinneapple_core.onboarding.recommend (v01)
- recommend f — pinneapple_llm.workbench.models (v07)
- recommend f — pinneapple_physics.physics_audit.improvement (v02)
- recommend f — pinneapple_systems.operations.control (v06)
- recommend_architecture f — pinneapple_analysis.verification.architecture_recommendation (v05)
- recommend_for_problem f — pinneapple_app.backend.core.model_catalog (v07)
- recommend_main f — pinneapple_core.onboarding.cli (v01)
- recommend_method f — pinneapple_problemdesign.method_selection (v02)

- recommend_method_from_spec f — pinneapple_problemdesign.method_selection (v02)
- recommend_models f — pinneapple_app.backend.core.problem_registry (v07)
- recommend_models f — pinneapple_app.backend.routers.models (v07)
- recommend_tools f — pinneapple_analysis.verification.tool_recommendation (v05)
- Recommendation C — pinneapple_core.onboarding.recommend (v01)
- Recommendation C — pinneapple_llm.workbench.models (v07)
- Recommendation C — pinneapple_physics.physics_audit.improvement (v02)
- RecommendationResult C — pinneapple_veriphysics.recommend (v05)
- recommended_solver f — pinneapple_app.backend.core.problem_registry (v07)
- reconstruct_interpolate f — pinneapple_systems.digital_twin.signal_reconstruction (v06)
- reconstruct_model_based f — pinneapple_systems.digital_twin.signal_reconstruction (v06)
- reconstruct_spectral f — pinneapple_systems.digital_twin.signal_reconstruction (v06)
- reconstruct_ssa f — pinneapple_systems.digital_twin.signal_reconstruction (v06)
- reconstruct_wavelet f — pinneapple_systems.digital_twin.signal_reconstruction (v06)
- reconstruction_view f — pinneapple_neural.architectures.rom.views (v04)
- ReconstructionResult C — pinneapple_systems.digital_twin.signal_reconstruction (v06)
- record f — pinneapple_hub.library.flowruns (v07)
- record_checks f — pinneapple_veriphysics.ledger (v05)
- record_replay f — pinneapple_autolab.dashboard (v07)
- record_runs f — pinneapple_hub.library (v07)
- recorded_dashboard f — pinneapple_autolab.domains.racecar (v07)
- rect_cavity_modes f — pinneapple_simulation.electromagnetics.waves (v03)
- rect_cavity_resonance f — pinneapple_simulation.electromagnetics.waves (v03)
- rect_cavity_te101_q f — pinneapple_simulation.electromagnetics.waves (v03)
- rect_mesh f — pinneapple_simulation.electromagnetics.mesh (v03)
- rect_te_mn_fields f — pinneapple_simulation.electromagnetics.waves (v03)
- rect_waveguide_cutoff f — pinneapple_simulation.electromagnetics.waves (v03)
- rect_waveguide_modes f — pinneapple_simulation.electromagnetics.waves (v03)
- Rectangle C — pinneapple_worldmodel.geometry (v04)
- rectangle f — pinneapple_design.geometry.gen.sdf_shapes (v06)
- rectangle_mask f — pinneapple_simulation.numerical_solvers.lbm (v03)
- rectangular_beam f — pinneapple_design.surrogate_inverse.meshes (v06)
- rectangular_section_properties f — pinneapple_systems.process_components.beam_statics (v06)
- RecurrentCatalog C — pinneapple_neural.architectures.recurrent.registry (v04)
- RecurrentConfig C — pinneapple_systems.time_series.models.recurrent (v06)
- RecurrentModelBase C — pinneapple_neural.architectures.recurrent.base (v04)
- redact f — pinneapple_flow.store (v07)
- redact f — pinneapple_llm.workbench.finetune (v07)
- redact f — pinneapple_security.privacy (v05)
- Reduced C — pinneapple_condense.algebraic (v02)
- reduced_block f — pinneapple_condense.learned (v02)
- ReducedModel C — pinneapple_neural.architectures.rom.study (v04)
- redundancy f — pinneapple_data.doe.coverage (v04)
- ref f — pinneapple_flow.references (v07)
- refan_concave_caps f — pinneapple_design.geometry_library.meshops (v06)
- Reference C — pinneapple_check.config (v05)
- Reference C — pinneapple_condense.query (v02)
- Reference C — pinneapple_design.fitmesh.catalog (v06)
- Reference C — pinneapple_presets.core (v02)
- Reference C — pinneapple_science.core (v02)
- reference f — pinneapple_design.aero.geometry_onnx (v06)
- reference_cantilever f — pinneapple_design.bricks.goals (v06)
- reference_data f — pinneapple_hub.zoo.web (v07)
- reference_decay f — pinneapple_physics.astro.decay (v02)
- reference_for_spec f — pinneapple_design.design_optimizer.spec (v06)
- reference_observables f — pinneapple_condense.substructure (v02)

- reference_solve f — pinneapple_design.thermal.fin_array_speed (v06)
- reference_solve f — pinneapple_design.thermal.heatsink.surrogate (v06)
- reference_solve_batch f — pinneapple_design.thermal.heatsink.surrogate (v06)
- reference_stack f — pinneapple_condense.package (v02)
- reference_trajectory f — pinneapple_physics.astro.surrogate (v02)
- reference_transient f — pinneapple_condense.transient (v02)
- reference_value f — pinneapple_flow.nodes_ext.examples (v07)
- ReferenceData C — pinneapple_llm.qa.references (v07)
- refilter f — pinneapple_physics.multiscale.filters (v02)
- refine_ladder f — pinneapple_science.numerics (v02)
- refinement f — pinneapple_condense.figures (v02)
- refinement_study f — pinneapple_condense.benchmark (v02)
- RefinementConfig C — pinneapple_design.design_optimizer.refinement (v06)
- RefinementResult C — pinneapple_design.design_optimizer.refinement (v06)
- reflect_box f — pinneapple_simulation.numerical_solvers.sph_boundaries (v03)
- reflect_field f — pinneapple_analysis.validation.metamorphic (v05)
- ReflectorMesh C — pinneapple_simulation.electromagnetics.reflector.geometry (v03)
- refraction_deg f — pinneapple_physics.astro.astrometry (v02)
- refractory_lining f — pinneapple_physics.pde_environment.presets.engineering (v02)
- regime_features f — pinneapple_physics.physics_audit.regime (v02)
- regime_series f — pinneapple_flow.nodes_ext.core_ts (v07)
- regime_tags_for f — pinneapple_pdb.shard (v04)
- region_integral_A f — pinneapple_simulation.electromagnetics.postprocess (v03)
- region_mask f — pinneapple_physics.weather.events (v02)
- region_masks f — pinneapple_physics.darcy_checks (v02)
- region_rel_l2 f — pinneapple_analysis.surrogate.heldout (v05)
- region_stencils f — pinneapple_condense.meshless (v02)
- regional f — pinneapple_physics.weather.viz (v02)
- regional_errors f — pinneapple_physics.darcy_checks (v02)
- regional_lines f — pinneapple_physics.weather.viz (v02)
- regional_turboprop f — pinneapple_simulation.flight.catalog (v03)
- regions f — pinneapple_data.cae.geometry (v04)
- register f — pinneapple_design.geometry_library.core (v06)
- register f — pinneapple_design.intent.parse (v06)
- register f — pinneapple_lab.spec (v07)
- register f — pinneapple_llm.qa.validators.base (v07)
- register f — pinneapple_presets.registry (v02)
- register f — pinneapple_science.registry (v02)
- register f — pinneapple_tools.automation.adapters.base (v07)
- register f — pinneapple_tools.automation.runner_step (v07)
- register_adapter f — pinneapple_core.workflows.adapters (v01)
- register_advanced_templates f — pinneapple_design.geometry.gen.cadquery_advanced (v06)
- register_all f — pinneapple_neural.architectures.register_all (v04)
- register_all f — pinneapple_simulation.numerical_solvers.registry (v03)
- register_analytic f — pinneapple_physics.solving (v02)
- register_backend f — pinneapple_core.backend (v01)
- register_backend f — pinneapple_tools.benchmark_suite.registry (v07)
- register_benchmark f — pinneapple_pdb.benchmarks (v04)
- register_builder f — pinneapple_design.geometry.gen.primitives (v06)
- register_capability f — pinneapple_flow.planner.extensions (v07)
- register_component f — pinneapple_systems.component_library.registry (v06)
- register_countermeasure f — pinneapple_reason.library (v05)
- register_custom_nodes f — pinneapple_flow.runtime (v07)
- register_default_templates f — pinneapple_design.geometry.gen.cadquery_gen (v06)
- register_enhancer f — pinneapple_worldmodel.photorealistic_enhancer (v04)
- register_example_run f — pinneapple_flow.examples (v07)

- register_executor f — pinneapple_core.workflows.executors (v01)
- register_family_registry f — pinneapple_neural.architectures._registry_bridge (v04)
- register_hook f — pinneapple_flow.examples (v07)
- register_into_global f — pinneapple_neural.architectures.autoencoders.registry (v04)
- register_into_global f — pinneapple_neural.architectures.benchmarks.registry (v04)
- register_into_global f — pinneapple_neural.architectures.classical_ts.registry (v04)
- register_into_global f — pinneapple_neural.architectures.continuous.registry (v04)
- register_into_global f — pinneapple_neural.architectures.convolutions.registry (v04)
- register_into_global f — pinneapple_neural.architectures.graphnn.registry (v04)
- register_into_global f — pinneapple_neural.architectures.group_b_registry (v04)
- register_into_global f — pinneapple_neural.architectures.neural_operators.registry (v04)
- register_into_global f — pinneapple_neural.architectures.physics_aware.registry (v04)
- register_into_global f — pinneapple_neural.architectures.pinns.registry (v04)
- register_into_global f — pinneapple_neural.architectures.recurrent.registry (v04)
- register_into_global f — pinneapple_neural.architectures.reservoir_computing.registry (v04)
- register_into_global f — pinneapple_neural.architectures.rom.registry (v04)
- register_into_global f — pinneapple_neural.architectures.transformers.registry (v04)
- register_manifest f — pinneapple_lab.capture (v07)
- register_method f — pinneapple_analysis.benchmark.registry (v05)
- register_method f — pinneapple_physics.solving (v02)
- register_mode f — pinneapple_reason.library (v05)
- register_port_type f — pinneapple_flow.types (v07)
- register_powersgd_hook f — pinneapple_neural.trainer.hpc (v04)
- register_preset f — pinneapple_physics.pde_environment.presets.registry (v02)
- register_problem f — pinneapple_analysis.benchmark.registry (v05)
- register_problem f — pinneapple_arena.problems (v04)
- register_rea_tools f — pinneapple_tools.rea_bridge.tools (v07)
- register_references f — pinneapple_flow (v07)
- register_references f — pinneapple_flow.references (v07)
- register_rule f — pinneapple_flow (v07)
- register_rule f — pinneapple_flow.rules (v07)
- register_soil f — pinneapple_simulation.terramechanics.soils (v03)
- register_task f — pinneapple_tools.benchmark_suite.registry (v07)
- register_topk_hook f — pinneapple_neural.trainer.hpc (v04)
- Registry C — pinneapple_flow.registry (v07)
- Registry C — pinneapple_tools.model_services.registry (v07)
- registry f — pinneapple_flow.references (v07)
- registry f — pinneapple_models (v01)
- registry_key f — pinneapple_arena.pointwise.architectures (v04)
- registry_listing f — pinneapple_flow.nodes_ext.architectures (v07)
- Regression C — pinneapple_analysis.cost.profiler (v05)
- regression_metrics f — pinneapple_tools.model_evaluation.metrics (v07)
- regression_metrics_bundle f — pinneapple_neural.trainer.metrics (v04)
- RegressionMetrics C — pinneapple_neural.trainer.metrics (v04)
- RegularizerBase C — pinneapple_analysis.inverse_problems.regularization (v05)
- rel f — pinneapple_hub.library.sources (v07)
- rel_l2 f — pinneapple_analysis.reproduction (v05)
- rel_l2 f — pinneapple_check.checks.common (v05)
- rel_l2 f — pinneapple_simulation.numerical_solvers.operator_datasets (v03)
- rel_l2_rows f — pinneapple_flow.learn (v07)
- rel_rms f — pinneapple_check.numerics (v05)
- Relation C — pinneapple_analysis.validation.metamorphic (v05)
- RelationResult C — pinneapple_analysis.validation.metamorphic (v05)
- relations_from_spec f — pinneapple_analysis.validation.metamorphic (v05)
- relative_error f — pinneapple_design.surrogate_inverse.core (v06)
- relative_errors f — pinneapple_tools.nvidia.physicsnemo (v07)

- relative_l2 f — pinneapple_analysis.cost.equal_budget (v05)
- relative_l2 f — pinneapple_physics.ensemble (v02)
- relative_l2 f — pinneapple_physics.metrics (v02)
- relative_linf f — pinneapple_physics.metrics (v02)
- relative_residual f — pinneapple_physics.maxwell_checks (v02)
- release_rate_mg_day f — pinneapple_design.surrogate_inverse.transdermal_patch (v06)
- reliability f — pinneapple_systems.sports.football.metrics (v06)
- reliability_figure f — pinneapple_systems.sports.football.plots (v06)
- relief_noise f — pinneapple_tools.visualization.studio.shadows (v07)
- RelL2 C — pinneapple_neural.trainer.metrics (v04)
- ReLoBRaLo C — pinneapple_neural.trainer.loss_balancer (v04)
- reluctance_rotor f — pinneapple_flow.nodes_ext.design (v07)
- remaining_life_fields f — pinneapple_twin3d.wear.build (v06)
- remap f — pinneapple_physics.blackhole.convergence (v02)
- remap_r f — pinneapple_physics.blackhole.convergence (v02)
- remesh_surface f — pinneapple_design.geometry.ops.remesh (v06)
- remove_outlier_vertices f — pinneapple_design.geometry.ops.clean (v06)
- remove_self_intersections f — pinneapple_design.geometry.ops.clean (v06)
- render f — pinneapple_check.report (v05)
- render f — pinneapple_design.aero.render_blender (v06)
- render f — pinneapple_design.geometry_library (v06)
- render f — pinneapple_flow.nodes.viz (v07)
- render f — pinneapple_flow.nodes_ext.render_worker (v07)
- render f — pinneapple_lab.explorer (v07)
- render f — pinneapple_physics.blackhole.raytrace (v02)
- render f — pinneapple_tools.automation.workflow (v07)
- render f — pinneapple_tools.visualization.molecular (v07)
- render f — pinneapple_tools.visualization.studio (v07)
- render f — pinneapple_tools.visualization.studio.blender (v07)
- render_3d f — pinneapple_design.thermal.board_cooling.report (v06)
- render_3d f — pinneapple_simulation.electromagnetics.antenna.study (v03)
- render_3d f — pinneapple_systems.digital_twin.pipe_loop.report (v06)
- render_animation f — pinneapple_tools.visualization.studio.blender (v07)
- render_attractors f — pinneapple_systems.time_series.forecast_lab.renders (v06)
- render_blender f — pinneapple_design.bricks.visuals (v06)
- render_comparison_figure f — pinneapple_perception.change_detection (v04)
- render_cones f — pinneapple_systems.time_series.forecast_lab.renders (v06)
- render_datasheet f — pinneapple_data.formfill.report (v04)
- render_ensembles f — pinneapple_systems.time_series.forecast_lab.renders (v06)
- render_evidence_report_pdf f — pinneapple_veriphysics.evidence_report (v05)
- render_frame f — pinneapple_simulation.vehicle.viz (v03)
- render_frames f — pinneapple_design.bricks.visuals (v06)
- render_frames f — pinneapple_simulation.vehicle.dashboard (v03)
- render_frames f — pinneapple_systems.time_series.enso.render (v06)
- render_frames f — pinneapple_tools.visualization.molecular (v07)
- render_geometry f — pinneapple_design.geometry_library.render (v06)
- render_gif f — pinneapple_tools.visualization.pyvista_bridge (v07)
- render_hero f — pinneapple_simulation.rover.autolab.render (v03)
- render_html f — pinneapple_design.fitmesh.report (v06)
- render_html f — pinneapple_lab.reports (v07)
- render_html f — pinneapple_veriphysics.evidence_package (v05)
- render_html f — pinneapple_veriphysics.validation_lab (v05)
- render_html_report f — pinneapple_perception.change_detection (v04)
- render_json_report f — pinneapple_problemdesign.renderers.report_json (v02)
- render_landscape f — pinneapple_systems.time_series.forecast_lab.renders (v06)
- render_legend f — pinneapple_lab.seal (v07)

- render_like f — pinneapple_worldmodel.physical_world (v04)
- render_markdown_report f — pinneapple_problemdesign.renderers.report_md (v02)
- render_md f — pinneapple_lab.reports (v07)
- render_overview f — pinneapple_simulation.rover.autolab.render (v03)
- render_page f — pinneapple_lab.html (v07)
- render_particle_frames f — pinneapple_tools.visualization.studio.particles (v07)
- render_pdf f — pinneapple_lab.reports (v07)
- render_pdf f — pinneapple_veriphysics.evidence_package (v05)
- render_png f — pinneapple_tools.visualization.pyvista_bridge (v07)
- render_run f — pinneapple_autolab.domains.racecar (v07)
- render_scene f — pinneapple_tools.visualization.studio.pyvista_render (v07)
- render_seal f — pinneapple_lab.seal (v07)
- render_stage_gif f — pinneapple_tools.nvidia.omniverse (v07)
- render_stage_png f — pinneapple_tools.nvidia.omniverse (v07)
- render_still f — pinneapple_tools.visualization.studio.blender (v07)
- render_surface f — pinneapple_tools.visualization.studio.planetary (v07)
- render_traverse f — pinneapple_simulation.rover.autolab.render (v03)
- render_visualizations f — pinneapple_neural.predictor.visualize (v04)
- RendererConfig C — pinneapple_worldmodel.physics_renderer (v04)
- RenderResult C — pinneapple_worldmodel.physics_renderer (v04)
- repair f — pinneapple_design.bricks.search (v06)
- repair f — pinneapple_flow.planner.llm (v07)
- repair_mesh f — pinneapple_design.geometry.ops.repair (v06)
- RepairAttempt C — pinneapple_llm.qa.repair (v07)
- RepairLoop C — pinneapple_llm.qa.repair (v07)
- RepairOutcome C — pinneapple_llm.qa.repair (v07)
- Replay C — pinneapple_design.bricks.dynamics (v06)
- replay f — pinneapple_analysis.validation.arena.cases (v05)
- replay f — pinneapple_design.bricks.dynamics (v06)
- replay f — pinneapple_systems.operations.live (v06)
- replay_frames f — pinneapple_design.bricks.visuals (v06)
- replay_score f — pinneapple_security.anomaly (v05)
- ReplayResult C — pinneapple_analysis.validation.arena.cases (v05)
- ReplaySimulator C — pinneapple_autolab.replay (v07)
- replicate_sector f — pinneapple_simulation.electromagnetics.machines.mesher (v03)
- Reply C — pinneapple_llm.workbench.backend (v07)
- RepoFile C — pinneapple_tools.hpo_experiments.sources.github_fetcher (v07)
- RepoResults C — pinneapple_lab.experiments.repo_results (v07)
- Report C — pinneapple_check.result (v05)
- Report C — pinneapple_data.preflight.common (v04)
- report f — pinneapple_analysis.benchmark.api (v05)
- report f — pinneapple_flow.nodes.reason (v07)
- report f — pinneapple_flow.nodes.viz (v07)
- report f — pinneapple_simulation.electromagnetics.reflector.study_report (v03)
- report f — pinneapple_twin3d.wear.build (v06)
- report_equations f — pinneapple_check.checks.results (v05)
- report_html f — pinneapple_flow.decisions (v07)
- report_markdown f — pinneapple_flow.decisions (v07)
- report_markdown f — pinneapple_security.compliance (v05)
- report_markdown f — pinneapple_simulation.rover.autolab.report (v03)
- ReportDiff C — pinneapple_analysis.validation.report_artifact (v05)
- representatives f — pinneapple_design.design_optimizer.pareto (v06)
- reproduce f — pinneapple_tools.hpo_experiments.pipelines.reproduce (v07)
- ReproducerAgent C — pinneapple_tools.hpo_experiments.agents.reproducer (v07)
- ReproduceResult C — pinneapple_tools.hpo_experiments.agents.reproducer (v07)
- ReptileConfig C — pinneapple_adaptation.meta_learning.config (v04)

- ReptileTrainer C — pinneapple_adaptation.meta_learning.reptile (v04)
- require_admin f — pinneapple_app.backend.core.admin_auth (v07)
- require_allowed f — pinneapple_catalog.resources (v01)
- require_deepxde f — pinneapple_physics.deepxde_bridge.adapter (v02)
- require_physicsnemo_model f — pinneapple_neural.physicsnemo_bridge.adapters (v04)
- require_prepared f — pinneapple_analysis.benchmark.runner (v05)
- require_problem f — pinneapple_flow.nodes.problem (v07)
- require_pxr f — pinneapple_tools.nvidia.omniverse (v07)
- require_research_use f — pinneapple_neural._licencas (v04)
- required_speed_for_head f — pinneapple_systems.process_components.similarity_map (v06)
- Requirement C — pinneapple_reason.chain (v05)
- requirement f — pinneapple_flow.nodes.reason (v07)
- Requirements C — pinneapple_design.aero.aircraft (v06)
- Requirements3D C — pinneapple_design.aero.aircraft3d (v06)
- RequirementsAL C — pinneapple_design.aero.airliner3d (v06)
- RequirementSpec C — pinneapple_design.intent.spec (v06)
- RerunLogger C — pinneapple_tools.visualization.integrations.rerun_bridge (v07)
- ResearcherConfig C — pinneapple_tools.hpo_experiments.config (v07)
- ResearchLoopResult C — pinneapple_problemdesign.research_loop (v02)
- ResearchOnlyError C — pinneapple_neural._licencas (v04)
- ResearchOnlyWarning C — pinneapple_neural._licencas (v04)
- ResearchReport C — pinneapple_llm.research (v07)
- reservoir_mass_mg f — pinneapple_design.surrogate_inverse.transdermal_patch (v06)
- ReservoirCatalog C — pinneapple_neural.architectures.reservoir_computing.registry (v04)
- reset_nonuniform f — pinneapple_simulation.external_solvers.openfoam.campaign (v03)
- residual_baseline f — pinneapple_physics.ensemble (v02)
- residual_check f — pinneapple_physics.physics_audit.monitor (v02)
- residual_error_bound f — pinneapple_physics.darcy_checks (v02)
- residual_from_stencil f — pinneapple_tools.benchmark_suite.crossframework.problems (v07)
- residual_l2_bound f — pinneapple_physics.darcy_checks (v02)
- residual_metrics f — pinneapple_analysis.inverse_problems.calibration (v05)
- residual_plot f — pinneapple_tools.automation.adapters.openfoam (v07)
- residual_points f — pinneapple_tools.benchmark_suite.crossframework.problems (v07)
- residual_scale f — pinneapple_app.backend.core.surrogate_report (v07)
- residual_sigma f — pinneapple_systems.operations.rom (v06)
- ResidualAnalysisResult C — pinneapple_analysis.inverse_problems.missing_term (v05)
- ResidualAnalyzer C — pinneapple_analysis.inverse_problems.missing_term (v05)
- ResidualBasedAL C — pinneapple_data.active_learning (v04)
- ResidualBlock3D C — pinneapple_neural.architectures.convolutions.conv3d (v04)
- ResidualComponent C — pinneapple_security.cps (v05)
- ResidualCorrectionForecaster C — pinneapple_systems.time_series.forecast_lab.hybrid (v06)
- ResidualCUSUM C — pinneapple_security.anomaly (v05)
- ResidualDetector C — pinneapple_systems.digital_twin.monitoring.anomaly (v06)
- ResidualErrorCalibrator C — pinneapple_analysis.uncertainty.residual_bound (v05)
- ResidualLoss C — pinneapple_analysis.uncertainty.residual_bound (v05)
- ResidualLSTMConfig C — pinneapple_systems.time_series.decomposition.fft_lstm (v06)
- ResidualLSTMForecaster C — pinneapple_systems.time_series.decomposition.fft_lstm (v06)
- Residuals C — pinneapple_systems.digital_twin.pipe_loop.diagnose (v06)
- ResidualSpec C — pinneapple_neural.architectures.neural_operators.pino (v04)
- residue_class f — pinneapple_tools.visualization.molecular (v07)
- resign_version f — pinneapple_security.integrate (v05)
- resolution_study f — pinneapple_physics.blackhole.convergence (v02)
- ResolutionConsistency C — pinneapple_analysis.validation.metamorphic (v05)
- resolve f — pinneapple_decision.constraints (v05)
- resolve f — pinneapple_flow.libcalls (v07)
- resolve f — pinneapple_physics.astro.facts (v02)

- resolve f — pinneapple_simulation.external_solvers.openfoam.campaign (v03)
- resolve_function f — pinneapple_core.workflows.adapters (v01)
- resolve_physics f — pinneapple_simulation.pipelines.freecad_openfoam.pipeline (v03)
- resolve_problem f — pinneapple_llm.catalog_match (v07)
- resolve_rea_command f — pinneapple_tools.rea_bridge.client (v07)
- resolve_sets f — pinneapple_data.cae.inp (v04)
- ResolveResult C — pinneapple_llm.catalog_match (v07)
- resonant_frequency_hz f — pinneapple_design.surrogate_inverse.reflex_port (v06)
- Resource C — pinneapple_catalog.resources (v01)
- response f — pinneapple_design.thermal.board_cooling.simulate (v06)
- RestartResult C — pinneapple_neural.trainer.multistart (v04)
- resting_contact f — pinneapple_simulation.external_solvers.newton_physics.validation (v03)
- restrict f — pinneapple_physics.multiscale.filters (v02)
- Result C — pinneapple_presets.core (v02)
- Result C — pinneapple_science.core (v02)
- Result C — pinneapple_simulation.external_solvers.calculix.study (v03)
- results f — pinneapple_flow.nodes_ext.structures (v07)
- results_to_samples f — pinneapple_simulation.external_solvers.fluidx3d (v03)
- ResultTable C — pinneapple_simulation.electromagnetics.antenna.runner (v03)
- ResultTarget C — pinneapple_check.config (v05)
- retarget f — pinneapple_flow.templates (v07)
- RetrainConfig C — pinneapple_neural.physicsnemo_bridge.active_learning_bridge (v04)
- retreat_point f — pinneapple_simulation.rover.autolab.mission (v03)
- RetrievalIndex C — pinneapple_analysis.retrieval.index (v05)
- reverse_field f — pinneapple_simulation.electromagnetics.machines.thermal (v03)
- review_mesh_plan f — pinneapple_design.fitmesh.llm (v06)
- revisions f — pinneapple_hub.zoo._artifacts (v07)
- RevoluteJoint C — pinneapple_simulation.multibody.tree (v03)
- revolve f — pinneapple_design.geometry_library.kit (v06)
- revolve f — pinneapple_twin3d.primitives (v06)
- revolve_closed f — pinneapple_design.geometry.vehicle_cad (v06)
- reynolds f — pinneapple_physics.physics_audit.regime (v02)
- rga_matrix f — pinneapple_systems.time_series.eda.plots (v06)
- rgb_preview_stretch f — pinneapple_perception.change_detection (v04)
- rh_degasser f — pinneapple_twin3d.wear.presets (v06)
- rhs f — pinneapple_physics.blackhole.hydro_numba (v02)
- RIAFConfig C — pinneapple_physics.blackhole.hydro (v02)
- richardson f — pinneapple_physics.physics_audit.regime (v02)
- richardson_extrapolate f — pinneapple_analysis.verification.convergence (v05)
- RichardsResult C — pinneapple_simulation.geophysics.richards (v03)
- ricker f — pinneapple_simulation.electromagnetics.fdtd (v03)
- RidgeAR C — pinneapple_systems.time_series.adaptive (v06)
- rigid_body_derivative f — pinneapple_simulation.flight.rigid_body (v03)
- rigid_column_max_head f — pinneapple_presets.cases.water_hammer (v02)
- rigid_scene f — pinneapple_design.bricks.dynamics (v06)
- RigidBody C — pinneapple_simulation.particle_dynamics.rigid_body (v03)
- RigidBodyState C — pinneapple_simulation.particle_dynamics.rigid_body (v03)
- RigidBodySystem C — pinneapple_simulation.particle_dynamics.rigid_body (v03)
- RigidRock C — pinneapple_simulation.terramechanics.terrain (v03)
- RigidScene C — pinneapple_simulation.external_solvers.newton_physics.scene (v03)
- RigidTrajectory C — pinneapple_simulation.external_solvers.newton_physics.runner (v03)
- Ring C — pinneapple_simulation.electromagnetics.mesh (v03)
- ring_height f — pinneapple_tools.visualization.studio.planetary (v07)
- ring_mesh f — pinneapple_presets.cases.pressure_vessel (v02)
- ring_potential f — pinneapple_science.em.statics (v02)
- rise_transit_set f — pinneapple_physics.astro.observing (v02)

- rise_transit_set f — pinneapple_physics.astro.phenomena (v02)
- Risk C — pinneapple_problemdesign.schema (v02)
- RiskPlanner C — pinneapple_simulation.rover.autolab.planner (v03)
- ritter f — pinneapple_simulation.numerical_solvers.shallow_water_fv (v03)
- rk4 f — pinneapple_analysis.data_assimilation.models (v05)
- rk4_step f — pinneapple_simulation.flight.rigid_body (v03)
- rk4_step f — pinneapple_systems.time_series.synthetic (v06)
- rmae f — pinneapple_analysis.reproduction (v05)
- rms f — pinneapple_simulation.flight.simulate (v03)
- rmsd f — pinneapple_tools.visualization.molecular (v07)
- RMSE C — pinneapple_neural.trainer.metrics (v04)
- rmse f — pinneapple_analysis.reproduction (v05)
- rmse f — pinneapple_physics.metrics (v02)
- rmse f — pinneapple_physics.weather.evaluate (v02)
- rmse f — pinneapple_systems.time_series.metrics (v06)
- rmse f — pinneapple_systems.time_series.metrics_ext.point (v06)
- rmse_fn f — pinneapple_systems.time_series.metrics (v06)
- RNNOutput C — pinneapple_neural.architectures.recurrent.base (v04)
- ro_coefficients f — pinneapple_systems.time_series.enso.oscillators (v06)
- ro_eigen f — pinneapple_systems.time_series.enso.oscillators (v06)
- ro_rhs f — pinneapple_systems.time_series.enso.forecasters (v06)
- ro_tendency f — pinneapple_systems.time_series.enso.oscillators (v06)
- road f — pinneapple_design.geometry.vehicles3d (v06)
- RoadCar C — pinneapple_design.geometry.vehicles3d (v06)
- RobinBC f — pinneapple_physics.pde_environment.conditions (v02)
- RobotRun C — pinneapple_simulation.robot_sim.sim (v03)
- RobotSim C — pinneapple_simulation.robot_sim.sim (v03)
- Robustness C — pinneapple_design.intent.pipeline (v06)
- robustness f — pinneapple_design.intent.pipeline (v06)
- RobustScaleStep C — pinneapple_neural.trainer.preprocess (v04)
- roc_auc f — pinneapple_security.cps (v05)
- Rock C — pinneapple_simulation.rover.terrain (v03)
- rocket f — pinneapple_twin3d.gallery (v06)
- rocket_nozzle_cfd f — pinneapple_physics.pde_environment.presets.engineering (v02)
- rocket_structural f — pinneapple_physics.pde_environment.presets.engineering (v02)
- rod f — pinneapple_simulation.flight.mass (v03)
- RodModel C — pinneapple_analysis.inverse_problems.models (v05)
- rolling_fit f — pinneapple_systems.sports.football.ball (v06)
- rolling_statistics f — pinneapple_systems.time_series.eda.plots (v06)
- rolling_stats f — pinneapple_systems.time_series.features.engineering (v06)
- rolling_travel_time f — pinneapple_systems.sports.football.ball (v06)
- RollingWindowSplitter C — pinneapple_systems.time_series.validation.splitters (v06)
- rollout f — pinneapple_analysis.validation.rollout (v05)
- rollout f — pinneapple_physics.blackhole.forecast (v02)
- rollout_dense_volume_operator f — pinneapple_neural.workflows.dense_volume_operator (v04)
- rollout_parametric_dense_volume_operator f — pinneapple_neural.workflows.parametric_dense_volume_operator (v04)
- rollout_relative_l2 f — pinneapple_physics.metrics (v02)
- RolloutReport C — pinneapple_analysis.validation.rollout (v05)
- rom_checks f — pinneapple_neural.architectures.rom.contracts (v04)
- rom_compare f — pinneapple_flow.wrappers (v07)
- rom_fit f — pinneapple_flow.nodes.methods (v07)
- rom_study f — pinneapple_neural.architectures.rom.study (v04)
- ROMBase C — pinneapple_neural.architectures.rom.base (v04)
- ROMCatalog C — pinneapple_neural.architectures.rom.registry (v04)
- ROMHybrid C — pinneapple_neural.architectures.rom.rom_hybrid (v04)

- ROMOutput C — pinneapple_neural.architectures.rom.base (v04)
- ROMStudy C — pinneapple_lab.experiments.rom (v07)
- ROMStudyResult C — pinneapple_neural.architectures.rom.study (v04)
- roof f — pinneapple_reason.cases (v05)
- roof_network f — pinneapple_reason.modes.thermal (v05)
- root_compressibility f — pinneapple_condense.benchmark (v02)
- ROParams C — pinneapple_systems.time_series.enso.oscillators (v06)
- rosenstein_lyapunov f — pinneapple_systems.time_series.predictability (v06)
- rossby_haurwitz_speed f — pinneapple_simulation.geophysics.sphere_shallow_water (v03)
- rossby_wave_test f — pinneapple_systems.time_series.enso.intermediate (v06)
- rot f — pinneapple_design.geometry.vehicle_cad (v06)
- rot f — pinneapple_design.geometry_library.kit (v06)
- rot_x f — pinneapple_physics.astro.frames (v02)
- rot_x f — pinneapple_simulation.rover.geometry (v03)
- rot_y f — pinneapple_physics.astro.frames (v02)
- rot_y f — pinneapple_simulation.rover.geometry (v03)
- rot_z f — pinneapple_physics.astro.frames (v02)
- rot_z f — pinneapple_simulation.rover.geometry (v03)
- rotary_coupling_torsion_default f — pinneapple_physics.pde_environment.presets.structural (v02)
- rotate f — pinneapple_design.geometry.vehicle_cad (v06)
- rotate_field f — pinneapple_analysis.validation.metamorphic (v05)
- rotating_bending_stress_cycle f — pinneapple_systems.process_components.pipe_stress_mechanics (v06)
- RotatingBendingCycleResult C — pinneapple_systems.process_components.pipe_stress_mechanics (v06)
- rotation f — pinneapple_simulation.electromagnetics.media (v03)
- rotation f — pinneapple_twin3d.primitives (v06)
- rotations f — pinneapple_decision.debias (v05)
- rotor_design f — pinneapple_flow.nodes_ext.em_prius (v07)
- rotor_design_problem f — pinneapple_simulation.electromagnetics.machines.design (v03)
- rotor_evaluator f — pinneapple_simulation.electromagnetics.machines.design (v03)
- rotor_parameter_space f — pinneapple_simulation.electromagnetics.machines.interop (v03)
- rotor_temperature_c f — pinneapple_design.surrogate_inverse.brake_rotor (v06)
- RotorDesign C — pinneapple_design.surrogate_inverse.brake_rotor (v06)
- RotorTorqueProblem C — pinneapple_design.topology.magnetic (v06)
- roughness_reynolds f — pinneapple_physics.physics_audit.regime (v02)
- round_conductor_eddy_loss_low_freq f — pinneapple_simulation.electromagnetics.analytic (v03)
- round_section f — pinneapple_design.geometry_library.kit (v06)
- round_sig f — pinneapple_core.jsonable (v01)
- RoundBottomSlots C — pinneapple_simulation.electromagnetics.machines.geometry (v03)
- rounded_box f — pinneapple_design.geometry_library.kit (v06)
- rounding_half_unit f — pinneapple_flow.nodes_ext.examples (v07)
- RoundReport C — pinneapple_analysis.validation.arena.loop (v05)
- Route C — pinneapple_simulation.numerical_solvers.internal_flow (v03)
- Route C — pinneapple_simulation.rover.autolab.planner (v03)
- route_a_sqrt_coupling f — pinneapple_physics.non_hermitian.ep_tuning (v02)
- route_c_asymmetric f — pinneapple_physics.non_hermitian.ep_tuning (v02)
- route_cases f — pinneapple_analysis.uncertainty.applicability (v05)
- route_lines f — pinneapple_simulation.rover.autolab.render (v03)
- RouteDecision C — pinneapple_physics.physics_audit.routing (v02)
- routing_summary f — pinneapple_analysis.uncertainty.applicability (v05)
- rover_meshes f — pinneapple_simulation.rover.geometry (v03)
- rover_wheel f — pinneapple_design.geometry.vehicle_cad (v06)
- RoverContact C — pinneapple_hub.zoo._models (v07)
- RoverMission C — pinneapple_simulation.rover.autolab.mission (v03)
- RoverParams C — pinneapple_simulation.rover.geometry (v03)
- RoverSimulator C — pinneapple_simulation.rover.dynamics (v03)

- Row C — pinneapple_tools.visualization.trust_panels (v07)
- row_status f — pinneapple_analysis.benchmark.checks (v05)
- rows_to_xy f — pinneapple_design.thermal.heatsink.surrogate (v06)
- rtn_basis f — pinneapple_physics.astro.neural_propagator (v02)
- Rule C — pinneapple_check.rules (v05)
- Rule C — pinneapple_flow.rules (v07)
- rule f — pinneapple_flow.rules (v07)
- rule_decision f — pinneapple_flow.decisions (v07)
- rule_eval f — pinneapple_simulation.pipelines.freecad_openfoam.cad (v03)
- RuleBasedBackend C — pinneapple_decision.backends (v05)
- RuleBasedBaseline C — pinneapple_systems.operations.control (v06)
- RuleParser C — pinneapple_simulation.rover.autolab.mission (v03)
- RULEstimate C — pinneapple_systems.digital_twin.prognostics (v06)
- Run C — pinneapple_autolab.core (v07)
- Run C — pinneapple_design.intent.physics.loop_track (v06)
- run f — pinneapple_analysis.benchmark.api (v05)
- run f — pinneapple_check.engine (v05)
- run f — pinneapple_condense.exceptions (v02)
- run f — pinneapple_design.design_optimizer.methods (v06)
- run f — pinneapple_flow (v07)
- run f — pinneapple_flow.nodes.cfd_pipeline (v07)
- run f — pinneapple_flow.runtime (v07)
- run f — pinneapple_lab.runner (v07)
- run f — pinneapple_llm.qa.validators.base (v07)
- run f — pinneapple_science.registry (v02)
- run f — pinneapple_simulation.electromagnetics.machines.verification (v03)
- run f — pinneapple_simulation.electromagnetics.reflector.study_report (v03)
- run f — pinneapple_simulation.electromagnetics.verification (v03)
- run f — pinneapple_simulation.flight.verification (v03)
- run_adaptive_sweep f — pinneapple_neural.trainer.adaptive_sweep (v04)
- run_adversarial_review f — pinneapple_analysis.verification.architecture_critique (v05)
- run_agent f — pinneapple_autolab.agent (v07)
- run_agent_loop f — pinneapple_llm.agent_loop (v07)
- run_all f — pinneapple_physics.astro.validation (v02)
- run_all f — pinneapple_physics.experiment (v02)
- run_arena f — pinneapple_analysis.validation.arena.loop (v05)
- run_arena_experiment f — pinneapple_tools.benchmark_suite.runner.run_arena_yaml (v07)
- run_benchmark f — pinneapple_arena.external.benchmark (v04)
- run_benchmark f — pinneapple_arena.showcase (v04)
- run_benchmark f — pinneapple_hub.zoo._bench (v07)
- run_benchmark f — pinneapple_security.cps_bench (v05)
- run_benchmark f — pinneapple_systems.digital_twin.pipe_loop.benchmark (v06)
- run_benchmark f — pinneapple_systems.operations.anomaly (v06)
- run_benchmark f — pinneapple_tools (v07)
- run_benchmark f — pinneapple_tools.benchmark_suite.runner.run_benchmark (v07)
- run_case f — pinneapple_design.aero.case (v06)
- run_case f — pinneapple_reason (v05)
- run_case f — pinneapple_simulation.accelerated_foam.case (v03)
- run_case f — pinneapple_simulation.numerical_solvers.external_flow (v03)
- run_case f — pinneapple_systems.time_series.forecast_lab.pipeline (v06)
- run_case3d f — pinneapple_design.aero.case3d (v06)
- run_ccx f — pinneapple_simulation.external_solvers.calculix.runner (v03)
- run_closed_loop f — pinneapple_systems.component_modeling.control (v06)
- run_closed_loop f — pinneapple_systems.operations.control (v06)
- run_command f — pinneapple_simulation.accelerated_foam.case (v03)
- run_compare f — pinneapple_arena.external.benchmark (v04)

- run_custom_solver f — pinneapple_tools.sandbox.custom_solver_runner (v07)
- run_data_pipeline f — pinneapple_app.backend.core.data_pipeline (v07)
- run_decision_mode f — pinneapple_arena.decision_mode (v04)
- run_design_optimization f — pinneapple_design.design_optimizer.airfoil_drag (v06)
- run_dir f — pinneapple_lab.seal (v07)
- run_ensembles f — pinneapple_physics.multiscale.comparison (v02)
- run_eval f — pinneapple_llm.workbench.evaluate (v07)
- run_example f — pinneapple_flow.examples (v07)
- run_example f — pinneapple_llm.qa.examples.pipe_wear_twin (v07)
- run_experiment f — pinneapple_flow.nodes.science (v07)
- run_fault_injection f — pinneapple_analysis.validation.fault_injection.matrix (v05)
- run_fdtd f — pinneapple_science.em.fdtd_study (v02)
- run_file f — pinneapple_hub.library (v07)
- run_fin f — pinneapple_analysis.inverse_problems.heat_transfer (v05)
- run_finetune f — pinneapple_arena.external.finetune (v04)
- run_fluent_case f — pinneapple_simulation.external_solvers.ansys.runner (v03)
- run_fluidx3d f — pinneapple_simulation.external_solvers.fluidx3d (v03)
- run_free_geometry f — pinneapple_design.fitmesh.llm (v06)
- run_full_pipeline f — pinneapple_tools.benchmark_suite.runner.run_pipeline (v07)
- run_geotaichi f — pinneapple_simulation.external_solvers.geotaichi (v03)
- run_in_freecad_gui f — pinneapple_tools.automation.adapters.freecad (v07)
- run_info f — pinneapple_simulation.rover.autolab.report (v03)
- run_intervention f — pinneapple_analysis.verification.causal_discrepancy (v05)
- run_inverse f — pinneapple_arena.trainer (v04)
- run_inverse_discovery f — pinneapple_analysis.inverse_problems.heat_diffusivity (v05)
- run_inverse_solver f — pinneapple_arena.pointwise.training (v04)
- run_job f — pinneapple_analysis.benchmark.runner (v05)
- run_job f — pinneapple_design.aero.dataset (v06)
- run_job f — pinneapple_tools.automation.scripts.freecad_gui_job (v07)
- run_live f — pinneapple_systems.digital_twin.product.runner (v06)
- run_manufactured_solution_check f — pinneapple_problemdesign.preset_authoring (v02)
- run_matlab_function f — pinneapple_simulation.external_solvers.matlab.runner (v03)
- run_matlab_script f — pinneapple_simulation.external_solvers.matlab.runner (v03)
- run_mission f — pinneapple_design.thermal.board_cooling.agent (v06)
- run_mission f — pinneapple_systems.digital_twin.pipe_loop.agent (v06)
- run_model f — pinneapple_systems.time_series.enso.study (v06)
- run_mpc f — pinneapple_systems.component_modeling.mpc (v06)
- run_omc_script f — pinneapple_simulation.external_solvers.modelica.omc (v03)
- run_one f — pinneapple_tools.benchmark_suite.runner.run_models_from_cfg (v07)
- run_onnx f — pinneapple_hub.zoo.export (v07)
- run_openfoam f — pinneapple_simulation.numerical_solvers.openfoam_bridge (v03)
- run_openfoam_case f — pinneapple_simulation.external_solvers.openfoam.runner (v03)
- run_openradioss_case f — pinneapple_simulation.external_solvers.openradioss.runner (v03)
- run_openradioss_cases f — pinneapple_simulation.external_solvers.openradioss.runner (v03)
- run_overfitting f — pinneapple_systems.time_series.forecast_lab.pipeline (v06)
- run_parallel_sweep f — pinneapple_neural.trainer.parallel (v04)
- run_pipeline f — pinneapple_simulation.pipelines.freecad_openfoam.pipeline (v03)
- run_pipeline f — pinneapple_systems.time_series.solar_cycle (v06)
- run_pipeline f — pinneapple_worldmodel.train (v04)
- run_plant f — pinneapple_systems.digital_twin.pipe_loop.plant (v06)
- run_plate f — pinneapple_analysis.inverse_problems.heat_transfer (v05)
- run_preset f — pinneapple_flow.nodes.presets (v07)
- run_regime_arena f — pinneapple_arena.regimes (v04)
- run_sampling f — pinneapple_simulation.external_solvers.openfoam.sampling (v03)
- run_series_flow f — pinneapple_systems.operations.series (v06)
- run_snippet f — pinneapple_hub.library.sandbox (v07)

- run_statistics f — pinneapple_physics.blackhole.convergence (v02)
- run_study f — pinneapple_analysis.surrogate.study (v05)
- run_sweep f — pinneapple_design.thermal.fin_array_speed (v06)
- run_sweep f — pinneapple_tools.benchmark_suite.runner.run_sweep (v07)
- run_testbed f — pinneapple_simulation.terramechanics.testbed (v03)
- run_track_b f — pinneapple_tools.benchmark_suite.crossframework.darcy (v07)
- run_tree f — pinneapple_decision.api (v05)
- run_tree f — pinneapple_decision.tree (v05)
- run_trial f — pinneapple_arena.autoresearch (v04)
- run_turbodesigner f — pinneapple_simulation.external_solvers.turbodesigner.bridge (v03)
- run_twin f — pinneapple_flow.twin (v07)
- run_twin f — pinneapple_systems.digital_twin.recession (v06)
- run_uq f — pinneapple_arena.trainer (v04)
- run_validation f — pinneapple_veriphysics.validation_lab (v05)
- run_zoo f — pinneapple_systems.time_series.forecast_lab.study (v06)
- RunContext C — pinneapple_lab.context (v07)
- RunInfo C — pinneapple_tools.nvidia.runinfo (v07)
- RunLogger C — pinneapple_neural.trainer.audit (v04)
- runnable f — pinneapple_core.onboarding.catalog (v01)
- runnable_actions f — pinneapple_lab.improve (v07)
- Runner C — pinneapple_tools.automation.runner (v07)
- runner_channel f — pinneapple_twin3d.gallery (v06)
- runner_parser f — pinneapple_tools.benchmark_suite.crossframework.protocol (v07)
- running f — pinneapple_mcp.jobs (v07)
- runout f — pinneapple_simulation.particle_dynamics.geo_mpm (v03)
- RunRecord C — pinneapple_autolab.ledger (v07)
- RunRecord C — pinneapple_simulation.electromagnetics.antenna.runner (v03)
- RunRecord C — pinneapple_tools.automation.record (v07)
- RunResult C — pinneapple_lab.runner (v07)
- RunResult C — pinneapple_simulation.accelerated_foam.case (v03)
- RunResult C — pinneapple_systems.digital_twin.product.runner (v06)
- RunResult C — pinneapple_tools.hpo_experiments.utils.verify_runtime (v07)
- runs_for_node f — pinneapple_hub.library.flowruns (v07)
- runtime_dir f — pinneapple_hub.zoo.web (v07)
- runway_surfaces f — pinneapple_simulation.flight.visuals (v03)

S

- safe_data_path f — pinneapple_flow.config (v07)
- safe_eval f — pinneapple_systems.process_components.explicit_equation_system (v06)
- safe_extract_zip f — pinneapple_security.archives (v05)
- safe_load f — pinneapple_security.model_security (v05)
- safe_name f — pinneapple_core.service.uploads (v01)
- safe_parse_iso8601 f — pinneapple_tools.hpo_experiments.ranking (v07)
- saltelli_design f — pinneapple_analysis.sensitivity.sobol (v05)
- saltelli_perturbation_sweep f — pinneapple_data.parameter_sampling (v04)
- same_dimension f — pinneapple_design.intent.spec (v06)
- same_gust f — pinneapple_flow.nodes_ext.flight (v07)
- same_origin_ok f — pinneapple_security.web (v05)
- sample f — pinneapple_design.fitmesh.suggest (v06)
- sample_annulus_tag_batch f — pinneapple_design.geometry.builders.analytic_domain_batch_builder (v06)
- sample_axisymmetric_wall_tag_batch f — pinneapple_design.geometry.builders.analytic_domain_batch_builder (v06)
- sample_barycentric_uv f — pinneapple_design.geometry.sample.barycentric (v06)
- sample_boundary_points f — pinneapple_design.geometry.sample.marching_squares (v06)
- sample_boundary_points_sdf2d f — pinneapple_design.geometry.io.sdf_meshing (v06)

- sample_box_particles f — pinneapple_design.geometry.sample.particles (v06)
- sample_box_tag_batch f — pinneapple_design.geometry.builders.analytic_domain_batch_builder (v06)
- sample_box_with_curve_tag_batch f — pinneapple_design.geometry.builders.analytic_domain_batch_builder (v06)
- sample_camera f — pinneapple_worldmodel.robot_perception (v04)
- sample_checks f — pinneapple_physics.darcy_checks (v02)
- sample_checks f — pinneapple_physics.maxwell_checks (v02)
- sample_circle_particles f — pinneapple_design.geometry.sample.particles (v06)
- sample_condition_points f — pinneapple_physics.pde_environment.condition_sampling (v02)
- sample_cylinder_tag_batch f — pinneapple_design.geometry.builders.analytic_domain_batch_builder (v06)
- sample_designs f — pinneapple_design.geometry.gen.car2d (v06)
- sample_designs f — pinneapple_tools.nvidia.heatsink (v07)
- sample_domain f — pinneapple_design (v06)
- sample_domain_1d f — pinneapple_quantum.data.quantum_dataset (v04)
- sample_domain_2d f — pinneapple_quantum.data.quantum_dataset (v04)
- sample_entropy f — pinneapple_systems.time_series.predictability (v06)
- sample_faces f — pinneapple_app.backend.core.physics_adapter (v07)
- sample_grf_2d_batch f — pinneapple_data.grf_sampler (v04)
- sample_grf_batch f — pinneapple_data.grf_sampler (v04)
- sample_grf_paths f — pinneapple_data.grf_sampler (v04)
- sample_initial f — pinneapple_app.backend.core.physics_adapter (v07)
- sample_interior f — pinneapple_app.backend.core.physics_adapter (v07)
- sample_latent f — pinneapple_physics.pinn_solver.stochastic (v02)
- sample_latin_hypercube_box f — pinneapple_design.geometry.sample.grids (v06)
- sample_lines f — pinneapple_simulation.numerical_solvers.free_surface (v03)
- sample_parameters f — pinneapple_data.parameter_sampling (v04)
- sample_physics_points f — pinneapple_design.geometry.mesh.mesher_3d (v06)
- sample_point_cloud f — pinneapple_data.geometry_features (v04)
- sample_pointcloud_from_mesh f — pinneapple_design.geometry.ops.pointcloud (v06)
- sample_points_on_triangles f — pinneapple_design.geometry.sample.barycentric (v06)
- sample_population f — pinneapple_physics.astro.neural_propagator (v02)
- sample_profile f — pinneapple_twin3d.wear.geometry (v06)
- sample_space f — pinneapple_analysis.uncertainty.distributions (v05)
- sample_surface_points f — pinneapple_design.geometry.sample.points (v06)
- sample_surface_points_weighted f — pinneapple_design.geometry.sample.points (v06)
- sample_uniform_box f — pinneapple_design.geometry.sample.grids (v06)
- sample_variation f — pinneapple_worldmodel.inspection_data (v04)
- sample_verdict f — pinneapple_physics.darcy_checks (v02)
- sample_verdict f — pinneapple_physics.maxwell_checks (v02)
- sample_voxelgrid f — pinneapple_design.geometry.ops.voxelize (v06)
- SampleLine C — pinneapple_simulation.numerical_solvers.free_surface (v03)
- Sampler C — pinneapple_core.data (v01)
- SamplingSpec C — pinneapple_data.upd_types (v04)
- Sand C — pinneapple_simulation.particle_dynamics.geo_mpm (v03)
- SandboxExecutor C — pinneapple_llm.qa.sandbox (v07)
- SandboxLimits C — pinneapple_llm.qa.sandbox (v07)
- SandboxResult C — pinneapple_llm.qa.sandbox (v07)
- sanitize f — pinneapple_tools.reporting.paperkit (v07)
- sanitize_log f — pinneapple_security.privacy (v05)
- sarif f — pinneapple_check.report (v05)
- Satellite C — pinneapple_physics.astro.satellites (v02)
- satellite f — pinneapple_twin3d.gallery (v06)
- satellite_j2_perturbation f — pinneapple_physics.pde_environment.presets.astrophysics (v02)
- saturation_pressure f — pinneapple_reason.props (v05)
- save f — pinneapple_flow.learn (v07)

- save f — pinneapple_lab.manifest (v07)
- save_cases f — pinneapple_analysis.validation.arena.cases (v05)
- save_checkpoint f — pinneapple_neural.trainer.checkpoint (v04)
- save_darcy_npz f — pinneapple_physics.darcy_checks (v02)
- save_forecaster f — pinneapple_physics.blackhole.forecast (v02)
- save_hdf5 f — pinneapple_data.serialization (v04)
- save_hdf5 f — pinneapple_data.serialization (v04)
- save_manifest f — pinneapple_data.serialization (v04)
- save_mat f — pinneapple_simulation.external_solvers.matlab.mat_io (v03)
- save_maxwell_npz f — pinneapple_physics.maxwell_checks (v02)
- save_mesh_cache f — pinneapple_data.adapters.splash_archive_adapter (v04)
- save_meshio f — pinneapple_design.geometry.io.meshio_bridge (v06)
- save_model f — pinneapple_systems.operations.contracts (v06)
- save_object f — pinneapple_flow.nodes_ext.examples (v07)
- save_point_set f — pinneapple_tools.benchmark_suite.crossframework.problems (v07)
- save_pt f — pinneapple_data.serialization (v04)
- save_review f — pinneapple_lab.curation (v07)
- save_seal f — pinneapple_lab.seal (v07)
- save_solution f — pinneapple_flow.nodes._common (v07)
- save_zarr f — pinneapple_data.serialization (v04)
- savefig f — pinneapple_tools.reporting.paperkit (v07)
- savgol_derivatives f — pinneapple_systems.sports.football.kinematics (v06)
- sbom f — pinneapple_security.attestation (v05)
- scaffold f — pinneapple_lab.manifest (v07)
- scalar f — pinneapple_check.fields (v05)
- scalar f — pinneapple_data.simulation_metadata.foam_dict (v04)
- scalar_coords f — pinneapple_condense.flow (v02)
- scalar_of f — pinneapple_tools.visualization.studio.scene (v07)
- ScalarIndex C — pinneapple_condense.flow (v02)
- ScalarSpec C — pinneapple_simulation.pipelines.freecad_openfoam.case (v03)
- Scale C — pinneapple_core.transforms (v01)
- scale_compatibility f — pinneapple_physics.multiscale.scales (v02)
- scale_conditioning_study f — pinneapple_physics.multiscale.turbulence (v02)
- scale_to_context f — pinneapple_design.fitmesh.suggest (v06)
- ScaleAwareRouter C — pinneapple_physics.multiscale.scales (v02)
- ScaleCheck C — pinneapple_physics.multiscale.scales (v02)
- ScaleConditionedFNO2d C — pinneapple_physics.multiscale.learned (v02)
- scaled f — pinneapple_simulation.electromagnetics.reflector.compact_range (v03)
- scaled_field_head f — pinneapple_analysis.verification.nozzle_crosscheck (v05)
- ScaledCurve C — pinneapple_simulation.electromagnetics.machine (v03)
- ScaleProfile C — pinneapple_physics.multiscale.scales (v02)
- ScaleQuery C — pinneapple_physics.multiscale.scales (v02)
- ScaleRoute C — pinneapple_physics.multiscale.scales (v02)
- ScaleSpec C — pinneapple_physics.pde_environment.scales (v02)
- scaling f — pinneapple_condense.figures (v02)
- scaling_point f — pinneapple_condense.benchmark (v02)
- scaling_study f — pinneapple_analysis.cost.profiler (v05)
- ScalingFit C — pinneapple_analysis.cost.profiler (v05)
- ScalingModel C — pinneapple_design.qualitative.models (v06)
- ScalingStudy C — pinneapple_analysis.cost.profiler (v05)
- scan f — pinneapple_check.models (v05)
- scan f — pinneapple_flow.autocalls (v07)
- scan f — pinneapple_flow.examples (v07)
- scan f — pinneapple_lab.manifest (v07)
- scan_checkpoint f — pinneapple_security.model_security (v05)
- scan_path f — pinneapple_security.secrets_scan (v05)

- scan_pickle_bytes f — pinneapple_security.model_security (v05)
- scan_text f — pinneapple_security.secrets_scan (v05)
- ScanReport C — pinneapple_security.model_security (v05)
- ScanReport C — pinneapple_twin3d.scan (v06)
- scatter_add f — pinneapple_neural.architectures.graphnn.utils (v04)
- scatter_add_flat f — pinneapple_neural.architectures.graphnn.utils (v04)
- Scenario C — pinneapple_systems.digital_twin.pipe_loop.plant (v06)
- Scenario C — pinneapple_systems.operations.plant (v06)
- ScenarioGenerator C — pinneapple_worldmodel.scenario_generator (v04)
- ScenarioSpec C — pinneapple_worldmodel.scenario_generator (v04)
- Scene C — pinneapple_tools.visualization.studio.scene (v07)
- Scene C — pinneapple_twin3d.scene (v06)
- scene f — pinneapple_flow.nodes_ext.flight (v07)
- scene f — pinneapple_physics.astro.journey (v02)
- scene3d_payload f — pinneapple_tools.visualization.dashboard.html (v07)
- scene_from_case f — pinneapple_twin3d.openfoam (v06)
- scene_from_mjcf f — pinneapple_twin3d.scan (v06)
- scene_from_wear f — pinneapple_twin3d.wear.build (v06)
- scene_html f — pinneapple_tools.visualization.integrations.plotly_export (v07)
- scene_summary f — pinneapple_simulation.flight.visuals (v03)
- scene_to_multiblock f — pinneapple_tools.visualization.pyvista_bridge (v07)
- scenery f — pinneapple_design.bricks.visuals (v06)
- schema_templates f — pinneapple_pdb.templates (v04)
- SchemaAdapter C — pinneapple_physics.pinn_solver.schemas.schema_adapter (v02)
- SchemaAdapterOptions C — pinneapple_physics.pinn_solver.schemas.schema_adapter (v02)
- schiller_naumann_cd f — pinneapple_physics.closed_form.particle_drag (v02)
- schwarzschild_deflection_angle_weak_field f — pinneapple_physics.pde_environment.presets.astrophysics (v02)
- schwarzschild_light_bending_u_exact f — pinneapple_physics.pde_environment.presets.astrophysics (v02)
- schwarzschild_light_bending_weak_field f — pinneapple_physics.pde_environment.presets.astrophysics (v02)
- SConv C — pinneapple_physics.weather.model (v02)
- scope_from_batch f — pinneapple_veriphysics.scope (v05)
- scope_from_verdicts f — pinneapple_veriphysics.scope (v05)
- ScopeOfUse C — pinneapple_veriphysics.scope (v05)
- score f — pinneapple_analysis.validation.fault_injection.checks (v05)
- score f — pinneapple_flow.decisions (v07)
- score f — pinneapple_flow.palette (v07)
- score f — pinneapple_flow.planner.extensions (v07)
- score f — pinneapple_hub.library (v07)
- score f — pinneapple_systems.sports.football.metrics (v06)
- score_against_reference f — pinneapple_analysis.verification.physics_confidence_score (v05)
- score_events f — pinneapple_systems.operations.anomaly (v06)
- score_forecasts f — pinneapple_physics.weather.evaluate (v02)
- score_matrix f — pinneapple_analysis.validation.arena.blue (v05)
- ScoringBackend C — pinneapple_decision.backends (v05)
- screen_cw_conjunction f — pinneapple_physics.astro.conjunction (v02)
- screen_nozzle f — pinneapple_analysis.verification.nozzle_crosscheck (v05)
- screenshot f — pinneapple_tools.visualization.studio.web (v07)
- ScriptedBackend C — pinneapple_llm.workbench.backend (v07)
- SDEDriftForecaster C — pinneapple_systems.time_series.forecast_lab.physical (v06)
- SDF C — pinneapple_design.geometry.gen.sdf_shapes (v06)
- sdf2d_annulus f — pinneapple_design.geometry.gen.sdf_shapes (v06)
- sdf2d_capsule f — pinneapple_design.geometry.gen.sdf_shapes (v06)
- sdf2d_circle f — pinneapple_design.geometry.gen.sdf_shapes (v06)
- sdf2d_convex_polygon f — pinneapple_design.geometry.gen.sdf_shapes (v06)

- sdf2d_ellipse f — pinneapple_design.geometry.gen.sdf_shapes (v06)
- sdf2d_line_segment f — pinneapple_design.geometry.gen.sdf_shapes (v06)
- sdf2d_rectangle f — pinneapple_design.geometry.gen.sdf_shapes (v06)
- sdf2d_rounded_rectangle f — pinneapple_design.geometry.gen.sdf_shapes (v06)
- sdf2d_sector f — pinneapple_design.geometry.gen.sdf_shapes (v06)
- sdf2d_to_pointcloud f — pinneapple_design.geometry.ops.pointcloud (v06)
- sdf2d_to_tri_mesh f — pinneapple_design.geometry.io.sdf_meshing (v06)
- sdf2d_triangle f — pinneapple_design.geometry.gen.sdf_shapes (v06)
- sdf3d_box f — pinneapple_design.geometry.gen.sdf_shapes (v06)
- sdf3d_capsule f — pinneapple_design.geometry.gen.sdf_shapes (v06)
- sdf3d_cone f — pinneapple_design.geometry.gen.sdf_shapes (v06)
- sdf3d_cylinder f — pinneapple_design.geometry.gen.sdf_shapes (v06)
- sdf3d_rounded_box f — pinneapple_design.geometry.gen.sdf_shapes (v06)
- sdf3d_sphere f — pinneapple_design.geometry.gen.sdf_shapes (v06)
- sdf3d_torus f — pinneapple_design.geometry.gen.sdf_shapes (v06)
- sdf_difference f — pinneapple_design.geometry.gen.sdf_shapes (v06)
- sdf_elongate_2d f — pinneapple_design.geometry.gen.sdf_shapes (v06)
- sdf_intersection f — pinneapple_design.geometry.gen.genetic_sdf (v06)
- sdf_intersection f — pinneapple_design.geometry.gen.sdf_shapes (v06)
- sdf_mirror_x f — pinneapple_design.geometry.gen.sdf_shapes (v06)
- sdf_mirror_y f — pinneapple_design.geometry.gen.sdf_shapes (v06)
- sdf_onion f — pinneapple_design.geometry.gen.sdf_shapes (v06)
- sdf_repeat_2d f — pinneapple_design.geometry.gen.sdf_shapes (v06)
- sdf_rotate_2d f — pinneapple_design.geometry.gen.sdf_shapes (v06)
- sdf_scale f — pinneapple_design.geometry.gen.sdf_shapes (v06)
- sdf_smooth_difference f — pinneapple_design.geometry.gen.sdf_shapes (v06)
- sdf_smooth_intersection f — pinneapple_design.geometry.gen.sdf_shapes (v06)
- sdf_smooth_union f — pinneapple_design.geometry.gen.sdf_shapes (v06)
- sdf_translate f — pinneapple_design.geometry.gen.sdf_shapes (v06)
- sdf_union f — pinneapple_design.geometry.gen.genetic_sdf (v06)
- sdf_union f — pinneapple_design.geometry.gen.sdf_shapes (v06)
- SDFConfig C — pinneapple_design.geometry.builders.stl_domain_batch_builder (v06)
- SDFDomain2D C — pinneapple_design.geometry.gen.domains (v06)
- SDFGrid2D C — pinneapple_design.geometry.io.sdf_meshing (v06)
- SDFShape C — pinneapple_design.geometry.csg (v06)
- Seal C — pinneapple_lab.seal (v07)
- seal_for f — pinneapple_lab.seal (v07)
- seal_png f — pinneapple_lab.explorer (v07)
- seal_status f — pinneapple_flow.lab_status (v07)
- sealed_savefig f — pinneapple_lab.seal (v07)
- SealError C — pinneapple_lab.seal (v07)
- search f — pinneapple_core.onboarding.catalog (v01)
- search f — pinneapple_flow.nodes_ext.bricks (v07)
- search f — pinneapple_flow.palette (v07)
- search f — pinneapple_hub.library (v07)
- search f — pinneapple_physics.astro.facts (v02)
- search_architecture f — pinneapple_arena.nas (v04)
- search_gates_bo f — pinneapple_simulation.numerical_solvers.gated_xtfc (v03)
- search_gates_nested f — pinneapple_simulation.numerical_solvers.gated_xtfc (v03)
- search_literature f — pinneapple_llm.research (v07)
- search_space f — pinneapple_flow.learn (v07)
- SearchResult C — pinneapple_analysis.cost.equal_budget (v05)
- SearchResult C — pinneapple_analysis.retrieval.index (v05)
- SearchResult C — pinneapple_analysis.validation.arena.search (v05)
- SearchResult C — pinneapple_design.bricks.search (v06)
- SearchTrial C — pinneapple_analysis.cost.equal_budget (v05)

- SeasonalNaive C — pinneapple_systems.time_series.forecast_lab.baselines (v06)
- SeasonalNaiveForecaster C — pinneapple_systems.time_series.baselines.naive (v06)
- seasons f — pinneapple_physics.astro.phenomena (v02)
- second_derivative f — pinneapple_simulation.numerical_solvers.beam_bvp_fdm (v03)
- seconds_since f — pinneapple_data.upd_types (v04)
- SecretFinding C — pinneapple_security.secrets_scan (v05)
- Section C — pinneapple_analysis.validation.physics_learning_cases (v05)
- Section C — pinneapple_simulation.pipelines.freecad_openfoam.post (v03)
- Section C — pinneapple_tools.reporting.datasheet (v07)
- section_aero f — pinneapple_design.design_optimizer.airfoil_drag (v06)
- section_average f — pinneapple_simulation.pipelines.freecad_openfoam.post (v03)
- section_class_ok f — pinneapple_presets.cases.footbridge (v02)
- section_of f — pinneapple_flow.palette (v07)
- section_profile f — pinneapple_design.qualitative.descriptors (v06)
- section_ring f — pinneapple_design.geometry_library.kit (v06)
- section_stress f — pinneapple_design.intent.physics.bracket (v06)
- SectionProps C — pinneapple_simulation.numerical_solvers.beam_bvp_fdm (v03)
- sections f — pinneapple_flow.palette (v07)
- SectorGeometry C — pinneapple_simulation.electromagnetics.machines.geometry (v03)
- SectorMesh C — pinneapple_simulation.electromagnetics.machines.mesher (v03)
- secure_save f — pinneapple_security.integrate (v05)
- SecurityHeaders C — pinneapple_security.web (v05)
- seed_spread f — pinneapple_analysis.benchmark.checks (v05)
- seed_stats f — pinneapple_analysis.reproduction (v05)
- seed_stats f — pinneapple_tools.benchmark_suite.crossframework.evaluate (v07)
- SeedStats C — pinneapple_analysis.reproduction (v05)
- Segment C — pinneapple_physics.astro.spk (v02)
- select f — pinneapple_lab.reports (v07)
- select f — pinneapple_simulation.numerical_solvers._bc (v03)
- select_adapter f — pinneapple_neural.architectures.adapters.base (v04)
- select_candidates f — pinneapple_design.design_optimizer.verify (v06)
- select_models f — pinneapple_tools.benchmark_suite.pipeline.model_selector (v07)
- select_params f — pinneapple_systems.sports.football.xg (v06)
- select_points f — pinneapple_analysis.validation.physics_learning (v05)
- select_rules f — pinneapple_check.rules (v05)
- select_slugs f — pinneapple_arena.external.benchmark (v04)
- select_solver_family f — pinneapple_analysis.verification.solver_orchestration (v05)
- select_template f — pinneapple_design.intent.parse (v06)
- self_check f — pinneapple_reason.library (v05)
- self_intersections f — pinneapple_design.geometry_library.meshops (v06)
- self_scaled_parameters f — pinneapple_neural.trainer.self_scaled_qn (v04)
- SelfAdaptiveWeights C — pinneapple_neural.trainer.weight_scheduler (v04)
- SelfScaledQuasiNewton C — pinneapple_neural.trainer.self_scaled_qn (v04)
- sensitivity f — pinneapple_design.bricks.report (v06)
- sensitivity f — pinneapple_flow.nodes.surrogates (v07)
- sensitivity_result f — pinneapple_analysis.study_contracts (v05)
- sensitivity_study f — pinneapple_analysis.sensitivity.study (v05)
- sensitivity_to_dict f — pinneapple_design.surrogate_inverse.suspension_arm (v06)
- SensitivityResult C — pinneapple_design.surrogate_inverse.suspension_arm (v06)
- Sensor C — pinneapple_simulation.numerical_solvers.shallow_water_fv (v03)
- Sensor C — pinneapple_systems.digital_twin.io.sensors (v06)
- Sensor C — pinneapple_twin3d.scene (v06)
- sensor_count f — pinneapple_flow.planner.heuristic (v07)
- sensor_layout f — pinneapple_flow.planner.heuristic (v07)
- SensorNoise C — pinneapple_systems.operations.plant (v06)
- SensorPlacement C — pinneapple_analysis.data_assimilation.sensor_placement (v05)

- SensorReading C — pinneapple_llm.qa.validators.base (v07)
- SensorRegistry C — pinneapple_systems.digital_twin.io.sensors (v06)
- Sensors C — pinneapple_simulation.rover.autolab.estimation (v03)
- SensorSpec C — pinneapple_systems.digital_twin.product.spec (v06)
- Sentinel2Frame C — pinneapple_perception.sentinel2 (v04)
- Seq2SeqRNN C — pinneapple_neural.architectures.recurrent.seq2seq (v04)
- Sequential C — pinneapple_core.module (v01)
- Series C — pinneapple_systems.digital_twin.io.usgs (v06)
- Series C — pinneapple_tools.visualization.dashboard.spec (v07)
- SeriesData C — pinneapple_systems.time_series.forecast_lab.base (v06)
- SeriesDecomp C — pinneapple_neural.architectures.transformers.autoformer (v04)
- serve f — pinneapple_autolab.dashboard (v07)
- serve f — pinneapple_flow.studio (v07)
- serve f — pinneapple_lab.server (v07)
- serve f — pinneapple_tools.visualization.studio (v07)
- serve f — pinneapple_tools.visualization.studio.web (v07)
- serve f — pinneapple_twin3d.serve (v06)
- serve_in_thread f — pinneapple_tools.model_services.server (v07)
- serve_mlx_command f — pinneapple_llm.workbench.finetune (v07)
- serve_spec f — pinneapple_presets.surrogate (v02)
- ServiceError C — pinneapple_tools.model_services.client (v07)
- Session C — pinneapple_llm.workbench.runtime (v07)
- SessionSettings C — pinneapple_llm.workbench.runtime (v07)
- set_backend f — pinneapple_tools.compute_backends.backend (v07)
- set_config f — pinneapple_mcp.config (v07)
- set_data_dir f — pinneapple_analysis.benchmark.builtin.airfoil_impl (v05)
- set_data_dir f — pinneapple_analysis.benchmark.builtin.inverse_heat_impl (v05)
- set_deterministic f — pinneapple_neural.trainer.audit (v04)
- set_entry f — pinneapple_simulation.external_solvers.openfoam.campaign (v03)
- set_lr f — pinneapple_analysis.cost.equal_budget (v05)
- set_parameters f — pinneapple_simulation.pipelines.freecad_openfoam.freecad_scripts.fc_job (v03)
- set_seed f — pinneapple_neural.trainer.audit (v04)
- set_shell_thickness f — pinneapple_simulation.external_solvers.openradioss.deck (v03)
- set_solver_entries f — pinneapple_simulation.accelerated_foam.benchmark (v03)
- Settings C — pinneapple_flow.config (v07)
- setup_mpl f — pinneapple_tools.reporting.paperkit (v07)
- setup_threads f — pinneapple_tools.benchmark_suite.crossframework.records (v07)
- severity_of f — pinneapple_systems.operations.anomaly (v06)
- sgp4_decay f — pinneapple_physics.astro.decay (v02)
- sha256 f — pinneapple_data.lineage.graph (v04)
- sha256 f — pinneapple_design.geometry_library.store (v06)
- sha256_bytes f — pinneapple_security.integrity (v05)
- sha256_bytes f — pinneapple_veriphysics.ledger (v05)
- sha256_file f — pinneapple_core.workflows.engine (v01)
- sha256_file f — pinneapple_data.contracts.models (v04)
- sha256_file f — pinneapple_hub.zoo._artifacts (v07)
- sha256_file f — pinneapple_security.integrity (v05)
- sha256_file f — pinneapple_veriphysics.ledger (v05)
- sha256_json f — pinneapple_veriphysics.ledger (v05)
- shade_terrain f — pinneapple_tools.visualization.studio.shadows (v07)
- shadow_diameter_rad f — pinneapple_physics.astro.lensing (v02)
- shakura_sunyaev_accretion_disk f — pinneapple_physics.pde_environment.presets.astrophysics (v02)
- shakura_sunyaev_flux_exact f — pinneapple_physics.pde_environment.presets.astrophysics (v02)
- shakura_sunyaev_teff_exact f — pinneapple_physics.pde_environment.presets.astrophysics (v02)
- shallow_water f — pinneapple_physics.physics_audit.assumptions (v02)
- shallow_water_pinn_study f — pinneapple_systems.time_series.enso.pipeline (v06)

- ShallowWater2D C — pinneapple_simulation.numerical_solvers.shallow_water_fv (v03)
- ShallowWaterOcean C — pinneapple_systems.time_series.enso.intermediate (v06)
- shannon_entropy f — pinneapple_security.secrets_scan (v05)
- Shape C — pinneapple_simulation.external_solvers.newton_physics.scene (v03)
- shape_am_two_tone f — pinneapple_systems.excitation (v06)
- shape_chirp f — pinneapple_systems.excitation (v06)
- shape_descriptor f — pinneapple_design.geometry.retrieval (v06)
- shape_from_handles f — pinneapple_design.aero.geometry (v06)
- shape_gaussian_bumps f — pinneapple_systems.excitation (v06)
- shape_piecewise_constant f — pinneapple_systems.excitation (v06)
- shape_prbs f — pinneapple_systems.excitation (v06)
- shape_ramp f — pinneapple_systems.excitation (v06)
- shape_sawtooth f — pinneapple_systems.excitation (v06)
- shape_sinusoid f — pinneapple_systems.excitation (v06)
- shape_smoothed_random_walk f — pinneapple_systems.excitation (v06)
- shape_step f — pinneapple_systems.excitation (v06)
- ShapeParametrization C — pinneapple_design.design_optimizer.adjoint (v06)
- ShapleyAttributionResult C — pinneapple_analysis.inverse_problems.counterfactual_attribution (v05)
- ShardAwareConfig C — pinneapple_data.zarr_shard_iterable (v04)
- ShardAwareZarrUPDIterable C — pinneapple_data.zarr_shard_iterable (v04)
- ShardSpec C — pinneapple_data.zarr_shards (v04)
- ShardSpec C — pinneapple_pdb.shard (v04)
- shear_layer_velocity_fns f — pinneapple_simulation.numerical_solvers.jax_cfd_solver (v03)
- ShearWind C — pinneapple_simulation.flight.turbulence (v03)
- sheet f — pinneapple_simulation.pipelines.freecad_openfoam.freecad_scripts.fc_job (v03)
- shell_solid f — pinneapple_design.geometry.ops.occ_features (v06)
- ShellTubeCooler C — pinneapple_presets.cases.shell_tube (v02)
- sheltered_faces f — pinneapple_design.qualitative.models (v06)
- shields_endpoint f — pinneapple_analysis.validation.report_artifact (v05)
- shots_arrays f — pinneapple_systems.sports.football.xg (v06)
- showcase f — pinneapple_autolab.lab (v07)
- showcase_score f — pinneapple_lab.html (v07)
- shrink_png f — pinneapple_autolab.report (v07)
- shuffle_surrogate f — pinneapple_systems.time_series.predictability (v06)
- ShuffledKFoldSplitter C — pinneapple_systems.time_series.validation.splitters (v06)
- si_unit_for f — pinneapple_data.physical_units (v04)
- sidecar_path f — pinneapple_data.contracts.tables (v04)
- sigma_is_default f — pinneapple_neural.architectures.reservoir_computing.rbf (v04)
- sign_bytes f — pinneapple_security.signing (v05)
- sign_file f — pinneapple_security.signing (v05)
- sign_json f — pinneapple_security.signing (v05)
- sign_model_card f — pinneapple_security.integrate (v05)
- sign_record f — pinneapple_security.integrate (v05)
- sign_report f — pinneapple_analysis.validation.report_artifact (v05)
- sign_test_p f — pinneapple_tools.benchmark_suite.crossframework.evaluate (v07)
- sign_violations f — pinneapple_tools.nvidia.physicsnemo (v07)
- SignalProfile C — pinneapple_systems.digital_twin.conditioning (v06)
- SignalReconstructor C — pinneapple_systems.digital_twin.signal_reconstruction (v06)
- signed_distance_inside f — pinneapple_design.bricks.statics (v06)
- signed_volume f — pinneapple_design.geometry.vehicle_cad (v06)
- signed_volume_sign f — pinneapple_data.cae.geometry (v04)
- SigningKey C — pinneapple_security.signing (v05)
- Similarity C — pinneapple_analysis.validation.metamorphic (v05)
- SimOutcome C — pinneapple_autolab.simulator (v07)
- SimpleExpSmoothing C — pinneapple_systems.time_series.adaptive (v06)
- SimplePhysicalSample C — pinneapple_data.synth.pde (v04)

- simplify f — pinneapple_simulation.rover.autolab.planner (v03)
- simplify_mesh f — pinneapple_design.geometry.ops.simplify (v06)
- simulate f — pinneapple_design.thermal.board_cooling.simulate (v06)
- simulate f — pinneapple_flow.nodes.vehicle (v07)
- simulate f — pinneapple_flow.nodes_ext.astro (v07)
- simulate f — pinneapple_simulation (v03)
- simulate f — pinneapple_simulation.electromagnetics.antenna.ava (v03)
- simulate f — pinneapple_simulation.electromagnetics.periodic.tcda (v03)
- simulate f — pinneapple_simulation.external_solvers.newton_physics.runner (v03)
- simulate f — pinneapple_simulation.flight.simulate (v03)
- simulate f — pinneapple_simulation.numerical_solvers.cylinder_wake (v03)
- simulate f — pinneapple_simulation.numerical_solvers.thermal_channel_datasets (v03)
- simulate f — pinneapple_systems.operations.plant (v06)
- simulate f — pinneapple_systems.sports.football.ball (v06)
- simulate f — pinneapple_twin3d.wear.optimize (v06)
- simulate_car f — pinneapple_lab.experiments.car (v07)
- simulate_delayed f — pinneapple_systems.time_series.enso.oscillators (v06)
- simulate_fmu f — pinneapple_simulation.external_solvers.modelica.fmi_runner (v03)
- simulate_many f — pinneapple_design.thermal.board_cooling.simulate (v06)
- simulate_mo f — pinneapple_simulation.external_solvers.modelica.omc (v03)
- simulate_openmodelica f — pinneapple_simulation.external_solvers.modelica.om_runner (v03)
- simulate_recharge f — pinneapple_systems.time_series.enso.oscillators (v06)
- simulate_release_timeseries f — pinneapple_design.surrogate_inverse.transdermal_patch (v06)
- simulate_release_timeseries f — pinneapple_physics.closed_form.membrane_diffusion (v02)
- simulate_run f — pinneapple_physics.blackhole.runs (v02)
- simulate_sensors f — pinneapple_flow.twin (v07)
- simulate_takeoff f — pinneapple_simulation.flight.takeoff (v03)
- simulate_three_tank f — pinneapple_security.cps_bench (v05)
- simulate_torsional_stickslip f — pinneapple_systems.process_components.torsional_stickslip (v06)
- SimulationBlowUp C — pinneapple_simulation.numerical_solvers.etdrk4 (v03)
- Simulator C — pinneapple_autolab.simulator (v07)
- SINDy C — pinneapple_neural.architectures.rom.sindy (v04)
- sindy f — pinneapple_lab.experiments.discovery (v07)
- SINDyForecaster C — pinneapple_systems.time_series.forecast_lab.physical (v06)
- SINDyIdentifier C — pinneapple_analysis.inverse_problems.missing_term (v05)
- SINDyResult C — pinneapple_analysis.inverse_problems.missing_term (v05)
- Sine C — pinneapple_neural.architectures.benchmarks.generic_pinn_models (v04)
- SineLayer C — pinneapple_neural.architectures.siren (v04)
- single_stud_torsion f — pinneapple_design.bricks.statics (v06)
- single_thread f — pinneapple_analysis.benchmark.builtin.inverse_heat_impl (v05)
- single_wheel_check f — pinneapple_simulation.rover.checks (v03)
- SingleBatchDataset C — pinneapple_physics.pinn_solver.compiler.dataset (v02)
- sink_mass_and_air f — pinneapple_design.thermal.board_cooling.simulate (v06)
- sinusoid f — pinneapple_simulation.electromagnetics.fdtd (v03)
- SinusoidalEmbedding C — pinneapple_neural.architectures.pinns.physics_diffusion (v04)
- SinusoidalPositionalEncoding C — pinneapple_neural.architectures.transformers.informer (v04)
- sir_epidemic f — pinneapple_physics.pde_environment.presets.multidisciplinary (v02)
- SIREN C — pinneapple_neural.architectures.siren (v04)
- site f — pinneapple_flow.nodes.vehicle (v07)
- size f — pinneapple_design.thermal.heatsink.sizer (v06)
- size_gear f — pinneapple_simulation.flight.gear (v03)
- size_layers f — pinneapple_design.fitmesh.layers (v06)
- size_to_pass f — pinneapple_reason.modes._util (v05)
- SizingRequest C — pinneapple_design.thermal.heatsink.sizer (v06)
- skew f — pinneapple_simulation.multibody.tree (v03)
- skewed_field_3d f — pinneapple_simulation.electromagnetics.machines.robustness (v03)

- skewed_no_load f — pinneapple_simulation.electromagnetics.machines.robustness (v03)
- skewed_torque f — pinneapple_simulation.electromagnetics.machines.robustness (v03)
- skill_against f — pinneapple_systems.time_series.enso.pipeline (v06)
- skill_figure f — pinneapple_physics.weather.viz (v02)
- skill_horizon f — pinneapple_systems.time_series.forecast_lab.study (v06)
- skill_score f — pinneapple_analysis.uncertainty.resampling (v05)
- skill_score f — pinneapple_systems.time_series.metrics_ext.comparison (v06)
- skill_vs_lead f — pinneapple_analysis.verification.forecast_skill (v05)
- skillcorner_ball f — pinneapple_systems.sports.football.data (v06)
- skillcorner_matches f — pinneapple_systems.sports.football.data (v06)
- SkillResult C — pinneapple_analysis.verification.forecast_skill (v05)
- skin_depth f — pinneapple_physics.closed_form.eddy_coil (v02)
- skin_depth f — pinneapple_simulation.electromagnetics.waves (v03)
- skin_effect_slab f — pinneapple_physics.closed_form.electromagnetics (v02)
- skin_effect_wire f — pinneapple_simulation.electromagnetics.analytic (v03)
- skipped_heavy f — pinneapple_arena.external.result (v04)
- sky_brightness f — pinneapple_physics.astro.observing (v02)
- sky_state f — pinneapple_physics.astro.phenomena (v02)
- sky_temperature f — pinneapple_reason.props (v05)
- sky_view f — pinneapple_physics.astro.conditions (v02)
- slab_for_footprint f — pinneapple_tools.visualization.molecular (v07)
- slab_frames f — pinneapple_systems.time_series.enso.pipeline (v06)
- slab_heat_solve f — pinneapple_core.workflows.templates (v01)
- slab_scene f — pinneapple_systems.time_series.enso.render (v06)
- slab_transient f — pinneapple_physics.closed_form.heat (v02)
- slerp f — pinneapple_tools.visualization.studio.animation (v07)
- slew f — pinneapple_physics.astro.observing (v02)
- Slice C — pinneapple_tools.visualization.studio.scene (v07)
- slice_viewer f — pinneapple_science.em.research (v02)
- sliced_wasserstein f — pinneapple_analysis.uncertainty.posterior_metrics (v05)
- sliding_friction f — pinneapple_simulation.external_solvers.newton_physics.validation (v03)
- slip_angle f — pinneapple_simulation.terramechanics.wong_reece (v03)
- slip_ratio f — pinneapple_simulation.terramechanics.wong_reece (v03)
- Slot C — pinneapple_design.intent.spec (v06)
- slugify f — pinneapple_tools.hpo_experiments.kb.store (v07)
- SLURMConfig C — pinneapple_neural.trainer.hpc (v04)
- SlurmExecutor C — pinneapple_core.workflows.executors (v01)
- Smagorinsky C — pinneapple_physics.multiscale.closures (v02)
- small_strain f — pinneapple_physics.physics_audit.assumptions (v02)
- SmallBody C — pinneapple_physics.astro.smallbodies (v02)
- smape f — pinneapple_systems.time_series.metrics (v06)
- smape f — pinneapple_systems.time_series.metrics_ext.point (v06)
- smape_fn f — pinneapple_systems.time_series.metrics (v06)
- smelting_reduction_reactor f — pinneapple_twin3d.wear.presets (v06)
- smoke f — pinneapple_flow.autocalls (v07)
- smoke_run f — pinneapple_tools.hpo_experiments.utils.verify_runtime (v07)
- smooth_block f — pinneapple_design.bricks.parts (v06)
- smooth_mesh f — pinneapple_design.geometry.ops.clean (v06)
- smoothing_figure f — pinneapple_systems.sports.football.plots (v06)
- sn_curve_cycles_to_failure f — pinneapple_systems.process_components.fatigue_analysis (v06)
- snap_to_box f — pinneapple_condense.meshless2d (v02)
- snappy_dict f — pinneapple_simulation.numerical_solvers.external_flow (v03)
- snapshot_values f — pinneapple_analysis.surrogate.data (v05)
- snapshots_from_arrays f — pinneapple_data.snapshots (v04)
- SnapshotSet C — pinneapple_data.snapshots (v04)
- snippet f — pinneapple_core.onboarding.catalog (v01)

- sobol_from_evaluations f — pinneapple_analysis.sensitivity.sobol (v05)
- sobol_from_runs f — pinneapple_analysis.sensitivity.sobol (v05)
- sobol_g f — pinneapple_analysis.sensitivity.testfunctions (v05)
- sobol_g_indices f — pinneapple_analysis.sensitivity.testfunctions (v05)
- sobol_g_space f — pinneapple_analysis.sensitivity.testfunctions (v05)
- sobol_indices f — pinneapple_analysis.sensitivity.sobol (v05)
- SobolIndices C — pinneapple_analysis.sensitivity.sobol (v05)
- SobolResult C — pinneapple_analysis.inverse_problems.sensitivity (v05)
- sod_exact f — pinneapple_simulation.numerical_solvers.compressible_fv (v03)
- sod_exact_solution f — pinneapple_physics.pde_environment.presets.astrophysics (v02)
- sod_shock_tube_astro f — pinneapple_physics.pde_environment.presets.astrophysics (v02)
- soft_sand f — pinneapple_simulation.rover.autolab.site (v03)
- SoftAdapt C — pinneapple_neural.trainer.loss_balancer (v04)
- softmax_dict f — pinneapple_decision.debias (v05)
- soi_from_mslp f — pinneapple_systems.time_series.enso.data (v06)
- Soil C — pinneapple_simulation.terramechanics.soils (v03)
- soil f — pinneapple_flow.nodes.vehicle (v07)
- soil_theta f — pinneapple_simulation.rover.autolab.estimation (v03)
- SoilBelief C — pinneapple_simulation.rover.autolab.estimation (v03)
- SoilParams C — pinneapple_simulation.particle_dynamics.terramechanics (v03)
- SoilRegion C — pinneapple_simulation.rover.terrain (v03)
- SoilUnit C — pinneapple_simulation.rover.autolab.site (v03)
- solar_eclipses f — pinneapple_physics.astro.phenomena (v02)
- solar_panel f — pinneapple_design.geometry.vehicle_cad (v06)
- solid_pipe_mesh f — pinneapple_llm.qa.examples.pipe_wear_twin (v07)
- SolidFEM C — pinneapple_simulation.numerical_solvers.solid_fem (v03)
- SolidFEMExperiment C — pinneapple_lab.experiments.structures (v07)
- solidify f — pinneapple_design.geometry_library.meshops (v06)
- Solution C — pinneapple_physics.solving (v02)
- solution_dirs f — pinneapple_data.cae.geometry (v04)
- solution_to_vtu f — pinneapple_simulation.electromagnetics.machines.export (v03)
- solve f — pinneapple_condense.flow (v02)
- solve f — pinneapple_condense.fv (v02)
- solve f — pinneapple_condense.meshless (v02)
- solve f — pinneapple_design.aero.vlm (v06)
- solve f — pinneapple_design.bricks.fea (v06)
- solve f — pinneapple_flow.nodes_ext.structures (v07)
- solve f — pinneapple_mcp.tools (v07)
- solve f — pinneapple_physics.pcb.solver (v02)
- solve f — pinneapple_physics.solving (v02)
- solve f — pinneapple_quantum.pipeline.presets (v04)
- solve f — pinneapple_simulation.electromagnetics.machines.getdp (v03)
- solve f — pinneapple_simulation.external_solvers.calculix.study (v03)
- solve f — pinneapple_simulation.meshless.methods (v03)
- solve_adaptive f — pinneapple_condense.learned (v02)
- solve_and_package f — pinneapple_simulation.external_solvers.fenics.solver (v03)
- solve_axisymmetric_eddy_current f — pinneapple_simulation.numerical_solvers.eddy_current_fdm (v03)
- solve_base_field f — pinneapple_design.thermal.heatsink.field (v06)
- solve_beam f — pinneapple_systems.process_components.beam_statics (v06)
- solve_blocks f — pinneapple_condense.learned (v02)
- solve_burgers f — pinneapple_simulation.numerical_solvers.operator_datasets (v03)
- solve_darcy f — pinneapple_simulation.numerical_solvers.operator_datasets (v03)
- solve_diffusion_reaction f — pinneapple_tools.benchmark_suite.deeponet_ground_truth (v07)
- solve_electrostatic f — pinneapple_simulation.electromagnetics.electrostatics (v03)
- solve_euler_bernoulli_beam_1d f — pinneapple_simulation.numerical_solvers.beam_bvp_fdm (v03)
- solve_exact f — pinneapple_condense.learned (v02)

- solve_fd f — pinneapple_simulation.numerical_solvers.symbolic_fd (v03)
- solve_fin_cell f — pinneapple_design.thermal.fin_cell (v06)
- solve_forward f — pinneapple_simulation.numerical_solvers.gated_xtfc (v03)
- solve_homogenized f — pinneapple_condense.homogenize (v02)
- solve_ibm_external_flow f — pinneapple_simulation.numerical_solvers.immersed_boundary_fdm (v03)
- solve_ibm_internal_flow f — pinneapple_simulation.numerical_solvers.immersed_boundary_fdm (v03)
- solve_ibm_navier_stokes f — pinneapple_simulation.numerical_solvers.immersed_boundary_fdm (v03)
- solve_inverse f — pinneapple_analysis.inverse_problems.studio (v05)
- solve_inverse_problem f — pinneapple_analysis.verification.uncertainty_inverse (v05)
- solve_kepler f — pinneapple_physics.astro.orbits (v02)
- solve_magnetostatic f — pinneapple_simulation.electromagnetics.fem (v03)
- solve_mfl_nonlinear f — pinneapple_analysis.inspection.magnetic_flux_leakage (v05)
- solve_network f — pinneapple_simulation.numerical_solvers.pipe_network (v03)
- solve_nodal_darcy f — pinneapple_physics.darcy_checks (v02)
- solve_nonlinear_beam_static f — pinneapple_systems.process_components.beam_nonlinear_fem (v06)
- solve_nonlinear_beam_transient f — pinneapple_simulation.numerical_solvers.nonlinear_beam_fem (v03)
- solve_ns f — pinneapple_simulation.numerical_solvers.operator_datasets (v03)
- solve_ode_linear f — pinneapple_tools.benchmark_suite.deeponet_ground_truth (v07)
- solve_ode_nonlinear f — pinneapple_tools.benchmark_suite.deeponet_ground_truth (v07)
- solve_package f — pinneapple_condense.fv (v02)
- solve_path_from_pressure_ratio f — pinneapple_systems.process_components.polytropic_path (v06)
- solve_path_from_work f — pinneapple_systems.process_components.polytropic_path (v06)
- solve_pde f — pinneapple_physics (v02)
- solve_pde_kwargs_from_batch f — pinneapple_physics.pde_environment.presets.tag_geometry (v02)
- solve_pendulum f — pinneapple_tools.benchmark_suite.deeponet_ground_truth (v07)
- solve_plane_stress f — pinneapple_simulation.numerical_solvers.plane_stress_fem (v03)
- solve_poisson f — pinneapple_core.fem (v01)
- solve_potential f — pinneapple_science.numerics (v02)
- solve_preset f — pinneapple_flow.wrappers (v07)
- solve_richards f — pinneapple_simulation.geophysics.richards (v03)
- solve_static_elasticity_3d f — pinneapple_simulation.numerical_solvers.elasticity3d_fdm (v03)
- solve_stiff_reaction_network f — pinneapple_simulation.numerical_solvers.stiff_kinetics_ode (v03)
- solve_time_harmonic f — pinneapple_simulation.electromagnetics.eddy (v03)
- solve_with_deepxde f — pinneapple_physics.deepxde_bridge.adapter (v02)
- solve_xtfc f — pinneapple_simulation.numerical_solvers.xtfc_general (v03)
- solve_xtfc_ode f — pinneapple_simulation.numerical_solvers.xtfc_ivp (v03)
- solve_xtfc_pde f — pinneapple_simulation.numerical_solvers.xtfc_pde (v03)
- solve_xtfc_subdomains f — pinneapple_simulation.numerical_solvers.xtfc_subdomain (v03)
- solved_files f — pinneapple_simulation.electromagnetics.antenna.scene3d (v03)
- SolveInfo C — pinneapple_condense.fv (v02)
- solver_available f — pinneapple_core.onboarding.catalog (v01)
- solver_checks f — pinneapple_systems.time_series.enso.pipeline (v06)
- solver_entries f — pinneapple_simulation.accelerated_foam.benchmark (v03)
- solver_from f — pinneapple_simulation.electromagnetics.periodic.benchmark (v03)
- solver_log f — pinneapple_tools.automation.adapters.openfoam (v07)
- solver_log_summary f — pinneapple_autolab.domains.racecar (v07)
- solver_qualification f — pinneapple_autolab.qualification (v07)
- solver_requirement f — pinneapple_core.onboarding.catalog (v01)
- solver_result_view f — pinneapple_tools.visualization.studio.solver_view (v07)
- SolverBase C — pinneapple_simulation.numerical_solvers.base (v03)
- SolverBridgeConfig C — pinneapple_worldmodel.physics_solver_bridge (v04)
- SolverCatalog C — pinneapple_simulation.numerical_solvers.registry (v03)
- SolveResult C — pinneapple_simulation.accelerated_foam.solvers (v03)
- SolverFeatureStep C — pinneapple_neural.trainer.preprocess (v04)
- SolverModule C — pinneapple_core.module (v01)

- SolverOutput C — pinneapple_simulation.numerical_solvers.base (v03)
- SolverOutput C — pinneapple_worldmodel.physics_solver_bridge (v04)
- SolverOutput3D C — pinneapple_simulation.numerical_solvers.fdm3d (v03)
- SolverRecommendation C — pinneapple_analysis.verification.solver_orchestration (v05)
- SolverRegistry C — pinneapple_simulation.numerical_solvers.registry (v03)
- SolverSpec C — pinneapple_simulation.numerical_solvers.registry (v03)
- sort_ranked f — pinneapple_tools.hpo_experiments.ranking (v07)
- Source C — pinneapple_design.geometry_library.core (v06)
- Source C — pinneapple_veriphysics.evidence_package (v05)
- source_root f — pinneapple_lab.experiments.landing (v07)
- source_vector f — pinneapple_condense.fv (v02)
- sources f — pinneapple_physics.astro.facts (v02)
- space_debris_cw_relative_motion f — pinneapple_physics.pde_environment.presets.astrophysics (v02)
- space_from_contract f — pinneapple_analysis.study_contracts (v05)
- space_readme f — pinneapple_hub.zoo.hub_cards (v07)
- space_to_contract f — pinneapple_analysis.study_contracts (v05)
- space_url f — pinneapple_hub.zoo.hub_cards (v07)
- spacecraft_attitude_euler_rotation f — pinneapple_physics.pde_environment.presets.astrophysics (v02)
- SpaceTime C — pinneapple_pdb.builder (v04)
- SpalartAllmarasResiduals C — pinneapple_physics.pde_environment.turbulence_presets (v02)
- SparsityRegularizer C — pinneapple_analysis.inverse_problems.regularization (v05)
- spatial_coords f — pinneapple_app.backend.core.physics_adapter (v07)
- SpatiotemporalGNN C — pinneapple_neural.architectures.graphnn.spatiotemporal_gnn (v04)
- spec f — pinneapple_systems.time_series.enso.study (v06)
- spec_for f — pinneapple_flow.libcalls (v07)
- spec_of f — pinneapple_analysis.validation.schema (v05)
- spec_to_urdf f — pinneapple_simulation.robot_sim.spec (v03)
- SpecError C — pinneapple_design.design_optimizer.spec (v06)
- SpecError C — pinneapple_design.intent.spec (v06)
- SpecError C — pinneapple_llm.qa.spec (v07)
- SpecError C — pinneapple_systems.digital_twin.product.spec (v06)
- SpecialistConfig C — pinneapple_worldmodel.specialist_trainer (v04)
- specialists f — pinneapple_flow.nodes_ext.worldmodel (v07)
- SpecialistTrainer C — pinneapple_worldmodel.specialist_trainer (v04)
- species_diffusion_residual f — pinneapple_systems.component_modeling.physics_residuals (v06)
- specific_impulse f — pinneapple_physics.closed_form.rocket_nozzle (v02)
- spectral f — pinneapple_check._reuse (v05)
- spectral_class f — pinneapple_physics.astro.stars_ml (v02)
- spectral_comparison f — pinneapple_analysis.validation.spectral (v05)
- spectral_derivative f — pinneapple_neural.architectures.neural_operators.pino (v04)
- spectral_derivative f — pinneapple_physics.multiscale.filters (v02)
- spectral_downsample f — pinneapple_simulation.numerical_solvers.operator_benchmarks (v03)
- spectral_drift f — pinneapple_analysis.validation.rollout (v05)
- spectral_error_shells f — pinneapple_analysis.validation.field_metrics (v05)
- spectral_fidelity f — pinneapple_analysis.validation.spectral (v05)
- spectral_peaks f — pinneapple_science.numerics (v02)
- spectral_relative_geomean f — pinneapple_analysis.uncertainty.posterior_metrics (v05)
- SpectralComparison C — pinneapple_analysis.validation.spectral (v05)
- SpectralConvND C — pinneapple_neural.architectures.neural_operators.uno (v04)
- SpectralFidelityReport C — pinneapple_analysis.validation.spectral (v05)
- SpectralNS3D C — pinneapple_simulation.numerical_solvers.ns3d_spectral (v03)
- SpectralSolver C — pinneapple_simulation.numerical_solvers.spectral (v03)
- spectrum_error f — pinneapple_physics.multiscale.closures (v02)
- sph_box f — pinneapple_flow.nodes_ext.solver_registry (v07)
- Sphere C — pinneapple_worldmodel.geometry (v04)

- sphere f — pinneapple_design.geometry.bodies (v06)
- sphere f — pinneapple_physics.blackhole.twin (v02)
- sphere f — pinneapple_simulation.numerical_solvers.lbm_tunnel (v03)
- sphere f — pinneapple_simulation.pipelines.freecad_openfoam.studies (v03)
- sphere3d f — pinneapple_design.geometry.gen.sdf_shapes (v06)
- sphere_drag f — pinneapple_flow.nodes_ext.lbm (v07)
- sphere_pad f — pinneapple_physics.weather.model (v02)
- sphere_stokes f — pinneapple_flow.nodes_ext.lbm (v07)
- SphereShallowWater C — pinneapple_simulation.geophysics.sphere_shallow_water (v03)
- SphereUNet C — pinneapple_physics.weather.model (v02)
- spherical_capacitance f — pinneapple_simulation.electromagnetics.statics (v03)
- SphericalHarmonics C — pinneapple_simulation.geophysics.sphere_shallow_water (v03)
- SPHParticles C — pinneapple_simulation.particle_dynamics.particles (v03)
- SPHSolver C — pinneapple_simulation.numerical_solvers.sph (v03)
- SPHState C — pinneapple_simulation.numerical_solvers.sph (v03)
- SpinChainDataset C — pinneapple_quantum.data.quantum_dataset (v04)
- SPK C — pinneapple_physics.astro.spk (v02)
- SplashCase C — pinneapple_data.adapters.splash_archive_adapter (v04)
- SplashMesh C — pinneapple_data.adapters.splash_archive_adapter (v04)
- Split C — pinneapple_neural.architectures.rom.study (v04)
- Split C — pinneapple_systems.time_series.validation.splitters (v06)
- split f — pinneapple_analysis.benchmark.builtin.airfoil_impl (v05)
- split f — pinneapple_condense.flow (v02)
- split f — pinneapple_design.aero.train (v06)
- split f — pinneapple_flow.learn (v07)
- split_files f — pinneapple_hub.zoo.datasets (v07)
- split_header f — pinneapple_data.physical_units (v04)
- split_indices f — pinneapple_analysis.uncertainty.conformal_study (v05)
- split_indices f — pinneapple_data.splits (v04)
- split_indices f — pinneapple_design.thermal.heatsink.monotone (v06)
- split_mask f — pinneapple_llm.qa.references (v07)
- split_order f — pinneapple_hub.zoo.hub_cards (v07)
- split_output_ports f — pinneapple_systems.cosimulation.node (v06)
- split_physical_samples f — pinneapple_data.noether_dataset (v04)
- split_procs f — pinneapple_simulation.pipelines.freecad_openfoam.mesh (v03)
- split_sections f — pinneapple_tools.hpo_experiments.sources.arxiv_pdf (v07)
- split_snapshots f — pinneapple_neural.architectures.rom.study (v04)
- split_surface f — pinneapple_simulation.numerical_solvers.free_surface (v03)
- Splits C — pinneapple_analysis.surrogate.splits (v05)
- Splits C — pinneapple_data.contracts.models (v04)
- splits f — pinneapple_analysis.benchmark.builtin.airfoil_impl (v05)
- SplitSpec C — pinneapple_data.splits (v04)
- spm_machine f — pinneapple_flow.nodes_ext.electromagnetics (v07)
- SPMMachine C — pinneapple_simulation.electromagnetics.machine (v03)
- SPMMotor C — pinneapple_presets.cases.spm_motor (v02)
- spp_wavenumber f — pinneapple_simulation.electromagnetics.waves (v03)
- spreading_resistance f — pinneapple_physics.closed_form.plate_fin_heatsink (v02)
- spring_barrier_hindcasts f — pinneapple_systems.time_series.enso.pipeline (v06)
- spur_gear f — pinneapple_design.geometry.gen.cadquery_advanced (v06)
- src f — pinneapple_analysis.uncertainty.tolerance (v05)
- ssa_decompose f — pinneapple_simulation.numerical_solvers.ssa (v03)
- SSASolver C — pinneapple_simulation.numerical_solvers.ssa (v03)
- sst_stft_1d f — pinneapple_simulation.numerical_solvers.sst (v03)
- sst_tendency f — pinneapple_systems.time_series.enso.intermediate (v06)
- SSTSolver C — pinneapple_simulation.numerical_solvers.sst (v03)
- stability_derivatives f — pinneapple_simulation.flight.trim (v03)

- stable_dt f — pinneapple_simulation.geophysics.sphere_shallow_water (v03)
- stable_time_step f — pinneapple_simulation.numerical_solvers.stable_fluids (v03)
- StableFluids2D C — pinneapple_simulation.numerical_solvers.stable_fluids (v03)
- StableFluidsState C — pinneapple_simulation.numerical_solvers.stable_fluids (v03)
- Stack C — pinneapple_condense.package (v02)
- stack f — pinneapple_design.bricks.assembly (v06)
- stacked f — pinneapple_simulation.electromagnetics.machines.steel (v03)
- stackup f — pinneapple_physics.closed_form.pcb_thermal (v02)
- Stage C — pinneapple_problemdesign.elicitation.stages (v02)
- stage_case f — pinneapple_simulation.external_solvers.openradioss.deck (v03)
- stage_case_for_scenario f — pinneapple_simulation.external_solvers.openfoam.case_builder (v03)
- stage_derivatives f — pinneapple_simulation.numerical_solvers.discrete_time_pinn (v03)
- stall_angle f — pinneapple_simulation.flight.encounter (v03)
- stamp_all f — pinneapple_lab.seal (v07)
- stamp_figure f — pinneapple_lab.seal (v07)
- stamp_file f — pinneapple_lab.seal (v07)
- stamp_image f — pinneapple_lab.seal (v07)
- stamp_video f — pinneapple_lab.seal (v07)
- stamped_jpeg f — pinneapple_lab.explorer (v07)
- standalone_snippet f — pinneapple_core.onboarding.catalog (v01)
- standard_features f — pinneapple_systems.sports.football.xg (v06)
- standard_volumetric_flow_to_mass_flow f — pinneapple_systems.process_components.real_gas_eos (v06)
- StandardConditions C — pinneapple_systems.process_components.real_gas_eos (v06)
- standardize f — pinneapple_data.unified (v04)
- standardize_dims f — pinneapple_pdb.validate (v04)
- StandardScaler C — pinneapple_data.transforms (v04)
- star_apparent f — pinneapple_physics.astro.astrometry (v02)
- star_field f — pinneapple_physics.blackhole.raytrace (v02)
- star_position f — pinneapple_physics.astro.journey (v02)
- star_properties f — pinneapple_physics.astro.stars (v02)
- star_splat f — pinneapple_physics.astro.lensing (v02)
- StarCatalog C — pinneapple_physics.astro.stars (v02)
- starccm_cgns_to_upd f — pinneapple_simulation.external_solvers.cfd_formats.starccm_reader (v03)
- starccm_table_to_upd f — pinneapple_simulation.external_solvers.cfd_formats.starccm_reader (v03)
- start_server f — pinneapple_llm.local_llm (v07)
- State C — pinneapple_condense.query (v02)
- state_from_elements f — pinneapple_physics.astro.orbits (v02)
- state_from_Ph f — pinneapple_systems.process_components.real_gas_eos (v06)
- state_from_Ps f — pinneapple_systems.process_components.real_gas_eos (v06)
- state_from_PT f — pinneapple_systems.process_components.real_gas_eos (v06)
- state_pairs f — pinneapple_worldmodel.physical_world (v04)
- state_vector f — pinneapple_simulation.flight.rigid_body (v03)
- StateEncoder C — pinneapple_worldmodel.physical_world (v04)
- statement f — pinneapple_security.attestation (v05)
- Static C — pinneapple_simulation.external_solvers.calculix.study (v03)
- static_fields f — pinneapple_physics.weather.model (v02)
- static_sinkage_check f — pinneapple_simulation.rover.checks (v03)
- StaticNonlinearBeamResult C — pinneapple_systems.process_components.beam_nonlinear_fem (v06)
- StaticPolicy C — pinneapple_llm.qa.static (v07)
- StaticResult C — pinneapple_llm.qa.static (v07)
- StaticsResult C — pinneapple_design.bricks.statics (v06)
- stationarity_report f — pinneapple_systems.time_series.eda.plots (v06)
- statistics f — pinneapple_flow.nodes_ext.astro (v07)
- statsbomb_shots f — pinneapple_systems.sports.football.data (v06)
- statsmodels_nino12 f — pinneapple_systems.time_series.enso.data (v06)
- StatsmodelsARIMAForecaster C — pinneapple_systems.time_series.classical_wrappers (v06)

- Status C — pinneapple_llm.qa.report (v07)
- status f — pinneapple_analysis.reproduction (v05)
- status f — pinneapple_flow.examples (v07)
- status f — pinneapple_hub.zoo (v07)
- status f — pinneapple_hub.zoo._artifacts (v07)
- status f — pinneapple_hub.zoo.datasets (v07)
- status f — pinneapple_mcp.jobs (v07)
- status f — pinneapple_twin3d.wear.forecast (v06)
- steady_heat_conduction_3d_default f — pinneapple_physics.pde_environment.presets.industry (v02)
- steady_state f — pinneapple_systems.digital_twin.pipe_loop.plant (v06)
- steady_state f — pinneapple_systems.operations.plant (v06)
- steady_state f — pinneapple_systems.process_components.heat_exchanger (v06)
- steady_state_release_rate_mg_day f — pinneapple_physics.closed_form.membrane_diffusion (v02)
- steady_values f — pinneapple_design.aero.dataset (v06)
- SteadyProfilePoint C — pinneapple_systems.process_components.pipe_network_1d (v06)
- SteamLineInsulation C — pinneapple_presets.cases.insulation (v02)
- steel_ladle f — pinneapple_twin3d.wear.presets (v06)
- SteelFootbridge C — pinneapple_presets.cases.footbridge (v02)
- SteelGrade C — pinneapple_simulation.electromagnetics.machines.steel (v03)
- stencil f — pinneapple_tools.benchmark_suite.crossframework.problems (v07)
- stencil_weights f — pinneapple_simulation.meshless.stencils (v03)
- Step C — pinneapple_autolab.core (v07)
- Step C — pinneapple_core.workflows.dag (v01)
- Step C — pinneapple_reason.chain (v05)
- Step C — pinneapple_tools.automation.workflow (v07)
- Step C — pinneapple_tools.reporting.datasheet (v07)
- step_check f — pinneapple_design.geometry_library.exporters (v06)
- step_hash f — pinneapple_core.workflows.engine (v01)
- step_irk f — pinneapple_simulation.numerical_solvers.irk_gauss_legendre (v03)
- step_offsets f — pinneapple_simulation.electromagnetics.machines.robustness (v03)
- step_response f — pinneapple_systems.time_series.eda.plots (v06)
- step_skew_factor f — pinneapple_simulation.electromagnetics.machines.robustness (v03)
- step_to_mesh f — pinneapple_design.geometry.io.step (v06)
- StepCancelled C — pinneapple_core.workflows.adapters (v01)
- StepContext C — pinneapple_core.workflows.adapters (v01)
- StepContext C — pinneapple_tools.automation.adapters.base (v07)
- StepFailed C — pinneapple_core.workflows.adapters (v01)
- StepImportConfig C — pinneapple_design.geometry.io.step (v06)
- StepLog C — pinneapple_simulation.rover.autolab.mission (v03)
- StepLog C — pinneapple_tools.visualization.molecular (v07)
- StepOutcome C — pinneapple_llm.qa.report (v07)
- StepPlanner C — pinneapple_autolab.core (v07)
- StepRecord C — pinneapple_tools.automation.record (v07)
- StepState C — pinneapple_simulation.rover.dynamics (v03)
- StickFixed C — pinneapple_simulation.flight.control (v03)
- StickSlipResult C — pinneapple_systems.process_components.torsional_stickslip (v06)
- stiffness f — pinneapple_core.fem (v01)
- stills f — pinneapple_presets.viz3d (v02)
- StirredTank C — pinneapple_simulation.numerical_solvers.particles (v03)
- stl_bytes f — pinneapple_design.geometry_library.exporters (v06)
- stl_to_upd f — pinneapple_data.adapters.geom_adapter (v04)
- STLDomainBatchBuilder C — pinneapple_design.geometry.builders.stl_domain_batch_builder (v06)
- STLDomainBatchConfig C — pinneapple_design.geometry.builders.stl_domain_batch_builder (v06)
- STLMesh C — pinneapple_data.stl_import (v04)
- STLSolver C — pinneapple_simulation.numerical_solvers.stl (v03)
- STLTemplateSynthGenerator C — pinneapple_data.synth.geometry_cadquery (v04)

- stochastic_oscillator f — pinneapple_systems.time_series.synthetic (v06)
- stoker f — pinneapple_simulation.numerical_solvers.shallow_water_fv (v03)
- stokes f — pinneapple_condense.meshless2d (v02)
- Stokes2D C — pinneapple_arena.problems (v04)
- stokes_sphere_field f — pinneapple_simulation.numerical_solvers.lbm_tunnel (v03)
- StokesDissipation2D C — pinneapple_design.topology.problems (v06)
- StopRecord C — pinneapple_twin3d.wear.calibrate (v06)
- stops f — pinneapple_tools.visualization.studio.compose (v07)
- Store C — pinneapple_llm.workbench.store (v07)
- stored f — pinneapple_hub.zoo.validation_reports (v07)
- strain f — pinneapple_physics.multiscale.closures (v02)
- strains_rz f — pinneapple_physics.solid.axisymmetric (v02)
- stream f — pinneapple_flow.nodes.twin_nodes (v07)
- StreamingConditioner C — pinneapple_systems.digital_twin.conditioning (v06)
- streamlines f — pinneapple_design.aero.vlm (v06)
- stress_field f — pinneapple_design.surrogate_inverse.suspension_arm (v06)
- stress_from_moment_pa f — pinneapple_physics.closed_form.cantilever_fatigue (v02)
- stress_tendency f — pinneapple_physics.multiscale.closures (v02)
- stresses_rz f — pinneapple_physics.solid.axisymmetric (v02)
- stribeck_friction_torque f — pinneapple_systems.process_components.torsional_stickslip (v06)
- strip_capacitor_2d f — pinneapple_science.em.statics (v02)
- strouhal f — pinneapple_physics.physics_audit.regime (v02)
- structural_checks f — pinneapple_analysis.validation.physics_learning (v05)
- StructuralObjective C — pinneapple_design.design_optimizer.objective (v06)
- Structure C — pinneapple_design.bricks.statics (v06)
- structure_functions f — pinneapple_analysis.validation.spectral (v05)
- StructureFunctions C — pinneapple_analysis.validation.spectral (v05)
- StructurePreservingNetwork C — pinneapple_neural.architectures.physics_aware.spn (v04)
- stud_cantilever_tension f — pinneapple_design.bricks.closed_form (v06)
- StudentTMisfit C — pinneapple_analysis.inverse_problems.noise_models (v05)
- studies_figure f — pinneapple_analysis.benchmark.report (v05)
- studies_markdown f — pinneapple_analysis.benchmark.report (v05)
- studio_scene f — pinneapple_design.bricks.mesh (v06)
- study f — pinneapple_flow.nodes.cfd_pipeline (v07)
- study_cost f — pinneapple_analysis.sensitivity.study (v05)
- study_figure f — pinneapple_simulation.electromagnetics.antenna.study (v03)
- study_metric_key f — pinneapple_analysis.benchmark.report (v05)
- study_result f — pinneapple_analysis.surrogate.contracts (v05)
- study_result f — pinneapple_design.design_optimizer.report (v06)
- study_result f — pinneapple_neural.architectures.rom.contracts (v04)
- study_result f — pinneapple_simulation.meshless.contracts (v03)
- StudyModel C — pinneapple_analysis.study_models (v05)
- StudyResult C — pinneapple_analysis.surrogate.study (v05)
- StudyResult C — pinneapple_data.contracts.models (v04)
- style f — pinneapple_science.plots (v02)
- SU2Client C — pinneapple_data.datasets.external_sources (v04)
- SubDataset C — pinneapple_arena.external.schema (v04)
- Subdomain C — pinneapple_physics.pinn_solver.domino (v02)
- SubdomainPINN C — pinneapple_physics.pinn_solver.domino (v02)
- subject f — pinneapple_security.attestation (v05)
- submit f — pinneapple_mcp.jobs (v07)
- SubnetWrapper C — pinneapple_neural.architectures.pinns.xpinn (v04)
- subsample f — pinneapple_tools.visualization.studio.fieldview (v07)
- subset_tile f — pinneapple_pdb.shard (v04)
- subset_time f — pinneapple_pdb.shard (v04)
- sufficiency f — pinneapple_analysis.validation.physics_learning (v05)

- suggest_geometries f — pinneapple_design.fitmesh.suggest (v06)
- suggest_parameters f — pinneapple_design.fitmesh.suggest (v06)
- suggest_problem_spec f — pinneapple_physics.pde_environment.capabilities (v02)
- suggest_roles f — pinneapple_data.process_optimizer (v04)
- Suggestion C — pinneapple_neural.trainer.advisor (v04)
- summarise f — pinneapple_analysis.benchmark.checks (v05)
- summarise f — pinneapple_data.simulation_metadata.openfoam (v04)
- summarise f — pinneapple_systems.sports.football.ball_study (v06)
- summarise_runs f — pinneapple_physics.multiscale.comparison (v02)
- summarize f — pinneapple_design.aero.optimize (v06)
- summarize f — pinneapple_design.thermal.fin_array_speed (v06)
- summarize f — pinneapple_flow.libcalls (v07)
- summarize f — pinneapple_physics.astro.conditions (v02)
- summarize f — pinneapple_tools.benchmark_suite.crossframework.evaluate (v07)
- summarize f — pinneapple_veriphysics.validation_lab (v05)
- summarize3d f — pinneapple_design.aero.optimize3d (v06)
- summarize_al f — pinneapple_design.aero.optimize_airliner (v06)
- summarize_array f — pinneapple_arena.external.result (v04)
- summarize_js_evidence f — pinneapple_tools.rea_bridge.evidence (v07)
- summarize_piv f — pinneapple_perception.piv_tools (v04)
- summarize_samples f — pinneapple_analysis.uncertainty.margin (v05)
- summary f — pinneapple_check.report (v05)
- summary f — pinneapple_flow.decisions (v07)
- summary f — pinneapple_flow.palette (v07)
- summary f — pinneapple_physics.metrics (v02)
- summary_figure f — pinneapple_physics.multiscale.comparison (v02)
- summary_metrics f — pinneapple_analysis.benchmark.report (v05)
- summary_table f — pinneapple_design.geometry_library.core (v06)
- SumWind C — pinneapple_simulation.flight.turbulence (v03)
- sun_deflection_arcsec f — pinneapple_physics.astro.lensing (v02)
- sun_itrs f — pinneapple_physics.astro.satellites (v02)
- superellipse f — pinneapple_design.geometry_library.kit (v06)
- superellipse_radius f — pinneapple_simulation.electromagnetics.reflector.geometry (v03)
- Superposition C — pinneapple_analysis.validation.metamorphic (v05)
- Superposition C — pinneapple_condense.query (v02)
- SupersededModelWarning C — pinneapple_hub.zoo._models (v07)
- supervised f — pinneapple_core.loss (v01)
- SupervisedLoss C — pinneapple_neural.trainer.losses (v04)
- Surface C — pinneapple_tools.visualization.studio.scene (v07)
- surface f — pinneapple_data.cae.compare (v04)
- surface_checks f — pinneapple_data.cae.quality (v04)
- surface_coefficient f — pinneapple_physics.closed_form.pipe_insulation (v02)
- surface_from_pointcloud f — pinneapple_design.geometry.mesh.mesher_3d (v06)
- surface_impedance f — pinneapple_simulation.electromagnetics.waves (v03)
- surface_scalars f — pinneapple_design.aero.case3d (v06)
- SurfaceRegion C — pinneapple_analysis.verification.geometry_intelligence (v05)
- surfaces f — pinneapple_design.aero.geometry (v06)
- SurfaceView C — pinneapple_tools.visualization.studio.planetary (v07)
- surprise_record f — pinneapple_systems.time_series.forecast_lab.pipeline (v06)
- Surrogate C — pinneapple_analysis.benchmark.builtin.inverse_heat_impl (v05)
- Surrogate C — pinneapple_design.thermal.heatsink.surrogate (v06)
- Surrogate C — pinneapple_neural.workflows.hybrid_surrogate_physics (v04)
- Surrogate C — pinneapple_systems.digital_twin.pipe_loop.diagnose (v06)
- surrogate f — pinneapple_flow.nodes.cfd_pipeline (v07)
- surrogate f — pinneapple_flow.nodes.surrogates (v07)
- surrogate_assisted f — pinneapple_design.design_optimizer.verify (v06)

- surrogate_from_table f — pinneapple_design.design_optimizer.report (v06)
- surrogate_inverse f — pinneapple_flow.nodes_ext.surrogate_inverse (v07)
- surrogate_job f — pinneapple_analysis.benchmark.builtin.inverse_heat_impl (v05)
- surrogate_model f — pinneapple_analysis.validation.physics_learning_cases (v05)
- surrogate_path f — pinneapple_presets.surrogate (v02)
- surrogate_report f — pinneapple_design.thermal.heatsink.trust (v06)
- surrogate_search f — pinneapple_design.bricks.search (v06)
- surrogate_service f — pinneapple_tools.nvidia.heatsink (v07)
- surrogate_steady_state f — pinneapple_simulation.rover.autolab.wheel_models (v03)
- surrogate_study f — pinneapple_flow.wrappers (v07)
- surrogate_test f — pinneapple_systems.time_series.predictability (v06)
- surrogate_trained f — pinneapple_systems.time_series.forecast_lab.study (v06)
- surrogate_vs_numerical f — pinneapple_simulation.rover.checks (v03)
- SurrogateConfig C — pinneapple_design.design_optimizer.surrogate (v06)
- SurrogateContact C — pinneapple_simulation.rover.contact (v03)
- SurrogateData C — pinneapple_analysis.surrogate.data (v05)
- SurrogateFit C — pinneapple_design.design_optimizer.report (v06)
- SurrogateFit C — pinneapple_design.surrogate_inverse.core (v06)
- SurrogateModel C — pinneapple_analysis.surrogate.model (v05)
- SurrogateModel C — pinneapple_systems.operations.twin (v06)
- SurrogatePolicy C — pinneapple_tools.surrogate_policy (v07)
- SurrogatePolicyResult C — pinneapple_tools.surrogate_policy (v07)
- SurrogatePrediction C — pinneapple_neural.workflows.hybrid_surrogate_physics (v04)
- SurrogateSpec C — pinneapple_presets.core (v02)
- SurrogateTest C — pinneapple_systems.time_series.predictability (v06)
- survey_cutout f — pinneapple_physics.astro.observing (v02)
- survey_url f — pinneapple_physics.astro.observing (v02)
- suspend f — pinneapple_simulation.numerical_solvers.particles (v03)
- sutherland_mu f — pinneapple_design.aero.buildup (v06)
- SWAGApproximation C — pinneapple_systems.component_modeling.bayesian (v06)
- Sweep C — pinneapple_simulation.pipelines.freecad_openfoam.sweep (v03)
- sweep f — pinneapple_flow.nodes.cfd_pipeline (v07)
- sweep f — pinneapple_hub.zoo.demo (v07)
- sweep f — pinneapple_lab.runner (v07)
- sweep f — pinneapple_simulation.electromagnetics.reflector.compact_range (v03)
- sweep f — pinneapple_twin3d.primitives (v06)
- sweep_figure f — pinneapple_simulation.pipelines.freecad_openfoam.report (v03)
- sweep_le_from_qc f — pinneapple_design.aero.airliner (v06)
- sweep_pareto_frontier f — pinneapple_design.surrogate_inverse.reflex_port (v06)
- sweep_profile f — pinneapple_design.geometry_library.kit (v06)
- SweepConfig C — pinneapple_neural.trainer.parallel (v04)
- SweepJob C — pinneapple_tools.benchmark_suite.runner.run_sweep (v07)
- switching_functions f — pinneapple_simulation.numerical_solvers.xtfc_general (v03)
- symbolic_identity f — pinneapple_analysis.proof.graph (v05)
- symbolic_true f — pinneapple_analysis.proof.graph (v05)
- SymbolicFDSynthGenerator C — pinneapple_data.synth.pde_symbolic (v04)
- SymbolicPDE C — pinneapple_physics.symbolic_pde.compiler (v02)
- SymbolicPDENode C — pinneapple_systems.cosimulation.adapters (v06)
- symbols f — pinneapple_hub.library.snippets (v07)
- symmetric_range f — pinneapple_tools.visualization.studio.compose (v07)
- Symmetry C — pinneapple_analysis.validation.metamorphic (v05)
- Symmetry C — pinneapple_core.transforms (v01)
- symmetry f — pinneapple_core.loss (v01)
- SymmetryCheck C — pinneapple_analysis.validation.symmetry (v05)
- SymmetryMap C — pinneapple_design.topology.filters (v06)
- SymplecticODENet C — pinneapple_neural.architectures.continuous.symplectic_ode (v04)

- SymplecticRNN C — pinneapple_neural.architectures.continuous.symplectic_rnn (v04)
- SympyBackend C — pinneapple_physics.pinn_solver.factory.sympy_backend (v02)
- SympyTorchCompiler C — pinneapple_physics.pinn_solver.factory.sympy_backend (v02)
- SynthCatalog C — pinneapple_data.synth.registry (v04)
- SynthConfig C — pinneapple_data.synth.base (v04)
- synthetic f — pinneapple_analysis.inverse_problems.presets (v05)
- synthetic_campaign f — pinneapple_twin3d.wear.synthetic (v06)
- synthetic_daily f — pinneapple_systems.time_series.solar_cycle (v06)
- synthetic_flights f — pinneapple_systems.sports.football.ball_study (v06)
- synthetic_recession f — pinneapple_systems.digital_twin.recession (v06)
- synthetic_recovery f — pinneapple_systems.sports.football.ball_study (v06)
- synthetic_runs f — pinneapple_systems.sports.football.kin_study (v06)
- synthetic_smoothing f — pinneapple_systems.sports.football.kin_study (v06)
- SyntheticDataFactory C — pinneapple_worldmodel.synthetic_factory (v04)
- SyntheticFactoryConfig C — pinneapple_worldmodel.synthetic_factory (v04)
- SyntheticSeries C — pinneapple_systems.time_series.synthetic (v06)
- SyntheticSite C — pinneapple_simulation.rover.autolab.site (v03)
- SynthGenerator C — pinneapple_data.synth.base (v04)
- SynthOutput C — pinneapple_data.synth.base (v04)
- System C — pinneapple_condense.fv (v02)
- system_of f — pinneapple_flow.nodes._common (v07)
- SystemState C — pinneapple_systems.digital_twin.state (v06)

T

- t10_pitch f — pinneapple_condense.falsify (v02)
- t1_point f — pinneapple_condense.falsify (v02)
- t1_reference f — pinneapple_condense.falsify (v02)
- t2_contrast f — pinneapple_condense.falsify (v02)
- t3_anisotropy f — pinneapple_condense.falsify (v02)
- t4_no_repetition f — pinneapple_condense.falsify (v02)
- t5_variants f — pinneapple_condense.falsify (v02)
- t6_t7_nonlinear f — pinneapple_condense.falsify (v02)
- t8_adversarial f — pinneapple_condense.falsify (v02)
- t90 f — pinneapple_twin3d.wear.forecast (v06)
- t9_thin f — pinneapple_condense.falsify (v02)
- t_baseline f — pinneapple_design.thermal.board_cooling.agent (v06)
- t_benchmark f — pinneapple_systems.digital_twin.pipe_loop.agent (v06)
- t_build_twin f — pinneapple_systems.digital_twin.pipe_loop.agent (v06)
- t_calibrate f — pinneapple_systems.digital_twin.pipe_loop.agent (v06)
- t_deliver f — pinneapple_design.thermal.board_cooling.agent (v06)
- t_deliver f — pinneapple_systems.digital_twin.pipe_loop.agent (v06)
- t_junction f — pinneapple_design.geometry.csg (v06)
- t_learn f — pinneapple_design.thermal.board_cooling.agent (v06)
- t_monitor f — pinneapple_systems.digital_twin.pipe_loop.agent (v06)
- t_optimize f — pinneapple_design.thermal.board_cooling.agent (v06)
- t_read_mission f — pinneapple_design.thermal.board_cooling.agent (v06)
- t_read_mission f — pinneapple_systems.digital_twin.pipe_loop.agent (v06)
- t_request_simulation f — pinneapple_systems.digital_twin.pipe_loop.agent (v06)
- t_sample f — pinneapple_design.thermal.board_cooling.agent (v06)
- t_screen_natural f — pinneapple_design.thermal.board_cooling.agent (v06)
- t_trust f — pinneapple_systems.digital_twin.pipe_loop.agent (v06)
- t_verify f — pinneapple_design.thermal.board_cooling.agent (v06)
- Table C — pinneapple_data.contracts.models (v04)
- Table C — pinneapple_tools.reporting.datasheet (v07)
- table f — pinneapple_hub.library.categories (v07)
- table f — pinneapple_science.report (v02)

- table_boxes f — pinneapple_design.bricks.dynamics (v06)
- table_evaluator f — pinneapple_design.design_optimizer.report (v06)
- table_format f — pinneapple_data.contracts.tables (v04)
- table_markdown f — pinneapple_simulation.flight.figures (v03)
- TableIronLoss C — pinneapple_simulation.electromagnetics.machines.steel (v03)
- tables_to_csv f — pinneapple_veriphysics.validation_lab (v05)
- TableSpec C — pinneapple_data.formfill.spec (v04)
- TabulatedModel C — pinneapple_analysis.inverse_problems.forward (v05)
- TabulatedSignal C — pinneapple_physics.pinn_solver.factory.signals (v02)
- tag_names f — pinneapple_core.onboarding.catalog (v01)
- tag_solve_kwargs f — pinneapple_physics.pde_environment.presets.tag_geometry (v02)
- TagConditionsUnresolved C — pinneapple_physics (v02)
- tagged_preset f — pinneapple_flow.nodes_ext.pde_env (v07)
- TagHeuristics C — pinneapple_design.geometry.builders.stl_domain_batch_builder (v06)
- tai_minus_utc f — pinneapple_physics.astro.timescales (v02)
- takeoff f — pinneapple_flow.nodes.systems (v07)
- takeoff_cases f — pinneapple_flow.nodes_ext.flight (v07)
- takeoff_derivatives f — pinneapple_simulation.flight.takeoff (v03)
- takeoff_figure f — pinneapple_simulation.flight.figures (v03)
- takeoff_metrics f — pinneapple_simulation.flight.takeoff (v03)
- TakeoffPilot C — pinneapple_simulation.flight.control (v03)
- TakeoffSchedule C — pinneapple_simulation.flight.control (v03)
- tangential_continuity f — pinneapple_physics.maxwell_checks (v02)
- tank_balance_rules f — pinneapple_systems.operations.anomaly (v06)
- tank_constants f — pinneapple_systems.operations.anomaly (v06)
- tank_rhs f — pinneapple_systems.operations.plant (v06)
- tank_step f — pinneapple_systems.operations.plant (v06)
- TankParams C — pinneapple_systems.operations.plant (v06)
- TankShellVanillaPINN C — pinneapple_systems.component_library.components (v06)
- TankSimulator C — pinneapple_systems.operations.plant (v06)
- target_altaz f — pinneapple_physics.astro.observing (v02)
- target_margin f — pinneapple_reason.modes._util (v05)
- targets_at f — pinneapple_systems.time_series.forecast_lab.base (v06)
- tarjan_scc f — pinneapple_systems.cosimulation.utils (v06)
- TaskDistribution C — pinneapple_worldmodel.meta_learning (v04)
- TaskResult C — pinneapple_llm.workbench.evaluate (v07)
- TaskResult C — pinneapple_tools.benchmark_suite.tasks.base (v07)
- taylor_green_ic f — pinneapple_simulation.numerical_solvers.ns3d_spectral (v03)
- taylor_green_velocity_fns f — pinneapple_simulation.numerical_solvers.jax_cfd_solver (v03)
- TC38Geometry C — pinneapple_design.geometry.gen.threaded_connection (v06)
- TC46Geometry C — pinneapple_design.geometry.gen.threaded_connection (v06)
- TC50Geometry C — pinneapple_design.geometry.gen.threaded_connection (v06)
- TCDACell C — pinneapple_simulation.electromagnetics.periodic.tcda (v03)
- TCN C — pinneapple_neural.architectures.classical_ts.tcn (v04)
- TCNConfig C — pinneapple_systems.time_series.models.tcn (v06)
- TCNForecaster C — pinneapple_systems.time_series.models.tcn (v06)
- tdb_minus_tt f — pinneapple_physics.astro.timescales (v02)
- teff_from_bv f — pinneapple_physics.astro.stars (v02)
- TelemetryConditioner C — pinneapple_systems.digital_twin.conditioning (v06)
- Telescope C — pinneapple_physics.astro.observing (v02)
- temperature_ratio f — pinneapple_physics.closed_form.rocket_nozzle (v02)
- Template C — pinneapple_core.workflows.templates (v01)
- Template C — pinneapple_design.fitmesh.catalog (v06)
- template f — pinneapple_flow.templates (v07)
- template_doc f — pinneapple_flow.planner.extensions (v07)
- template_plan f — pinneapple_design.thermal.board_cooling.agent (v06)

- `template_plan` f — `pinneapple_systems.digital_twin.pipe_loop.agent` (v06)
- `TemplatePlanner` C — `pinneapple_autolab.core` (v07)
- `templates` f — `pinneapple_design.intent.parse` (v06)
- `temporal_split` f — `pinneapple_systems.time_series.tuning.optuna_tuner` (v06)
- `TemporalFusionTransformer` C — `pinneapple_neural.architectures.transformers.tft` (v04)
- `tensor_grid` f — `pinneapple_physics.paper_problems` (v02)
- `TensorGrid` C — `pinneapple_science.numerics` (v02)
- `term_dimension` f — `pinneapple_simulation.electromagnetics.units` (v03)
- `terminal_velocity` f — `pinneapple_simulation.numerical_solvers.particles` (v03)
- `TerminalNode` C — `pinneapple_decision.tree` (v05)
- `terrain_at` f — `pinneapple_simulation.vehicle.viz` (v03)
- `terrain_colors` f — `pinneapple_simulation.rover.autolab.render` (v03)
- `terrain_grid` f — `pinneapple_simulation.rover.autolab.render` (v03)
- `terrain_height` f — `pinneapple_tools.visualization.studio.planetary` (v07)
- `terrain_self_shadow` f — `pinneapple_tools.visualization.studio.shadows` (v07)
- `TerramechanicsResiduals` C — `pinneapple_physics.pde_environment.presets.terramechanics` (v02)
- `test_points` f — `pinneapple_tools.benchmark_suite.crossframework.problems` (v07)
- `TestbedData` C — `pinneapple_simulation.terramechanics.testbed` (v03)
- `tet_boundary_faces` f — `pinneapple_simulation.external_solvers.cfd_formats.convert` (v03)
- `tetramer_threshold_branches` f — `pinneapple_analysis.proof.pt_chains` (v05)
- `text` f — `pinneapple_check.report` (v05)
- `text_of` f — `pinneapple_reason.chain` (v05)
- `TextChunk` C — `pinneapple_tools.hpo_experiments.kb.chunking` (v07)
- `tf` f — `pinneapple_design.geometry_library.kit` (v06)
- `tfc_available` f — `pinneapple_neural.architectures.pinns.xtfc` (v04)
- `TFCConstrainedExpression` C — `pinneapple_neural.architectures.pinns.xtfc` (v04)
- `TFTConfig` C — `pinneapple_systems.time_series.models.tft` (v06)
- `TFTForecaster` C — `pinneapple_systems.time_series.models.tft` (v06)
- `theorem_profile` f — `pinneapple_physics.non_hermitian.ep_tuning` (v02)
- `thermal_case` f — `pinneapple_condense.falsify` (v02)
- `thermal_field` f — `pinneapple_design.surrogate_inverse.brake_rotor` (v06)
- `thermal_field` f — `pinneapple_design.surrogate_inverse.heatsink_fins` (v06)
- `thermal_field` f — `pinneapple_design.thermal.fin_array_speed` (v06)
- `thermal_image` f — `pinneapple_worldmodel.inspection_data` (v04)
- `thermal_plate` f — `pinneapple_worldmodel.inspection_data` (v04)
- `thermal_resistance_k_w` f — `pinneapple_design.surrogate_inverse.brake_rotor` (v06)
- `thermal_resistance_k_w` f — `pinneapple_design.surrogate_inverse.heatsink_fins` (v06)
- `ThermalChannel2D` C — `pinneapple_simulation.numerical_solvers.thermal_channel_2d` (v03)
- `ThermalCompliance2D` C — `pinneapple_design.topology.problems` (v06)
- `ThermalEfficiencyObjective` C — `pinneapple_design.design_optimizer.objective` (v06)
- `ThermalNetwork` C — `pinneapple_simulation.electromagnetics.machines.thermal` (v03)
- `thermoelasticity_2d_default` f — `pinneapple_physics.pde_environment.presets.structural` (v02)
- `Theta` C — `pinneapple_systems.time_series.adaptive` (v06)
- `theta_soil` f — `pinneapple_simulation.rover.autolab.estimation` (v03)
- `thick_te` f — `pinneapple_design.geometry_library.kit` (v06)
- `thick_walled_cylinder_lame_default` f — `pinneapple_physics.pde_environment.presets.solid_mechanics` (v02)
- `thin_airfoil_cl` f — `pinneapple_physics.closed_form.thin_airfoil` (v02)
- `thin_airfoil_zero_lift` f — `pinneapple_presets.cases.airfoil` (v02)
- `threaded_coupling_tc50_box_default` f — `pinneapple_physics.pde_environment.presets.solid_mechanics` (v02)
- `threaded_coupling_tc50_pin_default` f — `pinneapple_physics.pde_environment.presets.solid_mechanics` (v02)
- `threaded_coupling_tc50_rotating_default` f — `pinneapple_physics.pde_environment.presets.solid_mechanics` (v02)
- `ThreadedConnectionMesher` C — `pinneapple_design.geometry.gen.threaded_connection` (v06)
- `ThreadProfile` C — `pinneapple_design.geometry.gen.threaded_connection` (v06)

- threejs_config f — pinneapple_tools.visualization.studio.lighting (v07)
- threshold_coefficient f — pinneapple_simulation.numerical_solvers.operator_benchmarks (v03)
- threshold_numeric f — pinneapple_analysis.proof.pt_chains (v05)
- ThresholdDetector C — pinneapple_systems.digital_twin.monitoring.anomaly (v06)
- ThresholdVerifier C — pinneapple_decision.verifiers (v05)
- ThroughputMonitor C — pinneapple_neural.trainer.parallel (v04)
- thrust_coefficient f — pinneapple_physics.closed_form.rocket_nozzle (v02)
- thrust_direction_body f — pinneapple_simulation.flight.propulsion (v03)
- thumbnail f — pinneapple_design.geometry_library (v06)
- thumbnail_bytes f — pinneapple_lab.html (v07)
- thumbnail_path f — pinneapple_design.geometry_library.store (v06)
- ticks f — pinneapple_tools.visualization.studio.colormap (v07)
- TikhonovRegularizer C — pinneapple_analysis.inverse_problems.regularization (v05)
- Tile C — pinneapple_lab.showcase (v07)
- Tile C — pinneapple_tools.visualization.dashboard.spec (v07)
- Time C — pinneapple_physics.astro.timescales (v02)
- time_average f — pinneapple_simulation.numerical_solvers.free_surface (v03)
- time_dependent_loss f — pinneapple_quantum.loss.schrodinger (v04)
- time_derivative f — pinneapple_physics.pinn_solver.compiler.autograd_ops (v02)
- time_directories f — pinneapple_twin3d.openfoam (v06)
- time_dirs f — pinneapple_data.cae.foam (v04)
- time_dirs f — pinneapple_design.aero.case (v06)
- time_dirs f — pinneapple_simulation.accelerated_foam.case (v03)
- time_per_query f — pinneapple_analysis.cost.equal_budget (v05)
- time_reversal_asymmetry f — pinneapple_systems.time_series.predictability (v06)
- time_sequence f — pinneapple_physics.paper_problems (v02)
- time_series_cv f — pinneapple_data.process_optimizer (v04)
- time_step f — pinneapple_physics.blackhole.hydro_numba (v02)
- time_surrogate f — pinneapple_design.thermal.fin_array_speed (v06)
- time_to_intercept f — pinneapple_systems.sports.football.pitch_control (v06)
- time_to_reach_region f — pinneapple_analysis.uncertainty.reachability (v05)
- TimeCap C — pinneapple_tools.benchmark_suite.crossframework.records (v07)
- TimeHarmonicSolution C — pinneapple_simulation.electromagnetics.eddy (v03)
- timeline_figure f — pinneapple_reason.report (v05)
- timeline_scene f — pinneapple_design.bricks.visuals (v06)
- timeline_scene f — pinneapple_design.thermal.board_cooling.scene (v06)
- TimeMarchingTrainer C — pinneapple_neural.trainer.time_marching (v04)
- Timer C — pinneapple_arena.external.result (v04)
- timeseries_spec f — pinneapple_flow.nodes_ext.arena_pipelines (v07)
- TimeSeriesAdapter C — pinneapple_neural.architectures.adapters.ts (v04)
- TimeSeriesBenchmarkSpec C — pinneapple_tools.benchmark_suite.timeseries_pipeline (v07)
- TimeSeriesCoSimNode C — pinneapple_systems.cosimulation.adapters (v06)
- TimeSeriesImputer C — pinneapple_systems.time_series.preparation.imputer (v06)
- TimeSeriesModelBase C — pinneapple_neural.architectures.transformers.base (v04)
- TimeSeriesResampler C — pinneapple_systems.time_series.preparation.resampler (v06)
- TimeSeriesScaler C — pinneapple_systems.time_series.csv_loader (v06)
- TimeSeriesSpec C — pinneapple_systems.time_series.spec (v06)
- TimesFMForecaster C — pinneapple_systems.time_series.foundation_models (v06)
- TimesNet C — pinneapple_neural.architectures.transformers.timesnet (v04)
- timestep_check f — pinneapple_simulation.rover.autolab.verify (v03)
- tip_mach_number f — pinneapple_systems.process_components.similarity_map (v06)
- tip_over_offset f — pinneapple_design.bricks.closed_form (v06)
- tip_speed_m_s f — pinneapple_systems.process_components.similarity_map (v06)
- tipping_report f — pinneapple_design.bricks.statics (v06)
- tisserand f — pinneapple_physics.astro.neural_propagator (v02)
- TJunctionDomain2D C — pinneapple_design.geometry.gen.domains (v06)

- `tm_cavity_mode` f — `pinneapple_physics.closed_form.electromagnetics` (v02)
- `tmax_cells` f — `pinneapple_condense.substructure` (v02)
- `to_aas` f — `pinneapple_systems.digital_twin.standards` (v06)
- `to_arrays` f — `pinneapple_simulation.electromagnetics.periodic.benchmark` (v03)
- `to_dataframe` f — `pinneapple_analysis.benchmark.report` (v05)
- `to_dbi` f — `pinneapple_simulation.electromagnetics.antenna.bounds` (v03)
- `to_device_sample` f — `pinneapple_data.device` (v04)
- `to_ditto_thing` f — `pinneapple_systems.digital_twin.standards` (v06)
- `to_dtdl` f — `pinneapple_systems.digital_twin.standards` (v06)
- `to_dxf` f — `pinneapple_simulation.electromagnetics.machines.interop` (v03)
- `to_exportable` f — `pinneapple_neural.architectures.neural_operators.fno_export` (v04)
- `to_glb` f — `pinneapple_design.aero.airframe` (v06)
- `to_glb` f — `pinneapple_design.geometry_library.exporters` (v06)
- `to_glb` f — `pinneapple_tools.visualization.molecular` (v07)
- `to_json` f — `pinneapple_check.report` (v05)
- `to_json` f — `pinneapple_flow.identify` (v07)
- `to_json` f — `pinneapple_simulation.rover.autolab.mission` (v03)
- `to_jsonable` f — `pinneapple_core.jsonable` (v01)
- `to_ldr` f — `pinneapple_design.bricks.ldraw` (v06)
- `to_markdown` f — `pinneapple_simulation.electromagnetics.verification` (v03)
- `to_mat` f — `pinneapple_simulation.electromagnetics.machines.interop` (v03)
- `to_mtl` f — `pinneapple_design.geometry_library.exporters` (v06)
- `to_ngsi_ld` f — `pinneapple_systems.digital_twin.standards` (v06)
- `to_obj` f — `pinneapple_design.geometry_library.exporters` (v06)
- `to_parameter_space` f — `pinneapple_analysis.inverse_problems.contracts` (v05)
- `to_parameter_space` f — `pinneapple_design.design_optimizer.report` (v06)
- `to_physical_sample` f — `pinneapple_data.synth.sample_adapter` (v04)
- `to_physical_sample` f — `pinneapple_data.unified` (v04)
- `to_problemdesign_dict` f — `pinneapple_tools.hpo_experiments.pipelines.export_problemdesign` (v07)
- `to_pyleecan` f — `pinneapple_simulation.electromagnetics.machines.interop` (v03)
- `to_rgb` f — `pinneapple_science.plots` (v02)
- `to_series` f — `pinneapple_systems.time_series.enso.dataset` (v06)
- `to_si` f — `pinneapple_design.intent.spec` (v06)
- `to_si` f — `pinneapple_physics.pde_environment.presets.astropy_bridge` (v02)
- `to_signal_batch` f — `pinneapple_simulation.numerical_solvers.adapters.upd_signal` (v03)
- `to_step` f — `pinneapple_design.geometry_library.exporters` (v06)
- `to_stl` f — `pinneapple_design.aero.airframe` (v06)
- `to_stl` f — `pinneapple_design.geometry_library.exporters` (v06)
- `to_stl` f — `pinneapple_design.surrogate_inverse.meshes` (v06)
- `to_study_result` f — `pinneapple_tools.automation.report` (v07)
- `to_study_result` f — `pinneapple_veriphysics.evidence_package` (v05)
- `to_toml` f — `pinneapple_veriphysics.validation_lab` (v05)
- `to_usda` f — `pinneapple_design.aero.airframe` (v06)
- `to_usda` f — `pinneapple_design.geometry_library.exporters` (v06)
- `to_values` f — `pinneapple_analysis.uncertainty.distributions` (v05)
- `to_workflow` f — `pinneapple_flow.runtime` (v07)
- `toa_insolation` f — `pinneapple_physics.weather.model` (v02)
- `tokenize` f — `pinneapple_tools.hpo_experiments.ranking` (v07)
- `tolerance_model` f — `pinneapple_simulation.electromagnetics.machines.robustness` (v03)
- `tolerance_verdicts` f — `pinneapple_data.cae.compare` (v04)
- `ToleranceResult` C — `pinneapple_analysis.uncertainty.tolerance` (v05)
- `Tolerances` C — `pinneapple_veriphysics.applicability` (v05)
- `tone_map` f — `pinneapple_physics.astro.lensing` (v02)
- `tonight` f — `pinneapple_physics.astro.facts` (v02)
- `Tool` C — `pinneapple_autolab.core` (v07)
- `Toolbox` C — `pinneapple_autolab.core` (v07)

- Toolbox C — pinneapple_systems.component_library.toolbox (v06)
- toolbox f — pinneapple_design.thermal.board_cooling.agent (v06)
- toolbox f — pinneapple_systems.digital_twin.pipe_loop.agent (v06)
- ToolboxRegistry C — pinneapple_systems.component_library.toolbox (v06)
- ToolCall C — pinneapple_llm.workbench.backend (v07)
- Toolchain C — pinneapple_tools.automation.toolchain (v07)
- ToolError C — pinneapple_llm.workbench.tools (v07)
- ToolRecommendation C — pinneapple_analysis.verification.tool_recommendation (v05)
- top_load f — pinneapple_design.bricks.statics (v06)
- topic_matches f — pinneapple_systems.digital_twin.io.mqtt_broker (v06)
- topological_sort f — pinneapple_systems.cosimulation.utils (v06)
- topology f — pinneapple_design.aero.mesh (v06)
- topology2d f — pinneapple_flow.nodes_ext.design (v07)
- topology_gradcheck f — pinneapple_flow.nodes_ext.design (v07)
- TopOptResult C — pinneapple_design.topology.driver (v06)
- torch_filter f — pinneapple_physics.multiscale.learned (v02)
- torch_from_handles f — pinneapple_design.aero.geometry_onnx (v06)
- torch_geometry f — pinneapple_design.aero.geometry_onnx (v06)
- torch_pcg f — pinneapple_simulation.accelerated_foam.solvers (v03)
- torch_refilter f — pinneapple_physics.multiscale.learned (v02)
- torch_transfer f — pinneapple_physics.multiscale.learned (v02)
- TorchModuleEvaluator C — pinneapple_design.design_optimizer.evaluators (v06)
- TorchNode C — pinneapple_systems.cosimulation.node (v06)
- TorchPhysicsBackend C — pinneapple_core.backend (v01)
- TorchRunConfig C — pinneapple_neural.trainer.hpc (v04)
- TorchVorticityLES2D C — pinneapple_physics.multiscale.learned (v02)
- torque_coenergy f — pinneapple_simulation.electromagnetics.postprocess (v03)
- torque_free_spin f — pinneapple_simulation.external_solvers.newton_physics.validation (v03)
- torque_maxwell_stress f — pinneapple_simulation.electromagnetics.postprocess (v03)
- torque_maxwell_stress_gradient f — pinneapple_simulation.electromagnetics.postprocess (v03)
- torque_trajectory f — pinneapple_simulation.robot_sim.sim (v03)
- torsion_optimizer f — pinneapple_flow.nodes_ext.core_ts (v07)
- torsional_shear_stress f — pinneapple_systems.process_components.pipe_stress_mechanics (v06)
- torus f — pinneapple_design.geometry_library.kit (v06)
- torus3d f — pinneapple_design.geometry.gen.sdf_shapes (v06)
- torus_graph f — pinneapple_physics.multiscale.learned (v02)
- torus_state f — pinneapple_physics.blackhole.hydro (v02)
- TotalVariationRegularizer C — pinneapple_analysis.inverse_problems.regularization (v05)
- tower_tip_force f — pinneapple_design.bricks.closed_form (v06)
- TowerGoal C — pinneapple_design.bricks.goals (v06)
- tr f — pinneapple_reason.chain (v05)
- trace_snapshots f — pinneapple_condense.substructure (v02)
- trace_thin_disc f — pinneapple_physics.astro.lensing (v02)
- track_meshes f — pinneapple_design.intent.physics.loop_track (v06)
- track_object f — pinneapple_worldmodel.physical_world (v04)
- track_setpoint f — pinneapple_systems.time_series.eda.plots (v06)
- TrackingReport C — pinneapple_systems.digital_twin.product.evidence (v06)
- traction_monitor f — pinneapple_simulation.rover.autolab.mission (v03)
- tradeoff_map f — pinneapple_simulation.electromagnetics.reflector.study (v03)
- train f — pinneapple_design.thermal.heatsink.surrogate (v06)
- train f — pinneapple_physics.weather.train (v02)
- train_acoustic f — pinneapple_analysis.inspection.acoustic (v05)
- train_and_predict f — pinneapple_neural (v04)
- train_closed_form_model f — pinneapple_arena.pointwise.training (v04)
- train_closure_aposteriori f — pinneapple_physics.multiscale.learned (v02)
- train_component_task f — pinneapple_orchestration.tasks.training_tasks (v01)

- `train_contact_surrogate` f — `pinneapple_simulation.terramechanics.surrogate` (v03)
- `train_decay_pinn` f — `pinneapple_physics.astro.decay` (v02)
- `train_dense_volume_operator` f — `pinneapple_neural.workflows.dense_volume_operator` (v04)
- `train_discrete_time` f — `pinneapple_simulation.numerical_solvers.discrete_time_pinn` (v03)
- `train_dynamics` f — `pinneapple_neural.architectures.graphnn.mgn_dynamics` (v04)
- `train_eddy_current` f — `pinneapple_analysis.inspection.eddy_current` (v05)
- `train_emat` f — `pinneapple_analysis.inspection.emat` (v05)
- `train_end` f — `pinneapple_systems.time_series.forecast_lab.base` (v06)
- `train_field_pinn` f — `pinneapple_analysis.benchmark.builtin.fields` (v05)
- `train_forecaster` f — `pinneapple_physics.astro.spaceweather` (v02)
- `train_forecaster` f — `pinneapple_physics.blackhole.forecast` (v02)
- `train_foundation` f — `pinneapple_worldmodel.train` (v04)
- `train_generic_inspection_pinn` f — `pinneapple_analysis.inspection.base` (v05)
- `train_gnn` f — `pinneapple_design.aero.train` (v06)
- `train_graph` f — `pinneapple_arena.trainer` (v04)
- `train_graybox` f — `pinneapple_neural.trainer.graybox` (v04)
- `train_grid_operator` f — `pinneapple_arena.trainer` (v04)
- `train_grid_operator` f — `pinneapple_neural.physicsnemo_bridge.field_operators` (v04)
- `train_guided_wave_ultrasonic` f — `pinneapple_analysis.inspection.guided_wave_ultrasonic` (v05)
- `train_magnetic_flux_leakage` f — `pinneapple_analysis.inspection.magnetic_flux_leakage` (v05)
- `train_meta` f — `pinneapple_worldmodel.train` (v04)
- `train_mgn` f — `pinneapple_analysis.benchmark.builtin.airfoil_impl` (v05)
- `train_mlp` f — `pinneapple_analysis.benchmark.builtin.airfoil_impl` (v05)
- `train_mlp` f — `pinneapple_design.aero.train` (v06)
- `train_mlp_fields` f — `pinneapple_analysis.benchmark.builtin.airfoil_impl` (v05)
- `train_mlx` f — `pinneapple_llm.workbench.finetune` (v07)
- `train_model` f — `pinneapple_neural` (v04)
- `train_monotone` f — `pinneapple_design.thermal.heatsink.monotone` (v06)
- `train_nozzle_pinn` f — `pinneapple_analysis.verification.nozzle_crosscheck` (v05)
- `train_on_trajectories` f — `pinneapple_physics.blackhole.forecast` (v02)
- `train_operator` f — `pinneapple_analysis.benchmark.builtin.fields` (v05)
- `train_operator` f — `pinneapple_tools.benchmark_suite.crossframework.darcy` (v07)
- `train_orbit_pinn` f — `pinneapple_physics.astro.surrogate` (v02)
- `train_parametric_dense_volume_operator` f — `pinneapple_neural.workflows.parametric_dense_volume_operator` (v04)
- `train_peft` f — `pinneapple_llm.workbench.finetune` (v07)
- `train_phased_array_ultrasonic` f — `pinneapple_analysis.inspection.phased_array_ultrasonic` (v05)
- `train_pinn` f — `pinneapple_analysis.benchmark.builtin.airfoil_impl` (v05)
- `train_pinn` f — `pinneapple_analysis.validation.pinn` (v05)
- `train_pinn` f — `pinneapple_arena.trainer` (v04)
- `train_pinn` f — `pinneapple_simulation.meshless.pinn` (v03)
- `train_specialist` f — `pinneapple_worldmodel.train` (v04)
- `train_supervised` f — `pinneapple_arena.trainer` (v04)
- `train_surrogate` f — `pinneapple_analysis.benchmark.builtin.inverse_heat_impl` (v05)
- `train_surrogate` f — `pinneapple_design.design_optimizer.airfoil_drag` (v06)
- `train_surrogate` f — `pinneapple_design.surrogate_inverse.brake_rotor` (v06)
- `train_surrogate` f — `pinneapple_design.surrogate_inverse.heatsink_fins` (v06)
- `train_surrogate` f — `pinneapple_design.surrogate_inverse.reflex_port` (v06)
- `train_surrogate` f — `pinneapple_design.surrogate_inverse.suspension_arm` (v06)
- `train_surrogate` f — `pinneapple_design.surrogate_inverse.transdermal_patch` (v06)
- `train_surrogate` f — `pinneapple_design.thermal.fin_array_speed` (v06)
- `train_surrogates` f — `pinneapple_design.aero.train` (v06)
- `train_torch_model` f — `pinneapple_arena.pointwise.training` (v04)
- `train_torch_regressor` f — `pinneapple_systems.time_series.forecast_lab.learned` (v06)
- `train_wheel_surrogate` f — `pinneapple_simulation.terramechanics.mobility` (v03)
- `train_within_budget` f — `pinneapple_analysis.cost.equal_budget` (v05)

- trainable f — pinneapple_core.onboarding.catalog (v01)
- TrainConfig C — pinneapple_llm.workbench.finetune (v07)
- TrainConfig C — pinneapple_neural.trainer.trainer (v04)
- TrainConfig C — pinneapple_physics.blackhole.forecast (v02)
- TrainContext C — pinneapple_analysis.surrogate.methods (v05)
- Trainer C — pinneapple_neural.trainer.trainer (v04)
- training_box f — pinneapple_simulation.electromagnetics.antenna.design (v03)
- training_distribution f — pinneapple_analysis.validation.physics_learning_cases (v05)
- training_gates f — pinneapple_analysis.benchmark.checks (v05)
- TrainingAdvisor C — pinneapple_neural.trainer.advisor (v04)
- TrainingConfig C — pinneapple_arena.config (v04)
- TrainingHooks C — pinneapple_worldmodel.training_hooks (v04)
- TrainingResult C — pinneapple_neural.physicsnemo_bridge.field_operators (v04)
- TrainingStrategySelector C — pinneapple_decision.selectors (v05)
- TrainResult C — pinneapple_arena.trainer (v04)
- trajectories_to_upd f — pinneapple_simulation.external_solvers.mujoco.adapter (v03)
- Trajectory C — pinneapple_physics.astro.smallbodies (v02)
- Trajectory C — pinneapple_simulation.rover.dynamics (v03)
- Trajectory C — pinneapple_systems.cosimulation.recorder (v06)
- trajectory_to_upd f — pinneapple_simulation.external_solvers.mujoco.adapter (v03)
- TrajectoryData C — pinneapple_worldmodel.simulator (v04)
- TrajectoryRecorder C — pinneapple_systems.cosimulation.recorder (v06)
- TrajectorySampler C — pinneapple_core.data (v01)
- transaction f — pinneapple_core.service.db (v01)
- transfer f — pinneapple_flow.nodes.methods (v07)
- transfer f — pinneapple_flow.nodes_ext.benchmarks (v07)
- transfer f — pinneapple_physics.multiscale.filters (v02)
- TransferBenchmarkConfig C — pinneapple_tools.benchmark_suite.transfer_benchmark (v07)
- TransferBenchmarkPipeline C — pinneapple_tools.benchmark_suite.transfer_benchmark (v07)
- TransferBenchmarkResult C — pinneapple_tools.benchmark_suite.transfer_benchmark (v07)
- TransferConfig C — pinneapple_adaptation.transfer_learning.config (v04)
- TransferScenario C — pinneapple_tools.benchmark_suite.transfer_benchmark (v07)
- TransferTrainer C — pinneapple_adaptation.transfer_learning.trainer (v04)
- Transform C — pinneapple_core.transforms (v01)
- transform f — pinneapple_twin3d.primitives (v06)
- transform_coordinates f — pinneapple_physics.pde_environment.presets.astropy_bridge (v02)
- TransformedSurrogate C — pinneapple_neural.workflows.hybrid_surrogate_physics (v04)
- TransformerCatalog C — pinneapple_neural.architectures.transformers.registry (v04)
- transient f — pinneapple_condense.figures (v02)
- transient f — pinneapple_reason.modes.fluids (v05)
- transient_heat_3d_default f — pinneapple_physics.pde_environment.presets.industry (v02)
- transient_rhs f — pinneapple_systems.process_components.heat_exchanger (v06)
- transient_study f — pinneapple_condense.benchmark (v02)
- TransientPipe C — pinneapple_systems.process_components.pipe_network_1d (v06)
- TransientResult C — pinneapple_systems.process_components.water_hammer (v06)
- translate f — pinneapple_design.geometry.vehicle_cad (v06)
- Transolver C — pinneapple_neural.architectures.neural_operators.transolver (v04)
- TransolverBlock C — pinneapple_neural.architectures.neural_operators.transolver (v04)
- TransolverLite C — pinneapple_neural.architectures.neural_operators.transolver (v04)
- travel_time f — pinneapple_systems.sports.football.ball (v06)
- travel_times f — pinneapple_physics.astro.journey (v02)
- TreeRun C — pinneapple_decision.tree (v05)
- TreeStep C — pinneapple_decision.tree (v05)
- Trial C — pinneapple_arena.autoresearch (v04)
- TriangleQualityStats C — pinneapple_analysis.verification.mesh_intelligence (v05)
- triangles_to_edges f — pinneapple_neural.architectures.graphnn.mgn_dynamics (v04)

- `trim` f — `pinneapple_simulation.flight.trim` (v03)
- `trim_checks` f — `pinneapple_simulation.flight.verification` (v03)
- `trim_stability` f — `pinneapple_flow.nodes_ext.flight` (v07)
- `TrimeshBridge` C — `pinneapple_design.geometry.io.trimesh_bridge` (v06)
- `TrimResult` C — `pinneapple_simulation.flight.trim` (v03)
- `trough_jet` f — `pinneapple_flow.nodes_ext.cfd` (v07)
- `TroughSection` C — `pinneapple_design.geometry.trough` (v06)
- `true_saving` f — `pinneapple_data.process_optimizer` (v04)
- `true_subfilter` f — `pinneapple_physics.multiscale.closures` (v02)
- `trust` f — `pinneapple_hub.zoo.demo` (v07)
- `trust_card` f — `pinneapple_analysis.benchmark.validation` (v05)
- `trust_card` f — `pinneapple_analysis.trust.field_card` (v05)
- `trust_card` f — `pinneapple_check.evidence` (v05)
- `trust_card` f — `pinneapple_mcp.tools` (v07)
- `trust_card` f — `pinneapple_reason.trust` (v05)
- `trust_card_service` f — `pinneapple_tools.model_services.registry` (v07)
- `trust_horizon` f — `pinneapple_physics.blackhole.forecast` (v02)
- `trust_panels` f — `pinneapple_tools.visualization.trust_panels` (v07)
- `trust_panels_gif` f — `pinneapple_tools.visualization.trust_panels` (v07)
- `trust_report` f — `pinneapple_physics.astro.neural_propagator` (v02)
- `trust_report` f — `pinneapple_physics.darcy_checks` (v02)
- `trust_report` f — `pinneapple_reason` (v05)
- `trust_report` f — `pinneapple_reason.trust` (v05)
- `trust_report` f — `pinneapple_systems.digital_twin.pipe_loop.report` (v06)
- `trust_stages` f — `pinneapple_check._reuse` (v05)
- `trust_stages` f — `pinneapple_lab.curation` (v07)
- `TrustedLatentSurrogate` C — `pinneapple_neural.architectures.rom.trusted` (v04)
- `TrustGate` C — `pinneapple_analysis.trust.trust_gate` (v05)
- `TrustReport` C — `pinneapple_analysis.trust.trust_report` (v05)
- `TrustScore` C — `pinneapple_analysis.trust.trust_gate` (v05)
- `try_parse_unit` f — `pinneapple_data.physical_units` (v04)
- `TSAuditor` C — `pinneapple_systems.time_series.audit.tests` (v06)
- `TSDataModule` C — `pinneapple_systems.time_series.datamodule` (v06)
- `TSFeatureEngineer` C — `pinneapple_systems.time_series.features.engineering` (v06)
- `TSModelBase` C — `pinneapple_systems.time_series.base` (v06)
- `TSModelCatalog` C — `pinneapple_systems.time_series.registry` (v06)
- `TSOutput` C — `pinneapple_neural.architectures.transformers.base` (v04)
- `TSOutput` C — `pinneapple_systems.time_series.base` (v06)
- `tube` f — `pinneapple_design.geometry_library.kit` (v06)
- `tube` f — `pinneapple_presets.viz3d` (v02)
- `tube` f — `pinneapple_systems.digital_twin.pipe_loop.scene` (v06)
- `tube` f — `pinneapple_tools.nvidia.omniverse` (v07)
- `tube_loop` f — `pinneapple_design.geometry_library.kit` (v06)
- `tube_shell` f — `pinneapple_design.geometry.vehicle_cad` (v06)
- `tune` f — `pinneapple_flow.learn` (v07)
- `tune` f — `pinneapple_systems.time_series.forecast_lab.study` (v06)
- `turbodesigner_to_upd` f — `pinneapple_simulation.external_solvers.turbodesigner.bridge` (v03)
- `TurboDesignerConfig` C — `pinneapple_simulation.external_solvers.turbodesigner.bridge` (v03)
- `TurboDesignerWorkflow` C — `pinneapple_simulation.external_solvers.turbodesigner.bridge` (v03)
- `Turbofan` C — `pinneapple_simulation.flight.propulsion` (v03)
- `turbulence_checks` f — `pinneapple_simulation.flight.verification` (v03)
- `turbulence_encounter` f — `pinneapple_flow.nodes_ext.flight` (v07)
- `turbulence_encounter` f — `pinneapple_simulation.flight.encounter` (v03)
- `turbulence_spec` f — `pinneapple_simulation.flight.turbulence` (v03)
- `TurbulenceModel` C — `pinneapple_physics.pde_environment.turbulence_selector` (v02)
- `TurbulenceSpec` C — `pinneapple_simulation.flight.turbulence` (v03)

- turbulent f — pinneapple_physics.physics_audit.assumptions (v02)
- turbulent_flat_plate_cf f — pinneapple_physics.closed_form.thin_airfoil (v02)
- Turn C — pinneapple_problemdesign.state (v02)
- turntable f — pinneapple_design.geometry_library.render (v06)
- turntable_blender f — pinneapple_design.bricks.visuals (v06)
- tutorial_comparison f — pinneapple_simulation.accelerated_foam.benchmark (v03)
- tvd_advection_rhs f — pinneapple_simulation.numerical_solvers.tvd_advection (v03)
- twilight_events f — pinneapple_physics.astro.phenomena (v02)
- twilight_mag f — pinneapple_physics.astro.observing (v02)
- twin f — pinneapple_flow.nodes.systems (v07)
- twin_auto f — pinneapple_flow.nodes.twin_nodes (v07)
- twin_boundary_layer f — pinneapple_simulation.numerical_solvers.gated_xtfc (v03)
- twin_experiment f — pinneapple_analysis.data_assimilation.var4d (v05)
- twin_scene f — pinneapple_simulation.rover.viz (v03)
- twin_study f — pinneapple_systems.operations.contracts (v06)
- TwinDescription C — pinneapple_systems.digital_twin.standards (v06)
- TwinDraftResult C — pinneapple_llm.twin_draft (v07)
- TwinEstimate C — pinneapple_systems.digital_twin.product.assimilation (v06)
- TwinEvidence C — pinneapple_systems.digital_twin.product.evidence (v06)
- TwinMode C — pinneapple_systems.digital_twin.product.policy (v06)
- TwinModel C — pinneapple_systems.digital_twin.standards (v06)
- TwinModelError C — pinneapple_systems.digital_twin.product.model (v06)
- TwinProperty C — pinneapple_systems.digital_twin.standards (v06)
- TwinRuntime C — pinneapple_systems.operations.twin (v06)
- TwinSpec C — pinneapple_systems.digital_twin.product.spec (v06)
- TwinStore C — pinneapple_systems.operations.twin (v06)
- two_body_j2_rhs f — pinneapple_physics.astro.conjunction (v02)
- two_dielectric_capacitor f — pinneapple_physics.closed_form.electromagnetics (v02)
- two_loop_network f — pinneapple_simulation.numerical_solvers.pipe_network (v03)
- TwoLayerEBM C — pinneapple_simulation.geophysics.energy_balance (v03)
- TwoLinkArm C — pinneapple_simulation.robot_sim.arms (v03)
- TwoPhaseConfig C — pinneapple_neural.trainer.two_phase (v04)
- TwoPhaseHistory C — pinneapple_neural.trainer.two_phase (v04)
- TwoPhaseTrainer C — pinneapple_neural.trainer.two_phase (v04)

U

- u_exact f — pinneapple_analysis.inverse_problems.heat_diffusivity (v05)
- UDEForecaster C — pinneapple_systems.time_series.forecast_lab.hybrid (v06)
- Uncertain C — pinneapple_design.intent.template (v06)
- UncertainParameter C — pinneapple_analysis.uncertainty.distributions (v05)
- uncertainty_checks f — pinneapple_reason.trust (v05)
- uncertainty_result f — pinneapple_analysis.study_contracts (v05)
- uncertainty_section f — pinneapple_app.backend.core.surrogate_report (v07)
- uncertainty_study f — pinneapple_analysis.uncertainty.study (v05)
- UncertaintyReport C — pinneapple_analysis.verification.uncertainty_inverse (v05)
- unfreeze_layers f — pinneapple_adaptation.transfer_learning.freeze (v04)
- UnifiedAgentResult C — pinneapple_problemdesign.unified_agent (v02)
- UnifiedPhysicsAgent C — pinneapple_problemdesign.unified_agent (v02)
- uniform_package f — pinneapple_condense.package (v02)
- Union C — pinneapple_worldmodel.geometry (v04)
- union f — pinneapple_design.geometry_library.meshops (v06)
- Unit C — pinneapple_data.physical_units (v04)
- Unit C — pinneapple_llm.qa.units (v07)
- unit_sample f — pinneapple_analysis.uncertainty.distributions (v05)
- UnitBelief C — pinneapple_simulation.rover.autolab.estimation (v03)
- UnitError C — pinneapple_data.physical_units (v04)

- `UnitError` C — `pinneapple_llm.qa.units` (v07)
- `UnitProblem` C — `pinneapple_simulation.electromagnetics.units` (v03)
- `Units` C — `pinneapple_simulation.numerical_solvers.lbm_tunnel` (v03)
- `units` f — `pinneapple_check._reuse` (v05)
- `UnitTables` C — `pinneapple_simulation.rover.autolab.planner` (v03)
- `UniversalUNO` C — `pinneapple_neural.architectures.neural_operators.uno` (v04)
- `UnnormModel` C — `pinneapple_neural.trainer.two_phase` (v04)
- `UNONdGrid` C — `pinneapple_neural.architectures.neural_operators.uno` (v04)
- `UNONdPoints` C — `pinneapple_neural.architectures.neural_operators.uno` (v04)
- `UnresolvedLimitError` C — `pinneapple_autolab.mission` (v07)
- `UnsafeArchive` C — `pinneapple_security.archives` (v05)
- `UnsafeCheckpoint` C — `pinneapple_check.models` (v05)
- `UnsafeModelError` C — `pinneapple_security.model_security` (v05)
- `UnsafeURL` C — `pinneapple_security.web` (v05)
- `UnscentedKalmanFilter` C — `pinneapple_neural.architectures.classical_ts.ukf` (v04)
- `unwrap` f — `pinneapple_app.backend.core.physics_adapter` (v07)
- `unwrap_model` f — `pinneapple_neural.trainer.parallel` (v04)
- `UpBlockND` C — `pinneapple_neural.architectures.neural_operators.uno` (v04)
- `upd_to_physical_sample` f — `pinneapple_data.adapters.upd_adapter` (v04)
- `update_leaderboard` f — `pinneapple_tools.benchmark_suite.runner.leaderboard` (v07)
- `UPDDataset` C — `pinneapple_data.upd_dataset` (v04)
- `UPDDataset` C — `pinneapple_physics.pinn_solver.io.upd_dataset` (v02)
- `UPDItem` C — `pinneapple_data.upd_types` (v04)
- `UPDZarrShardedWriter` C — `pinneapple_data.zarr_shards` (v04)
- `UPDZarrStore` C — `pinneapple_data.zarr_store` (v04)
- `UploadRejected` C — `pinneapple_core.service.uploads` (v01)
- `UploadStore` C — `pinneapple_core.service.uploads` (v01)
- `UploadTooLarge` C — `pinneapple_core.service.uploads` (v01)
- `upsample_sphere` f — `pinneapple_physics.weather.viz` (v02)
- `uq_predict` f — `pinneapple_analysis.uncertainty.core` (v05)
- `UQConfig` C — `pinneapple_arena.config` (v04)
- `UQResult` C — `pinneapple_analysis.uncertainty.core` (v05)
- `UsageLimitExceeded` C — `pinneapple_core.service.metering` (v01)
- `UsageMeter` C — `pinneapple_core.service.metering` (v01)
- `usd_validators` f — `pinneapple_tools.nvidia.omniverse` (v07)
- `use_backend` f — `pinneapple_core.backend` (v01)
- `use_cfd_style` f — `pinneapple_tools.visualization.style` (v07)
- `use_superlu` f — `pinneapple_simulation.electromagnetics.antenna.ava` (v03)
- `use_verdicts` f — `pinneapple_physics.darcy_checks` (v02)
- `used_by_examples` f — `pinneapple_hub.library.sources` (v07)
- `UseProfile` C — `pinneapple_physics.darcy_checks` (v02)
- `user_decision` f — `pinneapple_flow.decisions` (v07)
- `uses_cfd_approach` f — `pinneapple_problemdesign.schema` (v02)
- `uses_pinn_approach` f — `pinneapple_problemdesign.schema` (v02)
- `UseScope` C — `pinneapple_veriphysics.scope` (v05)
- `USGSDataError` C — `pinneapple_systems.digital_twin.io.usgs` (v06)
- `utc_now` f — `pinneapple_veriphysics.ledger` (v05)
- `utilisation` f — `pinneapple_design.bricks.visuals` (v06)

V

- `v_max_svpwm` f — `pinneapple_simulation.electromagnetics.machines.drive` (v03)
- `valid` f — `pinneapple_design.aero.geometry` (v06)
- `valid_cnpj` f — `pinneapple_security.privacy` (v05)
- `valid_cpf` f — `pinneapple_security.privacy` (v05)
- `validate` f — `pinneapple_analysis.validation.validate` (v05)
- `validate` f — `pinneapple_data.contracts.models` (v04)

- validate f — pinneapple_design.bricks.validation (v06)
- validate f — pinneapple_flow.nodes_ext.bricks (v07)
- validate f — pinneapple_hub.zoo (v07)
- validate f — pinneapple_lab.manifest (v07)
- validate f — pinneapple_physics.astro.facts (v02)
- validate f — pinneapple_physics.pcb.solver (v02)
- validate f — pinneapple_physics.pde_environment.presets.astroml_bridge (v02)
- validate f — pinneapple_science.atlas (v02)
- validate f — pinneapple_simulation.electromagnetics.reflector.study (v03)
- validate f — pinneapple_simulation.external_solvers.newton_physics.validation (v03)
- validate f — pinneapple_simulation.terramechanics.testbed (v03)
- validate_against_solver f — pinneapple_analysis.validation.analytical (v05)
- validate_and_suggest f — pinneapple_problemdesign.elicitation.validators (v02)
- validate_consistency f — pinneapple_tools.dataset_quality.checks (v07)
- validate_custom_problem f — pinneapple_app.backend.routers.problems (v07)
- validate_dataset f — pinneapple_pdb.validate (v04)
- validate_flow f — pinneapple_flow.schema (v07)
- validate_generated f — pinneapple_llm.qa.generation (v07)
- validate_json_artifact f — pinneapple_llm.qa.static (v07)
- validate_model f — pinneapple_analysis.validation.validator (v05)
- validate_physical_sample f — pinneapple_data.validators (v04)
- validate_physics f — pinneapple_tools.nvidia.isaac (v07)
- validate_pinn f — pinneapple_analysis.validation.pinn (v05)
- validate_preset f — pinneapple_flow.nodes.presets (v07)
- validate_python_artifact f — pinneapple_llm.qa.static (v07)
- validate_result f — pinneapple_tools.benchmark_suite.crossframework.records (v07)
- validate_rollout f — pinneapple_analysis.validation.rollout (v05)
- validate_stage f — pinneapple_tools.nvidia.omniverse (v07)
- validate_tableau f — pinneapple_simulation.numerical_solvers.irk_gauss_legendre (v03)
- validate_targets f — pinneapple_simulation.electromagnetics.antenna.study (v03)
- validated_methods f — pinneapple_catalog.methods (v01)
- ValidatedDomain C — pinneapple_autolab.domain (v07)
- ValidateNode C — pinneapple_decision.tree (v05)
- validation_energy f — pinneapple_simulation.numerical_solvers.gated_xtfc (v03)
- validation_markdown f — pinneapple_simulation.pipelines.freecad_openfoam.report (v03)
- validation_metrics_from_report f — pinneapple_hub.verified_card (v07)
- validation_summary f — pinneapple_physics.astro.smallbodies (v02)
- ValidationCheck C — pinneapple_analysis.validation.validate (v05)
- ValidationContext C — pinneapple_llm.qa.validators.base (v07)
- ValidationIssue C — pinneapple_data.validators (v04)
- ValidationReport C — pinneapple_analysis.validation.core (v05)
- ValidationReport C — pinneapple_analysis.validation.validate (v05)
- ValidationReport C — pinneapple_simulation.terramechanics.testbed (v03)
- ValidationRequiredError C — pinneapple_decision.loop (v05)
- ValidationResult C — pinneapple_simulation.electromagnetics.antenna.surrogates (v03)
- ValidationSpec C — pinneapple_pdb.validate (v04)
- ValidationSpec C — pinneapple_problemdesign.schema (v02)
- ValidationStrategySelector C — pinneapple_decision.selectors (v05)
- ValidatorResult C — pinneapple_llm.qa.validators.base (v07)
- Validity C — pinneapple_reason.chain (v05)
- validity f — pinneapple_reason.library (v05)
- ValidityCheck C — pinneapple_physics.physics_audit.monitor (v02)
- ValidityEnvelope C — pinneapple_systems.process_components.real_gas_eos (v06)
- ValidityReport C — pinneapple_physics.physics_audit.monitor (v02)
- value f — pinneapple_mcp.expressions (v07)
- values f — pinneapple_simulation.numerical_solvers._bc (v03)

- valve_closure_moc f — pinneapple_simulation.numerical_solvers.water_hammer_moc (v03)
- ValveFlowResult C — pinneapple_systems.process_components.control_valve (v06)
- ValveModifiedMLP C — pinneapple_systems.component_library.components (v06)
- ValveSpec C — pinneapple_systems.process_components.control_valve (v06)
- van_leer_limiter f — pinneapple_simulation.numerical_solvers.tvd_advection (v03)
- vane_temperature_at_radial_position_c f — pinneapple_design.surrogate_inverse.brake_rotor (v06)
- VanGenuchten C — pinneapple_simulation.geophysics.richards (v03)
- VanillaPINN C — pinneapple_neural.architectures.pinns.vanilla (v04)
- VanillaTransformer C — pinneapple_neural.architectures.transformers.transformer (v04)
- VAR C — pinneapple_neural.architectures.classical_ts.var (v04)
- Var4D C — pinneapple_analysis.data_assimilation.var4d (v05)
- Var4DResult C — pinneapple_analysis.data_assimilation.var4d (v05)
- Variable C — pinneapple_data.contracts.models (v04)
- Variable C — pinneapple_design.design_optimizer.study (v06)
- VariableEnvelope C — pinneapple_veriphysics.applicability (v05)
- variables_section f — pinneapple_app.backend.core.surrogate_report (v07)
- VariableSelection C — pinneapple_pdb.builder (v04)
- VarianceBasedAL C — pinneapple_data.active_learning (v04)
- Variant C — pinneapple_condense.package (v02)
- variant_field f — pinneapple_condense.learned (v02)
- VariantPreview C — pinneapple_design.qualitative.preview (v06)
- Variation C — pinneapple_worldmodel.inspection_data (v04)
- VariationalAutoencoder C — pinneapple_neural.architectures.autoencoders.vae (v04)
- VarMapping C — pinneapple_physics.pinn_solver.io.mappings (v02)
- vec f — pinneapple_data.preflight.common (v04)
- vector f — pinneapple_data.simulation_metadata.foam_dict (v04)
- vector_potential f — pinneapple_simulation.electromagnetics.statics (v03)
- Vehicle C — pinneapple_simulation.electromagnetics.machines.drive (v03)
- vehicle_dashboard f — pinneapple_simulation.vehicle.dashboard (v03)
- vehicle_presets f — pinneapple_flow.nodes.vehicle (v07)
- vehicle_scene3d f — pinneapple_simulation.vehicle.dashboard (v03)
- vehicle_spec f — pinneapple_flow.nodes.vehicle (v07)
- VehicleCAD C — pinneapple_design.geometry.vehicle_cad (v06)
- VehicleCFD C — pinneapple_lab.experiments.vehicles3d (v07)
- VehicleModel C — pinneapple_simulation.vehicle.build (v03)
- VehicleRun C — pinneapple_simulation.vehicle.sim (v03)
- VehicleSimulator C — pinneapple_simulation.vehicle.sim (v03)
- VehicleSpec C — pinneapple_simulation.vehicle.spec (v03)
- velocity_components f — pinneapple_check.fields (v05)
- velocity_from_vorticity f — pinneapple_physics.multiscale.ns2d (v02)
- velocity_name f — pinneapple_check.fields (v05)
- velocity_set f — pinneapple_simulation.numerical_solvers.lbm_tunnel (v03)
- vented_rotor f — pinneapple_design.surrogate_inverse.meshes (v06)
- verdict f — pinneapple_app.backend.core.surrogate_report (v07)
- verdict f — pinneapple_hub.zoo.checks (v07)
- verdict f — pinneapple_reason.modes._util (v05)
- Verification C — pinneapple_decision.schema (v05)
- Verification C — pinneapple_design.intent.pipeline (v06)
- Verification C — pinneapple_reason.chain (v05)
- verification f — pinneapple_flow.nodes_ext.flight (v07)
- verification_checks f — pinneapple_autolab.trust (v07)
- verification_jobs f — pinneapple_design.aero.verify (v06)
- VerificationCard C — pinneapple_design.surrogate_inverse.core (v06)
- VerificationError C — pinneapple_security.signing (v05)
- VerificationReport C — pinneapple_analysis.validation.physics_learning (v05)
- VerificationReport C — pinneapple_design.design_optimizer.verify (v06)

- `Verifier` C — `pinneapple_decision.verifiers` (v05)
- `VerifierAgent` C — `pinneapple_tools.hpo_experiments.agents.verifier` (v07)
- `verify` f — `pinneapple_analysis.validation.physics_learning` (v05)
- `verify` f — `pinneapple_decision.api` (v05)
- `verify` f — `pinneapple_design.intent.pipeline` (v06)
- `verify` f — `pinneapple_design.thermal.board_cooling.verify` (v06)
- `verify` f — `pinneapple_flow.nodes.reason` (v07)
- `verify` f — `pinneapple_reason` (v05)
- `verify` f — `pinneapple_simulation.numerical_solvers.cylinder_wake` (v03)
- `verify` f — `pinneapple_simulation.numerical_solvers.thermal_channel_datasets` (v03)
- `verify` f — `pinneapple_tools.nvidia.heatsink` (v07)
- `verify_bytes` f — `pinneapple_security.signing` (v05)
- `verify_candidates` f — `pinneapple_design.design_optimizer.verify` (v06)
- `verify_design` f — `pinneapple_design.surrogate_inverse.brake_rotor` (v06)
- `verify_design` f — `pinneapple_design.surrogate_inverse.heatsink_fins` (v06)
- `verify_design` f — `pinneapple_design.surrogate_inverse.reflex_port` (v06)
- `verify_design` f — `pinneapple_design.surrogate_inverse.suspension_arm` (v06)
- `verify_design` f — `pinneapple_design.surrogate_inverse.transdermal_patch` (v06)
- `verify_doi` f — `pinneapple_flow.references` (v07)
- `verify_file` f — `pinneapple_security.signing` (v05)
- `verify_files` f — `pinneapple_hub.zoo.publish` (v07)
- `verify_freeze` f — `pinneapple_analysis.sensitivity.effects` (v05)
- `verify_front` f — `pinneapple_design.design_optimizer.report` (v06)
- `verify_json` f — `pinneapple_security.signing` (v05)
- `verify_mission` f — `pinneapple_simulation.rover.autolab.verify` (v03)
- `verify_model` f — `pinneapple_hub.zoo.publish` (v07)
- `verify_model_card` f — `pinneapple_security.integrate` (v05)
- `verify_models` f — `pinneapple_hub.zoo.publish` (v07)
- `verify_package` f — `pinneapple_veriphysics.evidence_package` (v05)
- `verify_password` f — `pinneapple_security.passwords` (v05)
- `verify_quote` f — `pinneapple_data.formfill.llm` (v04)
- `verify_record` f — `pinneapple_security.integrate` (v05)
- `verify_registry` f — `pinneapple_flow.references` (v07)
- `verify_report` f — `pinneapple_analysis.validation.report_artifact` (v05)
- `verify_space` f — `pinneapple_hub.zoo.publish` (v07)
- `verify_splash_archive` f — `pinneapple_simulation.external_solvers.openfoam.splash_packer` (v03)
- `verify_uploaded_results` f — `pinneapple_design.design_optimizer.report` (v06)
- `verify_version` f — `pinneapple_security.integrate` (v05)
- `verify_viewer` f — `pinneapple_hub.zoo.publish` (v07)
- `VerifyAdapter` C — `pinneapple_tools.automation.adapters.checks` (v07)
- `VerifyKey` C — `pinneapple_security.signing` (v05)
- `VerifyResult` C — `pinneapple_tools.hpo_experiments.agents.verifier` (v07)
- `VeriPhysicsVerifier` C — `pinneapple_decision.verifiers` (v05)
- `versions` f — `pinneapple_hub.zoo` (v07)
- `versions` f — `pinneapple_hub.zoo._artifacts` (v07)
- `vertex_normals` f — `pinneapple_design.aero.airframe` (v06)
- `vertex_normals` f — `pinneapple_tools.nvidia.brep` (v07)
- `vertex_normals` f — `pinneapple_tools.visualization.studio.scene` (v07)
- `VesselSpec` C — `pinneapple_twin3d.wear.model` (v06)
- `via_copper_fraction` f — `pinneapple_physics.closed_form.pcb_thermal` (v02)
- `Video` C — `pinneapple_worldmodel.physical_world` (v04)
- `video_check` f — `pinneapple_worldmodel.physical_world` (v04)
- `view_payload` f — `pinneapple_data.cae.report` (v04)
- `viewer_status` f — `pinneapple_hub.zoo.publish` (v07)
- `views` f — `pinneapple_data.cae.compare` (v04)
- `viper` f — `pinneapple_simulation.vehicle.viper` (v03)

- viper_cad f — pinneapple_simulation.vehicle.viper (v03)
- virtual_display f — pinneapple_flow.nodes.vehicle (v07)
- VirtualDisplay C — pinneapple_tools.automation.display (v07)
- visibility f — pinneapple_physics.astro.observing (v02)
- visibility f — pinneapple_worldmodel.robot_perception (v04)
- VLMSolution C — pinneapple_design.aero.vlm (v06)
- vmap f — pinneapple_core.func (v01)
- vmap_residual f — pinneapple_tools.compute_backends.jax_backend (v07)
- vmd_1d f — pinneapple_simulation.numerical_solvers.vmd (v03)
- VMDSolver C — pinneapple_simulation.numerical_solvers.vmd (v03)
- Voltmeter f — pinneapple_science.instruments (v02)
- volume f — pinneapple_design.geometry_library.meshops (v06)
- von_karman_energy f — pinneapple_simulation.flight.turbulence (v03)
- von_mises f — pinneapple_simulation.numerical_solvers.solid_fem (v03)
- von_mises f — pinneapple_tools.visualization.pyvista_bridge (v07)
- von_mises_2d_default f — pinneapple_physics.pde_environment.presets.structural (v02)
- von_mises_from_components f — pinneapple_physics.solid.axisymmetric (v02)
- von_mises_rz f — pinneapple_physics.solid.axisymmetric (v02)
- von_mises_stress_rectangular_section f — pinneapple_systems.process_components.beam_statics (v06)
- von_mises_triaxial f — pinneapple_systems.process_components.pipe_stress_mechanics (v06)
- vorticity_residual f — pinneapple_physics.multiscale.ns2d (v02)
- VorticityNS2D C — pinneapple_physics.multiscale.ns2d (v02)
- voxel_fem_cantilever f — pinneapple_design.qualitative.quantitative (v06)
- voxel_mesh f — pinneapple_design.bricks.fea (v06)
- VoxelGrid C — pinneapple_design.geometry.ops.voxelize (v06)
- voxelgrid_to_collocation f — pinneapple_design.geometry.ops.voxelize (v06)
- VoxelInsideConfig C — pinneapple_design.geometry.builders.stl_domain_batch_builder (v06)
- voxelize_domain f — pinneapple_design.geometry.ops.voxelize (v06)
- voxelize_pointcloud f — pinneapple_design.geometry.ops.voxelize (v06)
- voxelize_sdf f — pinneapple_design.geometry.ops.voxelize (v06)
- voxelize_stl f — pinneapple_simulation.numerical_solvers.lbm_tunnel (v03)
- VoxelModel C — pinneapple_design.bricks.fea (v06)
- VPINN C — pinneapple_neural.architectures.pinns.vpinn (v04)
- VPNoiseScheduler C — pinneapple_neural.architectures.pinns.physics_diffusion (v04)
- vq_pinn f — pinneapple_quantum.pipeline.presets (v04)
- VRotor C — pinneapple_simulation.electromagnetics.machines.geometry (v03)
- vtail_cd0 f — pinneapple_design.aero.aircraft3d (v06)
- vtail_weight f — pinneapple_design.aero.aircraft3d (v06)
- vtk_type f — pinneapple_data.cae.inp (v04)

W

- WALEResiduals C — pinneapple_physics.pde_environment.turbulence_presets (v02)
- wall_distance f — pinneapple_analysis.benchmark.builtin.airfoil_impl (v05)
- wall_distance f — pinneapple_data.cae.metrics (v04)
- wall_geometry f — pinneapple_design.aero.case (v06)
- wall_loads f — pinneapple_simulation.pipelines.freecad_openfoam.post (v03)
- wall_loop f — pinneapple_design.aero.geometry (v06)
- wall_repulsion_force f — pinneapple_simulation.numerical_solvers.sph_boundaries (v03)
- wall_unit_height f — pinneapple_design.fitmesh.context (v06)
- WaMaModel C — pinneapple_worldmodel.model_zoo (v04)
- warm_solver f — pinneapple_simulation.accelerated_foam.case (v03)
- water f — pinneapple_systems.digital_twin.pipe_loop.plant (v06)
- water_density f — pinneapple_physics.closed_form.pump_npsh (v02)
- WaterHammer C — pinneapple_presets.cases.water_hammer (v02)
- WaterNetwork C — pinneapple_presets.cases.pipe_network (v02)
- Wave1D C — pinneapple_arena.problems (v04)

- Wave1DTask C — pinneapple_tools.benchmark_suite.tasks.wave_1d (v07)
- wave_exact f — pinneapple_physics.paper_problems (v02)
- wave_period f — pinneapple_systems.process_components.water_hammer (v06)
- wave_speed f — pinneapple_simulation.numerical_solvers.water_hammer_moc (v03)
- wave_ultrasound_3d_default f — pinneapple_physics.pde_environment.presets.industry (v02)
- wavedec_features f — pinneapple_simulation.numerical_solvers.wavelet (v03)
- WaveguideModes C — pinneapple_science.em.waves (v02)
- WaveguideUQ C — pinneapple_science.em.research (v02)
- wavelength f — pinneapple_simulation.electromagnetics.antenna.bounds (v03)
- WaveletSolver C — pinneapple_simulation.numerical_solvers.wavelet (v03)
- wavenumber f — pinneapple_simulation.electromagnetics.reflector.po (v03)
- wavenumbers f — pinneapple_physics.multiscale.filters (v02)
- weak_form_2nd_order f — pinneapple_lab.experiments.discovery (v07)
- WearDataset C — pinneapple_twin3d.wear.model (v06)
- WearMaterial C — pinneapple_physics.tribology.archard (v02)
- web_search f — pinneapple_llm.workbench.web (v07)
- web_viewer f — pinneapple_tools.visualization.studio (v07)
- web_viewer f — pinneapple_tools.visualization.studio.web (v07)
- WebAdapter C — pinneapple_tools.automation.adapters.checks (v07)
- weight_sensitivity_study f — pinneapple_veriphysics.robustness (v05)
- weight_vectors f — pinneapple_design.design_optimizer.methods (v06)
- weighted_mmd f — pinneapple_analysis.uncertainty.posterior_metrics (v05)
- weighted_wasserstein_1d f — pinneapple_analysis.uncertainty.posterior_metrics (v05)
- WeightMinimizationObjective C — pinneapple_design.design_optimizer.objective (v06)
- weights_section f — pinneapple_app.backend.core.surrogate_report (v07)
- WeightScheduler C — pinneapple_neural.trainer.weight_scheduler (v04)
- WeightSchedulerConfig C — pinneapple_neural.trainer.weight_scheduler (v04)
- welch_psd f — pinneapple_simulation.flight.turbulence (v03)
- weld f — pinneapple_design.geometry_library.meshops (v06)
- what_if f — pinneapple_physics.pcb.engine (v02)
- Wheel C — pinneapple_simulation.terramechanics.wheel (v03)
- wheel_model f — pinneapple_flow.nodes.vehicle (v07)
- wheel_on_soil f — pinneapple_flow.wrappers (v07)
- WheelParams C — pinneapple_simulation.particle_dynamics.terramechanics (v03)
- white_noise f — pinneapple_systems.time_series.synthetic (v06)
- wien_peak_m f — pinneapple_physics.astro.stars (v02)
- williamson_case2 f — pinneapple_simulation.geophysics.sphere_shallow_water (v03)
- williamson_case5 f — pinneapple_simulation.geophysics.sphere_shallow_water (v03)
- williamson_case6 f — pinneapple_simulation.geophysics.sphere_shallow_water (v03)
- wilson_interval f — pinneapple_analysis.uncertainty.tolerance (v05)
- wind_convection f — pinneapple_reason.props (v05)
- wind_stress f — pinneapple_systems.time_series.enso.data (v06)
- wind_turbine f — pinneapple_twin3d.gallery (v06)
- WindField C — pinneapple_simulation.flight.turbulence (v03)
- Winding C — pinneapple_simulation.electromagnetics.machines.pmsm (v03)
- window_auc f — pinneapple_security.cps (v05)
- window_features f — pinneapple_systems.time_series.features.engineering (v06)
- window_mask f — pinneapple_physics.blackhole.convergence (v02)
- window_mass f — pinneapple_physics.blackhole.forecast (v02)
- window_mass f — pinneapple_physics.blackhole.skill (v02)
- window_volumes f — pinneapple_physics.blackhole.dataset (v02)
- windowed_balance f — pinneapple_systems.operations.anomaly (v06)
- windowed_elm f — pinneapple_lab.experiments.discovery (v07)
- WindowedTimeSeriesDataset C — pinneapple_systems.time_series.datasets.windowed (v06)
- WindTunnelLBM C — pinneapple_simulation.numerical_solvers.lbm_tunnel (v03)
- WindTunnelSetup C — pinneapple_simulation.external_solvers.fluidx3d (v03)

- wing f — pinneapple_design.geometry_library.kit (v06)
- wing f — pinneapple_twin3d.primitives (v06)
- wing_figures f — pinneapple_design.intent.figures (v06)
- wing_weight f — pinneapple_design.aero.aircraft3d (v06)
- wing_weight_transport f — pinneapple_design.aero.airliner3d (v06)
- winkler_parabolic_contact f — pinneapple_physics.tribology.archard (v02)
- WinsorizeStep C — pinneapple_neural.trainer.preprocess (v04)
- wire_az f — pinneapple_physics.closed_form.electromagnetics (v02)
- wire_field f — pinneapple_simulation.electromagnetics.analytic (v03)
- with_loop f — pinneapple_systems.digital_twin.pipe_loop.plant (v06)
- with_mirror f — pinneapple_design.geometry_library.kit (v06)
- with_params f — pinneapple_systems.operations.plant (v06)
- with_params f — pinneapple_systems.sports.football.pc_study (v06)
- with_quick f — pinneapple_flow.examples (v07)
- with_units f — pinneapple_physics.pde_environment.presets.astropy_bridge (v02)
- wltc_class3b f — pinneapple_simulation.electromagnetics.machines.drive (v03)
- wod_d20_casts f — pinneapple_systems.time_series.enso.data (v06)
- WongReeceModel C — pinneapple_simulation.terramechanics.wong_reece (v03)
- words f — pinneapple_data.simulation_metadata.foam_dict (v04)
- worker_commands f — pinneapple_simulation.external_solvers.openfoam.campaign (v03)
- Workflow C — pinneapple_core.workflows.dag (v01)
- Workflow C — pinneapple_tools.automation.workflow (v07)
- workflow_from_cases f — pinneapple_core.workflows.dag (v01)
- WorkflowError C — pinneapple_core.workflows.dag (v01)
- WorkflowError C — pinneapple_tools.automation.workflow (v07)
- WorkflowRunner C — pinneapple_core.workflows.engine (v01)
- working_method f — pinneapple_core.onboarding.catalog (v01)
- Workspace C — pinneapple_analysis.benchmark.runner (v05)
- Workspace C — pinneapple_llm.workbench.tools (v07)
- WorldModel C — pinneapple_worldmodel.physical_world (v04)
- WorldModelConfig C — pinneapple_worldmodel.model (v04)
- WorldModelDataset C — pinneapple_worldmodel.dataset (v04)
- WorldModelLoss C — pinneapple_worldmodel.trainer (v04)
- WorldModelTrainConfig C — pinneapple_worldmodel.trainer (v04)
- WorldModelTrainer C — pinneapple_worldmodel.trainer (v04)
- worst_case f — pinneapple_simulation.electromagnetics.periodic.benchmark (v03)
- worst_cells f — pinneapple_data.cae.report (v04)
- worst_n_points f — pinneapple_tools.model_evaluation.metrics (v07)
- wrap f — pinneapple_simulation.flight.control (v03)
- wrap_data_parallel f — pinneapple_neural.trainer.parallel (v04)
- wrap_for_uq f — pinneapple_neural.physicsnemo_bridge.adapters (v04)
- wrap_fsdp f — pinneapple_neural.trainer.hpc (v04)
- wrap_zero_optimizer f — pinneapple_neural.trainer.hpc (v04)
- wrapped f — pinneapple_flow.libcalls (v07)
- write f — pinneapple_data.unified (v04)
- write f — pinneapple_flow.autocalls (v07)
- write f — pinneapple_science.report (v02)
- write_all f — pinneapple_science.report (v02)
- write_baseline f — pinneapple_check.baseline (v05)
- write_calculix_cantilever f — pinneapple_simulation.external_solvers.campaign_cases (v03)
- write_calculix_from_template f — pinneapple_simulation.external_solvers.campaign_cases (v03)
- write_campaign f — pinneapple_simulation.external_solvers.campaign_cases (v03)
- write_case f — pinneapple_design.aero.case (v06)
- write_case3d f — pinneapple_design.aero.case3d (v06)
- write_case_tables f — pinneapple_simulation.external_solvers.campaign_cases (v03)
- write_catalogue f — pinneapple_presets.report (v02)

- write_catalogue f — pinneapple_science.report (v02)
- write_cavity f — pinneapple_simulation.accelerated_foam.case (v03)
- write_csv f — pinneapple_analysis.benchmark.report (v05)
- write_csv f — pinneapple_data.cae.dataset (v04)
- write_dataset f — pinneapple_hub.zoo.parquet (v07)
- write_decompose f — pinneapple_simulation.external_solvers.openfoam.campaign (v03)
- write_design_files f — pinneapple_design.design_optimizer.report (v06)
- write_engine_anim f — pinneapple_simulation.external_solvers.openradioss.deck (v03)
- write_field_dataset f — pinneapple_design.aero.field_dataset (v06)
- write_flow_fields f — pinneapple_simulation.numerical_solvers.external_flow (v03)
- write_for_run f — pinneapple_lab.manifest (v07)
- write_frd f — pinneapple_simulation.external_solvers.calculix.frd (v03)
- write_geo f — pinneapple_simulation.electromagnetics.machines.getdp (v03)
- write_geo_3d f — pinneapple_simulation.electromagnetics.machines.getdp (v03)
- write_gif f — pinneapple_design.bricks.visuals (v06)
- write_gif f — pinneapple_simulation.vehicle.viz (v03)
- write_gif f — pinneapple_tools.visualization.dashboard.static (v07)
- write_glb f — pinneapple_tools.visualization.studio.scene (v07)
- write_gltf f — pinneapple_design.bricks.mesh (v06)
- write_hdf5 f — pinneapple_data.cae.dataset (v04)
- write_html f — pinneapple_design.intent.package (v06)
- write_html f — pinneapple_lab.html (v07)
- write_html f — pinneapple_tools.visualization.dashboard.html (v07)
- write_index f — pinneapple_hub.library (v07)
- write_index f — pinneapple_presets.registry (v02)
- write_inp f — pinneapple_simulation.external_solvers.calculix.inp (v03)
- write_inputs f — pinneapple_veriphysics.validation_lab (v05)
- write_instructions f — pinneapple_design.bricks.visuals (v06)
- write_internal_case f — pinneapple_simulation.pipelines.freecad_openfoam.case (v03)
- write_internal_mesh f — pinneapple_simulation.pipelines.freecad_openfoam.mesh (v03)
- write_json f — pinneapple_data.cae.dataset (v04)
- write_ldr f — pinneapple_design.bricks.ldraw (v06)
- write_manifest f — pinneapple_design.geometry_library.store (v06)
- write_markdown_block f — pinneapple_simulation.electromagnetics.verification (v03)
- write_motion_stage f — pinneapple_worldmodel.robot_perception (v04)
- write_mp4 f — pinneapple_design.bricks.visuals (v06)
- write_mpd f — pinneapple_design.bricks.ldraw (v06)
- write_npz f — pinneapple_data.cae.dataset (v04)
- write_npz f — pinneapple_physics.blackhole.dataset (v02)
- write_npz f — pinneapple_simulation.numerical_solvers.operator_datasets (v03)
- write_openfoam_airfoil f — pinneapple_simulation.external_solvers.campaign_cases (v03)
- write_openfoam_from_base f — pinneapple_simulation.external_solvers.campaign_cases (v03)
- write_parquet f — pinneapple_data.cae.dataset (v04)
- write_particle_frame f — pinneapple_simulation.particle_dynamics.geo_mpm (v03)
- write_pdf f — pinneapple_design.intent.package (v06)
- write_pinneapple f — pinneapple_data.cae.dataset (v04)
- write_ply f — pinneapple_blender.export (v07)
- write_png f — pinneapple_tools.visualization.dashboard.static (v07)
- write_polymesh f — pinneapple_design.aero.mesh (v06)
- write_pro f — pinneapple_simulation.electromagnetics.machines.getdp (v03)
- write_pro_3d f — pinneapple_simulation.electromagnetics.machines.getdp (v03)
- write_profile f — pinneapple_simulation.numerical_solvers.free_surface (v03)
- write_python_runner f — pinneapple_simulation.external_solvers.campaign_cases (v03)
- write_report f — pinneapple_autolab.report (v07)
- write_report f — pinneapple_design.bricks.report (v06)
- write_report f — pinneapple_design.fitmesh.report (v06)

- write_report f — pinneapple_design.thermal.board_cooling.report (v06)
- write_report f — pinneapple_lab.reports (v07)
- write_report f — pinneapple_presets.report (v02)
- write_report f — pinneapple_reason (v05)
- write_report f — pinneapple_reason.report (v05)
- write_report f — pinneapple_simulation.rover.autolab.report (v03)
- write_report f — pinneapple_systems.digital_twin.pipe_loop.report (v06)
- write_report f — pinneapple_tools.automation.report (v07)
- write_run f — pinneapple_hub.library (v07)
- write_run f — pinneapple_tools.benchmark_suite.crossframework.records (v07)
- write_run_artifacts f — pinneapple_tools.benchmark_suite.runner.report (v07)
- write_sample_dict_cloud f — pinneapple_simulation.external_solvers.openfoam.sampling (v03)
- write_setup f — pinneapple_simulation.external_solvers.fluidx3d (v03)
- write_sidecar f — pinneapple_data.contracts.tables (v04)
- write_site f — pinneapple_lab.html (v07)
- write_split_files f — pinneapple_physics.blackhole.dataset (v02)
- write_static f — pinneapple_lab.explorer (v07)
- write_stl f — pinneapple_design.bricks.mesh (v06)
- write_stl f — pinneapple_simulation.pipelines.freecad_openfoam.cad (v03)
- write_stl f — pinneapple_tools.visualization.studio.scene (v07)
- write_study_result f — pinneapple_tools.automation.report (v07)
- write_table f — pinneapple_data.contracts.tables (v04)
- write_table f — pinneapple_simulation.electromagnetics.antenna.runner (v03)
- write_usd f — pinneapple_design.bricks.mesh (v06)
- write_usda f — pinneapple_tools.visualization.studio.scene (v07)
- write_variation_stage f — pinneapple_worldmodel.inspection_data (v04)
- write_video f — pinneapple_tools.visualization.dashboard.static (v07)
- write_vtk f — pinneapple_simulation.numerical_solvers.lbm_tunnel (v03)
- write_vtu f — pinneapple_data.cae.dataset (v04)
- write_vtu f — pinneapple_simulation.electromagnetics.machines.export (v03)
- Written C — pinneapple_flow.nodes_ext.examples (v07)

X

- xavier_uniform_init f — pinneapple_tools.benchmark_suite.crossframework.weights (v07)
- xd_fit f — pinneapple_physics.pde_environment.presets.astroml_bridge (v02)
- XdotoolDriver C — pinneapple_tools.automation.display (v07)
- XDResult C — pinneapple_physics.pde_environment.presets.astroml_bridge (v02)
- XGBoostForecaster C — pinneapple_systems.time_series.models.classical (v06)
- XGParams C — pinneapple_systems.sports.football.xg (v06)
- XPINN C — pinneapple_neural.architectures.pinns.xpinn (v04)
- XTFC C — pinneapple_neural.architectures.pinns.xtfc (v04)
- xtfc_case f — pinneapple_physics.paper_problems_xtfc (v02)
- xtfc_pde f — pinneapple_flow.wrappers (v07)
- xtfc_pde_predict f — pinneapple_simulation.numerical_solvers.xtfc_pde (v03)
- xtfc_predict f — pinneapple_simulation.numerical_solvers.xtfc_ivp (v03)
- XTFCCase C — pinneapple_physics.paper_problems_xtfc (v02)
- XTFCConfig C — pinneapple_neural.architectures.pinns.xtfc (v04)
- XTFCFactoryModel C — pinneapple_neural.architectures.pinns.xtfc (v04)
- XTFCIVPSolver C — pinneapple_simulation.numerical_solvers.xtfc_ivp (v03)
- XTFCPDESolver C — pinneapple_simulation.numerical_solvers.xtfc_pde (v03)
- XTFCProblem C — pinneapple_simulation.numerical_solvers.xtfc_general (v03)
- XTFCResult C — pinneapple_simulation.numerical_solvers.xtfc_general (v03)
- XTFCSubdomainSolver C — pinneapple_simulation.numerical_solvers.xtfc_subdomain (v03)

Y

- yang_comparison f — pinneapple_simulation.electromagnetics.machines.verification (v03)
- YeeGrid C — pinneapple_simulation.electromagnetics.fdtd (v03)
- youtube_daily_aggregates f — pinneapple_systems.time_series.datasets.public (v06)
- YouTubeDaily C — pinneapple_systems.time_series.datasets.public (v06)
- yplus_check f — pinneapple_analysis.benchmark.validation (v05)
- YPlusEstimate C — pinneapple_analysis.verification.mesh_intelligence (v05)

Z

- ZarrByteCacheConfig C — pinneapple_data.zarr_cached_store_bytes (v04)
- ZarrCacheConfig C — pinneapple_data.zarr_cached_store (v04)
- ZarrUPDIterable C — pinneapple_data.zarr_iterable (v04)
- zipper f — pinneapple_simulation.electromagnetics.mesh (v03)
- ZoneForecast C — pinneapple_twin3d.wear.forecast (v06)
- zones f — pinneapple_neural.architectures.rom.views (v04)
- ZoneSpec C — pinneapple_twin3d.wear.model (v06)
- ZoneState C — pinneapple_twin3d.wear.optimize (v06)
- zoo f — pinneapple_systems.time_series.enso.study (v06)
- zoo_evaluate f — pinneapple_flow.nodes_ext.validation (v07)
- zoo_list f — pinneapple_mcp.tools (v07)
- zoo_predict f — pinneapple_mcp.tools (v07)
- zoo_problem f — pinneapple_design.design_optimizer.spec (v06)
- zoo_range f — pinneapple_flow.nodes_ext.validation_gallery (v07)
- zoo_service f — pinneapple_tools.model_services.registry (v07)
- zoo_surrogate f — pinneapple_flow.nodes.vehicle (v07)
- zoo_validate f — pinneapple_mcp.tools (v07)
- ZooContactSurrogate C — pinneapple_simulation.terramechanics.surrogate (v03)
- ZooEntry C — pinneapple_worldmodel.model_zoo (v04)
- ZooError C — pinneapple_hub.zoo._artifacts (v07)
- ZooEvaluator C — pinneapple_design.design_optimizer.evaluators (v06)
- ZooModel C — pinneapple_hub.zoo._models (v07)
- ZScoreDetector C — pinneapple_systems.digital_twin.monitoring.anomaly (v06)
- zwietering_njs f — pinneapple_simulation.numerical_solvers.particles (v03)

2. Glossário

termo	significado
Adjunto	Método que calcula o gradiente de um objetivo em relação a muitos parâmetros com custo de uma simulação extra.
Assimilação de dados	Combinar um modelo com medições ao longo do tempo (por exemplo, filtro de Kalman, EnKF).
Autolab	Laboratório autônomo: planeja e executa missões com ferramentas controladas, compara surrogates e escolhe a próxima simulação.
Co-simulação	Acoplar simuladores diferentes que trocam dados a cada passo de tempo (por exemplo, via FMI/FMU).
Condensação (Schur)	Eliminar os graus de liberdade internos de um bloco, ficando só com a interface; reaproveitado entre blocos idênticos.
Condição de contorno (BC)	Valor ou fluxo imposto na fronteira do domínio. Dirichlet fixa o valor; Neumann fixa a derivada normal; Robin combina os dois.
Condição inicial (IC)	Estado do sistema no instante inicial de um problema transiente.
Craig-Bampton	Redução de modelos para dinâmica: modos de interface fixa mais modos de restrição.
Credibilidade	Nível atribuído por pp.validate a cada fator verificado, com decisão final pass / warn / fail.
DeepONet	Operador neural com uma rede 'branch' (entrada funcional) e uma 'trunk' (coordenadas).
EKI	Ensemble Kalman Inversion: método de inversão sem gradiente baseado em ensembles.
Extra (pyproject)	Grupo opcional de dependências instalado com pip install "pinneapple[extra]".

termo	significado
FDM / FEM / FVM	Diferenças finitas, elementos finitos e volumes finitos: famílias clássicas de discretização de equações diferenciais.
Flow	Grafo de nós tipados (JSON pinneapple-flow/1) que encadeia chamadas da biblioteca; executado por pinneapple_flow.
FMI / FMU	Functional Mock-up Interface: padrão para trocar modelos de simulação empacotados (Functional Mock-up Units).
FNO	Fourier Neural Operator: operador neural que mistura informação no espaço de Fourier.
Frente de Pareto	Conjunto de soluções em que nenhum objetivo melhora sem piorar outro.
GNN / MeshGraphNet	Rede de grafos que opera sobre os nós e arestas de uma malha.
Guardrail físico	Verificação que bloqueia propostas (por exemplo, de um LLM) que violam a física ou o escopo.
Gêmeo digital (digital twin)	Modelo de um ativo real que é atualizado por medições e usado para monitorar, prever ou decidir.
Hard BC	Condição de contorno imposta pela própria forma da rede (por construção), e não por um termo de perda.
Hub / zoo	Repositório de modelos treinados e verificados, datasets e biblioteca pesquisável de ferramentas e exemplos.
Lab / selo	pinneapple_lab registra experimentos com manifesto, métricas, verificações e uma nota (A-F); o selo resume essa avaliação.
LBM	Lattice Boltzmann Method: método de fluidos baseado em distribuições de partículas numa rede regular.
MCP	Model Context Protocol: protocolo para expor ferramentas a agentes de IA; pinneapple_mcp é o servidor do PINNeAPPle.
NSGA-II	Algoritmo genético multiobjetivo que encontra uma frente de Pareto.
OOD (fora do domínio)	Entrada fora da região onde o modelo foi treinado ou validado; os resultados ali não são confiáveis.
OpenUSD / Omniverse	Formato de cena 3-D da Pixar e plataforma da NVIDIA que o usa; o PINNeAPPle exporta campos e decisões de confiança para ele.
Operador neural	Rede que aprende um mapa entre funções (por exemplo, condição de contorno → campo solução). Exemplos: FNO, DeepONet, AFNO.
Otimização bayesiana	Otimização com um modelo probabilístico (em geral processo gaussiano) que escolhe o próximo ponto a avaliar.
PhysicalProblem	Classe única do PINNeAPPle que descreve um problema físico para todas as partes da biblioteca.
PhysicsNeMo	Framework da NVIDIA para IA física; o PINNeAPPle confere surrogates feitos com ele.
PINN	Physics-Informed Neural Network: rede neural treinada para satisfazer a equação diferencial (resíduo na perda), além de dados e condições de contorno.
Ponto de colocação (collocation point)	Ponto do domínio onde o resíduo da equação é avaliado durante o treino de uma PINN.
Predição conformal	Método que produz intervalos com cobertura garantida a partir de um conjunto de calibração.
Preset	Problema pronto, com nome, equações, domínio e parâmetros padrão; pode ser carregado e resolvido diretamente.
Problema inverso	Estimar parâmetros ou entradas desconhecidas a partir de observações da saída.
ProblemSpec	Definição executável de uma EDP (coordenadas, campos, termo da EDP, contornos, domínio) consumida pelo compilador de PINN.
Referência independente	Valor de comparação que não vem do próprio código testado: forma fechada, solução manufaturada ou valor publicado/tabelado.
Resíduo	Quanto a solução deixa de satisfazer a equação num ponto. Resíduo zero = equação satisfeita.
ROM	Reduced-Order Model: modelo de ordem reduzida (por exemplo, POD) que aproxima o modelo completo com poucos graus de liberdade.
SDF / CSG	Signed Distance Function (distância com sinal até a fronteira) e Constructive Solid Geometry (união, interseção e diferença de sólidos).
Shim de compatibilidade	Pacote que só reexporta outro, mantido para que imports antigos funcionem.
Solution	Objeto devolvido por pp.solve: valores, pontos, método e metadados (info).
Solução manufaturada	Solução escolhida de antemão, inserida na equação para gerar o termo fonte; serve para verificar a ordem de convergência de um solver.
SPH	Smoothed Particle Hydrodynamics: método sem malha baseado em partículas.
Surrogate (modelo substituto)	Modelo rápido, em geral aprendido, que aproxima um simulador caro.
Teste de Diebold-Mariano (DM)	Teste estatístico que compara a precisão de duas previsões.

termo	significado
Trust card	Cartão de confiança: conjunto de verificações físicas de um campo (resíduo, conservação, contornos, referência, domínio) com decisão APPROVED / REVIEW / REJECT.
UQ	Uncertainty Quantification: quantificação de incerteza (ensembles, conformal, propagação de tolerâncias, Monte Carlo).
validated / tested / untested	Status do catálogo de métodos: validated = algum teste compara contra referência independente; tested = há testes, mas nenhum contra referência; untested = sem testes.
Walk-forward purgado	Protocolo de validação de séries temporais que treina no passado, testa no futuro e remove amostras vizinhas para evitar vazamento.
X-TFC / TFC	Theory of Functional Connections: expressões que satisfazem as restrições exatamente; X-TFC combina isso com redes de camada única (extreme learning machine).

3. Apêndice: números do inventário

item	valor
pacotes `pinneapple*`	41
módulos Python nesses pacotes	2.142
linhas de código nesses módulos	494.775
classes públicas	2.757
funções públicas de nível de módulo	6.130
métodos públicos de classes	6.220
módulos importados por pelo menos um arquivo de teste	1.073
arquivos de teste (`tests/**/test*.py`)	513
funções de teste (`test*`, inclusive em classes `Test*`)	4.611
nomes expostos em `pp.*`	297
extras do `pyproject.toml`	54
apps em `apps/`	64
entradas em `examples/`	88
entradas em `scripts/`	24
arquivos Markdown em `docs/`	190
workflows de CI	12
métodos no catálogo	201

Status dos métodos do catálogo (regra: validated = a test exercising the item compares against an independent reference (closed form, manufactured solution, published/tabulated value); arquivo gerado em 2026-10-17):

status	métodos
tested	47
untested	42
validated	112

3.1 Por pacote

pacote	módulos	linhas	classes	funções	testes*	módulos sem teste
`pinneapple`	3	866	0	2	431	2
`pinneapple_adaptation`	13	2.946	10	9	154	2
`pinneapple_analysis`	159	42.004	249	450	939	56
`pinneapple_app`	18	3.672	29	53	48	11
`pinneapple_arena`	54	21.853	69	243	99	39
`pinneapple_autolab`	16	3.596	38	39	37	10
`pinneapple_blender`	5	616	1	7	15	2
`pinneapple_catalog`	3	1.024	2	8	16	1
`pinneapple_check`	23	4.547	22	71	144	11
`pinneapple_condense`	18	6.006	29	115	27	3

pacote	módulos	linhas	classes	funções	testes*	módulos sem teste
pinneapple_core	34	8.747	71	92	233	15
pinneapple_data	115	29.019	129	256	507	70
pinneapple_decision	12	2.499	37	20	44	9
pinneapple_design	196	48.153	269	740	744	86
pinneapple_flow	89	18.086	23	381	263	58
pinneapple_hub	31	7.260	22	170	183	8
pinneapple_lab	42	11.775	45	119	96	27
pinneapple_llm	49	10.750	66	99	242	28
pinneapple_mcp	8	1.541	2	31	30	3
pinneapple_models	2	74	0	1	1	1
pinneapple_neural	198	36.217	334	150	438	152
pinneapple_orchestration	16	591	1	11	6	14
pinneapple_pdb	7	1.929	9	14	100	5
pinneapple_perception	7	1.037	6	19	29	2
pinneapple_physics	166	43.959	169	763	1.075	52
pinneapple_presets	21	5.247	22	31	50	16
pinneapple_problemdesign	28	5.642	40	21	123	19
pinneapple_quantum	16	2.636	17	24	0	16
pinneapple_query	3	252	7	2	7	2
pinneapple_reason	18	3.757	17	60	17	8
pinneapple_registry	10	1.699	10	3	92	3
pinneapple_science	16	4.253	33	49	19	10
pinneapple_security	20	3.398	33	93	105	0
pinneapple_simulation	298	66.085	364	931	1.119	113
pinneapple_solvers	2	71	0	0	1	1
pinneapple_systems	182	37.262	301	515	579	78
pinneapple_tools	177	34.350	155	384	465	107
pinneapple_train	4	109	0	0	1	1
pinneapple_twin3d	19	2.820	19	67	74	4
pinneapple_veriphysics	13	3.411	17	38	98	1
pinneapple_worldmodel	31	15.016	90	49	94	23

* funções de teste em arquivos que importam algum módulo do pacote (um teste pode contar em mais de um pacote).

3.2 Extras do `pyproject.toml`

Projeto `pinneapple` versão `0.7.0`, Python `>=3.10`. Dependências obrigatórias: `numpy>=1.24`, `scipy>=1.10`, `pandas>=2.0`, `pyyaml>=6.0`, `tqdm>=4.66`, `typing-extensions>=4.8`, `pydantic>=2.0`, `matplotlib>=3.8`, `torch>=2.2`, `sympy>=1.12`, `xarray`, `zarr`, `tomli>=2.0; python_version < '3.11'`.

extra	pacotes
`cfd`	manifold3d>=2.4, huggingface_hub>=0.20, cadquery>=2.4, gmsh>=4.11, trimesh>=4.0, pyvista>=0.43, plotly>=5.18, meshio>=5.3, h5py>=3.8
`enterprise`	fastapi>=0.111, uvicorn[standard]>=0.29, cryptography>=41, argon2-cffi>=23.1, meshio>=5.3, h5py>=3.8, huggingface_hub>=0.24, pyarrow>=14, safetensors>=0.4, mcp>=1.10
`mcp`	mcp>=1.10
`security`	cryptography>=41, argon2-cffi>=23.1
`check`	meshio>=5.3, h5py>=3.8, cryptography>=41
`sports`	scikit-learn>=1.3
`em`	miepython>=3.0

extra	pacotes
veriphysics	reportlab>=4.0
viz3d	rerun-sdk>=0.20, plotly>=5.18, trame>=3.0, trame-vtk>=2.5, trame-vuetify>=2.4
twin	basyx-python-sdk>=1.0, asyncua>=1.1
geometry	manifold3d>=2.4, huggingface_hub>=0.20
fitmesh	cadquery>=2.4, gmsh>=4.11, trimesh>=4.0
newton	newton>=1.6
sky	sgp4>=2.22, pymsis>=0.10, skyfield-data>=6.0, skyfield>=1.48
astroml	astroML>=1.0
fmi	fmpy>=0.3.20, pythonfmu>=0.6
tracking	mlflow>=2.9, wandb>=0.16
geom	trimesh>=4.0.0, meshio>=5.3.0, gmsh>=4.11
geom_simplify	fast-simplification>=0.1.0, open3d>=0.18.0
cad	cadquery>=2.4.0
solvers	numba>=0.59
process	CoolProp>=6.6
astro	astropy>=6.0
jax_cfd	jax-cfd>=0.2.0
fenics	fenics-dolfinx>=0.7.0, mpi4py>=3.1, petsc4py>=3.20
mujoco	mujoco>=3.0
mujoco_mjx	mujoco>=3.0, jax[cuda12]>=0.4.25
genesis	genesis-world>=0.2.0
noether	emmiai-noether>=2026.4.0
pinn	sympy>=1.12
uq	scipy>=1.10
serve	fastapi>=0.111, uvicorn[standard]>=0.29
export	onnx>=1.16, onnxruntime>=1.18
hub	huggingface_hub>=0.24, pyarrow>=14, safetensors>=0.4, onnx>=1.16, onnxruntime>=1.18
flow	sqlalchemy>=2.0, sympy>=1.12, fastapi>=0.111, uvicorn[standard]>=0.29, httpx>=0.27, pillow>=10
llm	anthropic>=0.34, openai>=1.40
orchestration	prefect>=2.19
tuning	optuna>=3.6

extra	pacotes
workbench	fastapi>=0.110, uvicorn>=0.29, requests>=2.31
finetune	peft>=0.12, datasets>=2.20, transformers>=4.44
vlm_curation	transformers>=4.45, accelerate>=0.30
digital_twin	paho-mqtt>=2.0, kafka-python>=2.0, asyncua>=1.1, pymodbus>=3.6
timeseries	statsmodels>=0.14, pmdarima>=2.0, scikit-learn>=1.4, PyWavelets>=1.6
synth	sympy>=1.12, trimesh>=4.0.0, meshio>=5.3.0
earthdata	requests>=2.31, earthaccess>=0.10.0
research	requests>=2.31, pdfminer.six>=20231228
retrieval	sentence-transformers>=3.0
all	cryptography>=41, trimesh>=4.0.0, meshio>=5.3.0, gmsh>=4.11, fast-simplification>=0.1.0, cadquery>=2.4.0, numba>=0.59, sympy>=1.12, fastapi>=0.111, uvicorn[standard]>=0.29, onnx>=1.16, onnxruntime>=1.18, paho-mqtt>=2.0, kafka-python>=2.0, sentence-transformers>=3.0, statsmodels>=0.14, requests>=2.31, mujoco>=3.0, fmpy>=0.3.20, genesis-world>=0.2.0, emmiai-noether>=2026.4.0, huggingface_hub>=0.24, anthropic>=0.34, openai>=1.40, transformers>=4.45, accelerate>=0.30
viz	matplotlib>=3.8, scipy>=1.10
viz_interactive	plotly>=5.18, kaleido>=0.2
pyvista	pyvista>=0.43, matplotlib>=3.8, imageio>=2.34
cost	psutil>=5.9
surrogate	scikit-learn>=1.4, pyarrow>=14, h5py>=3.8, onnx>=1.15, onnxruntime>=1.17
dev	ruff>=0.5, pytest>=8.0, pytest-cov>=4.1, mypy>=1.8, build>=1.2, twine>=5.0, ipykernel>=6.0, httpx>=0.27, jupyter>=1.0

3.3 Catálogo completo de métodos

id	método	status	testes contra referência	outros testes	código
E1	Laplace	validated	7	28	
E2	Poisson	validated	15	24	
E3	Helmholtz	validated	2	1	

id	método	status	testes contra referência	outros testes	código
E4	Heat (steady anisotropic / transient)	validated	4	13	
E5	Wave	validated	1	0	
E6	Advection-diffusion / convection	validated	1	0	
E7	Burgers (viscous / inviscid)	validated	10	50	
E8	Reaction-diffusion 1D/2D (Gray-Scott)	validated	2	1	
E9	Fisher-KPP	validated	1	3	
E10	Incompressible Navier-Stokes 2D/3D	tested	0	13	
E11	Navier-Stokes in a rotating frame	tested	0	2	
E12	Stokes	validated	1	1	
E13	Brinkman	validated	1	0	
E14	Darcy	tested	0	12	
E15	Buckley-Leverett two-phase	validated	1	0	
E16	Compressible Euler 1D/2D/axisymmetric/rotating 3D	validated	3	1	
E17	Shallow water 1D/2D	validated	1	6	
E18	Stommel gyre	validated	2	0	
E19	Plane-stress / plane-strain linear elasticity	validated	2	5	
E20	Axisymmetric elasticity (+ torsion) / Lame	validated	1	4	

id	método	status	testes contra referência	outros testes	código
E21	Neo-Hookean hyperelasticity	validated	1	0	
E22	Euler-Bernoulli beam	validated	1	11	pinneapple_systems/process_components/beam_statics.py
E23	von Karman beam / plate	validated	2	6	pinneapple_systems/process_components/beam_nonlinear_fem.py
E24	Thermoelasticity	validated	1	0	
E25	Phase-field fracture	untested	0	0	
E26	Biot poroelasticity	validated	1	0	
E27	Maxwell TE	validated	1	0	
E28	Bekker-Wong terramechanics	validated	1	2	
E29	Compressor meanline 1D	validated	2	3	
E30	Phonon Boltzmann transport 1D (gray)	validated	3	1	
E31	Black-Scholes	validated	1	2	
E32	Heston	validated	2	1	
E33	Opinion dynamics (continuum Hegselmann-Krause)	validated	1	0	
E34	Kepler two-body orbit	validated	1	6	
E35	Circular restricted three-body problem (planar, synodic)	validated	3	3	
E36	J2 perturbation	validated	1	1	

id	método	status	testes contra referência	outros testes	código
E37	Clohessy-Wiltshire relative motion	validated	1	1	
E38	Euler rigid-body attitude	validated	1	1	
E39	Lane-Emden polytrope	validated	5	19	
E40	Shakura-Sunyaev accretion disk	validated	6	4	
E41	Schwarzschild weak-field light bending	validated	2	3	
E42	SIR epidemic	validated	1	0	
E43	Pharmacokinetics two-compartment	validated	1	0	
E44	Reaction network / Arrhenius / mass action	tested	0	10	pinneapple_systems/process_components/reaction_kinetics.py
E45	Hagen-Poiseuille	tested	0	2	PINNeAPPle-CFD/pinneapple_cfd/etapas/e02_modelo_matematico/correlacoes.py
E46	Blasius friction factor	validated	2	10	PINNeAPPle-CFD/…/correlacoes.py
E47	Colebrook-White	tested	0	8	PINNeAPPle-CFD/…/correlacoes.py, pinneapple_systems/process_components/pipe_network_1d.py
E48	Darcy-Weisbach pressure drop	untested	0	0	PINNeAPPle-CFD/…/correlacoes.py
E49	Herschel-Bulkley + Metzner-Reed (non-Newtonian pipe flow)	tested	0	4	pinneapple_systems/process_components/non_newtonian_pipe_flow.py
E50	DNV-RP-O501 erosion (straight pipe and bend)	untested	0	0	PINNeAPPle-CFD/pinneapple_cfd/etapas/e02_modelo_matematico/erosao.py
E51	Lame thick-walled cylinder	tested	0	3	pinneapple_systems/process_components/pipe_stress_mechanics.py
E52	von Mises equivalent stress	tested	0	11	pinneapple_systems/process_components/pipe_stress_mechanics.py, pinneapple_systems/process_components/beam_statics.py
E53	Euler buckling + constrained rod buckling	validated	1	6	pinneapple_systems/process_components/pipe_stress_mechanics.py
E54	Beam-column moment amplification factor	tested	0	3	pinneapple_systems/process_components/pipe_stress_mechanics.py

id	método	status	testes contra referência	outros testes	código
E55	S-N curve + Goodman + Miner	validated	1	14	pinneapple_systems/process_components/fatigue_analysis.py, pinneapple_physics/closed_form/cantilever_fatigue.py
E56	Torsional stick-slip with Stribeck friction	tested	0	6	pinneapple_systems/process_components/torsional_stickslip.py
E57	Heat exchanger (effectiveness-NTU)	tested	0	8	pinneapple_systems/process_components/heat_exchanger.py
E58	Fin array conduction	tested	0	4	pinneapple_physics/closed_form/fin_array_conduction.py
E59	Control valve (Cv, compressible / incompressible)	tested	0	1	pinneapple_systems/process_components/control_valve.py
E60	Polytropic compression path	tested	0	1	pinneapple_systems/process_components/polytropic_path.py
E61	Real gas equation of state (CoolProp)	tested	0	2	pinneapple_systems/process_components/real_gas_eos.py
E62	Compressor similarity map	validated	2	0	pinneapple_systems/process_components/similarity_map.py
E63	Helmholtz resonator	tested	0	4	pinneapple_physics/closed_form/helmholtz_resonator.py
E64	Membrane diffusion	tested	0	5	pinneapple_physics/closed_form/membrane_diffusion.py
E65	Cantilever fatigue	validated	1	4	pinneapple_physics/closed_form/cantilever_fatigue.py
E66	Electrostatics (Gauss's law with permittivity), differential and integral form	validated	1	0	
E67	Magnetostatics in the vector potential A_z	validated	1	0	
E68	Magneto-quasistatic eddy currents (time-harmonic A_z)	validated	1	0	
E69	Maxwell TMz time domain, differential and integral (Ampere-Maxwell circulation, Faraday) form	validated	1	0	
E70	Maxwell's curl equations in 3-D with optional Gauss constraints	validated	1	0	
E71	Frequency-domain TMz wave equation with complex permittivity	validated	1	0	
P9	PINNeAPPle-CFD v1: pipes + particle erosion (CFD-04 pilot)	untested	0	0	PINNeAPPle-CFD/pinneapple_cfd/
P1.1		validated	9	43	
P1.2		tested	0	4	

id	método	status	testes contra referência	outros testes	código
P1.3		untested	0	0	
P1.4		untested	0	0	
P1.5		validated	6	15	
P1.6		untested	0	0	
P1.7		tested	0	7	
P1.8		untested	0	0	
P1.9		validated	4	15	
P2.1		untested	0	0	
P2.2		tested	0	5	
P2.3		tested	0	1	
P2.4		untested	0	0	
P2.5		untested	0	0	
P2.6		validated	1	0	
P2.7		untested	0	0	
P2.8		tested	1	0	
P2.9		tested	0	1	
P3.1		untested	0	0	
P3.2		untested	0	0	
P3.3		validated	1	3	
P3.4		untested	0	0	

id	método	status	testes contra referência	outros testes	código
P3.5		untested	0	0	
P3.6		untested	0	0	
P3.7		untested	0	0	
P4.1		untested	0	0	
P4.2		validated	1	2	
P4.3		validated	1	0	
P4.4		tested	0	1	
P5.1		validated	2	6	
P5.2		untested	0	0	
P5.3		untested	0	0	
P5.4		validated	1	0	
P5.5		untested	0	0	
P5.6		validated	1	2	
P5.7		validated	1	5	
P5.8		validated	1	0	
P6.1		validated	1	2	
P6.2		untested	0	0	
P6.3		untested	0	0	

id	método	status	testes contra referência	outros testes	código
P6.4		validated	1	0	
P6.5		untested	0	0	
P6.6		tested	2	1	
P6.7		validated	1	0	
P6.8		validated	1	0	
P6.9		untested	0	0	
P7.1		validated	3	2	
P7.2		validated	1	2	
P7.3		validated	2	8	
P7.4		validated	1	1	
P7.5		validated	1	1	
P7.6		validated	1	2	
P7.7		validated	1	3	
P7.8		untested	0	0	
P7.9		validated	1	1	
P8.1		validated	1	2	
P1.10		validated	2	1	

id	método	status	testes contra referência	outros testes	código
P1.11		tested	0	1	
P1.12		untested	0	0	
P1.13		untested	0	0	
P2.10		untested	0	0	
P2.11		untested	0	0	
P2.12		untested	0	0	
P2.13		untested	0	0	
P7.10		validated	1	1	
S1	Finite differences 1D/2D/3D	validated	2	11	pinneapple_simulation/numerical_solvers/fdm.py, pinneapple_simulation/numerical_solvers/fdm3d.py
S2	Finite elements (linear) + nonlinear beam FEM	validated	6	15	pinneapple_simulation/numerical_solvers/fem.py, pinneapple_simulation/numerical_solvers/nonlinear_beam_fem.py
S3	Finite volumes	validated	2	2	pinneapple_simulation/numerical_solvers/fvm.py
S4	Spectral (Fourier pseudo-spectral)	validated	3	2	pinneapple_simulation/numerical_solvers/spectral.py
S5	Lattice Boltzmann D2Q9 / D3Q19 (+ Smagorinsky LES)	validated	2	21	pinneapple_simulation/numerical_solvers/lbm.py
S6	SPH / ISPH / DFSPH	untested	0	0	pinneapple_simulation/numerical_solvers/sph.py, pinneapple_simulation/numerical_solvers/isph.py, pinneapple_simulation/numerical_solvers/dfsph.py
S7	Meshfree RBF collocation (Kansa)	validated	1	1	pinneapple_simulation/numerical_solvers/meshfree.py
S8	X-TFC (IVP, PDE, subdomains)	validated	4	8	pinneapple_simulation/numerical_solvers/xtfc_ivp.py, pinneapple_simulation/numerical_solvers/xtfc_pde.py, pinneapple_simulation/numerical_solvers/xtfc_subdomain.py
S9	Discrete-time PINN / implicit Runge-Kutta Gauss-Legendre	tested	0	2	pinneapple_simulation/numerical_solvers/discrete_time_pinn.py, pinneapple_simulation/numerical_solvers/irk_gauss_legendre.py

id	método	status	testes contra referência	outros testes	código
S10	Stiff kinetics ODE	validated	1	0	pinneapple_simulation/numerical_solvers/stiff_kinetics_ode.py
S11	TVD advection	validated	1	0	pinneapple_simulation/numerical_solvers/tvd_advection.py
S12	Immersed boundary (FDM)	validated	1	1	pinneapple_simulation/numerical_solvers/immersed_boundary_fdm.py
S13	Beam BVP FDM / 3D elasticity FDM	validated	2	0	pinneapple_simulation/numerical_solvers/beam_bvp_fdm.py, pinneapple_simulation/numerical_solvers/elasticity3d_fdm.py
S14	Eddy current FDM	validated	3	3	pinneapple_simulation/numerical_solvers/eddy_current_fdm.py
S15	Bekker-Wong terramechanics	validated	1	19	pinneapple_simulation/numerical_solvers/bekker_wong.py, pinneapple_simulation/particle_dynamics/terramechanics.py
S16	N-body (REBOUND)	validated	1	2	pinneapple_simulation/numerical_solvers/nbody_rebound.py
S17	JAX-CFD	tested	0	3	pinneapple_simulation/numerical_solvers/jax_cfd_solver.py
S18	Particle dynamics: rigid body, MPM, molecular dynamics	tested	0	13	pinneapple_simulation/particle_dynamics/
S19	Signal decomposition: CEEMDAN, EEMD, VMD, HHT, SSA, SST, STL, FFT, wavelet	tested	0	1	
S20	Co-simulation (graph + engine, algebraic loops by Tarjan SCC)	tested	0	11	pinneapple_systems/cosimulation/
S21	Digital twin state estimation: KF/EKF/UKF/EnKF, prognostics, signal reconstruction	tested	2	39	pinneapple_systems/digital_twin/, pinneapple_neural/architectures/classical_ts/
S22	1D process components (pipe network, transient pipe, heat exchanger, valve, reaction, polytropic path, compressor map)	tested	0	3	
S23	Explicit equation-system solver (evaluation order)	tested	0	1	pinneapple_systems/process_components/explicit_equation_system.py
S24	External solvers: OpenFOAM, FEniCS, ANSYS/CFD formats, MATLAB, Modelica/FMI, MuJoCo, Genesis, TurboDesigner	validated	1	80	pinneapple_simulation/external_solvers/
S25	Causal telemetry conditioning (Hampel one-step, rate limit, first-order filter, gaps)	validated	1	6	pinneapple_systems/digital_twin/conditioning.py
S26	2D shallow-water finite volumes (MUSCL + HLL, wet/dry, walls, gates, virtual sensors, health checks)	validated	1	3	pinneapple_simulation/numerical_solvers/shallow_water_fv.py
S27	ETDRK4 exponential integrator for stiff semilinear PDEs (contour phi-functions, blow-up guard)	validated	5	1	pinneapple_simulation/numerical_solvers/etdrk4.py

id	método	status	testes contra referência	outros testes	código
S28	Stable fluids 2D (semi-Lagrangian advection, spectral projection, vorticity confinement)	validated	2	4	pinneapple_simulation/numerical_solvers/stable_fluids.py
S29	2-D A_z magnetostatics, P1 FEM with Newton (nonlinear B-H, magnets, (anti-)periodic constraints)	validated	1	6	pinneapple_simulation/electromagnetics/fem.py, pinneapple_simulation/electromagnetics/mesh.py, pinneapple_simulation/electromagnetics/materials.py
S30	Time-harmonic 2-D eddy currents (A_z, solid conductors with imposed net currents)	validated	1	1	pinneapple_simulation/electromagnetics/eddy.py
S31	Surface-PM machine model: Maxwell-stress and co-energy torque, cogging, back-EMF, losses	validated	1	8	pinneapple_simulation/electromagnetics/machine.py, pinneapple_simulation/electromagnetics/postprocess.py
S32	2-D electrostatics, P1 FEM (capacitance from field energy)	validated	2	0	pinneapple_simulation/electromagnetics/electrostatics.py
S33	Biot-Savart integration of straight filament segments, loop and solenoid fields	validated	3	0	pinneapple_simulation/electromagnetics/statics.py
S34	Mie series for dielectric and perfectly conducting spheres	validated	2	0	pinneapple_simulation/electromagnetics/mie.py
S35	Fresnel coefficients and characteristic matrices of planar multilayers	validated	2	0	pinneapple_simulation/electromagnetics/waves.py
S36	Jiles-Atherton hysteresis model (RK4 along an applied field path)	validated	1	0	pinneapple_simulation/electromagnetics/hysteresis.py
S37	Dispersive media (Drude, Lorentz, Debye) and their auxiliary-differential-equation coefficients	validated	1	0	pinneapple_simulation/electromagnetics/media.py
S38	Maxwell physics checks of predicted fields (Gauss, Faraday, Ampere-Maxwell, continuity, Poynting, interfaces, reciprocity, passivity)	validated	4	2	pinneapple_physics/maxwell_checks.py
S39	Density-based topology optimisation (SIMP, filter, projection, OC, MMA; compliance, thermal, Stokes-Brinkman, machine rotor)	validated	1	2	pinneapple_design/topology/
S40	Yee FDTD in 1-D/2-D/3-D (CPML, Courant check, Drude/Lorentz/Debye ADE, PEC/PMC/periodic/Bloch, TF/SF plane waves, DFT monitors, sub-cell averaging)	validated	4	6	pinneapple_simulation/electromagnetics/fdtd.py
S41	Near-to-far-field transform on a closed box (FDTD radiation patterns)	validated	1	0	pinneapple_simulation/electromagnetics/fdtd.py
T1	Base trainer: AMP, gradient accumulation, auto batch size, CUDA graphs, profiler	tested	0	9	pinneapple_neural/trainer/trainer.py, pinneapple_neural/trainer/hpc.py
T2	Distributed: DDP, FSDP, SLURM, torchrun, PowerSGD	untested	0	0	pinneapple_neural/trainer/distributed.py, pinneapple_neural/trainer/hpc.py

id	método	status	testes contra referência	outros testes	código
T3	Causal training	untested	0	0	pinneapple_neural/trainer/causal.py
T4	Time marching	untested	0	0	pinneapple_neural/trainer/time_marching.py
T5	Two phases Adam -> L-BFGS + L-BFGS finetune	tested	0	6	pinneapple_neural/trainer/two_phase.py, pinneapple_neural/trainer/lbfgs_finetune.py
T6	Multi-restart	untested	0	0	pinneapple_neural/trainer/multistart.py
T7	Adaptive residual-based collocation	untested	0	0	pinneapple_neural/trainer/collocation.py
T8	Loss balancing: ReLoBRaLo, SoftAdapt, PCGrad, Augmented Lagrangian, Inverse Dirichlet, LR Annealing, AutoBalancer	validated	1	6	pinneapple_neural/trainer/loss_balancer.py
T9	Weight schedulers: SA-PINN, GradNorm, LossRatio, NTK	tested	0	2	pinneapple_neural/trainer/weight_scheduler.py
T10	Domain decomposition: XPINN, DoMINO/D3M	tested	0	2	pinneapple_neural/architectures/pinns/xpinn.py, pinneapple_physics/pinn_solver/domino.py
T11	Inverse PINN (learnable physical parameters)	tested	0	3	pinneapple_neural/trainer/inverse_pinn.py
T12	Gray-box + symbolic regression (PySR)	untested	0	0	pinneapple_neural/trainer/graybox.py
T13	Uncertainty: heteroscedastic, B-PINN, MC Dropout, deep ensembles	tested	0	20	pinneapple_neural/trainer/heteroscedastic.py, pinneapple_neural/architectures/pinns/bpinn.py, pinneapple_systems/component_modeling/
T14	SINDy	tested	0	2	pinneapple_neural/architectures/rom/sindy.py
T15	Adaptive hyperparameter sweep / architecture search (arena NAS)	untested	0	0	pinneapple_neural/trainer/adaptive_sweep.py, pinneapple_arena/nas.py
T16	Noether trainer (research only)	tested	0	1	pinneapple_neural/trainer/noether_trainer.py
T17	Post-training advisor / audit	untested	0	0	pinneapple_neural/trainer/advisor.py, pinneapple_neural/trainer/audit.py
T18	Meta-learning: MAML, Reptile	tested	0	6	pinneapple_adaptation/meta_learning/
T19	Transfer learning (freeze, adapters, parametric)	tested	0	3	pinneapple_adaptation/transfer_learning/
T20	Self-scaled quasi-Newton: BFGS / SSBFGS / SSBroyden (new, 2026-09-24)	validated	1	3	pinneapple_neural/trainer/self_scaled_qn.py
T21	Autoresearch loop (fixed-budget trials, keep/revert, random or LLM proposer)	tested	0	2	pinneapple_arena/autoresearch.py
T22	Large Physics Model blocks (Fourier encoding, multi-scale neighbourhoods, geometry code, ensemble UQ, OOD score, fine-tuning)	tested	0	1	pinneapple_neural/lpm.py
T23	MultiAdam: per-loss-term Adam moments for multi-term PINN losses	validated	1	3	pinneapple_neural/trainer/multiadam.py

3.4 Como os números foram obtidos

- Tudo vem de `docs/pt/_mapa/inventario.py`, que lê o código com `ast` (nada é importado ou executado).
- *Público* = nome de nível de módulo que não começa com _. Métodos de classes são contados à parte.
- *Testes de um módulo* = funções `test*` em arquivos de `tests/` que importam aquele módulo (`import`, `from … import`, `importlib.import_module`, `pytest.importorskip`). É uma medida de alcance, não de cobertura de linhas.
- O status de validação dos métodos é o de `pinneapple_catalog/method_status.json`, gerado por `scripts/build_method_status.py`.
- Itens ligados a clientes foram excluídos do inventário e do texto.